

ORBITER

Classification of Combinatorial Objects

User's Guide
Build Number 3344

Anton Betten

July 3, 2026

Contents

List of Tables	7
1 Introduction	11
1.1 What is Orbiter	11
1.2 Acknowledgements	12
1.3 Installing Orbiter for the First Time	13
1.4 Updating Orbiter	16
1.5 The Work Directory, Makefiles and GitHub	17
1.6 Typographic Conventions	20
2 The Orbiter Language	23
2.1 The Orbiter Session	23
2.2 Objects and Activities	26
2.3 The Orbiter Dash Code Language	30
2.4 Set Builder	40
2.5 Vector Builders	41
2.6 Vector of Group Elements	48
2.7 Symbolic Objects	50
3 Finite Fields	71
3.1 Prime Fields	71
3.2 Extension Fields	77
3.3 Linear Algebra Over Finite Fields	89
3.4 Advanced Topics in Finite Fields	94
4 Projective Geometry	103
4.1 Finite Projective Spaces	103
4.2 Indexing Points and Lines	109
4.3 Incidence Matrices	113
4.4 Advanced Topics	119
4.5 Geometric Objects	130
5 Group Theory	135
5.1 Generic Permutation Groups	135
5.2 Matrix Groups	147
5.3 Subgroups	171
5.4 New Actions from Old	189
5.5 Group Theoretic Activities	212
5.6 Group Theoretic Activities Based on Magma	229
5.7 Group Theoretic Activities Based on GAP	241
5.8 Group Theory, Advanced Topics	245
5.9 Activities for Vectors of Group Elements	254

6	Ring Theory	263
6.1	Univariate Polynomials Over Finite Fields	263
6.2	Multivariate Polynomial Rings	272
7	Algebraic Geometry	281
7.1	The Grassmannian	281
7.2	The Klein Quadric and the Plücker Map	284
7.3	Orthogonal Spaces	285
7.4	Hermitian Varieties	304
7.5	Algebraic Varieties	306
7.6	Activities for Algebraic Varieties	314
7.7	Mappings between Projective Spaces	324
8	Orbit Algorithms	327
8.1	Schreier Trees	327
8.2	Poset Classification	335
8.3	Orbits on Subsets	340
8.4	Orbits on Subspaces	349
8.5	Orbits on Set-Partitions	353
8.6	Arcs and Caps in Projective Spaces	355
8.7	Cubic Curves	359
9	Cubic Surfaces and Quartic Curves	363
9.1	Generalities	363
9.2	Creating Cubic Surfaces	366
9.3	Cubic Surface Activities	376
9.4	Classification of Cubic Surfaces	384
9.5	Quartic Curves	402
9.6	Classification of Quartic Curves	419
9.7	Interface to GAP	421
10	Applications	423
10.1	Number Theory	423
10.2	Representation Theory	436
10.3	Cryptography	438
11	Coding Theory	447
11.1	Introduction	447
11.2	Linear Codes	455
11.3	Golay Codes	466
11.4	CRC Codes	467
11.5	Reed-Muller Codes	476
11.6	BCH Codes	484
11.7	Reed-Solomon Codes	491
11.8	Twisted Tensor Product Codes	496
11.9	Bounds	498
11.10	Classification of Optimal Linear Codes	500
12	Combinatorics	503
12.1	Introduction	503
12.2	Combinatorial Objects	511
12.3	Activities for Geometric Objects	520
12.4	Combinatorial Objects with Group	523
12.5	Combinatorial Linear Spaces	527
12.6	Classification of Configurations and Geometries	530

12.7 Tactical Decompositions	539
12.8 The Plesken Ring	542
13 Graph Theory	551
13.1 Creating Graphs	551
13.2 Graph Theoretic Activities	565
13.3 Classification of Graphs and Tournaments	578
14 Design Theory	583
14.1 Creating Designs	583
14.2 Assuming Symmetry	592
14.3 Design Theory – Large Sets	599
14.4 Design Theory – Delandtsheer-Doyen	604
14.5 Design Theory – Flag transitive designs	608
15 Finite Geometry	613
15.1 Spreads	613
15.2 Translation Planes	629
15.3 Packings	634
15.4 BLT-Sets	642
16 Computer Science Primitives	653
16.1 Clique Finding	653
16.2 Rainbow Cliques	658
16.3 Diophantine Systems	661
17 Canonical Forms, Classification and Automorphism Groups	665
17.1 Overview of Canonical Forms	665
17.2 The Classification-By-Canonical-Form Algorithm	668
17.3 Canonical Forms of Incidence Geometries	685
17.4 Canonical Forms of Objects from Design Theory	703
17.5 Canonical Forms of Linear Codes	707
17.6 Canonical Forms of General Codes	714
17.7 Canonical Forms of Graphs	715
17.8 Canonical Forms of Quartic Curves	721
18 More Applied Algebra	725
18.1 The Automorphism Group of $\text{Sym}(6)$	725
19 Interfaces	739
19.1 Graphical Output	739
19.2 The Povray Interface	744
19.3 Creating Animations	757
19.4 Continuous Function Plotter	758
19.5 The Gnuplot Interface	762
20 Mathematical Data in Orbiter	765
20.1 Cubic Surfaces	765
20.2 BLT-Sets	767
21 Miscellaneous	769
21.1 Miscellaneous	769
21.2 Limitations	774
22 Orbiter on Windows	777
22.1 Using Windows Subsystem Linux	777

23	Templates	787
23.1	Templates	787
A	Orbiter Syntax Railroad Diagrams	791
A.1	Syntax Diagrams	791
B	Data on Cubic Surfaces	819
B.1	Data on Cubic Surfaces	819
C	Linux Background	841
C.1	Command Sheet	841
D	Testing	843
D.1	Testing	843
E	The Makefile	849
E.1	The Makefile	849
	References	1370

List of Tables

2.1	Orbiter session commands	25
2.2	Orbiter Objects (Part I)	26
2.3	Orbiter Objects (Part II)	27
2.4	Orbiter Activities (Part 1)	28
2.5	Orbiter Activities (Part 2)	29
2.6	Set Builder Commands	40
2.7	Vector Builder Commands	42
2.8	Vector of Integers Activities	47
2.9	Vector of Group Elements Builder Commands	48
2.10	Creating Symbolic Objects I	51
2.11	Creating Symbolic Objects II	52
2.12	Activities for Symbolic Objects	53
3.1	Creating Finite Fields	72
3.2	Finite Field Activities	73
3.3	More Finite Field Activities	78
3.4	Orbiter Primitive Polynomials	79
3.5	Finite Field Activities Pertaining to Linear Algebra	89
3.6	Finite Field Activities (Part I)	94
3.7	Finite Field Activities (Part II)	95
4.1	Options for Creating a Projective Space	103
4.2	Ranking Points in Small Projective Spaces	109
4.3	Ranking Lines in Small Projective Spaces	111
4.4	Projective Space Activities (Part 1)	120
4.5	Projective Space Activities (Part 2)	121
4.6	Projective Space Activities (Part 3)	122
4.7	Projective Space Activities (Part 4)	123
4.8	Global Commands for Projective Geometry	123
4.9	Geometric Objects (Part 1)	131
4.10	Geometric Objects (Part 2)	132
5.1	Creating a Generic Permutation Group	136
5.2	Creating a Matrix Group (Part 1)	147
5.3	Creating a Matrix Group (Part 2)	148
5.4	Basic actions	150
5.5	Creating Subgroups	171
5.6	Group Modifications	190
5.7	Creating Special Actions	200
5.8	Group theoretic activities (Part 1)	213
5.9	Group theoretic activities (Part 2)	214
5.10	Group theoretic activities (Part 3)	215
5.11	Group theoretic activities (Part 4)	216

5.12	Group theoretic activities (Part 5)	217
5.13	Element Processing Options	217
5.14	Group Theoretic Activities Based on Magma	229
5.15	Group Theoretic Activities Based on GAP	241
5.16	Vector of Group Elements Activities	255
6.1	Finite Field Activities Related to Polynomials	264
6.2	Creating Multivariate Polynomial Rings	272
6.3	Multivariate Polynomial Ring Activities	273
6.4	Monomial Ordering of Partition Type in the Plane	274
6.5	Monomial Ordering of Partition Type in Three-Space	275
7.1	Types of Quadrics	285
7.2	Creating an Orthogonal Space	285
7.3	Orthogonal Space Activities	288
7.4	Creating Varieties	306
7.5	Variety Activities	314
7.6	Mappings Between Projective Spaces	325
8.1	Computing Orbits	328
8.2	Computing Orbits	329
8.3	Orbit Based Activities	329
8.4	Options for Computing Posets	337
8.5	Activities for Posets of Orbits (Part 1)	338
8.6	Activities for Posets of Orbits (Part 2)	339
8.7	Poset Classification Report Options	339
8.8	Classifying Arcs	355
9.1	Creating a Cubic Surface (Part 1)	366
9.2	Creating a Cubic Surface (Part 2)	367
9.3	Creating a Cubic Surface (Part 3)	368
9.4	Cubic Surface Activities	377
9.5	Options for Exporting Data: Cubic Surfaces	378
9.6	Classifying Cubic Surfaces	384
9.7	Activity for Cubic Surfaces After Classification	385
9.8	Orbit Representatives	388
9.9	Creating Quartic Curves	402
9.10	Quartic Curve Activities	403
9.11	Options for Exporting Data: Quartic Curves	403
10.1	Global Algebra Commands	424
10.2	Basic Number Theory Commands	425
10.3	Number Theoretic Commands	426
10.4	Representation Theory Activities	436
10.5	Cryptographic Commands	438
10.6	Finite Field Activities for Cryptography	439
11.1	Global Coding Theoretic Commands	448
11.2	Creating Codes	456
11.3	Coding Theoretic Activities (Part 1)	457
11.4	Coding Theoretic Activities (Part 2)	458
11.5	Coding Theoretic Activities (Part 3)	459
11.6	Code Modifications	459
11.7	Defining a CRC-process	467
11.8	Options for CRC-Codes	467

11.9 Defining a CRC-Code	467
11.10 Options for Drawing Modulo N Objects	484
12.1 Combinatorics (Part 1)	504
12.2 Combinatorics (Part 2)	505
12.3 Defining Combinatorial Objects (Part 1)	512
12.4 Defining Combinatorial Objects (Part 2)	513
12.5 Activities for Combinatorial Objects	521
12.6 Options for Reporting After Classification	522
12.7 Activities for Combinatorial Objects with Group	523
12.8 Constructing and Classifying Geometries	531
12.9 Refining Tactical Decompositions	539
12.10 Activities for a Plesken Ring	542
13.1 Creating Graphs (Part 1)	552
13.2 Creating Graphs (Part 2)	553
13.3 Graph Modifiers	558
13.4 Graph Theoretic Activities (Part 1)	566
13.5 Graph Theoretic Activities (Part 2)	567
13.6 Classifying Graphs	578
13.7 Activities after Graph Classification	578
14.1 Creating Designs	584
14.2 Design Activities	585
14.3 Latin Squares of Order 5	589
14.4 Creating Large Sets with Assumed Symmetry	599
14.5 Large Set Activity	599
14.6 Creating Linear Spaces of Delandtsheer-Doyen Type	605
15.1 Creating a Spread	614
15.2 Spread Activities	614
15.3 Creating a Spread Table	615
15.4 Activities for Spread Tables	615
15.5 Classification of Spreads	616
15.6 Activities for the Classification of Spreads	616
15.7 Isomorph Testing	621
15.8 Activities for Translation Planes	629
15.9 Creating Packings with Assumed Symmetry	634
15.10 Classification of Packings	635
15.11 Activities for Packings	635
15.12 Activities for Packings with Fixpoints	635
15.13 Creating Packings from Long Orbits	635
15.14 Creating BLT-Sets	642
15.15 Families of BLT-sets	642
15.16 BLT-Set Activity	643
15.17 Classifying BLT-Sets	643
15.18 Activities for the Classification of BLT-sets	643
16.1 Finding Cliques	653
16.2 Creating Diophantine Systems	662
16.3 Diophantine System Activities	663
17.1 Classification by Canonical Form	666
17.2 Classification of Combinatorial Objects	667
17.3 Nauty Control Options	667

17.4 Creating a Bitmap Graphics	686
17.5 Options for Drawing Incidence Structures	686
19.1 Drawing Options for Layered Graph Files (Part 1)	740
19.2 Drawing Options for Layered Graph Files (Part 2)	741
19.3 Povray Interface	744
19.4 Povray Job Options	744
19.5 Povray Interface (Part 1)	745
19.6 Povray Interface (Part 2)	746
19.7 Scene Elements (Part 1)	747
19.8 Scene Elements (Part 2)	748
19.9 Scene Object Types	749
19.10 Preparing Frames for an Animation	757
21.1 Miscellaneous Orbiter Commands (Part 1)	770
21.2 Miscellaneous Orbiter Commands (Part 2)	771
21.3 Miscellaneous Orbiter Commands (Part 3)	772
21.4 Options for Drawing Trees	773

Chapter 1

Introduction

1.1 What is Orbiter

This is the User's Guide of Orbiter. Orbiter is a computer algebra system for the classification of combinatorial objects. It provides tools to construct, classify and investigate structures that are defined up to isomorphism. For background on Orbiter, see [13].

Orbiter is open source, and can be downloaded and installed from github. A C++ compiler environment similar to “gnu gcc” and the tool “make” are required. Compiler tools like “bison” and “flex” are required also. Latex is recommended.

For the most recent version of this guide, please visit

https://www.math.colostate.edu/~betten/orbiter/users_guide.pdf

For the github repository, see

<https://github.com/abetten/orbiter>

The version number of Orbiter that is required for this guide is indicated on the title page. It is the git repository number.

The examples in this guide are taken from a makefile. The complete makefile is distributed with Orbiter in the directory

`examples/users_guide`

There is no need to type the code. You can copy it from the orbiter source tree to a local directory of your choosing. Please do not run the makefile commands from the orbiter source tree. This will most likely create files, which will cause some trouble later on when updating Orbiter. The Orbiter source tree should be read only, unless you know what you are doing.

Orbiter interfaces with latex to display outputs of computations or to produce reports about objects. For this reason, it is recommended to have a working latex package installed. We will assume that pdfflatex exists. Some latex distributions come with a graphical user interface, where the tex file needs to be selected instead.

A few words about related software. Some of the commands rely on Magma. Magma is a commercial product. There is, however, a free web interface, see [71]. At times, we will be mentioning Maple [55] and GAP [32], as well as the GAP package Fining [7]. The software package Nauty [57] is important. It has been integrated seamlessly into Orbiter, which means we will be using it without notice. For instance, all of Chapter 17 relies on Nauty.

1.2 Acknowledgements

The following people contributed to Orbiter:

1. Abdullah Alazemi,
2. Sajeeb Roy Chowdhuri,
3. Fatma Karaoglu,
4. Awss Al-ogaidi.

The following software packages found their way into Orbiter:

1. Nauty [57] (Brendan McKay),
2. Possolve (Brendan McKay),
3. Eigen (Benoît Jacob and Gaël Guennebaud),
4. Pugixml (Arseny Kapoulkine, Kristen Wegner),
5. EasyBMP (Paul Macklin),
6. Part (Jeffrey Leon),
7. TDO (Dirk Kämpfer).

1.3 Installing Orbiter for the First Time

To install Orbiter, we first check if git is installed. Type

```
git
```

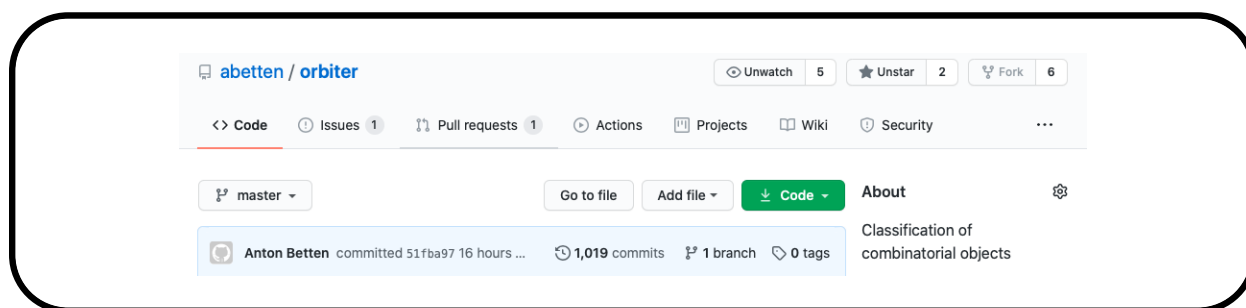
If you get an error message, please install git. If you get a long output with commands, you are fine. Git is installed.

The steps to install Orbiter differ slightly, depending on your operating system. For Windows, we need “Windows Subsystem Linux.” On Linux/unix based systems, including Macintosh, you will utilize a terminal window. You will also need the development tools. You need the C++ compiler and library tools. On Macintosh, you need Xcode and the command-line tools. You can get them for free on the App-store. You will also need the compiler generator tools flex and bison. They are open source projects. On Macintosh, you can use the package manager brew. On Linux, simply use your package manager to install these tools.

To install Orbiter, you first need to download the source code from github. So, navigate your web browser to

<https://github.com/abetten/orbiter>

Click on the green box that says “Code” and copy the address into the clipboard by clicking the clipboard symbol, see below:



Click on “Code” and copy paste the address. Then type

```
git clone XXX
```

Where XXX is to be replaced with the address that you copied (simply use CTRL-P to paste).

Once the command finishes, you need to check that the prerequisites are satisfied, which are

1. flex
2. bison

These are compiler generator tools, and they are required for compiling Orbiter. The required tools are called bison and flex (in the Unix world, they correspond to yacc and lex, respectively). Unfortunately, many Mac OS versions come with outdated versions of bison and flex, so we need to check the version that is installed (or install it in the first place). To check the version of bison, type

```
bison --version
```

If you get a message `bison not found` it means that you don't have bison. In this case you need to install bison. Otherwise, check out the version number. Orbiter requires 3.8.2 or higher. If your version number is below that, you need to upgrade your bison software. Regarding flex, type

```
flex --version
```

If you get a message `flex not found` it means that you don't have flex. In this case you need to install flex. Otherwise, check out the version number. Orbiter requires 2.6.4 or higher. If your version number is below that, you need to upgrade your flex software.

Next, we will show how to install bison and flex, and possibly update the version that is installed. On a Macintosh system, the package manager brew can be used. On Linux, different package managers exist. In either case, we have to also update certain shell variables, so that the correct version of flex and bison is used during the Orbiter build process.

On a Macintosh, using `cs`h (pronunciation: cee-shell), you can install bison using "brew." Type

```
brew install bison
```

The install will go to

```
/opt/homebrew/Cellar
```

Go there and check whether a directory bison exists. If so, do

```
cd bison
```

and

```
ls
```

and note the version number. You may still have an old version of bison somewhere else. In order to activate the new bison, you will have to change the path in your shell. For `cs`h, make sure you add the line

```
set path = ( /opt/homebrew/Cellar/bison/3.8.2/bin $path )
```

Here, the 3.8.2. part may have to be changed depending on your version of bison.

You can install flex using brew. Type

```
brew install flex
```

The install will go to

```
/opt/homebrew/Cellar
```

Go there and check whether a directory flex exists. If so, do

```
cd flex
```

and

```
ls
```

and note the version number. Like with bison earlier, you may still have an old version of flex somewhere else. In order to activate the new flex and not the old, you will have to change the path in your shell. For `cs`h, make sure you add the line

```
set path = ( /opt/homebrew/Cellar/flex/2.6.4_2/bin $path )
```

Here, the 2.6.4_2 part may have to be changed depending on your version of bison.

Finally, you have to add

```
set LDFLAGS = ( -L/usr/local/Cellar/flex/2.6.4_2/lib -L/usr/local/Cellar/bison/3.8.2/lib )
```

to your `~/.cshrc`. Here, the version numbers may have to be changed to match those that you used before.

After these changes, you need to read the `~/.cshrc` file again and rehash. Type

```
read ~/.cshrc
```

and

```
rehash
```

If you type

```
bison --version
```

you should get the correct version number of bison. Likewise, if you type

```
flex --version
```

you should get the correct version number of flex.

You are now ready to compile Orbiter. For this purpose, you need to enter the Orbiter home directory. Assuming that you are still in the directory where you did `git clone`, type

```
cd orbiter
```

and then

```
make
```

If your computer CPU has multiple cores, you can try

```
make -j 8
```

The number 8 can be adjusted to the number of cores that you have. This multitasking reduces the time it takes to compile Orbiter. Once done, you should have an orbiter executable in the directory

```
orbiter/src/apps/orbiter
```

The executable is called

```
orbiter.out
```

Note that you are already in orbiter, so you could do

```
ls src/apps/orbiter
```

to check if `orbiter.out` appears. In either case, you can do

```
cd ..
```

to leave the orbiter branch. You should only go back in the Orbiter branch if you want to update Orbiter. Otherwise, you should not be there. In particular, you should not change any files in this branch, as this would cause problems in the update.

Finally, if you have to update bison or flex, you can type

```
brew upgrade bison
```

or

```
brew upgrade flex
```

1.4 Updating Orbiter

To update Orbiter, we first check if “git” is installed. Type

```
git
```

If you get an error message, it means that git is not installed or that git cannot be found. In the former, you need to install git. In the latter, you may have to check the path of your shell. On the other hand, if you get a long output with commands, you are fine. Git is installed.

Next, we enter the `orbiter` directory. This is the directory that was created by `git clone`.

```
cd orbiter
```

Now type

```
git pull
```

to update the Orbiter sources. If you get an error message to the effect that you should stash something first, it means that you have made changes inside the orbiter directory. Most likely, you edited a file in the Orbiter directory tree by chance. In this case, it is best to let go of these changes by issuing

```
git reset --hard
```

and then doing

```
git pull
```

again. Then type

```
make clean
```

and

```
make
```

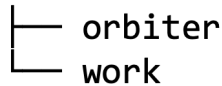
Orbiter is a large project, so this command may take some time. You can speed up things by using parallel compilation. For instance, use

```
make -j 8
```

for parallel compile with 8 threads. If necessary, change the number in the command. Use larger numbers if your hardware supports more CPU cores. You should get a new Orbiter executable.

1.5 The Work Directory, Makefiles and GitHub

The Orbiter directory is under the control of git, whose purpose it is to keep the sources up-to-date. The user should not make any changes in this directory, as it might cause trouble with updating Orbiter through git. It is recommended to create a separate directory for work, *outside the Orbiter directory*. For the following, we assume that a parallel directory work has been created, so that the directory tree structure looks like this:



There is a large makefile in

`orbiter/examples/users_guide`

containing all commands from this user's guide (listed in Appendix E). It would be good to copy this makefile into the work directory. Then, all commands in this user's guide can be executed immediately from within the work directory. This way, the orbiter directory stays clean, so github will not complain during any future updates of Orbiter.

As there can be only one makefile per directory, it is recommended to start separate directories for different projects. For instance, there could be multiple work directories, say work1, work2 and so forth. Each work directory would have its own makefile.

A "makefile" can store command sequences. They could be calls to Orbiter, or they could be other system commands. To call execute a makefile command, the make application is used.

Makefiles are plain text files. To edit a makefile, a text editor is required (something like "Notepad" on Windows, "vi" or "gedit" on Linux or Macintosh). A text editor is used to edit plain text files. This is different from a word processing application like "word." Word processing applications store files in a proprietary format (for instance, docx for Word). Word processing applications are not useful to edit makefiles.

Working with makefile requires a terminal window. The terminal is a application which runs a shell. A shell is responsible for executing the commands that you type. There are different shells, each with slightly different syntactical rules. However, most commands that are typed are really calls of system application.

Under Windows, there is the "Cmd" application. On Macintosh, the application is called Terminal. Under Windows, it is necessary to run `wsl.exe` before any make command. This invokes the "Windows Subsystem Linux" environment, which is necessary since Orbiter is a Linux project and not a native Windows application.

The terminal runs a command line processor called a shell. The shell executes the commands that are typed. When working with makefiles, the shell executes the make command. The make command is given exactly one argument, which refers to a command sequence in the makefile in the present directory. Make opens the makefile and searches for an entry that matches the given command label. Once found, make then executes the appropriate command sequence from the makefile. If there happen to be multiple entries with the same label, make will execute the first one. If make is called without any argument, the first label in the makefile will be executed.

The most important part of installation is the tailoring of the makefile. Specifically, we need to set the path to the Orbiter executable. The Orbiter is

`orbiter/src/apps/orbiter/orbiter.out.`

To get the makefile to work, you need to edit the first makefile variable, called

`ORBITER_PATH`

The `ORBITER_PATH` variable should point to the Orbiter directory. For instance, if you install Orbiter in your home directory, your variable should be

```
ORBITER_PATH=~/orbiter
```

The tilda is the abbreviation of the home directory. The shell will know where to find your home directory. If you installed Orbiter somewhere else, you would have to change the variable accordingly. To execute a command from the makefile associated with this user's guide, type

```
make test_orbiter_session
```

Remember, Windows users will instead type

```
wsl.exe make test_orbiter_session
```

If your `ORBITER_PATH` is incorrect, you may receive an error message of the form

```
.../orbiter.out: No such file or directory
```

where the initial three dots are a placeholder. The actual text that will appear depends on what you have in your `ORBITER_PATH`. By changing the `ORBITER_PATH` variable according to the place of Orbiter on your file system, the error message will go away.

If everything goes as planned, you should see a few lines of text produced by Orbiter. The first line should be something like this:

```
Welcome to Orbiter! Your build number is ...
```

Here, the final three dots are a placeholder for the “build number” of your Orbiter installation. You can use this number to compare it to the number on the title of this user's guide to detect potential version mismatches. Of course, you should always try to have the most recent version of Orbiter. If your Orbiter version is dated, follow the instruction in Section 1.4 to update your Orbiter. You can find the ultimate build number by going to Orbiter's source branch on GitHub and checking the file

```
build_number
```

It contains the most recent version number of the Orbiter sources.

In order to find out what files a particular Orbiter command has created, the shell command

```
ls -lrt
```

can be used. Remember, Windows users will type

```
wsl.exe ls -lrt
```

This command displays the list of files in the present directory in the reverse order of creation. So, the most recent files are at the bottom of the list. Many Orbiter commands create tex-files, which are meant to be processed by latex. Some commands create csv-files. The text output in the terminal window may contain useful information also. The text output can be collected into a file, using the shell output redirection syntax. For instance,

```
make test_orbiter_session >log
```

would execute the `test_orbiter_session` command and collect the text output in the file `log` (Windows users, remember to start the command with `wsl.exe`).

```
~/orbiter/src/apps/orbiter/orbiter.out
Welcome to Orbiter! Your build number is 3241.
do_orbiter_session
orbiter_top_level_session::orbiter_top_level_session before new orbiter_session
orbiter_session::orbiter_session
orbiter_session::orbiter_session done
orbiter_top_level_session::orbiter_top_level_session after new orbiter_session
```

```

do_orbiter_session before Top_level_session.execute_command_line
orbiter_top_level_session::execute_command_line
A user's guide is available here:
https://www.math.colostate.edu/~betten/orbiter/users_guide.pdf
The sources are available here:
https://github.com/abetten/orbiter
An example makefile with many commands from the user's guide is here:
https://github.com/abetten/orbiter/tree/master/examples/users_guide/makefile
SYSTEMUNIX is defined
SYSTEM_IS_MACINTOSH is defined
sizeof(int)=4
sizeof(long int)=8
session_verbose_level = 0
orbiter_top_level_session::execute_command_line before handle_everything
orbiter_top_level_session::execute_command_line after handle_everything
orbiter_top_level_session::execute_command_line done
do_orbiter_session after Top_level_session.execute_command_line
orbiter_top_level_session::~~orbiter_top_level_session
orbiter_top_level_session::~~orbiter_top_level_session before delete Orbiter_sessi
on
orbiter_session::~~orbiter_session
nb_times_finite_field_created=0
nb_times_projective_space_created=0
nb_times_action_created=0
nb_calls_to_densenauty=0
orbiter_session::~~orbiter_session done
orbiter_top_level_session::~~orbiter_top_level_session after delete Orbiter_sessio
n
orbiter_top_level_session::~~orbiter_top_level_session done
The Orbiter session is finished.
User time: 0.000000 of a second, dt=0 tps = 100

```

Orbiter accepts a parameter for the verbosity. A higher verbosity will make Orbiter produce more console text output, which may be useful to gain more insight into the computations that Orbiter performs. A high value of the verbosity is particularly useful for debugging purposes. A verbosity of 0 suppresses most console output.

1.6 Typographic Conventions

The following typographic conventions are used in this document:

Makefile excerpts are in color coded boxes. Line numbers have been added for clarity. A tabulator symbol is indicated as a little triangle pointing to the right. Tables are color coded:

A blue box indicates a piece of data that is given as a makefile variable, like so:

Code Snippet 1

```
1  HAMMING_CODE_GENERATOR="\
2  1,0,0,0,0,1,1, \
3  0,1,0,0,1,0,1, \
4  0,0,1,0,1,1,0, \
5  0,0,0,1,1,1,1"
6
```

A red box indicates a table of Orbiter commands.

Code Snippet 2

```
1  matrix_example2:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -define v -vector -field F -format 4 \
5  ▷ ▷ ▷ -dense $(HAMMING_CODE_GENERATOR) -end \
6  ▷ ▷ -print_symbols
7
```

A green box signifies some other kind of table.

Makefile variables are purely based on text. They can be utilized using the dollar-parenthesis syntax. For instance, the makefile variable

HAMMING_CODE_GENERATOR

has been defined above. To use a makefile variable, the dollar-parenthesis syntax is used. For instance,

\$(HAMMING_CODE_GENERATOR)

represents the contents of the variable.

A piece of output created by Orbiter is put in a black frame, like so:

```
~/orbiter/src/apps/orbiter/orbiter.out
Welcome to Orbiter! Your build number is 3241.
do_orbiter_session
orbiter_top_level_session::orbiter_top_level_session before new orbiter_session
orbiter_session::orbiter_session
orbiter_session::orbiter_session done
orbiter_top_level_session::orbiter_top_level_session after new orbiter_session
do_orbiter_session before Top_level_session.execute_command_line
orbiter_top_level_session::execute_command_line
A user's guide is available here:
```



```

https://www.math.colostate.edu/~betten/orbiter/users_guide.pdf
The sources are available here:
https://github.com/abetten/orbiter
An example makefile with many commands from the user's guide is here:
https://github.com/abetten/orbiter/tree/master/examples/users_guide/makefile
SYSTEMUNIX is defined
SYSTEM_IS_MACINTOSH is defined
sizeof(int)=4
sizeof(long int)=8
session_verbose.level = 0
orbiter_top_level_session::execute_command_line before handle_everything
orbiter_top_level_session::execute_command_line after handle_everything
orbiter_top_level_session::execute_command_line done
do_orbiter_session after Top_level_session.execute_command_line
orbiter_top_level_session::~~orbiter_top_level_session
orbiter_top_level_session::~~orbiter_top_level_session before delete Orbiter_sessi
on
orbiter_session::~~orbiter_session
nb_times_finite_field_created=0
nb_times_projective_space_created=0
nb_times_action_created=0
nb_calls_to_densenauty=0
orbiter_session::~~orbiter_session done
orbiter_top_level_session::~~orbiter_top_level_session after delete Orbiter_sessio
n
orbiter_top_level_session::~~orbiter_top_level_session done
The Orbiter session is finished.
User time: 0.000000 of a second, dt=0 tps = 100

```


Chapter 2

The Orbiter Language

2.1 The Orbiter Session

Orbiter is a command line program. There is no graphical user interface. The Orbiter executable is `orbiter.out`. It needs to be called from a terminal window, through the command shell. Orbiter takes commands from the command line for execution. The language is called Orbiter dash code. This refers to the fact that keywords are indicated with a leading dash, just like in the unix / linux command syntax. This user's guide will describe the orbiter dash code language.

Orbiter commands and command sequences can be stored either a makefile or in a shell script. Makefiles have the advantage that they can hold many command sequences, each with a different call name. This reduces clutter, as all these command sequences are stored in one file. On the other hand, makefiles come with some very stringent syntax requirements, so there is a learning curve. Shell scripts are little programs executed by the command line shell. Depending on the shell program, a slightly different syntax is required. However, the Orbiter commands are the same.

In this user's guide, we will make extensive use of makefiles. In fact, there will be only one makefile. It is available in the orbiter tree as

```
orbiter/examples/users_guide/makefile
```

It is strongly recommended to copy this file somewhere outside of the orbiter tree before running any commands.

Let us take a closer look at an Orbiter session. As a test, we can run Orbiter without any arguments. We use a makefile entry like this:

Code Snippet 3

```
1  test_orbiter_session:
2  ▷  $(ORBITER)
3
```

Once started, the Orbiter session will produce a short welcome message:

```
Welcome to Orbiter! Your build number is 1081.
A user's guide is available here:
https://www.math.colostate.edu/~betten/orbiter/users\_guide.pdf
The sources are available here:
https://github.com/abetten/orbiter
```

```
An example makefile with many commands from the user's guide is here:  
https://github.com/abetten/orbiter/tree/master/examples/users\_guide/makefile  
Orbiter session finished.  
User time: 0:00
```

The build number is the version number of the Orbiter software, as defined by the number of submits to the Git repository. Higher numbers mean more recent versions. After this message, Orbiter will start parsing the command line arguments. Once this is done, the session will execute these commands. At the end of the session, a short message is given that specifies the processor time used up by the session.

Several commands are global, and must appear right at the beginning of the orbiter dash code. These are the session commands. They are listed in Table 2.1.

Orbiter session commands		
Command	Arguments	Purpose
-v	v	Set verbosity to v . Larger values of v lead to more text output. $v = 0$ gives minimal to no output.
-list_arguments		Print the command line arguments.
-seed	s	Seed the pseudo random number generator with the integer value s .
-memory_debug		Turn on dynamic memory debugging.
-override_polynomial	poly	Specify the override polynomial for the creation on finite fields. See Section 3.2.
-orbiter_path	p	Set the orbiter path to p . This is useful in case the Orbiter session has to clone or fork new Orbiter sessions. In most cases, the orbiter path will end with a forward slash “/.”
-magma_path	p	Set the magma path to p . This is useful in case the Orbiter session has to create a magma process.
-fork	$L M s u d$	Fork new Orbiter sessions in parallel. The new sessions will be indexed by the values $i = s + kd$, where $k = 0, 1, \dots$ subject to $i < u$. Every occurrence of the string L in the argument list is replaced by the resulting value of the loop variable i . The forked process will write to a file. The filename is obtained from M by replacing %d with the value of i (similar to a printf command in C programming). All of the command line arguments after the fork command are passed through to the new Orbiter session, with all arguments L replaced by the integer value of the loop counter. The number of Orbiter sessions forked is $(u - s)/d$. The orbiter path is considered when starting the forked sessions.
-parse_commands_only		Parse the commands only. Do not execute the Orbiter code.
-show_births_and_deaths_stats		Show information about the number of class instantiations and destructions at the end of the run.

Table 2.1: Orbiter session commands

Orbiter Objects (Part I)		
Command	Reference	Type of Object
-finite_field	Sections 3.1 and 3.2	A finite field $\mathbb{F}_q$.
-polynomial_ring	Section 6.2	A multivariate polynomial ring.
-projective_space	Section 4.1	A projective space $\text{PG}(n, q)$.
-orthogonal_space	Section 7.3	A non-degenerate orthogonal space $O^\epsilon(n, q)$.
-BLT_set_classifier	Section 15.4	An object to classify BLT-sets.
-spread_classifier	Section 15.1	An object to classify spreads.
-linear_group	Section 5.2	A linear group.
-permutation_group	Section 5.1	A generic permutation group.
-modified_group	Section 5.4	A new group from an old.
-collection		A collection of objects.
-geometric_object	Section 4.5	A geometric object.
-graph	Section 13.1	A graph.
-code	Section 11.2	A code.
-spread	Section 15.1	A spread.
-cubic_surface	Section 9.2	A cubic surface.
-quartic_curve	Section 9.5	A quartic curve.
-BLT_set	Section 15.4	A BLT-set.
-translation_plane	Section 15.2	A translation plane.
-spread_table	Section 15.3	A table of spreads.
-packing_classify	Section 15.3	Classification of packings.
-packing_with_symmetry_assumption	Section 15.3	Classification of packings with assumed symmetry.
-packing_choose_fixed_points	Section 15.3	Classification of sets of fixed points during the classification of packings with assumed symmetry.

Table 2.2: Orbiter Objects (Part I)

2.2 Objects and Activities

Orbiter is based on objects and activities. Objects hold data and activities are the tasks which are associated to an object. A symbol table serves as a repository for objects.

Tables 2.2 and 2.3 summarize the Orbiter objects.

Tables 2.4-2.5 summarize the possible activities for Orbiter objects.

The objects and activities will be discussed throughout this user's guide.

Orbiter Objects (Part II)		
Command	Reference	Type of Object
-packing_long_orbits	Section 15.3	Classification of sets of long orbits during the classification of packings with assumed symmetry.
-graph_classification	Section 13.3	Classifying graphs and tournaments.
-diophant	Section 16.3	A diophantine system, i.e., a system of positive integer equations.
-design	Section 14.1	A combinatorial design.
-design_table	Section 14.1	A table of combinatorial designs, possibly isomorphic. It can be used to construct large sets of designs. A large set is a set of designs satisfying certain properties.
-large_set_with_symmetry_assumption	Section 14.1	Classifying large sets of designs.
-set	Section 2.4	A set of integers.
-vector	Section 2.5	A vector of integers.
-symbolic_object	Section 2.7	A symbolic algebraic expression.
-combinatorial_object	Section 12.2	A set of combinatorial objects.
-geometry_builder	Section 12.6	Classifying incidence geometries.
-vector_ge	Section 5.3	A vector of group elements.
-action_on_forms		The action of a projective group on homogeneous polynomials of a given degree.
-orbits	Section 8.1	An object to compute orbits of a group.
-poset_classification_control	Section 8.2	Options for the poset classification algorithm.
-poset_classification_report_options	Section 8.2	Options for a reporting a poset classification.
-draw_options	Section 19.1	Options for graphical output.
-draw_incidence_structure_options	Section 17.3	Options for graphical output of incidence structures.
-arc_generator_control	Section 8.6.	Options for the arc classification algorithm.
-poset_classification_activity	Section 15.4	An activity for a poset of orbits.
-crc_code	Section 11.6	A CRC-code.
-mapping	Section 7.6	A mapping between projective spaces.
-variety	Section 7.5	A variety over a finite field.
-isomorph_arguments	Section 15.1	Options for isomorph testing.

Table 2.3: Orbiter Objects (Part II)

Orbiter Activities (Part 1)		
Command	Reference	Type of Activity
-finite_field_activity	Sections 3.1 and 3.2	An activity for finite fields.
-polynomial_ring_activity	Table 6.3	An activity for a polynomial ring.
-projective_space_activity	Section 4.1	An activity for a projective space.
-orthogonal_space_activity	Section 7.3	An activity for an orthogonal space.
-group_theoretic_activity	Section 5.5	An activity for a group.
-coding_theoretic_activity	Tables 11.3-11.5	An activity for a code.
-cubic_surface_activity	Section 9.2	An activity for a cubic surface.
-quartic_curve_activity	Section 9.5	An activity for a quartic curve.
-blt_set_activity	Table 15.16	An activity for a BLT-set.
-combinatorial_object_activity	Section 12.2	An activity for a combinatorial object.
-graph_theoretic_activity	Section 13.1	An activity for a graph.
-classification_of_cubic_surfaces_ with_double_sixes_activity	Section 9.2	An activity for a cubic surface.
-spread_table_activity	Section 15.3	An activity associated with a table of spreads.

Table 2.4: Orbiter Activities (Part 1)

Orbiter Activities (Part 2)		
Command	Reference	Type of Activity
-packing_classify_activity	Section 15.3	An activity related to classifying packings.
-packing_with_symmetry_assumption_activity	Section 15.3	An activity related to classifying packings with assumed symmetry group.
-packing_fixed_points_activity	Section 15.3	An activity related to classifying fixed points for packings with assumed symmetry group.
-graph_classification_activity	Section 13.3	An activity for a classification of graphs problem.
-diophant_activity	Section 16.3	An activity for a diophantine system.
-design_activity	Section 14.1	An activity for a combinatorial design.
-large_set_with_symmetry_assumption_activity	Section 14.3	An activity related to creating large sets of designs with assumed symmetry group.
-symbolic_object_activity	Table 2.12	An activity related to a symbolic object.
-BLT_set_classify_activity	Table 15.18	An activity related to a classification of BLT-sets.
-spread_classify_activity	Table 15.6	An activity related to a classification of spreads.
-spread_activity	Table 15.2	An activity related to a spread.
-translation_plane_activity	Table 15.8	An activity related to a translation plane.
-action_on_forms_activity		An activity related to classifying forms.
-orbits_activity	Table 8.3	An activity related to a computation of orbits.
-variety_activity	Section 7.5	An activity related to an algebraic variety.
-vector_ge_activity	Section 5.9	An activity related to a vector of group elements.
-combo_activity	Section 12.3	An activity related to a set of combinatorial objects.

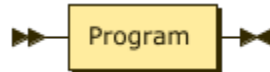
Table 2.5: Orbiter Activities (Part 2)

2.3 The Orbiter Dash Code Language

Orbiter commands are given in a language called Orbiter dash code. Here, we give a brief introduction into this language. We start with syntax diagrams. For a complete list of syntax diagrams of the Orbiter language, see Appendix A.1.

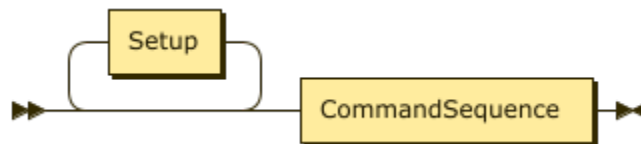
Every Orbiter Program follows the syntax derived from Grammar below:

Grammar:



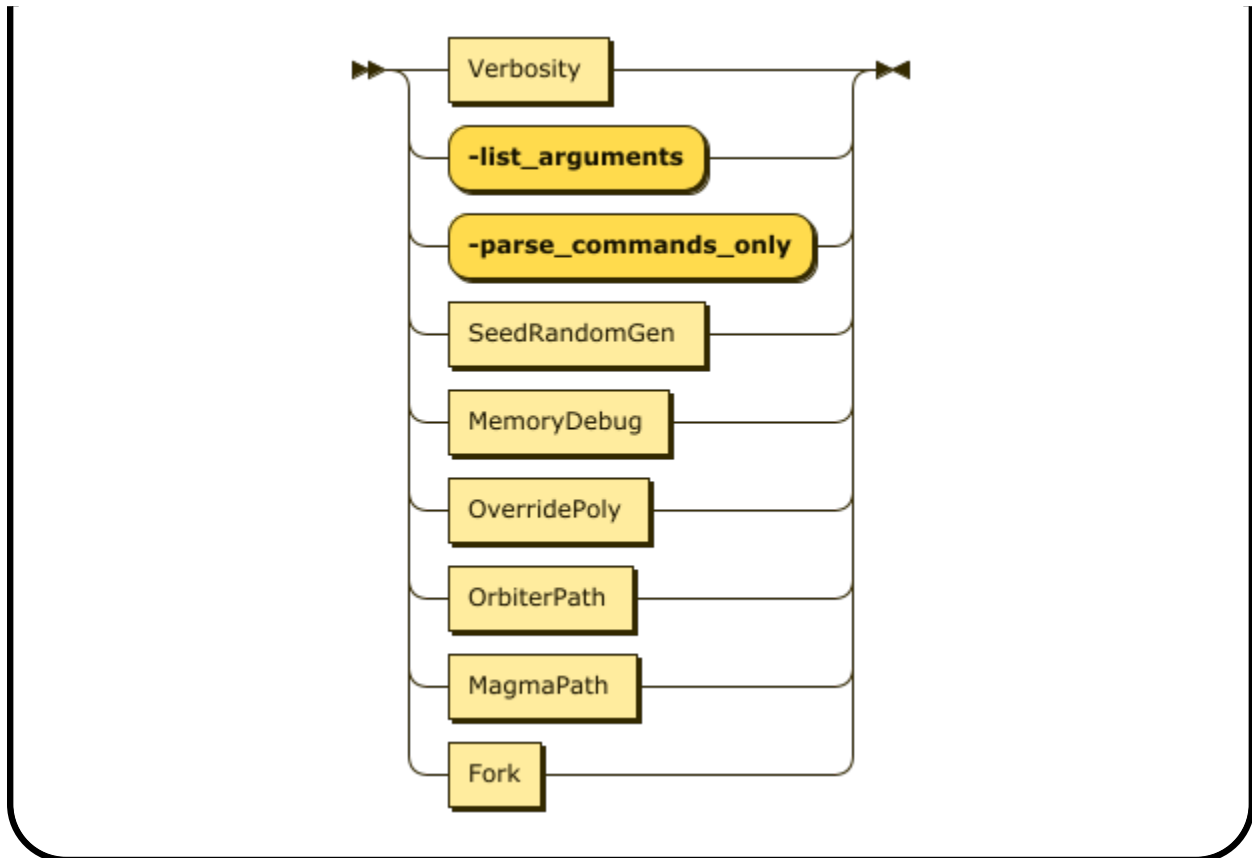
A Grammar is a Program:

Program:



A program has a setup phase and a sequence of commands. The setup phase happens only once, at the beginning of the program.

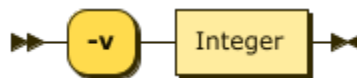
Setup:



Some of the setup commands require additional parameters. Here are three of these types of commands:

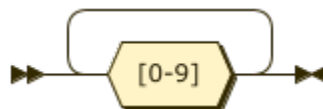
The verbosity is a parameter that determines how much text output should be produced on the console. Higher values of the verbosity lead to more text output. A verbosity value of 0 leads to minimal text output. To set the verbosity, the `-v` command is given in the setup. It expects the verbosity as an additional integer.

Verbosity:



An integer is given as a string of numerical characters:

Integer:



Some of the algorithms in Orbiter are randomized. This means that the output depends on a set of chosen pseudo random numbers. The generator for pseudo random numbers relies on a state, which is just an integer. Given a certain state, the pseudo random numbers are deterministic. In order to set the state of the random number generator, the `-seed` command can be used:

SeedRandomGen:

Orbiter relies on the C++ memory management system to store Objects and temporary data. One of the issues with memory management systems is that objects need to be freed once they become obsolete. It is a programming mistake to not free unused objects. Such a mistake is called a memory leak. Orbiter can help find memory leaks. To do so, the parameter `memory_debug` can be set to a positive number. Unfortunately, setting the `memory_debug` value can slow down the code quite a bit. So, this is not something that is recommended for the beginning user. Here is the syntax to set the `memory_debug` parameter:

MemoryDebug:

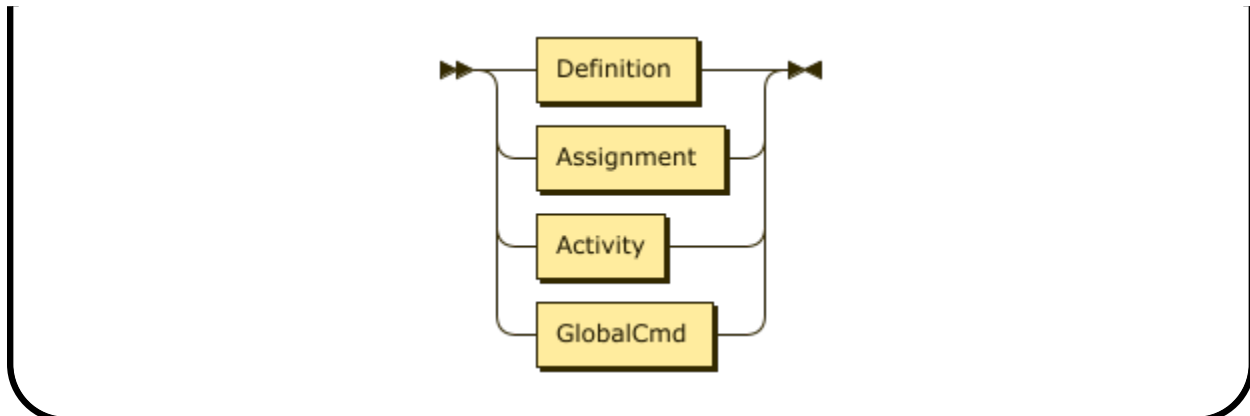
Let us go back to the main Orbiter program. After the initial setup, an Orbiter program consists of a command sequence. The command sequence consists of one or more commands.

CommandSequence:

An Orbiter command can be one of the following types:

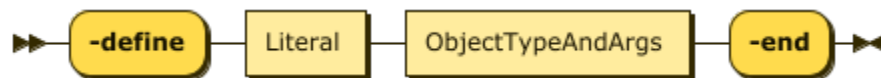
- a definition of a new Orbiter object,
- an assignment of an Object resulting from an activity,
- an activity for an existing Orbiter object.
- a global command which does not apply to a specific object.

Command:



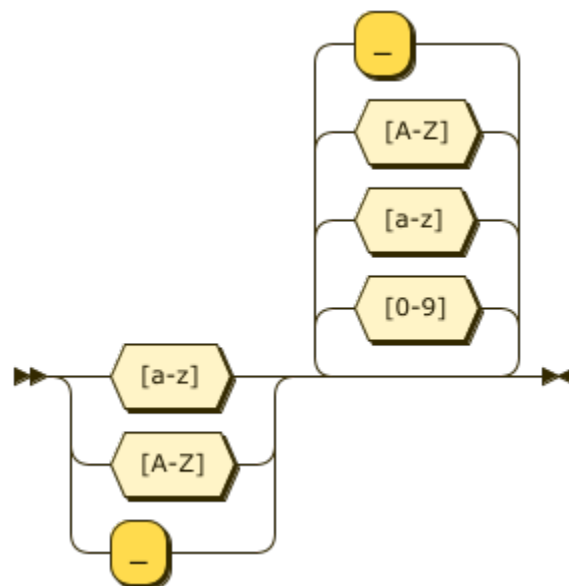
A definition defines a new Orbiter object and stores it in the symbol table, using a specified name. The name is given as a literal in the following syntax diagram. A literal is simply a string of symbols (letter, digit or underscore), but it must start with a character.

Definition:



A literal is a string of text and digits, possibly including the underscore character. It cannot start with a digit:

Literal:

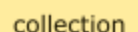


Because Orbiter has a large number of Objects, the syntax diagrams group them into six major categories.

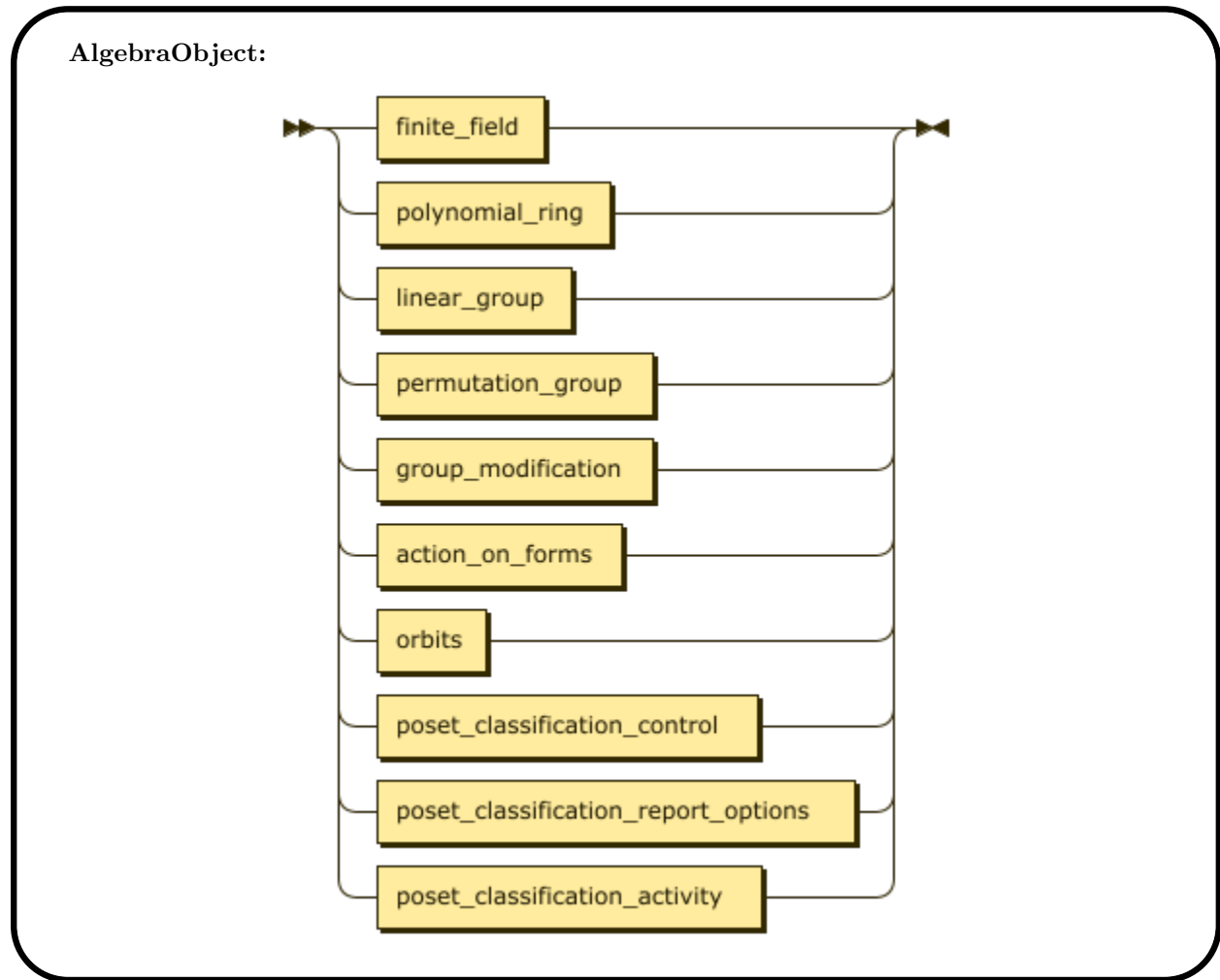


1. `DataSetObject`,
2. `AlgebraObject`,
3. `GeometryObject`,
4. `CombinatoricsObject`,
5. `CodingTheoryObject` and
6. `OtherObject`.

DataStructure:



Let us just look at the objects that arise in the Algebra category, and leave the other types for later:

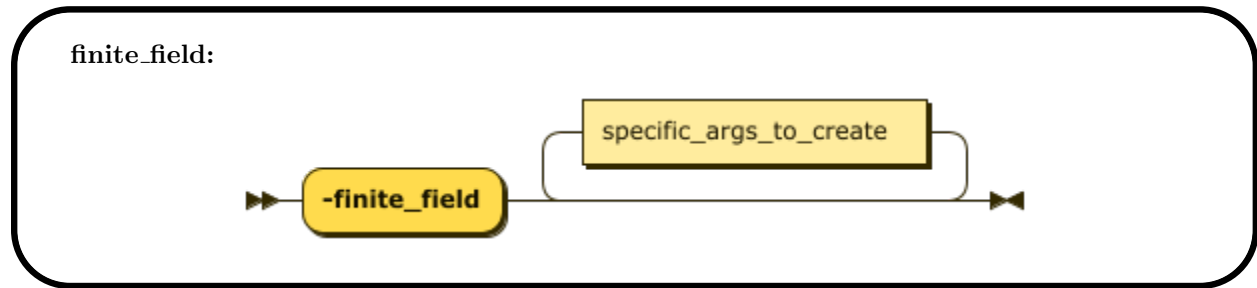


The AlgebraObject can be of several kinds:

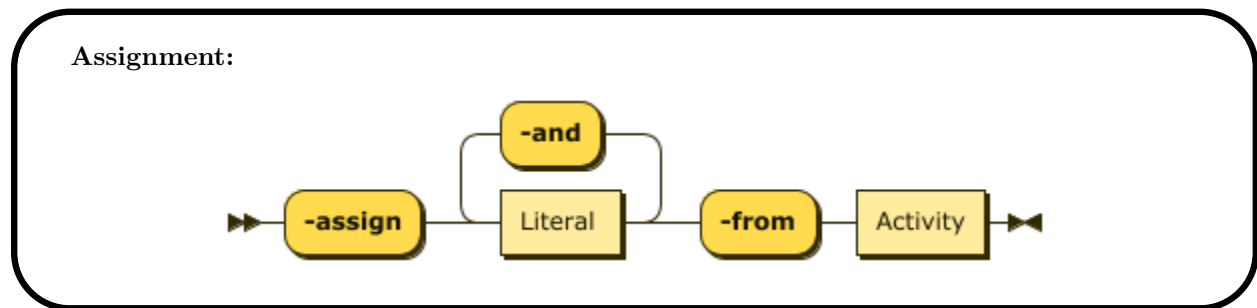
1. `finite_field`,
2. `polynomial_ring`,
3. `linear_group`,
4. `permutation_group`,
5. `group_modification`,
6. `action_on_forms`,
7. `orbits`,
8. `poset_classification_control`,
9. `poset_classification_report_options`,
10. `poset_classification_activity`.

Let us look at only the first type of object, called `finite_field`. This object provides a specific finite field $\mathbb{F}_q$. Of course, q must be a prime power. There are some limitations: The value of q cannot be too large, otherwise the system will run out of memory. For more on limitations, see Section 21.2.

The creation of a finite field object allows for several optional arguments. These optional arguments will be discussed in full detail in Chapter 3.

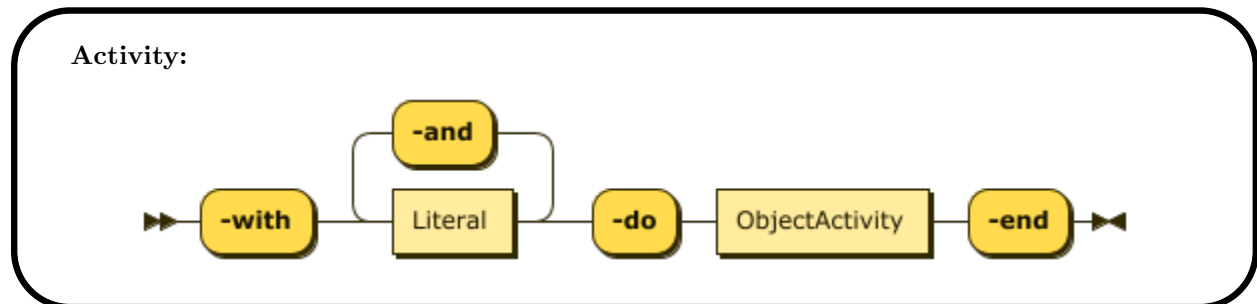


An assignment defines one or more new Orbiter objects and stores them in the symbol table, using a specified name. But this time, the objects that are assigned result from an activity.

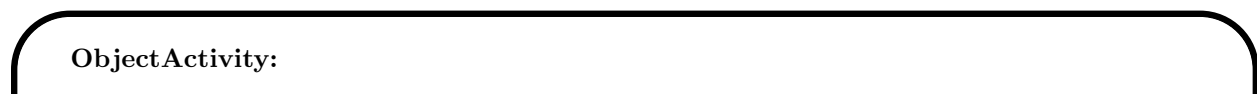


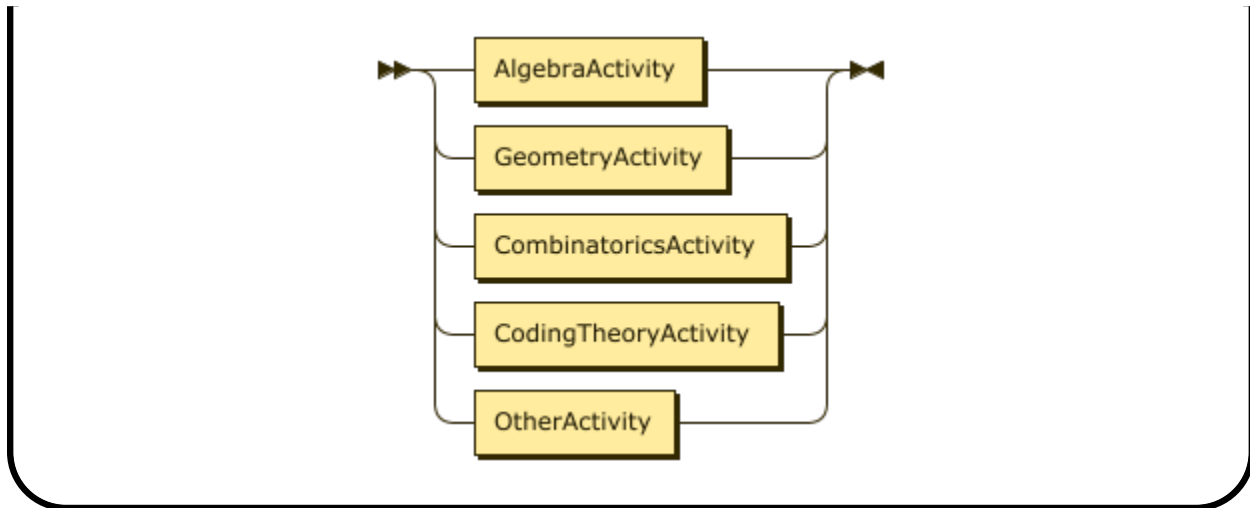
An assignment must be followed by an activity.

Activities are things we can do with Orbiter objects. They apply to one or more existing objects, which reside in the symbol table. Depending on the type of object, there is a list of activities that are available. An activity is a task that applies to the selected objects. The object is selected using the `-with` command. If necessary, additional objects can be selected using the `-and` clause. After that, the `do` command signifies that an activity is going to be selected. The type of activity is specified first. After that, optional commands specific to the activity may be given. The whole command sequence is terminated with a final `-end` command:

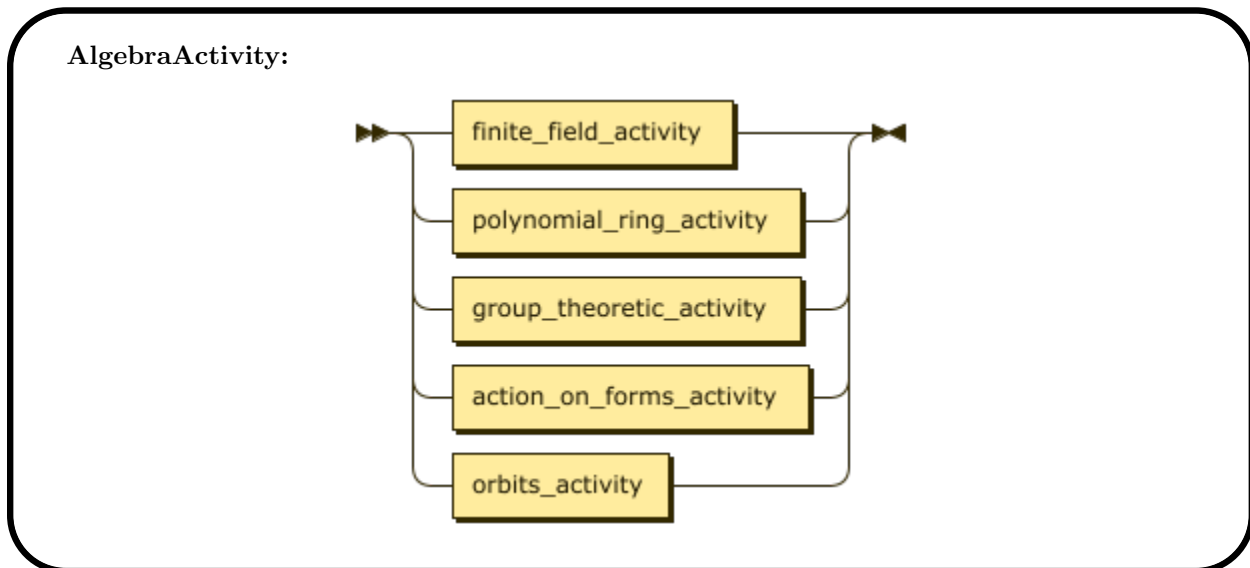


Different types of activities are available. The type of activity must match the type of object that was selected first. There are many different types of activities. For this reason, the activities are grouped into categories, just like the corresponding objects.





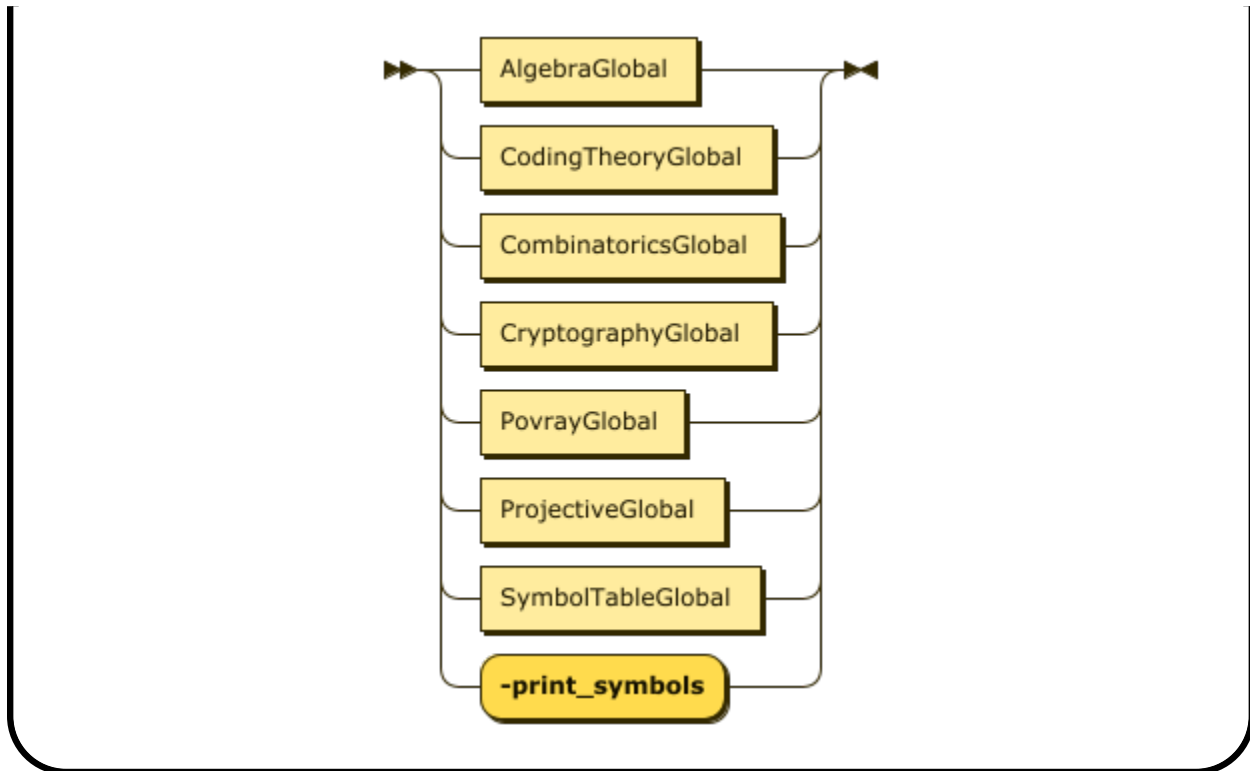
For instance, here are the activities related to algebra objects:



The activities related to the other types of objects can be found in Appendix A.1.

Finally, there are the global commands. They do not require an object or an activity.

GlobalCmd:



Global commands come in several categories:

1. AlgebraGlobal (see Table 10.1)
2. NumberTheoryGlobal (see Tables 10.2,10.3)
3. CodingTheoryGlobal (see Table 11.1)
4. CombinatoricsGlobal (see Tables 10.5,10.6)
5. CryptographyGlobal (see Tables 12.1,12.2)
6. PovrayGlobal (see Table 19.3)
7. ProjectiveGlobal (see Table 4.8)
8. SymbolTableGlobal
9. ToolkitGlobal (see Tables 21.1,21.2)

Let us now consider some simple examples of working with objects. Suppose we want to create the finite field $\mathbb{F}_2$ with two elements. The command

Code Snippet 4

```

1  object_F_2:
2  ▷ $(ORBITER) -v 3 -define F -finite_field -q 2 -end
3
```

creates the finite field $\mathbb{F}_2$ as object F . For more details, see Section 3.1. The command

Code Snippet 5

```
1    object_PG_3_2:
2    ▷ $(ORBITER) \
3    ▷ ▷ -define F -finite_field -q 2 -end \
4    ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end
5
```

creates the projective space $\text{PG}(3, 2)$. This is the three-dimensional space over the field $\mathbb{F}_2$. The `-projective_space` command requires additional options to set the dimension n and the field $\mathbb{F}_q$ in $\text{PG}(n, q)$. The `-n` command sets the dimension n . The `-field` command can be used to specify a particular field. The `-q` command can be used to create a generic field $\mathbb{F}_q$.

Set Builder Commands		
Command	Arguments	Purpose
-loop	a b c	creates the set $\{k \mid k = a + ic, i \in \mathbb{Z}, a \leq k < c\}$
-affine_function	a b	Apply the affine function $ax + b$.
-set_builder	set	Use a set as input to a construction.
-clone_with_affine_function	a b	Apply the affine function $ax + b$ to the elements x of a set specified using -set_builder.
-affine_function	a b	Apply the affine function $ax + b$ to the elements x of a set specified using -set_builder.
-here	elements	Create the set from the given list of elements. The elements of the list are comma separated.
-file	fname	Create a set from the elements listed in the given file.
-file_orbiter_format	fname	Create a set from the elements listed in the given file.

Table 2.6: Set Builder Commands

2.4 Set Builder

Table 2.6 shows Orbiter commands to create sets.

Here is an example. We create the set S of the first six prime numbers $\{2, 3, 5, 7, 11, 13\}$:

Code Snippet 6

```

1  set_of_primes:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define S -set -here "2,3,5,7,11,13" -end \
4  ▷ ▷ -print_symbols
5
```

The next command creates the integer interval $[0, 63]$. We use the -loop command to save us from typing out all elements of the set. The -loop command has three arguments: the start value s , the upper bound u , and the increment d . The numbers created have the form $s + kd$, $k = 0, 1, \dots$ as long as they are less than u .

Code Snippet 7

```

1  set_interval:
2  ▷ $(ORBITER) -v 2 -define S -set -loop 0 64 1 -end \
3  ▷ ▷ -print_symbols
4
```

2.5 Vector Builders

Table 2.7 shows Orbiter commands to create vector and matrix objects. Matrices are vectors with an additional format information. By specifying the number of rows, a vector turns into a matrix. So, to create a matrix, one creates a vector, and uses the `-format` command. Examples will be shown below.

Vectors can be defined in two different formats, called dense and sparse, respectively. In the dense format, the coefficients are listed in order from the lowest to the highest term. The `-dense` command creates the vector from a list of coefficients. The sparse format can be useful for coefficient vectors with few nonzero entries. It is a list of coefficient pairs, each of which describing one entry in the vector. One pair consists of the coefficient and the index of the term. The pairs are listed in sequence. The `-sparse` command creates the vectors from a given list of coefficient pairs.

If the option `-field` is given, Orbiter will assume that the vector entries lie in the interval $[0, q - 1]$, where q is the order of the finite field. If entries lie outside that interval, an error is raised. There are limits on the size of q , see Section 21.2. It is safe to assume that any number q less than $2^{31} - 1$ is fine. On 64 bit systems (with Orbiter compiled in 64 bit mode) we can have $q \leq 2^{63} - 1$.

Here is an example. We first create the field $\mathbb{F}_5$, and then create the vector $v = (0, 1, 2, 3, 4)$. The `-field` option refers to the finite field created previously. The `-dense` option allows to enter the vector coefficients on the command line.

Code Snippet 8

```
1    vector_example1:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 5 -end \
4    ▷ ▷ -define v -vector -field F -dense "0,1,2,3,4" -end \
5    ▷ ▷ -print_symbols
6
```

Vectors can also be read from file. The `-file` option can be used to name a csv file.

A vector can also serve as a matrix. The `-format  $k$` option can be used to specify the number k of rows. The number of columns is determined as n/k , where n is the length of the vector given. For instance, the next example creates a 2×3 matrix over $\mathbb{F}_5$:

Code Snippet 9

```
1    vector_example2:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 5 -end \
4    ▷ ▷ -define v -vector -field F -format 2 -dense "0,1,2,3,4,0" -end \
5    ▷ ▷ -print_symbols
6
```

Orbiter has a command to create vectors whose entries repeat. For instance, the following code creates a vector of length 11 whose entries repeat over the sequence 0, 1, 2, 3. It is not necessary that the vector length is an integer multiple of the length of the repeating sequence.

Vector Builder Commands		
Command	Arguments	Purpose
-field	F	Assume that the vector entries are in the range $[0, q - 1]$, where q is the order of the field F . An error is raised if not.
-allow_negatives		Allow negative numbers when using finite fields. A negative number will be considered modulo the characteristic of the field. For instance, in the field $\mathbb{F}_5$, a -1 is considered equal to 4.
-dense		Assume that the vector entries are given as a comma separated list (dense coding).
-compact		Assume that the vector entries are given as a continuous list without commas or other any kind of whitespace.
-repeat	pattern n	Repeat the given pattern n times.
-format	r	Assume that the data is a matrix with r rows.
-file	file	Read the data from the given csv-file.
-file_column	file column	Read the data from the given column in the given csv-file. The entries must be integers or vectors of integers. The entries must all have the same size.
-union_of_column	file column	Read the sets of integers from the given column in the given csv-file. Then forms the union of the sets.
-load_csv_no_border	file	Read the data from the given file, assuming that there are no headers.
-load_csv_data_column	file c	Read the data from column c in the given csv-file.
-sparse	data	Provide data in sparse form, i.e. as pairs value and position.
-flags	nb.rows nb.cols flags	Specify an incidence matrix by size and by flags.
-concatenate	list	Concatenate the given vectors. List is a comma separated list of vectors that have been created before.
-loop	$s \ u \ d$	Create vector entries of the form $s + id < u$, $i \in \mathbb{Z}$. The upper bound u itself will not be part of the sequence.
-index_of_support	vec	Create the index set of support of the vector vec. This is the set of all i such that $\text{vec}[i]$ is nonzero.
-permutation_matrix	list	Create the permutation matrix defined by the 0-based list of integers.
-permutation_matrix_inverse	list	Create the inverse permutation matrix defined by the 0-based list of integers.

Table 2.7: Vector Builder Commands

Code Snippet 10

```

1    vector_example_repeat:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define v -vector -repeat "0,1,2,3" 11 -end \
4    ▷ ▷ -print_symbols
5

```

The sequence 0,1,2,3 is repeated sufficiently often to make a vector of length 11. This creates the vector

$$(0, 1, 2, 3, 0, 1, 2, 3, 0, 1, 2).$$

In order to create a constant vector, the `-repeat` command can be used as well. Simply use a repeat sequence consisting of a single number. For instance, the following example creates the all-one vector of length 11:

Code Snippet 11

```

1    vector_example_all_one_11:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define v -vector -repeat 1 11 -end \
4    ▷ ▷ -print_symbols
5

```

This code will create the all-one vector of length 11:

$$(1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1).$$

A loop command can create sequences of numbers $a_0, a_1, \dots$. It takes three arguments: s , u , and d . The values of the sequence are

$$a_i = s + id, \dots k = 0, 1, 2, \dots, \quad a_i < u.$$

So, the values of the sequence increases in steps of d , and are forced to be less than the upper bound u . For instance, in order to create all integers from 0 to 7, we use the command:

Code Snippet 12

```

1    vector_loop_8:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define v -vector -loop 0 8 1 -end \
4

```

If we wanted to create the odd integers less than or equal to 16, we could do this:

Code Snippet 13

```

1    vector_loop_odd_16:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define v -vector -loop 1 16 2 -end \
4

```

This command creates the vector

$$(1, 3, 5, 7, 9, 11, 13, 15).$$

Two vectors can be concatenated, as demonstrated in the following example:

Code Snippet 14

```

1    vector_concatenate:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define a -vector -compact "1,2,3" -end \
4    ▷ ▷ -define b -vector -compact "4,5,6" -end \
5    ▷ ▷ -define c -vector -compact "7,8,9" -end \
6    ▷ ▷ -define abc -vector -concatenate a,b,c -end
7

```

The vectors (1, 2, 3), (4, 5, 6) and (7, 8, 9) are created. After that, the concatenation command produces

$$(1, 2, 3, 4, 5, 6, 7, 8, 9).$$

The vector command can also be used to create matrices. A matrix is a vector with additional formatting information. The vector is the coefficient vector, using row-major ordering. For instance, the following command creates the vector

$$(1, 2, 3, 4, 5, 6),$$

which by means of the `-format 2` command turns into the matrix

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}.$$

Here is the command:

Code Snippet 15

```

1    matrix_example1:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define v -vector -format 2 \
4    ▷ ▷ ▷ -dense "1,2,3,4,5,6" -end \
5    ▷ ▷ -print_symbols
6

```

For large vectors, the `sparse` command can be used to enter non-zero coefficients as a list of pairs. For instance,

Code Snippet 16

```

1    vector_example_sparse:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 5 -end \
4    ▷ ▷ -define v -vector -field F -format 4 -sparse 20 "1,0,1,19" -end \
5    ▷ ▷ -print_symbols
6

```

creates a vector of length 20 and sets the 0-th and the 19-th coefficient to 1. So, the vector is

$$(1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1).$$

Finally, the vector is displayed as a four-rowed matrix:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}.$$

Makefile variables can be used to hold the coefficients in a separate place. This helps reduce clutter. For instance, the following command creates the generator matrix of the Hamming code:

Code Snippet 17

```
1  HAMMING_CODE_GENERATOR="\
2  1,0,0,0,0,1,1, \
3  0,1,0,0,1,0,1, \
4  0,0,1,0,1,1,0, \
5  0,0,0,1,1,1,1"
6
```

Code Snippet 18

```
1  matrix_example2:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -define v -vector -field F -format 4 \
5  ▷ ▷ ▷ -dense $(HAMMING_CODE_GENERATOR) -end \
6  ▷ ▷ -print_symbols
7
```

For matrices over small fields, the `-compact` option can be given. This works similar as the `-dense` option, but the entries are not separated by commas. This is only good for single digit data, as it arises with very small fields. For instance, the following code creates a 22×22 matrix over the binary field:

Code Snippet 19

```
1  CONWAY_GEN1="\
2  1101110001000001010000\
3  1111010111110100001011\
4  0000001000000100010101\
5  1111100110110001001110\
6  0101010000000010011101\
7  0000010000000100010101\
8  0010000000000100010101\
9  0001000011000000111111\
10 1110100100110100010011\
11 0000000000000110010101\
12 00000000000100100010101\
13 0110111111010011101111\
14 0000000000001100010101\
15 0000000000000100000101\
16 0000000001000100010101\
```

```

17  0000000000000100011101\
18  0001000110000010011010\
19  00000000000000000010101\
20  0000000000000101010101\
21  0000000000000100010100\
22  0000000000000100010111\
23  0000000000000100010001"
24

```

Code Snippet 20

```

1  matrix_example_co_1:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -define v -vector -field F -format 22 \
5  ▷ ▷ ▷ -compact $(CONWAY_GEN1) -end \
6  ▷ ▷ -print_symbols
7

```

The next command creates a vector and then computes the set of position where the vector is nonzero:

Code Snippet 21

```

1  vector_index_set:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define v -vector -dense "0,1,0,1,0,1,0,1,0,1,0,1" -end \
4  ▷ ▷ -define v2 -vector \
5  ▷ ▷ ▷ -index_of_support \
6  ▷ ▷ ▷ v \
7  ▷ ▷ -end \
8

```

The next command creates the permutation matrix associated with the permutation $[1, 2, 3, 0]$

Code Snippet 22

```

1  vector_index_set:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define v -vector -dense "0,1,0,1,0,1,0,1,0,1,0,1" -end \
4  ▷ ▷ -define v2 -vector \
5  ▷ ▷ ▷ -index_of_support \
6  ▷ ▷ ▷ v \
7  ▷ ▷ -end \
8

```

The following matrix is created:

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}.$$

Vector of Integers Activities		
Command	Arguments	Purpose
-report		Produce a report of the vector or matrix.
-missing_sets	n	This assumes that the vector is a matrix whose rows describe a system of sets. Each set has size k, say. The entries are drawn from 0 to n - 1. The command produces a file containing the missing sets. The missing sets are the subsets of size k which are not member of the set system.

Table 2.8: Vector of Integers Activities

Table 2.8 shows Orbiter activities for vector and matrix objects.

Vector of Group Elements Builder Commands		
Command	Arguments	Purpose
-action	A	Set the action type for the group elements.
-read_csv	fname column	Read group elements from the given csv file. The data is taken from the given column.
-vector_data	vector	Construct group elements from the given vector data.

Table 2.9: Vector of Group Elements Builder Commands

2.6 Vector of Group Elements

Orbiter can create vectors of group elements. Table 2.9 shows the Orbiter commands that can be used to create a vector of group elements.

Let us consider an example. Our goal is to describe the action of the elements of class 2A in the stabilizer of the Hirschfeld surface. The group element is a Baer collineation:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1$$

The class representative is stored in a makefile variable:

Code Snippet 23

```
1 CLASS_2A_REP="1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,1"
2
```

The first command creates the conjugacy class 2A. The elements will be written to a csv file:

Code Snippet 24

```
1 Hirschfeld_Class_2A:
2   ▷ $(ORBITER) -v 9 \
3   ▷ ▷ -orbiter_path $(ORBITER_PATH) \
4   ▷ ▷ -define G -linear_group -PGGL 4 4 \
5   ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld_Stab" \
6   ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
7   ▷ ▷ ▷ -end \
8   ▷ ▷ -with G -do \
9   ▷ ▷ -group_theoretic_activity \
10  ▷ ▷ ▷ -conjugacy_class_of "2A" $(CLASS_2A_REP) \
11  ▷ ▷ -end
12
```

The filename is stored in a makefile variable:

Code Snippet 25

```

1  FILE_NAME_CLASS_2A="PGGL_4_4.Subgroup.Hirschfeld.Stab.51840_class_of_2A.csv"
2

```

The next command reads the elements of class 2A from this file. It then prints the action of these elements on the various objects associated with the cubic surface:

Code Snippet 26

```

1  surface_4_0_export_action:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define Class2A -vector -file_column $(FILE_NAME_CLASS_2A) elements -end \
4  ▷ ▷ -define F -finite_field -q 4 -end \
5  ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
6  ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
7  ▷ ▷ -define G -linear_group -PGGL 4 4 -end \
8  ▷ ▷ -define gens_ge -vector_ge \
9  ▷ ▷ ▷ -action G \
10 ▷ ▷ ▷ -vector_data Class2A \
11 ▷ ▷ -end \
12 ▷ ▷ -with S -do \
13 ▷ ▷ -cubic_surface_activity \
14 ▷ ▷ ▷ -export_something_with_group_element \
15 ▷ ▷ ▷ "action_on_tritangent_planes" gens_ge \
16 ▷ ▷ -end \
17 ▷ ▷ -with S -do \
18 ▷ ▷ -cubic_surface_activity \
19 ▷ ▷ ▷ -export_something_with_group_element \
20 ▷ ▷ ▷ "action_on_double_sixes" gens_ge \
21 ▷ ▷ -end \
22 ▷ ▷ -with S -do \
23 ▷ ▷ -cubic_surface_activity \
24 ▷ ▷ ▷ -export_something_with_group_element \
25 ▷ ▷ ▷ "action_on_lines" gens_ge \
26 ▷ ▷ -end
27

```

For activities with vectors of group elements, see Section 5.9.

2.7 Symbolic Objects

Symbolic objects in Orbiter represent algebraic expressions with coefficients in finite fields. They consist of the elementary arithmetic operations, numbers, variables and exponents. In Orbiter, an expression can be parsed from a text representation. The purpose of parsing is to produce an internal representation of the expression as a syntax tree. The syntax tree is an efficient way to represent the expression, and algebraic operations will be performed by manipulating the syntax tree. The process of creating the abstract syntax tree from the text representation is called parsing. Orbiter can deal with vectors and matrices of symbolic expressions. These are simply sequences of symbolic objects.

Tables 2.10-2.11 show Orbiter commands to create symbolic objects.

Table 2.12 show Orbiter activities for symbolic objects.

Let us consider some examples. The elements of the finite field $\mathbb{F}_q$ are coded as integers in the interval from 0 to $q - 1$. For more information about finite fields, see Chapter 3. It is fair to assume that 0 represents zero and 1 represents one. Numbers outside the interval 0 to $q - 1$ are not allowed and lead to error messages when encountered. However, when parsing a algebraic formula from text, a negative sign is interpreted as a unary or binary minus, as hence one can say that negative numbers are allowed at the time of input.

The first example creates the polynomial $1 - X - X^2$ over $\mathbb{F}_7$:

Code Snippet 27

```
1  symbolic_poly:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define F -finite_field -q 7 -end \
4  ▷ ▷ -define M -symbolic_object \
5  ▷ ▷ ▷ -field F \
6  ▷ ▷ ▷ -text "-X^2-X+1"\
7  ▷ ▷ -end \
8
```

The output is

$$M = 1 + (6X) + (6X^2)$$

The coefficients of X and X^2 are 6, because $6 \equiv -1 \pmod{7}$ and Orbiter chooses field elements of $\mathbb{F}_q$ to lie in the interval $[0, q - 1]$.

The second example creates the polynomial CRC32, using the makefile variable

Code Snippet 28

```
1  CRC32_ETHERNET_POLY="X^32+X^26+X^23+X^22+X^16+\
2  X^12+X^11+X^10+X^8+X^7+X^5+X^4+X^2+X^1+1"
3
```

The command

Code Snippet 29

```
1  symbolic_CRC32:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -define M -symbolic_object \
```

Creating Symbolic Objects I		
Command	Arguments	Purpose
-label_txt	label	Choose an ascii-label of the object.
-label_tex	label	Choose a tex-label of the object.
-managed_variables	list	A comma-separated list of managed variables may be given. The manage variables can later be matched to the variables of a polynomial ring. The other variables can be used as parameters.
-text	text	A comma separated sequence of algebraic expressions in text format.
-field	F	The finite field of coefficients in the expression.
-ring	R	
-file	filename	
-matrix	r	Consider the expression vector as a matrix with r rows, read in row-major ordering.
-determinant	s	Compute the symbolic determinant of the source object s . The object s must be a square matrix.
-characteristic_polynomial	λ s	Compute the characteristic polynomial of the square matrix s using the variable λ .
-substitute	v exp t	Substitute the values v for the variables t in the expressions given in exp . v and t must be vectors of the same length.
-simplify	s	Simplify the object s .
-reduce_exponents	s	Reduce exponents modulo $q - 1$ if greater than or equal to q . Don't reduce $q - 1$ to 0.
-expand	s	Expand the object s .
-right_nullspace	s	Compute the right nullspace of the object s , which must be a fully numerical matrix.
-minor	s i j	Compute the (i, j) -minor of the matrix object s . s must be a square matrix.
-symbolic_nullspace	s	Compute the right nullspace of the object s , which must be a symbolic matrix.
-stack_matrices_vertically	list	Stack the given matrices in the list vertically. The matrices must have the same number of columns.

Table 2.10: Creating Symbolic Objects I

Creating Symbolic Objects II		
Command	Arguments	Purpose
-multiply_2x2_from_the_left	$s \ A2 \ i \ j$	Multiply the 2×2 matrix $A2$ from the left to s , applying to rows i and j .
-matrix_entry	$s \ i \ j$	Select the (i, j) -entry of the matrix s .
-vector_entry	$s \ i$	Select the j -th entry of the vector s . Using row-major ordering, a matrix is considered a vector also.
-collect	$s \ t$	Collect by coefficients of t in the expression s .
-collect_by_degree	s	Collect the expression s by degree. The result is a vector of the homogeneous components of s .
-collect_by	$s \ R$	Collect the expression s by monomials of the ring R . The ring R must be univariate. For multivariate rings, use -collect_by_ring.
-collect_by_ring	$s \ R$	Collect the expression s by monomials of the ring R .
-encode_CRC	$l \ d \ c$	Encode the data d by blocks of length l using the check polynomial c .
-decode_CRC	$l \ d \ c$	Decode the data d by blocks of length l using the check polynomial c .
-submatrix	$M \ i \ m \ j \ n$	Form the $m \times n$ submatrix of M starting in position (i, j) .
-do_not_simplify		Do not simplify the output object.
-write_trees_during_expand		Export trees during the expand command.

Table 2.11: Creating Symbolic Objects II

Activities for Symbolic Objects		
Command	Arguments	Purpose
-export		Export the symbolic object.
-print		Print the symbolic object.
-as_vector		Convert the symbolic object to a vector of integers and print this vector.
-homogenize		Print the vector as a homogenized polynomial in the variables X and Y.
-latex		Print the symbolic object in latex format.
-evaluate_affine		Computes the evaluation vector of the symbolic object. The evaluation vector is indexed by all affine points coordinatized by the variables. The entry is the value of the formula when the coordinates of the affine point are substituted.
-collect_monomials_binary		Collects the monomials, and records the subscripts of terms which appear. This is made for polynomials over GF(2). A csv file is written. The function assumes that the formula is expanded, i.e., a sum of monomials.

Table 2.12: Activities for Symbolic Objects

```

5   ▷ ▷ ▷ -field F \
6   ▷ ▷ ▷ -text $(CRC32_ETHERNET_POLY) \
7   ▷ ▷ -end \
8   ▷ ▷ -define coeffs -symbolic_object \
9   ▷ ▷ ▷ -field F \
10  ▷ ▷ ▷ -collect M X \
11  ▷ ▷ -end \
12  ▷ ▷ -with coeffs -do -symbolic_object_activity \
13  ▷ ▷ ▷ -print \
14  ▷ ▷ -end
15

```

produces the following output:

$$M = 1 + X + X^2 + X^4 + X^5 + X^7 + X^8 + X^{10} + X^{11} + X^{12} + X^{16} + X^{22} + X^{23} + X^{26} + X^{32}$$

The second example creates the BCH polynomial Alfa, defined over the field $\mathbb{F}_{256}$, using the makefile variable

Code Snippet 30

```

1   BCH_POLYNOMIAL_ALFA_F256="X^2 + 167*X + 214"
2

```

The command

Code Snippet 31

```

1    symbolic_CRC_alfa:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 256 -end \
4    ▷ ▷ -define M -symbolic_object \
5    ▷ ▷ ▷ -field F \
6    ▷ ▷ ▷ -text $(BCH_POLYNOMIAL_ALFA_F256) \
7    ▷ ▷ -end \
8

```

produces the following output:

$$M = 214 + X^2 + (167X)$$

The next example creates the CRC polynomial Bravo over the field $\mathbb{F}_{256}$. The makefile variable is:

Code Snippet 32

```

1    BCH_POLYNOMIAL_BRAVO_F256="X^4+213*X^3+27*X^2+23*X+1"
2

```

The actual command is:

Code Snippet 33

```

1    symbolic_CRC_bravo:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 256 -end \
4    ▷ ▷ -define M -symbolic_object \
5    ▷ ▷ ▷ -field F \
6    ▷ ▷ ▷ -text $(BCH_POLYNOMIAL_BRAVO_F256) \
7    ▷ ▷ -end \
8

```

The output is

$$M = 1 + X^4 + (27X^2) + (23X) + (213X^3)$$

Let us consider the formula $(a + b)^3$ and expand:

Code Snippet 34

```

1    symbolic_test_5_expand:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 13 -end \
4    ▷ ▷ -define M -symbolic_object \
5    ▷ ▷ ▷ -field F \
6    ▷ ▷ ▷ -text "(a+b)^3" \
7    ▷ ▷ -end \
8    ▷ ▷ -define M1 -symbolic_object \
9    ▷ ▷ ▷ -field F \
10   ▷ ▷ ▷ -expand M \
11   ▷ ▷ -end \
12   ▷ $(PDFLATEX) M.tex

```

```

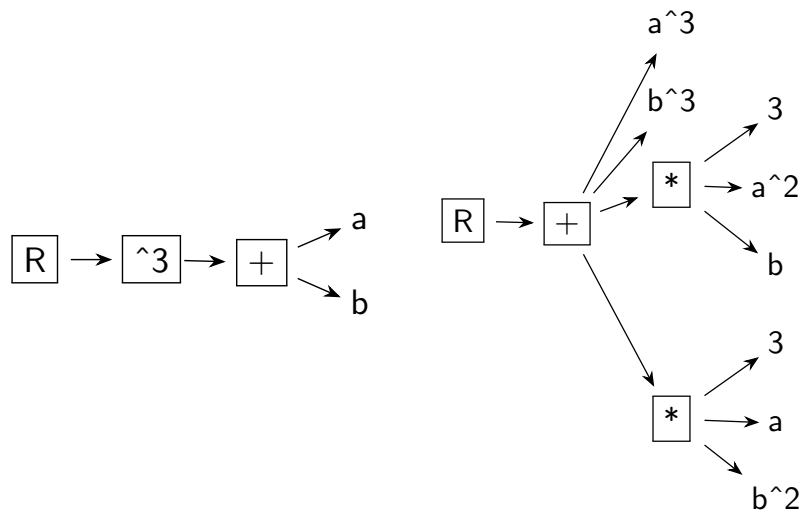
13   ▷ $(OPEN) M.pdf
14   ▷ $(PDFLATEX) M1.tex
15   ▷ $(OPEN) M1.pdf
16

```

The output is

$$M1 = \left[a^3 + b^3 + (3a^2b) + (3ab^2) \right]$$

It is also possible to inspect the representation of the expressions as abstract syntax trees. The node labeled R signifies the root node of the tree. The tree on the left is the syntax tree before expansion, whereas the tree on the right is the syntax tree after expansion:



The command

Code Snippet 35

```

1   test_expand_1:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define Fq -finite_field -q 31 -end \
4   ▷ ▷ -define f -symbolic_object \
5   ▷ ▷ ▷ -field Fq \
6   ▷ ▷ ▷ -text "a*(x+y)" \
7   ▷ ▷ -end \
8   ▷ ▷ -define fe -symbolic_object \
9   ▷ ▷ ▷ -field Fq \
10  ▷ ▷ ▷ -expand f \
11  ▷ ▷ ▷ -write_trees_during_expand \
12  ▷ ▷ -end
13  ▷ $(PDFLATEX) fe_0_efsxsfcscsc.tex
14  ▷ $(OPEN) fe_0_efsxsfcscsc.pdf
15

```

tests that

$$a(x + y) = ax + ay.$$

The command

Code Snippet 36

```

1  test_expand_2:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define Fq -finite_field -q 31 -end \
4  ▷ ▷ -define f -symbolic_object \
5  ▷ ▷ ▷ -field Fq \
6  ▷ ▷ ▷ -text "a*(-(x-y))" \
7  ▷ ▷ -end \
8  ▷ ▷ -define fe -symbolic_object \
9  ▷ ▷ ▷ -field Fq \
10 ▷ ▷ ▷ -expand f \
11 ▷ ▷ ▷ -write_trees_during_expand \
12 ▷ ▷ -end
13 ▷ $(PDFLATEX) fe_0_efsxsfsfscsc.tex
14 ▷ $(OPEN) fe_0_efsxsfsfscsc.pdf
15
```

tests that

$$a(-(x - y)) = -ax + ay.$$

The command

Code Snippet 37

```

1  test_expand_3:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define Fq -finite_field -q 31 -end \
4  ▷ ▷ -define f -symbolic_object \
5  ▷ ▷ ▷ -field Fq \
6  ▷ ▷ ▷ -text "a*(-(x-y)+(x+y))" \
7  ▷ ▷ -end \
8  ▷ ▷ -define fe -symbolic_object \
9  ▷ ▷ ▷ -field Fq \
10 ▷ ▷ ▷ -expand f \
11 ▷ ▷ ▷ -write_trees_during_expand \
12 ▷ ▷ -end
13 ▷ $(PDFLATEX) fe_0_efs.tex
14 ▷ $(OPEN) fe_0_efs.pdf
15 ▷ $(PDFLATEX) fe_0_efsx.tex
16 ▷ $(OPEN) fe_0_efsx.pdf
17 ▷ $(PDFLATEX) fe_0_efsxsfs.tex
18 ▷ $(OPEN) fe_0_efsxsfs.pdf
19 ▷ $(PDFLATEX) fe_0_efsxsfsfs.tex
20 ▷ $(OPEN) fe_0_efsxsfsfs.pdf
21 ▷ $(PDFLATEX) fe_0_efsxsfsfsc.tex
22 ▷ $(OPEN) fe_0_efsxsfsfsc.pdf
23 ▷ $(PDFLATEX) fe_0_efsxsfsfscsefsx.tex
```

```

24   ▷ $(OPEN) fe_0_efsxsfcsefsx.pdf
25   ▷ $(PDFLATEX) fe_0_efsxsfcsefsxsf.tex
26   ▷ $(OPEN) fe_0_efsxsfcsefsxsf.pdf
27   ▷ $(PDFLATEX) fe_0_efsxsfcsefsxsfscsc.tex
28   ▷ $(OPEN) fe_0_efsxsfcsefsxsfscsc.pdf
29

```

tests that

$$a(-(x - y) + (x + y)) = 2ay.$$

The output is

$$fe = (2ay)$$

The command shows the abstract syntax trees for the intermediate steps in the algebraic simplification.

The command

Code Snippet 38

```

1   test_expand_4:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define Fq -finite_field -q 31 -end \
4   ▷ ▷ -define f -symbolic_object \
5   ▷ ▷ ▷ -field Fq \
6   ▷ ▷ ▷ -text "x*(a+(x*(a+(x*(a+x*(a+x*(a+x))))))" \
7   ▷ ▷ -end \
8   ▷ ▷ -define fx -symbolic_object \
9   ▷ ▷ ▷ -field Fq \
10  ▷ ▷ ▷ -expand f \
11  ▷ ▷ ▷ -write_trees_during_expand \
12  ▷ ▷ -end
13  ▷ $(PDFLATEX) fx_0_efsxsfcsefsxsfscsefsxsfscsefsxsfscsefsxsfscsefsxsfscsc.tex
14  ▷ $(OPEN) fx_0_efsxsfcsefsxsfscsefsxsfscsefsxsfscsefsxsfscsefsxsfscsc.pdf
15

```

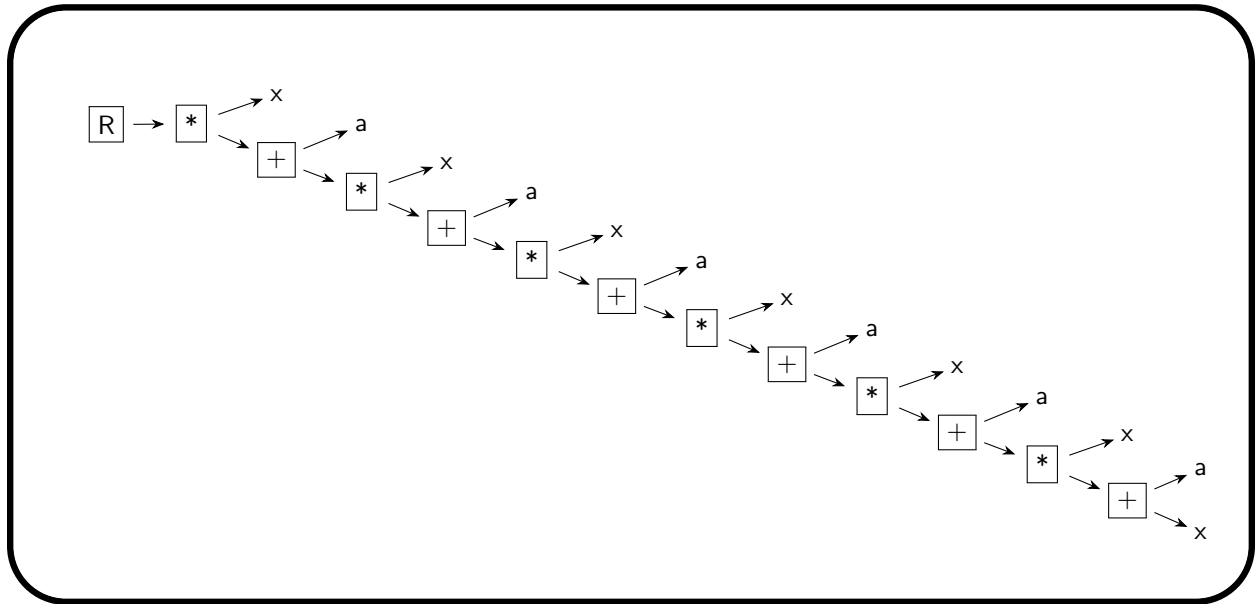
tests that

$$x * (a + (x * (a + (x * (a + x * (a + x * (a + x * (a + x))))))) = ax + ax^2 + ax^3 + ax^4 + ax^5 + ax^6 + x^7.$$

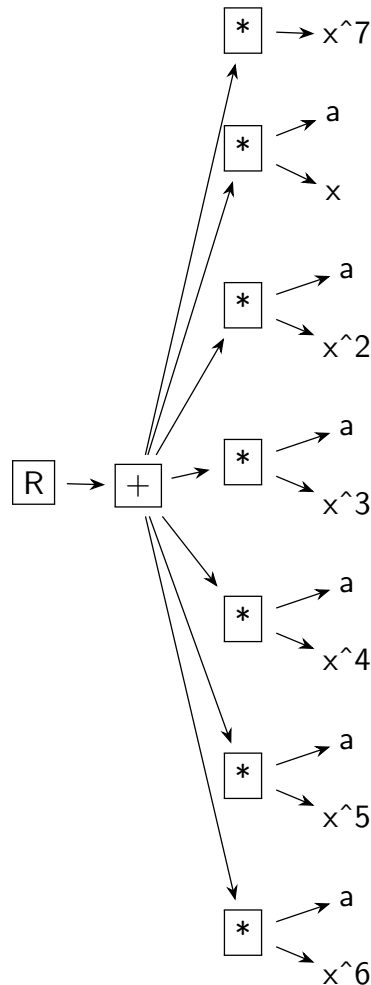
The output is

$$fx = x^7 + (ax^2) + (ax^3) + (ax^4) + (ax^5) + (ax^6) + (ax)$$

The abstract syntax tree before expansion is:



The abstract syntax tree after expansion is:



The abstract syntax trees for the intermediate steps during expansion are available also.

Objects are stored in the Orbiter symbol table. There is only one table. There can be only one object with any given name. Symbolic objects can recycle objects defined earlier using a process called implicit substitution. In this process, the object being referenced is taken from the symbol table and substituted immediately. For instance, in the next example we create an object $a = x + y$ and then another object $b = a + z$ which immediately expands into $b = x + y + z$. This is because the object a already exists. Implicit substitution is non-recursive, so the substituted syntax tree is not subjected to implicit substitution itself. This prevents unlimited recursion which would result in crashes.

Code Snippet 39

```

1    symbolic_objects_building_on_earlier_objects:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 2 -end \
4    ▷ ▷ -define a -symbolic_object \
5    ▷ ▷ ▷ -field F \
6    ▷ ▷ ▷ -text "x+y"\
7    ▷ ▷ -end \

```

```

8    ▷ ▷ -define b -symbolic_object \
9    ▷ ▷ ▷ -field F \
10   ▷ ▷ ▷ -text "a+z"\
11   ▷ ▷ -end
12

```

The output of the command is:

$$b = x + y + z$$

Self-referential objects are allowed. The following command is legal:

Code Snippet 40

```

1    symbolic_objects_self_referential_1:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 2 -end \
4    ▷ ▷ -define x -symbolic_object \
5    ▷ ▷ ▷ -field F \
6    ▷ ▷ ▷ -text "x"\
7    ▷ ▷ -end
8

```

The output is

$$x = x$$

Likewise, the following command is ok:

Code Snippet 41

```

1    symbolic_objects_self_referential_2:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 2 -end \
4    ▷ ▷ -define x -symbolic_object \
5    ▷ ▷ ▷ -field F \
6    ▷ ▷ ▷ -text "x"\
7    ▷ ▷ -end \
8    ▷ ▷ -define a -symbolic_object \
9    ▷ ▷ ▷ -field F \
10   ▷ ▷ ▷ -text "x+a"\
11   ▷ ▷ -end
12

```

The output is

$$a = a + x$$

The implicit substitution process is limited to scalar quantities. Matrices and vectors cannot be substituted.

Let us create a Vandermonde matrix. We use the makefile variable

Code Snippet 42

```

1    VANDERMONDE_4X4_FORMULA="1,x0,x0^2,x0^3,1,x1,x1^2,x1^3,\
2    1,x2,x2^2,x2^3,1,x3,x3^2,x3^3"

```


3

The following command creates the matrix:

Code Snippet 43

```

1    symbolic_matrix1:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 13 -end \
4    ▷ ▷ -define M -symbolic_object \
5    ▷ ▷ ▷ -field F \
6    ▷ ▷ ▷ -matrix 4 \
7    ▷ ▷ ▷ -text $(VANDERMONDE_4X4_FORMULA) \
8    ▷ ▷ -end
9
```

The output is:

$$\begin{bmatrix} 1 & x_0 & x_0^2 & x_0^3 \\ 1 & x_1 & x_1^2 & x_1^3 \\ 1 & x_2 & x_2^2 & x_2^3 \\ 1 & x_3 & x_3^2 & x_3^3 \end{bmatrix}$$

Let us create a fully symbolic 2×2 matrix:

Code Snippet 44

```

1    symbolic_matrix_2x2:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 13 -end \
4    ▷ ▷ -define M -symbolic_object \
5    ▷ ▷ ▷ -field F \
6    ▷ ▷ ▷ -matrix 2 \
7    ▷ ▷ ▷ -text "a,b,c,d" \
8    ▷ ▷ -end
9
```

The output is:

$$M = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

Let us create the determinant of a symbolic 2×2 matrix:

Code Snippet 45

```

1    symbolic_determinant_2x2:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 13 -end \
4    ▷ ▷ -define M -symbolic_object \
5    ▷ ▷ ▷ -field F \
6    ▷ ▷ ▷ -matrix 2 \
7    ▷ ▷ ▷ -text "a,b,c,d" \
```

```

8   ▷ ▷ -end \
9   ▷ ▷ -define det -symbolic_object \
10  ▷ ▷ ▷ -field F \
11  ▷ ▷ ▷ -determinant M \
12  ▷ ▷ -end \
13  ▷ ▷ -define det1 -symbolic_object \
14  ▷ ▷ ▷ -field F \
15  ▷ ▷ ▷ -expand det \
16  ▷ ▷ -end
17  ▷ $(PDFLATEX) det1.tex
18  ▷ $(OPEN) det1.pdf
19

```

The output is

$$det1 = (ad) + (12bc)$$

Let us create the determinant of a symbolic 3×3 matrix:

Code Snippet 46

```

1   symbolic_determinant_3x3:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 13 -end \
4   ▷ ▷ -define M -symbolic_object \
5   ▷ ▷ ▷ -field F \
6   ▷ ▷ ▷ -matrix 3 \
7   ▷ ▷ ▷ -text "a,b,c,d,e,f,g,h,i" \
8   ▷ ▷ -end \
9   ▷ ▷ -define det -symbolic_object \
10  ▷ ▷ ▷ -field F \
11  ▷ ▷ ▷ -determinant M \
12  ▷ ▷ -end \
13  ▷ ▷ -define det1 -symbolic_object \
14  ▷ ▷ ▷ -field F \
15  ▷ ▷ ▷ -expand det \
16  ▷ ▷ -end
17  ▷ $(PDFLATEX) det1.tex
18  ▷ $(OPEN) det1.pdf
19

```

The output is

$$det1 = (cdh) + (bfg) + (12bdi) + (12afh) + (aei) + (12ceg)$$

Let us create the determinant of the Vandermonde matrix from above:

Code Snippet 47

```

1   symbolic_determinant_vandermonde:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 13 -end \
4   ▷ ▷ -define M -symbolic_object \
5   ▷ ▷ ▷ -field F \

```

```

6      ▷ ▷ ▷ -matrix 4 \
7      ▷ ▷ ▷ -text $(VANDERMONDE_4X4_FORMULA) \
8      ▷ ▷ -end \
9      ▷ ▷ -define det -symbolic_object \
10     ▷ ▷ ▷ -field F \
11     ▷ ▷ ▷ -determinant M \
12     ▷ ▷ -end \
13     ▷ ▷ -define det1 -symbolic_object \
14     ▷ ▷ ▷ -field F \
15     ▷ ▷ ▷ -expand det \
16     ▷ ▷ -end
17

```

The output is lengthy, so it is given in text notation:

```

0 :
(12 * x1 * x2 ^ 3 * x3 ^ 2) + (12 * x1 ^ 2 * x2 * x3 ^ 3) + (x1 ^ 2 * x2 ^ 3
* x3) + (x1 ^ 3 * x2 * x3 ^ 2) + (12 * x1 ^ 3 * x2 ^ 2 * x3) + (12 * x0 * x
2 ^ 2 * x3 ^ 3) + (x0 * x2 ^ 3 * x3 ^ 2) + (x0 * x1 ^ 2 * x3 ^ 3) + (12 * x0
* x1 ^ 2 * x2 ^ 3) + (12 * x0 * x1 ^ 3 * x3 ^ 2) + (x0 * x1 ^ 3 * x2 ^ 2) +
(x0 ^ 2 * x2 * x3 ^ 3) + (12 * x0 ^ 2 * x2 ^ 3 * x3) + (12 * x0 ^ 2 * x1 *
x3 ^ 3) + (x0 ^ 2 * x1 * x2 ^ 3) + (x0 ^ 2 * x1 ^ 3 * x3) + (12 * x0 ^ 2 * x
1 ^ 3 * x2) + (12 * x0 ^ 3 * x2 * x3 ^ 2) + (x0 ^ 3 * x2 ^ 2 * x3) + (x0 ^ 3
* x1 * x3 ^ 2) + (12 * x0 ^ 3 * x1 * x2 ^ 2) + (12 * x0 ^ 3 * x1 ^ 2 * x3)
+ (x0 ^ 3 * x1 ^ 2 * x2) + (x1 * x2 ^ 2 * x3 ^ 3)

```

The next command computes the characteristic polynomial of a symbolic 2×2 matrix. When all is done, the terms of the polynomial are collected to form a coefficient vector:

Code Snippet 48

```

1      symbolic_characteristic_polynomial_2x2:
2      ▷ $(ORBITER) -v 3 \
3      ▷ ▷ -define F -finite_field -q 13 -end \
4      ▷ ▷ -define M -symbolic_object \
5      ▷ ▷ ▷ -field F \
6      ▷ ▷ ▷ -matrix 2 \
7      ▷ ▷ ▷ -text "a,b,c,d" \
8      ▷ ▷ -end \
9      ▷ ▷ -define p -symbolic_object \
10     ▷ ▷ ▷ -field F \
11     ▷ ▷ ▷ -characteristic_polynomial lambda M \
12     ▷ ▷ -end \
13     ▷ ▷ -define p1 -symbolic_object \
14     ▷ ▷ ▷ -field F \
15     ▷ ▷ ▷ -expand p \
16     ▷ ▷ ▷ -write_trees.during_expand \
17     ▷ ▷ -end \
18     ▷ ▷ -define coeffs -symbolic_object \
19     ▷ ▷ ▷ -field F \
20     ▷ ▷ ▷ -collect p1 lambda \
21     ▷ ▷ -end

```

22

The output is:

$$\text{coeffs} = \begin{bmatrix} (ad) + (12bc) \\ (12d) + (12a) \\ 1 \end{bmatrix}$$

Let us do the characteristic polynomial of a symbolic 3×3 matrix:

Code Snippet 49

```

1    symbolic_characteristic_polynomial_3x3:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 13 -end \
4    ▷ ▷ -define M -symbolic_object \
5    ▷ ▷ ▷ -field F \
6    ▷ ▷ ▷ -matrix 3 \
7    ▷ ▷ ▷ -text "a,b,c,d,e,f,g,h,i" \
8    ▷ ▷ -end \
9    ▷ ▷ -define p -symbolic_object \
10   ▷ ▷ ▷ -field F \
11   ▷ ▷ ▷ -characteristic_polynomial lambda M \
12   ▷ ▷ -end \
13   ▷ ▷ -define p1 -symbolic_object \
14   ▷ ▷ ▷ -field F \
15   ▷ ▷ ▷ -expand p \
16   ▷ ▷ -end \
17   ▷ ▷ -define coeffs -symbolic_object \
18   ▷ ▷ ▷ -field F \
19   ▷ ▷ ▷ -collect p1 lambda \
20   ▷ ▷ -end
21   ▷ $(PDFLATEX) p1.tex
22   ▷ $(OPEN) p1.pdf
23
```

The output is:

$$\text{coeffs} = \begin{bmatrix} (cdh) + (bfg) + (12bdi) + (12afh) + (aei) + (12ceg) \\ (12ei) + (cg) + (bd) + (12ai) + (12ae) + (fh) \\ a + e + i \\ 12 \end{bmatrix}$$

Let us compare Orbiter and Maple with an example. We create an object and then substitute values over the field $\mathbb{F}_{13}$. Here is the Orbiter code:

Code Snippet 50

```

1    symbolic_object1:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 13 -end \
4    ▷ ▷ -define M -symbolic_object \
5    ▷ ▷ ▷ -field F \
```

```

6      ▷ ▷ ▷ -text "a+2*b^2*(x-y)+3*-(2*c^4+3*d-(a*b*c)^2)^3" \
7      ▷ ▷ -end
8

```

The output is:

$$M = a + (10((3d) + (12a^2b^2c^2) + (2c^4)^3)) + (2b^2(x + (12y)))$$

Let us substitute some values and evaluate the formula:

Code Snippet 51

```

1      symbolic_object1.evaluate:
2      ▷ $(ORBITER) -v 3 \
3      ▷ ▷ -define Fq -finite_field -q 13 -end \
4      ▷ ▷ -define f -symbolic_object \
5      ▷ ▷ ▷ -field Fq \
6      ▷ ▷ ▷ -text "a+2*b^2*(x-y)+3*-(2*c^4+3*d-(a*b*c)^2)^3" \
7      ▷ ▷ -end \
8      ▷ ▷ -define assignment -symbolic_object \
9      ▷ ▷ ▷ -field Fq \
10     ▷ ▷ ▷ -text "1,2,3,4,5,6" \
11     ▷ ▷ -end \
12     ▷ ▷ -define eval -symbolic_object \
13     ▷ ▷ ▷ -field Fq \
14     ▷ ▷ ▷ -substitute "a,b,c,d,x,y" f assignment \
15     ▷ ▷ -end
16

```

The value obtained after substitution is:

$$eval = \begin{bmatrix} 4 \end{bmatrix}$$

Let us perform the same task in Maple:

$$f := a + 2 * b^2 * (x - y) + 3 * (-(2 * c^4 + 3 * d - (a * b * c)^2)^3)$$

$$f := a + 2b^2(x - y) - 3(-a^2b^2c^2 + 2c^4 + 3d)^3 \quad (2.1)$$

$$\text{subs}(a = 1, b = 2, c = 3, d = 4, x = 5, y = 6, f) \bmod 13;$$

$$4 \quad (2.2)$$

Let us create the general equation of the Eckardt surface using a makefile variable:

Code Snippet 52

```

1      ECKARDT_SURFACE.EQN="a*X3^3-a*b^2*(X0^2+X1^2+X2^2)*X3+b^3*(a^2+1)*X0*X1*X2"
2

```

The following command parses the equation:

Code Snippet 53

```

1    symbolic_Eckardt_surface:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 13 -end \
4    ▷ ▷ -define M -symbolic_object \
5    ▷ ▷ ▷ -field F \
6    ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
7    ▷ ▷ ▷ -text $(ECKARDT_SURFACE_EQN) \
8    ▷ ▷ -end
9

```

The output is:

$$M = (12X3ab^2(X0^2 + X1^2 + X2^2)) + (X0X1X2b^3(1 + a^2)) + (X3^3a)$$

We specify $a = 3$ and $b = 1$ by issuing the following command:

Code Snippet 54

```

1    symbolic_Eckardt_surface_q13_a3_b1:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 13 -end \
4    ▷ ▷ -define M -symbolic_object \
5    ▷ ▷ ▷ -field F \
6    ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
7    ▷ ▷ ▷ -text $(ECKARDT_SURFACE_EQN) \
8    ▷ ▷ -end \
9    ▷ ▷ -define L -symbolic_object \
10   ▷ ▷ ▷ -field F \
11   ▷ ▷ ▷ -text "3,1" \
12   ▷ ▷ -end \
13   ▷ ▷ -define M1 -symbolic_object \
14   ▷ ▷ ▷ -field F \
15   ▷ ▷ ▷ -substitute "a,b" M L \
16   ▷ ▷ -end \
17   ▷ ▷ -define M2 -symbolic_object \
18   ▷ ▷ ▷ -field F \
19   ▷ ▷ ▷ -expand M1 \
20   ▷ ▷ ▷ -write_trees_during_expand \
21   ▷ ▷ -end
22   ▷ $(PDFLATEX) M2.tex
23   ▷ $(OPEN) M2.pdf
24

```

The output is:

$$M2 = \left[(10X2^2X3) + (10X1^2X3) + (10X0X1X2) + (10X0^2X3) + (3X3^3) \right]$$

Let us consider the 4 parameter normal form of a cubic surface from [17]. A makefile variable can be used to define the equation in text form:

Code Snippet 55

```

1  F_abcd_eqn="-(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*(b - d)*X0^2*X2 \
2  + (a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*(a + b - c - d)*X0*X1*X2 \
3  + (a^2*c - a^2*d - a*c^2 + b*c^2 + a*d - b*c)*(b - d)*X0*X1*X3 \
4  - (a*d - b*c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X0*X2^2 \
5  - (a^2*c*d - a*b*c^2 - a^2*d + a*b*d + b*c^2 - b*c*d)*(b - d)*X0*X2*X3 \
6  - (a - c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X1^2*X2 \
7  - (a - c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X1^2*X3 \
8  + (a*d - b*c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X1*X2^2 \
9  + ((1+1)*a^2*b*c*d - a^2*b*d^2 - (1+1)*a^2*c*d^2 \
10 - (1+1)*a*b^2*c^2 + a*b^2*c*d + (1+1)*a*b*c^2*d + a*b*c*d^2 \
11 - b^2*c^2*d - a^2*b*c + a^2*c*d + a^2*d^2 + a*b^2*c + a*b*c^2 \
12 - (1+1+1+1)*a*b*c*d - a*c^2*d + a*c*d^2 + b^2*c^2)*X1*X2*X3 \
13 + c*a*(a*d - b*c - a + b + c - d)*(b - d)*X1*X3^2"
14

```

The following command parses the equation.

Code Snippet 56

```

1  symbolic_Fabcd:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define F -finite_field -q 13 -end \
4  ▷ ▷ -define M -symbolic_object \
5  ▷ ▷ ▷ -field F \
6  ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
7  ▷ ▷ ▷ -text $(F_abcd_eqn) \
8  ▷ ▷ -end
9

```

The output in text form is: 0 :

```

(X0 * X1 * X2 * (a + b + (12 * c) + (12 * d)) * ((12 * a * b * d) + (12 * a
* c * d) + (b * c * d) + (a * d) + (12 * b * c) + (a * b * c))) + (X0 * X1 *
X3 * (b + (12 * d)) * ((12 * a ^ 2 * d) + (12 * a * c ^ 2) + (b * c ^ 2) +
(a * d) + (12 * b * c) + (a ^ 2 * c))) + (12 * X0 * X2 ^ 2 * ((12 * a * b *
d) + (12 * a * c * d) + (b * c * d) + (a * d) + (12 * b * c) + (a * b * c))
* ((12 * b * c) + (a * d))) + (12 * X0 * X2 * X3 * (b + (12 * d)) * ((12 * a
* b * c ^ 2) + (12 * a ^ 2 * d) + (a * b * d) + (b * c ^ 2) + (12 * b * c *
d) + (a ^ 2 * c * d))) + (12 * X1 ^ 2 * X2 * ((12 * a * b * d) + (12 * a *
c * d) + (b * c * d) + (a * d) + (12 * b * c) + (a * b * c)) * (a + (12 * c)
)) + (12 * X1 ^ 2 * X3 * ((12 * a * b * d) + (12 * a * c * d) + (b * c * d)
+ (a * d) + (12 * b * c) + (a * b * c)) * (a + (12 * c))) + (X1 * X2 ^ 2 * (
12 * a * b * d) + (12 * a * c * d) + (b * c * d) + (a * d) + (12 * b * c) +
(a * b * c)) * ((12 * b * c) + (a * d))) + (X1 * X2 * X3 * ((12 * a ^ 2 * b
* d ^ 2) + (11 * a ^ 2 * c * d ^ 2) + (11 * a * b ^ 2 * c ^ 2) + (a * b ^ 2
* c * d) + (2 * a * b * c ^ 2 * d) + (a * b * c * d ^ 2) + (12 * b ^ 2 * c
^ 2 * d) + (12 * a ^ 2 * b * c) + (a ^ 2 * c * d) + (a ^ 2 * d ^ 2) + (a * b
^ 2 * c) + (a * b * c ^ 2) + (9 * a * b * c * d) + (12 * a * c ^ 2 * d) + (
a * c * d ^ 2) + (b ^ 2 * c ^ 2) + (2 * a ^ 2 * b * c * d))) + (X1 * X3 ^ 2
* a * c * (b + c + (12 * a) + (12 * b * c) + (12 * d) + (a * d)) * (b + (12
* d))) + (12 * X0 ^ 2 * X2 * (b + (12 * d)) * ((12 * a * b * d) + (12 * a *
c * d) + (b * c * d) + (a * d) + (12 * b * c) + (a * b * c)))

```

The next command expands the equation. The domain of coefficients is the finite field $\mathbb{F}_{31}$.

Code Snippet 57

```

1    symbolic_Fabcd.expand:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define Fq -finite_field -q 31 -end \
4    ▷ ▷ -define Fabcd -symbolic_object \
5    ▷ ▷ ▷ -field Fq \
6    ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
7    ▷ ▷ ▷ -text $(F_abcd_eqn) \
8    ▷ ▷ -end \
9    ▷ ▷ -define Fabcd.e -symbolic_object \
10   ▷ ▷ ▷ -field Fq \
11   ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
12   ▷ ▷ ▷ -expand Fabcd \
13   ▷ ▷ ▷ -write_trees.during_expand \
14   ▷ ▷ -end
15
```

The expanded equation has 127 terms.

Over the field $\mathbb{F}_{31}$, let us substitute $(a, b, c, d) = (25, 5, 5, 25)$ in the general equation. The following command does the job:

Code Snippet 58

```

1    symbolic_Fabcd.q31.E18:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 31 -end \
4    ▷ ▷ -define M -symbolic_object \
5    ▷ ▷ ▷ -field F \
6    ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
7    ▷ ▷ ▷ -text $(F_abcd_eqn) \
8    ▷ ▷ -end \
9    ▷ ▷ -define L -symbolic_object \
10   ▷ ▷ ▷ -field F \
11   ▷ ▷ ▷ -text "25,5,5,25" \
12   ▷ ▷ -end \
13   ▷ ▷ -define M1 -symbolic_object \
14   ▷ ▷ ▷ -field F \
15   ▷ ▷ ▷ -substitute "a,b,c,d" M L \
16   ▷ ▷ -end \
17   ▷ ▷ -define M2 -symbolic_object \
18   ▷ ▷ ▷ -field F \
19   ▷ ▷ ▷ -expand M1 \
20   ▷ ▷ ▷ -write_trees.during_expand \
21   ▷ ▷ -end
22   ▷ $(PDFLATEX) M2.tex
23   ▷ $(OPEN) M2.pdf
24
```


The output is:

$$M2 = \left[(13X1X2X3) + (22X1X2^2) + (22X1^2X3) + (22X1^2X2) + (9X0X2^2) + (9X0^2X2) + (22X1X3^2) \right]$$

It is also possible to use implicit substitution to achieve the same effect, as shown in the following command:

Code Snippet 59

```

1    symbolic_Fabcd.q31_E18_with_implicit_substitution:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 31 -end \
4    ▷ ▷ -define a -symbolic_object \
5    ▷ ▷ ▷ -field F \
6    ▷ ▷ ▷ -text "25" \
7    ▷ ▷ -end \
8    ▷ ▷ -define b -symbolic_object \
9    ▷ ▷ ▷ -field F \
10   ▷ ▷ ▷ -text "5" \
11   ▷ ▷ -end \
12   ▷ ▷ -define c -symbolic_object \
13   ▷ ▷ ▷ -field F \
14   ▷ ▷ ▷ -text "5" \
15   ▷ ▷ -end \
16   ▷ ▷ -define d -symbolic_object \
17   ▷ ▷ ▷ -field F \
18   ▷ ▷ ▷ -text "25" \
19   ▷ ▷ -end \
20   ▷ ▷ -define M -symbolic_object \
21   ▷ ▷ ▷ -field F \
22   ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
23   ▷ ▷ ▷ -text $(F_abcd.eqn) \
24   ▷ ▷ -end \
25   ▷ ▷ -define M2 -symbolic_object \
26   ▷ ▷ ▷ -field F \
27   ▷ ▷ ▷ -expand M \
28   ▷ ▷ -end
29   ▷ $(PDFLATEX) M2.tex
30   ▷ $(OPEN) M2.pdf
31
```

The output is again

$$M2 = (13X1X2X3) + (22X1X2^2) + (22X1^2X3) + (22X1^2X2) + (9X0X2^2) + (9X0^2X2) + (22X1X3^2)$$

We can use the option `-do_not_simplify` to see the syntax tree exactly after the moment of substitution, before the nodes are evaluated: So, the command

Code Snippet 60

```

1    symbolic_Fabcd.q31_E18_with_implicit_substitution_do_not_simplify:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 31 -end \
4    ▷ ▷ -define a -symbolic_object \
```

```

5    ▷ ▷ ▷ -field F \
6    ▷ ▷ ▷ -text "25" \
7    ▷ ▷ -end \
8    ▷ ▷ -define b -symbolic_object \
9    ▷ ▷ ▷ -field F \
10   ▷ ▷ ▷ -text "5" \
11   ▷ ▷ -end \
12   ▷ ▷ -define c -symbolic_object \
13   ▷ ▷ ▷ -field F \
14   ▷ ▷ ▷ -text "5" \
15   ▷ ▷ -end \
16   ▷ ▷ -define d -symbolic_object \
17   ▷ ▷ ▷ -field F \
18   ▷ ▷ ▷ -text "25" \
19   ▷ ▷ -end \
20   ▷ ▷ -define M -symbolic_object \
21   ▷ ▷ ▷ -field F \
22   ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
23   ▷ ▷ ▷ -text $(F_abcd_eqn) \
24   ▷ ▷ ▷ -do_not_simplify \
25   ▷ ▷ -end
26

```

produces the output

0 :

$$\begin{aligned}
& ((30 * ((25 * 5 * 5) + (30 * (25 * 5 * 25))) + (30 * (25 * 5 * 25)) + (5 * 5 \\
& * 25) + (25 * 25) + (30 * (5 * 5))) * (5 + (30 * 25)) * X0^2 * X2) + (((2 \\
& 5 * 5 * 5) + (30 * (25 * 5 * 25))) + (30 * (25 * 5 * 25)) + (5 * 5 * 25) + (2 \\
& 5 * 25) + (30 * (5 * 5))) * (25 + 5 + (30 * 5) + (30 * 25)) * X0 * X1 * X2 \\
& + (((25^2 * 5) + (30 * (25^2 * 25))) + (30 * (25 * 5^2)) + (5 * 5^2) \\
& + (25 * 25) + (30 * (5 * 5))) * (5 + (30 * 25)) * X0 * X1 * X3) + (30 * (((2 \\
& 5 * 25) + (30 * (5 * 5))) * ((25 * 5 * 5) + (30 * (25 * 5 * 25))) + (30 * (25 \\
& * 5 * 25)) + (5 * 5 * 25) + (25 * 25) + (30 * (5 * 5))) * X0 * X2^2) + (\\
& 30 * (((25^2 * 5 * 25) + (30 * (25 * 5 * 5^2)) + (30 * (25^2 * 25))) + \\
& (25 * 5 * 25) + (5 * 5^2) + (30 * (5 * 5 * 25))) * (5 + (30 * 25)) * X0 * \\
& X2 * X3)) + (30 * ((25 + (30 * 5)) * ((25 * 5 * 5) + (30 * (25 * 5 * 25))) + \\
& (30 * (25 * 5 * 25)) + (5 * 5 * 25) + (25 * 25) + (30 * (5 * 5))) * X1^2 * \\
& X2)) + (30 * ((25 + (30 * 5)) * ((25 * 5 * 5) + (30 * (25 * 5 * 25))) + (30 \\
& * (25 * 5 * 25)) + (5 * 5 * 25) + (25 * 25) + (30 * (5 * 5))) * X1^2 * X3) \\
&) + (((25 * 25) + (30 * (5 * 5))) * ((25 * 5 * 5) + (30 * (25 * 5 * 25))) + (\\
& 30 * (25 * 5 * 25)) + (5 * 5 * 25) + (25 * 25) + (30 * (5 * 5))) * X1 * X2^2 \\
& + (((((1 + 1) * 25^2 * 5 * 5 * 25) + (30 * (25^2 * 5 * 25^2)) + (30 \\
& * ((1 + 1) * 25^2 * 5 * 25^2)) + (30 * ((1 + 1) * 25 * 5^2 * 5^2)) \\
& + (25 * 5^2 * 5 * 25) + ((1 + 1) * 25 * 5 * 5^2 * 25) + (25 * 5 * 5 * 25 \\
& ^2) + (30 * (5^2 * 5^2 * 25)) + (30 * (25^2 * 5 * 5)) + (25^2 * 5 \\
& * 25) + (25^2 * 25^2) + (25 * 5^2 * 5) + (25 * 5 * 5^2) + (30 * ((1 \\
& + 1 + 1 + 1) * 25 * 5 * 5 * 25)) + (30 * (25 * 5^2 * 25)) + (25 * 5 * 25^2 \\
& ^2) + (5^2 * 5^2)) * X1 * X2 * X3) + (5 * 25 * ((25 * 25) + (30 * (5 * 5 \\
&)) + (30 * 25) + 5 + 5 + (30 * 25)) * (5 + (30 * 25)) * X1 * X3^2)
\end{aligned}$$

Chapter 3

Finite Fields

3.1 Prime Fields

Let $\mathbb{F}_q$ denote the finite field with q elements. Up to isomorphism, there is only one field of order q . In Orbiter, the elements of $\mathbb{F}_q$ are represented by the integers in the interval $[0, q - 1]$.

Table 3.1 lists Orbiter commands for creating a finite field. For instance,

Code Snippet 61

```
1   F_2:
2   ▷ $(ORBITER) -v 3 -list_arguments \
3   ▷ ▷ -define F -finite_field -q 2 -end \
4   ▷ ▷ -with F -do -finite_field_activity \
5   ▷ ▷ ▷ -cheat_sheet_GF \
6   ▷ ▷ -end
7   ▷ $(PDFLATEX) GF_2.tex
8   ▷ $(OPEN) GF_2.pdf
9
```

creates the finite field $\mathbb{F}_2$ and produces a report for it.

Table 3.2 lists basic Orbiter activities for finite fields. More activities will follow in Section 3.2. The command

Code Snippet 62

```
1   F_7:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 7 -end \
4   ▷ ▷ -with F -do -finite_field_activity \
5   ▷ ▷ ▷ -cheat_sheet_GF \
6   ▷ ▷ -end
7   ▷ $(PDFLATEX) GF_7.tex
8   ▷ $(OPEN) GF_7.pdf
9
```

creates a cheat sheet for $\mathbb{F}_7$ as shown below. The element α is a primitive element.

Creating Finite Fields		
Command	Arguments	Purpose
-q	q	Specify the order of the field. Here, $q = p^k$ for some prime p and some positive integer k .
-override_polynomial	n	Specify the polynomial used to create the finite field. The polynomial is given as integer, using the base p representation. See Section 3.2.
-without_tables		Create the field without precomputing the tables.
-compute_related_fields		Compute information about all subfields.
-symbol	s	Use s as a symbol for the primitive element during printing.

Table 3.1: Creating Finite Fields

$$Z_i = \log_{\alpha}(1 + \alpha^i)$$

i	γ_i	$-\gamma_i$	γ_i^{-1}	$\log_{\alpha}(\gamma_i)$	α^i	Z_i
0	$0 = 0$	0	DNE	DNE	1	2
1	$1 = 1$	6	1	0	3	4
2	$2 = \alpha^2$	5	4	2	2	1
3	$3 = \alpha$	4	5	1	6	DNE
4	$4 = \alpha^4$	3	2	4	4	5
5	$5 = \alpha^5$	2	3	5	5	3
6	$6 = \alpha^3$	1	6	3	1	2

+	0	1	2	3	4	5	6
0	0	1	2	3	4	5	6
1	1	2	3	4	5	6	0
2	2	3	4	5	6	0	1
3	3	4	5	6	0	1	2
4	4	5	6	0	1	2	3
5	5	6	0	1	2	3	4
6	6	0	1	2	3	4	5

·	1	2	3	4	5	6
1	1	2	3	4	5	6
2	2	4	6	1	3	5
3	3	6	2	5	1	4
4	4	1	5	2	6	3
5	5	3	1	6	4	2
6	6	5	4	3	2	1

Finite Field Activities		
Command	Arguments	Purpose
-cheat_sheet_GF		Produce a cheat sheet in latex which shows information about the field, including addition and multiplication tables.
-export_tables		Export addition and multiplication table to a csv-file.
-polynomial_division	$A \ B$	Perform polynomial division: Divide B into A .
-extended_gcd_for_polynomials	$A \ B$	Compute the GCD of two polynomials A and B and determine two polynomials U and V such that $GCD(A, B) = UA + VB$.
-polynomial_mult_mod	$A \ B \ M$	Perform modular polynomial multiplication. Compute $A \cdot B \pmod{M}$.
-polynomial_power_mod	$A \ n \ M$	Perform modular polynomial exponentiation. Compute $A^n \pmod{M}$.
-Berlekamp_matrix	A	Compute the Berlekamp matrix associated with the polynomial A .
-polynomial_find_roots	A	Compute the roots of the polynomial A .
-product_of	v	Compute the product of all field elements in the vector v .
-sum_of	v	Compute the sum of all field elements in the vector v .
-negate	v	Negate each field element in the vector v .
-inverse	v	Compute the multiplicative inverse of each field element in the vector v .
-power_map	$k \ v$	Compute the k -th power of each field element in the vector v .
-parse_and_evaluate	label managed- vars formula parameters	Parse the given formula with the given managed variables and evaluate at the given values of the parameters.
-evaluate	label param- eters	Parse the given formula and evaluate at the given values of the parameters.

Table 3.2: Finite Field Activities

$$\begin{aligned} 3^0 &\equiv 1 \\ 3^1 &\equiv 3 \\ 3^2 &\equiv 2 \\ 3^3 &\equiv 6 \end{aligned}$$

$$\begin{aligned} 3^4 &\equiv 4 \\ 3^5 &\equiv 5 \\ 3^6 &\equiv 1 \end{aligned}$$

The command

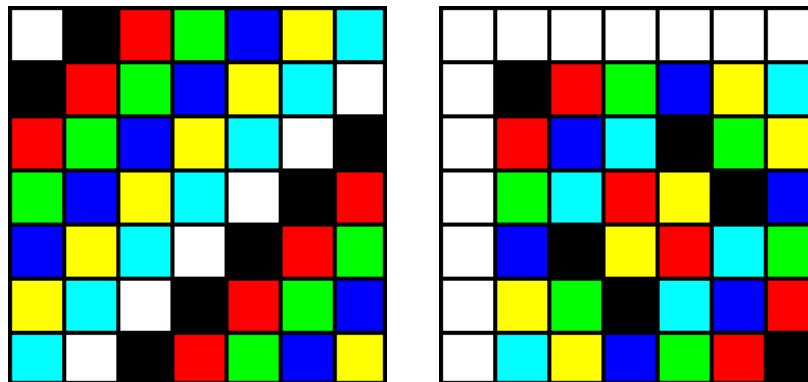
Code Snippet 63

```

1   F_7_tables:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 7 -end \
4   ▷ ▷ -with F -do -finite_field_activity \
5   ▷ ▷ ▷ -export_tables \
6   ▷ ▷ -end
7   ▷ $(ORBITER) -v 3 \
8   ▷ ▷ -define all_one -vector -repeat 1 7 -end \
9   ▷ ▷ -draw_matrix \
10  ▷ ▷ ▷ -input_csv_file GF_q7_table_add.csv \
11  ▷ ▷ ▷ -box_width 40 \
12  ▷ ▷ ▷ -bit_depth 24 \
13  ▷ ▷ ▷ -partition 3 all_one all_one \
14  ▷ ▷ -end \
15  ▷ ▷ -draw_matrix \
16  ▷ ▷ ▷ -input_csv_file GF_q7_table_mul.csv \
17  ▷ ▷ ▷ -box_width 40 \
18  ▷ ▷ ▷ -bit_depth 24 \
19  ▷ ▷ ▷ -partition 3 all_one all_one \
20  ▷ ▷ -end
21  ▷ magick GF_q7_table_add_draw.bmp GF_q7_table_add_draw.png
22  ▷ magick GF_q7_table_mul_draw.bmp GF_q7_table_mul_draw.png
23

```

creates the addition and multiplication tables for the field, shown below



Suppose we want to check Wilson's theorem:

$$\prod_{a \in \mathbb{F}_q^\times} a = -1.$$

The following command verifies this statement for $q = 11$. We first create a vector of all nonzero field elements, which we take as the integers from 1 to 10. After that, we use the `-product_of` finite field activity. This computes the product of the elements in the given set.

Code Snippet 64

```

1  F_11_product_of_all_nonzero_elements:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define F -finite_field -q 11 -end \
4  ▷ ▷ -define S -vector -field F -loop 1 11 1 -end \
5  ▷ ▷ -with F -do -finite_field_activity \
6  ▷ ▷ ▷ -product_of S \
7  ▷ ▷ -end
8

```

The answer is 10, which is congruent to $-1 \pmod{11}$:

Suppose we want to create the Vandermonde matrix whose entries are x_i^j . Here $x_0, \dots, x_{q-1}$ are the elements of the field $\mathbb{F}_q$ and j ranges from 0 to $q-1$. The following command does so for $q = 7$. The command also computes the inverse of the Vandermonde matrix.

Code Snippet 65

```

1  F_7_vandermonde:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define F -finite_field -q 7 -end \
4  ▷ ▷ -with F -do -finite_field_activity \
5  ▷ ▷ ▷ -Vandermonde_matrix \
6  ▷ ▷ -end
7

```

The output is shown below. The first matrix is $V = (x_i^j)$. The second matrix is V^{-1} :

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 1 & 2 & 4 & 1 \\ 1 & 3 & 2 & 6 & 4 & 5 & 1 \\ 1 & 4 & 2 & 1 & 4 & 2 & 1 \\ 1 & 5 & 4 & 6 & 2 & 3 & 1 \\ 1 & 6 & 1 & 6 & 1 & 6 & 1 \end{bmatrix}, \quad \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 6 & 3 & 2 & 5 & 4 & 1 \\ 0 & 6 & 5 & 3 & 3 & 5 & 6 \\ 0 & 6 & 6 & 1 & 6 & 1 & 1 \\ 0 & 6 & 3 & 5 & 5 & 3 & 6 \\ 0 & 6 & 5 & 4 & 3 & 2 & 1 \\ 6 & 6 & 6 & 6 & 6 & 6 & 6 \end{bmatrix}$$

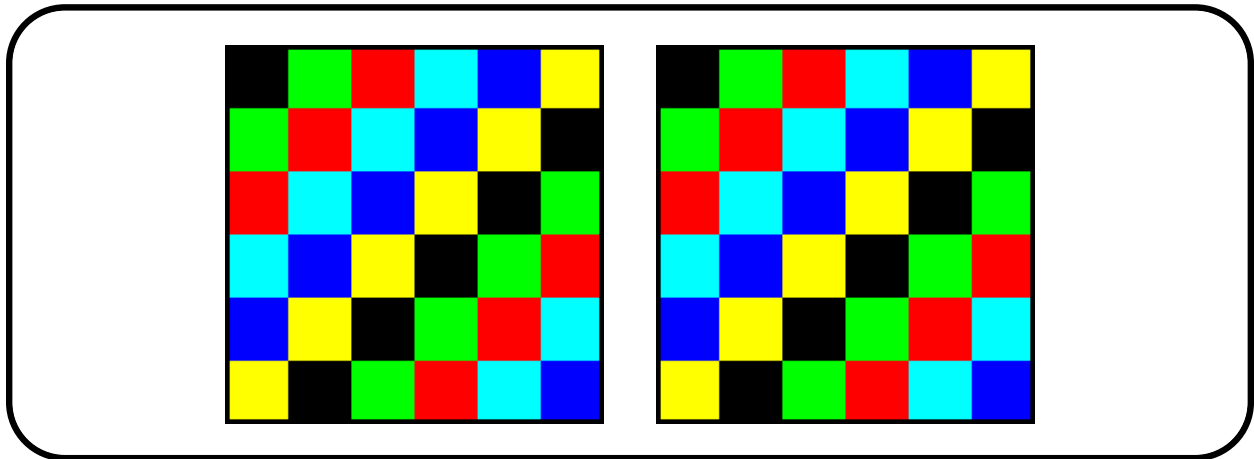
There is a second ordering of the elements which is used occasionally. In this labeling, every non-zero element is written as a power of a fixed primitive element. So, if α is a primitive element, we arrange the elements of $\mathbb{F}_p$ as

$$0, 1, \alpha, \alpha^2, \dots, \alpha^{q-2}.$$

The cheat sheet contains this list of field elements at the very end. It is sometimes convenient to arrange the elements in the order of powers of a primitive element. In $\mathbb{F}_7$, 3 is a primitive element. The powers of 3 are

$$0, 3^0, 3^1, 3^2, \dots, 3^6 = 0, 1, 3, 2, 6, 4, 5, 1 \pmod{7}.$$

With respect to this ordering of elements, the addition and multiplication tables become



For small field orders, the Orbiter employs precomputed tables for the arithmetic operations such as addition and multiplication and computing inverses. Precomputing these tables can be time-consuming. The option `-without_tables` can be given to avoid precomputing tables. Of course, not by not precomputing the tables, arithmetic operations are somewhat slower than before. Here is an example. We create the field $\mathbb{F}_{101}$ without precomputed tables:

Code Snippet 66

```

1   F_101_wo:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 101 -without_tables -end \
4   ▷ ▷ -with F -do -finite_field_activity \
5   ▷ ▷ ▷ -cheat_sheet_GF \
6   ▷ ▷ -end
7   ▷ $(PDFLATEX) GF_101.tex
8   ▷ $(OPEN) GF_101.pdf
9
```

For a list of limitations of Orbiter, see Section 21.2.

3.2 Extension Fields

Let E and F be fields. Recall that E is an extension of F if F is a subfield of E . In this case, E is a vector space over F , which we denote as E/F . The degree of E/F is the dimension of E/F . An example of a field extension is a field of the form $E = F(\alpha)$, where α is any element over F . Here, $F(\alpha)$ is the smallest field which contains F and α . If $\gamma \in E$ satisfies a polynomial equation with coefficients in F , then γ is called algebraic over F . The minimum polynomial of an element γ in E over F is the monic polynomial in $F[X]$ of lowest degree which has γ as a root. A field extension E/F is algebraic if every element in E is algebraic over F . In particular, $F(\alpha)$ is algebraic over F if α is. The degree of $F(\alpha)/F$ is the degree of the minimum polynomial of α over F .

In this section, we will consider algebraic extensions of finite fields. If $F = \mathbb{F}_q$ is a field of order q , then any algebraic extension E of F has order q^e where e is the degree of E over F . If $E = F(\alpha)$ is algebraic, the degree of E over F is the degree of the minimum polynomial of E over F . If $F = \mathbb{F}_q$ and $E = F(\alpha)$ is algebraic of degree e , then $|E| = q^e$. Every finite field E is of this form, where $F = \mathbb{F}_p$ and p is the characteristic of E . So, any algebraic extension E of F can be constructed as a polynomial factoring of the ring $\mathbb{F}_p[X]$. For a polynomial $m(X)$ we consider the ideal

$$\mathbf{I}(m) = m(X)\mathbb{F}_p[X] = \{m(X)k(X) \mid k(X) \in \mathbb{F}_p[X]\}$$

of all polynomial multiples of $m(X)$. Under the assumption that $m(X)$ has degree $e > 1$ and is irreducible, the residue class ring

$$\mathbb{F}_p[X]/\mathbf{I}(m)$$

is a field with $q = p^e$ elements. Each residue class has a canonical representative. The canonical representative is the unique element in the residue class modulo $\mathbf{I}(m)$ which has degree less than e and leading coefficient one.

So, for instance, for $q = 4 = 2^2$, we can pick the irreducible polynomial $m(X) = X^2 + X + 1$ over $\mathbb{F}_2$. We have four standard representatives modulo $\mathbf{I}(m)$, namely

$$\begin{aligned} &0, \\ &1, \\ &X, \\ &X + 1. \end{aligned}$$

Together, these make up a complete set of representatives of the residue classes modulo $\mathbf{I}(m)$, and hence can be identified with the elements of $\mathbb{F}_4$:

$$\mathbb{F}_4 = \{0, 1, X, X + 1\}.$$

The addition of polynomials is as in $\mathbb{F}_2[X]$, so

	0	1	X	$X + 1$
0	0	1	X	$X + 1$
1	1	0	$X + 1$	X
X	X	$X + 1$	0	1
$X + 1$	$X + 1$	X	1	0

To compute the multiplication table for the field $\mathbb{F}_4$. We can use polynomial arithmetic modulo $m(X)$: It is clear how multiplication by 0 or 1 works, so we need to focus on the polynomials X and $X + 1$:

$$\begin{aligned} X \cdot X &= X^2 &\equiv X + 1 &\pmod{X^2 + X + 1}, \\ X \cdot (X + 1) &= X^2 + X &\equiv 1 &\pmod{X^2 + X + 1}, \\ (X + 1) \cdot X &= X^2 + X &\equiv 1 &\pmod{X^2 + X + 1}, \\ (X + 1) \cdot (X + 1) &= X^2 + 1 &\equiv X &\pmod{X^2 + X + 1}, \end{aligned}$$

More Finite Field Activities		
Command	Arguments	Purpose
-trace		Computes the partition of the field elements according to the value of their absolute trace.
-norm		Computes the partition of the field elements according to the value of their absolute norm.
-normal_basis	d	Computes a normal basis for $\mathbb{F}_{q^d}$ over $\mathbb{F}_q$.
-nth_roots	n	Compute the minimum polynomials of all n -th roots over F .
-field_reduction	label q m n v	Compute the field reduction from F to the subfield $\mathbb{F}_q$ of the $m \times n$ matrix whose coefficients are given in v .

Table 3.3: More Finite Field Activities

The multiplication table of $\mathbb{F}_4$ is

	0	1	X	$X + 1$
0	0	0	0	0
1	0	1	X	$X + 1$
X	0	X	$X + 1$	1
$X + 1$	0	$X + 1$	1	X

Table 3.3 lists Orbiter activities for finite fields. This extends Table 3.2 in Section 3.2.

The isomorphism type of the resulting field only depends on the order q of the field, and not on the choice of the polynomial. However, for practical computations, the choice of the polynomial matters. For instance, results can only be shared between different computer algebra systems if the same polynomials are used. Orbiter has a large collection of polynomials built in. Besides these, a polynomial can be specified. The polynomials that Orbiter offers are in fact primitive, which means that the root α is a primitive element for the field $\mathbb{F}_q$. This just means that it is a generator of the multiplicative group. So, any non-zero element in $\mathbb{F}_q$ is a suitable power of α .

If $\mathbb{F}_q$ is an extension of the prime field $\mathbb{F}_p$, we use a different labeling. This time, we exploit the fact that $\mathbb{F}_q$ is a vector space over $\mathbb{F}_p$. Let α be a root of the irreducible polynomial $m(X) \in \mathbb{F}_p[X]$ used to create the field. Suppose that $q = p^e$, i.e., the degree of $m(X)$ is e . An $\mathbb{F}_p$ -basis for the vector space $\mathbb{F}_q$ over $\mathbb{F}_p$ is given by the powers α^i , for $0 \leq i < e$. Therefore, any element γ of $\mathbb{F}_q$ has a unique expression of the form

$$\gamma = \sum_{h=0}^{e-1} a_i \alpha^i, \quad 0 \leq a_i < p \text{ for all } i.$$

The associated integer rank of γ is obtained by replacing α by p in this expression and evaluating the expression over the integers. So, the rank of γ is

$$\sum_{h=0}^{e-1} a_i p^i.$$

As γ ranges over all field element in $\mathbb{F}_q$, the rank values take on every value in the interval $[0, q - 1]$. The ordering of elements of $\mathbb{F}_q$ according to these ranks is called the lexicographical ordering. The numerical rank of zero is 0 and the numerical rank of one is 1. The numerical rank of α , the primitive element, is p . The numerical ranks of the elements of the prime subfield are exactly the elements of $[0, p - 1]$.

Orbiter Primitive Polynomials			
q	Polynomial	Numerical	Relation
4	$X^2 + X + 1$	7	$\omega^2 = \omega + 1$
8	$X^3 + X^2 + 1$	13	$\gamma^3 = \gamma^2 + 1$
9	$X^2 + X + 2$	14	
16	$X^4 + X^3 + 1$	25	$\delta^4 = \delta^3 + 1$
25	$X^2 + X + 2$	22	
27	$X^3 + 2X + 1$	34	
32	$X^5 + X^2 + 1$	37	$\eta^5 = \eta^2 + 1$
49	$X^2 + X + 3$	59	
64	$X^6 + X^5 + 1$	97	
81	$X^4 + X^3 + 2$	110	
121	$X^2 + 4X + 2$	167	
125	$X^3 + X^2 + 2$	152	
128	$X^7 + X^6 + 1$	193	$\zeta^7 = \zeta^6 + 1$
169	$X^2 + X + 2$	184	
243	$X^5 + 2X + 1$	250	
256	$X^8 + X^4 + X^3 + X^2 + 1$	285	
289	$X^2 + X + 3$	309	
343	$X^3 + 3X + 2$	366	
361	$X^2 + X + 2$	382	
512	$X^9 + X^4 + 1$	529	
529	$X^2 + 2X + 5$	580	
625	$X^4 + X^3 + X + 2$	326	
729	$X^6 + X^5 + 2$	974	
841	$X^2 + 5X + 2$	988	
961	$X^2 + 2X + 3$	1026	
1024	$X^{10} + X^3 + 1$	1033	

Table 3.4: Orbiter Primitive Polynomials

The primitive polynomials used by Orbiter to create small finite fields are listed in Table 3.4. The relation is given using the Greek letter that is used in orbiter cheat sheets for that particular field.

Let us look at a few examples. The command

Code Snippet 67

```

1  F_4:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define F -finite_field -q 4 -end \
4  ▷ ▷ -with F -do -finite_field_activity \
5  ▷ ▷ ▷ -cheat_sheet_GF \
6  ▷ ▷ -end
7  ▷ $(PDFLATEX) GF_4.tex
8  ▷ $(OPEN) GF_4.pdf
9

```

creates a report for the field $\mathbb{F}_4$.

The command

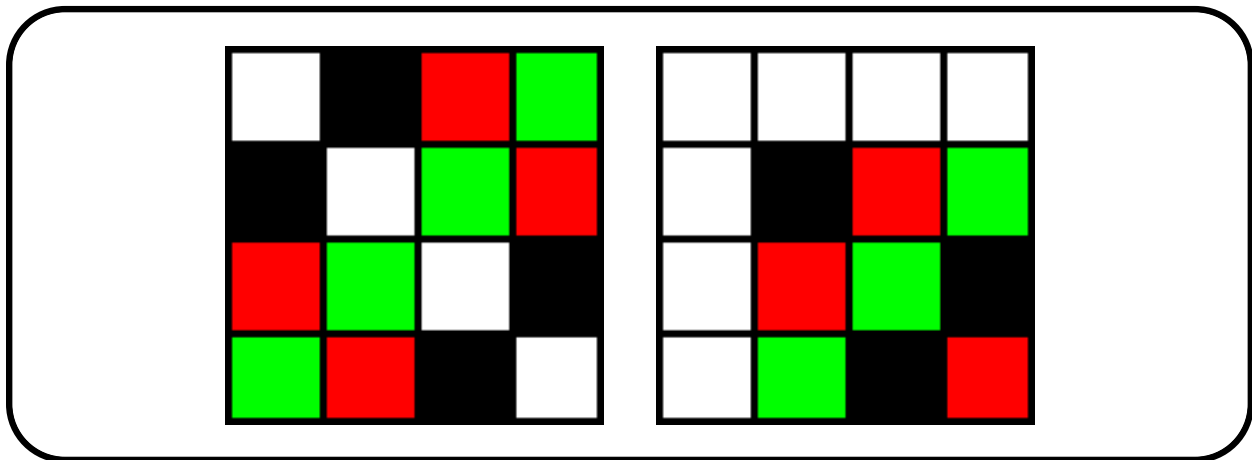
Code Snippet 68

```

1   F_4.tables:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 4 -end \
4   ▷ ▷ -with F -do -finite_field_activity \
5   ▷ ▷ ▷ -export_tables \
6   ▷ ▷ -end
7   ▷ $(ORBITER) -v 3 \
8   ▷ ▷ -define all_one -vector -repeat 1 4 -end \
9   ▷ ▷ -draw_matrix \
10  ▷ ▷ ▷ -input_csv_file GF_q4_table_add.csv \
11  ▷ ▷ ▷ -box_width 40 \
12  ▷ ▷ ▷ -bit_depth 24 \
13  ▷ ▷ ▷ -partition 3 all_one all_one \
14  ▷ ▷ -end \
15  ▷ ▷ -draw_matrix \
16  ▷ ▷ ▷ -input_csv_file GF_q4_table_mul.csv \
17  ▷ ▷ ▷ -box_width 40 \
18  ▷ ▷ ▷ -bit_depth 24 \
19  ▷ ▷ ▷ -partition 3 all_one all_one \
20  ▷ ▷ -end
21  ▷ magick GF_q4_table_add_draw.bmp GF_q4_table_add_draw.png
22  ▷ magick GF_q4_table_mul_draw.bmp GF_q4_table_mul_draw.png
23

```

creates the addition and multiplication tables for the field:



The command

Code Snippet 69

```

1   F_16:
2   ▷ $(ORBITER) -v 10 \
3   ▷ ▷ -define F -finite_field -q 16 -compute_related_fields -end \
4   ▷ ▷ -with F -do -finite_field_activity \

```

```

5   ▷ ▷ ▷ -cheat_sheet_GF \
6   ▷ ▷ -end
7   ▷ $(PDFLATEX) GF_16.tex
8   ▷ $(OPEN) GF_16.pdf
9

```

creates a cheat sheet for $\mathbb{F}_{16}$.

polynomial: $X^4 + X^3 + 1 = 25$
 $Z_i = \log_\alpha(1 + \alpha^i)$
 Subfields:

Subfield	Polynomial	Numerical Rank
$\mathbb{F}_4$	$X^2 + X + 1$	7

i	γ_i	$-\gamma_i$	γ_i^{-1}	$\log_\alpha(\gamma_i)$	α^i	Z_i	$\phi(\gamma_i)$	$T(\gamma_i)$	$N(\gamma_i)$	$T_2(\gamma_i)$	$N_2(\gamma_i)$
0	$0 = 0$	0	DNE	DNE	1	DNE	0	0	0	0	0
1	$1 = 1$	1	1	15	2	12	1	0	1	0	1
2	$\alpha = \delta$	2	12	1	4	9	4	1	1	11	11
3	$\alpha + 1 = \delta^{12}$	3	8	12	8	4	5	1	1	11	1
4	$\alpha^2 = \delta^2$	4	6	2	9	3	9	1	1	10	10
5	$\alpha^2 + 1 = \delta^9$	5	15	9	11	10	8	1	1	10	1
6	$\alpha^2 + \alpha = \delta^{13}$	6	4	13	15	8	13	0	1	1	11
7	$\alpha^2 + \alpha + 1 = \delta^7$	7	14	7	7	13	12	0	1	1	11
8	$\alpha^3 = \delta^3$	8	3	3	14	6	15	1	1	11	1
9	$\alpha^3 + 1 = \delta^4$	9	13	4	5	2	14	1	1	11	11
10	$\alpha^3 + \alpha = \delta^{10}$	10	11	10	10	5	11	0	1	0	11
11	$\alpha^3 + \alpha + 1 = \delta^5$	11	10	5	13	14	10	0	1	0	10
12	$\alpha^3 + \alpha^2 = \delta^{14}$	12	2	14	3	1	6	0	1	1	10
13	$\alpha^3 + \alpha^2 + 1 = \delta^{11}$	13	9	11	6	7	7	0	1	1	10
14	$\alpha^3 + \alpha^2 + \alpha = \delta^8$	14	7	8	12	11	2	1	1	10	10
15	$\alpha^3 + \alpha^2 + \alpha + 1 = \delta^6$	15	5	6	1	DNE	3	1	1	10	1

The command

Code Snippet 70

```

1   F_16_tables:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 16 -end \
4   ▷ ▷ -with F -do -finite_field_activity -export_tables -end
5   ▷ $(ORBITER) -v 3 \
6   ▷ ▷ -define all_one -vector -repeat 1 16 -end \
7   ▷ ▷ -draw_matrix \
8   ▷ ▷ ▷ -input_csv_file GF_q16_table_add.csv \
9   ▷ ▷ ▷ -box_width 40 -bit_depth 24 \
10  ▷ ▷ ▷ -partition 3 all_one all_one \
11  ▷ ▷ -end \
12  ▷ ▷ -draw_matrix \
13  ▷ ▷ ▷ -input_csv_file GF_q16_table_mul.csv \
14  ▷ ▷ ▷ -box_width 40 -bit_depth 24 \
15  ▷ ▷ ▷ -partition 3 all_one all_one \

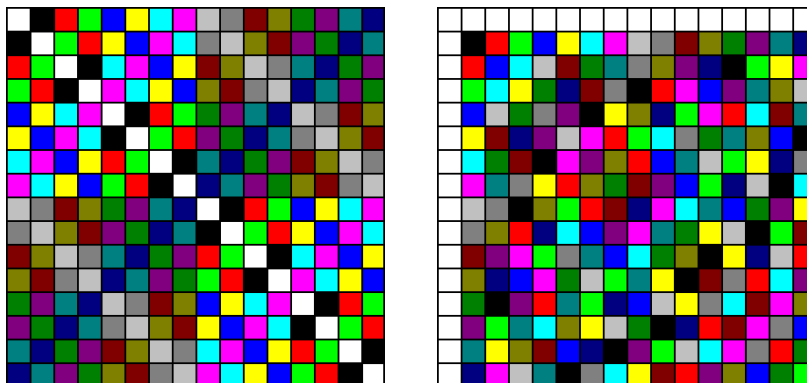
```

```

16  ▷ ▷ -end
17  ▷ magick GF_q16_table_add_draw.bmp GF_q16_table_add_draw.png
18  ▷ magick GF_q16_table_mul_draw.bmp GF_q16_table_mul_draw.png
19

```

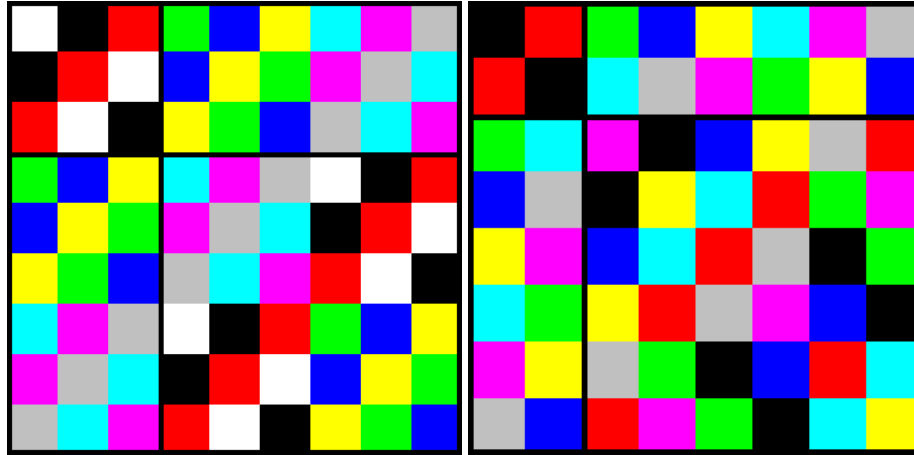
creates the addition and multiplication tables for the field:



Unlike GAP [32] and Magma [20], Orbiter does not use Conway polynomials to create field extensions. Instead, it provides the option to override the polynomial used to create the finite field. For subfield relationships, the cheat sheet will indicate the irreducible polynomials of all subfields for a given field. The table below is taken from a cheat sheet for the field $\mathbb{F}_{64}$ generated by the polynomial $X^6 + X^5 + 1$, whose numerical rank is 97:

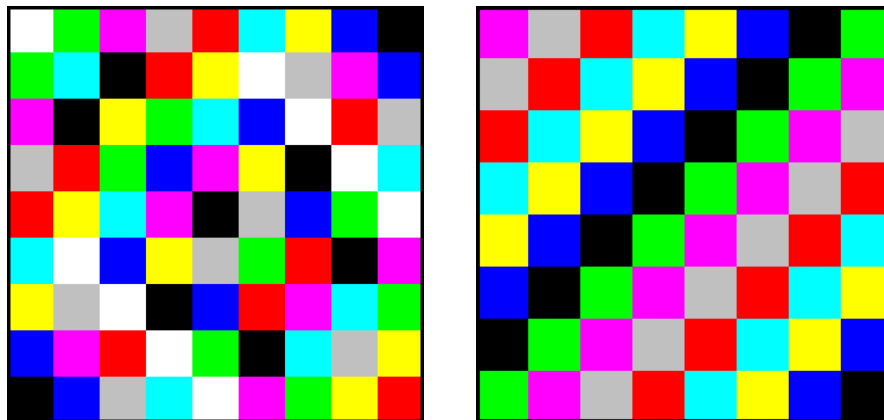
Subfield	Polynomial	Numerical rank
$\mathbb{F}_4$	$X^2 + X + 1$	7
$\mathbb{F}_8$	$X^3 + X + 1$	11

The lexicographic ordering has an interesting side-effect for the ordering of elements in extension fields. The elements of the prime subfield are always listed before any other elements in the extension field. For this reason, the addition and multiplication tables of the extension field contain the respective table of the prime field in the upper left corner, as shown below for the field $\mathbb{F}_9$:



Here, we omit the zero element in the multiplication table.

Orbiter uses primitive polynomials for creating extension fields. Because of this, the element α is always primitive. Since the numerical rank of α is p , this means that the rank p always represents a primitive element in an extension field. For the addition and multiplication tables of $\mathbb{F}_9$ arranged with respect to powers of a primitive element, see below:



The command

Code Snippet 71

```

1  nth_roots.63:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -with F -do -finite_field_activity \
5  ▷ ▷ ▷ -nth_roots 63 \
6  ▷ ▷ -end
7  ▷ $(PDFLATEX) Nth_roots_q2_n63.tex
8  ▷ $(OPEN) Nth_roots_q2_n63.pdf
9

```

creates the minimum polynomials of all elements in $\mathbb{F}_{64}^\times$. This corresponds to all irreducible polynomials over $\mathbb{F}_2$ of degree dividing 6, apart from the polynomial X .

i	r_i	$\text{Cyc}(r_i)$	$m_{\beta^{r_i}}(X)$	$m_{\beta^{r_i}}(X)$	rank
0	0	(0)	$(100000)X^0 + (100000)X^1$	$X + 1$	3
1	1	(1, 2, 4, 8, 16, 32)	$(100000)X^0 + (000000)X^1 + (000000)X^2 + (000000)X^3 + (000000)X^4 + (100000)X^5 + (100000)X^6$	$X^6 + X^5 + 1$	97
2	3	(3, 6, 12, 24, 48, 33)	$(100000)X^0 + (000000)X^1 + (100000)X^2 + (000000)X^3 + (100000)X^4 + (100000)X^5 + (100000)X^6$	$X^6 + X^5 + X^4 + X^2 + 1$	117
3	5	(5, 10, 20, 40, 17, 34)	$(100000)X^0 + (100000)X^1 + (000000)X^2 + (000000)X^3 + (100000)X^4 + (100000)X^5 + (100000)X^6$	$X^6 + X^5 + X^4 + X + 1$	115
4	7	(7, 14, 28, 56, 49, 35)	$(100000)X^0 + (000000)X^1 + (000000)X^2 + (100000)X^3 + (000000)X^4 + (000000)X^5 + (100000)X^6$	$X^6 + X^3 + 1$	73
5	9	(9, 18, 36)	$(100000)X^0 + (100000)X^1 + (000000)X^2 + (100000)X^3$	$X^3 + X + 1$	11
6	11	(11, 22, 44, 25, 50, 37)	$(100000)X^0 + (100000)X^1 + (000000)X^2 + (100000)X^3 + (100000)X^4 + (000000)X^5 + (100000)X^6$	$X^6 + X^4 + X^3 + X + 1$	91
7	13	(13, 26, 52, 41, 19, 38)	$(100000)X^0 + (000000)X^1 + (100000)X^2 + (100000)X^3 + (000000)X^4 + (100000)X^5 + (100000)X^6$	$X^6 + X^5 + X^3 + X^2 + 1$	109
8	15	(15, 30, 60, 57, 51, 39)	$(100000)X^0 + (100000)X^1 + (100000)X^2 + (000000)X^3 + (100000)X^4 + (000000)X^5 + (100000)X^6$	$X^6 + X^4 + X^2 + X + 1$	87
9	21	(21, 42)	$(100000)X^0 + (100000)X^1 + (100000)X^2$	$X^2 + X + 1$	7
10	23	(23, 46, 29, 58, 53, 43)	$(100000)X^0 + (100000)X^1 + (100000)X^2 + (000000)X^3 + (000000)X^4 + (100000)X^5 + (100000)X^6$	$X^6 + X^5 + X^2 + X + 1$	103
11	27	(27, 54, 45)	$(100000)X^0 + (000000)X^1 + (100000)X^2 + (100000)X^3$	$X^3 + X^2 + 1$	13
12	31	(31, 62, 61, 59, 55, 47)	$(100000)X^0 + (100000)X^1 + (000000)X^2 + (000000)X^3 + (000000)X^4 + (000000)X^5 + (100000)X^6$	$X^6 + X + 1$	67

The command

Code Snippet 72

```

1  F8_F2_field_reduction:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 8 -end \
4  ▷ ▷ -define v -vector -loop 0 8 1 -end \
5  ▷ ▷ -define C -code -field F -Reed_Solomon 7 3 -end \
6  ▷ ▷ -with F -do \
7  ▷ ▷ -finite_field_activity \
8  ▷ ▷ -field_reduction "F8.to.F2" \
9  ▷ ▷ ▷ 2 1 8 v \
10 ▷ ▷ -end
11 ▷ $(ORBITER) -v 2 \
12 ▷ ▷ -draw_matrix -input_csv_file F8.to.F2.csv \
13 ▷ ▷ ▷ -box_width 40 -bit_depth 24 \
14 ▷ ▷ ▷ -partition 4 "3" "3,3,3,3,3,3,3" \

```

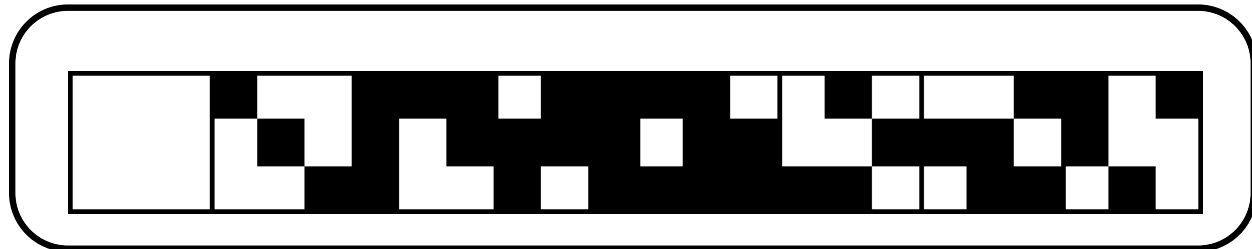


```

15  ▷ ▷ -end
16  ▷ magick F8_to_F2_draw.bmp F8_to_F2_draw.png
17  ▷ $(OPEN) F8_to_F2_draw.png
18

```

performs field reduction on the elements $0, \dots, 7$ in $\mathbb{F}_8$ down to $\mathbb{F}_2$. Here is a graphical representation of the matrices arising this way:



Let us discuss field embeddings in Orbiter. This is important for working in field extensions, as there is a restriction on the polynomials used to create the fields. Suppose we have fields $\mathbb{F}_q$ and $\mathbb{F}_Q$ with

$$\mathbb{F}_q \leq \mathbb{F}_Q,$$

i.e.

$$Q = q^e$$

for some positive integer e . Suppose α is a primitive element in $\mathbb{F}_Q$ and β is a primitive element in $\mathbb{F}_q$. Then Orbiter assumes that

$$\beta = \alpha^{\frac{Q-1}{q-1}},$$

where

$$\frac{Q-1}{q-1}$$

is the index of $\mathbb{F}_q^\times$ in the multiplicative group $\mathbb{F}_Q^\times$. With respect to this embedding, the minimum polynomial of β is determined. Let us consider an example. The command

Code Snippet 73

```

1  F_4096:
2  ▷ $(ORBITER) -v 4 \
3  ▷ ▷ -define F -finite_field -q 4096 \
4  ▷ ▷ ▷ -compute_related_fields \
5  ▷ ▷ -end \
6  ▷ ▷ -with F -do -finite_field_activity \
7  ▷ ▷ ▷ -cheat_sheet_GF \
8  ▷ ▷ -end
9  ▷ $(PDFLATEX) GF_4096.tex
10 ▷ $(OPEN) GF_4096.pdf
11

```

creates the field $\mathbb{F}_{2^{12}}$. Orbiter chooses the polynomial

$$X^{12} + X^6 + X^4 + X + 1$$

to create the field. This field has non-trivial subfields $\mathbb{F}_4$, $\mathbb{F}_8$, $\mathbb{F}_{16}$ and $\mathbb{F}_{64}$. The report shows the polynomials for the primitive elements of the subfields:

Subfield	Polynomial	Numerical Rank
$\mathbb{F}_4$	$X^2 + X + 1$	7
$\mathbb{F}_8$	$X^3 + X + 1$	11
$\mathbb{F}_{16}$	$X^4 + X + 1$	19
$\mathbb{F}_{64}$	$X^6 + X^5 + 1$	97

For each subfield, including the prime field, Orbiter computes a subfield basis and the embedding of the elements in the subfield. We use the convention that $d = \frac{Q-1}{q-1}$. Here is the output from the report:

Subfield $\mathbb{F}_2$ generated by polynomial 3:

Subfield of Order 2:

polynomial:

Field basis:

(1, 83, 342, 598, 1143, 518, 1492, 938, 1881, 3381, 1159, 227)

The power table shows $(\alpha^d)^i$ for $i = 0, 1, \dots, e$ where $q = p^e$:

i	$i \cdot d$	α^{id}	vector
0	0	1	(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1)
1	4095	1	(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1)

Subfield $\mathbb{F}_4$ generated by polynomial 7:

Subfield of Order 4:

polynomial: 7

Field basis:

(1, 64, 83, 1171, 342, 1183)

The power table shows $(\alpha^d)^i$ for $i = 0, 1, \dots, e$ where $q = p^e$:

i	$i \cdot d$	α^{id}	vector
0	0	1	(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1)
1	1365	70	(0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0)
2	2730	71	(0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1)

Subfield $\mathbb{F}_8$ generated by polynomial 11:

Subfield of Order 8:

polynomial: 11

Field basis:

(1, 16, 256, 83)

The power table shows $(\alpha^d)^i$ for $i = 0, 1, \dots, e$ where $q = p^e$:

i	$i \cdot d$	α^{id}	vector
0	0	1	(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1)
1	585	937	(0, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 1)
2	1170	1198	(0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0)
3	1755	936	(0, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0)

Subfield $\mathbb{F}_{16}$ generated by polynomial 19:

Subfield of Order 16:

polynomial: 19

Field basis:

$$(1, 8, 64)$$

The power table shows $(\alpha^d)^i$ for $i = 0, 1, \dots, e$ where $q = p^e$:

i	$i \cdot d$	α^{id}	vector
0	0	1	(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1)
1	273	1971	(0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1)
2	546	2037	(0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1)
3	819	3214	(1, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0)
4	1092	1970	(0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0)

Subfield $\mathbb{F}_{64}$ generated by polynomial 97:

Subfield of Order 64:

polynomial: 97

Field basis:

$$(1, 4)$$

The power table shows $(\alpha^d)^i$ for $i = 0, 1, \dots, e$ where $q = p^e$:

i	$i \cdot d$	α^{id}	vector
0	0	1	(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1)
1	65	396	(0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0)
2	130	1068	(0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0)
3	195	1315	(0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1)
4	260	1615	(0, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1)
5	325	811	(0, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 1)
6	390	810	(0, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0)

The command

Code Snippet 74

```
1      F_4489:
2      ▷ $(ORBITER) -v 4 \
3      ▷ ▷ -define F -finite_field -q 4489 -without_tables -end \
4
```

creates the field $\mathbb{F}_{67^2}$. For the sake of speed, the option `-without_tables` is given to prevent the field tables from being computed.

Finite Field Activities Pertaining to Linear Algebra		
Command	Arguments	Purpose
-nullspace	L	Compute a basis for the right nullspace of the matrix L
-RREF	L	Compute the RREF of the matrix L over $\mathbb{F}_q$
-RREF_random_matrix	$m \ n$	Produce a random $m \times n$ matrix over $\mathbb{F}_q$ and perform RREF.
-Walsh_matrix	n	Create the Walsh matrix of order n .
-Vandermonde_matrix		Create the Vandermonde matrix of order q and compute its inverse. The (i, j) entry is γ_i^j ($i, j = 0, \dots, q-1$). Here, $\gamma_0, \dots, \gamma_{q-1}$ is the list of elements in $\mathbb{F}_q$ in Orbiter ordering. The matrix and its inverse are written to file.

Table 3.5: Finite Field Activities Pertaining to Linear Algebra

3.3 Linear Algebra Over Finite Fields

In Table 3.5, some finite field activities regarding linear algebra are shown. For instance, the command

Code Snippet 75

```

1  RREF:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -define v -vector -field F -format 2 \
5  ▷ ▷ ▷ -dense "1,1,1,1,0,1,1,0,0,1" \
6  ▷ ▷ -end \
7  ▷ ▷ -with F -do -finite_field_activity \
8  ▷ ▷ ▷ -RREF v \
9  ▷ ▷ -end
10 ▷ $(PDFLATEX) v_rref.tex
11 ▷ $(OPEN) v_rref.pdf
12
```

computes the RREF form of the matrix

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

over $\mathbb{F}_2$. The output is the matrix

$$\begin{bmatrix} 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{bmatrix}.$$

The -RREF command produces a latex log of the steps. This can be used to follow the algorithm along. For a somewhat longer example, consider the Vandermonde matrix over the field $\mathbb{F}_7$. Suppose we want to compute the inverse matrix directly. We can use the following command to do so. Notice how we first create the matrix and an identity matrix next to it. After that we apply the -RREF command:

Code Snippet 76

```

1  V7_VANDERMONDE_EXTENDED="\
2  1,0,0,0,0,0,0,1,0,0,0,0,0,0, \
3  1,1,1,1,1,1,1,0,1,0,0,0,0,0, \
4  1,2,4,1,2,4,1,0,0,1,0,0,0,0, \
5  1,3,2,6,4,5,1,0,0,0,1,0,0,0, \
6  1,4,2,1,4,2,1,0,0,0,0,1,0,0, \
7  1,5,4,6,2,3,1,0,0,0,0,0,1,0, \
8  1,6,1,6,1,6,1,0,0,0,0,0,0,1"
9

```

Code Snippet 77

```

1  RREF_V7:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 7 -end \
4  ▷ ▷ -define V7 -vector -format 7 \
5  ▷ ▷ ▷ -dense $(V7_VANDERMONDE_EXTENDED) \
6  ▷ ▷ -end \
7  ▷ ▷ -with F -do -finite_field_activity \
8  ▷ ▷ ▷ -RREF V7 \
9  ▷ ▷ -end
10 ▷ $(PDFLATEX) V7_rref.tex
11 ▷ $(OPEN) V7_rref.pdf
12

```

The following (shortened) output is produced. Observe how the inverse matrix appears in the second half once the `-RREF` algorithm is finished:

A matrix over the field $\mathbb{F}_7$

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 2 & 4 & 1 & 2 & 4 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 3 & 2 & 6 & 4 & 5 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 4 & 2 & 1 & 4 & 2 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 5 & 4 & 6 & 2 & 3 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 6 & 1 & 6 & 1 & 6 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Position $(i, j) = (0, 0)$, found pivot in column 0

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 2 & 4 & 1 & 2 & 4 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 3 & 2 & 6 & 4 & 5 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 4 & 2 & 1 & 4 & 2 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 5 & 4 & 6 & 2 & 3 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 6 & 1 & 6 & 1 & 6 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

After making pivot 1:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 2 & 4 & 1 & 2 & 4 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 3 & 2 & 6 & 4 & 5 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 4 & 2 & 1 & 4 & 2 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 5 & 4 & 6 & 2 & 3 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 6 & 1 & 6 & 1 & 6 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

output truncated

After elimination above pivot 0 in position (0,0):

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 6 & 3 & 2 & 5 & 4 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 6 & 5 & 3 & 3 & 5 & 6 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 6 & 6 & 1 & 6 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 6 & 3 & 5 & 5 & 3 & 6 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 6 & 5 & 4 & 3 & 2 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 6 & 6 & 6 & 6 & 6 & 6 & 6 \end{bmatrix}$$

The inverse matrix agrees with the output obtained in Section 3.1.

Another task is computing the nullspace of a matrix. The command

Code Snippet 78

```
1  nullspace:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F2 -finite_field -q 2 -end \
4  ▷ ▷ -define v -vector -field F2 -format 2 \
5  ▷ ▷ ▷ -dense "1,1,1,1,0,1,1,0,0,1" \
6  ▷ ▷ -end \
7  ▷ ▷ -with F2 -do \
8  ▷ ▷ -finite_field_activity \
9  ▷ ▷ ▷ -nullspace v \
10 ▷ ▷ -end
11 ▷ $(PDFLATEX) v_nullspace.tex
12 ▷ $(OPEN) v_nullspace.pdf
13
```

computes the right nullspace of the matrix from the first example. The output is the matrix

$$\begin{bmatrix} 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix}.$$

Orbiter can compute eigenvalues and eigenvectors of matrices over finite fields. For instance, the command

Code Snippet 79

```

1   eigenstuff:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define F -finite_field -q 5 -end \
4   ▷ ▷ -eigenstuff F 4 "0,1,0,2,0,1,2,1,4,2,3,1,2,0,4,3"
5

```

computes all eigenvectors and eigenvalues of the matrix

$$\begin{bmatrix} 0 & 1 & 0 & 2 \\ 0 & 1 & 2 & 1 \\ 4 & 2 & 3 & 1 \\ 2 & 0 & 4 & 3 \end{bmatrix}$$

over $\mathbb{F}_5$.

Orbiter can produce a list of all conjugacy classes of endomorphisms of $\mathbb{F}_q^d$ by means of their rational normal forms. For instance

Code Snippet 80

```

1   classes_GL_3_2:
2   ▷ $(ORBITER) -v 7 \
3   ▷ ▷ -define F -finite_field -q 2 -end \
4   ▷ ▷ -all_rational_normal_forms F 3
5   ▷ $(PDFLATEX) Class_reps_GL_3_2.tex
6   ▷ $(OPEN) Class_reps_GL_3_2.pdf
7

```

produces a list of all conjugacy classes of $\text{GL}(3, 2)$. There are 6 of them. The report includes the order of the centralizer and the order of the conjugacy class. The order of the centralizer is computed using Kung's formula [47]. This command relies on the Orbiter catalogue of irreducible polynomials. For an introduction to the rational normal form of endomorphisms, see [53].

Conjugacy Classes of $\text{GL}(3, 2)$

The number of conjugacy classes of $\text{GL}(3, 2)$ is 6:

$$\begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Class 0 / 6

3, 1, 0

$$\begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

centralizer order 7

class size 24

Class 1 / 6

2, 1, 0

$$\begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix}$$

centralizer order 7

class size 24

Class 2 / 6

0, 1, 0; 1, 1, 0

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

centralizer order 3

class size 56

Class 3 / 6

0, 3, 0

$$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$$

centralizer order 4

class size 42

Class 4 / 6

0, 3, 1

$$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

centralizer order 8

class size 21

Class 5 / 6

0, 3, 2

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

centralizer order 168

class size 1

Finite Field Activities (Part I)		
Command	Arguments	Purpose
<code>-write_code_for_division</code>	<code>fname A B</code>	Write C++ source code for the polynomial division of A by B . See Section 11.4.
<code>-weight_enumerator</code>	<code>A</code>	Computes the weight enumerator of the code whose generator matrix is A .
<code>-Walsh_Hadamard_transform</code>	<code>fname n</code>	Computes the Walsh-Hadamard transform for the n -variable boolean function in the given file.
<code>-algebraic_normal_form_of_boolean_function</code>	<code>fname n</code>	Computes the algebraic normal form for the n -variable boolean function in the given file.
<code>-algebraic_normal_form</code>	<code>n input</code>	Computes the algebraic normal form for the n -variable function in the given file.
<code>-apply_trace_function</code>	<code>fname</code>	Applies the absolute trace function to the function in the given file.
<code>-apply_power_function</code>	<code>fname d</code>	Applies the raise-to-the-power- d function to the function in the given file.
<code>-identity_function</code>	<code>fname.csv</code>	Creates the identity function and stores in the given csv file.

Table 3.6: Finite Field Activities (Part I)

3.4 Advanced Topics in Finite Fields

Let us now look at some more advanced topics in the theory of finite fields.

In Tables 3.6-3.7, a summary of finite field activities is shown.

A normal basis for a field extension $\mathbb{F}_{q^d}$ over $\mathbb{F}_q$ is a basis of $\mathbb{F}_{q^d}$ as vector space over $\mathbb{F}_q$ which consists of one cycle of the Frobenius automorphism of $\mathbb{F}_{q^d}$ over $\mathbb{F}_q$. For instance, the command

Code Snippet 81

```

1    normal_basis_2_3:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 2 -end \
4    ▷ ▷ -with F -do -finite_field_activity \
5    ▷ ▷ ▷ -normal_basis 3 \
6    ▷ ▷ -end
7    ▷ $(PDFLATEX) normal_basis_q2_d3.tex
8    ▷ $(OPEN) normal_basis_q2_d3.pdf
9
```

computes a normal basis of $\mathbb{F}_8$ over $\mathbb{F}_2$. Using the polynomial $X^3 + X^2 + 1$, the normal basis in terms of the standard polynomial basis $1, X, X^2, \dots$ is given by the columns of the matrix

$$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}.$$

Reading the columns as coefficient vectors with respect to the standard basis, the normal basis is

$$b_1 = 1 + X + X^2, \quad b_2 = X, \quad b_3 = X^2.$$

Finite Field Activities (Part II)		
Command	Arguments	Purpose
-transversal	$L1\ L2\ P$	Computes the unique transversal to the lines $L1$ and $L2$ through the point P in $PG(3, q)$. The lines are given by a basis consisting of 8 field elements.
-intersection_of_two_lines	$L1\ L2$	Computes the intersection of two lines in $PG(3, q)$. The lines are given by a basis consisting of 8 field elements.
-rank_point_in_PG	P	Computes the orbiter point rank corresponding to the point P in $PG(n, q)$. P is a label of a vector, which is the coefficient vector.
-unrank_point_in_PG	r	Computes the coordinate vector of the Orbiter point in $PG(n, q)$ corresponding to the Orbiter rank value r .
-inverse_isomorphism_klein_quadric	$L36$	
-NTT	$k\ n$	Computes the Number-theoretic transform for $n = 2^k$, which must divide $q - 1$.

Table 3.7: Finite Field Activities (Part II)

Let us apply the Frobenius mapping Φ to the elements of the normal bases:

$$\begin{aligned}
 b_1^\Phi &= (1 + X + X^2)^2 = 1 + X^2 + X^4 = 1 + X^2 + X^3 + X = 1 + X + X^2 + X^2 + 1 = X = b_2, \\
 b_2^\Phi &= X^2 = b_3, \\
 b_3^\Phi &= X^4 = X^3 + X = X^2 + X + 1 = b_1.
 \end{aligned}$$

Thus,

$$b_1 \mapsto b_2 \mapsto b_3 \mapsto b_1$$

under Φ , as required.

A field is a vector space over any of its subfields. Using a field basis, the elements of the large field can be identified with invertible matrices. So, for $\mathbb{F}_{q^r}$ over $\mathbb{F}_q$, and for $a \in \mathbb{F}_{q^r}$, we consider the $\mathbb{F}_q$ -linear map

$$\mathbb{F}_{q^r} \rightarrow \mathbb{F}_{q^r}, x \mapsto ax.$$

The following code computes the field reduction from $\mathbb{F}_{64}$ to $\mathbb{F}_8$. Elements in the small field are represented as colors. The (i, j) -th block is the matrix of $a = i8 + j$ in the chosen basis.

Code Snippet 82

```

1  F_64_over_F8_field_reduction:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 64 -end \
4  ▷ ▷ -define elts -vector -field F -loop 0 64 1 -end \
5  ▷ ▷ -with F -do \
6  ▷ ▷ -finite_field_activity \
7  ▷ ▷ ▷ -field_reduction "F64_over_F8" 8 8 8 \
8  ▷ ▷ ▷ elts \
9  ▷ ▷ -end
10 ▷ $(ORBITER) -v 2 \

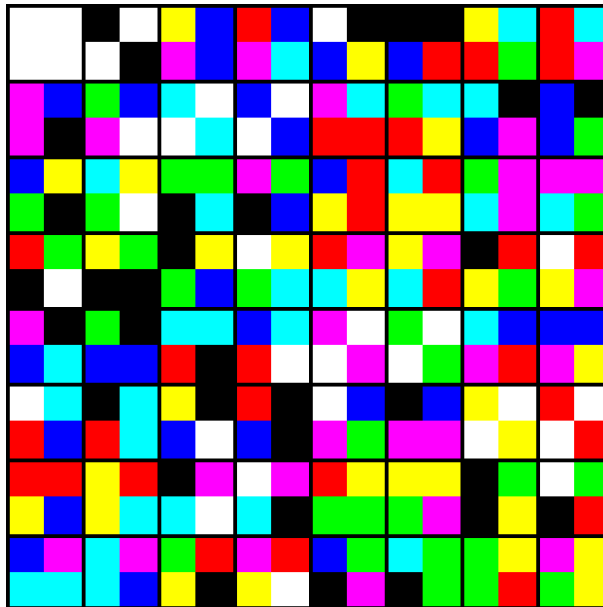
```

```

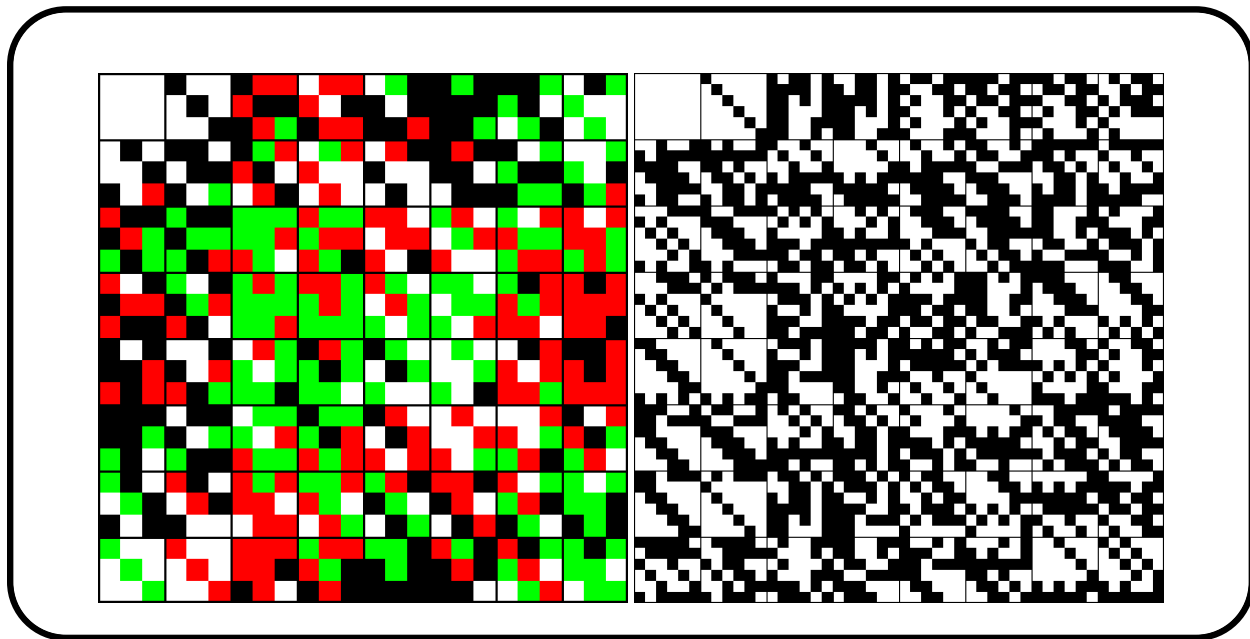
11  ▷ ▷ -draw_matrix \
12  ▷ ▷ ▷ -input_csv_file F64_over_F8.csv \
13  ▷ ▷ ▷ -box_width 40 -bit_depth 24 \
14  ▷ ▷ ▷ -partition 4 "2,2,2,2,2,2,2,2" "2,2,2,2,2,2,2,2" \
15  ▷ ▷ -end
16  ▷ $(OPEN) F64_over_F8_draw.bmp
17  ▷ $(PDFLATEX) field_reduction_Q64_q8_8_8.tex
18  ▷ $(OPEN) field_reduction_Q64_q8_8_8.pdf
19

```

The output is shown below:



Note that the dimension of the vector space is 2, so the block matrices are 2×2 . Observe that $\mathbb{F}_{64}$ has many subfields. The reduction from $\mathbb{F}_{64}$ to $\mathbb{F}_4$ and to $\mathbb{F}_2$ is also possible:



Here, the block matrices have size 3×3 when reducing to $\mathbb{F}_4$ (left picture) and 6×6 when reducing to $\mathbb{F}_2$.

The minimum polynomials associated with the n -th roots over $\mathbb{F}_q$ can be computed using the `-nth_roots` command, which is a finite field activity. The activity is applied to the field $\mathbb{F}_q$ over which the n -th roots are defined. The command constructs the field extension $\mathbb{F}_{q^m}$ where m is the order of q modulo n . This field extension contains the n -th roots of unity. Let α be a primitive element of $\mathbb{F}_{q^m}$ and let β be a generator of the subgroup of n -th roots. Also, let γ be the generator of the subgroup of $q - 1$ th roots, which are the elements of the multiplicative group of $\mathbb{F}_q$. The output lists the n -th roots first, generated by β . After that, the $q - 1$ th roots are shown, generated by γ . Finally, a table is produced which shows the irreducible polynomials over $\mathbb{F}_q$ associated with the n -th roots of unity. For instance, the following command computes the minimum polynomials of all 21st roots of unity over $\mathbb{F}_8$:

Code Snippet 83

```

1  F_8.Nth_roots_21:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
4  ▷ ▷ -with F -do -finite_field_activity \
5  ▷ ▷ ▷ -nth_roots 21 \
6  ▷ ▷ -end
7  ▷ $(PDFLATEX) Nth_roots_q8_n21.tex
8  ▷ $(OPEN) Nth_roots_q8_n21.pdf
9

```

The output is:

Let α be a primitive element of $\text{GF}(64)$. Let β be a primitive 21-th root in $\text{GF}(64)$, so $\beta = \alpha^3$.

$$\beta^0 = 100000 = 1$$

$$\beta^1 = 000100 = \alpha^3$$

$$\beta^2 = 100001 = \alpha^5 + 1$$

$$\beta^3 = 111101 = \alpha^5 + \alpha^3 + \alpha^2 + \alpha + 1$$

$$\beta^4 = 011111 = \alpha^5 + \alpha^4 + \alpha^3 + \alpha^2 + \alpha$$

$$\begin{aligned}
\beta^5 &= 101010 = \alpha^4 + \alpha^2 + 1 \\
\beta^6 &= 110100 = \alpha^3 + \alpha + 1 \\
\beta^7 &= 100111 = \alpha^5 + \alpha^4 + \alpha^3 + 1 \\
\beta^8 &= 101101 = \alpha^5 + \alpha^3 + \alpha^2 + 1 \\
\beta^9 &= 011101 = \alpha^5 + \alpha^3 + \alpha^2 + \alpha \\
\beta^{10} &= 011011 = \alpha^5 + \alpha^4 + \alpha^2 + \alpha \\
\beta^{11} &= 001011 = \alpha^5 + \alpha^4 + \alpha^2 \\
\beta^{12} &= 001001 = \alpha^5 + \alpha^2 \\
\beta^{13} &= 111000 = \alpha^2 + \alpha + 1 \\
\beta^{14} &= 000111 = \alpha^5 + \alpha^4 + \alpha^3 \\
\beta^{15} &= 101001 = \alpha^5 + \alpha^2 + 1 \\
\beta^{16} &= 111100 = \alpha^3 + \alpha^2 + \alpha + 1 \\
\beta^{17} &= 100110 = \alpha^4 + \alpha^3 + 1 \\
\beta^{18} &= 010100 = \alpha^3 + \alpha \\
\beta^{19} &= 100011 = \alpha^5 + \alpha^4 + 1 \\
\beta^{20} &= 001100 = \alpha^3 + \alpha^2
\end{aligned}$$

Let γ be a primitive 7-th root in $\text{GF}(64)$, so $\gamma = \alpha^9$.

$$\begin{aligned}
\gamma^0 &= 100000 = 1 \\
\gamma^1 &= 111101 = \alpha^5 + \alpha^3 + \alpha^2 + \alpha + 1 \\
\gamma^2 &= 110100 = \alpha^3 + \alpha + 1 \\
\gamma^3 &= 011101 = \alpha^5 + \alpha^3 + \alpha^2 + \alpha \\
\gamma^4 &= 001001 = \alpha^5 + \alpha^2 \\
\gamma^5 &= 101001 = \alpha^5 + \alpha^2 + 1 \\
\gamma^6 &= 010100 = \alpha^3 + \alpha
\end{aligned}$$

The q -cyclotomic set for $q = 8$ are:

$$\begin{aligned}
&\{ 0 \} \\
&\{ 1, 8 \} \\
&\{ 2, 16 \} \\
&\{ 3 \} \\
&\{ 4, 11 \} \\
&\{ 5, 19 \} \\
&\{ 6 \} \\
&\{ 7, 14 \} \\
&\{ 9 \} \\
&\{ 10, 17 \} \\
&\{ 12 \} \\
&\{ 13, 20 \} \\
&\{ 15 \} \\
&\{ 18 \}
\end{aligned}$$

Subfield basis, a basis for $\text{GF}(8)$ inside $\text{GF}(64)$:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \end{bmatrix}$$

The irreducible polynomials associated with the 21-th roots over $\text{GF}(8)$ are:

i	r_i	$\text{Cyc}(r_i)$	$m_{\beta^{r_i}}(X)$	$m_{\beta^{r_i}}(X)$
0	0	(0)	$(100000)X^0 + (100000)X^1$	$X + 1$
1	1	(1, 8)	$(011101)X^0 + (101001)X^1 + (100000)X^2$	$X^2 + 7X + 3$
2	2	(2, 16)	$(010100)X^0 + (011101)X^1 + (100000)X^2$	$X^2 + 3X + 5$
3	3	(3)	$(111101)X^0 + (100000)X^1$	$X + 2$
4	4	(4, 11)	$(101001)X^0 + (010100)X^1 + (100000)X^2$	$X^2 + 5X + 7$
5	5	(5, 19)	$(111101)X^0 + (001001)X^1 + (100000)X^2$	$X^2 + 6X + 2$
6	6	(6)	$(110100)X^0 + (100000)X^1$	$X + 4$
7	7	(7, 14)	$(100000)X^0 + (100000)X^1 + (100000)X^2$	$X^2 + X + 1$
8	9	(9)	$(011101)X^0 + (100000)X^1$	$X + 3$
9	10	(10, 17)	$(110100)X^0 + (111101)X^1 + (100000)X^2$	$X^2 + 2X + 4$
10	12	(12)	$(001001)X^0 + (100000)X^1$	$X + 6$
11	13	(13, 20)	$(001001)X^0 + (110100)X^1 + (100000)X^2$	$X^2 + 4X + 6$
12	15	(15)	$(101001)X^0 + (100000)X^1$	$X + 7$
13	18	(18)	$(010100)X^0 + (100000)X^1$	$X + 5$

In Section 3.1, we have considered the Vandermonde matrix over $\mathbb{F}_7$. Let us do the same for the field $\mathbb{F}_8$ instead. We use the following command:

Code Snippet 84

```

1  F_8_vandermonde:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define F -finite_field -q 8 -end \
4  ▷ ▷ -with F -do -finite_field_activity \
5  ▷ ▷ ▷ -Vandermonde_matrix \
6  ▷ ▷ -end
7
```

The output is shown below. Again, the first matrix is $V = (x_i^j)$. The second matrix is V^{-1} :

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 5 & 7 & 3 & 6 & 1 \\ 1 & 3 & 5 & 2 & 6 & 7 & 4 & 1 \\ 1 & 4 & 7 & 6 & 2 & 5 & 3 & 1 \\ 1 & 5 & 6 & 4 & 3 & 2 & 7 & 1 \\ 1 & 6 & 3 & 7 & 5 & 4 & 2 & 1 \\ 1 & 7 & 2 & 3 & 4 & 6 & 5 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 6 & 4 & 3 & 7 & 2 & 5 \\ 0 & 1 & 3 & 7 & 5 & 2 & 4 & 6 \\ 0 & 1 & 7 & 6 & 2 & 3 & 5 & 4 \\ 0 & 1 & 5 & 2 & 6 & 4 & 7 & 3 \\ 0 & 1 & 4 & 5 & 7 & 6 & 3 & 2 \\ 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Let us now do a somewhat larger example of the same problem. The next command computes the Vandermonde matrix and its inverse over the field $\mathbb{F}_{512}$:

Code Snippet 85

```
1 F_512_vandermonde:
2 ▷ $(ORBITER) -v 3 \
3 ▷ ▷ -define F -finite_field -q 512 -end \
4 ▷ ▷ -with F -do -finite_field_activity \
5 ▷ ▷ ▷ -Vandermonde_matrix \
6 ▷ ▷ -end
7 ▷ - rm Vandermonde_q512.csv
8 ▷ - rm Vandermonde_q512_inv.csv
9
```

The matrix is not shown. It would be too big to be printed. In order to save disc space, we delete the output files, using the `rm` command.

Orbiter can create code for the number theoretic transform. This is the discrete Fourier transform performed over finite fields. The generated code can be compiled with the Orbiter library. Compiling code requires additional makefile options are necessary. Because of this, we define the following makefile variables at the top of the makefile.

Code Snippet 86

```
1 SRC=$(ORBITER_PATH)/src
2 MY_CPP = g++
3 MY_CC = gcc
4 CPPFLAGS = -Wall -I../../DEV.23/orbiter/src/lib -std=c++14
5 LIB = $(SRC)/lib/liborbiter.a -lpthread
6 LFLAGS = -lm -Wl,-rpath -Wl,/usr/local/gcc-8.2.0/lib64
7
```

Suppose we want to create the number theoretic transform for the 16th roots of unity inside the field $\mathbb{F}_{17}$. Here is the command to generate the Orbiter source code:

Code Snippet 87

```
1   NTT_k4_q17.cpp:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 17 -end \
4   ▷ ▷ -with F -do -coding_theoretic_activity \
5   ▷ ▷ ▷ -NTT 4 17 \
6   ▷ ▷ -end
7
```

This produces a C++ file `NTT_k4_q17.cpp`. This file should be compiled and linked against the Orbiter library. The command

Code Snippet 88

```
1   F_17_NTT_compile: NTT_k4_q17.cpp
2   ▷ $(MY_CPP) NTT_k4_q17.cpp $(CPPFLAGS) \
3   ▷ ▷ $(LIB) $(LFLAGS) -o NTT_k4_q17.out
4   ▷ ./NTT_k4_q17.out
5
```

can be used to compile the code and run it. Note the dependency on the file `NTT_k4_q17.cpp`. This means that `make` would automatically invoke the first command if only the second one was issued.

Chapter 4

Projective Geometry

4.1 Finite Projective Spaces

Orbiter commands to create a finite projective space $\text{PG}(n, q)$ are listed in Table 4.1. The command also creates the group of the space, which is $\text{PTL}(n + 1, q)$. Let us look at a very simple example. Suppose we want to create $\text{PG}(3, 2)$. The following command sequence creates the finite field $\mathbb{F}_2$ and stores the object in the symbol table, using the label F . After that, the projective space $\text{PG}(3, F)$ is created, using the field F . The projective space is stored in the symbol table under the label P .

Code Snippet 89

```
1 PG_3_2_easy:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 2 -end \
4   ▷ ▷ -define P -projective_space -n 3 -field F -end
5
```

Orbiter offers indexing for points and for subspaces of all $\text{PG}(n, q)$ of a fixed dimension. The incidence matrix between points and lines with respect to this ordering can be computed. The indexing is used to establish the permutation representations of the projective group, as will be described in Section 5.2. The indexing of points is not the lexicographic ordering. It emphasizes the role of frames in the geometry by

Options for Creating a Projective Space		
Command	Arguments	Purpose
-n	n	Specify the projective dimension n .
-q	q	Specify the order q of the underlying finite field $\mathbb{F}_q$. A default version of the field $\mathbb{F}_q$ will be created.
-field	F	Specify the underlying field F .
-use_projectivity_subgroup		Create the projectivity group instead of the collineation group.
-v	k	Set verbosity to k during creation of the projective space. Higher values of k give more output. $k = 0$ is silent mode.

Table 4.1: Options for Creating a Projective Space

assigning the smallest rank values to the members of the standard frame. After that, the other points are listed.

Orbiter can create a cheat sheet for $PG(n, q)$. The cheat sheet lists the objects in the space, and basic properties. Here are two examples. The first is a projective plane. The second is a projective 3-space.

The following command creates a cheat sheet for $PG(2, 4)$:

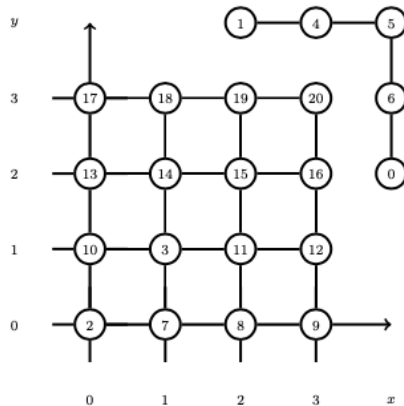
Code Snippet 90

```

1   PG_2_4:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw.options -end \
4   ▷ ▷ -define F -finite_field -q 4 -compute_related_fields -end \
5   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
6   ▷ ▷ -with P -do -projective_space_activity \
7   ▷ ▷ ▷ -cheat_sheet Do \
8   ▷ ▷ -end
9   ▷ $(PDFLATEX) PG_2_4.tex
10  ▷ $(OPEN) PG_2_4.pdf
11

```

The cheat sheet contains a drawing of the plane:



The affine plane is shown in the cartesian plane, while the line at infinity is wrapped around the top right corner. The cheat sheet continues by listing the points, including the canonical Baer subgeometry $PG(2, 2)$. After that, the points are listed again, but with left-normalized vectors. Finally, the lines are shown.

$PG(2, 4)$ has 21 points:

$$\begin{aligned}
P_0 &= (1, 0, 0) = (1, 0, 0) \\
P_1 &= (0, 1, 0) = (0, 1, 0) \\
P_2 &= (0, 0, 1) = (0, 0, 1) \\
P_3 &= (1, 1, 1) = (1, 1, 1) \\
P_4 &= (1, 1, 0) = (1, 1, 0) \\
P_5 &= (2, 1, 0) = (\alpha, 1, 0) \\
P_6 &= (3, 1, 0) = (\alpha^2, 1, 0) \\
P_7 &= (1, 0, 1) = (1, 0, 1) \\
P_8 &= (2, 0, 1) = (\alpha, 0, 1) \\
P_9 &= (3, 0, 1) = (\alpha^2, 0, 1) \\
P_{10} &= (0, 1, 1) = (0, 1, 1)
\end{aligned}$$

$$\begin{aligned}
P_{11} &= (2, 1, 1) = (\alpha, 1, 1) \\
P_{12} &= (3, 1, 1) = (\alpha^2, 1, 1) \\
P_{13} &= (0, 2, 1) = (0, \alpha, 1) \\
P_{14} &= (1, 2, 1) = (1, \alpha, 1) \\
P_{15} &= (2, 2, 1) = (\alpha, \alpha, 1) \\
P_{16} &= (3, 2, 1) = (\alpha^2, \alpha, 1) \\
P_{17} &= (0, 3, 1) = (0, \alpha^2, 1) \\
P_{18} &= (1, 3, 1) = (1, \alpha^2, 1) \\
P_{19} &= (2, 3, 1) = (\alpha, \alpha^2, 1) \\
P_{20} &= (3, 3, 1) = (\alpha^2, \alpha^2, 1)
\end{aligned}$$

Baer subgeometry:

$$\begin{array}{llll}
P_0 = (1, 0, 0) & P_2 = (0, 0, 1) & P_4 = (1, 1, 0) & P_{10} = (0, 1, 1) \\
P_1 = (0, 1, 0) & P_3 = (1, 1, 1) & P_7 = (1, 0, 1) &
\end{array}$$

There are 7 elements in the Baer subgeometry.
Normalized from the left:

$$\begin{array}{llll}
P_0 = (1, 0, 0) & P_6 = (1, 2, 0) & P_{12} = (1, 2, 2) & P_{18} = (1, 3, 1) \\
P_1 = (0, 1, 0) & P_7 = (1, 0, 1) & P_{13} = (0, 1, 3) & P_{19} = (1, 2, 3) \\
P_2 = (0, 0, 1) & P_8 = (1, 0, 3) & P_{14} = (1, 2, 1) & P_{20} = (1, 1, 2) \\
P_3 = (1, 1, 1) & P_9 = (1, 0, 2) & P_{15} = (1, 1, 3) & \\
P_4 = (1, 1, 0) & P_{10} = (0, 1, 1) & P_{16} = (1, 3, 2) & \\
P_5 = (1, 3, 0) & P_{11} = (1, 3, 3) & P_{17} = (0, 1, 2) &
\end{array}$$

The Lines of $\text{PG}(2, 4)$. $\text{PG}(2, 4)$ has 21 1-subspaces:

$$\begin{array}{lll}
L_0 = \begin{bmatrix} 100 \\ 010 \end{bmatrix} & L_7 = \begin{bmatrix} 101 \\ 012 \end{bmatrix} & L_{14} = \begin{bmatrix} 120 \\ 001 \end{bmatrix} \\
L_1 = \begin{bmatrix} 100 \\ 011 \end{bmatrix} & L_8 = \begin{bmatrix} 101 \\ 013 \end{bmatrix} & L_{15} = \begin{bmatrix} 103 \\ 010 \end{bmatrix} \\
L_2 = \begin{bmatrix} 100 \\ 012 \end{bmatrix} & L_9 = \begin{bmatrix} 110 \\ 001 \end{bmatrix} & L_{16} = \begin{bmatrix} 103 \\ 011 \end{bmatrix} \\
L_3 = \begin{bmatrix} 100 \\ 013 \end{bmatrix} & L_{10} = \begin{bmatrix} 102 \\ 010 \end{bmatrix} & L_{17} = \begin{bmatrix} 103 \\ 012 \end{bmatrix} \\
L_4 = \begin{bmatrix} 100 \\ 001 \end{bmatrix} & L_{11} = \begin{bmatrix} 102 \\ 011 \end{bmatrix} & L_{18} = \begin{bmatrix} 103 \\ 013 \end{bmatrix} \\
L_5 = \begin{bmatrix} 101 \\ 010 \end{bmatrix} & L_{12} = \begin{bmatrix} 102 \\ 012 \end{bmatrix} & L_{19} = \begin{bmatrix} 130 \\ 001 \end{bmatrix} \\
L_6 = \begin{bmatrix} 101 \\ 011 \end{bmatrix} & L_{13} = \begin{bmatrix} 102 \\ 013 \end{bmatrix} & L_{20} = \begin{bmatrix} 010 \\ 001 \end{bmatrix}
\end{array}$$

The following command creates a cheat sheet for $\text{PG}(3, 2)$.

Code Snippet 91

```

1   PG_3_2:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options -end \
4   ▷ ▷ -define F -finite_field -q 2 -end \
5   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
6   ▷ ▷ -with P -do -projective_space_activity \
7   ▷ ▷ ▷ -cheat_sheet Do \
8   ▷ ▷ -end
9   ▷ $(PDFLATEX) PG_3_2.tex
10  ▷ $(OPEN) PG_3_2.pdf
11

```

The cheat sheet shows points, lines and planes. The lines are shown together with their Plücker coordinates. The lines whose Plücker coordinates are unit vectors are shown separately.

The projective space $\text{PG}(3, 2)$

$$q = 2$$

$$p = 2$$

$$e = 1$$

$$n = 3$$

Number of points = 15

Number of lines = 35

Number of lines on a point = 7

Number of points on a line = 3

The points of $\text{PG}(3, 2)$

$\text{PG}(3, 2)$ has 15 points:

$$P_0 = (1, 0, 0, 0)$$

$$P_4 = (1, 1, 1, 1)$$

$$P_8 = (1, 1, 1, 0)$$

$$P_{12} = (0, 0, 1, 1)$$

$$P_1 = (0, 1, 0, 0)$$

$$P_5 = (1, 1, 0, 0)$$

$$P_9 = (1, 0, 0, 1)$$

$$P_{13} = (1, 0, 1, 1)$$

$$P_2 = (0, 0, 1, 0)$$

$$P_6 = (1, 0, 1, 0)$$

$$P_{10} = (0, 1, 0, 1)$$

$$P_{14} = (0, 1, 1, 1)$$

$$P_3 = (0, 0, 0, 1)$$

$$P_7 = (0, 1, 1, 0)$$

$$P_{11} = (1, 1, 0, 1)$$

Normalized from the left:

$$P_0 = (1, 0, 0, 0)$$

$$P_4 = (1, 1, 1, 1)$$

$$P_8 = (1, 1, 1, 0)$$

$$P_{12} = (0, 0, 1, 1)$$

$$P_1 = (0, 1, 0, 0)$$

$$P_5 = (1, 1, 0, 0)$$

$$P_9 = (1, 0, 0, 1)$$

$$P_{13} = (1, 0, 1, 1)$$

$$P_2 = (0, 0, 1, 0)$$

$$P_6 = (1, 0, 1, 0)$$

$$P_{10} = (0, 1, 0, 1)$$

$$P_{14} = (0, 1, 1, 1)$$

$$P_3 = (0, 0, 0, 1)$$

$$P_7 = (0, 1, 1, 0)$$

$$P_{11} = (1, 1, 0, 1)$$

The lines of $\text{PG}(3, 2)$

$\text{PG}(3, 2)$ has 35 1-subspaces:

$$L_0 = \begin{bmatrix} 1000 \\ 0100 \end{bmatrix} = \mathbf{Pl}(1, 0, 0, 0, 0, 0)$$

$$L_1 = \begin{bmatrix} 1000 \\ 0110 \end{bmatrix} = \mathbf{Pl}(1, 0, 1, 0, 0, 0)$$

$$L_2 = \begin{bmatrix} 1000 \\ 0101 \end{bmatrix} = \mathbf{Pl}(1, 0, 0, 0, 1, 0)$$

$$L_3 = \begin{bmatrix} 1000 \\ 0111 \end{bmatrix} = \mathbf{Pl}(1, 0, 1, 0, 1, 0)$$

$$L_4 = \begin{bmatrix} 1000 \\ 0010 \end{bmatrix} = \mathbf{Pl}(0, 0, 1, 0, 0, 0)$$

$$L_5 = \begin{bmatrix} 1000 \\ 0011 \end{bmatrix} = \mathbf{Pl}(0, 0, 1, 0, 1, 0)$$

$\vdots$

$$L_{34} = \begin{bmatrix} 0010 \\ 0001 \end{bmatrix} = \mathbf{Pl}(0, 1, 0, 0, 0, 0)$$

Lines sorted by Pluecker coordinates

$$0 = \mathbf{Pl}(1, 0, 0, 0, 0, 0) = L_0 = \begin{bmatrix} 1000 \\ 0100 \end{bmatrix}$$

$$1 = \mathbf{Pl}(0, 1, 0, 0, 0, 0) = L_{34} = \begin{bmatrix} 0010 \\ 0001 \end{bmatrix}$$

$$2 = \mathbf{Pl}(0, 0, 1, 0, 0, 0) = L_4 = \begin{bmatrix} 1000 \\ 0010 \end{bmatrix}$$

$$3 = \mathbf{Pl}(0, 0, 0, 1, 0, 0) = L_{30} = \begin{bmatrix} 0100 \\ 0001 \end{bmatrix}$$

$$4 = \mathbf{Pl}(0, 0, 0, 0, 1, 0) = L_6 = \begin{bmatrix} 1000 \\ 0001 \end{bmatrix}$$

$$5 = \mathbf{Pl}(0, 0, 0, 0, 0, 1) = L_{28} = \begin{bmatrix} 0100 \\ 0010 \end{bmatrix}$$

$\vdots$

$$34 = \mathbf{Pl}(0, 1, 1, 1, 1, 1) = L_{26} = \begin{bmatrix} 1101 \\ 0011 \end{bmatrix}$$

$\text{PG}(3, 2)$ has the following low weight Pluecker lines:

$$L_0 = \begin{bmatrix} 1000 \\ 0100 \end{bmatrix} = \mathbf{Pl}(1, 0, 0, 0, 0, 0)$$

$$L_4 = \begin{bmatrix} 1000 \\ 0010 \end{bmatrix} = \mathbf{Pl}(0, 0, 1, 0, 0, 0)$$

$$L_6 = \begin{bmatrix} 1000 \\ 0001 \end{bmatrix} = \mathbf{Pl}(0, 0, 0, 0, 1, 0)$$

$$\begin{aligned}
L_{28} &= \begin{bmatrix} 0100 \\ 0010 \end{bmatrix} = \mathbf{Pl}(0, 0, 0, 0, 0, 1) \\
L_{30} &= \begin{bmatrix} 0100 \\ 0001 \end{bmatrix} = \mathbf{Pl}(0, 0, 0, 1, 0, 0) \\
L_{34} &= \begin{bmatrix} 0010 \\ 0001 \end{bmatrix} = \mathbf{Pl}(0, 1, 0, 0, 0, 0)
\end{aligned}$$

The planes of $\text{PG}(3, 2)$

$\text{PG}(3, 2)$ has 15 2-subspaces:

$$\begin{aligned}
L_0 &= \begin{bmatrix} 1000 \\ 0100 \\ 0010 \end{bmatrix} \\
L_1 &= \begin{bmatrix} 1000 \\ 0100 \\ 0011 \end{bmatrix} \\
&\vdots \\
L_{14} &= \begin{bmatrix} 0100 \\ 0010 \\ 0001 \end{bmatrix}
\end{aligned}$$

The polynomial rings associated with $\text{PG}(3, 2)$

h	monomial	vector
0	X_0	$(1, 0, 0, 0)$
1	X_1	$(0, 1, 0, 0)$
2	X_2	$(0, 0, 1, 0)$
3	X_3	$(0, 0, 0, 1)$

Ranking Points in Small Projective Spaces		
PG(2, 2)	PG(3, 2)	PG(2, 3)
$P_0 = \mathbf{P}(1, 0, 0)$ $P_1 = \mathbf{P}(0, 1, 0)$ $P_2 = \mathbf{P}(0, 0, 1)$ $P_3 = \mathbf{P}(1, 1, 1)$ $P_4 = \mathbf{P}(1, 1, 0)$ $P_5 = \mathbf{P}(1, 0, 1)$ $P_6 = \mathbf{P}(0, 1, 1)$	$P_0 = \mathbf{P}(1, 0, 0, 0)$ $P_1 = \mathbf{P}(0, 1, 0, 0)$ $P_2 = \mathbf{P}(0, 0, 1, 0)$ $P_3 = \mathbf{P}(0, 0, 0, 1)$ $P_4 = \mathbf{P}(1, 1, 1, 1)$ $P_5 = \mathbf{P}(1, 1, 0, 0)$ $P_6 = \mathbf{P}(1, 0, 1, 0)$ $P_7 = \mathbf{P}(0, 1, 1, 0)$ $P_8 = \mathbf{P}(1, 1, 1, 0)$ $P_9 = \mathbf{P}(1, 0, 0, 1)$ $P_{10} = \mathbf{P}(0, 1, 0, 1)$ $P_{11} = \mathbf{P}(1, 1, 0, 1)$ $P_{12} = \mathbf{P}(0, 0, 1, 1)$ $P_{13} = \mathbf{P}(1, 0, 1, 1)$ $P_{14} = \mathbf{P}(0, 1, 1, 1)$	$P_0 = \mathbf{P}(1, 0, 0)$ $P_1 = \mathbf{P}(0, 1, 0)$ $P_2 = \mathbf{P}(0, 0, 1)$ $P_3 = \mathbf{P}(1, 1, 1)$ $P_4 = \mathbf{P}(1, 1, 0)$ $P_5 = \mathbf{P}(2, 1, 0)$ $P_6 = \mathbf{P}(1, 0, 1)$ $P_7 = \mathbf{P}(2, 0, 1)$ $P_8 = \mathbf{P}(0, 1, 1)$ $P_9 = \mathbf{P}(2, 1, 1)$ $P_{10} = \mathbf{P}(0, 2, 1)$ $P_{11} = \mathbf{P}(1, 2, 1)$ $P_{12} = \mathbf{P}(2, 2, 1)$

Table 4.2: Ranking Points in Small Projective Spaces

4.2 Indexing Points and Lines

The enumerator for points establishes a bijection between the set of points and the integers on the interval $[0, \theta_n(q) - 1]$, where

$$\theta_n(q) = \frac{q^{n+1} - 1}{q - 1}.$$

In order to facilitate the bijection, Orbiter enumerates representative vectors for the one-dimensional subspaces. The conditions on the vectors are summarized below:

1. The vector is not the zero vector.
2. The rightmost nonzero entry in the vector is one. If it is not, we normalize the vector so that the rightmost nonzero vector is indeed one. This operation does not change the projective point which is associated with the vector.

The second condition ensures that we list each projective point exactly once. We require two functions, RANK and UNRANK. The function RANK takes a vector $\mathbf{x} \in \mathbb{F}_q^n$, not zero, and maps it to the element in $\mathbb{Z}_N$ representing the projective point $\mathbf{P}(\mathbf{x})$. A frame in $\text{PG}(n, q)$ is a set of $n + 2$ points, no $n + 1$ in a hyperplane. We assume that the coordinates of a vector are indexed by the elements of $\mathbb{Z}_n$. Also, we let $\mathbf{e}_i$ be the i -th unit vector. A frame for $\text{PG}(n, q)$ is

$$\mathbf{e}_0, \dots, \mathbf{e}_{n-1}, \mathbf{e}_0 + \dots + \mathbf{e}_{n-1}.$$

This is the *standard frame*. We start the labeling of points with the standard frame. After these $n + 2$ points, we list the remaining points in lexicographic ordering (utilizing right-normalized representative). The points in the small spaces $\text{PG}(2, 2)$, $\text{PG}(3, 2)$ and $\text{PG}(2, 3)$ are listed by their ranks in Table 4.2.

It is possible to request ranks of specific points given by means of homogeneous coordinate vectors. Conversely, given a point rank, the coordinate vector can be produced. The following command computes the

rank of

$$\mathbf{P}(3, 3, 1) = \mathbf{P}(\omega + 1, \omega + 1, 1)$$

in $\text{PG}(2, 4)$:

Code Snippet 92

```

1   PG_2.4.rank_point:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define v -vector -dense "3,3,1" -format 1 -end \
4   ▷ ▷ -define F -finite_field -q 4 -end \
5   ▷ ▷ -with F -do -finite_field_activity \
6   ▷ ▷ ▷ -rank_point_in_PG v \
7   ▷ ▷ -end
8
```

The rank turns out to be 20. Conversely, running

Code Snippet 93

```

1   PG_2.4.unrank_point:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define v -vector -dense "20" -end \
4   ▷ ▷ -define F -finite_field -q 4 -end \
5   ▷ ▷ -with F -do -finite_field_activity \
6   ▷ ▷ ▷ -unrank_point_in_PG 2 v \
7   ▷ ▷ -end
8
```

shows that the point with rank 20 is $\mathbf{P}(3, 3, 1)$.

Next we show how to print an arbitrary set of points. In the example, we print all 21 points of $\text{PG}(2, 4)$. At first, we create a vector containing all point ranks, i.e., all integers from 0 to 20. After that, we invoke the `-print_points` command, which is a projective space activity. In this case, because the field is an extension field, the list of points is printed twice. The first list shows the points and the coordinates in extension field notation. The second list shows the points with coordinates in numeric form. In the numeric form, a field element in $\mathbb{F}_q$ is encoded as integer in the interval $[0, q - 1]$.

Code Snippet 94

```

1   PG_2.4.print_points:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define Pts -vector \
4   ▷ ▷ ▷ -loop 0 21 1 \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 4 -end \
7   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
8   ▷ ▷ -with P -do \
9   ▷ ▷ -projective_space_activity \
10  ▷ ▷ ▷ -print_points Pts \
11  ▷ ▷ -end
12  ▷ $(PDFLATEX) PG_2.4.points.tex
13  ▷ $(OPEN) PG_2.4.points.pdf
14
```

Ranking Lines in Small Projective Spaces	
PG(2,2)	PG(2,3)
$L_0 = \mathbf{L} \begin{bmatrix} 100 \\ 010 \end{bmatrix} \{0, 1, 4\}$ $L_1 = \mathbf{L} \begin{bmatrix} 100 \\ 011 \end{bmatrix} \{0, 3, 6\}$ $L_2 = \mathbf{L} \begin{bmatrix} 100 \\ 001 \end{bmatrix} \{0, 2, 5\}$ $L_3 = \mathbf{L} \begin{bmatrix} 101 \\ 010 \end{bmatrix} \{1, 3, 5\}$ $L_4 = \mathbf{L} \begin{bmatrix} 101 \\ 011 \end{bmatrix} \{4, 5, 6\}$ $L_5 = \mathbf{L} \begin{bmatrix} 110 \\ 001 \end{bmatrix} \{2, 3, 4\}$ $L_6 = \mathbf{L} \begin{bmatrix} 010 \\ 001 \end{bmatrix} \{1, 2, 6\}$	$L_0 = \mathbf{L} \begin{bmatrix} 100 \\ 010 \end{bmatrix} \{0, 1, 4, 5\}$ $L_1 = \mathbf{L} \begin{bmatrix} 100 \\ 011 \end{bmatrix} \{0, 3, 8, 9\}$ $L_2 = \mathbf{L} \begin{bmatrix} 100 \\ 012 \end{bmatrix} \{0, 10, 11, 12\}$ $L_3 = \mathbf{L} \begin{bmatrix} 100 \\ 001 \end{bmatrix} \{0, 2, 6, 7\}$ $L_4 = \mathbf{L} \begin{bmatrix} 101 \\ 010 \end{bmatrix} \{1, 3, 6, 11\}$ $L_5 = \mathbf{L} \begin{bmatrix} 101 \\ 011 \end{bmatrix} \{5, 6, 8, 12\}$ $L_6 = \mathbf{L} \begin{bmatrix} 101 \\ 012 \end{bmatrix} \{4, 6, 9, 10\}$ $L_7 = \mathbf{L} \begin{bmatrix} 110 \\ 001 \end{bmatrix} \{2, 3, 4, 12\}$ $L_8 = \mathbf{L} \begin{bmatrix} 102 \\ 010 \end{bmatrix} \{1, 7, 9, 12\}$ $L_9 = \mathbf{L} \begin{bmatrix} 102 \\ 011 \end{bmatrix} \{4, 7, 8, 11\}$ $L_{10} = \mathbf{L} \begin{bmatrix} 102 \\ 012 \end{bmatrix} \{3, 5, 7, 10\}$ $L_{11} = \mathbf{L} \begin{bmatrix} 120 \\ 001 \end{bmatrix} \{2, 5, 9, 11\}$ $L_{12} = \mathbf{L} \begin{bmatrix} 010 \\ 001 \end{bmatrix} \{1, 2, 8, 10\}$

Table 4.3: Ranking Lines in Small Projective Spaces

Line ranking will be discussed next. The lines in the small spaces PG(2,2) and PG(2,3) are listed by their ranks in Table 4.3.

The command

Code Snippet 95

```

1 PG_2.2_rank_lines:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define v -vector -format 4 \
4   ▷ ▷ -dense "1,0,0, 0,1,0, 1,0,0, 0,0,1, 1,1,1, 0,1,0, 1,1,1, 0,0,1" \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 2 -end \
7   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \

```

```

8      ▷ ▷ -with P -do \
9      ▷ ▷ -projective_space_activity \
10     ▷ ▷ ▷ -rank_lines_in_PG v \
11     ▷ ▷ -end \
12

```

computes the Orbiter ranks of 4 lines in $PG(2,2)$. The lines are given by two points each. The data is formatted as a matrix with 4 rows of size 6. Each row represents one line, with the coordinates of the two points written one after the other.

The command

Code Snippet 96

```

1      PG_2_2_unrank_lines:
2      ▷ $(ORBITER) -v 2 \
3      ▷ ▷ -define v -vector \
4      ▷ ▷ -dense "0,1,2,3,4,5,6" \
5      ▷ ▷ -end \
6      ▷ ▷ -define F -finite_field -q 2 -end \
7      ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
8      ▷ ▷ -with P -do \
9      ▷ ▷ -projective_space_activity \
10     ▷ ▷ ▷ -unrank_lines_in_PG v \
11     ▷ ▷ -end \
12

```

lists the 7 lines of $PG(2,2)$ by their Orbiter ranks from 0 to 6.

4.3 Incidence Matrices

The projective spaces $\text{PG}(2, q)$ are examples of a more general structure called projective planes. The $\text{PG}(2, F)$, F a field, are distinguished in the class of projective planes by the fact that the theorem of Desargues always holds. They are called the desarguesian projective planes. Other classes of projective planes are the translation planes, see Section 15.2.

The points in the dearguesian projective plane $\text{PG}(2, q)$ have the coordinates $\mathbf{P}(x, y, z)$, with $x, y, z \in \mathbb{F}_q$, not all three zero. We distinguish one line called the line at infinity. The points not on this line form an affine plane $\text{AG}(2, q)$.

The command

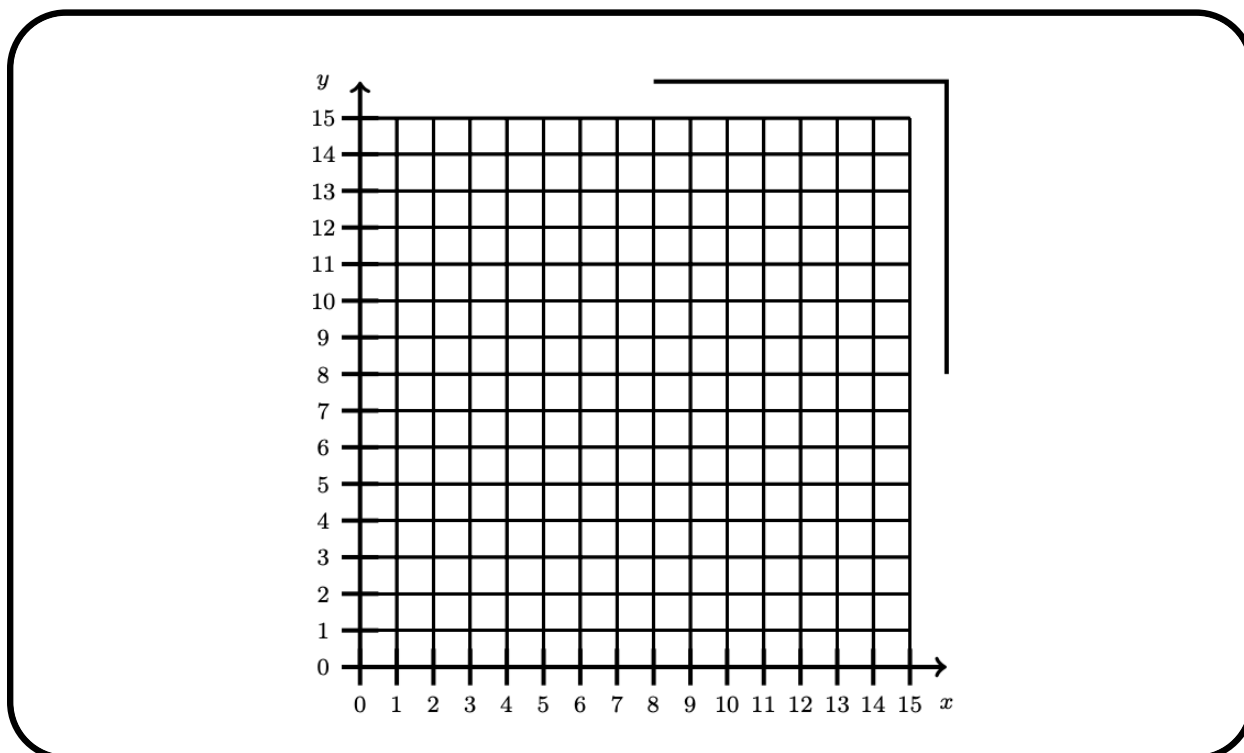
Code Snippet 97

```

1   PG_2_16:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 16 -compute_related_fields -end \
7   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
8   ▷ ▷ -with P -do -projective_space_activity \
9   ▷ ▷ ▷ -cheat_sheet Do \
10  ▷ ▷ -end
11  ▷ $(PDFLATEX) PG_2_16.tex
12  ▷ $(OPEN) PG_2_16.pdf
13

```

produces the drawing of $\text{PG}(2, 16)$ shown below



The `-nodes_empty` command is used to suppress the drawing of the nodes. The `-xin 20000` and `-yin 20000` options double the input coordinate system (recall from Table 19.1 that the default values are 10000), which has the effect that the text appears smaller relative to the grid.

It is possible to export the incidence matrix of a projective space to a file. The following example creates $PG(2,8)$ and exports the incidence matrix to a csv file. After that, a graphical representation is produced.

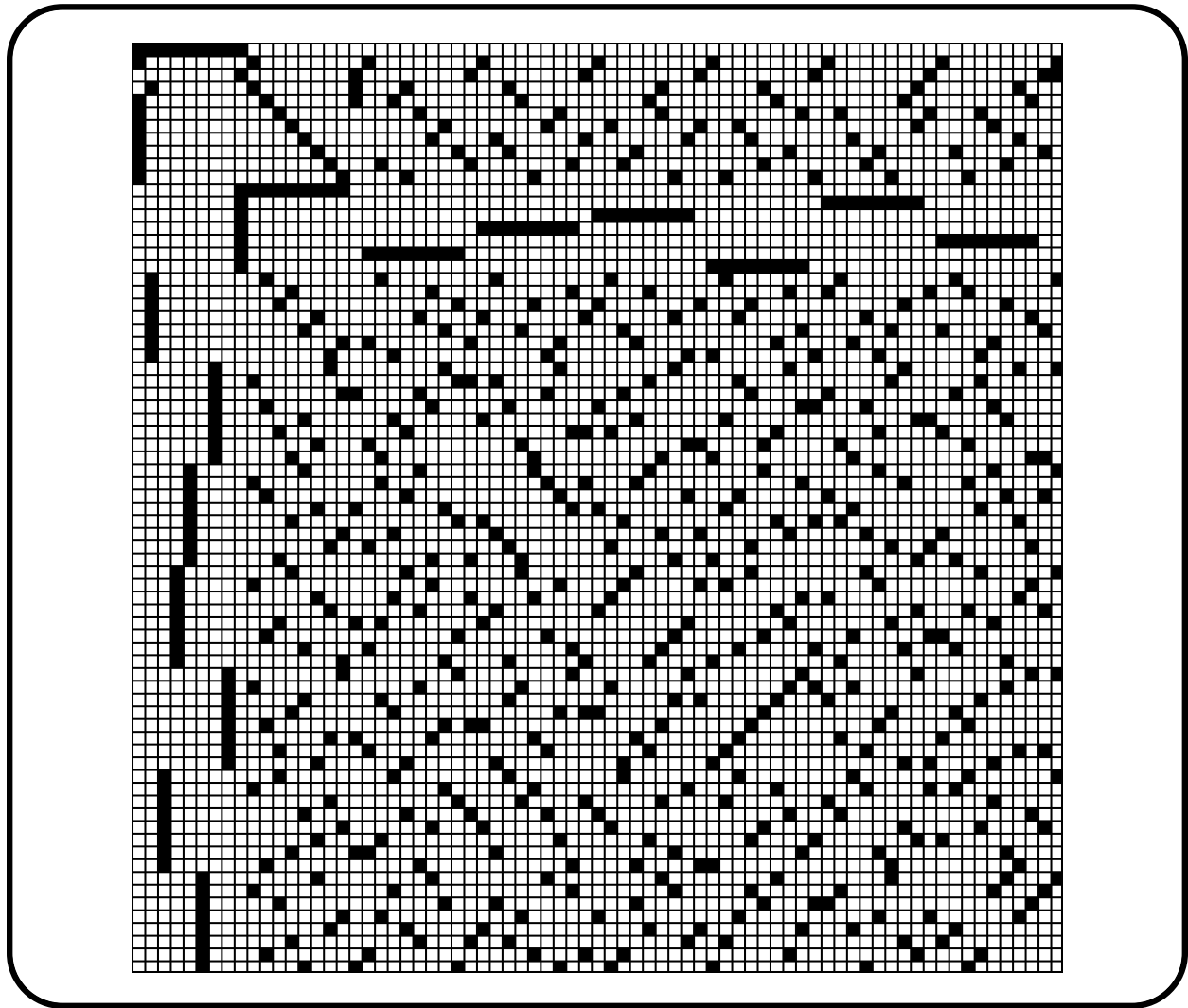
Code Snippet 98

```

1   PG_2_8_incidence_matrix:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 8 -end \
4   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5   ▷ ▷ -with P -do -projective_space_activity \
6   ▷ ▷ ▷ -export_point_line_incidence_matrix \
7   ▷ ▷ -end
8   ▷ $(ORBITER) -v 2 \
9   ▷ ▷ -define all_one -vector -repeat 1 73 -end \
10  ▷ ▷ -draw_matrix \
11  ▷ ▷ ▷ -input_csv_file PG_n2_q8_incidence_matrix.csv \
12  ▷ ▷ ▷ -box_width 20 -bit_depth 8 \
13  ▷ ▷ ▷ -partition 3 \
14  ▷ ▷ ▷ ▷ all_one all_one \
15  ▷ ▷ -end
16  ▷ $(OPEN) PG_n2_q8_incidence_matrix_draw.bmp
17  ▷ magick PG_n2_q8_incidence_matrix_draw.bmp PG_n2_q8_incidence_matrix_draw.png
18

```

The incidence matrix is shown below



The rows and columns correspond to points and lines, respectively, and are ordered according to their respective Orbiter ranks.

Projective spaces admit a cyclic group action on points and hyperplanes. Such a group is often called a Singer cycle. It is generated from a projectivity defined by the companion matrix of an irreducible polynomial. Let us look at an example. The following command creates a Singer cycle of $PG(2, 4)$

Code Snippet 99

```

1   PG_2.4_with_decomposition:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 4 -end \
4   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5   ▷ ▷ -with P -do -projective_space_activity \
6   ▷ ▷ ▷ -decomposition_by_element_PG \
7   ▷ ▷ ▷ 1 "0,1,0, 0,0,1, 2,1,1, 0" \
8   ▷ ▷ ▷ "PG.2.4_singer" \
9   ▷ ▷ -end
10  ▷ $(PDFLATEX) PG_2.4_singer.tex
11  ▷ $(OPEN) PG_2.4_singer.pdf

```

12

The output is shown below:

Considering the cyclic group generated by

$$\begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ \omega & 1 & 1 \end{bmatrix}_0 = \begin{bmatrix} 010 \\ 001 \\ 211 \end{bmatrix}_0$$

The group is transitive on points and on lines.

Orbits on points:

There are 1 orbits, the orbit lengths are 21

Orbits on lines:

There are 1 orbits, the orbit lengths are 21

Fixed points:

Fixed lines:

Row scheme:

$$\begin{array}{c|c} \rightarrow & 21 \\ \hline 21 & 5 \end{array}$$

Column scheme:

$$\begin{array}{c|c} \downarrow & 21 \\ \hline 21 & 5 \end{array}$$

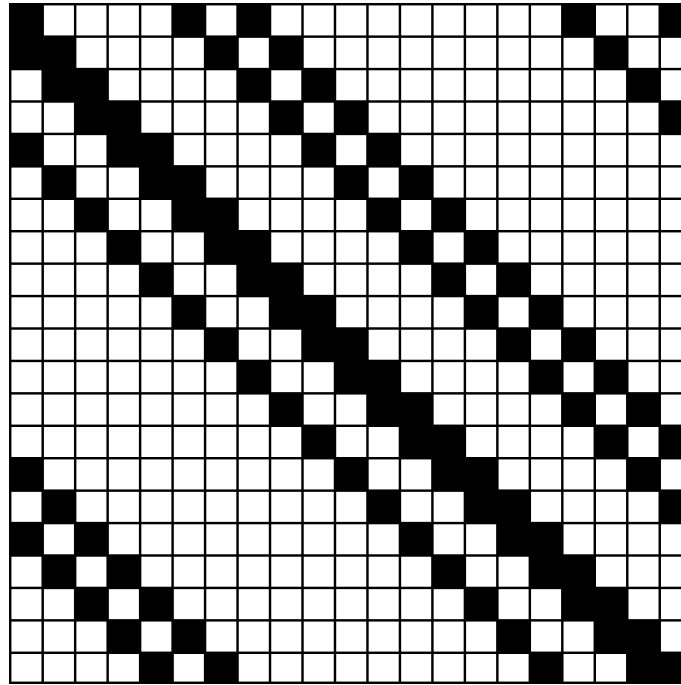
The command produces a csv file containing the cyclic incidence matrix, which can be drawn using the following command:

Code Snippet 100

```

1 PG_2.4_incma_cyclic:
2 ▷ $(ORBITER) -v 2 \
3 ▷ ▷ -list_arguments \
4 ▷ ▷ -define R -vector -repeat 1 21 -end \
5 ▷ ▷ -define C -vector -repeat 1 21 -end \
6 ▷ ▷ -draw_matrix \
7 ▷ ▷ ▷ -input_csv_file PG_2.4_singer_incma_cyclic.csv \
8 ▷ ▷ ▷ -box_width 40 -bit_depth 24 \
9 ▷ ▷ ▷ -partition 3 R C \
10 ▷ ▷ -end
11 ▷ $(OPEN) PG_2.4_singer_incma_cyclic_draw.bmp
12 ▷ magick PG_2.4_singer_incma_cyclic_draw.bmp PG_2.4_singer_incma_cyclic_draw.png
13
```

The cyclic incidence matrix is



If the number of points is not a prime power, the group acts imprimitively. By considering various subgroups of the Singer group, tactical decompositions are created. For instance, for $\text{PG}(2, 4)$, with 21 points, we can consider a subgroup the Singer cycle of order 3, which induces a partition with 7 classes of size 3 on both points and lines:

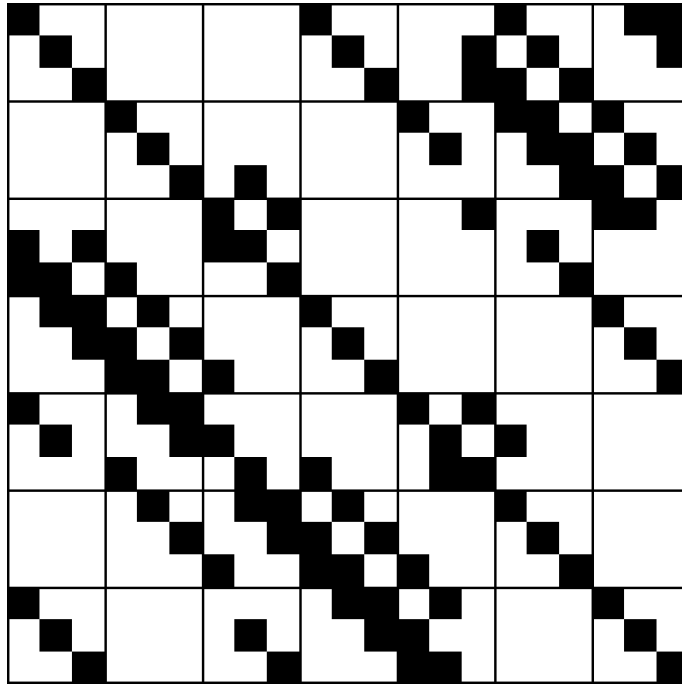
Code Snippet 101

```

1   PG_2_4_incma_singer_sub_3:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -list_arguments \
4   ▷ ▷ -define R -vector -repeat 3 7 -end \
5   ▷ ▷ -define C -vector -repeat 3 7 -end \
6   ▷ ▷ -draw_matrix \
7   ▷ ▷ ▷ -input_csv_file PG_2_4_singer_incma_subgroup_index_3.csv \
8   ▷ ▷ ▷ -box_width 40 -bit_depth 24 \
9   ▷ ▷ ▷ -partition 3 R C \
10  ▷ ▷ -end
11  ▷ $(OPEN) PG_2_4_singer_incma_subgroup_index_3_draw.bmp
12  ▷ magick PG_2_4_singer_incma_subgroup_index_3_draw.bmp PG_2_4_singer_incma_subgro
   up_index_3_draw.png
13

```

The tactical decomposition of the incidence matrix is



4.4 Advanced Topics

The Orbiter commands associated with projective space objects are summarized in Tables 4.4-4.7.

Table 4.8 lists Orbiter global commands related to projective geometries. These commands do not need an object of type projective space in order to be invoked.

Suppose we want to study the fix structure of a collineation in projective space. Suppose we want to do so for the Baer collineation

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1.$$

It fixes a subgeometry PG(3,2). The command

Code Snippet 102

```
1    fixed_objects_2A:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 4 -end \
4    ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5    ▷ ▷ -with P -do \
6    ▷ ▷ -projective_space_activity \
7    ▷ ▷ ▷ -report_fixed_objects \
8    ▷ ▷ ▷ ▷ "1,0,0,0, 0,1,0,0, 0,0,1,0, 0,0,0,1, 1" \
9    ▷ ▷ ▷ ▷ fix_structure_2A.tex \
10   ▷ ▷ -end
11   ▷ $(PDFLATEX) fix_structure_2A.tex
12   ▷ $(OPEN) fix_structure_2A.pdf
13
```

can be used. The output is shown below. There are 15 fixed points, 35 fixed lines, and 15 fixed planes. These objects form a Baer subgeometry.

The element

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1$$

has the following fixed objects:

There are 15 / 85 fixed points, they are:

```
0 / 15 = 0 : ( 1, 0, 0, 0 )
1 / 15 = 1 : ( 0, 1, 0, 0 )
2 / 15 = 2 : ( 0, 0, 1, 0 )
3 / 15 = 3 : ( 0, 0, 0, 1 )
4 / 15 = 4 : ( 1, 1, 1, 1 )
5 / 15 = 5 : ( 1, 1, 0, 0 )
6 / 15 = 8 : ( 1, 0, 1, 0 )
7 / 15 = 11 : ( 0, 1, 1, 0 )
8 / 15 = 12 : ( 1, 1, 1, 0 )
```

Projective Space Activities (Part 1)		
Command	Arguments	Purpose
-cheat_sheet	options	Produce a cheat sheet for $\text{PG}(n, q)$. Considers the given drawing options object for drawing options.
-print_points	label	Print the given set of points.
-export_point_line_incidence_matrix		Create a csv file of the point line incidence matrix.
-export_restricted_point_line_incidence_matrix	points lines	Create a csv file of the incidence matrix between the given set of points (rows) and lines (columns).
-export_cubic_surface_line_vs_line_incidence_matrix		See Section 9.1.
-export_cubic_surface_line_tritangent_plane_incidence_matrix		See Section 9.1.
-export_double_sixes		See Section 9.1.
-table_of_cubic_surfaces_compute_properties	fname q_0 col-offset	Computes the properties of cubic surfaces over a different field.
-cubic_surface_properties_analyze	fname q_0	Explores the properties of cubic surfaces over a different field.
-canonical_form_of_code	label m n matrix	Compute the automorphism group of a linear code using Nauty. See Section 11.
-map	eqn parameters	Evaluate the equation using the given parameters
-affine_map	R eqn parameters	Evaluate an affine map given by the equation using the given parameters over the ring R.
-projective_variety	R eqn parameters	Create a projective variety defined by the given equation using the ring R .
-analyze_del_Pezzo_surface	label parameters	
-decomposition_by_element_PG	power elt fname	Analyze the orbit structure of the cyclic group generated by the given element in the action on $\text{PG}(n, q)$. The command also produces a latex report.
-decomposition_by_subgroup	label descr	Analyze the orbit structure of the subgroup H in the action on $\text{PG}(n, q)$. The subgroup must be a linear group, and the description of H must follow the commands from Section 5.2. Produce a latex report.
-table_of_quartic_curves		Export the classification of quartic curves to a csv file.
-table_of_cubic_surfaces		Export the classification of cubic surfaces to a csv file.

Table 4.4: Projective Space Activities (Part 1)

Projective Space Activities (Part 2)		
Command	Arguments	Purpose
-sweep	options surface	Perform a parameter sweep for all cubic surface, given a surface description. Options can be given to direct the sweep.
-set_stabilizer	k fname-mask N col-label fname-out	Compute canonical form of sets using the substructure algorithm.
-conic_type	t set	Compute the conic type of the given set (given by its label). Record intersections of size $\geq t$ only.
-arc_with_given_set_as_ s_lines_after_dualizing	sz d $d_{\min}$ s	Finds arcs with the given set as s -lines.
-arc_with_two_given_sets_ of_lines_after_dualizing	sz d $d_{\min}$ s t T	Finds arcs with the two given sets as s -lines and t -lines, respectively.
-arc_with_three_given_sets_ of_lines_after_dualizing	sz d $d_{\min}$ s t T u U	Finds arcs with the three given sets as s -lines and t -lines and u -lines, respectively.
-dualize_hyperplanes_to_points		Turns ranks of hyperplanes into ranks of points.
-dualize_points_to_hyperplanes		Turns ranks of points into ranks of hyperplanes.
-dualize_rank_k_subspaces	k	Turns ranks of k -subspaces into ranks of $n-k$ subspaces.

Table 4.5: Projective Space Activities (Part 2)

Projective Space Activities (Part 3)		
Command	Arguments	Purpose
-lines_on_point_but_within_a_plane	pt-rk plane-rk	Compute the lines through a given point contained in a given plane.
-union_of_lines	lines	Compute the union of points on the given lines. Duplicates will be removed.
-rank_lines_in_PG	M	Rank lines. The generating systems for the lines are given in rows of the matrix M .
-unrank_lines_in_PG	v	Unrank lines. The ranks are given in the vector v .
-points_on_lines	L	Create the ranks of points on the given line L .
-move_two_lines_in_hyperplane_stabilizer	$l1\ l2\ m1\ m2$	Find the unique transvection fixing the hyperplane at infinity moving $l1$ and $l2$ to $m1$ and $m2$.
-move_two_lines_in_hyperplane_stabilizer_text	$l1\ l2\ m1\ m2$	Find the unique transvection fixing the hyperplane at infinity moving $l1$ and $l2$ to $m1$ and $m2$.
-planes_through_line	l	Find all planes through the line l .
-restricted_incidence_matrix	$t1\ t2\ R\ C$ fname	Compute the restricted incidence matrix between objects R of type $t1$ and objects C of type $t2$. Write output to the given file in csv format.
-plane_intersection_type	t input	Compute the plane intersection type of the lines given in input. Use a threshold of t to filter planes. Only planes which contain at least t of the given lines are considered.
-plane_intersection_type_of_klein_image	t input	Compute the plane intersection type of the Klein image of the lines given in input. Use a threshold of t to filter planes. Only planes which intersect in at least t points on the Klein quadric are considered.
-report_Grassmannian	k	Produce a report on the Grassmannian of k -dimensional subspaces.
-report_fixed_objects	data fname	Produce a report about the fixed points of the group element given by data in the action on objects of various types of the projective space. Writes a latex report using the given file name.
-evaluation_matrix	ring	Computes the matrix whose (i, j) entry is the value of the i -th monomial of the given ring on (a fixed representative of) the j -th projective point.

Table 4.6: Projective Space Activities (Part 3)

Projective Space Activities (Part 4)		
Command	Arguments	Purpose
-classify_surfaces_through_arcs_and_two_lines		Classify cubic surfaces using the approach of six-arcs and two skew lines. See Section 9.4.
-classify_surfaces_through_arcs_and_trihedral_pairs	options	See Section 9.4.
-test_nb_Eckardt_points	e	Restrict to e Eckardt points. See Section 9.4. To be used in conjunction with -classify_surfaces_through_arcs_and_trihedral_pairs.
-trihedra1_control	poset-control	For -classify_surfaces_through_arcs_and_trihedral_pairs
-trihedra2_control	poset-control	For -classify_surfaces_through_arcs_and_trihedral_pairs
-control_six_arcs	poset-control	For -classify_surfaces_through_arcs_and_trihedral_pairs

Table 4.7: Projective Space Activities (Part 4)

Global Commands for Projective Geometry		
Command	Arguments	Purpose
-create_points_on_quartic	ϵ	Create a table of points on a specific quartic curve. Consecutive points are no more than ϵ apart.
-create_points_on_parabola	$\epsilon \ a \ b \ c$	Create a table of points on the parabola $y = ax^2 + bx + c$. Consecutive points are no more than ϵ apart.
-smooth_curve	$\epsilon \ N \ b \ t_{\min} \ t_{\max}$ function	Create at least N points on a continuous curve given by "function". Consecutive points are no more than ϵ apart. The function must be in terms of a parameter t . The values of t are taken from the interval $[t_{\min}, t_{\max}]$.
-make_table_of_surfaces		Produce a latex table summarizing the surfaces in the Orbiter catalogue.
-create_surface_reports	field-orders	Produce reports for all surfaces in the Orbiter catalogue over the give field orders.
-create_surface_atlas	$q_{\max}$	Produce reports for all surfaces in the Orbiter catalogue for field orders $q \leq q_{\max}$.
-create_dickson_atlas		Produce reports of Dickson surfaces.

Table 4.8: Global Commands for Projective Geometry

$$9 / 15 = 23 : (1, 0, 0, 1)$$

$$10 / 15 = 26 : (0, 1, 0, 1)$$

$$11 / 15 = 27 : (1, 1, 0, 1)$$

$$12 / 15 = 38 : (0, 0, 1, 1)$$

$$13 / 15 = 39 : (1, 0, 1, 1)$$

$$14 / 15 = 42 : (0, 1, 1, 1)$$

There are 35 / 357 fixed subspaces of rank 2, they are:

$$\begin{aligned}
 0 / 35 = 0 : & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \\
 1 / 35 = 1 : & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{bmatrix} \\
 2 / 35 = 4 : & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} \\
 3 / 35 = 5 : & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix} \\
 4 / 35 = 16 : & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \\
 5 / 35 = 17 : & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} \\
 6 / 35 = 20 : & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
 7 / 35 = 21 : & \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \\
 8 / 35 = 22 : & \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \end{bmatrix} \\
 9 / 35 = 25 : & \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} \\
 10 / 35 = 26 : & \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix} \\
 11 / 35 = 37 : & \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \\
 12 / 35 = 38 : & \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} \\
 13 / 35 = 41 : & \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
 14 / 35 = 84 : & \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix} \\
 15 / 35 = 85 : & \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} \\
 16 / 35 = 88 : & \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \end{bmatrix}
 \end{aligned}$$

$$\begin{aligned}
17 / 35 = 89 : & \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix} \\
18 / 35 = 100 : & \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \\
19 / 35 = 101 : & \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \\
20 / 35 = 104 : & \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
21 / 35 = 105 : & \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix} \\
22 / 35 = 106 : & \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} \\
23 / 35 = 109 : & \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \end{bmatrix} \\
24 / 35 = 110 : & \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix} \\
25 / 35 = 121 : & \begin{bmatrix} 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \\
26 / 35 = 122 : & \begin{bmatrix} 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \\
27 / 35 = 125 : & \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
28 / 35 = 336 : & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \\
29 / 35 = 337 : & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} \\
30 / 35 = 340 : & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
31 / 35 = 341 : & \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \\
32 / 35 = 342 : & \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \\
33 / 35 = 345 : & \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
34 / 35 = 356 : & \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}
\end{aligned}$$

There are 15 / 85 fixed subspaces of rank 3, they are:

$$0 / 15 = 0 : \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

[illegible]

Suppose we want to study the fix structure of a collineation group in projective space. Suppose we want to do so for the elementary abelian group of order 4 generated by the two elements

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}_0, \quad \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}_0.$$

It fixes no points, 5 lines, and no plane. The following command sequence can be used. It uses two makefile variables: First, we set a label for the group:

Code Snippet 103

```
1 H1_LABEL="H1"
2
```

Now we define the subgroup:

Code Snippet 104

```
1 H1_GENS=PGGL 4 4 \
2 > -subgroup_by_generators "H1" 4 2 \
3 > "1,0,0,0,1,1,0,0,0,0,1,0,0,0,1,1,0, \
4 > 1,0,0,0,0,1,0,0,1,0,1,0,0,1,0,1,0" \
5 > -end
6
```

And finally the command itself:

Code Snippet 105

```
1 fix_structure_G1:
2 > $(ORBITER) -v 2 \
3 > > -define F -finite_field -q 4 -end \
4 > > -define P -projective_space -n 3 -field F -v 0 -end \
5 > > -with P -do \
6 > > -projective_space_activity \
7 > > > -decomposition_by_subgroup \
8 > > > > $(H1_LABEL) $(H1_GENS) \
9 > > > -end
10 > $(PDFLATEX) PGGL_4_4_Subgroup_$(H1_LABEL)_4_decomp.tex
11 > $(OPEN) PGGL_4_4_Subgroup_$(H1_LABEL)_4_decomp.pdf
12
```

The decomposition based on orbit lengths is shown below:

Decomposition based on orbit lengths:
Row scheme:

$\rightarrow$	3	2	48	16	16	144	128
1	3	2	0	16	0	0	0
12	1	0	4	0	4	12	0
8	0	1	0	0	4	0	16
64	0	0	3	1	0	9	8

Column scheme:

$\downarrow$	3	2	48	16	16	144	128
1	1	1	0	1	0	0	0
12	4	0	1	0	3	1	0
8	0	4	0	0	2	0	1
64	0	0	4	4	0	4	4

The decomposition of orbits is shown as well, but it is too large to be shown.

Suppose we are looking for a projectivity of $\text{PG}(3, 16)$ fixing the plane $v(X_3)$ pointwise and mapping a pair of skew lines not in that plane to another pair of skew lines not in that plane. For instance, suppose we want to map

$$M_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \mapsto N_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_2 = \begin{bmatrix} 1 & 1 & 0 & \delta \\ 0 & 0 & 1 & 0 \end{bmatrix} \mapsto N_2 = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

The command

Code Snippet 106

```

1    trans:
2    ▷ $(ORBITER) -v 5 \
3    ▷ ▷ -define F -finite_field -q 16 -end \
4    ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5    ▷ ▷ -with P -do \
6    ▷ ▷ -projective_space_activity \
7    ▷ ▷ ▷ -move_two_lines_in_hyperplane_stabilizer_text \
8    ▷ ▷ ▷ ▷ "1,0,0,0, 0,0,0,1" "1,1,0,2, 0,0,1,0" \
9    ▷ ▷ ▷ ▷ "1,0,0,0, 0,0,0,1" "0,1,0,1, 0,0,1,0" \
10   ▷ ▷ -end
11
```

computes a projectivity (transvection) to do so:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ \delta^{14} & 0 & 0 & \delta^{14} \end{bmatrix}$$

Here, δ is the primitive element in the built-in field $\mathbb{F}_{16}$, satisfying $\delta^4 = \delta^3 + 1$.

4.5 Geometric Objects

Orbiter can create objects in projective space. To do so, define an object of type `-geometric_object`. The definition of a geometric object requires a projective geometry object. For this reason, the definition requires an extra argument, which is the label of a previously created projective geometry object. After that, one of the commands shown in Tables 4.9 and 4.10 can be issued.

The following command creates an elliptic quadric ovoid on $PG(3, 8)$:

Code Snippet 107

```
1    elliptic_quadric_ovoid_q8:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 8 -end \
4    ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5    ▷ ▷ -define Geo -geometric_object P \
6    ▷ ▷ ▷ -elliptic_quadric_ovoid \
7    ▷ ▷ -end \
8    ▷ ▷ -define C -combinatorial_object \
9    ▷ ▷ ▷ -label ovoid ovoid \
10   ▷ ▷ ▷ -geometric_object Geo \
11   ▷ ▷ -end
12
```

The next command creates the Suzuki-Tits ovoid in $PG(3, 8)$:

Code Snippet 108

```
1    ovoid_ST_q8:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 8 -end \
4    ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5    ▷ ▷ -define Geo -geometric_object P \
6    ▷ ▷ ▷ -ovoid_ST \
7    ▷ ▷ -end \
8    ▷ ▷ -define C -combinatorial_object \
9    ▷ ▷ ▷ -label ovoid_ST ovoid\_ST \
10   ▷ ▷ ▷ -geometric_object Geo \
11   ▷ ▷ -end
12
```

The following command creates the Baer subplane of $PG(2, 4)$:

Code Snippet 109

```
1    Baer_PG_2.4:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 4 -end \
4    ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5    ▷ ▷ -define Geo -geometric_object P \
6    ▷ ▷ ▷ -Baer_substructure \
7    ▷ ▷ -end \
8    ▷ ▷ -define C -combinatorial_object \
9    ▷ ▷ ▷ -label Baer Baer \
```

Geometric Objects (Part 1)		
Command	Arguments	Purpose
-hyperoval		To create a hyperoval
-subiaco_oval	f_short	Create the Subiaco oval
-subiaco_hyperoval		Create the Subiaco hyperoval
-adelaide_hyperoval		Create the Adelaide hyperoval
-translation	i	Create the translation hyperoval with exponent i
-Segre		Create the Segre hyperoval
-Payne		Create the Payne hyperoval
-Cherowitzo		Create the Cherowitzo hyperoval
-O'Keefe_Penttila		Create the O'Keefe, Penttila hyperoval
-BLT_database	k	Create the k th BLT-set of order q from the database ($k = 0, 1, \dots$)
-elliptic_quadric_ovoid		Create an elliptic quadric ovoid in $\text{PG}(3, q)$.
-ovoid_ST		Create the Suzuki Tits ovoid in $\text{PG}(3, q)$. Here, $q = 2^{2r+1}$.
-Baer		Create the (standard) Baer subgeometry
-orthogonal	ϵ	Create the $Q^\epsilon(n, q)$ quadric
-hermitian		Create the Hermitian variety given by $\sum_{i=0}^n X_i^{\sqrt{q}+1} = 0$
-cuspidal_cubic		Create the cuspidal cubic (s^3, ts^2, t^3) in $\text{PG}(2, q)$
-twisted_cubic		Create a twisted cubic (s^3, s^2t, st^2, t^3) in $\text{PG}(3, q)$
-elliptic_curve	$a \ b$	Create the elliptic curve $y^2 = x^3 + ax + b$
-ttp_construction_A		Create the twisted tensor product code of type A [11]
-ttp_construction_A_hyperoval		Create the twisted tensor product code of type A [11]
-ttp_construction_B		Create the twisted tensor product code of type B [11]

Table 4.9: Geometric Objects (Part 1)

Geometric Objects (Part 2)		
Command	Arguments	Purpose
-unital_XXq_YZq_ZYq		Create the unital with equation $XX^q + YZ^q + ZY^q = 0$
-desarguesian_line_spread_in_PG_3_q		Create the desarguesian line spread in $PG(3, q)$ as a set of 2-subspaces
-Buekenhout_Metz		Create the Buekenhout Metz unital
-Uab	$a \ b$	Create the Buekenhout Metz unital in the form of Barwick and Ebert [9]
-whole_space		Create the whole space
-hyperplane	pt	Create the hyperplane given by dual coordinates associated with the given point
-segre_variety	$a \ b$	Create the Segre variety
-arc1_BCKM		Create arc from family 1 in [15].
-arc2_BCKM		Create arc from family 2 in [15].
-Maruta_Hamada_arc		Create the Maruta Hamada arc
-projective_variety	lab_ascii lab_tex d coeffs	Create a projective variety of degree d from an equation. By default, the coefficients of the equation are listed in the partition ordering. A different ordering can be specified. A label for the variety in ascii and in tex is required.
-projective_curve	$l \ r \ d \ \mathcal{C}$	Create the projective curve of degree d with label l , with coefficient vector $\mathcal{C}$ in r variables

Table 4.10: Geometric Objects (Part 2)


```

10  ▷ ▷ ▷ -geometric_object Geo \
11  ▷ ▷ -end \
12  ▷ ▷ -with C -do -combinatorial_object_activity -save \
13  ▷ ▷ -end
14

```

The next command creates the Baer subgeometry of $\text{PG}(3, 4)$:

Code Snippet 110

```

1  Baer_PG_3_4:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 4 -end \
4  ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5  ▷ ▷ -define Geo -geometric_object P \
6  ▷ ▷ ▷ -Baer_substructure \
7  ▷ ▷ -end \
8  ▷ ▷ -define C -combinatorial_object \
9  ▷ ▷ ▷ -label Baer Baer \
10 ▷ ▷ ▷ -geometric_object Geo \
11 ▷ ▷ -end
12

```

Let us consider BLT-sets. The following command recalls the unique BLT-set of order 5 from the database:

Code Snippet 111

```

1  BLT_database_5_0:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 5 -end \
4  ▷ ▷ -define P -projective_space -n 4 -field F -v 0 -end \
5  ▷ ▷ -define O -geometric_object P \
6  ▷ ▷ ▷ -BLT_database O \
7  ▷ ▷ -end \
8  ▷ ▷ -with O -do -combinatorial_object_activity -save \
9  ▷ ▷ -end
10

```

Let us pull the first BLT-set of order 7 and print it to latex:

Code Snippet 112

```

1  BLT_database_7_0_print:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 7 -end \
4  ▷ ▷ -define P -projective_space -n 4 -field F -v 0 -end \
5  ▷ ▷ -define Obj -geometric_object P \
6  ▷ ▷ ▷ -BLT_database O \
7  ▷ ▷ -end \
8  ▷ ▷ -with Obj -do -combinatorial_object_activity -save \
9  ▷ ▷ -end \

```

```
10  ▷ ▷ -define 0 -orthogonal_space 0 5 F -without_group -end \  
11  ▷ ▷ -define BLT_7_0 -set -file_orbiter_format BLT_7_0.txt -end \  
12  ▷ ▷ -with 0 -do -orthogonal_space_activity \  
13  ▷ ▷ ▷ -print_points BLT_7_0 \  
14  ▷ ▷ -end  
15  ▷ $(PDFLATEX) BLT_7_0_set_report.tex  
16  ▷ $(OPEN) BLT_7_0_set_report.pdf  
17
```

The output is shown below:

A set of points of size 8
The Points:
0 : $P_0 = (0, 1, 0, 0, 0)$
1 : $P_1 = (0, 0, 1, 0, 0)$
2 : $P_{40} = (0, 1, 2, 6, 2)$
3 : $P_{41} = (0, 1, 4, 3, 1)$
4 : $P_{42} = (0, 1, 1, 2, 3)$
5 : $P_{43} = (0, 1, 1, 5, 4)$
6 : $P_{44} = (0, 1, 4, 4, 6)$
7 : $P_{45} = (0, 1, 2, 1, 5)$

Chapter 5

Group Theory

5.1 Generic Permutation Groups

In this chapter, we will look at Orbiter's approach to permutation groups.

A permutation group G is a group that comes with a distinguished action on a set X . The set X is called the permutation domain of G . One group can have different actions. In Orbiter, the set is often taken as the interval

$$X = \{0, 1, 2, \dots, n - 1\}.$$

The size of the permutation domain is also known as the degree of the action, or the degree of the permutation group. It is the integer n in the definition of X . A group element may be stored as a mapping. For instance, the cyclic group of order 6, denoted C_6 , is generated by the permutation

$$\begin{bmatrix} 0 & 1 & 2 & 3 & 4 & 5 \\ 1 & 2 & 3 & 4 & 5 & 0 \end{bmatrix}.$$

If we wish to define this group element in Orbiter, we would list the second row only, simply as a comma-separated list of numbers:

$$1, 2, 3, 4, 5, 0.$$

Groups can also be defined as sets of algebraic functions. A matrix group is an example. Here, the function is defined by a matrix or a field automorphism or a vector, or a combination of all three.

Orbiter distinguishes two types of permutation groups: generic permutation groups and matrix groups. Both groups come with a permutation action, i.e. with an underlying set X on which the group G acts. In a generic permutation group, a group element is stored as a permutation of X . In a matrix group, the set is implicitly assigned a structure. It could be a vector space or a projective space. A matrix group is a group of algebraic functions. It has the advantage that the group elements can be stored very compact. There are three types of algebraic functions: linear maps, semilinear maps, affine maps. Depending on the type, the group element consists of a matrix or a matrix plus a field automorphism or a matrix plus a field automorphism plus a vector. Whenever we desire the image of a given element $x \in X$, we simply compute it using the algebraic function. There is no need to store the list of images of elements of X . Those types of groups will be discussed in Section 5.2.

In this section, we will talk about generic permutation groups in Orbiter.

In order to establish the permutation representation of a group, the technique of indexing is used. Indexing sets up a fixed bijection between the permutation domain X , and the integer interval $[0, n - 1]$ for some n . The integer associated to an element in the permutation domain is called the rank. Conversely, given an integer in $[0, n - 1]$, the element in the permutation domain associated with it is obtained by the unrank function.

Creating a Generic Permutation Group		
Command	Arguments	Purpose
<code>-symmetric_group</code>	n	Symmetric group of degree n and order $n!$.
<code>-cyclic_group</code>	n	Cyclic group of degree n and order n .
<code>-elementary_abelian_group</code>	n	Elementary abelian group of order n and degree hp where $n = p^h$ and p prime.
<code>-identity_group</code>	n	Identity group of degree n and order 1.
<code>-dihedral_group</code>	n	Dihedral group of degree n and order $2n$.
<code>-bsgs</code>	lab1 lab2 n order base k gens	Group with a given base and a given set of generators. Here, lab1 and lab2 are labels in ascii and latex, respectively, k is the number of generators, gens is the generators as an Orbiter vector of integers.
<code>-subgroup_by_generators</code>	lab order k gens	Subgroup given by generators. Here, k is the number of generators, gens is an Orbiter vector of integers.

Table 5.1: Creating a Generic Permutation Group

The process of converting integers to elements of the permutation domain and vice-versa is indexing. We have seen indexing for projective points in Section 4.1.

The enumerators for projective points from Section 4.1 are used to realize the permutation domain. This enumerator in turn relies on an enumerator for finite fields, as discussed in Sections 3.1 and 3.2. For extension fields, the enumerator for finite fields in turn depends on the choice of the irreducible polynomial which is used to create the field. For affine groups, a different enumerator is used to describe the permutation domain. This enumerator uses the base- q representation of integers, which associates a vector over $\mathbb{F}_q$ of length n with an integer in $[0, q^n - 1]$.

Permutation groups and matrix groups can be represented on a computer using the technique of stabilizer chains, or Sims chains (cf. [41, 68]). A Stabilizer chain is a chain of subgroups of a given group G which terminates in the identity subgroup, such as

$$G = G_1 \geq G_2 \geq \cdots \geq G_r = 1.$$

For computational efficiency, it is desired that the index of one group in the previous is small. The easiest way to define subgroup chains in a permutation group G is by means of stabilizer chains. A stabilizer chain is defined by successively stabilizing points in the permutation domain. The sequence of points that define the stabilizer chain is called the base. A set of generators which allows to generate each group along the chain is called a strong generating set. Many algorithms for permutation groups rely on knowing a base and strong generating set. In Orbiter, permutation groups can be created from a base and strong generating set. Many types of groups come with their own built-in base and strong generating set. On the other hand, it is also possible to create groups from generating sets which are either not strong or for which a base is not known. A good base has short basic orbits. Some groups come with known bases. For instance, Orbiter uses a default base for semilinear groups.

Table 5.1 lists Orbiter commands to create a generic permutation group.

Let us start with a cyclic group. The following command creates a cyclic group of order 6:

Code Snippet 113

```

1    Cyclic_6:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define Do -draw.options \
4    ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5    ▷ ▷ -end \
6    ▷ ▷ -define G -permutation_group -cyclic_group 6 -end \
7    ▷ ▷ -with G -do \
8    ▷ ▷ ▷ -group_theoretic_activity \
9    ▷ ▷ ▷ -report Do \
10   ▷ ▷ -end
11   ▷ $(PDFLATEX) C_6_report.tex
12   ▷ $(OPEN) C_6_report.pdf
13

```

It is possible to specify the generating set directly. Here, we describe the permutation in list notation, i.e. the sequence 1, 2, 3, 4, 5, 0.

Code Snippet 114

```

1    Cyclic_6_from_generator:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define Do -draw.options \
4    ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5    ▷ ▷ -end \
6    ▷ ▷ -define gens -vector -dense \
7    ▷ ▷ ▷ "1,2,3,4,5,0" \
8    ▷ ▷ -end \
9    ▷ ▷ -define G -permutation_group -symmetric_group 6 \
10   ▷ ▷ ▷ -subgroup_by_generators G1 6 1 gens -end \
11   ▷ ▷ -with G -do \
12   ▷ ▷ -group_theoretic_activity \
13   ▷ ▷ ▷ -report Do \
14   ▷ ▷ -end
15   ▷ $(PDFLATEX) Sym_6_Subgroup_G1_6_report.tex
16   ▷ $(OPEN) Sym_6_Subgroup_G1_6_report.pdf
17

```

The following command

Code Snippet 115

```

1    Cyclic_6_group_table:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define G -permutation_group -cyclic_group 6 -end \
4    ▷ ▷ -with G -do \
5    ▷ ▷ -group_theoretic_activity \
6    ▷ ▷ ▷ -export_group_table \
7    ▷ ▷ -end
8    ▷ $(ORBITER) -v 2 \
9    ▷ ▷ -define all_one_r -vector -repeat 1 6 -end \

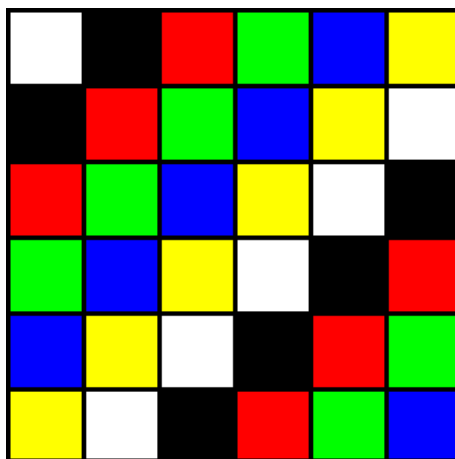
```

```

10  ▷ ▷ -define all_one_c -vector -repeat 1 6 -end \
11  ▷ ▷ -draw_matrix \
12  ▷ ▷ ▷ -input_csv_file C_6_group_table.csv \
13  ▷ ▷ ▷ -box_width 50 -bit_depth 24 \
14  ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
15  ▷ ▷ -end
16  ▷ $(OPEN) C_6_group_table_draw.bmp
17  ▷ magick C_6_group_table_draw.bmp C_6_group_table_draw.png
18

```

produces a graphical representation of the group table of the cyclic group C_6 , shown below



The following command creates the symmetric group $\text{Sym}(3)$:

Code Snippet 116

```

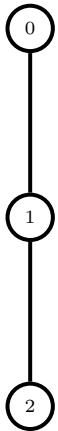
1  Symmetric_3:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define Do -draw_options \
4  ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5  ▷ ▷ -end \
6  ▷ ▷ -define G -permutation_group -symmetric_group 3 -end \
7  ▷ ▷ -with G -do \
8  ▷ ▷ -group_theoretic_activity \
9  ▷ ▷ ▷ -report Do \
10 ▷ ▷ -end
11 ▷ $(PDFLATEX) Sym_3_report.tex
12 ▷ $(OPEN) Sym_3_report.pdf
13

```

The report is shown below:

Stabilizer chain

Level	Base pt	Orbit length	Subgroup order
0	0	3	6
1	1	2	2

Basic Orbit 0

Basic orbit 0 has size 3
0, 1, 2

Basic Orbit 1

Basic orbit 1 has size 2
1, 2

The following command

Code Snippet 117

```

1 Symmetric_3_group_table:
2 ▷ $(ORBITER) -v 3 \

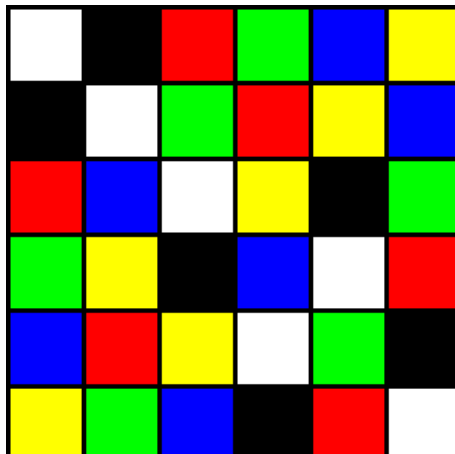
```

```

3  ▷ ▷ -define G -permutation_group -symmetric_group 3 -end \
4  ▷ ▷ -with G -do \
5  ▷ ▷ -group_theoretic_activity \
6  ▷ ▷ ▷ -export_group_table \
7  ▷ ▷ -end
8  ▷ $(ORBITER) -v 2 \
9  ▷ ▷ -define all_one_r -vector -repeat 1 6 -end \
10 ▷ ▷ -define all_one_c -vector -repeat 1 6 -end \
11 ▷ ▷ -draw_matrix \
12 ▷ ▷ ▷ -input_csv_file Sym_3_group_table.csv \
13 ▷ ▷ ▷ -box_width 50 \
14 ▷ ▷ ▷ -bit_depth 24 \
15 ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
16 ▷ ▷ -end
17 ▷ $(OPEN) Sym_3_group_table_draw.bmp
18

```

produces a graphical representation of the group table of the symmetric group $\text{Sym}(3)$, shown below



The next command

Code Snippet 118

```

1  Symmetric_3.elements:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define G -permutation_group -symmetric_group 3 -end \
4  ▷ ▷ -with G -do \
5  ▷ ▷ -group_theoretic_activity \
6  ▷ ▷ ▷ -save_elements_csv "Symmetric3.elts.csv" \
7  ▷ ▷ -end
8  ▷ $(ORBITER) -v 2 \
9  ▷ ▷ -define Sym3.elts -vector -load_csv_data_column \
10 ▷ ▷ ▷ Symmetric3.elts.csv 1 -end \
11 ▷ ▷ -save_matrix_csv Sym3.elts
12 ▷ $(ORBITER) -v 2 \

```

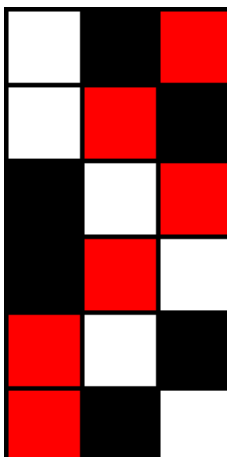


```

13  ▷ ▷ -define all_one_r -vector -repeat 1 6 -end \
14  ▷ ▷ -define all_one_c -vector -repeat 1 3 -end \
15  ▷ ▷ -draw_matrix \
16  ▷ ▷ ▷ -input_csv_file Sym3_elts_matrix.csv \
17  ▷ ▷ ▷ -box_width 50 \
18  ▷ ▷ ▷ -bit_depth 8 \
19  ▷ ▷ ▷ -partition 3 \
20  ▷ ▷ ▷ ▷ all_one_r all_one_c \
21  ▷ ▷ -end
22  ▷ $(OPEN) Sym3_elts_matrix_draw.bmp
23

```

produces a graphical representation of the elements of the symmetric group $\text{Sym}(3)$, shown below



Next, let us consider the elementary abelian group of order 8. The following command creates this group:

Code Snippet 119

```

1  Elementary_abelian_8:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define Do -draw_options \
4  ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5  ▷ ▷ -end \
6  ▷ ▷ -define G -permutation_group -elementary_abelian_group 8 -end \
7  ▷ ▷ -with G -do \
8  ▷ ▷ ▷ -group_theoretic_activity \
9  ▷ ▷ ▷ -report Do \
10 ▷ ▷ -end
11 ▷ $(PDFLATEX) E_8_report.tex
12 ▷ $(OPEN) E_8_report.pdf
13

```

In total, there are 5 groups of order 8. Three groups are abelian (C_8 , $C_4 \times C_2$ and $C_2 \times C_2 \times C_2$) and two are not (Dihedral group of order 8 and the quaternion group). The following commands can be used to create these five groups one-by-one:

Code Snippet 120

```

1   Group.8_1:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define gens -vector -dense \
4   ▷ ▷ ▷ "1,2,3,4,5,6,7,0" \
5   ▷ ▷ -end \
6   ▷ ▷ -define G -permutation_group -symmetric_group 8 \
7   ▷ ▷ ▷ -subgroup_by_generators G1 8 1 gens -end \
8   ▷ ▷ -with G -do \
9   ▷ ▷ -group_theoretic_activity \
10  ▷ ▷ ▷ -export_group_table \
11  ▷ ▷ -end
12  ▷ $(ORBITER) -v 2 \
13  ▷ ▷ -define all_one_r -vector -repeat 1 8 -end \
14  ▷ ▷ -define all_one_c -vector -repeat 1 8 -end \
15  ▷ ▷ -draw_matrix \
16  ▷ ▷ ▷ -input_csv_file \
17  ▷ ▷ ▷ ▷ Sym_8.Subgroup.G1_8.group_table.csv \
18  ▷ ▷ ▷ -box_width 50 \
19  ▷ ▷ ▷ -bit_depth 24 \
20  ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
21  ▷ ▷ -end
22  ▷ magick Sym_8.Subgroup.G1_8.group_table_draw.bmp \
23  ▷ ▷ Sym_8.Subgroup.G1_8.group_table_draw.png
24  ▷ $(OPEN) Sym_8.Subgroup.G1_8.group_table_draw.png
25

```

Code Snippet 121

```

1   Group.8_2:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define gens -vector -dense \
4   ▷ ▷ ▷ "1,2,3,0,4,5, 0,1,2,3,5,4" \
5   ▷ ▷ -end \
6   ▷ ▷ -define G -permutation_group -symmetric_group 6 \
7   ▷ ▷ ▷ -subgroup_by_generators G2 8 2 gens -end \
8   ▷ ▷ -with G -do \
9   ▷ ▷ -group_theoretic_activity \
10  ▷ ▷ ▷ -export_group_table \
11  ▷ ▷ -end
12  ▷ $(ORBITER) -v 2 \
13  ▷ ▷ -define all_one_r -vector -repeat 1 8 -end \
14  ▷ ▷ -define all_one_c -vector -repeat 1 8 -end \
15  ▷ ▷ -draw_matrix \
16  ▷ ▷ ▷ -input_csv_file \
17  ▷ ▷ ▷ ▷ Sym_6.Subgroup.G2_8.group_table.csv \
18  ▷ ▷ ▷ -box_width 50 \
19  ▷ ▷ ▷ -bit_depth 24 \
20  ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
21  ▷ ▷ -end
22  ▷ magick Sym_6.Subgroup.G2_8.group_table_draw.bmp \

```

```

23  ▷ ▷ Sym_6.Subgroup.G2_8.group.table.draw.png
24  ▷ $(OPEN) Sym_6.Subgroup.G2_8.group.table.draw.png
25

```

Code Snippet 122

```

1    Group_8_3:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define gens -vector -dense \
4    ▷ ▷ ▷ "1,0,2,3,4,5, 0,1,3,2,4,5, 0,1,2,3,5,4" \
5    ▷ ▷ -end \
6    ▷ ▷ -define G -permutation_group -symmetric_group 6 \
7    ▷ ▷ ▷ -subgroup_by_generators G3 8 3 gens -end \
8    ▷ ▷ -with G -do \
9    ▷ ▷ -group_theoretic_activity \
10   ▷ ▷ ▷ -export_group_table \
11   ▷ ▷ -end
12   ▷ $(ORBITER) -v 2 \
13   ▷ ▷ -define all_one_r -vector -repeat 1 8 -end \
14   ▷ ▷ -define all_one_c -vector -repeat 1 8 -end \
15   ▷ ▷ -draw_matrix \
16   ▷ ▷ ▷ -input_csv_file \
17   ▷ ▷ ▷ ▷ Sym_6.Subgroup.G3_8.group.table.csv \
18   ▷ ▷ ▷ -box_width 50 \
19   ▷ ▷ ▷ -bit_depth 24 \
20   ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
21   ▷ ▷ -end
22   ▷ magick Sym_6.Subgroup.G3_8.group.table.draw.bmp \
23   ▷ ▷ Sym_6.Subgroup.G3_8.group.table.draw.png
24   ▷ $(OPEN) Sym_6.Subgroup.G3_8.group.table.draw.png
25

```

Code Snippet 123

```

1    Group_8_4:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define gens -vector -dense \
4    ▷ ▷ ▷ "1,2,3,0, 0,3,2,1" \
5    ▷ ▷ -end \
6    ▷ ▷ -define G -permutation_group -symmetric_group 4 \
7    ▷ ▷ ▷ -subgroup_by_generators G4 8 2 gens -end \
8    ▷ ▷ -with G -do \
9    ▷ ▷ -group_theoretic_activity \
10   ▷ ▷ ▷ -export_group_table \
11   ▷ ▷ -end
12   ▷ $(ORBITER) -v 2 \
13   ▷ ▷ -define all_one_r -vector -repeat 1 8 -end \
14   ▷ ▷ -define all_one_c -vector -repeat 1 8 -end \
15   ▷ ▷ -draw_matrix \
16   ▷ ▷ ▷ -input_csv_file \
17   ▷ ▷ ▷ ▷ Sym_4.Subgroup.G4_8.group.table.csv \

```

```

18  ▷ ▷ ▷ -box_width 50 \
19  ▷ ▷ ▷ -bit_depth 24 \
20  ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
21  ▷ ▷ -end
22  ▷ magick Sym_4.Subgroup.G4_8.group_table_draw.bmp \
23  ▷ ▷ Sym_4.Subgroup.G4_8.group_table_draw.png
24  ▷ $(OPEN) Sym_4.Subgroup.G4_8.group_table_draw.png
25

```

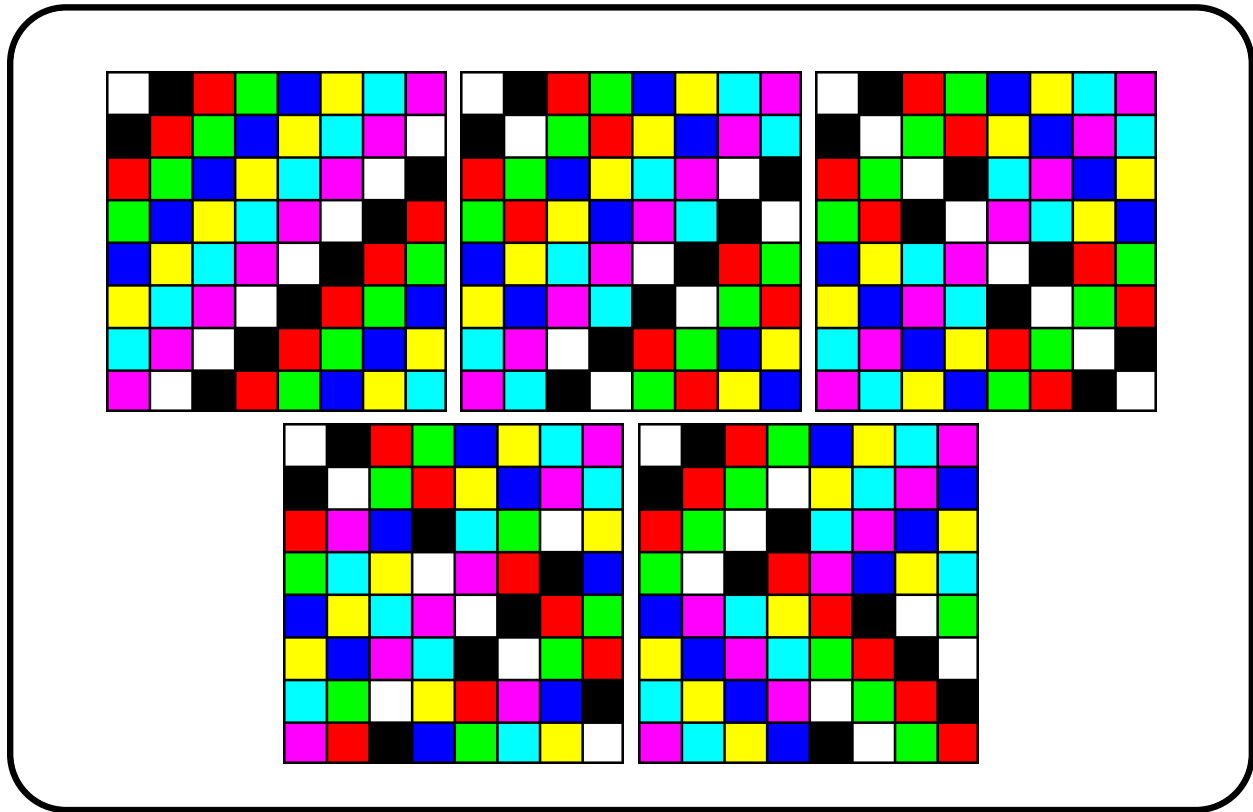
Code Snippet 124

```

1  Group_8_5:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define gens -vector -dense \
4  ▷ ▷ ▷ "2,3,1,0,6,7,5,4, 4,5,7,6,1,0,2,3" \
5  ▷ ▷ -end \
6  ▷ ▷ -define G -permutation_group -symmetric_group 8 \
7  ▷ ▷ ▷ -subgroup_by_generators G5 8 2 gens -end \
8  ▷ ▷ -with G -do \
9  ▷ ▷ -group_theoretic_activity \
10 ▷ ▷ ▷ -export_group_table \
11 ▷ ▷ -end
12 ▷ $(ORBITER) -v 2 \
13 ▷ ▷ -define all_one_r -vector -repeat 1 8 -end \
14 ▷ ▷ -define all_one_c -vector -repeat 1 8 -end \
15 ▷ ▷ -draw_matrix \
16 ▷ ▷ ▷ -input_csv_file \
17 ▷ ▷ ▷ ▷ Sym_8.Subgroup.G5_8.group_table.csv \
18 ▷ ▷ ▷ -box_width 50 \
19 ▷ ▷ ▷ -bit_depth 24 \
20 ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
21 ▷ ▷ -end
22 ▷ magick Sym_8.Subgroup.G5_8.group_table_draw.bmp \
23 ▷ ▷ Sym_8.Subgroup.G5_8.group_table_draw.png
24 ▷ $(OPEN) Sym_8.Subgroup.G5_8.group_table_draw.png
25

```

The Cayley tables are shown below



The Mathieu group M_{12} of order 95040 is generated by the permutations

$$(0, 3)(2, 9)(4, 10)(5, 11), \quad (0, 7, 8)(1, 2, 3)(4, 11, 10)(5, 9, 6),$$

taken from [76]. The generators can be written in list notation, which is the second row in the arrays

$$\begin{bmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 \\ 3 & 1 & 9 & 0 & 10 & 11 & 6 & 7 & 8 & 2 & 4 & 5 \end{bmatrix} \text{ and } \begin{bmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 \\ 7 & 2 & 3 & 1 & 11 & 9 & 5 & 8 & 0 & 6 & 4 & 10 \end{bmatrix}.$$

We use a makefile variable to store these generators, written one after another:

Code Snippet 125

```
1 GENERATORS.M12="3,1,9,0,10,11,6,7,8,2,4,5, 7,2,3,1,11,9,5,8,0,6,4,10"
2
```

The following command creates the group and produces a report:

Code Snippet 126

```
1 Group.M12:
2   ▷ $(ORBITER) -v 2 \
3   ▷   ▷ -define Do -draw_options \
4   ▷   ▷   ▷ -radius 200 -line_width 0.3 \
5   ▷   ▷ -end \
6   ▷   ▷ -define gens -vector -dense $(GENERATORS.M12) -end \
7   ▷   ▷ -define G -permutation_group -symmetric_group 12 \
```

```
8      ▷ ▷ ▷ -subgroup_by_generators M12 95040 2 gens -end \  
9      ▷ ▷ -with G -do \  
10     ▷ ▷ -group_theoretic_activity \  
11     ▷ ▷ ▷ -report Do \  
12     ▷ ▷ -end  
13     ▷ $(PDFLATEX) Sym_12_Subgroup_M12_95040_report.tex  
14     ▷ $(OPEN) Sym_12_Subgroup_M12_95040_report.pdf  
15
```

Creating a Matrix Group (Part 1)		
Command	Arguments	Purpose
-GL	$n F$	$GL(n, q)$. Here (and in all of below), F is an Orbiter object for a finite field $\mathbb{F}_q$.
-GGL	$n F$	$\Gamma L(n, q)$.
-SL	$n F$	$SL(n, q)$.
-SSL	$n F$	$\Sigma L(n, q)$.
-PGL	$n F$	$PGL(n, q)$.
-PGGL	$n F$	$P\Gamma L(n, q)$.
-PSL	$n F$	$PSL(n, q)$.
-PSSL	$n F$	$P\Sigma L(n, q)$.
-AGL	$n F$	$AGL(n, q)$.
-AGGL	$n F$	$A\Gamma L(n, q)$.
-ASL	$n F$	$ASL(n, q)$.
-ASSL	$n F$	$A\Sigma L(n, q)$.
-GL_d_q_wr_Sym_n	$d F n$	$GL(d, q)$ wreath product $Sym(n)$.
-PGO	$n F$	$PGO(n, q)$.
-PGOp	$n F$	$PGO^+(n, q)$.
-PGOm	$n F$	$PGO^-(n, q)$.
-PGGO	$n F$	$P\Gamma O(n, q)$.
-PGGOp	$n F$	$P\Gamma O^+(n, q)$.
-PGGOm	$n F$	$P\Gamma O^-(n, q)$.

Table 5.2: Creating a Matrix Group (Part 1)

5.2 Matrix Groups

Tables 5.2-5.3 shows the Orbiter commands to create a matrix group. Matrix groups are treated as permutation groups, so that a stabilizer chain can be established. Therefore, when creating a matrix group, a permutation representation must be chosen. By default, this will be the natural permutation representation. In the case of a projective matrix group, this is the action on the points of the underlying projective space. In case of an affine group or a general linear group, it is the set of vectors of the underlying vector space. At times, there are other permutation actions which are more suitable. Here, suitable often means that the degree of the permutation representation is small. This helps improve the performance of the permutation group algorithms. For background information about the classical groups of matrices over finite fields and their actions, see [73], for instance.

Let $V \simeq \mathbb{F}_q^n$ be a finite dimensional vector space over $\mathbb{F}_q$. The set of subspaces of V form the projective geometry $PG(n-1, q)$. The points of the geometry are the one-dimensional subspaces of $\mathbb{F}_q^n$. The hyperplanes are the codimension one subspaces. For each i with $0 \leq i \leq n$, there are subspaces of that dimension. They form the elements of a given type. For $i = 1$ and $i = n-1$ we have points and hyperplanes. For $i = 2$ we get lines. For $i = 3$ we get planes. The easiest way to denote a subspace is by means of a generator matrix. This is a matrix whose rows form a basis for the subspace in question.

Let π be a projective space. A collineation of a projective space π is a bijective mapping from the points of π to themselves which preserves collinearity. That is, a collineation φ maps any three collinear points P, Q, R to another collinear triple $\varphi(P), \varphi(Q), \varphi(R)$. The collineations form a group with respect to composition, the collineation group. If M is the matrix of an endomorphism, then Ψ_M is the induced map on projective space. By considering the homomorphism $M \mapsto \Psi_M$, the group $GL(n+1, q)$ of invertible endomorphisms becomes a subgroup of the group of collineations of $PG(n, q)$. This is the projectivity group $PGL(n+1, q)$.

Creating a Matrix Group (Part 2)		
Command	Arguments	Purpose
-wedge		Create the group in the action on the wedge product.
-wedge_detached		Create the group in the action on the wedge product.
-PGL20nConic		Create the action of $\text{PGL}(2, q)$ on the conic.
-monomial		Create the group in the monomial action.
-diagonal		Create the group in the diagonal action.
-null_polarity_group		Create the subgroup corresponding to a null polarity.
-symplectic_group		Create the subgroup of type symplectic group.
-singer		Create the Singer subgroup.
-singer_and_frobenius	k	Create the Singer subgroup extended by a Frobenius automorphism. Here, k is the power of the Frobenius endomorphism.
-subfield_structure_action	s	Create the action on the field reduction w.r.t. the subfield $\mathbb{F}_r$, where $r^s = q$.
-subgroup_from_file	fname label	Create the subgroup from the given file and assign the given label.
-borel_upper		Create the upper Borel subgroup.
-borel_lower		Create the lower Borel subgroup.
-identity_group		Create the identity subgroup.
-orthogonal	epsilon	Create the orthogonal group of type epsilon as subgroup, where $\epsilon \in \{0, 1, -1\}$.
-on_tensors		Create the tensor action.
-on_rank_one_tensors		Create the action on rank-one tensors.
-subgroup_by_generators	label order k gens	Create a subgroup with a known order from a set of generators. There are k generators. gens is a orbiter vector object.
-Janko1		Create the Janko group J_1 as a subgroup.
-export_magma		Export the group to Magma.
-import_group_of_plane	plane	Import the group of a translation plane. Here, plane must be an Orbiter object of type translation plane.
-lex_least_base		Create the lex-least base. This will make indexing of group elements deterministic.

Table 5.3: Creating a Matrix Group (Part 2)

It is isomorphic to $\text{GL}(n+1, q)/\mathbb{F}_q^\times$. Another source of collineations is this: Let $\Phi \in \text{Aut}(\mathbb{F}_q)$ be a field automorphism. Then Φ acts on projective space by sending $P(\mathbf{x})$ to $P(\mathbf{x}\Phi)$. This map is another type of collineation, called automorphic collineation. This way, $\text{Aut}(\mathbb{F}_q)$ gives rise to a group of collineations. If $q = p^h$ for some prime p and some integer h then

$$\Phi_0 : \mathbb{F}_q \rightarrow \mathbb{F}_q, x \mapsto x^p$$

is a generator for the cyclic group $C_h \simeq \text{Aut}(\mathbb{F}_q)$. The collineation group of $\text{PG}(n, q)$ ($n \geq 2$) is isomorphic to the semidirect product of the projectivity group and the automorphism group of the field. The collineation group is $\text{P}\Gamma\text{L}(n+1, q) = \text{PGL}(n+1, q) \ltimes \text{Aut}(\mathbb{F}_q)$. We use the following notation for elements of $\text{P}\Gamma\text{L}(n+1, q)$. Let Φ_0 be a generator for $\text{Aut}(\mathbb{F}_q)$ and let $M \in \text{GL}(n+1, q)$. The map

$$(\Psi_M, \Phi_0^k) : \text{PG}(n, q) \rightarrow \text{PG}(n, q), P(\mathbf{x}) \mapsto P(\mathbf{y}), \quad \mathbf{y} = (\mathbf{x} \cdot M)^{\Phi_0^k}$$

is denoted as

$$M_k. \tag{5.1}$$

The identity element is I_0 , where I is the identity matrix and 0 is the residue class modulo h . The rules for multiplication and inversion in the collineation group are given as

$$M_k \cdot N_l = \left(M \cdot N^{\Phi^{-k}} \right)_{k+l}, \tag{5.2}$$

$$\left(M_k \right)^{-1} = \left(\left(M^{-1} \right)^{\Phi^k} \right)_{-k}. \tag{5.3}$$

The affine group $\text{AGL}(n, q)$ is the semidirect product of $\text{GL}(n, q)$ with $\mathbb{F}_q^n$. The affine semilinear group $\text{A}\Gamma\text{L}(n, q)$ is the semidirect product of $\text{AGL}(n, q)$ with $\text{Aut}(\mathbb{F}_q)$. The elements of $\text{A}\Gamma\text{L}(n, q)$ are triples

$$M_{\mathbf{a}, k} := (M, \mathbf{a}, k) \in \text{GL}(n, q) \times \mathbb{F}_q^n \times \text{Aut}(\mathbb{F}_q),$$

which act on $\mathbb{F}_q^n$:

$$\left(\mathbf{x}, (M, \mathbf{a}, k) \right) \mapsto (\mathbf{x} \cdot M + \mathbf{a})^{\Phi^k}.$$

The multiplication in $\text{A}\Gamma\text{L}(n, q)$ is

$$M_{\mathbf{a}, k} \cdot N_{\mathbf{b}, l} = (MN)_{\mathbf{a}N^{\Phi^{-k}} + \mathbf{b}^{\Phi^{-k}}, k+l}.$$

The inverse of an element is

$$\left(M_{\mathbf{a}, k} \right)^{-1} = \left(M^{-1} \right)_{\mathbf{a}^{\Phi^k} M^{-1}, -k}.$$

A correlation is a one-to-one mapping between the set of points and the set of hyperplanes which reverses incidence. So, if ρ is a correlation and P is a point and ℓ is a hyperplane then P^ρ is a hyperplane and ℓ^ρ is a point and

$$\ell^\rho \in P^\rho \iff P \in \ell.$$

A correlation of order two is called polarity. The standard polarity is the map

$$\rho : \mathcal{P} \leftrightarrow \mathcal{L}, P(\mathbf{x}) \leftrightarrow [\mathbf{x}].$$

A group G can act on V in one of the types listed in Table 5.4.

To create a matrix group over a finite field, it is possible to specify the field by its order. It is also possible to use a specific field in the construction of the group. In this case, one can create the field beforehand, and reference the field object in the creation of the group. The second way allows more control, as it allows to select the irreducible polynomial that is used to create the field. If only the field order is given, the default polynomial is used.

The command

Basic actions		
Name	Object	Number of points
General linear $GL(n, q)$	all vectors of V	q^n
Affine $AGL(n, q)$	all vectors of V	q^n
Projective $PGL(n, q)$	$\mathfrak{G}r_1(V)$	$\frac{q^n-1}{q-1}$
Wreath product $GL(d, q) \wr \text{Sym}(n)$	$\mathfrak{G}r_1((\mathbb{F}_q^d)^{\otimes n})$ extended	$n + nq^d + \frac{q^{dn}-1}{q-1}$
Orthogonal $PGO(n, q)$	$Q(V)$	$\frac{q^{n-1}-1}{q-1}$
Orthogonal $PGO^+(n, q)$	$Q^+(V)$	$\frac{(q^{n/2}-1)(q^{(n-2)/2}+1)}{q-1}$
Orthogonal $PGO^-(n, q)$	$Q^-(V)$	$\frac{(q^{n/2}+1)(q^{(n-2)/2}-1)}{q-1}$

Table 5.4: Basic actions

Code Snippet 127

```

1  GL_2.3_elements_dense:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 3 -end \
4  ▷ ▷ -define G -linear_group -GL 2 F -end \
5  ▷ ▷ -assign elements -from \
6  ▷ ▷ -with G -do \
7  ▷ ▷ -group_theoretic_activity \
8  ▷ ▷ ▷ -elements \
9  ▷ ▷ -end \
10 ▷ ▷ -with elements -do \
11 ▷ ▷ -vector_ge_activity \
12 ▷ ▷ ▷ -report_with_options "dense=on" \
13 ▷ ▷ -end \
14 ▷ ▷ -print_symbols
15 ▷ $(PDFLATEX) GL_2.3_elements.tex
16 ▷ $(OPEN) GL_2.3_elements.pdf
17

```

creates the group $GL(2, 3)$ acting on the 9 vectors of $\mathbb{F}_3^2$. In this case, the field is created explicitly. The group has 48 elements, which are printed in latex format. The latex output is shown below:

$$\begin{aligned}
& \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 1 & 2 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 2 & 2 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}, \\
& \begin{bmatrix} 0 & 1 \\ 1 & 2 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 2 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 2 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 2 & 2 \end{bmatrix}, \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 1 \\ 0 & 2 \end{bmatrix}, \begin{bmatrix} 1 & 1 \\ 2 & 0 \end{bmatrix}, \\
& \begin{bmatrix} 1 & 1 \\ 0 & 2 \end{bmatrix}, \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix}, \begin{bmatrix} 1 & 1 \\ 0 & 2 \end{bmatrix}, \begin{bmatrix} 1 & 1 \\ 2 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix}, \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 1 \\ 1 & 0 \end{bmatrix}, \begin{bmatrix} 2 & 1 \\ 2 & 0 \end{bmatrix}, \\
& \begin{bmatrix} 2 & 1 \\ 2 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 1 \\ 2 & 2 \end{bmatrix}, \begin{bmatrix} 2 & 1 \\ 2 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 1 \\ 2 & 0 \end{bmatrix}, \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}, \begin{bmatrix} 1 & 2 \\ 1 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 2 \\ 2 & 2 \end{bmatrix},
\end{aligned}$$

$$\left[\begin{array}{cc} 1 & 2 \\ 0 & 1 \end{array} \right], \left[\begin{array}{cc} 1 & 2 \\ 2 & 0 \end{array} \right], \left[\begin{array}{cc} 1 & 2 \\ 0 & 2 \end{array} \right], \left[\begin{array}{cc} 1 & 2 \\ 1 & 1 \end{array} \right], \left[\begin{array}{cc} 2 & 2 \\ 1 & 0 \end{array} \right], \left[\begin{array}{cc} 2 & 2 \\ 0 & 2 \end{array} \right], \left[\begin{array}{cc} 2 & 2 \\ 2 & 1 \end{array} \right], \left[\begin{array}{cc} 2 & 2 \\ 2 & 0 \end{array} \right], \\ \left[\begin{array}{cc} 2 & 2 \\ 1 & 2 \end{array} \right], \left[\begin{array}{cc} 2 & 2 \\ 0 & 1 \end{array} \right], \left[\begin{array}{cc} 2 & 0 \\ 0 & 1 \end{array} \right], \left[\begin{array}{cc} 2 & 0 \\ 2 & 1 \end{array} \right], \left[\begin{array}{cc} 2 & 0 \\ 1 & 1 \end{array} \right], \left[\begin{array}{cc} 2 & 0 \\ 0 & 2 \end{array} \right], \left[\begin{array}{cc} 2 & 0 \\ 2 & 2 \end{array} \right], \left[\begin{array}{cc} 2 & 0 \\ 1 & 2 \end{array} \right]$$

Code Snippet 128

```

1 PGL_2_3.elements.dense:
2 ▷ $(ORBITER) -v 2 \
3 ▷ ▷ -define Do -draw_options \
4 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5 ▷ ▷ -end \
6 ▷ ▷ -define G -linear_group -PGL 2 3 -end \
7 ▷ ▷ -assign elements -from \
8 ▷ ▷ -with G -do \
9 ▷ ▷ -group_theoretic_activity \
10 ▷ ▷ ▷ -elements \
11 ▷ ▷ -end \
12 ▷ ▷ -with elements -do \
13 ▷ ▷ -vector_ge_activity \
14 ▷ ▷ ▷ -report_with_options "dense=on" \
15 ▷ ▷ -end \
16 ▷ ▷ -print_symbols
17 ▷ $(PDFLATEX) PGL_2_3.elements.tex
18 ▷ $(OPEN) PGL_2_3.elements.pdf
19

```

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 2 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 2 \\ 1 & 0 \end{bmatrix}, \\ \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 2 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 2 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 2 \\ 2 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}, \begin{bmatrix} 2 & 2 \\ 1 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 2 \\ 2 & 1 \end{bmatrix}, \\ \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 2 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 2 \\ 1 & 0 \end{bmatrix}, \begin{bmatrix} 2 & 1 \\ 1 & 0 \end{bmatrix}, \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 2 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$$

Code Snippet 129

```

1   PGL_2_17:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \

```

```

5    ▷ ▷ -end \
6    ▷ ▷ -define F -finite_field -q 17 -end \
7    ▷ ▷ -define G -linear_group -PGL 2 F -end \
8    ▷ ▷ -with G -do \
9    ▷ ▷ -group_theoretic_activity \
10   ▷ ▷ ▷ -report Do \
11   ▷ ▷ -end
12   ▷ $(PDFLATEX) PGL_2.17_report.tex
13   ▷ $(OPEN) PGL_2.17_report.pdf
14

```

creates the group $\text{PGL}(2, 17)$ of order 4896 acting on the 18 points of $\text{PG}(1, 18)$. The group is not simple. However, the subgroup $\text{PSL}(2, 17)$ of order 2448 is. To create it, we used the following command:

Code Snippet 130

```

1    PSL_2.17:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define Do -draw_options \
4    ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5    ▷ ▷ -end \
6    ▷ ▷ -define F -finite_field -q 17 -end \
7    ▷ ▷ -define G -linear_group -PSL 2 F -end \
8    ▷ ▷ -with G -do \
9    ▷ ▷ -group_theoretic_activity \
10   ▷ ▷ ▷ -report Do \
11   ▷ ▷ -end
12   ▷ $(PDFLATEX) PSL_2.17_report.tex
13   ▷ $(OPEN) PSL_2.17_report.pdf
14

```

The command

Code Snippet 131

```

1    PGL_4.2:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define G -linear_group -PGL 4 2 -end
4

```

creates the group $\text{PGL}(4, 2)$ of order 20160 acting on the 15 elements of $\text{PG}(3, 2)$. In this case, the field is created implicitly.

The command

Code Snippet 132

```

1    PGL_4.2_with_field:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 2 -end \
4    ▷ ▷ -define G -linear_group -PGL 4 F -end
5

```

creates the field $\mathbb{F}_2$ first, and then builds the group based in this field. The group is the same as the group created by the previous command. However, it is good to be able to have more options when creating the finite field. For instance, when using extension fields, it is sometimes necessary to pick the appropriate polynomial that defines the field, as described in Section 3.2.

The command

Code Snippet 133

```

1   PGL_4_2_report:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define G -linear_group -PGL 4 2 -end \
7   ▷ ▷ -with G -do \
8   ▷ ▷ -group_theoretic_activity \
9   ▷ ▷ ▷ -report Do \
10  ▷ ▷ -end
11  ▷ $(PDFLATEX) PGL_4_2_report.tex
12  ▷ $(OPEN) PGL_4_2_report.pdf
13

```

creates a report about the group $\text{PGL}(4, 2)$. The report gives information about the underlying field and the projective geometry, as well as about the permutation action, including the stabilizer chain that is used to represent the group effectively.

The Group $\text{PGL}(4, 2)$

The order of the group $\text{PGL}(4, 2)$ is 20160

The group acts on a set of size 15

Strong generators for a group of order 20160:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}, \\
 \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

1,0,0,0,0,1,0,0,0,0,1,0,1,0,0,1,
 1,0,0,0,0,1,0,0,0,0,1,0,0,1,0,1,
 1,0,0,0,0,1,0,0,0,0,1,0,0,0,1,1,
 1,0,0,0,0,1,0,0,0,0,0,1,0,0,1,0,
 1,0,0,0,0,0,1,0,0,1,0,0,0,0,0,1,
 0,1,0,0,1,0,0,0,0,0,1,0,0,0,0,1,

The Action

Group action $\text{PGL}(4, 2)$ of degree 15

We act on the following set:

$$0 = (1, 0, 0, 0)$$

$$1 = (0, 1, 0, 0)$$

$$2 = (0, 0, 1, 0)$$

$$3 = (0, 0, 0, 1)$$

$$4 = (1, 1, 1, 1)$$

$$5 = (1, 1, 0, 0)$$

$$6 = (1, 0, 1, 0)$$

$$7 = (0, 1, 1, 0)$$

$$8 = (1, 1, 1, 0)$$

$$9 = (1, 0, 0, 1)$$

$$10 = (0, 1, 0, 1)$$

$$11 = (1, 1, 0, 1)$$

$$12 = (0, 0, 1, 1)$$

$$13 = (1, 0, 1, 1)$$

$$14 = (0, 1, 1, 1)$$

The group is a matrix group.

The group acts on projective space $\text{PG}(3, 2)$

$$q = 2$$

$$p = 2$$

$$e = 1$$

$$n = 3$$

Number of points = 15

Number of lines = 35

Number of lines on a point = 7

Number of points on a line = 3

The finite field $\mathbb{F}_2$

$$Z_i = \log_\alpha(1 + \alpha^i)$$

i	γ_i	$-\gamma_i$	γ_i^{-1}	$\log_\alpha(\gamma_i)$	α^i	Z_i
0	$0 = 0$	0	DNE	DNE	1	DNE
1	$1 = 1$	1	1	0	1	DNE

+	0	1
0	0	1
1	1	0

·	1
1	1

$$1^0 \equiv 1$$

$$1^1 \equiv 1$$

Base and Stabilizer Chain

Group order 20160

tl=15, 14, 12, 8,

Base: $(0, 1, 2, 3)$

Strong generators for a group of order 20160:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix},$$

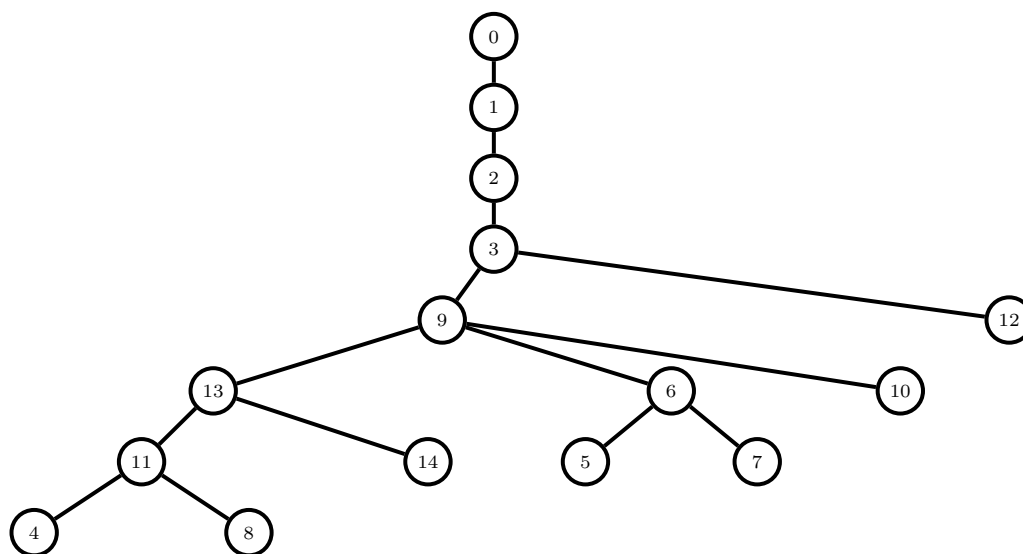
$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

1,0,0,0,0,1,0,0,0,0,1,0,1,0,0,1,
1,0,0,0,0,1,0,0,0,0,1,0,0,1,0,1,
1,0,0,0,0,1,0,0,0,0,1,0,0,0,1,1,
1,0,0,0,0,1,0,0,0,0,1,0,0,1,0,
1,0,0,0,0,1,0,0,1,0,0,0,0,1,
0,1,0,0,1,0,0,0,0,0,1,0,0,0,0,1,

Stabilizer chain

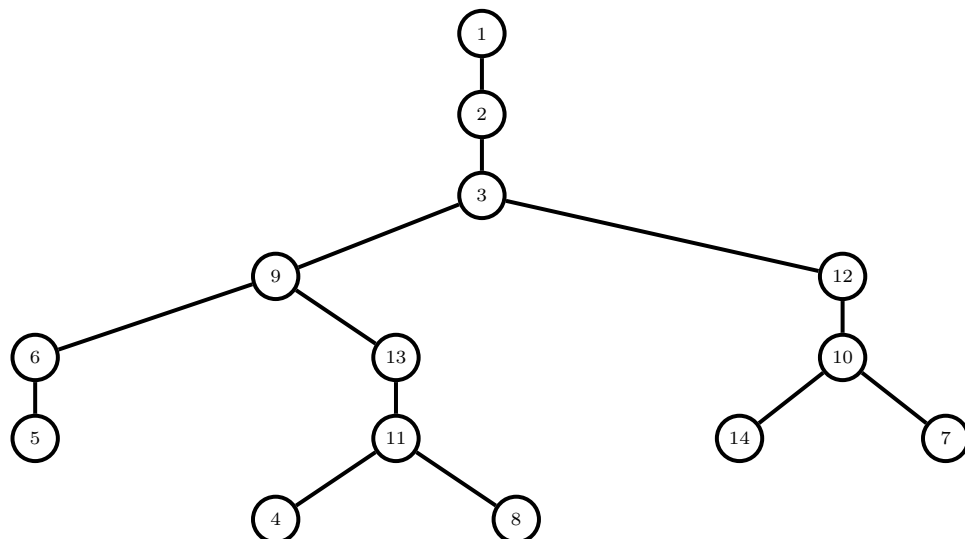
Level	Base pt	Orbit length	Subgroup order
0	0	15	20160
1	1	14	1344
2	2	12	96
3	3	8	8

Basic Orbit 0



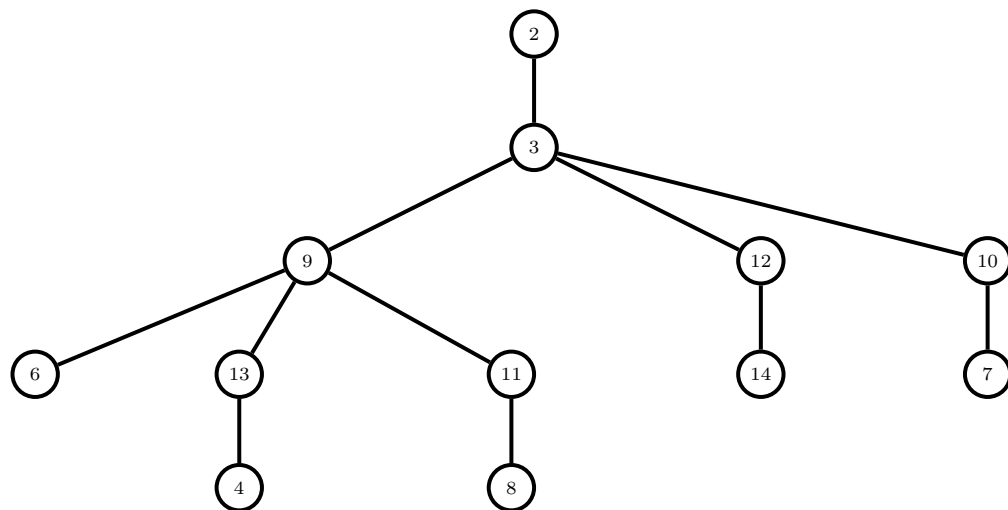
Basic orbit 0 has size 15

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14

Basic Orbit 1

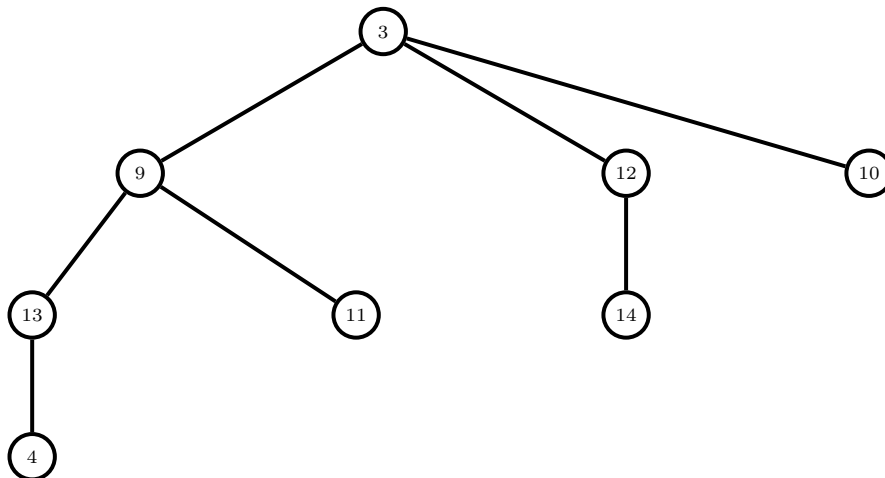
Basic orbit 1 has size 14

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14

Basic Orbit 2

Basic orbit 2 has size 12

2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13, 14

Basic Orbit 3

Basic orbit 3 has size 8
 3, 4, 9, 10, 11, 12, 13, 14
 GAP export:

Generators in GAP format are:
`G := Group([(4, 10)(5, 15)(11, 12)(13, 14),
 (4, 11)(5, 14)(10, 12)(13, 15),
 (4, 13)(5, 12)(10, 14)(11, 15),
 (3, 4)(7, 10)(8, 11)(9, 12),
 (2, 3)(6, 7)(11, 13)(12, 14),
 (1, 2)(7, 8)(10, 11)(14, 15)]);`

Magma export:

```

G := GeneralLinearGroup(4, GF(2));
H := sub< G | [1,0,0,0, 0,1,0,0, 0,0,1,0, 1,0,0,1],
[1,0,0,0, 0,1,0,0, 0,0,1,0, 0,1,0,1],
[1,0,0,0, 0,1,0,0, 0,0,1,0, 0,0,1,1],
[1,0,0,0, 0,1,0,0, 0,0,0,1, 0,0,1,0],
[1,0,0,0, 0,0,1,0, 0,1,0,0, 0,0,0,1],
[0,1,0,0, 1,0,0,0, 0,0,1,0, 0,0,0,1] >;

```

Compact form:

Generators in compact permutation form are:
 6 15
 0 1 2 9 14 5 6 7 8 3 11 10 13 12 4
 0 1 2 10 13 5 6 7 8 11 3 9 14 4 12
 0 1 2 12 11 5 6 7 8 13 14 4 3 9 10
 0 1 3 2 4 5 9 10 11 6 7 8 12 13 14
 0 2 1 3 4 6 5 7 8 9 12 13 10 11 14
 1 0 2 3 4 5 7 6 8 10 9 11 12 14 13
 -1

The base has length 4
 The basic orbits are:
 Basic orbit 0 is orbit of 0 of length 15

Basic orbit 1 is orbit of 1 of length 14
 Basic orbit 2 is orbit of 2 of length 12
 Basic orbit 3 is orbit of 3 of length 8

We use the following Orbiter command creates $\text{PGL}(4, 2)$ again. The command invokes two activities. The first creates a latex report for the group in the file `PGL_4_2_report.tex`. The second activity exports the permutation representation in Orbiter makefile format.

Code Snippet 134

```

1   PGL_4_2.export:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 2 -end \
7   ▷ ▷ -define G -linear_group -PGL 4 F -end \
8   ▷ ▷ -with G -do \
9   ▷ ▷ -group_theoretic_activity \
10  ▷ ▷ ▷ -report Do \
11  ▷ ▷ -end \
12  ▷ ▷ -with G -do \
13  ▷ ▷ -group_theoretic_activity \
14  ▷ ▷ ▷ -export_orbiter \
15  ▷ ▷ -end
16  ▷ $(PDFLATEX) PGL_4_2_report.tex
17  ▷ $(OPEN) PGL_4_2_report.pdf
18
```

The file `PGL_4_2.makefile` is created:

Code Snippet 135

```

1   PGL_4_2.generated:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define gens -vector -file PGL_4_2_gens.csv -end \
4   ▷ ▷ -define G -permutation_group \
5   ▷ ▷ -bsgs PGL_4_2 "{\rm PGL}(4,2)" 15 20160 "0,1,2,3" 6 gens -end \
6
```

This command can be used to recreate the group as permutation group directly. This group will be considered again in Section 5.2 below. The permutation representation itself is stored in the file `PGL_4_2_gens.csv`:

Row	C0	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14
0	0	1	2	9	14	5	6	7	8	3	11	10	13	12	4
1	0	1	2	10	13	5	6	7	8	11	3	9	14	4	12
2	0	1	2	12	11	5	6	7	8	13	14	4	3	9	10
3	0	1	3	2	4	5	9	10	11	6	7	8	12	13	14
4	0	2	1	3	4	6	5	7	8	9	12	13	10	11	14
5	1	0	2	3	4	5	7	6	8	10	9	11	12	14	13

END

The command

Code Snippet 136

```

1   L_5.3:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 3 -end \
7   ▷ ▷ -define G -linear_group -PSL 5 F -end \
8   ▷ ▷ -with G -do \
9   ▷ ▷ -group_theoretic_activity \
10  ▷ ▷ ▷ -report Do \
11  ▷ ▷ -end
12  ▷ $(PDFLATEX) PSL_5.3_report.tex
13  ▷ $(OPEN) PSL_5.3_report.pdf
14

```

creates $\text{PSL}(5, 3)$ of order 237783237120 in the action on the 121 points of $\text{PG}(4, 3)$.

The command

Code Snippet 137

```

1   L_4.5:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 5 -end \
7   ▷ ▷ -define G -linear_group -PSL 4 F -end \
8   ▷ ▷ -with G -do \
9   ▷ ▷ -group_theoretic_activity \
10  ▷ ▷ ▷ -report Do \
11  ▷ ▷ -end
12  ▷ $(PDFLATEX) PSL_4.5_report.tex
13  ▷ $(OPEN) PSL_4.5_report.pdf
14

```

creates $\text{PSL}(4, 5)$ of order 7254000000 in the action on the 156 points of $\text{PG}(3, 5)$.

The command

Code Snippet 138

```

1   PGL_4.5:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \

```

```

6    ▷ ▷ -define F -finite_field -q 5 -end \
7    ▷ ▷ -define G -linear_group -PGL 4 F -end \
8    ▷ ▷ -with G -do \
9    ▷ ▷ -group_theoretic_activity \
10   ▷ ▷ ▷ -report Do \
11   ▷ ▷ -end
12   ▷ $(PDFLATEX) PGL_4_5_report.tex
13   ▷ $(OPEN) PGL_4_5_report.pdf
14

```

creates $\text{PGL}(4, 5)$ of order 29016000000 in the action on the 156 points of $\text{PG}(3, 5)$. The group $\text{PSL}(4, 5)$ is a subgroup of $\text{PGL}(4, 5)$ of index 4.

The command

Code Snippet 139

```

1    PGGL_3_4:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define Do -draw_options \
4    ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5    ▷ ▷ -end \
6    ▷ ▷ -define G -linear_group -PGGL 3 4 -end \
7    ▷ ▷ -with G -do \
8    ▷ ▷ -group_theoretic_activity \
9    ▷ ▷ ▷ -report Do \
10   ▷ ▷ -end \
11   ▷ ▷ -with G -do \
12   ▷ ▷ -group_theoretic_activity \
13   ▷ ▷ ▷ -syLOW \
14   ▷ ▷ -end
15   ▷ $(PDFLATEX) PGGL_3_4_report.tex
16   ▷ $(OPEN) PGGL_3_4_report.pdf
17   ▷ $(PDFLATEX) PGGL_3_4_syLOW_report.tex
18   ▷ $(OPEN) PGGL_3_4_syLOW_report.pdf
19

```

creates $\text{PGL}(3, 4)$ of order 120960 in the action on the 21 points of $\text{PG}(2, 4)$. The report contains one p -Sylow subgroup for each of the primes p dividing the order of the group.

The command

Code Snippet 140

```

1    PGGL_3_8:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define Do -draw_options \
4    ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5    ▷ ▷ -end \
6    ▷ ▷ -define G -linear_group -PGGL 3 8 -end \
7    ▷ ▷ -with G -do \
8    ▷ ▷ -group_theoretic_activity \
9    ▷ ▷ ▷ -report Do \

```

```

10  ▷ ▷ -end \
11  ▷ ▷ -with G -do \
12  ▷ ▷ -group_theoretic_activity \
13  ▷ ▷ ▷ -syllow \
14  ▷ ▷ -end
15  ▷ $(PDFLATEX) PGGL_3_8_report.tex
16  ▷ $(OPEN) PGGL_3_8_report.pdf
17

```

creates $PTL(3, 8)$ of order 49448448 in the action on the 73 points of $PG(2, 8)$. The report contains one p -Sylow subgroup for each of the primes p dividing the order of the group.

The command

Code Snippet 141

```

1  AGL_1_27:
2  ▷ $(ORBITER) -v 4 \
3  ▷ ▷ -define Do -draw_options \
4  ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5  ▷ ▷ -end \
6  ▷ ▷ -define F -finite_field -q 27 -end \
7  ▷ ▷ -define G -linear_group -AGL 1 F -end \
8  ▷ ▷ -with G -do \
9  ▷ ▷ -group_theoretic_activity \
10 ▷ ▷ ▷ -report Do \
11 ▷ ▷ -end
12 ▷ $(PDFLATEX) AGL_1_27_report.tex
13 ▷ $(OPEN) AGL_1_27_report.pdf
14

```

creates $AGL(1, 27)$ of order 702 in the action on the 27 points of $AG(1, 27)$.

The command

Code Snippet 142

```

1  AGGL_2_27:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define Do -draw_options \
4  ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5  ▷ ▷ -end \
6  ▷ ▷ -define F -finite_field -q 27 -compute_related_fields -end \
7  ▷ ▷ -define G -linear_group -AGGL 2 F -end \
8  ▷ ▷ -with G -do \
9  ▷ ▷ -group_theoretic_activity \
10 ▷ ▷ ▷ -report Do \
11 ▷ ▷ -end
12 ▷ $(PDFLATEX) AGGL_2_27_report.tex
13 ▷ $(OPEN) AGGL_2_27_report.pdf
14

```

creates $ATL(2, 27)$ of order 1117679472 in the action on the 729 points of $AG(2, 27)$.

The command

Code Snippet 143

```

1    SP_4.2:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define Do -draw_options \
4    ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5    ▷ ▷ -end \
6    ▷ ▷ -define F -finite_field -q 2 -end \
7    ▷ ▷ -define G -linear_group -GL 4 F \
8    ▷ ▷ ▷ -symplectic_group \
9    ▷ ▷ -end \
10   ▷ ▷ -with G -do \
11   ▷ ▷ -group_theoretic_activity \
12   ▷ ▷ ▷ -report Do \
13   ▷ ▷ -end
14   ▷ $(PDFLATEX) GL_4.2_Sp_4.2_report.tex
15   ▷ $(OPEN) GL_4.2_Sp_4.2_report.pdf
16

```

creates the symplectic group $\text{Sp}(4, 2)$ of order 720 acting on the 16 points of $\text{AG}(4, 2)$.

The command

Code Snippet 144

```

1    PSP_4.4:
2    ▷ $(ORBITER) -v 5 \
3    ▷ ▷ -define Do -draw_options \
4    ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5    ▷ ▷ -end \
6    ▷ ▷ -define F -finite_field -q 4 -end \
7    ▷ ▷ -define G -linear_group -PGL 4 F \
8    ▷ ▷ ▷ -symplectic_group \
9    ▷ ▷ -end \
10   ▷ ▷ -with G -do \
11   ▷ ▷ -group_theoretic_activity \
12   ▷ ▷ ▷ -report Do \
13   ▷ ▷ -end
14   ▷ $(PDFLATEX) PSp_4.4_report.tex
15   ▷ $(OPEN) PSp_4.4_report.pdf
16

```

creates the symplectic group $\text{PSp}(4, 4)$ of order 979200 acting on the 85 points of $\text{PG}(3, 4)$.

The command

Code Snippet 145

```

1    PGO_5.2:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define Do -draw_options \
4    ▷ ▷ ▷ -radius 200 -line_width 0.3 \

```

```

5    ▷ ▷ -end \
6    ▷ ▷ -define F -finite_field -q 2 -end \
7    ▷ ▷ -define G -linear_group -PGO 5 F -end \
8    ▷ ▷ -with G -do \
9    ▷ ▷ -group_theoretic_activity \
10   ▷ ▷ ▷ -report Do \
11   ▷ ▷ -end
12   ▷ $(PDFLATEX) PGO_5_2_report.tex
13   ▷ $(OPEN) PGO_5_2_report.pdf
14

```

creates the group $\text{PGO}(5, 2)$ acting on the 15 points of the $Q(4, 2)$ quadric. The following latex report is produced:

The Group $\text{PGO}(5, 2)$

The order of the group $\text{PGO}(5, 2)$ is 720

The group acts on a set of size 15

Strong generators for a group of order 720:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \\
 \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

1,0,0,0,0,0,1,0,0,0,0,0,1,0,0,0,0,1,0,1,0,0,1,1,
 1,0,0,0,0,0,1,0,0,0,0,0,1,0,0,1,0,0,1,1,0,0,0,0,1,
 1,0,0,0,0,0,1,0,0,0,1,1,1,0,1,1,1,0,1,1,0,0,0,0,1,
 1,0,0,0,0,0,1,0,0,0,0,0,1,0,1,1,1,0,1,1,0,1,0,1,0,
 1,0,0,0,0,1,1,1,0,0,0,0,1,0,0,0,0,0,1,0,0,0,0,0,1,
 1,0,0,0,0,0,1,0,0,1,0,0,1,0,0,0,0,1,1,0,0,0,0,0,1,

The Action

Group action $\text{PGO}(5, 2)$ of degree 15

We act on the following set:

$0 = (0, 1, 0, 0, 0)$
 $1 = (0, 0, 1, 0, 0)$
 $2 = (0, 0, 0, 1, 0)$
 $3 = (0, 1, 0, 1, 0)$
 $4 = (0, 0, 1, 1, 0)$
 $5 = (0, 0, 0, 0, 1)$
 $6 = (0, 1, 0, 0, 1)$
 $7 = (0, 0, 1, 0, 1)$

$8 = (0, 1, 1, 1, 1)$
 $9 = (1, 1, 1, 0, 0)$
 $10 = (1, 1, 1, 1, 0)$
 $11 = (1, 1, 1, 0, 1)$
 $12 = (1, 0, 0, 1, 1)$
 $13 = (1, 1, 0, 1, 1)$
 $14 = (1, 0, 1, 1, 1)$

The group is a matrix group.

The base action is on projective space $\text{PG}(4, 2)$

$q = 2$

$p = 2$

$e = 1$

$n = 4$

Number of points = 31

Number of lines = 155

Number of lines on a point = 15

Number of points on a line = 3

The finite field $\mathbb{F}_2$

$Z_i = \log_\alpha(1 + \alpha^i)$

i	γ_i	$-\gamma_i$	γ_i^{-1}	$\log_\alpha(\gamma_i)$	α^i	Z_i
0	$0 = 0$	0	DNE	DNE	1	DNE
1	$1 = 1$	1	1	0	1	DNE

+	0	1
0	0	1
1	1	0

·	1
1	1

$1^0 \equiv 1$

$1^1 \equiv 1$

Base and Stabilizer Chain

Group order 720

tl=15, 8, 3, 1, 1, 2,

Base: (0, 1, 2, 3, 4, 5)

Strong generators for a group of order 720:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix},$$

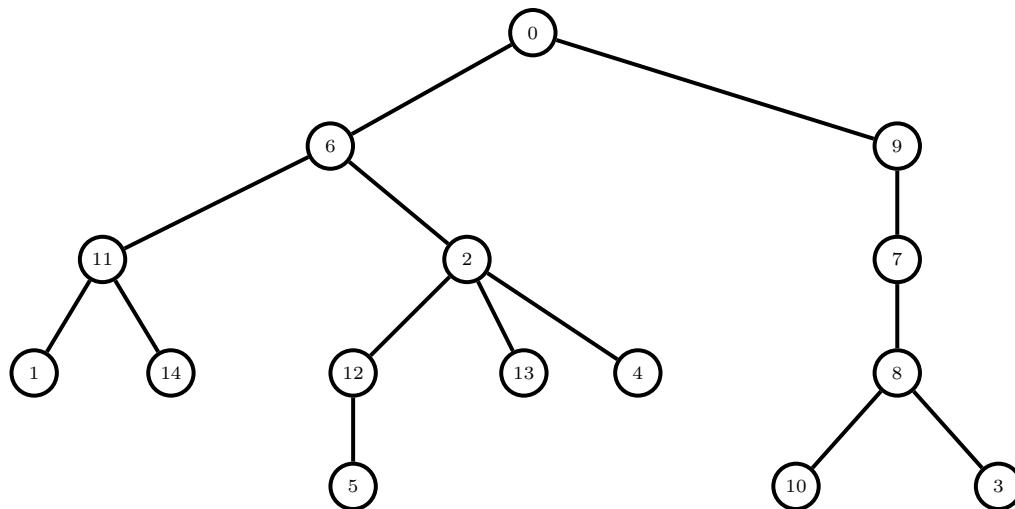
$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

1,0,0,0,0,0,1,0,0,0,0,0,1,0,0,0,0,1,0,1,0,0,1,1,
 1,0,0,0,0,0,1,0,0,0,0,0,1,0,0,1,0,0,1,1,0,0,0,0,1,
 1,0,0,0,0,0,1,0,0,0,1,1,1,0,1,1,1,0,1,1,0,0,0,0,1,
 1,0,0,0,0,0,1,0,0,0,0,1,0,1,1,1,0,1,1,0,1,0,1,0,
 1,0,0,0,0,0,1,0,0,0,0,0,1,0,1,1,1,0,1,1,0,1,0,1,0,
 1,0,0,0,0,1,1,1,0,0,0,0,1,0,0,0,0,0,1,0,0,0,0,0,1,
 1,0,0,0,0,0,1,0,0,1,0,0,0,1,0,0,0,0,1,1,0,0,0,0,1,

Stabilizer chain

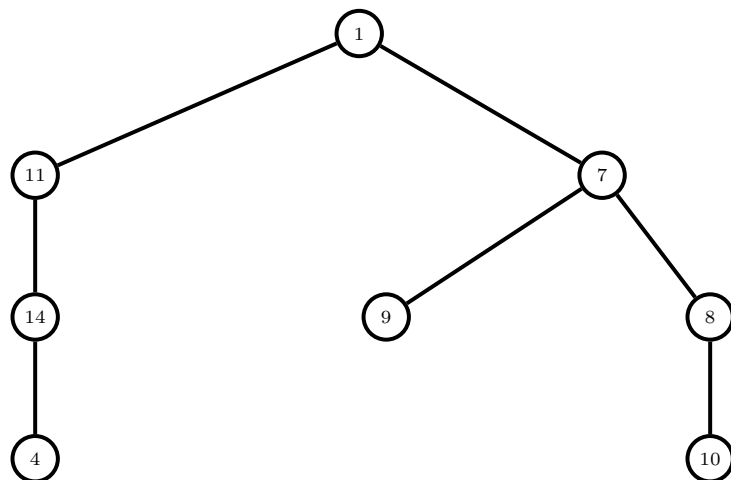
Level	Base pt	Orbit length	Subgroup order
0	0	15	720
1	1	8	48
2	2	3	6
3	3	1	2
4	4	1	2
5	5	2	2

Basic Orbit 0

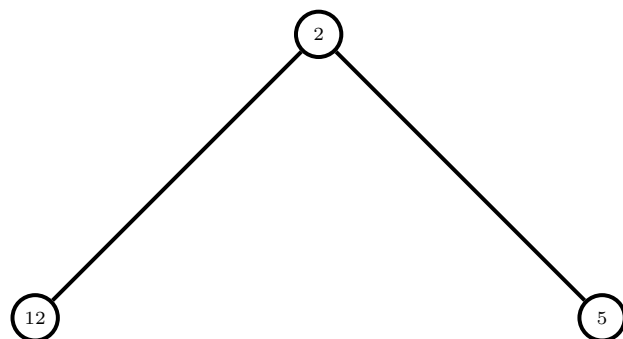


Basic orbit 0 has size 15

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14

Basic Orbit 1

Basic orbit 1 has size 8
1, 4, 7, 8, 9, 10, 11, 14

Basic Orbit 2

Basic orbit 2 has size 3
2, 5, 12

Basic Orbit 3

Basic orbit 3 has size 1
3

Basic Orbit 4

Basic orbit 4 has size 1
4

Basic Orbit 5

Basic orbit 5 has size 2

5, 12

GAP export:

Generators in GAP format are:

```
G := Group([(6, 13)(7, 14)(8, 15)(9, 12),
(3, 13)(4, 14)(5, 15)(9, 11),
(2, 12)(3, 14)(4, 13)(8, 10),
(2, 8, 9, 10, 12, 15)(3, 14, 7)(4, 13, 6)(5, 11),
(1, 10)(4, 11)(7, 12)(9, 14),
(1, 7)(3, 5)(4, 9)(10, 12)(11, 14)(13, 15)]);
```

Magma export:

Compact form:

Generators in compact permutation form are:

```
6 15
0 1 2 3 4 12 13 14 11 9 10 8 5 6 7
0 1 12 13 14 5 6 7 10 9 8 11 2 3 4
0 11 13 12 4 5 6 9 8 7 10 1 3 2 14
0 7 13 12 10 3 2 8 9 11 4 14 5 6 1
9 1 2 10 4 5 11 7 13 0 3 6 12 8 14
6 1 4 8 2 5 0 7 3 11 13 9 14 10 12
-1
```

The base has length 6

The basic orbits are:

Basic orbit 0 is orbit of 0 of length 15

Basic orbit 1 is orbit of 1 of length 8

Basic orbit 2 is orbit of 2 of length 3

Basic orbit 3 is orbit of 3 of length 1

Basic orbit 4 is orbit of 4 of length 1

Basic orbit 5 is orbit of 5 of length 2

The group order is 720.

The command

Code Snippet 146

```

1   PGG0_5_4:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define F4 -finite_field -q 4 -compute_related_fields -end \
7   ▷ ▷ -define G -linear_group -PGG0 5 F4 -end \
8   ▷ ▷ -with G -do \
9   ▷ ▷ -group_theoretic_activity \
10  ▷ ▷ ▷ -report Do \
11  ▷ ▷ -end
12  ▷ $(PDFLATEX) PGG0_5_4_report.tex
13  ▷ $(OPEN) PGG0_5_4_report.pdf
14

```

creates the group $\text{PFO}(5, 4)$ of order 1958400 acting on the 85 points of the $Q(4, 4)$ quadric.

The command

Code Snippet 147

```

1   PGOp_6_2:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 2 -end \
7   ▷ ▷ -define G -linear_group -PGOp 6 F -end \
8   ▷ ▷ -with G -do \
9   ▷ ▷ -group_theoretic_activity \
10  ▷ ▷ ▷ -report Do \
11  ▷ ▷ -end
12  ▷ $(PDFLATEX) PGOp_6_2_report.tex
13  ▷ $(OPEN) PGOp_6_2_report.pdf
14

```

creates the group $\text{PFO}^+(6, 2)$ of order 40320 acting on the 35 points of the $Q^+(5, 2)$ quadric.

The command

Code Snippet 148

```

1   PGOm_6_2:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 2 -end \
7   ▷ ▷ -define G -linear_group -PGOm 6 F -end \
8   ▷ ▷ -with G -do \
9   ▷ ▷ -group_theoretic_activity \

```

```

10  ▷ ▷ ▷ -report Do \
11  ▷ ▷ -end
12  ▷ $(PDFLATEX) PG0m_6_2_report.tex
13  ▷ $(OPEN) PG0m_6_2_report.pdf
14

```

creates the group $\text{PTO}^-(6, 2)$ of order 51840 acting on the 27 points of the $Q^-(5, 2)$ quadric.

The symplectic group $\text{PSp}(6, 2)$ can be created using the following command:

Code Snippet 149

```

1    PSP_6_2:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define Do -draw_options \
4    ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5    ▷ ▷ -end \
6    ▷ ▷ -define F -finite_field -q 2 -end \
7    ▷ ▷ -define G -linear_group -PGL 6 F \
8    ▷ ▷ ▷ -symplectic_group \
9    ▷ ▷ -end \
10   ▷ ▷ -with G -do \
11   ▷ ▷ -group_theoretic_activity \
12   ▷ ▷ ▷ -report Do \
13   ▷ ▷ -end
14   ▷ $(PDFLATEX) PGL_6_2_Sp_6_2_report.tex
15   ▷ $(OPEN) PGL_6_2_Sp_6_2_report.pdf
16

```

The group order is 1451520. The degree of the action is 63.

The group $\text{PGO}(7, 2)$, isomorphic to $\text{PSp}(6, 2)$, can be created using the following command:

Code Snippet 150

```

1    PGO_7_2:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define Do -draw_options \
4    ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5    ▷ ▷ -end \
6    ▷ ▷ -define F -finite_field -q 2 -end \
7    ▷ ▷ -define G -linear_group -PGO 7 F -end \
8    ▷ ▷ -with G -do \
9    ▷ ▷ -group_theoretic_activity \
10   ▷ ▷ ▷ -report Do \
11   ▷ ▷ -end
12   ▷ $(PDFLATEX) PGO_7_2_report.tex
13   ▷ $(OPEN) PGO_7_2_report.pdf
14

```

The group order is 1451520. The degree of the action is 63.

The command

Code Snippet 151

```

1    NullPolarity_6_2:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define Do -draw_options \
4    ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5    ▷ ▷ -end \
6    ▷ ▷ -define F -finite_field -q 2 -end \
7    ▷ ▷ -define G -linear_group -PGL 6 F \
8    ▷ ▷ ▷ -null_polarity_group \
9    ▷ ▷ -end \
10   ▷ ▷ -with G -do \
11   ▷ ▷ -group_theoretic_activity \
12   ▷ ▷ ▷ -report Do \
13   ▷ ▷ -end
14   ▷ $(PDFLATEX) PGL_6_2_NullPolarity_6_2_report.tex
15   ▷ $(OPEN) PGL_6_2_NullPolarity_6_2_report.pdf
16

```

creates the null polarity group of $\text{PG}(6, 2)$ acting on the 63 points of $\text{PG}(5, 2)$. This is the group preserving the dot product

$$\mathbf{x} \cdot \mathbf{y} = \sum_{i=1}^n x_i y_i.$$

The group order is 23040. The degree of the action is 63.

The command

Code Snippet 152

```

1    M24_matrix_group:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define Do -draw_options \
4    ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5    ▷ ▷ -end \
6    ▷ ▷ -define F -finite_field -q 2 -end \
7    ▷ ▷ -define G -linear_group -PGL 12 2 \
8    ▷ ▷ -subgroup_by_generators "M24" "244823040" 12 \
9    ▷ ▷ ▷ $(GOLAY_24_CODE_AUT_GENS) \
10   ▷ ▷ -end \
11   ▷ ▷ -with G -do \
12   ▷ ▷ -group_theoretic_activity \
13   ▷ ▷ ▷ -report Do \
14   ▷ ▷ -end
15   ▷ $(PDFLATEX) PGL_12_2_Subgroup_M24_244823040_report.tex
16   ▷ $(OPEN) PGL_12_2_Subgroup_M24_244823040_report.pdf
17

```

creates the matrix group M_{24} as a subgroup of $\text{PGL}(12, 2)$. The group is given by means of a generating set of 12 matrices, stored in the makefile variable

GOLAY_24_CODE_AUT_GENS.

Creating Subgroups		
Command	Arguments	Purpose
-Janko1		first Janko group, needs PGL(7, 11)
-monomial		subgroup of monomial matrices
-diagonal		subgroup of diagonal matrices
-null_polarity_group		null polarity group
-symplectic_group		symplectic group
-singer	k	subgroup of index k in the Singer cycle
-singer_and_frobenius	k	subgroup of index k in the Singer cycle, extended by the Frobenius automorphism of $\mathbb{F}_{q^n}$ over $\mathbb{F}_q$
-borel_upper		Borel subgroup of upper triangular matrices
-borel_lower		Borel subgroup of lower triangular matrices
-identity_group		identity subgroup
-subgroup_from_file	$f\ l$	read subgroup from file f and give it the label l
-orthogonal	ϵ	orthogonal group $O^\epsilon(n, q)$, with $\epsilon \in \{\pm 1\}$ when n is even
-subgroup_by_generators	$l\ o\ n\ s_1 \dots s_n$	Generate a subgroup from generators. The label "l" is used to denote the subgroup; o is the order of the subgroup; n is the number of generators and $s_1, \dots, s_n$ are the generators for the subgroup in vector form.

Table 5.5: Creating Subgroups

5.3 Subgroups

There are many ways to create subgroups of a group. Table 5.5 lists some commands to do so.

We start with an example of an explicit permutation group using a known base and strong generating set, using the `bsgs` command. Here is the cyclic group of order 13 acting on the permutation domain $[0, 12]$. The base is (0). When creating a group, we supply a label in ascii text and in tex. Then we specify the degree of the action, and the group order. After that, we specify the number of generators and the generators themselves. The labels will be used in reports about the group, for instance.

Code Snippet 153

```

1  GEN_C13="1,2,3,4,5,6,7,8,9,10,11,12,0"
2  # (0,1,2,3,4,5,6,7,8,9,10,11,12)
3
```

Code Snippet 154

```

1  C13:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define Do -draw_options \
4  ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5  ▷ ▷ -end \
```

```

6   ▷ ▷ -define gens -vector -dense $(GEN_C13) -end \
7   ▷ ▷ -define G -permutation.group \
8   ▷ ▷ ▷ -bsgs C13 C_{13} 13 13 0 1 \
9   ▷ ▷ ▷ ▷ gens \
10  ▷ ▷ -end \
11  ▷ ▷ -with G -do \
12  ▷ ▷ -group_theoretic_activity \
13  ▷ ▷ ▷ -export_orbiter \
14  ▷ ▷ -end \
15  ▷ ▷ -with G -do \
16  ▷ ▷ -group_theoretic_activity \
17  ▷ ▷ ▷ -export_group_table \
18  ▷ ▷ -end \
19  ▷ ▷ -with G -do \
20  ▷ ▷ -group_theoretic_activity \
21  ▷ ▷ ▷ -report Do \
22  ▷ ▷ -end \
23  ▷ ▷ -with G -do \
24  ▷ ▷ -group_theoretic_activity \
25  ▷ ▷ ▷ -save_elements_csv "C13.elts.csv" \
26  ▷ ▷ -end
27  ▷ $(PDFLATEX) C13_report.tex
28  ▷ $(OPEN) C13_report.pdf
29

```

The makefile variable GEN_C13 is used to define the generator of the group, which is the cycle

$$(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12).$$

The command creates the group from the known base 0. After that, several activities are invoked. Specifically, these are group theoretic activities. They will be discussed in more detail in Section 5.5.

Let us take a closer look at the three activities performed in this example. The `-export_orbiter` command exports the group in Orbiter makefile format. The file C13.makefile is generated, which can be used to recreate the permutation group in an Orbiter makefile. Here is the content of the file:

Code Snippet 155

```

1   C13_generated:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define gens -vector -file C13_gens.csv -end \
4   ▷ ▷ -define G -permutation.group \
5   ▷ ▷ ▷ -bsgs C13 "C_{13}" 13 13 "0" 1 gens \
6   ▷ ▷ -end \
7

```

The activity `-report` produces a report for the cyclic group, shown below:

Stabilizer chain

Level	Base pt	Orbit length	Subgroup order
0	0	13	13

Basic Orbit 0

Basic orbit 0 has size 13

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12

The command `-save_elements_csv` creates a csv file containing all group elements. Each group element is listed one-by-one, using the list notation of permutations. The csv file `C13_elts.csv` has the following content:

```

Row,Element
0,"0,1,2,3,4,5,6,7,8,9,10,11,12"
1,"1,2,3,4,5,6,7,8,9,10,11,12,0"
2,"2,3,4,5,6,7,8,9,10,11,12,0,1"
3,"3,4,5,6,7,8,9,10,11,12,0,1,2"
4,"4,5,6,7,8,9,10,11,12,0,1,2,3"
5,"5,6,7,8,9,10,11,12,0,1,2,3,4"
6,"6,7,8,9,10,11,12,0,1,2,3,4,5"
7,"7,8,9,10,11,12,0,1,2,3,4,5,6"
8,"8,9,10,11,12,0,1,2,3,4,5,6,7"
9,"9,10,11,12,0,1,2,3,4,5,6,7,8"
10,"10,11,12,0,1,2,3,4,5,6,7,8,9"
11,"11,12,0,1,2,3,4,5,6,7,8,9,10"
12,"12,0,1,2,3,4,5,6,7,8,9,10,11"
END

```

It is possible to create a permutation group as a subgroup of the symmetric group, using the known base for the symmetric group. Because the base of the symmetric group is large, this way of creating the group is less efficient than creating the group with a known (small) base. Here is an example. We create C_{13} as a subgroup of $\text{Sym}(13)$.

Code Snippet 156

```

1      C13_as_subgroup:
2      ▷ $(ORBITER) -v 2 \
3      ▷ ▷ -define Do -draw_options \
4      ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5      ▷ ▷ -end \
6      ▷ ▷ -define G -permutation_group -symmetric_group 13 \
7      ▷ ▷ ▷ -subgroup_by_generators C13 13 1 $(GEN_C13) \
8      ▷ ▷ -end \
9      ▷ ▷ -with G -do \
10     ▷ ▷ -group_theoretic_activity \
11     ▷ ▷ ▷ -export_orbiter \
12     ▷ ▷ -end \
13     ▷ ▷ -with G -do \
14     ▷ ▷ -group_theoretic_activity \
15     ▷ ▷ ▷ -report Do \
16     ▷ ▷ -end \
17     ▷ ▷ -with G -do \
18     ▷ ▷ -group_theoretic_activity \
19     ▷ ▷ ▷ -save_elements_csv "C13.elts.csv" \
20     ▷ ▷ -end
21     ▷ #$(PDFLATEX) Perm13.Subgroup.C13_13_report.tex
22     ▷ #$(OPEN) Perm13.Subgroup.C13_13_report.pdf
23
```

For instance, the command

Code Snippet 157

```

1      J1:
2      ▷ $(ORBITER) -v 2 \
3      ▷ ▷ -define Do -draw_options \
4      ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5      ▷ ▷ -end \
6      ▷ ▷ -define G -linear_group -PGL 7 11 -Janko1 -end \
7      ▷ ▷ -with G -do \
8      ▷ ▷ -group_theoretic_activity \
9      ▷ ▷ ▷ -report Do \
10     ▷ ▷ -end
11     ▷ #$(PDFLATEX) Janko1_report.tex
12     ▷ #$(OPEN) Janko1_report.pdf
13
```

creates the first Janko group as a subgroup of $\text{PGL}(7, 11)$.

The command

Code Snippet 158

```

1   PGL_3_11_singer:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define G -linear_group -PGL 3 11 -singer 19 -end \
7   ▷ ▷ -with G -do \
8   ▷ ▷ -group_theoretic_activity \
9   ▷ ▷ ▷ -report Do \
10  ▷ ▷ -end
11  ▷ $(PDFLATEX) PGL_3_11_Singer_3_11_19_report.tex
12  ▷ $(OPEN) PGL_3_11_Singer_3_11_19_report.pdf
13

```

creates a subgroup of the Singer cycle of order 7. The Singer cycle in $GL(d, q)$ is a generator for a subgroup of order $q^d - 1$. It induces an element of order $\frac{q^d - 1}{q - 1}$ on the associated projective geometry $PG(d - 1, q)$. The additional integer parameter k after the `-singer` command is used to create the subgroup of index k of the Singer cycle.

The command

Code Snippet 159

```

1   PGL_3_11_singer_and_frobenius:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define G -linear_group -PGL 3 11 \
7   ▷ ▷ ▷ -singer_and_frobenius 19 \
8   ▷ ▷ -end \
9   ▷ ▷ -with G -do \
10  ▷ ▷ -group_theoretic_activity \
11  ▷ ▷ ▷ -report Do \
12  ▷ ▷ -end
13  ▷ ▷ $(PDFLATEX) PGL_3_11_Singer_and_Frob3_11_19_report.tex
14  ▷ ▷ $(OPEN) PGL_3_11_Singer_and_Frob3_11_19_report.pdf
15

```

creates a subgroup of index 19 of the Singer cycle of $PG(2, 11)$, extended by a group of order 3 that arises from the field extension $\mathbb{F}_{11}^3$ over $\mathbb{F}_{11}$. The group created by this command has order 21.

The group $\text{Sym}(3)$ can be realized as a group of mappings of $\mathbb{R}^* = \mathbb{R} \cup \{\infty\}$. The six mappings are

$$z, \frac{1}{1-z}, \frac{1-z}{z}, \frac{1}{z}, 1-z, \frac{z}{1-z}.$$

We can find a projective representation acting on $PG(1, 3)$. The elements of $PG(1, 3)$ have the form

$$\left\{ \begin{bmatrix} z \\ 1 \end{bmatrix}, z \in \mathbb{F}_3 \right\} \cup \left\{ \begin{bmatrix} 1 \\ 0 \end{bmatrix} \right\}.$$

The representing elements in $\text{PGL}(2, 3)$ are

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 2 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 2 \\ 1 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 1 & 2 \end{bmatrix}.$$

Using the second and fourth matrix as generators, encoded as

$$0, 1, 2, 1, \quad 0, 1, 1, 0,$$

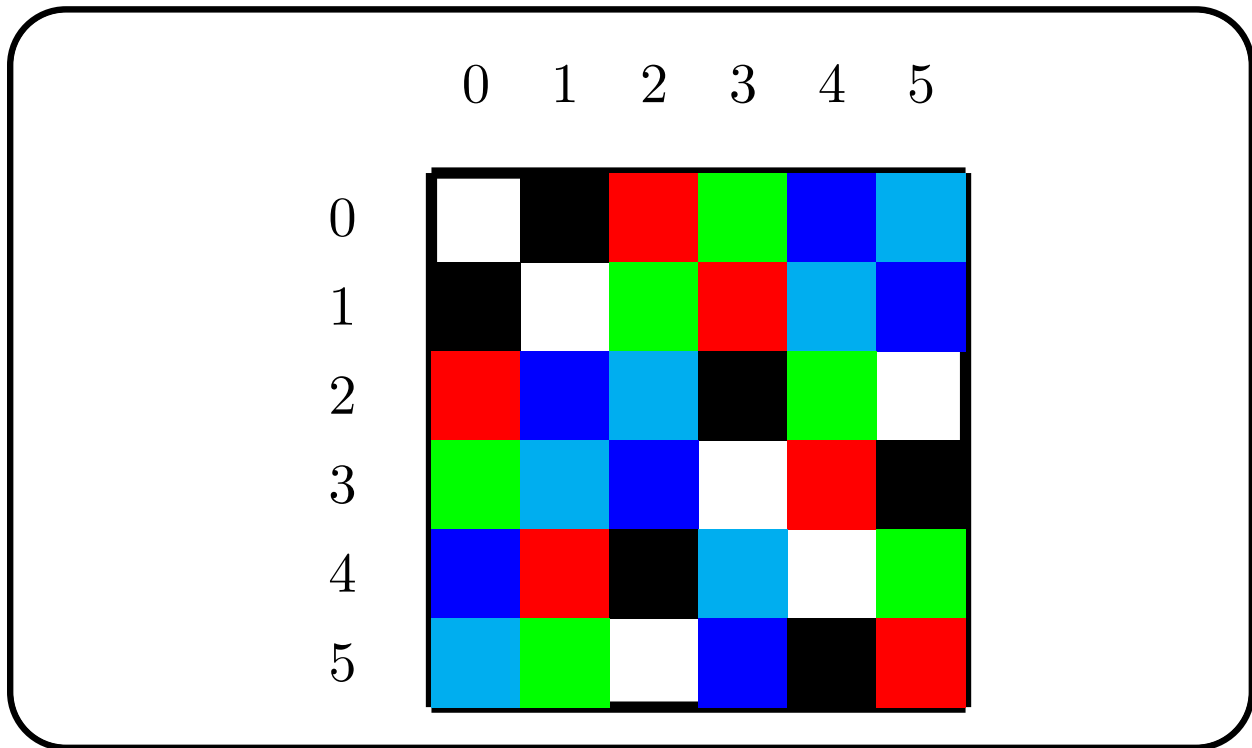
the following Orbiter command creates the group and produces the group table:

Code Snippet 160

```

1   Sym3_in_PGL_2_3_group_table:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -xout 10000 -yout 10000 \
5   ▷ ▷ ▷ -embedded -radius 50 \
6   ▷ ▷ ▷ -scale 0.1 -line_width 2 \
7   ▷ ▷ -end \
8   ▷ ▷ -define G -linear_group -PGL 2 3 \
9   ▷ ▷ -subgroup_by_generators "Sym3" "6" 2 \
10  ▷ ▷ ▷ "0,1,2,1, 0,1,1,0" \
11  ▷ ▷ -end \
12  ▷ ▷ -with G -do \
13  ▷ ▷ -group_theoretic_activity \
14  ▷ ▷ ▷ -report Do \
15  ▷ ▷ -end \
16  ▷ ▷ -with G -do \
17  ▷ ▷ -group_theoretic_activity \
18  ▷ ▷ ▷ -group_table Do \
19  ▷ ▷ -end
20  ▷ $(PDFLATEX) PGL_2_3_group_table_order_6.tex
21  ▷ $(OPEN) PGL_2_3_group_table_order_6.pdf
22
```

The group table is:



The next command creates a report of the group and the tree of elements, as well as a list of the group elements:

Code Snippet 161

```

1  Sym3.in_PGL_2.3_report:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define Do -draw.options -xout 1000000 -yout 1000000 \
4  ▷ ▷ ▷ -radius 250 \
5  ▷ ▷ ▷ -scale 0.5 -line_width 2 \
6  ▷ ▷ -end \
7  ▷ ▷ -define G -linear_group -PGL 2 3 \
8  ▷ ▷ -subgroup_by_generators "Sym3" "6" 2 \
9  ▷ ▷ ▷ "0,1,2,1, 0,1,1,0" \
10 ▷ ▷ -end \
11 ▷ ▷ -with G -do \
12 ▷ ▷ -group_theoretic_activity \
13 ▷ ▷ ▷ -report Do \
14 ▷ ▷ -end \
15 ▷ ▷ -with G -do \
16 ▷ ▷ -group_theoretic_activity \
17 ▷ ▷ ▷ -print_elements_tex \
18 ▷ ▷ -end \
19 ▷ ▷ -with G -do \
20 ▷ ▷ -group_theoretic_activity \
21 ▷ ▷ ▷ -make_element_tree \
22 ▷ ▷ -end
23 ▷ $(ORBITER) -v 2 \
24 ▷ ▷ -define Do -draw.options \

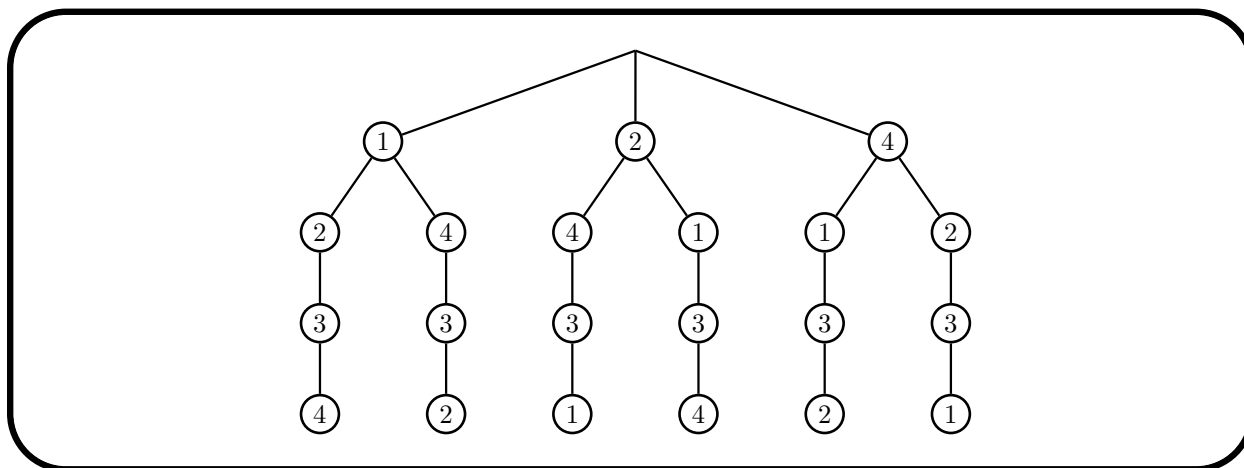
```

```

25  ▷ ▷ ▷ -nodes \
26  ▷ ▷ ▷ -embedded -radius 250 \
27  ▷ ▷ ▷ -xin 10000 -yin 10000 \
28  ▷ ▷ ▷ -xout 1000000 -yout 600000 \
29  ▷ ▷ ▷ -scale 0.3 -line_width 1.0 \
30  ▷ ▷ -end \
31  ▷ ▷ -tree_draw \
32  ▷ ▷ ▷ -file PGL_2_3_Subgroup_Sym3_6_elements_tree.txt \
33  ▷ ▷ ▷ -draw_options Do \
34  ▷ ▷ -end
35  ▷ $(PDFLATEX) PGL_2_3_Subgroup_Sym3_6_report.tex
36  ▷ $(OPEN) PGL_2_3_Subgroup_Sym3_6_report.pdf
37  ▷ $(PDFLATEX) PGL_2_3_Subgroup_Sym3_6_elements.tex
38  ▷ $(OPEN) PGL_2_3_Subgroup_Sym3_6_elements.pdf
39  ▷ $(PDFLATEX) PGL_2_3_Subgroup_Sym3_6_elements_tree_draw.tex
40  ▷ $(OPEN) PGL_2_3_Subgroup_Sym3_6_elements_tree_draw.pdf
41

```

The tree of elements is:



The list of elements is:

Group elements in action $\text{PGL}(2, 3)$

Element 0 / 6 of order 1:

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$(0)(1)(2)(3)$

0, 1, 2, 3,

Element 1 / 6 of order 2:

$$\begin{bmatrix} 2 & 0 \\ 2 & 1 \end{bmatrix}$$

$(0)(1, 3)(2)$

0, 3, 2, 1,

Element 2 / 6 of order 3:

$$\begin{bmatrix} 0 & 1 \\ 2 & 1 \end{bmatrix}$$

(0, 1, 3)(2)

1, 3, 2, 0,

Element 3 / 6 of order 2:

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

(0, 1)(2)(3)

1, 0, 2, 3,

Element 4 / 6 of order 3:

$$\begin{bmatrix} 1 & 2 \\ 1 & 0 \end{bmatrix}$$

(0, 3, 1)(2)

3, 0, 2, 1,

Element 5 / 6 of order 2:

$$\begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}$$

(0, 3)(1)(2)

3, 1, 2, 0,

Order structure:

$3^2, 2^3, 1$

The quaternion group is a group of order 8 generated by the following matrices over $\mathbb{R}$:

$$i = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad j = \begin{bmatrix} -1 & 1 \\ 1 & 1 \end{bmatrix}, \quad k = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}.$$

It is isomorphic to a subgroup of $\text{SL}(2,3)$. The Orbiter command

Code Snippet 162

```
1 quaternion:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define G -linear_group -SL 2 3 \
7   ▷ ▷ -subgroup_by_generators "quaternion" "8" 3 \
8   ▷ ▷ ▷ "1,1,1,2, 2,1,1,1, 0,2,1,0" \
9   ▷ ▷ -end \
10  ▷ ▷ -with G -do \
11  ▷ ▷ -group_theoretic_activity \
12  ▷ ▷ ▷ -group_table Do \
13  ▷ ▷ -end \
14  ▷ ▷ -with G -do \
15  ▷ ▷ -group_theoretic_activity \
16  ▷ ▷ ▷ -report Do \
```

```

17  ▷ ▷ -end \
18  ▷ ▷ -with G -do \
19  ▷ ▷ -group_theoretic_activity \
20  ▷ ▷ ▷ -print_elements_tex \
21  ▷ ▷ -end
22  ▷ $(PDFLATEX) SL_2_3_Subgroup_quaternion_8_report.tex
23  ▷ $(OPEN) SL_2_3_Subgroup_quaternion_8_report.pdf
24  ▷ $(PDFLATEX) SL_2_3_Subgroup_quaternion_8_group_table_report.tex
25  ▷ $(OPEN) SL_2_3_Subgroup_quaternion_8_group_table_report.pdf
26

```

creates the group. The command produces the list of group elements shown below.

Element 0 / 8 of order 1:

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$(0)(1)(2)(3)(4)(5)(6)(7)(8)$$

Element 1 / 8 of order 4:

$$\begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$$

$$(0)(1, 5, 2, 7)(3, 4, 6, 8)$$

Element 2 / 8 of order 2:

$$\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

$$(0)(1, 2)(3, 6)(4, 8)(5, 7)$$

Element 3 / 8 of order 4:

$$\begin{bmatrix} 1 & 2 \\ 2 & 2 \end{bmatrix}$$

$$(0)(1, 7, 2, 5)(3, 8, 6, 4)$$

Element 4 / 8 of order 4:

$$\begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$$(0)(1, 4, 2, 8)(3, 7, 6, 5)$$

Element 5 / 8 of order 4:

$$\begin{bmatrix} 0 & 1 \\ 2 & 0 \end{bmatrix}$$

$$(0)(1, 3, 2, 6)(4, 5, 8, 7)$$

Element 6 / 8 of order 4:

$$\begin{bmatrix} 2 & 2 \\ 2 & 1 \end{bmatrix}$$

$$(0)(1, 8, 2, 4)(3, 5, 6, 7)$$

Element 7 / 8 of order 4:

$$\begin{bmatrix} 0 & 2 \\ 1 & 0 \end{bmatrix}$$

$$(0)(1, 6, 2, 3)(4, 7, 8, 5)$$

The group table is created as csv file:

```
Row,C0,C1,C2,C3,C4,C5,C6,C7
0,0,1,2,3,4,5,6,7
1,1,2,3,0,5,6,7,4
2,2,3,0,1,6,7,4,5
3,3,0,1,2,7,4,5,6
4,4,7,6,5,2,1,0,3
5,5,4,7,6,3,2,1,0
6,6,5,4,7,0,3,2,1
7,7,6,5,4,1,0,3,2
END
```

The group of the cube can be created over the field $\mathbb{F}_3$:

Code Snippet 163

```
1   cube_group:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define gens -vector -dense \
7   ▷ ▷ ▷ "0,1,0,2,0,0,0,0,1, \
8   ▷ ▷ ▷ 0,0,1,0,1,0,2,0,0, \
9   ▷ ▷ ▷ 2,0,0,0,1,0,0,0,1" \
10  ▷ ▷ -end \
11  ▷ ▷ -define G -linear_group -GL 3 3 \
12  ▷ ▷ -subgroup_by_generators "cube" "48" 3 \
13  ▷ ▷ ▷ gens \
14  ▷ ▷ -end \
15  ▷ ▷ -with G -do \
16  ▷ ▷ -group_theoretic_activity \
17  ▷ ▷ ▷ -report Do \
18  ▷ ▷ -end \
19  ▷ ▷ -with G -do \
20  ▷ ▷ -group_theoretic_activity \
21  ▷ ▷ ▷ -print_elements_tex \
22  ▷ ▷ -end
23  ▷ $(PDFLATEX) GL_3_3_Subgroup_cube_48_report.tex
24  ▷ $(OPEN) GL_3_3_Subgroup_cube_48_report.pdf
25
```

The tetrahedral subgroup can be created as well:

Code Snippet 164

```

1   tetra_group:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define G -linear_group -GL 3 3 \
7   ▷ ▷ -subgroup_by_generators "tetra" "12" 2 \
8   ▷ ▷ ▷ "0,1,0,0,0,1,1,0,0, 0,0,1,2,0,0,0,2,0" \
9   ▷ ▷ -end \
10  ▷ ▷ -with G -do \
11  ▷ ▷ -group_theoretic_activity \
12  ▷ ▷ ▷ -report Do \
13  ▷ ▷ -end \
14  ▷ ▷ -with G -do \
15  ▷ ▷ -group_theoretic_activity \
16  ▷ ▷ ▷ -print_elements_tex \
17  ▷ ▷ -end
18  ▷ $(PDFLATEX) GL_3_3.Subgroup.tetra_12.report.tex
19  ▷ $(OPEN) GL_3_3.Subgroup.tetra_12.report.pdf
20

```

The Hesse group of order 216 extended by the automorphism group of the field can be created in PG(3,4)

Code Snippet 165

```

1   GENERATORS_HESSE_GROUP="\
2   3000300030 \
3   2000201230 \
4   1000100111 \
5   1000220200 \
6   1002312010 \
7   0331003211 \
8   2200011331"
9

```

Code Snippet 166

```

1   Hesse_group:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define gens -vector -compact \
7   ▷ ▷ ▷ $(GENERATORS_HESSE_GROUP) \
8   ▷ ▷ -end \
9   ▷ ▷ -define G -linear_group -PGGL 3 4 \
10  ▷ ▷ -subgroup_by_generators "Hesse" "432" 7 gens \
11  ▷ ▷ -end \
12  ▷ ▷ -with G -do \
13  ▷ ▷ -group_theoretic_activity \

```

```

14  ▷ ▷ ▷ -report Do \
15  ▷ ▷ -end \
16  ▷ ▷ -with G -do \
17  ▷ ▷ -group_theoretic_activity \
18  ▷ ▷ ▷ -print_elements_tex \
19  ▷ ▷ -end
20  ▷ $(PDFLATEX) PGGL_3_4_Subgroup_Hesse_432_report.tex
21  ▷ $(OPEN) PGGL_3_4_Subgroup_Hesse_432_report.pdf
22

```

The group has order 432.

The Weyl group of type E_8 can be generated as a subgroup of $GL(8, 3)$ using the following command:

Code Snippet 167

```

1  GENERATORS_WEYL_GROUP_E8="\
2  -1,-1,-1,-1,0,0,0,0, \
3  0,0,0,1,0,0,0,0, \
4  1,0,0,0,0,0,0,0, \
5  0,0,1,0,0,0,0,0, \
6  0,1,0,1,1,0,0,0, \
7  0,0,0,0,0,1,0,0, \
8  0,0,0,0,0,0,1,0, \
9  0,0,0,0,0,0,0,1, \
10 -1,0,-1,-1,-1,-1,-1,-1, \
11 0,1,0,1,1,1,1,1, \
12 1,0,0,0,0,0,0,0, \
13 0,0,1,0,0,0,0,0, \
14 0,0,0,1,0,0,0,0, \
15 0,0,0,0,1,0,0,0, \
16 0,0,0,0,0,1,0,0, \
17 0,0,0,0,0,0,1,0"
18

```

Code Snippet 168

```

1  Weyl_E8:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define Do -draw_options \
4  ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5  ▷ ▷ -end \
6  ▷ ▷ -define gens -vector -dense \
7  ▷ ▷ ▷ $(GENERATORS_WEYL_GROUP_E8) \
8  ▷ ▷ -end \
9  ▷ ▷ -define G -linear_group -GL 8 3 \
10 ▷ ▷ -subgroup_by_generators \
11 ▷ ▷ ▷ "Weyl_E8" "696729600" 2 gens \
12 ▷ ▷ -end \
13 ▷ ▷ -with G -do \
14 ▷ ▷ -group_theoretic_activity \
15 ▷ ▷ ▷ -report Do \

```

```

16  ▷ ▷ -end
17  ▷ $(PDFLATEX) GL_8_3_Subgroup_Weyl_E8_696729600_report.tex
18  ▷ $(OPEN) GL_8_3_Subgroup_Weyl_E8_696729600_report.pdf
19

```

A latex report is generated in the file `GL_8_3_Subgroup_Weyl_E8_696729600_report.tex`. This command uses generators found by Gabi Nebe:

<http://www.math.rwth-aachen.de/~Gabriele.Nebe/LATTICES/E8.b.html>

We can test if a group is a subgroup of another. In the following example, we test whether $\text{PGO}^+(6, 2)$ is a subgroup of $\text{PSp}(6, 2)$. The fact that it is depends on the choice of forms associated with the groups and on the fact that the characteristic is two.

Code Snippet 169

```

1    test_subgroup:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 2 -end \
4    ▷ ▷ -define G1 -linear_group -PGOp 6 F -end \
5    ▷ ▷ -define G2 -linear_group -PGL 6 F \
6    ▷ ▷ ▷ -symplectic_group \
7    ▷ ▷ -end \
8    ▷ ▷ -with G1 -and G2 -do \
9    ▷ ▷ -group_theoretic_activity \
10   ▷ ▷ ▷ -is_subgroup_of \
11   ▷ ▷ -end
12

```

Since the subgroup index is small (36), we create a set of coset representatives using the following command:

Code Snippet 170

```

1    coset_reps:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 2 -end \
4    ▷ ▷ -define G1 -linear_group -PGOp 6 F -end \
5    ▷ ▷ -define G2 -linear_group -PGL 6 F \
6    ▷ ▷ ▷ -symplectic_group \
7    ▷ ▷ -end \
8    ▷ ▷ -with G1 -and G2 -do \
9    ▷ ▷ -group_theoretic_activity \
10   ▷ ▷ ▷ -coset_reps \
11   ▷ ▷ -end
12   ▷ $(PDFLATEX) PGOp_6_2_coset_reps.tex
13   ▷ $(OPEN) PGOp_6_2_coset_reps.pdf
14

```

The coset representatives are written to a csv file. The (shortened) list of coset representatives in latex is:

coset 0 / 36:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 100000 \\ 010000 \\ 001000 \\ 000100 \\ 000010 \\ 000001 \end{bmatrix}$$

⋮

coset 35 / 36:

$$\begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 101110 \\ 101000 \\ 011101 \\ 011111 \\ 100010 \\ 110100 \end{bmatrix}$$

The following command reads the vector of coset representatives from the file just created.

Code Snippet 171

```

1  coset_reps_read:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -define G1 -linear_group -PGOp 6 F -end \
5  ▷ ▷ -define G2 -linear_group -PGL 6 F \
6  ▷ ▷ ▷ -symplectic_group \
7  ▷ ▷ -end \
8  ▷ ▷ -define CR -vector_ge -action G2 \
9  ▷ ▷ ▷ -read_csv \
10 ▷ ▷ ▷ PGOp_6_2_coset_reps.csv Element \
11 ▷ ▷ -end
12
```

The following command creates the group $\mathrm{PSp}(6, 2)$ and computes a point stabilizer subgroup.

Code Snippet 172

```

1  SP_6_2_point_stab_subgroup:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define Do -draw_options \
4  ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5  ▷ ▷ -end \
6  ▷ ▷ -define F -finite_field -q 2 -end \
7  ▷ ▷ -define G -linear_group -PGL 6 F \
8  ▷ ▷ ▷ -symplectic_group \
9  ▷ ▷ -end \
```

```

10  ▷ ▷ -define G0 -modified_group -from G \
11  ▷ ▷ ▷ -point_stabilizer 0 \
12  ▷ ▷ -end \
13  ▷ ▷ -with G0 -do \
14  ▷ ▷ -group_theoretic_activity \
15  ▷ ▷ ▷ -report Do \
16  ▷ ▷ -end
17  ▷ $(PDFLATEX) PSp_6_2_Stab0_report.tex
18  ▷ $(OPEN) PSp_6_2_Stab0_report.pdf
19

```

The next command creates the group $\text{PGO}^+(6, 2)$ and produces a report.

Code Snippet 173

```

1  PGOp_6_2_report:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define Do -draw_options \
4  ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5  ▷ ▷ -end \
6  ▷ ▷ -define F -finite_field -q 2 -end \
7  ▷ ▷ -define G -linear_group -PGOp 6 F -end \
8  ▷ ▷ -with G -do \
9  ▷ ▷ -group_theoretic_activity \
10 ▷ ▷ ▷ -report Do \
11 ▷ ▷ -end
12 ▷ $(PDFLATEX) PGOp_6_2_report.tex
13 ▷ $(OPEN) PGOp_6_2_report.pdf
14

```

The next command creates the group $\text{PGO}^-(6, 2)$ and produces a report.

Code Snippet 174

```

1  PGOm_6_2_report:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define Do -draw_options \
4  ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5  ▷ ▷ -end \
6  ▷ ▷ -define F -finite_field -q 2 -end \
7  ▷ ▷ -define G -linear_group -PGOm 6 F -end \
8  ▷ ▷ -with G -do \
9  ▷ ▷ -group_theoretic_activity \
10 ▷ ▷ ▷ -report Do \
11 ▷ ▷ -end
12 ▷ $(PDFLATEX) PGOm_6_2_report.tex
13 ▷ $(OPEN) PGOm_6_2_report.pdf
14

```

The next command creates the group $\text{PGO}^-(6, 2)$ and exports the group to Magma.

Code Snippet 175

```

1   PGOm_6_2.export_magma:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 2 -end \
4   ▷ ▷ -define G -linear_group -PGOm 6 F -end \
5   ▷ ▷ -with G -do \
6   ▷ ▷ -group_theoretic_activity \
7   ▷ ▷ ▷ -export_magma \
8   ▷ ▷ -end
9

```

The next command creates the group $\text{PGO}^+(6, 2)$ and computes a point stabilizer subgroup.

Code Snippet 176

```

1   PGOp_6_2.point_stab_subgroup:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 2 -end \
7   ▷ ▷ -define G -linear_group -PGOp 6 F -end \
8   ▷ ▷ -define G0 -modified_group -from G \
9   ▷ ▷ ▷ -point_stabilizer 0 \
10  ▷ ▷ -end \
11  ▷ ▷ -with G0 -do \
12  ▷ ▷ -group_theoretic_activity \
13  ▷ ▷ ▷ -report Do \
14  ▷ ▷ -end
15  ▷ $(PDFLATEX) PGOp_6_2_report.tex
16  ▷ $(OPEN) PGOp_6_2_report.pdf
17

```

We can save the generators of $\text{PGO}^+(6, 2)$ from the report to a makefile variable:

Code Snippet 177

```

1   PGOp_6_2.GENS="\
2   1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,1,0, \
3   1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,1,1,1,1,0,0,1,0,0,1,0,0,1,0,1,0, \
4   1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,1,0,0,0,1,0,0,1,0,0,0,0,1,0, \
5   1,0,0,0,0,0,0,1,0,0,0,0,0,0,0,1,1,0,0,0,0,0,0,1,0,0,1,0,0,1,0,0,0,1,0,0, \
6   1,0,0,0,0,0,0,1,0,0,0,0,0,0,0,1,0,0,0,0,1,0,1,0,0,0,0,0,1,0,0,0,0,1,0,1, \
7   1,0,0,0,0,0,0,1,1,1,1,1,1,0,0,0,1,0,1,0,0,0,0,1,1,0,1,0,0,0,1,0,0,1,0,0, \
8   1,0,0,0,0,0,0,1,0,1,0,0,1,0,1,0,0,1,0,0,0,1,0,0,0,0,0,1,1,0,0,0,0,0,1, \
9   0,1,0,1,1,0,0,0,1,0,0,0,0,1,0,0,0,0,1,0,1,0,1,0,0,0,0,0,1,0,0,1,1,0,0,1, \
10  0,0,1,1,1,1,1,0,1,0,0,0,0,1,1,1,1,1,1,1,1,1,0,0,0,1,0,1,0,1,0,1,0,1,1,0, \
11  0,0,0,0,1,0,1,0,1,1,1,1,1,0,1,0,1,0,0,1,1,0,1,0,0,0,0,0,1,1,1,1,1,0"
12

```

Using this variable, we can use the following command to recreate the group as a subgroup of $\text{PGL}(6, 2)$ and write a report:

Code Snippet 178

```

1   PGOp_6_2_linear:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 2 -end \
7   ▷ ▷ -define G -linear_group -PGL 6 F \
8   ▷ ▷ ▷ -subgroup_by_generators PGOp_6_2 \
9   ▷ ▷ ▷ ▷ 40320 10 $(PGOp_6_2_GENS) \
10  ▷ ▷ -end \
11  ▷ ▷ -with G -do \
12  ▷ ▷ -group_theoretic_activity \
13  ▷ ▷ ▷ -report Do \
14  ▷ ▷ -end
15  ▷ $(PDFLATEX) PGL_6_2_Subgroup_PGOp_6_2_40320_report.tex
16  ▷ $(OPEN) PGL_6_2_Subgroup_PGOp_6_2_40320_report.pdf
17

```

In the following command, we compute a point stabilizer subgroup:

Code Snippet 179

```

1   PGOp_6_2_linear_stab_6:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 2 -end \
7   ▷ ▷ -define G -linear_group -PGL 6 F \
8   ▷ ▷ ▷ -subgroup_by_generators PGOp_6_2 \
9   ▷ ▷ ▷ ▷ 40320 10 $(PGOp_6_2_GENS) \
10  ▷ ▷ -end \
11  ▷ ▷ -define G6 -modified_group -from G \
12  ▷ ▷ ▷ -point_stabilizer 6 \
13  ▷ ▷ -end \
14  ▷ ▷ -with G6 -do \
15  ▷ ▷ -group_theoretic_activity \
16  ▷ ▷ ▷ -report Do \
17  ▷ ▷ -end
18  ▷ $(PDFLATEX) PGL_6_2_Subgroup_PGOp_6_2_40320_Stab6_report.tex
19  ▷ $(OPEN) PGL_6_2_Subgroup_PGOp_6_2_40320_Stab6_report.pdf
20

```


5.4 New Actions from Old

It is possible to create new groups and group actions from old. This is done using the command `-modified_group`. Table 5.6 lists Orbiter commands that are available.

Suppose we are interested in the automorphism group of the cyclic group of order 6. The next command creates the group first and the automorphism group second. So, $G = C_6$ and $A = \text{Aut}(G)$. A report for the group A is produced. The automorphism group has order 2.

Code Snippet 180

```

1   Cyclic_6_aut:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define G -permutation_group -cyclic_group 6 -end \
7   ▷ ▷ -with G -do \
8   ▷ ▷ ▷ -group_theoretic_activity \
9   ▷ ▷ ▷ ▷ -report Do \
10  ▷ ▷ -end \
11  ▷ ▷ -define A -modified_group -from G \
12  ▷ ▷ ▷ -automorphism_group \
13  ▷ ▷ -end \
14  ▷ ▷ -with A -do \
15  ▷ ▷ ▷ -group_theoretic_activity \
16  ▷ ▷ ▷ ▷ -report Do \
17  ▷ ▷ -end
18  ▷ $(PDFLATEX) C_6_aut_report.tex
19  ▷ $(OPEN) C_6_aut_report.pdf
20

```

The next command creates $\text{Aut}(E_8)$, the automorphism group of an elementary abelian group of order 8, of order 168:

Code Snippet 181

```

1   Elementary_abelian_8_aut:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define G -permutation_group -elementary_abelian_group 8 -end \
7   ▷ ▷ -with G -do \
8   ▷ ▷ ▷ -group_theoretic_activity \
9   ▷ ▷ ▷ ▷ -report Do \
10  ▷ ▷ -end \
11  ▷ ▷ -define A -modified_group -from G \
12  ▷ ▷ ▷ -automorphism_group \
13  ▷ ▷ -end \
14  ▷ ▷ -with A -do \
15  ▷ ▷ ▷ -group_theoretic_activity \
16  ▷ ▷ ▷ ▷ -report Do \
17  ▷ ▷ -end

```

Group Modifications		
Command	Arguments	Purpose
-restricted_action	S label-tex	restricted action on the set S using the given tex-label.
-on_k_subspaces	k	induced action on k dimensional subspaces
-on_k_subsets	k	induced action on k subsets.
-on_wedge_product		action on the exterior square
-on_cosets_of_subgroup	H	Action on the cosets of H in the given group. H must be a subgroup of the given group.
-create_special_subgroup		Create the subgroup of special matrices.
-create_even_subgroup		Create the subgroup of even permutations.
-derived_subgroup		Create the derived subgroup.
-point_stabilizer	pt	Create the subgroup which stabilizes pt.
-set_stabilizer	set control	Create the subgroup which stabilizes the given set. Requires a poset classification control object to generate the orbits on subsets.
-projectivity_subgroup		Create the projectivity subgroup of a semilinear group.
-field_reduction		Create the field reduced group.
-action_on_self_by_right_multiplication		Create the action on the group itself by right multiplication.
-action_on_self_by_conjugation		Create the action on the group itself by conjugation.
-direct_product	list order gens	Create the direct product action of the two actions given in list (comma separated). Then create the subgroup of the given order from the given generators.
-polarity_extension	G0 P	Create the polarity extension of G0. P is the underlying projective space on which G0 acts.
-automorphism_group		Create the automorphism group of the input group (which is given in -from). This command requires Magma.
-subgroup_by_lattice	i	Create the representative of the i -th conjugacy class of subgroups in the lattice of subgroups. The conjugacy classes of subgroups must have been computed before using the group theoretic activity -subgroup_lattice_magma. A shortened base will be computed for the subgroup. The shortened base contains only those base points for which the basic orbit is non-trivial. See Section 5.8.
-stabilizer_of_variety	variety	Create the stabilizer of the given variety.
-subgroup_by_generators	elements	Create the subgroup generated by the given set of elements.
-nauty_control	options	Nauty options, see Table 17.3.
-from	label	Specify the input group action (this option can be repeated).

Table 5.6: Group Modifications

```

18   ▷ $(PDFLATEX) E_8_aut_report.tex
19   ▷ $(OPEN) E_8_aut_report.pdf
20

```

Of course, this group is just $GL(3, 2)$. However, this time the group is a permutation group of degree 8, namely employing the regular action on the elements of E_8 .

Let us create the action of $Sym(4)$ on pairs:

Code Snippet 182

```

1   Symmetric_4_on_pairs:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define G -permutation_group -symmetric_group 4 -end \
7   ▷ ▷ -define G_on_2subsets -modified_group -from G \
8   ▷ ▷ ▷ -on_k_subsets 2 \
9   ▷ ▷ -end \
10  ▷ ▷ -with G_on_2subsets -do \
11  ▷ ▷ -group_theoretic_activity \
12  ▷ ▷ ▷ -report Do \
13  ▷ ▷ -end
14  ▷ $(PDFLATEX) Sym_4_OnSubsets_2_report.tex
15  ▷ $(OPEN) Sym_4_OnSubsets_2_report.pdf
16

```

The next command creates $Alt(5)$, alternating group of degree 5. It is the smallest nonabelian simple group. The group is created as the even subgroup of $Sym(5)$.

Code Snippet 183

```

1   Alternating_5:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define Sym5 -permutation_group -symmetric_group 5 -end \
7   ▷ ▷ -define Alt5 -modified_group -from Sym5 \
8   ▷ ▷ ▷ -create_even_subgroup \
9   ▷ ▷ -end \
10  ▷ ▷ -with Alt5 -do \
11  ▷ ▷ -group_theoretic_activity \
12  ▷ ▷ ▷ -report Do \
13  ▷ ▷ -end \
14  ▷ ▷ -with Alt5 -do \
15  ▷ ▷ -group_theoretic_activity \
16  ▷ ▷ ▷ -group_table Do \
17  ▷ ▷ -end \
18  ▷ ▷ -with Alt5 -do \
19  ▷ ▷ -group_theoretic_activity \

```

```

20  ▷ ▷ ▷ -export_group_table \
21  ▷ ▷ -end
22  ▷ $(ORBITER) -v 2 \
23  ▷ ▷ -define all_one_r -vector -repeat 1 60 -end \
24  ▷ ▷ -define all_one_c -vector -repeat 1 60 -end \
25  ▷ ▷ -draw_matrix \
26  ▷ ▷ ▷ -input_csv_file \
27  ▷ ▷ ▷ ▷ Sym_5_EvenSub_group_table.csv \
28  ▷ ▷ ▷ -box_width 20 \
29  ▷ ▷ ▷ -bit_depth 24 \
30  ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
31  ▷ ▷ -end
32  ▷ $(PDFLATEX) Sym_5_EvenSub_report.tex
33  ▷ $(OPEN) Sym_5_EvenSub_report.pdf
34  ▷ $(PDFLATEX) Sym_5_EvenSub_group_table_report.tex
35  ▷ $(OPEN) Sym_5_EvenSub_group_table_report.pdf
36  ▷ magick Sym_5_EvenSub_group_table_draw.bmp Sym_5_EvenSub_group_table_draw.png
37  ▷ $(OPEN) Sym_5_EvenSub_group_table_draw.png
38

```

There is a different way to create the same group, namely as the derived subgroup of $Sym(5)$. The following command illustrates this method:

Code Snippet 184

```

1    Alternating_5_derived:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define Do -draw_options \
4    ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5    ▷ ▷ -end \
6    ▷ ▷ -define Sym5 -permutation_group -symmetric_group 5 -end \
7    ▷ ▷ -define Alt5 -modified_group -from Sym5 \
8    ▷ ▷ ▷ -derived_subgroup \
9    ▷ ▷ -end \
10   ▷ ▷ -with Alt5 -do \
11   ▷ ▷ -group_theoretic_activity \
12   ▷ ▷ ▷ -report Do \
13   ▷ ▷ -end
14   ▷ $(PDFLATEX) Sym_5_Derived_report.tex
15   ▷ $(OPEN) Sym_5_Derived_report.pdf
16

```

The next example creates a group in product action. Let C_n be the cyclic group of order n . The group we wish to create is C_{21} . Because of $C_{21} \simeq C_3 \times C_7$, there is a product action on a grid of type 3×7 . To create the cyclic group, we use the translation complement in the respective one-dimensional affine groups. This works because the numbers are prime. So, we create the group as a subgroup of $AGL(1, 3) \times AGL(1, 7)$. Recall the encoding of affine elements: The affine map

$$f(\mathbf{x}) = A\mathbf{x} + \mathbf{b}$$

with

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}, \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

is encoded in row-major ordering, with the entries of $\mathbf{b}$ following the entries of A

$$(a_{11}, a_{12}, \dots, a_{nn}, b_1, \dots, b_n).$$

The cyclic group is generated by the mapping

$$f : \mathbb{F}_3 \times \mathbb{F}_7 \rightarrow \mathbb{F}_3 \times \mathbb{F}_7, f(x, y) = (x + 1, y + 1),$$

which encodes as

$$(1, 1, 1, 1)$$

The command to create the product action and the subgroup of order 21 is:

Code Snippet 185

```
1  direct_product_AGL:
2  ▷ $(ORBITER) -v 4 \
3  ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4  ▷ ▷ -define Do -draw_options \
5  ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6  ▷ ▷ -end \
7  ▷ ▷ -define G1 -linear_group -AGL 1 3 \
8  ▷ ▷ ▷ -end \
9  ▷ ▷ -define G2 -linear_group -AGL 1 7 \
10 ▷ ▷ ▷ -end \
11 ▷ ▷ -define G -modified_group -direct_product "G1,G2" \
12 ▷ ▷ ▷ "21" "1,1,1,1" \
13 ▷ ▷ -end \
14 ▷ ▷ -with G -do \
15 ▷ ▷ -group_theoretic_activity \
16 ▷ ▷ ▷ -report Do \
17 ▷ ▷ -end
18 ▷ $(PDFLATEX) product_AGL_1_3_AGL_1_7_report.tex
19 ▷ $(OPEN) product_AGL_1_3_AGL_1_7_report.pdf
20
```

Let G be a linear group, acting on the vector space $\mathbb{F}_q^n$. The polarity extension G^* arises from G by extending with the standard polarity ρ , which sends a subspace U of V to $U^\perp$, where $\perp$ is the bilinear form $\sum_i x_i y_i$. The polarity group acts on the set of subspaces, so the degree is larger than the degree of G . For technical reasons, Orbiter creates a sum action, with an addition two element set. This set is an indicator set for the polarity. The two elements are permuted if the group element involves a polarity. So, the degree of the action of the polarity group is $2 + \sum_{i=1}^{n-1} d_i$ where d_i is the number of subspaces of algebraic dimension i . The polarity group G^* contains G as a subgroup of index two.

Let us do some examples of this construction.

The following command creates $\text{PGL}^*(3, 2)$ of order 336:

Code Snippet 186

```

1   PGLstar_3.2:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define Do -draw.options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 2 -end \
7   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
8   ▷ ▷ -define G0 -linear_group -PGL 3 F -end \
9   ▷ ▷ -define G -modified_group -polarity_extension \
10  ▷ ▷ ▷ G0 P \
11  ▷ ▷ -end \
12  ▷ ▷ -with G -do \
13  ▷ ▷ -group_theoretic_activity \
14  ▷ ▷ ▷ -report Do \
15  ▷ ▷ -end
16  ▷ $(PDFLATEX) PGL_3.2.polarity_ext.report.tex
17  ▷ $(OPEN) PGL_3.2.polarity_ext.report.pdf
18

```

The degree of the action is $16 = 2 + 7 + 7$.

The following command creates the same group, but the action is restricted to the 14 points and lines of $\text{PG}(2,2)$:

Code Snippet 187

```

1   PGLstar_3.2_restrict:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define Do -draw.options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 2 -end \
7   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
8   ▷ ▷ -define H -set -loop 2 14 1 -end \
9   ▷ ▷ -define G0 -linear_group -PGL 3 F -end \
10  ▷ ▷ -define G -modified_group -polarity_extension \
11  ▷ ▷ ▷ G0 P \
12  ▷ ▷ -end \
13  ▷ ▷ -define Gr -modified_group -from G \
14  ▷ ▷ ▷ -restricted_action H points\\_and\\_lines \
15  ▷ ▷ -end \
16  ▷ ▷ -with Gr -do \
17  ▷ ▷ -group_theoretic_activity \
18  ▷ ▷ ▷ -report Do \
19  ▷ ▷ -end
20  ▷ $(PDFLATEX) PGL_3.2.polarity_extension_res_H.report.tex
21  ▷ $(OPEN) PGL_3.2.polarity_extension_res_H.report.pdf
22

```

The degree of the action is $14 = 7 + 7$.

The following command creates $\text{PTL}^*(3,4)$ of order 241920:

Code Snippet 188

```

1   PGLstar_3.4:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define Do -draw.options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 4 -end \
7   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
8   ▷ ▷ -define G0 -linear_group -PGL 3 F -end \
9   ▷ ▷ -define G -modified_group -polarity_extension \
10  ▷ ▷ ▷ G0 P \
11  ▷ ▷ -end \
12  ▷ ▷ -with G -do \
13  ▷ ▷ -group_theoretic_activity \
14  ▷ ▷ ▷ -report Do \
15  ▷ ▷ -end
16  ▷ $(PDFLATEX) PGL_3.4_polarity_ext_report.tex
17  ▷ $(OPEN) PGL_3.4_polarity_ext_report.pdf
18

```

The degree of the action is $44 = 2 + 21 + 21$.

The following command creates $\text{PGL}^*(4, 2)$ of order 40320:

Code Snippet 189

```

1   PGLstar_4.2:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define Do -draw.options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 2 -end \
7   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
8   ▷ ▷ -define G0 -linear_group -PGL 4 F -end \
9   ▷ ▷ -define G -modified_group -polarity_extension \
10  ▷ ▷ ▷ G0 P \
11  ▷ ▷ -end \
12  ▷ ▷ -with G -do \
13  ▷ ▷ -group_theoretic_activity \
14  ▷ ▷ ▷ -report Do \
15  ▷ ▷ -end
16  ▷ $(PDFLATEX) PGL_4.2_polarity_ext_report.tex
17  ▷ $(OPEN) PGL_4.2_polarity_ext_report.pdf
18

```

The degree of the action is $67 = 2 + 15 + 35 + 15$.

We can have point stabilizers. To do so, we utilize a group modification. Continuing an example from Section 5.1, we consider the Mathieu group M_{12} , with its point stabilizer subgroup M_{11} of order 7920. The following command also produces a report for the subgroup.

Code Snippet 190

```

1   Group.M11:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define gens -vector -dense $(GENERATORS.M12) -end \
7   ▷ ▷ -define G -permutation_group -symmetric_group 12 \
8   ▷ ▷ ▷ -subgroup_by_generators M12 95040 2 gens -end \
9   ▷ ▷ -define G0 -modified_group -from G \
10  ▷ ▷ ▷ -point_stabilizer 0 \
11  ▷ ▷ -end \
12  ▷ ▷ -with G0 -do \
13  ▷ ▷ -group_theoretic_activity \
14  ▷ ▷ ▷ -report Do \
15  ▷ ▷ -end
16  ▷ $(PDFLATEX) Sym_12.Subgroup.M12.95040.Stab0.report.tex
17  ▷ $(OPEN) Sym_12.Subgroup.M12.95040.Stab0.report.pdf
18

```

In the following example, we will demonstrate two types of induced actions. One is the action induced on k -dimensional subspaces. The second is the restricted action on an invariant subset. The example we show is related to cubic surfaces. At first, we create the Eckardt surface in $\text{PG}(3, 13)$ from the arc

$$\{0, 1, 2, 3, 43, 113\}.$$

Then we export the set of 45 tritangent planes to file and we produce a report about the surface and its automorphism group. The next command creates the stabilizer of the surface from the generators given in the report, creates the induced action on planes, and restricts the action to the 45 tritangent planes stored in the file. The command sequence includes a makefile variable for the generators of the stabilizer of the surface:

Code Snippet 191

```

1   SURFACE_Q13_STAB_GENS="1,0,0,0,9,12,0,0,10,0,12,0,9,0,0,12, \
2   1,0,0,0,0,12,12,6,6,0,0,7,1,0,2,0, \
3   0,1,1,7,3,9,9,11,2,10,10,3,9,9,1,11"
4

```

Code Snippet 192

```

1   surface_q13.Eckardt_on_tritangent_planes:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4   ▷ ▷ -define Do -draw_options \
5   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6   ▷ ▷ -end \
7   ▷ ▷ -define F -finite_field -q 13 -end \
8   ▷ ▷ -define gens -vector -field F \
9   ▷ ▷ ▷ -dense $(SURFACE_Q13_STAB_GENS) \
10  ▷ ▷ -end \
11  ▷ ▷ -define TriP -set -file \

```



```

12  ▷ ▷ ▷ surface_arc_lifting_triangular_q13\
13  arc_0_1_2_3_43_113_tritangent_planes.csv \
14  ▷ ▷ -end \
15  ▷ ▷ -define G -linear_group -PGL 4 F \
16  ▷ ▷ ▷ -subgroup_by_generators "stab" \
17  ▷ ▷ ▷ ▷ 24 3 gens \
18  ▷ ▷ ▷ -end \
19  ▷ ▷ -define G_on_planes -modified_group -from G \
20  ▷ ▷ ▷ -on_k_subspaces 3 \
21  ▷ ▷ -end \
22  ▷ ▷ -define Gr -modified_group -from G_on_planes \
23  ▷ ▷ ▷ -restricted_action TriP TriP \
24  ▷ ▷ -end \
25  ▷ ▷ -with Gr -do \
26  ▷ ▷ -group_theoretic_activity \
27  ▷ ▷ ▷ -report Do \
28  ▷ ▷ -end \
29  ▷ ▷ -define Orb -orbits -group Gr \
30  ▷ ▷ ▷ -on_points \
31  ▷ ▷ -end \
32  ▷ ▷ -with Orb -do -orbits_activity \
33  ▷ ▷ ▷ -report Do \
34  ▷ ▷ -end \
35  ▷ ▷ -with Orb -do -orbits_activity \
36  ▷ ▷ ▷ -draw_tree 0 Do \
37  ▷ ▷ -end \
38  ▷ ▷ -with Orb -do -orbits_activity \
39  ▷ ▷ ▷ -draw_tree 1 Do \
40  ▷ ▷ -end \
41  ▷ ▷ -with Orb -do -orbits_activity \
42  ▷ ▷ ▷ -stabilizer 36 \
43  ▷ ▷ -end
44  ▷ $(PDFLATEX) PGL_4.13_Gr_4.3_res_TriP_orbits_report.tex
45  ▷ $(OPEN) PGL_4.13_Gr_4.3_res_TriP_orbits_report.pdf
46

```

Let us create the stabilizer of the Hirschfeld surface, isomorphic to the Weyl group of type E_6 , of order 51840. The group is created as a subgroup of $\text{PTL}(4, 4)$. The following command computes the projectivity subgroup, of order 25920, and produces a report.

Code Snippet 193

```

1  Hirschfeld_stab_projectivity_group:
2  ▷ $(ORBITER) -v 4 \
3  ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4  ▷ ▷ -define Do -draw_options \
5  ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6  ▷ ▷ -end \
7  ▷ ▷ -define G -linear_group -PGGL 4 4 \
8  ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld_Stab" \
9  ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \

```

```

10  ▷ ▷ -end \
11  ▷ ▷ -define G0 -modified_group -from G \
12  ▷ ▷ ▷ -projectivity_subgroup \
13  ▷ ▷ -end \
14  ▷ ▷ -with G0 -do \
15  ▷ ▷ -group_theoretic_activity \
16  ▷ ▷ ▷ -report Do \
17  ▷ ▷ -end
18  ▷ $(PDFLATEX) PGGL_4_4_Subgroup_Hirschfeld_Stab_51840_ProjectivitySubgroup_report
    .tex
19  ▷ $(OPEN) PGGL_4_4_Subgroup_Hirschfeld_Stab_51840_ProjectivitySubgroup_report.pdf
20

```

The following command performs field reduction on the group of the Hirschfeld surface. The resulting group has order 155520 and is a subgroup of $\text{PGL}(8, 2)$.

Code Snippet 194

```

1  Hirschfeld_stab_field_reduction:
2  ▷ $(ORBITER) -v 4 \
3  ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4  ▷ ▷ -define Do -draw_options \
5  ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6  ▷ ▷ -end \
7  ▷ ▷ -define G -linear_group -PGGL 4 4 \
8  ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld_Stab" \
9  ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
10 ▷ ▷ -end \
11 ▷ ▷ -define G0 -modified_group -from G \
12 ▷ ▷ ▷ -field_reduction \
13 ▷ ▷ -end \
14 ▷ ▷ -with G0 -do \
15 ▷ ▷ -group_theoretic_activity \
16 ▷ ▷ ▷ -report Do \
17 ▷ ▷ -end
18 ▷ $(PDFLATEX) PGL_8_2_Field.Reduction_report.tex
19 ▷ $(OPEN) PGL_8_2_Field.Reduction_report.pdf
20

```

In the next example, we consider the action of

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 2 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

in $\text{PGL}(4, 8)$ on the set of planes through a the fixed line 0. The Orbiter ranks of the set of planes through the line have been computed in Section 7.1. They are

$$0, 8, 1, 6, 4, 3, 7, 2, 5,$$

and are given as the first argument to the `apply` command. The second argument is the coding of the group element.

Code Snippet 195

```

1   on_planes:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 8 -end \
4   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5   ▷ ▷ -define G -linear_group -PGL 4 F -end \
6   ▷ ▷ -define G_on_planes -modified_group -from G \
7   ▷ ▷ ▷ -on_k_subspaces 3 \
8   ▷ ▷ -end \
9   ▷ ▷ -with G_on_planes -do \
10  ▷ ▷ -group_theoretic_activity \
11  ▷ ▷ ▷ -apply "0,8,1,6,4,3,7,2,5" \
12  ▷ ▷ ▷ "1,0,0,0, 0,1,0,0, 0,0,0,2, 0,0,1,1" \
13  ▷ ▷ -end
14  ▷ $(PDFLATEX) PGL_4.8_Gr_4.3_apply.tex
15  ▷ $(OPEN) PGL_4.8_Gr_4.3_apply.pdf
16

```

The `apply` command computes the permutation representation of the group element. In order to create the group acting on the planes, we use a restricted action. At first, we create the cyclic group of order 9 as a subgroup of $\text{PGL}(4, 8)$. Then we create the action on planes and restrict the action to the 9 planes through the line that we fix. Here is the command:

Code Snippet 196

```

1   on_planes_restricted:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define planes -vector -dense "0,8,1,6,4,3,7,2,5" -end \
7   ▷ ▷ -define F -finite_field -q 8 -end \
8   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
9   ▷ ▷ -define G -linear_group -PGL 4 F \
10  ▷ ▷ ▷ -subgroup_by_generators "cyclic" \
11  ▷ ▷ ▷ 9 1 "1,0,0,0, 0,1,0,0, 0,0,0,2, 0,0,1,1" \
12  ▷ ▷ -end \
13  ▷ ▷ -define G_on_planes -modified_group -from G \
14  ▷ ▷ ▷ -on_k_subspaces 3 \
15  ▷ ▷ -end \
16  ▷ ▷ -define Gr -modified_group -from G_on_planes \
17  ▷ ▷ ▷ -restricted_action planes planes \
18  ▷ ▷ -end \
19  ▷ ▷ -with Gr -do \
20  ▷ ▷ -group_theoretic_activity \
21  ▷ ▷ ▷ -report Do \
22  ▷ ▷ -end
23  ▷ $(PDFLATEX) PGL_4.8_Gr_4.3_res_planes_report.tex
24  ▷ $(OPEN) PGL_4.8_Gr_4.3_res_planes_report.pdf

```

Creating Special Actions		
Command	Arguments	Purpose
<code>-wedge_detached</code>		action on the exterior square. Unlike <code>-wedge</code> , this command does not establish the homomorphism to the original group. Instead, the group is created as subgroup of the larger general linear group.
<code>-PGL2nConic</code>		induced action of $\text{PGL}(2, q)$ on the conic in the plane $\text{PG}(2, q)$
<code>-subfield_structure_action</code>	s	action by field reduction to the subfield of index s
<code>-on_tensors</code>		induced action of $\text{GL}(d, q) \wr \text{Sym}(n)$ on the tensor space
<code>-on_rank_one_tensors</code>		induced action of $\text{GL}(d, q) \wr \text{Sym}(n)$ on the tensor space

Table 5.7: Creating Special Actions

25

Table 5.7 lists specific actions that can be requested when creating a linear group. Let us consider an example of a tensor product action. For instance, the command

Code Snippet 197

```

1   T3_on_tensors:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define G \
7   ▷ ▷ -linear_group -GL_d.q.wr_Sym_n 2 2 3 \
8   ▷ ▷ ▷ -on_tensors -end \
9   ▷ ▷ -with G -do \
10  ▷ ▷ -group_theoretic_activity \
11  ▷ ▷ ▷ -report Do \
12  ▷ ▷ -end
13  ▷ $(PDFLATEX) GL_2_2_wreath_Sym3_report.tex
14  ▷ $(OPEN) GL_2_2_wreath_Sym3_report.pdf
15  ▷
16

```

creates the group $\text{GL}(2, 2) \wr \text{Sym}(3)$ acting on the 255 elements of $\text{PG}(7, 2)$ which are identified with the tensors of type $(2, 2, 2)$ over $\mathbb{F}_2$. Elements of this group are denoted in the notation of the semidirect product. A vector of elements in the linear group is followed by a permutation of the components.

The Group $GL(2, 2) \wr \text{Sym}(3)$

The order of the group $GL(2, 2) \wr \text{Sym}(3)$ is 1296

The group acts on a set of size 255

The Action

Group action $GL(2, 2) \wr \text{Sym}(3)$ res255 of degree 255

Base and Stabilizer Chain

Group order 1296

tl=3, 2, 1, 3, 2, 3, 2, 3, 2,

Strong generators for a group of order 1296.

$$\begin{aligned} & \left(\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}; id \right), \left(\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}; id \right), \left(\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; id \right), \\ & \left(\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; id \right), \left(\begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; id \right), \left(\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; id \right), \\ & \left(\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; (1, 2) \right), \left(\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; (0, 1) \right) \end{aligned}$$

0,1,2,1,0,0,1,1,0,0,1,1,0,1,1,
 0,1,2,1,0,0,1,1,0,0,1,0,1,1,0,
 0,1,2,1,0,0,1,1,0,1,1,1,0,0,1,
 0,1,2,1,0,0,1,0,1,1,0,1,0,0,1,
 0,1,2,1,0,1,1,1,0,0,1,1,0,0,1,
 0,1,2,0,1,1,0,1,0,0,1,1,0,0,1,
 0,2,1,1,0,0,1,1,0,0,1,1,0,0,1,
 1,0,2,1,0,0,1,1,0,0,1,1,0,0,1,

Stabilizer chain

Level	Base pt	Orbit length	Subgroup order
0	0	3	1296
1	1	2	432
2	2	1	216
3	4	3	216
4	5	2	72
5	8	3	36
6	9	2	12
7	12	3	6
8	13	2	2

It is also possible to restrict the action on all rank-one tensors, as the following example shows:

Code Snippet 198

```

1   T3r1:
2   ▷ $(ORBITER) -v 4 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define G \
7   ▷ ▷ -linear_group -GL_d.q.wr.Sym.n 2 2 3 \
8   ▷ ▷ ▷ -on_rank_one_tensors -end \
9   ▷ ▷ -with G -do \
10  ▷ ▷ -group_theoretic_activity \
11  ▷ ▷ ▷ -report Do \
12  ▷ ▷ -end
13  ▷ $(PDFLATEX) GL_2.2.wreath_Sym3_report.tex
14  ▷ $(OPEN) GL_2.2.wreath_Sym3_report.pdf
15

```

This creates an action of degree 27. The Orbiter report shows all points in the permutation domain.

The Group $GL(2, 2) \wr \text{Sym}(3)$

The order of the group $GL(2, 2) \wr \text{Sym}(3)$ is 1296

The group acts on a set of size 27

The Action

Group action $GL(2, 2) \wr \text{Sym}(3)_{\text{res}27}$ of degree 27

We act on the following set:

0 = (1, 0, 0, 0, 0, 0, 0, 0)	14 = (0, 0, 0, 0, 0, 0, 1, 1)
1 = (0, 1, 0, 0, 0, 0, 0, 0)	15 = (0, 0, 0, 0, 1, 0, 1, 0)
2 = (1, 1, 0, 0, 0, 0, 0, 0)	16 = (0, 0, 0, 0, 0, 1, 0, 1)
3 = (0, 0, 1, 0, 0, 0, 0, 0)	17 = (0, 0, 0, 0, 1, 1, 1, 1)
4 = (0, 0, 0, 1, 0, 0, 0, 0)	18 = (1, 0, 0, 0, 1, 0, 0, 0)
5 = (0, 0, 1, 1, 0, 0, 0, 0)	19 = (0, 1, 0, 0, 0, 1, 0, 0)
6 = (1, 0, 1, 0, 0, 0, 0, 0)	20 = (1, 1, 0, 0, 1, 1, 0, 0)
7 = (0, 1, 0, 1, 0, 0, 0, 0)	21 = (0, 0, 1, 0, 0, 0, 1, 0)
8 = (1, 1, 1, 1, 0, 0, 0, 0)	22 = (0, 0, 0, 1, 0, 0, 0, 1)
9 = (0, 0, 0, 0, 1, 0, 0, 0)	23 = (0, 0, 1, 1, 0, 0, 1, 1)
10 = (0, 0, 0, 0, 0, 1, 0, 0)	24 = (1, 0, 1, 0, 1, 0, 1, 0)
11 = (0, 0, 0, 0, 1, 1, 0, 0)	25 = (0, 1, 0, 1, 0, 1, 0, 1)
12 = (0, 0, 0, 0, 0, 0, 1, 0)	26 = (1, 1, 1, 1, 1, 1, 1, 1)
13 = (0, 0, 0, 0, 0, 0, 0, 1)	

Base and Stabilizer Chain

Group order 1296

tl=3, 2, 1, 3, 2, 3, 2, 3, 2,

Strong generators for a group of order 1296:

$$\left(\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}; id \right), \left(\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}; id \right), \left(\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; id \right),$$

$$\left(\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; id \right), \left(\begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; id \right), \left(\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; id \right),$$

$$\left(\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; (1, 2) \right), \left(\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; (0, 1) \right)$$

0,1,2,1,0,0,1,1,0,0,1,1,0,1,1,
 0,1,2,1,0,0,1,1,0,0,1,0,1,1,0,
 0,1,2,1,0,0,1,1,0,1,1,1,0,0,1,
 0,1,2,1,0,0,1,0,1,1,0,1,0,0,1,
 0,1,2,1,0,1,1,1,0,0,1,1,0,0,1,
 0,1,2,0,1,1,0,1,0,0,1,1,0,0,1,
 0,2,1,1,0,0,1,1,0,0,1,1,0,0,1,
 1,0,2,1,0,0,1,1,0,0,1,1,0,0,1,

Orbiter supports the induced action on the wedge product. The following command shows how the induced action of $\text{PGL}(4, 2)$ on the wedge product is created:

Code Snippet 199

```

1   PGL_4_2_wedge:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define G -linear_group -PGL 4 2 -end \
7   ▷ ▷ -define G2 -modified_group -from G \
8   ▷ ▷ ▷ -on_wedge_product \
9   ▷ ▷ -end \
10  ▷ ▷ -with G -do \
11  ▷ ▷ -group_theoretic_activity \
12  ▷ ▷ ▷ -report Do \
13  ▷ ▷ -end
14  ▷ $(PDFLATEX) PGL_4_2_report.tex
15  ▷ $(OPEN) PGL_4_2_report.pdf
16
```

This creates a matrix group of order 20160 and dimension 4. The representation is only evaluated when needed, for instance when acting on elements of the wedge product.

A different way of creating wedge actions is by applying an attribute during the creation of the group. This has the effect that the group is created in the wedge representation right away. The following example creates the representation of $\text{PGL}(4, 2)$ in the wedge product action.

Code Snippet 200

```

1   PGL_4.2_wd:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define G -linear_group -PGL 4 2 -wedge_detached -end \
7   ▷ ▷ -with G -do \
8   ▷ ▷ -group_theoretic_activity \
9   ▷ ▷ ▷ -report Do \
10  ▷ ▷ -end \
11  ▷ ▷ -print_symbols
12  ▷ $(PDFLATEX) PGL_6.2.Wedge_4.2.detached_report.tex
13  ▷ $(OPEN) PGL_6.2.Wedge_4.2.detached_report.pdf
14

```

This creates a matrix group of order 20160 and dimension 10.

The next command creates the permutation representation of M_{24} in the action on a set of size 24 of points on $\text{PG}(11, 2)$.

Code Snippet 201

```

1   M24_perm_group:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define v -vector \
7   ▷ ▷ ▷ -dense $(GOLAY_24_CODE_PROJECTIVE) \
8   ▷ ▷ -end \
9   ▷ ▷ -define F -finite_field -q 2 -end \
10  ▷ ▷ -define G -linear_group -PGL 12 2 \
11  ▷ ▷ -subgroup_by_generators "M24" "244823040" 12 \
12  ▷ ▷ ▷ $(GOLAY_24_CODE_AUT_GEN) \
13  ▷ ▷ -end \
14  ▷ ▷ -define M24_perm -modified_group -from G \
15  ▷ ▷ ▷ -restricted_action v \_on\_code \
16  ▷ ▷ -end \
17  ▷ ▷ -with M24_perm -do \
18  ▷ ▷ -group_theoretic_activity \
19  ▷ ▷ ▷ -report Do \
20  ▷ ▷ -end
21  ▷ $(PDFLATEX) PGL_12.2.res.v.report.tex
22  ▷ $(OPEN) PGL_12.2.res.v.report.pdf
23

```

Next, we compute the two point set-stabilizer in M_{24} . Because M_{24} is five-fold transitive on the set of 24 points, it does not matter which 2 points are chosen. We pick the points 0 and 1.

Code Snippet 202

```

1  M22_bar:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define Do -draw_options \
4  ▷ ▷ ▷ -radius 50 -scale 0.5 -line_width 0.3 \
5  ▷ ▷ -end \
6  ▷ ▷ -define code -vector \
7  ▷ ▷ ▷ -dense $(GOLAY_24_CODE_PROJECTIVE) \
8  ▷ ▷ -end \
9  ▷ ▷ -define Control2 -poset_classification_control \
10 ▷ ▷ ▷ -problem_label M24_pairs -W -depth 2 \
11 ▷ ▷ ▷ -draw_options Do \
12 ▷ ▷ -end \
13 ▷ ▷ -define F -finite_field -q 2 -end \
14 ▷ ▷ -define G -linear_group -PGL 12 2 \
15 ▷ ▷ -subgroup_by_generators "M24" "244823040" 12 \
16 ▷ ▷ ▷ $(GOLAY_24_CODE_AUT_GENES) \
17 ▷ ▷ -end \
18 ▷ ▷ -define M24_perm -modified_group \
19 ▷ ▷ ▷ -from G -restricted_action code \\_on\\_code \
20 ▷ ▷ -end \
21 ▷ ▷ -define M22_perm -modified_group \
22 ▷ ▷ ▷ -from M24_perm \
23 ▷ ▷ ▷ -set_stabilizer 0,1 Control2 \
24 ▷ ▷ -end \
25 ▷ ▷ -define pts -set -loop 2 22 1 -end \
26 ▷ ▷ -define M22 -modified_group \
27 ▷ ▷ ▷ -from M22_perm \
28 ▷ ▷ ▷ -restricted_action pts M22 \
29 ▷ ▷ -end \
30 ▷ ▷ -with M22 -do \
31 ▷ ▷ -group_theoretic_activity \
32 ▷ ▷ ▷ -report Do \
33 ▷ ▷ -end
34 ▷ $(PDFLATEX) PGL_12_2_res_code_res_pts_report.tex
35 ▷ $(OPEN) PGL_12_2_res_code_res_pts_report.pdf
36

```

The next command creates the stabilizer of a variety. We continue an example from Section 7.5. The variety is defined in a makefile variable:

Code Snippet 203

```

1  CURVE_Q13_F0_CLEAN2_EQUATION="X0^4 + 4*X1^4 + 9*X2^4 + X0^2*X1^2 + 4*X0^2*X2^2 +
2  3*X1^2*X2^2"

```

We have seen how to create the variety object, and how to compute the group. The following command computes the group again, but then creates an Orbiter object for it:

Code Snippet 204

```

1   curve.q13_f0_clean2_variety_make_group:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define Do -draw.options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 13 -end \
7   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
8   ▷ ▷ -define R -polynomial_ring \
9   ▷ ▷ ▷ -field F \
10  ▷ ▷ ▷ -number_of_variables 3 \
11  ▷ ▷ ▷ -homogeneous_of_degree 4 \
12  ▷ ▷ ▷ -monomial_ordering_partition \
13  ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
14  ▷ ▷ -end \
15  ▷ ▷ -define V -variety \
16  ▷ ▷ ▷ -projective_space P \
17  ▷ ▷ ▷ -ring R \
18  ▷ ▷ ▷ -equation_in_algebraic_form \
19  ▷ ▷ ▷ ▷ $(CURVE_Q13_F0_CLEAN2_EQUATION) \
20  ▷ ▷ ▷ -label_txt curve.q13_f0_clean2 \
21  ▷ ▷ ▷ -label_tex "{\\rm curve\\_q13\\_f0\\_clean2}" \
22  ▷ ▷ -end \
23  ▷ ▷ -define G -modified_group \
24  ▷ ▷ ▷ -stabilizer_of_variety V \
25  ▷ ▷ ▷ -nauty_control -nauty_log "quartic.curve.q13.f0.nauty.log.txt" -end \
26  ▷ ▷ -end \
27  ▷ ▷ -with G -do \
28  ▷ ▷ -group_theoretic_activity \
29  ▷ ▷ ▷ -report Do \
30  ▷ ▷ -end
31  ▷ $(PDFLATEX) PGL_3_13_stab_of_curve.q13_f0_clean2_report.tex
32  ▷ $(OPEN) PGL_3_13_stab_of_curve.q13_f0_clean2_report.pdf
33  ▷ $(ORBITER) -v 2 \
34  ▷ ▷ -define Do -draw.options -embedded -radius 150 \
35  ▷ ▷ ▷ -nodes_empty \
36  ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
37  ▷ ▷ -tree_draw \
38  ▷ ▷ ▷ -file quartic.curve.q13_f0_nauty_log.txt \
39  ▷ ▷ ▷ -draw_options Do \
40  ▷ ▷ -end
41  ▷ $(PDFLATEX) quartic.curve.q13_f0_nauty_log_draw.tex
42  ▷ $(OPEN) quartic.curve.q13_f0_nauty_log_draw.pdf
43

```

Once the group object is created, we can apply group theoretic activities like producing a report for the group.

Let us compute the stabilizer of a conic over the field $\mathbb{F}_5$:

Code Snippet 205

```

1   conic_group.q5:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 5 -end \
7   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
8   ▷ ▷ -define R -polynomial_ring \
9   ▷ ▷ ▷ -field F \
10  ▷ ▷ ▷ -number_of_variables 3 \
11  ▷ ▷ ▷ -homogeneous_of_degree 2 \
12  ▷ ▷ ▷ -monomial_ordering_partition \
13  ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
14  ▷ ▷ -end \
15  ▷ ▷ -define V -variety \
16  ▷ ▷ ▷ -projective_space P \
17  ▷ ▷ ▷ -ring R \
18  ▷ ▷ ▷ -equation_in_algebraic_form "X1^2-X0*X2" \
19  ▷ ▷ ▷ -label_txt conic_q5 \
20  ▷ ▷ ▷ -label_tex "{\\rm conic\\_q5}" \
21  ▷ ▷ -end \
22  ▷ ▷ -define G -modified_group \
23  ▷ ▷ ▷ -stabilizer_of_variety V \
24  ▷ ▷ ▷ -nauty_control -nauty_log "conic_q5_nauty_log.txt" \
25  ▷ ▷ ▷ -end \
26  ▷ ▷ -end \
27  ▷ ▷ -with G -do \
28  ▷ ▷ -group_theoretic_activity \
29  ▷ ▷ ▷ -report Do \
30  ▷ ▷ -end
31  ▷ $(PDFLATEX) PGL_3_5_stab_of_conic_q5_report.tex
32  ▷ $(OPEN) PGL_3_5_stab_of_conic_q5_report.pdf
33

```

The group has order 120.

Next, we compute the stabilizer of the Hirschfeld surface over the field $\mathbb{F}_4$:

Code Snippet 206

```

1   group_of_Hirschfeld_surface.q4:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 4 -compute_related_fields -end \
7   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
8   ▷ ▷ -define R -polynomial_ring \
9   ▷ ▷ ▷ -field F \
10  ▷ ▷ ▷ -number_of_variables 4 \
11  ▷ ▷ ▷ -homogeneous_of_degree 3 \

```

```

12  ▷ ▷ ▷ -monomial_ordering_partition \
13  ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
14  ▷ ▷ -end \
15  ▷ ▷ -define V -variety \
16  ▷ ▷ ▷ -projective_space P \
17  ▷ ▷ ▷ -ring R \
18  ▷ ▷ ▷ -equation_in_algebraic_form \
19  ▷ ▷ ▷ ▷ "X0^2*X3+X0*X3^2+X1^2*X2+X1*X2^2" \
20  ▷ ▷ ▷ -label_txt Hirschfeld_surface_q4 \
21  ▷ ▷ ▷ -label_tex "{\\rm Hirschfeld\\_surface\\_q4}" \
22  ▷ ▷ -end \
23  ▷ ▷ -define G -modified_group \
24  ▷ ▷ ▷ -stabilizer_of_variety V -nauty_control -nauty_log "Hirschfeld_q4.nauty_log
    .txt" -end \
25  ▷ ▷ -end \
26  ▷ ▷ -with G -do \
27  ▷ ▷ -group_theoretic_activity \
28  ▷ ▷ ▷ -report Do \
29  ▷ ▷ -end \
30  ▷ ▷ -with V -do \
31  ▷ ▷ -variety_activity \
32  ▷ ▷ ▷ -report \
33  ▷ ▷ -end
34  ▷ $(PDFLATEX) variety_Hirschfeld_surface_q4_report.tex
35  ▷ $(OPEN) variety_Hirschfeld_surface_q4_report.pdf
36  ▷ $(PDFLATEX) PGGL_4.4_stab_of_Hirschfeld_surface_q4_report.tex
37  ▷ $(OPEN) PGGL_4.4_stab_of_Hirschfeld_surface_q4_report.pdf
38

```

The group has order 51840.

The next command creates the action of the Higman-Sims group of degree 176. To do so, we act on the cosets of a subgroup of index 176.

Code Snippet 207

```

1  Higman_Sims_on_176_points:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define Do -draw_options \
4  ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5  ▷ ▷ -end \
6  ▷ ▷ -define gens -vector -dense \
7  ▷ ▷ ▷ $(HIGMAN_SIMS_GENS) \
8  ▷ ▷ -end \
9  ▷ ▷ -define G -permutation_group -symmetric_group 100 \
10 ▷ ▷ ▷ -subgroup_by_generators HS \
11 ▷ ▷ ▷ $(HIGMAN_SIMS_GO) \
12 ▷ ▷ ▷ $(HIGMAN_SIMS_NB_GENS) gens \
13 ▷ ▷ -end \
14 ▷ ▷ -with G -do \
15 ▷ ▷ -group_theoretic_activity \
16 ▷ ▷ ▷ -subgroup_lattice_magma \

```

```

17  ▷ ▷ -end \
18  ▷ ▷ -define H176a -modified_group -from G \
19  ▷ ▷ ▷ -subgroup_by_lattice 585 \
20  ▷ ▷ -end \
21  ▷ ▷ -define G176a -modified_group -from G \
22  ▷ ▷ ▷ -on_cosets_of_subgroup H176a \
23  ▷ ▷ -end \
24  ▷ ▷ -with G176a -do \
25  ▷ ▷ -group_theoretic_activity \
26  ▷ ▷ ▷ -report Do \
27  ▷ ▷ -end
28  ▷ $(PDFLATEX) Sym_100_Subgroup_HS_44352000_OnCosetsOfSubgroup_report.tex
29  ▷ $(OPEN) Sym_100_Subgroup_HS_44352000_OnCosetsOfSubgroup_report.pdf
30

```

Let us create the unitary group $U(4, 3)$. The hermitian surface in $PG(3, 3)$ is given by the equation

$$X_0^4 + X_1^4 + X_2^4 + X_3^4 = 0,$$

which we encode as a makefile variable

Code Snippet 208

```

1  HERMITIAN_SURFACE_Q9_EQUATION="X0^4+X1^4+X2^4+X3^4"
2

```

We compute the stabilizer inside $P\Gamma L(4, 9)$. After that, we compute the projectivity subgroup and then the derived subgroup. Finally, we restrict the action to the points of the Hermitian surface, which we store in a makefile variable

Code Snippet 209

```

1  HERMITIAN_SURFACE_Q9_POINTS="7, 8, 10, 12, 15, 16, 18, 20, 22, 23, 26, 28, 31, 32
, 35, 37, 39, 48, 58, 59, 62, 64, 66, 76, 77, 80, 82, 84, 95, 96, 98, 100, 102, 1
03, 106, 108, 111, 112, 115, 117, 119, 128, 138, 139, 142, 144, 146, 156, 157, 16
0, 162, 164, 174, 175, 178, 180, 182, 190, 202, 203, 205, 207, 211, 212, 214, 216
, 217, 229, 230, 232, 234, 235, 247, 248, 250, 252, 254, 255, 258, 260, 262, 271,
283, 284, 286, 288, 292, 293, 295, 297, 298, 310, 311, 313, 315, 316, 328, 329,
331, 333, 334, 346, 347, 349, 351, 355, 356, 358, 360, 362, 363, 366, 368, 371, 3
72, 375, 377, 382, 383, 385, 387, 389, 390, 393, 395, 400, 401, 403, 405, 407, 40
8, 411, 413, 415, 427, 428, 430, 432, 436, 437, 439, 441, 443, 444, 447, 449, 452
, 453, 456, 458, 463, 464, 466, 468, 470, 471, 474, 476, 481, 482, 484, 486, 488,
489, 492, 494, 497, 498, 501, 503, 505, 514, 526, 527, 529, 531, 535, 536, 538,
540, 541, 553, 554, 556, 558, 559, 571, 572, 574, 576, 577, 589, 590, 592, 594, 5
98, 599, 601, 603, 605, 606, 609, 611, 614, 615, 618, 620, 625, 626, 628, 630, 63
2, 633, 636, 638, 643, 644, 646, 648, 650, 651, 654, 656, 659, 660, 663, 665, 667
, 676, 688, 689, 691, 693, 697, 698, 700, 702, 703, 715, 716, 718, 720, 721, 733,
734, 736, 738, 739, 751, 752, 754, 756, 760, 761, 763, 765, 767, 768, 771, 773,
776, 777, 780, 782, 787, 788, 790, 792, 794, 795, 798, 800, 805, 806, 808, 810, 8
12, 813, 816, 818"
2

```

We also produce a report about the group. Here is the complete command sequence:

Code Snippet 210

```

1  group_U4.3:
2  ▷ $(ORBITER) -v 6 \
3  ▷ ▷ -define Do -draw_options \
4  ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5  ▷ ▷ -end \
6  ▷ ▷ -define F -finite_field -q 9 -compute_related_fields -end \
7  ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
8  ▷ ▷ -define R -polynomial_ring \
9  ▷ ▷ ▷ -field F \
10 ▷ ▷ ▷ -number_of_variables 4 \
11 ▷ ▷ ▷ -homogeneous_of_degree 4 \
12 ▷ ▷ ▷ -monomial_ordering_partition \
13 ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
14 ▷ ▷ -end \
15 ▷ ▷ -define V -variety \
16 ▷ ▷ ▷ -projective_space P \
17 ▷ ▷ ▷ -ring R \
18 ▷ ▷ ▷ -equation_in_algebraic_form \
19 ▷ ▷ ▷ ▷ $(HERMITIAN_SURFACE_Q9_EQUATION) \
20 ▷ ▷ ▷ -label_txt hermitian_surface_q3 \
21 ▷ ▷ ▷ -label_tex "{\\rm hermitian\\_surface\\_q3}" \
22 ▷ ▷ -end \
23 ▷ ▷ -define G -modified_group \
24 ▷ ▷ ▷ -stabilizer_of_variety V -nauty_control -nauty_log "U4.3.nauty.log.txt" -e
    nd \
25 ▷ ▷ -end \
26 ▷ ▷ -with G -do \
27 ▷ ▷ -group_theoretic_activity \
28 ▷ ▷ ▷ -report Do \
29 ▷ ▷ -end \
30 ▷ ▷ -define GP -modified_group -from G \
31 ▷ ▷ ▷ -projectivity_subgroup \
32 ▷ ▷ -end \
33 ▷ ▷ -define GPD -modified_group -from GP \
34 ▷ ▷ ▷ -derived_subgroup \
35 ▷ ▷ -end \
36 ▷ ▷ -with GPD -do \
37 ▷ ▷ -group_theoretic_activity \
38 ▷ ▷ ▷ -report Do \
39 ▷ ▷ -end \
40 ▷ ▷ -define v -vector \
41 ▷ ▷ ▷ -dense $(HERMITIAN_SURFACE_Q9_POINTS) \
42 ▷ ▷ -end \
43 ▷ ▷ -define GPDr -modified_group \
44 ▷ ▷ ▷ -from GPD \
45 ▷ ▷ ▷ -restricted_action v \
46 ▷ ▷ ▷ "{\\rm \\_on\\_hermitiansurface}" \
47 ▷ ▷ -end \
48 ▷ ▷ -with GPDr -do \
49 ▷ ▷ -group_theoretic_activity \
50 ▷ ▷ ▷ -report Do \

```

```
51    ▷ ▷ -end
52    ▷ $(PDFLATEX) PGGL_4_9_stab_of_hermitian_surface_q3_res_v_report.tex
53    ▷ $(OPEN) PGGL_4_9_stab_of_hermitian_surface_q3_res_v_report.pdf
54
```

The group has order 3265920 and an action of degree 280. It is a simple group.

5.5 Group Theoretic Activities

Once a group has been created as in Sections 5.1 and 5.2, a group theoretic activity can be performed. For this purpose, Orbiter provides the `-group_theoretic_activity` option. Tables 5.8-5.12 list the available commands. Table 5.13 lists commands for element processing.

The command

Code Snippet 211

```

1   PGL_3_2_order_invariant:
2   ▷ $(ORBITER) -v 5 \
3   ▷ ▷ -define G -linear_group -PGL 3 2 -end \
4   ▷ ▷ -with G -do \
5   ▷ ▷ -group_theoretic_activity \
6   ▷ ▷ ▷ -order_invariant \
7   ▷ ▷ -end
8   ▷ $(PDFLATEX) PGL_3_2_order_invariant.tex
9   ▷ $(OPEN) PGL_3_2_order_invariant.pdf
10

```

computes the order invariant of $\text{PGL}(3,2)$. This is the statistic of the orders of the elements in the group. The following latex output will be created:

The Group $\text{PGL}(3,2)$

The order of the group $\text{PGL}(3,2)$ is 168
 The order invariant is 1, 2^{21} , 3^{56} , 4^{42} , 7^{48}

This shows us that the group contains 48 elements of order 7, 42 elements of order 4 and so forth. The order invariant is invariant under group isomorphisms, and hence can be used to distinguish non-isomorphic groups. If the order invariants of two groups differ, the groups are not isomorphic.

The command

Code Snippet 212

```

1   PGL_3_2_elements:
2   ▷ $(ORBITER) -v 5 \
3   ▷ ▷ -define G -linear_group -PGL 3 2 -end \
4   ▷ ▷ -with G -do \
5   ▷ ▷ -group_theoretic_activity \
6   ▷ ▷ ▷ -save_elements_csv "PGL_3_2_elements.csv" \
7   ▷ ▷ -end
8

```

creates all elements of $\text{PGL}(3,2)$ and writes them into the file `PGL_3_2_elements.csv`.

The command

Group theoretic activities (Part 1)		
Command	Arguments	Purpose
-report	options	Produce a latex report for the group. Considers the given draw options.
-order_invariant		Produce a latex report for the order invariant of the group.
-group_table	options	Produce a latex report for the group table. Considers the given draw options.
-sylow		Produce a latex report for the sylow subgroups.
-generators		Returns the generators of the group as a vector of group elements.
-elements		Returns the elements of the group as a vector of group elements.
-elements_by_class	order i	Returns the elements of the i -th class whose elements have the given order.
-select_elements	list	Returns the elements of the group whose ranks are given in the list as a vector of group elements.
-export_group_table		Export the group table as csv-file.
-random_element	label	Creates a random element. Writes the element to file in coded form.
-permutation_representation_of_element	g	Compute the permutation representation of g
-apply	$a \ g$	Applies the group element g to the element a .
-apply_to_set	$a \ g$	Applies the group element g to the set a . The object a must be a vector or a matrix. The image set will be sorted.
-element_processing	options	Perform an activity for a sequence of group elements. For options, see Table 5.13.
-multiply	$s_1 \ s_2$	Multiplies group elements s_1 and s_2 , assuming the elements are given in coded form. Produces a latex report.
-inverse	s	Computes the inverse of s , which is given in coded form. Produces a latex report.
-consecutive_powers	$s \ n$	Computes all powers s^i for $i = 1, \dots, n$. s is given in coded form. Produces a latex report.
-raise_to_the_power	$s \ n$	Computes the n -th power of s , which is given in coded form. Produces a latex report.
-export_orbiter		Exports the group to Orbiter.
-export_gap		Exports the group to GAP [32].
-export_magma		Exports the group to Magma [20].
-search_element_of_order	i	Finds all elements of order i in the group ($i \in \mathbb{N}$).
-find_standard_generators	$a \ b \ c$	Finds all pairs of elements x, y in G such that $ x = a$, $ y = b$, and $ xy = c$. Such pairs are called standard generators.
-find_standard_generators_M24		Finds a pair of standard generators for M_{24} .
-element_rank	s	Determines the rank of the group element s in the given group. s is given in coded form.
-element_unrank	r	Produces the group element whose rank is r .

Group theoretic activities (Part 2)		
Command	Arguments	Purpose
<code>-find_small_generating_set</code>	<code>sz nb</code>	Find a generating set of the group consisting of <code>sz</code> generators using a randomized algorithm. Give up after <code>nb</code> trials.
<code>-find_singer_cycle</code>		Finds all Singer cycles whose matrix is a companion matrix.
<code>-classes_based_on_normal_form</code>		
<code>-normalizer</code>		See Section 5.6.
<code>-centralizer_of_element</code>		See Section 5.6.
<code>-normalizer_of_cyclic_subgroup</code>		Computes the normalizer of a cyclic group, see Section 5.6.
<code>-classes</code>		Computes the conjugacy classes of elements, see Section 5.6.
<code>-split_by_classes</code>	<code>fname</code> <code>column</code>	Splits the sets from the given csv file in the given column according to the different conjugacy classes of the group.
<code>-identify_elements_by_class</code>	<code>fname</code> <code>column</code> <code>go</code> <code>supergroup</code>	Identifies the conjugacy class of each of the elements in the given column in the given file.
<code>-find_subgroup</code>	<code>go</code>	Finds all subgroups of the given order and produces a latex report.
<code>-conjugacy_class_of</code>	<code>label</code> g	Computes the conjugacy class of g .
<code>-isomorphism_Klein_quadric</code>	<code>fname</code>	
<code>-print_elements</code>		Produces a printout of all group elements.
<code>-print_elements_tex</code>		Produces a latex report of all group elements.
<code>-make_element_tree</code>		Produces the tree of all group elements.
<code>-vector_ge_print_elements_tex</code>	<code>vector</code>	Produces a latex report of all group elements in the vector object.
<code>-save_elements_csv</code>	<code>fname</code>	Writes the elements of the group to the given csv file.
<code>-export_inversion_graphs</code>	<code>fname</code>	Exports the inversion graphs associated to the group elements to the given file.
<code>-evaluate_word</code>	<code>word</code> <code>gens</code>	Expresses a word in terms of the given set of generators.
<code>-multiply_all_elements_in_lex_order</code>		Multiplies all group elements together. The output depends on the chosen stabilizer chain.
<code>-stats</code>	<code>prefix</code>	Report statistics on group operations.
<code>-move_a_b</code>	<code>a</code> <code>b</code>	Determine a group element mapping <code>a</code> to <code>b</code> . Computes the stabilizer of <code>b</code> in the given group G (the group for which the activity is applied).
<code>-rational_normal_form</code>	<code>element</code>	Computes the rational normal form of the given group element (must be a matrix group).

Table 5.9: Group theoretic activities (Part 2)

Group theoretic activities (Part 3)		
Command	Arguments	Purpose
-find_conjugating_element	$a\ b$	Computes an element g which conjugates a to b or states that the given elements a and b are not conjugate.
-group_of_automorphisms_by_images_of_generators	label v images	Creates a group of automorphisms by assigning image values to certain generators. The vector v is a vector of group elements. The image values are numerical values representing indices in the vector v .
-order_of_products	$g_1 \dots g_n$	Computes the orders of all products $g_i g_j$, $1 \leq i, j \leq n$. Writes a table with these orders to file.
-reverse_isomorphism_exterior_square		Given a set of generators of a subgroup of $\text{PGO}^+(6, q)$ as 6×6 matrixes, the command computes the inverse image of the generators in $\text{PGL}(4, q)$ (if possible).
-reverse_isomorphism_exterior_square_vector_of_ge	vector	applies reverse isomorphism exterior square to all elements in the given vector object.
-is_subgroup_of		This function is applied to two groups, say G_1 and G_2 , using the syntax -with G1 -and G2. It tests if G_1 is a subgroup of G_2 .
-coset_reps		This function is applied to two groups, say G_1 and G_2 , using the syntax -with G1 -and G2. It assumes that G_1 is a subgroup of G_2 . The command produces a set of coset representatives for G_1 in G_2 .
-subgroup_lattice		Computes the subgroup lattice of the given group. A csv file containing the subgroup lattice is written.
-subgroup_lattice_load	fname	Reads a previously computed subgroup lattice from the csv file.
-subgroup_lattice_draw_by_orbits	options	Produces a drawing of the subgroup lattice in terms of the conjugacy classes of subgroups. Each node represents one conjugacy class of subgroups. Uses the given draw options.
-subgroup_lattice_draw_by_groups	options	Produces a drawing of the subgroup lattice in terms of subgroups. Each node represents one single subgroup. Uses the given draw options.
-subgroup_lattice_intersection_orbit_orbit	orb1 orb2	Intersect all groups of orbit 1 with groups of orbit 2.
-subgroup_lattice_find_overgroup_in_orbit	orb1global gidx orb2global	Find the group on orb2global which normalizes the group gidx in orb1global.

Table 5.10: Group theoretic activities (Part 3)

Group theoretic activities (Part 4)		
Command	Arguments	Purpose
<code>-subgroup_lattice_create_flag_transitive_geometry_with_partition</code>	Pidx Qidx Ridx Rgrpidx s	Create flag transitive large set from groups of orbit Pidx and Qidx, assuming that group Rgrpidx from orbit Ridx normalizes the orbit representative of orbit Pidx. Incidence between groups is defined whenever the intersection has size s.
<code>-subgroup_lattice_create_coset_geometry</code>	Pidx Pgrpidx Qidx Qgrpidx s	Create flag transitive incidence structure from the gosets of the group Pidx in orbit Pidx and group Qgrpidx in orbit Qidx. Incidence between cosets is defined whenever the intersection has size s.
<code>-subgroup_lattice_identify_subgroup</code>	grp	Identify a subgroup in the lattice.
<code>-create_flag_transitive_geometry</code>	P Q	Create a matrix of intersections of all groups in the class of P and all groups in the class of Q. The entry in the matrix is the size of the intersection of the i-th group in the orbit of P with the j-th group in the orbit of Q. The ordering of groups in the conjugacy classes depends on the generating set for the group G. It is the ordering determined by the Schreier algorithm. The groups in the first row and first column are the groups P and Q which are given.
<code>-linear_codes</code>	control d $n_{\max}$	Classify linear codes with prescribed minimum distance d . Assumes that the group is $\text{PGL}(r, q)$ or $\text{PTL}(r, q)$. For each $n \leq n_{\max}$, the $[n, k, \geq d]$ codes are classified with $n - k = r$. See Section 11.
<code>-tensor_permutations</code>		Computes the permutation representation of the generators of the wreath product.
<code>-classify_ovoids</code>	descr	
<code>-representation_on_polynomials</code>	R	Computes the representation of the built-in generating set of the present group in the action on homogeneous polynomials as defined in the ring R. This function assumes a linear group.
<code>-canonical_image_GAP</code>	S	Compute the canonical image of the set S using GAP.
<code>-canonical_image</code>	S	Compute the canonical image of the set S using Orbiter.

Table 5.11: Group theoretic activities (Part 4)

Group theoretic activities (Part 5)		
Command	Arguments	Purpose
-subgroup_lattice_magma		Computes the lattice of subgroups by means of conjugacy classes of subgroups, see Section 5.6.
-find_overgroup	order sub-group	Computes the lattice of subgroups by means of conjugacy classes of subgroups, see Section 5.6. Then identified the overgroups of the given subgroup which have the desired size.
-identify_subgroups_from_file	file col-label grouporder	Identifies subgroups of a given order in the lattice of subgroups (which is computed using magma just as in the previous command). The subgroups are read from the given column in the given file. Subgroups can have any size. Groups whose size does not match are ignored. A file with the results of the identification is written.
-permutation_subgroup		Computes the permutation subgroup of a matrix group. An element belongs to the permutation subgroup if each row of the matrix has Hamming weight one. This function is in experimental stage.

Table 5.12: Group theoretic activities (Part 5)

Element Processing Options		
Command	Arguments	Purpose
-input	label	Specify the input source for a sequence of group elements.
-print		Print each group element.
-with_permutation		Print with permutation representation.
-with_fix_structure		Print with fix structure on the underlying projective space.
-conjugate	data	Conjugate each group element by the element in data.
-print_action_on_surface	surface	Print the action on a cubic surface object.

Table 5.13: Element Processing Options

Code Snippet 213

```

1   Sym_3_elements:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define G -permutation_group -symmetric_group 3 -end \
4   ▷ ▷ -with G -do \
5   ▷ ▷ -group_theoretic_activity \
6   ▷ ▷ ▷ -print_elements_tex \
7   ▷ ▷ -end
8   ▷ $(PDFLATEX) Sym_3_elements.tex
9   ▷ $(OPEN) Sym_3_elements.pdf
10

```

creates a report of the elements of the group $\text{Sym}(3)$:

Group elements in action Sym_3

Element 0 / 6 of order 1:	$(0)(1)(2)$
Element 1 / 6 of order 2:	$(0)(1, 2)$
Element 2 / 6 of order 2:	$(0, 1)(2)$
Element 3 / 6 of order 3:	$(0, 1, 2)$
Element 4 / 6 of order 3:	$(0, 2, 1)$
Element 5 / 6 of order 2:	$(0, 2)(1)$
Order structure:	
$3^2, 2^3, 1$	

The command

Code Snippet 214

```

1   Sym_3_make_element_tree:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define G -permutation_group -symmetric_group 3 -end \
4   ▷ ▷ -with G -do \
5   ▷ ▷ -group_theoretic_activity \
6   ▷ ▷ ▷ -make_element_tree \
7   ▷ ▷ -end
8   ▷ ▷ -define Do -draw_options \
9   ▷ ▷ ▷ -nodes \
10  ▷ ▷ ▷ -embedded -radius 250 \
11  ▷ ▷ ▷ -xin 10000 -yin 10000 \
12  ▷ ▷ ▷ -xout 1000000 -yout 600000 \

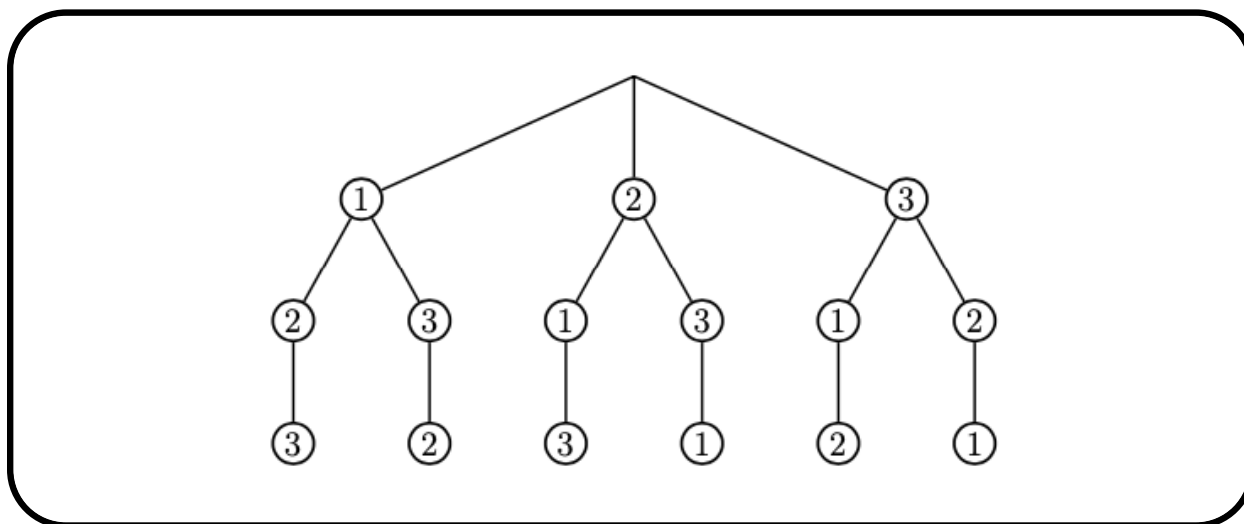
```

```

13  ▷ ▷ ▷ -scale 0.3 -line_width 1.0 \
14  ▷ ▷ -end \
15  ▷ ▷ -tree_draw \
16  ▷ ▷ ▷ -file Sym_3_elements_tree.txt \
17  ▷ ▷ ▷ -draw_options Do \
18  ▷ ▷ -end
19  ▷ $(PDFLATEX) Sym_3_elements_tree_draw.tex
20  ▷ $(OPEN) Sym_3_elements_tree_draw.pdf
21

```

produces a tree of all group elements of $\text{Sym}(3)$ according to the selected stabilizer chain.



The stabilizer chain is defined by a sequence of points in the permutation domain called a base. For $\text{Sym}(n)$ the default base is the sequence $0, 1, 2, \dots, n-1$.

It is possible to compute all powers of a fixed element, as in the following command:

Code Snippet 215

```

1  Cycle_12_power:
2  ▷ $(ORBITER) -v 5 \
3  ▷ ▷ -define G -permutation_group -symmetric_group 12 -end \
4  ▷ ▷ -with G -do \
5  ▷ ▷ -group_theoretic_activity \
6  ▷ ▷ ▷ -consecutive_powers \
7  ▷ ▷ ▷ "1,2,3,4,5,6,7,8,9,10,11,0" 12 \
8  ▷ ▷ -end
9  ▷ $(PDFLATEX) Sym_12_all_powers.tex
10 ▷ $(OPEN) Sym_12_all_powers.pdf
11 ▷
12

```

We create the 12 powers of the cycle

$$(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11).$$

The output is

i	$(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11)^i$
1	(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11)
2	(0, 2, 4, 6, 8, 10)(1, 3, 5, 7, 9, 11)
3	(0, 3, 6, 9)(1, 4, 7, 10)(2, 5, 8, 11)
4	(0, 4, 8)(1, 5, 9)(2, 6, 10)(3, 7, 11)
5	(0, 5, 10, 3, 8, 1, 6, 11, 4, 9, 2, 7)
6	(0, 6)(1, 7)(2, 8)(3, 9)(4, 10)(5, 11)
7	(0, 7, 2, 9, 4, 11, 6, 1, 8, 3, 10, 5)
8	(0, 8, 4)(1, 9, 5)(2, 10, 6)(3, 11, 7)
9	(0, 9, 6, 3)(1, 10, 7, 4)(2, 11, 8, 5)
10	(0, 10, 8, 6, 4, 2)(1, 11, 9, 7, 5, 3)
11	(0, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1)
12	id

The command

Code Snippet 216

```

1  PGL_3_4_singer:
2  ▷ $(ORBITER) -v 5 \
3  ▷ ▷ -define G -linear_group -PGL 3 4 -end \
4  ▷ ▷ -with G -do \
5  ▷ ▷ -group_theoretic_activity \
6  ▷ ▷ ▷ -find_singer_cycle \
7  ▷ ▷ -end
8

```

finds all Singer cycles in $\text{PGL}(3, 4)$ whose matrix is the companion matrix of a polynomial. The first one found is

$$\begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 2 & 3 & 2 \end{bmatrix}$$

whose projective order is 21. Here, we are using the numeric form of field elements, so 2 is ω and 3 is $\omega + 1$.

Suppose we want to multiply two group elements. The following command shows an example in $\text{GL}(2, 8)$. The elements of $\text{GL}(2, 8)$ are coded as vectors of length 4 with entries coded as integers in the interval $[0, 7]$. We multiply the elements coded by 0,1,2,3 and 4,5,6,7:

Code Snippet 217

```

1    GL_2.8_multiply:
2    ▷ $(ORBITER) -v 5 \
3    ▷ ▷ -define G -linear_group -GL 2 8 -end \
4    ▷ ▷ -with G -do \
5    ▷ ▷ -group_theoretic_activity \
6    ▷ ▷ ▷ -multiply "0,1,2,3" "4,5,6,7" \
7    ▷ ▷ -end
8    ▷ $(PDFLATEX) GL_2.8_mult.tex
9    ▷ $(OPEN) GL_2.8_mult.pdf
10

```

The output is

$$\begin{bmatrix} 0 & 1 \\ \gamma & \gamma^5 \end{bmatrix} \cdot \begin{bmatrix} \gamma^2 & \gamma^3 \\ \gamma^6 & \gamma^4 \end{bmatrix} = \begin{bmatrix} \gamma^6 & \gamma^4 \\ \gamma & \gamma^5 \end{bmatrix}$$

0,1,2,3,
4,5,6,7,
6,7,2,3,

Note that the output shows the codings of the three group elements. This way, the result of this computation can be processed further easily. The same example over $\mathbb{F}_7$ is:

Code Snippet 218

```

1    GL_2.7_multiply:
2    ▷ $(ORBITER) -v 5 \
3    ▷ ▷ -define G -linear_group -GL 2 7 -end \
4    ▷ ▷ -with G -do \
5    ▷ ▷ -group_theoretic_activity \
6    ▷ ▷ ▷ -multiply "0,1,2,3" "4,5,6,0" \
7    ▷ ▷ -end
8    ▷ $(PDFLATEX) GL_2.7_mult.tex
9    ▷ $(OPEN) GL_2.7_mult.pdf
10

```

The output is

$$\begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix} \cdot \begin{bmatrix} 4 & 5 \\ 6 & 0 \end{bmatrix} = \begin{bmatrix} 6 & 0 \\ 5 & 3 \end{bmatrix}$$

0,1,2,3,
4,5,6,0,
6,0,5,3,

We can compute the inverse of a group element:

Code Snippet 219

```

1    GL_2_7_inv:
2    ▷ $(ORBITER) -v 5 \
3    ▷ ▷ -define G -linear_group -GL 2 7 -end \
4    ▷ ▷ -with G -do \
5    ▷ ▷ -group_theoretic_activity \
6    ▷ ▷ ▷ -inverse "0,1,2,3" \
7    ▷ ▷ -end
8    ▷ $(PDFLATEX) GL_2_7_inv.tex
9    ▷ $(OPEN) GL_2_7_inv.pdf
10

```

The output is

$$\begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}^{-1} = \begin{bmatrix} 2 & 4 \\ 1 & 0 \end{bmatrix}$$

0,1,2,3,
2,4,1,0,

We can raise a group element to a power:

Code Snippet 220

```

1    GL_2_7_power:
2    ▷ $(ORBITER) -v 5 \
3    ▷ ▷ -define G -linear_group -GL 2 7 -end \
4    ▷ ▷ -with G -do \
5    ▷ ▷ -group_theoretic_activity \
6    ▷ ▷ ▷ -raise_to_the_power "0,1,2,3" 2 \
7    ▷ ▷ -end
8    ▷ $(PDFLATEX) GL_2_7_power.tex
9    ▷ $(OPEN) GL_2_7_power.pdf
10

```

The output is

$$\begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}^2 = \begin{bmatrix} 2 & 3 \\ 6 & 4 \end{bmatrix}$$

0,1,2,3,
2,3,6,4,

Suppose we want to find a small generating set for a group. The following command invokes a randomized algorithm to find a generating set of 2 elements for the group $\text{PGL}(4, 4)$. The command specifies the size of the desired generating system and an upper bound for the number of attempts.

Code Snippet 221

```

1   PGGL_4.4_small_generating_set:
2   ▷ $(ORBITER) -v 4 \
3   ▷ ▷ -define F -finite_field -q 4 -end \
4   ▷ ▷ -define G -linear_group -PGGL 4 F -end \
5   ▷ ▷ -with G -do \
6   ▷ ▷ -group_theoretic_activity \
7   ▷ ▷ ▷ -find_small_generating_set 2 1000 \
8   ▷ ▷ -end
9

```

The next example computes the action of a specific group element on the set of planes through a line. The planes have been computed in Section 7.1.

Code Snippet 222

```

1   on_planes:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 8 -end \
4   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5   ▷ ▷ -define G -linear_group -PGL 4 F -end \
6   ▷ ▷ -define G_on_planes -modified_group -from G \
7   ▷ ▷ ▷ -on_k_subspaces 3 \
8   ▷ ▷ -end \
9   ▷ ▷ -with G_on_planes -do \
10  ▷ ▷ -group_theoretic_activity \
11  ▷ ▷ ▷ -apply "0,8,1,6,4,3,7,2,5" \
12  ▷ ▷ ▷ "1,0,0,0, 0,1,0,0, 0,0,0,2, 0,0,1,1" \
13  ▷ ▷ -end
14  ▷ $(PDFLATEX) PGL_4.8_Gr_4.3_apply.tex
15  ▷ $(OPEN) PGL_4.8_Gr_4.3_apply.pdf
16

```

The output is

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & \gamma \\ 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1000 \\ 0100 \\ 0002 \\ 0011 \end{bmatrix}$$

1,0,0,0,0,1,0,0,0,0,0,2,0,0,1,1,

maps:

0 $\mapsto$ 8

8 $\mapsto$ 1

1 $\mapsto$ 3

6 $\mapsto$ 5

4 $\mapsto$ 7

3 $\mapsto$ 4

7 $\mapsto$ 6

2 $\mapsto$ 0

$$5 \mapsto 2$$

The command `-random_element` can be used to pick a random element in a group. Here is an example. We create the group $\text{PGL}(4, 2)$ and then ask for a random element:

Code Snippet 223

```

1   PGL_4_2_random_element:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 2 -end \
4   ▷ ▷ -define G -linear_group -PGL 4 F -end \
5   ▷ ▷ -with G -do \
6   ▷ ▷ -group_theoretic_activity \
7   ▷ ▷ -random_element r1 \
8   ▷ ▷ -end \
9   ▷ ▷ -print_symbols
10

```

The output is the following csv file (the actual content may differ, as the output is pseudo-random).

r1

Row	Rk	Path	Tl	ElementLatex	ElementCoded	Perm	CycleType	Sign	Order
0	9883	7,4,11,3	15,14,12,8	$\left[\begin{array}{c} 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{array} \right]$	1,0,1,1,0,1,1,0,0,1,0,1,1,0,1,0	13,7,10,6,2,11,8,12,0,3,5,14,4,1,9	15	1	15
END									

The contents is as follows:

1. The column **Rk** is the rank number of the group element in the group. It is a number between zero and group order minus one.
2. The column **Path** is the coset decomposition in the basic orbits determined by the rank.
3. The column **Tl** is the vector whose i -th entry is the length of the i -th basic orbit.
4. The column **ElementLatex** is a latex description of the group element.
5. The column **ElementCoded** is the coded description of the group element.
6. The column **Perm** is the permutation action of the group element in the natural action.
7. The column **CycleType** is the cycle type of the permutation action of the group element in the natural action.
8. The column **Sign** is the sign of the permutation action of the group element in the natural action (even=1, odd=-1).

9. The column `Order` is the order of the group element in the natural action.

The command

Code Snippet 224

```

1   PGL_2_3_subgroup_lattice:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 3 -end \
4   ▷ ▷ -define G -linear_group -PGL 2 F -end \
5   ▷ ▷ -with G -do \
6   ▷ ▷ -group_theoretic_activity \
7   ▷ ▷ ▷ -subgroup_lattice \
8   ▷ ▷ -end
9
```

computes the subgroup lattice of $\text{PGL}(2, 3)$ and writes the data to a csv file called

`PGL_2_3_subgroup_lattice.csv`.

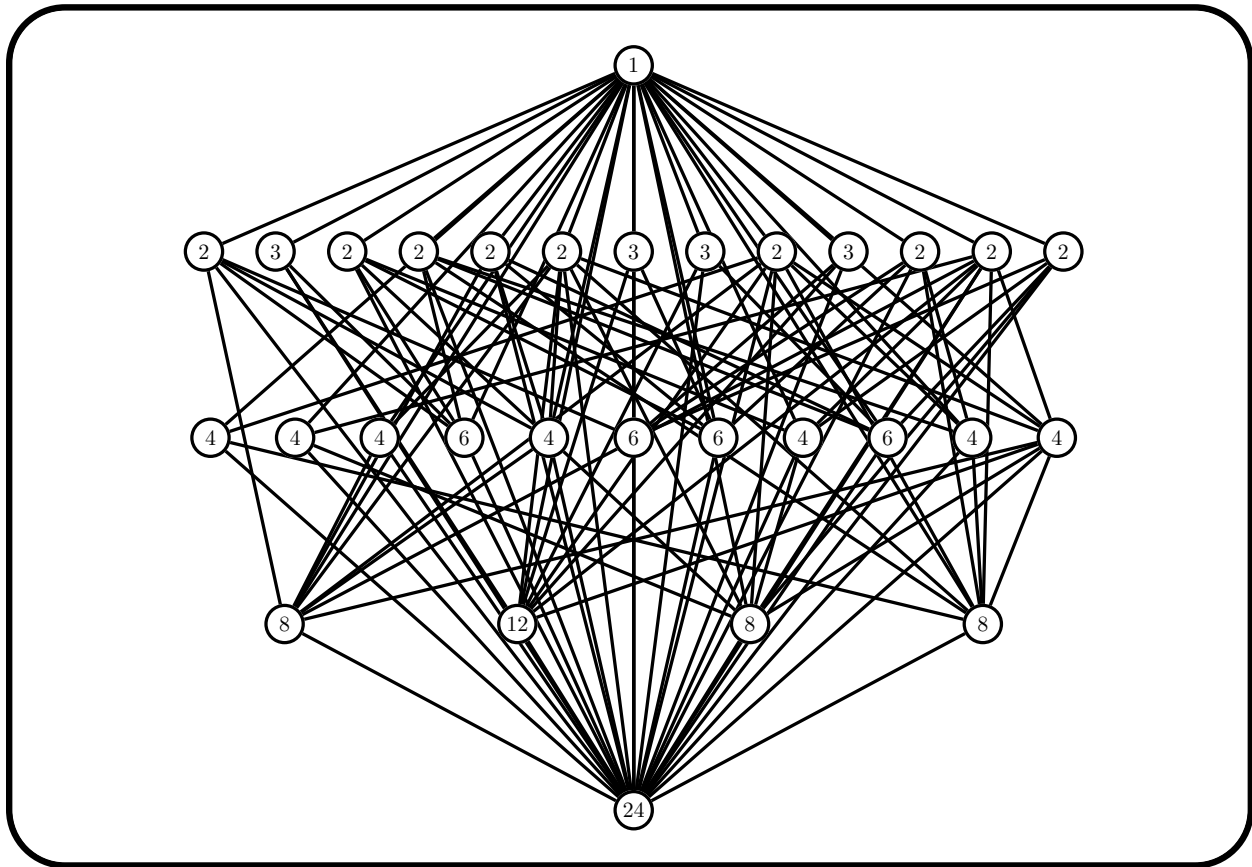
The command

Code Snippet 225

```

1   PGL_2_3_subgroup_lattice_load_and_draw_by_groups:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define Do -draw_options -embedded -end \
4   ▷ ▷ -define F -finite_field -q 3 -end \
5   ▷ ▷ -define G -linear_group -PGL 2 F -end \
6   ▷ ▷ -with G -do \
7   ▷ ▷ -group_theoretic_activity \
8   ▷ ▷ ▷ -subgroup_lattice_load PGL_2_3_subgroup_lattice.csv \
9   ▷ ▷ -end \
10  ▷ ▷ -with G -do \
11  ▷ ▷ -group_theoretic_activity \
12  ▷ ▷ ▷ -subgroup_lattice_draw_by_groups Do \
13  ▷ ▷ -end
14  ▷ $(PDFLATEX) PGL_2_3_drawing.tex
15  ▷ $(OPEN) PGL_2_3_drawing.pdf
16
```

reads this file and produces a drawing of the lattice by groups. The nodes correspond to subgroups. The lattice is “upside down,” i.e. the unit subgroup is at the top and the full group is at the bottom. The maximal inclusions are drawn as edges between the nodes:



The command

Code Snippet 226

```

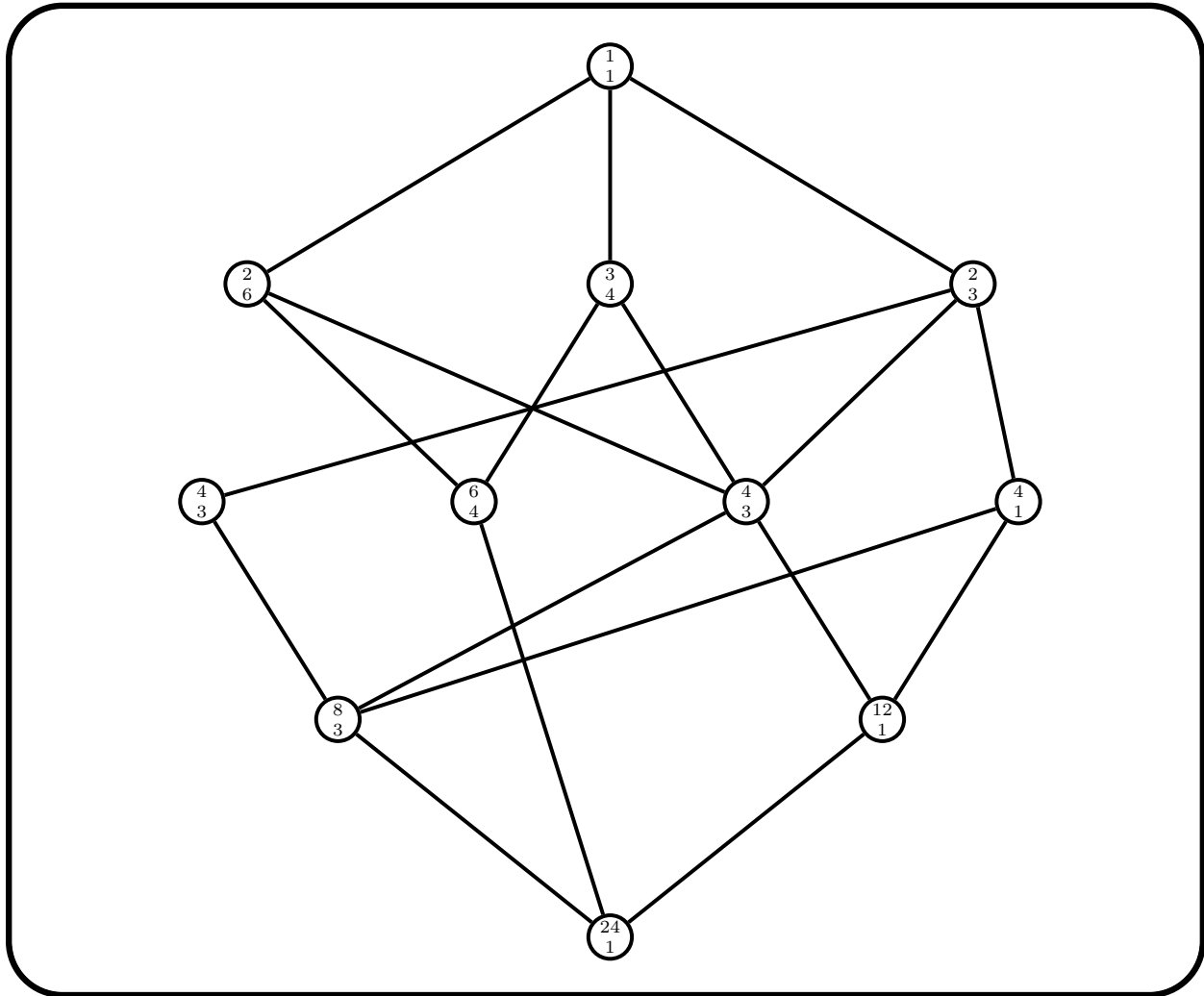
1   PGL_2_3_subgroup_lattice_load_and_draw_by_orbits:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define Do -draw_options -embedded -end \
4   ▷ ▷ -define F -finite_field -q 3 -end \
5   ▷ ▷ -define G -linear_group -PGL 2 F -end \
6   ▷ ▷ -with G -do \
7   ▷ ▷ -group_theoretic_activity \
8   ▷ ▷ ▷ -subgroup_lattice_load PGL_2_3_subgroup_lattice.csv \
9   ▷ ▷ -end \
10  ▷ ▷ -with G -do \
11  ▷ ▷ -group_theoretic_activity \
12  ▷ ▷ ▷ -subgroup_lattice_draw_by_orbits Do \
13  ▷ ▷ -end
14  ▷ $(PDFLATEX) PGL_2_3_drawing_by_orbits.tex
15  ▷ $(OPEN) PGL_2_3_drawing_by_orbits.pdf
16

```

also reads the csv file and produces a drawing of the lattice by orbits. The nodes correspond to conjugacy classes of subgroups. The nodes are labeled by symbols of the form

$$\begin{matrix} a \\ b \end{matrix}.$$

Here, a is the order of the subgroups in the orbit and b is the number of conjugate groups.



The command

Code Snippet 227

```

1   PGL_3_13_2B_RNF:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 13 -end \
4   ▷ ▷ -define G -linear_group -PGL 3 F -end \
5   ▷ ▷ -with G -do \
6   ▷ ▷ -group_theoretic_activity \
7   ▷ ▷ ▷ -rational_normal_form $(PGL_3_13_CLASS_2B_REP) \
8   ▷ ▷ -end
9

```

computes the rational normal form of the matrix

$$A = \begin{bmatrix} 1 & 1 & 11 \\ 1 & 6 & 12 \\ 7 & 10 & 5 \end{bmatrix}$$

in $\text{PGL}(3, 13)$. The rational normal form is

$$R = \begin{bmatrix} 12 & 0 & 0 \\ 0 & 12 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

A matrix B such that $B^{-1}AB = R$ is

$$B = \begin{bmatrix} 7 & 12 & 9 \\ 12 & 0 & 11 \\ 0 & 12 & 12 \end{bmatrix}.$$

Group Theoretic Activities Based on Magma		
Command	Arguments	Purpose
-classes		Compute the conjugacy classes of elements in the group.
-subgroup_lattice_magma		Compute the lattice of subgroups by means of conjugacy classes of subgroups.
-centralizer_of_element	label coding	Compute the centralizer of the group element, using label to create file names.
-normalizer_of_cyclic_subgroup	label s	Compute the normalizer of the cyclic subgroup generated by the element s .
-normalizer		Compute the normalizer of a subgroup in the larger group.

Table 5.14: Group Theoretic Activities Based on Magma

5.6 Group Theoretic Activities Based on Magma

Orbiter has an interface to Magma [20]. This allows Orbiter commands to utilize Magma for specialized (and often difficult) computations in Group theory. Applications are computing conjugacy classes in groups, normalizers and other things. We will see examples below.

Table 5.14 lists the group theoretic commands in Orbiter that rely on Magma.

The actual computation of conjugacy classes happens inside Magma, using a permutation representation of the group. The class representatives and centralizer groups are presented in Orbiter as matrix groups, using the latex interface to display the result. The communication between Magma and Orbiter is based on files. Orbiter writes a file that is used as Magma input. Because of this, the command has to be run twice. In the first execution, a magma session is started. Once the Magma session is done, the

quit;

command needs to be issued in the terminal to end the Magma session. The Magma run produces a file containing the results of the computation in machine-readable form. The Orbiter command needs to run one more time after that. Orbiter will recognize the file written by Magma and read it. Based on the data, Orbiter will produce a latex report.

The command

Code Snippet 228

```

1   PSP_4_4_export_magma:
2   ▷ $(ORBITER) -v 5 \
3   ▷ ▷ -define F -finite_field -q 4 -end \
4   ▷ ▷ -define G -linear_group -PGL 4 F \
5   ▷ ▷ ▷ -symplectic_group \
6   ▷ ▷ -end \
7   ▷ ▷ -with G -do \
8   ▷ ▷ -group_theoretic_activity \
9   ▷ ▷ ▷ -export_magma \
10  ▷ ▷ -end
11
```

exports the group $\mathrm{PSp}(4, 4)$ to Magma.

The command

Code Snippet 229

```

1   PGGL_2_4.export_magma:
2   ▷ $(ORBITER) -v 20 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 4 \
7   ▷ ▷ ▷ -compute_related_fields \
8   ▷ ▷ -end \
9   ▷ ▷ -define G -linear_group -PGGL 2 F \
10  ▷ ▷ -end \
11  ▷ ▷ -with G -do \
12  ▷ ▷ -group_theoretic_activity \
13  ▷ ▷ ▷ -report Do \
14  ▷ ▷ -end \
15  ▷ ▷ -with G -do \
16  ▷ ▷ -group_theoretic_activity \
17  ▷ ▷ ▷ -export_magma \
18  ▷ ▷ -end
19  ▷ $(PDFLATEX) PGGL_2_4.report.tex
20  ▷ $(OPEN) PGGL_2_4.report.pdf
21

```

exports the group $\text{PGL}(4, 4)$ to Magma.

Next, we wish to export the Weyl group of type E_6 (of order 51,840) into Magma. We wish to use a specific representation obtained by stabilizing a cubic surface over the field $\mathbb{F}_4$ (the Hirschfeld surface). There are in fact two representations. The first is a semilinear matrix group representation of degree 4 over the field $\mathbb{F}_4$. The second representation is a permutation representation on the points of the projective space $\text{PG}(3, 4)$. We start with the matrix representation. Since it is semilinear, we write each group element as a $x \times 4$ matrix together with the exponent of the associated field automorphism w.r.t. the generator $x \mapsto x^2$. In total, this amounts to 17 ($= 4 \times 4 + 1$) integers. The first 16 integers are in the interval $[0, 3]$ (encoding elements of $\mathbb{F}_4$). The 17th integer is from the interval $[0, 1]$, encoding an automorphism of the field (noting that $\text{Aut}(\mathbb{F}_4) \simeq C_2$). We use a makefile variable for storing the desired generators as matrices over $\mathbb{F}_4$:

Code Snippet 230

```

1   HIRSCHFELD_STAB_GENERATORS="\
2   1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,1, \
3   1,0,0,0,0,2,0,0,0,0,2,0,0,0,0,1,0, \
4   1,0,0,0,0,3,0,0,0,0,3,0,1,0,0,1,0, \
5   1,0,0,0,0,1,0,0,1,1,1,0,0,1,0,1,1, \
6   1,0,0,0,0,0,1,0,0,1,0,0,0,0,0,1,0, \
7   0,1,0,0,1,0,0,0,0,0,0,1,0,0,1,0,0"
8

```

We use the following command to create the group in Orbiter and to export it into Magma:

Code Snippet 231

```

1   Hirschfeld_stab.export_magma:
2   ▷ $(ORBITER) -v 9 \
3   ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4   ▷ ▷ -define F -finite_field -q 4 \
5   ▷ ▷ ▷ -compute_related_fields \
6   ▷ ▷ -end \
7   ▷ ▷ -define G -linear_group -PGGL 4 F \
8   ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld.Stab" \
9   ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
10  ▷ ▷ ▷ -end \
11  ▷ ▷ -with G -do \
12  ▷ ▷ -group_theoretic_activity \
13  ▷ ▷ ▷ -export_magma \
14  ▷ ▷ -end
15

```

The command creates a magma file to create the group in Magma. The file offers two different versions of the group. The first one is the matrix representation of degree 8 over the binary field. The second one is the permutation representation of degree 85. This raises the question: why do we get a matrix representation of degree 8, when the group has degree 4? The reason is that the group of the surface is semilinear, and Magma only allows linear groups. Therefore, Orbiter performs field reduction from $\mathbb{F}_4$ to $\mathbb{F}_2$. An element of $\mathbb{F}_4$ is written as a 2×2 matrix over $\mathbb{F}_2$. An automorphic collineation becomes a block diagonal matrix with two (identical) blocks of size 2×2 on the diagonal. All told, this creates a linear representation of degree 8. Here is the Magma code generated by Orbiter:

```

F:=GF(2);
G := GeneralLinearGroup(8, F);
H := sub< G | [1,0,0,0,0,0,0,0, 1,1,0,0,0,0,0,0, 0,0,1,0,0,0,0,0, 0,0,1,1,0,0,0,0,
, 0,0,0,0,1,0,0,0, 0,0,0,0,1,1,0,0, 0,0,0,0,0,0,1,0, 0,0,0,0,0,0,1,1],
[1,0,0,0,0,0,0,0, 0,1,0,0,0,0,0,0, 0,0,1,1,0,0,0,0, 0,0,1,0,0,0,0,0, 0,0,0,0,1,1,
0,0, 0,0,0,0,1,0,0,0, 0,0,0,0,0,0,1,0, 0,0,0,0,0,0,0,1],
[1,0,0,0,0,0,0,0, 0,1,0,0,0,0,0,0, 0,0,0,1,0,0,0,0, 0,0,1,1,0,0,0,0, 0,0,0,0,0,1,
0,0, 0,0,0,0,1,1,0,0, 1,0,0,0,0,0,1,0, 0,1,0,0,0,0,0,1],
[1,0,0,0,0,0,0,0, 1,1,0,0,0,0,0,0, 0,0,1,0,0,0,0,0, 0,0,1,1,0,0,0,0, 1,0,1,0,1,0,
0,0, 1,1,1,1,1,1,0,0, 0,0,1,0,0,0,1,0, 0,0,1,1,0,0,1,1],
[1,0,0,0,0,0,0,0, 0,1,0,0,0,0,0,0, 0,0,0,0,1,0,0,0, 0,0,0,0,0,1,0,0, 0,0,1,0,0,0,
0,0, 0,0,0,1,0,0,0,0, 0,0,0,0,0,0,1,0, 0,0,0,0,0,0,1],
[0,0,1,0,0,0,0,0, 0,0,0,1,0,0,0,0, 1,0,0,0,0,0,0,0, 0,1,0,0,0,0,0,0, 0,0,0,0,0,0,
1,0, 0,0,0,0,0,0,1, 0,0,0,0,1,0,0,0, 0,0,0,0,1,0,0], >;
G := sub< Sym(85) |
(7, 8)(10, 11)(14, 15)(16, 20)(17, 21)(18, 23)(19, 22)(25, 26)(29, 30)(31, 35)(32,
, 36)(33, 38)(34, 37)(41, 42)(44, 45)(46, 50)(47, 51)(48, 53)(49, 52)(54, 70)(55,
71)(56, 73)(57, 72)(58, 74)(59, 75)(60, 77)(61, 76)(62, 82)(63, 83)(64, 85)(65,
84)(66, 78)(67, 79)(68, 81)(69, 80),
(5, 63, 83)(6, 8, 7)(9, 11, 10)(13, 15, 14)(17, 19, 18)(21, 23, 22)(27, 31, 35)(2
8, 32, 36)(29, 33, 37)(30, 34, 38)(39, 54, 70)(40, 55, 71)(41, 56, 72)(42, 57, 73
)(43, 62, 82)(44, 64, 84)(45, 65, 85)(46, 66, 74)(47, 67, 75)(48, 68, 76)(49, 69,
77)(50, 58, 78)(51, 59, 79)(52, 60, 80)(53, 61, 81),
(4, 24)(5, 82, 63, 43, 83, 62)(6, 7, 8)(9, 10, 11)(13, 14, 15)(17, 18, 19)(21, 22
, 23)(25, 26)(27, 36, 31, 28, 35, 32)(29, 38, 33, 30, 37, 34)(39, 71, 54, 40, 70,
55)(41, 73, 56, 42, 72, 57)(44, 85, 64, 45, 84, 65)(46, 75, 66, 47, 74, 67)(48,
77, 68, 49, 76, 69)(50, 79, 58, 51, 78, 59)(52, 81, 60, 53, 80, 61),

```

```
(3, 13)(4, 27)(5, 43)(7, 8)(9, 12)(10, 14)(11, 15)(16, 17)(20, 21)(24, 28)(25, 30)
(26, 29)(33, 34)(37, 38)(39, 40)(46, 51)(47, 50)(48, 52)(49, 53)(54, 81)(55, 80)
(56, 78)(57, 79)(58, 85)(59, 84)(60, 82)(61, 83)(62, 77)(63, 76)(64, 74)(65, 75)(
66, 73)(67, 72)(68, 70)(69, 71),
(2, 3)(6, 9)(7, 10)(8, 11)(16, 20)(17, 23)(18, 21)(19, 22)(27, 39)(28, 40)(29, 41)
(30, 42)(31, 54)(32, 55)(33, 56)(34, 57)(35, 70)(36, 71)(37, 72)(38, 73)(46, 58)
(47, 59)(48, 60)(49, 61)(50, 74)(51, 75)(52, 76)(53, 77)(66, 78)(67, 79)(68, 80)(
69, 81),
(1, 2)(3, 4)(7, 8)(9, 27)(10, 31)(11, 35)(12, 24)(13, 28)(14, 32)(15, 36)(16, 25)
(17, 29)(18, 33)(19, 37)(20, 26)(21, 30)(22, 34)(23, 38)(40, 43)(41, 46)(42, 50)(
44, 47)(45, 51)(49, 52)(54, 70)(55, 82)(56, 74)(57, 78)(58, 73)(59, 85)(60, 77)(6
1, 81)(62, 71)(63, 83)(64, 75)(65, 79)(66, 72)(67, 84)(68, 76)(69, 80)>;
```

The command

Code Snippet 232

```
1  Hirschfeld_stab.subgroup.lattice.magma:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4  ▷ ▷ -define F -finite_field -q 4 \
5  ▷ ▷ ▷ -compute_related_fields \
6  ▷ ▷ -end \
7  ▷ ▷ -define G -linear_group -PGGL 4 F \
8  ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld.Stab" \
9  ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
10 ▷ ▷ ▷ -end \
11 ▷ ▷ -with G -do \
12 ▷ ▷ -group_theoretic_activity \
13 ▷ ▷ ▷ -subgroup_lattice_magma \
14 ▷ ▷ -end
15 ▷ $(ORBITER) -v 3 \
16 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
17 ▷ ▷ -define F -finite_field -q 4 \
18 ▷ ▷ ▷ -compute_related_fields \
19 ▷ ▷ -end \
20 ▷ ▷ -define G -linear_group -PGGL 4 F \
21 ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld.Stab" \
22 ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
23 ▷ ▷ ▷ -end \
24 ▷ ▷ -with G -do \
25 ▷ ▷ -group_theoretic_activity \
26 ▷ ▷ ▷ -subgroup_lattice_magma \
27 ▷ ▷ -end
28 ▷ $(PDFLATEX) PGGL_4.4_Subgroup_Hirschfeld_Stab_51840_subgroup_lattice_out.tex
29 ▷ $(OPEN) PGGL_4.4_Subgroup_Hirschfeld_Stab_51840_subgroup_lattice_out.pdf
30
```

exports the lattice of subgroups of the group of the Hirschfeld surface to Magma. The command produces a latex report. This same Orbiter command is run twice. In between, the Magma shell has to be terminated on the command line, using the `quit;` command as described above.

The command sequence

Code Snippet 233

```

1   PGGL_2_4_classes:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define G \
4   ▷ ▷ -linear_group -PGGL 2 4 \
5   ▷ ▷ -end \
6   ▷ ▷ -with G -do \
7   ▷ ▷ -group_theoretic_activity \
8   ▷ ▷ ▷ -classes \
9   ▷ ▷ -end \
10  ▷ ▷ -print_symbols
11  ▷ $(PDFLATEX) PGGL_2_4_classes.tex
12  ▷ $(OPEN) PGGL_2_4_classes.pdf
13

```

computes the conjugacy classes of elements in $\text{P}\Gamma\text{L}(2, 4)$. The ordering of conjugacy classes is chosen by Magma. Here is the shortened output of the command:

Conjugacy classes in $\text{P}\Gamma\text{L}(2, 4)$

The group order is

120

The following table shows the conjugacy classes of the group. The columns display: the number of the class, the order of elements in the class, an ATLAS style label for the class, the size of the class, the order of the centralizer of an element of the class, the order of the normalizer of the group generated by an element of the class.

0	1	1A	1	120	120	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	0
1	2	2A	10	12	12	$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$	1
2	2	2B	15	8	8	$\begin{bmatrix} 0 & \omega^2 \\ 1 & 0 \end{bmatrix}$	0
3	3	3A	20	6	12	$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$	0
4	4	4A	30	4	8	$\begin{bmatrix} 0 & \omega^2 \\ 1 & 1 \end{bmatrix}$	1
5	5	5A	24	5	20	$\begin{bmatrix} 0 & \omega \\ 1 & 1 \end{bmatrix}$	0
6	6	6A	20	6	12	$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$	1

The command

Code Snippet 234

```

1   Hirschfeld_stab_classes:
2   ▷ $(ORBITER) -v 9 \
3   ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4   ▷ ▷ -define G -linear_group -PGGL 4 4 \
5   ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld_Stab" \
6   ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
7   ▷ ▷ ▷ -end \
8   ▷ ▷ -with G -do \
9   ▷ ▷ -group_theoretic_activity \
10  ▷ ▷ ▷ -classes \
11  ▷ ▷ -end
12  ▷ $(ORBITER) -v 9 \
13  ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
14  ▷ ▷ -define G -linear_group -PGGL 4 4 \
15  ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld_Stab" \
16  ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
17  ▷ ▷ ▷ -end \
18  ▷ ▷ -with G -do \
19  ▷ ▷ -group_theoretic_activity \
20  ▷ ▷ ▷ -classes \
21  ▷ ▷ -end
22  ▷ $(PDFLATEX) PGGL_4_4_Subgroup_Hirschfeld_Stab_51840_report_classes_of_elements.
    tex
23  ▷ $(OPEN) PGGL_4_4_Subgroup_Hirschfeld_Stab_51840_report_classes_of_elements.pdf
24

```

computes the conjugacy classes of the Weyl group of type E_6 . The group is the stabilizer of the Hirschfeld surface (see Section 9.2). The generators have been stored in a makefile variable. They have been computed with the commands described in Section 17.2.

The command sequence

Code Snippet 235

```

1   PGGL_2_4_cent_2A:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define G \
4   ▷ ▷ -linear_group -PGGL 2 4 -end \
5   ▷ ▷ -with G -do \
6   ▷ ▷ -group_theoretic_activity \
7   ▷ ▷ ▷ -centralizer_of_element "2A" "1,0, 0,1, 1" \
8   ▷ ▷ -end
9   ▷ $(ORBITER) -v 3 \
10  ▷ ▷ -define G \
11  ▷ ▷ -linear_group -PGGL 2 4 -end \
12  ▷ ▷ -with G -do \
13  ▷ ▷ -group_theoretic_activity \
14  ▷ ▷ ▷ -centralizer_of_element "2A" "1,0, 0,1, 1" \
15  ▷ ▷ -end

```

```

16  ▷ $(PDFLATEX) PGGL_2_4_elt_2A_centralizer.tex
17  ▷ $(OPEN) PGGL_2_4_elt_2A_centralizer.pdf
18  ▷
19

```

computes the centralizer of the Baer involution

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1.$$

The centralizer is a group of order 40320, isomorphic to $\text{PGL}(4,2).Z_2$. Orbiter produces a list of strong generators, shown below:

Strong generators for a group of order 40320:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}_0,$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}_0,$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}_0, \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix}_0$$

1,0,0,0,0,1,0,0,0,0,1,0,0,0,1,1,
 1,0,0,0,0,1,0,0,0,0,1,0,1,0,0,1,1,
 1,0,0,0,0,1,0,0,0,0,1,0,1,1,1,0,
 1,0,0,0,0,1,0,0,0,0,1,0,1,1,0,1,0,
 1,0,0,0,0,1,0,0,1,1,0,1,1,1,1,0,0,
 1,0,0,0,0,1,0,0,0,0,1,1,1,1,1,0,0,
 1,0,0,0,0,1,0,0,1,1,1,0,0,0,0,1,0,
 1,0,0,0,1,1,1,1,1,0,1,0,0,1,0,0,0,
 0,1,0,1,1,1,1,1,1,0,0,1,0,1,1,1,0,

Orbiter can compute the normalizer of a subgroup. The group must be constructed as a subgroup H of a larger group G containing H . Typically, the group G is the group in which H is generated as a subgroup (either the full linear group of the full symmetric group). Then, the normalizer of H in G is computed. Here is an example in a symmetric group. We first create a subgroup of order 5, with generator

$$(0, 1, 2, 3, 4)(5, 6, 7, 8, 9)(10)(11)(12).$$

We store the generator in a makefile variable:

Code Snippet 236

```

1  GENERATORS_H5="1,2,3,4,0,6,7,8,9,5,10,11,12"
2  # (0, 1, 2, 3, 4)(5, 6, 7, 8, 9)
3

```

The command

Code Snippet 237

```

1  Normalizer_of_H5:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define G -permutation_group -symmetric_group 13 \
4  ▷ ▷ ▷ -subgroup_by_generators H5 5 1 \
5  ▷ ▷ ▷ ▷ $(GENERATORS_H5) -end \
6  ▷ ▷ -with G -do \
7  ▷ ▷ -group_theoretic_activity \
8  ▷ ▷ ▷ -normalizer \
9  ▷ ▷ -end
10 ▷ $(PDFLATEX) Sym_13.Subgroup_H5_5.normalizer.tex
11 ▷ $(OPEN) Sym_13.Subgroup_H5_5.normalizer.pdf
12

```

computes the normalizer of H inside $\text{Sym}(13)$. It is a group of order 1200. The following report is produced:

The group *Perm13SubgroupH5order5* of order 5 is:

Strong generators for a group of order 5:

$$(0, 1, 2, 3, 4)(5, 6, 7, 8, 9)$$

1, 2, 3, 4, 0, 6, 7, 8, 9, 5, 10, 11, 12,

Inside the group of order 6227020800, the normalizer has order 1200:

Strong generators for a group of order 1200:

$$(11, 12),$$

$$(10, 11),$$

$$(5, 9, 8, 7, 6),$$

$$(1, 2, 4, 3)(6, 7, 9, 8),$$

$$(0, 5)(1, 9)(2, 8)(3, 7)(4, 6)$$

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 11,

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 10, 12,

0, 1, 2, 3, 4, 9, 5, 6, 7, 8, 10, 11, 12,

0, 2, 4, 1, 3, 5, 7, 9, 6, 8, 10, 11, 12,

5, 9, 8, 7, 6, 0, 4, 3, 2, 1, 10, 11, 12,

The group

$$H = \left\langle \begin{bmatrix} \alpha^4 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \right\rangle \simeq C_2 \times C_2$$

is a non-cyclic subgroup of $G = \text{PGL}(2, 9)$. To compute the normalizer of H in G , the following command sequence can be used:

Code Snippet 238

```

1   Normalizer_of_Z22_in_PGL_2_9:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define G -linear_group -PGL 2 9 \
4   ▷ ▷ -subgroup_by_generators Z22 4 2 \
5   ▷ ▷ ▷ "2,0,0,1, 0,1,1,0" -end \
6   ▷ ▷ -with G -do \
7   ▷ ▷ -group_theoretic_activity \
8   ▷ ▷ ▷ -normalizer \
9   ▷ ▷ -end
10  ▷ $(PDFLATEX) PGL_2_9_Subgroup_Z22_4_normalizer.tex
11  ▷ $(OPEN) PGL_2_9_Subgroup_Z22_4_normalizer.pdf
12
```

It produces a report showing that the normalizer is a group of order 24 (it is isomorphic to $\text{Sym}(4)$, though the report does not tell us this fact directly):

The group $\text{PGL}(2, 9)\text{SubgroupZ22order4}$ of order 4 is:

Strong generators for a group of order 4:

$$\begin{bmatrix} \alpha^4 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

1,0,0,2,

0,1,1,0,

Inside the group of order 720, the normalizer has order 24:

Strong generators for a group of order 24:

$$\begin{bmatrix} \alpha^4 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} \alpha^2 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} \alpha^4 & \alpha^4 \\ \alpha^4 & 1 \end{bmatrix},$$

$$\begin{bmatrix} \alpha^4 & \alpha^6 \\ \alpha^2 & 1 \end{bmatrix}$$

1,0,0,2,

1,0,0,5,

1,1,1,2,

1,7,5,2,

The command

Code Snippet 239

```

1   PGOm_6_2.classes:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 2 -end \
4   ▷ ▷ -define G -linear_group -PGOm 6 F -end \
5   ▷ ▷ -with G -do \
6   ▷ ▷ -group_theoretic_activity \
7   ▷ ▷ ▷ -classes \
8   ▷ ▷ -end
9   ▷ $(PDFLATEX) PGOm_6_2.classes.out.tex
10  ▷ $(OPEN) PGOm_6_2.classes.out.pdf
11

```

computes the conjugacy classes of $\text{PGO}^-(6, 2)$. The command has to be executed twice. Let us look at the class 2A of size 36. We store the representative as a makefile variable:

Code Snippet 240

```

1   PGOM_6_2.CLASS_2A.REP="1,0,0,0,0,0,0,1,0,0,\
2   0,0,0,0,1,0,0,0,0,0,1,1,1,1,0,0,1,0,0,1,0,0,1,0,1,0"
3

```

To create the whole conjugacy class, we use the following command:

Code Snippet 241

```

1   PGOm_6_2.Class_2A:
2   ▷ $(ORBITER) -v 9 \
3   ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4   ▷ ▷ -define F -finite_field -q 2 -end \
5   ▷ ▷ -define G -linear_group -PGOm 6 F -end \
6   ▷ ▷ -with G -do \
7   ▷ ▷ -group_theoretic_activity \
8   ▷ ▷ ▷ -conjugacy_class_of "2A" $(PGOM_6_2.CLASS_2A.REP) \
9   ▷ ▷ -end
10

```

The elements are stored in a csv file. We use a makefile variable to store the file name:

Code Snippet 242

```

1   FILE_NAME_PGOM_6_2.CLASS_2A="PGOm_6_2_class_of_2A.csv"
2

```

The following command prints all group elements in the class:

Code Snippet 243

```

1   PGOm_6_2.Class_2A.print:
2   ▷ $(ORBITER) -v 9 \
3   ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4   ▷ ▷ -define F -finite_field -q 2 -end \

```

```

5    ▷ ▷ -define G -linear_group -PGOm 6 F -end \
6    ▷ ▷ -define PGOm_6_2_Class2A -vector_ge \
7    ▷ ▷ ▷ -action G \
8    ▷ ▷ ▷ -read_csv $(FILE_NAME_PGOM_6_2_CLASS_2A) elements \
9    ▷ ▷ -end \
10   ▷ ▷ -with G -do \
11   ▷ ▷ -group_theoretic_activity \
12   ▷ ▷ ▷ -element_processing \
13   ▷ ▷ ▷ ▷ -input PGOm_6_2_Class2A \
14   ▷ ▷ ▷ ▷ -print \
15   ▷ ▷ ▷ -end \
16   ▷ ▷ -end
17   ▷ $(PDFLATEX) PGOm_6_2_Class2A.elements.tex
18   ▷ $(OPEN) PGOm_6_2_Class2A.elements.pdf
19

```

We decide to pick group element 8 as our favorite class representative. We store the element in a makefile variable and compute the centralizer:

Code Snippet 244

```

1    PGOM_6_2_CLASS_2A_REP_CLEAN="1,0,0,0,0,0,0,1,0,0,\
2    0,0,0,0,0,1,0,0,0,0,1,0,0,0,0,0,0,0,1,0,0,0,0,0,1"
3

```

Code Snippet 245

```

1    PGOm_6_2_Class_2A.centralizer:
2    ▷ $(ORBITER) -v 9 \
3    ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4    ▷ ▷ -define F -finite_field -q 2 -end \
5    ▷ ▷ -define G -linear_group -PGOm 6 F -end \
6    ▷ ▷ -with G -do \
7    ▷ ▷ -group_theoretic_activity \
8    ▷ ▷ ▷ -centralizer_of_element \
9    ▷ ▷ ▷ ▷ "2Arep" $(PGOM_6_2_CLASS_2A_REP_CLEAN) \
10   ▷ ▷ -end
11   ▷ $(PDFLATEX) PGOm_6_2_elt_2Arep.centralizer.tex
12   ▷ $(OPEN) PGOm_6_2_elt_2Arep.centralizer.pdf
13

```

We store the generators for the centralizer in a makefile variable.

Code Snippet 246

```

1    PGOM_6_2_CLASS_2A_CENTRALIZER_GEN=" \
2    1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,1, \
3    1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,1,0, \
4    1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,1, \
5    1,0,0,0,0,0,1,1,0,0,0,1,0,0,1,0,0,0,0,0,0,1,0,0,1,0,0,0,1, \
6    1,0,0,0,0,0,1,1,0,0,1,0,0,0,1,0,0,0,0,0,1,0,0,0,0,0,1,0,1, \
7    1,0,0,0,0,0,1,1,1,1,0,0,1,0,0,1,0,0,1,0,1,0,0,0,0,0,0,1,0, \

```

```

8      0,1,1,1,1,0,1,0,1,1,1,0,1,1,0,1,1,0,1,1,0,1,0,0,0,0,0,1,0,1,1,1,1,1, \
9      1,1,0,0,0,1,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,1,0,0,1,1,0,0,0,0,1, \
10     0,1,0,0,0,0,1,0,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,1,0,0,0,0,0,1"
11

```

We recompute the centralizer as a subgroup of $\text{PGL}(6, 2)$:

Code Snippet 247

```

1      PG0m_6_2_Class_2A_centralizer_recover:
2      ▷ $(ORBITER) -v 2 \
3      ▷ ▷ -define Do -draw_options \
4      ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5      ▷ ▷ -end \
6      ▷ ▷ -define F -finite_field -q 2 -end \
7      ▷ ▷ -define Class2A_cent_gens -vector \
8      ▷ ▷ ▷ -dense $(PGOM_6_2_CLASS_2A_CENTRALIZER_GENS) \
9      ▷ ▷ -end \
10     ▷ ▷ -define G -linear_group -PGL 6 2 \
11     ▷ ▷ ▷ -subgroup_by_generators "2ACent" \
12     ▷ ▷ ▷ 1440 9 Class2A_cent_gens \
13     ▷ ▷ ▷ -end \
14     ▷ ▷ -with G -do \
15     ▷ ▷ -group_theoretic_activity \
16     ▷ ▷ ▷ -report Do \
17     ▷ ▷ -end
18     ▷ $(PDFLATEX) PGL_6_2_Subgroup_2ACent_1440_report.tex
19     ▷ $(OPEN) PGL_6_2_Subgroup_2ACent_1440_report.pdf
20

```

Group Theoretic Activities Based on GAP		
Command	Arguments	Purpose
-canonical_image	set	Compute the canonical image of the given set under the group G .
-export_gap		Export the group G to GAP [32] / Fining [7].

Table 5.15: Group Theoretic Activities Based on GAP

5.7 Group Theoretic Activities Based on GAP

Orbiter has an interface to GAP [32]. This allows Orbiter commands to utilize GAP specific functionality. One application is the computation of canonical images of sets in the context of permutation groups. This is facilitated by means of the GAP package images [42].

Table 5.15 lists the group theoretic commands in Orbiter that rely on GAP.

How can we export a group from Orbiter to GAP? For semilinear matrix groups, we need to export to the GAP package Fining [7]. This is because semilinear groups are not available in GAP. Here is an example. We create the stabilizer of the Hirschfeld surface in $\text{PG}(3, 4)$ and export the group to GAP/Fining as a group of collineations.

Code Snippet 248

```

1   Hirschfeld_stab.export_gap:
2   ▷ $(ORBITER) -v 9 \
3   ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4   ▷ ▷ -define G -linear_group -PGGL 4 4 \
5   ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld_Stab" \
6   ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
7   ▷ ▷ ▷ -end \
8   ▷ ▷ -with G -do \
9   ▷ ▷ -group_theoretic_activity \
10  ▷ ▷ ▷ -export_gap \
11  ▷ ▷ -end
12

```

The command creates a GAP/Fining file shown below:

```

LoadPackage("fining");
mat1 := [[Z(4)^0,0*Z(4),0*Z(4),0*Z(4)],
[0*Z(4),Z(4)^0,0*Z(4),0*Z(4)],
[0*Z(4),0*Z(4),Z(4)^0,0*Z(4)],
[0*Z(4),0*Z(4),0*Z(4),Z(4)^0]];
frob1 := FrobeniusAutomorphism(GF(4))^1;
psi1 := ProjectiveSemilinearMap(mat1, frob1,GF(4));
mat2 := [[Z(4)^0,0*Z(4),0*Z(4),0*Z(4)],
[0*Z(4),Z(4)^1,0*Z(4),0*Z(4)],
[0*Z(4),0*Z(4),Z(4)^1,0*Z(4)],
[0*Z(4),0*Z(4),0*Z(4),Z(4)^0]];
frob2 := FrobeniusAutomorphism(GF(4))^0;
psi2 := ProjectiveSemilinearMap(mat2, frob2,GF(4));

```

```

mat3 := [[Z(4)^0, 0*Z(4), 0*Z(4), 0*Z(4)],
[0*Z(4), Z(4)^2, 0*Z(4), 0*Z(4)],
[0*Z(4), 0*Z(4), Z(4)^2, 0*Z(4)],
[Z(4)^0, 0*Z(4), 0*Z(4), Z(4)^0]];
frob3 := FrobeniusAutomorphism(GF(4))^0;
psi3 := ProjectiveSemilinearMap(mat3, frob3, GF(4));
mat4 := [[Z(4)^0, 0*Z(4), 0*Z(4), 0*Z(4)],
[0*Z(4), Z(4)^0, 0*Z(4), 0*Z(4)],
[Z(4)^0, Z(4)^0, Z(4)^0, 0*Z(4)],
[0*Z(4), Z(4)^0, 0*Z(4), Z(4)^0]];
frob4 := FrobeniusAutomorphism(GF(4))^1;
psi4 := ProjectiveSemilinearMap(mat4, frob4, GF(4));
mat5 := [[Z(4)^0, 0*Z(4), 0*Z(4), 0*Z(4)],
[0*Z(4), 0*Z(4), Z(4)^0, 0*Z(4)],
[0*Z(4), Z(4)^0, 0*Z(4), 0*Z(4)],
[0*Z(4), 0*Z(4), 0*Z(4), Z(4)^0]];
frob5 := FrobeniusAutomorphism(GF(4))^0;
psi5 := ProjectiveSemilinearMap(mat5, frob5, GF(4));
mat6 := [[0*Z(4), Z(4)^0, 0*Z(4), 0*Z(4)],
[Z(4)^0, 0*Z(4), 0*Z(4), 0*Z(4)],
[0*Z(4), 0*Z(4), 0*Z(4), Z(4)^0],
[0*Z(4), 0*Z(4), Z(4)^0, 0*Z(4)]];
frob6 := FrobeniusAutomorphism(GF(4))^0;
psi6 := ProjectiveSemilinearMap(mat6, frob6, GF(4));
gens := [psi1, psi2, psi3, psi4, psi5, psi6];
G := Group(gens);
Size(G);

```

When we load this file into GAP, we see the following output (for printing purposes, it has been shortened and reformatted slightly):

```

gap> LoadPackage("fining");
true
gap> mat1 := [[Z(4)^0, 0*Z(4), 0*Z(4), 0*Z(4)],
> [0*Z(4), Z(4)^0, 0*Z(4), 0*Z(4)],
> [0*Z(4), 0*Z(4), Z(4)^0, 0*Z(4)],
> [0*Z(4), 0*Z(4), 0*Z(4), Z(4)^0]];
[ [ Z(2)^0, 0*Z(2), 0*Z(2), 0*Z(2) ],
[ 0*Z(2), Z(2)^0, 0*Z(2), 0*Z(2) ],
[ 0*Z(2), 0*Z(2), Z(2)^0, 0*Z(2) ],
[ 0*Z(2), 0*Z(2), 0*Z(2), Z(2)^0 ] ]
gap> frob1 := FrobeniusAutomorphism(GF(4))^1;
FrobeniusAutomorphism( GF(2^2) )
gap> psi1 := ProjectiveSemilinearMap(mat1, frob1, GF(4));
< a collineation: <cmat 4x4 over GF(2,2)>, F^2>
...
gap> gens := [psi1, psi2, psi3, psi4, psi5, psi6];
[ < a collineation: <cmat 4x4 over GF(2,2)>, F^2>,
< a collineation: <cmat 4x4 over GF(2,2)>, F^0>,
< a collineation: <cmat 4x4 over GF(2,2)>, F^0>,
< a collineation: <cmat 4x4 over GF(2,2)>, F^2>,
< a collineation: <cmat 4x4 over GF(2,2)>, F^0>,

```

```

< a collineation: <cmat 4x4 over GF(2,2)>, F^0> ]
gap> G := Group(gens);
<projective collineation group with 6 generators>
gap> Size(G);
51840
gap>

```

Suppose we want to compute the least image of the set $\{3,5\}$ in the permutation group C_6 . The following command sets up the group C_6 and invokes the Orbiter `-canonical_image` command.

Code Snippet 249

```

1   Cyclic_6_canonical_image:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define G -permutation_group -cyclic_group 6 -end \
4   ▷ ▷ -with G -do -group_theoretic_activity \
5   ▷ ▷ ▷ -canonical_image_GAP 3,5 \
6   ▷ ▷ -end
7

```

The command creates a GAP file shown below:

```

G := Group([(1, 2, 3, 4, 5, 6)]);
LoadPackage("images");
MinimalImage(G, [4,6], OnSets);

```

Running this command in GAP yields:

```

gap> MinimalImage(G, [4,6], OnSets);
[ 1, 3 ]

```

Note that the permutation representations in GAP are 1-based, while Orbiter uses 0-based representations. So, the least image of $\{1,3\}$ in GAP really corresponds to the set $\{0,2\}$ in Orbiter.

Let us try a larger example. We want to compute the canonical form of the Edge curve by means of their rational points over $\mathbb{F}_{17}$. We use the following makefile variable to specify the set of rational points:

Code Snippet 250

```

1   EDGE_CURVE_Q17_AS_POINTS="4, 7, 16, 19, 20, 23, 32, 35, 89, 100, 244, 251"
2

```

The next command creates $\text{PGL}(3,17)$ and invokes the `-canonical_image` image command.

Code Snippet 251

```

1   Edge_curve_q17_canonical_image:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define G -linear_group -PGL 3 17 -end \

```

```
4    ▷ ▷ -with G -do -group_theoretic_activity \  
5    ▷ ▷ ▷ -canonical_image_GAP \  
6    ▷ ▷ ▷ $(EDGE_CURVE_Q17_AS_POINTS) \  
7    ▷ ▷ -end  
8
```

The command creates a GAP input file called `PGL_3_17_canonical_image.gap`. The file is too large to be shown here. Running it through GAP yields the canonical form:

```
gap> MinimalImage(G, [5,8,17,20,21,24,33,36,90,101,245,252], OnSets);  
[ 1, 2, 3, 4, 5, 6, 55, 104, 108, 112, 206, 228 ]
```

Keep in mind that the permutation representation in Gap is 1-based, so the numbers have been incremented by one.

5.8 Group Theory, Advanced Topics

It is sometimes necessary to control the finite field that is used in the construction of a matrix group. For prime fields, this is not an issue. For extension fields, the choice of polynomial does matter, as the generators depend on specific choices made for the finite field. Magma and GAP use Conway polynomials, which are difficult to compute. Orbiter has a built-in table of primitive polynomials. As explained in Section 3.2, Orbiter allows to specify the polynomial that should be used to create the finite field. The next example shows an instance where choosing the polynomial is important. We are recreating a group from the electronic Atlas on finite simple groups [76].

The electronic Atlas of finite simple groups [76] lists generators for $U_3(3)$ as 3×3 matrices over the field $\mathbb{F}_9$ using the following short Magma [20] program:

```
F<w>:=GF(9);
x:=CambridgeMatrix(1,F,3,[
"164",
"506",
"851"]);
y:=CambridgeMatrix(1,F,3,[
"621",
"784",
"066"]);
G<x,y>:=MatrixGroup<3,F|x,y>;
```

The generators are given using the Magma command `CambridgeMatrix`, which allows for more efficient coding of field elements. The field elements are coded as base-3 integers (like in Orbiter) with respect to the Magma version of $\mathbb{F}_9$. The polynomial for $\mathbb{F}_9$ can be determined using the following Magma command:

```
F<w>:=GF(9);
print DefiningPolynomial(F);
```

It results in

```
$.1^2 + 2*$.1 + 2
```

which is the Magma way of printing the polynomial $X^2 + 2X + 2$. If α is a root of the polynomial over $\mathbb{F}_3$, then

$$\alpha^2 = \alpha + 1.$$

The coefficient vector of the polynomial is $(1, 2, 2)$. As an integer written in base-3, we obtain

$$1 \cdot 3^2 + 2 \cdot 3 + 2 = 17.$$

The desired subgroup can now be created using the command

Code Snippet 252

```
1  U_3.3:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define Do -draw_options \
4  ▷ ▷ ▷ -radius 200 -line_width 0.3 \
```

```

5    ▷ ▷ -end \
6    ▷ ▷ -define F -finite_field -q 9 \
7    ▷ ▷ ▷ -override_polynomial "17" \
8    ▷ ▷ -end \
9    ▷ ▷ -define G -linear_group -PGL 3 F \
10   ▷ ▷ ▷ -subgroup_by_generators \
11   ▷ ▷ ▷ "U_3_3" "6048" 2 \
12   ▷ ▷ ▷ "1,6,4, 5,0,6, 8,5,1, \
13   ▷ ▷ ▷ 6,2,1, 7,8,4, 0,6,6" \
14   ▷ ▷ ▷ -end \
15   ▷ ▷ -with G -do \
16   ▷ ▷ -group_theoretic_activity \
17   ▷ ▷ ▷ -report Do \
18   ▷ ▷ -end
19   ▷ $(PDFLATEX) PGL_3_9_Subgroup_U_3_3_6048_report.tex
20   ▷ #$(OPEN) PGL_3_9_Subgroup_U_3_3_6048_report.pdf
21

```

Group theoretic activities will be discussed in Section 5.5.

As an example of a large group, consider the Conway group Co_3 . Following [70], the group can be generated using two matrices of dimension 22 over $\mathbb{F}_2$. We use the makefile variables to give each generator in compact form.

Code Snippet 253

```

1    CONWAY_GEN1="\
2    1101110001000001010000\
3    1111010111110100001011\
4    0000001000000100010101\
5    1111100110110001001110\
6    0101010000000010011101\
7    0000010000000100010101\
8    0010000000000100010101\
9    0001000011000000111111\
10   1110100100110100010011\
11   0000000000000110010101\
12   0000000000100100010101\
13   0110111111010011101111\
14   0000000000001100010101\
15   0000000000000100000101\
16   0000000001000100010101\
17   0000000000000100011101\
18   0001000110000010011010\
19   00000000000000000010101\
20   0000000000000101010101\
21   0000000000000100010100\
22   0000000000000100010111\
23   0000000000000100010001"
24

```

Code Snippet 254

```

1  CONWAY_GEN2=""\
2  0101000010111010111111\
3  0110010100011110110000\
4  001101000011111010111\
5  0001101110001011010011\
6  1010010000100001011110\
7  110100000001010100011\
8  1100101010001111010101\
9  1000110100110101010101\
10 0100110001010000000111\
11 1100000010100101010010\
12 0101110110011100000101\
13 0101111101010011111001\
14 1000010101010101010001\
15 0001010000111100100111\
16 0011010010111011001111\
17 0100110010110011111010\
18 1101011001111101100011\
19 0100101001001000100001\
20 1100101100001001110011\
21 0101110110010100000001\
22 0000001101111000101110\
23 1101101010101110000101"
24

```

Then we define vectors for each of the generators. We concatenate the two generators to form one long vector, which is passed to the `-subgroup_by_generators` command. Finally, we create a report for the group.

Code Snippet 255

```

1  Co3:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define Do -draw_options \
4  ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5  ▷ ▷ -end \
6  ▷ ▷ -define F -finite_field -q 2 -end \
7  ▷ ▷ -define g1 -vector -field F -format 22 \
8  ▷ ▷ ▷ -compact $(CONWAY_GEN1) -end \
9  ▷ ▷ -define g2 -vector -field F -format 22 \
10 ▷ ▷ ▷ -compact $(CONWAY_GEN2) -end \
11 ▷ ▷ -define gens -vector -concatenate g1,g2 -end \
12 ▷ ▷ -define G -linear_group -PGL 22 2 \
13 ▷ ▷ ▷ -subgroup_by_generators "Co3" "495766656000" 2 gens \
14 ▷ ▷ ▷ -end \
15 ▷ ▷ -with G -do \
16 ▷ ▷ -group_theoretic_activity \
17 ▷ ▷ ▷ -report Do \
18 ▷ ▷ -end
19 ▷ $(PDFLATEX) PGL_22_2_Subgroup_Co3_495766656000_report.tex
20 ▷ #$(OPEN) PGL_22_2_Subgroup_Co3_495766656000_report.pdf
21

```

The next example creates the Ree group in 7 dimensions over the field $\mathbb{F}_{27}$. Again, we use makefile variables to specify the two generators as 7×7 matrices over $\mathbb{F}_{27}$ and concatenate them, before passing them to the `-subgroup_by_generators` command.

Code Snippet 256

```
1  Ree_gen1="21,5,1,6,17,1,1, 3,13,5,21,6,6,18, 21,3,21,21,22,6,14, \
2  14,18,1,5,13,6,7, 3,3,2,1,24,16,3, 17,3,22,10,16,24,26, \
3  21,21,6,18,20,2,5"
4
```

Code Snippet 257

```
1  Ree_gen2="16,3,11,5,16,22,20, 24,6,18,24,7,1,26, 9,23,17,18,23,20,13, \
2  9,7,2,15,17,5,11, 3,3,6,21,4,24,16, 25,8,6,24,21,12,7, \
3  24,15,2,13,11,14,24"
4
```

Code Snippet 258

```
1  Ree_27:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define Do -draw_options \
4  ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5  ▷ ▷ -end \
6  ▷ ▷ -define F -finite_field -q 27 \
7  ▷ ▷ ▷ -override_polynomial "34" \
8  ▷ ▷ -end \
9  ▷ ▷ -define g1 -vector -field F -format 7 \
10 ▷ ▷ ▷ -dense $(Ree_gen1) -end \
11 ▷ ▷ -define g2 -vector -field F -format 7 \
12 ▷ ▷ ▷ -dense $(Ree_gen2) -end \
13 ▷ ▷ -define gens -vector -concatenate g1,g2 -end \
14 ▷ ▷ -define G -linear_group -PGL 7 F \
15 ▷ ▷ ▷ -subgroup_by_generators "Ree_27" "10073444472" 2 gens \
16 ▷ ▷ ▷ -end \
17 ▷ ▷ -with G -do \
18 ▷ ▷ -group_theoretic_activity \
19 ▷ ▷ ▷ -report Do \
20 ▷ ▷ -end
21
```

Let us take a look at the Higman-Sims group. It is a permutation group acting on 100 points. More precisely, the group is related to the automorphism group of a certain graph that arises from the Witt design on 22 points. For a construction of the design, see Section 14.2. For a construction of the graph, see Section 13.1.

We use the following three makefile variables (for the sake of space, the third one, `HIGMAN_SIMS_GENS` is not shown),

Code Snippet 259

```

1    HIGMAN_SIMS_NB_GENS=12
2

```

Code Snippet 260

```

1    HIGMAN_SIMS_GO=44352000
2

```

The following command creates the group in the permutation representation:

Code Snippet 261

```

1    Higman.Sims.group:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define Do -draw_options \
4    ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5    ▷ ▷ -end \
6    ▷ ▷ -define gens -vector -dense \
7    ▷ ▷ ▷ $(HIGMAN_SIMS_GENS) \
8    ▷ ▷ -end \
9    ▷ ▷ -define G -permutation_group -symmetric_group 100 \
10   ▷ ▷ ▷ -subgroup_by_generators HS \
11   ▷ ▷ ▷ ▷ $(HIGMAN_SIMS_GO) \
12   ▷ ▷ ▷ ▷ $(HIGMAN_SIMS_NB_GENS) gens \
13   ▷ ▷ -end \
14   ▷ ▷ -with G -do \
15   ▷ ▷ -group_theoretic_activity \
16   ▷ ▷ ▷ -report Do \
17   ▷ ▷ -end
18   ▷ $(PDFLATEX) Sym_100_Subgroup_HS_44352000_report.tex
19   ▷ $(OPEN) Sym_100_Subgroup_HS_44352000_report.pdf
20

```

The next command computes the conjugacy classes of elements:

Code Snippet 262

```

1    Higman.Sims.group.classes:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define gens -vector -dense \
4    ▷ ▷ ▷ $(HIGMAN_SIMS_GENS) \
5    ▷ ▷ -end \
6    ▷ ▷ -define G -permutation_group -symmetric_group 100 \
7    ▷ ▷ ▷ -subgroup_by_generators HS \
8    ▷ ▷ ▷ ▷ $(HIGMAN_SIMS_GO) \
9    ▷ ▷ ▷ ▷ $(HIGMAN_SIMS_NB_GENS) gens \
10   ▷ ▷ -end \
11   ▷ ▷ -with G -do \
12   ▷ ▷ -group_theoretic_activity \
13   ▷ ▷ ▷ -classes \

```

```

14   ▷ ▷ -end
15   ▷ $(PDFLATEX) Sym_100_Subgroup_HS_44352000_classes.out.tex
16   ▷ $(OPEN) Sym_100_Subgroup_HS_44352000_classes.out.pdf
17

```

We also compute the conjugacy classes of subgroups:

Code Snippet 263

```

1   Higman_Sims_group_subgroup_lattice:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define gens -vector -dense \
4   ▷ ▷ ▷ $(HIGMAN_SIMS_GENS) \
5   ▷ ▷ -end \
6   ▷ ▷ -define G -permutation_group -symmetric_group 100 \
7   ▷ ▷ ▷ -subgroup_by_generators HS \
8   ▷ ▷ ▷ $(HIGMAN_SIMS_GO) \
9   ▷ ▷ ▷ $(HIGMAN_SIMS_NB_GENS) gens \
10  ▷ ▷ -end \
11  ▷ ▷ -with G -do \
12  ▷ ▷ -group_theoretic_activity \
13  ▷ ▷ ▷ -subgroup_lattice_magma \
14  ▷ ▷ -end
15

```

The next two commands define the representatives for the two distinct conjugacy classes of subgroups of order 252000:

Code Snippet 264

```

1   Higman_Sims_subgroup_176a:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define gens -vector -dense \
7   ▷ ▷ ▷ $(HIGMAN_SIMS_GENS) \
8   ▷ ▷ -end \
9   ▷ ▷ -define G -permutation_group -symmetric_group 100 \
10  ▷ ▷ ▷ -subgroup_by_generators HS \
11  ▷ ▷ ▷ $(HIGMAN_SIMS_GO) \
12  ▷ ▷ ▷ $(HIGMAN_SIMS_NB_GENS) gens \
13  ▷ ▷ -end \
14  ▷ ▷ -with G -do \
15  ▷ ▷ -group_theoretic_activity \
16  ▷ ▷ ▷ -subgroup_lattice_magma \
17  ▷ ▷ -end \
18  ▷ ▷ -define G176a -modified_group -from G \
19  ▷ ▷ ▷ -subgroup_by_lattice 585 \
20  ▷ ▷ -end \
21  ▷ ▷ -with G176a -do \

```

```

22  ▷ ▷ -group_theoretic_activity \
23  ▷ ▷ ▷ -report Do \
24  ▷ ▷ -end
25  ▷ $(PDFLATEX) Sym_100_Subgroup_HS.44352000_subgroup_by_lattice_585_report.tex
26  ▷ $(OPEN) Sym_100_Subgroup_HS.44352000_subgroup_by_lattice_585_report.pdf
27

```

Code Snippet 265

```

1    Higman_Sims_subgroup_176b:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define Do -draw_options \
4    ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5    ▷ ▷ -end \
6    ▷ ▷ -define gens -vector -dense \
7    ▷ ▷ ▷ $(HIGMAN_SIMS_GENS) \
8    ▷ ▷ -end \
9    ▷ ▷ -define G -permutation_group -symmetric_group 100 \
10   ▷ ▷ ▷ -subgroup_by_generators HS \
11   ▷ ▷ ▷ $(HIGMAN_SIMS_GO) \
12   ▷ ▷ ▷ $(HIGMAN_SIMS_NB_GENS) gens \
13   ▷ ▷ -end \
14   ▷ ▷ -with G -do \
15   ▷ ▷ -group_theoretic_activity \
16   ▷ ▷ ▷ -subgroup_lattice_magma \
17   ▷ ▷ -end \
18   ▷ ▷ -define G176b -modified_group -from G \
19   ▷ ▷ ▷ -subgroup_by_lattice 586 \
20   ▷ ▷ -end \
21   ▷ ▷ -with G176b -do \
22   ▷ ▷ -group_theoretic_activity \
23   ▷ ▷ ▷ -report Do \
24   ▷ ▷ -end
25   ▷ $(PDFLATEX) Sym_100_Subgroup_HS.44352000_subgroup_by_lattice_586_report.tex
26   ▷ $(OPEN) Sym_100_Subgroup_HS.44352000_subgroup_by_lattice_586_report.pdf
27

```

The command to define a subgroup from the previously computed lattice of subgroups performs a shortening of the base. Base points whose basic orbit is trivial are dropped.

It is sometimes desirable to change the underlying form of a classical group. The following command creates generators for the group $\mathrm{PSp}(6, 2)$ with respect to the form

$$\begin{bmatrix} 0 & I \\ I & 0 \end{bmatrix}.$$

Code Snippet 266

```

1    Butler12_PSP_6.2_conjugate:
2    ▷ $(ORBITER) -v 10 \
3    ▷ ▷ -define F -finite_field -q 2 -end \
4    ▷ ▷ -define G -linear_group -PGL 6 F \

```

```

5   ▷ ▷ ▷ -symplectic_group \
6   ▷ ▷ -end \
7   ▷ ▷ -assign gens -from \
8   ▷ ▷ -with G -do \
9   ▷ ▷ -group_theoretic_activity \
10  ▷ ▷ ▷ -generators \
11  ▷ ▷ -end \
12  ▷ ▷ -define m1 -vector \
13  ▷ ▷ ▷ -permutation_matrix "0,2,4,1,3,5" \
14  ▷ ▷ -end \
15  ▷ ▷ -define m1t -vector \
16  ▷ ▷ ▷ -permutation_matrix_inverse "0,2,4,1,3,5" \
17  ▷ ▷ -end \
18  ▷ ▷ -define M1 -vector_ge \
19  ▷ ▷ ▷ -action G \
20  ▷ ▷ ▷ -vector_data m1 \
21  ▷ ▷ -end \
22  ▷ ▷ -define M1t -vector_ge \
23  ▷ ▷ ▷ -action G \
24  ▷ ▷ ▷ -vector_data m1t \
25  ▷ ▷ -end \
26  ▷ ▷ -assign gens2 -from \
27  ▷ ▷ -with gens -and M1t -do \
28  ▷ ▷ -vector_ge_activity \
29  ▷ ▷ ▷ -conjugate \
30  ▷ ▷ -end \
31  ▷ ▷ -with gens2 -do \
32  ▷ ▷ -vector_ge_activity \
33  ▷ ▷ ▷ -report \
34  ▷ ▷ -end
35  ▷ $(PDFLATEX) PGL_6_2_elements.tex
36  ▷ $(OPEN) PGL_6_2_elements.pdf
37

```

The following command creates the subfield subgroup of the Weyl Group of type E_6 .

Code Snippet 267

```

1   Hirschfeld_surface_get_subfield_subgroup:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 4 -end \
7   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
8   ▷ ▷ -define R -polynomial_ring \
9   ▷ ▷ ▷ -field F \
10  ▷ ▷ ▷ -number_of_variables 4 \
11  ▷ ▷ ▷ -homogeneous_of_degree 3 \
12  ▷ ▷ ▷ -monomial_ordering_partition \
13  ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \

```



```

14  ▷ ▷ -end \
15  ▷ ▷ -define V -variety \
16  ▷ ▷ ▷ -projective_space P \
17  ▷ ▷ ▷ -ring R \
18  ▷ ▷ ▷ -equation_in_algebraic_form \
19  ▷ ▷ ▷ ▷ $(HIRSCHFELD_SURFACE_EQUATION_ALGEBRAIC_FORM) \
20  ▷ ▷ ▷ -label_txt Hirschfeld_4 \
21  ▷ ▷ ▷ -label_tex "{\\rm Hirschfeld\\_4}" \
22  ▷ ▷ -end \
23  ▷ ▷ -with V -do -variety_activity \
24  ▷ ▷ ▷ -compute_group \
25  ▷ ▷ ▷ -nauty_control -save_orbit_of_equations eqn_ -end \
26  ▷ ▷ -end \
27  ▷ ▷ -define G -modified_group \
28  ▷ ▷ ▷ -stabilizer_of_variety V \
29  ▷ ▷ ▷ -nauty_control -end \
30  ▷ ▷ -end \
31  ▷ ▷ -with G -do \
32  ▷ ▷ -group_theoretic_activity \
33  ▷ ▷ ▷ -report Do \
34  ▷ ▷ -end \
35  ▷ ▷ -assign elements -from \
36  ▷ ▷ -with G -do \
37  ▷ ▷ -group_theoretic_activity \
38  ▷ ▷ ▷ -elements \
39  ▷ ▷ -end \
40  ▷ ▷ -assign elements2 -from \
41  ▷ ▷ -with elements -do \
42  ▷ ▷ -vector_ge_activity \
43  ▷ ▷ ▷ -filter_subfield 2 \
44  ▷ ▷ -end \
45  ▷ ▷ -with elements2 -do \
46  ▷ ▷ -vector_ge_activity \
47  ▷ ▷ ▷ -report \
48  ▷ ▷ -end \
49  ▷ ▷ -with elements2 -do \
50  ▷ ▷ -vector_ge_activity \
51  ▷ ▷ ▷ -report_elements_coded \
52  ▷ ▷ -end \
53  ▷ ▷ -print_symbols
54  ▷ $(PDFLATEX) PGGL_4.4_stab_of_Hirschfeld_4_elements2_elements.tex
55  ▷ $(OPEN) PGGL_4.4_stab_of_Hirschfeld_4_elements2_elements.pdf
56

```

5.9 Activities for Vectors of Group Elements

Table 5.16 lists the commands that are available for activities for vectors of group elements.

The following command computes and prints the conjugacy classes in $\text{PTL}(2, 4)$:

Code Snippet 268

```

1    PGGL_2_4.classes_reps:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define G \
4    ▷ ▷ -linear_group -PGGL 2 4 \
5    ▷ ▷ -end \
6    ▷ ▷ -assign reps -from \
7    ▷ ▷ -with G -do \
8    ▷ ▷ -group_theoretic_activity \
9    ▷ ▷ ▷ -classes \
10   ▷ ▷ -end \
11   ▷ ▷ -with reps -do \
12   ▷ ▷ -vector_ge_activity \
13   ▷ ▷ ▷ -report \
14   ▷ ▷ -end \
15   ▷ ▷ -print_symbols
16   ▷ $(PDFLATEX) PGGL_2_4.reps.elements.tex
17   ▷ $(OPEN) PGGL_2_4.reps.elements.pdf
18
```

The class representatives are chosen by Magma.

The next command computes and prints the rational normal form for the representatives of the conjugacy classes in $\text{PGL}(4, 4)$

Code Snippet 269

```

1    PGL_4_4.classes_reps:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define G \
4    ▷ ▷ -linear_group -PGL 4 4 \
5    ▷ ▷ -end \
6    ▷ ▷ -assign reps -from \
7    ▷ ▷ -with G -do \
8    ▷ ▷ -group_theoretic_activity \
9    ▷ ▷ ▷ -classes \
10   ▷ ▷ -end \
11   ▷ ▷ -with reps -do \
12   ▷ ▷ -vector_ge_activity \
13   ▷ ▷ ▷ -report \
14   ▷ ▷ -end \
15   ▷ ▷ -assign RNF -and BaseChange -from \
16   ▷ ▷ -with reps -do \
17   ▷ ▷ -vector_ge_activity \
18   ▷ ▷ ▷ -rational_canonical_form \
19   ▷ ▷ -end \

```

Vector of Group Elements Activities		
Command	Arguments	Purpose
-report		Produce a report of the group elements in the vector.
-report_with_options	options	Produce a report of the group elements in the vector using the given options. Options must be a comma separated list of “key=value” pairs.
-report_elements_coded		Produce a report of the group elements in the vector in coded form.
-export_GAP		Export the group elements to GAP and generate the group.
-transform_variety	variety	Transform the given variety by the group elements in the vector.
-multiply		Multiply corresponding elements in the input vectors. Multiple input vectors can be given using the -with command.
-conjugate		Conjugate all elements of the first vector by the first (and only) entry in the second vector. Multiple input vectors can be given using the -with command.
-conjugate_by	elt	Conjugate all elements of the given vector by the element given in text form.
-conjugate_inverse		Same as -conjugate, but use the inverse element instead.
-select_subset	indexset	Create a new vector from the elements given in the index set. The index set must be a label of a vector of integers. Indexing of elements of the vector is zero-based.
-field_reduction	s	Perform field reduction w.r.t. the subfield of the given index s. The group must be a matrix group over a field admitting a subfield of index s.
-rational_canonical_form		Computes the rational normal form of each element. Returns a vector containing all rational normal forms and a second vector containing all transformation matrices.
-products_of_pairs		Computes a matrix whose entry a_{ij} is the order of the product of element i with element j .
-apply_isomorphism_wedge_product_4to6		For each vector element, compute the matrix for the induced action on the wedge product. The result will be written to csv file.
-filter_subfield	q	Produces a new vector whose elements are the elements in that belong to the subfield (and – for semilinear groups – whose associated field automorphism is trivial).

Table 5.16: Vector of Group Elements Activities

```

20  ▷ ▷ -with RNF -do \
21  ▷ ▷ -vector_ge_activity \
22  ▷ ▷ ▷ -report_with_options "dense=on" \
23  ▷ ▷ -end \
24  ▷ ▷ -print_symbols
25  ▷ $(PDFLATEX) PGL_4_4_RNF_elements.tex
26  ▷ $(OPEN) PGL_4_4_RNF_elements.pdf
27  ▷ #$(PDFLATEX) PGL_4_4_reps_elements.tex
28  ▷ #$(OPEN) PGL_4_4_reps_elements.pdf
29

```

The next command computes and prints the conjugacy classes in $PTL(4, 4)$:

Code Snippet 270

```

1    PGGL_4_4.classes_reps:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define G \
4    ▷ ▷ -linear_group -PGGL 4 4 \
5    ▷ ▷ -end \
6    ▷ ▷ -assign_reps -from \
7    ▷ ▷ -with G -do \
8    ▷ ▷ -group_theoretic_activity \
9    ▷ ▷ ▷ -classes \
10   ▷ ▷ -end \
11   ▷ ▷ -with_reps -do \
12   ▷ ▷ -vector_ge_activity \
13   ▷ ▷ ▷ -report \
14   ▷ ▷ -end \
15   ▷ ▷ -print_symbols
16   ▷ $(PDFLATEX) PGGL_4_4_reps_elements.tex
17   ▷ $(OPEN) PGGL_4_4_reps_elements.pdf
18

```

The class representatives are chosen by Magma.

The next command illustrates how to obtain the generators of a group as a vector of group elements object. In the example, we consider the group $Sym(6)$.

Code Snippet 271

```

1    Sym_6.generators:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define G -permutation_group -symmetric_group 6 -end \
4    ▷ ▷ -assign_gens -from \
5    ▷ ▷ -with G -do \
6    ▷ ▷ -group_theoretic_activity \
7    ▷ ▷ ▷ -generators \
8    ▷ ▷ -end \
9    ▷ ▷ -with_gens -do \
10   ▷ ▷ -vector_ge_activity \
11   ▷ ▷ ▷ -report \

```

```

12  ▷ ▷ -end \
13  ▷ ▷ -print_symbols
14  ▷ $(PDFLATEX) Sym_6_gens_elements.tex
15  ▷ $(OPEN) Sym_6_gens_elements.pdf
16

```

The command produces a latex report of the generators, which are the Coxeter generators.

```

Group elements in action Sym6
Element 0 / 5 of order 2:
                                (0)(1)(2)(3)(4,5)

(0)(1)(2)(3)(4,5)
0, 1, 2, 3, 5, 4,
Element 1 / 5 of order 2:
                                (0)(1)(2)(3,4)(5)

(0)(1)(2)(3,4)(5)
0, 1, 2, 4, 3, 5,
Element 2 / 5 of order 2:
                                (0)(1)(2,3)(4)(5)

(0)(1)(2,3)(4)(5)
0, 1, 3, 2, 4, 5,
Element 3 / 5 of order 2:
                                (0)(1,2)(3)(4)(5)

(0)(1,2)(3)(4)(5)
0, 2, 1, 3, 4, 5,
Element 4 / 5 of order 2:
                                (0,1)(2)(3)(4)(5)

(0,1)(2)(3)(4)(5)
1, 0, 2, 3, 4, 5,
Order structure:
25

```

The next command illustrates how to obtain the elements of a group as a vector of group elements object. In the example, we consider the group $\text{Sym}(6)$ with 720 elements. The command produces a latex report of the group elements.

Code Snippet 272

```

1  Sym_6_elements:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define G -permutation.group -symmetric.group 6 -end \
4  ▷ ▷ -assign elements -from \
5  ▷ ▷ -with G -do \
6  ▷ ▷ -group.theoretic.activity \
7  ▷ ▷ -elements \
8  ▷ ▷ -end \

```

```

9      ▷ ▷ -with elements -do \
10     ▷ ▷ -vector_ge.activity \
11     ▷ ▷ ▷ -report \
12     ▷ ▷ -end \
13     ▷ ▷ -print_symbols
14     ▷ $(PDFLATEX) Sym_6_elements_elements.tex
15     ▷ $(OPEN) Sym_6_elements_elements.pdf
16

```

The following command creates the conjugacy class of orthogonal reflection in $\text{PGO}^+(6, 2)$ and pushes them back into $\text{PGL}(4, 2)$ using the inverse of the exterior square action. Because the group elements lie in $\text{PGO}^+(6, 2)$ but not in the subgroup isomorphic to $\text{PGL}(4, 2)$, a fixed polarity is divided off before the push back. After that, a report of the 28 group elements is produced. Finally, the group elements are exported to GAP. Using GAP, the order of the group generated by these elements turns out to be 20160.

The command

Code Snippet 273

```

1      GL_2_3_RNF_elements:
2      ▷ $(ORBITER) -v 2 \
3      ▷ ▷ -define Do -draw_options \
4      ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5      ▷ ▷ -end \
6      ▷ ▷ -define F -finite_field -q 3 -end \
7      ▷ ▷ -define G -linear_group -GL 2 F -end \
8      ▷ ▷ -assign elements -from \
9      ▷ ▷ -with G -do \
10     ▷ ▷ -group_theoretic_activity \
11     ▷ ▷ ▷ -elements \
12     ▷ ▷ -end \
13     ▷ ▷ -assign RNF -and BaseChange -from \
14     ▷ ▷ -with elements -do \
15     ▷ ▷ -vector_ge.activity \
16     ▷ ▷ ▷ -rational_canonical_form \
17     ▷ ▷ -end \
18     ▷ ▷ -with RNF -do \
19     ▷ ▷ -vector_ge.activity \
20     ▷ ▷ ▷ -report_with_options "dense=on" \
21     ▷ ▷ -end \
22     ▷ ▷ -with BaseChange -do \
23     ▷ ▷ -vector_ge.activity \
24     ▷ ▷ ▷ -report_with_options "dense=on" \
25     ▷ ▷ -end \
26     ▷ ▷ -print_symbols
27     ▷ $(PDFLATEX) GL_2_3_RNF_elements.tex
28     ▷ $(OPEN) GL_2_3_RNF_elements.pdf
29

```

computes the rational normal form of each element in $\text{GL}(2, 3)$. A transformation matrix is computed as well. The output is:

$$\begin{aligned}
& \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}, \\
& \begin{bmatrix} 0 & 1 \\ 1 & 2 \end{bmatrix}, \begin{bmatrix} 0 & 2 \\ 1 & 0 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 1 & 2 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}, \\
& \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 2 \\ 1 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 2 \\ 1 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 1 & 2 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}, \\
& \begin{bmatrix} 0 & 1 \\ 1 & 2 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}, \begin{bmatrix} 0 & 2 \\ 1 & 0 \end{bmatrix}, \\
& \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 1 & 2 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}, \begin{bmatrix} 0 & 2 \\ 1 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 1 & 2 \end{bmatrix}, \\
& \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}
\end{aligned}$$

Code Snippet 274

```

1   PGL_4_2.orthogonal_reflections_report_vector_ge:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Pts -vector \
4   ▷ ▷ ▷ -dense $(ORTHOGONAL_OP_6_2_POINTS_OFF) \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 2 -end \
7   ▷ ▷ -define O -orthogonal_space 1 6 F -end \
8   ▷ ▷ -define G4 -linear_group -PGL 4 2 -end \
9   ▷ ▷ -with O -do -orthogonal_space_activity \
10  ▷ ▷ ▷ -create_orthogonal_reflection_6_and_4 Pts G4 \
11  ▷ ▷ -end
12  ▷ $(ORBITER) -v 2 \
13  ▷ ▷ -define F -finite_field -q 2 -end \
14  ▷ ▷ -define G -linear_group -PGL 4 F -end \
15  ▷ ▷ -define reflections -vector_ge \
16  ▷ ▷ ▷ -action G \
17  ▷ ▷ ▷ -read_csv 0.1_6.2.orthogonal_reflections_4x4.csv Element \
18  ▷ ▷ -end \
19  ▷ ▷ -with reflections -do \
20  ▷ ▷ -vector_ge_activity \
21  ▷ ▷ ▷ -report \
22  ▷ ▷ -end \
23  ▷ ▷ -with reflections -do \
24  ▷ ▷ -vector_ge_activity \
25  ▷ ▷ ▷ -export_GAP \
26  ▷ ▷ -end \
27  ▷ ▷ -assign RNF -and BaseChange -from \
28  ▷ ▷ -with reflections -do \
29  ▷ ▷ -vector_ge_activity \
30  ▷ ▷ ▷ -rational_canonical_form \
31  ▷ ▷ -end \

```

```

32  ▷ ▷ -with RNF -do \
33  ▷ ▷ -vector_ge_activity \
34  ▷ ▷ ▷ -report_with_options "dense=on" \
35  ▷ ▷ -end \
36  ▷ ▷ -print_symbols
37  ▷ $(PDFLATEX) PGL_4_2_reflections_elements.tex
38  ▷ $(OPEN) PGL_4_2_reflections_elements.pdf
39  ▷ $(PDFLATEX) PGL_4_2_RNF_elements.tex
40  ▷ $(OPEN) PGL_4_2_RNF_elements.pdf
41

```

The command produces a latex report of the group elements, which are matrices of size 4. The associated permutation action on the 15 points of $PG(3, 2)$ is shown also. For reasons of space, the output has been truncated.

Group elements in action $PGL(4, 2)$

Element 0 / 28 of order 2:

$$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

(0, 14)(1, 13)(2, 11)(3, 8)(4)(5)(6)(7)(9)(10)(12)

14, 13, 11, 8, 4, 5, 6, 7, 3, 9, 10, 2, 12, 1, 0,

Element 1 / 28 of order 2:

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

(0, 1)(2, 3)(4)(5)(6, 10)(7, 9)(8, 11)(12)(13, 14)

1, 0, 3, 2, 4, 5, 10, 9, 11, 7, 6, 8, 12, 14, 13,

Element 2 / 28 of order 6:

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

(0, 1, 9, 2, 3, 7)(4, 6, 10)(5, 11, 13, 12, 14, 8)

1, 9, 3, 7, 6, 11, 10, 0, 5, 2, 4, 13, 14, 12, 8,

Element 3 / 28 of order 6:

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

(0, 7, 3, 2, 9, 1)(4, 10, 6)(5, 8, 14, 12, 13, 11)

7, 0, 9, 2, 10, 8, 4, 3, 14, 1, 6, 5, 13, 11, 12,

Output truncated

Order structure:

$6^{12}, 4^{12}, 2^4$

Chapter 6

Ring Theory

6.1 Univariate Polynomials Over Finite Fields

For p prime, the finite field $\mathbb{F}_p$ of order p can be constructed as factorring of the integers moulo p . In this section, we will consider polynomials over $\mathbb{F}_p$. The ring of polynomials in one variable with coefficients in $\mathbb{F}_p$ is denoted as $\mathbb{F}_p[X]$. Table 6.1 lists Orbiter finite field activities related to polynomials. For instance, the command

Code Snippet 275

```
1  poly_division:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -with F -do \
5  ▷ ▷ -finite_field_activity \
6  ▷ ▷ ▷ -polynomial_division \
7  ▷ ▷ ▷ "1,0,0,0,0,0,0,0,0,0,1" "1,0,1,1" \
8  ▷ ▷ -end
9
```

computes the polynomial long division of $A(X)$ by $B(X)$ over $\mathbb{F}_2$ where

$$A(X) = X^{10} + 1, \quad B(X) = X^3 + X^2 + 1.$$

The result is $Q(X)$ and $R(X)$ with

$$A(X) = Q(X) \cdot B(X) + R(X)$$

with

$$Q(X) = X^7 + X^6 + X^5 + X^3 + 1, \quad R(X) = X^2.$$

The coefficient lists in the arguments are from the lowest term up.

It is perhaps more convenient to create the polynomials as vectors, as in Section 2.5. The following example uses vectors named A and B . After that, the division command is called.

Code Snippet 276

```
1  poly_division2:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
```

Finite Field Activities Related to Polynomials		
Command	Arguments	Purpose
-polynomial_division	$A(X) \ B(X)$	Polynomial division of $A(X)$ by $B(X)$ over $\mathbb{F}_q$. $A(X)$ and $B(X)$ are given as coefficient list, starting from the lowest coefficient.
-extended_gcd_for_polynomials	$A(X) \ B(X)$	Extended gcd for polynomials $A(X)$ and $B(X)$ over $\mathbb{F}_q$. $A(X)$ and $B(X)$ are given as coefficient list, starting from the lowest coefficient.
-polynomial_mult_mod	$A(X) \ B(X) \ M(X)$	Multiply the polynomials $A(X)$ and $B(X)$ modulo $M(X)$ in $\mathbb{F}_q[X]$.
-Berlekamp_matrix	$A(X)$	Computes the rank of the Berlekamp matrix associated to the polynomial $A(X)$ over $\mathbb{F}_q$. The polynomial $A(X)$ is irreducible over $\mathbb{F}_q$ if the Berlekamp matrix has rank $d-1$, where d is the degree of $A(X)$. The Berlekamp matrix is $F - I$ where F is the Frobenius matrix and I is the identity matrix. The Frobenius matrix is the matrix of the Frobenius endomorphism with respect to the standard basis of the polynomial ring: $1, X, X^2, \dots, X^{d-1}$.
-polynomial_find_roots	$A(X)$	Find the roots of $A(X)$ over $\mathbb{F}_q$.
-get_primitive_polynomial	d	Get a primitive polynomial of degree d over $\mathbb{F}_q$.
-get_primitive_polynomial_in_range	$d_{\min} \ d_{\max}$	Get a primitive polynomial of degree d over $\mathbb{F}_q$ for all $d \in [d_{\min}, d_{\max}]$.
-make_table_of_irreducible_polynomials	d	Produces a list of all irreducible polynomials of degree d over $\mathbb{F}_q$.

Table 6.1: Finite Field Activities Related to Polynomials

```

4    ▷ ▷ -define A -vector -field F -sparse 11 "1,0,1,10" -end \
5    ▷ ▷ -define B -vector -field F -dense "1,0,1,1" -end \
6    ▷ ▷ -with F -do \
7    ▷ ▷ -finite_field_activity \
8    ▷ ▷ ▷ -polynomial_division A B \
9    ▷ ▷ -end
10

```

The command `-extended_gcd_for_polynomials` takes two polynomials $A(X)$ and $B(X)$ and computes polynomials $U(X)$ and $V(X)$ and $G(X)$ such that $G(X)$ is the greatest common divisor of $A(X)$ and $B(X)$ and

$$G(X) = U(X) \cdot A(X) + V(X) \cdot B(X).$$

For instance,

Code Snippet 277

```

1    poly_gcd:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 2 -end \
4    ▷ ▷ -with F -do \
5    ▷ ▷ -finite_field_activity \
6    ▷ ▷ ▷ -extended_gcd_for_polynomials \
7    ▷ ▷ ▷ ▷ "1,0,0,0,0,0,0,0,0,1" "1,0,1,1" \
8    ▷ ▷ -end
9

```

computes

$$U(X) = X + 1, \quad V(X) = X^8 + X^5 + X^4 + X^3 + X, \quad G(X) = 1.$$

The next command computes

$$(3X^2 + 2X + 1) \cdot (5X^2 + 4X + 3) \mod (X^3 + 6) \mod 7.$$

Code Snippet 278

```

1    poly_mult_mod1:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 7 -end \
4    ▷ ▷ -with F -do \
5    ▷ ▷ -finite_field_activity \
6    ▷ ▷ ▷ -polynomial_mult_mod "1,2,3" "3,4,5" "6,0,0,1" \
7    ▷ ▷ -end
8

```

which has a result of

$$X^2 + 4X + 4.$$

The coefficients are given from the lowest to the highest term. For the opposite order, the following command computes

$$(2X^2 + X + 3) \cdot (4X^2 + 3X + 5) \mod (X^3 + 6) \mod 7.$$

Code Snippet 279

```

1  poly_mult_mod2:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 7 -end \
4  ▷ ▷ -with F -do \
5  ▷ ▷ -finite_field_activity \
6  ▷ ▷ ▷ -polynomial_mult_mod "3,1,2" "5,3,4" "6,0,0,1" \
7  ▷ ▷ -end
8

```

The result is

$$4X^2 + X + 4.$$

The finite field $\mathbb{F}_4$ can be defined by using polynomial arithmetic over $\mathbb{F}_2$ modulo $X^2 + X + 1$. Here is a command that computes the three non-trivial products of polynomials:

Code Snippet 280

```

1  poly_mult_mod_F4:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -with F -do \
5  ▷ ▷ -finite_field_activity \
6  ▷ ▷ ▷ -polynomial_mult_mod "1,1" "1,1" "1,1,1" \
7  ▷ ▷ -end
8  ▷ $(ORBITER) -v 2 \
9  ▷ ▷ -define F -finite_field -q 2 -end \
10 ▷ ▷ -with F -do \
11 ▷ ▷ -finite_field_activity \
12 ▷ ▷ ▷ -polynomial_mult_mod "0,1" "1,1" "1,1,1" \
13 ▷ ▷ -end
14 ▷ $(ORBITER) -v 2 \
15 ▷ ▷ -define F -finite_field -q 2 -end \
16 ▷ ▷ -with F -do \
17 ▷ ▷ -finite_field_activity \
18 ▷ ▷ ▷ -polynomial_mult_mod "0,1" "0,1" "1,1,1" \
19 ▷ ▷ -end
20

```

It is possible to use numerical values for polynomials, using the representation in radix q . The following command computes the product of the polynomials 5 and 7 over $\mathbb{F}_2$:

Code Snippet 281

```

1  mult_polynomials_2.5.7:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -with F -do \
5  ▷ ▷ -finite_field_activity \
6  ▷ ▷ ▷ -mult_polynomials 5 7 \
7  ▷ ▷ -end

```

```

8    ▷ $(PDFLATEX) polynomial_mult_5_7.tex
9    ▷ $(OPEN) polynomial_mult_5_7.pdf
10

```

The next command performs polynomial long division based on numerical polynomials:

Code Snippet 282

```

1    polynomial_division_ranked_2_27_13:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 2 -end \
4    ▷ ▷ -with F -do \
5    ▷ ▷ -finite_field_activity \
6    ▷ ▷ ▷ -polynomial_division_ranked 27 13 \
7    ▷ ▷ -end
8    ▷ $(PDFLATEX) polynomial_division_27_13.tex
9    ▷ $(OPEN) polynomial_division_27_13.pdf
10

```

Here is a somewhat larger example for numerical arguments. We wish to multiply 999 by 997 modulo 1033. The first command performs multiplication:

Code Snippet 283

```

1    mult_polynomials_1024_999_997:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 2 -end \
4    ▷ ▷ -with F -do \
5    ▷ ▷ -finite_field_activity \
6    ▷ ▷ ▷ -mult_polynomials 999 997 \
7    ▷ ▷ -end
8    ▷ $(PDFLATEX) polynomial_mult_999_997.tex
9    ▷ $(OPEN) polynomial_mult_999_997.pdf
10

```

The next command performs division with remainder:

Code Snippet 284

```

1    polynomial_division_ranked_2_349147_1033:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 2 -end \
4    ▷ ▷ -with F -do \
5    ▷ ▷ -finite_field_activity \
6    ▷ ▷ ▷ -polynomial_division_ranked 349147 1033 \
7    ▷ ▷ -end
8    ▷ $(PDFLATEX) polynomial_division_349147_1033.tex
9    ▷ $(OPEN) polynomial_division_349147_1033.pdf
10

```

Orbiter allows polynomial arithmetic modulo a factor polynomial. The coefficient vector of the polynomial can be created as a vector. Here is an example which performs arithmetic modulo the CRC32 polynomial.

The goal is to compute the multiplicative inverse of X . In order to do so, we use the fact that the CRC32 polynomial is irreducible, and hence the factor ring is a finite field of order 2^{32} . The inverse of a polynomial can be computed by raising to the power of $2^{32} - 2$:

Code Snippet 285

```
1 CRC32_SPARSE="1,32,1,26,1,23,1,22,1,16,1,12,1,11,\
2 1,10,1,8,1,7,1,5,1,4,1,2,1,1,1,0"
3
```

Code Snippet 286

```
1 TWO_TO_THE_32_MINUS_2=4294967294
2
```

Code Snippet 287

```
1 power_mod_inverse:
2 ▷ $(ORBITER) -v 2 \
3 ▷ ▷ -define F -finite_field -q 2 -end \
4 ▷ ▷ -define M -vector -field F -sparse 33 $(CRC32_SPARSE) -end \
5 ▷ ▷ -define A -vector -field F -sparse 2 "1,1" -end \
6 ▷ ▷ -with F -do \
7 ▷ ▷ -finite_field_activity \
8 ▷ ▷ ▷ -polynomial_power_mod A $(TWO_TO_THE_32_MINUS_2) M \
9 ▷ ▷ -end
10
```

This command produces the polynomial

$$B(X) = X^{31} + X^{25} + X^{22} + X^{21} + X^{15} + X^{11} + X^{10} + X^9 + X^7 + X^6 + X^4 + X^3 + X + 1$$

In order to test that this polynomial really is the multiplicative inverse of X modulo CRC32, we perform the following command:

Code Snippet 288

```
1 INVERSE_SPARSE="1,31,1,25,1,22,1,21,1,15,\
2 1,11,1,10,1,9,1,7,1,6,1,4,1,3,1,1,1,0"
3
```

Code Snippet 289

```
1 mult_mod_to_get_one:
2 ▷ $(ORBITER) -v 2 \
3 ▷ ▷ -define F -finite_field -q 2 -end \
4 ▷ ▷ -define M -vector -field F -sparse 33 $(CRC32_SPARSE) -end \
5 ▷ ▷ -define A -vector -field F -sparse 2 "1,1" -end \
6 ▷ ▷ -define B -vector -field F -sparse 33 $(INVERSE_SPARSE) -end \
7 ▷ ▷ -with F -do \
8 ▷ ▷ -finite_field_activity \
9 ▷ ▷ ▷ -polynomial_mult_mod A B M \
10 ▷ ▷ -end
```


11

The product is indeed 1.

The Berlekamp matrix can be used to test if a polynomial is irreducible over a given finite field. The polynomial is irreducible if and only if the rank of the Berlekamp matrix is $d - 1$, where d is the degree of the polynomial. For instance, the command

Code Snippet 290

```

1   Berlekamp_matrix_2_3:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 2 -end \
4   ▷ ▷ -define v -vector -field F -dense "1,1,0,1" -end \
5   ▷ ▷ -with F -do \
6   ▷ ▷ -finite_field_activity \
7   ▷ ▷ ▷ -Berlekamp_matrix v \
8   ▷ ▷ -end
9   ▷ $(PDFLATEX) Berlekamp_matrix_q2_d3.tex
10  ▷ $(OPEN) Berlekamp_matrix_q2_d3.pdf
11

```

computes the Berlekamp matrix associated with the polynomial $X^3 + X + 1$ over $\mathbb{F}_2$. The matrix is

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}.$$

Since the matrix has rank 2, the polynomial is irreducible.

Orbiter can compute irreducible and primitive polynomials. The following command creates a list of all irreducible polynomials of degree 3 over $\mathbb{F}_4$:

Code Snippet 291

```

1   irred_3_4:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define F -finite_field -q 4 -end \
4   ▷ ▷ -with F -do \
5   ▷ ▷ -finite_field_activity \
6   ▷ ▷ ▷ -make_table_of_irreducible_polynomials 3 \
7   ▷ ▷ -end
8   ▷ $(PDFLATEX) Irred_q4_d3.tex
9   ▷ $(OPEN) Irred_q4_d3.pdf
10

```

The output is:

There are 20 irreducible polynomials of degree 3 over the field F4:

```

0 : 3211 : 91 :  $X^3 + X^2 + 2 * X + 3$ 
1 : 1301 : 77 :  $X^3 + 3 * X + 1$ 
2 : 3121 : 103 :  $X^3 + 2 * X^2 + X + 3$ 
3 : 3231 : 123 :  $X^3 + 3 * X^2 + 2 * X + 3$ 

```

```

4 : 2231 : 122 :  $X^3 + 3 * X^2 + 2 * X + 2$ 
5 : 2221 : 106 :  $X^3 + 2 * X^2 + 2 * X + 2$ 
6 : 1201 : 73 :  $X^3 + 2 * X + 1$ 
7 : 1011 : 81 :  $X^3 + X^2 + 1$ 
8 : 3331 : 127 :  $X^3 + 3 * X^2 + 3 * X + 3$ 
9 : 2321 : 110 :  $X^3 + 2 * X^2 + 3 * X + 2$ 
10 : 3111 : 87 :  $X^3 + X^2 + X + 3$ 
11 : 3321 : 111 :  $X^3 + 2 * X^2 + 3 * X + 3$ 
12 : 1031 : 113 :  $X^3 + 3 * X^2 + 1$ 
13 : 3001 : 67 :  $X^3 + 3$ 
14 : 2111 : 86 :  $X^3 + X^2 + X + 2$ 
15 : 2001 : 66 :  $X^3 + 2$ 
16 : 2131 : 118 :  $X^3 + 3 * X^2 + X + 2$ 
17 : 1101 : 69 :  $X^3 + X + 1$ 
18 : 2311 : 94 :  $X^3 + X^2 + 3 * X + 2$ 
19 : 1021 : 97 :  $X^3 + 2 * X^2 + 1$ 

```

The following command creates a primitive polynomial of degree 10 over $\mathbb{F}_2$:

Code Snippet 292

```

1  F2_get_primitive_poly_of_deg_10:
2  ▷ $(ORBITER) -v 6 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -with F -do \
5  ▷ ▷ -finite_field_activity \
6  ▷ ▷ ▷ -get_primitive_polynomial 10 \
7  ▷ ▷ -end
8

```

The following command creates one primitive polynomial over $\mathbb{F}_2$ for each degree in the range [2, 10]:

Code Snippet 293

```

1  F2_get_primitive_poly_range_2_10:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -with F -do \
5  ▷ ▷ -finite_field_activity \
6  ▷ ▷ ▷ -get_primitive_polynomial_in_range 2 10 \
7  ▷ ▷ -end
8

```

The unix command `grep` is used to filter the output for lines containing the given pattern “//”. This yields the list

```

"7", //  $X^2 + X + 1$ 
"13", //  $X^3 + X^2 + 1$ 
"25", //  $X^4 + X^3 + 1$ 
"37", //  $X^5 + X^2 + 1$ 

```

```

"97", // X^{6} + X^{5} + 1
"193", // X^{7} + X^{6} + 1
"285", // X^{8} + X^{4} + X^{3} + X^{2} + 1
"529", // X^{9} + X^{4} + 1
"1033", // X^{10} + X^{3} + 1

```

Primitive polynomials over the base field $\mathbb{F}_s$ are converted into integers, using the base- s representation of integers. For instance, the polynomial $X^2 + X + 1$ is read as binary string 111, which in turn translates to the integer 7 (we use $s = 2$).

Creating Multivariate Polynomial Rings		
Command	Arguments	Purpose
-field	label	Specify the field of coefficients.
-homogeneous_of_degree	d	Specify the degree d of polynomials.
-number_of_variables	n	Specify the number n of variables.
-monomial_ordering_partition		Set monomial ordering to partition ordering.
-monomial_ordering_lex		Set monomial ordering to lexicographic ordering.
-variables	label-txt label-tex	Specify variable labels in ascii and in latex.

Table 6.2: Creating Multivariate Polynomial Rings

6.2 Multivariate Polynomial Rings

Orbiter can work with multivariate polynomial rings. Table 6.2 lists the commands for creating a multivariate polynomial ring.

Table 6.3 lists the activities for a multivariate polynomial ring.

Orbiter supports two orderings of the monomials.

1. The lexicographic ordering orders the monomials lexicographically.
2. In the partition ordering, the monomials are first ordered according to the partition arising from the exponent vector. For monomials with the same partition, the lexicographic ordering is used as a tie breaker.

By default, the partition ordering is used. Table 6.4 shows the monomials in the partition ordering for degrees 1, 2, 3 and 4 in a plane.

Table 6.5 shows the partition ordering of monomials of degree at most 3 in $\text{PG}(3, q)$.

The following command creates a polynomial ring in 4 variables over the field $\mathbb{F}_{13}$. The homogeneous component of degree 4 is created. A report is produced using the command `-cheat_sheet`.

Code Snippet 294

```

1  polynomial_ring_degree3_vars4_q13:
2  ▷ $(ORBITER) -v 6 \
3  ▷ ▷ -define Do -draw_options \
4  ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5  ▷ ▷ -end \
6  ▷ ▷ -define F -finite_field -q 13 -end \
7  ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
8  ▷ ▷ -define R -polynomial_ring \
9  ▷ ▷ ▷ -field F \
10 ▷ ▷ ▷ -number_of_variables 4 \
11 ▷ ▷ ▷ -homogeneous_of_degree 3 \
12 ▷ ▷ ▷ -monomial_ordering_partition \
13 ▷ ▷ ▷ -variables "x0,x1,x2,x3" "x_0,x_1,x_2,x_3" \
14 ▷ ▷ -end \
15 ▷ ▷ -with R -do \
16 ▷ ▷ ▷ -ring_theoretic_activity \

```

Multivariate Polynomial Ring Activities		
Command	Arguments	Purpose
-cheat_sheet		Create a report about the polynomial ring.
-export_partials		Export the matrix associated with the partial derivatives.
-ideal	label_txt label_tex label_set	Compute the ideal of the given set of points.
-apply_transformation	P eqn elt	Apply the transformation given by the group element to the given equation. P is the underlying projective space.
-set_variable_names	txt tex	Choose new variable names in text and tex format.
-print_equation	equation	Print the given equation.
-parse_equation_wo_parameters	label_txt label_tex eqn_algebraic	Parse an equation given in algebraic form.
-parse_equation	label_txt label_tex eqn_algebraic params param_val	Parse an equation given in algebraic form and perform parameter substitution.
-table_of_monomials_write_csv	label	Produce a csv file with the list of monomials in the chosen ordering.

Table 6.3: Multivariate Polynomial Ring Activities

Monomial Ordering of Partition Type in the Plane		
Degrees 1 and 2		
h	mon	vector
0	X_0	$(1, 0, 0)$
1	X_1	$(0, 1, 0)$
2	X_2	$(0, 0, 1)$
h	mon	vector
0	X_0^2	$(2, 0, 0)$
1	X_1^2	$(0, 2, 0)$
2	X_2^2	$(0, 0, 2)$
3	X_0X_1	$(1, 1, 0)$
4	X_0X_2	$(1, 0, 1)$
5	X_1X_2	$(0, 1, 1)$
Degree 3		
h	mon	vector
0	X_0^3	$(3, 0, 0)$
1	X_1^3	$(0, 3, 0)$
2	X_2^3	$(0, 0, 3)$
3	$X_0^2X_1$	$(2, 1, 0)$
4	$X_0^2X_2$	$(2, 0, 1)$
5	$X_0X_1^2$	$(1, 2, 0)$
6	$X_1^2X_2$	$(0, 2, 1)$
7	$X_0X_2^2$	$(1, 0, 2)$
8	$X_1X_2^2$	$(0, 1, 2)$
9	$X_0X_1X_2$	$(1, 1, 1)$
Degree 4		
h	mon	vector
0	X_0^4	$(4, 0, 0)$
1	X_1^4	$(0, 4, 0)$
2	X_2^4	$(0, 0, 4)$
3	$X_0^3X_1$	$(3, 1, 0)$
4	$X_0^3X_2$	$(3, 0, 1)$
5	$X_0X_1^3$	$(1, 3, 0)$
6	$X_1^3X_2$	$(0, 3, 1)$
7	$X_0X_2^3$	$(1, 0, 3)$
8	$X_1X_2^3$	$(0, 1, 3)$
9	$X_0^2X_1^2$	$(2, 2, 0)$
10	$X_0^2X_2^2$	$(2, 0, 2)$
11	$X_1^2X_2^2$	$(0, 2, 2)$
12	$X_0^2X_1X_2$	$(2, 1, 1)$
13	$X_0X_1^2X_2$	$(1, 2, 1)$
14	$X_0X_1X_2^2$	$(1, 1, 2)$

Table 6.4: Monomial Ordering of Partition Type in the Plane

Monomial Ordering of Partition Type in Three-Space		
Degree 1		
h	mon	vector
0	X_0	$(1, 0, 0, 0)$
1	X_1	$(0, 1, 0, 0)$
2	X_2	$(0, 0, 1, 0)$
3	X_3	$(0, 0, 0, 1)$
Degree 2		
h	mon	vector
0	X_0^2	$(2, 0, 0, 0)$
1	X_1^2	$(0, 2, 0, 0)$
2	X_2^2	$(0, 0, 2, 0)$
3	X_3^2	$(0, 0, 0, 2)$
4	X_0X_1	$(1, 1, 0, 0)$
5	X_0X_2	$(1, 0, 1, 0)$
6	X_0X_3	$(1, 0, 0, 1)$
7	X_1X_2	$(0, 1, 1, 0)$
8	X_1X_3	$(0, 1, 0, 1)$
9	X_2X_3	$(0, 0, 1, 1)$
Degree 3		
h	mon	vector
0	X_0^3	$(3, 0, 0, 0)$
1	X_1^3	$(0, 3, 0, 0)$
2	X_2^3	$(0, 0, 3, 0)$
3	X_3^3	$(0, 0, 0, 3)$
4	$X_0^2X_1$	$(2, 1, 0, 0)$
5	$X_0^2X_2$	$(2, 0, 1, 0)$
6	$X_0^2X_3$	$(2, 0, 0, 1)$
7	$X_0X_1^2$	$(1, 2, 0, 0)$
8	$X_1^2X_2$	$(0, 2, 1, 0)$
9	$X_1^2X_3$	$(0, 2, 0, 1)$
10	$X_0X_2^2$	$(1, 0, 2, 0)$
11	$X_1X_2^2$	$(0, 1, 2, 0)$
12	$X_2^2X_3$	$(0, 0, 2, 1)$
13	$X_0X_3^2$	$(1, 0, 0, 2)$
14	$X_1X_3^2$	$(0, 1, 0, 2)$
15	$X_2X_3^2$	$(0, 0, 1, 2)$
16	$X_0X_1X_2$	$(1, 1, 1, 0)$
17	$X_0X_1X_3$	$(1, 1, 0, 1)$
18	$X_0X_2X_3$	$(1, 0, 1, 1)$
19	$X_1X_2X_3$	$(0, 1, 1, 1)$

Table 6.5: Monomial Ordering of Partition Type in Three-Space

```

17  ▷ ▷ ▷ -cheat_sheet \
18  ▷ ▷ -end
19  ▷ $(PDFLATEX) poly_ring_deg3_vars4_cheat_sheet.tex
20  ▷ $(OPEN) poly_ring_deg3_vars4_cheat_sheet.pdf
21

```

The `-parse_equation_wo_parameters` command is the easiest way to read a homogeneous algebraic equation. The command is a ring theoretic activity. At first, we define an appropriate polynomial ring. The following example defines the equation of the Fermat cubic curve in $\text{PG}(2, 13)$, using the coordinates y_0, y_1, y_2 . The formula is given in algebraic notation. The variables are written as y_0, y_1, y_2 , respectively. The multiplication symbol “ $\ast$ ” must be written. Exponentiation is indicated by the “ $\wedge$ ” symbol.

Code Snippet 295

```

1  parse_homogeneous_polynomial:
2  ▷ $(ORBITER) -v 6 \
3  ▷ ▷ -define F -finite_field -q 13 -end \
4  ▷ ▷ -define R -polynomial_ring \
5  ▷ ▷ ▷ -field F \
6  ▷ ▷ ▷ -number_of_variables 3 \
7  ▷ ▷ ▷ -homogeneous_of_degree 3 \
8  ▷ ▷ ▷ -monomial_ordering_partition \
9  ▷ ▷ ▷ -variables "y0,y1,y2" "y_0,y_1,y_2" \
10 ▷ ▷ -end \
11 ▷ ▷ -with R -do \
12 ▷ ▷ ▷ -ring_theoretic_activity \
13 ▷ ▷ ▷ -parse_equation_wo_parameters "eqn" "eqn" "y0^3+y1^3+y2^3" \
14 ▷ ▷ -end
15

```

The `-parse_equation` command parses an algebraic equation and allows for possible parameter substitutions. Suppose we want to define the Edge curve

$$X^4 - Y^4 - Z^4 + 2f^2Y^2Z^2 + 4fX^2YZ = 0,$$

where f is a primitive element. To define the curve, we use a makefile variable:

Code Snippet 296

```

1  EDGE_CURVE_EQUATION_ALGEBRAIC="X^4 -Y^4 -Z^4 +2*f^2*Y^2*Z^2 +4*f*X^2*Y*Z"
2

```

The following command creates the curve over $\mathbb{F}_{17}$. We may pick $f = 3$.

Code Snippet 297

```

1  parse_homogeneous_polynomial_with_parameters:
2  ▷ $(ORBITER) -v 6 \
3  ▷ ▷ -define F -finite_field -q 17 -end \
4  ▷ ▷ -define R -polynomial_ring \
5  ▷ ▷ ▷ -field F \

```



```

6      ▷ ▷ ▷ -number_of_variables 3 \
7      ▷ ▷ ▷ -homogeneous_of_degree 4 \
8      ▷ ▷ ▷ -monomial_ordering_partition \
9      ▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \
10     ▷ ▷ ▷ -end \
11     ▷ ▷ ▷ -with R -do \
12     ▷ ▷ ▷ -ring_theoretic_activity \
13     ▷ ▷ ▷ -parse_equation \
14     ▷ ▷ ▷ ▷ "Edge_curve" "Edge\_curve" \
15     ▷ ▷ ▷ ▷ $(EDGE_CURVE_EQUATION_ALGEBRAIC) \
16     ▷ ▷ ▷ ▷ f 3 \
17     ▷ ▷ ▷ -end
18

```

In the ordering of monomials, the equation is

$$(1, -1, -1, 0, 0, 0, 0, 0, 0, 0, 0, 2f^2, 4f, 0, 0) = (1, 16, 16, 0, 0, 0, 0, 0, 0, 0, 0, 1, 12, 0, 0).$$

The following command creates the permutation polynomial associated with the map

$$\sigma : \mathbb{F}_5 \rightarrow \mathbb{F}_5, \begin{bmatrix} 0 & 1 & 2 & 3 & 4 \\ 3 & 0 & 4 & 1 & 2 \end{bmatrix}$$

Code Snippet 298

```

1      permutation_polynomial_F5:
2      ▷ $(ORBITER) -v 6 \
3      ▷ ▷ -define F -finite_field -q 5 -end \
4      ▷ ▷ -define P -projective_space -n 1 -field F -v 0 -end \
5      ▷ ▷ -define R -polynomial_ring \
6      ▷ ▷ ▷ -field F \
7      ▷ ▷ ▷ -number_of_variables 2 \
8      ▷ ▷ ▷ -homogeneous_of_degree 4 \
9      ▷ ▷ ▷ -monomial_ordering_lex \
10     ▷ ▷ ▷ -variables "x,y" "x,y" \
11     ▷ ▷ ▷ -end \
12     ▷ ▷ -define Y0 -symbolic_object \
13     ▷ ▷ ▷ -field F \
14     ▷ ▷ ▷ -text "3*(1-x^4)+4*(1-(x-2)^4)+1*(1-(x-3)^4)+2*(1-(x-4)^4)" \
15     ▷ ▷ ▷ -end \
16     ▷ ▷ -define Y -symbolic_object \
17     ▷ ▷ ▷ -field F \
18     ▷ ▷ ▷ -text "Y0" \
19     ▷ ▷ ▷ -end \
20     ▷ ▷ -with P -do \
21     ▷ ▷ -projective_space_activity \
22     ▷ ▷ ▷ -affine_map R Y "" \
23     ▷ ▷ -end \
24     ▷ ▷ -define Yx -symbolic_object \
25     ▷ ▷ ▷ -field F \
26     ▷ ▷ ▷ -expand Y \
27     ▷ ▷ -end

```

28

The code utilizes the basis functions

$$\delta_{x,i} = 1 - (x - i)^{q-1}$$

of $\mathbb{F}_5$, which are one if $x = i$ and zero otherwise. Because Orbiter offers homogeneous polynomials only, a polynomial ring in two-variables is utilized. The polynomials are homogeneous of degree $q - 1 = 4$. We can write

$$\begin{aligned} \sigma &= \sum_{i=0}^4 \sigma(i) \delta_{x,i} \\ &= \sigma(0) \delta_{x,0} + \sigma(1) \delta_{x,1} + \sigma(2) \delta_{x,2} + \sigma(3) \delta_{x,3} + \sigma(4) \delta_{x,4} \\ &= 3\delta_{x,0} + 0\delta_{x,1} + 4\delta_{x,2} + 1\delta_{x,3} + 2\delta_{x,4}, \\ &= 3(1 - x^4) + 4(1 - (x - 2)^4) + (1 - (x - 3)^4) + 2(1 - (x - 4)^4), \end{aligned}$$

because

$$\sigma(0) = 3, \sigma(1) = 0, \sigma(2) = 4, \sigma(3) = 1, \sigma(4) = 2.$$

After expansion, the permutation polynomial associated with σ turns out to be

$$3 + x^3 + (3x^2) + (3x).$$

The next example is about a conic. We want to find the set of rational points of the conic

$$X_0^2 - X_1 * X_2$$

over $\mathbb{F}_{13}$. After that we want to find the ideal in the polynomial ring which vanishes on this point set. At first, we compute the $\mathbb{F}_{13}$ -rational points of the variety using the following command. The command utilizes a variety object, which will be discussed in Section 7.5.

Code Snippet 299

```

1   conic_variety:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define F -finite_field -q 13 -end \
4   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5   ▷ ▷ -define R -polynomial_ring \
6   ▷ ▷ ▷ -field F \
7   ▷ ▷ ▷ -number_of_variables 3 \
8   ▷ ▷ ▷ -homogeneous_of_degree 2 \
9   ▷ ▷ ▷ -monomial_ordering_partition \
10  ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
11  ▷ ▷ -end \
12  ▷ ▷ -define V -variety \
13  ▷ ▷ ▷ -projective_space P \
14  ▷ ▷ ▷ -ring R \
15  ▷ ▷ ▷ -equation_in_algebraic_form \
16  ▷ ▷ ▷ ▷ "X0^2-X1*X2" \
17  ▷ ▷ ▷ -label_txt conic \
18  ▷ ▷ ▷ -label_tex "{\\rm conic}" \
19  ▷ ▷ -end
20
```

The set of points in Orbiter numbers is

$$1, 2, 3, 39, 57, 62, 68, 77, 134, 141, 150, 151, 175, 178.$$

The following command creates the ideal of quadratic polynomials vanishing on this set:

Code Snippet 300

```

1  ideal_of_conic:
2  ▷ $(ORBITER) -v 6 \
3  ▷ ▷ -define F -finite_field -q 13 -end \
4  ▷ ▷ -define R -polynomial_ring \
5  ▷ ▷ ▷ -field F \
6  ▷ ▷ ▷ -number_of_variables 3 \
7  ▷ ▷ ▷ -homogeneous_of_degree 2 \
8  ▷ ▷ ▷ -monomial_ordering_partition \
9  ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
10 ▷ ▷ -end \
11 ▷ ▷ -with R -do \
12 ▷ ▷ ▷ -ring_theoretic_activity \
13 ▷ ▷ ▷ -ideal "conic" "conic" "1, 2, 3, 39, 57, 62, 68, 77, 134, 141, 150, 151, 175, 178" \
14 ▷ ▷ -end
15
```

The ideal is generated by

$$X_0^2 + 12 * X_1 * X_2.$$

Here, we recall that -1 is represented as 12 in $\mathbb{F}_{13}$.

Suppose we want to confirm that two algebraic varieties are isomorphic under a given transformation. Consider the case of two quartic curves over the field $\mathbb{F}_{27}$. Suppose we have two equations given by a list of coefficients, for instance as in the following makefile variables

Code Snippet 301

```

1  QC_Q27_F0159_COEFFS="15,6,12,2,4,23,24,20,15,0,4,3,0,0,23"
2
```

Code Snippet 302

```

1  QC_Q27_F0388_COEFFS="1,25,11,12,16,8,13,25,0,14,2,2,19,19,2"
2
```

Suppose further that we have a transformation as in the following makefile variable:

Code Snippet 303

```

1  QC_Q27_F0388_TRANS="19,4,0,0,26,19,10,10,11,1"
2
```

The following command applies the transformation to the second equation:

Code Snippet 304

```

1    quartic_curve_q27_fo338_apply_transformation:
2    ▷ $(ORBITER) -v 6 \
3    ▷ ▷ -define F -finite_field -q 27 -end \
4    ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5    ▷ ▷ -define R -polynomial_ring \
6    ▷ ▷ ▷ -field F \
7    ▷ ▷ ▷ -number_of_variables 3 \
8    ▷ ▷ ▷ -homogeneous_of_degree 4 \
9    ▷ ▷ ▷ -monomial_ordering_partition \
10   ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
11   ▷ ▷ -end \
12   ▷ ▷ -define eqn -vector -dense \
13   ▷ ▷ ▷ $(QC_Q27_F0388_COEFFS) \
14   ▷ ▷ ▷ -end \
15   ▷ ▷ -define trans_coeffs -vector -dense \
16   ▷ ▷ ▷ $(QC_Q27_F0388_TRANS) \
17   ▷ ▷ ▷ -end \
18   ▷ ▷ -define G -linear_group -PGGL 3 F -end \
19   ▷ ▷ -define trans -vector_ge \
20   ▷ ▷ ▷ -action G \
21   ▷ ▷ ▷ -vector_data trans_coeffs \
22   ▷ ▷ -end \
23   ▷ ▷ -with R -do \
24   ▷ ▷ ▷ -ring_theoretic_activity \
25   ▷ ▷ ▷ -apply_transformation P eqn trans \
26   ▷ ▷ -end
27

```

The output of the command shows the image equation as

$$(1, 12, 13, 4, 23, 3, 26, 15, 1, 0, 23, 24, 0, 0, 3)$$

which is the first equation up to a scalar multiple.

Chapter 7

Algebraic Geometry

7.1 The Grassmannian

Let V be a finite dimensional vector space and let $\mathfrak{Gr}_k(V)$ be the Grassmannian of k -dimensional subspaces of V . If $\dim(V) = n$, the notation $\mathfrak{Gr}_{n,k}$ is used for $\mathfrak{Gr}_k(V)$. If $V = \mathbb{F}_q^n$, the notation $\mathfrak{Gr}_{n,k,q}$ is used for $\mathfrak{Gr}_k(V)$. The order of the set $\mathfrak{Gr}_{n,k,q}$ can be computed as

$$\begin{bmatrix} n \\ k \end{bmatrix}_q = \prod_{i=0}^{k-1} \frac{q^{n-i} - 1}{q^{k-i} - 1},$$

using the q -binomial coefficient.

Orbiter has an enumerator for the Grassmannian. The purpose of this enumerator is to establish a bijection between the Grassmannian and the integers in the interval $[0, N - 1]$, where $N = \begin{bmatrix} n \\ k \end{bmatrix}_q$. In order to do so, Orbiter picks a basis for each subspace. By writing the elements of the basis in the rows of a matrix, a $k \times n$ matrix is obtained. In order to make the matrix unique, we assume it to be in RREF. In coding theory, such a matrix is called a generator matrix.

The Orbiter cheat sheets for $\text{PG}(n, q)$ (see Section 4.1) contain lists of all Grassmannians, provided they are not too big. It is also possible to create cheat sheets specifically for one Grassmannian. For instance, the command

Code Snippet 305

```
1 GR_3.2.2:
2   ▷ $(ORBITER) -v 2 \
3   ▷   -define F -finite_field -q 2 -end \
4   ▷   -define P -projective_space -n 2 -field F -v 0 -end \
5   ▷   -with P -do \
6   ▷   -projective_space_activity \
7   ▷   ▷ -report_Grassmannian 2 \
8   ▷   -end
9   ▷ $(PDFLATEX) Gr_3.2.2.tex
10  ▷ $(OPEN) Gr_3.2.2.pdf
11
```

produces a list of 2-dimensional subspaces of $\mathbb{F}_2^3$, i.e. the lines of $\text{PG}(2, 2)$:

$$\begin{aligned}
L_0 &= \begin{bmatrix} 100 \\ 010 \end{bmatrix} & L_3 &= \begin{bmatrix} 101 \\ 010 \end{bmatrix} & L_6 &= \begin{bmatrix} 010 \\ 001 \end{bmatrix} \\
L_1 &= \begin{bmatrix} 100 \\ 011 \end{bmatrix} & L_4 &= \begin{bmatrix} 101 \\ 011 \end{bmatrix} \\
L_2 &= \begin{bmatrix} 100 \\ 001 \end{bmatrix} & L_5 &= \begin{bmatrix} 110 \\ 001 \end{bmatrix}
\end{aligned}$$

The following command illustrates how to rank lines. In the example, we consider lines in $\text{PG}(3,3)$. The lines are given as vectors of length 8. Three lines are given in $v1$ and three lines are given in $v2$.

Code Snippet 306

```

1  PG_3_3_rank_lines:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define v1 -vector -format 3 \
4  ▷ ▷ -dense "1,0,2,2,0,1,1,2, 1,0,2,0,0,1,1,2, 1,0,2,2,0,1,2,1" \
5  ▷ ▷ -end \
6  ▷ ▷ -define v2 -vector -format 3 \
7  ▷ ▷ -dense "1,0,0,0,0,1,0,0, 1,0,0,0,0,0,0,1, 0,1,0,0,0,0,2,1" \
8  ▷ ▷ -end \
9  ▷ ▷ -define F -finite_field -q 3 -end \
10 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
11 ▷ ▷ -with P -do \
12 ▷ ▷ -projective_space_activity \
13 ▷ ▷ ▷ -rank_lines_in_PG v1 \
14 ▷ ▷ -end \
15 ▷ ▷ -with P -do \
16 ▷ ▷ -projective_space_activity \
17 ▷ ▷ ▷ -rank_lines_in_PG v2 \
18 ▷ ▷ -end
19
```

In the next example, we unrank six lines in $\text{PG}(3,5)$.

Code Snippet 307

```

1  PG_3_5_unrank_lines:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define v -vector \
4  ▷ ▷ ▷ -dense "0,36,72,108,144,805" \
5  ▷ ▷ -end \
6  ▷ ▷ -define F -finite_field -q 5 -end \
7  ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
8  ▷ ▷ -with P -do \
9  ▷ ▷ -projective_space_activity \
10 ▷ ▷ ▷ -unrank_lines_in_PG v \
11 ▷ ▷ -end
12
```

The following command produces a list of planes through a line. In the example, the line is 0. The projective space is $\text{PG}(3, 8)$

Code Snippet 308

```
1 planes_in_pencil:
2 ▷ $(ORBITER) -v 2 \
3 ▷ ▷ -define F -finite_field -q 8 -end \
4 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5 ▷ ▷ -with P -do \
6 ▷ ▷ -projective_space_activity \
7 ▷ ▷ ▷ -planes_through_line 0 \
8 ▷ ▷ -end
9
```

7.2 The Klein Quadric and the Plücker Map

Orbiter can work with the Grassmannian over a finite field. In particular, Orbiter offers indexing for subspaces. For the special case of the Grassmannian $\mathfrak{Gr}_{4,2}(V)$, Plücker coordinates can be used to identify $\mathfrak{Gr}_{4,2}(V)$ with the $Q^+(5, q)$ (Klein) quadric. Here is an example. The command

Code Snippet 309

```

1  GR_4_2_2:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5  ▷ ▷ -with P -do \
6  ▷ ▷ -projective_space_activity \
7  ▷ ▷ ▷ -report_Grassmannian 2 \
8  ▷ ▷ -end
9  ▷ $(PDFLATEX) Gr_4_2_2.tex
10 ▷ $(OPEN) Gr_4_2_2.pdf
11
```

creates the elements of $\mathfrak{Gr}_{4,2,2}$ and lists them together with their Plücker coordinates. The following list is produced (output shortened):

There are 35 lines:

$$\begin{aligned}
 L_0 &= \begin{bmatrix} 1000 \\ 0100 \end{bmatrix} = \mathbf{Pl}(1, 0, 0, 0, 0, 0) & L_2 &= \begin{bmatrix} 1000 \\ 0101 \end{bmatrix} = \mathbf{Pl}(1, 0, 0, 0, 1, 0) \\
 L_1 &= \begin{bmatrix} 1000 \\ 0110 \end{bmatrix} = \mathbf{Pl}(1, 0, 1, 0, 0, 0) & \vdots & \\
 & & L_{34} &= \begin{bmatrix} 0010 \\ 0001 \end{bmatrix} = \mathbf{Pl}(0, 1, 0, 0, 0, 0)
 \end{aligned}$$

The Plücker coordinates satisfy

$$p_{12}p_{34} - p_{13}p_{24} + p_{14}p_{23} = 0$$

and hence belong to the Klein quadric $Q^+(5, q)$. Orthogonal spaces and quadrics will be discussed in Section 7.3.

The Orbiter labeling of points of the $Q^+(5, q)$ quadric (see Section 7.3) can then be used to enumerate the lines of $\text{PG}(3, q)$ in a second, different way. In the example of $\text{PG}(3, 2)$, this yields the following list (output shortened):

$$\begin{aligned}
 0 &= \mathbf{Pl}(1, 0, 0, 0, 0, 0) = L_0 = \begin{bmatrix} 1000 \\ 0100 \end{bmatrix} & 2 &= \mathbf{Pl}(0, 0, 1, 0, 0, 0) = L_4 = \begin{bmatrix} 1000 \\ 0010 \end{bmatrix} \\
 1 &= \mathbf{Pl}(0, 1, 0, 0, 0, 0) = L_{34} = \begin{bmatrix} 0010 \\ 0001 \end{bmatrix} & \vdots & \\
 & & 34 &= \mathbf{Pl}(0, 1, 1, 1, 1, 1) = L_{26} = \begin{bmatrix} 1101 \\ 0011 \end{bmatrix}
 \end{aligned}$$

Types of Quadrics		
Type	Quadratic Form	# Points
$Q^+(n, q)$ Hyperbolic (n is odd)	$\sum_{i=0}^{\frac{n-1}{2}-1} X_{2i} X_{2i+1} = 0$	$\frac{(q^{(n+1)/2} - 1)(q^{(n-1)/2} + 1)}{q - 1}$
$Q^-(n, q)$ Elliptic (n is odd)	$p(X_{n-1}, X_n) + \sum_{i=0}^{\frac{n-1}{2}-1} X_{2i} X_{2i+1} = 0$	$\frac{(q^{(n+1)/2} + 1)(q^{(n-1)/2} - 1)}{q - 1}$
$Q(n, q)$ Parabolic (n is even)	$X_0^2 + \sum_{i=0}^{\frac{n}{2}-1} X_{2i+1} X_{2i+2} = 0$	$\frac{q^n - 1}{q - 1}$

Table 7.1: Types of Quadrics

Creating an Orthogonal Space		
Command	Arguments	Purpose
-epsilon	ϵ	Specify the type ϵ .
-n	n	Specify the projective dimension.
-q	q	Specify the order of the field.
-label_txt	L	Set the ascii-label of the space. The label is used for things like file names etc. A default label will be used if this option is not given.
-label_tex	L	Set the tex-label of the space. The label is used within latex reports. A default label will be used if this option is not given.
-without_group		Do not create the orthogonal group.
-create_extension_fields		Create extension fields of degree two and three of $\text{GF}(q)$.

Table 7.2: Creating an Orthogonal Space

7.3 Orthogonal Spaces

Orbiter can create and work with orthogonal spaces and their groups. An orthogonal space is created by a quadratic form. We assume that the form is nondegenerate. There are three types of nondegenerate quadratic forms in $\text{PG}(n, q)$. Two when n is odd (hyperbolic and elliptic) and one if n is even (parabolic). Basic information about these quadrics and their representative quadratic forms in Orbiter is given in Table 7.1. Here, $p(X, Y) = c_1 X^2 + c_2 XY + c_3 Y^2 \in \mathbb{F}_q[X, Y]$ is irreducible over $\mathbb{F}_q$.

To create an orthogonal space, the

-orthogonal_space ϵ d q -end

command can be used. Here, $d = n + 1$, q is the order of the finite field, and

$$\epsilon = \begin{cases} 1 & \text{hyperbolic type } Q^+(d-1, q), & d \text{ even} \\ 0 & \text{elliptic type } Q(d-1, q), & d \text{ odd} \\ -1 & \text{hyperbolic type } Q^-(d-1, q), & d \text{ even} \end{cases}$$

In Table 7.2, Orbiter command options for creating orthogonal spaces are shown.

For instance, the following command creates $Q(3, 2)$ together with its group $\text{PGO}^+(4, 2)$:

Code Snippet 310

```

1   Op_4.2:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 2 -end \
7   ▷ ▷ -define O -orthogonal_space 1 4 F -without_group -end \
8   ▷ ▷ -with O -do -orthogonal_space_activity \
9   ▷ ▷ ▷ -cheat_sheet.orthogonal Do \
10  ▷ ▷ -end
11  ▷ $(PDFLATEX) 0.1_4.2_report.tex
12  ▷ $(OPEN) 0.1_4.2_report.pdf
13

```

The next command creates $Q(4, 2)$ together with its group $PGO(5, 2)$. There are 15 points and 15 lines. The geometry is a configuration 15_3 which is also known as the Cremona-Richmond configuration.

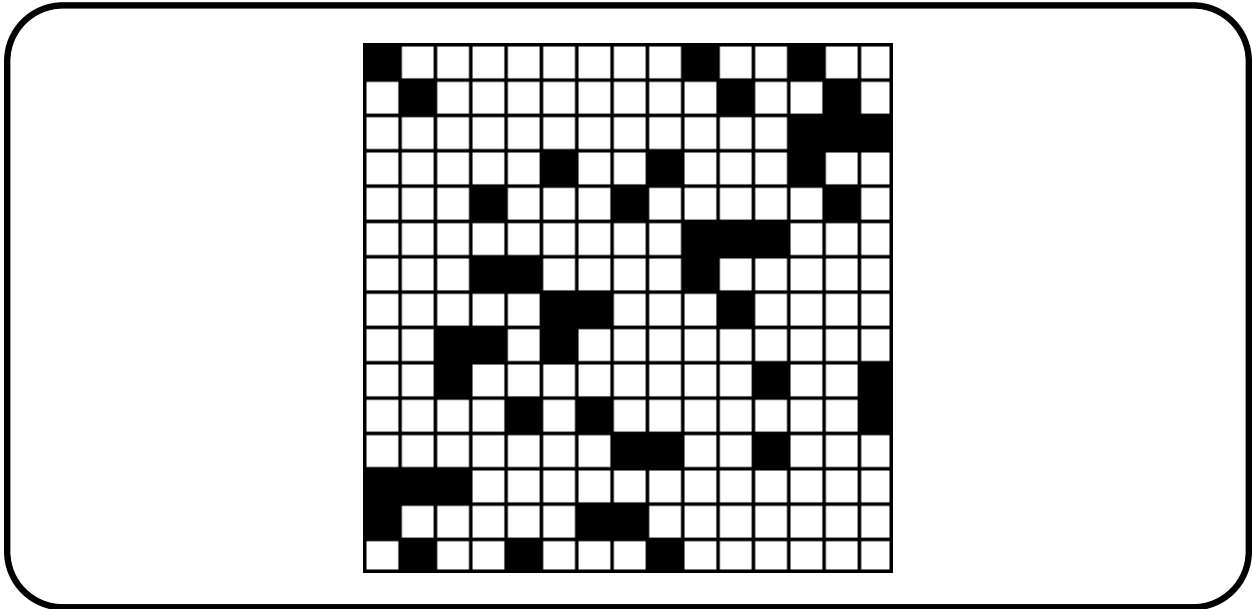
Code Snippet 311

```

1   0.5.2_incidence_matrix.csv:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 2 -end \
4   ▷ ▷ -define O -orthogonal_space 0 5 F -without_group -end \
5   ▷ ▷ -with O -do -orthogonal_space_activity \
6   ▷ ▷ ▷ -export_point_line_incidence_matrix \
7   ▷ ▷ -end
8   ▷ $(ORBITER) -v 2 \
9   ▷ ▷ -define all_one_r -vector -repeat 1 15 -end \
10  ▷ ▷ -define all_one_c -vector -repeat 1 15 -end \
11  ▷ ▷ -draw_matrix \
12  ▷ ▷ ▷ -input_csv_file 0.5.2_incidence_matrix.csv \
13  ▷ ▷ ▷ -box_width 20 -bit_depth 8 \
14  ▷ ▷ ▷ -partition 2 \
15  ▷ ▷ ▷ ▷ all_one_r all_one_c \
16  ▷ ▷ -end
17  ▷ $(OPEN) 0.5.2_incidence_matrix_draw.bmp
18

```

The command also creates a bitmap drawing of the incidence matrix between points and lines of $Q(4, 2)$.



The Orbiter indexing for points and lines of quadrics is used to address rows and columns.

By default, the orthogonal space is created together with the orthogonal group $\text{PGO}(n+1, q)$. When q is prime, the group $\text{PGO}(n+1, q)$ is created instead (the groups are isomorphic in this case, and $\text{PGO}(n+1, q)$ is a bit more efficient). For large orthogonal spaces, creating the group is expensive in terms of time and memory. The a command `-without_group` can be used to prevent the group from being created. For instance

```
-define 0 -orthogonal_space 1 6 2 -end
```

creates an object O of type $Q^+(5, 2)$. In Table 7.3, Orbiter activities for orthogonal spaces are shown.

The command

Code Snippet 312

```
1   Op_6.2:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 2 -end \
7   ▷ ▷ -define O -orthogonal_space 1 6 F -without_group -end \
8   ▷ ▷ -with O -do -orthogonal_space_activity \
9   ▷ ▷ ▷ -cheat_sheet.orthogonal Do \
10  ▷ ▷ -end
11  ▷ $(PDFLATEX) 0.1.6.2_report.tex
12  ▷ $(OPEN) 0.1.6.2_report.pdf
13
```

produces a cheat sheet for the quadric $Q^+(5, 2)$. This is the Klein quadric from Section 7.2. Orbiter produces the following output. At the top is the tactical decomposition of the incidence matrix between points and lines with respect to a hyperbolic pair. After that, the points and lines are listed (output shortened):

Orthogonal Space Activities		
Command	Arguments	Purpose
-cheat_sheet_orthogonal	options	Create a cheat sheet. Considers the given draw options object.
-print_points	v	Print the points whose ranks are given in the vector v .
-print_lines	v	Print the lines whose ranks are given in the vector v .
-unrank_line_through_two_points	$p1\ p2$	Determine the rank of the line through the points whose ranks are $p1$ and $p2$.
-lines_on_point	p	Create the ranks of all lines through the point whose rank is p .
-perp	L	Determine the common perp of a set of points. The point ranks are given in the list L .
-export_point_line_incidence_matrix		Create a csv file with the point line incidence matrix of the space.
-intersect_with_subspace	M	Find the points in the intersection of the quadric with the subspace whose generating matrix has label M .
-intersect_with_subspace_given_by_base	$base$	Find the points in the intersection of the quadric with the subspace whose base elements are given by means of their Orbiter ranks.
-table_of_blt_sets		Export a table of BLT-sets. We must be in an $Q^4(q)$ space.
-create_orthogonal_reflection	points	Create the orthogonal reflections associated with the given set of points, encoded as a vector. The points must be off the quadric. The points are given as point ranks in projective space.
-create_perp_of_points	points	For each point p in the given set of points, compute the set of points q on the quadric which are perpendicular to p . The input set of points is coded in the underlying projective geometry. The output points are coded as points on the quadric.
-create_Siegel_transformation	$u\ v$	Create the Siegel transformation defined by the vectors u and v . The Siegel transformation is defined as $\rho_{u,v}(x) := x + \beta(x,v)u - \beta(x,u)v - Q(v)\beta(x,u)u$. Here, Q is the quadratic form and β is the associated bilinear form. The vector u must be singular and v must be in $u^\perp$.
-make_all_Siegel_transformations		Make all Siegel transformations by looping over all possible pairs for u and v .
-create_orthogonal_reflection_6_and_4	points A4	Create the orthogonal reflections associated with the given set of point. Once done, turn the mappings into mappings of $PG(3, q)$, using the action A4.

Table 7.3: Orthogonal Space Activities

$\rightarrow$	9	36	18	18	6	9	9
6	3	6	0	0	0	0	0
9	0	4	4	0	0	1	0
9	0	4	0	4	0	0	1
9	1	0	2	2	2	1	1
1	0	0	0	0	0	9	0
1	0	0	0	0	0	0	9
$\downarrow$	9	36	18	18	6	9	9
6	2	1	0	0	0	0	0
9	0	1	2	0	0	1	0
9	0	1	0	2	0	0	1
9	1	0	1	1	3	1	1
1	0	0	0	0	0	1	0
1	0	0	0	0	0	0	1

The number of points is 35
points:

$$P_0 = (1, 0, 0, 0, 0, 0)$$

$$P_1 = (0, 1, 0, 0, 0, 0)$$

$$P_2 = (0, 0, 1, 0, 0, 0)$$

$$P_3 = (1, 0, 1, 0, 0, 0)$$

$$P_4 = (0, 1, 1, 0, 0, 0)$$

$$P_5 = (0, 0, 0, 1, 0, 0)$$

$$P_6 = (1, 0, 0, 1, 0, 0)$$

$$P_7 = (0, 1, 0, 1, 0, 0)$$

$$P_8 = (1, 1, 1, 1, 0, 0)$$

$$P_9 = (0, 0, 0, 0, 1, 0)$$

$$P_{10} = (1, 0, 0, 0, 1, 0)$$

$$P_{11} = (0, 1, 0, 0, 1, 0)$$

$$P_{12} = (0, 0, 1, 0, 1, 0)$$

$$P_{13} = (1, 0, 1, 0, 1, 0)$$

$$P_{14} = (0, 1, 1, 0, 1, 0)$$

$$P_{15} = (0, 0, 0, 1, 1, 0)$$

$$P_{16} = (1, 0, 0, 1, 1, 0)$$

$$P_{17} = (0, 1, 0, 1, 1, 0)$$

$$P_{18} = (1, 1, 1, 1, 1, 0)$$

$$P_{19} = (0, 0, 0, 0, 0, 1)$$

$$P_{20} = (1, 0, 0, 0, 0, 1)$$

$$P_{21} = (0, 1, 0, 0, 0, 1)$$

$$P_{22} = (0, 0, 1, 0, 0, 1)$$

$$P_{23} = (1, 0, 1, 0, 0, 1)$$

$$P_{24} = (0, 1, 1, 0, 0, 1)$$

$$P_{25} = (0, 0, 0, 1, 0, 1)$$

$$P_{26} = (1, 0, 0, 1, 0, 1)$$

$$P_{27} = (0, 1, 0, 1, 0, 1)$$

$$P_{28} = (1, 1, 1, 1, 0, 1)$$

$$P_{29} = (1, 1, 0, 0, 1, 1)$$

$$P_{30} = (1, 1, 1, 0, 1, 1)$$

$$P_{31} = (1, 1, 0, 1, 1, 1)$$

$$\begin{aligned}
P_{32} &= (0, 0, 1, 1, 1, 1) \\
P_{33} &= (1, 0, 1, 1, 1, 1) \\
P_{34} &= (0, 1, 1, 1, 1, 1) \\
\text{The number of lines is } 105 \\
L_0 &= \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 \end{bmatrix} \{P_0, P_{32}, P_{33}\} \\
L_1 &= \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 \end{bmatrix} \{P_1, P_{32}, P_{34}\} \\
L_{104} &= \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix} \{P_8, P_9, P_{18}\}
\end{aligned}$$

Orbiter has enumerators for points and lines in orthogonal spaces. For small spaces, the cheat sheet lists points and lines in the Orbiter ordering. Creating the groups can be expensive. For large spaces, it may be necessary to disable the group using the `-without_group` option. The command

Code Snippet 313

```

1   Op_64_line_rank:
2   ▷ $(ORBITER) -v 4 \
3   ▷ ▷ -define F -finite_field -q 64 -end \
4   ▷ ▷ -define O -orthogonal_space 1 6 F -without_group -end \
5   ▷ ▷ -with O -do -orthogonal_space_activity \
6   ▷ ▷ ▷ -unrank_line_through_two_points 15447347 15225451 \
7   ▷ ▷ -end
8

```

computes the Orbiter rank of the line through the points with rank 15447347 and 15225451, respectively. The rank of the line is 16767254. These ranks refer to the orthogonal geometry. They are different from the ranks of points and lines in projective spaces.

It is possible to create reports for orthogonal spaces without group. In this case, the group information will be skipped. For instance, the following command creates a report for $Q(5, 64)$:

Code Snippet 314

```

1   Op_64_report:
2   ▷ $(ORBITER) -v 4 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 64 -end \
7   ▷ ▷ -define O -orthogonal_space 1 6 F -without_group -end \
8   ▷ ▷ -with O -do -orthogonal_space_activity \
9   ▷ ▷ ▷ -cheat_sheet.orthogonal Do \
10  ▷ ▷ -end
11  ▷ $(PDFLATEX) 0_1_6_64_report.tex
12  ▷ $(OPEN) 0_1_6_64_report.pdf
13

```

The report does not show information about the group. However, it still contains the tactical decomposition with respect to a hyperbolic pair. The printing of points is restricted to small spaces only.

The group is not available.

The quadratic form is:

$$X_0X_1 + X_2X_3 + X_4X_5 = 0$$

→	16769025	1090252800	532350	532350	130	4225	4225
16511040	65	4160	0	0	0	0	0
266175	0	4096	128	0	0	1	0
266175	0	4096	0	128	0	0	1
4225	3969	0	126	126	2	1	1
1	0	0	0	0	0	4225	0
1	0	0	0	0	0	0	4225
↓	16769025	1090252800	532350	532350	130	4225	4225
16511040	64	63	0	0	0	0	0
266175	0	1	64	0	0	63	0
266175	0	1	0	64	0	0	63
4225	1	0	1	1	65	1	1
1	0	0	0	0	0	1	0
1	0	0	0	0	0	0	1

The number of points is 17047617

Too many points to print.

The number of lines is 1108095105

To study BLT-sets in $Q(4, q)$, see Section 15.4.

According to Table 7.1, Orbiter uses the equation

$$X_0X_1 + X_2X_3 + X_4X_5 = 0$$

to define the Klein quadric. An elliptic quadric is an ovoid of the Klein quadric that is obtained by intersecting the quadric with a suitable solid. In $PG(5, 5)$, the subspace generated by the rows of the matrix

$$\begin{bmatrix} 1 & 3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 \end{bmatrix}$$

is such a subspace. The ordering of columns corresponds to the natural ordering of the variables as $X_0, X_1, X_2, X_3, X_4, X_5$. The following command produces a list of points of an elliptic quadric ovoid in $Q^+(5, 5)$.

Code Snippet 315

```
1    elliptic_quadric_subspace:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 5 -end \
4    ▷ ▷ -define v -vector -format 4 \
5    ▷ ▷ ▷ -dense "1,3,0,0,0,0, 0,0,1,0,0,0, 0,0,0,1,0,0, 0,0,0,0,1,1" \
```

```

6      ▷ ▷ -end \
7      ▷ ▷ -define O -orthogonal_space 1 6 F -end \
8      ▷ ▷ -with O -do -orthogonal_space_activity \
9      ▷ ▷ ▷ -intersect_with_subspace v \
10     ▷ ▷ -end
11

```

The elliptic quadric has 26 points.

The coding of points and line in orthogonal spaces is different from the coding of points in projective spaces. We will create and print a set called BLT set (after [6]). This is a set of $q + 1$ points on the $Q(4, q)$ quadric satisfying a special geometric property. According to Table 7.1, Orbiter uses the equation

$$X_0^2 + X_1X_2 + X_3X_4 = 0$$

to define the $Q(4, q)$ quadric. The following example creates the BLT-set with Orbiter catalogue number #1 in $Q(4, 7)$:

Code Snippet 316

```

1      BLT_database_7.1:
2      ▷ $(ORBITER) -v 2 \
3      ▷ ▷ -define F -finite_field -q 7 -end \
4      ▷ ▷ -define P -projective_space -n 4 -field F -v 0 -end \
5      ▷ ▷ -define S -geometric_object P \
6      ▷ ▷ ▷ -BLT_database 1 \
7      ▷ ▷ -end \
8      ▷ ▷ -with S -do -combinatorial_object_activity -save \
9      ▷ ▷ -end
10

```

The set is stored in a file. The next command reads the file and prints the elements of the set:

Code Snippet 317

```

1      BLT_database_7.1_print:
2      ▷ $(ORBITER) -v 2 \
3      ▷ ▷ -define F -finite_field -q 7 -end \
4      ▷ ▷ -define O -orthogonal_space 0 5 F -without_group -end \
5      ▷ ▷ -define S -set -file_orbiter_format BLT_7.1.txt -end \
6      ▷ ▷ -with O -do -orthogonal_space_activity \
7      ▷ ▷ ▷ -print_points S -end
8      ▷ $(PDFLATEX) S_set_report.tex
9      ▷ $(OPEN) S_set_report.pdf
10

```

The command produces the following list of points, comprising the second BLT-set over $\mathbb{F}_7$.

A set of points of size 8

The Points:

0 : $P_0 = (0, 1, 0, 0, 0)$

1 : $P_1 = (0, 0, 1, 0, 0)$

2 : $P_{40} = (0, 1, 2, 6, 2)$


```

3 :  $P_{41} = (0, 1, 4, 3, 1)$ 
4 :  $P_{225} = (1, 6, 6, 5, 1)$ 
5 :  $P_{270} = (1, 5, 5, 4, 4)$ 
6 :  $P_{241} = (1, 2, 2, 2, 1)$ 
7 :  $P_{340} = (1, 1, 1, 4, 3)$ 

```

More on BLT-sets can be found in Section 15.4.

The next command prints points and lines of the $W(2)$, also known as the Doily. It is an example of a generalized quadrangle.

Code Snippet 318

```

1   Doily_W2:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 2 -end \
4   ▷ ▷ -define O -orthogonal_space 0 5 F -without_group -end \
5   ▷ ▷ -define W2_points -set -loop 0 15 1 -end \
6   ▷ ▷ -define W2_lines -set -loop 0 15 1 -end \
7   ▷ ▷ -with O -do \
8   ▷ ▷ -orthogonal_space_activity \
9   ▷ ▷ ▷ -print_points W2_points \
10  ▷ ▷ -end \
11  ▷ ▷ -with O -do \
12  ▷ ▷ -orthogonal_space_activity \
13  ▷ ▷ ▷ -print_lines W2_lines \
14  ▷ ▷ -end
15  ▷ $(PDFLATEX) W2_points_set_report.tex
16  ▷ $(OPEN) W2_points_set_report.pdf
17  ▷ $(PDFLATEX) W2_lines_set_of_lines_report.tex
18  ▷ $(OPEN) W2_lines_set_of_lines_report.pdf
19

```

Let us create orthogonal reflections. To this end, we need a list of points off the quadric. Here are the points off the $Q^+(5, 2)$ quadric. There are 28 of them:

Code Snippet 319

```

1   ORTHOGONAL_OP_6_2_POINTS_OFF="6,7,10,13,14,15,16,20,24,28,29,30,31,35,39,43, 44,
    45, 46, 48, 49, 50, 52, 53, 54, 56, 57, 58"
2

```

Here is a command to create the associated orthogonal reflections:

Code Snippet 320

```

1   Op_6_2_orthogonal_reflections:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Pts -vector \
4   ▷ ▷ ▷ -dense $(ORTHOGONAL_OP_6_2_POINTS_OFF) \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 2 -end \

```

```

7   ▷ ▷ -define O -orthogonal_space 1 6 F -end \
8   ▷ ▷ -with O -do -orthogonal_space_activity \
9   ▷ ▷ ▷ -create_orthogonal_reflection Pts \
10  ▷ ▷ -end \
11  ▷ ▷ -define G -linear_group -PGOp 6 F -end \
12  ▷ ▷ -define Op_6_2_orthogonal_reflections -vector_ge \
13  ▷ ▷ ▷ -action G \
14  ▷ ▷ ▷ -read_csv 0.1_6_2_orthogonal_reflections.csv Element \
15  ▷ ▷ -end \
16  ▷ ▷ -with G -do \
17  ▷ ▷ -group_theoretic_activity \
18  ▷ ▷ ▷ -element_processing \
19  ▷ ▷ ▷ ▷ -input Op_6_2_orthogonal_reflections \
20  ▷ ▷ ▷ ▷ -print \
21  ▷ ▷ ▷ -end \
22  ▷ ▷ -end
23  ▷ $(PDFLATEX) Op_6_2_orthogonal_reflections_elements.tex
24  ▷ $(OPEN) Op_6_2_orthogonal_reflections_elements.pdf
25

```

The following output is produced:

Element 0 / 28 of order 2:

$$\begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 \end{bmatrix}$$

0,1,1,1,1,1,0,1,1,1,1,1,0,1,1,1,1,1,0,1,1,1,1,1,0

Element 1 / 28 of order 2:

$$\begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

0,1,0,0,0,0,1,0,0,0,0,0,0,0,1,0,0,0,0,0,1,0,0,0,0,0,1

Element 2 / 28 of order 2:

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

0,1,1,0,0,0,1,0,1,0,0,0,0,1,0,0,0,1,1,1,0,0,0,0,0,1,0,0,0,0,0,1
 Element 3 / 28 of order 2:

$$\begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

0,1,0,1,0,0,1,0,0,1,0,0,1,1,1,0,0,0,0,0,1,0,0,0,0,0,1,0,0,0,0,0,1
 Element 4 / 28 of order 2:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

1,0,0,0,0,0,1,0,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,1
 Element 5 / 28 of order 2:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

1,0,0,0,0,0,1,1,1,0,0,1,0,0,1,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1
 Element 6 / 28 of order 2:

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

1,1,1,1,0,0,0,1,0,0,0,0,0,1,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1
 Element 7 / 28 of order 2:

$$\begin{bmatrix} 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 \end{bmatrix}$$

0,1,0,0,1,0,1,0,0,0,1,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,1,0,1,1,0,0,1,1

Element 8 / 28 of order 2:

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 & 1 \end{bmatrix}$$

0,1,1,0,1,0,1,0,1,0,0,0,1,0,0,0,1,1,1,1,0,0,0,0,1,0,1,1,0,1,1

Element 9 / 28 of order 2:

$$\begin{bmatrix} 0 & 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}$$

0,1,0,1,1,0,1,0,0,1,1,0,1,1,1,1,0,0,0,0,1,0,0,0,0,0,1,0,1,1,0,1,1,1

Element 10 / 28 of order 2:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

1,0,0,0,0,0,1,0,0,0,0,0,0,0,1,1,0,0,0,1,0,1,0,0,0,0,1,0,0,0,1,1,1,1

Element 11 / 28 of order 2:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

1,0,0,0,0,0,1,1,1,1,0,1,0,0,1,1,0,1,0,1,0,0,0,0,1,0,1,0,1,1,1,1

Element 12 / 28 of order 2:

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

1,1,1,1,1,0,0,1,0,0,0,0,0,1,0,1,1,0,0,1,1,0,1,0,0,0,0,1,0,0,1,1,1,1

Element 13 / 28 of order 2:

$$\begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

0,1,0,0,0,1,1,0,0,0,1,0,0,1,0,0,0,0,0,1,0,0,1,1,0,0,1,1,0,0,0,0,1

Element 14 / 28 of order 2:

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

0,1,1,0,0,1,1,0,1,0,0,1,0,0,1,0,0,0,1,1,1,0,1,1,1,0,1,1,0,0,0,0,1

Element 15 / 28 of order 2:

$$\begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

0,1,0,1,0,1,1,0,0,1,0,1,1,1,1,0,1,0,0,0,1,0,0,1,1,0,1,1,1,0,0,0,0,1

Element 16 / 28 of order 2:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

1,0,0,0,0,0,1,0,0,0,0,0,0,0,1,0,1,0,0,1,0,0,1,0,0,1,1,1,1,0,0,0,0,1

Element 17 / 28 of order 2:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

1,0,0,0,0,0,1,1,1,0,1,1,0,0,1,0,1,1,0,0,1,1,0,1,1,1,1,0,0,0,0,1

Element 18 / 28 of order 2:

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

1,1,1,1,0,1,0,1,0,0,0,0,1,0,1,0,1,1,0,0,1,0,1,1,1,1,0,0,0,0,1

Element 19 / 28 of order 2:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

1,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,0

Element 20 / 28 of order 2:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

1,0,0,0,0,0,1,1,0,0,1,1,0,0,1,0,0,0,0,0,1,0,0,1,0,0,0,1,1,0,0,0,1,0

Element 21 / 28 of order 2:

$$\begin{bmatrix} 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 \end{bmatrix}$$

1,1,0,0,1,1,0,1,0,0,0,0,0,1,0,0,0,0,0,1,0,0,0,1,0,1,0,0,1,0

Element 22 / 28 of order 2:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}$$

1,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,1,1,1,0,0,1,0,0,1,0,0,1,0

Element 23 / 28 of order 2:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}$$

1,0,0,0,0,1,1,1,0,1,1,0,0,1,0,0,1,1,1,1,0,1,0,0,1,1,0,1,0,1,0

Element 24 / 28 of order 2:

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \end{bmatrix}$$

1,1,1,0,1,1,0,1,0,0,0,0,0,1,0,0,0,0,1,1,1,1,0,1,1,0,0,1,0,1,1,0,1,0

Element 25 / 28 of order 2:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

1,0,0,0,0,0,1,0,0,0,0,0,0,1,1,1,0,0,0,1,0,0,0,0,0,1,0,1,0,0,0,1,1,0

Element 26 / 28 of order 2:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

1,0,0,0,0,1,1,0,1,1,1,0,1,1,1,0,0,0,1,0,0,1,0,0,1,0,1,1,0,0,1,1,0

Element 27 / 28 of order 2:

$$\begin{bmatrix} 1 & 1 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 \end{bmatrix}$$

1,1,0,1,1,1,0,1,0,0,0,0,0,1,1,1,1,0,0,0,1,0,0,0,1,0,1,0,1,0,1,1,0

Code Snippet 321

```

1   Op_6.2_create_Siegel_transformation_2B:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 2 -end \
4   ▷ ▷ -define G -linear_group -PGOp 6 F -end \
5   ▷ ▷ -define O -orthogonal_space 1 6 F -end \
6   ▷ ▷ -with O -do -orthogonal_space_activity \
7   ▷ ▷ ▷ -create_Siegel_transformation "1,0,0,0,0,0" "0,0,1,0,0,0" \
8   ▷ ▷ -end \
9   ▷ ▷ -define G -linear_group -PGOp 6 F -end \
10  ▷ ▷ -define siegel1 -vector_ge \
11  ▷ ▷ ▷ -action G \
12  ▷ ▷ ▷ -read_csv 0.1_6.2_Siegel_transformation.csv Element \
13  ▷ ▷ -end \
14  ▷ ▷ -with G -do \
15  ▷ ▷ -group_theoretic_activity \
16  ▷ ▷ -element_processing \
17  ▷ ▷ ▷ -input siegel1 \
18  ▷ ▷ ▷ -print \
19  ▷ ▷ -end \
20  ▷ ▷ -end
21  ▷ $(PDFLATEX) siegel1_elements.tex
22  ▷ $(OPEN) siegel1_elements.pdf
23

```

creates the Siegel transformation

$$\rho_{\mathbf{u},\mathbf{v}}, \mathbf{u} = (1, 0, 0, 0, 0, 0), \mathbf{v} = (0, 0, 1, 0, 0, 0).$$

The output is

Action $\text{PGO}^+(6, 2)$:

Element 0 / 1 of order 2:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

1,0,0,0,0,0,1,1,0,0,0,0,1,0,0,0,1,0,0,1,0,0,0,0,0,1,0,0,0,0,0,1

The command

Code Snippet 322

```

1   Op_6.2_create_Siegel_transformation_2C:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 2 -end \
4   ▷ ▷ -define O -orthogonal_space 1 6 F -end \
5   ▷ ▷ -with O -do -orthogonal_space_activity \
6   ▷ ▷ ▷ -create_Siegel_transformation "1,0,0,0,0,0" "0,0,1,1,0,0" \
7   ▷ ▷ -end \
8   ▷ ▷ -define G -linear_group -PGOp 6 F -end \
9   ▷ ▷ -define siegel2 -vector_ge \
10  ▷ ▷ ▷ -action G \
11  ▷ ▷ ▷ -read_csv 0_1_6.2_Siegel_transformation.csv Element \
12  ▷ ▷ -end \
13  ▷ ▷ -with G -do \
14  ▷ ▷ -group_theoretic_activity \
15  ▷ ▷ ▷ -element_processing \
16  ▷ ▷ ▷ ▷ -input siegel2 \
17  ▷ ▷ ▷ ▷ -print \
18  ▷ ▷ ▷ -end \
19  ▷ ▷ -end
20  ▷ $(PDFLATEX) siegel2_elements.tex
21  ▷ $(OPEN) siegel2_elements.pdf
22

```

creates the Siegel transformation

$$\rho_{\mathbf{u},\mathbf{v}}, \mathbf{u} = (1, 0, 0, 0, 0, 0), \mathbf{v} = (0, 0, 1, 1, 0, 0).$$

The output is

Element 0 / 1 of order 2:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

1,0,0,0,0,0,1,1,1,1,0,0,1,0,1,0,0,0,1,0,0,1,0,0,0,0,0,1,0,0,0,0,0,1

The next command expands the conjugacy class of the first Siegel transformation:

Code Snippet 323

```

1   Op_6.2_Siegel_transformation_expand_class_2A:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Pts -vector \
4   ▷ ▷ ▷ -dense $(ORTHOGONAL_OP_6.2_POINTS_OFF) \

```

```

5    ▷ ▷ -end \
6    ▷ ▷ -define F -finite_field -q 2 -end \
7    ▷ ▷ -define G -linear_group -PGOp 6 F -end \
8    ▷ ▷ -define Siegel -vector_ge \
9    ▷ ▷ ▷ -action G \
10   ▷ ▷ ▷ -read_csv 0.1.6.2.Siegel.transformation.csv Element \
11   ▷ ▷ -end \
12   ▷ ▷ -with G -do \
13   ▷ ▷ -group_theoretic_activity \
14   ▷ ▷ ▷ -conjugacy_class_of "Siegel" \
15   ▷ ▷ ▷ $(SIEGEL_TRANSFORM_UV_2A) \
16   ▷ ▷ -end
17   ▷ $(PDFLATEX) PGOp_6.2.Siegel.elements.tex
18   ▷ $(OPEN) PGOp_6.2.Siegel.elements.pdf
19

```

The class has size 105.

The next command

Code Snippet 324

```

1    Op_6.2.Siegel.transformation.expand_class_2C:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define Pts -vector \
4    ▷ ▷ ▷ -dense $(ORTHOGONAL_OP_6.2.POINTS.OFF) \
5    ▷ ▷ -end \
6    ▷ ▷ -define F -finite_field -q 2 -end \
7    ▷ ▷ -define G -linear_group -PGOp 6 F -end \
8    ▷ ▷ -define Siegel -vector_ge \
9    ▷ ▷ ▷ -action G \
10   ▷ ▷ ▷ -read_csv 0.1.6.2.Siegel.transformation.csv Element \
11   ▷ ▷ -end \
12   ▷ ▷ -with G -do \
13   ▷ ▷ -group_theoretic_activity \
14   ▷ ▷ ▷ -conjugacy_class_of "Siegel" \
15   ▷ ▷ ▷ $(SIEGEL_TRANSFORM_UV_2C) \
16   ▷ ▷ -end
17   ▷ #$(PDFLATEX) PGOp_6.2.Siegel.elements.tex
18   ▷ #$(OPEN) PGOp_6.2.Siegel.elements.pdf
19

```

expands the conjugacy class of the second Siegel transformation. The class has size 210. Because the class sizes are different, we know that the two Siegel transformations are not conjugate.

Next, let us create *all* Siegel transformations:

Code Snippet 325

```

1    Op_6.2.make_all_Siegel_transformations:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define Pts -vector \
4    ▷ ▷ ▷ -dense $(ORTHOGONAL_OP_6.2.POINTS.OFF) \
5    ▷ ▷ -end \

```

```
6      ▷ ▷ -define F -finite_field -q 2 -end \  
7      ▷ ▷ -define O -orthogonal_space 1 6 F -end \  
8      ▷ ▷ -with O -do -orthogonal_space_activity \  
9      ▷ ▷ ▷ -make_all_Siegel_transformations \  
10     ▷ ▷ -end  
11
```

In total, we get 1120 Siegel transformations. But they are not all distinct, and some of them are the identity matrix. The same Siegel transformation may arise from different pairs of the parameter $(\mathbf{u}, \mathbf{v})$. The next command produces a tally of the distinct Siegel transformations:

Code Snippet 326

```
1      Op_6.2_all_Siegel_transformations.tally:  
2      ▷ $(ORBITER) -v 2 \  
3      ▷ ▷ -tally_column all_Siegel.csv transform  
4
```

7.4 Hermitian Varieties

Orbiter has enumerators for points of the hermitian variety $H(k, Q)$. Here, Q is a square, and so $q = \sqrt{Q}$ is an integer. The equation of the variety is

$$\sum_{i=0}^k X_i^{q+1} = 0.$$

The command

Code Snippet 327

```
1 H_2.4:
2 ▷ $(ORBITER) -v 2 \
3 ▷ ▷ -define F -finite_field -q 4 -end \
4 ▷ ▷ -with F -do -finite_field_activity \
5 ▷ ▷ ▷ -cheat_sheet_hermitian 2 \
6 ▷ ▷ -end
7 ▷ $(PDFLATEX) H_2.4.tex
8 ▷ $(OPEN) H_2.4.pdf
9
```

produces a cheat sheet for the variety $H(2, 4)$:

The Hermitian variety $H(2, 4)$ contains 9 points:

$$P_0 = (1, 1, 0) = 4$$

$$P_1 = (2, 1, 0) = 5$$

$$P_2 = (3, 1, 0) = 6$$

$$P_3 = (1, 0, 1) = 7$$

$$P_4 = (2, 0, 1) = 8$$

$$P_5 = (3, 0, 1) = 9$$

$$P_6 = (0, 1, 1) = 10$$

$$P_7 = (0, 2, 1) = 13$$

$$P_8 = (0, 3, 1) = 17$$

All points: (4, 5, 6, 7, 8, 9, 10, 13, 17)

The command

Code Snippet 328

```
1 H_3.4:
2 ▷ $(ORBITER) -v 2 \
3 ▷ ▷ -define F -finite_field -q 4 -end \
4 ▷ ▷ -with F -do -finite_field_activity \
5 ▷ ▷ ▷ -cheat_sheet_hermitian 3 \
6 ▷ ▷ -end
7 ▷ $(PDFLATEX) H_3.4.tex
8 ▷ $(OPEN) H_3.4.pdf
9
```

produces a cheat sheet for the variety $H(3, 4)$.

The Hermitian variety $H(3, 4)$ contains 45 points:

$P_0 = (1, 1, 0, 0) = 5$	$P_{23} = (3, 3, 1, 1) = 52$
$P_1 = (2, 1, 0, 0) = 6$	$P_{24} = (0, 0, 1, 1) = 38$
$P_2 = (3, 1, 0, 0) = 7$	$P_{25} = (1, 1, 2, 1) = 58$
$P_3 = (1, 0, 1, 0) = 8$	$P_{26} = (2, 1, 2, 1) = 59$
$P_4 = (2, 0, 1, 0) = 9$	$P_{27} = (3, 1, 2, 1) = 60$
$P_5 = (3, 0, 1, 0) = 10$	$P_{28} = (1, 2, 2, 1) = 62$
$P_6 = (0, 1, 1, 0) = 11$	$P_{29} = (2, 2, 2, 1) = 63$
$P_7 = (0, 2, 1, 0) = 15$	$P_{30} = (3, 2, 2, 1) = 64$
$P_8 = (0, 3, 1, 0) = 19$	$P_{31} = (1, 3, 2, 1) = 66$
$P_9 = (1, 0, 0, 1) = 23$	$P_{32} = (2, 3, 2, 1) = 67$
$P_{10} = (2, 0, 0, 1) = 24$	$P_{33} = (3, 3, 2, 1) = 68$
$P_{11} = (3, 0, 0, 1) = 25$	$P_{34} = (0, 0, 2, 1) = 53$
$P_{12} = (0, 1, 0, 1) = 26$	$P_{35} = (1, 1, 3, 1) = 74$
$P_{13} = (0, 2, 0, 1) = 30$	$P_{36} = (2, 1, 3, 1) = 75$
$P_{14} = (0, 3, 0, 1) = 34$	$P_{37} = (3, 1, 3, 1) = 76$
$P_{15} = (1, 1, 1, 1) = 4$	$P_{38} = (1, 2, 3, 1) = 78$
$P_{16} = (2, 1, 1, 1) = 43$	$P_{39} = (2, 2, 3, 1) = 79$
$P_{17} = (3, 1, 1, 1) = 44$	$P_{40} = (3, 2, 3, 1) = 80$
$P_{18} = (1, 2, 1, 1) = 46$	$P_{41} = (1, 3, 3, 1) = 82$
$P_{19} = (2, 2, 1, 1) = 47$	$P_{42} = (2, 3, 3, 1) = 83$
$P_{20} = (3, 2, 1, 1) = 48$	$P_{43} = (3, 3, 3, 1) = 84$
$P_{21} = (1, 3, 1, 1) = 50$	$P_{44} = (0, 0, 3, 1) = 69$
$P_{22} = (2, 3, 1, 1) = 51$	

All points: (5, 6, 7, 8, 9, 10, 11, 15, 19, 23, 24, 25, 26, 30, 34, 4, 43, 44, 46, 47, 48, 50, 51, 52, 38, 58, 59, 60, 62, 63, 64, 66, 67, 68, 53, 74, 75, 76, 78, 79, 80, 82, 83, 84, 69)

Coincidentally, this Hermitian variety is the Hirschfeld cubic surface over $\mathbb{F}_4$.

Creating Varieties		
Command	Arguments	Purpose
-label_txt	label	Specify an ASCII label for the variety.
-label_tex	label	Specify a tex label for the variety.
-projective_space	P	Specify the underlying projective space.
-ring	R	Specify the underlying polynomial ring.
-equation_in_algebraic_form	eqn	The defining equation in algebraic form.
-set_parameters	label lable_tex parame- ter_values	Set parameter values for the equation given in algebraic form.
-equation_by_coefficients	eqn	The defining equation is given as a list of coefficients in the chosen ordering of monomials.
-equation_by_rank	rank	The defining equation is given as orbiter rank of a projective point.
-points	pts	The set of $\mathbb{F}_q$ -rational points (optional).
-bitangents	lines	A set of lines (optional).
-compute_lines		Compute the lines.
-transform	element	Apply the transformation by the given group element.
-transform_inverse	element	Apply the inverse transformation by the given group element.

Table 7.4: Creating Varieties

7.5 Algebraic Varieties

A set of points V in $\text{PG}(n, q)$ is algebraic if there is a set of homogeneous polynomials $p_1, \dots, p_r$ whose roots over $\mathbb{F}_q$ are the given set. In this case, we write $V = \mathbf{v}(p_1, \dots, p_r)$. The set V is also called the algebraic variety of $p_1, \dots, p_r$.

Conversely, given a set of points V in $\text{PG}(n, q)$, the ideal $I(V)$ is the set of homogeneous polynomials in $\mathbb{F}_q[X_0, \dots, X_n]$ which vanish on all of V . This set is an ideal in the polynomial ring. In $\text{PG}(n, q)$, every set is algebraic of degree at most $(n+1)(q-1)$ [33]. The associated polynomial is unique and known as the algebraic normal form of the set.

In order to work with algebraic sets, multivariate polynomial rings are required.

Table 7.4 lists the commands for creating an algebraic variety.

A projective variety requires a projective space and an associated ring of coordinate functions. The number of variables is one more than the projective dimension of the space. The variety can be given in two ways. It can be given by an equation in algebraic form or by a list of coefficients. In the second case, the coefficients must be listed in the chosen monomial ordering of the ring. Once a variety is created, activities for varieties can be invoked.

Suppose we want to create an algebraic variety. To this end, we need to first create a finite field and a suitable polynomial ring. Because we create projective variety, we need to specify the graded component of the ring, i.e., the degree of the homogeneous polynomial. After that, we specify the (homogeneous) algebraic equation which defines the variety. The degree of the equation must match the degree selected in the polynomial ring.

It is assumed that one side of the equation is zero, and only the non-zero side of the equation is given. The multiplication signs must be written and cannot be omitted. Here is an example:

We consider the conic

$$X_0^2 - X_1X_2$$

over the field $\mathbb{F}_{13}$. We write the equation as

$$X0^2-X1*X2$$

and invoke the following command:

Code Snippet 329

```

1   conic_variety_q13:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define F -finite_field -q 13 -end \
4   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5   ▷ ▷ -define R -polynomial_ring \
6   ▷ ▷ ▷ -field F \
7   ▷ ▷ ▷ -number_of_variables 3 \
8   ▷ ▷ ▷ -homogeneous_of_degree 2 \
9   ▷ ▷ ▷ -monomial_ordering_partition \
10  ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
11  ▷ ▷ -end \
12  ▷ ▷ -define V -variety \
13  ▷ ▷ ▷ -projective_space P \
14  ▷ ▷ ▷ -ring R \
15  ▷ ▷ ▷ -equation_in_algebraic_form \
16  ▷ ▷ ▷ ▷ "X0^2-X1*X2" \
17  ▷ ▷ ▷ -label_txt conic \
18  ▷ ▷ ▷ -label_tex "{\\rm conic}" \
19  ▷ ▷ -end
20
```

The command creates the field $\mathbb{F}_{13}$, the projective plane $\text{PG}(2, 13)$, the degree-2 component of the polynomial ring

$$\mathbb{F}_{13}[X_0, X_1, X_2],$$

and then finally the conic $X_0^2 - X_1X_2$. The variety is given the name `conic`. The name is useful should any output files be created (not in this example). Using the ordering of monomials in the polynomial ring, the equation is translated into a coefficient vector as follows:

$$(1, 0, 0, 0, 0, 12).$$

The conic has 14 points. Using Orbiter codes for projective points, the numbers are

$$1, 2, 3, 39, 57, 62, 68, 77, 134, 141, 150, 151, 175, 178.$$

In the next example, we consider the Klein quadric with equation

$$X_0X_1 + X_2X_3 + X_4X_5 = 0.$$

We encode the equation as a makefile variable:

Code Snippet 330

```

1   Op_6_eqn="X0*X1+X2*X3+X4*X5"
2

```

The following command creates the associated algebraic variety:

Code Snippet 331

```

1   Op_6_2_variety:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define F -finite_field -q 2 -end \
4   ▷ ▷ -define P -projective_space -n 5 -field F -v 0 -end \
5   ▷ ▷ -define R -polynomial_ring \
6   ▷ ▷ ▷ -field F \
7   ▷ ▷ ▷ -number_of_variables 6 \
8   ▷ ▷ ▷ -homogeneous_of_degree 2 \
9   ▷ ▷ ▷ -monomial_ordering_partition \
10  ▷ ▷ ▷ -variables "X0,X1,X2,X3,X4,X5" "X_0,X_1,X_2,X_3,X_4,X_5" \
11  ▷ ▷ -end \
12  ▷ ▷ -define V -variety \
13  ▷ ▷ ▷ -projective_space P \
14  ▷ ▷ ▷ -ring R \
15  ▷ ▷ ▷ -equation_in_algebraic_form \
16  ▷ ▷ ▷ ▷ $(Op_6_eqn) \
17  ▷ ▷ ▷ -label_txt Op_6_2 \
18  ▷ ▷ ▷ -label_tex "{\\rm Op\\_6\\_2}" \
19  ▷ ▷ -end
20

```

We consider the quartic curve in $\text{PG}(2, 13)$ given by the equation

$$X_0^4 + 4X_1^4 + 9X_2^4 + X_0^2X_1^2 + 4X_0^2X_2^2 + 3X_1^2X_2^2 = 0.$$

Code Snippet 332

```

1   CURVE_Q13_F0_CLEAN2_EQUATION="X0^4 + 4*X1^4 + 9*X2^4 + X0^2*X1^2 + 4*X0^2*X2^2 +
2   3*X1^2*X2^2"

```

The following command creates the algebraic variety. Since we are interested in a curve of degree 4, we create the degree-4 component of the polynomial ring. Finally, we create the variety V over F , utilizing the projective space P and the ring R and the makefile variable containing the equation. For possible use later, we define a label of the variety, one for ASCII text and one for latex output.

Code Snippet 333

```

1   curve.q13.f0:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define F -finite_field -q 13 -end \
4   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5   ▷ ▷ -define R -polynomial_ring \
6   ▷ ▷ ▷ -field F \

```



```

7   ▷ ▷ ▷ -number_of_variables 3 \
8   ▷ ▷ ▷ -homogeneous_of_degree 4 \
9   ▷ ▷ ▷ -monomial_ordering_partition \
10  ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
11  ▷ ▷ ▷ -end \
12  ▷ ▷ ▷ -define V -variety \
13  ▷ ▷ ▷ -projective_space P \
14  ▷ ▷ ▷ -ring R \
15  ▷ ▷ ▷ -equation_in_algebraic_form \
16  ▷ ▷ ▷ ▷ $(CURVE_Q13_F0_CLEAN2_EQUATION) \
17  ▷ ▷ ▷ -label_txt curve_q13_f0_clean2 \
18  ▷ ▷ ▷ -label_tex curve\_q13\_f0\_clean2 \
19  ▷ ▷ ▷ -end
20

```

In a similar vein, let us create the Hirschfeld surface over $\mathbb{F}_2$ in projective three-space, with equation

$$X^2W + XW^2 + Y^2Z + YZ^2 = 0.$$

We encode the equation in a makefile variable:

Code Snippet 334

```

1   HIRSCHFELD_EQUATION_XYZW="X^2*W+X*W^2+Y^2*Z+Y*Z^2"
2

```

The following command creates the variety:

Code Snippet 335

```

1   Hirschfeld.q2:
2   ▷   $(ORBITER) -v 6 \
3   ▷   ▷ -define F -finite_field -q 2 -end \
4   ▷   ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5   ▷   ▷ -define R -polynomial_ring \
6   ▷   ▷ ▷ -field F \
7   ▷   ▷ ▷ -number_of_variables 4 \
8   ▷   ▷ ▷ -homogeneous_of_degree 3 \
9   ▷   ▷ ▷ -monomial_ordering_partition \
10  ▷   ▷ ▷ -variables "X,Y,Z,W" "X,Y,Z,W" \
11  ▷   ▷ ▷ -end \
12  ▷   ▷ -define V -variety \
13  ▷   ▷ ▷ -label_txt Hirschfeld.q2 \
14  ▷   ▷ ▷ -label_tex Hirschfeld\_q2 \
15  ▷   ▷ ▷ -projective_space P \
16  ▷   ▷ ▷ -ring R \
17  ▷   ▷ ▷ -equation_in_algebraic_form \
18  ▷   ▷ ▷ ▷ $(HIRSCHFELD_EQUATION_XYZW) \
19  ▷   ▷ ▷ -end
20

```

It is possible to transform a projective variety under a linear transformation. There are two different

strategies. The first is by using a variety object. The second is by applying the transformation directly to the equation, without going through a variety object.

Here are two examples which illustrate both strategies. In the first example, we apply two transformations while we create the variety. At first, we create makefile variables for the equation in algebraic form, for the precomputed set of points and bitangents, and for the two transformations. The transformations are given as matrices.

Code Snippet 336

```
1 CURVE_Q13_FLAG_1_EQN="10*x1^4 + 12*x2^4 + 9*x3^4 + 8*x1^3*x2 + 2*x1*x2^3 + 5*x2^3
  *x3 + 3*x2*x3^3 + 4*x1^2*x2^2 + 3*x1^2*x3^2 + x2^2*x3^2 + 12*x1^2*x2*x3 + 10*x1*x
  2^2*x3 + 9*x1*x2*x3^2"
2
```

Code Snippet 337

```
1 CURVE_Q13_FLAG_1_POINTS="68,71,75,78"
2
```

Code Snippet 338

```
1 CURVE_Q13_FLAG_1_BITANGENTS="13,126,121,91,150,105,96,170,18,164,29,0,48,100,81,6
  8,137,69,47,8,162,4,149,31,182,169,19,66"
2
```

Code Snippet 339

```
1 CURVE_Q13_FLAG_1_TRANS_1="4,12,0,1,11,5,12,10,6"
2
```

Code Snippet 340

```
1 CURVE_Q13_FLAG_1_TRANS_2="1,0,0,12,3,0,9,7,8"
2
```

And now the command to create the variety:

Code Snippet 341

```
1 quartic_curve_q13_f1_apply_transformation:
2 ▷ $(ORBITER) -v 10 \
3 ▷ ▷ -define F -finite_field -q 13 -end \
4 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5 ▷ ▷ -define R -polynomial_ring \
6 ▷ ▷ ▷ -field F \
7 ▷ ▷ ▷ -number_of_variables 3 \
8 ▷ ▷ ▷ -homogeneous_of_degree 4 \
9 ▷ ▷ ▷ -monomial_ordering_partition \
10 ▷ ▷ ▷ -variables "x1,x2,x3" "x_1,x_2,x_3" \
11 ▷ ▷ -end \
```

```

12  ▷ ▷ -define V_q13_f1 -variety \
13  ▷ ▷ ▷ -label_txt curve_q13_f1_clean2 \
14  ▷ ▷ ▷ -label_tex curve\q13\f1\clean2 \
15  ▷ ▷ ▷ -projective_space P \
16  ▷ ▷ ▷ -ring R \
17  ▷ ▷ ▷ -equation_in_algebraic_form $(CURVE_Q13_FLAG_1_EQN) \
18  ▷ ▷ ▷ -points $(CURVE_Q13_FLAG_1_POINTS) \
19  ▷ ▷ ▷ -bitangents $(CURVE_Q13_FLAG_1_BITANGENTS) \
20  ▷ ▷ ▷ -transform $(CURVE_Q13_FLAG_1_TRANS_1) \
21  ▷ ▷ ▷ -transform $(CURVE_Q13_FLAG_1_TRANS_2) \
22  ▷ ▷ -end
23

```

This example will be continued in Section 9.5.

In the second example, do not use a variety object. Instead, we perform the transformation directly on the equation, using a polynomial ring object. We utilize two makefile variables. The first is for the equation, given by the list of coefficients in the partition ordering of monomials:

Code Snippet 342

```

1  QUARTIC_CURVE_Q13_FLAG_0_EQN_COEFFICIENTS="9, 1, 3, 8, 7, 6, 3, 2, 5, 3, 4, 2, 1,
    10, 2"
2

```

The second makefile variable defines the transformation matrix:

Code Snippet 343

```

1  QUARTIC_CURVE_Q13_FLAG_0_TRANS="0,6,8,6,8,10,3,12,1"
2

```

Finally, the Orbiter command to perform the linear substitution of the equation:

Code Snippet 344

```

1  quartic_curve_q13_f0_apply_transformation:
2  ▷ $(ORBITER) -v 6 \
3  ▷ ▷ -define F -finite_field -q 13 -end \
4  ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5  ▷ ▷ -define R -polynomial_ring \
6  ▷ ▷ ▷ -field F \
7  ▷ ▷ ▷ -number_of_variables 3 \
8  ▷ ▷ ▷ -homogeneous_of_degree 4 \
9  ▷ ▷ ▷ -monomial_ordering_partition \
10 ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
11 ▷ ▷ -end \
12 ▷ ▷ -define eqn -vector -dense \
13 ▷ ▷ ▷ $(QUARTIC_CURVE_Q13_FLAG_0_EQN_COEFFICIENTS) \
14 ▷ ▷ ▷ -end \
15 ▷ ▷ -define trans_coeffs -vector -dense \
16 ▷ ▷ ▷ $(QUARTIC_CURVE_Q13_FLAG_0_TRANS) \

```

```

17  ▷ ▷ ▷ -end \
18  ▷ ▷ -define G -linear_group -PGL 3 F -end \
19  ▷ ▷ -define trans -vector_ge \
20  ▷ ▷ ▷ -action G \
21  ▷ ▷ ▷ -vector_data trans.coeffs \
22  ▷ ▷ -end \
23  ▷ ▷ -with R -do \
24  ▷ ▷ ▷ -ring_theoretic_activity \
25  ▷ ▷ ▷ -apply_transformation P eqn trans \
26  ▷ ▷ -end
27

```

The command creates a finite field, a projective space and an associate polynomial ring. After that, the equation is defined by means of a dense coefficient vector in the partition ordering of monomials. After that, the coefficients of a transformation matrix are given, again using a dense vector. Then the appropriate projective linear group is defined. Using this group, the transformation is defined as a group element from the vector of coefficients. Finally, the `-apply_transformation` command is applied to the polynomial ring R as a ring theoretic activity.

We can create a variety by giving the Orbiter rank of the equation. In this case, the coefficient vector of the equation is identified with the homogeneous coordinates of a point in a projective space of suitable dimension. The following example creates a cubic curve:

Code Snippet 345

```

1  cubic_eqn.q3_0:
2  ▷ $(ORBITER) -v 6 \
3  ▷ ▷ -define F -finite_field -q 3 -end \
4  ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5  ▷ ▷ -define R -polynomial_ring \
6  ▷ ▷ ▷ -field F \
7  ▷ ▷ ▷ -number_of_variables 4 \
8  ▷ ▷ ▷ -homogeneous_of_degree 3 \
9  ▷ ▷ ▷ -monomial_ordering_partition \
10 ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
11 ▷ ▷ -end \
12 ▷ ▷ -define V -variety \
13 ▷ ▷ ▷ -projective_space P \
14 ▷ ▷ ▷ -ring R \
15 ▷ ▷ ▷ -equation_by_rank \
16 ▷ ▷ ▷ ▷ 0 \
17 ▷ ▷ ▷ -compute_lines \
18 ▷ ▷ ▷ -label_txt eqn.q3_0 \
19 ▷ ▷ ▷ -label_tex "{\\rm eqn\\_q3\\_0}" \
20 ▷ ▷ -end \
21 ▷ ▷ -with V -do -variety_activity -compute_group \
22 ▷ ▷ ▷ -nauty_control -save_orbit_of_equations eqn_ -end \
23 ▷ ▷ -end \
24 ▷ ▷ -with V -do -variety_activity -singular_points -end \
25 ▷ ▷ -with V -do -variety_activity -report -end
26 ▷ $(PDFLATEX) variety_eqn.q3_0_report.tex
27 ▷ $(OPEN) variety_eqn.q3_0_report.pdf

```


Variety Activities		
Command	Arguments	Purpose
-compute_group		Compute the group of the variety (i.e., the stabilizer of the equation).
-test_isomorphism		Test if two varieties are isomorphic. If yes, compute an isomorphism from the second to the first.
-compute_set_stabilizer		Compute the set stabilizer of the variety. This group contains the stabilizer of the equation as a subgroup.
-nauty_control	options	Nauty options, see Table 17.3.
-report		Produce a report of the variety.
-export		Export data about the variety to a csv file.
-apply_transformation_to_self	M	Apply the given transformation to the variety. M must be a coded semilinear map.
-singular_points		Compute the singular points of the variety.
-output_fname_base	fname	Set output filename base.

Table 7.5: Variety Activities

7.6 Activities for Algebraic Varieties

Table 7.5 lists the activities for varieties.

The following command continues the example of a conic in $\text{PG}(2,13)$ from Section 7.5. The command invokes the `-report` activity for the variety to produce a latex report.

Code Snippet 346

```

1  conic_variety_q13_report:
2  ▷ $(ORBITER) -v 6 \
3  ▷ ▷ -define F -finite_field -q 13 -end \
4  ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5  ▷ ▷ -define R -polynomial_ring \
6  ▷ ▷ ▷ -field F \
7  ▷ ▷ ▷ -number_of_variables 3 \
8  ▷ ▷ ▷ -homogeneous_of_degree 2 \
9  ▷ ▷ ▷ -monomial_ordering_partition \
10 ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
11 ▷ ▷ -end \
12 ▷ ▷ -define V -variety \
13 ▷ ▷ ▷ -projective_space P \
14 ▷ ▷ ▷ -ring R \
15 ▷ ▷ ▷ -equation_in_algebraic_form \
16 ▷ ▷ ▷ ▷ "X0^2-X1*X2" \
17 ▷ ▷ ▷ -label.txt conic \
18 ▷ ▷ ▷ -label.tex "{\\rm conic}" \
19 ▷ ▷ -end \
20 ▷ ▷ -with V -do -variety_activity -report -end
21 ▷ $(PDFLATEX) variety_conic_report.tex

```

```
22  ▷ $(OPEN) variety_conic_report.pdf
23
```

The report contains the list of points with their coordinates, as shown below:

The equation

The equation of the variety is :

$$X_0^2 + 12X_1X_2 = 0$$

(1, 0, 0, 0, 0, 12)

$X_0^2 + 12X_1X_2$

Summary

Number of automorphisms	-1
Number of points	14
Number of lines	0
Number of singular points	0

The variety has 14 points. They are: (1, 2, 3, 39, 57, 62, 68, 77, 134, 141, 150, 151, 175, 178)

0 : $P_1 = (0, 1, 0)$	5 : $P_{62} = (9, 3, 1)$	10 : $P_{150} = (6, 10, 1)$
1 : $P_2 = (0, 0, 1)$	6 : $P_{68} = (2, 4, 1)$	11 : $P_{151} = (7, 10, 1)$
2 : $P_3 = (1, 1, 1)$	7 : $P_{77} = (11, 4, 1)$	12 : $P_{175} = (5, 12, 1)$
3 : $P_{39} = (12, 1, 1)$	8 : $P_{134} = (3, 9, 1)$	13 : $P_{178} = (8, 12, 1)$
4 : $P_{57} = (4, 3, 1)$	9 : $P_{141} = (10, 9, 1)$	

If the automorphism group has not yet been computed, the order is shown as -1.

Orbiter can compute the group of the variety, using the algorithm from [4].

Let us compute the group of the Klein quadric over the field $\mathbb{F}_2$. This continues an earlier example from Section 7.5.

Code Snippet 347

```
1  Op_6.2_group:
2  ▷ $(ORBITER) -v 6 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -define P -projective_space -n 5 -field F -v 0 -end \
5  ▷ ▷ -define R -polynomial_ring \
6  ▷ ▷ ▷ -field F \
7  ▷ ▷ ▷ -number_of_variables 6 \
8  ▷ ▷ ▷ -homogeneous_of_degree 2 \
9  ▷ ▷ ▷ -monomial_ordering_partition \
```

```

10  ▷ ▷ ▷ -variables "X0,X1,X2,X3,X4,X5" \
11  ▷ ▷ ▷ ▷ "X_0,X_1,X_2,X_3,X_4,X_5" \
12  ▷ ▷ -end \
13  ▷ ▷ -define V -variety \
14  ▷ ▷ ▷ -projective_space P \
15  ▷ ▷ ▷ -ring R \
16  ▷ ▷ ▷ -equation_in_algebraic_form \
17  ▷ ▷ ▷ ▷ $(Op_6_eqn) \
18  ▷ ▷ ▷ -label_txt Op_6_2 \
19  ▷ ▷ ▷ -label_tex "{\\rm Op\\_6\\_2}" \
20  ▷ ▷ -end \
21  ▷ ▷ -with V -do -variety_activity -compute_group \
22  ▷ ▷ ▷ -nauty_control -save_orbit_of_equations eqn_ -nauty_log "Op6_2.nauty.log.tx
t" -end \
23  ▷ ▷ -end \
24  ▷ ▷ -with V -do -variety_activity -compute_set_stabilizer \
25  ▷ ▷ ▷ -nauty_control -save_orbit_of_equations eqn_ -end \
26  ▷ ▷ -end \
27  ▷ ▷ -with V -do -variety_activity -singular_points -end \
28  ▷ ▷ -with V -do -variety_activity -export -end \
29  ▷ ▷ -with V -do -variety_activity -report -end
30  ▷ $(PDFLATEX) variety_Op_6_2_report.tex
31  ▷ $(OPEN) variety_Op_6_2_report.pdf
32  ▷ $(ORBITER) -v 2 \
33  ▷ ▷ -define Do -draw_options -embedded -radius 150 \
34  ▷ ▷ ▷ -nodes_empty \
35  ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
36  ▷ ▷ -tree_draw \
37  ▷ ▷ ▷ -file Op6_2.nauty_log.txt \
38  ▷ ▷ ▷ -draw_options Do \
39  ▷ ▷ -end
40  ▷ $(PDFLATEX) Op6_2.nauty_log_draw.tex
41  ▷ $(OPEN) Op6_2.nauty_log_draw.pdf
42

```

The group of the Klein quadric over $\mathbb{F}_2$ is $\text{PGO}^+(6, 2)$ of order 40320. It contains $\text{PGL}(4, 2)$ as a subgroup of index two.

We consider the quartic curve in $\text{PG}(2, 13)$ from Section 7.5, given by the equation

$$X_0^4 + 4X_1^4 + 9X_2^4 + X_0^2X_1^2 + 4X_0^2X_2^2 + 3X_1^2X_2^2 = 0.$$

We encode the equation as a makefile variable:

Code Snippet 348

```

1  CURVE_Q13_F0_CLEAN2_EQUATION="X0^4 + 4*X1^4 + 9*X2^4 + X0^2*X1^2 + 4*X0^2*X2^2 +
   3*X1^2*X2^2"
2

```

Next, we invoke an activity to compute the group of the variety:

Code Snippet 349

```

1  curve.q13_f0_clean2.variety_compute_group:
2  ▷ $(ORBITER) -v 6 \
3  ▷ ▷ -define F -finite_field -q 13 -end \
4  ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5  ▷ ▷ -define R -polynomial_ring \
6  ▷ ▷ ▷ -field F \
7  ▷ ▷ ▷ -number_of_variables 3 \
8  ▷ ▷ ▷ -homogeneous_of_degree 4 \
9  ▷ ▷ ▷ -monomial_ordering_partition \
10 ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
11 ▷ ▷ -end \
12 ▷ ▷ -define V -variety \
13 ▷ ▷ ▷ -projective_space P \
14 ▷ ▷ ▷ -ring R \
15 ▷ ▷ ▷ -equation_in_algebraic_form \
16 ▷ ▷ ▷ ▷ $(CURVE_Q13_F0_CLEAN2_EQUATION) \
17 ▷ ▷ ▷ -label_txt curve.q13_f0_clean2 \
18 ▷ ▷ ▷ -label_tex "{\rm curve\_q13\_f0\_clean2}" \
19 ▷ ▷ -end \
20 ▷ ▷ -with V -do -variety_activity \
21 ▷ ▷ ▷ -compute_group -nauty_control -end \
22 ▷ ▷ -end
23 ▷ $(PDFLATEX) variety_curve.q13_f0_clean2.report.tex
24 ▷ $(OPEN) variety_curve.q13_f0_clean2_report.pdf
25

```

The group turns out to be of order 24.

In the next example, we will compute the stabilizer of the Barth sextic [8] over a small finite field. The Barth sextic has 65 singular points, which is the extreme value for this degree. The definition requires the golden ratio, which involves $\sqrt{5}$. For $p = 11$, 5 is a square, so we can pick $\mathbb{F}_{11}$ as our base field. At first, we define the equation in algebraic form as a makefile variable:

Code Snippet 350

```

1  BARTH_EQUATION_GENERAL="4*(PHI^2*X^2-Y^2)*(PHI^2*Y^2-Z^2)*(PHI^2*Z^2-X^2)-(1+2*PH
  I)*(X^2+Y^2+Z^2-W^2)^2*W^2"
2

```

Next, we create the object over the field $\mathbb{F}_{11}$ and compute both the group and the set of singular points:

Code Snippet 351

```

1  Barth_sextic_mod_11:
2  ▷ $(ORBITER) -v 10 \
3  ▷ ▷ -define F -finite_field -q 11 -end \
4  ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5  ▷ ▷ -define R -polynomial_ring \
6  ▷ ▷ ▷ -field F \
7  ▷ ▷ ▷ -number_of_variables 4 \
8  ▷ ▷ ▷ -homogeneous_of_degree 6 \

```

```

9      ▷ ▷ ▷ -monomial_ordering_partition \
10     ▷ ▷ ▷ -variables "X,Y,Z,W" "X,Y,Z,W" \
11     ▷ ▷ -end \
12     ▷ ▷ -define V -variety \
13     ▷ ▷ ▷ -projective_space P \
14     ▷ ▷ ▷ -ring R \
15     ▷ ▷ ▷ -equation_in_algebraic_form \
16     ▷ ▷ ▷ ▷ $(BARTH_EQUATION_GENERAL) \
17     ▷ ▷ ▷ -set_parameters "PHI" "\Phi" "8" \
18     ▷ ▷ ▷ -label_txt Barth_q11 \
19     ▷ ▷ ▷ -label_tex Barth\_q11 \
20     ▷ ▷ -end \
21     ▷ ▷ -with V -do -variety_activity \
22     ▷ ▷ ▷ -compute_group -nauty_control -end \
23     ▷ ▷ -end \
24     ▷ ▷ -with V -do -variety_activity -singular_points -end \
25     ▷ ▷ -with V -do -variety_activity -report -end
26     ▷ $(PDFLATEX) variety_Barth_q11_report.tex
27     ▷ $(OPEN) variety_Barth_q11_report.pdf
28

```

Next, we will compute the stabilizer of the Hirschfeld surface from Section 7.5 over the field $\mathbb{F}_2$. The following command creates the variety and computes the stabilizer, which is a group of order 720:

Code Snippet 352

```

1      Hirschfeld.q2_group:
2      ▷ $(ORBITER) -v 6 \
3      ▷ ▷ -define F -finite_field -q 2 -end \
4      ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5      ▷ ▷ -define R -polynomial_ring \
6      ▷ ▷ ▷ -field F \
7      ▷ ▷ ▷ -number_of_variables 4 \
8      ▷ ▷ ▷ -homogeneous_of_degree 3 \
9      ▷ ▷ ▷ -monomial_ordering_partition \
10     ▷ ▷ ▷ -variables "X,Y,Z,W" "X,Y,Z,W" \
11     ▷ ▷ -end \
12     ▷ ▷ -define V -variety \
13     ▷ ▷ ▷ -label_txt Hirschfeld_q2 \
14     ▷ ▷ ▷ -label_tex Hirschfeld\_q2 \
15     ▷ ▷ ▷ -projective_space P \
16     ▷ ▷ ▷ -ring R \
17     ▷ ▷ ▷ -equation_in_algebraic_form \
18     ▷ ▷ ▷ ▷ $(HIRSCHFELD_EQUATION_XYZW) \
19     ▷ ▷ -end \
20     ▷ ▷ -with V -do -variety_activity \
21     ▷ ▷ ▷ -compute_group -nauty_control -end \
22     ▷ ▷ -end \
23     ▷ ▷ -with V -do -variety_activity -singular_points -end \
24     ▷ ▷ -with V -do -variety_activity -report -end
25     ▷ $(PDFLATEX) variety_Hirschfeld.q2_report.tex

```

```

26   ▷ $(OPEN) variety_Hirschfeld_q2_report.pdf
27

```

The next example investigates a nonsingular cubic surface with 9 lines over the field $\mathbb{F}_4$. The variety is called Dickson36, since it appears as the 36th example in [29]. The automorphism group is computed as well. It is a group of order 216. A latex report is produced.

Code Snippet 353

```

1   variety_Dickson36_9lines_q4_group:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define F -finite_field -q 4 -end \
4   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5   ▷ ▷ -define R -polynomial_ring \
6   ▷ ▷ ▷ -field F \
7   ▷ ▷ ▷ -number_of_variables 4 \
8   ▷ ▷ ▷ -homogeneous_of_degree 3 \
9   ▷ ▷ ▷ -monomial_ordering_partition \
10  ▷ ▷ ▷ -variables "X0,X1,X2,X3" \
11  ▷ ▷ ▷ ▷ "X_0,X_1,X_2,X_3" \
12  ▷ ▷ -end \
13  ▷ ▷ -define V -variety \
14  ▷ ▷ ▷ -projective_space P \
15  ▷ ▷ ▷ -ring R \
16  ▷ ▷ ▷ -equation_in_algebraic_form \
17  ▷ ▷ ▷ ▷ "X0^3+X1^3+X2^3+X3^3+X0^2*X3+X1^2*X2" \
18  ▷ ▷ ▷ -compute_lines \
19  ▷ ▷ ▷ -label_txt Dickson36 \
20  ▷ ▷ ▷ -label_tex "{\\rm Dickson36}" \
21  ▷ ▷ -end \
22  ▷ ▷ -with V -do -variety_activity -compute_group \
23  ▷ ▷ ▷ -nauty_control -save_orbit_of_equations eqn_ -end \
24  ▷ ▷ -end \
25  ▷ ▷ -with V -do -variety_activity -compute_set_stabilizer \
26  ▷ ▷ ▷ -nauty_control -save_orbit_of_equations eqn_ -end \
27  ▷ ▷ -end \
28  ▷ ▷ -with V -do -variety_activity -singular_points -end \
29  ▷ ▷ -with V -do -variety_activity -export -end \
30  ▷ ▷ -with V -do -variety_activity -report -end
31  ▷ $(PDFLATEX) variety_Dickson36_report.tex
32  ▷ $(OPEN) variety_Dickson36_report.pdf
33

```

In the next example, we test if two varieties are isomorphic. We continue the example from Section 6.2. The varieties are given as vector of coefficients:

Code Snippet 354

```

1   QC_Q27_F0159_COEFFS="15,6,12,2,4,23,24,20,15,0,4,3,0,0,23"
2

```

Code Snippet 355

```

1   QC_Q27_F0388_COEFFS="1,25,11,12,16,8,13,25,0,14,2,2,19,19,2"
2

```

The following command performs isomorphism testing. The curves are isomorphic, and an isomorphism is computed.

Code Snippet 356

```

1   quartic_curve_q27_fo159_fo338.test.isomorphism:
2   ▷ $(ORBITER) -v 12 \
3   ▷ ▷ -define F -finite_field -q 27 -end \
4   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5   ▷ ▷ -define R -polynomial_ring \
6   ▷ ▷ ▷ -field F \
7   ▷ ▷ ▷ -number_of_variables 3 \
8   ▷ ▷ ▷ -homogeneous_of_degree 4 \
9   ▷ ▷ ▷ -monomial_ordering_partition \
10  ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
11  ▷ ▷ -end \
12  ▷ ▷ -define V159 -variety \
13  ▷ ▷ ▷ -label_txt curve_q27_fo159 \
14  ▷ ▷ ▷ -label_tex curve\_q27\_fo159 \
15  ▷ ▷ ▷ -projective_space P \
16  ▷ ▷ ▷ -ring R \
17  ▷ ▷ ▷ -equation_by_coefficients $(QC_Q27_F0159_COEFFS) \
18  ▷ ▷ -end \
19  ▷ ▷ -define V388 -variety \
20  ▷ ▷ ▷ -label_txt curve_q27_fo388 \
21  ▷ ▷ ▷ -label_tex curve\_q27\_fo388 \
22  ▷ ▷ ▷ -projective_space P \
23  ▷ ▷ ▷ -ring R \
24  ▷ ▷ ▷ -equation_by_coefficients $(QC_Q27_F0388_COEFFS) \
25  ▷ ▷ -end \
26  ▷ ▷ -with V159 -and V388 -do -variety_activity \
27  ▷ ▷ ▷ -test_isomorphism \
28  ▷ ▷ ▷ -nauty_control -nauty_log "quartic_curve_q27_nauty_log.txt" -end \
29  ▷ ▷ -end
30  ▷ $(ORBITER) -v 2 \
31  ▷ ▷ -define Do -draw_options -embedded -radius 150 \
32  ▷ ▷ ▷ -nodes_empty \
33  ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
34  ▷ ▷ -tree_draw \
35  ▷ ▷ ▷ -file quartic_curve_q27_nauty_log.txt \
36  ▷ ▷ ▷ -draw_options Do \
37  ▷ ▷ -end
38  ▷ $(PDFLATEX) quartic_curve_q27_nauty_log.draw.tex
39  ▷ $(OPEN) quartic_curve_q27_nauty_log_draw.pdf
40

```

Let us look at the command in some detail. To begin with, we create one variety object for each of the two curves. After that, we invoke the `-test_isomorphism` command as an activity applied to the two varieties. This command will construct a mapping from the second variety to the first or prove that none exists. In

the example, the command will create the isomorphism that was already mentioned in Section 6.2. The mapping is the point map, which means it takes the set of rational points of the second variety and maps it to the set of rational points of the first variety. In order to map the second equation to the first, the contragredient action is to be used.

Here is another example of isomorphism testing:

Code Snippet 357

```
1 QC_Q9_COEFFS="0, 1, 0, 0, 6, 0, 0, 4, 0, 0, 0, 0, 0, 0, 0"
2
```

Code Snippet 358

```
1 QC_Q9_HERMITIAN_COEFFS="1,1,1,0,0,0,0,0,0,0,0,0,0,0,0"
2
```

Code Snippet 359

```
1 quartic_curve_q9_hermitian_fo0_test_isomorphism:
2   ▷ $(ORBITER) -v 12 \
3   ▷ ▷ -define F -finite_field -q 9 -end \
4   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5   ▷ ▷ -define R -polynomial_ring \
6   ▷ ▷ ▷ -field F \
7   ▷ ▷ ▷ -number_of_variables 3 \
8   ▷ ▷ ▷ -homogeneous_of_degree 4 \
9   ▷ ▷ ▷ -monomial_ordering_partition \
10  ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
11  ▷ ▷ -end \
12  ▷ ▷ -define V0 -variety \
13  ▷ ▷ ▷ -label_txt curve_q9_fo0 \
14  ▷ ▷ ▷ -label_tex curve\_q9\_fo0 \
15  ▷ ▷ ▷ -projective_space P \
16  ▷ ▷ ▷ -ring R \
17  ▷ ▷ ▷ -equation_by_coefficients $(QC_Q9_COEFFS) \
18  ▷ ▷ -end \
19  ▷ ▷ -define VH -variety \
20  ▷ ▷ ▷ -label_txt curve_q9_Hermitian \
21  ▷ ▷ ▷ -label_tex curve\_q9\_Hermitian \
22  ▷ ▷ ▷ -projective_space P \
23  ▷ ▷ ▷ -ring R \
24  ▷ ▷ ▷ -equation_by_coefficients $(QC_Q9_HERMITIAN_COEFFS) \
25  ▷ ▷ -end \
26  ▷ ▷ -with VH -and V0 -do -variety_activity \
27  ▷ ▷ ▷ -test_isomorphism \
28  ▷ ▷ ▷ -nauty_control -nauty_log "quartic_curve_q9_nauty_log.txt" -end \
29  ▷ ▷ -end
30
```

We can specify the variety in algebraic form, as in the next example:

Code Snippet 360

```

1   QC_Q9_ORBITER_CATALOGUE_0_AF="X0^4 + X1^4 + X2^4 + X0^3*X1 + 5*X0^3*X2 + X0*X1^3
    + 7*X1^3*X2 + 7*X0*X2^3 + 5*X1*X2^3"
2

```

Code Snippet 361

```

1   quartic_curve_q9_hermitian_and_OCN0_test_isomorphism:
2   ▷ $(ORBITER) -v 12 \
3   ▷ ▷ -define F -finite_field -q 9 -end \
4   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5   ▷ ▷ -define R -polynomial_ring \
6   ▷ ▷ ▷ -field F \
7   ▷ ▷ ▷ -number_of_variables 3 \
8   ▷ ▷ ▷ -homogeneous_of_degree 4 \
9   ▷ ▷ ▷ -monomial_ordering_partition \
10  ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
11  ▷ ▷ -end \
12  ▷ ▷ -define VH -variety \
13  ▷ ▷ ▷ -label_txt curve_q9_Hermitian \
14  ▷ ▷ ▷ -label_tex curve\_q9\_Hermitian \
15  ▷ ▷ ▷ -projective_space P \
16  ▷ ▷ ▷ -ring R \
17  ▷ ▷ ▷ -equation_by_coefficients $(QC_Q9_HERMITIAN_COEFFS) \
18  ▷ ▷ -end \
19  ▷ ▷ -define V2 -variety \
20  ▷ ▷ ▷ -projective_space P \
21  ▷ ▷ ▷ -ring R \
22  ▷ ▷ ▷ -equation_in_algebraic_form \
23  ▷ ▷ ▷ ▷ $(QC_Q9_ORBITER_CATALOGUE_0_AF) \
24  ▷ ▷ ▷ -label_txt QC_Q9_0 \
25  ▷ ▷ ▷ -label_tex "${\rm QC\_Q9\_0}" \
26  ▷ ▷ -end \
27  ▷ ▷ -with VH -and V2 -do -variety_activity \
28  ▷ ▷ ▷ -test_isomorphism \
29  ▷ ▷ ▷ -nauty_control -nauty_log "quartic_curve_q9_nauty_log.txt" -end \
30  ▷ ▷ -end
31

```

Once an isomorphism is computed, we want to check it. We apply the isomorphism to the equation to produce the other equation. We use the quartic curve, including the set of lines (the bitangents), and the previously computed mapping, all stored as makefile variables:

Code Snippet 362

```

1   QUARTIC_CURVE_Q9_CLEAN="X2^4+6*X1^3*X3+4*X1*X3^3"
2

```

Code Snippet 363

```

1   QC_Q9_CLEAN_LINES="85, 48, 73, 78, 16, 55, 44, 87, 74, 76, 57, 13, 90, 51, 18, 27
    , 22, 14, 52, 21, 25, 60, 81, 46, 43, 82, 30, 0"
2

```

Code Snippet 364

```

1   QC_Q9_TRANSFORM_CLEAN_TO_HERMITIAN="5,8,6,8,1,6,0,3,6,1"
2

```

And here is the command to create the transformed variety:

Code Snippet 365

```

1   quartic_curve_q9_Hermitian_transform:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define F -finite_field -q 9 -end \
4   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5   ▷ ▷ -define R -polynomial_ring \
6   ▷ ▷ ▷ -field F \
7   ▷ ▷ ▷ -number_of_variables 3 \
8   ▷ ▷ ▷ -homogeneous_of_degree 4 \
9   ▷ ▷ ▷ -monomial_ordering_partition \
10  ▷ ▷ ▷ -variables "X1,X2,X3" "X_1,X_2,X_3" \
11  ▷ ▷ -end \
12  ▷ ▷ -define V -variety \
13  ▷ ▷ ▷ -projective_space P \
14  ▷ ▷ ▷ -ring R \
15  ▷ ▷ ▷ -equation_in_algebraic_form $(QUARTIC_CURVE_Q9_CLEAN) \
16  ▷ ▷ ▷ -bitangents $(QC_Q9_CLEAN_LINES) \
17  ▷ ▷ ▷ -transform $(QC_Q9_TRANSFORM_CLEAN_TO_HERMITIAN) \
18  ▷ ▷ ▷ -label_txt qc_q9 \
19  ▷ ▷ ▷ -label_tex "{\\rm qc\\_q9}" \
20  ▷ ▷ -end
21

```

7.7 Mappings between Projective Spaces

We can create maps between projective spaces. There are two possible ways to do so. The first is by using the `-map` command, which is a projective space activity. The second is by using a map object. We will demonstrate both ways next.

The first example shows a Cremona map from $\text{PG}(2, 13)$ to $\text{PG}(3, 13)$. At first, we define the 4 coordinate functions as makefile variables. These coordinate functions must be polynomials. In the command, we create the finite field, the projective space for the domain of the map, and the polynomial ring associated with the domain. After that, we will create symbolic objects for the 4 coordinate functions. The object called “Cremona” is a vector of the coordinate functions. Finally, the `-map` command is used to define and evaluate the map. This command is a projective space activity. Here are the necessary makefile definitions and commands:

Code Snippet 366

```
1 CREMONA_MAP_Y0="3*y0^5*y2+4*y0^3*y1^2*y2\
2 +2*y0^3*y2^3+y0*y1^4*y2\
3 +6*y0*y1^2*y2^3+9*y0*y2^5"
4
```

Code Snippet 367

```
1 CREMONA_MAP_Y1="y0^5*y1+5*y0^3*y1^3\
2 +12*y0^3*y1*y2^2+3*y0*y1^5\
3 +5*y0*y1^3*y2^2+y0*y1*y2^4"
4
```

Code Snippet 368

```
1 CREMONA_MAP_Y2="10*y0^6+11*y0^4*y1^2\
2 +11*y0^4*y2^2+4*y0^2*y1^4\
3 +9*y0^2*y1^2*y2^2+4*y0^2*y2^4"
4
```

Code Snippet 369

```
1 CREMONA_MAP_Y3=""
2
```

Code Snippet 370

```
1 Cremona_map:
2 ▷ $(ORBITER) -v 6 \
3 ▷ ▷ -define F -finite_field -q 13 -end \
4 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5 ▷ ▷ -define R -polynomial_ring \
6 ▷ ▷ ▷ -field F \
7 ▷ ▷ ▷ -number_of_variables 3 \
8 ▷ ▷ ▷ -homogeneous_of_degree 6 \
9 ▷ ▷ ▷ -monomial_ordering_lex \
10 ▷ ▷ ▷ -variables "y0,y1,y2" "y_0,y_1,y_2" \
```


Mappings Between Projective Spaces		
Command	Arguments	Purpose
-domain	D	Set the domain to D (a projective space).
-codomain	C	Set the codomain to C (a projective space).
-ring	R	Set the coordinate ring for D.
-formula	f	Use the symbolic object f to define the mapping.
-substitute	s	Substitute s into the equation before doing the mapping.
-affine		Treat the domain as an affine space.
-object_in_codomain_cubic_surface	S	Set the codomain to the cubic surface object S. The map will now output images as indices of points on the surface.

Table 7.6: Mappings Between Projective Spaces

```

11  ▷ ▷ ▷ -end \
12  ▷ ▷ -define Y0 -symbolic_object \
13  ▷ ▷ ▷ -field F \
14  ▷ ▷ ▷ -text $(CREMONA_MAP_Y0) \
15  ▷ ▷ ▷ -end \
16  ▷ ▷ -define Y1 -symbolic_object \
17  ▷ ▷ ▷ -field F \
18  ▷ ▷ ▷ -text $(CREMONA_MAP_Y1) \
19  ▷ ▷ ▷ -end \
20  ▷ ▷ -define Y2 -symbolic_object \
21  ▷ ▷ ▷ -field F \
22  ▷ ▷ ▷ -text $(CREMONA_MAP_Y2) \
23  ▷ ▷ ▷ -end \
24  ▷ ▷ -define Cremona -symbolic_object \
25  ▷ ▷ ▷ -field F \
26  ▷ ▷ ▷ -text "Y0,Y1,Y2" \
27  ▷ ▷ ▷ -end \
28  ▷ ▷ -with P -do \
29  ▷ ▷ -projective_space_activity \
30  ▷ ▷ ▷ -map R Cremona "" \
31  ▷ ▷ -end
32

```

Next, we demonstrate the mapping object to define a map between projective spaces. Table 7.6 lists Orbiter commands for mappings between projective spaces.

The first example creates the Veronese variety of $\text{PG}(1, 7)$ in $\text{PG}(3, 7)$:

Code Snippet 371

```

1  Veronese_1.3.q7:
2  ▷ $(ORBITER) -v 5 \
3  ▷ ▷ -define Fq -finite_field -q 7 -end \
4  ▷ ▷ -define C -symbolic_object \

```

```

5    ▷ ▷ ▷ -field Fq \
6    ▷ ▷ ▷ -managed_variables "y0,y1" \
7    ▷ ▷ ▷ -text "y1^3,y0*y1^2,y0^2*y1,y0^3" \
8    ▷ ▷ ▷ -end \
9    ▷ ▷ ▷ -define P1 -projective_space -n 1 -field Fq -v 0 -end \
10   ▷ ▷ ▷ -define P3 -projective_space -n 3 -field Fq -v 0 -end \
11   ▷ ▷ ▷ -define R2 -polynomial_ring \
12   ▷ ▷ ▷ -field Fq \
13   ▷ ▷ ▷ -number_of_variables 2 \
14   ▷ ▷ ▷ -homogeneous_of_degree 3 \
15   ▷ ▷ ▷ -monomial_ordering_partition \
16   ▷ ▷ ▷ -variables "y0,y1" "y_0,y_1" \
17   ▷ ▷ ▷ -end \
18   ▷ ▷ ▷ -define map -mapping \
19   ▷ ▷ ▷ -domain P1 \
20   ▷ ▷ ▷ -codomain P3 \
21   ▷ ▷ ▷ -ring R2 \
22   ▷ ▷ ▷ -formula C \
23   ▷ ▷ ▷ -substitute "" \
24   ▷ ▷ ▷ -end
25

```

The image is written as a set of points in Orbiter ranks:

(3, 0, 4, 268, 336, 184, 224, 364)

For a deeper exploration of the combinatorial and group-theoretic properties of this object, see Section 17.3.

Chapter 8

Orbit Algorithms

8.1 Schreier Trees

Orbiter provides three different orbit algorithms:

- (i) Schreier trees,
- (ii) poset classification, and
- (iii) orderly generation using canonical forms.

The “orbit” Orbiter object provides a front end to all three orbit algorithms. For background on Schreier trees, see [23, 41, 68]. Poset based algorithms will be discussed in Section 8.2. Orderly generation using canonical forms is discussed in Section 17.2.

To create an orbit object, the commands listed in Tables 8.1-8.2 can be used. Table 8.3 lists activities for an object of type orbit.

Consider the group $\text{PGL}(4, 2)$ in the natural action on the set of points of $\text{PG}(3, 2)$. The action is transitive on the 15 points of $\text{PG}(3, 2)$. The following example computes the Schreier tree for the action:

Code Snippet 372

```
1  orbits_PGL_4_2_on_points.draw_tree:
2  ▷ $(ORBITER) -v 8 \
3  ▷ ▷ -define Do -draw_options -embedded -end \
4  ▷ ▷ -define G -linear_group -PGL 4 2 -end \
5  ▷ ▷ -define Orb -orbits -group G \
6  ▷ ▷ ▷ -on_points \
7  ▷ ▷ -end \
8  ▷ ▷ -with Orb -do -orbits_activity \
9  ▷ ▷ ▷ -report Do \
10 ▷ ▷ -end \
11 ▷ ▷ -with Orb -do -orbits_activity \
12 ▷ ▷ ▷ -export_something "orbit" 0 \
13 ▷ ▷ -end \
14 ▷ ▷ -with Orb -do -orbits_activity \
15 ▷ ▷ ▷ -draw_tree 0 Do \
16 ▷ ▷ -end \
17 ▷ ▷ -with Orb -do -orbits_activity \
18 ▷ ▷ ▷ -schreier_poset 0 PGL_4_2_on_points \
```

Computing Orbits		
Command	Arguments	Purpose
-on_points		Perform Schreier algorithm to compute the orbits of G in the natural action and under the group included in the action.
-on_points_with_generators	gens	Perform Schreier algorithm to compute the orbits of G in the natural action and under the group given by a set of generators.
-on_subsets	k control	Consider the induced action on subsets of size k . This option will invoke the poset classification algorithm.
-of_one_subset	set-label	Compute the orbit of the given set under the group G . This algorithm will build up the Schreier tree. The orbit will be saved to a csv file.
-on_subspaces	k control	Consider the induced action on subspaces of size k . We assume that the group action is linear. This option will invoke the poset classification algorithm.
-on_tensors	d control	Compute the orbits on the d -fold tensor product.
-on_partition	k control	Compute the orbits on partitions of type $k + k + k$. In this case, $3k$ must equal the degree of the defining action of G . This option will invoke the poset classification algorithm.
-on_polynomials	R	Consider the action of G on homogenous polynomials as defined in ring R. The action must be linear and the number of variables in R must match the degree of the matrix group. This option will invoke the Schreier orbit algorithm.
-on_one_polynomial	R equation	Consider the action of G on polynomials of degree d in n variables. G must be a matrix group. Here, the action must be linear on an n -dimensional space.
-on_cubic_curves	options	Classify cubic curves by means of $(9, 3)$ -arcs. For valid options, see Tab. 8.8.
-on_cubic_surfaces	P control	Classify cubic surfaces in the projective space P, using the given poset classification control options.
-on_arcs	P control	Classify arcs in the projective space P, using the given arc classification control options, see Tab. 8.8. This option will invoke the poset classification algorithm.
-classification_by_canonical_form	description	Perform the classification-by-canonical-form algorithm using Nauty. See Chapter 17.

Table 8.1: Computing Orbits

Computing Orbits		
Command	Arguments	Purpose
-group	G	Specify the group. The argument must be an object of type group.
-group_action	G	Specify the action. The argument must be an object of type group.
-generators	G	Specify the generators for the group or subgroup whose orbits we wish to compute. The argument must be an object of type group. This option is only required when poset classification is used (i.e., not for the Schreier algorithm).
-print_interval	n	Set the print interval for the Schreier orbit algorithm to n .

Table 8.2: Computing Orbits

Orbit Based Activities		
Command	Arguments	Purpose
-report	options	Create a report on the orbits.
-export_something	type data	Export orbit data of the given type.
-export_trees		Create drawings of the Schreier trees.
-export_source_code		Create a cpp file for the orbit representatives.
-export_levels	idx	
-draw_tree	i options	Create a drawing of the i -th orbit as a tree. Uses the given draw options.
-stabilizer	i	Compute the stabilizer of the given point i .
-stabilizer_of_orbit_rep	i	Compute the stabilizer of the orbit representative of orbit i .
-Kramer_Mesner_matrix	t k	Compute the Kramer Mesner matrix associated with the orbits on t -subsets and the orbits on k -subsets. This option requires a poset type problem.
-recognize	S	Recognize S in the orbit data structure.
-transporter	S	For each $s \in S$, which is the index of an element in the orbit, compute a transporter that maps the element to the orbit representative. At present, this is only implemented for orbits on equations.
-schreier_poset	i prefix	Export the orbit whose index is i as a partially ordered set. The file name is constructed by using the given prefix.
-draw_options	label	Specify draw options.
-report_options	descr	Set additional parameters for the report, see Table 8.7.
-poset_classification_activity	label	Perform the given poset classification activity. This option requires a poset type problem.

Table 8.3: Orbit Based Activities

```

19   ▷ ▷ -end
20   ▷ $(ORBITER) -v 3 \
21   ▷ ▷ -draw_layered_graph \
22   ▷ ▷ ▷ PGL_4.2_on_points_poset.layered_graph \
23   ▷ ▷ ▷ -place \
24   ▷ ▷ -radius 500 -embedded \
25   ▷ ▷ ▷ -line_width 1.1 -x_stretch 1.4 -scale 0.25 \
26   ▷ ▷ -end
27   ▷ $(PDFLATEX) PGL_4.2_on_points_poset.draw.tex
28   ▷ $(OPEN) PGL_4.2_on_points_poset.draw.pdf
29   ▷ $(PDFLATEX) PGL_4.2_on_points_poset.draw.direct.tex
30   ▷ $(OPEN) PGL_4.2_on_points_poset.draw.direct.pdf
31   ▷ #$(PDFLATEX) PGL_4.2_orbits_report.tex
32   ▷ #$(OPEN) PGL_4.2_orbits_report.pdf
33   ▷ #$(PDFLATEX) PGL_4.2_orbit_0_tree.tex
34   ▷ #$(OPEN) PGL_4.2_orbit_0_tree.pdf
35

```

To draw the tree, the following command is used:

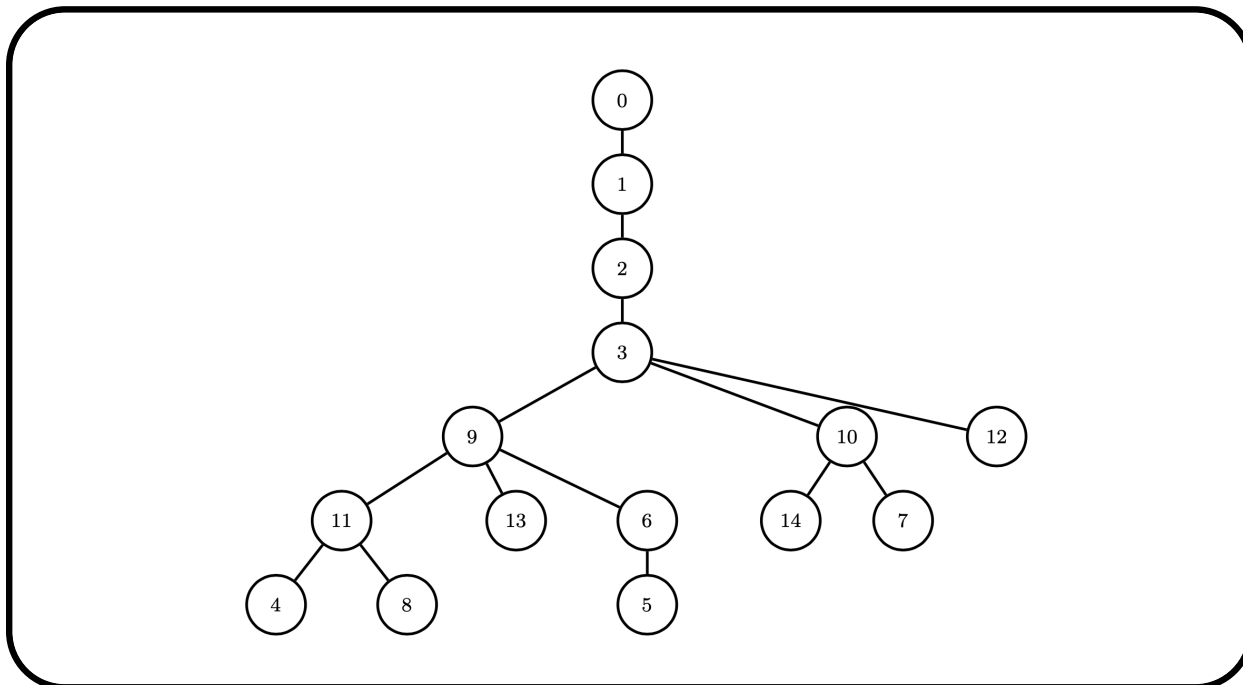
Code Snippet 373

```

1   orbits_PGL_4.2_on_points_export_trees:
2   ▷ $(ORBITER) -v 4 \
3   ▷ ▷ -define Do -draw_options -embedded -end \
4   ▷ ▷ -define G -linear_group -PGL 4 2 -end \
5   ▷ ▷ -define Orb -orbits -group G \
6   ▷ ▷ ▷ -on_points \
7   ▷ ▷ -end \
8   ▷ ▷ -define ro \
9   ▷ ▷ ▷ -poset_classification_report_options \
10  ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
11  ▷ ▷ ▷ ▷ -draw_options Do \
12  ▷ ▷ -end \
13  ▷ ▷ -with Orb -do -orbits_activity \
14  ▷ ▷ ▷ -report "" \
15  ▷ ▷ ▷ -report_options ro \
16  ▷ ▷ -end \
17  ▷ ▷ -with Orb -do -orbits_activity \
18  ▷ ▷ ▷ -export_trees \
19  ▷ ▷ -end
20  ▷ $(ORBITER) -v 3 \
21  ▷ ▷ -draw_layered_graph \
22  ▷ ▷ ▷ orbit_PGL_4.2_0.layered_graph \
23  ▷ ▷ ▷ -place \
24  ▷ ▷ -radius 500 -spanning_tree -embedded \
25  ▷ ▷ ▷ -line_width 1.1 -x_stretch 1.4 -scale 0.25 \
26  ▷ ▷ -end
27  ▷ $(PDFLATEX) orbit_PGL_4.2_0.draw.tex
28  ▷ $(OPEN) orbit_PGL_4.2_0.draw.pdf
29

```

The Schreier tree is shown below



The next command creates the projective plane of order 4 using a line-transitive group. The design is the orbit of a certain set of size 5. The group is created in product action. The set results from a search for a special class of linear spaces as described in Section 14.4.

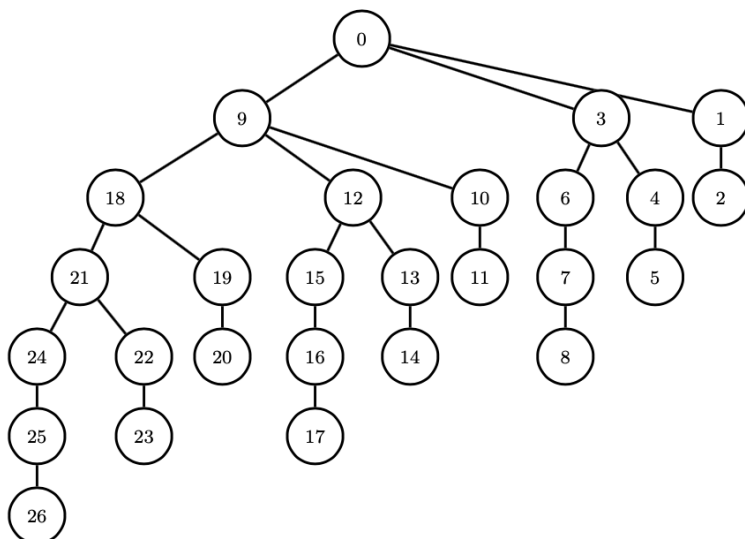
Code Snippet 374

```

1    DD_PP4_orbit:
2    ▷ $(ORBITER) -v 4 \
3    ▷ ▷ -define Do -draw_options \
4    ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5    ▷ ▷ -end \
6    ▷ ▷ -define G1 -linear_group -AGL 1 3 \
7    ▷ ▷ ▷ -end \
8    ▷ ▷ -define G2 -linear_group -AGL 1 7 \
9    ▷ ▷ ▷ -end \
10   ▷ ▷ -define G -modified_group -direct_product "G1,G2" \
11   ▷ ▷ ▷ "21" "1,1,1,1" \
12   ▷ ▷ -end \
13   ▷ ▷ -with G -do \
14   ▷ ▷ -group_theoretic_activity \
15   ▷ ▷ ▷ -report Do \
16   ▷ ▷ -end \
17   ▷ ▷ -define S -vector -dense "0,1,3,13,20" -end \
18   ▷ ▷ -define Orb -orbits -group G \
19   ▷ ▷ ▷ -of_one_subset S \
20   ▷ ▷ -end
21

```

Consider the wreath product acting on rank-one tensors from Section 5.4. The following command sequence computes the orbits, exports the Schreier tree, and produces the drawing



Code Snippet 375

```

1   T3r1_orbits:
2   ▷ $(ORBITER) -v 4 \
3   ▷ ▷ -define Do -draw_options -embedded -end \
4   ▷ ▷ -define G \
5   ▷ ▷ -linear_group -GL_d.q.wr.Sym.n 2 2 3 \
6   ▷ ▷ ▷ -on_rank_one_tensors -end \
7   ▷ ▷ -define Orb -orbits -group G \
8   ▷ ▷ ▷ -on_points \
9   ▷ ▷ -end \
10  ▷ ▷ -with Orb -do -orbits_activity \
11  ▷ ▷ ▷ -report Do \
12  ▷ ▷ -end \
13  ▷ ▷ -with Orb -do -orbits_activity \
14  ▷ ▷ ▷ -draw_tree 0 Do \
15  ▷ ▷ -end
16  ▷ $(PDFLATEX) GL_2.2.wreath.Sym3_orbit_0_tree.tex
17  ▷ $(OPEN) GL_2.2.wreath.Sym3_orbit_0_tree.pdf
18

```

In the next example, we compute the orbits of the linear group $\text{PGL}(3, 2)$ on plane cubics:

Code Snippet 376

```

1   poly_orbits_d3.n2.q2.compute:
2   ▷ $(ORBITER) -v 4 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -yout 500000 -radius 15 -nodes_empty \
5   ▷ ▷ ▷ -line_width 0.5 -y_stretch 0.25 -embedded \
6   ▷ ▷ -end \

```



```

7   ▷ ▷ -define F -finite_field -q 2 -end \
8   ▷ ▷ -define G -linear_group -PGL 3 F -end \
9   ▷ ▷ -define R -polynomial_ring \
10  ▷ ▷ ▷ -field F \
11  ▷ ▷ ▷ -number_of_variables 3 \
12  ▷ ▷ ▷ -homogeneous_of_degree 3 \
13  ▷ ▷ ▷ -monomial_ordering_partition \
14  ▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \
15  ▷ ▷ -end \
16  ▷ ▷ -define Orb -orbits -group G \
17  ▷ ▷ ▷ -print_interval 1000000 \
18  ▷ ▷ ▷ -on_polynomials R \
19  ▷ ▷ -end \
20  ▷ ▷ -with Orb -do -orbits_activity \
21  ▷ ▷ ▷ -report Do \
22  ▷ ▷ -end \
23  ▷ ▷ -with Orb -do -orbits_activity \
24  ▷ ▷ ▷ -export_something representatives -1 \
25  ▷ ▷ -end
26  ▷ ▷ -print_symbols
27  ▷ $(PDFLATEX) poly_orbits_d3_n2_q2.tex
28  ▷ $(OPEN) poly_orbits_d3_n2_q2.pdf
29

```

The command produces a latex report about the orbits and a csv-file containing additional data. The csv-file is shown below, as seen through a spreadsheet program.

Row	OrbIdx	Go	EqnCode	EqnVec	EqnAf	NbPts	Pts
0	0	24	0	1,0,0,0,0,0,0,0	X^3	3	1, 2, 6
1	1	4	3	0,0,0,1,0,0,0,0	X^2Y	5	0, 1, 2, 5, 6
2	2	6	9	0,0,0,0,0,0,0,1	X^2YZ	6	0, 1, 2, 4, 5, 6
3	3	6	10	1,1,1,1,1,1,1,1	$X^3 + Y^3 + Z^3 + X^2Y + X^2Z + X^2Y^2 + Y^2Z + X^2Z^2 + Y^2Z^2 + X^2YZ$	4	3, 4, 5, 6
4	4	8	11	1,1,0,0,0,0,0,0	$X^3 + Y^3$	3	2, 3, 4
5	5	6	14	1,1,0,0,0,0,0,0	$X^3 + Y^3 + Z^3$	3	4, 5, 6
6	6	12	17	1,1,0,1,0,0,0,0	$X^3 + Y^3 + X^2Y$	1	2
7	7	1	18	0,0,1,1,0,0,0,0	$Z^3 + X^2Y$	3	0, 1, 3
8	8	3	21	1,1,1,1,0,0,0,0	$X^3 + Y^3 + Z^3 + X^2Y$	3	3, 5, 6
9	9	2	35	0,1,1,1,1,0,0,0	$Y^3 + Z^3 + X^2Y + X^2Z$	5	0, 3, 4, 5, 6
10	10	4	36	1,1,1,1,0,0,0,0	$X^3 + Y^3 + Z^3 + X^2Y + X^2Z$	1	6
11	11	24	44	0,0,0,1,0,1,0,0	$X^2Y + X^2Y^2$	7	0, 1, 2, 3, 4, 5, 6
12	12	4	64	0,0,1,1,1,1,0,0	$Z^3 + X^2Y + X^2Z + X^2Y^2$	5	0, 1, 3, 4, 5
13	13	2	513	1,0,0,0,0,0,0,1	$X^3 + X^2YZ$	4	1, 2, 3, 6
14	14	2	515	1,1,0,0,0,0,0,1	$X^3 + Y^3 + X^2YZ$	2	2, 4
15	15	6	519	1,1,1,0,0,0,0,1	$X^3 + Y^3 + Z^3 + X^2YZ$	4	3, 4, 5, 6
16	16	2	526	0,1,1,1,0,0,0,1	$Y^3 + Z^3 + X^2Y + X^2YZ$	4	0, 3, 4, 6
17	17	2	527	1,1,1,1,0,0,0,1	$X^3 + Y^3 + Z^3 + X^2Y + X^2YZ$	2	5, 6
18	18	6	596	0,0,1,0,1,0,0,1	$Z^3 + X^2Z + Y^2Z + X^2YZ$	6	0, 1, 3, 4, 5, 6
19	19	3	600	0,0,0,1,1,0,1,0	$X^2Y + X^2Z + Y^2Z + X^2YZ$	4	0, 1, 2, 3
20	20	21	607	1,1,1,1,1,0,1,0	$X^3 + Y^3 + Z^3 + X^2Y + X^2Z + Y^2Z + X^2YZ$	0	
END							

The columns are

Row – the row in the table.

OrbIdx – The orbit index (same as Row).

Go – The order of the stabilizer of the variety in the group that was used to compute the orbits.

EqnCode – To Orbiter rank of the equation.

EqnVec – The coefficients of the equation in vector form.

EqnAf – The equation of the variety written in algebraic form.

NbPts – The number of $\mathbb{F}_q$ -rational points of the variety.

Pts – The set of $\mathbb{F}_q$ -rational points of the variety in Orbiter codes.

In the next example, we compute the orbits of the linear group $\text{PGL}(4, 2)$ on homogeneous polynomials of degree 3 in 4 variables:

Code Snippet 377

```

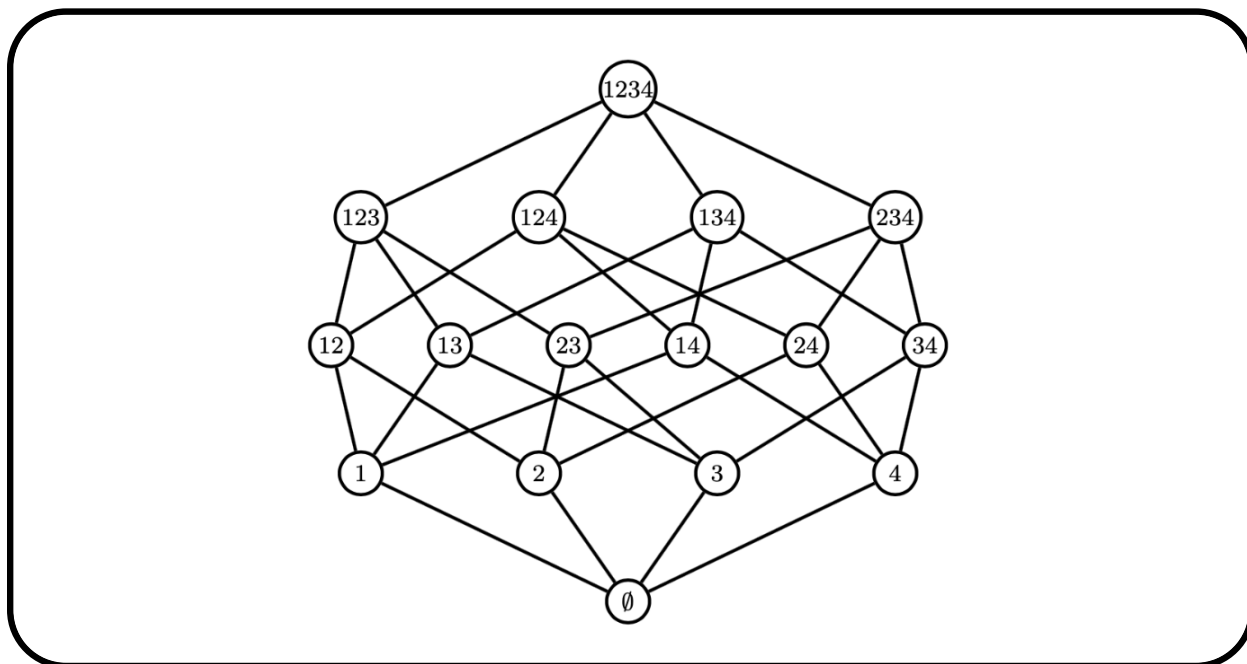
1    poly_orbits_d3_n3_q2.csv:
2    ▷ $(ORBITER) -v 4 \
3    ▷ ▷ -define Do -draw_options \
4    ▷ ▷ ▷ -yout 500000 -radius 15 -nodes_empty \
5    ▷ ▷ ▷ -line_width 0.5 -y_stretch 0.25 -embedded \
6    ▷ ▷ -end \
7    ▷ ▷ -define F -finite_field -q 2 -end \
8    ▷ ▷ -define G -linear_group -PGL 4 F -end \
9    ▷ ▷ -define R -polynomial_ring \
10   ▷ ▷ ▷ -field F \
11   ▷ ▷ ▷ -number_of_variables 4 \
12   ▷ ▷ ▷ -homogeneous_of_degree 3 \
13   ▷ ▷ ▷ -monomial_ordering_partition \
14   ▷ ▷ ▷ -variables "X,Y,Z,W" "X,Y,Z,W" \
15   ▷ ▷ -end \
16   ▷ ▷ -define Orb -orbits -group G \
17   ▷ ▷ ▷ -print_interval 1000000 \
18   ▷ ▷ ▷ -on_polynomials R \
19   ▷ ▷ -end \
20   ▷ ▷ -with Orb -do -orbits_activity \
21   ▷ ▷ ▷ -report Do \
22   ▷ ▷ -end \
23   ▷ ▷ -with Orb -do -orbits_activity \
24   ▷ ▷ ▷ -draw_tree 6 Do \
25   ▷ ▷ -end
26   ▷ $(PDFLATEX) PGL_4_2_orbit_6_tree.tex
27   ▷ $(OPEN) PGL_4_2_orbit_6_tree.pdf
28
```

This command computes the orbits of all cubic forms in 4 variables over the field $\mathbb{F}_2$, confirming the work of Dickson [29] and an enumerative result of Cooley [26].

8.2 Poset Classification

A partially ordered set (poset) is a set together with a partial order. For instance, the set of subsets of a fixed set form an order structure with respect to set-inclusion. The Hasse diagram is a diagram whose nodes represent the element. Nodes are arranged from top to bottom, and relations are indicated by lines. Only the maximal inclusions are drawn. The other relations can be inferred from transitivity.

For example, consider the set of subsets of a 4-element set, say $\{1, 2, 3, 4\}$:



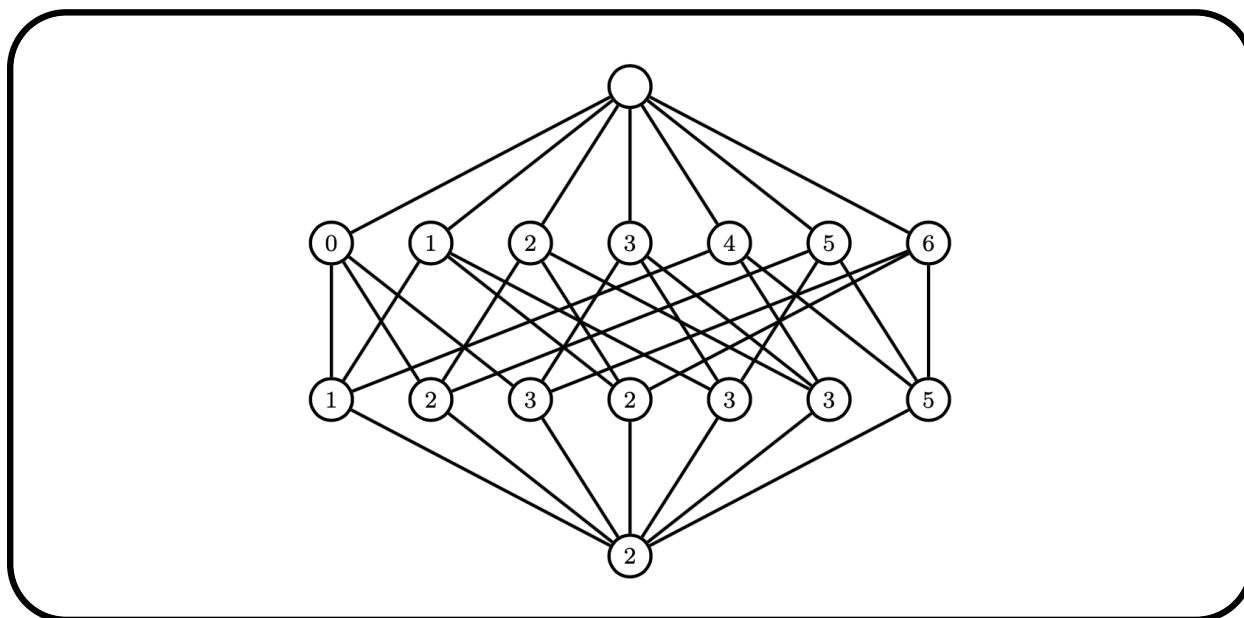
Here, $\{1\}$ is a subset of $\{1, 2\}$, which is a subset of $\{1, 2, 3\}$. There is no line from $\{1\}$ to $\{1, 2, 3\}$ as the relation is not maximal.

Partial orders can be useful in the problem of classifying combinatorial objects by means of substructures. A poset action is group action on a poset. Let G be a group and let $\mathcal{P}$ be a poset. For a poset action, we require that

$$x \leq y \Rightarrow xg \leq yg \quad \text{for all } x, y \in \mathcal{P}, g \in G.$$

For background on poset actions, see Plesken [63]. The orbits of G on $\mathcal{P}$ form another poset, the poset of orbits. A ranked poset can be decomposed into layers according to the rank function. It is then possible to build the poset by considering adjacent levels inductively. In each step, the next layer is constructed from the previous layer and all the layers below. This allows one to reduce a classification problem to a sequence of smaller, and in many cases easier steps.

Orbiter uses the algorithm of Schmalz [67] to perform poset classification. Two versions are available: one for subset-type posets and one for subspace-type posets. The subspace lattice of $V(3, 2) = \mathbb{F}_2^3$ is shown below:



The basis elements are listed, using the enumerator for elements of the projective geometry $PG(2, 2)$ explained in Section 4.1.

The commands listed in Table 8.4 can be used to control the poset classification algorithm.

By default, Orbiter will choose the lexicographically least orbit representatives. It is possible to direct Orbiter to choose different orbit representatives. To this end, the nodes in the orbit tree are labeled. The node number is the zero-based number of a given node in the tree, using the breadth first ordering.

For posets whose orbits have been computed, there are activities as shown in Tables 8.5-8.6.

For reporting purposes, the options in Table 8.7 can be used.

Suppose that Orbiter chooses element a at node n . If we prefer b over a , we can issue the command

```
-preferred_choice n a b
```

It forces Orbiter to choose b instead of a at node n .

Options for Computing Posets		
Command	Arguments	Purpose
-problem_label	str	Use str as a prefix for files that are created.
-path	p	Use path p for files that are created.
-depth	d	Set search depth to d .
-v	v	Set verbosity to v . Larger numbers mean more output.
-gv	v	Set verbosity for group theoretic operations to v . Larger numbers mean more output.
-recover	fname	Recover from the given file.
-lex		Use the lexicographic ordering to speed up the search.
-w		Save orbits at level d only.
-W		Save orbits at all levels.
-write_data_files		Save data to files.
-t		Write a file containing the search tree at level d .
-T		Write a file containing the search tree at all levels.
-draw_options	options	Drawing options according to Table 19.1.
-preferred_choice	$n a b$	At node n , choose b instead of a as orbit representative. This option can be repeated.
-clique_test	graph	Classify cliques in the given graph.
-test_mindist_nonlinear	d	Search for non-linear codes with minimum distance at least d . See Chapter 11.
-test_max_line_intersections		

Table 8.4: Options for Computing Posets

Activities for Posets of Orbits (Part 1)		
Command	Arguments	Purpose
-report	options	Produce a latex report. For options, see Table 8.7. Requires -orbiter_path option from Section 2.1.
-export_level_to_cpp	i	Export level i to cpp file.
-export_history_to_cpp	i	Export history up to level i to cpp file.
-write_tree	options	Create the depth first tree from the poset of orbits. A tree file is created. Uses the given draw options.
-find_node_by_stabilizer_order	k	Search for all nodes whose stabilizer has order k.
-draw_poset	options	Draw the poset of orbits. Uses the given draw options.
-draw_full_poset	options	Draw the poset of orbits in full. Uses the given draw options.
-plesken_ring		Compute the Plesken matrix of the poset of orbits. See [63]. This command returns an object of type Plesken_ring.
-print_data_structure		Print the data structure of the poset of orbits.
-list		List the nodes in the poset of orbits.
-list_all		List all nodes in the poset of orbits.
-table_of_nodes		Produce a table of all nodes in the poset of orbits.
-make_relations_with_flag_orbits		Produce a bitmap drawing of the neighboring relations in the poset with flag orbits.
-level_summary_csv		Write a summary of number of orbits at each level to a csv file.
-orbit_reps_csv		Write orbit representatives to a csv file.
-node_label_is_group_order		When drawing the poset of orbits, display the group order in the orbit nodes.
-node_label_is_element		When drawing the poset of orbits, display the element rank in the orbit nodes.
-show_orbit_decomposition		Show the orbits of the stabilizers on the orbit representatives (in the induced action on the set).
-show_stab		Show the stabilizers.
-save_stab		Save the stabilizer generators.
-show_whole_orbits		Show the whole orbits.
-export_schreier_trees		Export all Schreier trees.
-draw_schreier_trees		Draw all Schreier trees.

Table 8.5: Activities for Posets of Orbits (Part 1)

Activities for Posets of Orbits (Part 2)		
Command	Arguments	Purpose
-test_multi_edge_in_decomposition_matrix		
-recognize	set	Recognizes the given set in the poset of orbits. This option can be repeated.
-pair_relations_within_orbit	orbit	Computes the pairwise join of elements in the given orbit and identifies the orbit in the poset. Two matrices are written. In the first matrix, the ij -entry is the index of the orbit containing the join of orbit element i and orbit element j . In the second matrix, the ij -entry is the orbit index of the join of orbit element i and orbit element j in its respective orbit.
-export_orbits_long		Exports all orbits in long form. For each orbit, each orbit element is listed as a set.

Table 8.6: Activities for Posets of Orbits (Part 2)

Poset Classification Report Options		
Command	Arguments	Purpose
-select_orbits_by_level	level	Select the levels which should be drawn.
-select_orbits_by_stabilizer_order	m	Select the orbits whose stabilizer order is equal to m .
-select_orbits_by_stabilizer_order_multiple_of	m	Select the orbits whose stabilizer order is a multiple of m .
-include_projective_stabilizer		Include the projective stabilizer in the report. This option assumes that the acting group is a collineation group. The projective stabilizer is a subgroup of the collineation stabilizer.
-draw_poset		Include a drawing of the poset of orbits in the report. This option requires one of the following four options to set the type of drawing.
-type_aux		Poset drawing of auxiliary type. The auxiliary poset includes the flag orbits between layers.
-type_ordinary		Poset drawing of ordinary type.
-type_tree		Poset drawing of type spanning tree.
-type_detailed		Poset drawing of type detailed. The detailed drawing includes the flag orbits between layers.
-fname	fname	Set the file name.
-draw_options	label	Set draw options. label must refer to an object of type draw_options, see Table 19.1.

Table 8.7: Poset Classification Report Options

8.3 Orbits on Subsets

The lattice of subsets of a set X is $\mathfrak{P}(X)$, the set of all subsets of X , ordered with respect to inclusion. Assume that a group G acts on X , and hence on the lattice by means of the induced action on subsets. The orbits of G on subsets form a new poset, the poset of orbits. Poset classification is the process of computing the poset of orbits. Orbiter has an algorithm to perform poset classification. In many cases, we are not interested in the full lattice of subsets $\mathfrak{P}(X)$ but rather in a subposet of it. We require that the subposet is closed under the group action and that the following property holds:

$$x, y \in \mathfrak{P}(X) \text{ and } x \leq y \Rightarrow (y \in \mathcal{P} \rightarrow x \in \mathcal{P}).$$

The join of two subsets in the poset may or may not belong to the poset.

Consider the action of the group $G = C_6$ on the set $X = \{0, 1, \dots, 5\}$. To compute the orbits of G on subsets of X , we use the following command.

Code Snippet 378

```

1   poset_of_subsets_under_C6:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4   ▷ ▷ -define Control -poset_classification_control \
5   ▷ ▷ ▷ -problem_label poset_C6 \
6   ▷ ▷ ▷ -W -depth 6 \
7   ▷ ▷ -end \
8   ▷ ▷ -define Do -draw_options -radius 150 \
9   ▷ ▷ -end \
10  ▷ ▷ -define DoA -draw_options \
11  ▷ ▷ ▷ -radius 200 -line_width 1.0 \
12  ▷ ▷ ▷ -poset_orbits_x_stretch 750 \
13  ▷ ▷ ▷ -embedded \
14  ▷ ▷ ▷ -poset_with_horizontal_lines \
15  ▷ ▷ -end \
16  ▷ ▷ -define G -permutation_group -cyclic_group 6 -end \
17  ▷ ▷ -define Orb -orbits \
18  ▷ ▷ ▷ -group G \
19  ▷ ▷ ▷ -group_action G \
20  ▷ ▷ ▷ -generators G \
21  ▷ ▷ ▷ -on_subsets 6 Control \
22  ▷ ▷ -end \
23  ▷ ▷ -define ro \
24  ▷ ▷ ▷ -poset_classification_report_options \
25  ▷ ▷ ▷ ▷ -draw_poset -type_detailed \
26  ▷ ▷ ▷ -draw_options Do \
27  ▷ ▷ -end \
28  ▷ ▷ -with Orb -do -orbits_activity \
29  ▷ ▷ ▷ -report "" \
30  ▷ ▷ ▷ -report_options ro \
31  ▷ ▷ -end \
32  ▷ ▷ -define PCActivity -poset_classification_activity \
33  ▷ ▷ ▷ -draw_full_poset DoA \
34  ▷ ▷ -end \
35  ▷ ▷ -with Orb -do -orbits_activity \
36  ▷ ▷ ▷ -poset_classification_activity PCActivity \
37  ▷ ▷ -end

```

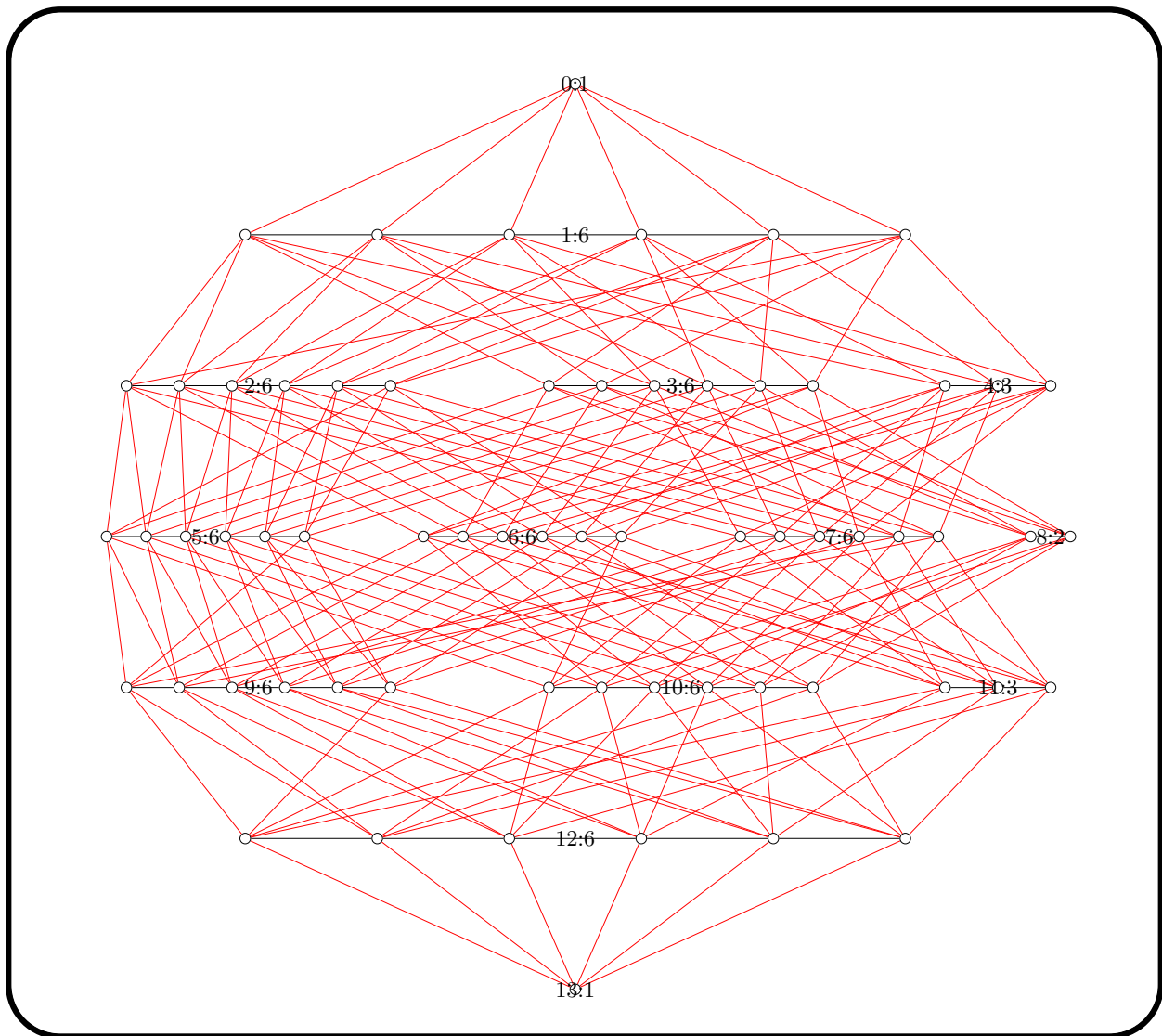


```

38 ▷ $(ORBITER) -v 3 \
39 ▷ ▷ -draw_factor_poset \
40 ▷ ▷ ▷ poset_C6_poset_full_lvl_6.factor_poset \
41 ▷ ▷ -radius 50 -embedded \
42 ▷ ▷ ▷ -line_width 0.2 -x_stretch 1.4 -scale 0.5 \
43 ▷ ▷ -end
44 ▷ pdflatex poset_C6_poset_full_lvl_6.draw.tex
45 ▷ open poset_C6_poset_full_lvl_6.draw.pdf
46

```

The poset of orbits is shown below:



The orbits are numbered from 0 to 13. The elements of one orbit are drawn in groups, indicated by a horizontal line. The orbits are labeled. The label of any orbit has the form $a : b$ where a is the number of the orbit and b is the orbit length. The orbit numbered 0 is the bottom element of the poset, corresponding to the empty set. The empty set forms an orbit of length 1.

Next we consider the induced action of $\text{Sym}(4)$ on the set of unordered pairs. We use the following command

to compute the orbits on subsets. These orbits are in one-to-one correspondence to the isomorphism types of simple graphs of 4 vertices.

Code Snippet 379

```

1  orbits_on_subsets_in_action_on_pairs:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4  ▷ ▷ -define Do -draw_options \
5  ▷ ▷ ▷ -radius 200 -embedded \
6  ▷ ▷ -end \
7  ▷ ▷ -define DoA -draw_options \
8  ▷ ▷ ▷ -radius 200 -line_width 1.0 \
9  ▷ ▷ ▷ -poset_orbits_x_stretch 750 \
10 ▷ ▷ ▷ -embedded \
11 ▷ ▷ ▷ -poset_with_horizontal_lines \
12 ▷ ▷ -end \
13 ▷ ▷ -define Control -poset_classification_control \
14 ▷ ▷ ▷ -problem_label graphs4 \
15 ▷ ▷ ▷ -depth 6 \
16 ▷ ▷ ▷ -draw_options Do \
17 ▷ ▷ -end \
18 ▷ ▷ -define G -permutation_group -symmetric_group 4 -end \
19 ▷ ▷ -define G_on_2subsets -modified_group -from G \
20 ▷ ▷ ▷ -on_k_subsets 2 \
21 ▷ ▷ -end \
22 ▷ ▷ -define Orb -orbits \
23 ▷ ▷ ▷ -group G \
24 ▷ ▷ ▷ -group_action G_on_2subsets \
25 ▷ ▷ ▷ -generators G \
26 ▷ ▷ ▷ -on_subsets 6 Control \
27 ▷ ▷ -end \
28 ▷ ▷ -define ro \
29 ▷ ▷ ▷ -poset_classification_report_options \
30 ▷ ▷ ▷ ▷ -draw_poset -type_detailed \
31 ▷ ▷ ▷ ▷ -draw_options Do \
32 ▷ ▷ -end \
33 ▷ ▷ -with Orb -do -orbits_activity \
34 ▷ ▷ ▷ -report "" \
35 ▷ ▷ ▷ -report_options ro \
36 ▷ ▷ -end \
37 ▷ ▷ -define PCActivity -poset_classification_activity \
38 ▷ ▷ ▷ -draw_full_poset DoA \
39 ▷ ▷ -end \
40 ▷ ▷ -with Orb -do -orbits_activity \
41 ▷ ▷ ▷ -poset_classification_activity PCActivity \
42 ▷ ▷ -end \
43 ▷ ▷ -with G_on_2subsets -do \
44 ▷ ▷ -group_theoretic_activity \
45 ▷ ▷ ▷ -stats "graphs4" \
46 ▷ ▷ -end
47 ▷ $(ORBITER) -v 3 \
48 ▷ ▷ -draw_factor_poset \

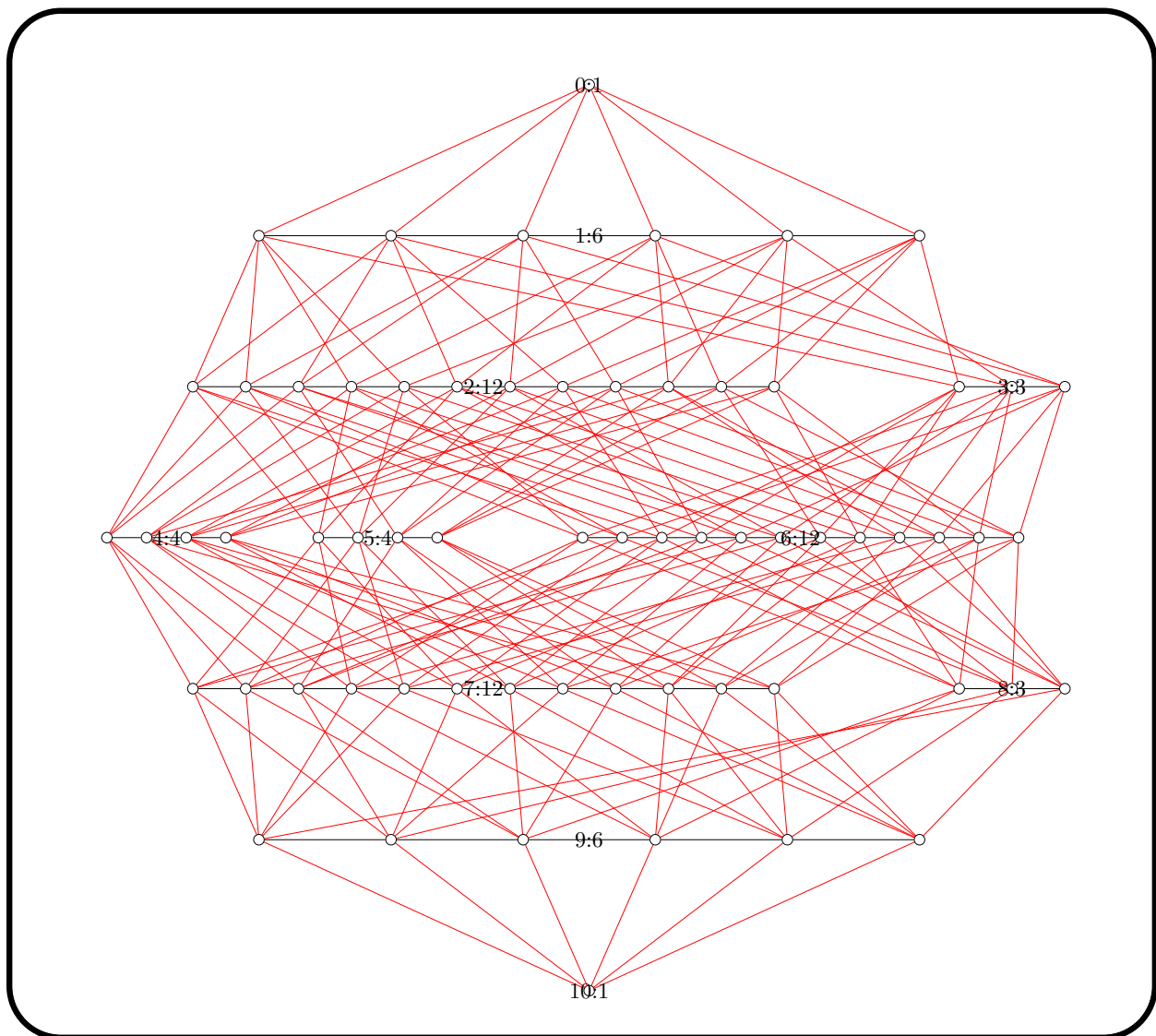
```

```

49  ▷ ▷ ▷ graphs4_poset_full_lvl_6.factor_poset \
50  ▷ ▷ -radius 50 -embedded \
51  ▷ ▷ ▷ -line_width 0.2 -x_stretch 1.4 -scale 0.5 \
52  ▷ ▷ -end
53  ▷ pdflatex graphs4_poset_full_lvl_6_draw.tex
54  ▷ open graphs4_poset_full_lvl_6_draw.pdf
55

```

The poset of orbits is shown below:



Let us consider the action of the Singer cycle on $\text{PG}(3, 2)$. The following command computes the orbits of the group G generated by a Singer cycle in $\text{PG}(3, 2)$:

Code Snippet 380

```

1   PGL_3.2.singer:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4   ▷ ▷ -define Do -draw_options \
5   ▷ ▷ ▷ -radius 200 -embedded \
6   ▷ ▷ -end \
7   ▷ ▷ -define Control -poset_classification_control \
8   ▷ ▷ ▷ -problem_label PGL_3.2.singer_1 \
9   ▷ ▷ ▷ -W -depth 7 \
10  ▷ ▷ ▷ -draw_options Do \
11  ▷ ▷ -end \
12  ▷ ▷ -define G -linear_group -PGL 3 2 -singer 1 -end \
13  ▷ ▷ -define Orb -orbits \
14  ▷ ▷ ▷ -group G \
15  ▷ ▷ ▷ -group_action G \
16  ▷ ▷ ▷ -generators G \
17  ▷ ▷ ▷ -on_subsets 7 Control \
18  ▷ ▷ -end \
19  ▷ ▷ -define ro \
20  ▷ ▷ ▷ -poset_classification_report_options \
21  ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
22  ▷ ▷ ▷ ▷ -draw_options Do \
23  ▷ ▷ -end \
24  ▷ ▷ -with Orb -do -orbits_activity \
25  ▷ ▷ ▷ -report "" \
26  ▷ ▷ ▷ -report_options ro \
27  ▷ ▷ -end
28  ▷ $(PDFLATEX) PGL_3.2.singer_1_poset.tex
29  ▷ $(OPEN) PGL_3.2.singer_1_poset.pdf
30

```

The next command computes the orbits of the projective group $\text{PGL}(4, 2)$ acting on all subsets of $\text{PG}(3, 2)$:

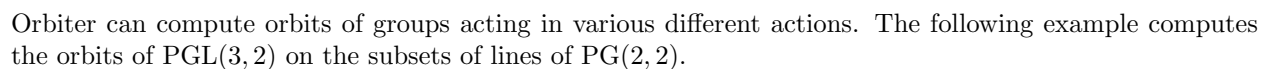
Code Snippet 381

```

1   PG_3.2.subsets:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4   ▷ ▷ -define Do -draw_options \
5   ▷ ▷ ▷ -radius 200 -embedded \
6   ▷ ▷ -end \
7   ▷ ▷ -define Control -poset_classification_control \
8   ▷ ▷ ▷ -problem_label PGL_4.2 \
9   ▷ ▷ ▷ -depth 15 \
10  ▷ ▷ ▷ -draw_options Do \
11  ▷ ▷ -end \
12  ▷ ▷ -define F -finite_field -q 2 -end \
13  ▷ ▷ -define G -linear_group -PGL 4 F -end \
14  ▷ ▷ -define Orb -orbits \
15  ▷ ▷ ▷ -group G \

```

A diagram showing the poset of orbits is produced:



Code Snippet 382

```

1   PGL_3.2_on_lines:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4   ▷ ▷ -define Do -draw_options \
5   ▷ ▷ ▷ -radius 200 -embedded \
6   ▷ ▷ -end \
7   ▷ ▷ -define Control -poset_classification_control \
8   ▷ ▷ ▷ -problem_label PGL_3.2_lines \
9   ▷ ▷ ▷ -W -depth 7 \
10  ▷ ▷ -end \
11  ▷ ▷ -define G -linear_group -PGL 3 2 -end \
12  ▷ ▷ -define G_on_lines -modified_group -from G \
13  ▷ ▷ ▷ -on_k_subspaces 2 \
14  ▷ ▷ -end \
15  ▷ ▷ -define Orb -orbits \
16  ▷ ▷ ▷ -group G \
17  ▷ ▷ ▷ -group_action G_on_lines \
18  ▷ ▷ ▷ -generators G \
19  ▷ ▷ ▷ -on_subsets 7 Control \
20  ▷ ▷ -end \
21  ▷ ▷ -define ro \
22  ▷ ▷ ▷ -poset_classification_report_options \
23  ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
24  ▷ ▷ ▷ ▷ -draw_options Do \
25  ▷ ▷ -end \
26  ▷ ▷ -with Orb -do -orbits_activity \
27  ▷ ▷ ▷ -report "" \
28  ▷ ▷ ▷ -report_options ro \
29  ▷ ▷ -end
30  ▷ $(PDFLATEX) PGL_3.2_lines_poset.tex
31  ▷ $(OPEN) PGL_3.2_lines_poset.pdf
32

```

The following example computes the orbits of $\text{PGO}(5, 2)$ on the power set lattice of points of $Q(4, 2)$:

Code Snippet 383

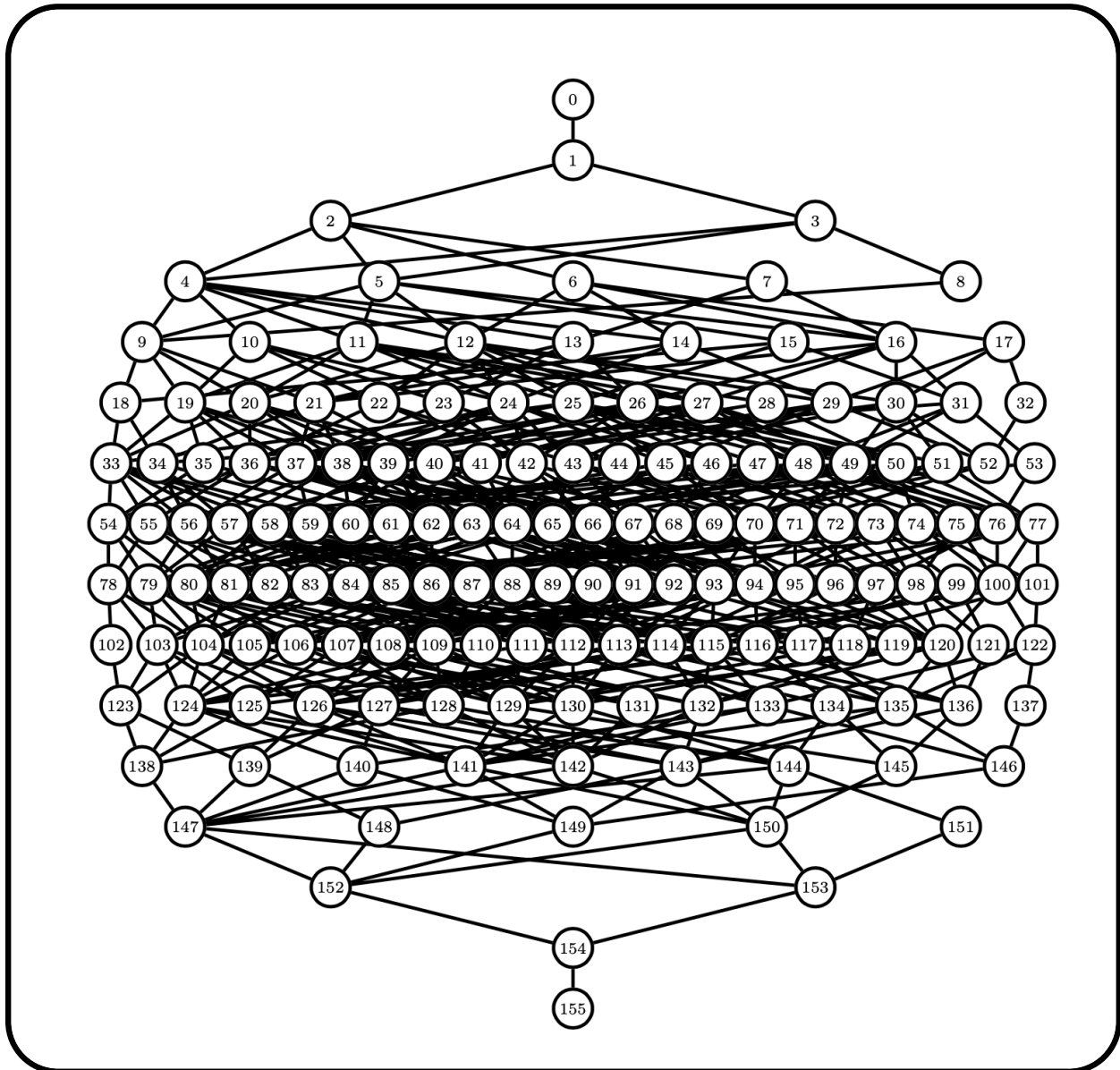
```

1   PGO_5.2_on_subsets:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4   ▷ ▷ -define Do -draw_options \
5   ▷ ▷ ▷ -radius 200 -embedded \
6   ▷ ▷ -end \
7   ▷ ▷ -define Control -poset_classification_control \
8   ▷ ▷ ▷ -problem_label PGO_5.2 \
9   ▷ ▷ ▷ -depth 15 \
10  ▷ ▷ ▷ -w \
11  ▷ ▷ -end \
12  ▷ ▷ -define F -finite_field -q 2 -end \
13  ▷ ▷ -define G -linear_group -PGO 5 F -end \

```

```
14  ▷ ▷ -define Orb -orbits \  
15  ▷ ▷ ▷ -group G \  
16  ▷ ▷ ▷ -group_action G \  
17  ▷ ▷ ▷ -generators G \  
18  ▷ ▷ ▷ -on_subsets 15 Control \  
19  ▷ ▷ -end \  
20  ▷ ▷ -define ro \  
21  ▷ ▷ ▷ -poset_classification_report_options \  
22  ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \  
23  ▷ ▷ ▷ ▷ -draw_options Do \  
24  ▷ ▷ -end \  
25  ▷ ▷ -with Orb -do -orbits_activity \  
26  ▷ ▷ ▷ -report "" \  
27  ▷ ▷ ▷ -report_options ro \  
28  ▷ ▷ -end  
29  ▷ $(PDFLATEX) PG0_5_2_poset.tex  
30  ▷ $(OPEN) PG0_5_2_poset.pdf  
31
```

The poset of orbits is shown below:



8.4 Orbits on Subspaces

Orbiter can compute the orbits of a group on the lattice of subspaces of a finite vector space.

The orthogonal group is the stabilizer of a non-degenerate quadric. Suppose we want to classify the subspaces in $\text{PG}(3, 2)$ under the action of the orthogonal group. In $\text{PG}(3, 2)$, there are two distinct nondegenerate quadrics, $Q^+(3, 2)$ and $Q^-(3, 2)$. The $Q^+(3, 2)$ quadric is given by the equation

$$x_0x_1 + x_2x_3 = 0.$$

The stabilizer of the quadric is the group $\text{PGO}^+(4, 2)$. Suppose we want to classify the orbits of this group on the underlying space $\text{PG}(3, 2)$. The command

Code Snippet 384

```

1   subspaces_Op_4_2:
2   ▷ $(ORBITER) -v 10 \
3   ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4   ▷ ▷ -define Do -draw_options \
5   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6   ▷ ▷ ▷ -poset_orbits_x_stretch 750 \
7   ▷ ▷ ▷ -embedded \
8   ▷ ▷ -end \
9   ▷ ▷ -define Control -poset_classification_control \
10  ▷ ▷ ▷ -problem_label Op_4_2 -W -depth 4 \
11  ▷ ▷ -end \
12  ▷ ▷ -define G -linear_group -PGL 4 2 -orthogonal 1 -end \
13  ▷ ▷ -define Orb -orbits \
14  ▷ ▷ ▷ -group G \
15  ▷ ▷ ▷ -group_action G \
16  ▷ ▷ ▷ -generators G \
17  ▷ ▷ ▷ -on_subspaces 4 Control \
18  ▷ ▷ -end \
19  ▷ ▷ -define ro \
20  ▷ ▷ ▷ -poset_classification_report_options \
21  ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
22  ▷ ▷ ▷ -draw_options Do \
23  ▷ ▷ -end \
24  ▷ ▷ -with Orb -do -orbits_activity \
25  ▷ ▷ ▷ -report "" \
26  ▷ ▷ ▷ -report_options ro \
27  ▷ ▷ -end \
28  ▷ ▷ -define PC_Activity -poset_classification_activity \
29  ▷ ▷ ▷ -draw_full_poset Do \
30  ▷ ▷ -end \
31  ▷ ▷ -with Orb -do -orbits_activity \
32  ▷ ▷ ▷ -poset_classification_activity PC_Activity \
33  ▷ ▷ -end
34  ▷ $(ORBITER) -v 3 \
35  ▷ ▷ -draw_layered_graph \
36  ▷ ▷ ▷ Op_4_2_poset_full_lvl_4.layered_graph \
37  ▷ ▷ -radius 150 -embedded \
38  ▷ ▷ ▷ -line_width 0.2 -x_stretch 1.4 -scale 0.5 \
39  ▷ ▷ -end
40  ▷ $(PDFLATEX) Op_4_2_poset_full_lvl_4.draw.tex

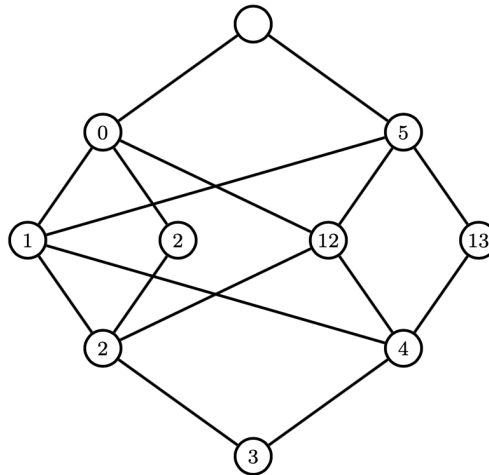
```

```

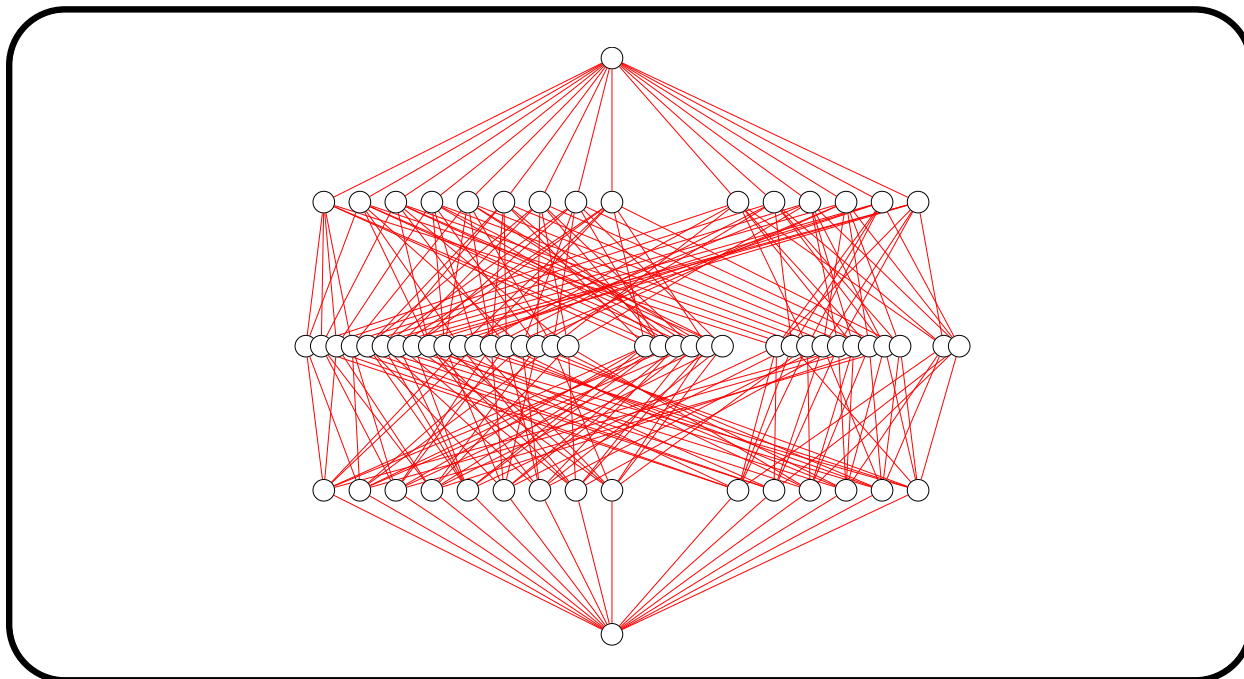
41   ▷ $(OPEN) Op_4_2_poset_full_lvl_4_draw.pdf
42   ▷ $(PDFLATEX) Op_4_2_poset.tex
43   ▷ $(OPEN) Op_4_2_poset.pdf
44   ▷ $$ (PDFLATEX) Op_4_2_poset_full_lvl_4.tex
45   ▷ $$ (OPEN) Op_4_2_poset_full_lvl_4.pdf
46   ▷ $$ (PDFLATEX) Op_4_2_poset_full_lvl_4_draw.tex
47   ▷ $$ (OPEN) Op_4_2_poset_full_lvl_4_draw.pdf
48

```

produces a classification of all subspaces of $\text{PG}(3, 2)$ under $\text{PGO}^+(4, 2)$. The poset of orbits is shown below. Each node represents one orbit on subspaces:



The nodes show the ranks of points in $\text{PG}(3, 2)$ as described in Section 4.1. It is also possible to expand the poset to the level of orbit elements. The command also produces a detailed view of the poset, showing every subspace. Orbits of subspaces are indicated by grouping the corresponding nodes at a horizontal level:



Suppose we want to compute the orbits of the projective group on the subspaces of the exterior algebra. The following command creates $\text{PGL}(5, 3)$ in the wedge action as in Section 5.4. It then computes the orbits of subspaces of the exterior algebra of dimension 5. Because of the nature of the poset classification algorithm, all orbits of dimension i with $0 \leq i \leq 5$ are computed.

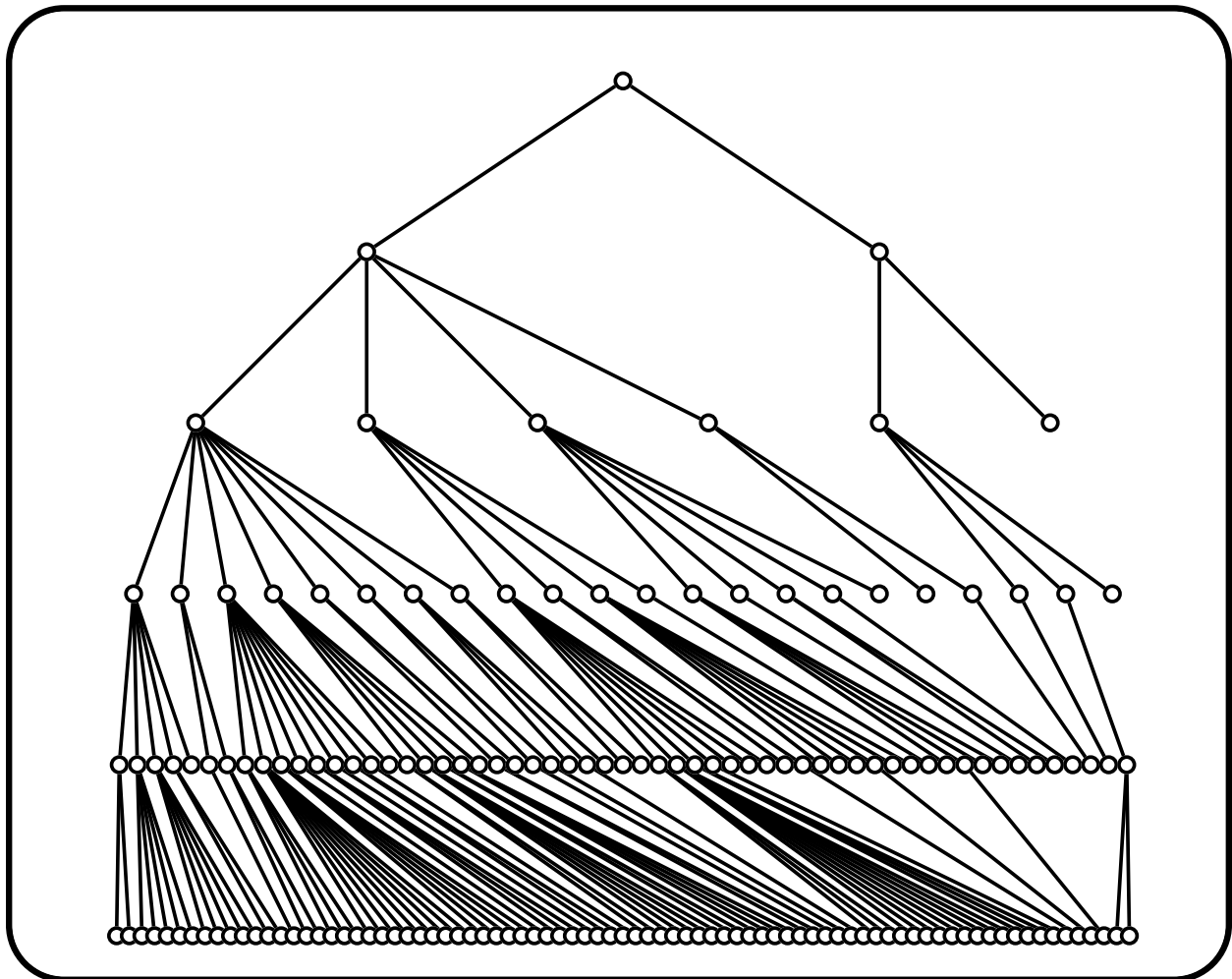
Code Snippet 385

```

1   PGL_5_3_wedge_subspace_orbits:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 75 -embedded -nodes_empty \
5   ▷ ▷ -end \
6   ▷ ▷ -define G -linear_group -PGL 5 3 -end \
7   ▷ ▷ -define G2 -modified_group -from G \
8   ▷ ▷ ▷ -on_wedge_product \
9   ▷ ▷ -end \
10  ▷ ▷ -define Control -poset_classification_control \
11  ▷ ▷ ▷ -problem_label PGL_5_3_wedge -W -depth 5 \
12  ▷ ▷ -end \
13  ▷ ▷ -define Orb -orbits \
14  ▷ ▷ ▷ -group G \
15  ▷ ▷ ▷ -group_action G2 \
16  ▷ ▷ ▷ -generators G \
17  ▷ ▷ ▷ -on_subspaces 5 Control \
18  ▷ ▷ -end \
19  ▷ ▷ -with Orb -do -orbits_activity \
20  ▷ ▷ ▷ -draw_tree 0 Do \
21  ▷ ▷ -end
22  ▷ $(PDFLATEX) PGL_5_3_wedge_tree_lvl_5.tex
23  ▷ $(OPEN) PGL_5_3_wedge_tree_lvl_5.pdf
24

```

The computation finds 81 orbits at level 5. The lex-least spanning tree for the orbits is shown below:



8.5 Orbits on Set-Partitions

Orbiter can compute the orbits of a group on set-partitions. The set-partition needs to have three parts of equal size.

The command

Code Snippet 386

```

1  C6_on_partition:
2  ▷ $(ORBITER) -v 5 \
3  ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4  ▷ ▷ -define Control -poset_classification_control \
5  ▷ ▷ ▷ -problem_label C6 \
6  ▷ ▷ ▷ -depth 2 \
7  ▷ ▷ ▷ -W \
8  ▷ ▷ -end \
9  ▷ ▷ -define G -permutation_group -cyclic_group 6 -end \
10 ▷ ▷ -define Orb -orbits \
11 ▷ ▷ ▷ -group G \
12 ▷ ▷ ▷ -group_action G \
13 ▷ ▷ ▷ -generators G \
14 ▷ ▷ ▷ -on_partition 2 Control \
15 ▷ ▷ -end
16
```

computes the orbits of the cyclic group C_6 on set-partitions of type $2 + 2 + 2$. There are 15 set-partitions, and they fall into 5 orbits, with stabilizer orders

$$3, 1, 2, 2, 6.$$

The orbit count gives

$$6\left(\frac{1}{3} + \frac{1}{1} + \frac{1}{2} + \frac{1}{2} + \frac{1}{6}\right) = 15.$$

The command

Code Snippet 387

```

1  PGL_2_17_on_partition:
2  ▷ $(ORBITER) -v 5 \
3  ▷ ▷ -define Control -poset_classification_control \
4  ▷ ▷ ▷ -problem_label PGL_2_17 \
5  ▷ ▷ ▷ -depth 6 \
6  ▷ ▷ ▷ -W \
7  ▷ ▷ -end \
8  ▷ ▷ -define G -linear_group -PGL 2 17 -end \
9  ▷ ▷ -define Orb -orbits \
10 ▷ ▷ ▷ -group G \
11 ▷ ▷ ▷ -group_action G \
12 ▷ ▷ ▷ -generators G \
13 ▷ ▷ ▷ -on_partition 6 Control \
14 ▷ ▷ -end
15
```

computes the orbits of the group $\text{PGL}(2, 17)$ on set-partitions of type $6+6+6$. The number of set-partitions is

$$\frac{\binom{18}{6} \cdot \binom{12}{6}}{3!} = 2858856$$

There are 720 orbits. The orbit stabilizer statistic is

$$(1^{480}, 2^{184}, 3^{11}, 4^{20}, 6^{15}, 8, 12^6, 18, 24, 36).$$

The orbit-stabilizer count confirms that

$$4896 \left(\frac{480}{1} + \frac{184}{2} + \frac{11}{3} + \frac{20}{4} + \frac{15}{6} + \frac{1}{8} + \frac{6}{12} + \frac{1}{18} + \frac{1}{24} + \frac{1}{36} + \right) = 2858856.$$

Classifying Arcs		
Command	Arguments	Purpose
-control	label	Set the label of the poset classification control object.
-projective_space	P	Set the projective space to P.
-d	d	Require that no more than d points lie on a line.
-target_size	t	Specify the size of the arc to be t .
-conic_test		Require that no 6 points of the arc lie on a conic.
-test_nb_Eckardt_points	n	Require exactly n Eckardt points.
-affine		Classify arcs in the affine geometry, assuming that $x_0 = 0$ is the hyperplane at infinity. The condition that no more that d point lie on a line applies to affine lines only.
-no_arc_testing		Do not test the “at most d points per line” condition.
-forbidden_point_set	set	The arc must not contain any of the given points.
-override_group	label	Override the group under which we classify.

Table 8.8: Classifying Arcs

8.6 Arcs and Caps in Projective Spaces

In $\text{PG}(n, q)$, an arc is a set of points, no $n+1$ in a hyperplane. A cap is set of points, no three collinear. Here, we restrict our attention to arcs in $\text{PG}(2, q)$. Arcs in higher dimensional projective spaces are equivalent to MDS codes and will be treated in Section 11. Our main examples will be the construction of the Lunelli-Sce hyperoval in $\text{PG}(2, 16)$ (cf. [54]) and the Pellegrino cap in $\text{AG}(4, 3)$. The uniqueness of this cap was proven by Hill [35].

A (k, d) -arc in a projective plane π is a set S of k points such that any line intersects S in at most d points. Two arcs S_1 and S_2 are equivalent if there is a projectivity Φ such that $\Phi(A) = B$. The problem of classifying arcs is the problem of determining the orbits of the projectivity group on arcs. At times, we consider the larger group of collineations. In that case, the problem of classifying arcs is the problem of determining the orbits of the collineation group on arcs. Table 8.8 list the commands available to classify arcs. Here is an example. A hyperoval in a finite plane $\text{PG}(2, 2^e)$ is a $(2^e + 2, 2)$ -arc. It is an interesting but difficult problem to classify the hyperovals in finite projective planes under the appropriate group action. For small planes, computer search with Orbiter can help. The command

Code Snippet 388

```

1  hyperoval_16_classify:
2  ▷ $(ORBITER) -v 4 \
3  ▷ ▷ -orbiter_path $(ORBITER.EXE_PATH) \
4  ▷ ▷ -define F -finite_field -q 16 -end \
5  ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
6  ▷ ▷ -define Control -poset_classification_control \
7  ▷ ▷ ▷ -problem_label hyperoval.q16 \
8  ▷ ▷ ▷ -W -depth 18 \

```

```

9    ▷ ▷ -end \
10   ▷ ▷ -define Arc_control -arc_generator_control \
11   ▷ ▷ ▷ -projective_space P \
12   ▷ ▷ ▷ -control Control \
13   ▷ ▷ ▷ -target_size 18 \
14   ▷ ▷ ▷ -d 2 \
15   ▷ ▷ -end \
16   ▷ ▷ -define Orb -orbits \
17   ▷ ▷ ▷ -on_arcs Arc_control \
18   ▷ ▷ -end
19

```

performs the classification of hyperovals in $PG(2, 16)$. There are exactly two hyperovals in this plane. Orbiter also finds the stabilizers of these arcs. They have orders 16320 and 144, respectively. The two hyperovals are the regular hyperoval and the Lunelli-Sce hyperoval. Here is the relevant output from the Orbiter report (in the output, the Lunelli-Sce hyperoval is orbit 0, and the regular hyperoval is orbit 1):

Orbits at Level 18

There are 2 orbits at level 18.

Orbit 0 / 2 at Level 18

Node number: 4212

$\{0, 1, 2, 3, 52, 67, 89, 106, 126, 141, 159, 176, 184, 199, 220, 235, 245, 262\}_{144}$

0 : 0 = (1, 0, 0)	10 : 159 = (14, 8, 1)
1 : 1 = (0, 1, 0)	11 : 176 = (15, 9, 1)
2 : 2 = (0, 0, 1)	12 : 184 = (7, 10, 1)
3 : 3 = (1, 1, 1)	13 : 199 = (6, 11, 1)
4 : 52 = (3, 2, 1)	14 : 220 = (11, 12, 1)
5 : 67 = (2, 3, 1)	15 : 235 = (10, 13, 1)
6 : 89 = (8, 4, 1)	16 : 245 = (4, 14, 1)
7 : 106 = (9, 5, 1)	17 : 262 = (5, 15, 1)
8 : 126 = (13, 6, 1)	
9 : 141 = (12, 7, 1)	

Strong generators for a group of order 144:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ \delta^4 & \delta^9 & 1 \end{bmatrix}_1, \begin{bmatrix} 1 & \delta^7 & \delta^{13} \\ \delta^8 & \delta^9 & \delta^{10} \\ \delta & \delta^6 & 1 \end{bmatrix}_3, \begin{bmatrix} \delta^5 & \delta^5 & \delta^5 \\ \delta^5 & \delta^2 & \delta^{11} \\ \delta^5 & \delta^{14} & 1 \end{bmatrix}_0$$

1,0,0,0,1,0,9,5,1,1,

1,7,6,14,5,10,2,15,1,3,

1,1,1,1,3,15,1,5,10,0,

There are 0 extensions

Number of generators 3

Orbit 1 / 2 at Level 18

Node number: 4213

$$\{0, 1, 2, 3, 52, 70, 83, 109, 127, 139, 156, 174, 186, 199, 217, 229, 256, 264\}_{16320}$$

$$\begin{array}{ll} 0 : 0 = (1, 0, 0) & 10 : 156 = (11, 8, 1) \\ 1 : 1 = (0, 1, 0) & 11 : 174 = (13, 9, 1) \\ 2 : 2 = (0, 0, 1) & 12 : 186 = (9, 10, 1) \\ 3 : 3 = (1, 1, 1) & 13 : 199 = (6, 11, 1) \\ 4 : 52 = (3, 2, 1) & 14 : 217 = (8, 12, 1) \\ 5 : 70 = (5, 3, 1) & 15 : 229 = (4, 13, 1) \\ 6 : 83 = (2, 4, 1) & 16 : 256 = (15, 14, 1) \\ 7 : 109 = (12, 5, 1) & 17 : 264 = (7, 15, 1) \\ 8 : 127 = (14, 6, 1) & \\ 9 : 139 = (10, 7, 1) & \end{array}$$

Strong generators for a group of order 16320:

$$\begin{aligned} & \begin{bmatrix} \delta^6 & 0 & 0 \\ 0 & \delta^3 & 0 \\ 0 & 0 & 1 \end{bmatrix}_2, \begin{bmatrix} \delta^9 & 0 & 0 \\ 0 & \delta^7 & 0 \\ 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} \delta^2 & 0 & 0 \\ 0 & \delta^{11} & 0 \\ \delta^4 & \delta^7 & 1 \end{bmatrix}_3, \\ & \begin{bmatrix} \delta^{10} & 0 & 0 \\ 0 & \delta^3 & 0 \\ \delta & \delta^{11} & 1 \end{bmatrix}_3, \begin{bmatrix} \delta & 0 & 0 \\ \delta^{12} & \delta^2 & \delta^5 \\ \delta^{14} & \delta^{10} & 1 \end{bmatrix}_1, \begin{bmatrix} \delta^5 & 0 & 0 \\ \delta & \delta & \delta \\ \delta^6 & \delta^8 & 1 \end{bmatrix}_0, \\ & \begin{bmatrix} \delta^{12} & 1 & \delta^2 \\ \delta^4 & \delta^3 & \delta^7 \\ \delta^6 & \delta^3 & 1 \end{bmatrix}_2, \begin{bmatrix} \delta^5 & \delta^3 & \delta^6 \\ \delta^{11} & \delta^6 & \delta^{10} \\ \delta^{10} & \delta^6 & 1 \end{bmatrix}_3 \end{aligned}$$

1,0,0,0,3,0,0,0,5,2,
1,0,0,0,6,0,0,0,15,1,
1,0,0,0,5,0,4,11,6,3,
1,0,0,0,14,0,15,2,11,3,
1,0,0,13,2,9,6,5,12,1,
1,0,0,13,13,13,2,8,10,0,
1,8,11,7,15,10,5,15,8,2,
1,6,2,15,2,11,11,2,10,3,
There are 0 extensions
Number of generators 8

In the theory of cubic surfaces, we are interested in non-conical arcs of size 6. Here, non-conical means that the set of points does not lie on a conic. Cubic surfaces are associated with non-conical arcs of size 6 (in a many-to-one relationship when considering isomorphism classes). The following example demonstrates how non-conical 6-arcs can be classified in Orbiter:

Code Snippet 389

```

1   nc_arcs_16:
2   ▷ $(ORBITER) -v 4 \
3   ▷ ▷ -define F -finite_field -q 16 -end \
4   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5   ▷ ▷ -define Control -poset_classification_control \
6   ▷ ▷ ▷ -problem_label nc_arcs.q16.d2 \
7   ▷ ▷ ▷ -W -depth 6 \
8   ▷ ▷ -end \
9   ▷ ▷ -define Arc_control -arc_generator_control \
10  ▷ ▷ ▷ -projective_space P \
11  ▷ ▷ ▷ -control Control \
12  ▷ ▷ ▷ -target_size 6 \
13  ▷ ▷ ▷ -d 2 \
14  ▷ ▷ -end \
15  ▷ ▷ -define Orb -orbits \
16  ▷ ▷ ▷ -on_arcs Arc_control \
17  ▷ ▷ -end
18  ▷ #$(PDFLATEX) nc_arcs.q16.d2.poset.tex
19  ▷ #$(OPEN) nc_arcs.q16.d2.poset.pdf
20

```

The number of Eckardt points of the surface can be recovered from properties of the arc. For this reason, it is interesting to classify arcs so that the associated cubic surface has a fixed number of Eckardt points.

8.7 Cubic Curves

Orbiter can classify cubic curves in $\text{PG}(2, q)$. To this end, the $(9, 3)$ -arcs in $\text{PG}(2, q)$ are classified first. From this classification, the classification of curves is computed. This classification only works for arcs which contain a $(9, 3)$ arc. For very small fields, this is not always the case. Here is an example. The command

Code Snippet 390

```

1    cubic_curves_PG.2.4:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4    ▷ ▷ -define Do -draw_options \
5    ▷ ▷ ▷ -radius 200 -embedded \
6    ▷ ▷ -end \
7    ▷ ▷ -define F -finite_field -q 4 -end \
8    ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
9    ▷ ▷ -define Control -poset_classification_control \
10   ▷ ▷ ▷ -problem_label cc_4 -W -depth 9 \
11   ▷ ▷ -end \
12   ▷ ▷ -define Arc_control -arc_generator_control \
13   ▷ ▷ ▷ -projective_space P \
14   ▷ ▷ ▷ -control Control \
15   ▷ ▷ ▷ -target_size 9 \
16   ▷ ▷ ▷ -d 3 \
17   ▷ ▷ -end \
18   ▷ ▷ -define Orb -orbits \
19   ▷ ▷ ▷ -on_cubic_curves Arc_control \
20   ▷ ▷ -end \
21   ▷ ▷ -define ro \
22   ▷ ▷ ▷ -poset_classification_report_options \
23   ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
24   ▷ ▷ ▷ -draw_options Do \
25   ▷ ▷ -end \
26   ▷ ▷ -with Orb -do -orbits_activity \
27   ▷ ▷ ▷ -report "" \
28   ▷ ▷ ▷ -report_options ro \
29   ▷ ▷ -end
30   ▷ ▷ -with Orb -do -orbits_activity \
31   ▷ ▷ ▷ -draw_tree 0 Do \
32   ▷ ▷ -end
33   ▷ #$(PDFLATEX) cc_4_poset_lvl_9.tex
34   ▷ #$(OPEN) cc_4_poset_lvl_9.pdf
35   ▷ $(PDFLATEX) Cubic_curves_q4.tex
36   ▷ $(OPEN) Cubic_curves_q4.pdf
37

```

classifies the cubic curves in $\text{PG}(2, 4)$ containing at least 9 points. There are none.

The command

Code Snippet 391

```

1    poly_orbits_q4:
2    ▷ $(ORBITER) -v 4 \

```

```

3    ▷ ▷ -define Do -draw_options \
4    ▷ ▷ ▷ -yout 500000 -radius 15 -nodes_empty \
5    ▷ ▷ ▷ -line_width 0.5 -y_stretch 0.25 -embedded -end \
6    ▷ ▷ -define F -finite_field -q 4 -print_numerically -end \
7    ▷ ▷ -define R -polynomial_ring \
8    ▷ ▷ ▷ -field F \
9    ▷ ▷ ▷ -number_of_variables 3 \
10   ▷ ▷ ▷ -homogeneous_of_degree 3 \
11   ▷ ▷ ▷ -monomial_ordering_partition \
12   ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
13   ▷ ▷ ▷ -end \
14   ▷ ▷ -define G -linear_group -PGGL 3 F -end \
15   ▷ ▷ -define Orb -orbits -group G \
16   ▷ ▷ ▷ -print_interval 1000000 \
17   ▷ ▷ ▷ -on_polynomials R \
18   ▷ ▷ -end \
19   ▷ ▷ -define ro \
20   ▷ ▷ ▷ -poset_classification_report_options \
21   ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
22   ▷ ▷ ▷ ▷ -draw_options Do \
23   ▷ ▷ -end \
24   ▷ ▷ -with Orb -do -orbits_activity \
25   ▷ ▷ ▷ -report "" \
26   ▷ ▷ ▷ -report_options ro \
27   ▷ ▷ -end \
28   ▷ ▷ -with Orb -do -orbits_activity \
29   ▷ ▷ ▷ -export_something "orbit" 0 \
30   ▷ ▷ -end
31   ▷ $(PDFLATEX) poly_orbits_d3_n2_q4.tex
32   ▷ $(OPEN) poly_orbits_d3_n2_q4.pdf
33

```

uses the basic Schreier algorithm to classify all cubic curves in $PG(2, 4)$. This is done using the action of the group $PTL(3, 4)$ on all cubic forms over $\mathbb{F}_4$. The output is shown below:

The Varieties of degree 3 in $PG(2, 4)$, summary

Objects with 13 Points

There are 1 objects with 13 Points:

orbit : rep : go : poly : Pts

4 : (11) : 576 : 11= $X_0^3 + X_1^3$: 2,3,4,5,6,11,12,14,15,16,18,19,20

Objects with 12 Points

There are 1 objects with 12 Points:

orbit : rep : go : poly : Pts

2 : (9) : 108 : 9= $X_0X_1X_2$: 0,1,2,4,5,6,7,8,9,10,13,17

Objects with 10 Points

There are 1 objects with 10 Points:

orbit : rep : go : poly : Pts

25 : (87743) : 20 : $87743 = X_0^3 + 2X_1^3 + 2X_2^3 + X_0^2X_1 + X_0^2X_2 + X_0X_1X_2$: 3,5,8,10,13,14,15,17,18,20

Objects with 9 Points

There are 4 objects with 9 Points:

orbit : rep : go : poly : Pts

1 : (3) : 288 : $3 = X_0^2X_1$: 0,1,2,7,8,9,10,13,17

6 : (18) : 432 : $18 = X_0^3 + X_1^3 + X_2^3$: 4,5,6,7,8,9,10,13,17

8 : (24) : 54 : $24 = 3X_0^3 + 2X_1^3 + X_2^3$: 3,11,12,14,15,16,18,19,20

13 : (175) : 24 : $175 = X_1^3 + X_2^3 + X_0^2X_1 + X_0^2X_2$: 0,3,4,7,10,11,12,13,17

Objects with 8 Points

There are 2 objects with 8 Points:

orbit : rep : go : poly : Pts

16 : (87383) : 12 : $87383 = X_0^3 + X_0X_1X_2$: 1,2,3,10,13,16,17,19

20 : (87466) : 4 : $87466 = X_1^3 + X_2^3 + X_0^2X_1 + X_0X_1X_2$: 0,3,4,10,13,14,17,18

Objects with 7 Points

There are 1 objects with 7 Points:

orbit : rep : go : poly : Pts

11 : (50) : 6 : $50 = 2X_0^3 + X_1^3 + X_2^3 + X_0^2X_1$: 5,10,12,13,14,17,19

Objects with 6 Points

There are 4 objects with 6 Points:

orbit : rep : go : poly : Pts

18 : (87404) : 6 : $87404 = 2X_0^3 + X_1^3 + X_2^3 + X_0X_1X_2$: 10,11,13,14,17,20

21 : (87468) : 3 : $87468 = 2X_0^3 + X_1^3 + X_2^3 + X_0^2X_1 + X_0X_1X_2$: 5,10,13,15,16,17

23 : (87471) : 4 : $87471 = X_0^3 + 2X_1^3 + X_2^3 + X_0^2X_1 + X_0X_1X_2$: 5,7,8,9,15,16

26 : (91990) : 60 : $91990 = 2X_0^2X_2 + X_1^2X_2 + X_0X_1X_2$: 0,1,2,4,5,6

Objects with 5 Points

There are 4 objects with 5 Points:

orbit : rep : go : poly : Pts

0 : (0) : 5760 : $0 = X_0^3$: 1,2,10,13,17

9 : (34) : 192 : $34 = 2X_0^3 + X_1^3 + X_0^2X_1$: 2,5,11,16,18

10 : (44) : 6 : $44 = X_2^3 + X_0^2X_1$: 0,1,3,15,20

14 : (176) : 8 : $176 = X_0^3 + X_1^3 + X_2^3 + X_0^2X_1 + X_0^2X_2$: 10,13,16,17,19

Objects with 4 Points

There are 3 objects with 4 Points:

orbit : rep : go : poly : Pts

3 : (10) : 12 : 10= $X_0^3 + X_1^3 + X_2^3 + X_0^2X_1 + X_0^2X_2 + X_0X_1^2 + X_1^2X_2 + X_0X_2^2 + X_1X_2^2 + X_0X_1X_2$
: 3,4,7,10

17 : (87388) : 6 : 87388= $2X_0^3 + X_1^3 + X_0X_1X_2$: 2,12,15,18

22 : (87470) : 2 : 87470= $2X_1^3 + X_2^3 + X_0^2X_1 + X_0X_1X_2$: 0,6,19,20

Objects with 3 Points

There are 2 objects with 3 Points:

orbit : rep : go : poly : Pts

7 : (19) : 18 : 19= $2X_0^3 + X_1^3 + X_2^3$: 10,13,17

15 : (180) : 9 : 180= $X_0^3 + 2X_1^3 + X_2^3 + X_0^2X_1 + X_0^2X_2$: 5,15,18

Objects with 2 Points

There are 1 objects with 2 Points:

orbit : rep : go : poly : Pts

24 : (87472) : 4 : 87472= $2X_0^3 + 2X_1^3 + X_2^3 + X_0^2X_1 + X_0X_1X_2$: 11,12

Objects with 1 Points

There are 2 objects with 1 Points:

orbit : rep : go : poly : Pts

5 : (12) : 288 : 12= $2X_0^3 + X_1^3$: 2

12 : (55) : 48 : 55= $3X_0^3 + 2X_1^3 + X_2^3 + X_0^2X_1$: 4

Objects with 0 Points

There are 1 objects with 0 Points:

orbit : rep : go : poly : Pts

19 : (87409) : 126 : 87409= $3X_0^3 + 2X_1^3 + X_2^3 + X_0X_1X_2$:

This classification involves curves that are degenerate or singular, or contain lines.

Chapter 9

Cubic Surfaces and Quartic Curves

9.1 Generalities

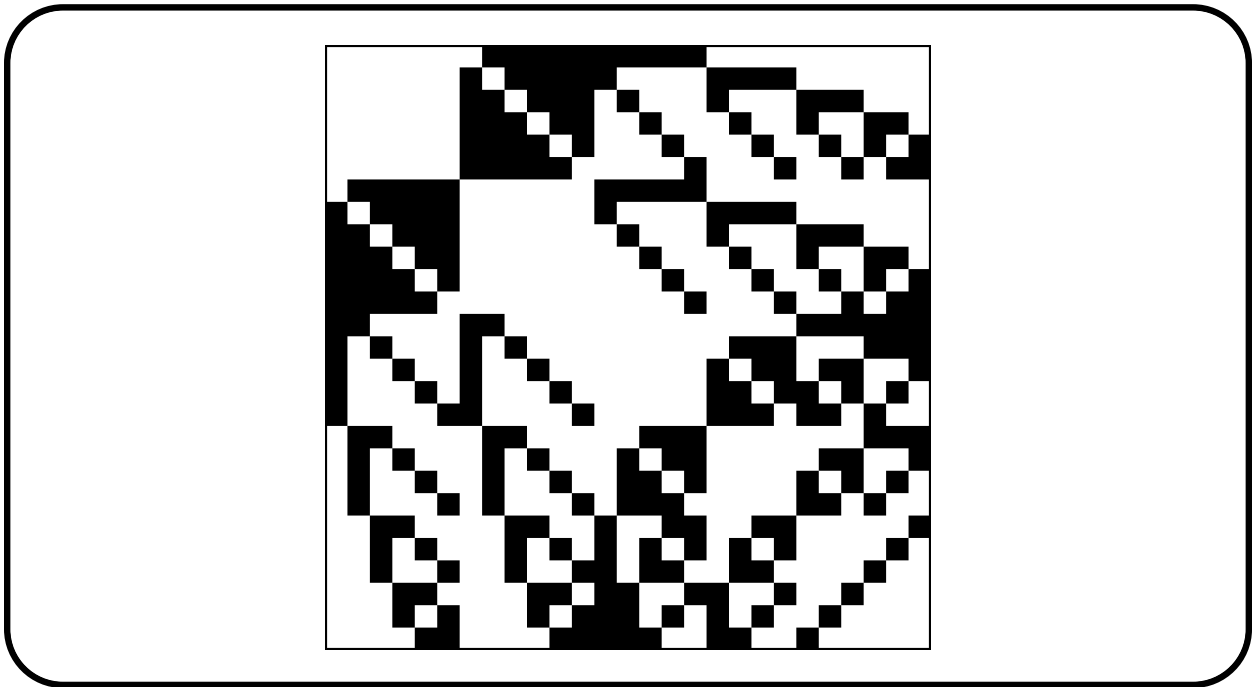
Orbiter can create, classify and investigate cubic surfaces over small finite fields. In this section, some global commands associated with cubic surfaces will be shown. Section 9.2 demonstrates how cubic surfaces can be created in Orbiter. Section 9.5 shows Orbiter commands related to quartic curves. Section 9.4 discusses how cubic surfaces can be classified. Section 9.7 covers the GAP interface for cubic surfaces.

The command

Code Snippet 392

```
1    surface_4.0_lines_vs_lines:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 4 -end \
4    ▷ ▷ -define P -projective_space -n 3 -field F -end \
5    ▷ ▷ -with P -do \
6    ▷ ▷ -projective_space_activity \
7    ▷ ▷ ▷ -export_cubic_surface_line_vs_line_incidence_matrix \
8    ▷ ▷ -end
9    ▷ $(ORBITER) -v 2 -draw_matrix \
10   ▷ ▷ -input_csv_file PG_3.4_lines_vs_lines_incma.csv \
11   ▷ ▷ -box_width 32 -bit_depth 24 \
12   ▷ ▷ -partition 3 27 27 \
13   ▷ ▷ -end
14   ▷ magick PG_3.4_lines_vs_lines_incma_draw.bmp PG_3.4_lines_vs_lines_incma_draw.png
15
```

exports the adjacency matrix of the graph of incident lines of a cubic surface. The lines are ordered w.r.t. the Schläefli labeling. The command invokes a projective space activity, and hence creates a projective space first. The order of the field is irrelevant. The command also produces a drawing of the adjacency matrix, shown below:



Using Schlaefli labels, the lines are ordered

$$a_1, \dots, a_6, b_1, \dots, b_6, c_{12}, c_{13}, \dots, c_{56}.$$

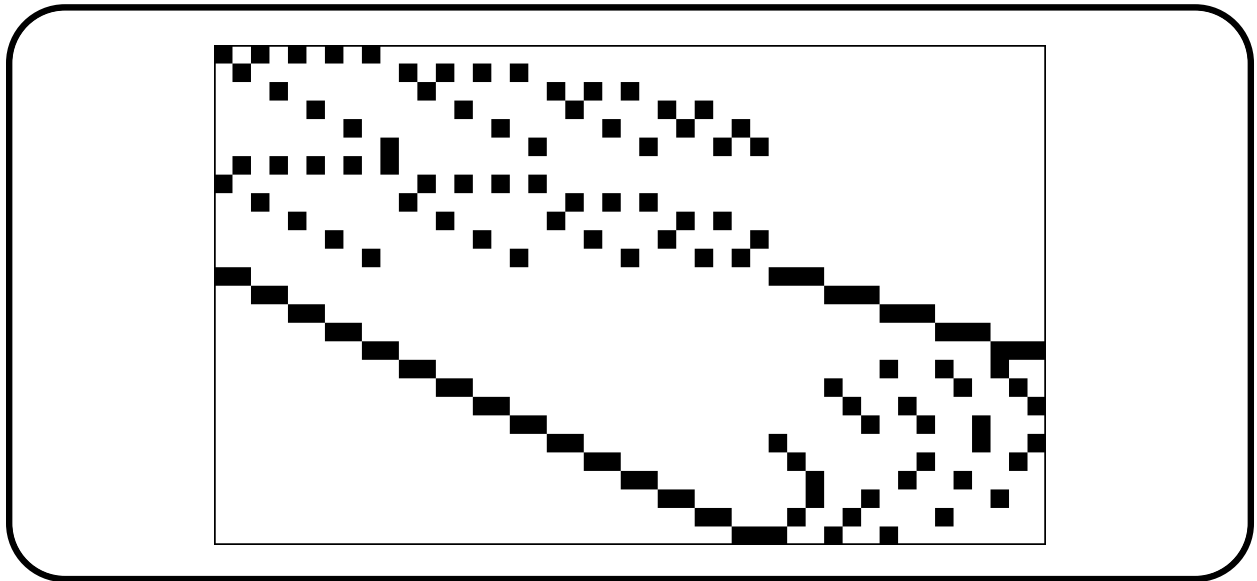
The command

Code Snippet 393

```

1    surface_4_0_lines_vs_tritangent_planes:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 4 -end \
4    ▷ ▷ -define P -projective_space -n 3 -field F -end \
5    ▷ ▷ -with P -do \
6    ▷ ▷ -projective_space_activity \
7    ▷ ▷ ▷ -export_cubic_surface_line_tritangent_plane_incidence_matrix \
8    ▷ ▷ -end
9    ▷ $(ORBITER) -v 2 -draw_matrix \
10   ▷ ▷ -input_csv_file PG_3_4_lines_tritplanes_incma.csv \
11   ▷ ▷ -box_width 32 -bit_depth 24 \
12   ▷ ▷ -partition 3 27 45 \
13   ▷ ▷ -end
14   ▷ magick PG_3_4_lines_tritplanes_incma_draw.bmp PG_3_4_lines_tritplanes_incma_dra
    w.png
15
```

exports the incidence matrix between lines and tritangent planes of a cubic surface. The lines are ordered w.r.t. the Schlaefli labeling. The tritangent planes are ordered w.r.t. the Schlaefli labeling. The command invokes a projective space activity, and hence creates a projective space first. The order of the field is irrelevant. The command also produces a drawing of the incidence matrix, shown below:



Creating a Cubic Surface (Part 1)		
Command	Arguments	Purpose
-space	P	Specify the 3-dimensional projective space $P = PG(3, q)$.
-label_txt	label	Override the ascii label of the curve.
-label_tex	label	Override the latex label of the curve.
-label_for_summary	label	Override the ascii label of the curve, to be used in summary commands.
-catalogue	i	Create the i -th surface in the Orbiter catalogue. Here, i is an index variable used to index all surfaces in $PG(3, q)$. The index i is zero-based. The automorphism group is created as well.
-by_coefficients	list-of-coeff-pairs	Create a surface from a list of coefficient-monomial pairs. The automorphism group is not created.
-by_rank	rank q_0	Create a surface from the numerical rank of the equation over the subfield $\mathbb{F}_{q_0}$. Here, we think of the equation as a point in $PG(19, q_0)$ and use the Orbiter point rank.
-family_Eckardt	a b	Create the Eckardt surface with parameters (a, b) as in see [16] (where it is called the Hilbert, Cohn-Vossen surface). The equation is $X_3^3 - b^2(X_0^2 + X_1^2 + X_2^2)X_3 + \frac{b^3}{a}(a^2 + 1)X_0X_1X_2 = 0$. The automorphism group is created as well.

Table 9.1: Creating a Cubic Surface (Part 1)

9.2 Creating Cubic Surfaces

In this section, we describe ways in which surfaces can be created in Orbiter.

Orbiter contains a built-in catalogue of cubic surfaces with 27 lines for small finite fields $\mathbb{F}_q$. The surfaces in the catalogue all come with their automorphism group. It is also possible to create surfaces from known families, or from associated objects like 6-arcs. Some of these constructions only create the surface, not the automorphism group.

Tables 9.1-9.3 summarize the Orbiter commands that can be used to create cubic surfaces. The commands require a projective space object. Not all of the surfaces created have 27 lines, and some of the constructions may yield degenerate surfaces.

Let us look at some examples. The next command creates the unique surface with 27 lines over the field $\mathbb{F}_4$. It is the Hirschfeld surface. In this command, the surface is pulled from Orbiter's built-in catalogue of cubic surfaces. The surface has Orbiter Catalogue Number (OCN) equal to 0.

Code Snippet 394

```

1  surface_4.0:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define F -finite_field -q 4 -end \
4  ▷ ▷ -define P -projective_space -n 3 -field F -end \

```

Creating a Cubic Surface (Part 2)		
Command	Arguments	Purpose
-family_G13	a	Create a member of the G_{13} family with parameter a . The surface has 13 Eckardt points.
-family_F13	a	Create a member of the F_{13} family with parameter a . The surface has 13 Eckardt points.
-family_bes	$a\ c$	Create a member of the “bes”-family with parameter a . The surface has 5 Eckardt points.
-family_general_abcd	$a\ b\ c\ d$	Create a member of the general family with parameters a, b, c, d .
-arc_lifting_with_trihedral_pair	A	Create the surface associated with the arc $A = a_1, \dots, a_6$ in $\text{PG}(2, q)$ by means of the Clebsch map. Each of the a_i is the rank of a point in $\text{PG}(2, q)$. Use the trihedral pair algorithm. Here, A is a comma-separated string containing the numerical ranks of the P_i in $\text{PG}(2, q)$.
-arc_lifting_with_two_lines	$A\ L$	"Create the surface associated with the arc $A = a_1, \dots, a_6$ in $\text{PG}(2, q)$ by means of the Clebsch map, defined by the lines ℓ_1 and ℓ_2 whose ranks are given in L . If both of the lines are given as 0, Orbiter will pick a suitable set of lines."

Table 9.2: Creating a Cubic Surface (Part 2)

Creating a Cubic Surface (Part 3)		
Command	Arguments	Purpose
-Cayley_form	$k\ l\ m\ n$	Create the surface from Cayley's normal form, using the parameters k, l, m, n .
-Pentahedral_form	params25	Create the surface from Sylvester's pentahedral normal form, using the give 25 parameters. The first 20 parameters are 5 sets of 4 coefficients each of planes, the latter 5 parameters are the coefficients of the linear combination of the third powers of the five planes.
-by_equation	R label label_tex vars eqn para para_tex para_values	Create the surface from an equation.
-by_symbolic_object	R formula	Create the surface from an equation. Here, R is a ring and formula is the equation of the surface.
-by_double_six	label label_tex D	Create the surface from a double six D .
-by_skew_hexagon	label label	Create the surface from a skew hexagon.
-select_double_six	L	Establish a new set of Schlaefli labels by distinguishing a double six, using an existing Schlaefli labeling. The double six is given in L , and consists of 12 comma separated entries (first the six a_i , then the corresponding six b_j). The entries in L can be either Schlaefli symbols or numerical line indices. The latter are integers in the interval $[0, 26]$. This command can be repeated. In each application, the selection takes place with respect to the previous Schlaefli labeling.
-override_group	descr	Override the automorphism group of the surface by the given group.
-random		Create a random cubic surface with 27 lines over the given field.
-transform	elt	Apply the transformation given by the group element.
-transform_inverse	elt	Apply the inverse transformation given by the group element.

Table 9.3: Creating a Cubic Surface (Part 3)

```

5    ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
6

```

It is possible to create the same surface directly from the equation. At first, we define a variable holding the equation in algebraic form:

Code Snippet 395

```

1    HIRSCHFELD_SURFACE_EQN_AF="X0^2*X3+X0*X3^2+X1^2*X2+X1*X2^2"
2

```

The following command creates the surface from the algebraic form and produces a report. In order to do so, the command creates a suitable polynomial ring R and a symbolic object for the equation. Only then the surface is created and a report is written.

Code Snippet 396

```

1    Hirschfeld_surface_q4:
2    ▷ $(ORBITER) -v 6 \
3    ▷ ▷ -define Do -draw_options \
4    ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5    ▷ ▷ -end \
6    ▷ ▷ -define F -finite_field -q 4 -compute_related_fields -end \
7    ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
8    ▷ ▷ -define R -polynomial_ring \
9    ▷ ▷ ▷ -field F \
10   ▷ ▷ ▷ -number_of_variables 4 \
11   ▷ ▷ ▷ -homogeneous_of_degree 3 \
12   ▷ ▷ ▷ -monomial_ordering_partition \
13   ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
14   ▷ ▷ -end \
15   ▷ ▷ -define f -symbolic_object \
16   ▷ ▷ ▷ -field F \
17   ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
18   ▷ ▷ ▷ -text $(HIRSCHFELD_SURFACE_EQN_AF) \
19   ▷ ▷ -end \
20   ▷ ▷ -define Surf -cubic_surface \
21   ▷ ▷ ▷ -space P \
22   ▷ ▷ ▷ -by_symbolic_object R f \
23   ▷ ▷ -end \
24   ▷ ▷ -with Surf -do \
25   ▷ ▷ -cubic_surface_activity \
26   ▷ ▷ ▷ -report Do \
27   ▷ ▷ -end
28   ▷ pdflatex surface_equation_f_q4_report.tex
29   ▷ open surface_equation_f_q4_report.pdf
30

```

There is a 4-parameter normal form $\mathcal{F}_{abcd}$ for cubic surfaces with 27 lines. Here, a, b, c, d are field elements in $\mathbb{F}_q$. The following command creates the surface $\mathcal{F}_{abcd}$ with $(a, b, c, d) = (2, 3, 3, 4)$ over the field $\mathbb{F}_7$:

Code Snippet 397

```

1  Family_general_F7:
2  ▷ $(ORBITER) -v 10 \
3  ▷ ▷ -define F -finite_field -q 7 -end \
4  ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5  ▷ ▷ -define S7_abcd_2_3_3_4 -cubic_surface \
6  ▷ ▷ ▷ -space P -family_general_abcd 2 3 3 4 \
7  ▷ ▷ -end
8

```

This command uses a built-in command to realize the surface of type $\mathcal{F}_{abcd}$. We will see later that it is possible to create surfaces symbolically from an equation.

Another way of creating a cubic surface is as member of a known infinite family. However, different choices of the parameters may lead to isomorphic surfaces. For instance,

Code Snippet 398

```

1  surface_eckardt_13_4_12:
2  ▷ $(ORBITER) -v 6 \
3  ▷ ▷ -define F -finite_field -q 13 -end \
4  ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5  ▷ ▷ -define Eckardt_4_12 -cubic_surface \
6  ▷ ▷ ▷ -space P -family_Eckardt 4 12 \
7  ▷ ▷ -end
8

```

creates the member of the Eckardt family described in [16] with parameters $(a, b) = (4, 12)$ over the field $\mathbb{F}_{13}$. On the other hand, the command

Code Snippet 399

```

1  Eckardt_13:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define F -finite_field -q 13 -end \
4  ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5  ▷ ▷ -define Eckardt_3_1 -cubic_surface \
6  ▷ ▷ ▷ -space P -family_Eckardt 3 1 \
7  ▷ ▷ -end
8

```

creates the Eckardt surface over the field $\mathbb{F}_{13}$ from the parameter set $(a, b) = (3, 1)$. The two surfaces are isomorphic.

It is possible to create a random cubic surface of order q . The following example creates a random cubic surface over the field $\mathbb{F}_{11}$:

Code Snippet 400

```

1  surface_11_random:
2  ▷ $(ORBITER) -v 10 \
3  ▷ ▷ -define F -finite_field -q 11 -end \

```

```

4      ▷ ▷ -define P -projective_space -n 3 -field F -v 6 -end \
5      ▷ ▷ -define S -cubic_surface -space P -random -end
6

```

The only restriction is that q is within the range for which the classification of cubic surfaces is available in Orbiter.

The normal form of [18] can be used to create any cubic surface over $\mathbb{F}_q$. This means that up to isomorphism, any cubic surface with 27 lines can be written as $\mathcal{F}_{a,b,c,d}$ for a suitable choice of parameters $a, b, c, d \in \mathbb{F}_q$. For instance, the following command uses $(a, b, c, d) = (2, 30, 30, 2)$ to create the Eckardt surface over $\mathbb{F}_{31}$:

Code Snippet 401

```

1      surface_F_abcd_Eckardt_q31_by_equation:
2      ▷ $(ORBITER) -v 10 \
3      ▷ ▷ -define F -finite_field -q 31 -end \
4      ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5      ▷ ▷ -define R -polynomial_ring \
6      ▷ ▷ ▷ -field F \
7      ▷ ▷ ▷ -number_of_variables 4 \
8      ▷ ▷ ▷ -homogeneous_of_degree 3 \
9      ▷ ▷ ▷ -monomial_ordering_partition \
10     ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
11     ▷ ▷ ▷ -end \
12     ▷ ▷ -define F_abcd -cubic_surface -space P \
13     ▷ ▷ ▷ -by_equation \
14     ▷ ▷ ▷ R \
15     ▷ ▷ ▷ "F_abcd" \
16     ▷ ▷ ▷ "\DF_{a,b,c,d}\D" "X0,X1,X2,X3" \
17     ▷ ▷ ▷ $(F_abcd_eqn) \
18     ▷ ▷ ▷ "a,b,c,d" \
19     ▷ ▷ ▷ "\D(a,b,c,d)=(2,30,30,2)\D" \
20     ▷ ▷ ▷ "2,30,30,2" \
21     ▷ ▷ -end
22

```

A cubic surface is determined uniquely up to isomorphism by a non-conical 6-arc in a plane. The points of the arc are given in Orbiter ranks. The following command creates the cubic surface which is associated with the arc 0, 1, 2, 3, 43, 113 over the field $\mathbb{F}_{13}$.

Code Snippet 402

```

1      Eckardt_13_by_arc_lifting:
2      ▷ $(ORBITER) -v 3 \
3      ▷ ▷ -define F -finite_field -q 13 -end \
4      ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5      ▷ ▷ -define S -cubic_surface -space P \
6      ▷ ▷ ▷ -arc_lifting_with_triangular_pair "0,1,2,3,43,113" \
7      ▷ ▷ -end
8

```

The surface is of Eckardt type.

The formula for $\mathcal{F}_{a,b,c,d}$ can be used symbolically. To this end, we encode it as a makefile variable:

Code Snippet 403

```

1   F_abcd_eqn="- (a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*(b - d)*X0^2*X2 \
2   + (a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*(a + b - c - d)*X0*X1*X2 \
3   + (a^2*c - a^2*d - a*c^2 + b*c^2 + a*d - b*c)*(b - d)*X0*X1*X3 \
4   - (a*d - b*c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X0*X2^2 \
5   - (a^2*c*d - a*b*c^2 - a^2*d + a*b*d + b*c^2 - b*c*d)*(b - d)*X0*X2*X3 \
6   - (a - c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X1^2*X2 \
7   - (a - c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X1^2*X3 \
8   + (a*d - b*c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X1*X2^2 \
9   + ((1+1)*a^2*b*c*d - a^2*b*d^2 - (1+1)*a^2*c*d^2 \
10  - (1+1)*a*b^2*c^2 + a*b^2*c*d + (1+1)*a*b*c^2*d + a*b*c*d^2 \
11  - b^2*c^2*d - a^2*b*c + a^2*c*d + a^2*d^2 + a*b^2*c + a*b*c^2 \
12  - (1+1+1)*a*b*c*d - a*c^2*d + a*c*d^2 + b^2*c^2)*X1*X2*X3 \
13  + c*a*(a*d - b*c - a + b + c - d)*(b - d)*X1*X3^2"
14

```

We use notation such as $1 + 1 + \dots + 1$ (k times) for the integer k . This makes the formula valid over very small fields such as $\mathbb{F}_3$ and $\mathbb{F}_4$, where the Orbiter conventions for field elements prohibit using 3 or 4. There is no automatic reduction modulo the characteristic when specifying field elements.

To do so, we first create a symbolic object for the equation. Then we perform the substitution of (a, b, c, d) with the desired values. Finally, we create the surface from the equation:

Code Snippet 404

```

1   surface_F_abcd.Eckardt.q31_by_symbolic_object:
2   ▷ $(ORBITER) -v 10 \
3   ▷ ▷ -define F -finite_field -q 31 -end \
4   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5   ▷ ▷ -define R -polynomial_ring \
6   ▷ ▷ ▷ -field F \
7   ▷ ▷ ▷ -number_of_variables 4 \
8   ▷ ▷ ▷ -homogeneous_of_degree 3 \
9   ▷ ▷ ▷ -monomial_ordering_partition \
10  ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
11  ▷ ▷ ▷ -end \
12  ▷ ▷ -define F_abcd -symbolic_object \
13  ▷ ▷ ▷ -field F \
14  ▷ ▷ ▷ -text $(F_abcd_eqn) \
15  ▷ ▷ ▷ -end \
16  ▷ ▷ -define abcd_values -symbolic_object \
17  ▷ ▷ ▷ -field F \
18  ▷ ▷ ▷ -text "2,30,30,2" \
19  ▷ ▷ ▷ -end \
20  ▷ ▷ -define F_abcd_sub -symbolic_object \
21  ▷ ▷ ▷ -field F \
22  ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
23  ▷ ▷ ▷ -substitute "a,b,c,d" F_abcd abcd_values \
24  ▷ ▷ -end \
25  ▷ ▷ -define S -cubic_surface -space P \

```



```

26  ▷ ▷ ▷ -by_symbolic_object \
27  ▷ ▷ ▷ R \
28  ▷ ▷ ▷ F_abcd_sub \
29  ▷ ▷ -end
30

```

Using the Clebsch map, we can create a cubic surface from a non-conical 6-arc. The command expects the 6-arc in a plane in $\text{PG}(3, q)$. The command also expects the rank numbers of two skew lines not in the plane containing the arc. If two rank numbers are given as two zeros, then Orbiter will pick a suitable set of lines. Here is the non-conical 6-arc arising from the hyperoval in $\text{PG}(2, 4)$. The associated cubic surface is the Hirschfeld-surface. This can be verified from the fact that the surface has 45 Eckardt points:

Code Snippet 405

```

1  Hirschfeld_surface_by_arc_lifting_with_two_lines:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define F -finite_field -q 4 -end \
4  ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5  ▷ ▷ -define Surf -cubic_surface -space P \
6  ▷ ▷ ▷ -arc_lifting_with_two_lines "0,1,2,12,18,21" "0,0" \
7  ▷ ▷ -end \
8  ▷ ▷ -define Do -draw_options \
9  ▷ ▷ ▷ -radius 200 -line_width 0.3 \
10 ▷ ▷ -end \
11 ▷ ▷ -with Surf -do \
12 ▷ ▷ -cubic_surface_activity \
13 ▷ ▷ ▷ -report Do \
14 ▷ ▷ -end \
15 ▷ ▷ -print_symbols
16 ▷ pdflatex surface_arc_lifting_with_two_lines_q4_lines_0_0_arc_0_1_2_12_18_21_report.tex
17 ▷ open surface_arc_lifting_with_two_lines_q4_lines_0_0_arc_0_1_2_12_18_21_report.pdf
18

```

We can also create a cubic surface as an algebraic variety. For instance, consider the second surface from Dickson's list [29]. In algebraic form, the curve is given by using the following makefile variable:

Code Snippet 406

```

1  Dickson2_eqn_af="X1^3 + X2^3 + X3^3 + X0^2*X1 + X0*X1^2 + X1*X2^2 + X1*X3^2 + X2*
   X3^2 + X1*X2*X3"
2

```

The following command creates the algebraic variety defined by this equation

Code Snippet 407

```

1  Dickson2_group:
2  ▷ $(ORBITER) -v 6 \
3  ▷ ▷ -define F -finite_field -q 2 -end \

```

```

4   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5   ▷ ▷ -define R -polynomial_ring \
6   ▷ ▷ ▷ -field F \
7   ▷ ▷ ▷ -number_of_variables 4 \
8   ▷ ▷ ▷ -homogeneous_of_degree 3 \
9   ▷ ▷ ▷ -monomial_ordering_partition \
10  ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
11  ▷ ▷ -end \
12  ▷ ▷ -define V -variety \
13  ▷ ▷ ▷ -projective_space P \
14  ▷ ▷ ▷ -ring R \
15  ▷ ▷ ▷ -equation_in_algebraic_form \
16  ▷ ▷ ▷ ▷ $(Dickson2_eqn_af) \
17  ▷ ▷ ▷ -label_txt Dickson_2 \
18  ▷ ▷ ▷ -label_tex "{\\rm Dickson\\_2}" \
19  ▷ ▷ -end \
20  ▷ ▷ -with V -do -variety_activity \
21  ▷ ▷ ▷ -compute_group \
22  ▷ ▷ ▷ -nauty_control -nauty_log "Dickson2_nauty_log.txt" -save_orbit_of_equations
    eqn_ -end \
23  ▷ ▷ -end \
24  ▷ ▷ -with V -do -variety_activity -singular_points -end \
25  ▷ ▷ -with V -do -variety_activity -report -end
26  ▷ $(PDFLATEX) variety_Dickson_2_report.tex
27  ▷ $(OPEN) variety_Dickson_2_report.pdf
28

```

It is also possible to create a surface from an equation given as coefficient/value pairs. For instance, the Dickson surface D_6 can be created from the following makefile variable:

Code Snippet 408

```

1   Dickson6="1,2,1,3,1,4,1,7,1,15"
2

```

This means that the monomials m_2, m_3, m_4, m_7 , and m_{15} have a coefficient of one. The other monomials do not appear in the equation. The monomial ordering is determined by the polynomial ring. In this case, since we use the default ring, the partition ordering is used. The command to create the surface over $\mathbb{F}_2$ is:

Code Snippet 409

```

1   Dickson6_q2:
2   ▷ $(ORBITER) -v 10 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 2 -end \
7   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
8   ▷ ▷ -define S -cubic_surface \
9   ▷ ▷ ▷ -space P -by_coefficients $(Dickson6) \
10  ▷ ▷ -end \

```

```
11  ▷ ▷ -with S -do \  
12  ▷ ▷ -cubic_surface_activity \  
13  ▷ ▷ ▷ -report Do \  
14  ▷ ▷ -end  
15  ▷ $(PDFLATEX) surface_by_coefficients_q2_report.tex  
16  ▷ $(OPEN) surface_by_coefficients_q2_report.pdf  
17
```

9.3 Cubic Surface Activities

Table 9.4 lists activities for a cubic surface object.

Table 9.5 lists options for exporting data.

The following command creates the Hirschfeld surface over $\mathbb{F}_4$ and produces a report for it:

Code Snippet 410

```

1    surface_4.0_report:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define Do -draw_options \
4    ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
5    ▷ ▷ -end \
6    ▷ ▷ -define F -finite_field -q 4 -end \
7    ▷ ▷ -define P -projective_space -n 3 -field F -end \
8    ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
9    ▷ ▷ -with S -do \
10   ▷ ▷ -cubic_surface_activity \
11   ▷ ▷ ▷ -report Do \
12   ▷ ▷ -end
13   ▷ $(PDFLATEX) surface_catalogue_q4_iso0_report.tex
14   ▷ $(OPEN) surface_catalogue_q4_iso0_report.pdf
15
```

We may decide to choose a nice labeling of lines. For this, we utilize the `-select_double_six` command. This command can be repeated:

Code Snippet 411

```

1    surface_4.0_clean_report:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define Do -draw_options \
4    ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
5    ▷ ▷ -end \
6    ▷ ▷ -define F -finite_field -q 4 -end \
7    ▷ ▷ -define P -projective_space -n 3 -field F -end \
8    ▷ ▷ -define S -cubic_surface -space P -catalogue 0 \
9    ▷ ▷ ▷ -select_double_six "a4,b4,c15,c25,c35,c56,a5,b5,c14,c24,c34,c46" \
10   ▷ ▷ ▷ -select_double_six "a1,a6,a2,a3,a4,a5,b1,b6,b2,b3,b4,b5" \
11   ▷ ▷ ▷ -select_double_six "a1,a2,a5,a6,a3,a4,b1,b2,b5,b6,b3,b4" \
12   ▷ ▷ -end \
13   ▷ ▷ -with S -do \
14   ▷ ▷ -cubic_surface_activity \
15   ▷ ▷ ▷ -report Do \
16   ▷ ▷ -end
17   ▷ $(PDFLATEX) surface_catalogue_q4_iso0_report.tex
18   ▷ $(OPEN) surface_catalogue_q4_iso0_report.pdf
19
```

The following command exports some data about the surface:

Cubic Surface Activities		
Command	Arguments	Purpose
-report	options	Produce a latex report about the cubic surface. This function considers the given options for drawing.
-report_group_elements	csv-file heading	Report properties of elements of the automorphism group. The group elements must be given in make-element-form in a column in a csv-file.
-export_something	type	Exports data about the surface to a file. The type of data to be exported must be specified according to Table 9.5.
-export_gap		Export a cubic surface to GAP. See Section 9.7.
-report_all_flag_orbits		Creates all flag orbits of quartic curves with 28 bitangents associated with the cubic surface. The flag orbits are in 1-1 correspondence to the orbits on surface points not on any line of the surface.
-export_all_quartic_curves		Export all quartic curves arising from the orbits of points not on lines of the surface. The curves are written to a csv file.
-export_something_with_group_element	what label	Exports data about the surface to a file. The type of data to be exported must be specified according to Table 9.5.
-action_on_module	type basis generators	
-Clebsch_map_up	line1 line2	Define the birational Clebsch map from a plane to the surface. The skew lines line1 and line2 are used to define the map.
-Clebsch_map_up_single_point	pt line1 line2	Map a single point to the surface under the Clebsch map defined by the pair of lines line1 and line2.
-recognize_Fabcd	Arc-classification	Recognize the surface as a member of the F_{abcd} family. Computes the parameters a, b, c, d . The parameter Arc-classification must be an object of type orbit, representing the classification of 6-arcs in the associated plane $\text{PG}(2, q)$.

Table 9.4: Cubic Surface Activities

Options for Exporting Data: Cubic Surfaces		
Label		What
points		The points on the surface.
points_off		The points off the surface.
Eckardt_points		The Eckardt points of the surface.
double_points		The double points of the surface.
single_points		The single points of the surface.
zero_points		The zero points of the surface. A point is a zero point if it does not lie on any line.
singular_points		The singular points of the surface.
lines		The set of lines of the surface.
tritangent_planes		The set of tritangent planes of the surface.
Hesse_planes		The set of Hesse planes of the surface.

Table 9.5: Options for Exporting Data: Cubic Surfaces

Code Snippet 412

```

1  surface_4.0_export:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define F -finite_field -q 4 -end \
4  ▷ ▷ -define P -projective_space -n 3 -field F -end \
5  ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
6  ▷ ▷ -with S -do \
7  ▷ ▷ -cubic_surface_activity \
8  ▷ ▷ ▷ -export_something "points" \
9  ▷ ▷ -end \
10 ▷ ▷ -with S -do \
11 ▷ ▷ -cubic_surface_activity \
12 ▷ ▷ ▷ -export_something "points_off" \
13 ▷ ▷ -end \
14 ▷ ▷ -with S -do \
15 ▷ ▷ -cubic_surface_activity \
16 ▷ ▷ ▷ -export_something "lines" \
17 ▷ ▷ -end \
18 ▷ ▷ -with S -do \
19 ▷ ▷ -cubic_surface_activity \
20 ▷ ▷ ▷ -export_something "Hesse_planes" \
21 ▷ ▷ -end
22

```

The following data is exported: the set of points on and off the surface, the set of lines, and the set of Hesse planes.

The following command creates the cubic surface over $\mathbb{F}_7$ with catalogue number 0. It also writes a report and exports the set of lines:

Code Snippet 413

```

1   surface_7_0:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define Do -draw.options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 7 -end \
7   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
8   ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
9   ▷ ▷ -with S -do \
10  ▷ ▷ -cubic_surface_activity \
11  ▷ ▷ ▷ -report Do \
12  ▷ ▷ -end \
13  ▷ ▷ -with S -do \
14  ▷ ▷ -cubic_surface_activity \
15  ▷ ▷ ▷ -export_something "lines" \
16  ▷ ▷ -end
17  ▷ $(PDFLATEX) surface_catalogue.q7_iso0_report.tex
18  ▷ $(OPEN) surface_catalogue.q7_iso0_report.pdf
19

```

It is possible to apply a transformation, i.e., a change of coordinates. For instance, the following command produce an equation of a cubic surface over $\mathbb{F}_{16}$. Starting from the surface with catalogue number 0, two transformations are applied:

Code Snippet 414

```

1   surface_16_0:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define Do -draw.options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 16 -end \
7   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
8   ▷ ▷ -define S16_0 -cubic_surface -space P -catalogue 0 \
9   ▷ ▷ ▷ -transform "1,0,0,0,0,1,0,12,0,0,1,12,0,0,0,1,0" \
10  ▷ ▷ ▷ -transform "15,11,4,0,0,0,12,0,0,12,0,0,0,0,0,1,3" \
11  ▷ ▷ -end \
12  ▷ ▷ -with S16_0 -do \
13  ▷ ▷ -cubic_surface_activity \
14  ▷ ▷ ▷ -report Do \
15  ▷ ▷ -end
16  ▷ $(PDFLATEX) surface_catalogue.q16_iso0_report.tex
17  ▷ $(OPEN) surface_catalogue.q16_iso0_report.pdf
18

```

The following command creates the surface $\mathcal{F}_{a,b,c,d}$ with $(a, b, c, d) = (2, 30, 30, 2)$ over $\mathbb{F}_{31}$ from the previous section and produces a report:

Code Snippet 415

```

1   surface_F_abcd.Eckardt.q31_by_symbolic.object.report:
2   ▷ $(ORBITER) -v 7 \
3   ▷ ▷ -define Do -draw.options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 31 -end \
7   ▷ ▷ -define P -projective_space -n 3 -field F -end \
8   ▷ ▷ -define R -polynomial_ring \
9   ▷ ▷ ▷ -field F \
10  ▷ ▷ ▷ -number_of_variables 4 \
11  ▷ ▷ ▷ -homogeneous_of_degree 3 \
12  ▷ ▷ ▷ -monomial_ordering_partition \
13  ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
14  ▷ ▷ ▷ -end \
15  ▷ ▷ -define F_abcd -symbolic_object \
16  ▷ ▷ ▷ -field F \
17  ▷ ▷ ▷ -text $(F_abcd_eqn) \
18  ▷ ▷ ▷ -end \
19  ▷ ▷ -define abcd_values -symbolic_object \
20  ▷ ▷ ▷ -field F \
21  ▷ ▷ ▷ -text "2,30,30,2" \
22  ▷ ▷ ▷ -end \
23  ▷ ▷ -define F_abcd_sub -symbolic_object \
24  ▷ ▷ ▷ -field F \
25  ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
26  ▷ ▷ ▷ -substitute "a,b,c,d" F_abcd abcd_values \
27  ▷ ▷ -end \
28  ▷ ▷ -define S -cubic_surface -space P \
29  ▷ ▷ ▷ -by_symbolic_object \
30  ▷ ▷ ▷ R \
31  ▷ ▷ ▷ F_abcd_sub \
32  ▷ ▷ -end \
33  ▷ ▷ -with S -do \
34  ▷ ▷ -cubic_surface_activity \
35  ▷ ▷ ▷ -report Do \
36  ▷ ▷ -end
37  ▷ $(PDFLATEX) surface.equation_F_abcd_sub.q31.report.tex
38  ▷ $(OPEN) surface.equation_F_abcd_sub.q31.report.pdf
39

```

The report shows information about the surface like the number of points and lines or the number of Eckardt points. In this case, the surface has 6 Eckardt points.

As described in the previous section, it is possible to create a cubic surface at random. The following command creates a random surface over the field $\mathbb{F}_{11}$. Random means that each isomorphism type of surface appears with equal probability. Since surfaces may have different stabilizer orders, this is different from the equidistribution over all surfaces with 27 lines for a given finite field.

Code Snippet 416

```

1   surface_11_random_report:
2   ▷ $(ORBITER) -v 10 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 11 -end \
7   ▷ ▷ -define P -projective_space -n 3 -field F -v 6 -end \
8   ▷ ▷ -define S -cubic_surface -space P -random -end \
9   ▷ ▷ -with S -do \
10  ▷ ▷ -cubic_surface_activity \
11  ▷ ▷ ▷ -report Do \
12  ▷ ▷ -end
13  ▷ $(PDFLATEX) surface_random_q11_report.tex
14  ▷ $(OPEN) surface_random_q11_report.pdf
15

```

The following report will be produced:

The equation of the surface is :

$$\begin{aligned}
 &X_0^3 + 2X_1^3 + 4X_2^3 + 7X_3^3 + 7X_0^2X_1 + 2X_0^2X_2 + 2X_0^2X_3 + 9X_1^2X_2 + 9X_1^2X_3 + 6X_0X_2^2 \\
 &\quad + 9X_1X_2^2 + 4X_2^2X_3 + 10X_0X_3^2 + 9X_1X_3^2 + 3X_2X_3^2 + 8X_0X_1X_2 + 4X_0X_1X_3 = 0
 \end{aligned}$$

(1, 2, 4, 7, 7, 2, 2, 0, 9, 9, 6, 9, 4, 10, 9, 3, 8, 4, 0, 0)

Summary

Object	Number	Orbit type
Lines	27	12, 15
Points on surface	199	10, 15, 24, 30, 60 ²
Singular points	0	
Eckardt points	10	10
Double points	105	15, 30, 60
Single points	84	24, 60
Points off lines	0	
Hesse planes	0	
Axes	10	10
Single sixes	72	2, 20 ² , 30
Double sixes	36	1, 10 ² , 15
Tritangent planes	45	5, 10, 30
Trihedral pairs	120	10, 15 ² , 20, 60
Type of points on lines	12 ²⁷	
Type of lines on points	3 ¹⁰ , 2 ¹⁰⁵ , 1 ⁸⁴	

It is possible to recognize a cubic surface as a member of the $\mathcal{F}_{a,b,c,d}$ family. We can determine elements a, b, c, d in $\mathbb{F}_q$ such that $\mathcal{F}_{a,b,c,d}$ is isomorphic to the given smooth surface. This relies on the classification of non-conical 6-arcs in the associated projective plane $\text{PG}(2, q)$. The following command shows how to classify the non-conical 6-arcs in the plane and how to recognize a surface. In the example, we create the surface over $\mathbb{F}_8$ with catalogue number 0:

Code Snippet 417

```

1  surface.8.0_recognize_abcd:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define Do -draw_options \
4  ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
5  ▷ ▷ -end \
6  ▷ ▷ -define F -finite_field -q 8 -end \
7  ▷ ▷ -define P3 -projective_space -n 3 -field F -end \
8  ▷ ▷ -define P2 -projective_space -n 2 -field F -v 0 -end \
9  ▷ ▷ -define Control -poset_classification_control \
10 ▷ ▷ ▷ -problem_label nc_arcs.q8_d2 \
11 ▷ ▷ ▷ -W -depth 6 \
12 ▷ ▷ -end \
13 ▷ ▷ -define Arc_control -arc_generator_control \
14 ▷ ▷ ▷ -projective_space P2 \
15 ▷ ▷ ▷ -control Control \
16 ▷ ▷ ▷ -target_size 6 \
17 ▷ ▷ ▷ -d 2 \

```

```
18  ▷ ▷ -end \  
19  ▷ ▷ -define Orb -orbits \  
20  ▷ ▷ ▷ -on_arcs Arc_control \  
21  ▷ ▷ -end \  
22  ▷ ▷ -define S -cubic_surface -space P3 -catalogue 0 -end \  
23  ▷ ▷ -with S -do \  
24  ▷ ▷ -cubic_surface_activity \  
25  ▷ ▷ ▷ -recognize_Fabcd Orb \  
26  ▷ ▷ -end  
27
```

Classifying Cubic Surfaces		
Command	Arguments	Purpose
<code>-use_double_sixes</code>		Use the method of substructures based on double sixes.
<code>-projective_space</code>	P	Specify the projective space object P.
<code>-poset_classification_control</code>	C	Use the poset classification control object C.

Table 9.6: Classifying Cubic Surfaces

9.4 Classification of Cubic Surfaces

There are several different approaches to classify cubic surfaces with 27 lines over finite fields $\mathbb{F}_q$ in Orbiter. Classification means to determine the non-equivalent surfaces under the action of the collineation group $\text{PTL}(4, q)$ of $\text{PG}(3, q)$. The approach described in [16] relies on Schlaefli's notion of a double six as a substructure [66]. The approach described in [43] utilizes the relation to non-conical six-arcs in a plane. A third approach is described in [44]. All three approaches are available in Orbiter.

Table 9.6 lists the Orbiter commands to classify cubic surfaces.

Table 9.7 lists the Orbiter commands that can be performed once the classification is completed.

In $\text{PG}(3, 4)$, there is only one type of cubic surfaces with 27 lines. It is a member of the Hirschfeld family, described in [37]. The following Orbiter command can be used to construct this surface and to prove its uniqueness for $\mathbb{F}_4$. The following command utilizes the algorithm of [16] to do so:

Code Snippet 418

```

1   surface_classify_q4:
2   ▷ $(ORBITER) -v 5 \
3   ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4   ▷ ▷ -define F -finite_field -q 4 -end \
5   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
6   ▷ ▷ -define Control -poset_classification_control -W \
7   ▷ ▷ ▷ -problem_label "surf_q4" \
8   ▷ ▷ -end \
9   ▷ ▷ -define Classify -classify_cubic_surfaces \
10  ▷ ▷ ▷ -use_double_sixes \
11  ▷ ▷ ▷ -projective_space P \
12  ▷ ▷ ▷ -poset_classification_control Control \
13  ▷ ▷ -end \
14  ▷ ▷ -define Orb -orbits \
15  ▷ ▷ ▷ -on_cubic_surfaces Classify \
16  ▷ ▷ -end \
17  ▷ ▷ -define Do -draw_options \
18  ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
19  ▷ ▷ -end \
20  ▷ ▷ -define ro \
21  ▷ ▷ ▷ -poset_classification_report_options \
22  ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
23  ▷ ▷ ▷ -draw_options Do \
24  ▷ ▷ -end \
25  ▷ ▷ -with Orb -do -orbits_activity \

```

Activity for Cubic Surfaces After Classification		
Command	Arguments	Purpose
-report	options	Create a latex report.
-stats	prefix	Report statistics on group operations.
-identify_Eckardt		
-identify_F13		
-identify_Bes		
-identify_general_abcd		
-isomorphism_testing	surf1 surf2	Compute an isomorphism from surface 1 to surface 2. surf1 and surf2 must be surface objects.
-recognize	surf	Recognizes the isomorphism type of the given surface object in the Orbiter Catalogue of cubic surfaces.
-create_source_code		Create C++ source code for the cubic surfaces of the given order.
-sweep_Cayley		Identify the isomorphism type of all surfaces of Cayley type.
-sweep_Sylvester	fname col options	Identify the isomorphism type of all surfaces of Sylvester pentahedral type based on a list of 5-sets of planes, given by Orbiter rank. Options is an extra parameter that must be given (it could be an empty string).

Table 9.7: Activity for Cubic Surfaces After Classification

```

26  ▷ ▷ ▷ -report "5plus1=off,double_sixes=off,flag_orbits=off,show_group=off,report_
    options=ro" \
27  ▷ ▷ -end \
28  ▷ ▷ -with Orb -do \
29  ▷ ▷ ▷ -orbits_activity \
30  ▷ ▷ ▷ -export_source_code \
31  ▷ ▷ -end \
32  ▷ ▷ -with Orb -do \
33  ▷ ▷ ▷ -cubic_surfaces_after_classification_activity \
34  ▷ ▷ ▷ -identify_general_abcd \
35  ▷ ▷ -end
36  ▷ $(PDFLATEX) Surfaces.q4.tex
37  ▷ $(OPEN) Surfaces.q4.pdf
38

```

The `-report` option creates a latex report. After some redactions, the report contains the following elements.

The semilinear group

The Action

Group action $\text{PTL}(4, 4)$ of degree 85

The group is a matrix group.

⋮

The base action is on projective space $\text{PG}(3, 4)$

$q = 4$

$p = 2$

$e = 2$

$n = 3$

Number of points = 85

Number of lines = 357

Number of lines on a point = 21

Number of points on a line = 5

⋮

The orthogonal group

The Action

Group action $\text{PTL}(4, 4)\text{OnWedge}$ of degree 1365

The group is a matrix group.

The base action is on projective space $\text{PG}(3, 4)$

$q = 4$

$p = 2$

$e = 2$

$n = 3$

Number of points = 85
 Number of lines = 357
 Number of lines on a point = 21
 Number of points on a line = 5
 ⋮

The group stabilizing the fixed line

The Action

Group action $\text{PTL}(4, 4)$ On Wedgeres100 of degree 100

⋮

Strong generators for a group of order 5529600: ⋮

The classification of five-plus-ones

Poset classification up to depth 5

The Orbits

Number of Orbits By Level

Depth	Nb of orbits
0	1
1	1
2	1
3	1
4	3
5	4

Summary of Orbit Representatives

N = node

D = depth or level

O = orbit with a level

Rep = orbit representative

(S,O) = (order of stabilizer, orbit length)

L = number of live points

F = number of flags

Gen = number of generators for the stabilizer of the orbit rep.

Table 9.8: Orbit Representatives

N	D	O	Rep	(S,O)	L	F	Gen
0	0	0	{ }	(5529600, 1)	100	1	12
1	1	0	{ 0 }	(55296, 100)	64	1	9
2	2	0	{ 0, 3 }	(1728, 3200)	36	1	7
3	3	0	{ 0, 3, 56 }	(144, 38400)	16	3	6
4	4	0	{ 0, 3, 56, 76 }	(288, 19200)	4	2	7
5	4	1	{ 0, 3, 56, 77 }	(96, 57600)	4	1	6
6	4	2	{ 0, 3, 56, 80 }	(72, 76800)	4	2	6
7	5	0	{ 0, 3, 56, 76, 96 }	(1440, 3840)			10
8	5	1	{ 0, 3, 56, 76, 97 }	(96, 57600)			5
9	5	2	{ 0, 3, 56, 80, 92 }	(360, 15360)			7
10	5	3	{ 0, 3, 56, 80, 93 }	(120, 46080)			7

Poset of Orbits in Detail

⋮

Classification of 5 + 1 Configurations in PG(3, 4)

The order of the group is 1974067200

The group has 4 orbits on five plus one configurations in PG(3, 4).

Of these, 1 impose 19 conditions.

Of these, 1 are associated with double sixes. They are:

0/1 is orbit 3/4 $\{0, 3, 56, 80, 93\}_{120}$ orbit length 46080

The overall number of five plus one configurations associated with double sixes in PG(3, 4) is: 46080

Flag orbits for double sixes

The number of primary orbits below is 4

The number of primary orbits above is 1

The number of flag orbits is 1

The flag orbits are:

- (1) Flag orbit 0 / 1 down=(3,0) up=(0,-1) is (0, 3, 56, 80, 93, 16, 340, 38, 61, 156, 0, 16, 340, 38, 61, 156, 165, 155, 72, 54, 25, 356, 0) with a stabilizer of order 120

Strong generators for a group of order 120:

$$\begin{aligned} & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ 0 & 0 & \omega^2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega & 0 & 0 \\ \omega^2 & 0 & \omega & 0 \\ 0 & \omega^2 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 1 & 0 & 0 & 0 \\ \omega^2 & \omega & 0 & 0 \\ 0 & 0 & \omega & 0 \\ 0 & 0 & \omega^2 & 1 \end{bmatrix}_1, \\ & \begin{bmatrix} 1 & 0 & 0 & 0 \\ \omega^2 & \omega^2 & 0 & 0 \\ 0 & 0 & \omega^2 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ \omega^2 & \omega & 0 & 0 \\ \omega & 0 & \omega & 0 \\ \omega^2 & 1 & \omega^2 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 1 & 0 & 0 \\ \omega^2 & 0 & 0 & 0 \\ 0 & 0 & \omega^2 & \omega^2 \\ 0 & 0 & 1 & 0 \end{bmatrix}_1, \\ & \begin{bmatrix} 1 & \omega & 0 & 0 \\ 0 & 1 & 0 & 0 \\ \omega & \omega & 1 & \omega^2 \\ 0 & \omega^2 & 0 & 1 \end{bmatrix}_1 \end{aligned}$$

1,0,0,0,0,3,0,0,0,0,3,0,0,0,1,1,
1,0,0,0,0,2,0,0,3,0,2,0,0,3,0,1,1,
1,0,0,0,3,2,0,0,0,0,2,0,0,0,3,1,1,
1,0,0,0,3,3,0,0,0,0,3,0,0,0,1,1,0,
1,0,0,0,3,2,0,0,2,0,2,0,3,1,3,1,0,
1,1,0,0,3,0,0,0,0,0,3,3,0,0,1,0,1,

1,2,0,0,0,1,0,0,2,2,1,3,0,3,0,1,1,
nb received = 0

Double Sixes

The order of the group is 1974067200

The group has 1 orbits:

The orbits are:

- (1) 0/1 {16, 340, 38, 61, 156, 165, 155, 72, 54, 25, 356, 0}₁₄₄₀ orbit length 1370880
Strong generators for a group of order 1440:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ 0 & 0 & \omega^2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ \omega^2 & 0 & \omega^2 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ \omega^2 & 0 & \omega & 0 \\ \omega & \omega & 0 & 0 \\ \omega^2 & \omega^2 & 1 & 1 \end{bmatrix}_0,$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ \omega & 0 & \omega & 0 \\ 1 & \omega & 0 & 0 \\ \omega & 1 & \omega & 1 \end{bmatrix}_1, \begin{bmatrix} 0 & 0 & \omega^2 & 0 \\ 0 & 0 & 1 & \omega^2 \\ \omega^2 & 0 & \omega & 0 \\ \omega & \omega^2 & \omega & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 1 & 0 & 0 \\ \omega^2 & 0 & 0 & 0 \\ 0 & 0 & \omega^2 & \omega^2 \\ 0 & 0 & 1 & 0 \end{bmatrix}_1$$

1,0,0,0,0,3,0,0,0,0,3,0,0,0,0,1,1,
1,0,0,0,0,3,0,0,3,0,3,0,0,1,0,1,0,
1,0,0,0,3,0,2,0,2,2,0,0,3,3,1,1,0,
1,0,0,0,2,0,2,0,1,2,0,0,2,1,2,1,1,
0,0,1,0,0,0,2,1,1,0,3,0,3,1,3,2,0,
1,1,0,0,3,0,0,0,0,0,3,3,0,0,1,0,1,

The overall number of objects is: 1370880

Flag orbits for surfaces

The number of primary orbits below is 1

The number of primary orbits above is 1

The number of flag orbits is 1

The flag orbits are:

- (1) Flag orbit 0 / 1 down=(0,0) up=(0,-1) is (16, 340, 38, 61, 156, 165, 155, 72, 54, 25, 356, 0, 71, 55, 26, 100, 1, 138, 109, 345, 84, 85, 122, 110, 145, 139, 81) with a stabilizer of order 1440
Strong generators for a group of order 1440:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ 0 & 0 & \omega^2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ \omega^2 & 0 & \omega^2 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ \omega^2 & 0 & \omega & 0 \\ \omega & \omega & 0 & 0 \\ \omega^2 & \omega^2 & 1 & 1 \end{bmatrix}_0,$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ \omega & 0 & \omega & 0 \\ 1 & \omega & 0 & 0 \\ \omega & 1 & \omega & 1 \end{bmatrix}_1, \begin{bmatrix} 0 & 0 & \omega^2 & 0 \\ 0 & 0 & 1 & \omega^2 \\ \omega^2 & 0 & \omega & 0 \\ \omega & \omega^2 & \omega & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 1 & 0 & 0 \\ \omega^2 & 0 & 0 & 0 \\ 0 & 0 & \omega^2 & \omega^2 \\ 0 & 0 & 1 & 0 \end{bmatrix}_1$$

1,0,0,0,0,3,0,0,0,0,3,0,0,0,1,1,
 1,0,0,0,0,3,0,0,3,0,3,0,0,1,0,1,0,
 1,0,0,0,3,0,2,0,2,2,0,0,3,3,1,1,0,
 1,0,0,0,2,0,2,0,1,2,0,0,2,1,2,1,1,
 0,0,1,0,0,0,2,1,1,0,3,0,3,1,3,2,0,
 1,1,0,0,3,0,0,0,0,0,3,3,0,0,1,0,1,
 nb received = 0

Surfaces

The order of the group is 1974067200

The group has 1 orbits:

The orbits are:

- (1) 0/1 {16, 340, 38, 61, 156, 165, 155, 72, 54, 25, 356, 0, 71, 55, 26, 100, 1, 138, 109, 345, 84, 85, 122, 110, 145, 139, 81}₅₁₈₄₀ orbit length 38080

Strong generators for a group of order 51840:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega & 0 & 0 \\ 0 & 0 & \omega & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ 0 & 0 & \omega^2 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}_0,$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ \omega^2 & \omega & \omega & 0 \\ 0 & 0 & \omega & 0 \\ 1 & 0 & \omega^2 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & \omega & 0 \\ \omega & \omega & 0 & 0 \\ \omega & \omega & 1 & 1 \end{bmatrix}_0,$$

$$\begin{bmatrix} \omega^2 & \omega & \omega^2 & 1 \\ \omega^2 & 0 & 1 & 0 \\ \omega & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}_0, \begin{bmatrix} \omega & \omega & 1 & 1 \\ 0 & 1 & 0 & \omega \\ \omega & \omega^2 & 0 & \omega \\ 0 & 1 & 0 & 0 \end{bmatrix}_0$$

1,0,0,0,0,1,0,0,0,0,1,0,0,0,1,1,
 1,0,0,0,0,2,0,0,0,0,2,0,0,0,1,0,
 1,0,0,0,0,3,0,0,0,0,3,0,1,0,0,1,0,
 1,0,0,0,0,1,0,0,1,1,1,0,1,1,0,1,0,
 1,0,0,0,3,2,2,0,0,0,2,0,1,0,3,1,0,
 1,0,0,0,1,0,2,0,2,2,0,0,2,2,1,1,0,
 1,3,1,2,1,0,2,0,3,2,0,0,2,0,0,0,0,
 1,1,3,3,0,3,0,1,1,2,0,1,0,3,0,0,0,

The overall number of objects is: 38080

The Group PGL(4, 4)

The order of the group is 1974067200

Cubic Surfaces with 27 Lines in PG(3, 4)

The order of the group is 1974067200

The group has 1 orbits:

The orbits are:

- (1) 0/1 {16, 340, 38, 61, 156, 165, 155, 72, 54, 25, 356, 0, 71, 55, 26, 100, 1, 138, 109, 345, 84, 85, 122, 110, 145, 139, 81}₅₁₈₄₀ orbit length 38080

Strong generators for a group of order 51840:

$$\begin{aligned} & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega & 0 & 0 \\ 0 & 0 & \omega & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ 0 & 0 & \omega^2 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}_0, \\ & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ \omega^2 & \omega & \omega & 0 \\ 0 & 0 & \omega & 0 \\ 1 & 0 & \omega^2 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & \omega & 0 \\ \omega & \omega & 0 & 0 \\ \omega & \omega & 1 & 1 \end{bmatrix}_0, \\ & \begin{bmatrix} \omega^2 & \omega & \omega^2 & 1 \\ \omega^2 & 0 & 1 & 0 \\ \omega & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}_0, \begin{bmatrix} \omega & \omega & 1 & 1 \\ 0 & 1 & 0 & \omega \\ \omega & \omega^2 & 0 & \omega \\ 0 & 1 & 0 & 0 \end{bmatrix}_0 \end{aligned}$$

1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,1,
1,0,0,0,0,2,0,0,0,0,2,0,0,0,0,1,0,
1,0,0,0,0,3,0,0,0,0,3,0,1,0,0,1,0,
1,0,0,0,0,1,0,0,1,1,1,0,1,1,0,1,0,
1,0,0,0,3,2,2,0,0,0,2,0,1,0,3,1,0,
1,0,0,0,1,0,2,0,2,2,0,0,2,2,1,1,0,
1,3,1,2,1,0,2,0,3,2,0,0,2,0,0,0,0,
1,1,3,3,0,3,0,1,1,2,0,1,0,3,0,0,0,

The overall number of objects is: 38080

Surface 4#0**The equation**

The equation of the surface is :

$$X_0^2 X_3 + X_1^2 X_2 + X_1 X_2^2 + X_0 X_3^2 = 0$$

(0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0)

Number of points on the surface 45

The automorphism group of the surface has order 51840

The automorphism group is the following group

Strong generators for a group of order 51840:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega & 0 & 0 \\ 0 & 0 & \omega & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ 0 & 0 & \omega^2 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}_0,$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ \omega^2 & \omega & \omega & 0 \\ 0 & 0 & \omega & 0 \\ 1 & 0 & \omega^2 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & \omega & 0 \\ \omega & \omega & 0 & 0 \\ \omega & \omega & 1 & 1 \end{bmatrix}_0,$$

$$\begin{bmatrix} \omega^2 & \omega & \omega^2 & 1 \\ \omega^2 & 0 & 1 & 0 \\ \omega & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}_0, \begin{bmatrix} \omega & \omega & 1 & 1 \\ 0 & 1 & 0 & \omega \\ \omega & \omega^2 & 0 & \omega \\ 0 & 1 & 0 & 0 \end{bmatrix}_0$$

1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,1,
1,0,0,0,0,2,0,0,0,0,2,0,0,0,0,1,0,
1,0,0,0,0,3,0,0,0,0,3,0,1,0,0,1,0,
1,0,0,0,0,1,0,0,1,1,1,0,1,1,0,1,0,
1,0,0,0,3,2,2,0,0,0,2,0,1,0,3,1,0,
1,0,0,0,1,0,2,0,2,2,0,0,2,2,1,1,0,
1,3,1,2,1,0,2,0,3,2,0,0,2,0,0,0,0,
1,1,3,3,0,3,0,1,1,2,0,1,0,3,0,0,0,

General information

Points on lines:

$$5^{27}$$

Lines on points:

$$3^{45}$$

The 27 Lines

$$\ell_0 = a_1 = \begin{bmatrix} 1 & 0 & \omega^2 & 0 \\ 0 & 1 & 1 & \omega \end{bmatrix}_{72} = \begin{bmatrix} 1 & 0 & 3 & 0 \\ 0 & 1 & 1 & 2 \end{bmatrix}_{72} = \mathbf{Pl}(3, 2, 3, 0, 3, 1)_{308}$$

$$\ell_1 = a_2 = \begin{bmatrix} 1 & 0 & \omega & 0 \\ 0 & 1 & 0 & \omega^2 \end{bmatrix}_{54} = \begin{bmatrix} 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 3 \end{bmatrix}_{54} = \mathbf{Pl}(2, 3, 0, 0, 2, 1)_{238}$$

$$\ell_2 = a_3 = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}_{25} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}_{25} = \mathbf{Pl}(1, 1, 0, 0, 1, 1)_{177}$$

$$\ell_3 = a_4 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_{356} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_{356} = \mathbf{Pl}(0, 1, 0, 0, 0, 0)_1$$

$$\begin{aligned}
\ell_4 = a_5 &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}_0 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}_0 = \mathbf{Pl}(1, 0, 0, 0, 0, 0)_0 \\
\ell_5 = a_6 &= \begin{bmatrix} 1 & 0 & \omega^2 & 1 \\ 0 & 1 & 0 & \omega \end{bmatrix}_{155} = \begin{bmatrix} 1 & 0 & 3 & 1 \\ 0 & 1 & 0 & 2 \end{bmatrix}_{155} = \mathbf{Pl}(3, 2, 0, 2, 3, 1)_{314} \\
\ell_6 = b_1 &= \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_{340} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_{340} = \mathbf{Pl}(0, 0, 0, 1, 0, 0)_9 \\
\ell_7 = b_2 &= \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}_{38} = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}_{38} = \mathbf{Pl}(0, 0, 1, 1, 1, 1)_{198} \\
\ell_8 = b_3 &= \begin{bmatrix} 1 & \omega & 0 & 0 \\ 0 & 0 & 1 & \omega^2 \end{bmatrix}_{61} = \begin{bmatrix} 1 & 2 & 0 & 0 \\ 0 & 0 & 1 & 3 \end{bmatrix}_{61} = \mathbf{Pl}(0, 0, 2, 3, 2, 1)_{265} \\
\ell_9 = b_4 &= \begin{bmatrix} 1 & 0 & \omega^2 & 1 \\ 0 & 1 & 1 & \omega \end{bmatrix}_{156} = \begin{bmatrix} 1 & 0 & 3 & 1 \\ 0 & 1 & 1 & 2 \end{bmatrix}_{156} = \mathbf{Pl}(3, 0, 3, 2, 3, 1)_{335} \\
\ell_{10} = b_5 &= \begin{bmatrix} 1 & \omega^2 & 0 & 1 \\ 0 & 0 & 1 & \omega \end{bmatrix}_{165} = \begin{bmatrix} 1 & 3 & 0 & 1 \\ 0 & 0 & 1 & 2 \end{bmatrix}_{165} = \mathbf{Pl}(0, 2, 3, 2, 3, 1)_{337} \\
\ell_{11} = b_6 &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}_{16} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}_{16} = \mathbf{Pl}(0, 0, 1, 0, 0, 0)_2 \\
\ell_{12} = c_{12} &= \begin{bmatrix} 1 & 0 & \omega & 1 \\ 0 & 1 & 0 & \omega^2 \end{bmatrix}_{138} = \begin{bmatrix} 1 & 0 & 2 & 1 \\ 0 & 1 & 0 & 3 \end{bmatrix}_{138} = \mathbf{Pl}(2, 3, 0, 3, 2, 1)_{256} \\
\ell_{13} = c_{13} &= \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \end{bmatrix}_{109} = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \end{bmatrix}_{109} = \mathbf{Pl}(1, 1, 0, 1, 1, 1)_{189} \\
\ell_{14} = c_{14} &= \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_{345} = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_{345} = \mathbf{Pl}(0, 1, 0, 1, 0, 0)_{13} \\
\ell_{15} = c_{15} &= \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix}_{84} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix}_{84} = \mathbf{Pl}(1, 0, 0, 1, 0, 0)_{10} \\
\ell_{16} = c_{16} &= \begin{bmatrix} 1 & 0 & \omega^2 & 0 \\ 0 & 1 & 0 & \omega \end{bmatrix}_{71} = \begin{bmatrix} 1 & 0 & 3 & 0 \\ 0 & 1 & 0 & 2 \end{bmatrix}_{71} = \mathbf{Pl}(3, 2, 0, 0, 3, 1)_{299} \\
\ell_{17} = c_{23} &= \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}_{85} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}_{85} = \mathbf{Pl}(1, 1, 1, 1, 0, 0)_{16} \\
\ell_{18} = c_{24} &= \begin{bmatrix} 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}_{122} = \begin{bmatrix} 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}_{122} = \mathbf{Pl}(0, 1, 1, 1, 1, 1)_{202} \\
\ell_{19} = c_{25} &= \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix}_{110} = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix}_{110} = \mathbf{Pl}(1, 0, 1, 1, 1, 1)_{199} \\
\ell_{20} = c_{26} &= \begin{bmatrix} 1 & 0 & \omega & 0 \\ 0 & 1 & 1 & \omega^2 \end{bmatrix}_{55} = \begin{bmatrix} 1 & 0 & 2 & 0 \\ 0 & 1 & 1 & 3 \end{bmatrix}_{55} = \mathbf{Pl}(2, 3, 2, 0, 2, 1)_{244}
\end{aligned}$$

$$\begin{aligned}
\ell_{21} = c_{34} &= \begin{bmatrix} 1 & \omega & 0 & 1 \\ 0 & 0 & 1 & \omega^2 \end{bmatrix}_{145} = \begin{bmatrix} 1 & 2 & 0 & 1 \\ 0 & 0 & 1 & 3 \end{bmatrix}_{145} = \mathbf{Pl}(0, 3, 2, 3, 2, 1)_{271} \\
\ell_{22} = c_{35} &= \begin{bmatrix} 1 & 0 & \omega & 1 \\ 0 & 1 & 1 & \omega^2 \end{bmatrix}_{139} = \begin{bmatrix} 1 & 0 & 2 & 1 \\ 0 & 1 & 1 & 3 \end{bmatrix}_{139} = \mathbf{Pl}(2, 0, 2, 3, 2, 1)_{267} \\
\ell_{23} = c_{36} &= \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix}_{26} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix}_{26} = \mathbf{Pl}(1, 1, 1, 0, 1, 1)_{180} \\
\ell_{24} = c_{45} &= \begin{bmatrix} 1 & \omega^2 & 0 & 0 \\ 0 & 0 & 1 & \omega \end{bmatrix}_{81} = \begin{bmatrix} 1 & 3 & 0 & 0 \\ 0 & 0 & 1 & 2 \end{bmatrix}_{81} = \mathbf{Pl}(0, 0, 3, 2, 3, 1)_{332} \\
\ell_{25} = c_{46} &= \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}_{100} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}_{100} = \mathbf{Pl}(0, 1, 1, 0, 0, 0)_6 \\
\ell_{26} = c_{56} &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{bmatrix}_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{bmatrix}_1 = \mathbf{Pl}(1, 0, 1, 0, 0, 0)_3
\end{aligned}$$

Rank of lines: (72, 54, 25, 356, 0, 155, 340, 38, 61, 156, 165, 16, 138, 109, 345, 84, 71, 85, 122, 110, 55, 145, 139, 26, 81, 100, 1)

Rank of points on Klein quadric: (308, 238, 177, 1, 0, 314, 9, 198, 265, 335, 337, 2, 256, 189, 13, 10, 299, 16, 202, 199, 244, 271, 267, 180, 332, 6, 3)

All Points on surface

The surface has 45 points

Eckardt Points

The surface has 45 Eckardt points:

$$\begin{aligned}
0 : E_{56} &= a_5 \cap b_6 \cap c_{56} = P_0 = P_0 = \mathbf{P}(1, 0, 0, 0) = \mathbf{P}(1, 0, 0, 0), \quad T = 0 \\
1 : E_{51} &= a_5 \cap b_1 \cap c_{15} = P_1 = P_1 = \mathbf{P}(0, 1, 0, 0) = \mathbf{P}(0, 1, 0, 0), \quad T = 4 \\
2 : E_{46} &= a_4 \cap b_6 \cap c_{46} = P_2 = P_2 = \mathbf{P}(0, 0, 1, 0) = \mathbf{P}(0, 0, 1, 0), \quad T = 20 \\
3 : E_{41} &= a_4 \cap b_1 \cap c_{14} = P_3 = P_3 = \mathbf{P}(0, 0, 0, 1) = \mathbf{P}(0, 0, 0, 1), \quad T = 84 \\
4 : E_{32} &= a_3 \cap b_2 \cap c_{23} = P_4 = P_4 = \mathbf{P}(1, 1, 1, 1) = \mathbf{P}(1, 1, 1, 1), \quad T = 27 \\
5 : E_{52} &= a_5 \cap b_2 \cap c_{25} = P_5 = P_5 = \mathbf{P}(1, 1, 0, 0) = \mathbf{P}(1, 1, 0, 0), \quad T = 1 \\
6 : E_{54} &= a_5 \cap b_4 \cap c_{45} = P_6 = P_6 = \mathbf{P}(\omega, 1, 0, 0) = \mathbf{P}(2, 1, 0, 0), \quad T = 2 \\
7 : E_{53} &= a_5 \cap b_3 \cap c_{35} = P_7 = P_7 = \mathbf{P}(\omega^2, 1, 0, 0) = \mathbf{P}(3, 1, 0, 0), \quad T = 3 \\
8 : E_{36} &= a_3 \cap b_6 \cap c_{36} = P_8 = P_8 = \mathbf{P}(1, 0, 1, 0) = \mathbf{P}(1, 0, 1, 0), \quad T = 5 \\
9 : E_{16} &= a_1 \cap b_6 \cap c_{16} = P_9 = P_9 = \mathbf{P}(\omega, 0, 1, 0) = \mathbf{P}(2, 0, 1, 0), \quad T = 10 \\
10 : E_{26} &= a_2 \cap b_6 \cap c_{26} = P_{10} = P_{10} = \mathbf{P}(\omega^2, 0, 1, 0) = \mathbf{P}(3, 0, 1, 0), \quad T = 15 \\
11 : E_{14,23,56} &= c_{14} \cap c_{23} \cap c_{56} = P_{11} = P_{11} = \mathbf{P}(0, 1, 1, 0) = \mathbf{P}(0, 1, 1, 0), \quad T = 9 \\
12 : E_{13,24,56} &= c_{13} \cap c_{24} \cap c_{56} = P_{12} = P_{12} = \mathbf{P}(1, 1, 1, 0) = \mathbf{P}(1, 1, 1, 0), \quad T = 6 \\
13 : E_{65} &= a_6 \cap b_5 \cap c_{56} = P_{13} = P_{13} = \mathbf{P}(\omega, 1, 1, 0) = \mathbf{P}(2, 1, 1, 0), \quad T = 12 \\
14 : E_{12,34,56} &= c_{12} \cap c_{34} \cap c_{56} = P_{14} = P_{14} = \mathbf{P}(\omega^2, 1, 1, 0) = \mathbf{P}(3, 1, 1, 0), \quad T = 18 \\
15 : E_{15,23,46} &= c_{15} \cap c_{23} \cap c_{46} = P_{15} = P_{23} = \mathbf{P}(1, 0, 0, 1) = \mathbf{P}(1, 0, 0, 1), \quad T = 21 \\
16 : E_{31} &= a_3 \cap b_1 \cap c_{13} = P_{16} = P_{26} = \mathbf{P}(0, 1, 0, 1) = \mathbf{P}(0, 1, 0, 1), \quad T = 25 \\
17 : E_{15,24,36} &= c_{15} \cap c_{24} \cap c_{36} = P_{17} = P_{27} = \mathbf{P}(1, 1, 0, 1) = \mathbf{P}(1, 1, 0, 1), \quad T = 22 \\
18 : E_{21} &= a_2 \cap b_1 \cap c_{12} = P_{18} = P_{30} = \mathbf{P}(0, \omega, 0, 1) = \mathbf{P}(0, 2, 0, 1), \quad T = 46 \\
19 : E_{15,26,34} &= c_{15} \cap c_{26} \cap c_{34} = P_{19} = P_{31} = \mathbf{P}(1, \omega, 0, 1) = \mathbf{P}(1, 2, 0, 1), \quad T = 24 \\
20 : E_{61} &= a_6 \cap b_1 \cap c_{16} = P_{20} = P_{34} = \mathbf{P}(0, \omega^2, 0, 1) = \mathbf{P}(0, 3, 0, 1), \quad T = 67
\end{aligned}$$

21 : $E_{15} = a_1 \cap b_5 \cap c_{15} = P_{21} = P_{35} = \mathbf{P}(1, \omega^2, 0, 1) = \mathbf{P}(1, 3, 0, 1)$, $T = 23$
 22 : $E_{42} = a_4 \cap b_2 \cap c_{24} = P_{22} = P_{38} = \mathbf{P}(0, 0, 1, 1) = \mathbf{P}(0, 0, 1, 1)$, $T = 41$
 23 : $E_{13,25,46} = c_{13} \cap c_{25} \cap c_{46} = P_{23} = P_{39} = \mathbf{P}(1, 0, 1, 1) = \mathbf{P}(1, 0, 1, 1)$, $T = 26$
 24 : $E_{14,25,36} = c_{14} \cap c_{25} \cap c_{36} = P_{24} = P_{42} = \mathbf{P}(0, 1, 1, 1) = \mathbf{P}(0, 1, 1, 1)$, $T = 30$
 25 : $E_{62} = a_6 \cap b_2 \cap c_{26} = P_{25} = P_{47} = \mathbf{P}(\omega, \omega, 1, 1) = \mathbf{P}(2, 2, 1, 1)$, $T = 53$
 26 : $E_{25} = a_2 \cap b_5 \cap c_{25} = P_{26} = P_{48} = \mathbf{P}(\omega^2, \omega, 1, 1) = \mathbf{P}(3, 2, 1, 1)$, $T = 80$
 27 : $E_{16,25,34} = c_{16} \cap c_{25} \cap c_{34} = P_{27} = P_{51} = \mathbf{P}(\omega, \omega^2, 1, 1) = \mathbf{P}(2, 3, 1, 1)$, $T = 55$
 28 : $E_{12} = a_1 \cap b_2 \cap c_{12} = P_{28} = P_{52} = \mathbf{P}(\omega^2, \omega^2, 1, 1) = \mathbf{P}(3, 3, 1, 1)$, $T = 79$
 29 : $E_{43} = a_4 \cap b_3 \cap c_{34} = P_{29} = P_{53} = \mathbf{P}(0, 0, \omega, 1) = \mathbf{P}(0, 0, 2, 1)$, $T = 62$
 30 : $E_{12,35,46} = c_{12} \cap c_{35} \cap c_{46} = P_{30} = P_{54} = \mathbf{P}(1, 0, \omega, 1) = \mathbf{P}(1, 0, 2, 1)$, $T = 36$
 31 : $E_{35} = a_3 \cap b_5 \cap c_{35} = P_{31} = P_{59} = \mathbf{P}(\omega, 1, \omega, 1) = \mathbf{P}(2, 1, 2, 1)$, $T = 49$
 32 : $E_{63} = a_6 \cap b_3 \cap c_{36} = P_{32} = P_{60} = \mathbf{P}(\omega^2, 1, \omega, 1) = \mathbf{P}(3, 1, 2, 1)$, $T = 76$
 33 : $E_{14,26,35} = c_{14} \cap c_{26} \cap c_{35} = P_{33} = P_{61} = \mathbf{P}(0, \omega, \omega, 1) = \mathbf{P}(0, 2, 2, 1)$, $T = 51$
 34 : $E_{23} = a_2 \cap b_3 \cap c_{23} = P_{34} = P_{62} = \mathbf{P}(1, \omega, \omega, 1) = \mathbf{P}(1, 2, 2, 1)$, $T = 39$
 35 : $E_{13} = a_1 \cap b_3 \cap c_{13} = P_{35} = P_{67} = \mathbf{P}(\omega, \omega^2, \omega, 1) = \mathbf{P}(2, 3, 2, 1)$, $T = 50$
 36 : $E_{16,24,35} = c_{16} \cap c_{24} \cap c_{35} = P_{36} = P_{68} = \mathbf{P}(\omega^2, \omega^2, \omega, 1) = \mathbf{P}(3, 3, 2, 1)$, $T = 74$
 37 : $E_{45} = a_4 \cap b_5 \cap c_{45} = P_{37} = P_{69} = \mathbf{P}(0, 0, \omega^2, 1) = \mathbf{P}(0, 0, 3, 1)$, $T = 83$
 38 : $E_{64} = a_6 \cap b_4 \cap c_{46} = P_{38} = P_{70} = \mathbf{P}(1, 0, \omega^2, 1) = \mathbf{P}(1, 0, 3, 1)$, $T = 31$
 39 : $E_{12,36,45} = c_{12} \cap c_{36} \cap c_{45} = P_{39} = P_{75} = \mathbf{P}(\omega, 1, \omega^2, 1) = \mathbf{P}(2, 1, 3, 1)$, $T = 59$
 40 : $E_{34} = a_3 \cap b_4 \cap c_{34} = P_{40} = P_{76} = \mathbf{P}(\omega^2, 1, \omega^2, 1) = \mathbf{P}(3, 1, 3, 1)$, $T = 71$
 41 : $E_{24} = a_2 \cap b_4 \cap c_{24} = P_{41} = P_{79} = \mathbf{P}(\omega, \omega, \omega^2, 1) = \mathbf{P}(2, 2, 3, 1)$, $T = 58$
 42 : $E_{13,26,45} = c_{13} \cap c_{26} \cap c_{45} = P_{42} = P_{80} = \mathbf{P}(\omega^2, \omega, \omega^2, 1) = \mathbf{P}(3, 2, 3, 1)$, $T = 70$
 43 : $E_{14} = a_1 \cap b_4 \cap c_{14} = P_{43} = P_{81} = \mathbf{P}(0, \omega^2, \omega^2, 1) = \mathbf{P}(0, 3, 3, 1)$, $T = 72$
 44 : $E_{16,23,45} = c_{16} \cap c_{23} \cap c_{45} = P_{44} = P_{82} = \mathbf{P}(1, \omega^2, \omega^2, 1) = \mathbf{P}(1, 3, 3, 1)$, $T = 33$
 Set of tangent planes: (0, 4, 20, 84, 27, 1, 2, 3, 5, 10, 15, 9, 6, 12, 18, 21, 25, 22, 46, 24, 67, 23, 41, 26, 30, 53, 80, 55, 79, 62, 36, 49, 76, 51, 39, 50, 74, 83, 31, 59, 71, 58, 70, 72, 33)
 Line type of Eckardt points: $5^{27}, 3^{240}, 1^{90}$
 Plane type of Eckardt points: $13^{45}, 9^{40}$

Hesse planes

Number of Hesse planes: 40

Set of Hesse planes: (7, 8, 11, 13, 14, 16, 17, 19, 28, 29, 32, 34, 35, 37, 38, 40, 42, 43, 44, 45, 47, 48, 52, 54, 56, 57, 60, 61, 63, 64, 65, 66, 68, 69, 73, 75, 77, 78, 81, 82)
 subspace 0 / 40 is 7:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & \omega \end{bmatrix}$$

⋮

subspace 39 / 40 is 82:

$$\begin{bmatrix} 1 & 0 & \omega^2 & 0 \\ 0 & 1 & \omega^2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

0 : 7 : $E_{56}, E_{31}, E_{15,24,36}, E_{16,25,34}, E_{12}, E_{14,26,35}, E_{23}, E_{45}, E_{64}$

⋮

39 : 82 : $E_{41}, E_{52}, E_{16}, E_{12,34,56}, E_{15,24,36}, E_{35}, E_{23}, E_{64}, E_{13,26,45}$

Axes

Number of axes: 240

Axes:

$$0 : 0 = 0, 0 = E_{23}, E_{31}, E_{12}$$

$\vdots$

$$239 : 239 = 119, 1 = E_{12,36,45}, E_{14,26,35}, E_{13,25,46}$$

$\vdots$

Tritangent planes

The 45 tritangent planes are:

$$\begin{aligned} \pi_{12} = \pi_0 = 79 &= \begin{bmatrix} 1 & 0 & 0 & \omega^2 \\ 0 & 1 & 0 & \omega^2 \\ 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 3 \\ 0 & 1 & 0 & 3 \\ 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 3 \\ 0 & 1 & 0 & 3 \\ 0 & 0 & 1 & 1 \end{bmatrix} \\ &= V(\omega^2 X_0 + \omega^2 X_1 + X_2 + X_3) = V(3X_0 + 3X_1 + X_2 + X_3) \end{aligned}$$

dual pt rank = 52 = (3, 3, 1, 1).

$\vdots$

$$\begin{aligned} \pi_{16,25,34} = \pi_{44} = 55 &= \begin{bmatrix} 1 & 0 & 0 & \omega \\ 0 & 1 & 0 & \omega \\ 0 & 0 & 1 & \omega^2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 3 \end{bmatrix} \\ &= V(\omega X_0 + \omega X_1 + \omega^2 X_2 + X_3) = V(2X_0 + 2X_1 + 3X_2 + X_3) \end{aligned}$$

dual pt rank = 79 = (2, 2, 3, 1).

Karaoglu [43] describes a different algorithm, based on non-conical six-arcs and Steiner trihedral pairs. The command

Code Snippet 419

```

1  surface_classify_q4_arc_lifting_two_lines:
2  ▷ $(ORBITER) -v 10 \
3  ▷ ▷ -define F -finite_field -q 4 -end \
4  ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5  ▷ ▷ -define Do -draw_options \
6  ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
7  ▷ ▷ -end \
8  ▷ ▷ -define Control -poset_classification_control \
9  ▷ ▷ ▷ -problem_label sixarcs_q4 -W \
10 ▷ ▷ ▷ -draw_options Do \
11 ▷ ▷ -end \
12 ▷ ▷ -with P -do \
13 ▷ ▷ -projective_space_activity \
14 ▷ ▷ ▷ -control_six_arcs Control \
15 ▷ ▷ ▷ -classify_surfaces_through_arcs_and_two_lines \

```

```

16  ▷ ▷ -end
17  ▷ $(PDFLATEX) surfaces_arc_lifting_4.tex
18  ▷ $(OPEN) surfaces_arc_lifting_4.pdf
19

```

classifies all cubic surfaces with 27 lines over the field $\mathbb{F}_4$ using the algorithm of Karaoglu. The result agrees with the previous algorithm. In $\text{PG}(3, 4)$, the only surface with 27 lines is the Hirschfeld surface.

The classification of cubic surfaces with double sixes can be used to perform isomorphism testing and recognition of surfaces. The surfaces chosen in the classification algorithm are called canonical surfaces. Any surface is isomorphic to exactly one canonical surface. The canonical surface is often called the fanonical form. It is possible to efficiently compute a map that takes a given surface to the associated canonical surface. Suppose we want to compute the canonical surface of $F_{a,b,c,d}$ with $(a, b, c, d) = (25, 5, 5, 25)$ over $\mathbb{F}_{31}$. We can use the following command to create the surface, to classify all surfaces with 27 lines over $\mathbb{F}_{31}$, and to establish an isomorphism between the given surface and the canonical form.

Code Snippet 420

```

1  Family_general_F31_25_5_5_25_recognize:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define Do -draw_options \
4  ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
5  ▷ ▷ -end \
6  ▷ ▷ -define F -finite_field -q 31 -end \
7  ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
8  ▷ ▷ -define Surf -cubic_surface \
9  ▷ ▷ ▷ -space P -family_general_abcd 25 5 5 25 \
10 ▷ ▷ -end \
11 ▷ ▷ -with Surf -do \
12 ▷ ▷ -cubic_surface_activity \
13 ▷ ▷ ▷ -report Do \
14 ▷ ▷ -end \
15 ▷ ▷ -define Control -poset_classification_control \
16 ▷ ▷ ▷ -problem_label "surf_q31" \
17 ▷ ▷ -end \
18 ▷ ▷ -define Classify -classify_cubic_surfaces \
19 ▷ ▷ ▷ -use_double_sixes \
20 ▷ ▷ ▷ -projective_space P \
21 ▷ ▷ ▷ -poset_classification_control Control \
22 ▷ ▷ -end \
23 ▷ ▷ -define Orb -orbits \
24 ▷ ▷ ▷ -on_cubic_surfaces Classify \
25 ▷ ▷ -end \
26 ▷ ▷ -define Do2 -draw_options \
27 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
28 ▷ ▷ -end \
29 ▷ ▷ -define ro \
30 ▷ ▷ ▷ -poset_classification_report_options \
31 ▷ ▷ ▷ ▷ -draw_poset -type_aux \
32 ▷ ▷ ▷ -draw_options Do2 \
33 ▷ ▷ -end \
34 ▷ ▷ -with Orb -do -orbits_activity \

```

```

35  ▷ ▷ ▷ -report "5plus1=off,double_sixes=off,flag_orbits=off,show_group=off,report_
    options=ro" \
36  ▷ ▷ -end \
37  ▷ ▷ -with Orb -do \
38  ▷ ▷ ▷ -orbits_activity \
39  ▷ ▷ ▷ -export_source_code \
40  ▷ ▷ -end \
41  ▷ ▷ -with Orb -do \
42  ▷ ▷ ▷ -cubic_surfaces_after_classification_activity \
43  ▷ ▷ ▷ -recognize Surf \
44  ▷ ▷ -end
45

```

The following command creates all surfaces in the form F_{abcd} over $\mathbb{F}_9$ and identifies the isomorphism type of each surface. The result is written to a csv file.

Code Snippet 421

```

1    surface_F_abcd_identify_q9:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 9 -end \
4    ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5    ▷ ▷ -define Control -poset_classification_control -W \
6    ▷ ▷ ▷ -problem_label "surf_q9" \
7    ▷ ▷ -end \
8    ▷ ▷ -define Classify -classify_cubic_surfaces \
9    ▷ ▷ ▷ -use_double_sixes \
10   ▷ ▷ ▷ -projective_space P \
11   ▷ ▷ ▷ -poset_classification_control Control \
12   ▷ ▷ -end \
13   ▷ ▷ -define Orb -orbits \
14   ▷ ▷ ▷ -on_cubic_surfaces Classify \
15   ▷ ▷ -end \
16   ▷ ▷ -with Orb -do \
17   ▷ ▷ ▷ -cubic_surfaces_after_classification_activity \
18   ▷ ▷ ▷ -identify_general_abcd \
19   ▷ ▷ -end
20

```

The next example illustrates isomorphism testing for smooth cubic surfaces with 27 lines. In order to perform isomorphism testing, the classification of cubic surfaces of that order is required. Let us consider an example over the field $\mathbb{F}_{11}$. Let us see whether the two surfaces of type F_{abcd} with $(a, b, c, d) = (2, 4, 4, 2)$ and $(a, b, c, d) = (5, 10, 7, 5)$ over $\mathbb{F}_{11}$ are isomorphic. Both surfaces have 10 Eckardt points. However, having the same number of Eckardt points is only a necessary condition for surfaces to be isomorphic. It is not a sufficient condition. To determine whether the surfaces are isomorphic, we can use the following command:

Code Snippet 422

```

1    Family_general_F11_iso_test:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 11 -end \

```

```

4    ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5    ▷ ▷ -define Surf1 -cubic_surface \
6    ▷ ▷ ▷ -space P -family_general_abcd 2 4 4 2 \
7    ▷ ▷ -end \
8    ▷ ▷ -define Surf2 -cubic_surface \
9    ▷ ▷ ▷ -space P -family_general_abcd 5 10 7 5 \
10   ▷ ▷ -end \
11   ▷ ▷ -define Control -poset_classification_control -W \
12   ▷ ▷ ▷ -problem_label "surf_q11" \
13   ▷ ▷ -end \
14   ▷ ▷ -define Classify -classify_cubic_surfaces \
15   ▷ ▷ ▷ -use_double_sixes \
16   ▷ ▷ ▷ -projective_space P \
17   ▷ ▷ ▷ -poset_classification_control Control \
18   ▷ ▷ -end \
19   ▷ ▷ -define Orb -orbits \
20   ▷ ▷ ▷ -on_cubic_surfaces Classify \
21   ▷ ▷ -end \
22   ▷ ▷ -with Orb -do \
23   ▷ ▷ ▷ -cubic_surfaces_after_classification_activity \
24   ▷ ▷ ▷ -isomorphism_testing Surf1 Surf2 \
25   ▷ ▷ -end
26

```

The command creates both surfaces, and then performs an isomorphism test for the two given surfaces. The surfaces are found to be isomorphic. A transformation mapping one surface to the other is computed.

Doing isomorphism testing of cubic surfaces is difficult. The approach taken here is the following: Classify all cubic surfaces over the given field and use the data obtained during this classification to compute the canonical form of each of the given surfaces. The surfaces are isomorphic if and only if the respective canonical forms agree. By multiplying the group elements that move each of the surfaces to their associated canonical form (one element inverted), we obtain an isomorphism mapping one to the other. Obviously, the computation of one isomorphism is as costly as the classification of all surfaces. However, once the classification is finished, isomorphisms can be computed in near constant time, as the data from the classification allows to determine a canonizing group element in almost constant time.

It is possible to identify cubic surfaces in special forms. For instance, the Cayley normal form is parametrized by 4 field elements (k, l, m, n) as

$$(X_0^2 + X_1^2 + X_2^2 + X_3^2) X_3 + (aX_0 + bX_1 + cX_2) X_3^2 + kX_0X_1X_2 + dX_0X_1X_3 + eX_0X_2X_3 + fX_1X_2X_3,$$

where

$$a = l + \frac{1}{l}, \quad b = m + \frac{1}{m}, \quad c = n + \frac{1}{n}, \quad d = lm + \frac{1}{lm}, \quad e = nl + \frac{1}{nl}, \quad f = mn + \frac{1}{mn}.$$

The following command identifies the Cayley normal form among all cubic surfaces with 27 lines for $q = 7$.

Code Snippet 423

```

1    surface_classify_q7_sweep.Cayley:
2    ▷ $(ORBITER) -v 5 \
3    ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4    ▷ ▷ -define F -finite_field -q 7 -end \
5    ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \

```

```

6    ▷ ▷ -define Control -poset_classification_control -W \
7    ▷ ▷ ▷ -problem_label "surf_q7" \
8    ▷ ▷ -end \
9    ▷ ▷ -define Classify -classify_cubic_surfaces \
10   ▷ ▷ ▷ -use_double_sixes \
11   ▷ ▷ ▷ -projective_space P \
12   ▷ ▷ ▷ -poset_classification_control Control \
13   ▷ ▷ -end \
14   ▷ ▷ -define Orb -orbits \
15   ▷ ▷ ▷ -on_cubic_surfaces Classify \
16   ▷ ▷ -end \
17   ▷ ▷ -define Do -draw_options \
18   ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
19   ▷ ▷ -end \
20   ▷ ▷ -define ro \
21   ▷ ▷ ▷ -poset_classification_report_options \
22   ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
23   ▷ ▷ ▷ -draw_options Do \
24   ▷ ▷ -end \
25   ▷ ▷ -with Orb -do -orbits_activity \
26   ▷ ▷ ▷ -report "5plus1=off,double_sixes=off,flag_orbits=off,show_group=off,report_
options=ro" \
27   ▷ ▷ -end \
28   ▷ ▷ -with Orb -do \
29   ▷ ▷ ▷ -orbits_activity \
30   ▷ ▷ ▷ -export_source_code \
31   ▷ ▷ -end \
32   ▷ ▷ -with Orb -do \
33   ▷ ▷ ▷ -cubic_surfaces_after_classification_activity \
34   ▷ ▷ ▷ -sweep_Cayley \
35   ▷ ▷ -end
36   ▷ $(PDFLATEX) Surfaces.q7.tex
37   ▷ $(OPEN) Surfaces.q7.pdf
38

```

The command writes two csv files. The first maps (k, l, m, n) to isomorphism types of surfaces. The second file writes for each isomorphism type a suitable (k, l, m, n) pair. There is only one cubic surface over $\mathbb{F}_7$. The computation shows that it can be obtained in the Cayley form with parameters $(k, l, m, n) = (1, 2, 2, 2)$. This is not the only way. In total, there are 64 parameter tuples which all produce isomorphic copies of the same surface.

Creating Quartic Curves		
Command	Arguments	Purpose
-space	P	Specify the 2-dimensional projective space $P = PG(2, q)$.
-label_txt	label	Override the ascii label of the curve.
-label_tex	label	Override the latex label of the curve.
-label_for_summary	label	Override the ascii label of the curve, to be used in summary commands.
-catalogue	OCN	Create the quartic curve from the Orbiter catalogue with the given Orbiter catalogue number.
-by_coefficients	coeffs	Create a quartic curve given the coefficients of the equation.
-by_normal_form	abcdef	Create a quartic curve in the 6-parameter normal form.
-by_equation	eqn	Create a quartic curve from an equation.
-by_symbolic_object	R equation	Create a quartic curve from an equation using the polynomial ring R . The equation is given as a symbolic object.
-from_cubic_surface	S i	Create the quartic curve from i th orbit of the automorphism group of the surface S on points not on lines (i is zero based).
-from_variety	V	Create the quartic curve defined by the variety V .
-override_group	descr	Override the automorphism group of the curve by the given group.
-transform	elt	Apply the transformation given by the group element.
-transform_inverse	elt	Apply the inverse transformation given by the group element.

Table 9.9: Creating Quartic Curves

9.5 Quartic Curves

Cubic surfaces with 27 lines are associated with quartic curves with 28 bitangents (see [36]), which in turn are associated with del Pezzo surfaces. Orbiter can classify quartic curves based on a known classification of cubic surfaces. Orbiter also has a catalogue of quartic curves for small field sizes.

Table 9.9 lists options to create a quartic curve object.

Table 9.10 lists activities for a quartic curve object.

Table 9.11 lists options for exporting data.

Suppose we want to create a quartic curve. We can utilize the Orbiter catalogue of isomorphism types of quartic curves. The following command creates the unique curve over the field $\mathbb{F}_9$. The curve is isomorphic to the Hermitian curve.

Quartic Curve Activities		
Command	Arguments	Purpose
-report		Produce a latex report about the curve.
-export		Export data of the quartic curve to a csv file.
-export_something	type	Export selected data of the quartic curve to a file. The type of data to be exported must be specified according to Table 9.11.
-create_surface		Create a cubic surface from the curve.
-extract_orbit_on_bitangents_by_length	l	Extract the bitangents in the unique orbit of length l . If there is no orbit of length l , or if there are multiple orbits of length l , an error is raised.
-extract_specific_orbit_on_bitangents_by_length	$l\ i$	Extract the i th orbit on bitangents of length l .
-extract_specific_orbit_on_kovalevski_points_by_length	$l\ i$	Extract the i th orbit on Kovalevski points of length l .
-get_Kovalevski_configuration		Extract the incidence structure between Kovalevski points and bitangents.

Table 9.10: Quartic Curve Activities

Options for Exporting Data: Quartic Curves		
Label		What
points		The points on the quartic curve.
equation		The equation of the quartic curves as a coefficient vector of length 15.
bitangents		The 28 bitangents.
Kovalevski_points		The Kovalevski points (points off the curve where 4 bitangents meet).
singular_points		The singular points of the curve.

Table 9.11: Options for Exporting Data: Quartic Curves

Code Snippet 424

```

1   quartic_curve_9_0:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 9 -end \
4   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5   ▷ ▷ -define C -quartic_curve -space P -catalogue 0 -end
6

```

The following command creates the same curve and produces a report:

Code Snippet 425

```

1   quartic_curve_9_0_report:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 9 -end \
4   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5   ▷ ▷ -define C -quartic_curve -space P -catalogue 0 -end \
6   ▷ ▷ -with C -do \
7   ▷ ▷ ▷ -quartic_curve_activity \
8   ▷ ▷ ▷ ▷ -report \
9   ▷ ▷ ▷ -end
10  ▷ $(PDFLATEX) quartic_curve_catalogue_q9_iso0_report.tex
11  ▷ $(OPEN) quartic_curve_catalogue_q9_iso0_report.pdf
12

```

The report contains the following information:

The equation

The equation of the quartic curve is :

$$\alpha^3 X_0^3 X_1 + \alpha^4 X_0^3 X_2 + \alpha^7 X_0 X_1^3 + \alpha^6 X_1^3 X_2 + \alpha^2 X_0 X_2^3 + X_1 X_2^3$$

$$(0, 0, 0, 8, 2, 4, 5, 7, 1, 0, 0, 0, 0, 0, 0)$$

The gradient

The gradient of the quartic curve is :

$$\alpha^7 X_1^3 + \alpha^2 X_2^3$$

$$(0, 4, 7, 0, 0, 0, 0, 0, 0, 0)$$

$$\alpha^3 X_0^3 + X_2^3$$

$$(8, 0, 1, 0, 0, 0, 0, 0, 0, 0)$$

$$\alpha^4 X_0^3 + \alpha^6 X_1^3$$

$$(2, 5, 0, 0, 0, 0, 0, 0, 0, 0)$$

General information

Number of bitangents	28
Number of points	28
Fullness	is full
Number of Kovalevski points	63
Bitangent line type (a_0, a_1, a_2)	$(0, 28, 0)$
Number of singular points	0

All points on the curve

The surface has 28 points:

The points on the quartic curve are:

0 : $P_0 = (1, 0, 0)$	10 : $P_{30} = (2, 2, 1)$	20 : $P_{58} = (3, 5, 1)$
1 : $P_1 = (0, 1, 0)$	11 : $P_{32} = (4, 2, 1)$	21 : $P_{62} = (7, 5, 1)$
2 : $P_2 = (0, 0, 1)$	12 : $P_{34} = (6, 2, 1)$	22 : $P_{76} = (3, 7, 1)$
3 : $P_3 = (1, 1, 1)$	13 : $P_{38} = (1, 3, 1)$	23 : $P_{77} = (4, 7, 1)$
4 : $P_4 = (1, 1, 0)$	14 : $P_{41} = (4, 3, 1)$	24 : $P_{78} = (5, 7, 1)$
5 : $P_5 = (2, 1, 0)$	15 : $P_{44} = (7, 3, 1)$	25 : $P_{82} = (0, 8, 1)$
6 : $P_{14} = (3, 0, 1)$	16 : $P_{46} = (0, 4, 1)$	26 : $P_{83} = (1, 8, 1)$
7 : $P_{17} = (6, 0, 1)$	17 : $P_{51} = (5, 4, 1)$	27 : $P_{84} = (2, 8, 1)$
8 : $P_{24} = (5, 1, 1)$	18 : $P_{53} = (7, 4, 1)$	
9 : $P_{25} = (6, 1, 1)$	19 : $P_{57} = (2, 5, 1)$	

The points by rank are: (0, 1, 2, 3, 4, 5, 14, 17, 24, 25, 30, 32, 34, 38, 41, 44, 46, 51, 53, 57, 58, 62, 76, 77, 78, 82, 83, 84)

The Kovalevski points are:

0 : $P_7 = (4, 1, 0) = c_{13} \cap c_{14} \cap c_{36} \cap c_{46}$
1 : $P_8 = (5, 1, 0) = a_2 \cap a_4 \cap c_{25} \cap c_{45}$
2 : $P_9 = (6, 1, 0) = b_1 \cap b_6 \cap c_{12} \cap c_{26}$
3 : $P_{10} = (7, 1, 0) = a_3 \cap b_5 \cap c_{35} \cap d$
4 : $P_{11} = (8, 1, 0) = b_2 \cap b_3 \cap c_{24} \cap c_{34}$
5 : $P_{12} = (1, 0, 1) = a_3 \cap a_4 \cap c_{23} \cap c_{24}$
6 : $P_{13} = (2, 0, 1) = c_{34} \cap c_{36} \cap c_{45} \cap c_{56}$
7 : $P_{15} = (4, 0, 1) = b_3 \cap b_6 \cap c_{13} \cap c_{16}$
8 : $P_{16} = (5, 0, 1) = a_5 \cap a_6 \cap c_{25} \cap c_{26}$
9 : $P_{18} = (7, 0, 1) = a_2 \cap b_1 \cap c_{35} \cap c_{46}$
10 : $P_{19} = (8, 0, 1) = b_4 \cap b_5 \cap c_{14} \cap c_{15}$
11 : $P_{20} = (0, 1, 1) = a_2 \cap b_3 \cap c_{14} \cap c_{56}$
12 : $P_{21} = (2, 1, 1) = b_2 \cap b_4 \cap c_{26} \cap c_{46}$
13 : $P_{22} = (3, 1, 1) = a_4 \cap b_5 \cap c_{12} \cap c_{36}$
14 : $P_{23} = (4, 1, 1) = a_6 \cap b_1 \cap c_{23} \cap c_{45}$
15 : $P_{26} = (7, 1, 1) = c_{16} \cap c_{25} \cap c_{34} \cap d$
16 : $P_{27} = (8, 1, 1) = a_3 \cap a_5 \cap c_{13} \cap c_{15}$
17 : $P_{28} = (0, 2, 1) = c_{12} \cap c_{13} \cap c_{25} \cap c_{35}$
18 : $P_{29} = (1, 2, 1) = b_1 \cap b_5 \cap c_{16} \cap c_{56}$
19 : $P_{31} = (3, 2, 1) = a_3 \cap a_6 \cap c_{34} \cap c_{46}$
20 : $P_{33} = (5, 2, 1) = a_2 \cap b_4 \cap c_{24} \cap d$

- 21 : $P_{35} = (7, 2, 1) = b_2 \cap b_6 \cap c_{23} \cap c_{36}$
- 22 : $P_{36} = (8, 2, 1) = a_4 \cap b_3 \cap c_{15} \cap c_{26}$
- 23 : $P_{37} = (0, 3, 1) = a_5 \cap b_1 \cap c_{24} \cap c_{36}$
- 24 : $P_{39} = (2, 3, 1) = a_2 \cap a_6 \cap c_{12} \cap c_{16}$
- 25 : $P_{40} = (3, 3, 1) = b_3 \cap b_4 \cap c_{35} \cap c_{45}$
- 26 : $P_{42} = (5, 3, 1) = a_4 \cap b_2 \cap c_{13} \cap c_{56}$
- 27 : $P_{43} = (6, 3, 1) = a_3 \cap b_6 \cap c_{14} \cap c_{25}$
- 28 : $P_{45} = (8, 3, 1) = c_{15} \cap c_{23} \cap c_{46} \cap d$
- 29 : $P_{47} = (1, 4, 1) = a_6 \cap b_2 \cap c_{14} \cap c_{35}$
- 30 : $P_{48} = (2, 4, 1) = b_3 \cap b_5 \cap c_{23} \cap c_{25}$
- 31 : $P_{49} = (3, 4, 1) = a_5 \cap b_6 \cap c_{56} \cap d$
- 32 : $P_{50} = (4, 4, 1) = a_2 \cap a_3 \cap c_{26} \cap c_{36}$
- 33 : $P_{52} = (6, 4, 1) = b_1 \cap b_4 \cap c_{13} \cap c_{34}$
- 34 : $P_{54} = (8, 4, 1) = c_{12} \cap c_{15} \cap c_{24} \cap c_{45}$
- 35 : $P_{55} = (0, 5, 1) = a_4 \cap a_6 \cap b_4 \cap b_6$
- 36 : $P_{56} = (1, 5, 1) = c_{13} \cap c_{26} \cap c_{45} \cap d$
- 37 : $P_{59} = (4, 5, 1) = c_{24} \cap c_{25} \cap c_{46} \cap c_{56}$
- 38 : $P_{60} = (5, 5, 1) = c_{12} \cap c_{14} \cap c_{23} \cap c_{34}$
- 39 : $P_{61} = (6, 5, 1) = a_2 \cap a_5 \cap b_2 \cap b_5$
- 40 : $P_{63} = (8, 5, 1) = c_{15} \cap c_{16} \cap c_{35} \cap c_{36}$
- 41 : $P_{64} = (0, 6, 1) = a_1 \cap b_5 \cap c_{26} \cap c_{34}$
- 42 : $P_{65} = (1, 6, 1) = a_1 \cap b_4 \cap c_{25} \cap c_{36}$
- 43 : $P_{66} = (2, 6, 1) = a_1 \cap b_6 \cap c_{24} \cap c_{35}$
- 44 : $P_{67} = (3, 6, 1) = a_1 \cap a_2 \cap c_{13} \cap c_{23}$
- 45 : $P_{68} = (4, 6, 1) = a_1 \cap b_2 \cap c_{12} \cap d$
- 46 : $P_{69} = (5, 6, 1) = a_1 \cap a_3 \cap b_1 \cap b_3$
- 47 : $P_{70} = (6, 6, 1) = a_1 \cap a_4 \cap c_{16} \cap c_{46}$
- 48 : $P_{71} = (7, 6, 1) = a_1 \cap a_5 \cap c_{14} \cap c_{45}$
- 49 : $P_{72} = (8, 6, 1) = a_1 \cap a_6 \cap c_{15} \cap c_{56}$
- 50 : $P_{73} = (0, 7, 1) = a_3 \cap b_2 \cap c_{16} \cap c_{45}$
- 51 : $P_{74} = (1, 7, 1) = a_5 \cap b_3 \cap c_{12} \cap c_{46}$
- 52 : $P_{75} = (2, 7, 1) = a_4 \cap b_1 \cap c_{14} \cap d$
- 53 : $P_{79} = (6, 7, 1) = c_{23} \cap c_{26} \cap c_{35} \cap c_{56}$
- 54 : $P_{80} = (7, 7, 1) = a_6 \cap b_5 \cap c_{13} \cap c_{24}$
- 55 : $P_{81} = (8, 7, 1) = a_2 \cap b_6 \cap c_{15} \cap c_{34}$
- 56 : $P_{85} = (3, 8, 1) = c_{14} \cap c_{16} \cap c_{24} \cap c_{26}$
- 57 : $P_{86} = (4, 8, 1) = a_4 \cap a_5 \cap c_{34} \cap c_{35}$
- 58 : $P_{87} = (5, 8, 1) = b_5 \cap b_6 \cap c_{45} \cap c_{46}$
- 59 : $P_{88} = (6, 8, 1) = a_6 \cap b_3 \cap c_{36} \cap d$
- 60 : $P_{89} = (7, 8, 1) = a_3 \cap b_4 \cap c_{12} \cap c_{56}$
- 61 : $P_{90} = (8, 8, 1) = b_1 \cap b_2 \cap c_{15} \cap c_{25}$
- 62 : $P_6 = (3, 1, 0) = a_5 \cap b_4 \cap c_{16} \cap c_{23}$

The Kovalevski points by rank are: (7, 8, 9, 10, 11, 12, 13, 15, 16, 18, 19, 20, 21, 22, 23, 26, 27, 28, 29, 31, 33, 35, 36, 37, 39, 40, 42, 43, 45, 47, 48, 49, 50, 52, 54, 55, 56, 59, 60, 61, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 79, 80, 81, 85, 86, 87, 88, 89, 90, 6)

The points off the curve are:

0 : $P_6 = (3, 1, 0)$	22 : $P_{35} = (7, 2, 1)$	44 : $P_{66} = (2, 6, 1)$
1 : $P_7 = (4, 1, 0)$	23 : $P_{36} = (8, 2, 1)$	45 : $P_{67} = (3, 6, 1)$
2 : $P_8 = (5, 1, 0)$	24 : $P_{37} = (0, 3, 1)$	46 : $P_{68} = (4, 6, 1)$
3 : $P_9 = (6, 1, 0)$	25 : $P_{39} = (2, 3, 1)$	47 : $P_{69} = (5, 6, 1)$
4 : $P_{10} = (7, 1, 0)$	26 : $P_{40} = (3, 3, 1)$	48 : $P_{70} = (6, 6, 1)$
5 : $P_{11} = (8, 1, 0)$	27 : $P_{42} = (5, 3, 1)$	49 : $P_{71} = (7, 6, 1)$
6 : $P_{12} = (1, 0, 1)$	28 : $P_{43} = (6, 3, 1)$	50 : $P_{72} = (8, 6, 1)$
7 : $P_{13} = (2, 0, 1)$	29 : $P_{45} = (8, 3, 1)$	51 : $P_{73} = (0, 7, 1)$
8 : $P_{15} = (4, 0, 1)$	30 : $P_{47} = (1, 4, 1)$	52 : $P_{74} = (1, 7, 1)$
9 : $P_{16} = (5, 0, 1)$	31 : $P_{48} = (2, 4, 1)$	53 : $P_{75} = (2, 7, 1)$
10 : $P_{18} = (7, 0, 1)$	32 : $P_{49} = (3, 4, 1)$	54 : $P_{79} = (6, 7, 1)$
11 : $P_{19} = (8, 0, 1)$	33 : $P_{50} = (4, 4, 1)$	55 : $P_{80} = (7, 7, 1)$
12 : $P_{20} = (0, 1, 1)$	34 : $P_{52} = (6, 4, 1)$	56 : $P_{81} = (8, 7, 1)$
13 : $P_{21} = (2, 1, 1)$	35 : $P_{54} = (8, 4, 1)$	57 : $P_{85} = (3, 8, 1)$
14 : $P_{22} = (3, 1, 1)$	36 : $P_{55} = (0, 5, 1)$	58 : $P_{86} = (4, 8, 1)$
15 : $P_{23} = (4, 1, 1)$	37 : $P_{56} = (1, 5, 1)$	59 : $P_{87} = (5, 8, 1)$
16 : $P_{26} = (7, 1, 1)$	38 : $P_{59} = (4, 5, 1)$	60 : $P_{88} = (6, 8, 1)$
17 : $P_{27} = (8, 1, 1)$	39 : $P_{60} = (5, 5, 1)$	61 : $P_{89} = (7, 8, 1)$
18 : $P_{28} = (0, 2, 1)$	40 : $P_{61} = (6, 5, 1)$	62 : $P_{90} = (8, 8, 1)$
19 : $P_{29} = (1, 2, 1)$	41 : $P_{63} = (8, 5, 1)$	
20 : $P_{31} = (3, 2, 1)$	42 : $P_{64} = (0, 6, 1)$	
21 : $P_{33} = (5, 2, 1)$	43 : $P_{65} = (1, 6, 1)$	

(6, 7, 8, 9, 10, 11, 12, 13, 15, 16, 18, 19, 20, 21, 22, 23, 26, 27, 28, 29, 31, 33, 35, 36, 37, 39, 40, 42, 43, 45, 47, 48, 49, 50, 52, 54, 55, 56, 59, 60, 61, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 79, 80, 81, 85, 86, 87, 88, 89, 90)

The lines and their points of contact are:

$$\begin{aligned}
 a_1 &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & \alpha^3 \end{bmatrix}^8 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 8 \end{bmatrix}^8 P_0 = \mathbf{P}(1, 0, 0) \ 4\times \\
 a_2 &= \begin{bmatrix} 1 & 0 & \alpha^6 \\ 0 & 1 & 1 \end{bmatrix}^{51} = \begin{bmatrix} 1 & 0 & 5 \\ 0 & 1 & 1 \end{bmatrix}^{51} P_{83} = \mathbf{P}(1, 8, 1) \ 4\times \\
 a_3 &= \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & \alpha^6 \end{bmatrix}^{15} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 5 \end{bmatrix}^{15} P_{57} = \mathbf{P}(2, 5, 1) \ 4\times \\
 a_4 &= \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & \alpha^2 \end{bmatrix}^{17} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 7 \end{bmatrix}^{17} P_{53} = \mathbf{P}(7, 4, 1) \ 4\times \\
 a_5 &= \begin{bmatrix} 1 & 0 & \alpha^2 \\ 0 & 1 & \alpha^7 \end{bmatrix}^{74} = \begin{bmatrix} 1 & 0 & 7 \\ 0 & 1 & 4 \end{bmatrix}^{74} P_{30} = \mathbf{P}(2, 2, 1) \ 4\times \\
 a_6 &= \begin{bmatrix} 1 & 0 & \alpha^2 \\ 0 & 1 & \alpha^2 \end{bmatrix}^{77} = \begin{bmatrix} 1 & 0 & 7 \\ 0 & 1 & 7 \end{bmatrix}^{77} P_5 = \mathbf{P}(2, 1, 0) \ 4\times \\
 b_1 &= \begin{bmatrix} 1 & 0 & \alpha^6 \\ 0 & 1 & \alpha^7 \end{bmatrix}^{54} = \begin{bmatrix} 1 & 0 & 5 \\ 0 & 1 & 4 \end{bmatrix}^{54} P_{58} = \mathbf{P}(3, 5, 1) \ 4\times \\
 b_2 &= \begin{bmatrix} 1 & 0 & \alpha^7 \\ 0 & 1 & \alpha^6 \end{bmatrix}^{45} = \begin{bmatrix} 1 & 0 & 4 \\ 0 & 1 & 5 \end{bmatrix}^{45} P_{14} = \mathbf{P}(3, 0, 1) \ 4\times \\
 b_3 &= \begin{bmatrix} 1 & 0 & \alpha \\ 0 & 1 & 1 \end{bmatrix}^{31} = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 1 \end{bmatrix}^{31} P_{62} = \mathbf{P}(7, 5, 1) \ 4\times \\
 b_4 &= \begin{bmatrix} 1 & 0 & \alpha^5 \\ 0 & 1 & \alpha^2 \end{bmatrix}^{67} = \begin{bmatrix} 1 & 0 & 6 \\ 0 & 1 & 7 \end{bmatrix}^{67} P_{77} = \mathbf{P}(4, 7, 1) \ 4\times
 \end{aligned}$$

$$\begin{aligned}
b_5 &= \begin{bmatrix} 1 & 0 & \alpha^5 \\ 0 & 1 & \alpha^3 \end{bmatrix}_{68} = \begin{bmatrix} 1 & 0 & 6 \\ 0 & 1 & 8 \end{bmatrix}_{68} & P_{41} &= \mathbf{P}(4, 3, 1) \ 4\times \\
b_6 &= \begin{bmatrix} 1 & 0 & \alpha \\ 0 & 1 & \alpha^2 \end{bmatrix}_{37} = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 7 \end{bmatrix}_{37} & P_3 &= \mathbf{P}(1, 1, 1) \ 4\times \\
c_{12} &= \begin{bmatrix} 1 & 0 & \alpha^3 \\ 0 & 1 & \alpha^4 \end{bmatrix}_{82} = \begin{bmatrix} 1 & 0 & 8 \\ 0 & 1 & 2 \end{bmatrix}_{82} & P_{17} &= \mathbf{P}(6, 0, 1) \ 4\times \\
c_{13} &= \begin{bmatrix} 1 & 0 & \alpha \\ 0 & 1 & \alpha^4 \end{bmatrix}_{32} = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 2 \end{bmatrix}_{32} & P_{84} &= \mathbf{P}(2, 8, 1) \ 4\times \\
c_{14} &= \begin{bmatrix} 1 & 0 & \alpha^5 \\ 0 & 1 & 1 \end{bmatrix}_{61} = \begin{bmatrix} 1 & 0 & 6 \\ 0 & 1 & 1 \end{bmatrix}_{61} & P_{32} &= \mathbf{P}(4, 2, 1) \ 4\times \\
c_{15} &= \begin{bmatrix} 1 & 0 & \alpha^5 \\ 0 & 1 & 0 \end{bmatrix}_{60} = \begin{bmatrix} 1 & 0 & 6 \\ 0 & 1 & 0 \end{bmatrix}_{60} & P_1 &= \mathbf{P}(0, 1, 0) \ 4\times \\
c_{16} &= \begin{bmatrix} 1 & 0 & \alpha \\ 0 & 1 & \alpha^6 \end{bmatrix}_{35} = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 5 \end{bmatrix}_{35} & P_{51} &= \mathbf{P}(5, 4, 1) \ 4\times \\
c_{23} &= \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & \alpha^5 \end{bmatrix}_{16} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 6 \end{bmatrix}_{16} & P_{82} &= \mathbf{P}(0, 8, 1) \ 4\times \\
c_{24} &= \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & \alpha^7 \end{bmatrix}_{14} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 4 \end{bmatrix}_{14} & P_{25} &= \mathbf{P}(6, 1, 1) \ 4\times \\
c_{25} &= \begin{bmatrix} 1 & 0 & \alpha^2 \\ 0 & 1 & \alpha^4 \end{bmatrix}_{72} = \begin{bmatrix} 1 & 0 & 7 \\ 0 & 1 & 2 \end{bmatrix}_{72} & P_{76} &= \mathbf{P}(3, 7, 1) \ 4\times \\
c_{26} &= \begin{bmatrix} 1 & 0 & \alpha^2 \\ 0 & 1 & \alpha^3 \end{bmatrix}_{78} = \begin{bmatrix} 1 & 0 & 7 \\ 0 & 1 & 8 \end{bmatrix}_{78} & P_{44} &= \mathbf{P}(7, 3, 1) \ 4\times \\
c_{34} &= \begin{bmatrix} 1 & 0 & \alpha^4 \\ 0 & 1 & \alpha^3 \end{bmatrix}_{28} = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 8 \end{bmatrix}_{28} & P_{38} &= \mathbf{P}(1, 3, 1) \ 4\times \\
c_{35} &= \begin{bmatrix} 1 & 0 & \alpha^6 \\ 0 & 1 & \alpha^4 \end{bmatrix}_{52} = \begin{bmatrix} 1 & 0 & 5 \\ 0 & 1 & 2 \end{bmatrix}_{52} & P_{24} &= \mathbf{P}(5, 1, 1) \ 4\times \\
c_{36} &= \begin{bmatrix} 1 & 0 & \alpha^4 \\ 0 & 1 & \alpha^7 \end{bmatrix}_{24} = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 4 \end{bmatrix}_{24} & P_{78} &= \mathbf{P}(5, 7, 1) \ 4\times \\
c_{45} &= \begin{bmatrix} 1 & 0 & \alpha^4 \\ 0 & 1 & \alpha^6 \end{bmatrix}_{25} = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 5 \end{bmatrix}_{25} & P_{34} &= \mathbf{P}(6, 2, 1) \ 4\times \\
c_{46} &= \begin{bmatrix} 1 & 0 & \alpha^6 \\ 0 & 1 & \alpha \end{bmatrix}_{53} = \begin{bmatrix} 1 & 0 & 5 \\ 0 & 1 & 3 \end{bmatrix}_{53} & P_{46} &= \mathbf{P}(0, 4, 1) \ 4\times \\
c_{56} &= \begin{bmatrix} 1 & 0 & \alpha^4 \\ 0 & 1 & 1 \end{bmatrix}_{21} = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \end{bmatrix}_{21} & P_4 &= \mathbf{P}(1, 1, 0) \ 4\times \\
d &= \begin{bmatrix} 1 & \alpha^6 & 0 \\ 0 & 0 & 1 \end{bmatrix}_{59} = \begin{bmatrix} 1 & 5 & 0 \\ 0 & 0 & 1 \end{bmatrix}_{59} & P_2 &= \mathbf{P}(0, 0, 1) \ 4\times
\end{aligned}$$

Rank of lines: (8, 51, 15, 17, 74, 77, 54, 45, 31, 67, 68, 37, 82, 32, 61, 60, 35, 16, 14, 72, 78, 28, 52, 24, 25, 53, 21, 59)

Line type: 1^{28}

28	1	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 0
----	---	--

point types: 1^{28}

28	1	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 0
----	---	--

point types for points off the curve: 4^{63}

63	4	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 0
----	---	--

Lines on points off the curve:

- Off point $0 = P_6 = (3, 1, 0)$ lies on 4 bisecants : $\{ 4, 9, 16, 17 \}$
Off point $1 = P_7 = (4, 1, 0)$ lies on 4 bisecants : $\{ 13, 14, 23, 25 \}$
Off point $2 = P_8 = (5, 1, 0)$ lies on 4 bisecants : $\{ 1, 3, 19, 24 \}$
Off point $3 = P_9 = (6, 1, 0)$ lies on 4 bisecants : $\{ 6, 11, 12, 20 \}$
Off point $4 = P_{10} = (7, 1, 0)$ lies on 4 bisecants : $\{ 2, 10, 22, 27 \}$
Off point $5 = P_{11} = (8, 1, 0)$ lies on 4 bisecants : $\{ 7, 8, 18, 21 \}$
Off point $6 = P_{12} = (1, 0, 1)$ lies on 4 bisecants : $\{ 2, 3, 17, 18 \}$
Off point $7 = P_{13} = (2, 0, 1)$ lies on 4 bisecants : $\{ 21, 23, 24, 26 \}$
Off point $8 = P_{15} = (4, 0, 1)$ lies on 4 bisecants : $\{ 8, 11, 13, 16 \}$
Off point $9 = P_{16} = (5, 0, 1)$ lies on 4 bisecants : $\{ 4, 5, 19, 20 \}$
Off point $10 = P_{18} = (7, 0, 1)$ lies on 4 bisecants : $\{ 1, 6, 22, 25 \}$
Off point $11 = P_{19} = (8, 0, 1)$ lies on 4 bisecants : $\{ 9, 10, 14, 15 \}$
Off point $12 = P_{20} = (0, 1, 1)$ lies on 4 bisecants : $\{ 1, 8, 14, 26 \}$
Off point $13 = P_{21} = (2, 1, 1)$ lies on 4 bisecants : $\{ 7, 9, 20, 25 \}$
Off point $14 = P_{22} = (3, 1, 1)$ lies on 4 bisecants : $\{ 3, 10, 12, 23 \}$
Off point $15 = P_{23} = (4, 1, 1)$ lies on 4 bisecants : $\{ 5, 6, 17, 24 \}$
Off point $16 = P_{26} = (7, 1, 1)$ lies on 4 bisecants : $\{ 16, 19, 21, 27 \}$
Off point $17 = P_{27} = (8, 1, 1)$ lies on 4 bisecants : $\{ 2, 4, 13, 15 \}$
Off point $18 = P_{28} = (0, 2, 1)$ lies on 4 bisecants : $\{ 12, 13, 19, 22 \}$
Off point $19 = P_{29} = (1, 2, 1)$ lies on 4 bisecants : $\{ 6, 10, 16, 26 \}$
Off point $20 = P_{31} = (3, 2, 1)$ lies on 4 bisecants : $\{ 2, 5, 21, 25 \}$
Off point $21 = P_{33} = (5, 2, 1)$ lies on 4 bisecants : $\{ 1, 9, 18, 27 \}$
Off point $22 = P_{35} = (7, 2, 1)$ lies on 4 bisecants : $\{ 7, 11, 17, 23 \}$
Off point $23 = P_{36} = (8, 2, 1)$ lies on 4 bisecants : $\{ 3, 8, 15, 20 \}$
Off point $24 = P_{37} = (0, 3, 1)$ lies on 4 bisecants : $\{ 4, 6, 18, 23 \}$
Off point $25 = P_{39} = (2, 3, 1)$ lies on 4 bisecants : $\{ 1, 5, 12, 16 \}$
Off point $26 = P_{40} = (3, 3, 1)$ lies on 4 bisecants : $\{ 8, 9, 22, 24 \}$
Off point $27 = P_{42} = (5, 3, 1)$ lies on 4 bisecants : $\{ 3, 7, 13, 26 \}$
Off point $28 = P_{43} = (6, 3, 1)$ lies on 4 bisecants : $\{ 2, 11, 14, 19 \}$
Off point $29 = P_{45} = (8, 3, 1)$ lies on 4 bisecants : $\{ 15, 17, 25, 27 \}$

Off point 30 = $P_{47} = (1, 4, 1)$ lies on 4 bisecants : { 5, 7, 14, 22 }
 Off point 31 = $P_{48} = (2, 4, 1)$ lies on 4 bisecants : { 8, 10, 17, 19 }
 Off point 32 = $P_{49} = (3, 4, 1)$ lies on 4 bisecants : { 4, 11, 26, 27 }
 Off point 33 = $P_{50} = (4, 4, 1)$ lies on 4 bisecants : { 1, 2, 20, 23 }
 Off point 34 = $P_{52} = (6, 4, 1)$ lies on 4 bisecants : { 6, 9, 13, 21 }
 Off point 35 = $P_{54} = (8, 4, 1)$ lies on 4 bisecants : { 12, 15, 18, 24 }
 Off point 36 = $P_{55} = (0, 5, 1)$ lies on 4 bisecants : { 3, 5, 9, 11 }
 Off point 37 = $P_{56} = (1, 5, 1)$ lies on 4 bisecants : { 13, 20, 24, 27 }
 Off point 38 = $P_{59} = (4, 5, 1)$ lies on 4 bisecants : { 18, 19, 25, 26 }
 Off point 39 = $P_{60} = (5, 5, 1)$ lies on 4 bisecants : { 12, 14, 17, 21 }
 Off point 40 = $P_{61} = (6, 5, 1)$ lies on 4 bisecants : { 1, 4, 7, 10 }
 Off point 41 = $P_{63} = (8, 5, 1)$ lies on 4 bisecants : { 15, 16, 22, 23 }
 Off point 42 = $P_{64} = (0, 6, 1)$ lies on 4 bisecants : { 0, 10, 20, 21 }
 Off point 43 = $P_{65} = (1, 6, 1)$ lies on 4 bisecants : { 0, 9, 19, 23 }
 Off point 44 = $P_{66} = (2, 6, 1)$ lies on 4 bisecants : { 0, 11, 18, 22 }
 Off point 45 = $P_{67} = (3, 6, 1)$ lies on 4 bisecants : { 0, 1, 13, 17 }
 Off point 46 = $P_{68} = (4, 6, 1)$ lies on 4 bisecants : { 0, 7, 12, 27 }
 Off point 47 = $P_{69} = (5, 6, 1)$ lies on 4 bisecants : { 0, 2, 6, 8 }
 Off point 48 = $P_{70} = (6, 6, 1)$ lies on 4 bisecants : { 0, 3, 16, 25 }
 Off point 49 = $P_{71} = (7, 6, 1)$ lies on 4 bisecants : { 0, 4, 14, 24 }
 Off point 50 = $P_{72} = (8, 6, 1)$ lies on 4 bisecants : { 0, 5, 15, 26 }
 Off point 51 = $P_{73} = (0, 7, 1)$ lies on 4 bisecants : { 2, 7, 16, 24 }
 Off point 52 = $P_{74} = (1, 7, 1)$ lies on 4 bisecants : { 4, 8, 12, 25 }
 Off point 53 = $P_{75} = (2, 7, 1)$ lies on 4 bisecants : { 3, 6, 14, 27 }
 Off point 54 = $P_{79} = (6, 7, 1)$ lies on 4 bisecants : { 17, 20, 22, 26 }
 Off point 55 = $P_{80} = (7, 7, 1)$ lies on 4 bisecants : { 5, 10, 13, 18 }
 Off point 56 = $P_{81} = (8, 7, 1)$ lies on 4 bisecants : { 1, 11, 15, 21 }
 Off point 57 = $P_{85} = (3, 8, 1)$ lies on 4 bisecants : { 14, 16, 18, 20 }
 Off point 58 = $P_{86} = (4, 8, 1)$ lies on 4 bisecants : { 3, 4, 21, 22 }
 Off point 59 = $P_{87} = (5, 8, 1)$ lies on 4 bisecants : { 10, 11, 24, 25 }
 Off point 60 = $P_{88} = (6, 8, 1)$ lies on 4 bisecants : { 5, 8, 23, 27 }
 Off point 61 = $P_{89} = (7, 8, 1)$ lies on 4 bisecants : { 2, 9, 12, 26 }
 Off point 62 = $P_{90} = (8, 8, 1)$ lies on 4 bisecants : { 6, 7, 15, 19 }

The following command creates the same curve in the normal form. The normal form requires 6 parameters, called a, b, c, d, e, f . We choose

$$(a, b, c, d, e, f) = (6, 3, 4, 6, 2, 5)$$

to create the Hermitian curve.

Code Snippet 426

```

1  quartic_curve_9_0_normal_form_report:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define F -finite_field -q 9 -end \
4  ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5  ▷ ▷ -define C -quartic_curve -space P -by_normal_form "6,3,4,6,2,5" -end \
6  ▷ ▷ -with C -do \
7  ▷ ▷ ▷ -quartic_curve_activity \
8  ▷ ▷ ▷ ▷ -report \
9  ▷ ▷ ▷ -end \
10 ▷ ▷ -print_symbols

```

```

11  ▷ $(PDFLATEX) quartic_curve_by_normal_form.q9_report.tex
12  ▷ $(OPEN) quartic_curve_by_normal_form.q9_report.pdf
13

```

The Edge curve over $\mathbb{F}_q$ has an equation of the form

$$X^4 - Y^4 - Z^4 + 2f^2Y^2Z^2 + 4fX^2YZ = 0,$$

with f a primitive element of $\mathbb{F}_q$. Over $\mathbb{F}_9$, since we can assume that Orbiter uses a primitive polynomials, we use 3 as a coding for the primitive element α in the field. The following command creates the Edge curve:

Code Snippet 427

```

1    quartic_curve_edge.q9:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 9 -end \
4    ▷ ▷ -define P2 -projective_space -n 2 -field F -end \
5    ▷ ▷ -define R -polynomial_ring \
6    ▷ ▷ ▷ -field F \
7    ▷ ▷ ▷ -number_of_variables 3 \
8    ▷ ▷ ▷ -homogeneous_of_degree 4 \
9    ▷ ▷ ▷ -monomial_ordering_partition \
10   ▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \
11   ▷ ▷ -end \
12   ▷ ▷ -define Edge -symbolic_object \
13   ▷ ▷ ▷ -field F \
14   ▷ ▷ ▷ -managed_variables "X,Y,Z" \
15   ▷ ▷ ▷ -text "X^4-Y^4-Z^4+2*f^2*Y^2*Z^2+1*f*X^2*Y*Z" \
16   ▷ ▷ -end \
17   ▷ ▷ -define f -symbolic_object \
18   ▷ ▷ ▷ -field F \
19   ▷ ▷ ▷ -text "3" \
20   ▷ ▷ -end \
21   ▷ ▷ -define Edge_f3 -symbolic_object \
22   ▷ ▷ ▷ -field F \
23   ▷ ▷ ▷ -managed_variables "X,Y,Z" \
24   ▷ ▷ ▷ -substitute "f" Edge f \
25   ▷ ▷ -end \
26   ▷ ▷ -define C -quartic_curve \
27   ▷ ▷ ▷ -space P2 \
28   ▷ ▷ ▷ -by_symbolic_object R Edge_f3 -end \
29   ▷ ▷ -with C -do \
30   ▷ ▷ ▷ -quartic_curve_activity \
31   ▷ ▷ ▷ ▷ -report \
32   ▷ ▷ ▷ -end
33   ▷ $(PDFLATEX) quartic_curve.equation.Edge_f3.q9_report.tex
34   ▷ $(OPEN) quartic_curve.equation.Edge_f3.q9_report.pdf
35

```

The report shows that the curve is non-singular and has 8 rational points.

Orbiter can create quartic curves from cubic surfaces. The following command creates all quartic curves arising from points on no line of the cubic surface over $\mathbb{F}_7$:

Code Snippet 428

```

1   surface_7_0_make_quartic_curves:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define Do -draw.options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 7 -end \
7   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
8   ▷ ▷ -define P2 -projective_space -n 2 -field F -v 0 -end \
9   ▷ ▷ -define Control -poset_classification_control \
10  ▷ ▷ ▷ -problem_label nc_arcs.q7 \
11  ▷ ▷ ▷ -W -depth 6 \
12  ▷ ▷ -end \
13  ▷ ▷ -define Arc_control -arc_generator_control \
14  ▷ ▷ ▷ -projective_space P2 \
15  ▷ ▷ ▷ -control Control \
16  ▷ ▷ ▷ -target_size 6 \
17  ▷ ▷ ▷ -d 2 \
18  ▷ ▷ -end \
19  ▷ ▷ -define Orb -orbits \
20  ▷ ▷ ▷ -on_arcs Arc_control \
21  ▷ ▷ -end \
22  ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
23  ▷ ▷ -with S -do \
24  ▷ ▷ -cubic_surface_activity \
25  ▷ ▷ ▷ -report Do \
26  ▷ ▷ ▷ -report_all_flag_orbits Orb \
27  ▷ ▷ -end
28  ▷ $(PDFLATEX) surface_catalogue.q7_iso0_report.tex
29  ▷ $(OPEN) surface_catalogue.q7_iso0_report.pdf
30

```

Code Snippet 429

```

1   quartic_curves.q7:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 7 -end \
4   ▷ ▷ -define P -projective_space -n 3 -field F -end \
5   ▷ ▷ -define P2 -projective_space -n 2 -field F -v 0 -end \
6   ▷ ▷ -define Control -poset_classification_control \
7   ▷ ▷ ▷ -problem_label nc_arcs.q7 \
8   ▷ ▷ ▷ -W -depth 6 \
9   ▷ ▷ -end \
10  ▷ ▷ -define Arc_control -arc_generator_control \
11  ▷ ▷ ▷ -projective_space P2 \
12  ▷ ▷ ▷ -control Control \
13  ▷ ▷ ▷ -target_size 6 \
14  ▷ ▷ ▷ -d 2 \
15  ▷ ▷ -end \
16  ▷ ▷ -define Orb -orbits \

```



```

17  ▷ ▷ ▷ -on_arcs Arc_control \
18  ▷ ▷ -end \
19  ▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q7) 1 \
20  ▷ ▷ ▷ -define S_%L -cubic_surface -space P -catalogue %L -end \
21  ▷ ▷ -end_loop L \
22  ▷ ▷ -print_symbols \
23  ▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q7) 1 \
24  ▷ ▷ ▷ -with S_%L -do \
25  ▷ ▷ ▷ -cubic_surface_activity \
26  ▷ ▷ ▷ ▷ -export_all_quartic_curves Orb \
27  ▷ ▷ ▷ -end \
28  ▷ ▷ -end_loop L \
29  ▷ ▷ -print_symbols
30

```

The next command creates all cubic surfaces arising from the second isomorphism class of cubic surfaces over the field $\mathbb{F}_9$:

Code Snippet 430

```

1  surface_9_1_make_quartic_curves:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define Do -draw_options \
4  ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
5  ▷ ▷ -end \
6  ▷ ▷ -define F -finite_field -q 9 -end \
7  ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
8  ▷ ▷ -define P2 -projective_space -n 2 -field F -v 0 -end \
9  ▷ ▷ -define Control -poset_classification_control \
10 ▷ ▷ ▷ -problem_label nc_arcs.q9 \
11 ▷ ▷ ▷ -W -depth 6 \
12 ▷ ▷ -end \
13 ▷ ▷ -define Arc_control -arc_generator_control \
14 ▷ ▷ ▷ -projective_space P2 \
15 ▷ ▷ ▷ -control Control \
16 ▷ ▷ ▷ -target_size 6 \
17 ▷ ▷ ▷ -d 2 \
18 ▷ ▷ -end \
19 ▷ ▷ -define Orb -orbits \
20 ▷ ▷ ▷ -on_arcs Arc_control \
21 ▷ ▷ -end \
22 ▷ ▷ -define S -cubic_surface -space P -catalogue 1 -end \
23 ▷ ▷ -with S -do \
24 ▷ ▷ -cubic_surface_activity \
25 ▷ ▷ ▷ -report Do \
26 ▷ ▷ ▷ -report_all_flag_orbits Orb \
27 ▷ ▷ -end
28 ▷ #$(PDFLATEX) surface_catalogue.q9_iso1_report.tex
29 ▷ #$(OPEN) surface_catalogue.q9_iso1_report.pdf
30

```

Here we apply some transformations to get a nicer form of the equation of this quartic curve.

Code Snippet 431

```

1   quartic_curve_q9_1_clean2:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 9 -print_as_exponentials -end \
7   ▷ ▷ -define P2 -projective_space -n 2 -field F -end \
8   ▷ ▷ -define P3 -projective_space -n 3 -field F -end \
9   ▷ ▷ -define S9_1 -cubic_surface -space P3 -catalogue 1 \
10  ▷ ▷ ▷ -select_double_six "c12,c23,a4,c13,a5,a6,b3,b1,c56,b2,c46,c45" \
11  ▷ ▷ ▷ -transform_inverse "6,4,6,1, 0,1,0,0, 0,0,1,0, 0,0,0,1, 0" \
12  ▷ ▷ ▷ -transform_inverse "1,0,0,0, 0,1,0,5, 0,0,1,1, 0,0,0,1, 0" \
13  ▷ ▷ ▷ -transform "1,0,0,0, 0,0,1,0, 1,1,5,0, 0,0,0,1, 0" \
14  ▷ ▷ ▷ -transform "1,0,0,0, 0,1,0,0, 0,0,1,0, 0,0,8,1, 0" \
15  ▷ ▷ ▷ -end \
16  ▷ ▷ -with S9_1 -do \
17  ▷ ▷ -cubic_surface_activity \
18  ▷ ▷ ▷ -report Do \
19  ▷ ▷ -end \
20  ▷ ▷ -define C -quartic_curve -space P2 -from_cubic_surface S9_1 0 -end \
21  ▷ ▷ -with C -do \
22  ▷ ▷ ▷ -quartic_curve_activity \
23  ▷ ▷ ▷ ▷ -report \
24  ▷ ▷ ▷ -end
25  ▷ $(PDFLATEX) surface_catalogue_q9_iso1_report.tex
26  ▷ $(PDFLATEX) quartic_curve_surface_catalogue_q9_iso1_pt_orb_0_report.tex
27  ▷ $(OPEN) quartic_curve_surface_catalogue_q9_iso1_pt_orb_0_report.pdf
28

```

Suppose we want to create reports on all quartic curves for a given value of q , utilizing the Orbiter knowledge base of quartic curves. Here is an example, considering the quartic curves over $\mathbb{F}_{19}$. We use a makefile variable to set the number of curves:

Code Snippet 432

```

1   NB_QUARTIC_CURVES_Q19=14
2

```

Now, we create each curve one-by-one and produce a report. To do so, we utilize the loop command. Once done, we translate the latex files:

Code Snippet 433

```

1   quartic_curves_19_report:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 19 -end \
4   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5   ▷ ▷ -loop L 0 $(NB_QUARTIC_CURVES_Q19) 1 \
6   ▷ ▷ -define C -quartic_curve -space P -catalogue %L -end \

```

```

7   ▷ ▷ -with C -do \
8   ▷ ▷ ▷ -quartic_curve_activity \
9   ▷ ▷ ▷ ▷ -report \
10  ▷ ▷ ▷ -end \
11  ▷ ▷ -end_loop L
12  ▷ ▷ $(PDFLATEX) quartic_curve_catalogue_q19_iso0_report.tex
13  ▷ ▷ $(PDFLATEX) quartic_curve_catalogue_q19_iso1_report.tex
14  ▷ ▷ $(PDFLATEX) quartic_curve_catalogue_q19_iso2_report.tex
15  ▷ ▷ $(PDFLATEX) quartic_curve_catalogue_q19_iso3_report.tex
16  ▷ ▷ $(PDFLATEX) quartic_curve_catalogue_q19_iso4_report.tex
17  ▷ ▷ $(PDFLATEX) quartic_curve_catalogue_q19_iso5_report.tex
18  ▷ ▷ $(PDFLATEX) quartic_curve_catalogue_q19_iso6_report.tex
19  ▷ ▷ $(PDFLATEX) quartic_curve_catalogue_q19_iso7_report.tex
20  ▷ ▷ $(PDFLATEX) quartic_curve_catalogue_q19_iso8_report.tex
21  ▷ ▷ $(PDFLATEX) quartic_curve_catalogue_q19_iso9_report.tex
22  ▷ ▷ $(PDFLATEX) quartic_curve_catalogue_q19_iso10_report.tex
23  ▷ ▷ $(PDFLATEX) quartic_curve_catalogue_q19_iso11_report.tex
24  ▷ ▷ $(PDFLATEX) quartic_curve_catalogue_q19_iso12_report.tex
25  ▷ ▷ $(PDFLATEX) quartic_curve_catalogue_q19_iso13_report.tex
26

```

In the next example, we will study the generation of a quartic curve from a cubic surface. The example is the quartic curve over $\mathbb{F}_9$. It is interesting, as it contains the maximum number of Kovalevski points, which is 63 (a Kovalevski point is a point where 4 bitangents meet). There are two cubic surfaces with 27 lines over $\mathbb{F}_9$. The first one has 10 Eckardt points, whereas the second one has 9. The quartic curve arises from the surface with 9 Eckardt points (OCN=1). The surface has a unique point not on any line. For this reason, we set the orbit number on points not on lines to zero. Here is the command:

Code Snippet 434

```

1   quartic_curve_q9_1:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 9 -end \
4   ▷ ▷ -define P2 -projective_space -n 2 -field F -end \
5   ▷ ▷ -define P3 -projective_space -n 3 -field F -end \
6   ▷ ▷ -define S9_1 -cubic_surface -space P3 -catalogue 1 -end \
7   ▷ ▷ -define C -quartic_curve -space P2 -from_cubic_surface S9_1 0 -end \
8   ▷ ▷ -with C -do \
9   ▷ ▷ ▷ -quartic_curve_activity \
10  ▷ ▷ ▷ ▷ -report \
11  ▷ ▷ ▷ -end
12  ▷ $(PDFLATEX) quartic_curve_surface_catalogue_q9_iso1_pt_orb_0_report.tex
13  ▷ $(OPEN) quartic_curve_surface_catalogue_q9_iso1_pt_orb_0_report.pdf
14

```

Let us create all flag orbits of quartic curves associated with the cubic surface over $\mathbb{F}_{29}$ of catalogue number 29:

Code Snippet 435

```

1    quartic_curves.q29_po4_export:
2    ▷ $(ORBITER) -v 10 \
3    ▷ ▷ -define F -finite_field -q 29 -end \
4    ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5    ▷ ▷ -define P2 -projective_space -n 2 -field F -v 0 -end \
6    ▷ ▷ -define Control -poset_classification_control \
7    ▷ ▷ ▷ -problem_label nc_arcs.q29 \
8    ▷ ▷ ▷ -W -depth 6 \
9    ▷ ▷ -end \
10   ▷ ▷ -define Arc_control -arc_generator_control \
11   ▷ ▷ ▷ -projective_space P2 \
12   ▷ ▷ ▷ -control Control \
13   ▷ ▷ ▷ -target_size 6 \
14   ▷ ▷ ▷ -d 2 \
15   ▷ ▷ -end \
16   ▷ ▷ -define Orb -orbits \
17   ▷ ▷ ▷ -on_arcs Arc_control \
18   ▷ ▷ -end \
19   ▷ ▷ -define S_4 -cubic_surface -space P -catalogue 4 -end \
20   ▷ ▷ -print_symbols \
21   ▷ ▷ -with S_4 -do \
22   ▷ ▷ -cubic_surface_activity \
23   ▷ ▷ ▷ -export_all_quartic_curves Orb \
24   ▷ ▷ -end \
25   ▷ ▷ -print_symbols
26

```

The following command creates a latex report of these flag orbits:

Code Snippet 436

```

1    quartic_curves.q29_po4_flag_orbits_report:
2    ▷ $(ORBITER) -v 10 \
3    ▷ ▷ -define F -finite_field -q 29 -end \
4    ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5    ▷ ▷ -define P2 -projective_space -n 2 -field F -v 0 -end \
6    ▷ ▷ -define Control -poset_classification_control \
7    ▷ ▷ ▷ -problem_label nc_arcs.q29 \
8    ▷ ▷ ▷ -W -depth 6 \
9    ▷ ▷ -end \
10   ▷ ▷ -define Arc_control -arc_generator_control \
11   ▷ ▷ ▷ -projective_space P2 \
12   ▷ ▷ ▷ -control Control \
13   ▷ ▷ ▷ -target_size 6 \
14   ▷ ▷ ▷ -d 2 \
15   ▷ ▷ -end \
16   ▷ ▷ -define Orb -orbits \
17   ▷ ▷ ▷ -on_arcs Arc_control \
18   ▷ ▷ -end \
19   ▷ ▷ -define S_4 -cubic_surface -space P -catalogue 4 -end \

```

```

20  ▷ ▷ -print_symbols \
21  ▷ ▷ -with S_4 -do \
22  ▷ ▷ -cubic_surface_activity \
23  ▷ ▷ ▷ -report_all_flag_orbits Orb \
24  ▷ ▷ -end \
25  ▷ ▷ -print_symbols
26  ▷ $(PDFLATEX) surface_catalogue.q29_iso4_quartics.tex
27  ▷ $(OPEN) surface_catalogue.q29_iso4_quartics.pdf
28

```

Next, we create the quartic curve explicitly, using the polynomials f_1 , f_2 and f_3 computed after moving the point of projection to $\mathbf{P}(1, 0, 0, 0)$. To begin with, we define the polynomials as makefile variables:

Code Snippet 437

```

1  EQUATION_SURFACE_Q29_ISO4_ORB7_F1="11*x1 + 6*x2 + 7*x3"
2

```

Code Snippet 438

```

1  EQUATION_SURFACE_Q29_ISO4_ORB7_F2="17*x1^2 + 27*x2^2 + 8*x3^2 + 13*x1*x2 + 25*x1*
   x3 + 20*x2*x3"
2

```

Code Snippet 439

```

1  EQUATION_SURFACE_Q29_ISO4_ORB7_F3="27*x3^3 + 28*x1^2*x3 + 17*x2^2*x3 + x1*x3^2 +
   2*x2*x3^2 + 17*x1*x2*x3"
2

```

Using the following command, we create the equation of the quartic curve as

$$f_2^2 - 4f_1f_3.$$

The object will be a symbolic expression representing the equation of the quartic curve.

Code Snippet 440

```

1  symbolic_equation_of_curve.q29_po4_so7:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define F -finite_field -q 29 -end \
4  ▷ ▷ -define f1 -symbolic_object \
5  ▷ ▷ ▷ -field F \
6  ▷ ▷ ▷ -text $(EQUATION_SURFACE_Q29_ISO4_ORB7_F1) \
7  ▷ ▷ -end \
8  ▷ ▷ -define f2 -symbolic_object \
9  ▷ ▷ ▷ -field F \
10 ▷ ▷ ▷ -text $(EQUATION_SURFACE_Q29_ISO4_ORB7_F2) \
11 ▷ ▷ -end \
12 ▷ ▷ -define f3 -symbolic_object \
13 ▷ ▷ ▷ -field F \

```

```

14  ▷ ▷ ▷ -text $(EQUATION_SURFACE_Q29.IS04.ORB7.F3) \
15  ▷ ▷ -end \
16  ▷ ▷ -define curve -symbolic_object \
17  ▷ ▷ ▷ -field F \
18  ▷ ▷ ▷ -text "f2^2-4*f1*f3" \
19  ▷ ▷ -end \
20  ▷ ▷ -define curve_x -symbolic_object \
21  ▷ ▷ ▷ -field F \
22  ▷ ▷ ▷ -expand curve \
23  ▷ ▷ -end
24

```

Finally, we create a latex report on the quartic curve with catalogue number 121 over the field $\mathbb{F}_{29}$:

Code Snippet 441

```

1    quartic_curve_29_121:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 29 -end \
4    ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5    ▷ ▷ -define C -quartic_curve -space P -catalogue 121 -end \
6    ▷ ▷ -with C -do \
7    ▷ ▷ ▷ -quartic_curve_activity \
8    ▷ ▷ ▷ ▷ -report \
9    ▷ ▷ ▷ -end
10   ▷ $(PDFLATEX) quartic_curve_catalogue_q29_iso121_report.tex
11   ▷ $(OPEN) quartic_curve_catalogue_q29_iso121_report.pdf
12

```

9.6 Classification of Quartic Curves

Let us now address the problem of classification of quartic curves. For a fixed field $\mathbb{F}_q$, q odd, we can exploit the previously classified list of cubic surfaces as a start. For each surface, and for each orbit on points not on lines, we perform the projection along the tangent cone to create one quartic curve. This guarantees that each isomorphism type of quartic curve with 28 bitangents will be created. Afterwards, we apply isomorphism testing based on canonical forms. This is based on substructures and using canonical forms for graphs using the built-in package Nauty.

Let us look at some examples of this algorithm. We try $q = 7$ and then $q = 13$. We will set a makefile variable to the number of (isomorphism types of) cubic surfaces with 27 lines over $\mathbb{F}_q$. For $q = 7$, there is exactly one isomorphism type, so we put

Code Snippet 442

```
1 NB_CUBIC_SURFACES_Q7=1
2
```

The next command creates a list of quartic curves using a projection construction. For each isomorphism type of cubic surface, and for each point not on any line, we consider the intersection of the tangent cone with the surface and project onto a plane not containing the point. Because of symmetry, it suffices to perform this projection only for a set of representatives of the orbits on points not on lines. Here is the Orbiter command for $q = 7$:

Code Snippet 443

```
1 quartic_curves.q7:
2 ▷ $(ORBITER) -v 3 \
3 ▷ ▷ -define F -finite_field -q 7 -end \
4 ▷ ▷ -define P -projective_space -n 3 -field F -end \
5 ▷ ▷ -define P2 -projective_space -n 2 -field F -v 0 -end \
6 ▷ ▷ -define Control -poset_classification_control \
7 ▷ ▷ ▷ -problem_label nc_arcs.q7 \
8 ▷ ▷ ▷ -W -depth 6 \
9 ▷ ▷ -end \
10 ▷ ▷ -define Arc_control -arc_generator_control \
11 ▷ ▷ ▷ -projective_space P2 \
12 ▷ ▷ ▷ -control Control \
13 ▷ ▷ ▷ -target_size 6 \
14 ▷ ▷ ▷ -d 2 \
15 ▷ ▷ -end \
16 ▷ ▷ -define Orb -orbits \
17 ▷ ▷ ▷ -on_arcs Arc_control \
18 ▷ ▷ -end \
19 ▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q7) 1 \
20 ▷ ▷ ▷ -define S_%L -cubic_surface -space P -catalogue %L -end \
21 ▷ ▷ -end_loop L \
22 ▷ ▷ -print_symbols \
23 ▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q7) 1 \
24 ▷ ▷ ▷ -with S_%L -do \
25 ▷ ▷ ▷ -cubic_surface_activity \
26 ▷ ▷ ▷ ▷ -export_all_quartic_curves Orb \
27 ▷ ▷ ▷ -end \
28 ▷ ▷ -end_loop L \
```

```

29  ▷ ▷ -print_symbols
30

```

The resulting curves are written to file. Unfortunately, in this example, there is no point which does not lie on any line of the surface. This means that no quartic curve with 28 lines exists over $\mathbb{F}_7$.

We move on to the next example, which is $q = 13$. Again, we use a makefile variable to set the number of isomorphism types of cubic surfaces with 27 lines over $\mathbb{F}_{13}$. There are exactly 4 types:

Code Snippet 444

```

1  NB_CUBIC_SURFACES_Q13=4
2

```

Just like before, we create all quartic curves arising from projections:

Code Snippet 445

```

1  quartic_curves_q13:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define F -finite_field -q 13 -end \
4  ▷ ▷ -define P -projective_space -n 3 -field F -end \
5  ▷ ▷ -define P2 -projective_space -n 2 -field F -v 0 -end \
6  ▷ ▷ -define Control -poset_classification_control \
7  ▷ ▷ ▷ -problem_label nc_arcs.q13 \
8  ▷ ▷ ▷ -W -depth 6 \
9  ▷ ▷ -end \
10 ▷ ▷ -define Arc_control -arc_generator_control \
11 ▷ ▷ ▷ -projective_space P2 \
12 ▷ ▷ ▷ -control Control \
13 ▷ ▷ ▷ -target_size 6 \
14 ▷ ▷ ▷ -d 2 \
15 ▷ ▷ -end \
16 ▷ ▷ -define Orb -orbits \
17 ▷ ▷ ▷ -on_arcs Arc_control \
18 ▷ ▷ -end \
19 ▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q13) 1 \
20 ▷ ▷ ▷ -define S_%L -cubic_surface -space P -catalogue %L -end \
21 ▷ ▷ -end_loop L \
22 ▷ ▷ -print_symbols \
23 ▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q13) 1 \
24 ▷ ▷ ▷ -with S_%L -do \
25 ▷ ▷ ▷ -cubic_surface_activity \
26 ▷ ▷ ▷ ▷ -export_all_quartic_curves Orb \
27 ▷ ▷ ▷ -end \
28 ▷ ▷ -end_loop L \
29 ▷ ▷ -print_symbols
30

```

Every cubic surface creates a number of quartic curves. Namely, each orbit of the automorphism group on points not on lines of the cubic surface creates one quartic curve. The curves arising this way are written to file. There is one file for each surface. In Section 17.2, we will continue with the classification of quartic curves.

9.7 Interface to GAP

We wish to export a cubic surface to GAP. The following command creates the Hirschfeld surface and exports it to GAP

Code Snippet 446

```

1    surface_4_0_export_gap:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 4 -end \
4    ▷ ▷ -define P -projective_space -n 3 -field F -end \
5    ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
6    ▷ ▷ -with S -do \
7    ▷ ▷ -cubic_surface_activity \
8    ▷ ▷ ▷ -export_gap \
9    ▷ ▷ -end
10

```

The following GAP file is written:

```

LoadPackage("fining");
# Cubic surface catalogue_q4_iso0
# Group:
mat1 := [[Z(4)^1,0*Z(4),0*Z(4),0*Z(4)],
[0*Z(4),Z(4)^1,0*Z(4),0*Z(4)],
[0*Z(4),0*Z(4),Z(4)^1,0*Z(4)],
[0*Z(4),0*Z(4),0*Z(4),Z(4)^1]];
frob1 := FrobeniusAutomorphism(GF(4))^1;
psi1 := ProjectiveSemilinearMap(mat1, frob1,GF(4));
mat2 := [[Z(4)^2,0*Z(4),0*Z(4),0*Z(4)],
[0*Z(4),Z(4)^1,0*Z(4),0*Z(4)],
[0*Z(4),0*Z(4),Z(4)^1,0*Z(4)],
[0*Z(4),0*Z(4),0*Z(4),Z(4)^2]];
frob2 := FrobeniusAutomorphism(GF(4))^1;
psi2 := ProjectiveSemilinearMap(mat2, frob2,GF(4));
mat3 := [[Z(4)^0,0*Z(4),0*Z(4),0*Z(4)],
[0*Z(4),Z(4)^2,0*Z(4),0*Z(4)],
[0*Z(4),0*Z(4),Z(4)^2,0*Z(4)],
[0*Z(4),0*Z(4),0*Z(4),Z(4)^0]];
frob3 := FrobeniusAutomorphism(GF(4))^0;
psi3 := ProjectiveSemilinearMap(mat3, frob3,GF(4));
mat4 := [[Z(4)^2,0*Z(4),0*Z(4),0*Z(4)],
[0*Z(4),Z(4)^2,0*Z(4),0*Z(4)],
[0*Z(4),0*Z(4),Z(4)^2,0*Z(4)],
[Z(4)^2,0*Z(4),0*Z(4),Z(4)^2]];
frob4 := FrobeniusAutomorphism(GF(4))^0;
psi4 := ProjectiveSemilinearMap(mat4, frob4,GF(4));
mat5 := [[Z(4)^0,0*Z(4),0*Z(4),0*Z(4)],
[0*Z(4),Z(4)^2,0*Z(4),0*Z(4)],
[Z(4)^1,0*Z(4),Z(4)^2,0*Z(4)],
[0*Z(4),Z(4)^1,0*Z(4),Z(4)^0]];
frob5 := FrobeniusAutomorphism(GF(4))^1;
psi5 := ProjectiveSemilinearMap(mat5, frob5,GF(4));

```

```

mat6 := [[Z(4)^0, 0*Z(4), 0*Z(4), 0*Z(4)],
[0*Z(4), Z(4)^2, 0*Z(4), 0*Z(4)],
[Z(4)^1, Z(4)^2, Z(4)^2, 0*Z(4)],
[Z(4)^0, Z(4)^1, 0*Z(4), Z(4)^0]];
frob6 := FrobeniusAutomorphism(GF(4))^1;
psi6 := ProjectiveSemilinearMap(mat6, frob6, GF(4));
mat7 := [[Z(4)^1, 0*Z(4), 0*Z(4), 0*Z(4)],
[Z(4)^1, Z(4)^1, 0*Z(4), 0*Z(4)],
[0*Z(4), 0*Z(4), Z(4)^1, 0*Z(4)],
[0*Z(4), 0*Z(4), Z(4)^1, Z(4)^1]];
frob7 := FrobeniusAutomorphism(GF(4))^1;
psi7 := ProjectiveSemilinearMap(mat7, frob7, GF(4));
mat8 := [[Z(4)^2, 0*Z(4), 0*Z(4), 0*Z(4)],
[Z(4)^2, Z(4)^0, Z(4)^0, 0*Z(4)],
[0*Z(4), Z(4)^0, 0*Z(4), 0*Z(4)],
[Z(4)^2, Z(4)^0, 0*Z(4), Z(4)^2]];
frob8 := FrobeniusAutomorphism(GF(4))^1;
psi8 := ProjectiveSemilinearMap(mat8, frob8, GF(4));
mat9 := [[Z(4)^0, Z(4)^2, Z(4)^0, Z(4)^2],
[0*Z(4), Z(4)^2, Z(4)^2, Z(4)^1],
[Z(4)^2, Z(4)^0, Z(4)^2, Z(4)^0],
[Z(4)^0, Z(4)^1, 0*Z(4), Z(4)^0]];
frob9 := FrobeniusAutomorphism(GF(4))^0;
psi9 := ProjectiveSemilinearMap(mat9, frob9, GF(4));
gens := [psi1, psi2, psi3, psi4, psi5, psi6, psi7, psi8, psi9];
G := Group(gens);
Size(G);
r := PolynomialRing(GF(4), ["X0", "X1", "X2", "X3"]);
Eqn := (r.1)^2*(r.4)+(r.2)^2*(r.3)+(r.2)*(r.3)^2+(r.1)*(r.4)^2;

```

Chapter 10

Applications

10.1 Number Theory

In Table 10.1, global Orbiter commands for Algebra are summarized.

Table 10.2 shows Orbiter commands for basic number theory, including integer factor rings and the Euclidean algorithm.

In Table 10.3, some number theoretic commands are shown.

To compute primitive roots, there exists a randomized fast algorithm. On the other hand, there is also an algorithm that finds the smallest primitive root. For instance,

Code Snippet 447

```
1    PR101:
2    ▷ $(ORBITER) -v 5 -primitive_root 101
3
```

finds a primitive root modulo 101 using the randomized algorithm. The command finds the root 75. On the other hand, the command

Code Snippet 448

```
1    PR101_smallest:
2    ▷ $(ORBITER) -v 5 -smallest_primitive_root 101
3
```

finds the smallest primitive root modulo 101, which turns out to be 2.

Code Snippet 449

```
1    PR29:
2    ▷ $(ORBITER) -v 1 -smallest_primitive_root 29
3
```

computes a primitive root modulo 29. The answer in this case is 2.

It is possible to compute primitive roots for all prime numbers in a given interval. The command

Global Algebra Commands		
Command	Arguments	Purpose
-power_function_mod_n	k n	
-all_rational_normal_forms	F d	Compute all rational normal forms of elements in $GL(d, F)$, where F is a field.
-compute_rational_normal_form	F d matrix	Compute the rational normal form of the given $d \times d$ matrix over the field F .
-eigenstuff	F n coeffs fname	Compute the eigenvalues and eigenvectors of a matrix.
-smith_normal_form	M	Compute the Smith Normal Form of the matrix M , together with matrices P, P^{-1} and Q, Q^{-1} such that $P^{-1} * M' * Q^{-1} = M$, where M' is the Smith Normal Form of A .
-smith_normal_form_ from_the_left_only	M	Compute the rref of the matrix M over the ring of integers, together with P and P^{-1} such that $P^{-1} * M' = M$, where M' is the integer rref of M .
-character_table_symmetric_group	n	Compute the character table of the symmetric group of degree n.
-young_symmetrizer	n	Compute the Young symmetrizer in the symmetric group of degree n.
-young_symmetrizer_sym_4		Compute the Young symmetrizer in the symmetric group of degree 4.
-make_A5_in_PSL_2_q	q	Compute generators for a subgroup of $PSL(2, q)$ isomorphic to $Alt(5)$.
-order_of_group_Anq	n q	Compute the order of the group of Lie type $A_n(q)$
-order_of_group_Bnq	n q	Compute the order of the group of Lie type $B_n(q)$
-order_of_group_Dnq	n q	Compute the order of the group of Lie type $D_n(q)$
-boolean_function_as_polynomial	n v	Compute the polynomial representation of the boolean function v in n variables. The function refers to a vector object. The length of the vector is 2^n . The entries of the vector are ordered according to the binary representation of integers.

Table 10.1: Global Algebra Commands

Basic Number Theory Commands		
Command	Arguments	Purpose
-primitive_root	p	Computes a primitive root modulo p .
-smallest_primitive_root	p	Computes the smallest primitive root modulo p .
-smallest_primitive_root_interval	$m1\ m2$	Computes the smallest primitive root modulo m for all m with $m1 \leq m \leq m2$.
-number_of_primitive_roots_interval	$m1\ m2$	Counts the number of primitive roots modulo m for all m with $m1 \leq m \leq m2$.
-inverse_mod	$a\ p$	Computes the modular inverse of a modulo p , i.e. an integer b with $ab \equiv 1 \pmod{p}$.
-extended_gcd	$a\ b$	Computes integers g , u , and v such that $g = \gcd(a, b) = ua + vb$.
-power_mod	$a\ k\ n$	Compute the power $a^k \pmod{n}$.
-discrete_log	$y\ a\ m$	Computes x such that $a^x \equiv y \pmod{m}$. Employs a brute force algorithm.
-square_root	n	computes $\lfloor \sqrt{a} \rfloor$ over the integers.
-square_root_mod	$a\ p$	Computes an integer b such that $b^2 \equiv a \pmod{p}$. Here, p must be a prime number.
-all_square_roots_mod_n	$a\ n$	Computes all integers b such that $b^2 \equiv a \pmod{n}$.
-count_subprimitive	Q_max H_max	Count subprimitive polynomials for $q \leq Q_max$
-order_of_q_mod_n	$q\ n_{\min}\ n_{\max}$	Computes the order $\text{ord}(q, n)$ of q modulo n for all n with $n_{\min} \leq n \leq n_{\max}$ for which $\gcd(n, q) = 1$. Also computes $\varphi(n)$ and $\varphi(n)/\text{ord}(q, n)$.
-eulerfunction_interval	$n1\ n2$	Compute Euler's totient function $\varphi(n)$ for all n with $n1 \leq n \leq n2$.
-jacobi	$a\ p$	Computes the Jacobi symbol $\left(\frac{a}{p}\right)$
-Chinese_remainders	$R\ M$	Solves a system of congruences with remainders R and moduli M . R and M must be vectors whose labels are given.
-draw_mod_n	option-string options	Draws the integers modulo n on a circle. For possible options, see Table 11.10.

Table 10.2: Basic Number Theory Commands

Number Theoretic Commands		
Command	Arguments	Purpose
<code>-sift_smooth</code>	a n primes	Computes all smooth numbers in the interval $[a, a+n-1]$. Smooth means that they factor completely over the list of primes given.
<code>-random</code>	n fname	Creates n random numbers and writes them to the csv file fname
<code>-random_last</code>	n	Creates n random numbers prints the last one
<code>-affine_sequence</code>	a b p	Splits the interval $[0, p-1]$ into affine sequences of the form $x_{n+1} = ax_n + b \pmod{p}$

Table 10.3: Number Theoretic Commands

Code Snippet 450

```

1    PR_2_1000:
2    ▷ $(ORBITER) -v 1 -smallest_primitive_root_interval 2 1000
3    ▷ $(ORBITER) -v 1 -csv_file_latex 1 \
4    ▷ ▷ primitive_element_table_2_1000.csv
5    ▷ $(PDFLATEX) primitive_element_table_2_1000.tex
6    ▷ $(OPEN) primitive_element_table_2_1000.pdf
7

```

finds all primitive roots for all primes between 2 and 1000.

The command

Code Snippet 451

```

1    PE_number_2_1000:
2    ▷ $(ORBITER) -v 1 -number_of_primitive_roots_interval 2 1000
3    ▷ $(ORBITER) -v 1 -csv_file_latex 1 table_number_of_pe_2_1000.csv
4    ▷ $(PDFLATEX) table_number_of_pe_2_1000.tex
5    ▷ $(OPEN) table_number_of_pe_2_1000.pdf
6

```

determines the number of primitive elements for all primes between 2 and 1000.

The command

Code Snippet 452

```

1    PR_915839:
2    ▷ $(ORBITER) -v 5 -primitive_root 915839
3

```

computes a primitive root modulo 915839. The answer is 43085. Let us check this result. The command

Code Snippet 453

```

1    PR_915839_check:
2    ▷ $(ORBITER) -v 5 -power_mod 43085 49842 915839
3

```

computes

$$43085^{49842} \bmod 915839$$

which is 487320.

The command `-discrete_log` can be used to compute the discrete logarithm of a modulo p with respect to b . This means, a number k is computed such that

$$b^k \equiv a \pmod{p}.$$

For instance, the discrete log of 487320 with respect to the base 43085 modulo 915839 is 49842, based on the previous example. We can compute the discrete logarithm using the command

Code Snippet 454

```

1    DL_915839:
2    ▷ $(ORBITER) -v 5 -discrete_log 487320 43085 915839
3

```

This command can be quite expensive.

Computing inverses modulo a prime p is possible using the `-inverse_mod` command. The command

Code Snippet 455

```

1    IM:
2    ▷ $(ORBITER) -v 5 -inverse_mod 1865025205 2147483647
3

```

computes the inverse of 1865025205 modulo 2147483647 which is 579785381.

A different way of computing the inverse is by using the 1-trick. This approach utilizes the lattice linear combination of the gcd. Given two numbers a and b , say, we can find two integers u and v , say, such that

$$\gcd(a, b) = ua + vb, \quad u, v \in \mathbb{Z}.$$

The `-extended_gcd` command can be used to compute g and u and v from a and b . For instance, the following code computes the gcd of $a = 2147483647$ and $b = 1865025205$.

Code Snippet 456

```

1    IM_gcd:
2    ▷ $(ORBITER) -v 5 -extended_gcd 1865025205 2147483647
3

```

The output is

$$1 = -503526232 * 2147483647 + 579785381 * 1865025205,$$

from which we see that $\gcd(a, b) = 1$ and $u = -503526232$ and $v = 579785381$. which is the gcd written as a lattice combination of the input arguments. The inverse of 1865025205 mod 2147483647 is $v = 579785381$.

In order to compute the modular power

$$a^e \bmod n,$$

the `-power_mod` command can be used. For instance,

Code Snippet 457

```
1    PM3a:
2    ▷ $(ORBITER) -v 5 -power_mod 16807 1073741823 2147483647
3
```

computes 16807 raised to the power 1073741823 modulo 2147483647, which is 2147483646.

The modular square root of a modulo p is any x such that

$$x^2 \equiv a \bmod p.$$

The command `-square_root_mod` can be used to compute modular square roots using the algorithm of Tonelli and Shanks (cf. [25]). For instance,

Code Snippet 458

```
1    sqrt_mod:
2    ▷ $(ORBITER) -v 2 -square_root_mod 33 41
3
```

finds that the square root of 33 mod 41 is 19, i.e.

$$19^2 \equiv 33 \bmod 41.$$

The command `order_of_q_mod_n` computes $\text{ord}(q, n)$, the order of q modulo n , for all n with $n_{\min} \leq n \leq n_{\max}$ and $\gcd(n, q) = 1$. It also computes Euler's totient function $\varphi(n)$ and the cofactor $\varphi(n)/\text{ord}(q, n)$. For instance,

Code Snippet 459

```
1    order_of_2_mod_n:
2    ▷ $(ORBITER) -v 3 -order_of_q_mod_n 2 3 51
3    ▷ $(ORBITER) -v 1 -csv_file_latex 1 \
4    ▷ ▷ order_of_q_mod_n_q2_3_51.csv
5    ▷ $(PDFLATEX) order_of_q_mod_n_q2_3_51.tex
6    ▷ $(OPEN) order_of_q_mod_n_q2_3_51.pdf
7
```

produces the output shown below

N	ORD	PHI	COF	N	ORD	PHI	COF	N	ORD	PHI	COF
3	2	2	1	53	52	52	1	103	51	102	2
5	4	4	1	55	20	40	2	105	12	48	4
7	3	6	2	57	18	36	2	107	106	106	1
9	6	6	1	59	58	58	1	109	36	108	3
11	10	10	1	61	60	60	1	111	36	72	2
13	12	12	1	63	6	36	6	113	28	112	4
15	4	8	2	65	12	48	4	115	44	88	2
17	8	16	2	67	66	66	1	117	12	72	6
19	18	18	1	69	22	44	2	119	24	96	4
21	6	12	2	71	35	70	2	121	110	110	1
23	11	22	2	73	9	72	8	123	20	80	4
25	20	20	1	75	20	40	2	125	100	100	1
27	18	18	1	77	30	60	2	127	7	126	18
29	28	28	1	79	39	78	2	129	14	84	6
31	5	30	6	81	54	54	1	131	130	130	1
33	10	20	2	83	82	82	1	133	18	108	6
35	12	24	2	85	8	64	8	135	36	72	2
37	36	36	1	87	28	56	2	137	68	136	2
39	12	24	2	89	11	88	8	139	138	138	1
41	20	40	2	91	12	72	6	141	46	92	2
43	14	42	3	93	10	60	6	143	60	120	2
45	12	24	2	95	36	72	2	145	28	112	4
47	23	46	2	97	48	96	2	147	42	84	2
49	21	42	2	99	30	60	2	149	148	148	1
51	8	32	4	101	100	100	1	151	15	150	10

The command

Code Snippet 460

```

1 Eulerfunction_150:
2   ▷ $(ORBITER) -v 1 -eulerfunction.interval 1 150
3   ▷ $(ORBITER) -v 1 -csv_file.latex 1 \
4   ▷   table.eulerfunction_1_150.csv
5   ▷ $(PDFLATEX) table.eulerfunction_1_150.tex
6   ▷ $(OPEN) table.eulerfunction_1_150.pdf
7

```

computes Euler's totient function for all integers n with $1 \leq n \leq 150$. The result is shown below:

N	PHI	N	PHI	N	PHI	N	PHI	N	PHI	N	PHI
1	1	26	12	51	32	76	36	101	100	126	36
2	1	27	18	52	24	77	60	102	32	127	126
3	2	28	12	53	52	78	24	103	102	128	64
4	2	29	28	54	18	79	78	104	48	129	84
5	4	30	8	55	40	80	32	105	48	130	48
6	2	31	30	56	24	81	54	106	52	131	130
7	6	32	16	57	36	82	40	107	106	132	40
8	4	33	20	58	28	83	82	108	36	133	108
9	6	34	16	59	58	84	24	109	108	134	66
10	4	35	24	60	16	85	64	110	40	135	72
11	10	36	12	61	60	86	42	111	72	136	64
12	4	37	36	62	30	87	56	112	48	137	136
13	12	38	18	63	36	88	40	113	112	138	44
14	6	39	24	64	32	89	88	114	36	139	138
15	8	40	16	65	48	90	24	115	88	140	48
16	8	41	40	66	20	91	72	116	56	141	92
17	16	42	12	67	66	92	44	117	72	142	70
18	6	43	42	68	32	93	60	118	58	143	120
19	18	44	20	69	44	94	46	119	96	144	48
20	8	45	24	70	24	95	72	120	32	145	112
21	12	46	22	71	70	96	32	121	110	146	72
22	10	47	46	72	24	97	96	122	60	147	84
23	22	48	16	73	72	98	42	123	80	148	72
24	8	49	42	74	36	99	60	124	60	149	148
25	20	50	20	75	40	100	40	125	100	150	40

A power map sends a to a^k for some fixed k . Orbiter can compute power maps modulo p . For instance, the following command computes the function $a \mapsto a^k \pmod{11}$:

Code Snippet 461

```

1  power_function_2_mod_11:
2  ▷ $(ORBITER) -v 5 -power_function_mod_n 2 11
3  ▷ $(ORBITER) -v 1 -csv_file_latex 1 power_function_k2_n11.csv
4  ▷ $(PDFLATEX) power_function_k2_n11.tex
5  ▷ $(OPEN) power_function_k2_n11.pdf
6

```

The result is shown below:

A	APOWK
0	0
1	1
2	4
3	9
4	5
5	3
6	3
7	5
8	9
9	4
10	1

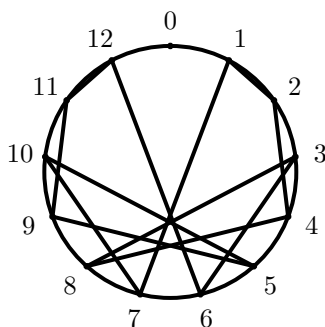
It is sometimes helpful to draw the elements modulo n on a circle, using the vertices of an n -gon to represent the field elements. For instance, for the command

Code Snippet 462

```

1  draw_mod_13:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define Do -draw_options -embedded -end \
4  ▷ ▷ -draw_mod_n \
5  ▷ ▷ ▷ "mod_n=on" \
6  ▷ ▷ ▷ -n 13 \
7  ▷ ▷ ▷ -file mod_13 \
8  ▷ ▷ ▷ -power_cycle 2 \
9  ▷ ▷ ▷ -label_nodes \
10 ▷ ▷ ▷ -draw_options Do \
11 ▷ ▷ -end
12 ▷ $(PDFLATEX) mod_13_draw.tex
13 ▷ $(OPEN) mod_13_draw.pdf
14
```

uses a 13-gon to represent the elements modulo 13 as shown below



Consecutive powers of 2 mod 13 are connected.

The command

Code Snippet 463

```

1  inverse_mod_a:
2  ▷ $(ORBITER) -v 2 -inverse_mod 18059241 58014043
3

```

computes the inverse of 18059241 modulo 58014043.

Orbiter can compute the Smith-Normal-Form of matrices with integer entries. The command

Code Snippet 464

```

1  smith_normal_form_1:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define M -vector \
4  ▷ ▷ ▷ -dense "1,2,3,4,5,6,7,8,9" \
5  ▷ ▷ ▷ -format 3 \
6  ▷ ▷ -end \
7  ▷ ▷ -smith_normal_form M
8

```

computes the Smith-Normal-Form of the matrix

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

The Legendre symbol evaluates if a number a is a square modulo an odd prime p . By definition,

$$\left(\frac{a}{p}\right) = \begin{cases} 1 & \text{if there exists } r \text{ s.t. } r^2 \equiv a \pmod{p} \\ -1 & \text{if there does not exist } r \text{ s.t. } r^2 \equiv a \pmod{p} \\ 0 & \text{if } p \text{ divides } a. \end{cases}$$

The Jacobi symbol generalizes the Legendre symbol to allow non-prime bottom arguments. By definition,

$$\left(\frac{a}{b}\right) = \prod_{i=1}^k \left(\frac{a}{r_i}\right)^{e_i},$$

where

$$b = \prod_{i=1}^k r_i^{e_i}$$

is the prime factorization of b with pairwise distinct primes r_i . The Jacobi symbol agrees with the Legendre symbol whenever the bottom argument b is an odd prime. Because there is no ambiguity, the same notation is used for the Jacobi symbol as for the Legendre symbol. Orbiter can compute Jacobi symbols. For instance, the command

Code Snippet 465

```

1  jacobi_a:
2  ▷ $(ORBITER) -v 5 -jacobi 2221 7817
3

```

computes the Jacobi symbol

$$\left(\frac{2221}{7817}\right).$$

In the Jacobi symbol, the denominator p has to be a positive odd integer. This command creates the file `jacobi_2221_7817.tex` which contains a detailed step-by-step description of the computation. After reformatting, the description looks like this:

$$\begin{aligned}
& \left(\frac{2221}{7817}\right) \\
&= \left(\frac{7817}{2221}\right) \cdot (-1)^{\frac{2221-1}{2} \cdot \frac{7817-1}{2}} \\
&= \left(\frac{7817}{2221}\right) \\
&= \left(\frac{1154}{2221}\right) \\
&= \left(\frac{2}{2221}\right) \cdot \left(\frac{577}{2221}\right) \\
&= (-1)^{\frac{2221^2-1}{8}} \cdot \left(\frac{577}{2221}\right) \\
&= (-1) \cdot \left(\frac{577}{2221}\right) \\
&= (-1) \cdot \left(\frac{2221}{577}\right) \cdot (-1)^{\frac{577-1}{2} \cdot \frac{2221-1}{2}} \\
&= (-1) \cdot \left(\frac{2221}{577}\right) \\
&= (-1) \cdot \left(\frac{490}{577}\right) \\
&= (-1) \cdot \left(\frac{2}{577}\right) \cdot \left(\frac{245}{577}\right) \\
&= (-1) \cdot (-1)^{\frac{577^2-1}{8}} \cdot \left(\frac{245}{577}\right) \\
&= (-1) \cdot \left(\frac{245}{577}\right) \\
&= (-1) \cdot \left(\frac{577}{245}\right) \cdot (-1)^{\frac{245-1}{2} \cdot \frac{577-1}{2}} \\
&= (-1) \cdot \left(\frac{577}{245}\right) \\
&= (-1) \cdot \left(\frac{87}{245}\right) \\
&= (-1) \cdot \left(\frac{245}{87}\right) \cdot (-1)^{\frac{87-1}{2} \cdot \frac{245-1}{2}} \\
&= (-1) \cdot \left(\frac{245}{87}\right) \\
&= (-1) \cdot \left(\frac{71}{87}\right) \\
&= (-1) \cdot \left(\frac{87}{71}\right) \cdot (-1)^{\frac{71-1}{2} \cdot \frac{87-1}{2}} \\
&= \left(\frac{87}{71}\right) \\
&= \left(\frac{16}{71}\right) \\
&= \left(\frac{2}{71}\right)^4 \cdot \left(\frac{1}{71}\right) \\
&= \left((-1)^{\frac{71^2-1}{8}}\right)^4 \cdot \left(\frac{1}{71}\right) \\
&= \left(\frac{1}{71}\right) \\
&= 1
\end{aligned}$$

The answer 1 tells us that 2221 is a square modulo 7817. Because 7817 is prime, the Jacobi symbol and the Legendre symbol agree on this input pair. We can use the `square_root_mod` command to compute a square root of 2221 modulo 7817 and verify this fact. The command

Code Snippet 466

```

1      sqrt_mod_7817:
2      ▷ ▷ $(ORBITER) -v 2 -square_root_mod 2221 7817
3

```

yields that 7634 is a square root. Indeed,

$$7634^2 \equiv 2221 \pmod{7817}.$$

The next command illustrates how to solve a system of congruences with coprime moduli using the Chinese remainder theorem. Suppose we want to find the integer x such that

$$x \equiv 2 \pmod{5}$$

$$x \equiv 2 \pmod{6}$$

$$x \equiv 5 \pmod{7}$$

The following command creates one vector for the remainders and one for the moduli and then invokes the `-Chinese_remainders` command.

Code Snippet 467

```

1      Chinese_remainders_A:
2      ▷ ▷ $(ORBITER) -v 2 \
3      ▷ ▷ -define R -vector -dense "2,2,5" -end \
4      ▷ ▷ -define M -vector -dense "5,6,7" -end \
5      ▷ ▷ -Chinese_remainders R M
6

```

The answer is $x \equiv 152$ modulo 210.

The next example shows that the Chinese remainder algorithm is safe for large numbers. We pick two 10 digit prime numbers as moduli and solve

$$x \equiv 2 \pmod{2147483647}$$

$$x \equiv 3 \pmod{5915587277}$$

using the command

Code Snippet 468

```

1      Chinese_remainders_C2:
2      ▷ ▷ $(ORBITER) -v 2 \
3      ▷ ▷ -define R -vector -dense "2,3" -end \
4      ▷ ▷ -define M -vector -dense "2147483647,5915587277" -end \
5      ▷ ▷ -Chinese_remainders R M
6

```

The answer is

$$x \equiv 5684294357108828365 \pmod{12703626939758759219}.$$

A quick check with Maple shows that

$$\begin{aligned}5684294357108828365 \bmod 2147483647 &\equiv 2 \\5684294357108828365 \bmod 5915587277 &\equiv 3\end{aligned}$$

as required.

Representation Theory Activities		
Command	Arguments	Purpose
<code>-representation_on_polynomials</code>	<code>R</code>	Computes the representation of the given group on homogeneous polynomials from the ring <code>R</code> . This is a group theoretic activity as described in Section 5.5.

Table 10.4: Representation Theory Activities

10.2 Representation Theory

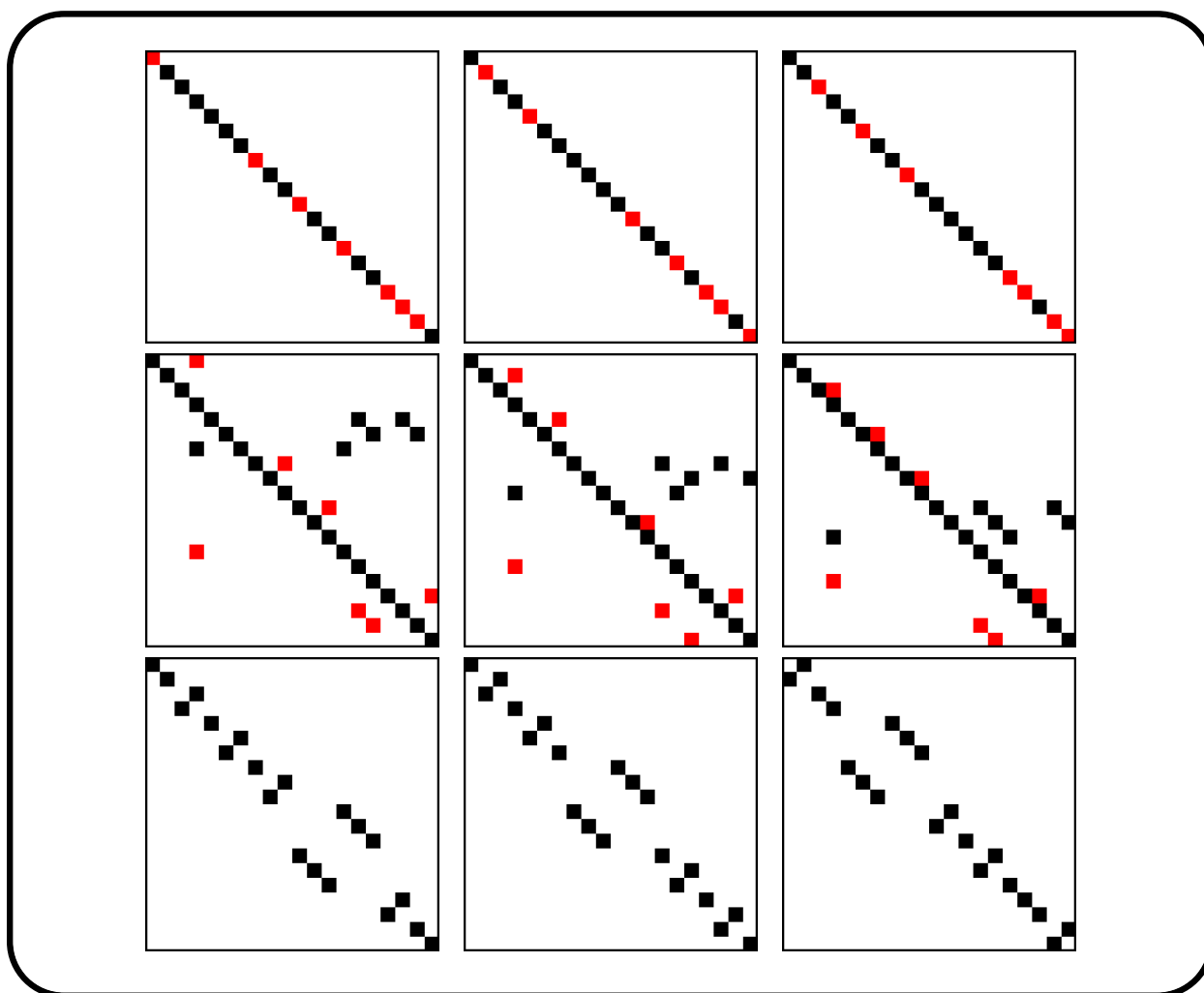
Table 10.4 list the Orbiter commands for group representations. The command

Code Snippet 469

```

1  representation_on_polynomials_of_degree_3:
2  ▷ $(ORBITER) -v 4 \
3  ▷ ▷ -define F -finite_field -q 3 -end \
4  ▷ ▷ -define G -linear_group -PGL 4 F -end \
5  ▷ ▷ -define R -polynomial_ring \
6  ▷ ▷ ▷ -field F \
7  ▷ ▷ ▷ -number_of_variables 4 \
8  ▷ ▷ ▷ -homogeneous_of_degree 3 \
9  ▷ ▷ ▷ -monomial_ordering_partition \
10 ▷ ▷ ▷ -variables "X,Y,Z,W" "X,Y,Z,W" \
11 ▷ ▷ -end \
12 ▷ ▷ -with G -do \
13 ▷ ▷ -group_theoretic_activity \
14 ▷ ▷ ▷ -representation_on_polynomials R \
15 ▷ ▷ -end
16 ▷ $(ORBITER) -v 2 \
17 ▷ ▷ -loop L 0 9 1 -draw_matrix \
18 ▷ ▷ ▷ -input_csv_file PGL_4_3_rep_3.%L.csv \
19 ▷ ▷ ▷ -box_width 40 -bit_depth 24 -partition 3 20 20 -end \
20 ▷ ▷ -end_loop L
21
```

creates $G = \text{PGL}(4, 3)$ and computes the representation on polynomials of degree 3 in 4 variables. The representation has degree 20. The second command produces bitmap drawings for the representing matrices associated with a generating set of the group.



Cryptographic Commands		
Command	Arguments	Purpose
-solovay_strassen	$a \ n$	Performs n Solovay / Strassen tests on the number a
-miller_rabin	$a \ n$	Performs n Miller / Rabin tests on the number a
-fermat_test	$a \ n$	Performs n Fermat tests on the number a
-find_pseudoprime	$a \ n_1 \ n_2 \ n_3$	Computes a pseudoprime which survives n_1 Fermat tests, n_2 Miller Rabin tests, n_3 Solovay Strassen tests
-find_strong_pseudoprime	$a \ n_1 \ n_2$	Computes a pseudoprime which survives n_1 Fermat tests and n_2 Miller Rabin tests
-RSA_encrypt_text	$d \ n \ b \ \text{text}$	Using blocks of b letters at a time, encrypt "text" using RSA with exponent d modulo n
-RSA	$d \ n \ \text{list-of-integers}$	encrypt the given sequence of integers using RSA with exponent d modulo n
-RSA_setup	$n \ n_1 \ n_2$	Setup RSA-key with n bits using a pseudoprime with n_1 Solovay-Strassen tests and n_2 Miller-Rabin tests

Table 10.5: Cryptographic Commands

10.3 Cryptography

In Table 10.5, some global cryptographic commands are shown. Some of these commands require a finite field and appear as a finite field activity, see Table 10.6. For instance,

Code Snippet 470

```

1  EC_add:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 11 -end \
4  ▷ ▷ -with F -do \
5  ▷ ▷ -finite_field_activity \
6  ▷ ▷ -EC_add 1 3 "1,4" "1,4" -end
7
```

adds the point $(1, 4)$ on the curve $y^2 = x^3 + x + 3 \pmod{11}$ to itself. The command

Code Snippet 471

```

1  EC_cyclic_subgroup:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 11 -end \
4  ▷ ▷ -with F -do \
5  ▷ ▷ -finite_field_activity \
6  ▷ ▷ -EC_cyclic_subgroup 1 3 "1,4" -end
7
```

computes the cyclic subgroup generated by the point $(1, 4)$ on the curve $y^2 = x^3 + x + 3 \pmod{11}$. The command

Finite Field Activities for Cryptography		
Command	Arguments	Purpose
-EC_add	$a \ b \ i_1 \ i_2$	On the elliptic curve $y^2 \equiv x^3 + ax + b$ in $\mathbb{F}_q$, add the points with indices i_1 and i_2 , each given as a pair x, y
-EC_points	$a \ b$	Computes all points of the elliptic curve $y^2 \equiv x^3 + ax + b$ over $\mathbb{F}_q$
-EC_multiple_of	$a \ b \ pt \ n$	Computes the n fold multiple of the given point pt on the elliptic curve $y^2 \equiv x^3 + ax + b$ over $\mathbb{F}_q$
-EC_cyclic_subgroup	$a \ b \ pt$	Computes the cyclic subgroup generated by the given point pt on the elliptic curve $y^2 \equiv x^3 + ax + b$ over $\mathbb{F}_q$
-EC_Koblitz_encoding	$a \ b \ s \ pt \ plain$	Computes the Koblitz encoding of “plain” (all caps) on the elliptic curve $y^2 \equiv x^3 + ax + b$ over $\mathbb{F}_q$ using the base point pt and the secret exponent s
-EC_bsgs	$a \ b \ pt \ n \ cipher$	Prepare the baby-step giant-step tables for the ciphertext “cipher” on the elliptic curve $y^2 \equiv x^3 + ax + b$ over $\mathbb{F}_q$ using the base point pt of order n
-EC_bsgs_decode	$a \ b \ pt \ n \ cipher \ round-keys$	Decodes the ciphertext “cipher” on the elliptic curve $y^2 \equiv x^3 + ax + b$ over $\mathbb{F}_q$ using the base point pt of order n and the round keys “keys”
-EC_discrete_log	$a \ b \ pt \ base-pt$	Computes the elliptic curve discrete log analogue of pt with respect to $base-pt$ on the elliptic curve $y^2 \equiv x^3 + ax + b$ over $\mathbb{F}_q$
-NTRU_encrypt	$N \ p \ H \ R \ M$	NTRU encryption for the message $M(X)$ using the public key $H(X)$ and one-time-key $R(X)$.
-polynomial_center_lift	$A(X)$	Compute the center lift mod q for the coefficients of A
-polynomial_reduce_mod_p	$p \ A(X)$	Reduce the coefficients of the polynomial A modulo p

Table 10.6: Finite Field Activities for Cryptography

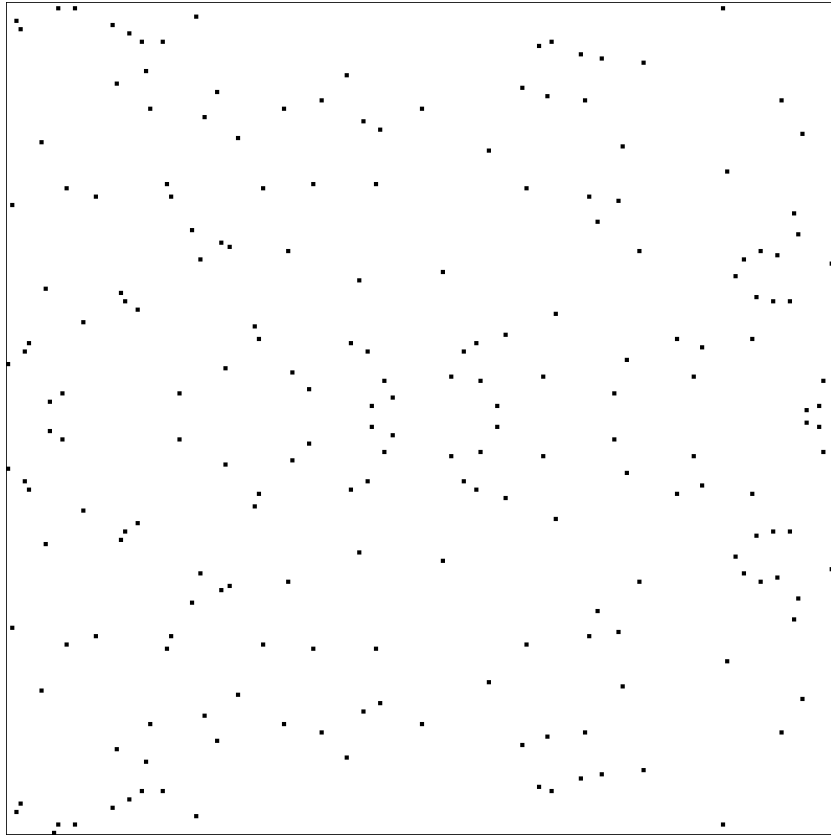
Code Snippet 472

```

1   EC_points_199:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 199 -end \
4   ▷ ▷ -with F -do \
5   ▷ ▷ -finite_field_activity \
6   ▷ ▷ -EC_points "EC_5_7_q199" 5 7 -end
7   ▷ $(ORBITER) -v 2 \
8   ▷ ▷ -draw_matrix -input_csv_file EC_5_7_q199_points.xy.csv \
9   ▷ ▷ -box_width 10 -bit_depth 24 \
10  ▷ ▷ -partition 2 199 199 -end
11

```

computes all points on the curve $y^2 = x^3 + 5x + 7 \pmod{199}$ and produces a bitmap drawing of the points in the affine plane:



Both the x -axis and the y -axis are indexed by the field elements from 0 to 198.

The command

Code Snippet 473

```

1   EC_Koblitz_encoding:
2   ▷ $(ORBITER) -v 6 -seed 17 \

```

```

3      ▷ ▷ -define F -finite_field -q 199 -end \
4      ▷ ▷ -with F -do \
5      ▷ ▷ -finite_field_activity \
6      ▷ ▷ -EC_Koblitz_encoding 5 7 67 "147,164" "DEADBEEF" \
7      ▷ ▷ -end
8

```

encode the message “DEADBEEF” on the curve $y^2 = x^3 + 5x + 7 \pmod{199}$ using the base point $(147, 164)$ and the secret key 67. The i th input character is encoded as two points (R_i, T_i) on the curve using the Elgamal scheme. A random round key is generated for each plaintext symbol. As seen in this example, the `-seed` command can be used to seed the random number generator with an arbitrary integer (here 17).

The command

Code Snippet 474

```

1      EC_bsgs:
2      ▷ $(ORBITER) -v 2 \
3      ▷ ▷ -define F -finite_field -q 199 -end \
4      ▷ ▷ -with F -do \
5      ▷ ▷ -finite_field_activity \
6      ▷ ▷ -EC_bsgs 5 7 "147,164" 212 \
7      ▷ ▷ "172,158,45,195,50,22,10,103,55,33,50,22,\
8      ▷ ▷ 145,105,31,74,73,155,67,60,25,6" \
9      ▷ ▷ -end
10

```

performs a baby-step-giant-step brute force attack on the ciphertext sequence

$$R_i = (172, 158), (45, 195), (50, 22), (10, 103), (55, 33), \\ (50, 22), (145, 105), (31, 74), (73, 155), (67, 60), (25, 6),$$

using the base point $(147, 164)$ on the curve $y^2 = x^3 + 5x + 7 \pmod{199}$, assuming a group order of 212. The command

Code Snippet 475

```

1      EC_bsgs_decode:
2      ▷ $(ORBITER) -v 2 \
3      ▷ ▷ -define F -finite_field -q 199 -end \
4      ▷ ▷ -with F -do \
5      ▷ ▷ -finite_field_activity \
6      ▷ ▷ -EC_bsgs_decode 5 7 "129,176" 212 \
7      ▷ ▷ "127,188,51,141,85,29,106,90,41,105,179,71,\
8      ▷ ▷ 171,2,16,197,183,72,27,129,37,10" \
9      ▷ ▷ "50,179,169,13,153,169,115,116,188,110,176" \
10     ▷ ▷ -end
11

```

decodes the ciphertext sequence

$$T_i = (127, 188), (51, 141), (85, 29), (106, 90), (41, 105), (179, 71), \\ (171, 2), (16, 197), (183, 72), (27, 129), (37, 10),$$

assuming round keys

$$k_i = 50, 179, 169, 13, 153, 169, 115, 116, 188, 110, 176,$$

using the base point $(147, 164)$ on the curve $y^2 = x^3 + 5x + 7 \pmod{199}$, and assuming a group order of 212.

The next sequence of examples discusses the NTRU cryptosystem (cf. Example 7.53 in [40]). In the example, we choose the parameters of the cryptosystem to be $(N, p, q, d) = (7, 41, 3, 2)$. Orbiter uses the following convention for polynomials over a finite field $\mathbb{F}_q$: The coefficients of $A(X) = a_0 + a_1X + \cdots + a_dX^d$ are listed as a sequence, starting with the constant term and ending with the leading coefficient. The cryptosystem requires coefficients a_i in the range $-\frac{p}{2} \leq a_i \leq \frac{p}{2}$. So, in an extension to the conventions for field elements in $\mathbb{F}_q$, Orbiter allows negative coefficients as well. The assumption is that q is prime and negative coefficients are considered modulo q . In the example, Alice picks the private polynomials $f(x) = x^6 - x^4 + x^3 + x^2 - 1$ (with $d+1$ coefficients equal to plus one and d coefficients equal to minus one) and $g(x) = x^6 + x^4 - x^2 - x$ with d coefficients plus one and d coefficients minus one. We also need the polynomial $x^N - 1$. The makefile commands

Code Snippet 476

```
1  NTRU_N=7
2
```

Code Snippet 477

```
1  NTRU_P=3
2
```

Code Snippet 478

```
1  NTRU_Q=41
2
```

Code Snippet 479

```
1  NTRU_D=2
2
```

Code Snippet 480

```
1  NTRU_XN1="-1,0,0,0,0,0,1,"
2
```

Code Snippet 481

```
1  ALICE_PRIVATE_F="-1,0,1,1,-1,0,1"
2
```

Code Snippet 482

```
1  ALICE_PRIVATE_G="0,-1,-1,0,1,0,1"
2
```

are used to set up the appropriate variables according to these choices.

Regarding the NTRU set-up, Alice needs to compute her private keys $F_p(x)$ and $F_q(x)$. These two polynomials are defined as follows:

1. $F_p(x)$ is the inverse of $f(x)$ in $\mathbb{F}_p[x]/(x^n - 1)$,
2. $F_q(x)$ the inverse of $f(x)$ in $\mathbb{F}_q[x]/(x^n - 1)$.

To this end, we can use the `extended_gcd_for_polynomials` command from Table 10.3. The following two makefile commands do the job:

Code Snippet 483

```
1  NTRU-Alice1:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q $(NTRU_Q) -end \
4  ▷ ▷ -with F -do \
5  ▷ ▷ -finite_field_activity \
6  ▷ ▷ -extended_gcd_for_polynomials \
7  ▷ ▷ ▷ $(NTRU_XN1) $(ALICE_PRIVATE_F) \
8  ▷ ▷ -end
9
```

Code Snippet 484

```
1  ALICE_PRIVATE_FQ="37,2,40,21,31,26,8"
2
```

Code Snippet 485

```
1  NTRU-Alice2:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q $(NTRU_P) -end \
4  ▷ ▷ -with F -do \
5  ▷ ▷ -finite_field_activity \
6  ▷ ▷ -extended_gcd_for_polynomials \
7  ▷ ▷ ▷ $(NTRU_XN1) $(ALICE_PRIVATE_F) \
8  ▷ ▷ -end
9
```

The resulting polynomials (indicated as comments by means of the `#` symbol) are again encoded as makefile variables.

Code Snippet 486

```
1  ALICE_PRIVATE_FP="1,1,1,1,0,2,1"
2
```

There is a chance that the polynomial $f(x)$ does not have an inverse in either $\mathbb{F}_p[x]$ or in $\mathbb{F}_q[x]$. In that case, Alice simply chooses a different polynomial $f(x)$ and tries again. Alice can now compute her public key:

Code Snippet 487

```

1  NTRU_Alice_public_key:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q $(NTRU_Q) -end \
4  ▷ ▷ -with F -do \
5  ▷ ▷ -finite_field_activity \
6  ▷ ▷ -polynomial_mult_mod $(ALICE_PRIVATE_F) \
7  ▷ ▷ ▷ $(ALICE_PRIVATE_G) $(NTRU_XN1) \
8  ▷ ▷ -end
9

```

Code Snippet 488

```

1  ALICE_PUBLIC_KEY="30,26,8,38,2,40,20"
2

```

The public key is assigned to the makefile variable `ALICE_PUBLIC_KEY`. Now, Bob chooses his message to Alice and his one-time-key. The message must be the center lift of a polynomial in $\mathbb{F}_p[x]$. The round-key must have exactly d coefficients one and d coefficients -1 (rest zeroes).

Code Snippet 489

```

1  BOB_MESSAGE="1,-1,1,1,0,-1"
2

```

Code Snippet 490

```

1  BOB_ONE_TIME_KEY="-1,1,0,0,0,-1,1"
2

```

The encryption proceeds using the `NTRU_encrypt` command, and the result is stored in the makefile variable `BOB_ENCRYPT`:

Code Snippet 491

```

1  NTRU_encrypt:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q $(NTRU_Q) -end \
4  ▷ ▷ -with F -do \
5  ▷ ▷ -finite_field_activity \
6  ▷ ▷ -NTRU_encrypt \
7  ▷ ▷ $(NTRU_N) $(NTRU_P) $(ALICE_PUBLIC_KEY) \
8  ▷ ▷ $(BOB_ONE_TIME_KEY) $(BOB_MESSAGE) \
9  ▷ ▷ -end
10

```

Code Snippet 492

```

1  BOB_ENCRYPT="25,3,40,2,4,19,31"
2

```


Decryption is done in five steps.

Code Snippet 493

```

1  NTRU_decrypt1:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q $(NTRU_Q) -end \
4  ▷ ▷ -with F -do \
5  ▷ ▷ -finite_field_activity \
6  ▷ ▷ -polynomial_mult_mod $(ALICE_PRIVATE_F) \
7  ▷ ▷ ▷ $(BOB_ENCRYPT) $(NTRU_XN1) \
8  ▷ ▷ -end
9

```

Code Snippet 494

```

1  ALICE_C1="40,1,40,40,33,10,1"
2

```

Code Snippet 495

```

1  NTRU_decrypt2:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q $(NTRU_Q) -end \
4  ▷ ▷ -with F -do \
5  ▷ ▷ -finite_field_activity \
6  ▷ ▷ -polynomial_center_lift $(ALICE_C1) -end
7

```

Code Snippet 496

```

1  ALICE_C2="-1,1,-1,-1,-8,10,1"
2

```

Code Snippet 497

```

1  NTRU_decrypt3:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q $(NTRU_P) -end \
4  ▷ ▷ -with F -do \
5  ▷ ▷ -finite_field_activity \
6  ▷ ▷ -polynomial_reduce_mod_p $(ALICE_C2) -end
7

```

Code Snippet 498

```

1  ALICE_C3="2,1,2,2,1,1,1"
2

```

Code Snippet 499

```
1  NTRU_decrypt4:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q $(NTRU_Q) -end \
4  ▷ ▷ -with F -do \
5  ▷ ▷ -finite_field_activity \
6  ▷ ▷ -polynomial_mult_mod $(ALICE_PRIVATE_FP) \
7  ▷ ▷ ▷ $(ALICE_C3) $(NTRU_XN1) \
8  ▷ ▷ -end
9
```

Code Snippet 500

```
1  ALICE_C4="1,2,1,1,0,2"
2
```

Code Snippet 501

```
1  NTRU_decrypt5:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q $(NTRU_P) -end \
4  ▷ ▷ -with F -do \
5  ▷ ▷ -finite_field_activity \
6  ▷ ▷ -polynomial_center_lift $(ALICE_C4) -end
7
```

Decryption produces Bob's message to Alice.

Chapter 11

Coding Theory

11.1 Introduction

Table 11.1 lists global Orbiter commands related to coding theory.

The command

Code Snippet 502

```
1   Allen.Gates_noise_1_percent:
2   ▷ cp $(ORBITER_PATH)/examples/users_guide/allen.Gates.bmp .
3   ▷ $(ORBITER) -v 3 \
4   ▷ ▷ -random_noise_in_bitmap_file \
5   ▷ ▷ allen.Gates.bmp \
6   ▷ ▷ allen.Gates_1.bmp \
7   ▷ ▷ 1 100
8   ▷ $(OPEN) allen.Gates_1.bmp
9
```

simulates random noise at the 1 percent level applied to the file `allen.Gates.bmp`, see below

Global Coding Theoretic Commands		
Command	Arguments	Purpose
-make_macwilliams_system	$q\ n\ k$	Create the MacWilliams equations for the weight enumerator of the dual code.
-table_of_bounds	$n_{\max}\ q$	Make a table of bounds for q -ary linear codes for all $k \leq n \leq n_{\max}$
-make_bounds_for_d_given_n_and_k_and_q	$n\ k\ q$	Make bounds for the minimum distance of an $[n, k]_q$ code
-introduce_errors	params	Introduce errors to a file. See Table 11.8.
-check_errors	params	Find errors in a CRC coded file. See Table 11.8.
-extract_block	params	Extract a block from a CRC coded file. See Table 11.8.
-random_noise_in_bitmap_file	input output $n\ d$	Apply random noise at the d/n level to the bitmap file "input" and write to "output."
-random_noise_of_burst_type_in_bitmap_file	input output $n\ d\ b$	Apply random noise of type burst with max burst length b at the d/n level to the bitmap file "input" and write to "output"

Table 11.1: Global Coding Theoretic Commands



The original is on the top. The effect of noise can be seen in the picture below. The picture shows Paul Allen and Bill Gates in the early 1970s.

The command

Code Snippet 503

```

1  nonlinear_code.long.n7.k4:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -define C -code -field F \
5  ▷ ▷ ▷ -nonlinear_code.long 7 4 \
6  ▷ ▷ ▷ "0,1" \
7  ▷ ▷ ▷ "1,2,3" \
8  ▷ ▷ ▷ "0,1,3,4,6" \
9  ▷ ▷ ▷ "2,3,5,6" \
10 ▷ ▷ -end \
11 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
12 ▷ ▷ ▷ -export_codewords codewords.csv \
13 ▷ ▷ -end \

```

```

14  ▷ ▷ -with C -and F -do -coding_theoretic_activity \
15  ▷ ▷ ▷ -all_distances \
16  ▷ ▷ -end \
17  ▷ ▷ -with C -and F -do -coding_theoretic_activity \
18  ▷ ▷ ▷ -all_external_distances \
19  ▷ ▷ -end \
20  ▷ ▷ -tally_column nonlinear_code_n7_k4_all_external_distances.csv C0
21

```

creates the code $\mathcal{C}$ in $H(7, 2)$ whose codewords are

$$\mathbf{w}_1 = 1100000, \mathbf{w}_2 = 0111000, \mathbf{w}_3 = 1101101, \mathbf{w}_4 = 0011011.$$

The command computes the distance matrix (d_{ij}) where $d_{ij} = d(\mathbf{w}_i, \mathbf{w}_j)$ the distance between codeword $\mathbf{w}_i$ and codeword $\mathbf{w}_j$. Moreover, the command computes the external distances for all elements not in the code. Here, for $\mathbf{y} \notin \mathcal{C}$ we have

$$d(\mathbf{y}, \mathcal{C}) = \min\{d(\mathbf{y}, \mathbf{w}) \mid \mathbf{w} \in \mathcal{C}\}$$

the external distance of $\mathbf{y}$ w.r.t. $\mathcal{C}$. Both, the distance matrix and the external distance vector are written to csv files.

The command

Code Snippet 504

```

1  nonlinear_code_n7_d4:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -define C -code -field F \
5  ▷ ▷ ▷ -nonlinear_code 7 \
6  ▷ ▷ ▷ "0,15,51,60,85,90,102,105" \
7  ▷ ▷ -end \
8  ▷ ▷ -with C -and F -do -coding_theoretic_activity \
9  ▷ ▷ ▷ -all_distances \
10 ▷ ▷ -end \
11 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
12 ▷ ▷ ▷ -all_external_distances \
13 ▷ ▷ -end \
14 ▷ ▷ -tally_column nonlinear_code_n7_k8_all_external_distances.csv C0
15

```

creates the code $\mathcal{C}$ in $H(7, 2)$ whose codewords are

$$0, 15, 51, 60, 85, 90, 102, 105.$$

Again, the distance matrix and the external distances are computed and written to file.

The command

Code Snippet 505

```

1  nonlinear_code_n7_d4.classify:
2  ▷ $(ORBITER) -v 5 \
3  ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \

```

```

4      ▷ ▷ -define Do -draw_options \
5      ▷ ▷ ▷ -radius 200 -embedded \
6      ▷ ▷ -end \
7      ▷ ▷ -define Control -poset_classification_control \
8      ▷ ▷ ▷ -problem_label codes_n7_d4 \
9      ▷ ▷ ▷ -depth 10 \
10     ▷ ▷ ▷ -test_mindist_nonlinear 4 \
11     ▷ ▷ ▷ -draw_options Do \
12     ▷ ▷ ▷ -W \
13     ▷ ▷ -end \
14     ▷ ▷ -define F -finite_field -q 2 -end \
15     ▷ ▷ -define G -linear_group -GL 7 F -monomial -end \
16     ▷ ▷ -define Orb -orbits \
17     ▷ ▷ ▷ -group G \
18     ▷ ▷ ▷ -group_action G \
19     ▷ ▷ ▷ -generators G \
20     ▷ ▷ ▷ -on_subsets 10 Control \
21     ▷ ▷ -end \
22     ▷ ▷ -define ro \
23     ▷ ▷ ▷ -poset_classification_report_options \
24     ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
25     ▷ ▷ ▷ ▷ -draw_options Do \
26     ▷ ▷ -end \
27     ▷ ▷ -with Orb -do -orbits_activity \
28     ▷ ▷ ▷ -report "" \
29     ▷ ▷ ▷ -report_options ro \
30     ▷ ▷ -end \
31

```

classifies the codes in $H(7, 2)$ whose codewords are at least 4 apart.

The command

Code Snippet 506

```

1      Hamming_space_4.2_distance_matrix:
2      ▷ $(ORBITER) -v 2 \
3      ▷ ▷ -define F -finite_field -q 2 -end \
4      ▷ ▷ -with F -do -coding_theoretic_activity \
5      ▷ ▷ ▷ -Hamming_space_distance_matrix 4 \
6      ▷ ▷ -end
7

```

creates the distance matrix of the Hamming graph $H(n, q)$. The data is written to the file `Hamming_n4_q2.csv`. The command

Code Snippet 507

```

1      Hamming_space_4.2_distance_matrix.draw:
2      ▷ $(ORBITER) -v 2 \
3      ▷ ▷ -draw_matrix \
4      ▷ ▷ ▷ -input_csv_file Hamming_n4_q2.csv \

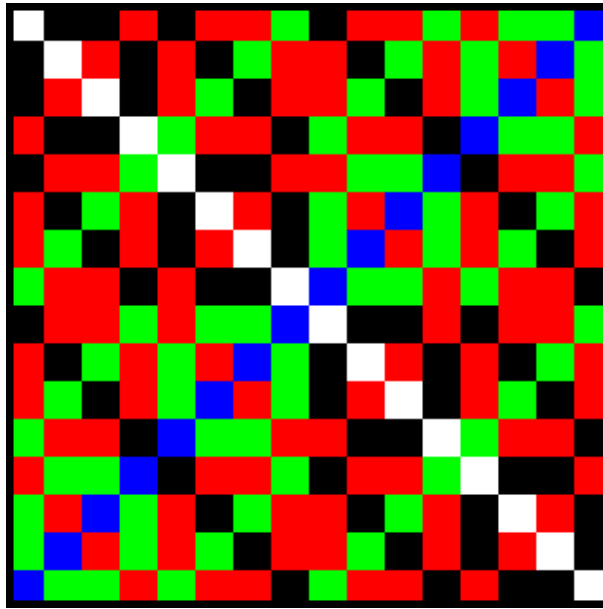
```

```

5   ▷ ▷ ▷ -box_width 20 -bit_depth 24 \
6   ▷ ▷ ▷ -partition 4 16 16 \
7   ▷ ▷ -end
8   ▷ $(OPEN) Hamming_n4.q2_draw.bmp
9

```

produces the bitmap graphic Hamming_n4.q2_draw.bmp shown below:



The command

Code Snippet 508

```

1   Hamming_code_macwilliams:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -make_macwilliams_system 7 4 2
4   ▷ $(PDFLATEX) MacWilliams_n7_k4.q2.tex
5   ▷ $(OPEN) MacWilliams_n7_k4.q2.pdf
6

```

creates the coefficient matrix of the MacWilliams system for the $[7, 4, 2]$ Hamming code:

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 7 & 5 & 3 & 1 & -1 & -3 & -5 & -7 \\ 21 & 9 & 1 & -3 & -3 & 1 & 9 & 21 \\ 35 & 5 & -5 & -3 & 3 & 5 & -5 & -35 \\ 35 & -5 & -5 & 3 & 3 & -5 & -5 & 35 \\ 21 & -9 & 1 & 3 & -3 & -1 & 9 & -21 \\ 7 & -5 & 3 & -1 & -1 & 3 & -5 & 7 \\ 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \end{bmatrix}$$

For examples concerning the bounds, see Section 11.9.

Consider the $[5, 2]_2$ code whose codewords are $\{0, 7, 25, 30\}$. We start with a makefile variable:

Code Snippet 509

```
1 CODE_5_2_3_CODEWORDS="0,7,25,30"
2
```

The following command creates the code and draws the code diagram in Hamming space:

Code Snippet 510

```
1 code_5_2_3_diagram:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 2 -end \
4   ▷ ▷ -with F -do -coding_theoretic_activity \
5   ▷ ▷ ▷ -general_code_binary 5 "code_5_2_3" \
6   ▷ ▷ ▷ $(CODE_5_2_3_CODEWORDS) \
7   ▷ ▷ ▷ -metric_balls 1 \
8   ▷ ▷ -end
9   ▷ $(ORBITER) -v 2 \
10  ▷ ▷ -draw_matrix \
11  ▷ ▷ ▷ -input_csv_file code_5_2_3_char_func_5.4.csv \
12  ▷ ▷ ▷ -box_width 25 -bit_depth 24 \
13  ▷ ▷ ▷ -partition 4 8 4 \
14  ▷ ▷ -end
15
```

The Hamming graph $H(5, 2)$ can be created with the following command:

Code Snippet 511

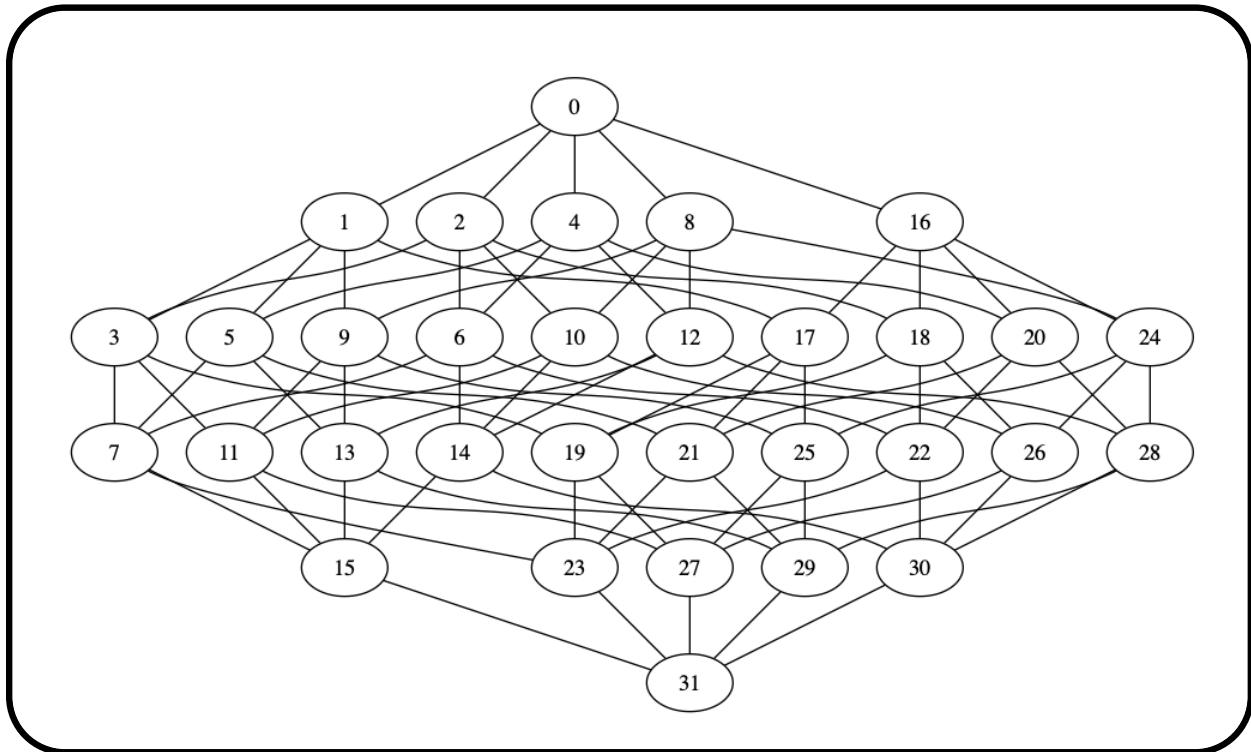
```
1 Hamming_5_2_graph:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define G -graph -Hamming 5 2 -end \
4   ▷ ▷ -with G -do \
5   ▷ ▷ -graph_theoretic_activity -export_csv -end \
6   ▷ ▷ -with G -do \
7   ▷ ▷ -graph_theoretic_activity -export_graphviz -end \
```

```

8   ▷ ▷ -with G -do \
9   ▷ ▷ -graph_theoretic_activity -save -end
10  ▷ $(ORBITER) -v 2 -draw_matrix \
11  ▷ ▷ -input_csv_file Hamming_5.2.csv \
12  ▷ ▷ -box_width 8 -bit_depth 24 -partition 4 32 32 -end
13  ▷ dot -Tpng Hamming_5.2.gv >Hamming_5.2.png
14

```

Using the unix dot program, this command sequence creates the drawing of $H(5, 2)$



The command

Code Snippet 512

```

1   Sylvester_Hadamard_code_8:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 3 -end \
4   ▷ ▷ -with F -do -coding_theoretic_activity \
5   ▷ ▷ ▷ -Sylvester_Hadamard_code 8 \
6   ▷ ▷ -end
7

```

creates the code arising from a Sylvester-type Hadamard matrix of order 8.

11.2 Linear Codes

Table 11.2 lists Orbiter commands to create codes. Tables 11.3-11.5 list coding theoretic activities in Orbiter. Table 11.6 shows ways to create new codes from old.

The following command creates the first order Reed-Muller code in three variables:

Code Snippet 513

```

1    RM_3_1:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 2 -end \
4    ▷ ▷ -define C -code -field F \
5    ▷ ▷ ▷ -Reed_Muller 3 \
6    ▷ ▷ -end \
7    ▷ ▷ -with C -and F -do -coding_theoretic_activity \
8    ▷ ▷ ▷ -export_magma RM_3_1.magma \
9    ▷ ▷ -end \
10   ▷ ▷ -with C -do -coding_theoretic_activity \
11   ▷ ▷ ▷ -report \
12   ▷ ▷ -end
13   ▷ $(PDFLATEX) RM_m3_code_n8_k4_q2.tex
14   ▷ $(OPEN) RM_m3_code_n8_k4_q2.pdf
15
```

Consider the linear code with generator matrix

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}.$$

The row vectors correspond to the integers 60, 50, 41 in binary. We can use the following command to create the code:

Code Snippet 514

```

1    code_6:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 2 -end \
4    ▷ ▷ -define C -code -field F \
5    ▷ ▷ ▷ -basis 6 "60,50,41" \
6    ▷ ▷ -end \
7    ▷ ▷ -with C -and F -do -coding_theoretic_activity \
8    ▷ ▷ ▷ -make_diagram \
9    ▷ ▷ -end \
10   ▷ ▷ -with C -do -coding_theoretic_activity \
11   ▷ ▷ ▷ -report \
12   ▷ ▷ -end
13   ▷ $(ORBITER) -v 2 -draw_matrix \
14   ▷ ▷ -input_csv_file by_basis_n6_k3_char_func_6.8.csv \
15   ▷ ▷ -box_width 20 -bit_depth 24 \
16   ▷ ▷ -partition 2 "1,1,1,1,1,1,1" "1,1,1,1,1,1,1" \
17   ▷ ▷ -end

```

Creating Codes		
Command	Arguments	Purpose
-field	F	The field $F = \mathbb{F}_q$ over which the code is defined.
-generator_matrix	M	A generator matrix over F defining the code as Orbiter object.
-basis	n basis	A basis for the code as Orbiter ranks. The rows of the generator matrix are coded as integers using the binary representation.
-long_code	n genma-by-rows	Create a code over $\mathbb{F}_q$ defined by a generator matrix. The rows of the generator matrix are given as a list of strings. Each string lists the positions in the codeword which are one.
-nonlinear_code	n codewords	Create a code over $\mathbb{F}_q$ of length n defined by a list of words. The codewords are given as string of comma-separated integers in base q .
-nonlinear_code_long	n k code-words	Create a code over $\mathbb{F}_q$ of length n with k codewords defined by a list of words. The codewords are given as a list of strings. Each string lists the positions in the codeword which are one. The command expects exactly k strings to follow.
-projective_set	nmk	A projective set defining the columns of a check matrix of a code. The projective set is in $\text{PG}(n - k - 1, q)$ where $nmk = n - k$ is given.
-columns_of_generator_matrix	k cols	The columns of a check matrix defining the code are given, using the ranking of elements of $\text{AG}(k, q)$.
-Reed_Muller	m	The first order Reed-Muller code with m variables.
-BCH	n d	BCH-code over $\mathbb{F}_q$ of length n with designed distance d . The minimum distance will be at least the designed distance.
-Reed_Solomon	n d	A Reed-Solomon code of length n with designed distance d . This requires that n divides $q - 1$.
-Gilbert_Varshamov	n k d	A Gilbert-Varshamov code with length n , dimension k and minimum distance d .
-ttpA	FQ	Twisted tensor product code of type A using the extension field FQ . The code arises from a hyperoval in $\mathbb{F}_{q^2}$ (Theorem 1 in [11])
-ttpB	FQ	Twisted tensor product code of type B using the extension field FQ . The code arises from the projective line over $\mathbb{F}_{q^3}$ (Theorem 2 in [11]).
-dual		Create the dual code instead.

Table 11.2: Creating Codes

Coding Theoretic Activities (Part 1)		
Command	Arguments	Purpose
-report		Produce a latex report.
-general_code_binary	n label code-words	Produces a plot of the codewords in the Hamming space. The code is given by a set of codewords in binary. The code has length n . If -metric_balls r is given, the metric balls of radius r centered around codewords are included in the plot. The command produces several csv files.
-encode_text_5bits	input fname	
-field_induction	fname-in fname-out nb-bits	
-weight_enumerator	matrix	Compute the complete weight enumerator of the code C .
-minimum_distance	code-object-label	Compute the minimum distance of the linear code object.
-generator_matrix_cyclic_code	n poly	
-Sylvester_Hadamard_code	n	Create the codewords of a Sylvester type Hadamard matrix of order n . Here, n must be a power of 2.
-fixed_code	perm	Create the subcode which is fix under the group generated by perm, which acts on the generator matrix by column permutations.
-export_magma	fname	
-export_codewords	fname	Export a list of codewords using coding as binary numbers.
-all_distances		Computes then distance for all pairs of codewords. The resulting matrix is written to file.
-all_external_distances		Computes the external distance for all words not in the code. The external distance is the distance to the nearest codeword. The resulting matrix is written to file.
-export_codewords_long	fname	Export a table of codewords.
-export_codewords_by_weight	fname	
-export_genma	fname	
-export_checkma	fname	
-export_checkma_as_projective_set	fname	

Table 11.3: Coding Theoretic Activities (Part 1)

Coding Theoretic Activities (Part 2)		
Command	Arguments	Purpose
-make_diagram		Creates a diagram of the code in Hamming space. Uses -embellish r to emphasize codewords by drawing a circle of radius r . If -metric_balls r is given, the metric balls around codewords of radius r are indicated.
-boolean_function_of_code		Computes the boolean function of the code C .
-embellish	r	See -make_diagram.
-metric_balls	r	See -make_diagram.
-Hamming_space_distance_matrix	n	Make the distance matrix of the Hamming graph $H(n, q)$.

Table 11.4: Coding Theoretic Activities (Part 2)

```

18  ▷ $(PDFLATEX) by_basis_n6_k3_code_n6_k3_q2.tex
19  ▷ $(OPEN) by_basis_n6_k3_code_n6_k3_q2.pdf
20  ▷ $(OPEN) by_basis_n6_k3_char_func_6_8_draw.bmp
21

```

The command also draws the code in the Hamming space.

Let us create the Hamming code. The dual of the Hamming code is the simplex code, so we create the simplex code first. The following makefile variable is defined to hold the generator matrix of the simplex code:

Code Snippet 515

```

1  SIMPLEX_CODE_GENERATOR="\
2  1,0,1,0,1,0,1, \
3  0,1,1,0,0,1,1, \
4  0,0,0,1,1,1,1"
5

```

The following command computes the nullspace of this matrix, which is the Hamming code:

Code Snippet 516

```

1  simplex_code:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -define v -vector -field F -format 3 \
5  ▷ ▷ ▷ -dense $(SIMPLEX_CODE_GENERATOR) \
6  ▷ ▷ -end \
7  ▷ ▷ -define C -code -field F \
8  ▷ ▷ ▷ -generator_matrix v \
9  ▷ ▷ -end \
10 ▷ ▷ -with C -do -coding_theoretic_activity \
11 ▷ ▷ ▷ -report \

```

Coding Theoretic Activities (Part 3)		
Command	Arguments	Purpose
-crc32	text	Tests CRC32.
-crc32_hexdata	hexdata	
-crc32_test	block_length	
-crc256_test	message_length R k	
-crc32_remainders	msg_length	
-crc_encode_file_based	fname_in fname_out block_length crc_type block_length	
-crc_compare	fname code1 code2 error_weight nb_tests_per_block	
-crc_compare_read_output_file	fname nb_lines code1 code2	
-all_errors_of_a_given_weight	fname block_idx code1 code2 max_weight	
-weight_enumerator_bottom_up	code max_weight	
-convert_data_to_polynomials	fname_in fname_out block_length symbol_size	
-find_CRC_polynomials	nb_errors info_bits check_bits	
-write_code_for_division	fname A B	
-polynomial_division_from_file	fname $r1$	
-polynomial_division_from_file_all_k_bit_error_patterns	fname $r1$ k	

Table 11.5: Coding Theoretic Activities (Part 3)

Code Modifications		
Command	Arguments	Purpose
-dual		Create the dual code.

Table 11.6: Code Modifications

```

12  ▷ ▷ -end
13  ▷ $(PDFLATEX) by_genma_n7_k3_code_n7_k3_q2.tex
14  ▷ $(OPEN) by_genma_n7_k3_code_n7_k3_q2.pdf
15

```

The following latex output is produced:

Input matrix:

$$\begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

RREF:

$$\begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Basis for Perp:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

It is possible to create the Hamming code by taking the dual of the simplex code. The following command does so:

Code Snippet 517

```

1  Hamming_code:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -define v -vector -field F -format 3 \
5  ▷ ▷ ▷ -dense $(SIMPLEX_CODE_GENERATOR) \
6  ▷ ▷ -end \
7  ▷ ▷ -define C -code -field F \
8  ▷ ▷ ▷ -generator_matrix v \
9  ▷ ▷ ▷ -dual -end \
10 ▷ ▷ -end \
11 ▷ ▷ -with C -do -coding_theoretic_activity \
12 ▷ ▷ ▷ -export_magma Hamming.magma \
13 ▷ ▷ -end
14

```

The command also exports the code to magma by means of the magma file Hamming.magma, shown below:

```

K<w> := GF(2);
V := VectorSpace(K, 7);
C := LinearCode(sub<V |

```



```
[1,1,1,0,0,0,0],
[1,0,0,1,1,0,0],
[0,1,0,1,0,1,0],
[1,1,0,1,0,0,1]>);
```

The next command creates the first order Reed-Muller code in 3 variables. All codewords are created. The codewords and the generator matrix are exported to files.

Code Snippet 518

```
1  RM_3_1_and_codewords:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -define C -code -field F \
5  ▷ ▷ ▷ -Reed_Muller 3 \
6  ▷ ▷ -end \
7  ▷ ▷ -with C -and F -do -coding_theoretic_activity \
8  ▷ ▷ ▷ -export_magma RM_3_1.magma \
9  ▷ ▷ -end \
10 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
11 ▷ ▷ ▷ -export_codewords RM_3_1.codewords.csv \
12 ▷ ▷ -end \
13 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
14 ▷ ▷ ▷ -export_genma RM_3_1.genma.csv \
15 ▷ ▷ -end
16
```

We can create the code from scratch, using a generator matrix. To do so, we use a makefile variable:

Code Snippet 519

```
1  CODE_RM_3_1_GENMA="\
2  11111111\
3  01010101\
4  00110011\
5  00001111"
6
```

The following command creates the Reed-Muller code from its generator matrix directly:

Code Snippet 520

```
1  RM_3_1_from_generator_matrix:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -define genma -vector -format 8 -field F \
5  ▷ ▷ ▷ -compact $(CODE_RM_3_1_GENMA) \
6  ▷ ▷ -end \
7  ▷ ▷ -define C -code -field F \
8  ▷ ▷ ▷ -generator_matrix genma \
9  ▷ ▷ -end
10 ▷ #$(PDFLATEX) code_n8_k4.q2.tex
```

```

11  ▷  #$(OPEN) code_n8_k4_q2.pdf
12

```

Let us look at the Hamming code. The following command creates the $[7, 4]$ Hamming code.

Code Snippet 521

```

1  Hamming_code_by_rows:
2  ▷  $(ORBITER) -v 2 \
3  ▷  ▷  -define F -finite_field -q 2 -end \
4  ▷  ▷  -define v -vector -dense $(HAMMING_CODE_ROWS_IN_BINARY_RANKS) -end \
5  ▷  ▷  -define C -code -field F \
6  ▷  ▷  ▷  -basis 7 v \
7  ▷  ▷  -end
8  ▷  #$(PDFLATEX) code_n7_k4_q2.tex
9  ▷  #$(OPEN) code_n7_k4_q2.pdf
10

```

A different way to create the same code is using the `long_code` command:

Code Snippet 522

```

1  Hamming_code_long:
2  ▷  $(ORBITER) -v 2 \
3  ▷  ▷  -define F -finite_field -q 2 -end \
4  ▷  ▷  -define C -code -field F \
5  ▷  ▷  ▷  -long_code 7 4 \
6  ▷  ▷  ▷  "0,5,6" \
7  ▷  ▷  ▷  "1,4,6" \
8  ▷  ▷  ▷  "2,4,5" \
9  ▷  ▷  ▷  "3,4,5,6" \
10 ▷  ▷  -end
11

```

The following command creates a diagram for the Hamming code inside the Hamming space.

Code Snippet 523

```

1  Hamming_code_diagram:
2  ▷  $(ORBITER) -v 2 \
3  ▷  ▷  -define F -finite_field -q 2 -end \
4  ▷  ▷  -define v -vector -dense $(HAMMING_CODE_ROWS_IN_BINARY_RANKS) -end \
5  ▷  ▷  -define C -code -field F \
6  ▷  ▷  ▷  -basis 7 v \
7  ▷  ▷  -end \
8  ▷  ▷  -with C -and F -do -coding_theoretic_activity \
9  ▷  ▷  ▷  -make_diagram \
10 ▷  ▷  -end
11 ▷  $(ORBITER) -v 2 \
12 ▷  ▷  -draw_matrix \
13 ▷  ▷  ▷  -input_csv_file by_basis_n7_k4_char_func_7_16.csv \

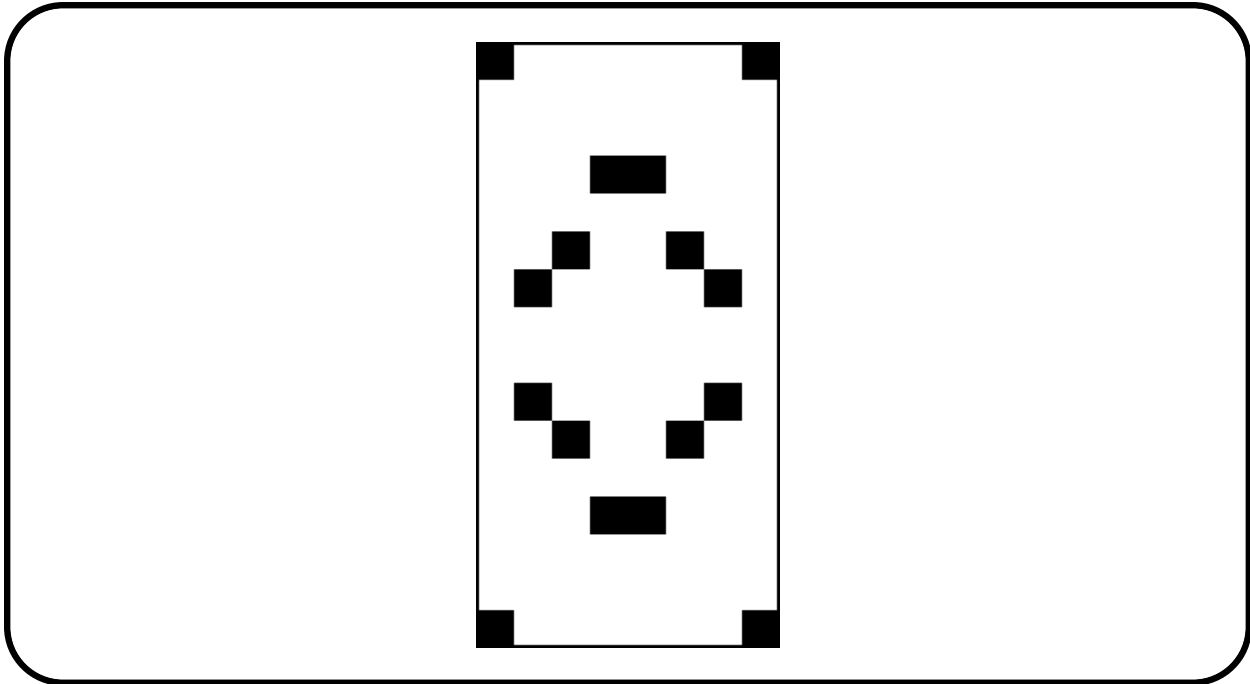
```

```

14  ▷ ▷ ▷ -box_width 25 -bit_depth 24 \
15  ▷ ▷ ▷ -partition 4 16 8 \
16  ▷ ▷ -end
17  ▷ $(OPEN) by_basis_n7_k4_char_func_7_16_draw.bmp
18

```

Here is the output:



The next command creates the metric balls of radius one centered at each of the 16 codewords of the Hamming code. At first, we define the numerical values of the codewords of the Hamming code:

Code Snippet 524

```

1  HAMMING_CODE_CODEWORDS="0, 67, 37, 102, 22, 85, \
2  51, 112, 15, 76, 42, 105, 25, 90, 60, 127"
3

```

Then we create the diagrams of all codewords. We utilize a loop-over command for the codewords and a loop command for the graphical output.

Code Snippet 525

```

1  Hamming_code_words:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define v -vector -dense $(HAMMING_CODE_CODEWORDS) -end \
4  ▷ ▷ -define F -finite_field -q 2 -end \
5  ▷ ▷ -loop_over i v \
6  ▷ ▷ ▷ -with F -do -coding_theoretic_activity \
7  ▷ ▷ ▷ -general_code_binary 7 "Hamming_7.4_word%i" "%i[v]" \
8  ▷ ▷ ▷ -metric_balls 1 \

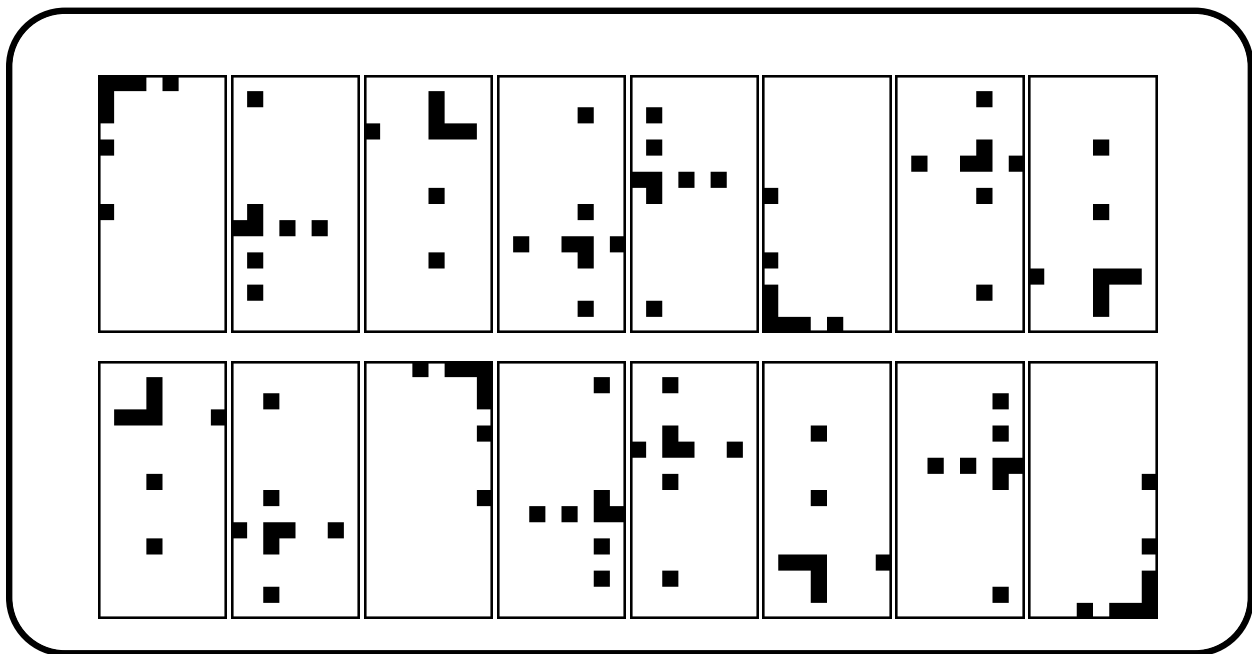
```

```

9   ▷ ▷ ▷ -end \
10  ▷ ▷ -end_loop_over i
11  ▷ $(ORBITER) -v 2 \
12  ▷ ▷ -loop i 0 16 1 -draw_matrix \
13  ▷ ▷ ▷ -input_csv_file Hamming_7_4_word_%i_char_func_7_1.csv \
14  ▷ ▷ ▷ -box_width 25 -bit_depth 8 \
15  ▷ ▷ ▷ -partition 4 16 8 -end \
16  ▷ ▷ ▷ -system "magick Hamming_7_4_word_%i_char_func_7_1_draw.bmp \
17  ▷ ▷ ▷ -frame 8 Hamming_7_4_word_%i_char_func_7_1_draw.png" \
18  ▷ ▷ -end_loop i
19

```

These metric balls form a partition of the Hamming space. The partition can be shown in binary Hamming space:



Orbiter can compute the weight enumerator and the minimum distance of codes. Let us consider the Hamming code, for example. We use a makefile variable for the generator matrix:

Code Snippet 526

```

1   HAMMING_CODE_GENERATOR="\
2   1,0,0,0,0,1,1, \
3   0,1,0,0,1,0,1, \
4   0,0,1,0,1,1,0, \
5   0,0,0,1,1,1,1"
6

```

The next command computes the weight enumerator:

Code Snippet 527

```

1      Hamming_weight_enumerator:
2      ▷ $(ORBITER) -v 2 \
3      ▷ ▷ -define F -finite_field -q 2 -end \
4      ▷ ▷ -define v -vector -field F -format 4 \
5      ▷ ▷ ▷ -dense $(HAMMING_CODE_GENERATOR) \
6      ▷ ▷ -end \
7      ▷ ▷ -define C -code -field F \
8      ▷ ▷ ▷ -generator_matrix v \
9      ▷ ▷ -end \
10     ▷ ▷ -with C -do \
11     ▷ ▷ -coding_theoretic_activity \
12     ▷ ▷ ▷ -weight_enumerator \
13     ▷ ▷ -end
14

```

We find that the weight enumerator is

$$(1, 0, 0, 7, 7, 0, 0, 1).$$

The next command computes the minimum distance of the code:

Code Snippet 528

```

1      Hamming_minimum_distance:
2      ▷ $(ORBITER) -v 2 \
3      ▷ ▷ -define F -finite_field -q 2 -end \
4      ▷ ▷ -define v -vector -field F -format 4 \
5      ▷ ▷ ▷ -dense $(HAMMING_CODE_GENERATOR) \
6      ▷ ▷ -end \
7      ▷ ▷ -with F -do \
8      ▷ ▷ -coding_theoretic_activity \
9      ▷ ▷ ▷ -minimum_distance v \
10     ▷ ▷ -end
11

```

The following command computes the minimum distance of the Golay code of length 23:

Code Snippet 529

```

1      Golay23_minimum_distance:
2      ▷ $(ORBITER) -v 2 \
3      ▷ ▷ -define F -finite_field -q 2 -end \
4      ▷ ▷ -define v -vector -field F -format 12 \
5      ▷ ▷ ▷ -dense $(GOLAY23_CODE_GENERATOR) \
6      ▷ ▷ -end \
7      ▷ ▷ -with F -do \
8      ▷ ▷ -coding_theoretic_activity \
9      ▷ ▷ ▷ -minimum_distance v \
10     ▷ ▷ -end
11

```

11.3 Golay Codes

The Golay code of length 23 is a perfect code of dimension 12 and minimum distance 7. The metric balls of radius three centered around codewords cover the whole Hamming space. We can create the code by listing the columns of a generator matrix in Orbiter ranks of points in $\text{PG}(11, 2)$. The following makefile variable does that:

Code Snippet 530

```
1 GOLAY_23_COLUMN_RANKS_PROJECTIVELY="0, 1, 2, 3, 4, 5, 6, 7, \
2 8, 9, 10, 11, 132, 913, 1460, 1750, 1898, 2518, 2787, 2874, \
3 3320, 3357, 3662"
4
```

Suppose we want to list the code words. The following command can be used:

Code Snippet 531

```
1 Golay23_code_words:
2 ▷ $(ORBITER) -v 2 \
3 ▷ ▷ -define v -vector -dense $(GOLAY_23_COLUMN_RANKS_PROJECTIVELY) -end \
4 ▷ ▷ -define F -finite_field -q 2 -end \
5 ▷ ▷ -define C -code -field F \
6 ▷ ▷ ▷ -projective_set 12 v -end \
7 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
8 ▷ ▷ ▷ -export_magma Golay23.magma \
9 ▷ ▷ -end \
10 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
11 ▷ ▷ ▷ -export_codewords Golay23.codewords.csv \
12 ▷ ▷ -end \
13 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
14 ▷ ▷ ▷ -export_genma Golay23.genma.csv \
15 ▷ ▷ -end
16 ▷ #$(PDFLATEX) code_n23_k12_q2.tex
17 ▷ #$(OPEN) code_n23_k12_q2.pdf
18
```

Defining a CRC-process		
Command	Arguments	Purpose
-code	label	Set the a label.
-crc_options	options	Specify the CRC-code using commands from Table 11.8.

Table 11.7: Defining a CRC-process

Options for CRC-Codes		
Command	Arguments	Purpose
-input	fname	Input file name.
-output	fname	Output file name.
-crc_type	type	The type of CRC-code.
-block_length	L	Set the block length to L field elements.
-block_based_error_generator		Apply block-based error generator.
-file_based_error_generator	threshold	Apply file-based error generator.
-nb_repeats	N	Set the number of repeats to N .
-threshold	t	Set probability of error per experiment to $t/1000000$.
-error_log	fname	Set file name for error logging.
-selected_block	i	Set block number.

Table 11.8: Options for CRC-Codes

11.4 CRC Codes

A CRC code can be used to detect communication errors. It is a cyclic code, and hence generated by a polynomial over a finite field. The message is encoded as a string, which is then thought of as a polynomial, called the information polynomial. Assume that the check polynomial has degree d . The information polynomial is then divided by the check polynomial. The remainder is added to the information polynomial multiplied by X^d . This is the codeword, which is sent.

Table 11.7 lists the commands for defining a CRC-process.

Table 11.8 summarizes options associated with commands for CRC-codes.

Table 11.9 lists the available commands for defining a CRC-code.

The following command performs an exhaustive search over all binary CRC polynomials of degree $k = 10$ which can detect every error pattern of Hamming weight at most $t = 3$ in messages of length $n = 128$.

Defining a CRC-Code		
Command	Arguments	Purpose
-type	type	Select the type of CRC-code.
-block_length	length	Select the symbol length of the CRC-code.

Table 11.9: Defining a CRC-Code

Code Snippet 532

```

1   CRC_3_128_10:
2   ▷ $(ORBITER) -v 1 \
3   ▷ ▷ -define F -finite_field -q 2 -end \
4   ▷ ▷ -with F -do -coding_theoretic_activity \
5   ▷ ▷ ▷ -find_CRC_polynomials 3 128 10 \
6   ▷ ▷ -end
7

```

The program finds 244 polynomials in about 1 minute.

Here is a collection of CRC polynomials from various sources:

Code Snippet 533

```

1   CRC4="1,4,1,2,1,1,1,0"
2

```

Code Snippet 534

```

1   CRC7="1,7,1,3,1,0"
2

```

Code Snippet 535

```

1   CRC8_ATM="1,8,1,2,1,1,1,0"
2

```

Code Snippet 536

```

1   CRC16_CCITT="1,16,1,12,1,5,1,0"
2

```

Code Snippet 537

```

1   CRC32_ETHERNET="1,32,1,26,1,23,1,22,1,16,1,12,1,11,1,10,1,8,1,7,\
2   1,5,1,4,1,2,1,1,1,0"
3

```

Code Snippet 538

```

1   CRC32_CASTAGNOLI="1,32,1,28,1,27,1,26,1,25,1,23,1,22,1,20,1,19,1,\
2   18,1,14,1,13,1,11,1,10,1,9,1,8,1,6,1,0"
3

```

Code Snippet 539

```

1   CRC64_ECMA182="1,64,1,62,1,57,1,55,1,54,1,53,1,52,1,47,1,46,1,45,\
2   1,40,1,39,1,38,1,37,1,35,1,33,1,32,1,31,1,29,1,27,1,24,1,23,1,22,\

```



```

3      1,21,1,19,1,17,1,13,1,12,1,10,1,9,1,7,1,4,1,1,1,0"
4

```

Code Snippet 540

```

1      CRC64_ROCKSOFT="1,64,1,63,1,61,1,59,1,58,1,56,1,55,1,52,1,49,1,48,\
2      1,47,1,46,1,44,1,41,1,37,1,36,1,34,1,32,1,31,1,28,1,26,1,23,1,22,1,\
3      19,1,16,1,13,1,12,1,10,1,9,1,6,1,4,1,3,1,0"
4

```

We test whether the polynomial `crc32` is irreducible:

Code Snippet 541

```

1      crc32_Berlekamp_matrix:
2      ▷ $(ORBITER) -v 2 \
3      ▷ ▷ -define F -finite_field -q 2 -end \
4      ▷ ▷ -define v -vector -field F -sparse 33 $(CRC32_ETHERNET) -end \
5      ▷ ▷ -with F -do \
6      ▷ ▷ -finite_field_activity \
7      ▷ ▷ ▷ -Berlekamp_matrix v \
8      ▷ ▷ -end
9

```

Now, we create some new CRC polynomials over the field $\mathbb{F}_{256}$. To begin with, we create the 771st roots over $\mathbb{F}_{256}$:

Code Snippet 542

```

1      CRC_F256_roots_771:
2      ▷ $(ORBITER) -v 3 \
3      ▷ ▷ -define F -finite_field -q 256 -end \
4      ▷ ▷ -with F -do -finite_field_activity \
5      ▷ ▷ ▷ -nth_roots 771 \
6      ▷ ▷ -end
7

```

We create a BCH code of length 771 over $\mathbb{F}_{256}$ with designed distance 2:

Code Snippet 543

```

1      CRC_F256_BCH_code_d2:
2      ▷ $(ORBITER) -v 2 \
3      ▷ ▷ -define F -finite_field -q 256 -end \
4      ▷ ▷ -define C -code -field F \
5      ▷ ▷ ▷ -BCH 771 2 \
6      ▷ ▷ -end \
7      ▷ ▷ -with C -and F -do -coding_theoretic_activity \
8      ▷ ▷ ▷ -export_magma BCH_q8_n771_d2.magma \
9      ▷ ▷ -end

```

```

10  ▷  $(PDFLATEX) BCH_codes_q256_n771_d2.tex
11  ▷  $(OPEN) BCH_codes_q256_n771_d2.pdf
12

```

The polynomial in dense coding

Code Snippet 544

```

1  CRC_POLY_Q256_DEG2_DENSE="214,167,1"
2

```

We generate C++ source code for the use of this polynomial:

Code Snippet 545

```

1  CRC_F256_BCH_write_code_for_division_d2:
2  ▷  $(ORBITER) -v 2 \
3  ▷  ▷  -define F -finite_field -q 256 -end \
4  ▷  ▷  -define A -vector -field F -sparse 772 "1,771,1,0" -end \
5  ▷  ▷  -define B -vector -field F -dense $(CRC_POLY_Q256_DEG2_DENSE) -end \
6  ▷  ▷  -with F -do \
7  ▷  ▷  -coding_theoretic_activity \
8  ▷  ▷  ▷  -write_code_for_division \
9  ▷  ▷  ▷  alfa A B \
10 ▷  ▷  -end
11 ▷  g++ crc_alfa.cpp -o crc_alfa.out
12 ▷  ./crc_alfa.out
13

```

We create a BCH code of length 771 over $\mathbb{F}_{256}$ with designed distance 16:

Code Snippet 546

```

1  F256_BCH_code_d16:
2  ▷  $(ORBITER) -v 3 \
3  ▷  ▷  -define F -finite_field -q 256 -end \
4  ▷  ▷  -define C -code -field F \
5  ▷  ▷  ▷  -BCH 771 16 \
6  ▷  ▷  -end \
7  ▷  ▷  -with C -and F -do -coding_theoretic_activity \
8  ▷  ▷  ▷  -report \
9  ▷  ▷  -end
10 ▷  $(PDFLATEX) BCH_n771_d16_code_n771_k741_q256.tex
11 ▷  $(OPEN) BCH_n771_d16_code_n771_k741_q256.pdf
12

```

The polynomial in sparse coding is:

Code Snippet 547

```

1  POLY_Q256_DEG30_SPARSE="1,0,26,1,210,2,24,3,\
2  138,4,148,5,160,6,58,7,108,8,199,9,95,10,56,\

```

```

3    11,9,12,205,13,194,14,193,15,3,16,248,17,110,\
4    18,150,19,24,20,169,21,192,22,212,23,112,24,\
5    144,25,97,26,109,27,174,28,253,29,1,30"
6

```

The polynomial in dense coding is:

Code Snippet 548

```

1    POLY_Q256_DEG30_DENSE="1,26,210,24,138,148,\
2    160,58,108,199,95,56,9,205,194,193,3,248,110,\
3    150,24,169,192,212,112,144,97,109,174,253,1"
4

```

We generate C++ source code for the use of this polynomial:

Code Snippet 549

```

1    F256_BCH_write_code_for_division_d16:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 256 -end \
4    ▷ ▷ -define A -vector -field F -sparse 772 "1,771,1,0" -end \
5    ▷ ▷ -define B -vector -field F -dense $(POLY_Q256_DEG30_DENSE) -end \
6    ▷ ▷ -with F -do \
7    ▷ ▷ -coding_theoretic_activity \
8    ▷ ▷ ▷ -write_code_for_division \
9    ▷ ▷ ▷ check_q256_n771_r30 A B \
10   ▷ ▷ -end
11   ▷ g++ crc_check_q256_n771_r30.cpp -o crc_check_q256_n771_r30.out
12   ▷ ./crc_check_q256_n771_r30.out
13

```

We confirm that the polynomial divides $X^{771} - 1$ as it should:

Code Snippet 550

```

1    F256_BCH_code_d16_division:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 256 -end \
4    ▷ ▷ -define A -vector -field F -sparse 772 "1,771,1,0" -end \
5    ▷ ▷ -define B -vector -field F -dense $(POLY_Q256_DEG30_DENSE) -end \
6    ▷ ▷ -with F -do \
7    ▷ ▷ -finite_field_activity \
8    ▷ ▷ -polynomial_division A B -end
9

```

The next example introduces three errors. The remainder is not zero, so the errors are detected:

Code Snippet 551

```

1    F256_BCH_code_d16_error:
2    ▷ $(ORBITER) -v 2 \

```

```

3   ▷ ▷ -define F -finite_field -q 256 -end \
4   ▷ ▷ -define A -vector -field F -sparse 771 "2,30,3,31,55,770" -end \
5   ▷ ▷ -define B -vector -field F -dense $(POLY_Q256_DEG30_DENSE) -end \
6   ▷ ▷ -with F -do \
7   ▷ ▷ -finite_field_activity \
8   ▷ ▷ -polynomial_division A B -end
9

```

We create the field $\mathbb{F}_{256}$:

Code Snippet 552

```

1   F_256:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 256 -end \
4   ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
5   ▷ $(PDFLATEX) GF_256.tex
6   ▷ $(OPEN) GF_256.pdf
7

```

Code Snippet 553

```

1   CRC_F_16:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 16 -override_polynomial 19 -end \
4   ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
5   ▷ $(PDFLATEX) GF_16_poly19.tex
6   ▷ $(OPEN) GF_16_poly19.pdf
7

```

Code Snippet 554

```

1   nth_roots_256:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 256 -end \
4   ▷ ▷ -with F -do -finite_field_activity \
5   ▷ ▷ ▷ -nth_roots 51 \
6   ▷ ▷ -end
7   ▷ $(PDFLATEX) Nth_roots_q256_n51.tex
8   ▷ $(OPEN) Nth_roots_q256_n51.pdf
9

```

Code Snippet 555

```

1   CRC_F16_roots_51:
2   ▷ $(ORBITER) -v 3 \

```

```

3   ▷ ▷ -define F -finite_field -q 16 -override_polynomial 19 -end \
4   ▷ ▷ -with F -do -finite_field_activity \
5   ▷ ▷ ▷ -nth_roots 51 \
6   ▷ ▷ -end
7   ▷ $(PDFLATEX) Nth_roots_q16_n51.tex
8   ▷ $(OPEN) Nth_roots_q16_n51.pdf
9

```

Code Snippet 556

```

1   CRC_F16_BCH_code_d3:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 16 -override_polynomial 19 -end \
4   ▷ ▷ -define C -code -field F \
5   ▷ ▷ ▷ -BCH 51 3 \
6   ▷ ▷ -end \
7   ▷ ▷ -with C -and F -do -coding_theoretic_activity \
8   ▷ ▷ ▷ -export_magma BCH_q16_n51_d3.magma \
9   ▷ ▷ -end \
10  ▷ ▷ -with C -and F -do -coding_theoretic_activity \
11  ▷ ▷ ▷ -export_checkma BCH_q16_n51_d3_check.csv \
12  ▷ ▷ -end \
13  ▷ ▷ -with C -and F -do -coding_theoretic_activity \
14  ▷ ▷ ▷ -export_checkma_as_projective_set BCH_q16_n51_d3_check_ranks.csv \
15  ▷ ▷ -end \
16  ▷ ▷ -with C -and F -do -coding_theoretic_activity \
17  ▷ ▷ ▷ -report \
18  ▷ ▷ -end
19  ▷ $(PDFLATEX) BCH_n51_d3_code_n51_k47_q16.tex
20  ▷ $(OPEN) BCH_n51_d3_code_n51_k47_q16.pdf
21

```

Code Snippet 557

```

1   CRC_F16_BCH_code_d3_mindist_dual:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 16 -override_polynomial 19 -end \
4   ▷ ▷ -define v -vector -file BCH_q16_n51_d3_check.csv \
5   ▷ ▷ ▷ -field F -format 4 -file BCH_q16_n51_d3_check.csv -end \
6   ▷ ▷ -with F -do \
7   ▷ ▷ -coding_theoretic_activity \
8   ▷ ▷ ▷ -minimum_distance v \
9   ▷ ▷ -end
10

```

Code Snippet 558

```

1   CRC_F16_BCH_code_d3_dual_weight_enumerator:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 16 -override_polynomial 19 -end \
4   ▷ ▷ -define v -vector -file BCH_q16_n51_d3_check.csv \
5   ▷ ▷ ▷ -field F -format 4 -file BCH_q16_n51_d3_check.csv -end \
6   ▷ ▷ -define C -code -field F \
7   ▷ ▷ ▷ -generator_matrix v \
8   ▷ ▷ ▷ -end \
9   ▷ ▷ -with C -do \
10  ▷ ▷ -coding_theoretic_activity \
11  ▷ ▷ ▷ -weight_enumerator \
12  ▷ ▷ -end
13

```

Code Snippet 559

```

1   CRC_F16_BCH_code_d3_projective_set:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 16 -override_polynomial 19 -end \
4   ▷ ▷ -define v -vector -file BCH_q16_n51_d3_check_ranks.csv \
5   ▷ ▷ ▷ -file BCH_q16_n51_d3_check_ranks.csv -end \
6

```

Code Snippet 560

```

1   CRC_F16_BCH_code_d3_macwilliams:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -make_macwilliams_system 51 4 16
4

```

Code Snippet 561

```

1   CRC_F16_BCH_write_code_for_division_Delta:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 16 -override_polynomial 19 -end \
4   ▷ ▷ -define A -vector -field F -sparse 52 "1,51,1,0" -end \
5   ▷ ▷ -define B -vector -field F -dense $(CRC_POLY_DELTA_DENSE) -end \
6   ▷ ▷ -with F -do \
7   ▷ ▷ -coding_theoretic_activity \
8   ▷ ▷ ▷ -write_code_for_division \
9   ▷ ▷ ▷ delta A B \
10  ▷ ▷ -end
11  ▷ g++ crc_delta.cpp -o crc_delta.out
12  ▷ ./crc_delta.out
13

```

Code Snippet 562

```
1  crc_encode_16:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -define CRC16 -crc_code -type crc16 -block_length 771 -end \
5  ▷ ▷ -with F -do \
6  ▷ ▷ ▷ -coding_theoretic_activity \
7  ▷ ▷ ▷ ▷ -crc_encode_file_based \
8  ▷ ▷ ▷ ▷ $(CRC_FILE).$(CRC_FILE_EXTENSION) \
9  ▷ ▷ ▷ ▷ $(CRC_FILE)_crc16.bin CRC16 \
10 ▷ ▷ -end
11
```

Code Snippet 563

```
1  crc_encode_32:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -define CRC32 -crc_code -type crc32 -block_length 771 -end \
5  ▷ ▷ -with F -do \
6  ▷ ▷ ▷ -coding_theoretic_activity \
7  ▷ ▷ ▷ ▷ -crc_encode_file_based \
8  ▷ ▷ ▷ ▷ $(CRC_FILE).$(CRC_FILE_EXTENSION) \
9  ▷ ▷ ▷ ▷ $(CRC_FILE)_crc32.bin CRC32 \
10 ▷ ▷ -end
11
```

11.5 Reed-Muller Codes

The following command creates the Reed Muller code RM_3 . The command exports the code to magma and writes a file of all codewords.

Code Snippet 564

```
1  RM_3_export:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -define C -code -field F \
5  ▷ ▷ ▷ -Reed_Muller 3 \
6  ▷ ▷ -end \
7  ▷ ▷ -with C -and F -do -coding_theoretic_activity \
8  ▷ ▷ ▷ -export_magma RM_3.magma \
9  ▷ ▷ -end \
10 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
11 ▷ ▷ ▷ -export_codewords_long RM_3.codewords.csv \
12 ▷ ▷ -end
13
```

Here is the csv file containing the codewords.

RM_3_codewords

Row	Idx	Message	Codeword	CodewordRank
0	0	0,0,0,0	0,0,0,0,0,0,0,0	0
1	1	1,0,0,0	1,1,1,1,1,1,1,1	255
2	2	0,1,0,0	0,1,0,1,0,1,0,1	170
3	3	1,1,0,0	1,0,1,0,1,0,1,0	85
4	4	0,0,1,0	0,0,1,1,0,0,1,1	204
5	5	1,0,1,0	1,1,0,0,1,1,0,0	51
6	6	0,1,1,0	0,1,1,0,0,1,1,0	102
7	7	1,1,1,0	1,0,0,1,1,0,0,1	153
8	8	0,0,0,1	0,0,0,0,1,1,1,1	240
9	9	1,0,0,1	1,1,1,1,0,0,0,0	15
10	10	0,1,0,1	0,1,0,1,1,0,1,0	90
11	11	1,1,0,1	1,0,1,0,0,1,0,1	165
12	12	0,0,1,1	0,0,1,1,1,1,0,0	60
13	13	1,0,1,1	1,1,0,0,0,0,1,1	195
14	14	0,1,1,1	0,1,1,0,1,0,0,1	150
15	15	1,1,1,1	1,0,0,1,0,1,1,0	105
END				

The four columns represent:

1. Idx – The numerical value of the message.
2. Message – The message.
3. Codeword – The associated codeword.
4. CodewordRank – The numerical value of the codeword.

The next command creates a diagram of the codewords in the appropriate Hamming space, which is $H(8, 2)$ in this case.

Code Snippet 565

```

1   RM_3_Hamming_space_diagram:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 2 -end \
4   ▷ ▷ -define C -code -field F \
5   ▷ ▷ ▷ -Reed_Muller 3 \
6   ▷ ▷ -end \
7   ▷ ▷ -with C -do \
8   ▷ ▷ -coding_theoretic_activity \
9   ▷ ▷ ▷ -make_diagram \
10  ▷ ▷ ▷ -metric_balls 1 \
11  ▷ ▷ -end
12

```

The following command produces a diagram of the characteristic function of the code in the Hamming space $H(8, 2)$.

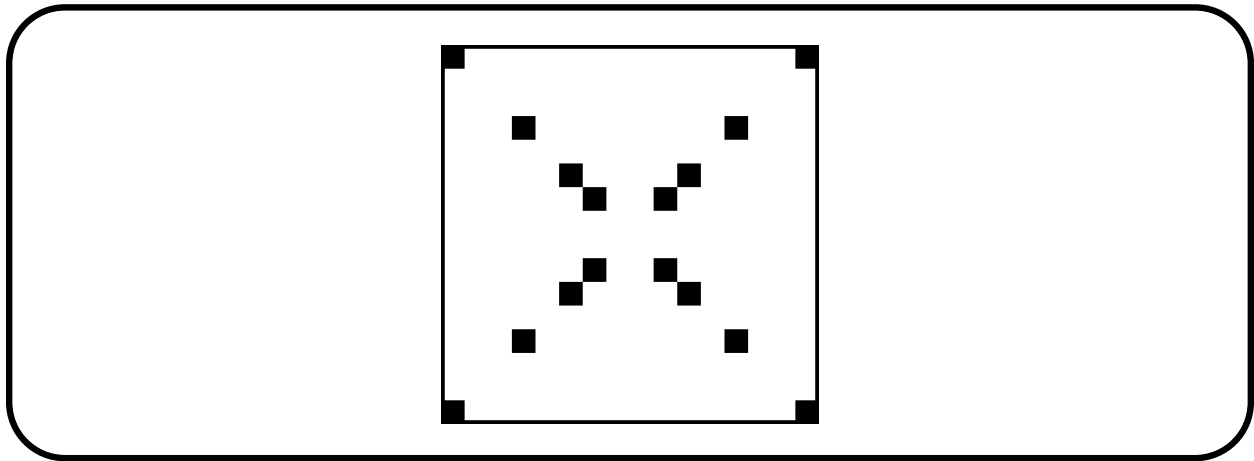
Code Snippet 566

```

1   RM_3_draw:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -draw_matrix \
4   ▷ ▷ ▷ -input_csv_file RM_m3_distance_H_8_16.csv \
5   ▷ ▷ ▷ -box_width 25 -bit_depth 8 \
6   ▷ ▷ ▷ -partition 4 16 16 \
7   ▷ ▷ -end
8   ▷ $(ORBITER) -v 2 \
9   ▷ ▷ -draw_matrix \
10  ▷ ▷ ▷ -input_csv_file RM_m3_char_func_8_16.csv \
11  ▷ ▷ ▷ -box_width 25 -bit_depth 8 \
12  ▷ ▷ ▷ -partition 4 16 16 \
13  ▷ ▷ -end
14  ▷ $(ORBITER) -v 2 \
15  ▷ ▷ -draw_matrix \
16  ▷ ▷ ▷ -input_csv_file RM_m3_idx_8_16.csv \
17  ▷ ▷ ▷ -box_width 25 -bit_depth 8 \
18  ▷ ▷ ▷ -partition 4 16 16 \
19  ▷ ▷ -end
20  ▷ magick RM_m3_distance_H_8_16_draw.bmp RM_m3_distance_H_8_16_draw.png
21  ▷ magick RM_m3_char_func_8_16_draw.bmp RM_m3_char_func_8_16_draw.png
22  ▷ magick RM_m3_idx_8_16_draw.bmp RM_m3_idx_8_16_draw.png
23  ▷ $(OPEN) RM_m3_distance_H_8_16_draw.bmp
24

```

The following boolean function representation of RM_3 in $H(8, 2)$ is produced:



The next command splits the vectors according to their distance from the code:

Code Snippet 567

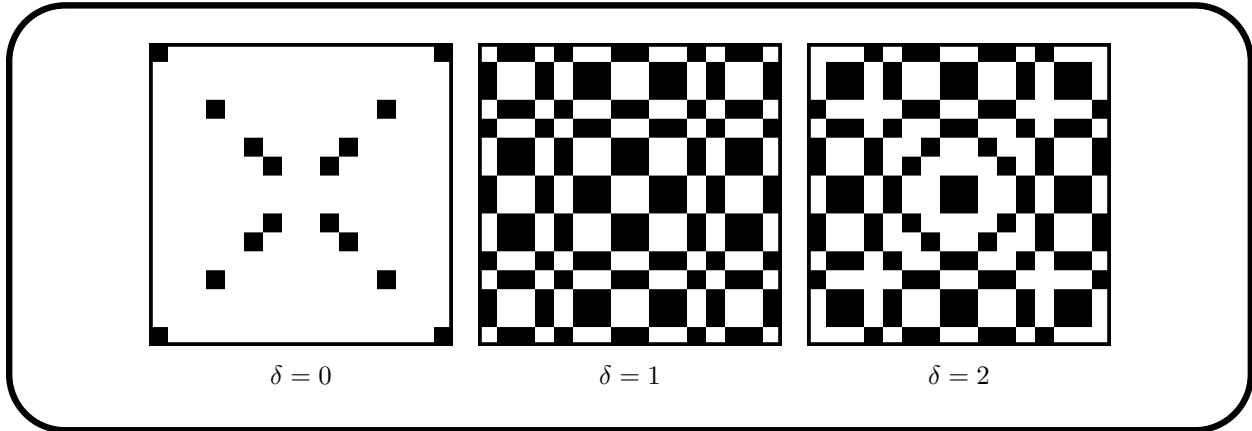
```
1  RM_3_split:
2  ▷ $(ORBITER) -split_by_values RM_m3_distance_H_8_16.csv
3
```

We produce a diagram of each distance set. We loop over all distances $\delta = 0, 1, 2$:

Code Snippet 568

```
1  RM_3_distance_draw:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -loop L 0 3 1 \
4  ▷ ▷ ▷ -draw_matrix \
5  ▷ ▷ ▷ ▷ -input_csv_file RM_m3_distance_H_8_16_value%L.csv \
6  ▷ ▷ ▷ ▷ -box_width 25 -bit_depth 8 \
7  ▷ ▷ ▷ ▷ -partition 5 16 16 \
8  ▷ ▷ ▷ -end \
9  ▷ ▷ -end_loop L
10 ▷ magick RM_m3_distance_H_8_16_value0_draw.bmp RM_m3_distance_H_8_16_value0_draw.
    png
11 ▷ magick RM_m3_distance_H_8_16_value1_draw.bmp RM_m3_distance_H_8_16_value1_draw.
    png
12 ▷ magick RM_m3_distance_H_8_16_value2_draw.bmp RM_m3_distance_H_8_16_value2_draw.
    png
13 ▷ $(OPEN) RM_m3_distance_H_8_16_value0_draw.png
14 ▷ $(OPEN) RM_m3_distance_H_8_16_value1_draw.png
15 ▷ $(OPEN) RM_m3_distance_H_8_16_value2_draw.png
16
```

The distance sets are shown below



For each distance, we compute the algebraic normal form of the characteristic function of the distance set:

Code Snippet 569

```

1  RM_3_algebraic_normal_form:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -loop L 0 3 1 \
5  ▷ ▷ ▷ -define v%L -vector -field F \
6  ▷ ▷ ▷ ▷ -file RM_m3_distance_H_8_16_value%L.csv \
7  ▷ ▷ ▷ -end \
8  ▷ ▷ ▷ -with F -do -finite_field_activity \
9  ▷ ▷ ▷ ▷ -algebraic_normal_form 8 v%L \
10 ▷ ▷ ▷ -end \
11 ▷ ▷ -end_loop L
12
```

In the example, we will compute a graphical representation of the codewords of the Reed-Muller code of order 6. First, we use makefile variables to define the rows of the generator matrix:

Code Snippet 570

```

1  RM_6_GENERATOR_1="0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,\
2  22,23,24,25,26,27,28,29,30,31,32,33,34,35,36,37,38,39,40,41,42,43,44,45,\
3  46,47,48,49,50,51,52,53,54,55,56,57,58,59,60,61,62,63"
4
```

Code Snippet 571

```

1  RM_6_GENERATOR_2="1,3,5,7,9,11,13,15,17,19,21,23,25,27,29,31,33,35,37,39,\
2  41,43,45,47,49,51,53,55,57,59,61,63"
3
```

Code Snippet 572

```

1  RM_6_GENERATOR_3="2,3,6,7,18,19,22,23,10,11,14,15,26,27,30,31,34,35,38,\
2  39,42,43,46,47,50,51,54,55,58,59,62,63"

```

3

Code Snippet 573

```

1  RM_6_GENERATOR_4="4,6,12,14,36,38,52,54,5,7,13,15,37,39,53,55,20,22,28,\
2  30,44,46,60,62,21,23,29,31,45,47,61,63"
3

```

Code Snippet 574

```

1  RM_6_GENERATOR_5="8,9,12,13,24,25,28,29,10,11,14,15,26,27,30,31,40,41,\
2  44,45,56,57,60,61,42,43,46,47,58,59,62,63"
3

```

Code Snippet 575

```

1  RM_6_GENERATOR_6="16,18,24,26,48,50,56,58,17,19,25,27,49,51,57,59,20,22,\
2  28,30,52,54,60,62,21,23,29,31,53,55,61,63"
3

```

Code Snippet 576

```

1  RM_6_GENERATOR_7="32,34,48,50,33,35,49,51,36,38,52,54,37,39,53,55,40,42,\
2  56,58,41,43,57,59,44,46,60,62,45,47,61,63"
3

```

The following command creates the generator matrix, and exports the generator matrix and the set of codewords to two csv files.

Code Snippet 577

```

1  RM_6:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -define C -code -field F \
5  ▷ ▷ ▷ -long_code 64 7 \
6  ▷ ▷ ▷ $(RM_6_GENERATOR_1) \
7  ▷ ▷ ▷ $(RM_6_GENERATOR_2) \
8  ▷ ▷ ▷ $(RM_6_GENERATOR_3) \
9  ▷ ▷ ▷ $(RM_6_GENERATOR_4) \
10 ▷ ▷ ▷ $(RM_6_GENERATOR_5) \
11 ▷ ▷ ▷ $(RM_6_GENERATOR_6) \
12 ▷ ▷ ▷ $(RM_6_GENERATOR_7) \
13 ▷ ▷ -end \
14 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
15 ▷ ▷ ▷ -export_genma RM6_genma.csv \
16 ▷ ▷ -end \
17 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
18 ▷ ▷ ▷ -export_codewords_long RM6_codewords.csv \
19 ▷ ▷ -end \

```

20

The next command produces a drawing of the generator matrix:

Code Snippet 578

```

1  RM_6_genma_draw:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -draw_matrix \
4  ▷ ▷ ▷ -input_csv_file RM6_genma.csv \
5  ▷ ▷ ▷ -box_width 25 -bit_depth 8 \
6  ▷ ▷ ▷ -partition 3 7 64 \
7  ▷ ▷ -end
8  ▷ magick RM6_genma_draw.bmp RM6_genma_draw.png
9  ▷ open RM6_genma_draw.png
10

```

The next command produces individual pictures for each of the 128 codewords. The command utilizes the `-loop` command to automatize the processing. Afterwards, the codeword pictures are composed in one poster, using the `magick` tool on the command line.

Code Snippet 579

```

1  RM_6_codewords_draw:
2  ▷ $(ORBITER) -v 4 \
3  ▷ ▷ -loop i 0 128 1 \
4  ▷ ▷ ▷ -csv_file_select_rows RM6_codewords.csv %i \
5  ▷ ▷ ▷ -mv RM6_codewords.select.csv RM6_codewords_select.%i.csv \
6  ▷ ▷ ▷ -destring RM6_codewords_select.%i.csv "Codeword" RM6_codewords_sel.%i.csv \

7  ▷ ▷ ▷ -reformat RM6_codewords_sel.%i.csv RM6_codewords_ref.%i.csv 8 \
8  ▷ ▷ ▷ -draw_matrix \
9  ▷ ▷ ▷ ▷ -input_csv_file RM6_codewords_ref.%i.csv \
10 ▷ ▷ ▷ ▷ -box_width 25 -bit_depth 8 \
11 ▷ ▷ ▷ ▷ -partition 3 8 8 \
12 ▷ ▷ ▷ -end \
13 ▷ ▷ ▷ -system "magick RM6_codewords_ref.%i_draw.bmp -frame 8 %i.png" \
14 ▷ ▷ -end_loop i
15 ▷ magick 0.png 1.png 2.png 3.png 4.png 5.png \
16 ▷ ▷ 6.png 7.png 8.png 9.png 10.png +append a0
17 ▷ magick 11.png 12.png 13.png 14.png 15.png \
18 ▷ ▷ 16.png 17.png 18.png 19.png 20.png 21.png +append a1
19 ▷ magick 22.png 23.png 24.png 25.png 26.png \
20 ▷ ▷ 27.png 28.png 29.png 30.png 31.png 32.png +append a2
21 ▷ magick 33.png 34.png 35.png 36.png 37.png \
22 ▷ ▷ 38.png 39.png 40.png 41.png 42.png 43.png +append a3
23 ▷ magick 44.png 45.png 46.png 47.png 48.png \
24 ▷ ▷ 49.png 50.png 51.png 52.png 53.png 54.png +append a4
25 ▷ magick 55.png 56.png 57.png 58.png 59.png \
26 ▷ ▷ 60.png 61.png 62.png 63.png 64.png 65.png +append a5
27 ▷ magick 66.png 67.png 68.png 69.png 70.png \
28 ▷ ▷ 71.png 72.png 73.png 74.png 75.png 76.png +append a6

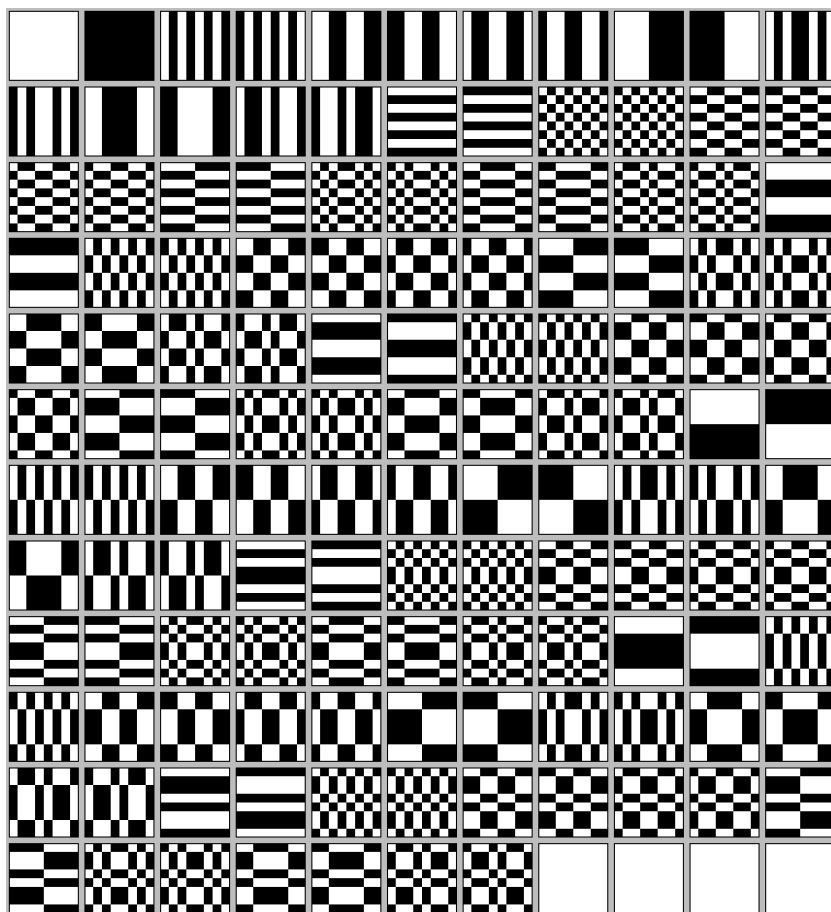
```

```

29  ▷ magick 77.png 78.png 79.png 80.png 81.png \
30  ▷ ▷ 82.png 83.png 84.png 85.png 86.png 87.png +append a7
31  ▷ magick 88.png 89.png 90.png 91.png 92.png \
32  ▷ ▷ 93.png 94.png 95.png 96.png 97.png 98.png +append a8
33  ▷ magick 99.png 100.png 101.png 102.png 103.png \
34  ▷ ▷ 104.png 105.png 106.png 107.png 108.png 109.png +append a9
35  ▷ magick 110.png 111.png 112.png 113.png 114.png \
36  ▷ ▷ 115.png 116.png 117.png 118.png 119.png 120.png +append a10
37  ▷ magick 121.png 122.png 123.png 124.png 125.png \
38  ▷ ▷ 126.png 127.png 0.png 0.png 0.png 0.png +append a11
39  ▷ magick a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 a10 \
40  ▷ ▷ a11 -append poster_RM_1_6.png
41

```

Finally, the 128 codewords are stitched together in the form of a tapestry. Each codeword is a bitvector of length 64, rearranged as 8×8 bitmatrix.



Options for Drawing Modulo N Objects		
Command	Arguments	Purpose
-n	n	Set the number of dots.
-mod_s	s	
-divide_out_by	k	Divide by k.
-file	fname	Read from file.
-inverse		Connect i with $i^{-1} \bmod n$.
-additive_inverse		Connect i with $-i \bmod n$.
-power_cycle	a	Draw a cycle connecting the powers of $a \bmod n$.
-cyclotomic_sets	q list	Draw the q -cyclotomic sets of all elements in the list.
-cyclotomic_sets_thickness		Set the thickness for the drawing of cyclotomic sets.
-draw_option	label	Set the draw options as defined by the given object of type -draw_options.

Table 11.10: Options for Drawing Modulo N Objects

11.6 BCH Codes

Let β be an n -th root of unity over $\mathbb{F}_q$. The minimum polynomial of β over $\mathbb{F}_q$ is denoted as $m_{\beta, \mathbb{F}_q}$. The BCH code of length n and designed distance d is the cyclic code with generator polynomial

$$\text{lcm}\left(m_{\beta^1, \mathbb{F}_q}, m_{\beta^2, \mathbb{F}_q}, \dots, m_{\beta^{d-1}, \mathbb{F}_q}\right).$$

To create the polynomial $m_{\beta^a, \mathbb{F}_q}$, we consider the q -cyclotomic set of a modulo n , which is

$$\{aq^i \bmod n \mid i \in \mathbb{Z}\}.$$

Orbiter can illustrate cyclic sets using the commands of Table 11.10.

Suppose we want to make a BCH-code of length 21 over $\mathbb{F}_8$. In Section 3.2, we considered the q -cyclotomic sets modulo 21 for $q = 8$. Let us produce a pictorial representation. Omitting the singletons, a transversal is given by the sets containing 1, 2, 4, 5, 7, 10, 13. For this reason, we issue the command

Code Snippet 580

```

1  draw_cyclotomic_mod_21_q8:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define Do -draw_options \
4  ▷ ▷ ▷ -radius 100 \
5  ▷ ▷ ▷ -line_width 1.0 -embedded \
6  ▷ ▷ -end \
7  ▷ ▷ -draw_mod_n \
8  ▷ ▷ ▷ "mod.n=on" \
9  ▷ ▷ ▷ -n 21 \
10 ▷ ▷ ▷ -file mod_21_cyclotomic \
11 ▷ ▷ ▷ -cyclotomic_sets 8 "1,2,4,5,7,10,13" \
12 ▷ ▷ ▷ -label_nodes \

```

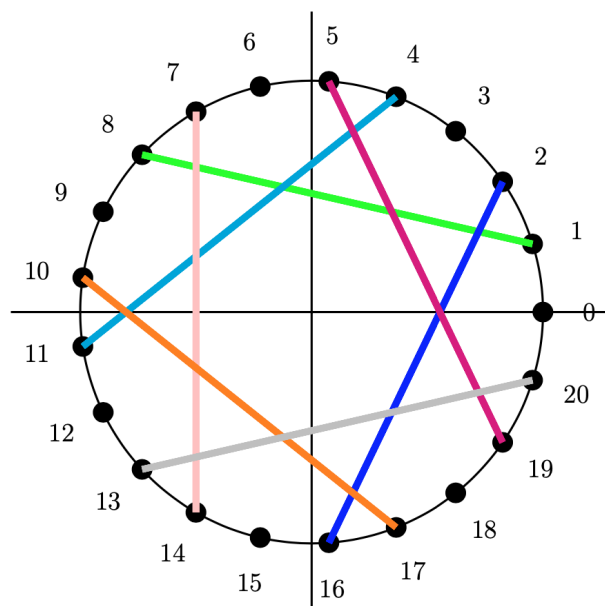


```

13  ▷ ▷ ▷ -draw_options Do \
14  ▷ ▷ -end
15  ▷ $(PDFLATEX) mod_21_cyclotomic_draw.tex
16  ▷ $(OPEN) mod_21_cyclotomic_draw.pdf
17

```

The output is shown below:



We will try BCH-codes with minimum distances 3, 5 and 7. Here is distance 3:

Code Snippet 581

```

1    F_8.BCH_code.d3:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
4    ▷ ▷ -define C -code -field F \
5    ▷ ▷ ▷ -BCH 21 3 \
6    ▷ ▷ -end \
7    ▷ ▷ -with C -do -coding_theoretic_activity -report -end
8    ▷ $(PDFLATEX) BCH_n21_d3_code_n21_k17_q8.tex
9    ▷ $(OPEN) BCH_n21_d3_code_n21_k17_q8.pdf
10

```

The code construction is shown in a latex report:

BCH-code:

$n = 21$, $k = 17$, $d_0 = 3$, $q = 8$,

$g(x) = m_1 m_2 = X^4 + 4X^3 + 4X^2 + 3X + 4$

Chosen cyclotomic sets:

$\{ 1, 8 \}$
 $\{ 2, 16 \}$

The generator polynomial has degree 4

-dense "4,3,4,4,1"

-sparse "4,0,3,1,4,2,4,3,1,4"

The generator matrix is:

$$\begin{bmatrix} 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 & 3 & 4 & 4 & 1 \end{bmatrix}$$

Let us stick to the same parameters for the field and the length but increase the minimum distance to $d = 5$:

Code Snippet 582

```
1 F_8_BCH_code_d5:
2   > $(ORBITER) -v 3 \
3   > > -define F -finite_field -q 8 -override_polynomial 11 -end \
4   > > -define C -code -field F \
5   > > > -BCH 21 5 \
6   > > > -end \
7   > > -with C -do -coding_theoretic_activity -report -end
8   > $(PDFLATEX) BCH_n21_d5_code_n21_k14_q8.tex
9   > $(OPEN) BCH_n21_d5_code_n21_k14_q8.pdf
10
```

The output file is:

BCH-code:
 $n = 21, k = 14, d_0 = 5, q = 8,$

$$g(x) = m_1 m_2 m_3 m_4 = X^7 + 3X^6 + 3X^5 + 2X^4 + X^3 + 2X^2 + X + 2$$

Chosen cyclotomic sets:

$\{1, 8\}$

$\{2, 16\}$

$\{3\}$

$\{4, 11\}$

The generator polynomial has degree 7

-dense "2,1,2,1,2,3,3,1"

-sparse "2,0,1,1,2,2,1,3,2,4,3,5,3,6,1,7"

The generator matrix is:

$$\begin{bmatrix} 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 1 & 2 & 1 & 2 & 3 & 3 \end{bmatrix}$$

We compute the minimum distance:

Code Snippet 583

```
1 F_8_BCH_code_d5_minimum_distance:
2 ▷ $(ORBITER) -v 2 \
3 ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
4 ▷ ▷ -define v -vector -format 14 -field F \
5 ▷ ▷ ▷ -compact $(CODE_BCH_F8_N21_D5_GENMA_OVERRIDE_POLYNOMIAL11) \
6 ▷ ▷ -end \
7 ▷ ▷ -with F -do \
8 ▷ ▷ -coding_theoretic_activity \
9 ▷ ▷ ▷ -minimum_distance v \
10 ▷ ▷ -end
11 # important: use the same polynomial as when creating the code.
12 #
13 # d=5
14
```

The minimum distance turns out to be $d = 5$.

Finally, we create the BCH code with minimum distance $d = 7$:

Code Snippet 584

```

1    F_8_BCH_code.d7:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
4    ▷ ▷ -define C -code -field F \
5    ▷ ▷ ▷ -BCH 21 7 \
6    ▷ ▷ -end \
7    ▷ ▷ -with C -do -coding_theoretic_activity -report -end
8    ▷ $(PDFLATEX) BCH_n21_d7_code_n21_k11_q8.tex
9    ▷ $(OPEN) BCH_n21_d7_code_n21_k11_q8.pdf
10

```

The output file is:

BCH-code:

$n = 21, k = 11, d_0 = 7, q = 8,$

$g(x) = m_1 m_2 m_3 m_4 m_5 m_6 = X^{10} + X^9 + 2X^8 + 5X^7 + 2X^6 + 4X^4 + 6X^3 + 5X^2 + 6X + 6$

Chosen cyclotomic sets:

$\{ 1, 8 \}$

$\{ 2, 16 \}$

$\{ 3 \}$

$\{ 4, 11 \}$

$\{ 5, 19 \}$

$\{ 6 \}$

The generator polynomial has degree 10

-dense "6,6,5,6,4,0,2,5,2,1,1"

-sparse "6,0,6,1,5,2,6,3,4,4,2,6,5,7,2,8,1,9,1,10"

The generator matrix is:

$$\begin{bmatrix}
 6 & 6 & 5 & 6 & 4 & 0 & 2 & 5 & 2 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 6 & 6 & 5 & 6 & 4 & 0 & 2 & 5 & 2 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 6 & 6 & 5 & 6 & 4 & 0 & 2 & 5 & 2 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 6 & 6 & 5 & 6 & 4 & 0 & 2 & 5 & 2 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 6 & 6 & 5 & 6 & 4 & 0 & 2 & 5 & 2 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 6 & 6 & 5 & 6 & 4 & 0 & 2 & 5 & 2 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 6 & 6 & 5 & 6 & 4 & 0 & 2 & 5 & 2 & 1 & 1 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 6 & 6 & 5 & 6 & 4 & 0 & 2 & 5 & 2 & 1 & 1 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 6 & 6 & 5 & 6 & 4 & 0 & 2 & 5 & 2 & 1 & 1 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 6 & 6 & 5 & 6 & 4 & 0 & 2 & 5 & 2 & 1 & 1 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 6 & 6 & 5 & 6 & 4 & 0 & 2 & 5 & 2 & 1 & 1 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 6 & 6 & 5 & 6 & 4 & 0 & 2 & 5 & 2 & 1 & 1
 \end{bmatrix}$$

As a larger example, let us consider the 2-cyclotomic sets of 2 and 3 modulo 255. The following command

produces a graphical representation on a circle (similar to the unit circle in complex analysis). The 255-th roots of unity are placed in the appropriate position.

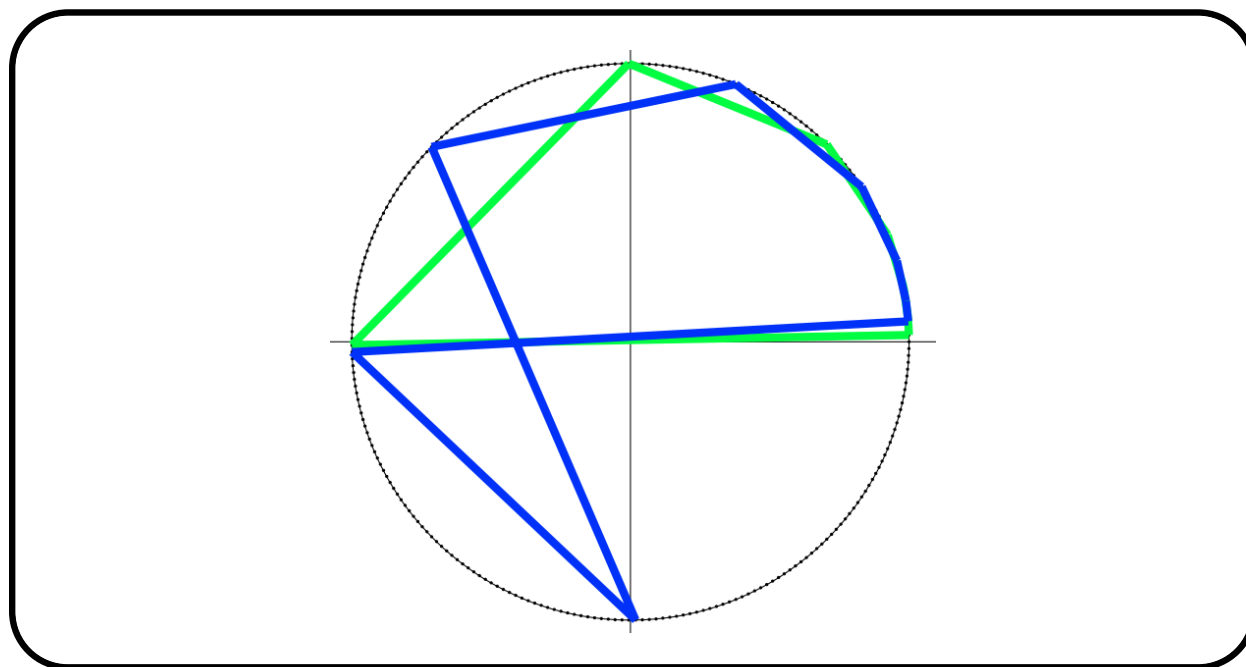
Code Snippet 585

```

1  draw_mod_255_cyclotomic_1_and_3:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define Do -draw_options -nodes_empty -radius 10 \
4  ▷ ▷ ▷ -line_width 0.4 -embedded -end \
5  ▷ ▷ -draw_mod_n \
6  ▷ ▷ ▷ "mod_n=on" \
7  ▷ ▷ ▷ -n 255 \
8  ▷ ▷ ▷ -file mod_255_cyclotomic_1_and_3 \
9  ▷ ▷ ▷ -label_nodes \
10 ▷ ▷ ▷ -cyclotomic_sets 2 "1,3" \
11 ▷ ▷ ▷ -draw_options Do \
12 ▷ ▷ -end
13 ▷ $(PDFLATEX) mod_255_cyclotomic_1_and_3_draw.tex
14 ▷ $(OPEN) mod_255_cyclotomic_1_and_3_draw.pdf
15

```

The drawing is shown below



Suppose we want to make a BCH-code over $\mathbb{F}_{256}$. In order to keep the degree of the generator polynomial low, we try a quadratic field extension. This way, each cyclotomic set has size either 1 or 2. Since

$$256^2 - 1 = (256 + 1)(256 - 1) = 257 \cdot 3 \cdot 5 \cdot 17,$$

we can consider a code of length $n = 771 = 257 \cdot 3$. The following command computes the 256-cyclotomic cosets modulo 771:

Code Snippet 586

```

1   BCH_F256_roots_771:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 256 -end \
4   ▷ ▷ -with F -do -finite_field_activity \
5   ▷ ▷ ▷ -nth_roots 771 \
6   ▷ ▷ -end
7

```

The next command creates a BCH-code of length 771 over $\mathbb{F}_{256}$ with minimum distance at least 16:

Code Snippet 587

```

1   BCH_F256_BCH_code_d16:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 256 -end \
4   ▷ ▷ -define C -code -field F \
5   ▷ ▷ ▷ -BCH 771 16 \
6   ▷ ▷ -end \
7   ▷ ▷ -with C -do -coding_theoretic_activity -report -end
8   ▷ $(PDFLATEX) BCH_n771_d16_code_n771_k741_q256.tex
9   ▷ $(OPEN) BCH_n771_d16_code_n771_k741_q256.pdf
10

```

We compute the minimum distance:

Code Snippet 588

```

1   F_8_BCH_code_d5_minimum_distance:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
4   ▷ ▷ -define v -vector -format 14 -field F \
5   ▷ ▷ ▷ -compact $(CODE_BCH_F8_N21_D5_GENMA_OVERRIDE_POLYNOMIAL11) \
6   ▷ ▷ -end \
7   ▷ ▷ -with F -do \
8   ▷ ▷ -coding_theoretic_activity \
9   ▷ ▷ ▷ -minimum_distance v \
10  ▷ ▷ -end
11  # important: use the same polynomial as when creating the code.
12  #
13  # d=5
14

```

The minimum distance turns out to be $d = 5$.

11.7 Reed-Solomon Codes

Reed-Solomon codes are BCH-codes where the length n divides $q - 1$. In particular, they are cyclic codes. They are almost never binary.

To create a Reed-Solomon code over $\mathbb{F}_7$ with minimum distance at least 3. The command

Code Snippet 589

```
1 RS_6_3:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 7 -end \
4   ▷ ▷ -define C -code -field F -ReedSolomon 6 3 -end \
5   ▷ ▷ -with C -do -coding_theoretic_activity \
6   ▷ ▷ ▷ -export_magma RS_6_3.magma \
7   ▷ ▷ -end \
8   ▷ ▷ -with C -do -coding_theoretic_activity -report -end
9   ▷ $(PDFLATEX) RS_n6_d3_code_n6_k4_q7.tex
10  ▷ $(OPEN) RS_n6_d3_code_n6_k4_q7.pdf
11
```

creates this code. The following report is produced:

RS-code:

$$n = 6, k = 4, d_0 = 3, q = 7,$$

$$g(x) = m_1 m_2 = X^2 + 2X + 6$$

The generator polynomial has degree 2

-dense "6,2,1"

-sparse "6,0,2,1,1,2"

The generator matrix is:

$$\begin{bmatrix} 6 & 2 & 1 & 0 & 0 & 0 \\ 0 & 6 & 2 & 1 & 0 & 0 \\ 0 & 0 & 6 & 2 & 1 & 0 \\ 0 & 0 & 0 & 6 & 2 & 1 \end{bmatrix}$$

As the report shows, the code is cyclic, generated by

$$(X - \alpha)(X - \alpha^2) = (X - 3)(X - 2) = X^2 + 2X + 6.$$

Let us investigate this code. We start with the weight enumerator. The command

Code Snippet 590

```
1 RS_6_3_weight_enumerator:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 7 -end \
4   ▷ ▷ -define C -code -field F -ReedSolomon 6 3 -end \
5   ▷ ▷ -with C -do -coding_theoretic_activity \
```

```

6      ▷ ▷ ▷ -export_magma RS_6_3.magma \
7      ▷ ▷ -end \
8      ▷ ▷ -with C -do \
9      ▷ ▷ -coding_theoretic_activity \
10     ▷ ▷ ▷ -weight_enumerator \
11     ▷ ▷ -end
12

```

computes the weight enumerator, which turns out to be

$$(1, 0, 0, 120, 360, 972, 948).$$

In polynomial form, this is

$$1y^6 + 120x^3y^3 + 360x^4y^2 + 972x^5y + 948x^6.$$

This confirms that the minimum distance is three.

Let us consider an example of a Reed-Solomon code in characteristic two. We will create a Reed Solomon code of designed distance 3 over $\mathbb{F}_8$. The command

Code Snippet 591

```

1      RS_7_3:
2      ▷ $(ORBITER) -v 2 \
3      ▷ ▷ -define F -finite_field -q 8 -end \
4      ▷ ▷ -define C -code -field F -ReedSolomon 7 3 -end \
5      ▷ ▷ -with C -do -coding_theoretic_activity \
6      ▷ ▷ ▷ -export_magma RS_7_3.magma \
7      ▷ ▷ -end \
8      ▷ ▷ -with C -do -coding_theoretic_activity -report -end
9      ▷ $(PDFLATEX) RS_n7_d3_code_n7_k5_q8.tex
10     ▷ $(OPEN) RS_n7_d3_code_n7_k5_q8.pdf
11

```

creates this code. The following report is produced:

RS-code:

$$n = 7, k = 5, d_0 = 3, q = 8,$$

$$g(x) = m_1m_2 = X^2 + 6X + 5$$

The generator polynomial has degree 2

-dense "5,6,1"

-sparse "5,0,6,1,1,2"

The generator matrix is:

$$\begin{bmatrix} 5 & 6 & 1 & 0 & 0 & 0 & 0 \\ 0 & 5 & 6 & 1 & 0 & 0 & 0 \\ 0 & 0 & 5 & 6 & 1 & 0 & 0 \\ 0 & 0 & 0 & 5 & 6 & 1 & 0 \\ 0 & 0 & 0 & 0 & 5 & 6 & 1 \end{bmatrix}$$

As we can see, the code is a cyclic code generated by the polynomial

$$(X - \alpha)(X - \alpha^2) = X^2 + 6X + 5.$$

What is the minimum weight of the code? The following command

Code Snippet 592

```
1  RS_7_3_weight_enumerator:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 8 -end \
4  ▷ ▷ -define C -code -field F -Reed_Solomon 7 3 -end \
5  ▷ ▷ -with C -do -coding_theoretic_activity \
6  ▷ ▷ ▷ -export_magma RS_7_3.magma \
7  ▷ ▷ -end \
8  ▷ ▷ -with C -do \
9  ▷ ▷ -coding_theoretic_activity \
10 ▷ ▷ ▷ -weight_enumerator \
11 ▷ ▷ -end
12
```

computes the weight enumerator, which turns out to be

$$y^7 + 245x^3y^4 + 1225x^4y^3 + 5586x^5y^2 + 12838x^6y + 12873x^7.$$

This shows that the minimum distance is 3, as expected. Computing the automorphism group of the code is computationally infeasible.

The next command performs field reduction of the code.

Code Snippet 593

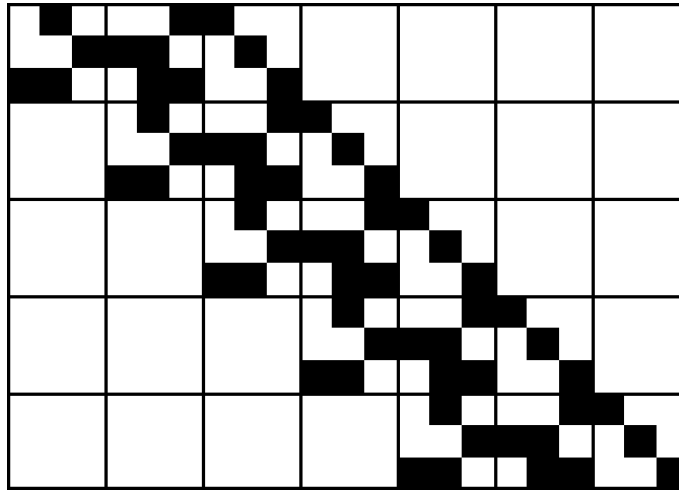
```
1  RS_7_3_field_reduction:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 8 -end \
4  ▷ ▷ -define C -code -field F -Reed_Solomon 7 3 -end \
5  ▷ ▷ -with F -do \
6  ▷ ▷ -finite_field_activity \
7  ▷ ▷ -field_reduction "RS_8_red_2" \
8  ▷ ▷ ▷ 2 5 7 $(CODE_RS_F8_N7_K5_D3_GENMA) \
9  ▷ ▷ -end
10 ▷ $(ORBITER) -v 2 \
11 ▷ ▷ -draw_matrix -input_csv_file RS_8_red_2.csv \
12 ▷ ▷ -box_width 40 -bit_depth 24 \
13 ▷ ▷ -partition 4 "3,3,3,3,3" "3,3,3,3,3,3,3" -end
14 ▷ $(PDFLATEX) field_reduction_Q8.q2.5.7.tex
```

```

15  ▷ $(OPEN) field_reduction_Q8_q2_5_7.pdf
16

```

This produces a $[21, 15]_2$ code. The reduced matrix is below



We store the generator matrix in a makefile variable:

Code Snippet 594

```

1  RS_8_reduced="\
2  01000110000000000000\
3  00111001000000000000\
4  11001100100000000000\
5  00001000110000000000\
6  00000111001000000000\
7  00011001100100000000\
8  00000001000110000000\
9  00000000111001000000\
10 00000011001100100000\
11 00000000001000110000\
12 00000000000111001000\
13 00000000011001100100\
14 00000000000001000110\
15 00000000000000111001\
16 000000000000110011001"
17

```

Let us compute the weight enumerator of the reduced code. The command

Code Snippet 595

```

1  RS_7.3_reduced_weight_enumerator:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 2 -end \

```

```

4    ▷ ▷ -define v -vector -format 15 -field F \
5    ▷ ▷ ▷ -compact $(RS_8_reduced) \
6    ▷ ▷ -end \
7    ▷ ▷ -define C -code -field F \
8    ▷ ▷ ▷ -generator_matrix v \
9    ▷ ▷ -end \
10   ▷ ▷ -with C -do \
11   ▷ ▷ -coding_theoretic_activity \
12   ▷ ▷ ▷ -weight_enumerator \
13   ▷ ▷ -end
14

```

computes the weight enumerator of the binary code. It is

$$\begin{aligned}
 &1y^{21} + 28x^3y^{18} + 84x^4y^{17} + 273x^5y^{16} + 924x^6y^{15} + 1956x^7y^{14} + 2982x^8y^{13} + \\
 &4340x^9y^{12} + 5796x^{10}y^{11} + 5796x^{11}y^{10} + 4340x^{12}y^9 + 2982x^{13}y^8 + 1956x^{14}y^7 + \\
 &924x^{15}y^6 + 273x^{16}y^5 + 84x^{17}y^4 + 28x^{18}y^3 + 1x^{21}
 \end{aligned}$$

In particular, the field reduced Reed-Solomon code has minimum distance three. Since there are other codes of minimum distance 4, the field-reduced code is not optimal.

11.8 Twisted Tensor Product Codes

The command

Code Snippet 596

```

1   TTP_A_4:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F4 -finite_field -q 4 -end \
4   ▷ ▷ -define F16 -finite_field -q 16 -end \
5   ▷ ▷ -define C -code -field F4 -ttpA F16 -end \
6   ▷ ▷ -with C -do -coding_theoretic_activity \
7   ▷ ▷ ▷ -export_magma TTP_A_q4.magma \
8   ▷ ▷ -end \
9   ▷ ▷ -with C -do -coding_theoretic_activity \
10  ▷ ▷ ▷ -report \
11  ▷ ▷ -end
12  ▷ $(PDFLATEX) ttpA_n18_k9_code_n18_k9_q4.tex
13  ▷ $(OPEN) ttpA_n18_k9_code_n18_k9_q4.pdf
14

```

creates the twisted tensor product code arising from the hyperoval of $PG(2, 16)$. The command also produces a report of the code, as shown below.

The generator matrix is:

$$\begin{bmatrix}
 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 1 & 3 & 1 & 3 & 1 & 2 & 3 & 0 & 3 & 2 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 2 & 0 & 2 & 1 & 2 & 3 & 0 & 2 & 3 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\
 1 & 2 & 3 & 1 & 0 & 3 & 2 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
 0 & 3 & 3 & 1 & 1 & 2 & 2 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\
 1 & 1 & 3 & 2 & 0 & 1 & 0 & 0 & 3 & 2 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\
 1 & 1 & 2 & 3 & 1 & 0 & 0 & 0 & 2 & 3 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
 1 & 2 & 2 & 1 & 2 & 1 & 3 & 0 & 2 & 2 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\
 1 & 2 & 0 & 3 & 3 & 0 & 3 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1
 \end{bmatrix}$$

The command also exports the code to magma. The magma file is shown below:

```

K<w> := GF(4);
V := VectorSpace(K, 18);
C := LinearCode(sub<V |
[1,1,1,1,1,1,1,1,0,0,0,0,0,0,0,0,0,0],
[1,w^2,1,w^2,1,w^1,w^2,0,w^2,w^1,1,0,0,0,0,0,0,0],
[0,w^1,0,w^1,1,w^1,w^2,0,w^1,w^2,0,1,0,0,0,0,0,0],
[1,w^1,w^2,1,0,w^2,w^1,0,1,0,0,0,1,0,0,0,0,0],
[0,w^2,w^2,1,1,w^1,w^1,0,0,1,0,0,0,1,0,0,0,0],
[1,1,w^2,w^1,0,1,0,0,w^2,w^1,0,0,0,0,1,0,0,0],
[1,1,w^1,w^2,1,0,0,0,w^1,w^2,0,0,0,0,1,0,0,0],

```

```
[1,w^1,w^1,1,w^1,1,w^2,0,w^1,w^1,0,0,0,0,0,0,1,0],
[1,w^1,0,w^2,w^2,0,w^2,0,1,1,0,0,0,0,0,0,0,1]>);
```

11.9 Bounds

In coding theory, one main question is to determine the best value of $d_{\max}$ for a fixed n , k and q such that a linear $[n, k, d]_q$ code exists. There are many bounds, both upper and lower bounds. An upper bound tells us that no code with $d \geq d_{\max}$ exists. A lower bound tells us that a code with $d \geq d_{\max}$ exists. The command

Code Snippet 597

```
1 bounds_for_d_given_n15_k6_q2:
2 ▷ $(ORBITER) -v 2 \
3 ▷ ▷ -make_bounds_for_d_given_n_and_k_and_q 15 6 2
4
```

gives upper and lower bounds on the optimal minimum distance $d_{\max}$ of a $[15, 6]_2$ code. The values of the Gilbert-Varshamov lower bound and the Singleton, Hamming, Plotkin and Griesmer upper bounds are computed. The output is:

```
n = 15 k=6 q=2
d_GV = 5
d_singleton = 10
d_hamming = 6
d_plotkin = 7
d_griesmer = 6
```

This shows that $5 \leq d_{\max} \leq 6$. The command

Code Snippet 598

```
1 coding_theory_bounds.q2:
2 ▷ $(ORBITER) -v 2 -table_of_bounds 12 2
3 ▷ $(ORBITER) -v 2 \
4 ▷ ▷ -csv_file_latex 1 table_of_bounds_n12_q2.csv
5 ▷ $(PDFLATEX) table_of_bounds_n12_q2.tex
6 ▷ $(OPEN) table_of_bounds_n12_q2.pdf
7
```

produces a table of bounds for binary codes with $n, k \leq 12$. The command

Code Snippet 599

```
1 GV_n15_k6_d5:
2 ▷ $(ORBITER) -v 2 \
3 ▷ ▷ -define F -finite_field -q 2 -end \
4 ▷ ▷ -define C -code -field F \
5 ▷ ▷ ▷ -Gilbert_Varshamov 15 6 5 \
6 ▷ ▷ -end
7
```

creates a $[15, 6, d]_2$ with minimum distance $g \geq 5$ using a greedy algorithm based on the proof of the Gilbert-Varshamov bound. The code that is produced has the following generator matrix:

```

1 1 1 1 1 1 1 1 1 1 0 0 0 0 0
1 1 1 1 1 0 0 0 0 0 1 0 0 0 0
1 1 1 0 0 1 1 0 0 0 0 1 0 0 0
1 1 0 1 0 1 0 1 0 0 0 0 1 0 0
1 0 1 0 1 0 1 1 0 0 0 0 0 1 0
1 0 1 1 0 1 0 0 1 0 0 0 0 0 1

```

To compute the minimum distance of the code, we store the generator matrix

Code Snippet 600

```

1 CODE_GV_N15_K6="\
2 11111111110000\
3 1111100001000\
4 1110011000100\
5 1101010100010\
6 101011100001\
7 10110100100001"
8

```

and apply the following command to determine the weight enumerator of the code:

Code Snippet 601

```

1 GV_n15_k6_d5_weight_enumerator:
2 ▷ $(ORBITER) -v 2 \
3 ▷ ▷ -define F -finite_field -q 2 -end \
4 ▷ ▷ -define v -vector -format 6 -field F \
5 ▷ ▷ ▷ -compact $(CODE_GV_N15_K6) \
6 ▷ ▷ -end \
7 ▷ ▷ -define C -code -field F \
8 ▷ ▷ ▷ -generator_matrix v \
9 ▷ ▷ -end \
10 ▷ ▷ -with C -do \
11 ▷ ▷ -coding_theoretic_activity \
12 ▷ ▷ ▷ -weight_enumerator \
13 ▷ ▷ -end
14

```

The weight enumerator turns out to be

$$1y^{15} + 27x^6y^9 + 24x^8y^7 + 9x^{10}y^5 + 3x^{12}y^3.$$

From this, we see that the code has minimum distance 6, which is better than predicted.

11.10 Classification of Optimal Linear Codes

The classification problem of optimal codes in coding theory is the problem of determining the equivalence classes of codes for a given set of values of n and k and q with a lower bound on d . Orbiter can be used to classify linear codes with given redundancy and bounded minimum distance. The redundancy of a linear $[n, k]$ code is the parameter $r = n - k$. Codes with redundancy r can be identified with subsets of $\text{PG}(r-1, q)$. Under this correspondence, a code with minimum distance at least d corresponds to a subset such that any $d-1$ elements are independent. We use the notation $\Lambda_{r-1,s}(q)$ to denote the poset of subsets of $\text{PG}(r-1, q)$ for which any $d-1$ -subset (if any) is independent. Under the correspondence, the action of $\text{PGL}(r, q)$ on $\Lambda_{r-1,s}(q)$ corresponds to the orbits of equivalent linear codes. For this reason, we are interested in determining the orbits of $\text{PGL}(r, q)$ on $\Lambda_{r-1,s}(q)$. An orbit of size n represents an isometry class of $[n, n-r, d; q]$ codes with $d \geq s+1$. The projective stabilizer of the subset is the automorphism group of the code. The Orbiter command

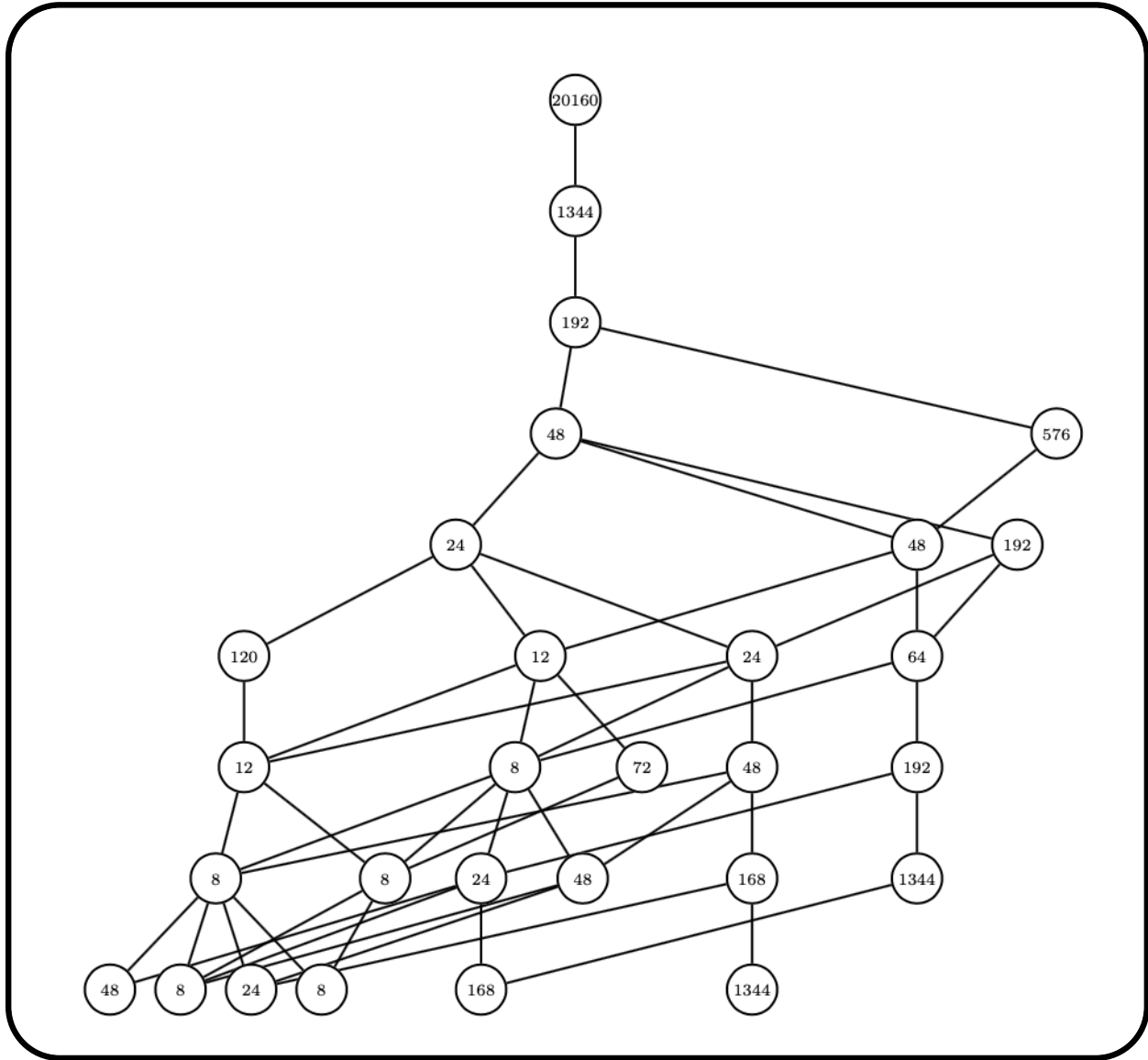
Code Snippet 602

```

1 codes_8_4_4:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4   ▷ ▷ -define Do -draw_options \
5   ▷ ▷ ▷ -embedded -radius 250 \
6   ▷ ▷ ▷ -line_width 1.0 -spanning_tree \
7   ▷ ▷ -end \
8   ▷ ▷ -define Control -poset_classification_control \
9   ▷ ▷ ▷ -problem_label codes_8_4_4 \
10  ▷ ▷ ▷ -draw_options Do \
11  ▷ ▷ ▷ -W \
12  ▷ ▷ -end \
13  ▷ ▷ -define G \
14  ▷ ▷ -linear_group -PGL 4 2 -end \
15  ▷ ▷ -with G -do \
16  ▷ ▷ -group_theoretic_activity \
17  ▷ ▷ ▷ -linear_codes Control 3 8 \
18  ▷ ▷ -end
19  ▷ #$(PDFLATEX) codes_8_4_4_poset_lvl1.8.tex
20  ▷ #$(OPEN) codes_8_4_4_poset_lvl1.8.pdf
21  ▷ #$(PDFLATEX) codes_8_4_4_poset.tex
22  ▷ #$(OPEN) codes_8_4_4_poset.pdf
23

```

classifies linear codes with redundancy 4 and minimum distance at least 4. Orbiter confirms that there is exactly one such code, and it computes the code together with the projective stabilizer. Orbiter creates the action of the group $\text{PGL}(4, 2)$ on the poset $\Lambda_{3,3}(2)$. Using poset classification, Orbiter then produces the poset of orbits shown below



In this diagram, the numbers stand for Orbiter ranks of points in $\text{PG}(3, 2)$. All nodes except for the root node have a number attached to it. The nodes represent subsets. In order to determine the set associated to a node, follow the path from the root node to the node and collect the points according to their labels. The root node represents the empty set. The $[8, 4, 4; 2]$ -code is represented by the set $\{0, 1, 2, 3, 8, 11, 13, 14\}$. The fact that there is only one node at level 8 in the poset of orbits tells us that the code is unique up to equivalence. Let us look at the code. The elements of the set $\{0, 1, 2, 3, 8, 11, 13, 14\}$ are points in $\text{PG}(3, 2)$. We write the coordinate vectors in the columns of a matrix H :

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}.$$

This matrix is the parity check matrix H of the code $\mathcal{C}$. This means that the words of the code are the vectors $\mathbf{c}$ such that $\mathbf{c} \cdot H^\top = 0$. Observe that the vectors that we put in the columns of H all have odd weight. They are in fact the points of the hyperplane $x + y + z + w = 0$. This shows that the stabilizer of the code which is the stabilizer of the set is equal to $\text{AGL}(3, 2)$, a group of order 1344.

Chapter 12

Combinatorics

12.1 Introduction

In Tables 12.1 and 12.2, global Orbiter commands for Combinatorics are summarized.

The command

Code Snippet 603

```
1    random_permutation_12:
2    ▷ $(ORBITER) -v 4 \
3    ▷ ▷ -seed 31415 \
4    ▷ ▷ -random_permutation 12 random.csv
5
```

creates a random permutation in $\text{Sym}(12)$. The permutation is written to the csv file named in the command. The command seeds the pseudo-random number generator with a number (here, the first few digits of π), just to put the random number generator into a state different from the zero state. On some systems, this may prevent that the sequence of random numbers starts with 0.

The command

Code Snippet 604

```
1    Sym_10_conj_classes:
2    ▷ $(ORBITER) -v 2 -conjugacy_classes_Sym_n 10
3    ▷ $(OPEN) classes_Sym_10.csv
4
```

produces a list of the conjugacy classes of $\text{Sym}(10)$. The list is written to a csv file.

The next command computes the character table of the symmetric group $\text{Sym}(4)$:

Code Snippet 605

```
1    Char_Sym_4:
2    ▷ $(ORBITER) -v 2 -character_table_symmetric_group 4
3
```

The command produces the following output:

Combinatorics (Part 1)		
Command	Arguments	Purpose
-random_permutation	n fname	Creates a random permutation in $\text{Sym}(n)$ and stores it in the given file.
-create_random_k_subsets	n k N	Creates N random k -subsets of an n -set.
-read_poset_file	fname	Reads a poset from the given file.
-grouping	x-stretch	Override placement of orbits while reading poset-file. Apply x-stretch to elements in orbit.
-list_parameters_of_SRG	$v_{\max}$	Performs a sift for putative parameter sets of SRGs.
-conjugacy_classes_Sym_n	n	Compute a list of conjugacy classes of $\text{Sym}(n)$.
-tree_of_all_k_subsets	n k	Creates a tree-file for all k -subsets of an n -set.
-Delandtsheer_Doyen	params	For valid parameters, see Section 14.4.
-tdo_refinement	params	For valid parameters, see Section 12.7.
-tdo_print	fname	See Section 12.7.
-convert_stack_to_tdo	fname	See Section 12.7.
-maximal_arc_parameters	q r	See Section 12.7.
-arc_parameters	q s r	See Section 12.7.
-pentomino_puzzle		

Table 12.1: Combinatorics (Part 1)

Combinatorics (Part 2)		
Command	Arguments	Purpose
-read_solutions_and_tally	fname sz	Reads solutions from the given file.
-make_elementary_symmetric_functions	n $k_{\max}$	Computes all elementary symmetric functions in n variables of degree k with $1 \leq k \leq k_{\max}$.
-make_elementary_symmetric_function	n k	Computes the elementary symmetric function in n variables of degree k .
-Dedekind_numbers	$n_{\min}$ $n_{\max}$ $q_{\min}$ $q_{\max}$	Computes the Dedekind numbers $D_{n,q}$ for $n_{\min} \leq n \leq n_{\max}$ and $q_{\min} \leq q \leq q_{\max}$.
-q_binomial	n k q	Computes the q -binomial n, k, q .
-rank_k_subset	n k L	Computes the ranks of k -subsets chosen from an n -set. L is a list of k -sets taken from an n -set.
-geometry_builder	params	For parameters, see Section 12.6.
-union	file input-file output-file	Perform the union of sets given in files.
-dot_product_of_columns	file	Perform the dot product of all columns of the matrix in the given file.
-dot_product_of_rows	file	Perform the dot product of all rows of the matrix in the given file.
-matrix_multiply_over_Z	m1 m2	Perform the multiplication (over $\mathbb{Z}$) of the given matrix objects.
-rowspan_over_R	matrix	Compute a basis of the rowspan of the given matrix over $\mathbb{R}$.
-read_widor	file	Create a tactical decomposition from a file in Widor format.
-Kaempfer		Invoke Kaempfer's TDO-refinement algorithm.
-domino_portrait	D s fname options	Computes a domino portrait for a graphics file in r/g/b format using double- D domino sets.
-test_if_distance_regular_graph	fname	Tests whether the layered graph in the given file is distance regular. If so, compute the parameters.

Table 12.2: Combinatorics (Part 2)

The character table of $\text{Sym}(4)$ is the matrix

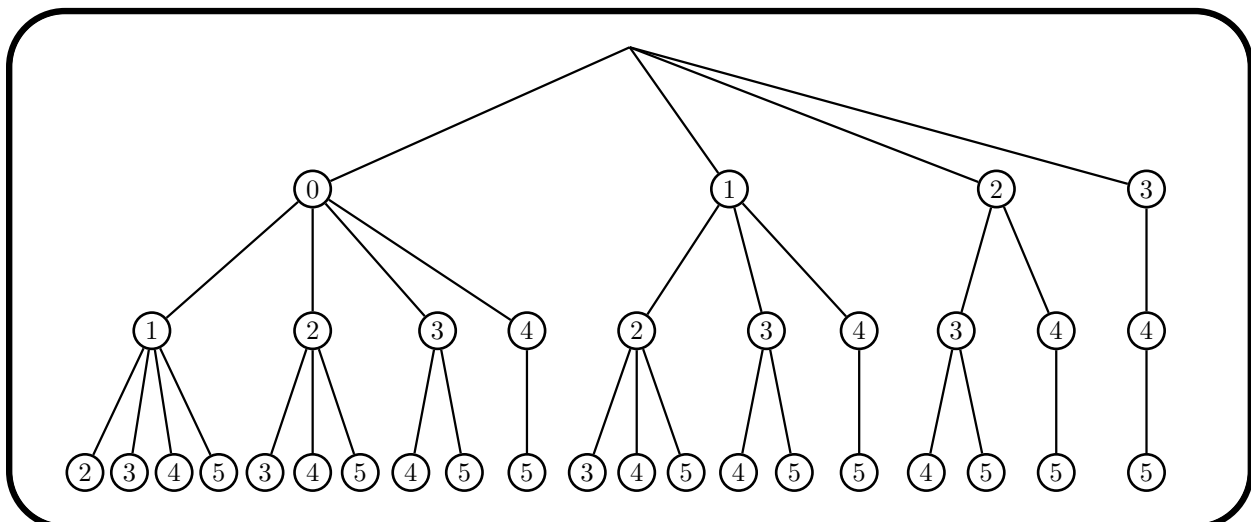
$$\begin{bmatrix} 1 & -1 & 1 & 1 & -1 \\ 3 & 1 & 0 & -1 & -1 \\ 2 & 0 & -1 & 2 & 0 \\ 3 & -1 & 0 & -1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}.$$

Orbiter often uses the lexicographic order to arrange things. Consider the k -subsets of an n -set. The following example creates the 3-subsets of a 6-set in lexicographic order. A tree representation is produced. The leaves of the tree represent the subsets:

Code Snippet 606

```
1 all_subsets_6_3:
2   ▷ $(ORBITER) -v 2 -tree_of_all_k_subsets 6 3
3   ▷ $(ORBITER) -v 20 \
4   ▷ ▷ -define Do -draw_options -embedded \
5   ▷ ▷ ▷ -nodes -radius 80 \
6   ▷ ▷ ▷ -xin 5000 -yin 5000 \
7   ▷ ▷ ▷ -xout 1000000 -yout 500000 \
8   ▷ ▷ ▷ -scale 0.5 -line_width 1.0 \
9   ▷ ▷ -end \
10  ▷ ▷ -tree_draw \
11  ▷ ▷ ▷ -file all_k_subsets_n6_k3.tree \
12  ▷ ▷ ▷ -draw_options Do \
13  ▷ ▷ -end
14  ▷ $(PDFLATEX) all_k_subsets_n6_k3_draw.tex
15  ▷ $(OPEN) all_k_subsets_n6_k3_draw.pdf
16
```

Here is the tree:



The following command illustrates how to create random k -subsets of a set of size n . In the example, we create 20 5-subsets of a 10-element set:

Code Snippet 607

```
1  random_k_subsets:
2  ▷ $(ORBITER) -v 4 \
3  ▷ ▷ -create_random_k_subsets 10 5 20
4
```

Using the lexicographic order, the k -subsets of an n -element set are ranked. The following command computes the ranks of a number of 3-subsets of a 10-element set:

Code Snippet 608

```
1  rank_k_subsets_test:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define Blocks -vector -format 7 \
4  ▷ ▷ ▷ -dense "0,1,2,0,3,4,1,3,5,2,4,5,3,6,7,1,6,8,0,6,9" \
5  ▷ ▷ ▷ -end \
6  ▷ ▷ -rank_k_subset 10 3 Blocks
7
```

Orbiter can create the Sylvester type Hadamard matrix of size 2^n (also called the Walsh matrix). The following command creates the matrix of size $2^4 \times 2^4$ and produces a graphical representation:

Code Snippet 609

```
1  Walsh_matrix_4:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -with F -do -finite_field_activity \
5  ▷ ▷ ▷ -Walsh_matrix 4 -end
6  ▷ $(ORBITER) -v 2 -draw_matrix \
7  ▷ ▷ -input_csv_file Walsh_01.4.csv \
8  ▷ ▷ -box_width 10 -bit_depth 24 -partition 3 16 16 -end
9  ▷ #$(PDFLATEX) GF_2.tex
10 ▷ #$(OPEN) GF_2.pdf
11
```

The following command creates the matrix of Dedekind numbers of order at most 10:

Code Snippet 610

```
1  Dedekind_10_10:
2  ▷ $(ORBITER) -v 3 -Dedekind_numbers 2 10 2 10
3  ▷ $(ORBITER) -v 3 -csv_file_latex 1 Dedekind_2.10.2.10.csv
4  ▷ $(PDFLATEX) Dedekind_2.10.2.10.tex
5  ▷ $(OPEN) Dedekind_2.10.2.10.pdf
6
```

The following table is produced:

2	3	4	5	6	7	8	9	10
2	1	3	6	10	15	21	28	36
3	2	8	20	40	70	112	168	240
4	3	18	60	150	315	588	1008	1620
5	6	48	204	624	1554	3360	6552	11808
6	9	116	670	2580	7735	19544	43596	88440
7	18	312	2340	11160	39990	117648	299592	683280
8	30	810	8160	48750	209790	720300	2096640	5380020
9	56	2184	29120	217000	1119720	4483696	14913024	43046640
10	99	5880	104754	976248	6045837	28245840	107370900	348672528

The following command computes the q -binomial

$$\begin{bmatrix} 8 \\ 3 \end{bmatrix}_2$$

Code Snippet 611

```
1  q_binomial_8_3_2:
2  ▷ $(ORBITER) -v 3 \
3  ▷ -q_binomial 8 3 2
4
```

The following command creates the elementary symmetric functions in 4 variables.

Code Snippet 612

```
1  elementary_symmetric_functions_4:
2  ▷ $(ORBITER) -make_elementary_symmetric_functions 4 4
3
```

The output is:

```
k=1 :
x0 + x1 + x2 + x3
k=2 :
x0*x1 + x0*x2 + x0*x3 + x1*x2 + x1*x3 + x2*x3
k=3 :
x0*x1*x2 + x0*x1*x3 + x0*x2*x3 + x1*x2*x3
k=4 :
x0*x1*x2*x3
```

Orbiter can compute Domino portraits. To do so, we need an input file in r/g/b format of size $(D+1)s \times Ds$, where $D = 7$ for double-six dominos.

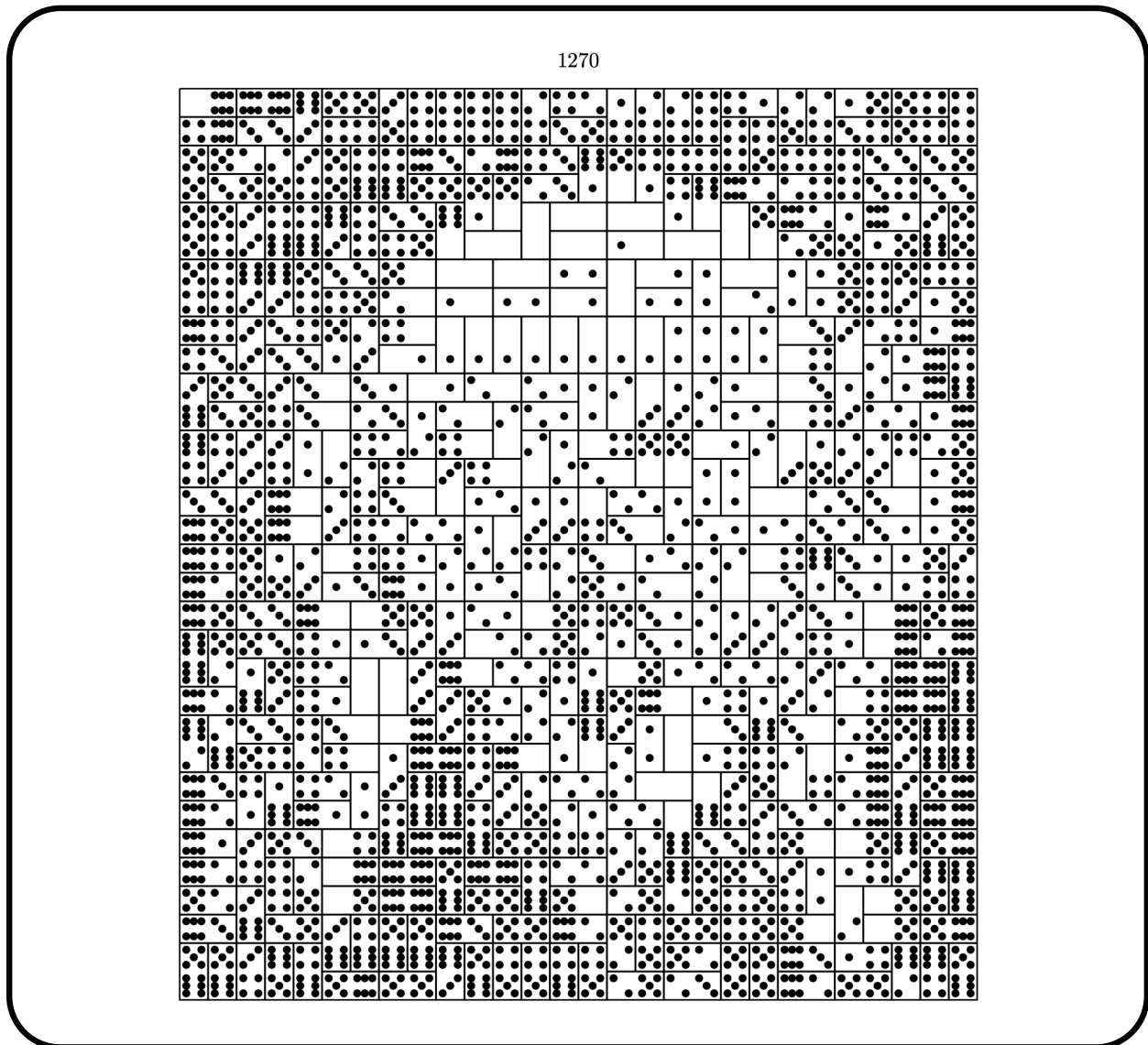
Code Snippet 613

```

1  domino_portrait:
2  ▷ cp $(ORBITER_PATH)/examples/users_guide/Munch.3.? .
3  ▷ $(ORBITER) -v 3 -domino_portrait 7 4 Munch_3 -end
4

```

The portrait is shown below



It is possible to compare the domino portrait with a grayscale version of the input image. The following command creates a grayscale image of the input file that was written during the previous command.

Code Snippet 614

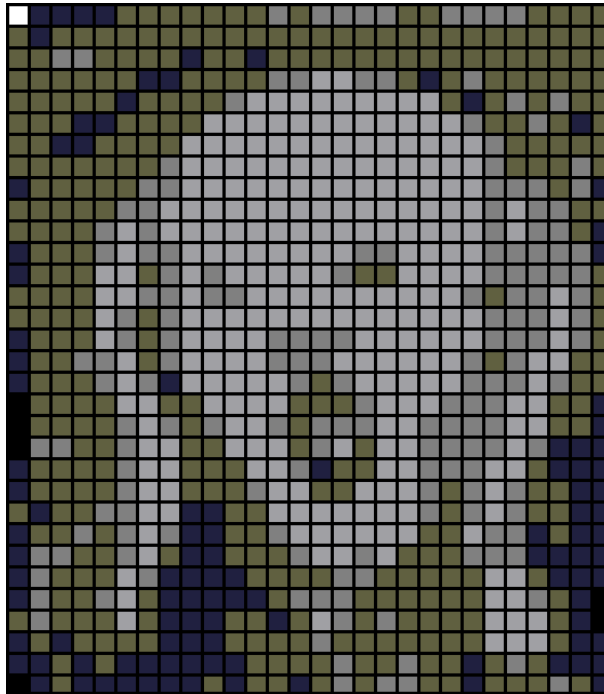
```

1  domino_portrait_input:
2  ▷ cp $(ORBITER_PATH)/examples/users_guide/Munch.3.? .
3  ▷ $(ORBITER) -v 2 \

```

```
4    ▷ ▷ -define all_one_r -vector -repeat 1 28 -end \  
5    ▷ ▷ -define all_one_c -vector -repeat 1 32 -end \  
6    ▷ ▷ -draw_matrix \  
7    ▷ ▷ ▷ -grayscale \  
8    ▷ ▷ ▷ -invert_colors \  
9    ▷ ▷ ▷ -input_csv_file Munch_3_m.csv \  
10   ▷ ▷ ▷ -box_width 20 -bit_depth 8 \  
11   ▷ ▷ ▷ -partition 3 \  
12   ▷ ▷ ▷ ▷ all_one_c all_one_r \  
13   ▷ ▷ -end  
14   ▷ $(OPEN) Munch_3_m.draw.bmp  
15
```

The grayscale version of the input file is



12.2 Combinatorial Objects

Orbiter facilitates processing batch processing of combinatorial data. The data is described by means of input streams. An input stream defines a sequence of combinatorial objects, all of the same type. Once an input stream is defined, there are commands for activities that can be applied to the objects in the stream. It does not matter how many objects are in the stream.

Tables 12.3 and 12.4 show the commands that are available to define input streams.

Let us consider some examples. First, we create the Hirschfeld surface in $\text{PG}(3, 4)$, using point ranks as described in Section 4.2. The 45 points on the surface can be coded as in the following makefile variable:

Code Snippet 615

```
1  HIRSCHFELD_SURFACE_Q4.SET_OF_POINTS="0,1,2,3,4,5,6,7,8,9,\
2  10,11,12,13,14,23,26,27,30,31,34,35,38,39,42,47,48,51,52,\
3  53,54,59,60,61,62,67,68,69,70,75,76,79,80,81,82"
4
```

The next command creates a combinatorial object from this set of points:

Code Snippet 616

```
1  Hirschfeld_q4_from_set:
2  ▷ $(ORBITER) -v 4 \
3  ▷ ▷ -define H -set -here \
4  ▷ ▷ ▷ $(HIRSCHFELD_SURFACE_Q4.SET_OF_POINTS) \
5  ▷ ▷ -end \
6  ▷ ▷ -define C -combinatorial_object \
7  ▷ ▷ ▷ -label Hirschfeld_surface Hirschfeld\_surface \
8  ▷ ▷ ▷ -set_of_points H \
9  ▷ ▷ -end
10
```

The next example creates the two hyperovals in $\text{PG}(2, 16)$. The hyperovals are stored in two makefile variables:

Code Snippet 617

```
1  HYPEROVAL_16_144="0, 1, 2, 3, 52, 67, 89, 106, 126, \
2  141, 159, 176, 184, 199, 220, 235, 245, 262"
3
```

Code Snippet 618

```
1  HYPEROVAL_16_16320="0, 1, 2, 3, 52, 70, 83, 109, 127, \
2  139, 156, 174, 186, 199, 217, 229, 256, 264"
3
```

The next command creates combinatorial objects for these two hyperovals:

Defining Combinatorial Objects (Part 1)		
Command	Arguments	Purpose
-label	label-txt label-tex	A label for the objects.
-set_of_points	S	A set S consisting of points.
-set_of_lines	S	A set S consisting of lines.
-set_of_points_and_lines	$S_1 S_2$	A set S_1 consisting of points and a second set S_2 consisting of lines.
-set_of_packing	$S q$	A set S of packings of $\text{PG}(3, q)$.
-file_of_points	fname	A set consisting of points read from file.
-file_of_points_csv	fname col- header	A csv-file containing sets of points in a specific column. The column header is given.
-file_of_lines	fname	A set consisting of lines read from file.
-file_of_packings	fname	A set consisting of packings read from file.
-file_of_packings_through_spread_table	spread-table fname- packings q	A file of packings of $\text{PG}(3, q)$. This relies on the presence of a spread-table.
-file_of_designs_through_block_orbits	fname1 fname2 v k	A set of designs (or configurations) built from a set of orbits on k -subsets. The file fname1 is a matrix of zero-one vectors, for instance the solutions of a Diophantine system (see Section 16.3). The file fname2 is a list of orbits of a group on k -subsets created using the -export_something activity from Table 8.3 using the keyword set_orbits. The design will be invariant under the group used to compute the k -orbits.
-file_of_designs_through_blocks	fname col- umn v b k	A set of designs (or configurations) built from a set of k -subsets. The file fname is a csv file, and the blocks are listed as coded k -subsets in the given column.
-file_of_point_set	fname	A file containing sets of points.
-file_of_designs	fname v b k c	A file containing designs or large sets. Here, v is the number of points in the design, b is the number of blocks, k is the block size, and c is the size of one class of the partition.

Table 12.3: Defining Combinatorial Objects (Part 1)

Defining Combinatorial Objects (Part 2)		
Command	Arguments	Purpose
-file_of_incidence_geometries	fname v b f	A file of incidence geometries. The incidence structure is listed as set of flags. Here, v is the number of points, b is the number of blocks and f is the number of flags. If point i and block j are incident, this corresponds to the value $ib + j$ being listed as a flag (recall that i and j are zero-based).
-file_of_incidence_geometries_by_row_ranks	fname v b r	A file describing incidence geometries defined by their row ranks.
-incidence_geometry	fname v b f	An incidence geometry defined by a set of flags. Here, v is the number of points, b is the number of blocks and f is the number of flags.
-incidence_geometry_by_row_ranks	fname v b r	An incidence geometry defined by row ranks.
-from_parallel_search	fname-mask nb_cases cases.fname	
-orbiter_file	fname	Orbiter file with orbit representatives created during poset classification.
-csv_file	fname column	The given column in a csv-file.
-graph_by_adjacency_matrix	adjacencylist N	A graph on N vertices whose adjacency matrix is given (upper triangle only).
-graph_by_adjacency_matrix_from_file	fname column N	A set of graphs on N vertices whose adjacency matrices are given in the given column in the given file (upper triangle only).
-graph_object	label	A graph object from the symbol table.
-design_object	label	A design object from the symbol table.
-multi_matrix	s1 s2	s1 is a string of three integers m,n,max. s2 is the entries of an $m \times n$ matrix. max is an upper bound for the entries of the matrix.
-geometric_object	label	A geometric object from the symbol table.
-Kaempfer_file	fname	A set of tactical decompositions from a Kaempfer output file.

Table 12.4: Defining Combinatorial Objects (Part 2)

Code Snippet 619

```

1    hyperoval_16_create:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define C -combinatorial_object \
4    ▷ ▷ ▷ -label hyperoval_q16 hyperoval\_q16 \
5    ▷ ▷ ▷ -set_of_points $(HYPEROVAL_16_16320) \
6    ▷ ▷ ▷ -set_of_points $(HYPEROVAL_16_144) \
7    ▷ ▷ -end \
8

```

In the next example, we read the points of an elliptic curve from file (see Section 4.2):

Code Snippet 620

```

1    EC_read:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 11 -end \
4    ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5    ▷ ▷ -define Curve -geometric_object P \
6    ▷ ▷ ▷ -elliptic_curve 1 3 \
7    ▷ ▷ -end \
8    ▷ ▷ -with Curve -do \
9    ▷ ▷ ▷ -combinatorial_object_activity -save \
10   ▷ ▷ -end
11   ▷ $(ORBITER) -v 4 \
12   ▷ ▷ -define C -combinatorial_object \
13   ▷ ▷ ▷ -label EC EC \
14   ▷ ▷ ▷ -file_of_points elliptic_curve_b1_c3.q11.txt \
15   ▷ ▷ -end
16

```

In the next example, we read a packing, using a previously defined table of spreads, stored in a csv file.

Code Snippet 621

```

1    PG_3_5_assume_31_read:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define C -combinatorial_object \
4    ▷ ▷ ▷ -label packings_25 packings_25 \
5    ▷ ▷ ▷ -file_of_packings_through_spread_table \
6    ▷ ▷ ▷ ▷ H31_packings.csv \
7    ▷ ▷ ▷ ▷ SPREAD_TABLES_5_REG/spread_25_spreads.csv \
8    ▷ ▷ ▷ ▷ SpreadRep \
9    ▷ ▷ ▷ ▷ 5 \
10   ▷ ▷ -end
11

```

The following command reads a file of large sets of designs:

Code Snippet 622

```

1    LS_AG_2_3_read:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define C -combinatorial_object \
4    ▷ ▷ ▷ -label designs designs \
5    ▷ ▷ ▷ -file_of_designs \
6    ▷ ▷ ▷ solutions.csv 9 84 3 12 \
7    ▷ ▷ -end
8

```

The next command reads incidence geometries from a file containing the flags:

Code Snippet 623

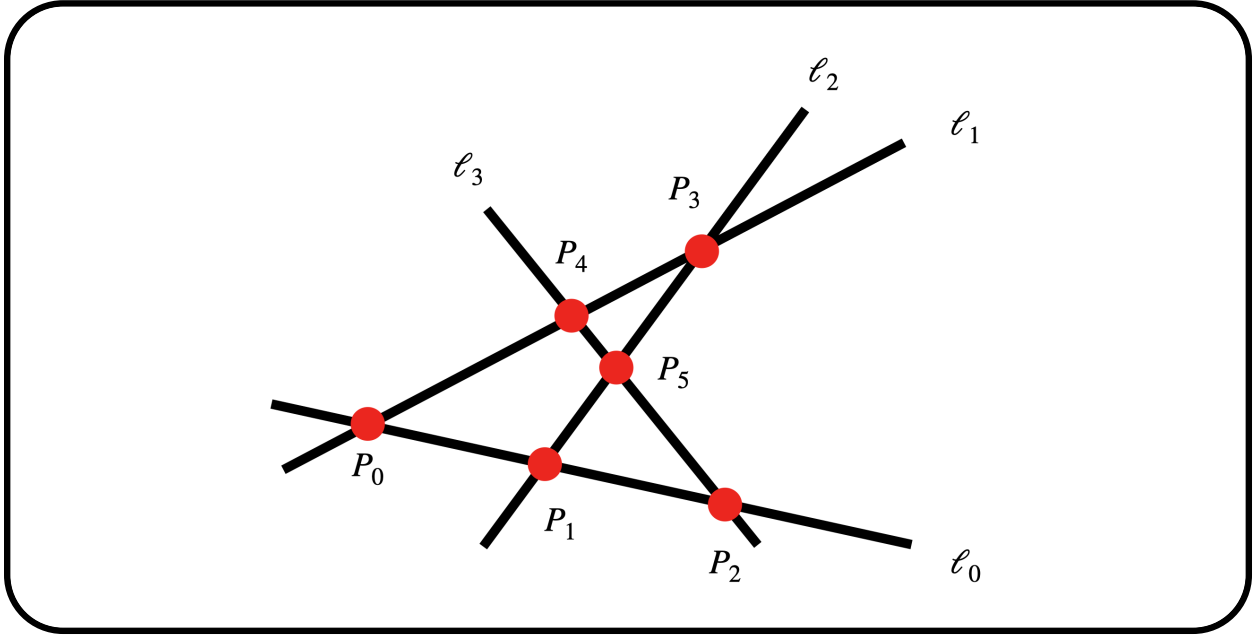
```

1    geo_7_3_read:
2    ▷ $(ORBITER) -v 10 \
3    ▷ ▷ -define Doinc -draw_incidence_structure_options \
4    ▷ ▷ ▷ -width 60 -width_10 6 -end \
5    ▷ ▷ -define C -combinatorial_object \
6    ▷ ▷ ▷ -label geo_7_3 geo\7\3 \
7    ▷ ▷ ▷ -file_of_incidence_geometries \
8    ▷ ▷ ▷ 7_3.inc 7 7 21 \
9    ▷ ▷ -end
10

```

Combinatorial objects can be stored in text files. There can be any number of objects in one file. The objects themselves are coded. For instance, a set of points in projective space is given as a set of integers, using the Orbiter point ranks. Likewise, a set of lines is coded using Orbiter line ranks. For designs, there are several ways in which the object can be stored. One way is by listing the incidences (flags) in a numerical form. An incidence between point i and line j is recorded as $ib + j$. Here, i and j are the zero based index values of the point and line pair, and b is the number of lines. Another way is by recording the flags in a row of the incidence matrix as a subset. In this case, the number of incidences per row must be constant, say r . The flags in a given row thus comprise an r -subset of a set of size b , and the ranking of subsets is used to encode the flags in one row. Proceeding in a row-by-row fashion, the incidence matrix is encoded as a vector of ranks of length v . Yet another way is by listing the columns of the incidence matrix, provided the number of incidences is constant across columns, say k . In this case, we encode the flags in one column as a rank of a k -subset of a set of size v .

The Pasch configuration is the configuration of 6 points and 4 lines shown below:



In the geometry, we have 4 lines, which we can identify with the index sets of the points as $\{0, 1, 2\}$, $\{0, 3, 4\}$, $\{1, 3, 5\}$ and $\{2, 4, 5\}$. The incidence matrix of the configuration is

	ℓ_0	ℓ_1	ℓ_2	ℓ_3
P_0	1	1	0	0
P_1	1	0	1	0
P_2	1	0	0	1
P_3	0	1	1	0
P_4	0	1	0	1
P_5	0	0	1	1

The rows correspond to the points, the columns correspond to the lines. The labels of points and lines are shown.

How to encode an incidence structure for computer storage? There are three ways to encode the incidence structure. One way encodes the flags of the geometry. This will be described next. The flag space is the set of all possible flags in the incidence matrix between the given number of points and lines. The space is totally ordered using the row-major ordering

	ℓ_0	ℓ_1	ℓ_2	ℓ_3
P_0	0	1	2	3
P_1	4	5	6	7
P_2	8	9	10	11
P_3	12	13	14	15
P_4	16	17	18	19
P_5	20	21	22	23

The Pasch configuration can now be coded as

$$\{0, 1, 4, 6, 8, 11, 13, 14, 17, 19, 22, 23\}.$$

The file `pasch.inc` contains:

```
6 4 12
0 1 4 6 8 11 13 14 17 19 22 23
-1 1
24
```

The first line lists the number of rows and columns of the incidence matrix, and the number of incidences. The geometry is encoded on the next line. After that, a marker of -1 shows that this is the only geometry in this file (the file format allows for any number of incidence geometries, all with the same parameters). The final row is the order of the automorphism group of the geometry. This row is optional. In case that there are several geometries in the file, the orders will all be listed. In this case, the possible values will be collected with multiplicities, and listed in decreasing order. The command

Code Snippet 624

```
1  geo_pasch_read:
2  ▷ $(ORBITER) -v 10 \
3  ▷ ▷ -define C -combinatorial_object \
4  ▷ ▷ ▷ -label Pasch Pasch \
5  ▷ ▷ ▷ -file_of_incidence_geometries \
6  ▷ ▷ ▷ pasch.inc 6 4 12 \
7  ▷ ▷ -end
8
```

reads the incidence geometry from the file `pasch.inc`. It is also possible to enter the incidence geometry directly from the command line. The following example uses the `-incidence_geometry` command to do so:

Code Snippet 625

```
1  geo_pasch_given:
2  ▷ $(ORBITER) -v 10 \
3  ▷ ▷ -define C -combinatorial_object \
4  ▷ ▷ ▷ -label Pasch Pasch \
5  ▷ ▷ ▷ -incidence_geometry \
6  ▷ ▷ ▷ ▷ "0,1,4,6,8,11,13,14,17,19,22,23" \
7  ▷ ▷ ▷ ▷ 6 4 12 \
8  ▷ ▷ -end
9
```

Let us study small Neumaier graphs. The makefile variables

Code Snippet 626

```
1  NEUMAIER_GRAPH_16_1_ADJ="1,1,1,1,1,0,0,1,1,0,0,1,1,0,0,1,1,1,0,0,0,0,1,1,0,0,1,
1,1,0,0,1,1,1,0,0,0,0,1,1,0,0,1,1,0,0,1,1,1,1,0,0,1,1,1,1,0,1,0,1,0,1,0,1,1,0,1
,0,1,0,1,0,1,1,1,0,1,0,0,1,0,1,0,1,0,1,1,0,1,0,1,1,1,1,0,0,1,1,1,0,1,1,0,1,0,1,1,
0,1,0,0,1,1,1,1,1,1,1"
```

2

and

Code Snippet 627

```

1  NEUMAIER_GRAPH_16_2_ADJ="1,1,1,1,0,0,1,1,0,0,1,1,0,0,1,1,0,0,0,0,1,1,0,0,1,
    1,1,0,0,1,1,1,0,1,0,1,0,1,0,0,0,1,1,0,1,0,1,0,1,1,1,1,1,0,0,0,0,1,1,1,1,0,0,
    ,1,1,1,1,0,0,1,1,0,1,0,0,1,0,1,0,1,0,1,1,1,1,1,1,1,1,0,0,0,0,1,1,1,1,0,0,
    0,1,1,0,0,1,1,1,1,1,1"
2

```

hold the adjacency matrices of two Neumaier graphs on 16 vertices.

The command

Code Snippet 628

```

1  neumaier_graphs_16:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define C -combinatorial_object \
4  ▷ ▷ ▷ -label neumaier_graph neumaier\_graph \
5  ▷ ▷ ▷ -graph_by_adjacency_matrix \
6  ▷ ▷ ▷ $(NEUMAIER_GRAPH_16_1_ADJ) 16 \
7  ▷ ▷ ▷ -graph_by_adjacency_matrix \
8  ▷ ▷ ▷ $(NEUMAIER_GRAPH_16_2_ADJ) 16 \
9  ▷ ▷ -end
10

```

creates the associated graphs as Orbiter objects.

The command

Code Snippet 629

```

1  neumaier_graphs_16_from_file:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define C -combinatorial_object \
4  ▷ ▷ ▷ -label neumaier_graph neumaier\_graph \
5  ▷ ▷ ▷ -graph_by_adjacency_matrix_from_file \
6  ▷ ▷ ▷ NG_16_9_4_2_4.csv ADJ 16 \
7  ▷ ▷ -end
8

```

reads Neumaier graphs on 16 vertices from an csv file called

NG_16_9_4_2_4.csv.

The command

Code Snippet 630

```
1      Neumaier_graph_25:
2      ▷ $(ORBITER) -v 10 \
3      ▷ ▷ -define Gamma -graph -Neumaier_graph_25 -end \
4      ▷ ▷ -with Gamma -do \
5      ▷ ▷ -graph_theoretic_activity \
6      ▷ ▷ ▷ -test_SRG_property \
7      ▷ ▷ -end \
8      ▷ ▷ -with Gamma -do \
9      ▷ ▷ -graph_theoretic_activity \
10     ▷ ▷ ▷ -eigenvalues \
11     ▷ ▷ -end \
12     ▷ ▷ -with Gamma -do \
13     ▷ ▷ -graph_theoretic_activity \
14     ▷ ▷ ▷ -find_cliques -target_size 5 -end \
15     ▷ ▷ -end \
16     ▷ ▷ -with Gamma -do \
17     ▷ ▷ -graph_theoretic_activity \
18     ▷ ▷ ▷ -automorphism_group \
19     ▷ ▷ -end \
20     ▷ ▷ -with Gamma -do \
21     ▷ ▷ -graph_theoretic_activity \
22     ▷ ▷ ▷ -test_Neumaier_property 5 \
23     ▷ ▷ -end
24
```

creates a specific Neumaier graph on 25 vertices and computes properties of the graph. It verifies the strongly regular graph property, computes the eigenvalues, finds all cliques of size 5, computes the automorphism group, and tests the Neumaier property.

12.3 Activities for Geometric Objects

Table 12.5 shows activities for geometric objects:

Some commands require extra options. Table 12.6 shows options for the report which is generated after a classification. For more details on classification, please see Section 17.1.

The Edge curve is given by the equation

$$X^4 - Y^4 - Z^4 + 2f^2Y^2Z^2 + 4fX^2YZ = 0$$

where f is a primitive element of $\mathbb{F}_q$. Let us pick $q = 17$. The next example creates the Edge curve in $\text{PG}(2, 17)$ using the primitive element $f = 3$. For more details, see Section 7.5.

Code Snippet 631

```

1   Edge_curve_17:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define F -finite_field -q 17 -end \
4   ▷ ▷ -define R -polynomial_ring \
5   ▷ ▷ ▷ -field F \
6   ▷ ▷ ▷ -number_of_variables 3 \
7   ▷ ▷ ▷ -homogeneous_of_degree 4 \
8   ▷ ▷ ▷ -monomial_ordering_partition \
9   ▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \
10  ▷ ▷ -end \
11  ▷ ▷ -assign Eqn -from \
12  ▷ ▷ -with R -do \
13  ▷ ▷ ▷ -ring_theoretic_activity \
14  ▷ ▷ ▷ -parse_equation \
15  ▷ ▷ ▷ ▷ "Edge_curve" "Edge\_curve" \
16  ▷ ▷ ▷ ▷ $(EDGE_CURVE_EQUATION_ALGEBRAIC) \
17  ▷ ▷ ▷ ▷ f 3 \
18  ▷ ▷ -end \
19  ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
20  ▷ ▷ -define C -geometric_object P \
21  ▷ ▷ ▷ -projective_variety R \
22  ▷ ▷ ▷ ▷ "Edge_q17" "Edge\_q17" \
23  ▷ ▷ ▷ ▷ Eqn \
24  ▷ ▷ -end
25
```

The following command computes the line type of the Edge curve:

Code Snippet 632

```

1   Edge_curve_17_line_type:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define F -finite_field -q 17 -end \
4   ▷ ▷ -define R -polynomial_ring \
5   ▷ ▷ ▷ -field F \
6   ▷ ▷ ▷ -number_of_variables 3 \
7   ▷ ▷ ▷ -homogeneous_of_degree 4 \
8   ▷ ▷ ▷ -monomial_ordering_partition \
9   ▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \
10  ▷ ▷ -end \
```

Activities for Combinatorial Objects		
Command	Arguments	Purpose
-save		Save the objects under the name that was assigned automatically.
-save_as	fname	Save the objects under the given name.
-extract_subset	set fname	Extract a subset of the objects.
-line_type_old		Compute the line type of a set in projective space.
-line_type		Compute the line type of a set in projective space.
-conic_type		Compute the conic type of a set in projective space.
-non_conical_type		
-ideal	R	Compute the ideal of the set, using the ring R.
-canonical_form_PG	P options	Requires a projective space P. See Tab. 17.2 for options.
-canonical_form	options	Classification by canonical form. See Tab. 17.2 for options.
-post_processing		Perform post processing for objects after the canonical form has been computed.
-get_combo_with_group	i	Returns the i-th combinatorial object and its group.
-report	options	See Tab. 12.6 for options.
-draw_incidence_matrices	prefix option	Draws the incidence matrices, using the given prefix and the given draw incidence options object.
-test_distinguishing_property	G	Test the distinguishing property for the graph G.
-covering_type	orbits k	Compute the covering numbers of the combinatorial objects (which are sets) w.r.t. the orbits on k subsets.
-filter_by_Steiner_property		When computing the covering type, filter those which have the Steiner property. This means that no k -orbit is covered more than once.
-compute_frequency	graph	
-unpack_from_restricted_action	prefix G	
-line_covering_type	prefix P L	Computes the line type of the lines in L in the projective space P.
-activity	description	Perform an Orbiter activity according to the given description.
-algebraic_degree	P	Compute the algebraic degree of the set inside the given projective space P.

Table 12.5: Activities for Combinatorial Objects

Options for Reporting After Classification		
Command	Arguments	Purpose
-export_flag_orbits		Export flag orbits to csv file.
-show_incidence_matrices		Show incidence matrices in the report.
-show_TDO		Compute the TDO for the report.
-show_TDA		Compute the TDA for the report.
-export_labels		
-export_group_orbiter		Export the automorphism group in Orbiter format.
-export_group_GAP		Export the automorphism group in GAP format.
-export_group_magma		Export the automorphism group in Magma format.
-lex_least	geo	Show lex-least orbit representatives. Here, geo is a geometry builder object.
-canonical_forms		Report the canonical forms of each input object.

Table 12.6: Options for Reporting After Classification

```

11  ▷ ▷ -assign Eqn -from \
12  ▷ ▷ -with R -do \
13  ▷ ▷ ▷ -ring_theoretic_activity \
14  ▷ ▷ ▷ -parse_equation \
15  ▷ ▷ ▷ ▷ "Edge_curve" "Edge\_curve" \
16  ▷ ▷ ▷ ▷ $(EDGE_CURVE_EQUATION_ALGEBRAIC) \
17  ▷ ▷ ▷ ▷ f 3 \
18  ▷ ▷ -end \
19  ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
20  ▷ ▷ -define C -geometric_object P \
21  ▷ ▷ ▷ -projective_variety R \
22  ▷ ▷ ▷ ▷ "Edge_q17" "Edge\_q17" \
23  ▷ ▷ ▷ ▷ Eqn \
24  ▷ ▷ -end \
25  ▷ ▷ -with C -do \
26  ▷ ▷ -combinatorial_object_activity \
27  ▷ ▷ ▷ -line_type_old \
28  ▷ ▷ -end \
29  ▷ ▷ -print_symbols
30

```

The line type is

$$(4^6, 2^{30}, 1^{132}, 0^{139})$$

This means that there are 6 4-secants, 30 2-secants, 132 tangent lines, and 139 external lines to the curve.

Activities for Combinatorial Objects with Group		
Command	Arguments	Purpose
-report	options	Produce a report. See Tab. 12.6 for options.
-get_group		Returns the group of the object as permutation group.

Table 12.7: Activities for Combinatorial Objects with Group

12.4 Combinatorial Objects with Group

In Table 12.7, activities for combinatorial objects with group are listed.

The following Orbiter command demonstrates how we can extract the Desargues configuration from the list of 10_3 configurations (see Section 17.3). The corresponding object type is called `combo` (for combinatorial object). Once extracted, we can write a report on only the Desargues configuration, using an activity of type `-combo_activity`

Code Snippet 633

```

1   geo_10_3.Desargues.configuration-c:
2   ▷ $(ORBITER) -v 10 \
3   ▷ ▷ -define Doinc -draw-incidence-structure-options \
4   ▷ ▷ ▷ -width 60 -width_10 6 \
5   ▷ ▷ -end \
6   ▷ ▷ -define Test_lines -set -loop 4 11 1 -end \
7   ▷ ▷ -define Geo -geometry_builder \
8   ▷ ▷ ▷ -V 10 -B 10 -TDO 3 -fuse 1 \
9   ▷ ▷ ▷ -fname_GEO 10_3 \
10  ▷ ▷ ▷ -test Test_lines \
11  ▷ ▷ ▷ -output_to_inc_file \
12  ▷ ▷ -end \
13  ▷ ▷ -define C -combinatorial_object \
14  ▷ ▷ ▷ -label 10_3 "{\rm 10\\3}" \
15  ▷ ▷ ▷ -file_of_incidence_geometries 10_3.inc 10 10 30 \
16  ▷ ▷ -end \
17  ▷ ▷ -with C -do \
18  ▷ ▷ -combinatorial_object_activity \
19  ▷ ▷ ▷ -canonical_form \
20  ▷ ▷ ▷ ▷ -save_ago \
21  ▷ ▷ ▷ ▷ -save_transversal \
22  ▷ ▷ ▷ ▷ -max_TDO_depth 10 \
23  ▷ ▷ ▷ ▷ -nauty_control -nauty_log "10_3.nauty_log.txt" -end \
24  ▷ ▷ ▷ -end \
25  ▷ ▷ -end \
26  ▷ ▷ -with C -do \
27  ▷ ▷ -combinatorial_object_activity \
28  ▷ ▷ ▷ -post_processing \
29  ▷ ▷ -end \
30  ▷ ▷ -assign Object7 -from \
31  ▷ ▷ -with C -do \
32  ▷ ▷ -combinatorial_object_activity \

```

```

33  ▷ ▷ ▷ -get_combo_with_group 7 \
34  ▷ ▷ -end \
35  ▷ ▷ -with Object7 -do \
36  ▷ ▷ ▷ -combo_activity \
37  ▷ ▷ ▷ ▷ -report \
38  ▷ ▷ ▷ ▷ ▷ -show_TDO \
39  ▷ ▷ ▷ ▷ ▷ -show_TDA \
40  ▷ ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
41  ▷ ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
42  ▷ ▷ ▷ ▷ -end \
43  ▷ ▷ -end \
44  ▷ ▷ -print_symbols
45  ▷ $(PDFLATEX) 10_3_object7_report.tex
46  ▷ $(OPEN) 10_3_object7_report.pdf
47

```

Here is the report of the Desargues configuration produced by this command:

Automorphism Group as Permutation Group

Automorphism group has order: 120

Generators for the automorphism group:

The stabilizer of order 120 is generated by:

$$\begin{aligned}
 g_1 &= (0)(1)(2)(3, 5)(4, 6)(7, 8)(9)(10)(11, 12)(13, 14)(15, 16)(17)(18)(19) \\
 g_2 &= (0)(1, 3)(2, 4)(5)(6)(7)(8, 9)(10, 11)(12)(13)(14, 17)(15)(16, 18)(19) \\
 g_3 &= (0)(1, 2)(3, 4)(5, 6)(7)(8)(9)(10)(11)(12)(13, 15)(14, 16)(17, 18)(19) \\
 g_4 &= (0, 1)(2)(3)(4, 7)(5)(6, 8)(9)(10)(11, 13)(12, 14)(15)(16)(17)(18, 19)
 \end{aligned}$$

0, 1, 2, 5, 6, 3, 4, 8, 7, 9, 10, 12, 11, 14, 13, 16, 15, 17, 18, 19,
 0, 3, 4, 1, 2, 5, 6, 7, 9, 8, 11, 10, 12, 13, 17, 15, 18, 14, 16, 19,
 0, 2, 1, 4, 3, 6, 5, 7, 8, 9, 10, 11, 12, 15, 16, 13, 14, 18, 17, 19,
 1, 0, 2, 3, 7, 5, 8, 4, 6, 9, 10, 13, 14, 11, 12, 15, 16, 17, 19, 18,

#Generators in GAP format are:

```

G := Group([(4, 6)(5, 7)(8, 9)(12, 13)(14, 15)(16, 17),
(2, 4)(3, 5)(9, 10)(11, 12)(15, 18)(17, 19),
(2, 3)(4, 5)(6, 7)(14, 16)(15, 17)(18, 19),
(1, 2)(5, 8)(7, 9)(12, 14)(13, 15)(19, 20)]);

```

Incidence structure by flags:

(0, 1, 2, 10, 13, 14, 20, 25, 26, 31, 33, 37, 41, 45, 48, 52, 54, 57, 62, 66, 68, 73, 75, 79, 84, 86, 89,
 97, 98, 99)

TDO

Decomposition by combinatorial refinement (TDO):

The decomposition scheme has size 1×1

$$\begin{array}{c|c} \rightarrow & 10_1 \\ \hline 10_0 & 3 \end{array}$$

$$\begin{array}{c|c} \downarrow & 10_1 \\ \hline 10_0 & 3 \end{array}$$

V0 of size 10 contains 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

B0 of size 10 contains 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Incidence matrix not shown. Use option `-show_incidence_matrices` to show incidence matrix.

TDA

Automorphism group has order: 120

Decomposition by automorphism group:

The decomposition scheme has size 1×1

$$\begin{array}{c|c} \rightarrow & 10_1 \\ \hline 10_0 & 3 \end{array}$$

$$\begin{array}{c|c} \downarrow & 10_1 \\ \hline 10_0 & 3 \end{array}$$

V0 of size 10 contains 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

B0 of size 10 contains 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Incidence matrix not shown. Use option `-show_incidence_matrices` to show incidence matrix.

Canonical labeling

Canonical labeling:

canonical row = 3

canonical orbit number = 0

Row sets of the encoded object:

$\{ 0, 1, 2 \} = 0,$

$\{ 7, 8, 9 \} = 119,$

$\{ 3, 5, 9 \} = 93,$

$\{ 4, 6, 9 \} = 106,$

$\{ 0, 3, 4 \} = 15,$

$\{ 0, 5, 6 \} = 26,$

$\{ 2, 6, 7 \} = 79,$

$\{ 1, 5, 7 \} = 55,$

$\{ 2, 4, 8 \} = 73,$

$\{ 1, 3, 8 \} = 47$

Column sets of the encoded object:

$\{ 0, 4, 5 \} = 21,$

$\{ 0, 7, 9 \} = 34,$

$\{ 0, 6, 8 \} = 31,$

$\{ 2, 4, 9 \} = 74,$

$\{ 3, 4, 8 \} = 88,$

$\{ 2, 5, 7 \} = 76,$

$\{ 3, 5, 6 \} = 90,$

$\{ 1, 6, 7 \} = 58,$

$$\{ 1, 8, 9 \} = 63,$$

$$\{ 1, 2, 3 \} = 36$$

Orbits on Flags and Anti-Flags

Flag orbits:

orbit length : number of orbits of that length:

$$30 \quad 1$$

Anti-Flag orbits:

orbit length : number of orbits of that length:

$$10 \quad 1$$

$$60 \quad 1$$

12.5 Combinatorial Linear Spaces

A linear space is a pair $(S, \mathcal{L})$ where S is a set and $\mathcal{L}$ is a set of subsets of S such that each set $L \in \mathcal{L}$ satisfies $|L| \geq 2$ and moreover for any two $a, b \in S$ there is exactly one element $L \in \mathcal{L}$ such that both a and b belong to L . The usual notions of isomorphism and automorphism apply. For finite linear spaces, a first combinatorial property is the number a_i which counts the number of sets $L \in \mathcal{L}$ of size i . The vector $(a_2, \dots, a_n)$ is the line type of $(S, \mathcal{L})$. The equation

$$\binom{n}{2} = \sum_{j=2}^n a_j \binom{j}{2} \quad (12.1)$$

is satisfied. The equation can be used to generate all possible line types of a putative linear space. Here is an example. For $|S|=6$, (12.1) becomes

$$x_0 \binom{6}{2} + x_1 \binom{5}{2} + x_2 \binom{4}{2} + x_3 \binom{3}{2} + x_4 \binom{2}{2} = \binom{6}{2}.$$

Here, $(x_0, x_1, \dots, x_4)$ is the line type of a putative linear space on 6 points. That is, $x_i = a_{6-i}$ is the number of lines of size $6-i$. The extended coefficient matrix of the system is

$$[15 \ 10 \ 6 \ 3 \ 1 \mid 15].$$

The Orbiter command

Code Snippet 634

```
1  linsp6:
2  ▷ $(ORBITER) -v 4 \
3  ▷ ▷ -define A -vector -format 1 -dense "15,10,6,3,1" -end \
4  ▷ ▷ -define D -diophant -label linsp6 \
5  ▷ ▷ -coefficient_matrix A \
6  ▷ ▷ -RHS "mult=1,EQ=15" \
7  ▷ ▷ -x_min_global 0 \
8  ▷ ▷ -x_max_global 15 \
9  ▷ ▷ -end \
10 ▷ ▷ -with D -do \
11 ▷ ▷ ▷ -diophant_activity -solve_mckay \
12 ▷ ▷ -end
13 ▷
14 # Found 15 solutions with 22 backtrack steps
15
```

solves the system using McKay's program possolve [56]. The program finds 15 solutions, written to the file `linsp6.sol`.

Let us consider a problem from [14]. Suppose we are interested in linear spaces on 30 points and with line type $(7, 5^{27}, 4^{24})$. This notation means that we assume one 7-line, 27 5-lines and 24 4-lines. The type of a point P is the set of integers

$$p_j = \#j\text{-lines through } P.$$

We are trying to precompute the matrix of point types

$$(p_{ij})$$

where $j = 7, 5, 4$ and i belongs to an index set of all possible point types. Fixing a point P , counting points $Q \neq P$ collinear with P yields

$$6p_7 + 4p_5 + 3p_4 = 29, \quad p_7 \leq 1, \quad p_5 \leq 27, \quad p_4 \leq 24.$$

Using the Orbiter commands

Code Snippet 635

```

1   linsp30_pt_types:
2   ▷ $(ORBITER) -v 4 \
3   ▷ ▷ -define A -vector -format 1 -dense "6,4,3" -end \
4   ▷ ▷ -define D -diophant \
5   ▷ ▷ ▷ -label linsp30_pt_types \
6   ▷ ▷ ▷ -coefficient_matrix A \
7   ▷ ▷ ▷ -RHS "mult=1,EQ=29" -x_bounds "0,1,0,27,0,24" \
8   ▷ ▷ -end \
9   ▷ ▷ -with D -do \
10  ▷ ▷ ▷ -diophant_activity -solve_mckay \
11  ▷ ▷ -end
12

```

we determine the possibilities

$$(p_7, p_5, p_4) = \begin{cases} 1 & 5 & 1 \\ 1 & 2 & 5 \\ 0 & 5 & 3 \\ 0 & 2 & 7 \end{cases}$$

The rows in this matrix are called the point types ($i = 0, 1, 2, 3$). Let b_i be the number of points of type i . By counting points, incident (point, line) pairs by j -lines and pairs of intersecting j -lines, we arrive at the following system:

$$\begin{aligned}
b_0 + b_1 + b_2 + b_3 &= 30 \\
b_0 + b_1 &= 7 \\
5b_0 + 2b_1 + 5b_2 + 2b_3 &= 135 = 27 \cdot 5 \\
b_0 + 5b_1 + 3b_2 + 7b_3 &= 96 = 24 \cdot 4 \\
10b_0 + b_1 + 10b_2 + b_3 &\leq 351 = \binom{27}{2} \\
10b_1 + 3b_2 + 21b_3 &\leq 276 = \binom{24}{2}
\end{aligned}$$

Using the Orbiter commands

Code Snippet 636

```

1   linsp30_pt_distribution:
2   ▷ $(ORBITER) -v 4 \
3   ▷ ▷ -define A -vector -format 6 -dense \
4   ▷ ▷ ▷ "1,1,1,1,1,1,0,0,5,2,5,2,1,5,3,7,10,1,10,1,0,10,3,21" \
5   ▷ ▷ -end \
6   ▷ ▷ -define D -diophant \
7   ▷ ▷ ▷ -label linsp30_pt_distribution \
8   ▷ ▷ ▷ -coefficient_matrix A \
9   ▷ ▷ ▷ -RHS "mult=1,EQ=30" \
10  ▷ ▷ ▷ -RHS "mult=1,EQ=7" \
11  ▷ ▷ ▷ -RHS "mult=1,EQ=135" \
12  ▷ ▷ ▷ -RHS "mult=1,EQ=96" \
13  ▷ ▷ ▷ -RHS "mult=1,LE=351" \
14  ▷ ▷ ▷ -RHS "mult=1,LE=276" \

```

```

15  ▷ ▷ ▷ -x_min_global 0 -x_max_global 30 \
16  ▷ ▷ -end \
17  ▷ ▷ -with D -do \
18  ▷ ▷ ▷ -diophant_activity -solve_mckay \
19  ▷ ▷ -end \
20  ▷ ▷ -with D -do \
21  ▷ ▷ ▷ -diophant_activity -draw_as_bitmap 20 8 \
22  ▷ ▷ -end
23

```

we determine the possibilities

$$(b_0, b_1, b_2, b_3) = \begin{cases} 2 & 5 & 23 & 0 \\ 3 & 4 & 22 & 1 \\ 4 & 3 & 21 & 2 \\ 5 & 2 & 20 & 3 \\ 6 & 1 & 19 & 4 \\ 7 & 0 & 18 & 5 \end{cases}$$

12.6 Classification of Configurations and Geometries

A partial linear space is a set system on a fixed set V . We write $\mathcal{L} = (V, \mathcal{B})$, where $\mathcal{B}$ is a set of distinct subsets of V , called lines. The members of $V \cup \mathcal{B}$ are called elements. For two elements x, y , we say that x is incident with y , written xIy , if either $x \in y$ or $y \in x$. We require that any line has at least two points and that any two points are contained in at most one line. A *decomposition* of a linear space is a partition $\Pi = (C_1, \dots, C_n)$ of $V \cup \mathcal{B}$ such that each C_i either is a subset of V or a subset of $\mathcal{B}$. A decomposition is called *tactical* if for all i , the incidence number

$$\iota(C_i, C_j) = \#\{y \in C_j, xIy\}$$

does not depend on the choice of $x \in C_i$. Any linear space has a tactical decomposition, as the discrete partition (every element is in its own class) is tactical. Let $\text{Aut}(\mathcal{L})$ be the automorphism group of the linear space, which is the subgroup of $\text{Sym}(V)$ which preserves incidence. For $\alpha \in \text{Aut}(\mathcal{L})$, we say that the decomposition Π preserves α if α fixes every class of Π . For $A \leq \text{Aut}(\mathcal{L})$, we say that Π preserves A if Π preserves every element $\alpha \in A$. Mostly, we are interested in those decompositions Π which preserve $\text{Aut}(\mathcal{L})$. In light of this, the discrete decomposition is not that interesting.

Any linear space has a coarsest tactical decomposition that preserves its automorphism group: The orbit partition of the automorphism group acting on $V \cup \mathcal{B}$ will do. Up to ordering of the classes, the coarsest tactical refinement is unique. Computing the orbit decomposition is challenging as it involves computing the automorphism group. Computationally, there are easier ways to get to admissible decompositions. One way is by means of successive refinements. If a class C_i does not have the property that $\iota(C_i, C_j)$ is well-defined for all $x \in C_i$, then a refinement of C_i will do. The coarsest refinement of C_i has the property that if C_i preserves some group A then the refinement will do, too. This shows that there is an algorithm to compute a tactical decomposition of any given linear space $\mathcal{P}$. Simply start with the decomposition of two classes, one the set of points and one the set of blocks, and refine. The output may or may not be equal to the decomposition arising from the orbit partition of $\text{Aut}(\mathcal{L})$.

Let us consider the opposite question. Given a tactical decomposition, can we classify the linear spaces which admit a given tactical decomposition. The `-geometry_builder` option can help. Table 12.8 shows the options for the geometry builder.

The command

Code Snippet 637

```

1  geo_10_3:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define Test_lines -set -loop 4 11 1 -end \
4  ▷ ▷ -define Geo -geometry_builder \
5  ▷ ▷ ▷ -V 10 -B 10 -TDO 3 -fuse 1 \
6  ▷ ▷ ▷ -fname_GEO 10_3 \
7  ▷ ▷ ▷ -output_to_inc_file \
8  ▷ ▷ ▷ -output_to_sage_file \
9  ▷ ▷ ▷ -output_to_blocks_file \
10 ▷ ▷ ▷ -test Test_lines \
11 ▷ ▷ ▷ -save_canonical_forms \
12 ▷ ▷ -end
13
```

classifies the configurations 10_3 . It uses isomorphism tests after 4, 5, 6, 7, 8, 9 and 10 points. The positions of the tests is defined using a set called `Test_lines`. The set of test lines is defined using a loop command. The command shows that there are exactly 10 configurations of this kind. One of them is the Desargues configuration. Four different output files can be written. Each contains all geometries, but the file format is different.

Constructing and Classifying Geometries		
Command	Arguments	Purpose
-V	part	The initial partition of points (rows).
-B	part	The initial partition of blocks (columns).
-TDO	tdo	The initial row-tactical decomposition scheme.
-fuse	fuse	The partition of row classes.
-girth_test	g	Require the girth of the collinearity graph to be at least g .
-lambda	λ	Set λ for two-design test. Every pair of points is required to lie in λ blocks.
-find_square		Enable the test for linear spaces.
-simple		Construct simple designs only (needs -lambda)
-search_tree		Write a file containing the search tree (at the level of rows of the partial geometry).
-search_tree_flags		Write a file containing the search tree (at the level of flags of the partial geometry). A flag is an incident point-block pair.
-orderly		Apply orderly generation.
-special_test_not_orderly		Apply the special test. This option only applies to generation which is not orderly.
-test_lines	lines	Install isomorphism checking at the given set of lines. Lines must be an object of type vector or set.
-test2_lines	lines	Install isomorphism checking of the second type at the given set of lines. Lines must be an object of type vector or set.
-split	$l \ r \ m$	At line l , proceed only with the cases whose number is congruent to r modulo m . This allows splitting the search tree into smaller subtrees.
-print_at_line	lines	Print the geometries considered at the given set of lines.
-fname_GEO	fname	Set the output file name base (no extension).
-output_to_inc_file		Set output to inc file.
-output_to_sage_file		Set output to sage file.
-output_to_blocks_file		Set output to a file containing the blocks in coded form.
-output_to_blocks_latex_file		Set output to a file containing the blocks in latex.
-save_canonical_forms		Saves the canonical forms to a csv file.

Table 12.8: Constructing and Classifying Geometries

1. The option `-output_to_inc_file` writes `10_3.inc`. The file contains the incidences in increasing order. The position in the incidence matrix is given. Apart from the header and footer, each row in the file describes exactly one geometry. The incidences are given in numeric form. Each incidence is the numerical position of the point/block pair in the incidence matrix. The position is the numbering of the matrix entries in the incidence matrix in row-major ordering, starting with zero for the top left entry. The index of the incidence in row i (zero-based) and column j is $b \cdot i + j$, where b is the number of blocks in the geometry. The header consists of one row, containing the number of points, the number of lines, and the number of incidences. The footer consists of two rows. A single `-1` indicates the beginning of the footer. The second row in the footer summarizes the automorphism group orders of the geometries. Here is an example: The file `10_3.inc` lists the 10 configurations 10_3 :

```

10 10 30
0 1 2 10 13 14 20 25 26 31 33 35 41 44 47 52 53 58 62 66 69 74 78 79 85 87 89 96 97 98
0 1 2 10 13 14 20 25 26 31 33 35 41 44 47 52 54 58 62 66 69 73 78 79 85 87 89 96 97 98
0 1 2 10 13 14 20 25 26 31 33 35 41 44 47 52 54 58 62 67 69 73 76 79 85 88 89 96 97 98
0 1 2 10 13 14 20 25 26 31 33 35 41 44 47 52 56 58 62 67 69 73 78 79 84 86 89 95 97 98
0 1 2 10 13 14 20 25 26 31 33 35 41 47 48 52 54 57 62 66 68 73 77 79 84 86 89 95 98 99
0 1 2 10 13 14 20 25 26 31 33 35 41 47 48 52 54 57 62 66 68 73 78 79 84 86 89 95 97 99
0 1 2 10 13 14 20 25 26 31 33 35 41 47 48 52 54 57 62 66 69 73 78 79 84 86 88 95 97 99
0 1 2 10 13 14 20 25 26 31 33 37 41 45 48 52 54 57 62 66 68 73 75 79 84 86 89 97 98 99
0 1 2 10 13 14 20 25 26 31 33 37 41 45 48 52 54 57 62 66 68 73 75 79 84 88 89 96 97 99
-1 10
120, 24, 12, 10, 6, 4^2, 3^2, 2

```

2. The option `-output_to_sage_file` writes `10_3.sage`. This file is meant to be read by Sage [69].
3. The option `-output_to_blocks_file` writes `10_3.blocks`. Here is the content of the file `10_3.blocks`:

```

10 10 3
0 15 26 44 51 68 81 109 114 116
0 15 26 46 49 68 81 109 114 116
0 15 26 46 49 68 83 106 115 116
0 15 26 46 52 69 77 106 114 116
0 15 26 46 56 69 80 101 106 119
0 15 26 46 56 69 80 103 104 119
0 15 26 46 56 69 80 103 107 117
0 15 26 46 56 72 80 93 106 119
0 15 26 46 56 72 81 93 105 119
0 15 26 46 56 74 79 93 105 119
-1 10
120, 24, 12, 10, 6, 4^2, 3^2, 2

```

contains the blocks. Each number represents the rank of a 3-subset corresponding to a block in the lexicographic ordering of all 3-subsets.

4. The option `-output_to_blocks_latex_file` writes `10_3.blocks_long`. The file `10_3.blocks_long` contains a list of all blocks written out in a format ready for use in latex.

It is possible to create graphical representations of the search tree. The additional option `-search_tree` can be given, as shown in the following command:

Code Snippet 638

```

1   geo_10_3_tree:
2   ▷ $(ORBITER) -v 20 \
3   ▷ ▷ -define Test_lines -set -loop 0 11 1 -end \
4   ▷ ▷ -define GEO -geometry_builder \
5   ▷ ▷ ▷ -V 10 -B 10 -TDO 3 -fuse 1 \
6   ▷ ▷ ▷ -fname_GEO 10_3 \
7   ▷ ▷ ▷ -output_to_inc_file \
8   ▷ ▷ ▷ -search_tree \
9   ▷ ▷ ▷ -test Test_lines \
10  ▷ ▷ -end
11  ▷ $(ORBITER) -v 20 \
12  ▷ ▷ -define Do -draw_options -embedded -radius 40 \
13  ▷ ▷ ▷ -paperheight 220 \
14  ▷ ▷ ▷ -paperwidth 330 \
15  ▷ ▷ ▷ -xin 10000 -yin 10000 \
16  ▷ ▷ ▷ -xout 1000000 -yout 500000 \
17  ▷ ▷ ▷ -scale 2 -line_width 0.3 \
18  ▷ ▷ ▷ -nodes_empty \
19  ▷ ▷ -end \
20  ▷ ▷ -tree_draw \
21  ▷ ▷ ▷ -file 10_3_tree.txt \
22  ▷ ▷ ▷ -draw_options Do \
23  ▷ ▷ -end
24  ▷ $(PDFLATEX) 10_3_tree_draw.tex
25  ▷ $(OPEN) 10_3_tree_draw.pdf
26

```

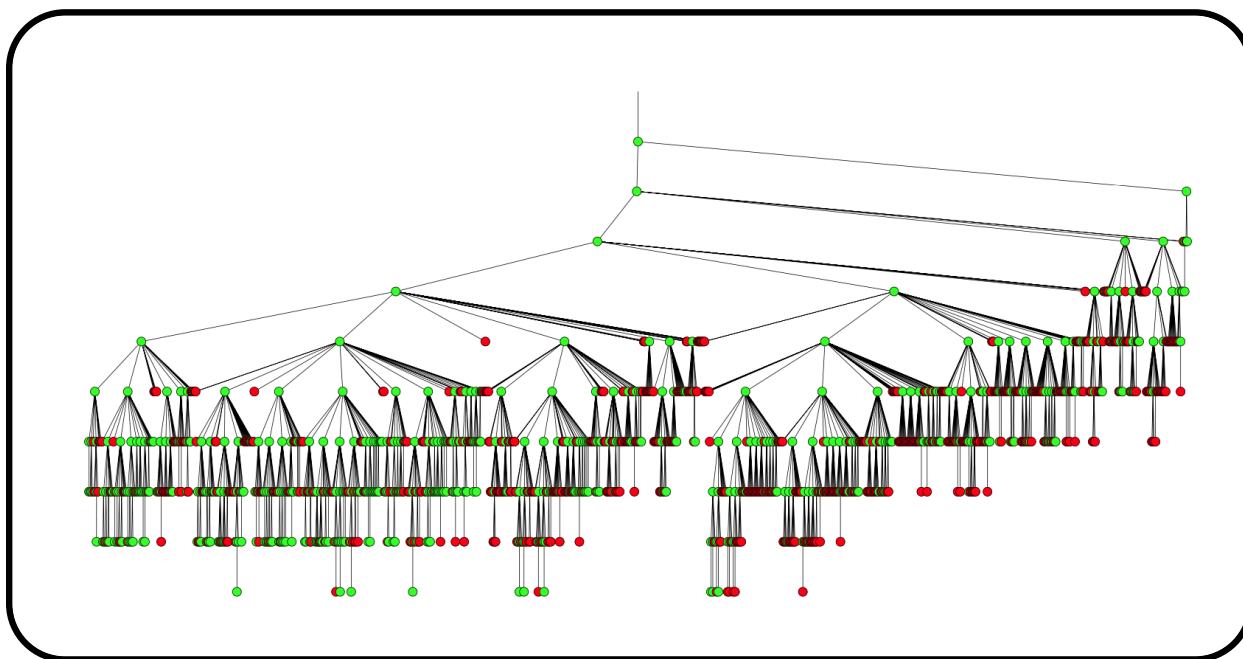
The `-search_tree` option causes Orbiter to create a file containing the search tree. The name of the file is derived from the file name given with the `fname_GEO` option. Here, the `fname_GEO` option sets the output file to `10_3`. The `-search_tree` option then creates the file `10_3_tree.txt`. Once the classification is completed, the `-tree_draw` command can be used to create a drawing of the search tree from the file just created. The vertex color represents the outcome of the isomorphism test. A green node is accepted. A red node is rejected. The search will continue for the green nodes only. A green node at the bottom of the tree corresponds to an isomorphism type of a geometry satisfying all the requirements. In the example of the configurations 10_3 , there are 10 green nodes at the very bottom of the search tree. They represent the 10 isomorphism types of configurations 10_3 . The size of the tree can be determined by counting the lines of the file `10_3_tree.txt` and subtracting one:

```
wc 10_3_tree.txt
```

The word count command yields the following output:

```
1471  13543  37633 10_3_tree.txt
```

This means that there are 1471 lines in the file. Hence the search tree has 1470 nodes. The resulting tree is shown below



Any incidence structure defines a graph on its underlying set of points. The vertices of the collinearity graph are the points of the incidence structure. Two vertices are adjacent if the incidence structure contains a block which contains the associated points. The distance between two points in the geometry is the distance of the associated vertices in the collinearity graph. The girth is the length of the shortest cycle. Incidence structures with high girth are often interesting. Orbiter allows classifying incidence structures with a given girth. More specifically, Orbiter classifies all incidence structures whose girth is at least a given number, or proves that there are none. This is interesting because the girth can be used as a filter, and classifying with a given girth may be faster than classifying all incidence geometries. For instance, the following example classifies all cubic graphs on 10 vertices with girth at least 5:

Code Snippet 639

```

1   geo.petersen:
2   ▷ $(ORBITER) -v 8 \
3   ▷ ▷ -define Test_lines -set -loop 3 11 1 -end \
4   ▷ ▷ -define Geo -geometry_builder \
5   ▷ ▷ ▷ -V 10 -B 15 -TDO 3 -fuse 1 \
6   ▷ ▷ ▷ -fname_GEO petersen -girth 5 \
7   ▷ ▷ ▷ -output_to_inc_file \
8   ▷ ▷ ▷ -search_tree \
9   ▷ ▷ ▷ -test Test_lines \
10  ▷ ▷ -end
11

```

There is a unique graph with these properties. It is the Petersen graph. Its automorphism group is $\text{Sym}(5)$ of order 120.

Let us look at the girth of configurations. For instance, while there are 245342 isomorphism classes of configurations 15_3 , only one of them has girth 4. This is the Cremona Richmond configuration with automorphism group $\text{Sym}(6)$ of order 720. It is associated to a cubic surface. Let us compare the classification with and without girth condition. The following command classifies all configurations 15_3 :

Code Snippet 640

```

1    15_3.inc:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define Test_lines -set -loop 4 16 1 -end \
4    ▷ ▷ -define Geo -geometry_builder \
5    ▷ ▷ ▷ -V 15 -B 15 -TDO 3 \
6    ▷ ▷ ▷ -fuse 1 -fname_GEO 15_3 \
7    ▷ ▷ ▷ -output_to_inc_file \
8    ▷ ▷ ▷ -test Test_lines \
9    ▷ ▷ ▷ -special_test_not_orderly \
10   ▷ ▷ -end
11

```

The next command classifies the configurations 15_3 with girth 4.

Code Snippet 641

```

1    geo_15_3.g4:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define Test_lines -set -loop 4 16 1 -end \
4    ▷ ▷ -define Geo -geometry_builder \
5    ▷ ▷ ▷ -V 15 -B 15 -TDO 3 \
6    ▷ ▷ ▷ -fuse 1 -fname_GEO 15_3.g4 \
7    ▷ ▷ ▷ -output_to_inc_file \
8    ▷ ▷ ▷ -girth 4 \
9    ▷ ▷ ▷ -search_tree \
10   ▷ ▷ ▷ -special_test_not_orderly \
11   ▷ ▷ ▷ -test Test_lines \
12   ▷ ▷ -end
13   ▷ $(ORBITER) -v 2 \
14   ▷ ▷ -define Do -draw_options -embedded -radius 50 \
15   ▷ ▷ ▷ -nodes_empty \
16   ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
17   ▷ ▷ -tree_draw \
18   ▷ ▷ ▷ -file 15_3.g4_tree.txt \
19   ▷ ▷ ▷ -draw_options Do \
20   ▷ ▷ -end
21   ▷ $(PDFLATEX) 15_3.g4_tree_draw.tex
22   ▷ $(OPEN) 15_3.g4_tree_draw.pdf
23

```

The next command classifies the configurations 40_4 with girth 4. Exactly two configuration arise, both with a group of order 51840. Note the extra option `-special_test_not_orderly` to speed up the search.

Code Snippet 642

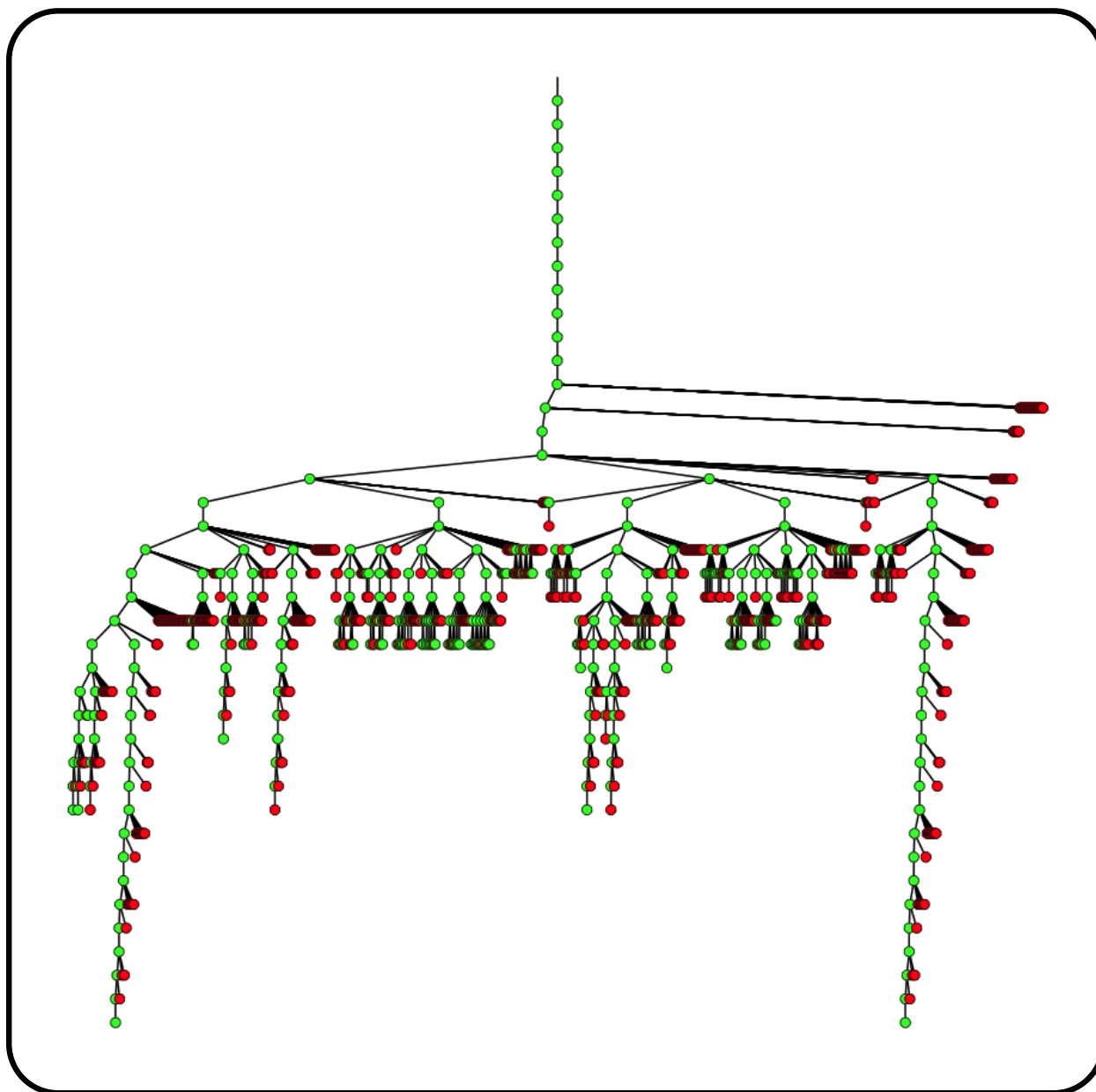
```

1    geo_40_4.g4:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define Test_lines -set -loop 0 41 1 -end \
4    ▷ ▷ -define Geo -geometry_builder \
5    ▷ ▷ ▷ -V 40 -B 40 -TDO 4 \

```

```
6    ▷ ▷ ▷ -fuse 1 \  
7    ▷ ▷ ▷ -fname_GEO 40_4.g4 \  
8    ▷ ▷ ▷ -girth 4 \  
9    ▷ ▷ ▷ -search_tree \  
10   ▷ ▷ ▷ -special_test_not_orderly \  
11   ▷ ▷ ▷ -test Test_lines \  
12   ▷ ▷ ▷ -output_to_sage_file \  
13   ▷ ▷ ▷ -output_to_inc_file \  
14   ▷ ▷ ▷ -save_canonical_forms \  
15   ▷ ▷ -end  
16   ▷ $(ORBITER) -v 2 \  
17   ▷ ▷ -define Do -draw_options -embedded -radius 50 \  
18   ▷ ▷ ▷ -xin 10000 -yin 10000 \  
19   ▷ ▷ ▷ -xout 1000000 -yout 1000000 \  
20   ▷ ▷ ▷ -nodes_empty \  
21   ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \  
22   ▷ ▷ -tree_draw \  
23   ▷ ▷ ▷ -file 40_4.g4_tree.txt \  
24   ▷ ▷ ▷ -draw_options Do \  
25   ▷ ▷ -end  
26   ▷ $(PDFLATEX) 40_4.g4_tree_draw.tex  
27   ▷ $(OPEN) 40_4.g4_tree_draw.pdf  
28
```

The search tree is shown below



The next command classifies the configurations 63_3 with girth 6. Exactly two configuration arise, both with a group of order 12096. Note the extra option `-special_test_not_orderly` to speed up the search.

Code Snippet 643

```

1  geo_63_3.g6:
2  ▷ $(ORBITER) -v 1 \
3  ▷ ▷ -define Test_lines -set -loop 4 64 1 -end \
4  ▷ ▷ -define Geo -geometry_builder \
5  ▷ ▷ ▷ -V 63 -B 63 -TDO 3 \
6  ▷ ▷ ▷ -fuse 1 -fname_GEO 63_3.g6 \
7  ▷ ▷ ▷ -girth 6 \
8  ▷ ▷ ▷ -test Test_lines \

```

```

9      ▷ ▷ ▷ -special_test_not_orderly \
10     ▷ ▷ ▷ -output_to_sage_file \
11     ▷ ▷ ▷ -output_to_inc_file \
12     ▷ ▷ ▷ -search_tree \
13     ▷ ▷ ▷ -save_canonical_forms \
14     ▷ ▷ -end
15

```

The configuration is also known as the split Cayley hexagon.

The next command classifies the Latin squares of order 6.

Code Snippet 644

```

1      geo.LSQ6.inc:
2      ▷ $(ORBITER) -v 1 \
3      ▷ ▷ -define Test_lines -set -loop 1 19 1 -end \
4      ▷ ▷ -define Geo -geometry_builder \
5      ▷ ▷ ▷ -V 6,6,6 -B 1,1,1,36 -TDO \
6      ▷ ▷ ▷ "1,0,0,0,6, 0,1,0,6, 0,0,1,6" \
7      ▷ ▷ ▷ -fuse 3 \
8      ▷ ▷ ▷ -fname_GEO LSQ6 \
9      ▷ ▷ ▷ -output_to_inc_file \
10     ▷ ▷ ▷ -test Test_lines \
11     ▷ ▷ -end
12

```

Up to isomorphism, there are exactly 12 such objects.

The next command classifies the nonexistence of a pair of orthogonal Latin squares of order 6 (Euler's problem).

Code Snippet 645

```

1      geo.MOL6.inc:
2      ▷ $(ORBITER) -v 1 \
3      ▷ ▷ -define Test_lines -set -loop 1 25 1 -end \
4      ▷ ▷ -define Geo -geometry_builder \
5      ▷ ▷ ▷ -V 6,6,6,6 -B 1,1,1,1,36 -TDO \
6      ▷ ▷ ▷ "1,0,0,0,0,6, 0,1,0,0,6, 0,0,1,0,6, 0,0,0,1,6" \
7      ▷ ▷ ▷ -fuse 4 \
8      ▷ ▷ ▷ -fname_GEO MOL6 \
9      ▷ ▷ ▷ -output_to_inc_file \
10     ▷ ▷ ▷ -test Test_lines \
11     ▷ ▷ -end
12

```

No geometry is found, proving that no MOL(6) exists.

Refining Tactical Decompositions		
Command	Arguments	Purpose
-lambda3	λ_3 s	Refine as 3-design with λ_3 and with block-size s
-solution	i fname	Use solutions to system i from file fname.
-range	f l	Refine cases i with $f \leq i < f + l$ only.
-select	label	Select the case for refinement by label.
-o1	s	Omit s variables from the first refinement system.
-o2	s	Omit s variables from the second refinement system.
-D1_upper_bound_x0	b	Add the bound $x_0 \leq b$ in the first refinement.
-reverse		Sort the distributions in reverse order.
-reverse_inverse		
-nopacking		Do not use packing inequalities.
-dual_is_linear_space		Assume that the dual incidence structure is a linear space also. This is valid for projective planes, for instance.
-geometric_test		Subject the distributions to the geometric test.
-once		Find at most one refinement in each case. This can be used to test which cases can be refined.
-mckay		Use McKay's solver instead (by default, a lexicographic solver is used).
-input_file	fname	Specify the input TDO-file for refinement.

Table 12.9: Refining Tactical Decompositions

12.7 Tactical Decompositions

Table 12.9 lists the Orbiter commands for decomposition refinement.

Suppose we want to study projective planes of order 16. It is a linear space with $16^2 + 16 + 1 = 273$ points and equally many lines. Each point lies on 17 lines and each line contains 17 points. Any two points lie on exactly one line and any two lines intersect in exactly one point.

We decide to study maximal arcs of degree 4 in this plane (the degree has to divide the order of the plane). A maximal arc of degree d is a set of points so that each line intersects in either d or zero points. A line which intersects in d points is called a secant. A line which intersects in no point is called an external line. The command

Code Snippet 646

```

1    max_arc_16_4.start:
2    ▷ $(ORBITER) -v 4 -maximal_arc_parameters 16 4
3
```

creates a decomposition stack for the parameters of the arc and writes the file max_arc_q16_r4.stack

```

<HTDO type=pt ptanz=2 btanz=2 fuse=simple>
      221      52
    52      17      0
    221      13      4

    1  1
</HTDO>

```

This is a point-tactical decomposition with 2 point-classes and 2 block-classes. The point classes are associated with the rows. The block-classes are associated with the columns. The first row and column indicates the size of the classes. The entries a_{ij} count the number of blocks in the column class j that are incident with a given point in the i th row class. The fuse information at the bottom (1 1) is a partition of the row classes which indicates the ancestor decomposition which was column tactical. The next step is to convert the stack file to a tdo file. The command

Code Snippet 647

```

1  max_arc_16_4.convert_stack_tdo:
2  ▷ $(ORBITER) -v 4 -convert_stack_to_tdo max_arc_q16_r4.stack
3

```

does that. It creates the file max_arc_q16_r4.tdo. It also prints the decomposition stack:

```

lambda_scheme at level 2 :
is 1 x 1
      | 273_{ 1}
=====
273_{ 0} |

row_scheme at level 4 :
is 2 x 2
      | 221_{ 1} 52_{ 2}
=====
52_{ 0} |      17      0
221_{ 3} |      13      4

col_scheme at level 3 :
is 1 x 2
      | 221_{ 1} 52_{ 2}
=====
273_{ 0} |      17      17

```

Next, we can compute all coarsest column-tactical refinements of the decomposition. To this end, the command

Code Snippet 648

```

1  max_arc_16_4.refine:
2  ▷ $(ORBITER) -v 4 -tdo_refinement \
3  ▷ ▷ -input_file max_arc_q16_r4.tdo -dual_is_linear_space -end
4  ▷

```



```

5    max_arc_16_4r_print:
6    ▷ $(ORBITER) -v 4 -tdo_print max_arc.q16_r4r.tdo
7

```

is used. Because the incidence structure is a projective plane, the dual is a linear space also. Hence the option `-dual_is_linear_space` can be used, which is helpful to reduce possibilities. As it turns out, there is exactly one refinement, and it is tactical. The file `max_arc.q16_r4r.tdo` is produced. Note the added letter `r` at the end of the file name (`r` for refinement). We can use the following command to display the decomposition stack in the file:

Code Snippet 649

```

1    max_arc_16_4r_print:
2    ▷ $(ORBITER) -v 4 -tdo_print max_arc.q16_r4r.tdo
3

```

This produces the following output:

```

decomposition 0.1:
lambda_scheme at level 2 :
is 1 x 1
      | 273_{ 1}
=====
273_{ 0} |

row_scheme at level 4 :
is 2 x 2
      | 221_{ 1} 52_{ 2}
=====
52_{ 0} |      17      0
221_{ 3} |      13      4

col_scheme at level 4 :
is 2 x 2
      | 221_{ 1} 52_{ 2}
=====
52_{ 0} |      4      0
221_{ 3} |      13     17

extra_col_scheme at level 3 :
is 1 x 2
      | 221_{ 1} 52_{ 2}
=====
273_{ 0} |      17     17

```

Activities for a Plesken Ring		
Command	Arguments	Purpose
-report	options	Produce a latex report. For options, see Table 8.7.
-evaluate_join	ring formula	Evaluate the given formula in the Plesken ring $A^\vee$. Use the given polynomial ring to map symbols to orbits.
-evaluate_meet	ring formula	Evaluate the given formula in the Plesken ring $A^\wedge$. Use the given polynomial ring to map symbols to orbits.

Table 12.10: Activities for a Plesken Ring

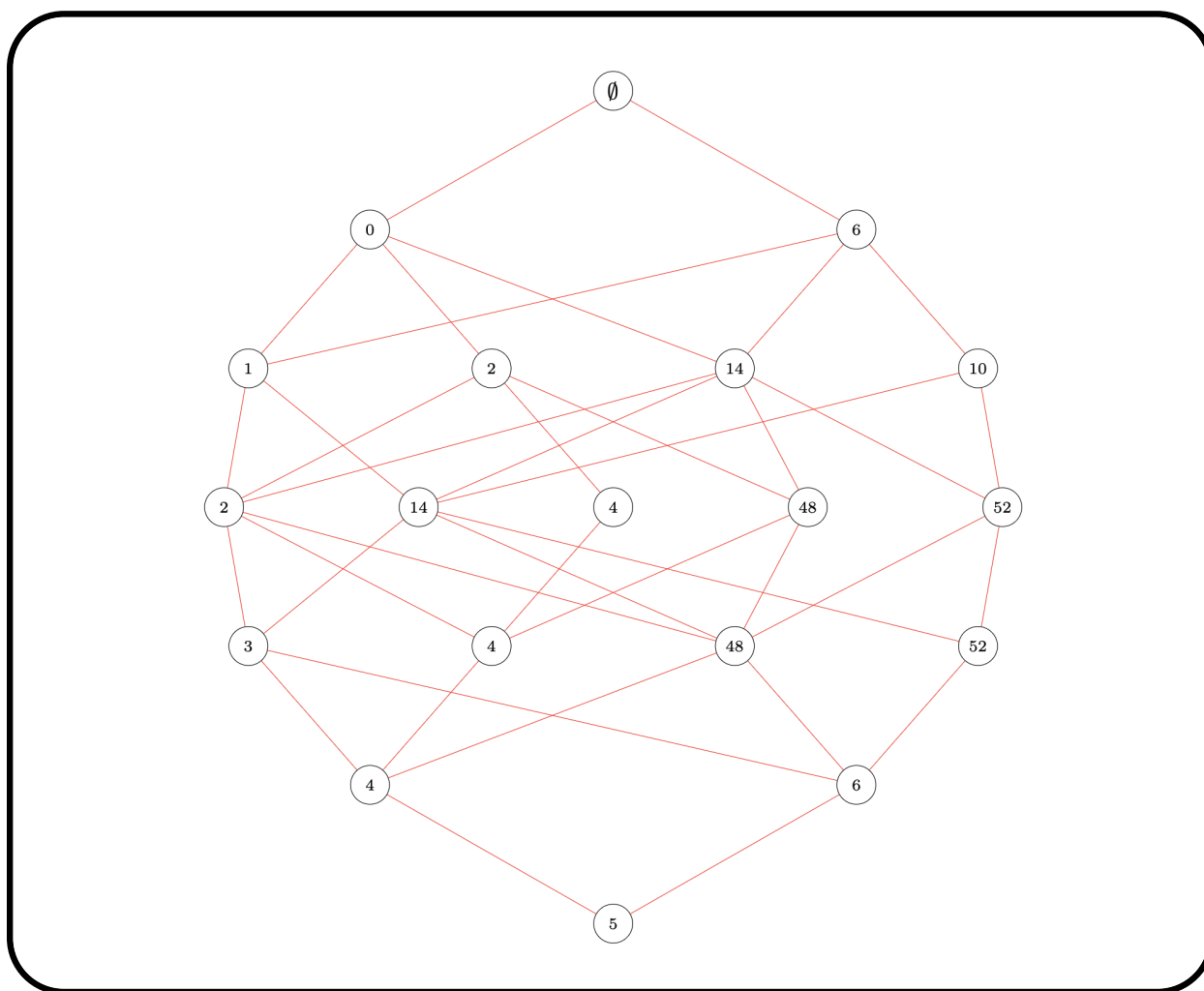
12.8 The Plesken Ring

Table 12.10 shows activities for objects of type Plesken-ring. For background on Plesken rings, see [63].

Consider the action of $\text{PGO}^+(6, 2)$ on the vector space $\mathbb{F}_2^6$ equipped with the quadratic form

$$X_0X_1 + X_2X_3 + X_4X_5.$$

The command computes the poset of orbits and produces a report. The report contains the following drawing of the poset or orbits:



The next command produces a full drawing of all elements of the poset, grouped by orbits. Not all relations are drawn. Only the relations relevant for the $P^\vee$ matrix are drawn.

Code Snippet 650

```

1   subspaces_GOp_6_2:
2   ▷ $(ORBITER) -v 5 \
3   ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4   ▷ ▷ -define Do -draw_options \
5   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6   ▷ ▷ ▷ -poset_orbits_x_stretch 750 \
7   ▷ ▷ ▷ -embedded \
8   ▷ ▷ ▷ -poset_type_Asup \
9   ▷ ▷ -end \
10  ▷ ▷ -define Control -poset_classification_control \
11  ▷ ▷ ▷ -problem_label GOp_6_2 -W -depth 6 \
12  ▷ ▷ -end \
13  ▷ ▷ -define G -linear_group -PGL 6 2 -orthogonal 1 -end \
14  ▷ ▷ -define Orb -orbits \
15  ▷ ▷ ▷ -group G \
16  ▷ ▷ ▷ -group_action G \

```

```

17  ▷ ▷ ▷ -generators G \
18  ▷ ▷ ▷ -on_subspaces 6 Control \
19  ▷ ▷ -end \
20  ▷ ▷ -define ro \
21  ▷ ▷ ▷ -poset_classification_report_options \
22  ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
23  ▷ ▷ ▷ -draw_options Do \
24  ▷ ▷ -end \
25  ▷ ▷ -with Orb -do -orbits_activity \
26  ▷ ▷ ▷ -report "" \
27  ▷ ▷ ▷ -report_options ro \
28  ▷ ▷ -end \
29  ▷ ▷ -print_symbols
30  ▷ $(PDFLATEX) GOp_6_2_poset.tex
31  ▷ $(OPEN) GOp_6_2_poset.pdf
32

```

The next command produces a drawing of the poset illustrating the P^V constants for the Plesken ring:

Code Snippet 651

```

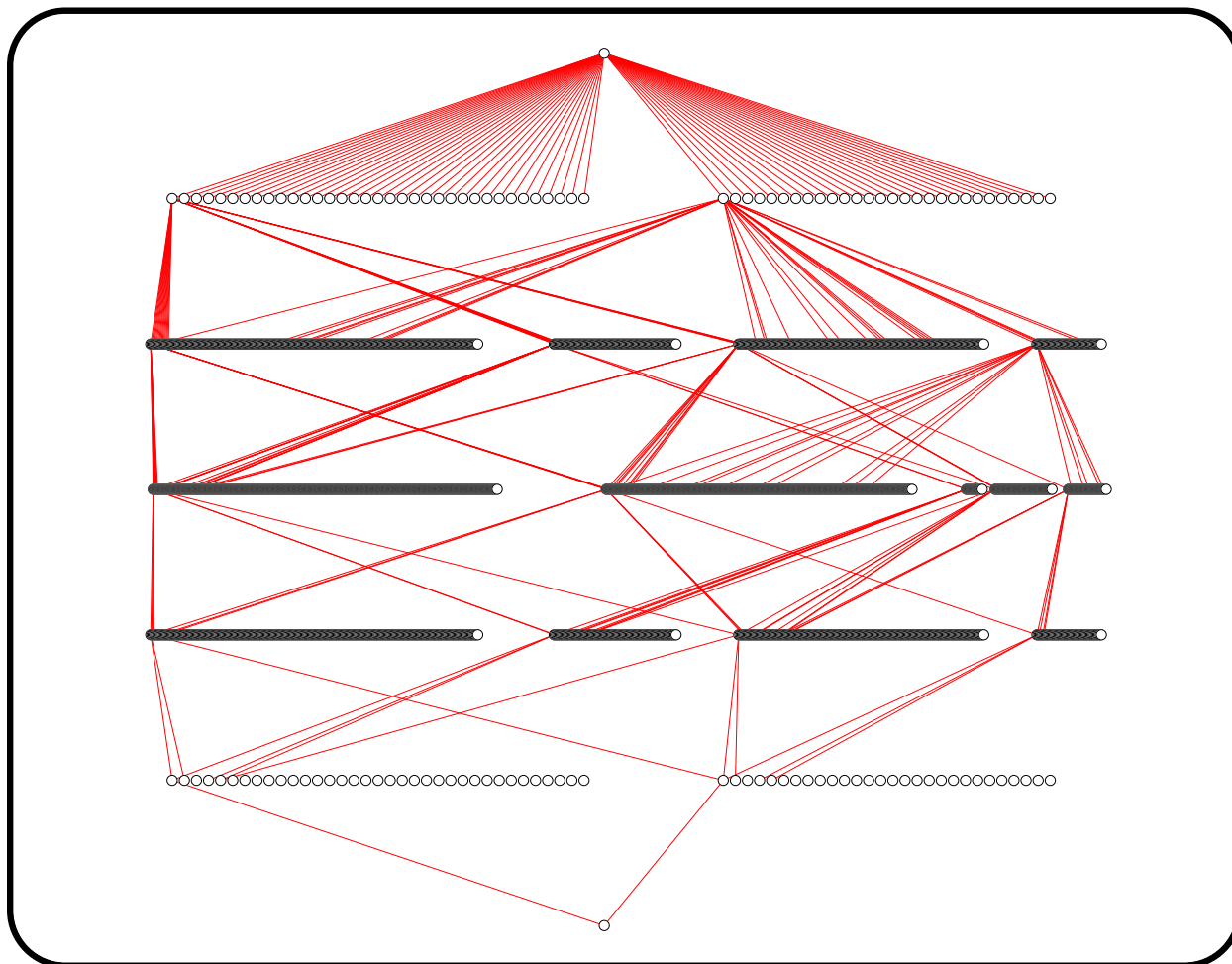
1    subspaces_GOp_6_2_draw_Asup:
2    ▷ $(ORBITER) -v 5 \
3    ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4    ▷ ▷ -define Do -draw_options \
5    ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6    ▷ ▷ ▷ -poset_orbits_x_stretch 750 \
7    ▷ ▷ ▷ -embedded \
8    ▷ ▷ ▷ -poset_type_Asup \
9    ▷ ▷ -end \
10   ▷ ▷ -define Control -poset_classification_control \
11   ▷ ▷ ▷ -problem_label GOp_6_2 -W -depth 6 \
12   ▷ ▷ -end \
13   ▷ ▷ -define G -linear_group -PGL 6 2 -orthogonal 1 -end \
14   ▷ ▷ -define Orb -orbits \
15   ▷ ▷ ▷ -group G \
16   ▷ ▷ ▷ -group_action G \
17   ▷ ▷ ▷ -generators G \
18   ▷ ▷ ▷ -on_subspaces 6 Control \
19   ▷ ▷ -end \
20   ▷ ▷ -define PCActivity -poset_classification_activity \
21   ▷ ▷ ▷ -draw_full_poset Do \
22   ▷ ▷ -end \
23   ▷ ▷ -with Orb -do -orbits_activity \
24   ▷ ▷ ▷ -poset_classification_activity PCActivity \
25   ▷ ▷ -end \
26   ▷ ▷ -print_symbols
27   ▷ $(ORBITER) -v 3 \
28   ▷ ▷ -draw_layered_graph \
29   ▷ ▷ ▷ GOp_6_2_poset_full_lvl_6_Asup.layered_graph \
30   ▷ ▷ -radius 50 -embedded \
31   ▷ ▷ ▷ -line_width 0.2 -x_stretch 1.4 -scale 0.5 \

```

```

32  ▷ ▷ -end
33  ▷ $(PDFLATEX) GOp_6_2_poset_full_lvl_6_Asup_draw.tex
34  ▷ $(OPEN) GOp_6_2_poset_full_lvl_6_Asup_draw.pdf
35

```



The next command creates the Plesken ring associated to the poset action of $\text{PGO}^+(6, 2)$ on the orthogonal space:

Code Snippet 652

```

1  subspaces_GOp_6_2_plesken_ring:
2  ▷ $(ORBITER) -v 5 \
3  ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4  ▷ ▷ -define Do -draw_options \
5  ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6  ▷ ▷ ▷ -poset_orbits_x_stretch 750 \
7  ▷ ▷ ▷ -embedded \
8  ▷ ▷ ▷ -poset_type_Asup \
9  ▷ ▷ -end \
10 ▷ ▷ -define Control -poset_classification_control \
11 ▷ ▷ ▷ -problem_label GOp_6_2 -W -depth 6 \

```

```

12  ▷ ▷ -end \
13  ▷ ▷ -define G -linear_group -PGL 6 2 -orthogonal 1 -end \
14  ▷ ▷ -define Orb -orbits \
15  ▷ ▷ ▷ -group G \
16  ▷ ▷ ▷ -group_action G \
17  ▷ ▷ ▷ -generators G \
18  ▷ ▷ ▷ -on_subspaces 6 Control \
19  ▷ ▷ -end \
20  ▷ ▷ -define PC.Plesken -poset_classification_activity \
21  ▷ ▷ ▷ -plesken_ring \
22  ▷ ▷ -end \
23  ▷ ▷ -assign P -from \
24  ▷ ▷ -with Orb -do -orbits_activity \
25  ▷ ▷ ▷ -poset_classification_activity PC.Plesken \
26  ▷ ▷ -end \
27  ▷ ▷ -with P -do -plesken_ring_activity \
28  ▷ ▷ ▷ -report Do \
29  ▷ ▷ -end \
30  ▷ ▷ -print_symbols
31  ▷ $(PDFLATEX) GOp_6_2_plesken_ring.tex
32  ▷ $(OPEN) GOp_6_2_plesken_ring.pdf
33

```

The command produces a latex report for the Plesken ring. The report contains the $P^\vee$ and $P^\wedge$ matrices. Here is the $P^\vee$ matrix.

$$\begin{array}{c}
N = 19 \\
P^\vee = \begin{bmatrix}
1 & 35 & 28 & 280 & 105 & 210 & 56 & 630 & 560 & 30 & 105 & 70 & 280 & 105 & 210 & 56 & 35 & 28 & 1 \\
0 & 1 & 0 & 16 & 9 & 6 & 0 & 90 & 48 & 6 & 9 & 2 & 72 & 33 & 42 & 8 & 19 & 12 & 1 \\
0 & 0 & 1 & 10 & 0 & 15 & 6 & 45 & 80 & 0 & 15 & 15 & 60 & 15 & 60 & 20 & 15 & 16 & 1 \\
0 & 0 & 0 & 1 & 0 & 0 & 0 & 9 & 6 & 0 & 0 & 0 & 18 & 6 & 9 & 2 & 9 & 6 & 1 \\
0 & 0 & 0 & 0 & 1 & 0 & 0 & 12 & 0 & 2 & 1 & 0 & 16 & 13 & 6 & 0 & 11 & 4 & 1 \\
0 & 0 & 0 & 0 & 0 & 1 & 0 & 3 & 8 & 0 & 3 & 1 & 12 & 3 & 16 & 4 & 7 & 8 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 10 & 0 & 0 & 5 & 10 & 0 & 15 & 10 & 5 & 10 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 4 & 2 & 1 & 0 & 5 & 2 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 3 & 0 & 3 & 1 & 3 & 4 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 7 & 0 & 0 & 7 & 0 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 6 & 0 & 3 & 4 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 3 & 4 & 1 & 6 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 2 & 1 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 3 & 0 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 2 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 3 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1
\end{bmatrix} \\
r = (2825, 374, 374, 67, 67, 67, 67, 16, 16, 16, 16, 16, 5, 5, 5, 5, 2, 2, 1) \\
c = (1, 36, 29, 307, 115, 232, 63, 790, 713, 39, 134, 94, 476, 186, 372, 106, 127, 107, 19)
\end{array}$$

For convenience, the output includes the number of orbits and the rows sums and the column sums of the $P^\vee$ matrix. The report contains the $P^\wedge$ matrix also.

Finally, we wish to demonstrate the Plesken ring. We will evaluate the product X_6^2 in the ring $P^\vee$. To do so, we need to define a suitable polynomial ring. We use the field $\mathbb{F}_2$ as the field of coefficients. This field only applies to the input formula. The Plesken ring disregards the finite field and evaluates over the integers $\mathbb{Z}$. Our input formula is X_6^2 , which is the same when read over $\mathbb{F}_2$. If our input formula had larger coefficients, we should use a larger input field. This is because the Orbiter parser for formulae expects the constants to lie in a finite field $\mathbb{F}_q$ and the value of the constant must be in the interval $[0, q - 1]$.

Code Snippet 653

```

1    subspaces_GOp_6_2_plesken_ring_evaluate_join:
2    ▷ $(ORBITER) -v 5 \
3    ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4    ▷ ▷ -define Do -draw_options \
5    ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6    ▷ ▷ ▷ -poset_orbits_x_stretch 750 \
7    ▷ ▷ ▷ -embedded \
8    ▷ ▷ ▷ -poset_type_Asup \
9    ▷ ▷ -end \
10   ▷ ▷ -define Control -poset_classification_control \
11   ▷ ▷ ▷ -problem_label Op_6_2 -W -depth 6 \
12   ▷ ▷ -end \
13   ▷ ▷ -define G -linear_group -PGL 6 2 -orthogonal 1 -end \
14   ▷ ▷ -define Orb -orbits \
15   ▷ ▷ ▷ -group G \
16   ▷ ▷ ▷ -group_action G \
17   ▷ ▷ ▷ -generators G \
18   ▷ ▷ ▷ -on_subspaces 6 Control \
19   ▷ ▷ -end \
20   ▷ ▷ -define ro \
21   ▷ ▷ ▷ -poset_classification_report_options \
22   ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
23   ▷ ▷ ▷ -draw_options Do \
24   ▷ ▷ -end \
25   ▷ ▷ -with Orb -do -orbits_activity \
26   ▷ ▷ ▷ -report "" \
27   ▷ ▷ ▷ -report_options ro \
28   ▷ ▷ -end \
29   ▷ ▷ -define PC_Plesken -poset_classification_activity \
30   ▷ ▷ ▷ -plesken_ring \
31   ▷ ▷ -end \
32   ▷ ▷ -assign P -from \
33   ▷ ▷ -with Orb -do -orbits_activity \
34   ▷ ▷ ▷ -poset_classification_activity PC_Plesken \
35   ▷ ▷ -end \
36   ▷ ▷ -with P -do -plesken_ring_activity \
37   ▷ ▷ ▷ -report Do \
38   ▷ ▷ -end \
39   ▷ ▷ -define F -finite_field -q 2 -end \
40   ▷ ▷ -define formula -symbolic_object \
41   ▷ ▷ ▷ -field F \
42   ▷ ▷ ▷ -text "X6^2" \
43   ▷ ▷ -end \
44   ▷ ▷ -define R -polynomial_ring \
45   ▷ ▷ ▷ -field F \
46   ▷ ▷ ▷ -number_of_variables 19 \
47   ▷ ▷ ▷ -homogeneous_of_degree 2 \
48   ▷ ▷ ▷ -monomial_ordering_partition \
49   ▷ ▷ ▷ -variables "X0,X1,X2,X3,X4,X5,X6,X7,X8,X9,X10,X11,X12,X13,X14,X15,X16,X17,X
18" \
50   ▷ ▷ ▷ "X_0,X_1,X_2,X_3,X_4,X_5,X_6,X_7,X_8,X_9,X_{10},X_{11},X_{12},X_{13},X_{14}

```

```

,X_{15},X_{16},X_{17},X_{18}" \
51  ▷ ▷ -end \
52  ▷ ▷ -with P -do -plesken_ring_activity \
53  ▷ ▷ ▷ -evaluate_join R formula \
54  ▷ ▷ -end \
55  ▷ ▷ -print_symbols
56

```

The output of the command is

$$(0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 12, 2, 0, 0, 30, 0, 0, 0)$$

which shows that

$$X_6^2 = X_6 + 12X_{11} + 2X_{12} + 30X_{15}$$

in the Plesken ring $P^\vee$ associated to the action of $\text{PGO}^+(6, 2)$ on the underlying orthogonal space.

The next command performs the pairwise join of all elements in orbit 6. The command creates two matrices, which are written as csv files. Let $a_1, \dots, a_\ell$ be the elements of orbit 6. The ij -entry in the first matrix is the index of the orbit containing $a_i \vee a_j$. The ij -entry in the second matrix is the index of $a_i \vee a_j$ inside the respective orbit.

Code Snippet 654

```

1  subspaces_GOp_6_2_make_pair_relations:
2  ▷ $(ORBITER) -v 5 \
3  ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4  ▷ ▷ -define Do -draw_options \
5  ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6  ▷ ▷ ▷ -poset_orbits_x_stretch 750 \
7  ▷ ▷ ▷ -embedded \
8  ▷ ▷ ▷ -poset_type_Asup \
9  ▷ ▷ -end \
10 ▷ ▷ -define Control -poset_classification_control \
11 ▷ ▷ ▷ -problem_label Op_6_2 -W -depth 6 \
12 ▷ ▷ -end \
13 ▷ ▷ -define G -linear_group -PGL 6 2 -orthogonal 1 -end \
14 ▷ ▷ -define Orb -orbits \
15 ▷ ▷ ▷ -group G \
16 ▷ ▷ ▷ -group_action G \
17 ▷ ▷ ▷ -generators G \
18 ▷ ▷ ▷ -on_subspaces 6 Control \
19 ▷ ▷ -end \
20 ▷ ▷ -define ro \
21 ▷ ▷ ▷ -poset_classification_report_options \
22 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
23 ▷ ▷ ▷ -draw_options Do \
24 ▷ ▷ -end \
25 ▷ ▷ -with Orb -do -orbits_activity \
26 ▷ ▷ ▷ -report "" \
27 ▷ ▷ ▷ -report_options ro \
28 ▷ ▷ -end \
29 ▷ ▷ -define PC_Activity -poset_classification_activity \
30 ▷ ▷ ▷ -pair_relations_within_orbit 6 \

```



```
31   ▷ ▷ -end \  
32   ▷ ▷ -with Orb -do -orbits_activity \  
33   ▷ ▷ ▷ -poset_classification_activity PC_Activity \  
34   ▷ ▷ -end \  
35   ▷ ▷ -print_symbols  
36
```


Chapter 13

Graph Theory

13.1 Creating Graphs

Tables 13.1-13.2 list Orbiter commands to create graphs.

Let us consider some examples.

The command

Code Snippet 655

```
1  make_graph_from_adjacency_matrix:
2  ▷ $(ORBITER) -v 6 \
3  ▷ ▷ -define triangle -vector -dense "0,1,1, 1,0,1, 1,1,0" -end \
4  ▷ ▷ -define G -graph \
5  ▷ ▷ ▷ -adjacency_matrix \
6  ▷ ▷ ▷ triangle \
7  ▷ ▷ -end
8
```

creates a graph on three vertices from a given adjacency matrix. The adjacency matrix must be given as a vector object, with the matrix entries listed in row-major ordering. In the example, the adjacency matrix is

$$\begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}$$

which is given as a vector

$$0, 1, 1, 1, 0, 1, 1, 1, 0.$$

The adjacency matrix of a graph can be read from a file. The following command sequence uses a makefile variable to store the adjacency matrix of a graph. The matrix is then copied into a file and the graph is created from the file. Here is the makefile variable containing the adjacency matrix:

Code Snippet 656

```
1  TRIANGLE_GRAPH="0,1,1\n1,0,1\n1,1,0\n"
2
```

Creating Graphs (Part 1)		
Command	Arguments	Purpose
-adjacency_matrix	vector	Create graph from the adjacency matrix. A vector object must be given.
-load	filename	Read a graph from file in Orbiter format.
-load_csv_no_border	filename	Read a graph from a csv file. The first row and the first column are part of the adjacency matrix.
-load_adjacency_matrix_from_csv_and_select_value	filename v	Read a graph from a csv file. The first row and the first column are ignored. Entries equal to v define adjacencies. All other values are ignored.
-load_dimacs	filename	Read a graph from file in dimacs format.
-load_Brouwer	filename	Read a graph from file in Brouwer format.
-edge_list	n list-of-edges	Create a graph on n vertices from a list of edges as ranked pairs.
-edges_as_pairs	n edges-as-pairs	Create a graph on n vertices from a list of edges as pairs.
-cycle	n	Cycle graph on n vertices.
-inversion_graph	text	Read inversion graph.
-Hamming	n q	Hamming graph $H(n, q)$
-Johnson	n k s	Johnson graph
-Paley	F	Paley graph over the field $F = \mathbb{F}_q$.
-Sarnak	p q	Lubotzky-Phillips-Sarnak graph [52]
-Schlaefli	F	Schlaefli graph
-Shrikhande		Shrikhande graph
-Winnie_Li	F i	Winnie-Li graph (see [51]) over the field $F = \mathbb{F}_q$ with subfield index i .
-Grassmann	n k F r	Grassmann graph over $F = \mathbb{F}_q$.
-coll_orthogonal	O S	Collinearity graph of the set S in the orthogonal geometry O .
-affine_polar	O	Affine polar graph associated to the orthogonal geometry O .
-triedral_pair_disjointness_graph		Trihedral pair disjointness graph.
-non_attacking_queens_graph	n	Create the graph for non-attacking queens on a $n \times n$ chess board.

Table 13.1: Creating Graphs (Part 1)

Creating Graphs (Part 2)		
Command	Arguments	Purpose
-subset	label labeltex subset	Define vertex coloring with two colors based on a subset of vertices.
-disjoint_sets_graph	fname	Reads a set of sets from the given file. Define a graph on the sets. Vertices correspond to the given sets. Two vertices are adjacent if the associated sets are disjoint.
-orbital_graph	G i	Define orbital graph from the i -th orbit of the group G acting on pairs.
-collinearity_graph	inc-matrix	Collinearity graph of the given incidence matrix.
-chain_graph	P1 P2	Chain graph with respect to the partitions P1 and P2.
-Neumaier_graph_16		Create a Neumaier graph on 16 vertices.
-Neumaier_graph_25		Create a Neumaier graph on 25 vertices, see [1]
-adjacency_bitvector	bitvec N	Create a graph on N vertices from a bitvector.
-Cayley_graph	G gens	Cayley graph with respect to group G and generating set gens.

Table 13.2: Creating Graphs (Part 2)

The following command writes the adjacency matrix to a csv file and then creates the graph from the csv file:

Code Snippet 657

```

1  make_triangle_graph:
2  ▷ echo $(TRIANGLE_GRAPH) >triangle_graph.csv
3  ▷ $(ORBITER) -v 6 \
4  ▷ ▷ -define G -graph \
5  ▷ ▷ ▷ -load_csv_no_border \
6  ▷ ▷ ▷ triangle_graph.csv \
7  ▷ ▷ -end
8
```

This will create the same triangle graph that we have created before.

The command

Code Snippet 658

```

1  Cycle_graph_13:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define Gamma -graph \
4  ▷ ▷ ▷ -cycle 13 \
5  ▷ ▷ -end \
6  ▷ ▷ -with Gamma -do \
```

```

7   ▷ ▷ ▷ -graph_theoretic_activity -export_csv \
8   ▷ ▷ -end
9

```

creates the cycle graph of degree 13.

The command

Code Snippet 659

```

1   Chain_232:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define P1 -vector -dense 2,3,2 -end \
4   ▷ ▷ -define P2 -vector -dense 2,3,2 -end \
5   ▷ ▷ -define Gamma -graph \
6   ▷ ▷ ▷ -chain_graph P1 P2 \
7   ▷ ▷ -end
8

```

creates the chain graph with respect to the partitions (2, 3, 2) and (2, 3, 2).

The command

Code Snippet 660

```

1   Paley_13_graph:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 13 -end \
4   ▷ ▷ -define Gamma -graph -Paley F -end \
5   ▷ ▷ -with Gamma -do \
6   ▷ ▷ ▷ -graph_theoretic_activity -export_csv \
7   ▷ ▷ -end
8

```

creates the Paley graph on 13 vertices. The command first creates the field $\mathbb{F}_{13}$ and then the associated Paley graph.

The command

Code Snippet 661

```

1   Paley_13_graph_complement:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 13 -end \
4   ▷ ▷ -define Gamma -graph -Paley F -complement -end \
5

```

creates the complement of the Paley graph on 13 vertices.

The command

Code Snippet 662

```

1   Winnie_Li_graph_q16_index2:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 16 -end \
4   ▷ ▷ -define Gamma -graph -Winnie_Li F 2 -end \
5

```

creates the Winnie Li graph [51] on 16 vertices with index 2. The command first creates the field $\mathbb{F}_{16}$ and then the associated graph.

The command

Code Snippet 663

```

1   Sarnak_graph_29_5:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Gamma -graph -Sarnak 29 5 -end \
4

```

creates the graph $X_{29,5}$ from [52] on 60 vertices.

The command

Code Snippet 664

```

1   Neumaier_graph_16:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Gamma -graph -Neumaier_graph_16 -end \
4

```

creates a Neumaier graph on 16 vertices.

The command

Code Snippet 665

```

1   tritangent_planes_graph:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Gamma \
4   ▷ ▷ ▷ -graph -tritangent_planes_disjointness_graph \
5   ▷ ▷ -end
6

```

creates the graph of tritangent planes in a general cubic surface with 27 lines. Two vertices are adjacent if the associated tritangent planes are line-disjoint.

The command

Code Snippet 666

```

1   trihedral_pair_graph:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Gamma \

```

```

4    ▷ ▷ ▷ -graph -trihedral_pair_disjointness_graph \
5    ▷ ▷ -end
6

```

creates the graph of trihedral pairs in a general cubic surface with 27 lines. Two vertices are adjacent if the associated trihedral pairs are line-disjoint.

The command

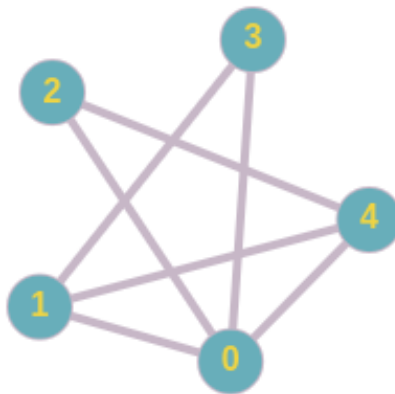
Code Snippet 667

```

1    small_graph:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define G -graph -edges_as_pairs \
4    ▷ ▷ ▷ 5 "0,1,0,2,0,3,0,4,1,3,1,4,2,4" \
5    ▷ ▷ -end
6

```

creates a graph by listing the edges in pairs. The following graph is created:



The command

Code Snippet 668

```

1    petersen:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define G -graph -Johnson 5 2 0 -end
4

```

creates the Petersen graph.

The command

Code Snippet 669

```

1   Johnson_6_2_0:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define G -graph -Johnson 6 2 0 -end
4

```

creates the Johnson graph $J(6, 2, 0)$.

The command

Code Snippet 670

```

1   Hamming_graph_3:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define G -graph -Hamming 3 2 -end
4

```

creates the Hamming graph of order 3.

The command

Code Snippet 671

```

1   Hamming_graph_7:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define G -graph -Hamming 7 2 -end
4

```

creates the Hamming graph of order 7.

The command

Code Snippet 672

```

1   Op_6_2_coll_graph:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 13 -end \
4   ▷ ▷ -define O -orthogonal_space 1 6 F -end \
5   ▷ ▷ -define v -vector -loop 0 35 1 -end \
6   ▷ ▷ -define Gamma -graph -coll_orthogonal O v -end \
7   ▷ ▷ -with Gamma -do \
8   ▷ ▷ ▷ -graph_theoretic_activity -save \
9   ▷ ▷ -end
10

```

creates the collinearity graph of the orthogonal space $O^+(6, 2)$.

There is a graph on 315 vertices that arises from the Cohen-Tits near octagon (see [21]). The graph was first constructed in [24] and has automorphism group equal to $\text{Aut}(HJ)$, the automorphism group of the Higman-Sims sporadic simple group. The graph is distance-regular. An incidence matrix can be found in Ascii format on the web site [10]. In the following, we assume that a file `halljanko315.csv` is present, containing the incidence matrix of the graph. The following command creates the graph from the file:

Graph Modifiers		
Command	Arguments	Purpose
-complement		Complementary graph.
-distance_2		Distance two graph: Two vertices are adjacent if and only if they have distance two in the original graph.
-reorder	perm	Apply the given permutation to the vertices of the graph.

Table 13.3: Graph Modifiers

Code Snippet 673

```

1  HJ_graph:
2  ▷ cp $(ORBITER_PATH)/examples/users_guide/halljanko315.csv .
3  ▷ $(ORBITER) -v 6 \
4  ▷ ▷ -define G -graph \
5  ▷ ▷ ▷ -load_csv_no_border \
6  ▷ ▷ ▷ halljanko315.csv \
7  ▷ ▷ -end
8

```

In Section 17.7, we will compute the automorphism group of the graph (of order 1209600). This will create a file `halljanko315_gens.csv` which we use here to create an orbital graph. An orbital graph is a graph whose adjacency matrix corresponds to an orbit of a permutation group in the action on pairs. The group is the automorphism group of the graph. The following command creates the third orbital graph:

Code Snippet 674

```

1  Group_Perm315_Orbital_3.colored_graph:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define gens -vector -format 5 -file \
4  ▷ ▷ ▷ File_halljanko315_gens.csv -end \
5  ▷ ▷ -define G -permutation_group \
6  ▷ ▷ ▷ -bsgs halljanko315 "File\halljanko315" \
7  ▷ ▷ ▷ 315 1209600 "0,1,2" 5 gens \
8  ▷ ▷ -end \
9  ▷ ▷ -define Gamma -graph \
10 ▷ ▷ ▷ -orbital_graph G 3 \
11 ▷ ▷ -end \
12 ▷ ▷ -with Gamma -do \
13 ▷ ▷ ▷ -graph_theoretic_activity -save \
14 ▷ ▷ -end
15

```

Table 13.3 shows some Orbiter commands to modify graphs. The commands replace the given graph by the graph obtained after applying the specified modification.

For a graph Γ , the distance 2 graph Δ has the same vertices as Γ , and two vertices in Δ are adjacent if and only if the distance in Γ is two. The following command creates the distance 2 graph of the Cohen-Tits graph.

Code Snippet 675

```

1    HJ_d2_graph:
2    ▷ cp $(ORBITER_PATH)/examples/users_guide/halljanko315.csv .
3    ▷ $(ORBITER) -v 6 \
4    ▷ ▷ -define G -graph \
5    ▷ ▷ ▷ -load_csv_no_border \
6    ▷ ▷ ▷ halljanko315.csv \
7    ▷ ▷ ▷ -distance_2 \
8    ▷ ▷ -end
9

```

Let us look at some examples of Cayley graphs. The first graph has $G = \mathbb{Z}_{11}$ and connection set all elements congruent 1 mod 3. We create the group as a subgroup of the one-dimensional affine group over $\mathbb{F}_{11}$. This means that the map $x \mapsto ax + b \bmod 11$ is encoded as a pair (a, b) .

Code Snippet 676

```

1    Cayley_Z11_1mod3:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 11 -end \
4    ▷ ▷ -define S -vector -dense \
5    ▷ ▷ ▷ "1,1, 1,4, 1,7, 1,10" -end \
6    ▷ ▷ -define G -linear_group -AGL 1 F \
7    ▷ ▷ ▷ -subgroup_by_generators "Z11" 11 1 "1,1" \
8    ▷ ▷ -end \
9    ▷ ▷ -define Gamma -graph \
10   ▷ ▷ ▷ -Cayley_graph G S \
11   ▷ ▷ -end
12

```

The vertices of the Cayley graph are ordered. The ordering is determined by the stabilizer chain. This is a total ordering.

In the following example, we create a Cayley graph based on the symmetric group on 4 things. We take the Coxeter generators as connection set:

Code Snippet 677

```

1    Cayley_Sym4_coxeter:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define S -vector -dense "1,0,2,3, 0,2,1,3, 0,1,3,2" -end \
4    ▷ ▷ -define G -permutation_group -symmetric_group 4 \
5    ▷ ▷ -end \
6    ▷ ▷ -define Gamma -graph \
7    ▷ ▷ ▷ -Cayley_graph G S \
8    ▷ ▷ -end
9

```

The star graph is another Cayley graph for the symmetric group. The connection set is given by the permutations $(0, i)$ for $i = 1, \dots, n - 1$. The next example creates the star graph on 4 vertices:

Code Snippet 678

```

1   Cayley_Sym4.star:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define S -vector -dense "1,0,2,3, 2,1,0,3, 3,1,2,0" -end \
4   ▷ ▷ -define G -permutation_group -symmetric_group 4 \
5   ▷ ▷ -end \
6   ▷ ▷ -define Gamma -graph \
7   ▷ ▷ ▷ -Cayley_graph G S \
8   ▷ ▷ -end
9

```

In the following example, we create a graph of nonattacking queens on a chess board.

Code Snippet 679

```

1   Queens_graph:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define G -graph -non_attacking_queens_graph 8 -end \
4

```

In the following example, we create an affine polar graph based on the $O^+(4, 2)$ space.

Code Snippet 680

```

1   Op_4_2_affine_polar_graph:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 2 -end \
4   ▷ ▷ -define O -orthogonal_space 1 4 F -end \
5   ▷ ▷ -define Gamma -graph -affine_polar O -end \
6   ▷ ▷ -with Gamma -do \
7   ▷ ▷ ▷ -graph_theoretic_activity -save \
8   ▷ ▷ -end
9

```

Next, we create the first and then the 15th graph in the list of Paulus and Rozenfeld.

Code Snippet 681

```

1   Paulus_1:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define P25_1 -graph -load_Brouwer \
4   ▷ ▷ ▷ $(ORBITER_PATH)/examples/users_guide/PAULUS_ROSENFELD/P25_1.txt \
5   ▷ ▷ -end
6

```

```

1 Paulus_15:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define P25_15 -graph -load_Brouwer \
4   ▷ ▷ ▷ ▷ $(ORBITER_PATH)/examples/users_guide/PAULUS.ROSENFELD/P25.15.txt \
5   ▷ ▷ -end
6

```

set:

```
1 GOLAY_24_CODE_PROJECTIVE="0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 132, 913, 1460, 1
2 750, 1898, 2518, 2787, 2874, 3320, 3357, 3662, 4004"
```

on $\text{PG}(11, 2)$ and stabilizing the set S . They are encoded as a makefile variable:

[illegible]

[illegible]

The following code snippet creates the graph:

Code Snippet 685

```

1      Higman_Sims_graph:
2      ▷ $(ORBITER) -v 2 \
3      ▷ ▷ -orbiter_path $(ORBITER.EXE_PATH) \
4      ▷ ▷ -define Do -draw_options \
5      ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6      ▷ ▷ -end \
7      ▷ ▷ -define code -vector \
8      ▷ ▷ ▷ -dense $(GOLAY_24_CODE_PROJECTIVE) \
9      ▷ ▷ -end \
10     ▷ ▷ -define Control2 -poset_classification_control \
11     ▷ ▷ ▷ -problem_label M24_pairs -W -depth 2 \
12     ▷ ▷ ▷ -draw_options Do \
13     ▷ ▷ -end \
14     ▷ ▷ -define F -finite_field -q 2 -end \
15     ▷ ▷ -define G -linear_group -PGL 12 2 \
16     ▷ ▷ -subgroup_by_generators "M24" "244823040" 12 \
17     ▷ ▷ ▷ $(GOLAY_24_CODE_AUT_GEN) \
18     ▷ ▷ -end \
19     ▷ ▷ -define M24_perm -modified_group -from G \
20     ▷ ▷ ▷ -restricted_action code \\on\\code \
21     ▷ ▷ -end \
22     ▷ ▷ -define M22_perm -modified_group -from M24_perm \
23     ▷ ▷ ▷ -set_stabilizer 0,1 Control2 \

```

```

24  ▷ ▷ -end \
25  ▷ ▷ -define pts -set -loop 2 22 1 -end \
26  ▷ ▷ -define M22 -modified_group -from M22_perm \
27  ▷ ▷ ▷ -restricted_action pts M22 \
28  ▷ ▷ -end \
29  ▷ ▷ -with M22 -do \
30  ▷ ▷ -group_theoretic_activity \
31  ▷ ▷ ▷ -report Do \
32  ▷ ▷ -end \
33  ▷ ▷ -define Control -poset_classification_control \
34  ▷ ▷ ▷ -problem_label M22 -W -depth 6 \
35  ▷ ▷ ▷ -draw_options Do \
36  ▷ ▷ -end \
37  ▷ ▷ -define Orb -orbits \
38  ▷ ▷ ▷ -group M22 \
39  ▷ ▷ ▷ -group_action M22 \
40  ▷ ▷ ▷ -generators M22 \
41  ▷ ▷ ▷ -on_subsets 6 Control \
42  ▷ ▷ -end \
43  ▷ ▷ -with Orb -do -orbits_activity \
44  ▷ ▷ ▷ -Kramer_Mesner_matrix 3 6 \
45  ▷ ▷ -end \
46  ▷ ▷ -define ro \
47  ▷ ▷ ▷ -poset_classification_report_options \
48  ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
49  ▷ ▷ ▷ -draw_options Do \
50  ▷ ▷ -end \
51  ▷ ▷ -with Orb -do -orbits_activity \
52  ▷ ▷ ▷ -report "" \
53  ▷ ▷ ▷ -report_options ro \
54  ▷ ▷ -end \
55  ▷ ▷ -define D -design -orbit_on_sets \
56  ▷ ▷ ▷ Orb 6 1 \
57  ▷ ▷ -end \
58  ▷ ▷ -with D -do \
59  ▷ ▷ ▷ -design_activity \
60  ▷ ▷ ▷ ▷ -export_flags \
61  ▷ ▷ -end \
62  ▷ ▷ -with D -do \
63  ▷ ▷ ▷ -design_activity \
64  ▷ ▷ ▷ ▷ -intersection_matrix \
65  ▷ ▷ -end \
66  ▷ ▷ -define Control_block_orbits -poset_classification_control \
67  ▷ ▷ ▷ -problem_label M22bar_block_orbits -W -depth 2 \
68  ▷ ▷ ▷ -draw_options Do \
69  ▷ ▷ -end \
70  ▷ ▷ -with D -do \
71  ▷ ▷ ▷ -design_activity \
72  ▷ ▷ ▷ ▷ -orbits_on_blocks 2 Control_block_orbits \
73  ▷ ▷ -end \
74  ▷ ▷ -with D -do \
75  ▷ ▷ ▷ -design_activity \

```

```

76  ▷ ▷ ▷ ▷ -one_point_extension 1 Control_block_orbits \
77  ▷ ▷ -end \
78  ▷ ▷ -define Gamma -graph \
79  ▷ ▷ ▷ -load_adjacency_matrix_from_csv_and_select_value \
80  ▷ ▷ ▷ blocks_v22_k6_one_pt_ext_orb1.csv 1 \
81  ▷ ▷ -end
82

```

In lines 7-9, we create the set S of size 24. In line 14, we create the field $\mathbb{F}_2$. In lines 15-18, we create the group M_{24} as a matrix subgroup of $\text{PGL}(12, 2)$. In lines 19-21, we create the permutation representation of M_{24} of degree 24 acting on the set S . In lines 22-24, we create the subgroup $M_{22}.2$ which stabilizes the elements $\{0, 1\}$ of S as a set. In lines 25-28, we create the permutation action of $M_{22}.2$ on the 22 points of S different from $\{0, 1\}$. Note that the action is re-indexing the point set as $0 - 21$. In lines 37-42, we invoke the poset classification algorithm to compute the orbits of $M_{22}.2$ on subsets of size at most 6. This computation takes place in the action of degree 22 just constructed. In lines 43-45, we create the Kramer-Mesner matrix for the orbits on 3-subsets versus the orbits on 6-subsets. This matrix allows us to identify the orbit on 6-subsets which forms a Steiner 3-design. In the numbering chosen by the poset classification algorithm, it is the second orbit. Since indexing is zero-based, this is orbit 1. In lines 55-57, we create the design from this orbit. This is the Witt design on 22 points. In lines 58-61, we export the design to file by means of the set of flags. In lines 62-65, we compute the intersection matrix of the design. This matrix can be used to give an independent verification that the chosen orbit has the design property. In lines 70-73, we compute the orbits of the group of the design (which is $M_{11}.2$) on pairs of blocks. In lines 74-77, we create the one-point extension of the Witt design based on the orbit on pairs of index 1. Finally, in lines 78-81, the Higman-Sims graph is constructed from the one-point extension.

13.2 Graph Theoretic Activities

Graph theoretic activities allow us to perform tasks on graphs. Tables 13.4-13.5 show the commands for graph theoretic activities. These are activities that can be applied to objects of type graph.

Let us continue the example of the three-cycle from Section 13.1. The command

Code Snippet 686

```
1      triangle_graph_properties:
2      ▷ echo $(TRIANGLE_GRAPH) >triangle_graph.csv
3      ▷ $(ORBITER) -v 6 \
4      ▷ ▷ -define G -graph \
5      ▷ ▷ ▷ -load_csv_no_border \
6      ▷ ▷ ▷ triangle_graph.csv \
7      ▷ ▷ -end \
8      ▷ ▷ -with G -do \
9      ▷ ▷ ▷ -graph_theoretic_activity -properties \
10     ▷ ▷ -end
11
```

computes the degree type of the graph. This is the distribution of degrees in the degree sequence of the graph. It prints the distribution of degree values in exponential notation. The multiplicities are indicated as exponent. For instance, the graph in this example has three vertices of degree 2, so the degree type is written as 2^3 .

The command

Code Snippet 687

```
1      Hamming_graph_7_drg:
2      ▷ $(ORBITER) -v 2 \
3      ▷ ▷ -define Gamma -graph -Hamming 7 2 -end \
4      ▷ ▷ -with Gamma -do -graph_theoretic_activity \
5      ▷ ▷ ▷ -test_if_distance_regular 0 \
6      ▷ ▷ -end
7
```

verifies that the Hamming graph of order 7 is distance regular. The command assumes that the graph is known to be vertex transitive. The computation is performed using vertex 0 as a distinguished vertex. The vertex for which the computation is to be performed is an argument to the command. If the graph is distance regular, the parameters a_i, b_i, c_i are computed for $i = 0, \dots, d$, where d is the diameter of the graph.

The command

Code Snippet 688

```
1      Johnson_8_3_2_drg:
2      ▷ $(ORBITER) -v 2 \
3      ▷ ▷ -define Gamma -graph -Johnson 8 3 2 -end \
4      ▷ ▷ -with Gamma -do -graph_theoretic_activity \
5      ▷ ▷ ▷ -test_if_distance_regular 0 \
6      ▷ ▷ -end
7
```

Graph Theoretic Activities (Part 1)		
Command	Arguments	Purpose
-find_cliques	options	Find all cliques. See Section 16.1.
-test_SRG_property		Test whether a graph is strongly regular.
-test_Neumaier_property	s	Test whether a graph is Neumaier w.r.t. cliques of size s .
-find_subgraph	type	Finds all labelings of a Dynkin diagram of a given type. This needs two graphs. The vertices correspond to the involutions. The first graph is the pairs whose product has order 2. The second graph is the pairs whose product has order 3.
-test_automorphism_property_of_group	group	Tests if the given group preserves the graph.
-common_neighbors	S	Find the common neighbors of the set S . S must be a vector object.
-export_magma		Export to Magma [20].
-export_maple		Export to Maple [55].
-export_csv		Export to csv-file.
-export_graphviz		Export to graphviz-file.
-print		Print the graph.
-sort_by_colors		Sort the vertices by color classes.
-A_powers	$k_{\max}$	Computes the powers A^i for $i=0,1,\dots,k_{\max}$ and tries to find a linear relation between them over the field of real numbers (i.e., in the vector space $\mathbb{R}^{n \times n}$). Here, A is the adjacency matrix of the graph.
-distance_from	v_0	Computes the distances of all vertices from the given vertex v_0 . The output is written to a csv file.
-split	input file	Split the graph into subgraphs.
-split_by_starters	fname_reps.csv col_label	Split the graph into subgraphs according to starters listed in the specified column of the given csv file.
-combine_by_starters	fname_reps col_label solu- tion_fname_mask	Combine the graph from subgraphs according to starters listed in the specified column of the given csv file.
-split_by_clique	label clique	Compute the neighborhood graph of the given clique.
-save		Save the graph to file in binary format.

Table 13.4: Graph Theoretic Activities (Part 1)

Graph Theoretic Activities (Part 2)		
Command	Arguments	Purpose
-automorphism_group		Compute the automorphism group and write a report. See Section 17.7.
-automorphism_group_colored_graph		Compute the automorphism group of a vertex colored graph and write a report. See Section 17.7.
-properties		Compute properties of the graph.
-eigenvalues		Compute the eigenvalues of the graph (ordinary and Laplace) and write to a csv file.
-eigenvalues_report		Compute the eigenvalues of the graph (ordinary and Laplace) and produce a latex report.
-draw	options	Draw the graph. Uses a draw options object as in Table 19.1 for drawing.
-create_distance_poset	v	Create the distance poset with respect to the vertex v .
-test_if_distance_regular	v	Test if distance regular with respect to the vertex v . This is meaningful only if the graph is vertex transitive.

Table 13.5: Graph Theoretic Activities (Part 2)

verifies that the Johnson graph $J_{8,3}$ is distance regular.

The following command exports the adjacency matrix and creates a bitmap drawing of it.

Code Snippet 689

```

1   Cycle_13_draw:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Gamma -graph -cycle 13 -end \
4   ▷ ▷ -with Gamma -do \
5   ▷ ▷ -graph_theoretic_activity -export_csv -end \
6   ▷ ▷ -with Gamma -do \
7   ▷ ▷ -graph_theoretic_activity -export_graphviz -end
8   ▷ $(ORBITER) -v 2 -draw_matrix \
9   ▷ ▷ -input_csv_file Cycle_13.csv \
10  ▷ ▷ -box_width 20 -bit_depth 8 -partition 4 13 13 -end
11  ▷ dot -Tpng Cycle_13.gv >Cycle_13.png
12  ▷ #twopi -Tpng Cycle_13.gv >Cycle_13.png
13  ▷ #$(OPEN) Cycle_13_draw.bmp
14  ▷ #$(PDFLATEX) Cycle_13_report.tex
15  ▷ #$(OPEN) Cycle_13_report.pdf
16

```

The command first creates the cycle graph of order 13, and then exports the adjacency matrix as csv file. It then draws the adjacency matrix as a bitmap graphics.

Suppose we want to compute the eigenvalues of the adjacency matrix of a graph. In the following example, the command `-eigenvalues` is used to compute the eigenvalues (both regular and Laplace) of the 9-cycle:

Code Snippet 690

```

1   Cycle_9_eigenvalues:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Gamma -graph \
4   ▷ ▷ ▷ -cycle 9 \
5   ▷ ▷ -end \
6   ▷ ▷ -with Gamma -do \
7   ▷ ▷ -graph_theoretic_activity -eigenvalues -end \
8   ▷ ▷ -with Gamma -do \
9   ▷ ▷ -graph_theoretic_activity -eigenvalue_report -end
10  ▷ $(PDFLATEX) Cycle_9_eigenvalues.tex
11  ▷ $(OPEN) Cycle_9_eigenvalues.pdf
12

```

The following output is produced:

i	λ_i	θ_i
0	2	3.87939
1	1.53209	3.87939
2	1.53209	3
3	0.347296	3
4	0.347296	1.6527
5	-1	1.6527
6	-1	0.467911
7	-1.87939	0.467911
8	-1.87939	$-2.26243e - 16$

The energy is 11.5175
 Eigenvalues: λ_i
 Laplace eigenvalues: θ_i

The command

Code Snippet 691

```

1   Paley_13_draw:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 13 -end \
4   ▷ ▷ -define Gamma -graph -Paley F -end \
5   ▷ ▷ -with Gamma -do \
6   ▷ ▷ -graph_theoretic_activity -export_csv -end \
7   ▷ ▷ -with Gamma -do \
8   ▷ ▷ -graph_theoretic_activity -export_graphviz -end
9   ▷ $(ORBITER) -v 2 -draw_matrix \
10  ▷ ▷ -input_csv_file Paley_13.csv \
11  ▷ ▷ -box_width 20 -bit_depth 8 -partition 4 13 13 -end
12  ▷ dot -Tpng Paley_13.gv >Paley_13.png

```

```

13  ▷ $(OPEN) Paley_13_draw.bmp
14

```

draws the Paley graph of order 13 created in Section 13.1 using the external tool graphviz.

Let us consider the Cayley graphs from Section 13.1. Here is a command that draws the first graph and computes the eigenvalues:

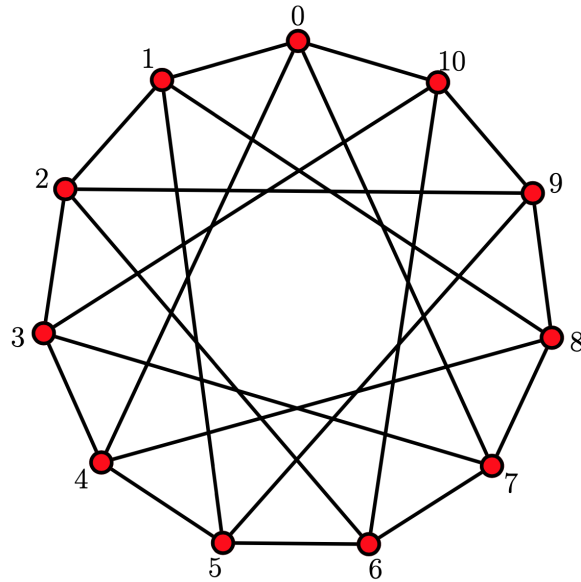
Code Snippet 692

```

1  Cayley_Z11_1mod3_eigenvalues_and_draw:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define Do -draw_options -xin 20000 \
4  ▷ ▷ ▷ -yin 20000 -embedded -radius 200 -end \
5  ▷ ▷ -define F -finite_field -q 11 -end \
6  ▷ ▷ -define S -vector -dense \
7  ▷ ▷ ▷ "1,1, 1,4, 1,7, 1,10" -end \
8  ▷ ▷ -define G -linear_group -AGL 1 F \
9  ▷ ▷ ▷ -subgroup_by_generators "Z11" 11 1 "1,1" \
10 ▷ ▷ -end \
11 ▷ ▷ -define Gamma -graph \
12 ▷ ▷ ▷ -Cayley_graph G S \
13 ▷ ▷ -end \
14 ▷ ▷ -with Gamma -do \
15 ▷ ▷ -graph_theoretic_activity -eigenvalues -end \
16 ▷ ▷ -with Gamma -do \
17 ▷ ▷ -graph_theoretic_activity -draw Do -end
18 ▷ $(PDFLATEX) Cayley_graph_AGL_1_11_draw.tex
19 ▷ $(OPEN) Cayley_graph_AGL_1_11_draw.pdf
20

```

The drawing is shown below



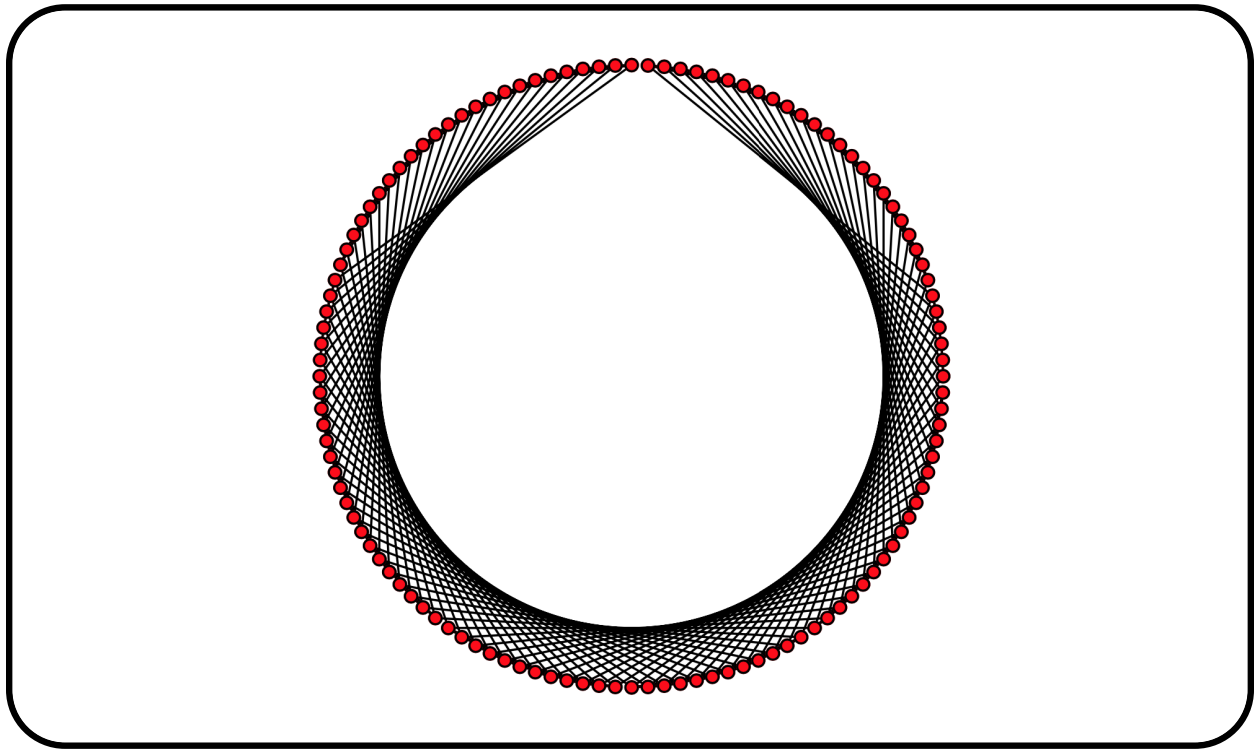
Let us draw the Cayley graph of $\text{Sym}(5)$ with respect to the Coxeter generators:

Code Snippet 693

```

1  Cayley_Sym5_coxeter_draw:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define Do -draw_options -xin 1000000 -yin 1000000 \
4  ▷ ▷ ▷ -embedded -radius 10000 -nodes_empty -end \
5  ▷ ▷ -define S -vector -dense \
6  ▷ ▷ ▷ "1,0,2,3,4, 0,2,1,3,4, 0,1,3,2,4, 0,1,2,4,3" -end \
7  ▷ ▷ -define G -permutation_group -symmetric_group 5 \
8  ▷ ▷ -end \
9  ▷ ▷ -define Gamma -graph \
10 ▷ ▷ ▷ -Cayley_graph G S \
11 ▷ ▷ -end \
12 ▷ ▷ -with Gamma -do \
13 ▷ ▷ -graph_theoretic_activity -draw Do -end
14 ▷ $(PDFLATEX) Cayley_graph_Sym_5_draw.tex
15 ▷ $(OPEN) Cayley_graph_Sym_5_draw.pdf
16
```

The drawing is shown below



It is possible to create the collinearity graph of an incidence structure. The incidence structure must be encoded by means of an incidence matrix. We can search for disjoint sets in the generalized quadrangle $W(2)$ (the Doily). For starters, here is the makefile variable to create the geometry:

Code Snippet 694

```

1    DOILY="Row,C0,C1,C2\
2    \n0,0,12,13\
3    \n1,1,12,14\
4    \n2,8,9,12\
5    \n3,4,6,8\
6    \n4,6,10,14\
7    \n5,3,7,8\
8    \n6,7,10,13\
9    \n7,4,11,13\
10   \n8,3,11,14\
11   \n9,0,5,6\
12   \n10,1,5,7\
13   \n11,5,9,11\
14   \n12,0,2,3\
15   \n13,1,2,4\
16   \n14,2,9,10\
17   \nEND"
18

```

Next, we read the incidence matrix from file and create the collinearity graph of it:

Code Snippet 695

```

1   PGO_5.2_collinearity_graph:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 2 -end \
4   ▷ ▷ -define O -orthogonal_space 0 5 F -without_group -end \
5   ▷ ▷ -with O -do -orthogonal_space_activity \
6   ▷ ▷ ▷ -export_point_line_incidence_matrix \
7   ▷ ▷ -end
8   ▷ $(ORBITER) -v 3 \
9   ▷ ▷ -define Inc -vector -file 0.5.2_incidence_matrix.csv -end \
10  ▷ ▷ -define Gamma -graph -collinearity_graph Inc -end \
11  ▷ ▷ -with Gamma -do \
12  ▷ ▷ -graph_theoretic_activity \
13  ▷ ▷ ▷ -properties \
14  ▷ ▷ -end
15

```

The command also computes properties of the graph. The graph has 15 vertices of degree 6. This is because in the geometry, each point lies on three lines, and hence is collinear with 6 other points.

The command

Code Snippet 696

```

1   Op_4.2_affine_polar_graph_info:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 2 -end \
4   ▷ ▷ -define O -orthogonal_space 1 4 F -end \
5   ▷ ▷ -define Gamma -graph -affine_polar O -end \
6   ▷ ▷ -with Gamma -do \
7   ▷ ▷ ▷ -graph_theoretic_activity -save \
8   ▷ ▷ -end \
9   ▷ ▷ -with Gamma -do \
10  ▷ ▷ ▷ -graph_theoretic_activity -test_SRG_property \
11  ▷ ▷ -end \
12  ▷ ▷ -with Gamma -do \
13  ▷ ▷ -graph_theoretic_activity \
14  ▷ ▷ ▷ -find_cliques -target_size 4 \
15  ▷ ▷ -end \
16  ▷ ▷ -with Gamma -do \
17  ▷ ▷ -graph_theoretic_activity \
18  ▷ ▷ ▷ -automorphism_group \
19  ▷ ▷ -end
20  ▷ #$(PDFLATEX) affine_polar_0.1.4.2_report.tex
21  ▷ #$(OPEN) affine_polar_0.1.4.2_report.pdf
22

```

computes properties of the affine polar graph based on $O^+(4, 2)$.

The command

Code Snippet 697

```

1   Neumaier_16_test_SRG.property:
2   ▷ $(ORBITER) -v 10 \
3   ▷ ▷ -define Gamma -graph -Neumaier_graph_16 -end \
4   ▷ ▷ -with Gamma -do \
5   ▷ ▷ -graph_theoretic_activity \
6   ▷ ▷ ▷ -test_SRG_property \
7   ▷ ▷ -end
8

```

tests the Neumaier property for the Neumaier graph on 16 vertices.

The command

Code Snippet 698

```

1   Paulus_1_aut:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define P25_1 -graph -load.Brouwer \
4   ▷ ▷ ▷ ▷ $(ORBITER_PATH)/examples/users_guide/PAULUS.ROSENFELD/P25_1.txt \
5   ▷ ▷ -end \
6   ▷ ▷ -with P25_1 -do \
7   ▷ ▷ -graph_theoretic_activity \
8   ▷ ▷ ▷ -automorphism_group \
9   ▷ ▷ -end
10  ▷ $(PDFLATEX) P25_1_report.tex
11  ▷ $(OPEN) P25_1_report.pdf
12

```

computes the automorphism group of the first graph in the list of Paulus and Rozenfeldt.

The command

Code Snippet 699

```

1   Paulus_15_aut:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define P25_15 -graph -load.Brouwer \
4   ▷ ▷ ▷ ▷ $(ORBITER_PATH)/examples/users_guide/PAULUS.ROSENFELD/P25_15.txt \
5   ▷ ▷ -end \
6   ▷ ▷ -with P25_15 -do \
7   ▷ ▷ -graph_theoretic_activity \
8   ▷ ▷ ▷ -automorphism_group \
9   ▷ ▷ -end
10  ▷ $(PDFLATEX) P25_15_report.tex
11  ▷ $(OPEN) P25_15_report.pdf
12

```

computes the automorphism group of the 15th graph in the list of Paulus and Rozenfeldt.

The command

Code Snippet 700

```

1   PG_3.2.packing.make_group_and_test:
2   ▷ - mkdir SPREAD_TABLES_2
3   ▷ $(ORBITER) -v 5 \
4   ▷ ▷ -define Control -poset_classification_control \
5   ▷ ▷ ▷ -problem_label spreads_PG_3.2 \
6   ▷ ▷ ▷ -W \
7   ▷ ▷ -end \
8   ▷ ▷ -define F -finite_field -q 2 -end \
9   ▷ ▷ -define P3 -projective_space -n 3 -field F -v 0 -end \
10  ▷ ▷ -define P5 -projective_space -n 5 -field F -v 0 -end \
11  ▷ ▷ -define T -spread_table -space P3 -rk 2 \
12  ▷ ▷ ▷ -iso_types "0" \
13  ▷ ▷ ▷ -path "SPREAD_TABLES_2/" \
14  ▷ ▷ ▷ -control Control \
15  ▷ ▷ ▷ -end \
16  ▷ ▷ -define PC -packing_classify P3 P5 T \
17  ▷ ▷ -with PC -do -packing_classify_activity \
18  ▷ ▷ ▷ -make_graph_of_disjoint_spreads \
19  ▷ ▷ -end
20  ▷ cp SPREAD_TABLES_2/spread_4_disjoint.graph .
21  ▷ $(ORBITER) -v 5 \
22  ▷ ▷ -define Control -poset_classification_control \
23  ▷ ▷ ▷ -problem_label packings_PG_3.2 \
24  ▷ ▷ ▷ -W \
25  ▷ ▷ -end \
26  ▷ ▷ -define F -finite_field -q 2 -end \
27  ▷ ▷ -define P3 -projective_space -n 3 -field F -v 0 -end \
28  ▷ ▷ -define P5 -projective_space -n 5 -field F -v 0 -end \
29  ▷ ▷ -define T -spread_table -space P3 -rk 2 \
30  ▷ ▷ ▷ -iso_types "0" \
31  ▷ ▷ ▷ -path "SPREAD_TABLES_2/" \
32  ▷ ▷ ▷ -control Control \
33  ▷ ▷ ▷ -end \
34  ▷ ▷ -define PC -packing_classify P3 P5 T \
35  ▷ ▷ -with PC -do -packing_classify_activity \
36  ▷ ▷ ▷ -export_group_on_spreads \
37  ▷ ▷ -end \
38  ▷ ▷ -define G -modified_group \
39  ▷ ▷ ▷ -import \
40  ▷ ▷ -end \
41  ▷ ▷ -define Orb -orbits -group G \
42  ▷ ▷ ▷ -on_points \
43  ▷ ▷ -end \
44  ▷ ▷ -define Gamma -graph \
45  ▷ ▷ ▷ -load \
46  ▷ ▷ ▷ spread_4_disjoint.graph \
47  ▷ ▷ -end \
48  ▷ ▷ -with Gamma -do \
49  ▷ ▷ ▷ -graph_theoretic_activity \
50  ▷ ▷ ▷ -test_automorphism_property_of_group G \
51  ▷ ▷ -end \

```

52

creates the set of all labeled spreads in $\text{PG}(3, 2)$. It then creates the graph of disjointness on the set of spreads. Afterwards, it creates the action of the projective group on the set of all labeled spreads in $\text{PG}(3, 2)$. Finally, it tests whether the group preserves the graph.

The command

Code Snippet 701

```

1   Petersen_graph_distance_poset:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Gamma -graph -Johnson 5 2 0 -end \
4   ▷ ▷ -with Gamma -do \
5   ▷ ▷ ▷ -graph_theoretic_activity -create_distance_poset 0 \
6   ▷ ▷ -end
7

```

computes the poset of neighborhood sets of the Petersen graph with respect to vertex 0.

The command

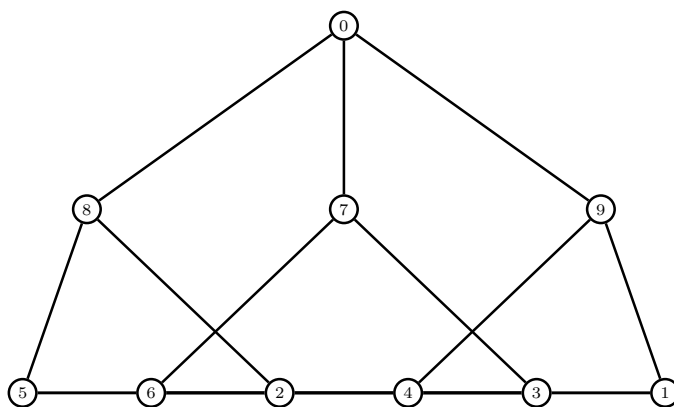
Code Snippet 702

```

1   Petersen_graph_distance_poset_draw:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -draw_layered_graph \
4   ▷ ▷ ▷ Johnson_5_2_0_distance_poset.layered_graph \
5   ▷ ▷ ▷ -place \
6   ▷ ▷ ▷ -radius 250 -embedded \
7   ▷ ▷ ▷ -nodes \
8   ▷ ▷ ▷ -line_width 1.1 -x_stretch 1.4 -scale 0.25 \
9   ▷ ▷ -end
10  ▷ $(PDFLATEX) Johnson_5_2_0_distance_poset_draw.tex
11  ▷ $(OPEN) Johnson_5_2_0_distance_poset_draw.pdf
12

```

produces a drawing of the poset, shown below:



The command

Code Snippet 703

```

1   Petersen_test_if_distance_regular:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Gamma -graph -Johnson 5 2 0 -end \
4   ▷ ▷ -test_if_distance_regular_graph \
5   ▷ ▷ Johnson_5_2_0_distance_poset.layered_graph
6

```

tests whether the Petersen graph is distance regular. The answer is positive.

The command

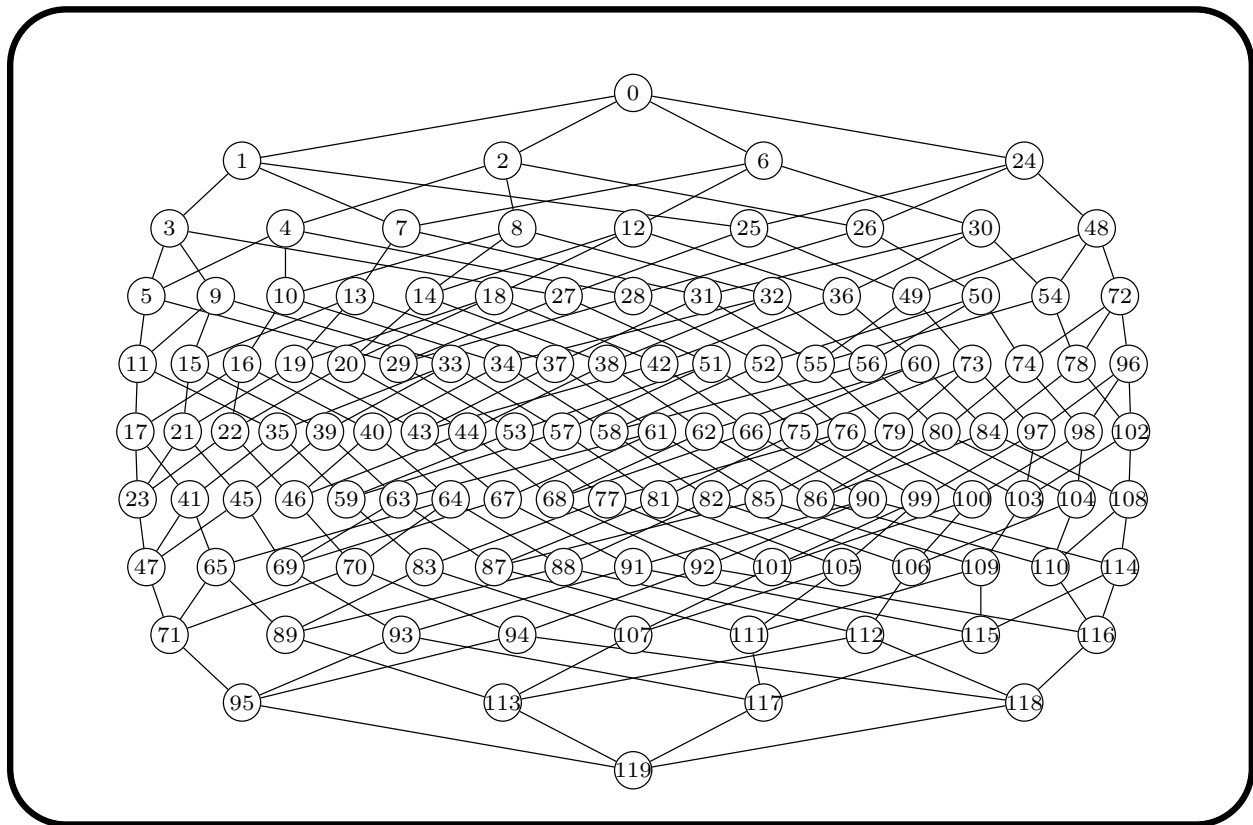
Code Snippet 704

```

1   Cayley_Sym5_distance_poset:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options -xin 1000000 -yin 1000000 \
4   ▷ ▷ ▷ -embedded -radius 10000 -nodes_empty -end \
5   ▷ ▷ -define S -vector -dense \
6   ▷ ▷ ▷ "1,0,2,3,4, 0,2,1,3,4, 0,1,3,2,4, 0,1,2,4,3" -end \
7   ▷ ▷ -define G -permutation_group -symmetric_group 5 \
8   ▷ ▷ -end \
9   ▷ ▷ -define Gamma -graph \
10  ▷ ▷ ▷ -Cayley_graph G S \
11  ▷ ▷ -end \
12  ▷ ▷ -with Gamma -do \
13  ▷ ▷ ▷ -graph_theoretic_activity -create_distance_poset 0 \
14  ▷ ▷ -end
15  ▷ $(ORBITER) -v 3 \
16  ▷ ▷ -draw_layered_graph \
17  ▷ ▷ ▷ Cayley_graph_Sym_5_distance_poset.layered_graph \
18  ▷ ▷ ▷ -place \
19  ▷ ▷ ▷ -radius 250 -embedded \
20  ▷ ▷ ▷ -nodes \
21  ▷ ▷ ▷ -line_width 0.4 -x_stretch 1.4 -scale 0.25 \
22  ▷ ▷ -end
23  ▷ $(PDFLATEX) Cayley_graph_Sym_5_distance_poset_draw.tex
24  ▷ $(OPEN) Cayley_graph_Sym_5_distance_poset_draw.pdf
25

```

computes the neighborhood poset for the Coxeter group $\text{Sym}(5)$. This poset is related to the Bruhat order. The following drawing is produced:



Classifying Graphs		
Command	Arguments	Purpose
-regular	r	Regular of degree r
-n	n	Number of vertices n
-girth	d	Girth at least d
-tournament		Classify tournaments.
-no_transmitter		Tournament without transmitter (requires -tournament)
-test_multi_edge		
-identify		
-depth		

Table 13.6: Classifying Graphs

Activities after Graph Classification		
Command	Arguments	Purpose
-draw_level_graph	k	
-draw_graphs		
-list_graphs_at_level	min max	
-draw_graphs_at_level	k	
-draw_options		
-recognize_graphs_from_adjacency_matrix_csv		

Table 13.7: Activities after Graph Classification

13.3 Classification of Graphs and Tournaments

Table 13.6 lists the Orbiter commands to classify graphs and tournaments. Table 13.7 lists activities after the classification has been completed.

The following command classifies all graphs on 5 vertices:

Code Snippet 705

```

1  graph.classify_5:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4  ▷ ▷ -define Do -draw_options \
5  ▷ ▷ ▷ -radius 300 -nodes_empty \
6  ▷ ▷ ▷ -line_width 1.5 \
7  ▷ ▷ ▷ -scale 0.1 \
8  ▷ ▷ -end \
9  ▷ ▷ -define GC -graph_classification \
10 ▷ ▷ ▷ -n 5 \
11 ▷ ▷ ▷ -poset_classification_control \
12 ▷ ▷ ▷ -problem_label graphs.v5 \
13 ▷ ▷ ▷ -depth 10 \
14 ▷ ▷ ▷ -draw_options Do \

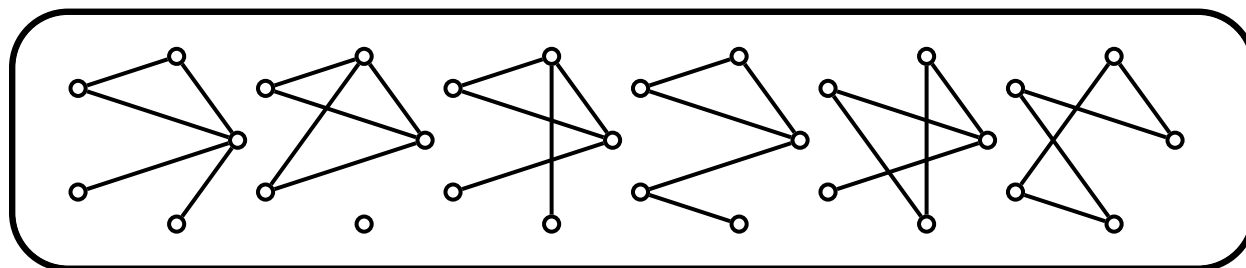
```

```

15   ▷ ▷ ▷ -end \
16   ▷ ▷ -end \
17   ▷ ▷ -with GC -do \
18   ▷ ▷ -graph_classification.activity \
19   ▷ ▷ ▷ -list_graphs_at_level 5 5 \
20   ▷ ▷ -end \
21   ▷ ▷ -with GC -do \
22   ▷ ▷ -graph_classification.activity \
23   ▷ ▷ ▷ -draw_options Do \
24   ▷ ▷ ▷ -draw_graphs_at_level 5 \
25   ▷ ▷ -end \
26   ▷ ▷ -print_symbols
27   ▷ $(PDFLATEX) graphs.v5_level_5_reps.tex
28   ▷ $(OPEN) graphs.v5_level_5_reps.pdf
29   ▷ #$(PDFLATEX) graphs.v5_poset.tex
30   ▷ #$(OPEN) graphs.v5_poset.pdf
31

```

After the classification, the graphs with 5 edges are shown.



The next command classifies all tournaments on 4 vertices:

Code Snippet 706

```

1   tournament_classify_4:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 400 \
5   ▷ ▷ ▷ -line_width 2 -scale 0.10 \
6   ▷ ▷ -end \
7   ▷ ▷ -define GC -graph_classification \
8   ▷ ▷ ▷ -n 4 -tournament \
9   ▷ ▷ ▷ -poset_classification_control \
10  ▷ ▷ ▷ ▷ -problem_label tournament_4 \
11  ▷ ▷ ▷ ▷ -depth 6 \
12  ▷ ▷ ▷ ▷ -draw_options Do \
13  ▷ ▷ ▷ -end \
14  ▷ ▷ -end \
15  ▷ ▷ -with GC -do \
16  ▷ ▷ -graph_classification.activity \
17  ▷ ▷ ▷ -draw_options Do \
18  ▷ ▷ ▷ -draw_graphs_at_level 6 \
19  ▷ ▷ -end \

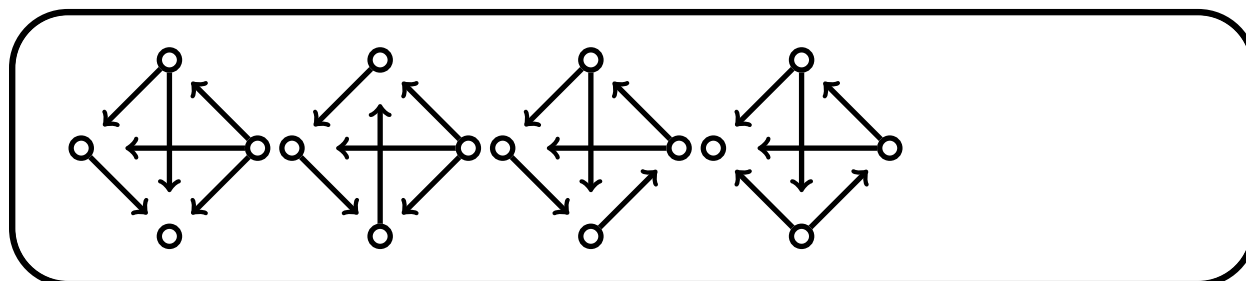
```

```

20  ▷ ▷ -print_symbols
21  ▷ $(PDFLATEX) tournament_4_level_6_reps.tex
22  ▷ $(OPEN) tournament_4_level_6_reps.pdf
23  ▷
24  # error after
25  #do_orbiter_session after execute_command_line
26

```

There are four tournaments, shown below:



The next command classifies all cubic graphs on 8 vertices:

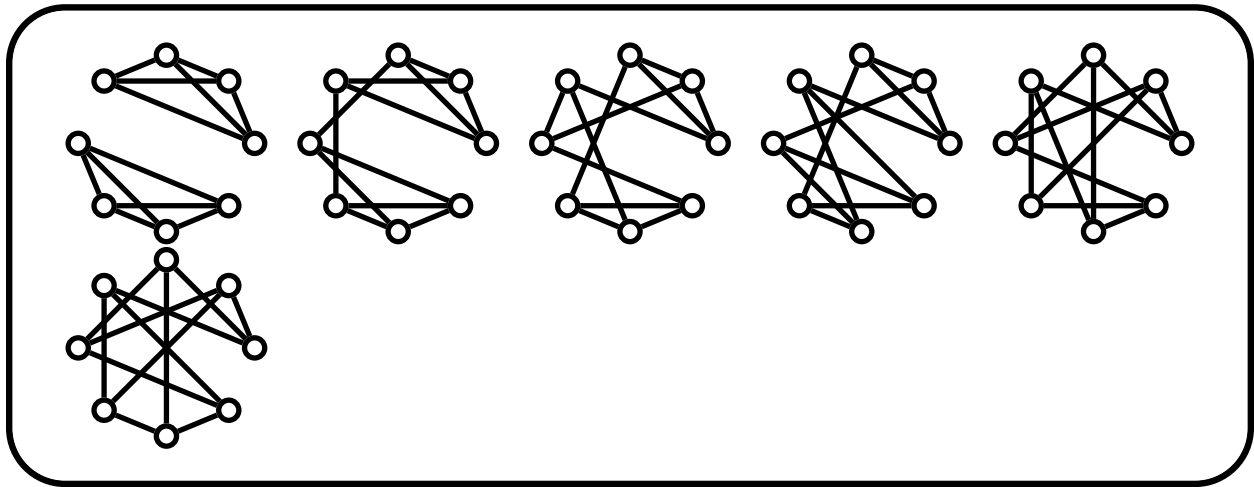
Code Snippet 707

```

1  graph_classify_8_r3:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define Do -draw_options \
4  ▷ ▷ ▷ -radius 400 \
5  ▷ ▷ ▷ -line_width 2 -scale 0.10 \
6  ▷ ▷ -end \
7  ▷ ▷ -define GC -graph_classification \
8  ▷ ▷ ▷ -n 8 -regular 3 \
9  ▷ ▷ ▷ -poset_classification_control \
10 ▷ ▷ ▷ ▷ -problem_label graphs_v8_r3 \
11 ▷ ▷ ▷ ▷ -depth 12 \
12 ▷ ▷ ▷ ▷ -draw_options Do \
13 ▷ ▷ ▷ -end \
14 ▷ ▷ -end \
15 ▷ ▷ -with GC -do \
16 ▷ ▷ -graph_classification_activity \
17 ▷ ▷ ▷ -draw_options Do \
18 ▷ ▷ ▷ -draw_graphs_at_level 12 \
19 ▷ ▷ -end \
20 ▷ ▷ -print_symbols
21 ▷ #$(PDFLATEX) graphs_v8_r3_poset_lvl_12.tex
22 ▷ #$(OPEN) graphs_v8_r3_poset_lvl_12.pdf
23

```

There are six cubic graphs. The graph drawings are shown below:



Chapter 14

Design Theory

14.1 Creating Designs

A design is an incidence structure of points and blocks. The incidence matrix of a design has rows corresponding to the points and columns corresponding to the blocks. An entry in a certain row and column is one if and only if the point associated with the row is contained in the block associated with the column, zero otherwise. A decomposition of the design is a partition of the points and blocks such that each class consists either exclusively of points or exclusively of blocks.

A decomposition is point-tactical if for all points, the number of incident lines in the j th block class depends only on the class of the point. If the point belongs to class i , this number is denoted as a_{ij} . A decomposition is block-tactical if for all blocks, the number of incident points in the i th point class depends only on the class of the block. If the block belongs to class j , this number is denoted as b_{ij} .

A projective plane of order n is a design with $n^2 + n + 1$ points and equally many blocks (also called lines), each of size $n+1$ such that any two points lie in exactly one block and any two blocks have exactly one point in common. Projective planes are known to exist for all $n = q$ which are a power of a prime. This follows from a construction which utilizes the projective geometry $PG(2, q)$. Points are the one-dimensional subspaces of $\mathbb{F}_q^3$, blocks are the two-dimensional subspaces of $\mathbb{F}_q^3$, and incidence is natural (inclusion of subspaces). The automorphism group of this design is the collineation group of the projective space. Projective planes other than these exist, though none are known when n is not a prime power. The number of lines through a point equals the number of points on a line. The fact that these numbers exist imply that there is a tactical decomposition. Namely, the trivial decomposition with two classes, one containing all points and one containing all lines. The structure constants of the decomposition are the numbers just described.

Table 14.1 lists Orbiter commands to create designs. Table 14.2 lists Orbiter activities for designs. The command

Code Snippet 708

```
1    design_PG_2_3:
2    ▷ $(ORBITER) -v 8 \
3    ▷ ▷ -define F -finite_field -q 3 -end \
4    ▷ ▷ -define D -design -field F -family PG_2.q -end \
5    ▷ ▷ -with D -do \
6    ▷ ▷ ▷ -design_activity \
7    ▷ ▷ ▷ ▷ -export_flags \
8    ▷ ▷ -end \
9    ▷ ▷ -with D -do \
10   ▷ ▷ ▷ -design_activity \
```

Creating Designs		
Command	Arguments	Purpose
-label	txt tex	Set a label of the design. One label is for text output. The other label is for latex output.
-field	F	Some constructions of designs require a finite field. Set the field to be F .
-catalogue	iso	Recall a design of a given family from the Orbiter catalogue. The parameter iso is the isomorphism type in the catalogue. It is zero-based.
-family	familyname	Select a family for a design to be constructed. The following family names are supported: PG_2_q for the projective plane PG(2, q).
-sporadic	keyword	Select a sporadic design to be constructed. The following keywords are recognized: Baker_1978 (Baker's elliptic semi-plane), Mathon_1987 (Mathon's elliptic semi-plane).
-from_incidence_matrix	matrix	Create a design from the given incidence matrix. The incidence matrix is an orbiter matrix object. The number of ones in each column of the matrix must be constant.
-from_graph	graph	Create a design from adjacency matrix of the given graph. The graph is an orbiter graph object. The graph must be regular.
-list_of_base_blocks	G fname col	Create a design by spanning the orbits of the base blocks given in column col in file fname under the group G. See Section 14.4.
-list_of_blocks_coded	v k L	Create a design on v points and with block size k from the coded list of sets L.
-list_of_sets_coded	v label	Create a design on v points and with block codes in the list L.
-list_of_blocks_coded_from_file	v k file	Create a design on v points and with block size k from the coded list of sets read from the given file.
-list_of_blocks_from_file	v fname column	Create a design on v points and with blocks from the given column in the given file.
-wreath_product_designs	n k	Create a class of designs with wreath product action.
-linear_space_from_latin_square	matrix	Create a linear space associated with a Latin Square given as matrix. The matrix must be an Orbiter object of type vector.
-orbit_on_sets	Orbit sz idx	Create a design from an orbit on subsets. Orbit must be a subset classification object. The orbit selected is given by its local index (idx) among the orbits on subsets of size sz.
-no_group		
-block_partition	sz	Partition the set of blocks in classes of the given size.

Table 14.1: Creating Designs

Design Activities		
Command	Arguments	Purpose
-load_table	label group H_label H_group_order H_gens l	
-canonical_form	options	
-extract_solutions_by_index_csv	label group fname- solutions-in fname- solutions-out prefix	Used by large set desargues.
-extract_solutions_by_index_txt	label group fname- solutions-in fname- solutions-out prefix	
-export_flags		Export the flags of the incidence to file.
-export_incidence_matrix		Export the incidence matrix as csv file. The adjacency matrix will be written row-by-row on v rows.
-export_incidence_matrix_as_flags		Export the incidence matrix as csv file. The whole design will be coded in one row.
-export_incidence_matrix_latex	options	Export the incidence matrix as latex file. Used the options for drawing incidence matrices.
-intersection_matrix		Compute the intersection matrix. If -save is given, writes the intersection matrix to file.
-save		See -intersection_matrix.
-export_blocks		Export the incidence matrix as blocks file
-row_sums		Compute the row sums of the incidence matrix.
-tactical_decomposition		Compute the tactical decomposition.
-orbits_on_blocks	s control	Computes orbits on blocks of size s (s -subsets of the set of blocks). Requires a poset classification control structure.
-one_point_extension	idx control	Construct the one-point extension graph from the incidence matrix of a design, using a relation on pairs of blocks. Idx is the orbit on pairs of blocks that defines the relation. Points in the graph correspond to points of the design, blocks of the design, and to the special point.

Table 14.2: Design Activities

```

11  ▷ ▷ ▷ ▷ -tactical_decomposition \
12  ▷ ▷ -end
13

```

creates the design PG(2, 3).

We have created the following design:

{19, 79, 126, 219, 256, 284, 371, 392, 465, 541, 619, 627, 653}

The stabilizer is generated by:

Strong generators for a group of order 5616:

$$\begin{bmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}, \\
 \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

1,0,0,0,2,0,0,0,2,
 1,0,0,0,2,0,0,0,1,
 1,0,0,0,1,0,1,0,1,
 1,0,0,0,1,0,0,1,1,
 1,0,0,0,0,1,0,1,0,
 0,1,0,1,0,0,0,0,1,

The blocks of the design are encoded in the lexicographic ordering of k -subsets (here $k = 4$). The program also displays the tactical decomposition scheme of the design, which is

$$\begin{array}{c|c} \rightarrow & 13_1 \\ \hline 13_0 & 4 \end{array} \quad \begin{array}{c|c} \downarrow & 13_1 \\ \hline 13_0 & 4 \end{array}$$

In Section 17.4, we will show how to compute further properties of the design.

The command

Code Snippet 709

```

1  wreath_product_designs_n4_k2.inc.txt:
2  ▷ $(ORBITER) -v 8 \
3  ▷ ▷ -define D -design -wreath_product_designs 4 2 -end \
4  ▷ ▷ -with D -do \
5  ▷ ▷ ▷ -design_activity \
6  ▷ ▷ ▷ ▷ -export_flags \
7  ▷ ▷ -end
8

```

creates a design on 8 points invariant under the wreath product $\text{Sym}(4) \wr \text{Sym}(2)$. The design has 12 blocks of size 4. The command

Code Snippet 710

```
1 wreath_product_designs_n8_k6.inc.txt:
2 ▷ $(ORBITER) -v 8 \
3 ▷ ▷ -define D -design -wreath_product_designs 8 6 -end \
4 ▷ ▷ -with D -do \
5 ▷ ▷ ▷ -design_activity \
6 ▷ ▷ ▷ ▷ -export_flags \
7 ▷ ▷ -end
8
```

creates a design on 16 points invariant under the wreath product $\text{Sym}(8) \wr \text{Sym}(2)$. The design has 3920 blocks of size 6. We will compute the automorphism groups of these two designs in Section 17.3.

A design can be created by listing the blocks in coded fashion. The codes correspond to the ranking of k -subsets of the v set which is the underlying set of the design. The following command creates a configuration $15_4 20_3$ from its set of blocks.

Code Snippet 711

```
1 design_20:
2 ▷ $(ORBITER) -v 8 \
3 ▷ ▷ -define D -design -list_of_blocks_coded 15 3 \
4 ▷ ▷ "61, 67, 72, 76, 129, 147, 152, 156, 197, 204, 215, 224, 249, 261, 267, 276,
5 ▷ ▷ 296, 303, 309, 319" \
6 ▷ ▷ -end \
7 ▷ ▷ -with D -do \
8 ▷ ▷ ▷ -design_activity \
9 ▷ ▷ ▷ ▷ -export_flags \
10 ▷ ▷ -end
10
```

It is also possible to create a design by listing the blocks as sets. Here is an example. At first, we create a makefile variable for a csv file containing the blocks:

Code Snippet 712

```
1 GEO_BLOCKS_600="Row,Block\n\
2 0,\"0, 5, 14\", \n\
3 1,\"1, 6, 10\", \n\
4 2,\"2, 7, 11\", \n\
5 3,\"3, 8, 12\", \n\
6 4,\"4, 9, 13\", \n\
7 5,\"0, 6, 13\", \n\
8 6,\"1, 7, 14\", \n\
9 7,\"2, 8, 10\", \n\
10 8,\"3, 9, 11\", \n\
11 9,\"4, 5, 12\", \n\
12 10,\"0, 7, 12\", \n\
13 11,\"1, 8, 13\", \n\
```

```

14 12,\"2, 9, 14\", \\n\\
15 13,\"3, 5, 10\", \\n\\
16 14,\"4, 6, 11\", \\n\\
17 15,\"0, 8, 11\", \\n\\
18 16,\"1, 9, 12\", \\n\\
19 17,\"2, 5, 13\", \\n\\
20 18,\"3, 6, 14\", \\n\\
21 19,\"4, 7, 10\", \\n\\
22 20,\"0, 9, 10\", \\n\\
23 21,\"1, 5, 11\", \\n\\
24 22,\"2, 6, 12\", \\n\\
25 23,\"3, 7, 13\", \\n\\
26 24,\"4, 8, 14\", \\n\\
27 END\\n\"
28

```

The following command creates the csv file from the makefile variable using the unix echo command. The sed command is necessary to remove space characters that are inserted by the echo command. The Orbiter command then reads the list of blocks from the csv file. After that, it creates the design from the list of blocks. As we will see below, the design has an automorphism group of order 600 and is related to the Latin Square of order 5 associated with the cyclic group of order 5.

Code Snippet 713

```

1  design_600:
2  ▷ echo $(GEO_BLOCKS_600) | sed 's/ //' >geo_600_blocks.csv
3  ▷ $(ORBITER) -v 8 \
4  ▷ ▷ -define D -design -list_of_blocks_from_file \
5  ▷ ▷ ▷ 15 geo_600_blocks.csv Block \
6  ▷ ▷ -end \
7  ▷ ▷ -with D -do \
8  ▷ ▷ ▷ -design_activity \
9  ▷ ▷ ▷ ▷ -export_flags \
10 ▷ ▷ -end \
11 ▷ ▷ -with D -do \
12 ▷ ▷ ▷ -design_activity \
13 ▷ ▷ ▷ ▷ -save \
14 ▷ ▷ ▷ ▷ -intersection_matrix \
15 ▷ ▷ -end
16

```

The command also computes the intersection matrix of the design. The design is a transversal design. The configuration graph is three copies of the complete graph K_5 .

How can we convince ourselves that the design created previously is indeed associated to a certain Latin Square? There are exactly two Latin Squares of order 5. One is the Cayley table of the cyclic group of order 5, the other is the multiplication table of the non-associative loop of order 5, see Table 14.3.

We create makefile variables for the two tables:

Latin Squares of Order 5				
Cyclic 5				
0	1	2	3	4
1	2	3	4	0
2	3	4	0	1
3	4	0	1	2
4	0	1	2	3

Nonassociative Loop				
0	1	2	3	4
1	0	4	2	3
2	4	3	1	0
3	2	0	4	1
4	3	1	0	2

Table 14.3: Latin Squares of Order 5

Code Snippet 714

```
1 LSQ_5A_TABLE="0,1,2,3,4,1,2,3,4,0,2,3,4,0,1,3,4,0,1,2,4,0,1,2,3"
2
```

Code Snippet 715

```
1 LSQ_5B_TABLE="0,1,2,3,4,1,0,4,2,3,2,4,3,1,0,3,2,0,4,1,4,3,1,0,2"
2
```

The next two commands create the associated linear spaces:

Code Snippet 716

```
1 LSQ_5A:
2 ▷ $(ORBITER) -v 8 \
3 ▷ ▷ -define LSQ5A -vector -format 5 -dense $(LSQ_5A_TABLE) -end \
4 ▷ ▷ -define D -design -linear_space_from_latin_square \
5 ▷ ▷ ▷ LSQ5A \
6 ▷ ▷ -end \
7 ▷ ▷ -with D -do \
8 ▷ ▷ ▷ -design_activity \
9 ▷ ▷ ▷ ▷ -export_flags \
10 ▷ ▷ -end
11
```

Code Snippet 717

```
1 LSQ_5B:
2 ▷ $(ORBITER) -v 8 \
3 ▷ ▷ -define LSQ5B -vector -format 5 -dense $(LSQ_5B_TABLE) -end \
```

```

4    ▷ ▷ -define D -design -linear_space_from_latin_square \
5    ▷ ▷ ▷ LSQ5B \
6    ▷ ▷ -end \
7    ▷ ▷ -with D -do \
8    ▷ ▷ ▷ -design_activity \
9    ▷ ▷ ▷ ▷ -export_flags \
10   ▷ ▷ -end
11

```

Finally, we perform the canonical form algorithm on these two linear spaces. We note that any isomorphism between the Latin Squares corresponds to an isomorphism between the associated linear spaces. Likewise, any automorphism of a given Latin Square corresponds to an automorphism of the associated linear space.

Code Snippet 718

```

1    LSQ_5_c:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define Doinc -draw_incidence_structure_options \
4    ▷ ▷ ▷ -width 60 -width_10 6 -end \
5    ▷ ▷ -define C -combinatorial_object \
6    ▷ ▷ ▷ -label LSQ_5 LSQ\5 \
7    ▷ ▷ ▷ -file_of_incidence_geometries latin_square_order5_LSQ5A_flags.txt 15 28 90 \
8    ▷ ▷ ▷ -file_of_incidence_geometries latin_square_order5_LSQ5B_flags.txt 15 28 90 \
9    ▷ ▷ -end \
10   ▷ ▷ -with C -do \
11   ▷ ▷ -combinatorial_object_activity \
12   ▷ ▷ ▷ -canonical_form \
13   ▷ ▷ ▷ ▷ -save_ago \
14   ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
15   ▷ ▷ ▷ ▷ -nauty_control -nauty_log "LSQ5_nauty_log.txt" -end \
16   ▷ ▷ ▷ -end \
17   ▷ ▷ -end \
18   ▷ ▷ -with C -do \
19   ▷ ▷ -combinatorial_object_activity \
20   ▷ ▷ ▷ -post_processing \
21   ▷ ▷ -end \
22   ▷ ▷ -with C -do \
23   ▷ ▷ -combinatorial_object_activity \
24   ▷ ▷ ▷ -report \
25   ▷ ▷ ▷ ▷ -export_flag_orbits \
26   ▷ ▷ ▷ ▷ -show_TDO \
27   ▷ ▷ ▷ ▷ -show_TDA \
28   ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
29   ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
30   ▷ ▷ ▷ -end \
31   ▷ ▷ -end
32   ▷ $(PDFLATEX) LSQ_5_classification.tex
33   ▷ $(OPEN) LSQ_5_classification.pdf
34

```

This establishes the fact that the geometries are not isomorphic, and it also produces the automorphism

groups of each geometry. It turns out that the Latin Square of order 5 arising from the cyclic group has a symmetry group of order 600. The second Latin Square arising from the loop has a symmetry group of order 72. Orbiter produces a report listing generators for each group. For more information about the canonical form of designs, see Section 17.4. Since there is only one linear space of Latin Square 5 type with a group of order 600, the object constructed earlier must be that same space which is associated to the Cayley table of the cyclic group of order 5.

14.2 Assuming Symmetry

One way to construct combinatorial designs is by assuming a suitable group of symmetries. Let us consider an example. It is possible to construct t -(v, k, λ) designs invariant under a permutation group G acting on a set V with $|V| = v$ as follows: Classify the orbits of G on subsets of size k and less. Construct a matrix which describes the relationship between the orbits on t -sets and the orbits on k -sets. This matrix is often referred to as the Kramer-Mesner matrix (cf. [46]). For each pair of t -orbit and k -orbit, for instance with representatives T and K , say, we count the number of elements in the orbit of K which contain T . The rows of the matrix are in correspondence to the t -orbits, while the columns are in correspondence to the k -orbits. The matrix entry a_{ij} is the number just defined where T is the representative of the i -th orbit on t -sets, and where K is the representative of the j -th orbit on k -sets. Let $M_{t,k}(G)$ be the Kramer-Mesner matrix for the group $G \leq \text{Sym}(V)$ defined in this way. The t -(v, k, λ) designs invariant under G are in one-to-one correspondence to the solutions of

$$M_{t,k}(G) \cdot \mathbf{x} = \lambda \mathbf{1},$$

where $\mathbf{x}$ is a column vector of zeros and ones and $\mathbf{1}$ is the column vector of all ones. The length of $\mathbf{x}$ is the number of k -orbits of G on V , while the length of $\mathbf{1}$ is the number of t -orbits of G on V . Any vector $\mathbf{x}$ satisfying the matrix equation corresponds to a design invariant under G . Simply take the blocks of the design to be the union of those orbits of G on k -subsets whose associated entry in $\mathbf{x}$ is one. We assume the group $\text{PTL}(2, 32)$ in the action on points of the projective line $\text{PG}(1, 32)$ over the field $\mathbb{F}_{32}$. The parameters of the design are 7-(33, 8, 10), that is, each 7-subset of $\text{PG}(1, 32)$ is covered exactly 10 times by the chosen 8-subsets comprising the design. Let us consider the following Orbiter command:

Code Snippet 719

```

1   KM_PGGL_2_32:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4   ▷ ▷ -define Do -draw_options -embedded -sideways -radius 50 \
5   ▷ ▷ ▷ -scale 0.5 -line_width 0.3 \
6   ▷ ▷ -end \
7   ▷ ▷ -define Control -poset_classification_control \
8   ▷ ▷ ▷ -problem_label KM_PGGL_2_32 \
9   ▷ ▷ ▷ -W -depth 8 \
10  ▷ ▷ ▷ -draw_options Do \
11  ▷ ▷ -end \
12  ▷ ▷ -define G -linear_group -PGGL 2 32 -end \
13  ▷ ▷ -define Orb -orbits \
14  ▷ ▷ ▷ -group G \
15  ▷ ▷ ▷ -group_action G \
16  ▷ ▷ ▷ -generators G \
17  ▷ ▷ ▷ -on_subsets 8 Control \
18  ▷ ▷ -end \
19  ▷ ▷ -with Orb -do -orbits_activity \
20  ▷ ▷ ▷ -Kramer_Mesner_matrix 7 8 \
21  ▷ ▷ -end \
22  ▷ ▷ -define ro \
23  ▷ ▷ ▷ -poset_classification_report_options \
24  ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
25  ▷ ▷ ▷ ▷ -draw_options Do \
26  ▷ ▷ -end \
27  ▷ ▷ -with Orb -do -orbits_activity \
28  ▷ ▷ ▷ -report "" \
29  ▷ ▷ ▷ -report_options ro \

```

```

30  ▷ ▷ -end
31  ▷ $(ORBITER) -v 2 -draw_matrix \
32  ▷ ▷ -input_csv_file KM_PGGL_2_32_KM_7_8.csv \
33  ▷ ▷ -box_width 20 -bit_depth 24 \
34  ▷ ▷ -partition 3 32 97 -end
35  ▷ $(PDFLATEX) KM_PGGL_2_32_poset_lvl_8.tex
36  ▷ $(OPEN) KM_PGGL_2_32_poset_lvl_8.pdf
37  ▷ $(OPEN) KM_PGGL_2_32_KM_7_8_draw.bmp
38

```

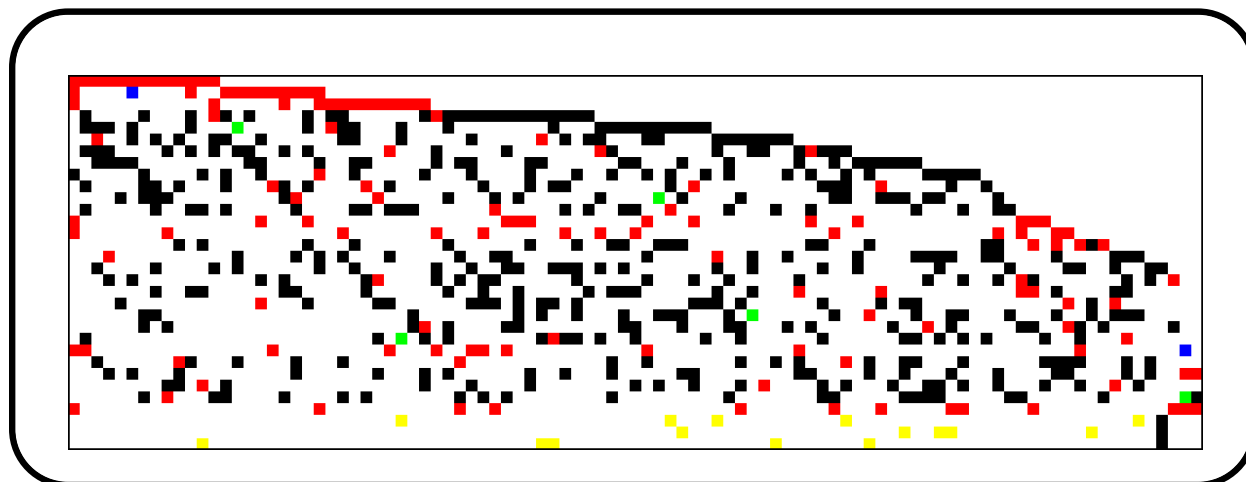
The first command creates the group $\text{PTL}(2, 32)$ and computes the Kramer-Mesner matrix

$$M_{7,8}(\text{PTL}(2, 32)).$$

The number of 7-orbits is 32. The number of 8-orbits is 97. Correspondingly, the Kramer-Mesner matrix has 32 rows and 97 columns. The matrix is stored in the csv-file

KM_PGGL_2_32_KM_7_8.csv.

The second command produces the graphical representation of the matrix shown below



(different colors represent different values of the entries in the matrix). The command

Code Snippet 720

```

1  KM_PGGL_2_32_solve:
2  ▷ $(ORBITER) -v 4 \
3  ▷ ▷ -define A -vector -file KM_PGGL_2_32_KM_7_8.csv -end \
4  ▷ ▷ -define D -diophant \
5  ▷ ▷ -label "KM_PGGL_2_32_KM_7_8.system" \
6  ▷ ▷ -coefficient_matrix A \
7  ▷ ▷ -RHS_constant "EQ=10" \
8  ▷ ▷ -x_min_global 0 -x_max_global 1 \
9  ▷ ▷ -end \
10 ▷ ▷ -with D -do \
11 ▷ ▷ ▷ -diophant_activity -solve_mckay \
12 ▷ ▷ -end
13

```

creates the diophantine system associated with the Kramer-Mesner matrix and solves it. It performs a complete enumeration of all solutions by solving the system and producing the solution vectors $\mathbf{x}$ corresponding to the designs.

Let us construct configurations with an assumed symmetry group. Suppose we are interested in configurations with $v = 15$ points, and 25 blocks of size 3. We assume a group of order 5 acting semi regularly. In cycle notation, the generator may be taken as

$$(0, 1, 2, 3, 4)(5, 6, 7, 8, 9)(10, 11, 12, 13, 14).$$

In two row notation, we write

$$\begin{bmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 \\ 1 & 2 & 3 & 4 & 0 & 6 & 7 & 8 & 9 & 5 & 11 & 12 & 13 & 14 & 10 \end{bmatrix}$$

A makefile variable will hold the generator (the second row in the two-row notation):

Code Snippet 721

```
1 GEN_C5="1,2,3,4,0,6,7,8,9,5,11,12,13,14,10"
2
```

Next, we need to compute the orbits on pairs and triples. We also need the Kramer-Mesner matrices

$$M_{1,3}, \quad M_{2,3}$$

Every point lies on 5 blocks, and every pair orbit is covered at most once. This leads to the system

$$M_{1,3}\mathbf{x} = 5 \cdot \mathbf{1}, \quad M_{2,3}\mathbf{x} \leq \mathbf{1}, \quad \mathbf{0} \leq \mathbf{x} \leq \mathbf{1},$$

where $\mathbf{0}$ is the all-zero vector and $\mathbf{1}$ is the all-one vector. The following command computes $M_{1,3}$:

Code Snippet 722

```
1 C5_KM_1.3:
2 ▷ $(ORBITER) -v 3 \
3 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4 ▷ ▷ -define gens -vector -dense $(GEN_C5) -end \
5 ▷ ▷ -define G -permutation_group -symmetric_group 15 \
6 ▷ ▷ ▷ -subgroup_by_generators "C5" 5 1 gens \
7 ▷ ▷ -end \
8 ▷ ▷ -define Do -draw_options -embedded -sideways -radius 50 \
9 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 \
10 ▷ ▷ -end \
11 ▷ ▷ -define Control -poset_classification_control \
12 ▷ ▷ ▷ -problem_label C5 -W -depth 3 \
13 ▷ ▷ ▷ -draw_options Do \
14 ▷ ▷ -end \
15 ▷ ▷ -define Orb -orbits \
16 ▷ ▷ ▷ -group G \
17 ▷ ▷ ▷ -group_action G \
18 ▷ ▷ ▷ -generators G \
19 ▷ ▷ ▷ -on_subsets 3 Control \
20 ▷ ▷ -end \
21 ▷ ▷ -with Orb -do -orbits_activity \
```

```

22   ▷ ▷ ▷ -Kramer_Mesner_matrix 1 3 \
23   ▷ ▷ -end \
24   ▷ ▷ -with Orb -do -orbits_activity \
25   ▷ ▷ ▷ -export_something set_orbits 3 \
26   ▷ ▷ -end \
27

```

The matrix has 3 rows and 91 columns. The command also exports the orbits on 3-subsets to a file with the name `C5_set_orbits_level_3.csv`. The following command computes $M_{2,3}$:

Code Snippet 723

```

1   C5_KM_2_3:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4   ▷ ▷ -define gens -vector -dense $(GEN_C5) -end \
5   ▷ ▷ -define G -permutation_group -symmetric_group 15 \
6   ▷ ▷ ▷ -subgroup_by_generators "C5" 5 1 gens \
7   ▷ ▷ -end \
8   ▷ ▷ -define Do -draw_options -embedded -sideways -radius 50 \
9   ▷ ▷ ▷ -scale 0.5 -line_width 0.3 \
10  ▷ ▷ -end \
11  ▷ ▷ -define Control -poset_classification_control \
12  ▷ ▷ ▷ -problem_label C5 -W -depth 3 \
13  ▷ ▷ ▷ -draw_options Do \
14  ▷ ▷ -end \
15  ▷ ▷ -define Orb -orbits \
16  ▷ ▷ ▷ -group G \
17  ▷ ▷ ▷ -group_action G \
18  ▷ ▷ ▷ -generators G \
19  ▷ ▷ ▷ -on_subsets 3 Control \
20  ▷ ▷ -end \
21  ▷ ▷ -with Orb -do -orbits_activity \
22  ▷ ▷ ▷ -Kramer_Mesner_matrix 2 3 \
23  ▷ ▷ -end
24

```

The matrix has 21 rows and 91 columns. The next command stacks the two matrices together, one on top of the other:

Code Snippet 724

```

1   C5_concatenate:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -csv_file_concatenate C5_KM_combined.csv 2 C5_KM_1_3.csv C5_KM_2_3.csv
4

```

Next, we need to solve the system. To this end, we use the following command.

Code Snippet 725

```

1   C5_solve:
2   ▷ $(ORBITER) -v 4 \
3   ▷ ▷ -define A -vector -file C5_KM_combined.csv -end \
4   ▷ ▷ -define D -diophant \
5   ▷ ▷ ▷ -label "C5_KM_combined.system" \
6   ▷ ▷ ▷ -coefficient_matrix A \
7   ▷ ▷ ▷ -x_min_global 0 \
8   ▷ ▷ ▷ -x_max_global 1 \
9   ▷ ▷ ▷ -RHS "mult=3,EQ=5" \
10  ▷ ▷ ▷ -RHS "mult=21,LE=1" \
11  ▷ ▷ -end \
12  ▷ ▷ -with D -do \
13  ▷ ▷ ▷ -diophant_activity -solve_mckay \
14  ▷ ▷ -end
15

```

Orbiter finds 21540 solutions in 61001 backtrack steps. Next, we need to assemble the designs by concatenating the set orbits on 3-subsets that are selected by the solution vector $\mathbf{x}$. Once the designs are created, we need to classify them up to isomorphism. We utilize the classification algorithm in Orbiter which is based on canonical forms computed by Nauty:

Code Snippet 726

```

1   C5_design_classify:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define C -combinatorial_object \
4   ▷ ▷ ▷ -label C5_config C5\_config \
5   ▷ ▷ ▷ -file_of_designs_through_block_orbits \
6   ▷ ▷ ▷ ▷ C5_KM_combined.system_sol.csv \
7   ▷ ▷ ▷ ▷ C5_set_orbits.level_3.csv \
8   ▷ ▷ ▷ ▷ 15 3 \
9   ▷ ▷ -end \
10  ▷ ▷ -with C -do \
11  ▷ ▷ -combinatorial_object_activity \
12  ▷ ▷ ▷ -canonical_form \
13  ▷ ▷ ▷ ▷ -save_ago \
14  ▷ ▷ ▷ ▷ -save_transversal \
15  ▷ ▷ ▷ ▷ -nauty_control -end \
16  ▷ ▷ ▷ -end \
17  ▷ ▷ -end \
18  ▷ ▷ -with C -do \
19  ▷ ▷ -combinatorial_object_activity \
20  ▷ ▷ ▷ -post_processing \
21  ▷ ▷ -end \
22  ▷ ▷ -define Doinc -draw_incidence_structure_options \
23  ▷ ▷ ▷ -width 60 -width_10 6 -end \
24  ▷ ▷ -with C -do \
25  ▷ ▷ -combinatorial_object_activity \
26  ▷ ▷ ▷ -report \
27  ▷ ▷ ▷ ▷ -show_incidence_matrices \
28  ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \

```



```

29   ▷ ▷ ▷ -end \
30   ▷ ▷ -end \
31   ▷ ▷ -print_symbols
32   ▷ $(PDFLATEX) C5_config_classification.tex
33   ▷ $(OPEN) C5_config_classification.pdf
34

```

A latex report is produced. Orbiter finds 42 isomorphism types of designs. The largest automorphism group associated with these designs has order 600.

The command

Code Snippet 727

```

1   Witt_design_on_22_points:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
4   ▷ ▷ -define Do -draw_options \
5   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6   ▷ ▷ -end \
7   ▷ ▷ -define code -vector \
8   ▷ ▷ ▷ -dense $(GOLAY_24_CODE_PROJECTIVE) \
9   ▷ ▷ -end \
10  ▷ ▷ -define Do_embedded -draw_options -embedded -sideways -radius 50 \
11  ▷ ▷ ▷ -scale 0.5 -line_width 0.3 \
12  ▷ ▷ -end \
13  ▷ ▷ -define Control2 -poset_classification_control \
14  ▷ ▷ ▷ -problem_label M24_pairs -W -depth 2 \
15  ▷ ▷ ▷ -draw_options Do \
16  ▷ ▷ -end \
17  ▷ ▷ -define F -finite_field -q 2 -end \
18  ▷ ▷ -define G -linear_group -PGL 12 2 \
19  ▷ ▷ -subgroup_by_generators "M24" "244823040" 12 \
20  ▷ ▷ ▷ $(GOLAY_24_CODE_AUT_GENs) \
21  ▷ ▷ -end \
22  ▷ ▷ -define M24_perm -modified_group \
23  ▷ ▷ ▷ -from G -restricted_action code \\on\\code \
24  ▷ ▷ -end \
25  ▷ ▷ -define M22_perm -modified_group \
26  ▷ ▷ ▷ -from M24_perm \
27  ▷ ▷ ▷ -set_stabilizer 0,1 Control2 \
28  ▷ ▷ -end \
29  ▷ ▷ -define pts -set -loop 2 22 1 -end \
30  ▷ ▷ -define M22 -modified_group -from M22_perm \
31  ▷ ▷ ▷ -restricted_action pts M22 \
32  ▷ ▷ -end \
33  ▷ ▷ -with M22 -do \
34  ▷ ▷ -group_theoretic_activity \
35  ▷ ▷ ▷ -report Do \
36  ▷ ▷ -end \
37  ▷ ▷ -define Control -poset_classification_control \
38  ▷ ▷ ▷ -problem_label M22 -W -depth 6 \

```

```

39  ▷ ▷ ▷ -draw_options Do \
40  ▷ ▷ -end \
41  ▷ ▷ -define Orb -orbits \
42  ▷ ▷ ▷ -group M22 \
43  ▷ ▷ ▷ -group_action M22 \
44  ▷ ▷ ▷ -generators M22 \
45  ▷ ▷ ▷ -on_subsets 6 Control \
46  ▷ ▷ -end \
47  ▷ ▷ -with Orb -do -orbits_activity \
48  ▷ ▷ ▷ -Kramer_Mesner_matrix 3 6 \
49  ▷ ▷ -end \
50  ▷ ▷ -define ro \
51  ▷ ▷ ▷ -poset_classification_report_options \
52  ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
53  ▷ ▷ ▷ ▷ -draw_options Do_embedded \
54  ▷ ▷ -end \
55  ▷ ▷ -with Orb -do -orbits_activity \
56  ▷ ▷ ▷ -report "" \
57  ▷ ▷ ▷ -report_options ro \
58  ▷ ▷ -end \
59  ▷ ▷ -define D -design -orbit_on_sets \
60  ▷ ▷ ▷ Orb 6 1 \
61  ▷ ▷ -end \
62  ▷ ▷ -with D -do \
63  ▷ ▷ ▷ -design_activity \
64  ▷ ▷ ▷ ▷ -export_incidence_matrix \
65  ▷ ▷ -end \
66  ▷ ▷ -with D -do \
67  ▷ ▷ ▷ -design_activity \
68  ▷ ▷ ▷ ▷ -intersection_matrix \
69  ▷ ▷ -end
70

```

creates the Witt design on 22 points.

Creating Large Sets with Assumed Symmetry		
Command	Arguments	Purpose
-H	go gens	Specify the assumed group H .
-N	go gens	Specify the normalizer N of the group H .
-report		Create a latex report.
-prefix	prefix	Set a prefix for output files.
-selected_orbit_length	l	Select orbit length l .

Table 14.4: Creating Large Sets with Assumed Symmetry

Large Set Activity		
Command	Arguments	Purpose
-normalizer_on_orbits_of_ a_given_length	l nb_orbits control	
-create_graph_on_orbits_of_length	fname l	
-create_graph_on_orbits_ of_length_based_on_N_orbits	fname-mask l nb_orbits r m	
-read_solution_file	l fname	

Table 14.5: Large Set Activity

14.3 Design Theory – Large Sets

Fix a set of size v and an integer k with $1 < k < v$. Is it possible to partition the set of k -subsets of v into designs, all with the same parameters? If so, the resulting set of designs is called a large set (of designs). So, a large set of designs is a set of designs, all of the same types, on a fixed v -element set whose block sets are pairwise disjoint and partition the set of k -subsets. Let us see how Orbiter can help construct and classify small large sets.

Tables 14.4 shows Orbiter commands to construct large sets with assumed symmetry group. Table 14.5 lists Orbiter activities for objects of type large-set-with-assumed-symmetry.

Suppose we consider $AG(2, 3)$, the affine plane of order 3. It is a configuration with 9 points, 12 lines, 4 lines on each point and 3 points on each line. To see whether or not it is unique, we use the following command:

Code Snippet 728

```

1    AG_2.3.inc:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define Geo -geometry_builder \
4    ▷ ▷ ▷ -V 9 -B 12 \
5    ▷ ▷ ▷ -TDO 4 -fuse 1 \
6    ▷ ▷ ▷ -fname_GEO AG_2.3 \
7    ▷ ▷ ▷ -test 3,4,5,6,7,8,9 \
8    ▷ ▷ ▷ -output_to_inc_file \
9    ▷ ▷ -end
10
```

The command produces the file `AG_2.3.inc`, which contains the following lines:

```

9 12 36
0 1 2 3 12 16 17 18 24 31 32 33 37 40 43 46 49 53 56 59 62 64 69 71 74 78 80 82 87 89 93 94 99 102 103 107
-1 1
432

```

This shows that the design is unique, and has an automorphism group of order 432. From now on, we will encode blocks as integers, using the ranking of k -subsets. The coding of the AG(2,3) design is stored in a makefile variable:

Code Snippet 729

```

1   AG_2_3_BLOCKS="0,13,22,27,35,41,47,53,55,59,71,76"
2

```

Next, we create a table of all designs AG(2,3), as orbit under the group Sym(9). The number of designs is $|\text{Sym}(9)|/432 = 362880/432 = 840$. The following command creates a table of these designs:

Code Snippet 730

```

1   LS_AG_2_3_design_table_create:
2   ▷ $(ORBITER) -v 50 \
3   ▷ ▷ -define B -vector -dense $(AG_2_3_BLOCKS) -end \
4   ▷ ▷ -define D -design -list_of_blocks_coded 9 3 B -end \
5   ▷ ▷ -define Sym9 -permutation_group -symmetric_group 9 -end \
6   ▷ ▷ -define T -design_table D "AG_2_3" Sym9 -end
7

```

Next, we compute the induced action of the group of AG(2,3) on the designs in the table:

Code Snippet 731

```

1   AG_2_3_on_designs:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define gens -vector -file AG_2_3_gens.csv -end \
4   ▷ ▷ -define G -permutation_group \
5   ▷ ▷ ▷ -bsgs AG_2_3 "AG_2_3" 840 362880 "0,1,2,3,4,5,6,7" \
6   ▷ ▷ ▷ 8 gens \
7   ▷ ▷ -end \
8   ▷ ▷ -define Orb -orbits -group G \
9   ▷ ▷ ▷ -on_points \
10  ▷ ▷ -end \
11  ▷ ▷ -with Orb -do -orbits_activity \
12  ▷ ▷ ▷ -stabilizer 0 \
13  ▷ ▷ -end
14

```

We need the designs that are disjoint from the original design. For this, we define a makefile variable:

Code Snippet 732

```

1   LARGE_SET_AG_2_3_NEIGHBOR_SET="129,130,133,134,136,\
2   139,141,142,153,154,156,160,165,166,178,179,183,184,\

```

```

3    185,190,192,194,197,203,204,206,218,221,222,225,227,\
4    231,248,251,252,255,256,259,261,262,272,277,279,283,\
5    285,287,299,301,303,305,306,309,313,315,319,320,323,\
6    325,341,342,343,344,345,349,368,371,375,378,381,383,\
7    392,393,397,402,403,405,416,419,421,422,425,426,429,\
8    430,440,443,447,449,453,454,464,467,468,473,474,479,\
9    490,493,494,497,500,503,513,517,518,520,523,527,536,\
10   539,541,542,544,547,548,551,563,566,567,571,572,573,\
11   585,589,590,593,595,596,600,601,603,611,614,615,625,\
12   629,631,635,637,638,657,659,661,667,668,671,681,683,\
13   686,689,691,693,705,706,709,710,712,715,717,718,720,\
14   723,724,729,733,735,747,748,750,752,754,757,777,780,\
15   781,784,790,791,802,804,807,808,811,814,824,827,828,\
16   831,832,835,837,838"
17

```

And now we compute the stabilizer of the first design, which of course is still the affine group $\text{AGL}(2, 3)$ of order 432.

Code Snippet 733

```

1    AG_2_3_stab_orb_0:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define gens -vector -file AG_2_3_stab_pt_0_gens.csv -end \
4    ▷ ▷ -define Set192 -set -here $(LARGE_SET_AG_2_3_NEIGHBOR_SET) -end \
5    ▷ ▷ -define G -permutation_group \
6    ▷ ▷ -bsgs AG_2_3_stab_orb_0 "AG_2_3_stab_pt_0" \
7    ▷ ▷ ▷ 840 432 "0,1,2,3,4,5,6,7,8" 4 gens \
8    ▷ ▷ -end \
9    ▷ ▷ -define Gr -modified_group -from G \
10   ▷ ▷ ▷ -restricted_action Set192 Set192 \
11   ▷ ▷ -end \
12   ▷ ▷ -with Gr -do \
13   ▷ ▷ -group_theoretic_activity \
14   ▷ ▷ ▷ -export_orbiter \
15   ▷ ▷ -end
16

```

Now, we compute the permutation representation of $\text{AGL}(2, 3)$ on the 192 designs that are disjoint from the original design:

Code Snippet 734

```

1    AG_2_3_stab_orb_0_Perm840_res192:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define Do -draw_options \
4    ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5    ▷ ▷ -end \
6    ▷ ▷ -define gens -vector -file Perm_840_res_Set192_gens.csv -end \
7    ▷ ▷ -define G -permutation_group \
8    ▷ ▷ -bsgs Perm840_res192 "Perm840 {\rm res192}" \
9    ▷ ▷ ▷ 192 432 "0,1,2,3,4,5,6,7,8" 4 gens \

```

```

10  ▷ ▷ -end \
11  ▷ ▷ -with G -do \
12  ▷ ▷ -group_theoretic_activity \
13  ▷ ▷ ▷ -report Do \
14  ▷ ▷ -end
15  ▷ $(PDFLATEX) Perm840_res192_report.tex
16  ▷ $(OPEN) Perm840_res192_report.pdf
17

```

Next, we create the graph whose vertices are the 192 designs. Two designs are adjacent on the graph if the blocks are distinct as sets. In order to create the large sets of $AGL(2, 3)$, we also compute the cliques of size 12 in this graph:

Code Snippet 735

```

1  LS_AG_2_3_disjoint_sets_graph_and_cliques:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define Gamma -graph \
4  ▷ ▷ ▷ -disjoint_sets_graph \
5  ▷ ▷ ▷ AG_2_3_design_table.csv \
6  ▷ ▷ -end \
7  ▷ ▷ -with Gamma -do \
8  ▷ ▷ -graph_theoretic_activity \
9  ▷ ▷ ▷ -save \
10 ▷ ▷ -end \
11 ▷ ▷ -with Gamma -do \
12 ▷ ▷ -graph_theoretic_activity \
13 ▷ ▷ ▷ -find_cliques -target_size 7 \
14 ▷ ▷ ▷ ▷ -output_file AG_2_3_design_table_cliques.csv \
15 ▷ ▷ -end \
16 ▷ ▷ -print_symbols
17

```

We find 15360 cliques. The next command creates the associated large sets and writes them to file:

Code Snippet 736

```

1  LS_AG_2_3_export_large_sets:
2  ▷ $(ORBITER) -v 20 \
3  ▷ ▷ -define B -vector -dense $(AG_2_3_BLOCKS) -end \
4  ▷ ▷ -define D -design -list_of_blocks_coded 9 3 B -end \
5  ▷ ▷ -define Sym9 -permutation_group -symmetric_group 9 -end \
6  ▷ ▷ -define T -design_table D "AG_2_3" Sym9 -end \
7  ▷ ▷ -with D -do \
8  ▷ ▷ -design_activity \
9  ▷ ▷ ▷ -extract_solutions_by_index.csv "AG_2_3" Sym9 \
10 ▷ ▷ ▷ ▷ AG_2_3_design_table_cliques.csv \
11 ▷ ▷ ▷ ▷ "Solution" \
12 ▷ ▷ ▷ ▷ AG_2_3_large_sets.csv \
13 ▷ ▷ ▷ ▷ "" \
14 ▷ ▷ -end
15

```

It remains to classify the resulting 15360 large sets up to isomorphism. This final step will be discussed in Section 17.4.

14.4 Design Theory – Delandtsheer-Doyen

Delandtsheer and Doyen in [28] study line-transitive and point-imprimitive designs and show that they are rare in a certain sense. Orbiter can be used to construct such designs assuming that there is a grid structure on the set of points and assuming that the design is invariant under a chosen group G . The group G is assumed to be a subgroup of the group $\text{AGL}(d_1, q_1) \times \text{AGL}(d_2, q_2)$ acting on a grid of size $q_1^{d_1} \times q_2^{d_2}$ in product action.

Table 14.6 shows Orbiter commands to search for linear spaces of Delandtsheer/Doyen type. Finite projective planes often arise in this context. However, not all examples are projective planes. Orbiter can help to classify small examples.

Let us consider an example. Suppose we want to classify all designs on 21 points with blocks of size $k = 5$ invariant under a cyclic group of order 21 preserving a grid of type 3×7 . To this end, we consider the group $\text{AGL}(1, 3) \times \text{AGL}(1, 7)$. The subgroup is generated by the map

$$(\tau_1, \tau_2), \mathbb{Z}_3 \times \mathbb{Z}_7 \rightarrow \mathbb{Z}_3 \times \mathbb{Z}_7,$$

where

$$\tau_1 : \mathbb{Z}_3 \rightarrow \mathbb{Z}_3, x \mapsto x + 1 \pmod{3}, \quad \tau_2 : \mathbb{Z}_7 \rightarrow \mathbb{Z}_7, y \mapsto y + 1 \pmod{7}.$$

With blocks of size 5, we cover 10 pairs each. The group of order 21 allows to cover each of the $210 = \binom{21}{2}$ pairs exactly once using a single orbit of a block. Under the group G , there are 55 orbits on pairs. The question remains to construct all blocks and to classify the resulting designs.

At first, three makefile variables are defined:

Code Snippet 737

```
1 PP4=-d1 1 -q1 3 -d2 1 -q2 7 -K 5 -problem_label PP4
2
```

Code Snippet 738

```
1 PP4_GROUP1=-subgroup "1,1,1,1, " "21" -group_label "cyclic21"
2
```

Code Snippet 739

```
1 PP4_MASK1=\
2 ▷ -nb_orbits_on_blocks 1 \
3 ▷ -mask_label "no_mask"
4
```

The next command performs a classification of the sets of size 5 under the given group and using the given mask conditions:

Code Snippet 740

```
1 DD_PP4:
2 ▷ $(ORBITER) -v 6 \
3 ▷ ▷ -define Do -draw_options -end \
4 ▷ ▷ -define pair_search_control -poset_classification_control \
5 ▷ ▷ ▷ -problem_label PP4_pairs -W \
6 ▷ ▷ ▷ -draw_options Do \
```


Creating Linear Spaces of Delandtsheer-Doyen Type		
Command	Arguments	Purpose
-d1	d1	Affine group parameter.
-d2	d2	Affine group parameter.
-q1	q1	Affine group parameter.
-q2	q2	Affine group parameter.
-group_label	label	
-mask_label	label	
-problem_label	label	
-DDx	x	Deladtsheer-Doyen parameter x .
-DDy	y	Secondary Deladtsheer-Doyen parameter y .
-K	k	Specify the size of the base-line to generate the linear space under the affine group.
-pair_search_control	label	Options for the classification of pair orbits, see Tab. 8.4.
-search_control	label	Options for the classification of the base lines by means of subsets, see Tab. 8.4. The depth of the search for starters (partial base-lines) is set here, using the -depth command.
-R	nb_row_types row_type	
-C	nb_col_types col_type	
-nb_orbits_on_blocks	n	
-masktest	options	Specify conditions on the mask, which is the pattern induced by the base-line when considering the grid structure.
-search_partial_base_lines		Search for partial base-lines. The size of the partial base-line is specified in the search_control data structure, using the -depth argument. Once the classification of partial base-lines is finished (of size s , say), the covering matrix of pair orbits is written. For each orbit on partial base-lines of size s , the $\binom{s}{2}$ disjoint pair-orbits orbits covered by that representative are written.
-singletons	sz	Search for singletons. This requires that a classification of partial base-lines of size sz has been completed before. This command initiates the completion of the partial base-lines.
-subgroup	gens order	Specify generators for the subgroup G which acts line-transitive and point-imprimitive on the linear space.
-search_wrt_subgroup		

Table 14.6: Creating Linear Spaces of Delandtsheer-Doyen Type

```

7      ▷ ▷ -end \
8      ▷ ▷ -define search_control -poset_classification_control \
9      ▷ ▷ ▷ -problem_label PP4_search -W \
10     ▷ ▷ ▷ -draw_options Do \
11     ▷ ▷ ▷ -depth 5 \
12     ▷ ▷ -end \
13     ▷ ▷ -Delandtsheer_Doyen \
14     ▷ ▷ ▷ -pair_search_control pair_search_control \
15     ▷ ▷ ▷ -search_control search_control \
16     ▷ ▷ ▷ $(PP4) $(PP4_GROUP1) $(PP4_MASK1) \
17     ▷ ▷ ▷ -create_starter \
18     ▷ ▷ ▷ -end \
19

```

We find exactly one solution. The following command spans the design from the base line by computing the orbit under G :

Code Snippet 741

```

1      DD_PP4_span_design:
2      ▷ $(ORBITER) -v 6 \
3      ▷ ▷ -define G1 -linear_group -AGL 1 3 \
4      ▷ ▷ ▷ -end \
5      ▷ ▷ -define G2 -linear_group -AGL 1 7 \
6      ▷ ▷ ▷ -end \
7      ▷ ▷ -define G -modified_group -direct_product "G1,G2" \
8      ▷ ▷ ▷ "21" "1,1,1,1" \
9      ▷ ▷ -end \
10     ▷ ▷ -define D -design -list_of_base_blocks \
11     ▷ ▷ ▷ G \
12     ▷ ▷ ▷ PP4_no_mask_cyclic21_cases.csv
13     ▷ ▷ ▷ Starter \
14     ▷ ▷ -end
15

```

The next command creates a design object from the base line under the group G :

Code Snippet 742

```

1      DD_PP4_design_orbit:
2      ▷ $(ORBITER) -v 4 \
3      ▷ ▷ -define Do -draw_options \
4      ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5      ▷ ▷ -end \
6      ▷ ▷ -define G1 -linear_group -AGL 1 3 \
7      ▷ ▷ ▷ -end \
8      ▷ ▷ -define G2 -linear_group -AGL 1 7 \
9      ▷ ▷ ▷ -end \
10     ▷ ▷ -define G -modified_group -direct_product "G1,G2" \
11     ▷ ▷ ▷ "21" "1,1,1,1" \
12     ▷ ▷ -end \
13     ▷ ▷ -with G -do \

```

```
14  ▷ ▷ -group_theoretic_activity \  
15  ▷ ▷ ▷ -report Do \  
16  ▷ ▷ -end \  
17  ▷ ▷ -define D -design -list_of_base_blocks \  
18  ▷ ▷ ▷ G \  
19  ▷ ▷ ▷ PP4_no_mask_cyclic21_cases.csv \  
20  ▷ ▷ ▷ Starter \  
21  ▷ ▷ -end  
22
```

As $\text{PG}(2, 4)$ admits an action by a cyclic group of order 21 in product action, and we find only one design, the design we found must be $\text{PG}(2, 4)$. It is the only line-transitive point-imprimitive design with these parameters.

14.5 Design Theory – Flag transitive designs

It is possible to create flag transitive designs from a group G and two conjugacy classes of subgroups, represented by groups P and Q , both subgroups of G . The groups in the class of P serve as the point stabilizers. The groups in the class of Q serve as the block stabilizers. The following sequence of commands recreates the design of [65].

Using a makefile variable, we define 8 elements in $\text{AGL}(1, 81)$ as follows:

Code Snippet 743

```
1   PRAEGER_DESIGN_45_12_3_GENS="1,1,0, 1,3,0, 1,51,0, 1,76,0, 2,0,0, 3,0,1, 16,0,0,
    3,0,1"
2
```

Let G be the group of order 3240 generated by these elements. We create a flag-transitive design from the group generated by these elements. The following command is used:

Code Snippet 744

```
1   Praeger_design_45_12_3:
2   ▷ $(ORBITER) -v 4 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 81 -compute_related_fields \
7   ▷ ▷ ▷ -override_polynomial 89 \
8   ▷ ▷ -end \
9   ▷ ▷ -define AGGL -linear_group -AGGL 1 F \
10  ▷ ▷ ▷ -end \
11  ▷ ▷ -define G -linear_group -AGGL 1 F \
12  ▷ ▷ ▷ -subgroup_by_generators G 3240 \
13  ▷ ▷ ▷ 8 $(PRAEGER_DESIGN_45_12_3_GENS) \
14  ▷ ▷ ▷ -end \
15  ▷ ▷ -with G -do \
16  ▷ ▷ -group_theoretic_activity \
17  ▷ ▷ ▷ -report Do \
18  ▷ ▷ -end \
19  ▷ ▷ -define v -vector \
20  ▷ ▷ ▷ -dense $(PRAEGER_DESIGN_45_12_3_GENS) \
21  ▷ ▷ -end \
22  ▷ ▷ -define gens -vector_ge \
23  ▷ ▷ ▷ -action AGGL \
24  ▷ ▷ ▷ -vector_data v \
25  ▷ ▷ -end \
26  ▷ ▷ -with gens -do \
27  ▷ ▷ -vector_ge_activity \
28  ▷ ▷ ▷ -report \
29  ▷ ▷ -end \
30  ▷ ▷ -with gens -do \
31  ▷ ▷ -vector_ge_activity \
32  ▷ ▷ ▷ -field_reduction 4 \
33  ▷ ▷ -end \
34  ▷ ▷ -with G -do \
```

```

35  ▷ ▷ -group_theoretic_activity \
36  ▷ ▷ ▷ -subgroup_lattice_magma \
37  ▷ ▷ -end \
38  ▷ ▷ -define L -modified_group -from G \
39  ▷ ▷ ▷ -subgroup_by_lattice 43 \
40  ▷ ▷ -end \
41  ▷ ▷ -define R -modified_group -from G \
42  ▷ ▷ ▷ -subgroup_by_lattice 44 \
43  ▷ ▷ -end \
44  ▷ ▷ -with L -do \
45  ▷ ▷ -group_theoretic_activity \
46  ▷ ▷ ▷ -report Do \
47  ▷ ▷ -end \
48  ▷ ▷ -with R -do \
49  ▷ ▷ -group_theoretic_activity \
50  ▷ ▷ ▷ -report Do \
51  ▷ ▷ -end \
52  ▷ ▷ -with G -do \
53  ▷ ▷ -group_theoretic_activity \
54  ▷ ▷ ▷ -create_flag_transitive_geometry L R \
55  ▷ ▷ -end
56

```

The command creates the intersection matrix between the groups in $\tilde{L}$ and in $\tilde{R}$ where $\tilde{P}$ denotes the conjugacy class of subgroups of P inside G .

The next command creates the relation defined by the pairs of groups whose intersection of size 6.

Code Snippet 745

```

1  Praeger_design_change_values:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -orbiter_path $(ORBITER_PATH) \
4  ▷ ▷ -change_values \
5  ▷ ▷ ▷ AGGL_1.81_Subgroup_G.3240_AGGL_1.81_Subgroup_G.3240_subgroup_by_lattice_43_
AGGL_1.81_Subgroup_G.3240_subgroup_by_lattice_44_intersection_size.csv \
6  ▷ ▷ ▷ Praeger_6.csv \
7  ▷ ▷ ▷ 3,6,8 \
8  ▷ ▷ ▷ 0,1,0
9

```

The following command creates the design from the relation created before and performs some basic activities for it:

Code Snippet 746

```

1  Praeger_45.12.3.design:
2  ▷ $(ORBITER) -v 8 \
3  ▷ ▷ -define Praeger_45 -vector \
4  ▷ ▷ ▷ -file Praeger_6.csv -format 45 \
5  ▷ ▷ -end \
6  ▷ ▷ -define D -design \
7  ▷ ▷ ▷ -from_incidence_matrix Praeger_45 \

```

```

8   ▷ ▷ -end \
9   ▷ ▷ -with D -do \
10  ▷ ▷ ▷ -design_activity \
11  ▷ ▷ ▷ ▷ -export_flags \
12  ▷ ▷ ▷ -end \
13  ▷ ▷ ▷ -with D -do \
14  ▷ ▷ ▷ ▷ -design_activity \
15  ▷ ▷ ▷ ▷ -export_blocks \
16  ▷ ▷ ▷ -end \
17  ▷ ▷ ▷ -with D -do \
18  ▷ ▷ ▷ ▷ -design_activity \
19  ▷ ▷ ▷ ▷ -save \
20  ▷ ▷ ▷ ▷ -intersection_matrix \
21  ▷ ▷ -end
22

```

The following command creates the full stabilizer of the design:

Code Snippet 747

```

1   Praeger_45.12.3_design.c:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define Doinc -draw_incidence_structure_options \
4   ▷ ▷ ▷ -width 60 -width_10 6 -end \
5   ▷ ▷ -define C -combinatorial_object \
6   ▷ ▷ ▷ -label Praeger Praeger \
7   ▷ ▷ ▷ -file_of_incidence_geometries \
8   ▷ ▷ ▷ ▷ Praeger_45_flags.txt 45 45 540 \
9   ▷ ▷ -end \
10  ▷ ▷ -with C -do \
11  ▷ ▷ -combinatorial_object_activity \
12  ▷ ▷ ▷ -canonical_form \
13  ▷ ▷ ▷ ▷ -save_ago \
14  ▷ ▷ ▷ ▷ -save_transversal \
15  ▷ ▷ ▷ ▷ -nauty_control -nauty_log "praeger_design.nauty_log.txt" -end \
16  ▷ ▷ ▷ -end \
17  ▷ ▷ -end \
18  ▷ ▷ -with C -do \
19  ▷ ▷ -combinatorial_object_activity \
20  ▷ ▷ ▷ -post_processing \
21  ▷ ▷ -end \
22  ▷ ▷ -with C -do \
23  ▷ ▷ -combinatorial_object_activity \
24  ▷ ▷ ▷ -report \
25  ▷ ▷ ▷ ▷ -export_flag_orbits \
26  ▷ ▷ ▷ ▷ -show_incidence_matrices \
27  ▷ ▷ ▷ ▷ -export_group_GAP \
28  ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
29  ▷ ▷ ▷ -end \
30  ▷ ▷ -end
31  ▷ $(PDFLATEX) Praeger_classification.tex
32  ▷ $(OPEN) Praeger_classification.pdf

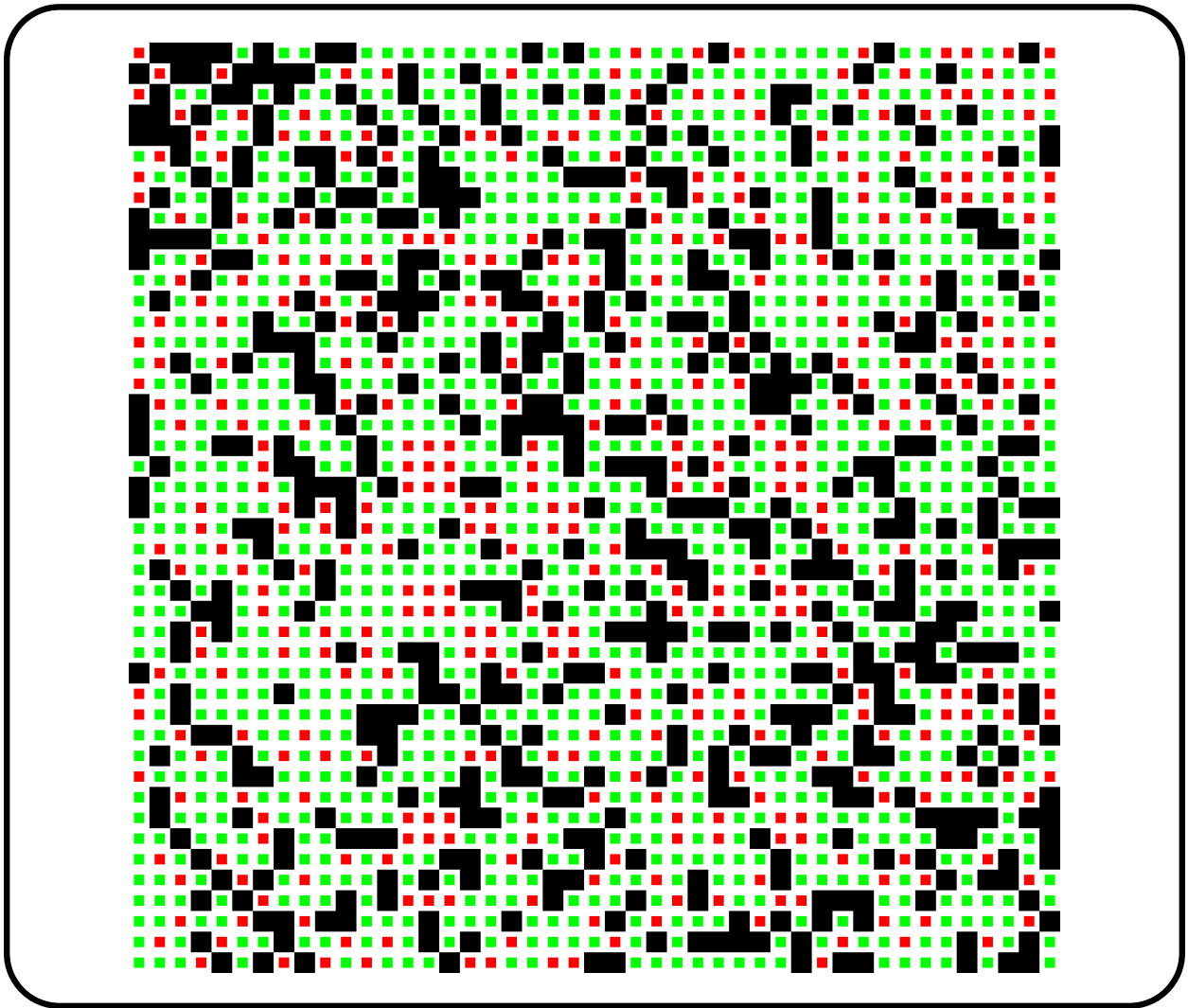
```

```

33   ▷ $(ORBITER) -v 2 -draw_matrix \
34   ▷ ▷ -input_csv_file Praeger_object_0_INP_flag_orbits.csv \
35   ▷ ▷ -secondary_input_csv_file Praeger_object_0_INP.csv \
36   ▷ ▷ -box_width 32 -bit_depth 24 \
37   ▷ ▷ -end
38   ▷ magick \
39   ▷ ▷ Praeger_object_0_INP_flag_orbits_draw.bmp \
40   ▷ ▷ Praeger_object_0_INP_flag_orbits_draw.png
41   ▷ $(OPEN) Praeger_object_0_INP_flag_orbits_draw.png
42   ▷ $(ORBITER) -v 2 \
43   ▷ ▷ -define Do -draw_options -embedded -radius 150 \
44   ▷ ▷ ▷ -nodes_empty \
45   ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
46   ▷ ▷ -tree_draw \
47   ▷ ▷ ▷ -file praeger_design_nauty_log.txt \
48   ▷ ▷ ▷ -draw_options Do \
49   ▷ ▷ -end
50   ▷ $(PDFLATEX) praeger_design_nauty_log_draw.tex
51   ▷ $(OPEN) praeger_design_nauty_log_draw.pdf
52

```

The full stabilizer of the design has order 19440. The command also produces a drawing of the action of the group on the flags and anti-flags, shown below:



Chapter 15

Finite Geometry

15.1 Spreads

A t -spread of $\text{PG}(n, q)$ is a set of disjoint $\text{PG}(t, q)$ that cover all of $\text{PG}(n, q)$ pointwise. t -spreads in $\text{PG}(n, q)$ exist if and only if $t + 1$ divides $n + 1$.

Table 15.1 lists the Orbiter commands to create spreads.

Table 15.2 lists the Orbiter spread activities.

Table 15.3 lists the Orbiter commands to create a table of spreads.

Table 15.4 lists the Orbiter activities for spread tables.

Table 15.5 list the Orbiter commands to create an object that can classify spreads.

Table 15.6 lists the Orbiter commands that apply to an object that can classify spreads.

The following two commands create the two spreads of order 9, relying on the Orbiter knowledge base.

Code Snippet 748

```
1   create_spread_9a:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 3 -end \
4   ▷ ▷ -define G -linear_group -PGL 4 F -end \
5   ▷ ▷ -define S -spread -kernel_field F \
6   ▷ ▷ ▷ -group G -k 2 -catalogue 0 \
7   ▷ ▷ ▷ -end \
8   ▷ ▷ -with S -do -spread_activity \
9   ▷ ▷ ▷ -report \
10  ▷ ▷ -end
11
```

Code Snippet 749

```
1   create_spread_9b:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 3 -end \
4   ▷ ▷ -define G -linear_group -PGL 4 F -end \
5   ▷ ▷ -define S -spread -kernel_field F \
6   ▷ ▷ ▷ -group G -k 2 -catalogue 1 \
7   ▷ ▷ ▷ -end \
8   ▷ ▷ -with S -do -spread_activity \
9   ▷ ▷ ▷ -report \
```

Creating a Spread		
Command	Arguments	Purpose
-kernel_field	F	Define the kernel of the spread. F must be an object of type finite field.
-group	G	Define the group acting on the spread. Should be $\text{P}\Gamma\text{L}(2k, F)$.
-group_on_subspaces	G	
-k	k	Set the dimension of the spread.
-catalogue	i	Pull spread number i from the catalogue of spreads associated with the given field and the given dimension.
-family	L	Define a spread from a named family L . So far, no family has been provided.
-spread_set	label-txt label-tex S	Define a spread from the named spreadset S . The spreadset S must be a vector object. It must contain $q^3 k^2$ entries over F .
-transform	elt	Apply the transformation given by the group element.
-transform_inverse	elt	Apply the inverse transformation given by the group element.

Table 15.1: Creating a Spread

Spread Activities		
Command	Arguments	Purpose
-report		Create a latex report.

Table 15.2: Spread Activities

Creating a Spread Table		
Command	Arguments	Purpose
-space	label	Set the projective space object. The space must be an object of type $\text{PG}(3, q)$.
-rk	rk	Set the vector space dimension of the spread elements (i.e., 2 for line-spreads).
-iso_types	list	Selection of isomorphism types of spreads for the table. The numbers refer to the Orbiter catalogue of spreads. The spreads in the orbits of the chosen representatives will be added to the table.
-path	path	Set a path for files associated with the table of spreads.
-control	control	Specify a control structure for poset classification.
-load		Load the table from previously created files.
-restricted_table	label subset	Create a new table containing a subset of the spreads. Subset is the list of indices of spreads that should go in the table.

Table 15.3: Creating a Spread Table

Activities for Spread Tables		
Command	Arguments	Purpose
-find_spread	spread	Find a given spread in the table.
-find_spread_and_dualize	spread	Find a given spread in the table and dualize.
-dualize_packing	packing	Dualize a packing.
-print_spreads	spreads	Print the given spreads.
-export_spreads_to_csv	fname index	Export the spread whose index is given to csv-file.
-find_spreads_containing_one_line	index	Find all spreads which contain the given line.
-isomorphism_type_of_spreads	spreads-by-index	Determine the isomorphism type of each spread in the list.
-dualize_packings	fname_in fname_out col_label	Reads the packings from fname_in and dualizes them. The dual packings are written to fname_out. Here, col_label is the column containing the data.

Table 15.4: Activities for Spread Tables

Classification of Spreads		
Command	Arguments	Purpose
-projective_space	P	Select the projective space object P.
-starter_size	k	Choose starter size k .
-k	k	Choose dimension of the spread elements.
-poset_classification_control	control	Select poset classification control.
-output_prefix	prefix	Set output prefix for file i/o.
-recoordinatize		Turn recoordinatization on (recommended).

Table 15.5: Classification of Spreads

Activities for the Classification of Spreads		
Command	Arguments	Purpose
-compute_starter		Perform classification of partial spreads.
-prepare_lifting_single_case	k	Prepare to lift single case k . Here (and below) case means isomorphism type in the classification of partial spreads.
-prepare_lifting_all_cases		Prepare to lift all cases of partial spreads.
-split	r m	Lift only those cases k with $k \equiv r \pmod{m}$.
-isomorph	prefix1 prefix2 options	Perform isomorphism testing after lifting is complete.

Table 15.6: Activities for the Classification of Spreads

```

10  ▷ ▷ -end
11

```

The first spread is the Desarguesian spread, with automorphism group of order 5760. The second spread is the Hall spread with automorphism group of order 1920.

Spreads can be defined using spread sets. A spread set is a set of q^k matrices of size $k \times k$ over $\mathbb{F}_q$ such that $A_i - A_j$ is nonsingular for all $i \neq j$. Let us look at an example. The spread due to Rao and Rao [60] can be defined using the following makefile variable.

Code Snippet 750

```

1  SPREAD_SET_27_RAO_RAO="\
2  0,0,0,0,0,0,0,0,0, \
3  1,1,0,2,1,1,0,0,2, \
4  1,0,1,1,2,2,0,1,0, \
5  1,2,2,1,2,0,2,2,2, \
6  0,0,2,2,2,0,1,2,0, \
7  1,1,2,0,2,1,2,1,0, \
8  0,1,0,1,0,1,0,2,1, \
9  2,0,2,0,0,2,1,1,0, \
10 2,2,2,0,1,1,0,1,2, \
11 2,0,0,1,0,2,1,2,1, \
12 0,2,2,2,2,2,2,0,2, \

```

```

13  2,1,2,0,2,0,2,0,1, \
14  0,1,2,2,0,1,0,1,1, \
15  1,0,0,0,1,0,0,0,1, \
16  2,1,0,1,2,1,0,2,0, \
17  0,2,0,0,2,2,1,1,2, \
18  0,0,1,0,1,2,2,2,1, \
19  2,0,1,2,2,1,1,0,1, \
20  0,1,1,1,1,0,1,2,2, \
21  2,2,0,2,0,0,0,2,2, \
22  2,1,1,1,1,2,2,1,2, \
23  2,2,1,2,1,0,2,0,0, \
24  1,2,0,2,0,2,1,0,0, \
25  1,2,1,1,0,0,1,1,1, \
26  0,2,1,1,1,1,2,2,0, \
27  1,1,1,0,0,1,1,0,2, \
28  1,0,2,2,1,2,2,1,1 \
29  "
30

```

Each line represents one matrix of the spread set, with matrix entries being listed consecutively. The following command can be used to define the spread:

Code Snippet 751

```

1  create_spread_Rao_Rao_27:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define F -finite_field -q 3 -end \
4  ▷ ▷ -define SS -vector -dense $(SPREAD.SET_27_RA0_RA0) -end \
5  ▷ ▷ -define G -linear_group -PGL 6 F -end \
6  ▷ ▷ -define S -spread -kernel_field F \
7  ▷ ▷ ▷ -group G -k 3 -spread_set "Rao_Rao" "Rao\_Rao" SS \
8  ▷ ▷ ▷ -end
9

```

The following command creates the Desarguesian line-spread in $\text{PG}(3, 2)$:

Code Snippet 752

```

1  desarguesian_spread_in_PG_3_2:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define FQ -finite_field -q 4 -end \
4  ▷ ▷ -define Fq -finite_field -q 2 -end \
5  ▷ ▷ -with FQ -and Fq -do -finite_field_activity \
6  ▷ ▷ ▷ -cheat_sheet_desarguesian_spread 2 -end
7  ▷ $(PDFLATEX) Desarguesian_Spread_3_2.tex
8  ▷ $(OPEN) Desarguesian_Spread_3_2.pdf
9

```

The cheat sheet contains the following spread:

$$\begin{aligned}
\text{Spread element 0 is } (1,0) &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}_0 \\
\text{Spread element 1 is } (0,1) &= \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_{34} \\
\text{Spread element 2 is } (1,1) &= \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}_9 \\
\text{Spread element 3 is } (2,1) &= \begin{bmatrix} 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}_{17} \\
\text{Spread element 4 is } (3,1) &= \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix}_{22} \\
\text{Spread elements by rank: } &(0, 34, 9, 17, 22).
\end{aligned}$$

The following command creates the Desarguesian plane-spread in PG(5, 2):

Code Snippet 753

```

1   desarguesian_spread_in_PG_5.2:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define FQ -finite_field -q 8 -end \
4   ▷ ▷ -define Fq -finite_field -q 2 -end \
5   ▷ ▷ -with FQ -and Fq -do -finite_field_activity \
6   ▷ ▷ ▷ -cheat_sheet_desarguesian_spread 2 -end
7   ▷ $(PDFLATEX) Desarguesian_Spread_5.2.tex
8   ▷ $(OPEN) Desarguesian_Spread_5.2.pdf
9

```

$$\begin{aligned}
\text{Spread element 0 is } (1,0) &= \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}_0 \\
\text{Spread element 1 is } (0,1) &= \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}_{1394} \\
\text{Spread element 2 is } (1,1) &= \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}_{189} \\
\text{Spread element 3 is } (2,1) &= \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}_{671} \\
\text{Spread element 4 is } (3,1) &= \begin{bmatrix} 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}_{562}
\end{aligned}$$

$$\begin{aligned}
\text{Spread element 5 is } (4,1) &= \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}^{1040} \\
\text{Spread element 6 is } (5,1) &= \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}^{792} \\
\text{Spread element 7 is } (6,1) &= \begin{bmatrix} 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}^{1161} \\
\text{Spread element 8 is } (7,1) &= \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}^{373} \\
\text{Spread elements by rank: } & (0, 1394, 189, 671, 562, 1040, 792, 1161, 373)
\end{aligned}$$

A spread table can be used to enumerate labeled spreads. For instance, we can create all labeled spreads associated with one particular isomorphism type. This means we can create spreads by orbits. We can choose the isomorphism type of spreads for which we want the orbits computed. In the following example, we create the labeled spreads associated with the unique spread in $\text{PG}(3, 2)$. There are exactly 56 spreads in the orbit:

Code Snippet 754

```

1   PG_3.2_spread_table:
2   ▷ - mkdir SPREAD_TABLES_2
3   ▷ $(ORBITER) -v 5 \
4   ▷ ▷ -define Control -poset_classification_control \
5   ▷ ▷ ▷ -problem_label spreads_PG_3.2 \
6   ▷ ▷ ▷ -W \
7   ▷ ▷ -end \
8   ▷ ▷ -define F -finite_field -q 2 -end \
9   ▷ ▷ -define P3 -projective_space -n 3 -field F -v 0 -end \
10  ▷ ▷ -define T -spread_table -space P3 -rk 2 \
11  ▷ ▷ ▷ -iso_types "0" \
12  ▷ ▷ ▷ -path "SPREAD_TABLES_2/" \
13  ▷ ▷ ▷ -control Control \
14  ▷ ▷ ▷ -end \
15

```

The spreads are stored in a file

SPREAD_TABLES_2/spread_4_table.csv

Here, the integer 4 in the file name represents the order of the translation plane associated with the spreads.

Two t -spreads are isomorphic if there is a collineation which maps one to the other. The classification problem for t -spreads is the problem of determining a complete set of pairwise non-isomorphic t -spreads. Orbiter can be used to classify spreads for small parameters. To enhance the algorithm, the method of classification by substructure is used. Classification by substructure is a method to classify objects. In this method, we do not classify the whole poset. Instead, we classify the substructures, and then move directly to the desired objects, thereby skipping over intermediate objects. The jump over intermediate objects

is facilitated by combinatorial searching, which may process nodes faster than classification with isomorphism rejection. The combinatorial search is facilitated by an auxiliary algorithm. This auxiliary algorithm can be clique finding, exact cover, or any other computer science primitives. Table 15.7 shows the commands available for classification by substructure.

Let us look at some examples.

At first, we look at an example which is sufficiently small and can be solved using the standard method. Here, the standard method is poset classification algorithm for partial spreads. Suppose we want to classify the line spreads in $\text{PG}(3, 4)$ under the action of $\text{P}\Gamma\text{L}(4, 4)$. Under the André, Bruck-Bose construction [5, 22], these spreads correspond to translation planes of order 16 with kernel $\mathbb{F}_4$. In order to classify the spreads of $\text{PG}(3, 4)$, we utilize poset classification, as shown in the following command

Code Snippet 755

```

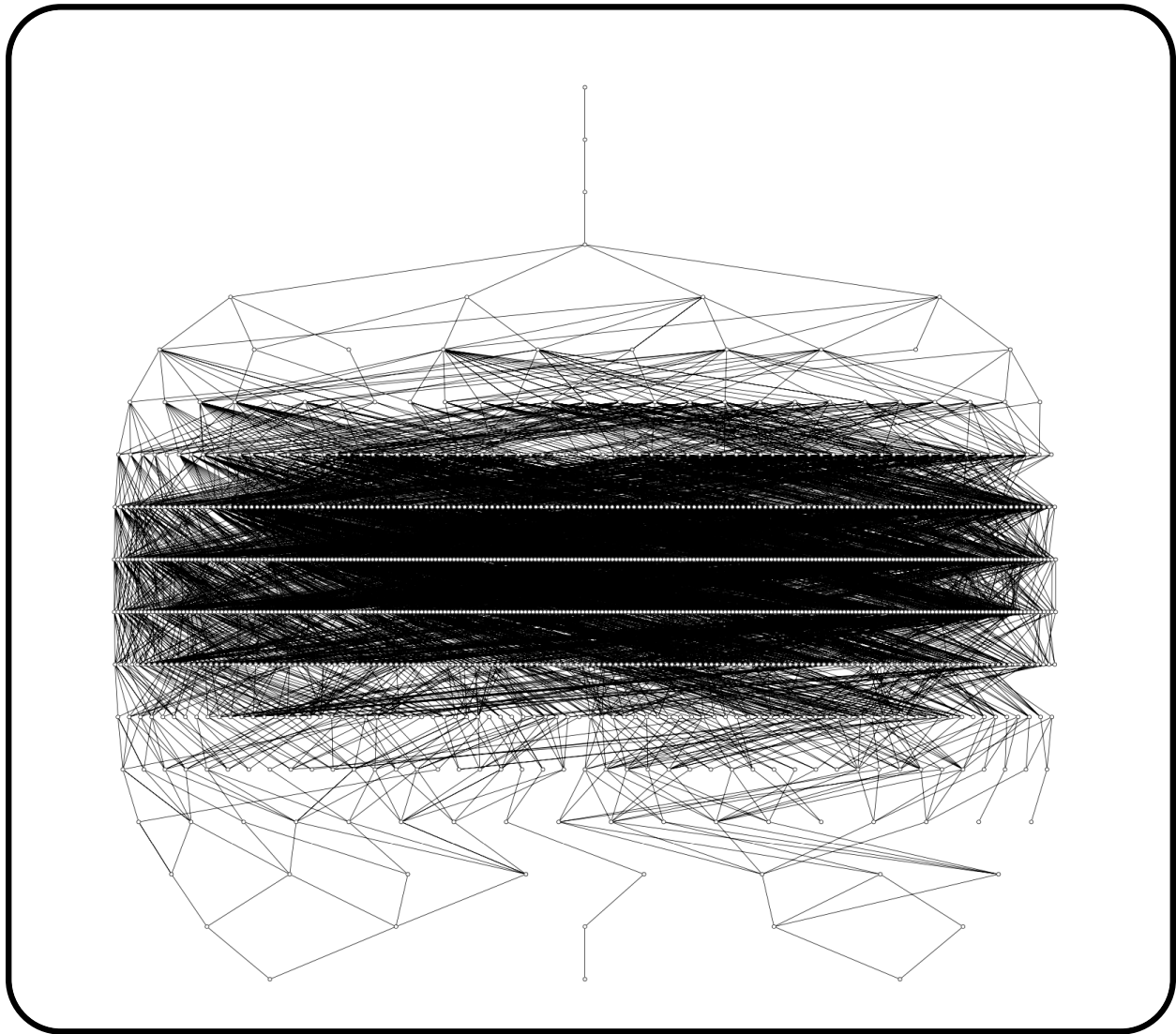
1   classify_spreads_16_4:
2   ▷ $(ORBITER) -v 4 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 20 \
5   ▷ ▷ ▷ -nodes_empty \
6   ▷ ▷ ▷ -line_width 0.2 \
7   ▷ ▷ ▷ -embedded \
8   ▷ ▷ -end \
9   ▷ ▷ -define Control -poset_classification_control \
10  ▷ ▷ ▷ -problem_label spreads_16_4 \
11  ▷ ▷ ▷ -draw_options Do \
12  ▷ ▷ -end \
13  ▷ ▷ -define F -finite_field -q 4 -end \
14  ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
15  ▷ ▷ -define C -spread_classifier \
16  ▷ ▷ ▷ -projective_space P \
17  ▷ ▷ ▷ -poset_classification_control Control \
18  ▷ ▷ ▷ -k 2 \
19  ▷ ▷ ▷ -starter_size 17 \
20  ▷ ▷ ▷ -output_prefix "." \
21  ▷ ▷ -end \
22  ▷ ▷ -with C -do -spread_classify_activity \
23  ▷ ▷ ▷ -compute_starter \
24  ▷ ▷ -end
25  ▷ #$(PDFLATEX) spreads_16_4_poset_lvl17.tex
26  ▷ #$(OPEN) spreads_16_4_poset_lvl17.pdf
27

```

The algorithm computes the poset of orbits for the group $G = \text{P}\Gamma\text{L}(4, 4)$ acting on the poset of partial spreads in $\text{PG}(3, 4)$. The poset of orbits is shown below:

Isomorph Testing		
Command	Arguments	Purpose
-prefix_iso	prefix	Set a prefix for files related to isomorph testing.
-prefix_with_directory	prefix	Set a prefix for files related to isomorph testing.
-prefix_classify	prefix	Set a prefix for files related to classification.
-solution_prefix	prefix	Set a prefix for solution files.
-base_fname	base	Set the main file name.
-use_database_for_starter		Use database to store subobjects.
-implicit_fusion		Use implicit fusion during the isomorphism testing.
-build_db		Initialize the database.
-read_solutions		Read solutions.
-list_of_cases	fname	Read a list of cases from the given file.
-read_solutions_after_split	m	Read the solution files after split modulo m.
-read_statistics_after_split	m	Read the statistics files after split modulo m.
-recognize	label	
-compute_orbits		Compute the orbits of the stabilizer. These are the flag orbits.
-isomorph_testing		Apply isomorphism testing to the flag orbits to produce the classification.
-classification_graph		Compute the classification graph (after classification).
-event_file	fname	Reuse a previously written event file during the classification.
-print_mod	N	Set print internal to N.
-isomorph_report		Write a report on the classification.
-export_source_code		Export the classification as C++ source code.
-subset_orbits		
-subset_orbits_file	fname	
-eliminate_graphs_if_possible		
-down_orbits		

Table 15.7: Isomorph Testing



Up to isomorphism, there are exactly three line-spreads in $\text{PG}(3, 4)$ (corresponding to the three nodes at the bottom of the poset of orbits in the figure). These three spreads are the Desarguesian spread, the Hall spread, and the semifield spread, respectively. Here is the relevant output taken from the latex report:

There are 3 orbits at level 17.

Orbit 0 / 3 at Level 17

Node number: 1126

$\{0, 25, 50, 75, 90, 107, 122, 140, 144, 157, 179, 204, 213, 238, 268, 334, 345\}_{1200}$

Strong generators for a group of order 1200:

$$\begin{bmatrix} \omega^2 & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ 1 & \omega & 0 & 1 \\ \omega^2 & \omega^2 & \omega & 1 \end{bmatrix}_0, \begin{bmatrix} \omega^2 & 0 & 0 & 0 \\ \omega & \omega^2 & 0 & 0 \\ \omega & \omega & 1 & \omega^2 \\ 0 & 1 & 1 & 0 \end{bmatrix}_1, \begin{bmatrix} \omega & 1 & \omega & \omega \\ \omega & \omega^2 & \omega^2 & 0 \\ \omega & 0 & 0 & 1 \\ 0 & \omega & \omega & 1 \end{bmatrix}_0$$

1,0,0,0,0,1,0,0,2,3,0,2,1,1,3,2,0,

1,0,0,0,3,1,0,0,3,3,2,1,0,2,2,0,1,

1,3,1,1,1,2,2,0,1,0,0,3,0,1,1,3,0,

There are 0 extensions

Number of generators 3

Orbit 1 / 3 at Level 17

Node number: 1127

$$\{0, 25, 50, 75, 90, 107, 140, 157, 179, 204, 213, 238, 265, 282, 299, 316, 356\}_{81600}$$

Strong generators for a group of order 81600:

$$\begin{bmatrix} \omega & 0 & 0 & 0 \\ 0 & \omega & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \omega & \omega^2 \\ 0 & 0 & 1 & 1 \end{bmatrix}_0, \begin{bmatrix} \omega^2 & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ 1 & \omega^2 & \omega & \omega^2 \\ 1 & \omega & 1 & 1 \end{bmatrix}_0,$$

$$\begin{bmatrix} \omega^2 & 0 & 0 & 0 \\ \omega & \omega^2 & 0 & 0 \\ 0 & 0 & 0 & \omega^2 \\ 0 & 0 & \omega & 1 \end{bmatrix}_1, \begin{bmatrix} \omega^2 & 0 & 0 & \omega \\ \omega & \omega^2 & 1 & \omega^2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & \omega^2 & 1 \end{bmatrix}_1, \begin{bmatrix} 0 & \omega^2 & \omega^2 & 0 \\ 1 & 0 & \omega^2 & \omega^2 \\ 0 & 0 & 1 & \omega^2 \\ 0 & 0 & 0 & 1 \end{bmatrix}_0,$$

$$\begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & \omega & 1 \\ 0 & 1 & \omega & \omega^2 \\ \omega & 1 & 1 & 1 \end{bmatrix}_0$$

1,0,0,0,0,1,0,0,0,0,3,0,0,0,0,3,0,

1,0,0,0,0,1,0,0,0,0,2,3,0,0,1,1,0,

1,0,0,0,0,1,0,0,2,1,3,1,2,3,2,2,0,

1,0,0,0,3,1,0,0,0,0,0,1,0,0,3,2,1,

1,0,0,3,3,1,2,1,0,0,2,0,0,0,1,2,1,

0,1,1,0,2,0,1,1,0,0,2,1,0,0,0,2,0,

0,0,0,1,0,0,2,1,0,1,2,3,2,1,1,1,0,

There are 0 extensions

Number of generators 7

Orbit 2 / 3 at Level 17

Node number: 1128

$$\{0, 25, 50, 75, 90, 108, 122, 140, 158, 183, 199, 217, 233, 250, 268, 312, 345\}_{576}$$

Strong generators for a group of order 576:

$$\begin{bmatrix} \omega & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ 0 & 0 & \omega^2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} \omega^2 & 0 & 0 & 0 \\ 0 & \omega^2 & 0 & 0 \\ \omega & 0 & \omega & 1 \\ \omega^2 & 0 & 0 & 1 \end{bmatrix}_0, \begin{bmatrix} \omega^2 & 0 & 0 & 0 \\ \omega & \omega^2 & 0 & 0 \\ \omega & 0 & 1 & 1 \\ \omega^2 & 0 & \omega^2 & 1 \end{bmatrix}_0,$$

$$\begin{bmatrix} \omega & \omega & \omega & \omega \\ \omega^2 & 0 & \omega^2 & 0 \\ \omega^2 & 0 & \omega^2 & \omega \\ 0 & \omega^2 & \omega^2 & 1 \end{bmatrix}_0, \begin{bmatrix} 1 & 0 & \omega^2 & 1 \\ 1 & \omega^2 & 1 & 0 \\ 1 & 0 & \omega & \omega \\ 0 & 0 & 0 & 1 \end{bmatrix}_1, \begin{bmatrix} 0 & \omega^2 & \omega^2 & 0 \\ 0 & 0 & 0 & \omega^2 \\ 1 & 0 & 1 & \omega^2 \\ \omega & 1 & \omega & 1 \end{bmatrix}_0$$

1,0,0,0,0,2,0,0,0,0,2,0,0,0,3,1,

1,0,0,0,0,1,0,0,3,0,3,2,1,0,2,0,

1,0,0,0,3,1,0,0,3,0,2,2,1,0,1,2,0,

1,1,1,1,2,0,2,0,2,0,2,1,0,2,2,3,0,

1,0,3,1,1,3,1,0,1,0,2,2,0,0,1,1,

0,1,1,0,0,0,0,1,2,0,2,1,3,2,3,2,0,

There are 0 extensions

Number of generators 6

The three spreads in $\text{PG}(3, 4)$ can be distinguished by their stabilizer orders.

Let us illustrate the technique of classification by distinguished substructure. We consider spreads in $\text{PG}(3, 5)$. These are sets of lines which partition the set of points. The search poset is the set of partial spreads, that is, the set of sets of pairwise disjoint lines. We decide to pick as substructure the partial spreads consisting of five disjoint lines. The number $s = 5$ is somewhat arbitrary, chosen only because it seems to work best. For each partial spreads of size s , the lifting constructs all spreads of $\text{PG}(3, 5)$ containing the chosen partial spread as a subset. In a final step, we will perform an isomorph classification on the set of liftings. This will furnish the desired classification of spreads of $\text{PG}(3, 5)$.

Computationally, the lifting process is the bottleneck in this procedure. To enhance the performance of the lifting, we use specialized algorithms from graph theory, based on cliques. A clique in a graph is a complete subgraph, i.e. a subset of the vertices such that any two in the subset are adjacent. A variant of this is rainbow clique finding. Here, we have a coloring of the vertices, so that edges are only present between vertices of different color. The goal of rainbow clique finding is to find cliques whose vertices represent all the different colors. Many problems in the field of combinatorial designs can be attacked using rainbow cliques.

To illustrate the technique, let us discuss some examples. Regarding the substructure, we choose the parameter $s = 5$. This is based on some experimentation. The command

Code Snippet 756

```
1  classify_spreads_25_starter_lift_case_0:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define Do -draw.options \
4  ▷ ▷ ▷ -radius 20 \
5  ▷ ▷ ▷ -nodes_empty \
6  ▷ ▷ ▷ -line_width 0.2 \
7  ▷ ▷ ▷ -embedded \
8  ▷ ▷ -end \
9  ▷ ▷ -define Control -poset_classification_control \
```

```

10  ▷ ▷ ▷ -W \
11  ▷ ▷ ▷ -problem_label spreads_25 \
12  ▷ ▷ ▷ -draw_options Do \
13  ▷ ▷ ▷ -end \
14  ▷ ▷ ▷ -define F -finite_field -q 5 -end \
15  ▷ ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
16  ▷ ▷ ▷ -define C -spread_classifier \
17  ▷ ▷ ▷ -projective_space P \
18  ▷ ▷ ▷ -k 2 \
19  ▷ ▷ ▷ -starter_size 5 \
20  ▷ ▷ ▷ -recoordinatize \
21  ▷ ▷ ▷ -poset_classification_control Control \
22  ▷ ▷ ▷ -output_prefix "" \
23  ▷ ▷ ▷ -end \
24  ▷ ▷ ▷ -with C -do -spread_classify_activity \
25  ▷ ▷ ▷ -compute_starter \
26  ▷ ▷ ▷ -end \
27  ▷ ▷ ▷ -with C -do -spread_classify_activity \
28  ▷ ▷ ▷ -prepare_lifting_single_case 0 \
29  ▷ ▷ ▷ -end
30

```

classifies the partial spreads of size $s = 5$ and prepares for the lifting of the first case only. In order to prepare for the lifting, a graph is constructed which describes the lines that can be added to the first partial spread. The vertices of the graph are the lines disjoint from the initial set of 5 lines in the partial spread. Two vertices are joined by an edge if the associated lines are disjoint. The vertices of the graph are colored according to the very first basis vector in the generator matrix of the subspace in reduced row echelon form. In order to find the rainbow cliques in the graph, the command

Code Snippet 757

```

1  spreads_25_starter_0_cliques:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ ▷ -define G -graph -load spreads_25_graph_0.bin -end \
4  ▷ ▷ ▷ -with G -do \
5  ▷ ▷ ▷ -graph_theoretic_activity \
6  ▷ ▷ ▷ ▷ -find_cliques -rainbow -target_size 21 \
7  ▷ ▷ ▷ ▷ -output_file spreads_25_0_cliques.csv \
8  ▷ ▷ ▷ -end \
9

```

can be used.

The command

Code Snippet 758

```

1  classify_spreads_25_starter_lift_all_cases:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define Do -draw_options \
4  ▷ ▷ ▷ -radius 20 \
5  ▷ ▷ ▷ -nodes_empty \

```

```

6    ▷ ▷ ▷ -line_width 0.2 \
7    ▷ ▷ ▷ -embedded \
8    ▷ ▷ -end \
9    ▷ ▷ -define Control -poset_classification_control \
10   ▷ ▷ ▷ -W \
11   ▷ ▷ ▷ -problem_label spreads_25 \
12   ▷ ▷ ▷ -draw_options Do \
13   ▷ ▷ -end \
14   ▷ ▷ -define F -finite_field -q 5 -end \
15   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
16   ▷ ▷ -define C -spread_classifier \
17   ▷ ▷ ▷ -projective_space P \
18   ▷ ▷ ▷ -k 2 \
19   ▷ ▷ ▷ -starter_size 5 \
20   ▷ ▷ ▷ -recoordinatize \
21   ▷ ▷ ▷ -poset_classification_control Control \
22   ▷ ▷ ▷ -output_prefix "" \
23   ▷ ▷ -end \
24   ▷ ▷ -with C -do -spread_classify_activity \
25   ▷ ▷ ▷ -compute_starter \
26   ▷ ▷ -end \
27   ▷ ▷ -with C -do -spread_classify_activity \
28   ▷ ▷ ▷ -prepare_lifting_all_cases \
29   ▷ ▷ -end
30

```

recomputes the partial spreads of size $s = 5$ and prepares for the lifting of all orbit representatives (there are 28). This leads to 28 graphs, each of which is written to a file. The next command performs the rainbow clique finding in each of the 28 graphs:

Code Snippet 759

```

1    spreads_25_starter_cliques:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -loop L 0 29 1 \
4    ▷ ▷ ▷ -define G -graph -load spreads_25_graph_%L.bin -end \
5    ▷ ▷ ▷ -with G -do \
6    ▷ ▷ ▷ -graph_theoretic_activity \
7    ▷ ▷ ▷ ▷ -find_cliques -rainbow -target_size 21 \
8    ▷ ▷ ▷ ▷ -output_file spreads_25_graph_%L_sol.csv \
9    ▷ ▷ ▷ -end \
10   ▷ ▷ -end_loop L
11

```

The resulting cliques are stored in files. The command

Code Snippet 760

```

1    classify_spreads_25_isomorph:
2    ▷ $(ORBITER) -v 4 \
3    ▷ ▷ -define Do -draw_options \
4    ▷ ▷ ▷ -radius 20 \

```

```

5    ▷ ▷ ▷ -nodes_empty \
6    ▷ ▷ ▷ -line_width 0.2 \
7    ▷ ▷ ▷ -embedded \
8    ▷ ▷ -end \
9    ▷ ▷ -define Control -poset_classification_control \
10   ▷ ▷ ▷ -W \
11   ▷ ▷ ▷ -problem_label spreads_25 \
12   ▷ ▷ ▷ -draw_options Do \
13   ▷ ▷ -end \
14   ▷ ▷ -define Iso -isomorph_arguments \
15   ▷ ▷ ▷ -prefix_iso "./spreads_25" \
16   ▷ ▷ ▷ -use_database_for_starter \
17   ▷ ▷ ▷ -solution_prefix "spreads_25_graph" \
18   ▷ ▷ ▷ -base_fname "" \
19   ▷ ▷ -end \
20   ▷ ▷ -define F -finite_field -q 5 -end \
21   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
22   ▷ ▷ -define C -spread_classifier \
23   ▷ ▷ ▷ -projective_space P \
24   ▷ ▷ ▷ -k 2 \
25   ▷ ▷ ▷ -starter_size 5 \
26   ▷ ▷ ▷ -recoordinatize \
27   ▷ ▷ ▷ -poset_classification_control Control \
28   ▷ ▷ ▷ -output_prefix "" \
29   ▷ ▷ -end \
30   ▷ ▷ -with C -do -spread_classify_activity \
31   ▷ ▷ ▷ -compute_starter \
32   ▷ ▷ -end \
33   ▷ ▷ -with C -do -spread_classify_activity \
34   ▷ ▷ ▷ -isomorph Iso \
35   ▷ ▷ ▷ -build_db \
36   ▷ ▷ -end \
37   ▷ ▷ -with C -do -spread_classify_activity \
38   ▷ ▷ ▷ -isomorph Iso \
39   ▷ ▷ ▷ -read_solutions \
40   ▷ ▷ -end \
41   ▷ ▷ -with C -do -spread_classify_activity \
42   ▷ ▷ ▷ -isomorph Iso \
43   ▷ ▷ ▷ -compute_orbits \
44   ▷ ▷ -end \
45   ▷ ▷ -with C -do -spread_classify_activity \
46   ▷ ▷ ▷ -isomorph Iso \
47   ▷ ▷ ▷ -isomorph_testing \
48   ▷ ▷ -end \
49   ▷ ▷ -with C -do -spread_classify_activity \
50   ▷ ▷ ▷ -isomorph Iso \
51   ▷ ▷ ▷ -isomorph_report \
52   ▷ ▷ -end
53   ▷ $(PDFLATEX) spreads_25_isomorphism_types.tex
54   ▷ $(OPEN) spreads_25_isomorphism_types.pdf
55

```

performs the final isomorph rejection on the spreads arising from the rainbow cliques in all cases. It results

in a transversal of the isomorphism classes of spreads of $\text{PG}(3, 5)$. In total, 21 spreads are found. This agrees with the results in the literature, see [27].

Activities for Translation Planes		
Command	Arguments	Purpose
-export_incma		Export the incidence matrix to file.
-p_rank	p	Compute the p-rank of the incidence matrix for the given prime p .
-report		Produce a latex report.

Table 15.8: Activities for Translation Planes

15.2 Translation Planes

Orbiter can create translation planes from spreads. The construction of translation planes from spreads is due to André and Bruck, Bose (cf. [5, 22]). In order to perform the construction, we need a field $F = \mathbb{F}_q$ which is the kernel of the plane, a spread of k -subspaces, and the groups $\text{PTL}(2k, F)$ and $\text{PTL}(2k + 1, F)$.

Table 15.8 lists Orbiter activities for translation planes.

For instance, the command

Code Snippet 761

```

1   create_translation_plane_9b:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 3 -end \
4   ▷ ▷ -define G -linear_group -PGL 4 F -end \
5   ▷ ▷ -define G1 -linear_group -PGL 5 F -end \
6   ▷ ▷ -define S -spread -kernel_field F \
7   ▷ ▷ ▷ -group G -k 2 -catalogue 1 \
8   ▷ ▷ ▷ -end \
9   ▷ ▷ -define T -translation_plane S G G1 -end \
10  ▷ ▷ -with T -do -translation_plane_activity \
11  ▷ ▷ ▷ -export_incma \
12  ▷ ▷ -end \
13  ▷ ▷ -with T -do -translation_plane_activity \
14  ▷ ▷ ▷ -report \
15  ▷ ▷ -end \
16  ▷ ▷ -define A -linear_group -import_group_of_plane T -end \
17  ▷ ▷ -define Orb -orbits -group A \
18  ▷ ▷ ▷ -on_points \
19  ▷ ▷ -end \
20  ▷ ▷ -define Do -draw_options -embedded -sideways -radius 50 \
21  ▷ ▷ ▷ -scale 0.5 -line_width 0.3 \
22  ▷ ▷ -end \
23  ▷ ▷ -with Orb -do -orbits_activity \
24  ▷ ▷ ▷ -report Do \
25  ▷ ▷ -end \
26  ▷ ▷ -with Orb -do -orbits_activity \
27  ▷ ▷ ▷ -stabilizer 92 \
28  ▷ ▷ -end \
29  ▷ ▷ -with Orb -do -orbits_activity \
30  ▷ ▷ ▷ -export_trees \
31  ▷ ▷ -end
32  ▷ $(ORBITER) -v 2 \

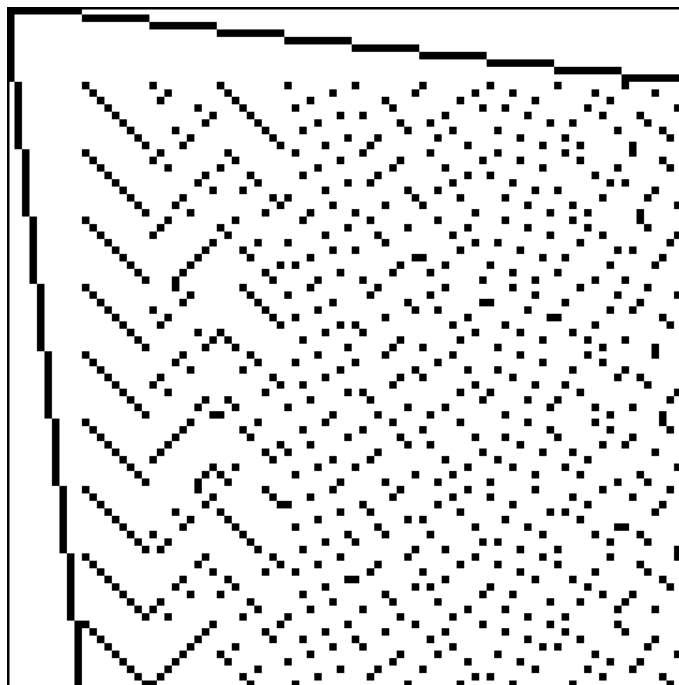
```

```

33  ▷ ▷ -draw_matrix \
34  ▷ ▷ ▷ -input_csv_file plane_catalogue_q3_k2_1_incma.csv \
35  ▷ ▷ ▷ -box_width 6 -bit_depth 8 \
36  ▷ ▷ ▷ -partition 2 91 91 \
37  ▷ ▷ -end
38  ▷ $(ORBITER) -v 3 \
39  ▷ ▷ -draw_layered_graph \
40  ▷ ▷ ▷ orbit_PGL_5_3_on_andre_3.layered_graph \
41  ▷ ▷ ▷ -place \
42  ▷ ▷ ▷ -radius 250 -spanning_tree -embedded -nodes_empty \
43  ▷ ▷ ▷ -line_width 1.1 -x_stretch 2.4 -scale 0.15 \
44  ▷ ▷ -end
45  ▷ $(PDFLATEX) orbit_PGL_5_3_on_andre_3.draw.tex
46  ▷ $(OPEN) orbit_PGL_5_3_on_andre_3.draw.pdf
47  ▷ $(PDFLATEX) group_of_plane_plane_catalogue_q3_k2_1.orbits_report.tex
48  ▷ $(OPEN) group_of_plane_plane_catalogue_q3_k2_1.orbits_report.pdf
49  ▷ $(PDFLATEX) group_of_plane_plane_catalogue_q3_k2_1.stab_pt_92_report.tex
50  ▷ $(OPEN) group_of_plane_plane_catalogue_q3_k2_1.stab_pt_92_report.pdf
51

```

creates the (projective) Hall plane of order 9 from the Hall spread. In this example, we use the fact that $P\Gamma L(n, q) = PGL(n, q)$ if q is prime. The example also creates a bitmap drawing of the incidence matrix of the plane, shown below:



In the next example, we will create a translation plane of order 16 with kernel of order 4:

Code Snippet 762

```

1   create_translation_plane_16_4_0:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 4 -end \
4   ▷ ▷ -define G -linear_group -PGGL 4 F -end \
5   ▷ ▷ -define G1 -linear_group -PGGL 5 F -end \
6   ▷ ▷ -define S -spread -kernel_field F \
7   ▷ ▷ ▷ -group G -k 2 -catalogue 0 \
8   ▷ ▷ ▷ -end \
9   ▷ ▷ -define T -translation_plane S G G1 -end \
10  ▷ ▷ -with T -do -translation_plane_activity \
11  ▷ ▷ ▷ -export_incma \
12  ▷ ▷ -end
13  ▷ $(ORBITER) -v 2 \
14  ▷ ▷ -draw_matrix \
15  ▷ ▷ ▷ -input_csv_file plane_catalogue_q4_k2_0_incma.csv \
16  ▷ ▷ ▷ -box_width 6 -bit_depth 8 \
17  ▷ ▷ ▷ -partition 4 273 273 \
18  ▷ ▷ -end
19  ▷ $(OPEN) plane_catalogue_q4_k2_0_incma_draw.bmp
20

```

This plane is the Hall plane, and the spread is the Hall spread. The stabilizer of the spread has order 1200.

In the next example, we will create a translation plane of order 16 with kernel of order 2:

Code Snippet 763

```

1   create_translation_plane_16_2_0:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 2 -end \
4   ▷ ▷ -define G -linear_group -PGL 8 F -end \
5   ▷ ▷ -define G1 -linear_group -PGL 9 F -end \
6   ▷ ▷ -define S -spread -kernel_field F \
7   ▷ ▷ ▷ -group G -k 4 -catalogue 0 \
8   ▷ ▷ ▷ -end \
9   ▷ ▷ -define T -translation_plane S G G1 -end \
10  ▷ ▷ -with T -do -translation_plane_activity \
11  ▷ ▷ ▷ -export_incma \
12  ▷ ▷ -end
13  ▷ $(ORBITER) -v 2 \
14  ▷ ▷ -draw_matrix \
15  ▷ ▷ ▷ -input_csv_file plane_catalogue_q2_k4_0_incma.csv \
16  ▷ ▷ ▷ -box_width 6 -bit_depth 8 \
17  ▷ ▷ ▷ -partition 4 273 273 \
18  ▷ ▷ -end
19  ▷ $(OPEN) plane_catalogue_q2_k4_0_incma_draw.bmp
20

```

The spread has a stabilizer of order 1008, which means that the associated translation plane has a stabilizer of order $1008 \cdot 256 = 258045$. According to [58], there are two planes whose associated spreads have an automorphism group of this order. They can be distinguished by the 2-rank of their incidence matrices. The

Johnson-Walker plane has a 2-rank of 100. The Lorimer-Rahilly plane has a 2-rank of 106. Using Orbiter, we compute the 2-rank of the translation plane that we have created:

Code Snippet 764

```

1   RREF_plane_16_2_0_rank_of_incma:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 2 -end \
4   ▷ ▷ -define v -vector -field F \
5   ▷ ▷ ▷ -file plane_catalogue_q2_k4_0_incma.csv \
6   ▷ ▷ -end \
7   ▷ ▷ -with F -do -finite_field_activity \
8   ▷ ▷ -RREF v \
9   ▷ ▷ -end
10
```

It turns out that the 2-rank of our plane is 106, so the plane is Lorimer-Rahilly.

Let us investigate the Rao / Rao plane from Section 15.1, which we know is isomorphic to the spread in the Orbiter catalogue with number 0 and projective stabilizer of order 84. The command

Code Snippet 765

```

1   create_translation_plane_27_Rao_Rao:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 3 -end \
4   ▷ ▷ -define SS -vector -dense $(SPREAD_SET_27_RA0_RA0) -end \
5   ▷ ▷ -define G -linear_group -PGL 6 F -end \
6   ▷ ▷ -define G1 -linear_group -PGL 7 F -end \
7   ▷ ▷ -define S -spread -kernel_field F \
8   ▷ ▷ ▷ -group G -k 3 -spread_set "Rao_Rao" "Rao\_Rao" SS \
9   ▷ ▷ ▷ -end \
10  ▷ ▷ -define T -translation_plane S G G1 -end \
11  ▷ ▷ -with T -do -translation_plane_activity \
12  ▷ ▷ ▷ -export_incma \
13  ▷ ▷ -end
14
```

creates the translation plane from the spread. To compute the 3-rank of the incidence matrix, we issue the following command:

Code Snippet 766

```

1   RREF_Rao_Rao_plane_incma_rank:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 3 -end \
4   ▷ ▷ -define v -vector -field F \
5   ▷ ▷ ▷ -file plane_Rao_Rao_incma.csv \
6   ▷ ▷ -end \
7   ▷ ▷ -with F -do -finite_field_activity \
8   ▷ ▷ ▷ -RREF v \
9   ▷ ▷ -end
10
```

The 3-rank turns out to be 271. According to the Moorhouse tables [59], the plane has Moorhouse number IV.

Suppose we want to study the orbits of the group of a translation plane on points and blocks. The following command creates the translation plane of order 27 with Orbiter number 4. Once done, it creates the action on points and blocks and invokes the Schreier orbit algorithm (see Section 8.1).

Code Snippet 767

```

1      translation_plane_27_4_orbits:
2      ▷  $(ORBITER) -v 3 \
3      ▷  ▷  -define F -finite_field -q 3 -end \
4      ▷  ▷  -define G -linear_group -PGL 6 F -end \
5      ▷  ▷  -define G1 -linear_group -PGL 7 F -end \
6      ▷  ▷  -define S -spread -kernel_field F \
7      ▷  ▷  ▷  -group G -k 3 -catalogue 4 \
8      ▷  ▷  ▷  -end \
9      ▷  ▷  -define T -translation_plane S G G1 -end \
10     ▷  ▷  -with T -do -translation_plane_activity \
11     ▷  ▷  ▷  -export_incma \
12     ▷  ▷  -end \
13     ▷  ▷  -with T -do -translation_plane_activity \
14     ▷  ▷  ▷  -report \
15     ▷  ▷  -end \
16     ▷  ▷  -define A -linear_group -import_group_of_plane T -end \
17     ▷  ▷  -define Orb -orbits -group A \
18     ▷  ▷  ▷  -on_points \
19     ▷  ▷  -end
20
```

The group of the plane has order 1592136. The degree of the action is 1514. The orbit lengths are

1, 28, 729, 756.

Creating Packings with Assumed Symmetry		
Command	Arguments	Purpose
-H	description	Specify the assumed group H of symmetries. The orbits of H on the set of spreads are considered. The packings will be constructed as union of orbits.
-N	description	Specify the normalizer of H .
-cliques_on_fixpoint_graph	s	Using poset classification, classify the orbits of N on cliques of size $\leq s$ in the graph on fixed points.
-cliques_on_fixpoint_graph_control	descr	Specify poset classification options related to the classification of cliques on the fixed point graph as in Table 8.4.
-fixp_clique_types_save_individually		Sort the cliques on fixed points by the type of their spreads and write one csv file for each possible type containing the index of the cliques of the given type.
-process_long_orbits	descr	Proceed on to long orbits using Table 15.13.
-spread_tables_prefix	P	Use prefix P to access spread tables.
-report		Create a report of the classification process.
-regular_packing		Initialize Klein correspondence and identify (regular) spreads with external lines to the Klein quadric using the polarity of the Klein quadric.

Table 15.9: Creating Packings with Assumed Symmetry

15.3 Packings

A packing of $\text{PG}(3, q)$ is a set of pairwise line-disjoint spreads of $\text{PG}(3, q)$ of size $q^2 + q + 1$. Each spread contains $q^2 + 1$ lines. A simple counting argument shows that every line is contained in exactly one spread of the packing. The classification problem for packings is the problem of determining a complete set of pairwise non-isomorphic packings. Orbiter can be used to classify packings for small parameters. It is sometimes useful to make a symmetry assumption. This means that only those packings will be found that satisfy the symmetry assumption. The reason for making such an assumption is that the problem becomes easier and hence more tractable. Often, an assumption is made that the packings are invariant under a (nontrivial) group H . This section describes various ways in which Orbiter can help find and classify packings, with or without symmetry assumption.

Table 15.9 list Orbiter commands related to the construction of packings with assumed symmetry.

Table 15.10 list Orbiter activities for the classification of packings.

Table 15.11 list Orbiter activities for packings with assumed symmetry.

Table 15.12 list Orbiter activities for packings with assumed symmetry and with fixpoints.

Table 15.13 list Orbiter commands related to the construction of packings with assumed symmetry by picking long orbits.

There are two isomorphism types of spreads in $\text{PG}(3, 3)$. The following command computes the orbits of each and creates a table of all (labeled) spreads in $\text{PG}(3, 3)$.

Classification of Packings		
Command	Arguments	Purpose
-report		Produce a report.
-classify	control	Classify packings using poset classification. Use control for arguments.
-make_graph_of_disjoint_spreads		Make the graph of disjoint spreads.
-export_group_on_spreads		Export the group in the action on the spreads.

Table 15.10: Classification of Packings

Activities for Packings		
Command	Arguments	Purpose
-report		Produce a latex report
-export_reduced_spread_orbits	fname	Export reduced spread orbits.
-create_graph_on_mixed_orbits	L	Create the graph of orbits whose length is listed in L.

Table 15.11: Activities for Packings

Activities for Packings with Fixpoints		
Command	Arguments	Purpose
-report		Produce a latex report
-print_packing	text	Print the packing.
-compare_files_of_packings	fname1 fname2	Compare packings in the two given files.

Table 15.12: Activities for Packings with Fixpoints

Creating Packings from Long Orbits		
Command	Arguments	Purpose
-split	r m	Define a subset of cases of fixed point cliques to be worked on. Only those cases whose number is congruent to r modulo m are considered.
-orbit_length	l	Use orbits of length l .
-mixed_orbits	length	
-list_of_cases_from_file	fname	Define a subset of cases of fixed point cliques to be worked on. Only the cases listed the given file are considered.
-solution_path	P	Use P as a prefix for all solution files.
-create_graphs		For each case, create the graph that describes whether two orbits of length l are compatible.
-solve		Perform clique finding and write solutions to file.
-read_solutions		Read solutions from file.

Table 15.13: Creating Packings from Long Orbits

Code Snippet 768

```

1    PG_3_3_spread_table:
2    ▷ - mkdir SPREAD_TABLES_3
3    ▷ $(ORBITER) -v 5 \
4    ▷ ▷ -define Control -poset_classification_control \
5    ▷ ▷ ▷ -problem_label spreads_PG_3_3 \
6    ▷ ▷ ▷ -W \
7    ▷ ▷ -end \
8    ▷ ▷ -define F -finite_field -q 3 -end \
9    ▷ ▷ -define P3 -projective_space -n 3 -field F -v 0 -end \
10   ▷ ▷ -define P5 -projective_space -n 5 -field F -v 0 -end \
11   ▷ ▷ -define T -spread_table -space P3 -rk 2 \
12   ▷ ▷ ▷ -iso_types "0,1" \
13   ▷ ▷ ▷ -path "SPREAD_TABLES_3/" \
14   ▷ ▷ ▷ -control Control \
15   ▷ ▷ ▷ -end \
16

```

There are 21 isomorphism types of spreads in $\text{PG}(3, 5)$. The regular spread has Orbiter catalogue number equal to 12. The following command creates a table of all regular spreads in $\text{PG}(3, 5)$:

Code Snippet 769

```

1    spread_table_PG_3_5_regular:
2    ▷ - mkdir SPREAD_TABLES_5_REG
3    ▷ $(ORBITER) -v 6 \
4    ▷ ▷ -define Control -poset_classification_control \
5    ▷ ▷ ▷ -problem_label spreads_PG_3_5 \
6    ▷ ▷ -end \
7    ▷ ▷ -define F -finite_field -q 5 -end \
8    ▷ ▷ -define P3 -projective_space -n 3 -field F -end \
9    ▷ ▷ -define T -spread_table -space P3 -rk 2 \
10   ▷ ▷ ▷ -iso_types "12" \
11   ▷ ▷ ▷ -path "SPREAD_TABLES_5_REG/" \
12   ▷ ▷ ▷ -control Control \
13   ▷ ▷ ▷ -end \
14   ▷ ▷ -print_symbols
15

```

There are 155000 regular spreads.

Suppose we are interested in spreads of $\text{PG}(3, 3)$ invariant under a group of order 5. To do so, we can apply a sequence of Orbiter commands as follows. To begin with, we need to find a generator for a subgroup of order 5 in $\text{PGL}(4, 3)$. To this end, we compute the conjugacy classes of the group, using the following command:

Code Snippet 770

```

1    PGL_4_3_classes:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define Do -draw_options \
4    ▷ ▷ ▷ -radius 200 -line_width 0.3 \

```



```

5    ▷ ▷ -end \
6    ▷ ▷ -define G -linear_group -PGL 4 3 -end \
7    ▷ ▷ -with G -do \
8    ▷ ▷ -group_theoretic_activity \
9    ▷ ▷ ▷ -report Do \
10   ▷ ▷ -end \
11   ▷ ▷ -with G -do \
12   ▷ ▷ -group_theoretic_activity \
13   ▷ ▷ ▷ -classes \
14   ▷ ▷ -end
15   ▷ $(PDFLATEX) PGL_4.3_report_classes_of_elements.tex
16   ▷ $(OPEN) PGL_4.3_report_classes_of_elements.pdf
17

```

The actual computation is done in Magma, but Orbiter provides us the group as matrix group and produces a latex report. The report contains all the information we need. At first, we pick the representative of the unique class of order 5 (note that Magma does not behave deterministically, so different runs of Magma may result in different elements of order 5):

Code Snippet 771

```

1    PGL_4.3_CLASS_5A_REP="2,2,2,0,1,1,1,2,2,1,0,0,1,0,1,2"
2

```

Let H be the subgroup generated by this element. We also need the generators for the normalizer N of H . The subgroup N has order 160, and generators can be found in the report on conjugacy classes as well. Using copy-paste, we store the generators in a makefile variable like so:

Code Snippet 772

```

1    PGL_4.3_CLASS_5A_N_GENs="1,0,0,0,2,0,1,1,2,1,2,1,1,1,0,0, 1,1,1,0,2,2,2,1,1,2,0,0
    ,2,0,2,1, 1,1,1,2,2,2,0,1,0,2,2,0,1,2,2,1"
2

```

The next step is to prepare for a search for all packings invariant under the chosen group H of order 5. To this end, we apply the following command:

Code Snippet 773

```

1    PG_3.3_assume_5_graph:
2    ▷ $(ORBITER) -v 5 \
3    ▷ ▷ -define Control -poset_classification_control \
4    ▷ ▷ ▷ -problem_label spreads_PG_3.3 \
5    ▷ ▷ ▷ -W \
6    ▷ ▷ -end \
7    ▷ ▷ -define Control_Fix -poset_classification_control \
8    ▷ ▷ ▷ -problem_label N5_fixp_cliques \
9    ▷ ▷ ▷ -W \
10   ▷ ▷ -end \
11   ▷ ▷ -define F -finite_field -q 3 -end \
12   ▷ ▷ -define P3 -projective_space -n 3 -field F -v 0 -end \
13   ▷ ▷ -define P5 -projective_space -n 5 -field F -v 0 -end \

```

```

14  ▷ ▷ -define T -spread_table -space P3 -rk 2 \
15  ▷ ▷ ▷ -iso_types "0,1" \
16  ▷ ▷ ▷ -path "SPREAD.TABLES_3/" \
17  ▷ ▷ ▷ -control Control \
18  ▷ ▷ ▷ -end \
19  ▷ ▷ -define PC -packing_classify P3 P5 T \
20  ▷ ▷ -define PW -packing_with_symmetry_assumption PC \
21  ▷ ▷ ▷ ▷ -H "H5" -PGL 4 3 -subgroup_by_generators \
22  ▷ ▷ ▷ ▷ ▷ "5" 5 1 $(PGL_4_3_CLASS_5A_REP) -end \
23  ▷ ▷ ▷ ▷ -N "N5" -PGL 4 3 -subgroup_by_generators \
24  ▷ ▷ ▷ ▷ ▷ "N5" 160 3 $(PGL_4_3_CLASS_5A_N_GENS) -end \
25  ▷ ▷ ▷ -end \
26  ▷ ▷ -define PWF -packing_choose_fixed_points PW 3 Control.Fix \
27  ▷ ▷ -define Long -packing_long_orbits PWF \
28  ▷ ▷ ▷ -orbit_length 5 \
29  ▷ ▷ ▷ -list_of_cases_from_file N5_fixp_cliques.csv \
30  ▷ ▷ ▷ -create_graphs \
31  ▷ ▷ ▷ -end \
32  ▷ ▷ -print_symbols
33

```

It produces the orbits of the group H on packings, filtering out those orbits which cannot be used for packings because the spreads are not line disjoint. It then classifies the orbits by length. The only possible lengths are one and 5. The orbits of length one are called the fixpoints. Because we need 13 spreads in a packing, we must have at least 3 fixed points (recall that point means spread). The command classifies the ways in which one can pick 3 fixed points up to the action of N . It turns out that there are three ways, up to isomorphism. This creates three disjoint cases. For each case, we need to continue the construction by adding two orbits of length 5. In order to do so, we consider each of the three cases separately. In each case, we first filter the set of orbits of length 5 according to whether they are compatible with the fixed spread configuration. This means that the spreads in the orbit are supposed to be line disjoint from the spreads in the three fixed spreads. Once done, we compute a graph. The vertices of the graph are the orbits of spreads of length 5 that are compatible with the fixed point configuration. Two vertices are adjacent in the graph if the associated spread orbits are compatible, which means that they are line-disjoint. The packings that contain the fixed configuration and consist of an additional set of 2 orbits of length 5 are in one-to-one correspondence to the cliques of size 2 in the graph. We employ Orbiter's clique finding algorithm (see Section 16.1) to compute all these cliques (in each of the three cases):

Code Snippet 774

```

1  PG_3_3_assume_5_fpc0_lo_cliques:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define G -graph -load H5_fpc0_lo.graph -end \
4  ▷ ▷ -with G -do \
5  ▷ ▷ ▷ -graph_theoretic_activity \
6  ▷ ▷ ▷ -find_cliques -target_size 2 -output_file H5_fpc0_lo_cliques.csv -end \
7  ▷ ▷ -print_symbols
8

```

Code Snippet 775

```

1  PG_3_3_assume_5_fpc1_lo_cliques:
2  ▷ $(ORBITER) -v 2 \

```

```

3   ▷ ▷ -define G -graph -load H5_fpc1_lo.graph -end \
4   ▷ ▷ -with G -do \
5   ▷ ▷ ▷ -graph_theoretic_activity \
6   ▷ ▷ ▷ -find_cliques -target_size 2 -output_file H5_fpc1_lo_cliques.csv -end \
7   ▷ ▷ -print_symbols
8

```

Code Snippet 776

```

1   PG_3_3_assume_5_fpc2_lo_cliques:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define G -graph -load H5_fpc2_lo.graph -end \
4   ▷ ▷ -with G -do \
5   ▷ ▷ ▷ -graph_theoretic_activity \
6   ▷ ▷ ▷ -find_cliques -target_size 2 -output_file H5_fpc2_lo_cliques.csv -end \
7   ▷ ▷ -print_symbols
8

```

Once we are done with the cliques, we need to assemble the packings that arise by joining the orbits of length 5 associated with the elements of the clique to the fixed point configuration. Once this is done, we assemble the packings:

Code Snippet 777

```

1   PG_3_3_assume_5_read_solution:
2   ▷ $(ORBITER) -v 5 \
3   ▷ ▷ -define Control -poset_classification_control \
4   ▷ ▷ ▷ -problem_label spreads_PG_3_3 \
5   ▷ ▷ ▷ -W \
6   ▷ ▷ -end \
7   ▷ ▷ -define Control_Fix -poset_classification_control \
8   ▷ ▷ ▷ -problem_label N5_fixp_cliques \
9   ▷ ▷ ▷ -W \
10  ▷ ▷ -end \
11  ▷ ▷ -define F -finite_field -q 3 -end \
12  ▷ ▷ -define P3 -projective_space -n 3 -field F -v 0 -end \
13  ▷ ▷ -define P5 -projective_space -n 5 -field F -v 0 -end \
14  ▷ ▷ -define T -spread_table -space P3 -rk 2 \
15  ▷ ▷ ▷ -iso_types "0,1" \
16  ▷ ▷ ▷ -path "SPREAD_TABLES_3/" \
17  ▷ ▷ ▷ -control Control \
18  ▷ ▷ ▷ -end \
19  ▷ ▷ -define PC -packing_classify P3 P5 T \
20  ▷ ▷ -define PW -packing_with_symmetry_assumption PC \
21  ▷ ▷ ▷ ▷ -H "H5" -PGL 4 3 -subgroup_by_generators \
22  ▷ ▷ ▷ ▷ ▷ "5" 5 1 $(PGL_4_3_CLASS_5A.REP) -end \
23  ▷ ▷ ▷ ▷ -N "N5" -PGL 4 3 -subgroup_by_generators \
24  ▷ ▷ ▷ ▷ ▷ "N5" 160 3 $(PGL_4_3_CLASS_5A.N.GENS) -end \
25  ▷ ▷ ▷ -end \
26  ▷ ▷ -define PWF -packing_choose_fixed_points PW 3 Control_Fix \
27  ▷ ▷ -define Long -packing_long_orbits PWF \
28  ▷ ▷ ▷ -orbit_length 5 \

```

```

29  ▷ ▷ ▷ -list_of_cases_from_file N5_fixp_cliques.csv \
30  ▷ ▷ ▷ -read_solutions \
31  ▷ ▷ ▷ -end \
32  ▷ ▷ -print_symbols
33

```

As it turns out, the clique finding commands find $0 + 4 + 10 = 14$ cliques, for a total of 14 packings. However some of these packings can be isomorphic. For this reason, we perform a final isomorphism test, based on Nauty. We use Nauty through the Orbiter interface (for more on that, see Chapter 17). The following command does the job:

Code Snippet 778

```

1  PG_3_3_assume_5_packings_classify:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define C -combinatorial_object \
4  ▷ ▷ ▷ -label PG_3_3_assuming_5 \
5  ▷ ▷ ▷ "${\rm PG\3\3\_assuming\5}" \
6  ▷ ▷ ▷ -file_of_packings_through_spread_table \
7  ▷ ▷ ▷ H5_packings.csv \
8  ▷ ▷ ▷ SPREAD_TABLES_3/spread_9_table.csv \
9  ▷ ▷ ▷ SpreadRep \
10 ▷ ▷ ▷ ▷ 3 \
11 ▷ ▷ -end \
12 ▷ ▷ -define F -finite_field -q 3 -end \
13 ▷ ▷ -define P3 -projective_space -n 3 -field F -v 0 -end \
14 ▷ ▷ -with C -do \
15 ▷ ▷ -combinatorial_object_activity \
16 ▷ ▷ ▷ -canonical_form_PG P3 \
17 ▷ ▷ ▷ -nauty_control -nauty_log "PG_3_3_packings_5_nauty_log.txt" -end \
18 ▷ ▷ ▷ -end \
19 ▷ ▷ -end \
20 ▷ ▷ -define Doinc -draw_incidence_structure_options \
21 ▷ ▷ ▷ -width 40 -width_10 6 \
22 ▷ ▷ -end \
23 ▷ ▷ -with C -do \
24 ▷ ▷ -combinatorial_object_activity \
25 ▷ ▷ ▷ -post_processing \
26 ▷ ▷ -end \
27 ▷ ▷ -with C -do \
28 ▷ ▷ -combinatorial_object_activity \
29 ▷ ▷ ▷ -report \
30 ▷ ▷ ▷ ▷ -export_flag_orbits \
31 ▷ ▷ ▷ ▷ -show_TDO \
32 ▷ ▷ ▷ ▷ -show_TDA \
33 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
34 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
35 ▷ ▷ ▷ -end \
36 ▷ ▷ -end
37 ▷ $(PDFLATEX) PG_3_3_assuming_5_classification.tex
38 ▷ $(OPEN) PG_3_3_assuming_5_classification.pdf
39

```

We end up with a list of isomorphism types of packings in $\text{PG}(3, 3)$ invariant under the chosen group H of order 5. Finally, the command prepares a latex report for these packings.

Creating BLT-Sets		
Command	Arguments	Purpose
-catalogue	i	Create the BLT-set number i from the Orbiter catalogue (i is zero-based).
-family	name	Create a BLT-set from the named family. See Table 15.15 for possibilities for name.
-flock	flock	Create a BLT-set from a given flock.
-space	space	Set the space.
-invariants		Compute isomorphism invariants.

Table 15.14: Creating BLT-Sets

Families of BLT-sets		
Label	Condition	Explanation
Linear		Linear BLT-set.
Fisher		Fisher BLT-set [31].
Mondello	$q \equiv \pm 1 \pmod{10}$	Mondello BLT-set due to Penttila [61].
FTWKB	$q \equiv \pm 2 \pmod{3}$	Fisher, Thas, Walker [74], Kantor, Betten [19] BLT-set.
Kantor1	$q = p^e, e > 1$	Kantor's first family.
Kantor2	$q \equiv \pm 2 \pmod{5}$	Kantor's second family.
LP_37_72	$q = 37$	BLT-set for $q = 37$ with ago=72 due to Law and Penttila [50].
LP_37_41a	$q = 37$	First BLT-set for $q = 37$ with ago=4, due to Law and Penttila [50].
LP_37_41b	$q = 37$	Second BLT-set for $q = 37$ with ago=4, due to Law and Penttila [50].
LP_71	$q = 71$	BLT-set for $q = 71$ due to Law and Penttila [50].

Table 15.15: Families of BLT-sets

15.4 BLT-Sets

A BLT-set of $Q(4, q)$ is a set of $q + 1$ point on the quadric such that no point on the quadric is collinear to more than two points of the set. BLT sets are related to spreads of $PG(3, q)$, to flocks of the quadratic cone in $PG(3, q)$, and other objects in finite geometry. These sets have been introduced in [6] for the purposes of creating new flocks from old. A BLT-set exists if and only if q is odd. It is an interesting problem to classify BLT-sets of $Q(4, q)$ under the orthogonal group. Some references are Law [48], Penttila-Royle [62], Penttila-Law [49, 50], Betten [12], AlAzemi-Betten-Chowdhury [2]. For small values of q , computer search has been used extensively. As we will discuss below, Orbiter is useful to supporting such computations.

Before we go into the problem of classification, let us first discuss how known BLT-sets can be constructed in Orbiter. Orbiter maintains a catalogue of BLT-sets over small fields. Besides that, Orbiter can be used to create instances of known families of BLT-sets.

Table 15.14 shows options to create known BLT-sets.

Table 15.15 shows names for known families of sporadic sets.

BLT-Set Activity		
Command	Arguments	Purpose
-report		Produce a latex report.
-export_gap	index	Export to GAP/fining based on the indexed point.
-create_flock		Create a flock.
-BLT_test		Test of the set has the BLT-property.
-export_set_in_PG		Export the set in ranks of the underlying projective space.
-plane_invariant		Compute the plane invariant.

Table 15.16: BLT-Set Activity

Classifying BLT-Sets		
Command	Arguments	Purpose
-orthogonal_space	O	Specify the orthogonal space.
-starter_size	s	Set the size of partial BLT-sets that is used as subobjects.

Table 15.17: Classifying BLT-Sets

Table 15.16 shows Orbiter activities for a BLT-set object.

Table 15.17 shows options to prepare for the classification of BLT-sets.

Table 15.18 shows commands for the classification of BLT-sets.

Let us demonstrate some examples. The command

Code Snippet 779

```

1   BLT_5_1:
2   ▷ $(ORBITER) -v 4 \
3   ▷ ▷ -define F -finite_field -q 5 -end \
4   ▷ ▷ -define O -orthogonal_space 0 5 F -create_extension_fields -end \
5   ▷ ▷ -define BLT -BLT_set \
6   ▷ ▷ ▷ -space 0 -catalogue 1 -invariants \

```

Activities for the Classification of BLT-sets		
Command	Arguments	Purpose
-compute_starter		Perform classification of partial spreads.
-poset_classification_activity	label	
-create_graphs		Prepare to lift all cases of partial BLT-sets by creating the associated colored graphs.
-split	r m	Lift only those cases k with $k \equiv r \pmod{m}$.
-isomorph	options	Perform isomorphism testing after lifting is complete.

Table 15.18: Activities for the Classification of BLT-sets

```

7    ▷ ▷ -end \
8    ▷ ▷ -with BLT -do -blt_set_activity \
9    ▷ ▷ ▷ -report \
10   ▷ ▷ -end
11   ▷ $(PDFLATEX) BLT_catalogue_q5_iso1.tex
12   ▷ $(OPEN) BLT_catalogue_q5_iso1.pdf
13

```

creates the first BLT-set of order 5 from the catalogue. Suppose we want to export the BLT-set to GAP/fining. The following command can do that:

Code Snippet 780

```

1    BLT_5_1.export_gap:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 5 -end \
4    ▷ ▷ -define O -orthogonal_space 0 5 F -create_extension_fields -end \
5    ▷ ▷ -define BLT -BLT_set \
6    ▷ ▷ ▷ -space 0 -catalogue 1 \
7    ▷ ▷ -end \
8    ▷ ▷ -with BLT -do -blt_set_activity \
9    ▷ ▷ ▷ -export_gap \
10   ▷ ▷ -end
11

```

The command creates the following GAP/fining file:

```

LoadPackage("fining");
# BLT-set catalogue_q5_iso1
# Quadratic form:  $X_0^2 + X_1X_2 + X_3X_4$ 
# Group of order 720
mat1 := [[Z(5)^2, 0*Z(5), 0*Z(5), Z(5)^3, 0*Z(5)],
[0*Z(5), Z(5)^2, 0*Z(5), 0*Z(5), 0*Z(5)],
[0*Z(5), 0*Z(5), Z(5)^2, 0*Z(5), 0*Z(5)],
[0*Z(5), 0*Z(5), 0*Z(5), Z(5)^0, 0*Z(5)],
[Z(5)^2, 0*Z(5), 0*Z(5), Z(5)^2, Z(5)^0]];
frob1 := FrobeniusAutomorphism(GF(5))^0;
psi1 := ProjectiveSemilinearMap(mat1, frob1, GF(5));
mat2 := [[Z(5)^0, 0*Z(5), 0*Z(5), Z(5)^2, 0*Z(5)],
[0*Z(5), Z(5)^2, 0*Z(5), 0*Z(5), 0*Z(5)],
[0*Z(5), 0*Z(5), Z(5)^2, 0*Z(5), 0*Z(5)],
[0*Z(5), 0*Z(5), 0*Z(5), Z(5)^1, 0*Z(5)],
[Z(5)^2, 0*Z(5), 0*Z(5), Z(5)^3, Z(5)^3]];
frob2 := FrobeniusAutomorphism(GF(5))^0;
psi2 := ProjectiveSemilinearMap(mat2, frob2, GF(5));
mat3 := [[Z(5)^3, 0*Z(5), 0*Z(5), Z(5)^1, Z(5)^0],
[0*Z(5), Z(5)^2, 0*Z(5), 0*Z(5), 0*Z(5)],
[0*Z(5), 0*Z(5), Z(5)^2, 0*Z(5), 0*Z(5)],
[Z(5)^3, 0*Z(5), 0*Z(5), Z(5)^2, Z(5)^2],
[Z(5)^0, 0*Z(5), 0*Z(5), Z(5)^0, Z(5)^2]];
frob3 := FrobeniusAutomorphism(GF(5))^0;
psi3 := ProjectiveSemilinearMap(mat3, frob3, GF(5));

```



```

mat4 := [[Z(5)^3, 0*Z(5), 0*Z(5), Z(5)^0, Z(5)^1],
[0*Z(5), Z(5)^2, 0*Z(5), 0*Z(5), 0*Z(5)],
[0*Z(5), 0*Z(5), Z(5)^2, 0*Z(5), 0*Z(5)],
[Z(5)^2, 0*Z(5), 0*Z(5), Z(5)^0, Z(5)^2],
[Z(5)^1, 0*Z(5), 0*Z(5), Z(5)^0, Z(5)^0]];
frob4 := FrobeniusAutomorphism(GF(5))^0;
psi4 := ProjectiveSemilinearMap(mat4, frob4, GF(5));
mat5 := [[Z(5)^1, Z(5)^0, 0*Z(5), Z(5)^1, Z(5)^0],
[0*Z(5), Z(5)^0, 0*Z(5), 0*Z(5), 0*Z(5)],
[Z(5)^3, Z(5)^3, Z(5)^0, Z(5)^0, Z(5)^3],
[Z(5)^3, Z(5)^3, 0*Z(5), Z(5)^0, Z(5)^0],
[Z(5)^0, Z(5)^0, 0*Z(5), Z(5)^2, Z(5)^0]];
frob5 := FrobeniusAutomorphism(GF(5))^0;
psi5 := ProjectiveSemilinearMap(mat5, frob5, GF(5));
mat6 := [[Z(5)^1, 0*Z(5), 0*Z(5), Z(5)^3, Z(5)^2],
[Z(5)^0, Z(5)^0, Z(5)^1, Z(5)^1, Z(5)^0],
[0*Z(5), 0*Z(5), Z(5)^0, 0*Z(5), 0*Z(5)],
[Z(5)^3, 0*Z(5), Z(5)^1, Z(5)^3, Z(5)^1],
[Z(5)^0, 0*Z(5), Z(5)^2, Z(5)^3, Z(5)^3]];
frob6 := FrobeniusAutomorphism(GF(5))^0;
psi6 := ProjectiveSemilinearMap(mat6, frob6, GF(5));
gens := [psi1, psi2, psi3, psi4, psi5, psi6];
G := Group(gens);
Size(G);
pg := ProjectiveSpace(4,5);
S:=
[0*Z(5), Z(5)^0, 0*Z(5), 0*Z(5), 0*Z(5)],
[0*Z(5), 0*Z(5), Z(5)^0, 0*Z(5), 0*Z(5)],
[0*Z(5), Z(5)^0, Z(5)^1, Z(5)^2, Z(5)^1],
[Z(5)^0, Z(5)^0, Z(5)^1, Z(5)^1, Z(5)^0],
[Z(5)^0, Z(5)^3, Z(5)^0, Z(5)^2, Z(5)^2],
[Z(5)^0, Z(5)^3, Z(5)^0, Z(5)^3, Z(5)^1]
];
S := List(S, x -> VectorSpaceToElement(pg, x));

```

The command

Code Snippet 781

```

1   BLT_11_0:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 11 -end \
4   ▷ ▷ -define O -orthogonal_space 0 5 F -create_extension_fields -end \
5   ▷ ▷ -define BLT -BLT_set \
6   ▷ ▷ ▷ -space 0 -catalogue 0 -invariants \
7   ▷ ▷ -end \
8   ▷ ▷ -with BLT -do -blt_set_activity \
9   ▷ ▷ ▷ -report \
10  ▷ ▷ -end
11  ▷ $(PDFLATEX) BLT_catalogue.q11_iso0.tex
12  ▷ $(OPEN) BLT_catalogue.q11_iso0.pdf
13

```

creates the BLT-set #0 in $Q(4, 11)$. The command

Code Snippet 782

```

1   BLT_11_Mondello:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 11 -end \
4   ▷ ▷ -define O -orthogonal_space 0 5 F -end \
5   ▷ ▷ -define BLT -BLT_set \
6   ▷ ▷ ▷ -space 0 -family "Mondello" -invariants \
7   ▷ ▷ -end \
8   ▷ ▷ -with BLT -do -blt_set_activity \
9   ▷ ▷ ▷ -report \
10  ▷ ▷ -end
11  ▷ $(PDFLATEX) BLT_Mondello.q11.tex
12  ▷ $(OPEN) BLT_Mondello.q11.pdf
13

```

creates the Mondello BLT-set in $Q(4, 11)$. Orbiter creates the following report:

The quadratic form is:

$$X_0^2 + X_1X_2 + X_3X_4 = 0$$

The BLT-set is:

i	Rank	Point	(a, b, c)
0	846	(1, 6, 4, 10, 3)	(22, 11, 1)
1	851	(1, 5, 7, 10, 3)	(22, 110, 1)
2	1234	(1, 5, 1, 7, 7)	(37, 11, 1)
3	613	(1, 6, 10, 5, 1)	(73, 110, 1)
4	1307	(1, 1, 3, 8, 5)	(59, 36, 1)
5	1418	(1, 3, 9, 6, 10)	(95, 36, 1)
6	1022	(1, 9, 5, 10, 2)	(99, 96, 1)
7	835	(1, 2, 6, 3, 3)	(99, 36, 1)
8	950	(1, 10, 8, 2, 9)	(95, 96, 1)
9	789	(1, 8, 2, 4, 4)	(59, 96, 1)
10	611	(1, 7, 7, 5, 1)	(73, 11, 1)
11	1236	(1, 4, 4, 7, 7)	(37, 110, 1)

Plane intersection type is $4^{18} 3^{148}$

Plane invariant is too big (18 planes)

$$\begin{array}{c|c} \rightarrow & 18_1 \\ \hline 12_0 & 6 \end{array} \quad \begin{array}{c|c} \downarrow & 18_1 \\ \hline 12_0 & 4 \end{array}$$

$$C_0 = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11\}_{12}$$

$$C_1 = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17\}_{18}$$

$$\begin{array}{c|c} \rightarrow & 18_1 \\ \hline 12_0 & 6 \end{array}$$

$$\begin{array}{c|c} \downarrow & 18_1 \\ \hline 12_0 & 4 \end{array}$$

$$C_0 = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11\}_{12}$$

$$C_1 = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17\}_{18}$$

The classification of BLT-sets is a difficult problem. One approach is by means of the poset of partial BLT-sets. The orbits at level $q + 1$ in the poset of partial BLT-sets are the isomorphism classes of BLT-sets. We illustrate this method by a small example, comparing different construction algorithms.

We begin with a classification of the complete poset of partial BLT sets. This method is very limited, as the number of orbits of partial BLT-sets gets out of hand quickly as q increases.

Code Snippet 783

```

1   BLT_13_deep_search:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 13 -end \
4   ▷ ▷ -define O -orthogonal_space 0 5 F -end \
5   ▷ ▷ -define C -BLT_set_classifier \
6   ▷ ▷ ▷ -orthogonal_space 0 -starter_size 14 \
7   ▷ ▷ -end \
8   ▷ ▷ -with C -do -BLT_set_classify_activity \
9   ▷ ▷ ▷ -compute_starter \
10  ▷ ▷ ▷ ▷ -problem_label BLT_q13 \
11  ▷ ▷ ▷ ▷ -W -depth 14 \
12  ▷ ▷ ▷ -end \
13  ▷ ▷ -end
14  ▷ #$(PDFLATEX) BLT_q13_poset.tex
15  ▷ #$(OPEN) BLT_q13_poset.pdf
16

```

The technique of classification via substructure can help. Various authors have described different versions of this technique, see [2, 12, 48]. As an example, we classify the BLT-sets of order 13 with the technique of substructures and rainbow cliques.

We consider the classification problem of BLT-sets of order 13. We use the technique of substructures of size 5. So, to begin with, we classify all partial BLT-sets of size 5. For each isomorphism type of partial BLT-set of size 5, we create a colored graph. The vertices in the graph are the points in $Q(4, 13)$ which can be added to the partial BLT-set. Two vertices are connected if they can both be added. Here is the command:

Code Snippet 784

```

1   BLT_13_classify_starter:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 13 -end \
4   ▷ ▷ -define O -orthogonal_space 0 5 F -end \
5   ▷ ▷ -define C -BLT_set_classifier \
6   ▷ ▷ ▷ -orthogonal_space 0 -starter_size 5 \
7   ▷ ▷ -end \
8   ▷ ▷ -with C -do -BLT_set_classify_activity \
9   ▷ ▷ ▷ -compute_starter \
10  ▷ ▷ ▷ ▷ -problem_label BLT_q13 \

```

```

11  ▷ ▷ ▷ ▷ -W -depth 5 \
12  ▷ ▷ ▷ -end \
13  ▷ ▷ -end \
14  ▷ ▷ -with C -do -BLT_set_classify_activity \
15  ▷ ▷ ▷ -create_graphs \
16  ▷ ▷ -end
17

```

In the next step, we compute all rainbow cliques in each of the graphs:

Code Snippet 785

```

1    BLT_13_cliques:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -loop L 0 38 1 \
4    ▷ ▷ ▷ -define G -graph \
5    ▷ ▷ ▷ ▷ -load BLT_q13_graph_5_%L.bin \
6    ▷ ▷ ▷ -end \
7    ▷ ▷ ▷ -with G -do \
8    ▷ ▷ ▷ -graph_theoretic_activity \
9    ▷ ▷ ▷ ▷ -find_cliques -rainbow -target_size 9 \
10   ▷ ▷ ▷ -end \
11   ▷ ▷ -end_loop L
12

```

Next, we create a data structure for isomorphism testing. The first step is to create a database of all partial BLT-sets of order at most 5:

Code Snippet 786

```

1    BLT_13_isomorph_read_DB:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 13 -end \
4    ▷ ▷ -define O -orthogonal_space 0 5 F -end \
5    ▷ ▷ -define C -BLT_set_classifier \
6    ▷ ▷ ▷ -orthogonal_space O -starter_size 5 \
7    ▷ ▷ -end \
8    ▷ ▷ -with C -do -BLT_set_classify_activity \
9    ▷ ▷ ▷ -compute_starter \
10   ▷ ▷ ▷ ▷ -problem_label BLT_q13 \
11   ▷ ▷ ▷ ▷ -W -depth 5 \
12   ▷ ▷ ▷ -end \
13   ▷ ▷ -end \
14   ▷ ▷ -with C -do -BLT_set_classify_activity \
15   ▷ ▷ ▷ -isomorph \
16   ▷ ▷ ▷ ▷ -prefix_iso "./BLT_q13" \
17   ▷ ▷ ▷ ▷ -use_database_for_starter \
18   ▷ ▷ ▷ ▷ -build_db \
19   ▷ ▷ ▷ ▷ -solution_prefix "" \
20   ▷ ▷ ▷ ▷ -base_fname "" \
21   ▷ ▷ ▷ -end \
22   ▷ ▷ -end

```

23

The next step is to read the rainbow cliques from the clique finding process:

Code Snippet 787

```

1    BLT_13_isomorph_read_solutions:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 13 -end \
4    ▷ ▷ -define O -orthogonal_space 0 5 F -end \
5    ▷ ▷ -define C -BLT_set_classifier \
6    ▷ ▷ ▷ -orthogonal_space 0 -starter_size 5 \
7    ▷ ▷ -end \
8    ▷ ▷ -with C -do -BLT_set_classify_activity \
9    ▷ ▷ ▷ -compute_starter \
10   ▷ ▷ ▷ ▷ -problem_label BLT_q13 \
11   ▷ ▷ ▷ ▷ -W -depth 5 \
12   ▷ ▷ ▷ -end \
13   ▷ ▷ -end \
14   ▷ ▷ -with C -do -BLT_set_classify_activity \
15   ▷ ▷ ▷ -isomorph \
16   ▷ ▷ ▷ ▷ -prefix_iso "./BLT_q13" \
17   ▷ ▷ ▷ ▷ -use_database_for_starter \
18   ▷ ▷ ▷ ▷ -read_solutions \
19   ▷ ▷ ▷ ▷ -list_of_cases BLT_q13_list_of_cases_5_0_1.csv \
20   ▷ ▷ ▷ ▷ -solution_prefix "" \
21   ▷ ▷ ▷ ▷ -base_fname "BLT_q13_graph" \
22   ▷ ▷ ▷ -end \
23   ▷ ▷ -end
24
```

Once the cliques have been read from file, we compute the flag orbits in the relation between partial BLT-sets and full BLT-sets. For each orbit representative of a partial BLT-set, we compute the orbits of the stabilizer on the BLT-sets incident with it. The following command can be used.

Code Snippet 788

```

1    BLT_13_isomorph_stabilizer_orbits:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 13 -end \
4    ▷ ▷ -define O -orthogonal_space 0 5 F -end \
5    ▷ ▷ -define C -BLT_set_classifier \
6    ▷ ▷ ▷ -orthogonal_space 0 -starter_size 5 \
7    ▷ ▷ -end \
8    ▷ ▷ -with C -do -BLT_set_classify_activity \
9    ▷ ▷ ▷ -compute_starter \
10   ▷ ▷ ▷ ▷ -problem_label BLT_q13 \
11   ▷ ▷ ▷ ▷ -W -depth 5 \
12   ▷ ▷ ▷ -end \
13   ▷ ▷ -end \
14   ▷ ▷ -with C -do -BLT_set_classify_activity \
15   ▷ ▷ ▷ -isomorph \

```

```

16  ▷ ▷ ▷ ▷ -prefix_iso "./BLT_q13" \
17  ▷ ▷ ▷ ▷ -use_database_for_starter \
18  ▷ ▷ ▷ ▷ -compute_orbits \
19  ▷ ▷ ▷ ▷ -list_of_cases BLT_q13_list_of_cases_5_0_1.csv \
20  ▷ ▷ ▷ ▷ -solution_prefix "" \
21  ▷ ▷ ▷ ▷ -base_fname "BLT_q13_graph" \
22  ▷ ▷ ▷ -end \
23  ▷ ▷ -end
24

```

Finally, we perform isomorphism testing:

Code Snippet 789

```

1    BLT_13_isomorph_testing:
2    ▷ $(ORBITER) -v 4 \
3    ▷ ▷ -define F -finite_field -q 13 -end \
4    ▷ ▷ -define O -orthogonal_space 0 5 F -end \
5    ▷ ▷ -define C -BLT_set_classifier \
6    ▷ ▷ ▷ -orthogonal_space 0 -starter_size 5 \
7    ▷ ▷ -end \
8    ▷ ▷ -with C -do -BLT_set_classify_activity \
9    ▷ ▷ ▷ -compute_starter \
10   ▷ ▷ ▷ ▷ -problem_label BLT_q13 \
11   ▷ ▷ ▷ ▷ -W -depth 5 \
12   ▷ ▷ ▷ -end \
13   ▷ ▷ -end \
14   ▷ ▷ -with C -do -BLT_set_classify_activity \
15   ▷ ▷ ▷ -isomorph \
16   ▷ ▷ ▷ ▷ -prefix_iso "./BLT_q13" \
17   ▷ ▷ ▷ ▷ -use_database_for_starter \
18   ▷ ▷ ▷ ▷ -isomorph_testing \
19   ▷ ▷ ▷ ▷ -solution_prefix "" \
20   ▷ ▷ ▷ ▷ -base_fname "BLT_q13_graph" \
21   ▷ ▷ ▷ -end \
22   ▷ ▷ -end \
23   ▷ ▷ -with C -do -BLT_set_classify_activity \
24   ▷ ▷ ▷ -isomorph \
25   ▷ ▷ ▷ ▷ -prefix_iso "./BLT_q13" \
26   ▷ ▷ ▷ ▷ -use_database_for_starter \
27   ▷ ▷ ▷ ▷ -isomorph_report \
28   ▷ ▷ ▷ ▷ -solution_prefix "" \
29   ▷ ▷ ▷ ▷ -base_fname "BLT_q13_graph" \
30   ▷ ▷ ▷ -end \
31   ▷ ▷ -end
32   ▷ $(PDFLATEX) BLT_q13_isomorphism_types.tex
33   ▷ $(OPEN) BLT_q13_isomorphism_types.pdf
34

```

This last command results in exactly three isomorphism types of BLT-sets of order 13. Of course, these results match with the results from the classification of the complete poset of partial BLT-sets before. The difference is that the second approach is more efficient, as the number of orbits to be considered by the algorithm is much smaller. In essence, we are skipping over the worst part of the poset. Many partial

BLT-sets do not complete to BLT-sets, so these braches of the search tree will die out in the clique finding algorithm.

The command

Code Snippet 790

```

1   blt_set_export_gap.all:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define Q -vector -dense \
4   ▷ ▷ ▷ $(BLT_ORDER.Q) \
5   ▷ ▷ -end \
6   ▷ ▷ -define NB -vector -dense \
7   ▷ ▷ ▷ $(BLT_NUMBER.ISO) \
8   ▷ ▷ -end \
9   ▷ ▷ -loop_over i Q \
10  ▷ ▷ ▷ -define F%i -finite_field -q "%i[Q]" -end \
11  ▷ ▷ ▷ -define O%i -orthogonal_space 0 5 F%i -create_extension_fields -end \
12  ▷ ▷ ▷ -loop j 0 %i[NB] 1 \
13  ▷ ▷ ▷ ▷ -define BLT_%i_%j -BLT_set \
14  ▷ ▷ ▷ ▷ ▷ -space O%i -catalogue %j \
15  ▷ ▷ ▷ ▷ -end \
16  ▷ ▷ ▷ ▷ -with BLT_%i_%j -do -blt_set_activity \
17  ▷ ▷ ▷ ▷ ▷ -export_gap \
18  ▷ ▷ ▷ ▷ -end \
19  ▷ ▷ ▷ -end_loop j \
20  ▷ ▷ -end_loop_over i \
21  ▷ ▷ -print_symbols
22

```

exports all BLT-sets in the Orbiter catalogue to GAP.

Chapter 16

Computer Science Primitives

16.1 Clique Finding

A clique in a graph $\Gamma = (V, E)$ is a subset S of the vertices such that any two elements of S are adjacent in Γ . Orbiter supports finding and classifying cliques in graphs. The command `-find_cliques` from Table 13.4-13.5 can be used to find all cliques in a graph. Additional options for this command are shown in Table 16.1.

For instance, the cliques of size 3 in the graph `graph_v5_e7.colored_graph` from Section 13.1 can all be found using the following command:

Code Snippet 791

```
1  small_graph_cliques:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define G -graph -edges_as_pairs 5 \
4  ▷ ▷ ▷ "0,1,0,2,0,3,0,4,1,3,1,4,2,4" \
5  ▷ ▷ -end \
6  ▷ ▷ -with G -do \
7  ▷ ▷ -graph_theoretic_activity -save -end
8  ▷ $(ORBITER) -v 2 \
9  ▷ ▷ -define G -graph -load graph_v5_e7.colored_graph -end \
```

Finding Cliques

Command	Arguments	Purpose
<code>-rainbow</code>		Find all rainbow cliques. The size of the cliques is the number of vertex colors.
<code>-target_size</code>	s	Find all cliques of size s .
<code>-weighted</code>	s	Find weighted cliques.
<code>-Sajeeb</code>		Use the implementation by Sajeeb Chowdhury.
<code>-output_file</code>	$fname$	Write cliques to the named file.
<code>-restrictions</code>	$l\ r\ m$	Restricted search: At level l , restrict to all branches congruent to r modulo m . Here, $0 \leq r < m$.

Table 16.1: Finding Cliques

```

10  ▷ ▷ -with G -do \
11  ▷ ▷ -graph_theoretic_activity \
12  ▷ ▷ ▷ -find_cliques -target_size 3 \
13  ▷ ▷ -end
14

```

It is also possible to classify all cliques under the automorphism group of the graph. This is a multi-step process, though. At first, the automorphism group of the graph has to be computed. Then, poset classification can be invoked to classify the cliques of a certain size. Here is an example. We wish to classify the cliques in the Paley graph of order 13. The first command creates the graph and computes the automorphism group:

Code Snippet 792

```

1  Paley_13_aut:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 13 -end \
4  ▷ ▷ -define Gamma -graph -Paley F -end \
5  ▷ ▷ -with Gamma -do \
6  ▷ ▷ -graph_theoretic_activity \
7  ▷ ▷ ▷ -automorphism_group \
8  ▷ ▷ -end
9

```

The command writes a file `Paley_13_group.makefile`. The group is given using a base and strong generating set. The base consists of the two points 0, 1. Three strong generators with respect to this base are given in a csv file. Using this group, the next command classifies all cliques of size at most 5 in the Paley graph of order 13 under the action of the automorphism group. It turns out that there are no 5-cliques, and that the largest cliques have size 3. The command shows that there is a unique orbit of 3-cliques:

Code Snippet 793

```

1  Paley_13_cliques_classify:
2  ▷ $(ORBITER) -v 4 \
3  ▷ ▷ -define Control -poset_classification_control \
4  ▷ ▷ ▷ -W \
5  ▷ ▷ ▷ -problem_label Paley13_cliques \
6  ▷ ▷ ▷ -clique_test Gamma \
7  ▷ ▷ ▷ -depth 5 \
8  ▷ ▷ -end \
9  ▷ ▷ -define F -finite_field -q 13 -end \
10 ▷ ▷ -define gens -vector -file Paley_13_gens.csv -end \
11 ▷ ▷ -define G -permutation_group \
12 ▷ ▷ ▷ -bsgs Paley_13 "Paley\13" 13 78 "0,1" 3 gens -end \
13 ▷ ▷ -define Gamma -graph -Paley F -end \
14 ▷ ▷ -define Orb -orbits \
15 ▷ ▷ ▷ -group G \
16 ▷ ▷ ▷ -group_action G \
17 ▷ ▷ ▷ -generators G \
18 ▷ ▷ ▷ -on_subsets 5 Control \
19 ▷ ▷ -end

```

20

For comparison, the command

Code Snippet 794

```

1   Paley_13_cliques_all:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 13 -end \
4   ▷ ▷ -define Gamma -graph -Paley F -end \
5   ▷ ▷ -with Gamma -do \
6   ▷ ▷ -graph_theoretic_activity \
7   ▷ ▷ ▷ -find_cliques -target_size 3 \
8   ▷ ▷ -end
9
```

finds all cliques of size 3. There are exactly 26 of them. Because of the previous command, we know that they are all equivalent under the automorphism group of the graph.

Let us consider the orbital graph created in Section 13.1. We wish to study the 5-cliques. We first determine the number of 5-cliques, and then the number of orbits of 5-cliques under the automorphism group. The following command computes all 5-cliques:

Code Snippet 795

```

1   HJ64_cliques5:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define Gamma -graph \
4   ▷ ▷ ▷ -load \
5   ▷ ▷ ▷ Group_Perm315_Orbital_3.colored_graph \
6   ▷ ▷ -end \
7   ▷ ▷ -with Gamma -do \
8   ▷ ▷ -graph_theoretic_activity \
9   ▷ ▷ ▷ -find_cliques -target_size 5 -end \
10  ▷ ▷ -end
11
```

It turns out that there are exactly 1008 5-cliques. We can use the following command to classify these cliques under the action of the automorphism group of the graph:

Code Snippet 796

```

1   HJ64_cliques5_classify:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define Control -poset_classification_control \
4   ▷ ▷ ▷ -W \
5   ▷ ▷ ▷ -problem_label HJ64_cliques \
6   ▷ ▷ ▷ -clique_test Gamma \
7   ▷ ▷ ▷ -depth 5 \
8   ▷ ▷ -end \
9   ▷ ▷ -define Gamma -graph \
10  ▷ ▷ ▷ -load \
```

```

11  ▷ ▷ ▷ Group_Perm315_Orbital_3.colored_graph \
12  ▷ ▷ -end \
13  ▷ ▷ -define gens -vector \
14  ▷ ▷ ▷ -file halljanko315_gens.csv \
15  ▷ ▷ -end \
16  ▷ ▷ -define G -permutation_group \
17  ▷ ▷ -bsgs halljanko315 "File\halljanko315" \
18  ▷ ▷ ▷ 315 1209600 "0,1,42,95" 5 gens -end \
19  ▷ ▷ -define Orb -orbits -group G \
20  ▷ ▷ ▷ -on_subsets 5 Control \
21  ▷ ▷ -end
22

```

It shows that all of the 1008 5-cliques lie in one orbit under the group. The orbit representative picked by Orbiter is {0,8,31,110,283}. These numbers refer to the vertices of the graph. They are zero-based. The stabilizer of the clique has order 1200.

Let us look at the collinearity graph of $Q(4, 2)$ one more time. The following command computes the cliques of size 3:

Code Snippet 797

```

1    PGO_5_2_cliques:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 2 -end \
4    ▷ ▷ -define O -orthogonal_space 0 5 F -without_group -end \
5    ▷ ▷ -with O -do -orthogonal_space_activity \
6    ▷ ▷ ▷ -export_point_line_incidence_matrix \
7    ▷ ▷ -end
8    ▷ $(ORBITER) -v 3 \
9    ▷ ▷ -define Inc -vector -file 0_5_2_incidence_matrix.csv -end \
10   ▷ ▷ -define Gamma -graph -collinearity_graph Inc -end \
11   ▷ ▷ -with Gamma -do \
12   ▷ ▷ -graph_theoretic_activity \
13   ▷ ▷ ▷ -find_cliques -target_size 3 \
14   ▷ ▷ -end
15

```

There are 15 cliques of size 3. They correspond to the lines in the geometry.

And here is the command to find all 5-cliques in the graph of disjoint lines in the Doily:

Code Snippet 798

```

1    Doily_disjoint_sets_graph_cliques_5:
2    ▷ echo $(DOILY) >doily.csv
3    ▷ $(ORBITER) -v 2 \
4    ▷ ▷ -define G -graph -disjoint_sets_graph \
5    ▷ ▷ ▷ doily.csv \
6    ▷ ▷ -end \
7    ▷ ▷ -with G -do \
8    ▷ ▷ ▷ -graph_theoretic_activity \
9    ▷ ▷ ▷ -find_cliques \

```

```
10  ▷ ▷ ▷ ▷ -target_size 5 \  
11  ▷ ▷ ▷ ▷ -output_file doily_cliques.5 \  
12  ▷ ▷ ▷ -end \  
13  ▷ ▷ -end \  
14  ▷ ▷ -print_symbols  
15
```

There are exactly 6 cliques.

16.2 Rainbow Cliques

Consider a graph $\Gamma = (V, E)$. A vertex coloring of Γ is a map $f : V \rightarrow C$ where C is a finite set whose elements we call colors. Let $k = |C|$. A rainbow clique in Γ is a clique X such that $|X \cap f^{-1}(c)| = 1$ for all $c \in C$. This means that the clique contains exactly one vertex from every color. Rainbow cliques can be helpful in the search for combinatorial objects. This is because the conditions for rainbow cliques are stronger than the conditions for ordinary cliques, which leads to better pruning in a backtrack search algorithm for cliques. Orbiter offers two algorithms for rainbow cliques. Let us consider an example. The classification of BLT-sets can be facilitated using rainbow cliques. The following algorithm was introduced in [12], building on earlier work of Penttila. At first, the partial BLT-sets of a certain size (usually 5) are computed. The orbit representatives of this classification are called starter sets. Then, for each orbit representative, a certain graph is defined. The graph helps us to find the BLT-sets that contain the given starter set. The graph has a vertex coloring, and the extensions correspond to rainbow cliques. Let us look at an example. We classify the BLT-sets of size 11. The first command classifies the starters of size 5 and creates the graphs. The graphs are $\Gamma_0, \dots, \Gamma_{s-1}$, where s is the number of orbits on partial BLT-sets.

Code Snippet 799

```

1   BLT_11_classify_starter:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 11 -end \
4   ▷ ▷ -define O -orthogonal_space 0 5 F -end \
5   ▷ ▷ -define C -BLT_set_classifier -orthogonal_space 0 -starter_size 5 -end \
6   ▷ ▷ -with C -do -BLT_set_classify_activity \
7   ▷ ▷ ▷ -compute_starter \
8   ▷ ▷ ▷ ▷ -problem_label BLT_q11 \
9   ▷ ▷ ▷ ▷ -W -depth 5 \
10  ▷ ▷ ▷ -end \
11  ▷ ▷ -end \
12  ▷ ▷ -with C -do -BLT_set_classify_activity \
13  ▷ ▷ ▷ -create_graphs \
14  ▷ ▷ -end
15

```

Consider just one graph, say the first. The following command computes the rainbow cliques in Γ_0 :

Code Snippet 800

```

1   BLT_11_clique_0:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define G -graph \
4   ▷ ▷ ▷ -load BLT_q11_graph_5.0.bin \
5   ▷ ▷ -end \
6   ▷ ▷ -with G -do \
7   ▷ ▷ -graph_theoretic_activity \
8   ▷ ▷ ▷ -find_cliques -rainbow -target_size 12 \
9   ▷ ▷ -end
10

```

There are three rainbow cliques. The following command computes the rainbow cliques in Γ_0 using the second algorithm. The second algorithm, due to Sajeeb Chowdhury, allows for parallel computing using multithreading.

Code Snippet 801

```

1   BLT_11_clique_0_Sajeeb:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define G -graph \
4   ▷ ▷ ▷ -load BLT_q11_graph_5.0.bin \
5   ▷ ▷ -end \
6   ▷ ▷ -with G -do \
7   ▷ ▷ -graph_theoretic_activity \
8   ▷ ▷ ▷ -find_cliques -rainbow -Sajeeb \
9   ▷ ▷ -end
10

```

Let us look at the graph Γ_0 . At first, we sort the vertices by color class:

Code Snippet 802

```

1   BLT_11_0_sort:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define G -graph \
4   ▷ ▷ ▷ -load BLT_q11_graph_5.0.bin \
5   ▷ ▷ -end \
6   ▷ ▷ -with G -do \
7   ▷ ▷ -graph_theoretic_activity \
8   ▷ ▷ ▷ -sort_by_colors \
9   ▷ ▷ -end
10

```

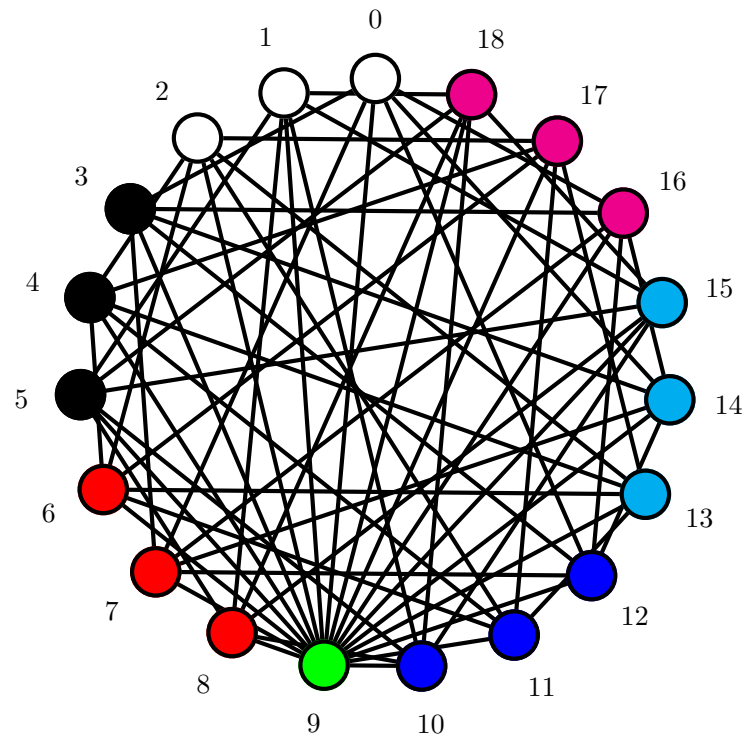
Next, we produce a drawing of the graph. The subscripts at the vertex label indicate the vertex color:

Code Snippet 803

```

1   BLT_11_0_sorted_draw:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 100 \
5   ▷ ▷ ▷ -scale 0.4 \
6   ▷ ▷ ▷ -line_width 1.0 -embedded \
7   ▷ ▷ -end \
8   ▷ ▷ -define G -graph \
9   ▷ ▷ ▷ -load BLT_q11_graph_5.0_sorted.bin \
10  ▷ ▷ -end \
11  ▷ ▷ -with G -do \
12  ▷ ▷ -graph_theoretic_activity \
13  ▷ ▷ ▷ -draw Do \
14  ▷ ▷ -end
15  ▷ $(PDFLATEX) BLT_q11_graph_5.0_sorted_draw.tex
16  ▷ $(OPEN) BLT_q11_graph_5.0_sorted_draw.pdf
17

```



16.3 Diophantine Systems

Diophantine systems of equations and inequalities arise frequently in Combinatorics. In Table 16.2, Orbiter commands for creating and solving diophantine systems are shown. In Table 16.3, Orbiter activities for diophantine systems are shown.

Consider the matrix

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 \end{bmatrix}$$

Suppose we want to find all column vectors $\mathbf{x}$ with entries in 0,1 such that

$$A\mathbf{x} = \mathbf{1}$$

where $\mathbf{1}$ is the all-one column vector. Orbiter offers two algorithms to do this. One is McKay's possolve, the other is Knuth's dancing links (DLX). In order to get started, we need to create a diophant object. In the following example, we use the makefile variable TEST_SYSTEM for the coefficient matrix.

Code Snippet 804

```
1 TEST_SYSTEM="\
2 0,1,0,1,0,0, \
3 0,0,1,0,1,0, \
4 1,0,1,0,0,0, \
5 0,1,0,1,0,1, \
6 1,0,0,0,0,1, \
7 1,0,1,0,0,0, \
8 0,1,0,0,1,1"
9
```

The right hand side will be specified using the RHS command below:

Code Snippet 805

```
1 solve_test_system:
2 ▷ $(ORBITER) -v 4 \
3 ▷ ▷ -define A -vector -format 7 -dense $(TEST_SYSTEM) -end \
4 ▷ ▷ -define D -diophant \
5 ▷ ▷ ▷ -label test_system \
6 ▷ ▷ ▷ -coefficient_matrix A \
7 ▷ ▷ ▷ -RHS "mult=7,EQ=1" \
8 ▷ ▷ ▷ -x_min_global 0 -x_max_global 1 \
9 ▷ ▷ -end
10
```

There are two commands to solve a diophantine system: `-solve_mckay` and `-solve_DLX`. The latter is more restrictive, as it allows only 0,1 variables. Here is the McKay solver:

Creating Diophantine Systems		
Command	Arguments	Purpose
-label	label	Use the given name as file name.
-coefficient_matrix	A	Set the coefficient matrix to the previously created vector with format information.
-coefficient_matrix_csv	fname	Read the coefficient matrix from the given csv-file.
-RHS	options	Set RHS info for a set of m rows. Possible types of conditions are EQ, LE, INT, ZOR. This command can be repeated. Example: mult=2,EQ=1 is condition for 2 rows where the RHS is equal to zero. Example: mult=1,LE=3 is one condition where the RHS is at most three. Example: mult=1,INT=1to2 is one condition where the RHS is between 1 and 2. Example: mult=1,ZOR=2 is one condition where the RHS is either 0 or 2.
-RHS_csv	fname	Read the RHS information from the given csv file.
-RHS_constant	options	Set the RHS as in -RHS (without the mult component).
-x_max_global	a	Set the upper bound for all variables to a
-x_min_global	a	Set the lower bound for all variables to a
-x_bounds	list-of-values	Set the lower and upper bounds for all variables.
-x_bounds_csv	fname	Read the lower and upper bounds for all variables from the given file.
-has_sum	s	Force the sum of the variables to be s .
-problem_of_Steiner_type	N fname	Create an instance of an exact cover problem based on the matrix in the given file. The underlying set has size N .
-maximal_arc	s d secants subset	Create system for a maximal arc of size s and degree d in $PG(2, q)$. Use the given set of two pencil lines. The subset picks the lines from the given pencils which are external.
-field	F	Use the field F. Related to -maximal_arc.

Table 16.2: Creating Diophantine Systems

Diophantine System Activities		
Command	Arguments	Purpose
-print		Print the system.
-solve_mckay		Solve the system using McKay's possolve.
-solve_DLX		Solve the system using Knuth's dancing links.
-solve_standard		Solve the system using the standard solver.
-draw	options	Produce a drawing of the coefficient matrix of the system. Here, options is the name of a draw options object.
-draw_as_bitmap	$w\ b$	Produce a bitmap drawing of the coefficient matrix of the system, using boxes of w pixels with. Set the color bit-depth to b ($b = 8$ or $b = 24$). The output is a bmp-file.
-perform_column_reductions		Eliminate variables which must be zero and write a reduced system.
-test_single_equation		For each row of the system, compute the number of solutions of the system restricted to the nonzero coefficients.
-project_to_single_equation_and_solve	$i\ j$	Solve the system assuming the j th solution to the restricted system consisting of the i th row.
-project_to_two_equations_and_solve	$i\ j\ r\ m$	Solve the system assuming any solution to the restricted system consisting of the i th and the j -th row whose number is congruent to r mod m .

Table 16.3: Diophantine System Activities

Code Snippet 806

```
1   McKay_test:
2   ▷ $(ORBITER) -v 4 \
3   ▷ ▷ -define A -vector -format 7 -dense $(TEST_SYSTEM) -end \
4   ▷ ▷ -define D -diophant \
5   ▷ ▷ ▷ -label test_system \
6   ▷ ▷ ▷ -coefficient_matrix A \
7   ▷ ▷ ▷ -RHS "mult=7,EQ=1" \
8   ▷ ▷ ▷ -x_min_global 0 -x_max_global 1 \
9   ▷ ▷ -end \
10  ▷ ▷ -with D -do \
11  ▷ ▷ ▷ -diophant_activity -solve_mckay \
12  ▷ ▷ -end
13
```

The solutions are written to the file `DLX_test.sol`. Next, we invoke the dancing links solver:

Code Snippet 807

```
1   DLX_test:
2   ▷ $(ORBITER) -v 4 \
3   ▷ ▷ -define A -vector -format 7 -dense $(TEST_SYSTEM) -end \
4   ▷ ▷ -define D \
5   ▷ ▷ -diophant -label test_system \
6   ▷ ▷ ▷ -coefficient_matrix A \
7   ▷ ▷ ▷ -RHS "mult=7,EQ=1" \
8   ▷ ▷ ▷ -x_min_global 0 -x_max_global 1 \
9   ▷ ▷ -end \
10  ▷ ▷ -with D -do \
11  ▷ ▷ ▷ -diophant_activity -solve_DLX \
12  ▷ ▷ -end
13
```

Chapter 17

Canonical Forms, Classification and Automorphism Groups

17.1 Overview of Canonical Forms

The problem of classification of combinatorial objects is related to that of computing canonical forms and computing automorphism groups. First, a combinatorial object is any kind of object that can be described as a sets of sets (always finite). This includes objects in finite projective geometries, as the geometry can be represented as sets itself (for instance the set of lines). Orbiter uses the graph theory package Nauty [57] to perform the underlying computations. The combinatorial object is turned into an incidence structure, which in turn is converted into a bipartite graph. Then, the canonical form and automorphism group of the graph is computed with Nauty. From this information, the classification problem can be solved, and the automorphism group of the object can be converted in the type of group that we expect. For instance, an object in projective space will have an automorphism group which is a subgroup of the collineation group, which means that in effect we produce matrices which generate the group. For background on the reduction of the isomorphism problem of combinatorial objects to graphs, see [3].

Orbiter can go beyond combinatorial objects. A variety is not a true combinatorial object, as the action of the group is by means of linear substitution, and the object is more than just the set of rational points. For varieties, Orbiter offers the algorithm of [4] to perform classification and for computing the automorphism group.

Table 17.1 lists Orbiter commands for classification of combinatorial/geometric objects.

Table 17.2 lists Orbiter commands for classification of combinatorial objects.

Table 17.3 lists Orbiter commands for the classification using Nauty.

Classification by Canonical Form		
Command	Arguments	Purpose
-space	L	A projective space is required for some types of combinatorial objects.
-ring	R	A polynomial ring is required for algebraic varieties.
-input_fname_mask	fname-mask	Input file name mask.
-nb_files	n	Number of input files.
-output_fname	fname	Output file name.
-label_po_go	label	Column label for po-go columns. This is the stabilizer of the orbit representative of the primary orbit.
-label_po_index	label	Column label for the subgroup index of the secondary orbit representative inside the stabilizer of the representative of the primary orbit.
-label_po	label	Column label for the primary orbit number.
-label_so	label	Column label for the secondary orbit number.
-label_equation_algebraic	label	Column label for the algebraic equation (in case of a variety).
-label_equation_by_coefficients	label	Column label for the equation (in vector of coefficients form).
-label_equation2_algebraic	label	Column label for the optional second algebraic equation (in case of a variety).
-label_equation2_by_coefficients	label	Column label for the optional second equation (in vector of coefficients form).
-label_points	label	Column label for the set of points of the object (optional).
-label_lines	label	Column label for the set of lines of the object (optional).
-nauty_control	options	Nauty options, see Table 17.3.
-has_nauty_output		Whether data from a previous run of Nauty is available.
-carry_through	label	Column label of a property that is carried through from the input file to the output file. This option can be repeated.
-skip	list	List of cases to be skipped. Case numbers are 0-based.

Table 17.1: Classification by Canonical Form

Classification of Combinatorial Objects		
Command	Arguments	Purpose
-save_classification	prefix	Save classification when finished.
-max_TDO_depth	d	Limit TDO depth to d in the report.
-save_canonical_labeling		Save the canonical labeling.
-save_ago		Save the automorphism group orders to file.
-save_transversal		Save the indices of the elements chosen for the transversal.
-filter_ago_at_least	ago0	Filter out the objects whose automorphisssm group order is at least ago0. At present, this option is only available for combinatorial objects of type incidence geometry.
-nauty_control	options	Set options that control Nauty. See Table 17.3.

Table 17.2: Classification of Combinatorial Objects

Nauty Control Options		
Command	Arguments	Purpose
-save_nauty_input_graphs	prefix	Save nauty input graphs during classification. The graphs will be written in Dimacs format. The file name will be constructed from the given prefix and the name of the combinatorial object.
-save_orbit_of_equations	prefix	Save the orbit of equations (only for objects of type variety). The file name will be constructed from the given prefix and the name of the combinatorial object.
-partition	part	Compute the canonical form with respect to the stabilizer of the partition.
-show_canonical_form		Show the canonical form during classification.
-reduce_memory_footprint	k	Reduce the memory footprint (for $k = 1$).

Table 17.3: Nauty Control Options

17.2 The Classification-By-Canonical-Form Algorithm

Suppose we want to check whether the 2 sets of points

$$\{0, 1, 2, 3\}, \{1, 3, 4, 6\}$$

are equivalent in $\text{PG}(2, 2)$. The following command creates a combinatorial object for each of the two sets and invokes the nauty-based classification-by-canonical-form algorithm.

Code Snippet 808

```

1   quadrangles_PG_2_2:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define C -combinatorial_object \
4   ▷ ▷ ▷ -label quadrangle quadrangle \
5   ▷ ▷ ▷ -set_of_points "0,1,2,3" \
6   ▷ ▷ ▷ -set_of_points "1,3,4,6" \
7   ▷ ▷ -end \
8   ▷ ▷ -define F -finite_field -q 2 -end \
9   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
10  ▷ ▷ -with C -do \
11  ▷ ▷ -combinatorial_object_activity \
12  ▷ ▷ ▷ -canonical_form_PG P \
13  ▷ ▷ ▷ -nauty_control -nauty_log "quadrangles_PG_2_2_nauty_log.txt" -end \
14  ▷ ▷ ▷ -end \
15  ▷ ▷ -end \
16  ▷ ▷ -define Doinc -draw_incidence_structure_options \
17  ▷ ▷ ▷ -width 40 -width_10 6 \
18  ▷ ▷ -end \
19  ▷ ▷ -with C -do \
20  ▷ ▷ -combinatorial_object_activity \
21  ▷ ▷ ▷ -post_processing \
22  ▷ ▷ -end \
23  ▷ ▷ -with C -do \
24  ▷ ▷ -combinatorial_object_activity \
25  ▷ ▷ ▷ -report \
26  ▷ ▷ ▷ ▷ -export_flag_orbits \
27  ▷ ▷ ▷ ▷ -show_TDO \
28  ▷ ▷ ▷ ▷ -show_TDA \
29  ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
30  ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
31  ▷ ▷ ▷ -end \
32  ▷ ▷ -end
33  ▷ $(PDFLATEX) quadrangle_classification.tex
34  ▷ $(OPEN) quadrangle_classification.pdf
35

```

It turns out that the two sets are equivalent. Both are quadrangles. The stabilizer is computed to be of order 24. Generators for the group are given.

Let us compute the group of the Barth sextic. In Section 7.6, we computed the set of 65 singular points. Next, we will compute the stabilizer of these points. To begin with, we define a makefile variable with these points in Orbiter codes:

Code Snippet 809

```

1  BARTH_SEXTIC_MOD_11_SINGULAR_POINTS="0, 1, 2, 4, 39, 44, 61, 63, 64, 66, 94, 96,
    97, 99, 116, 121, 135, 144, 145, 171, 174, 226, 229, 244, 255, 275, 365, 374, 424
    , 425, 457, 458, 540, 573, 620, 625, 674, 681, 685, 692, 764, 767, 841, 844, 885,
    888, 962, 965, 983, 988, 1037, 1044, 1048, 1055, 1145, 1178, 1271, 1272, 1304, 1
    305, 1343, 1355, 1364, 1454, 1463"
2

```

The next command performs the classification-by-canonical-form algorithm. At first, we create a combinatorial object from the set of singular points. After creating the finite field $\mathbb{F}_{11}$ and the projective geometry $\text{PG}(3, 11)$, a combinatorial object activity of type `canonical_form_PG` is called to compute the canonical form and the automorphism group of the object. A second combinatorial object activity of type `report` is called to produce a latex report. The report includes the tactical decompositions of the decorated incidence matrix called the TDO and TDA.

Code Snippet 810

```

1  Barth_mod.11_singular_pts:
2  ▷ $(ORBITER) -v 4 \
3  ▷ ▷ -define H -set -here \
4  ▷ ▷ ▷ $(BARTH_SEXTIC_MOD_11_SINGULAR_POINTS) \
5  ▷ ▷ -end \
6  ▷ ▷ -define C -combinatorial_object \
7  ▷ ▷ ▷ -label Barth.q11_sing Barth\q11\_sing \
8  ▷ ▷ ▷ -set_of_points H \
9  ▷ ▷ -end \
10 ▷ ▷ -define F -finite_field -q 11 -end \
11 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
12 ▷ ▷ -with C -do \
13 ▷ ▷ -combinatorial_object_activity \
14 ▷ ▷ ▷ -canonical_form_PG P \
15 ▷ ▷ ▷ ▷ -save_ago \
16 ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
17 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "Barth_mod.11_nauty_log.txt" -end \
18 ▷ ▷ ▷ -end \
19 ▷ ▷ -end \
20 ▷ ▷ -define Doinc -draw_incidence_structure_options \
21 ▷ ▷ ▷ -width 40 -width_10 6 \
22 ▷ ▷ -end \
23 ▷ ▷ -with C -do \
24 ▷ ▷ -combinatorial_object_activity \
25 ▷ ▷ ▷ -post_processing \
26 ▷ ▷ -end \
27 ▷ ▷ -with C -do \
28 ▷ ▷ -combinatorial_object_activity \
29 ▷ ▷ ▷ -report \
30 ▷ ▷ ▷ ▷ -show_TDO \
31 ▷ ▷ ▷ ▷ -show_TDA \
32 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
33 ▷ ▷ ▷ -end \
34 ▷ ▷ -end
35 ▷ $(PDFLATEX) Barth.q11_sing_classification.tex

```

```

36   ▷ $(OPEN) Barth.q11_sing_classification.pdf
37

```

It turns out that the set is stabilized by a group of order 120. Generators mod 11 are given.

The following sequence of commands generates and classifies all quartic curves with 28 bitangents in the plane over the field $\mathbb{F}_{13}$. As first, we need a makefile variable for the number of isomorphism types of cubic surfaces with 27 lines over this field:

Code Snippet 811

```

1   NB_CUBIC_SURFACES_Q13=4
2

```

Next, we create candidates for the quartic curves by projecting from a point on the surface but not on any line. There can be multiple such points. For each orbit under the automorphism group, one point is chosen as a representative of this orbit. Using the procedure described in [38], the associated quartic curve is created. We then concatenate the output files. Finally, we perform a classification of the curves. We employ the nauty-based classification-by-canonical-form algorithm. The following command sequence does the job:

Code Snippet 812

```

1   quartic_curves.q13_classify_nauty:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define F -finite_field -q 13 -end \
4   ▷ ▷ -define P -projective_space -n 3 -field F -end \
5   ▷ ▷ -define P2 -projective_space -n 2 -field F -v 0 -end \
6   ▷ ▷ -define Control -poset_classification_control \
7   ▷ ▷ ▷ -problem_label nc_arcs.q12 \
8   ▷ ▷ ▷ -W -depth 6 \
9   ▷ ▷ -end \
10  ▷ ▷ -define Arc_control -arc_generator_control \
11  ▷ ▷ ▷ -projective_space P2 \
12  ▷ ▷ ▷ -control Control \
13  ▷ ▷ ▷ -target_size 6 \
14  ▷ ▷ ▷ -d 2 \
15  ▷ ▷ -end \
16  ▷ ▷ -define Orb -orbits \
17  ▷ ▷ ▷ -on_arcs Arc_control \
18  ▷ ▷ -end \
19  ▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q13) 1 \
20  ▷ ▷ ▷ -define S_%L -cubic_surface -space P -catalogue %L -end \
21  ▷ ▷ -end_loop L \
22  ▷ ▷ -print_symbols \
23  ▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q13) 1 \
24  ▷ ▷ ▷ -with S_%L -do \
25  ▷ ▷ ▷ -cubic_surface_activity \
26  ▷ ▷ ▷ ▷ -export_all_quartic_curves Orb \
27  ▷ ▷ ▷ -end \
28  ▷ ▷ -end_loop L \
29  ▷ ▷ -print_symbols
30  ▷ $(ORBITER) -v 3 \

```

```

31  ▷ ▷ -csv_file_concatenate_from_mask 0 $(NB_CUBIC_SURFACES_Q13) \
32  ▷ ▷ ▷ surface_catalogue_q13_iso%ld_quartics.csv \
33  ▷ ▷ ▷ quartics.q13.csv
34  ▷ $(ORBITER) -v 5 \
35  ▷ ▷ -define F -finite_field -q 13 -end \
36  ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
37  ▷ ▷ -define R -polynomial_ring \
38  ▷ ▷ ▷ -field F \
39  ▷ ▷ ▷ -number_of_variables 3 \
40  ▷ ▷ ▷ -homogeneous_of_degree 4 \
41  ▷ ▷ ▷ -monomial_ordering_partition \
42  ▷ ▷ ▷ -variables "y0,y1,y2" "y_0,y_1,y_2" \
43  ▷ ▷ -end \
44  ▷ ▷ -define Orb -orbits -classification_by_canonical_form \
45  ▷ ▷ ▷ -space P \
46  ▷ ▷ ▷ -ring R \
47  ▷ ▷ ▷ -input_fname_mask quartics.q13.csv \
48  ▷ ▷ ▷ -nb_files 1 \
49  ▷ ▷ ▷ -output_fname quartic_curves.q13 \
50  ▷ ▷ ▷ -label_po "File" \
51  ▷ ▷ ▷ -label_so "orbit" \
52  ▷ ▷ ▷ -carry_through "PT" \
53  ▷ ▷ ▷ -carry_through "PTCOEFF" \
54  ▷ ▷ ▷ -carry_through "TANGENT_PLANE_RK" \
55  ▷ ▷ ▷ -carry_through "abcdef" \
56  ▷ ▷ ▷ -carry_through "NB.E" \
57  ▷ ▷ ▷ -carry_through "NB.ET" \
58  ▷ ▷ ▷ -carry_through "NB.K" \
59  ▷ ▷ ▷ -carry_through "NB.DOUBLE" \
60  ▷ ▷ ▷ -carry_through "NB.SINGLE" \
61  ▷ ▷ ▷ -carry_through "NB.ZERO" \
62  ▷ ▷ ▷ -carry_through "SEQN" \
63  ▷ ▷ ▷ -carry_through "SEQNALG" \
64  ▷ ▷ ▷ -carry_through "curve" \
65  ▷ ▷ ▷ -carry_through "CURVEALG" \
66  ▷ ▷ ▷ -label_equation_by_coefficients "curve" \
67  ▷ ▷ ▷ -label_points "pts.on.curve" \
68  ▷ ▷ ▷ -label_lines "bitangents" \
69  ▷ ▷ ▷ -carry_through "Kovalevski_pts" \
70  ▷ ▷ ▷ -nauty_control -reduce_memory_footprint 1 -end \
71  ▷ ▷ ▷ -end \
72  ▷ ▷ -end \
73  ▷ ▷ -with Orb -do \
74  ▷ ▷ ▷ -orbits_activity \
75  ▷ ▷ ▷ -export_source_code \
76  ▷ ▷ -end \
77  ▷ ▷ -print_symbols
78  ▷ #$(PDFLATEX) quartic_curves.q13_orbits.tex
79  ▷ #$(OPEN) quartic_curves.q13_orbits.pdf
80

```

Over the given field $q = 13$, we find exactly two isomorphism types. The Orbiter knowledge base of combinatorial objects is built by assembling output files from classifications like this one in the form of

C++ source code. The previous command creates a C++ source file containing these curves for importing into Orbiter. The command also creates a csv file containing the classification of all flag orbits. The output of the command is shown below, showing the canonical equation of each curve, the automorphism group by means of generators and group order, and the number of rational points of the curve over the given field.

Classification

$q = 13$

Number of isomorphism classes: 2

Automorphism group order statistic: 48, 24

The isomorphism classes are:

Isomorphism class 0 / 2:

Number of rational points over $\mathbb{F}_{13}$: 8

Canonical equation:

$$5X_0^3X_1 + 4X_0^3X_2 + 7X_0X_1^3 + 11X_1^3X_2 + 10X_0X_2^3 + 6X_1X_2^3 + 8X_0^2X_1^2 \\ + 7X_0^2X_2^2 + 7X_1^2X_2^2 + 5X_0^2X_1X_2 + 7X_0X_1^2X_2 + X_0X_1X_2^2$$

Stabilizer order: 24

The stabilizer:

Strong generators for a group of order 24:

$$\begin{bmatrix} 10 & 0 & 0 \\ 4 & 2 & 6 \\ 1 & 1 & 1 \end{bmatrix}, \begin{bmatrix} 11 & 2 & 4 \\ 0 & 0 & 4 \\ 0 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 12 & 12 & 12 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

1,0,0,3,8,11,4,4,4,

1,12,11,0,0,11,0,6,0,

1,1,1,0,12,0,12,0,0,

Isomorphism class 1 / 2:

Number of rational points over $\mathbb{F}_{13}$: 4

Canonical equation:

$$5X_2^4 + 5X_0^3X_1 + 7X_0X_1^3 + X_0^2X_1^2$$

Stabilizer order: 48

The stabilizer:

Strong generators for a group of order 48:

$$\begin{bmatrix} 12 & 0 & 0 \\ 0 & 12 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 8 & 0 & 0 \\ 0 & 8 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 4 & 0 & 0 \\ 9 & 12 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 10 & 0 \\ 3 & 3 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

1,0,0,0,1,0,0,0,12,

1,0,0,0,1,0,0,0,5,

1,0,0,12,3,0,0,0,10,

0,1,0,12,12,0,0,0,4,

In order to assemble a list of isomorphism types of quartic curves from the csv file, we need to extract those lines for which the entry in column F_Fst is equal to one. The following command does so:

Code Snippet 813

```

1   quartic_curves_q13_filter_isomorphism_reps:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -csv_file_filter \
4   ▷ ▷ ▷ quartic_curves_q13_classified.csv \
5   ▷ ▷ ▷ "F_Fst" \
6   ▷ ▷ ▷ "1"
7   ▷ mv quartic_curves_q13_classified_f_F_Fst_1.csv quartic_curves_q13_iso.csv
8

```

The next command collects the orders of the automorphism groups of the representatives of the isomorphism classes:

Code Snippet 814

```

1   quartic_curves_q13_tally_ago:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -tally_column \
4   ▷ ▷ quartic_curves_q13_iso.csv AGO
5

```

The following example computes the set stabilizer of an elliptic curve over the field $\mathbb{F}_{11}$.

Code Snippet 815

```

1   EC_canon:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 11 -end \
4   ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
5   ▷ ▷ -define Geo -geometric_object P \
6   ▷ ▷ ▷ -elliptic_curve 1 3 \
7   ▷ ▷ -end \
8   ▷ ▷ -define C -combinatorial_object \
9   ▷ ▷ ▷ -label EC EC \
10  ▷ ▷ ▷ -geometric_object Geo \
11  ▷ ▷ -end \
12  ▷ ▷ -with C -do \
13  ▷ ▷ -combinatorial_object_activity \
14  ▷ ▷ ▷ -canonical_form_PG P \
15  ▷ ▷ ▷ ▷ -save_ago \
16  ▷ ▷ ▷ ▷ -save_transversal \
17  ▷ ▷ ▷ ▷ -max_TDO_depth 400 \
18  ▷ ▷ ▷ ▷ -nauty_control -nauty_log "EC_nauty_log.txt" -end \
19  ▷ ▷ ▷ -end \
20  ▷ ▷ -end \
21  ▷ ▷ -define Doinc -draw_incidence_structure_options \
22  ▷ ▷ ▷ -width 60 -width_10 6 \
23  ▷ ▷ -end \
24  ▷ ▷ -with C -do \
25  ▷ ▷ -combinatorial_object_activity \

```

```

26  ▷ ▷ ▷ -post_processing \
27  ▷ ▷ -end \
28  ▷ ▷ -with C -do \
29  ▷ ▷ -combinatorial_object_activity \
30  ▷ ▷ ▷ -report \
31  ▷ ▷ ▷ ▷ -export_flag_orbits \
32  ▷ ▷ ▷ ▷ -show_TDO \
33  ▷ ▷ ▷ ▷ -show_TDA \
34  ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
35  ▷ ▷ ▷ ▷ -export_group_GAP \
36  ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
37  ▷ ▷ ▷ -end \
38  ▷ ▷ -end
39  ▷ $(PDFLATEX) EC_classification.tex
40  ▷ $(OPEN) EC_classification.pdf
41  ▷ $(ORBITER) -v 2 -draw_matrix \
42  ▷ ▷ -input_csv_file EC_object_0_TDA_flag_orbits.csv \
43  ▷ ▷ -secondary_input_csv_file EC_object_0_TDA.csv \
44  ▷ ▷ -box_width 20 -bit_depth 24 \
45  ▷ ▷ -end
46  ▷ $(OPEN) EC_object_0_TDA_flag_orbits_draw.bmp
47

```

Orbiter shows that the curve has a collineation stabilizer of order 6, generated by

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 10 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 1 & 8 \\ 5 & 9 & 5 \\ 8 & 1 & 1 \end{bmatrix}.$$

The following example computes the canonical form and the automorphism group of the Hirschfeld surface in PG(3, 4). We use a makefile variable to store the equation of the surface in algebraic form:

Code Snippet 816

```

1  HIRSCHFELD_SURFACE_EQUATION_ALGEBRAIC_FORM="X0^2*X3 + X1^2*X2 + X1*X2^2 + X0*X3^2
   "
2

```

The following command creates the surface in PG(3, 4) and invokes the classification-by-canonical-form algorithm. This produces the collineation stabilizer of the surface:

Code Snippet 817

```

1  Hirschfeld.q4.c:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 4 -end \
4  ▷ ▷ -define R -polynomial_ring -field F \
5  ▷ ▷ ▷ -number_of_variables 4 \
6  ▷ ▷ ▷ -homogeneous_of_degree 3 \

```

```

7   ▷ ▷ ▷ -end \
8   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
9   ▷ ▷ -assign Eqn -from \
10  ▷ ▷ -with R -do \
11  ▷ ▷ ▷ -ring_theoretic_activity \
12  ▷ ▷ ▷ -parse_equation_wo_parameters \
13  ▷ ▷ ▷ ▷ "Hirschfeld_surface_q4" "Hirschfeld\_surface\_q4" \
14  ▷ ▷ ▷ ▷ $(HIRSCHFELD_SURFACE_EQUATION_ALGEBRAIC_FORM) \
15  ▷ ▷ -end \
16  ▷ ▷ -define Geo -geometric_object P \
17  ▷ ▷ ▷ -projective_variety R \
18  ▷ ▷ ▷ ▷ "Hirschfeld_surface_q4" "Hirschfeld\_surface\_q4" \
19  ▷ ▷ ▷ ▷ Eqn \
20  ▷ ▷ -end \
21  ▷ ▷ -define C -combinatorial_object \
22  ▷ ▷ ▷ -label Hirschfeld_surface_q4 Hirschfeld\_surface\_q4 \
23  ▷ ▷ ▷ -geometric_object Geo \
24  ▷ ▷ -end \
25  ▷ ▷ -with C -do \
26  ▷ ▷ -combinatorial_object_activity \
27  ▷ ▷ ▷ -canonical_form_PG P \
28  ▷ ▷ ▷ ▷ -save_ago \
29  ▷ ▷ ▷ ▷ -max_TDO_depth 10 \
30  ▷ ▷ ▷ ▷ -nauty_control \
31  ▷ ▷ ▷ ▷ ▷ -nauty_log "Hirschfeld.q4_nauty_log.txt" \
32  ▷ ▷ ▷ ▷ ▷ -save_nauty_input_graphs Hirschfeld.q4 \
33  ▷ ▷ ▷ ▷ -end \
34  ▷ ▷ ▷ -end \
35  ▷ ▷ -end \
36  ▷ ▷ -define Doinc -draw_incidence_structure_options \
37  ▷ ▷ ▷ -width 60 -width_10 6 \
38  ▷ ▷ -end \
39  ▷ ▷ -with C -do \
40  ▷ ▷ -combinatorial_object_activity \
41  ▷ ▷ ▷ -post_processing \
42  ▷ ▷ -end \
43  ▷ ▷ -with C -do \
44  ▷ ▷ -combinatorial_object_activity \
45  ▷ ▷ ▷ -report \
46  ▷ ▷ ▷ ▷ -export_flag_orbits \
47  ▷ ▷ ▷ ▷ -show_TDO \
48  ▷ ▷ ▷ ▷ -show_TDA \
49  ▷ ▷ ▷ ▷ -export_labels \
50  ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
51  ▷ ▷ ▷ ▷ -export_group_GAP \
52  ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
53  ▷ ▷ ▷ -end \
54  ▷ ▷ -end
55  ▷ $(PDFLATEX) Hirschfeld_surface.q4_classification.tex
56  ▷ $(OPEN) Hirschfeld_surface.q4_classification.pdf
57  ▷ $(ORBITER) -v 2 \
58  ▷ ▷ -define Do -draw_options -embedded -radius 150 \

```

```

59  ▷ ▷ ▷ -nodes_empty \
60  ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
61  ▷ ▷ ▷ -tree_draw \
62  ▷ ▷ ▷ -file Hirschfeld.q4.nauty_log.txt \
63  ▷ ▷ ▷ -draw_options Do \
64  ▷ ▷ ▷ -end
65  ▷ $(PDFLATEX) Hirschfeld.q4.nauty_log_draw.tex
66  ▷ $(OPEN) Hirschfeld.q4.nauty_log_draw.pdf
67

```

It is also possible to precompute the points of the surface and to pass this set to the classification-by-canonical-form algorithm. Using indexing of points in $\text{PG}(3,4)$, we encode the surface as a set of points using Orbiter ranks. We use a makefile variable to provide these ranks as input for the canonical form procedure.

Code Snippet 818

```

1  HIRSCHFELD_SURFACE_Q4.SET_OF_POINTS="0,1,2,3,4,5,6,7,8,9,\
2  10,11,12,13,14,23,26,27,30,31,34,35,38,39,42,47,48,51,52,\
3  53,54,59,60,61,62,67,68,69,70,75,76,79,80,81,82"
4

```

The following command invokes the classification-by-canonical-form algorithm to compute the collineation stabilizer of the surface:

Code Snippet 819

```

1  Hirschfeld.q4_set.c:
2  ▷ $(ORBITER) -v 4 \
3  ▷ ▷ -define H -set -here \
4  ▷ ▷ ▷ $(HIRSCHFELD_SURFACE_Q4.SET_OF_POINTS) \
5  ▷ ▷ -end \
6  ▷ ▷ -define C -combinatorial_object \
7  ▷ ▷ ▷ -label Hirschfeld_surface_q4 Hirschfeld\surface\_q4 \
8  ▷ ▷ ▷ -set_of_points H \
9  ▷ ▷ -end \
10 ▷ ▷ -define F -finite_field -q 4 -end \
11 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
12 ▷ ▷ -with C -do \
13 ▷ ▷ -combinatorial_object_activity \
14 ▷ ▷ ▷ -canonical_form_PG P \
15 ▷ ▷ ▷ ▷ -save_ago \
16 ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
17 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "Hirschfeld.q4_set_nauty_log.txt" -end \
18 ▷ ▷ ▷ -end \
19 ▷ ▷ -end \
20 ▷ ▷ -define Doinc -draw_incidence_structure_options \
21 ▷ ▷ ▷ -width 40 -width_10 6 \
22 ▷ ▷ -end \
23 ▷ ▷ -with C -do \
24 ▷ ▷ -combinatorial_object_activity \
25 ▷ ▷ ▷ -post_processing \

```



```

26  ▷ ▷ -end \
27  ▷ ▷ -with C -do \
28  ▷ ▷ -combinatorial_object_activity \
29  ▷ ▷ ▷ -report \
30  ▷ ▷ ▷ ▷ -show_TDO \
31  ▷ ▷ ▷ ▷ -show_TDA \
32  ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
33  ▷ ▷ ▷ -end \
34  ▷ ▷ -end
35  ▷ $(PDFLATEX) Hirschfeld_surface_q4_classification.tex
36  ▷ $(OPEN) Hirschfeld_surface_q4_classification.pdf
37

```

In the next example, we compute the canonical form of the two hyperovals in $\text{PG}(2, 16)$. The hyperovals are precomputed and given by means of their Orbiter point indices, as defined in makefile variables. The first hyperoval is called `HYPEROVAL_16_16320`:

Code Snippet 820

```

1  HYPEROVAL_16_16320="0, 1, 2, 3, 52, 70, 83, 109, 127, \
2  139, 156, 174, 186, 199, 217, 229, 256, 264"
3

```

The second hyperoval is called `HYPEROVAL_16_144`:

Code Snippet 821

```

1  HYPEROVAL_16_144="0, 1, 2, 3, 52, 67, 89, 106, 126, \
2  141, 159, 176, 184, 199, 220, 235, 245, 262"
3

```

The following command invokes the classification-by-canonical-form algorithm:

Code Snippet 822

```

1  hyperoval_16_canonical_form:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define C -combinatorial_object \
4  ▷ ▷ ▷ -label hyperoval_q16 hyperoval\_q16 \
5  ▷ ▷ ▷ -set_of_points $(HYPEROVAL_16_16320) \
6  ▷ ▷ ▷ -set_of_points $(HYPEROVAL_16_144) \
7  ▷ ▷ -end \
8  ▷ ▷ -define F -finite_field -q 16 -end \
9  ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
10 ▷ ▷ -with C -do \
11 ▷ ▷ -combinatorial_object_activity \
12 ▷ ▷ ▷ -canonical_form_PG P \
13 ▷ ▷ ▷ ▷ -save_ago \
14 ▷ ▷ ▷ ▷ -save_transversal \
15 ▷ ▷ ▷ ▷ -max_TDO_depth 10 \
16 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "hyperoval_16_nauty_log.txt" -end \

```

```

17   ▷ ▷ ▷ -end \
18   ▷ ▷ -end \
19   ▷ ▷ -define Doinc -draw_incidence_structure_options \
20   ▷ ▷ ▷ -width 40 -width_10 6 \
21   ▷ ▷ -end \
22   ▷ ▷ -with C -do \
23   ▷ ▷ -combinatorial_object_activity \
24   ▷ ▷ ▷ -post_processing \
25   ▷ ▷ -end \
26   ▷ ▷ -with C -do \
27   ▷ ▷ -combinatorial_object_activity \
28   ▷ ▷ ▷ -report \
29   ▷ ▷ ▷ ▷ -export_flag_orbits \
30   ▷ ▷ ▷ ▷ -show_TDO \
31   ▷ ▷ ▷ ▷ -show_TDA \
32   ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
33   ▷ ▷ ▷ ▷ -export_group_GAP \
34   ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
35   ▷ ▷ ▷ -end \
36   ▷ ▷ -end
37   ▷ $(PDFLATEX) hyperoval_q16_classification.tex
38   ▷ $(OPEN) hyperoval_q16_classification.pdf
39   ▷ $(ORBITER) -v 2 -draw_matrix \
40   ▷ ▷ -input_csv_file hyperoval_q16_object_0_TDA_flag_orbits.csv \
41   ▷ ▷ -secondary_input_csv_file hyperoval_q16_object_0_TDA.csv \
42   ▷ ▷ -box_width 4 -bit_depth 24 \
43   ▷ ▷ -end
44   ▷ $(OPEN) hyperoval_q16_object_0_TDA_flag_orbits.draw.bmp
45   ▷ $(ORBITER) -v 2 -draw_matrix \
46   ▷ ▷ -input_csv_file hyperoval_q16_object_1_TDA_flag_orbits.csv \
47   ▷ ▷ -secondary_input_csv_file hyperoval_q16_object_1_TDA.csv \
48   ▷ ▷ -box_width 4 -bit_depth 24 \
49   ▷ ▷ -end
50   ▷ $(OPEN) hyperoval_q16_object_1_TDA_flag_orbits.draw.bmp
51   ▷ $(ORBITER) -v 2 \
52   ▷ ▷ -define Do -draw_options -embedded -radius 150 \
53   ▷ ▷ ▷ -nodes_empty \
54   ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
55   ▷ ▷ -tree_draw \
56   ▷ ▷ ▷ -file hyperoval_16_nauty_log.txt \
57   ▷ ▷ ▷ -draw_options Do \
58   ▷ ▷ -end
59   ▷ $(PDFLATEX) hyperoval_16_nauty_log.draw.tex
60   ▷ $(OPEN) hyperoval_16_nauty_log.draw.pdf
61

```

In the work in Dickson [29], the set stabilizers of all subsets of $\text{PG}(3, 2)$ of odd size is required. In Section 8.3, we computed these orbits using the command `PG_3_2_subsets`. The next command invokes the classification-by-canonical-form algorithm for these sets:

Code Snippet 823

```

1   Dickson_sets_stabilizer:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define C -combinatorial_object \
4   ▷ ▷ ▷ -label Dickson_sets Dickson\_sets \
5   ▷ ▷ ▷ -set_of_points "0,1,2,5,6" \
6   ▷ ▷ ▷ -set_of_points "0,1,2,3,6" \
7   ▷ ▷ ▷ -set_of_points "0,1,2,3,4" \
8   ▷ ▷ ▷ -set_of_points "0,1,2,3,8" \
9   ▷ ▷ ▷ -set_of_points "0,1,2,5,6,7,8" \
10  ▷ ▷ ▷ -set_of_points "0,1,2,3,5,6,7" \
11  ▷ ▷ ▷ -set_of_points "0,1,2,3,5,6,9" \
12  ▷ ▷ ▷ -set_of_points "0,1,2,3,5,6,10" \
13  ▷ ▷ ▷ -set_of_points "0,1,2,3,5,6,4" \
14  ▷ ▷ ▷ -set_of_points "0,1,2,3,8,11,13" \
15  ▷ ▷ ▷ -set_of_points "3,6,9,7,10,12,8,11,13,14,4" \
16  ▷ ▷ ▷ -set_of_points "3,5,6,9,7,10,12,11,13,14,4" \
17  ▷ ▷ ▷ -set_of_points "0,1,2,3,5,6,9,7,10,12,4" \
18  ▷ ▷ -end \
19  ▷ ▷ -define F -finite_field -q 2 -end \
20  ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
21  ▷ ▷ -with C -do \
22  ▷ ▷ -combinatorial_object_activity \
23  ▷ ▷ ▷ -canonical_form_PG P \
24  ▷ ▷ ▷ ▷ -save_ago \
25  ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
26  ▷ ▷ ▷ ▷ -nauty_control -nauty_log "Dickson_sets_nauty_log.txt" -end \
27  ▷ ▷ ▷ -end \
28  ▷ ▷ -end \
29  ▷ ▷ -define Doinc -draw_incidence_structure_options \
30  ▷ ▷ ▷ -width 40 -width_10 6 \
31  ▷ ▷ -end \
32  ▷ ▷ -with C -do \
33  ▷ ▷ -combinatorial_object_activity \
34  ▷ ▷ ▷ -post_processing \
35  ▷ ▷ -end \
36  ▷ ▷ -with C -do \
37  ▷ ▷ -combinatorial_object_activity \
38  ▷ ▷ ▷ -report \
39  ▷ ▷ ▷ ▷ -show_TDO \
40  ▷ ▷ ▷ ▷ -show_TDA \
41  ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
42  ▷ ▷ ▷ -end \
43  ▷ ▷ -end
44  ▷ $(PDFLATEX) Dickson_sets_classification.tex
45  ▷ $(OPEN) Dickson_sets_classification.pdf
46

```

There are two ovoids in $PG(3,2)$, the classical ovoid and the Suzuki-Tits ovoid. The classical ovoid is the elliptic quadric. In Section 4.5, it was created using the command `elliptic_quadric_ovoid_q8`. The following command invokes the classification-by-canonical-form algorithm to compute the stabilizer of the ovoid:

Code Snippet 824

```

1   ovoid.q8_canon:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 8 -end \
4   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
5   ▷ ▷ -define Geo -geometric_object P \
6   ▷ ▷ ▷ -elliptic_quadric_ovoid \
7   ▷ ▷ -end \
8   ▷ ▷ -define C -combinatorial_object \
9   ▷ ▷ ▷ -label ovoid ovoid \
10  ▷ ▷ ▷ -geometric_object Geo \
11  ▷ ▷ -end \
12  ▷ ▷ -with C -do \
13  ▷ ▷ -combinatorial_object_activity \
14  ▷ ▷ ▷ -canonical_form_PG P \
15  ▷ ▷ ▷ ▷ -save_ago \
16  ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
17  ▷ ▷ ▷ ▷ -nauty_control -nauty_log "ovoid.q8_nauty_log.txt" -show_canonical_form
    -end \
18  ▷ ▷ ▷ -end \
19  ▷ ▷ -end \
20  ▷ ▷ -define Doinc -draw_incidence_structure_options \
21  ▷ ▷ ▷ -width 40 -width_10 6 \
22  ▷ ▷ -end \
23  ▷ ▷ -with C -do \
24  ▷ ▷ -combinatorial_object_activity \
25  ▷ ▷ ▷ -post_processing \
26  ▷ ▷ -end \
27  ▷ ▷ -with C -do \
28  ▷ ▷ -combinatorial_object_activity \
29  ▷ ▷ ▷ -report \
30  ▷ ▷ ▷ ▷ -show_TDO \
31  ▷ ▷ ▷ ▷ -show_TDA \
32  ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
33  ▷ ▷ ▷ ▷ -export_group_GAP \
34  ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
35  ▷ ▷ ▷ -end \
36  ▷ ▷ -end
37  ▷ $(PDFLATEX) ovoid_classification.tex
38  ▷ $(OPEN) ovoid_classification.pdf
39

```

The stabilizer of the Suzuki-Tits ovoid is the Suzuki group. In Section 4.5, we have created the Suzuki-Tits ovoid using the command `ovoid_ST_q8`. The following command invokes the classification-by-canonical-form algorithm to compute the stabilizer of the Suzuki ovoid, which is the Suzuki group for $q = 8$.

Code Snippet 825

```

1   ovoid.ST_q8_canon:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 8 -end \
4   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \

```

```

5  ▷ ▷ -define Geo -geometric_object P \
6  ▷ ▷ ▷ -ovoid_ST \
7  ▷ ▷ -end \
8  ▷ ▷ -define C -combinatorial_object \
9  ▷ ▷ ▷ -label ovoid_ST ovoid\_ST \
10 ▷ ▷ ▷ -geometric_object Geo \
11 ▷ ▷ -end \
12 ▷ ▷ -with C -do \
13 ▷ ▷ -combinatorial_object_activity \
14 ▷ ▷ ▷ -canonical_form_PG P \
15 ▷ ▷ ▷ ▷ -save_ago \
16 ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
17 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "ovoid_ST.nauty_log.txt" -end \
18 ▷ ▷ ▷ -end \
19 ▷ ▷ -end \
20 ▷ ▷ -define Doinc -draw_incidence_structure_options \
21 ▷ ▷ ▷ -width 40 -width_10 6 \
22 ▷ ▷ -end \
23 ▷ ▷ -with C -do \
24 ▷ ▷ -combinatorial_object_activity \
25 ▷ ▷ ▷ -post_processing \
26 ▷ ▷ -end \
27 ▷ ▷ -with C -do \
28 ▷ ▷ -combinatorial_object_activity \
29 ▷ ▷ ▷ -report \
30 ▷ ▷ ▷ ▷ -show_TDO \
31 ▷ ▷ ▷ ▷ -show_TDA \
32 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
33 ▷ ▷ ▷ ▷ -export_group_GAP \
34 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
35 ▷ ▷ ▷ -end \
36 ▷ ▷ -end
37 ▷ $(PDFLATEX) ovoid_ST_classification.tex
38 ▷ $(OPEN) ovoid_ST_classification.pdf
39

```

We may store the generators of the Suzuki group in a makefile variable as follows:

Code Snippet 826

```

1  SUZUKI_8_GENERATORS="\
2  1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,1, \
3  1,0,0,0,0,6,0,0,0,0,2,0,0,0,0,3,0, \
4  1,0,0,0,1,1,1,0,0,0,1,0,1,0,0,1,0, \
5  1,0,0,0,3,6,2,2,5,0,2,0,3,0,6,3,2, \
6  0,1,0,0,1,0,0,0,0,0,0,1,0,0,1,0,2"
7  # group order 87360 = 3 * 29120
8

```

The following command recreates the Suzuki group for $q = 8$ from the given set of generators.

Code Snippet 827

```

1   Suzuki_8:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define F -finite_field -q 8 -end \
7   ▷ ▷ -define gens -vector -field F \
8   ▷ ▷ ▷ -compact $(SUZUKI_8.GENERATORS) -end \
9   ▷ ▷ -define G -linear_group -PGGL 4 8 \
10  ▷ ▷ -subgroup_by_generators "Sz8" "87360" 5 gens \
11  ▷ ▷ ▷ -end \
12  ▷ ▷ -with G -do \
13  ▷ ▷ -group_theoretic_activity \
14  ▷ ▷ ▷ -report Do \
15  ▷ ▷ -end
16  ▷ $(PDFLATEX) PGGL_4.8_Subgroup_Sz8.87360_report.tex
17  ▷ $(OPEN) PGGL_4.8_Subgroup_Sz8.87360_report.pdf
18

```

In Section 7.6, we have created a Veronese variety over the field $\mathbb{F}_7$. The following command computes the stabilizer of this variety.

Code Snippet 828

```

1   Veronese_1.3_q7_c:
2   ▷ $(ORBITER) -v 6 \
3   ▷ ▷ -define C -combinatorial_object \
4   ▷ ▷ ▷ -label veronese_1.3_q7 veronese\_1\_3\_q7 \
5   ▷ ▷ ▷ -set_of_points "3, 0, 4, 268, 336, 184, 224, 364" \
6   ▷ ▷ -end \
7   ▷ ▷ -define F -finite_field -q 7 -end \
8   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
9   ▷ ▷ -with C -do \
10  ▷ ▷ -combinatorial_object_activity \
11  ▷ ▷ ▷ -canonical_form_PG P \
12  ▷ ▷ ▷ ▷ -save_ago \
13  ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
14  ▷ ▷ ▷ ▷ -nauty_control -nauty_log "Veronese_1.3_q7_nauty_log.txt" -end \
15  ▷ ▷ ▷ -end \
16  ▷ ▷ -end \
17  ▷ ▷ -define DoInc -draw_incidence_structure_options \
18  ▷ ▷ ▷ -width 40 -width_10 6 \
19  ▷ ▷ -end \
20  ▷ ▷ -with C -do \
21  ▷ ▷ -combinatorial_object_activity \
22  ▷ ▷ ▷ -post_processing \
23  ▷ ▷ -end \
24  ▷ ▷ -with C -do \
25  ▷ ▷ -combinatorial_object_activity \
26  ▷ ▷ ▷ -report \

```

```

27  ▷ ▷ ▷ ▷ -show_TDO \
28  ▷ ▷ ▷ ▷ -show_TDA \
29  ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
30  ▷ ▷ ▷ ▷ -export_group_GAP \
31  ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
32  ▷ ▷ ▷ -end \
33  ▷ ▷ -end
34  ▷ $(PDFLATEX) veronese_1_3_q7_classification.tex
35  ▷ $(OPEN) veronese_1_3_q7_classification.pdf
36

```

The points are printed with their homogeneous coordinates:

i	Rank	Point
0	3	(0, 0, 0, 1)
1	0	(1, 0, 0, 0)
2	4	(1, 1, 1, 1)
3	268	(1, 2, 4, 1)
4	336	(6, 4, 5, 1)
5	184	(1, 4, 2, 1)
6	224	(6, 2, 3, 1)
7	364	(6, 1, 6, 1)

The automorphism group is determined to have order 336. Generators are computed:

The stabilizer of order 336 is generated by:

$$\begin{aligned}
 g_1 &= \begin{bmatrix} 6 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 6 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} && \text{of order 2 and with 16 fixed points.} \\
 g_2 &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} && \text{of order 3 and with 10 fixed points.} \\
 g_3 &= \begin{bmatrix} 6 & 0 & 0 & 0 \\ 4 & 2 & 0 & 0 \\ 4 & 4 & 3 & 0 \\ 6 & 2 & 3 & 1 \end{bmatrix} && \text{of order 6 and with 4 fixed points.} \\
 g_4 &= \begin{bmatrix} 1 & 5 & 4 & 6 \\ 6 & 1 & 0 & 3 \\ 5 & 0 & 2 & 4 \\ 1 & 1 & 1 & 1 \end{bmatrix} && \text{of order 8 and with 0 fixed points.}
 \end{aligned}$$

1,0,0,0,0,6,0,0,0,0,1,0,0,0,0,6,

```

1,0,0,0,0,2,0,0,0,0,4,0,0,0,0,1,
1,0,0,0,3,5,0,0,3,3,4,0,1,5,4,6,
1,5,4,6,6,1,0,3,5,0,2,4,1,1,1,1,

```

A coding of the generators is printed at the very end. This coding can be used later to recreate the group. Finally, a tactical decomposition is produced. This is the extended incidence matrix of $\text{PG}(3, 7)$. One artificial point and one artificial line are introduced to encode the variety as a combinatorial object. The TDO is obtained by refining the partition.

Decomposition by combinatorial refinement:

$\rightarrow$	1_1	28_2	400_5	2422_6
8_0	1	7	50	0
1_4	1	0	0	0
168_3	0	1	6	50
224_7	0	0	8	49
$\downarrow$	1_1	28_2	400_5	2422_6
8_0	8	2	1	0
1_4	1	0	0	0
168_3	0	6	2	3
224_7	0	0	5	5

17.3 Canonical Forms of Incidence Geometries

Let us consider incidence structures, which are sets of subsets. These subsets are chosen from the same set, which we call the carrier set. The elements of the set are often called points. The distinguished subsets are called blocks or sometimes lines. There are conditions that restrict the way in which the sets can be chosen. There is a notion of isomorphism on such set systems. Two set systems are isomorphic if there is a bijection between the underlying sets which takes one to the other. The incidence matrix is the 0/1 matrix whose rows correspond to the elements of the group set, and whose columns correspond to the chosen subsets. An entry 1 indicates that the corresponding point belongs to the corresponding block. A flag is an incident point-line pair. An anti-flag is a non-incident point-line pair. Two points are said to be collinear if there is a line containing both.

It is interesting to study the action of the automorphism group on the elements of a geometry. Properties of interest are various levels of transitivity on the elements of the geometry. For instance, a geometry is line-transitive if the automorphism group is transitive on lines. Likewise, it is flag transitive if the automorphism group is transitive on flags. The collinearity graph of a geometry is the graph whose vertices correspond to the points, with two vertices adjacent if the associated points are collinear. The girth of the incidence geometry is the girth of the associated collineation graph. A geometry is triangle free if its girth is at least 4.

A configuration $v_r b_k$ is an incidence geometry on a set of size v and with b lines such that each line has size k and each point is contained in exactly r lines. In the special case where $b = v$ and $k = r$, the name symmetric configuration v_r is used (the term symmetric is somewhat misleading because the incidence matrix of a symmetric configuration need not be symmetric). Orbiter can be used to classify incidence geometries. One of the important steps in this process is computing a canonical form of the incidence geometry.

We will also be producing drawings of the incidence matrices of geometries. There are two types of drawings:

1. Bitmap drawings,
2. Latex drawings.

In the bitmap drawings, flags are indicated as heavy squares while anti-flags are drawn as small squares. The coloring will indicate the orbits of the automorphism group on flags and anti-flags. Objects with the same color belong to the same orbit. In a flag-transitive geometry, all incidences have the same color, which is black. Bitmaps can be created using the `-draw_matrix` command. The input is an integer-valued matrix in csv format. The matrix entries are translated into colors. The possible commands after `-draw_matrix` are shown in Table 17.4. Orbiter uses latex drawings for instance in reports. Table 17.5 shows the available commands to customize such drawings.

The following command computes the canonical form and a report of the projective plane $\text{PG}(2, 2)$, which is a configuration 7_3 .

Code Snippet 829

```

1  geo_7_3_c:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define Test_lines -set -loop 3 5 1 -end \
4  ▷ ▷ -define Geo -geometry_builder \
5  ▷ ▷ ▷ -V 7 -B 7 -TDO 3 \
6  ▷ ▷ ▷ -fuse 1 \
7  ▷ ▷ ▷ -fname_GEO 7_3 \
8  ▷ ▷ ▷ -output_to_inc_file \
9  ▷ ▷ ▷ -test Test_lines \
10 ▷ ▷ -end

```

Creating a Bitmap Graphics		
Command	Arguments	Purpose
-input_csv_file	csv-file	Specify the input csv file.
-input_csv_file_column	csv-file column-label	Specify the input csv file and the column from which to pull the data.
-secondary_input_csv_file	csv-file	Specify a secondary input csv file (optional).
-input_object	l	Specify input to be Orbiter object l .
-partition	$w \ R \ C$	Specify a partition R of rows and C of columns. Use separating lines of width w .
-box_width	w	Use w pixels per matrix entry.
-bit_depth	d	Use color bit depth of d bits ($d = 8$ or $d = 24$).
-invert_colors		Use an inverted color scheme.
-grayscale		Use a grayscale color set.
-indent	i	Shrink the width and height of a box to achieve padding. The value for indentation is i , which is given in percent. The color white is used for padding.
-color_offset	i	Apply the given offset to the choice of colors. This is useful to distinguish the colors in the matrix from the color used for padding.

Table 17.4: Creating a Bitmap Graphics

Options for Drawing Incidence Structures		
Command	Arguments	Purpose
-width	w	Set the width of one box in units of 0.1mm.
-width_10	w	Set the width of one tenth of a box. Should with the value of width divided by 10.
-outline_thin		Draw a thin outline.
-unit_length		Default: 0.065mm.
-thick_lines		Default: 0.5mm.
-thin_lines		Default: 0.15mm.
-geo_line_width		Default: 0.25mm.

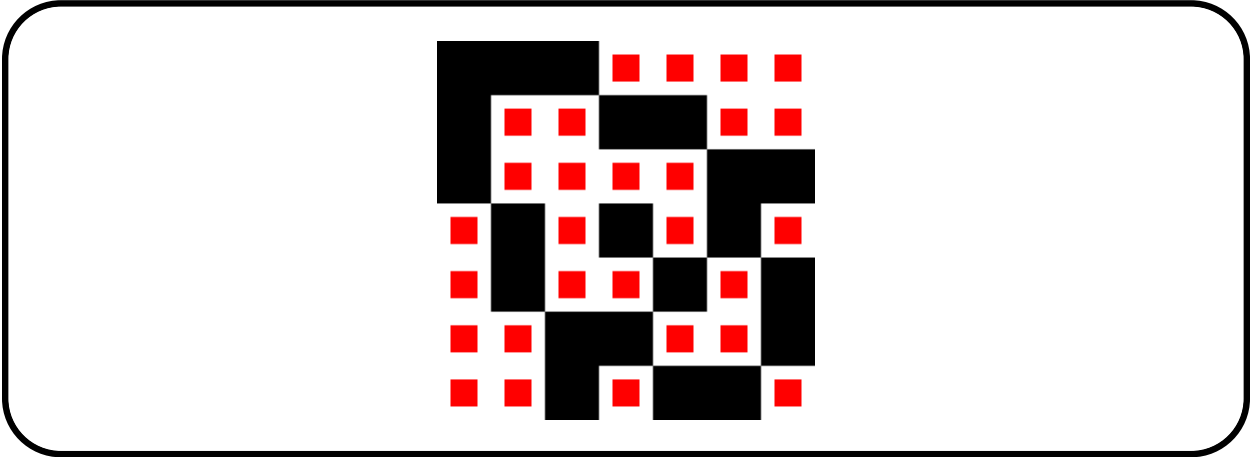
Table 17.5: Options for Drawing Incidence Structures

```

11  ▷ $(ORBITER) -v 10 \
12  ▷ ▷ -define Doinc -draw_incidence_structure_options \
13  ▷ ▷ ▷ -width 60 -width_10 6 -end \
14  ▷ ▷ -define C -combinatorial_object \
15  ▷ ▷ ▷ -label geo_7_3 geo\7\3 \
16  ▷ ▷ ▷ -file_of_incidence_geometries 7_3.inc 7 7 21 \
17  ▷ ▷ -end \
18  ▷ ▷ -with C -do \
19  ▷ ▷ -combinatorial_object_activity \
20  ▷ ▷ ▷ -canonical_form \
21  ▷ ▷ ▷ ▷ -save_ago \
22  ▷ ▷ ▷ ▷ -save_transversal \
23  ▷ ▷ ▷ ▷ -nauty_control -nauty_log "geo_7_3.nauty_log.txt" -show_canonical_form -e
    nd \
24  ▷ ▷ ▷ -end \
25  ▷ ▷ -end \
26  ▷ ▷ -with C -do \
27  ▷ ▷ -combinatorial_object_activity \
28  ▷ ▷ ▷ -post_processing \
29  ▷ ▷ -end \
30  ▷ ▷ -with C -do \
31  ▷ ▷ -combinatorial_object_activity \
32  ▷ ▷ ▷ -report \
33  ▷ ▷ ▷ ▷ -export_flag_orbits \
34  ▷ ▷ ▷ ▷ -show_incidence_matrices \
35  ▷ ▷ ▷ ▷ -export_group_GAP \
36  ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
37  ▷ ▷ ▷ -end \
38  ▷ ▷ -end
39  ▷ $(PDFLATEX) geo_7_3_classification.tex
40  ▷ $(OPEN) geo_7_3_classification.pdf
41  ▷ $(ORBITER) -v 2 -draw_matrix \
42  ▷ ▷ -input_csv_file geo_7_3.object_0.TDA.flag_orbits.csv \
43  ▷ ▷ -secondary_input_csv_file geo_7_3.object_0.TDA.csv \
44  ▷ ▷ -box_width 32 -bit_depth 24 \
45  ▷ ▷ -end
46  ▷ $(ORBITER) -v 2 -draw_matrix \
47  ▷ ▷ -input_csv_file geo_7_3.object_0.INP.flag_orbits.csv \
48  ▷ ▷ -secondary_input_csv_file geo_7_3.object_0.INP.csv \
49  ▷ ▷ -box_width 32 -bit_depth 24 \
50  ▷ ▷ -end
51  ▷ $(OPEN) geo_7_3.object_0.INP.flag_orbits.draw.bmp
52

```

A bitmap drawing is produced, shown below:



The command also produces the following report of the geometry:

Summary of Orbits

	Rep	#	Ago	Objects
0	0	1	168	0

Ago : 168

Isomorphism type 0 / 1

Isomorphism type 0 / 1 is original object 0 and appears 1 times:

This isomorphism type appears 1 times, namely for the following 1 input objects: {0}
incidence structure:

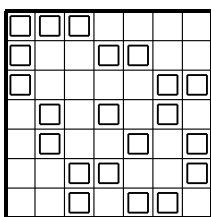
(0, 1, 2, 7, 10, 11, 14, 19, 20, 22, 24, 26, 29, 32, 34, 37, 38, 41, 44, 46, 47)

Column sets of the encoded object:

- { 0, 1, 2 }
- { 0, 3, 4 }
- { 0, 5, 6 }
- { 1, 3, 5 }
- { 1, 4, 6 }
- { 2, 3, 6 }
- { 2, 4, 5 }

Row sets of the encoded object:

- { 0, 1, 2 } = 0
- { 0, 3, 4 } = 9
- { 0, 5, 6 } = 14
- { 1, 3, 5 } = 20
- { 1, 4, 6 } = 23
- { 2, 3, 6 } = 27
- { 2, 4, 5 } = 28



Generators for the automorphism group:

The stabilizer of order 168 is generated by:

$g_1 = (3, 5)(4, 6)(8, 9)(12, 13)$ of order 2 and with 6 fixed points.

$g_2 = (3, 4)(5, 6)(10, 11)(12, 13)$ of order 2 and with 6 fixed points.

$g_3 = (1, 2)(5, 6)(10, 12)(11, 13)$ of order 2 and with 6 fixed points.

$g_4 = (1, 3)(2, 4)(7, 8)(11, 12)$ of order 2 and with 6 fixed points.

$g_5 = (0, 1)(4, 5)(8, 10)(9, 11)$ of order 2 and with 6 fixed points.

Canonical labeling:

canonical row = 6

canonical orbit number = 0

Flags : (0, 1, 2, 7, 10, 11, 14, 19, 20, 22, 24, 26, 30, 31, 34, 36, 39, 41, 44, 46, 47)

	7	8	9	10	11	12	13
*0	□	□	□				
*1	□			□	□		
*2	□					□	□
*3		□			□		□
*5			□	□			□
*4		□			□		□
*6			□		□	□	

Flag orbits:

orbit length : number of orbits of that length:

21 1

Anti-Flag orbits:

orbit length : number of orbits of that length:

28 1

The following command computes the canonical form and a report of the affine plane $AG(2, 3)$, which is a configuration $9_4 12_3$.

Code Snippet 830

```

1    AG_2_3_c: AG_2_3.inc
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define C -combinatorial_object \
4    ▷ ▷ ▷ -label AG_2_3 AG\_2\_3 \
5    ▷ ▷ ▷ -file_of_incidence_geometries \
6    ▷ ▷ ▷ AG_2_3.inc 9 12 36 \
7    ▷ ▷ -end \
8    ▷ ▷ -with C -do \
9    ▷ ▷ -combinatorial_object_activity \

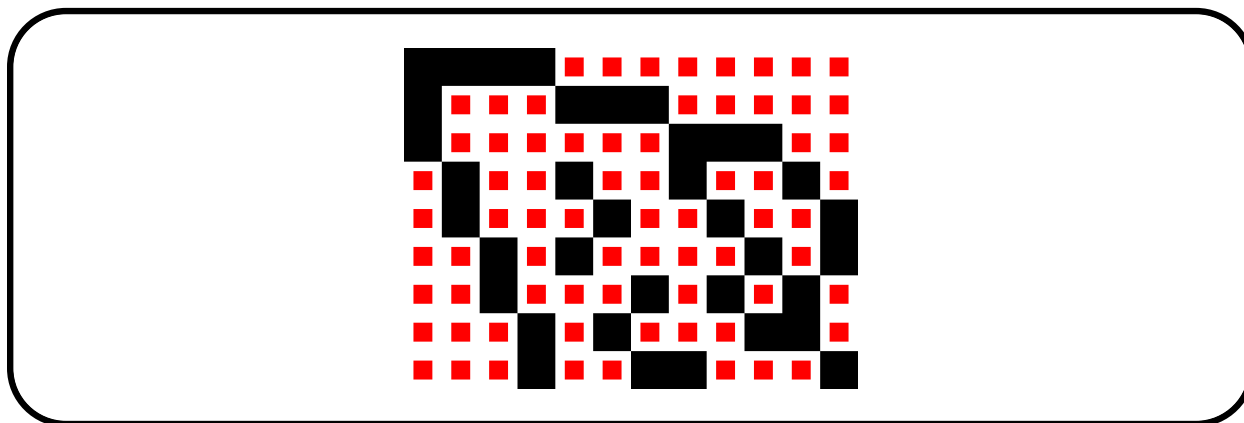
```

```

10  ▷ ▷ ▷ -canonical_form \
11  ▷ ▷ ▷ ▷ -save_ago \
12  ▷ ▷ ▷ ▷ -max_TDO_depth 10 \
13  ▷ ▷ ▷ ▷ -nauty_control -nauty_log "AG_2.3_nauty_log.txt" -end \
14  ▷ ▷ ▷ -end \
15  ▷ ▷ -end \
16  ▷ ▷ -define Doinc -draw_incidence_structure_options \
17  ▷ ▷ ▷ -width 40 -width_10 6 \
18  ▷ ▷ -end \
19  ▷ ▷ -with C -do \
20  ▷ ▷ -combinatorial_object_activity \
21  ▷ ▷ ▷ -post_processing \
22  ▷ ▷ -end \
23  ▷ ▷ -with C -do \
24  ▷ ▷ -combinatorial_object_activity \
25  ▷ ▷ ▷ -report \
26  ▷ ▷ ▷ ▷ -export_flag_orbits \
27  ▷ ▷ ▷ ▷ -show_TDO \
28  ▷ ▷ ▷ ▷ -show_TDA \
29  ▷ ▷ ▷ ▷ -show_incidence_matrices \
30  ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
31  ▷ ▷ ▷ -end \
32  ▷ ▷ -end
33  ▷ $(PDFLATEX) AG_2.3_classification.tex
34  ▷ $(OPEN) AG_2.3_classification.pdf
35  ▷ $(ORBITER) -v 2 -draw_matrix \
36  ▷ ▷ -input_csv_file AG_2.3_object_0_INP_flag_orbits.csv \
37  ▷ ▷ -secondary_input_csv_file AG_2.3_object_0_INP.csv \
38  ▷ ▷ -box_width 40 -bit_depth 24 \
39  ▷ ▷ -end
40  ▷ $(OPEN) AG_2.3_object_0_INP_flag_orbits_draw.bmp
41

```

The following bitmap drawing is produced:



Because the geometry is flag transitive, there is only one color being used for the incidence (black). In addition, the geometry is also anti-flag transitive. The smallish boxes all have the same color (red). Orbiter also produces the following report of the geometry:

Summary of Orbits

	Rep	#	Ago	Objects
0	0	1	432	0

Ago :432

Isomorphism type 0 / 1

Isomorphism type 0 / 1 is original object 0 and appears 1 times:

This isomorphism type appears 1 times, namely for the following 1 input objects: {0}

incidence structure:

(0, 1, 2, 3, 12, 16, 17, 18, 24, 31, 32, 33, 37, 40, 43, 46, 49, 53, 56, 59, 62, 64, 69, 71, 74, 78, 80, 82, 87, 89, 93, 94, 99, 102, 103, 107)

Generators for the automorphism group:

The stabilizer of order 432 is generated by:

$g_1 = (3, 4)(5, 7)(6, 8)(11, 12)(13, 14)(16, 17)(19, 20)$ of order 2 and with 7 fixed points.

$g_2 = (3, 5)(4, 6)(7, 8)(10, 11)(14, 15)(16, 18)(19, 20)$ of order 2 and with 7 fixed points.

$g_3 = (1, 3)(2, 4)(7, 8)(9, 10)(14, 16)(15, 19)(18, 20)$ of order 2 and with 7 fixed points.

$g_4 = (0, 1)(4, 5)(6, 7)(10, 13)(11, 14)(12, 15)(17, 18)$ of order 2 and with 7 fixed points.

Decomposition by combinatorial refinement:

$\rightarrow$	12_1
9_0	4
$\downarrow$	12_1
9_0	3

Decomposition by automorphism group:

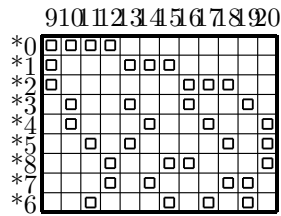
91011324569780

Canonical labeling:

canonical row = 6

canonical orbit number = 0

Flags : (0, 1, 2, 3, 12, 16, 17, 18, 24, 31, 32, 33, 37, 40, 43, 46, 49, 53, 56, 59, 62, 64, 69, 71, 75, 78, 79, 83, 87, 89, 93, 94, 98, 102, 104, 106)



Flag orbits:

orbit length : number of orbits of that length:

36 1

Anti-Flag orbits:

orbit length : number of orbits of that length:

72 1

It is possible to perform isomorph classification for configurations based on incidence files. Suppose we want to check that the configurations in 10_3 are in fact all distinct, in the sense of being pairwise-nonisomorphic. We apply the canonical form algorithm given by Nauty. This produces a transversal of the isomorphism types of incidence geometries from the given list of input objects. The objects are specified by means of the `combinatorial_objects` command.

Code Snippet 831

```

1   geo_10_3.c:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Test_lines -set -loop 3 8 1 -end \
4   ▷ ▷ -define Geo -geometry_builder \
5   ▷ ▷ ▷ -V 10 -B 10 -TDO 3 \
6   ▷ ▷ ▷ -fuse 1 \
7   ▷ ▷ ▷ -fname_GEO 10_3 \
8   ▷ ▷ ▷ -output_to_inc_file \
9   ▷ ▷ ▷ -test Test_lines \
10  ▷ ▷ -end
11  ▷ $(ORBITER) -v 10 \
12  ▷ ▷ -define Doinc -draw_incidence_structure_options \
13  ▷ ▷ ▷ -width 60 -width_10 6 -end \
14  ▷ ▷ -define C -combinatorial_object \
15  ▷ ▷ ▷ -label geo_10_3 geo\10\3 \
16  ▷ ▷ ▷ -file_of_incidence_geometries 10_3.inc 10 10 30 \
17  ▷ ▷ -end \
18  ▷ ▷ -with C -do \
19  ▷ ▷ -combinatorial_object_activity \
20  ▷ ▷ ▷ -canonical_form \
21  ▷ ▷ ▷ ▷ -save_ago \
22  ▷ ▷ ▷ ▷ -save_transversal \
23  ▷ ▷ ▷ ▷ -nauty_control -nauty_log "geo_10_3.nauty_log.txt" -end \
24  ▷ ▷ ▷ -end \
25  ▷ ▷ -end \
26  ▷ ▷ -with C -do \
27  ▷ ▷ -combinatorial_object_activity \

```



```

28   ▷ ▷ ▷ -post_processing \
29   ▷ ▷ -end \
30   ▷ ▷ -with C -do \
31   ▷ ▷ -combinatorial_object_activity \
32   ▷ ▷ ▷ -report \
33   ▷ ▷ ▷ ▷ -export_flag_orbits \
34   ▷ ▷ ▷ ▷ -show_incidence_matrices \
35   ▷ ▷ ▷ ▷ -export_group_GAP \
36   ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
37   ▷ ▷ ▷ -end \
38   ▷ ▷ -end
39   ▷ $(PDFLATEX) geo_10_3_classification.tex
40   ▷ $(OPEN) geo_10_3_classification.pdf
41   ▷ $(ORBITER) -v 2 \
42   ▷ ▷ -loop L 0 10 1 \
43   ▷ ▷ ▷ -draw_matrix \
44   ▷ ▷ ▷ -input_csv_file geo_10_3_object_%L.TDA.flag_orbits.csv \
45   ▷ ▷ ▷ -secondary_input_csv_file geo_10_3_object_%L.TDA.csv \
46   ▷ ▷ ▷ -box_width 16 -bit_depth 24 \
47   ▷ ▷ ▷ -end \
48   ▷ ▷ -end_loop L
49   ▷ $(ORBITER) -v 2 \
50   ▷ ▷ -loop L 0 10 1 \
51   ▷ ▷ ▷ -draw_matrix \
52   ▷ ▷ ▷ -input_csv_file geo_10_3_object_%L.INP.flag_orbits.csv \
53   ▷ ▷ ▷ -secondary_input_csv_file geo_10_3_object_%L.INP.csv \
54   ▷ ▷ ▷ -box_width 16 -bit_depth 24 \
55   ▷ ▷ ▷ -end \
56   ▷ ▷ -end_loop L
57   ▷ #magick font_[0-3].gif -delete 1 +append seq_delete.gif
58   ▷ magick geo_10_3_object_[0-5].TDA.flag_orbits_draw.bmp -frame 5 +append a0.png
59   ▷ magick geo_10_3_object_[5-9].TDA.flag_orbits_draw.bmp geo_10_3_object_9.TDA fla
    g_orbits_draw.bmp -frame 5 +append a1.png
60   ▷ magick a0.png a1.png -frame 5 -append 10_3.png
61

```

The classification algorithm prints a report. The report, shown below, lists the transversal of the isomorphism types. For the sake of space, the report is shown only partially. Note that the ordering of geometries in the report may be different from the ordering in the input file. This is because the classification program sorts the geometries according to the canonical form. Also, note that the report includes the incidence geometry in the original form as well as the tactical decomposition induced by the orbits of the automorphism group.

Summary of Orbits

	Rep	#	Ago	Objects
0	9	1	3	9
1	1	1	2	1
2	6	1	10	6
3	5	1	4	5
4	4	1	24	4
5	7	1	4	7
6	8	1	3	8
7	3	1	120	3
8	2	1	12	2
9	0	1	6	0

Ago :2, 3^2 , 4^2 , 6, 10, 12, 24, 120

Isomorphism type 0 / 10

Isomorphism type 0 / 10 is original object 9 and appears 1 times:

This isomorphism type appears 1 times, namely for the following 1 input objects: {9}

incidence structure:

(0, 1, 2, 10, 13, 14, 20, 25, 26, 31, 33, 35, 41, 44, 47, 52, 53, 58, 62, 66, 69, 74, 78, 79, 85, 87, 89, 96, 97, 98)

Generators for the automorphism group:

The stabilizer of order 3 is generated by:

$g_1 = (0, 1, 3)(2, 5, 4)(6, 7, 8)(10, 13, 11)(12, 14, 15)(16, 18, 17)$ of order 3 and with 2 fixed points.

Decomposition by automorphism group:

	10	13	11	12	14	15	16	18	17	19
0	□		□	□						
1	□	□			□					
3		□	□			□				
2	□					□	□			
5		□		□				□		
4			□		□				□	
6				□			□			□
7					□			□		□
8						□			□	□
9							□	□	□	

Canonical labeling:

canonical row = 5

canonical orbit number = 1

Flags : 0,1,2,16,17,18,25,27,29,34,38,39,40,43,45,51,53,56,62,63,64,70,74,77,82,86,89,91,95,98,

	16	18	17	19	15	12	14	10	13	11
9	□	□	□							
1							□	□	□	
0						□		□		□
3				□					□	□
6	□			□		□				
7		□			□		□			
8			□	□	□					
*2	□				□			□		
*4		□					□			□
*5		□				□			□	

Isomorphism type 1 / 10

Isomorphism type 1 / 10 is original object 1 and appears 1 times:

This isomorphism type appears 1 times, namely for the following 1 input objects: {1}

incidence structure:

(0, 1, 2, 10, 13, 14, 20, 25, 26, 31, 33, 35, 41, 44, 47, 52, 54, 58, 62, 66, 69, 73, 78, 79, 85, 87, 89, 96, 97, 98)

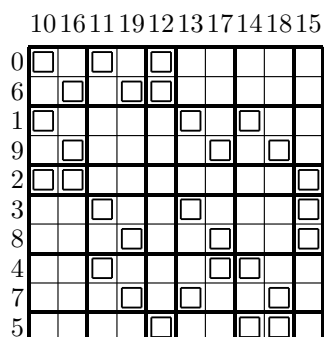
□	□	□								
□			□	□						
□					□	□				
	□		□		□					
	□			□			□			
		□		□				□		
			□				□		□	
				□					□	□
					□		□			□
						□	□			□
							□	□	□	

Generators for the automorphism group:

The stabilizer of order 2 is generated by:

$g_1 = (0, 6)(1, 9)(3, 8)(4, 7)(10, 16)(11, 19)(13, 17)(14, 18)$ of order 2 and with 4 fixed points.

Decomposition by automorphism group:

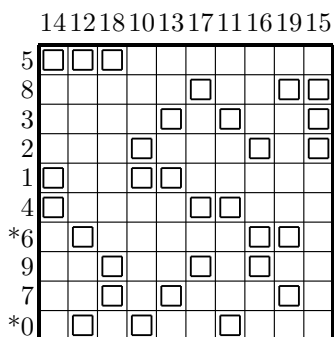


Canonical labeling:

canonical row = 0

canonical orbit number = 0

Flags : 0,1,2,15,18,19,24,26,29,33,37,39,40,43,44,50,55,56,61,67,68,72,75,77,82,84,88,91,93,96,



Besides the report, the command also produces a diagram of each geometry. The diagram indicates the orbits on flags and anti-flags. All the diagrams are pasted together, to form one large tapestry:



At times, we have large lists of geometries. It may help to filter them by the order of the automorphism group. The following example illustrates this technique. We assume that the file 15_3.inc (created in Section 12.6) is available. The first command reclassifies the geometries and filters out those whose automorphism group order is at least 72.

Code Snippet 832

```

1   geo_15_3.c_filter72:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Doinc -draw_incidence_structure_options \
4   ▷ ▷ ▷ -width 50 -width_10 5 -end \
5   ▷ ▷ -define C -combinatorial_object \
6   ▷ ▷ ▷ -label geo_15_3 geo\_15\_3 \
7   ▷ ▷ ▷ -file_of_incidence_geometries 15_3.inc 15 15 45 \
8   ▷ ▷ -end \
9   ▷ ▷ -with C -do \
10  ▷ ▷ -combinatorial_object_activity \
11  ▷ ▷ ▷ -canonical_form \
12  ▷ ▷ ▷ -save_ago \
13  ▷ ▷ ▷ -nauty_control -end \
14  ▷ ▷ ▷ -filter_ago_at_least 72 \
15  ▷ ▷ ▷ -end \
16  ▷ ▷ -end \
17

```

The next command runs the classification for the filtered geometries only:

Code Snippet 833

```

1   geo_15_3.filtered.c:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Doinc -draw_incidence_structure_options \
4   ▷ ▷ ▷ -width 50 -width_10 5 -end \
5   ▷ ▷ -define C -combinatorial_object \
6   ▷ ▷ ▷ -label geo_15_3.ago_ge72 geo\_15\_3\_ago\_ge72 \
7   ▷ ▷ ▷ -file_of_incidence_geometries 15_3.filtered.inc 15 15 45 \
8   ▷ ▷ -end \
9   ▷ ▷ -with C -do \
10  ▷ ▷ -combinatorial_object_activity \
11  ▷ ▷ ▷ -canonical_form \
12  ▷ ▷ ▷ -save_ago \
13  ▷ ▷ ▷ -nauty_control -end \
14  ▷ ▷ ▷ -end \
15  ▷ ▷ -end \
16  ▷ ▷ -with C -do \
17  ▷ ▷ -combinatorial_object_activity \
18  ▷ ▷ ▷ -post_processing \
19  ▷ ▷ -end \
20  ▷ ▷ -with C -do \
21  ▷ ▷ -combinatorial_object_activity \
22  ▷ ▷ ▷ -report \
23  ▷ ▷ ▷ ▷ -export_flag_orbits \
24  ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
25  ▷ ▷ ▷ -end \
26  ▷ ▷ -end
27  ▷ $(PDFLATEX) geo_15_3.ago_ge72_classification.tex
28  ▷ $(OPEN) geo_15_3.ago_ge72_classification.pdf
29

```

The following command computes the canonical form for the three triangle-free configurations 24_3 found by Abdullah Alazemi. These configurations have 24 points, 24 lines, each line consists of 3 points and each point is on 3 lines.

Code Snippet 834

```

1  FILE_24_3.TFC_INC="24 24 72\
2  \n0 1 2 24 27 28 48 53 54 73 79 80 97 105 106 122 131 \
3  132 146 157 158 171 175 183 195 203 208 220 225 233 244 \
4  258 259 269 272 282 293 300 308 318 325 333 342 352 358 \
5  367 379 381 392 398 400 417 428 429 442 443 450 466 471 \
6  479 492 497 502 517 519 521 542 548 551 571 574 575 \
7  48\
8  \n0 1 2 24 27 28 48 53 54 73 79 80 97 105 106 122 131 \
9  132 146 157 158 171 175 183 195 203 208 220 225 233 244 \
10 258 259 269 272 281 293 301 308 318 324 327 342 354 357 \
11 367 373 378 392 400 403 417 419 430 442 446 447 466 472 \
12 479 492 500 503 518 525 526 545 549 551 571 572 574 \
13 48\
14 \n0 1 2 24 27 28 48 53 54 73 79 80 97 105 106 122 131 \
15 132 146 157 158 171 175 179 195 201 207 220 226 232 244 \
16 257 258 269 274 277 293 300 307 318 323 329 342 352 356 \
17 367 374 381 392 397 406 416 423 431 441 450 454 468 476 \
18 477 494 499 503 519 521 525 544 547 550 570 572 575 \
19 144\
20 \n-1 3"
21
```

Code Snippet 835

```

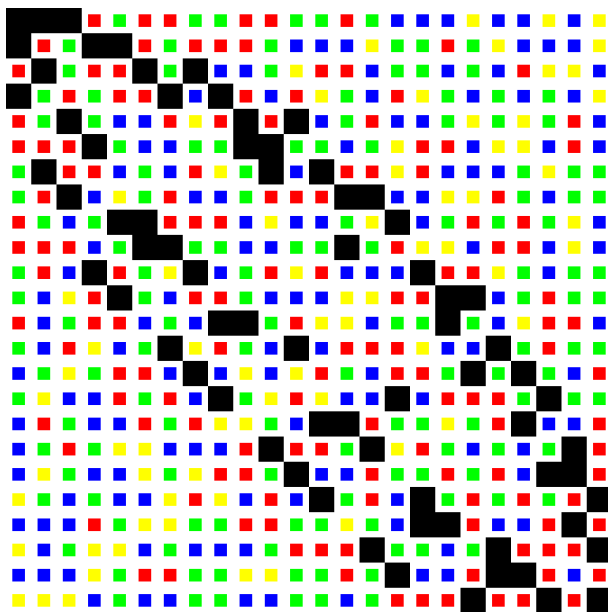
1  TFC_24_3.c:
2  ▷ echo $(FILE_24_3.TFC_INC) >24_3.TFC.inc
3  ▷ $(ORBITER) -v 20 \
4  ▷ ▷ -define C -combinatorial_object \
5  ▷ ▷ ▷ -label 24_3.TFC 24\3\TFC \
6  ▷ ▷ ▷ -file_of_incidence_geometries 24_3.TFC.inc 24 24 72 \
7  ▷ ▷ -end \
8  ▷ ▷ -with C -do \
9  ▷ ▷ -combinatorial_object_activity \
10 ▷ ▷ ▷ -canonical_form \
11 ▷ ▷ ▷ -save_ago \
12 ▷ ▷ ▷ -nauty_control -nauty_log "TFC_24_3_nauty_log.txt" -end \
13 ▷ ▷ ▷ -end \
14 ▷ ▷ -end \
15 ▷ ▷ -define Doinc -draw_incidence_structure_options \
16 ▷ ▷ ▷ -width 40 -width_10 6 \
17 ▷ ▷ -end \
18 ▷ ▷ -with C -do \
19 ▷ ▷ -combinatorial_object_activity \
20 ▷ ▷ ▷ -post_processing \
21 ▷ ▷ -end \
22 ▷ ▷ -with C -do \
23 ▷ ▷ -combinatorial_object_activity \
```

```

24  ▷ ▷ ▷ -report \
25  ▷ ▷ ▷ ▷ -export_flag_orbits \
26  ▷ ▷ ▷ ▷ -show_TDO \
27  ▷ ▷ ▷ ▷ -show_TDA \
28  ▷ ▷ ▷ ▷ -show_incidence_matrices \
29  ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
30  ▷ ▷ ▷ -end \
31  ▷ ▷ -end
32  ▷ $(PDFLATEX) 24_3_TFC_classification.tex
33  ▷ $(OPEN) 24_3_TFC_classification.pdf
34  ▷ $(ORBITER) -v 2 -draw_matrix \
35  ▷ ▷ -input_csv_file 24_3_TFC_object_2_TDA_flag_orbits.csv \
36  ▷ ▷ -secondary_input_csv_file 24_3_TFC_object_2_TDA.csv \
37  ▷ ▷ -box_width 40 -bit_depth 24 \
38  ▷ ▷ -end
39  ▷ $(OPEN) 24_3_TFC_object_2_TDA_flag_orbits_draw.bmp
40  ▷ $(ORBITER) -v 2 \
41  ▷ ▷ -define Do -draw_options -embedded -radius 150 \
42  ▷ ▷ ▷ -nodes_empty \
43  ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
44  ▷ ▷ -tree_draw \
45  ▷ ▷ ▷ -file TFC_24_3_nauty_log.txt \
46  ▷ ▷ ▷ -draw_options Do \
47  ▷ ▷ -end
48  ▷ $(PDFLATEX) TFC_24_3_nauty_log_draw.tex
49  ▷ $(OPEN) TFC_24_3_nauty_log_draw.pdf
50

```

The command also computes the tactical decomposition induced by the automorphism group. In addition, the command also computes the orbits on flags and on anti-flags. The third of the three geometries is flag transitive. A bitmap drawing is produced, shown below:



The geometry is flag transitive, so all incidences are drawn in the same color (black).

The next command produces a diagram of the flag orbits for each of the twelve Latin squares of order 6.

Code Snippet 836

```

1   geo.LSQ6_c:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Test_lines -set -loop 1 19 1 -end \
4   ▷ ▷ -define Geo -geometry_builder \
5   ▷ ▷ ▷ -V 6,6,6 -B 1,1,1,36 -TDO \
6   ▷ ▷ ▷ "1,0,0,6, 0,1,0,6, 0,0,1,6" \
7   ▷ ▷ ▷ -fuse 3 \
8   ▷ ▷ ▷ -fname_GEO LSQ6 \
9   ▷ ▷ ▷ -output_to_inc_file \
10  ▷ ▷ ▷ -test Test_lines \
11  ▷ ▷ -end
12  ▷ $(ORBITER) -v 3 \
13  ▷ ▷ -define Doinc -draw_incidence_structure_options \
14  ▷ ▷ ▷ -width 60 -width_10 6 -end \
15  ▷ ▷ -define C -combinatorial_object \
16  ▷ ▷ ▷ -label LSQ6 LSQ6 \
17  ▷ ▷ ▷ -file_of_incidence_geometries \
18  ▷ ▷ ▷ ▷ LSQ6.inc 18 39 126 \
19  ▷ ▷ -end \
20  ▷ ▷ -with C -do \
21  ▷ ▷ -combinatorial_object_activity \
22  ▷ ▷ ▷ -canonical_form \
23  ▷ ▷ ▷ ▷ -save_ago \
24  ▷ ▷ ▷ ▷ -save_transversal \
25  ▷ ▷ ▷ ▷ -nauty_control -nauty_log "LSQ6.nauty.log.txt" -end \
26  ▷ ▷ ▷ -end \
27  ▷ ▷ -end \
28  ▷ ▷ -with C -do \
29  ▷ ▷ -combinatorial_object_activity \
30  ▷ ▷ ▷ -post_processing \
31  ▷ ▷ -end \
32  ▷ ▷ -with C -do \
33  ▷ ▷ -combinatorial_object_activity \
34  ▷ ▷ ▷ -report \
35  ▷ ▷ ▷ ▷ -export_flag_orbits \
36  ▷ ▷ ▷ ▷ -show_incidence_matrices \
37  ▷ ▷ ▷ ▷ -export_group_GAP \
38  ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
39  ▷ ▷ ▷ -end \
40  ▷ ▷ -end
41  ▷ $(PDFLATEX) LSQ6_classification.tex
42  ▷ $(OPEN) LSQ6_classification.pdf
43  ▷ $(ORBITER) -v 2 \
44  ▷ ▷ -loop L 0 12 1 \
45  ▷ ▷ ▷ -draw_matrix \
46  ▷ ▷ ▷ -input_csv_file LSQ6_object.%L_TDA_flag_orbits.csv \
47  ▷ ▷ ▷ -secondary_input_csv_file LSQ6_object.%L_TDA.csv \

```

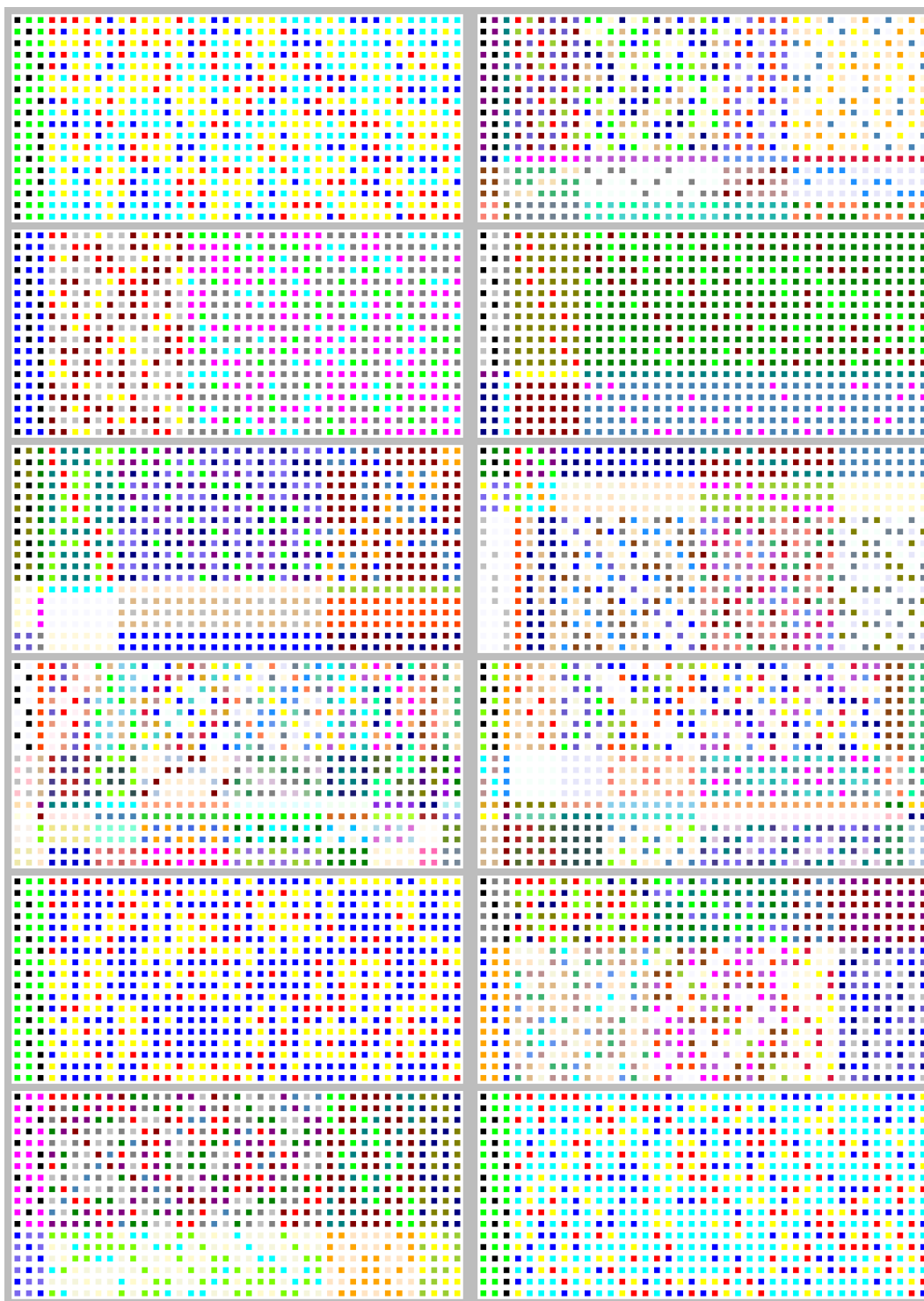


```

48  ▷ ▷ ▷ -box_width 32 -bit_depth 24 \
49  ▷ ▷ ▷ -end \
50  ▷ ▷ ▷ -system "magick \
51  ▷ ▷ ▷ ▷ LSQ6_object_%L_TDA_flag_orbits_draw.bmp \
52  ▷ ▷ ▷ ▷ LSQ6_object_%L_TDA_flag_orbits_draw.png" \
53  ▷ ▷ -end_loop L
54  ▷ $(ORBITER) -v 2 \
55  ▷ ▷ -loop L 0 12 1 \
56  ▷ ▷ ▷ -draw_matrix \
57  ▷ ▷ ▷ -input_csv_file LSQ6_object_%L_INP_flag_orbits.csv \
58  ▷ ▷ ▷ -secondary_input_csv_file LSQ6_object_%L_INP.csv \
59  ▷ ▷ ▷ -box_width 32 -bit_depth 24 \
60  ▷ ▷ ▷ -end \
61  ▷ ▷ -end_loop L
62  ▷ magick LSQ6_object_[0-5]_TDA_flag_orbits_draw.bmp -frame 10 -append a0.png
63  ▷ magick LSQ6_object_[6-9]_TDA_flag_orbits_draw.bmp \
64  ▷ ▷ LSQ6_object_10_TDA_flag_orbits_draw.bmp \
65  ▷ ▷ LSQ6_object_11_TDA_flag_orbits_draw.bmp \
66  ▷ ▷ -frame 10 -append a1.png
67  ▷ magick a0.png a1.png -frame 10 +append LSQ6.png
68  ▷ cp LSQ6.png ~/TEX.26/ORBITER_USERS_GUIDE/GRAPHICS/LSQ_6
69

```

The output is shown below



17.4 Canonical Forms of Objects from Design Theory

In Section 14.1, we discussed how to create designs in Orbiter. In this section, we wish to compute properties of designs related to the automorphism group. The first step is to export the incidence matrix of the design to file. After that, we compute the canonical form of the design, which allows us to determine many properties. The following example computes the properties of $\text{PG}(2, 3)$:

Code Snippet 837

```

1  design_PG_2_3_canonical:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define F -finite_field -q 3 -end \
4  ▷ ▷ -define D -design -field F -family PG_2_q -end \
5  ▷ ▷ -with D -do \
6  ▷ ▷ ▷ -design_activity \
7  ▷ ▷ ▷ ▷ -export_incidence_matrix_as_flags \
8  ▷ ▷ -end
9  ▷ $(ORBITER) -v 3 \
10 ▷ ▷ -define Doinc -draw_incidence_structure_options \
11 ▷ ▷ ▷ -width 60 -width_10 6 -end \
12 ▷ ▷ -define C -combinatorial_object \
13 ▷ ▷ ▷ -label PG_2_3 PG\_2\_3 \
14 ▷ ▷ ▷ -file_of_incidence_geometries_csv \
15 ▷ ▷ ▷ ▷ PG_2_q3_flags.csv flags 13 13 52 \
16 ▷ ▷ -end \
17 ▷ ▷ -with C -do \
18 ▷ ▷ -combinatorial_object_activity \
19 ▷ ▷ ▷ -canonical_form \
20 ▷ ▷ ▷ ▷ -save_ago \
21 ▷ ▷ ▷ ▷ -save_transversal \
22 ▷ ▷ ▷ ▷ -nauty_control -end \
23 ▷ ▷ ▷ -end \
24 ▷ ▷ -end \
25 ▷ ▷ -with C -do \
26 ▷ ▷ -combinatorial_object_activity \
27 ▷ ▷ ▷ -post_processing \
28 ▷ ▷ -end \
29 ▷ ▷ -with C -do \
30 ▷ ▷ -combinatorial_object_activity \
31 ▷ ▷ ▷ -report \
32 ▷ ▷ ▷ ▷ -export_flag_orbits \
33 ▷ ▷ ▷ ▷ -show_incidence_matrices \
34 ▷ ▷ ▷ ▷ -export_group_GAP \
35 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
36 ▷ ▷ ▷ -end \
37 ▷ ▷ -end
38 ▷ $(PDFLATEX) PG_2_3_classification.tex
39 ▷ $(OPEN) PG_2_3_classification.pdf
40 ▷ $(ORBITER) -v 2 -draw_matrix \
41 ▷ ▷ -input_csv_file PG_2_3_object_0_TDA_flag_orbits.csv \
42 ▷ ▷ -secondary_input_csv_file PG_2_3_object_0_TDA.csv \
43 ▷ ▷ -box_width 32 -bit_depth 24 \
44 ▷ ▷ -end

```

```

45 ▷ $(OPEN) PG_2.3_object_0_TDA_flag_orbits.draw.bmp
46 ▷
47

```

The command

Code Snippet 838

```

1 wreath_product_designs.n4.k2.c:
2 ▷ $(ORBITER) -v 8 \
3 ▷ ▷ -define D -design -wreath_product_designs 4 2 -end \
4 ▷ ▷ -with D -do \
5 ▷ ▷ ▷ -design_activity \
6 ▷ ▷ ▷ ▷ -export_incidence_matrix_as_flags \
7 ▷ ▷ -end
8 ▷ $(ORBITER) -v 10 \
9 ▷ ▷ -define Doinc -draw_incidence_structure_options \
10 ▷ ▷ ▷ -width 60 -width_10 6 -end \
11 ▷ ▷ -define C -combinatorial_object \
12 ▷ ▷ ▷ -label wreath_4_2 wreath\4\2 \
13 ▷ ▷ ▷ -file_of_incidence_geometries \
14 ▷ ▷ ▷ wreath_product_designs.n4.k2_flags.txt \
15 ▷ ▷ ▷ 8 12 24 \
16 ▷ ▷ -end \
17 ▷ ▷ -with C -do \
18 ▷ ▷ -combinatorial_object_activity \
19 ▷ ▷ ▷ -canonical_form \
20 ▷ ▷ ▷ ▷ -save_ago \
21 ▷ ▷ ▷ ▷ -save_transversal \
22 ▷ ▷ ▷ ▷ -nauty_control -end \
23 ▷ ▷ ▷ -end \
24 ▷ ▷ -end \
25 ▷ ▷ -with C -do \
26 ▷ ▷ -combinatorial_object_activity \
27 ▷ ▷ ▷ -post_processing \
28 ▷ ▷ -end \
29 ▷ ▷ -with C -do \
30 ▷ ▷ -combinatorial_object_activity \
31 ▷ ▷ ▷ -report \
32 ▷ ▷ ▷ ▷ -export_flag_orbits \
33 ▷ ▷ ▷ ▷ -show_incidence_matrices \
34 ▷ ▷ ▷ ▷ -export_group_GAP \
35 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
36 ▷ ▷ ▷ -end \
37 ▷ ▷ -end
38 ▷ $(PDFLATEX) wreath_4_2_classification.tex
39 ▷ $(OPEN) wreath_4_2_classification.pdf
40

```

computes the automorphism group of the design on 8 points created in Section 14.1. The group is $\text{Sym}(4) \wr \text{Sym}(2)$. The command

Code Snippet 839

```

1    wreath_product_designs_n8_k6.c:
2    ▷ $(ORBITER) -v 8 \
3    ▷ ▷ -define D -design -wreath_product_designs 8 6 -end \
4    ▷ ▷ -with D -do \
5    ▷ ▷ ▷ -design_activity \
6    ▷ ▷ ▷ ▷ -export_incidence_matrix_as_flags \
7    ▷ ▷ -end
8    ▷ $(ORBITER) -v 10 \
9    ▷ ▷ -define Doinc -draw_incidence_structure_options \
10   ▷ ▷ ▷ -width 60 -width_10 6 -end \
11   ▷ ▷ -define C -combinatorial_object \
12   ▷ ▷ ▷ -label wreath_8_6 wreath\_8\_6 \
13   ▷ ▷ ▷ -file_of_incidence_geometries \
14   ▷ ▷ ▷ wreath_product_designs_n8_k6_flags.txt \
15   ▷ ▷ ▷ 16 3920 23520 \
16   ▷ ▷ -end \
17   ▷ ▷ -with C -do \
18   ▷ ▷ -combinatorial_object_activity \
19   ▷ ▷ ▷ -canonical_form \
20   ▷ ▷ ▷ ▷ -save_ago \
21   ▷ ▷ ▷ ▷ -save_transversal \
22   ▷ ▷ ▷ ▷ -nauty_control -end \
23   ▷ ▷ ▷ -end \
24   ▷ ▷ -end \
25   ▷ ▷ -with C -do \
26   ▷ ▷ -combinatorial_object_activity \
27   ▷ ▷ ▷ -post_processing \
28   ▷ ▷ -end \
29   ▷ ▷ -with C -do \
30   ▷ ▷ -combinatorial_object_activity \
31   ▷ ▷ ▷ -report \
32   ▷ ▷ ▷ ▷ -export_flag_orbits \
33   ▷ ▷ ▷ ▷ -export_group_GAP \
34   ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
35   ▷ ▷ ▷ -end \
36   ▷ ▷ -end
37   ▷ $(PDFLATEX) wreath_8_6_classification.tex
38   ▷ $(OPEN) wreath_8_6_classification.pdf
39

```

computes the automorphism group of the design on 16 points created in Section 14.1. The group is $\text{Sym}(8) \wr \text{Sym}(2)$.

In Section 14.3, the large sets of $\text{AG}(2, 3)$ were constructed. However, the isomorphism classification of the created objects is still outstanding. We will do this final step now. The isomorphism classification is performed using the Nauty interface. A list of combinatorial objects is created, and the `-canonical_form` command is applied as activity. This will produce a list of pairwise non-isomorphic designs. The size of this list is the number of isomorphism types of large sets of $\text{AG}(2, 3)$.

Code Snippet 840

```

1   LS_AG_2_3_solutions_classify:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Doinc -draw_incidence_structure_options \
4   ▷ ▷ ▷ -width 20 -width_10 2 -end \
5   ▷ ▷ -define C -combinatorial_object \
6   ▷ ▷ ▷ -label LS_AG_2_3 LS\AG\2\3 \
7   ▷ ▷ ▷ -file_of_designs \
8   ▷ ▷ ▷ AG_2_3_large_sets.csv large_set 9 84 3 12 \
9   ▷ ▷ -end \
10  ▷ ▷ -with C -do \
11  ▷ ▷ -combinatorial_object_activity \
12  ▷ ▷ ▷ -canonical_form \
13  ▷ ▷ ▷ ▷ -save_ago \
14  ▷ ▷ ▷ ▷ -save_transversal \
15  ▷ ▷ ▷ ▷ -max_TDO_depth 10 \
16  ▷ ▷ ▷ ▷ -nauty_control -end \
17  ▷ ▷ ▷ -end \
18  ▷ ▷ -end \
19  ▷ ▷ -with C -do \
20  ▷ ▷ -combinatorial_object_activity \
21  ▷ ▷ ▷ -post_processing \
22  ▷ ▷ -end \
23  ▷ ▷ -with C -do \
24  ▷ ▷ -combinatorial_object_activity \
25  ▷ ▷ ▷ -report \
26  ▷ ▷ ▷ ▷ -export_flag_orbits \
27  ▷ ▷ ▷ ▷ -show_TDO \
28  ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
29  ▷ ▷ ▷ -end \
30  ▷ ▷ -end
31  ▷ $(PDFLATEX) LS_AG_2_3_classification.tex
32  ▷ $(OPEN) LS_AG_2_3_classification.pdf
33  ▷ $(ORBITER) -v 2 -draw_matrix \
34  ▷ ▷ -input_csv_file LS_AG_2_3_object_0_INP_flag_orbits.csv \
35  ▷ ▷ -secondary_input_csv_file LS_AG_2_3_object_0_INP.csv \
36  ▷ ▷ -box_width 12 -bit_depth 24 \
37  ▷ ▷ -end
38  ▷ $(OPEN) LS_AG_2_3_object_0_INP_flag_orbits_draw.bmp
39  ▷ $(ORBITER) -v 2 -draw_matrix \
40  ▷ ▷ -input_csv_file LS_AG_2_3_object_1_INP_flag_orbits.csv \
41  ▷ ▷ -secondary_input_csv_file LS_AG_2_3_object_1_INP.csv \
42  ▷ ▷ -box_width 12 -bit_depth 24 \
43  ▷ ▷ -end
44  ▷ $(OPEN) LS_AG_2_3_object_1_INP_flag_orbits_draw.bmp
45

```

It turns out that there are exactly two isomorphism types, with automorphism groups of order 54 and 42, respectively.

17.5 Canonical Forms of Linear Codes

Using Nauty, Orbiter can compute canonical forms and automorphism groups of codes. For linear codes, the semilinear automorphism group can be computed.

Consider the $[3, 2, 2]$ code generated by

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

Using the following command, the semilinear automorphism group of the code can be computed:

Code Snippet 841

```

1   code_3_2.aut:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 2 -end \
4   ▷ ▷ -define genma -vector -field F -format 2 \
5   ▷ ▷ ▷ -dense $(CODE_N3_K2_Q2_GENMA) \
6   ▷ ▷ -end \
7   ▷ ▷ -define P -projective_space -n 1 -field F -v 0 -end \
8   ▷ ▷ -with P -do \
9   ▷ ▷ -projective_space_activity \
10  ▷ ▷ ▷ -canonical_form_of_code \
11  ▷ ▷ ▷ ▷ "3_2" genma \
12  ▷ ▷ ▷ ▷ -save_ago \
13  ▷ ▷ ▷ ▷ -save_classification "3_2" \
14  ▷ ▷ ▷ ▷ -nauty_control -end \
15  ▷ ▷ ▷ -end \
16  ▷ ▷ -end
17  ▷ $(PDFLATEX) 3_2_classification.tex
18  ▷ $(OPEN) 3_2_classification.pdf
19  ▷ $(ORBITER) -v 2 -draw_matrix \
20  ▷ ▷ -input_csv_file 3_2_object0_TDA_flag_orbits.csv \
21  ▷ ▷ -secondary_input_csv_file 3_2_object0_TDA.csv \
22  ▷ ▷ -box_width 16 -bit_depth 24 \
23  ▷ ▷ -end
24  ▷ $(OPEN) 3_2_object0_TDA_flag_orbits_draw.bmp
25

```

The code has a semilinear automorphism group of order 6. The following report is written:

Summary of Orbits

	Rep	#	Ago	Objects
0	0	1	6	0

Ago : 6

Isomorphism type 0 / 1

Isomorphism type 0 / 1 is original object 0 and appears 1 times:

This isomorphism type appears 1 times, namely for the following 1 input objects: $\{0\}$
set of points of size 3: $(0, 1, 2)$

i	Rank	Point
0	0	(1, 0)
1	1	(0, 1)
2	2	(1, 1)

Column sets of the encoded object:

$\{0, 1, 2\}$

$\{0, 1, 2, 3\}$

Row sets of the encoded object:

$\{0, 1\} = 0$

$\{0, 1\} = 0$

$\{0, 1\} = 0$

$\{1\} = 1$

0	0
0	0
0	0
0	0
0	0
1	1

Generators for the automorphism group:

The stabilizer of order 6 is generated by:

$g_1 = (1, 2)$ of order 2 and with 4 fixed points.

$g_2 = (0, 1)$ of order 2 and with 4 fixed points.

Generators for the automorphism group as matrix group:

The stabilizer of order 6 is generated by:

$$g_1 = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 10 \\ 11 \end{bmatrix} \text{ of order 2 and with 1 fixed points.}$$

$$g_2 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 01 \\ 10 \end{bmatrix} \text{ of order 2 and with 1 fixed points.}$$

Decomposition by combinatorial refinement:

$$\begin{array}{c|c} \rightarrow & 2_1 \\ \hline 4_0 & 2 \end{array}$$

$$\begin{array}{c|c} \downarrow & 2_1 \\ \hline 4_0 & 3 \end{array}$$

Decomposition by automorphism group:

4	5
0	0
1	0
2	0
3	0
3	1

Canonical labeling:

canonical row = 3

canonical orbit number = 1

Flags : $(0, 1, 2, 3, 4, 5, 7)$

$\begin{array}{cc} & 4 \ 5 \\ 0 & \begin{array}{|c|c|} \hline \square & \square \\ \hline \end{array} \\ 1 & \begin{array}{|c|c|} \hline \square & \square \\ \hline \end{array} \\ 2 & \begin{array}{|c|c|} \hline \square & \square \\ \hline \end{array} \\ *3 & \begin{array}{|c|c|} \hline \square & \square \\ \hline \end{array} \end{array}$

Flag orbits:

orbit length : number of orbits of that length:

$\begin{array}{cc} 1 & 1 \\ 3 & 2 \end{array}$

Anti-Flag orbits:

orbit length : number of orbits of that length:

$\begin{array}{cc} 1 & 1 \end{array}$

The command

Code Snippet 842

```

1  CODE_RM_3_1_GENMA="\
2  11111111\
3  01010101\
4  00110011\
5  00001111"
6

```

Code Snippet 843

```

1  RM_3_1_group:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 2 -end \
4  ▷ ▷ -define genma -vector -field F -format 4 \
5  ▷ ▷ ▷ -compact $(CODE_RM_3_1_GENMA) \
6  ▷ ▷ -end \
7  ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
8  ▷ ▷ -with P -do \
9  ▷ ▷ -projective_space_activity \
10 ▷ ▷ ▷ -canonical_form_of_code \
11 ▷ ▷ ▷ ▷ "RM_3_1" genma \
12 ▷ ▷ ▷ ▷ -save_ago \
13 ▷ ▷ ▷ ▷ -save_classification "RM_3_1" \
14 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "RM_3_1.nauty_log.txt" -end \
15 ▷ ▷ ▷ -end \
16 ▷ ▷ -end
17 ▷ $(PDFLATEX) RM_3_1_classification.tex
18 ▷ $(OPEN) RM_3_1_classification.pdf
19

```

computes the automorphism group of the Reed-Muller code. The group is an affine linear group $AGL(3, 2)$ of order 1344. A report is created, showing the automorphism group and the action on $PG(3, 2)$, with the Reed-Muller code distinguished.

The following command creates a drawing of the incidence matrix between points and lines in $\text{PG}(3, 2)$, with the Reed-Muller code distinguished:

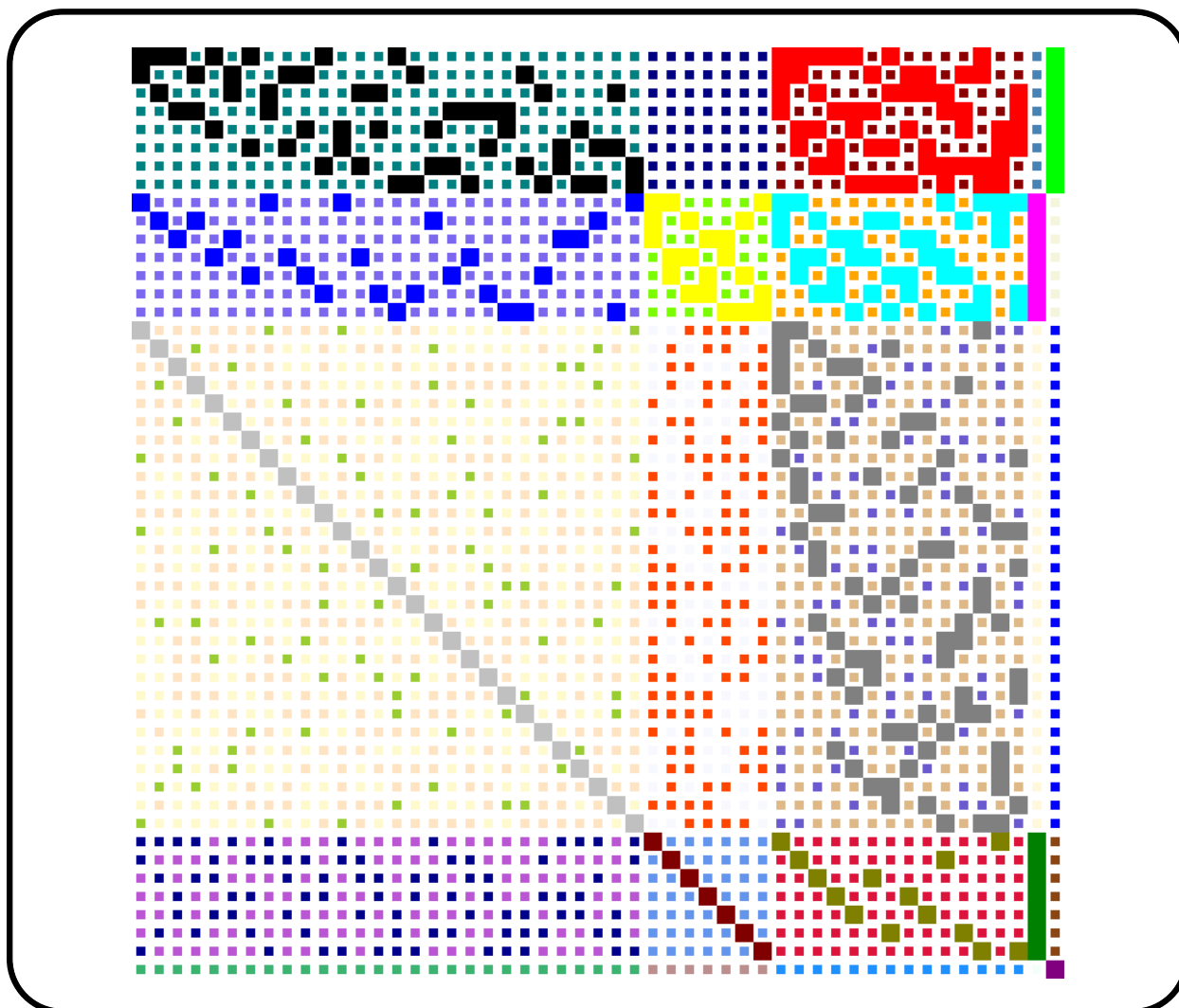
Code Snippet 844

```

1   RM_3_1_group_and_diagram:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 2 -end \
4   ▷ ▷ -define genma -vector -field F -format 4 \
5   ▷ ▷ ▷ -compact $(CODE_RM_3_1_GENMA) \
6   ▷ ▷ -end \
7   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
8   ▷ ▷ -with P -do \
9   ▷ ▷ -projective_space_activity \
10  ▷ ▷ ▷ -canonical_form_of_code \
11  ▷ ▷ ▷ ▷ "RM_3_1" genma \
12  ▷ ▷ ▷ ▷ -save_ago \
13  ▷ ▷ ▷ ▷ -save_classification "RM_3_1" \
14  ▷ ▷ ▷ ▷ -nauty_control -end \
15  ▷ ▷ ▷ -end \
16  ▷ ▷ -end
17  ▷ $(PDFLATEX) RM_3_1_classification.tex
18  ▷ $(OPEN) RM_3_1_classification.pdf
19  ▷ $(ORBITER) -v 2 -draw_matrix \
20  ▷ ▷ -input_csv_file RM_3_1_object0_INP_flag_orbits.csv \
21  ▷ ▷ -secondary_input_csv_file RM_3_1_object0_INP.csv \
22  ▷ ▷ -box_width 16 -bit_depth 24 \
23  ▷ ▷ -end
24  ▷ $(ORBITER) -v 2 -draw_matrix \
25  ▷ ▷ -input_csv_file RM_3_1_object0_TDA_flag_orbits.csv \
26  ▷ ▷ -secondary_input_csv_file RM_3_1_object0_TDA.csv \
27  ▷ ▷ -box_width 16 -bit_depth 24 \
28  ▷ ▷ -end
29  ▷ $(OPEN) RM_3_1_object0_INP_flag_orbits_draw.bmp
30  ▷ $(OPEN) RM_3_1_object0_TDA_flag_orbits_draw.bmp
31

```

The drawing is shown below:



The command

Code Snippet 845

```

1  RS_6.4.7_group:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define F -finite_field -q 7 -end \
4  ▷ ▷ -define genma -vector -field F -format 4 \
5  ▷ ▷ ▷ -compact $(CODE_RS_6.4.7) \
6  ▷ ▷ -end \
7  ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
8  ▷ ▷ -with P -do \
9  ▷ ▷ -projective_space_activity \
10 ▷ ▷ ▷ -canonical_form_of_code \
11 ▷ ▷ ▷ ▷ "RS_6" genma \
12 ▷ ▷ ▷ ▷ -save_ago \
13 ▷ ▷ ▷ ▷ -save_classification "RS_6" \
14 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "RS_6.4.7_nauty_log.txt" -end \
15 ▷ ▷ ▷ -end \

```

```

16    ▷ ▷ -end
17

```

shows that the automorphism group has order 12. After some shortening, the output is:

Isomorphism type 0 / 1 is original object 0 and appears 1 times:
 set of points of size 6: $\{(0, 9, 51, 344, 253, 3)\}$

i	Rank	Point
0	0	(1, 0, 0, 0)
1	9	(5, 1, 0, 0)
2	51	(6, 5, 1, 0)
3	344	(0, 6, 5, 1)
4	253	(0, 0, 4, 1)
5	3	(0, 0, 0, 1)

Group order 12

This isomorphism type appears 1 times, namely for the following 1 input objects: $\{0\}$

Stabilizer:

Strong generators for a group of order 12:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 2 & 0 & 0 & 1 \\ 5 & 0 & 6 & 0 \\ 5 & 1 & 0 & 0 \end{bmatrix},$$

$$\begin{bmatrix} 0 & 0 & 0 & 3 \\ 4 & 0 & 0 & 6 \\ 0 & 4 & 0 & 1 \\ 0 & 0 & 4 & 1 \end{bmatrix}$$

1,0,0,0,2,0,0,1,5,0,6,0,5,1,0,0,
 0,0,0,1,6,0,0,2,0,6,0,5,0,0,6,5,

$$\begin{array}{c|cc} \rightarrow & 2850_1 & 1_2 \\ \hline 401_0 & 57 & 1 \end{array}$$

The command

Code Snippet 846

```

1    GV_n15_k6.d5.group:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define F -finite_field -q 2 -end \
4    ▷ ▷ -define genma -vector -field F -format 6 \
5    ▷ ▷ ▷ -compact $(CODE_GV_N15_K6) \
6    ▷ ▷ -end \

```

```
7    ▷ ▷ -define P -projective_space -n 5 -field F -v 0 -end \
8    ▷ ▷ -with P -do \
9    ▷ ▷ -projective_space_activity \
10   ▷ ▷ ▷ -canonical_form_of_code \
11   ▷ ▷ ▷ ▷ "GV_n15_k6_d5" genma \
12   ▷ ▷ ▷ ▷ -save_ago \
13   ▷ ▷ ▷ ▷ -save_classification "GV_n15_k6_d5" \
14   ▷ ▷ ▷ ▷ -nauty_control -end \
15   ▷ ▷ ▷ -end \
16   ▷ ▷ -end
17   ▷ $(PDFLATEX) GV_n15_k6_d5_classification.tex
18   ▷ $(OPEN) GV_n15_k6_d5_classification.pdf
19
```

computes the automorphism group of the Gilbert-Varshamov code from Section 11.9. It has order 12.

17.6 Canonical Forms of General Codes

The command

Code Snippet 847

```
1  HAMMING_CODE_CODEWORDS="0, 67, 37, 102, 22, 85, \
2  51, 112, 15, 76, 42, 105, 25, 90, 60, 127"
3
```

Code Snippet 848

```
1  Hamming_graph_7_with_Hamming_code:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define G -graph -Hamming 7 2 \
4  ▷ ▷ ▷ -subset "_Hamming_code" "\\_with\\_Hamming\\_code" \
5  ▷ ▷ ▷ $(HAMMING_CODE_CODEWORDS) -end \
6  ▷ ▷ -with G -do \
7  ▷ ▷ -graph_theoretic_activity -export_csv -end \
8  ▷ ▷ -with G -do \
9  ▷ ▷ -graph_theoretic_activity -export_graphviz -end \
10 ▷ ▷ -with G -do \
11 ▷ ▷ -graph_theoretic_activity -save -end \
12 ▷ ▷ -with G -do \
13 ▷ ▷ -graph_theoretic_activity -automorphism_group -end
14 ▷ $(PDFLATEX) Hamming_7_2_Hamming_code_report.tex
15 ▷ $(OPEN) Hamming_7_2_Hamming_code_report.pdf
16
```

computes the set stabilizer of the Hamming code inside the automorphism group of the Hamming graph. The group has order $2688 = 16 \cdot 168$.

17.7 Canonical Forms of Graphs

Orbiter can compute isomorphism and automorphism between graphs. Here are some examples.

Suppose we want to compute the automorphism group of the cycle graph of order 13:

Code Snippet 849

```
1  Cycle_13.aut:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define Gamma -graph -cycle 13 -end \
4  ▷ ▷ -with Gamma -do \
5  ▷ ▷ ▷ -graph_theoretic_activity -automorphism_group \
6  ▷ ▷ -end
7  ▷ $(PDFLATEX) Cycle_13_report.tex
8  ▷ $(OPEN) Cycle_13_report.pdf
9
```

The output is two files: The first one, `Cycle_13_group.makefile` is a makefile containing an Orbiter command to create the automorphism group: The second file is `Cycle_13_gens.csv`, which contains the permutation representation of the group, and which is needed for the makefile.

The next command computes the automorphism group of the chain graph with respect to the partition $(2, 3, 2)$.

Code Snippet 850

```
1  Chain_232.aut:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -define P1 -vector -dense 2,3,2 -end \
4  ▷ ▷ -define P2 -vector -dense 2,3,2 -end \
5  ▷ ▷ -define Gamma -graph \
6  ▷ ▷ ▷ -chain_graph P1 P2 \
7  ▷ ▷ -end \
8  ▷ ▷ -with Gamma -do \
9  ▷ ▷ ▷ -graph_theoretic_activity -automorphism_group \
10 ▷ ▷ -end
11 ▷ $(PDFLATEX) chain_graph_report.tex
12 ▷ $(OPEN) chain_graph_report.pdf
13
```

The following report is written:

The automorphism group of *chain_graph* has order 1152 and is generated by:
Strong generators for a group of order 1152:

(12, 13),
(3, 4),
(2, 3),
(10, 11),
(9, 10),

```

(5, 6),
(7, 8),
(0, 1),
(0, 12)(1, 13)(2, 9)(3, 10)(4, 11)(5, 7)(6, 8)
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 13, 12, 14,
0, 1, 2, 4, 3, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,
0, 1, 3, 2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 10, 12, 13, 14,
0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 9, 11, 12, 13, 14,
0, 1, 2, 3, 4, 6, 5, 7, 8, 9, 10, 11, 12, 13, 14,
0, 1, 2, 3, 4, 5, 6, 8, 7, 9, 10, 11, 12, 13, 14,
1, 0, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,
12, 13, 9, 10, 11, 7, 8, 5, 6, 2, 3, 4, 0, 1, 14,

```

Junttila and Kaski maintain a collection of graphs that can be used as test cases. The graphs are stored in Dimacs format and can be read in through the Orbiter `-load_dimacs` command. For instance, the following command computes the automorphism group of the Levi graph of the desarguesian projective plane of order 16:

Code Snippet 851

```

1   JK_graph_pp16_1:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define Gamma -graph -load_dimacs \
4   ▷ ▷ ../JUNTILLA_KASKI/benchmarks/pp/pp16-1 \
5   ▷ ▷ -end \
6   ▷ ▷ -with Gamma -do \
7   ▷ ▷ -graph_theoretic_activity -save -end \
8   ▷ ▷ -with Gamma -do \
9   ▷ ▷ -graph_theoretic_activity -automorphism_group -end \
10

```

The command shows a group of order 34217164800. As a measurement of the complexity, the number of backtrack nodes in Nauty is recorded:

```

nb_backtrack1 = 6
nb_backtrack2 = 134
nb_backtrack3 = 134
nb_backtrack4 = 1

```

Here, `nb_backtrack1` is the number of calls to `firstpathnode`, `nb_backtrack2` is the number of calls to `othernode`, `nb_backtrack3` is the number of calls to `processnode`, `nb_backtrack4` is the number of calls to `firstterminal`. These are the four recursive functions in Nauty.

Unfortunately, the complexity of graph isomorphism is not well-understood. We can see this here. While the first projective plane of order 16 can be handled relatively easily, the second one causes problems. The following command hardly finishes:

Code Snippet 852

```

1    JK_graph_pp16_2:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define Gamma -graph -load_dimacs \
4    ▷ ▷ ../JUNTTILA_KASKI/benchmarks/pp/pp16-2 \
5    ▷ ▷ -end \
6    ▷ ▷ -with Gamma -do \
7    ▷ ▷ -graph_theoretic_activity -save -end \
8    ▷ ▷ -with Gamma -do \
9    ▷ ▷ -graph_theoretic_activity -automorphism_group -end \
10

```

The difference between the two planes is that the first plane has a very large automorphism group, while the second one has not. For any q , the Desarguesian plane $\text{PG}(2, q)$ has the largest automorphism group of all projective planes of order q .

The following example considers the block intersection graph of a Steiner triple system (“STS”) of order 13. There are exactly two $\text{STS}(13)$. The one we consider here has a group of order 39. The block intersection graph has the same automorphism group.

Code Snippet 853

```

1    JK_graph_sts_13:
2    ▷ $(ORBITER) -v 2 \
3    ▷ ▷ -define Gamma -graph -load_dimacs \
4    ▷ ▷ ▷ ../JUNTTILA_KASKI/benchmarks/srg/sts-13 \
5    ▷ ▷ -end \
6    ▷ ▷ -with Gamma -do \
7    ▷ ▷ -graph_theoretic_activity -save -end \
8    ▷ ▷ -with Gamma -do \
9    ▷ ▷ -graph_theoretic_activity -automorphism_group -end
10    ▷ make ORBITER_PATH=$(ORBITER_EXE_PATH) -f sts-13-group.makefile sts-13
11

```

The automorphism group has order 39 and is generated by:

```

(
(1, 25, 16)(2, 18, 20)(3, 7, 15)(4, 13, 11)(5, 6, 17)
(9, 12, 19)(10, 14, 24)(21, 23, 22),
(0, 1, 2)(3, 4, 5)(7, 8, 9)(11, 12, 13)(14, 16, 18)
(15, 17, 19)(20, 22, 24)(21, 23, 25))

```

Graphs can be created from groups by means of orbitals. An orbital is an orbit of a permutation group G on the set of pairs. Here is an example. We start from the Coxeter-Tits graph on 315 vertices, whose automorphism group is the Hall-Janko group $J_2 : 2$. We first read the graph from file `halljanko315.csv` and compute the automorphism group using Nauty:

Code Snippet 854

```

1  HJ_group:
2  ▷ cp $(ORBITER_PATH)/examples/users_guide/halljanko315.csv .
3  ▷ $(ORBITER) -v 6 \
4  ▷ ▷ -define G -graph \
5  ▷ ▷ ▷ -load_csv_no_border \
6  ▷ ▷ ▷ halljanko315.csv \
7  ▷ ▷ -end \
8  ▷ ▷ -with G -do \
9  ▷ ▷ ▷ -graph_theoretic_activity -automorphism_group \
10 ▷ ▷ -end \
11 ▷ ▷ -with G -do \
12 ▷ ▷ ▷ -graph_theoretic_activity -properties \
13 ▷ ▷ -end
14 ▷ $(PDFLATEX) File_halljanko315_report.tex
15 ▷ $(OPEN) File_halljanko315_report.pdf
16

```

The next step is to compute the orbits of the automorphism group on pairs, using the following command:

Code Snippet 855

```

1  HJ_group_and_orbits:
2  ▷ cp $(ORBITER_PATH)/examples/users_guide/halljanko315.csv .
3  ▷ $(ORBITER) -v 6 \
4  ▷ ▷ -define G -graph \
5  ▷ ▷ ▷ -load_csv_no_border \
6  ▷ ▷ ▷ halljanko315.csv \
7  ▷ ▷ -end \
8  ▷ ▷ -with G -do \
9  ▷ ▷ ▷ -graph_theoretic_activity -automorphism_group \
10 ▷ ▷ -end \
11 ▷ ▷ -with G -do \
12 ▷ ▷ ▷ -graph_theoretic_activity -properties \
13 ▷ ▷ -end
14 ▷ $(ORBITER) -v 2 \
15 ▷ ▷ -define Control -poset_classification_control \
16 ▷ ▷ ▷ -W \
17 ▷ ▷ ▷ -problem_label HJ_orbits \
18 ▷ ▷ ▷ -depth 2 \
19 ▷ ▷ -end \
20 ▷ ▷ -define gens -vector -file \
21 ▷ ▷ ▷ File_halljanko315_gens.csv -end \
22 ▷ ▷ -define G -permutation_group \
23 ▷ ▷ ▷ -bsgs halljanko315 "File\halljanko315" \
24 ▷ ▷ ▷ 315 1209600 "0,1,2" 5 gens \
25 ▷ ▷ -end \
26 ▷ ▷ -define Orb -orbits \
27 ▷ ▷ ▷ -group G \
28 ▷ ▷ ▷ -group_action G \
29 ▷ ▷ ▷ -generators G \
30 ▷ ▷ ▷ -on_subsets 2 Control \

```

```

31   ▷ ▷ -end
32

```

There are 4 orbits on pairs. We decide to pick the fourth orbit to create a new graph. Because indexing is zero-based, we give the orbit index of 3:

Code Snippet 856

```

1   HJ_orbital_graph_3:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define gens -vector -file \
4   ▷ ▷ ▷ File_halljanko315_gens.csv -end \
5   ▷ ▷ -define G -permutation_group \
6   ▷ ▷ ▷ -bsgs halljanko315 "File\_halljanko315" \
7   ▷ ▷ ▷ 315 1209600 "0,1,2" 5 gens \
8   ▷ ▷ -end \
9   ▷ ▷ -define Gamma -graph \
10  ▷ ▷ ▷ -orbital_graph G 3 \
11  ▷ ▷ -end \
12  ▷ ▷ -with Gamma -do \
13  ▷ ▷ ▷ -graph_theoretic_activity \
14  ▷ ▷ ▷ -properties \
15  ▷ ▷ -end \
16  ▷ ▷ -with Gamma -do \
17  ▷ ▷ ▷ -graph_theoretic_activity \
18  ▷ ▷ ▷ -save \
19  ▷ ▷ -end
20

```

The graph is regular of degree 64.

The next command computes the automorphism group of the collinearity graph of the $Q(4, 2)$ quadric.

Code Snippet 857

```

1   PGO_5_2_graph_group:
2   ▷ $(ORBITER) -v 2 \
3   ▷ ▷ -define F -finite_field -q 2 -end \
4   ▷ ▷ -define O -orthogonal_space 0 5 F -without_group -end \
5   ▷ ▷ -with O -do -orthogonal_space_activity \
6   ▷ ▷ ▷ -export_point_line_incidence_matrix \
7   ▷ ▷ -end
8   ▷ $(ORBITER) -v 3 \
9   ▷ ▷ -define Inc -vector -file 0.5_2_incidence_matrix.csv -end \
10  ▷ ▷ -define Gamma -graph -collinearity_graph Inc -end \
11  ▷ ▷ -with Gamma -do \
12  ▷ ▷ -graph_theoretic_activity \
13  ▷ ▷ ▷ -automorphism_group \
14  ▷ ▷ -end \
15  ▷ ▷ -with Gamma -do \
16  ▷ ▷ -graph_theoretic_activity \
17  ▷ ▷ ▷ -eigenvalues \

```

```
18   ▷ ▷ -end \  
19   ▷ ▷ -with Gamma -do \  
20   ▷ ▷ -graph_theoretic_activity \  
21   ▷ ▷ ▷ -eigenvalue_report \  
22   ▷ ▷ -end  
23   ▷ $(PDFLATEX) collinearity_graph_eigenvalues.tex  
24   ▷ $(OPEN) collinearity_graph_eigenvalues.pdf  
25
```

The group is $\text{PGO}(5, 2)$ of order 720. The command creates the group as a permutation group on the 15 vertices of the graph. The group is no longer treated as a matrix group.

17.8 Canonical Forms of Quartic Curves

We wish to study the automorphism groups of the quartic curves of Edge that have been considered in Section 4.5. We start by creating a cheat sheet of the field $\mathbb{F}_{17}$

Code Snippet 858

```

1      F_17_edge:
2      ▷ $(ORBITER) -v 3 \
3      ▷ ▷ -define F -finite_field -q 17 -end \
4      ▷ ▷ -with F -do -finite_field_activity \
5      ▷ ▷ ▷ -cheat_sheet_GF -end
6      ▷ $(PDFLATEX) GF_17.tex
7      ▷ $(OPEN) GF_17.pdf
8
```

Next, we compute the canonical form of the Edge quartic. This command also computes generators for the automorphism group of the curve.

Code Snippet 859

```

1      Edge_curve_17_nauty:
2      ▷ $(ORBITER) -v 6 \
3      ▷ ▷ -define F -finite_field -q 17 -end \
4      ▷ ▷ -define R -polynomial_ring \
5      ▷ ▷ ▷ -field F \
6      ▷ ▷ ▷ -number_of_variables 3 \
7      ▷ ▷ ▷ -homogeneous_of_degree 4 \
8      ▷ ▷ ▷ -monomial_ordering_partition \
9      ▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \
10     ▷ ▷ -end \
11     ▷ ▷ -assign Eqn -from \
12     ▷ ▷ -with R -do \
13     ▷ ▷ ▷ -ring_theoretic_activity \
14     ▷ ▷ ▷ -parse_equation \
15     ▷ ▷ ▷ ▷ "Edge_curve" "Edge\_curve" \
16     ▷ ▷ ▷ ▷ $(EDGE_CURVE_EQUATION_ALGEBRAIC) \
17     ▷ ▷ ▷ ▷ f 3 \
18     ▷ ▷ -end \
19     ▷ ▷ -define P -projective_space \
20     ▷ ▷ ▷ -n 2 -field F -v 0 \
21     ▷ ▷ -end \
22     ▷ ▷ -define Geo -geometric_object P \
23     ▷ ▷ ▷ -projective_variety R \
24     ▷ ▷ ▷ ▷ "Edge_curve_q17" "Edge\_curve\_q17" \
25     ▷ ▷ ▷ ▷ Eqn \
26     ▷ ▷ -end \
27     ▷ ▷ -define C -combinatorial_object \
28     ▷ ▷ ▷ -label "Edge_curve_q17" "Edge\_curve\_q17" \
29     ▷ ▷ ▷ -geometric_object Geo \
30     ▷ ▷ -end \
31     ▷ ▷ -with C -do \
32     ▷ ▷ -combinatorial_object_activity \

```

```

33 ▷ ▷ ▷ -canonical_form_PG P \
34 ▷ ▷ ▷ ▷ -save_ago \
35 ▷ ▷ ▷ ▷ -save_transversal \
36 ▷ ▷ ▷ ▷ -max_TDO_depth 10 \
37 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "Edge_curve_q17_nauty_log.txt" -end \
38 ▷ ▷ ▷ ▷ -end \
39 ▷ ▷ ▷ -end \
40 ▷ ▷ -define Doinc -draw_incidence_structure_options \
41 ▷ ▷ ▷ -width 40 -width_10 6 \
42 ▷ ▷ -end \
43 ▷ ▷ -with C -do \
44 ▷ ▷ -combinatorial_object_activity \
45 ▷ ▷ ▷ -post_processing \
46 ▷ ▷ -end \
47 ▷ ▷ -with C -do \
48 ▷ ▷ -combinatorial_object_activity \
49 ▷ ▷ ▷ -report \
50 ▷ ▷ ▷ ▷ -export_flag_orbits \
51 ▷ ▷ ▷ ▷ -show_TDO \
52 ▷ ▷ ▷ ▷ -show_TDA \
53 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
54 ▷ ▷ ▷ ▷ -export_group_GAP \
55 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
56 ▷ ▷ ▷ -end \
57 ▷ ▷ -end
58 ▷ $(PDFLATEX) Edge_curve_q17_classification.tex
59 ▷ $(OPEN) Edge_curve_q17_classification.pdf
60 ▷ $(ORBITER) -v 2 -draw_matrix \
61 ▷ ▷ -input_csv_file Edge_curve_q17_object_0_TDA_flag_orbits.csv \
62 ▷ ▷ -secondary_input_csv_file Edge_curve_q17_object_0_TDA.csv \
63 ▷ ▷ -box_width 4 -bit_depth 24 \
64 ▷ ▷ -end
65 ▷ $(OPEN) Edge_curve_q17_object_0_TDA_flag_orbits_draw.bmp
66 ▷ $(ORBITER) -v 2 \
67 ▷ ▷ -define Do -draw_options -embedded -radius 150 \
68 ▷ ▷ ▷ -nodes_empty \
69 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
70 ▷ ▷ -tree_draw \
71 ▷ ▷ ▷ -file Edge_curve_q17_nauty_log.txt \
72 ▷ ▷ ▷ -draw_options Do \
73 ▷ ▷ -end
74 ▷ $(PDFLATEX) Edge_curve_q17_nauty_log_draw.tex
75 ▷ $(OPEN) Edge_curve_q17_nauty_log_draw.pdf
76

```

Using the generators that have just been computed, we can recreate the group of the quartic curve:

Code Snippet 860

```

1 Edge_curve_17_group:
2 ▷ $(ORBITER) -v 3 \
3 ▷ ▷ -define Do -draw_options \

```

```

4      ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5      ▷ ▷ -end \
6      ▷ ▷ -define G -linear_group -PGL 3 17 \
7      ▷ ▷ -subgroup_by_generators "Stab_Edge" "24" 3 \
8      ▷ ▷ ▷ "1,0,0,0,13,0,0,0,4, \
9      ▷ ▷ ▷ 1,0,0,0,0,16,0,16,0, \
10     ▷ ▷ ▷ 0,1,16,2,4,4,15,4,4" \
11     ▷ ▷ -end \
12     ▷ ▷ -with G -do \
13     ▷ ▷ ▷ -group_theoretic_activity \
14     ▷ ▷ ▷ ▷ -group_table Do \
15     ▷ ▷ -end \
16     ▷ ▷ -with G -do \
17     ▷ ▷ ▷ -group_theoretic_activity \
18     ▷ ▷ ▷ ▷ -report Do \
19     ▷ ▷ -end \
20     ▷ ▷ -with G -do \
21     ▷ ▷ ▷ -group_theoretic_activity \
22     ▷ ▷ ▷ ▷ -print_elements_tex \
23     ▷ ▷ -end
24     ▷ $(PDFLATEX) PGL_3_17_Subgroup_Stab_Edge_24_group_table_report.tex
25     ▷ $(OPEN) PGL_3_17_Subgroup_Stab_Edge_24_group_table_report.pdf
26     ▷ $(PDFLATEX) PGL_3_17_Subgroup_Stab_Edge_24_report.tex
27     ▷ $(OPEN) PGL_3_17_Subgroup_Stab_Edge_24_report.pdf
28

```


Chapter 18

More Applied Algebra

18.1 The Automorphism Group of $\text{Sym}(6)$

It is well-known that $\text{Sym}(6)$ has an outer automorphism of order two, and that the full automorphism group has order 1440. Let us create this group. We exploit the structure of $\text{Sym}(6)$ as a Coxeter group. To this end, we find all possible labelings of the Dynkin diagram of type A_5 by involutions.

The first command creates the group $\text{Sym}(6)$ as a permutation group of order 720 of degree 6:

Code Snippet 861

```
1 Sym_6:
2 ▷ $(ORBITER) -v 3 \
3 ▷ ▷ -define Do -draw_options \
4 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5 ▷ ▷ -end \
6 ▷ ▷ -define G -permutation_group -symmetric_group 6 -end \
7 ▷ ▷ -with G -do \
8 ▷ ▷ -group_theoretic_activity \
9 ▷ ▷ ▷ -report Do \
10 ▷ ▷ -end
11 ▷ $(PDFLATEX) Sym_6_report.tex
12 ▷ $(OPEN) Sym_6_report.pdf
13
```

The next command computes the conjugacy classes of the group (using magma):

Code Snippet 862

```
1 Sym_6_classes:
2 ▷ $(ORBITER) -v 3 \
3 ▷ ▷ -define G -permutation_group -symmetric_group 6 -end \
4 ▷ ▷ -with G -do \
5 ▷ ▷ -group_theoretic_activity \
6 ▷ ▷ ▷ -classes \
7 ▷ ▷ -end
8 ▷ $(PDFLATEX) Sym_6_classes_out.tex
9 ▷ $(OPEN) Sym_6_classes_out.pdf
10
```

The ordering of conjugacy classes is chosen by Magma. Here is the output of the command:

Conjugacy classes in Sym_6

The group order is

720

The following table shows the conjugacy classes of the group. The columns display: the number of the class, the order of elements in the class, an ATLAS style label for the class, the size of the class, the order of the centralizer of an element of the class, the order of the normalizer of the group generated by an element of the class.

0	1	1A	1	720	720
1	2	2A	15	48	48
2	2	2B	15	48	48
3	2	2C	45	16	16
4	3	3A	40	18	36
5	3	3B	40	18	36
6	4	4A	90	8	16
7	4	4B	90	8	16
8	5	5A	144	5	20
9	6	6A	120	6	12
10	6	6B	120	6	12

Class 0 / 11

Order of element = 1

Class size = 1

Centralizer order = 720

Normalizer order = 720

Representing element is

$$c_0 = id$$

of order 1 and with 6 fixed points. 0, 1, 2, 3, 4, 5

The normalizer is generated by:

Strong generators for a group of order 720:

(4, 5) , (3, 4) , (2, 3) , (1, 2) , (0, 1)

0, 1, 2, 3, 5, 4,

0, 1, 2, 4, 3, 5,

0, 1, 3, 2, 4, 5,

0, 2, 1, 3, 4, 5,

1, 0, 2, 3, 4, 5,

Class 1 / 11

Order of element = 2

Class size = 15

Centralizer order = 48

Normalizer order = 48

Representing element is

$$c_1 = (0, 1)(2, 3)(4, 5)$$

of order 2 and with 0 fixed points. 1, 0, 3, 2, 5, 4

The normalizer is generated by:

Strong generators for a group of order 48:

(4, 5) , (2, 3) , (2, 4)(3, 5) , (0, 2)(1, 3)

0, 1, 2, 3, 5, 4,

0, 1, 3, 2, 4, 5,

0, 1, 4, 5, 2, 3,

2, 3, 0, 1, 4, 5,

Class 2 / 11

Order of element = 2

Class size = 15

Centralizer order = 48

Normalizer order = 48

Representing element is

$$c_2 = (0, 1)$$

of order 2 and with 4 fixed points. 1, 0, 2, 3, 4, 5

The normalizer is generated by:

Strong generators for a group of order 48:

(4, 5) , (3, 4) , (2, 3) , (0, 1)

0, 1, 2, 3, 5, 4,

0, 1, 2, 4, 3, 5,

0, 1, 3, 2, 4, 5,

1, 0, 2, 3, 4, 5,

Class 3 / 11

Order of element = 2

Class size = 45

Centralizer order = 16

Normalizer order = 16

Representing element is

$$c_3 = (0, 1)(2, 3)$$

of order 2 and with 2 fixed points. 1, 0, 3, 2, 4, 5

The normalizer is generated by:

Strong generators for a group of order 16:

(4, 5) , (2, 3) , (0, 1)(2, 3) , (0, 2)(1, 3)

0, 1, 2, 3, 5, 4,

0, 1, 3, 2, 4, 5,

1, 0, 3, 2, 4, 5,
2, 3, 0, 1, 4, 5,

Class 4 / 11

Order of element = 3

Class size = 40

Centralizer order = 18

Normalizer order = 36

Representing element is

$$c_4 = (0, 1, 2)(3, 4, 5)$$

of order 3 and with 0 fixed points. 1, 2, 0, 4, 5, 3

The normalizer is generated by:

Strong generators for a group of order 36:

$(3, 5, 4)$, $(1, 2)(4, 5)$, $(0, 3)(1, 5)(2, 4)$

0, 1, 2, 5, 3, 4,

0, 2, 1, 3, 5, 4,

3, 5, 4, 0, 2, 1,

Class 5 / 11

Order of element = 3

Class size = 40

Centralizer order = 18

Normalizer order = 36

Representing element is

$$c_5 = (0, 1, 2)$$

of order 3 and with 3 fixed points. 1, 2, 0, 3, 4, 5

The normalizer is generated by:

Strong generators for a group of order 36:

$(4, 5)$, $(3, 4)$, $(1, 2)$, $(0, 1, 2)$

0, 1, 2, 3, 5, 4,

0, 1, 2, 4, 3, 5,

0, 2, 1, 3, 4, 5,

1, 2, 0, 3, 4, 5,

Class 6 / 11

Order of element = 4

Class size = 90

Centralizer order = 8

Normalizer order = 16

Representing element is

$$c_6 = (0, 1, 2, 3)(4, 5)$$

of order 4 and with 0 fixed points. 1, 2, 3, 0, 5, 4

The normalizer is generated by:

Strong generators for a group of order 16:

$(4, 5)$, $(1, 3)$, $(0, 1, 2, 3)(4, 5)$

0, 1, 2, 3, 5, 4,

0, 3, 2, 1, 4, 5,

1, 2, 3, 0, 5, 4,

Class 7 / 11

Order of element = 4

Class size = 90

Centralizer order = 8

Normalizer order = 16

Representing element is

$$c_7 = (0, 1, 2, 3)$$

of order 4 and with 2 fixed points. 1, 2, 3, 0, 4, 5

The normalizer is generated by:

Strong generators for a group of order 16:

$(4, 5)$, $(1, 3)$, $(0, 1, 2, 3)$

0, 1, 2, 3, 5, 4,

0, 3, 2, 1, 4, 5,

1, 2, 3, 0, 4, 5,

Class 8 / 11

Order of element = 5

Class size = 144

Centralizer order = 5

Normalizer order = 20

Representing element is

$$c_8 = (0, 1, 2, 3, 4)$$

of order 5 and with 1 fixed points. 1, 2, 3, 4, 0, 5

The normalizer is generated by:

Strong generators for a group of order 20:

$(1, 2, 4, 3)$, $(0, 1, 2, 3, 4)$

0, 2, 4, 1, 3, 5,

1, 2, 3, 4, 0, 5,

Class 9 / 11

Order of element = 6

Class size = 120

Centralizer order = 6

Normalizer order = 12

Representing element is

$$c_9 = (0, 1, 2, 3, 4, 5)$$

of order 6 and with 0 fixed points. 1, 2, 3, 4, 5, 0

The normalizer is generated by:

Strong generators for a group of order 12:

(1, 5)(2, 4) , (0, 1, 2, 3, 4, 5)

0, 5, 4, 3, 2, 1,

1, 2, 3, 4, 5, 0,

Class 10 / 11

Order of element = 6

Class size = 120

Centralizer order = 6

Normalizer order = 12

Representing element is

$$c_{10} = (0, 1, 2)(3, 4)$$

of order 6 and with 1 fixed points. 1, 2, 0, 4, 3, 5

The normalizer is generated by:

Strong generators for a group of order 12:

(3, 4) , (1, 2) , (0, 1, 2)(3, 4)

0, 1, 2, 4, 3, 5,

0, 2, 1, 3, 4, 5,

1, 2, 0, 4, 3, 5,

There are three classes of involutions. We store representatives for each class in a separate makefile variable:

Code Snippet 863

```
1 SYM_6_CLASS_2A_REP="1,0,3,2,5,4"
2
```

Code Snippet 864

```
1 SYM_6_CLASS_2B_REP="1,0,3,2,4,5"
2
```

Code Snippet 865

```
1 SYM_6_CLASS_2C_REP="1,0,2,3,4,5"
2
```

Using the following three commands, we create the full class in each case. Each time, the set of elements of a class is written to file.

Code Snippet 866

```
1 Sym_6_Class_2A:
2 ▷ $(ORBITER) -v 9 \
3 ▷ ▷ -orbiter_path $(ORBITER_PATH) \
```

```

4   ▷ ▷ -define G -permutation_group -symmetric_group 6 -end \
5   ▷ ▷ -with G -do \
6   ▷ ▷ -group_theoretic_activity \
7   ▷ ▷ ▷ -conjugacy_class_of "2A" $(SYM_6_CLASS_2A.REP) \
8   ▷ ▷ -end
9

```

Code Snippet 867

```

1   Sym_6_Class_2B:
2   ▷ $(ORBITER) -v 9 \
3   ▷ ▷ -orbiter_path $(ORBITER_PATH) \
4   ▷ ▷ -define G -permutation_group -symmetric_group 6 -end \
5   ▷ ▷ -with G -do \
6   ▷ ▷ -group_theoretic_activity \
7   ▷ ▷ ▷ -conjugacy_class_of "2B" $(SYM_6_CLASS_2B.REP) \
8   ▷ ▷ -end
9

```

Code Snippet 868

```

1   Sym_6_Class_2C:
2   ▷ $(ORBITER) -v 9 \
3   ▷ ▷ -orbiter_path $(ORBITER_PATH) \
4   ▷ ▷ -define G -permutation_group -symmetric_group 6 -end \
5   ▷ ▷ -with G -do \
6   ▷ ▷ -group_theoretic_activity \
7   ▷ ▷ ▷ -conjugacy_class_of "2C" $(SYM_6_CLASS_2C.REP) \
8   ▷ ▷ -end
9

```

We store the filenames of each class in separate makefile variables:

Code Snippet 869

```

1   FILE_NAME_SYM_6_CLASS_2A="Sym_6_class_of_2A.csv"
2

```

Code Snippet 870

```

1   FILE_NAME_SYM_6_CLASS_2B="Sym_6_class_of_2B.csv"
2

```

Code Snippet 871

```

1   FILE_NAME_SYM_6_CLASS_2C="Sym_6_class_of_2C.csv"
2

```

We form the union of the three classes by concatenating the three files containing the three conjugacy classes:

Code Snippet 872

```

1 Sym_6_concatenate_classes:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -csv_file_concatenate Sym_6_class_2ABC.csv 3 \
4   ▷ ▷ ▷ $(FILE_NAME_SYM_6_CLASS_2A) \
5   ▷ ▷ ▷ $(FILE_NAME_SYM_6_CLASS_2B) \
6   ▷ ▷ ▷ $(FILE_NAME_SYM_6_CLASS_2C)
7

```

Next, we compute the orders of the products of pairs of involutions. The following command reads the file created previously and writes a matrix containing the orders of pairs. In the (i, j) -entry, the order of the product of involution i and involution j is recorded:

Code Snippet 873

```

1 Sym_6_class_2ABC_order_of_pairs:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -orbiter_path $(ORBITER_PATH) \
4   ▷ ▷ -define G -permutation_group -symmetric_group 6 -end \
5   ▷ ▷ -define Sym_6_class_2ABC -vector_ge \
6   ▷ ▷ ▷ -action G \
7   ▷ ▷ ▷ -read_csv Sym_6_class_2ABC.csv elements \
8   ▷ ▷ -end \
9   ▷ ▷ -with Sym_6_class_2ABC -do -vector_ge_activity \
10  ▷ ▷ ▷ -order_of_products_of_pairs \
11  ▷ ▷ -end \
12

```

For the purposes of labeling the Dynkin diagram of type A_5 , we are interested in those pairs for which the product has order either two or three. We create two matrices. The first for products of order 2, the second for products of order 3:

Code Snippet 874

```

1 Sym_6_order_of_products_change_values:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -orbiter_path $(ORBITER_PATH) \
4   ▷ ▷ -change_values \
5   ▷ ▷ ▷ Sym_6_class_2ABC_order_of_product_of_pairs.csv \
6   ▷ ▷ ▷ Sym_6_class_2ABC_order_3.csv \
7   ▷ ▷ ▷ 1,2,3,4,5,6 \
8   ▷ ▷ ▷ 0,0,1,0,0,0
9   ▷ $(ORBITER) -v 3 \
10  ▷ ▷ -orbiter_path $(ORBITER_PATH) \
11  ▷ ▷ -change_values \
12  ▷ ▷ ▷ Sym_6_class_2ABC_order_of_product_of_pairs.csv \
13  ▷ ▷ ▷ Sym_6_class_2ABC_order_2.csv \
14  ▷ ▷ ▷ 1,2,3,4,5,6 \
15  ▷ ▷ ▷ 0,1,0,0,0,0
16

```

We create two graphs from these matrices. For fun, we compute the automorphism group of each graph, as well as the eigenvalues:

Code Snippet 875

```

1   Sym_6_class_2ABC_make_graph_G2:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define G -graph \
4   ▷ ▷ ▷ -load_adjacency_matrix_from_csv_and_select_value \
5   ▷ ▷ ▷ Sym_6_class_2ABC_order_2.csv 1 \
6   ▷ ▷ -end \
7   ▷ ▷ -with G -do \
8   ▷ ▷ ▷ -graph_theoretic_activity -properties \
9   ▷ ▷ -end \
10  ▷ ▷ -with G -do \
11  ▷ ▷ ▷ -graph_theoretic_activity -eigenvalues \
12  ▷ ▷ -end \
13  ▷ ▷ -with G -do \
14  ▷ ▷ ▷ -graph_theoretic_activity -automorphism_group \
15  ▷ ▷ -end
16  ▷ $(PDFLATEX) File_Sym_6_class_2ABC_order_2_report.tex
17  ▷ $(OPEN) File_Sym_6_class_2ABC_order_2_report.pdf
18

```

Code Snippet 876

```

1   Sym_6_class_2ABC_make_graph_G3:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define G -graph \
4   ▷ ▷ ▷ -load_adjacency_matrix_from_csv_and_select_value \
5   ▷ ▷ ▷ Sym_6_class_2ABC_order_3.csv 1 \
6   ▷ ▷ -end \
7   ▷ ▷ -with G -do \
8   ▷ ▷ ▷ -graph_theoretic_activity -properties \
9   ▷ ▷ -end \
10  ▷ ▷ -with G -do \
11  ▷ ▷ ▷ -graph_theoretic_activity -eigenvalues \
12  ▷ ▷ -end \
13  ▷ ▷ -with G -do \
14  ▷ ▷ ▷ -graph_theoretic_activity -automorphism_group \
15  ▷ ▷ -end
16  ▷ $(PDFLATEX) File_Sym_6_class_2ABC_order_3_report.tex
17  ▷ $(OPEN) File_Sym_6_class_2ABC_order_3_report.pdf
18

```

The next step is to find all possible labelings of the Dynkin diagram of type A_5 with involutions, so that the relations are satisfied. To this end, we need the two graphs created before. The program writes a file which contains all possible labelings. It turns out that there are exactly 1440 labelings.

Code Snippet 877

```

1   Sym_6_class_2ABC_subgraph_A5:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define G2 -graph \
4   ▷ ▷ ▷ -load_adjacency_matrix_from_csv_and_select_value \

```

```

5    ▷ ▷ ▷ Sym_6_class_2ABC_order_2.csv 1 \
6    ▷ ▷ -end \
7    ▷ ▷ -define G3 -graph \
8    ▷ ▷ ▷ -load_adjacency_matrix_from_csv_and_select_value \
9    ▷ ▷ ▷ Sym_6_class_2ABC_order_3.csv 1 \
10   ▷ ▷ -end \
11   ▷ ▷ -with G2 -and G3 -do \
12   ▷ ▷ ▷ -graph_theoretic_activity -find_subgraph "A5" \
13   ▷ ▷ -end \
14

```

Next, we create the automorphisms associated to the different labelings of the Dynkin diagram:

Code Snippet 878

```

1    Sym_6_aut_group:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define images -vector -file_column \
4    ▷ ▷ ▷ File_Sym_6_class_2ABC_order_2_all_A5.csv "C0" \
5    ▷ ▷ -end \
6    ▷ ▷ -define G -permutation_group -symmetric_group 6 -end \
7    ▷ ▷ -define elements_of_order2_ge -vector_ge \
8    ▷ ▷ ▷ -action G \
9    ▷ ▷ ▷ -read_csv Sym_6_class_2ABC.csv "elements" \
10   ▷ ▷ -end \
11   ▷ ▷ -with G -do \
12   ▷ ▷ -group_theoretic_activity \
13   ▷ ▷ ▷ -group_of_automorphisms_by_images_of_generators \
14   ▷ ▷ ▷ sym_6_aut_group \
15   ▷ ▷ ▷ elements_of_order2_ge \
16   ▷ ▷ ▷ images \
17   ▷ ▷ -end
18

```

We copy paste the generators and add information about the group in the form of makefile variables:

Code Snippet 879

```

1    AUT_SYM_6_GROUP_ORDER=1440
2

```

Code Snippet 880

```

1    AUT_SYM_6_DEGREE=720
2

```

Code Snippet 881

```

1    AUT_SYM_6_BASE="1,2,6,24"
2

```

Code Snippet 882

```

1   AUT_SYM_6_NB_GENS=6
2

```

For the sake of saving space, the variable `AUT_SYM_6_GENS` is omitted.

The next command creates the automorphism group of $\text{Sym}(6)$ from the data that is in the makefile variables. The group has order 1440 and a permutation representation of degree 720.

Code Snippet 883

```

1   Aut_Sym_6_group:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define gens -vector -dense $(AUT_SYM_6_GENS) -end \
7   ▷ ▷ -define G -permutation_group \
8   ▷ ▷ ▷ -bsgs AutSym6 "{\rm AutSym}{6}" \
9   ▷ ▷ ▷ $(AUT_SYM_6_DEGREE) \
10  ▷ ▷ ▷ $(AUT_SYM_6_GROUP_ORDER) \
11  ▷ ▷ ▷ $(AUT_SYM_6_BASE) \
12  ▷ ▷ ▷ $(AUT_SYM_6_NB_GENS) \
13  ▷ ▷ ▷ gens \
14  ▷ ▷ -end \
15  ▷ ▷ -with G -do \
16  ▷ ▷ -group_theoretic_activity \
17  ▷ ▷ ▷ -report Do \
18  ▷ ▷ -end
19  ▷ $(PDFLATEX) AutSym6_report.tex
20  ▷ $(OPEN) AutSym6_report.pdf
21

```

For fun, we compute the subgroup lattice of $\text{Aut}(\text{Sym}(6))$. Warning: the following command is very slow.

Code Snippet 884

```

1   Aut_Sym_6_subgroup_lattice:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define gens -vector -dense $(AUT_SYM_6_GENS) -end \
4   ▷ ▷ -define G -permutation_group \
5   ▷ ▷ ▷ -bsgs AutSym6 "{\rm AutSym}{6}" \
6   ▷ ▷ ▷ $(AUT_SYM_6_DEGREE) \
7   ▷ ▷ ▷ $(AUT_SYM_6_GROUP_ORDER) \
8   ▷ ▷ ▷ $(AUT_SYM_6_BASE) \
9   ▷ ▷ ▷ $(AUT_SYM_6_NB_GENS) \
10  ▷ ▷ ▷ gens \
11  ▷ ▷ -end \
12  ▷ ▷ -with G -do \
13  ▷ ▷ -group_theoretic_activity \
14  ▷ ▷ ▷ -subgroup_lattice \
15  ▷ ▷ -end
16

```

We produce a drawing of the subgroup lattice.

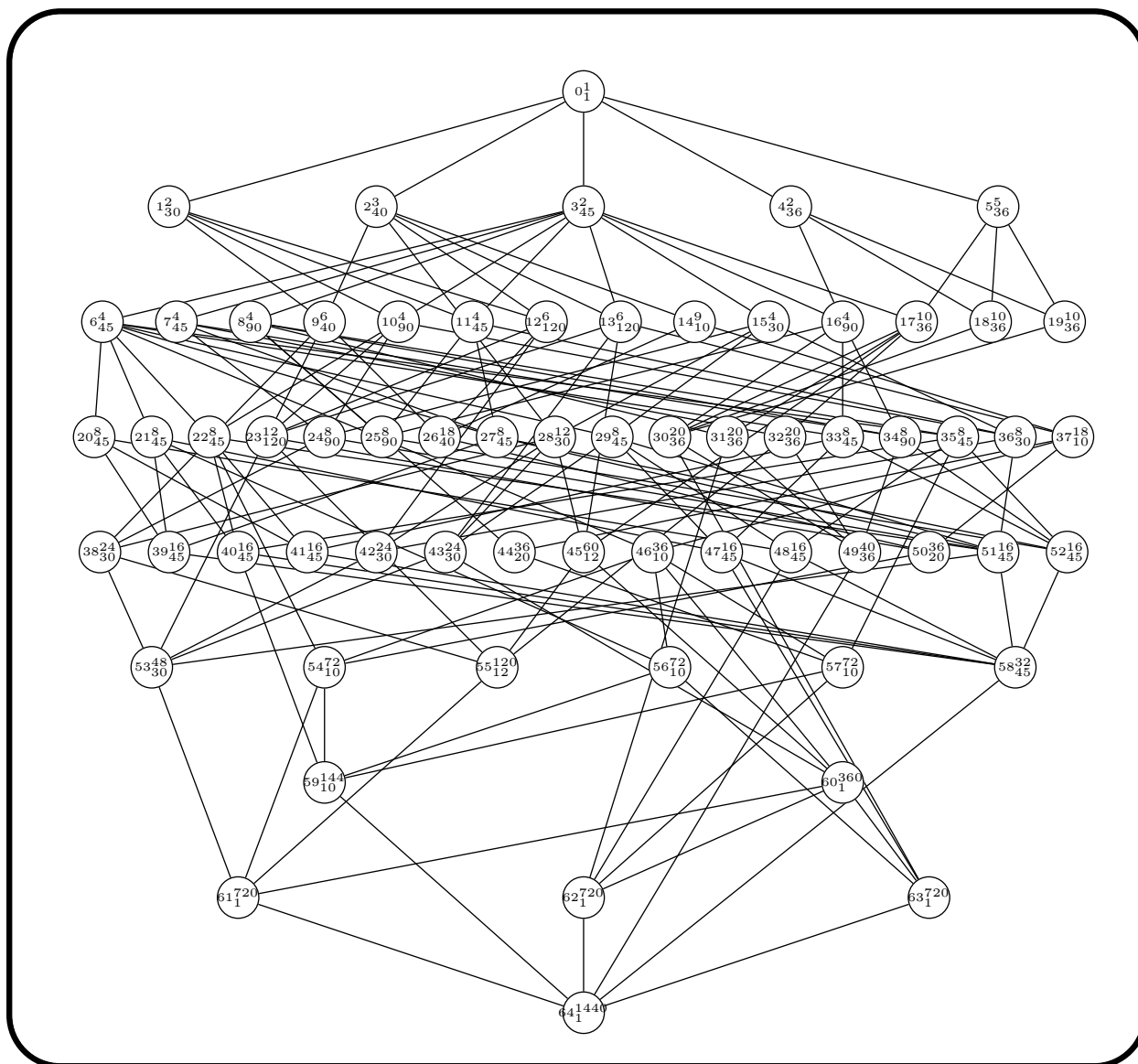
Code Snippet 885

```

1 Aut_Sym_6_subgroup_lattice_load_and_draw:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.5 \
5   ▷ ▷ -end \
6   ▷ ▷ -define gens -vector -dense $(AUT_SYM_6_GENS) -end \
7   ▷ ▷ -define G -permutation_group \
8   ▷ ▷ ▷ -bsgs AutSym6 "{\rm AutSym}{6}" \
9   ▷ ▷ ▷ $(AUT_SYM_6_DEGREE) \
10  ▷ ▷ ▷ $(AUT_SYM_6_GROUP_ORDER) \
11  ▷ ▷ ▷ $(AUT_SYM_6_BASE) \
12  ▷ ▷ ▷ $(AUT_SYM_6_NB_GENS) \
13  ▷ ▷ ▷ gens \
14  ▷ ▷ -end \
15  ▷ ▷ -with G -do \
16  ▷ ▷ -group_theoretic_activity \
17  ▷ ▷ ▷ -subgroup_lattice_load AutSym6_subgroup_lattice.csv \
18  ▷ ▷ -end \
19  ▷ ▷ -with G -do \
20  ▷ ▷ -group_theoretic_activity \
21  ▷ ▷ ▷ -subgroup_lattice_draw_by_orbits Do \
22  ▷ ▷ -end
23  ▷ #$(PDFLATEX) AutSym6_drawing.tex
24  ▷ #$(OPEN) AutSym6_drawing.pdf
25  ▷ $(PDFLATEX) AutSym6_drawing_by_orbits.tex
26  ▷ $(OPEN) AutSym6_drawing_by_orbits.pdf
27

```

As there are too many subgroups, we simply draw the poset of orbits, shown below:



Chapter 19

Interfaces

19.1 Graphical Output

Orbiter can produce graphical output in a variety of formats:

1. TikZ / Latex [72], see Table 17.5,
2. Metapost [39],
3. Bitmap files (.bmp) [75], see Table 17.4,
4. Povray, see Section 19.2,
5. Gnuplot [34], see Section 19.5.

The poset classification algorithm from Sections 8.3 and 8.4 computes partially ordered sets. The posets are created using the `-draw_poset` option in the poset classification control command package, see Table 8.4. The posets are stored in a file with extension `.layered_graph`. These files can be drawn using the `-draw_layered_graph` command. The commands in Table 19.1 and Table 19.2 show ways in which to customize the drawings. Let us consider an example. Suppose we are interested in the Schreier trees of a permutation group represented in a stabilizer chain. We take $\text{PGL}(4, 2)$ in its action on the wedge product. The command

Code Snippet 886

```
1   PGL_6_2.Wedge_4_2.detached.report.tex:
2   ▷ $(ORBITER) -v 4 \
3   ▷ ▷ -define Do -draw_options \
4   ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5   ▷ ▷ -end \
6   ▷ ▷ -define G -linear_group -PGL 4 2 \
7   ▷ ▷ ▷ -wedge_detached \
8   ▷ ▷ -end \
9   ▷ ▷ -with G -do \
10  ▷ ▷ -group_theoretic_activity \
11  ▷ ▷ ▷ -report Do \
12  ▷ ▷ -end
13  ▷ $(PDFLATEX) PGL_6_2.Wedge_4_2.detached.report.tex
14  ▷ $(OPEN) PGL_6_2.Wedge_4_2.detached.report.pdf
15
```

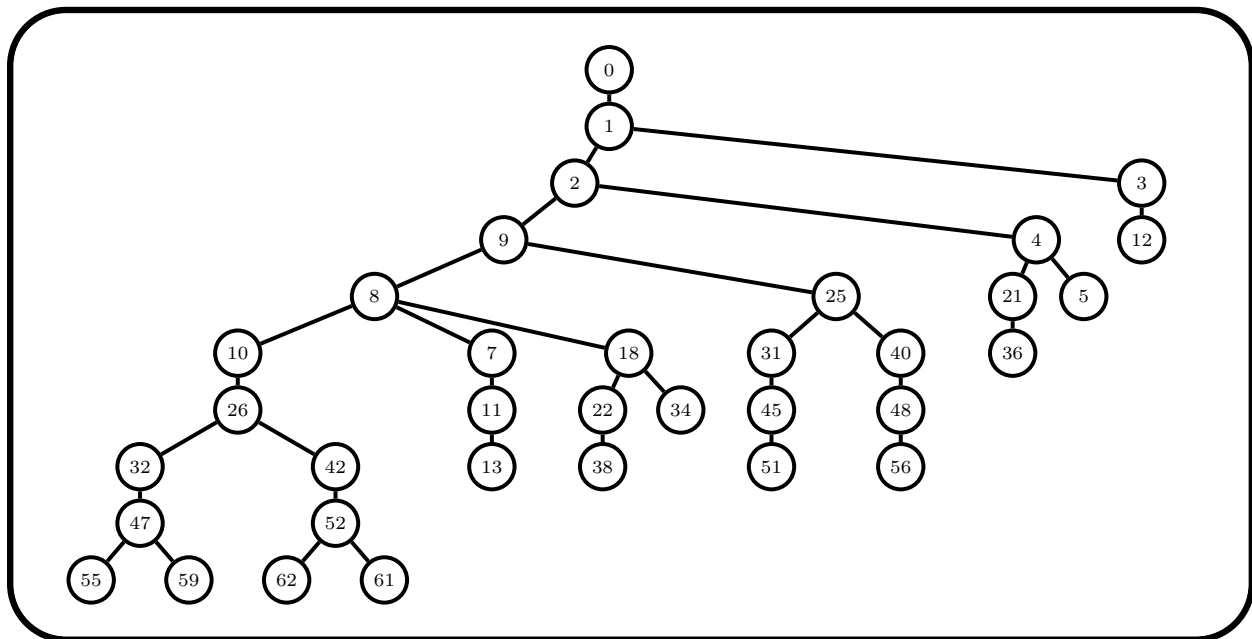
Drawing Options for Layered Graph Files (Part 1)		
Command	Arguments	Purpose
-paperheight	h	Set paperheight for the latex document. The given value is measures the paperheight in tens of an inch.
-paperwidth	w	Set paperwidth for the latex document. The given value is measures the paperwidth in tens of an inch.
-xin	a	Assume input x -coordinates are in the interval $[0, a]$. Default value: 10000.
-yin	a	Assume input y -coordinates are in the interval $[0, a]$. Default value: 10000.
-xout	a	Assume output x -coordinates are in the interval $[0, a]$. Default value: 1000000.
-yout	a	Assume output y -coordinates are in the interval $[0, a]$. Default value: 1000000.
-spanning_tree		Place nodes according to a spanning tree. Default value: off.
-circle		Circle all nodes. Default value: on.
-corners		Draw corners at the outside of the drawing. Default value: off.
-radius	r	Use radius r for drawing circles around nodes.
-embedded		Create latex headers for stand-alone latex files. Default value: off.
-sideways		Create latex figure sideways. Default value: off.
-label_edges		Label the edges. This option is relevant for the drawing of Schreier trees. Default value: off.
-x_stretch	s	Apply x -axis scaling by a factor of s . Default value: $s = 1.0$.
-y_stretch	s	Apply y -axis scaling by a factor of s . Default value: $s = 1.0$.

Table 19.1: Drawing Options for Layered Graph Files (Part 1)

Drawing Options for Layered Graph Files (Part 2)		
Command	Arguments	Purpose
-scale	s	Use Tikz global scale-factor of s . Default value: $s = 0.45$.
-line_width	s	Set Tikz line width to s . Default value: $s = 1.5$.
-rotated		Rotate the output.
-upside_down		Flip top and bottom.
-nodes		Turn on node drawing.
-nodes_empty		Do not label the nodes. Default value: off.
-no_vertices		Do not draw vertices.
-node_color	c	Set the color for nodes to c (unless the graph already has vertex colors).
-select_layers	S	Draw layers whose index is given in the list S only.
-paths_in_between	$l_1 \ i_1 \ l_2 \ i_2$	Draw all paths from node (l_1, i_1) to node (l_2, i_2) . Here, (l, i) is the i -th node at layer l (counting from zero). Delete all other edges between layers l_1 and l_2 .
-poset_orbits_x_stretch	f1000	Specify a stretch factor for grouping of poset orbits in 1000th.
-place		Layout the graph.
-poset_type_Asup		Draw poset in supremum style.
-poset_type_Ainf		Draw poset in infimum style.
-poset_with_horizontal_lines		Draw horizontal lines to indicate orbits.

Table 19.2: Drawing Options for Layered Graph Files (Part 2)

produces a report. The following tree is taken from this report. It represents the first basic orbit in the stabilizer chain of the group in that action.



The command

Code Snippet 887

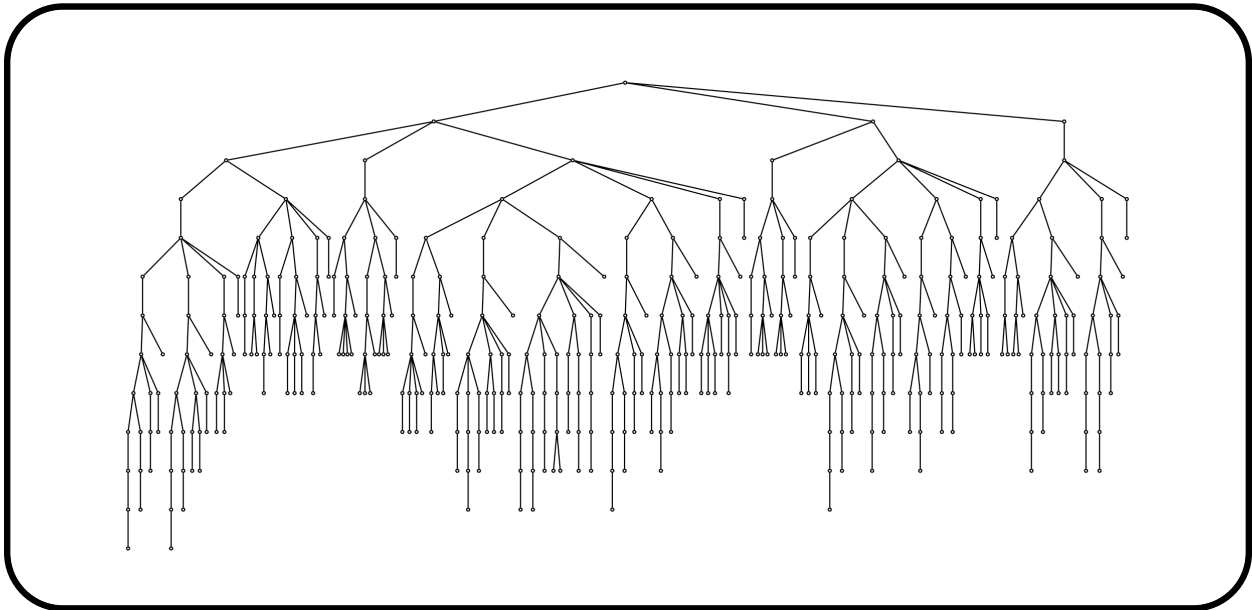
```

1  schreier_tree_graphical_output:
2  ▷ $(ORBITER) -v 4 \
3  ▷ ▷ -define Do -draw_options \
4  ▷ ▷ ▷ -yout 500000 \
5  ▷ ▷ ▷ -radius 15 -nodes_empty \
6  ▷ ▷ ▷ -line_width 0.5 -y_stretch 0.25 \
7  ▷ ▷ ▷ -embedded \
8  ▷ ▷ -end \
9  ▷ ▷ -define F -finite_field -q 2 -end \
10 ▷ ▷ -define G -linear_group -PGL 4 F -end \
11 ▷ ▷ -define R -polynomial_ring \
12 ▷ ▷ ▷ -field F \
13 ▷ ▷ ▷ -number_of_variables 4 \
14 ▷ ▷ ▷ -homogeneous_of_degree 3 \
15 ▷ ▷ ▷ -monomial_ordering_partition \
16 ▷ ▷ ▷ -variables "X,Y,Z,W" "X,Y,Z,W" \
17 ▷ ▷ -end \
18 ▷ ▷ -define Orb -orbits \
19 ▷ ▷ ▷ -group G \
20 ▷ ▷ ▷ -print_interval 1000000 \
21 ▷ ▷ ▷ -on_polynomials R \
22 ▷ ▷ -end \
23 ▷ ▷ -with Orb -do -orbits_activity \
24 ▷ ▷ ▷ -export_something "orbit" 6 \
25 ▷ ▷ -end
26 ▷ ▷ -with Orb -do -orbits_activity \

```

```
27   ▷ ▷ ▷ -draw_tree 6 Do \  
28   ▷ ▷ -end  
29
```

draws the 6th Schreier tree in the classification of orbits of $\text{PGL}(4, 2)$ on homogeneous polynomials of degree 3 in 4 variables, shown below



The orbit has length 420.

Povray Interface		
Command	Arguments	Purpose
-povray	description	Povray job description, see Table 19.4.
-prepare_frames	n	Prepare frames description, see Table 19.10.

Table 19.3: Povray Interface

Povray Job Options		
Command	Arguments	Purpose
-video_options		Set options from Tables 19.5 and 19.6.
-round	n	Select round n for rendering only.
-rounds	list	Select a list of rounds for rendering.
-nb_frames_default	f	Select the number of frames per round.
-output_mask	fname-mask	Set mask for output file name generation.
-scene_objects	options	Define Scene objects according to Table 19.9.

Table 19.4: Povray Job Options

19.2 The Povray Interface

Orbiter can be used to create raytracing 3D-graphics. Orbiter serves as a front end for the raytracing software Povray [64]. This is a multi step process: A 3D scene is defined through orbiter commands. Next, Orbiter produces Povray files. After that, the povray files are processed through povray, and turned into graphics files (png), called frames. The frames can be turned into a video by using tools like ffmpeg (see Section 19.3). By default, an rotational animation is produced. The main commands are listed in Table 19.3.

To describe a povray job, the commands in Table 19.4 can be used.

The Orbiter Povray interface requires some general information about the animation, the camera position, the boundary box for clipping, the font size for text and others. Tables 19.5- 19.6 list the commands to control the 3D-povray frontend.

A povray graphics starts with a description of the scene. The scene will be rendered to the final frame. A scene is composed of geometric objects. Various types of objects are available: points, lines, planes, faces, algebraic surfaces, reguli, 3D-text, and others. Some complex objects are predefined, for instance the Eckardt surface. Once the objects are defined, output commands can be invoked to draw them in various colors and with various options. At times, there are many objects in one scene. In order to ease drawing, objects can be grouped together. Only objects of the same type may be grouped. A group of object can be drawn at once. Tables 19.7 and 19.8 summarize the Orbiter commands to build objects of a 3D scene. The commands in Table 19.9 are used to build the scene. Each of these commands applies to a group of objects of the same kind. Groups of objects are created using the commands in Table 19.8 which start with `group_of`. Here is a simple example which combines scene building and graphical output. The example creates a cube with vertices, edges and faces:

Code Snippet 888

```

1   cube:
2   ▷ cp $(ORBITER_PATH)/examples/users_guide/cube_centered.obj .
3   ▷ $(ORBITER) -v 2 -povray \
4   ▷ ▷ -round 0 -nb_frames_default 30 \

```

Povray Interface (Part 1)		
Command	Arguments	Purpose
-do_not_rotate		No rotation. By default, the animation consists of a full rotation around a vertical axis.
-rotate_about_z_axis		Rotate around z -axis.
-rotate_about_111		Rotate around $(1, 1, 1)$ -axis (default).
-rotate_about_custom_axis	axis	Rotate around a custom axis. The axis is specified as a vector of length 3.
-boundary_none		Remove the clipping.
-boundary_box		Clip using a box shape.
-boundary_sphere		Clip using a sphere (default).
-font_size	s	Set font size to s .
-stroke_width	s	Set text depth to s .
-omit_bottom_plane		Remove the bottom plane.
-W	w	Set output dimension to w pixels wide.
-H	h	Set output dimension to h pixels height.
-nb_frames	n	Set number of frames to n . One revolution around the axis is split into n frames.
-zoom	$r \ a_s \ a_t \ c_s \ c_t$	Set zoom angle and clipping with in round r to change from a_s to a_t and from c_s to c_t , respectively.
-pan	$r \ F \ T \ C$	In round r , pan the camera from location F to location T in a rotational movement with center at C . Each of F, T, C are three dimensional coordinates.
-pan_reverse	$r \ F \ T \ C$	Same as -pan, but camera movement is in opposite order.
-no_background		Remove background.
-no_bottom_plane	r	Remove bottom plane in round r .
-camera	$r \ S \ C \ L$	In round r , set camera location at C , sky at S and pointing towards L . Each of S, C, L are three-dimensional coordinate vectors.

Table 19.5: Povray Interface (Part 1)

Povray Interface (Part 2)		
Command	Arguments	Purpose
-clipping	$r\ c$	In round r , set clipping radius to c .
-text	$r\ a\ \text{text}$	In round r , produce running text text with sustain value a .
-label	$r\ s\ a\ g\ \text{text}$	In round r , produce running text text with start value s , sustain s and gravity g .
-latex	$r\ s\ a\ \text{praeamble}\ g\ \text{text}\ l\ \text{fname}$	In round r , produce running latex text text with start value s , sustain s and gravity g . Put praeamble in the latex source code. Use fname for the latex file names (no extension).
-global_picture_scale	d	Set scaling factor to d .
-picture	$r\ d\ \text{fname}\ \text{options}$	In round r , place picture fname scaled by d using options.
-picture	$r\ d\ \text{fname}\ \text{options}$	In round r , place picture fname scaled by d using options.
-look_at	L	Override camera look-at value to L . L is a three-dimensional vector.
-default_angle	a	Set default camera angle to a .
-clipping_radius	f	Set default clipping radius to f .
-scale_factor	s	Set default scale factor to s .
-line_radius	s	Set default line radius to s .

Table 19.6: Povray Interface (Part 2)

```

5    ▷ ▷ -output_mask cube_%d_%03d.pov \
6    ▷ ▷ -video_options -W 1024 -H 768 \
7    ▷ ▷ -global_picture_scale 0.5 \
8    ▷ ▷ -default_angle 75 \
9    ▷ ▷ -clipping_radius 2.7 \
10   ▷ ▷ -end \
11   ▷ ▷ -scene_objects \
12   ▷ ▷ ▷ -obj_file cube_centered.obj \
13   ▷ ▷ ▷ -edge "0, 1" \
14   ▷ ▷ ▷ -edge "0, 2" \
15   ▷ ▷ ▷ -edge "0, 4" \
16   ▷ ▷ ▷ -edge "1, 3" \
17   ▷ ▷ ▷ -edge "1, 5" \
18   ▷ ▷ ▷ -edge "2, 3" \
19   ▷ ▷ ▷ -edge "2, 6" \
20   ▷ ▷ ▷ -edge "3, 7" \
21   ▷ ▷ ▷ -edge "4, 5" \
22   ▷ ▷ ▷ -edge "4, 6" \
23   ▷ ▷ ▷ -edge "5, 7" \
24   ▷ ▷ ▷ -edge "6, 7" \
25   ▷ ▷ ▷ -group_of_things.as_interval 0 8 \
26   ▷ ▷ ▷ -spheres 0 0.3 $(POLISHED_CHROME_WHITE) \
27   ▷ ▷ ▷ -group_of_things.as_interval 0 6 \

```

Scene Elements (Part 1)		
Command	Arguments	Purpose
-cubic_lex	coeffs	Cubic surface given by 20 coefficients in lexicographic ordering
-cubic_orbiter	coeffs	Cubic surface given by 20 coefficients in Orbiter ordering
-cubic_Goursat	$A B C$	Cubic surface with tetrahedral symmetry given by 3 Goursat coefficients as $Axyz + B(x^2 + y^2 + z^2) + C = 0$
-quadric_lex_10	coeffs	Quadric surface given by 10 coefficients in lexicographic ordering
-quartic_lex_35	coeffs	Quartic surface given by 35 coefficients in lexicographic ordering
-octic_lex_165	coeffs	Octic surface given by 165 coefficients in lexicographic ordering
-point	coeffs	Point given by three coordinates
-point_list_from_csv_file	fname	List of points with coordinates given in a csv file
-line_through_two_points_recentered_from_csv_file	fname	List of lines through two points with point coordinates given in a csv file
-line_through_two_points_from_csv_file	fname	List of lines through two points with point coordinates given in a csv file
-point_as_intersection_of_two_lines	$i_1 i_2$	Create a point from the intersection of two lines i_1 and i_2
-edge	$i_1 i_2$	Create an edge (line segment) between points i_1 and i_2
-text	$i_1 s$	Create a label s located at the point i
-triangular_face_given_by_three_lines	$i_1 i_2 i_3$	Create a triangular face give by three lines i_1, i_2, i_3
-face	pts	Create a face through the vertices pts, ordered cyclically
-quadric_through_three_skew_lines	$i_1 i_2 i_3$	Create a quadric through three skew lines
-plane_defined_by_three_points	$i_1 i_2 i_3$	Create a plane through three noncollinear points
-line_through_two_points_recentered	pt-coords	Create a line through two points given by 6 coordinates, recentered

Table 19.7: Scene Elements (Part 1)

Scene Elements (Part 2)		
Command	Arguments	Purpose
-line_through_two_points	pt-coords	Create a line through two points given by 6 coordinates
-line_through_two_existing_points	$i_1 i_2$	Create a line through two points
-line_through_point_with_direction	$x y z u_x u_y u_z$	Create a line through a point (x, y, z) with a given direction (u_x, u_y, u_z)
-plane_by_dual_coordinates	$a b c d$	Create the plane $ax + by + cz + d = 0$ given in dual coordinates
-dodecahedron		Create a Dodecahedron centered at the origin (20 points, 30 edges, 12 faces)
-Hilbert_Cohn_Vossen_surface		Create the Hilbert, Cohn-Vossen surface (1 cubic surface, 45 tritangent planes, 27 lines)
-obj_file	fname	Read points and faces from the given .obj file
-group_of_things	list	Create a group of things from the given list
-group_of_things_with_offset	list offset	Create a group of things from the given list, each value is increase by offset
-group_of_things_as_interval	$a b$	Create a group of things indexed by the interval $a, \dots, a + b - 1$
-group_of_things_as_interval_with_exceptions	$a b \text{ ex}$	Create a group of things indexed by the interval $a, \dots, a + b - 1$ with the exceptional elements in the list ex removed
-group_of_all_points		Create a group of things from all points currently defined
-group_of_all_faces		Create a group of things from all faces currently defined
-group_subset_at_random	$i f$	Create a group of things from the existing group i by picking a random subset with probability f
-create_regulus	$i N$	Create a regulus for quadric i with N lines

Table 19.8: Scene Elements (Part 2)

Scene Object Types		
Command	Arguments	Purpose
-spheres	i r prop	For each element in point group i , create a sphere of radius r with given Povray properties.
-cylinders	i r prop	For each element in edge group i , create a cylinder of radius r with given Povray properties.
-prisms	i d prop	For each element in face group i , create a prism of half-thickness d with given Povray properties.
-planes	i prop	For each element in plane group i , create a plane with given Povray properties.
-lines	i r prop	For each element in line group i , create a line of radius r with given Povray properties.
-cubics	i prop	For each element in group i of cubics, create a surface with given Povray properties.
-quadrics	i prop	For each element in group i of quadrics, create a surface with given Povray properties.
-quartics	i prop	For each element in group i of quartics, create a surface with given Povray properties.
-octics	i prop	For each element in group i of octics, create a surface with given Povray properties.
-texts	i d s prop	For each element in group i of labels, create a text element with half-thickness d and size s with given Povray properties.

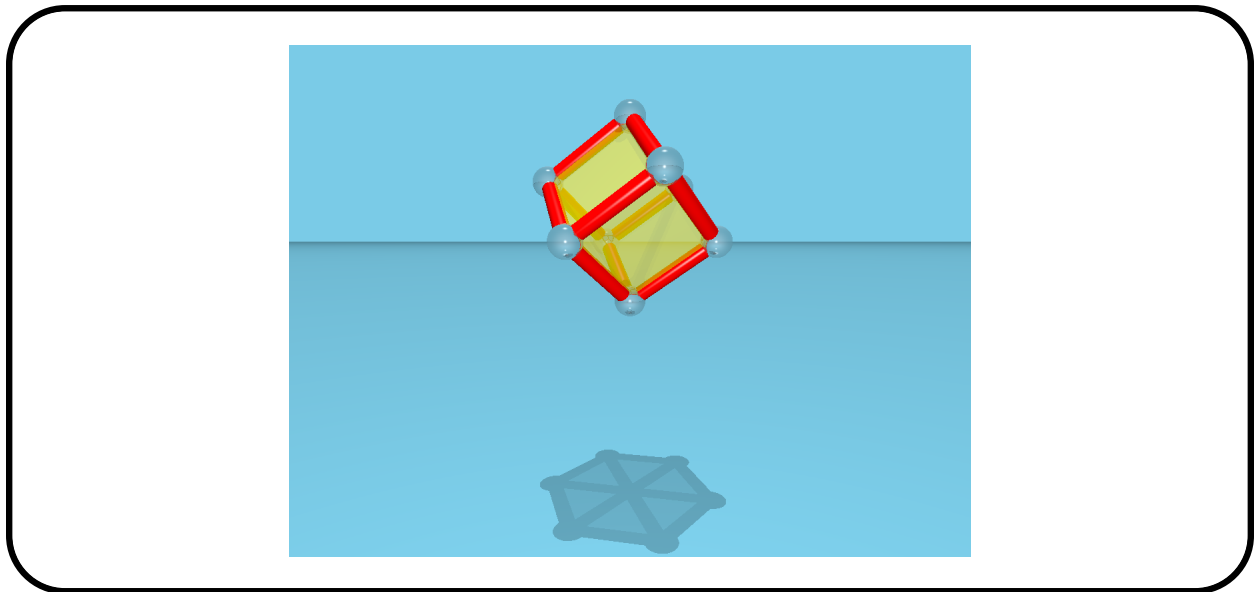
Table 19.9: Scene Object Types

```

28  ▷ ▷ ▷ -prisms 1 0.05 $(YELLOW_TRANSPARENT) \
29  ▷ ▷ ▷ -group_of_things_as_interval 0 12 \
30  ▷ ▷ ▷ -cylinders 2 0.15 $(COLOR_RED) \
31  ▷ ▷ -scene_objects_end \
32  ▷ ▷ -povray_end
33  ▷ - rm -rf POV
34  ▷ mkdir POV
35  ▷ mv cube_0_*.pov POV
36  ▷ mv makefile_animation POV
37

```

This command instructs Orbiter to create 30 povray files (extension .pov), one for each frame of a rotating scene. The scene contains a cube whose vertices are shown in chrome, whose edges are in red, and whose faces are yellow and transparent. The cube turns around a vertical axis of symmetry. Here is the first frame of the result:



The coordinates of the cube are stored in an object file `cube_centered.obj`. The content of this file is:

```

v -1 -1 -1
v 1 -1 -1
v -1 1 -1
v 1 1 -1
v -1 -1 1
v 1 -1 1
v -1 1 1
v 1 1 1
f 1 2 4 3
f 1 2 6 5
f 1 3 7 5
f 2 4 8 6
f 3 4 8 7

```

f 5 6 8 7

The monkey saddle is a cubic surface, given by the equation

$$z = x^3 - 3xy^2$$

The next example plots the surface known as the monkey saddle. The tangent plane at $(0, 0, 0)$ is drawn as well. An animation is created by rotating the scene around the z -axis.

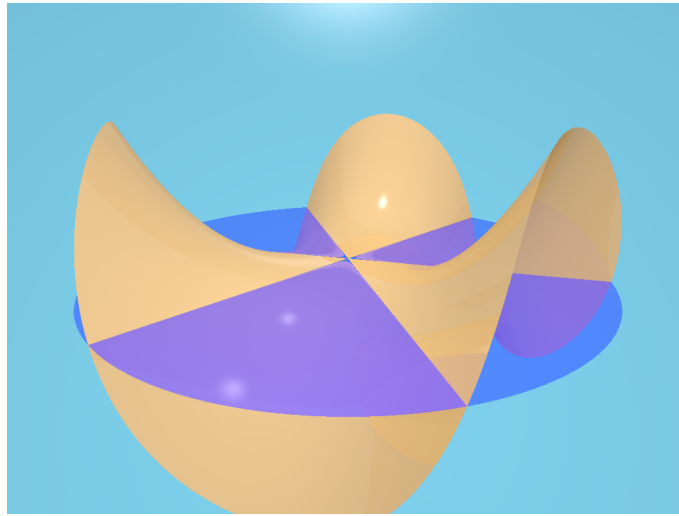
Code Snippet 889

```
1 MONKEY_SADDLE_CUBIC="1,0,0,0,-3,0,0,0,0,0,0,0,0,0,0,0,-1,0"
2
```

Code Snippet 890

```
1 monkey:
2   ▷ $(ORBITER) -v 2 -povray \
3   ▷ ▷ -round 0 -nb_frames_default 30 \
4   ▷ ▷ -output_mask monkey_%d_%03d.pov \
5   ▷ ▷ -video_options -W 1024 -H 768 \
6   ▷ ▷ -global_picture_scale 0.8 \
7   ▷ ▷ -default_angle 75 \
8   ▷ ▷ -clipping_radius 0.8 \
9   ▷ ▷ -camera 0 "0,0,1" "1,1,0.5" "0,0,0" \
10  ▷ ▷ -rotate_about_z_axis \
11  ▷ ▷ -end \
12  ▷ ▷ -scene_objects \
13  ▷ ▷ ▷ -cubic_lex $(MONKEY_SADDLE_CUBIC) \
14  ▷ ▷ ▷ -plane_by_dual_coordinates "0,0,1,0" \
15  ▷ ▷ ▷ -group_of_things "0" \
16  ▷ ▷ ▷ -group_of_things "0" \
17  ▷ ▷ ▷ -cubics 0 $(COLOR_GOLD) \
18  ▷ ▷ ▷ -planes 1 $(COLOR_BLUE) \
19  ▷ ▷ -scene_objects_end \
20  ▷ ▷ -povray_end
21  ▷ - rm -rf POV
22  ▷ mkdir POV
23  ▷ mv monkey_0_*.pov POV
24  ▷ mv makefile_animation POV
25
```

Here is one of the frames that are created:



The Eckardt surface is given by the equation

$$\frac{5}{2}xyz - (x^2 + y^2 + z^2) + 1 = 0.$$

We use the following code to plot the surface and the lines on it. The Schläfli labeling of the lines is indicated.

Code Snippet 891

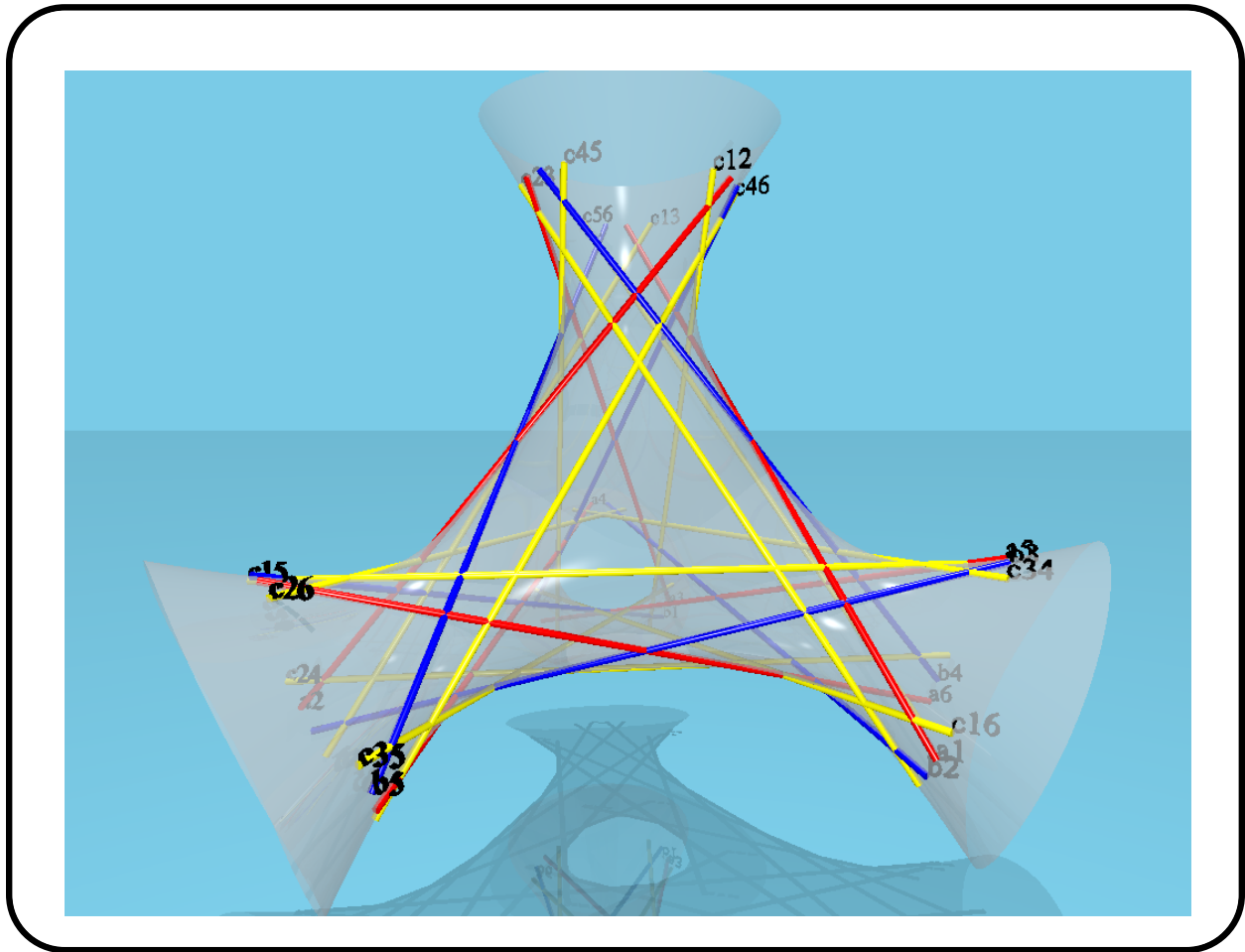
```

1   Eckardt:
2   ▷ $(ORBITER) -v 2 -povray \
3   ▷ ▷ -round 0 -nb_frames_default 30 \
4   ▷ ▷ -output_mask Eckardt_%d_%03d.pov \
5   ▷ ▷ -video_options -W 1024 -H 768 \
6   ▷ ▷ -global_picture_scale 0.9 \
7   ▷ ▷ -default_angle 75 \
8   ▷ ▷ -clipping_radius 2.4 \
9   ▷ ▷ -camera 0 "1,1,1" "-3,1,3" "0.12,0.12,0.12" \
10  ▷ ▷ -end \
11  ▷ ▷ -scene_objects \
12  ▷ ▷ ▷ -Hilbert_Cohn_Vossen_surface \
13  ▷ ▷ ▷ -group_of_things "0" \
14  ▷ ▷ ▷ -cubics 0 $(SURFACE_COLOR) \
15  ▷ ▷ ▷ -group_of_things_as_interval 0 6 \
16  ▷ ▷ ▷ -group_of_things_as_interval 6 6 \
17  ▷ ▷ ▷ -group_of_things_as_interval_with_exceptions 12 15 \
18  ▷ ▷ ▷ ▷ "14,19,23" \
19  ▷ ▷ ▷ -lines 1 0.02 $(COLOR_RED_SHINY) \
20  ▷ ▷ ▷ -lines 2 0.02 $(COLOR_BLUE_SHINY) \
21  ▷ ▷ ▷ -lines 3 0.02 $(COLOR_YELLOW_SHINY) \
22  ▷ ▷ ▷ -label 0 "a1" \
23  ▷ ▷ ▷ -label 2 "a2" \
24  ▷ ▷ ▷ -label 4 "a3" \
25  ▷ ▷ ▷ -label 6 "a4" \
26  ▷ ▷ ▷ -label 8 "a5" \
27  ▷ ▷ ▷ -label 10 "a6" \

```

```
28  ▷ ▷ ▷ -label 12 "b1" \
29  ▷ ▷ ▷ -label 14 "b2" \
30  ▷ ▷ ▷ -label 16 "b3" \
31  ▷ ▷ ▷ -label 18 "b4" \
32  ▷ ▷ ▷ -label 20 "b5" \
33  ▷ ▷ ▷ -label 22 "b6" \
34  ▷ ▷ ▷ -label 24 "c12" \
35  ▷ ▷ ▷ -label 26 "c13" \
36  ▷ ▷ ▷ -label 30 "c15" \
37  ▷ ▷ ▷ -label 32 "c16" \
38  ▷ ▷ ▷ -label 34 "c23" \
39  ▷ ▷ ▷ -label 36 "c24" \
40  ▷ ▷ ▷ -label 40 "c26" \
41  ▷ ▷ ▷ -label 42 "c34" \
42  ▷ ▷ ▷ -label 44 "c35" \
43  ▷ ▷ ▷ -label 48 "c45" \
44  ▷ ▷ ▷ -label 50 "c46" \
45  ▷ ▷ ▷ -label 52 "c56" \
46  ▷ ▷ ▷ -group_of_things.as_interval 0 6 \
47  ▷ ▷ ▷ -texts 4 0.2 0.15 $(COLOR.BLACK.NO.SHADOW) \
48  ▷ ▷ ▷ -group_of_things.as_interval 6 6 \
49  ▷ ▷ ▷ -texts 5 0.2 0.15 $(COLOR.BLACK.NO.SHADOW) \
50  ▷ ▷ ▷ -group_of_things.as_interval 12 12 \
51  ▷ ▷ ▷ -texts 6 0.2 0.15 $(COLOR.BLACK.NO.SHADOW) \
52  ▷ ▷ -scene_objects_end \
53  ▷ ▷ -povray_end
54  ▷ - rm -rf POV
55  ▷ mkdir POV
56  ▷ mv Eckardt_0_*.pov POV
57  ▷ mv makefile_animation POV
58
```

The rendering yields:



The Endrass octic [30] is the algebraic surface given by the equation

$$X8 := 64 (-w^2 + x^2) (-w^2 + y^2) ((x+y)^2 - 2w^2) ((x-y)^2 - 2w^2) - (-4(1 + \sqrt{2})(x^2 + y^2)^2 + (8(2 + \sqrt{2})z^2 + 2(2 + 7\sqrt{2})w^2)(x^2 + y^2) - 16z^4 + 8(1 - 2\sqrt{2})z^2w^2 - (1 + 12\sqrt{2})w^4)^2$$

The following Orbiter command creates a povray graphics of the octic.

Code Snippet 892

```
1  ENDRASS_OCTIC_LEX_165="-93.2548,0,0,0,-309.019,0,0,527.529,0,395.647,\
2  0,0,0,0,0,0,0,0,0,0,-687.529,0,0,1582.59,0,1186.94,0,0,0,0,-1055.06,0,\
3  -1582.59,0,-593.47,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,-309.019,\
4  0,0,1582.59,0,1186.94,0,0,0,0,-2110.12,0,-3165.17,0,-1186.94,0,0,0,0,0,\
5  0,874.039,0,1560.63,0,1677.92,0,343.362,0,0,0,0,0,0,0,0,0,0,0,0,0,\
6  0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,-93.2548,0,0,527.529,0,395.647,\
7  0,0,0,0,-1055.06,0,-1582.59,0,-593.47,0,0,0,0,0,0,874.039,0,1560.63,0,\
8  1677.92,0,343.362,0,0,0,0,0,0,0,0,-256,0,-468.077,0,-789.019,0,\
9  -525.726,0,0.941125"
10
```

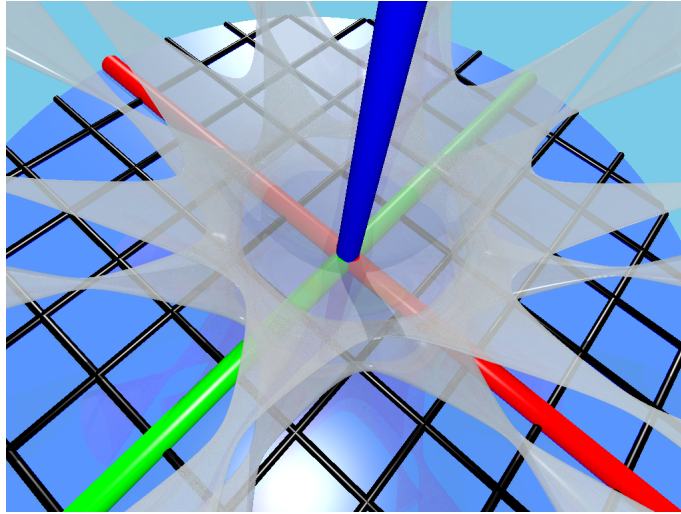
Code Snippet 893

```

1   endrass8:
2   ▷ cp $(ORBITER_PATH)/examples/users_guide/coordinate_grid.csv .
3   ▷ $(ORBITER) -v 2 -povray \
4   ▷ ▷ -round 0 -nb_frames_default 30 \
5   ▷ ▷ -output_mask endrass_octic.%d.%03d.pov \
6   ▷ ▷ -video_options -W 1024 -H 768 \
7   ▷ ▷ -global_picture_scale 0.75 \
8   ▷ ▷ -default_angle 75 \
9   ▷ ▷ -clipping_radius 3.7 \
10  ▷ ▷ -no_bottom_plane 0 \
11  ▷ ▷ -camera 0 "1,1,1" "6,6,3" "0,0,0" \
12  ▷ ▷ -rotate_about_111 \
13  ▷ ▷ -end \
14  ▷ ▷ -scene_objects \
15  ▷ ▷ ▷ -line_through_two_points_recentered_from_csv_file \
16  ▷ ▷ ▷ ▷ coordinate_grid.csv \
17  ▷ ▷ ▷ -group_of_things "0" \
18  ▷ ▷ ▷ -group_of_things "1" \
19  ▷ ▷ ▷ -group_of_things "2" \
20  ▷ ▷ ▷ -group_of_things_as_interval 3 39 \
21  ▷ ▷ ▷ -lines 0 0.15 $(COLOR_RED_SHINY) \
22  ▷ ▷ ▷ -lines 1 0.15 $(COLOR_GREEN_SHINY) \
23  ▷ ▷ ▷ -lines 2 0.15 $(COLOR_BLUE_SHINY) \
24  ▷ ▷ ▷ -lines 3 0.05 $(COLOR_BLACK_SHINY) \
25  ▷ ▷ ▷ -octic_lex_165 $(ENDRASS_OCTIC_LEX_165) \
26  ▷ ▷ ▷ -plane_by_dual_coordinates "0,0,1,0" \
27  ▷ ▷ ▷ -group_of_things "0" \
28  ▷ ▷ ▷ -group_of_things "0" \
29  ▷ ▷ ▷ -octics 4 $(SURFACE_COLOR_SEETHROUGH) \
30  ▷ ▷ ▷ -planes 5 "texture{ pigment{ color Blue transmit 0.5 } \
31  finish { diffuse 0.9 phong 1}}" \
32  ▷ ▷ -scene_objects_end \
33  ▷ ▷ -povray_end
34  ▷ - rm -rf POV
35  ▷ mkdir POV
36  ▷ mv endrass_octic_0_*.pov POV
37  ▷ mv makefile_animation POV
38

```

Here is the rendering:



It includes coordinate axes and the x, y -plane.

Preparing Frames for an Animation		
Command	Arguments	Purpose
<code>-i</code>	<code>s l mask</code>	Specify the input file names by running a <code>printf</code> command with the given mask applied to the index i where i loops over all integers from s to $s + l - 1$. This option can be repeated.
<code>-step</code>	<code>s</code>	Increment the index in steps of size s .
<code>-o</code>	<code>mask</code>	Create the output file using the given mask.
<code>-output_starts_at</code>	<code>i</code>	Start output file indices at i (default is 0).

Table 19.10: Preparing Frames for an Animation

19.3 Creating Animations

Orbiter can be used to create animations. This relies on the software `ffmpeg`. In a first step, all frames (i.e. individual graphics files) are created using Orbiter's `povray` interface. After that, the frames are used to create the animation. In order to use `ffmpeg`, the frames should have a uniform file naming scheme, using a consecutive numbering to arrange the files in order. This is achieved by using a `printf` style mask, with `%d` representing the number of the current frame. In order to do so, Orbiter can be used to copy and rename files. A temporary directory can be used to collect the files. The Orbiter command `prepare_frames` can be used. For a list of commands, see Tables 19.10. For instance, the command

Code Snippet 894

```

1  monkey_video:
2  ▷ - rm -r FRAMES
3  ▷ - mkdir FRAMES
4  ▷ - rm monkey.mp4
5  ▷ $(ORBITER) \
6  ▷ ▷ -prepare_frames \
7  ▷ ▷ ▷ -i 0 30 monkey_0_%03d.png \
8  ▷ ▷ ▷ -output_starts_at 0 \
9  ▷ ▷ ▷ -o FRAMES/frame%04d.png \
10 ▷ ▷ -end
11 ▷ ffmpeg -r 5 -f image2 -i FRAMES/frame%04d.png \
12 ▷ ▷ -f mp4 -q:v 0 -vcodec mpeg4 monkey.mp4
13
```

creates a video `monkey.mp4` from a set of 30 files. The individual filenames are created using the `printf` format string `monkey_0_%03d.png`, with an integer index that is drawn from the interval $[0, 29]$. The part that starts with a percent sign and ends with a “d” character defines the way in which the integer is formatted. The number three before the “d” indicates that three characters will be printed. The zero indicates the use of leading zeros. So, the first file would be `monkey_0_000.png` and the very last file is `monkey_0_029.png`. The description of the `printf` format string can be found in the documentation of the C standard library [45].

19.4 Continuous Function Plotter

Orbiter can plot functions using a built-in function tracker. The functions must be continuous apart from a finite number of poles. The function can have multiple components, each described using an expression. Each expression is specified in Reverse Polish Notation (RPN). Consider an example. A Lissajous curve is defined using coordinate functions of the form

$$x = r \sin(at + c), \quad y = r \sin(bt), \quad a, b, c, r \in \mathbb{R}.$$

The terms

$$r \sin(at + c), \quad r \sin(bt)$$

are the expressions of the two coordinate functions. RPN means that the operator is listed after the operands. A stack data structure is used to hold temporary values. Operators are pushed to the top of the stack using the push commands. A binary operator pops the two elements from the stack, performs the operation, and pushes the resulting value back onto the stack. For a unary operator, only one element is popped and replaced by the result. Here are some examples of expressions rewritten in RPN:

$$\begin{aligned} \sin(x) &\mapsto \text{push } x \text{ sin,} \\ a + b &\mapsto \text{push } a \text{ push } b \text{ add,} \\ a \cdot b &\mapsto \text{push } a \text{ push } b \text{ mult.} \end{aligned}$$

The coordinate functions are enclosed between `-code` and `-code_end` commands. Each coordinate function is described in RPN and terminated using a `return` keyword. By the time the `return` keyword is reached, the RPN expression must have exactly one value on the stack which is considered the value of the expression. Constants are declared between the `-const` and `-const_end` keywords. Likewise, variables are declared between the `-var` and `-var_end` keywords. Picking $a = 3$, $b = 2$, $c = \pi/2$ and $r = 7$, the function is computed using

Code Snippet 895

```

1  lissajous:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -smooth_curve "lissajous" 0.07 2000 15 0 18.85 \
4  ▷ ▷ -const a 3 b 2 c 1.57 r 7 -const_end \
5  ▷ ▷ -var t -var_end \
6  ▷ ▷ -code \
7  ▷ ▷ ▷ push t push a mult push c add sin push r mult return \
8  ▷ ▷ ▷ push t push b mult sin push r mult return \
9  ▷ ▷ -code_end \
10
```

The sequence

push t push a mult push c add sin push r mult

is $r \sin(at + c)$ expressed in RPN. The constants are defined in the line

-const a 3 b 2 c 1.57 r 7 -const_end

The input variable is defined using the line

-var t -var_end

The sequence

-smooth_curve "lissajous" 0.07 2000 15 0 18.85

defines the name of the output file, the fact that two consecutive points are never further than $\epsilon = 0.07$ away, the fact that points that are 15 or more away from the origin should be ignored, and the fact that the variable t loops over the range $[0, 18.85]$ with a default of 2000 steps. The evaluator automatically reduces the step-size if consecutive image points are more than ϵ apart. The code to produce the plot is

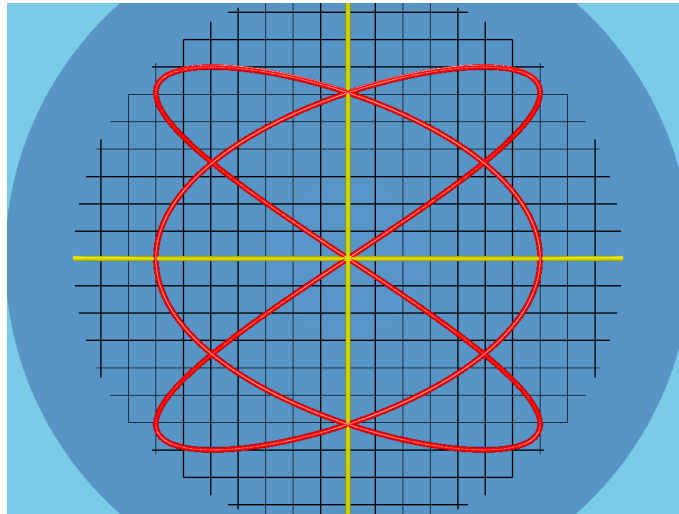
Code Snippet 896

```

1    lissajous_plot:
2    ▷ cp $(ORBITER_PATH)/examples/users_guide/coordinate_grid.csv .
3    ▷ $(ORBITER) -v 2 -povray \
4    ▷ ▷ -round 0 -nb_frames_default 1 \
5    ▷ ▷ -output_mask lissajous_%d%03d.pov \
6    ▷ ▷ -video_options -W 1024 -H 768 \
7    ▷ ▷ -global_picture_scale 0.40 \
8    ▷ ▷ -default_angle 45 \
9    ▷ ▷ -clipping_radius 5 \
10   ▷ ▷ -omit_bottom_plane \
11   ▷ ▷ -camera 0 "0,-1,0" "0,0,12" "0,0,0" \
12   ▷ ▷ -rotate_about_z_axis \
13   ▷ ▷ -end \
14   ▷ ▷ -scene_objects \
15   ▷ ▷ ▷ -line_through_two_points_recentered_from_csv_file \
16   ▷ ▷ ▷ coordinate_grid.csv \
17   ▷ ▷ ▷ -group_of_things "0" \
18   ▷ ▷ ▷ -group_of_things "1" \
19   ▷ ▷ ▷ -group_of_things "2" \
20   ▷ ▷ ▷ -lines 0 0.09 "texture{ pigment{ color Yellow } }" \
21   ▷ ▷ ▷ -lines 1 0.09 "texture{ pigment{ color Yellow } }" \
22   ▷ ▷ ▷ -lines 2 0.09 "texture{ pigment{ color Yellow } }" \
23   ▷ ▷ ▷ -group_of_things_as_interval 3 39 \
24   ▷ ▷ ▷ -lines 3 0.02 "texture{ pigment{ color Black } }" \
25   ▷ ▷ ▷ -point_list_from_csv_file \
26   ▷ ▷ ▷ function_lissajous_N2000_points.csv \
27   ▷ ▷ ▷ -group_of_things_as_interval 0 6524\
28   ▷ ▷ ▷ -spheres 4 0.1 "texture{ pigment{ color Red } }" \
29   finish { diffuse 0.9 phong 1}}" \
30   ▷ ▷ ▷ -plane_by_dual_coordinates "0,0,1,0" \
31   ▷ ▷ ▷ -group_of_things "0" \
32   ▷ ▷ ▷ -planes 5 "texture{ pigment{ color Blue*0.5 \
33   transmit 0.5 } }" \
34   ▷ ▷ -scene_objects_end \
35   ▷ ▷ -povray_end
36   ▷ - rm -rf POV
37   ▷ mkdir POV
38   ▷ mv lissajous_0_*.pov POV
39   ▷ mv makefile_animation POV
40

```

The plot is shown below



We can turn it into a 3D plot by using the t value for the z coordinate. The function is computed using the command

Code Snippet 897

```

1  lissajous_3d:
2  ▷ $(ORBITER) -v 2 \
3  ▷ ▷ -smooth_curve "lissajous_3d" 0.07 2000 50 0 18.85 \
4  ▷ ▷ -const a 3 b 2 c 1.57 r 7 -const_end \
5  ▷ ▷ -var t -var_end \
6  ▷ ▷ -code \
7  ▷ ▷ ▷ push t push a mult push c add sin push r mult return \
8  ▷ ▷ ▷ push t push b mult sin push r mult return \
9  ▷ ▷ ▷ push t return \
10 ▷ ▷ -code_end \
11
```

The code to produce the 3D plot is

Code Snippet 898

```

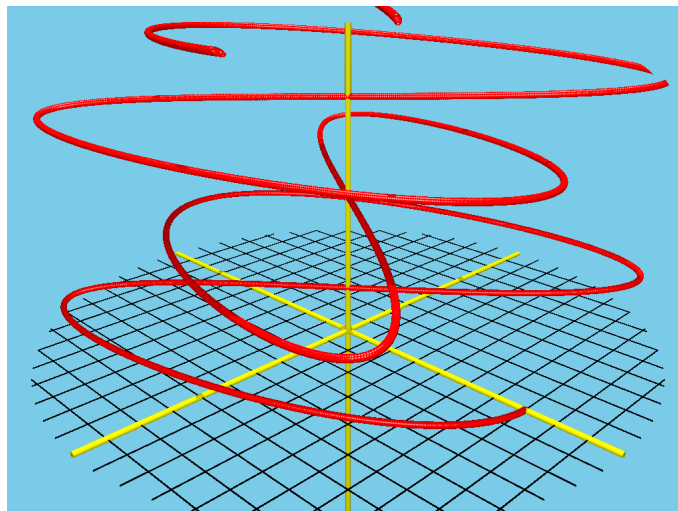
1  lissajous_3d_plot:
2  ▷ cp $(ORBITER_PATH)/examples/users_guide/coordinate_grid.csv .
3  ▷ $(ORBITER) -v 2 -povray \
4  ▷ ▷ -round 0 -nb_frames_default 30 \
5  ▷ ▷ -output_mask lissajous_3d_%d%03d.pov \
6  ▷ ▷ -video_options -W 1024 -H 768 \
7  ▷ ▷ -global_picture_scale 0.40 \
8  ▷ ▷ -default_angle 45 \
9  ▷ ▷ -clipping_radius 5 \
10 ▷ ▷ -omit_bottom_plane \
11 ▷ ▷ -camera 0 "0,0,1" "7,7,5" "0,0,1" \
12 ▷ ▷ -rotate_about_z_axis \
```

```

13  ▷ ▷ -end \
14  ▷ ▷ -scene_objects \
15  ▷ ▷ ▷ -line_through_two_points_recentered_from_csv_file \
16  ▷ ▷ ▷ coordinate_grid.csv \
17  ▷ ▷ ▷ -group_of_things "0" \
18  ▷ ▷ ▷ -group_of_things "1" \
19  ▷ ▷ ▷ -group_of_things "2" \
20  ▷ ▷ ▷ -lines 0 0.09 "texture{ pigment{ color Yellow } }" \
21  ▷ ▷ ▷ -lines 1 0.09 "texture{ pigment{ color Yellow } }" \
22  ▷ ▷ ▷ -lines 2 0.09 "texture{ pigment{ color Yellow } }" \
23  ▷ ▷ ▷ -group_of_things_as_interval 3 39 \
24  ▷ ▷ ▷ -lines 3 0.02 "texture{ pigment{ color Black } }" \
25  ▷ ▷ ▷ -point_list_from_csv_file \
26  ▷ ▷ ▷ function_lissajous_3d_N2000_points.csv \
27  ▷ ▷ ▷ -group_of_things_as_interval 0 6538\
28  ▷ ▷ ▷ -spheres 4 0.1 "texture{ pigment{ color Red } }" \
29  finish { diffuse 0.9 phong 1}}" \
30  ▷ ▷ ▷ -plane_by_dual_coordinates "0,0,1,0" \
31  ▷ ▷ ▷ -group_of_things "0" \
32  ▷ ▷ -scene_objects_end \
33  ▷ ▷ -povray_end
34  ▷ - rm -rf POV
35  ▷ mkdir POV
36  ▷ mv lissajous_3d_0_*.pov POV
37  ▷ mv makefile_animation POV
38

```

The 3D curve is shown below



19.5 The Gnuplot Interface

Orbiter can interface with gnuplot [34] to draw diagrams. Here is an example. Suppose we have data in a csv file like this:

```
Row, Curve-1, Curve-2
1, 2, 4
2, 6, 8
3, 9, 10
4, 13, 17
5, 17, 20
6, 19, 25
7, 23, 30
END
```

The following command creates this csv file from a makefile variable

Code Snippet 899

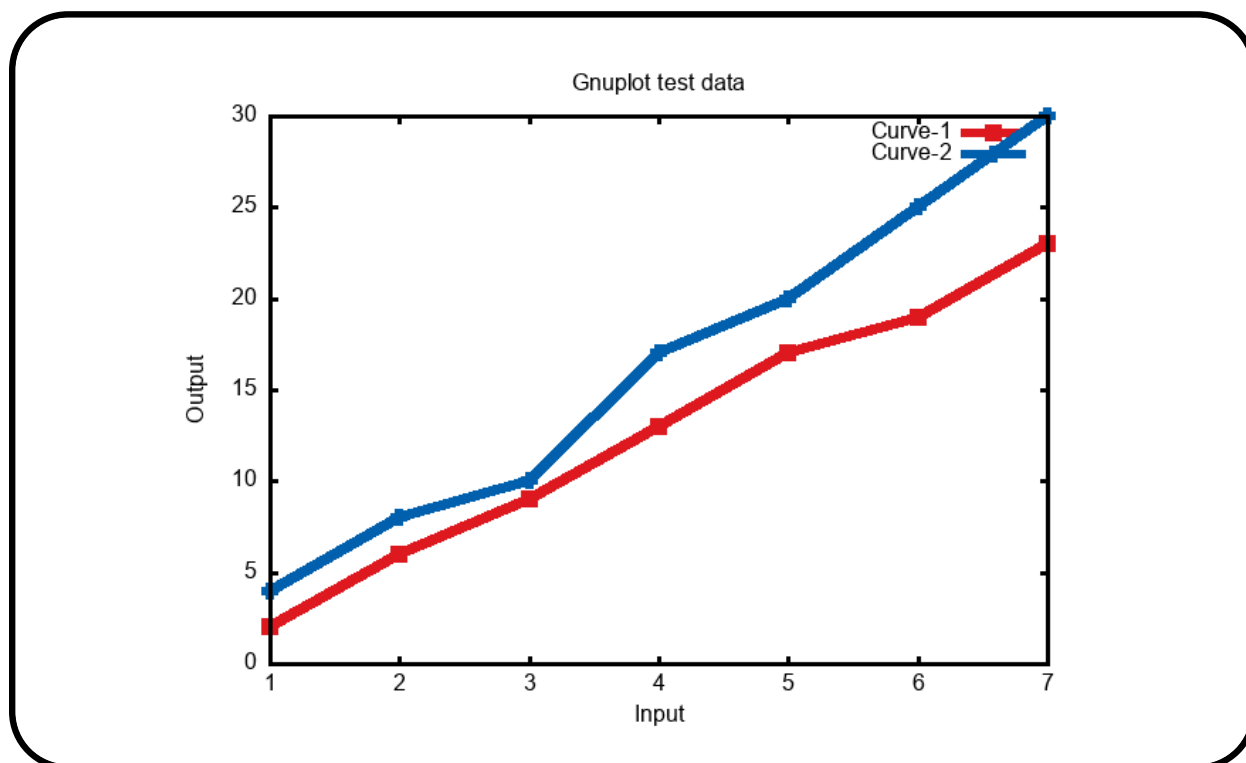
```
1  GNUPLOT_TEST_INPUT="Row, Curve-1, Curve-2\n1, 2, 4\n2, 6, 8\n3, 9, 10\n4, 13, 17\n5, 17, 20\n
2  6, 19, 25\n7, 23, 30\nEND\n"
```

The next command invokes the Orbiter gnuplot interface:

Code Snippet 900

```
1  gnuplot_test_data:
2  ▷ echo $(GNUPLOT_TEST_INPUT) >gnuplot_test_data.csv
3  ▷ $(ORBITER) -v 3 \
4  ▷ ▷ -gnuplot gnuplot_test_data.csv \
5  ▷ ▷ "Gnuplot test data" \
6  ▷ ▷ "Input" \
7  ▷ ▷ "Output"
8  ▷ $(OPEN) gnuplot_test_data_gnuplot.png
9
```

The resulting plot created by gnuplot is:



The next command plots the Eulerfunction, the number of prime divisors and the number of distinct prime divisors of all integers n between 1 and 50:

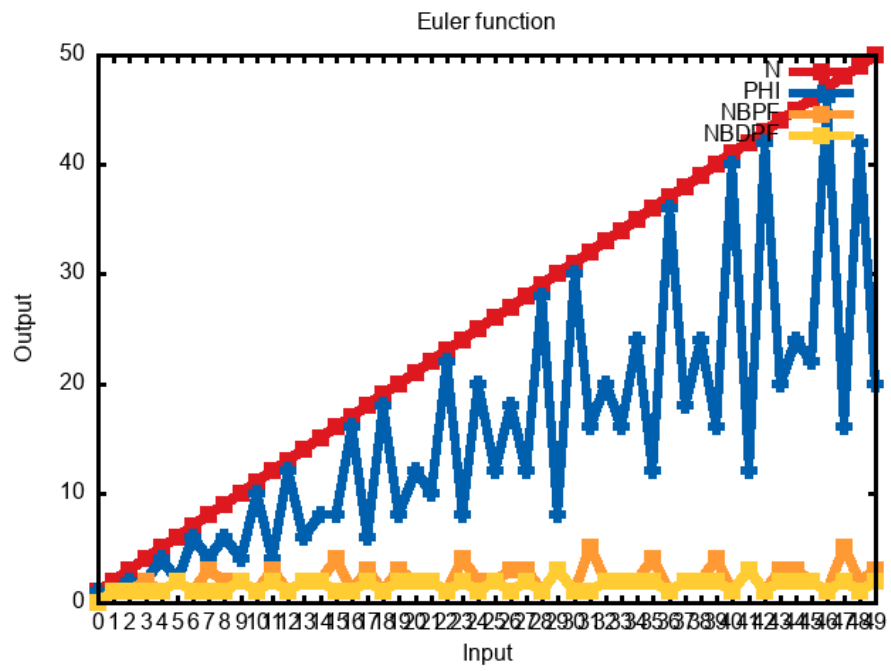
Code Snippet 901

```

1  gnuplot_Eulerfunction_50:
2  ▷ $(ORBITER) -v 1 -eulerfunction_interval 1 50
3  ▷ $(ORBITER) -v 3 \
4  ▷ ▷ -gnuplot table_eulerfunction_1_50.csv \
5  ▷ ▷ "Euler function" \
6  ▷ ▷ "Input" \
7  ▷ ▷ "Output"
8

```

The resulting plot is:



Chapter 20

Mathematical Data in Orbiter

20.1 Cubic Surfaces

Orbiter contains a knowledge base of mathematical data. This is like a database, except for the way that the data is stored. The data does not reside in files, like in a database. Rather, the data is compiled into the Orbiter executable. This means that the data is immediately available, without any delay. We will now discuss what data exists in Orbiter and how to access it.

In most cases, the data that Orbiter stores includes the automorphism group of the objects. Individual objects can be accessed by creating them from the catalogue. Other functions exist that can write reports about all objects of a certain type.

The command

Code Snippet 902

```
1   make_table_of_surfaces:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -make_table_of_surfaces
4   ▷ $(PDFLATEX) surfaces_report.tex
5   ▷ $(OPEN) surfaces_report.pdf
6
```

creates a table of all cubic surfaces contained in the Orbiter knowledge base. The output is shown in Appendix B.1.

The command

Code Snippet 903

```
1   make_table_of_quartic_curves:
2   ▷ $(ORBITER) -v 3 \
3   ▷ ▷ -make_table_of_quartic_curves
4   ▷ $(PDFLATEX) quartic_curves_report.tex
5   ▷ $(OPEN) quartic_curves_report.pdf
6
```

creates a table of all quartic curves contained in the Orbiter knowledge base.

It is possible to export data on cubic surfaces and quartic curves. Two file types are supported: csv and sql. For instance, the command

Code Snippet 904

```

1    cubic_surfaces_table.q17:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 17 -end \
4    ▷ ▷ -define P -projective_space \
5    ▷ ▷ ▷ -n 3 -field F -v 0 \
6    ▷ ▷ -end \
7    ▷ ▷ -with P -do \
8    ▷ ▷ ▷ -projective_space_activity \
9    ▷ ▷ ▷ ▷ -table_of_cubic_surfaces \
10   ▷ ▷ -end
11

```

exports the cubic surfaces over the field $\mathbb{F}_{17}$ contained in the Orbiter knowledge base. The files `table_of_cubic_surfaces_q17_info.csv` and `table_of_cubic_surfaces_q17_data.sql` are created. For each isomorphism class of cubic surfaces, one line is generated in the file. The command

Code Snippet 905

```

1    quartic_curves_table.q17:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 17 -end \
4    ▷ ▷ -define P -projective_space \
5    ▷ ▷ ▷ -n 2 -field F -v 0 \
6    ▷ ▷ -end \
7    ▷ ▷ -with P -do \
8    ▷ ▷ ▷ -projective_space_activity \
9    ▷ ▷ ▷ ▷ -table_of_quartic_curves \
10   ▷ ▷ -end
11

```

exports all quartic curves over the field $\mathbb{F}_{17}$

20.2 BLT-Sets

Orbiter contains a library of BLT-sets. It is possible to export data on BLT-sets to csv-files. For instance, the command

Code Snippet 906

```

1    blt_set_table.q13:
2    ▷ $(ORBITER) -v 3 \
3    ▷ ▷ -define F -finite_field -q 13 -end \
4    ▷ ▷ -define O -orthogonal_space 0 5 F -create_extension_fields -end \
5    ▷ ▷ -with O -do \
6    ▷ ▷ ▷ -orthogonal_space_activity \
7    ▷ ▷ ▷ ▷ -table_of_blt_sets \
8    ▷ ▷ -end
9

```

exports the BLT-sets over the field $\mathbb{F}_{13}$ contained in the Orbiter knowledge base.

The command

Code Snippet 907

```

1    blt_set_table:
2    ▷ $(ORBITER) -v 30 \
3    ▷ ▷ -define Q -vector -dense "3,5,7,9,11,13,17,19,23,25,27,29,31" -end \
4    ▷ ▷ -define NB -vector -dense "1,2,2,3,4,3,6,5,9,6,6,9,8" -end \
5    ▷ ▷ -loop_over i Q \
6    ▷ ▷ ▷ -define F%i -finite_field -q "%i[Q]" -end \
7    ▷ ▷ ▷ -define O%i -orthogonal_space 0 5 F%i -create_extension_fields -end \
8    ▷ ▷ ▷ -with O%i -do \
9    ▷ ▷ ▷ ▷ -orthogonal_space_activity \
10   ▷ ▷ ▷ ▷ ▷ -table_of_blt_sets \
11   ▷ ▷ ▷ ▷ -end \
12   ▷ ▷ ▷ -end \
13   ▷ ▷ -end_loop_over i \
14   ▷ ▷ -print_symbols
15

```

creates tables of BLT-sets for all orders up to 31.

The command

Code Snippet 908

```

1    BLT_ORDER_Q="3,5,7,9,11,13,17,19,23,25,27,\
2    29,31,37,41,43,47,49,53,59,61,67,71,73"
3

```

Code Snippet 909

```

1    BLT_NUMBER_ISO="1,2,2,3,4,3,6,5,9,6,6,\
2    9,8,7,10,6,10,8,8,9,5,6,8,5"

```

3

Code Snippet 910

```

1  blt_set_export_gap:
2  ▷ $(ORBITER) -v 30 \
3  ▷ ▷ -define Q -vector -dense \
4  ▷ ▷ ▷ $(BLT_ORDER.Q) \
5  ▷ ▷ -end \
6  ▷ ▷ -define NB -vector -dense \
7  ▷ ▷ ▷ $(BLT_NUMBER.ISO) \
8  ▷ ▷ -end \
9  ▷ ▷ -loop_over i Q \
10 ▷ ▷ ▷ -define F%i -finite_field -q "%i[Q]" -end \
11 ▷ ▷ ▷ -define O%i -orthogonal_space 0 5 F%i -create_extension_fields -end \
12 ▷ ▷ ▷ -loop j 0 %i[NB] 1 \
13 ▷ ▷ ▷ ▷ -define BLT_%i_%j -BLT_set \
14 ▷ ▷ ▷ ▷ ▷ -space O%i -catalogue %j \
15 ▷ ▷ ▷ ▷ -end \
16 ▷ ▷ ▷ ▷ -with BLT_%i_%j -do -blt_set_activity \
17 ▷ ▷ ▷ ▷ ▷ -export_gap \
18 ▷ ▷ ▷ ▷ -end \
19 ▷ ▷ ▷ -end_loop j \
20 ▷ ▷ -end_loop_over i \
21 ▷ ▷ -print_symbols
22

```

exports all BLT-sets of order at most 73 to GAP.

Chapter 21

Miscellaneous

21.1 Miscellaneous

Tables 21.1-21.3 list miscellaneous Orbiter commands.

The command `-csv_file_select_rows` can be used to select rows from a csv file. The command `-csv_file_select_cols` can be used to select columns from a csv file. The command `-csv_file_select_rows_and_cols` selects rows and columns. Here is an example. We create the multiplication table of the finite field $\mathbb{F}_7$, ordered according to the powers of a primitive element:

$$\alpha^0, \alpha^1, \alpha^2, \alpha^3, \alpha^4, \alpha^5.$$

The even powers of α create a multiplicative subgroup. We select the rows and columns corresponding to even powers $\alpha^0, \alpha^2, \alpha^4$.

Code Snippet 911

```
1  misc.select:
2  ▷ $(ORBITER) -v 3 \
3  ▷ ▷ -define F -finite_field -q 7 -end \
4  ▷ ▷ -with F -do -finite_field_activity \
5  ▷ ▷ ▷ -export_tables \
6  ▷ ▷ -end
7  ▷ $(ORBITER) -v 3 \
8  ▷ ▷ -define F -finite_field -q 7 -end \
9  ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
10 ▷ $(ORBITER) -v 4 -csv_file_select_rows_and_cols \
11 ▷ ▷ GF_q7_table_mul_r.csv \
12 ▷ ▷ "0,2,4" "0,2,4"
13
```

Here is the file `GF_q7_multiplication_table_reordered.csv`

```
Row,C0,C1,C2,C3,C4,C5
0,1,3,2,6,4,5
1,3,2,6,4,5,1
2,2,6,4,5,1,3
3,6,4,5,1,3,2
4,4,5,1,3,2,6
```

Miscellaneous Orbiter Commands (Part 1)		
Command	Arguments	Purpose
-create_files_direct	fname_mask content_mask list-of-labels	Create text files using a file name mask and a list of labels. The list of labels must terminate in -end. The file will contain the text obtained after creating content through content_mask.
-create_files	descr	Create text files.
-save_matrix_csv	label	Save matrix to a csv file.
-csv_file_tally	fname	Tally a csv file.
-tally_column	fname col- umn	Tally the contents of the given column in the given csv file. The column contains integer vector data. The data has constant size.
-collect_stats	fname_mask fname_out from last step	Gather data from a list of files. Specifically, count the number of rows in each file.
-csv_file_select_rows	fname R	Selects rows listed in R from the csv-file fname.
-csv_file_select_rows_by_file	fname R	Selects rows listed in R from the csv-file fname.
-csv_file_select_rows_complement	fname R	Selects rows listed in R from the csv-file fname.
-csv_file_split_rows_modulo	fname n	Splits the rows from the csv-file fname modulo n .
-csv_file_select_cols	fname R	Selects columns listed in R from the csv-file fname.
-csv_file_select_cols_by_label	fname suffix R	Selects columns listed in R from the csv-file fname. Appends the given suffix to the output file name.
-csv_file_select_rows_and_cols	fname R C	Selects rows listed in R and columns listed in C from the csv-file fname.
-csv_file_sort_each_row	fname	Sorts each individual row in the given csv file, thinking of the entries as a set.
-csv_file_sort_rows	fname	Sorts the rows in the given csv file.
-csv_file_sort_rows_and_remove_duplicates	fname	Sorts the rows in the given csv file. Removes duplicate rows.

Table 21.1: Miscellaneous Orbiter Commands (Part 1)

Miscellaneous Orbiter Commands (Part 2)		
Command	Arguments	Purpose
-csv_file_join	fname n fname-label- pair-list	Joins csv file fname based on a list of fname-label pairs of length n .
-csv_file_concatenate	fname-out n f1 f2 ... fn	Join the n csv files f1, f2, ..., fn. The output is written to fname-out.
-csv_file_concatenate_from_mask	f N fname- mask fname- out	Join csv files. The input files arise from fname-mask with all values $i = f, f + 1, \dots, f + N - 1$. The output is written to fname-out.
-csv_file_extract_column_to_txt	fname col- label	
-csv_file_filter	fname col- label value	Extracts those rows whose entry in the given column matches the given value.
-csv_file_latex	header fname	Produces a latex table from the given csv-file. The binary variable header determines whether a header is produced.
-draw_matrix	descr	Produce a bitmap graphic, see Table 17.4.
-destring	file-in column file-out	Unstring a text vector by removing quotes. The text is from the given file in the given column and will be written to the given output file.
-reformat	fname-in fname-out nb-cols	Reformat the data from fname-in in nb-cols columns. Write the output to fname-out.
-split_by_values	fname-in	Split the file fname-in by values. For each value, a separate file is written.
-change_values	fname-in fname-out map-in map- out	Change the values of the file in fname-in according to a mapping. The output is written to fname-out. The mapping is defined by two corresponding sequences. The input values are in map-in, the corresponding output values are in map-out.
-store_as_csv_file	fname m n L	Stores the data in L to a csv file. The data is an $m \times n$ matrix in row-major ordering.
-mv	fname-in fname-out	Rename file fname-in to fname-out.
-cp	fname-in fname-out	Copy file fname-in to fname-out.
-system	command	Execute the given system command.
-loop	variable val_from val_to val_step	Create a loop with the given loop variable, whose values range from loop_from to val_to minus one in steps of val_step.
-loop_over	variable domain	Create a loop with the given loop variable, whose values range over the given domain.

Table 21.2: Miscellaneous Orbiter Commands (Part 2)

Miscellaneous Orbiter Commands (Part 3)		
Command	Arguments	Purpose
-plot_function	function-name	
-draw_projective_curve	options	
-tree_draw	options	Draw a tree from a description. For options, see Table 21.4.
-extract_from_file	fname-in key fname-out	Extract lines from a text file, starting with the given key and continuing until the next empty row.
-extract_from_file_with_tail	fname-in key tail fname-out	Extract lines from a text file, starting with the given key and continuing until the next empty row.
-serialize_file_names	fname output-mask	
-save_4_bit_data_file	fname vector-data	
-gnuplot	fname title label_x label_y	Produce an diagram of data series using gnuplot [34].
-compare_columns	fname col1 col2	
-gcd_worksheet	nb_problems N key	
-draw_layered_graph	options	Draw layered graph with options.
-draw_factor_poset	options	Draw factor poset with options.
-read_xml	fname fname_out	Produces cross references for classes from a Doxygen output file.
-read_column_and_tally	fname col-header	
-intersect_with	matrix data	Counts the number of times an element of data appears in any given row of the given matrix.
-make_set_of_sets	fname-in col-label list	
-copy_and_edit	fname-in fname-out-mask params search-and-replace	
-join_columns	fname-in fname-out col1 col2	
-decomposition_matrix	fname-in po fst iso n1 n2 n3	
-stats	fname-base	
-intersect_set_of_sets	fname col	
-eliminate_duplicate_columns	fname	Eliminates duplicate columns in the given file and writes a file with one column per each type.
-density_of_ones_in_bitvector_file	fname	Computes the density of ones in the given bitvector file.

Table 21.3: Miscellaneous Orbiter Commands (Part 3)

Options for Drawing Trees		
Command	Arguments	Purpose
-file	fname	File in .tree format
-restrict	color	
-select_path	path	
-draw_options	options	Select the draw options object for drawing options.

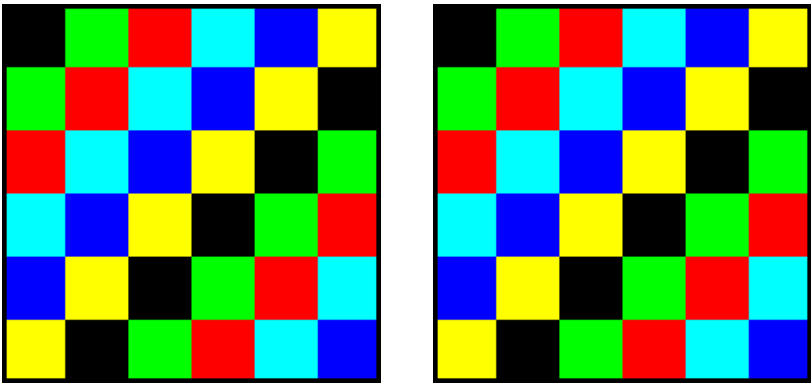
Table 21.4: Options for Drawing Trees

```
5,5,1,3,2,6,4
END
```

and next the file that is created by selecting rows and columns 0, 2, 4:

```
Row,"C0","C2","C4"
0,"1","2","4"
1,"2","4","1"
2,"4","1","2"
END
```

A graphical output is shown below:



21.2 Limitations

Several limitations exist in Orbiter. Here is a list:

1. Field elements are encoded as `int`. This limits the size of fields that can be handled to 2^{8s-1} where $s = \text{sizeof}(\text{int})$.
2. The ranks of elements in the permutation domain are encoded as `long int`. This limits the size of permutation domains that can be handled. The degree of a permutation group must be less than 2^{8s-1} where $s = \text{sizeof}(\text{long int})$.
3. The finite field class builds tables for the addition and multiplication of field elements. This restricts the size of the fields that can be created.
4. The projective geometry class tries to build a bitmatrix for the adjacency matrix if the number of lines is less than `MAX_NUMBER_OF_LINES_FOR_INCIDENCE_MATRIX` which is defined in `src/lib/foundations/geometry/projective_space.cpp`. If the number of lines is too big, the table is not created. In this case, the projective geometry class may behave slower.
5. The projective geometry class tries to build a table for the lines if the number of points is less than `MAX_NUMBER_OF_POINTS_FOR_POINT_TABLE` and the number of lines is less than `MAX_NUMBER_OF_LINES_FOR_LINE_TABLE`, both of which are defined in `src/lib/foundations/geometry/projective_space.cpp`. If the number of points is too big, the table is not created. In this case, the projective geometry class may behave slow.
6. The projective geometry class tries to build a table for the lines through any two points if the number of points is less than `MAX_NB_POINTS_FOR_LINE_THROUGH_TWO_POINTS_TABLE` which is defined in `src/lib/foundations/geometry/projective_space.cpp`. If the number of points is too big, the table is not created. In this case, the projective geometry class may behave slow.
7. The projective geometry class tries to build a table for the intersection points of pairs of lines if the number of points is less than `MAX_NB_POINTS_FOR_LINE_INTERSECTION_TABLE` which is defined in `src/lib/foundations/geometry/projective_space.cpp`. If the number of points or lines is too big, the table is not created. In this case, the projective geometry class may behave slow.
8. For Windows users: Cygwin by default uses 32 bit integers for both `int` and `long int`. Using Cygwin 64 to compile Orbiter recommended.
9. A limited list of primitive polynomials are hard-coded in Orbiter. For large fields, the user must provide their own primitive polynomial. The polynomials encoded in orbiter are not guaranteed to be compatible with the subfield relationship.

The command

Code Snippet 912

```

1  F_15bit:
2  ▷ $(ORBITER) -v 10 \
3  ▷ ▷ -define F -finite_field -q 32749 -without_tables -end \
4  ▷ ▷ -define v -vector -field F -allow_negatives \
5  ▷ ▷ ▷ -dense "-1,-1" -end \
6  ▷ ▷ -with F -do -finite_field_activity -product_of v -end
7  ▷
8
```

creates the field $\mathbb{F}_{32749}$. Tables are suppressed for speed. Note that

$$2^{15} - 19 = 32749$$

is prime (see <https://primes.utm.edu/lists/2small/0bit.html>). The command creates two instances of -1 in the field and multiplies them.

The command

Code Snippet 913

```
1  F_31bit:
2  ▷ $(ORBITER) -v 10 \
3  ▷ ▷ -define F -finite_field -q 2147483647 -without_tables -end \
4  ▷ ▷ -define v -vector -field F -allow_negatives \
5  ▷ ▷ ▷ -dense "-1,-1" -end \
6  ▷ ▷ -with F -do -finite_field_activity -product_of v -end
7
```

creates the field $\mathbb{F}_{2147483647}$. Tables are suppressed for speed. Note that

$$2^{31} - 1 = 2147483647 = p$$

is prime (see <https://primes.utm.edu/lists/2small/0bit.html>). The command creates two instances of -1 in the field $\mathbb{Z}_p$ and multiplies them. The fact that this is done correctly is remarkable, because the intermediate value exceeds the size of 32 bit signed integers. Recall that -1 is stored as $p - 1$, so

$$-1 \equiv 2147483646.$$

But

$$-1 \cdot -1 \equiv 2147483646 \cdot 2147483646 = 4611686009837453316,$$

which exceeds the range of 32 bit signed integers. The reason why this command works is that the multiplication of integers is performed using long integer arithmetic. Long integer arithmetic treats integers as text strings, and hence is not bound by the size of a machine word. Unfortunately, long integer arithmetic is considerably slower than arithmetic in machine words. So, there is a performance drawback with large finite fields in Orbiter.

The command

Code Snippet 914

```
1  F_32bit:
2  ▷ $(ORBITER) -v 10 \
3  ▷ ▷ -define F -finite_field -q 4294967291 -without_tables -end \
4
```

creates the field $\mathbb{F}_{4294967291}$. Note that

$$2^{32} - 5 = 4294967291$$

is prime (see <https://primes.utm.edu/lists/2small/0bit.html>). The command will likely not run, and terminate with an error message. The reason is hardware dependent. The command tests the size of a machine word. Most compilers map the C++ data type *signed int* to 32 bit machine words, so this command will fail in such an environment. This is true even with 64 bit hardware. There may be some compiler options to change this.

Machines for which the previous command works correctly, the following command should also be fine:

Code Snippet 915

```

1      F_63bit:
2      ▷ $(ORBITER) -v 10 \
3      ▷ ▷ -define F -finite_field -q 9223372036854775783 -without_tables -end \
4      ▷ ▷ -define v -vector -field F -allow_negatives \
5      ▷ ▷ ▷ -dense "-1,-1" -end \
6      ▷ ▷ -with F -do -finite_field_activity -product_of v -end
7

```

Note that

$$2^{63} - 25 = 9223372036854775783$$

is prime (see <https://primes.utm.edu/lists/2small/0bit.html>). However, the next command will almost surely fail:

Code Snippet 916

```

1      F_64bit:
2      ▷ $(ORBITER) -v 10 \
3      ▷ ▷ -define F -finite_field -q 18446744073709551557 -without_tables -end \
4      ▷ ▷ -define v -vector -field F -allow_negatives \
5      ▷ ▷ ▷ -dense "-1,-1" -end \
6      ▷ ▷ -with F -do -finite_field_activity -product_of v -end
7

```

This is because we are exceeding the range that can be stored in a 64 bit unsigned integer machine word. Note that

$$2^{64} - 59 = 18446744073709551557$$

is prime (see <https://primes.utm.edu/lists/2small/0bit.html>).

Chapter 22

Orbiter on Windows

22.1 Using Windows Subsystem Linux

The following quote from <https://docs.microsoft.com/en-us/windows/wsl/> summarizes the function of the Windows Subsystem for Linux:

Windows Subsystem for Linux (WSL) lets developers run a GNU/Linux environment – including most command-line tools, utilities, and applications – directly on Windows, unmodified, without the overhead of a traditional virtual machine or dual-boot setup. You can:

1. Choose your favorite GNU/Linux distributions from the Microsoft Store.
2. Run common command-line tools such as grep, sed, awk, or other ELF-64 binaries.
3. Run Bash shell scripts and GNU/Linux command-line applications including:
4. Tools: vim, emacs, tmux
5. Languages: NodeJS, Javascript, Python, Ruby, C/C++, C# & F#, Rust, Go, etc.
6. Services: SSHD, MySQL, Apache, lighttpd, MongoDB, PostgreSQL.
7. Install additional software using your own GNU/Linux distribution package manager.
8. Invoke Windows applications using a Unix-like command-line shell.
9. Invoke GNU/Linux applications on Windows.

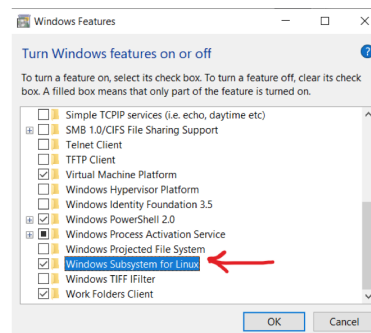
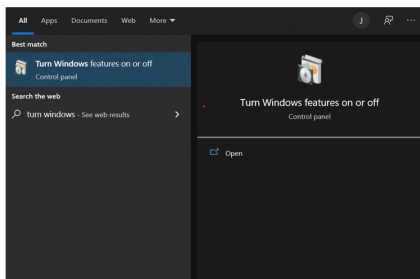
The following set of slides will illustrate the installation of Orbiter under WSL.

Resources

- Many of the steps will be taken from the following sources:
 - https://okunhardt.github.io/documents/Installing_WSL.pdf
 - <https://docs.microsoft.com/en-us/windows/wsl/basic-commands>
- Consult the two links for further help and suggestions.

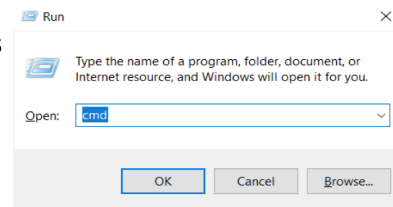
Installing WSL

- Search “Turn Windows features on or off” in the Windows search bar
- Search for “Windows Subsystem for Linux”, the box must be checked
- Restart the computer



Update

- The Windows Subsystem for Linux kernel does not automatically update due to system settings
- Updates must be done manually
- To update, first you need to command prompt as admin
 - Press Windows + R to open the “Run” box
 - Type “cmd” into the box
 - Press Ctrl + Shift + Enter
 - When the window prompt opens, click “Yes”
 - Command prompt will now open as admin
 - In command prompt
 - Type `wsl --update`
 - Type `wsl --shutdown`



```
Administrator: C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.19044.1415]
(c) Microsoft Corporation. All rights reserved.

C:\WINDOWS\system32>wsl --update
Checking for updates...
Downloading updates...
Installing updates...
This change will take effect on the next full restart of WSL. To force a restart, please run 'wsl --shutdown'.
Kernel version: 5.10.60.1
```

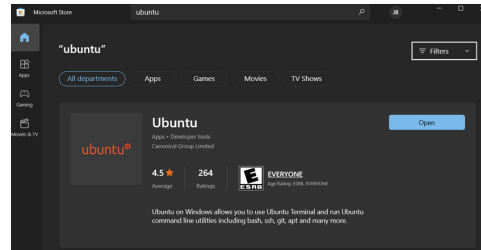
WSL1, WSL2

- When using WSL, you can adjust the configurations according to the Linux distribution that you are using
- To run Ubuntu distribution, we need the WSL1 configuration
- To check the status, in the command prompt enter
 - `wsl --status`
- To change WSL configuration type
 - `wsl --set-default-version 1`
 - `wsl --shutdown`

```
C:\Users\Joel\CPP_Workspace>wsl --status
Default Distribution: Ubuntu
Default Version: 1
```

Ubuntu - installation

- Generally, the Ubuntu distribution is installed by default when WSL is installed
 - `wsl --status`
 - Displays the default distribution
- If you find that Ubuntu was not installed, you can find it in the Microsoft store
- Launch Ubuntu after installation



Ubuntu - launching

- After launching Ubuntu, allow the installation to be initiated
- If you receive an error, this could be a result of the configuration
 - Set configuration to WSL1
 - `wsl --set-default-version 1`
 - Make sure to terminate Ubuntu and reboot
 - `wsl --terminate Ubuntu`
 - Start Ubuntu again
- Once Ubuntu starts correctly
 - Create Username & Password to complete installation
 - Note: the password will not appear when you type it

```
joebarr@LAPTOP-KQU42VS8: ~  
Installing, this may take a few minutes...  
Please create a default UNIX user account. The username does not need to match your Windows username.  
For more information visit: https://aka.ms/wslusers  
Enter new UNIX username: joebarr  
New password:
```


Ubuntu - update

- Ubuntu does not update automatically, to update run the command
 - `sudo apt update && sudo apt upgrade`
- You will be prompted to enter your password
- When update are ready to be installed the message will appear
 - Do you want to continue? [Y/n]
 - Y + enter

```

joebarr@LAPTOP-KQU42VS8: ~
Welcome to Ubuntu 20.04.3 LTS (GNU/Linux 4.4.0-19041-Microsoft x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Tue Jan 11 22:39:50 MST 2022

System load:          0.52
Usage of /home:       unknown
Memory usage:        53%
Swap usage:          2%
Processes:            7
Users logged in:      0
IPv4 address for wifi0: 192.168.7.65
IPv6 address for wifi0: fd45:1ae2:adef:1:6cc6:baae:e12c:92d7
IPv6 address for wifi0: fd45:1ae2:adef:1:54fa:5f35:b84:2f87
IPv6 address for wifi0: fd45:1ae2:adef:1:700e:4c46:55cb:ef48

updates can be applied immediately.

This message is shown once a day. To disable it please create the
/home/joebarr/.hushlogin file.
joebarr@LAPTOP-KQU42VS8: ~$ sudo apt update && sudo apt upgrade

```

Ubuntu – g++ and make

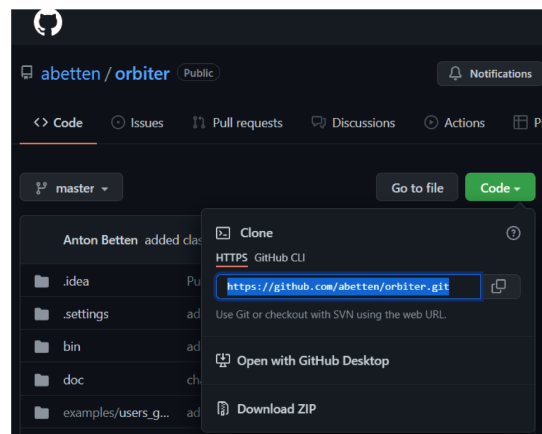
- At this point, you have successfully installed and setup WSL, and now you can use the terminal as you would on Ubuntu
- Terminate and reboot Ubuntu
- Run the command in Ubuntu
 - `sudo apt install g++`
 - You can now compile C++ in WSL
- Run the command in Ubuntu
 - `sudo apt install make`
 - You can now use `makefiles` in WSL

Orbiter - installation

- The easiest way to run make is through the command prompt, not Ubuntu
- To run WSL commands in command prompt, use either
 - `wsl <command>`
 - `wsl.exe <command>`
- Open command prompt
- Change directory to Users\username
 - `cd C:\Users\“your username”`

Orbiter - installation

- In web, go to <https://github.com/abetten/orbiter>
- Click on the green icon “Code” that opens a drop-down menu
- You want to copy HTTPS URL



Orbiter - installation

- In command prompt, once you are in C:\Users\Joel type the command
 - wsl.exe git clone <https://github.com/abetten/orbiter.git>
 - Hit enter
- Now, orbiter will begin the cloning process

```
C:\Users\Joel>wsl.exe git clone https://github.com/abetten/orbiter.git
Cloning into 'orbiter'...
remote: Enumerating objects: 87873, done.
remote: Counting objects: 100% (7029/7029), done.
remote: Compressing objects: 100% (5007/5007), done.
remote: Total 87873 (delta 5245), reused 3408 (delta 1713), pack-reused 80844
Receiving objects: 100% (87873/87873), 1.55 GiB | 18.87 MiB/s, done.
Resolving deltas: 100% (47649/47649), done.
Updating files: 100% (1154/1154), done.
```

Orbiter - compile

- After cloning orbiter, run the command
 - `dir`
- You will find a new directory created called “orbiter”
- Change directory to “orbiter”
 - `cd orbiter`

```
C:\Users\Joel>dir
Volume in drive C is Windows
Volume Serial Number is 6C2E-AA7C

Directory of C:\Users\Joel

01/11/2022 12:00 AM <DIR> .
01/11/2022 12:00 AM <DIR> ..
12/23/2020 02:51 PM <DIR> 3D Objects
12/23/2020 02:51 PM <DIR> Contacts
12/23/2020 02:51 PM <DIR> Desktop
01/09/2022 06:11 PM <DIR> Documents
12/23/2020 02:51 PM <DIR> Downloads
02/09/2018 03:27 PM <DIR> Dropbox
12/23/2020 02:51 PM <DIR> Favorites
07/10/2020 10:27 AM <DIR> HP
08/09/2019 07:25 AM <DIR> Intel
12/23/2020 02:51 PM <DIR> Links
12/23/2020 02:51 PM <DIR> Music
08/04/2021 09:20 AM <DIR> OneDrive
01/11/2022 12:02 AM <DIR> orbiter
01/09/2022 09:19 PM <DIR> Pictures
05/27/2018 03:38 PM <DIR> Roaming
12/23/2020 02:51 PM <DIR> Saved Games
12/23/2020 02:51 PM <DIR> Searches
01/09/2022 04:36 PM <DIR> source
12/23/2020 02:51 PM <DIR> Videos
0 File(s) 0 bytes
21 Dir(s) 70,471,704,576 bytes free
```

Orbiter - compile

- Now that you are in C:\Users\“your username”\orbiter, run the command
 - wsl.exe make
- The orbiter library will now be compiled, give it some time

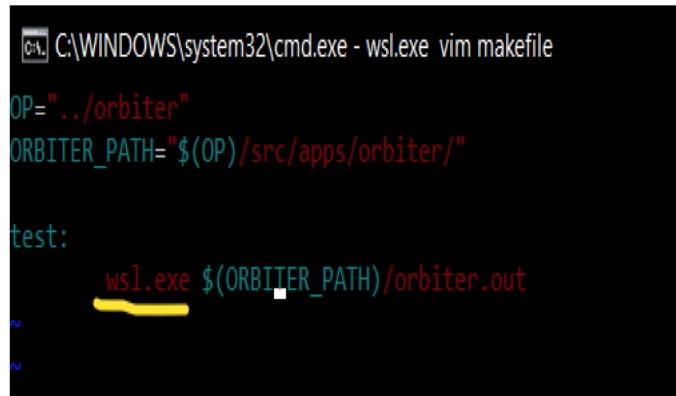
```
C:\Users\Joel\orbiter>wsl.exe make ←
cd src; make all
make[1]: Entering directory '/mnt/c/Users/Joel/orbiter/src'
cd lib; make all
make[2]: Entering directory '/mnt/c/Users/Joel/orbiter/src/lib'
cd foundations; make
```

Makefile

- Now that orbiter has been successfully compiled, in the directory C:\Users\“your username”\orbiter
 - Change directory to C:\Users\“your username” and create a new directory
 - Ex: `mkdir CPP_Workspace`
- Change directory into `CPP_Workspace`
 - `cd CPP_Workspace`
- In C:\Users\“your username”\“new directory”, run the command
 - `wsl.exe vim makefile`
- Vim (an IDE) will create the file “`makefile`”
- For Vim commands, go to <https://vim.rtorr.com/>
- Remember: all Ubuntu commands must begin with either
 - `wsl` or `wsl.exe`

Makefile

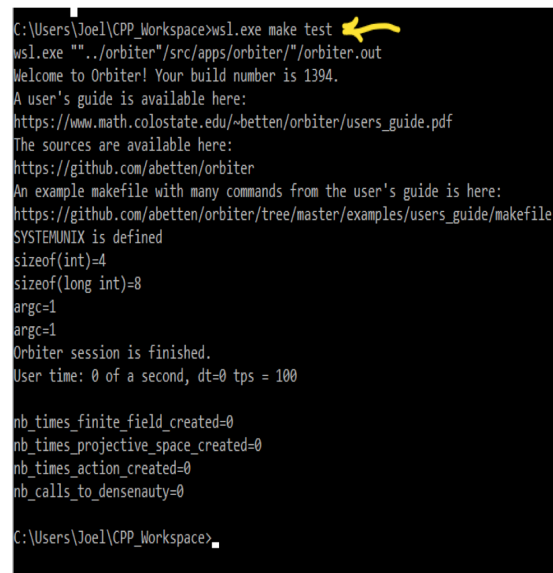
- To edit file in vim, click “i”
- You will see --insert-- in the lower left-hand corner
- The example to the right demonstrates a simple test to assure that orbiter is running correctly
- Assuming that orbiter directory is located in C:\Users\“your username” then the variable OP and ORBITER_PATH should work just fine
- Note where wsl.exe is inserted
- Makefile contains Ubuntu commands not windows commands



```
C:\WINDOWS\system32\cmd.exe - wsl.exe vim makefile
OP=" ../orbiter"
ORBITER_PATH="$(OP)/src/apps/orbiter/"
test:
    wsl.exe $(ORBITER_PATH)/orbiter.out
```

Running makefile

- Now that you have created the makefile,
 - Click “esc” to finish editing in vim
 - Run the command
 - :wq + enter
 - This saves & closes the makefile in vim
- You will be returned to
 - C:\Users\“your username”\“new directory”
- In the directory run,
 - wsl.exe make test
 - Hit “enter”
- If everything runs correctly, you will see



```
C:\Users\Joel\CPP_Workspace>wsl.exe make test
wsl.exe " ../orbiter/src/apps/orbiter/"orbiter.out
Welcome to Orbiter! Your build number is 1394.
A user's guide is available here:
https://www.math.colostate.edu/~betten/orbiter/users_guide.pdf
The sources are available here:
https://github.com/abetten/orbiter
An example makefile with many commands from the user's guide is here:
https://github.com/abetten/orbiter/tree/master/examples/users_guide/makefile
SYSTEMUNIX is defined
sizeof(int)=4
sizeof(long int)=8
argc=1
argc=1
Orbiter session is finished.
User time: 0 of a second, dt=0 tps = 100

nb_times_finite_field_created=0
nb_times_projective_space_created=0
nb_times_action_created=0
nb_calls_to_densnauty=0
C:\Users\Joel\CPP_Workspace>
```

Orbiter - notes

- Now that everything runs correctly, visit https://www.math.colostate.edu/~betten/orbiter/users_guide.pdf
- This is the Orbiter User's guide
- Remember that you must use "wsl.exe make <target>" or "wsl make <target>" to run make correctly on linux distribution
- Also, note how "wsl.exe" is used inside of the makefile
- Ubuntu commands are used in makefile

Orbiter - update

- To update orbiter, change directories to
 - C:\Users\"your username"\orbiter
- Run the commands
 - wsl.exe make clean ; wsl.exe make
- Good luck!

Chapter 23

Templates

23.1 Templates

h	mon	vector					
0	X_0^2	$(2, 0, 0, 0)$					
1	X_1^2	$(0, 2, 0, 0)$					
2	X_2^2	$(0, 0, 2, 0)$					
3	X_3^2	$(0, 0, 0, 2)$					
4	X_0X_1	$(1, 1, 0, 0)$					
5	X_0X_2	$(1, 0, 1, 0)$					
6	X_0X_3	$(1, 0, 0, 1)$					
7	X_1X_2	$(0, 1, 1, 0)$					
8	X_1X_3	$(0, 1, 0, 1)$					
9	X_2X_3	$(0, 0, 1, 1)$					

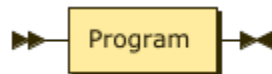
h	mon	vector					
0	X_0^3	$(3, 0, 0, 0)$					
1	X_1^3	$(0, 3, 0, 0)$					
2	X_2^3	$(0, 0, 3, 0)$					
3	X_3^3	$(0, 0, 0, 3)$					
4	$X_0^2 X_1$	$(2, 1, 0, 0)$					
5	$X_0^2 X_2$	$(2, 0, 1, 0)$					
6	$X_0^2 X_3$	$(2, 0, 0, 1)$					
7	$X_0 X_1^2$	$(1, 2, 0, 0)$					
8	$X_1^2 X_2$	$(0, 2, 1, 0)$					
9	$X_1^2 X_3$	$(0, 2, 0, 1)$					
10	$X_0 X_2^2$	$(1, 0, 2, 0)$					
11	$X_1 X_2^2$	$(0, 1, 2, 0)$					
12	$X_2^2 X_3$	$(0, 0, 2, 1)$					
13	$X_0 X_3^2$	$(1, 0, 0, 2)$					
14	$X_1 X_3^2$	$(0, 1, 0, 2)$					
15	$X_2 X_3^2$	$(0, 0, 1, 2)$					
16	$X_0 X_1 X_2$	$(1, 1, 1, 0)$					
17	$X_0 X_1 X_3$	$(1, 1, 0, 1)$					
18	$X_0 X_2 X_3$	$(1, 0, 1, 1)$					
19	$X_1 X_2 X_3$	$(0, 1, 1, 1)$					

Appendix A

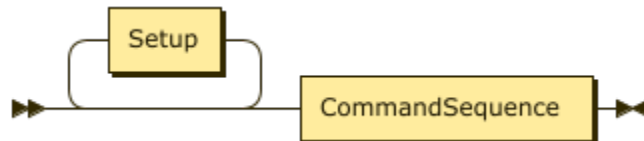
Orbiter Syntax Railroad Diagrams

A.1 Syntax Diagrams

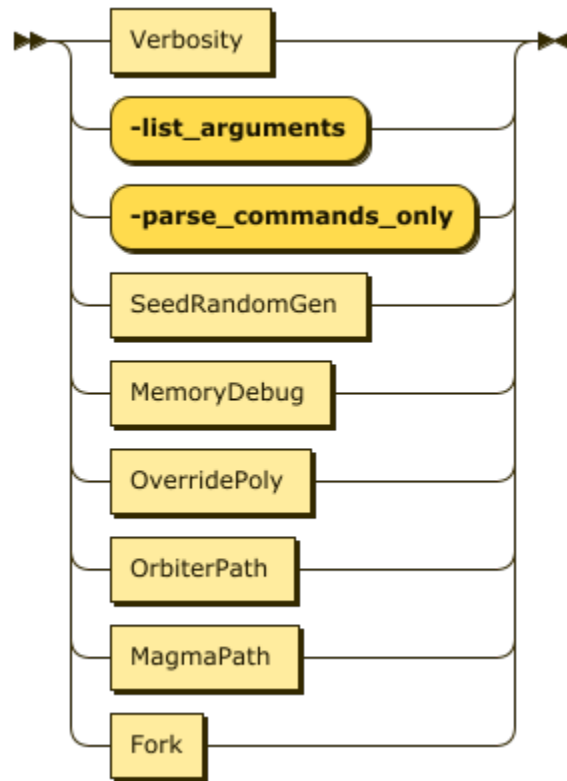
Grammar:



Program:



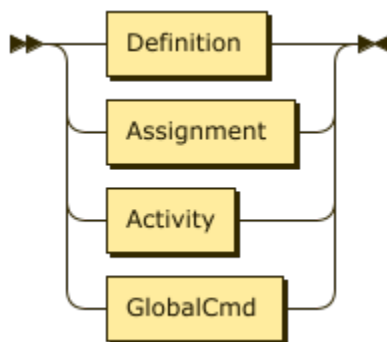
Setup:



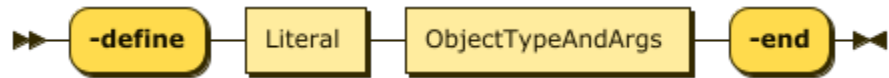
CommandSequence:



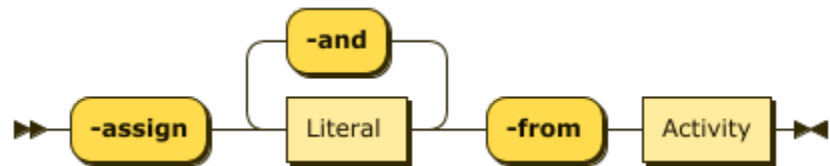
Command:



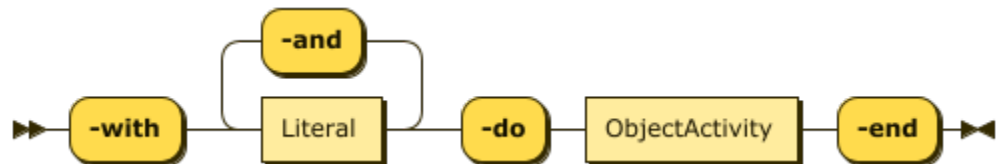
Definition:



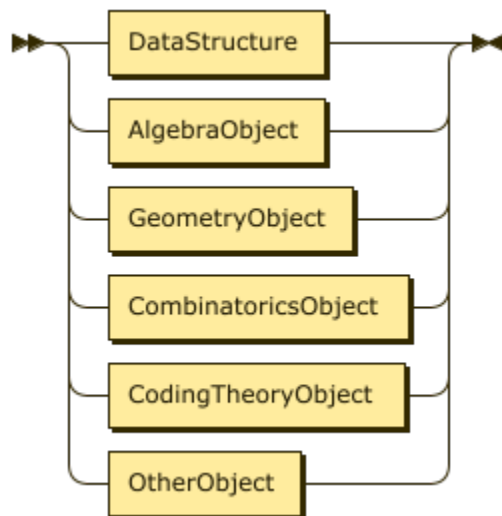
Assignment:



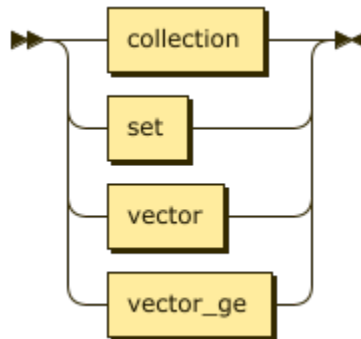
Activity:



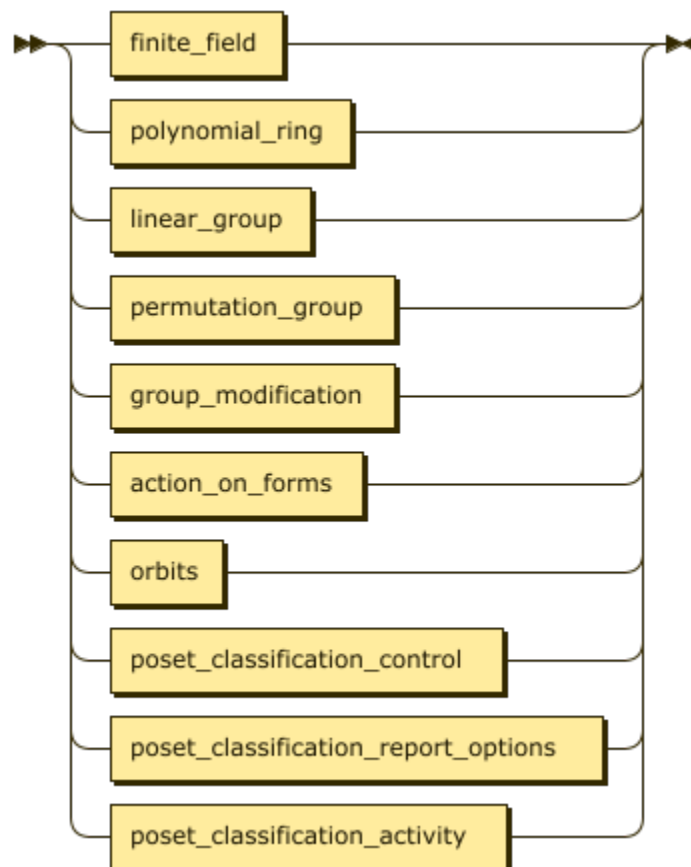
ObjectTypeAndArgs:



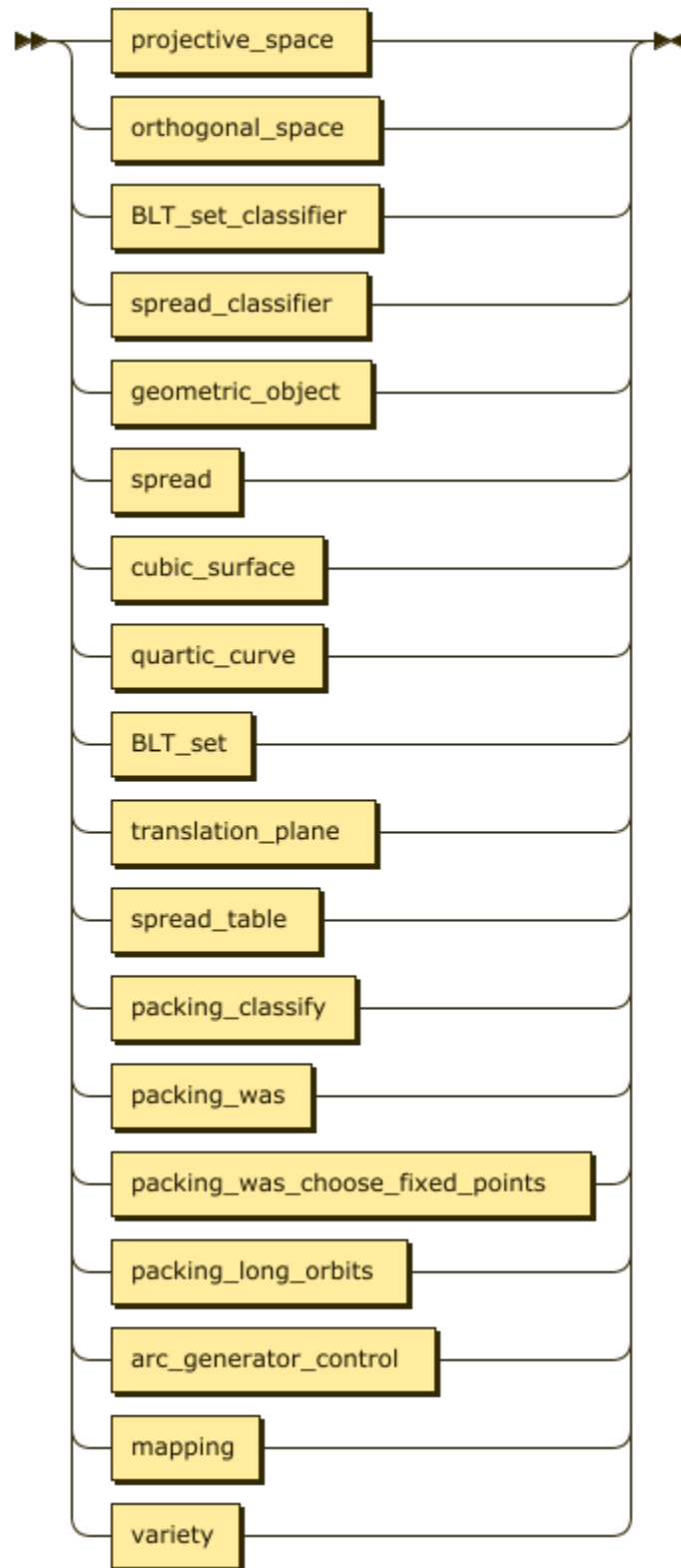
DataSetructure:



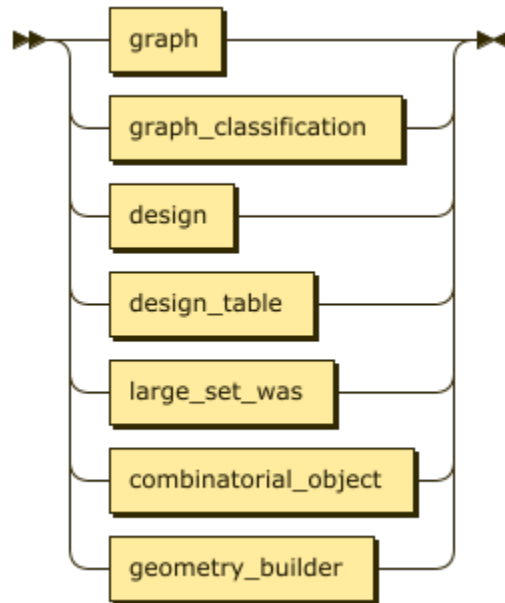
AlgebraObject:



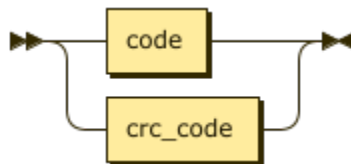
GeometryObject:



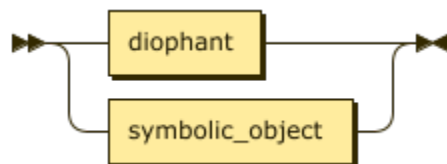
CombinatoricsObject:



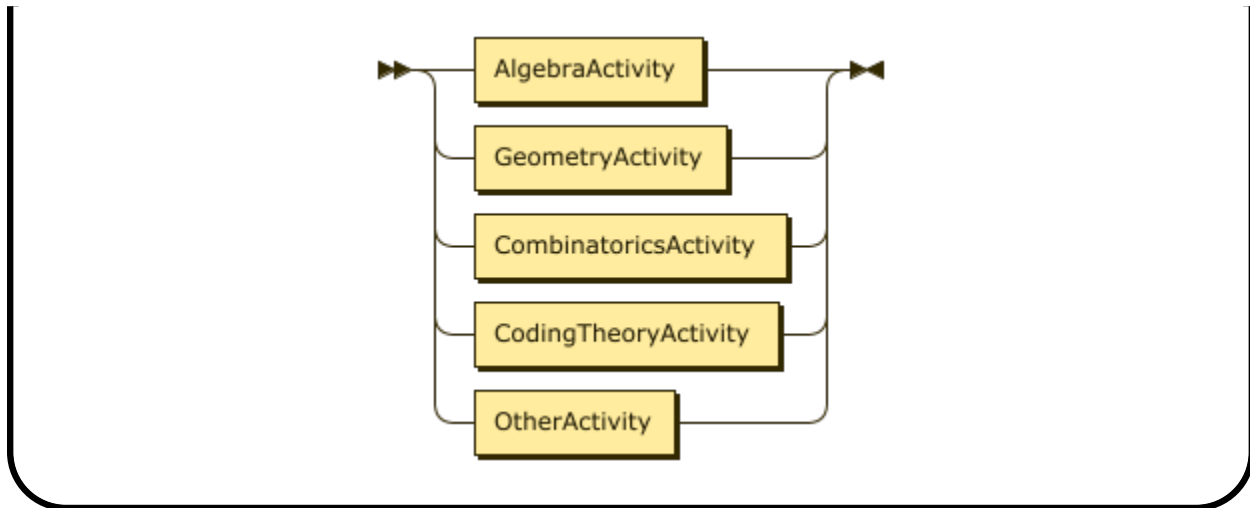
CodingTheoryObject:



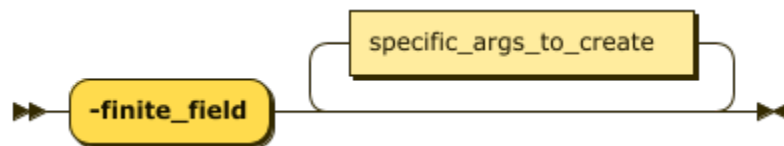
OtherObject:



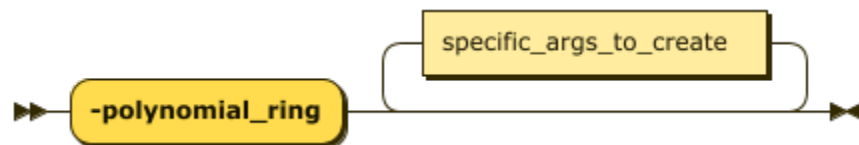
ObjectActivity:



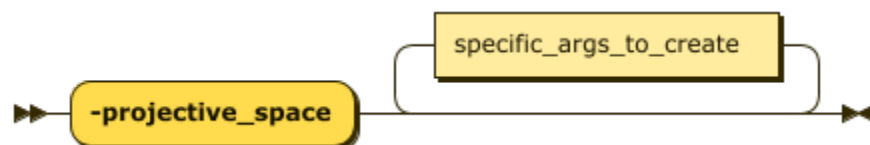
finite_field:



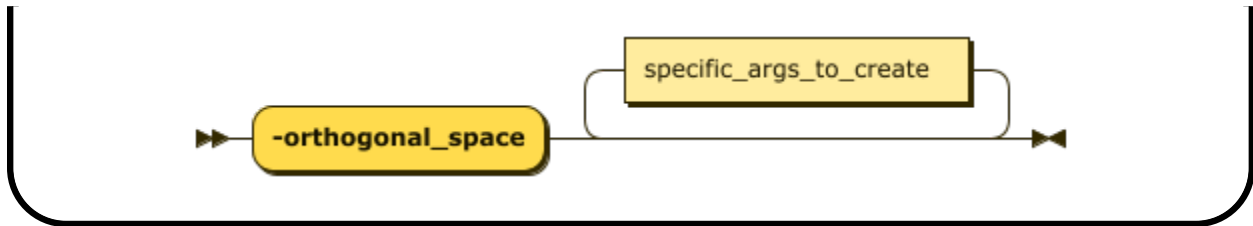
polynomial_ring:



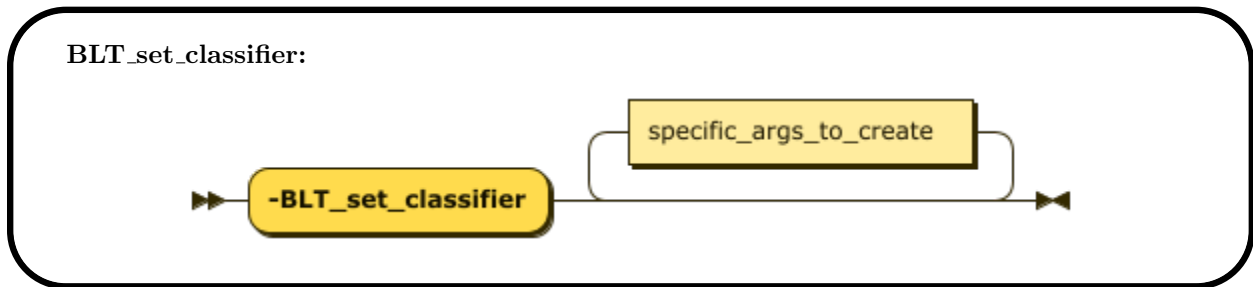
projective_space:



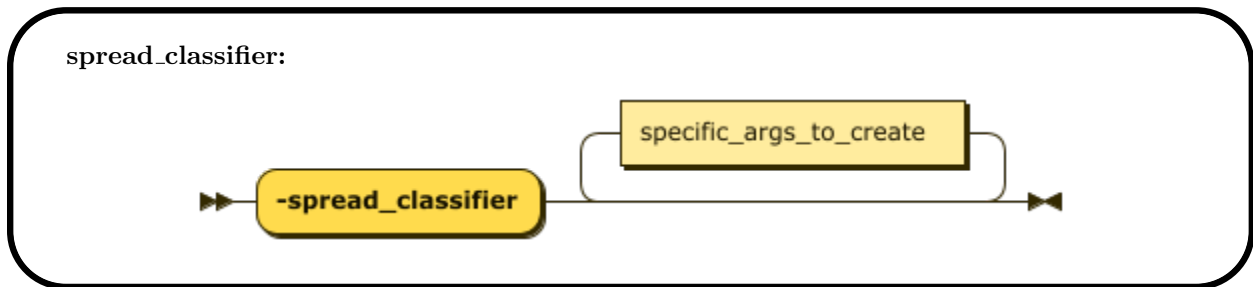
orthogonal_space:



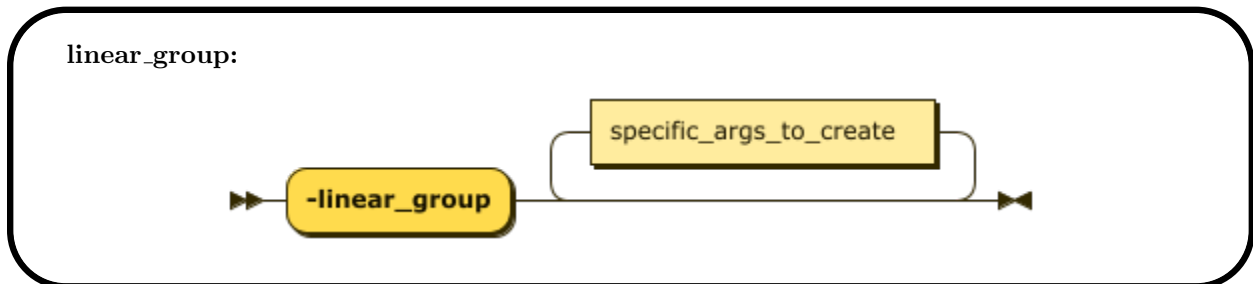
`BLT_set_classifier:`



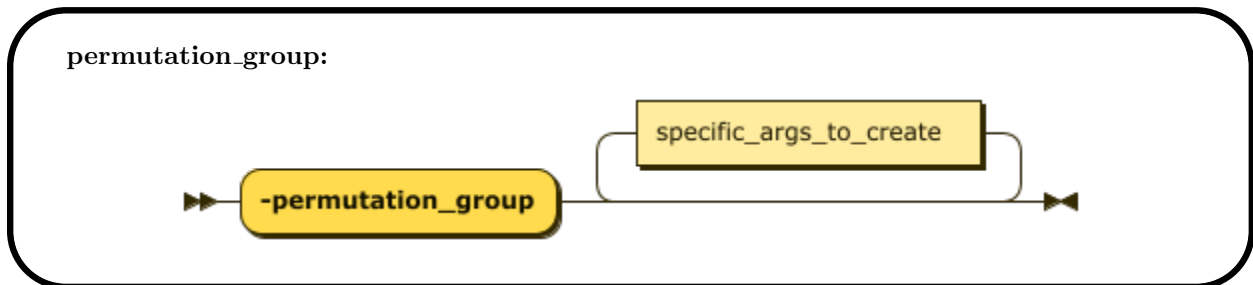
`spread_classifier:`



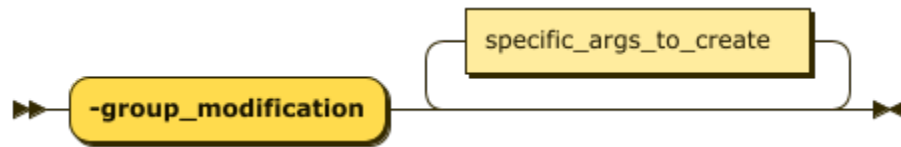
`linear_group:`



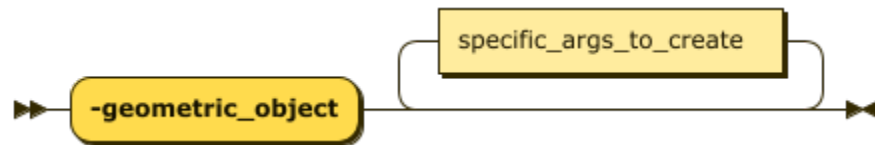
`permutation_group:`



`group_modification:`



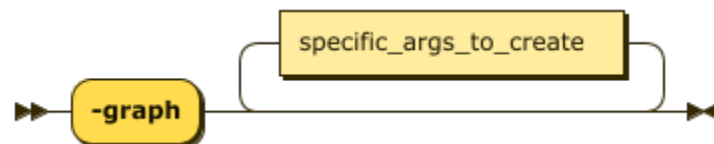
`geometric_object:`



`collection:`



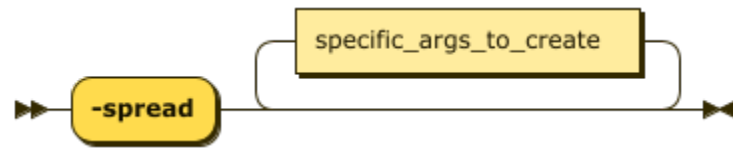
`graph:`



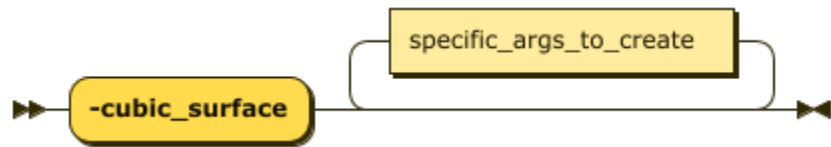
`code:`



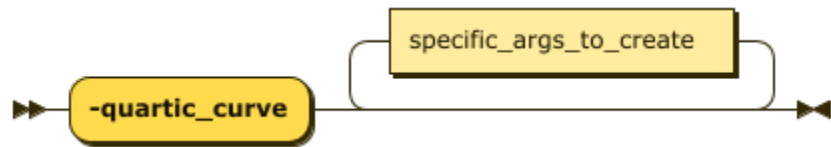
`spread:`



`cubic_surface:`



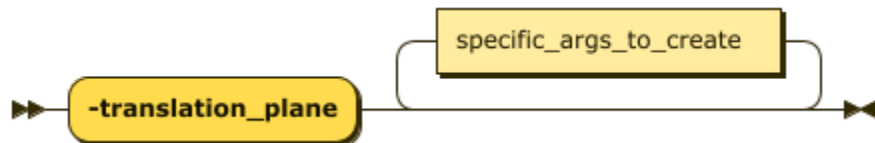
`quartic_curve:`



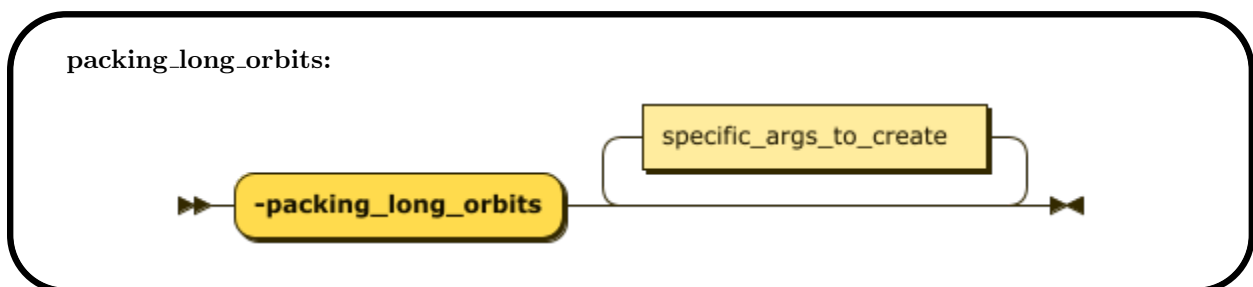
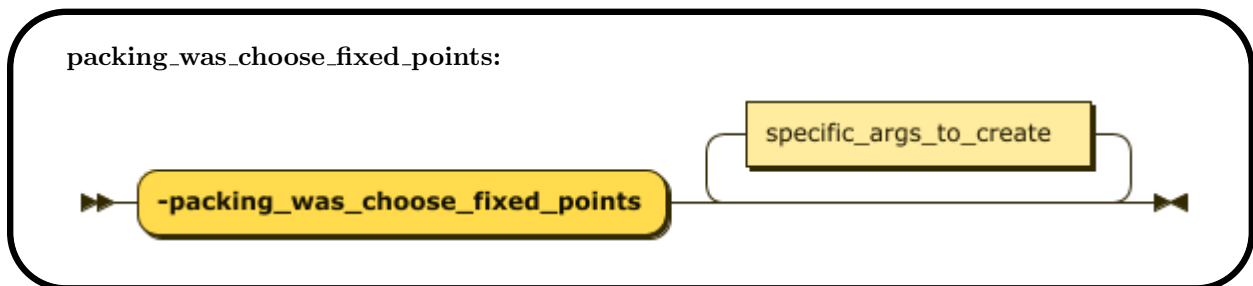
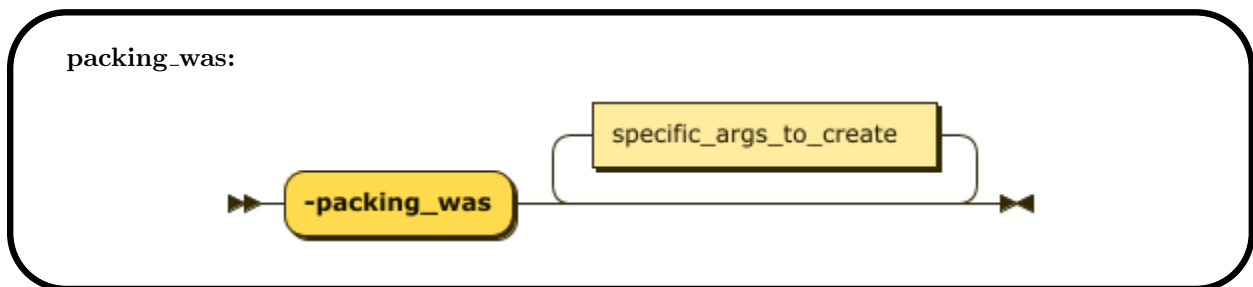
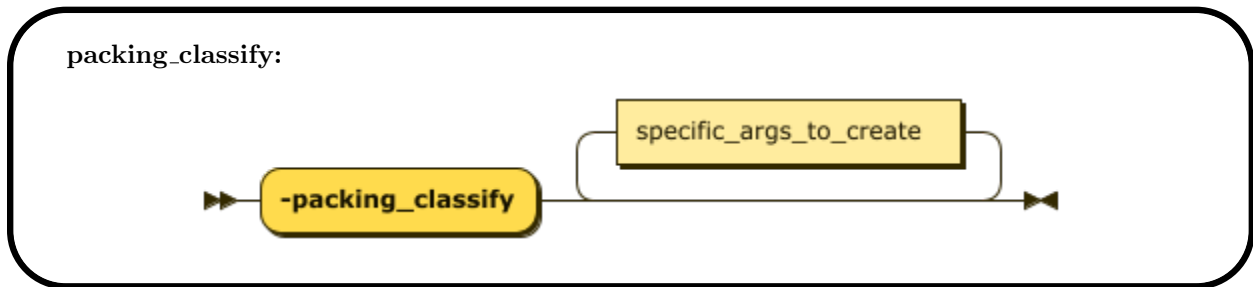
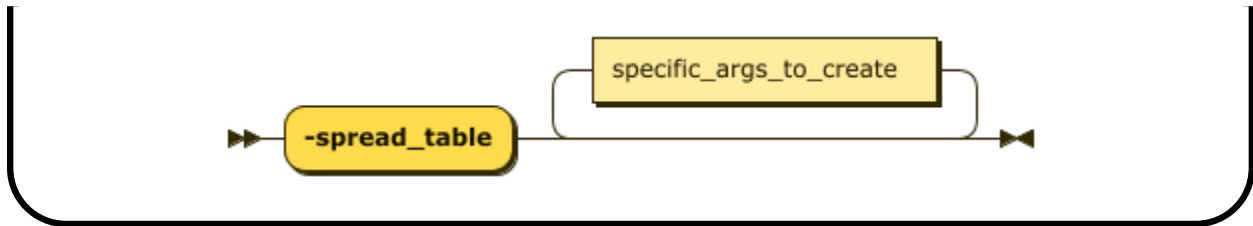
`BLT_set:`

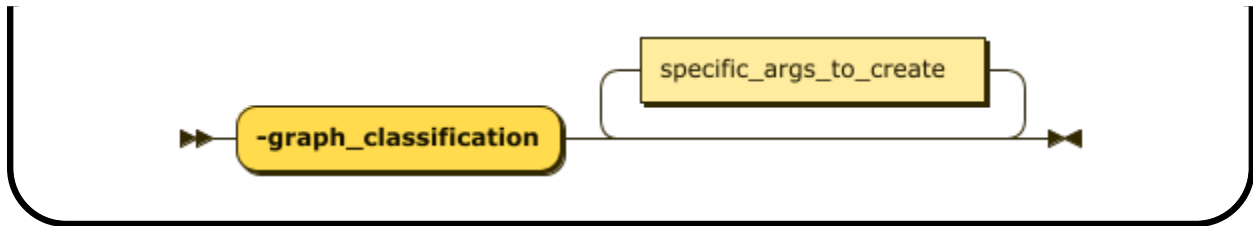


`translation_plane:`

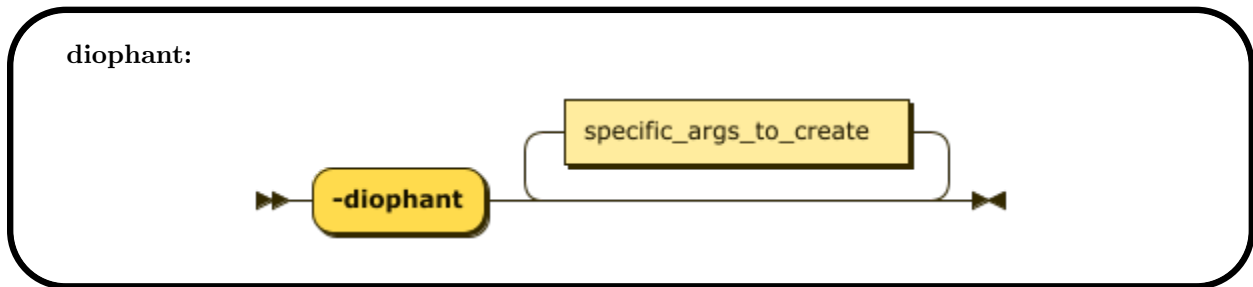


`spread_table:`

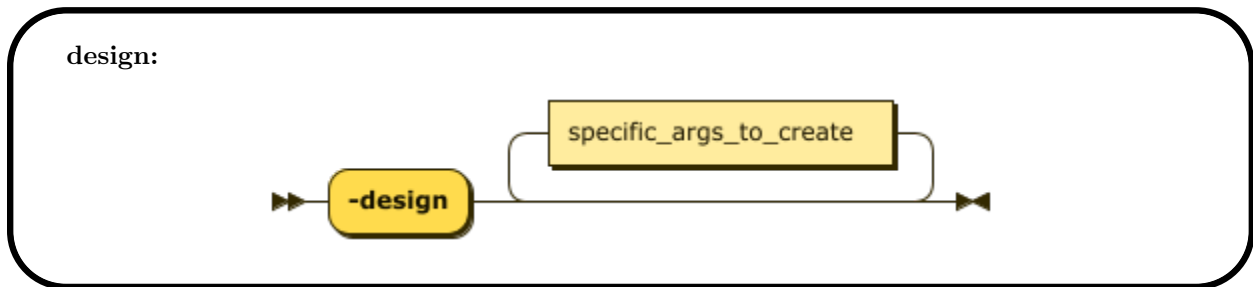




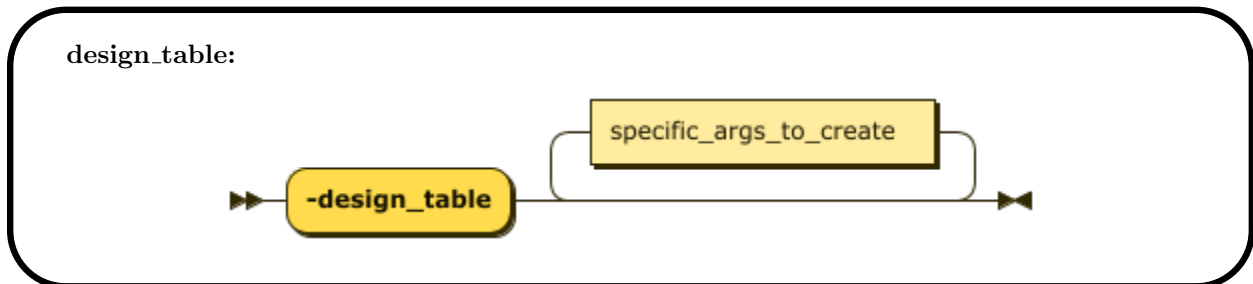
diophant:



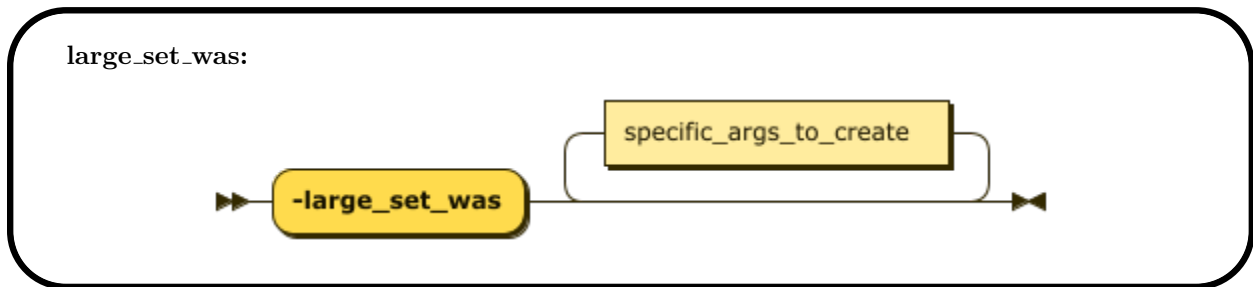
design:



design_table:

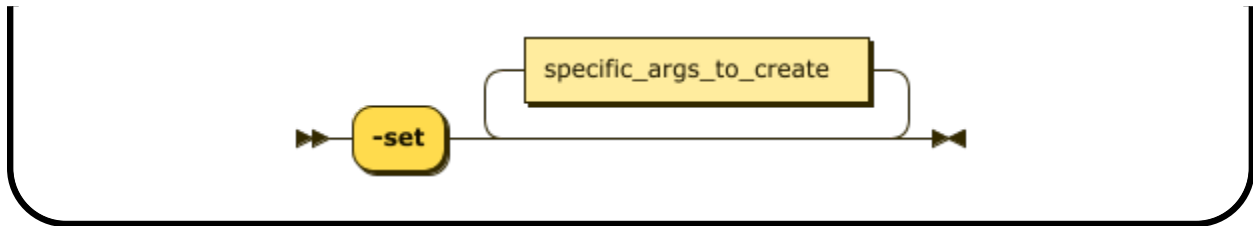


large_set_was:

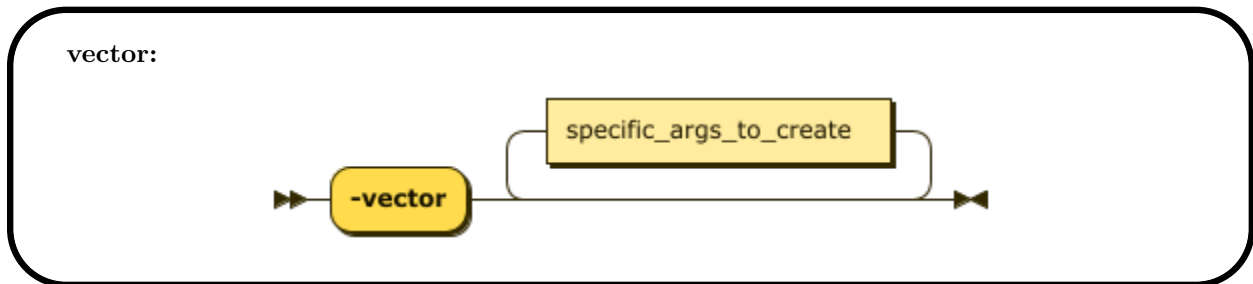


set:

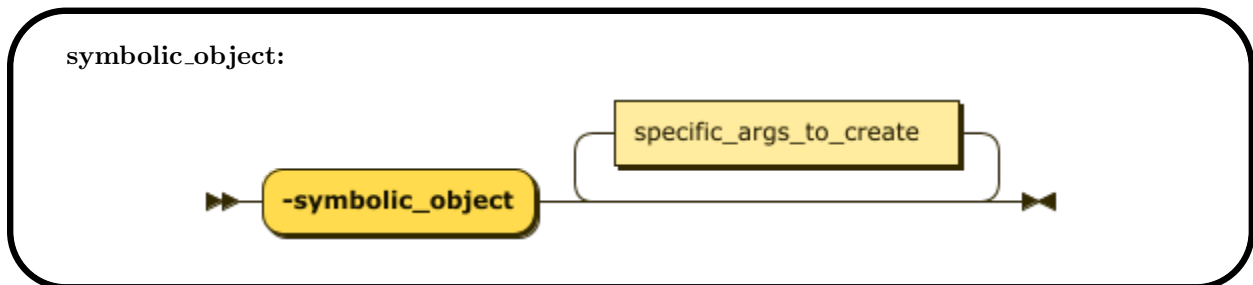




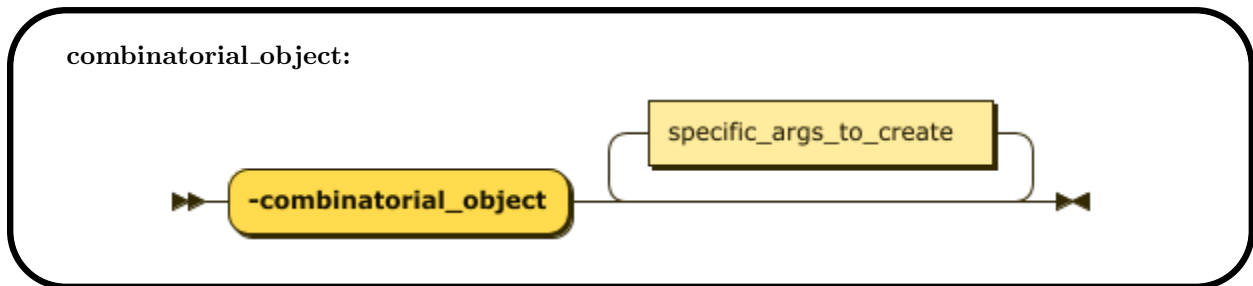
`vector:`



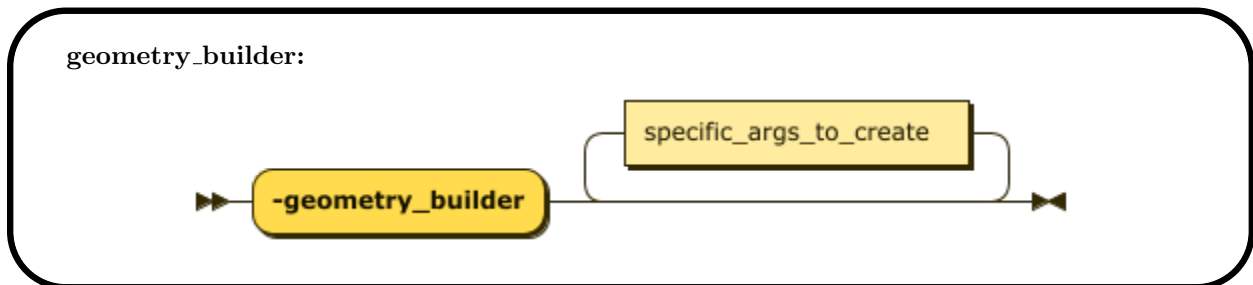
`symbolic_object:`



`combinatorial_object:`

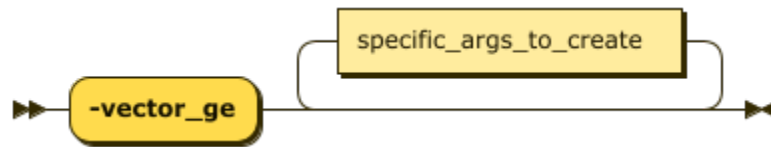


`geometry_builder:`

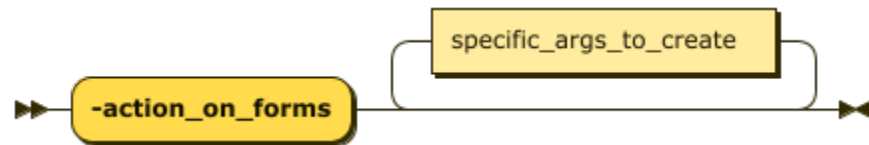


`vector_ge:`

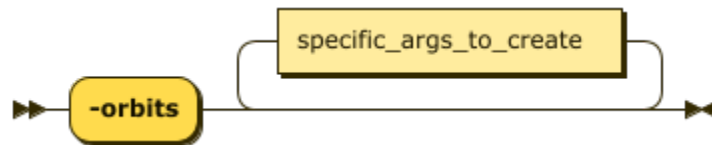




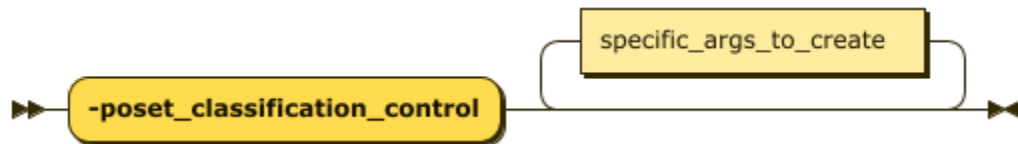
`action_on_forms:`



`orbits:`



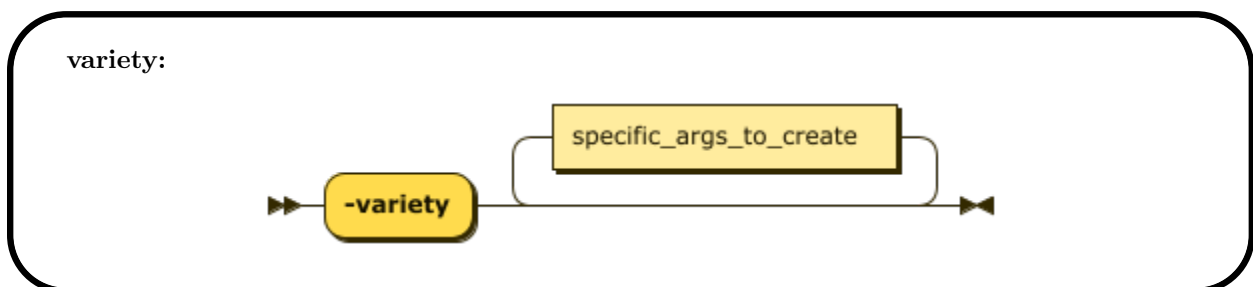
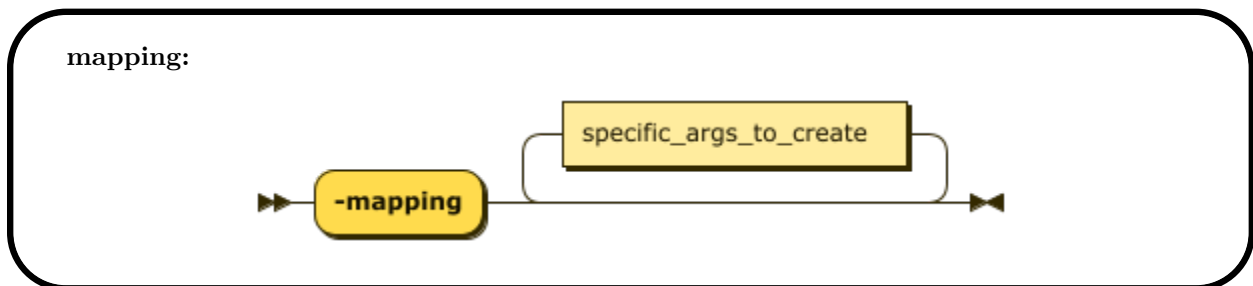
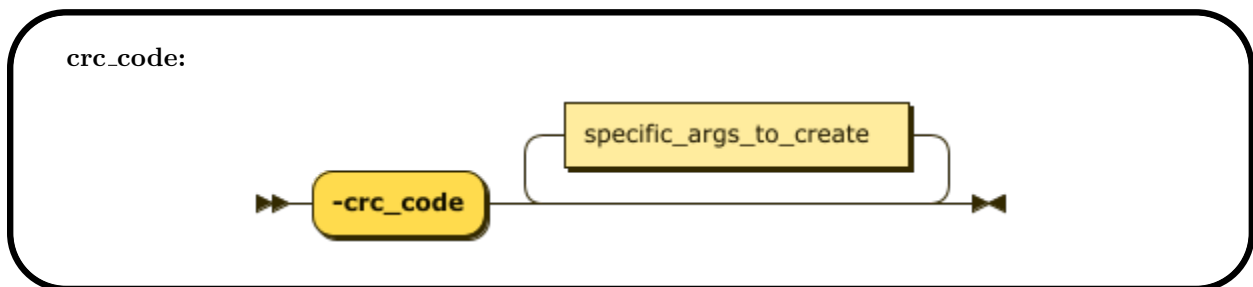
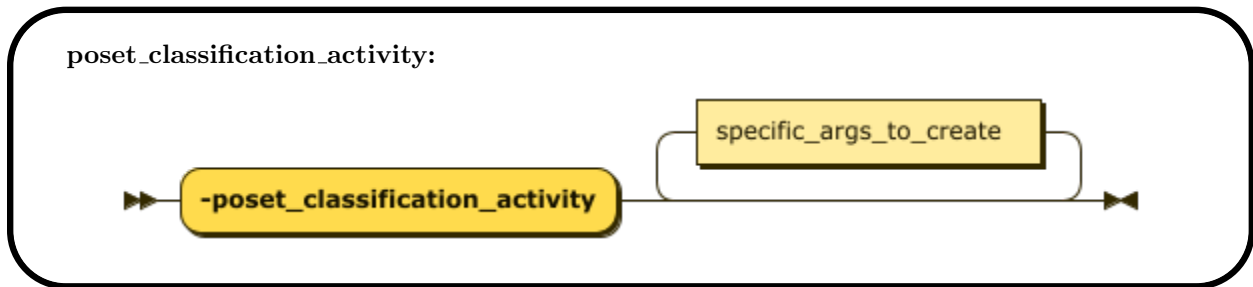
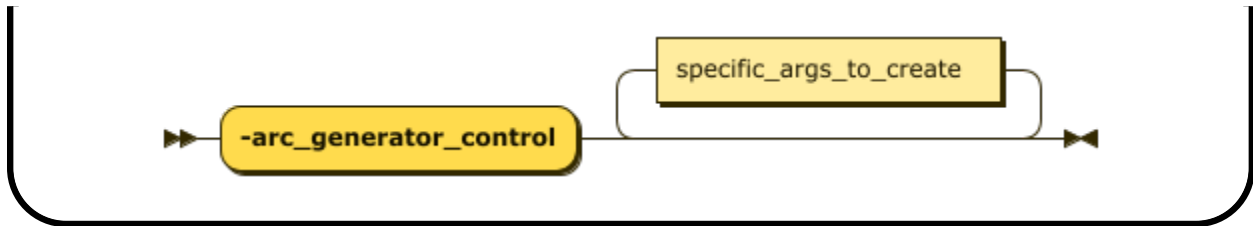
`poset_classification_control:`

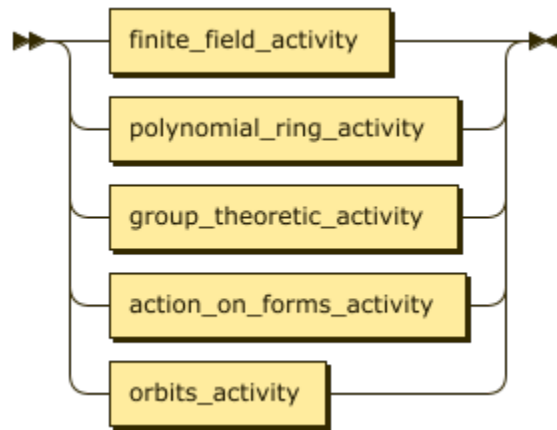


`poset_classification_report_options:`

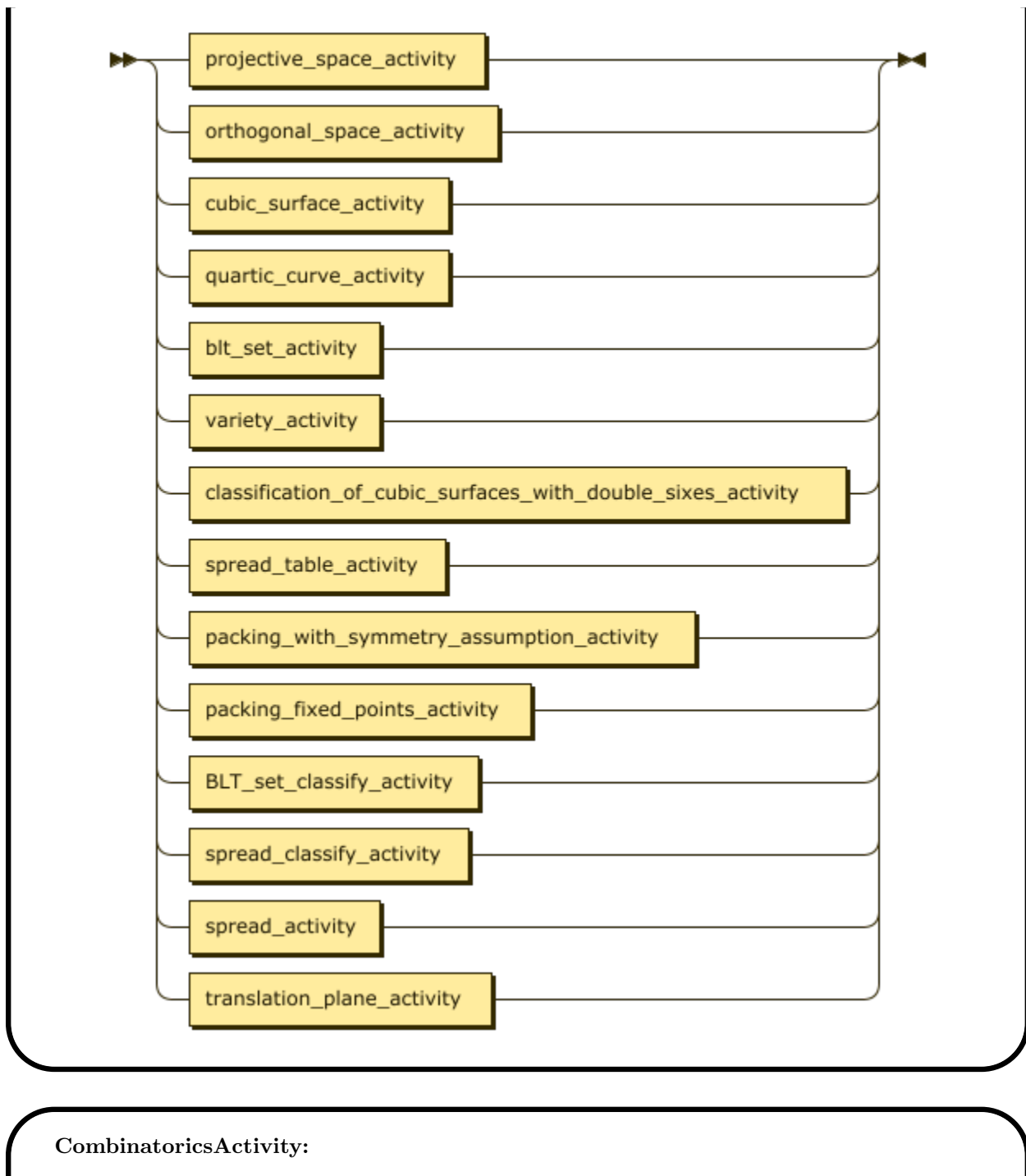


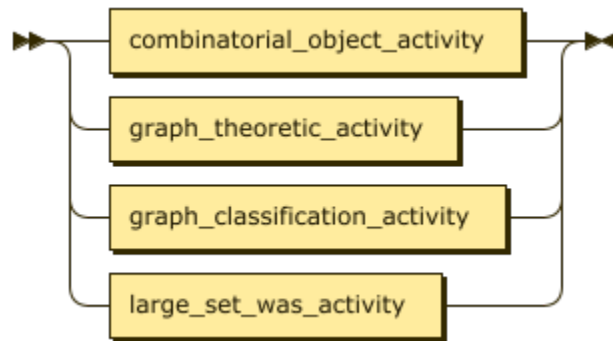
`arc_generator_control:`



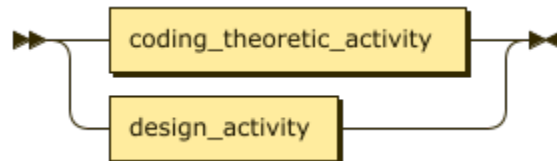


GeometryActivity:

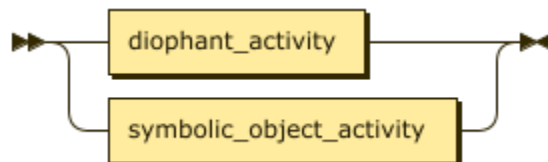




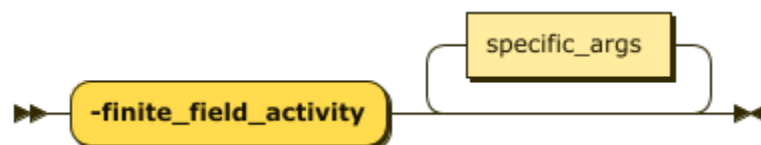
CodingTheoryActivity:



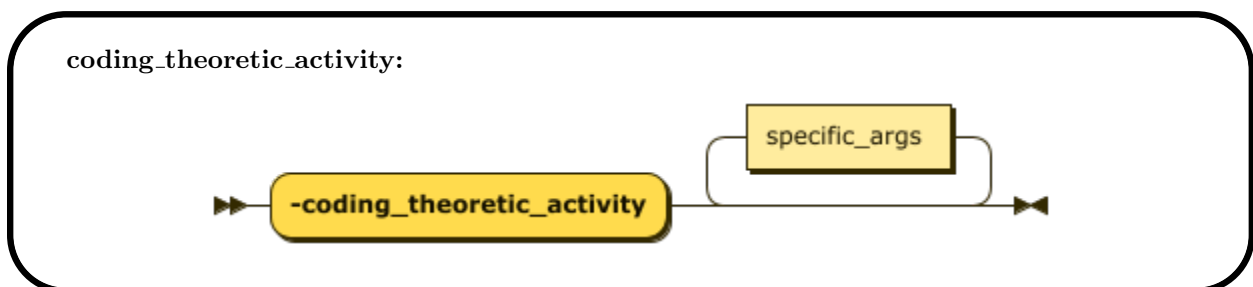
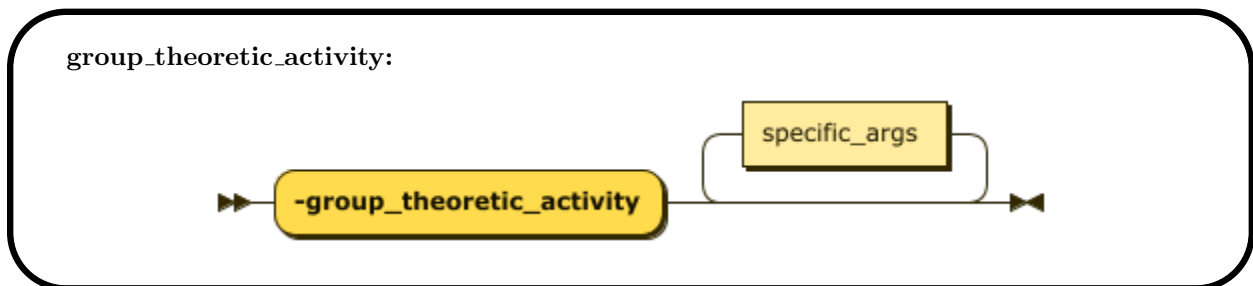
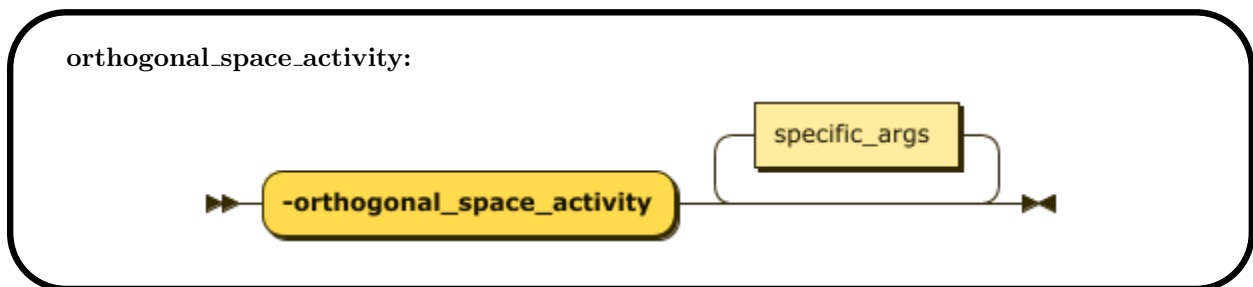
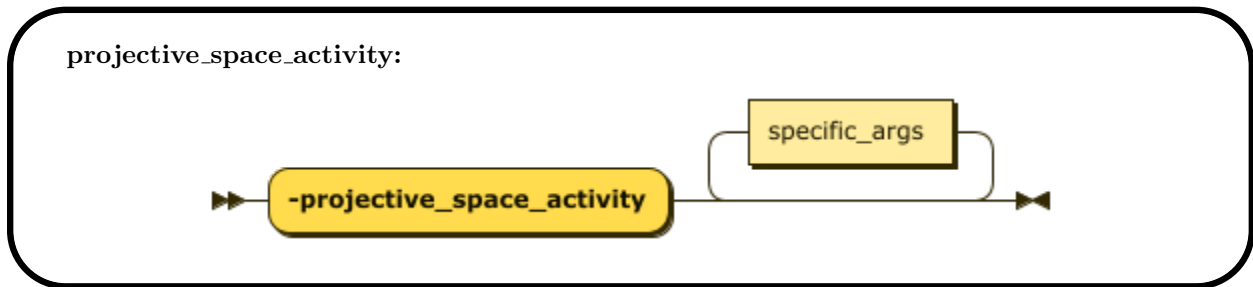
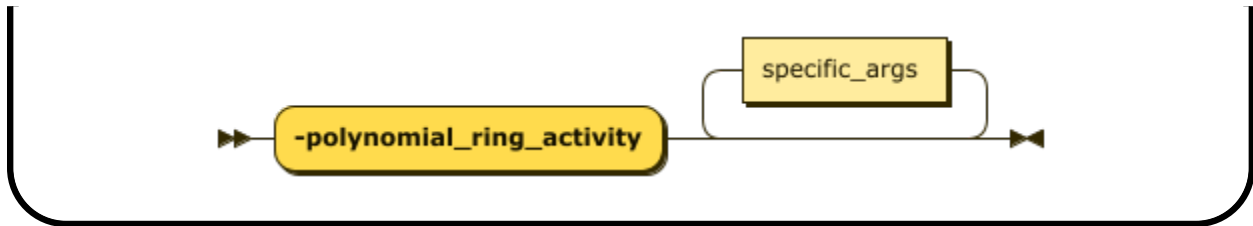
OtherActivity:

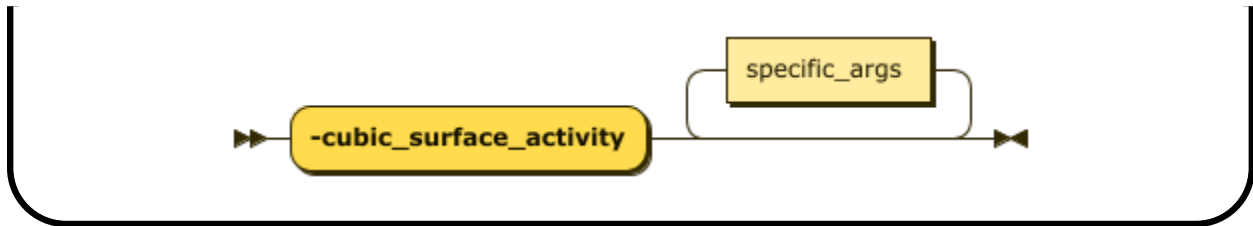


finite_field_activity:

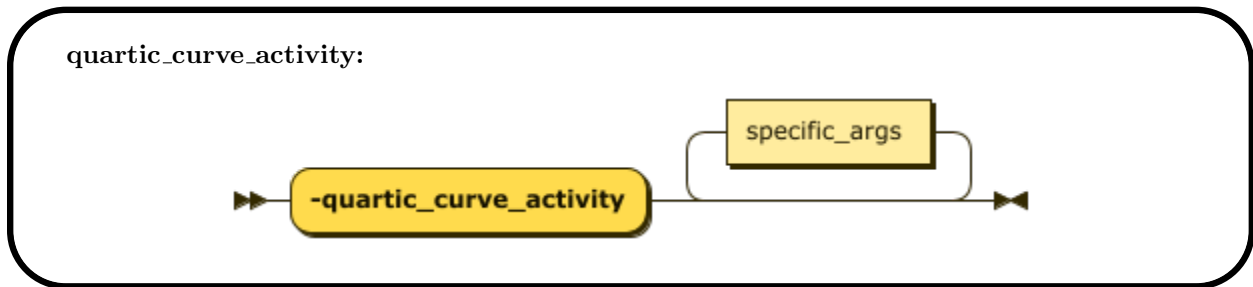


polynomial_ring_activity:

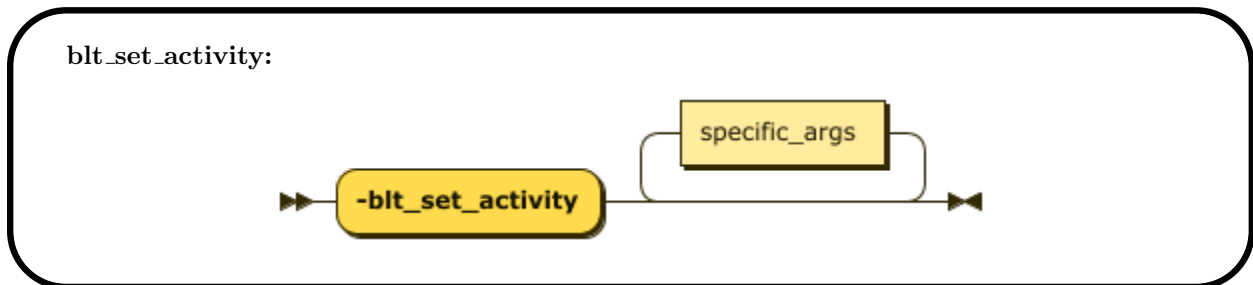




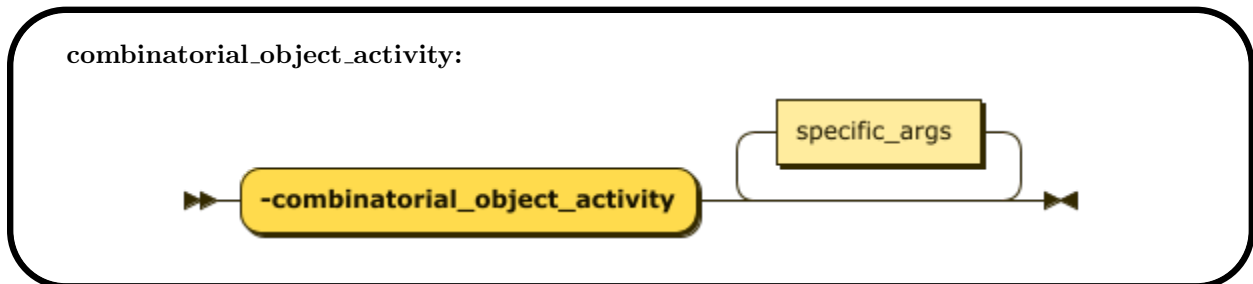
`quartic_curve_activity:`



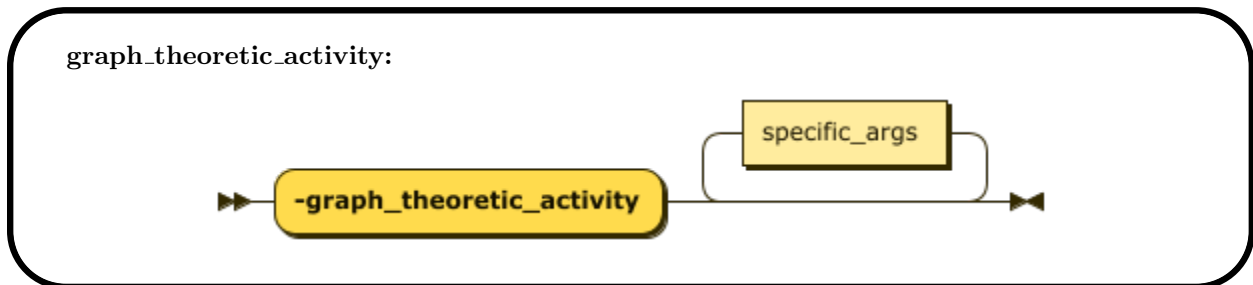
`blt_set_activity:`



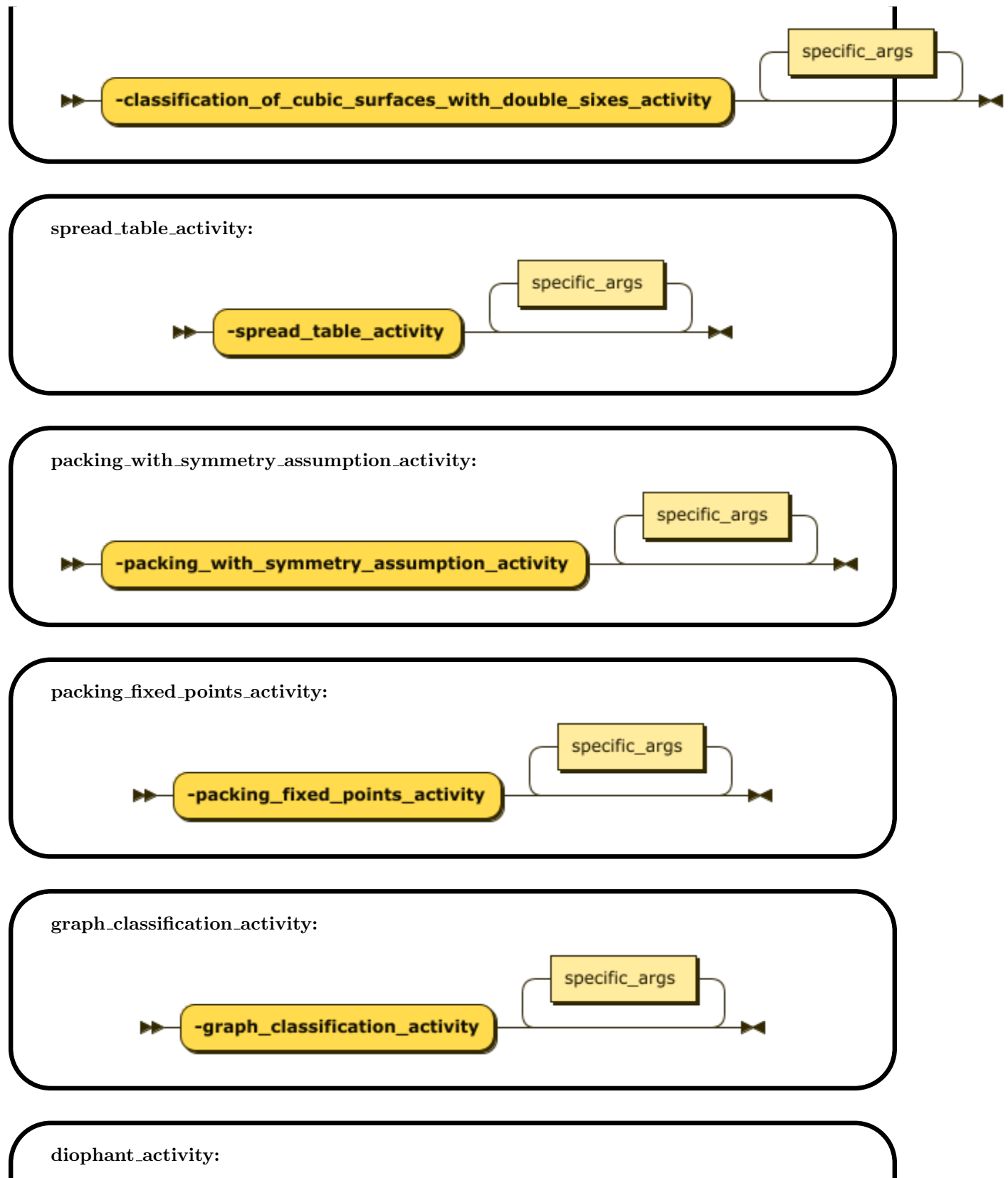
`combinatorial_object_activity:`

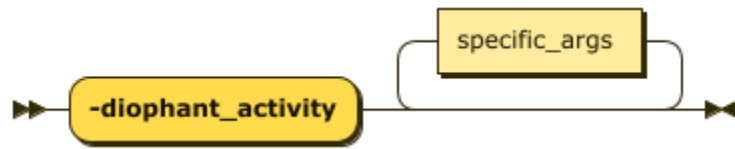


`graph_theoretic_activity:`

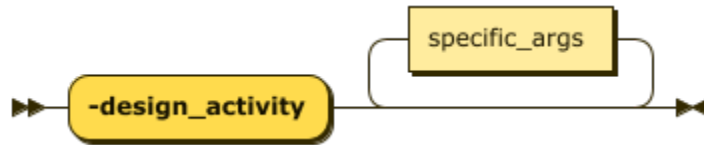


`classification_of_cubic_surfaces_with_double_sixes_activity:`

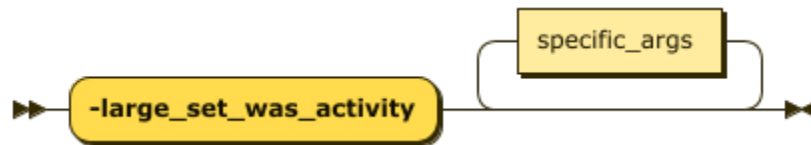




`design_activity:`



`large_set_was_activity:`



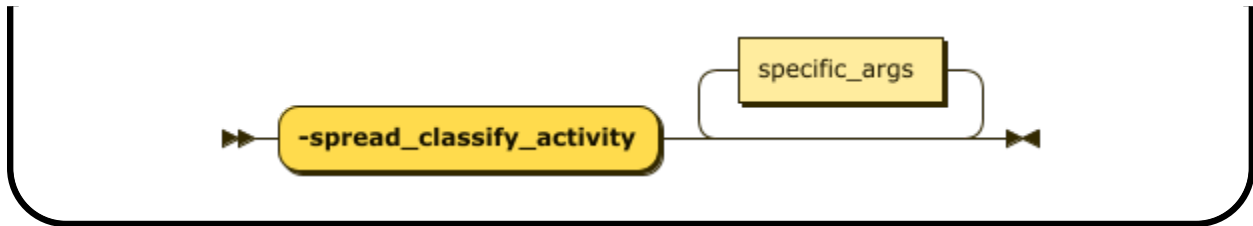
`symbolic_object_activity:`



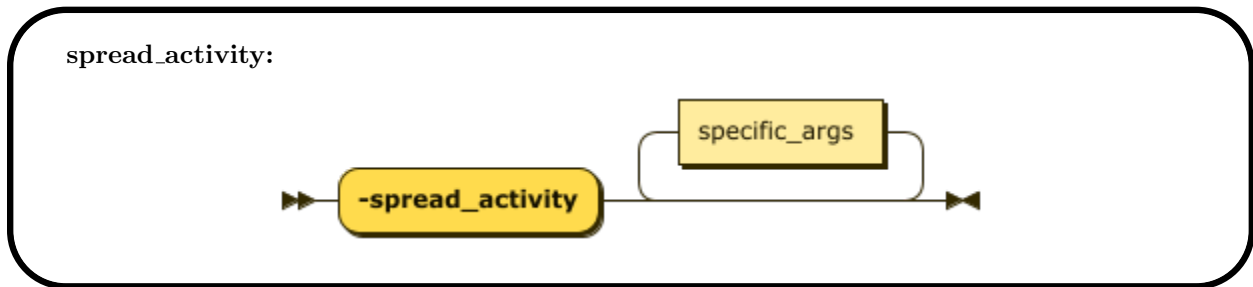
`BLT_set_classify_activity:`



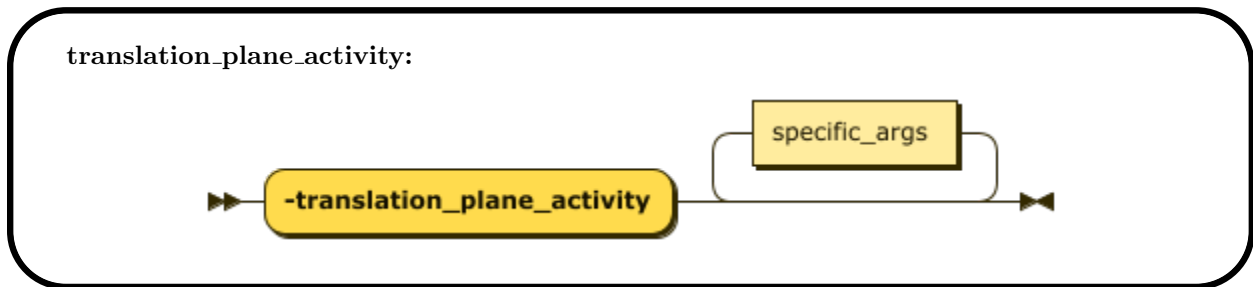
`spread_classify_activity:`



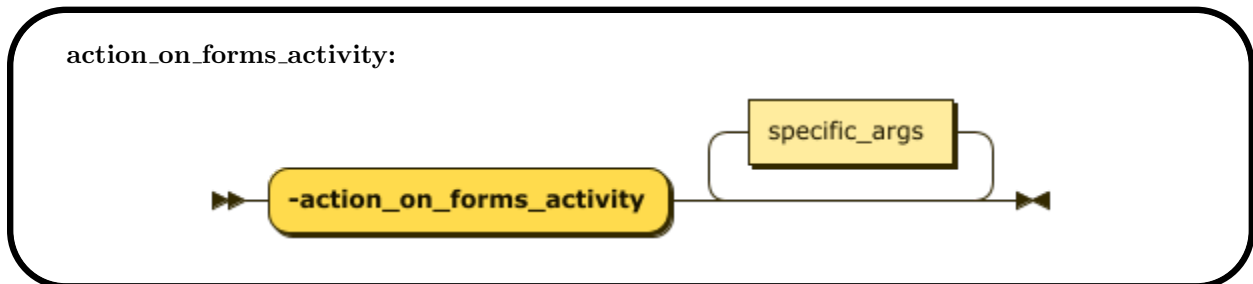
`spread_activity:`



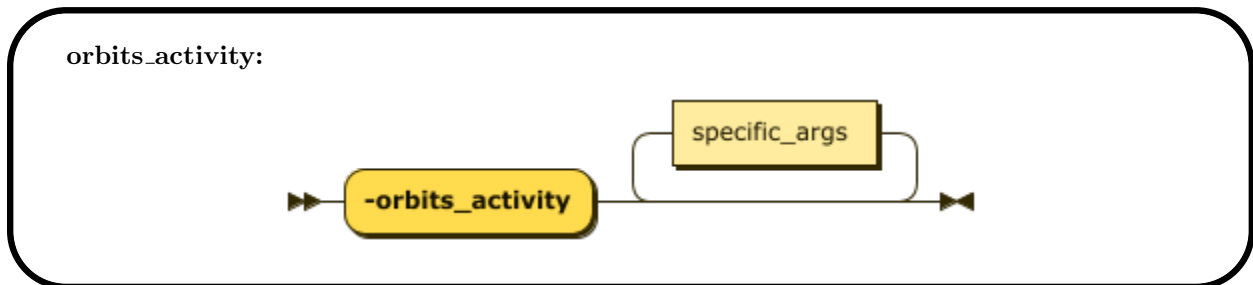
`translation_plane_activity:`



`action_on_forms_activity:`

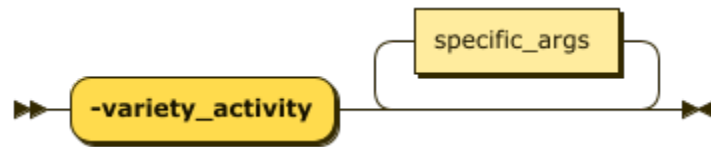


`orbits_activity:`

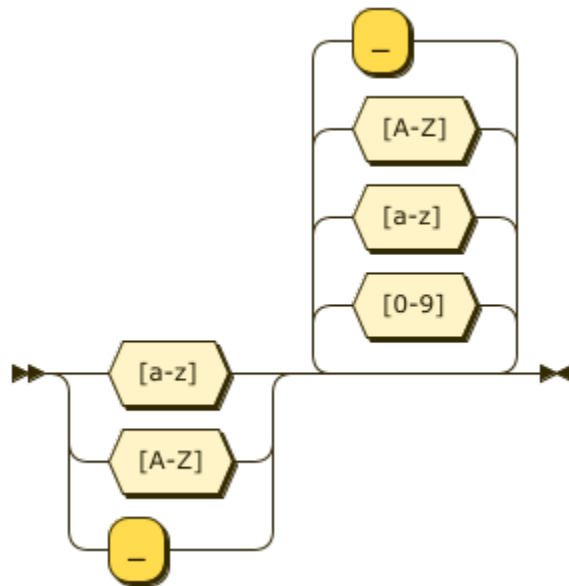


`variety_activity:`

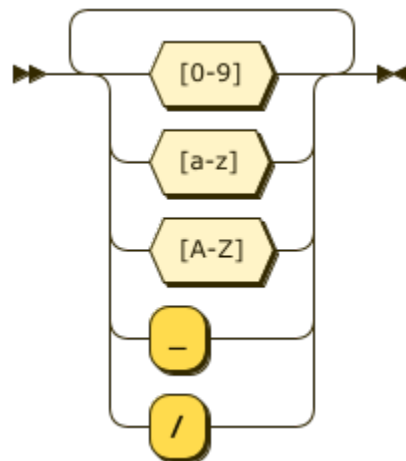


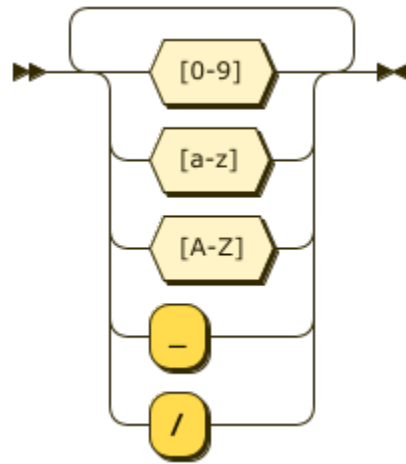
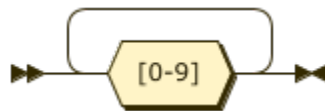


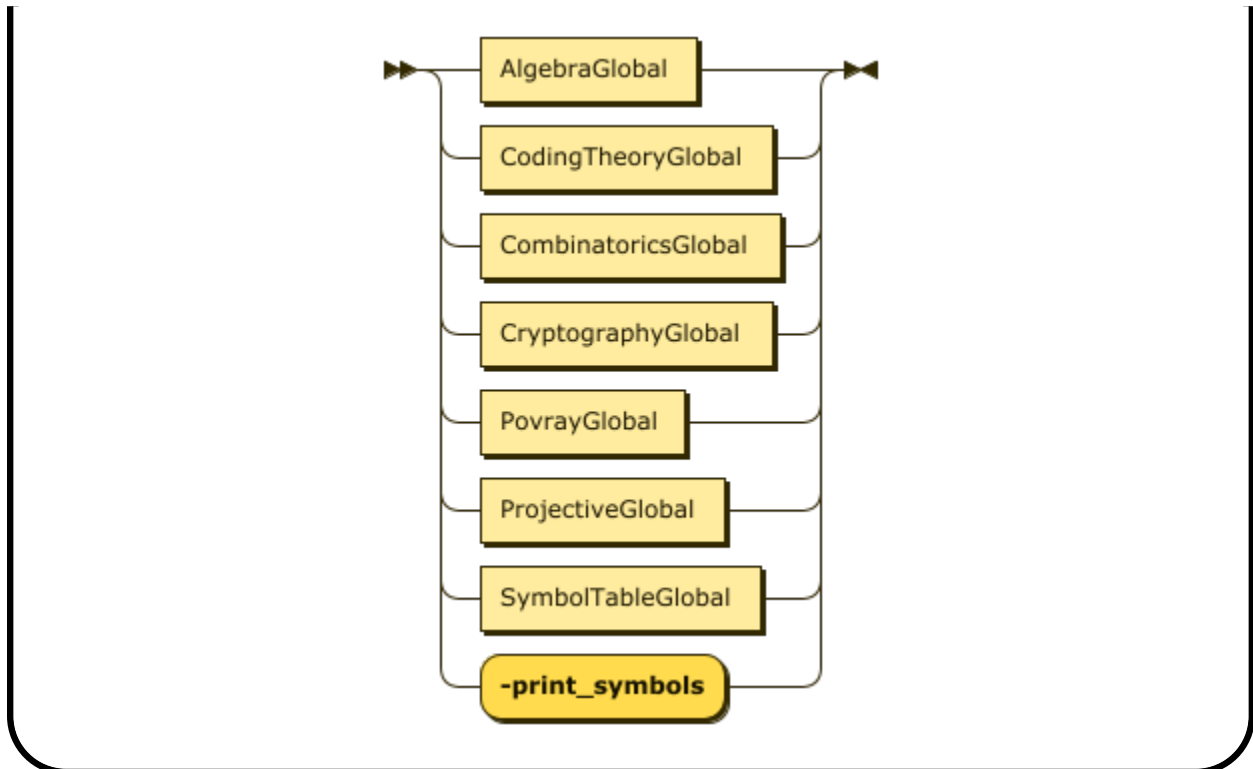
Literal:



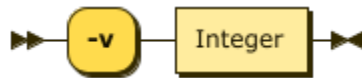
FilePath:



FileMask:**Integer:****GlobalCmd:**



Verbosity:



SeedRandomGen:



MemoryDebug:



OverridePoly:



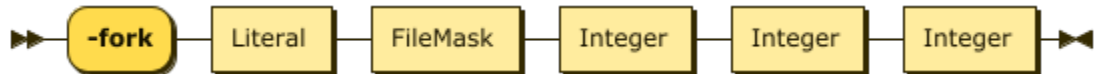
OrbiterPath:



MagmaPath:



Fork:



Appendix B

Data on Cubic Surfaces

B.1 Data on Cubic Surfaces

Cubic Surfaces by Field Order and Number of Eckardt Points

q	total	0	1	2	3	4	5	6	9	10	13	18	45
4	1	0	0	0	0	0	0	0	0	0	0	0	1
7	1	0	0	0	0	0	0	0	0	0	0	1	0
8	1	0	0	0	0	0	0	0	0	0	1	0	0
9	2	0	0	0	0	0	0	0	1	1	0	0	0
11	2	0	0	0	0	0	0	1	0	1	0	0	0
13	4	0	0	0	0	1	0	1	1	0	0	1	0
16	5	0	0	0	1	0	1	0	1	0	1	0	1
17	7	0	1	0	1	2	0	3	0	0	0	0	0
19	10	0	0	2	2	1	0	2	1	1	0	1	0
23	16	0	2	2	4	3	0	5	0	0	0	0	0
25	18	0	4	3	3	2	0	3	2	0	0	1	0
27	11	0	2	2	2	2	0	2	1	0	0	0	0
29	34	1	6	7	11	3	0	5	0	1	0	0	0
31	43	1	10	9	11	3	0	5	2	1	0	1	0
32	11	1	3	0	4	0	2	0	0	0	1	0	0
37	77	4	25	14	18	5	0	7	3	0	0	1	0
41	107	9	37	19	28	5	0	8	0	1	0	0	0
43	126	11	48	21	27	6	0	9	3	0	0	1	0
47	169	20	67	26	38	7	0	11	0	0	0	0	0
49	121	16	46	19	25	4	0	6	3	1	0	1	0

q	total	0	1	2	3	4	5	6	9	10	13	18	45
53	258	40	110	36	52	8	0	12	0	0	0	0	0
59	376	72	166	48	68	8	0	13	0	1	0	0	0
61	427	85	193	53	69	8	0	12	5	1	0	1	0
64	101	20	51	0	17	0	7	0	2	0	3	0	1
67	595	139	275	65	85	10	0	15	5	0	0	1	0
71	731	189	335	75	105	10	0	16	0	1	0	0	0
73	813	216	378	80	105	11	0	16	6	0	0	1	0
79	1081	321	500	97	127	11	0	17	6	1	0	1	0
81	331	100	149	30	38	4	0	6	3	1	0	0	0
83	1292	411	592	107	149	13	0	20	0	0	0	0	0
89	1673	577	759	127	176	13	0	20	0	1	0	0	0
97	2304	868	1033	154	203	15	0	22	8	0	0	1	0
101	2673	1053	1179	169	233	15	0	23	0	1	0	0	0
103	2877	1152	1268	176	232	16	0	24	8	0	0	1	0
107	3313	1379	1437	191	263	17	0	26	0	0	0	0	0
109	3557	1501	1539	201	265	16	0	24	9	1	0	1	0
113	4067	1776	1733	216	297	18	0	27	0	0	0	0	0
121	2753	1262	1135	140	179	11	0	17	7	1	0	1	0
128	929	443	405	0	60	0	18	0	0	0	3	0	0

Total: 30917

0 Eckardt Points

q	total	Ago
29	1	1
31	1	1
32	1	5
37	4	1^4
41	9	1^9
43	11	1^{11}
47	20	1^{20}
49	16	$2^7, 1^9$
53	40	1^{40}
59	72	1^{72}
61	85	1^{85}
64	20	$2^6, 1^{14}$
67	139	1^{139}
71	189	1^{189}
73	216	1^{216}
79	321	1^{321}
81	100	$4^2, 2^{15}, 1^{83}$
83	411	1^{411}
89	577	1^{577}
97	868	1^{868}
101	1053	1^{1053}
103	1152	1^{1152}
107	1379	1^{1379}
109	1501	1^{1501}
113	1776	1^{1776}
121	1262	$2^{95}, 1^{1167}$
128	443	1^{443}

Total: 11667

1 Eckardt Points

q	total	Ago
17	1	8
23	2	2^2
25	4	$16, 4^3$
27	2	2^2
29	6	$4, 2^5$
31	10	2^{10}
32	3	2^3
37	25	$4, 2^{24}$
41	37	$8, 4, 2^{35}$
43	48	2^{48}
47	67	2^{67}
49	46	$16, 8, 4^8, 2^{36}$
53	110	$4^2, 2^{108}$
59	166	2^{166}
61	193	$4^2, 2^{191}$
64	51	$12^3, 4^7, 2^{41}$
67	275	2^{275}
71	335	2^{335}
73	378	$8, 4^2, 2^{375}$
79	500	2^{500}
81	149	$16, 8^6, 4^{15}, 2^{127}$
83	592	2^{592}
89	759	$8, 4^3, 2^{755}$
97	1033	$8, 4^3, 2^{1029}$
101	1179	$4^4, 2^{1175}$
103	1268	2^{1268}
107	1437	2^{1437}
109	1539	$4^4, 2^{1535}$
113	1733	$8, 4^4, 2^{1728}$
121	1135	$16, 8^2, 4^{77}, 2^{1055}$
128	405	2^{405}

Total: 13488

2 Eckardt Points

q	total	Ago
19	2	4^2
23	2	4^2
25	3	$8^2, 4$
27	2	$12, 4$
29	7	4^7
31	9	4^9
37	14	4^{14}
41	19	4^{19}
43	21	4^{21}
47	26	4^{26}
49	19	$8^7, 4^{12}$
53	36	4^{36}
59	48	4^{48}
61	53	4^{53}
67	65	4^{65}
71	75	4^{75}
73	80	4^{80}
79	97	4^{97}
81	30	$16^2, 8^6, 4^{22}$
83	107	4^{107}
89	127	4^{127}
97	154	4^{154}
101	169	4^{169}
103	176	4^{176}
107	191	4^{191}
109	201	4^{201}
113	216	4^{216}
121	140	$8^{27}, 4^{113}$

Total: 2089

3 Eckardt Points

q	total	Ago
16	1	12
17	1	6
19	2	6^2
23	4	6^4
25	3	12, 6^2
27	2	6^2
29	11	6^{11}
31	11	6^{11}
32	4	6^4
37	18	6^{18}
41	28	6^{28}
43	27	6^{27}
47	38	6^{38}
49	25	12^{10} , 6^{15}
53	52	6^{52}
59	68	6^{68}
61	69	6^{69}
64	17	12^4 , 6^{13}
67	85	6^{85}
71	105	6^{105}
73	105	6^{105}
79	127	6^{127}
81	38	12^7 , 6^{31}
83	149	6^{149}
89	176	6^{176}
97	203	6^{203}
101	233	6^{233}
103	232	6^{232}
107	263	6^{263}
109	265	6^{265}
113	297	6^{297}
121	179	12^{24} , 6^{155}
128	60	6^{60}

Total: 2898

4 Eckardt Points

q	total	Ago
13	1	12
17	2	12^2
19	1	12
23	3	12^3
25	2	24, 12
27	2	36, 12
29	3	12^3
31	3	12^3
37	5	12^5
41	5	12^5
43	6	12^6
47	7	12^7
49	4	24^2 , 12^2
53	8	12^8
59	8	12^8
61	8	12^8
67	10	12^{10}
71	10	12^{10}
73	11	12^{11}
79	11	12^{11}
81	4	24^2 , 12^2
83	13	12^{13}
89	13	12^{13}
97	15	12^{15}
101	15	12^{15}
103	16	12^{16}
107	17	12^{17}
109	16	12^{16}
113	18	12^{18}
121	11	24^4 , 12^7

Total: 248

5 Eckardt Points

q	total	Ago
16	1	64
32	2	80, 16
64	7	48, 32^3 , 16^3
128	18	16^{18}

Total: 28

6 Eckardt Points

q	total	Ago
11	1	24
13	1	24
17	3	24^3
19	2	24^2
23	5	24^5
25	3	$48^2, 24$
27	2	24^2
29	5	24^5
31	5	24^5
37	7	24^7
41	8	24^8
43	9	24^9
47	11	24^{11}
49	6	$48^3, 24^3$
53	12	24^{12}
59	13	24^{13}
61	12	24^{12}
67	15	24^{15}
71	16	24^{16}
73	16	24^{16}
79	17	24^{17}
81	6	$48^3, 24^3$
83	20	24^{20}
89	20	24^{20}
97	22	24^{22}
101	23	24^{23}
103	24	24^{24}
107	26	24^{26}
109	24	24^{24}
113	27	24^{27}
121	17	$48^7, 24^{10}$

Total: 378

9 Eckardt Points

q	total	Ago
9	1	432
13	1	108
16	1	216
19	1	54
25	2	216, 108
27	1	162
31	2	54^2
37	3	108, 54^2
43	3	54^3
49	3	216, 108, 54
61	5	108, 54^4
64	2	162, 108
67	5	54^5
73	6	108, 54^5
79	6	54^6
81	3	864, 216, 108
97	8	108, 54^7
103	8	54^8
109	9	108, 54^8
121	7	216, 108^3 , 54^3

Total: 77

10 Eckardt Points

q	total	Ago
9	1	240
11	1	120
19	1	120
29	1	120
31	1	120
41	1	120
49	1	240
59	1	120
61	1	120
71	1	120
79	1	120
81	1	480
89	1	120
101	1	120
109	1	120
121	1	240

Total: 16

13 Eckardt Points

q	total	Ago
8	1	576
16	1	384
32	1	192
64	3	1152, 384, 192
128	3	192^3

Total: 9

18 Eckardt Points

q	total	Ago
7	1	648
13	1	648
19	1	648
25	1	1296
31	1	648
37	1	648
43	1	648
49	1	1296
61	1	648
67	1	648
73	1	648
79	1	648
97	1	648
103	1	648
109	1	648
121	1	1296

Total: 16

45 Eckardt Points

q	total	Ago
4	1	51840
16	1	103680
64	1	155520

Total: 3

Even Characteristic

q	total	0	1	3	5	9	13	45
4	1	0	0	0	0	0	0	1
8	1	0	0	0	0	0	1	0
16	5	0	0	1	1	1	1	1
32	11	1	3	4	2	0	1	0
64	101	20	51	17	7	2	3	1
128	929	443	405	60	18	0	3	0

Total: 1048

0 Eckardt Points

q	total	Ago
32	1	5
64	20	$2^6, 1^{14}$
128	443	1^{443}

Total: 464

1 Eckardt Points

q	total	Ago
32	3	2^3
64	51	$12^3, 4^7, 2^{41}$
128	405	2^{405}

Total: 459

3 Eckardt Points

q	total	Ago
16	1	12
32	4	6^4
64	17	$12^4, 6^{13}$
128	60	6^{60}

Total: 82

5 Eckardt Points

q	total	Ago
16	1	64
32	2	80, 16
64	7	48, 32^3 , 16^3
128	18	16^{18}

Total: 28

9 Eckardt Points

q	total	Ago
16	1	216
64	2	162, 108

Total: 3

13 Eckardt Points

q	total	Ago
8	1	576
16	1	384
32	1	192
64	3	1152, 384, 192
128	3	192^3

Total: 9

45 Eckardt Points

q	total	Ago
4	1	51840
16	1	103680
64	1	155520

Total: 3

Odd Characteristic

q	total	0	1	2	3	4	6	9	10	18
7	1	0	0	0	0	0	0	0	0	1
9	2	0	0	0	0	0	0	1	1	0
11	2	0	0	0	0	0	1	0	1	0
13	4	0	0	0	0	1	1	1	0	1
17	7	0	1	0	1	2	3	0	0	0
19	10	0	0	2	2	1	2	1	1	1
23	16	0	2	2	4	3	5	0	0	0
25	18	0	4	3	3	2	3	2	0	1
27	11	0	2	2	2	2	2	1	0	0
29	34	1	6	7	11	3	5	0	1	0
31	43	1	10	9	11	3	5	2	1	1
37	77	4	25	14	18	5	7	3	0	1
41	107	9	37	19	28	5	8	0	1	0
43	126	11	48	21	27	6	9	3	0	1
47	169	20	67	26	38	7	11	0	0	0
49	121	16	46	19	25	4	6	3	1	1
53	258	40	110	36	52	8	12	0	0	0
59	376	72	166	48	68	8	13	0	1	0
61	427	85	193	53	69	8	12	5	1	1
67	595	139	275	65	85	10	15	5	0	1
71	731	189	335	75	105	10	16	0	1	0
73	813	216	378	80	105	11	16	6	0	1
79	1081	321	500	97	127	11	17	6	1	1
81	331	100	149	30	38	4	6	3	1	0
83	1292	411	592	107	149	13	20	0	0	0
89	1673	577	759	127	176	13	20	0	1	0
97	2304	868	1033	154	203	15	22	8	0	1
101	2673	1053	1179	169	233	15	23	0	1	0
103	2877	1152	1268	176	232	16	24	8	0	1
107	3313	1379	1437	191	263	17	26	0	0	0
109	3557	1501	1539	201	265	16	24	9	1	1
113	4067	1776	1733	216	297	18	27	0	0	0
121	2753	1262	1135	140	179	11	17	7	1	1

Total: 29869

0 Eckardt Points

q	total	Ago
29	1	1
31	1	1
37	4	1^4
41	9	1^9
43	11	1^{11}
47	20	1^{20}
49	16	$2^7, 1^9$
53	40	1^{40}
59	72	1^{72}
61	85	1^{85}
67	139	1^{139}
71	189	1^{189}
73	216	1^{216}
79	321	1^{321}
81	100	$4^2, 2^{15}, 1^{83}$
83	411	1^{411}
89	577	1^{577}
97	868	1^{868}
101	1053	1^{1053}
103	1152	1^{1152}
107	1379	1^{1379}
109	1501	1^{1501}
113	1776	1^{1776}
121	1262	$2^{95}, 1^{1167}$

Total: 11203

1 Eckardt Points

q	total	Ago
17	1	8
23	2	2^2
25	4	$16, 4^3$
27	2	2^2
29	6	$4, 2^5$
31	10	2^{10}
37	25	$4, 2^{24}$
41	37	$8, 4, 2^{35}$
43	48	2^{48}
47	67	2^{67}
49	46	$16, 8, 4^8, 2^{36}$
53	110	$4^2, 2^{108}$
59	166	2^{166}
61	193	$4^2, 2^{191}$
67	275	2^{275}
71	335	2^{335}
73	378	$8, 4^2, 2^{375}$
79	500	2^{500}
81	149	$16, 8^6, 4^{15}, 2^{127}$
83	592	2^{592}
89	759	$8, 4^3, 2^{755}$
97	1033	$8, 4^3, 2^{1029}$
101	1179	$4^4, 2^{1175}$
103	1268	2^{1268}
107	1437	2^{1437}
109	1539	$4^4, 2^{1535}$
113	1733	$8, 4^4, 2^{1728}$
121	1135	$16, 8^2, 4^{77}, 2^{1055}$

Total: 13029

2 Eckardt Points

q	total	Ago
19	2	4^2
23	2	4^2
25	3	$8^2, 4$
27	2	$12, 4$
29	7	4^7
31	9	4^9
37	14	4^{14}
41	19	4^{19}
43	21	4^{21}
47	26	4^{26}
49	19	$8^7, 4^{12}$
53	36	4^{36}
59	48	4^{48}
61	53	4^{53}
67	65	4^{65}
71	75	4^{75}
73	80	4^{80}
79	97	4^{97}
81	30	$16^2, 8^6, 4^{22}$
83	107	4^{107}
89	127	4^{127}
97	154	4^{154}
101	169	4^{169}
103	176	4^{176}
107	191	4^{191}
109	201	4^{201}
113	216	4^{216}
121	140	$8^{27}, 4^{113}$

Total: 2089

3 Eckardt Points

q	total	Ago
17	1	6
19	2	6^2
23	4	6^4
25	3	$12, 6^2$
27	2	6^2
29	11	6^{11}
31	11	6^{11}
37	18	6^{18}
41	28	6^{28}
43	27	6^{27}
47	38	6^{38}
49	25	$12^{10}, 6^{15}$
53	52	6^{52}
59	68	6^{68}
61	69	6^{69}
67	85	6^{85}
71	105	6^{105}
73	105	6^{105}
79	127	6^{127}
81	38	$12^7, 6^{31}$
83	149	6^{149}
89	176	6^{176}
97	203	6^{203}
101	233	6^{233}
103	232	6^{232}
107	263	6^{263}
109	265	6^{265}
113	297	6^{297}
121	179	$12^{24}, 6^{155}$

Total: 2816

4 Eckardt Points

q	total	Ago
13	1	12
17	2	12^2
19	1	12
23	3	12^3
25	2	24, 12
27	2	36, 12
29	3	12^3
31	3	12^3
37	5	12^5
41	5	12^5
43	6	12^6
47	7	12^7
49	4	24^2 , 12^2
53	8	12^8
59	8	12^8
61	8	12^8
67	10	12^{10}
71	10	12^{10}
73	11	12^{11}
79	11	12^{11}
81	4	24^2 , 12^2
83	13	12^{13}
89	13	12^{13}
97	15	12^{15}
101	15	12^{15}
103	16	12^{16}
107	17	12^{17}
109	16	12^{16}
113	18	12^{18}
121	11	24^4 , 12^7

Total: 248

6 Eckardt Points

q	total	Ago
11	1	24
13	1	24
17	3	24^3
19	2	24^2
23	5	24^5
25	3	$48^2, 24$
27	2	24^2
29	5	24^5
31	5	24^5
37	7	24^7
41	8	24^8
43	9	24^9
47	11	24^{11}
49	6	$48^3, 24^3$
53	12	24^{12}
59	13	24^{13}
61	12	24^{12}
67	15	24^{15}
71	16	24^{16}
73	16	24^{16}
79	17	24^{17}
81	6	$48^3, 24^3$
83	20	24^{20}
89	20	24^{20}
97	22	24^{22}
101	23	24^{23}
103	24	24^{24}
107	26	24^{26}
109	24	24^{24}
113	27	24^{27}
121	17	$48^7, 24^{10}$

Total: 378

9 Eckardt Points

q	total	Ago
9	1	432
13	1	108
19	1	54
25	2	216, 108
27	1	162
31	2	54^2
37	3	108, 54^2
43	3	54^3
49	3	216, 108, 54
61	5	108, 54^4
67	5	54^5
73	6	108, 54^5
79	6	54^6
81	3	864, 216, 108
97	8	108, 54^7
103	8	54^8
109	9	108, 54^8
121	7	216, 108^3 , 54^3

Total: 74

10 Eckardt Points

q	total	Ago
9	1	240
11	1	120
19	1	120
29	1	120
31	1	120
41	1	120
49	1	240
59	1	120
61	1	120
71	1	120
79	1	120
81	1	480
89	1	120
101	1	120
109	1	120
121	1	240

Total: 16

18 Eckardt Points

q	total	Ago
7	1	648
13	1	648
19	1	648
25	1	1296
31	1	648
37	1	648
43	1	648
49	1	1296
61	1	648
67	1	648
73	1	648
79	1	648
97	1	648
103	1	648
109	1	648
121	1	1296

Total: 16

Appendix C

Linux Background

C.1 Command Sheet

This is a collection of unix/linux commands that are frequently used. For Windows users under wsl, please type wsl (and space) immediately before each command.

Unix/Linux Command Reference

FOSSwire.com

File Commands	System Info
ls - directory listing	date - show the current date and time
ls -al - formatted listing with hidden files	cal - show this month's calendar
cd dir - change directory to <i>dir</i>	uptime - show current uptime
cd - change to home	w - display who is online
pwd - show current directory	whoami - who you are logged in as
mkdir dir - create a directory <i>dir</i>	finger user - display information about <i>user</i>
rm file - delete <i>file</i>	uname -a - show kernel information
rm -r dir - delete directory <i>dir</i>	cat /proc/cpuinfo - cpu information
rm -f file - force remove <i>file</i>	cat /proc/meminfo - memory information
rm -rf dir - force remove directory <i>dir</i> *	man command - show the manual for <i>command</i>
cp file1 file2 - copy <i>file1</i> to <i>file2</i>	df - show disk usage
cp -r dir1 dir2 - copy <i>dir1</i> to <i>dir2</i> ; create <i>dir2</i> if it doesn't exist	du - show directory space usage
mv file1 file2 - rename or move <i>file1</i> to <i>file2</i> if <i>file2</i> is an existing directory, moves <i>file1</i> into directory <i>file2</i>	free - show memory and swap usage
ln -s file link - create symbolic link <i>link</i> to <i>file</i>	whereis app - show possible locations of <i>app</i>
touch file - create or update <i>file</i>	which app - show which <i>app</i> will be run by default
cat > file - places standard input into <i>file</i>	
more file - output the contents of <i>file</i>	
head file - output the first 10 lines of <i>file</i>	
tail file - output the last 10 lines of <i>file</i>	
tail -f file - output the contents of <i>file</i> as it grows, starting with the last 10 lines	
Process Management	Compression
ps - display your currently active processes	tar cf file.tar files - create a tar named <i>file.tar</i> containing <i>files</i>
top - display all running processes	tar xf file.tar - extract the files from <i>file.tar</i>
kill pid - kill process id <i>pid</i>	tar czf file.tar.gz files - create a tar with Gzip compression
killall proc - kill all processes named <i>proc</i> *	tar xzf file.tar.gz - extract a tar using Gzip
bg - lists stopped or background jobs; resume a stopped job in the background	tar cjf file.tar.bz2 - create a tar with Bzip2 compression
fg - brings the most recent job to foreground	tar xjf file.tar.bz2 - extract a tar using Bzip2
fg n - brings job <i>n</i> to the foreground	gzip file - compresses <i>file</i> and renames it to <i>file.gz</i>
File Permissions	gzip -d file.gz - decompresses <i>file.gz</i> back to <i>file</i>
chmod octal file - change the permissions of <i>file</i> to <i>octal</i> , which can be found separately for user, group, and world by adding:	
4 - read (r) 2 - write (w) 1 - execute (x)	
Examples:	
chmod 777 - read, write, execute for all	
chmod 755 - rwx for owner, rx for group and world	
For more options, see man chmod .	
SSH	Network
ssh user@host - connect to <i>host</i> as <i>user</i>	ping host - ping <i>host</i> and output results
ssh -p port user@host - connect to <i>host</i> on port <i>port</i> as <i>user</i>	whois domain - get whois information for <i>domain</i>
ssh-copy-id user@host - add your key to <i>host</i> for <i>user</i> to enable a keyed or passwordless login	dig domain - get DNS information for <i>domain</i>
	dig -x host - reverse lookup <i>host</i>
	wget file - download <i>file</i>
	wget -c file - continue a stopped download
Searching	Installation
grep pattern files - search for <i>pattern</i> in <i>files</i>	Install from source:
grep -r pattern dir - search recursively for <i>pattern</i> in <i>dir</i>	./configure
command grep pattern - search for <i>pattern</i> in the output of <i>command</i>	make
locate file - find all instances of <i>file</i>	make install
	dpkg -i pkg.deb - install a package (Debian)
	rpm -Uvh pkg.rpm - install a package (RPM)
	Shortcuts
	Ctrl+C - halts the current command
	Ctrl+Z - stops the current command, resume with fg in the foreground or bg in the background
	Ctrl+D - log out of current session, similar to exit
	Ctrl+W - erases one word in the current line
	Ctrl+U - erases the whole line
	Ctrl+R - type to bring up a recent command
	!! - repeats the last command
	exit - log out of current session
	* use with extreme caution.



Appendix D

Testing

D.1 Testing

Date of Test	Name of Tester

Section	Check	Problem report
Chapter 2		
2.1		
2.2		
2.3		
2.4		
2.5		
2.6		
Chapter 3		
3.1		
3.2		
3.3		
3.4		
Chapter 4		
4.1		
4.2		
4.3		
4.4		
4.5		

Section	Check	Problem report
Chapter 5		
5.1		
5.2		
5.3		
5.4		
5.5		
5.6		
5.7		
5.8		
5.9		
Chapter 6		
6.1		
6.2		
Chapter 7		
7.1		
7.2		
7.3		
7.4		
7.5		
7.6		
Chapter 8		
8.1		
8.2		
8.3		
8.4		
8.5		
8.6		
8.7		

Section	Check	Problem report
Chapter 9		
9.1		
9.2		
9.3		
9.4		
9.5		
9.6		
9.7		
Chapter 10		
10.1		
10.2		
10.3		
Chapter 11		
11.1		
11.2		
11.3		
11.4		
11.5		
11.6		
11.7		
11.8		
11.9		
11.10		

Section	Check	Problem report
Chapter 12		
12.1		
12.2		
12.3		
12.4		
12.5		
12.6		
12.7		
12.8		
Chapter 13		
13.1		
13.2		
13.3		
Chapter 14		
14.1		
14.2		
14.3		
14.4		
14.5		
Chapter 15		
15.1		
15.2		
15.3		
15.4		

Section	Check	Problem report
Chapter 16		
16.1		
16.2		
16.3		
Chapter 17		
17.2		
17.3		
17.4		
17.5		
17.6		
17.7		
17.8		
Chapter 18		
18.1		
Chapter 19		
19.1		
19.2		
19.3		
19.4		
19.5		
Chapter 20		
20.1		
20.2		
Chapter 21		
21.1		
21.2		

Appendix E

The Makefile

E.1 The Makefile

The examples in this guide are taken from a makefile. The complete makefile is distributed with Orbiter in the directory

examples/users_guide

The makefile can be used as a sort of index of all Orbiter commands. In most cases, it is possible to find an example for a given Orbiter command by simply searching for this particular command inside the makefile.

We advise against running any commands in the orbiter directory tree. Running the makefile within the orbiter tree will create files that make updating orbiter through git more difficult. Therefore, we recommend to copy the makefile to some location *outside* the orbiter tree, see Section 1.5.

The contents of the makefile is shown below. Note that all makefile variables are defined in the header. The actual commands follow the header. The ordering follows the ordering of sections in this user’s guide. The “test”-commands are useful for testing all commands in a section or chapter in a single go. There is even a test command that runs all commands in the user’s guide.

```
1 #####
2 # orbiter/examples/users_guide/makefile
3 #####
4
5
6
7 ORBITER_PATH=~/orbiter
8
9 #ORBITER_PATH=/s/hipft-logger/b/nobackup/abetten/orbiter
10 #ORBITER_PATH=/scratch/betten/COMPILE/orbiter
11 #ORBITER_PATH=/data/accounts/fac/betten/orbiter
12
13
14 # uncomment exactly one of the following two lines.
15 # uncomment the first if you want to run orbiter through docker.
16 # uncomment the second if you have an installed copy of orbiter and you want to r
   un it directly
17 #ORBITER_PATH=docker run -it --volume ${PWD}:/mnt -w /mnt abetten/orbiter
18
19 ORBITER_EXE_PATH=$(ORBITER_PATH)/src/apps/orbiter/
20
21 ORBITER=$(ORBITER_EXE_PATH)orbiter.out
22 #ORBITER=$(ORBITER_EXE_PATH)orbiter.out -parse_commands_only
```

```

23
24 SANDBOX=$(ORBITER_PATH)/src/apps/sandbox/sandbox.out
25
26 #####
27 # additional configurations for when you want to
28 # compile automatically generated code
29 #####
30
31 SRC=$(ORBITER_PATH)/src
32 MY_CPP = g++
33 MY_CC = gcc
34 CPPFLAGS = -Wall -I../DEV.23/orbiter/src/lib -std=c++14
35 LIB = $(SRC)/lib/liborbiter.a -lpthread
36 LFLAGS = -lm -Wl,-rpath -Wl,/usr/local/gcc-8.2.0/lib64
37
38
39 #MAGMA_PATH=/usr/local/magma
40 MAGMA_PATH=""
41
42 MAGMA_EXE=$(MAGMA_PATH)magma
43 #MAGMA_EXE=echo
44
45 #OPEN=open
46 OPEN=echo
47
48
49 #PDFLATEX=pdflatex
50 PDFLATEX=echo
51
52 #####
53 # End of configuration part
54 #####
55
56 protect:
57 ▷ echo "please type make followed by a command name"
58
59
60
61 GINAC_PATH=$(ORBITER_PATH)/src/apps/ginac
62 SANDBOX_PATH=$(ORBITER_PATH)/src/apps/sandbox
63
64 update:
65 ▷ cd $(ORBITER_PATH); make clean;
66 ▷ cd $(ORBITER_PATH); make cleana; git pull; make
67
68
69 update_all:
70 ▷ cd $(ORBITER_PATH); make clean; git pull; make
71
72
73 sandbox:
74 ▷ $(SANDBOX_PATH)/sandbox.out
75
76
77 #####
78 # testing:
79 #####
80
81 test:

```

```

82  ▷ make test_2
83  ▷ make test_3
84  ▷ make test_4
85  ▷ make test_5
86  ▷ make test_6
87  ▷ make test_7
88  ▷ make test_8
89  ▷ make test_9
90  ▷ make test_10
91  ▷ make test_11
92  ▷ make test_12
93  ▷ make test_13
94  ▷ make test_14
95  ▷ make test_15
96  ▷ make test_16
97  ▷ make test_17
98  ▷ make test_18
99  ▷ make test_19
100 ▷ make test_20
101 ▷ make test_21
102
103
104
105 test_problem:
106
107
108
109 #####
110 # Makefile Variables
111
112
113 # Section 2.4 set builder
114
115
116
117 # Section 2.5 vector builder
118
119
120
121
122 # Section 2.6 Vector of group element
123
124
125 HIRSCHFELD_STAB_GENERATORS="\
126 1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,1, \
127 1,0,0,0,0,2,0,0,0,0,2,0,0,0,0,1,0, \
128 1,0,0,0,0,3,0,0,0,0,3,0,1,0,0,1,0, \
129 1,0,0,0,0,1,0,0,1,1,1,0,0,1,0,1,1, \
130 1,0,0,0,0,0,1,0,0,1,0,0,0,0,0,1,0, \
131 0,1,0,0,1,0,0,0,0,0,0,1,0,0,1,0,0"
132
133
134 CLASS_2A_REP="1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,1"
135
136 FILE_NAME_CLASS_2A="PGL_4_4-Subgroup-Hirschfeld-Stab-51840-class-of_2A.csv"
137
138
139
140 # Section 2.7 Symbolic Algebra

```

```

141
142 TEST_FORMULA="(-a+b*b)*x*x+a*b*x"
143
144 BCH_POLYNOMIAL_ALFA_F256="X^2 + 167*X + 214"
145
146 BCH_POLYNOMIAL_BRAVO_F256="X^4+213*X^3+27*X^2+23*X+1"
147
148 VANDERMONDE_4X4_FORMULA="1,x0,x0^2,x0^3,1,x1,x1^2,x1^3,\
149 1,x2,x2^2,x2^3,1,x3,x3^2,x3^3"
150
151 ECKARDT_SURFACE_EQN="a*X3^3-a*b^2*(X0^2+X1^2+X2^2)*X3+b^3*(a^2+1)*X0*X1*X2"
152
153 # general normal form for cubic surfaces with 27 rational lines:
154
155 F_abcd_eqn="-(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*(b - d)*X0^2*X2 \
156 + (a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*(a + b - c - d)*X0*X1*X2 \
157 + (a^2*c - a^2*d - a*c^2 + b*c^2 + a*d - b*c)*(b - d)*X0*X1*X3 \
158 - (a*d - b*c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X0*X2^2 \
159 - (a^2*c*d - a*b*c^2 - a^2*d + a*b*d + b*c^2 - b*c*d)*(b - d)*X0*X2*X3 \
160 - (a - c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X1^2*X2 \
161 - (a - c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X1^2*X3 \
162 + (a*d - b*c)*(a*b*c - a*b*d - a*c*d + b*c*d + a*d - b*c)*X1*X2^2 \
163 + ((1+1)*a^2*b*c*d - a^2*b*d^2 - (1+1)*a^2*c*d^2 \
164 - (1+1)*a*b^2*c^2 + a*b^2*c*d + (1+1)*a*b*c^2*d + a*b*c*d^2 \
165 - b^2*c^2*d - a^2*b*c + a^2*c*d + a^2*d^2 + a*b^2*c + a*b*c^2 \
166 - (1+1+1)*a*b*c*d - a*c^2*d + a*c*d^2 + b^2*c^2)*X1*X2*X3 \
167 + c*a*(a*d - b*c - a + b + c - d)*(b - d)*X1*X3^2"
168
169
170 # Chapter 3 Basic algebra
171
172
173
174 # Section 3.3 Linear algebra over finite fields
175
176
177
178
179 V7_VANDERMONDE_EXTENDED="\
180 1,0,0,0,0,0,0,1,0,0,0,0,0,0, \
181 1,1,1,1,1,1,1,0,1,0,0,0,0,0, \
182 1,2,4,1,2,4,1,0,0,1,0,0,0,0, \
183 1,3,2,6,4,5,1,0,0,0,1,0,0,0, \
184 1,4,2,1,4,2,1,0,0,0,0,1,0,0, \
185 1,5,4,6,2,3,1,0,0,0,0,0,1,0, \
186 1,6,1,6,1,6,1,0,0,0,0,0,0,1"
187
188
189
190 CODE_RS_6_4_7="\
191 621000 \
192 062100 \
193 006210 \
194 000621"
195
196
197
198 # Chapter 4 Projective Geometry
199

```



```

200
201
202 # Section 4.4 Advanced topics
203
204 H1_LABEL="H1"
205
206 H1_GENs=-PGGL 4 4 \
207 ▷ -subgroup_by_generators "H1" 4 2 \
208 ▷ "1,0,0,0,1,1,0,0,0,0,1,0,0,0,1,1,0, \
209 ▷ 1,0,0,0,0,1,0,0,1,0,1,0,0,1,0,1,0" \
210 ▷ -end
211
212
213
214
215 # Section 4.5 Geometric objects
216
217
218 DOILY="Row,C0,C1,C2\
219 \n0,0,12,13\
220 \n1,1,12,14\
221 \n2,8,9,12\
222 \n3,4,6,8\
223 \n4,6,10,14\
224 \n5,3,7,8\
225 \n6,7,10,13\
226 \n7,4,11,13\
227 \n8,3,11,14\
228 \n9,0,5,6\
229 \n10,1,5,7\
230 \n11,5,9,11\
231 \n12,0,2,3\
232 \n13,1,2,4\
233 \n14,2,9,10\
234 \nEND"
235
236
237 # q=17:
238 # 3 is p.e. mod 17.
239 # so we pick f=3.
240 # then,  $2f^2=18=1$ 
241 #  $4f = 12$ 
242
243 #  $X^4 - Y^4 - Z^4 + 2f^2YZ^2 + 4fX^2YZ$ 
244
245 #(1,-1,-1,0,0,0,0,0,0,0,2f^2,4f,0,0)
246
247 EDGE_CURVE_Q17_EQUATION="1,16,16,0,0,0,0,0,0,0,1,12,0,0"
248
249 EDGE_CURVE_Q17_AS_POINTS="4, 7, 16, 19, 20, 23, 32, 35, 89, 100, 244, 251"
250
251
252
253
254
255 # Chapter 5 Group Theory
256
257
258

```

```

259 # Section 5.1 generic permutation groups
260
261
262
263 GENERATORS_M12="3,1,9,0,10,11,6,7,8,2,4,5, 7,2,3,1,11,9,5,8,0,6,4,10"
264
265 # Section 5.2 Matrix groups
266
267
268 # Section 5.3 Subgroups
269
270 GEN_C13="1,2,3,4,5,6,7,8,9,10,11,12,0"
271 # (0,1,2,3,4,5,6,7,8,9,10,11,12)
272
273
274
275
276
277
278 GENERATORS_HESSE_GROUP="\
279 3000300030 \
280 2000201230 \
281 1000100111 \
282 1000220200 \
283 1002312010 \
284 0331003211 \
285 2200011331"
286
287
288
289
290 GENERATORS_WEYL_GROUP_E8="\
291 -1,-1,-1,-1,0,0,0,0, \
292 0,0,0,1,0,0,0,0, \
293 1,0,0,0,0,0,0,0, \
294 0,0,1,0,0,0,0,0, \
295 0,1,0,1,1,0,0,0, \
296 0,0,0,0,0,1,0,0, \
297 0,0,0,0,0,0,1,0, \
298 0,0,0,0,0,0,0,1, \
299 -1,0,-1,-1,-1,-1,-1, \
300 0,1,0,1,1,1,1,1, \
301 1,0,0,0,0,0,0,0, \
302 0,0,1,0,0,0,0,0, \
303 0,0,0,1,0,0,0,0, \
304 0,0,0,0,1,0,0,0, \
305 0,0,0,0,0,1,0,0, \
306 0,0,0,0,0,0,1,0"
307
308
309
310 PGOp_6_2_GENS="\
311 1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,1,0, \
312 1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,1,1,1,0,0,1,0,0,1,0, \
313 1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,1,0,0,0,1,0,0,0, \
314 1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,1,0,0,0,0,0,0,1,0,0,1,0,0,0,1,0, \
315 1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,1,0,1,0,0,0,0,1,0,0,1, \
316 1,0,0,0,0,0,0,1,1,1,1,1,1,0,0,0,1,0,1,0,0,0,0,1,1,0,1,0,0,1,0, \
317 1,0,0,0,0,0,0,1,0,1,0,0,1,0,1,0,0,1,0,0,0,0,1,1,0,0,0,0,0,1, \

```

```

318 0,1,0,1,1,0,0,0,1,0,0,0,0,1,0,0,0,0,1,0,1,0,1,0,0,0,0,0,1,0,0,1,1,0,0,1, \
319 0,0,1,1,1,1,1,0,1,0,0,0,0,1,1,1,1,1,1,1,1,0,0,0,1,0,1,0,1,0,1,1,0, \
320 0,0,0,0,1,0,1,0,1,1,1,1,1,0,1,0,1,0,1,1,0,0,1,1,0,0,0,0,1,1,1,1,1,0"
321
322
323
324
325
326
327 # Section 5.4 New actions from old
328
329
330 SURFACE_Q13_STAB_GENS="1,0,0,0,9,12,0,0,10,0,12,0,9,0,0,12, \
331 1,0,0,0,0,12,12,6,6,0,0,7,1,0,2,0, \
332 0,1,1,7,3,9,9,11,2,10,10,3,9,9,1,11"
333
334
335
336
337
338 HERMITIAN_SURFACE_Q9_EQUATION="X0^4+X1^4+X2^4+X3^4"
339
340 HERMITIAN_SURFACE_Q9_POINTS="7, 8, 10, 12, 15, 16, 18, 20, 22, 23, 26, 28, 31, 32
, 35, 37, 39, 48, 58, 59, 62, 64, 66, 76, 77, 80, 82, 84, 95, 96, 98, 100, 102, 1
03, 106, 108, 111, 112, 115, 117, 119, 128, 138, 139, 142, 144, 146, 156, 157, 16
0, 162, 164, 174, 175, 178, 180, 182, 190, 202, 203, 205, 207, 211, 212, 214, 216
, 217, 229, 230, 232, 234, 235, 247, 248, 250, 252, 254, 255, 258, 260, 262, 271,
283, 284, 286, 288, 292, 293, 295, 297, 298, 310, 311, 313, 315, 316, 328, 329,
331, 333, 334, 346, 347, 349, 351, 355, 356, 358, 360, 362, 363, 366, 368, 371, 3
72, 375, 377, 382, 383, 385, 387, 389, 390, 393, 395, 400, 401, 403, 405, 407, 40
8, 411, 413, 415, 427, 428, 430, 432, 436, 437, 439, 441, 443, 444, 447, 449, 452
, 453, 456, 458, 463, 464, 466, 468, 470, 471, 474, 476, 481, 482, 484, 486, 488,
489, 492, 494, 497, 498, 501, 503, 505, 514, 526, 527, 529, 531, 535, 536, 538,
540, 541, 553, 554, 556, 558, 559, 571, 572, 574, 576, 577, 589, 590, 592, 594, 5
98, 599, 601, 603, 605, 606, 609, 611, 614, 615, 618, 620, 625, 626, 628, 630, 63
2, 633, 636, 638, 643, 644, 646, 648, 650, 651, 654, 656, 659, 660, 663, 665, 667
, 676, 688, 689, 691, 693, 697, 698, 700, 702, 703, 715, 716, 718, 720, 721, 733,
734, 736, 738, 739, 751, 752, 754, 756, 760, 761, 763, 765, 767, 768, 771, 773,
776, 777, 780, 782, 787, 788, 790, 792, 794, 795, 798, 800, 805, 806, 808, 810, 8
12, 813, 816, 818"
341
342 HERMITIAN_SURFACE_Q9_NB_POINTS=280
343
344
345 PGL_3_13_CLASS_2A_REP="1, 1, 5, 9, 9, 8, 5, 11, 0"
346
347 PGL_3_13_CLASS_2B_REP="1, 1, 11, 1, 6, 12, 7, 10, 5"
348
349
350 # Section 5.6 Group theoretic activities based on Magma
351
352 # large sets of PG(2,3):
353
354 GENERATORS_H5="1,2,3,4,0,6,7,8,9,5,10,11,12"
355 # (0, 1, 2, 3, 4)(5, 6, 7, 8, 9)
356
357 GENERATORS_N5="\
358 0,1,2,3,4,9,5,6,7,8,10,11,12, \
359 0,1,2,3,4,5,6,7,8,9,10,12,11, \

```

```

360 0,4,3,2,1,5,9,8,7,6,10,11,12, \
361 0,2,4,1,3,5,7,9,6,8,10,11,12, \
362 0,1,2,3,4,5,6,7,8,9,11,10,12, \
363 1,2,3,4,0,6,7,8,9,5,10,11,12, \
364 5,9,8,7,6,0,4,3,2,1,10,11,12"
365
366
367 PGOM_6_2_CLASS_2A_REP="1,0,0,0,0,0,0,1,0,0,\
368 0,0,0,0,1,0,0,0,0,0,1,1,1,1,0,0,1,0,0,1,0,0,1,0,1,0"
369
370 FILE_NAME_PGOM_6_2_CLASS_2A="PGom_6_2_class_of_2A.csv"
371
372 PGOM_6_2_CLASS_2A_REP_CLEAN="1,0,0,0,0,0,0,1,0,0,\
373 0,0,0,0,0,1,0,0,0,0,1,0,0,0,0,0,0,0,1,0,0,0,0,0,0,1"
374
375 PGOM_6_2_CLASS_2A_CENTRALIZER_GENS="\
376 1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,1,1, \
377 1,0,0,0,0,0,0,1,0,0,0,0,0,0,0,1,0,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0, \
378 1,0,0,0,0,0,0,1,0,0,0,0,0,0,0,1,0,0,0,0,0,0,0,1,0,0,0,0,0,0,1, \
379 1,0,0,0,0,0,1,1,0,0,0,1,0,0,1,0,0,0,0,0,0,1,0,0,1,0,0,0,1,1,0,0,0,0,1, \
380 1,0,0,0,0,0,1,1,0,0,1,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,1,0,1,0,0,0,1,1, \
381 1,0,0,0,0,0,1,1,1,1,0,0,1,0,0,1,0,0,1,0,1,0,0,0,0,0,0,1,0,0,0,0,0,1, \
382 0,1,1,1,1,0,1,0,1,1,1,0,1,1,0,1,1,0,1,1,1,0,1,0,0,0,0,1,0,1,1,1,1,1, \
383 1,1,0,0,0,1,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,1,0,0,1,1,0,0,0,0,1, \
384 0,1,0,0,0,0,1,0,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,1"
385
386
387
388 # Section 5.7 Group theoretic activities based on GAP
389
390
391
392 # Section 5.8 Group theory, advanced topics
393
394
395
396
397 #Co3 from Conway et al., 1985 (ATLAS)
398 #order = 495766656000
399 #Co3 from the paper by Suleiman and Wilson 1997
400
401
402 CONWAY_GEN1="\
403 1101110001000001010000\
404 1111010111110100001011\
405 00000010000000100010101\
406 1111100110110001001110\
407 0101010000000010011101\
408 00000100000000100010101\
409 0010000000000100010101\
410 0001000011000000111111\
411 1110100100110100010011\
412 0000000000000110010101\
413 0000000000100100010101\
414 0110111111010011101111\
415 0000000000001100010101\
416 0000000000000100000101\
417 00000000001000100010101\
418 0000000000000100011101\

```

```

419 0001000110000010011010\
420 0000000000000000010101\
421 0000000000000101010101\
422 0000000000000100010100\
423 0000000000000100010111\
424 0000000000000100010001"
425
426 CONWAY_GEN2="\
427 0101000010111010111111\
428 0110010100011110110000\
429 0011010000111111010111\
430 0001101110001011010011\
431 1010010000100001011110\
432 1101000000001010100011\
433 1100101010001111010101\
434 1000110100110101010101\
435 0100110001010000000111\
436 1100000010100101010010\
437 0101110110011100000101\
438 0101111101010011111001\
439 1000010101010101010001\
440 0001010000111100100111\
441 0011010010111011001111\
442 0100110010110011111010\
443 1101011001111101100011\
444 0100101001001000100001\
445 1100101100001001110011\
446 0101110110010100000001\
447 0000001101111000101110\
448 1101101010101110000101"
449
450
451
452
453
454 Ree_gen1="21,5,1,6,17,1,1, 3,13,5,21,6,6,18, 21,3,21,21,22,6,14, \
455 14,18,1,5,13,6,7, 3,3,2,1,24,16,3, 17,3,22,10,16,24,26, \
456 21,21,6,18,20,2,5"
457
458 Ree_gen2="16,3,11,5,16,22,20, 24,6,18,24,7,1,26, 9,23,17,18,23,20,13, \
459 9,7,2,15,17,5,11, 3,3,6,21,4,24,16, 25,8,6,24,21,12,7, \
460 24,15,2,13,11,14,24"
461
462
463 HIGMAN_SIMS_GENS="0, 1, 2, 3, 4, 5, 10, 16, 17, 19, 6, 11, 18, 20, 21, 22, 7, 8,
12, 9, 13, 14, 15, 23, 27, 26, 25, 24, 30, 31, 28, 29, 33, 32, 35, 34, 36, 37, 38
, 39, 41, 40, 43, 42, 44, 45, 46, 47, 50, 51, 48, 49, 55, 54, 53, 52, 57, 56, 59,
58, 63, 62, 61, 60, 65, 64, 67, 66, 70, 71, 68, 69, 72, 73, 74, 75, 79, 78, 77,
76, 83, 82, 81, 80, 89, 95, 99, 94, 97, 84, 92, 96, 90, 98, 87, 85, 91, 88, 93, 8
6,\
464 0, 1, 2, 3, 4, 5, 22, 21, 20, 18, 15, 11, 19, 17, 16, 10, 14, 13, 9, 12, 8, 7, 6,
23, 26, 27, 24, 25, 30, 31, 28, 29, 32, 33, 34, 35, 38, 39, 36, 37, 42, 43, 40,
41, 45, 44, 47, 46, 50, 51, 48, 49, 54, 55, 52, 53, 56, 57, 58, 59, 62, 63, 60, 6
1, 66, 67, 64, 65, 68, 69, 70, 71, 74, 75, 72, 73, 77, 76, 79, 78, 83, 82, 81, 80
, 86, 96, 84, 93, 92, 99, 97, 95, 88, 87, 98, 91, 85, 90, 94, 89,\
465 0, 1, 2, 3, 4, 5, 12, 8, 7, 15, 18, 11, 6, 14, 13, 9, 17, 16, 10, 22, 21, 20, 19,
23, 24, 25, 26, 27, 31, 30, 29, 28, 35, 34, 33, 32, 38, 39, 36, 37, 41, 40, 43,
42, 46, 47, 44, 45, 50, 51, 48, 49, 53, 52, 55, 54, 59, 58, 57, 56, 61, 60, 63, 6
2, 65, 64, 67, 66, 68, 69, 70, 71, 73, 72, 75, 74, 76, 77, 78, 79, 81, 80, 83, 82

```

```

, 95, 89, 91, 92, 93, 85, 94, 86, 87, 88, 90, 84, 99, 98, 97, 96,\
466 0, 1, 2, 3, 4, 5, 20, 19, 22, 16, 13, 11, 21, 10, 18, 17, 9, 15, 14, 7, 6, 12, 8,
    23, 26, 27, 24, 25, 28, 29, 30, 31, 34, 35, 32, 33, 39, 38, 37, 36, 41, 40, 43,
    42, 46, 47, 44, 45, 49, 48, 51, 50, 52, 53, 54, 55, 57, 56, 59, 58, 63, 62, 61, 6
    0, 66, 67, 64, 65, 69, 68, 71, 70, 74, 75, 72, 73, 76, 77, 78, 79, 82, 83, 80, 81
    , 88, 98, 92, 91, 84, 97, 99, 87, 86, 95, 96, 93, 94, 89, 85, 90,\
467 0, 1, 2, 3, 11, 4, 9, 14, 15, 22, 17, 5, 13, 20, 19, 21, 10, 16, 18, 7, 12, 8, 6,
    23, 25, 26, 24, 27, 31, 29, 28, 30, 36, 37, 38, 39, 42, 43, 40, 41, 34, 35, 32,
    33, 47, 44, 46, 45, 58, 59, 56, 57, 50, 51, 48, 49, 52, 53, 54, 55, 66, 67, 64, 6
    5, 70, 71, 68, 69, 60, 61, 62, 63, 78, 79, 76, 77, 81, 80, 83, 82, 75, 74, 73, 72
    , 85, 96, 86, 94, 91, 97, 98, 93, 89, 88, 99, 90, 84, 92, 95, 87,\
468 0, 1, 2, 8, 6, 7, 22, 21, 20, 15, 18, 12, 19, 9, 14, 13, 16, 10, 17, 11, 3, 5, 4,
    24, 26, 25, 23, 27, 30, 37, 34, 43, 32, 40, 28, 36, 38, 41, 35, 31, 42, 29, 33,
    39, 54, 51, 47, 57, 56, 44, 48, 52, 45, 55, 49, 58, 50, 46, 53, 59, 86, 88, 72, 6
    4, 92, 95, 61, 81, 77, 69, 90, 94, 84, 87, 60, 80, 68, 76, 89, 85, 93, 91, 65, 73
    , 62, 99, 74, 83, 66, 96, 97, 67, 63, 75, 98, 82, 78, 70, 71, 79,\
469 0, 1, 2, 4, 3, 11, 17, 18, 10, 22, 7, 5, 16, 21, 20, 19, 6, 12, 8, 9, 13, 14, 15,
    23, 27, 26, 25, 24, 35, 34, 33, 32, 28, 29, 30, 31, 42, 41, 43, 40, 37, 39, 38,
    36, 49, 48, 51, 50, 47, 46, 45, 44, 56, 59, 57, 58, 53, 55, 54, 52, 62, 63, 61, 6
    0, 72, 73, 74, 75, 78, 79, 77, 76, 65, 64, 67, 66, 71, 70, 68, 69, 82, 83, 81, 80
    , 90, 92, 99, 85, 97, 87, 95, 96, 89, 98, 84, 94, 86, 93, 88, 91,\
470 0, 1, 13, 21, 5, 6, 8, 3, 10, 20, 4, 18, 22, 7, 16, 11, 19, 17, 12, 9, 14, 2, 15,
    40, 30, 39, 25, 35, 37, 31, 33, 26, 43, 23, 41, 42, 34, 27, 38, 29, 24, 32, 28,
    36, 51, 67, 93, 97, 81, 71, 74, 55, 88, 56, 89, 60, 90, 79, 46, 96, 69, 83, 53, 7
    5, 95, 62, 98, 58, 86, 44, 76, 87, 72, 68, 80, 52, 54, 70, 82, 73, 64, 84, 85, 48
    , 61, 92, 57, 47, 94, 77, 65, 66, 99, 45, 63, 50, 91, 49, 59, 78,\
471 0, 17, 8, 13, 7, 15, 9, 3, 19, 6, 12, 18, 2, 4, 5, 10, 22, 20, 16, 14, 21, 11, 1,
    90, 47, 72, 37, 92, 97, 71, 35, 56, 42, 65, 87, 49, 85, 82, 27, 62, 78, 53, 28,
    99, 81, 30, 51, 88, 94, 69, 24, 77, 86, 57, 43, 61, 34, 95, 64, 54, 60, 55, 39, 9
    3, 96, 46, 40, 79, 25, 67, 89, 74, 91, 45, 33, 80, 58, 29, 84, 73, 70, 48, 98, 38
    , 31, 52, 36, 66, 75, 50, 63, 44, 26, 23, 68, 59, 32, 76, 83, 41,\
472 0, 7, 20, 3, 5, 8, 16, 13, 1, 17, 19, 21, 11, 14, 4, 2, 12, 15, 9, 6, 22, 10, 18,
    69, 39, 49, 84, 91, 45, 60, 29, 65, 73, 55, 42, 80, 87, 33, 58, 93, 94, 77, 90,
    24, 63, 46, 66, 28, 52, 74, 82, 41, 92, 57, 34, 88, 98, 26, 97, 76, 71, 23, 70, 6
    8, 31, 67, 62, 44, 64, 30, 47, 61, 43, 81, 54, 72, 83, 40, 53, 75, 51, 37, 86, 95
    , 96, 78, 27, 89, 32, 59, 25, 56, 38, 79, 35, 36, 85, 99, 50, 48,\
473 51, 42, 91, 70, 85, 66, 82, 38, 21, 84, 4, 36, 2, 29, 65, 80, 28, 68, 13, 8, 41,
    16, 99, 9, 44, 7, 64, 79, 5, 88, 76, 59, 17, 63, 67, 57, 61, 98, 46, 1, 89, 48, 2
    2, 95, 32, 15, 56, 37, 30, 12, 23, 83, 20, 27, 40, 72, 19, 62, 71, 54, 18, 26, 39
    , 77, 93, 49, 10, 92, 24, 87, 35, 3, 33, 55, 14, 43, 45, 69, 6, 74, 11, 53, 90, 7
    3, 60, 25, 31, 94, 86, 50, 97, 47, 58, 52, 75, 0, 81, 78, 96, 34,\
474 40, 77, 78, 21, 91, 6, 50, 63, 49, 45, 99, 92, 61, 1, 5, 47, 13, 18, 94, 66, 65,
    59, 57, 44, 4, 41, 11, 46, 98, 53, 31, 97, 71, 19, 7, 27, 22, 70, 8, 25, 28, 52,
    85, 89, 69, 93, 26, 95, 9, 54, 15, 30, 88, 84, 68, 24, 29, 17, 16, 55, 37, 83, 62
    , 51, 67, 58, 75, 33, 87, 86, 76, 0, 64, 56, 32, 72, 12, 20, 79, 23, 80, 36, 48,
    60, 10, 82, 2, 90, 43, 39, 38, 3, 96, 42, 81, 14, 35, 34, 74, 73"
475
476
477 HIGMAN_SIMS_NB_GENS=12
478
479 HIGMAN_SIMS_GO=44352000
480
481
482 # Section 5.9 Group theory, activities for vectors of group elements
483
484
485
486
487

```

```

488
489 # Chapter 6 Ring Theory
490
491 # Section 6.1 polynomials over finite fields
492
493
494
495 CRC32_SPARSE="1,32,1,26,1,23,1,22,1,16,1,12,1,11,\
496 1,10,1,8,1,7,1,5,1,4,1,2,1,1,1,0"
497
498
499 TWO_TO_THE_32_MINUS_2=4294967294
500
501 INVERSE_SPARSE="1,31,1,25,1,22,1,21,1,15,\
502 1,11,1,10,1,9,1,7,1,6,1,4,1,3,1,1,1,0"
503
504
505
506 # Section 6.2 multivariate polynomial rings
507
508 EDGE_CURVE_EQUATION_ALGEBRAIC="X^4 -Y^4 -Z^4 +2*f^2*Y^2*Z^2 +4*f*X^2*Y*Z"
509
510
511
512 QC_Q27_F0159_COEFFS="15,6,12,2,4,23,24,20,15,0,4,3,0,0,23"
513
514 QC_Q27_F0388_COEFFS="1,25,11,12,16,8,13,25,0,14,2,2,19,19,2"
515
516 QC_Q27_F0388_TRANS="19,4,0,0,26,19,10,10,11,1"
517
518
519 # Section 7.1 The Grassmannian
520
521 # Section 7.2 The Klein Quadric and the Pluecker Map
522
523 # Section 7.3 Orthogonal Spaces
524
525
526 ORTHOGONAL_OP_6_2_POINTS_OFF="6,7,10,13,14,15,16,20,24,28,29,30,31,35,39,43, 44,
527 45, 46, 48, 49, 50, 52, 53, 54, 56, 57, 58"
528
529 SIEGEL_TRANSFORM_UV_2A="1,0,0,0,0,0,0,1,1,0,0,0,0,0,1,0,0,0,1,0,0,1,0,0,0,0,0,1
530 ,0,0,0,0,0,0,1"
531
532 SIEGEL_TRANSFORM_UV_2C="1,0,0,0,0,0, 1,1,1,1,0,0, 1,0,1,0,0,0, 1,0,0,1,0,0, 0,0,0
533 ,0,1,0, 0,0,0,0,0,1"
534
535 # Section 7.4 Hermitian Varieties
536
537 # Section 7.5 Algebraic Varieties
538
539 Op_6_eqn="X0*X1+X2*X3+X4*X5"
540
541
542 CURVE_Q13_F0_CLEAN2_EQUATION="X0^4 + 4*X1^4 + 9*X2^4 + X0^2*X1^2 + 4*X0^2*X2^2 +
3*X1^2*X2^2"

```

860

APPENDIX E. THE MAKEFILE

```

543
544
545 HIRSCHFELD_EQUATION_XYZW="X^2*W+X*W^2+Y^2*Z+Y*Z^2"
546
547
548
549 CURVE_Q13_FLAG_1_EQN="10*x1^4 + 12*x2^4 + 9*x3^4 + 8*x1^3*x2 + 2*x1*x2^3 + 5*x2^3
    *x3 + 3*x2*x3^3 + 4*x1^2*x2^2 + 3*x1^2*x3^2 + x2^2*x3^2 + 12*x1^2*x2*x3 + 10*x1*x
    2^2*x3 + 9*x1*x2*x3^2"
550
551 CURVE_Q13_FLAG_1_POINTS="68,71,75,78"
552
553 CURVE_Q13_FLAG_1_BITANGENTS="13,126,121,91,150,105,96,170,18,164,29,0,48,100,81,6
    8,137,69,47,8,162,4,149,31,182,169,19,66"
554
555 CURVE_Q13_FLAG_1_TRANS_1="4,12,0,1,11,5,12,10,6"
556
557 CURVE_Q13_FLAG_1_TRANS_2="1,0,0,12,3,0,9,7,8"
558
559
560
561 QUARTIC_CURVE_Q13_FLAG_0_EQN_COEFFICIENTS="9, 1, 3, 8, 7, 6, 3, 2, 5, 3, 4, 2, 1,
    10, 2"
562
563 QUARTIC_CURVE_Q13_FLAG_0_TRANS="0,6,8,6,8,10,3,12,1"
564
565 # 0 6 8
566 # 6 8 10
567 # 3 12 1
568
569
570 # Section 7.6 Activities for Algebraic Varieties
571
572
573
574 BARTH_EQUATION_GENERAL="4*(PHI^2*X^2-Y^2)*(PHI^2*Y^2-Z^2)*(PHI^2*Z^2-X^2)-(1+2*PH
    I)*(X^2+Y^2+Z^2-W^2)^2*W^2"
575
576
577 QC_Q9_COEFFS="0, 1, 0, 0, 6, 0, 0, 4, 0, 0, 0, 0, 0, 0, 0"
578
579 QC_Q9_ORBITER_CATALOGUE_0_AF="X0^4 + X1^4 + X2^4 + X0^3*X1 + 5*X0^3*X2 + X0*X1^3
    + 7*X1^3*X2 + 7*X0*X2^3 + 5*X1*X2^3"
580
581
582 QC_Q9_HERMITIAN_COEFFS="1,1,1,0,0,0,0,0,0,0,0,0,0,0,0"
583
584 QUARTIC_CURVE_Q9_CLEAN="X2^4+6*X1^3*X3+4*X1*X3^3"
585
586
587 QC_Q9_CLEAN_EQUATION="0,1,0,0,6,0,0,4,0,0,0,0,0,0,0"
588
589 QC_Q9_CLEAN_LINES="85, 48, 73, 78, 16, 55, 44, 87, 74, 76, 57, 13, 90, 51, 18, 27
    , 22, 14, 52, 21, 25, 60, 81, 46, 43, 82, 30, 0"
590
591
592 QC_Q9_TRANSFORM_CLEAN_TO_HERMITIAN="5,8,6,8,1,6,0,3,6,1"
593
594

```



```

595
596
597 QUARTIC_Q9_EQUATION="x1 ^ 4 + 4 * x1 ^ 3 * x2 + 8 * x1 ^ 3 * x3 + 6 * x1 * x2 ^ 3
    + 3 * x1 * x3 ^ 3 + x2 ^ 4 + 6 * x2 ^ 3 * x3 + 4 * x2 * x3 ^ 3 + 2 * x3 ^ 4"
598
599 QUARTIC_Q9_NF_COEFFICIENTS=" 1, 1, 2, 4, 8, 6, 6, 3, 4, 0, 0, 0, 0, 0, 0 "
600
601
602 QC_Q9_UNITAL_COEFFICIENTS="1,1,1,0,0,0,0,0,0,0,0,0,0,0,0"
603
604
605
606 # Section 7.7 Mappings between Projective Spaces
607
608
609
610
611 CREMONA_MAP_Y0="3*y0^5*y2+4*y0^3*y1^2*y2\
612 +2*y0^3*y2^3+y0*y1^4*y2\
613 +6*y0*y1^2*y2^3+9*y0*y2^5"
614
615 CREMONA_MAP_Y1="y0^5*y1+5*y0^3*y1^3\
616 +12*y0^3*y1*y2^2+3*y0*y1^5\
617 +5*y0*y1^3*y2^2+y0*y1*y2^4"
618
619 CREMONA_MAP_Y2="10*y0^6+11*y0^4*y1^2\
620 +11*y0^4*y2^2+4*y0^2*y1^4\
621 +9*y0^2*y1^2*y2^2+4*y0^2*y2^4"
622
623 CREMONA_MAP_Y3="0"
624
625
626
627
628
629
630 # End of Section 7
631
632
633
634
635
636
637
638
639 # Chapter 8 Orbit Algorithms
640
641
642 # Section 8.7
643
644 SURFACE_F9_1_EQN_RESTRICTED="4*X0*X1^2+4*X1^2*X2+8*X0*X2^2+3*X1*X2^2+3*X0*X1*X2"
645
646
647 # Chapter 9 Cubic Surfaces
648
649
650
651 # Section 9.2 Creation
652

```

```

653
654 HIRSCHFELD_SURFACE_EQN_AF="X0^2*X3+X0*X3^2+X1^2*X2+X1*X2^2"
655
656
657 # Dickson 1915:
658
659 Dickson1="1,4,1,7,1,12,1,15"
660
661 Dickson2="1,1,1,2,1,3,1,4,1,7,1,11,1,14,1,15,1,19"
662
663 Dickson3="1,4,1,7,1,12,1,14,1,15,1,19"
664
665 Dickson4="1,3,1,4,1,11,1,9,1,10,1,14,1,17,1,18,1,19"
666
667 Dickson5="1,3,1,4,1,8,1,10,1,17,1,18,1,19"
668
669 Dickson6="1,2,1,3,1,4,1,7,1,15"
670
671 Dickson7="1,2,1,3,1,5,1,9,1,10,1,14,1,15"
672
673 Dickson8="1,3,1,4,1,5,1,7,1,10,1,11,1,19"
674
675 Dickson9="1,3,1,4,1,7,1,11,1,12,1,15,1,19"
676
677 Dickson10="1,4,1,7,1,8,1,9,1,10,1,13,1,15,1,16,1,17"
678
679 Dickson11="1,4,1,7,1,8,1,10,1,13,1,14,1,15,1,16,1,17"
680
681 Dickson12="1,4,1,6,1,7,1,8,1,9,1,10,1,15,1,16,1,17"
682
683 Dickson13="1,6,1,8,1,9,1,10,1,15,1,16,1,17"
684
685 Dickson14="1,6,1,7,1,10,1,11,1,14,1,15"
686
687 Dickson15="1,4,1,5,1,6,1,8,1,12,1,14,1,16"
688
689 Dickson16=" 1,4,1,7,1,3,1,15,1,12"
690
691 Dickson17="1,4,1,7,1,8,1,9,1,11,1,13,1,15,1,16,1,17,1,18,1,19"
692
693 Dickson18="1,4,1,7,1,8,1,11,1,13,1,14,1,15,1,16,1,17,1,18,1,19"
694
695 Dickson19=" 1,4,1,7,1,6,1,8,1,11,1,14,1,12,1,16,1,17,1,18,1,19 "
696
697 Dickson20="1,4,1,5,1,6,1,7,1,8,1,10,1,13,1,14,1,15"
698
699 Dickson21="1,5,1,6,1,8,1,10,1,13,1,14,1,15"
700
701 Dickson22="1,4,1,5,1,7,1,9,1,10,1,11,1,13,1,14,1,15,1,18,1,19"
702
703 Dickson23="1,4,1,7,1,9,1,11,1,13,1,14,1,15,1,18,1,19"
704
705 Dickson24="1,5,1,8,1,9,1,10,1,13,1,14,1,15,1,18,1,19"
706
707 Dickson25=" 1,5,1,10,1,13,1,11,1,9,1,14,1,15,1,18,1,19 "
708
709 Dickson26="1,5,1,6,1,8,1,10,1,15,1,18,1,19"
710
711 Dickson27="1,4,1,5,1,7,1,10,1,11,1,12,1,13,1,14,1,17,1,18"

```

```

712
713 Dickson28="1,4,1,5,1,6,1,7,1,10,1,11,1,14,1,15,1,17,1,18"
714
715 Dickson29="1,4,1,7,1,13,1,11,1,9,1,15,1,17,1,18"
716
717 Dickson30="1,0,1,1,1,2,1,3,1,7,1,10,1,11,1,16"
718
719 Dickson31="1,0,1,1,1,2,1,5,1,7,1,11,1,13,1,14,1,15,1,17,1,18,1,19"
720
721 Dickson32="1,0,1,1,1,2,1,3,1,7,1,8,1,10,1,11,1,12,1,15,1,16,1,19"
722
723 Dickson33="1,0,1,1,1,2,1,3,1,6,1,7,1,10,1,12,1,13,1,15,1,16,1,19"
724
725 Dickson34="1,0,1,1,1,2,1,3,1,7,1,10,1,16,1,19"
726
727 Dickson35="1,0,1,1,1,2,1,3,1,6,1,7,1,8,1,11,1,13,1,15"
728
729 Dickson36="1,0,1,1,1,2,1,3,1,7,1,15"
730
731 Dickson37="1,0,1,1,1,2,1,3,1,7,1,12,1,15,1,19"
732
733 Dickson38="1,0,1,1,1,2,1,3,1,6,1,7,1,12,1,13,1,15,1,19"
734
735 Dickson39="1,0,1,1,1,2,1,3,1,7,1,5,1,10,1,6,1,13,1,8,1,11,1,9,1,14,1,12,1,15,1,19"
736
737 Dickson40="1,0,1,1,1,2,1,3,1,7,1,5,1,10,1,6,1,13,1,19"
738
739 Dickson41="1,2,1,4,1,7,1,5,1,10,1,11,1,14,1,19"
740
741 Dickson42="1,2,1,4,1,7,1,11,1,12,1,14,1,15,1,19"
742
743 Dickson43="1,0,1,1,1,2,1,7,1,13,1,8,1,11,1,14,1,15,1,17"
744
745 Dickson44="1,0,1,1,1,2,1,6,1,8,1,11,1,12,1,14"
746
747 Dickson45="1,6,1,8,1,9,1,11,1,12,1,13,1,14,1,15,1,16,1,17,1,18,1,19"
748
749 Dickson46="1,5,1,10,1,6,1,13,1,8,1,11,1,9,1,14,1,12,1,15,1,16,1,17,1,18,1,19"
750
751 Dickson47="1,5,1,10,1,6,1,13,1,8,1,11,1,9,1,14,1,16,1,17,1,18,1,19"
752
753
754
755
756 Dickson1_eqn_af="X0^2*X1 + X0*X1^2 + X2^2*X3 + X2*X3^2"
757
758 Dickson2_eqn_af="X1^3 + X2^3 + X3^3 + X0^2*X1 + X0*X1^2 + X1*X2^2 + X1*X3^2 + X2*
X3^2 + X1*X2*X3"
759
760 Dickson3_eqn_af="X0^2*X1 + X0*X1^2 + X2^2*X3 + X1*X3^2 + X2*X3^2 + X1*X2*X3"
761
762 Dickson4_eqn_af="X3^3 + X0^2*X1 + X1^2*X3 + X0*X2^2 + X1*X2^2 + X1*X3^2 + X0*X1*X
3 + X0*X2*X3 + X1*X2*X3"
763
764 Dickson5_eqn_af="X3^3 + X0^2*X1 + X1^2*X2 + X0*X2^2 + X0*X1*X3 + X0*X2*X3 + X1*X2
*X3"
765
766 Dickson6_eqn_af="X2^3 + X3^3 + X0^2*X1 + X0*X1^2 + X2*X3^2"

```

```

767
768 Dickson7_eqn_af="X2^3 + X3^3 + X0^2*X2 + X1^2*X3 + X0*X2^2 + X1*X3^2 + X2*X3^2"
769
770 Dickson8_eqn_af="X3^3 + X0^2*X1 + X0^2*X2 + X0*X1^2 + X0*X2^2 + X1*X2^2 + X1*X2*X
3"
771
772 Dickson9_eqn_af="X3^3 + X0^2*X1 + X0*X1^2 + X1*X2^2 + X2^2*X3 + X2*X3^2 + X1*X2*X
3"
773
774 Dickson10_eqn_af="X0^2*X1 + X0*X1^2 + X1^2*X2 + X1^2*X3 + X0*X2^2 + X0*X3^2 + X2*
X3^2 + X0*X1*X2 + X0*X1*X3"
775
776 Dickson11_eqn_af="X0^2*X1 + X0*X1^2 + X1^2*X2 + X0*X2^2 + X0*X3^2 + X1*X3^2 + X2*
X3^2 + X0*X1*X2 + X0*X1*X3"
777
778 Dickson12_eqn_af="X0^2*X1 + X0^2*X3 + X0*X1^2 + X1^2*X2 + X1^2*X3 + X0*X2^2 + X2*
X3^2 + X0*X1*X2 + X0*X1*X3"
779
780 Dickson13_eqn_af="X0^2*X3 + X1^2*X2 + X1^2*X3 + X0*X2^2 + X2*X3^2 + X0*X1*X2 + X0
*X1*X3"
781
782 Dickson14_eqn_af="X0^2*X3 + X0*X1^2 + X0*X2^2 + X1*X2^2 + X1*X3^2 + X2*X3^2"
783
784 Dickson15_eqn_af="X0^2*X1 + X0^2*X2 + X0^2*X3 + X1^2*X2 + X2^2*X3 + X1*X3^2 + X0*
X1*X2"
785
786 Dickson16_eqn_af="X3^3 + X0^2*X1 + X0*X1^2 + X2^2*X3 + X2*X3^2"
787
788 Dickson17_eqn_af="X0^2*X1 + X0*X1^2 + X1^2*X2 + X1^2*X3 + X1*X2^2 + X0*X3^2 + X2*
X3^2 + X0*X1*X2 + X0*X1*X3 + X0*X2*X3 + X1*X2*X3"
789
790 Dickson18_eqn_af="X0^2*X1 + X0*X1^2 + X1^2*X2 + X1*X2^2 + X0*X3^2 + X1*X3^2 + X2*
X3^2 + X0*X1*X2 + X0*X1*X3 + X0*X2*X3 + X1*X2*X3"
791
792 Dickson19_eqn_af="X0^2*X1 + X0^2*X3 + X0*X1^2 + X1^2*X2 + X1*X2^2 + X2^2*X3 + X1*
X3^2 + X0*X1*X2 + X0*X1*X3 + X0*X2*X3 + X1*X2*X3"
793
794 Dickson20_eqn_af="X0^2*X1 + X0^2*X2 + X0^2*X3 + X0*X1^2 + X1^2*X2 + X0*X2^2 + X0*
X3^2 + X1*X3^2 + X2*X3^2"
795
796 Dickson21_eqn_af="X0^2*X2 + X0^2*X3 + X1^2*X2 + X0*X2^2 + X0*X3^2 + X1*X3^2 + X2*
X3^2"
797
798 Dickson22_eqn_af="X0^2*X1 + X0^2*X2 + X0*X1^2 + X1^2*X3 + X0*X2^2 + X1*X2^2 + X0*
X3^2 + X1*X3^2 + X2*X3^2 + X0*X2*X3 + X1*X2*X3"
799
800 Dickson23_eqn_af="X0^2*X1 + X0*X1^2 + X1^2*X3 + X1*X2^2 + X0*X3^2 + X1*X3^2 + X2*
X3^2 + X0*X2*X3 + X1*X2*X3"
801
802 Dickson24_eqn_af="X0^2*X2 + X1^2*X2 + X1^2*X3 + X0*X2^2 + X0*X3^2 + X1*X3^2 + X2*
X3^2 + X0*X2*X3 + X1*X2*X3"
803
804 Dickson25_eqn_af="X0^2*X2 + X1^2*X3 + X0*X2^2 + X1*X2^2 + X0*X3^2 + X1*X3^2 + X2*
X3^2 + X0*X2*X3 + X1*X2*X3"
805
806 Dickson26_eqn_af="X0^2*X2 + X0^2*X3 + X1^2*X2 + X0*X2^2 + X2*X3^2 + X0*X2*X3 + X1
*X2*X3"
807
808 Dickson27_eqn_af="X0^2*X1 + X0^2*X2 + X0*X1^2 + X0*X2^2 + X1*X2^2 + X2^2*X3 + X0*

```

```

X3^2 + X1*X3^2 + X0*X1*X3 + X0*X2*X3"
809
810 Dickson28_eqn_af="X0^2*X1 + X0^2*X2 + X0^2*X3 + X0*X1^2 + X0*X2^2 + X1*X2^2 + X1*
X3^2 + X2*X3^2 + X0*X1*X3 + X0*X2*X3"
811
812 Dickson29_eqn_af="X0^2*X1 + X0*X1^2 + X1^2*X3 + X1*X2^2 + X0*X3^2 + X2*X3^2 + X0*
X1*X3 + X0*X2*X3"
813
814 Dickson30_eqn_af="X0^3 + X1^3 + X2^3 + X3^3 + X0*X1^2 + X0*X2^2 + X1*X2^2 + X0*X1
*X2"
815
816 Dickson31_eqn_af="X0^3 + X1^3 + X2^3 + X0^2*X2 + X0*X1^2 + X1*X2^2 + X0*X3^2 + X1
*X3^2 + X2*X3^2 + X0*X1*X3 + X0*X2*X3 + X1*X2*X3"
817
818 Dickson32_eqn_af="X0^3 + X1^3 + X2^3 + X3^3 + X0*X1^2 + X1^2*X2 + X0*X2^2 + X1*X2
^2 + X2^2*X3 + X2*X3^2 + X0*X1*X2 + X1*X2*X3"
819
820 Dickson33_eqn_af="X0^3 + X1^3 + X2^3 + X3^3 + X0^2*X3 + X0*X1^2 + X0*X2^2 + X2^2*
X3 + X0*X3^2 + X2*X3^2 + X0*X1*X2 + X1*X2*X3"
821
822 Dickson34_eqn_af="X0^3 + X1^3 + X2^3 + X3^3 + X0*X1^2 + X0*X2^2 + X0*X1*X2 + X1*X
2*X3"
823
824 Dickson35_eqn_af="X0^3 + X1^3 + X2^3 + X3^3 + X0^2*X3 + X0*X1^2 + X1^2*X2 + X1*X2
^2 + X0*X3^2 + X2*X3^2"
825
826 Dickson36_eqn_af="X0^3 + X1^3 + X2^3 + X3^3 + X0*X1^2 + X2*X3^2"
827
828 Dickson37_eqn_af="X0^3 + X1^3 + X2^3 + X3^3 + X0*X1^2 + X2^2*X3 + X2*X3^2 + X1*X2
*X3"
829
830 Dickson38_eqn_af="X0^3 + X1^3 + X2^3 + X3^3 + X0^2*X3 + X0*X1^2 + X2^2*X3 + X0*X3
^2 + X2*X3^2 + X1*X2*X3"
831
832 Dickson39_eqn_af="X0^3 + X1^3 + X2^3 + X3^3 + X0^2*X2 + X0^2*X3 + X0*X1^2 + X1^2*
X2 + X1^2*X3 + X0*X2^2 + X1*X2^2 + X2^2*X3 + X0*X3^2 + X1*X3^2 + X2*X3^2 + X1*X2*
X3"
833
834 Dickson40_eqn_af="X0^3 + X1^3 + X2^3 + X3^3 + X0^2*X2 + X0^2*X3 + X0*X1^2 + X0*X2
^2 + X0*X3^2 + X1*X2*X3"
835
836 Dickson41_eqn_af="X2^3 + X0^2*X1 + X0^2*X2 + X0*X1^2 + X0*X2^2 + X1*X2^2 + X1*X3^
2 + X1*X2*X3"
837
838 Dickson42_eqn_af="X2^3 + X0^2*X1 + X0*X1^2 + X1*X2^2 + X2^2*X3 + X1*X3^2 + X2*X3^
2 + X1*X2*X3"
839
840 Dickson43_eqn_af="X0^3 + X1^3 + X2^3 + X0*X1^2 + X1^2*X2 + X1*X2^2 + X0*X3^2 + X1
*X3^2 + X2*X3^2 + X0*X1*X3"
841
842 Dickson44_eqn_af="X0^3 + X1^3 + X2^3 + X0^2*X3 + X1^2*X2 + X1*X2^2 + X2^2*X3 + X1
*X3^2"
843
844 Dickson45_eqn_af="X0^2*X3 + X1^2*X2 + X1^2*X3 + X1*X2^2 + X2^2*X3 + X0*X3^2 + X1*
X3^2 + X2*X3^2 + X0*X1*X2 + X0*X1*X3 + X0*X2*X3 + X1*X2*X3"
845
846 Dickson46_eqn_af="X0^2*X2 + X0^2*X3 + X1^2*X2 + X1^2*X3 + X0*X2^2 + X1*X2^2 + X2^
2*X3 + X0*X3^2 + X1*X3^2 + X2*X3^2 + X0*X1*X2 + X0*X1*X3 + X0*X2*X3 + X1*X2*X3"
847

```

```

848 Dickson47_eqn_af="X0^2*X2 + X0^2*X3 + X1^2*X2 + X1^2*X3 + X0*X2^2 + X1*X2^2 + X0*
X3^2 + X1*X3^2 + X0*X1*X2 + X0*X1*X3 + X0*X2*X3 + X1*X2*X3"
849
850
851
852
853
854 # Section 9.3 Activities
855
856 # points:
857 #0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,23,26,27,30,31,34,35,38,39,42,47,48,51,52,53,
54,59,60,61,62,67,68,69,70,75,76,79,80,81,82
858
859 # points off:
860 #15,16,17,18,19,20,21,22,24,25,28,29,32,33,36,37,40,41,43,44,45,46,49,50,55,56,57
,58,63,64,65,66,71,72,73,74,77,78,83,84
861
862 HIRSCHFELD_SURFACE.POINTS.OFF="15,16,17,18,19,20,21,22,24,\
863 25,28,29,32,33,36,37,40,41,43,44,45,46,49,50,55,56,57,58,\
864 63,64,65,66,71,72,73,74,77,78,83,84"
865
866 #Hesse planes:
867 #7,8,11,13,14,16,17,19,28,29,32,34,35,37,38,40,42,43,44,45,47,48,52,54,56,57,60,6
1,63,64,65,66,68,69,73,75,77,78,81,82
868
869 HIRSCHFELD_SURFACE.HESSE_PLANES="7,8,11,13,14,16,17,19,28,\
870 29,32,34,35,37,38,40,42,43,44,45,47,48,52,54,56,57,60,61,\
871 63,64,65,66,68,69,73,75,77,78,81,82"
872
873
874
875
876 # Section 9.4 Classification of cubic surfaces
877
878 # Section 9.5 Quartic curves
879
880
881 EQUATION_SURFACE_Q29_ISO4_ORB7_F1="11*x1 + 6*x2 + 7*x3"
882
883 EQUATION_SURFACE_Q29_ISO4_ORB7_F2="17*x1^2 + 27*x2^2 + 8*x3^2 + 13*x1*x2 + 25*x1*
x3 + 20*x2*x3"
884
885 EQUATION_SURFACE_Q29_ISO4_ORB7_F3="27*x3^3 + 28*x1^2*x3 + 17*x2^2*x3 + x1*x3^2 +
2*x2*x3^2 + 17*x1*x2*x3"
886
887 #QC_Q9_CLEAN.EQUATION="0,1,0,0,6,0,0,4,0,0,0,0,0,0,0"
888
889 #QC_Q9_CLEAN.LINES="85, 48, 73, 78, 16, 55, 44, 87, 74, 76, 57, 13, 90, 51, 18, 2
7, 22, 14, 52, 21, 25, 60, 81, 46, 43, 82, 30, 0"
890
891
892
893
894 # Section 9.6 Classification of quartic curves
895
896
897
898 NB_CUBIC_SURFACES.Q7=1
899

```

```

900
901 NB_CUBIC_SURFACES.Q13=4
902
903
904 NB_CUBIC_SURFACES.Q19=10
905
906 NB_QUARTIC_CURVES.Q19=14
907
908
909
910 # Section 9.7 Interface to GAP
911
912
913
914 # Chapter 10 Applications
915
916
917 # Section 10.1 Number Theory
918
919 # Section 10.2 Representation Theory
920
921
922 # Section 10.3 Cryptography
923
924
925 NTRU_N=7
926
927 NTRU_P=3
928
929 NTRU_Q=41
930
931 NTRU_D=2
932
933 NTRU_XN1="-1,0,0,0,0,0,0,1,"
934
935 # D + 1 plus ones and D minus ones
936
937 ALICE_PRIVATE_F="-1,0,1,1,-1,0,1"
938
939 # D plus ones and D minus ones
940
941 ALICE_PRIVATE_G="0,-1,-1,0,1,0,1"
942
943 #F_q(x) = 8X^6 + 26X^5 + 31X^4 + 21X^3 + 40X^2 + 2X + 37
944
945 ALICE_PRIVATE_FQ="37,2,40,21,31,26,8"
946
947
948 #F_p(x) = X^6 + 2X^5 + X^3 + X^2 + X + 1
949 ALICE_PRIVATE_FP="1,1,1,1,0,2,1"
950
951 #C(X)=20X^6 + 40X^5 + 2X^4 + 38X^3 + 8X^2 + 26X + 30
952 ALICE_PUBLIC_KEY="30,26,8,38,2,40,20"
953
954 BOB_MESSAGE="1,-1,1,1,0,-1"
955
956 BOB_ONE_TIME_KEY="-1,1,0,0,0,-1,1"
957
958 # E(X) = 31X^6 + 19X^5 + 4X^4 + 2X^3 + 40X^2 + 3X + 25

```

```

959 BOB_ENCRYPT= "25,3,40,2,4,19,31"
960
961 #C(X)=X{6} + 10X{5} + 33X{4} + 40X{3} + 40X{2} + X + 40
962 ALICE_C1="40,1,40,40,33,10,1"
963
964 #A(X)=X{6} + 10X{5} - 8X{4} - X{3} - X{2} + X - 1
965 ALICE_C2="-1,1,-1,-1,-8,10,1"
966
967 #A(X)=X{6} + X{5} + X{4} + 2X{3} + 2X{2} + X + 2
968 ALICE_C3="2,1,2,2,1,1,1"
969
970
971 #C(X)=2X{5} + X{3} + X{2} + 2X + 1
972 ALICE_C4="1,2,1,1,0,2"
973
974
975
976 # Chapter 11 Coding Theory
977
978 # Section 11.1 Introduction
979
980 CODE_5_2_3_CODEWORDS="0,7,25,30"
981
982
983 # Section 11.2 Linear codes
984
985
986 SIMPLEX_CODE_GENERATOR="\
987 1,0,1,0,1,0,1, \
988 0,1,1,0,0,1,1, \
989 0,0,0,1,1,1,1"
990
991
992
993 CODE_RM_3_1_GENMA="\
994 11111111\
995 01010101\
996 00110011\
997 00001111"
998
999 CODE_RM_4_1_GENMA="\
1000 1111111111111111\
1001 01010101010101\
1002 0011001100110011\
1003 0000111100001111\
1004 0000000011111111"
1005
1006
1007 HAMMING_CODE_ROWS_IN_BINARY_RANKS="67,37,22,15"
1008
1009 HAMMING_CODE_CODEWORDS="0, 67, 37, 102, 22, 85, \
1010 51, 112, 15, 76, 42, 105, 25, 90, 60, 127"
1011
1012
1013 HAMMING_CODE_GENERATOR="\
1014 1,0,0,0,0,1,1, \
1015 0,1,0,0,1,0,1, \
1016 0,0,1,0,1,1,0, \
1017 0,0,0,1,1,1,1"

```


1090

```

1091
1092 M24_2B_PERM="2, 5, 0, 8, 11, 1, 16, 14, 3, 19, 21, 4, \
1093 22, 23, 7, 20, 6, 18, 17, 9, 15, 10, 12, 13"
1094
1095 GOLAY24_2A_FIX_SUBCODE="1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, \
1096 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, \
1097 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 0, \
1098 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, \
1099 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, \
1100 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, \
1101 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, \
1102 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 0, \
1103 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, \
1104 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, \
1105 1, 1, 0, 0, 0, 1, 1, 0, 0, 1"
1106
1107 GOLAY24_2B_FIX_SUBCODE="1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, \
1108 0, 0, 0, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, \
1109 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 0, \
1110 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 1, \
1111 0, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, \
1112 1, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, \
1113 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, \
1114 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1"
1115
1116
1117
1118
1119 # Section 11.4 CRC codes
1120
1121
1122 CRC4="1,4,1,2,1,1,1,0"
1123
1124 CRC7="1,7,1,3,1,0"
1125
1126 CRC8_ATM="1,8,1,2,1,1,1,0"
1127
1128 CRC16_CCITT="1,16,1,12,1,5,1,0"
1129
1130 CRC32_ETHERNET="1,32,1,26,1,23,1,22,1,16,1,12,1,11,1,10,1,8,1,7,\
1131 1,5,1,4,1,2,1,1,1,0"
1132
1133 CRC32_ETHERNET_POLY="X^32+X^26+X^23+X^22+X^16+\
1134 X^12+X^11+X^10+X^8+X^7+X^5+X^4+X^2+X^1+1"
1135
1136 CRC32_CASTAGNOLI="1,32,1,28,1,27,1,26,1,25,1,23,1,22,1,20,1,19,1,\
1137 18,1,14,1,13,1,11,1,10,1,9,1,8,1,6,1,0"
1138
1139 CRC64_ECMA182="1,64,1,62,1,57,1,55,1,54,1,53,1,52,1,47,1,46,1,45,\
1140 1,40,1,39,1,38,1,37,1,35,1,33,1,32,1,31,1,29,1,27,1,24,1,23,1,22,\
1141 1,21,1,19,1,17,1,13,1,12,1,10,1,9,1,7,1,4,1,1,1,0"
1142
1143 CRC64_ROCKSOFT="1,64,1,63,1,61,1,59,1,58,1,56,1,55,1,52,1,49,1,48,\
1144 1,47,1,46,1,44,1,41,1,37,1,36,1,34,1,32,1,31,1,28,1,26,1,23,1,22,1,\
1145 19,1,16,1,13,1,12,1,10,1,9,1,6,1,4,1,3,1,0"
1146
1147
1148
1149 CRC_POLY_Q256_DEG2_DENSE="214,167,1"

```

```

1150
1151
1152
1153 CRC_POLY_BRAVO_DENSE="1,23,27,213,1"
1154
1155
1156 CRC_POLY_CHARLIE_DENSE="1,126,25,1,196,209,244,3,121,126,35,65,1"
1157
1158
1159 POLY_Q256_DEG30_SPARSE="1,0,26,1,210,2,24,3,\
1160 138,4,148,5,160,6,58,7,108,8,199,9,95,10,56,\
1161 11,9,12,205,13,194,14,193,15,3,16,248,17,110,\
1162 18,150,19,24,20,169,21,192,22,212,23,112,24,\
1163 144,25,97,26,109,27,174,28,253,29,1,30"
1164
1165 POLY_Q256_DEG30_DENSE="1,26,210,24,138,148,\
1166 160,58,108,199,95,56,9,205,194,193,3,248,110,\
1167 150,24,169,192,212,112,144,97,109,174,253,1"
1168
1169 CRC_FILE=allen.Gates
1170
1171 CRC_FILE_EXTENSION=bmp
1172
1173 TEN_MILLION=10000000
1174
1175
1176 CRC_BCH_Q16_N51_D3_PROJECTIVE_SET="1900, 2703, 2347, 1653, 232, 3681, 752, 511, 3
    916, 1222, 632, 2014, 3951, 4006, 2198, 3258, 2804, 3771, 3120, 2132, 3754, 2640,
    1995, 943, 3319, 1748, 36, 546, 1198, 4296, 4013, 2118, 2234, 3034, 2441, 4085,
    4326, 318, 1180, 2488, 4229, 765, 1470, 2306, 666, 3454, 1763, 0, 1, 2, 3 "
1177
1178 CRC_POLY_DELTA_DENSE="1, 2, 13, 5, 1"
1179 #create_BCH_code::init Q=X^{4} + 5X^{3} + 13X^{2} + 2X + 1
1180 #create_BCH_code::init BCH_POLYNOMIAL_F16="X^4+5*X^3+13*X^2+2*X+1"
1181
1182 ERROR_POLY_BRAVO_COEFF="737, 417, 171, 634, 158, 30, 480, 688, 94, 739, 147, 694,
    724, 741, 51, 411, 753, 622, 592, 667, 25, 709, 55, 450, 255, 744, 28, 718, 82,
    740, 372, 144, 435, 505, 478, 233, 325, 231, 27, 518, 492, 350, 621, 468, 297, 14
    1, 735, 338, 227, 434, 54, 180, 390, 245, 301, 617, 423, 408, 521, 42, 697, 719,
    668, 618, 107, 746, 441, 376, 705, 353, 368, 254, 393, 291, 762, 188, 506, 580, 4
    19, 525, 500, 652, 433, 130, 52, 212, 511, 191, 182, 100, 676, 706, 248, 268, 365
    , 626, 303, 5, 490, 524, 237, 210, 326, 619, 485, 767, 432, 721, 391, 650, 88, 62
    7, 177, 700, 354, 682, 38, 360, 70, 636, 609, 766, 203, 7, 369, 584, 475, 84, 87,
    251, 680, 479, 256, 121, 473, 604, 614, 265, 585, 40, 458, 551, 163, 661, 358, 6
    10, 349, 748, 371, 310, 31, 136, 153, 649, 662, 547, 120, 324, 162, 671, 62, 725,
    603, 645, 670, 554, 364, 127, 312, 708, 386, 286, 532, 58, 658, 366, 74, 442, 69
    9, 707, 164, 102, 593, 657, 293, 638, 370, 716, 454, 126, 204, 232, 637, 409, 491
    , 666, 402, 209, 639, 560, 578, 467, 24, 629, 93, 453, 352, 425, 406, 37, 684, 18
    9, 648, 186, 345, 691, 673, 193, 264, 247, 714, 267, 540, 215, 134, 729, 588, 295
    , 294, 241, 577, 398, 722, 46, 214, 60, 640, 770, 477, 106, 97, 381, 280, 675, 41
    8, 29, 385, 269, 151, 665, 109, 514, 305, 377, 95, 544, 570, 552, 128, 152, 599,
    397, 415, 149, 451, 738, 743, 302, 140, 273, 103, 284, 396, 726, 327, 221, 288, 2
    00, 344, 285, 112, 542, 22, 313, 263, 218, 61, 157, 362, 559, 507, 243, 664, 206,
    550, 242, 244, 230, 145, 761, 413, 679, 392, 447, 17, 330, 156, 279, 752, 487, 7
    31, 651, 388, 535, 154, 502, 108, 452, 363, 240, 569, 564, 713, 704, 495, 589, 83
    , 15, 483, 261, 537, 595, 219, 573, 516, 252, 199, 337, 616, 339, 486, 555, 653,
    728, 137, 216, 56, 211, 734, 449, 44, 26, 348, 426, 91, 139, 238, 539, 444, 754,
    672, 695, 543, 659, 601, 654, 742, 300, 69, 115, 756, 122, 582, 374, 236, 656, 53
    3, 755, 512, 594, 404, 65, 611, 316, 703, 361, 318, 80, 92, 12, 720, 378, 763, 22

```

8, 429, 623, 608, 712, 33, 612, 185, 747, 11, 620, 78, 234, 202, 165, 498, 173, 1
60, 494, 768, 75, 306, 482, 641, 53, 222, 379, 460, 258, 567, 50, 531, 548, 207,
424, 320, 383, 124, 757, 249, 10, 405, 575, 597, 587, 341, 328, 336, 513, 317, 17
2, 681, 169, 187, 430, 351, 523, 81, 553, 389, 496, 689, 461, 607, 359, 436, 105,
229, 8, 101, 176, 663, 43, 701, 356, 630, 562, 678, 272, 224, 387, 271, 311, 456
, 530, 401, 278, 239, 520, 529, 287, 343, 59, 367, 342, 21, 503, 420, 466, 322, 1
46, 504, 563, 196, 545, 558, 161, 394, 257, 49, 99, 598, 6, 77, 143, 68, 118, 133
, 690, 410, 235, 536, 48, 205, 270, 445, 283, 198, 660, 90, 47, 57, 727, 41, 463,
76, 581, 576, 192, 250, 686, 509, 34, 175, 246, 400, 168, 111, 334, 403, 517, 67
7, 373, 497, 431, 470, 501, 174, 225"

1183

1184 ERROR_POLY_BRAVO_PLACE="100, 202, 178, 171, 102, 243, 43, 238, 132, 200, 126, 107
, 61, 252, 150, 5, 18, 51, 161, 248, 230, 166, 207, 48, 138, 132, 221, 248, 228,
42, 126, 117, 253, 85, 133, 165, 56, 125, 224, 92, 223, 129, 108, 179, 197, 224,
163, 241, 196, 207, 53, 73, 183, 98, 172, 7, 65, 90, 35, 15, 59, 219, 95, 41, 93,
52, 51, 19, 205, 78, 66, 6, 218, 47, 109, 228, 45, 134, 141, 173, 225, 93, 194,
5, 164, 100, 199, 10, 118, 30, 163, 110, 152, 247, 212, 214, 146, 104, 164, 7, 11
7, 87, 129, 159, 235, 129, 233, 224, 72, 250, 167, 128, 208, 220, 150, 195, 40, 1
68, 73, 21, 98, 185, 128, 27, 30, 74, 251, 134, 186, 40, 109, 138, 181, 202, 137,
86, 72, 230, 117, 18, 123, 180, 10, 23, 122, 47, 241, 16, 193, 71, 189, 203, 212
, 68, 241, 143, 191, 62, 228, 255, 82, 12, 178, 134, 161, 178, 251, 119, 66, 209,
33, 32, 17, 124, 47, 96, 191, 20, 10, 4, 82, 36, 65, 36, 164, 113, 230, 30, 33,
176, 75, 54, 130, 25, 171, 227, 1, 41, 182, 164, 87, 232, 210, 74, 161, 84, 24, 1
87, 106, 106, 52, 153, 247, 152, 252, 245, 182, 253, 213, 203, 45, 168, 193, 190,
42, 105, 201, 1, 211, 224, 153, 189, 89, 187, 59, 238, 53, 95, 105, 103, 24, 2,
62, 250, 57, 70, 68, 110, 45, 45, 226, 254, 194, 103, 186, 140, 209, 1, 37, 87, 1
5, 200, 26, 22, 216, 251, 175, 137, 33, 228, 132, 152, 86, 179, 106, 68, 69, 49,
202, 146, 67, 168, 130, 223, 55, 234, 177, 88, 146, 195, 100, 176, 210, 177, 61,
161, 1, 58, 5, 191, 30, 49, 183, 83, 125, 13, 170, 211, 93, 110, 219, 93, 246, 12
0, 169, 97, 206, 86, 241, 208, 195, 199, 53, 31, 133, 253, 181, 247, 192, 209, 19
, 159, 175, 39, 247, 154, 12, 225, 43, 231, 198, 83, 125, 33, 175, 79, 242, 224,
150, 251, 213, 125, 155, 230, 61, 109, 32, 239, 227, 40, 8, 133, 186, 118, 232, 1
57, 218, 41, 24, 168, 32, 205, 172, 216, 36, 94, 179, 37, 212, 13, 35, 171, 16, 1
71, 39, 69, 241, 165, 62, 91, 136, 4, 227, 168, 53, 6, 226, 251, 58, 251, 38, 234
, 245, 255, 168, 119, 169, 47, 179, 223, 28, 32, 17, 185, 49, 44, 196, 122, 89, 1
16, 157, 223, 252, 204, 40, 119, 89, 8, 7, 178, 235, 52, 12, 51, 217, 60, 104, 10
0, 59, 111, 14, 244, 49, 223, 225, 245, 187, 61, 242, 174, 29, 145, 172, 177, 131
, 188, 157, 76, 104, 244, 94, 39, 96, 246, 131, 77, 255, 70, 211, 201, 166, 105,
68, 129, 4, 80, 20, 89, 58, 222, 159, 208, 82, 255, 245, 121, 210, 149, 202, 105,
212, 42, 35, 188, 3, 121, 23, 139, 50, 24, 224, 20, 140, 119, 55, 118, 53, 155,
19, 70, 138, 178, 202, 153, 78, 44, 90, 109, 32, 253, 64, 130, 238, 243, 87, 12,
204, 63, 116, 163, 82, 25, 14, 31, 197, 193, 203, 235, 79, 86, 118, 118, 88, 190,
26, 55, 179, 156, 153, 89, 227, 221"

1185

1186 CRC_ERROR_POLY_BRAVO="104*X^5+118*X^6+27*X^7+39*X^8+51*X^10+245*X^11+91*X^12+247*
X^15+125*X^17+202*X^21+130*X^22+210*X^24+230*X^25+125*X^26+224*X^27+221*X^28+70*X
^29+243*X^30+189*X^31+58*X^33+193*X^34+106*X^37+40*X^38+18*X^40+204*X^41+15*X^42+
77*X^43+213*X^44+187*X^46+243*X^47+44*X^48+140*X^49+204*X^50+150*X^51+164*X^52+12
2*X^53+53*X^54+207*X^55+242*X^56+87*X^57+124*X^58+121*X^59+238*X^60+177*X^61+82*X
^62+171*X^65+19*X^68+24*X^69+73*X^70+191*X^74+185*X^75+116*X^76+53*X^77+168*X^78+
165*X^80+29*X^81+228*X^82+181*X^83+134*X^84+186*X^87+167*X^88+238*X^90+61*X^91+62
*X^92+161*X^93+132*X^94+186*X^95+24*X^97+119*X^99+30*X^100+96*X^101+36*X^102+132*
X^103+244*X^105+103*X^106+93*X^107+206*X^108+226*X^109+118*X^111+67*X^112+168*X^1
15+70*X^118+191*X^120+202*X^121+205*X^122+235*X^124+176*X^126+119*X^127+37*X^128+
5*X^130+138*X^133+42*X^134+203*X^136+175*X^137+109*X^139+33*X^140+224*X^141+155*X
^143+117*X^144+5*X^145+188*X^146+126*X^147+22*X^149+45*X^151+87*X^152+212*X^153+1
69*X^154+170*X^156+88*X^157+102*X^158+28*X^160+24*X^161+228*X^162+10*X^163+82*X^1
64+47*X^165+86*X^168+245*X^169+178*X^171+223*X^172+223*X^173+227*X^174+203*X^175+
246*X^176+208*X^177+73*X^180+118*X^182+38*X^185+152*X^186+187*X^187+228*X^188+153

```

*189*10*191+25*192+253*193+23*196+64*198+12*199+49*200+169*202+
128*203+75*204+90*205+177*206+89*207+41*209+87*210+224*211+100*212+
59*214+190*215+79*216+234*218+175*219+68*221+89*222+68*224+22
1*225+196*227+168*228+94*229+58*230+125*231+54*232+165*233+119*234+
153*235+36*236+117*237+32*238+159*239+208*240+224*241+161*242+
176*243+1*244+98*245+235*246+203*247+152*248+12*249+14*250+40*251+
154*252+6*254+138*255+181*256+20*257+223*258+209*261+55*263+2
13*264+230*265+168*267+247*268+110*269+109*270+4*271+105*272+228*
273+222*278+211*279+62*280+253*283+152*284+146*285+32*286+255*287+
69*288+47*291+164*293+211*294+1*295+197*297+41*300+172*301+137
*302+146*303+194*305+49*306+71*310+80*311+66*312+223*313+171*316+
49*317+241*318+7*320+35*322+62*324+56*325+129*326+106*327+111*328+
13*330+118*334+14*336+225*337+241*338+231*339+59*341+149*342+
245*343+202*344+252*345+155*348+241*349+129*350+242*351+24*352+7
8*353+150*354+70*356+122*358+76*359+168*360+69*361+146*362+241*363+
251*364+212*365+96*366+210*367+66*368+30*369+230*370+193*371+
126*372+55*373+216*374+19*376+103*377+4*378+116*379+2*381+178*383+
68*385+33*386+129*387+246*388+172*389+183*390+72*391+183*392+2
18*393+224*394+86*396+200*397+189*398+79*400+58*401+1*402+88*403+
35*404+217*405+106*406+90*408+25*409+202*410+5*411+30*413+26*415+
202*417+57*418+141*419+212*420+65*423+8*424+187*425+230*426+5
3*429+61*430+156*431+233*432+194*433+207*434+253*435+104*436+51*441+
20*442+227*444+32*445+83*447+251*449+48*450+216*451+86*452+8
4*453+33*454+20*456+123*458+157*460+188*461+63*463+42*466+232*467+
179*468+153*470+137*473+251*475+105*477+133*478+138*479+43*480+
44*482+192*483+235*485+198*486+110*487+164*490+171*491+223*492+3
2*494+133*495+177*496+179*497+179*498+225*500+89*501+97*502+105*503+
3*504+85*505+45*506+100*507+197*509+199*511+212*512+244*513+
254*514+247*516+190*517+92*518+208*520+35*521+174*523+7*524+173*525+
82*529+89*530+40*531+17*532+179*533+120*535+78*536+19*537+23
9*539+193*540+168*542+186*543+140*544+139*545+143*547+119*548+61*
550+180*551+1*552+145*553+178*554+83*555+50*558+195*559+164*560+
201*562+121*563+199*564+252*567+195*569+209*570+39*573+60*575+82
*576+153*577+87*578+134*580+163*581+172*582+74*584+117*585+100*587+
201*588+253*589+161*592+65*593+13*594+159*595+104*597+55*598+
15*599+232*601+178*603+86*604+157*607+226*608+98*609+47*610+16*611+
251*612+72*614+43*616+7*617+41*618+159*619+255*620+108*621+51
*622+6*623+214*626+128*627+74*629+211*630+171*634+21*636+130*637+
113*638+182*639+53*640+196*641+134*645+247*648+68*649+250*650+9
3*651+93*652+125*653+157*654+94*656+36*657+47*658+118*659+130*660+
23*661+241*662+131*663+210*664+45*665+227*666+248*667+95*668+1
61*670+255*671+8*672+182*673+250*675+163*676+26*677+166*678+49*679+
109*680+225*681+195*682+52*684+31*686+238*688+131*689+178*690+
245*691+107*694+133*695+59*697+10*699+220*700+255*701+39*703+31*
704+205*705+110*706+4*707+209*708+166*709+251*712+53*713+45*714+
30*716+248*718+219*719+136*720+224*721+89*722+61*724+12*725+179*
726+12*727+33*728+105*729+219*731+150*734+163*735+100*737+251*738+
200*739+42*740+252*741+218*742+175*743+132*744+52*746+234*747+
16*748+93*752+18*753+40*754+37*755+32*756+52*757+191*761+109*762+
227*763+185*766+129*767+17*768+95*770"

```

1187

1188

1189

1190 # Section 11.5 Reed-Muller codes

1191

1192

1193

1194

1195 RM_6_GENERATOR_1="0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,\

```

1196 22,23,24,25,26,27,28,29,30,31,32,33,34,35,36,37,38,39,40,41,42,43,44,45,\
1197 46,47,48,49,50,51,52,53,54,55,56,57,58,59,60,61,62,63"
1198
1199 RM_6_GENERATOR_2="1,3,5,7,9,11,13,15,17,19,21,23,25,27,29,31,33,35,37,39,\
1200 41,43,45,47,49,51,53,55,57,59,61,63"
1201
1202 RM_6_GENERATOR_3="2,3,6,7,18,19,22,23,10,11,14,15,26,27,30,31,34,35,38,\
1203 39,42,43,46,47,50,51,54,55,58,59,62,63"
1204
1205 RM_6_GENERATOR_4="4,6,12,14,36,38,52,54,5,7,13,15,37,39,53,55,20,22,28,\
1206 30,44,46,60,62,21,23,29,31,45,47,61,63"
1207
1208 RM_6_GENERATOR_5="8,9,12,13,24,25,28,29,10,11,14,15,26,27,30,31,40,41,\
1209 44,45,56,57,60,61,42,43,46,47,58,59,62,63"
1210
1211 RM_6_GENERATOR_6="16,18,24,26,48,50,56,58,17,19,25,27,49,51,57,59,20,22,\
1212 28,30,52,54,60,62,21,23,29,31,53,55,61,63"
1213
1214 RM_6_GENERATOR_7="32,34,48,50,33,35,49,51,36,38,52,54,37,39,53,55,40,42,\
1215 56,58,41,43,57,59,44,46,60,62,45,47,61,63"
1216
1217
1218
1219
1220 # Section 11.6 BCH-codes
1221
1222
1223 #CODE_BCH_F8_N21_D5_GENMA
1224
1225 CODE_BCH_F8_N21_D5_GENMA_OVERRIDE_POLYNOMIAL11="\
1226 2,1,2,1,2,3,3,1,0,0,0,0,0,0,0,0,0,0,0,0,\
1227 0,2,1,2,1,2,3,3,1,0,0,0,0,0,0,0,0,0,0,0,\
1228 0,0,2,1,2,1,2,3,3,1,0,0,0,0,0,0,0,0,0,0,\
1229 0,0,0,2,1,2,1,2,3,3,1,0,0,0,0,0,0,0,0,0,\
1230 0,0,0,0,2,1,2,1,2,3,3,1,0,0,0,0,0,0,0,0,\
1231 0,0,0,0,0,2,1,2,1,2,3,3,1,0,0,0,0,0,0,0,\
1232 0,0,0,0,0,0,2,1,2,1,2,3,3,1,0,0,0,0,0,0,\
1233 0,0,0,0,0,0,0,2,1,2,1,2,3,3,1,0,0,0,0,0,\
1234 0,0,0,0,0,0,0,0,2,1,2,1,2,3,3,1,0,0,0,0,\
1235 0,0,0,0,0,0,0,0,0,2,1,2,1,2,3,3,1,0,0,0,\
1236 0,0,0,0,0,0,0,0,0,0,2,1,2,1,2,3,3,1,0,0,\
1237 0,0,0,0,0,0,0,0,0,0,0,2,1,2,1,2,3,3,1,0,\
1238 0,0,0,0,0,0,0,0,0,0,0,0,2,1,2,1,2,3,3,1,\
1239 0,0,0,0,0,0,0,0,0,0,0,0,0,2,1,2,1,2,3,3,1"
1240
1241 CODE_BCH_F8_N21_K14_D5_GENMA="\
1242 6,1,5,5,4,4,3,1,0,0,0,0,0,0,0,0,0,0,0,0,\
1243 0,6,1,5,5,4,4,3,1,0,0,0,0,0,0,0,0,0,0,0,\
1244 0,0,6,1,5,5,4,4,3,1,0,0,0,0,0,0,0,0,0,0,\
1245 0,0,0,6,1,5,5,4,4,3,1,0,0,0,0,0,0,0,0,0,\
1246 0,0,0,0,6,1,5,5,4,4,3,1,0,0,0,0,0,0,0,0,\
1247 0,0,0,0,0,6,1,5,5,4,4,3,1,0,0,0,0,0,0,0,\
1248 0,0,0,0,0,0,6,1,5,5,4,4,3,1,0,0,0,0,0,0,\
1249 0,0,0,0,0,0,0,6,1,5,5,4,4,3,1,0,0,0,0,0,\
1250 0,0,0,0,0,0,0,0,6,1,5,5,4,4,3,1,0,0,0,0,\
1251 0,0,0,0,0,0,0,0,0,6,1,5,5,4,4,3,1,0,0,0,\
1252 0,0,0,0,0,0,0,0,0,0,6,1,5,5,4,4,3,1,0,0,\
1253 0,0,0,0,0,0,0,0,0,0,0,6,1,5,5,4,4,3,1,0,\
1254 0,0,0,0,0,0,0,0,0,0,0,0,6,1,5,5,4,4,3,1,0,\

```

[illegible]


```
1288 010001100000000000000000\
1289 001110010000000000000000\
1290 110011001000000000000000\
1291 000010001100000000000000\
1292 000001110010000000000000\
1293 000110011001000000000000\
1294 000000010001100000000000\
1295 000000001110010000000000\
1296 000000110011001000000000\
1297 000000000100011000000000\
1298 000000000001110010000000\
1299 000000000110011001000000\
1300 000000000000010001100000\
1301 000000000000001110010000\
1302 000000000000110011001"
1303
1304
1305 CODE_21_15_4="\
1306 111000100000000000000000 \
1307 110100010000000000000000 \
1308 101100001000000000000000 \
1309 011100000100000000000000 \
1310 110010000010000000000000 \
1311 101010000001000000000000 \
1312 011010000000100000000000 \
1313 100110000000010000000000 \
1314 010110000000001000000000 \
1315 001110000000000100000000 \
1316 111110000000000010000000 \
1317 110001000000000001000000 \
1318 101001000000000000100000 \
1319 011001000000000000010000 \
1320 100101000000000000001"
1321
1322
1323 # there are 5 [15,6,6]
1324
1325 # ago=12
1326 CODE_15_6_6_A="\
1327 111111111100000 \
1328 111110000010000 \
1329 111001100001000 \
1330 110101010000100 \
1331 101010110000010 \
1332 101101001000001"
1333
1334 # ago=12
1335 CODE_15_6_6_B="\
1336 111111111100000 \
1337 111110000010000 \
1338 111001100001000 \
1339 110101010000100 \
1340 101010110000010 \
1341 011011001000001"
1342
1343 #ago=720:
1344 CODE_15_6_6_C="\
1345 111111111100000 \
1346 111110000010000 \
```

```

1347 111001100001000 \
1348 110101010000100 \
1349 101101001000010 \
1350 100010111000001"
1351
1352 #ago=96:
1353 CODE_15_6_D="\
1354 111111111100000 \
1355 111110000010000 \
1356 111001100001000 \
1357 110101010000100 \
1358 101011001000010 \
1359 011001011000001"
1360
1361 #ago=360
1362 CODE_15_6_E="\
1363 111111111100000 \
1364 111110000010000 \
1365 111001100001000 \
1366 100111010000100 \
1367 010101110000010 \
1368 010110101000001"
1369
1370
1371
1372 # Section 11.9 Bounds
1373
1374
1375 CODE_GV_N15_K6="\
1376 111111111100000\
1377 111110000010000\
1378 111001100001000\
1379 110101010000100\
1380 101010110000010\
1381 101101001000001"
1382
1383
1384 CODE_GV_N15_K6_CHECK="\
1385 100000000111111\
1386 010000000111100\
1387 001000000111011\
1388 000100000110101\
1389 000010000110010\
1390 000001000101101\
1391 000000100101010\
1392 000000010100110\
1393 000000001100001"
1394
1395
1396
1397 # Chapter 12 Combinatorics
1398
1399
1400
1401 # Section 12.2 Combinatorial Objects
1402
1403 HIRSCHFELD_SURFACE_Q4_SET_OF_POINTS="0,1,2,3,4,5,6,7,8,9,\
1404 10,11,12,13,14,23,26,27,30,31,34,35,38,39,42,47,48,51,52,\
1405 53,54,59,60,61,62,67,68,69,70,75,76,79,80,81,82"

```

```

1406
1407
1408 HYPEROVAL_16_144="0, 1, 2, 3, 52, 67, 89, 106, 126, \
1409 141, 159, 176, 184, 199, 220, 235, 245, 262"
1410
1411 HYPEROVAL_16_16320="0, 1, 2, 3, 52, 70, 83, 109, 127, \
1412 139, 156, 174, 186, 199, 217, 229, 256, 264"
1413
1414
1415 NEUMAIER_GRAPH_16_1_ADJ="1,1,1,1,1,0,0,1,1,0,0,1,1,0,0,1,1,1,1,0,0,0,0,1,1,0,0,1,
1416 1,1,0,0,1,1,1,1,0,0,0,0,1,1,0,0,1,1,0,0,1,1,1,0,0,1,0,1,0,1,0,1,1,0,1,
1417 0,1,0,1,0,1,1,1,0,0,1,0,0,1,0,1,0,1,1,0,1,1,1,0,0,1,1,1,0,0,1,1,0,1,1,
1418 0,1,0,0,1,1,1,1,1,1,1"
1419
1420
1421 # Section 12.5 Classification of Configurations and Geometries
1422
1423
1424
1425 GEO_BLOCKS_600="Row,Block\n\
1426 0,\"0, 5, 14\", \n\
1427 1,\"1, 6, 10\", \n\
1428 2,\"2, 7, 11\", \n\
1429 3,\"3, 8, 12\", \n\
1430 4,\"4, 9, 13\", \n\
1431 5,\"0, 6, 13\", \n\
1432 6,\"1, 7, 14\", \n\
1433 7,\"2, 8, 10\", \n\
1434 8,\"3, 9, 11\", \n\
1435 9,\"4, 5, 12\", \n\
1436 10,\"0, 7, 12\", \n\
1437 11,\"1, 8, 13\", \n\
1438 12,\"2, 9, 14\", \n\
1439 13,\"3, 5, 10\", \n\
1440 14,\"4, 6, 11\", \n\
1441 15,\"0, 8, 11\", \n\
1442 16,\"1, 9, 12\", \n\
1443 17,\"2, 5, 13\", \n\
1444 18,\"3, 6, 14\", \n\
1445 19,\"4, 7, 10\", \n\
1446 20,\"0, 9, 10\", \n\
1447 21,\"1, 5, 11\", \n\
1448 22,\"2, 6, 12\", \n\
1449 23,\"3, 7, 13\", \n\
1450 24,\"4, 8, 14\", \n\
1451 END\n"
1452
1453
1454 LSQ_5A_TABLE="0,1,2,3,4,1,2,3,4,0,2,3,4,0,1,3,4,0,1,2,4,0,1,2,3"
1455
1456 LSQ_5B_TABLE="0,1,2,3,4,1,0,4,2,3,2,4,3,1,0,3,2,0,4,1,4,3,1,0,2"
1457
1458

```

```

1459
1460
1461 # Chapter 13 Graph theory
1462
1463 # Section 13.1 Creating graphs
1464
1465 TRIANGLE_GRAPH="0,1,1\n1,0,1\n1,1,0\n"
1466
1467
1468
1469
1470
1471
1472 # Chapter 14 Design theory
1473
1474
1475
1476
1477 # Section 14.2 Assumed symmetry
1478
1479
1480 GEN_C5="1,2,3,4,0,6,7,8,9,5,11,12,13,14,10"
1481
1482
1483
1484 # Section 14.3 Design Theory - Large sets
1485
1486
1487
1488
1489 AG_2_3_BLOCKS="0,13,22,27,35,41,47,53,55,59,71,76"
1490
1491 LARGE_SET_AG_2_3_NEIGHBOR_SET="129,130,133,134,136,\
1492 139,141,142,153,154,156,160,165,166,178,179,183,184,\
1493 185,190,192,194,197,203,204,206,218,221,222,225,227,\
1494 231,248,251,252,255,256,259,261,262,272,277,279,283,\
1495 285,287,299,301,303,305,306,309,313,315,319,320,323,\
1496 325,341,342,343,344,345,349,368,371,375,378,381,383,\
1497 392,393,397,402,403,405,416,419,421,422,425,426,429,\
1498 430,440,443,447,449,453,454,464,467,468,473,474,479,\
1499 490,493,494,497,500,503,513,517,518,520,523,527,536,\
1500 539,541,542,544,547,548,551,563,566,567,571,572,573,\
1501 585,589,590,593,595,596,600,601,603,611,614,615,625,\
1502 629,631,635,637,638,657,659,661,667,668,671,681,683,\
1503 686,689,691,693,705,706,709,710,712,715,717,718,720,\
1504 723,724,729,733,735,747,748,750,752,754,757,777,780,\
1505 781,784,790,791,802,804,807,808,811,814,824,827,828,\
1506 831,832,835,837,838"
1507
1508
1509
1510 # Section 14.4: Design Theory - Delandtsheer Doyen
1511
1512
1513
1514 PP4=-d1 1 -q1 3 -d2 1 -q2 7 -K 5 -problem_label PP4
1515
1516 PP4.GROUP1=-subgroup "1,1,1,1, " "21" -group_label "cyclic21"
1517

```

```

1518 PP4_MASK1=\
1519 ▷ -nb_orbits_on_blocks 1 \
1520 ▷ -mask_label "no_mask"
1521
1522
1523
1524 DELANDTSHEER_DOYEN_PROBLEM_COLBOURN_COLBOURN_7_13= -d1 1 -q1 7 -d2 1 -q2 13 -K 6
    -search_control -W -end -problem_label DD_CC_7_13
1525
1526 DELANDTSHEER_DOYEN_PROBLEM_COLBOURN_COLBOURN_7_13.GROUP1=-subgroup "1,1,1,1, " "9
    1" -group_label "cyclic91"
1527
1528 DELANDTSHEER_DOYEN_PROBLEM_COLBOURN_COLBOURN_7_13_MASK1=\
1529 ▷ -nb_orbits_on_blocks 3 \
1530 ▷ -depth 6 \
1531 ▷ -mask_label "no_mask"
1532
1533
1534
1535 DELANDTSHEER_DOYEN_PROBLEM_27_53= -d1 1 -q1 27 -d2 1 -q2 53 -K 11 -DDx 2 -DDy 1 -
    search_control -W -end
1536
1537 DELANDTSHEER_DOYEN_PROBLEM_27_53.GROUP1=-subgroup \
1538 ▷ "1,1,1,0, 1,3,1,0, 1,9,1,0, 1,0,1,1, -2,0,-4,0" "18603" -group_label "group1"
1539 # mask 1:
1540 # XX.
1541 # X.X+
1542
1543 DELANDTSHEER_DOYEN_PROBLEM_27_53_MASK1=\
1544 ▷ -masktest 3 x ge 1 \
1545 ▷ -masktest 4 x+y ge 3 \
1546 ▷ -depth 4 \
1547 ▷ -mask_label "mask1"
1548
1549 DELANDTSHEER_DOYEN_PROBLEM_3_7= -d1 1 -q1 3 -d2 1 -q2 7 -K 5 -DDx 3 -DDy 1 -searc
    h_control -W -end
1550
1551 DELANDTSHEER_DOYEN_PROBLEM_3_7.GROUP1=-subgroup \
1552 ▷ "1,1,1,0, 1,0,1,1 " "21" -group_label "group-cyclic"
1553
1554 DELANDTSHEER_DOYEN_PROBLEM_3_7_MASK1= -mask_label "mask1" -depth 5
1555
1556
1557
1558 # Section 14.5: Design Theory - Flag transitive designs
1559
1560
1561
1562 PRAEGER_DESIGN_45_12_3_GENS="1,1,0, 1,3,0, 1,51,0, 1,76,0, 2,0,0, 3,0,1, 16,0,0,
    3,0,1"
1563
1564
1565
1566
1567
1568 # Chapter 15 Finite geometry
1569
1570 # Section 15.1 Spreads
1571

```

```

1572
1573 SPREAD_SET_27_RAO_RAO="\
1574 0,0,0,0,0,0,0,0,0, \
1575 1,1,0,2,1,1,0,0,2, \
1576 1,0,1,1,2,2,0,1,0, \
1577 1,2,2,1,2,0,2,2,2, \
1578 0,0,2,2,2,0,1,2,0, \
1579 1,1,2,0,2,1,2,1,0, \
1580 0,1,0,1,0,1,0,2,1, \
1581 2,0,2,0,0,2,1,1,0, \
1582 2,2,2,0,1,1,0,1,2, \
1583 2,0,0,1,0,2,1,2,1, \
1584 0,2,2,2,2,2,2,0,2, \
1585 2,1,2,0,2,0,2,0,1, \
1586 0,1,2,2,0,1,0,1,1, \
1587 1,0,0,0,1,0,0,0,1, \
1588 2,1,0,1,2,1,0,2,0, \
1589 0,2,0,0,2,2,1,1,2, \
1590 0,0,1,0,1,2,2,2,1, \
1591 2,0,1,2,2,1,1,0,1, \
1592 0,1,1,1,1,0,1,2,2, \
1593 2,2,0,2,0,0,0,2,2, \
1594 2,1,1,1,1,2,2,1,2, \
1595 2,2,1,2,1,0,2,0,0, \
1596 1,2,0,2,0,2,1,0,0, \
1597 1,2,1,1,0,0,1,1,1, \
1598 0,2,1,1,1,1,2,2,0, \
1599 1,1,1,0,0,1,1,0,2, \
1600 1,0,2,2,1,2,2,1,1 \
1601 "
1602
1603 SPREAD_S27_RAO_RAO="\
1604 0, 33879, 5418, 13103, 30556, 22107, 27225, 4045, 24924, 31961, \
1605 3196, 30100, 28081, 25862, 1339, 6696, 8242, 11747, 14000, 14705, \
1606 9784, 17843, 20772, 9271, 19413, 18678, 16109, 23924"
1607
1608
1609
1610
1611
1612 SPREADS_27_ISO_0="\
1613 0, 33879, 1339, 2678, 3994, 7671, 10180, 5862, 9524, 6852, 22243, \
1614 12745, 24295, 11062, 13615, 23894, 15056, 29367, 16429, 31521, 17726, \
1615 31103, 18887, 26333, 19566, 28400, 21531, 27228"
1616
1617 SPREADS_27_ISO_1="\
1618 0, 33879, 1339, 2678, 3994, 7671, 10182, 5761, 6796, 9327, 15339, \
1619 31914, 24415, 12713, 22748, 11666, 13353, 23555, 30103, 16395, 17827, \
1620 30790, 18254, 26422, 20046, 28112, 20900, 26801"
1621
1622 SPREADS_27_ISO_2="\
1623 0, 33879, 1339, 2678, 3994, 7671, 10182, 5817, 6796, 9276, 23891, \
1624 15368, 11666, 22124, 12713, 24415, 13353, 29619, 15910, 31914, 17030, \
1625 30931, 19213, 26422, 19905, 28112, 21217, 27545"
1626
1627 SPREADS_27_ISO_3="\
1628 0, 33879, 1339, 2678, 3994, 7671, 10625, 6590, 9476, 5576, 24688, \
1629 23043, 10996, 22124, 12723, 13522, 15421, 29894, 16532, 32442, 17997, \
1630 31015, 18311, 26109, 19807, 28113, 21220, 27195"

```

```

1631
1632 SPREADS.27_ISO_4="\
1633 0, 33879, 1339, 2678, 3994, 7674, 7051, 10666, 9327, 5419, 19806, \
1634 21332, 22124, 13353, 24415, 12401, 11062, 23717, 15056, 29660, 16395, \
1635 31950, 17873, 31153, 19212, 26221, 28515, 26708"
1636
1637 SPREADS.27_ISO_5="\
1638 0, 33879, 1339, 2678, 3994, 7988, 5333, 6672, 10666, 9327, 11062, \
1639 22124, 13353, 15056, 16395, 17347, 31055, 27061, 24415, 12401, 23076, \
1640 30103, 32394, 19041, 26109, 20380, 28400, 21332"
1641
1642 SPREADS.27_ISO_6="\
1643 0, 33879, 1339, 2679, 3992, 7672, 31140, 6913, 13513, 23167, 5653, \
1644 10607, 9131, 11225, 22548, 13074, 24645, 15124, 29345, 16226, 31506, \
1645 17684, 18732, 26116, 19458, 28361, 21571, 27680"
1646
1647
1648
1649 PENTTILA_WILLIAMS_PRINCE_REG_PACKING_0="444,43313,154402,46682,108254,75363,27729
,32139,5244,60442,142811,111115,94209,120678,89533,13798,103994,129953,82168,1368
38,19253,23017,145985,134996,54705,36267,55066,117542,96699,69154,72460"
1650
1651 PENTTILA_WILLIAMS_PRINCE_REG_PACKING_1="616,42728,152655,48576,105431,79607,28634
,32817,9799,62356,141176,110085,92557,122136,86312,13975,101942,126869,81478,1393
52,18028,24325,147284,130370,52074,36843,55602,118454,95973,69642,74036"
1652
1653
1654
1655 # Section 15.3 Packings
1656
1657
1658
1659
1660 PGL_4.3_CLASS_5A_REP="2,2,2,0,1,1,1,2,2,1,0,0,1,0,1,2"
1661
1662 PGL_4.3_CLASS_5A_N_GENS="1,0,0,0,2,0,1,1,2,1,2,1,1,0,0, 1,1,1,0,2,2,2,1,1,2,0,0
,2,0,2,1, 1,1,1,2,2,2,0,1,0,2,2,0,1,2,2,1"
1663
1664
1665 # nice generators, from Michael Epstein:
1666
1667 PGL_4.5_SUBGROUP_3B_ME--PGL 4 5 \
1668 ▷ -subgroup_by_generators "3B" 3 1 \
1669 ▷ "1,0,0,0, 0,1,0,0, 0,0,2,2, 0,0,4,2"
1670
1671 PGL_4.5_SUBGROUP_3B_ME_NORMALIZER--PGL 4 5 \
1672 ▷ -subgroup_by_generators "normalizer_3B" "5760" 8 \
1673 ▷ "1,0,0,0,0,4,0,0,0,0,1,0,0,0,0,1, \
1674 ▷ 1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,4, \
1675 ▷ 1,0,0,0,0,4,0,0,0,0,4,0,0,0,0,4, \
1676 ▷ 1,0,0,0,0,1,0,0,0,0,3,0,0,0,0,3, \
1677 ▷ 1,0,0,0,0,3,0,0,0,0,1,0,0,0,0,1, \
1678 ▷ 1,0,0,0,0,1,0,0,0,0,2,4,0,0,2,3, \
1679 ▷ 1,0,0,0,4,4,0,0,0,0,1,0,0,0,0,1, \
1680 ▷ 0,1,0,0,1,0,0,0,0,0,4,0,0,0,0,4,"
1681
1682 PGL_4.5_SUBGROUP_31_ME--PGL 4 5 \
1683 ▷ -subgroup_by_generators "31" 31 1 \
1684 ▷ "1,0,0,0, 0,3,4,3, 0,3,3,4, 0,3,2,3"

```

```

1685
1686 PGL_4_5_SUBGROUP_31_ME_NORMALIZER=-PGL 4 5 \
1687 ▷ -subgroup_by_generators "normalizer_31" "372" 4 \
1688 ▷ "1,0,0,0,0,4,0,0,0,0,4,0,0,0,0,4, \
1689 ▷ 1,0,0,0,0,3,0,0,0,0,3,0,0,0,0,3, \
1690 ▷ 1,0,0,0,0,4,0,0,0,0,2,1,0,3,2,4, \
1691 ▷ 1,0,0,0,0,0,1,0,0,0,0,1,0,1,1,3,"
1692
1693
1694
1695
1696
1697 # Section 15.4 BLT-Sets
1698
1699
1700 BLT_ORDER_Q="3,5,7,9,11,13,17,19,23,25,27,\
1701 29,31,37,41,43,47,49,53,59,61,67,71,73"
1702
1703 BLT_NUMBER_ISO="1,2,2,3,4,3,6,5,9,6,6,\
1704 9,8,7,10,6,10,8,8,9,5,6,8,5"
1705
1706
1707
1708 # Chapter 16 Computer Science Primitives
1709
1710 # Section 16.2 Diophantine systems
1711
1712
1713 TEST_SYSTEM="\
1714 0,1,0,1,0,0, \
1715 0,0,1,0,1,0, \
1716 1,0,1,0,0,0, \
1717 0,1,0,1,0,1, \
1718 1,0,0,0,0,1, \
1719 1,0,1,0,0,0, \
1720 0,1,0,0,1,1"
1721
1722 PRAEGER_DESIGN_LINE_TYPE="\
1723 10,4,1,0,0,0, \
1724 10,6,3,1,0,0, \
1725 5,4,3,2,1,0, \
1726 1,1,1,1,1,1"
1727
1728
1729 # Chapter 17 Canonical Forms with Nauty
1730
1731 # Section 17.2 Canonical forms of objects in projective space
1732
1733 KLEIN_CUBIC_THREEFOLD_EQN="X0^2*X1 + X1^2*X2 + X2^2*X3 + X3^2*X4 + X4^2*X0"
1734
1735
1736 BARTH_SEXTIC_MOD_11_SINGULAR_POINTS="0, 1, 2, 4, 39, 44, 61, 63, 64, 66, 94, 96,
1737 97, 99, 116, 121, 135, 144, 145, 171, 174, 226, 229, 244, 255, 275, 365, 374, 424
1738 , 425, 457, 458, 540, 573, 620, 625, 674, 681, 685, 692, 764, 767, 841, 844, 885,
1739 888, 962, 965, 983, 988, 1037, 1044, 1048, 1055, 1145, 1178, 1271, 1272, 1304, 1
1740 305, 1343, 1355, 1364, 1454, 1463"

```



```

1740
1741 HIRSCHFELD_SURFACE_EQUATION_ALGEBRAIC_FORM="X0^2*X3 + X1^2*X2 + X1*X2^2 + X0*X3^2
"
1742
1743
1744 SUZUKI_8_GENERATORS="\
1745 1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,1, \
1746 1,0,0,0,0,6,0,0,0,0,2,0,0,0,0,3,0, \
1747 1,0,0,0,1,1,1,0,0,0,1,0,1,0,0,1,0, \
1748 1,0,0,0,3,6,2,2,5,0,2,0,3,0,6,3,2, \
1749 0,1,0,0,1,0,0,0,0,0,0,1,0,0,1,0,2"
1750 # group order 87360 = 3 * 29120
1751
1752
1753
1754 # Section 17.3 Canonical Forms of Incidence Geometries
1755
1756
1757
1758 FILE_24_3_TFC_INC="24 24 72\
1759 \n0 1 2 24 27 28 48 53 54 73 79 80 97 105 106 122 131 \
1760 132 146 157 158 171 175 183 195 203 208 220 225 233 244 \
1761 258 259 269 272 282 293 300 308 318 325 333 342 352 358 \
1762 367 379 381 392 398 400 417 428 429 442 443 450 466 471 \
1763 479 492 497 502 517 519 521 542 548 551 571 574 575 \
1764 48\
1765 \n0 1 2 24 27 28 48 53 54 73 79 80 97 105 106 122 131 \
1766 132 146 157 158 171 175 183 195 203 208 220 225 233 244 \
1767 258 259 269 272 281 293 301 308 318 324 327 342 354 357 \
1768 367 373 378 392 400 403 417 419 430 442 446 447 466 472 \
1769 479 492 500 503 518 525 526 545 549 551 571 572 574 \
1770 48\
1771 \n0 1 2 24 27 28 48 53 54 73 79 80 97 105 106 122 131 \
1772 132 146 157 158 171 175 179 195 201 207 220 226 232 244 \
1773 257 258 269 274 277 293 300 307 318 323 329 342 352 356 \
1774 367 374 381 392 397 406 416 423 431 441 450 454 468 476 \
1775 477 494 499 503 519 521 525 544 547 550 570 572 575 \
1776 144\
1777 \n-1 3"
1778
1779
1780 # Section 17.5 Canonical forms of linear codes
1781
1782 # consider the binary code with generator matrix:
1783 # 1 0 1
1784 # 0 1 1
1785
1786 CODE_N3_K2_Q2_GENMA="1,0,1, 0,1,1"
1787
1788 CODE_N6_K3_Q2_GENMA="\
1789 111100\
1790 110010\
1791 101001"
1792
1793
1794
1795
1796 # Section 18.1 Automorphism group of Sym(6)
1797

```

```
1798
1799 SYM_6_CLASS_2A_REP="1,0,3,2,5,4"
1800
1801 SYM_6_CLASS_2B_REP="1,0,3,2,4,5"
1802
1803 SYM_6_CLASS_2C_REP="1,0,2,3,4,5"
1804
1805 FILE_NAME_SYM_6_CLASS_2A="Sym_6_class_of_2A.csv"
1806
1807 FILE_NAME_SYM_6_CLASS_2B="Sym_6_class_of_2B.csv"
1808
1809 FILE_NAME_SYM_6_CLASS_2C="Sym_6_class_of_2C.csv"
1810
1811
1812 AUT_SYM_6_GROUP_ORDER=1440
1813
1814 AUT_SYM_6_DEGREE=720
1815
1816 AUT_SYM_6_BASE="1,2,6,24"
1817
1818 AUT_SYM_6_NB_GENS=6
1819
1820 AUT_SYM_6_GENS="0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18,
    19, 20, 21, 22, 23, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273, 274, 275,
    276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 384, 385, 386, 387, 3
    88, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 40
    4, 405, 406, 407, 504, 505, 506, 507, 508, 509, 510, 511, 512, 513, 514, 515, 516
    , 517, 518, 519, 520, 521, 522, 523, 524, 525, 526, 527, 624, 625, 626, 627, 628,
    629, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644,
    645, 646, 647, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 1
    33, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 240, 241, 242, 243, 244, 24
    5, 246, 247, 248, 249, 250, 251, 252, 253, 254, 255, 256, 257, 258, 259, 260, 261
    , 262, 263, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373,
    374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 480, 481, 482, 483, 484, 485,
    486, 487, 488, 489, 490, 491, 492, 493, 494, 495, 496, 497, 498, 499, 500, 501, 5
    02, 503, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 61
    4, 615, 616, 617, 618, 619, 620, 621, 622, 623, 144, 145, 146, 147, 148, 149, 150
    , 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166,
    167, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42,
    43, 44, 45, 46, 47, 414, 415, 416, 417, 418, 419, 408, 409, 410, 411, 412, 413,
    422, 423, 420, 421, 425, 424, 428, 429, 426, 427, 431, 430, 534, 535, 536, 537, 5
    38, 539, 528, 529, 530, 531, 532, 533, 542, 543, 540, 541, 545, 544, 548, 549, 54
    6, 547, 551, 550, 654, 655, 656, 657, 658, 659, 648, 649, 650, 651, 652, 653, 662
    , 663, 660, 661, 665, 664, 668, 669, 666, 667, 671, 670, 168, 169, 170, 171, 172,
    173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188,
    189, 190, 191, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64
    , 65, 66, 67, 68, 69, 70, 71, 294, 295, 296, 297, 298, 299, 288, 289, 290, 291, 2
    92, 293, 302, 303, 300, 301, 305, 304, 308, 309, 306, 307, 311, 310, 558, 559, 56
    0, 561, 562, 563, 552, 553, 554, 555, 556, 557, 566, 567, 564, 565, 569, 568, 572
    , 573, 570, 571, 575, 574, 678, 679, 680, 681, 682, 683, 672, 673, 674, 675, 676,
    677, 686, 687, 684, 685, 689, 688, 692, 693, 690, 691, 695, 694, 192, 193, 194,
    195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 2
    11, 212, 213, 214, 215, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 8
    6, 87, 88, 89, 90, 91, 92, 93, 94, 95, 318, 319, 320, 321, 322, 323, 312, 313, 31
    4, 315, 316, 317, 326, 327, 324, 325, 329, 328, 332, 333, 330, 331, 335, 334, 438
    , 439, 440, 441, 442, 443, 432, 433, 434, 435, 436, 437, 446, 447, 444, 445, 449,
    448, 452, 453, 450, 451, 455, 454, 702, 703, 704, 705, 706, 707, 696, 697, 698,
    699, 700, 701, 710, 711, 708, 709, 713, 712, 716, 717, 714, 715, 719, 718, 216, 2
    17, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 23
```

```

3, 234, 235, 236, 237, 238, 239, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 10
6, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 342, 343, 344
, 345, 346, 347, 336, 337, 338, 339, 340, 341, 350, 351, 348, 349, 353, 352, 356,
357, 354, 355, 359, 358, 462, 463, 464, 465, 466, 467, 456, 457, 458, 459, 460,
461, 470, 471, 468, 469, 473, 472, 476, 477, 474, 475, 479, 478, 582, 583, 584, 5
85, 586, 587, 576, 577, 578, 579, 580, 581, 590, 591, 588, 589, 593, 592, 596, 59
7, 594, 595, 599, 598, \
1821 0, 1, 2, 3, 4, 5, 54, 55, 56, 57, 58, 59, 78, 79, 80, 81, 82, 83, 102, 103, 104,
105, 106, 107, 24, 25, 26, 27, 28, 29, 48, 49, 50, 51, 52, 53, 72, 73, 74, 75, 76
, 77, 96, 97, 98, 99, 100, 101, 30, 31, 32, 33, 34, 35, 6, 7, 8, 9, 10, 11, 86, 8
7, 84, 85, 89, 88, 110, 111, 108, 109, 113, 112, 36, 37, 38, 39, 40, 41, 12, 13,
14, 15, 16, 17, 62, 63, 60, 61, 65, 64, 116, 117, 114, 115, 119, 118, 42, 43, 44,
45, 46, 47, 18, 19, 20, 21, 22, 23, 68, 69, 66, 67, 71, 70, 92, 93, 90, 91, 95,
94, 264, 265, 266, 267, 268, 269, 288, 289, 290, 291, 292, 293, 312, 313, 314, 31
5, 316, 317, 336, 337, 338, 339, 340, 341, 240, 241, 242, 243, 244, 245, 294, 295
, 296, 297, 298, 299, 318, 319, 320, 321, 322, 323, 342, 343, 344, 345, 346, 347,
246, 247, 248, 249, 250, 251, 270, 271, 272, 273, 274, 275, 324, 325, 326, 327,
328, 329, 348, 349, 350, 351, 352, 353, 252, 253, 254, 255, 256, 257, 276, 277, 2
78, 279, 280, 281, 300, 301, 302, 303, 304, 305, 354, 355, 356, 357, 358, 359, 25
8, 259, 260, 261, 262, 263, 282, 283, 284, 285, 286, 287, 306, 307, 308, 309, 310
, 311, 330, 331, 332, 333, 334, 335, 144, 145, 146, 147, 148, 149, 168, 169, 170,
171, 172, 173, 192, 193, 194, 195, 196, 197, 216, 217, 218, 219, 220, 221, 120,
121, 122, 123, 124, 125, 174, 175, 176, 177, 178, 179, 198, 199, 200, 201, 202, 2
03, 222, 223, 224, 225, 226, 227, 126, 127, 128, 129, 130, 131, 150, 151, 152, 15
3, 154, 155, 204, 205, 206, 207, 208, 209, 228, 229, 230, 231, 232, 233, 132, 133
, 134, 135, 136, 137, 156, 157, 158, 159, 160, 161, 180, 181, 182, 183, 184, 185,
234, 235, 236, 237, 238, 239, 138, 139, 140, 141, 142, 143, 162, 163, 164, 165,
166, 167, 186, 187, 188, 189, 190, 191, 210, 211, 212, 213, 214, 215, 408, 409, 4
10, 411, 412, 413, 384, 385, 386, 387, 388, 389, 434, 435, 432, 433, 437, 436, 45
8, 459, 456, 457, 461, 460, 366, 367, 368, 369, 370, 371, 390, 391, 392, 393, 394
, 395, 444, 445, 446, 447, 448, 449, 468, 469, 470, 471, 472, 473, 360, 361, 362,
363, 364, 365, 414, 415, 416, 417, 418, 419, 438, 439, 440, 441, 442, 443, 462,
463, 464, 465, 466, 467, 374, 375, 372, 373, 377, 376, 420, 421, 422, 423, 424, 4
25, 396, 397, 398, 399, 400, 401, 475, 474, 478, 479, 476, 477, 380, 381, 378, 37
9, 383, 382, 426, 427, 428, 429, 430, 431, 402, 403, 404, 405, 406, 407, 451, 450
, 454, 455, 452, 453, 528, 529, 530, 531, 532, 533, 504, 505, 506, 507, 508, 509,
554, 555, 552, 553, 557, 556, 578, 579, 576, 577, 581, 580, 486, 487, 488, 489,
490, 491, 510, 511, 512, 513, 514, 515, 564, 565, 566, 567, 568, 569, 588, 589, 5
90, 591, 592, 593, 480, 481, 482, 483, 484, 485, 534, 535, 536, 537, 538, 539, 55
8, 559, 560, 561, 562, 563, 582, 583, 584, 585, 586, 587, 494, 495, 492, 493, 497
, 496, 540, 541, 542, 543, 544, 545, 516, 517, 518, 519, 520, 521, 595, 594, 598,
599, 596, 597, 500, 501, 498, 499, 503, 502, 546, 547, 548, 549, 550, 551, 522,
523, 524, 525, 526, 527, 571, 570, 574, 575, 572, 573, 648, 649, 650, 651, 652, 6
53, 624, 625, 626, 627, 628, 629, 674, 675, 672, 673, 677, 676, 698, 699, 696, 69
7, 701, 700, 606, 607, 608, 609, 610, 611, 630, 631, 632, 633, 634, 635, 684, 685
, 686, 687, 688, 689, 708, 709, 710, 711, 712, 713, 600, 601, 602, 603, 604, 605,
654, 655, 656, 657, 658, 659, 678, 679, 680, 681, 682, 683, 702, 703, 704, 705,
706, 707, 614, 615, 612, 613, 617, 616, 660, 661, 662, 663, 664, 665, 636, 637, 6
38, 639, 640, 641, 715, 714, 718, 719, 716, 717, 620, 621, 618, 619, 623, 622, 66
6, 667, 668, 669, 670, 671, 642, 643, 644, 645, 646, 647, 691, 690, 694, 695, 692
, 693, \
1822 0, 1, 21, 20, 15, 14, 6, 7, 19, 18, 13, 12, 11, 10, 5, 4, 16, 17, 9, 8, 3, 2, 22,
23, 54, 55, 67, 66, 61, 60, 48, 49, 69, 68, 63, 62, 53, 52, 59, 58, 65, 64, 51,
50, 57, 56, 71, 70, 30, 31, 43, 42, 37, 36, 24, 25, 45, 44, 39, 38, 29, 28, 35, 3
4, 41, 40, 27, 26, 33, 32, 47, 46, 109, 108, 103, 102, 114, 115, 99, 98, 105, 104
, 119, 118, 97, 96, 111, 110, 117, 116, 100, 101, 113, 112, 107, 106, 85, 84, 79,
78, 90, 91, 75, 74, 81, 80, 95, 94, 73, 72, 87, 86, 93, 92, 76, 77, 89, 88, 83,
82, 120, 121, 141, 140, 135, 134, 126, 127, 139, 138, 133, 132, 131, 130, 125, 12
4, 136, 137, 129, 128, 123, 122, 142, 143, 174, 175, 187, 186, 181, 180, 168, 169

```

, 189, 188, 183, 182, 173, 172, 179, 178, 185, 184, 171, 170, 177, 176, 191, 190,
 150, 151, 163, 162, 157, 156, 144, 145, 165, 164, 159, 158, 149, 148, 155, 154,
 161, 160, 147, 146, 153, 152, 167, 166, 229, 228, 223, 222, 234, 235, 219, 218, 2
 25, 224, 239, 238, 217, 216, 231, 230, 237, 236, 220, 221, 233, 232, 227, 226, 20
 5, 204, 199, 198, 210, 211, 195, 194, 201, 200, 215, 214, 193, 192, 207, 206, 213
 , 212, 196, 197, 209, 208, 203, 202, 366, 367, 379, 378, 373, 372, 360, 361, 381,
 380, 375, 374, 365, 364, 371, 370, 377, 376, 363, 362, 369, 368, 383, 382, 390,
 391, 403, 402, 397, 396, 384, 385, 405, 404, 399, 398, 389, 388, 395, 394, 401, 4
 00, 387, 386, 393, 392, 407, 406, 414, 415, 427, 426, 421, 420, 408, 409, 429, 42
 8, 423, 422, 413, 412, 419, 418, 425, 424, 411, 410, 417, 416, 431, 430, 463, 462
 , 469, 468, 475, 474, 457, 456, 471, 470, 477, 476, 459, 458, 465, 464, 479, 478,
 461, 460, 467, 466, 473, 472, 439, 438, 445, 444, 451, 450, 433, 432, 447, 446,
 453, 452, 435, 434, 441, 440, 455, 454, 437, 436, 443, 442, 449, 448, 246, 247, 2
 59, 258, 253, 252, 240, 241, 261, 260, 255, 254, 245, 244, 251, 250, 257, 256, 24
 3, 242, 249, 248, 263, 262, 270, 271, 283, 282, 277, 276, 264, 265, 285, 284, 279
 , 278, 269, 268, 275, 274, 281, 280, 267, 266, 273, 272, 287, 286, 294, 295, 307,
 306, 301, 300, 288, 289, 309, 308, 303, 302, 293, 292, 299, 298, 305, 304, 291,
 290, 297, 296, 311, 310, 343, 342, 349, 348, 355, 354, 337, 336, 351, 350, 357, 3
 56, 339, 338, 345, 344, 359, 358, 341, 340, 347, 346, 353, 352, 319, 318, 325, 32
 4, 331, 330, 313, 312, 327, 326, 333, 332, 315, 314, 321, 320, 335, 334, 317, 316
 , 323, 322, 329, 328, 613, 612, 607, 606, 618, 619, 603, 602, 609, 608, 623, 622,
 601, 600, 615, 614, 621, 620, 604, 605, 617, 616, 611, 610, 637, 636, 631, 630,
 642, 643, 627, 626, 633, 632, 647, 646, 625, 624, 639, 638, 645, 644, 628, 629, 6
 41, 640, 635, 634, 679, 678, 685, 684, 691, 690, 673, 672, 687, 686, 693, 692, 67
 5, 674, 681, 680, 695, 694, 677, 676, 683, 682, 689, 688, 655, 654, 661, 660, 667
 , 666, 649, 648, 663, 662, 669, 668, 651, 650, 657, 656, 671, 670, 653, 652, 659,
 658, 665, 664, 702, 703, 715, 714, 709, 708, 696, 697, 717, 716, 711, 710, 701,
 700, 707, 706, 713, 712, 699, 698, 705, 704, 719, 718, 493, 492, 487, 486, 498, 4
 99, 483, 482, 489, 488, 503, 502, 481, 480, 495, 494, 501, 500, 484, 485, 497, 49
 6, 491, 490, 517, 516, 511, 510, 522, 523, 507, 506, 513, 512, 527, 526, 505, 504
 , 519, 518, 525, 524, 508, 509, 521, 520, 515, 514, 559, 558, 565, 564, 571, 570,
 553, 552, 567, 566, 573, 572, 555, 554, 561, 560, 575, 574, 557, 556, 563, 562,
 569, 568, 535, 534, 541, 540, 547, 546, 529, 528, 543, 542, 549, 548, 531, 530, 5
 37, 536, 551, 550, 533, 532, 539, 538, 545, 544, 582, 583, 595, 594, 589, 588, 57
 6, 577, 597, 596, 591, 590, 581, 580, 587, 586, 593, 592, 579, 578, 585, 584, 599
 , 598, \

1823 0, 143, 475, 680, 543, 316, 659, 339, 496, 79, 398, 186, 212, 516, 289, 425, 107,
 603, 58, 373, 702, 598, 165, 266, 450, 566, 25, 257, 220, 639, 356, 660, 168, 40
 7, 77, 489, 515, 195, 617, 97, 428, 300, 381, 50, 279, 148, 588, 716, 557, 441, 6
 34, 229, 260, 36, 127, 16, 333, 530, 685, 466, 414, 310, 118, 606, 505, 209, 577,
 713, 152, 283, 364, 63, 246, 46, 237, 626, 559, 448, 676, 471, 538, 325, 134, 18
 , 95, 480, 391, 184, 353, 649, 274, 157, 699, 587, 67, 368, 643, 224, 35, 243, 43
 2, 575, 320, 547, 458, 693, 9, 125, 177, 389, 498, 86, 670, 342, 620, 108, 299, 4
 11, 198, 526, 341, 657, 518, 210, 376, 55, 141, 2, 423, 292, 595, 704, 678, 478,
 82, 493, 264, 167, 313, 545, 188, 396, 605, 105, 561, 437, 262, 30, 632, 235, 10,
 133, 709, 586, 309, 410, 457, 689, 158, 282, 484, 87, 534, 334, 112, 607, 385, 1
 85, 446, 570, 218, 645, 27, 251, 666, 350, 53, 369, 192, 527, 501, 74, 273, 149,
 468, 692, 395, 171, 623, 96, 548, 324, 41, 241, 637, 226, 455, 552, 201, 509, 378
 , 62, 664, 343, 296, 427, 578, 717, 15, 124, 114, 614, 174, 406, 323, 531, 231, 6
 28, 252, 44, 568, 439, 71, 360, 511, 208, 359, 648, 700, 591, 418, 301, 128, 19,
 151, 280, 91, 488, 675, 467, 662, 354, 197, 513, 48, 383, 564, 452, 100, 615, 145
 , 281, 255, 28, 405, 170, 714, 590, 641, 217, 491, 75, 302, 426, 136, 7, 687, 460
 , 331, 536, 435, 563, 284, 156, 610, 109, 32, 259, 583, 712, 183, 388, 238, 630,
 408, 311, 89, 481, 23, 120, 541, 322, 477, 674, 345, 653, 374, 66, 520, 199, 178,
 397, 696, 599, 45, 242, 92, 492, 295, 424, 227, 627, 464, 691, 314, 549, 5, 129,
 522, 206, 57, 365, 336, 671, 619, 104, 155, 267, 438, 574, 417, 293, 502, 78, 64
 4, 228, 532, 327, 682, 469, 138, 14, 367, 64, 215, 504, 655, 352, 270, 166, 117,
 602, 553, 449, 581, 705, 392, 187, 250, 37, 13, 130, 307, 416, 711, 580, 443, 555
 , 608, 115, 286, 150, 232, 631, 528, 335, 65, 361, 38, 258, 463, 688, 207, 508, 3

48, 668, 72, 503, 173, 393, 572, 444, 147, 275, 98, 621, 647, 216, 371, 51, 326,
 546, 249, 29, 525, 194, 690, 470, 651, 347, 400, 175, 494, 90, 122, 21, 597, 698,
 421, 298, 319, 544, 68, 372, 629, 225, 672, 479, 202, 517, 240, 47, 487, 88, 191
 , 384, 654, 358, 412, 303, 706, 589, 139, 8, 111, 604, 276, 164, 569, 433, 512, 2
 11, 461, 681, 31, 256, 402, 182, 81, 485, 337, 665, 584, 715, 290, 429, 4, 135, 1
 61, 265, 613, 106, 454, 558, 382, 54, 537, 317, 234, 638, 473, 673, 482, 93, 160,
 271, 43, 248, 181, 394, 585, 701, 624, 239, 328, 535, 366, 70, 445, 562, 20, 132
 , 652, 351, 507, 203, 430, 294, 612, 116, 244, 39, 719, 576, 403, 176, 223, 640,
 60, 380, 533, 321, 554, 453, 346, 661, 121, 17, 204, 524, 102, 622, 291, 419, 694
 , 462, 269, 153, 80, 499, 551, 312, 363, 59, 230, 642, 131, 3, 669, 338, 571, 440
 , 305, 409, 601, 113, 214, 510, 85, 490, 162, 278, 683, 459, 386, 189, 592, 703,
 253, 34, 355, 656, 6, 142, 436, 567, 99, 611, 420, 308, 521, 193, 287, 144, 495,
 76, 476, 684, 710, 594, 172, 399, 26, 261, 49, 377, 330, 542, 633, 221, 550, 318,
 387, 179, 110, 618, 222, 646, 84, 500, 413, 297, 245, 33, 695, 456, 523, 200, 57
 3, 434, 123, 11, 344, 667, 362, 69, 593, 697, 277, 154, 625, 233, 304, 415, 486,
 94, 42, 254, 205, 514, 465, 677, 442, 565, 650, 357, 22, 126, 61, 370, 163, 272,
 707, 579, 329, 529, 600, 119, 190, 390, 472, 679, 506, 213, 40, 247, 12, 140, 349
 , 658, 447, 556, 718, 582, 268, 159, 56, 379, 180, 404, 103, 616, 315, 539, 483,
 83, 431, 288, 636, 236, 663, 340, 137, 1, 560, 451, 146, 285, 596, 708, 375, 52,
 609, 101, 401, 169, 540, 332, 306, 422, 73, 497, 219, 635, 196, 519, 686, 474, 26
 3, 24, \

1824 0, 80, 14, 38, 74, 24, 6, 86, 12, 62, 72, 48, 8, 32, 2, 56, 26, 50, 84, 30, 78, 5
 4, 36, 60, 5, 82, 16, 40, 76, 29, 19, 92, 13, 116, 73, 97, 22, 46, 3, 106, 27, 10
 0, 90, 43, 79, 103, 37, 114, 11, 88, 17, 64, 77, 53, 21, 94, 15, 118, 75, 99, 23,
 70, 9, 112, 51, 101, 91, 67, 85, 109, 61, 115, 10, 34, 4, 58, 28, 52, 20, 44, 1,
 104, 25, 98, 18, 68, 7, 110, 49, 96, 42, 66, 31, 108, 55, 102, 89, 35, 83, 59, 4
 1, 65, 95, 45, 81, 105, 39, 119, 93, 69, 87, 111, 63, 117, 47, 71, 33, 113, 57, 1
 07, 633, 713, 647, 671, 707, 657, 639, 719, 645, 695, 705, 681, 641, 665, 635, 68
 9, 659, 683, 717, 663, 711, 687, 669, 693, 631, 708, 642, 666, 702, 655, 625, 698
 , 644, 620, 704, 601, 628, 652, 634, 610, 658, 604, 696, 649, 710, 607, 668, 618,
 637, 714, 643, 690, 703, 679, 627, 700, 646, 622, 706, 603, 629, 676, 640, 616,
 682, 605, 697, 673, 716, 613, 692, 619, 636, 660, 630, 684, 654, 678, 626, 650, 6
 32, 608, 656, 602, 624, 674, 638, 614, 680, 600, 648, 672, 662, 612, 686, 606, 71
 5, 661, 709, 685, 667, 691, 701, 651, 712, 609, 670, 623, 699, 675, 718, 615, 694
 , 621, 653, 677, 664, 617, 688, 611, 395, 449, 401, 425, 443, 419, 405, 455, 399,
 479, 441, 465, 407, 431, 393, 473, 417, 467, 453, 429, 447, 471, 423, 477, 390,
 444, 396, 420, 438, 414, 384, 434, 398, 374, 440, 360, 386, 410, 392, 368, 416, 3
 62, 432, 408, 446, 366, 422, 372, 403, 450, 397, 474, 439, 463, 389, 436, 400, 37
 6, 442, 365, 387, 460, 406, 382, 466, 363, 433, 457, 452, 379, 476, 373, 402, 426
 , 391, 468, 415, 462, 388, 412, 394, 370, 418, 364, 385, 458, 404, 380, 464, 361,
 409, 456, 428, 378, 470, 367, 451, 427, 445, 469, 421, 475, 437, 413, 448, 371,
 424, 377, 435, 459, 454, 381, 478, 375, 411, 461, 430, 383, 472, 369, 275, 329, 2
 81, 305, 323, 299, 285, 335, 279, 359, 321, 345, 287, 311, 273, 353, 297, 347, 33
 3, 309, 327, 351, 303, 357, 270, 324, 276, 300, 318, 294, 264, 314, 278, 254, 320
 , 240, 266, 290, 272, 248, 296, 242, 312, 288, 326, 246, 302, 252, 283, 330, 277,
 354, 319, 343, 269, 316, 280, 256, 322, 245, 267, 340, 286, 262, 346, 243, 313,
 337, 332, 259, 356, 253, 282, 306, 271, 348, 295, 342, 268, 292, 274, 250, 298, 2
 44, 265, 338, 284, 260, 344, 241, 289, 336, 308, 258, 350, 247, 331, 307, 325, 34
 9, 301, 355, 317, 293, 328, 251, 304, 257, 315, 339, 334, 261, 358, 255, 291, 341
 , 310, 263, 352, 249, 521, 545, 515, 569, 539, 563, 527, 551, 513, 593, 537, 587,
 525, 575, 519, 599, 561, 585, 549, 573, 543, 597, 567, 591, 516, 540, 510, 564,
 534, 558, 506, 530, 512, 488, 536, 482, 504, 554, 518, 494, 560, 480, 528, 552, 5
 42, 492, 566, 486, 522, 546, 511, 588, 535, 582, 508, 532, 514, 490, 538, 484, 50
 5, 578, 524, 500, 584, 481, 529, 576, 548, 498, 590, 487, 523, 570, 517, 594, 559
 , 583, 509, 556, 520, 496, 562, 485, 507, 580, 526, 502, 586, 483, 553, 577, 572,
 499, 596, 493, 547, 571, 541, 595, 565, 589, 533, 557, 544, 497, 568, 491, 531,
 581, 550, 503, 592, 489, 555, 579, 574, 501, 598, 495, 209, 155, 203, 179, 161, 1
 85, 215, 165, 201, 225, 159, 239, 213, 189, 207, 231, 183, 237, 167, 191, 153, 23
 3, 177, 227, 204, 150, 198, 174, 156, 180, 194, 144, 200, 120, 158, 134, 192, 168

```

, 206, 126, 182, 132, 146, 170, 152, 128, 176, 122, 210, 163, 199, 223, 157, 234,
  196, 149, 202, 125, 160, 136, 193, 217, 212, 139, 236, 133, 147, 220, 166, 142,
226, 123, 211, 187, 205, 229, 181, 235, 197, 173, 208, 131, 184, 137, 195, 219, 2
14, 141, 238, 135, 171, 221, 190, 143, 232, 129, 162, 186, 151, 228, 175, 222, 14
8, 172, 154, 130, 178, 124, 145, 218, 164, 140, 224, 121, 169, 216, 188, 138, 230
, 127, \
1825 0, 475, 572, 147, 244, 719, 136, 687, 348, 72, 507, 430, 435, 284, 659, 496, 60,
223, 32, 583, 212, 289, 371, 647, 450, 25, 122, 597, 694, 269, 561, 262, 651, 400
, 204, 102, 10, 709, 356, 168, 363, 551, 457, 158, 515, 617, 68, 319, 662, 197, 1
3, 307, 473, 482, 341, 518, 443, 608, 43, 181, 141, 423, 564, 100, 624, 328, 678,
82, 255, 405, 232, 528, 345, 374, 557, 634, 162, 85, 666, 53, 127, 333, 592, 386
, 446, 218, 23, 541, 305, 601, 273, 501, 696, 178, 414, 118, 522, 57, 246, 237, 7
06, 412, 201, 378, 676, 538, 276, 111, 41, 637, 464, 314, 487, 191, 578, 296, 155
, 619, 95, 391, 693, 125, 265, 454, 594, 26, 469, 14, 485, 665, 308, 193, 166, 55
3, 86, 342, 633, 377, 705, 250, 411, 526, 234, 54, 148, 716, 479, 240, 3, 571, 27
2, 579, 90, 175, 419, 622, 679, 40, 489, 407, 230, 312, 140, 447, 300, 97, 629, 5
44, 75, 426, 688, 508, 132, 351, 500, 297, 150, 115, 701, 394, 456, 200, 281, 615
, 70, 535, 11, 667, 590, 170, 361, 335, 492, 227, 283, 63, 436, 656, 83, 636, 466
, 530, 253, 189, 159, 379, 674, 322, 510, 113, 560, 340, 45, 397, 209, 606, 293,
644, 587, 368, 31, 211, 422, 219, 18, 325, 569, 604, 708, 52, 129, 549, 358, 384,
263, 519, 438, 104, 649, 184, 243, 575, 715, 4, 144, 476, 44, 439, 182, 337, 611
, 521, 591, 128, 389, 670, 330, 49, 280, 675, 108, 198, 537, 382, 598, 266, 29, 6
90, 453, 121, 697, 154, 393, 503, 116, 294, 254, 465, 186, 79, 533, 640, 565, 22,
603, 425, 326, 216, 396, 105, 258, 207, 562, 652, 179, 618, 580, 416, 271, 93, 3
3, 523, 704, 292, 366, 239, 434, 344, 167, 493, 65, 631, 171, 548, 713, 364, 6, 3
55, 404, 315, 36, 229, 683, 490, 582, 56, 251, 645, 214, 409, 137, 663, 468, 74,
505, 310, 614, 323, 157, 67, 461, 512, 101, 540, 448, 626, 139, 303, 285, 375, 55
2, 226, 654, 88, 686, 196, 15, 427, 353, 480, 504, 352, 429, 135, 76, 684, 208, 6
48, 106, 558, 399, 261, 301, 19, 602, 449, 542, 221, 488, 467, 187, 37, 317, 638,
311, 481, 194, 470, 661, 17, 415, 94, 621, 275, 176, 576, 514, 677, 109, 156, 32
1, 380, 357, 126, 388, 712, 546, 51, 607, 185, 517, 47, 338, 440, 119, 390, 298,
698, 499, 153, 213, 247, 410, 586, 642, 59, 658, 556, 87, 282, 225, 372, 96, 324,
424, 627, 142, 567, 616, 539, 199, 66, 459, 278, 288, 236, 527, 369, 34, 703, 1,
451, 692, 149, 242, 599, 406, 531, 78, 228, 681, 256, 169, 332, 671, 365, 8, 589
, 497, 635, 343, 62, 433, 164, 474, 24, 124, 717, 574, 267, 64, 655, 224, 432, 49
5, 287, 360, 359, 547, 9, 172, 710, 628, 568, 327, 138, 99, 420, 91, 151, 392, 58
1, 620, 299, 630, 89, 373, 165, 346, 554, 233, 486, 316, 680, 403, 39, 69, 277, 5
36, 460, 612, 203, 650, 442, 183, 259, 107, 516, 334, 385, 50, 588, 669, 131, 529
, 190, 639, 257, 80, 462, 370, 707, 235, 30, 291, 524, 349, 12, 484, 689, 428, 19
5, 545, 605, 217, 302, 445, 20, 318, 110, 383, 513, 160, 673, 646, 413, 55, 210,
585, 248, 573, 123, 264, 478, 714, 28, 174, 114, 502, 417, 274, 699, 401, 609, 33
6, 206, 559, 46, 73, 306, 664, 509, 134, 471, 146, 596, 455, 241, 5, 691, 367, 21
5, 643, 35, 290, 584, 71, 511, 320, 458, 613, 161, 231, 252, 532, 682, 402, 81, 4
18, 700, 117, 270, 177, 498, 238, 408, 58, 702, 525, 249, 625, 304, 543, 143, 98,
444, 362, 593, 331, 7, 173, 668, 205, 42, 610, 563, 398, 339, 534, 112, 381, 279
, 202, 672, 329, 600, 220, 566, 421, 21, 61, 163, 632, 437, 494, 347, 506, 472, 3
09, 133, 77, 660, 313, 188, 641, 491, 38, 463, 550, 387, 48, 354, 711, 130, 222,
84, 376, 657, 286, 555, 245, 695, 595, 2, 145, 452, 623, 395, 295, 92, 577, 152,
103, 180, 520, 653, 260, 441, 431, 483, 192, 350, 685, 16, 718, 268, 27, 570, 477
, 120"
1826
1827
1828
1829
1830
1831
1832
1833
1834 # Chapter 19 Interfaces

```

```
1835
1836
1837 # Section 19.2 Poyvray interface
1838
1839 # povray colors:
1840 POLISHED_CHROME_WHITE=\
1841 ▷ "texture{ Polished_Chrome pigment{quick_color White} }"
1842
1843 YELLOW_TRANSPARENT=\
1844 ▷ "texture{ pigment{ color Yellow transmit 0.7 } \
1845 ▷ finish {diffuse 0.9 phong 0.6} }"
1846
1847 COLOR_RED=\
1848 ▷ "texture{ pigment{ color Red } \
1849 ▷ finish {diffuse 0.9 phong 0.6} }"
1850
1851 COLOR_RED_SHINY=\
1852 ▷ "texture{ pigment{ color Red } \
1853 ▷ finish { diffuse 0.9 phong 1}]"
1854
1855 COLOR_GREEN_SHINY=\
1856 ▷ "texture{ pigment{ color Green } \
1857 ▷ finish { diffuse 0.9 phong 1}]"
1858
1859 COLOR_BLUE_SHINY=\
1860 ▷ "texture{ pigment{ color Blue } \
1861 ▷ finish { diffuse 0.9 phong 1}]"
1862
1863 COLOR_YELLOW_SHINY=\
1864 ▷ "texture{ pigment{ color Yellow } \
1865 ▷ finish { diffuse 0.9 phong 1}]"
1866
1867 COLOR_BLACK_SHINY=\
1868 ▷ "texture{ pigment{ color Black } \
1869 ▷ finish { diffuse 0.9 phong 1}]"
1870
1871 COLOR_RED_SEE_THROUGH=\
1872 ▷ "texture{ pigment{ color Red transmit 0.5 } \
1873 ▷ finish { diffuse 0.9 phong 1}]"
1874
1875 COLOR_GREEN_SEE_THROUGH=\
1876 ▷ "texture{ pigment{ color Green transmit 0.5 } \
1877 ▷ finish { diffuse 0.9 phong 1}]"
1878
1879 COLOR_BLUE_SEE_THROUGH=\
1880 ▷ "texture{ pigment{ color Blue transmit 0.5 } \
1881 ▷ finish { diffuse 0.9 phong 1}]"
1882
1883 COLOR_YELLOW_SEE_THROUGH=\
1884 ▷ "texture{ pigment{ color Yellow transmit 0.5 } \
1885 ▷ finish { diffuse 0.9 phong 1}]"
1886
1887 COLOR_YELLOW_THICK=\
1888 ▷ "texture{ pigment{ color Yellow } \
1889 ▷ finish { diffuse 0.9 phong 1}]"
1890
1891 COLOR_BLACK_NO_SHADOW=\
1892 ▷ "texture{ pigment{Black} } no_shadow"
1893
```

```

1894 SURFACE.COLOR=\
1895 ▷ "texture{ pigment{ White*0.5 } \
1896 ▷ finish {ambient 0.4 diffuse 0.5 roughness 0.001 \
1897 ▷ reflection 0.1 specular .8} }"
1898
1899 SURFACE.COLOR.SEETHROUGH=\
1900 ▷ "texture{ pigment{ White*0.5 transmit 0.5 } \
1901 ▷ finish {ambient 0.4 diffuse 0.5 roughness 0.001 \
1902 ▷ reflection 0.1 specular .8} }"
1903
1904 COLOR.GOLD=\
1905 ▷ "texture{ pigment{ Gold } finish \
1906 ▷ {ambient 0.4 diffuse 0.5 roughness 0.001 \
1907 ▷ reflection 0.1 specular .8} }"
1908
1909 COLOR.TURQUOISE=\
1910 ▷ "texture{ pigment{Cyan*1.3} \
1911 ▷ finish {ambient 0.4 diffuse 0.6 roughness 0.001 \
1912 ▷ reflection 0 specular .8} }"
1913
1914 MONKEY_SADDLE_CUBIC="1,0,0,0,-3,0,0,0,0,0,0,0,0,0,0,0,-1,0"
1915
1916 ECKARDT_CUBIC_DEFORM1_LEX="0, 10, 0, -8, 10, \
1917 25, 2, 0, -20, -8, -20, -10, -24, 10, -2, 12, \
1918 0, -8, 8, 16"
1919
1920 ECKARDT_CUBIC_DEFORM2_LEX="0, -5, 0, -5, -5, \
1921 10, -1, 0, 10, 4, 10, 5, 3, -5, 1, -6, 0, -5, \
1922 -4, 1"
1923
1924 KUMMER_QUARTIC_LEX_35="-2,0,0,0,2,0,0,2,0,0,\
1925 0,0,0,0,0,0,0,-2,0,0,2,0,2,0,0,0,-2,0,2,0,-2"
1926
1927
1928
1929
1930 BEAUVILLE_QUINTIC_LEX_56="-44, 228, 400, 315, -396, -852, \
1931 -512, -553, -1050, -354, 284, 504, -62, -707, -1390, -1010, \
1932 281, -167, -1644, -1024, -72, -196, 192, 373, 322, 78, 150, \
1933 966, 1540, 348, -475, -492, 1063, 1550, 390, 0, 96, 3, -337, \
1934 -426, -66, 425, 673, -156, -216, -223, -60, 1543, 1998, 618, \
1935 263, -250, -919, 557, 1800, 741"
1936
1937 ENDRASS_OCTIC_LEX_165="-93.2548,0,0,0,-309.019,0,0,527.529,0,395.647,\
1938 0,0,0,0,0,0,0,0,-687.529,0,0,1582.59,0,1186.94,0,0,0,0,-1055.06,0,\
1939 -1582.59,0,-593.47,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,-309.019,\
1940 0,0,1582.59,0,1186.94,0,0,0,0,-2110.12,0,-3165.17,0,-1186.94,0,0,0,0,\
1941 0,874.039,0,1560.63,0,1677.92,0,343.362,0,0,0,0,0,0,0,0,0,0,0,0,\
1942 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,-93.2548,0,527.529,0,395.647,\
1943 0,0,0,0,-1055.06,0,-1582.59,0,-593.47,0,0,0,0,0,874.039,0,1560.63,0,\
1944 1677.92,0,343.362,0,0,0,0,0,0,0,0,-256,0,-468.077,0,-789.019,0,\
1945 -525.726,0,0.941125"
1946
1947
1948 # Clebsch map up for surface created using arc lifting
1949 # We take a circle of radius r centered at the origin in the affine real plane
1950 # and map it up on the surface.
1951 # The Clebsch surface has
1952 # a = d = 2.618033988 = (3+sqrt(5))/2

```



```
1953 # b = c = 1.618033988 = (1+sqrt(5))/2
1954 #
1955 CLEBSCH_A=2.618033988
1956
1957 CLEBSCH_D=2.618033988
1958
1959 CLEBSCH_B=1.618033988
1960
1961 CLEBSCH_C=1.618033988
1962
1963 TWO_PI=6.283185308
1964
1965
1966 # to go from the arclifting surface to the defining equation:
1967 #Matrix(4, 4, [[-0.44721360215312733, 1.1708204000530853, 1.1708204000530853, -0.
    4472135957999158], [-1.1708204000530853, 0.4472136021531272, 1.4472136021531272,
    0.4472135957999158], [4.2360680044124255, -4.2360680044124255, -4.236068004412425
    5, 0.], [1.6180340022062127, -2.6180340022062127, -1.6180340022062127, 0.]])
1968 #-0.44721360215312733, 1.1708204000530853, 1.1708204000530853, -0.447213595799915
    8
1969 #-1.1708204000530853, 0.4472136021531272, 1.4472136021531272, 0.4472135957999158
1970 #4.2360680044124255, -4.2360680044124255, -4.2360680044124255, 0.
1971 #1.6180340022062127, -2.6180340022062127, -1.6180340022062127, 0.
1972
1973
1974 T00=-0.44721360215312733
1975
1976 T01=1.1708204000530853
1977
1978 T02=1.1708204000530853
1979
1980 T03=-0.4472135957999158
1981
1982 T10=-1.1708204000530853
1983
1984 T11=0.4472136021531272
1985
1986 T12=1.4472136021531272
1987
1988 T13=0.4472135957999158
1989
1990 T20=4.2360680044124255
1991
1992 T21=-4.2360680044124255
1993
1994 T22=-4.2360680044124255
1995
1996 T23=0.
1997
1998 T30=1.6180340022062127
1999
2000 T31=-2.6180340022062127
2001
2002 T32=-1.6180340022062127
2003
2004 T33=0.
2005
2006
2007 CLEBSCH_CUBICS=\
```

```

2008 ▷ push b push b mult push d push c push m mult add mult \
2009 ▷ push b push c push d push d push m mult mult add mult \
2010 ▷ push a push d push d push m add mult mult add add \
2011 ▷ push a push c push m mult add mult \
2012 ▷ store c001 \
2013 ▷ push b push d mult \
2014 ▷ push b push 1 push m push c mult add mult \
2015 ▷ push d push a push 1 push m mult add mult add \
2016 ▷ push m push a mult add push c add \
2017 ▷ push c push m push a mult add \
2018 ▷ mult mult \
2019 ▷ store c002 \
2020 ▷ push b \
2021 ▷ push d push c push a push m mult add mult \
2022 ▷ push c push a push m push 1 mult add mult add mult \
2023 ▷ push a push d mult push c push 1 push m mult add mult \
2024 ▷ push m mult add \
2025 ▷ push a push c push m mult add mult \
2026 ▷ store c011 \
2027 ▷ push b push b push c mult mult \
2028 ▷ push 1 push d push m mult add mult \
2029 ▷ push a push b mult push c push d push d push m mult mult add mult \
2030 ▷ push m mult add \
2031 ▷ push a push d mult push c push d push m mult add mult add \
2032 ▷ push a push c push m mult add mult \
2033 ▷ store c012 \
2034 ▷ push m \
2035 ▷ push b push d push m mult add push c mult \
2036 ▷ push d push b push 1 push m mult add mult push m mult add push a mult \
2037 ▷ push b push c mult push d push 1 push m mult add mult add mult \
2038 ▷ push b push d push m mult add mult \
2039 ▷ store d001 \
2040 ▷ push m \
2041 ▷ push d push c push m mult add push a push a mult mult \
2042 ▷ push c push c mult push d push m mult add push a mult add \
2043 ▷ push m push b push c mult mult push c push 1 push m mult add mult add mult \
2044 ▷ push b push d push m mult add mult \
2045 ▷ store d011 \
2046 ▷ push m \
2047 ▷ push c push d mult push d push m mult add push a push a mult mult \
2048 ▷ push c push c mult push d push m mult add push a push b push m mult mult a
dd \
2049 ▷ push b push d push c push m mult add mult push c push m mult mult add \
2050 ▷ push b push d push m mult add mult mult \
2051 ▷ store d012 \
2052 ▷ push d push 1 push m mult add push a mult push m push b mult push 1 add push c
mult add \
2053 ▷ push b add push m push d mult add \
2054 ▷ push a push c mult mult \
2055 ▷ push b push d push m mult add mult \
2056 ▷ store d112 \
2057 ▷ push m \
2058 ▷ push b push d push m mult add push c mult push d push b push 1 push m mult add
mult \
2059 ▷ push m mult add push a mult push b push c mult push d push 1 push m mult add mu
lt add \
2060 ▷ push b push d push m mult add mult mult \
2061 ▷ store m002 \
2062 ▷ push m \

```

```

2063 ▷ push d push c push m mult add push a push a mult mult \
2064 ▷ push c push c mult push d push m mult add push a mult add \
2065 ▷ push b push c push m mult mult push c push 1 push m mult add mult add \
2066 ▷ push b push d push m mult add mult mult \
2067 ▷ store m012 \
2068 ▷ push m\
2069 ▷ push c push d mult push d push m mult add push a push a mult mult \
2070 ▷ push m push c push c mult push d push m mult add push a push b mult mult mult a
    dd \
2071 ▷ push m push b push d push c push m mult add push c mult mult mult add \
2072 ▷ push b push d push m mult add mult mult \
2073 ▷ store m022 \
2074 ▷ push d push 1 push m mult add push a mult \
2075 ▷ push m push b mult push 1 add push c mult add \
2076 ▷ push b add push m push d mult add \
2077 ▷ push a push c mult mult \
2078 ▷ push b push d push m mult add mult \
2079 ▷ store m122 \
2080 ▷ push m push a mult push c add push d mult push c push a push 1 push m mult add
    mult add \
2081 ▷ push b mult \
2082 ▷ push m push a push d mult mult push c push 1 push m mult add mult add \
2083 ▷ push b push d push m mult add mult \
2084 ▷ store n002 \
2085 ▷ push m \
2086 ▷ push c push d push m mult add push b mult push m push d push c push 1 push m mu
    lt add mult mult add \
2087 ▷ push a mult \
2088 ▷ push b push c mult push d push 1 push m mult add mult add mult \
2089 ▷ push a push b push c push m mult push d push m mult add add add mult \
2090 ▷ store n012 \
2091 ▷ push c push d push m mult add push b mult \
2092 ▷ push m push d push c push 1 push m mult add mult mult add \
2093 ▷ push a mult \
2094 ▷ push b push c mult push d push 1 push m mult add mult add \
2095 ▷ push a push d mult push m push b push c mult mult add mult \
2096 ▷ store n022 \
2097 ▷ push m \
2098 ▷ push c push d push m mult add push b mult \
2099 ▷ push m push d push c push 1 push m mult add mult mult add \
2100 ▷ push a mult \
2101 ▷ push b push c mult push d push 1 push m mult add mult add \
2102 ▷ push m push a mult push c add mult mult \
2103 ▷ store n112 \
2104 ▷ push m \
2105 ▷ push c push d push m mult add push b mult \
2106 ▷ push m push d push c push 1 push m mult add mult mult add \
2107 ▷ push a mult \
2108 ▷ push b push c mult push d push 1 push m mult add mult add \
2109 ▷ push a push d mult push m push b push c mult mult add mult mult \
2110 ▷ store n122
2111
2112
2113
2114 # Section 19.5 The Gnuplot interface
2115
2116 GNUPLOT_TEST_INPUT="Row,Curve-1,Curve-2\n1,2,4\n2,6,8\n3,9,10\n4,13,17\n5,17,20\n
    6,19,25\n7,23,30\nEND\n"
2117

```

```

2118 # Chapter 20 Mathematical Data in Orbiter
2119
2120
2121 # Section 20.1 Mathematical data on Cubic Surfaces
2122
2123 # Section 20.2 Mathematical data on BLT-sets
2124
2125 BLT_ORDER_Q="3,5,7,9,11,13,17,19,23,25,27,29,31,37,41,43,47,49,53,59,61,67,71,73"

2126
2127 BLT_NUMBER_ISO="1,2,2,3,4,3,6,5,9,6,6,9,8,7,10,6,10,8,8,9,5,6,8,5"
2128
2129
2130
2131
2132
2133
2134 #####
2135 # Chapter 1 - Introduction
2136 #####
2137
2138
2139
2140 #####
2141 # Chapter 2 - The Orbiter Language
2142 #####
2143
2144 test_2:
2145 ▷ make test_2.1
2146 ▷ make test_2.2
2147 ▷ make test_2.3
2148 ▷ make test_2.4
2149 ▷ make test_2.5
2150 ▷ make test_2.6
2151
2152
2153
2154 #####
2155 # Section 2.1: Orbiter Session
2156 ▷
2157 ▷
2158 SECTION_ORBITER_SESSION:
2159
2160 test_2.1:
2161 ▷ make test_orbiter_session
2162
2163
2164 test_orbiter_session:
2165 ▷ $(ORBITER)
2166
2167
2168
2169
2170 #####
2171 # Section 2.2: Objects and Activities
2172
2173 SECTION_OBJECTS_AND_ACTIVITIES:
2174
2175 test_2.2:

```

```

2176
2177
2178
2179 #####
2180 # Section 2.3: Orbiter Dash Code Language
2181
2182 SECTION_ORBITER_DASH_CODE_LANGUAGE:
2183
2184 test_2.3:
2185   ▷ make object_F_2
2186   ▷ make object_PG_3_2
2187   ▷ make vector_ex
2188
2189
2190
2191 object_F_2:
2192   ▷ $(ORBITER) -v 3 -define F -finite_field -q 2 -end
2193
2194
2195 object_PG_3_2:
2196   ▷ $(ORBITER) \
2197   ▷ ▷ -define F -finite_field -q 2 -end \
2198   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end
2199
2200
2201 vector_ex:
2202   ▷ $(ORBITER) -v 2 \
2203   ▷ ▷ -define F -finite_field -q 5 -end \
2204   ▷ ▷ -define v -vector -field F -dense "0,1,2,3,4" -end \
2205   ▷ ▷ -print_symbols
2206
2207
2208
2209
2210
2211 #####
2212 # Section 2.4: Set Builder
2213
2214 SECTION_SET_BUILDER:
2215
2216
2217 test_2.4:
2218   ▷ make set_of_primes
2219   ▷ make set_interval
2220
2221
2222
2223
2224 set_of_primes:
2225   ▷ $(ORBITER) -v 2 \
2226   ▷ ▷ -define S -set -here "2,3,5,7,11,13" -end \
2227   ▷ ▷ -print_symbols
2228
2229 set_interval:
2230   ▷ $(ORBITER) -v 2 -define S -set -loop 0 64 1 -end \
2231   ▷ ▷ -print_symbols
2232
2233
2234

```

```

2235
2236 #####
2237 # Section 2.5: Vector Builder
2238
2239 SECTION.VECTOR.BUILDER:
2240
2241
2242 test_2.5:
2243   ▷ make vector_example1
2244   ▷ make vector_example2
2245   ▷ make vector_example_repeat
2246   ▷ make vector_example_all_one_11
2247   ▷ make vector_loop_8
2248   ▷ make vector_loop_odd_16
2249   ▷ make vector_concatenate
2250   ▷ make matrix_example1
2251   ▷ make vector_example_sparse
2252   ▷ make matrix_example2
2253   ▷ make matrix_example_co_1
2254   ▷ make vector_index_set
2255   ▷ make permutation_matrix
2256
2257
2258 vector_example1:
2259   ▷ $(ORBITER) -v 2 \
2260   ▷ ▷ -define F -finite_field -q 5 -end \
2261   ▷ ▷ -define v -vector -field F -dense "0,1,2,3,4" -end \
2262   ▷ ▷ -print_symbols
2263
2264
2265 vector_example2:
2266   ▷ $(ORBITER) -v 2 \
2267   ▷ ▷ -define F -finite_field -q 5 -end \
2268   ▷ ▷ -define v -vector -field F -format 2 -dense "0,1,2,3,4,0" -end \
2269   ▷ ▷ -print_symbols
2270
2271 vector_example_repeat:
2272   ▷ $(ORBITER) -v 2 \
2273   ▷ ▷ -define v -vector -repeat "0,1,2,3" 11 -end \
2274   ▷ ▷ -print_symbols
2275
2276
2277 vector_example_all_one_11:
2278   ▷ $(ORBITER) -v 2 \
2279   ▷ ▷ -define v -vector -repeat 1 11 -end \
2280   ▷ ▷ -print_symbols
2281
2282 vector_loop_8:
2283   ▷ $(ORBITER) -v 2 \
2284   ▷ ▷ -define v -vector -loop 0 8 1 -end \
2285
2286 vector_loop_odd_16:
2287   ▷ $(ORBITER) -v 2 \
2288   ▷ ▷ -define v -vector -loop 1 16 2 -end \
2289
2290
2291 vector_concatenate:
2292   ▷ $(ORBITER) -v 2 \
2293   ▷ ▷ -define a -vector -compact "1,2,3" -end \

```

```

2294 ▷ ▷ -define b -vector -compact "4,5,6" -end \
2295 ▷ ▷ -define c -vector -compact "7,8,9" -end \
2296 ▷ ▷ -define abc -vector -concatenate a,b,c -end
2297
2298
2299 matrix_example1:
2300 ▷ $(ORBITER) -v 2 \
2301 ▷ ▷ -define v -vector -format 2 \
2302 ▷ ▷ ▷ -dense "1,2,3,4,5,6" -end \
2303 ▷ ▷ -print_symbols
2304
2305
2306 vector_example_sparse:
2307 ▷ $(ORBITER) -v 2 \
2308 ▷ ▷ -define F -finite_field -q 5 -end \
2309 ▷ ▷ -define v -vector -field F -format 4 -sparse 20 "1,0,1,19" -end \
2310 ▷ ▷ -print_symbols
2311
2312
2313
2314 matrix_example2:
2315 ▷ $(ORBITER) -v 2 \
2316 ▷ ▷ -define F -finite_field -q 2 -end \
2317 ▷ ▷ -define v -vector -field F -format 4 \
2318 ▷ ▷ ▷ -dense $(HAMMING_CODE_GENERATOR) -end \
2319 ▷ ▷ -print_symbols
2320
2321
2322
2323
2324 matrix_example_co_1:
2325 ▷ $(ORBITER) -v 2 \
2326 ▷ ▷ -define F -finite_field -q 2 -end \
2327 ▷ ▷ -define v -vector -field F -format 22 \
2328 ▷ ▷ ▷ -compact $(CONWAY_GEN1) -end \
2329 ▷ ▷ -print_symbols
2330
2331
2332
2333
2334
2335 vector_index_set:
2336 ▷ $(ORBITER) -v 2 \
2337 ▷ ▷ -define v -vector -dense "0,1,0,1,0,1,0,1,0,1,0,1" -end \
2338 ▷ ▷ -define v2 -vector \
2339 ▷ ▷ ▷ -index_of_support \
2340 ▷ ▷ ▷ v \
2341 ▷ ▷ -end \
2342
2343
2344 #1, 3, 5, 7, 9, 11
2345
2346
2347
2348 permutation_matrix:
2349 ▷ $(ORBITER) -v 2 \
2350 ▷ ▷ -define P -vector -permutation_matrix "1,2,3,0" -end \
2351
2352

```

```

2353 #0 1 0 0
2354 #0 0 1 0
2355 #0 0 0 1
2356 #1 0 0 0
2357
2358
2359
2360 #####
2361 # Section 2.6: Vector of Group Elements Builder
2362
2363 SECTION_VECTOR_OF_GROUP_ELEMENTS_BUILDER:
2364
2365
2366 test_2.6:
2367   ▷ make Hirschfeld_Class_2A
2368   ▷ make surface_4_0_export_action
2369
2370
2371 Hirschfeld_Class_2A:
2372   ▷ $(ORBITER) -v 9 \
2373   ▷ ▷ -orbiter_path $(ORBITER_PATH) \
2374   ▷ ▷ -define G -linear_group -PGGL 4 4 \
2375   ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld.Stab" \
2376   ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
2377   ▷ ▷ ▷ -end \
2378   ▷ ▷ -with G -do \
2379   ▷ ▷ -group_theoretic_activity \
2380   ▷ ▷ ▷ -conjugacy_class_of "2A" $(CLASS_2A.REP) \
2381   ▷ ▷ -end
2382
2383
2384
2385
2386
2387 surface_4_0_export_action:
2388   ▷ $(ORBITER) -v 3 \
2389   ▷ ▷ -define Class2A -vector -file_column $(FILE_NAME_CLASS_2A) elements -end \
2390   ▷ ▷ -define F -finite_field -q 4 -end \
2391   ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
2392   ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
2393   ▷ ▷ -define G -linear_group -PGGL 4 4 -end \
2394   ▷ ▷ -define gens_ge -vector_ge \
2395   ▷ ▷ ▷ -action G \
2396   ▷ ▷ ▷ -vector_data Class2A \
2397   ▷ ▷ -end \
2398   ▷ ▷ -with S -do \
2399   ▷ ▷ -cubic_surface_activity \
2400   ▷ ▷ ▷ -export_something_with_group_element \
2401   ▷ ▷ ▷ "action_on_tritangent_planes" gens_ge \
2402   ▷ ▷ -end \
2403   ▷ ▷ -with S -do \
2404   ▷ ▷ -cubic_surface_activity \
2405   ▷ ▷ ▷ -export_something_with_group_element \
2406   ▷ ▷ ▷ "action_on_double_sixes" gens_ge \
2407   ▷ ▷ -end \
2408   ▷ ▷ -with S -do \
2409   ▷ ▷ -cubic_surface_activity \
2410   ▷ ▷ ▷ -export_something_with_group_element \
2411   ▷ ▷ ▷ "action_on_lines" gens_ge \

```



```

2412 ▷ ▷ -end
2413
2414
2415
2416
2417 #####
2418 # Section 2.7: Symbolic Objects
2419
2420 SECTION.SYMBOLIC.OBJECTS:
2421
2422 test_2.7:
2423 ▷ make symbolic_poly
2424 ▷ make symbolic_poly2
2425 ▷ make symbolic_CRC32
2426 ▷ make symbolic_CRC_alfa
2427 ▷ make symbolic_CRC_alfa.draw.trees
2428 ▷ make symbolic_CRC_bravo
2429 ▷ make symbolic_CRC.bravo.draw.trees
2430 ▷ make symbolic_test_4.expand
2431 ▷ make symbolic_test_4.expand.draw
2432 ▷ make symbolic_test_5.expand
2433 ▷ make test_expand_1
2434 ▷ make test_expand_2
2435 ▷ make test_expand_3
2436 ▷ make test_expand_4
2437 ▷ make symbolic_objects_building_on_earlier_objects
2438 ▷ make symbolic_objects_self_referential_1
2439 ▷ make symbolic_objects_self_referential_2
2440 ▷ make symbolic_test_6
2441 ▷ make symbolic_matrix1
2442 ▷ make symbolic_matrix_2x2
2443 ▷ make symbolic_matrix_3x3
2444 ▷ make symbolic_determinant_2x2
2445 ▷ make symbolic_determinant_3x3
2446 ▷ make symbolic_determinant_vandermonde
2447 ▷ make symbolic_characteristic_polynomial_2x2
2448 ▷ make symbolic_characteristic_polynomial_2x2.draw
2449 ▷ make symbolic_characteristic_polynomial_3x3
2450 ▷ make symbolic_object1
2451 ▷ make symbolic_object1.expand
2452 ▷ make symbolic_object1.draw
2453 ▷ make symbolic_object1.evaluate
2454 ▷ make symbolic_Eckardt_surface
2455 ▷ make symbolic_Eckardt_surface.q13.a3.b1
2456 ▷ make symbolic_Fabcd
2457 ▷ make symbolic_Fabcd.expand
2458 ▷ make symbolic_Fabcd.q31.E18
2459 ▷ make symbolic_Fabcd.q31.E18.with.implicit.substitution
2460 ▷ make symbolic_Fabcd.q31.E18.with.implicit.substitution.do.not.simplify
2461 ▷ make symbolic_matrix_minor
2462 ▷ make symbolic_matrix_nullspace_1
2463 ▷ make symbolic_matrix_nullspace_2
2464 ▷ make symbolic_Clebsch
2465 ▷ make Clebsch.map_up_Fabcd
2466 ▷ make Hirschfeld.evaluate_symbolic
2467
2468
2469
2470

```

```

2471
2472
2473 symbolic_poly:
2474 ▷ $(ORBITER) -v 3 \
2475 ▷ ▷ -define F -finite.field -q 7 -end \
2476 ▷ ▷ -define M -symbolic.object \
2477 ▷ ▷ ▷ -field F \
2478 ▷ ▷ ▷ -text "-X^2-X+1"\
2479 ▷ ▷ -end \
2480
2481 symbolic_poly2:
2482 ▷ $(ORBITER) -v 3 \
2483 ▷ ▷ -define F -finite.field -q 7 -end \
2484 ▷ ▷ -define M -symbolic.object \
2485 ▷ ▷ ▷ -field F \
2486 ▷ ▷ ▷ -text "X*Y*X"\
2487 ▷ ▷ -end
2488 ▷ $(PDFLATEX) sajeeb_tree0.tex
2489 ▷ $(OPEN) sajeeb_tree0.pdf
2490
2491
2492 symbolic_CRC32:
2493 ▷ $(ORBITER) -v 3 \
2494 ▷ ▷ -define F -finite.field -q 2 -end \
2495 ▷ ▷ -define M -symbolic.object \
2496 ▷ ▷ ▷ -field F \
2497 ▷ ▷ ▷ -text $(CRC32_ETHERNET_POLY) \
2498 ▷ ▷ -end \
2499 ▷ ▷ -define coeffs -symbolic.object \
2500 ▷ ▷ ▷ -field F \
2501 ▷ ▷ ▷ -collect M X \
2502 ▷ ▷ -end \
2503 ▷ ▷ -with coeffs -do -symbolic.object.activity \
2504 ▷ ▷ ▷ -print \
2505 ▷ ▷ -end
2506
2507
2508
2509 symbolic_CRC_alfa:
2510 ▷ $(ORBITER) -v 3 \
2511 ▷ ▷ -define F -finite.field -q 256 -end \
2512 ▷ ▷ -define M -symbolic.object \
2513 ▷ ▷ ▷ -field F \
2514 ▷ ▷ ▷ -text $(BCH_POLYNOMIAL_ALFA_F256) \
2515 ▷ ▷ -end \
2516
2517
2518
2519
2520 symbolic_CRC_alfa.draw_trees:
2521 ▷ $(ORBITER) -v 3 \
2522 ▷ ▷ -define F -finite.field -q 256 -end \
2523 ▷ ▷ -define M -symbolic.object \
2524 ▷ ▷ ▷ -field F \
2525 ▷ ▷ ▷ -text $(BCH_POLYNOMIAL_ALFA_F256) \
2526 ▷ ▷ -end \
2527 ▷ ▷ -define Mx -symbolic.object \
2528 ▷ ▷ ▷ -field F \
2529 ▷ ▷ ▷ -expand M \

```

```

2530 ▷ ▷ ▷ -write.trees.during.expand \
2531 ▷ ▷ -end
2532 ▷ ▷ $(PDFLATEX) Mx_0_efsxsfsfscsc.tex
2533 ▷ ▷ $(OPEN) Mx_0_efsxsfsfscsc.pdf
2534
2535
2536 symbolic_CRC_bravo:
2537 ▷ $(ORBITER) -v 3 \
2538 ▷ ▷ -define F -finite.field -q 256 -end \
2539 ▷ ▷ -define M -symbolic.object \
2540 ▷ ▷ ▷ -field F \
2541 ▷ ▷ ▷ -text $(BCH.POLYNOMIAL.BRAVO.F256) \
2542 ▷ ▷ -end \
2543
2544 symbolic_CRC_bravo_draw_trees:
2545 ▷ $(ORBITER) -v 3 \
2546 ▷ ▷ -define F -finite.field -q 256 -end \
2547 ▷ ▷ -define M -symbolic.object \
2548 ▷ ▷ ▷ -field F \
2549 ▷ ▷ ▷ -text $(BCH.POLYNOMIAL.BRAVO.F256) \
2550 ▷ ▷ -end \
2551 ▷ ▷ -define Mx -symbolic.object \
2552 ▷ ▷ ▷ -field F \
2553 ▷ ▷ ▷ -expand M \
2554 ▷ ▷ ▷ -write.trees.during.expand \
2555 ▷ ▷ -end
2556 ▷ ▷ $(PDFLATEX) Mx_0_efsxsfsfscsc.tex
2557 ▷ ▷ $(OPEN) Mx_0_efsxsfsfscsc.pdf
2558
2559
2560 symbolic_test_4_expand:
2561 ▷ $(ORBITER) -v 3 \
2562 ▷ ▷ -define F -finite.field -q 13 -end \
2563 ▷ ▷ -define M -symbolic.object \
2564 ▷ ▷ ▷ -field F \
2565 ▷ ▷ ▷ -matrix 1 \
2566 ▷ ▷ ▷ -text "(a+b)^2" \
2567 ▷ ▷ -end \
2568 ▷ ▷ -define M1 -symbolic.object \
2569 ▷ ▷ ▷ -field F \
2570 ▷ ▷ ▷ -expand M \
2571 ▷ ▷ ▷ -write.trees.during.expand \
2572 ▷ ▷ -end
2573
2574 symbolic_test_4_expand_draw:
2575 ▷ $(PDFLATEX) M.tex
2576 ▷ $(OPEN) M.pdf
2577 ▷ $(PDFLATEX) M1.tex
2578 ▷ $(OPEN) M1.pdf
2579 ▷ $(PDFLATEX) M1_0.tex
2580 ▷ $(OPEN) M1_0.pdf
2581 ▷ $(PDFLATEX) M1_0.e.tex
2582 ▷ $(OPEN) M1_0.e.pdf
2583 ▷ $(PDFLATEX) M1_0.ef.tex
2584 ▷ $(OPEN) M1_0.ef.pdf
2585 ▷ $(PDFLATEX) M1_0.efs.tex
2586 ▷ $(OPEN) M1_0.efs.pdf
2587 ▷ $(PDFLATEX) M1_0.efsx.tex
2588 ▷ $(OPEN) M1_0.efsx.pdf

```

```

2589 ▷ $(PDFLATEX) M1_0_efsxs.tex
2590 ▷ $(OPEN) M1_0_efsxs.pdf
2591 ▷ $(PDFLATEX) M1_0_efsxsf.tex
2592 ▷ $(OPEN) M1_0_efsxsf.pdf
2593 ▷ $(PDFLATEX) M1_0_efsxsfs.tex
2594 ▷ $(OPEN) M1_0_efsxsfs.pdf
2595 ▷ $(PDFLATEX) M1_0_efsxsfsfsc.tex
2596 ▷ $(OPEN) M1_0_efsxsfsfsc.pdf
2597 ▷ $(PDFLATEX) M1_0_efsxsfsfscs.tex
2598 ▷ $(OPEN) M1_0_efsxsfsfscs.pdf
2599 ▷ $(PDFLATEX) M1_0_efsxsfsfscsc.tex
2600 ▷ $(OPEN) M1_0_efsxsfsfscsc.pdf
2601
2602
2603 symbolic_test_5_expand:
2604 ▷ $(ORBITER) -v 3 \
2605 ▷ ▷ -define F -finite_field -q 13 -end \
2606 ▷ ▷ -define M -symbolic_object \
2607 ▷ ▷ ▷ -field F \
2608 ▷ ▷ ▷ -text "(a+b)^3" \
2609 ▷ ▷ -end \
2610 ▷ ▷ -define M1 -symbolic_object \
2611 ▷ ▷ ▷ -field F \
2612 ▷ ▷ ▷ -expand M \
2613 ▷ ▷ -end
2614 ▷ $(PDFLATEX) M.tex
2615 ▷ $(OPEN) M.pdf
2616 ▷ $(PDFLATEX) M1.tex
2617 ▷ $(OPEN) M1.pdf
2618
2619
2620
2621 test_expand_1:
2622 ▷ $(ORBITER) -v 3 \
2623 ▷ ▷ -define Fq -finite_field -q 31 -end \
2624 ▷ ▷ -define f -symbolic_object \
2625 ▷ ▷ ▷ -field Fq \
2626 ▷ ▷ ▷ -text "a*(x+y)" \
2627 ▷ ▷ -end \
2628 ▷ ▷ -define fe -symbolic_object \
2629 ▷ ▷ ▷ -field Fq \
2630 ▷ ▷ ▷ -expand f \
2631 ▷ ▷ ▷ -write_trees_during_expand \
2632 ▷ ▷ -end
2633 ▷ $(PDFLATEX) fe_0_efsxsfsfscsc.tex
2634 ▷ $(OPEN) fe_0_efsxsfsfscsc.pdf
2635
2636 test_expand_2:
2637 ▷ $(ORBITER) -v 3 \
2638 ▷ ▷ -define Fq -finite_field -q 31 -end \
2639 ▷ ▷ -define f -symbolic_object \
2640 ▷ ▷ ▷ -field Fq \
2641 ▷ ▷ ▷ -text "a*(-(x-y))" \
2642 ▷ ▷ -end \
2643 ▷ ▷ -define fe -symbolic_object \
2644 ▷ ▷ ▷ -field Fq \
2645 ▷ ▷ ▷ -expand f \
2646 ▷ ▷ ▷ -write_trees_during_expand \
2647 ▷ ▷ -end

```

```

2648 > $(PDFLATEX) fe_0_efsxsfsfscsc.tex
2649 > $(OPEN) fe_0_efsxsfsfscsc.pdf
2650
2651 test_expand_3:
2652 > $(ORBITER) -v 3 \
2653 > > -define Fq -finite_field -q 31 -end \
2654 > > -define f -symbolic_object \
2655 > > > -field Fq \
2656 > > > -text "a*(-(x-y)+(x+y))" \
2657 > > -end \
2658 > > -define fe -symbolic_object \
2659 > > > -field Fq \
2660 > > > -expand f \
2661 > > > -write_trees_during_expand \
2662 > > -end
2663 > $(PDFLATEX) fe_0_efs.tex
2664 > $(OPEN) fe_0_efs.pdf
2665 > $(PDFLATEX) fe_0_efsx.tex
2666 > $(OPEN) fe_0_efsx.pdf
2667 > $(PDFLATEX) fe_0_efsxsf.tex
2668 > $(OPEN) fe_0_efsxsf.pdf
2669 > $(PDFLATEX) fe_0_efsxsfs.tex
2670 > $(OPEN) fe_0_efsxsfs.pdf
2671 > $(PDFLATEX) fe_0_efsxsfsfsc.tex
2672 > $(OPEN) fe_0_efsxsfsfsc.pdf
2673 > $(PDFLATEX) fe_0_efsxsfsfscsefsx.tex
2674 > $(OPEN) fe_0_efsxsfsfscsefsx.pdf
2675 > $(PDFLATEX) fe_0_efsxsfsfscsefsxsfsf.tex
2676 > $(OPEN) fe_0_efsxsfsfscsefsxsfsf.pdf
2677 > $(PDFLATEX) fe_0_efsxsfsfscsefsxsfsfscsc.tex
2678 > $(OPEN) fe_0_efsxsfsfscsefsxsfsfscsc.pdf
2679
2680
2681 test_expand_4:
2682 > $(ORBITER) -v 3 \
2683 > > -define Fq -finite_field -q 31 -end \
2684 > > -define f -symbolic_object \
2685 > > > -field Fq \
2686 > > > -text "x*(a+(x*(a+(x*(a+x*(a+x*(a+x*(a+x))))))))" \
2687 > > -end \
2688 > > -define fx -symbolic_object \
2689 > > > -field Fq \
2690 > > > -expand f \
2691 > > > -write_trees_during_expand \
2692 > > -end
2693 > $(PDFLATEX) fx_0_efsxsfsfscsefsxsfsfscsefsxsfsfscsefsxsfsfscsefsxsfsfscsc.tex
2694 > $(OPEN) fx_0_efsxsfsfscsefsxsfsfscsefsxsfsfscsefsxsfsfscsefsxsfsfscsc.pdf
2695
2696
2697 symbolic_objects_building_on_earlier_objects:
2698 > $(ORBITER) -v 3 \
2699 > > -define F -finite_field -q 2 -end \
2700 > > -define a -symbolic_object \
2701 > > > -field F \
2702 > > > -text "x+y" \
2703 > > -end \
2704 > > -define b -symbolic_object \
2705 > > > -field F \
2706 > > > -text "a+z" \

```

```

2707 ▷ ▷ -end
2708
2709
2710 symbolic_objects_self_referential_1:
2711 ▷ $(ORBITER) -v 3 \
2712 ▷ ▷ -define F -finite_field -q 2 -end \
2713 ▷ ▷ -define x -symbolic_object \
2714 ▷ ▷ ▷ -field F \
2715 ▷ ▷ ▷ -text "x" \
2716 ▷ ▷ -end
2717
2718 symbolic_objects_self_referential_2:
2719 ▷ $(ORBITER) -v 3 \
2720 ▷ ▷ -define F -finite_field -q 2 -end \
2721 ▷ ▷ -define x -symbolic_object \
2722 ▷ ▷ ▷ -field F \
2723 ▷ ▷ ▷ -text "x" \
2724 ▷ ▷ -end \
2725 ▷ ▷ -define a -symbolic_object \
2726 ▷ ▷ ▷ -field F \
2727 ▷ ▷ ▷ -text "x+a" \
2728 ▷ ▷ -end
2729
2730
2731 symbolic_test_6:
2732 ▷ $(ORBITER) -v 3 \
2733 ▷ ▷ -define F -finite_field -q 13 -end \
2734 ▷ ▷ -define a -symbolic_object \
2735 ▷ ▷ ▷ -field F \
2736 ▷ ▷ ▷ -text "x" \
2737 ▷ ▷ -end \
2738 ▷ ▷ -define a -symbolic_object \
2739 ▷ ▷ ▷ -field F \
2740 ▷ ▷ ▷ -text "a^2" \
2741 ▷ ▷ -end \
2742 ▷ ▷ -define a -symbolic_object \
2743 ▷ ▷ ▷ -field F \
2744 ▷ ▷ ▷ -text "a^2" \
2745 ▷ ▷ -end \
2746 ▷ ▷ -define a -symbolic_object \
2747 ▷ ▷ ▷ -field F \
2748 ▷ ▷ ▷ -text "a^2" \
2749 ▷ ▷ -end
2750 ▷ $(PDFLATEX) a.tex
2751 ▷ $(OPEN) a.pdf
2752
2753
2754
2755 symbolic_test_7:
2756 ▷ $(ORBITER) -v 10 \
2757 ▷ ▷ -define Fq -finite_field -q 13 -end \
2758 ▷ ▷ -define M -symbolic_object \
2759 ▷ ▷ ▷ -field Fq \
2760 ▷ ▷ ▷ -text "a*b^-1" \
2761 ▷ ▷ -end \
2762 ▷ ▷ -define values -symbolic_object \
2763 ▷ ▷ ▷ -field Fq \
2764 ▷ ▷ ▷ -text "1,3" \
2765 ▷ ▷ -end \

```

```

2766 ▷ ▷ -define eval -symbolic_object \
2767 ▷ ▷ ▷ -field Fq \
2768 ▷ ▷ ▷ -substitute "a,b" M values \
2769 ▷ ▷ -end
2770 ▷ #$(PDFLATEX) M.tex
2771 ▷ #$(OPEN) M.pdf
2772
2773
2774
2775 symbolic_matrix1:
2776 ▷ $(ORBITER) -v 3 \
2777 ▷ ▷ -define F -finite_field -q 13 -end \
2778 ▷ ▷ -define M -symbolic_object \
2779 ▷ ▷ ▷ -field F \
2780 ▷ ▷ ▷ -matrix 4 \
2781 ▷ ▷ ▷ -text $(VANDERMONDE_4X4_FORMULA) \
2782 ▷ ▷ -end
2783
2784
2785 symbolic_matrix_2x2:
2786 ▷ $(ORBITER) -v 3 \
2787 ▷ ▷ -define F -finite_field -q 13 -end \
2788 ▷ ▷ -define M -symbolic_object \
2789 ▷ ▷ ▷ -field F \
2790 ▷ ▷ ▷ -matrix 2 \
2791 ▷ ▷ ▷ -text "a,b,c,d" \
2792 ▷ ▷ -end
2793
2794
2795 symbolic_matrix_3x3:
2796 ▷ $(ORBITER) -v 3 \
2797 ▷ ▷ -define F -finite_field -q 13 -end \
2798 ▷ ▷ -define M -symbolic_object \
2799 ▷ ▷ ▷ -field F \
2800 ▷ ▷ ▷ -matrix 3 \
2801 ▷ ▷ ▷ -text "a,b,c,d,e,f,g,h,i" \
2802 ▷ ▷ -end
2803
2804
2805 symbolic_determinant_2x2:
2806 ▷ $(ORBITER) -v 3 \
2807 ▷ ▷ -define F -finite_field -q 13 -end \
2808 ▷ ▷ -define M -symbolic_object \
2809 ▷ ▷ ▷ -field F \
2810 ▷ ▷ ▷ -matrix 2 \
2811 ▷ ▷ ▷ -text "a,b,c,d" \
2812 ▷ ▷ -end \
2813 ▷ ▷ -define det -symbolic_object \
2814 ▷ ▷ ▷ -field F \
2815 ▷ ▷ ▷ -determinant M \
2816 ▷ ▷ -end \
2817 ▷ ▷ -define det1 -symbolic_object \
2818 ▷ ▷ ▷ -field F \
2819 ▷ ▷ ▷ -expand det \
2820 ▷ ▷ -end
2821 ▷ #$(PDFLATEX) det1.tex
2822 ▷ #$(OPEN) det1.pdf
2823
2824

```

```

2825 symbolic_determinant_3x3:
2826 ▷ $(ORBITER) -v 3 \
2827 ▷ ▷ -define F -finite.field -q 13 -end \
2828 ▷ ▷ -define M -symbolic.object \
2829 ▷ ▷ ▷ -field F \
2830 ▷ ▷ ▷ -matrix 3 \
2831 ▷ ▷ ▷ -text "a,b,c,d,e,f,g,h,i" \
2832 ▷ ▷ -end \
2833 ▷ ▷ -define det -symbolic.object \
2834 ▷ ▷ ▷ -field F \
2835 ▷ ▷ ▷ -determinant M \
2836 ▷ ▷ -end \
2837 ▷ ▷ -define det1 -symbolic.object \
2838 ▷ ▷ ▷ -field F \
2839 ▷ ▷ ▷ -expand det \
2840 ▷ ▷ -end
2841 ▷ $(PDFLATEX) det1.tex
2842 ▷ $(OPEN) det1.pdf
2843
2844
2845 symbolic_determinant_vandermonde:
2846 ▷ $(ORBITER) -v 3 \
2847 ▷ ▷ -define F -finite.field -q 13 -end \
2848 ▷ ▷ -define M -symbolic.object \
2849 ▷ ▷ ▷ -field F \
2850 ▷ ▷ ▷ -matrix 4 \
2851 ▷ ▷ ▷ -text $(VANDERMONDE_4X4_FORMULA) \
2852 ▷ ▷ -end \
2853 ▷ ▷ -define det -symbolic.object \
2854 ▷ ▷ ▷ -field F \
2855 ▷ ▷ ▷ -determinant M \
2856 ▷ ▷ -end \
2857 ▷ ▷ -define det1 -symbolic.object \
2858 ▷ ▷ ▷ -field F \
2859 ▷ ▷ ▷ -expand det \
2860 ▷ ▷ -end
2861
2862
2863 symbolic_characteristic_polynomial_2x2:
2864 ▷ $(ORBITER) -v 3 \
2865 ▷ ▷ -define F -finite.field -q 13 -end \
2866 ▷ ▷ -define M -symbolic.object \
2867 ▷ ▷ ▷ -field F \
2868 ▷ ▷ ▷ -matrix 2 \
2869 ▷ ▷ ▷ -text "a,b,c,d" \
2870 ▷ ▷ -end \
2871 ▷ ▷ -define p -symbolic.object \
2872 ▷ ▷ ▷ -field F \
2873 ▷ ▷ ▷ -characteristic.polynomial lambda M \
2874 ▷ ▷ -end \
2875 ▷ ▷ -define p1 -symbolic.object \
2876 ▷ ▷ ▷ -field F \
2877 ▷ ▷ ▷ -expand p \
2878 ▷ ▷ ▷ -write.trees.during.expand \
2879 ▷ ▷ -end \
2880 ▷ ▷ -define coeffs -symbolic.object \
2881 ▷ ▷ ▷ -field F \
2882 ▷ ▷ ▷ -collect p1 lambda \
2883 ▷ ▷ -end

```



```

2884
2885 symbolic_characteristic_polynomial_2x2_draw:
2886 ▷ $(PDFLATEX) p1.tex
2887 ▷ $(OPEN) p1.pdf
2888 ▷ $(PDFLATEX) p1_0.tex
2889 ▷ $(OPEN) p1_0.pdf
2890 ▷ $(PDFLATEX) p1_0_e.tex
2891 ▷ $(OPEN) p1_0_e.pdf
2892 ▷ $(PDFLATEX) p1_0_ef.tex
2893 ▷ $(OPEN) p1_0_ef.pdf
2894 ▷ $(PDFLATEX) p1_0_efs.tex
2895 ▷ $(OPEN) p1_0_efs.pdf
2896 ▷ $(PDFLATEX) p1_0_efsx.tex
2897 ▷ $(OPEN) p1_0_efsx.pdf
2898 ▷ $(PDFLATEX) p1_0_efsxs.tex
2899 ▷ $(OPEN) p1_0_efsxs.pdf
2900 ▷ $(PDFLATEX) p1_0_efsxsf.tex
2901 ▷ $(OPEN) p1_0_efsxsf.pdf
2902 ▷ $(PDFLATEX) p1_0_efsxsfs.tex
2903 ▷ $(OPEN) p1_0_efsxsfs.pdf
2904 ▷ $(PDFLATEX) p1_0_efsxsfsc.tex
2905 ▷ $(OPEN) p1_0_efsxsfsc.pdf
2906 ▷ $(PDFLATEX) p1_0_efsxsfscs.tex
2907 ▷ $(OPEN) p1_0_efsxsfscs.pdf
2908 ▷ $(PDFLATEX) p1_0_efsxsfscsc.tex
2909 ▷ $(OPEN) p1_0_efsxsfscsc.pdf
2910
2911 symbolic_characteristic_polynomial_3x3:
2912 ▷ $(ORBITER) -v 3 \
2913 ▷ ▷ -define F -finite_field -q 13 -end \
2914 ▷ ▷ -define M -symbolic_object \
2915 ▷ ▷ ▷ -field F \
2916 ▷ ▷ ▷ -matrix 3 \
2917 ▷ ▷ ▷ -text "a,b,c,d,e,f,g,h,i" \
2918 ▷ ▷ -end \
2919 ▷ ▷ -define p -symbolic_object \
2920 ▷ ▷ ▷ -field F \
2921 ▷ ▷ ▷ -characteristic_polynomial lambda M \
2922 ▷ ▷ -end \
2923 ▷ ▷ -define p1 -symbolic_object \
2924 ▷ ▷ ▷ -field F \
2925 ▷ ▷ ▷ -expand p \
2926 ▷ ▷ -end \
2927 ▷ ▷ -define coeffs -symbolic_object \
2928 ▷ ▷ ▷ -field F \
2929 ▷ ▷ ▷ -collect p1 lambda \
2930 ▷ ▷ -end
2931 ▷ #$(PDFLATEX) p1.tex
2932 ▷ #$(OPEN) p1.pdf
2933
2934
2935
2936 symbolic_object1:
2937 ▷ $(ORBITER) -v 3 \
2938 ▷ ▷ -define F -finite_field -q 13 -end \
2939 ▷ ▷ -define M -symbolic_object \
2940 ▷ ▷ ▷ -field F \
2941 ▷ ▷ ▷ -text "a+2*b^2*(x-y)+3*-(2*c^4+3*d-(a*b*c)^2)^3" \
2942 ▷ ▷ -end

```

```

2943
2944
2945 symbolic_object1_expand:
2946 ▷ $(ORBITER) -v 3 \
2947 ▷ ▷ -define F -finite_field -q 13 -end \
2948 ▷ ▷ -define M -symbolic_object \
2949 ▷ ▷ ▷ -field F \
2950 ▷ ▷ ▷ -text "a+2*b^2*(x-y)+3*-(2*c^4+3*d-(a*b*c)^2)^3" \
2951 ▷ ▷ -end \
2952 ▷ ▷ -define M1 -symbolic_object \
2953 ▷ ▷ ▷ -field F \
2954 ▷ ▷ ▷ -expand M \
2955 ▷ ▷ -end
2956 ▷ $(PDFLATEX) M1.tex
2957 ▷ $(OPEN) M1.pdf
2958
2959
2960
2961 symbolic_object1_draw:
2962 ▷ $(ORBITER) -v 3 \
2963 ▷ ▷ -define F -finite_field -q 13 -end \
2964 ▷ ▷ -define M -symbolic_object \
2965 ▷ ▷ ▷ -field F \
2966 ▷ ▷ ▷ -text "a+2*b^2*(x-y)+3*-(2*c^4+3*d-(a*b*c)^2)^3" \
2967 ▷ ▷ -end
2968 ▷ $(PDFLATEX) sajeeb_tree0.tex
2969 ▷ $(OPEN) sajeeb_tree0.pdf
2970 ▷ $(PDFLATEX) M.tex
2971 ▷ $(OPEN) M.pdf
2972
2973
2974
2975 symbolic_object1_evaluate:
2976 ▷ $(ORBITER) -v 3 \
2977 ▷ ▷ -define Fq -finite_field -q 13 -end \
2978 ▷ ▷ -define f -symbolic_object \
2979 ▷ ▷ ▷ -field Fq \
2980 ▷ ▷ ▷ -text "a+2*b^2*(x-y)+3*-(2*c^4+3*d-(a*b*c)^2)^3" \
2981 ▷ ▷ -end \
2982 ▷ ▷ -define assignment -symbolic_object \
2983 ▷ ▷ ▷ -field Fq \
2984 ▷ ▷ ▷ -text "1,2,3,4,5,6" \
2985 ▷ ▷ -end \
2986 ▷ ▷ -define eval -symbolic_object \
2987 ▷ ▷ ▷ -field Fq \
2988 ▷ ▷ ▷ -substitute "a,b,c,d,x,y" f assignment \
2989 ▷ ▷ -end
2990
2991
2992
2993
2994
2995 symbolic_Eckardt_surface:
2996 ▷ $(ORBITER) -v 3 \
2997 ▷ ▷ -define F -finite_field -q 13 -end \
2998 ▷ ▷ -define M -symbolic_object \
2999 ▷ ▷ ▷ -field F \
3000 ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
3001 ▷ ▷ ▷ -text $(ECKARDT_SURFACE_EQN) \

```

```

3002 ▷ ▷ -end
3003
3004
3005
3006 symbolic_Eckardt_surface.q13.a3.b1:
3007 ▷ $(ORBITER) -v 3 \
3008 ▷ ▷ -define F -finite_field -q 13 -end \
3009 ▷ ▷ -define M -symbolic_object \
3010 ▷ ▷ ▷ -field F \
3011 ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
3012 ▷ ▷ ▷ -text $(ECKARDT_SURFACE_EQN) \
3013 ▷ ▷ -end \
3014 ▷ ▷ -define L -symbolic_object \
3015 ▷ ▷ ▷ -field F \
3016 ▷ ▷ ▷ -text "3,1" \
3017 ▷ ▷ -end \
3018 ▷ ▷ -define M1 -symbolic_object \
3019 ▷ ▷ ▷ -field F \
3020 ▷ ▷ ▷ -substitute "a,b" M L \
3021 ▷ ▷ -end \
3022 ▷ ▷ -define M2 -symbolic_object \
3023 ▷ ▷ ▷ -field F \
3024 ▷ ▷ ▷ -expand M1 \
3025 ▷ ▷ ▷ -write_trees_during_expand \
3026 ▷ ▷ -end
3027 ▷ $(PDFLATEX) M2.tex
3028 ▷ $(OPEN) M2.pdf
3029
3030
3031 symbolic_Fabcd:
3032 ▷ $(ORBITER) -v 3 \
3033 ▷ ▷ -define F -finite_field -q 13 -end \
3034 ▷ ▷ -define M -symbolic_object \
3035 ▷ ▷ ▷ -field F \
3036 ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
3037 ▷ ▷ ▷ -text $(F_abcd_eqn) \
3038 ▷ ▷ -end
3039
3040
3041 symbolic_Fabcd.expand:
3042 ▷ $(ORBITER) -v 3 \
3043 ▷ ▷ -define Fq -finite_field -q 31 -end \
3044 ▷ ▷ -define Fabcd -symbolic_object \
3045 ▷ ▷ ▷ -field Fq \
3046 ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
3047 ▷ ▷ ▷ -text $(F_abcd_eqn) \
3048 ▷ ▷ -end \
3049 ▷ ▷ -define Fabcd_e -symbolic_object \
3050 ▷ ▷ ▷ -field Fq \
3051 ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
3052 ▷ ▷ ▷ -expand Fabcd \
3053 ▷ ▷ ▷ -write_trees_during_expand \
3054 ▷ ▷ -end
3055
3056
3057 # q=31
3058 #E=18 (a,b,c,d) = (25,5,5,25)
3059 #E=10 (a,b,c,d) = (14,13,13,14)
3060 #E=9 (a,b,c,d) = (5,3,12,5)

```

```

3061 #E=6 (a,b,c,d) = (2,30,30,2)
3062 #E=4 (a,b,c,d) = (4,2,2,4)
3063 #E=3 (a,b,c,d) = (6,2,3,6)
3064 #E=2 (a,b,c,d) = (2,3,3,2)
3065 #E=1 (a,b,c,d) = (2,3,5,2)
3066 #E=0 (a,b,c,d) = (2,5,9,7)
3067
3068
3069 symbolic_Fabcd.q31_E18:
3070 ▷ $(ORBITER) -v 3 \
3071 ▷ ▷ -define F -finite.field -q 31 -end \
3072 ▷ ▷ -define M -symbolic.object \
3073 ▷ ▷ ▷ -field F \
3074 ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
3075 ▷ ▷ ▷ -text $(F_abcd_eqn) \
3076 ▷ ▷ -end \
3077 ▷ ▷ -define L -symbolic.object \
3078 ▷ ▷ ▷ -field F \
3079 ▷ ▷ ▷ -text "25,5,5,25" \
3080 ▷ ▷ -end \
3081 ▷ ▷ -define M1 -symbolic.object \
3082 ▷ ▷ ▷ -field F \
3083 ▷ ▷ ▷ -substitute "a,b,c,d" M L \
3084 ▷ ▷ -end \
3085 ▷ ▷ -define M2 -symbolic.object \
3086 ▷ ▷ ▷ -field F \
3087 ▷ ▷ ▷ -expand M1 \
3088 ▷ ▷ ▷ -write_trees.during.expand \
3089 ▷ ▷ -end
3090 ▷ $(PDFLATEX) M2.tex
3091 ▷ $(OPEN) M2.pdf
3092
3093
3094 #(13 X1 X2 X3) + (22 X1 X2^{2}) + (22 X1^{2} X3) + (22 X1^{2} X2) + (9 X0 X2^{2})
+ (9 X0^{2} X2) + (22 X1 X3^{2})\\
3095
3096
3097
3098 symbolic_Fabcd.q31_E18_with_implicit_substitution:
3099 ▷ $(ORBITER) -v 3 \
3100 ▷ ▷ -define F -finite.field -q 31 -end \
3101 ▷ ▷ -define a -symbolic.object \
3102 ▷ ▷ ▷ -field F \
3103 ▷ ▷ ▷ -text "25" \
3104 ▷ ▷ -end \
3105 ▷ ▷ -define b -symbolic.object \
3106 ▷ ▷ ▷ -field F \
3107 ▷ ▷ ▷ -text "5" \
3108 ▷ ▷ -end \
3109 ▷ ▷ -define c -symbolic.object \
3110 ▷ ▷ ▷ -field F \
3111 ▷ ▷ ▷ -text "5" \
3112 ▷ ▷ -end \
3113 ▷ ▷ -define d -symbolic.object \
3114 ▷ ▷ ▷ -field F \
3115 ▷ ▷ ▷ -text "25" \
3116 ▷ ▷ -end \
3117 ▷ ▷ -define M -symbolic.object \
3118 ▷ ▷ ▷ -field F \

```

```

3119 ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
3120 ▷ ▷ ▷ -text $(F_abcd_eqn) \
3121 ▷ ▷ -end \
3122 ▷ ▷ -define M2 -symbolic_object \
3123 ▷ ▷ ▷ -field F \
3124 ▷ ▷ ▷ -expand M \
3125 ▷ ▷ -end
3126 ▷ $(PDFLATEX) M2.tex
3127 ▷ $(OPEN) M2.pdf
3128
3129
3130
3131 symbolic_Fabcd.q31_E18_with_implicit_substitution.do_not_simplify:
3132 ▷ $(ORBITER) -v 3 \
3133 ▷ ▷ -define F -finite_field -q 31 -end \
3134 ▷ ▷ -define a -symbolic_object \
3135 ▷ ▷ ▷ -field F \
3136 ▷ ▷ ▷ -text "25" \
3137 ▷ ▷ -end \
3138 ▷ ▷ -define b -symbolic_object \
3139 ▷ ▷ ▷ -field F \
3140 ▷ ▷ ▷ -text "5" \
3141 ▷ ▷ -end \
3142 ▷ ▷ -define c -symbolic_object \
3143 ▷ ▷ ▷ -field F \
3144 ▷ ▷ ▷ -text "5" \
3145 ▷ ▷ -end \
3146 ▷ ▷ -define d -symbolic_object \
3147 ▷ ▷ ▷ -field F \
3148 ▷ ▷ ▷ -text "25" \
3149 ▷ ▷ -end \
3150 ▷ ▷ -define M -symbolic_object \
3151 ▷ ▷ ▷ -field F \
3152 ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
3153 ▷ ▷ ▷ -text $(F_abcd_eqn) \
3154 ▷ ▷ ▷ -do_not_simplify \
3155 ▷ ▷ -end
3156
3157
3158
3159
3160 symbolic_matrix_minor:
3161 ▷ $(ORBITER) -v 3 \
3162 ▷ ▷ -define F -finite_field -q 13 -end \
3163 ▷ ▷ -define A -symbolic_object \
3164 ▷ ▷ ▷ -field F \
3165 ▷ ▷ ▷ -matrix 4 \
3166 ▷ ▷ ▷ -text "1,0,0,0,0,0,0,1,x0,x1,x2,x3,a,b,c,d" \
3167 ▷ ▷ -end \
3168 ▷ ▷ -define M -symbolic_object \
3169 ▷ ▷ ▷ -field F \
3170 ▷ ▷ ▷ -minor A 3 1 \
3171 ▷ ▷ -end
3172
3173
3174 symbolic_matrix_nullspace_1:
3175 ▷ $(ORBITER) -v 3 \
3176 ▷ ▷ -define F -finite_field -q 13 -end \
3177 ▷ ▷ -define A1 -symbolic_object \

```

```

3178 ▷ ▷ ▷ -field F \
3179 ▷ ▷ ▷ -matrix 3 \
3180 ▷ ▷ ▷ -text "1,0,0,0,0,0,0,1,y0,y1,y2,0" \
3181 ▷ ▷ -end \
3182 ▷ ▷ -define M1 -symbolic.object \
3183 ▷ ▷ ▷ -field F \
3184 ▷ ▷ ▷ -symbolic.nullspace A1 \
3185 ▷ ▷ -end
3186
3187 #0, ((12) * (y2)), (y1), 0
3188
3189
3190 symbolic_matrix_nullspace_2:
3191 ▷ $(ORBITER) -v 3 \
3192 ▷ ▷ -define F -finite.field -q 13 -end \
3193 ▷ ▷ -define A2 -symbolic.object \
3194 ▷ ▷ ▷ -field F \
3195 ▷ ▷ ▷ -matrix 3 \
3196 ▷ ▷ ▷ -text "0,1,0,0,0,0,-1,1,y0,y1,y2,0" \
3197 ▷ ▷ -end \
3198 ▷ ▷ -define M2 -symbolic.object \
3199 ▷ ▷ ▷ -field F \
3200 ▷ ▷ ▷ -symbolic.nullspace A2 \
3201 ▷ ▷ -end
3202
3203
3204 #(y2), 0, (12) * (y0), ((12) * (y0))
3205
3206 symbolic_Clebsch:
3207 ▷ $(ORBITER) -v 3 \
3208 ▷ ▷ -define F -finite.field -q 13 -end \
3209 ▷ ▷ -define A1 -symbolic.object \
3210 ▷ ▷ ▷ -field F \
3211 ▷ ▷ ▷ -matrix 3 \
3212 ▷ ▷ ▷ -text "1,0,0,0,0,0,0,1,y0,y1,y2,0" \
3213 ▷ ▷ -end \
3214 ▷ ▷ -define A2 -symbolic.object \
3215 ▷ ▷ ▷ -field F \
3216 ▷ ▷ ▷ -matrix 3 \
3217 ▷ ▷ ▷ -text "0,1,0,0,0,0,-1,1,y0,y1,y2,0" \
3218 ▷ ▷ -end \
3219 ▷ ▷ -define A3 -symbolic.object \
3220 ▷ ▷ ▷ -field F \
3221 ▷ ▷ ▷ -matrix 2 \
3222 ▷ ▷ ▷ -text "0,0,1,0,0,0,0,1" \
3223 ▷ ▷ -end \
3224 ▷ ▷ -define M1 -symbolic.object \
3225 ▷ ▷ ▷ -field F \
3226 ▷ ▷ ▷ -symbolic.nullspace A1 \
3227 ▷ ▷ -end \
3228 ▷ ▷ -define M2 -symbolic.object \
3229 ▷ ▷ ▷ -field F \
3230 ▷ ▷ ▷ -symbolic.nullspace A2 \
3231 ▷ ▷ -end \
3232 ▷ ▷ -define L -symbolic.object \
3233 ▷ ▷ ▷ -field F \
3234 ▷ ▷ ▷ -stack_matrices_vertically M1,M2,A3 \
3235 ▷ ▷ -end
3236

```

```

3237
3238
3239
3240
3241 Clebsch_map_up_Fabcd:
3242 ▷ $(ORBITER) -v 3 \
3243 ▷ ▷ -define F -finite_field -q 13 -end \
3244 ▷ ▷ -define M -symbolic_object \
3245 ▷ ▷ ▷ -field F \
3246 ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
3247 ▷ ▷ ▷ -text $(F_abcd_eqn) \
3248 ▷ ▷ -end \
3249 ▷ ▷ -define L -symbolic_object \
3250 ▷ ▷ ▷ -field F \
3251 ▷ ▷ ▷ -text "y0+t*y0,t*y1,t*y2,y2" \
3252 ▷ ▷ -end \
3253 ▷ ▷ -define C1 -symbolic_object \
3254 ▷ ▷ ▷ -field F \
3255 ▷ ▷ ▷ -managed_variables "y0,y1,y2" \
3256 ▷ ▷ ▷ -substitute "X0,X1,X2,X3" M L \
3257 ▷ ▷ -end \
3258 ▷ ▷ -define M -symbolic_object \
3259 ▷ ▷ ▷ -field F \
3260 ▷ ▷ ▷ -expand C1 \
3261 ▷ ▷ ▷ -managed_variables "y0,y1,y2" \
3262 ▷ ▷ ▷ -write_trees_during_expand \
3263 ▷ ▷ -end \
3264 ▷ ▷ -define coeff_t -symbolic_object \
3265 ▷ ▷ ▷ -field F \
3266 ▷ ▷ ▷ -managed_variables "y0,y1,y2" \
3267 ▷ ▷ ▷ -collect M t \
3268 ▷ ▷ -end
3269
3270
3271 Hirschfeld_evaluate_symbolic:
3272 ▷ $(ORBITER) -v 3 \
3273 ▷ ▷ -define F -finite_field -q 4 -end \
3274 ▷ ▷ -define M -symbolic_object \
3275 ▷ ▷ ▷ -field F \
3276 ▷ ▷ ▷ -text "X0*X1^2+X0^2*X1+X2*X3^2+X2^2*X3" \
3277 ▷ ▷ -end \
3278 ▷ ▷ -define Mx -symbolic_object \
3279 ▷ ▷ ▷ -field F \
3280 ▷ ▷ ▷ -expand M \
3281 ▷ ▷ -end \
3282 ▷ ▷ -with Mx -do -symbolic_object_activity \
3283 ▷ ▷ ▷ -evaluate_affine \
3284 ▷ ▷ -end
3285
3286
3287
3288 #####
3289 # Chapter 3 - Finite Fields
3290 #####
3291
3292
3293 test_3:
3294 ▷ make test_3.1
3295 ▷ make test_3.2

```

```

3296 ▷ make test_3.3
3297 ▷ make test_3.4
3298
3299
3300
3301
3302 #####
3303 # Section 3.1: Prime Fields
3304
3305 SECTION.PRIME.FIELDS:
3306
3307
3308 test_3.1:
3309 ▷ make F_2
3310 ▷ make F_3
3311 ▷ make F_5
3312 ▷ make F_5_add_table
3313 ▷ make F_7
3314 ▷ make F_7_tables
3315 ▷ make F_13
3316 ▷ make F_19
3317 ▷ make F_127
3318 ▷ make F_11_product_of_all_nonzero_elements
3319 ▷ make F_7_vandermonde
3320 ▷ make F_101_wo
3321 ▷ make F_1021_wo
3322
3323
3324
3325 F_2:
3326 ▷ $(ORBITER) -v 3 -list_arguments \
3327 ▷ ▷ -define F -finite_field -q 2 -end \
3328 ▷ ▷ -with F -do -finite_field_activity \
3329 ▷ ▷ ▷ -cheat_sheet_GF \
3330 ▷ ▷ -end
3331 ▷ $(PDFLATEX) GF_2.tex
3332 ▷ $(OPEN) GF_2.pdf
3333
3334
3335 F_3:
3336 ▷ $(ORBITER) -v 3 \
3337 ▷ ▷ -define F -finite_field -q 3 -end \
3338 ▷ ▷ -with F -do -finite_field_activity \
3339 ▷ ▷ ▷ -cheat_sheet_GF \
3340 ▷ ▷ -end
3341 ▷ $(PDFLATEX) GF_3.tex
3342 ▷ $(OPEN) GF_3.pdf
3343
3344 F_5:
3345 ▷ $(ORBITER) -v 3 \
3346 ▷ ▷ -define F -finite_field -q 5 -end \
3347 ▷ ▷ -with F -do -finite_field_activity \
3348 ▷ ▷ ▷ -cheat_sheet_GF \
3349 ▷ ▷ -end
3350 ▷ $(PDFLATEX) GF_5.tex
3351 ▷ $(OPEN) GF_5.pdf
3352
3353
3354 F_5_add_table:

```



```

3355 ▷ $(ORBITER) -v 3 \
3356 ▷ ▷ -define F -finite_field -q 5 -end \
3357 ▷ ▷ -with F -do -finite_field_activity -export_tables -end \
3358 ▷ ▷ -draw_matrix \
3359 ▷ ▷ ▷ -input_csv_file GF_q5_table_add.csv \
3360 ▷ ▷ ▷ -box_width 40 \
3361 ▷ ▷ ▷ -bit_depth 24 \
3362 ▷ ▷ ▷ -partition 3 5 5 \
3363 ▷ ▷ -end \
3364
3365
3366 F_7:
3367 ▷ $(ORBITER) -v 3 \
3368 ▷ ▷ -define F -finite_field -q 7 -end \
3369 ▷ ▷ -with F -do -finite_field_activity \
3370 ▷ ▷ ▷ -cheat_sheet_GF \
3371 ▷ ▷ -end
3372 ▷ $(PDFLATEX) GF_7.tex
3373 ▷ $(OPEN) GF_7.pdf
3374
3375
3376 F_7_tables:
3377 ▷ $(ORBITER) -v 3 \
3378 ▷ ▷ -define F -finite_field -q 7 -end \
3379 ▷ ▷ -with F -do -finite_field_activity \
3380 ▷ ▷ ▷ -export_tables \
3381 ▷ ▷ -end
3382 ▷ $(ORBITER) -v 3 \
3383 ▷ ▷ -define all_one -vector -repeat 1 7 -end \
3384 ▷ ▷ -draw_matrix \
3385 ▷ ▷ ▷ -input_csv_file GF_q7_table_add.csv \
3386 ▷ ▷ ▷ -box_width 40 \
3387 ▷ ▷ ▷ -bit_depth 24 \
3388 ▷ ▷ ▷ -partition 3 all_one all_one \
3389 ▷ ▷ -end \
3390 ▷ ▷ -draw_matrix \
3391 ▷ ▷ ▷ -input_csv_file GF_q7_table_mul.csv \
3392 ▷ ▷ ▷ -box_width 40 \
3393 ▷ ▷ ▷ -bit_depth 24 \
3394 ▷ ▷ ▷ -partition 3 all_one all_one \
3395 ▷ ▷ -end
3396 ▷ magick GF_q7_table_add_draw.bmp GF_q7_table_add_draw.png
3397 ▷ magick GF_q7_table_mul_draw.bmp GF_q7_table_mul_draw.png
3398
3399
3400 F_13:
3401 ▷ $(ORBITER) -v 6 \
3402 ▷ ▷ -define F -finite_field -q 13 -end \
3403 ▷ ▷ -with F -do -finite_field_activity \
3404 ▷ ▷ ▷ -cheat_sheet_GF \
3405 ▷ ▷ -end
3406 ▷ $(PDFLATEX) GF_13.tex
3407 ▷ $(OPEN) GF_13.pdf
3408
3409
3410 F_19:
3411 ▷ $(ORBITER) -v 6 \
3412 ▷ ▷ -define F -finite_field -q 19 -end \
3413 ▷ ▷ -with F -do -finite_field_activity \

```

```

3414 ▷ ▷ ▷ -cheat_sheet_GF \
3415 ▷ ▷ -end
3416 ▷ $(PDFLATEX) GF_19.tex
3417 ▷ $(OPEN) GF_19.pdf
3418
3419
3420 F_127:
3421 ▷ $(ORBITER) -v 3 \
3422 ▷ ▷ -define F -finite_field -q 127 -end \
3423 ▷ ▷ -with F -do -finite_field_activity \
3424 ▷ ▷ ▷ -cheat_sheet_GF \
3425 ▷ ▷ -end
3426
3427
3428 F_11_product_of_all_nonzero_elements:
3429 ▷ $(ORBITER) -v 3 \
3430 ▷ ▷ -define F -finite_field -q 11 -end \
3431 ▷ ▷ -define S -vector -field F -loop 1 11 1 -end \
3432 ▷ ▷ -with F -do -finite_field_activity \
3433 ▷ ▷ ▷ -product_of S \
3434 ▷ ▷ -end
3435
3436
3437 F_7_vandermonde:
3438 ▷ $(ORBITER) -v 3 \
3439 ▷ ▷ -define F -finite_field -q 7 -end \
3440 ▷ ▷ -with F -do -finite_field_activity \
3441 ▷ ▷ ▷ -Vandermonde_matrix \
3442 ▷ ▷ -end
3443
3444
3445 F_101_wo:
3446 ▷ $(ORBITER) -v 3 \
3447 ▷ ▷ -define F -finite_field -q 101 -without_tables -end \
3448 ▷ ▷ -with F -do -finite_field_activity \
3449 ▷ ▷ ▷ -cheat_sheet_GF \
3450 ▷ ▷ -end
3451 ▷ $(PDFLATEX) GF_101.tex
3452 ▷ $(OPEN) GF_101.pdf
3453
3454
3455 F_1021_wo:
3456 ▷ $(ORBITER) -v 3 \
3457 ▷ ▷ -define F -finite_field -q 1021 -without_tables -end \
3458 ▷ ▷ -with F -do -finite_field_activity \
3459 ▷ ▷ ▷ -cheat_sheet_GF \
3460 ▷ ▷ -end
3461
3462
3463
3464
3465 #####
3466 # Section 3.2: Extension Fields
3467
3468 SECTION_EXTENSION_FIELDS:
3469 ▷
3470
3471 test_3.2:
3472 ▷ make F_4

```

```

3473 ▷ make F_4_tables
3474 ▷ make F_8
3475 ▷ make F_8_tables
3476 ▷ make F_8b
3477 ▷ make F_9
3478 ▷ make F_16
3479 ▷ make F_16_calculate
3480 ▷ make F_16_tables
3481 ▷ make F_64
3482 ▷ make nth_roots_63
3483 ▷ make F8_F2_field_reduction
3484 ▷ make F_125
3485 ▷ make F_4096
3486 ▷ #make F_4489
3487 ▷ #make F_2_power_24
3488
3489
3490 F_4:
3491 ▷ $(ORBITER) -v 3 \
3492 ▷ ▷ -define F -finite_field -q 4 -end \
3493 ▷ ▷ -with F -do -finite_field_activity \
3494 ▷ ▷ ▷ -cheat_sheet_GF \
3495 ▷ ▷ -end
3496 ▷ $(PDFLATEX) GF_4.tex
3497 ▷ $(OPEN) GF_4.pdf
3498
3499
3500
3501
3502 F_4_tables:
3503 ▷ $(ORBITER) -v 3 \
3504 ▷ ▷ -define F -finite_field -q 4 -end \
3505 ▷ ▷ -with F -do -finite_field_activity \
3506 ▷ ▷ ▷ -export_tables \
3507 ▷ ▷ -end
3508 ▷ $(ORBITER) -v 3 \
3509 ▷ ▷ -define all_one -vector -repeat 1 4 -end \
3510 ▷ ▷ -draw_matrix \
3511 ▷ ▷ ▷ -input_csv_file GF_q4_table_add.csv \
3512 ▷ ▷ ▷ -box_width 40 \
3513 ▷ ▷ ▷ -bit_depth 24 \
3514 ▷ ▷ ▷ -partition 3 all_one all_one \
3515 ▷ ▷ -end \
3516 ▷ ▷ -draw_matrix \
3517 ▷ ▷ ▷ -input_csv_file GF_q4_table_mul.csv \
3518 ▷ ▷ ▷ -box_width 40 \
3519 ▷ ▷ ▷ -bit_depth 24 \
3520 ▷ ▷ ▷ -partition 3 all_one all_one \
3521 ▷ ▷ -end
3522 ▷ magick GF_q4_table_add_draw.bmp GF_q4_table_add_draw.png
3523 ▷ magick GF_q4_table_mul_draw.bmp GF_q4_table_mul_draw.png
3524
3525
3526
3527 F_8:
3528 ▷ $(ORBITER) -v 3 \
3529 ▷ ▷ -define F -finite_field -q 8 -end \
3530 ▷ ▷ -with F -do -finite_field_activity \
3531 ▷ ▷ ▷ -cheat_sheet_GF \

```

```

3532 ▷ ▷ -end
3533 ▷ $(PDFLATEX) GF_8.tex
3534 ▷ $(OPEN) GF_8.pdf
3535
3536
3537 F_8_tables:
3538 ▷ $(ORBITER) -v 3 \
3539 ▷ ▷ -define all_one -vector -repeat 1 8 -end \
3540 ▷ ▷ -define F -finite_field -q 8 -end \
3541 ▷ ▷ -with F -do -finite_field_activity -export_tables -end \
3542 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end \
3543 ▷ ▷ -draw_matrix \
3544 ▷ ▷ ▷ -input_csv_file GF_q8_table_add.csv \
3545 ▷ ▷ ▷ -box_width 40 \
3546 ▷ ▷ ▷ -bit_depth 24 \
3547 ▷ ▷ ▷ -partition 3 all_one all_one \
3548 ▷ ▷ -end \
3549 ▷ ▷ -draw_matrix \
3550 ▷ ▷ ▷ -input_csv_file GF_q8_table_mul.csv \
3551 ▷ ▷ ▷ -box_width 40 \
3552 ▷ ▷ ▷ -bit_depth 24 \
3553 ▷ ▷ ▷ -partition 3 all_one all_one \
3554 ▷ ▷ -end
3555
3556
3557 F_8b:
3558 ▷ $(ORBITER) -v 3 \
3559 ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
3560 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
3561 ▷ $(PDFLATEX) GF_8.tex
3562 ▷ $(OPEN) GF_8.pdf
3563
3564
3565
3566 F_9:
3567 ▷ $(ORBITER) -v 3 \
3568 ▷ ▷ -define F -finite_field -q 9 -end \
3569 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
3570 ▷ $(PDFLATEX) GF_9.tex
3571 ▷ $(OPEN) GF_9.pdf
3572
3573
3574
3575 F_16:
3576 ▷ $(ORBITER) -v 10 \
3577 ▷ ▷ -define F -finite_field -q 16 -compute_related_fields -end \
3578 ▷ ▷ -with F -do -finite_field_activity \
3579 ▷ ▷ ▷ -cheat_sheet_GF \
3580 ▷ ▷ -end
3581 ▷ $(PDFLATEX) GF_16.tex
3582 ▷ $(OPEN) GF_16.pdf
3583
3584
3585 F_16_calculate:
3586 ▷ $(ORBITER) -v 3 \
3587 ▷ ▷ -define F -finite_field -q 16 -end \
3588 ▷ ▷ -define M -symbolic_object \
3589 ▷ ▷ ▷ -field F \
3590 ▷ ▷ ▷ -text "2*12" \

```

```

3591 ▷ ▷ -end
3592
3593
3594 F_16_tables:
3595 ▷ $(ORBITER) -v 3 \
3596 ▷ ▷ -define F -finite_field -q 16 -end \
3597 ▷ ▷ -with F -do -finite_field_activity -export_tables -end
3598 ▷ $(ORBITER) -v 3 \
3599 ▷ ▷ -define all_one -vector -repeat 1 16 -end \
3600 ▷ ▷ -draw_matrix \
3601 ▷ ▷ ▷ -input_csv_file GF_q16_table_add.csv \
3602 ▷ ▷ ▷ -box_width 40 -bit_depth 24 \
3603 ▷ ▷ ▷ -partition 3 all_one all_one \
3604 ▷ ▷ -end \
3605 ▷ ▷ -draw_matrix \
3606 ▷ ▷ ▷ -input_csv_file GF_q16_table_mul.csv \
3607 ▷ ▷ ▷ -box_width 40 -bit_depth 24 \
3608 ▷ ▷ ▷ -partition 3 all_one all_one \
3609 ▷ ▷ -end
3610 ▷ magick GF_q16_table_add_draw.bmp GF_q16_table_add_draw.png
3611 ▷ magick GF_q16_table_mul_draw.bmp GF_q16_table_mul_draw.png
3612
3613
3614
3615 F_25:
3616 ▷ $(ORBITER) -v 10 \
3617 ▷ ▷ -define F -finite_field -q 25 -compute_related_fields -end \
3618 ▷ ▷ -with F -do -finite_field_activity \
3619 ▷ ▷ ▷ -cheat_sheet_GF \
3620 ▷ ▷ -end
3621 ▷ $(PDFLATEX) GF_25.tex
3622 ▷ $(OPEN) GF_25.pdf
3623
3624
3625
3626 F_64:
3627 ▷ $(ORBITER) -v 3 \
3628 ▷ ▷ -define F -finite_field -q 64 -end \
3629 ▷ ▷ -with F -do -finite_field_activity \
3630 ▷ ▷ ▷ -cheat_sheet_GF \
3631 ▷ ▷ -end
3632 ▷ $(PDFLATEX) GF_64.tex
3633 ▷ $(OPEN) GF_64.pdf
3634
3635
3636
3637 nth_roots_63:
3638 ▷ $(ORBITER) -v 3 \
3639 ▷ ▷ -define F -finite_field -q 2 -end \
3640 ▷ ▷ -with F -do -finite_field_activity \
3641 ▷ ▷ ▷ -nth_roots 63 \
3642 ▷ ▷ -end
3643 ▷ $(PDFLATEX) Nth_roots_q2_n63.tex
3644 ▷ $(OPEN) Nth_roots_q2_n63.pdf
3645
3646
3647 F8_F2_field_reduction:
3648 ▷ $(ORBITER) -v 2 \
3649 ▷ ▷ -define F -finite_field -q 8 -end \

```

```

3650 ▷ ▷ -define v -vector -loop 0 8 1 -end \
3651 ▷ ▷ -define C -code -field F -ReedSolomon 7 3 -end \
3652 ▷ ▷ -with F -do \
3653 ▷ ▷ -finite_field_activity \
3654 ▷ ▷ -field_reduction "F8_to_F2" \
3655 ▷ ▷ ▷ 2 1 8 v \
3656 ▷ ▷ -end
3657 ▷ $(ORBITER) -v 2 \
3658 ▷ ▷ -draw_matrix -input_csv_file F8_to_F2.csv \
3659 ▷ ▷ ▷ -box_width 40 -bit_depth 24 \
3660 ▷ ▷ ▷ -partition 4 "3" "3,3,3,3,3,3,3" \
3661 ▷ ▷ -end
3662 ▷ magick F8_to_F2_draw.bmp F8_to_F2_draw.png
3663 ▷ $(OPEN) F8_to_F2_draw.png
3664
3665
3666 F_125:
3667 ▷ $(ORBITER) -v 3 \
3668 ▷ ▷ -define F -finite_field -q 125 -end \
3669 ▷ ▷ -with F -do -finite_field_activity \
3670 ▷ ▷ ▷ -cheat_sheet_GF \
3671 ▷ ▷ -end
3672 ▷ $(PDFLATEX) GF_125.tex
3673 ▷ $(OPEN) GF_125.pdf
3674
3675
3676
3677
3678 F_4096:
3679 ▷ $(ORBITER) -v 4 \
3680 ▷ ▷ -define F -finite_field -q 4096 \
3681 ▷ ▷ ▷ -compute_related_fields \
3682 ▷ ▷ -end \
3683 ▷ ▷ -with F -do -finite_field_activity \
3684 ▷ ▷ ▷ -cheat_sheet_GF \
3685 ▷ ▷ -end
3686 ▷ $(PDFLATEX) GF_4096.tex
3687 ▷ $(OPEN) GF_4096.pdf
3688
3689
3690
3691 # 67^2 = 4489
3692
3693 F_4489:
3694 ▷ $(ORBITER) -v 4 \
3695 ▷ ▷ -define F -finite_field -q 4489 -without_tables -end \
3696
3697 # ToDo error message
3698 #finite_field::alpha_power !f.has_table
3699
3700
3701 F_2_power_24:
3702 ▷ $(ORBITER) -v 4 \
3703 ▷ ▷ -define F -finite_field -q 16777216 \
3704 ▷ ▷ ▷ -compute_related_fields \
3705 ▷ ▷ ▷ -without_tables \
3706 ▷ ▷ -end \
3707 ▷ ▷ -with F -do -finite_field_activity \
3708 ▷ ▷ ▷ -cheat_sheet_GF \

```

```

3709 ▷ ▷ -end
3710 ▷ $(PDFLATEX) GF_16777216.tex
3711 ▷ $(OPEN) GF_16777216.pdf
3712
3713
3714 #####
3715 # Section 3.3: Linear Algebra over Finite Fields
3716
3717
3718 SECTION_LINEAR_ALGEBRA:
3719
3720
3721 test_3_3:
3722 ▷ make RREF
3723 ▷ make RREF_V7
3724 ▷ make nullspace
3725 ▷ make RREF_RS_6_4_7
3726 ▷ make eigenstuff
3727 ▷ make classes_GL_3_2
3728 ▷ make classes_GL_4_2
3729
3730
3731 RREF:
3732 ▷ $(ORBITER) -v 2 \
3733 ▷ ▷ -define F -finite_field -q 2 -end \
3734 ▷ ▷ -define v -vector -field F -format 2 \
3735 ▷ ▷ ▷ -dense "1,1,1,1,0,1,1,0,0,1" \
3736 ▷ ▷ -end \
3737 ▷ ▷ -with F -do -finite_field_activity \
3738 ▷ ▷ ▷ -RREF v \
3739 ▷ ▷ -end
3740 ▷ $(PDFLATEX) v_rref.tex
3741 ▷ $(OPEN) v_rref.pdf
3742
3743
3744 RREF_V7:
3745 ▷ $(ORBITER) -v 2 \
3746 ▷ ▷ -define F -finite_field -q 7 -end \
3747 ▷ ▷ -define V7 -vector -format 7 \
3748 ▷ ▷ ▷ -dense $(V7_VANDERMONDE_EXTENDED) \
3749 ▷ ▷ -end \
3750 ▷ ▷ -with F -do -finite_field_activity \
3751 ▷ ▷ ▷ -RREF V7 \
3752 ▷ ▷ -end
3753 ▷ $(PDFLATEX) V7_rref.tex
3754 ▷ $(OPEN) V7_rref.pdf
3755
3756
3757
3758 nullspace:
3759 ▷ $(ORBITER) -v 2 \
3760 ▷ ▷ -define F2 -finite_field -q 2 -end \
3761 ▷ ▷ -define v -vector -field F2 -format 2 \
3762 ▷ ▷ ▷ -dense "1,1,1,1,0,1,1,0,0,1" \
3763 ▷ ▷ -end \
3764 ▷ ▷ -with F2 -do \
3765 ▷ ▷ -finite_field_activity \
3766 ▷ ▷ ▷ -nullspace v \
3767 ▷ ▷ -end

```

```

3768 ▷ $(PDFLATEX) v_nullspace.tex
3769 ▷ $(OPEN) v_nullspace.pdf
3770
3771 RREF_RS_6_4_7:
3772 ▷ $(ORBITER) -v 2 \
3773 ▷ ▷ -define F -finite_field -q 7 -end \
3774 ▷ ▷ -define v -vector -field F -format 4 \
3775 ▷ ▷ ▷ -compact $(CODE_RS_6_4_7) \
3776 ▷ ▷ -end \
3777 ▷ ▷ -with F -do \
3778 ▷ ▷ -finite_field_activity \
3779 ▷ ▷ ▷ -RREF v \
3780 ▷ ▷ -end
3781 ▷ $(PDFLATEX) v_rref.tex
3782 ▷ $(OPEN) v_rref.pdf
3783
3784
3785 ## these are global algebra functions:
3786
3787
3788 eigenstuff:
3789 ▷ $(ORBITER) -v 6 \
3790 ▷ ▷ -define F -finite_field -q 5 -end \
3791 ▷ ▷ -eigenstuff F 4 "0,1,0,2,0,1,2,1,4,2,3,1,2,0,4,3"
3792
3793 classes_GL_3_2:
3794 ▷ $(ORBITER) -v 7 \
3795 ▷ ▷ -define F -finite_field -q 2 -end \
3796 ▷ ▷ -all_rational_normal_forms F 3
3797 ▷ $(PDFLATEX) Class_reps_GL_3_2.tex
3798 ▷ $(OPEN) Class_reps_GL_3_2.pdf
3799
3800
3801 classes_GL_4_2:
3802 ▷ $(ORBITER) -v 7 \
3803 ▷ ▷ -define F -finite_field -q 2 -end \
3804 ▷ ▷ -all_rational_normal_forms F 4
3805 ▷ $(PDFLATEX) Class_reps_GL_4_2.tex
3806 ▷ $(OPEN) Class_reps_GL_4_2.pdf
3807
3808
3809 #####
3810 # Section 3.4: Advanced Topics in finite fields
3811
3812
3813
3814 SECTION_ADVANCED_TOPICS_IN_FINITE_FIELDS:
3815
3816
3817 test_3_4:
3818 ▷ make normal_basis_2_3
3819 ▷ make normal_basis_2_4
3820 ▷ make normal_basis_2_4.poly19
3821 ▷ make normal_basis_2_6
3822 ▷ make normal_basis_2_8
3823 ▷ make F8_over_F2_field_reduction
3824 ▷ make F_64_over_F8_field_reduction
3825 ▷ make F_64_over_F4_field_reduction
3826 ▷ make F_64_over_F2_field_reduction

```



```

3827 ▷ make F_8.Nth_roots_21
3828 ▷ make F_8.vandermonde
3829 ▷ make F_512.vandermonde
3830 ▷ #make NTT_k4_q17.cpp
3831 ▷ #make F_17.NTT.compile
3832
3833
3834
3835 normal_basis_2.3:
3836 ▷ $(ORBITER) -v 2 \
3837 ▷ ▷ -define F -finite_field -q 2 -end \
3838 ▷ ▷ -with F -do -finite_field_activity \
3839 ▷ ▷ ▷ -normal_basis 3 \
3840 ▷ ▷ -end
3841 ▷ $(PDFLATEX) normal_basis_q2_d3.tex
3842 ▷ $(OPEN) normal_basis_q2_d3.pdf
3843
3844
3845 normal_basis_2.4:
3846 ▷ $(ORBITER) -v 2 \
3847 ▷ ▷ -define F -finite_field -q 2 -end \
3848 ▷ ▷ -with F -do -finite_field_activity \
3849 ▷ ▷ ▷ -normal_basis 4 \
3850 ▷ ▷ -end
3851
3852
3853 normal_basis_2.4.poly19:
3854 ▷ $(ORBITER) -v 2 \
3855 ▷ ▷ -define F -finite_field -q 2 -end \
3856 ▷ ▷ -with F -do -finite_field_activity \
3857 ▷ ▷ ▷ -normal_basis_with_given_polynomial 19 4 \
3858 ▷ ▷ -end
3859 ▷ $(PDFLATEX) normal_basis_q2_d4_poly19.tex
3860 ▷ $(OPEN) normal_basis_q2_d4_poly19.pdf
3861
3862
3863 normal_basis_2.6:
3864 ▷ $(ORBITER) -v 2 \
3865 ▷ ▷ -define F -finite_field -q 2 -end \
3866 ▷ ▷ -with F -do -finite_field_activity \
3867 ▷ ▷ ▷ -normal_basis 6 \
3868 ▷ ▷ -end
3869
3870
3871
3872 normal_basis_2.8:
3873 ▷ $(ORBITER) -v 2 \
3874 ▷ ▷ -define F -finite_field -q 2 -end \
3875 ▷ ▷ -with F -do -finite_field_activity \
3876 ▷ ▷ ▷ -normal_basis 8 \
3877 ▷ ▷ -end
3878 ▷ $(PDFLATEX) normal_basis_q2_d8.tex
3879 ▷ $(OPEN) normal_basis_q2_d8.pdf
3880
3881
3882
3883 F8_over_F2_field_reduction:
3884 ▷ $(ORBITER) -v 2 \
3885 ▷ ▷ -define F -finite_field -q 8 -end \

```

```

3886 ▷ ▷ -loop L 0 8 1 \
3887 ▷ ▷ ▷ -with F -do \
3888 ▷ ▷ ▷ -finite_field_activity \
3889 ▷ ▷ ▷ ▷ -field_reduction "F8_red_%L" 2 1 1 "%L" \
3890 ▷ ▷ ▷ -end \
3891 ▷ ▷ -end_loop L
3892 ▷ $(ORBITER) -v 2 \
3893 ▷ ▷ -loop L 0 8 1 \
3894 ▷ ▷ ▷ -draw_matrix \
3895 ▷ ▷ ▷ ▷ -input_csv_file F8_red_%L.csv \
3896 ▷ ▷ ▷ ▷ -box_width 40 -bit_depth 24 \
3897 ▷ ▷ ▷ ▷ -partition 4 3 3 \
3898 ▷ ▷ ▷ -end \
3899 ▷ ▷ -end_loop L
3900 ▷ #$(PDFLATEX) field_reduction_Q8-q2.5.7.tex
3901
3902
3903
3904
3905 F_64_over_F8.field_reduction:
3906 ▷ $(ORBITER) -v 2 \
3907 ▷ ▷ -define F -finite_field -q 64 -end \
3908 ▷ ▷ -define elts -vector -field F -loop 0 64 1 -end \
3909 ▷ ▷ -with F -do \
3910 ▷ ▷ -finite_field_activity \
3911 ▷ ▷ ▷ -field_reduction "F64_over_F8" 8 8 8 \
3912 ▷ ▷ ▷ elts \
3913 ▷ ▷ -end
3914 ▷ $(ORBITER) -v 2 \
3915 ▷ ▷ -draw_matrix \
3916 ▷ ▷ ▷ -input_csv_file F64_over_F8.csv \
3917 ▷ ▷ ▷ -box_width 40 -bit_depth 24 \
3918 ▷ ▷ ▷ -partition 4 "2,2,2,2,2,2,2,2" "2,2,2,2,2,2,2,2" \
3919 ▷ ▷ -end
3920 ▷ $(OPEN) F64_over_F8.draw.bmp
3921 ▷ $(PDFLATEX) field_reduction_Q64-q8.8.8.tex
3922 ▷ $(OPEN) field_reduction_Q64-q8.8.8.pdf
3923
3924
3925 F_64_over_F4.field_reduction:
3926 ▷ $(ORBITER) -v 2 \
3927 ▷ ▷ -define F -finite_field -q 64 -end \
3928 ▷ ▷ -define elts -vector -field F -loop 0 64 1 -end \
3929 ▷ ▷ -with F -do \
3930 ▷ ▷ -finite_field_activity \
3931 ▷ ▷ ▷ -field_reduction "F64_over_F4" 4 8 8 elts \
3932 ▷ ▷ -end
3933 ▷ $(ORBITER) -v 2 \
3934 ▷ ▷ -draw_matrix \
3935 ▷ ▷ ▷ -input_csv_file F64_over_F4.csv \
3936 ▷ ▷ ▷ -box_width 40 -bit_depth 24 \
3937 ▷ ▷ ▷ -partition 4 "3,3,3,3,3,3,3,3" "3,3,3,3,3,3,3,3" \
3938 ▷ ▷ -end
3939 ▷ $(OPEN) F64_over_F4.draw.bmp
3940 ▷ $(PDFLATEX) field_reduction_Q64-q4.8.8.tex
3941 ▷ $(OPEN) field_reduction_Q64-q4.8.8.pdf
3942
3943
3944 F_64_over_F2.field_reduction:

```

```

3945 ▷ $(ORBITER) -v 2 \
3946 ▷ ▷ -define F -finite_field -q 64 -end \
3947 ▷ ▷ -define elts -vector -field F -loop 0 64 1 -end \
3948 ▷ ▷ -with F -do \
3949 ▷ ▷ -finite_field_activity \
3950 ▷ ▷ ▷ -field_reduction "F64_over_F2" 2 8 8 elts \
3951 ▷ ▷ -end
3952 ▷ $(ORBITER) -v 2 \
3953 ▷ ▷ -draw_matrix \
3954 ▷ ▷ ▷ -input_csv_file F64_over_F2.csv \
3955 ▷ ▷ ▷ -box_width 40 -bit_depth 24 \
3956 ▷ ▷ ▷ -partition 4 "6,6,6,6,6,6,6" "6,6,6,6,6,6,6" \
3957 ▷ ▷ -end
3958 ▷ $(OPEN) F64_over_F2.draw.bmp
3959 ▷ $(PDFLATEX) field_reduction_Q64_q2_8_8.tex
3960 ▷ $(OPEN) field_reduction_Q64_q2_8_8.pdf
3961
3962
3963
3964
3965
3966 F_8_Nth_roots_21:
3967 ▷ $(ORBITER) -v 3 \
3968 ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
3969 ▷ ▷ -with F -do -finite_field_activity \
3970 ▷ ▷ ▷ -nth_roots 21 \
3971 ▷ ▷ -end
3972 ▷ $(PDFLATEX) Nth_roots_q8_n21.tex
3973 ▷ $(OPEN) Nth_roots_q8_n21.pdf
3974
3975
3976
3977
3978 F_8_vandermonde:
3979 ▷ $(ORBITER) -v 3 \
3980 ▷ ▷ -define F -finite_field -q 8 -end \
3981 ▷ ▷ -with F -do -finite_field_activity \
3982 ▷ ▷ ▷ -Vandermonde_matrix \
3983 ▷ ▷ -end
3984
3985
3986
3987 F_512_vandermonde:
3988 ▷ $(ORBITER) -v 3 \
3989 ▷ ▷ -define F -finite_field -q 512 -end \
3990 ▷ ▷ -with F -do -finite_field_activity \
3991 ▷ ▷ ▷ -Vandermonde_matrix \
3992 ▷ ▷ -end
3993 ▷ - rm Vandermonde_q512.csv
3994 ▷ - rm Vandermonde_q512_inv.csv
3995
3996
3997
3998
3999
4000 NTT_k4_q17.cpp:
4001 ▷ $(ORBITER) -v 3 \
4002 ▷ ▷ -define F -finite_field -q 17 -end \
4003 ▷ ▷ -with F -do -coding_theoretic_activity \

```

```

4004 ▷ ▷ ▷ -NTT 4 17 \
4005 ▷ ▷ -end
4006
4007 F_17_NTT_compile: NTT_k4_q17.cpp
4008 ▷ $(MY_CPP) NTT_k4_q17.cpp $(CPPFLAGS) \
4009 ▷ ▷ $(LIB) $(LFLAGS) -o NTT_k4_q17.out
4010 ▷ ./NTT_k4_q17.out
4011
4012 F64_1:
4013 ▷ $(ORBITER) -v 3 \
4014 ▷ ▷ -define F -finite_field -q 64 -override_polynomial 97 -end \
4015 ▷ ▷ -with F -do -finite_field_activity \
4016 ▷ ▷ ▷ -nth_roots 7 \
4017 ▷ ▷ -end
4018 ▷ $(PDFLATEX) Nth_roots_q64_n7.tex
4019 ▷ $(OPEN) Nth_roots_q64_n7.pdf
4020
4021
4022
4023
4024 #####
4025 # Chapter 4 - Projective Geometry
4026 #####
4027
4028
4029 test_4:
4030 ▷ make test_4.1
4031 ▷ make test_4.2
4032 ▷ make test_4.3
4033 ▷ make test_4.4
4034 ▷ make test_4.5
4035
4036
4037
4038 #####
4039 # Section 4.1: Finite Projective Spaces
4040
4041 SECTION.FINITE_PROJECTIVE_SPACES:
4042
4043
4044 test_4.1:
4045 ▷ make PG_3_2.easy
4046 ▷ make PG_1_16
4047 ▷ make PG_2_2
4048 ▷ make PG_2_3
4049 ▷ make PG_2_4
4050 ▷ make PG_2_13
4051 ▷ make PG_2_64
4052 ▷ make PG_3_2
4053 ▷ make PG_3_4
4054 ▷ make PG_3_5
4055 ▷ make PG_3_7
4056 ▷ make PG_3_8
4057 ▷ make PG_3_16
4058 ▷ make PG_3_25
4059 ▷ make PG_4_2
4060 ▷ make PG_4_3
4061 ▷ make PG_8_2
4062

```

```

4063
4064 PG_3_2_easy:
4065 ▷ $(ORBITER) -v 2 \
4066 ▷ ▷ -define F -finite_field -q 2 -end \
4067 ▷ ▷ -define P -projective_space -n 3 -field F -end
4068
4069
4070
4071
4072 PG_1_16:
4073 ▷ $(ORBITER) -v 2 \
4074 ▷ ▷ -define Do -draw_options -end \
4075 ▷ ▷ -define F -finite_field -q 16 -compute_related_fields -end \
4076 ▷ ▷ -define P -projective_space -n 1 -field F -v 0 -end \
4077 ▷ ▷ -with P -do -projective_space_activity \
4078 ▷ ▷ ▷ -cheat_sheet Do \
4079 ▷ ▷ -end
4080 ▷ $(PDFLATEX) PG_1_16.tex
4081 ▷ $(OPEN) PG_1_16.pdf
4082
4083
4084 PG_2_2:
4085 ▷ $(ORBITER) -v 2 \
4086 ▷ ▷ -define Do -draw_options -end \
4087 ▷ ▷ -define F -finite_field -q 2 -end \
4088 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
4089 ▷ ▷ -with P -do -projective_space_activity \
4090 ▷ ▷ ▷ -cheat_sheet Do \
4091 ▷ ▷ -end
4092 ▷ $(PDFLATEX) PG_2_2.tex
4093 ▷ $(OPEN) PG_2_2.pdf
4094
4095 PG_2_3:
4096 ▷ $(ORBITER) -v 2 \
4097 ▷ ▷ -define Do -draw_options -end \
4098 ▷ ▷ -define F -finite_field -q 3 -end \
4099 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
4100 ▷ ▷ -with P -do -projective_space_activity \
4101 ▷ ▷ ▷ -cheat_sheet Do \
4102 ▷ ▷ -end
4103 ▷ $(PDFLATEX) PG_2_3.tex
4104 ▷ $(OPEN) PG_2_3.pdf
4105
4106
4107 PG_2_4:
4108 ▷ $(ORBITER) -v 2 \
4109 ▷ ▷ -define Do -draw_options -end \
4110 ▷ ▷ -define F -finite_field -q 4 -compute_related_fields -end \
4111 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
4112 ▷ ▷ -with P -do -projective_space_activity \
4113 ▷ ▷ ▷ -cheat_sheet Do \
4114 ▷ ▷ -end
4115 ▷ $(PDFLATEX) PG_2_4.tex
4116 ▷ $(OPEN) PG_2_4.pdf
4117
4118
4119
4120
4121 PG_2_13:

```

```

4122 ▷ $(ORBITER) -v 2 \
4123 ▷ ▷ -define Do -draw_options -end \
4124 ▷ ▷ -define F -finite_field -q 13 -end \
4125 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
4126 ▷ ▷ -with P -do -projective_space_activity \
4127 ▷ ▷ ▷ -cheat_sheet Do \
4128 ▷ ▷ -end
4129 ▷ $(PDFLATEX) PG_2_13.tex
4130 ▷ $(OPEN) PG_2_13.pdf
4131
4132
4133
4134
4135 PG_2_64:
4136 ▷ $(ORBITER) -v 2 \
4137 ▷ ▷ -define Do -draw_options -end \
4138 ▷ ▷ -define F -finite_field -q 64 -compute_related_fields -end \
4139 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
4140 ▷ ▷ -with P -do -projective_space_activity \
4141 ▷ ▷ ▷ -cheat_sheet Do \
4142 ▷ ▷ -end
4143 ▷ $(PDFLATEX) PG_2_64.tex
4144 ▷ $(OPEN) PG_2_64.pdf
4145
4146
4147
4148 PG_3_2:
4149 ▷ $(ORBITER) -v 2 \
4150 ▷ ▷ -define Do -draw_options -end \
4151 ▷ ▷ -define F -finite_field -q 2 -end \
4152 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4153 ▷ ▷ -with P -do -projective_space_activity \
4154 ▷ ▷ ▷ -cheat_sheet Do \
4155 ▷ ▷ -end
4156 ▷ $(PDFLATEX) PG_3_2.tex
4157 ▷ $(OPEN) PG_3_2.pdf
4158
4159
4160 PG_3_4:
4161 ▷ $(ORBITER) -v 2 \
4162 ▷ ▷ -define Do -draw_options -end \
4163 ▷ ▷ -define F -finite_field -q 4 -compute_related_fields -end \
4164 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4165 ▷ ▷ -with P -do -projective_space_activity \
4166 ▷ ▷ ▷ -cheat_sheet Do \
4167 ▷ ▷ -end
4168 ▷ $(PDFLATEX) PG_3_4.tex
4169 ▷ $(OPEN) PG_3_4.pdf
4170
4171 PG_3_5:
4172 ▷ $(ORBITER) -v 2 \
4173 ▷ ▷ -define Do -draw_options -end \
4174 ▷ ▷ -define F -finite_field -q 5 -end \
4175 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4176 ▷ ▷ -with P -do -projective_space_activity \
4177 ▷ ▷ ▷ -cheat_sheet Do \
4178 ▷ ▷ -end
4179 ▷ $(PDFLATEX) PG_3_5.tex
4180 ▷ $(OPEN) PG_3_5.pdf

```

```

4181
4182
4183 PG_3_7:
4184 ▷ $(ORBITER) -v 2 \
4185 ▷ ▷ -define Do -draw.options -end \
4186 ▷ ▷ -define F -finite.field -q 7 -end \
4187 ▷ ▷ -define P -projective.space -n 3 -field F -v 0 -end \
4188 ▷ ▷ -with P -do -projective.space.activity \
4189 ▷ ▷ ▷ -cheat.sheet Do \
4190 ▷ ▷ -end
4191 ▷ $(PDFLATEX) PG_3_7.tex
4192 ▷ $(OPEN) PG_3_7.pdf
4193
4194
4195
4196
4197 PG_3_8:
4198 ▷ $(ORBITER) -v 2 \
4199 ▷ ▷ -define Do -draw.options -end \
4200 ▷ ▷ -define F -finite.field -q 8 -compute.related.fields -end \
4201 ▷ ▷ -define P -projective.space -n 3 -field F -v 0 -end \
4202 ▷ ▷ -with P -do -projective.space.activity \
4203 ▷ ▷ ▷ -cheat.sheet Do \
4204 ▷ ▷ -end
4205 ▷ $(PDFLATEX) PG_3_8.tex
4206 ▷ $(OPEN) PG_3_8.pdf
4207
4208
4209 PG_3_16:
4210 ▷ $(ORBITER) -v 2 \
4211 ▷ ▷ -define Do -draw.options -end \
4212 ▷ ▷ -define F -finite.field -q 16 -compute.related.fields -end \
4213 ▷ ▷ -define P -projective.space -n 3 -field F -v 0 -end \
4214 ▷ ▷ -with P -do -projective.space.activity \
4215 ▷ ▷ ▷ -cheat.sheet Do \
4216 ▷ ▷ -end
4217 ▷ $(PDFLATEX) PG_3_16.tex
4218 ▷ $(OPEN) PG_3_16.pdf
4219
4220
4221
4222
4223 PG_3_25:
4224 ▷ $(ORBITER) -v 2 \
4225 ▷ ▷ -define Do -draw.options -end \
4226 ▷ ▷ -define F -finite.field -q 25 -compute.related.fields -end \
4227 ▷ ▷ -define P -projective.space -n 3 -field F -v 0 -end \
4228 ▷ ▷ -with P -do -projective.space.activity \
4229 ▷ ▷ ▷ -cheat.sheet Do \
4230 ▷ ▷ -end
4231 ▷ $(PDFLATEX) PG_3_25.tex
4232 ▷ $(OPEN) PG_3_25.pdf
4233
4234
4235
4236 PG_4_2:
4237 ▷ $(ORBITER) -v 2 \
4238 ▷ ▷ -define Do -draw.options -end \
4239 ▷ ▷ -define F -finite.field -q 2 -end \

```

```

4240 ▷ ▷ -define P -projective_space -n 4 -field F -v 0 -end \
4241 ▷ ▷ -with P -do -projective_space_activity \
4242 ▷ ▷ ▷ -cheat_sheet Do \
4243 ▷ ▷ -end
4244 ▷ $(PDFLATEX) PG_4_2.tex
4245 ▷ $(OPEN) PG_4_2.pdf
4246
4247 PG_4_3:
4248 ▷ $(ORBITER) -v 2 \
4249 ▷ ▷ -define Do -draw_options -end \
4250 ▷ ▷ -define F -finite_field -q 3 -end \
4251 ▷ ▷ -define P -projective_space -n 4 -field F -v 0 -end \
4252 ▷ ▷ -with P -do -projective_space_activity \
4253 ▷ ▷ ▷ -cheat_sheet Do \
4254 ▷ ▷ -end
4255 ▷ $(PDFLATEX) PG_4_3.tex
4256 ▷ $(OPEN) PG_4_3.pdf
4257
4258
4259 PG_8_2:
4260 ▷ $(ORBITER) -v 2 \
4261 ▷ ▷ -define Do -draw_options -end \
4262 ▷ ▷ -define F -finite_field -q 2 -end \
4263 ▷ ▷ -define P -projective_space -n 8 -field F -v 0 -end \
4264 ▷ ▷ -with P -do -projective_space_activity \
4265 ▷ ▷ ▷ -cheat_sheet Do \
4266 ▷ ▷ -end
4267 ▷ $(PDFLATEX) PG_8_2.tex
4268 ▷ $(OPEN) PG_8_2.pdf
4269
4270
4271
4272
4273 #####
4274 # Section 4.2: Indexing Points and Lines
4275
4276 SECTION_INDEXING_POINTS_AND_LINES:
4277
4278 test_4_2:
4279 ▷ make PG_2_4_rank_point
4280 ▷ make PG_2_4_unrank_point
4281 ▷ make PG_2_4_print_points
4282 ▷ make PG_2_2_rank_lines
4283 ▷ make PG_2_2_unrank_lines
4284 ▷ make PG_3_3_rank_lines
4285 ▷ make PG_3_5_unrank_lines
4286
4287
4288 PG_2_4_rank_point:
4289 ▷ $(ORBITER) -v 2 \
4290 ▷ ▷ -define v -vector -dense "3,3,1" -format 1 -end \
4291 ▷ ▷ -define F -finite_field -q 4 -end \
4292 ▷ ▷ -with F -do -finite_field_activity \
4293 ▷ ▷ ▷ -rank_point_in_PG v \
4294 ▷ ▷ -end
4295
4296 # geometry_global::do_rank_point_in_PG coeff: ( 3, 3, 1 ) has rank 20
4297
4298 PG_2_4_unrank_point:

```



```

4299 ▷ $(ORBITER) -v 2 \
4300 ▷ ▷ -define v -vector -dense "20" -end \
4301 ▷ ▷ -define F -finite_field -q 4 -end \
4302 ▷ ▷ -with F -do -finite_field_activity \
4303 ▷ ▷ ▷ -unrank_point_in_PG 2 v \
4304 ▷ ▷ -end
4305
4306
4307
4308 PG_2_4_print_points:
4309 ▷ $(ORBITER) -v 6 \
4310 ▷ ▷ -define Pts -vector \
4311 ▷ ▷ ▷ -loop 0 21 1 \
4312 ▷ ▷ -end \
4313 ▷ ▷ -define F -finite_field -q 4 -end \
4314 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
4315 ▷ ▷ -with P -do \
4316 ▷ ▷ -projective_space_activity \
4317 ▷ ▷ ▷ -print_points Pts \
4318 ▷ ▷ -end
4319 ▷ $(PDFLATEX) PG_2_4_points.tex
4320 ▷ $(OPEN) PG_2_4_points.pdf
4321
4322
4323
4324 PG_2_2_rank_lines:
4325 ▷ $(ORBITER) -v 2 \
4326 ▷ ▷ -define v -vector -format 4 \
4327 ▷ ▷ -dense "1,0,0, 0,1,0, 1,0,0, 0,0,1, 1,1,1, 0,1,0, 1,1,1, 0,0,1" \
4328 ▷ ▷ -end \
4329 ▷ ▷ -define F -finite_field -q 2 -end \
4330 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
4331 ▷ ▷ -with P -do \
4332 ▷ ▷ -projective_space_activity \
4333 ▷ ▷ ▷ -rank_lines_in_PG v \
4334 ▷ ▷ -end \
4335
4336
4337 PG_2_2_unrank_lines:
4338 ▷ $(ORBITER) -v 2 \
4339 ▷ ▷ -define v -vector \
4340 ▷ ▷ -dense "0,1,2,3,4,5,6" \
4341 ▷ ▷ -end \
4342 ▷ ▷ -define F -finite_field -q 2 -end \
4343 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
4344 ▷ ▷ -with P -do \
4345 ▷ ▷ -projective_space_activity \
4346 ▷ ▷ ▷ -unrank_lines_in_PG v \
4347 ▷ ▷ -end \
4348
4349
4350
4351 PG_3_3_rank_lines:
4352 ▷ $(ORBITER) -v 2 \
4353 ▷ ▷ -define v1 -vector -format 3 \
4354 ▷ ▷ -dense "1,0,2,2,0,1,1,2, 1,0,2,0,0,1,1,2, 1,0,2,2,0,1,2,1" \
4355 ▷ ▷ -end \
4356 ▷ ▷ -define v2 -vector -format 3 \
4357 ▷ ▷ -dense "1,0,0,0,0,1,0,0, 1,0,0,0,0,0,0,1, 0,1,0,0,0,0,2,1" \

```

```

4358 ▷ ▷ -end \
4359 ▷ ▷ -define F -finite.field -q 3 -end \
4360 ▷ ▷ -define P -projective.space -n 3 -field F -v 0 -end \
4361 ▷ ▷ -with P -do \
4362 ▷ ▷ -projective.space.activity \
4363 ▷ ▷ ▷ -rank.lines.in_PG v1 \
4364 ▷ ▷ -end \
4365 ▷ ▷ -with P -do \
4366 ▷ ▷ -projective.space.activity \
4367 ▷ ▷ ▷ -rank.lines.in_PG v2 \
4368 ▷ ▷ -end
4369
4370
4371 PG_3_5_unrank.lines:
4372 ▷ $(ORBITER) -v 2 \
4373 ▷ ▷ -define v -vector \
4374 ▷ ▷ ▷ -dense "0,36,72,108,144,805" \
4375 ▷ ▷ -end \
4376 ▷ ▷ -define F -finite.field -q 5 -end \
4377 ▷ ▷ -define P -projective.space -n 3 -field F -v 0 -end \
4378 ▷ ▷ -with P -do \
4379 ▷ ▷ -projective.space.activity \
4380 ▷ ▷ ▷ -unrank.lines.in_PG v \
4381 ▷ ▷ -end
4382
4383
4384
4385
4386
4387
4388 #####
4389 # Section 4.3: Incidence Matrices
4390
4391
4392 SECTION_INCIDENCE_MATRICES:
4393
4394
4395 test_4_3:
4396 ▷ make PG_2_16
4397 ▷ make PG_2_2_incidence_matrix
4398 ▷ make PG_2_4_incidence_matrix
4399 ▷ make PG_2_8_incidence_matrix
4400 ▷ make PG_2_4_with_decomposition
4401 ▷ make PG_2_4_incma_cyclic
4402 ▷ make PG_2_4_incma_singer_sub_3
4403 ▷ make PG_2_4_incma_singer_sub_7
4404
4405
4406 PG_2_16:
4407 ▷ $(ORBITER) -v 2 \
4408 ▷ ▷ -define Do -draw.options \
4409 ▷ ▷ ▷ -radius 200 -line.width 0.3 -nodes.empty \
4410 ▷ ▷ -end \
4411 ▷ ▷ -define F -finite.field -q 16 -compute.related.fields -end \
4412 ▷ ▷ -define P -projective.space -n 2 -field F -v 0 -end \
4413 ▷ ▷ -with P -do -projective.space.activity \
4414 ▷ ▷ ▷ -cheat.sheet Do \
4415 ▷ ▷ -end
4416 ▷ $(PDFLATEX) PG_2_16.tex

```

```

4417 ▷ $(OPEN) PG_2_16.pdf
4418
4419 #magma_interface::export_matrix_group_with_field_reduction related fields have no
    t yet been computed
4420
4421
4422 PG_2_2.incidence_matrix:
4423 ▷ $(ORBITER) -v 2 \
4424 ▷ ▷ -define F -finite_field -q 2 -end \
4425 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
4426 ▷ ▷ -with P -do -projective_space_activity \
4427 ▷ ▷ ▷ -export_point_line_incidence_matrix \
4428 ▷ ▷ -end
4429 ▷ $(ORBITER) -v 2 \
4430 ▷ ▷ -define all_one -vector -repeat 1 7 -end \
4431 ▷ ▷ -draw_matrix \
4432 ▷ ▷ ▷ -input_csv_file PG_n2_q2_incidence_matrix.csv \
4433 ▷ ▷ ▷ -box_width 20 -bit_depth 8 \
4434 ▷ ▷ ▷ -partition 3 \
4435 ▷ ▷ ▷ ▷ all_one all_one \
4436 ▷ ▷ -end
4437 ▷ $(OPEN) PG_n2_q2_incidence_matrix_draw.bmp
4438 ▷ magick PG_n2_q2_incidence_matrix_draw.bmp PG_n2_q2_incidence_matrix_draw.png
4439
4440
4441 PG_2_4.incidence_matrix:
4442 ▷ $(ORBITER) -v 2 \
4443 ▷ ▷ -define F -finite_field -q 4 -end \
4444 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
4445 ▷ ▷ -with P -do -projective_space_activity \
4446 ▷ ▷ ▷ -export_point_line_incidence_matrix \
4447 ▷ ▷ -end
4448 ▷ $(ORBITER) -v 2 \
4449 ▷ ▷ -define all_one -vector -repeat 1 21 -end \
4450 ▷ ▷ -draw_matrix \
4451 ▷ ▷ ▷ -input_csv_file PG_n2_q4_incidence_matrix.csv \
4452 ▷ ▷ ▷ -box_width 20 -bit_depth 8 \
4453 ▷ ▷ ▷ -partition 3 \
4454 ▷ ▷ ▷ ▷ all_one all_one \
4455 ▷ ▷ -end
4456 ▷ $(OPEN) PG_n2_q4_incidence_matrix_draw.bmp
4457 ▷ magick PG_n2_q4_incidence_matrix_draw.bmp PG_n2_q4_incidence_matrix_draw.png
4458
4459 # writes PG_n2_q4_incidence_matrix.csv
4460
4461
4462
4463 PG_2_8.incidence_matrix:
4464 ▷ $(ORBITER) -v 2 \
4465 ▷ ▷ -define F -finite_field -q 8 -end \
4466 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
4467 ▷ ▷ -with P -do -projective_space_activity \
4468 ▷ ▷ ▷ -export_point_line_incidence_matrix \
4469 ▷ ▷ -end
4470 ▷ $(ORBITER) -v 2 \
4471 ▷ ▷ -define all_one -vector -repeat 1 73 -end \
4472 ▷ ▷ -draw_matrix \
4473 ▷ ▷ ▷ -input_csv_file PG_n2_q8_incidence_matrix.csv \
4474 ▷ ▷ ▷ -box_width 20 -bit_depth 8 \

```

```

4475 ▷ ▷ ▷ -partition 3 \
4476 ▷ ▷ ▷ ▷ all_one all_one \
4477 ▷ ▷ -end
4478 ▷ $(OPEN) PG_n2_q8_incidence_matrix_draw.bmp
4479 ▷ magick PG_n2_q8_incidence_matrix_draw.bmp PG_n2_q8_incidence_matrix_draw.png
4480
4481
4482
4483
4484 PG_2_4_with_decomposition:
4485 ▷ $(ORBITER) -v 2 \
4486 ▷ ▷ -define F -finite_field -q 4 -end \
4487 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
4488 ▷ ▷ -with P -do -projective_space_activity \
4489 ▷ ▷ ▷ -decomposition_by_element_PG \
4490 ▷ ▷ ▷ 1 "0,1,0, 0,0,1, 2,1,1, 0" \
4491 ▷ ▷ ▷ "PG_2_4.singer" \
4492 ▷ ▷ -end
4493 ▷ $(PDFLATEX) PG_2_4.singer.tex
4494 ▷ $(OPEN) PG_2_4.singer.pdf
4495
4496
4497 #PG_2_4.singer_incma_cyclic.csv
4498 #PG_2_4.singer_incma_subgroup_index_3.csv
4499 #PG_2_4.singer_incma_subgroup_index_7.csv
4500
4501
4502
4503 PG_2_4_incma_cyclic:
4504 ▷ $(ORBITER) -v 2 \
4505 ▷ ▷ -list_arguments \
4506 ▷ ▷ -define R -vector -repeat 1 21 -end \
4507 ▷ ▷ -define C -vector -repeat 1 21 -end \
4508 ▷ ▷ -draw_matrix \
4509 ▷ ▷ ▷ -input_csv_file PG_2_4.singer_incma_cyclic.csv \
4510 ▷ ▷ ▷ -box_width 40 -bit_depth 24 \
4511 ▷ ▷ ▷ -partition 3 R C \
4512 ▷ ▷ -end
4513 ▷ $(OPEN) PG_2_4.singer_incma_cyclic_draw.bmp
4514 ▷ magick PG_2_4.singer_incma_cyclic_draw.bmp PG_2_4.singer_incma_cyclic_draw.png
4515
4516
4517 PG_2_4_incma_singer_sub_3:
4518 ▷ $(ORBITER) -v 2 \
4519 ▷ ▷ -list_arguments \
4520 ▷ ▷ -define R -vector -repeat 3 7 -end \
4521 ▷ ▷ -define C -vector -repeat 3 7 -end \
4522 ▷ ▷ -draw_matrix \
4523 ▷ ▷ ▷ -input_csv_file PG_2_4.singer_incma_subgroup_index_3.csv \
4524 ▷ ▷ ▷ -box_width 40 -bit_depth 24 \
4525 ▷ ▷ ▷ -partition 3 R C \
4526 ▷ ▷ -end
4527 ▷ $(OPEN) PG_2_4.singer_incma_subgroup_index_3_draw.bmp
4528 ▷ magick PG_2_4.singer_incma_subgroup_index_3_draw.bmp PG_2_4.singer_incma_subgro
up_index_3_draw.png
4529
4530 PG_2_4_incma_singer_sub_7:
4531 ▷ $(ORBITER) -v 2 \
4532 ▷ ▷ -draw_matrix \

```

```

4533 ▷ ▷ ▷ -input_csv_file PG_2.4.singer_incma_subgroup_index.7.csv \
4534 ▷ ▷ ▷ -box_width 20 -bit_depth 24 \
4535 ▷ ▷ ▷ -partition 3 3,3,3,3,3,3 3,3,3,3,3,3 \
4536 ▷ ▷ -end
4537 ▷ $(OPEN) PG_2.4.singer_incma_subgroup_index.7_draw.bmp
4538 ▷ magick PG_2.4.singer_incma_subgroup_index.7_draw.bmp PG_2.4.singer_incma_subgro
up_index.7_draw.png
4539
4540
4541
4542
4543
4544
4545
4546
4547
4548
4549 #####
4550 # Section 4.4: Projective Space Advanced Topics
4551
4552
4553 SECTION_PROJECTIVE_SPACE_ADVANCED_TOPICS:
4554
4555
4556 test_4.4:
4557 ▷ make fixed_objects_2A
4558 ▷ make fixed_objects_2B
4559 ▷ make fixed_objects_2C
4560 ▷ make fix_structure.G1
4561 ▷ make trans
4562 ▷ make PG_2.3.plane.pencil
4563
4564
4565
4566 fixed_objects_2A:
4567 ▷ $(ORBITER) -v 2 \
4568 ▷ ▷ -define F -finite_field -q 4 -end \
4569 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4570 ▷ ▷ -with P -do \
4571 ▷ ▷ -projective_space_activity \
4572 ▷ ▷ ▷ -report_fixed_objects \
4573 ▷ ▷ ▷ ▷ "1,0,0,0, 0,1,0,0, 0,0,1,0, 0,0,0,1, 1" \
4574 ▷ ▷ ▷ ▷ fix_structure_2A.tex \
4575 ▷ ▷ -end
4576 ▷ $(PDFLATEX) fix_structure_2A.tex
4577 ▷ $(OPEN) fix_structure_2A.pdf
4578
4579
4580 fixed_objects_2B:
4581 ▷ $(ORBITER) -v 2 \
4582 ▷ ▷ -define F -finite_field -q 4 -end \
4583 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4584 ▷ ▷ -with P -do \
4585 ▷ ▷ -projective_space_activity \
4586 ▷ ▷ ▷ -report_fixed_objects \
4587 ▷ ▷ ▷ ▷ "1,0,0,0, 1,1,0,0, 0,0,1,0, 0,0,0,1, 0" \
4588 ▷ ▷ ▷ ▷ fix_structure_2B.tex \
4589 ▷ ▷ -end
4590 ▷ $(PDFLATEX) fix_structure_2B.tex

```

```

4591 ▷ $(OPEN) fix_structure_2B.pdf
4592
4593
4594 fixed_objects_2C:
4595 ▷ $(ORBITER) -v 2 \
4596 ▷ ▷ -define F -finite_field -q 4 -end \
4597 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4598 ▷ ▷ -with P -do \
4599 ▷ ▷ -projective_space_activity \
4600 ▷ ▷ ▷ -report_fixed_objects \
4601 ▷ ▷ ▷ ▷ "1,0,0,0, 1,1,0,0, 0,0,1,0, 0,0,1,1, 0" \
4602 ▷ ▷ ▷ ▷ fix_structure_2C.tex \
4603 ▷ ▷ -end
4604 ▷ $(PDFLATEX) fix_structure_2C.tex
4605 ▷ $(OPEN) fix_structure_2C.pdf
4606
4607
4608
4609
4610 fix_structure_G1:
4611 ▷ $(ORBITER) -v 2 \
4612 ▷ ▷ -define F -finite_field -q 4 -end \
4613 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4614 ▷ ▷ -with P -do \
4615 ▷ ▷ -projective_space_activity \
4616 ▷ ▷ ▷ -decomposition_by_subgroup \
4617 ▷ ▷ ▷ ▷ $(H1_LABEL) $(H1_GENS) \
4618 ▷ ▷ -end
4619 ▷ $(PDFLATEX) PGGL_4_4_Subgroup_$(H1_LABEL)_4_decomp.tex
4620 ▷ $(OPEN) PGGL_4_4_Subgroup_$(H1_LABEL)_4_decomp.pdf
4621
4622
4623
4624
4625 trans:
4626 ▷ $(ORBITER) -v 5 \
4627 ▷ ▷ -define F -finite_field -q 16 -end \
4628 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4629 ▷ ▷ -with P -do \
4630 ▷ ▷ -projective_space_activity \
4631 ▷ ▷ ▷ -move_two_lines_in_hyperplane_stabilizer_text \
4632 ▷ ▷ ▷ ▷ "1,0,0,0, 0,0,0,1" "1,1,0,2, 0,0,1,0" \
4633 ▷ ▷ ▷ ▷ "1,0,0,0, 0,0,0,1" "0,1,0,1, 0,0,1,0" \
4634 ▷ ▷ -end
4635
4636
4637 PG_2_3_plane_pencil:
4638 ▷ $(ORBITER) -v 2 \
4639 ▷ ▷ -define F -finite_field -q 2 -end \
4640 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4641 ▷ ▷ -with P -do \
4642 ▷ ▷ -projective_space_activity \
4643 ▷ ▷ ▷ -planes_through_line 0 \
4644 ▷ ▷ -end \
4645
4646
4647
4648
4649 #####

```

```

4650 # Section 4.5: Geometric Objects
4651
4652
4653 SECTION_GEOMETRIC_OBJECTS:
4654
4655
4656
4657 test_4.5:
4658 ▷ make elliptic_quadric_ovoid_q8
4659 ▷ make ovoid_ST_q8
4660 ▷ make Baer_PG_2.4
4661 ▷ make Baer_PG_3.4
4662 ▷ make BLT_database_5_0
4663 ▷ make BLT_database_7_0
4664 ▷ make BLT_database_7_0_print
4665 ▷ make BLT_database_67_4
4666
4667
4668
4669 elliptic_quadric_ovoid_q8:
4670 ▷ $(ORBITER) -v 2 \
4671 ▷ ▷ -define F -finite_field -q 8 -end \
4672 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4673 ▷ ▷ -define Geo -geometric_object P \
4674 ▷ ▷ ▷ -elliptic_quadric_ovoid \
4675 ▷ ▷ -end \
4676 ▷ ▷ -define C -combinatorial_object \
4677 ▷ ▷ ▷ -label ovoid ovoid \
4678 ▷ ▷ ▷ -geometric_object Geo \
4679 ▷ ▷ -end
4680
4681 #ovoid_q8.txt
4682 # 65 points
4683
4684 ovoid_ST_q8:
4685 ▷ $(ORBITER) -v 2 \
4686 ▷ ▷ -define F -finite_field -q 8 -end \
4687 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4688 ▷ ▷ -define Geo -geometric_object P \
4689 ▷ ▷ ▷ -ovoid_ST \
4690 ▷ ▷ -end \
4691 ▷ ▷ -define C -combinatorial_object \
4692 ▷ ▷ ▷ -label ovoid_ST ovoid_ST \
4693 ▷ ▷ ▷ -geometric_object Geo \
4694 ▷ ▷ -end
4695
4696 #ovoid_ST_q8.txt
4697
4698
4699 Baer_PG_2.4:
4700 ▷ $(ORBITER) -v 2 \
4701 ▷ ▷ -define F -finite_field -q 4 -end \
4702 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
4703 ▷ ▷ -define Geo -geometric_object P \
4704 ▷ ▷ ▷ -Baer_substructure \
4705 ▷ ▷ -end \
4706 ▷ ▷ -define C -combinatorial_object \
4707 ▷ ▷ ▷ -label Baer Baer \
4708 ▷ ▷ ▷ -geometric_object Geo \

```

```

4709 ▷ ▷ -end \
4710 ▷ ▷ -with C -do -combinatorial_object_activity -save \
4711 ▷ ▷ -end
4712
4713 Baer_PG_3.4:
4714 ▷ $(ORBITER) -v 2 \
4715 ▷ ▷ -define F -finite_field -q 4 -end \
4716 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
4717 ▷ ▷ -define Geo -geometric_object P \
4718 ▷ ▷ ▷ -Baer_substructure \
4719 ▷ ▷ -end \
4720 ▷ ▷ -define C -combinatorial_object \
4721 ▷ ▷ ▷ -label Baer Baer \
4722 ▷ ▷ ▷ -geometric_object Geo \
4723 ▷ ▷ -end
4724
4725 BLT_database_5.0:
4726 ▷ $(ORBITER) -v 2 \
4727 ▷ ▷ -define F -finite_field -q 5 -end \
4728 ▷ ▷ -define P -projective_space -n 4 -field F -v 0 -end \
4729 ▷ ▷ -define O -geometric_object P \
4730 ▷ ▷ ▷ -BLT_database 0 \
4731 ▷ ▷ -end \
4732 ▷ ▷ -with O -do -combinatorial_object_activity -save \
4733 ▷ ▷ -end
4734
4735 # writes BLT_5.0.txt
4736
4737
4738 BLT_database_7.0:
4739 ▷ $(ORBITER) -v 2 \
4740 ▷ ▷ -define F -finite_field -q 7 -end \
4741 ▷ ▷ -define P -projective_space -n 4 -field F -v 0 -end \
4742 ▷ ▷ -define O -geometric_object P \
4743 ▷ ▷ ▷ -BLT_database 0 \
4744 ▷ ▷ -end \
4745 ▷ ▷ -with O -do -combinatorial_object_activity -save \
4746 ▷ ▷ -end
4747
4748 # writes BLT_7.0.txt
4749
4750
4751 BLT_database_7.0_print:
4752 ▷ $(ORBITER) -v 2 \
4753 ▷ ▷ -define F -finite_field -q 7 -end \
4754 ▷ ▷ -define P -projective_space -n 4 -field F -v 0 -end \
4755 ▷ ▷ -define Obj -geometric_object P \
4756 ▷ ▷ ▷ -BLT_database 0 \
4757 ▷ ▷ -end \
4758 ▷ ▷ -with Obj -do -combinatorial_object_activity -save \
4759 ▷ ▷ -end \
4760 ▷ ▷ -define O -orthogonal_space 0 5 F -without_group -end \
4761 ▷ ▷ -define BLT_7.0 -set -file_orbiter_format BLT_7.0.txt -end \
4762 ▷ ▷ -with O -do -orthogonal_space_activity \
4763 ▷ ▷ ▷ -print_points BLT_7.0 \
4764 ▷ ▷ -end
4765 ▷ $(PDFLATEX) BLT_7.0_set_report.tex
4766 ▷ $(OPEN) BLT_7.0_set_report.pdf
4767

```



```

4768
4769 BLT_database_67_4:
4770 ▷ $(ORBITER) -v 2 \
4771 ▷ ▷ -define F -finite_field -q 67 -end \
4772 ▷ ▷ -define P -projective_space -n 4 -field F -v 0 -end \
4773 ▷ ▷ -define Obj -geometric_object P \
4774 ▷ ▷ ▷ -BLT_database 4 \
4775 ▷ ▷ -end \
4776 ▷ ▷ -with Obj -do -combinatorial_object_activity -save \
4777 ▷ ▷ -end \
4778 ▷ ▷ -define O -orthogonal_space 0 5 F -without_group -end \
4779 ▷ ▷ -define BLT_67_4 -set -file_orbiter_format BLT_67_4.txt -end \
4780 ▷ ▷ -with O -do -orthogonal_space_activity \
4781 ▷ ▷ ▷ -print_points BLT_67_4 \
4782 ▷ ▷ -end
4783 ▷ $(PDFLATEX) BLT_67_4_set_report.tex
4784 ▷ $(OPEN) BLT_67_4_set_report.pdf
4785
4786
4787
4788
4789
4790
4791
4792
4793
4794
4795
4796
4797 #####
4798 # Chapter 5 - Group Theory
4799 #####
4800
4801
4802 test_5:
4803 ▷ make test_5.1
4804 ▷ make test_5.2
4805 ▷ make test_5.3
4806 ▷ make test_5.4
4807 ▷ make test_5.5
4808 ▷ make test_5.6 # Magma stuff
4809 ▷ make test_5.7 # gap stuff
4810 ▷ make test_5.8 # Linear Groups, Advanced Topics
4811 ▷ make test_5.9 # activities for vectors of group elements
4812
4813
4814
4815 #####
4816 # Section 5.1: Permutation groups
4817
4818
4819 SECTION_PERMUTATION_GROUPS:
4820
4821 test_5.1:
4822 ▷ make Cyclic_6
4823 ▷ make Cyclic_6_from_generator
4824 ▷ make Cyclic_6_group_table
4825 ▷ make Symmetric_3
4826 ▷ make Symmetric_3_group_table

```

```

4827 ▷ make Symmetric_3.elements
4828 ▷ make Symmetric_3.long
4829 ▷ make Elementary_abelian_8
4830 ▷ make Symmetric_4
4831 ▷ make Symmetric_4.group.table
4832 ▷ make Symmetric_4.long
4833 ▷ make Group_8.1
4834 ▷ make Group_8.2
4835 ▷ make Group_8.3
4836 ▷ make Group_8.4
4837 ▷ make Group_8.5
4838 ▷ make Group_M12
4839
4840
4841 Cyclic_6:
4842 ▷ $(ORBITER) -v 3 \
4843 ▷ ▷ -define Do -draw.options \
4844 ▷ ▷ ▷ -radius 200 -line.width 0.3 \
4845 ▷ ▷ -end \
4846 ▷ ▷ -define G -permutation_group -cyclic_group 6 -end \
4847 ▷ ▷ -with G -do \
4848 ▷ ▷ ▷ -group.theoretic.activity \
4849 ▷ ▷ ▷ -report Do \
4850 ▷ ▷ -end
4851 ▷ $(PDFLATEX) C_6.report.tex
4852 ▷ $(OPEN) C_6.report.pdf
4853
4854
4855 Cyclic_6_from_generator:
4856 ▷ $(ORBITER) -v 2 \
4857 ▷ ▷ -define Do -draw.options \
4858 ▷ ▷ ▷ -radius 200 -line.width 0.3 \
4859 ▷ ▷ -end \
4860 ▷ ▷ -define gens -vector -dense \
4861 ▷ ▷ ▷ "1,2,3,4,5,0" \
4862 ▷ ▷ -end \
4863 ▷ ▷ -define G -permutation_group -symmetric_group 6 \
4864 ▷ ▷ ▷ -subgroup_by_generators G1 6 1 gens -end \
4865 ▷ ▷ -with G -do \
4866 ▷ ▷ -group.theoretic.activity \
4867 ▷ ▷ ▷ -report Do \
4868 ▷ ▷ -end
4869 ▷ $(PDFLATEX) Sym_6_Subgroup_G1_6.report.tex
4870 ▷ $(OPEN) Sym_6_Subgroup_G1_6.report.pdf
4871
4872
4873 Cyclic_6_group_table:
4874 ▷ $(ORBITER) -v 3 \
4875 ▷ ▷ -define G -permutation_group -cyclic_group 6 -end \
4876 ▷ ▷ -with G -do \
4877 ▷ ▷ -group.theoretic.activity \
4878 ▷ ▷ ▷ -export_group_table \
4879 ▷ ▷ -end
4880 ▷ $(ORBITER) -v 2 \
4881 ▷ ▷ -define all_one_r -vector -repeat 1 6 -end \
4882 ▷ ▷ -define all_one_c -vector -repeat 1 6 -end \
4883 ▷ ▷ -draw_matrix \
4884 ▷ ▷ ▷ -input_csv_file C_6.group_table.csv \
4885 ▷ ▷ ▷ -box_width 50 -bit_depth 24 \

```

```

4886 ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
4887 ▷ ▷ -end
4888 ▷ $(OPEN) C_6_group_table_draw.bmp
4889 ▷ magick C_6_group_table_draw.bmp C_6_group_table_draw.png
4890
4891
4892 Symmetric_3:
4893 ▷ $(ORBITER) -v 3 \
4894 ▷ ▷ -define Do -draw_options \
4895 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
4896 ▷ ▷ -end \
4897 ▷ ▷ -define G -permutation_group -symmetric_group 3 -end \
4898 ▷ ▷ -with G -do \
4899 ▷ ▷ -group_theoretic_activity \
4900 ▷ ▷ ▷ -report Do \
4901 ▷ ▷ -end
4902 ▷ $(PDFLATEX) Sym_3_report.tex
4903 ▷ $(OPEN) Sym_3_report.pdf
4904
4905
4906 Symmetric_3_group_table:
4907 ▷ $(ORBITER) -v 3 \
4908 ▷ ▷ -define G -permutation_group -symmetric_group 3 -end \
4909 ▷ ▷ -with G -do \
4910 ▷ ▷ -group_theoretic_activity \
4911 ▷ ▷ ▷ -export_group_table \
4912 ▷ ▷ -end
4913 ▷ $(ORBITER) -v 2 \
4914 ▷ ▷ -define all_one_r -vector -repeat 1 6 -end \
4915 ▷ ▷ -define all_one_c -vector -repeat 1 6 -end \
4916 ▷ ▷ -draw_matrix \
4917 ▷ ▷ ▷ -input_csv_file Sym_3_group_table.csv \
4918 ▷ ▷ ▷ -box_width 50 \
4919 ▷ ▷ ▷ -bit_depth 24 \
4920 ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
4921 ▷ ▷ -end
4922 ▷ $(OPEN) Sym_3_group_table_draw.bmp
4923
4924 Symmetric_3_elements:
4925 ▷ $(ORBITER) -v 3 \
4926 ▷ ▷ -define G -permutation_group -symmetric_group 3 -end \
4927 ▷ ▷ -with G -do \
4928 ▷ ▷ -group_theoretic_activity \
4929 ▷ ▷ ▷ -save_elements_csv "Symmetric3_elts.csv" \
4930 ▷ ▷ -end
4931 ▷ $(ORBITER) -v 2 \
4932 ▷ ▷ -define Sym3_elts -vector -load_csv_data_column \
4933 ▷ ▷ ▷ Symmetric3_elts.csv 1 -end \
4934 ▷ ▷ -save_matrix_csv Sym3_elts
4935 ▷ $(ORBITER) -v 2 \
4936 ▷ ▷ -define all_one_r -vector -repeat 1 6 -end \
4937 ▷ ▷ -define all_one_c -vector -repeat 1 3 -end \
4938 ▷ ▷ -draw_matrix \
4939 ▷ ▷ ▷ -input_csv_file Sym3_elts_matrix.csv \
4940 ▷ ▷ ▷ -box_width 50 \
4941 ▷ ▷ ▷ -bit_depth 8 \
4942 ▷ ▷ ▷ -partition 3 \
4943 ▷ ▷ ▷ ▷ all_one_r all_one_c \
4944 ▷ ▷ -end

```

```

4945 ▷ $(OPEN) Sym3.elts.matrix.draw.bmp
4946
4947 Symmetric_3_long:
4948 ▷ $(ORBITER) -v 3 \
4949 ▷ ▷ -define Do -draw.options \
4950 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
4951 ▷ ▷ -end \
4952 ▷ ▷ -define G -permutation_group -symmetric_group 3 -end \
4953 ▷ ▷ -with G -do \
4954 ▷ ▷ -group_theoretic_activity \
4955 ▷ ▷ ▷ -export_orbiter \
4956 ▷ ▷ -end \
4957 ▷ ▷ -with G -do \
4958 ▷ ▷ -group_theoretic_activity \
4959 ▷ ▷ ▷ -print_elements_tex \
4960 ▷ ▷ -end \
4961 ▷ ▷ -with G -do \
4962 ▷ ▷ -group_theoretic_activity \
4963 ▷ ▷ ▷ -report Do \
4964 ▷ ▷ -end \
4965 ▷ ▷ -with G -do \
4966 ▷ ▷ -group_theoretic_activity \
4967 ▷ ▷ ▷ -save_elements_csv "Symmetric3.elts.csv" \
4968 ▷ ▷ -end
4969 ▷ $(ORBITER) -v 3 \
4970 ▷ ▷ -define Do -draw.options \
4971 ▷ ▷ ▷ -nodes \
4972 ▷ ▷ ▷ -embedded -radius 250 \
4973 ▷ ▷ ▷ -xin 10000 -yin 10000 \
4974 ▷ ▷ ▷ -xout 1000000 -yout 600000 \
4975 ▷ ▷ ▷ -scale 0.3 -line_width 1.0 \
4976 ▷ ▷ -end \
4977 ▷ ▷ -tree_draw \
4978 ▷ ▷ ▷ -file Sym_3_elements_tree.txt \
4979 ▷ ▷ ▷ -draw.options Do \
4980 ▷ ▷ -end
4981 ▷ $(ORBITER) -v 2 \
4982 ▷ ▷ -define M -vector -load_csv_data_column \
4983 ▷ ▷ ▷ Symmetric3.elts.csv 1 \
4984 ▷ ▷ -end \
4985 ▷ ▷ -save_matrix_csv M
4986 ▷ $(ORBITER) -v 2 \
4987 ▷ ▷ -define all_one_r -vector -repeat 1 6 -end \
4988 ▷ ▷ -define all_one_c -vector -repeat 1 3 -end \
4989 ▷ ▷ -draw_matrix \
4990 ▷ ▷ ▷ -input_csv_file M.matrix.csv \
4991 ▷ ▷ ▷ -box_width 50 -bit_depth 8 \
4992 ▷ ▷ ▷ -partition 3 \
4993 ▷ ▷ ▷ ▷ all_one_r all_one_c \
4994 ▷ ▷ -end
4995 ▷ $(PDFLATEX) Sym_3_elements_tree_draw.tex
4996 ▷ $(OPEN) Sym_3_elements_tree_draw.pdf
4997 ▷ #$(PDFLATEX) Perm3_report.tex
4998 ▷ #$(OPEN) Perm3_report.pdf
4999
5000
5001 Elementary_abelian_8:
5002 ▷ $(ORBITER) -v 3 \
5003 ▷ ▷ -define Do -draw.options \

```

```

5004 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5005 ▷ ▷ -end \
5006 ▷ ▷ -define G -permutation_group -elementary_abelian_group 8 -end \
5007 ▷ ▷ -with G -do \
5008 ▷ ▷ ▷ -group_theoretic_activity \
5009 ▷ ▷ ▷ -report Do \
5010 ▷ ▷ -end
5011 ▷ $(PDFLATEX) E_8_report.tex
5012 ▷ $(OPEN) E_8_report.pdf
5013
5014
5015 Symmetric_4:
5016 ▷ $(ORBITER) -v 3 \
5017 ▷ ▷ -define Do -draw_options \
5018 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5019 ▷ ▷ -end \
5020 ▷ ▷ -define G -permutation_group -symmetric_group 4 -end \
5021 ▷ ▷ -with G -do \
5022 ▷ ▷ -group_theoretic_activity \
5023 ▷ ▷ ▷ -report Do \
5024 ▷ ▷ -end
5025 ▷ $(PDFLATEX) Sym_4_report.tex
5026 ▷ $(OPEN) Sym_4_report.pdf
5027
5028
5029 Symmetric_4_group_table:
5030 ▷ $(ORBITER) -v 3 \
5031 ▷ ▷ -define G -permutation_group -symmetric_group 4 -end \
5032 ▷ ▷ -with G -do \
5033 ▷ ▷ -group_theoretic_activity \
5034 ▷ ▷ ▷ -export_group_table \
5035 ▷ ▷ -end
5036 ▷ $(ORBITER) -v 2 \
5037 ▷ ▷ -define all_one_r -vector -repeat 1 24 -end \
5038 ▷ ▷ -define all_one_c -vector -repeat 1 24 -end \
5039 ▷ ▷ -draw_matrix \
5040 ▷ ▷ ▷ -input_csv_file Sym_4_group_table.csv \
5041 ▷ ▷ ▷ -box_width 50 \
5042 ▷ ▷ ▷ -bit_depth 24 \
5043 ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
5044 ▷ ▷ -end
5045 ▷ $(OPEN) Sym_4_group_table.draw.bmp
5046
5047
5048
5049 Symmetric_4_long:
5050 ▷ $(ORBITER) -v 3 \
5051 ▷ ▷ -define Do -draw_options \
5052 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5053 ▷ ▷ -end \
5054 ▷ ▷ -define G -permutation_group -symmetric_group 4 -end \
5055 ▷ ▷ -with G -do \
5056 ▷ ▷ -group_theoretic_activity \
5057 ▷ ▷ ▷ -export_orbiter \
5058 ▷ ▷ -end \
5059 ▷ ▷ -with G -do \
5060 ▷ ▷ -group_theoretic_activity \
5061 ▷ ▷ ▷ -export_group_table \
5062 ▷ ▷ -end \

```

```

5063 ▷ ▷ -with G -do \
5064 ▷ ▷ -group_theoretic_activity \
5065 ▷ ▷ ▷ -print_elements_tex \
5066 ▷ ▷ -end \
5067 ▷ ▷ -with G -do \
5068 ▷ ▷ -group_theoretic_activity \
5069 ▷ ▷ ▷ -report Do \
5070 ▷ ▷ -end \
5071 ▷ ▷ -with G -do \
5072 ▷ ▷ -group_theoretic_activity \
5073 ▷ ▷ ▷ -save_elements_csv "Sym_4_elts.csv" \
5074 ▷ ▷ -end \
5075 ▷ ▷ -with G -do \
5076 ▷ ▷ -group_theoretic_activity \
5077 ▷ ▷ ▷ -make_element_tree \
5078 ▷ ▷ -end \
5079 ▷ ▷ -with G -do \
5080 ▷ ▷ -group_theoretic_activity \
5081 ▷ ▷ ▷ -export_inversion_graphs "Sym_4_inversion_graphs.csv" \
5082 ▷ ▷ -end
5083 ▷ $(ORBITER) -v 2 \
5084 ▷ ▷ -define Do -draw_options \
5085 ▷ ▷ ▷ -nodes \
5086 ▷ ▷ ▷ -embedded -radius 175 \
5087 ▷ ▷ ▷ -xin 10000 -yin 10000 \
5088 ▷ ▷ ▷ -xout 1500000 -yout 600000 \
5089 ▷ ▷ ▷ -scale 0.3 -line_width 1.0 \
5090 ▷ ▷ -end \
5091 ▷ ▷ -tree_draw \
5092 ▷ ▷ ▷ -file Sym_4_elements_tree.txt \
5093 ▷ ▷ ▷ -draw_options Do \
5094 ▷ ▷ -end
5095 ▷ $(ORBITER) -v 2 -draw_matrix \
5096 ▷ ▷ -input_csv_file Sym_4_group_table.csv \
5097 ▷ ▷ -box_width 50 -bit_depth 24 -end
5098 ▷ $(ORBITER) -v 2 \
5099 ▷ ▷ -define M -vector -load_csv_data_column \
5100 ▷ ▷ ▷ Sym_4_elts.csv 1 -end \
5101 ▷ ▷ -save_matrix_csv M
5102 ▷ $(ORBITER) -v 2 \
5103 ▷ ▷ -define all_one_r -vector -repeat 1 24 -end \
5104 ▷ ▷ -define all_one_c -vector -repeat 1 4 -end \
5105 ▷ ▷ -draw_matrix \
5106 ▷ ▷ ▷ -input_csv_file M_matrix.csv \
5107 ▷ ▷ ▷ -box_width 50 \
5108 ▷ ▷ ▷ -bit_depth 8 \
5109 ▷ ▷ ▷ -partition 3 \
5110 ▷ ▷ ▷ ▷ all_one_r all_one_c \
5111 ▷ ▷ -end
5112 ▷ $(PDFLATEX) Sym_4_elements_tree_draw.tex
5113 ▷ #$(OPEN) Sym_4_elements_tree_draw.pdf
5114
5115
5116
5117 # C8:
5118
5119 Group_8_1:
5120 ▷ $(ORBITER) -v 2 \
5121 ▷ ▷ -define gens -vector -dense \

```

```

5122 ▷ ▷ ▷ "1,2,3,4,5,6,7,0" \
5123 ▷ ▷ -end \
5124 ▷ ▷ -define G -permutation_group -symmetric_group 8 \
5125 ▷ ▷ ▷ -subgroup_by_generators G1 8 1 gens -end \
5126 ▷ ▷ -with G -do \
5127 ▷ ▷ -group_theoretic_activity \
5128 ▷ ▷ ▷ -export_group_table \
5129 ▷ ▷ -end
5130 ▷ $(ORBITER) -v 2 \
5131 ▷ ▷ -define all_one_r -vector -repeat 1 8 -end \
5132 ▷ ▷ -define all_one_c -vector -repeat 1 8 -end \
5133 ▷ ▷ -draw_matrix \
5134 ▷ ▷ ▷ -input_csv_file \
5135 ▷ ▷ ▷ ▷ Sym_8.Subgroup_G1_8.group_table.csv \
5136 ▷ ▷ ▷ -box_width 50 \
5137 ▷ ▷ ▷ -bit_depth 24 \
5138 ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
5139 ▷ ▷ -end
5140 ▷ magick Sym_8.Subgroup_G1_8.group_table.draw.bmp \
5141 ▷ ▷ Sym_8.Subgroup_G1_8.group_table.draw.png
5142 ▷ $(OPEN) Sym_8.Subgroup_G1_8.group_table.draw.png
5143
5144
5145
5146 # C4C2:
5147
5148 Group_8_2:
5149 ▷ $(ORBITER) -v 2 \
5150 ▷ ▷ -define gens -vector -dense \
5151 ▷ ▷ ▷ "1,2,3,0,4,5, 0,1,2,3,5,4" \
5152 ▷ ▷ -end \
5153 ▷ ▷ -define G -permutation_group -symmetric_group 6 \
5154 ▷ ▷ ▷ -subgroup_by_generators G2 8 2 gens -end \
5155 ▷ ▷ -with G -do \
5156 ▷ ▷ -group_theoretic_activity \
5157 ▷ ▷ ▷ -export_group_table \
5158 ▷ ▷ -end
5159 ▷ $(ORBITER) -v 2 \
5160 ▷ ▷ -define all_one_r -vector -repeat 1 8 -end \
5161 ▷ ▷ -define all_one_c -vector -repeat 1 8 -end \
5162 ▷ ▷ -draw_matrix \
5163 ▷ ▷ ▷ -input_csv_file \
5164 ▷ ▷ ▷ ▷ Sym_6.Subgroup_G2_8.group_table.csv \
5165 ▷ ▷ ▷ -box_width 50 \
5166 ▷ ▷ ▷ -bit_depth 24 \
5167 ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
5168 ▷ ▷ -end
5169 ▷ magick Sym_6.Subgroup_G2_8.group_table.draw.bmp \
5170 ▷ ▷ Sym_6.Subgroup_G2_8.group_table.draw.png
5171 ▷ $(OPEN) Sym_6.Subgroup_G2_8.group_table.draw.png
5172
5173
5174 # Group C2C2C2:
5175
5176 Group_8_3:
5177 ▷ $(ORBITER) -v 2 \
5178 ▷ ▷ -define gens -vector -dense \
5179 ▷ ▷ ▷ "1,0,2,3,4,5, 0,1,3,2,4,5, 0,1,2,3,5,4" \
5180 ▷ ▷ -end \

```

```

5181 ▷ ▷ -define G -permutation_group -symmetric_group 6 \
5182 ▷ ▷ ▷ -subgroup_by_generators G3 8 3 gens -end \
5183 ▷ ▷ -with G -do \
5184 ▷ ▷ -group_theoretic_activity \
5185 ▷ ▷ ▷ -export_group_table \
5186 ▷ ▷ -end
5187 ▷ $(ORBITER) -v 2 \
5188 ▷ ▷ -define all_one_r -vector -repeat 1 8 -end \
5189 ▷ ▷ -define all_one_c -vector -repeat 1 8 -end \
5190 ▷ ▷ -draw_matrix \
5191 ▷ ▷ ▷ -input_csv_file \
5192 ▷ ▷ ▷ ▷ Sym_6.Subgroup_G3.8.group_table.csv \
5193 ▷ ▷ ▷ -box_width 50 \
5194 ▷ ▷ ▷ -bit_depth 24 \
5195 ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
5196 ▷ ▷ -end
5197 ▷ magick Sym_6.Subgroup_G3.8.group_table.draw.bmp \
5198 ▷ ▷ Sym_6.Subgroup_G3.8.group_table.draw.png
5199 ▷ $(OPEN) Sym_6.Subgroup_G3.8.group_table.draw.png
5200
5201
5202 # Dihedral group D4 of order 8:
5203
5204 Group_8.4:
5205 ▷ $(ORBITER) -v 2 \
5206 ▷ ▷ -define gens -vector -dense \
5207 ▷ ▷ ▷ "1,2,3,0, 0,3,2,1" \
5208 ▷ ▷ -end \
5209 ▷ ▷ -define G -permutation_group -symmetric_group 4 \
5210 ▷ ▷ ▷ -subgroup_by_generators G4 8 2 gens -end \
5211 ▷ ▷ -with G -do \
5212 ▷ ▷ -group_theoretic_activity \
5213 ▷ ▷ ▷ -export_group_table \
5214 ▷ ▷ -end
5215 ▷ $(ORBITER) -v 2 \
5216 ▷ ▷ -define all_one_r -vector -repeat 1 8 -end \
5217 ▷ ▷ -define all_one_c -vector -repeat 1 8 -end \
5218 ▷ ▷ -draw_matrix \
5219 ▷ ▷ ▷ -input_csv_file \
5220 ▷ ▷ ▷ ▷ Sym_4.Subgroup_G4.8.group_table.csv \
5221 ▷ ▷ ▷ -box_width 50 \
5222 ▷ ▷ ▷ -bit_depth 24 \
5223 ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
5224 ▷ ▷ -end
5225 ▷ magick Sym_4.Subgroup_G4.8.group_table.draw.bmp \
5226 ▷ ▷ Sym_4.Subgroup_G4.8.group_table.draw.png
5227 ▷ $(OPEN) Sym_4.Subgroup_G4.8.group_table.draw.png
5228
5229 # Quaternion group:
5230
5231 Group_8.5:
5232 ▷ $(ORBITER) -v 2 \
5233 ▷ ▷ -define gens -vector -dense \
5234 ▷ ▷ ▷ "2,3,1,0,6,7,5,4, 4,5,7,6,1,0,2,3" \
5235 ▷ ▷ -end \
5236 ▷ ▷ -define G -permutation_group -symmetric_group 8 \
5237 ▷ ▷ ▷ -subgroup_by_generators G5 8 2 gens -end \
5238 ▷ ▷ -with G -do \
5239 ▷ ▷ -group_theoretic_activity \

```



```

5240 ▷ ▷ ▷ -export_group_table \
5241 ▷ ▷ -end
5242 ▷ $(ORBITER) -v 2 \
5243 ▷ ▷ -define all_one_r -vector -repeat 1 8 -end \
5244 ▷ ▷ -define all_one_c -vector -repeat 1 8 -end \
5245 ▷ ▷ -draw_matrix \
5246 ▷ ▷ ▷ -input_csv_file \
5247 ▷ ▷ ▷ ▷ Sym_8.Subgroup_G5_8.group_table.csv \
5248 ▷ ▷ ▷ -box_width 50 \
5249 ▷ ▷ ▷ -bit_depth 24 \
5250 ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
5251 ▷ ▷ -end
5252 ▷ magick Sym_8.Subgroup_G5_8.group_table.draw.bmp \
5253 ▷ ▷ Sym_8.Subgroup_G5_8.group_table.draw.png
5254 ▷ $(OPEN) Sym_8.Subgroup_G5_8.group_table.draw.png
5255
5256
5257
5258 # M12 of order 95040:
5259 #
5260 #(1,4)(3,10)(5,11)(6,12)
5261 #(1,8,9)(2,3,4)(5,12,11)(6,10,7)
5262 #[3,1,9,0,10,11,6,7,8,2,4,5]
5263 #[7,2,3,1,11,9,5,8,0,6,4,10]
5264
5265
5266
5267
5268 Group_M12:
5269 ▷ $(ORBITER) -v 2 \
5270 ▷ ▷ -define Do -draw_options \
5271 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5272 ▷ ▷ -end \
5273 ▷ ▷ -define gens -vector -dense $(GENERATORS.M12) -end \
5274 ▷ ▷ -define G -permutation_group -symmetric_group 12 \
5275 ▷ ▷ ▷ -subgroup_by_generators M12 95040 2 gens -end \
5276 ▷ ▷ -with G -do \
5277 ▷ ▷ -group_theoretic_activity \
5278 ▷ ▷ ▷ -report Do \
5279 ▷ ▷ -end
5280 ▷ $(PDFLATEX) Sym_12.Subgroup_M12_95040.report.tex
5281 ▷ $(OPEN) Sym_12.Subgroup_M12_95040.report.pdf
5282
5283
5284
5285 #####
5286 # Section 5.2: Linear Groups
5287
5288
5289 SECTION_LINEAR_GROUPS:
5290
5291 test_5.2:
5292 ▷ make GL_2_3.elements.dense
5293 ▷ make PGL_2_3.elements.dense
5294 ▷ make PGL_2_17
5295 ▷ make PSL_2_17
5296 ▷ make PGL_3_2
5297 ▷ make PGL_4_2
5298 ▷ make PGL_4_2.with.field

```

```

5299 ▷ make PGL_4.2_report
5300 ▷ make PGL_4.2_multiply_all_elements
5301 ▷ make PGL_4.2_export
5302 ▷ make PGL_4.2_export_GAP
5303 ▷ make PGL_4.2_generated
5304 ▷ make PGL_5.2
5305 ▷ #make PGL_5.2_multiply_all_elements too slow
5306 ▷ make L_5.3
5307 ▷ make L_4.5
5308 ▷ make PGL_4.5
5309 ▷ make PGGL_2.9
5310 ▷ make PGGL_3.4
5311 ▷ #make PGGL_3.8 problem in Syl->init (too slow)
5312 ▷ make AGGL_1.4
5313 ▷ make AGGL_1.8
5314 ▷ make AGL_1.27
5315 ▷ make AGGL_2.27
5316 ▷ make AGGL_1.81
5317 ▷ make SP_4.2
5318 ▷ make PSP_4.4
5319 ▷ make PGOp_4.2
5320 ▷ make PGO_5.2
5321 ▷ make PGO_5.4
5322 ▷ make PGOp_6.2
5323 ▷ make PGOm_6.2
5324 ▷ make PSP_6.2
5325 ▷ make PGO_7.2
5326 ▷ make NullPolarity_6.2
5327 ▷ make M24_matrix_group
5328
5329
5330 GL_2.3_elements_dense:
5331 ▷ $(ORBITER) -v 2 \
5332 ▷ ▷ -define F -finite_field -q 3 -end \
5333 ▷ ▷ -define G -linear_group -GL 2 F -end \
5334 ▷ ▷ -assign elements -from \
5335 ▷ ▷ -with G -do \
5336 ▷ ▷ -group_theoretic_activity \
5337 ▷ ▷ ▷ -elements \
5338 ▷ ▷ -end \
5339 ▷ ▷ -with elements -do \
5340 ▷ ▷ -vector_ge_activity \
5341 ▷ ▷ ▷ -report_with_options "dense=on" \
5342 ▷ ▷ -end \
5343 ▷ ▷ -print_symbols
5344 ▷ $(PDFLATEX) GL_2.3_elements.tex
5345 ▷ $(OPEN) GL_2.3_elements.pdf
5346
5347
5348 PGL_2.3_elements_dense:
5349 ▷ $(ORBITER) -v 2 \
5350 ▷ ▷ -define Do -draw_options \
5351 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5352 ▷ ▷ -end \
5353 ▷ ▷ -define G -linear_group -PGL 2 3 -end \
5354 ▷ ▷ -assign elements -from \
5355 ▷ ▷ -with G -do \
5356 ▷ ▷ -group_theoretic_activity \
5357 ▷ ▷ ▷ -elements \

```

```

5358 ▷ ▷ -end \
5359 ▷ ▷ -with elements -do \
5360 ▷ ▷ -vector_ge_activity \
5361 ▷ ▷ ▷ -report_with_options "dense=on" \
5362 ▷ ▷ -end \
5363 ▷ ▷ -print_symbols
5364 ▷ $(PDFLATEX) PGL_2_3.elements.tex
5365 ▷ $(OPEN) PGL_2_3.elements.pdf
5366
5367
5368 PGL_2_17:
5369 ▷ $(ORBITER) -v 2 \
5370 ▷ ▷ -define Do -draw_options \
5371 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5372 ▷ ▷ -end \
5373 ▷ ▷ -define F -finite_field -q 17 -end \
5374 ▷ ▷ -define G -linear_group -PGL 2 F -end \
5375 ▷ ▷ -with G -do \
5376 ▷ ▷ -group_theoretic_activity \
5377 ▷ ▷ ▷ -report Do \
5378 ▷ ▷ -end
5379 ▷ $(PDFLATEX) PGL_2_17_report.tex
5380 ▷ $(OPEN) PGL_2_17_report.pdf
5381
5382 PSL_2_17:
5383 ▷ $(ORBITER) -v 2 \
5384 ▷ ▷ -define Do -draw_options \
5385 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5386 ▷ ▷ -end \
5387 ▷ ▷ -define F -finite_field -q 17 -end \
5388 ▷ ▷ -define G -linear_group -PSL 2 F -end \
5389 ▷ ▷ -with G -do \
5390 ▷ ▷ -group_theoretic_activity \
5391 ▷ ▷ ▷ -report Do \
5392 ▷ ▷ -end
5393 ▷ $(PDFLATEX) PSL_2_17_report.tex
5394 ▷ $(OPEN) PSL_2_17_report.pdf
5395
5396
5397
5398 PGL_3_2:
5399 ▷ $(ORBITER) -v 2 \
5400 ▷ ▷ -define Do -draw_options \
5401 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5402 ▷ ▷ -end \
5403 ▷ ▷ -define F -finite_field -q 2 -end \
5404 ▷ ▷ -define G -linear_group -PGL 3 F -end \
5405 ▷ ▷ -with G -do \
5406 ▷ ▷ -group_theoretic_activity \
5407 ▷ ▷ ▷ -report Do \
5408 ▷ ▷ -end
5409 ▷ $(PDFLATEX) PGL_3_2_report.tex
5410 ▷ $(OPEN) PGL_3_2_report.pdf
5411
5412
5413 PGL_4_2:
5414 ▷ $(ORBITER) -v 2 \
5415 ▷ ▷ -define G -linear_group -PGL 4 2 -end
5416

```

```

5417
5418 # order 20160
5419
5420
5421 PGL_4.2.with.field:
5422 ▷ $(ORBITER) -v 2 \
5423 ▷ ▷ -define F -finite_field -q 2 -end \
5424 ▷ ▷ -define G -linear_group -PGL 4 F -end
5425
5426
5427 PGL_4.2.report:
5428 ▷ $(ORBITER) -v 2 \
5429 ▷ ▷ -define Do -draw_options \
5430 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5431 ▷ ▷ -end \
5432 ▷ ▷ -define G -linear_group -PGL 4 2 -end \
5433 ▷ ▷ -with G -do \
5434 ▷ ▷ -group_theoretic_activity \
5435 ▷ ▷ ▷ -report Do \
5436 ▷ ▷ -end
5437 ▷ $(PDFLATEX) PGL_4.2.report.tex
5438 ▷ $(OPEN) PGL_4.2.report.pdf
5439
5440
5441 PGL_4.2.multiply_all_elements:
5442 ▷ $(ORBITER) -v 5 \
5443 ▷ ▷ -define F -finite_field -q 2 -end \
5444 ▷ ▷ -define G -linear_group -PGL 4 F -lex_least_base -end \
5445 ▷ ▷ -with G -do \
5446 ▷ ▷ -group_theoretic_activity \
5447 ▷ ▷ ▷ -multiply_all_elements_in_lex_order \
5448 ▷ ▷ -end
5449
5450 # 0.37 of a second, dt=37 tps = 100
5451
5452
5453 PGL_4.2.export:
5454 ▷ $(ORBITER) -v 2 \
5455 ▷ ▷ -define Do -draw_options \
5456 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5457 ▷ ▷ -end \
5458 ▷ ▷ -define F -finite_field -q 2 -end \
5459 ▷ ▷ -define G -linear_group -PGL 4 F -end \
5460 ▷ ▷ -with G -do \
5461 ▷ ▷ -group_theoretic_activity \
5462 ▷ ▷ ▷ -report Do \
5463 ▷ ▷ -end \
5464 ▷ ▷ -with G -do \
5465 ▷ ▷ -group_theoretic_activity \
5466 ▷ ▷ ▷ -export_orbiter \
5467 ▷ ▷ -end
5468 ▷ $(PDFLATEX) PGL_4.2.report.tex
5469 ▷ $(OPEN) PGL_4.2.report.pdf
5470
5471 # created by PGL_4.2.export
5472
5473
5474 PGL_4.2.export_GAP:
5475 ▷ $(ORBITER) -v 2 \

```

```

5476 ▷ ▷ -define Do -draw.options \
5477 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5478 ▷ ▷ -end \
5479 ▷ ▷ -define F -finite_field -q 2 -end \
5480 ▷ ▷ -define G -linear_group -PGL 4 F -end \
5481 ▷ ▷ -with G -do \
5482 ▷ ▷ -group_theoretic_activity \
5483 ▷ ▷ ▷ -report Do \
5484 ▷ ▷ -end \
5485 ▷ ▷ -with G -do \
5486 ▷ ▷ -group_theoretic_activity \
5487 ▷ ▷ ▷ -export_gap \
5488 ▷ ▷ -end
5489 ▷ #$(PDFLATEX) PGL_4.2_report.tex
5490 ▷ #$(OPEN) PGL_4.2_report.pdf
5491
5492
5493 PGL_4.2_generated:
5494 ▷ $(ORBITER) -v 2 \
5495 ▷ ▷ -define gens -vector -file PGL_4.2_gens.csv -end \
5496 ▷ ▷ -define G -permutation_group \
5497 ▷ ▷ -bsgs PGL_4.2 "{\rm PGL}(4,2)" 15 20160 "0,1,2,3" 6 gens -end \
5498
5499 PGL_5.2:
5500 ▷ $(ORBITER) -v 2 \
5501 ▷ ▷ -define Do -draw.options \
5502 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5503 ▷ ▷ -end \
5504 ▷ ▷ -define F -finite_field -q 2 -end \
5505 ▷ ▷ -define G -linear_group -PGL 5 F -lex_least_base -end \
5506 ▷ ▷ -with G -do \
5507 ▷ ▷ -group_theoretic_activity \
5508 ▷ ▷ ▷ -report Do \
5509 ▷ ▷ -end
5510 ▷ $(PDFLATEX) PGL_5.2_report.tex
5511 ▷ $(OPEN) PGL_5.2_report.pdf
5512
5513 #The order of the group PGGL(5, 2) is 9999360
5514
5515 PGL_5.2_multiply_all_elements:
5516 ▷ $(ORBITER) -v 5 \
5517 ▷ ▷ -define F -finite_field -q 2 -end \
5518 ▷ ▷ -define G -linear_group -PGL 5 F -lex_least_base -end \
5519 ▷ ▷ -with G -do \
5520 ▷ ▷ -group_theoretic_activity \
5521 ▷ ▷ ▷ -multiply_all_elements_in_lex_order \
5522 ▷ ▷ -end
5523
5524 # time: 9 min :28 sec on Macbook
5525
5526
5527 L_5.3:
5528 ▷ $(ORBITER) -v 2 \
5529 ▷ ▷ -define Do -draw.options \
5530 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5531 ▷ ▷ -end \
5532 ▷ ▷ -define F -finite_field -q 3 -end \
5533 ▷ ▷ -define G -linear_group -PSL 5 F -end \
5534 ▷ ▷ -with G -do \

```

```

5535 ▷ ▷ -group_theoretic_activity \
5536 ▷ ▷ ▷ -report Do \
5537 ▷ ▷ -end
5538 ▷ $(PDFLATEX) PSL_5_3_report.tex
5539 ▷ $(OPEN) PSL_5_3_report.pdf
5540
5541 #PSL(5,3): Order 237783237120 = 121 * 120 * 117 * 108 * 81 * 16
5542
5543
5544
5545 L_4_5:
5546 ▷ $(ORBITER) -v 2 \
5547 ▷ ▷ -define Do -draw_options \
5548 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5549 ▷ ▷ -end \
5550 ▷ ▷ -define F -finite_field -q 5 -end \
5551 ▷ ▷ -define G -linear_group -PSL 4 F -end \
5552 ▷ ▷ -with G -do \
5553 ▷ ▷ -group_theoretic_activity \
5554 ▷ ▷ ▷ -report Do \
5555 ▷ ▷ -end
5556 ▷ $(PDFLATEX) PSL_4_5_report.tex
5557 ▷ $(OPEN) PSL_4_5_report.pdf
5558
5559 #PSL(4,5): Order 7254000000
5560
5561
5562
5563
5564
5565
5566
5567
5568
5569 PGL_4_5:
5570 ▷ $(ORBITER) -v 2 \
5571 ▷ ▷ -define Do -draw_options \
5572 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5573 ▷ ▷ -end \
5574 ▷ ▷ -define F -finite_field -q 5 -end \
5575 ▷ ▷ -define G -linear_group -PGL 4 F -end \
5576 ▷ ▷ -with G -do \
5577 ▷ ▷ -group_theoretic_activity \
5578 ▷ ▷ ▷ -report Do \
5579 ▷ ▷ -end
5580 ▷ $(PDFLATEX) PGL_4_5_report.tex
5581 ▷ $(OPEN) PGL_4_5_report.pdf
5582
5583
5584
5585 PGGL_2_9:
5586 ▷ $(ORBITER) -v 2 \
5587 ▷ ▷ -define Do -draw_options \
5588 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5589 ▷ ▷ -end \
5590 ▷ ▷ -define G -linear_group -PGGL 2 9 -end \
5591 ▷ ▷ -with G -do \
5592 ▷ ▷ -group_theoretic_activity \
5593 ▷ ▷ ▷ -report Do \

```

```

5594 ▷ ▷ -end \
5595 ▷ ▷ -print_symbols
5596 ▷ $(PDFLATEX) PGGL_2.9_report.tex
5597 ▷ $(OPEN) PGGL_2.9_report.pdf
5598
5599
5600 PGGL_3.4:
5601 ▷ $(ORBITER) -v 2 \
5602 ▷ ▷ -define Do -draw_options \
5603 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5604 ▷ ▷ -end \
5605 ▷ ▷ -define G -linear_group -PGGL 3 4 -end \
5606 ▷ ▷ -with G -do \
5607 ▷ ▷ -group_theoretic_activity \
5608 ▷ ▷ ▷ -report Do \
5609 ▷ ▷ -end \
5610 ▷ ▷ -with G -do \
5611 ▷ ▷ -group_theoretic_activity \
5612 ▷ ▷ ▷ -syllow \
5613 ▷ ▷ -end
5614 ▷ $(PDFLATEX) PGGL_3.4_report.tex
5615 ▷ $(OPEN) PGGL_3.4_report.pdf
5616 ▷ $(PDFLATEX) PGGL_3.4_syllow_report.tex
5617 ▷ $(OPEN) PGGL_3.4_syllow_report.pdf
5618
5619
5620
5621 #problem:
5622
5623 PGGL_3.8:
5624 ▷ $(ORBITER) -v 2 \
5625 ▷ ▷ -define Do -draw_options \
5626 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5627 ▷ ▷ -end \
5628 ▷ ▷ -define G -linear_group -PGGL 3 8 -end \
5629 ▷ ▷ -with G -do \
5630 ▷ ▷ -group_theoretic_activity \
5631 ▷ ▷ ▷ -report Do \
5632 ▷ ▷ -end \
5633 ▷ ▷ -with G -do \
5634 ▷ ▷ -group_theoretic_activity \
5635 ▷ ▷ ▷ -syllow \
5636 ▷ ▷ -end
5637 ▷ $(PDFLATEX) PGGL_3.8_report.tex
5638 ▷ $(OPEN) PGGL_3.8_report.pdf
5639
5640 # problem in Syl->init
5641
5642
5643
5644 AGGL_1.4:
5645 ▷ $(ORBITER) -v 4 \
5646 ▷ ▷ -define F -finite_field -q 4 -end \
5647 ▷ ▷ -define AGGL -linear_group -AGGL 1 F \
5648 ▷ ▷ ▷ -end
5649
5650
5651 AGGL_1.8:
5652 ▷ $(ORBITER) -v 4 \

```

```

5653 ▷ ▷ -define F -finite_field -q 8 -end \
5654 ▷ ▷ -define AGGL -linear_group -AGGL 1 F \
5655 ▷ ▷ ▷ -end
5656
5657
5658
5659 AGL_1.27:
5660 ▷ $(ORBITER) -v 4 \
5661 ▷ ▷ -define Do -draw_options \
5662 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5663 ▷ ▷ -end \
5664 ▷ ▷ -define F -finite_field -q 27 -end \
5665 ▷ ▷ -define G -linear_group -AGL 1 F -end \
5666 ▷ ▷ -with G -do \
5667 ▷ ▷ -group_theoretic_activity \
5668 ▷ ▷ ▷ -report Do \
5669 ▷ ▷ -end
5670 ▷ $(PDFLATEX) AGL_1.27_report.tex
5671 ▷ $(OPEN) AGL_1.27_report.pdf
5672
5673
5674 AGGL_2.27:
5675 ▷ $(ORBITER) -v 2 \
5676 ▷ ▷ -define Do -draw_options \
5677 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5678 ▷ ▷ -end \
5679 ▷ ▷ -define F -finite_field -q 27 -compute_related_fields -end \
5680 ▷ ▷ -define G -linear_group -AGGL 2 F -end \
5681 ▷ ▷ -with G -do \
5682 ▷ ▷ -group_theoretic_activity \
5683 ▷ ▷ ▷ -report Do \
5684 ▷ ▷ -end
5685 ▷ $(PDFLATEX) AGGL_2.27_report.tex
5686 ▷ $(OPEN) AGGL_2.27_report.pdf
5687
5688 #The order of the group AGGL(2, 27) is 1117679472
5689
5690 AGGL_1.81:
5691 ▷ $(ORBITER) -v 4 \
5692 ▷ ▷ -define Do -draw_options \
5693 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5694 ▷ ▷ -end \
5695 ▷ ▷ -define F -finite_field -q 81 \
5696 ▷ ▷ ▷ -override_polynomial 89 \
5697 ▷ ▷ ▷ -compute_related_fields \
5698 ▷ ▷ -end \
5699 ▷ ▷ -define AGGL -linear_group -AGGL 1 F \
5700 ▷ ▷ ▷ -end \
5701 ▷ ▷ -with AGGL -do \
5702 ▷ ▷ -group_theoretic_activity \
5703 ▷ ▷ ▷ -report Do \
5704 ▷ ▷ -end
5705 ▷ $(PDFLATEX) AGGL_1.81_report.tex
5706 ▷ $(OPEN) AGGL_1.81_report.pdf
5707
5708
5709
5710 SP_4.2:
5711 ▷ $(ORBITER) -v 2 \

```



```

5712 ▷ ▷ -define Do -draw.options \
5713 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5714 ▷ ▷ -end \
5715 ▷ ▷ -define F -finite_field -q 2 -end \
5716 ▷ ▷ -define G -linear_group -GL 4 F \
5717 ▷ ▷ ▷ -symplectic_group \
5718 ▷ ▷ -end \
5719 ▷ ▷ -with G -do \
5720 ▷ ▷ -group_theoretic_activity \
5721 ▷ ▷ ▷ -report Do \
5722 ▷ ▷ -end
5723 ▷ $(PDFLATEX) GL_4.2.Sp.4.2.report.tex
5724 ▷ $(OPEN) GL_4.2.Sp.4.2.report.pdf
5725
5726 # order 720
5727
5728
5729 PSP.4.4:
5730 ▷ $(ORBITER) -v 5 \
5731 ▷ ▷ -define Do -draw.options \
5732 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5733 ▷ ▷ -end \
5734 ▷ ▷ -define F -finite_field -q 4 -end \
5735 ▷ ▷ -define G -linear_group -PGL 4 F \
5736 ▷ ▷ ▷ -symplectic_group \
5737 ▷ ▷ -end \
5738 ▷ ▷ -with G -do \
5739 ▷ ▷ -group_theoretic_activity \
5740 ▷ ▷ ▷ -report Do \
5741 ▷ ▷ -end
5742 ▷ $(PDFLATEX) PSp_4.4.report.tex
5743 ▷ $(OPEN) PSp_4.4.report.pdf
5744
5745 #order 979200
5746
5747
5748 PGOp.4.2:
5749 ▷ $(ORBITER) -v 2 \
5750 ▷ ▷ -define Do -draw.options \
5751 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5752 ▷ ▷ -end \
5753 ▷ ▷ -define F -finite_field -q 2 -end \
5754 ▷ ▷ -define G -linear_group -PGOp 4 F -end \
5755 ▷ ▷ -with G -do \
5756 ▷ ▷ -group_theoretic_activity \
5757 ▷ ▷ ▷ -report Do \
5758 ▷ ▷ -end
5759 ▷ $(PDFLATEX) PGOp.4.2.report.tex
5760 ▷ $(OPEN) PGOp.4.2.report.pdf
5761
5762 # order 72
5763
5764
5765 PGO.5.2:
5766 ▷ $(ORBITER) -v 2 \
5767 ▷ ▷ -define Do -draw.options \
5768 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5769 ▷ ▷ -end \
5770 ▷ ▷ -define F -finite_field -q 2 -end \

```

```

5771 ▷ ▷ -define G -linear_group -PGO 5 F -end \
5772 ▷ ▷ -with G -do \
5773 ▷ ▷ -group_theoretic_activity \
5774 ▷ ▷ ▷ -report Do \
5775 ▷ ▷ -end
5776 ▷ $(PDFLATEX) PGO_5.2_report.tex
5777 ▷ $(OPEN) PGO_5.2_report.pdf
5778
5779 # order 720
5780
5781
5782 PGG0_5.4:
5783 ▷ $(ORBITER) -v 2 \
5784 ▷ ▷ -define Do -draw_options \
5785 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5786 ▷ ▷ -end \
5787 ▷ ▷ -define F4 -finite_field -q 4 -compute_related_fields -end \
5788 ▷ ▷ -define G -linear_group -PGGO 5 F4 -end \
5789 ▷ ▷ -with G -do \
5790 ▷ ▷ -group_theoretic_activity \
5791 ▷ ▷ ▷ -report Do \
5792 ▷ ▷ -end
5793 ▷ $(PDFLATEX) PGG0_5.4_report.tex
5794 ▷ $(OPEN) PGG0_5.4_report.pdf
5795
5796 # order 1958400
5797
5798 PGG0_5.4.export_gap:
5799 ▷ $(ORBITER) -v 2 \
5800 ▷ ▷ -define F4 -finite_field -q 4 -end \
5801 ▷ ▷ -define G -linear_group -PGGO 5 F4 -end \
5802 ▷ ▷ -with G -do \
5803 ▷ ▷ -group_theoretic_activity \
5804 ▷ ▷ ▷ -export_gap \
5805 ▷ ▷ -end
5806
5807
5808 PGG0_5.4.multiply_all_elements:
5809 ▷ $(ORBITER) -v 5 \
5810 ▷ ▷ -define F4 -finite_field -q 4 -end \
5811 ▷ ▷ -define G -linear_group -PGGO 5 F4 -end \
5812 ▷ ▷ -with G -do \
5813 ▷ ▷ -group_theoretic_activity \
5814 ▷ ▷ ▷ -multiply_all_elements_in_lex_order \
5815 ▷ ▷ -end
5816
5817
5818 PGOp_6.2:
5819 ▷ $(ORBITER) -v 2 \
5820 ▷ ▷ -define Do -draw_options \
5821 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5822 ▷ ▷ -end \
5823 ▷ ▷ -define F -finite_field -q 2 -end \
5824 ▷ ▷ -define G -linear_group -PGOp 6 F -end \
5825 ▷ ▷ -with G -do \
5826 ▷ ▷ -group_theoretic_activity \
5827 ▷ ▷ ▷ -report Do \
5828 ▷ ▷ -end
5829 ▷ $(PDFLATEX) PGOp_6.2_report.tex

```

```

5830 ▷ $(OPEN) PGOp.6.2_report.pdf
5831
5832 PGOm.6.2:
5833 ▷ $(ORBITER) -v 2 \
5834 ▷ ▷ -define Do -draw.options \
5835 ▷ ▷ ▷ -radius 200 -line.width 0.3 \
5836 ▷ ▷ -end \
5837 ▷ ▷ -define F -finite.field -q 2 -end \
5838 ▷ ▷ -define G -linear.group -PGOm 6 F -end \
5839 ▷ ▷ -with G -do \
5840 ▷ ▷ -group.theoretic.activity \
5841 ▷ ▷ ▷ -report Do \
5842 ▷ ▷ -end
5843 ▷ $(PDFLATEX) PGOm.6.2_report.tex
5844 ▷ $(OPEN) PGOm.6.2_report.pdf
5845
5846
5847
5848
5849
5850 # the following two groups are isomorphic:
5851
5852 PSP.6.2:
5853 ▷ $(ORBITER) -v 2 \
5854 ▷ ▷ -define Do -draw.options \
5855 ▷ ▷ ▷ -radius 200 -line.width 0.3 \
5856 ▷ ▷ -end \
5857 ▷ ▷ -define F -finite.field -q 2 -end \
5858 ▷ ▷ -define G -linear.group -PGL 6 F \
5859 ▷ ▷ ▷ -symplectic.group \
5860 ▷ ▷ -end \
5861 ▷ ▷ -with G -do \
5862 ▷ ▷ -group.theoretic.activity \
5863 ▷ ▷ ▷ -report Do \
5864 ▷ ▷ -end
5865 ▷ $(PDFLATEX) PGL.6.2.Sp.6.2_report.tex
5866 ▷ $(OPEN) PGL.6.2.Sp.6.2_report.pdf
5867
5868 # group order 1451520, acting on 63 points
5869
5870
5871 PGO.7.2:
5872 ▷ $(ORBITER) -v 2 \
5873 ▷ ▷ -define Do -draw.options \
5874 ▷ ▷ ▷ -radius 200 -line.width 0.3 \
5875 ▷ ▷ -end \
5876 ▷ ▷ -define F -finite.field -q 2 -end \
5877 ▷ ▷ -define G -linear.group -PGO 7 F -end \
5878 ▷ ▷ -with G -do \
5879 ▷ ▷ -group.theoretic.activity \
5880 ▷ ▷ ▷ -report Do \
5881 ▷ ▷ -end
5882 ▷ $(PDFLATEX) PGO.7.2_report.tex
5883 ▷ $(OPEN) PGO.7.2_report.pdf
5884
5885
5886 # group order 1451520, acting on 63 points
5887
5888

```

```

5889
5890
5891
5892 NullPolarity_6_2:
5893 ▷ $(ORBITER) -v 2 \
5894 ▷ ▷ -define Do -draw.options \
5895 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5896 ▷ ▷ -end \
5897 ▷ ▷ -define F -finite.field -q 2 -end \
5898 ▷ ▷ -define G -linear_group -PGL 6 F \
5899 ▷ ▷ ▷ -null.polarity_group \
5900 ▷ ▷ -end \
5901 ▷ ▷ -with G -do \
5902 ▷ ▷ -group.theoretic.activity \
5903 ▷ ▷ ▷ -report Do \
5904 ▷ ▷ -end
5905 ▷ $(PDFLATEX) PGL_6_2_NullPolarity_6_2_report.tex
5906 ▷ $(OPEN) PGL_6_2_NullPolarity_6_2_report.pdf
5907
5908 # group order 23040, acting on 63 points
5909
5910
5911 M24_matrix_group:
5912 ▷ $(ORBITER) -v 2 \
5913 ▷ ▷ -define Do -draw.options \
5914 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5915 ▷ ▷ -end \
5916 ▷ ▷ -define F -finite.field -q 2 -end \
5917 ▷ ▷ -define G -linear_group -PGL 12 2 \
5918 ▷ ▷ -subgroup.by.generators "M24" "244823040" 12 \
5919 ▷ ▷ ▷ $(GOLAY_24_CODE_AUT_GENS) \
5920 ▷ ▷ -end \
5921 ▷ ▷ -with G -do \
5922 ▷ ▷ -group.theoretic.activity \
5923 ▷ ▷ ▷ -report Do \
5924 ▷ ▷ -end
5925 ▷ $(PDFLATEX) PGL_12_2_Subgroup.M24.244823040_report.tex
5926 ▷ $(OPEN) PGL_12_2_Subgroup.M24.244823040_report.pdf
5927
5928 # see also Section 10.3
5929
5930
5931
5932
5933
5934 #####
5935 # Section 5.3: Subgroups
5936
5937 SECTION.SUBGROUPS:
5938
5939
5940 test_5.3:
5941 ▷ make C13
5942 ▷ make C13.generated
5943 ▷ make C13.as.subgroup
5944 ▷ make J1
5945 ▷ make PGL_3.11.singer
5946 ▷ make PGL_3.11.singer.and.frobenius
5947 ▷ make PG_2.4.order.21

```

```

5948 ▷ make Sym3_in_PGL_2.3_group_table
5949 ▷ make Sym3_in_PGL_2.3_report
5950 ▷ make quaternion
5951 ▷ make cube_group
5952 ▷ make tetra_group
5953 ▷ make Hesse_group
5954 ▷ make Weyl_E8
5955 ▷ make test_subgroup
5956 ▷ make coset_reps
5957 ▷ make coset_reps_read
5958 ▷ make SP_6.2_point_stab_subgroup
5959 ▷ make PGOp_6.2_report
5960 ▷ make PGOm_6.2_report
5961 ▷ make PGOm_6.2_export_magma
5962 ▷ make PGOp_6.2_point_stab_subgroup
5963 ▷ make PGOp_6.2_linear
5964 ▷ make PGOp_6.2_linear_stab_6
5965
5966
5967
5968
5969 C13:
5970 ▷ $(ORBITER) -v 2 \
5971 ▷ ▷ -define Do -draw.options \
5972 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
5973 ▷ ▷ -end \
5974 ▷ ▷ -define gens -vector -dense $(GEN.C13) -end \
5975 ▷ ▷ -define G -permutation_group \
5976 ▷ ▷ ▷ -bsgs C13 C_{13} 13 13 0 1 \
5977 ▷ ▷ ▷ ▷ gens \
5978 ▷ ▷ -end \
5979 ▷ ▷ -with G -do \
5980 ▷ ▷ -group_theoretic_activity \
5981 ▷ ▷ ▷ -export_orbiter \
5982 ▷ ▷ -end \
5983 ▷ ▷ -with G -do \
5984 ▷ ▷ -group_theoretic_activity \
5985 ▷ ▷ ▷ -export_group_table \
5986 ▷ ▷ -end \
5987 ▷ ▷ -with G -do \
5988 ▷ ▷ -group_theoretic_activity \
5989 ▷ ▷ ▷ -report Do \
5990 ▷ ▷ -end \
5991 ▷ ▷ -with G -do \
5992 ▷ ▷ -group_theoretic_activity \
5993 ▷ ▷ ▷ -save_elements_csv "C13.elts.csv" \
5994 ▷ ▷ -end
5995 ▷ $(PDFLATEX) C13_report.tex
5996 ▷ $(OPEN) C13_report.pdf
5997
5998
5999 C13_generated:
6000 ▷ $(ORBITER) -v 2 \
6001 ▷ ▷ -define gens -vector -file C13_gens.csv -end \
6002 ▷ ▷ -define G -permutation_group \
6003 ▷ ▷ ▷ -bsgs C13 "C_{13}" 13 13 "0" 1 gens \
6004 ▷ ▷ -end \
6005
6006

```

```

6007 C13.as.subgroup:
6008 ▷ $(ORBITER) -v 2 \
6009 ▷ ▷ -define Do -draw.options \
6010 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6011 ▷ ▷ -end \
6012 ▷ ▷ -define G -permutation_group -symmetric_group 13 \
6013 ▷ ▷ ▷ -subgroup_by_generators C13 13 1 $(GEN_C13) \
6014 ▷ ▷ -end \
6015 ▷ ▷ -with G -do \
6016 ▷ ▷ -group_theoretic_activity \
6017 ▷ ▷ ▷ -export_orbiter \
6018 ▷ ▷ -end \
6019 ▷ ▷ -with G -do \
6020 ▷ ▷ -group_theoretic_activity \
6021 ▷ ▷ ▷ -report Do \
6022 ▷ ▷ -end \
6023 ▷ ▷ -with G -do \
6024 ▷ ▷ -group_theoretic_activity \
6025 ▷ ▷ ▷ -save_elements_csv "C13.elts.csv" \
6026 ▷ ▷ -end
6027 ▷ $(PDFLATEX) Perm13.Subgroup_C13_13_report.tex
6028 ▷ $(OPEN) Perm13.Subgroup_C13_13_report.pdf
6029
6030
6031
6032
6033 J1:
6034 ▷ $(ORBITER) -v 2 \
6035 ▷ ▷ -define Do -draw.options \
6036 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6037 ▷ ▷ -end \
6038 ▷ ▷ -define G -linear_group -PGL 7 11 -Janko1 -end \
6039 ▷ ▷ -with G -do \
6040 ▷ ▷ -group_theoretic_activity \
6041 ▷ ▷ ▷ -report Do \
6042 ▷ ▷ -end
6043 ▷ $(PDFLATEX) Janko1_report.tex
6044 ▷ $(OPEN) Janko1_report.pdf
6045
6046
6047 PGL_3.11.singer:
6048 ▷ $(ORBITER) -v 2 \
6049 ▷ ▷ -define Do -draw.options \
6050 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6051 ▷ ▷ -end \
6052 ▷ ▷ -define G -linear_group -PGL 3 11 -singer 19 -end \
6053 ▷ ▷ -with G -do \
6054 ▷ ▷ -group_theoretic_activity \
6055 ▷ ▷ ▷ -report Do \
6056 ▷ ▷ -end
6057 ▷ $(PDFLATEX) PGL_3.11.Singer_3.11.19_report.tex
6058 ▷ $(OPEN) PGL_3.11.Singer_3.11.19_report.pdf
6059
6060
6061 PGL_3.11.singer_and_frobenius:
6062 ▷ $(ORBITER) -v 2 \
6063 ▷ ▷ -define Do -draw.options \
6064 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6065 ▷ ▷ -end \

```

```

6066 ▷ ▷ -define G -linear_group -PGL 3 11 \
6067 ▷ ▷ ▷ -singer_and_frobenius 19 \
6068 ▷ ▷ -end \
6069 ▷ ▷ -with G -do \
6070 ▷ ▷ -group_theoretic_activity \
6071 ▷ ▷ ▷ -report Do \
6072 ▷ ▷ -end
6073 ▷ ▷ $(PDFLATEX) PGL_3_11_Singer_and_Frob3_11_19_report.tex
6074 ▷ ▷ $(OPEN) PGL_3_11_Singer_and_Frob3_11_19_report.pdf
6075
6076 PG_2_4_order_21:
6077 ▷ $(ORBITER) -v 2 \
6078 ▷ ▷ -define G -linear_group -PGL 3 4 -end \
6079 ▷ ▷ -with G -do \
6080 ▷ ▷ -group_theoretic_activity \
6081 ▷ ▷ ▷ -search_element_of_order 21 \
6082 ▷ ▷ -end
6083
6084 Sym3_in_PGL_2_3_group_table:
6085 ▷ $(ORBITER) -v 2 \
6086 ▷ ▷ -define Do -draw_options \
6087 ▷ ▷ ▷ -xout 10000 -yout 10000 \
6088 ▷ ▷ ▷ -embedded -radius 50 \
6089 ▷ ▷ ▷ -scale 0.1 -line_width 2 \
6090 ▷ ▷ -end \
6091 ▷ ▷ -define G -linear_group -PGL 2 3 \
6092 ▷ ▷ -subgroup_by_generators "Sym3" "6" 2 \
6093 ▷ ▷ ▷ "0,1,2,1, 0,1,1,0" \
6094 ▷ ▷ -end \
6095 ▷ ▷ -with G -do \
6096 ▷ ▷ -group_theoretic_activity \
6097 ▷ ▷ ▷ -report Do \
6098 ▷ ▷ -end \
6099 ▷ ▷ -with G -do \
6100 ▷ ▷ -group_theoretic_activity \
6101 ▷ ▷ ▷ -group_table Do \
6102 ▷ ▷ -end
6103 ▷ ▷ $(PDFLATEX) PGL_2_3_group_table_order_6.tex
6104 ▷ ▷ $(OPEN) PGL_2_3_group_table_order_6.pdf
6105
6106 Sym3_in_PGL_2_3_report:
6107 ▷ $(ORBITER) -v 2 \
6108 ▷ ▷ -define Do -draw_options -xout 1000000 -yout 1000000 \
6109 ▷ ▷ ▷ -radius 250 \
6110 ▷ ▷ ▷ -scale 0.5 -line_width 2 \
6111 ▷ ▷ -end \
6112 ▷ ▷ -define G -linear_group -PGL 2 3 \
6113 ▷ ▷ -subgroup_by_generators "Sym3" "6" 2 \
6114 ▷ ▷ ▷ "0,1,2,1, 0,1,1,0" \
6115 ▷ ▷ -end \
6116 ▷ ▷ -with G -do \
6117 ▷ ▷ -group_theoretic_activity \
6118 ▷ ▷ ▷ -report Do \
6119 ▷ ▷ -end \
6120 ▷ ▷ -with G -do \
6121 ▷ ▷ -group_theoretic_activity \
6122 ▷ ▷ ▷ -print_elements_tex \
6123 ▷ ▷ -end \
6124 ▷ ▷ -with G -do \

```

```

6125 ▷ ▷ -group_theoretic_activity \
6126 ▷ ▷ ▷ -make_element_tree \
6127 ▷ ▷ -end
6128 ▷ $(ORBITER) -v 2 \
6129 ▷ ▷ -define Do -draw_options \
6130 ▷ ▷ ▷ -nodes \
6131 ▷ ▷ ▷ -embedded -radius 250 \
6132 ▷ ▷ ▷ -xin 10000 -yin 10000 \
6133 ▷ ▷ ▷ -xout 1000000 -yout 600000 \
6134 ▷ ▷ ▷ -scale 0.3 -line_width 1.0 \
6135 ▷ ▷ -end \
6136 ▷ ▷ -tree_draw \
6137 ▷ ▷ ▷ -file PGL_2_3.Subgroup.Sym3_6.elements_tree.txt \
6138 ▷ ▷ ▷ -draw_options Do \
6139 ▷ ▷ -end
6140 ▷ $(PDFLATEX) PGL_2_3.Subgroup.Sym3_6.report.tex
6141 ▷ $(OPEN) PGL_2_3.Subgroup.Sym3_6.report.pdf
6142 ▷ $(PDFLATEX) PGL_2_3.Subgroup.Sym3_6.elements.tex
6143 ▷ $(OPEN) PGL_2_3.Subgroup.Sym3_6.elements.pdf
6144 ▷ $(PDFLATEX) PGL_2_3.Subgroup.Sym3_6.elements_tree.draw.tex
6145 ▷ $(OPEN) PGL_2_3.Subgroup.Sym3_6.elements_tree.draw.pdf
6146
6147
6148 #PGL_2_3.Subgroup.Sym3_6.elements_tree.txt
6149
6150 quaternion:
6151 ▷ $(ORBITER) -v 2 \
6152 ▷ ▷ -define Do -draw_options \
6153 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6154 ▷ ▷ -end \
6155 ▷ ▷ -define G -linear_group -SL 2 3 \
6156 ▷ ▷ -subgroup_by_generators "quaternion" "8" 3 \
6157 ▷ ▷ ▷ "1,1,1,2, 2,1,1,1, 0,2,1,0" \
6158 ▷ ▷ -end \
6159 ▷ ▷ -with G -do \
6160 ▷ ▷ -group_theoretic_activity \
6161 ▷ ▷ ▷ -group_table Do \
6162 ▷ ▷ -end \
6163 ▷ ▷ -with G -do \
6164 ▷ ▷ -group_theoretic_activity \
6165 ▷ ▷ ▷ -report Do \
6166 ▷ ▷ -end \
6167 ▷ ▷ -with G -do \
6168 ▷ ▷ -group_theoretic_activity \
6169 ▷ ▷ ▷ -print_elements_tex \
6170 ▷ ▷ -end
6171 ▷ $(PDFLATEX) SL_2_3.Subgroup.quaternion_8.report.tex
6172 ▷ $(OPEN) SL_2_3.Subgroup.quaternion_8.report.pdf
6173 ▷ $(PDFLATEX) SL_2_3.Subgroup.quaternion_8.group_table.report.tex
6174 ▷ $(OPEN) SL_2_3.Subgroup.quaternion_8.group_table.report.pdf
6175
6176
6177 cube_group:
6178 ▷ $(ORBITER) -v 2 \
6179 ▷ ▷ -define Do -draw_options \
6180 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6181 ▷ ▷ -end \
6182 ▷ ▷ -define gens -vector -dense \
6183 ▷ ▷ ▷ "0,1,0,2,0,0,0,0,1, \

```



```

6184 ▷ ▷ ▷ 0,0,1,0,1,0,2,0,0, \
6185 ▷ ▷ ▷ 2,0,0,0,1,0,0,0,1" \
6186 ▷ ▷ -end \
6187 ▷ ▷ -define G -linear_group -GL 3 3 \
6188 ▷ ▷ -subgroup_by_generators "cube" "48" 3 \
6189 ▷ ▷ ▷ gens \
6190 ▷ ▷ -end \
6191 ▷ ▷ -with G -do \
6192 ▷ ▷ -group_theoretic_activity \
6193 ▷ ▷ ▷ -report Do \
6194 ▷ ▷ -end \
6195 ▷ ▷ -with G -do \
6196 ▷ ▷ -group_theoretic_activity \
6197 ▷ ▷ ▷ -print_elements_tex \
6198 ▷ ▷ -end
6199 ▷ $(PDFLATEX) GL_3_3.Subgroup_cube_48.report.tex
6200 ▷ $(OPEN) GL_3_3.Subgroup_cube_48.report.pdf
6201
6202
6203 tetra_group:
6204 ▷ $(ORBITER) -v 3 \
6205 ▷ ▷ -define Do -draw_options \
6206 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6207 ▷ ▷ -end \
6208 ▷ ▷ -define G -linear_group -GL 3 3 \
6209 ▷ ▷ -subgroup_by_generators "tetra" "12" 2 \
6210 ▷ ▷ ▷ "0,1,0,0,0,1,1,0,0, 0,0,1,2,0,0,0,2,0" \
6211 ▷ ▷ -end \
6212 ▷ ▷ -with G -do \
6213 ▷ ▷ -group_theoretic_activity \
6214 ▷ ▷ ▷ -report Do \
6215 ▷ ▷ -end \
6216 ▷ ▷ -with G -do \
6217 ▷ ▷ -group_theoretic_activity \
6218 ▷ ▷ ▷ -print_elements_tex \
6219 ▷ ▷ -end
6220 ▷ $(PDFLATEX) GL_3_3.Subgroup_tetra_12.report.tex
6221 ▷ $(OPEN) GL_3_3.Subgroup_tetra_12.report.pdf
6222
6223
6224
6225 Hesse_group:
6226 ▷ $(ORBITER) -v 3 \
6227 ▷ ▷ -define Do -draw_options \
6228 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6229 ▷ ▷ -end \
6230 ▷ ▷ -define gens -vector -compact \
6231 ▷ ▷ ▷ $(GENERATORS_HESSE_GROUP) \
6232 ▷ ▷ -end \
6233 ▷ ▷ -define G -linear_group -PGGL 3 4 \
6234 ▷ ▷ -subgroup_by_generators "Hesse" "432" 7 gens \
6235 ▷ ▷ -end \
6236 ▷ ▷ -with G -do \
6237 ▷ ▷ -group_theoretic_activity \
6238 ▷ ▷ ▷ -report Do \
6239 ▷ ▷ -end \
6240 ▷ ▷ -with G -do \
6241 ▷ ▷ -group_theoretic_activity \
6242 ▷ ▷ ▷ -print_elements_tex \

```

```

6243 ▷ ▷ -end
6244 ▷ $(PDFLATEX) PGGL_3_4_Subgroup.Hesse.432.report.tex
6245 ▷ $(OPEN) PGGL_3_4_Subgroup.Hesse.432.report.pdf
6246
6247
6248
6249 Weyl_E8:
6250 ▷ $(ORBITER) -v 3 \
6251 ▷ ▷ -define Do -draw_options \
6252 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6253 ▷ ▷ -end \
6254 ▷ ▷ -define gens -vector -dense \
6255 ▷ ▷ ▷ $(GENERATORS_WEYL_GROUP_E8) \
6256 ▷ ▷ -end \
6257 ▷ ▷ -define G -linear_group -GL 8 3 \
6258 ▷ ▷ -subgroup_by_generators \
6259 ▷ ▷ ▷ "Weyl_E8" "696729600" 2 gens \
6260 ▷ ▷ -end \
6261 ▷ ▷ -with G -do \
6262 ▷ ▷ -group_theoretic_activity \
6263 ▷ ▷ ▷ -report Do \
6264 ▷ ▷ -end
6265 ▷ $(PDFLATEX) GL_8_3_Subgroup.Weyl_E8.696729600.report.tex
6266 ▷ $(OPEN) GL_8_3_Subgroup.Weyl_E8.696729600.report.pdf
6267
6268 # group generators from http://www.math.rwth-aachen.de/~Gabriele.Nebe/LATTICES/E8
6269 .b.html
6270
6271
6272 test_subgroup:
6273 ▷ $(ORBITER) -v 2 \
6274 ▷ ▷ -define F -finite_field -q 2 -end \
6275 ▷ ▷ -define G1 -linear_group -PGOp 6 F -end \
6276 ▷ ▷ -define G2 -linear_group -PGL 6 F \
6277 ▷ ▷ ▷ -symplectic_group \
6278 ▷ ▷ -end \
6279 ▷ ▷ -with G1 -and G2 -do \
6280 ▷ ▷ -group_theoretic_activity \
6281 ▷ ▷ ▷ -is_subgroup_of \
6282 ▷ ▷ -end
6283
6284
6285 coset_reps:
6286 ▷ $(ORBITER) -v 2 \
6287 ▷ ▷ -define F -finite_field -q 2 -end \
6288 ▷ ▷ -define G1 -linear_group -PGOp 6 F -end \
6289 ▷ ▷ -define G2 -linear_group -PGL 6 F \
6290 ▷ ▷ ▷ -symplectic_group \
6291 ▷ ▷ -end \
6292 ▷ ▷ -with G1 -and G2 -do \
6293 ▷ ▷ -group_theoretic_activity \
6294 ▷ ▷ ▷ -coset_reps \
6295 ▷ ▷ -end
6296 ▷ $(PDFLATEX) PGOp_6_2_coset_reps.tex
6297 ▷ $(OPEN) PGOp_6_2_coset_reps.pdf
6298
6299 coset_reps_read:
6300 ▷ $(ORBITER) -v 2 \

```

```

6301 ▷ ▷ -define F -finite_field -q 2 -end \
6302 ▷ ▷ -define G1 -linear_group -PGOp 6 F -end \
6303 ▷ ▷ -define G2 -linear_group -PGL 6 F \
6304 ▷ ▷ ▷ -symplectic_group \
6305 ▷ ▷ -end \
6306 ▷ ▷ -define CR -vector_ge -action G2 \
6307 ▷ ▷ ▷ -read_csv \
6308 ▷ ▷ ▷ PGOp_6_2_coset_reps.csv Element \
6309 ▷ ▷ -end
6310
6311 SP_6_2_point_stab_subgroup:
6312 ▷ $(ORBITER) -v 2 \
6313 ▷ ▷ -define Do -draw_options \
6314 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6315 ▷ ▷ -end \
6316 ▷ ▷ -define F -finite_field -q 2 -end \
6317 ▷ ▷ -define G -linear_group -PGL 6 F \
6318 ▷ ▷ ▷ -symplectic_group \
6319 ▷ ▷ -end \
6320 ▷ ▷ -define G0 -modified_group -from G \
6321 ▷ ▷ ▷ -point_stabilizer 0 \
6322 ▷ ▷ -end \
6323 ▷ ▷ -with G0 -do \
6324 ▷ ▷ -group_theoretic_activity \
6325 ▷ ▷ ▷ -report Do \
6326 ▷ ▷ -end
6327 ▷ $(PDFLATEX) PSp_6_2_Stab0_report.tex
6328 ▷ $(OPEN) PSp_6_2_Stab0_report.pdf
6329
6330
6331 # group of order 23040
6332
6333
6334 PGOp_6_2_report:
6335 ▷ $(ORBITER) -v 2 \
6336 ▷ ▷ -define Do -draw_options \
6337 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6338 ▷ ▷ -end \
6339 ▷ ▷ -define F -finite_field -q 2 -end \
6340 ▷ ▷ -define G -linear_group -PGOp 6 F -end \
6341 ▷ ▷ -with G -do \
6342 ▷ ▷ -group_theoretic_activity \
6343 ▷ ▷ ▷ -report Do \
6344 ▷ ▷ -end
6345 ▷ $(PDFLATEX) PGOp_6_2_report.tex
6346 ▷ $(OPEN) PGOp_6_2_report.pdf
6347
6348 # group order 40320
6349
6350
6351 PGOm_6_2_report:
6352 ▷ $(ORBITER) -v 2 \
6353 ▷ ▷ -define Do -draw_options \
6354 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6355 ▷ ▷ -end \
6356 ▷ ▷ -define F -finite_field -q 2 -end \
6357 ▷ ▷ -define G -linear_group -PGOm 6 F -end \
6358 ▷ ▷ -with G -do \
6359 ▷ ▷ -group_theoretic_activity \

```

```

6360 ▷ ▷ ▷ -report Do \
6361 ▷ ▷ -end
6362 ▷ $(PDFLATEX) PG0m_6.2_report.tex
6363 ▷ $(OPEN) PG0m_6.2_report.pdf
6364
6365 # group order 51840
6366 # degree 27
6367
6368
6369 PG0m_6.2_export_magma:
6370 ▷ $(ORBITER) -v 2 \
6371 ▷ ▷ -define F -finite_field -q 2 -end \
6372 ▷ ▷ -define G -linear_group -PG0m 6 F -end \
6373 ▷ ▷ -with G -do \
6374 ▷ ▷ -group_theoretic_activity \
6375 ▷ ▷ ▷ -export_magma \
6376 ▷ ▷ -end
6377
6378
6379
6380 PG0p_6.2_point_stab_subgroup:
6381 ▷ $(ORBITER) -v 2 \
6382 ▷ ▷ -define Do -draw_options \
6383 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6384 ▷ ▷ -end \
6385 ▷ ▷ -define F -finite_field -q 2 -end \
6386 ▷ ▷ -define G -linear_group -PG0p 6 F -end \
6387 ▷ ▷ -define G0 -modified_group -from G \
6388 ▷ ▷ ▷ -point_stabilizer 0 \
6389 ▷ ▷ -end \
6390 ▷ ▷ -with G0 -do \
6391 ▷ ▷ -group_theoretic_activity \
6392 ▷ ▷ ▷ -report Do \
6393 ▷ ▷ -end
6394 ▷ $(PDFLATEX) PG0p_6.2_report.tex
6395 ▷ $(OPEN) PG0p_6.2_report.pdf
6396
6397
6398 # group of order 1152
6399
6400
6401 PG0p_6.2_linear:
6402 ▷ $(ORBITER) -v 2 \
6403 ▷ ▷ -define Do -draw_options \
6404 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6405 ▷ ▷ -end \
6406 ▷ ▷ -define F -finite_field -q 2 -end \
6407 ▷ ▷ -define G -linear_group -PGL 6 F \
6408 ▷ ▷ ▷ -subgroup_by_generators PG0p_6.2 \
6409 ▷ ▷ ▷ ▷ 40320 10 $(PG0p_6.2_GENS) \
6410 ▷ ▷ -end \
6411 ▷ ▷ -with G -do \
6412 ▷ ▷ -group_theoretic_activity \
6413 ▷ ▷ ▷ -report Do \
6414 ▷ ▷ -end
6415 ▷ $(PDFLATEX) PGL_6.2_Subgroup_PG0p_6.2_40320_report.tex
6416 ▷ $(OPEN) PGL_6.2_Subgroup_PG0p_6.2_40320_report.pdf
6417
6418

```

```

6419
6420 PGOp_6.2.linear_stab.6:
6421 ▷ $(ORBITER) -v 2 \
6422 ▷ ▷ -define Do -draw_options \
6423 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6424 ▷ ▷ -end \
6425 ▷ ▷ -define F -finite_field -q 2 -end \
6426 ▷ ▷ -define G -linear_group -PGL 6 F \
6427 ▷ ▷ ▷ -subgroup_by_generators PGOp_6.2 \
6428 ▷ ▷ ▷ 40320 10 $(PGOp_6.2.GENS) \
6429 ▷ ▷ -end \
6430 ▷ ▷ -define G6 -modified_group -from G \
6431 ▷ ▷ ▷ -point_stabilizer 6 \
6432 ▷ ▷ -end \
6433 ▷ ▷ -with G6 -do \
6434 ▷ ▷ -group_theoretic_activity \
6435 ▷ ▷ ▷ -report Do \
6436 ▷ ▷ -end
6437 ▷ $(PDFLATEX) PGL_6.2.Subgroup.PGOp_6.2.40320_Stab6_report.tex
6438 ▷ $(OPEN) PGL_6.2.Subgroup.PGOp_6.2.40320_Stab6_report.pdf
6439
6440
6441 # group of order 1440, of index 28 in PGOp(6,2)
6442
6443
6444
6445
6446 #####
6447 # Section 5.4: New Actions from Old
6448
6449
6450 SECTION_NEW_ACTIONS_FROM_OLD:
6451
6452
6453 test_5.4:
6454 ▷ make Cyclic_6_aut
6455 ▷ make Elementary_abelian_8_aut
6456 ▷ make Symmetric_4_on_pairs
6457 ▷ make Alternating_5
6458 ▷ make Alternating_5_derived
6459 ▷ #make direct_product_AGL
6460 ▷ make PGLstar_3.2
6461 ▷ make PGLstar_3.2_restrict
6462 ▷ make PGGLstar_3.4
6463 ▷ make PGLstar_4.2
6464 ▷ make Group_M11
6465 ▷ make surface_q13_Eckardt
6466 ▷ make surface_q13_Eckardt_on_tritangent_planes
6467 ▷ make Hirschfeld_stab_projectivity_group
6468 ▷ make Hirschfeld_stab_field_reduction
6469 ▷ make on_planes
6470 ▷ make on_planes_restricted
6471 ▷ make T3_on_tensors
6472 ▷ make T3r1
6473 ▷ make T4_on_tensors
6474 ▷ make T4r1
6475 ▷ #make PGGL_2.8_on_conic
6476 ▷ make PGL_4.2_wedge
6477 ▷ make PGL_5.3_wedge

```

```

6478 ▷ make PGL_4_2_wd
6479 ▷ make PGL_5_3_wd
6480 ▷ #make PGL_4_2_wd_reverse
6481 ▷ make M24_perm_group
6482 ▷ #make M22_bar
6483 ▷ make curve_q13_f0_clean2_variety.make_group
6484 ▷ make conic_group_q5
6485 ▷ make group_of_Hirschfeld_surface_q4
6486 ▷ #make Higman_Sims_on_176_points
6487 ▷ make group_U_4_3
6488
6489
6490
6491 Cyclic_6_aut:
6492 ▷ $(ORBITER) -v 3 \
6493 ▷ ▷ -define Do -draw.options \
6494 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6495 ▷ ▷ -end \
6496 ▷ ▷ -define G -permutation_group -cyclic_group 6 -end \
6497 ▷ ▷ -with G -do \
6498 ▷ ▷ ▷ -group.theoretic.activity \
6499 ▷ ▷ ▷ ▷ -report Do \
6500 ▷ ▷ -end \
6501 ▷ ▷ -define A -modified_group -from G \
6502 ▷ ▷ ▷ -automorphism_group \
6503 ▷ ▷ -end \
6504 ▷ ▷ -with A -do \
6505 ▷ ▷ ▷ -group.theoretic.activity \
6506 ▷ ▷ ▷ ▷ -report Do \
6507 ▷ ▷ -end
6508 ▷ $(PDFLATEX) C_6_aut_report.tex
6509 ▷ $(OPEN) C_6_aut_report.pdf
6510
6511 Elementary_abelian_8_aut:
6512 ▷ $(ORBITER) -v 3 \
6513 ▷ ▷ -define Do -draw.options \
6514 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6515 ▷ ▷ -end \
6516 ▷ ▷ -define G -permutation_group -elementary_abelian_group 8 -end \
6517 ▷ ▷ -with G -do \
6518 ▷ ▷ ▷ -group.theoretic.activity \
6519 ▷ ▷ ▷ ▷ -report Do \
6520 ▷ ▷ -end \
6521 ▷ ▷ -define A -modified_group -from G \
6522 ▷ ▷ ▷ -automorphism_group \
6523 ▷ ▷ -end \
6524 ▷ ▷ -with A -do \
6525 ▷ ▷ ▷ -group.theoretic.activity \
6526 ▷ ▷ ▷ ▷ -report Do \
6527 ▷ ▷ -end
6528 ▷ $(PDFLATEX) E_8_aut_report.tex
6529 ▷ $(OPEN) E_8_aut_report.pdf
6530
6531
6532
6533 Symmetric_4_on_pairs:
6534 ▷ $(ORBITER) -v 3 \
6535 ▷ ▷ -define Do -draw.options \
6536 ▷ ▷ ▷ -radius 200 -line_width 0.3 \

```

```

6537 ▷ ▷ -end \
6538 ▷ ▷ -define G -permutation_group -symmetric_group 4 -end \
6539 ▷ ▷ -define G.on_2subsets -modified_group -from G \
6540 ▷ ▷ ▷ -on_k_subsets 2 \
6541 ▷ ▷ -end \
6542 ▷ ▷ -with G.on_2subsets -do \
6543 ▷ ▷ -group_theoretic_activity \
6544 ▷ ▷ ▷ -report Do \
6545 ▷ ▷ -end
6546 ▷ $(PDFLATEX) Sym_4_OnSubsets_2_report.tex
6547 ▷ $(OPEN) Sym_4_OnSubsets_2_report.pdf
6548
6549
6550
6551 Alternating_5:
6552 ▷ $(ORBITER) -v 3 \
6553 ▷ ▷ -define Do -draw_options \
6554 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6555 ▷ ▷ -end \
6556 ▷ ▷ -define Sym5 -permutation_group -symmetric_group 5 -end \
6557 ▷ ▷ -define Alt5 -modified_group -from Sym5 \
6558 ▷ ▷ ▷ -create_even_subgroup \
6559 ▷ ▷ -end \
6560 ▷ ▷ -with Alt5 -do \
6561 ▷ ▷ -group_theoretic_activity \
6562 ▷ ▷ ▷ -report Do \
6563 ▷ ▷ -end \
6564 ▷ ▷ -with Alt5 -do \
6565 ▷ ▷ -group_theoretic_activity \
6566 ▷ ▷ ▷ -group_table Do \
6567 ▷ ▷ -end \
6568 ▷ ▷ -with Alt5 -do \
6569 ▷ ▷ -group_theoretic_activity \
6570 ▷ ▷ ▷ -export_group_table \
6571 ▷ ▷ -end
6572 ▷ $(ORBITER) -v 2 \
6573 ▷ ▷ -define all_one_r -vector -repeat 1 60 -end \
6574 ▷ ▷ -define all_one_c -vector -repeat 1 60 -end \
6575 ▷ ▷ -draw_matrix \
6576 ▷ ▷ ▷ -input_csv_file \
6577 ▷ ▷ ▷ ▷ Sym_5_EvenSub_group_table.csv \
6578 ▷ ▷ ▷ -box_width 20 \
6579 ▷ ▷ ▷ -bit_depth 24 \
6580 ▷ ▷ ▷ -partition 3 all_one_r all_one_c \
6581 ▷ ▷ -end
6582 ▷ $(PDFLATEX) Sym_5_EvenSub_report.tex
6583 ▷ $(OPEN) Sym_5_EvenSub_report.pdf
6584 ▷ $(PDFLATEX) Sym_5_EvenSub_group_table_report.tex
6585 ▷ $(OPEN) Sym_5_EvenSub_group_table_report.pdf
6586 ▷ magick Sym_5_EvenSub_group_table_draw.bmp Sym_5_EvenSub_group_table_draw.png
6587 ▷ $(OPEN) Sym_5_EvenSub_group_table_draw.png
6588
6589 # group order 60
6590 #Sym_5_EvenSub_group_table.csv
6591
6592
6593 Alternating_5_derived:
6594 ▷ $(ORBITER) -v 3 \
6595 ▷ ▷ -define Do -draw_options \

```

```

6596 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6597 ▷ ▷ -end \
6598 ▷ ▷ -define Sym5 -permutation_group -symmetric_group 5 -end \
6599 ▷ ▷ -define Alt5 -modified_group -from Sym5 \
6600 ▷ ▷ ▷ -derived_subgroup \
6601 ▷ ▷ -end \
6602 ▷ ▷ -with Alt5 -do \
6603 ▷ ▷ -group_theoretic_activity \
6604 ▷ ▷ ▷ -report Do \
6605 ▷ ▷ -end
6606 ▷ $(PDFLATEX) Sym_5_Derived_report.tex
6607 ▷ $(OPEN) Sym_5_Derived_report.pdf
6608
6609
6610 # product action:
6611
6612 direct_product_AGL:
6613 ▷ $(ORBITER) -v 4 \
6614 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
6615 ▷ ▷ -define Do -draw_options \
6616 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6617 ▷ ▷ -end \
6618 ▷ ▷ -define G1 -linear_group -AGL 1 3 \
6619 ▷ ▷ ▷ -end \
6620 ▷ ▷ -define G2 -linear_group -AGL 1 7 \
6621 ▷ ▷ ▷ -end \
6622 ▷ ▷ -define G -modified_group -direct_product "G1,G2" \
6623 ▷ ▷ ▷ "21" "1,1,1,1" \
6624 ▷ ▷ -end \
6625 ▷ ▷ -with G -do \
6626 ▷ ▷ -group_theoretic_activity \
6627 ▷ ▷ ▷ -report Do \
6628 ▷ ▷ -end
6629 ▷ $(PDFLATEX) product_AGL_1.3_AGL_1.7_report.tex
6630 ▷ $(OPEN) product_AGL_1.3_AGL_1.7_report.pdf
6631
6632 #error: direct_product_group_element_code_for_make_element not yet implemented
6633
6634
6635 PGLstar_3.2:
6636 ▷ $(ORBITER) -v 6 \
6637 ▷ ▷ -define Do -draw_options \
6638 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6639 ▷ ▷ -end \
6640 ▷ ▷ -define F -finite_field -q 2 -end \
6641 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
6642 ▷ ▷ -define G0 -linear_group -PGL 3 F -end \
6643 ▷ ▷ -define G -modified_group -polarity_extension \
6644 ▷ ▷ ▷ G0 P \
6645 ▷ ▷ -end \
6646 ▷ ▷ -with G -do \
6647 ▷ ▷ -group_theoretic_activity \
6648 ▷ ▷ ▷ -report Do \
6649 ▷ ▷ -end
6650 ▷ $(PDFLATEX) PGL_3.2.polarity_ext_report.tex
6651 ▷ $(OPEN) PGL_3.2.polarity_ext_report.pdf
6652
6653 PGLstar_3.2_restrict:
6654 ▷ $(ORBITER) -v 6 \

```



```

6655 ▷ ▷ -define Do -draw.options \
6656 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6657 ▷ ▷ -end \
6658 ▷ ▷ -define F -finite_field -q 2 -end \
6659 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
6660 ▷ ▷ -define H -set -loop 2 14 1 -end \
6661 ▷ ▷ -define G0 -linear_group -PGL 3 F -end \
6662 ▷ ▷ -define G -modified_group -polarity_extension \
6663 ▷ ▷ ▷ G0 P \
6664 ▷ ▷ -end \
6665 ▷ ▷ -define Gr -modified_group -from G \
6666 ▷ ▷ ▷ -restricted_action H points\\_and\\_lines \
6667 ▷ ▷ -end \
6668 ▷ ▷ -with Gr -do \
6669 ▷ ▷ -group_theoretic_activity \
6670 ▷ ▷ ▷ -report Do \
6671 ▷ ▷ -end
6672 ▷ $(PDFLATEX) PGL_3.2.polarity_extension_res_H.report.tex
6673 ▷ $(OPEN) PGL_3.2.polarity_extension_res_H.report.pdf
6674
6675
6676 PGLstar_3.4:
6677 ▷ $(ORBITER) -v 6 \
6678 ▷ ▷ -define Do -draw.options \
6679 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6680 ▷ ▷ -end \
6681 ▷ ▷ -define F -finite_field -q 4 -end \
6682 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
6683 ▷ ▷ -define G0 -linear_group -PGGL 3 F -end \
6684 ▷ ▷ -define G -modified_group -polarity_extension \
6685 ▷ ▷ ▷ G0 P \
6686 ▷ ▷ -end \
6687 ▷ ▷ -with G -do \
6688 ▷ ▷ -group_theoretic_activity \
6689 ▷ ▷ ▷ -report Do \
6690 ▷ ▷ -end
6691 ▷ $(PDFLATEX) PGGL_3.4.polarity_ext_report.tex
6692 ▷ $(OPEN) PGGL_3.4.polarity_ext_report.pdf
6693
6694 PGLstar_4.2:
6695 ▷ $(ORBITER) -v 6 \
6696 ▷ ▷ -define Do -draw.options \
6697 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6698 ▷ ▷ -end \
6699 ▷ ▷ -define F -finite_field -q 2 -end \
6700 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
6701 ▷ ▷ -define G0 -linear_group -PGL 4 F -end \
6702 ▷ ▷ -define G -modified_group -polarity_extension \
6703 ▷ ▷ ▷ G0 P \
6704 ▷ ▷ -end \
6705 ▷ ▷ -with G -do \
6706 ▷ ▷ -group_theoretic_activity \
6707 ▷ ▷ ▷ -report Do \
6708 ▷ ▷ -end
6709 ▷ $(PDFLATEX) PGL_4.2.polarity_ext_report.tex
6710 ▷ $(OPEN) PGL_4.2.polarity_ext_report.pdf
6711
6712
6713

```

```

6714 Group_M11:
6715 ▷ $(ORBITER) -v 2 \
6716 ▷ ▷ -define Do -draw.options \
6717 ▷ ▷ ▷ -radius 200 -line.width 0.3 \
6718 ▷ ▷ -end \
6719 ▷ ▷ -define gens -vector -dense $(GENERATORS.M12) -end \
6720 ▷ ▷ -define G -permutation_group -symmetric_group 12 \
6721 ▷ ▷ ▷ -subgroup_by_generators M12 95040 2 gens -end \
6722 ▷ ▷ -define G0 -modified_group -from G \
6723 ▷ ▷ ▷ -point_stabilizer 0 \
6724 ▷ ▷ -end \
6725 ▷ ▷ -with G0 -do \
6726 ▷ ▷ -group_theoretic_activity \
6727 ▷ ▷ ▷ -report Do \
6728 ▷ ▷ -end
6729 ▷ $(PDFLATEX) Sym_12_Subgroup_M12_95040_Stab0_report.tex
6730 ▷ $(OPEN) Sym_12_Subgroup_M12_95040_Stab0_report.pdf
6731
6732
6733
6734 surface_q13_Eckardt:
6735 ▷ $(ORBITER) -v 10 \
6736 ▷ ▷ -define Do -draw.options \
6737 ▷ ▷ ▷ -radius 200 -line.width 0.3 \
6738 ▷ ▷ -end \
6739 ▷ ▷ -define F -finite_field -q 13 -end \
6740 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
6741 ▷ ▷ -define S -cubic_surface -space P \
6742 ▷ ▷ ▷ -arc_lifting_with_trihedral_pair "0,1,2,3,43,113" \
6743 ▷ ▷ -end \
6744 ▷ ▷ -with S -do \
6745 ▷ ▷ -cubic_surface_activity \
6746 ▷ ▷ ▷ -report Do \
6747 ▷ ▷ -end \
6748 ▷ ▷ -with S -do \
6749 ▷ ▷ -cubic_surface_activity \
6750 ▷ ▷ ▷ -export_something tritangent_planes \
6751 ▷ ▷ -end
6752 ▷ $(PDFLATEX) surface_arc_lifting_trihedral_q13_arc_0_1_2_3_43_113_report.tex
6753 ▷ $(OPEN) surface_arc_lifting_trihedral_q13_arc_0_1_2_3_43_113_report.pdf
6754
6755 #surface_equation_F_Hirschfeld_q4_tritangent_planes.csv
6756
6757
6758 surface_q13_Eckardt_on_tritangent_planes:
6759 ▷ $(ORBITER) -v 2 \
6760 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
6761 ▷ ▷ -define Do -draw.options \
6762 ▷ ▷ ▷ -radius 200 -line.width 0.3 \
6763 ▷ ▷ -end \
6764 ▷ ▷ -define F -finite_field -q 13 -end \
6765 ▷ ▷ -define gens -vector -field F \
6766 ▷ ▷ ▷ -dense $(SURFACE.Q13_STAB_GENS) \
6767 ▷ ▷ -end \
6768 ▷ ▷ -define TriP -set -file \
6769 ▷ ▷ ▷ surface_arc_lifting_trihedral_q13_ \
6770 arc_0_1_2_3_43_113_tritangent_planes.csv \
6771 ▷ ▷ -end \
6772 ▷ ▷ -define G -linear_group -PGL 4 F \

```

```

6773 ▷ ▷ ▷ -subgroup_by_generators "stab" \
6774 ▷ ▷ ▷ ▷ 24 3 gens \
6775 ▷ ▷ ▷ -end \
6776 ▷ ▷ -define G_on_planes -modified_group -from G \
6777 ▷ ▷ ▷ -on_k.subspaces 3 \
6778 ▷ ▷ -end \
6779 ▷ ▷ -define Gr -modified_group -from G_on_planes \
6780 ▷ ▷ ▷ -restricted_action TriP TriP \
6781 ▷ ▷ -end \
6782 ▷ ▷ -with Gr -do \
6783 ▷ ▷ -group_theoretic_activity \
6784 ▷ ▷ ▷ -report Do \
6785 ▷ ▷ -end \
6786 ▷ ▷ -define Orb -orbits -group Gr \
6787 ▷ ▷ ▷ -on_points \
6788 ▷ ▷ -end \
6789 ▷ ▷ -with Orb -do -orbits_activity \
6790 ▷ ▷ ▷ -report Do \
6791 ▷ ▷ -end \
6792 ▷ ▷ -with Orb -do -orbits_activity \
6793 ▷ ▷ ▷ -draw_tree 0 Do \
6794 ▷ ▷ -end \
6795 ▷ ▷ -with Orb -do -orbits_activity \
6796 ▷ ▷ ▷ -draw_tree 1 Do \
6797 ▷ ▷ -end \
6798 ▷ ▷ -with Orb -do -orbits_activity \
6799 ▷ ▷ ▷ -stabilizer 36 \
6800 ▷ ▷ -end
6801 ▷ $(PDFLATEX) PGL_4_13_Gr_4_3_res.TriP-orbits-report.tex
6802 ▷ $(OPEN) PGL_4_13_Gr_4_3_res.TriP-orbits-report.pdf
6803
6804
6805 Hirschfeld_group:
6806 ▷ $(ORBITER) -v 4 \
6807 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
6808 ▷ ▷ -define Do -draw_options \
6809 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6810 ▷ ▷ -end \
6811 ▷ ▷ -define G -linear_group -PGGL 4 4 \
6812 ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld.Stab" \
6813 ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
6814 ▷ ▷ -end \
6815 ▷ ▷ -with G -do \
6816 ▷ ▷ -group_theoretic_activity \
6817 ▷ ▷ ▷ -report Do \
6818 ▷ ▷ -end
6819 ▷ $(PDFLATEX) PGGL_4_4_Subgroup_Hirschfeld.Stab_51840_report.tex
6820 ▷ $(OPEN) PGGL_4_4_Subgroup_Hirschfeld.Stab_51840_report.pdf
6821
6822
6823
6824 #F:=GF(2);
6825 #G := GeneralLinearGroup(8, F);
6826 #H := sub< G | [1,0,0,0,0,0,0,0, 1,1,0,0,0,0,0,0, 0,0,1,0,0,0,0,0, 0,0,1,1,0,0,0,
6827 0, 0,0,0,0,1,0,0,0, 0,0,0,0,1,1,0,0, 0,0,0,0,0,0,1,0, 0,0,0,0,0,0,1,1],
6827 #[1,0,0,0,0,0,0,0, 0,1,0,0,0,0,0,0, 0,0,1,1,0,0,0,0, 0,0,1,0,0,0,0,0, 0,0,0,0,1,1
6827 ,0,0, 0,0,0,0,1,0,0,0, 0,0,0,0,0,0,1,0, 0,0,0,0,0,0,0,1],
6828 #[1,0,0,0,0,0,0,0, 0,1,0,0,0,0,0,0, 0,0,0,1,0,0,0,0, 0,0,1,1,0,0,0,0, 0,0,0,0,0,1
6828 ,0,0, 0,0,0,0,1,1,0,0, 1,0,0,0,0,0,1,0, 0,1,0,0,0,0,0,1],

```

```

6829 #[1,0,0,0,0,0,0,0, 1,1,0,0,0,0,0,0, 0,0,1,0,0,0,0,0, 0,0,1,1,0,0,0,0, 1,0,1,0,1,0
,0,0, 1,1,1,1,1,1,0,0, 0,0,1,0,0,0,1,0, 0,0,1,1,0,0,1,1],
6830 #[1,0,0,0,0,0,0,0, 0,1,0,0,0,0,0,0, 0,0,0,0,1,0,0,0, 0,0,0,0,0,1,0,0, 0,0,1,0,0,0
,0,0, 0,0,0,1,0,0,0,0, 0,0,0,0,0,0,1,0, 0,0,0,0,0,0,0,1],
6831 #[0,0,1,0,0,0,0,0, 0,0,0,1,0,0,0,0, 1,0,0,0,0,0,0,0, 0,1,0,0,0,0,0,0, 0,0,0,0,0,0
,1,0, 0,0,0,0,0,0,1, 0,0,0,0,1,0,0,0, 0,0,0,0,0,1,0,0] >;
6832
6833 # group order = 155520
6834
6835
6836
6837 Hirschfeld_stab_projectivity_group:
6838 ▷ $(ORBITER) -v 4 \
6839 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
6840 ▷ ▷ -define Do -draw_options \
6841 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6842 ▷ ▷ -end \
6843 ▷ ▷ -define G -linear_group -PGGL 4 4 \
6844 ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld_Stab" \
6845 ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
6846 ▷ ▷ -end \
6847 ▷ ▷ -define G0 -modified_group -from G \
6848 ▷ ▷ ▷ -projectivity_subgroup \
6849 ▷ ▷ -end \
6850 ▷ ▷ -with G0 -do \
6851 ▷ ▷ -group_theoretic_activity \
6852 ▷ ▷ ▷ -report Do \
6853 ▷ ▷ -end
6854 ▷ $(PDFLATEX) PGGL_4_4_Subgroup_Hirschfeld_Stab_51840_ProjectivitySubgroup_report
.tex
6855 ▷ $(OPEN) PGGL_4_4_Subgroup_Hirschfeld_Stab_51840_ProjectivitySubgroup_report.pdf

6856
6857
6858
6859 Hirschfeld_stab_field_reduction:
6860 ▷ $(ORBITER) -v 4 \
6861 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
6862 ▷ ▷ -define Do -draw_options \
6863 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6864 ▷ ▷ -end \
6865 ▷ ▷ -define G -linear_group -PGGL 4 4 \
6866 ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld_Stab" \
6867 ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
6868 ▷ ▷ -end \
6869 ▷ ▷ -define G0 -modified_group -from G \
6870 ▷ ▷ ▷ -field_reduction \
6871 ▷ ▷ -end \
6872 ▷ ▷ -with G0 -do \
6873 ▷ ▷ -group_theoretic_activity \
6874 ▷ ▷ ▷ -report Do \
6875 ▷ ▷ -end
6876 ▷ $(PDFLATEX) PGL_8_2_Field.Reduction_report.tex
6877 ▷ $(OPEN) PGL_8_2_Field.Reduction_report.pdf
6878
6879
6880
6881 # related to planes.in.pencil:
6882 # we are computing the action on the planes through the line 0.

```

```

6883
6884
6885
6886 on_planes:
6887 ▷ $(ORBITER) -v 2 \
6888 ▷ ▷ -define F -finite_field -q 8 -end \
6889 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
6890 ▷ ▷ -define G -linear_group -PGL 4 F -end \
6891 ▷ ▷ -define G.on_planes -modified_group -from G \
6892 ▷ ▷ ▷ -on_k_subspaces 3 \
6893 ▷ ▷ -end \
6894 ▷ ▷ -with G.on_planes -do \
6895 ▷ ▷ -group_theoretic_activity \
6896 ▷ ▷ ▷ -apply "0,8,1,6,4,3,7,2,5" \
6897 ▷ ▷ ▷ "1,0,0,0, 0,1,0,0, 0,0,0,2, 0,0,1,1" \
6898 ▷ ▷ -end
6899 ▷ $(PDFLATEX) PGL_4_8_Gr_4_3_apply.tex
6900 ▷ $(OPEN) PGL_4_8_Gr_4_3_apply.pdf
6901
6902
6903 on_planes_restricted:
6904 ▷ $(ORBITER) -v 2 \
6905 ▷ ▷ -define Do -draw_options \
6906 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6907 ▷ ▷ -end \
6908 ▷ ▷ -define planes -vector -dense "0,8,1,6,4,3,7,2,5" -end \
6909 ▷ ▷ -define F -finite_field -q 8 -end \
6910 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
6911 ▷ ▷ -define G -linear_group -PGL 4 F \
6912 ▷ ▷ ▷ -subgroup_by_generators "cyclic" \
6913 ▷ ▷ ▷ 9 1 "1,0,0,0, 0,1,0,0, 0,0,0,2, 0,0,1,1" \
6914 ▷ ▷ -end \
6915 ▷ ▷ -define G.on_planes -modified_group -from G \
6916 ▷ ▷ ▷ -on_k_subspaces 3 \
6917 ▷ ▷ -end \
6918 ▷ ▷ -define Gr -modified_group -from G.on_planes \
6919 ▷ ▷ ▷ -restricted_action planes planes \
6920 ▷ ▷ -end \
6921 ▷ ▷ -with Gr -do \
6922 ▷ ▷ -group_theoretic_activity \
6923 ▷ ▷ ▷ -report Do \
6924 ▷ ▷ -end
6925 ▷ $(PDFLATEX) PGL_4_8_Gr_4_3_res_planes_report.tex
6926 ▷ $(OPEN) PGL_4_8_Gr_4_3_res_planes_report.pdf
6927
6928
6929 T3_on_tensors:
6930 ▷ $(ORBITER) -v 2 \
6931 ▷ ▷ -define Do -draw_options \
6932 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6933 ▷ ▷ -end \
6934 ▷ ▷ -define G \
6935 ▷ ▷ -linear_group -GL_d.q.wr.Sym.n 2 2 3 \
6936 ▷ ▷ ▷ -on_tensors -end \
6937 ▷ ▷ -with G -do \
6938 ▷ ▷ -group_theoretic_activity \
6939 ▷ ▷ ▷ -report Do \
6940 ▷ ▷ -end
6941 ▷ $(PDFLATEX) GL_2_2_wreath_Sym3_report.tex

```

```

6942 ▷ $(OPEN) GL_2.2.wreath.Sym3_report.pdf
6943 ▷
6944
6945
6946 T3r1:
6947 ▷ $(ORBITER) -v 4 \
6948 ▷ ▷ -define Do -draw.options \
6949 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6950 ▷ ▷ -end \
6951 ▷ ▷ -define G \
6952 ▷ ▷ -linear_group -GL_d.q.wr.Sym.n 2 2 3 \
6953 ▷ ▷ ▷ -on_rank_one_tensors -end \
6954 ▷ ▷ -with G -do \
6955 ▷ ▷ -group_theoretic.activity \
6956 ▷ ▷ ▷ -report Do \
6957 ▷ ▷ -end
6958 ▷ $(PDFLATEX) GL_2.2.wreath.Sym3_report.tex
6959 ▷ $(OPEN) GL_2.2.wreath.Sym3_report.pdf
6960
6961
6962
6963
6964
6965 T4_on_tensors:
6966 ▷ $(ORBITER) -v 4 \
6967 ▷ ▷ -define Do -draw.options \
6968 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6969 ▷ ▷ -end \
6970 ▷ ▷ -define G \
6971 ▷ ▷ -linear_group -GL_d.q.wr.Sym.n 2 2 4 \
6972 ▷ ▷ ▷ -on_tensors -end \
6973 ▷ ▷ -with G -do \
6974 ▷ ▷ -group_theoretic.activity \
6975 ▷ ▷ ▷ -report Do \
6976 ▷ ▷ -end
6977 ▷ $(PDFLATEX) GL_2.2.wreath.Sym4_report.tex
6978 ▷ $(OPEN) GL_2.2.wreath.Sym4_report.pdf
6979 ▷
6980 ▷
6981 T4r1:
6982 ▷ $(ORBITER) -v 4 \
6983 ▷ ▷ -define Do -draw.options \
6984 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
6985 ▷ ▷ -end \
6986 ▷ ▷ -define G \
6987 ▷ ▷ -linear_group -GL_d.q.wr.Sym.n 2 2 4 \
6988 ▷ ▷ ▷ -on_rank_one_tensors -end \
6989 ▷ ▷ -with G -do \
6990 ▷ ▷ -group_theoretic.activity \
6991 ▷ ▷ ▷ -report Do \
6992 ▷ ▷ -end
6993 ▷ $(PDFLATEX) GL_2.2.wreath.Sym4_report.tex
6994 ▷ $(OPEN) GL_2.2.wreath.Sym4_report.pdf
6995
6996
6997 # ToDo error message:
6998 #any_group::init_linear_group !LG->f_has.strong_generators
6999
7000 PGGL_2.8_on_conic:

```

```

7001 ▷ $(ORBITER) -v 4 \
7002 ▷ ▷ -define Do -draw_options \
7003 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
7004 ▷ ▷ -end \
7005 ▷ ▷ -define G -linear_group \
7006 ▷ ▷ ▷ -PGL 2 8 -PGL2OnConic \
7007 ▷ ▷ -end \
7008 ▷ ▷ -with G -do \
7009 ▷ ▷ -group_theoretic_activity \
7010 ▷ ▷ ▷ -report Do \
7011 ▷ ▷ -end
7012 ▷ #$(PDFLATEX) PGGL_2.8_OnConic.2.8_report.tex
7013 ▷ #$(OPEN) PGGL_2.8_OnConic.2.8_report.pdf
7014
7015 PGL_4.2_wedge:
7016 ▷ $(ORBITER) -v 3 \
7017 ▷ ▷ -define Do -draw_options \
7018 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
7019 ▷ ▷ -end \
7020 ▷ ▷ -define G -linear_group -PGL 4 2 -end \
7021 ▷ ▷ -define G2 -modified_group -from G \
7022 ▷ ▷ ▷ -on_wedge_product \
7023 ▷ ▷ -end \
7024 ▷ ▷ -with G -do \
7025 ▷ ▷ -group_theoretic_activity \
7026 ▷ ▷ ▷ -report Do \
7027 ▷ ▷ -end
7028 ▷ #$(PDFLATEX) PGL_4.2_report.tex
7029 ▷ #$(OPEN) PGL_4.2_report.pdf
7030
7031
7032
7033
7034 PGL_5.3_wedge:
7035 ▷ $(ORBITER) -v 3 \
7036 ▷ ▷ -define Do -draw_options \
7037 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
7038 ▷ ▷ -end \
7039 ▷ ▷ -define G -linear_group -PGL 5 3 -end \
7040 ▷ ▷ -define G2 -modified_group -from G \
7041 ▷ ▷ ▷ -on_wedge_product \
7042 ▷ ▷ -end \
7043 ▷ ▷ -with G -do \
7044 ▷ ▷ -group_theoretic_activity \
7045 ▷ ▷ ▷ -report Do \
7046 ▷ ▷ -end
7047 ▷ #$(PDFLATEX) PGL_5.3_report.tex
7048 ▷ #$(OPEN) PGL_5.3_report.pdf
7049
7050
7051
7052
7053
7054 PGL_4.2_wd:
7055 ▷ $(ORBITER) -v 3 \
7056 ▷ ▷ -define Do -draw_options \
7057 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
7058 ▷ ▷ -end \
7059 ▷ ▷ -define G -linear_group -PGL 4 2 -wedge_detached -end \

```

```

7060 ▷ ▷ -with G -do \
7061 ▷ ▷ -group_theoretic_activity \
7062 ▷ ▷ ▷ -report Do \
7063 ▷ ▷ -end \
7064 ▷ ▷ -print_symbols
7065 ▷ $(PDFLATEX) PGL_6_2.Wedge_4.2.detached_report.tex
7066 ▷ $(OPEN) PGL_6_2.Wedge_4.2.detached_report.pdf
7067
7068
7069
7070 PGL_5.3_wd:
7071 ▷ $(ORBITER) -v 3 \
7072 ▷ ▷ -define Do -draw_options \
7073 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
7074 ▷ ▷ -end \
7075 ▷ ▷ -define G -linear_group -PGL 5 3 -wedge_detached -end \
7076 ▷ ▷ -with G -do \
7077 ▷ ▷ -group_theoretic_activity \
7078 ▷ ▷ ▷ -report Do \
7079 ▷ ▷ -end
7080 ▷ $(PDFLATEX) PGL_10_3.Wedge_5.3.detached_report.tex
7081 ▷ $(OPEN) PGL_10_3.Wedge_5.3.detached_report.pdf
7082
7083
7084 PGL_4.2_wd_reverse:
7085 ▷ $(ORBITER) -v 3 \
7086 ▷ ▷ -define G -linear_group -PGL 4 2 -wedge_detached -end \
7087 ▷ ▷ -with G -do \
7088 ▷ ▷ -group_theoretic_activity \
7089 ▷ ▷ ▷ -reverse_isomorphism_exterior_square \
7090 ▷ ▷ -end
7091
7092 #linear_algebra::invert_matrix matrix is not invertible, the rank is 3
7093
7094
7095 M24_perm_group:
7096 ▷ $(ORBITER) -v 2 \
7097 ▷ ▷ -define Do -draw_options \
7098 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
7099 ▷ ▷ -end \
7100 ▷ ▷ -define v -vector \
7101 ▷ ▷ ▷ -dense $(GOLAY_24.CODE_PROJECTIVE) \
7102 ▷ ▷ -end \
7103 ▷ ▷ -define F -finite_field -q 2 -end \
7104 ▷ ▷ -define G -linear_group -PGL 12 2 \
7105 ▷ ▷ -subgroup_by_generators "M24" "244823040" 12 \
7106 ▷ ▷ ▷ $(GOLAY_24.CODE_AUT_GEN) \
7107 ▷ ▷ -end \
7108 ▷ ▷ -define M24_perm -modified_group -from G \
7109 ▷ ▷ ▷ -restricted_action v \\\_on\\\_code \
7110 ▷ ▷ -end \
7111 ▷ ▷ -with M24_perm -do \
7112 ▷ ▷ -group_theoretic_activity \
7113 ▷ ▷ ▷ -report Do \
7114 ▷ ▷ -end
7115 ▷ $(PDFLATEX) PGL_12.2_res.v_report.tex
7116 ▷ $(OPEN) PGL_12.2_res.v_report.pdf
7117
7118

```



```

7119
7120 # 2 point set stabilizer in M24:
7121
7122 M22_bar:
7123 ▷ $(ORBITER) -v 2 \
7124 ▷ ▷ -define Do -draw_options \
7125 ▷ ▷ ▷ -radius 50 -scale 0.5 -line_width 0.3 \
7126 ▷ ▷ -end \
7127 ▷ ▷ -define code -vector \
7128 ▷ ▷ ▷ -dense $(GOLAY_24_CODE_PROJECTIVE) \
7129 ▷ ▷ -end \
7130 ▷ ▷ -define Control2 -poset_classification_control \
7131 ▷ ▷ ▷ -problem_label M24_pairs -W -depth 2 \
7132 ▷ ▷ ▷ -draw_options Do \
7133 ▷ ▷ -end \
7134 ▷ ▷ -define F -finite_field -q 2 -end \
7135 ▷ ▷ -define G -linear_group -PGL 12 2 \
7136 ▷ ▷ -subgroup_by_generators "M24" "244823040" 12 \
7137 ▷ ▷ ▷ $(GOLAY_24_CODE_AUT_GEN) \
7138 ▷ ▷ -end \
7139 ▷ ▷ -define M24_perm -modified_group \
7140 ▷ ▷ ▷ -from G -restricted_action code \_on\_code \
7141 ▷ ▷ -end \
7142 ▷ ▷ -define M22_perm -modified_group \
7143 ▷ ▷ ▷ -from M24_perm \
7144 ▷ ▷ ▷ -set_stabilizer 0,1 Control2 \
7145 ▷ ▷ -end \
7146 ▷ ▷ -define pts -set -loop 2 22 1 -end \
7147 ▷ ▷ -define M22 -modified_group \
7148 ▷ ▷ ▷ -from M22_perm \
7149 ▷ ▷ ▷ -restricted_action pts M22 \
7150 ▷ ▷ -end \
7151 ▷ ▷ -with M22 -do \
7152 ▷ ▷ -group_theoretic_activity \
7153 ▷ ▷ ▷ -report Do \
7154 ▷ ▷ -end
7155 ▷ $(PDFLATEX) PGL_12_2_res_code_res_pts_report.tex
7156 ▷ $(OPEN) PGL_12_2_res_code_res_pts_report.pdf
7157
7158
7159 # M22_bar has order 887040
7160
7161
7162
7163 curve_q13_f0_clean2_variety_make_group:
7164 ▷ $(ORBITER) -v 6 \
7165 ▷ ▷ -define Do -draw_options \
7166 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
7167 ▷ ▷ -end \
7168 ▷ ▷ -define F -finite_field -q 13 -end \
7169 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
7170 ▷ ▷ -define R -polynomial_ring \
7171 ▷ ▷ ▷ -field F \
7172 ▷ ▷ ▷ -number_of_variables 3 \
7173 ▷ ▷ ▷ -homogeneous_of_degree 4 \
7174 ▷ ▷ ▷ -monomial_ordering_partition \
7175 ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
7176 ▷ ▷ -end \
7177 ▷ ▷ -define V -variety \

```

```

7178 ▷ ▷ ▷ -projective_space P \
7179 ▷ ▷ ▷ -ring R \
7180 ▷ ▷ ▷ -equation_in_algebraic_form \
7181 ▷ ▷ ▷ ▷ $(CURVE_Q13_F0_CLEAN2_EQUATION) \
7182 ▷ ▷ ▷ -label.txt curve.q13.f0.clean2 \
7183 ▷ ▷ ▷ -label.tex "{\\rm curve\\_q13\\_f0\\_clean2}" \
7184 ▷ ▷ -end \
7185 ▷ ▷ -define G -modified_group \
7186 ▷ ▷ ▷ -stabilizer_of_variety V \
7187 ▷ ▷ ▷ -nauty_control -nauty_log "quartic_curve.q13.f0.nauty_log.txt" -end \
7188 ▷ ▷ -end \
7189 ▷ ▷ -with G -do \
7190 ▷ ▷ -group_theoretic_activity \
7191 ▷ ▷ ▷ -report Do \
7192 ▷ ▷ -end
7193 ▷ $(PDFLATEX) PGL_3_13_stab.of.curve.q13.f0.clean2.report.tex
7194 ▷ $(OPEN) PGL_3_13_stab.of.curve.q13.f0.clean2.report.pdf
7195 ▷ $(ORBITER) -v 2 \
7196 ▷ ▷ -define Do -draw_options -embedded -radius 150 \
7197 ▷ ▷ ▷ -nodes_empty \
7198 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
7199 ▷ ▷ -tree_draw \
7200 ▷ ▷ ▷ -file quartic_curve.q13.f0.nauty_log.txt \
7201 ▷ ▷ ▷ -draw_options Do \
7202 ▷ ▷ -end
7203 ▷ $(PDFLATEX) quartic_curve.q13.f0.nauty_log_draw.tex
7204 ▷ $(OPEN) quartic_curve.q13.f0.nauty_log_draw.pdf
7205
7206
7207 #quartic_curve.q13.f0.nauty_log.txt
7208
7209 conic_group_q5:
7210 ▷ $(ORBITER) -v 6 \
7211 ▷ ▷ -define Do -draw_options \
7212 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
7213 ▷ ▷ -end \
7214 ▷ ▷ -define F -finite_field -q 5 -end \
7215 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
7216 ▷ ▷ -define R -polynomial_ring \
7217 ▷ ▷ ▷ -field F \
7218 ▷ ▷ ▷ -number_of_variables 3 \
7219 ▷ ▷ ▷ -homogeneous_of_degree 2 \
7220 ▷ ▷ ▷ -monomial_ordering_partition \
7221 ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
7222 ▷ ▷ -end \
7223 ▷ ▷ -define V -variety \
7224 ▷ ▷ ▷ -projective_space P \
7225 ▷ ▷ ▷ -ring R \
7226 ▷ ▷ ▷ -equation_in_algebraic_form "X1^2-X0*X2" \
7227 ▷ ▷ ▷ -label.txt conic.q5 \
7228 ▷ ▷ ▷ -label.tex "{\\rm conic\\_q5}" \
7229 ▷ ▷ -end \
7230 ▷ ▷ -define G -modified_group \
7231 ▷ ▷ ▷ -stabilizer_of_variety V \
7232 ▷ ▷ ▷ -nauty_control -nauty_log "conic.q5.nauty_log.txt" \
7233 ▷ ▷ ▷ -end \
7234 ▷ ▷ -end \
7235 ▷ ▷ -with G -do \
7236 ▷ ▷ -group_theoretic_activity \

```

```

7237 ▷ ▷ ▷ -report Do \
7238 ▷ ▷ -end
7239 ▷ $(PDFLATEX) PGL_3_5_stab_of_conic_q5_report.tex
7240 ▷ $(OPEN) PGL_3_5_stab_of_conic_q5_report.pdf
7241
7242 group_of_Hirschfeld_surface_q4:
7243 ▷ $(ORBITER) -v 6 \
7244 ▷ ▷ -define Do -draw.options \
7245 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
7246 ▷ ▷ -end \
7247 ▷ ▷ -define F -finite_field -q 4 -compute_related_fields -end \
7248 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
7249 ▷ ▷ -define R -polynomial_ring \
7250 ▷ ▷ ▷ -field F \
7251 ▷ ▷ ▷ -number_of_variables 4 \
7252 ▷ ▷ ▷ -homogeneous_of_degree 3 \
7253 ▷ ▷ ▷ -monomial_ordering_partition \
7254 ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
7255 ▷ ▷ -end \
7256 ▷ ▷ -define V -variety \
7257 ▷ ▷ ▷ -projective_space P \
7258 ▷ ▷ ▷ -ring R \
7259 ▷ ▷ ▷ -equation_in_algebraic_form \
7260 ▷ ▷ ▷ ▷ "X0^2*X3+X0*X3^2+X1^2*X2+X1*X2^2" \
7261 ▷ ▷ ▷ -label.txt Hirschfeld_surface_q4 \
7262 ▷ ▷ ▷ -label.tex "{\\rm Hirschfeld\\_surface\\_q4}" \
7263 ▷ ▷ -end \
7264 ▷ ▷ -define G -modified_group \
7265 ▷ ▷ ▷ -stabilizer_of_variety V -nauty_control -nauty_log "Hirschfeld.q4.nauty.log
7266 ▷ ▷ -end \
7267 ▷ ▷ -with G -do \
7268 ▷ ▷ -group_theoretic_activity \
7269 ▷ ▷ ▷ -report Do \
7270 ▷ ▷ -end \
7271 ▷ ▷ -with V -do \
7272 ▷ ▷ -variety_activity \
7273 ▷ ▷ ▷ -report \
7274 ▷ ▷ -end
7275 ▷ $(PDFLATEX) variety_Hirschfeld_surface_q4_report.tex
7276 ▷ $(OPEN) variety_Hirschfeld_surface_q4_report.pdf
7277 ▷ $(PDFLATEX) PGGL_4_4_stab_of_Hirschfeld_surface_q4_report.tex
7278 ▷ $(OPEN) PGGL_4_4_stab_of_Hirschfeld_surface_q4_report.pdf
7279
7280 Higman_Sims_on_176_points:
7281 ▷ $(ORBITER) -v 2 \
7282 ▷ ▷ -define Do -draw.options \
7283 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
7284 ▷ ▷ -end \
7285 ▷ ▷ -define gens -vector -dense \
7286 ▷ ▷ ▷ $(HIGMAN_SIMS_GENS) \
7287 ▷ ▷ -end \
7288 ▷ ▷ -define G -permutation_group -symmetric_group 100 \
7289 ▷ ▷ ▷ -subgroup_by_generators HS \
7290 ▷ ▷ ▷ $(HIGMAN_SIMS_GO) \
7291 ▷ ▷ ▷ $(HIGMAN_SIMS_NB_GENS) gens \
7292 ▷ ▷ -end \
7293 ▷ ▷ -with G -do \
7294 ▷ ▷ -group_theoretic_activity \

```

```

7295 ▷ ▷ ▷ -subgroup_lattice_magma \
7296 ▷ ▷ -end \
7297 ▷ ▷ -define H176a -modified_group -from G \
7298 ▷ ▷ ▷ -subgroup_by_lattice 585 \
7299 ▷ ▷ -end \
7300 ▷ ▷ -define G176a -modified_group -from G \
7301 ▷ ▷ ▷ -on_cosets_of_subgroup H176a \
7302 ▷ ▷ -end \
7303 ▷ ▷ -with G176a -do \
7304 ▷ ▷ -group_theoretic_activity \
7305 ▷ ▷ ▷ -report Do \
7306 ▷ ▷ -end
7307 ▷ $(PDFLATEX) Sym_100_Subgroup_HS_44352000_OnCosetsOfSubgroup_report.tex
7308 ▷ $(OPEN) Sym_100_Subgroup_HS_44352000_OnCosetsOfSubgroup_report.pdf
7309
7310
7311 #error in action_global::report after A->report_what_we_act_on
7312
7313
7314
7315 group_U_4_3:
7316 ▷ $(ORBITER) -v 6 \
7317 ▷ ▷ -define Do -draw_options \
7318 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
7319 ▷ ▷ -end \
7320 ▷ ▷ -define F -finite_field -q 9 -compute_related_fields -end \
7321 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
7322 ▷ ▷ -define R -polynomial_ring \
7323 ▷ ▷ ▷ -field F \
7324 ▷ ▷ ▷ -number_of_variables 4 \
7325 ▷ ▷ ▷ -homogeneous_of_degree 4 \
7326 ▷ ▷ ▷ -monomial_ordering_partition \
7327 ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
7328 ▷ ▷ -end \
7329 ▷ ▷ -define V -variety \
7330 ▷ ▷ ▷ -projective_space P \
7331 ▷ ▷ ▷ -ring R \
7332 ▷ ▷ ▷ -equation_in_algebraic_form \
7333 ▷ ▷ ▷ ▷ $(HERMITIAN_SURFACE_Q9_EQUATION) \
7334 ▷ ▷ ▷ -label_txt hermitian_surface.q3 \
7335 ▷ ▷ ▷ -label_tex "{\\rm hermitian\\_surface\\_q3}" \
7336 ▷ ▷ -end \
7337 ▷ ▷ -define G -modified_group \
7338 ▷ ▷ -stabilizer_of_variety V -nauty_control -nauty_log "U_4_3_nauty_log.txt" -e
7339 nd \
7339 ▷ ▷ -end \
7340 ▷ ▷ -with G -do \
7341 ▷ ▷ -group_theoretic_activity \
7342 ▷ ▷ ▷ -report Do \
7343 ▷ ▷ -end \
7344 ▷ ▷ -define GP -modified_group -from G \
7345 ▷ ▷ ▷ -projectivity_subgroup \
7346 ▷ ▷ -end \
7347 ▷ ▷ -define GPD -modified_group -from GP \
7348 ▷ ▷ ▷ -derived_subgroup \
7349 ▷ ▷ -end \
7350 ▷ ▷ -with GPD -do \
7351 ▷ ▷ -group_theoretic_activity \
7352 ▷ ▷ ▷ -report Do \

```

```

7353 ▷ ▷ -end \
7354 ▷ ▷ -define v -vector \
7355 ▷ ▷ ▷ -dense $(HERMITIAN_SURFACE_Q9_POINTS) \
7356 ▷ ▷ -end \
7357 ▷ ▷ -define GPDr -modified_group \
7358 ▷ ▷ ▷ -from GPD \
7359 ▷ ▷ ▷ -restricted_action v \
7360 ▷ ▷ ▷ "{\\rm \\on\\_hermitiansurface}" \
7361 ▷ ▷ -end \
7362 ▷ ▷ -with GPDr -do \
7363 ▷ ▷ -group_theoretic_activity \
7364 ▷ ▷ ▷ -report Do \
7365 ▷ ▷ -end
7366 ▷ $(PDFLATEX) PGGL_4.9_stab_of_hermitian_surface.q3_res.v_report.tex
7367 ▷ $(OPEN) PGGL_4.9_stab_of_hermitian_surface.q3_res.v_report.pdf
7368
7369
7370
7371 #####
7372 # Section 5.5: Group Theoretic Activities
7373
7374
7375 SECTION.GROUP.THEORETIC.ACTIVITIES:
7376
7377
7378 test_5.5:
7379 ▷ make PGL_3.2_order_invariant
7380 ▷ make PGL_3.2_elements
7381 ▷ make Sym_3_elements
7382 ▷ make Sym_3_make_element_tree
7383 ▷ make Cycle_13_power
7384 ▷ make Cycle_12_power
7385 ▷ make PGL_3.4_singer
7386 ▷ make GL_2.8_multiply
7387 ▷ make GL_2.7_multiply
7388 ▷ make GL_2.7_inv
7389 ▷ make GL_2.7_power
7390 ▷ make PGL_3.2_classes
7391 ▷ make PGL_4.2_classes_based_on_normal_form
7392 ▷ make PGL_10.2_classes_based_on_normal_form
7393 ▷ make PGGL_4.4_small_generating_set
7394 ▷ make normal_forms_PGL_4.4
7395 ▷ make PGL_4.4.2A_rank
7396 ▷ make PGL_4.4.2A_unrank
7397 ▷ make PGL_4.5.3B_rank
7398 ▷ make PGL_4.5.3B_unrank
7399 ▷ make normal_forms_PGL_4.5
7400 ▷ make on_planes
7401 ▷ make PGL_4.2_random_element
7402 ▷ make PGL_2.3_subgroup_lattice
7403 ▷ make PGL_2.3_subgroup_lattice_load_and_draw_by_groups
7404 ▷ make PGL_2.3_subgroup_lattice_load_and_draw_by_orbits
7405
7406
7407 PGL_3.2_order_invariant:
7408 ▷ $(ORBITER) -v 5 \
7409 ▷ ▷ -define G -linear_group -PGL 3 2 -end \
7410 ▷ ▷ -with G -do \
7411 ▷ ▷ -group_theoretic_activity \

```

```

7412 ▷ ▷ ▷ -order_invariant \
7413 ▷ ▷ -end
7414 ▷ $(PDFLATEX) PGL_3_2_order_invariant.tex
7415 ▷ $(OPEN) PGL_3_2_order_invariant.pdf
7416
7417
7418
7419 PGL_3_2_elements:
7420 ▷ $(ORBITER) -v 5 \
7421 ▷ ▷ -define G -linear_group -PGL 3 2 -end \
7422 ▷ ▷ -with G -do \
7423 ▷ ▷ -group_theoretic_activity \
7424 ▷ ▷ ▷ -save_elements_csv "PGL_3_2_elements.csv" \
7425 ▷ ▷ -end
7426
7427 # creates PGL_3_2_elements.csv
7428
7429
7430
7431
7432 Sym_3_elements:
7433 ▷ $(ORBITER) -v 3 \
7434 ▷ ▷ -define G -permutation_group -symmetric_group 3 -end \
7435 ▷ ▷ -with G -do \
7436 ▷ ▷ -group_theoretic_activity \
7437 ▷ ▷ ▷ -print_elements_tex \
7438 ▷ ▷ -end
7439 ▷ $(PDFLATEX) Sym_3_elements.tex
7440 ▷ $(OPEN) Sym_3_elements.pdf
7441
7442
7443 Sym_3_make_element_tree:
7444 ▷ $(ORBITER) -v 2 \
7445 ▷ ▷ -define G -permutation_group -symmetric_group 3 -end \
7446 ▷ ▷ -with G -do \
7447 ▷ ▷ -group_theoretic_activity \
7448 ▷ ▷ ▷ -make_element_tree \
7449 ▷ ▷ -end
7450 ▷ ▷ -define Do -draw_options \
7451 ▷ ▷ ▷ -nodes \
7452 ▷ ▷ ▷ -embedded -radius 250 \
7453 ▷ ▷ ▷ -xin 10000 -yin 10000 \
7454 ▷ ▷ ▷ -xout 1000000 -yout 600000 \
7455 ▷ ▷ ▷ -scale 0.3 -line_width 1.0 \
7456 ▷ ▷ -end \
7457 ▷ ▷ -tree_draw \
7458 ▷ ▷ ▷ -file Sym_3_elements_tree.txt \
7459 ▷ ▷ ▷ -draw_options Do \
7460 ▷ ▷ -end
7461 ▷ $(PDFLATEX) Sym_3_elements_tree_draw.tex
7462 ▷ $(OPEN) Sym_3_elements_tree_draw.pdf
7463
7464
7465
7466 Cycle_13_power:
7467 ▷ $(ORBITER) -v 5 \
7468 ▷ ▷ -define G -permutation_group -symmetric_group 13 -end \
7469 ▷ ▷ -with G -do \
7470 ▷ ▷ -group_theoretic_activity \

```

```

7471 ▷ ▷ ▷ -consecutive_powers \
7472 ▷ ▷ ▷ "1,2,3,4,5,6,7,8,9,10,11,12,0" 13 \
7473 ▷ ▷ -end
7474 ▷ $(PDFLATEX) Sym_13_all_powers.tex
7475 ▷ $(OPEN) Sym_13_all_powers.pdf
7476
7477
7478 Cycle_12_power:
7479 ▷ $(ORBITER) -v 5 \
7480 ▷ ▷ -define G -permutation_group -symmetric_group 12 -end \
7481 ▷ ▷ -with G -do \
7482 ▷ ▷ -group_theoretic_activity \
7483 ▷ ▷ ▷ -consecutive_powers \
7484 ▷ ▷ ▷ "1,2,3,4,5,6,7,8,9,10,11,0" 12 \
7485 ▷ ▷ -end
7486 ▷ $(PDFLATEX) Sym_12_all_powers.tex
7487 ▷ $(OPEN) Sym_12_all_powers.pdf
7488 ▷
7489
7490
7491
7492
7493 PGL_3_4_singer:
7494 ▷ $(ORBITER) -v 5 \
7495 ▷ ▷ -define G -linear_group -PGL 3 4 -end \
7496 ▷ ▷ -with G -do \
7497 ▷ ▷ -group_theoretic_activity \
7498 ▷ ▷ ▷ -find_singer_cycle \
7499 ▷ ▷ -end
7500
7501
7502 GL_2_8_multiply:
7503 ▷ $(ORBITER) -v 5 \
7504 ▷ ▷ -define G -linear_group -GL 2 8 -end \
7505 ▷ ▷ -with G -do \
7506 ▷ ▷ -group_theoretic_activity \
7507 ▷ ▷ ▷ -multiply "0,1,2,3" "4,5,6,7" \
7508 ▷ ▷ -end
7509 ▷ $(PDFLATEX) GL_2_8_mult.tex
7510 ▷ $(OPEN) GL_2_8_mult.pdf
7511
7512
7513 GL_2_7_multiply:
7514 ▷ $(ORBITER) -v 5 \
7515 ▷ ▷ -define G -linear_group -GL 2 7 -end \
7516 ▷ ▷ -with G -do \
7517 ▷ ▷ -group_theoretic_activity \
7518 ▷ ▷ ▷ -multiply "0,1,2,3" "4,5,6,0" \
7519 ▷ ▷ -end
7520 ▷ $(PDFLATEX) GL_2_7_mult.tex
7521 ▷ $(OPEN) GL_2_7_mult.pdf
7522
7523
7524 GL_2_7_inv:
7525 ▷ $(ORBITER) -v 5 \
7526 ▷ ▷ -define G -linear_group -GL 2 7 -end \
7527 ▷ ▷ -with G -do \
7528 ▷ ▷ -group_theoretic_activity \
7529 ▷ ▷ ▷ -inverse "0,1,2,3" \

```

```

7530 ▷ ▷ -end
7531 ▷ $(PDFLATEX) GL_2_7_inv.tex
7532 ▷ $(OPEN) GL_2_7_inv.pdf
7533
7534 GL_2_7_power:
7535 ▷ $(ORBITER) -v 5 \
7536 ▷ ▷ -define G -linear_group -GL 2 7 -end \
7537 ▷ ▷ -with G -do \
7538 ▷ ▷ -group_theoretic_activity \
7539 ▷ ▷ ▷ -raise_to_the_power "0,1,2,3" 2 \
7540 ▷ ▷ -end
7541 ▷ $(PDFLATEX) GL_2_7_power.tex
7542 ▷ $(OPEN) GL_2_7_power.pdf
7543
7544
7545 PGL_3_2_classes:
7546 ▷ $(ORBITER) -v 3 \
7547 ▷ ▷ -define G -linear_group -PGL 3 2 -end \
7548 ▷ ▷ -with G -do \
7549 ▷ ▷ -group_theoretic_activity \
7550 ▷ ▷ ▷ -classes_based_on_normal_form \
7551 ▷ ▷ -end
7552 ▷ $(PDFLATEX) PGL_3_2_classes_based_on_normal_forms_3_2.tex
7553 ▷ $(OPEN) PGL_3_2_classes_based_on_normal_forms_3_2.pdf
7554 ▷ #$(PDFLATEX) PGL_3_2_classes_out.tex
7555 ▷ #$(OPEN) PGL_3_2_classes_out.pdf
7556
7557 #▷ ▷ -classes \
7558
7559
7560
7561 PGL_4_2_classes_based_on_normal_form:
7562 ▷ $(ORBITER) -v 3 \
7563 ▷ ▷ -define G -linear_group -PGL 4 2 -end \
7564 ▷ ▷ -with G -do \
7565 ▷ ▷ -group_theoretic_activity \
7566 ▷ ▷ ▷ -classes_based_on_normal_form \
7567 ▷ ▷ -end
7568 ▷ $(PDFLATEX) PGL_4_2_classes_based_on_normal_forms_4_2.tex
7569 ▷ $(OPEN) PGL_4_2_classes_based_on_normal_forms_4_2.pdf
7570
7571
7572 PGL_10_2_classes_based_on_normal_form:
7573 ▷ $(ORBITER) -v 3 \
7574 ▷ ▷ -define G -linear_group -PGL 10 2 -end \
7575 ▷ ▷ -with G -do \
7576 ▷ ▷ -group_theoretic_activity \
7577 ▷ ▷ ▷ -classes_based_on_normal_form \
7578 ▷ ▷ -end
7579 ▷ $(PDFLATEX) PGL_10_2_classes_based_on_normal_forms_10_2.tex
7580 ▷ $(OPEN) PGL_10_2_classes_based_on_normal_forms_10_2.pdf
7581
7582
7583
7584 PGGL_4_4_small_generating_set:
7585 ▷ $(ORBITER) -v 4 \
7586 ▷ ▷ -define F -finite_field -q 4 -end \
7587 ▷ ▷ -define G -linear_group -PGGL 4 F -end \
7588 ▷ ▷ -with G -do \

```



```

7589 ▷ ▷ -group_theoretic_activity \
7590 ▷ ▷ ▷ -find_small_generating_set 2 1000 \
7591 ▷ ▷ -end
7592
7593
7594
7595
7596 normal_forms_PGL_4_4:
7597 ▷ $(ORBITER) -v 7 \
7598 ▷ ▷ -define G -linear_group -PGGL 4 4 -end \
7599 ▷ ▷ -with G -do \
7600 ▷ ▷ -group_theoretic_activity \
7601 ▷ ▷ ▷ -classes_based_on_normal_form \
7602 ▷ ▷ -end
7603 ▷ $(PDFLATEX) PGGL_4_4_classes_based_on_normal_forms_4_4.tex
7604 ▷ $(OPEN) PGGL_4_4_classes_based_on_normal_forms_4_4.pdf
7605
7606
7607
7608
7609 PGL_4_4_2A_rank:
7610 ▷ $(ORBITER) -v 6 \
7611 ▷ ▷ -define G -linear_group -PGGL 4 4 -end \
7612 ▷ ▷ -with G -do \
7613 ▷ ▷ -group_theoretic_activity \
7614 ▷ ▷ ▷ -element_rank \
7615 ▷ ▷ ▷ "1,0,0,0, 0,1,0,0, 0,0,1,0, 0,0,0,1, 1" \
7616 ▷ ▷ -end
7617
7618
7619 PGL_4_4_2A_unrank:
7620 ▷ $(ORBITER) -v 6 \
7621 ▷ ▷ -define G -linear_group -PGGL 4 4 -end \
7622 ▷ ▷ -with G -do \
7623 ▷ ▷ -group_theoretic_activity \
7624 ▷ ▷ ▷ -element_unrank "1" \
7625 ▷ ▷ -end
7626 ▷ ▷
7627
7628
7629
7630
7631
7632
7633 PGL_4_5_3B_rank:
7634 ▷ $(ORBITER) -v 6 \
7635 ▷ ▷ -define G -linear_group -PGL 4 5 -end \
7636 ▷ ▷ -with G -do \
7637 ▷ ▷ -group_theoretic_activity \
7638 ▷ ▷ ▷ -element_rank "0,0,0,1, 2,3,0,1, 0,3,4,4, 0,1,2,1" \
7639 ▷ ▷ -end
7640
7641
7642 PGL_4_5_3B_unrank:
7643 ▷ $(ORBITER) -v 6 \
7644 ▷ ▷ -define G -linear_group -PGL 4 5 -end \
7645 ▷ ▷ -with G -do \
7646 ▷ ▷ -group_theoretic_activity \
7647 ▷ ▷ ▷ -element_unrank "701459351" \

```

```

7648 ▷ ▷ -end
7649 ▷ ▷
7650
7651
7652
7653 normal_forms_PGL_4_5:
7654 ▷ $(ORBITER) -v 7 \
7655 ▷ ▷ -define G -linear_group -PGL 4 5 -end \
7656 ▷ ▷ -with G -do \
7657 ▷ ▷ -group_theoretic_activity \
7658 ▷ ▷ ▷ -classes_based_on_normal_form \
7659 ▷ ▷ -end
7660 ▷ $(PDFLATEX) PGL_4_5_classes_based_on_normal_forms_4_5.tex
7661 ▷ $(OPEN) PGL_4_5_classes_based_on_normal_forms_4_5.pdf
7662
7663
7664
7665 PGL_4_2_random_element:
7666 ▷ $(ORBITER) -v 2 \
7667 ▷ ▷ -define F -finite_field -q 2 -end \
7668 ▷ ▷ -define G -linear_group -PGL 4 F -end \
7669 ▷ ▷ -with G -do \
7670 ▷ ▷ -group_theoretic_activity \
7671 ▷ ▷ ▷ -random_element r1 \
7672 ▷ ▷ -end \
7673 ▷ ▷ -print_symbols
7674
7675
7676 #group order H = 20160
7677 #Element :
7678 #1 0 1 1
7679 #0 1 1 0
7680 #0 1 0 1
7681 #1 0 1 0
7682
7683
7684 #coded: ( 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0 )
7685 #Element as permutation:
7686 #(0, 13, 1, 7, 12, 4, 2, 10, 5, 11, 14, 9, 3, 6, 8)
7687 #In list notation:
7688 #( 13, 7, 10, 6, 2, 11, 8, 12, 0, 3, 5, 14, 4, 1, 9 )
7689 #The rank of the element is 9883
7690
7691 #\{0,1,2,3,4\} -> \{13,7,10,6,2\} = \{2,6,7,10,13\}
7692
7693
7694
7695
7696
7697
7698
7699
7700
7701
7702 PGL_2_3_subgroup_lattice:
7703 ▷ $(ORBITER) -v 3 \
7704 ▷ ▷ -define F -finite_field -q 3 -end \
7705 ▷ ▷ -define G -linear_group -PGL 2 F -end \
7706 ▷ ▷ -with G -do \

```

```

7707 ▷ ▷ -group_theoretic.activity \
7708 ▷ ▷ ▷ -subgroup_lattice \
7709 ▷ ▷ -end
7710
7711
7712 # writes PGL_2.3.subgroup_lattice.csv
7713
7714
7715 PGL_2.3.subgroup_lattice_load_and_draw_by_groups:
7716 ▷ $(ORBITER) -v 3 \
7717 ▷ ▷ -define Do -draw.options -embedded -end \
7718 ▷ ▷ -define F -finite_field -q 3 -end \
7719 ▷ ▷ -define G -linear_group -PGL 2 F -end \
7720 ▷ ▷ -with G -do \
7721 ▷ ▷ -group_theoretic.activity \
7722 ▷ ▷ ▷ -subgroup_lattice_load PGL_2.3.subgroup_lattice.csv \
7723 ▷ ▷ -end \
7724 ▷ ▷ -with G -do \
7725 ▷ ▷ -group_theoretic.activity \
7726 ▷ ▷ ▷ -subgroup_lattice_draw_by_groups Do \
7727 ▷ ▷ -end
7728 ▷ $(PDFLATEX) PGL_2.3.drawing.tex
7729 ▷ $(OPEN) PGL_2.3.drawing.pdf
7730
7731
7732
7733 PGL_2.3.subgroup_lattice_load_and_draw_by_orbits:
7734 ▷ $(ORBITER) -v 3 \
7735 ▷ ▷ -define Do -draw.options -embedded -end \
7736 ▷ ▷ -define F -finite_field -q 3 -end \
7737 ▷ ▷ -define G -linear_group -PGL 2 F -end \
7738 ▷ ▷ -with G -do \
7739 ▷ ▷ -group_theoretic.activity \
7740 ▷ ▷ ▷ -subgroup_lattice_load PGL_2.3.subgroup_lattice.csv \
7741 ▷ ▷ -end \
7742 ▷ ▷ -with G -do \
7743 ▷ ▷ -group_theoretic.activity \
7744 ▷ ▷ ▷ -subgroup_lattice_draw_by_orbits Do \
7745 ▷ ▷ -end
7746 ▷ $(PDFLATEX) PGL_2.3.drawing_by_orbits.tex
7747 ▷ $(OPEN) PGL_2.3.drawing_by_orbits.pdf
7748
7749
7750
7751
7752
7753
7754 PGL_3.13.2B_RNF:
7755 ▷ $(ORBITER) -v 3 \
7756 ▷ ▷ -define F -finite_field -q 13 -end \
7757 ▷ ▷ -define G -linear_group -PGL 3 F -end \
7758 ▷ ▷ -with G -do \
7759 ▷ ▷ -group_theoretic.activity \
7760 ▷ ▷ ▷ -rational_normal_form $(PGL_3.13_CLASS_2B.REP) \
7761 ▷ ▷ -end
7762
7763
7764
7765

```

```

7766 #####
7767 # Section 5.6: Group Theoretic Activities Based on Magma
7768
7769
7770 SECTION.GROUP.THEORETIC.ACTIVITIES.BASED.ON.MAGMA:
7771
7772
7773 test_5.6:
7774 ▷ make Symmetric_4_diagram_of_conjugacy_classes
7775 ▷ make PSP_4_4_export_magma
7776 ▷ make PGGL_2_4_export_magma
7777 ▷ make Hirschfeld_stab_export_magma
7778 ▷ make Hirschfeld_stab_subgroup_lattice_magma
7779 ▷ make PGGL_2_4_classes
7780 ▷ make PGLstar_3_2_classes
7781 ▷ make PGL_4_2_classes
7782 ▷ make PGL_3_11_classes
7783 ▷ make PGL_7_2_classes
7784 ▷ make PGL_8_2_classes
7785 ▷ make Hirschfeld_stab_classes
7786 ▷ make PGGL_2_4_cent_2A
7787 ▷ make Normalizer_of_H5
7788 ▷ make PGGL_3_4_classes
7789 ▷ make classes_PGGL_4_4
7790 ▷ make classes_PGL_4_5
7791 ▷ #make PGL_4_5_3B_class_again
7792 ▷ make Normalizer_of_Z22_in_PGL_2_9
7793 ▷ make PGOm_6_2_classes
7794 ▷ make PGOm_6_2_Class_2A
7795 ▷ #make PGOm_6_2_Class_2A_print
7796 ▷ #make PGOm_6_2_Class_2A_centralizer
7797 ▷ #make PGOm_6_2_Class_2A_centralizer_recover
7798
7799
7800 Symmetric_4_diagram_of_conjugacy_classes:
7801 ▷ $(ORBITER) -v 3 \
7802 ▷ ▷ -define G -permutation-group -symmetric.group 4 -end \
7803 ▷ ▷ -with G -do \
7804 ▷ ▷ -group-theoretic.activity \
7805 ▷ ▷ ▷ -diagram \
7806 ▷ ▷ -end
7807 ▷ $(ORBITER) -v 2 \
7808 ▷ ▷ -draw.matrix \
7809 ▷ ▷ ▷ -input.csv_file.column Sym_4_class_diagram.csv Class \
7810 ▷ ▷ ▷ -box_width 50 \
7811 ▷ ▷ ▷ -bit_depth 24 \
7812 ▷ ▷ ▷ -indent 15 \
7813 ▷ ▷ ▷ -color_offset 1 \
7814 ▷ ▷ -end
7815 ▷ magick Sym_4_class_diagram_draw.bmp Sym_4_class_diagram_draw.pdf
7816 ▷ $(OPEN) Sym_4_class_diagram_draw.pdf
7817
7818
7819 PSP_4_4_export_magma:
7820 ▷ $(ORBITER) -v 5 \
7821 ▷ ▷ -define F -finite_field -q 4 -end \
7822 ▷ ▷ -define G -linear_group -PGL 4 F \
7823 ▷ ▷ ▷ -symplectic.group \
7824 ▷ ▷ -end \

```

```

7825 ▷ ▷ -with G -do \
7826 ▷ ▷ -group_theoretic_activity \
7827 ▷ ▷ ▷ -export_magma \
7828 ▷ ▷ -end
7829
7830 # creates PGL_4_4.Sp_4_4.generators.magma
7831
7832
7833 PGGL_2_4_export_magma:
7834 ▷ $(ORBITER) -v 20 \
7835 ▷ ▷ -define Do -draw_options \
7836 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
7837 ▷ ▷ -end \
7838 ▷ ▷ -define F -finite_field -q 4 \
7839 ▷ ▷ ▷ -compute_related_fields \
7840 ▷ ▷ -end \
7841 ▷ ▷ -define G -linear_group -PGGL 2 F \
7842 ▷ ▷ -end \
7843 ▷ ▷ -with G -do \
7844 ▷ ▷ -group_theoretic_activity \
7845 ▷ ▷ ▷ -report Do \
7846 ▷ ▷ -end \
7847 ▷ ▷ -with G -do \
7848 ▷ ▷ -group_theoretic_activity \
7849 ▷ ▷ ▷ -export_magma \
7850 ▷ ▷ -end
7851 ▷ $(PDFLATEX) PGGL_2_4_report.tex
7852 ▷ $(OPEN) PGGL_2_4_report.pdf
7853
7854
7855
7856
7857 Hirschfeld_stab_export_magma:
7858 ▷ $(ORBITER) -v 9 \
7859 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
7860 ▷ ▷ -define F -finite_field -q 4 \
7861 ▷ ▷ ▷ -compute_related_fields \
7862 ▷ ▷ -end \
7863 ▷ ▷ -define G -linear_group -PGGL 4 F \
7864 ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld_Stab" \
7865 ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
7866 ▷ ▷ ▷ -end \
7867 ▷ ▷ -with G -do \
7868 ▷ ▷ -group_theoretic_activity \
7869 ▷ ▷ ▷ -export_magma \
7870 ▷ ▷ -end
7871
7872
7873 Hirschfeld_stab_subgroup_lattice_magma:
7874 ▷ $(ORBITER) -v 3 \
7875 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
7876 ▷ ▷ -define F -finite_field -q 4 \
7877 ▷ ▷ ▷ -compute_related_fields \
7878 ▷ ▷ -end \
7879 ▷ ▷ -define G -linear_group -PGGL 4 F \
7880 ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld_Stab" \
7881 ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
7882 ▷ ▷ ▷ -end \
7883 ▷ ▷ -with G -do \

```

```

7884 ▷ ▷ -group_theoretic_activity \
7885 ▷ ▷ ▷ -subgroup_lattice_magma \
7886 ▷ ▷ -end
7887 ▷ $(ORBITER) -v 3 \
7888 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
7889 ▷ ▷ -define F -finite_field -q 4 \
7890 ▷ ▷ ▷ -compute_related_fields \
7891 ▷ ▷ -end \
7892 ▷ ▷ -define G -linear_group -PGGL 4 F \
7893 ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld_Stab" \
7894 ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
7895 ▷ ▷ ▷ -end \
7896 ▷ ▷ -with G -do \
7897 ▷ ▷ -group_theoretic_activity \
7898 ▷ ▷ ▷ -subgroup_lattice_magma \
7899 ▷ ▷ -end
7900 ▷ $(PDFLATEX) PGGL_4.4_Subgroup_Hirschfeld_Stab_51840_subgroup_lattice_out.tex
7901 ▷ $(OPEN) PGGL_4.4_Subgroup_Hirschfeld_Stab_51840_subgroup_lattice_out.pdf
7902
7903 PGGL_2.4_classes:
7904 ▷ $(ORBITER) -v 3 \
7905 ▷ ▷ -define G \
7906 ▷ ▷ -linear_group -PGGL 2 4 \
7907 ▷ ▷ -end \
7908 ▷ ▷ -with G -do \
7909 ▷ ▷ -group_theoretic_activity \
7910 ▷ ▷ ▷ -classes \
7911 ▷ ▷ -end \
7912 ▷ ▷ -print_symbols
7913 ▷ $(PDFLATEX) PGGL_2.4_classes.tex
7914 ▷ $(OPEN) PGGL_2.4_classes.pdf
7915
7916
7917
7918
7919 PGLstar_3.2_classes:
7920 ▷ $(ORBITER) -v 10 \
7921 ▷ ▷ -define F -finite_field -q 2 -end \
7922 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
7923 ▷ ▷ -define G0 -linear_group -PGL 3 F -end \
7924 ▷ ▷ -define G -modified_group -polarity_extension \
7925 ▷ ▷ ▷ G0 P \
7926 ▷ ▷ -end \
7927 ▷ ▷ -with G -do \
7928 ▷ ▷ -group_theoretic_activity \
7929 ▷ ▷ ▷ -classes \
7930 ▷ ▷ -end
7931
7932 PGL_4.2_classes:
7933 ▷ $(ORBITER) -v 3 \
7934 ▷ ▷ -define G -linear_group -PGL 4 2 -end \
7935 ▷ ▷ -with G -do \
7936 ▷ ▷ -group_theoretic_activity \
7937 ▷ ▷ ▷ -classes \
7938 ▷ ▷ -end
7939 ▷ $(PDFLATEX) PGL_4.2_report_classes_of_elements.tex
7940 ▷ $(OPEN) PGL_4.2_report_classes_of_elements.pdf
7941
7942

```

```

7943
7944 PGL_3.11.classes:
7945 ▷ $(ORBITER) -v 3 \
7946 ▷ ▷ -define G \
7947 ▷ ▷ -linear_group -PGL 3 11 \
7948 ▷ ▷ -end \
7949 ▷ ▷ -with G -do \
7950 ▷ ▷ -group_theoretic.activity \
7951 ▷ ▷ ▷ -classes \
7952 ▷ ▷ -end
7953 ▷ $(MAGMA.EXE) PGL_3.11.classes.magma
7954 ▷ $(ORBITER) -v 3 \
7955 ▷ ▷ -define G \
7956 ▷ ▷ -linear_group -PGL 3 11 \
7957 ▷ ▷ -end \
7958 ▷ ▷ -with G -do \
7959 ▷ ▷ -group_theoretic.activity \
7960 ▷ ▷ ▷ -classes \
7961 ▷ ▷ -end
7962 ▷ $(PDFLATEX) PGL_3.11.classes.out.tex
7963 ▷ $(OPEN) PGL_3.11.classes.out.pdf
7964
7965 PGL_7.2.classes:
7966 ▷ $(ORBITER) -v 3 \
7967 ▷ ▷ -define G \
7968 ▷ ▷ -linear_group -PGL 7 2 \
7969 ▷ ▷ -end \
7970 ▷ ▷ -with G -do \
7971 ▷ ▷ -group_theoretic.activity \
7972 ▷ ▷ ▷ -classes \
7973 ▷ ▷ -end
7974 ▷ $(MAGMA.EXE) PGL_7.2.classes.magma
7975 ▷ $(ORBITER) -v 3 \
7976 ▷ ▷ -define G \
7977 ▷ ▷ -linear_group -PGL 7 2 \
7978 ▷ ▷ -end \
7979 ▷ ▷ -with G -do \
7980 ▷ ▷ -group_theoretic.activity \
7981 ▷ ▷ ▷ -classes \
7982 ▷ ▷ -end
7983
7984
7985
7986 PGL_8.2.classes:
7987 ▷ $(ORBITER) -v 3 \
7988 ▷ ▷ -define G \
7989 ▷ ▷ -linear_group -PGL 8 2 \
7990 ▷ ▷ -end \
7991 ▷ ▷ -with G -do \
7992 ▷ ▷ -group_theoretic.activity \
7993 ▷ ▷ ▷ -classes \
7994 ▷ ▷ -end
7995 ▷ $(ORBITER) -v 3 \
7996 ▷ ▷ -define G \
7997 ▷ ▷ -linear_group -PGL 8 2 \
7998 ▷ ▷ -end \
7999 ▷ ▷ -with G -do \
8000 ▷ ▷ -group_theoretic.activity \
8001 ▷ ▷ ▷ -classes \

```

```

8002 ▷ ▷ -end
8003 ▷ $(PDFLATEX) PGL_8_2_classes_out.tex
8004 ▷ $(OPEN) PGL_8_2_classes_out.pdf
8005
8006
8007 ▷ #$(MAGMA.EXE) PGL_8_2_classes.magma
8008
8009
8010
8011 Hirschfeld_stab_classes:
8012 ▷ $(ORBITER) -v 9 \
8013 ▷ ▷ -orbiter_path $(ORBITER.EXE_PATH) \
8014 ▷ ▷ -define G -linear_group -PGGL 4 4 \
8015 ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld.Stab" \
8016 ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
8017 ▷ ▷ ▷ -end \
8018 ▷ ▷ -with G -do \
8019 ▷ ▷ -group_theoretic_activity \
8020 ▷ ▷ ▷ -classes \
8021 ▷ ▷ -end
8022 ▷ $(ORBITER) -v 9 \
8023 ▷ ▷ -orbiter_path $(ORBITER.EXE_PATH) \
8024 ▷ ▷ -define G -linear_group -PGGL 4 4 \
8025 ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld.Stab" \
8026 ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
8027 ▷ ▷ ▷ -end \
8028 ▷ ▷ -with G -do \
8029 ▷ ▷ -group_theoretic_activity \
8030 ▷ ▷ ▷ -classes \
8031 ▷ ▷ -end
8032 ▷ $(PDFLATEX) PGGL_4_4_Subgroup.Hirschfeld.Stab.51840_report_classes_of_elements.
    tex
8033 ▷ $(OPEN) PGGL_4_4_Subgroup.Hirschfeld.Stab.51840_report_classes_of_elements.pdf
8034
8035
8036
8037
8038 PGGL_2_4_cent_2A:
8039 ▷ $(ORBITER) -v 3 \
8040 ▷ ▷ -define G \
8041 ▷ ▷ -linear_group -PGGL 2 4 -end \
8042 ▷ ▷ -with G -do \
8043 ▷ ▷ -group_theoretic_activity \
8044 ▷ ▷ ▷ -centralizer_of_element "2A" "1,0, 0,1, 1" \
8045 ▷ ▷ -end
8046 ▷ $(ORBITER) -v 3 \
8047 ▷ ▷ -define G \
8048 ▷ ▷ -linear_group -PGGL 2 4 -end \
8049 ▷ ▷ -with G -do \
8050 ▷ ▷ -group_theoretic_activity \
8051 ▷ ▷ ▷ -centralizer_of_element "2A" "1,0, 0,1, 1" \
8052 ▷ ▷ -end
8053 ▷ $(PDFLATEX) PGGL_2_4_elt_2A_centralizer.tex
8054 ▷ $(OPEN) PGGL_2_4_elt_2A_centralizer.pdf
8055 ▷
8056
8057
8058 Normalizer_of_H5:
8059 ▷ $(ORBITER) -v 2 \

```



```

8060 ▷ ▷ -define G -permutation_group -symmetric_group 13 \
8061 ▷ ▷ ▷ -subgroup_by_generators H5 5 1 \
8062 ▷ ▷ ▷ ▷ $(GENERATORS_H5) -end \
8063 ▷ ▷ -with G -do \
8064 ▷ ▷ -group_theoretic_activity \
8065 ▷ ▷ ▷ -normalizer \
8066 ▷ ▷ -end
8067 ▷ $(PDFLATEX) Sym_13_Subgroup_H5_5_normalizer.tex
8068 ▷ $(OPEN) Sym_13_Subgroup_H5_5_normalizer.pdf
8069
8070
8071
8072
8073
8074 PGGL_3.4_classes:
8075 ▷ $(ORBITER) -v 3 \
8076 ▷ ▷ -define G \
8077 ▷ ▷ -linear_group -PGGL 3 4 \
8078 ▷ ▷ -end \
8079 ▷ ▷ -with G -do \
8080 ▷ ▷ -group_theoretic_activity \
8081 ▷ ▷ ▷ -classes \
8082 ▷ ▷ -end
8083 ▷ $(PDFLATEX) PGGL_3.4_classes_out.tex
8084 ▷ $(OPEN) PGGL_3.4_classes_out.pdf
8085
8086
8087
8088
8089
8090
8091
8092 classes_PGGL_4.4:
8093 ▷ $(ORBITER) -v 3 \
8094 ▷ ▷ -magma_path $(MAGMA_PATH) \
8095 ▷ ▷ -define G -linear_group -PGGL 4 4 -end \
8096 ▷ ▷ -with G -do \
8097 ▷ ▷ -group_theoretic_activity \
8098 ▷ ▷ ▷ -classes \
8099 ▷ ▷ -end
8100 ▷ $(PDFLATEX) PGGL_4.4_classes_out.tex
8101 ▷ $(OPEN) PGGL_4.4_classes_out.pdf
8102
8103 # group order 1974067200 = 2^13 * 3^4 * 5^2 * 7 * 17
8104
8105
8106
8107
8108
8109
8110
8111 classes_PGL_4.5:
8112 ▷ $(ORBITER) -v 6 \
8113 ▷ ▷ -define G \
8114 ▷ ▷ -linear_group -PGL 4 5 -end \
8115 ▷ ▷ -with G -do \
8116 ▷ ▷ -group_theoretic_activity \
8117 ▷ ▷ ▷ -classes \
8118 ▷ ▷ -end

```

```

8119 ▷ $(PDFLATEX) PGL_4_5.classes.out.tex
8120 ▷ $(OPEN) PGL_4_5.classes.out.pdf
8121 ▷
8122 # 163 classes
8123
8124
8125 # two classes of elements of order 3
8126 #Order of element = 3 Class size = 310000 Centralizer order = 93600 Normalizer or
    der = 187200
8127 #of order 3 and with 0 fixed points.
8128 #0,1,0,2,0,1,2,1,4,2,3,1,2,0,4,3,
8129
8130 #Class size = 10075000 Centralizer order = 2880 Normalizer order = 5760
8131 #of order 3 and with 6 fixed points.
8132 #0,0,0,1,2,3,0,1,0,3,4,4,0,1,2,1,
8133
8134
8135 PGL_4_5_3B.class.again:
8136 ▷ $(ORBITER) -v 6 \
8137 ▷ ▷ -define G -linear_group -PGL 4 5 -end \
8138 ▷ ▷ -with G -do \
8139 ▷ ▷ -group_theoretic_activity \
8140 ▷ ▷ ▷ -conjugacy_class_of \
8141 ▷ ▷ ▷ "3B" "0,0,0,1, 2,3,0,1, 0,3,4,4, 0,1,2,1" \
8142 ▷ ▷ -end
8143
8144 # very slow
8145
8146
8147
8148
8149 Normalizer_of_Z22_in_PGL_2_9:
8150 ▷ $(ORBITER) -v 2 \
8151 ▷ ▷ -define G -linear_group -PGL 2 9 \
8152 ▷ ▷ -subgroup_by_generators Z22 4 2 \
8153 ▷ ▷ ▷ "2,0,0,1, 0,1,1,0" -end \
8154 ▷ ▷ -with G -do \
8155 ▷ ▷ -group_theoretic_activity \
8156 ▷ ▷ ▷ -normalizer \
8157 ▷ ▷ -end
8158 ▷ $(PDFLATEX) PGL_2_9.Subgroup.Z22.4.normalizer.tex
8159 ▷ $(OPEN) PGL_2_9.Subgroup.Z22.4.normalizer.pdf
8160
8161
8162
8163
8164 PG0m_6_2.classes:
8165 ▷ $(ORBITER) -v 2 \
8166 ▷ ▷ -define F -finite_field -q 2 -end \
8167 ▷ ▷ -define G -linear_group -PG0m 6 F -end \
8168 ▷ ▷ -with G -do \
8169 ▷ ▷ -group_theoretic_activity \
8170 ▷ ▷ ▷ -classes \
8171 ▷ ▷ -end
8172 ▷ $(PDFLATEX) PG0m_6_2.classes.out.tex
8173 ▷ $(OPEN) PG0m_6_2.classes.out.pdf
8174
8175
8176 PG0m_6_2.Class_2A:

```

```

8177 ▷ $(ORBITER) -v 9 \
8178 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
8179 ▷ ▷ -define F -finite_field -q 2 -end \
8180 ▷ ▷ -define G -linear_group -PGOm 6 F -end \
8181 ▷ ▷ -with G -do \
8182 ▷ ▷ -group_theoretic_activity \
8183 ▷ ▷ ▷ -conjugacy_class_of "2A" $(PGOM_6_2_CLASS_2A_REP) \
8184 ▷ ▷ -end
8185
8186
8187
8188 #PGOm_6_2_class_of_2A.csv
8189
8190
8191 PGOm_6_2_Class_2A_print:
8192 ▷ $(ORBITER) -v 9 \
8193 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
8194 ▷ ▷ -define F -finite_field -q 2 -end \
8195 ▷ ▷ -define G -linear_group -PGOm 6 F -end \
8196 ▷ ▷ -define PGOm_6_2_Class2A -vector_ge \
8197 ▷ ▷ ▷ -action G \
8198 ▷ ▷ ▷ -read_csv $(FILE_NAME_PGOm_6_2_CLASS_2A) elements \
8199 ▷ ▷ -end \
8200 ▷ ▷ -with G -do \
8201 ▷ ▷ -group_theoretic_activity \
8202 ▷ ▷ ▷ -element_processing \
8203 ▷ ▷ ▷ ▷ -input PGOm_6_2_Class2A \
8204 ▷ ▷ ▷ ▷ -print \
8205 ▷ ▷ ▷ -end \
8206 ▷ ▷ -end
8207 ▷ $(PDFLATEX) PGOm_6_2_Class2A_elements.tex
8208 ▷ $(OPEN) PGOm_6_2_Class2A_elements.pdf
8209
8210 # 36 elements
8211
8212
8213 PGOm_6_2_Class_2A_centralizer:
8214 ▷ $(ORBITER) -v 9 \
8215 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
8216 ▷ ▷ -define F -finite_field -q 2 -end \
8217 ▷ ▷ -define G -linear_group -PGOm 6 F -end \
8218 ▷ ▷ -with G -do \
8219 ▷ ▷ -group_theoretic_activity \
8220 ▷ ▷ ▷ -centralizer_of_element \
8221 ▷ ▷ ▷ ▷ "2Arep" $(PGOM_6_2_CLASS_2A_REP_CLEAN) \
8222 ▷ ▷ -end
8223 ▷ $(PDFLATEX) PGOm_6_2_elt_2Arep_centralizer.tex
8224 ▷ $(OPEN) PGOm_6_2_elt_2Arep_centralizer.pdf
8225
8226
8227
8228 PGOm_6_2_Class_2A_centralizer_recover:
8229 ▷ $(ORBITER) -v 2 \
8230 ▷ ▷ -define Do -draw_options \
8231 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
8232 ▷ ▷ -end \
8233 ▷ ▷ -define F -finite_field -q 2 -end \
8234 ▷ ▷ -define Class2A_cent_gens -vector \
8235 ▷ ▷ ▷ -dense $(PGOM_6_2_CLASS_2A_CENTRALIZER_GENS) \

```

```

8236 ▷ ▷ -end \
8237 ▷ ▷ -define G -linear_group -PGL 6 2 \
8238 ▷ ▷ ▷ -subgroup_by_generators "2ACent" \
8239 ▷ ▷ ▷ 1440 9 Class2A.cent_gens \
8240 ▷ ▷ ▷ -end \
8241 ▷ ▷ -with G -do \
8242 ▷ ▷ -group_theoretic_activity \
8243 ▷ ▷ ▷ -report Do \
8244 ▷ ▷ -end
8245 ▷ $(PDFLATEX) PGL_6_2_Subgroup_2ACent_1440_report.tex
8246 ▷ $(OPEN) PGL_6_2_Subgroup_2ACent_1440_report.pdf
8247
8248
8249 #####
8250 # Section 5.7: The GAP Interface
8251
8252
8253 SECTION.GAP_INTERFACE:
8254
8255 test_5.7:
8256 ▷ make Hirschfeld_stab_export_gap
8257 ▷ make Cyclic_6_canonical_image
8258 ▷ make Edge_curve_q17_canonical_image
8259
8260
8261 Hirschfeld_stab_export_gap:
8262 ▷ $(ORBITER) -v 9 \
8263 ▷ ▷ -orbiter_path $(ORBITER.EXE_PATH) \
8264 ▷ ▷ -define G -linear_group -PGGL 4 4 \
8265 ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld_Stab" \
8266 ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
8267 ▷ ▷ ▷ -end \
8268 ▷ ▷ -with G -do \
8269 ▷ ▷ -group_theoretic_activity \
8270 ▷ ▷ ▷ -export_gap \
8271 ▷ ▷ -end
8272
8273
8274
8275 Cyclic_6_canonical_image:
8276 ▷ $(ORBITER) -v 3 \
8277 ▷ ▷ -define G -permutation_group -cyclic_group 6 -end \
8278 ▷ ▷ -with G -do -group_theoretic_activity \
8279 ▷ ▷ ▷ -canonical_image_GAP 3,5 \
8280 ▷ ▷ -end
8281
8282
8283
8284 Edge_curve_q17_canonical_image:
8285 ▷ $(ORBITER) -v 3 \
8286 ▷ ▷ -define G -linear_group -PGL 3 17 -end \
8287 ▷ ▷ -with G -do -group_theoretic_activity \
8288 ▷ ▷ ▷ -canonical_image_GAP \
8289 ▷ ▷ ▷ $(EDGE_CURVE_Q17_AS.POINTS) \
8290 ▷ ▷ -end
8291
8292 #PGL_3_17.canonical_image.gap
8293
8294

```

```

8295
8296 #####
8297 # Section 5.8: Group Theory, Advanced Topics
8298
8299
8300 SECTION.GROUP_THEORY_ADVANCED_TOPICS:
8301
8302 test_5.8:
8303   ▷ make U_3_3
8304   ▷ make PGL_2_3_group_table
8305   ▷ make Co3
8306   ▷ #make Ree_27 # needs too much memory
8307   ▷ make Higman_Sims_group
8308   ▷ make Higman_Sims_group_classes
8309   ▷ make Higman_Sims_group_subgroup_lattice
8310   ▷ make Higman_Sims_subgroup_176a
8311   ▷ make Higman_Sims_subgroup_176b
8312   ▷ make Butler12_PSP_6_2_conjugate
8313   ▷ make Hirschfeld_surface.get_subfield_subgroup
8314
8315
8316 U_3_3:
8317   ▷ $(ORBITER) -v 3 \
8318   ▷   ▷ -define Do -draw.options \
8319   ▷   ▷   ▷ -radius 200 -line_width 0.3 \
8320   ▷   ▷ -end \
8321   ▷   ▷ -define F -finite_field -q 9 \
8322   ▷   ▷   ▷ -override_polynomial "17" \
8323   ▷   ▷ -end \
8324   ▷   ▷ -define G -linear_group -PGL 3 F \
8325   ▷   ▷   ▷ -subgroup_by_generators \
8326   ▷   ▷   ▷ "U_3_3" "6048" 2 \
8327   ▷   ▷   ▷ "1,6,4, 5,0,6, 8,5,1, \
8328   ▷   ▷   ▷ 6,2,1, 7,8,4, 0,6,6" \
8329   ▷   ▷   ▷ -end \
8330   ▷   ▷ -with G -do \
8331   ▷   ▷ -group_theoretic_activity \
8332   ▷   ▷   ▷ -report Do \
8333   ▷   ▷ -end
8334   ▷ $(PDFLATEX) PGL_3_9_Subgroup.U_3_3.6048_report.tex
8335   ▷ #$(OPEN) PGL_3_9_Subgroup.U_3_3.6048_report.pdf
8336
8337
8338
8339
8340
8341 PGL_2_3_group_table:
8342   ▷ $(ORBITER) -v 3 \
8343   ▷   ▷ -define Do -draw.options \
8344   ▷   ▷   ▷ -radius 200 -line_width 0.3 \
8345   ▷   ▷ -end \
8346   ▷   ▷ -define G -linear_group -PGL 2 3 -end \
8347   ▷   ▷ -with G -do \
8348   ▷   ▷ -group_theoretic_activity \
8349   ▷   ▷   ▷ -report Do \
8350   ▷   ▷ -end
8351   ▷ $(PDFLATEX) PGL_2_3_report.tex
8352   ▷ $(OPEN) PGL_2_3_report.pdf
8353   ▷ #$(PDFLATEX) PGL_2_3_group_table_order_24.tex

```

```

8354 ▷ #$(OPEN) PGL_2.3_group_table_order_24.pdf
8355 ▷ #$(OPEN) PGL_2.3_report.pdf
8356
8357
8358
8359
8360
8361
8362 #Co3 from Conway et al., 1985 (ATLAS)
8363 #order = 495766656000
8364 #Co3 from the paper by Suleiman and Wilson 1997
8365
8366
8367 Co3:
8368 ▷ $(ORBITER) -v 2 \
8369 ▷ ▷ -define Do -draw_options \
8370 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
8371 ▷ ▷ -end \
8372 ▷ ▷ -define F -finite_field -q 2 -end \
8373 ▷ ▷ -define g1 -vector -field F -format 22 \
8374 ▷ ▷ ▷ -compact $(CONWAY_GEN1) -end \
8375 ▷ ▷ -define g2 -vector -field F -format 22 \
8376 ▷ ▷ ▷ -compact $(CONWAY_GEN2) -end \
8377 ▷ ▷ -define gens -vector -concatenate g1,g2 -end \
8378 ▷ ▷ -define G -linear_group -PGL 22 2 \
8379 ▷ ▷ ▷ -subgroup_by_generators "Co3" "495766656000" 2 gens \
8380 ▷ ▷ ▷ -end \
8381 ▷ ▷ -with G -do \
8382 ▷ ▷ -group_theoretic_activity \
8383 ▷ ▷ ▷ -report Do \
8384 ▷ ▷ -end
8385 ▷ $(PDFLATEX) PGL_22.2_Subgroup_Co3_495766656000_report.tex
8386 ▷ #$(OPEN) PGL_22.2_Subgroup_Co3_495766656000_report.pdf
8387
8388 # needs a lot of memory to run!
8389
8390 Ree_27:
8391 ▷ $(ORBITER) -v 2 \
8392 ▷ ▷ -define Do -draw_options \
8393 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
8394 ▷ ▷ -end \
8395 ▷ ▷ -define F -finite_field -q 27 \
8396 ▷ ▷ ▷ -override_polynomial "34" \
8397 ▷ ▷ -end \
8398 ▷ ▷ -define g1 -vector -field F -format 7 \
8399 ▷ ▷ ▷ -dense $(Ree_gen1) -end \
8400 ▷ ▷ -define g2 -vector -field F -format 7 \
8401 ▷ ▷ ▷ -dense $(Ree_gen2) -end \
8402 ▷ ▷ -define gens -vector -concatenate g1,g2 -end \
8403 ▷ ▷ -define G -linear_group -PGL 7 F \
8404 ▷ ▷ ▷ -subgroup_by_generators "Ree_27" "10073444472" 2 gens \
8405 ▷ ▷ ▷ -end \
8406 ▷ ▷ -with G -do \
8407 ▷ ▷ -group_theoretic_activity \
8408 ▷ ▷ ▷ -report Do \
8409 ▷ ▷ -end
8410
8411 # not enough memory on Mac laptop
8412 # needs a lot of memory to run!

```

```

8413
8414
8415
8416 Higman.Sims_group:
8417 ▷ $(ORBITER) -v 2 \
8418 ▷ ▷ -define Do -draw.options \
8419 ▷ ▷ ▷ -radius 200 -line.width 0.3 \
8420 ▷ ▷ -end \
8421 ▷ ▷ -define gens -vector -dense \
8422 ▷ ▷ ▷ $(HIGMAN_SIMS_GENS) \
8423 ▷ ▷ -end \
8424 ▷ ▷ -define G -permutation_group -symmetric_group 100 \
8425 ▷ ▷ ▷ -subgroup_by_generators HS \
8426 ▷ ▷ ▷ ▷ $(HIGMAN_SIMS_GO) \
8427 ▷ ▷ ▷ ▷ $(HIGMAN_SIMS_NB_GENS) gens \
8428 ▷ ▷ -end \
8429 ▷ ▷ -with G -do \
8430 ▷ ▷ -group_theoretic_activity \
8431 ▷ ▷ ▷ -report Do \
8432 ▷ ▷ -end
8433 ▷ $(PDFLATEX) Sym_100_Subgroup_HS_44352000_report.tex
8434 ▷ $(OPEN) Sym_100_Subgroup_HS_44352000_report.pdf
8435
8436
8437 Higman.Sims_group.classes:
8438 ▷ $(ORBITER) -v 2 \
8439 ▷ ▷ -define gens -vector -dense \
8440 ▷ ▷ ▷ $(HIGMAN_SIMS_GENS) \
8441 ▷ ▷ -end \
8442 ▷ ▷ -define G -permutation_group -symmetric_group 100 \
8443 ▷ ▷ ▷ -subgroup_by_generators HS \
8444 ▷ ▷ ▷ ▷ $(HIGMAN_SIMS_GO) \
8445 ▷ ▷ ▷ ▷ $(HIGMAN_SIMS_NB_GENS) gens \
8446 ▷ ▷ -end \
8447 ▷ ▷ -with G -do \
8448 ▷ ▷ -group_theoretic_activity \
8449 ▷ ▷ ▷ -classes \
8450 ▷ ▷ -end
8451 ▷ $(PDFLATEX) Sym_100_Subgroup_HS_44352000_classes_out.tex
8452 ▷ $(OPEN) Sym_100_Subgroup_HS_44352000_classes_out.pdf
8453
8454
8455 Higman.Sims_group.subgroup_lattice:
8456 ▷ $(ORBITER) -v 2 \
8457 ▷ ▷ -define gens -vector -dense \
8458 ▷ ▷ ▷ $(HIGMAN_SIMS_GENS) \
8459 ▷ ▷ -end \
8460 ▷ ▷ -define G -permutation_group -symmetric_group 100 \
8461 ▷ ▷ ▷ -subgroup_by_generators HS \
8462 ▷ ▷ ▷ ▷ $(HIGMAN_SIMS_GO) \
8463 ▷ ▷ ▷ ▷ $(HIGMAN_SIMS_NB_GENS) gens \
8464 ▷ ▷ -end \
8465 ▷ ▷ -with G -do \
8466 ▷ ▷ -group_theoretic_activity \
8467 ▷ ▷ ▷ -subgroup_lattice_magma \
8468 ▷ ▷ -end
8469
8470
8471 Higman.Sims_subgroup_176a:

```

```

8472 ▷ $(ORBITER) -v 2 \
8473 ▷ ▷ -define Do -draw.options \
8474 ▷ ▷ ▷ -radius 200 -line.width 0.3 \
8475 ▷ ▷ -end \
8476 ▷ ▷ -define gens -vector -dense \
8477 ▷ ▷ ▷ $(HIGMAN_SIMS_GENS) \
8478 ▷ ▷ -end \
8479 ▷ ▷ -define G -permutation_group -symmetric_group 100 \
8480 ▷ ▷ ▷ -subgroup_by_generators HS \
8481 ▷ ▷ ▷ $(HIGMAN_SIMS_GO) \
8482 ▷ ▷ ▷ $(HIGMAN_SIMS_NB_GENS) gens \
8483 ▷ ▷ -end \
8484 ▷ ▷ -with G -do \
8485 ▷ ▷ -group_theoretic_activity \
8486 ▷ ▷ ▷ -subgroup_lattice_magma \
8487 ▷ ▷ -end \
8488 ▷ ▷ -define G176a -modified_group -from G \
8489 ▷ ▷ ▷ -subgroup_by_lattice 585 \
8490 ▷ ▷ -end \
8491 ▷ ▷ -with G176a -do \
8492 ▷ ▷ -group_theoretic_activity \
8493 ▷ ▷ ▷ -report Do \
8494 ▷ ▷ -end
8495 ▷ $(PDFLATEX) Sym_100_Subgroup_HS_44352000_subgroup_by_lattice_585_report.tex
8496 ▷ $(OPEN) Sym_100_Subgroup_HS_44352000_subgroup_by_lattice_585_report.pdf
8497
8498
8499 # group order 252000
8500
8501
8502 HigmanSims_subgroup_176b:
8503 ▷ $(ORBITER) -v 2 \
8504 ▷ ▷ -define Do -draw.options \
8505 ▷ ▷ ▷ -radius 200 -line.width 0.3 \
8506 ▷ ▷ -end \
8507 ▷ ▷ -define gens -vector -dense \
8508 ▷ ▷ ▷ $(HIGMAN_SIMS_GENS) \
8509 ▷ ▷ -end \
8510 ▷ ▷ -define G -permutation_group -symmetric_group 100 \
8511 ▷ ▷ ▷ -subgroup_by_generators HS \
8512 ▷ ▷ ▷ $(HIGMAN_SIMS_GO) \
8513 ▷ ▷ ▷ $(HIGMAN_SIMS_NB_GENS) gens \
8514 ▷ ▷ -end \
8515 ▷ ▷ -with G -do \
8516 ▷ ▷ -group_theoretic_activity \
8517 ▷ ▷ ▷ -subgroup_lattice_magma \
8518 ▷ ▷ -end \
8519 ▷ ▷ -define G176b -modified_group -from G \
8520 ▷ ▷ ▷ -subgroup_by_lattice 586 \
8521 ▷ ▷ -end \
8522 ▷ ▷ -with G176b -do \
8523 ▷ ▷ -group_theoretic_activity \
8524 ▷ ▷ ▷ -report Do \
8525 ▷ ▷ -end
8526 ▷ $(PDFLATEX) Sym_100_Subgroup_HS_44352000_subgroup_by_lattice_586_report.tex
8527 ▷ $(OPEN) Sym_100_Subgroup_HS_44352000_subgroup_by_lattice_586_report.pdf
8528
8529
8530

```



```

8531 Butler12.PSP_6.2.conjugate:
8532 ▷ $(ORBITER) -v 10 \
8533 ▷ ▷ -define F -finite_field -q 2 -end \
8534 ▷ ▷ -define G -linear_group -PGL 6 F \
8535 ▷ ▷ ▷ -symplectic_group \
8536 ▷ ▷ -end \
8537 ▷ ▷ -assign gens -from \
8538 ▷ ▷ -with G -do \
8539 ▷ ▷ -group_theoretic_activity \
8540 ▷ ▷ ▷ -generators \
8541 ▷ ▷ -end \
8542 ▷ ▷ -define m1 -vector \
8543 ▷ ▷ ▷ -permutation_matrix "0,2,4,1,3,5" \
8544 ▷ ▷ -end \
8545 ▷ ▷ -define m1t -vector \
8546 ▷ ▷ ▷ -permutation_matrix_inverse "0,2,4,1,3,5" \
8547 ▷ ▷ -end \
8548 ▷ ▷ -define M1 -vector_ge \
8549 ▷ ▷ ▷ -action G \
8550 ▷ ▷ ▷ -vector_data m1 \
8551 ▷ ▷ -end \
8552 ▷ ▷ -define M1t -vector_ge \
8553 ▷ ▷ ▷ -action G \
8554 ▷ ▷ ▷ -vector_data m1t \
8555 ▷ ▷ -end \
8556 ▷ ▷ -assign gens2 -from \
8557 ▷ ▷ -with gens -and M1t -do \
8558 ▷ ▷ -vector_ge_activity \
8559 ▷ ▷ ▷ -conjugate \
8560 ▷ ▷ -end \
8561 ▷ ▷ -with gens2 -do \
8562 ▷ ▷ -vector_ge_activity \
8563 ▷ ▷ ▷ -report \
8564 ▷ ▷ -end
8565 ▷ $(PDFLATEX) PGL_6.2.elements.tex
8566 ▷ $(OPEN) PGL_6.2.elements.pdf
8567
8568 # generates the symplectic group associated with the bilinear form
8569 # whose gram matrix is
8570 # 0 I
8571 # I 0
8572 # (only in characteristic two)
8573
8574
8575
8576 Hirschfeld_surface_get_subfield_subgroup:
8577 ▷ $(ORBITER) -v 6 \
8578 ▷ ▷ -define Do -draw_options \
8579 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
8580 ▷ ▷ -end \
8581 ▷ ▷ -define F -finite_field -q 4 -end \
8582 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
8583 ▷ ▷ -define R -polynomial_ring \
8584 ▷ ▷ ▷ -field F \
8585 ▷ ▷ ▷ -number_of_variables 4 \
8586 ▷ ▷ ▷ -homogeneous_of_degree 3 \
8587 ▷ ▷ ▷ -monomial_ordering_partition \
8588 ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
8589 ▷ ▷ -end \

```

```

8590 ▷ ▷ -define V -variety \
8591 ▷ ▷ ▷ -projective_space P \
8592 ▷ ▷ ▷ -ring R \
8593 ▷ ▷ ▷ -equation_in_algebraic_form \
8594 ▷ ▷ ▷ ▷ $(HIRSCHFELD_SURFACE_EQUATION_ALGEBRAIC_FORM) \
8595 ▷ ▷ ▷ -label.txt Hirschfeld.4 \
8596 ▷ ▷ ▷ -label.tex "{\\rm Hirschfeld\\_4}" \
8597 ▷ ▷ -end \
8598 ▷ ▷ -with V -do -variety_activity \
8599 ▷ ▷ ▷ -compute_group \
8600 ▷ ▷ ▷ -nauty_control -save_orbit_of_equations eqn_ -end \
8601 ▷ ▷ -end \
8602 ▷ ▷ -define G -modified_group \
8603 ▷ ▷ ▷ -stabilizer_of_variety V \
8604 ▷ ▷ ▷ -nauty_control -end \
8605 ▷ ▷ -end \
8606 ▷ ▷ -with G -do \
8607 ▷ ▷ -group_theoretic_activity \
8608 ▷ ▷ ▷ -report Do \
8609 ▷ ▷ -end \
8610 ▷ ▷ -assign_elements -from \
8611 ▷ ▷ -with G -do \
8612 ▷ ▷ -group_theoretic_activity \
8613 ▷ ▷ ▷ -elements \
8614 ▷ ▷ -end \
8615 ▷ ▷ -assign_elements2 -from \
8616 ▷ ▷ -with elements -do \
8617 ▷ ▷ -vector_ge_activity \
8618 ▷ ▷ ▷ -filter_subfield 2 \
8619 ▷ ▷ -end \
8620 ▷ ▷ -with elements2 -do \
8621 ▷ ▷ -vector_ge_activity \
8622 ▷ ▷ ▷ -report \
8623 ▷ ▷ -end \
8624 ▷ ▷ -with elements2 -do \
8625 ▷ ▷ -vector_ge_activity \
8626 ▷ ▷ ▷ -report_elements_coded \
8627 ▷ ▷ -end \
8628 ▷ ▷ -print_symbols
8629 ▷ $(PDFLATEX) PGGL_4.4_stab_of_Hirschfeld.4_elements2_elements.tex
8630 ▷ $(OPEN) PGGL_4.4_stab_of_Hirschfeld.4_elements2_elements.pdf
8631
8632
8633
8634
8635 #####
8636 # Section 5.9: Group Theory, activities for vectors of group elements
8637
8638
8639 SECTION_GROUP_THEORY_ACTIVITIES_FOR_VECTORS:
8640
8641 test_5.9:
8642 ▷ make PGGL_2.4_classes_reps
8643 ▷ make PGL_4.4_classes_reps
8644 ▷ make PGGL_4.4_classes_reps
8645 ▷ make Sym_6_generators
8646 ▷ make Sym_6_elements
8647 ▷ make GL_2.3_RNF_elements
8648 ▷ make PGL_4.2_orthogonal_reflections_report_vector_ge

```

```

8649
8650
8651 PGGL_2.4_classes.reps:
8652 ▷ $(ORBITER) -v 3 \
8653 ▷ ▷ -define G \
8654 ▷ ▷ -linear_group -PGGL 2 4 \
8655 ▷ ▷ -end \
8656 ▷ ▷ -assign reps -from \
8657 ▷ ▷ -with G -do \
8658 ▷ ▷ -group_theoretic_activity \
8659 ▷ ▷ ▷ -classes \
8660 ▷ ▷ -end \
8661 ▷ ▷ -with reps -do \
8662 ▷ ▷ -vector_ge_activity \
8663 ▷ ▷ ▷ -report \
8664 ▷ ▷ -end \
8665 ▷ ▷ -print_symbols
8666 ▷ $(PDFLATEX) PGGL_2.4_reps.elements.tex
8667 ▷ $(OPEN) PGGL_2.4_reps.elements.pdf
8668
8669
8670 PGL_4.4_classes.reps:
8671 ▷ $(ORBITER) -v 3 \
8672 ▷ ▷ -define G \
8673 ▷ ▷ -linear_group -PGL 4 4 \
8674 ▷ ▷ -end \
8675 ▷ ▷ -assign reps -from \
8676 ▷ ▷ -with G -do \
8677 ▷ ▷ -group_theoretic_activity \
8678 ▷ ▷ ▷ -classes \
8679 ▷ ▷ -end \
8680 ▷ ▷ -with reps -do \
8681 ▷ ▷ -vector_ge_activity \
8682 ▷ ▷ ▷ -report \
8683 ▷ ▷ -end \
8684 ▷ ▷ -assign RNF -and BaseChange -from \
8685 ▷ ▷ -with reps -do \
8686 ▷ ▷ -vector_ge_activity \
8687 ▷ ▷ ▷ -rational_canonical_form \
8688 ▷ ▷ -end \
8689 ▷ ▷ -with RNF -do \
8690 ▷ ▷ -vector_ge_activity \
8691 ▷ ▷ ▷ -report_with_options "dense=on" \
8692 ▷ ▷ -end \
8693 ▷ ▷ -print_symbols
8694 ▷ $(PDFLATEX) PGL_4.4.RNF.elements.tex
8695 ▷ $(OPEN) PGL_4.4.RNF.elements.pdf
8696 ▷ #$(PDFLATEX) PGL_4.4_reps.elements.tex
8697 ▷ #$(OPEN) PGL_4.4_reps.elements.pdf
8698
8699
8700
8701 PGGL_4.4_classes.reps:
8702 ▷ $(ORBITER) -v 3 \
8703 ▷ ▷ -define G \
8704 ▷ ▷ -linear_group -PGGL 4 4 \
8705 ▷ ▷ -end \
8706 ▷ ▷ -assign reps -from \
8707 ▷ ▷ -with G -do \

```

```

8708 ▷ ▷ -group_theoretic.activity \
8709 ▷ ▷ ▷ -classes \
8710 ▷ ▷ -end \
8711 ▷ ▷ -with reps -do \
8712 ▷ ▷ -vector_ge.activity \
8713 ▷ ▷ ▷ -report \
8714 ▷ ▷ -end \
8715 ▷ ▷ -print_symbols
8716 ▷ $(PDFLATEX) PGGL_4.4_reps.elements.tex
8717 ▷ $(OPEN) PGGL_4.4_reps.elements.pdf
8718
8719
8720 Sym_6_generators:
8721 ▷ $(ORBITER) -v 3 \
8722 ▷ ▷ -define G -permutation_group -symmetric_group 6 -end \
8723 ▷ ▷ -assign gens -from \
8724 ▷ ▷ -with G -do \
8725 ▷ ▷ -group_theoretic.activity \
8726 ▷ ▷ ▷ -generators \
8727 ▷ ▷ -end \
8728 ▷ ▷ -with gens -do \
8729 ▷ ▷ -vector_ge.activity \
8730 ▷ ▷ ▷ -report \
8731 ▷ ▷ -end \
8732 ▷ ▷ -print_symbols
8733 ▷ $(PDFLATEX) Sym_6_gens.elements.tex
8734 ▷ $(OPEN) Sym_6_gens.elements.pdf
8735
8736
8737 Sym_6_elements:
8738 ▷ $(ORBITER) -v 3 \
8739 ▷ ▷ -define G -permutation_group -symmetric_group 6 -end \
8740 ▷ ▷ -assign elements -from \
8741 ▷ ▷ -with G -do \
8742 ▷ ▷ -group_theoretic.activity \
8743 ▷ ▷ ▷ -elements \
8744 ▷ ▷ -end \
8745 ▷ ▷ -with elements -do \
8746 ▷ ▷ -vector_ge.activity \
8747 ▷ ▷ ▷ -report \
8748 ▷ ▷ -end \
8749 ▷ ▷ -print_symbols
8750 ▷ $(PDFLATEX) Sym_6_elements.elements.tex
8751 ▷ $(OPEN) Sym_6_elements.elements.pdf
8752
8753
8754
8755 GL_2_3_RNF_elements:
8756 ▷ $(ORBITER) -v 2 \
8757 ▷ ▷ -define Do -draw_options \
8758 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
8759 ▷ ▷ -end \
8760 ▷ ▷ -define F -finite_field -q 3 -end \
8761 ▷ ▷ -define G -linear_group -GL 2 F -end \
8762 ▷ ▷ -assign elements -from \
8763 ▷ ▷ -with G -do \
8764 ▷ ▷ -group_theoretic.activity \
8765 ▷ ▷ ▷ -elements \
8766 ▷ ▷ -end \

```

```

8767 ▷ ▷ -assign RNF -and BaseChange -from \
8768 ▷ ▷ -with elements -do \
8769 ▷ ▷ -vector_ge_activity \
8770 ▷ ▷ ▷ -rational_canonical_form \
8771 ▷ ▷ -end \
8772 ▷ ▷ -with RNF -do \
8773 ▷ ▷ -vector_ge_activity \
8774 ▷ ▷ ▷ -report_with_options "dense=on" \
8775 ▷ ▷ -end \
8776 ▷ ▷ -with BaseChange -do \
8777 ▷ ▷ -vector_ge_activity \
8778 ▷ ▷ ▷ -report_with_options "dense=on" \
8779 ▷ ▷ -end \
8780 ▷ ▷ -print_symbols
8781 ▷ $(PDFLATEX) GL_2_3_RNF_elements.tex
8782 ▷ $(OPEN) GL_2_3_RNF_elements.pdf
8783
8784
8785
8786
8787 PGL_4.2_orthogonal_reflections_report_vector_ge:
8788 ▷ $(ORBITER) -v 2 \
8789 ▷ ▷ -define Pts -vector \
8790 ▷ ▷ ▷ -dense $(ORTHOGONAL_OP_6.2_POINTS_OFF) \
8791 ▷ ▷ -end \
8792 ▷ ▷ -define F -finite_field -q 2 -end \
8793 ▷ ▷ -define O -orthogonal_space 1 6 F -end \
8794 ▷ ▷ -define G4 -linear_group -PGL 4 2 -end \
8795 ▷ ▷ -with O -do -orthogonal_space_activity \
8796 ▷ ▷ ▷ -create_orthogonal_reflection_6.and.4 Pts G4 \
8797 ▷ ▷ -end
8798 ▷ $(ORBITER) -v 2 \
8799 ▷ ▷ -define F -finite_field -q 2 -end \
8800 ▷ ▷ -define G -linear_group -PGL 4 F -end \
8801 ▷ ▷ -define reflections -vector_ge \
8802 ▷ ▷ ▷ -action G \
8803 ▷ ▷ ▷ -read_csv 0_1_6.2_orthogonal_reflections_4x4.csv Element \
8804 ▷ ▷ -end \
8805 ▷ ▷ -with reflections -do \
8806 ▷ ▷ -vector_ge_activity \
8807 ▷ ▷ ▷ -report \
8808 ▷ ▷ -end \
8809 ▷ ▷ -with reflections -do \
8810 ▷ ▷ -vector_ge_activity \
8811 ▷ ▷ ▷ -export_GAP \
8812 ▷ ▷ -end \
8813 ▷ ▷ -assign RNF -and BaseChange -from \
8814 ▷ ▷ -with reflections -do \
8815 ▷ ▷ -vector_ge_activity \
8816 ▷ ▷ ▷ -rational_canonical_form \
8817 ▷ ▷ -end \
8818 ▷ ▷ -with RNF -do \
8819 ▷ ▷ -vector_ge_activity \
8820 ▷ ▷ ▷ -report_with_options "dense=on" \
8821 ▷ ▷ -end \
8822 ▷ ▷ -print_symbols
8823 ▷ $(PDFLATEX) PGL_4.2_reflections_elements.tex
8824 ▷ $(OPEN) PGL_4.2_reflections_elements.pdf
8825 ▷ $(PDFLATEX) PGL_4.2_RNF_elements.tex

```

```

8826 ▷ $(OPEN) PGL_4_2.RNF_elements.pdf
8827
8828
8829
8830
8831
8832 #####
8833 # Chapter 6 - Ring Theory
8834 #####
8835
8836
8837 test_6:
8838 ▷ make test_6.1
8839 ▷ make test_6.2
8840
8841
8842
8843
8844 #####
8845 # Section 6.1: Polynomials over Finite Fields
8846
8847 SECTION_POLYNOMIALS:
8848
8849
8850 test_6.1:
8851 ▷ make sift_polynomials_deg3.q2
8852 ▷ make sift_polynomials_deg4.q2
8853 ▷ make poly_division
8854 ▷ make poly_division2
8855 ▷ make poly_gcd
8856 ▷ make poly_mult_mod1
8857 ▷ make poly_mult_mod2
8858 ▷ make poly_mult_mod_F4
8859 ▷ make mult_polynomials_2.5.7
8860 ▷ make polynomial_division_ranked_2.27.13
8861 ▷ make mult_polynomials_2.8.15
8862 ▷ make polynomial_division_ranked_2.120.25
8863 ▷ make mult_polynomials_2.7.7
8864 ▷ make mult_polynomials_2.4.6
8865 ▷ make polynomial_division_ranked_2.24.13
8866 ▷ make mult_polynomials_1024.999.997
8867 ▷ make polynomial_division_ranked_2.349147.1033
8868 ▷ make mult_polynomials_17.12
8869 ▷ make polynomial_division_ranked_2.204.37
8870 ▷ make Berlekamp_matrix_crc32
8871 ▷ make power_mod_inverse
8872 ▷ make mult_mod_to_get_one
8873 ▷ make Berlekamp_matrix_2.3
8874 ▷ make Berlekamp_matrix_2.4
8875 ▷ make Berlekamp_matrix_4.3a
8876 ▷ make Berlekamp_matrix_4.3b
8877 ▷ make find_roots.a
8878 ▷ make find_roots.b
8879 ▷ make find_roots.c
8880 ▷ make find_roots.d
8881 ▷ make find_roots.e
8882 ▷ make roots_over_F2
8883 ▷ make roots_over_F8
8884 ▷ make irred_3.2

```

```

8885 ▷ make irred_4.2
8886 ▷ make irred_5.2
8887 ▷ make irred_6.2
8888 ▷ make irred_7.2
8889 ▷ make irred_8.2
8890 ▷ make irred_9.2
8891 ▷ make irred_10.2
8892 ▷ make irred_2.4
8893 ▷ make irred_3.4
8894 ▷ make F2_get_primitive_poly_of_deg_10
8895 ▷ make F2_get_primitive_poly_range_2.10
8896
8897
8898 # check which polynomials are irreducible and which are primitive:
8899
8900 sift_polynomials_deg3_q2:
8901 ▷ $(ORBITER) -v 2 \
8902 ▷ ▷ -define F -finite_field -q 2 -end \
8903 ▷ ▷ -with F -do \
8904 ▷ ▷ -finite_field_activity \
8905 ▷ ▷ ▷ -sift_polynomials 8 16 \
8906 ▷ ▷ -end
8907
8908
8909
8910 sift_polynomials_deg4_q2:
8911 ▷ $(ORBITER) -v 2 \
8912 ▷ ▷ -define F -finite_field -q 2 -end \
8913 ▷ ▷ -with F -do \
8914 ▷ ▷ -finite_field_activity \
8915 ▷ ▷ ▷ -sift_polynomials 16 32 \
8916 ▷ ▷ -end
8917
8918
8919
8920
8921 poly_division:
8922 ▷ $(ORBITER) -v 2 \
8923 ▷ ▷ -define F -finite_field -q 2 -end \
8924 ▷ ▷ -with F -do \
8925 ▷ ▷ -finite_field_activity \
8926 ▷ ▷ ▷ -polynomial_division \
8927 ▷ ▷ ▷ "1,0,0,0,0,0,0,0,0,1" "1,0,1,1" \
8928 ▷ ▷ -end
8929
8930 poly_division2:
8931 ▷ $(ORBITER) -v 2 \
8932 ▷ ▷ -define F -finite_field -q 2 -end \
8933 ▷ ▷ -define A -vector -field F -sparse 11 "1,0,1,10" -end \
8934 ▷ ▷ -define B -vector -field F -dense "1,0,1,1" -end \
8935 ▷ ▷ -with F -do \
8936 ▷ ▷ -finite_field_activity \
8937 ▷ ▷ ▷ -polynomial_division A B \
8938 ▷ ▷ -end
8939
8940
8941 poly_gcd:
8942 ▷ $(ORBITER) -v 2 \
8943 ▷ ▷ -define F -finite_field -q 2 -end \

```

```

8944 ▷ ▷ -with F -do \
8945 ▷ ▷ -finite_field_activity \
8946 ▷ ▷ ▷ -extended_gcd_for_polynomials \
8947 ▷ ▷ ▷ ▷ "1,0,0,0,0,0,0,0,0,1" "1,0,1,1" \
8948 ▷ ▷ -end
8949
8950 poly_mult_mod1:
8951 ▷ $(ORBITER) -v 2 \
8952 ▷ ▷ -define F -finite_field -q 7 -end \
8953 ▷ ▷ -with F -do \
8954 ▷ ▷ -finite_field_activity \
8955 ▷ ▷ ▷ -polynomial_mult_mod "1,2,3" "3,4,5" "6,0,0,1" \
8956 ▷ ▷ -end
8957
8958 poly_mult_mod2:
8959 ▷ $(ORBITER) -v 2 \
8960 ▷ ▷ -define F -finite_field -q 7 -end \
8961 ▷ ▷ -with F -do \
8962 ▷ ▷ -finite_field_activity \
8963 ▷ ▷ ▷ -polynomial_mult_mod "3,1,2" "5,3,4" "6,0,0,1" \
8964 ▷ ▷ -end
8965
8966
8967
8968 poly_mult_mod.F4:
8969 ▷ $(ORBITER) -v 2 \
8970 ▷ ▷ -define F -finite_field -q 2 -end \
8971 ▷ ▷ -with F -do \
8972 ▷ ▷ -finite_field_activity \
8973 ▷ ▷ ▷ -polynomial_mult_mod "1,1" "1,1" "1,1,1" \
8974 ▷ ▷ -end
8975 ▷ $(ORBITER) -v 2 \
8976 ▷ ▷ -define F -finite_field -q 2 -end \
8977 ▷ ▷ -with F -do \
8978 ▷ ▷ -finite_field_activity \
8979 ▷ ▷ ▷ -polynomial_mult_mod "0,1" "1,1" "1,1,1" \
8980 ▷ ▷ -end
8981 ▷ $(ORBITER) -v 2 \
8982 ▷ ▷ -define F -finite_field -q 2 -end \
8983 ▷ ▷ -with F -do \
8984 ▷ ▷ -finite_field_activity \
8985 ▷ ▷ ▷ -polynomial_mult_mod "0,1" "0,1" "1,1,1" \
8986 ▷ ▷ -end
8987
8988
8989
8990
8991
8992 mult_polynomials_2.5.7:
8993 ▷ $(ORBITER) -v 2 \
8994 ▷ ▷ -define F -finite_field -q 2 -end \
8995 ▷ ▷ -with F -do \
8996 ▷ ▷ -finite_field_activity \
8997 ▷ ▷ ▷ -mult_polynomials 5 7 \
8998 ▷ ▷ -end
8999 ▷ $(PDFLATEX) polynomial_mult_5.7.tex
9000 ▷ $(OPEN) polynomial_mult_5.7.pdf
9001
9002

```



```

9003 polynomial_division_ranked.2.27.13:
9004 ▷ $(ORBITER) -v 2 \
9005 ▷ ▷ -define F -finite_field -q 2 -end \
9006 ▷ ▷ -with F -do \
9007 ▷ ▷ -finite_field.activity \
9008 ▷ ▷ ▷ -polynomial_division_ranked 27 13 \
9009 ▷ ▷ -end
9010 ▷ $(PDFLATEX) polynomial_division.27.13.tex
9011 ▷ $(OPEN) polynomial_division.27.13.pdf
9012
9013
9014
9015
9016 mult_polynomials.2.8.15:
9017 ▷ $(ORBITER) -v 2 \
9018 ▷ ▷ -define F -finite_field -q 2 -end \
9019 ▷ ▷ -with F -do \
9020 ▷ ▷ -finite_field.activity \
9021 ▷ ▷ ▷ -mult_polynomials 8 15 \
9022 ▷ ▷ -end
9023 ▷ $(PDFLATEX) polynomial_mult.8.15.tex
9024 ▷ $(OPEN) polynomial_mult.8.15.pdf
9025
9026 polynomial_division_ranked.2.120.25:
9027 ▷ $(ORBITER) -v 2 \
9028 ▷ ▷ -define F -finite_field -q 2 -end \
9029 ▷ ▷ -with F -do \
9030 ▷ ▷ -finite_field.activity \
9031 ▷ ▷ ▷ -polynomial_division_ranked 120 25 \
9032 ▷ ▷ -end
9033 ▷ $(PDFLATEX) polynomial_division.120.25.tex
9034 ▷ $(OPEN) polynomial_division.120.25.pdf
9035
9036 # the answer is 5
9037
9038
9039 mult_polynomials.2.7.7:
9040 ▷ $(ORBITER) -v 2 \
9041 ▷ ▷ -define F -finite_field -q 2 -end \
9042 ▷ ▷ -with F -do \
9043 ▷ ▷ -finite_field.activity \
9044 ▷ ▷ ▷ -mult_polynomials 7 7 \
9045 ▷ ▷ -end
9046 ▷ $(PDFLATEX) polynomial_mult.7.7.tex
9047 ▷ $(OPEN) polynomial_mult.7.7.pdf
9048
9049
9050
9051 mult_polynomials.2.4.6:
9052 ▷ $(ORBITER) -v 2 \
9053 ▷ ▷ -define F -finite_field -q 2 -end \
9054 ▷ ▷ -with F -do \
9055 ▷ ▷ -finite_field.activity \
9056 ▷ ▷ ▷ -mult_polynomials 4 6 \
9057 ▷ ▷ -end
9058 ▷ $(PDFLATEX) polynomial_mult.4.6.tex
9059 ▷ $(OPEN) polynomial_mult.4.6.pdf
9060
9061 polynomial_division_ranked.2.24.13:

```

```

9062 ▷ $(ORBITER) -v 2 \
9063 ▷ ▷ -define F -finite_field -q 2 -end \
9064 ▷ ▷ -with F -do \
9065 ▷ ▷ -finite_field_activity \
9066 ▷ ▷ ▷ -polynomial_division_ranked 24 13 \
9067 ▷ ▷ -end
9068 ▷ $(PDFLATEX) polynomial_division_24_13.tex
9069 ▷ $(OPEN) polynomial_division_24_13.pdf
9070
9071
9072
9073 mult_polynomials_1024_999_997:
9074 ▷ $(ORBITER) -v 2 \
9075 ▷ ▷ -define F -finite_field -q 2 -end \
9076 ▷ ▷ -with F -do \
9077 ▷ ▷ -finite_field_activity \
9078 ▷ ▷ ▷ -mult_polynomials 999 997 \
9079 ▷ ▷ -end
9080 ▷ $(PDFLATEX) polynomial_mult_999_997.tex
9081 ▷ $(OPEN) polynomial_mult_999_997.pdf
9082
9083
9084 polynomial_division_ranked_2_349147_1033:
9085 ▷ $(ORBITER) -v 2 \
9086 ▷ ▷ -define F -finite_field -q 2 -end \
9087 ▷ ▷ -with F -do \
9088 ▷ ▷ -finite_field_activity \
9089 ▷ ▷ ▷ -polynomial_division_ranked 349147 1033 \
9090 ▷ ▷ -end
9091 ▷ $(PDFLATEX) polynomial_division_349147_1033.tex
9092 ▷ $(OPEN) polynomial_division_349147_1033.pdf
9093
9094
9095
9096
9097
9098 mult_polynomials_17_12:
9099 ▷ $(ORBITER) -v 2 \
9100 ▷ ▷ -define F -finite_field -q 2 -end \
9101 ▷ ▷ -with F -do \
9102 ▷ ▷ -finite_field_activity \
9103 ▷ ▷ ▷ -mult_polynomials 17 12 \
9104 ▷ ▷ -end
9105 ▷ $(PDFLATEX) polynomial_mult_17_12.tex
9106 ▷ $(OPEN) polynomial_mult_17_12.pdf
9107 ▷
9108 # gives 204
9109
9110 polynomial_division_ranked_2_204_37:
9111 ▷ $(ORBITER) -v 2 \
9112 ▷ ▷ -define F -finite_field -q 2 -end \
9113 ▷ ▷ -with F -do \
9114 ▷ ▷ -finite_field_activity \
9115 ▷ ▷ ▷ -polynomial_division_ranked 204 37 \
9116 ▷ ▷ -end
9117 ▷ $(PDFLATEX) polynomial_division_204_37.tex
9118 ▷ $(OPEN) polynomial_division_204_37.pdf
9119 ▷
9120 # answer is 18

```

```

9121 ▷
9122 ▷
9123
9124
9125
9126
9127 #ToDo:
9128
9129 #test_crc32:
9130 #▷ $(ORBITER) -v 3 \
9131 #▷ ▷ -crc32 "123456789"
9132
9133
9134 Berlekamp_matrix_crc32:
9135 ▷ $(ORBITER) -v 2 \
9136 ▷ ▷ -define F -finite_field -q 2 -end \
9137 ▷ ▷ -define v -vector -field F -sparse 33 $(CRC32_SPARSE) -end \
9138 ▷ ▷ -with F -do \
9139 ▷ ▷ -finite_field_activity \
9140 ▷ ▷ ▷ -Berlekamp_matrix v \
9141 ▷ ▷ -end
9142
9143
9144 # N = 2^32-1 = 3 * 5 * 17 * 257 * 65537
9145 # N / 3 = 1431655765
9146 # N / 5 = 858993459
9147 # N / 17 = 252645135
9148 # N / 257 = 16711935
9149 # N / 65537 = 65535
9150
9151
9152
9153 power_mod_inverse:
9154 ▷ $(ORBITER) -v 2 \
9155 ▷ ▷ -define F -finite_field -q 2 -end \
9156 ▷ ▷ -define M -vector -field F -sparse 33 $(CRC32_SPARSE) -end \
9157 ▷ ▷ -define A -vector -field F -sparse 2 "1,1" -end \
9158 ▷ ▷ -with F -do \
9159 ▷ ▷ -finite_field_activity \
9160 ▷ ▷ ▷ -polynomial_power_mod A $(TWO_TO_THE_32_MINUS_2) M \
9161 ▷ ▷ -end
9162
9163
9164 #A(X)=X^{31} + X^{25} + X^{22} + X^{21} + X^{15}
9165 #+ X^{11} + X^{10} + X^9 + X^7 + X^6 + X^4 + X^3 + X + 1
9166
9167 mult_mod_to_get_one:
9168 ▷ $(ORBITER) -v 2 \
9169 ▷ ▷ -define F -finite_field -q 2 -end \
9170 ▷ ▷ -define M -vector -field F -sparse 33 $(CRC32_SPARSE) -end \
9171 ▷ ▷ -define A -vector -field F -sparse 2 "1,1" -end \
9172 ▷ ▷ -define B -vector -field F -sparse 33 $(INVERSE_SPARSE) -end \
9173 ▷ ▷ -with F -do \
9174 ▷ ▷ -finite_field_activity \
9175 ▷ ▷ ▷ -polynomial_mult_mod A B M \
9176 ▷ ▷ -end
9177
9178 #C(X)=1
9179

```

```

9180
9181
9182
9183
9184
9185
9186 Berlekamp_matrix_2.3:
9187 ▷ $(ORBITER) -v 2 \
9188 ▷ ▷ -define F -finite_field -q 2 -end \
9189 ▷ ▷ -define v -vector -field F -dense "1,1,0,1" -end \
9190 ▷ ▷ -with F -do \
9191 ▷ ▷ -finite_field_activity \
9192 ▷ ▷ ▷ -Berlekamp_matrix v \
9193 ▷ ▷ -end
9194 ▷ $(PDFLATEX) Berlekamp_matrix_q2_d3.tex
9195 ▷ $(OPEN) Berlekamp_matrix_q2_d3.pdf
9196
9197 # the polynomial  $X^3+X+1$  is irreducible over  $GF(2)$  because the rank of the Berlek
amp matrix is 2.
9198
9199 Berlekamp_matrix_2.4:
9200 ▷ $(ORBITER) -v 2 \
9201 ▷ ▷ -define F -finite_field -q 2 -end \
9202 ▷ ▷ -define v -vector -field F -dense "1,1,0,0,1" -end \
9203 ▷ ▷ -with F -do \
9204 ▷ ▷ -finite_field_activity \
9205 ▷ ▷ ▷ -Berlekamp_matrix v \
9206 ▷ ▷ -end
9207
9208 # the polynomial  $X^4+X+1$  is irreducible over  $GF(2)$  because the rank of the Berlek
amp matrix is 3.
9209
9210
9211 Berlekamp_matrix_4.3a:
9212 ▷ $(ORBITER) -v 2 \
9213 ▷ ▷ -define F -finite_field -q 4 -end \
9214 ▷ ▷ -define v -vector -field F -dense "1,3,0,1" -end \
9215 ▷ ▷ -with F -do \
9216 ▷ ▷ -finite_field_activity \
9217 ▷ ▷ ▷ -Berlekamp_matrix v \
9218 ▷ ▷ -end
9219
9220 Berlekamp_matrix_4.3b:
9221 ▷ $(ORBITER) -v 2 \
9222 ▷ ▷ -define F -finite_field -q 4 -end \
9223 ▷ ▷ -define v -vector -field F -dense "1,3,1,1" -end \
9224 ▷ ▷ -with F -do \
9225 ▷ ▷ -finite_field_activity \
9226 ▷ ▷ ▷ -Berlekamp_matrix v \
9227 ▷ ▷ -end
9228
9229
9230
9231
9232 find_roots.a:
9233 ▷ $(ORBITER) -v 2 \
9234 ▷ ▷ -define F -finite_field -q 19 -end \
9235 ▷ ▷ -define v -vector -field F -dense "18,1,1" -end \
9236 ▷ ▷ -with F -do \

```

```

9237 ▷ ▷ -finite_field.activity \
9238 ▷ ▷ ▷ -polynomial.find_roots v \
9239 ▷ ▷ -end
9240
9241 find_roots.b:
9242 ▷ $(ORBITER) -v 2 \
9243 ▷ ▷ -define F -finite_field -q 19 -end \
9244 ▷ ▷ -define v -vector -field F -dense "1,3,1" -end \
9245 ▷ ▷ -with F -do \
9246 ▷ ▷ -finite_field.activity \
9247 ▷ ▷ ▷ -polynomial.find_roots v \
9248 ▷ ▷ -end
9249
9250 find_roots.c:
9251 ▷ $(ORBITER) -v 2 \
9252 ▷ ▷ -define F -finite_field -q 19 -end \
9253 ▷ ▷ -define v -vector -field F -dense "1,16,1" -end \
9254 ▷ ▷ -with F -do \
9255 ▷ ▷ -finite_field.activity \
9256 ▷ ▷ ▷ -polynomial.find_roots v \
9257 ▷ ▷ -end
9258
9259 find_roots.d:
9260 ▷ $(ORBITER) -v 2 \
9261 ▷ ▷ -define F -finite_field -q 19 -end \
9262 ▷ ▷ -define v -vector -field F -dense "1,18,1" -end \
9263 ▷ ▷ -with F -do \
9264 ▷ ▷ -finite_field.activity \
9265 ▷ ▷ ▷ -polynomial.find_roots v \
9266 ▷ ▷ -end
9267
9268 find_roots.e:
9269 ▷ $(ORBITER) -v 2 \
9270 ▷ ▷ -define F -finite_field -q 19 -end \
9271 ▷ ▷ -define v -vector -field F -dense "1,16,3" -end \
9272 ▷ ▷ -with F -do \
9273 ▷ ▷ -finite_field.activity \
9274 ▷ ▷ ▷ -polynomial.find_roots v \
9275 ▷ ▷ -end
9276
9277
9278 roots_over_F2:
9279 ▷ $(ORBITER) -v 2 \
9280 ▷ ▷ -define F -finite_field -q 2 -end \
9281 ▷ ▷ -define v -vector -field F -dense "0,1,0,1,1,1" -end \
9282 ▷ ▷ -with F -do \
9283 ▷ ▷ -finite_field.activity \
9284 ▷ ▷ ▷ -polynomial.find_roots v \
9285 ▷ ▷ -end
9286
9287
9288
9289 roots_over_F8:
9290 ▷ $(ORBITER) -v 2 \
9291 ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
9292 ▷ ▷ -define v -vector -field F -dense "0,1,0,1,1,1" -end \
9293 ▷ ▷ -with F -do \
9294 ▷ ▷ -finite_field.activity \
9295 ▷ ▷ ▷ -polynomial.find_roots v \

```

```

9296 ▷ ▷ -end
9297
9298
9299
9300
9301 # degree and then order of the field of coefficients:
9302
9303 irred_3.2:
9304 ▷ $(ORBITER) -v 3 \
9305 ▷ ▷ -define F -finite_field -q 2 -end \
9306 ▷ ▷ -with F -do \
9307 ▷ ▷ -finite_field.activity \
9308 ▷ ▷ ▷ -make_table_of_irreducible_polynomials 3 \
9309 ▷ ▷ -end
9310 ▷ $(PDFLATEX) Irred_q2_d3.tex
9311 ▷ $(OPEN) Irred_q2_d3.pdf
9312
9313
9314 irred_4.2:
9315 ▷ $(ORBITER) -v 3 \
9316 ▷ ▷ -define F -finite_field -q 2 -end \
9317 ▷ ▷ -with F -do \
9318 ▷ ▷ -finite_field.activity \
9319 ▷ ▷ ▷ -make_table_of_irreducible_polynomials 4 \
9320 ▷ ▷ -end
9321 ▷ $(PDFLATEX) Irred_q2_d4.tex
9322 ▷ $(OPEN) Irred_q2_d4.pdf
9323
9324 # 3 polys
9325
9326 irred_5.2:
9327 ▷ $(ORBITER) -v 3 \
9328 ▷ ▷ -define F -finite_field -q 2 -end \
9329 ▷ ▷ -with F -do \
9330 ▷ ▷ -finite_field.activity \
9331 ▷ ▷ ▷ -make_table_of_irreducible_polynomials 5 \
9332 ▷ ▷ -end
9333 ▷ $(PDFLATEX) Irred_q2_d5.tex
9334 ▷ $(OPEN) Irred_q2_d5.pdf
9335
9336 # 6 polys
9337
9338 irred_6.2:
9339 ▷ $(ORBITER) -v 3 \
9340 ▷ ▷ -define F -finite_field -q 2 -end \
9341 ▷ ▷ -with F -do \
9342 ▷ ▷ -finite_field.activity \
9343 ▷ ▷ ▷ -make_table_of_irreducible_polynomials 6 \
9344 ▷ ▷ -end
9345 ▷ $(PDFLATEX) Irred_q2_d6.tex
9346 ▷ $(OPEN) Irred_q2_d6.pdf
9347
9348 # 9 polys: 97, 73, 67, 115, 117, 109, 91, 87, 103.
9349
9350 irred_7.2:
9351 ▷ $(ORBITER) -v 3 \
9352 ▷ ▷ -define F -finite_field -q 2 -end \
9353 ▷ ▷ -with F -do \
9354 ▷ ▷ -finite_field.activity \

```

```

9355 ▷ ▷ ▷ -make_table_of_irreducible_polynomials 7 \
9356 ▷ ▷ -end
9357 ▷ $(PDFLATEX) Irred.q2_d7.tex
9358 ▷ $(OPEN) Irred.q2_d7.pdf
9359
9360 # 18 polys
9361
9362 irred_8.2:
9363 ▷ $(ORBITER) -v 3 \
9364 ▷ ▷ -define F -finite_field -q 2 -end \
9365 ▷ ▷ -with F -do \
9366 ▷ ▷ -finite_field_activity \
9367 ▷ ▷ ▷ -make_table_of_irreducible_polynomials 8 \
9368 ▷ ▷ -end
9369 ▷ $(PDFLATEX) Irred.q2_d8.tex
9370 ▷ $(OPEN) Irred.q2_d8.pdf
9371
9372 # 30 polys
9373
9374 irred_9.2:
9375 ▷ $(ORBITER) -v 3 \
9376 ▷ ▷ -define F -finite_field -q 2 -end \
9377 ▷ ▷ -with F -do \
9378 ▷ ▷ -finite_field_activity \
9379 ▷ ▷ ▷ -make_table_of_irreducible_polynomials 9 \
9380 ▷ ▷ -end
9381 ▷ $(PDFLATEX) Irred.q2_d9.tex
9382 ▷ $(OPEN) Irred.q2_d9.pdf
9383
9384 # 56 polys
9385
9386 irred_10.2:
9387 ▷ $(ORBITER) -v 3 \
9388 ▷ ▷ -define F -finite_field -q 2 -end \
9389 ▷ ▷ -with F -do \
9390 ▷ ▷ -finite_field_activity \
9391 ▷ ▷ ▷ -make_table_of_irreducible_polynomials 10 \
9392 ▷ ▷ -end
9393 ▷ $(PDFLATEX) Irred.q2_d10.tex
9394 ▷ $(OPEN) Irred.q2_d10.pdf
9395
9396 # 99 polys
9397
9398 irred_2.4:
9399 ▷ $(ORBITER) -v 3 \
9400 ▷ ▷ -define F -finite_field -q 4 -end \
9401 ▷ ▷ -with F -do \
9402 ▷ ▷ -finite_field_activity \
9403 ▷ ▷ ▷ -make_table_of_irreducible_polynomials 2 \
9404 ▷ ▷ -end
9405 ▷ $(PDFLATEX) Irred.q4_d2.tex
9406 ▷ $(OPEN) Irred.q4_d2.pdf
9407
9408 # 6 polys
9409
9410
9411 irred_3.4:
9412 ▷ $(ORBITER) -v 6 \
9413 ▷ ▷ -define F -finite_field -q 4 -end \

```

```

9414 ▷ ▷ -with F -do \
9415 ▷ ▷ -finite_field_activity \
9416 ▷ ▷ ▷ -make_table_of_irreducible_polynomials 3 \
9417 ▷ ▷ -end
9418 ▷ $(PDFLATEX) Irred.q4.d3.tex
9419 ▷ $(OPEN) Irred.q4.d3.pdf
9420
9421 # 20 polys
9422
9423
9424 F2_get_primitive_poly_of_deg_10:
9425 ▷ $(ORBITER) -v 6 \
9426 ▷ ▷ -define F -finite_field -q 2 -end \
9427 ▷ ▷ -with F -do \
9428 ▷ ▷ -finite_field_activity \
9429 ▷ ▷ ▷ -get_primitive_polynomial 10 \
9430 ▷ ▷ -end
9431
9432
9433 F2_get_primitive_poly_range_2_10:
9434 ▷ $(ORBITER) -v 2 \
9435 ▷ ▷ -define F -finite_field -q 2 -end \
9436 ▷ ▷ -with F -do \
9437 ▷ ▷ -finite_field_activity \
9438 ▷ ▷ ▷ -get_primitive_polynomial_in_range 2 10 \
9439 ▷ ▷ -end
9440
9441 #| grep //
9442 # for degree 61, it gets stuck in factoring 2^61-1 (which is prime)
9443
9444
9445
9446
9447
9448
9449
9450
9451 #####
9452 # Section 6.2: Multivariate Polynomials
9453
9454
9455
9456 SECTION_MULTIVARIATE_POLYNOMIALS:
9457
9458
9459 test_6.2:
9460 ▷ make polynomial_ring_degree3_vars4_q13
9461 ▷ make parse_homogeneous_polynomial
9462 ▷ make parse_homogeneous_polynomial_with_parameters
9463 ▷ make affine_map_F5_0
9464 ▷ make affine_map_F5_1
9465 ▷ make affine_map_F5_2
9466 ▷ make affine_map_F5_3
9467 ▷ make affine_map_F5_4
9468 ▷ make permutation_polynomial_F5
9469 ▷ make conic_variety
9470 ▷ make ideal_of_conic
9471 ▷ make quartic_curve_q27_fo338_apply_transformation
9472

```



```

9473
9474
9475 polynomial_ring_degree3_vars4.q13:
9476 ▷ $(ORBITER) -v 6 \
9477 ▷ ▷ -define Do -draw.options \
9478 ▷ ▷ ▷ -radius 200 -line.width 0.3 \
9479 ▷ ▷ -end \
9480 ▷ ▷ -define F -finite.field -q 13 -end \
9481 ▷ ▷ -define P -projective.space -n 3 -field F -v 0 -end \
9482 ▷ ▷ -define R -polynomial_ring \
9483 ▷ ▷ ▷ -field F \
9484 ▷ ▷ ▷ -number_of_variables 4 \
9485 ▷ ▷ ▷ -homogeneous_of_degree 3 \
9486 ▷ ▷ ▷ -monomial_ordering_partition \
9487 ▷ ▷ ▷ -variables "x0,x1,x2,x3" "x_0,x_1,x_2,x_3" \
9488 ▷ ▷ -end \
9489 ▷ ▷ -with R -do \
9490 ▷ ▷ ▷ -ring_theoretic_activity \
9491 ▷ ▷ ▷ -cheat_sheet \
9492 ▷ ▷ -end
9493 ▷ $(PDFLATEX) poly_ring_deg3_vars4_cheat_sheet.tex
9494 ▷ $(OPEN) poly_ring_deg3_vars4_cheat_sheet.pdf
9495
9496
9497
9498 parse_homogeneous_polynomial:
9499 ▷ $(ORBITER) -v 6 \
9500 ▷ ▷ -define F -finite.field -q 13 -end \
9501 ▷ ▷ -define R -polynomial_ring \
9502 ▷ ▷ ▷ -field F \
9503 ▷ ▷ ▷ -number_of_variables 3 \
9504 ▷ ▷ ▷ -homogeneous_of_degree 3 \
9505 ▷ ▷ ▷ -monomial_ordering_partition \
9506 ▷ ▷ ▷ -variables "y0,y1,y2" "y_0,y_1,y_2" \
9507 ▷ ▷ -end \
9508 ▷ ▷ -with R -do \
9509 ▷ ▷ ▷ -ring_theoretic_activity \
9510 ▷ ▷ ▷ -parse_equation_wo_parameters "eqn" "eqn" "y0^3+y1^3+y2^3" \
9511 ▷ ▷ -end
9512
9513 parse_homogeneous_polynomial_with_parameters:
9514 ▷ $(ORBITER) -v 6 \
9515 ▷ ▷ -define F -finite.field -q 17 -end \
9516 ▷ ▷ -define R -polynomial_ring \
9517 ▷ ▷ ▷ -field F \
9518 ▷ ▷ ▷ -number_of_variables 3 \
9519 ▷ ▷ ▷ -homogeneous_of_degree 4 \
9520 ▷ ▷ ▷ -monomial_ordering_partition \
9521 ▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \
9522 ▷ ▷ -end \
9523 ▷ ▷ -with R -do \
9524 ▷ ▷ ▷ -ring_theoretic_activity \
9525 ▷ ▷ ▷ -parse_equation \
9526 ▷ ▷ ▷ ▷ "Edge_curve" "Edge\_curve" \
9527 ▷ ▷ ▷ ▷ $(EDGE_CURVE_EQUATION_ALGEBRAIC) \
9528 ▷ ▷ ▷ ▷ f 3 \
9529 ▷ ▷ -end
9530
9531

```

```

9532 #-parse.equation The equation is:
9533 #( 1, 16, 16, 0, 0, 0, 0, 0, 0, 0, 1, 12, 0, 0 )
9534
9535
9536 affine_map_F5.0:
9537 ▷ $(ORBITER) -v 8 \
9538 ▷ ▷ -define F -finite_field -q 5 -end \
9539 ▷ ▷ -define P -projective_space -n 1 -field F -v 0 -end \
9540 ▷ ▷ -define R -polynomial_ring \
9541 ▷ ▷ ▷ -field F \
9542 ▷ ▷ ▷ -number_of_variables 2 \
9543 ▷ ▷ ▷ -homogeneous_of_degree 4 \
9544 ▷ ▷ ▷ -monomial_ordering_lex \
9545 ▷ ▷ ▷ -variables "x,y" "x,y" \
9546 ▷ ▷ ▷ -end \
9547 ▷ ▷ -define Y0 -symbolic_object \
9548 ▷ ▷ ▷ -field F \
9549 ▷ ▷ ▷ -text "1-x^4" \
9550 ▷ ▷ ▷ -end \
9551 ▷ ▷ -define Y -symbolic_object \
9552 ▷ ▷ ▷ -field F \
9553 ▷ ▷ ▷ -text "Y0" \
9554 ▷ ▷ ▷ -end \
9555 ▷ ▷ -with P -do \
9556 ▷ ▷ -projective_space_activity \
9557 ▷ ▷ ▷ -affine_map R Y "" \
9558 ▷ ▷ -end
9559
9560 #( 1, 0, 0, 0, 0 )
9561
9562 affine_map_F5.1:
9563 ▷ $(ORBITER) -v 6 \
9564 ▷ ▷ -define F -finite_field -q 5 -end \
9565 ▷ ▷ -define P -projective_space -n 1 -field F -v 0 -end \
9566 ▷ ▷ -define R -polynomial_ring \
9567 ▷ ▷ ▷ -field F \
9568 ▷ ▷ ▷ -number_of_variables 2 \
9569 ▷ ▷ ▷ -homogeneous_of_degree 4 \
9570 ▷ ▷ ▷ -monomial_ordering_lex \
9571 ▷ ▷ ▷ -variables "x,y" "x,y" \
9572 ▷ ▷ ▷ -end \
9573 ▷ ▷ -define Y0 -symbolic_object \
9574 ▷ ▷ ▷ -field F \
9575 ▷ ▷ ▷ -text "1-(x-1)^4" \
9576 ▷ ▷ ▷ -end \
9577 ▷ ▷ -define Y -symbolic_object \
9578 ▷ ▷ ▷ -field F \
9579 ▷ ▷ ▷ -text "Y0" \
9580 ▷ ▷ ▷ -end \
9581 ▷ ▷ -with P -do \
9582 ▷ ▷ -projective_space_activity \
9583 ▷ ▷ ▷ -affine_map R Y "" \
9584 ▷ ▷ -end
9585
9586 #( 0, 1, 0, 0, 0 )
9587
9588 affine_map_F5.2:
9589 ▷ $(ORBITER) -v 6 \
9590 ▷ ▷ -define F -finite_field -q 5 -end \

```

```

9591 ▷ ▷ -define P -projective_space -n 1 -field F -v 0 -end \
9592 ▷ ▷ -define R -polynomial_ring \
9593 ▷ ▷ ▷ -field F \
9594 ▷ ▷ ▷ -number_of_variables 2 \
9595 ▷ ▷ ▷ -homogeneous_of_degree 4 \
9596 ▷ ▷ ▷ -monomial_ordering_lex \
9597 ▷ ▷ ▷ -variables "x,y" "x,y" \
9598 ▷ ▷ ▷ -end \
9599 ▷ ▷ -define Y0 -symbolic_object \
9600 ▷ ▷ ▷ -field F \
9601 ▷ ▷ ▷ -text "1-(x-2)^4" \
9602 ▷ ▷ ▷ -end \
9603 ▷ ▷ -define Y -symbolic_object \
9604 ▷ ▷ ▷ -field F \
9605 ▷ ▷ ▷ -text "Y0" \
9606 ▷ ▷ ▷ -end \
9607 ▷ ▷ -with P -do \
9608 ▷ ▷ -projective_space_activity \
9609 ▷ ▷ ▷ -affine_map R Y "" \
9610 ▷ ▷ -end
9611
9612 #( 0, 0, 1, 0, 0 )
9613
9614 affine_map_F5.3:
9615 ▷ $(ORBITER) -v 6 \
9616 ▷ ▷ -define F -finite_field -q 5 -end \
9617 ▷ ▷ -define P -projective_space -n 1 -field F -v 0 -end \
9618 ▷ ▷ -define R -polynomial_ring \
9619 ▷ ▷ ▷ -field F \
9620 ▷ ▷ ▷ -number_of_variables 2 \
9621 ▷ ▷ ▷ -homogeneous_of_degree 4 \
9622 ▷ ▷ ▷ -monomial_ordering_lex \
9623 ▷ ▷ ▷ -variables "x,y" "x,y" \
9624 ▷ ▷ ▷ -end \
9625 ▷ ▷ -define Y0 -symbolic_object \
9626 ▷ ▷ ▷ -field F \
9627 ▷ ▷ ▷ -text "1-(x-3)^4" \
9628 ▷ ▷ ▷ -end \
9629 ▷ ▷ -define Y -symbolic_object \
9630 ▷ ▷ ▷ -field F \
9631 ▷ ▷ ▷ -text "Y0" \
9632 ▷ ▷ ▷ -end \
9633 ▷ ▷ -with P -do \
9634 ▷ ▷ -projective_space_activity \
9635 ▷ ▷ ▷ -affine_map R Y "" \
9636 ▷ ▷ -end
9637
9638
9639 #( 0, 0, 0, 1, 0 )
9640
9641
9642
9643 affine_map_F5.4:
9644 ▷ $(ORBITER) -v 6 \
9645 ▷ ▷ -define F -finite_field -q 5 -end \
9646 ▷ ▷ -define P -projective_space -n 1 -field F -v 0 -end \
9647 ▷ ▷ -define R -polynomial_ring \
9648 ▷ ▷ ▷ -field F \
9649 ▷ ▷ ▷ -number_of_variables 2 \

```

```

9650 ▷ ▷ ▷ -homogeneous_of_degree 4 \
9651 ▷ ▷ ▷ -monomial_ordering_lex \
9652 ▷ ▷ ▷ -variables "x,y" "x,y" \
9653 ▷ ▷ ▷ -end \
9654 ▷ ▷ -define Y0 -symbolic_object \
9655 ▷ ▷ ▷ -field F \
9656 ▷ ▷ ▷ -text "1-(x-4)^4" \
9657 ▷ ▷ ▷ -end \
9658 ▷ ▷ -define Y -symbolic_object \
9659 ▷ ▷ ▷ -field F \
9660 ▷ ▷ ▷ -text "Y0" \
9661 ▷ ▷ ▷ -end \
9662 ▷ ▷ -with P -do \
9663 ▷ ▷ -projective_space_activity \
9664 ▷ ▷ ▷ -affine_map R Y "" \
9665 ▷ ▷ -end
9666
9667
9668 #( 0, 0, 0, 0, 1 )
9669
9670
9671
9672 permutation_polynomial_F5:
9673 ▷ $(ORBITER) -v 6 \
9674 ▷ ▷ -define F -finite_field -q 5 -end \
9675 ▷ ▷ -define P -projective_space -n 1 -field F -v 0 -end \
9676 ▷ ▷ -define R -polynomial_ring \
9677 ▷ ▷ ▷ -field F \
9678 ▷ ▷ ▷ -number_of_variables 2 \
9679 ▷ ▷ ▷ -homogeneous_of_degree 4 \
9680 ▷ ▷ ▷ -monomial_ordering_lex \
9681 ▷ ▷ ▷ -variables "x,y" "x,y" \
9682 ▷ ▷ ▷ -end \
9683 ▷ ▷ -define Y0 -symbolic_object \
9684 ▷ ▷ ▷ -field F \
9685 ▷ ▷ ▷ -text "3*(1-x^4)+4*(1-(x-2)^4)+1*(1-(x-3)^4)+2*(1-(x-4)^4)" \
9686 ▷ ▷ ▷ -end \
9687 ▷ ▷ -define Y -symbolic_object \
9688 ▷ ▷ ▷ -field F \
9689 ▷ ▷ ▷ -text "Y0" \
9690 ▷ ▷ ▷ -end \
9691 ▷ ▷ -with P -do \
9692 ▷ ▷ -projective_space_activity \
9693 ▷ ▷ ▷ -affine_map R Y "" \
9694 ▷ ▷ -end \
9695 ▷ ▷ -define Yx -symbolic_object \
9696 ▷ ▷ ▷ -field F \
9697 ▷ ▷ ▷ -expand Y \
9698 ▷ ▷ -end
9699
9700
9701
9702 conic_variety:
9703 ▷ $(ORBITER) -v 6 \
9704 ▷ ▷ -define F -finite_field -q 13 -end \
9705 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
9706 ▷ ▷ -define R -polynomial_ring \
9707 ▷ ▷ ▷ -field F \
9708 ▷ ▷ ▷ -number_of_variables 3 \

```

```

9709 ▷ ▷ ▷ -homogeneous_of_degree 2 \
9710 ▷ ▷ ▷ -monomial_ordering_partition \
9711 ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
9712 ▷ ▷ -end \
9713 ▷ ▷ -define V -variety \
9714 ▷ ▷ ▷ -projective_space P \
9715 ▷ ▷ ▷ -ring R \
9716 ▷ ▷ ▷ -equation_in_algebraic_form \
9717 ▷ ▷ ▷ ▷ "X0^2-X1*X2" \
9718 ▷ ▷ ▷ -label_txt conic \
9719 ▷ ▷ ▷ -label_tex "{\\rm conic}" \
9720 ▷ ▷ -end
9721
9722 # 1, 2, 3, 39, 57, 62, 68, 77, 134, 141, 150, 151, 175, 178
9723
9724 ideal_of_conic:
9725 ▷ $(ORBITER) -v 6 \
9726 ▷ ▷ -define F -finite_field -q 13 -end \
9727 ▷ ▷ -define R -polynomial_ring \
9728 ▷ ▷ ▷ -field F \
9729 ▷ ▷ ▷ -number_of_variables 3 \
9730 ▷ ▷ ▷ -homogeneous_of_degree 2 \
9731 ▷ ▷ ▷ -monomial_ordering_partition \
9732 ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
9733 ▷ ▷ -end \
9734 ▷ ▷ -with R -do \
9735 ▷ ▷ ▷ -ring_theoretic_activity \
9736 ▷ ▷ ▷ -ideal "conic" "conic" "1, 2, 3, 39, 57, 62, 68, 77, 134, 141, 150, 151, 175, 178" \
9737 ▷ ▷ -end
9738
9739 #X0^2 + 12*X1*X2
9740
9741
9742
9743
9744
9745 quartic_curve_q27_fo338_apply_transformation:
9746 ▷ $(ORBITER) -v 6 \
9747 ▷ ▷ -define F -finite_field -q 27 -end \
9748 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
9749 ▷ ▷ -define R -polynomial_ring \
9750 ▷ ▷ ▷ -field F \
9751 ▷ ▷ ▷ -number_of_variables 3 \
9752 ▷ ▷ ▷ -homogeneous_of_degree 4 \
9753 ▷ ▷ ▷ -monomial_ordering_partition \
9754 ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
9755 ▷ ▷ -end \
9756 ▷ ▷ -define eqn -vector -dense \
9757 ▷ ▷ ▷ $(QC_Q27_F0388_COEFFS) \
9758 ▷ ▷ -end \
9759 ▷ ▷ -define trans_coeffs -vector -dense \
9760 ▷ ▷ ▷ $(QC_Q27_F0388_TRANS) \
9761 ▷ ▷ -end \
9762 ▷ ▷ -define G -linear_group -PGGL 3 F -end \
9763 ▷ ▷ -define trans -vector_ge \
9764 ▷ ▷ ▷ -action G \
9765 ▷ ▷ ▷ -vector_data trans_coeffs \
9766 ▷ ▷ -end \

```

```

9767 ▷ ▷ -with R -do \
9768 ▷ ▷ ▷ -ring_theoretic_activity \
9769 ▷ ▷ ▷ -apply_transformation P eqn trans \
9770 ▷ ▷ -end
9771
9772
9773
9774
9775
9776 #####
9777 # Chapter 7 - Algebraic Geometry
9778 #####
9779
9780
9781 test_7:
9782 ▷ make test_7.1
9783 ▷ make test_7.2
9784 ▷ make test_7.3
9785 ▷ make test_7.4
9786 ▷ make test_7.5
9787 ▷ make test_7.6
9788
9789
9790
9791 #####
9792 # Section 7.1: The Grassmannian
9793
9794
9795 SECTION.GRASSMANNIAN:
9796
9797
9798
9799 test_7.1:
9800 ▷ make GR_3.2.2
9801 ▷ make planes_in_pencil
9802
9803
9804
9805
9806
9807 GR_3.2.2:
9808 ▷ $(ORBITER) -v 2 \
9809 ▷ ▷ -define F -finite_field -q 2 -end \
9810 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
9811 ▷ ▷ -with P -do \
9812 ▷ ▷ -projective_space_activity \
9813 ▷ ▷ ▷ -report_Grassmannian 2 \
9814 ▷ ▷ -end
9815 ▷ $(PDFLATEX) Gr_3.2.2.tex
9816 ▷ $(OPEN) Gr_3.2.2.pdf
9817
9818
9819
9820
9821
9822
9823 planes_in_pencil:
9824 ▷ $(ORBITER) -v 2 \
9825 ▷ ▷ -define F -finite_field -q 8 -end \

```

```

9826 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
9827 ▷ ▷ -with P -do \
9828 ▷ ▷ -projective_space_activity \
9829 ▷ ▷ ▷ -planes_through_line 0 \
9830 ▷ ▷ -end
9831
9832
9833
9834
9835
9836 #####
9837 # Section 7.2: The Klein Quadric and Pluecker coordinates
9838
9839
9840 SECTION_KLEIN_QUADRIC_AND_PLUECKER_COORDINATES:
9841
9842
9843 test_7.2:
9844 ▷ make GR_4.2.2
9845
9846
9847 GR_4.2.2:
9848 ▷ $(ORBITER) -v 2 \
9849 ▷ ▷ -define F -finite_field -q 2 -end \
9850 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
9851 ▷ ▷ -with P -do \
9852 ▷ ▷ -projective_space_activity \
9853 ▷ ▷ ▷ -report_Grassmannian 2 \
9854 ▷ ▷ -end
9855 ▷ $(PDFLATEX) Gr_4.2.2.tex
9856 ▷ $(OPEN) Gr_4.2.2.pdf
9857
9858
9859
9860
9861
9862 #####
9863 # Section 7.3: Orthogonal spaces
9864
9865
9866 SECTION_ORTHOGONAL_SPACES:
9867
9868
9869 test_7.3:
9870 ▷ make Op_4.2
9871 ▷ make 0.5.2.incidence_matrix.csv
9872 ▷ make Op_6.2
9873 ▷ make Op_6.2.incidence_matrix.csv
9874 ▷ make Op_6.2.with_group
9875 ▷ make Op_6.8
9876 ▷ make Op_8.2
9877 ▷ make Op_6.64
9878 ▷ #make Op_6.64.line_rank_problem
9879 ▷ make Op_6.64.line_rank
9880 ▷ make Op_6.64.report
9881 ▷ #make elliptic_quadric_subspace fails
9882 ▷ make BLT_database_7.1
9883 ▷ make BLT_database_7.1.print
9884 ▷ make Doily_W.2

```

```

9885 ▷ make Op_6_2.orthogonal_reflections
9886 ▷ make Op_6_2.create_Siegel_transformation_2B
9887 ▷ make Op_6_2.create_Siegel_transformation_2C
9888 ▷ make Op_6_2.report_Siegel_transformation
9889 ▷ make Op_6_2.Siegel_transformation_expand_class_2A
9890 ▷ make Op_6_2.Siegel_transformation_expand_class_2C
9891 ▷ make Op_6_2.make_all_Siegel_transformations
9892 ▷ make Op_6_2.all_Siegel_transformations_tally
9893
9894
9895
9896 Op_4_2:
9897 ▷ $(ORBITER) -v 2 \
9898 ▷ ▷ -define Do -draw_options \
9899 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
9900 ▷ ▷ -end \
9901 ▷ ▷ -define F -finite_field -q 2 -end \
9902 ▷ ▷ -define O -orthogonal_space 1 4 F -without_group -end \
9903 ▷ ▷ -with O -do -orthogonal_space_activity \
9904 ▷ ▷ ▷ -cheat_sheet_orthogonal Do \
9905 ▷ ▷ -end
9906 ▷ $(PDFLATEX) 0_1_4_2_report.tex
9907 ▷ $(OPEN) 0_1_4_2_report.pdf
9908
9909
9910 0_5_2_incidence_matrix.csv:
9911 ▷ $(ORBITER) -v 2 \
9912 ▷ ▷ -define F -finite_field -q 2 -end \
9913 ▷ ▷ -define O -orthogonal_space 0 5 F -without_group -end \
9914 ▷ ▷ -with O -do -orthogonal_space_activity \
9915 ▷ ▷ ▷ -export_point_line_incidence_matrix \
9916 ▷ ▷ -end
9917 ▷ $(ORBITER) -v 2 \
9918 ▷ ▷ -define all_one_r -vector -repeat 1 15 -end \
9919 ▷ ▷ -define all_one_c -vector -repeat 1 15 -end \
9920 ▷ ▷ -draw_matrix \
9921 ▷ ▷ ▷ -input_csv_file 0_5_2_incidence_matrix.csv \
9922 ▷ ▷ ▷ -box_width 20 -bit_depth 8 \
9923 ▷ ▷ ▷ -partition 2 \
9924 ▷ ▷ ▷ ▷ all_one_r all_one_c \
9925 ▷ ▷ -end
9926 ▷ $(OPEN) 0_5_2_incidence_matrix_draw.bmp
9927
9928 #0(5,2) projectively =  $Q(4,2)$  = (dual of)  $W(3,2)$  =  $W(3,2)$ 
9929 # recall that  $W(3,2)$  and  $Q(4,q)$  are self dual if  $q$  is even
9930
9931
9932 Op_6_2:
9933 ▷ $(ORBITER) -v 2 \
9934 ▷ ▷ -define Do -draw_options \
9935 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
9936 ▷ ▷ -end \
9937 ▷ ▷ -define F -finite_field -q 2 -end \
9938 ▷ ▷ -define O -orthogonal_space 1 6 F -without_group -end \
9939 ▷ ▷ -with O -do -orthogonal_space_activity \
9940 ▷ ▷ ▷ -cheat_sheet_orthogonal Do \
9941 ▷ ▷ -end
9942 ▷ $(PDFLATEX) 0_1_6_2_report.tex
9943 ▷ $(OPEN) 0_1_6_2_report.pdf

```



```

9944
9945
9946 Op_6_2.incidence_matrix.csv:
9947 ▷ $(ORBITER) -v 2 \
9948 ▷ ▷ -define F -finite_field -q 2 -end \
9949 ▷ ▷ -define O -orthogonal_space 1 6 F -without_group -end \
9950 ▷ ▷ -with O -do -orthogonal_space_activity \
9951 ▷ ▷ ▷ -export_point_line_incidence_matrix \
9952 ▷ ▷ -end
9953 ▷ $(ORBITER) -v 2 \
9954 ▷ ▷ -define all_one_r -vector -repeat 1 35 -end \
9955 ▷ ▷ -define all_one_c -vector -repeat 1 105 -end \
9956 ▷ ▷ -draw_matrix \
9957 ▷ ▷ ▷ -input_csv_file Op_6_2.incidence_matrix.csv \
9958 ▷ ▷ ▷ -box_width 20 -bit_depth 8 \
9959 ▷ ▷ ▷ -partition 2 \
9960 ▷ ▷ ▷ ▷ all_one_r all_one_c \
9961 ▷ ▷ -end
9962 ▷ $(OPEN) Op_6_2.incidence_matrix_draw.bmp
9963
9964
9965 Op_6_2.with_group:
9966 ▷ $(ORBITER) -v 2 \
9967 ▷ ▷ -define Do -draw_options \
9968 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
9969 ▷ ▷ -end \
9970 ▷ ▷ -define F -finite_field -q 2 -end \
9971 ▷ ▷ -define O -orthogonal_space 1 6 F -end \
9972 ▷ ▷ -with O -do -orthogonal_space_activity \
9973 ▷ ▷ ▷ -cheat_sheet_orthogonal Do \
9974 ▷ ▷ -end
9975 ▷ $(PDFLATEX) 0_1.6.2.report.tex
9976 ▷ $(OPEN) 0_1.6.2.report.pdf
9977
9978
9979 Op_6_8:
9980 ▷ $(ORBITER) -v 2 \
9981 ▷ ▷ -define Do -draw_options \
9982 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
9983 ▷ ▷ -end \
9984 ▷ ▷ -define F -finite_field -q 8 -end \
9985 ▷ ▷ -define O -orthogonal_space 1 6 F -without_group -end \
9986 ▷ ▷ -with O -do -orthogonal_space_activity \
9987 ▷ ▷ ▷ -cheat_sheet_orthogonal Do \
9988 ▷ ▷ -end
9989 ▷ $(PDFLATEX) 0_1.6.8.report.tex
9990 ▷ $(OPEN) 0_1.6.8.report.pdf
9991
9992
9993 Op_8_2:
9994 ▷ $(ORBITER) -v 2 \
9995 ▷ ▷ -define Do -draw_options \
9996 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
9997 ▷ ▷ -end \
9998 ▷ ▷ -define F -finite_field -q 2 -end \
9999 ▷ ▷ -define O -orthogonal_space 1 8 F -without_group -end \
10000 ▷ ▷ -with O -do -orthogonal_space_activity \
10001 ▷ ▷ ▷ -cheat_sheet_orthogonal Do \
10002 ▷ ▷ -end

```

```

10003 ▷ $(PDFLATEX) 0_1.8.2_report.tex
10004 ▷ $(OPEN) 0_1.8.2_report.pdf
10005
10006
10007 Op_6_64:
10008 ▷ $(ORBITER) -v 2 \
10009 ▷ ▷ -define Do -draw_options \
10010 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
10011 ▷ ▷ -end \
10012 ▷ ▷ -define F -finite_field -q 64 -end \
10013 ▷ ▷ -define O -orthogonal_space 1 6 F -without_group -end \
10014 ▷ ▷ -with O -do -orthogonal_space_activity \
10015 ▷ ▷ ▷ -cheat_sheet_orthogonal Do \
10016 ▷ ▷ -end
10017 ▷ $(PDFLATEX) 0_1.6.64_report.tex
10018 ▷ $(OPEN) 0_1.6.64_report.pdf
10019
10020
10021 # problem, because we are trying to create PGL(6,64):
10022
10023 Op_6_64_line_rank_problem:
10024 ▷ $(ORBITER) -v 4 \
10025 ▷ ▷ -define F -finite_field -q 64 -end \
10026 ▷ ▷ -define O -orthogonal_space 1 6 F -end \
10027 ▷ ▷ -with O -do -orthogonal_space_activity \
10028 ▷ ▷ ▷ -unrank_line_through_two_points 15447347 15225451 \
10029 ▷ ▷ -end
10030
10031 # use option -without_group to skip the group. This will work:
10032
10033 Op_6_64_line_rank:
10034 ▷ $(ORBITER) -v 4 \
10035 ▷ ▷ -define F -finite_field -q 64 -end \
10036 ▷ ▷ -define O -orthogonal_space 1 6 F -without_group -end \
10037 ▷ ▷ -with O -do -orthogonal_space_activity \
10038 ▷ ▷ ▷ -unrank_line_through_two_points 15447347 15225451 \
10039 ▷ ▷ -end
10040
10041 # this will create a basic report without the group:
10042
10043 Op_6_64_report:
10044 ▷ $(ORBITER) -v 4 \
10045 ▷ ▷ -define Do -draw_options \
10046 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
10047 ▷ ▷ -end \
10048 ▷ ▷ -define F -finite_field -q 64 -end \
10049 ▷ ▷ -define O -orthogonal_space 1 6 F -without_group -end \
10050 ▷ ▷ -with O -do -orthogonal_space_activity \
10051 ▷ ▷ ▷ -cheat_sheet_orthogonal Do \
10052 ▷ ▷ -end
10053 ▷ $(PDFLATEX) 0_1.6.64_report.tex
10054 ▷ $(OPEN) 0_1.6.64_report.pdf
10055
10056
10057
10058 # ToDo: this command fails:
10059
10060 elliptic_quadric_subspace:
10061 ▷ $(ORBITER) -v 3 \

```

```

10062 ▷ ▷ -define F -finite_field -q 5 -end \
10063 ▷ ▷ -define v -vector -format 4 \
10064 ▷ ▷ ▷ -dense "1,3,0,0,0,0, 0,0,1,0,0,0, 0,0,0,1,0,0, 0,0,0,0,1,1" \
10065 ▷ ▷ -end \
10066 ▷ ▷ -define O -orthogonal_space 1 6 F -end \
10067 ▷ ▷ -with O -do -orthogonal_space_activity \
10068 ▷ ▷ ▷ -intersect_with_subspace v \
10069 ▷ ▷ -end
10070
10071 #We found that the intersection contains 26 points:
10072 #( 2, 3, 1572, 100, 98, 116, 132, 1570, 1588, 1604, 1631, 1688, 1704, 1706, 1747,
    1745, 1847, 1845, 1856, 1888, 1904, 1931, 1988, 2004, 2022, 2020 )
10073
10074
10075
10076 BLT_database_7.1:
10077 ▷ $(ORBITER) -v 2 \
10078 ▷ ▷ -define F -finite_field -q 7 -end \
10079 ▷ ▷ -define P -projective_space -n 4 -field F -v 0 -end \
10080 ▷ ▷ -define S -geometric_object P \
10081 ▷ ▷ ▷ -BLT_database 1 \
10082 ▷ ▷ -end \
10083 ▷ ▷ -with S -do -combinatorial_object_activity -save \
10084 ▷ ▷ -end
10085
10086 # writes BLT_7.1.txt
10087
10088 BLT_database_7.1.print:
10089 ▷ $(ORBITER) -v 2 \
10090 ▷ ▷ -define F -finite_field -q 7 -end \
10091 ▷ ▷ -define O -orthogonal_space 0 5 F -without_group -end \
10092 ▷ ▷ -define S -set -file_orbiter_format BLT_7.1.txt -end \
10093 ▷ ▷ -with O -do -orthogonal_space_activity \
10094 ▷ ▷ ▷ -print_points S -end
10095 ▷ $(PDFLATEX) S_set_report.tex
10096 ▷ $(OPEN) S_set_report.pdf
10097
10098
10099
10100 # O(5,2) space:
10101
10102 Doily_W.2:
10103 ▷ $(ORBITER) -v 2 \
10104 ▷ ▷ -define F -finite_field -q 2 -end \
10105 ▷ ▷ -define O -orthogonal_space 0 5 F -without_group -end \
10106 ▷ ▷ -define W2_points -set -loop 0 15 1 -end \
10107 ▷ ▷ -define W2_lines -set -loop 0 15 1 -end \
10108 ▷ ▷ -with O -do \
10109 ▷ ▷ -orthogonal_space_activity \
10110 ▷ ▷ ▷ -print_points W2_points \
10111 ▷ ▷ -end \
10112 ▷ ▷ -with O -do \
10113 ▷ ▷ -orthogonal_space_activity \
10114 ▷ ▷ ▷ -print_lines W2_lines \
10115 ▷ ▷ -end
10116 ▷ $(PDFLATEX) W2_points_set_report.tex
10117 ▷ $(OPEN) W2_points_set_report.pdf
10118 ▷ $(PDFLATEX) W2_lines_set_of_lines_report.tex
10119 ▷ $(OPEN) W2_lines_set_of_lines_report.pdf

```

```

10120
10121
10122 # the output defines doily.csv
10123
10124
10125
10126
10127
10128 Op_6_2.orthogonal.reflections:
10129 ▷ $(ORBITER) -v 2 \
10130 ▷ ▷ -define Pts -vector \
10131 ▷ ▷ ▷ -dense $(ORTHOGONAL_OP_6_2.POINTS.OFF) \
10132 ▷ ▷ -end \
10133 ▷ ▷ -define F -finite_field -q 2 -end \
10134 ▷ ▷ -define O -orthogonal_space 1 6 F -end \
10135 ▷ ▷ -with O -do -orthogonal_space_activity \
10136 ▷ ▷ ▷ -create_orthogonal_reflection Pts \
10137 ▷ ▷ -end \
10138 ▷ ▷ -define G -linear_group -PGOp 6 F -end \
10139 ▷ ▷ -define Op_6_2.orthogonal.reflections -vector_ge \
10140 ▷ ▷ ▷ -action G \
10141 ▷ ▷ ▷ -read_csv 0.1.6.2.orthogonal.reflections.csv Element \
10142 ▷ ▷ -end \
10143 ▷ ▷ -with G -do \
10144 ▷ ▷ -group_theoretic_activity \
10145 ▷ ▷ ▷ -element_processing \
10146 ▷ ▷ ▷ ▷ -input Op_6_2.orthogonal.reflections \
10147 ▷ ▷ ▷ ▷ -print \
10148 ▷ ▷ ▷ -end \
10149 ▷ ▷ -end
10150 ▷ $(PDFLATEX) Op_6_2.orthogonal.reflections.elements.tex
10151 ▷ $(OPEN) Op_6_2.orthogonal.reflections.elements.pdf
10152
10153
10154
10155 # 0.1.6.2.orthogonal.reflections.csv
10156
10157
10158 # create a Siegel transformation of class 2B:
10159
10160 Op_6_2.create.Siegel.transformation.2B:
10161 ▷ $(ORBITER) -v 2 \
10162 ▷ ▷ -define F -finite_field -q 2 -end \
10163 ▷ ▷ -define G -linear_group -PGOp 6 F -end \
10164 ▷ ▷ -define O -orthogonal_space 1 6 F -end \
10165 ▷ ▷ -with O -do -orthogonal_space_activity \
10166 ▷ ▷ ▷ -create_Siegel_transformation "1,0,0,0,0,0" "0,0,1,0,0,0" \
10167 ▷ ▷ -end \
10168 ▷ ▷ -define G -linear_group -PGOp 6 F -end \
10169 ▷ ▷ -define siegel1 -vector_ge \
10170 ▷ ▷ ▷ -action G \
10171 ▷ ▷ ▷ -read_csv 0.1.6.2.Siegel.transformation.csv Element \
10172 ▷ ▷ -end \
10173 ▷ ▷ -with G -do \
10174 ▷ ▷ -group_theoretic_activity \
10175 ▷ ▷ ▷ -element_processing \
10176 ▷ ▷ ▷ ▷ -input siegel1 \
10177 ▷ ▷ ▷ ▷ -print \
10178 ▷ ▷ ▷ -end \

```

```

10179 ▷ ▷ -end
10180 ▷ $(PDFLATEX) siegel1.elements.tex
10181 ▷ $(OPEN) siegel1.elements.pdf
10182
10183
10184 #0.1.6.2.Siegel.transformation.csv
10185
10186
10187
10188 Op_6.2.create_Siegel.transformation_2C:
10189 ▷ $(ORBITER) -v 2 \
10190 ▷ ▷ -define F -finite_field -q 2 -end \
10191 ▷ ▷ -define O -orthogonal_space 1 6 F -end \
10192 ▷ ▷ -with O -do -orthogonal_space_activity \
10193 ▷ ▷ ▷ -create_Siegel_transformation "1,0,0,0,0,0" "0,0,1,1,0,0" \
10194 ▷ ▷ -end \
10195 ▷ ▷ -define G -linear_group -PGOp 6 F -end \
10196 ▷ ▷ -define siegel2 -vector_ge \
10197 ▷ ▷ ▷ -action G \
10198 ▷ ▷ ▷ -read_csv 0.1.6.2.Siegel.transformation.csv Element \
10199 ▷ ▷ -end \
10200 ▷ ▷ -with G -do \
10201 ▷ ▷ -group_theoretic_activity \
10202 ▷ ▷ ▷ -element_processing \
10203 ▷ ▷ ▷ ▷ -input siegel2 \
10204 ▷ ▷ ▷ ▷ -print \
10205 ▷ ▷ ▷ -end \
10206 ▷ ▷ -end
10207 ▷ $(PDFLATEX) siegel2.elements.tex
10208 ▷ $(OPEN) siegel2.elements.pdf
10209
10210
10211 #0.1.6.2.Siegel.transformation.csv
10212
10213
10214
10215 Op_6.2.report_Siegel.transformation:
10216 ▷ $(ORBITER) -v 2 \
10217 ▷ ▷ -define Pts -vector \
10218 ▷ ▷ ▷ -dense $(ORTHOGONAL_OP_6.2.POINTS.OFF) \
10219 ▷ ▷ -end \
10220 ▷ ▷ -define F -finite_field -q 2 -end \
10221 ▷ ▷ -define G -linear_group -PGOp 6 F -end \
10222 ▷ ▷ -define Siegel -vector_ge \
10223 ▷ ▷ ▷ -action G \
10224 ▷ ▷ ▷ -read_csv 0.1.6.2.Siegel.transformation.csv Element \
10225 ▷ ▷ -end \
10226 ▷ ▷ -with Siegel -do \
10227 ▷ ▷ -vector_ge_activity \
10228 ▷ ▷ ▷ -report_with_options "dense=on" \
10229 ▷ ▷ -end
10230 ▷ $(PDFLATEX) PGOp_6.2.Siegel.elements.tex
10231 ▷ $(OPEN) PGOp_6.2.Siegel.elements.pdf
10232
10233
10234 Op_6.2.Siegel.transformation_expand_class_2A:
10235 ▷ $(ORBITER) -v 2 \
10236 ▷ ▷ -define Pts -vector \
10237 ▷ ▷ ▷ -dense $(ORTHOGONAL_OP_6.2.POINTS.OFF) \

```

```

10238 ▷ ▷ -end \
10239 ▷ ▷ -define F -finite_field -q 2 -end \
10240 ▷ ▷ -define G -linear_group -PGOp 6 F -end \
10241 ▷ ▷ -define Siegel -vector_ge \
10242 ▷ ▷ ▷ -action G \
10243 ▷ ▷ ▷ -read_csv 0.1.6.2.Siegel.transformation.csv Element \
10244 ▷ ▷ -end \
10245 ▷ ▷ -with G -do \
10246 ▷ ▷ -group_theoretic_activity \
10247 ▷ ▷ ▷ -conjugacy_class_of "Siegel" \
10248 ▷ ▷ ▷ $(SIEGEL_TRANSFORM_UV_2A) \
10249 ▷ ▷ -end
10250 ▷ $(PDFLATEX) PGOp_6.2.Siegel.elements.tex
10251 ▷ $(OPEN) PGOp_6.2.Siegel.elements.pdf
10252
10253
10254
10255 #PGOp_6.2.class_of.Siegel.csv
10256 # 105 elements, so class 2B
10257
10258
10259 Op_6.2.Siegel.transformation.expand.class.2C:
10260 ▷ $(ORBITER) -v 2 \
10261 ▷ ▷ -define Pts -vector \
10262 ▷ ▷ ▷ -dense $(ORTHOGONAL_OP_6.2.POINTS.OFF) \
10263 ▷ ▷ -end \
10264 ▷ ▷ -define F -finite_field -q 2 -end \
10265 ▷ ▷ -define G -linear_group -PGOp 6 F -end \
10266 ▷ ▷ -define Siegel -vector_ge \
10267 ▷ ▷ ▷ -action G \
10268 ▷ ▷ ▷ -read_csv 0.1.6.2.Siegel.transformation.csv Element \
10269 ▷ ▷ -end \
10270 ▷ ▷ -with G -do \
10271 ▷ ▷ -group_theoretic_activity \
10272 ▷ ▷ ▷ -conjugacy_class_of "Siegel" \
10273 ▷ ▷ ▷ $(SIEGEL_TRANSFORM_UV_2C) \
10274 ▷ ▷ -end
10275 ▷ #$(PDFLATEX) PGOp_6.2.Siegel.elements.tex
10276 ▷ #$(OPEN) PGOp_6.2.Siegel.elements.pdf
10277
10278 #PGOp_6.2.class_of.Siegel.csv
10279 # class size 210, so 2C
10280
10281 Op_6.2.make_all.Siegel.transformations:
10282 ▷ $(ORBITER) -v 2 \
10283 ▷ ▷ -define Pts -vector \
10284 ▷ ▷ ▷ -dense $(ORTHOGONAL_OP_6.2.POINTS.OFF) \
10285 ▷ ▷ -end \
10286 ▷ ▷ -define F -finite_field -q 2 -end \
10287 ▷ ▷ -define O -orthogonal_space 1 6 F -end \
10288 ▷ ▷ -with O -do -orthogonal_space_activity \
10289 ▷ ▷ ▷ -make_all.Siegel.transformations \
10290 ▷ ▷ -end
10291
10292
10293 # all_Siegel.csv
10294
10295 Op_6.2.all.Siegel.transformations.tally:
10296 ▷ $(ORBITER) -v 2 \

```

```

10297 ▷ ▷ -tally_column all_Siegel.csv transform
10298
10299
10300
10301
10302 #####
10303 # Section 7.4: Hermitian varieties
10304
10305
10306 SECTION.HERMITIAN.VARIETIES:
10307
10308 test_7.4:
10309 ▷ make H_2.4
10310 ▷ make H_2.9
10311 ▷ make H_3.4
10312
10313
10314 H_2.4:
10315 ▷ $(ORBITER) -v 2 \
10316 ▷ ▷ -define F -finite_field -q 4 -end \
10317 ▷ ▷ -with F -do -finite_field_activity \
10318 ▷ ▷ ▷ -cheat_sheet_hermitian 2 \
10319 ▷ ▷ -end
10320 ▷ $(PDFLATEX) H_2.4.tex
10321 ▷ $(OPEN) H_2.4.pdf
10322
10323
10324 H_2.9:
10325 ▷ $(ORBITER) -v 2 \
10326 ▷ ▷ -define F -finite_field -q 9 -end \
10327 ▷ ▷ -with F -do -finite_field_activity \
10328 ▷ ▷ ▷ -cheat_sheet_hermitian 2 \
10329 ▷ ▷ -end
10330 ▷ $(PDFLATEX) H_2.9.tex
10331 ▷ $(OPEN) H_2.9.pdf
10332
10333
10334 # 28 points: 6, 11, 9, 7, 14, 19, 17, 15, 80, 75, 78, 74, 35, 30, 33, 29, 62, 57,
        60, 56, 26, 21, 24, 3, 37, 82, 64, 46
10335
10336
10337 H_3.4:
10338 ▷ $(ORBITER) -v 2 \
10339 ▷ ▷ -define F -finite_field -q 4 -end \
10340 ▷ ▷ -with F -do -finite_field_activity \
10341 ▷ ▷ ▷ -cheat_sheet_hermitian 3 \
10342 ▷ ▷ -end
10343 ▷ $(PDFLATEX) H_3.4.tex
10344 ▷ $(OPEN) H_3.4.pdf
10345
10346
10347 # H_3.4 = the Hirschfeld surface
10348
10349
10350
10351
10352
10353 #####
10354 # Section 7.5: Algebraic Varieties

```

```

10355
10356 test_7.5:
10357 ▷ make conic_variety_q13
10358 ▷ make Op_6_2_variety
10359 ▷ make curve_q13.f0
10360 ▷ make Hirschfeld.q2
10361 ▷ make quartic_curve_q13.f1.apply_transformation
10362 ▷ make quartic_curve_q13.f0.apply_transformation
10363 ▷ make cubic_eqn.q3_0
10364 ▷
10365
10366
10367
10368
10369 conic_variety_q13:
10370 ▷ $(ORBITER) -v 6 \
10371 ▷ ▷ -define F -finite_field -q 13 -end \
10372 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
10373 ▷ ▷ -define R -polynomial_ring \
10374 ▷ ▷ ▷ -field F \
10375 ▷ ▷ ▷ -number_of_variables 3 \
10376 ▷ ▷ ▷ -homogeneous_of_degree 2 \
10377 ▷ ▷ ▷ -monomial_ordering_partition \
10378 ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
10379 ▷ ▷ -end \
10380 ▷ ▷ -define V -variety \
10381 ▷ ▷ ▷ -projective_space P \
10382 ▷ ▷ ▷ -ring R \
10383 ▷ ▷ ▷ -equation_in_algebraic_form \
10384 ▷ ▷ ▷ ▷ "X0^2-X1*X2" \
10385 ▷ ▷ ▷ -label_txt conic \
10386 ▷ ▷ ▷ -label_tex "{\\rm conic}" \
10387 ▷ ▷ -end
10388
10389
10390
10391 Op_6_2_variety:
10392 ▷ $(ORBITER) -v 6 \
10393 ▷ ▷ -define F -finite_field -q 2 -end \
10394 ▷ ▷ -define P -projective_space -n 5 -field F -v 0 -end \
10395 ▷ ▷ -define R -polynomial_ring \
10396 ▷ ▷ ▷ -field F \
10397 ▷ ▷ ▷ -number_of_variables 6 \
10398 ▷ ▷ ▷ -homogeneous_of_degree 2 \
10399 ▷ ▷ ▷ -monomial_ordering_partition \
10400 ▷ ▷ ▷ -variables "X0,X1,X2,X3,X4,X5" "X_0,X_1,X_2,X_3,X_4,X_5" \
10401 ▷ ▷ -end \
10402 ▷ ▷ -define V -variety \
10403 ▷ ▷ ▷ -projective_space P \
10404 ▷ ▷ ▷ -ring R \
10405 ▷ ▷ ▷ -equation_in_algebraic_form \
10406 ▷ ▷ ▷ ▷ $(Op_6_eqn) \
10407 ▷ ▷ ▷ -label_txt Op_6_2 \
10408 ▷ ▷ ▷ -label_tex "{\\rm Op\\_6\\_2}" \
10409 ▷ ▷ -end
10410
10411
10412
10413

```



```

10414 curve_q13_f0:
10415 ▷ $(ORBITER) -v 6 \
10416 ▷ ▷ -define F -finite_field -q 13 -end \
10417 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
10418 ▷ ▷ -define R -polynomial_ring \
10419 ▷ ▷ ▷ -field F \
10420 ▷ ▷ ▷ -number_of_variables 3 \
10421 ▷ ▷ ▷ -homogeneous_of_degree 4 \
10422 ▷ ▷ ▷ -monomial_ordering_partition \
10423 ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
10424 ▷ ▷ ▷ -end \
10425 ▷ ▷ -define V -variety \
10426 ▷ ▷ ▷ -projective_space P \
10427 ▷ ▷ ▷ -ring R \
10428 ▷ ▷ ▷ -equation_in_algebraic_form \
10429 ▷ ▷ ▷ ▷ $(CURVE_Q13_F0_CLEAN2_EQUATION) \
10430 ▷ ▷ ▷ -label_txt curve_q13_f0_clean2 \
10431 ▷ ▷ ▷ -label_tex curve\_q13\_f0\_clean2 \
10432 ▷ ▷ -end
10433
10434
10435 Hirschfeld.q2:
10436 ▷ $(ORBITER) -v 6 \
10437 ▷ ▷ -define F -finite_field -q 2 -end \
10438 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10439 ▷ ▷ -define R -polynomial_ring \
10440 ▷ ▷ ▷ -field F \
10441 ▷ ▷ ▷ -number_of_variables 4 \
10442 ▷ ▷ ▷ -homogeneous_of_degree 3 \
10443 ▷ ▷ ▷ -monomial_ordering_partition \
10444 ▷ ▷ ▷ -variables "X,Y,Z,W" "X,Y,Z,W" \
10445 ▷ ▷ ▷ -end \
10446 ▷ ▷ -define V -variety \
10447 ▷ ▷ ▷ -label_txt Hirschfeld.q2 \
10448 ▷ ▷ ▷ -label_tex Hirschfeld\_q2 \
10449 ▷ ▷ ▷ -projective_space P \
10450 ▷ ▷ ▷ -ring R \
10451 ▷ ▷ ▷ -equation_in_algebraic_form \
10452 ▷ ▷ ▷ ▷ $(HIRSCHFELD_EQUATION_XYZW) \
10453 ▷ ▷ -end
10454
10455
10456
10457 quartic_curve_q13_f1_apply_transformation:
10458 ▷ $(ORBITER) -v 10 \
10459 ▷ ▷ -define F -finite_field -q 13 -end \
10460 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
10461 ▷ ▷ -define R -polynomial_ring \
10462 ▷ ▷ ▷ -field F \
10463 ▷ ▷ ▷ -number_of_variables 3 \
10464 ▷ ▷ ▷ -homogeneous_of_degree 4 \
10465 ▷ ▷ ▷ -monomial_ordering_partition \
10466 ▷ ▷ ▷ -variables "x1,x2,x3" "x_1,x_2,x_3" \
10467 ▷ ▷ ▷ -end \
10468 ▷ ▷ -define V_q13_f1 -variety \
10469 ▷ ▷ ▷ -label_txt curve_q13_f1_clean2 \
10470 ▷ ▷ ▷ -label_tex curve\_q13\_f1\_clean2 \
10471 ▷ ▷ ▷ -projective_space P \
10472 ▷ ▷ ▷ -ring R \

```

```

10473 ▷ ▷ ▷ -equation_in_algebraic_form $(CURVE_Q13_FLAG_1_EQN) \
10474 ▷ ▷ ▷ -points $(CURVE_Q13_FLAG_1_POINTS) \
10475 ▷ ▷ ▷ -bitangents $(CURVE_Q13_FLAG_1_BITANGENTS) \
10476 ▷ ▷ ▷ -transform $(CURVE_Q13_FLAG_1_TRANS_1) \
10477 ▷ ▷ ▷ -transform $(CURVE_Q13_FLAG_1_TRANS_2) \
10478 ▷ ▷ -end
10479
10480
10481
10482
10483 quartic_curve_q13.f0.apply_transformation:
10484 ▷ $(ORBITER) -v 6 \
10485 ▷ ▷ -define F -finite_field -q 13 -end \
10486 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
10487 ▷ ▷ -define R -polynomial_ring \
10488 ▷ ▷ ▷ -field F \
10489 ▷ ▷ ▷ -number_of_variables 3 \
10490 ▷ ▷ ▷ -homogeneous_of_degree 4 \
10491 ▷ ▷ ▷ -monomial_ordering_partition \
10492 ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
10493 ▷ ▷ -end \
10494 ▷ ▷ -define eqn -vector -dense \
10495 ▷ ▷ ▷ $(QUARTIC_CURVE_Q13_FLAG_0_EQN_COEFFICIENTS) \
10496 ▷ ▷ ▷ -end \
10497 ▷ ▷ -define trans_coeffs -vector -dense \
10498 ▷ ▷ ▷ $(QUARTIC_CURVE_Q13_FLAG_0_TRANS) \
10499 ▷ ▷ ▷ -end \
10500 ▷ ▷ -define G -linear_group -PGL 3 F -end \
10501 ▷ ▷ -define trans -vector_ge \
10502 ▷ ▷ ▷ -action G \
10503 ▷ ▷ ▷ -vector_data trans_coeffs \
10504 ▷ ▷ -end \
10505 ▷ ▷ -with R -do \
10506 ▷ ▷ ▷ -ring_theoretic_activity \
10507 ▷ ▷ ▷ -apply_transformation P eqn trans \
10508 ▷ ▷ -end
10509
10510
10511
10512 cubic_eqn_q3_0:
10513 ▷ $(ORBITER) -v 6 \
10514 ▷ ▷ -define F -finite_field -q 3 -end \
10515 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10516 ▷ ▷ -define R -polynomial_ring \
10517 ▷ ▷ ▷ -field F \
10518 ▷ ▷ ▷ -number_of_variables 4 \
10519 ▷ ▷ ▷ -homogeneous_of_degree 3 \
10520 ▷ ▷ ▷ -monomial_ordering_partition \
10521 ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
10522 ▷ ▷ -end \
10523 ▷ ▷ -define V -variety \
10524 ▷ ▷ ▷ -projective_space P \
10525 ▷ ▷ ▷ -ring R \
10526 ▷ ▷ ▷ -equation_by_rank \
10527 ▷ ▷ ▷ ▷ 0 \
10528 ▷ ▷ ▷ -compute_lines \
10529 ▷ ▷ ▷ -label_txt eqn_q3_0 \
10530 ▷ ▷ ▷ -label_tex "{\\rm eqn\\_q3\\_0}" \
10531 ▷ ▷ -end \

```

```

10532 ▷ ▷ -with V -do -variety_activity -compute_group \
10533 ▷ ▷ ▷ -nauty_control -save_orbit_of_equations eqn_ -end \
10534 ▷ ▷ -end \
10535 ▷ ▷ -with V -do -variety_activity -singular_points -end \
10536 ▷ ▷ -with V -do -variety_activity -report -end
10537 ▷ $(PDFLATEX) variety_eqn.q3.0.report.tex
10538 ▷ $(OPEN) variety_eqn.q3.0.report.pdf
10539
10540
10541
10542
10543 #####
10544 # Section 7.6: Activities of Varieties
10545
10546
10547
10548 test_7.6:
10549 ▷ make conic_variety_q13_report
10550 ▷ make Op_6_2_group
10551 ▷ make curve_q13_f0_clean2_variety_compute_group
10552 ▷ make Hirschfeld_q2_group
10553 ▷ make Hirschfeld_q4_group
10554 ▷ make Barth_sextic_mod_11
10555 ▷ make variety_Dickson36_9lines_q4_group
10556 ▷ make quartic_curve_test_isomorphism
10557 ▷ make quartic_curve_q27_fo159_fo338_test_isomorphism
10558 ▷ make quartic_curve_q9_hermitian_fo0_test_isomorphism
10559 ▷ make quartic_curve_q9_hermitian_and_OCN0_test_isomorphism
10560 ▷ make quartic_curve_q9_Hermitian_transform
10561
10562
10563
10564 conic_variety_q13_report:
10565 ▷ $(ORBITER) -v 6 \
10566 ▷ ▷ -define F -finite_field -q 13 -end \
10567 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
10568 ▷ ▷ -define R -polynomial_ring \
10569 ▷ ▷ ▷ -field F \
10570 ▷ ▷ ▷ -number_of_variables 3 \
10571 ▷ ▷ ▷ -homogeneous_of_degree 2 \
10572 ▷ ▷ ▷ -monomial_ordering_partition \
10573 ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
10574 ▷ ▷ -end \
10575 ▷ ▷ -define V -variety \
10576 ▷ ▷ ▷ -projective_space P \
10577 ▷ ▷ ▷ -ring R \
10578 ▷ ▷ ▷ -equation_in_algebraic_form \
10579 ▷ ▷ ▷ ▷ "X0^2-X1*X2" \
10580 ▷ ▷ ▷ -label.txt conic \
10581 ▷ ▷ ▷ -label.tex "{\\rm conic}" \
10582 ▷ ▷ -end \
10583 ▷ ▷ -with V -do -variety_activity -report -end
10584 ▷ $(PDFLATEX) variety_conic_report.tex
10585 ▷ $(OPEN) variety_conic_report.pdf
10586
10587
10588
10589 Op_6_2_group:
10590 ▷ $(ORBITER) -v 6 \

```

```

10591 ▷ ▷ -define F -finite_field -q 2 -end \
10592 ▷ ▷ -define P -projective_space -n 5 -field F -v 0 -end \
10593 ▷ ▷ -define R -polynomial_ring \
10594 ▷ ▷ ▷ -field F \
10595 ▷ ▷ ▷ -number_of_variables 6 \
10596 ▷ ▷ ▷ -homogeneous_of_degree 2 \
10597 ▷ ▷ ▷ -monomial_ordering_partition \
10598 ▷ ▷ ▷ -variables "X0,X1,X2,X3,X4,X5" \
10599 ▷ ▷ ▷ ▷ "X_0,X_1,X_2,X_3,X_4,X_5" \
10600 ▷ ▷ -end \
10601 ▷ ▷ -define V -variety \
10602 ▷ ▷ ▷ -projective_space P \
10603 ▷ ▷ ▷ -ring R \
10604 ▷ ▷ ▷ -equation_in_algebraic_form \
10605 ▷ ▷ ▷ ▷ $(Op_6_eqn) \
10606 ▷ ▷ ▷ -label_txt Op_6.2 \
10607 ▷ ▷ ▷ -label_tex "${\rm Op}_6.2}" \
10608 ▷ ▷ -end \
10609 ▷ ▷ -with V -do -variety_activity -compute_group \
10610 ▷ ▷ ▷ -nauty_control -save_orbit_of_equations eqn_ -nauty_log "Op6.2_nauty_log.tx
t" -end \
10611 ▷ ▷ -end \
10612 ▷ ▷ -with V -do -variety_activity -compute_set_stabilizer \
10613 ▷ ▷ ▷ -nauty_control -save_orbit_of_equations eqn_ -end \
10614 ▷ ▷ -end \
10615 ▷ ▷ -with V -do -variety_activity -singular_points -end \
10616 ▷ ▷ -with V -do -variety_activity -export -end \
10617 ▷ ▷ -with V -do -variety_activity -report -end
10618 ▷ $(PDFLATEX) variety_Op_6.2_report.tex
10619 ▷ $(OPEN) variety_Op_6.2_report.pdf
10620 ▷ $(ORBITER) -v 2 \
10621 ▷ ▷ -define Do -draw_options -embedded -radius 150 \
10622 ▷ ▷ ▷ -nodes_empty \
10623 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
10624 ▷ ▷ -tree_draw \
10625 ▷ ▷ ▷ -file Op6.2_nauty_log.txt \
10626 ▷ ▷ ▷ -draw_options Do \
10627 ▷ ▷ -end
10628 ▷ $(PDFLATEX) Op6.2_nauty_log_draw.tex
10629 ▷ $(OPEN) Op6.2_nauty_log_draw.pdf
10630
10631
10632 #Op6.2_nauty_log.txt
10633
10634
10635 curve_q13_f0_clean2_variety_compute_group:
10636 ▷ $(ORBITER) -v 6 \
10637 ▷ ▷ -define F -finite_field -q 13 -end \
10638 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
10639 ▷ ▷ -define R -polynomial_ring \
10640 ▷ ▷ ▷ -field F \
10641 ▷ ▷ ▷ -number_of_variables 3 \
10642 ▷ ▷ ▷ -homogeneous_of_degree 4 \
10643 ▷ ▷ ▷ -monomial_ordering_partition \
10644 ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
10645 ▷ ▷ -end \
10646 ▷ ▷ -define V -variety \
10647 ▷ ▷ ▷ -projective_space P \
10648 ▷ ▷ ▷ -ring R \

```

```

10649 ▷ ▷ ▷ -equation_in_algebraic_form \
10650 ▷ ▷ ▷ ▷ $(CURVE.Q13.F0.CLEAN2.EQUATION) \
10651 ▷ ▷ ▷ -label.txt curve_q13.f0_clean2 \
10652 ▷ ▷ ▷ -label.tex "{\rm curve\q13\fo\clean2}" \
10653 ▷ ▷ -end \
10654 ▷ ▷ -with V -do -variety_activity \
10655 ▷ ▷ ▷ -compute_group -nauty_control -end \
10656 ▷ ▷ -end
10657 ▷ $(PDFLATEX) variety_curve_q13.f0_clean2.report.tex
10658 ▷ $(OPEN) variety_curve_q13.f0_clean2.report.pdf
10659
10660
10661 Hirschfeld.q2.group:
10662 ▷ $(ORBITER) -v 6 \
10663 ▷ ▷ -define F -finite_field -q 2 -end \
10664 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10665 ▷ ▷ -define R -polynomial_ring \
10666 ▷ ▷ ▷ -field F \
10667 ▷ ▷ ▷ -number_of_variables 4 \
10668 ▷ ▷ ▷ -homogeneous_of_degree 3 \
10669 ▷ ▷ ▷ -monomial_ordering_partition \
10670 ▷ ▷ ▷ -variables "X,Y,Z,W" "X,Y,Z,W" \
10671 ▷ ▷ -end \
10672 ▷ ▷ -define V -variety \
10673 ▷ ▷ ▷ -label.txt Hirschfeld.q2 \
10674 ▷ ▷ ▷ -label.tex Hirschfeld\q2 \
10675 ▷ ▷ ▷ -projective_space P \
10676 ▷ ▷ ▷ -ring R \
10677 ▷ ▷ ▷ -equation_in_algebraic_form \
10678 ▷ ▷ ▷ ▷ $(HIRSCHFELD.EQUATION.XYZW) \
10679 ▷ ▷ -end \
10680 ▷ ▷ -with V -do -variety_activity \
10681 ▷ ▷ ▷ -compute_group -nauty_control -end \
10682 ▷ ▷ -end \
10683 ▷ ▷ -with V -do -variety_activity -singular_points -end \
10684 ▷ ▷ -with V -do -variety_activity -report -end
10685 ▷ $(PDFLATEX) variety_Hirschfeld.q2.report.tex
10686 ▷ $(OPEN) variety_Hirschfeld.q2.report.pdf
10687
10688
10689 Hirschfeld.q4.group:
10690 ▷ $(ORBITER) -v 6 \
10691 ▷ ▷ -define F -finite_field -q 4 -end \
10692 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10693 ▷ ▷ -define R -polynomial_ring \
10694 ▷ ▷ ▷ -field F \
10695 ▷ ▷ ▷ -number_of_variables 4 \
10696 ▷ ▷ ▷ -homogeneous_of_degree 3 \
10697 ▷ ▷ ▷ -monomial_ordering_partition \
10698 ▷ ▷ ▷ -variables "X,Y,Z,W" "X,Y,Z,W" \
10699 ▷ ▷ -end \
10700 ▷ ▷ -define V -variety \
10701 ▷ ▷ ▷ -label.txt Hirschfeld.q4 \
10702 ▷ ▷ ▷ -label.tex Hirschfeld\q4 \
10703 ▷ ▷ ▷ -projective_space P \
10704 ▷ ▷ ▷ -ring R \
10705 ▷ ▷ ▷ -equation_in_algebraic_form \
10706 ▷ ▷ ▷ ▷ $(HIRSCHFELD.EQUATION.XYZW) \
10707 ▷ ▷ -end \

```

```

10708 ▷ ▷ -with V -do -variety_activity \
10709 ▷ ▷ ▷ -compute_group -nauty_control -end \
10710 ▷ ▷ -end \
10711 ▷ ▷ -with V -do -variety_activity -singular_points -end \
10712 ▷ ▷ -with V -do -variety_activity -report -end
10713 ▷ $(PDFLATEX) variety_Hirschfeld.q4.report.tex
10714 ▷ $(OPEN) variety_Hirschfeld.q4.report.pdf
10715
10716
10717
10718 Barth_sextic_mod.11:
10719 ▷ $(ORBITER) -v 10 \
10720 ▷ ▷ -define F -finite_field -q 11 -end \
10721 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10722 ▷ ▷ -define R -polynomial_ring \
10723 ▷ ▷ ▷ -field F \
10724 ▷ ▷ ▷ -number_of_variables 4 \
10725 ▷ ▷ ▷ -homogeneous_of_degree 6 \
10726 ▷ ▷ ▷ -monomial_ordering_partition \
10727 ▷ ▷ ▷ -variables "X,Y,Z,W" "X,Y,Z,W" \
10728 ▷ ▷ -end \
10729 ▷ ▷ -define V -variety \
10730 ▷ ▷ ▷ -projective_space P \
10731 ▷ ▷ ▷ -ring R \
10732 ▷ ▷ ▷ -equation_in_algebraic_form \
10733 ▷ ▷ ▷ ▷ $(BARTH_EQUATION_GENERAL) \
10734 ▷ ▷ ▷ -set_parameters "PHI" "\Phi" "8" \
10735 ▷ ▷ ▷ -label.txt Barth.q11 \
10736 ▷ ▷ ▷ -label.tex Barth\_q11 \
10737 ▷ ▷ -end \
10738 ▷ ▷ -with V -do -variety_activity \
10739 ▷ ▷ ▷ -compute_group -nauty_control -end \
10740 ▷ ▷ -end \
10741 ▷ ▷ -with V -do -variety_activity -singular_points -end \
10742 ▷ ▷ -with V -do -variety_activity -report -end
10743 ▷ $(PDFLATEX) variety_Barth.q11.report.tex
10744 ▷ $(OPEN) variety_Barth.q11.report.pdf
10745
10746
10747
10748 variety_Dickson36.9lines.q4.group:
10749 ▷ $(ORBITER) -v 6 \
10750 ▷ ▷ -define F -finite_field -q 4 -end \
10751 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
10752 ▷ ▷ -define R -polynomial_ring \
10753 ▷ ▷ ▷ -field F \
10754 ▷ ▷ ▷ -number_of_variables 4 \
10755 ▷ ▷ ▷ -homogeneous_of_degree 3 \
10756 ▷ ▷ ▷ -monomial_ordering_partition \
10757 ▷ ▷ ▷ -variables "X0,X1,X2,X3" \
10758 ▷ ▷ ▷ ▷ "X_0,X_1,X_2,X_3" \
10759 ▷ ▷ -end \
10760 ▷ ▷ -define V -variety \
10761 ▷ ▷ ▷ -projective_space P \
10762 ▷ ▷ ▷ -ring R \
10763 ▷ ▷ ▷ -equation_in_algebraic_form \
10764 ▷ ▷ ▷ ▷ " $X_0^3+X_1^3+X_2^3+X_3^3+X_0^2X_3+X_1^2X_2$ " \
10765 ▷ ▷ ▷ -compute_lines \
10766 ▷ ▷ ▷ -label.txt Dickson36 \

```

```

10767 ▷ ▷ ▷ -label.tex "{\\rm Dickson36}" \
10768 ▷ ▷ -end \
10769 ▷ ▷ -with V -do -variety_activity -compute_group \
10770 ▷ ▷ ▷ -nauty_control -save_orbit_of_equations eqn_ -end \
10771 ▷ ▷ -end \
10772 ▷ ▷ -with V -do -variety_activity -compute_set_stabilizer \
10773 ▷ ▷ ▷ -nauty_control -save_orbit_of_equations eqn_ -end \
10774 ▷ ▷ -end \
10775 ▷ ▷ -with V -do -variety_activity -singular_points -end \
10776 ▷ ▷ -with V -do -variety_activity -export -end \
10777 ▷ ▷ -with V -do -variety_activity -report -end
10778 ▷ $(PDFLATEX) variety_Dickson36_report.tex
10779 ▷ $(OPEN) variety_Dickson36_report.pdf
10780
10781
10782 quartic_curve_test_isomorphism:
10783 ▷ $(ORBITER) -v 6 \
10784 ▷ ▷ -define F -finite_field -q 9 -end \
10785 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
10786 ▷ ▷ -define R -polynomial_ring \
10787 ▷ ▷ ▷ -field F \
10788 ▷ ▷ ▷ -number_of_variables 3 \
10789 ▷ ▷ ▷ -homogeneous_of_degree 4 \
10790 ▷ ▷ ▷ -monomial_ordering_partition \
10791 ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
10792 ▷ ▷ -end \
10793 ▷ ▷ -define V1 -variety \
10794 ▷ ▷ ▷ -label.txt curve.q9 \
10795 ▷ ▷ ▷ -label.tex curve\_q9 \
10796 ▷ ▷ ▷ -projective_space P \
10797 ▷ ▷ ▷ -ring R \
10798 ▷ ▷ ▷ -equation_by_coefficients $(QC_Q9_UNITAL_COEFFICIENTS) \
10799 ▷ ▷ -end \
10800 ▷ ▷ -define V2 -variety \
10801 ▷ ▷ ▷ -label.txt curve.q9_nf \
10802 ▷ ▷ ▷ -label.tex curve\_q9\_nf \
10803 ▷ ▷ ▷ -projective_space P \
10804 ▷ ▷ ▷ -ring R \
10805 ▷ ▷ ▷ -equation_by_coefficients $(QUARTIC_Q9_NF_COEFFICIENTS) \
10806 ▷ ▷ -end \
10807 ▷ ▷ -with V2 -and V1 -do -variety_activity \
10808 ▷ ▷ ▷ -test_isomorphism \
10809 ▷ ▷ ▷ -nauty_control -nauty_log "nauty_log.txt" -end \
10810 ▷ ▷ -end \
10811 ▷ ▷ -stats "iso.q9"
10812 ▷ $(ORBITER) -v 2 \
10813 ▷ ▷ -define Do -draw_options -embedded -radius 150 \
10814 ▷ ▷ ▷ -nodes_empty \
10815 ▷ ▷ ▷ -scale 0.5 -line_width 1.3 \
10816 ▷ ▷ ▷ -xin 10000 \
10817 ▷ ▷ ▷ -yin 10000 \
10818 ▷ ▷ ▷ -xout 1000000 \
10819 ▷ ▷ ▷ -yout 500000 \
10820 ▷ ▷ -end \
10821 ▷ ▷ -tree_draw \
10822 ▷ ▷ -file nauty_log.txt \
10823 ▷ ▷ -line_width 100 \
10824 ▷ ▷ ▷ -draw_options Do \
10825 ▷ ▷ -end

```

```

10826 ▷ $(PDFLATEX) nauty_log_draw.tex
10827 ▷ $(OPEN) nauty_log_draw.pdf
10828
10829 # from V1 to V2
10830 #classification.of.varieties.nauty::handle_one_input_case an isomorphism is given
      by phi=1,5,5,5,3,8,5,0,2,0
10831 #nauty_log.txt
10832
10833
10834 quartic_curve_q27_fo159_fo338_test_isomorphism:
10835 ▷ $(ORBITER) -v 12 \
10836 ▷ ▷ -define F -finite_field -q 27 -end \
10837 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
10838 ▷ ▷ -define R -polynomial_ring \
10839 ▷ ▷ ▷ -field F \
10840 ▷ ▷ ▷ -number_of_variables 3 \
10841 ▷ ▷ ▷ -homogeneous_of_degree 4 \
10842 ▷ ▷ ▷ -monomial_ordering_partition \
10843 ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
10844 ▷ ▷ -end \
10845 ▷ ▷ -define V159 -variety \
10846 ▷ ▷ ▷ -label.txt curve_q27_fo159 \
10847 ▷ ▷ ▷ -label.tex curve\_q27\_fo159 \
10848 ▷ ▷ ▷ -projective_space P \
10849 ▷ ▷ ▷ -ring R \
10850 ▷ ▷ ▷ -equation_by_coefficients $(QC_Q27_F0159_COEFFS) \
10851 ▷ ▷ -end \
10852 ▷ ▷ -define V388 -variety \
10853 ▷ ▷ ▷ -label.txt curve_q27_fo388 \
10854 ▷ ▷ ▷ -label.tex curve\_q27\_fo388 \
10855 ▷ ▷ ▷ -projective_space P \
10856 ▷ ▷ ▷ -ring R \
10857 ▷ ▷ ▷ -equation_by_coefficients $(QC_Q27_F0388_COEFFS) \
10858 ▷ ▷ -end \
10859 ▷ ▷ -with V159 -and V388 -do -variety_activity \
10860 ▷ ▷ ▷ -test_isomorphism \
10861 ▷ ▷ ▷ -nauty_control -nauty_log "quartic_curve_q27_nauty_log.txt" -end \
10862 ▷ ▷ -end
10863 ▷ $(ORBITER) -v 2 \
10864 ▷ ▷ -define Do -draw_options -embedded -radius 150 \
10865 ▷ ▷ ▷ -nodes_empty \
10866 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
10867 ▷ ▷ -tree_draw \
10868 ▷ ▷ ▷ -file quartic_curve_q27_nauty_log.txt \
10869 ▷ ▷ ▷ -draw_options Do \
10870 ▷ ▷ -end
10871 ▷ $(PDFLATEX) quartic_curve_q27_nauty_log_draw.tex
10872 ▷ $(OPEN) quartic_curve_q27_nauty_log_draw.pdf
10873
10874
10875 #quartic_curve_q27_nauty_log.txt
10876
10877
10878
10879 quartic_curve_q9_hermitian_fo0_test_isomorphism:
10880 ▷ $(ORBITER) -v 12 \
10881 ▷ ▷ -define F -finite_field -q 9 -end \
10882 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
10883 ▷ ▷ -define R -polynomial_ring \

```



```

10884 ▷ ▷ ▷ -field F \
10885 ▷ ▷ ▷ -number_of_variables 3 \
10886 ▷ ▷ ▷ -homogeneous_of_degree 4 \
10887 ▷ ▷ ▷ -monomial_ordering_partition \
10888 ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
10889 ▷ ▷ -end \
10890 ▷ ▷ -define V0 -variety \
10891 ▷ ▷ ▷ -label.txt curve_q9_fo0 \
10892 ▷ ▷ ▷ -label.tex curve\_q9\_fo0 \
10893 ▷ ▷ ▷ -projective_space P \
10894 ▷ ▷ ▷ -ring R \
10895 ▷ ▷ ▷ -equation_by_coefficients $(QC_Q9_COEFFS) \
10896 ▷ ▷ -end \
10897 ▷ ▷ -define VH -variety \
10898 ▷ ▷ ▷ -label.txt curve_q9_Hermitian \
10899 ▷ ▷ ▷ -label.tex curve\_q9\_Hermitian \
10900 ▷ ▷ ▷ -projective_space P \
10901 ▷ ▷ ▷ -ring R \
10902 ▷ ▷ ▷ -equation_by_coefficients $(QC_Q9_HERMITIAN_COEFFS) \
10903 ▷ ▷ -end \
10904 ▷ ▷ -with VH -and V0 -do -variety_activity \
10905 ▷ ▷ ▷ -test_isomorphism \
10906 ▷ ▷ ▷ -nauty_control -nauty_log "quartic_curve_q9_nauty_log.txt" -end \
10907 ▷ ▷ -end
10908
10909
10910
10911 # 28 points
10912 # group order = 12096
10913
10914
10915 #classification_of_varieties_nauty::handle_one_input_case input_counter = 1 / 2 i
    s isomorphic to canonical form 0 and equation 0
10916 #classification_of_varieties_nauty::handle_one_input_case an isomorphism is given
    by phi=
10917 #5 8 6
10918 #8 1 6
10919 #0 3 6
10920 #, 1
10921
10922 #=
10923 #1 6 4
10924 #6 7 4
10925 #0 8 4
10926
10927
10928
10929 quartic_curve_q9_hermitian_and_OCN0_test_isomorphism:
10930 ▷ $(ORBITER) -v 12 \
10931 ▷ ▷ -define F -finite_field -q 9 -end \
10932 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
10933 ▷ ▷ -define R -polynomial_ring \
10934 ▷ ▷ ▷ -field F \
10935 ▷ ▷ ▷ -number_of_variables 3 \
10936 ▷ ▷ ▷ -homogeneous_of_degree 4 \
10937 ▷ ▷ ▷ -monomial_ordering_partition \
10938 ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
10939 ▷ ▷ -end \
10940 ▷ ▷ -define VH -variety \

```

```

10941 ▷ ▷ ▷ -label.txt curve.q9_Hermitian \
10942 ▷ ▷ ▷ -label.tex curve\_q9\_Hermitian \
10943 ▷ ▷ ▷ -projective_space P \
10944 ▷ ▷ ▷ -ring R \
10945 ▷ ▷ ▷ -equation_by_coefficients $(QC_Q9_HERMITIAN_COEFFS) \
10946 ▷ ▷ -end \
10947 ▷ ▷ -define V2 -variety \
10948 ▷ ▷ ▷ -projective_space P \
10949 ▷ ▷ ▷ -ring R \
10950 ▷ ▷ ▷ -equation_in_algebraic_form \
10951 ▷ ▷ ▷ ▷ $(QC_Q9_ORBITER_CATALOGUE_0_AF) \
10952 ▷ ▷ ▷ -label.txt QC.Q9_0 \
10953 ▷ ▷ ▷ -label.tex "${\rm QC\_Q9\_0}" \
10954 ▷ ▷ -end \
10955 ▷ ▷ -with VH -and V2 -do -variety_activity \
10956 ▷ ▷ ▷ -test_isomorphism \
10957 ▷ ▷ ▷ -nauty_control -nauty_log "quartic_curve_q9_nauty_log.txt" -end \
10958 ▷ ▷ -end
10959
10960
10961 # phi =
10962 #1 4 7
10963 #8 2 5
10964 #0 0 1
10965
10966
10967
10968
10969
10970 quartic_curve_q9_Hermitian_transform:
10971 ▷ $(ORBITER) -v 6 \
10972 ▷ ▷ -define F -finite_field -q 9 -end \
10973 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
10974 ▷ ▷ -define R -polynomial_ring \
10975 ▷ ▷ ▷ -field F \
10976 ▷ ▷ ▷ -number_of_variables 3 \
10977 ▷ ▷ ▷ -homogeneous_of_degree 4 \
10978 ▷ ▷ ▷ -monomial_ordering_partition \
10979 ▷ ▷ ▷ -variables "X1,X2,X3" "X_1,X_2,X_3" \
10980 ▷ ▷ -end \
10981 ▷ ▷ -define V -variety \
10982 ▷ ▷ ▷ -projective_space P \
10983 ▷ ▷ ▷ -ring R \
10984 ▷ ▷ ▷ -equation_in_algebraic_form $(QUARTIC_CURVE_Q9_CLEAN) \
10985 ▷ ▷ ▷ -bitangents $(QC_Q9_CLEAN_LINES) \
10986 ▷ ▷ ▷ -transform $(QC_Q9_TRANSFORM_CLEAN_TO_HERMITIAN) \
10987 ▷ ▷ ▷ -label.txt qc.q9 \
10988 ▷ ▷ ▷ -label.tex "${\rm qc\_q9}" \
10989 ▷ ▷ -end
10990
10991
10992 #####
10993 # Section 7.7 Mappings between Projective Spaces
10994
10995
10996
10997 test_7.7:
10998 ▷ make Cremona_map
10999 ▷ make Veronese_1.3.q7

```

```

11000
11001
11002
11003
11004
11005 Cremona_map:
11006 ▷ $(ORBITER) -v 6 \
11007 ▷ ▷ -define F -finite_field -q 13 -end \
11008 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
11009 ▷ ▷ -define R -polynomial_ring \
11010 ▷ ▷ ▷ -field F \
11011 ▷ ▷ ▷ -number_of_variables 3 \
11012 ▷ ▷ ▷ -homogeneous_of_degree 6 \
11013 ▷ ▷ ▷ -monomial_ordering_lex \
11014 ▷ ▷ ▷ -variables "y0,y1,y2" "y_0,y_1,y_2" \
11015 ▷ ▷ ▷ -end \
11016 ▷ ▷ -define Y0 -symbolic_object \
11017 ▷ ▷ ▷ -field F \
11018 ▷ ▷ ▷ -text $(CREMONA_MAP_Y0) \
11019 ▷ ▷ ▷ -end \
11020 ▷ ▷ -define Y1 -symbolic_object \
11021 ▷ ▷ ▷ -field F \
11022 ▷ ▷ ▷ -text $(CREMONA_MAP_Y1) \
11023 ▷ ▷ ▷ -end \
11024 ▷ ▷ -define Y2 -symbolic_object \
11025 ▷ ▷ ▷ -field F \
11026 ▷ ▷ ▷ -text $(CREMONA_MAP_Y2) \
11027 ▷ ▷ ▷ -end \
11028 ▷ ▷ -define Cremona -symbolic_object \
11029 ▷ ▷ ▷ -field F \
11030 ▷ ▷ ▷ -text "Y0,Y1,Y2" \
11031 ▷ ▷ ▷ -end \
11032 ▷ ▷ -with P -do \
11033 ▷ ▷ -projective_space_activity \
11034 ▷ ▷ ▷ -map R Cremona "" \
11035 ▷ ▷ -end
11036
11037
11038
11039 Veronese_1_3_q7:
11040 ▷ $(ORBITER) -v 5 \
11041 ▷ ▷ -define Fq -finite_field -q 7 -end \
11042 ▷ ▷ -define C -symbolic_object \
11043 ▷ ▷ ▷ -field Fq \
11044 ▷ ▷ ▷ -managed_variables "y0,y1" \
11045 ▷ ▷ ▷ -text "y1^3,y0*y1^2,y0^2*y1,y0^3" \
11046 ▷ ▷ ▷ -end \
11047 ▷ ▷ -define P1 -projective_space -n 1 -field Fq -v 0 -end \
11048 ▷ ▷ -define P3 -projective_space -n 3 -field Fq -v 0 -end \
11049 ▷ ▷ -define R2 -polynomial_ring \
11050 ▷ ▷ ▷ -field Fq \
11051 ▷ ▷ ▷ -number_of_variables 2 \
11052 ▷ ▷ ▷ -homogeneous_of_degree 3 \
11053 ▷ ▷ ▷ -monomial_ordering_partition \
11054 ▷ ▷ ▷ -variables "y0,y1" "y_0,y_1" \
11055 ▷ ▷ ▷ -end \
11056 ▷ ▷ -define map -mapping \
11057 ▷ ▷ ▷ -domain P1 \
11058 ▷ ▷ ▷ -codomain P3 \

```

```

11059 ▷ ▷ ▷ -ring R2 \
11060 ▷ ▷ ▷ -formula C \
11061 ▷ ▷ ▷ -substitute "" \
11062 ▷ ▷ -end
11063
11064
11065
11066
11067
11068
11069
11070 #####
11071 # Chapter 8 - Orbit Algorithms
11072 #####
11073
11074
11075 test_8:
11076 ▷ make test_8.1
11077 ▷ make test_8.2
11078 ▷ make test_8.3
11079 ▷ make test_8.4
11080 ▷ make test_8.5
11081 ▷ make test_8.6
11082 ▷ make test_8.7
11083
11084
11085 #####
11086 # Section 8.1: Orbit Algorithms
11087
11088
11089 SECTION_ORBIT_ALGORITHMS_SCHREIER_TREES:
11090
11091
11092 test_8.1:
11093 ▷ make orbits_PGL_4.2_on_points_draw_tree
11094 ▷ make orbits_PGL_4.2_on_points_export_trees
11095 ▷ make DD_PP4_orbit
11096 ▷ make T3r1_orbits
11097 ▷ #make T3r1_orbits_draw
11098 ▷ make 2C_orbit_under_PGGL_4.4_elements_coded.csv
11099 ▷ make orbits_on_conics_q13
11100 ▷ make orbits_cubic_curves_q2
11101 ▷ make orbits_cubic_curves_q2_with_draw_tree
11102 ▷ make poly_orbits_d3_n2_q2_compute
11103 ▷ make poly_orbits_d3_n3_q2.csv
11104 ▷ make poly_orbits_d3_n3_q2_get_ranks
11105 ▷ make T4_orbits
11106 ▷ make T4r1_orbits
11107 ▷ make T4r1_orbits_draw
11108 ▷ make T4r1_orbits_4
11109 ▷ make PGGL_7.8_orbits
11110
11111
11112
11113 orbits_PGL_4.2_on_points_draw_tree:
11114 ▷ $(ORBITER) -v 8 \
11115 ▷ ▷ -define Do -draw.options -embedded -end \
11116 ▷ ▷ -define G -linear.group -PGL 4 2 -end \
11117 ▷ ▷ -define Orb -orbits -group G \

```

```

11118 ▷ ▷ ▷ -on_points \
11119 ▷ ▷ -end \
11120 ▷ ▷ -with Orb -do -orbits_activity \
11121 ▷ ▷ ▷ -report Do \
11122 ▷ ▷ -end \
11123 ▷ ▷ -with Orb -do -orbits_activity \
11124 ▷ ▷ ▷ -export_something "orbit" 0 \
11125 ▷ ▷ -end \
11126 ▷ ▷ -with Orb -do -orbits_activity \
11127 ▷ ▷ ▷ -draw_tree 0 Do \
11128 ▷ ▷ -end \
11129 ▷ ▷ -with Orb -do -orbits_activity \
11130 ▷ ▷ ▷ -schreier_poset 0 PGL_4.2_on_points \
11131 ▷ ▷ -end
11132 ▷ $(ORBITER) -v 3 \
11133 ▷ ▷ -draw_layered_graph \
11134 ▷ ▷ ▷ PGL_4.2_on_points_poset.layered_graph \
11135 ▷ ▷ ▷ -place \
11136 ▷ ▷ -radius 500 -embedded \
11137 ▷ ▷ ▷ -line_width 1.1 -x_stretch 1.4 -scale 0.25 \
11138 ▷ ▷ -end
11139 ▷ $(PDFLATEX) PGL_4.2_on_points_poset.draw.tex
11140 ▷ $(OPEN) PGL_4.2_on_points_poset.draw.pdf
11141 ▷ $(PDFLATEX) PGL_4.2_on_points_poset.draw_direct.tex
11142 ▷ $(OPEN) PGL_4.2_on_points_poset.draw_direct.pdf
11143 ▷ #$(PDFLATEX) PGL_4.2_orbits_report.tex
11144 ▷ #$(OPEN) PGL_4.2_orbits_report.pdf
11145 ▷ #$(PDFLATEX) PGL_4.2_orbit_0.tree.tex
11146 ▷ #$(OPEN) PGL_4.2_orbit_0.tree.pdf
11147
11148 #-spanning_tree
11149 #PGL_4.2_on_points_poset.layered_graph
11150
11151
11152
11153
11154 orbits_PGL_4.2_on_points_export_trees:
11155 ▷ $(ORBITER) -v 4 \
11156 ▷ ▷ -define Do -draw_options -embedded -end \
11157 ▷ ▷ -define G -linear_group -PGL 4 2 -end \
11158 ▷ ▷ -define Orb -orbits -group G \
11159 ▷ ▷ ▷ -on_points \
11160 ▷ ▷ -end \
11161 ▷ ▷ -define ro \
11162 ▷ ▷ ▷ -poset_classification_report_options \
11163 ▷ ▷ ▷ -draw_poset -type_ordinary \
11164 ▷ ▷ ▷ -draw_options Do \
11165 ▷ ▷ -end \
11166 ▷ ▷ -with Orb -do -orbits_activity \
11167 ▷ ▷ ▷ -report "" \
11168 ▷ ▷ ▷ -report_options ro \
11169 ▷ ▷ -end \
11170 ▷ ▷ -with Orb -do -orbits_activity \
11171 ▷ ▷ ▷ -export_trees \
11172 ▷ ▷ -end
11173 ▷ $(ORBITER) -v 3 \
11174 ▷ ▷ -draw_layered_graph \
11175 ▷ ▷ ▷ orbit_PGL_4.2_0.layered_graph \
11176 ▷ ▷ ▷ -place \

```

```

11177 ▷ ▷ -radius 500 -spanning_tree -embedded \
11178 ▷ ▷ ▷ -line_width 1.1 -x_stretch 1.4 -scale 0.25 \
11179 ▷ ▷ -end
11180 ▷ $(PDFLATEX) orbit_PGL_4.2.0_draw.tex
11181 ▷ $(OPEN) orbit_PGL_4.2.0_draw.pdf
11182
11183
11184 DD_PP4_orbit:
11185 ▷ $(ORBITER) -v 4 \
11186 ▷ ▷ -define Do -draw_options \
11187 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
11188 ▷ ▷ -end \
11189 ▷ ▷ -define G1 -linear_group -AGL 1 3 \
11190 ▷ ▷ ▷ -end \
11191 ▷ ▷ -define G2 -linear_group -AGL 1 7 \
11192 ▷ ▷ ▷ -end \
11193 ▷ ▷ -define G -modified_group -direct_product "G1,G2" \
11194 ▷ ▷ ▷ "21" "1,1,1,1" \
11195 ▷ ▷ -end \
11196 ▷ ▷ -with G -do \
11197 ▷ ▷ -group_theoretic_activity \
11198 ▷ ▷ ▷ -report Do \
11199 ▷ ▷ -end \
11200 ▷ ▷ -define S -vector -dense "0,1,3,13,20" -end \
11201 ▷ ▷ -define Orb -orbits -group G \
11202 ▷ ▷ ▷ -of_one_subset S \
11203 ▷ ▷ -end
11204
11205
11206 T3r1_orbits:
11207 ▷ $(ORBITER) -v 4 \
11208 ▷ ▷ -define Do -draw_options -embedded -end \
11209 ▷ ▷ -define G \
11210 ▷ ▷ -linear_group -GL_d.q.wr.Sym.n 2 2 3 \
11211 ▷ ▷ ▷ -on_rank_one_tensors -end \
11212 ▷ ▷ -define Orb -orbits -group G \
11213 ▷ ▷ ▷ -on_points \
11214 ▷ ▷ -end \
11215 ▷ ▷ -with Orb -do -orbits_activity \
11216 ▷ ▷ ▷ -report Do \
11217 ▷ ▷ -end \
11218 ▷ ▷ -with Orb -do -orbits_activity \
11219 ▷ ▷ ▷ -draw_tree 0 Do \
11220 ▷ ▷ -end
11221 ▷ $(PDFLATEX) GL_2.2_wreath_Sym3_orbit_0_tree.tex
11222 ▷ $(OPEN) GL_2.2_wreath_Sym3_orbit_0_tree.pdf
11223
11224
11225 # ToDo: layered_graph file is not written
11226
11227 T3r1_orbits_draw:
11228 ▷ $(ORBITER) -v 3 \
11229 ▷ ▷ -draw_layered_graph \
11230 ▷ ▷ ▷ GL_2.2_wreath_Sym3_res27_0.layered_graph \
11231 ▷ ▷ ▷ -place \
11232 ▷ ▷ -radius 500 -spanning_tree -embedded \
11233 ▷ ▷ ▷ -line_width 1.1 -x_stretch 1.4 -scale 0.25 \
11234 ▷ ▷ -end
11235 ▷ #$(PDFLATEX) GL_2.2_wreath_Sym3_report.tex

```

```

11236 ▷ #$(OPEN) GL_2_2.wreath.Sym3.report.pdf
11237 ▷ $(PDFLATEX) GL_2_2.wreath.Sym3.res27_0.draw.tex
11238 ▷ $(OPEN) GL_2_2.wreath.Sym3.res27_0.draw.pdf
11239
11240
11241 # write GL_2_2.wreath.Sym3.res27_0.layered_graph
11242
11243
11244
11245 # very slow:
11246
11247 2C_orbit_under_PGGL_4_4.elements.coded.csv:
11248 ▷ $(ORBITER) -v 6 \
11249 ▷ ▷ -define G -linear_group -PGGL 4 4 -end \
11250 ▷ ▷ -with G -do \
11251 ▷ ▷ -group_theoretic_activity \
11252 ▷ ▷ ▷ -conjugacy_class_of \
11253 ▷ ▷ ▷ "2C" "1,0,0,0, 1,1,0,0, 0,0,1,0, 0,0,1,1, 0" \
11254 ▷ ▷ -end
11255
11256 # class of size 64260
11257 # creates:
11258 # PGGL_4_4.class.of_2C.csv
11259 # 1:33 on Mac
11260 #User time: 2:59 on Mac
11261 # now 1:02 on new Mac
11262
11263
11264
11265
11266 orbits_on_conics.q13:
11267 ▷ $(ORBITER) -v 4 \
11268 ▷ ▷ -define F -finite_field -q 13 -end \
11269 ▷ ▷ -define G -linear_group -PGL 3 F -end \
11270 ▷ ▷ -define R -polynomial_ring \
11271 ▷ ▷ ▷ -field F \
11272 ▷ ▷ ▷ -number_of_variables 3 \
11273 ▷ ▷ ▷ -homogeneous_of_degree 2 \
11274 ▷ ▷ ▷ -monomial_ordering_partition \
11275 ▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \
11276 ▷ ▷ -end \
11277 ▷ ▷ -define Orb -orbits -group G \
11278 ▷ ▷ ▷ -print_interval 1000000 \
11279 ▷ ▷ ▷ -on_polynomials R \
11280 ▷ ▷ -end
11281 ▷ #$(PDFLATEX) poly_orbits_d2_n2.q13.tex
11282 ▷ #$(OPEN) poly_orbits_d2_n2.q13.pdf
11283
11284
11285 orbits_cubic_curves.q2:
11286 ▷ $(ORBITER) -v 4 \
11287 ▷ ▷ -define F -finite_field -q 2 -end \
11288 ▷ ▷ -define G -linear_group -PGL 3 F -end \
11289 ▷ ▷ -define R -polynomial_ring \
11290 ▷ ▷ ▷ -field F \
11291 ▷ ▷ ▷ -number_of_variables 3 \
11292 ▷ ▷ ▷ -homogeneous_of_degree 3 \
11293 ▷ ▷ ▷ -monomial_ordering_partition \
11294 ▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \

```

```

11295 ▷ ▷ -end \
11296 ▷ ▷ -define Orb -orbits -group G \
11297 ▷ ▷ ▷ -print_interval 1000000 \
11298 ▷ ▷ ▷ -on_polynomials R \
11299 ▷ ▷ -end
11300 ▷ #$(PDFLATEX) poly_orbits_d3.n3.q2.tex
11301 ▷ #$(OPEN) poly_orbits_d3.n3.q2.pdf
11302
11303
11304 orbits_cubic_curves_q2.with.draw.tree:
11305 ▷ $(ORBITER) -v 4 \
11306 ▷ ▷ -define Do -draw_options \
11307 ▷ ▷ ▷ -yout 500000 -radius 150 -nodes_empty \
11308 ▷ ▷ ▷ -line_width 0.5 -y_stretch 0.25 -embedded \
11309 ▷ ▷ -end \
11310 ▷ ▷ -define F -finite_field -q 2 -end \
11311 ▷ ▷ -define G -linear_group -PGL 3 F -end \
11312 ▷ ▷ -define R -polynomial_ring \
11313 ▷ ▷ ▷ -field F \
11314 ▷ ▷ ▷ -number_of_variables 3 \
11315 ▷ ▷ ▷ -homogeneous_of_degree 3 \
11316 ▷ ▷ ▷ -monomial_ordering_partition \
11317 ▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \
11318 ▷ ▷ -end \
11319 ▷ ▷ -define Orb -orbits -group G \
11320 ▷ ▷ ▷ -print_interval 1000000 \
11321 ▷ ▷ ▷ -on_polynomials R \
11322 ▷ ▷ -end \
11323 ▷ ▷ -with Orb -do -orbits_activity \
11324 ▷ ▷ ▷ -draw_tree 6 Do \
11325 ▷ ▷ -end
11326 ▷ $(PDFLATEX) PGL_3_2.orbit.6.tree.tex
11327 ▷ $(OPEN) PGL_3_2.orbit.6.tree.pdf
11328
11329
11330
11331
11332 poly_orbits_d3.n2.q2.compute:
11333 ▷ $(ORBITER) -v 4 \
11334 ▷ ▷ -define Do -draw_options \
11335 ▷ ▷ ▷ -yout 500000 -radius 15 -nodes_empty \
11336 ▷ ▷ ▷ -line_width 0.5 -y_stretch 0.25 -embedded \
11337 ▷ ▷ -end \
11338 ▷ ▷ -define F -finite_field -q 2 -end \
11339 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
11340 ▷ ▷ -define G -linear_group -PGL 3 F -end \
11341 ▷ ▷ -define R -polynomial_ring \
11342 ▷ ▷ ▷ -field F \
11343 ▷ ▷ ▷ -number_of_variables 3 \
11344 ▷ ▷ ▷ -homogeneous_of_degree 3 \
11345 ▷ ▷ ▷ -monomial_ordering_partition \
11346 ▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \
11347 ▷ ▷ -end \
11348 ▷ ▷ -define Orb -orbits -group G \
11349 ▷ ▷ ▷ -print_interval 1000000 \
11350 ▷ ▷ ▷ -on_polynomials R \
11351 ▷ ▷ -end \
11352 ▷ ▷ -with Orb -do -orbits_activity \
11353 ▷ ▷ ▷ -report Do \

```



```

11354 ▷ ▷ -end \
11355 ▷ ▷ -with Orb -do -orbits_activity \
11356 ▷ ▷ ▷ -export_something representatives_detailed P \
11357 ▷ ▷ -end \
11358 ▷ ▷ -print_symbols
11359 ▷ $(PDFLATEX) poly_orbits_d3_n2.q2.tex
11360 ▷ $(OPEN) poly_orbits_d3_n2.q2.pdf
11361
11362
11363
11364 # slow:
11365
11366 poly_orbits_d3_n3.q2.csv:
11367 ▷ $(ORBITER) -v 4 \
11368 ▷ ▷ -define Do -draw_options \
11369 ▷ ▷ ▷ -yout 500000 -radius 15 -nodes_empty \
11370 ▷ ▷ ▷ -line_width 0.5 -y_stretch 0.25 -embedded \
11371 ▷ ▷ -end \
11372 ▷ ▷ -define F -finite_field -q 2 -end \
11373 ▷ ▷ -define G -linear_group -PGL 4 F -end \
11374 ▷ ▷ -define R -polynomial_ring \
11375 ▷ ▷ ▷ -field F \
11376 ▷ ▷ ▷ -number_of_variables 4 \
11377 ▷ ▷ ▷ -homogeneous_of_degree 3 \
11378 ▷ ▷ ▷ -monomial_ordering_partition \
11379 ▷ ▷ ▷ -variables "X,Y,Z,W" "X,Y,Z,W" \
11380 ▷ ▷ -end \
11381 ▷ ▷ -define Orb -orbits -group G \
11382 ▷ ▷ ▷ -print_interval 1000000 \
11383 ▷ ▷ ▷ -on_polynomials R \
11384 ▷ ▷ -end \
11385 ▷ ▷ -with Orb -do -orbits_activity \
11386 ▷ ▷ ▷ -report Do \
11387 ▷ ▷ -end \
11388 ▷ ▷ -with Orb -do -orbits_activity \
11389 ▷ ▷ ▷ -draw_tree 6 Do \
11390 ▷ ▷ -end
11391 ▷ $(PDFLATEX) PGL_4_2_orbit_6_tree.tex
11392 ▷ $(OPEN) PGL_4_2_orbit_6_tree.pdf
11393
11394 #written file PGL_4_2_orbit_6_tree.tex of size 79891
11395
11396
11397 poly_orbits_d3_n3.q2_get_ranks:
11398 ▷ $(ORBITER) -v 4 \
11399 ▷ ▷ -csv_file.select_cols poly_orbits_d3_n3.q2.csv "_reps" Rep
11400 ▷ #$(PDFLATEX) poly_orbits_d3_n3.q2.tex
11401 ▷ #$(OPEN) poly_orbits_d3_n3.q2.pdf
11402
11403
11404 #poly_orbits_d3_n3.q2_reps.csv
11405
11406
11407 T4_orbits:
11408 ▷ $(ORBITER) -v 4 \
11409 ▷ ▷ -define G -linear_group \
11410 ▷ ▷ ▷ -GL_d.q.wr.Sym_n 2 2 4 \
11411 ▷ ▷ ▷ -on_tensors \
11412 ▷ ▷ -end \

```

```

11413 ▷ ▷ -define Orb -orbits -group G \
11414 ▷ ▷ ▷ -on_points \
11415 ▷ ▷ -end
11416 ▷ #$(PDFLATEX) GL_2_2_wreath_Sym4_res65535_orbits.tex
11417 ▷ #$(OPEN) GL_2_2_wreath_Sym4_res65535_orbits.pdf
11418 ▷ #$(PDFLATEX) GL_2_2_wreath_Sym4_report.tex
11419 ▷ #$(OPEN) GL_2_2_wreath_Sym4_report.pdf
11420
11421
11422
11423
11424 T4r1_orbits:
11425 ▷ $(ORBITER) -v 4 \
11426 ▷ ▷ -define G -linear_group \
11427 ▷ ▷ ▷ -GL_d.q.wr.Sym.n 2 2 4 \
11428 ▷ ▷ ▷ -on_rank_one_tensors \
11429 ▷ ▷ -end \
11430 ▷ ▷ -define Orb -orbits -group G \
11431 ▷ ▷ ▷ -on_points \
11432 ▷ ▷ -end \
11433 ▷ ▷ -with Orb -do -orbits_activity \
11434 ▷ ▷ ▷ -export_trees \
11435 ▷ ▷ -end
11436
11437
11438 # problem in export_trees because of f_load.save = true in Algebra.orbits_on_poin
    ts
11439 # solution set f_load.save = false
11440
11441 #orbit_GL_2_2_wreath_Sym4_res_on_rank_one_tensors.0.layered_graph
11442
11443
11444 T4r1_orbits_draw:
11445 ▷ $(ORBITER) -v 3 \
11446 ▷ ▷ -draw_layered_graph \
11447 ▷ ▷ ▷ orbit_GL_2_2_wreath_Sym4_res_on_rank_one_tensors.0.layered_graph \
11448 ▷ ▷ ▷ -place \
11449 ▷ ▷ -radius 400 -spanning_tree -embedded \
11450 ▷ ▷ ▷ -line_width 1.1 -x_stretch 2.5 -scale 0.15 \
11451 ▷ ▷ -end
11452 ▷ $(PDFLATEX) orbit_GL_2_2_wreath_Sym4_res_on_rank_one_tensors.0_draw.tex
11453 ▷ $(OPEN) orbit_GL_2_2_wreath_Sym4_res_on_rank_one_tensors.0_draw.pdf
11454
11455
11456 T4r1_orbits_4:
11457 ▷ $(ORBITER) -v 4 \
11458 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
11459 ▷ ▷ -define Do -draw_options -end \
11460 ▷ ▷ -define Control -poset_classification_control \
11461 ▷ ▷ ▷ -problem_label T4r1 -W \
11462 ▷ ▷ ▷ -draw_options Do \
11463 ▷ ▷ -end \
11464 ▷ ▷ -define G -linear_group -GL_d.q.wr.Sym.n 2 2 4 \
11465 ▷ ▷ ▷ -on_rank_one_tensors -end \
11466 ▷ ▷ -define Orb -orbits \
11467 ▷ ▷ ▷ -group G \
11468 ▷ ▷ -group_action G \
11469 ▷ ▷ ▷ -generators G \
11470 ▷ ▷ ▷ -on_subsets 4 Control \

```

```

11471 ▷ ▷ -end \
11472 ▷ ▷ -define ro \
11473 ▷ ▷ ▷ -poset_classification_report_options \
11474 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
11475 ▷ ▷ ▷ ▷ -draw_options Do \
11476 ▷ ▷ -end \
11477 ▷ ▷ -with Orb -do -orbits_activity \
11478 ▷ ▷ ▷ -report "" \
11479 ▷ ▷ ▷ -report_options ro \
11480 ▷ ▷ -end
11481 ▷ $(PDFLATEX) T4r1_poset.tex
11482 ▷ $(OPEN) T4r1_poset.pdf
11483
11484 # error!
11485
11486
11487
11488 # example from the Fining manual, page 107:
11489
11490 PGGL_7.8_orbits:
11491 ▷ $(ORBITER) -v 4 \
11492 ▷ ▷ -define G \
11493 ▷ ▷ -linear_group -PGGL 7 8 -end \
11494 ▷ ▷ -define Orb -orbits -group G \
11495 ▷ ▷ ▷ -on_points \
11496 ▷ ▷ -end
11497
11498 # 0:34
11499
11500
11501
11502
11503
11504
11505 #####
11506 # Section 8.2: Poset Classification
11507
11508
11509 SECTION_POSET_CLASSIFICATION:
11510
11511 test_8.2:
11512 ▷ make poset_of_4subsets
11513 ▷ make poset_of_4subsets.draw
11514 ▷ make poset_of_5subsets
11515 ▷ make poset_of_5subsets.draw
11516 ▷ make Symmetric_4_on_pairs_poset
11517 ▷ make V_3.2_trivial
11518 ▷ make V_4.2_trivial
11519
11520
11521 poset_of_4subsets:
11522 ▷ $(ORBITER) -v 3 \
11523 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
11524 ▷ ▷ -define Control -poset_classification_control \
11525 ▷ ▷ ▷ -problem_label poset_4 \
11526 ▷ ▷ ▷ -W -depth 4 \
11527 ▷ ▷ -end \
11528 ▷ ▷ -define Do -draw_options -radius 150 -end \
11529 ▷ ▷ -define G -permutation_group -identity_group 4 -end \

```

```

11530 ▷ ▷ -define Orb -orbits \
11531 ▷ ▷ ▷ -group G \
11532 ▷ ▷ ▷ -group_action G \
11533 ▷ ▷ ▷ -generators G \
11534 ▷ ▷ ▷ -on_subsets 4 Control \
11535 ▷ ▷ -end \
11536 ▷ ▷ -define ro \
11537 ▷ ▷ ▷ -poset_classification_report_options \
11538 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
11539 ▷ ▷ ▷ -draw_options Do \
11540 ▷ ▷ -end \
11541 ▷ ▷ -with Orb -do -orbits_activity \
11542 ▷ ▷ ▷ -report "" \
11543 ▷ ▷ ▷ -report_options ro \
11544 ▷ ▷ -end
11545 ▷ $(PDFLATEX) poset_4_poset.tex
11546 ▷ $(OPEN) poset_4_poset.pdf
11547
11548
11549 poset_of_4subsets.draw:
11550 ▷ $(ORBITER) -v 3 \
11551 ▷ ▷ -draw_layered_graph \
11552 ▷ ▷ ▷ poset_4_poset_lvl_4.layered_graph \
11553 ▷ ▷ ▷ -place \
11554 ▷ ▷ ▷ -radius 300 -embedded -line_width 1.1 \
11555 ▷ ▷ ▷ -y_stretch 0.9 -scale 0.25 \
11556 ▷ ▷ -end
11557 ▷ $(PDFLATEX) poset_4_poset_lvl_4.draw.tex
11558 ▷ $(OPEN) poset_4_poset_lvl_4.draw.pdf
11559
11560
11561 poset_of_5subsets:
11562 ▷ $(ORBITER) -v 3 \
11563 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
11564 ▷ ▷ -define Do -draw_options -radius 150 -end \
11565 ▷ ▷ -define Control -poset_classification_control \
11566 ▷ ▷ ▷ -problem_label poset_5 \
11567 ▷ ▷ ▷ -W -depth 5 \
11568 ▷ ▷ ▷ -draw_options Do \
11569 ▷ ▷ -end \
11570 ▷ ▷ -define G -permutation_group -identity_group 5 -end \
11571 ▷ ▷ -define Orb -orbits \
11572 ▷ ▷ ▷ -group G \
11573 ▷ ▷ ▷ -group_action G \
11574 ▷ ▷ ▷ -generators G \
11575 ▷ ▷ ▷ -on_subsets 5 Control \
11576 ▷ ▷ -end \
11577 ▷ ▷ -define ro \
11578 ▷ ▷ ▷ -poset_classification_report_options \
11579 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
11580 ▷ ▷ ▷ -draw_options Do \
11581 ▷ ▷ -end \
11582 ▷ ▷ -with Orb -do -orbits_activity \
11583 ▷ ▷ ▷ -report "" \
11584 ▷ ▷ ▷ -report_options ro \
11585 ▷ ▷ -end
11586 ▷ $(PDFLATEX) poset_5_poset.tex
11587 ▷ $(OPEN) poset_5_poset.pdf
11588

```

```

11589
11590 poset_of_5subsets.draw:
11591 ▷ $(ORBITER) -v 3 \
11592 ▷ ▷ -draw_layered_graph \
11593 ▷ ▷ ▷ poset_5.poset_lvl_5.layered_graph \
11594 ▷ ▷ ▷ -place \
11595 ▷ ▷ ▷ -radius 300 -embedded \
11596 ▷ ▷ ▷ -line_width 1.1 -y_stretch 0.9 \
11597 ▷ ▷ ▷ -scale 0.25 \
11598 ▷ ▷ -end
11599 ▷ $(PDFLATEX) poset_5.poset_lvl_5.draw.tex
11600 ▷ $(OPEN) poset_5.poset_lvl_5.draw.pdf
11601
11602
11603
11604
11605 Symmetric_4_on_pairs.poset:
11606 ▷ $(ORBITER) -v 3 \
11607 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
11608 ▷ ▷ -define Do -draw_options -radius 150 -end \
11609 ▷ ▷ -define Control -poset_classification_control \
11610 ▷ ▷ ▷ -problem_label Sym4_on2 \
11611 ▷ ▷ ▷ -W -depth 6 \
11612 ▷ ▷ ▷ -draw_options Do \
11613 ▷ ▷ -end \
11614 ▷ ▷ -define G -permutation_group -symmetric_group 4 -end \
11615 ▷ ▷ -define G_on_2subsets -modified_group -from G \
11616 ▷ ▷ ▷ -on_k_subsets 2 \
11617 ▷ ▷ -end \
11618 ▷ ▷ -define Orb -orbits \
11619 ▷ ▷ ▷ -group G \
11620 ▷ ▷ ▷ -group_action G_on_2subsets \
11621 ▷ ▷ ▷ -generators G \
11622 ▷ ▷ ▷ -on_subsets 6 Control \
11623 ▷ ▷ -end \
11624 ▷ ▷ -define ro \
11625 ▷ ▷ ▷ -poset_classification_report_options \
11626 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
11627 ▷ ▷ ▷ -draw_options Do \
11628 ▷ ▷ -end \
11629 ▷ ▷ -with Orb -do -orbits_activity \
11630 ▷ ▷ ▷ -report "" \
11631 ▷ ▷ ▷ -report_options ro \
11632 ▷ ▷ -end \
11633 ▷ ▷ -define PC_Activity1 -poset_classification_activity \
11634 ▷ ▷ ▷ -orbit_reps_csv \
11635 ▷ ▷ -end \
11636 ▷ ▷ -with Orb -do -orbits_activity \
11637 ▷ ▷ ▷ -poset_classification_activity PC_Activity1 \
11638 ▷ ▷ -end
11639 ▷ ▷ -define PC_Activity2 -poset_classification_activity \
11640 ▷ ▷ ▷ -export_orbits_long \
11641 ▷ ▷ -end \
11642 ▷ ▷ -with Orb -do -orbits_activity \
11643 ▷ ▷ ▷ -poset_classification_activity PC_Activity2 \
11644 ▷ ▷ -end
11645 ▷ $(PDFLATEX) Sym4_on2_poset.tex
11646 ▷ $(OPEN) Sym4_on2_poset.pdf
11647

```

```

11648
11649 #Sym4_on2_all_orbits_expanded.csv
11650
11651
11652 V_3_2_trivial:
11653 ▷ $(ORBITER) -v 5 \
11654 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
11655 ▷ ▷ -define Do -draw_options \
11656 ▷ ▷ ▷ -radius 200 -embedded \
11657 ▷ ▷ -end \
11658 ▷ ▷ -define Control -poset_classification_control \
11659 ▷ ▷ ▷ -problem_label V_3_2_trivial \
11660 ▷ ▷ ▷ -W -depth 3 \
11661 ▷ ▷ ▷ -draw_options Do \
11662 ▷ ▷ -end \
11663 ▷ ▷ -define G -linear_group -PGL 3 2 -identity_group -end \
11664 ▷ ▷ -define Orb -orbits -group G \
11665 ▷ ▷ ▷ -on_subspaces 3 Control \
11666 ▷ ▷ -end \
11667 ▷ ▷ -define ro \
11668 ▷ ▷ ▷ -poset_classification_report_options \
11669 ▷ ▷ ▷ -draw_poset -type_ordinary \
11670 ▷ ▷ ▷ -draw_options Do \
11671 ▷ ▷ -end \
11672 ▷ ▷ -with Orb -do -orbits_activity \
11673 ▷ ▷ ▷ -report "" \
11674 ▷ ▷ ▷ -report_options ro \
11675 ▷ ▷ -end
11676 ▷ $(PDFLATEX) V_3_2_trivial_poset.tex
11677 ▷ $(OPEN) V_3_2_trivial_poset.pdf
11678
11679
11680
11681 V_4_2_trivial:
11682 ▷ $(ORBITER) -v 5 \
11683 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
11684 ▷ ▷ -define Do -draw_options \
11685 ▷ ▷ ▷ -radius 200 -embedded \
11686 ▷ ▷ ▷ -select_layers "1,2,3,4" \
11687 ▷ ▷ -end \
11688 ▷ ▷ -define Control -poset_classification_control \
11689 ▷ ▷ ▷ -problem_label V_4_2_trivial \
11690 ▷ ▷ ▷ -W -depth 3 \
11691 ▷ ▷ -end \
11692 ▷ ▷ -define G -linear_group -PGL 4 2 -identity_group -end \
11693 ▷ ▷ -define Orb -orbits -group G \
11694 ▷ ▷ ▷ -on_subspaces 4 Control \
11695 ▷ ▷ -end \
11696 ▷ ▷ -define ro \
11697 ▷ ▷ ▷ -poset_classification_report_options \
11698 ▷ ▷ ▷ -draw_poset -type_ordinary \
11699 ▷ ▷ ▷ -draw_options Do \
11700 ▷ ▷ -end \
11701 ▷ ▷ -with Orb -do -orbits_activity \
11702 ▷ ▷ ▷ -report "" \
11703 ▷ ▷ ▷ -report_options ro \
11704 ▷ ▷ -end
11705 ▷ $(PDFLATEX) V_4_2_trivial_poset.tex
11706 ▷ $(OPEN) V_4_2_trivial_poset.pdf

```

```

11707
11708 #V_4_2.trivial-poset.detailed_lvl_4.layered_graph
11709
11710 #####
11711 # Section 8.3: Orbits on Subsets
11712
11713
11714 SECTION_ORBITS_ON_SUBSETS:
11715
11716
11717 test_8.3:
11718 ▷ make poset_of_subsets_under_C6
11719 ▷ make orbits_on_subsets_in_action_on_pairs
11720 ▷ make PG_2_2.subsets
11721 ▷ make PG_3_2.subsets
11722 ▷ make PGL_3_2.singer
11723 ▷ make PGL_3_2.on_lines
11724 ▷ make PGL_2_5.on_subsets
11725 ▷ make PGL_2_7.on_subsets
11726 ▷ make PGGL_2_8.on_subsets
11727 ▷ make PGGL_2_9.on_subsets
11728 ▷ make PGL_2_11.on_subsets
11729 ▷ make PGGL_2_16.on_subsets
11730 ▷ make PGGL_2_32.on_subsets
11731 ▷ make PG_3_4.subsets
11732 ▷ make PGGL_2_9.orbits
11733 ▷ make PGO_5_2.on_subsets
11734
11735
11736
11737 poset_of_subsets_under_C6:
11738 ▷ $(ORBITER) -v 3 \
11739 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
11740 ▷ ▷ -define Control -poset_classification_control \
11741 ▷ ▷ ▷ -problem_label poset_C6 \
11742 ▷ ▷ ▷ -W -depth 6 \
11743 ▷ ▷ -end \
11744 ▷ ▷ -define Do -draw_options -radius 150 \
11745 ▷ ▷ -end \
11746 ▷ ▷ -define DoA -draw_options \
11747 ▷ ▷ ▷ -radius 200 -line_width 1.0 \
11748 ▷ ▷ ▷ -poset_orbits_x_stretch 750 \
11749 ▷ ▷ ▷ -embedded \
11750 ▷ ▷ ▷ -poset_with_horizontal_lines \
11751 ▷ ▷ -end \
11752 ▷ ▷ -define G -permutation_group -cyclic_group 6 -end \
11753 ▷ ▷ -define Orb -orbits \
11754 ▷ ▷ ▷ -group G \
11755 ▷ ▷ ▷ -group_action G \
11756 ▷ ▷ ▷ -generators G \
11757 ▷ ▷ ▷ -on_subsets 6 Control \
11758 ▷ ▷ -end \
11759 ▷ ▷ -define ro \
11760 ▷ ▷ ▷ -poset_classification_report_options \
11761 ▷ ▷ ▷ ▷ -draw_poset -type_detailed \
11762 ▷ ▷ ▷ -draw_options Do \
11763 ▷ ▷ -end \
11764 ▷ ▷ -with Orb -do -orbits_activity \
11765 ▷ ▷ ▷ -report "" \

```

```

11766 ▷ ▷ ▷ -report_options ro \
11767 ▷ ▷ -end \
11768 ▷ ▷ -define PC_Activity -poset_classification_activity \
11769 ▷ ▷ ▷ -draw_full_poset DoA \
11770 ▷ ▷ -end \
11771 ▷ ▷ -with Orb -do -orbits_activity \
11772 ▷ ▷ ▷ -poset_classification_activity PC_Activity \
11773 ▷ ▷ -end
11774 ▷ $(ORBITER) -v 3 \
11775 ▷ ▷ -draw_factor_poset \
11776 ▷ ▷ poset_C6_poset_full_lvl_6.factor_poset \
11777 ▷ ▷ -radius 50 -embedded \
11778 ▷ ▷ ▷ -line_width 0.2 -x_stretch 1.4 -scale 0.5 \
11779 ▷ ▷ -end
11780 ▷ pdflatex poset_C6_poset_full_lvl_6.draw.tex
11781 ▷ open poset_C6_poset_full_lvl_6.draw.pdf
11782
11783
11784
11785 orbits_on_subsets_in_action.on.pairs:
11786 ▷ $(ORBITER) -v 3 \
11787 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
11788 ▷ ▷ -define Do -draw_options \
11789 ▷ ▷ ▷ -radius 200 -embedded \
11790 ▷ ▷ -end \
11791 ▷ ▷ -define DoA -draw_options \
11792 ▷ ▷ ▷ -radius 200 -line_width 1.0 \
11793 ▷ ▷ ▷ -poset_orbits_x_stretch 750 \
11794 ▷ ▷ ▷ -embedded \
11795 ▷ ▷ ▷ -poset_with_horizontal_lines \
11796 ▷ ▷ -end \
11797 ▷ ▷ -define Control -poset_classification_control \
11798 ▷ ▷ ▷ -problem_label graphs4 \
11799 ▷ ▷ ▷ -depth 6 \
11800 ▷ ▷ ▷ -draw_options Do \
11801 ▷ ▷ -end \
11802 ▷ ▷ -define G -permutation_group -symmetric_group 4 -end \
11803 ▷ ▷ -define G_on_2subsets -modified_group -from G \
11804 ▷ ▷ ▷ -on_k_subsets 2 \
11805 ▷ ▷ -end \
11806 ▷ ▷ -define Orb -orbits \
11807 ▷ ▷ ▷ -group G \
11808 ▷ ▷ ▷ -group_action G_on_2subsets \
11809 ▷ ▷ ▷ -generators G \
11810 ▷ ▷ ▷ -on_subsets 6 Control \
11811 ▷ ▷ -end \
11812 ▷ ▷ -define ro \
11813 ▷ ▷ ▷ -poset_classification_report_options \
11814 ▷ ▷ ▷ ▷ -draw_poset -type_detailed \
11815 ▷ ▷ ▷ ▷ -draw_options Do \
11816 ▷ ▷ -end \
11817 ▷ ▷ -with Orb -do -orbits_activity \
11818 ▷ ▷ ▷ -report "" \
11819 ▷ ▷ ▷ -report_options ro \
11820 ▷ ▷ -end \
11821 ▷ ▷ -define PC_Activity -poset_classification_activity \
11822 ▷ ▷ ▷ -draw_full_poset DoA \
11823 ▷ ▷ -end \
11824 ▷ ▷ -with Orb -do -orbits_activity \

```



```

11825 ▷ ▷ ▷ -poset_classification_activity PCActivity \
11826 ▷ ▷ -end \
11827 ▷ ▷ -with G_on_2subsets -do \
11828 ▷ ▷ -group_theoretic_activity \
11829 ▷ ▷ ▷ -stats "graphs4" \
11830 ▷ ▷ -end
11831 ▷ $(ORBITER) -v 3 \
11832 ▷ ▷ -draw_factor_poset \
11833 ▷ ▷ ▷ graphs4_poset_full_lvl_6_factor_poset \
11834 ▷ ▷ -radius 50 -embedded \
11835 ▷ ▷ ▷ -line_width 0.2 -x_stretch 1.4 -scale 0.5 \
11836 ▷ ▷ -end
11837 ▷ pdflatex graphs4_poset_full_lvl_6_draw.tex
11838 ▷ open graphs4_poset_full_lvl_6_draw.pdf
11839
11840
11841
11842 PG_2_2_subsets:
11843 ▷ $(ORBITER) -v 3 \
11844 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
11845 ▷ ▷ -define Do -draw_options \
11846 ▷ ▷ ▷ -radius 200 -embedded \
11847 ▷ ▷ -end \
11848 ▷ ▷ -define Control -poset_classification_control \
11849 ▷ ▷ ▷ -problem_label PGL_3_2 \
11850 ▷ ▷ ▷ -depth 7 \
11851 ▷ ▷ ▷ -draw_options Do \
11852 ▷ ▷ -end \
11853 ▷ ▷ -define F -finite_field -q 2 -end \
11854 ▷ ▷ -define G -linear_group -PGL 3 F -end \
11855 ▷ ▷ -define Orb -orbits \
11856 ▷ ▷ ▷ -group G \
11857 ▷ ▷ ▷ -group_action G \
11858 ▷ ▷ ▷ -generators G \
11859 ▷ ▷ ▷ -on_subsets 7 Control \
11860 ▷ ▷ -end \
11861 ▷ ▷ -define ro \
11862 ▷ ▷ ▷ -poset_classification_report_options \
11863 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
11864 ▷ ▷ ▷ ▷ -draw_options Do \
11865 ▷ ▷ -end \
11866 ▷ ▷ -with Orb -do -orbits_activity \
11867 ▷ ▷ ▷ -report "" \
11868 ▷ ▷ ▷ -report_options ro \
11869 ▷ ▷ -end \
11870 ▷ ▷ -with G -do \
11871 ▷ ▷ -group_theoretic_activity \
11872 ▷ ▷ ▷ -stats "PG_2_2_subsets" \
11873 ▷ ▷ -end
11874 ▷ $(PDFLATEX) PGL_3_2_poset_lvl_7.tex
11875 ▷ $(OPEN) PGL_3_2_poset_lvl_7.pdf
11876 ▷ $(PDFLATEX) PGL_3_2_poset.tex
11877 ▷ $(OPEN) PGL_3_2_poset.pdf
11878
11879
11880
11881 # PG(3,2) has  $2^3+2^2+2^1+1 = 15$  points:
11882 # PG(3,3) has  $3^3+3^2+3^1+1 = 27 + 9 + 3 + 1 = 40$  points.
11883

```

```

11884
11885
11886
11887
11888
11889 PG_3_2.subsets:
11890 ▷ $(ORBITER) -v 3 \
11891 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
11892 ▷ ▷ -define Do -draw_options \
11893 ▷ ▷ ▷ -radius 200 -embedded \
11894 ▷ ▷ -end \
11895 ▷ ▷ -define Control -poset_classification_control \
11896 ▷ ▷ ▷ -problem_label PGL_4_2 \
11897 ▷ ▷ ▷ -depth 15 \
11898 ▷ ▷ ▷ -draw_options Do \
11899 ▷ ▷ -end \
11900 ▷ ▷ -define F -finite_field -q 2 -end \
11901 ▷ ▷ -define G -linear_group -PGL 4 F -end \
11902 ▷ ▷ -define Orb -orbits \
11903 ▷ ▷ ▷ -group G \
11904 ▷ ▷ ▷ -group_action G \
11905 ▷ ▷ ▷ -generators G \
11906 ▷ ▷ ▷ -on_subsets 15 Control \
11907 ▷ ▷ -end \
11908 ▷ ▷ -define ro \
11909 ▷ ▷ ▷ -poset_classification_report_options \
11910 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
11911 ▷ ▷ ▷ ▷ -draw_options Do \
11912 ▷ ▷ -end \
11913 ▷ ▷ -with Orb -do -orbits_activity \
11914 ▷ ▷ ▷ -report "" \
11915 ▷ ▷ ▷ -report_options ro \
11916 ▷ ▷ -end
11917 ▷ $(PDFLATEX) PGL_4_2_poset.tex
11918 ▷ $(OPEN) PGL_4_2_poset.pdf
11919
11920 PGL_3_2.singer:
11921 ▷ $(ORBITER) -v 3 \
11922 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
11923 ▷ ▷ -define Do -draw_options \
11924 ▷ ▷ ▷ -radius 200 -embedded \
11925 ▷ ▷ -end \
11926 ▷ ▷ -define Control -poset_classification_control \
11927 ▷ ▷ ▷ -problem_label PGL_3_2.singer_1 \
11928 ▷ ▷ ▷ -W -depth 7 \
11929 ▷ ▷ ▷ -draw_options Do \
11930 ▷ ▷ -end \
11931 ▷ ▷ -define G -linear_group -PGL 3 2 -singer 1 -end \
11932 ▷ ▷ -define Orb -orbits \
11933 ▷ ▷ ▷ -group G \
11934 ▷ ▷ ▷ -group_action G \
11935 ▷ ▷ ▷ -generators G \
11936 ▷ ▷ ▷ -on_subsets 7 Control \
11937 ▷ ▷ -end \
11938 ▷ ▷ -define ro \
11939 ▷ ▷ ▷ -poset_classification_report_options \
11940 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
11941 ▷ ▷ ▷ ▷ -draw_options Do \
11942 ▷ ▷ -end \

```

```

11943 ▷ ▷ -with Orb -do -orbits.activity \
11944 ▷ ▷ ▷ -report "" \
11945 ▷ ▷ ▷ -report_options ro \
11946 ▷ ▷ -end
11947 ▷ $(PDFLATEX) PGL_3.2_singer_1.poset.tex
11948 ▷ $(OPEN) PGL_3.2_singer_1.poset.pdf
11949
11950
11951
11952 PGL_3.2_on_lines:
11953 ▷ $(ORBITER) -v 3 \
11954 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
11955 ▷ ▷ -define Do -draw_options \
11956 ▷ ▷ ▷ -radius 200 -embedded \
11957 ▷ ▷ -end \
11958 ▷ ▷ -define Control -poset_classification_control \
11959 ▷ ▷ ▷ -problem_label PGL_3.2_lines \
11960 ▷ ▷ ▷ -W -depth 7 \
11961 ▷ ▷ -end \
11962 ▷ ▷ -define G -linear_group -PGL 3 2 -end \
11963 ▷ ▷ -define G_on_lines -modified_group -from G \
11964 ▷ ▷ ▷ -on_k_subspaces 2 \
11965 ▷ ▷ -end \
11966 ▷ ▷ -define Orb -orbits \
11967 ▷ ▷ ▷ -group G \
11968 ▷ ▷ ▷ -group_action G_on_lines \
11969 ▷ ▷ ▷ -generators G \
11970 ▷ ▷ ▷ -on_subsets 7 Control \
11971 ▷ ▷ -end \
11972 ▷ ▷ -define ro \
11973 ▷ ▷ ▷ -poset_classification_report_options \
11974 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
11975 ▷ ▷ ▷ ▷ -draw_options Do \
11976 ▷ ▷ -end \
11977 ▷ ▷ -with Orb -do -orbits.activity \
11978 ▷ ▷ ▷ -report "" \
11979 ▷ ▷ ▷ -report_options ro \
11980 ▷ ▷ -end
11981 ▷ $(PDFLATEX) PGL_3.2_lines.poset.tex
11982 ▷ $(OPEN) PGL_3.2_lines.poset.pdf
11983
11984
11985
11986 PGL_2.5_on_subsets:
11987 ▷ $(ORBITER) -v 5 \
11988 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
11989 ▷ ▷ -define Do -draw_options \
11990 ▷ ▷ ▷ -radius 200 -embedded \
11991 ▷ ▷ -end \
11992 ▷ ▷ -define Control -poset_classification_control \
11993 ▷ ▷ ▷ -problem_label PGL_2.5 \
11994 ▷ ▷ ▷ -W -depth 6 \
11995 ▷ ▷ -end \
11996 ▷ ▷ -define G -linear_group -PGL 2 5 -end \
11997 ▷ ▷ -define Orb -orbits \
11998 ▷ ▷ ▷ -group G \
11999 ▷ ▷ ▷ -group_action G \
12000 ▷ ▷ ▷ -generators G \
12001 ▷ ▷ ▷ -on_subsets 6 Control \

```

```

12002 ▷ ▷ -end \
12003 ▷ ▷ -define ro \
12004 ▷ ▷ ▷ -poset_classification_report_options \
12005 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
12006 ▷ ▷ ▷ ▷ -draw_options Do \
12007 ▷ ▷ -end \
12008 ▷ ▷ -with Orb -do -orbits_activity \
12009 ▷ ▷ ▷ -report "" \
12010 ▷ ▷ ▷ -report_options ro \
12011 ▷ ▷ -end
12012 ▷ $(PDFLATEX) PGL_2_5_poset.tex
12013 ▷ $(OPEN) PGL_2_5_poset.pdf
12014
12015 PGL_2_7_on_subsets:
12016 ▷ $(ORBITER) -v 10 \
12017 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
12018 ▷ ▷ -define Do -draw_options \
12019 ▷ ▷ ▷ -radius 200 -embedded \
12020 ▷ ▷ -end \
12021 ▷ ▷ -define Control -poset_classification_control \
12022 ▷ ▷ ▷ -problem_label PGL_2_7 \
12023 ▷ ▷ ▷ -W -depth 8 \
12024 ▷ ▷ -end \
12025 ▷ ▷ -define G -linear_group -PGL 2 7 -end \
12026 ▷ ▷ -define Orb -orbits \
12027 ▷ ▷ ▷ -group G \
12028 ▷ ▷ ▷ -group_action G \
12029 ▷ ▷ ▷ -generators G \
12030 ▷ ▷ ▷ -on_subsets 8 Control \
12031 ▷ ▷ -end \
12032 ▷ ▷ -define ro \
12033 ▷ ▷ ▷ -poset_classification_report_options \
12034 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
12035 ▷ ▷ ▷ ▷ -draw_options Do \
12036 ▷ ▷ -end \
12037 ▷ ▷ -with Orb -do -orbits_activity \
12038 ▷ ▷ ▷ -report "" \
12039 ▷ ▷ ▷ -report_options ro \
12040 ▷ ▷ -end
12041 ▷ $(PDFLATEX) PGL_2_7_poset.tex
12042 ▷ $(OPEN) PGL_2_7_poset.pdf
12043
12044 PGGL_2_8_on_subsets:
12045 ▷ $(ORBITER) -v 10 \
12046 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
12047 ▷ ▷ -define Do -draw_options \
12048 ▷ ▷ ▷ -radius 200 -embedded \
12049 ▷ ▷ -end \
12050 ▷ ▷ -define Control -poset_classification_control \
12051 ▷ ▷ ▷ -problem_label PGGL_2_8 \
12052 ▷ ▷ ▷ -W -depth 9 \
12053 ▷ ▷ -end \
12054 ▷ ▷ -define G -linear_group -PGGL 2 8 -end \
12055 ▷ ▷ -define Orb -orbits \
12056 ▷ ▷ ▷ -group G \
12057 ▷ ▷ ▷ -group_action G \
12058 ▷ ▷ ▷ -generators G \
12059 ▷ ▷ ▷ -on_subsets 9 Control \
12060 ▷ ▷ -end \

```

```

12061 ▷ ▷ -define ro \
12062 ▷ ▷ ▷ -poset_classification_report_options \
12063 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
12064 ▷ ▷ ▷ ▷ -draw_options Do \
12065 ▷ ▷ -end \
12066 ▷ ▷ -with Orb -do -orbits_activity \
12067 ▷ ▷ ▷ -report "" \
12068 ▷ ▷ ▷ -report_options ro \
12069 ▷ ▷ -end
12070 ▷ $(PDFLATEX) PGGL_2.8_poset.tex
12071 ▷ $(OPEN) PGGL_2.8_poset.pdf
12072
12073
12074 PGGL_2.9_on_subsets:
12075 ▷ $(ORBITER) -v 10 \
12076 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
12077 ▷ ▷ -define Do -draw_options \
12078 ▷ ▷ ▷ -radius 200 -embedded \
12079 ▷ ▷ -end \
12080 ▷ ▷ -define Control -poset_classification_control \
12081 ▷ ▷ ▷ -problem_label PGGL_2.9 \
12082 ▷ ▷ ▷ -W -depth 10 \
12083 ▷ ▷ -end \
12084 ▷ ▷ -define G -linear_group -PGGL 2 9 -end \
12085 ▷ ▷ -define Orb -orbits \
12086 ▷ ▷ ▷ -group G \
12087 ▷ ▷ ▷ -group_action G \
12088 ▷ ▷ ▷ -generators G \
12089 ▷ ▷ ▷ -on_subsets 10 Control \
12090 ▷ ▷ -end \
12091 ▷ ▷ -define ro \
12092 ▷ ▷ ▷ -poset_classification_report_options \
12093 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
12094 ▷ ▷ ▷ ▷ -draw_options Do \
12095 ▷ ▷ -end \
12096 ▷ ▷ -with Orb -do -orbits_activity \
12097 ▷ ▷ ▷ -report "" \
12098 ▷ ▷ ▷ -report_options ro \
12099 ▷ ▷ -end
12100 ▷ $(PDFLATEX) PGGL_2.9_poset.tex
12101 ▷ $(OPEN) PGGL_2.9_poset.pdf
12102
12103
12104 PGL_2.11_on_subsets:
12105 ▷ $(ORBITER) -v 10 \
12106 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
12107 ▷ ▷ -define Do -draw_options \
12108 ▷ ▷ ▷ -radius 200 -embedded \
12109 ▷ ▷ -end \
12110 ▷ ▷ -define Control -poset_classification_control \
12111 ▷ ▷ ▷ -problem_label PGL_2.11 \
12112 ▷ ▷ ▷ -W -depth 12 \
12113 ▷ ▷ -end \
12114 ▷ ▷ -define G -linear_group -PGL 2 11 -end \
12115 ▷ ▷ -define Orb -orbits \
12116 ▷ ▷ ▷ -group G \
12117 ▷ ▷ ▷ -group_action G \
12118 ▷ ▷ ▷ -generators G \
12119 ▷ ▷ ▷ -on_subsets 12 Control \

```

```

12120 ▷ ▷ -end \
12121 ▷ ▷ -define ro \
12122 ▷ ▷ ▷ -poset_classification_report_options \
12123 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
12124 ▷ ▷ ▷ ▷ -draw_options Do \
12125 ▷ ▷ -end \
12126 ▷ ▷ -with Orb -do -orbits_activity \
12127 ▷ ▷ ▷ -report "" \
12128 ▷ ▷ ▷ -report_options ro \
12129 ▷ ▷ -end
12130 ▷ $(PDFLATEX) PGL_2_11_poset.tex
12131 ▷ $(OPEN) PGL_2_11_poset.pdf
12132
12133
12134
12135 PGGL_2_16_on_subsets:
12136 ▷ $(ORBITER) -v 3 \
12137 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
12138 ▷ ▷ -define Do -draw_options \
12139 ▷ ▷ ▷ -radius 200 -embedded \
12140 ▷ ▷ -end \
12141 ▷ ▷ -define Control -poset_classification_control \
12142 ▷ ▷ ▷ -problem_label PGGL_2_16 \
12143 ▷ ▷ ▷ -W -depth 10 \
12144 ▷ ▷ -end \
12145 ▷ ▷ -define G -linear_group -PGGL 2 16 -end \
12146 ▷ ▷ -define Orb -orbits \
12147 ▷ ▷ ▷ -group G \
12148 ▷ ▷ ▷ -group_action G \
12149 ▷ ▷ ▷ -generators G \
12150 ▷ ▷ ▷ -on_subsets 10 Control \
12151 ▷ ▷ -end \
12152 ▷ ▷ -define ro \
12153 ▷ ▷ ▷ -poset_classification_report_options \
12154 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
12155 ▷ ▷ ▷ ▷ -draw_options Do \
12156 ▷ ▷ -end \
12157 ▷ ▷ -with Orb -do -orbits_activity \
12158 ▷ ▷ ▷ -report "" \
12159 ▷ ▷ ▷ -report_options ro \
12160 ▷ ▷ -end
12161 ▷ $(PDFLATEX) PGGL_2_16_poset.tex
12162 ▷ $(OPEN) PGGL_2_16_poset.pdf
12163
12164
12165 PGGL_2_32_on_subsets:
12166 ▷ $(ORBITER) -v 3 \
12167 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
12168 ▷ ▷ -define Do -draw_options \
12169 ▷ ▷ ▷ -radius 200 -embedded \
12170 ▷ ▷ -end \
12171 ▷ ▷ -define Control -poset_classification_control \
12172 ▷ ▷ ▷ -problem_label PGGL_2_32 \
12173 ▷ ▷ ▷ -W -depth 8 \
12174 ▷ ▷ -end \
12175 ▷ ▷ -define G -linear_group -PGGL 2 32 -end \
12176 ▷ ▷ -define Orb -orbits \
12177 ▷ ▷ ▷ -group G \
12178 ▷ ▷ ▷ -group_action G \

```

```

12179 ▷ ▷ ▷ -generators G \
12180 ▷ ▷ ▷ -on_subsets 8 Control \
12181 ▷ ▷ -end \
12182 ▷ ▷ -define ro \
12183 ▷ ▷ ▷ -poset.classification.report.options \
12184 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
12185 ▷ ▷ ▷ ▷ -draw_options Do \
12186 ▷ ▷ -end \
12187 ▷ ▷ -with Orb -do -orbits.activity \
12188 ▷ ▷ ▷ -report "" \
12189 ▷ ▷ ▷ -report_options ro \
12190 ▷ ▷ -end
12191 ▷ $(PDFLATEX) PGGL_2.32_poset.tex
12192 ▷ $(OPEN) PGGL_2.32_poset.pdf
12193
12194
12195 PG_3.4_subsets:
12196 ▷ $(ORBITER) -v 3 \
12197 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
12198 ▷ ▷ -define Do -draw_options \
12199 ▷ ▷ ▷ -radius 200 -embedded \
12200 ▷ ▷ -end \
12201 ▷ ▷ -define Control -poset.classification.control \
12202 ▷ ▷ ▷ -problem_label PGGL_4.4 \
12203 ▷ ▷ ▷ -depth 5 \
12204 ▷ ▷ -end \
12205 ▷ ▷ -define G -linear_group -PGGL 4 4 -end \
12206 ▷ ▷ -define Orb -orbits \
12207 ▷ ▷ ▷ -group G \
12208 ▷ ▷ ▷ -group_action G \
12209 ▷ ▷ ▷ -generators G \
12210 ▷ ▷ ▷ -on_subsets 5 Control \
12211 ▷ ▷ -end \
12212 ▷ ▷ -define ro \
12213 ▷ ▷ ▷ -poset.classification.report.options \
12214 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
12215 ▷ ▷ ▷ ▷ -draw_options Do \
12216 ▷ ▷ -end \
12217 ▷ ▷ -with Orb -do -orbits.activity \
12218 ▷ ▷ ▷ -report "" \
12219 ▷ ▷ ▷ -report_options ro \
12220 ▷ ▷ -end
12221 ▷ $(PDFLATEX) PGGL_4.4_poset.tex
12222 ▷ $(OPEN) PGGL_4.4_poset.pdf
12223
12224
12225 PGGL_2.9_orbits:
12226 ▷ $(ORBITER) -v 3 \
12227 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
12228 ▷ ▷ -define Do -draw_options \
12229 ▷ ▷ ▷ -radius 200 -embedded \
12230 ▷ ▷ -end \
12231 ▷ ▷ -define Control -poset.classification.control \
12232 ▷ ▷ ▷ -problem_label PGGL_2.9 -W -depth 5 \
12233 ▷ ▷ -end \
12234 ▷ ▷ -define G -linear_group -PGGL 2 9 -end \
12235 ▷ ▷ -define Orb -orbits \
12236 ▷ ▷ ▷ -group G \
12237 ▷ ▷ ▷ -group_action G \

```

```

12238 ▷ ▷ ▷ -generators G \
12239 ▷ ▷ ▷ -on_subsets 5 Control \
12240 ▷ ▷ -end \
12241 ▷ ▷ -define ro \
12242 ▷ ▷ ▷ -poset.classification.report.options \
12243 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
12244 ▷ ▷ ▷ ▷ -draw_options Do \
12245 ▷ ▷ -end \
12246 ▷ ▷ -with Orb -do -orbits.activity \
12247 ▷ ▷ ▷ -report "" \
12248 ▷ ▷ ▷ -report_options ro \
12249 ▷ ▷ -end
12250 ▷ $(PDFLATEX) PGGL_2_9_poset.tex
12251 ▷ $(OPEN) PGGL_2_9_poset.pdf
12252
12253
12254
12255
12256 PG0_5_2_on_subsets:
12257 ▷ $(ORBITER) -v 3 \
12258 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
12259 ▷ ▷ -define Do -draw_options \
12260 ▷ ▷ ▷ -radius 200 -embedded \
12261 ▷ ▷ -end \
12262 ▷ ▷ -define Control -poset_classification_control \
12263 ▷ ▷ ▷ -problem_label PG0_5_2 \
12264 ▷ ▷ ▷ -depth 15 \
12265 ▷ ▷ ▷ -w \
12266 ▷ ▷ -end \
12267 ▷ ▷ -define F -finite_field -q 2 -end \
12268 ▷ ▷ -define G -linear_group -PG0 5 F -end \
12269 ▷ ▷ -define Orb -orbits \
12270 ▷ ▷ ▷ -group G \
12271 ▷ ▷ ▷ -group_action G \
12272 ▷ ▷ ▷ -generators G \
12273 ▷ ▷ ▷ -on_subsets 15 Control \
12274 ▷ ▷ -end \
12275 ▷ ▷ -define ro \
12276 ▷ ▷ ▷ -poset.classification.report.options \
12277 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
12278 ▷ ▷ ▷ ▷ -draw_options Do \
12279 ▷ ▷ -end \
12280 ▷ ▷ -with Orb -do -orbits.activity \
12281 ▷ ▷ ▷ -report "" \
12282 ▷ ▷ ▷ -report_options ro \
12283 ▷ ▷ -end
12284 ▷ $(PDFLATEX) PG0_5_2_poset.tex
12285 ▷ $(OPEN) PG0_5_2_poset.pdf
12286
12287
12288
12289
12290 #####
12291 # Section 8.4: Orbits on Subspaces
12292
12293
12294 SECTION_ORBITS_ON_SUBSPACES:
12295
12296

```



```

12297 test_8.4:
12298 ▷ make subspaces_Op_4.2
12299 ▷ make PGL_4.2_on_subspaces
12300 ▷ make PGL_4.2_singer_on_subspaces
12301 ▷ make PGL_8.2_singer_on_subspaces
12302 ▷ make Op_6.2_orbits_on_subspaces
12303 ▷ make Op_6.3_orbits_on_subspaces
12304 ▷ make Op_6.11_orbits_on_subspaces
12305 ▷ make Op_8.2_orbits_on_subspaces
12306 ▷ make PGO_7.2_on_subspaces
12307 ▷ make PGL_5.3_wedge_subspace_orbits
12308
12309
12310
12311
12312
12313
12314
12315
12316 subspaces_Op_4.2:
12317 ▷ $(ORBITER) -v 10 \
12318 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
12319 ▷ ▷ -define Do -draw_options \
12320 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
12321 ▷ ▷ ▷ -poset_orbits_x_stretch 750 \
12322 ▷ ▷ ▷ -embedded \
12323 ▷ ▷ -end \
12324 ▷ ▷ -define Control -poset_classification_control \
12325 ▷ ▷ ▷ -problem_label Op_4.2 -W -depth 4 \
12326 ▷ ▷ -end \
12327 ▷ ▷ -define G -linear_group -PGL 4 2 -orthogonal 1 -end \
12328 ▷ ▷ -define Orb -orbits \
12329 ▷ ▷ ▷ -group G \
12330 ▷ ▷ ▷ -group_action G \
12331 ▷ ▷ ▷ -generators G \
12332 ▷ ▷ ▷ -on_subspaces 4 Control \
12333 ▷ ▷ -end \
12334 ▷ ▷ -define ro \
12335 ▷ ▷ ▷ -poset_classification_report_options \
12336 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
12337 ▷ ▷ ▷ -draw_options Do \
12338 ▷ ▷ -end \
12339 ▷ ▷ -with Orb -do -orbits_activity \
12340 ▷ ▷ ▷ -report "" \
12341 ▷ ▷ ▷ -report_options ro \
12342 ▷ ▷ -end \
12343 ▷ ▷ -define PCActivity -poset_classification_activity \
12344 ▷ ▷ ▷ -draw_full_poset Do \
12345 ▷ ▷ -end \
12346 ▷ ▷ -with Orb -do -orbits_activity \
12347 ▷ ▷ ▷ -poset_classification_activity PCActivity \
12348 ▷ ▷ -end
12349 ▷ $(ORBITER) -v 3 \
12350 ▷ ▷ -draw_layered_graph \
12351 ▷ ▷ ▷ Op_4.2_poset_full_lvl_4.layered_graph \
12352 ▷ ▷ ▷ -radius 150 -embedded \
12353 ▷ ▷ ▷ -line_width 0.2 -x_stretch 1.4 -scale 0.5 \
12354 ▷ ▷ -end
12355 ▷ $(PDFLATEX) Op_4.2_poset_full_lvl_4.draw.tex

```

```

12356 ▷ $(OPEN) Op_4.2_poset_full_lvl_4.draw.pdf
12357 ▷ $(PDFLATEX) Op_4.2_poset.tex
12358 ▷ $(OPEN) Op_4.2_poset.pdf
12359 ▷ #$(PDFLATEX) Op_4.2_poset_full_lvl_4.tex
12360 ▷ #$(OPEN) Op_4.2_poset_full_lvl_4.pdf
12361 ▷ #$(PDFLATEX) Op_4.2_poset_full_lvl_4.draw.tex
12362 ▷ #$(OPEN) Op_4.2_poset_full_lvl_4.draw.pdf
12363
12364
12365
12366 PGL_4.2_on_subspaces:
12367 ▷ $(ORBITER) -v 5 \
12368 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
12369 ▷ ▷ -define Do -draw_options \
12370 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
12371 ▷ ▷ -end \
12372 ▷ ▷ -define Control -poset_classification_control \
12373 ▷ ▷ ▷ -problem_label PGL_4.2 \
12374 ▷ ▷ ▷ -W -depth 4 \
12375 ▷ ▷ -end \
12376 ▷ ▷ -define G -linear_group -PGL 4 2 -end \
12377 ▷ ▷ -define Orb -orbits \
12378 ▷ ▷ ▷ -group G \
12379 ▷ ▷ ▷ -group_action G \
12380 ▷ ▷ ▷ -generators G \
12381 ▷ ▷ ▷ -on_subspaces 4 Control \
12382 ▷ ▷ -end \
12383 ▷ ▷ -define ro \
12384 ▷ ▷ ▷ -poset_classification_report_options \
12385 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
12386 ▷ ▷ ▷ -draw_options Do \
12387 ▷ ▷ -end \
12388 ▷ ▷ -with Orb -do -orbits_activity \
12389 ▷ ▷ ▷ -report "" \
12390 ▷ ▷ ▷ -report_options ro \
12391 ▷ ▷ -end
12392 ▷ $(PDFLATEX) PGL_4.2_poset.tex
12393 ▷ $(OPEN) PGL_4.2_poset.pdf
12394
12395
12396
12397
12398 PGL_4.2_singer_on_subspaces:
12399 ▷ $(ORBITER) -v 5 \
12400 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
12401 ▷ ▷ -define Do -draw_options \
12402 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
12403 ▷ ▷ -end \
12404 ▷ ▷ -define Control -poset_classification_control \
12405 ▷ ▷ ▷ -problem_label PGL_4.2_singer \
12406 ▷ ▷ ▷ -W -depth 4 \
12407 ▷ ▷ -end \
12408 ▷ ▷ -define G -linear_group -PGL 4 2 -singer 1 -end \
12409 ▷ ▷ -define Orb -orbits \
12410 ▷ ▷ ▷ -group G \
12411 ▷ ▷ ▷ -group_action G \
12412 ▷ ▷ ▷ -generators G \
12413 ▷ ▷ ▷ -on_subspaces 4 Control \
12414 ▷ ▷ -end \

```

```

12415 ▷ ▷ -define ro \
12416 ▷ ▷ ▷ -poset_classification_report_options \
12417 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
12418 ▷ ▷ ▷ -draw_options Do \
12419 ▷ ▷ -end \
12420 ▷ ▷ -with Orb -do -orbits_activity \
12421 ▷ ▷ ▷ -report "" \
12422 ▷ ▷ ▷ -report_options ro \
12423 ▷ ▷ -end
12424 ▷ $(PDFLATEX) PGL_4.2_singer_poset.tex
12425 ▷ $(OPEN) PGL_4.2_singer_poset.pdf
12426
12427
12428
12429
12430 PGL_8.2_singer_on_subspaces:
12431 ▷ $(ORBITER) -v 5 \
12432 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
12433 ▷ ▷ -define Do -draw_options \
12434 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
12435 ▷ ▷ -end \
12436 ▷ ▷ -define Control -poset_classification_control \
12437 ▷ ▷ ▷ -problem_label PGL_8.2_singer \
12438 ▷ ▷ ▷ -W -depth 8 \
12439 ▷ ▷ -end \
12440 ▷ ▷ -define G -linear_group -PGL 8 2 -singer 1 -end \
12441 ▷ ▷ -define Orb -orbits \
12442 ▷ ▷ ▷ -group G \
12443 ▷ ▷ ▷ -group_action G \
12444 ▷ ▷ ▷ -generators G \
12445 ▷ ▷ ▷ -on_subspaces 8 Control \
12446 ▷ ▷ -end \
12447 ▷ ▷ -define ro \
12448 ▷ ▷ ▷ -poset_classification_report_options \
12449 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
12450 ▷ ▷ ▷ -draw_options Do \
12451 ▷ ▷ -end \
12452 ▷ ▷ -with Orb -do -orbits_activity \
12453 ▷ ▷ ▷ -report "" \
12454 ▷ ▷ ▷ -report_options ro \
12455 ▷ ▷ -end
12456 ▷ #$(PDFLATEX) PGL_8.2_singer_poset.tex
12457 ▷ #$(OPEN) PGL_8.2_singer_poset.pdf
12458
12459 # May 7, 2020: 16 sec on Mac
12460 # 1643 orbits in total
12461 # Mar 25, 2025:
12462 #User time: 2:6 (slow)
12463
12464
12465
12466 Op_6.2_orbits_on_subspaces:
12467 ▷ $(ORBITER) -v 5 \
12468 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
12469 ▷ ▷ -define Do -draw_options \
12470 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
12471 ▷ ▷ -end \
12472 ▷ ▷ -define Control -poset_classification_control \
12473 ▷ ▷ ▷ -problem_label Op_6.2 -W \

```

```

12474 ▷ ▷ ▷ -depth 6 \
12475 ▷ ▷ -end \
12476 ▷ ▷ -define G -linear_group -PGL 6 2 -orthogonal 1 -end \
12477 ▷ ▷ -define Orb -orbits \
12478 ▷ ▷ ▷ -group G \
12479 ▷ ▷ ▷ -group.action G \
12480 ▷ ▷ ▷ -generators G \
12481 ▷ ▷ ▷ -on_subspaces 6 Control \
12482 ▷ ▷ -end \
12483 ▷ ▷ -define ro \
12484 ▷ ▷ ▷ -poset_classification_report_options \
12485 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
12486 ▷ ▷ ▷ -draw_options Do \
12487 ▷ ▷ -end \
12488 ▷ ▷ -with Orb -do -orbits_activity \
12489 ▷ ▷ ▷ -report "" \
12490 ▷ ▷ ▷ -report_options ro \
12491 ▷ ▷ -end
12492 ▷ $(PDFLATEX) Op_6_2_poset.tex
12493 ▷ $(OPEN) Op_6_2_poset.pdf
12494
12495
12496
12497 Op_6_3_orbits_on_subspaces:
12498 ▷ $(ORBITER) -v 5 \
12499 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
12500 ▷ ▷ -define Control -poset_classification_control \
12501 ▷ ▷ ▷ -problem_label Op_6_3 -W \
12502 ▷ ▷ ▷ -depth 6 \
12503 ▷ ▷ -end \
12504 ▷ ▷ -define G -linear_group -PGL 6 3 -orthogonal 1 -end \
12505 ▷ ▷ -define Orb -orbits \
12506 ▷ ▷ ▷ -group G \
12507 ▷ ▷ ▷ -group.action G \
12508 ▷ ▷ ▷ -generators G \
12509 ▷ ▷ ▷ -on_subspaces 6 Control \
12510 ▷ ▷ -end
12511
12512 # June 3, 2020 on Mac: 0 sec
12513
12514
12515
12516 Op_6_11_orbits_on_subspaces:
12517 ▷ $(ORBITER) -v 5 \
12518 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
12519 ▷ ▷ -define Control -poset_classification_control \
12520 ▷ ▷ ▷ -problem_label Op_6_11 -W \
12521 ▷ ▷ ▷ -depth 6 \
12522 ▷ ▷ -end \
12523 ▷ ▷ -define G -linear_group -PGL 6 11 -orthogonal 1 -end \
12524 ▷ ▷ -define Orb -orbits \
12525 ▷ ▷ ▷ -group G \
12526 ▷ ▷ ▷ -group.action G \
12527 ▷ ▷ ▷ -generators G \
12528 ▷ ▷ ▷ -on_subspaces 6 Control \
12529 ▷ ▷ -end
12530
12531
12532 # June 3, 2020 on Mac: 12 sec

```

```

12533
12534
12535 Op_8_2_orbits_on_subspaces:
12536 ▷ $(ORBITER) -v 5 \
12537 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
12538 ▷ ▷ -define Do -draw_options \
12539 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
12540 ▷ ▷ -end \
12541 ▷ ▷ -define Control -poset_classification_control \
12542 ▷ ▷ ▷ -problem_label Op_8_2 -W -depth 8 \
12543 ▷ ▷ -end \
12544 ▷ ▷ -define G -linear_group -PGL 8 2 -orthogonal 1 -end \
12545 ▷ ▷ -define Orb -orbits \
12546 ▷ ▷ ▷ -group G \
12547 ▷ ▷ ▷ -group_action G \
12548 ▷ ▷ ▷ -generators G \
12549 ▷ ▷ ▷ -on_subspaces 8 Control \
12550 ▷ ▷ -end \
12551 ▷ ▷ -define ro \
12552 ▷ ▷ ▷ -poset_classification_report_options \
12553 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
12554 ▷ ▷ ▷ -draw_options Do \
12555 ▷ ▷ -end \
12556 ▷ ▷ -with Orb -do -orbits_activity \
12557 ▷ ▷ ▷ -report "" \
12558 ▷ ▷ ▷ -report_options ro \
12559 ▷ ▷ -end
12560 ▷ $(PDFLATEX) Op_8_2_poset.tex
12561 ▷ $(OPEN) Op_8_2_poset.pdf
12562
12563
12564
12565 PGO_7_2_on_subspaces:
12566 ▷ $(ORBITER) -v 6 \
12567 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
12568 ▷ ▷ -define Control -poset_classification_control \
12569 ▷ ▷ ▷ -problem_label O_7_2 \
12570 ▷ ▷ ▷ -W -depth 7 \
12571 ▷ ▷ -end \
12572 ▷ ▷ -define F -finite_field -q 2 -end \
12573 ▷ ▷ -define G -linear_group -PGL 7 F -orthogonal 0 -end \
12574 ▷ ▷ -define Orb -orbits \
12575 ▷ ▷ ▷ -group G \
12576 ▷ ▷ ▷ -group_action G \
12577 ▷ ▷ ▷ -generators G \
12578 ▷ ▷ ▷ -on_subspaces 7 Control \
12579 ▷ ▷ -end
12580
12581
12582 PGL_5_3_wedge_subspace_orbits:
12583 ▷ $(ORBITER) -v 3 \
12584 ▷ ▷ -define Do -draw_options \
12585 ▷ ▷ ▷ -radius 75 -embedded -nodes_empty \
12586 ▷ ▷ -end \
12587 ▷ ▷ -define G -linear_group -PGL 5 3 -end \
12588 ▷ ▷ -define G2 -modified_group -from G \
12589 ▷ ▷ ▷ -on_wedge_product \
12590 ▷ ▷ -end \
12591 ▷ ▷ -define Control -poset_classification_control \

```

```

12592 ▷ ▷ ▷ -problem_label PGL_5.3.wedge -W -depth 5 \
12593 ▷ ▷ -end \
12594 ▷ ▷ -define Orb -orbits \
12595 ▷ ▷ ▷ -group G \
12596 ▷ ▷ ▷ -group.action G2 \
12597 ▷ ▷ ▷ -generators G \
12598 ▷ ▷ ▷ -on_subspaces 5 Control \
12599 ▷ ▷ -end \
12600 ▷ ▷ -with Orb -do -orbits.activity \
12601 ▷ ▷ ▷ -draw_tree 0 Do \
12602 ▷ ▷ -end
12603 ▷ $(PDFLATEX) PGL_5.3.wedge.tree_lvl5.tex
12604 ▷ $(OPEN) PGL_5.3.wedge.tree_lvl5.pdf
12605
12606
12607
12608
12609 #####
12610 # Section 8.5: Orbits on set partitions
12611
12612
12613 SECTION_ORBITS_ON_SET_PARTITIONS:
12614
12615
12616
12617 test_8.5:
12618 ▷ make C6_on_partition
12619 ▷ make PGL_2.17_on_partition
12620
12621
12622 C6_on_partition:
12623 ▷ $(ORBITER) -v 5 \
12624 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
12625 ▷ ▷ -define Control -poset_classification_control \
12626 ▷ ▷ ▷ -problem_label C6 \
12627 ▷ ▷ ▷ -depth 2 \
12628 ▷ ▷ ▷ -W \
12629 ▷ ▷ -end \
12630 ▷ ▷ -define G -permutation_group -cyclic_group 6 -end \
12631 ▷ ▷ -define Orb -orbits \
12632 ▷ ▷ ▷ -group G \
12633 ▷ ▷ ▷ -group.action G \
12634 ▷ ▷ ▷ -generators G \
12635 ▷ ▷ ▷ -on_partition 2 Control \
12636 ▷ ▷ -end
12637
12638
12639 PGL_2.17_on_partition:
12640 ▷ $(ORBITER) -v 5 \
12641 ▷ ▷ -define Control -poset_classification_control \
12642 ▷ ▷ ▷ -problem_label PGL_2.17 \
12643 ▷ ▷ ▷ -depth 6 \
12644 ▷ ▷ ▷ -W \
12645 ▷ ▷ -end \
12646 ▷ ▷ -define G -linear_group -PGL 2 17 -end \
12647 ▷ ▷ -define Orb -orbits \
12648 ▷ ▷ ▷ -group G \
12649 ▷ ▷ ▷ -group.action G \
12650 ▷ ▷ ▷ -generators G \

```

```

12651 ▷ ▷ ▷ -on_partition 6 Control \
12652 ▷ ▷ -end
12653
12654
12655
12656 #####
12657 # Section 8.6: Arcs and Caps in Projective Spaces
12658
12659
12660 SECTION_ARCS_AND_CAPS_IN_PROJECTIVE_SPACES:
12661
12662
12663
12664 test_8.6:
12665 ▷ make PGL_3_27
12666 ▷ make hyperoval_4_classify
12667 ▷ make hyperoval_8_classify
12668 ▷ make hyperoval_16_classify
12669 ▷ make hyperoval_16_1_conic_type
12670 ▷ make hyperoval_16_1_nonconical_type
12671 ▷ make hyperoval_16_2_nonconical_type
12672 ▷ #make hyperoval_16_stab_0_disjoint_sets_graph
12673 ▷ #make nc_arcs_16
12674 ▷ make five_arcs_q13
12675
12676
12677
12678
12679 PGL_3_27:
12680 ▷ $(ORBITER) -v 5 \
12681 ▷ ▷ -define Do -draw_options \
12682 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
12683 ▷ ▷ -end \
12684 ▷ ▷ -define G \
12685 ▷ ▷ -linear_group -PGL 3 27 -end \
12686 ▷ ▷ -with G -do \
12687 ▷ ▷ -group_theoretic_activity \
12688 ▷ ▷ ▷ -report Do \
12689 ▷ ▷ -end
12690 ▷ $(PDFLATEX) PGL_3_27_report.tex
12691 ▷ $(OPEN) PGL_3_27_report.pdf
12692 ▷
12693
12694
12695
12696
12697
12698 hyperoval_4_classify:
12699 ▷ $(ORBITER) -v 4 \
12700 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
12701 ▷ ▷ -define F -finite_field -q 4 -end \
12702 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
12703 ▷ ▷ -define Control -poset_classification_control \
12704 ▷ ▷ ▷ -problem_label hyperoval.q4 \
12705 ▷ ▷ ▷ -W -depth 6 \
12706 ▷ ▷ -end \
12707 ▷ ▷ -define Arc_control -arc_generator_control \
12708 ▷ ▷ ▷ -projective_space P \
12709 ▷ ▷ ▷ -control Control \

```

```

12710 ▷ ▷ ▷ -target_size 6 \
12711 ▷ ▷ ▷ -d 2 \
12712 ▷ ▷ -end \
12713 ▷ ▷ -define Orb -orbits \
12714 ▷ ▷ ▷ -on_arcs Arc_control \
12715 ▷ ▷ -end
12716
12717 #hyperoval_q4_reps_lvl_6.csv:
12718 #Row,Line,Node,Level,OrbitIdx,OrbitRep,StabOrder,OrbitLength,SV.length
12719 #0,0,6,6,0,"0,1,2,3,16,19",720,168,0
12720 #END
12721
12722
12723 hyperoval_8_classify:
12724 ▷ $(ORBITER) -v 4 \
12725 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
12726 ▷ ▷ -define F -finite_field -q 8 -end \
12727 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
12728 ▷ ▷ -define Control -poset_classification_control \
12729 ▷ ▷ ▷ -problem_label hyperoval_q8 \
12730 ▷ ▷ ▷ -W -depth 10 \
12731 ▷ ▷ -end \
12732 ▷ ▷ -define Arc_control -arc_generator_control \
12733 ▷ ▷ ▷ -projective_space P \
12734 ▷ ▷ ▷ -control Control \
12735 ▷ ▷ ▷ -target_size 10 \
12736 ▷ ▷ ▷ -d 2 \
12737 ▷ ▷ -end \
12738 ▷ ▷ -define Orb -orbits \
12739 ▷ ▷ ▷ -on_arcs Arc_control \
12740 ▷ ▷ -end
12741
12742
12743
12744 hyperoval_16_classify:
12745 ▷ $(ORBITER) -v 4 \
12746 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
12747 ▷ ▷ -define F -finite_field -q 16 -end \
12748 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
12749 ▷ ▷ -define Control -poset_classification_control \
12750 ▷ ▷ ▷ -problem_label hyperoval_q16 \
12751 ▷ ▷ ▷ -W -depth 18 \
12752 ▷ ▷ -end \
12753 ▷ ▷ -define Arc_control -arc_generator_control \
12754 ▷ ▷ ▷ -projective_space P \
12755 ▷ ▷ ▷ -control Control \
12756 ▷ ▷ ▷ -target_size 18 \
12757 ▷ ▷ ▷ -d 2 \
12758 ▷ ▷ -end \
12759 ▷ ▷ -define Orb -orbits \
12760 ▷ ▷ ▷ -on_arcs Arc_control \
12761 ▷ ▷ -end
12762
12763
12764 hyperoval_16_1_conic_type:
12765 ▷ $(ORBITER) -v 2 \
12766 ▷ ▷ -define F -finite_field -q 16 -end \
12767 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
12768 ▷ ▷ -define H_16_1 -geometric_object P \

```



```

12769 ▷ ▷ ▷ -set "hyperoval_16_144" "hyperoval\_16\_144" \
12770 ▷ ▷ ▷ $(HYPEROVAL_16_144) \
12771 ▷ ▷ -end \
12772 ▷ ▷ -with H_16_1 -do \
12773 ▷ ▷ -combinatorial_object_activity \
12774 ▷ ▷ ▷ -save \
12775 ▷ ▷ -end \
12776 ▷ ▷ -with H_16_1 -do \
12777 ▷ ▷ -combinatorial_object_activity \
12778 ▷ ▷ ▷ -conic_type 6 \
12779 ▷ ▷ -end \
12780 ▷ ▷ -print_symbols
12781
12782 hyperoval_16_1_nonconical_type:
12783 ▷ $(ORBITER) -v 2 \
12784 ▷ ▷ -define F -finite_field -q 16 -end \
12785 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
12786 ▷ ▷ -define H_16_1 -geometric_object P \
12787 ▷ ▷ ▷ -set "hyperoval_16_144" "hyperoval\_16\_144" \
12788 ▷ ▷ ▷ $(HYPEROVAL_16_144) \
12789 ▷ ▷ -end \
12790 ▷ ▷ -with H_16_1 -do \
12791 ▷ ▷ -combinatorial_object_activity \
12792 ▷ ▷ ▷ -save \
12793 ▷ ▷ -end \
12794 ▷ ▷ -with H_16_1 -do \
12795 ▷ ▷ -combinatorial_object_activity \
12796 ▷ ▷ ▷ -non_conical_type \
12797 ▷ ▷ -end \
12798 ▷ ▷ -print_symbols
12799
12800
12801 #We found 17028 non-conical 6 subsets
12802 #Eckardt point number distribution : $13^{252},\,$ $9^{720},\,$ $5^{2304},\,$ $3^{\{13752\}}$
12803
12804
12805 hyperoval_16_2_nonconical_type:
12806 ▷ $(ORBITER) -v 2 \
12807 ▷ ▷ -define F -finite_field -q 16 -end \
12808 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
12809 ▷ ▷ -define H_16_2 -geometric_object P \
12810 ▷ ▷ ▷ -set "hyperoval_16_16320" "hyperoval\_16\_16320" \
12811 ▷ ▷ ▷ $(HYPEROVAL_16_16320) \
12812 ▷ ▷ -end \
12813 ▷ ▷ -with H_16_2 -do \
12814 ▷ ▷ -combinatorial_object_activity \
12815 ▷ ▷ ▷ -save \
12816 ▷ ▷ -end \
12817 ▷ ▷ -with H_16_2 -do \
12818 ▷ ▷ -combinatorial_object_activity \
12819 ▷ ▷ ▷ -non_conical_type \
12820 ▷ ▷ -end \
12821 ▷ ▷ -print_symbols
12822
12823 #We found 6188 = {17 choose 5} non-conical 6 subsets
12824 #Eckardt point number distribution : $45^{68},\,$ $13^{2040},\,$ $5^{4080}$
12825
12826 #neighbors_of_0_with_4_removed.csv

```

```

12827 #Row,C0,C1,C2,C3
12828 #0,2,3,9,10
12829 #1,1,3,7,8
12830 #2,10,12,13,15
12831 #3,1,5,10,11
12832 #4,3,5,6,13
12833 #5,8,9,11,12
12834 #6,7,11,13,17
12835 #7,7,10,14,16
12836 #8,1,9,13,16
12837 #9,2,8,13,14
12838 #10,1,2,15,17
12839 #11,6,8,10,17
12840 #12,6,7,9,15
12841 #13,2,6,11,16
12842 #14,5,9,14,17
12843 #15,5,8,15,16
12844 #16,1,6,12,14
12845 #17,2,5,7,12
12846 #18,3,12,16,17
12847 #19,3,11,14,15
12848 #END
12849
12850
12851 # ToDo neighbors_of_0_with_4_removed.csv is missing
12852
12853 hyperoval_16_stab_0_disjoint_sets_graph:
12854 ▷ $(ORBITER) -v 2 \
12855 ▷ ▷ -define G -graph -disjoint_sets_graph \
12856 ▷ ▷ ▷ neighbors_of_0_with_4_removed.csv \
12857 ▷ ▷ -end \
12858 ▷ ▷ -with G -do \
12859 ▷ ▷ ▷ -graph_theoretic_activity \
12860 ▷ ▷ ▷ -find_cliques \
12861 ▷ ▷ ▷ ▷ -target_size 4 \
12862 ▷ ▷ ▷ -end \
12863 ▷ ▷ -end \
12864 ▷ ▷ -print_symbols
12865
12866
12867 # 5 cliques of size 4
12868 #ROW,C0,C1,C2,C3
12869 #0,0,6,15,16
12870 #1,1,2,13,14
12871 #2,3,9,12,18
12872 #3,4,5,7,10
12873 #4,8,11,17,19
12874 #END
12875
12876 #clique 0:
12877 #0,2,3,9,10
12878 #6,7,11,13,17
12879 #15,5,8,15,16
12880 #16,1,6,12,14
12881 # partition: (1,6,12,14|2,3,9,10|5,8,15,16|7,11,13,17)
12882 # 4 is missing, it is the nucleus
12883 # 0 is missing is the chosen point
12884
12885

```

```

12886
12887
12888
12889
12890
12891 # nonconical 6-arcs are used for classifying cubic surfaces:
12892
12893
12894
12895
12896
12897 nc_arcs_16:
12898 ▷ $(ORBITER) -v 4 \
12899 ▷ ▷ -define F -finite_field -q 16 -end \
12900 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
12901 ▷ ▷ -define Control -poset_classification_control \
12902 ▷ ▷ ▷ -problem_label nc_arcs_q16_d2 \
12903 ▷ ▷ ▷ -W -depth 6 \
12904 ▷ ▷ -end \
12905 ▷ ▷ -define Arc_control -arc_generator_control \
12906 ▷ ▷ ▷ -projective_space P \
12907 ▷ ▷ ▷ -control Control \
12908 ▷ ▷ ▷ -target_size 6 \
12909 ▷ ▷ ▷ -d 2 \
12910 ▷ ▷ -end \
12911 ▷ ▷ -define Orb -orbits \
12912 ▷ ▷ ▷ -on_arcs Arc_control \
12913 ▷ ▷ -end
12914 ▷ #$(PDFLATEX) nc_arcs_q16_d2_poset.tex
12915 ▷ #$(OPEN) nc_arcs_q16_d2_poset.pdf
12916
12917
12918 five_arcs_q13:
12919 ▷ $(ORBITER) -v 4 \
12920 ▷ ▷ -define F -finite_field -q 13 -end \
12921 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
12922 ▷ ▷ -define Control -poset_classification_control \
12923 ▷ ▷ ▷ -problem_label five_arcs_q13 -W -depth 5 \
12924 ▷ ▷ -end \
12925 ▷ ▷ -define Arc_control -arc_generator_control \
12926 ▷ ▷ ▷ -projective_space P \
12927 ▷ ▷ ▷ -control Control \
12928 ▷ ▷ ▷ -target_size 5 \
12929 ▷ ▷ ▷ -d 2 \
12930 ▷ ▷ -end \
12931 ▷ ▷ -define Orb -orbits \
12932 ▷ ▷ ▷ -on_arcs Arc_control \
12933 ▷ ▷ -end
12934
12935
12936
12937
12938
12939
12940
12941 #####
12942 # Section 8.7: Cubic Curves
12943
12944 SECTION_CUBIC_CURVES:

```

```

12945
12946
12947
12948 test_8_7:
12949 ▷ make cubic_curves_PG_2_4
12950 ▷ make poly_orbits_q4
12951 ▷ #make cubic_curves_PG_2_9
12952 ▷ #make cubic_curves_PG_2_9_draw
12953 ▷ make curve_orbit_mini
12954 ▷ #make curve_orbit # too slow
12955
12956
12957
12958 cubic_curves_PG_2_4:
12959 ▷ $(ORBITER) -v 3 \
12960 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
12961 ▷ ▷ -define Do -draw_options \
12962 ▷ ▷ ▷ -radius 200 -embedded \
12963 ▷ ▷ -end \
12964 ▷ ▷ -define F -finite_field -q 4 -end \
12965 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
12966 ▷ ▷ -define Control -poset_classification_control \
12967 ▷ ▷ ▷ -problem_label cc_4 -W -depth 9 \
12968 ▷ ▷ -end \
12969 ▷ ▷ -define Arc_control -arc_generator_control \
12970 ▷ ▷ ▷ -projective_space P \
12971 ▷ ▷ ▷ -control Control \
12972 ▷ ▷ ▷ -target_size 9 \
12973 ▷ ▷ ▷ -d 3 \
12974 ▷ ▷ -end \
12975 ▷ ▷ -define Orb -orbits \
12976 ▷ ▷ ▷ -on_cubic_curves Arc_control \
12977 ▷ ▷ -end \
12978 ▷ ▷ -define ro \
12979 ▷ ▷ ▷ -poset_classification_report_options \
12980 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
12981 ▷ ▷ ▷ -draw_options Do \
12982 ▷ ▷ -end \
12983 ▷ ▷ -with Orb -do -orbits_activity \
12984 ▷ ▷ ▷ -report "" \
12985 ▷ ▷ ▷ -report_options ro \
12986 ▷ ▷ -end
12987 ▷ ▷ -with Orb -do -orbits_activity \
12988 ▷ ▷ ▷ -draw_tree 0 Do \
12989 ▷ ▷ -end
12990 ▷ #$(PDFLATEX) cc_4_poset_lvl_9.tex
12991 ▷ #$(OPEN) cc_4_poset_lvl_9.pdf
12992 ▷ $(PDFLATEX) Cubic_curves_q4.tex
12993 ▷ $(OPEN) Cubic_curves_q4.pdf
12994
12995
12996
12997 poly_orbits_q4:
12998 ▷ $(ORBITER) -v 4 \
12999 ▷ ▷ -define Do -draw_options \
13000 ▷ ▷ ▷ -yout 500000 -radius 15 -nodes_empty \
13001 ▷ ▷ ▷ -line_width 0.5 -y_stretch 0.25 -embedded -end \
13002 ▷ ▷ -define F -finite_field -q 4 -print_numerically -end \
13003 ▷ ▷ -define R -polynomial_ring \

```

```

13004 ▷ ▷ ▷ -field F \
13005 ▷ ▷ ▷ -number_of_variables 3 \
13006 ▷ ▷ ▷ -homogeneous_of_degree 3 \
13007 ▷ ▷ ▷ -monomial_ordering_partition \
13008 ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
13009 ▷ ▷ ▷ -end \
13010 ▷ ▷ -define G -linear_group -PGGL 3 F -end \
13011 ▷ ▷ -define Orb -orbits -group G \
13012 ▷ ▷ ▷ -print_interval 1000000 \
13013 ▷ ▷ ▷ -on_polynomials R \
13014 ▷ ▷ -end \
13015 ▷ ▷ -define ro \
13016 ▷ ▷ ▷ -poset_classification_report_options \
13017 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
13018 ▷ ▷ ▷ ▷ -draw_options Do \
13019 ▷ ▷ -end \
13020 ▷ ▷ -with Orb -do -orbits_activity \
13021 ▷ ▷ ▷ -report "" \
13022 ▷ ▷ ▷ -report_options ro \
13023 ▷ ▷ -end \
13024 ▷ ▷ -with Orb -do -orbits_activity \
13025 ▷ ▷ ▷ -export_something "orbit" 0 \
13026 ▷ ▷ -end
13027 ▷ $(PDFLATEX) poly_orbits.d3.n2.q4.tex
13028 ▷ $(OPEN) poly_orbits.d3.n2.q4.pdf
13029
13030 #orbits_PGGL_3_4_orbit_0.csv
13031
13032
13033 # ToDo: error in cubic_curve::compute_inflexion_points
13034 #cubic_curve::compute_inflexion_points rank of line spanned by v and w is not two

13035
13036 cubic_curves_PG_2_9:
13037 ▷ $(ORBITER) -v 3 \
13038 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
13039 ▷ ▷ -define Do -draw_options \
13040 ▷ ▷ ▷ -radius 200 -embedded \
13041 ▷ ▷ -end \
13042 ▷ ▷ -define F -finite_field -q 9 -end \
13043 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
13044 ▷ ▷ -define Control -poset_classification_control \
13045 ▷ ▷ ▷ -problem_label cc_9 -W -depth 9 \
13046 ▷ ▷ -end \
13047 ▷ ▷ -define Arc_control -arc_generator_control \
13048 ▷ ▷ -projective_space P \
13049 ▷ ▷ ▷ -control Control \
13050 ▷ ▷ ▷ -target_size 9 \
13051 ▷ ▷ ▷ -d 3 \
13052 ▷ ▷ -end \
13053 ▷ ▷ -define Orb -orbits \
13054 ▷ ▷ ▷ -on_cubic_curves Arc_control \
13055 ▷ ▷ -end \
13056 ▷ ▷ -define ro \
13057 ▷ ▷ ▷ -poset_classification_report_options \
13058 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
13059 ▷ ▷ ▷ ▷ -draw_options Do \
13060 ▷ ▷ -end \
13061 ▷ ▷ -with Orb -do -orbits_activity \

```

```

13062 ▷ ▷ ▷ -report "" \
13063 ▷ ▷ ▷ -report_options ro \
13064 ▷ ▷ -end
13065 ▷ ▷ -with Orb -do -orbits_activity \
13066 ▷ ▷ ▷ -draw_tree 0 Do \
13067 ▷ ▷ -end
13068 ▷ #$(PDFLATEX) cc_9_poset_lvl_9.tex
13069 ▷ #$(OPEN) cc_9_poset_lvl_9.pdf
13070
13071 #cubic_curve::compute_inflexion_points rank of line spanned by v and w is not two

13072
13073
13074 # too big to latex:
13075
13076 cubic_curves_PG_2_9.draw:
13077 ▷ $(ORBITER) -v 3 \
13078 ▷ ▷ -draw_layered_graph cc_9_poset_lvl_9.layered_graph \
13079 ▷ ▷ ▷ -place \
13080 ▷ ▷ ▷ -radius 300 -embedded -line_width 1.1 \
13081 ▷ ▷ ▷ -y_stretch 0.9 -scale 0.25 \
13082 ▷ ▷ -end
13083 ▷ #$(PDFLATEX) cc_9_poset_lvl_9.draw.tex
13084 ▷ #$(OPEN) cc_9_poset_lvl_9.draw.pdf
13085
13086
13087 curve_orbit_mini:
13088 ▷ $(ORBITER) -v 4 \
13089 ▷ ▷ -define F -finite_field -q 3 -print_numerically -end \
13090 ▷ ▷ -define R -polynomial_ring \
13091 ▷ ▷ ▷ -field F \
13092 ▷ ▷ ▷ -number_of_variables 3 \
13093 ▷ ▷ ▷ -homogeneous_of_degree 3 \
13094 ▷ ▷ ▷ -monomial_ordering_partition \
13095 ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
13096 ▷ ▷ ▷ -end \
13097 ▷ ▷ -define cubic -symbolic_object \
13098 ▷ ▷ ▷ -field F \
13099 ▷ ▷ ▷ -text "X0^3+X1^3+X2^3" \
13100 ▷ ▷ ▷ -end \
13101 ▷ ▷ -define G -linear_group -PGL 3 F -end \
13102 ▷ ▷ -define Orb -orbits -group G \
13103 ▷ ▷ ▷ -of_one_polynomial R cubic \
13104 ▷ ▷ -end \
13105 ▷ ▷ -with Orb -do -orbits_activity \
13106 ▷ ▷ ▷ -export_something "orbit" 0 \
13107 ▷ ▷ -end
13108
13109
13110 curve_orbit:
13111 ▷ $(ORBITER) -v 4 \
13112 ▷ ▷ -define F -finite_field -q 9 -print_numerically -end \
13113 ▷ ▷ -define R -polynomial_ring \
13114 ▷ ▷ ▷ -field F \
13115 ▷ ▷ ▷ -number_of_variables 3 \
13116 ▷ ▷ ▷ -homogeneous_of_degree 3 \
13117 ▷ ▷ ▷ -monomial_ordering_partition \
13118 ▷ ▷ ▷ -variables "X0,X1,X2" "X_0,X_1,X_2" \
13119 ▷ ▷ ▷ -end \

```

```

13120 ▷ ▷ -define cubic -symbolic_object \
13121 ▷ ▷ ▷ -field F \
13122 ▷ ▷ ▷ -text $(SURFACE_F9_1_EQN_RESTRICTED) \
13123 ▷ ▷ ▷ -end \
13124 ▷ ▷ -define G -linear_group -PGL 3 F -end \
13125 ▷ ▷ -define Orb -orbits -group G \
13126 ▷ ▷ ▷ -of_one_polynomial R cubic \
13127 ▷ ▷ -end \
13128 ▷ ▷ -with Orb -do -orbits_activity \
13129 ▷ ▷ ▷ -export_something "orbit" 0 \
13130 ▷ ▷ -end
13131
13132
13133 #orbits.on.polynomials::orbit_of_one_polynomial found an orbit of length 589680
13134 #7:15
13135
13136
13137
13138
13139 #####
13140 # Chapter 9 - Cubic Surfaces
13141 #####
13142
13143
13144 test_9:
13145 ▷ make test_9.1
13146 ▷ make test_9.2
13147 ▷ make test_9.3
13148 ▷ make test_9.4
13149 ▷ make test_9.5
13150 ▷ make test_9.6
13151 ▷ make test_9.7
13152
13153
13154
13155
13156
13157 #####
13158 # Section 9.1: Global Commands for Cubic Surfaces
13159
13160
13161 SECTION_CUBIC_SURFACES_GLOBAL:
13162
13163
13164
13165 test_9.1:
13166 ▷ make surface_4.0_lines_vs_lines
13167 ▷ make surface_4.0_lines_vs_tritangent_planes
13168 ▷ make surface_4.0_dot_products_columns
13169 ▷ make surface_4.0_export_double_sixes
13170 ▷ make surface_4.0_multiply_half_double_six_vs_lines
13171 ▷ make surface_4.0_multiply_half_double_six_vs_incma
13172 ▷ make surface_4.0_dot_products_rows
13173 ▷ make surface_4.0_dot_products_rowspan
13174 ▷ make surface_4.0_single_six_times_lines_vs_lines_rowspan
13175
13176
13177 surface_4.0_lines_vs_lines:
13178 ▷ $(ORBITER) -v 3 \

```

```

13179 ▷ ▷ -define F -finite_field -q 4 -end \
13180 ▷ ▷ -define P -projective_space -n 3 -field F -end \
13181 ▷ ▷ -with P -do \
13182 ▷ ▷ -projective_space_activity \
13183 ▷ ▷ ▷ -export_cubic_surface_line_vs_line_incidence_matrix \
13184 ▷ ▷ -end
13185 ▷ $(ORBITER) -v 2 -draw_matrix \
13186 ▷ ▷ -input_csv_file PG_3.4.lines_vs_lines_incma.csv \
13187 ▷ ▷ -box_width 32 -bit_depth 24 \
13188 ▷ ▷ -partition 3 27 27 \
13189 ▷ ▷ -end
13190 ▷ magick PG_3.4.lines_vs_lines_incma.draw.bmp PG_3.4.lines_vs_lines_incma.draw.png
13191 g
13192
13193 # PG_3.4.lines_vs_lines_incma.csv
13194
13195
13196 surface_4.0.lines_vs_tritangent_planes:
13197 ▷ $(ORBITER) -v 3 \
13198 ▷ ▷ -define F -finite_field -q 4 -end \
13199 ▷ ▷ -define P -projective_space -n 3 -field F -end \
13200 ▷ ▷ -with P -do \
13201 ▷ ▷ -projective_space_activity \
13202 ▷ ▷ ▷ -export_cubic_surface_line_tritangent_plane_incidence_matrix \
13203 ▷ ▷ -end
13204 ▷ $(ORBITER) -v 2 -draw_matrix \
13205 ▷ ▷ -input_csv_file PG_3.4.lines_tritplanes_incma.csv \
13206 ▷ ▷ -box_width 32 -bit_depth 24 \
13207 ▷ ▷ -partition 3 27 45 \
13208 ▷ ▷ -end
13209 ▷ magick PG_3.4.lines_tritplanes_incma.draw.bmp PG_3.4.lines_tritplanes_incma.draw.png
13210 w.png
13211
13212
13213
13214 surface_4.0.dot_products_columns:
13215 ▷ $(ORBITER) -v 3 \
13216 ▷ ▷ -define line_vs_trit -vector \
13217 ▷ ▷ ▷ -file PG_3.4.lines_tritplanes_incma.csv \
13218 ▷ ▷ -end \
13219 ▷ ▷ -dot_product_of_columns line_vs_trit
13220 ▷ $(ORBITER) -v 2 -draw_matrix \
13221 ▷ ▷ -input_csv_file line_vs_trit.dot_products_columns.csv \
13222 ▷ ▷ -box_width 32 -bit_depth 24 \
13223 ▷ ▷ -partition 3 45 45 \
13224 ▷ ▷ -end
13225 ▷ magick line_vs_trit.dot_products_columns.draw.bmp \
13226 ▷ ▷ line_vs_trit.dot_products_columns.draw.png
13227
13228
13229
13230 surface_4.0.export_double_sixes:
13231 ▷ $(ORBITER) -v 3 \
13232 ▷ ▷ -define F -finite_field -q 4 -end \
13233 ▷ ▷ -define P -projective_space -n 3 -field F -end \
13234 ▷ ▷ -with P -do \
13235 ▷ ▷ -projective_space_activity \

```



```

13236 ▷ ▷ ▷ -export_double_sixes \
13237 ▷ ▷ -end \
13238
13239
13240 surface_4_0_multiply_half_double_six_vs_lines:
13241 ▷ $(ORBITER) -v 3 \
13242 ▷ ▷ -define lines_vs_lines -vector \
13243 ▷ ▷ ▷ -file PG_3_4_lines_vs_lines_incma.csv \
13244 ▷ ▷ -end \
13245 ▷ ▷ -define single_six -vector \
13246 ▷ ▷ ▷ -file PG_3_4_single_sixes_char_vec.csv \
13247 ▷ ▷ -end \
13248 ▷ ▷ -matrix_multiply_over_Z single_six lines_vs_lines
13249
13250 # single_six_times_lines_vs_lines.csv
13251
13252 surface_4_0_multiply_half_double_six_vs_incma:
13253 ▷ $(ORBITER) -v 3 \
13254 ▷ ▷ -define line_vs_trit -vector \
13255 ▷ ▷ ▷ -file PG_3_4_lines_tritplanes_incma.csv \
13256 ▷ ▷ -end \
13257 ▷ ▷ -define single_six -vector \
13258 ▷ ▷ ▷ -file PG_3_4_single_sixes_char_vec.csv \
13259 ▷ ▷ -end \
13260 ▷ ▷ -matrix_multiply_over_Z single_six line_vs_trit
13261
13262 # 72 x 45:
13263 #single_six_times_line_vs_trit.csv
13264
13265 surface_4_0_dot_products_rows:
13266 ▷ $(ORBITER) -v 3 \
13267 ▷ ▷ -define single_six_vs_trit_planes -vector \
13268 ▷ ▷ ▷ -file single_six_times_line_vs_trit.csv \
13269 ▷ ▷ -end \
13270 ▷ ▷ -dot_product_of_rows single_six_vs_trit_planes
13271
13272 # 72 x 72:
13273 #single_six_vs_trit_planes_dot_products_rows.csv
13274
13275 surface_4_0_dot_products_rowspan:
13276 ▷ $(ORBITER) -v 3 \
13277 ▷ ▷ -define single_six_vs_trit_planes -vector \
13278 ▷ ▷ ▷ -file single_six_times_line_vs_trit.csv \
13279 ▷ ▷ -end \
13280 ▷ ▷ -rowspan_over_R single_six_vs_trit_planes
13281
13282 surface_4_0_single_six_times_lines_vs_lines_rowspan:
13283 ▷ $(ORBITER) -v 3 \
13284 ▷ ▷ -define M -vector \
13285 ▷ ▷ ▷ -file single_six_times_lines_vs_lines.csv \
13286 ▷ ▷ -end \
13287 ▷ ▷ -rowspan_over_R M
13288
13289
13290
13291
13292 #####
13293 # Section 9.2: Cubic Surfaces Creation
13294

```

```

13295
13296 SECTION.CUBIC_SURFACES_CREATION:
13297
13298
13299
13300 test.9.2:
13301 ▷ make surface_4.0
13302 ▷ make Hirschfeld_surface.q4
13303 ▷ make Family_general.F7
13304 ▷ make surface_eckardt_13_4_12
13305 ▷ make Eckardt_13
13306 ▷ make surface_11_random
13307 ▷ make surface_F_abcd_Eckardt_q31_by_equation
13308 ▷ make Eckardt_13_by_arc_lifting
13309 ▷ make surface_F_abcd_Eckardt_q31_by_symbolic_object
13310 ▷ make Hirschfeld_surface_by_arc_lifting_with_two_lines
13311 ▷ make Dickson6_q2
13312 ▷ make Dickson1_group
13313 ▷ make Dickson2_group
13314
13315
13316
13317 surface_4.0:
13318 ▷ $(ORBITER) -v 3 \
13319 ▷ ▷ -define F -finite_field -q 4 -end \
13320 ▷ ▷ -define P -projective_space -n 3 -field F -end \
13321 ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
13322
13323
13324
13325 Hirschfeld_surface.q4:
13326 ▷ $(ORBITER) -v 6 \
13327 ▷ ▷ -define Do -draw_options \
13328 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
13329 ▷ ▷ ▷ -end \
13330 ▷ ▷ -define F -finite_field -q 4 -compute_related_fields -end \
13331 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
13332 ▷ ▷ -define R -polynomial_ring \
13333 ▷ ▷ ▷ -field F \
13334 ▷ ▷ ▷ -number_of_variables 4 \
13335 ▷ ▷ ▷ -homogeneous_of_degree 3 \
13336 ▷ ▷ ▷ -monomial_ordering_partition \
13337 ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
13338 ▷ ▷ -end \
13339 ▷ ▷ -define f -symbolic_object \
13340 ▷ ▷ ▷ -field F \
13341 ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
13342 ▷ ▷ ▷ -text $(HIRSCHFELD_SURFACE_EQN_AF) \
13343 ▷ ▷ -end \
13344 ▷ ▷ -define Surf -cubic_surface \
13345 ▷ ▷ ▷ -space P \
13346 ▷ ▷ ▷ -by_symbolic_object R f \
13347 ▷ ▷ -end \
13348 ▷ ▷ -with Surf -do \
13349 ▷ ▷ -cubic_surface_activity \
13350 ▷ ▷ ▷ -report Do \
13351 ▷ ▷ -end
13352 ▷ pdflatex surface.equation-f.q4.report.tex
13353 ▷ open surface.equation-f.q4.report.pdf

```

```

13354
13355
13356
13357 Family_general_F7:
13358 ▷ $(ORBITER) -v 10 \
13359 ▷ ▷ -define F -finite_field -q 7 -end \
13360 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
13361 ▷ ▷ -define S7_abcd_2_3_3_4 -cubic_surface \
13362 ▷ ▷ ▷ -space P -family_general_abcd 2 3 3 4 \
13363 ▷ ▷ -end
13364
13365
13366 surface_eckardt_13_4_12:
13367 ▷ $(ORBITER) -v 6 \
13368 ▷ ▷ -define F -finite_field -q 13 -end \
13369 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
13370 ▷ ▷ -define Eckardt_4_12 -cubic_surface \
13371 ▷ ▷ ▷ -space P -family_Eckardt 4 12 \
13372 ▷ ▷ -end
13373
13374
13375 Eckardt_13:
13376 ▷ $(ORBITER) -v 3 \
13377 ▷ ▷ -define F -finite_field -q 13 -end \
13378 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
13379 ▷ ▷ -define Eckardt_3_1 -cubic_surface \
13380 ▷ ▷ ▷ -space P -family_Eckardt 3 1 \
13381 ▷ ▷ -end
13382
13383
13384 surface_11_random:
13385 ▷ $(ORBITER) -v 10 \
13386 ▷ ▷ -define F -finite_field -q 11 -end \
13387 ▷ ▷ -define P -projective_space -n 3 -field F -v 6 -end \
13388 ▷ ▷ -define S -cubic_surface -space P -random -end
13389
13390
13391
13392 surface_F_abcd_Eckardt_q31_by_equation:
13393 ▷ $(ORBITER) -v 10 \
13394 ▷ ▷ -define F -finite_field -q 31 -end \
13395 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
13396 ▷ ▷ -define R -polynomial_ring \
13397 ▷ ▷ ▷ -field F \
13398 ▷ ▷ ▷ -number_of_variables 4 \
13399 ▷ ▷ ▷ -homogeneous_of_degree 3 \
13400 ▷ ▷ ▷ -monomial_ordering_partition \
13401 ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
13402 ▷ ▷ ▷ -end \
13403 ▷ ▷ -define F_abcd -cubic_surface -space P \
13404 ▷ ▷ ▷ -by_equation \
13405 ▷ ▷ ▷ R \
13406 ▷ ▷ ▷ "F_abcd" \
13407 ▷ ▷ ▷ "\DF_{a,b,c,d}\D" "X0,X1,X2,X3" \
13408 ▷ ▷ ▷ $(F_abcd_eqn) \
13409 ▷ ▷ ▷ "a,b,c,d" \
13410 ▷ ▷ ▷ "\D(a,b,c,d)=(2,30,30,2)\D" \
13411 ▷ ▷ ▷ "2,30,30,2" \
13412 ▷ ▷ -end

```

```

13413
13414 Eckardt_13.by_arc_lifting:
13415 ▷ $(ORBITER) -v 3 \
13416 ▷ ▷ -define F -finite_field -q 13 -end \
13417 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
13418 ▷ ▷ -define S -cubic_surface -space P \
13419 ▷ ▷ ▷ -arc_lifting_with_triangular_pair "0,1,2,3,43,113" \
13420 ▷ ▷ -end
13421
13422
13423
13424
13425
13426 surface_F_abcd_Eckardt_q31.by_symbolic_object:
13427 ▷ $(ORBITER) -v 10 \
13428 ▷ ▷ -define F -finite_field -q 31 -end \
13429 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
13430 ▷ ▷ -define R -polynomial_ring \
13431 ▷ ▷ ▷ -field F \
13432 ▷ ▷ ▷ -number_of_variables 4 \
13433 ▷ ▷ ▷ -homogeneous_of_degree 3 \
13434 ▷ ▷ ▷ -monomial_ordering_partition \
13435 ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
13436 ▷ ▷ ▷ -end \
13437 ▷ ▷ -define F_abcd -symbolic_object \
13438 ▷ ▷ ▷ -field F \
13439 ▷ ▷ ▷ -text $(F_abcd_eqn) \
13440 ▷ ▷ ▷ -end \
13441 ▷ ▷ -define abcd_values -symbolic_object \
13442 ▷ ▷ ▷ -field F \
13443 ▷ ▷ ▷ -text "2,30,30,2" \
13444 ▷ ▷ ▷ -end \
13445 ▷ ▷ -define F_abcd_sub -symbolic_object \
13446 ▷ ▷ ▷ -field F \
13447 ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
13448 ▷ ▷ ▷ -substitute "a,b,c,d" F_abcd abcd_values \
13449 ▷ ▷ -end \
13450 ▷ ▷ -define S -cubic_surface -space P \
13451 ▷ ▷ ▷ -by_symbolic_object \
13452 ▷ ▷ ▷ R \
13453 ▷ ▷ ▷ F_abcd_sub \
13454 ▷ ▷ -end
13455
13456
13457
13458 Hirschfeld_surface.by_arc_lifting_with_two_lines:
13459 ▷ $(ORBITER) -v 3 \
13460 ▷ ▷ -define F -finite_field -q 4 -end \
13461 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
13462 ▷ ▷ -define Surf -cubic_surface -space P \
13463 ▷ ▷ ▷ -arc_lifting_with_two_lines "0,1,2,12,18,21" "0,0" \
13464 ▷ ▷ -end \
13465 ▷ ▷ -define Do -draw_options \
13466 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
13467 ▷ ▷ -end \
13468 ▷ ▷ -with Surf -do \
13469 ▷ ▷ -cubic_surface_activity \
13470 ▷ ▷ ▷ -report Do \
13471 ▷ ▷ -end \

```

```

13472 ▷ ▷ -print_symbols
13473 ▷ pdflatex surface_arc_lifting_with_two_lines.q4.lines_0_0.arc_0_1.2.12.18.21_rep
    ort.tex
13474 ▷ open surface_arc_lifting_with_two_lines.q4.lines_0_0.arc_0_1.2.12.18.21_report.
    pdf
13475
13476
13477
13478
13479
13480
13481 Dickson6.q2:
13482 ▷ $(ORBITER) -v 10 \
13483 ▷ ▷ -define Do -draw_options \
13484 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
13485 ▷ ▷ -end \
13486 ▷ ▷ -define F -finite_field -q 2 -end \
13487 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
13488 ▷ ▷ -define S -cubic_surface \
13489 ▷ ▷ ▷ -space P -by_coefficients $(Dickson6) \
13490 ▷ ▷ -end \
13491 ▷ ▷ -with S -do \
13492 ▷ ▷ -cubic_surface_activity \
13493 ▷ ▷ ▷ -report Do \
13494 ▷ ▷ -end
13495 ▷ $(PDFLATEX) surface_by_coefficients_q2_report.tex
13496 ▷ $(OPEN) surface_by_coefficients_q2_report.pdf
13497
13498
13499 Dickson1_group:
13500 ▷ $(ORBITER) -v 6 \
13501 ▷ ▷ -define F -finite_field -q 2 -end \
13502 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
13503 ▷ ▷ -define R -polynomial_ring \
13504 ▷ ▷ ▷ -field F \
13505 ▷ ▷ ▷ -number_of_variables 4 \
13506 ▷ ▷ ▷ -homogeneous_of_degree 3 \
13507 ▷ ▷ ▷ -monomial_ordering_partition \
13508 ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
13509 ▷ ▷ -end \
13510 ▷ ▷ -define V -variety \
13511 ▷ ▷ ▷ -projective_space P \
13512 ▷ ▷ ▷ -ring R \
13513 ▷ ▷ ▷ -equation_in_algebraic_form \
13514 ▷ ▷ ▷ ▷ $(Dickson1_eqn.af) \
13515 ▷ ▷ ▷ -label_txt Dickson_1 \
13516 ▷ ▷ ▷ -label_tex "${\rm Dickson\1}" \
13517 ▷ ▷ -end \
13518 ▷ ▷ -with V -do -variety_activity \
13519 ▷ ▷ ▷ -compute_group \
13520 ▷ ▷ ▷ -nauty_control -nauty_log "Dickson1.nauty_log.txt" -save_orbit_of_equations
    eqn_ -end \
13521 ▷ ▷ -end \
13522 ▷ ▷ -with V -do -variety_activity \
13523 ▷ ▷ ▷ -compute_set_stabilizer \
13524 ▷ ▷ ▷ -nauty_control -save_orbit_of_equations eqn_ -end \
13525 ▷ ▷ -end \
13526 ▷ ▷ -with V -do -variety_activity -singular_points -end \
13527 ▷ ▷ -with V -do -variety_activity -report -end

```

```

13528 ▷ $(PDFLATEX) variety_Dickson_1_report.tex
13529 ▷ $(OPEN) variety_Dickson_1_report.pdf
13530
13531
13532
13533 Dickson2.group:
13534 ▷ $(ORBITER) -v 6 \
13535 ▷ ▷ -define F -finite_field -q 2 -end \
13536 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
13537 ▷ ▷ -define R -polynomial_ring \
13538 ▷ ▷ ▷ -field F \
13539 ▷ ▷ ▷ -number_of_variables 4 \
13540 ▷ ▷ ▷ -homogeneous_of_degree 3 \
13541 ▷ ▷ ▷ -monomial_ordering_partition \
13542 ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
13543 ▷ ▷ -end \
13544 ▷ ▷ -define V -variety \
13545 ▷ ▷ ▷ -projective_space P \
13546 ▷ ▷ ▷ -ring R \
13547 ▷ ▷ ▷ -equation_in_algebraic_form \
13548 ▷ ▷ ▷ ▷ $(Dickson2_eqn_af) \
13549 ▷ ▷ ▷ -label_txt Dickson_2 \
13550 ▷ ▷ ▷ -label_tex "{\\rm Dickson\\_2}" \
13551 ▷ ▷ -end \
13552 ▷ ▷ -with V -do -variety_activity \
13553 ▷ ▷ ▷ -compute_group \
13554 ▷ ▷ ▷ -nauty_control -nauty_log "Dickson2_nauty_log.txt" -save_orbit_of_equations
    eqn_ -end \
13555 ▷ ▷ -end \
13556 ▷ ▷ -with V -do -variety_activity -singular_points -end \
13557 ▷ ▷ -with V -do -variety_activity -report -end
13558 ▷ $(PDFLATEX) variety_Dickson_2_report.tex
13559 ▷ $(OPEN) variety_Dickson_2_report.pdf
13560
13561
13562
13563 #####
13564 # Section 9.3: Cubic Surface Activities
13565
13566
13567 SECTION.CUBIC.SURFACE.ACTIVITIES:
13568
13569
13570
13571 test_9.3:
13572 ▷ make surface_4.0_report
13573 ▷ make surface_4.0_clean_report
13574 ▷ make surface_4.0_export
13575 ▷ make Hirschfeld_surface.get_incidence_matrix.40.40
13576 ▷ make Hirschfeld_surface.incma.40.40.c
13577 ▷ make surface_4.0_plane_type_of_lines_on_Klein_quadric
13578 ▷ make surface_7.0
13579 ▷ #make surface_7.0_plane_type_of_lines_on_Klein_quadric # slow
13580 ▷ make Family_general_F7_report
13581 ▷ make surface_8.0
13582 ▷ #make surface_8.0_plane_type_of_lines_on_Klein_quadric # slow
13583 ▷ make surface_8.0_catalogue
13584 ▷ make surface_8.0_clean
13585 ▷ make surface_8.0b

```

```

13586 ▷ make surface_9.0
13587 ▷ #make surface_9.0_plane_type_of_lines_on_Klein_quadric # slow
13588 ▷ make surface_9.1
13589 ▷ #make surface_9.1_plane_type_of_lines_on_Klein_quadric # slow
13590 ▷ make surface_eckardt_13.4.12_report
13591 ▷ make Eckardt_13_report
13592 ▷ make Eckardt_13_by_arc_lifting_report
13593 ▷ make surface_13.0
13594 ▷ make surface_16.0
13595 ▷ make G13.8
13596 ▷ make F13.8
13597 ▷ make F13.16
13598 ▷ make F13.32
13599 ▷ #make F13.64a
13600 ▷ #make F13.64b
13601 ▷ #make Colorado1
13602 ▷ #make Colorado2
13603 ▷ #make Colorado3
13604 ▷ #make F13.128a
13605 ▷ #make F13.128b
13606 ▷ #make F13.128c
13607 ▷ make move_two_lines
13608 ▷ make surface_F_abcd_Eckardt_q31_by_symbolic_object_report
13609 ▷ make surface_11_random_report
13610 ▷ make surface_11.1
13611 ▷ make surface_8.0_recognize_abcd
13612
13613
13614
13615
13616
13617 surface_4.0_report:
13618 ▷ $(ORBITER) -v 3 \
13619 ▷ ▷ -define Do -draw_options \
13620 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
13621 ▷ ▷ ▷ -end \
13622 ▷ ▷ -define F -finite_field -q 4 -end \
13623 ▷ ▷ -define P -projective_space -n 3 -field F -end \
13624 ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
13625 ▷ ▷ -with S -do \
13626 ▷ ▷ -cubic_surface_activity \
13627 ▷ ▷ ▷ -report Do \
13628 ▷ ▷ -end
13629 ▷ $(PDFLATEX) surface_catalogue_q4_iso0_report.tex
13630 ▷ $(OPEN) surface_catalogue_q4_iso0_report.pdf
13631
13632
13633 surface_4.0_clean_report:
13634 ▷ $(ORBITER) -v 3 \
13635 ▷ ▷ -define Do -draw_options \
13636 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
13637 ▷ ▷ ▷ -end \
13638 ▷ ▷ -define F -finite_field -q 4 -end \
13639 ▷ ▷ -define P -projective_space -n 3 -field F -end \
13640 ▷ ▷ -define S -cubic_surface -space P -catalogue 0 \
13641 ▷ ▷ ▷ -select_double_six "a4,b4,c15,c25,c35,c56,a5,b5,c14,c24,c34,c46" \
13642 ▷ ▷ ▷ -select_double_six "a1,a6,a2,a3,a4,a5,b1,b6,b2,b3,b4,b5" \
13643 ▷ ▷ ▷ -select_double_six "a1,a2,a5,a6,a3,a4,b1,b2,b5,b6,b3,b4" \
13644 ▷ ▷ -end \

```

```

13645 ▷ ▷ -with S -do \
13646 ▷ ▷ -cubic_surface_activity \
13647 ▷ ▷ ▷ -report Do \
13648 ▷ ▷ -end
13649 ▷ $(PDFLATEX) surface_catalogue_q4_iso0_report.tex
13650 ▷ $(OPEN) surface_catalogue_q4_iso0_report.pdf
13651
13652
13653
13654
13655 surface_4.0.export:
13656 ▷ $(ORBITER) -v 3 \
13657 ▷ ▷ -define F -finite_field -q 4 -end \
13658 ▷ ▷ -define P -projective_space -n 3 -field F -end \
13659 ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
13660 ▷ ▷ -with S -do \
13661 ▷ ▷ -cubic_surface_activity \
13662 ▷ ▷ ▷ -export_something "points" \
13663 ▷ ▷ -end \
13664 ▷ ▷ -with S -do \
13665 ▷ ▷ -cubic_surface_activity \
13666 ▷ ▷ ▷ -export_something "points_off" \
13667 ▷ ▷ -end \
13668 ▷ ▷ -with S -do \
13669 ▷ ▷ -cubic_surface_activity \
13670 ▷ ▷ ▷ -export_something "lines" \
13671 ▷ ▷ -end \
13672 ▷ ▷ -with S -do \
13673 ▷ ▷ -cubic_surface_activity \
13674 ▷ ▷ ▷ -export_something "Hesse_planes" \
13675 ▷ ▷ -end
13676
13677
13678 Hirschfeld_surface_get_incidence_matrix_40.40:
13679 ▷ $(ORBITER) -v 3 \
13680 ▷ ▷ -define points -vector -dense $(HIRSCHFELD_SURFACE_POINTS_OFF) -end \
13681 ▷ ▷ -define planes -vector -dense $(HIRSCHFELD_SURFACE_HESSSE_PLANES) -end \
13682 ▷ ▷ -define F -finite_field -q 4 -end \
13683 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
13684 ▷ ▷ -with P -do \
13685 ▷ ▷ -projective_space_activity \
13686 ▷ ▷ ▷ -restricted_incidence_matrix 1 3 points planes "H_incma_40.40" \
13687 ▷ ▷ -end
13688
13689
13690
13691 Hirschfeld_surface_incma_40.40.c:
13692 ▷ $(ORBITER) -v 5 \
13693 ▷ ▷ -define Doinc -draw_incidence_structure_options \
13694 ▷ ▷ ▷ -width 40 -width_10 6 \
13695 ▷ ▷ -end \
13696 ▷ ▷ -define C -combinatorial_object \
13697 ▷ ▷ ▷ -label H_incma_40.40 H\incma\40\40 \
13698 ▷ ▷ ▷ -file_of_incidence_geometries H_incma_40.40.inc 40 40 480 \
13699 ▷ ▷ -end \
13700 ▷ ▷ -with C -do \
13701 ▷ ▷ -combinatorial_object_activity \
13702 ▷ ▷ ▷ -canonical_form \
13703 ▷ ▷ ▷ ▷ -save_ago \

```



```

13704 ▷ ▷ ▷ ▷ -save_transversal \
13705 ▷ ▷ ▷ ▷ -nauty_control -end \
13706 ▷ ▷ ▷ -end \
13707 ▷ ▷ -end \
13708 ▷ ▷ -with C -do \
13709 ▷ ▷ -combinatorial_object_activity \
13710 ▷ ▷ ▷ -post_processing \
13711 ▷ ▷ -end \
13712 ▷ ▷ -with C -do \
13713 ▷ ▷ -combinatorial_object_activity \
13714 ▷ ▷ ▷ -report \
13715 ▷ ▷ ▷ ▷ -export_flag_orbits \
13716 ▷ ▷ ▷ ▷ -show_incidence_matrices \
13717 ▷ ▷ ▷ ▷ -export_group_GAP \
13718 ▷ ▷ ▷ ▷ -export_group_magma \
13719 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
13720 ▷ ▷ ▷ -end \
13721 ▷ ▷ -end
13722 ▷ $(ORBITER) -v 2 -draw_matrix \
13723 ▷ ▷ -input_csv_file H_incma_40_40_object_0_TDA_flag_orbits.csv \
13724 ▷ ▷ -secondary_input_csv_file H_incma_40_40_object_0_TDA.csv \
13725 ▷ ▷ -box_width 32 -bit_depth 24 \
13726 ▷ ▷ -end
13727 ▷ $(PDFLATEX) H_incma_40_40_classification.tex
13728 ▷ $(OPEN) H_incma_40_40_classification.pdf
13729
13730
13731
13732 surface_4_0_plane_type_of_lines_on_Klein_quadric:
13733 ▷ $(ORBITER) -v 6 \
13734 ▷ ▷ -define L -vector -file surface_catalogue_q4_iso0_lines.csv -end \
13735 ▷ ▷ -define F -finite_field -q 4 -end \
13736 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
13737 ▷ ▷ -with P -do \
13738 ▷ ▷ -projective_space_activity \
13739 ▷ ▷ ▷ -plane_intersection_type_of_klein_image 4 L \
13740 ▷ ▷ -end
13741
13742 #The plane intersection type is (0^244496, 1^112914, 2^15120, 3^4005, 5^270)
13743 #The plane intersection type is (5^270)
13744 #L_highest_weight_objects.csv
13745
13746
13747
13748
13749 surface_7_0:
13750 ▷ $(ORBITER) -v 3 \
13751 ▷ ▷ -define Do -draw_options \
13752 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
13753 ▷ ▷ -end \
13754 ▷ ▷ -define F -finite_field -q 7 -end \
13755 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
13756 ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
13757 ▷ ▷ -with S -do \
13758 ▷ ▷ -cubic_surface_activity \
13759 ▷ ▷ ▷ -report Do \
13760 ▷ ▷ -end \
13761 ▷ ▷ -with S -do \
13762 ▷ ▷ -cubic_surface_activity \

```

```

13763 ▷ ▷ ▷ -export_something "lines" \
13764 ▷ ▷ -end
13765 ▷ $(PDFLATEX) surface_catalogue_q7_iso0_report.tex
13766 ▷ $(OPEN) surface_catalogue_q7_iso0_report.pdf
13767
13768
13769
13770 surface_7_0_plane_type_of_lines_on_Klein_quadric:
13771 ▷ $(ORBITER) -v 6 \
13772 ▷ ▷ -define L_7_0 -vector \
13773 ▷ ▷ -file surface_catalogue_q7_iso0_lines.csv -end \
13774 ▷ ▷ -define F -finite_field -q 7 -end \
13775 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
13776 ▷ ▷ -with P -do \
13777 ▷ ▷ -projective_space_activity \
13778 ▷ ▷ ▷ -plane_intersection_type_of_klein_image 4 L_7_0 \
13779 ▷ ▷ -end
13780
13781 # Time: 4:50
13782 #The plane intersection type is (0^44526575, 1^3529170, 2^112563, 3^8541, 4^324,
13783 #The plane intersection type is (4^324, 5^27)
13784 #L_7_0_highest_weight_objects.csv
13785
13786
13787 Family_general_F7_report:
13788 ▷ $(ORBITER) -v 3 \
13789 ▷ ▷ -define Do -draw_options \
13790 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
13791 ▷ ▷ -end \
13792 ▷ ▷ -define F -finite_field -q 7 -end \
13793 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
13794 ▷ ▷ -define S7_abcd_2_3_3_4 -cubic_surface \
13795 ▷ ▷ ▷ -space P -family_general_abcd 2 3 3 4 \
13796 ▷ ▷ -end \
13797 ▷ ▷ -with S7_abcd_2_3_3_4 -do \
13798 ▷ ▷ -cubic_surface_activity \
13799 ▷ ▷ ▷ -report Do \
13800 ▷ ▷ -end
13801 ▷ $(PDFLATEX) surface_F_a2_b3_c4_d4_report.tex
13802 ▷ $(OPEN) surface_F_a2_b3_c4_d4_report.pdf
13803
13804
13805 # Fermat with 18 Eckardt points
13806 # no automorphism group
13807
13808
13809 surface_8_0:
13810 ▷ $(ORBITER) -v 3 \
13811 ▷ ▷ -define Do -draw_options \
13812 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
13813 ▷ ▷ -end \
13814 ▷ ▷ -define F -finite_field -q 8 -end \
13815 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
13816 ▷ ▷ -define P2 -projective_space -n 2 -field F -v 0 -end \
13817 ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
13818 ▷ ▷ -define Control -poset_classification_control \
13819 ▷ ▷ ▷ -problem_label nc_arcs_q8 \
13820 ▷ ▷ ▷ -W -depth 6 \

```

```

13821 ▷ ▷ -end \
13822 ▷ ▷ -define Arc_control -arc_generator_control \
13823 ▷ ▷ ▷ -projective_space P2 \
13824 ▷ ▷ ▷ -control Control \
13825 ▷ ▷ ▷ -target_size 6 \
13826 ▷ ▷ ▷ -d 2 \
13827 ▷ ▷ -end \
13828 ▷ ▷ -define Orb -orbits \
13829 ▷ ▷ ▷ -on_arcs Arc_control \
13830 ▷ ▷ -end \
13831 ▷ ▷ -with S -do \
13832 ▷ ▷ -cubic_surface_activity \
13833 ▷ ▷ ▷ -report Do \
13834 ▷ ▷ -end \
13835 ▷ ▷ -with S -do \
13836 ▷ ▷ -cubic_surface_activity \
13837 ▷ ▷ ▷ -report_all_flag_orbits Orb \
13838 ▷ ▷ -end \
13839 ▷ ▷ -with S -do \
13840 ▷ ▷ -cubic_surface_activity \
13841 ▷ ▷ ▷ -export_something "lines" \
13842 ▷ ▷ -end
13843
13844
13845 # slow:
13846
13847 surface.8.0_plane_type_of_lines_on_Klein_quadric:
13848 ▷ $(ORBITER) -v 6 \
13849 ▷ ▷ -define L8.0 -vector -file surface_catalogue_q8_iso0_lines.csv -end \
13850 ▷ ▷ -define F -finite_field -q 8 -end \
13851 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
13852 ▷ ▷ -with P -do \
13853 ▷ ▷ -projective_space_activity \
13854 ▷ ▷ ▷ -plane_intersection_type_of_klein_image 4 L8.0 \
13855 ▷ ▷ -end
13856
13857 #L8.0_highest_weight_objects.csv
13858 #intersection numbers: ( 0, 0, 0, 0, 192, 30 )
13859 # time: 15:48
13860
13861
13862
13863
13864
13865
13866 surface.8.0_catalogue:
13867 ▷ $(ORBITER) -v 3 \
13868 ▷ ▷ -define Do -draw_options \
13869 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
13870 ▷ ▷ -end \
13871 ▷ ▷ -define F -finite_field -q 8 -end \
13872 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
13873 ▷ ▷ -define S8.0 -cubic_surface -space P -catalogue 0 -end \
13874 ▷ ▷ -with S8.0 -do \
13875 ▷ ▷ -cubic_surface_activity \
13876 ▷ ▷ ▷ -report Do \
13877 ▷ ▷ -end
13878 ▷ $(PDFLATEX) surface_catalogue_q8_iso0_report.tex
13879 ▷ $(OPEN) surface_catalogue_q8_iso0_report.pdf

```

```

13880
13881
13882
13883 surface_8_0_clean:
13884 ▷ $(ORBITER) -v 3 \
13885 ▷ ▷ -define Do -draw.options \
13886 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
13887 ▷ ▷ -end \
13888 ▷ ▷ -define F -finite_field -q 8 -end \
13889 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
13890 ▷ ▷ -define S8_0 -cubic_surface -space P -catalogue 0 \
13891 ▷ ▷ ▷ -select_double_six "15,11,22,19,24,5,16,10,23,20,25,4" \
13892 ▷ ▷ ▷ -select_double_six "3,2,1,0,5,4,9,8,7,6,11,10" \
13893 ▷ ▷ ▷ -transform_inverse "1,4,4,0,6,0,0,0,6,2,0,1,7,0,4,0,0" \
13894 ▷ ▷ ▷ -transform "4,4,0,0, 0,0,1,0, 1,0,0,0, 0,0,0,1, 0" \
13895 ▷ ▷ ▷ -transform_inverse "2,0,0,0,0,2,0,0,0,0,2,0,1,1,2,3,0" \
13896 ▷ ▷ ▷ -end \
13897 ▷ ▷ -with S8_0 -do \
13898 ▷ ▷ -cubic_surface_activity \
13899 ▷ ▷ ▷ -report Do \
13900 ▷ ▷ -end
13901 ▷ $(PDFLATEX) surface_catalogue_q8_iso0_report.tex
13902 ▷ $(OPEN) surface_catalogue_q8_iso0_report.pdf
13903
13904
13905
13906 # clean equation for Tekirdag-1:
13907
13908 surface_8_0b:
13909 ▷ $(ORBITER) -v 3 \
13910 ▷ ▷ -define Do -draw.options \
13911 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
13912 ▷ ▷ -end \
13913 ▷ ▷ -define F -finite_field -q 8 -end \
13914 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
13915 ▷ ▷ -define S8_0 -cubic_surface -space P -catalogue 0 \
13916 ▷ ▷ ▷ -select_double_six "15,11,22,19,24,5,16,10,23,20,25,4" \
13917 ▷ ▷ ▷ -select_double_six "3,2,1,0,5,4,9,8,7,6,11,10" \
13918 ▷ ▷ ▷ -transform "1,0,0,0,0,1,0,6,0,0,1,6,0,0,0,1,0" \
13919 ▷ ▷ ▷ -transform_inverse "3,1,1,0,0,1,0,0,0,0,1,0,0,0,0,1,0" \
13920 ▷ ▷ ▷ -transform_inverse "2,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,0" \
13921 ▷ ▷ -end \
13922 ▷ ▷ -with S8_0 -do \
13923 ▷ ▷ -cubic_surface_activity \
13924 ▷ ▷ ▷ -report Do \
13925 ▷ ▷ -end
13926
13927
13928
13929
13930
13931
13932 surface_9_0:
13933 ▷ $(ORBITER) -v 3 \
13934 ▷ ▷ -define Do -draw.options \
13935 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
13936 ▷ ▷ -end \
13937 ▷ ▷ -define F -finite_field -q 9 -end \
13938 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \

```

```

13939 ▷ ▷ -define P2 -projective_space -n 2 -field F -v 0 -end \
13940 ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
13941 ▷ ▷ -define Control -poset_classification_control \
13942 ▷ ▷ ▷ -problem_label nc_arcs_q9 \
13943 ▷ ▷ ▷ -W -depth 6 \
13944 ▷ ▷ -end \
13945 ▷ ▷ -define Arc_control -arc_generator_control \
13946 ▷ ▷ ▷ -projective_space P2 \
13947 ▷ ▷ ▷ -control Control \
13948 ▷ ▷ ▷ -target_size 6 \
13949 ▷ ▷ ▷ -d 2 \
13950 ▷ ▷ -end \
13951 ▷ ▷ -define Orb -orbits \
13952 ▷ ▷ ▷ -on_arcs Arc_control \
13953 ▷ ▷ -end \
13954 ▷ ▷ -with S -do \
13955 ▷ ▷ -cubic_surface_activity \
13956 ▷ ▷ ▷ -report Do \
13957 ▷ ▷ -end \
13958 ▷ ▷ -with S -do \
13959 ▷ ▷ -cubic_surface_activity \
13960 ▷ ▷ ▷ -report_all_flag_orbits Orb \
13961 ▷ ▷ -end \
13962 ▷ ▷ -with S -do \
13963 ▷ ▷ -cubic_surface_activity \
13964 ▷ ▷ ▷ -export_something "lines" \
13965 ▷ ▷ -end
13966
13967
13968 # slow:
13969
13970 surface_9_0_plane_type_of_lines_on_Klein_quadric:
13971 ▷ $(ORBITER) -v 6 \
13972 ▷ ▷ -define L_9_0 -vector -file surface_catalogue.q9.iso0_lines.csv -end \
13973 ▷ ▷ -define F -finite_field -q 9 -end \
13974 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
13975 ▷ ▷ -with P -do \
13976 ▷ ▷ -projective_space_activity \
13977 ▷ ▷ ▷ -plane_intersection_type_of_klein_image 4 L_9_0 \
13978 ▷ ▷ -end
13979
13980
13981 #intersection numbers: ( 0, 0, 0, 0, 180, 15 )
13982 #time: 44:42
13983
13984
13985 surface_9_1:
13986 ▷ $(ORBITER) -v 3 \
13987 ▷ ▷ -define Do -draw_options \
13988 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
13989 ▷ ▷ -end \
13990 ▷ ▷ -define F -finite_field -q 9 -end \
13991 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
13992 ▷ ▷ -define P2 -projective_space -n 2 -field F -v 0 -end \
13993 ▷ ▷ -define S -cubic_surface -space P -catalogue 1 -end \
13994 ▷ ▷ -define Control -poset_classification_control \
13995 ▷ ▷ ▷ -problem_label nc_arcs_q9 \
13996 ▷ ▷ ▷ -W -depth 6 \
13997 ▷ ▷ -end \

```

```

13998 ▷ ▷ -define Arc_control -arc.generator.control \
13999 ▷ ▷ ▷ -projective_space P2 \
14000 ▷ ▷ ▷ -control Control \
14001 ▷ ▷ ▷ -target_size 6 \
14002 ▷ ▷ ▷ -d 2 \
14003 ▷ ▷ -end \
14004 ▷ ▷ -define Orb -orbits \
14005 ▷ ▷ ▷ -on_arcs Arc_control \
14006 ▷ ▷ -end \
14007 ▷ ▷ -with S -do \
14008 ▷ ▷ -cubic_surface_activity \
14009 ▷ ▷ ▷ -report Do \
14010 ▷ ▷ -end \
14011 ▷ ▷ -with S -do \
14012 ▷ ▷ -cubic_surface_activity \
14013 ▷ ▷ ▷ -report_all_flag_orbits Orb \
14014 ▷ ▷ -end \
14015 ▷ ▷ -with S -do \
14016 ▷ ▷ -cubic_surface_activity \
14017 ▷ ▷ ▷ -export_something "lines" \
14018 ▷ ▷ -end
14019 ▷ $(PDFLATEX) surface.catalogue.q9_iso1_report.tex
14020 ▷ $(OPEN) surface.catalogue.q9_iso1_report.pdf
14021
14022
14023 # slow:
14024
14025
14026 surface_9_1_plane_type_of_lines_on_Klein_quadric:
14027 ▷ $(ORBITER) -v 6 \
14028 ▷ ▷ -define L_9_1 -vector -file surface.catalogue.q9_iso1_lines.csv -end \
14029 ▷ ▷ -define F -finite_field -q 9 -end \
14030 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14031 ▷ ▷ -with P -do \
14032 ▷ ▷ -projective_space_activity \
14033 ▷ ▷ ▷ -plane_intersection_type_of_klein_image 4 L_9_1 \
14034 ▷ ▷ -end
14035
14036 #intersection numbers: ( 0, 0, 0, 0, 216 )
14037
14038 #44:36
14039
14040
14041
14042 surface_eckardt_13_4_12_report:
14043 ▷ $(ORBITER) -v 6 \
14044 ▷ ▷ -define Do -draw_options \
14045 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14046 ▷ ▷ -end \
14047 ▷ ▷ -define F -finite_field -q 13 -end \
14048 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14049 ▷ ▷ -define Eckardt_4_12 -cubic_surface \
14050 ▷ ▷ ▷ -space P -family_Eckardt 4 12 \
14051 ▷ ▷ -end \
14052 ▷ ▷ -with Eckardt_4_12 -do \
14053 ▷ ▷ -cubic_surface_activity \
14054 ▷ ▷ ▷ -report Do \
14055 ▷ ▷ -end
14056 ▷ $(PDFLATEX) surface.Eckardt.alpha12.beta12_report.tex

```

```

14057 ▷ $(OPEN) surface.Eckardt.alpha12.beta12.report.pdf
14058
14059
14060
14061
14062
14063
14064 # 13_0 has 4 Eckardt points
14065 # 13_1 has 6 Eckardt points
14066 # 13_2 has 9 Eckardt points
14067 # 13_3 has 18 Eckardt points
14068 #
14069
14070
14071
14072
14073 Eckardt_13_report:
14074 ▷ $(ORBITER) -v 3 \
14075 ▷ ▷ -define Do -draw.options \
14076 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14077 ▷ ▷ -end \
14078 ▷ ▷ -define F -finite_field -q 13 -end \
14079 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14080 ▷ ▷ -define Eckardt_3_1 -cubic_surface \
14081 ▷ ▷ ▷ -space P -family_Eckardt 3 1 \
14082 ▷ ▷ -end \
14083 ▷ ▷ -with Eckardt_3_1 -do \
14084 ▷ ▷ -cubic_surface_activity \
14085 ▷ ▷ ▷ -report Do \
14086 ▷ ▷ -end
14087 ▷ $(PDFLATEX) surface.Eckardt.alpha12.beta12.report.tex
14088 ▷ $(OPEN) surface.Eckardt.alpha12.beta12.report.pdf
14089
14090
14091 Eckardt_13_by_arc_lifting_report:
14092 ▷ $(ORBITER) -v 3 \
14093 ▷ ▷ -define Do -draw.options \
14094 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14095 ▷ ▷ -end \
14096 ▷ ▷ -define F -finite_field -q 13 -end \
14097 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14098 ▷ ▷ -define S -cubic_surface -space P \
14099 ▷ ▷ ▷ -arc_lifting_with_triangular_pair "0,1,2,3,43,113" \
14100 ▷ ▷ -end \
14101 ▷ ▷ -with S -do \
14102 ▷ ▷ -cubic_surface_activity \
14103 ▷ ▷ ▷ -report Do \
14104 ▷ ▷ -end
14105 ▷ $(PDFLATEX) surface_arc_lifting_triangular.q13_arc_0.1.2.3.43.113_report.tex
14106 ▷ $(OPEN) surface_arc_lifting_triangular.q13_arc_0.1.2.3.43.113_report.pdf
14107
14108
14109 surface_13_0:
14110 ▷ $(ORBITER) -v 3 \
14111 ▷ ▷ -define Do -draw.options \
14112 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14113 ▷ ▷ -end \
14114 ▷ ▷ -define F -finite_field -q 13 -end \
14115 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \

```

```

14116 ▷ ▷ -define S13_0 -cubic_surface -space P -catalogue 0 -end \
14117 ▷ ▷ -with S13_0 -do \
14118 ▷ ▷ -cubic_surface_activity \
14119 ▷ ▷ ▷ -report Do \
14120 ▷ ▷ -end
14121 ▷ $(PDFLATEX) surface_catalogue_q13_iso0_report.tex
14122 ▷ $(OPEN) surface_catalogue_q13_iso0_report.pdf
14123
14124
14125 # clean equation for Tekirdag-2:
14126
14127
14128
14129 surface_16_0:
14130 ▷ $(ORBITER) -v 3 \
14131 ▷ ▷ -define Do -draw_options \
14132 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14133 ▷ ▷ -end \
14134 ▷ ▷ -define F -finite_field -q 16 -end \
14135 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14136 ▷ ▷ -define S16_0 -cubic_surface -space P -catalogue 0 \
14137 ▷ ▷ ▷ -transform "1,0,0,0,0,1,0,12,0,0,1,12,0,0,0,1,0" \
14138 ▷ ▷ ▷ -transform "15,11,4,0,0,0,12,0,0,12,0,0,0,0,0,1,3" \
14139 ▷ ▷ -end \
14140 ▷ ▷ -with S16_0 -do \
14141 ▷ ▷ -cubic_surface_activity \
14142 ▷ ▷ ▷ -report Do \
14143 ▷ ▷ -end
14144 ▷ $(PDFLATEX) surface_catalogue_q16_iso0_report.tex
14145 ▷ $(OPEN) surface_catalogue_q16_iso0_report.pdf
14146
14147
14148 #▷ ▷ -transform_inverse "3,0,0,0,0,1,1,0,0,0,1,0,0,0,0,1,0" \
14149 #▷ ▷ -transform_inverse "13,12,1,0,12,13,1,0,0,0,1,0,0,0,0,1,0" \
14150 #▷ ▷ -transform_inverse "1,0,0,0,0,1,0,0,12,12,1,0,0,0,0,1,0" \
14151 #▷ ▷ -transform_inverse "12,0,0,0,0,12,0,0,0,0,1,0,0,0,0,1,0" \
14152
14153 # rank of lines ( 66591, 26737, 4093, 69904, 28376, 26470, 70160, 69855, 26208, 5
      847, 369, 32230, 529, 30293, 70068, 2178, 261, 28666, 8575, 105, 31694, 0, 51784,
      25209, 22193, 49862, 274 )
14154 # Rank of points on Klein quadric: ( 29181, 4677, 29950, 33, 62496, 429, 1, 9205,
      37, 29964, 29364, 21501, 4656, 54735, 5425, 30105, 754, 6680, 13354, 758, 30106,
      0, 29209, 48736, 25595, 33780, 4657 )
14155
14156
14157 # ai: 29181, 4677, 29950, 33, 62496, 429
14158 # bi: 1, 9205, 37, 29964, 29364, 21501
14159
14160 # Tekirdag-1:
14161
14162 G13_8:
14163 ▷ $(ORBITER) -v 3 \
14164 ▷ ▷ -define Do -draw_options \
14165 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14166 ▷ ▷ -end \
14167 ▷ ▷ -define F -finite_field -q 8 -end \
14168 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14169 ▷ ▷ -define T1 -cubic_surface -space P -family_G13 2 -end \
14170 ▷ ▷ -with T1 -do \

```



```

14171 ▷ ▷ -cubic_surface_activity \
14172 ▷ ▷ ▷ -report Do \
14173 ▷ ▷ -end
14174 ▷ $(PDFLATEX) surface.G13.a2.report.tex
14175 ▷ $(OPEN) surface.G13.a2.report.pdf
14176
14177
14178 F13.8:
14179 ▷ $(ORBITER) -v 3 \
14180 ▷ ▷ -define Do -draw_options \
14181 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14182 ▷ ▷ -end \
14183 ▷ ▷ -define F -finite_field -q 8 -end \
14184 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14185 ▷ ▷ -define T1 -cubic_surface -space P -family_F13 2 -end \
14186 ▷ ▷ -with T1 -do \
14187 ▷ ▷ -cubic_surface_activity \
14188 ▷ ▷ ▷ -report Do \
14189 ▷ ▷ -end
14190 ▷ $(PDFLATEX) surface.F13.a2.report.tex
14191 ▷ $(OPEN) surface.F13.a2.report.pdf
14192
14193
14194
14195
14196 # Tekirdag-2:
14197
14198 F13.16:
14199 ▷ $(ORBITER) -v 3 \
14200 ▷ ▷ -define Do -draw_options \
14201 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14202 ▷ ▷ -end \
14203 ▷ ▷ -define F -finite_field -q 16 -end \
14204 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14205 ▷ ▷ -define T2 -cubic_surface -space P -family_F13 2 -end \
14206 ▷ ▷ -with T2 -do \
14207 ▷ ▷ -cubic_surface_activity \
14208 ▷ ▷ ▷ -report Do \
14209 ▷ ▷ -end
14210
14211
14212 # Tekirdag-3:
14213
14214 F13.32:
14215 ▷ $(ORBITER) -v 3 \
14216 ▷ ▷ -define Do -draw_options \
14217 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14218 ▷ ▷ -end \
14219 ▷ ▷ -define F -finite_field -q 32 -end \
14220 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14221 ▷ ▷ -define T3 -cubic_surface -space P -family_F13 2 -end \
14222 ▷ ▷ -with T3 -do \
14223 ▷ ▷ -cubic_surface_activity \
14224 ▷ ▷ ▷ -report Do \
14225 ▷ ▷ -end
14226
14227
14228 # Kapadokya-1:
14229

```

```

14230 F13.64a:
14231 ▷ $(ORBITER) -v 3 \
14232 ▷ ▷ -define Do -draw_options \
14233 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14234 ▷ ▷ -end \
14235 ▷ ▷ -define F -finite_field -q 64 -end \
14236 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14237 ▷ ▷ -define K1 -cubic_surface -space P -family_F13 2 -end \
14238 ▷ ▷ -with K1 -do \
14239 ▷ ▷ -cubic_surface_activity \
14240 ▷ ▷ ▷ -report Do \
14241 ▷ ▷ -end
14242
14243
14244 # Kapadokya-2:
14245
14246 F13.64b:
14247 ▷ $(ORBITER) -v 3 \
14248 ▷ ▷ -define Do -draw_options \
14249 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14250 ▷ ▷ -end \
14251 ▷ ▷ -define F -finite_field -q 64 -end \
14252 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14253 ▷ ▷ -define K2 -cubic_surface -space P -family_F13 18 -end \
14254 ▷ ▷ -with K2 -do \
14255 ▷ ▷ -cubic_surface_activity \
14256 ▷ ▷ ▷ -report Do \
14257 ▷ ▷ -end
14258
14259
14260 Colorado1:
14261 ▷ $(ORBITER) -v 3 \
14262 ▷ ▷ -define Do -draw_options \
14263 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14264 ▷ ▷ -end \
14265 ▷ ▷ -define F -finite_field -q 128 -end \
14266 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14267 ▷ ▷ -define CO-1 -cubic_surface -space P -catalogue 0 \
14268 ▷ ▷ ▷ -transform_inverse "1,0,0,0,0,1,0,96,0,0,1,96,0,0,0,1,0" \
14269 ▷ ▷ -end \
14270 ▷ ▷ -with CO-1 -do \
14271 ▷ ▷ -cubic_surface_activity \
14272 ▷ ▷ ▷ -report Do \
14273 ▷ ▷ -end
14274
14275
14276 # recognize the arcs from Colorado-1,2,3:
14277
14278
14279
14280 Colorado2:
14281 ▷ $(ORBITER) -v 3 \
14282 ▷ ▷ -define Do -draw_options \
14283 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14284 ▷ ▷ -end \
14285 ▷ ▷ -define F -finite_field -q 128 -end \
14286 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14287 ▷ ▷ -define CO-2 -cubic_surface -space P -catalogue 926 \
14288 ▷ ▷ ▷ -transform_inverse "1,0,0,0,0,1,0,32,0,0,1,32,0,0,0,1,0" \

```

```

14289 ▷ ▷ -end \
14290 ▷ ▷ -with CO-2 -do \
14291 ▷ ▷ -cubic_surface_activity \
14292 ▷ ▷ ▷ -report Do \
14293 ▷ ▷ -end
14294
14295 Colorado3:
14296 ▷ $(ORBITER) -v 3 \
14297 ▷ ▷ -define Do -draw.options \
14298 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14299 ▷ ▷ -end \
14300 ▷ ▷ -define F -finite_field -q 128 -end \
14301 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14302 ▷ ▷ -define CO-3 -cubic_surface -space P -catalogue 928 \
14303 ▷ ▷ ▷ -transform_inverse "1,0,0,0,0,1,0,59,0,0,1,59,0,0,0,1,0" \
14304 ▷ ▷ -end \
14305 ▷ ▷ -with CO-3 -do \
14306 ▷ ▷ -cubic_surface_activity \
14307 ▷ ▷ ▷ -report Do \
14308 ▷ ▷ -end
14309
14310
14311 # Colorado-1:
14312
14313 F13_128a:
14314 ▷ $(ORBITER) -v 3 \
14315 ▷ ▷ -define Do -draw.options \
14316 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14317 ▷ ▷ -end \
14318 ▷ ▷ -define F -finite_field -q 128 -end \
14319 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14320 ▷ ▷ -define CO-1 -cubic_surface -space P -family_F13 2 -end \
14321 ▷ ▷ -with CO-1 -do \
14322 ▷ ▷ -cubic_surface_activity \
14323 ▷ ▷ ▷ -report Do \
14324 ▷ ▷ -end
14325
14326 # Colorado-2:
14327
14328 F13_128b:
14329 ▷ $(ORBITER) -v 3 \
14330 ▷ ▷ -define Do -draw.options \
14331 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14332 ▷ ▷ -end \
14333 ▷ ▷ -define F -finite_field -q 128 -end \
14334 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14335 ▷ ▷ -define CO-2 -cubic_surface -space P -family_F13 6 -end \
14336 ▷ ▷ -with CO-2 -do \
14337 ▷ ▷ -cubic_surface_activity \
14338 ▷ ▷ ▷ -report Do \
14339 ▷ ▷ -end
14340
14341 # Colorado-3:
14342
14343 F13_128c:
14344 ▷ $(ORBITER) -v 3 \
14345 ▷ ▷ -define Do -draw.options \
14346 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14347 ▷ ▷ -end \

```

```

14348 ▷ ▷ -define F -finite.field -q 128 -end \
14349 ▷ ▷ -define P -projective.space -n 3 -field F -v 0 -end \
14350 ▷ ▷ -define CO-3 -cubic.surface -space P -family_F13 14 -end \
14351 ▷ ▷ -with CO-3 -do \
14352 ▷ ▷ -cubic.surface.activity \
14353 ▷ ▷ ▷ -report Do \
14354 ▷ ▷ -end
14355
14356
14357 move_two_lines:
14358 ▷ $(ORBITER) -v 5 \
14359 ▷ ▷ -define F -finite.field -q 8 -end \
14360 ▷ ▷ -define P -projective.space -n 3 -field F -v 0 -end \
14361 ▷ ▷ -with P -do -projective.space.activity \
14362 ▷ ▷ ▷ -move_two_lines_in_hyperplane_stabilizer \
14363 ▷ ▷ ▷ 65 4680 72 657 \
14364 ▷ ▷ -end
14365
14366
14367
14368 surface_F_abcd_Eckardt_q31_by_symbolic_object_report:
14369 ▷ $(ORBITER) -v 7 \
14370 ▷ ▷ -define Do -draw.options \
14371 ▷ ▷ ▷ -radius 200 -line.width 0.3 -nodes.empty \
14372 ▷ ▷ -end \
14373 ▷ ▷ -define F -finite.field -q 31 -end \
14374 ▷ ▷ -define P -projective.space -n 3 -field F -end \
14375 ▷ ▷ -define R -polynomial.ring \
14376 ▷ ▷ ▷ -field F \
14377 ▷ ▷ ▷ -number_of_variables 4 \
14378 ▷ ▷ ▷ -homogeneous_of_degree 3 \
14379 ▷ ▷ ▷ -monomial_ordering_partition \
14380 ▷ ▷ ▷ -variables "X0,X1,X2,X3" "X_0,X_1,X_2,X_3" \
14381 ▷ ▷ ▷ -end \
14382 ▷ ▷ -define F_abcd -symbolic.object \
14383 ▷ ▷ ▷ -field F \
14384 ▷ ▷ ▷ -text $(F_abcd_eqn) \
14385 ▷ ▷ ▷ -end \
14386 ▷ ▷ -define abcd_values -symbolic.object \
14387 ▷ ▷ ▷ -field F \
14388 ▷ ▷ ▷ -text "2,30,30,2" \
14389 ▷ ▷ ▷ -end \
14390 ▷ ▷ -define F_abcd_sub -symbolic.object \
14391 ▷ ▷ ▷ -field F \
14392 ▷ ▷ ▷ -managed_variables "X0,X1,X2,X3" \
14393 ▷ ▷ ▷ -substitute "a,b,c,d" F_abcd abcd_values \
14394 ▷ ▷ -end \
14395 ▷ ▷ -define S -cubic.surface -space P \
14396 ▷ ▷ ▷ -by_symbolic.object \
14397 ▷ ▷ ▷ R \
14398 ▷ ▷ ▷ F_abcd_sub \
14399 ▷ ▷ -end \
14400 ▷ ▷ -with S -do \
14401 ▷ ▷ -cubic.surface.activity \
14402 ▷ ▷ ▷ -report Do \
14403 ▷ ▷ -end
14404 ▷ $(PDFLATEX) surface.equation_F_abcd_sub_q31_report.tex
14405 ▷ $(OPEN) surface.equation_F_abcd_sub_q31_report.pdf
14406

```

```

14407
14408 surface_11_random_report:
14409 ▷ $(ORBITER) -v 10 \
14410 ▷ ▷ -define Do -draw_options \
14411 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14412 ▷ ▷ -end \
14413 ▷ ▷ -define F -finite_field -q 11 -end \
14414 ▷ ▷ -define P -projective_space -n 3 -field F -v 6 -end \
14415 ▷ ▷ -define S -cubic_surface -space P -random -end \
14416 ▷ ▷ -with S -do \
14417 ▷ ▷ -cubic_surface_activity \
14418 ▷ ▷ ▷ -report Do \
14419 ▷ ▷ -end
14420 ▷ $(PDFLATEX) surface_random_q11_report.tex
14421 ▷ $(OPEN) surface_random_q11_report.pdf
14422
14423
14424 surface_11_1:
14425 ▷ $(ORBITER) -v 3 \
14426 ▷ ▷ -define Do -draw_options \
14427 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14428 ▷ ▷ -end \
14429 ▷ ▷ -define F -finite_field -q 11 -end \
14430 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14431 ▷ ▷ -define S -cubic_surface -space P -catalogue 1 -end \
14432 ▷ ▷ -with S -do \
14433 ▷ ▷ -cubic_surface_activity \
14434 ▷ ▷ ▷ -report Do \
14435 ▷ ▷ -end
14436 ▷ $(PDFLATEX) surface_catalogue_q11_isol_report.tex
14437 ▷ $(OPEN) surface_catalogue_q11_isol_report.pdf
14438
14439
14440 surface_8_0_recognize_abcd:
14441 ▷ $(ORBITER) -v 3 \
14442 ▷ ▷ -define Do -draw_options \
14443 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14444 ▷ ▷ -end \
14445 ▷ ▷ -define F -finite_field -q 8 -end \
14446 ▷ ▷ -define P3 -projective_space -n 3 -field F -end \
14447 ▷ ▷ -define P2 -projective_space -n 2 -field F -v 0 -end \
14448 ▷ ▷ -define Control -poset_classification_control \
14449 ▷ ▷ ▷ -problem_label nc_arcs_q8_d2 \
14450 ▷ ▷ ▷ -W -depth 6 \
14451 ▷ ▷ -end \
14452 ▷ ▷ -define Arc_control -arc_generator_control \
14453 ▷ ▷ ▷ -projective_space P2 \
14454 ▷ ▷ ▷ -control Control \
14455 ▷ ▷ ▷ -target_size 6 \
14456 ▷ ▷ ▷ -d 2 \
14457 ▷ ▷ -end \
14458 ▷ ▷ -define Orb -orbits \
14459 ▷ ▷ ▷ -on_arcs Arc_control \
14460 ▷ ▷ -end \
14461 ▷ ▷ -define S -cubic_surface -space P3 -catalogue 0 -end \
14462 ▷ ▷ -with S -do \
14463 ▷ ▷ -cubic_surface_activity \
14464 ▷ ▷ ▷ -recognize_Fabcd Orb \
14465 ▷ ▷ -end

```

```

14466
14467
14468 #####
14469 # Section 9.4: Classification of Cubic Surfaces with 27 lines
14470
14471
14472 SECTION.CLASSIFICATION.OF.CUBIC.SURFACES.WITH.27.LINES:
14473
14474
14475 test_9.4:
14476 ▷ make surface_classify_q4
14477 ▷ make surface_classify_q4_arc.lifting.two.lines
14478 ▷ make surface_classify_q7
14479 ▷ make Family_general.F31_25.5.5.25.recognize
14480 ▷ make surface_F_abcd.identify.q9
14481 ▷ make Family_general.F11_iso_test
14482 ▷ make surface_classify_q7_sweep.Cayley
14483
14484
14485
14486
14487 surface_classify_q4:
14488 ▷ $(ORBITER) -v 5 \
14489 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
14490 ▷ ▷ -define F -finite_field -q 4 -end \
14491 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14492 ▷ ▷ -define Control -poset_classification_control -W \
14493 ▷ ▷ ▷ -problem_label "surf_q4" \
14494 ▷ ▷ -end \
14495 ▷ ▷ -define Classify -classify_cubic_surfaces \
14496 ▷ ▷ ▷ -use_double_sixes \
14497 ▷ ▷ ▷ -projective_space P \
14498 ▷ ▷ ▷ -poset_classification_control Control \
14499 ▷ ▷ -end \
14500 ▷ ▷ -define Orb -orbits \
14501 ▷ ▷ ▷ -on_cubic_surfaces Classify \
14502 ▷ ▷ -end \
14503 ▷ ▷ -define Do -draw_options \
14504 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14505 ▷ ▷ -end \
14506 ▷ ▷ -define ro \
14507 ▷ ▷ ▷ -poset_classification_report_options \
14508 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
14509 ▷ ▷ ▷ -draw_options Do \
14510 ▷ ▷ -end \
14511 ▷ ▷ -with Orb -do -orbits_activity \
14512 ▷ ▷ ▷ -report "5plus1=off,double_sixes=off,flag_orbits=off,show_group=off,report_
options=ro" \
14513 ▷ ▷ -end \
14514 ▷ ▷ -with Orb -do \
14515 ▷ ▷ ▷ -orbits_activity \
14516 ▷ ▷ ▷ -export_source_code \
14517 ▷ ▷ -end \
14518 ▷ ▷ -with Orb -do \
14519 ▷ ▷ ▷ -cubic_surfaces_after_classification_activity \
14520 ▷ ▷ ▷ -identify_general_abcd \
14521 ▷ ▷ -end
14522 ▷ $(PDFLATEX) Surfaces.q4.tex
14523 ▷ $(OPEN) Surfaces.q4.pdf

```

```

14524
14525 #▷ ▷ ▷ -draw_poset \
14526
14527 #surface_recognize_abcd_q4.csv
14528
14529
14530
14531
14532 surface_classify_q4_arc_lifting_two_lines:
14533 ▷ $(ORBITER) -v 10 \
14534 ▷ ▷ -define F -finite_field -q 4 -end \
14535 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14536 ▷ ▷ -define Do -draw_options \
14537 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14538 ▷ ▷ -end \
14539 ▷ ▷ -define Control -poset_classification_control \
14540 ▷ ▷ ▷ -problem_label sixarcs_q4 -W \
14541 ▷ ▷ ▷ -draw_options Do \
14542 ▷ ▷ -end \
14543 ▷ ▷ -with P -do \
14544 ▷ ▷ -projective_space_activity \
14545 ▷ ▷ ▷ -control_six_arcs Control \
14546 ▷ ▷ ▷ -classify_surfaces_through_arcs_and_two_lines \
14547 ▷ ▷ -end
14548 ▷ $(PDFLATEX) surfaces_arc_lifting_4.tex
14549 ▷ $(OPEN) surfaces_arc_lifting_4.pdf
14550
14551
14552
14553
14554 surface_classify_q7:
14555 ▷ $(ORBITER) -v 5 \
14556 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
14557 ▷ ▷ -define Do -draw_options \
14558 ▷ ▷ ▷ -xin 400000 \
14559 ▷ ▷ ▷ -xout 3000000 \
14560 ▷ ▷ ▷ -yout 3000000 \
14561 ▷ ▷ ▷ -y_stretch 0.5 \
14562 ▷ ▷ -end \
14563 ▷ ▷ -define F -finite_field -q 7 -end \
14564 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14565 ▷ ▷ -define Control -poset_classification_control -W \
14566 ▷ ▷ ▷ -problem_label "surf_q7" \
14567 ▷ ▷ ▷ -draw_options Do \
14568 ▷ ▷ -end \
14569 ▷ ▷ -define Do2 -draw_options \
14570 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14571 ▷ ▷ -end \
14572 ▷ ▷ -define ro \
14573 ▷ ▷ ▷ -poset_classification_report_options \
14574 ▷ ▷ ▷ ▷ -draw_poset -type_aux \
14575 ▷ ▷ ▷ -draw_options Do2 \
14576 ▷ ▷ -end \
14577 ▷ ▷ -define Classify -classify_cubic_surfaces \
14578 ▷ ▷ ▷ -use_double_sixes \
14579 ▷ ▷ ▷ -projective_space P \
14580 ▷ ▷ ▷ -poset_classification_control Control \
14581 ▷ ▷ -end \
14582 ▷ ▷ -define Orb -orbits \

```

```

14583 ▷ ▷ ▷ -on_cubic_surfaces Classify \
14584 ▷ ▷ -end \
14585 ▷ ▷ -with Orb -do -orbits_activity \
14586 ▷ ▷ ▷ -report "5plus1=off,double_sixes=off,flag_orbits=off,show_group=off,report_
options=ro,max_nb_elements_printed=10" \
14587 ▷ ▷ -end \
14588 ▷ ▷ -with Orb -do \
14589 ▷ ▷ ▷ -orbits_activity \
14590 ▷ ▷ ▷ -export_source_code \
14591 ▷ ▷ -end \
14592 ▷ ▷ -with Orb -do \
14593 ▷ ▷ ▷ -cubic_surfaces_after_classification_activity \
14594 ▷ ▷ ▷ -identify_general_abcd \
14595 ▷ ▷ -end
14596 ▷ $(PDFLATEX) Surfaces.q7.tex
14597 ▷ $(OPEN) Surfaces.q7.pdf
14598
14599
14600
14601
14602
14603
14604
14605 Family_general_F31_25_5_5_25_recognize:
14606 ▷ $(ORBITER) -v 3 \
14607 ▷ ▷ -define Do -draw_options \
14608 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14609 ▷ ▷ -end \
14610 ▷ ▷ -define F -finite_field -q 31 -end \
14611 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14612 ▷ ▷ -define Surf -cubic_surface \
14613 ▷ ▷ ▷ -space P -family_general_abcd 25 5 5 25 \
14614 ▷ ▷ -end \
14615 ▷ ▷ -with Surf -do \
14616 ▷ ▷ -cubic_surface_activity \
14617 ▷ ▷ ▷ -report Do \
14618 ▷ ▷ -end \
14619 ▷ ▷ -define Control -poset_classification_control \
14620 ▷ ▷ ▷ -problem_label "surf_q31" \
14621 ▷ ▷ -end \
14622 ▷ ▷ -define Classify -classify_cubic_surfaces \
14623 ▷ ▷ ▷ -use_double_sixes \
14624 ▷ ▷ ▷ -projective_space P \
14625 ▷ ▷ ▷ -poset_classification_control Control \
14626 ▷ ▷ -end \
14627 ▷ ▷ -define Orb -orbits \
14628 ▷ ▷ ▷ -on_cubic_surfaces Classify \
14629 ▷ ▷ -end \
14630 ▷ ▷ -define Do2 -draw_options \
14631 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14632 ▷ ▷ -end \
14633 ▷ ▷ -define ro \
14634 ▷ ▷ ▷ -poset_classification_report_options \
14635 ▷ ▷ ▷ ▷ -draw_poset -type_aux \
14636 ▷ ▷ ▷ -draw_options Do2 \
14637 ▷ ▷ -end \
14638 ▷ ▷ -with Orb -do -orbits_activity \
14639 ▷ ▷ ▷ -report "5plus1=off,double_sixes=off,flag_orbits=off,show_group=off,report_
options=ro" \

```



```

14640 ▷ ▷ -end \
14641 ▷ ▷ -with Orb -do \
14642 ▷ ▷ ▷ -orbits_activity \
14643 ▷ ▷ ▷ -export_source_code \
14644 ▷ ▷ -end \
14645 ▷ ▷ -with Orb -do \
14646 ▷ ▷ ▷ -cubic_surfaces_after_classification_activity \
14647 ▷ ▷ ▷ -recognize Surf \
14648 ▷ ▷ -end
14649
14650 # slow !
14651 #User time: 5:17
14652
14653
14654
14655 surface_F_abcd_identify_q9:
14656 ▷ $(ORBITER) -v 3 \
14657 ▷ ▷ -define F -finite_field -q 9 -end \
14658 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14659 ▷ ▷ -define Control -poset_classification_control -W \
14660 ▷ ▷ ▷ -problem_label "surf_q9" \
14661 ▷ ▷ -end \
14662 ▷ ▷ -define Classify -classify_cubic_surfaces \
14663 ▷ ▷ ▷ -use_double_sixes \
14664 ▷ ▷ ▷ -projective_space P \
14665 ▷ ▷ ▷ -poset_classification_control Control \
14666 ▷ ▷ -end \
14667 ▷ ▷ -define Orb -orbits \
14668 ▷ ▷ ▷ -on_cubic_surfaces Classify \
14669 ▷ ▷ -end \
14670 ▷ ▷ -with Orb -do \
14671 ▷ ▷ ▷ -cubic_surfaces_after_classification_activity \
14672 ▷ ▷ ▷ -identify_general_abcd \
14673 ▷ ▷ -end
14674
14675
14676 # slow
14677 #User time: 1:11
14678
14679
14680 Family_general_F11_iso_test:
14681 ▷ $(ORBITER) -v 3 \
14682 ▷ ▷ -define F -finite_field -q 11 -end \
14683 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14684 ▷ ▷ -define Surf1 -cubic_surface \
14685 ▷ ▷ ▷ -space P -family_general_abcd 2 4 4 2 \
14686 ▷ ▷ -end \
14687 ▷ ▷ -define Surf2 -cubic_surface \
14688 ▷ ▷ ▷ -space P -family_general_abcd 5 10 7 5 \
14689 ▷ ▷ -end \
14690 ▷ ▷ -define Control -poset_classification_control -W \
14691 ▷ ▷ ▷ -problem_label "surf_q11" \
14692 ▷ ▷ -end \
14693 ▷ ▷ -define Classify -classify_cubic_surfaces \
14694 ▷ ▷ ▷ -use_double_sixes \
14695 ▷ ▷ ▷ -projective_space P \
14696 ▷ ▷ ▷ -poset_classification_control Control \
14697 ▷ ▷ -end \
14698 ▷ ▷ -define Orb -orbits \

```

```

14699 ▷ ▷ ▷ -on_cubic_surfaces Classify \
14700 ▷ ▷ -end \
14701 ▷ ▷ -with Orb -do \
14702 ▷ ▷ ▷ -cubic_surfaces_after_classification_activity \
14703 ▷ ▷ ▷ -isomorphism_testing Surf1 Surf2 \
14704 ▷ ▷ -end
14705
14706
14707 #The surfaces are isomorphic, an isomorphism from the first surface to the second
      is given by
14708 # 0 1 1 9
14709 # 0 4 1 8
14710 # 9 2 5 7
14711 # 1 10 10 3
14712
14713
14714 surface_classify_q7_sweep_Cayley:
14715 ▷ $(ORBITER) -v 5 \
14716 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
14717 ▷ ▷ -define F -finite_field -q 7 -end \
14718 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14719 ▷ ▷ -define Control -poset_classification_control -W \
14720 ▷ ▷ ▷ -problem_label "surf.q7" \
14721 ▷ ▷ -end \
14722 ▷ ▷ -define Classify -classify_cubic_surfaces \
14723 ▷ ▷ ▷ -use_double_sixes \
14724 ▷ ▷ ▷ -projective_space P \
14725 ▷ ▷ ▷ -poset_classification_control Control \
14726 ▷ ▷ -end \
14727 ▷ ▷ -define Orb -orbits \
14728 ▷ ▷ ▷ -on_cubic_surfaces Classify \
14729 ▷ ▷ -end \
14730 ▷ ▷ -define Do -draw_options \
14731 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14732 ▷ ▷ -end \
14733 ▷ ▷ -define ro \
14734 ▷ ▷ ▷ -poset_classification_report_options \
14735 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
14736 ▷ ▷ ▷ -draw_options Do \
14737 ▷ ▷ -end \
14738 ▷ ▷ -with Orb -do -orbits_activity \
14739 ▷ ▷ ▷ -report "5plus1=off,double_sixes=off,flag_orbits=off,show_group=off,report_
      options=ro" \
14740 ▷ ▷ -end \
14741 ▷ ▷ -with Orb -do \
14742 ▷ ▷ ▷ -orbits_activity \
14743 ▷ ▷ ▷ -export_source_code \
14744 ▷ ▷ -end \
14745 ▷ ▷ -with Orb -do \
14746 ▷ ▷ ▷ -cubic_surfaces_after_classification_activity \
14747 ▷ ▷ ▷ -sweep_Cayley \
14748 ▷ ▷ -end
14749 ▷ $(PDFLATEX) Surfaces.q7.tex
14750 ▷ $(OPEN) Surfaces.q7.pdf
14751
14752
14753
14754 #####
14755 # Section 9.5: Quartic Curves Creation

```

```

14756
14757
14758 SECTION_QUARTIC_CURVES_CREATION:
14759
14760
14761 test.9.5:
14762 ▷ make quartic_curve_9_0
14763 ▷ make quartic_curve_9_0_report
14764 ▷ make quartic_curve_9_0_normal_form_report
14765 ▷ make quartic_curve_edge_q9
14766 ▷ make surface_7_0_make_quartic_curves
14767 ▷ make surface_9_1_make_quartic_curves
14768 ▷ make quartic_curve_q9_1_clean2
14769 ▷ make quartic_curves_q13
14770 ▷ make quartic_curves_q19
14771 ▷ make quartic_curves_q29_po4_export
14772 ▷ make quartic_curves_q29_po4_flag_orbits_report
14773 ▷ make symbolic_equation_of_curve_q29_po4_so7
14774 ▷ make quartic_curve_29_121
14775
14776 #slow
14777
14778
14779
14780 quartic_curve_9_0:
14781 ▷ $(ORBITER) -v 3 \
14782 ▷ ▷ -define F -finite_field -q 9 -end \
14783 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
14784 ▷ ▷ -define C -quartic_curve -space P -catalogue 0 -end
14785
14786
14787
14788 # the coefficient 1 is actually 4 mod 3:
14789
14790
14791
14792 quartic_curve_9_0_report:
14793 ▷ $(ORBITER) -v 3 \
14794 ▷ ▷ -define F -finite_field -q 9 -end \
14795 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
14796 ▷ ▷ -define C -quartic_curve -space P -catalogue 0 -end \
14797 ▷ ▷ -with C -do \
14798 ▷ ▷ ▷ -quartic_curve_activity \
14799 ▷ ▷ ▷ -report \
14800 ▷ ▷ ▷ -end
14801 ▷ $(PDFLATEX) quartic_curve_catalogue_q9_iso0_report.tex
14802 ▷ $(OPEN) quartic_curve_catalogue_q9_iso0_report.pdf
14803
14804
14805
14806 quartic_curve_9_0_normal_form_report:
14807 ▷ $(ORBITER) -v 3 \
14808 ▷ ▷ -define F -finite_field -q 9 -end \
14809 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
14810 ▷ ▷ -define C -quartic_curve -space P -by_normal_form "6,3,4,6,2,5" -end \
14811 ▷ ▷ -with C -do \
14812 ▷ ▷ ▷ -quartic_curve_activity \
14813 ▷ ▷ ▷ -report \
14814 ▷ ▷ ▷ -end \

```

```

14815 ▷ ▷ -print_symbols
14816 ▷ $(PDFLATEX) quartic_curve_by_normal_form.q9_report.tex
14817 ▷ $(OPEN) quartic_curve_by_normal_form.q9_report.pdf
14818
14819
14820
14821 quartic_curve_edge.q9:
14822 ▷ $(ORBITER) -v 3 \
14823 ▷ ▷ -define F -finite_field -q 9 -end \
14824 ▷ ▷ -define P2 -projective_space -n 2 -field F -end \
14825 ▷ ▷ -define R -polynomial_ring \
14826 ▷ ▷ ▷ -field F \
14827 ▷ ▷ ▷ -number_of_variables 3 \
14828 ▷ ▷ ▷ -homogeneous_of_degree 4 \
14829 ▷ ▷ ▷ -monomial_ordering_partition \
14830 ▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \
14831 ▷ ▷ -end \
14832 ▷ ▷ -define Edge -symbolic_object \
14833 ▷ ▷ ▷ -field F \
14834 ▷ ▷ ▷ -managed_variables "X,Y,Z" \
14835 ▷ ▷ ▷ -text "X^4-Y^4-Z^4+2*f^2*Y^2*Z^2+1*f*X^2*Y*Z" \
14836 ▷ ▷ -end \
14837 ▷ ▷ -define f -symbolic_object \
14838 ▷ ▷ ▷ -field F \
14839 ▷ ▷ ▷ -text "3" \
14840 ▷ ▷ -end \
14841 ▷ ▷ -define Edge_f3 -symbolic_object \
14842 ▷ ▷ ▷ -field F \
14843 ▷ ▷ ▷ -managed_variables "X,Y,Z" \
14844 ▷ ▷ ▷ -substitute "f" Edge f \
14845 ▷ ▷ -end \
14846 ▷ ▷ -define C -quartic_curve \
14847 ▷ ▷ ▷ -space P2 \
14848 ▷ ▷ ▷ -by_symbolic_object R Edge_f3 -end \
14849 ▷ ▷ -with C -do \
14850 ▷ ▷ ▷ -quartic_curve_activity \
14851 ▷ ▷ ▷ ▷ -report \
14852 ▷ ▷ ▷ -end
14853 ▷ $(PDFLATEX) quartic_curve_equation.Edge_f3.q9_report.tex
14854 ▷ $(OPEN) quartic_curve_equation.Edge_f3.q9_report.pdf
14855
14856
14857
14858 surface_7_0_make_quartic_curves:
14859 ▷ $(ORBITER) -v 3 \
14860 ▷ ▷ -define Do -draw_options \
14861 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14862 ▷ ▷ -end \
14863 ▷ ▷ -define F -finite_field -q 7 -end \
14864 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14865 ▷ ▷ -define P2 -projective_space -n 2 -field F -v 0 -end \
14866 ▷ ▷ -define Control -poset_classification_control \
14867 ▷ ▷ ▷ -problem_label nc_arcs.q7 \
14868 ▷ ▷ ▷ -W -depth 6 \
14869 ▷ ▷ -end \
14870 ▷ ▷ -define Arc_control -arc_generator_control \
14871 ▷ ▷ ▷ -projective_space P2 \
14872 ▷ ▷ ▷ -control Control \
14873 ▷ ▷ ▷ -target_size 6 \

```

```

14874 ▷ ▷ ▷ -d 2 \
14875 ▷ ▷ -end \
14876 ▷ ▷ -define Orb -orbits \
14877 ▷ ▷ ▷ -on_arcs Arc_control \
14878 ▷ ▷ -end \
14879 ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
14880 ▷ ▷ -with S -do \
14881 ▷ ▷ -cubic_surface_activity \
14882 ▷ ▷ ▷ -report Do \
14883 ▷ ▷ ▷ -report_all_flag_orbits Orb \
14884 ▷ ▷ -end
14885 ▷ $(PDFLATEX) surface_catalogue_q7_iso0_report.tex
14886 ▷ $(OPEN) surface_catalogue_q7_iso0_report.pdf
14887
14888
14889
14890 surface_9_1_make_quartic_curves:
14891 ▷ $(ORBITER) -v 3 \
14892 ▷ ▷ -define Do -draw_options \
14893 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14894 ▷ ▷ -end \
14895 ▷ ▷ -define F -finite_field -q 9 -end \
14896 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
14897 ▷ ▷ -define P2 -projective_space -n 2 -field F -v 0 -end \
14898 ▷ ▷ -define Control -poset_classification_control \
14899 ▷ ▷ ▷ -problem_label nc_arcs_q9 \
14900 ▷ ▷ ▷ -W -depth 6 \
14901 ▷ ▷ -end \
14902 ▷ ▷ -define Arc_control -arc_generator_control \
14903 ▷ ▷ ▷ -projective_space P2 \
14904 ▷ ▷ ▷ -control Control \
14905 ▷ ▷ ▷ -target_size 6 \
14906 ▷ ▷ ▷ -d 2 \
14907 ▷ ▷ -end \
14908 ▷ ▷ -define Orb -orbits \
14909 ▷ ▷ ▷ -on_arcs Arc_control \
14910 ▷ ▷ -end \
14911 ▷ ▷ -define S -cubic_surface -space P -catalogue 1 -end \
14912 ▷ ▷ -with S -do \
14913 ▷ ▷ -cubic_surface_activity \
14914 ▷ ▷ ▷ -report Do \
14915 ▷ ▷ ▷ -report_all_flag_orbits Orb \
14916 ▷ ▷ -end
14917 ▷ #$(PDFLATEX) surface_catalogue_q9_iso1_report.tex
14918 ▷ #$(OPEN) surface_catalogue_q9_iso1_report.pdf
14919
14920
14921
14922 quartic_curve_q9_1_clean2:
14923 ▷ $(ORBITER) -v 3 \
14924 ▷ ▷ -define Do -draw_options \
14925 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
14926 ▷ ▷ -end \
14927 ▷ ▷ -define F -finite_field -q 9 -print_as_exponentials -end \
14928 ▷ ▷ -define P2 -projective_space -n 2 -field F -end \
14929 ▷ ▷ -define P3 -projective_space -n 3 -field F -end \
14930 ▷ ▷ -define S9_1 -cubic_surface -space P3 -catalogue 1 \
14931 ▷ ▷ ▷ -select_double_six "c12,c23,a4,c13,a5,a6,b3,b1,c56,b2,c46,c45" \
14932 ▷ ▷ ▷ -transform_inverse "6,4,6,1, 0,1,0,0, 0,0,1,0, 0,0,0,1, 0" \

```

```

14933 ▷ ▷ ▷ -transform_inverse "1,0,0,0, 0,1,0,5, 0,0,1,1, 0,0,0,1, 0" \
14934 ▷ ▷ ▷ -transform "1,0,0,0, 0,0,1,0, 1,1,5,0, 0,0,0,1, 0" \
14935 ▷ ▷ ▷ -transform "1,0,0,0, 0,1,0,0, 0,0,1,0, 0,0,8,1, 0" \
14936 ▷ ▷ ▷ -end \
14937 ▷ ▷ -with S9_1 -do \
14938 ▷ ▷ -cubic_surface_activity \
14939 ▷ ▷ ▷ -report Do \
14940 ▷ ▷ -end \
14941 ▷ ▷ -define C -quartic_curve -space P2 -from_cubic_surface S9_1 0 -end \
14942 ▷ ▷ -with C -do \
14943 ▷ ▷ ▷ -quartic_curve_activity \
14944 ▷ ▷ ▷ ▷ -report \
14945 ▷ ▷ ▷ -end
14946 ▷ $(PDFLATEX) surface_catalogue_q9_iso1_report.tex
14947 ▷ $(PDFLATEX) quartic_curve_surface_catalogue_q9_iso1_pt_orb_0_report.tex
14948 ▷ $(OPEN) quartic_curve_surface_catalogue_q9_iso1_pt_orb_0_report.pdf
14949
14950
14951
14952 quartic_curves.q13:
14953 ▷ $(ORBITER) -v 3 \
14954 ▷ ▷ -define F -finite_field -q 13 -end \
14955 ▷ ▷ -define P -projective_space -n 3 -field F -end \
14956 ▷ ▷ -define P2 -projective_space -n 2 -field F -v 0 -end \
14957 ▷ ▷ -define Control -poset_classification_control \
14958 ▷ ▷ ▷ -problem_label nc_arcs_q13 \
14959 ▷ ▷ ▷ -W -depth 6 \
14960 ▷ ▷ -end \
14961 ▷ ▷ -define Arc_control -arc_generator_control \
14962 ▷ ▷ ▷ -projective_space P2 \
14963 ▷ ▷ ▷ -control Control \
14964 ▷ ▷ ▷ -target_size 6 \
14965 ▷ ▷ ▷ -d 2 \
14966 ▷ ▷ -end \
14967 ▷ ▷ -define Orb -orbits \
14968 ▷ ▷ ▷ -on_arcs Arc_control \
14969 ▷ ▷ -end \
14970 ▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q13) 1 \
14971 ▷ ▷ ▷ -define S_%L -cubic_surface -space P -catalogue %L -end \
14972 ▷ ▷ -end_loop L \
14973 ▷ ▷ -print_symbols \
14974 ▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q13) 1 \
14975 ▷ ▷ ▷ -with S_%L -do \
14976 ▷ ▷ ▷ -cubic_surface_activity \
14977 ▷ ▷ ▷ -export_all_quartic_curves Orb \
14978 ▷ ▷ ▷ -end \
14979 ▷ ▷ -end_loop L \
14980 ▷ ▷ -print_symbols
14981
14982
14983
14984
14985
14986
14987 quartic_curves.q19:
14988 ▷ $(ORBITER) -v 3 \
14989 ▷ ▷ -define F -finite_field -q 19 -end \
14990 ▷ ▷ -define P -projective_space -n 3 -field F -end \
14991 ▷ ▷ -define P2 -projective_space -n 2 -field F -v 0 -end \

```

```

14992 ▷ ▷ -define Control -poset_classification_control \
14993 ▷ ▷ ▷ -problem_label nc_arcs_q19 \
14994 ▷ ▷ ▷ -W -depth 6 \
14995 ▷ ▷ -end \
14996 ▷ ▷ -define Arc_control -arc_generator_control \
14997 ▷ ▷ ▷ -projective_space P2 \
14998 ▷ ▷ ▷ -control Control \
14999 ▷ ▷ ▷ -target_size 6 \
15000 ▷ ▷ ▷ -d 2 \
15001 ▷ ▷ -end \
15002 ▷ ▷ -define Orb -orbits \
15003 ▷ ▷ ▷ -on_arcs Arc_control \
15004 ▷ ▷ -end \
15005 ▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q19) 1 \
15006 ▷ ▷ ▷ -define S_%L -cubic_surface -space P -catalogue %L -end \
15007 ▷ ▷ -end_loop L \
15008 ▷ ▷ -print_symbols \
15009 ▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q19) 1 \
15010 ▷ ▷ ▷ -with S_%L -do \
15011 ▷ ▷ ▷ -cubic_surface_activity \
15012 ▷ ▷ ▷ ▷ -export_all_quartic_curves Orb \
15013 ▷ ▷ ▷ -end \
15014 ▷ ▷ -end_loop L \
15015 ▷ ▷ -print_symbols
15016
15017
15018
15019 quartic_curves.q29_po4_export:
15020 ▷ $(ORBITER) -v 10 \
15021 ▷ ▷ -define F -finite_field -q 29 -end \
15022 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
15023 ▷ ▷ -define P2 -projective_space -n 2 -field F -v 0 -end \
15024 ▷ ▷ -define Control -poset_classification_control \
15025 ▷ ▷ ▷ -problem_label nc_arcs_q29 \
15026 ▷ ▷ ▷ -W -depth 6 \
15027 ▷ ▷ -end \
15028 ▷ ▷ -define Arc_control -arc_generator_control \
15029 ▷ ▷ ▷ -projective_space P2 \
15030 ▷ ▷ ▷ -control Control \
15031 ▷ ▷ ▷ -target_size 6 \
15032 ▷ ▷ ▷ -d 2 \
15033 ▷ ▷ -end \
15034 ▷ ▷ -define Orb -orbits \
15035 ▷ ▷ ▷ -on_arcs Arc_control \
15036 ▷ ▷ -end \
15037 ▷ ▷ -define S_4 -cubic_surface -space P -catalogue 4 -end \
15038 ▷ ▷ -print_symbols \
15039 ▷ ▷ -with S_4 -do \
15040 ▷ ▷ -cubic_surface_activity \
15041 ▷ ▷ ▷ -export_all_quartic_curves Orb \
15042 ▷ ▷ -end \
15043 ▷ ▷ -print_symbols
15044
15045 #The equation is of the form  $X_{-0}^2 f_1(X_{-1}, X_{-2}, X_{-3}) + X_{-0} f_2(X_{-1}, X_{-2}, X_{-3}) + f_3(X_{-1}, X_{-2}, X_{-3})$ , where
15046 #f1=
15047 #11*x1 + 6*x2 + 7*x3
15048 #f2=
15049 #17*x1^2 + 27*x2^2 + 8*x3^2 + 13*x1*x2 + 25*x1*x3 + 20*x2*x3

```

```

15050 #f3=
15051 #27*x3^3 + 28*x1^2*x3 + 17*x2^2*x3 + x1*x3^2 + 2*x2*x3^2 + 17*x1*x2*x3
15052
15053
15054 quartic_curves.q29_po4_flag_orbits.report:
15055 ▷ $(ORBITER) -v 10 \
15056 ▷ ▷ -define F -finite_field -q 29 -end \
15057 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
15058 ▷ ▷ -define P2 -projective_space -n 2 -field F -v 0 -end \
15059 ▷ ▷ -define Control -poset_classification_control \
15060 ▷ ▷ -problem_label nc_arcs.q29 \
15061 ▷ ▷ ▷ -W -depth 6 \
15062 ▷ ▷ -end \
15063 ▷ ▷ -define Arc_control -arc_generator_control \
15064 ▷ ▷ ▷ -projective_space P2 \
15065 ▷ ▷ ▷ -control Control \
15066 ▷ ▷ ▷ -target_size 6 \
15067 ▷ ▷ ▷ -d 2 \
15068 ▷ ▷ -end \
15069 ▷ ▷ -define Orb -orbits \
15070 ▷ ▷ ▷ -on_arcs Arc_control \
15071 ▷ ▷ -end \
15072 ▷ ▷ -define S_4 -cubic_surface -space P -catalogue 4 -end \
15073 ▷ ▷ -print_symbols \
15074 ▷ ▷ -with S_4 -do \
15075 ▷ ▷ -cubic_surface_activity \
15076 ▷ ▷ ▷ -report_all_flag_orbits Orb \
15077 ▷ ▷ -end \
15078 ▷ ▷ -print_symbols
15079 ▷ $(PDFLATEX) surface_catalogue.q29_iso4.quartics.tex
15080 ▷ $(OPEN) surface_catalogue.q29_iso4.quartics.pdf
15081
15082
15083 #User time: 1:52
15084
15085 symbolic_equation_of_curve.q29_po4_so7:
15086 ▷ $(ORBITER) -v 3 \
15087 ▷ ▷ -define F -finite_field -q 29 -end \
15088 ▷ ▷ -define f1 -symbolic_object \
15089 ▷ ▷ ▷ -field F \
15090 ▷ ▷ ▷ -text $(EQUATION_SURFACE_Q29_ISO4_ORB7_F1) \
15091 ▷ ▷ -end \
15092 ▷ ▷ -define f2 -symbolic_object \
15093 ▷ ▷ ▷ -field F \
15094 ▷ ▷ ▷ -text $(EQUATION_SURFACE_Q29_ISO4_ORB7_F2) \
15095 ▷ ▷ -end \
15096 ▷ ▷ -define f3 -symbolic_object \
15097 ▷ ▷ ▷ -field F \
15098 ▷ ▷ ▷ -text $(EQUATION_SURFACE_Q29_ISO4_ORB7_F3) \
15099 ▷ ▷ -end \
15100 ▷ ▷ -define curve -symbolic_object \
15101 ▷ ▷ ▷ -field F \
15102 ▷ ▷ ▷ -text "f2^2-4*f1*f3" \
15103 ▷ ▷ -end \
15104 ▷ ▷ -define curve_x -symbolic_object \
15105 ▷ ▷ ▷ -field F \
15106 ▷ ▷ ▷ -expand curve \
15107 ▷ ▷ -end
15108

```



```

15109 # 0 : (28 * x1 ^ 4) + (7 * x1 ^ 3 * x2) + (24 * x1 ^ 3 * x3) + (14 * x1 ^ 2 * x2
      ^ 2) + (26 * x1 ^ 2 * x2 * x3) + (11 * x1 ^ 2 * x3 ^ 2) + (6 * x1 * x2 ^ 3) + (1
      8 * x1 * x2 ^ 2 * x3) + (11 * x1 * x2 * x3 ^ 2) + (25 * x1 * x3 ^ 3) + (4 * x2 ^
      4) + (5 * x2 ^ 3 * x3) + (18 * x2 ^ 2 * x3 ^ 2) + (22 * x2 * x3 ^ 3) + (4 * x3 ^
      4)
15110
15111
15112
15113 quartic_curve_29_121:
15114 ▷ $(ORBITER) -v 3 \
15115 ▷ ▷ -define F -finite_field -q 29 -end \
15116 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
15117 ▷ ▷ -define C -quartic_curve -space P -catalogue 121 -end \
15118 ▷ ▷ -with C -do \
15119 ▷ ▷ ▷ -quartic_curve_activity \
15120 ▷ ▷ ▷ ▷ -report \
15121 ▷ ▷ ▷ -end
15122 ▷ $(PDFLATEX) quartic_curve_catalogue_q29_iso121_report.tex
15123 ▷ $(OPEN) quartic_curve_catalogue_q29_iso121_report.pdf
15124
15125
15126
15127
15128 #####
15129 # Section 9.6: Classification of Quartic Curve
15130
15131
15132 SECTION.CLASSIFICATION.OF_QUARTIC_CURVE:
15133
15134
15135 test_9.6:
15136 ▷ make quartic_curves.q7
15137 ▷ make quartic_curve_13_0_report
15138 ▷ make PG_2_13_rank_lines
15139 ▷ make PG_2_13_orbits_on_lines
15140 ▷ make quartic_curve_13_1_report
15141 ▷ make quartic_curves_19_report
15142 ▷ make quartic_curve_q9_1
15143 ▷ make quartic_curve_q13_f1_from_variety
15144
15145
15146
15147
15148
15149 quartic_curves.q7:
15150 ▷ $(ORBITER) -v 3 \
15151 ▷ ▷ -define F -finite_field -q 7 -end \
15152 ▷ ▷ -define P -projective_space -n 3 -field F -end \
15153 ▷ ▷ -define P2 -projective_space -n 2 -field F -v 0 -end \
15154 ▷ ▷ -define Control -poset_classification_control \
15155 ▷ ▷ ▷ -problem_label nc_arcs_q7 \
15156 ▷ ▷ ▷ -W -depth 6 \
15157 ▷ ▷ -end \
15158 ▷ ▷ -define Arc_control -arc_generator_control \
15159 ▷ ▷ ▷ -projective_space P2 \
15160 ▷ ▷ ▷ -control Control \
15161 ▷ ▷ ▷ -target_size 6 \
15162 ▷ ▷ ▷ -d 2 \
15163 ▷ ▷ -end \

```

```

15164 ▷ ▷ -define Orb -orbits \
15165 ▷ ▷ ▷ -on_arcs Arc_control \
15166 ▷ ▷ -end \
15167 ▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q7) 1 \
15168 ▷ ▷ ▷ -define S_%L -cubic_surface -space P -catalogue %L -end \
15169 ▷ ▷ -end_loop L \
15170 ▷ ▷ -print_symbols \
15171 ▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q7) 1 \
15172 ▷ ▷ ▷ -with S_%L -do \
15173 ▷ ▷ ▷ -cubic_surface_activity \
15174 ▷ ▷ ▷ ▷ -export_all_quartic_curves Orb \
15175 ▷ ▷ ▷ -end \
15176 ▷ ▷ -end_loop L \
15177 ▷ ▷ -print_symbols
15178
15179
15180
15181
15182
15183
15184 quartic_curve_13_0_report:
15185 ▷ $(ORBITER) -v 3 \
15186 ▷ ▷ -define F -finite_field -q 13 -end \
15187 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
15188 ▷ ▷ -define C -quartic_curve -space P -catalogue 0 \
15189 ▷ ▷ ▷ -transform "10,4,1,11,5,11,4,1,1" \
15190 ▷ ▷ ▷ -transform_inverse "9,1,0,12,9,0,2,10,11" \
15191 ▷ ▷ -end \
15192 ▷ ▷ -with C -do \
15193 ▷ ▷ ▷ -quartic_curve_activity \
15194 ▷ ▷ ▷ ▷ -report \
15195 ▷ ▷ ▷ -end \
15196 ▷ ▷ -with C -do \
15197 ▷ ▷ ▷ -quartic_curve_activity \
15198 ▷ ▷ ▷ ▷ -extract_orbit_on_bitangents_by_length 4 \
15199 ▷ ▷ ▷ -end
15200 ▷ #$(PDFLATEX) quartic_curve_catalogue_q13_iso0_report.tex
15201 ▷ #$(OPEN) quartic_curve_catalogue_q13_iso0_report.pdf
15202
15203 # 170, 111, 140, 2
15204
15205
15206 PG_2_13_rank_lines:
15207 ▷ $(ORBITER) -v 2 \
15208 ▷ ▷ -define v -vector -format 4 \
15209 ▷ ▷ -dense "1,0,0, 0,1,0, 1,0,0, 0,0,1, 1,1,1, 0,1,0, 1,1,1, 0,0,1" \
15210 ▷ ▷ -end \
15211 ▷ ▷ -define F -finite_field -q 23 -end \
15212 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
15213 ▷ ▷ -with P -do \
15214 ▷ ▷ -projective_space_activity \
15215 ▷ ▷ ▷ -rank_lines_in_PG v \
15216 ▷ ▷ -end \
15217
15218
15219 PG_2_13_orbits_on_lines:
15220 ▷ $(ORBITER) -v 5 \
15221 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
15222 ▷ ▷ -define Control -poset_classification_control \

```

```

15223 ▷ ▷ ▷ -problem_label PGL_3.13 \
15224 ▷ ▷ ▷ -depth 4 \
15225 ▷ ▷ -end \
15226 ▷ ▷ -define G -linear_group -PGL 3 13 -end \
15227 ▷ ▷ -define G.on.lines -modified_group -from G \
15228 ▷ ▷ ▷ -on_k.subspaces 2 \
15229 ▷ ▷ -end \
15230 ▷ ▷ -define Orb -orbits \
15231 ▷ ▷ ▷ -group G \
15232 ▷ ▷ ▷ -group_action G.on.lines \
15233 ▷ ▷ ▷ -generators G \
15234 ▷ ▷ ▷ -on_subsets 4 Control \
15235 ▷ ▷ -end \
15236 ▷ ▷ -with Orb -do -orbits.activity \
15237 ▷ ▷ ▷ -recognize "170, 111, 140, 2" \
15238 ▷ ▷ ▷ -recognize "0,23,24,47" \
15239 ▷ ▷ -end
15240 ▷ #$(PDFLATEX) PGL_3.13_poset.tex
15241 ▷ #$(OPEN) PGL_3.13_poset.pdf
15242
15243 # stabilizer of {0,23,24,47}
15244 #1,0,0,7,9,0,9,5,3,
15245 #1,3,0,1,12,0,10,9,2,
15246 #1,1,11,7,9,6,10,5,2,
15247 #1,4,11,12,1,8,10,4,2,
15248
15249 quartic_curve.13.1.report:
15250 ▷ $(ORBITER) -v 3 \
15251 ▷ ▷ -define F -finite_field -q 13 -end \
15252 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
15253 ▷ ▷ -define C -quartic_curve -space P -catalogue 1 -end \
15254 ▷ ▷ -with C -do \
15255 ▷ ▷ ▷ -quartic_curve_activity \
15256 ▷ ▷ ▷ -report \
15257 ▷ ▷ ▷ -end
15258 ▷ $(PDFLATEX) quartic_curve_catalogue_q13_iso1_report.tex
15259 ▷ $(OPEN) quartic_curve_catalogue_q13_iso1_report.pdf
15260
15261
15262
15263 quartic_curves.19.report:
15264 ▷ $(ORBITER) -v 3 \
15265 ▷ ▷ -define F -finite_field -q 19 -end \
15266 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
15267 ▷ ▷ -loop L 0 $(NB_QUARTIC_CURVES_Q19) 1 \
15268 ▷ ▷ -define C -quartic_curve -space P -catalogue %L -end \
15269 ▷ ▷ -with C -do \
15270 ▷ ▷ ▷ -quartic_curve_activity \
15271 ▷ ▷ ▷ -report \
15272 ▷ ▷ ▷ -end \
15273 ▷ ▷ -end_loop L
15274 ▷ ▷ $(PDFLATEX) quartic_curve_catalogue_q19_iso0_report.tex
15275 ▷ ▷ $(PDFLATEX) quartic_curve_catalogue_q19_iso1_report.tex
15276 ▷ ▷ $(PDFLATEX) quartic_curve_catalogue_q19_iso2_report.tex
15277 ▷ ▷ $(PDFLATEX) quartic_curve_catalogue_q19_iso3_report.tex
15278 ▷ ▷ $(PDFLATEX) quartic_curve_catalogue_q19_iso4_report.tex
15279 ▷ ▷ $(PDFLATEX) quartic_curve_catalogue_q19_iso5_report.tex
15280 ▷ ▷ $(PDFLATEX) quartic_curve_catalogue_q19_iso6_report.tex
15281 ▷ ▷ $(PDFLATEX) quartic_curve_catalogue_q19_iso7_report.tex

```

```

15282 ▷ ▷ $(PDFLATEX) quartic_curve_catalogue.q19_iso8_report.tex
15283 ▷ ▷ $(PDFLATEX) quartic_curve_catalogue.q19_iso9_report.tex
15284 ▷ ▷ $(PDFLATEX) quartic_curve_catalogue.q19_iso10_report.tex
15285 ▷ ▷ $(PDFLATEX) quartic_curve_catalogue.q19_iso11_report.tex
15286 ▷ ▷ $(PDFLATEX) quartic_curve_catalogue.q19_iso12_report.tex
15287 ▷ ▷ $(PDFLATEX) quartic_curve_catalogue.q19_iso13_report.tex
15288
15289
15290
15291 quartic_curve.q9.1:
15292 ▷ $(ORBITER) -v 3 \
15293 ▷ ▷ -define F -finite_field -q 9 -end \
15294 ▷ ▷ -define P2 -projective_space -n 2 -field F -end \
15295 ▷ ▷ -define P3 -projective_space -n 3 -field F -end \
15296 ▷ ▷ -define S9_1 -cubic_surface -space P3 -catalogue 1 -end \
15297 ▷ ▷ -define C -quartic_curve -space P2 -from_cubic_surface S9_1 0 -end \
15298 ▷ ▷ -with C -do \
15299 ▷ ▷ ▷ -quartic_curve_activity \
15300 ▷ ▷ ▷ -report \
15301 ▷ ▷ ▷ -end
15302 ▷ $(PDFLATEX) quartic_curve_surface_catalogue.q9_iso1_pt_orb_0_report.tex
15303 ▷ $(OPEN) quartic_curve_surface_catalogue.q9_iso1_pt_orb_0_report.pdf
15304
15305
15306 #The points by rank are: ( 5, 18, 25, 36, 39, 40, 42, 43, 47, 50, 51, 54, 55, 58,
    59, 62, 68, 69, 70, 71, 76, 77, 79, 81, 85, 87, 89, 90 )
15307
15308 #eqn15:
15309 #(1,1,1,1,5,1,7,7,5,0,0,0,0,0,0)
15310
15311
15312 # the 27 single points (of the surface) are:
15313 #6, 17, 81, 118, 146, 192, 229, 265, 332, 351, 362, 386, 412, 432, 488, 505, 531,
    544, 576, 666, 715, 717, 723, 739, 760, 762, 805
15314
15315
15316
15317
15318 quartic_curve.q13.f1.from_variety:
15319 ▷ $(ORBITER) -v 10 \
15320 ▷ ▷ -define F -finite_field -q 13 -end \
15321 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
15322 ▷ ▷ -define R -polynomial_ring \
15323 ▷ ▷ ▷ -field F \
15324 ▷ ▷ ▷ -number_of_variables 3 \
15325 ▷ ▷ ▷ -homogeneous_of_degree 4 \
15326 ▷ ▷ ▷ -monomial_ordering_partition \
15327 ▷ ▷ ▷ -variables "x1,x2,x3" "x_1,x_2,x_3" \
15328 ▷ ▷ -end \
15329 ▷ ▷ -define V_q13_f1 -variety \
15330 ▷ ▷ ▷ -label.txt curve.q13.f1_clean2 \
15331 ▷ ▷ ▷ -label.tex curve\q13\f1\_clean2 \
15332 ▷ ▷ ▷ -projective_space P \
15333 ▷ ▷ ▷ -ring R \
15334 ▷ ▷ ▷ -equation_in_algebraic_form $(CURVE_Q13_FLAG_1_EQN) \
15335 ▷ ▷ ▷ -points $(CURVE_Q13_FLAG_1_POINTS) \
15336 ▷ ▷ ▷ -bitangents $(CURVE_Q13_FLAG_1_BITANGENTS) \
15337 ▷ ▷ ▷ -transform $(CURVE_Q13_FLAG_1_TRANS_1) \
15338 ▷ ▷ ▷ -transform $(CURVE_Q13_FLAG_1_TRANS_2) \

```

```

15339 ▷ ▷ -end \
15340 ▷ ▷ -with V_q13_f1 -do -variety_activity \
15341 ▷ ▷ ▷ -compute_group -nauty_control -end \
15342 ▷ ▷ -end \
15343 ▷ ▷ -define C -quartic_curve -space P -from_variety V_q13_f1 -end \
15344 ▷ ▷ -with C -do \
15345 ▷ ▷ ▷ -quartic_curve_activity \
15346 ▷ ▷ ▷ ▷ -report \
15347 ▷ ▷ ▷ -end
15348 ▷ $(PDFLATEX) quartic_curve_V_q13_f1_report.tex
15349 ▷ $(OPEN) quartic_curve_V_q13_f1_report.pdf
15350
15351
15352
15353
15354
15355
15356
15357 #####
15358 # Section 9.7: Interface to GAP
15359
15360 SECTION_CUBIC_SURFACES_INTERFACE_TO_GAP:
15361
15362
15363 test_9_7:
15364 ▷ make surface_4_0_export_gap
15365
15366 surface_4_0_export_gap:
15367 ▷ $(ORBITER) -v 3 \
15368 ▷ ▷ -define F -finite_field -q 4 -end \
15369 ▷ ▷ -define P -projective_space -n 3 -field F -end \
15370 ▷ ▷ -define S -cubic_surface -space P -catalogue 0 -end \
15371 ▷ ▷ -with S -do \
15372 ▷ ▷ -cubic_surface_activity \
15373 ▷ ▷ ▷ -export_gap \
15374 ▷ ▷ -end
15375
15376
15377
15378
15379 #####
15380 # Chapter 10 - Applications
15381 #####
15382
15383
15384 test_10:
15385 ▷ make test_10_1
15386 ▷ make test_10_2
15387 ▷ make test_10_3
15388
15389
15390
15391 #####
15392 # Section 10.1: Number Theory
15393
15394
15395 SECTION_NUMBER_THEORY:
15396
15397

```

```

15398 test_10_1:
15399 ▷ make PR101
15400 ▷ make PR101_smallest
15401 ▷ make PR29
15402 ▷ make PR31
15403 ▷ make PR37
15404 ▷ make PR_2_100
15405 ▷ make PR_2_1000
15406 ▷ make PE_number_2_1000
15407 ▷ make number_of_primitive_roots_10000
15408 ▷ make IM_3_19
15409 ▷ make IM_723
15410 ▷ make IM
15411 ▷ make IM_gcd
15412 ▷ make PM3a
15413 ▷ make PR_915839
15414 ▷ make PR_915839_check
15415 ▷ make DL_915839
15416 ▷ make sqrt_mod
15417 ▷ make sqrt_5_mod_11
15418 ▷ make sqrt_5_mod_19
15419 ▷ make sqrt_mod_20_31
15420 ▷ make order_of_2_mod_n
15421 ▷ make Eulerfunction_150
15422 ▷ make Eulerfunction_900
15423 ▷ make Eulerfunction_10000
15424 ▷ make power_function_2_mod_11
15425 ▷ make draw_mod_13
15426 ▷ make inverse_mod_a
15427 ▷ make smith_normal_form_1
15428 ▷ make smith_normal_form_2
15429 ▷ make jacobi_35_41
15430 ▷ make jacobi_33_41
15431 ▷ make jacobi_a
15432 ▷ make jacobi_5_19
15433 ▷ make sqrt_mod_7817
15434 ▷ make Chinese_remainders_A
15435 ▷ make Chinese_remainders_B
15436 ▷ make Chinese_remainders_C2
15437 ▷ make Chinese_remainders_C3
15438
15439
15440
15441 PR101:
15442 ▷ $(ORBITER) -v 5 -primitive_root 101
15443
15444
15445 PR101_smallest:
15446 ▷ $(ORBITER) -v 5 -smallest_primitive_root 101
15447
15448
15449 PR29:
15450 ▷ $(ORBITER) -v 1 -smallest_primitive_root 29
15451
15452 PR31:
15453 ▷ $(ORBITER) -v 1 -smallest_primitive_root 31
15454
15455 PR37:
15456 ▷ $(ORBITER) -v 1 -smallest_primitive_root 37

```

```

15457
15458 PR_2_100:
15459 ▷ $(ORBITER) -v 1 -smallest_primitive_root_interval 2 100
15460
15461
15462
15463
15464 PR_2_1000:
15465 ▷ $(ORBITER) -v 1 -smallest_primitive_root_interval 2 1000
15466 ▷ $(ORBITER) -v 1 -csv_file_latex 1 \
15467 ▷ primitive_element_table_2_1000.csv
15468 ▷ $(PDFLATEX) primitive_element_table_2_1000.tex
15469 ▷ $(OPEN) primitive_element_table_2_1000.pdf
15470
15471 PE_number_2_1000:
15472 ▷ $(ORBITER) -v 1 -number_of_primitive_roots_interval 2 1000
15473 ▷ $(ORBITER) -v 1 -csv_file_latex 1 table_number_of_pe_2_1000.csv
15474 ▷ $(PDFLATEX) table_number_of_pe_2_1000.tex
15475 ▷ $(OPEN) table_number_of_pe_2_1000.pdf
15476
15477 number_of_primitive_roots_10000:
15478 ▷ $(ORBITER) -v 1 -number_of_primitive_roots_interval 10000 10001
15479
15480
15481
15482 IM_3_19:
15483 ▷ $(ORBITER) -v 5 -inverse_mod 3 19
15484
15485
15486 IM_723:
15487 ▷ $(ORBITER) -v 5 -inverse_mod 723 4060
15488
15489
15490
15491 IM:
15492 ▷ $(ORBITER) -v 5 -inverse_mod 1865025205 2147483647
15493
15494
15495 IM_gcd:
15496 ▷ $(ORBITER) -v 5 -extended_gcd 1865025205 2147483647
15497
15498
15499 PM3a:
15500 ▷ $(ORBITER) -v 5 -power_mod 16807 1073741823 2147483647
15501
15502 # randomized algo:
15503
15504 PR_915839:
15505 ▷ $(ORBITER) -v 5 -primitive_root 915839
15506
15507 #a primitive root modulo 915839 is 43085
15508
15509 PR_915839_check:
15510 ▷ $(ORBITER) -v 5 -power_mod 43085 49842 915839
15511
15512 #the power of 43085 to the 49842 mod 915839 is 487320
15513
15514 DL_915839:
15515 ▷ $(ORBITER) -v 5 -discrete_log 487320 43085 915839

```

```

15516
15517
15518 #The discrete log is 49842 since  $487320 = 43085^{49842} \bmod 915839$ , time: 0:22
15519
15520 sqrt_mod:
15521 ▷ $(ORBITER) -v 2 -square_root_mod 33 41
15522
15523
15524 sqrt_5_mod_11:
15525 ▷ $(ORBITER) -v 2 -square_root_mod 5 11
15526
15527
15528 sqrt_5_mod_19:
15529 ▷ $(ORBITER) -v 2 -square_root_mod 5 19
15530
15531
15532 sqrt_mod_20_31:
15533 ▷ $(ORBITER) -v 2 -square_root_mod 20 31
15534
15535 order_of_2_mod_n:
15536 ▷ $(ORBITER) -v 3 -order_of_q_mod_n 2 3 51
15537 ▷ $(ORBITER) -v 1 -csv_file_latex 1 \
15538 ▷   order_of_q_mod_n.q2_3_51.csv
15539 ▷ $(PDFLATEX) order_of_q_mod_n.q2_3_51.tex
15540 ▷ $(OPEN) order_of_q_mod_n.q2_3_51.pdf
15541
15542
15543
15544
15545
15546
15547 Eulerfunction_150:
15548 ▷ $(ORBITER) -v 1 -eulerfunction_interval 1 150
15549 ▷ $(ORBITER) -v 1 -csv_file_latex 1 \
15550 ▷   table_eulerfunction_1_150.csv
15551 ▷ $(PDFLATEX) table_eulerfunction_1_150.tex
15552 ▷ $(OPEN) table_eulerfunction_1_150.pdf
15553
15554 Eulerfunction_900:
15555 ▷ $(ORBITER) -v 1 -eulerfunction_interval 900 1150
15556 ▷ $(ORBITER) -v 1 -csv_file_latex 1 \
15557 ▷   table_eulerfunction_900_1150.csv
15558 ▷ $(PDFLATEX) table_eulerfunction_900_1150.tex
15559 ▷ $(OPEN) table_eulerfunction_900_1150.pdf
15560
15561 Eulerfunction_10000:
15562 ▷ $(ORBITER) -v 1 -eulerfunction_interval 10000 10150
15563 ▷ $(ORBITER) -v 1 -csv_file_latex 1 \
15564 ▷   table_eulerfunction_10000_10150.csv
15565 ▷ $(PDFLATEX) table_eulerfunction_10000_10150.tex
15566 ▷ $(OPEN) table_eulerfunction_10000_10150.pdf
15567
15568
15569
15570 power_function_2_mod_11:
15571 ▷ $(ORBITER) -v 5 -power_function_mod_n 2 11
15572 ▷ $(ORBITER) -v 1 -csv_file_latex 1 power_function_k2_n11.csv
15573 ▷ $(PDFLATEX) power_function_k2_n11.tex
15574 ▷ $(OPEN) power_function_k2_n11.pdf

```



```

15575
15576
15577 draw_mod_13:
15578 ▷ $(ORBITER) -v 2 \
15579 ▷ ▷ -define Do -draw.options -embedded -end \
15580 ▷ ▷ -draw_mod_n \
15581 ▷ ▷ ▷ "mod_n=on" \
15582 ▷ ▷ ▷ -n 13 \
15583 ▷ ▷ ▷ -file mod_13 \
15584 ▷ ▷ ▷ -power_cycle 2 \
15585 ▷ ▷ ▷ -label_nodes \
15586 ▷ ▷ ▷ -draw.options Do \
15587 ▷ ▷ -end
15588 ▷ $(PDFLATEX) mod_13_draw.tex
15589 ▷ $(OPEN) mod_13_draw.pdf
15590
15591
15592
15593
15594 inverse_mod_26_99:
15595 ▷ $(ORBITER) -v 2 -inverse_mod 26 99
15596
15597
15598 inverse_mod_a:
15599 ▷ $(ORBITER) -v 2 -inverse_mod 18059241 58014043
15600
15601
15602
15603 smith_normal_form_1:
15604 ▷ $(ORBITER) -v 3 \
15605 ▷ ▷ -define M -vector \
15606 ▷ ▷ ▷ -dense "1,2,3,4,5,6,7,8,9" \
15607 ▷ ▷ ▷ -format 3 \
15608 ▷ ▷ -end \
15609 ▷ ▷ -smith_normal_form M
15610
15611
15612 smith_normal_form_2:
15613 ▷ $(ORBITER) -v 3 \
15614 ▷ ▷ -define M -vector \
15615 ▷ ▷ ▷ -dense "9,8,7,6,5,4,3,2,1" \
15616 ▷ ▷ ▷ -format 3 \
15617 ▷ ▷ -end \
15618 ▷ ▷ -smith_normal_form M
15619
15620
15621
15622 jacobi_35_41:
15623 ▷ $(ORBITER) -v 5 -jacobi 35 41
15624 ▷ $(PDFLATEX) jacobi_35_41.tex
15625 ▷ $(OPEN) jacobi_35_41.pdf
15626
15627
15628 jacobi_33_41:
15629 ▷ $(ORBITER) -v 5 -jacobi 33 41
15630 ▷ $(PDFLATEX) jacobi_33_41.tex
15631 ▷ $(OPEN) jacobi_33_41.pdf
15632
15633

```

```

15634 jacobi_a:
15635 ▷ $(ORBITER) -v 5 -jacobi 2221 7817
15636
15637 jacobi_5_19:
15638 ▷ $(ORBITER) -v 5 -jacobi 5 19
15639
15640 sqrt_mod_7817:
15641 ▷ ▷ $(ORBITER) -v 2 -square_root_mod 2221 7817
15642
15643
15644
15645
15646
15647 Chinese_remainders_A:
15648 ▷ $(ORBITER) -v 2 \
15649 ▷ ▷ -define R -vector -dense "2,2,5" -end \
15650 ▷ ▷ -define M -vector -dense "5,6,7" -end \
15651 ▷ ▷ -Chinese_remainders R M
15652
15653 Chinese_remainders_B:
15654 ▷ $(ORBITER) -v 2 \
15655 ▷ ▷ -define R -vector -dense "38,2" -end \
15656 ▷ ▷ -define M -vector -dense "74,27" -end \
15657 ▷ ▷ -Chinese_remainders R M
15658
15659 Chinese_remainders_C2:
15660 ▷ $(ORBITER) -v 2 \
15661 ▷ ▷ -define R -vector -dense "2,3" -end \
15662 ▷ ▷ -define M -vector -dense "2147483647,5915587277" -end \
15663 ▷ ▷ -Chinese_remainders R M
15664
15665
15666 #The solution is 5684294357108828365 modulo 12703626939758759219 (computed in lon
ginteger)
15667
15668 # checking with Maple:
15669 #5684294357108828365 mod 2147483647;
15670 #
2
15671
15672 #5684294357108828365 mod 5915587277;
15673 #
3
15674
15675
15676
15677 Chinese_remainders_C3:
15678 ▷ $(ORBITER) -v 2 \
15679 ▷ ▷ -define R -vector -dense "2,3,4" -end \
15680 ▷ ▷ -define M -vector -dense "2147483647,5915587277,3267000013" -end \
15681 ▷ ▷ -Chinese_remainders R M
15682
15683
15684 #The solution is 31431541759324477327451572539 modulo 415027493773390165853368698
47 (computed in longinteger)
15685
15686 # checking with Maple:
15687 #31431541759324477327451572539 mod 2147483647;
15688 #
2
15689
15690 #31431541759324477327451572539 mod 5915587277;

```

```

15691 #                               3
15692
15693 #31431541759324477327451572539 mod 3267000013;
15694 #                               4
15695
15696
15697
15698
15699
15700
15701
15702
15703 #####
15704 # Section 10.2: Representation Theory
15705
15706 SECTION.REPRESENTATION.THEORY:
15707
15708
15709 test_10.2:
15710 ▷ make representation_on_polynomials_of_degree_3
15711 ▷ make representation_tetrahedral_group_on_polynomials_of_degree_3
15712
15713
15714 representation_on_polynomials_of_degree_3:
15715 ▷ $(ORBITER) -v 4 \
15716 ▷ ▷ -define F -finite_field -q 3 -end \
15717 ▷ ▷ -define G -linear_group -PGL 4 F -end \
15718 ▷ ▷ -define R -polynomial_ring \
15719 ▷ ▷ ▷ -field F \
15720 ▷ ▷ ▷ -number_of_variables 4 \
15721 ▷ ▷ ▷ -homogeneous_of_degree 3 \
15722 ▷ ▷ ▷ -monomial_ordering_partition \
15723 ▷ ▷ ▷ -variables "X,Y,Z,W" "X,Y,Z,W" \
15724 ▷ ▷ -end \
15725 ▷ ▷ -with G -do \
15726 ▷ ▷ -group_theoretic_activity \
15727 ▷ ▷ ▷ -representation_on_polynomials R \
15728 ▷ ▷ -end
15729 ▷ $(ORBITER) -v 2 \
15730 ▷ ▷ -loop L 0 9 1 -draw_matrix \
15731 ▷ ▷ ▷ -input_csv_file PGL_4_3.rep.3.%L.csv \
15732 ▷ ▷ ▷ -box_width 40 -bit_depth 24 -partition 3 20 20 -end \
15733 ▷ ▷ -end_loop L
15734
15735
15736
15737
15738 representation_tetrahedral_group_on_polynomials_of_degree_3:
15739 ▷ $(ORBITER) -v 4 \
15740 ▷ ▷ -define F -finite_field -q 3 -end \
15741 ▷ ▷ -define G -linear_group -GL 3 F \
15742 ▷ ▷ -subgroup_by_generators "tetra" "12" 2 \
15743 ▷ ▷ "0,1,0,0,0,1,1,0,0, 0,0,1,2,0,0,0,2,0" \
15744 ▷ ▷ -end \
15745 ▷ ▷ -define R -polynomial_ring \
15746 ▷ ▷ ▷ -field F \
15747 ▷ ▷ ▷ -number_of_variables 3 \
15748 ▷ ▷ ▷ -homogeneous_of_degree 3 \
15749 ▷ ▷ ▷ -monomial_ordering_partition \

```

```

15750 ▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \
15751 ▷ ▷ -end \
15752 ▷ ▷ -with G -do \
15753 ▷ ▷ -group_theoretic_activity \
15754 ▷ ▷ ▷ -representation_on_polynomials R \
15755 ▷ ▷ -end
15756 ▷ $(ORBITER) -v 2 -loop L 0 2 1 -draw_matrix \
15757 ▷ ▷ -input_csv_file GL_3_3.Subgroup_tetra_12_rep_3_%L.csv \
15758 ▷ ▷ -box_width 40 -bit_depth 24 -partition 3 10 10 -end -end_loop L
15759 ▷ $(OPEN) GL_3_3.Subgroup_tetra_12_rep_3_0.draw.bmp
15760 ▷ $(OPEN) GL_3_3.Subgroup_tetra_12_rep_3_1.draw.bmp
15761
15762 # write GL_3_3.Subgroup_tetra_12_rep_3_0.csv
15763
15764
15765 #####
15766 # Section 10.3: Cryptography
15767
15768
15769 SECTION_CRYPTOGRAPHY:
15770
15771
15772
15773 test_10.3:
15774 ▷ make EC_add
15775 ▷ make EC_cyclic_subgroup
15776 ▷ make EC_points_13
15777 ▷ make EC_points_199
15778 ▷ make EC_Koblitz_encoding
15779 ▷ make EC_bsgs
15780 ▷ make EC_bsgs_decode
15781 ▷ make NTRU_Alice1
15782 ▷ make NTRU_Alice2
15783 ▷ make NTRU_Alice_public_key
15784 ▷ make NTRU_encrypt
15785 ▷ make NTRU_decrypt1
15786 ▷ make NTRU_decrypt2
15787 ▷ make NTRU_decrypt3
15788 ▷ make NTRU_decrypt4
15789 ▷ make NTRU_decrypt5
15790 ▷ make inv_59_mod
15791 ▷ make RSA_e
15792 ▷ make RSA_d
15793 ▷ make im1
15794 ▷ make RSA_e1
15795 ▷ make RSA_d1
15796 ▷ make im1061
15797 ▷ make RSA_e2
15798 ▷ make RSA_d2
15799 ▷ make im3
15800 ▷ make RSA_e3
15801 ▷ make RSA_d3
15802 ▷ make im4
15803 ▷ make RSA_e4
15804 ▷ make RSA_d4
15805 ▷ make im5
15806 ▷ make RSA_e5
15807 ▷ make RSA_d5
15808 ▷ make RSA_d6

```

```

15809 ▷ make smooth
15810 ▷ make im7
15811 ▷ make RSA.e7
15812 ▷ make im8
15813 ▷ make RSA.e8
15814 ▷ make sqrt.big
15815 ▷ make sqrt_mod_33_41
15816 ▷ #make quadratic_sieve
15817 ▷ make pseudoprime3
15818 ▷ make pseudoprime10
15819 ▷ make PR10_test1
15820 ▷ make pseudoprime11
15821 ▷ make pseudoprime20
15822 ▷ make PR10
15823 ▷ make pseudoprime50
15824 ▷ make pseudoprime51
15825 ▷ make pseudoprime30
15826 ▷ make pseudoprime31
15827 ▷ make pseudoprime33
15828 ▷ make pseudoprime34
15829 ▷ make pseudoprime35
15830 ▷ make pseudoprime36
15831 ▷ #make MATH360.hw2
15832 ▷ make F_256_Rijndahl
15833 ▷ make all_square_roots_mod_n_1549411
15834 ▷ make power_mod_211
15835
15836
15837
15838 EC.add:
15839 ▷ $(ORBITER) -v 2 \
15840 ▷ ▷ -define F -finite_field -q 11 -end \
15841 ▷ ▷ -with F -do \
15842 ▷ ▷ -finite_field_activity \
15843 ▷ ▷ -EC_add 1 3 "1,4" "1,4" -end
15844
15845 EC.cyclic_subgroup:
15846 ▷ $(ORBITER) -v 2 \
15847 ▷ ▷ -define F -finite_field -q 11 -end \
15848 ▷ ▷ -with F -do \
15849 ▷ ▷ -finite_field_activity \
15850 ▷ ▷ -EC_cyclic_subgroup 1 3 "1,4" -end
15851
15852
15853 EC.points_13:
15854 ▷ $(ORBITER) -v 2 \
15855 ▷ ▷ -define F -finite_field -q 13 -end \
15856 ▷ ▷ -with F -do \
15857 ▷ ▷ -finite_field_activity \
15858 ▷ ▷ -EC_points "EC_2_5.q13" 2 5 -end
15859 ▷ $(ORBITER) -v 2 -draw_matrix \
15860 ▷ ▷ -input_csv_file EC_2_5.q13_points_xy.csv \
15861 ▷ ▷ -box_width 20 -bit_depth 24 \
15862 ▷ ▷ -partition 2 "1,1,1,1,1,1,1,1,1,1,1,1" "1,1,1,1,1,1,1,1,1,1,1,1" -end
15863
15864
15865
15866
15867 EC.points_199:

```

```

15868 ▷ $(ORBITER) -v 2 \
15869 ▷ ▷ -define F -finite_field -q 199 -end \
15870 ▷ ▷ -with F -do \
15871 ▷ ▷ -finite_field_activity \
15872 ▷ ▷ -EC_points "EC_5_7.q199" 5 7 -end
15873 ▷ $(ORBITER) -v 2 \
15874 ▷ ▷ -draw_matrix -input_csv_file EC_5_7.q199.points.xy.csv \
15875 ▷ ▷ -box_width 10 -bit_depth 24 \
15876 ▷ ▷ -partition 2 199 199 -end
15877
15878 EC.Koblitz_encoding:
15879 ▷ $(ORBITER) -v 6 -seed 17 \
15880 ▷ ▷ -define F -finite_field -q 199 -end \
15881 ▷ ▷ -with F -do \
15882 ▷ ▷ -finite_field_activity \
15883 ▷ ▷ -EC_Koblitz_encoding 5 7 67 "147,164" "DEADBEEF" \
15884 ▷ ▷ -end
15885
15886 EC.bsgs:
15887 ▷ $(ORBITER) -v 2 \
15888 ▷ ▷ -define F -finite_field -q 199 -end \
15889 ▷ ▷ -with F -do \
15890 ▷ ▷ -finite_field_activity \
15891 ▷ ▷ -EC_bsgs 5 7 "147,164" 212 \
15892 ▷ ▷ "172,158,45,195,50,22,10,103,55,33,50,22,\
15893 ▷ ▷ 145,105,31,74,73,155,67,60,25,6" \
15894 ▷ ▷ -end
15895
15896 EC.bsgs.decode:
15897 ▷ $(ORBITER) -v 2 \
15898 ▷ ▷ -define F -finite_field -q 199 -end \
15899 ▷ ▷ -with F -do \
15900 ▷ ▷ -finite_field_activity \
15901 ▷ ▷ -EC_bsgs.decode 5 7 "129,176" 212 \
15902 ▷ ▷ "127,188,51,141,85,29,106,90,41,105,179,71,\
15903 ▷ ▷ 171,2,16,197,183,72,27,129,37,10" \
15904 ▷ ▷ "50,179,169,13,153,169,115,116,188,110,176" \
15905 ▷ ▷ -end
15906
15907
15908
15909 NTRU_Alice1:
15910 ▷ $(ORBITER) -v 2 \
15911 ▷ ▷ -define F -finite_field -q $(NTRU_Q) -end \
15912 ▷ ▷ -with F -do \
15913 ▷ ▷ -finite_field_activity \
15914 ▷ ▷ -extended_gcd_for_polynomials \
15915 ▷ ▷ ▷ $(NTRU_XN1) $(ALICE_PRIVATE_F) \
15916 ▷ ▷ -end
15917
15918
15919
15920
15921 NTRU_Alice2:
15922 ▷ $(ORBITER) -v 2 \
15923 ▷ ▷ -define F -finite_field -q $(NTRU_P) -end \
15924 ▷ ▷ -with F -do \
15925 ▷ ▷ -finite_field_activity \
15926 ▷ ▷ -extended_gcd_for_polynomials \

```

```

15927 ▷ ▷ ▷ $(NTRU_XN1) $(ALICE_PRIVATE_F) \
15928 ▷ ▷ -end
15929
15930
15931 NTRU_Alice_public_key:
15932 ▷ $(ORBITER) -v 2 \
15933 ▷ ▷ -define F -finite_field -q $(NTRU_Q) -end \
15934 ▷ ▷ -with F -do \
15935 ▷ ▷ -finite_field_activity \
15936 ▷ ▷ -polynomial_mult_mod $(ALICE_PRIVATE_F) \
15937 ▷ ▷ ▷ $(ALICE_PRIVATE_G) $(NTRU_XN1) \
15938 ▷ ▷ -end
15939
15940
15941 NTRU_encrypt:
15942 ▷ $(ORBITER) -v 2 \
15943 ▷ ▷ -define F -finite_field -q $(NTRU_Q) -end \
15944 ▷ ▷ -with F -do \
15945 ▷ ▷ -finite_field_activity \
15946 ▷ ▷ -NTRU_encrypt \
15947 ▷ ▷ $(NTRU_N) $(NTRU_P) $(ALICE_PUBLIC_KEY) \
15948 ▷ ▷ $(BOB_ONE_TIME_KEY) $(BOB_MESSAGE) \
15949 ▷ ▷ -end
15950
15951
15952
15953 NTRU_decrypt1:
15954 ▷ $(ORBITER) -v 2 \
15955 ▷ ▷ -define F -finite_field -q $(NTRU_Q) -end \
15956 ▷ ▷ -with F -do \
15957 ▷ ▷ -finite_field_activity \
15958 ▷ ▷ -polynomial_mult_mod $(ALICE_PRIVATE_F) \
15959 ▷ ▷ ▷ $(BOB_ENCRYPT) $(NTRU_XN1) \
15960 ▷ ▷ -end
15961
15962
15963
15964 NTRU_decrypt2:
15965 ▷ $(ORBITER) -v 2 \
15966 ▷ ▷ -define F -finite_field -q $(NTRU_Q) -end \
15967 ▷ ▷ -with F -do \
15968 ▷ ▷ -finite_field_activity \
15969 ▷ ▷ -polynomial_center_lift $(ALICE_C1) -end
15970
15971 NTRU_decrypt3:
15972 ▷ $(ORBITER) -v 2 \
15973 ▷ ▷ -define F -finite_field -q $(NTRU_P) -end \
15974 ▷ ▷ -with F -do \
15975 ▷ ▷ -finite_field_activity \
15976 ▷ ▷ -polynomial_reduce_mod_p $(ALICE_C2) -end
15977
15978 NTRU_decrypt4:
15979 ▷ $(ORBITER) -v 2 \
15980 ▷ ▷ -define F -finite_field -q $(NTRU_Q) -end \
15981 ▷ ▷ -with F -do \
15982 ▷ ▷ -finite_field_activity \
15983 ▷ ▷ -polynomial_mult_mod $(ALICE_PRIVATE_FP) \
15984 ▷ ▷ ▷ $(ALICE_C3) $(NTRU_XN1) \
15985 ▷ ▷ -end

```

```

15986
15987
15988
15989 NTRU_decrypt5:
15990 ▷ $(ORBITER) -v 2 \
15991 ▷ ▷ -define F -finite_field -q $(NTRU_P) -end \
15992 ▷ ▷ -with F -do \
15993 ▷ ▷ -finite_field_activity \
15994 ▷ ▷ -polynomial_center_lift $(ALICE_C4) -end
15995
15996 #A(X)= - X^{5} + X^{3} + X^{2} - X + 1
15997 #plaintext BOB_MESSAGE
15998
15999
16000
16001 #####
16002
16003
16004 inv_59_mod:
16005 ▷ $(ORBITER) -v 2 -inverse_mod 59 10200
16006
16007 # the inverse of 59 mod 10200 is 2939
16008
16009
16010
16011 RSA_e:
16012 ▷ $(ORBITER) -v 2 \
16013 ▷ ▷ -RSA 59 10403 2 "1921,1605,1804,2116,0518"
16014
16015
16016 RSA_d:
16017 ▷ $(ORBITER) -v 2 \
16018 ▷ ▷ -RSA 2939 10403 2 "902,3509,9833,3548,5181"
16019
16020
16021 im1:
16022 ▷ $(ORBITER) -v 2 -inverse_mod 869 1843488
16023
16024 #the inverse of 869 mod 1843488 is 386093
16025
16026
16027 # FUNFACTOR:
16028
16029 RSA_e1:
16030 ▷ $(ORBITER) -v 2 \
16031 ▷ ▷ -RSA 386093 1846303 3 "62114,60103,201518"
16032
16033
16034 RSA_d1:
16035 ▷ $(ORBITER) -v 2 \
16036 ▷ ▷ -RSA 869 1846303 3 "1248407,345776,317846"
16037
16038
16039
16040
16041
16042 #####
16043 # 5503*4603 = 25330309
16044 # 5502*4602 = 25320204

```



```

16045
16046
16047 im1061:
16048 ▷ $(ORBITER) -v 2 \
16049 ▷ ▷ -inverse_mod 1061 25320204
16050 ▷
16051 # the inverse of 1061 mod 25320204 is 2076209
16052
16053
16054
16055 RSA_e2:
16056 ▷ $(ORBITER) -v 2 \
16057 ▷ ▷ -RSA_encrypt_text 2076209 25330309 3 creamcheese
16058
16059 #-RSA_encrypt_text 386093 1846303 creamcheese
16060 #408918,1735142,239809,654636
16061
16062
16063 RSA_d2:
16064 ▷ $(ORBITER) -v 2 \
16065 ▷ ▷ -RSA 1061 25330309 3 "19019931,1619805,740498,2671344"
16066
16067
16068 #####
16069 # 7253*8171 = 59264263
16070 # 7252*8170 = 59248840
16071
16072
16073 im3:
16074 ▷ $(ORBITER) -v 2 \
16075 ▷ ▷ -inverse_mod 2909 59248840
16076 ▷
16077 #the inverse of 2909 mod 59248840 is 4358629
16078
16079 RSA_e3:
16080 ▷ $(ORBITER) -v 2 \
16081 ▷ ▷ -RSA_encrypt_text 2909 59264263 3 encrypted
16082
16083 RSA_d3:
16084 ▷ $(ORBITER) -v 2 \
16085 ▷ ▷ -RSA 4358629 59264263 3 "35270141,9642524,49091707"
16086
16087 #51403,182516,200504 = encrypted
16088
16089
16090 ####
16091 # 7879 * 7901 = 62251979
16092 # 7878 * 7900 = 62236200
16093
16094 # e =
16095
16096 im4:
16097 ▷ $(ORBITER) -v 2 -inverse_mod 583 62236200
16098
16099 # the inverse of 583 mod 62236200 is 32559247
16100
16101 RSA_e4:
16102 ▷ $(ORBITER) -v 2 \
16103 ▷ ▷ -RSA_encrypt_text 583 62251979 3 venividivici

```

```
16104
16105 #-RSA_encrypt_text 583 62251979 venividivici
16106 #40513610,53979973,56449676,35068535
16107
16108
16109
16110 RSA.d4:
16111 ▷ $(ORBITER) -v 2 \
16112 ▷ ▷ -RSA 32559247 62251979 4 "40513610,53979973,56449676,35068535"
16113
16114
16115 #####
16116 # 7369 * 7127 = 52518863
16117 # 7368 * 7126 = 52504368
16118
16119
16120 im5:
16121 ▷ $(ORBITER) -v 2 -inverse_mod 173 52504368
16122
16123 #the inverse of 173 mod 52504368 is 38543669
16124 ▷
16125 RSA.e5:
16126 ▷ $(ORBITER) -v 2 \
16127 ▷ ▷ -RSA_encrypt_text 38543669 52518863 3 fascinating
16128
16129 #-RSA_encrypt_text 38543669 52518863 fascinating
16130 #31526751,8962078,51045732,51894467
16131 ▷ ▷
16132
16133 RSA.d5:
16134 ▷ $(ORBITER) -v 2 \
16135 ▷ ▷ -RSA 173 52518863 5 "31526751,8962078,51045732,51894467"
16136
16137
16138 RSA.d6:
16139 ▷ $(ORBITER) -v 2 \
16140 ▷ ▷ -RSA 47177497 55040413 6 "28702119,48926559"
16141
16142
16143 smooth:
16144 ▷ $(ORBITER) -v 2 \
16145 ▷ ▷ -sift_smooth 100000 100 "2,3,5,7,11,13,17,19"
16146
16147
16148 ▷ ▷
16149 #####
16150 # 1999 * 7907 = 15806093
16151 # 1998 * 7906 = 15796188
16152
16153 im7:
16154 ▷ $(ORBITER) -v 2 -inverse_mod 3221 15796188
16155
16156 #the inverse of 3221 mod 15796188 is 10048553
16157
16158
16159 RSA.e7:
16160 ▷ $(ORBITER) -v 2 \
16161 ▷ ▷ -RSA_encrypt_text 10048553 15806093 3 beachandfun
16162 ▷ ▷
```

```
16163 #
16164 # 7853 * 7673 = 60256069
16165 # 7852 * 7672 = 60240544
16166
16167 im8:
16168 ▷ $(ORBITER) -v 2 -inverse_mod 9017 60240544
16169
16170
16171 #the inverse of 9017 mod 60240544 is 14430473
16172
16173 RSA_e8:
16174 ▷ $(ORBITER) -v 2 \
16175 ▷ ▷ -RSA_encrypt_text 9017 60256069 3 strawberry
16176
16177
16178 sqrt_big:
16179 ▷ $(ORBITER) -v 2 -square_root 1002001
16180 ▷
16181 sqrt_mod_33_41:
16182 ▷ $(ORBITER) -v 2 -square_root_mod 33 41
16183 ▷
16184 quadratic_sieve:
16185 ▷ $(ORBITER) -v 5 -quadratic_sieve 31 500 1
16186
16187
16188
16189 #####
16190
16191 pseudoprime3:
16192 ▷ $(ORBITER) -v 5 \
16193 ▷ ▷ -seed 2531011 -find_pseudoprime 3 5 0 0
16194 ▷ $(PDFLATEX) pseudoprime_3.tex
16195 ▷ $(OPEN) pseudoprime_3.pdf
16196
16197 pseudoprime10:
16198 ▷ $(ORBITER) -v 5 \
16199 ▷ ▷ -seed 2531011 -find_pseudoprime 10 5 5 5
16200 ▷ $(PDFLATEX) pseudoprime_10.tex
16201 ▷ $(OPEN) pseudoprime_10.pdf
16202
16203 # 4460190157
16204
16205
16206 PR10_test1:
16207 ▷ $(ORBITER) -v 5 -power_mod 1293 2230095078 4460190157
16208 ▷ $(ORBITER) -v 5 -power_mod 9865 2230095078 4460190157
16209 ▷ $(ORBITER) -v 5 -power_mod 19645 2230095078 4460190157
16210 ▷ $(ORBITER) -v 5 -power_mod 974586571 2230095078 4460190157
16211 ▷ $(ORBITER) -v 5 -power_mod 974586571 1486730052 4460190157
16212 ▷ $(ORBITER) -v 5 -power_mod 974586571 15222492 4460190157
16213 ▷ $(ORBITER) -v 5 -power_mod 974586571 284796 4460190157
16214
16215
16216
16217 pseudoprime11:
16218 ▷ $(ORBITER) -v 5 \
16219 ▷ ▷ -seed 2531011 -find_pseudoprime 11 5 5 5
16220 ▷ $(PDFLATEX) pseudoprime_11.tex
16221 ▷ $(OPEN) pseudoprime_11.pdf
```

```

16222
16223 # 63814633367
16224
16225 # product is 284625399616057168619
16226
16227
16228 pseudoprime20:
16229 ▷ $(ORBITER) -v 5 \
16230 ▷ ▷ -seed 2531011 -find_pseudoprime 20 5 5 5
16231 ▷ $(PDFLATEX) pseudoprime_20.tex
16232 ▷ $(OPEN) pseudoprime_20.pdf
16233
16234
16235 PR10:
16236 ▷ $(ORBITER) -v 5 -primitive_root 4460190157
16237
16238
16239 # mistake! long integer overflow
16240 # a primitive root modulo 165222861 is 1293
16241
16242
16243
16244
16245 pseudoprime50:
16246 ▷ $(ORBITER) -v 5 \
16247 ▷ ▷ -seed 2531011 -find_pseudoprime 50 5 0 0
16248 ▷ $(PDFLATEX) pseudoprime_50.tex
16249 ▷ $(OPEN) pseudoprime_50.pdf
16250
16251
16252 #91322792878581218181431392170986926262336688354473
16253
16254 pseudoprime51:
16255 ▷ $(ORBITER) -v 5 \
16256 ▷ ▷ -seed 2531011 -find_pseudoprime 51 5 5 5
16257 ▷ $(PDFLATEX) pseudoprime_51.tex
16258 ▷ $(OPEN) pseudoprime_51.pdf
16259
16260 #754600727746834470214089702490004944659715367045417
16261
16262 # product 68912245966050819606199994423264315732335295324400658436661744403244049
    572914094379904326661586100241
16263
16264 pseudoprime30:
16265 ▷ $(ORBITER) -v 5 \
16266 ▷ ▷ -seed 2531011 -find_pseudoprime 30 5 5 5
16267 ▷ $(PDFLATEX) pseudoprime_30.tex
16268 ▷ $(OPEN) pseudoprime_30.pdf
16269
16270 # 286525565474504516914595596387
16271
16272 pseudoprime31:
16273 ▷ $(ORBITER) -v 5 \
16274 ▷ ▷ -seed 2531011 -find_pseudoprime 31 5 5 5
16275 ▷ $(PDFLATEX) pseudoprime_31.tex
16276 ▷ $(OPEN) pseudoprime_31.pdf
16277 #8777266765422645523724129853331
16278
16279 # 2514911323283298698837184692002835573476743643265896783515097

```

```

16280
16281 # maybe 2 seconds
16282
16283
16284 pseudoprime33:
16285 ▷ $(ORBITER) -v 5 \
16286 ▷ ▷ -seed 2531011 -find_pseudoprime 33 5 5 5
16287 ▷ $(PDFLATEX) pseudoprime_33.tex
16288 ▷ $(OPEN) pseudoprime_33.pdf
16289
16290
16291 #371674199498295345543363004459891
16292
16293
16294 pseudoprime34:
16295 ▷ $(ORBITER) -v 5 \
16296 ▷ ▷ -seed 2531011 -find_pseudoprime 34 5 5 5
16297 ▷ $(PDFLATEX) pseudoprime_34.tex
16298 ▷ $(OPEN) pseudoprime_34.pdf
16299
16300 #9309708224110488378214945245346817
16301
16302 # 3460178351758962531912872979731874528849142238619677890786061016947
16303 # 18 sec
16304
16305
16306
16307 pseudoprime35:
16308 ▷ $(ORBITER) -v 5 \
16309 ▷ ▷ -seed 2531011 -find_pseudoprime 35 5 5 5
16310 ▷ $(PDFLATEX) pseudoprime_35.tex
16311 ▷ $(OPEN) pseudoprime_35.pdf
16312
16313 #81329557792505271120435930267680203
16314
16315 pseudoprime36:
16316 ▷ $(ORBITER) -v 5 \
16317 ▷ ▷ -seed 2531011 -find_pseudoprime 36 5 5 5
16318 ▷ $(PDFLATEX) pseudoprime_36.tex
16319 ▷ $(OPEN) pseudoprime_36.pdf
16320
16321 #16262468089199340433363207561599139
16322
16323 # 13226193383093105242537919350220354135219441323641636665484262532145217
16324 # factoring takes 46 seconds
16325
16326
16327 #ToDo:
16328
16329 MATH360_hw2:
16330 ▷ $(ORBITER) -v 3 \
16331 ▷ ▷ -define F -finite_field -q 16 -end \
16332 ▷ ▷ -with F -do -finite_field_activity \
16333 ▷ ▷ -parse_and_evaluate "test" "" "a+b" "a=8,b=14" -end
16334 ▷ $(ORBITER) -v 3 \
16335 ▷ ▷ -define F -finite_field -q 16 -end \
16336 ▷ ▷ -with F -do -finite_field_activity \
16337 ▷ ▷ -parse_and_evaluate "test" "" "a*b" "a=9,b=13" -end
16338 ▷ $(ORBITER) -v 3 \

```

```

16339 ▷ ▷ -define F -finite_field -q 16 -end \
16340 ▷ ▷ -with F -do -finite_field_activity \
16341 ▷ ▷ -parse_and_evaluate "test" "" "a*a*a*a*a" "a=9" -end
16342 ▷ $(ORBITER) -v 3 \
16343 ▷ ▷ -define F -finite_field -q 16 -end \
16344 ▷ ▷ -with F -do -finite_field_activity \
16345 ▷ ▷ -parse_and_evaluate "test" "" "(a+b)*(a+b)" "a=5,b=7" -end
16346 ▷ $(ORBITER) -v 3 \
16347 ▷ ▷ -define F -finite_field -q 16 -end \
16348 ▷ ▷ -with F -do -finite_field_activity \
16349 ▷ ▷ -parse_and_evaluate "test" "" "a*a+b*b" "a=5,b=7" -end
16350
16351
16352 F_256_Rijndahl:
16353 ▷ $(ORBITER) -v 3 \
16354 ▷ ▷ -define F -finite_field -q 256 -override_polynomial 283 -end \
16355 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
16356
16357
16358
16359
16360
16361
16362
16363 all_square_roots_mod_n_1549411:
16364 ▷ $(ORBITER) -v 3 -all_square_roots_mod_n 1075922 1549411
16365 ▷
16366
16367
16368 power_mod_211:
16369 ▷ $(ORBITER) -v 3 -power_function_mod_n 2 211
16370 ▷ $(ORBITER) -v 3 \
16371 ▷ ▷ -plot_function power_function_k2_n211.csv
16372 ▷ $(ORBITER) -v 2 -draw_matrix \
16373 ▷ ▷ -input_csv_file power_function_k2_n211_graph.csv \
16374 ▷ ▷ -box_width 10 -bit_depth 8 \
16375 ▷ ▷ -partition 3 211 211 -end
16376 ▷ $(OPEN) power_function_k2_n211_graph.draw.bmp
16377
16378
16379
16380 #####
16381 # Chapter 11 - Coding Theory
16382 #####
16383
16384
16385 test_11:
16386 ▷ make test_11_1
16387 ▷ make test_11_2
16388 ▷ make test_11_3
16389 ▷ make test_11_4
16390 ▷ make test_11_5
16391 ▷ make test_11_6
16392 ▷ make test_11_7
16393 ▷ make test_11_8
16394 ▷ make test_11_9
16395 ▷ make test_11_10
16396
16397

```

```

16398
16399
16400 #####
16401 # Section 11.1: Coding Theory
16402
16403 SECTION.CODING.THEORY.INTRODUCTION:
16404
16405 test_11_1:
16406 ▷ make Allen_Gates_noise_1_percent
16407 ▷ make nonlinear_code_long_n7_k4
16408 ▷ make nonlinear_code_n7_d4
16409 ▷ make nonlinear_code_n7_d4_classify
16410 ▷ make Hamming_space_4_2_distance_matrix
16411 ▷ make Hamming_space_4_2_distance_matrix_draw
16412 ▷ make Hamming_code_macwilliams
16413 ▷ make code_5_2_3_diagram
16414 ▷ make Hamming_5_2_graph
16415 ▷ make Hamming_5_2_with_5_2_3_code
16416 ▷ make Sylvester_Hadamard_code_8
16417
16418
16419 Allen_Gates_noise_1_percent:
16420 ▷ cp $(ORBITER_PATH)/examples/users_guide/allen.Gates.bmp .
16421 ▷ $(ORBITER) -v 3 \
16422 ▷ ▷ -random_noise_in_bitmap_file \
16423 ▷ ▷ allen.Gates.bmp \
16424 ▷ ▷ allen.Gates.1.bmp \
16425 ▷ ▷ 1 100
16426 ▷ $(OPEN) allen.Gates.1.bmp
16427
16428
16429
16430 nonlinear_code_long_n7_k4:
16431 ▷ $(ORBITER) -v 2 \
16432 ▷ ▷ -define F -finite_field -q 2 -end \
16433 ▷ ▷ -define C -code -field F \
16434 ▷ ▷ ▷ -nonlinear_code_long 7 4 \
16435 ▷ ▷ ▷ "0,1" \
16436 ▷ ▷ ▷ "1,2,3" \
16437 ▷ ▷ ▷ "0,1,3,4,6" \
16438 ▷ ▷ ▷ "2,3,5,6" \
16439 ▷ ▷ -end \
16440 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
16441 ▷ ▷ ▷ -export_codewords codewords.csv \
16442 ▷ ▷ -end \
16443 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
16444 ▷ ▷ ▷ -all_distances \
16445 ▷ ▷ -end \
16446 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
16447 ▷ ▷ ▷ -all_external_distances \
16448 ▷ ▷ -end \
16449 ▷ ▷ -tally_column nonlinear_code_n7_k4_all_external_distances.csv C0
16450
16451
16452 nonlinear_code_n7_d4:
16453 ▷ $(ORBITER) -v 2 \
16454 ▷ ▷ -define F -finite_field -q 2 -end \
16455 ▷ ▷ -define C -code -field F \
16456 ▷ ▷ ▷ -nonlinear_code 7 \

```

```

16457 ▷ ▷ ▷ "0,15,51,60,85,90,102,105" \
16458 ▷ ▷ -end \
16459 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
16460 ▷ ▷ ▷ -all_distances \
16461 ▷ ▷ -end \
16462 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
16463 ▷ ▷ ▷ -all_external_distances \
16464 ▷ ▷ -end \
16465 ▷ ▷ -tally_column nonlinear_code_n7_k8_all_external_distances.csv C0
16466
16467
16468 nonlinear_code_n7_d4_classify:
16469 ▷ $(ORBITER) -v 5 \
16470 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
16471 ▷ ▷ -define Do -draw_options \
16472 ▷ ▷ ▷ -radius 200 -embedded \
16473 ▷ ▷ -end \
16474 ▷ ▷ -define Control -poset_classification_control \
16475 ▷ ▷ ▷ -problem_label codes_n7_d4 \
16476 ▷ ▷ ▷ -depth 10 \
16477 ▷ ▷ ▷ -test_mindist_nonlinear 4 \
16478 ▷ ▷ ▷ -draw_options Do \
16479 ▷ ▷ ▷ -W \
16480 ▷ ▷ -end \
16481 ▷ ▷ -define F -finite_field -q 2 -end \
16482 ▷ ▷ -define G -linear_group -GL 7 F -monomial -end \
16483 ▷ ▷ -define Orb -orbits \
16484 ▷ ▷ ▷ -group G \
16485 ▷ ▷ ▷ -group_action G \
16486 ▷ ▷ ▷ -generators G \
16487 ▷ ▷ ▷ -on_subsets 10 Control \
16488 ▷ ▷ -end \
16489 ▷ ▷ -define ro \
16490 ▷ ▷ ▷ -poset_classification_report_options \
16491 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
16492 ▷ ▷ ▷ ▷ -draw_options Do \
16493 ▷ ▷ -end \
16494 ▷ ▷ -with Orb -do -orbits_activity \
16495 ▷ ▷ ▷ -report "" \
16496 ▷ ▷ ▷ -report_options ro \
16497 ▷ ▷ -end \
16498
16499
16500
16501
16502
16503 Hamming_space_4_2_distance_matrix:
16504 ▷ $(ORBITER) -v 2 \
16505 ▷ ▷ -define F -finite_field -q 2 -end \
16506 ▷ ▷ -with F -do -coding_theoretic_activity \
16507 ▷ ▷ ▷ -Hamming_space_distance_matrix 4 \
16508 ▷ ▷ -end
16509
16510
16511 Hamming_space_4_2_distance_matrix_draw:
16512 ▷ $(ORBITER) -v 2 \
16513 ▷ ▷ -draw_matrix \
16514 ▷ ▷ ▷ -input_csv_file Hamming_n4_q2.csv \
16515 ▷ ▷ ▷ -box_width 20 -bit_depth 24 \

```



```

16516 ▷ ▷ ▷ -partition 4 16 16 \
16517 ▷ ▷ -end
16518 ▷ $(OPEN) Hamming_n4.q2.draw.bmp
16519
16520
16521
16522 Hamming_code_macwilliams:
16523 ▷ $(ORBITER) -v 2 \
16524 ▷ ▷ -make_macwilliams_system 7 4 2
16525 ▷ $(PDFLATEX) MacWilliams.n7.k4.q2.tex
16526 ▷ $(OPEN) MacWilliams.n7.k4.q2.pdf
16527
16528
16529
16530
16531
16532 code_5.2.3_diagram:
16533 ▷ $(ORBITER) -v 2 \
16534 ▷ ▷ -define F -finite_field -q 2 -end \
16535 ▷ ▷ -with F -do -coding_theoretic_activity \
16536 ▷ ▷ ▷ -general_code_binary 5 "code_5.2.3" \
16537 ▷ ▷ ▷ $(CODE_5.2.3.CODEWORDS) \
16538 ▷ ▷ ▷ -metric_balls 1 \
16539 ▷ ▷ -end
16540 ▷ $(ORBITER) -v 2 \
16541 ▷ ▷ -draw_matrix \
16542 ▷ ▷ ▷ -input_csv_file code_5.2.3_char_func_5.4.csv \
16543 ▷ ▷ ▷ -box_width 25 -bit_depth 24 \
16544 ▷ ▷ ▷ -partition 4 8 4 \
16545 ▷ ▷ -end
16546
16547
16548
16549 Hamming_5.2_graph:
16550 ▷ $(ORBITER) -v 2 \
16551 ▷ ▷ -define G -graph -Hamming 5 2 -end \
16552 ▷ ▷ -with G -do \
16553 ▷ ▷ -graph_theoretic_activity -export_csv -end \
16554 ▷ ▷ -with G -do \
16555 ▷ ▷ -graph_theoretic_activity -export_graphviz -end \
16556 ▷ ▷ -with G -do \
16557 ▷ ▷ -graph_theoretic_activity -save -end
16558 ▷ $(ORBITER) -v 2 -draw_matrix \
16559 ▷ ▷ -input_csv_file Hamming_5.2.csv \
16560 ▷ ▷ -box_width 8 -bit_depth 24 -partition 4 32 32 -end
16561 ▷ dot -Tpng Hamming_5.2.gv >Hamming_5.2.png
16562
16563
16564 Hamming_5.2_with_5.2.3_code:
16565 ▷ $(ORBITER) -v 2 \
16566 ▷ ▷ -define G -graph -Hamming 5 2 \
16567 ▷ ▷ ▷ -subset "_code_5.2.3" "\\_code\\5\\_2\\_3" \
16568 ▷ ▷ ▷ $(CODE_5.2.3.CODEWORDS) -end \
16569 ▷ ▷ -with G -do \
16570 ▷ ▷ -graph_theoretic_activity -export_csv -end \
16571 ▷ ▷ -with G -do \
16572 ▷ ▷ -graph_theoretic_activity -export_graphviz -end \
16573 ▷ ▷ -with G -do \
16574 ▷ ▷ -graph_theoretic_activity -save -end \

```

```

16575 ▷ ▷ -with G -do \
16576 ▷ ▷ -graph_theoretic_activity -automorphism_group -end
16577 ▷ $(PDFLATEX) Hamming_5_2_code_5_2_3_report.tex
16578 ▷ $(OPEN) Hamming_5_2_code_5_2_3_report.pdf
16579
16580 # group has order 32
16581
16582
16583 # n must be a power of 2, and at least 4:
16584 # minus one is represented as 2.
16585
16586 Sylvester_Hadamard_code_8:
16587 ▷ $(ORBITER) -v 2 \
16588 ▷ ▷ -define F -finite_field -q 3 -end \
16589 ▷ ▷ -with F -do -coding_theoretic_activity \
16590 ▷ ▷ ▷ -Sylvester_Hadamard_code 8 \
16591 ▷ ▷ -end
16592
16593
16594
16595
16596 #####
16597 # Section 11.2: Linear codes
16598
16599 SECTION_CODING_THEORY_LINEAR_CODES:
16600
16601
16602
16603
16604 test_11_2:
16605 ▷ make RM_3_1
16606 ▷ make code_6
16607 ▷ make simplex_code
16608 ▷ make Hamming_generator
16609 ▷ make Hamming_code
16610 ▷ make RM_3_1_and_codewords
16611 ▷ make RM_3_1_from_generator_matrix
16612 ▷ make RM_4_1_and_codewords
16613 ▷ make RM_5_1_and_codewords
16614 ▷ make Hamming_code_by_rows
16615 ▷ make Hamming_code_long
16616 ▷ make Hamming_code_diagram
16617 ▷ make Hamming_code_words
16618 ▷ make Hamming_weight_enumerator
16619 ▷ make Hamming_minimum_distance
16620 ▷ make Golay23_minimum_distance
16621 ▷ make code_Hamming_systematic
16622 ▷ make Hamming_RREF
16623 ▷ make Hamming_nullspace
16624
16625
16626
16627 RM_3_1:
16628 ▷ $(ORBITER) -v 2 \
16629 ▷ ▷ -define F -finite_field -q 2 -end \
16630 ▷ ▷ -define C -code -field F \
16631 ▷ ▷ ▷ -Reed_Muller 3 \
16632 ▷ ▷ -end \
16633 ▷ ▷ -with C -and F -do -coding_theoretic_activity \

```

```

16634 ▷ ▷ ▷ -export_magma RM_3_1.magma \
16635 ▷ ▷ -end \
16636 ▷ ▷ -with C -do -coding_theoretic_activity \
16637 ▷ ▷ ▷ -report \
16638 ▷ ▷ -end
16639 ▷ $(PDFLATEX) RM_m3_code.n8.k4.q2.tex
16640 ▷ $(OPEN) RM_m3_code.n8.k4.q2.pdf
16641
16642
16643 code_6:
16644 ▷ $(ORBITER) -v 2 \
16645 ▷ ▷ -define F -finite_field -q 2 -end \
16646 ▷ ▷ -define C -code -field F \
16647 ▷ ▷ ▷ -basis 6 "60,50,41" \
16648 ▷ ▷ -end \
16649 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
16650 ▷ ▷ ▷ -make_diagram \
16651 ▷ ▷ -end \
16652 ▷ ▷ -with C -do -coding_theoretic_activity \
16653 ▷ ▷ ▷ -report \
16654 ▷ ▷ -end
16655 ▷ $(ORBITER) -v 2 -draw_matrix \
16656 ▷ ▷ -input_csv_file by_basis.n6.k3_char_func.6.8.csv \
16657 ▷ ▷ -box_width 20 -bit_depth 24 \
16658 ▷ ▷ -partition 2 "1,1,1,1,1,1,1,1" "1,1,1,1,1,1,1,1" \
16659 ▷ ▷ -end
16660 ▷ $(PDFLATEX) by_basis.n6.k3_code.n6.k3.q2.tex
16661 ▷ $(OPEN) by_basis.n6.k3_code.n6.k3.q2.pdf
16662 ▷ $(OPEN) by_basis.n6.k3_char_func.6.8.draw.bmp
16663
16664 # linear code with generator matrix
16665 # 111100 = 60
16666 # 110010 = 50
16667 # 101001 = 41
16668
16669
16670
16671
16672 simplex_code:
16673 ▷ $(ORBITER) -v 2 \
16674 ▷ ▷ -define F -finite_field -q 2 -end \
16675 ▷ ▷ -define v -vector -field F -format 3 \
16676 ▷ ▷ ▷ -dense $(SIMPLEX_CODE_GENERATOR) \
16677 ▷ ▷ -end \
16678 ▷ ▷ -define C -code -field F \
16679 ▷ ▷ ▷ -generator_matrix v \
16680 ▷ ▷ -end \
16681 ▷ ▷ -with C -do -coding_theoretic_activity \
16682 ▷ ▷ ▷ -report \
16683 ▷ ▷ -end
16684 ▷ $(PDFLATEX) by_genma.n7.k3_code.n7.k3.q2.tex
16685 ▷ $(OPEN) by_genma.n7.k3_code.n7.k3.q2.pdf
16686
16687
16688
16689 Hamming_generator:
16690 ▷ $(ORBITER) -v 2 \
16691 ▷ ▷ -define F -finite_field -q 2 -end \
16692 ▷ ▷ -define v -vector -field F -format 3 \

```

```

16693 ▷ ▷ ▷ -dense $(SIMPLEX.CODE.GENERATOR) \
16694 ▷ ▷ -end \
16695 ▷ ▷ -with F -do \
16696 ▷ ▷ -finite_field_activity \
16697 ▷ ▷ ▷ -nullspace v \
16698 ▷ ▷ -end
16699 ▷ $(PDFLATEX) v_nullspace.tex
16700 ▷ $(OPEN) v_nullspace.pdf
16701
16702 # basis in binary:
16703 # 67,37,22,15
16704
16705
16706
16707
16708 Hamming_code:
16709 ▷ $(ORBITER) -v 2 \
16710 ▷ ▷ -define F -finite_field -q 2 -end \
16711 ▷ ▷ -define v -vector -field F -format 3 \
16712 ▷ ▷ ▷ -dense $(SIMPLEX.CODE.GENERATOR) \
16713 ▷ ▷ -end \
16714 ▷ ▷ -define C -code -field F \
16715 ▷ ▷ ▷ -generator_matrix v \
16716 ▷ ▷ ▷ -dual -end \
16717 ▷ ▷ -end \
16718 ▷ ▷ -with C -do -coding_theoretic_activity \
16719 ▷ ▷ ▷ -export_magma Hamming.magma \
16720 ▷ ▷ -end
16721
16722 # writes Hamming.magma
16723
16724
16725
16726 RM_3_1_and_codewords:
16727 ▷ $(ORBITER) -v 2 \
16728 ▷ ▷ -define F -finite_field -q 2 -end \
16729 ▷ ▷ -define C -code -field F \
16730 ▷ ▷ ▷ -Reed_Muller 3 \
16731 ▷ ▷ -end \
16732 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
16733 ▷ ▷ ▷ -export_magma RM_3_1.magma \
16734 ▷ ▷ -end \
16735 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
16736 ▷ ▷ ▷ -export_codewords RM_3_1_codewords.csv \
16737 ▷ ▷ -end \
16738 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
16739 ▷ ▷ ▷ -export_genma RM_3_1_genma.csv \
16740 ▷ ▷ -end
16741
16742
16743 RM_3_1_from_generator_matrix:
16744 ▷ $(ORBITER) -v 2 \
16745 ▷ ▷ -define F -finite_field -q 2 -end \
16746 ▷ ▷ -define genma -vector -format 8 -field F \
16747 ▷ ▷ ▷ -compact $(CODE_RM_3_1_GENMA) \
16748 ▷ ▷ -end \
16749 ▷ ▷ -define C -code -field F \
16750 ▷ ▷ ▷ -generator_matrix genma \
16751 ▷ ▷ -end

```

```

16752 ▷ #$(PDFLATEX) code_n8_k4.q2.tex
16753 ▷ #$(OPEN) code_n8_k4.q2.pdf
16754
16755 #Codewords: (0,255,170,85,204,51,102,153,240,15,90,165,60,195,150,105)
16756
16757
16758 RM_4_1_and_codewords:
16759 ▷ $(ORBITER) -v 2 \
16760 ▷ ▷ -define F -finite_field -q 2 -end \
16761 ▷ ▷ -define C -code -field F -ReedMuller 4 -end \
16762 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
16763 ▷ ▷ ▷ -export_magma RM_4_1.magma \
16764 ▷ ▷ -end \
16765 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
16766 ▷ ▷ ▷ -export_codewords RM_4_1.codewords.csv \
16767 ▷ ▷ -end \
16768 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
16769 ▷ ▷ ▷ -export_genma RM_4_1.genma.csv \
16770 ▷ ▷ -end
16771
16772 RM_5_1_and_codewords:
16773 ▷ $(ORBITER) -v 2 \
16774 ▷ ▷ -define F -finite_field -q 2 -end \
16775 ▷ ▷ -define C -code -field F -ReedMuller 5 -end \
16776 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
16777 ▷ ▷ ▷ -export_magma RM_5_1.magma \
16778 ▷ ▷ -end \
16779 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
16780 ▷ ▷ ▷ -export_codewords RM_5_1.codewords.csv \
16781 ▷ ▷ -end \
16782 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
16783 ▷ ▷ ▷ -export_genma RM_5_1.genma.csv \
16784 ▷ ▷ -end
16785
16786
16787
16788
16789 Hamming_code_by_rows:
16790 ▷ $(ORBITER) -v 2 \
16791 ▷ ▷ -define F -finite_field -q 2 -end \
16792 ▷ ▷ -define v -vector -dense $(HAMMING_CODE_ROWS_IN_BINARY_RANKS) -end \
16793 ▷ ▷ -define C -code -field F \
16794 ▷ ▷ ▷ -basis 7 v \
16795 ▷ ▷ -end
16796 ▷ #$(PDFLATEX) code_n7_k4.q2.tex
16797 ▷ #$(OPEN) code_n7_k4.q2.pdf
16798
16799
16800
16801 Hamming_code_long:
16802 ▷ $(ORBITER) -v 2 \
16803 ▷ ▷ -define F -finite_field -q 2 -end \
16804 ▷ ▷ -define C -code -field F \
16805 ▷ ▷ ▷ -long_code 7 4 \
16806 ▷ ▷ ▷ "0,5,6" \
16807 ▷ ▷ ▷ "1,4,6" \
16808 ▷ ▷ ▷ "2,4,5" \
16809 ▷ ▷ ▷ "3,4,5,6" \
16810 ▷ ▷ -end

```

```

16811
16812
16813 Hamming_code_diagram:
16814 ▷ $(ORBITER) -v 2 \
16815 ▷ ▷ -define F -finite_field -q 2 -end \
16816 ▷ ▷ -define v -vector -dense $(HAMMING_CODE_ROWS_IN_BINARY_RANKS) -end \
16817 ▷ ▷ -define C -code -field F \
16818 ▷ ▷ ▷ -basis 7 v \
16819 ▷ ▷ -end \
16820 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
16821 ▷ ▷ ▷ -make_diagram \
16822 ▷ ▷ -end
16823 ▷ $(ORBITER) -v 2 \
16824 ▷ ▷ -draw_matrix \
16825 ▷ ▷ ▷ -input_csv_file by_basis_n7_k4_char_func_7_16.csv \
16826 ▷ ▷ ▷ -box_width 25 -bit_depth 24 \
16827 ▷ ▷ ▷ -partition 4 16 8 \
16828 ▷ ▷ -end
16829 ▷ $(OPEN) by_basis_n7_k4_char_func_7_16.draw.bmp
16830
16831
16832 Hamming_code_words:
16833 ▷ $(ORBITER) -v 2 \
16834 ▷ ▷ -define v -vector -dense $(HAMMING_CODE_CODEWORDS) -end \
16835 ▷ ▷ -define F -finite_field -q 2 -end \
16836 ▷ ▷ -loop_over i v \
16837 ▷ ▷ ▷ -with F -do -coding_theoretic_activity \
16838 ▷ ▷ ▷ -general_code_binary 7 "Hamming_7_4_word_%i" "%i[v]" \
16839 ▷ ▷ ▷ -metric_balls 1 \
16840 ▷ ▷ ▷ -end \
16841 ▷ ▷ -end_loop_over i
16842 ▷ $(ORBITER) -v 2 \
16843 ▷ ▷ -loop i 0 16 1 -draw_matrix \
16844 ▷ ▷ ▷ -input_csv_file Hamming_7_4_word_%i_char_func_7_1.csv \
16845 ▷ ▷ ▷ -box_width 25 -bit_depth 8 \
16846 ▷ ▷ ▷ -partition 4 16 8 -end \
16847 ▷ ▷ ▷ -system "magick Hamming_7_4_word_%i_char_func_7_1.draw.bmp \
16848 ▷ ▷ ▷ -frame 8 Hamming_7_4_word_%i_char_func_7_1.draw.png" \
16849 ▷ ▷ -end_loop i
16850
16851
16852
16853 Hamming_weight_enumerator:
16854 ▷ $(ORBITER) -v 2 \
16855 ▷ ▷ -define F -finite_field -q 2 -end \
16856 ▷ ▷ -define v -vector -field F -format 4 \
16857 ▷ ▷ ▷ -dense $(HAMMING_CODE_GENERATOR) \
16858 ▷ ▷ -end \
16859 ▷ ▷ -define C -code -field F \
16860 ▷ ▷ ▷ -generator_matrix v \
16861 ▷ ▷ -end \
16862 ▷ ▷ -with C -do \
16863 ▷ ▷ -coding_theoretic_activity \
16864 ▷ ▷ ▷ -weight_enumerator \
16865 ▷ ▷ -end
16866
16867 Hamming_minimum_distance:
16868 ▷ $(ORBITER) -v 2 \
16869 ▷ ▷ -define F -finite_field -q 2 -end \

```

```

16870 ▷ ▷ -define v -vector -field F -format 4 \
16871 ▷ ▷ ▷ -dense $(HAMMING_CODE_GENERATOR) \
16872 ▷ ▷ -end \
16873 ▷ ▷ -with F -do \
16874 ▷ ▷ -coding_theoretic_activity \
16875 ▷ ▷ ▷ -minimum_distance v \
16876 ▷ ▷ -end
16877
16878
16879 Golay23_minimum_distance:
16880 ▷ $(ORBITER) -v 2 \
16881 ▷ ▷ -define F -finite_field -q 2 -end \
16882 ▷ ▷ -define v -vector -field F -format 12 \
16883 ▷ ▷ ▷ -dense $(GOLAY23_CODE_GENERATOR) \
16884 ▷ ▷ -end \
16885 ▷ ▷ -with F -do \
16886 ▷ ▷ -coding_theoretic_activity \
16887 ▷ ▷ ▷ -minimum_distance v \
16888 ▷ ▷ -end
16889
16890 #d=7 in 0 sec
16891
16892
16893
16894
16895 code_Hamming_systematic:
16896 ▷ $(ORBITER) -v 2 \
16897 ▷ ▷ -define F -finite_field -q 2 -end \
16898 ▷ ▷ -define v -vector -dense $(HAMMING_CODE_ROWS_IN_BINARY_RANKS) -end \
16899 ▷ ▷ -define C -code -field F \
16900 ▷ ▷ ▷ -basis 7 v \
16901 ▷ ▷ -end
16902 ▷ #$(ORBITER) -v 2 -draw_matrix \
16903 ▷ #▷ -input_csv_file code_matrix_16.8.csv \
16904 ▷ #▷ -box_width 25 -bit_depth 8 -partition 2 16 8 -end
16905 ▷ #$(OPEN) code_matrix_16.8.draw.bmp
16906 ▷ #$(PDFLATEX) code_7.16.tex
16907 ▷ #$(OPEN) code_7.16.pdf
16908
16909
16910 #▷ $(DRAW_PATH)/draw_matrix.out -read code_matrix_256.256.csv -box_width 10
16911
16912 # 1 0 0 0 0 1 1 = 67
16913 # 0 1 0 0 1 0 1 = 37
16914 # 0 0 1 0 1 1 0 = 22
16915 # 0 0 0 1 1 1 1 = 15
16916
16917
16918
16919 Hamming_RREF:
16920 ▷ $(ORBITER) -v 2 \
16921 ▷ ▷ -define F -finite_field -q 2 -end \
16922 ▷ ▷ -define v -vector -format 4 -field F \
16923 ▷ ▷ ▷ -dense $(HAMMING_CODE_GENERATOR) \
16924 ▷ ▷ -end \
16925 ▷ ▷ -with F -do \
16926 ▷ ▷ -finite_field_activity \
16927 ▷ ▷ ▷ -RREF v \
16928 ▷ ▷ -end

```

```

16929 ▷ $(PDFLATEX) v_rref.tex
16930 ▷ $(OPEN) v_rref.pdf
16931 ▷ #$(PDFLATEX) RREF_example_q2_4_7.tex
16932 ▷ #gs -sDEVICE=png16 -dFIXEDMEDIA \
16933 ▷ #> -dDEVICEWIDTHPOINTS=500 -dDEVICEHEIGHTPOINTS=450 \
16934 ▷ #> -r240 -oHamming_dual_page%02d.png \
16935 ▷ #> RREF_example_q2_4_7.pdf
16936
16937
16938
16939 Hamming_nullspace:
16940 ▷ $(ORBITER) -v 2 \
16941 ▷ ▷ -define F2 -finite_field -q 2 -end \
16942 ▷ ▷ -define v -vector -format 4 -field F2 \
16943 ▷ ▷ ▷ -dense $(HAMMING_CODE_GENERATOR) \
16944 ▷ ▷ -end \
16945 ▷ ▷ -with F2 -do \
16946 ▷ ▷ -finite_field.activity \
16947 ▷ ▷ -nullspace v \
16948 ▷ ▷ -end
16949 ▷ $(PDFLATEX) v_nullspace.tex
16950 ▷ $(OPEN) v_nullspace.pdf
16951
16952
16953 #check equations of the Hamming code:
16954 # a4+a5+a6+a7 =1+0+1+0=0 mod2 OK.
16955 # a2+a3+a6+a7 =0+1+1+0=0 mod2 OK.
16956 # a1+a3+a5+a7 =1+1+0+0=0 mod2 OK.
16957
16958 #1010101
16959 #0110011
16960 #0001111
16961
16962
16963
16964
16965
16966 #####
16967 # Section 11.3: Coding Theory - Golay codes
16968
16969 SECTION.CODING.THEORY.GOLAY.CODES:
16970
16971
16972 test_11_3:
16973 ▷ make Golay23.code.words
16974 ▷ make Golay23.code.diagram
16975 ▷ #make Golay23.code.diagram.draw
16976 ▷ make Golay24.make.code
16977 ▷ make Golay24.make.aut.gens
16978 ▷ make Golay24.make.aut.gens.classes
16979 ▷ make M24_elt_2A
16980 ▷ make M24_elt_2B
16981 ▷ make M24_2A.restricted
16982 ▷ make M24_2B.restricted
16983 ▷ make Golay24_2A.fixed.subcode
16984 ▷ make Golay24_2B.fixed.subcode
16985 ▷ make Golay24_2A.fix.code
16986 ▷ make Golay24_2B.fix.code
16987 ▷ make Golay24_2A.fix.weight12

```



```

16988 ▷ make Golay24_2A_fix_weight12.c
16989 ▷ make Golay24_2A_fix_weight12_graph_24
16990
16991
16992
16993
16994 Golay23_code_words:
16995 ▷ $(ORBITER) -v 2 \
16996 ▷ ▷ -define v -vector -dense $(GOLAY_23.COLUMN.RANKS.PROJECTIVELY) -end \
16997 ▷ ▷ -define F -finite_field -q 2 -end \
16998 ▷ ▷ -define C -code -field F \
16999 ▷ ▷ ▷ -projective_set 12 v -end \
17000 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
17001 ▷ ▷ ▷ -export_magma Golay23.magma \
17002 ▷ ▷ -end \
17003 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
17004 ▷ ▷ ▷ -export_codewords Golay23.codewords.csv \
17005 ▷ ▷ -end \
17006 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
17007 ▷ ▷ ▷ -export_genma Golay23.genma.csv \
17008 ▷ ▷ -end
17009 ▷ #$(PDFLATEX) code_n23_k12_q2.tex
17010 ▷ #$(OPEN) code_n23_k12_q2.pdf
17011
17012
17013 Golay23_code_diagram:
17014 ▷ $(ORBITER) -v 2 \
17015 ▷ ▷ -define F -finite_field -q 2 -end \
17016 ▷ ▷ -define words -vector -file Golay23.codewords.csv -end \
17017 ▷ ▷ -with F -do \
17018 ▷ ▷ -coding_theoretic_activity \
17019 ▷ ▷ ▷ -general_code_binary 23 "Golay_23" \
17020 ▷ ▷ ▷ words \
17021 ▷ ▷ ▷ -embellish 4 \
17022 ▷ ▷ -end
17023
17024
17025 Golay23_code_diagram_draw:
17026 ▷ $(ORBITER) -v 2 \
17027 ▷ ▷ -draw_matrix \
17028 ▷ ▷ ▷ -input_csv_file Golay_23.diagram_01_23_4096.csv \
17029 ▷ ▷ ▷ -box_width 4 -bit_depth 8 \
17030 ▷ ▷ ▷ -partition 20 4096 2048 \
17031 ▷ ▷ -end
17032
17033
17034 Golay24_make_code:
17035 ▷ $(ORBITER) -v 2 \
17036 ▷ ▷ -define F -finite_field -q 2 -end \
17037 ▷ ▷ -define v -vector \
17038 ▷ ▷ ▷ -dense $(GOLAY_24.CODE.PROJECTIVE) \
17039 ▷ ▷ -end \
17040 ▷ ▷ -define C -code -field F \
17041 ▷ ▷ ▷ -projective_set 12 v \
17042 ▷ ▷ -end
17043
17044
17045 Golay24_make_aut_gens:
17046 ▷ $(ORBITER) -v 2 \

```

```

17047 ▷ ▷ -define Do -draw.options \
17048 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
17049 ▷ ▷ -end \
17050 ▷ ▷ -define F -finite_field -q 2 -end \
17051 ▷ ▷ -define G -linear_group -PGL 12 2 \
17052 ▷ ▷ -subgroup_by_generators "M24" "244823040" 12 \
17053 ▷ ▷ ▷ $(GOLAY_24_CODE_AUT_GENS) \
17054 ▷ ▷ -end \
17055 ▷ ▷ -with G -do \
17056 ▷ ▷ -group_theoretic_activity \
17057 ▷ ▷ ▷ -report Do \
17058 ▷ ▷ -end
17059 ▷ $(PDFLATEX) PGL_12_2_Subgroup.M24.244823040_report.tex
17060 ▷ $(OPEN) PGL_12_2_Subgroup.M24.244823040_report.pdf
17061
17062
17063 Golay24_make_aut_gens_classes:
17064 ▷ $(ORBITER) -v 3 \
17065 ▷ ▷ -define F -finite_field -q 2 -end \
17066 ▷ ▷ -define G -linear_group -PGL 12 2 \
17067 ▷ ▷ -subgroup_by_generators "M24" "244823040" 12 \
17068 ▷ ▷ ▷ $(GOLAY_24_CODE_AUT_GENS) \
17069 ▷ ▷ -end \
17070 ▷ ▷ -with G -do \
17071 ▷ ▷ -group_theoretic_activity \
17072 ▷ ▷ ▷ -classes \
17073 ▷ ▷ -end
17074 ▷ $(MAGMA.EXE) M24_classes.magma
17075 ▷ $(ORBITER) -v 3 \
17076 ▷ ▷ -define F -finite_field -q 2 -end \
17077 ▷ ▷ -define G -linear_group -PGL 12 2 \
17078 ▷ ▷ -subgroup_by_generators "M24" "244823040" 12 \
17079 ▷ ▷ ▷ $(GOLAY_24_CODE_AUT_GENS) \
17080 ▷ ▷ -end \
17081 ▷ ▷ -with G -do \
17082 ▷ ▷ -group_theoretic_activity \
17083 ▷ ▷ ▷ -classes \
17084 ▷ ▷ -end
17085 ▷ $(PDFLATEX) PGL_12_2_Subgroup.M24.244823040_classes_out.tex
17086 ▷ $(OPEN) PGL_12_2_Subgroup.M24.244823040_classes_out.pdf
17087
17088
17089
17090 M24_elt_2A:
17091 ▷ $(ORBITER) -v 2 \
17092 ▷ ▷ -define Do -draw.options \
17093 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
17094 ▷ ▷ -end \
17095 ▷ ▷ -define F -finite_field -q 2 -end \
17096 ▷ ▷ -define G -linear_group -PGL 12 2 \
17097 ▷ ▷ -subgroup_by_generators "M24_2A" "2" 1 \
17098 ▷ ▷ ▷ $(M24_ELEMENT_2A) \
17099 ▷ ▷ -end \
17100 ▷ ▷ -with G -do \
17101 ▷ ▷ -group_theoretic_activity \
17102 ▷ ▷ ▷ -report Do \
17103 ▷ ▷ -end
17104 ▷ $(PDFLATEX) PGL_12_2_Subgroup.M24.2A.2_report.tex
17105 ▷ $(OPEN) PGL_12_2_Subgroup.M24.2A.2_report.pdf

```

```

17106
17107 M24_elt.2B:
17108 ▷ $(ORBITER) -v 2 \
17109 ▷ ▷ -define Do -draw_options \
17110 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
17111 ▷ ▷ -end \
17112 ▷ ▷ -define F -finite_field -q 2 -end \
17113 ▷ ▷ -define G -linear_group -PGL 12 2 \
17114 ▷ ▷ -subgroup_by_generators "M24_2B" "2" 1 \
17115 ▷ ▷ ▷ $(M24_ELEMENT_2B) \
17116 ▷ ▷ -end \
17117 ▷ ▷ -with G -do \
17118 ▷ ▷ -group_theoretic_activity \
17119 ▷ ▷ ▷ -report Do \
17120 ▷ ▷ -end
17121 ▷ $(PDFLATEX) PGL_12_2_Subgroup_M24_2B_2_report.tex
17122 ▷ $(OPEN) PGL_12_2_Subgroup_M24_2B_2_report.pdf
17123
17124 M24_2A_restricted:
17125 ▷ $(ORBITER) -v 2 \
17126 ▷ ▷ -define v -vector \
17127 ▷ ▷ ▷ -dense $(GOLAY_24_CODE_PROJECTIVE) \
17128 ▷ ▷ -end \
17129 ▷ ▷ -define F -finite_field -q 2 -end \
17130 ▷ ▷ -define G -linear_group -PGL 12 2 \
17131 ▷ ▷ ▷ -subgroup_by_generators "M24_2A" "2" 1 \
17132 ▷ ▷ ▷ $(M24_ELEMENT_2A) \
17133 ▷ ▷ -end \
17134 ▷ ▷ -define M24_2Ar -modified_group \
17135 ▷ ▷ ▷ -from G \
17136 ▷ ▷ ▷ -restricted_action v \\_on\\_code \
17137 ▷ ▷ -end \
17138 ▷ ▷ -with M24_2Ar -do \
17139 ▷ ▷ ▷ -group_theoretic_activity \
17140 ▷ ▷ ▷ ▷ -print_elements_tex \
17141 ▷ ▷ -end
17142 ▷ $(PDFLATEX) PGL_12_2_res_v_elements.tex
17143 ▷ $(OPEN) PGL_12_2_res_v_elements.pdf
17144
17145
17146 # 8 fixed points, 8 cycles of length 2.
17147
17148
17149
17150
17151
17152 M24_2B_restricted:
17153 ▷ $(ORBITER) -v 2 \
17154 ▷ ▷ -define v -vector \
17155 ▷ ▷ ▷ -dense $(GOLAY_24_CODE_PROJECTIVE) \
17156 ▷ ▷ -end \
17157 ▷ ▷ -define F -finite_field -q 2 -end \
17158 ▷ ▷ -define G -linear_group -PGL 12 2 \
17159 ▷ ▷ ▷ -subgroup_by_generators "M24_2B" "2" 1 \
17160 ▷ ▷ ▷ $(M24_ELEMENT_2B) \
17161 ▷ ▷ -end \
17162 ▷ ▷ -define M24_2Br -modified_group \
17163 ▷ ▷ ▷ -from G -restricted_action v v \
17164 ▷ ▷ -end \

```

```

17165 ▷ ▷ -with M24.2Br -do \
17166 ▷ ▷ ▷ -group.theoretic.activity \
17167 ▷ ▷ ▷ ▷ -print.elements.tex \
17168 ▷ ▷ -end
17169 ▷ $(PDFLATEX) PGL_12.2_res.v.elements.tex
17170 ▷ $(OPEN) PGL_12.2_res.v.elements.pdf
17171
17172 # 12 cycles of length 2
17173
17174
17175
17176
17177 Golay24.2A.fixed_subcode:
17178 ▷ $(ORBITER) -v 2 \
17179 ▷ ▷ -define perm -vector \
17180 ▷ ▷ ▷ -dense $(M24.2A.PERM) \
17181 ▷ ▷ -end \
17182 ▷ ▷ -define F -finite.field -q 2 -end \
17183 ▷ ▷ -define v -vector \
17184 ▷ ▷ ▷ -dense $(GOLAY_24.CODE.PROJECTIVE) \
17185 ▷ ▷ -end \
17186 ▷ ▷ -define C -code -field F \
17187 ▷ ▷ ▷ -projective.set 12 v \
17188 ▷ ▷ -end \
17189 ▷ ▷ -with C -and F -do -coding.theoretic.activity \
17190 ▷ ▷ ▷ -fixed.code perm \
17191 ▷ ▷ -end
17192
17193 # 256 codewords
17194
17195 Golay24.2B.fixed_subcode:
17196 ▷ $(ORBITER) -v 2 \
17197 ▷ ▷ -define perm -vector \
17198 ▷ ▷ ▷ -dense $(M24.2B.PERM) \
17199 ▷ ▷ -end \
17200 ▷ ▷ -define F -finite.field -q 2 -end \
17201 ▷ ▷ -define v -vector \
17202 ▷ ▷ ▷ -dense $(GOLAY_24.CODE.PROJECTIVE) \
17203 ▷ ▷ -end \
17204 ▷ ▷ -define C -code -field F \
17205 ▷ ▷ ▷ -projective.set 12 v \
17206 ▷ ▷ -end \
17207 ▷ ▷ -with C -and F -do -coding.theoretic.activity \
17208 ▷ ▷ ▷ -fixed.code perm \
17209 ▷ ▷ -end
17210
17211 # 64 codewords
17212
17213
17214 Golay24.2A.fix_code:
17215 ▷ $(ORBITER) -v 2 \
17216 ▷ ▷ -define F -finite.field -q 2 -end \
17217 ▷ ▷ -define v -vector \
17218 ▷ ▷ ▷ -format 8 \
17219 ▷ ▷ ▷ -dense $(GOLAY24.2A.FIX.SUBCODE) \
17220 ▷ ▷ -end \
17221 ▷ ▷ -define C -code -field F \
17222 ▷ ▷ ▷ -generator.matrix v \
17223 ▷ ▷ -end \

```

```

17224 ▷ ▷ -with C -do \
17225 ▷ ▷ -coding_theoretic_activity \
17226 ▷ ▷ ▷ -weight_enumerator \
17227 ▷ ▷ -end \
17228 ▷ ▷ -with C -do \
17229 ▷ ▷ -coding_theoretic_activity \
17230 ▷ ▷ ▷ -export_codewords_by_weight Golay24_2A_fix \
17231 ▷ ▷ -end
17232
17233 #1y^{24} + 71x^8y^{16} + 112x^{12}y^{12} + 71x^{16}y^8 + 1x^{24}
17234 #weight enumerator:
17235 #( 1, 0, 0, 0, 0, 0, 0, 0, 71, 0, 0, 0, 112, 0, 0, 0, 71, 0, 0, 0, 0, 0, 0, 1
    )
17236
17237 # 112 codewords of weight 12:
17238 #Golay24_2A_fix_of_weight_12.csv
17239 #-rw-r--r-- 1 betten staff 79 Oct 5 21:10 Golay24_2A_fix_of_weight_0.c
    sv
17240 #-rw-r--r-- 1 betten staff 4008 Oct 5 21:10 Golay24_2A_fix_of_weight_8.c
    sv
17241 #-rw-r--r-- 1 betten staff 6367 Oct 5 21:10 Golay24_2A_fix_of_weight_12.
    csv
17242 #-rw-r--r-- 1 betten staff 4052 Oct 5 21:10 Golay24_2A_fix_of_weight_16.
    csv
17243 #-rw-r--r-- 1 betten staff 81 Oct 5 21:10 Golay24_2A_fix_of_weight_24.
    csv
17244
17245
17246 Golay24_2B_fix.code:
17247 ▷ $(ORBITER) -v 2 \
17248 ▷ ▷ -define F -finite_field -q 2 -end \
17249 ▷ ▷ -define v -vector \
17250 ▷ ▷ -format 6 \
17251 ▷ ▷ -dense $(GOLAY24_2B_FIX.SUBCODE) \
17252 ▷ ▷ -end \
17253 ▷ ▷ -define C -code -field F \
17254 ▷ ▷ ▷ -generator_matrix v \
17255 ▷ ▷ -end \
17256 ▷ ▷ -with C -do \
17257 ▷ ▷ -coding_theoretic_activity \
17258 ▷ ▷ ▷ -weight_enumerator \
17259 ▷ ▷ -end \
17260 ▷ ▷ -with C -do \
17261 ▷ ▷ -coding_theoretic_activity \
17262 ▷ ▷ ▷ -export_codewords_by_weight Golay24_2B_fix \
17263 ▷ ▷ -end
17264
17265 #1y^{24} + 15x^8y^{16} + 32x^{12}y^{12} + 15x^{16}y^8 + 1x^{24}
17266 #weight enumerator:
17267 #( 1, 0, 0, 0, 0, 0, 0, 0, 15, 0, 0, 0, 32, 0, 0, 0, 15, 0, 0, 0, 0, 0, 0, 1 )

17268 #-rw-r--r-- 1 betten staff 79 Oct 5 20:51 Golay24_2B_fix_of_weight_0.c
    sv
17269 #-rw-r--r-- 1 betten staff 851 Oct 5 20:51 Golay24_2B_fix_of_weight_8.c
    sv
17270 #-rw-r--r-- 1 betten staff 1802 Oct 5 20:51 Golay24_2B_fix_of_weight_12.
    csv
17271 #-rw-r--r-- 1 betten staff 855 Oct 5 20:51 Golay24_2B_fix_of_weight_16.
    csv

```

```

17272 #-rw-r--r-- 1 betten staff      80 Oct  5 20:51 Golay24_2B_fix_of_weight_24.
        csv
17273
17274
17275 Golay24_2A_fix_weight12:
17276 ▷ $(ORBITER) -v 2 \
17277 ▷ ▷ -define F -finite_field -q 2 -end \
17278 ▷ ▷ -define D -design -list_of_blocks_from_file \
17279 ▷ ▷ ▷ 24 Golay24_2A_fix_of_weight_12.csv "codeword" \
17280 ▷ ▷ -end \
17281 ▷ ▷ -with D -do \
17282 ▷ ▷ ▷ -design_activity \
17283 ▷ ▷ ▷ ▷ -export_flags \
17284 ▷ ▷ -end \
17285 ▷ ▷ -with D -do \
17286 ▷ ▷ ▷ -design_activity \
17287 ▷ ▷ ▷ ▷ -intersection_matrix \
17288 ▷ ▷ ▷ -save \
17289 ▷ ▷ -end
17290
17291 #▷ ▷ -define v -vector \
17292 #▷ ▷ ▷ -load_csv_data_column Golay24_2A_fix_of_weight_12.csv 2 \
17293 #▷ ▷ -end \
17294 # intersection numbers 24 and 28 and 56
17295 #-rw-r--r-- 1 betten staff      5626 Oct  5 21:02 blocks_v24_k12_inc.txt
17296 #-rw-r--r-- 1 betten staff      1755 Oct  5 21:02 blocks_v24_k12_AAt.csv
17297
17298
17299
17300 Golay24_2A_fix_weight12.c:
17301 ▷ $(ORBITER) -v 3 \
17302 ▷ ▷ -define Doinc -draw_incidence_structure_options \
17303 ▷ ▷ ▷ -width 60 -width_10 6 -end \
17304 ▷ ▷ -define C -combinatorial_object \
17305 ▷ ▷ ▷ -label Golay24_2A_fix_weight12 Golay24\_2A\_fix\_weight12 \
17306 ▷ ▷ ▷ -file_of_incidence_geometries blocks_v24_flags.txt 24 112 1344 \
17307 ▷ ▷ -end \
17308 ▷ ▷ -with C -do \
17309 ▷ ▷ -combinatorial_object_activity \
17310 ▷ ▷ ▷ -canonical_form \
17311 ▷ ▷ ▷ ▷ -save_ago \
17312 ▷ ▷ ▷ ▷ -save_transversal \
17313 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "Golay24_2A_fix_nauty_log.txt" -end \
17314 ▷ ▷ ▷ -end \
17315 ▷ ▷ -end \
17316 ▷ ▷ -with C -do \
17317 ▷ ▷ -combinatorial_object_activity \
17318 ▷ ▷ ▷ -post_processing \
17319 ▷ ▷ -end \
17320 ▷ ▷ -with C -do \
17321 ▷ ▷ -combinatorial_object_activity \
17322 ▷ ▷ ▷ -report \
17323 ▷ ▷ ▷ ▷ -export_flag_orbits \
17324 ▷ ▷ ▷ ▷ -show_incidence_matrices \
17325 ▷ ▷ ▷ ▷ -show_TDO \
17326 ▷ ▷ ▷ ▷ -show_TDA \
17327 ▷ ▷ ▷ ▷ -export_group_GAP \
17328 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
17329 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \

```

```

17330 ▷ ▷ ▷ -end \
17331 ▷ ▷ -end
17332 ▷ $(PDFLATEX) Golay24_2A_fix_weight12_classification.tex
17333 ▷ $(OPEN) Golay24_2A_fix_weight12_classification.pdf
17334 ▷ $(ORBITER) -v 2 \
17335 ▷ ▷ -define Do -draw_options -embedded -radius 150 \
17336 ▷ ▷ ▷ -nodes_empty \
17337 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
17338 ▷ ▷ -tree_draw \
17339 ▷ ▷ ▷ -file Golay24_2A_fix_nauty_log.txt \
17340 ▷ ▷ ▷ -draw_options Do \
17341 ▷ ▷ -end
17342 ▷ $(PDFLATEX) Golay24_2A_fix_nauty_log.draw.tex
17343 ▷ $(OPEN) Golay24_2A_fix_nauty_log.draw.pdf
17344
17345
17346
17347
17348 #Golay24_2A_fix_nauty_log.txt
17349
17350
17351 # stabilizer of order 2752512
17352
17353 Golay24_2A_fix_weight12_graph_24:
17354 ▷ $(ORBITER) -v 3 \
17355 ▷ ▷ -define Gamma -graph \
17356 ▷ ▷ ▷ -load_adjacency_matrix_from_csv_and_select_value \
17357 ▷ ▷ ▷ blocks.v24_AAt.csv 24 \
17358 ▷ ▷ -end \
17359 ▷ ▷ -with Gamma -do \
17360 ▷ ▷ ▷ -graph_theoretic_activity -automorphism_group \
17361 ▷ ▷ -end \
17362 ▷ ▷ -with Gamma -do \
17363 ▷ ▷ ▷ -graph_theoretic_activity \
17364 ▷ ▷ ▷ -properties \
17365 ▷ ▷ -end
17366 ▷ #$(PDFLATEX) _report.tex
17367 ▷ #$(OPEN) _report.pdf
17368
17369 #Degree type: (14^{16},\, 7^8 )
17370 # go = 416179814400
17371
17372
17373
17374 #####
17375 # Section 11.4: Coding Theory - CRC codes
17376
17377 SECTION_CODING_THEORY_CRC_CODES:
17378
17379
17380 test_11_4:
17381 ▷ #make CRC_3_128_10 # too slow
17382 ▷ make crc32_test
17383 ▷ make crc32_test_hexdata
17384 ▷ make crc32_Berlekamp_matrix
17385 ▷ make CRC_F256_roots_771
17386 ▷ make CRC_F256_BCH_code_d2
17387 ▷ make CRC_F256_BCH_write_code_for_division_d2
17388 ▷ make CRC_F256_BCH_code_d3

```

```

17389 ▷ make CRC_F256_BCH_write_code_for_division_Bravo
17390 ▷ make CRC_F256_BCH_code_d7
17391 ▷ make CRC_F256_BCH_write_code_for_division_Charlie
17392 ▷ make F256_BCH_code_d16
17393 ▷ make F256_BCH_write_code_for_division_d16
17394 ▷ make F256_BCH_code_d16_division
17395 ▷ make F256_BCH_code_d16_error
17396 ▷ make F_256
17397 ▷ make CRC_F_16
17398 ▷ make nth_roots_256
17399 ▷ make CRC_F16_roots_51
17400 ▷ make CRC_F16_BCH_code_d3
17401 ▷ make CRC_F16_BCH_code_d3_mindist_dual
17402 ▷ make CRC_F16_BCH_code_d3_dual_weight_enumerator
17403 ▷ make CRC_F16_BCH_code_d3_projective_set
17404 ▷ make CRC_F16_BCH_code_d3_macwilliams
17405 ▷ make CRC_F16_BCH_write_code_for_division_Delta
17406 ▷ make crc_encode_16
17407 ▷ make crc_encode_32
17408
17409
17410
17411 CRC_3_128_10:
17412 ▷ $(ORBITER) -v 1 \
17413 ▷ ▷ -define F -finite_field -q 2 -end \
17414 ▷ ▷ -with F -do -coding_theoretic_activity \
17415 ▷ ▷ ▷ -find_CRC_polynomials 3 128 10 \
17416 ▷ ▷ -end
17417
17418
17419
17420
17421 crc32_test:
17422 ▷ $(ORBITER) -v 3 \
17423 ▷ ▷ -define F -finite_field -q 2 -end \
17424 ▷ ▷ -with F -do -coding_theoretic_activity \
17425 ▷ ▷ ▷ -crc32 "123456789" \
17426 ▷ ▷ -end
17427
17428 crc32_test_hexdata:
17429 ▷ $(ORBITER) -v 3 \
17430 ▷ ▷ -define F -finite_field -q 2 -end \
17431 ▷ ▷ -with F -do -coding_theoretic_activity \
17432 ▷ ▷ ▷ -crc32_hexdata "7BD11C4010" \
17433 ▷ ▷ -end
17434
17435 crc32_Berlekamp_matrix:
17436 ▷ $(ORBITER) -v 2 \
17437 ▷ ▷ -define F -finite_field -q 2 -end \
17438 ▷ ▷ -define v -vector -field F -sparse 33 $(CRC32_ETHERNET) -end \
17439 ▷ ▷ -with F -do \
17440 ▷ ▷ -finite_field_activity \
17441 ▷ ▷ ▷ -Berlekamp_matrix v \
17442 ▷ ▷ -end
17443
17444
17445 CRC_F256_roots_771:
17446 ▷ $(ORBITER) -v 3 \
17447 ▷ ▷ -define F -finite_field -q 256 -end \

```



```

17448 ▷ ▷ -with F -do -finite_field.activity \
17449 ▷ ▷ ▷ -nth_roots 771 \
17450 ▷ ▷ -end
17451
17452
17453 # alfa:
17454 # 255 = 3 * 5 * 17
17455 # 771 = 3 * 257
17456
17457 CRC_F256_BCH_code_d2:
17458 ▷ $(ORBITER) -v 2 \
17459 ▷ ▷ -define F -finite_field -q 256 -end \
17460 ▷ ▷ -define C -code -field F \
17461 ▷ ▷ ▷ -BCH 771 2 \
17462 ▷ ▷ -end \
17463 ▷ ▷ -with C -and F -do -coding_theoretic.activity \
17464 ▷ ▷ ▷ -export_magma BCH.q8.n771_d2.magma \
17465 ▷ ▷ -end
17466 ▷ #$(PDFLATEX) BCH.codes.q256.n771_d2.tex
17467 ▷ #$(OPEN) BCH.codes.q256.n771_d2.pdf
17468
17469 #Q=X^{2} + 167X + 214
17470 # degree of polynomial = 3
17471 #-dense "214,167,1"
17472 #-sparse "214,0,167,1,1,2"
17473
17474
17475
17476 CRC_F256_BCH_write_code_for_division_d2:
17477 ▷ $(ORBITER) -v 2 \
17478 ▷ ▷ -define F -finite_field -q 256 -end \
17479 ▷ ▷ -define A -vector -field F -sparse 772 "1,771,1,0" -end \
17480 ▷ ▷ -define B -vector -field F -dense $(CRC.POLY_Q256_DEG2_DENSE) -end \
17481 ▷ ▷ -with F -do \
17482 ▷ ▷ -coding_theoretic.activity \
17483 ▷ ▷ ▷ -write_code_for_division \
17484 ▷ ▷ ▷ alfa A B \
17485 ▷ ▷ -end
17486 ▷ g++ crc_alfa.cpp -o crc_alfa.out
17487 ▷ ./crc_alfa.out
17488
17489
17490
17491
17492
17493 # bravo:
17494
17495
17496
17497 # degree of polynomial = 4
17498 #-dense "1,23,27,213,1"
17499 #-sparse "1,0,23,1,27,2,213,3,1,4"
17500
17501 CRC_F256_BCH_code_d3:
17502 ▷ $(ORBITER) -v 2 \
17503 ▷ ▷ -define F -finite_field -q 256 -end \
17504 ▷ ▷ -define C -code -field F \
17505 ▷ ▷ ▷ -BCH 771 3 \
17506 ▷ ▷ -end \

```

```

17507 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
17508 ▷ ▷ ▷ -export_magma BCH.q8.n771.d3.magma \
17509 ▷ ▷ -end
17510 ▷ #$(PDFLATEX) BCH.codes.q256.n771.d3.tex
17511 ▷ #$(OPEN) BCH.codes.q256.n771.d3.pdf
17512
17513
17514
17515 CRC_F256_BCH_write_code_for_division.Bravo:
17516 ▷ $(ORBITER) -v 2 \
17517 ▷ ▷ -define F -finite_field -q 256 -end \
17518 ▷ ▷ -define A -vector -field F -sparse 772 "1,771,1,0" -end \
17519 ▷ ▷ -define B -vector -field F -dense $(CRC_POLY_BRAVO_DENSE) -end \
17520 ▷ ▷ -with F -do \
17521 ▷ ▷ -coding_theoretic_activity \
17522 ▷ ▷ ▷ -write_code_for_division \
17523 ▷ ▷ ▷ bravo A B \
17524 ▷ ▷ -end
17525 ▷ g++ crc_bravo.cpp -o crc_bravo.out
17526 ▷ ./crc_bravo.out
17527
17528
17529 # Charlie
17530
17531 CRC_F256_BCH_code_d7:
17532 ▷ $(ORBITER) -v 2 \
17533 ▷ ▷ -define F -finite_field -q 256 -end \
17534 ▷ ▷ -define C -code -field F \
17535 ▷ ▷ ▷ -BCH 771 7 \
17536 ▷ ▷ -end \
17537 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
17538 ▷ ▷ ▷ -export_magma BCH.q8.n771.d7.magma \
17539 ▷ ▷ -end \
17540 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
17541 ▷ ▷ ▷ -report \
17542 ▷ ▷ -end
17543 ▷ #$(PDFLATEX) BCH.n771.d7_code.n771.k759.q256.tex
17544 ▷ #$(OPEN) BCH.n771.d7_code.n771.k759.q256.pdf
17545
17546
17547 # polynomial of degree 12:
17548 #-dense "1,126,25,1,196,209,244,3,121,126,35,65,1"
17549 #-sparse "1,0,126,1,25,2,1,3,196,4,209,5,244,6,3,7,121,8,126,9,35,10,65,11,1,12"
17550
17551
17552 CRC_F256_BCH_write_code_for_division.Charlie:
17553 ▷ $(ORBITER) -v 2 \
17554 ▷ ▷ -define F -finite_field -q 256 -end \
17555 ▷ ▷ -define A -vector -field F -sparse 772 "1,771,1,0" -end \
17556 ▷ ▷ -define B -vector -field F -dense $(CRC_POLY_CHARLIE_DENSE) -end \
17557 ▷ ▷ -with F -do \
17558 ▷ ▷ -coding_theoretic_activity \
17559 ▷ ▷ ▷ -write_code_for_division \
17560 ▷ ▷ ▷ charlie A B \
17561 ▷ ▷ -end
17562 ▷ g++ crc_charlie.cpp -o crc_charlie.out
17563 ▷ ./crc_charlie.out
17564
17565

```

```

17566
17567
17568
17569
17570 F256_BCH_code.d16:
17571 ▷ $(ORBITER) -v 3 \
17572 ▷ ▷ -define F -finite_field -q 256 -end \
17573 ▷ ▷ -define C -code -field F \
17574 ▷ ▷ ▷ -BCH 771 16 \
17575 ▷ ▷ -end \
17576 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
17577 ▷ ▷ ▷ -report \
17578 ▷ ▷ -end
17579 ▷ $(PDFLATEX) BCH_n771_d16_code.n771_k741.q256.tex
17580 ▷ $(OPEN) BCH_n771_d16_code.n771_k741.q256.pdf
17581
17582 #generator polynomial is  $X^{30} + 253X^{29} + 174X^{28} + 109X^{27} + 97X^{26} +$ 
     $144X^{25} + 112X^{24} + 212X^{23} + 192X^{22} + 169X^{21} + 24X^{20} + 150X^{19}$ 
     $+ 110X^{18} + 248X^{17} + 3X^{16} + 193X^{15} + 194X^{14} + 205X^{13} + 9X^{12} +$ 
     $56X^{11} + 95X^{10} + 199X^9 + 108X^8 + 58X^7 + 160X^6 + 148X^5 + 138X$ 
     $^4 + 24X^3 + 210X^2 + 26X + 1$ 
17583
17584
17585
17586
17587
17588 F256_BCH_write_code_for_division.d16:
17589 ▷ $(ORBITER) -v 2 \
17590 ▷ ▷ -define F -finite_field -q 256 -end \
17591 ▷ ▷ -define A -vector -field F -sparse 772 "1,771,1,0" -end \
17592 ▷ ▷ -define B -vector -field F -dense $(POLY.Q256_DEG30_DENSE) -end \
17593 ▷ ▷ -with F -do \
17594 ▷ ▷ -coding_theoretic_activity \
17595 ▷ ▷ ▷ -write_code_for_division \
17596 ▷ ▷ ▷ check.q256_n771_r30 A B \
17597 ▷ ▷ -end
17598 ▷ g++ crc_check.q256_n771_r30.cpp -o crc_check.q256_n771_r30.out
17599 ▷ ./crc_check.q256_n771_r30.out
17600
17601
17602
17603 F256_BCH_code.d16_division:
17604 ▷ $(ORBITER) -v 2 \
17605 ▷ ▷ -define F -finite_field -q 256 -end \
17606 ▷ ▷ -define A -vector -field F -sparse 772 "1,771,1,0" -end \
17607 ▷ ▷ -define B -vector -field F -dense $(POLY.Q256_DEG30_DENSE) -end \
17608 ▷ ▷ -with F -do \
17609 ▷ ▷ -finite_field_activity \
17610 ▷ ▷ -polynomial_division A B -end
17611
17612
17613
17614 F256_BCH_code.d16_error:
17615 ▷ $(ORBITER) -v 2 \
17616 ▷ ▷ -define F -finite_field -q 256 -end \
17617 ▷ ▷ -define A -vector -field F -sparse 771 "2,30,3,31,55,770" -end \
17618 ▷ ▷ -define B -vector -field F -dense $(POLY.Q256_DEG30_DENSE) -end \
17619 ▷ ▷ -with F -do \
17620 ▷ ▷ -finite_field_activity \

```

```

17621 ▷ ▷ -polynomial_division A B -end
17622
17623
17624
17625
17626 # delta:
17627 # 15 = 3 * 5
17628 # 51 = 3 * 17
17629
17630 F_256:
17631 ▷ $(ORBITER) -v 3 \
17632 ▷ ▷ -define F -finite_field -q 256 -end \
17633 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
17634 ▷ $(PDFLATEX) GF_256.tex
17635 ▷ $(OPEN) GF_256.pdf
17636
17637
17638 # subfield F16: X^4+X+1
17639
17640
17641 # F16: X^4 + X3 + 1 not compatible with F_256 and x mapsto x^17
17642
17643 CRC_F16:
17644 ▷ $(ORBITER) -v 3 \
17645 ▷ ▷ -define F -finite_field -q 16 -override_polynomial 19 -end \
17646 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
17647 ▷ $(PDFLATEX) GF_16_poly19.tex
17648 ▷ $(OPEN) GF_16_poly19.pdf
17649
17650 nth_roots_256:
17651 ▷ $(ORBITER) -v 3 \
17652 ▷ ▷ -define F -finite_field -q 256 -end \
17653 ▷ ▷ -with F -do -finite_field_activity \
17654 ▷ ▷ ▷ -nth_roots 51 \
17655 ▷ ▷ -end
17656 ▷ $(PDFLATEX) Nth_roots_q256_n51.tex
17657 ▷ $(OPEN) Nth_roots_q256_n51.pdf
17658
17659
17660 CRC_F16_roots_51:
17661 ▷ $(ORBITER) -v 3 \
17662 ▷ ▷ -define F -finite_field -q 16 -override_polynomial 19 -end \
17663 ▷ ▷ -with F -do -finite_field_activity \
17664 ▷ ▷ ▷ -nth_roots 51 \
17665 ▷ ▷ -end
17666 ▷ $(PDFLATEX) Nth_roots_q16_n51.tex
17667 ▷ $(OPEN) Nth_roots_q16_n51.pdf
17668
17669 CRC_F16_BCH_code_d3:
17670 ▷ $(ORBITER) -v 2 \
17671 ▷ ▷ -define F -finite_field -q 16 -override_polynomial 19 -end \
17672 ▷ ▷ -define C -code -field F \
17673 ▷ ▷ ▷ -BCH 51 3 \
17674 ▷ ▷ -end \
17675 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
17676 ▷ ▷ ▷ -export_magma BCH_q16_n51_d3.magma \
17677 ▷ ▷ -end \
17678 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
17679 ▷ ▷ ▷ -export_checkma BCH_q16_n51_d3.check.csv \

```

```

17680 ▷ ▷ -end \
17681 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
17682 ▷ ▷ ▷ -export_checkma_as_projective_set BCH.q16.n51.d3.check_ranks.csv \
17683 ▷ ▷ -end \
17684 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
17685 ▷ ▷ ▷ -report \
17686 ▷ ▷ -end
17687 ▷ $(PDFLATEX) BCH.n51.d3.code.n51.k47.q16.tex
17688 ▷ $(OPEN) BCH.n51.d3.code.n51.k47.q16.pdf
17689
17690 #BCH.q16.n51.d3.check.csv
17691 #BCH.q16.n51.d3.check_ranks.csv
17692
17693 CRC_F16_BCH_code.d3.mindist.dual:
17694 ▷ $(ORBITER) -v 2 \
17695 ▷ ▷ -define F -finite_field -q 16 -override_polynomial 19 -end \
17696 ▷ ▷ -define v -vector -file BCH.q16.n51.d3.check.csv \
17697 ▷ ▷ ▷ -field F -format 4 -file BCH.q16.n51.d3.check.csv -end \
17698 ▷ ▷ -with F -do \
17699 ▷ ▷ -coding_theoretic_activity \
17700 ▷ ▷ ▷ -minimum_distance v \
17701 ▷ ▷ -end
17702
17703 CRC_F16_BCH_code.d3.dual_weight_enumerator:
17704 ▷ $(ORBITER) -v 2 \
17705 ▷ ▷ -define F -finite_field -q 16 -override_polynomial 19 -end \
17706 ▷ ▷ -define v -vector -file BCH.q16.n51.d3.check.csv \
17707 ▷ ▷ ▷ -field F -format 4 -file BCH.q16.n51.d3.check.csv -end \
17708 ▷ ▷ -define C -code -field F \
17709 ▷ ▷ ▷ -generator_matrix v \
17710 ▷ ▷ ▷ -end \
17711 ▷ ▷ -with C -do \
17712 ▷ ▷ -coding_theoretic_activity \
17713 ▷ ▷ ▷ -weight_enumerator \
17714 ▷ ▷ -end
17715
17716 #weight enumerator:
17717 #( 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3060, 0, 18360, 0, 25755, 0
    , 18360, 0 )
17718 #1y{51} + 3060x{44}y{7} + 18360x{46}y{5} + 25755x{48}y{3} + 18360x{50}y
17719 # 1 + 3060 + 18360 + 25755 + 18360 = 65536 = 2{16} = 16{4}
17720
17721
17722 CRC_F16_BCH_code.d3.projective.set:
17723 ▷ $(ORBITER) -v 2 \
17724 ▷ ▷ -define F -finite_field -q 16 -override_polynomial 19 -end \
17725 ▷ ▷ -define v -vector -file BCH.q16.n51.d3.check_ranks.csv \
17726 ▷ ▷ ▷ -file BCH.q16.n51.d3.check_ranks.csv -end \
17727
17728
17729
17730
17731 CRC_F16_BCH_code.d3.macwilliams:
17732 ▷ $(ORBITER) -v 2 \
17733 ▷ ▷ -make_macwilliams_system 51 4 16
17734
17735
17736

```

```

17737 CRC_F16_BCH_write_code_for_division_Delta:
17738 ▷ $(ORBITER) -v 2 \
17739 ▷ ▷ -define F -finite_field -q 16 -override_polynomial 19 -end \
17740 ▷ ▷ -define A -vector -field F -sparse 52 "1,51,1,0" -end \
17741 ▷ ▷ -define B -vector -field F -dense $(CRC.POLY_DELTA.DENSE) -end \
17742 ▷ ▷ -with F -do \
17743 ▷ ▷ -coding_theoretic_activity \
17744 ▷ ▷ ▷ -write_code_for_division \
17745 ▷ ▷ ▷ delta A B \
17746 ▷ ▷ -end
17747 ▷ g++ crc_delta.cpp -o crc_delta.out
17748 ▷ ./crc_delta.out
17749
17750 #g++ crc_delta.cpp -o crc_delta.out
17751 #./crc_delta.out
17752 #0,0,0,0,0,
17753
17754
17755 crc_compare_Echo_crc32_Oben.e5:
17756 ▷ $(ORBITER) -v 3 \
17757 ▷ ▷ -define F -finite_field -q 256 -end \
17758 ▷ ▷ -define Echo -crc_code -type Echo -block_length 25 -end \
17759 ▷ ▷ -define CRC32 -crc_code -type crc32 -block_length 25 -end \
17760 ▷ ▷ -with F -do \
17761 ▷ ▷ ▷ -coding_theoretic_activity \
17762 ▷ ▷ ▷ ▷ -crc_compare \
17763 ▷ ▷ ▷ ▷ ▷ Oben.mp4 \
17764 ▷ ▷ ▷ ▷ ▷ Echo \
17765 ▷ ▷ ▷ ▷ ▷ CRC32 \
17766 ▷ ▷ ▷ ▷ ▷ 5 \
17767 ▷ ▷ -end
17768
17769
17770 crc_encode_16:
17771 ▷ $(ORBITER) -v 3 \
17772 ▷ ▷ -define F -finite_field -q 2 -end \
17773 ▷ ▷ -define CRC16 -crc_code -type crc16 -block_length 771 -end \
17774 ▷ ▷ -with F -do \
17775 ▷ ▷ ▷ -coding_theoretic_activity \
17776 ▷ ▷ ▷ ▷ -crc_encode_file_based \
17777 ▷ ▷ ▷ ▷ $(CRC.FILE).$(CRC.FILE.EXTENSION) \
17778 ▷ ▷ ▷ ▷ $(CRC.FILE)_crc16.bin CRC16 \
17779 ▷ ▷ -end
17780
17781 #-rw-r--r-- 1 betten staff 646576 Oct 7 08:12 allen.Gates_crc16.bin
17782 # old:
17783 #-rw-r--r-- 1 betten staff 646576 Aug 24 14:35 allen.Gates_crc16.bin
17784
17785
17786
17787 crc_encode_32:
17788 ▷ $(ORBITER) -v 3 \
17789 ▷ ▷ -define F -finite_field -q 2 -end \
17790 ▷ ▷ -define CRC32 -crc_code -type crc32 \
17791 ▷ ▷ ▷ -block_length 771 \
17792 ▷ ▷ -end \
17793 ▷ ▷ -with F -do \
17794 ▷ ▷ ▷ -coding_theoretic_activity \
17795 ▷ ▷ ▷ ▷ -crc_encode_file_based \

```

```

17796 ▷ ▷ ▷ ▷ $(CRC_FILE).$(CRC_FILE_EXTENSION) \
17797 ▷ ▷ ▷ ▷ $(CRC_FILE)_crc32.bin CRC32 \
17798 ▷ ▷ -end
17799
17800
17801 #coding_theory_domain::crc_encode_file_based input file size = 644898
17802 #coding_theory_domain::crc_encode_file_based nb_blocks = 841
17803 #Written file allen.Gates_crc32.bin of size 648262
17804 #nb_blocks = 841
17805
17806
17807
17808 #####
17809 # Section 11.5: Coding Theory - Reed-Muller codes
17810
17811 SECTION_CODING_THEORY_REED_MULLER_CODES:
17812
17813
17814
17815
17816 test_11_5:
17817 ▷ make RM_3_export
17818 ▷ make RM_3_Hamming_space_diagram
17819 ▷ make RM_3_draw
17820 ▷ make RM_3_split
17821 ▷ make RM_3_distance_draw
17822 ▷ make RM_3_algebraic_normal_form
17823 ▷ make RM_6
17824 ▷ make RM_6_genma_draw
17825 ▷ make RM_6_codewords_draw
17826
17827
17828
17829 RM_3_export:
17830 ▷ $(ORBITER) -v 2 \
17831 ▷ ▷ -define F -finite_field -q 2 -end \
17832 ▷ ▷ -define C -code -field F \
17833 ▷ ▷ ▷ -Reed_Muller 3 \
17834 ▷ ▷ -end \
17835 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
17836 ▷ ▷ ▷ -export_magma RM_3.magma \
17837 ▷ ▷ -end \
17838 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
17839 ▷ ▷ ▷ -export_codewords_long RM_3_codewords.csv \
17840 ▷ ▷ -end
17841
17842 #RM_3.magma
17843 #RM_3_codewords.csv
17844
17845
17846 RM_3_Hamming_space_diagram:
17847 ▷ $(ORBITER) -v 2 \
17848 ▷ ▷ -define F -finite_field -q 2 -end \
17849 ▷ ▷ -define C -code -field F \
17850 ▷ ▷ ▷ -Reed_Muller 3 \
17851 ▷ ▷ -end \
17852 ▷ ▷ -with C -do \
17853 ▷ ▷ -coding_theoretic_activity \
17854 ▷ ▷ ▷ -make_diagram \

```

```

17855 ▷ ▷ ▷ -metric_balls 1 \
17856 ▷ ▷ -end
17857 ▷ pdflatex code_n8_k4_q2.tex
17858 ▷ open code_n8_k4_q2.pdf
17859
17860
17861 # creates
17862 #code_n8_k4_q2.tex
17863 #RM_m3_distance_8_16.csv
17864 #RM_m3_distance_H_8_16.csv
17865 #RM_m3_idx_8_16.csv
17866 #RM_m3_char_func_8_16.csv
17867 #RM_m3_8_16.tex
17868
17869
17870 RM_3_draw:
17871 ▷ $(ORBITER) -v 2 \
17872 ▷ ▷ -draw_matrix \
17873 ▷ ▷ ▷ -input_csv_file RM_m3_distance_H_8_16.csv \
17874 ▷ ▷ ▷ -box_width 25 -bit_depth 8 \
17875 ▷ ▷ ▷ -partition 4 16 16 \
17876 ▷ ▷ -end
17877 ▷ $(ORBITER) -v 2 \
17878 ▷ ▷ -draw_matrix \
17879 ▷ ▷ ▷ -input_csv_file RM_m3_char_func_8_16.csv \
17880 ▷ ▷ ▷ -box_width 25 -bit_depth 8 \
17881 ▷ ▷ ▷ -partition 4 16 16 \
17882 ▷ ▷ -end
17883 ▷ $(ORBITER) -v 2 \
17884 ▷ ▷ -draw_matrix \
17885 ▷ ▷ ▷ -input_csv_file RM_m3_idx_8_16.csv \
17886 ▷ ▷ ▷ -box_width 25 -bit_depth 8 \
17887 ▷ ▷ ▷ -partition 4 16 16 \
17888 ▷ ▷ -end
17889 ▷ magick RM_m3_distance_H_8_16_draw.bmp RM_m3_distance_H_8_16_draw.png
17890 ▷ magick RM_m3_char_func_8_16_draw.bmp RM_m3_char_func_8_16_draw.png
17891 ▷ magick RM_m3_idx_8_16_draw.bmp RM_m3_idx_8_16_draw.png
17892 ▷ $(OPEN) RM_m3_distance_H_8_16_draw.bmp
17893
17894
17895 RM_3_split:
17896 ▷ $(ORBITER) -split_by_values RM_m3_distance_H_8_16.csv
17897
17898 RM_3_distance_draw:
17899 ▷ $(ORBITER) -v 2 \
17900 ▷ ▷ -loop L 0 3 1 \
17901 ▷ ▷ ▷ -draw_matrix \
17902 ▷ ▷ ▷ ▷ -input_csv_file RM_m3_distance_H_8_16_value%.csv \
17903 ▷ ▷ ▷ ▷ -box_width 25 -bit_depth 8 \
17904 ▷ ▷ ▷ ▷ -partition 5 16 16 \
17905 ▷ ▷ ▷ -end \
17906 ▷ ▷ -end_loop L
17907 ▷ magick RM_m3_distance_H_8_16_value0_draw.bmp RM_m3_distance_H_8_16_value0_draw.
    png
17908 ▷ magick RM_m3_distance_H_8_16_value1_draw.bmp RM_m3_distance_H_8_16_value1_draw.
    png
17909 ▷ magick RM_m3_distance_H_8_16_value2_draw.bmp RM_m3_distance_H_8_16_value2_draw.
    png
17910 ▷ $(OPEN) RM_m3_distance_H_8_16_value0_draw.png

```



```

17911 ▷ $(OPEN) RM_m3_distance.H.8.16_value1_draw.png
17912 ▷ $(OPEN) RM_m3_distance.H.8.16_value2_draw.png
17913
17914
17915 RM_3_algebraic_normal_form:
17916 ▷ $(ORBITER) -v 3 \
17917 ▷ ▷ -define F -finite_field -q 2 -end \
17918 ▷ ▷ -loop L 0 3 1 \
17919 ▷ ▷ ▷ -define v%L -vector -field F \
17920 ▷ ▷ ▷ ▷ -file RM_m3_distance.H.8.16_value%L.csv \
17921 ▷ ▷ ▷ -end \
17922 ▷ ▷ ▷ -with F -do -finite_field_activity \
17923 ▷ ▷ ▷ ▷ -algebraic_normal_form 8 v%L \
17924 ▷ ▷ ▷ -end \
17925 ▷ ▷ -end_loop L
17926
17927
17928 RM_6:
17929 ▷ $(ORBITER) -v 2 \
17930 ▷ ▷ -define F -finite_field -q 2 -end \
17931 ▷ ▷ -define C -code -field F \
17932 ▷ ▷ ▷ -long_code 64 7 \
17933 ▷ ▷ ▷ $(RM_6_GENERATOR_1) \
17934 ▷ ▷ ▷ $(RM_6_GENERATOR_2) \
17935 ▷ ▷ ▷ $(RM_6_GENERATOR_3) \
17936 ▷ ▷ ▷ $(RM_6_GENERATOR_4) \
17937 ▷ ▷ ▷ $(RM_6_GENERATOR_5) \
17938 ▷ ▷ ▷ $(RM_6_GENERATOR_6) \
17939 ▷ ▷ ▷ $(RM_6_GENERATOR_7) \
17940 ▷ ▷ -end \
17941 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
17942 ▷ ▷ ▷ -export_genma RM6_genma.csv \
17943 ▷ ▷ -end \
17944 ▷ ▷ -with C -and F -do -coding_theoretic_activity \
17945 ▷ ▷ ▷ -export_codewords_long RM6_codewords.csv \
17946 ▷ ▷ -end \
17947
17948
17949 RM_6_genma_draw:
17950 ▷ $(ORBITER) -v 2 \
17951 ▷ ▷ -draw_matrix \
17952 ▷ ▷ ▷ -input_csv_file RM6_genma.csv \
17953 ▷ ▷ ▷ -box_width 25 -bit_depth 8 \
17954 ▷ ▷ ▷ -partition 3 7 64 \
17955 ▷ ▷ -end
17956 ▷ magick RM6_genma_draw.bmp RM6_genma_draw.png
17957 ▷ open RM6_genma_draw.png
17958
17959
17960
17961
17962 RM_6_codewords_draw:
17963 ▷ $(ORBITER) -v 4 \
17964 ▷ ▷ -loop i 0 128 1 \
17965 ▷ ▷ ▷ -csv_file_select_rows RM6_codewords.csv %i \
17966 ▷ ▷ ▷ -mv RM6_codewords_select.csv RM6_codewords_select_%i.csv \
17967 ▷ ▷ ▷ -destring RM6_codewords_select_%i.csv "Codeword" RM6_codewords_sel_%i.csv \
17968 ▷ ▷ ▷ -reformat RM6_codewords_sel_%i.csv RM6_codewords_ref_%i.csv 8 \

```

```

17969 ▷ ▷ ▷ -draw_matrix \
17970 ▷ ▷ ▷ ▷ -input_csv_file RM6_codewords_ref_%i.csv \
17971 ▷ ▷ ▷ ▷ -box_width 25 -bit_depth 8 \
17972 ▷ ▷ ▷ ▷ -partition 3 8 8 \
17973 ▷ ▷ ▷ -end \
17974 ▷ ▷ ▷ -system "magick RM6_codewords_ref_%i-draw.bmp -frame 8 %i.png" \
17975 ▷ ▷ -end_loop i
17976 ▷ magick 0.png 1.png 2.png 3.png 4.png 5.png \
17977 ▷ ▷ 6.png 7.png 8.png 9.png 10.png +append a0
17978 ▷ magick 11.png 12.png 13.png 14.png 15.png \
17979 ▷ ▷ 16.png 17.png 18.png 19.png 20.png 21.png +append a1
17980 ▷ magick 22.png 23.png 24.png 25.png 26.png \
17981 ▷ ▷ 27.png 28.png 29.png 30.png 31.png 32.png +append a2
17982 ▷ magick 33.png 34.png 35.png 36.png 37.png \
17983 ▷ ▷ 38.png 39.png 40.png 41.png 42.png 43.png +append a3
17984 ▷ magick 44.png 45.png 46.png 47.png 48.png \
17985 ▷ ▷ 49.png 50.png 51.png 52.png 53.png 54.png +append a4
17986 ▷ magick 55.png 56.png 57.png 58.png 59.png \
17987 ▷ ▷ 60.png 61.png 62.png 63.png 64.png 65.png +append a5
17988 ▷ magick 66.png 67.png 68.png 69.png 70.png \
17989 ▷ ▷ 71.png 72.png 73.png 74.png 75.png 76.png +append a6
17990 ▷ magick 77.png 78.png 79.png 80.png 81.png \
17991 ▷ ▷ 82.png 83.png 84.png 85.png 86.png 87.png +append a7
17992 ▷ magick 88.png 89.png 90.png 91.png 92.png \
17993 ▷ ▷ 93.png 94.png 95.png 96.png 97.png 98.png +append a8
17994 ▷ magick 99.png 100.png 101.png 102.png 103.png \
17995 ▷ ▷ 104.png 105.png 106.png 107.png 108.png 109.png +append a9
17996 ▷ magick 110.png 111.png 112.png 113.png 114.png \
17997 ▷ ▷ 115.png 116.png 117.png 118.png 119.png 120.png +append a10
17998 ▷ magick 121.png 122.png 123.png 124.png 125.png \
17999 ▷ ▷ 126.png 127.png 0.png 0.png 0.png 0.png +append a11
18000 ▷ magick a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 a10 \
18001 ▷ ▷ a11 -append poster_RM_1.6.png
18002
18003
18004
18005
18006
18007 #####
18008 # Section 11.6: Coding Theory - BCH Codes
18009
18010 SECTION_CODING_THEORY_BCH_CODES:
18011
18012
18013 test_11_6:
18014 ▷ make draw_cyclotomic_mod_51_q16
18015 ▷ make draw_cyclotomic_mod_21_q8
18016 ▷ make F_8_BCH_code_d3
18017 ▷ make F_8_BCH_code_d4
18018 ▷ make F_8_BCH_code_d5
18019 ▷ make F_8_BCH_code_d5_minimum_distance
18020 ▷ make F_8_BCH_code_d7
18021 ▷ make F8_BCH_code_n63_d43
18022 ▷ #make F8_BCH_code_n63_d43_minimum_distance
18023 ▷ make F2_BCH_code_n21
18024 ▷ make F7_RS_code_n6
18025 ▷ make F_64_again
18026 ▷ make BCH_15_5
18027 ▷ make draw_mod_31

```

```

18028 ▷ make PR127
18029 ▷ make draw_mod_127_power
18030 ▷ make draw_mod_251
18031 ▷ make draw_mod_255_cyclotomic_1
18032 ▷ make draw_mod_255_cyclotomic_3
18033 ▷ make draw_mod_255_cyclotomic_1_and_3
18034 ▷ make draw_mod_63_4_cyclotomic_3_6
18035 ▷ make BCH_F_64
18036 ▷ make BCH_21_poly_mult_mod_F4
18037 ▷ make BCH_21_poly_division_a
18038 ▷ make BCH_21_poly_division_b
18039 ▷ make BCH_21_poly_division_ab
18040 ▷ make BCH_21_generator_matrix
18041 ▷ #make BCH_21_15_weight_enumerator
18042 ▷ make BCH_21_15_dual
18043 ▷ make BCH_21_6_weight_enumerator
18044 ▷ make BCH_21_6_4_macwilliams
18045 ▷ make BCH_21_15_4_field_reduction
18046 ▷ make BCH_21_poly_division_c
18047 ▷ make F16_roots_5
18048 ▷ make F64_roots_21
18049 ▷ make BCH_F256_roots_771
18050 ▷ make BCH_F256_BCH_code_d16
18051
18052
18053 draw_cyclotomic_mod_51_q16:
18054 ▷ $(ORBITER) -v 2 \
18055 ▷ ▷ -define Do -draw_options \
18056 ▷ ▷ ▷ -radius 100 \
18057 ▷ ▷ ▷ -line_width 1.0 -embedded \
18058 ▷ ▷ -end \
18059 ▷ ▷ -draw_mod_n \
18060 ▷ ▷ ▷ "mod_n=on" \
18061 ▷ ▷ ▷ -n 51 \
18062 ▷ ▷ ▷ -cyclotomic_sets 16 "1,2" \
18063 ▷ ▷ ▷ -file cyclotomic_51 \
18064 ▷ ▷ ▷ -label_nodes \
18065 ▷ ▷ ▷ -draw_options Do \
18066 ▷ ▷ -end
18067 ▷ $(PDFLATEX) cyclotomic_51_draw.tex
18068 ▷ $(OPEN) cyclotomic_51_draw.pdf
18069
18070 #▷ ▷ ▷ -cyclotomic_sets 8 "1,2,4,5,7,10,13" \
18071
18072
18073 draw_cyclotomic_mod_21_q8:
18074 ▷ $(ORBITER) -v 2 \
18075 ▷ ▷ -define Do -draw_options \
18076 ▷ ▷ ▷ -radius 100 \
18077 ▷ ▷ ▷ -line_width 1.0 -embedded \
18078 ▷ ▷ -end \
18079 ▷ ▷ -draw_mod_n \
18080 ▷ ▷ ▷ "mod_n=on" \
18081 ▷ ▷ ▷ -n 21 \
18082 ▷ ▷ ▷ -file mod_21_cyclotomic \
18083 ▷ ▷ ▷ -cyclotomic_sets 8 "1,2,4,5,7,10,13" \
18084 ▷ ▷ ▷ -label_nodes \
18085 ▷ ▷ ▷ -draw_options Do \
18086 ▷ ▷ -end

```

```

18087 ▷ $(PDFLATEX) mod_21_cyclotomic_draw.tex
18088 ▷ $(OPEN) mod_21_cyclotomic_draw.pdf
18089
18090
18091 F_8_BCH_code_d3:
18092 ▷ $(ORBITER) -v 3 \
18093 ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
18094 ▷ ▷ -define C -code -field F \
18095 ▷ ▷ ▷ -BCH 21 3 \
18096 ▷ ▷ -end \
18097 ▷ ▷ -with C -do -coding_theoretic_activity -report -end
18098 ▷ $(PDFLATEX) BCH_n21_d3_code_n21_k17.q8.tex
18099 ▷ $(OPEN) BCH_n21_d3_code_n21_k17.q8.pdf
18100
18101 #generator polynomial is  $X^4 + 4X^3 + 4X^2 + 3X + 4$ 
18102
18103 F_8_BCH_code_d4:
18104 ▷ $(ORBITER) -v 3 \
18105 ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
18106 ▷ ▷ -define C -code -field F \
18107 ▷ ▷ ▷ -BCH 21 4 \
18108 ▷ ▷ -end \
18109 ▷ ▷ -with C -do -coding_theoretic_activity -report -end
18110 ▷ $(PDFLATEX) BCH_n21_d4_code_n21_k16.q8.tex
18111 ▷ $(OPEN) BCH_n21_d4_code_n21_k16.q8.pdf
18112
18113 #generator polynomial is  $X^5 + 6X^4 + 7X^3 + 2X + 3$ 
18114
18115
18116 F_8_BCH_code_d5:
18117 ▷ $(ORBITER) -v 3 \
18118 ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
18119 ▷ ▷ -define C -code -field F \
18120 ▷ ▷ ▷ -BCH 21 5 \
18121 ▷ ▷ -end \
18122 ▷ ▷ -with C -do -coding_theoretic_activity -report -end
18123 ▷ $(PDFLATEX) BCH_n21_d5_code_n21_k14.q8.tex
18124 ▷ $(OPEN) BCH_n21_d5_code_n21_k14.q8.pdf
18125
18126 #-override_polynomial 11
18127 #generator polynomial is  $X^7 + 3X^6 + 3X^5 + 2X^4 + X^3 + 2X^2 + X + 2$ 
18128
18129 F_8_BCH_code_d5_minimum_distance:
18130 ▷ $(ORBITER) -v 2 \
18131 ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
18132 ▷ ▷ -define v -vector -format 14 -field F \
18133 ▷ ▷ ▷ -compact $(CODE_BCH_F8_N21_D5_GENMA_OVERRIDE_POLYNOMIAL11) \
18134 ▷ ▷ -end \
18135 ▷ ▷ -with F -do \
18136 ▷ ▷ -coding_theoretic_activity \
18137 ▷ ▷ ▷ -minimum_distance v \
18138 ▷ ▷ -end
18139 # important: use the same polynomial as when creating the code.
18140 #
18141 # d=5
18142
18143
18144

```

```

18145 F_8_BCH_code_d7:
18146 ▷ $(ORBITER) -v 3 \
18147 ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
18148 ▷ ▷ -define C -code -field F \
18149 ▷ ▷ ▷ -BCH 21 7 \
18150 ▷ ▷ -end \
18151 ▷ ▷ -with C -do -coding_theoretic_activity -report -end
18152 ▷ $(PDFLATEX) BCH_n21_d7_code_n21_k11.q8.tex
18153 ▷ $(OPEN) BCH_n21_d7_code_n21_k11.q8.pdf
18154
18155
18156
18157 F8_BCH_code_n63_d43:
18158 ▷ $(ORBITER) -v 3 \
18159 ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
18160 ▷ ▷ -define C -code -field F \
18161 ▷ ▷ ▷ -BCH 63 43 \
18162 ▷ ▷ -end \
18163 ▷ ▷ -with C -do -coding_theoretic_activity -report -end
18164 ▷ $(PDFLATEX) BCH_n63_d43_code_n63_k9.q8.tex
18165 ▷ $(OPEN) BCH_n63_d43_code_n63_k9.q8.pdf
18166
18167
18168
18169 # slow
18170
18171 F8_BCH_code_n63_d43_minimum_distance:
18172 ▷ $(ORBITER) -v 2 \
18173 ▷ ▷ -define F -finite_field -q 8 -override_polynomial 11 -end \
18174 ▷ ▷ -define v -vector -format 9 -field F \
18175 ▷ ▷ ▷ -compact $(CODE_BCH_F8_N63_K9_D43_GENMA) \
18176 ▷ ▷ -end \
18177 ▷ ▷ -with F -do \
18178 ▷ ▷ -coding_theoretic_activity \
18179 ▷ ▷ ▷ -minimum_distance v \
18180 ▷ ▷ -end
18181
18182
18183 #coding_theory_domain::do_minimum_distance The minimum distance is d = 45, computed in 0 days, 0 hours, 1 minutes, 32 seconds
18184 #1:32
18185
18186
18187
18188
18189
18190
18191 F2_BCH_code_n21:
18192 ▷ $(ORBITER) -v 3 \
18193 ▷ ▷ -define F -finite_field -q 2 -end \
18194 ▷ ▷ -define C -code -field F \
18195 ▷ ▷ ▷ -BCH 21 3 \
18196 ▷ ▷ -end \
18197 ▷ ▷ -with C -do -coding_theoretic_activity -report -end
18198 ▷ $(PDFLATEX) BCH_n21_d3_code_n21_k15.q2.tex
18199 ▷ $(OPEN) BCH_n21_d3_code_n21_k15.q2.pdf
18200
18201
18202

```

```

18203 F7_RS_code.n6:
18204 ▷ $(ORBITER) -v 3 \
18205 ▷ ▷ -define F -finite_field -q 7 -end \
18206 ▷ ▷ -define C -code -field F \
18207 ▷ ▷ ▷ -Reed_Solomon 6 3 \
18208 ▷ ▷ -end \
18209 ▷ ▷ -with C -do -coding_theoretic_activity -report -end
18210 ▷ $(PDFLATEX) RS_n6_d3_code.n6.k4.q7.tex
18211 ▷ $(OPEN) RS_n6_d3_code.n6.k4.q7.pdf
18212
18213
18214 F_64_again:
18215 ▷ $(ORBITER) -v 3 \
18216 ▷ ▷ -define F -finite_field -q 64 -end \
18217 ▷ ▷ -with F -do \
18218 ▷ ▷ ▷ -finite_field_activity \
18219 ▷ ▷ ▷ -cheat_sheet_GF \
18220 ▷ ▷ -end
18221 ▷ $(PDFLATEX) GF_64.tex
18222 ▷ $(OPEN) GF_64.pdf
18223
18224
18225
18226 BCH_15_5:
18227 ▷ $(ORBITER) \
18228 ▷ ▷ -define F -finite_field -q 2 -end \
18229 ▷ ▷ -define C -code -field F \
18230 ▷ ▷ ▷ -BCH 15 5 \
18231 ▷ ▷ -end \
18232 ▷ ▷ -with C -do -coding_theoretic_activity -report -end
18233 ▷ $(PDFLATEX) BCH_n15_d5_code.n15.k7.q2.tex
18234 ▷ $(OPEN) BCH_n15_d5_code.n15.k7.q2.pdf
18235
18236
18237
18238
18239
18240 draw_mod_31:
18241 ▷ $(ORBITER) -v 2 \
18242 ▷ ▷ -define Do -draw_options -embedded -radius 50 -line_width 0.3 -end \
18243 ▷ ▷ -draw_mod_n \
18244 ▷ ▷ ▷ "mod_n=on" \
18245 ▷ ▷ ▷ -n 31 \
18246 ▷ ▷ ▷ -file mod_31 \
18247 ▷ ▷ ▷ -power_cycle 2 \
18248 ▷ ▷ ▷ -label_nodes \
18249 ▷ ▷ ▷ -draw_options Do \
18250 ▷ ▷ -end
18251 ▷ $(PDFLATEX) mod_31_draw.tex
18252 ▷ $(OPEN) mod_31_draw.pdf
18253
18254
18255 PR127:
18256 ▷ $(ORBITER) -v 5 -primitive_root 127
18257
18258
18259 draw_mod_127_power:
18260 ▷ $(ORBITER) -v 2 \
18261 ▷ ▷ -define Do -draw_options -radius 50 -scale 0.4 -embedded -line_width 0.3 -end

```

```

\
18262 ▷ ▷ -draw_mod_n \
18263 ▷ ▷ ▷ "mod_n=on" \
18264 ▷ ▷ ▷ -n 127 \
18265 ▷ ▷ ▷ -dont_label_nodes \
18266 ▷ ▷ ▷ -file mod_127 \
18267 ▷ ▷ ▷ -power_cycle 3 \
18268 ▷ ▷ ▷ -draw_options Do \
18269 ▷ ▷ -end
18270 ▷ $(PDFLATEX) mod_127_draw.tex
18271 ▷ $(OPEN) mod_127_draw.pdf
18272
18273 draw_mod_251:
18274 ▷ $(ORBITER) -v 2 \
18275 ▷ ▷ -define Do -draw_options -nodes_empty -radius 10 -embedded -end \
18276 ▷ ▷ -draw_mod_n \
18277 ▷ ▷ ▷ "mod_n=on" \
18278 ▷ ▷ ▷ -n 251 \
18279 ▷ ▷ ▷ -file mod_251 \
18280 ▷ ▷ ▷ -label_nodes \
18281 ▷ ▷ ▷ -draw_options Do \
18282 ▷ ▷ -end
18283 ▷ $(PDFLATEX) mod_251_draw.tex
18284 ▷ $(OPEN) mod_251_draw.pdf
18285
18286 #-draw_mod_n_inverse
18287
18288
18289 draw_mod_255_cyclotomic_1:
18290 ▷ $(ORBITER) -v 2 \
18291 ▷ ▷ -define Do -draw_options -nodes_empty -radius 10 \
18292 ▷ ▷ ▷ -line_width 0.4 -embedded -end \
18293 ▷ ▷ -draw_mod_n \
18294 ▷ ▷ ▷ "mod_n=on" \
18295 ▷ ▷ ▷ -n 255 \
18296 ▷ ▷ ▷ -file mod_255_cyclotomic_1 \
18297 ▷ ▷ ▷ -label_nodes \
18298 ▷ ▷ ▷ -cyclotomic_sets 2 "1" \
18299 ▷ ▷ ▷ -draw_options Do \
18300 ▷ ▷ -end
18301 ▷ $(PDFLATEX) mod_255_cyclotomic_1_draw.tex
18302 ▷ $(OPEN) mod_255_cyclotomic_1_draw.pdf
18303
18304 draw_mod_255_cyclotomic_3:
18305 ▷ $(ORBITER) -v 2 \
18306 ▷ ▷ -define Do -draw_options -nodes_empty -radius 10 \
18307 ▷ ▷ ▷ -line_width 0.4 -embedded -end \
18308 ▷ ▷ -draw_mod_n \
18309 ▷ ▷ ▷ "mod_n=on" \
18310 ▷ ▷ ▷ -n 255 \
18311 ▷ ▷ ▷ -file mod_255_cyclotomic_3 \
18312 ▷ ▷ ▷ -label_nodes \
18313 ▷ ▷ ▷ -cyclotomic_sets 2 "3" \
18314 ▷ ▷ ▷ -draw_options Do \
18315 ▷ ▷ -end
18316 ▷ $(PDFLATEX) mod_255_cyclotomic_3_draw.tex
18317 ▷ $(OPEN) mod_255_cyclotomic_3_draw.pdf
18318
18319 draw_mod_255_cyclotomic_1_and_3:

```

```

18320 ▷ $(ORBITER) -v 2 \
18321 ▷ ▷ -define Do -draw_options -nodes_empty -radius 10 \
18322 ▷ ▷ ▷ -line_width 0.4 -embedded -end \
18323 ▷ ▷ -draw_mod_n \
18324 ▷ ▷ ▷ "mod_n=on" \
18325 ▷ ▷ ▷ -n 255 \
18326 ▷ ▷ ▷ -file mod_255_cyclotomic_1_and_3 \
18327 ▷ ▷ ▷ -label_nodes \
18328 ▷ ▷ ▷ -cyclotomic_sets 2 "1,3" \
18329 ▷ ▷ ▷ -draw_options Do \
18330 ▷ ▷ -end
18331 ▷ $(PDFLATEX) mod_255_cyclotomic_1_and_3_draw.tex
18332 ▷ $(OPEN) mod_255_cyclotomic_1_and_3_draw.pdf
18333
18334 draw_mod_63_4_cyclotomic_3_6:
18335 ▷ $(ORBITER) -v 2 \
18336 ▷ ▷ -define Do -draw_options -radius 20 \
18337 ▷ ▷ ▷ -line_width 0.1 -embedded -end \
18338 ▷ ▷ -draw_mod_n \
18339 ▷ ▷ ▷ "mod_n=on" \
18340 ▷ ▷ ▷ -n 63 \
18341 ▷ ▷ ▷ -file mod_63_4_cyclotomic_3_6 \
18342 ▷ ▷ ▷ -label_nodes \
18343 ▷ ▷ ▷ -cyclotomic_sets 4 "3,6" \
18344 ▷ ▷ ▷ -cyclotomic_sets_thickness 50 \
18345 ▷ ▷ ▷ -draw_options Do \
18346 ▷ ▷ -end
18347 ▷ $(PDFLATEX) mod_63_4_cyclotomic_3_6_draw.tex
18348 ▷ $(OPEN) mod_63_4_cyclotomic_3_6_draw.pdf
18349
18350 BCH_F_64:
18351 ▷ $(ORBITER) -v 3 \
18352 ▷ ▷ -define F -finite_field -q 64 -end \
18353 ▷ ▷ -with F -do -finite_field_activity \
18354 ▷ ▷ ▷ -cheat_sheet_GF \
18355 ▷ ▷ -end
18356 ▷ $(PDFLATEX) GF_64.tex
18357
18358
18359
18360
18361 BCH_21_poly_mult_mod_F4:
18362 ▷ $(ORBITER) -v 2 \
18363 ▷ ▷ -define F -finite_field -q 4 -end \
18364 ▷ ▷ -with F -do \
18365 ▷ ▷ -finite_field_activity \
18366 ▷ ▷ ▷ -polynomial_mult_mod "1,0,2,1" "1,0,3,1" \
18367 ▷ ▷ ▷ "1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1" \
18368 ▷ ▷ -end
18369
18370 #C(X)=X^{6} + X^{5} + X^{4} + X^{2} + 1
18371
18372 # poly 1,0,1,0,1,1,1
18373
18374
18375 BCH_21_poly_division_a:
18376 ▷ $(ORBITER) -v 2 \
18377 ▷ ▷ -define F -finite_field -q 4 -end \
18378 ▷ ▷ -with F -do \

```



```

18379 ▷ ▷ -finite_field_activity \
18380 ▷ ▷ -polynomial_division \
18381 ▷ ▷ ▷ "1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1" \
18382 ▷ ▷ ▷ "1,0,2,1" \
18383 ▷ ▷ -end
18384
18385 BCH_21_poly_division.b:
18386 ▷ $(ORBITER) -v 2 \
18387 ▷ ▷ -define F -finite_field -q 4 -end \
18388 ▷ ▷ -with F -do \
18389 ▷ ▷ -finite_field_activity \
18390 ▷ ▷ -polynomial_division \
18391 ▷ ▷ ▷ "1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1" \
18392 ▷ ▷ ▷ "1,0,3,1" \
18393 ▷ ▷ -end
18394
18395
18396 BCH_21_poly_division.ab:
18397 ▷ $(ORBITER) -v 2 \
18398 ▷ ▷ -define F -finite_field -q 4 -end \
18399 ▷ ▷ -with F -do \
18400 ▷ ▷ -finite_field_activity \
18401 ▷ ▷ -polynomial_division \
18402 ▷ ▷ ▷ "1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1" \
18403 ▷ ▷ ▷ "1,0,1,0,1,1,1" \
18404 ▷ ▷ -end
18405
18406 BCH_21_generator_matrix:
18407 ▷ $(ORBITER) -v 2 \
18408 ▷ ▷ -define F -finite_field -q 4 -end \
18409 ▷ ▷ -with F -do \
18410 ▷ ▷ -coding_theoretic_activity \
18411 ▷ ▷ ▷ -generator_matrix_cyclic_code \
18412 ▷ ▷ ▷ ▷ 21 "1,0,1,0,1,1,1" \
18413 ▷ ▷ -end
18414
18415
18416
18417
18418 BCH_21_15_weight_enumerator:
18419 ▷ $(ORBITER) -v 2 \
18420 ▷ ▷ -define F -finite_field -q 4 -end \
18421 ▷ ▷ -define v -vector -field F -format 15 \
18422 ▷ ▷ ▷ -dense $(BCH_21_15_GENERATOR_MATRIX) \
18423 ▷ ▷ -end \
18424 ▷ ▷ -define C -code -field F \
18425 ▷ ▷ ▷ -generator_matrix v \
18426 ▷ ▷ -end \
18427 ▷ ▷ -with C -do \
18428 ▷ ▷ -coding_theoretic_activity \
18429 ▷ ▷ ▷ -weight_enumerator \
18430 ▷ ▷ -end
18431
18432 # too slow!
18433
18434 BCH_21_15_dual:
18435 ▷ $(ORBITER) -v 2 \
18436 ▷ ▷ -define F -finite_field -q 4 -end \
18437 ▷ ▷ -define v -vector -field F -format 15 \

```

```

18438 ▷ ▷ ▷ -dense $(BCH_21_15_GENERATOR_MATRIX) -end \
18439 ▷ ▷ -with F -do -finite_field_activity \
18440 ▷ ▷ ▷ -nullspace v \
18441 ▷ ▷ -end
18442
18443
18444 BCH_21_6_weight_enumerator:
18445 ▷ $(ORBITER) -v 2 \
18446 ▷ ▷ -define F -finite_field -q 4 -end \
18447 ▷ ▷ -define v -vector -format 6 -field F \
18448 ▷ ▷ ▷ -dense $(BCH_21_6_GENERATOR_MATRIX) \
18449 ▷ ▷ -end \
18450 ▷ ▷ -define C -code -field F \
18451 ▷ ▷ ▷ -generator_matrix v \
18452 ▷ ▷ -end \
18453 ▷ ▷ -with C -do \
18454 ▷ ▷ -coding_theoretic_activity \
18455 ▷ ▷ ▷ -weight_enumerator \
18456 ▷ ▷ -end
18457
18458 #  $1y^{21} + 63x^8y^{13} + 294x^{12}y^9 + 756x^{14}y^7 + 1890x^{16}y^5 + 1092x^{18}y^3$ 
18459
18460 #( 1, 0, 0, 0, 0, 0, 0, 0, 63, 0, 0, 0, 294, 0, 756, 0, 1890, 0, 1092, 0, 0, 0 )
18461
18462
18463
18464 BCH_21_6_4_macwilliams:
18465 ▷ $(ORBITER) -v 2 \
18466 ▷ ▷ -make_macwilliams_system 21 6 4
18467 ▷ $(PDFLATEX) MacWilliams.n21.k6.q4.tex
18468 ▷ $(OPEN) MacWilliams.n21.k6.q4.pdf
18469
18470
18471
18472 #ww := [1, 0, 0, 84, 252, 1575, 10080, 58032, 319662, 1411116, 5133744, 15282792,
18473         37951620, 79336530, 135622080, 190615824, 213273081, 188911548, 125744304, 59721
18474         732, 17767512, 2580255]
18475
18476 BCH_21_15_4_field_reduction:
18477 ▷ $(ORBITER) -v 2 \
18478 ▷ ▷ -define F -finite_field -q 4 -end \
18479 ▷ ▷ -with F -do \
18480 ▷ ▷ -finite_field_activity \
18481 ▷ ▷ ▷ -field_reduction "BCH_21_15_4" 2 15 21 \
18482 ▷ ▷ ▷ $(BCH_21_15_GENERATOR_MATRIX) \
18483 ▷ ▷ -end
18484 ▷ $(ORBITER) -v 2 \
18485 ▷ ▷ -draw_matrix \
18486 ▷ ▷ ▷ -input_csv_file BCH_21_15_4.csv \
18487 ▷ ▷ ▷ -box_width 20 -bit_depth 24 \
18488 ▷ ▷ ▷ -partition 4 "30" "42" \
18489 ▷ ▷ -end
18490 ▷ $(PDFLATEX) field_reduction.Q4.q2.15.21.tex
18491 ▷ $(OPEN) BCH_21_15_4_draw.bmp
18492 ▷ #$(OPEN) field_reduction.Q4.q2.15.21.pdf
18493 # poly of degree 12: 1,0,1,0,1,0,0,0,1,0,0,0,1

```

[illegible]

```

18548
18549
18550
18551
18552
18553
18554
18555
18556 #####
18557 # Section 11.7: Coding Theory - Reed Solomon codes
18558
18559 SECTION_CODING_THEORY_REED_SOLOMON_CODES:
18560
18561
18562 test_11_7:
18563 ▷ make RS_6_3
18564 ▷ make RS_6_3_weight_enumerator
18565 ▷ make RREF_RS_6_4_7_weight_enumerator
18566 ▷ make Code_RS_11
18567 ▷ #make Code_RS_11_weight_enumerator
18568 ▷ make RS_7_3
18569 ▷ make RS_7_3_weight_enumerator
18570 ▷ make RS_7_3_field_reduction
18571 ▷ make RS_7_3_reduced_weight_enumerator
18572
18573
18574
18575 RS_6_3:
18576 ▷ $(ORBITER) -v 2 \
18577 ▷ ▷ -define F -finite_field -q 7 -end \
18578 ▷ ▷ -define C -code -field F -ReedSolomon 6 3 -end \
18579 ▷ ▷ -with C -do -coding_theoretic_activity \
18580 ▷ ▷ ▷ -export_magma RS_6_3.magma \
18581 ▷ ▷ -end \
18582 ▷ ▷ -with C -do -coding_theoretic_activity -report -end
18583 ▷ $(PDFLATEX) RS_n6_d3_code_n6.k4.q7.tex
18584 ▷ $(OPEN) RS_n6_d3_code_n6.k4.q7.pdf
18585
18586
18587 RS_6_3_weight_enumerator:
18588 ▷ $(ORBITER) -v 2 \
18589 ▷ ▷ -define F -finite_field -q 7 -end \
18590 ▷ ▷ -define C -code -field F -ReedSolomon 6 3 -end \
18591 ▷ ▷ -with C -do -coding_theoretic_activity \
18592 ▷ ▷ ▷ -export_magma RS_6_3.magma \
18593 ▷ ▷ -end \
18594 ▷ ▷ -with C -do \
18595 ▷ ▷ -coding_theoretic_activity \
18596 ▷ ▷ ▷ -weight_enumerator \
18597 ▷ ▷ -end
18598
18599 #[6,4,3]_7 code.
18600 #( 1, 0, 0, 120, 360, 972, 948 )
18601
18602 #1y^6 + 90x^4y^2 + 108x^5y + 144x^6
18603
18604 RREF_RS_6_4_7_weight_enumerator:
18605 ▷ $(ORBITER) -v 2 \
18606 ▷ ▷ -define F -finite_field -q 7 -end \

```

```

18607 ▷ ▷ -define v -vector -format 4 -field F \
18608 ▷ ▷ ▷ -compact $(CODE_RS_6_4_7) \
18609 ▷ ▷ -end \
18610 ▷ ▷ -define C -code -field F \
18611 ▷ ▷ ▷ -generator_matrix v \
18612 ▷ ▷ -end \
18613 ▷ ▷ -with C -do \
18614 ▷ ▷ -coding_theoretic_activity \
18615 ▷ ▷ ▷ -weight_enumerator \
18616 ▷ ▷ -end
18617
18618
18619 #1y^6 + 120x^3y^3 + 360x^4y^2 + 972x^5y + 948x^6
18620 #weight enumerator:
18621 #( 1, 0, 0, 120, 360, 972, 948 )
18622
18623
18624
18625
18626
18627 Code_RS_11:
18628 ▷ $(ORBITER) -v 2 \
18629 ▷ ▷ -define F -finite_field -q 11 -end \
18630 ▷ ▷ -define C -code -field F -ReedSolomon 10 3 -end \
18631 ▷ ▷ -with C -do -coding_theoretic_activity \
18632 ▷ ▷ ▷ -export_magma RS_11_3.magma \
18633 ▷ ▷ -end \
18634 ▷ ▷ -with C -do -coding_theoretic_activity -report -end
18635 ▷ $(PDFLATEX) RS_n10_d3_code_n10_k8.q11.tex
18636 ▷ $(OPEN) RS_n10_d3_code_n10_k8.q11.pdf
18637
18638
18639 # slow:
18640
18641 Code_RS_11_weight_enumerator:
18642 ▷ $(ORBITER) -v 2 \
18643 ▷ ▷ -define F -finite_field -q 11 -end \
18644 ▷ ▷ -define C -code -field F -ReedSolomon 10 3 -end \
18645 ▷ ▷ -with C -do -coding_theoretic_activity \
18646 ▷ ▷ ▷ -export_magma RS_11_3.magma \
18647 ▷ ▷ -end \
18648 ▷ ▷ -with C -do \
18649 ▷ ▷ -coding_theoretic_activity \
18650 ▷ ▷ ▷ -weight_enumerator \
18651 ▷ ▷ -end
18652
18653
18654 #1*y^(10) + 1200*x^3*y^7 + 16800*x^4*y^6 + 209160*x^5*y^5 + 1734600*x^6*y^4 + 991
      8000*x^7*y^3 + 37189800*x^8*y^2 + 82644700*x^9*y + 82644620*x^(10)
18655
18656
18657
18658 RS_7_3:
18659 ▷ $(ORBITER) -v 2 \
18660 ▷ ▷ -define F -finite_field -q 8 -end \
18661 ▷ ▷ -define C -code -field F -ReedSolomon 7 3 -end \
18662 ▷ ▷ -with C -do -coding_theoretic_activity \
18663 ▷ ▷ ▷ -export_magma RS_7_3.magma \
18664 ▷ ▷ -end \

```

```

18665 ▷ ▷ -with C -do -coding_theoretic_activity -report -end
18666 ▷ $(PDFLATEX) RS_n7_d3_code.n7.k5.q8.tex
18667 ▷ $(OPEN) RS_n7_d3_code.n7.k5.q8.pdf
18668
18669
18670 RS_7_3.weight.enumerator:
18671 ▷ $(ORBITER) -v 2 \
18672 ▷ ▷ -define F -finite_field -q 8 -end \
18673 ▷ ▷ -define C -code -field F -ReedSolomon 7 3 -end \
18674 ▷ ▷ -with C -do -coding_theoretic_activity \
18675 ▷ ▷ ▷ -export_magma RS_7_3.magma \
18676 ▷ ▷ -end \
18677 ▷ ▷ -with C -do \
18678 ▷ ▷ -coding_theoretic_activity \
18679 ▷ ▷ ▷ -weight_enumerator \
18680 ▷ ▷ -end
18681
18682
18683
18684 RS_7_3.field.reduction:
18685 ▷ $(ORBITER) -v 2 \
18686 ▷ ▷ -define F -finite_field -q 8 -end \
18687 ▷ ▷ -define C -code -field F -ReedSolomon 7 3 -end \
18688 ▷ ▷ -with F -do \
18689 ▷ ▷ -finite_field_activity \
18690 ▷ ▷ -field_reduction "RS_8_red_2" \
18691 ▷ ▷ ▷ 2 5 7 $(CODE_RS_F8_N7_K5_D3_GENMA) \
18692 ▷ ▷ -end
18693 ▷ $(ORBITER) -v 2 \
18694 ▷ ▷ -draw_matrix -input_csv_file RS_8_red_2.csv \
18695 ▷ ▷ -box_width 40 -bit_depth 24 \
18696 ▷ ▷ -partition 4 "3,3,3,3,3" "3,3,3,3,3,3,3" -end
18697 ▷ $(PDFLATEX) field_reduction_Q8.q2.5.7.tex
18698 ▷ $(OPEN) field_reduction_Q8.q2.5.7.pdf
18699
18700
18701 RS_7_3.reduced.weight.enumerator:
18702 ▷ $(ORBITER) -v 2 \
18703 ▷ ▷ -define F -finite_field -q 2 -end \
18704 ▷ ▷ -define v -vector -format 15 -field F \
18705 ▷ ▷ ▷ -compact $(RS_8_reduced) \
18706 ▷ ▷ -end \
18707 ▷ ▷ -define C -code -field F \
18708 ▷ ▷ ▷ -generator_matrix v \
18709 ▷ ▷ -end \
18710 ▷ ▷ -with C -do \
18711 ▷ ▷ -coding_theoretic_activity \
18712 ▷ ▷ ▷ -weight_enumerator \
18713 ▷ ▷ -end
18714
18715
18716
18717
18718 #####
18719 # Section 11.8: Coding Theory - Twisted tensor product codes
18720
18721 SECTION_CODING_THEORY_TWISTED_TENSOR_PRODUCT_CODES:
18722
18723

```

```

18724 test_11_8:
18725 ▷ ▷ make TTP_A_4
18726
18727
18728 TTP_A_4:
18729 ▷ $(ORBITER) -v 2 \
18730 ▷ ▷ -define F4 -finite_field -q 4 -end \
18731 ▷ ▷ -define F16 -finite_field -q 16 -end \
18732 ▷ ▷ -define C -code -field F4 -ttpA F16 -end \
18733 ▷ ▷ -with C -do -coding_theoretic_activity \
18734 ▷ ▷ -export_magma TTP_A.q4.magma \
18735 ▷ ▷ -end \
18736 ▷ ▷ -with C -do -coding_theoretic_activity \
18737 ▷ ▷ ▷ -report \
18738 ▷ ▷ -end
18739 ▷ $(PDFLATEX) ttpA_n18_k9_code_n18_k9.q4.tex
18740 ▷ $(OPEN) ttpA_n18_k9_code_n18_k9.q4.pdf
18741
18742
18743
18744 #####
18745 # Section 11.9: Coding Theory - Bounds
18746
18747 SECTION.CODING.THEORY.BOUNDS:
18748
18749
18750 test_11_9:
18751 ▷ make bounds_for_d_given_n6_k4_q7
18752 ▷ make bounds_for_d_given_n15_k6_q2
18753 ▷ make coding_theory_bounds_q2
18754 ▷ make coding_theory_bounds_q8
18755 ▷ make GV_n15_k6_d5
18756 ▷ make bounds_for_d_given_n12_k4_q13
18757 ▷ make GV_n15_k6_d5_weight_enumerator
18758 ▷ make code_n15_k6_d6_a_we
18759 ▷ make code_n15_k6_d6_RREF
18760 ▷ make code_n15_k6_d6_check_RREF
18761
18762
18763 bounds_for_d_given_n6_k4_q7:
18764 ▷ $(ORBITER) -v 2 \
18765 ▷ ▷ -make_bounds_for_d_given_n_and_k_and_q 6 4 7
18766 ▷
18767 bounds_for_d_given_n15_k6_q2:
18768 ▷ $(ORBITER) -v 2 \
18769 ▷ ▷ -make_bounds_for_d_given_n_and_k_and_q 15 6 2
18770
18771 #n = 15 k=6 q=2
18772 #d_GV = 5
18773 #d_singleton = 10
18774 #d_hamming = 6
18775 #d_plotkin = 7
18776 #d_griesmer = 6
18777
18778
18779 coding_theory_bounds_q2:
18780 ▷ $(ORBITER) -v 2 -table_of_bounds 12 2
18781 ▷ $(ORBITER) -v 2 \
18782 ▷ ▷ -csv_file_latex 1 table_of_bounds_n12.q2.csv

```

```

18783 ▷ $(PDFLATEX) table_of_bounds_n12.q2.tex
18784 ▷ $(OPEN) table_of_bounds_n12.q2.pdf
18785
18786 # produces table_of_bounds_n20.q2.csv
18787
18788 coding_theory_bounds_q8:
18789 ▷ $(ORBITER) -v 2 -table_of_bounds 20 8
18790
18791
18792 GV_n15_k6.d5:
18793 ▷ $(ORBITER) -v 2 \
18794 ▷ ▷ -define F -finite_field -q 2 -end \
18795 ▷ ▷ -define C -code -field F \
18796 ▷ ▷ ▷ -Gilbert_Varshamov 15 6 5 \
18797 ▷ ▷ -end
18798
18799
18800 # [15,6] code created
18801
18802
18803
18804 bounds_for_d_given_n12.k4.q13:
18805 ▷ $(ORBITER) -v 2 \
18806 ▷ ▷ -make_bounds_for_d_given_n_and_k_and_q 12 4 13
18807
18808
18809
18810
18811 GV_n15_k6.d5.weight_enumerator:
18812 ▷ $(ORBITER) -v 2 \
18813 ▷ ▷ -define F -finite_field -q 2 -end \
18814 ▷ ▷ -define v -vector -format 6 -field F \
18815 ▷ ▷ ▷ -compact $(CODE_GV_N15_K6) \
18816 ▷ ▷ -end \
18817 ▷ ▷ -define C -code -field F \
18818 ▷ ▷ ▷ -generator_matrix v \
18819 ▷ ▷ -end \
18820 ▷ ▷ -with C -do \
18821 ▷ ▷ -coding_theoretic_activity \
18822 ▷ ▷ ▷ -weight_enumerator \
18823 ▷ ▷ -end
18824
18825 # $1y^{15} + 27x^6y^9 + 24x^8y^7 + 9x^{10}y^5 + 3x^{12}y^3$ 
18826 # surprise:  $d = 6$ 
18827
18828
18829 code_n15_k6.d6.a.we:
18830 ▷ $(ORBITER) -v 2 \
18831 ▷ ▷ -define F -finite_field -q 2 -end \
18832 ▷ ▷ -define v -vector -format 6 -field F \
18833 ▷ ▷ ▷ -compact $(CODE_15_6_6.A) \
18834 ▷ ▷ -end \
18835 ▷ ▷ -define C -code -field F \
18836 ▷ ▷ ▷ -generator_matrix v \
18837 ▷ ▷ -end \
18838 ▷ ▷ -with C -do \
18839 ▷ ▷ -coding_theoretic_activity \
18840 ▷ ▷ ▷ -weight_enumerator \
18841 ▷ ▷ -end

```



```

18842
18843 #1y^{15} + 27x^6y^9 + 24x^8y^7 + 9x^{10}y^5 + 3x^{12}y^3
18844
18845
18846
18847 # weight enumerator
18848 #1y^{15} + 28x^6y^9 + 21x^8y^7 + 12x^{10}y^5 + 2x^{12}y^3
18849
18850
18851 code_n15_k6_d6_RREF:
18852 ▷ $(ORBITER) -v 2 \
18853 ▷ ▷ -define F -finite_field -q 2 -end \
18854 ▷ ▷ -define v -vector -format 6 -field F \
18855 ▷ ▷ ▷ -compact $(CODE_GV_N15_K6) \
18856 ▷ ▷ -end \
18857 ▷ ▷ -with F -do -finite_fieldactivity \
18858 ▷ ▷ ▷ -RREF v \
18859 ▷ ▷ -end
18860 ▷ #$(PDFLATEX) RREF_example_q2_6_15.tex
18861 ▷ #$(OPEN) RREF_example_q2_6_15.pdf
18862
18863 code_n15_k6_d6_check_RREF:
18864 ▷ $(ORBITER) -v 2 \
18865 ▷ ▷ -define F -finite_field -q 2 -end \
18866 ▷ ▷ -define v -vector -format 9 -field F \
18867 ▷ ▷ ▷ -compact $(CODE_GV_N15_K6_CHECK) \
18868 ▷ ▷ -end \
18869 ▷ ▷ -with F -do -finite_fieldactivity \
18870 ▷ ▷ -RREF v \
18871 ▷ ▷ -end
18872 ▷ #$(PDFLATEX) RREF_example_q2_9_15.tex
18873 ▷ #$(OPEN) RREF_example_q2_9_15.pdf
18874
18875
18876
18877
18878
18879 #####
18880 # Section 11.10: Coding Theory - Classification
18881
18882 SECTION.CODING.THEORY.CLASSIFICATION:
18883
18884 test_11_10:
18885 ▷ make codes_8_4_4
18886 ▷ #make codes_8_4_4.draw
18887 ▷ #make codes_14_4_9_3
18888 ▷ make codes_15_6_6_2
18889 ▷ make codes_d4
18890 ▷ make codes_24_12_8
18891 ▷ #make codes_24_12_8.draw
18892 ▷ make glynn_arc
18893 ▷ make five_points_in_general
18894 ▷ make codes_q13_12_4
18895
18896
18897 # code classification:
18898
18899 codes_8_4_4:
18900 ▷ $(ORBITER) -v 6 \

```

```

18901 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
18902 ▷ ▷ -define Do -draw_options \
18903 ▷ ▷ ▷ -embedded -radius 250 \
18904 ▷ ▷ ▷ -line_width 1.0 -spanning_tree \
18905 ▷ ▷ -end \
18906 ▷ ▷ -define Control -poset_classification_control \
18907 ▷ ▷ ▷ -problem_label codes_8_4_4 \
18908 ▷ ▷ ▷ -draw_options Do \
18909 ▷ ▷ ▷ -W \
18910 ▷ ▷ -end \
18911 ▷ ▷ -define G \
18912 ▷ ▷ -linear_group -PGL 4 2 -end \
18913 ▷ ▷ -with G -do \
18914 ▷ ▷ -group_theoretic_activity \
18915 ▷ ▷ ▷ -linear_codes Control 3 8 \
18916 ▷ ▷ -end
18917 ▷ #$(PDFLATEX) codes_8_4_4_poset_lvl_8.tex
18918 ▷ #$(OPEN) codes_8_4_4_poset_lvl_8.pdf
18919 ▷ #$(PDFLATEX) codes_8_4_4_poset.tex
18920 ▷ #$(OPEN) codes_8_4_4_poset.pdf
18921
18922 # There are six codes, one of them has a group of order 1344
18923
18924 #CODE_8_4_4_1344="0,1,2,3,8,11,13,14"
18925
18926
18927
18928 codes_8_4_4_draw:
18929 ▷ $(ORBITER) -v 3 \
18930 ▷ ▷ -draw_layered_graph \
18931 ▷ ▷ ▷ codes_8_4_4_poset_lvl_8_layered_graph \
18932 ▷ ▷ ▷ -place \
18933 ▷ ▷ ▷ -radius 250 -embedded -line_width 1.0 \
18934 ▷ ▷ ▷ -y_stretch 1.0 -scale 0.5 \
18935 ▷ ▷ -end
18936 ▷ $(PDFLATEX) codes_8_4_4_poset_lvl_8_draw.tex
18937 ▷ $(OPEN) codes_8_4_4_poset_lvl_8_draw.pdf
18938
18939
18940
18941 codes_14_4_9_3:
18942 ▷ $(ORBITER) -v 6 \
18943 ▷ ▷ -define Do -draw_options \
18944 ▷ ▷ ▷ -embedded -radius 250 \
18945 ▷ ▷ -end \
18946 ▷ ▷ -define Control -poset_classification_control \
18947 ▷ ▷ ▷ -problem_label codes_14_4_9_3 \
18948 ▷ ▷ ▷ -draw_options Do \
18949 ▷ ▷ -end \
18950 ▷ ▷ -define G \
18951 ▷ ▷ -linear_group -PGL 10 3 -end \
18952 ▷ ▷ -with G -do \
18953 ▷ ▷ -group_theoretic_activity \
18954 ▷ ▷ ▷ -linear_codes Control 9 14 \
18955 ▷ ▷ -end
18956 ▷ #$(PDFLATEX) codes_14_4_9_3_poset_lvl_13.tex
18957 ▷ #$(OPEN) codes_14_4_9_3_poset_lvl_13.pdf
18958
18959 #very slow

```

```

18960
18961
18962 # ToDo: report is not generated
18963
18964 codes_15_6_6_2:
18965 ▷ $(ORBITER) -v 6 \
18966 ▷ ▷ -define Do -draw_options \
18967 ▷ ▷ ▷ -embedded -radius 250 \
18968 ▷ ▷ -end \
18969 ▷ ▷ -define Control -poset_classification_control \
18970 ▷ ▷ ▷ -problem_label codes_15_6_6_2 \
18971 ▷ ▷ ▷ -draw_options Do \
18972 ▷ ▷ -end \
18973 ▷ ▷ -define G \
18974 ▷ ▷ -linear_group -PGL 9 2 -end \
18975 ▷ ▷ -with G -do \
18976 ▷ ▷ -group_theoretic_activity \
18977 ▷ ▷ ▷ -linear_codes Control 6 15 \
18978 ▷ ▷ -end
18979 ▷ #$(PDFLATEX) codes_15_6_6_2_poset_lvl15.tex
18980 ▷ #$(OPEN) codes_15_6_6_2_poset_lvl15.pdf
18981
18982
18983
18984
18985
18986 codes_d4:
18987 ▷ $(ORBITER) -v 3 \
18988 ▷ ▷ -define Do -draw_options -embedded -end \
18989 ▷ ▷ -define Control -poset_classification_control \
18990 ▷ ▷ ▷ -W \
18991 ▷ ▷ ▷ -problem_label codes_r4_d4 \
18992 ▷ ▷ ▷ -draw_options Do \
18993 ▷ ▷ -end \
18994 ▷ ▷ -define G -linear_group -PGL 4 2 -end \
18995 ▷ ▷ -with G -do \
18996 ▷ ▷ -group_theoretic_activity \
18997 ▷ ▷ ▷ -linear_codes Control 4 100 \
18998 ▷ ▷ -end
18999
19000
19001 codes_24_12_8:
19002 ▷ $(ORBITER) -v 6 \
19003 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
19004 ▷ ▷ -define Do -draw_options -embedded -radius 250 \
19005 ▷ ▷ ▷ -line_width 1.0 -spanning_tree \
19006 ▷ ▷ -end \
19007 ▷ ▷ -define Control -poset_classification_control \
19008 ▷ ▷ ▷ -problem_label codes_24_12_8 \
19009 ▷ ▷ ▷ -W \
19010 ▷ ▷ ▷ -draw_options Do \
19011 ▷ ▷ -end \
19012 ▷ ▷ -define G \
19013 ▷ ▷ -linear_group -PGL 12 2 -end \
19014 ▷ ▷ -with G -do \
19015 ▷ ▷ -group_theoretic_activity \
19016 ▷ ▷ ▷ -linear_codes Control 8 24 \
19017 ▷ ▷ -end
19018 ▷ #$(PDFLATEX) codes_24_12_8_poset.tex

```

```

19019 ▷ #$(OPEN) codes_24_12_8_poset.pdf
19020
19021 #codes_24_12_8_poset_lvl_24_layered_graph
19022
19023 codes_24_12_8_draw:
19024 ▷ $(ORBITER) -v 3 \
19025 ▷ ▷ -draw_layered_graph \
19026 ▷ ▷ ▷ codes_24_12_8_poset_lvl_24_layered_graph \
19027 ▷ ▷ ▷ -place \
19028 ▷ ▷ ▷ -radius 100 -spanning_tree -embedded \
19029 ▷ ▷ ▷ -line_width 0.5 -x_stretch 1.4 \
19030 ▷ ▷ ▷ -scale 0.25 -nodes_empty \
19031 ▷ ▷ -end
19032 ▷ $(PDFLATEX) codes_24_12_8_poset_lvl_24_draw.tex
19033 ▷ $(OPEN) codes_24_12_8_poset_lvl_24_draw.pdf
19034
19035 glynn_arc:
19036 ▷ $(ORBITER) -v 5 \
19037 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
19038 ▷ ▷ -define Do -draw_options -embedded -radius 250 \
19039 ▷ ▷ ▷ -line_width 1.0 -spanning_tree \
19040 ▷ ▷ -end \
19041 ▷ ▷ -define Control -poset_classification_control \
19042 ▷ ▷ ▷ -problem_label glynn_arc \
19043 ▷ ▷ ▷ -draw_options Do \
19044 ▷ ▷ -end \
19045 ▷ ▷ -define G \
19046 ▷ ▷ -linear_group -PGL 5 9 -end \
19047 ▷ ▷ -with G -do \
19048 ▷ ▷ -group_theoretic_activity \
19049 ▷ ▷ ▷ -linear_codes Control 6 10 \
19050 ▷ ▷ -end
19051 ▷ #$(PDFLATEX) glynn_arc_poset.tex
19052 ▷ #$(OPEN) glynn_arc_poset.pdf
19053
19054
19055
19056
19057
19058 five_points_in_general:
19059 ▷ $(ORBITER) -v 5 \
19060 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
19061 ▷ ▷ -define Do -draw_options \
19062 ▷ ▷ ▷ -embedded -radius 250 \
19063 ▷ ▷ ▷ -line_width 1.0 -spanning_tree \
19064 ▷ ▷ -end \
19065 ▷ ▷ -define Control -poset_classification_control \
19066 ▷ ▷ ▷ -problem_label five_points_in_general \
19067 ▷ ▷ ▷ -draw_options Do \
19068 ▷ ▷ -end \
19069 ▷ ▷ -define G \
19070 ▷ ▷ -linear_group -PGL 4 2 -end \
19071 ▷ ▷ -with G -do \
19072 ▷ ▷ -group_theoretic_activity \
19073 ▷ ▷ -linear_codes Control 4 5 \
19074 ▷ ▷ -end
19075 ▷ #$(PDFLATEX) five_points_in_general_poset.tex
19076 ▷ #$(OPEN) five_points_in_general_poset.pdf
19077

```

```

19078
19079
19080
19081 codes_q13_12_4:
19082 ▷ $(ORBITER) -v 6 \
19083 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
19084 ▷ ▷ -define Do -draw_options \
19085 ▷ ▷ ▷ -embedded -radius 250 \
19086 ▷ ▷ ▷ -line_width 1.0 -spanning_tree \
19087 ▷ ▷ -end \
19088 ▷ ▷ -define Control -poset_classification_control \
19089 ▷ ▷ ▷ -problem_label codes_q13 \
19090 ▷ ▷ ▷ -draw_options Do \
19091 ▷ ▷ -end \
19092 ▷ ▷ -define G \
19093 ▷ ▷ -linear_group -PGL 4 13 -end \
19094 ▷ ▷ -with G -do \
19095 ▷ ▷ -group_theoretic_activity \
19096 ▷ ▷ -linear_codes Control 6 12 \
19097 ▷ ▷ -end
19098 ▷ #$(PDFLATEX) codes_q13_poset.tex
19099 ▷ #$(OPEN) codes_q13_poset.pdf
19100
19101
19102
19103
19104 #####
19105 # Chapter 12 - Combinatorics
19106 #####
19107
19108
19109
19110 test_12:
19111 ▷ make test_12_1
19112 ▷ make test_12_2
19113 ▷ make test_12_3
19114 ▷ make test_12_4
19115 ▷ make test_12_5
19116 ▷ make test_12_6
19117 ▷ make test_12_7
19118
19119
19120
19121 #####
19122 # Section 12.1: Combinatorics
19123
19124
19125 SECTION_COMBINATORICS:
19126 ▷
19127
19128
19129 test_12_1:
19130 ▷ make random_permutation_12
19131 ▷ make Sym_4_conj_classes
19132 ▷ make Sym_10_conj_classes
19133 ▷ make Sym_15_conj_classes
19134 ▷ make Char_Sym_4
19135 ▷ make Char_Sym_5
19136 ▷ #make Char_Sym_6

```

```

19137 ▷ make all_subsets_6.3
19138 ▷ make random_k_subsets
19139 ▷ make rank_k_subsets_test
19140 ▷ make Walsh_matrix_4
19141 ▷ make q_binomial_8.3.2
19142 ▷ make Dedekind_10.10
19143 ▷ make Dedekind_30.2
19144 ▷ make Dedekind_100.2
19145 ▷ make elementary_symmetric_functions_4
19146 ▷ make elementary_symmetric_functions_8
19147 ▷ make domino_portrait
19148 ▷ make domino_portrait_input
19149
19150
19151 random_permutation_12:
19152 ▷ $(ORBITER) -v 4 \
19153 ▷ ▷ -seed 31415 \
19154 ▷ ▷ -random_permutation 12 random.csv
19155
19156
19157
19158
19159 Sym_4_conj_classes:
19160 ▷ $(ORBITER) -v 2 -conjugacy_classes_Sym_n 4
19161
19162 Sym_10_conj_classes:
19163 ▷ $(ORBITER) -v 2 -conjugacy_classes_Sym_n 10
19164 ▷ $(OPEN) classes_Sym_10.csv
19165
19166 Sym_15_conj_classes:
19167 ▷ $(ORBITER) -v 2 -conjugacy_classes_Sym_n 15
19168
19169 Char_Sym_4:
19170 ▷ $(ORBITER) -v 2 -character_table_symmetric_group 4
19171
19172 Char_Sym_5:
19173 ▷ $(ORBITER) -v 2 -character_table_symmetric_group 5
19174
19175
19176 # too slow:
19177
19178 Char_Sym_6:
19179 ▷ $(ORBITER) -v 2 -character_table_symmetric_group 6
19180
19181 all_subsets_6.3:
19182 ▷ $(ORBITER) -v 2 -tree_of_all_k_subsets 6 3
19183 ▷ $(ORBITER) -v 20 \
19184 ▷ ▷ -define Do -draw_options -embedded \
19185 ▷ ▷ ▷ -nodes -radius 80 \
19186 ▷ ▷ ▷ -xin 5000 -yin 5000 \
19187 ▷ ▷ ▷ -xout 1000000 -yout 500000 \
19188 ▷ ▷ ▷ -scale 0.5 -line_width 1.0 \
19189 ▷ ▷ -end \
19190 ▷ ▷ -tree_draw \
19191 ▷ ▷ ▷ -file all_k_subsets_n6_k3.tree \
19192 ▷ ▷ ▷ -draw_options Do \
19193 ▷ ▷ -end
19194 ▷ $(PDFLATEX) all_k_subsets_n6_k3_draw.tex
19195 ▷ $(OPEN) all_k_subsets_n6_k3_draw.pdf

```

```

19196
19197 random_k_subsets:
19198 ▷ $(ORBITER) -v 4 \
19199 ▷ ▷ -create_random_k_subsets 10 5 20
19200
19201
19202 rank_k_subsets_test:
19203 ▷ $(ORBITER) -v 2 \
19204 ▷ ▷ -define Blocks -vector -format 7 \
19205 ▷ ▷ ▷ -dense "0,1,2,0,3,4,1,3,5,2,4,5,3,6,7,1,6,8,0,6,9" \
19206 ▷ ▷ ▷ -end \
19207 ▷ ▷ -rank_k_subset 10 3 Blocks
19208
19209
19210 Walsh_matrix_4:
19211 ▷ $(ORBITER) -v 3 \
19212 ▷ ▷ -define F -finite_field -q 2 -end \
19213 ▷ ▷ -with F -do -finite_field_activity \
19214 ▷ ▷ ▷ -Walsh_matrix 4 -end
19215 ▷ $(ORBITER) -v 2 -draw_matrix \
19216 ▷ ▷ -input_csv_file Walsh_01_4.csv \
19217 ▷ ▷ -box_width 10 -bit_depth 24 -partition 3 16 16 -end
19218 ▷ #$(PDFLATEX) GF_2.tex
19219 ▷ #$(OPEN) GF_2.pdf
19220
19221 q_binomial_8_3_2:
19222 ▷ $(ORBITER) -v 3 \
19223 ▷ ▷ -q_binomial 8 3 2
19224
19225
19226 Dedekind_10_10:
19227 ▷ $(ORBITER) -v 3 -Dedekind_numbers 2 10 2 10
19228 ▷ $(ORBITER) -v 3 -csv_file_latex 1 Dedekind_2_10_2_10.csv
19229 ▷ $(PDFLATEX) Dedekind_2_10_2_10.tex
19230 ▷ $(OPEN) Dedekind_2_10_2_10.pdf
19231
19232
19233 Dedekind_30_2:
19234 ▷ $(ORBITER) -v 3 -Dedekind_numbers 2 30 2 2
19235
19236
19237 Dedekind_100_2:
19238 ▷ $(ORBITER) -v 3 -Dedekind_numbers 2 100 2 2
19239
19240
19241
19242 elementary_symmetric_functions_4:
19243 ▷ $(ORBITER) -make_elementary_symmetric_functions 4 4
19244
19245 elementary_symmetric_functions_8:
19246 ▷ $(ORBITER) -make_elementary_symmetric_functions 8 8
19247
19248
19249
19250 domino_portrait:
19251 ▷ cp $(ORBITER_PATH)/examples/users_guide/Munch_3.? .
19252 ▷ $(ORBITER) -v 3 -domino_portrait 7 4 Munch_3 -end
19253
19254 domino_portrait_input:

```

```

19255 ▷ cp $(ORBITER_PATH)/examples/users_guide/Munch_3.? .
19256 ▷ $(ORBITER) -v 2 \
19257 ▷ ▷ -define all_one_r -vector -repeat 1 28 -end \
19258 ▷ ▷ -define all_one_c -vector -repeat 1 32 -end \
19259 ▷ ▷ -draw_matrix \
19260 ▷ ▷ ▷ -grayscale \
19261 ▷ ▷ ▷ -invert_colors \
19262 ▷ ▷ ▷ -input_csv_file Munch_3_m.csv \
19263 ▷ ▷ ▷ -box_width 20 -bit_depth 8 \
19264 ▷ ▷ ▷ -partition 3 \
19265 ▷ ▷ ▷ ▷ all_one_c all_one_r \
19266 ▷ ▷ -end
19267 ▷ $(OPEN) Munch_3_m_draw.bmp
19268
19269
19270
19271 #####
19272 # Section 12.2: Combinatorial Objects
19273
19274 SECTION_COMBINATORIAL_OBJECTS:
19275
19276
19277 test_12.2:
19278 ▷ make Hirschfeld_q4_from_set
19279 ▷ make hyperoval_16_create
19280 ▷ make EC_read
19281 ▷ #make PG_3_5_assume_31_read
19282 ▷ #make LS_AG_2_3_read
19283 ▷ make geo_7_3_read
19284 ▷ make geo_20_3_g4_save_canonical_forms
19285 ▷ make geo_20_3_g4_read
19286 ▷ make geo_pasch_read
19287 ▷ make geo_pasch_given
19288 ▷ make neuumaier_graphs_16
19289 ▷ make neuumaier_graphs_16_from_file
19290 ▷ make Neuumaier_graph_25
19291
19292
19293
19294 Hirschfeld_q4_from_set:
19295 ▷ $(ORBITER) -v 4 \
19296 ▷ ▷ -define H -set -here \
19297 ▷ ▷ ▷ $(HIRSCHFELD_SURFACE_Q4_SET_OF_POINTS) \
19298 ▷ ▷ -end \
19299 ▷ ▷ -define C -combinatorial_object \
19300 ▷ ▷ ▷ -label Hirschfeld_surface Hirschfeld\_surface \
19301 ▷ ▷ ▷ -set_of_points H \
19302 ▷ ▷ -end
19303
19304
19305
19306 hyperoval_16_create:
19307 ▷ $(ORBITER) -v 2 \
19308 ▷ ▷ -define C -combinatorial_object \
19309 ▷ ▷ ▷ -label hyperoval_q16 hyperoval\_q16 \
19310 ▷ ▷ ▷ -set_of_points $(HYPEROVAL_16_16320) \
19311 ▷ ▷ ▷ -set_of_points $(HYPEROVAL_16_144) \
19312 ▷ ▷ -end \
19313

```



```

19314
19315 EC_read:
19316 ▷ $(ORBITER) -v 2 \
19317 ▷ ▷ -define F -finite_field -q 11 -end \
19318 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
19319 ▷ ▷ -define Curve -geometric_object P \
19320 ▷ ▷ ▷ -elliptic_curve 1 3 \
19321 ▷ ▷ -end \
19322 ▷ ▷ -with Curve -do \
19323 ▷ ▷ ▷ -combinatorial_object_activity -save \
19324 ▷ ▷ -end
19325 ▷ $(ORBITER) -v 4 \
19326 ▷ ▷ -define C -combinatorial_object \
19327 ▷ ▷ ▷ -label EC EC \
19328 ▷ ▷ ▷ -file_of_points elliptic_curve_b1_c3_q11.txt \
19329 ▷ ▷ -end
19330
19331
19332
19333 PG_3_5_assume_31_read:
19334 ▷ $(ORBITER) -v 2 \
19335 ▷ ▷ -define C -combinatorial_object \
19336 ▷ ▷ ▷ -label packings_25 packings_25 \
19337 ▷ ▷ ▷ -file_of_packings_through_spread_table \
19338 ▷ ▷ ▷ ▷ H31_packings.csv \
19339 ▷ ▷ ▷ ▷ SPREAD_TABLES_5_REG/spread_25_spreads.csv \
19340 ▷ ▷ ▷ ▷ SpreadRep \
19341 ▷ ▷ ▷ ▷ 5 \
19342 ▷ ▷ -end
19343
19344
19345
19346 LS_AG_2_3_read:
19347 ▷ $(ORBITER) -v 2 \
19348 ▷ ▷ -define C -combinatorial_object \
19349 ▷ ▷ ▷ -label designs designs \
19350 ▷ ▷ ▷ -file_of_designs \
19351 ▷ ▷ ▷ solutions.csv 9 84 3 12 \
19352 ▷ ▷ -end
19353
19354
19355
19356 geo_7_3_read:
19357 ▷ $(ORBITER) -v 10 \
19358 ▷ ▷ -define DoInc -draw_incidence_structure_options \
19359 ▷ ▷ ▷ -width 60 -width_10 6 -end \
19360 ▷ ▷ -define C -combinatorial_object \
19361 ▷ ▷ ▷ -label geo_7_3 geo_7_3 \
19362 ▷ ▷ ▷ -file_of_incidence_geometries \
19363 ▷ ▷ ▷ ▷ 7_3.inc 7 7 21 \
19364 ▷ ▷ -end
19365
19366
19367
19368 geo_20_3_g4_save_canonical_forms:
19369 ▷ $(ORBITER) -v 2 \
19370 ▷ ▷ -define Test_lines -set -loop 1 21 1 -end \
19371 ▷ ▷ -define Geo -geometry_builder \
19372 ▷ ▷ ▷ -V 20 -B 20 -TDO 3 \

```

```

19373 ▷ ▷ ▷ -fuse 1 \
19374 ▷ ▷ ▷ -fname.GEO 20_3_g4 \
19375 ▷ ▷ ▷ -output_to_inc_file \
19376 ▷ ▷ ▷ -girth 4 \
19377 ▷ ▷ ▷ -special_test.not_orderly \
19378 ▷ ▷ ▷ -test Test_lines \
19379 ▷ ▷ ▷ -save_canonical_forms \
19380 ▷ ▷ ▷ -end
19381
19382 #User time: 0:27
19383 #-1 162
19384 #240, 16, 12^3, 10^4, 6^4, 4, 3, 2^49, 1^98
19385
19386 #20_3_g4.inc
19387 #20_3_g4_canonical_forms.csv
19388
19389 geo_20_3_g4_read:
19390 ▷ $(ORBITER) -v 10 \
19391 ▷ ▷ -define DoInc -draw_incidence_structure_options \
19392 ▷ ▷ ▷ -width 60 -width_10 6 -end \
19393 ▷ ▷ -define C -combinatorial_object \
19394 ▷ ▷ ▷ -label geo_20_3_g4 geo\_20\_3\_g4 \
19395 ▷ ▷ ▷ -file_of_incidence_geometries.csv \
19396 ▷ ▷ ▷ ▷ 20_3_g4_canonical_forms.csv Object 20 20 60 \
19397 ▷ ▷ -end
19398
19399
19400
19401
19402 geo_pasch_read:
19403 ▷ $(ORBITER) -v 10 \
19404 ▷ ▷ -define C -combinatorial_object \
19405 ▷ ▷ ▷ -label Pasch Pasch \
19406 ▷ ▷ ▷ -file_of_incidence_geometries \
19407 ▷ ▷ ▷ ▷ pasch.inc 6 4 12 \
19408 ▷ ▷ -end
19409
19410 geo_pasch_given:
19411 ▷ $(ORBITER) -v 10 \
19412 ▷ ▷ -define C -combinatorial_object \
19413 ▷ ▷ ▷ -label Pasch Pasch \
19414 ▷ ▷ ▷ -incidence_geometry \
19415 ▷ ▷ ▷ ▷ "0,1,4,6,8,11,13,14,17,19,22,23" \
19416 ▷ ▷ ▷ ▷ 6 4 12 \
19417 ▷ ▷ -end
19418
19419
19420
19421 neuimaier_graphs_16:
19422 ▷ $(ORBITER) -v 2 \
19423 ▷ ▷ -define C -combinatorial_object \
19424 ▷ ▷ ▷ -label neuimaier_graph neuimaier\_graph \
19425 ▷ ▷ ▷ -graph_by_adjacency_matrix \
19426 ▷ ▷ ▷ $(NEUMAIER_GRAPH_16_1_ADJ) 16 \
19427 ▷ ▷ ▷ -graph_by_adjacency_matrix \
19428 ▷ ▷ ▷ $(NEUMAIER_GRAPH_16_2_ADJ) 16 \
19429 ▷ ▷ -end
19430
19431

```

```

19432 neumaier_graphs_16.from.file:
19433 ▷ $(ORBITER) -v 2 \
19434 ▷ ▷ -define C -combinatorial.object \
19435 ▷ ▷ ▷ -label neumaier_graph neumaier\_graph \
19436 ▷ ▷ ▷ -graph.by.adjacency.matrix.from.file \
19437 ▷ ▷ ▷ NG_16.9.4.2.4.csv ADJ 16 \
19438 ▷ ▷ -end
19439
19440
19441 Neumaier_graph_25:
19442 ▷ $(ORBITER) -v 10 \
19443 ▷ ▷ -define Gamma -graph -Neumaier_graph_25 -end \
19444 ▷ ▷ -with Gamma -do \
19445 ▷ ▷ -graph_theoretic.activity \
19446 ▷ ▷ ▷ -test_SRG.property \
19447 ▷ ▷ -end \
19448 ▷ ▷ -with Gamma -do \
19449 ▷ ▷ -graph_theoretic.activity \
19450 ▷ ▷ ▷ -eigenvalues \
19451 ▷ ▷ -end \
19452 ▷ ▷ -with Gamma -do \
19453 ▷ ▷ -graph_theoretic.activity \
19454 ▷ ▷ ▷ -find_cliques -target.size 5 -end \
19455 ▷ ▷ -end \
19456 ▷ ▷ -with Gamma -do \
19457 ▷ ▷ -graph_theoretic.activity \
19458 ▷ ▷ ▷ -automorphism_group \
19459 ▷ ▷ -end \
19460 ▷ ▷ -with Gamma -do \
19461 ▷ ▷ -graph_theoretic.activity \
19462 ▷ ▷ ▷ -test_Neumaier_property 5 \
19463 ▷ ▷ -end
19464
19465
19466
19467
19468
19469 #####
19470 # Section 12.3: Activities for Geometric Objects
19471
19472
19473 SECTION_ACTIVITIES_FOR_GEOMETRIC_OBJECTS:
19474
19475
19476
19477 test_12.3:
19478 ▷ make Edge_curve_17
19479 ▷ make Edge_curve_17_line_type
19480
19481
19482
19483
19484
19485 Edge_curve_17:
19486 ▷ $(ORBITER) -v 6 \
19487 ▷ ▷ -define F -finite_field -q 17 -end \
19488 ▷ ▷ -define R -polynomial_ring \
19489 ▷ ▷ ▷ -field F \
19490 ▷ ▷ ▷ -number_of_variables 3 \

```

```

19491 ▷ ▷ ▷ -homogeneous_of_degree 4 \
19492 ▷ ▷ ▷ -monomial_ordering_partition \
19493 ▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \
19494 ▷ ▷ -end \
19495 ▷ ▷ -assign Eqn -from \
19496 ▷ ▷ -with R -do \
19497 ▷ ▷ ▷ -ring_theoretic_activity \
19498 ▷ ▷ ▷ -parse.equation \
19499 ▷ ▷ ▷ ▷ "Edge_curve" "Edge\_curve" \
19500 ▷ ▷ ▷ ▷ $(EDGE_CURVE.EQUATION.ALGEBRAIC) \
19501 ▷ ▷ ▷ ▷ f 3 \
19502 ▷ ▷ -end \
19503 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
19504 ▷ ▷ -define C -geometric_object P \
19505 ▷ ▷ ▷ -projective_variety R \
19506 ▷ ▷ ▷ ▷ "Edge_q17" "Edge\_q17" \
19507 ▷ ▷ ▷ ▷ Eqn \
19508 ▷ ▷ -end
19509
19510
19511
19512
19513
19514 Edge_curve.17.line-type:
19515 ▷ $(ORBITER) -v 6 \
19516 ▷ ▷ -define F -finite_field -q 17 -end \
19517 ▷ ▷ -define R -polynomial_ring \
19518 ▷ ▷ ▷ -field F \
19519 ▷ ▷ ▷ -number_of_variables 3 \
19520 ▷ ▷ ▷ -homogeneous_of_degree 4 \
19521 ▷ ▷ ▷ -monomial_ordering_partition \
19522 ▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \
19523 ▷ ▷ -end \
19524 ▷ ▷ -assign Eqn -from \
19525 ▷ ▷ -with R -do \
19526 ▷ ▷ ▷ -ring_theoretic_activity \
19527 ▷ ▷ ▷ -parse.equation \
19528 ▷ ▷ ▷ ▷ "Edge_curve" "Edge\_curve" \
19529 ▷ ▷ ▷ ▷ $(EDGE_CURVE.EQUATION.ALGEBRAIC) \
19530 ▷ ▷ ▷ ▷ f 3 \
19531 ▷ ▷ -end \
19532 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
19533 ▷ ▷ -define C -geometric_object P \
19534 ▷ ▷ ▷ -projective_variety R \
19535 ▷ ▷ ▷ ▷ "Edge_q17" "Edge\_q17" \
19536 ▷ ▷ ▷ ▷ Eqn \
19537 ▷ ▷ -end \
19538 ▷ ▷ -with C -do \
19539 ▷ ▷ -combinatorial_object_activity \
19540 ▷ ▷ ▷ -line_type_old \
19541 ▷ ▷ -end \
19542 ▷ ▷ -print_symbols
19543
19544 #( 4^6, 2^30, 1^132, 0^139 )
19545
19546
19547
19548
19549

```

```

19550
19551
19552
19553 #####
19554 # Section 12.4: Combinatorial Objects with Group
19555
19556 SECTION.COMBINATORIAL_OBJECTS.WITH_GROUP:
19557
19558
19559 test_12_4:
19560 ▷ make geo_10_3.Desargues_configuration_c
19561 ▷ make Praeger_45.12.3.design_group
19562
19563
19564
19565 geo_10_3.Desargues_configuration_c:
19566 ▷ $(ORBITER) -v 10 \
19567 ▷ ▷ -define Doinc -draw_incidence_structure_options \
19568 ▷ ▷ ▷ -width 60 -width_10 6 \
19569 ▷ ▷ -end \
19570 ▷ ▷ -define Test_lines -set -loop 4 11 1 -end \
19571 ▷ ▷ -define Geo -geometry_builder \
19572 ▷ ▷ ▷ -V 10 -B 10 -TD0 3 -fuse 1 \
19573 ▷ ▷ ▷ -fname_GEO 10_3 \
19574 ▷ ▷ ▷ -test Test_lines \
19575 ▷ ▷ ▷ -output_to_inc_file \
19576 ▷ ▷ -end \
19577 ▷ ▷ -define C -combinatorial_object \
19578 ▷ ▷ ▷ -label 10_3 "{\rm 10\\_3}" \
19579 ▷ ▷ ▷ -file_of_incidence_geometries 10_3.inc 10 10 30 \
19580 ▷ ▷ -end \
19581 ▷ ▷ -with C -do \
19582 ▷ ▷ -combinatorial_object_activity \
19583 ▷ ▷ ▷ -canonical_form \
19584 ▷ ▷ ▷ ▷ -save_ago \
19585 ▷ ▷ ▷ ▷ -save_transversal \
19586 ▷ ▷ ▷ ▷ -max_TD0_depth 10 \
19587 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "10_3.nauty_log.txt" -end \
19588 ▷ ▷ ▷ -end \
19589 ▷ ▷ -end \
19590 ▷ ▷ -with C -do \
19591 ▷ ▷ -combinatorial_object_activity \
19592 ▷ ▷ ▷ -post_processing \
19593 ▷ ▷ -end \
19594 ▷ ▷ -assign Object7 -from \
19595 ▷ ▷ -with C -do \
19596 ▷ ▷ -combinatorial_object_activity \
19597 ▷ ▷ ▷ -get_combo_with_group 7 \
19598 ▷ ▷ -end \
19599 ▷ ▷ -with Object7 -do \
19600 ▷ ▷ ▷ -combo_activity \
19601 ▷ ▷ ▷ ▷ -report \
19602 ▷ ▷ ▷ ▷ ▷ -show_TD0 \
19603 ▷ ▷ ▷ ▷ ▷ -show_TDA \
19604 ▷ ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
19605 ▷ ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
19606 ▷ ▷ ▷ ▷ -end \
19607 ▷ ▷ -end \
19608 ▷ ▷ -print_symbols

```

```

19609 ▷ $(PDFLATEX) 10_3.object7_report.tex
19610 ▷ $(OPEN) 10_3.object7_report.pdf
19611
19612
19613
19614 Praeger_45.12.3.design-group:
19615 ▷ $(ORBITER) -v 4 \
19616 ▷ ▷ -define Do -draw.options \
19617 ▷ ▷ ▷ -radius 200 -line.width 0.3 \
19618 ▷ ▷ -end \
19619 ▷ ▷ -define F -finite_field -q 81 -compute.related.fields \
19620 ▷ ▷ ▷ -override.polynomial 89 \
19621 ▷ ▷ -end \
19622 ▷ ▷ -define AGGL -linear_group -AGGL 1 F \
19623 ▷ ▷ ▷ -end \
19624 ▷ ▷ -define G -linear_group -AGGL 1 F \
19625 ▷ ▷ ▷ -subgroup_by_generators G 3240 \
19626 ▷ ▷ ▷ 8 $(PRAEGER_DESIGN_45.12.3.GENS) \
19627 ▷ ▷ ▷ -end \
19628 ▷ ▷ -with G -do \
19629 ▷ ▷ -group_theoretic.activity \
19630 ▷ ▷ ▷ -report Do \
19631 ▷ ▷ -end \
19632 ▷ ▷ -define v -vector \
19633 ▷ ▷ ▷ -dense $(PRAEGER_DESIGN_45.12.3.GENS) \
19634 ▷ ▷ -end \
19635 ▷ ▷ -with G -do \
19636 ▷ ▷ -group_theoretic.activity \
19637 ▷ ▷ ▷ -subgroup_lattice_magma \
19638 ▷ ▷ -end \
19639 ▷ ▷ -define L -modified_group -from G \
19640 ▷ ▷ ▷ -subgroup_by_lattice 43 \
19641 ▷ ▷ -end \
19642 ▷ ▷ -define R -modified_group -from G \
19643 ▷ ▷ ▷ -subgroup_by_lattice 44 \
19644 ▷ ▷ -end \
19645 ▷ ▷ -with L -do \
19646 ▷ ▷ -group_theoretic.activity \
19647 ▷ ▷ ▷ -report Do \
19648 ▷ ▷ -end \
19649 ▷ ▷ -with R -do \
19650 ▷ ▷ -group_theoretic.activity \
19651 ▷ ▷ ▷ -report Do \
19652 ▷ ▷ -end \
19653 ▷ ▷ -with G -do \
19654 ▷ ▷ -group_theoretic.activity \
19655 ▷ ▷ ▷ -create_flag_transitive_geometry L R \
19656 ▷ ▷ -end
19657 ▷ $(ORBITER) -v 3 \
19658 ▷ ▷ -orbiter_path $(ORBITER_PATH) \
19659 ▷ ▷ -change_values \
19660 ▷ ▷ ▷ AGGL_1.81.Subgroup_G.3240_AGGL_1.81.Subgroup_G.3240_subgroup_by_lattice_43_
AGGL_1.81.Subgroup_G.3240_subgroup_by_lattice_44.intersection.size.csv \
19661 ▷ ▷ ▷ Praeger_6.csv \
19662 ▷ ▷ ▷ 3,6,8 \
19663 ▷ ▷ ▷ 0,1,0
19664 ▷ $(ORBITER) -v 8 \
19665 ▷ ▷ -define Praeger_45 -vector \
19666 ▷ ▷ ▷ -file Praeger_6.csv -format 45 \

```

```

19667 ▷ ▷ -end \
19668 ▷ ▷ -define D -design \
19669 ▷ ▷ ▷ -from_incidence_matrix Praeger_45 \
19670 ▷ ▷ -end \
19671 ▷ ▷ -with D -do \
19672 ▷ ▷ ▷ -design_activity \
19673 ▷ ▷ ▷ ▷ -export_flags \
19674 ▷ ▷ -end
19675 ▷ $(ORBITER) -v 3 \
19676 ▷ ▷ -define Doinc -draw_incidence_structure_options \
19677 ▷ ▷ ▷ -width 60 -width_10 6 -end \
19678 ▷ ▷ -define C -combinatorial_object \
19679 ▷ ▷ ▷ -label Praeger Praeger \
19680 ▷ ▷ ▷ -file_of_incidence_geometries \
19681 ▷ ▷ ▷ ▷ Praeger_45_flags.txt 45 45 540 \
19682 ▷ ▷ -end \
19683 ▷ ▷ -with C -do \
19684 ▷ ▷ -combinatorial_object_activity \
19685 ▷ ▷ ▷ -canonical_form \
19686 ▷ ▷ ▷ ▷ -save_ago \
19687 ▷ ▷ ▷ ▷ -save_transversal \
19688 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "praeger_design_nauty_log.txt" -end \
19689 ▷ ▷ ▷ -end \
19690 ▷ ▷ -end \
19691 ▷ ▷ -with C -do \
19692 ▷ ▷ -combinatorial_object_activity \
19693 ▷ ▷ ▷ -post_processing \
19694 ▷ ▷ -end \
19695 ▷ ▷ -assign Object -from \
19696 ▷ ▷ -with C -do \
19697 ▷ ▷ -combinatorial_object_activity \
19698 ▷ ▷ ▷ -get_combo_with_group 0 \
19699 ▷ ▷ -end \
19700 ▷ ▷ -with Object -do \
19701 ▷ ▷ ▷ -combo_activity \
19702 ▷ ▷ ▷ ▷ -report \
19703 ▷ ▷ ▷ ▷ ▷ -export_flag_orbits \
19704 ▷ ▷ ▷ ▷ ▷ -show_incidence_matrices \
19705 ▷ ▷ ▷ ▷ ▷ -show_TDO \
19706 ▷ ▷ ▷ ▷ ▷ -show_TDA \
19707 ▷ ▷ ▷ ▷ ▷ -export_group_GAP \
19708 ▷ ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
19709 ▷ ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
19710 ▷ ▷ ▷ ▷ -end \
19711 ▷ ▷ -end \
19712 ▷ ▷ -assign A -from \
19713 ▷ ▷ -with Object -do \
19714 ▷ ▷ ▷ -combo_activity \
19715 ▷ ▷ ▷ ▷ -get_group \
19716 ▷ ▷ -end \
19717 ▷ ▷ -print_symbols
19718 ▷ #$(PDFLATEX) Praeger_object0_report.tex
19719 ▷ #$(OPEN) Praeger_object0_report.pdf
19720
19721
19722
19723
19724 #####
19725 # Section 12.5: Combinatorial Linear Spaces

```

```

19726
19727 SECTION.COMBINATORIAL_LINEAR_SPACES:
19728
19729
19730 test_12.5:
19731 ▷ make linsp6
19732 ▷ make linsp7
19733 ▷ make linsp30_pt.types
19734 ▷ make linsp30_pt.distribution
19735
19736
19737 linsp6:
19738 ▷ $(ORBITER) -v 4 \
19739 ▷ ▷ -define A -vector -format 1 -dense "15,10,6,3,1" -end \
19740 ▷ ▷ -define D -diophant -label linsp6 \
19741 ▷ ▷ -coefficient_matrix A \
19742 ▷ ▷ -RHS "mult=1,EQ=15" \
19743 ▷ ▷ -x_min_global 0 \
19744 ▷ ▷ -x_max_global 15 \
19745 ▷ ▷ -end \
19746 ▷ ▷ -with D -do \
19747 ▷ ▷ ▷ -diophant_activity -solve_mckay \
19748 ▷ ▷ -end
19749 ▷
19750 # Found 15 solutions with 22 backtrack steps
19751
19752
19753
19754
19755 linsp7:
19756 ▷ $(ORBITER) -v 4 \
19757 ▷ ▷ -define A -vector -format 1 -dense "21,15,10,6,3,1" -end \
19758 ▷ ▷ -define D -diophant -label linsp7 \
19759 ▷ ▷ -coefficient_matrix A \
19760 ▷ ▷ -RHS "mult=1,EQ=21" \
19761 ▷ ▷ -x_min_global 0 \
19762 ▷ ▷ -x_max_global 21 \
19763 ▷ ▷ -end \
19764 ▷ ▷ -with D -do \
19765 ▷ ▷ ▷ -diophant_activity -solve_mckay \
19766 ▷ ▷ -end
19767 ▷
19768
19769 # 32 solutions in 45 backtrack steps
19770
19771
19772
19773
19774
19775
19776
19777 linsp30_pt.types:
19778 ▷ $(ORBITER) -v 4 \
19779 ▷ ▷ -define A -vector -format 1 -dense "6,4,3" -end \
19780 ▷ ▷ -define D -diophant \
19781 ▷ ▷ ▷ -label linsp30_pt.types \
19782 ▷ ▷ ▷ -coefficient_matrix A \
19783 ▷ ▷ ▷ -RHS "mult=1,EQ=29" -x_bounds "0,1,0,27,0,24" \
19784 ▷ ▷ -end \

```



```

19785 ▷ ▷ -with D -do \
19786 ▷ ▷ ▷ -diophant_activity -solve_mckay \
19787 ▷ ▷ -end
19788
19789 linsp30.pt.distribution:
19790 ▷ $(ORBITER) -v 4 \
19791 ▷ ▷ -define A -vector -format 6 -dense \
19792 ▷ ▷ ▷ "1,1,1,1,1,1,0,0,5,2,5,2,1,5,3,7,10,1,10,1,0,10,3,21" \
19793 ▷ ▷ -end \
19794 ▷ ▷ -define D -diophant \
19795 ▷ ▷ ▷ -label linsp30.pt.distribution \
19796 ▷ ▷ ▷ -coefficient_matrix A \
19797 ▷ ▷ ▷ -RHS "mult=1,EQ=30" \
19798 ▷ ▷ ▷ -RHS "mult=1,EQ=7" \
19799 ▷ ▷ ▷ -RHS "mult=1,EQ=135" \
19800 ▷ ▷ ▷ -RHS "mult=1,EQ=96" \
19801 ▷ ▷ ▷ -RHS "mult=1,LE=351" \
19802 ▷ ▷ ▷ -RHS "mult=1,LE=276" \
19803 ▷ ▷ ▷ -x_min_global 0 -x_max_global 30 \
19804 ▷ ▷ -end \
19805 ▷ ▷ -with D -do \
19806 ▷ ▷ ▷ -diophant_activity -solve_mckay \
19807 ▷ ▷ -end \
19808 ▷ ▷ -with D -do \
19809 ▷ ▷ ▷ -diophant_activity -draw_as_bitmap 20 8 \
19810 ▷ ▷ -end
19811
19812
19813
19814 #####
19815 # Section 12.6: Combinatorial Linear Spaces
19816
19817 test_12.6:
19818 ▷ make geo_pasch
19819 ▷ make geo_petersen
19820 ▷ make geo_7.3
19821 ▷ make geo_7.3_no_square_test
19822 ▷ make geo_7.3_no_square_test_draw
19823 ▷ make geo_7.3_orderly
19824 ▷ make geo_7.3_orderly_draw
19825 ▷ make geo_7.3_orderly_mem_debug
19826 ▷ make geo_8.3
19827 ▷ make geo_9.3
19828 ▷ make geo_10.3
19829 ▷ make geo_10.3_inc_draw
19830 ▷ make geo_10.3_orderly
19831 ▷ make geo_10.3_orderly_mem_debug
19832 ▷ make geo_10.3_tree
19833 ▷ make geo_11.3
19834 ▷ make geo_12.3
19835 ▷ make geo_12.3_orderly
19836 ▷ make geo_13.3
19837 ▷ make geo_13.3_orderly
19838 ▷ #make geo_14.3 #slow
19839 ▷ #make geo_14.3_orderly # slow
19840 ▷ #make 15.3.inc
19841 ▷ make geo_15.3.g4
19842 ▷ make geo_17.3.g4_orderly
19843 ▷ make geo_18.3.g4

```

```

19844 ▷ make geo.19.3.g4
19845 ▷ make geo.20.3.g4
19846 ▷ #make geo.21.3.g4 # too slow
19847 ▷ make geo.15.4
19848 ▷ make geo.16.4.g4
19849 ▷ make geo.16
19850 ▷ make geo.40.4.g4
19851 ▷ make geo.63.3.g6
19852 ▷ make geo.LSQ6.inc
19853 ▷ make geo.MOL6.inc
19854
19855
19856 geo.pasch:
19857 ▷ $(ORBITER) -v 8 \
19858 ▷ ▷ -define Test_lines -set -loop 1 7 1 -end \
19859 ▷ ▷ -define Geo -geometry_builder \
19860 ▷ ▷ ▷ -V 6 -B 4 -TDO 2 -fuse 1 \
19861 ▷ ▷ ▷ -fname_GEO pasch \
19862 ▷ ▷ ▷ -output_to_inc_file \
19863 ▷ ▷ ▷ -test Test_lines \
19864 ▷ ▷ ▷ -save_canonical_forms \
19865 ▷ ▷ -end
19866
19867
19868 geo.petersen:
19869 ▷ $(ORBITER) -v 8 \
19870 ▷ ▷ -define Test_lines -set -loop 3 11 1 -end \
19871 ▷ ▷ -define Geo -geometry_builder \
19872 ▷ ▷ ▷ -V 10 -B 15 -TDO 3 -fuse 1 \
19873 ▷ ▷ ▷ -fname_GEO petersen -girth 5 \
19874 ▷ ▷ ▷ -output_to_inc_file \
19875 ▷ ▷ ▷ -search_tree \
19876 ▷ ▷ ▷ -test Test_lines \
19877 ▷ ▷ -end
19878
19879 geo.7.3:
19880 ▷ $(ORBITER) -v 2 \
19881 ▷ ▷ -define Test_lines -set -loop 3 8 1 -end \
19882 ▷ ▷ -define Geo -geometry_builder \
19883 ▷ ▷ ▷ -V 7 -B 7 -TDO 3 \
19884 ▷ ▷ ▷ -fuse 1 \
19885 ▷ ▷ ▷ -fname_GEO 7.3 \
19886 ▷ ▷ ▷ -output_to_inc_file \
19887 ▷ ▷ ▷ -test Test_lines \
19888 ▷ ▷ -end
19889
19890 geo.7.3_no_square_test:
19891 ▷ $(ORBITER) -v 2 \
19892 ▷ ▷ -define Test_lines -set -loop 3 8 1 -end \
19893 ▷ ▷ -define Geo -geometry_builder \
19894 ▷ ▷ ▷ -V 7 -B 7 -TDO 3 \
19895 ▷ ▷ ▷ -fuse 1 \
19896 ▷ ▷ ▷ -fname_GEO 7.3_nst \
19897 ▷ ▷ ▷ -output_to_inc_file \
19898 ▷ ▷ ▷ -test Test_lines \
19899 ▷ ▷ ▷ -no_square_test \
19900 ▷ ▷ -end
19901
19902 geo.7.3_no_square_test.draw:

```

```

19903 ▷ $(ORBITER) -v 10 \
19904 ▷ ▷ -define Doinc -draw_incidence_structure_options \
19905 ▷ ▷ ▷ -width 60 -width_10 6 -end \
19906 ▷ ▷ -define C -combinatorial_object \
19907 ▷ ▷ ▷ -label geo.7.3_nst geo.7.3_nst \
19908 ▷ ▷ ▷ -file_of_incidence_geometries \
19909 ▷ ▷ ▷ ▷ 7.3_nst.inc 7 7 21 \
19910 ▷ ▷ -end \
19911 ▷ ▷ -with C -do \
19912 ▷ ▷ -combinatorial_object_activity \
19913 ▷ ▷ ▷ -draw_incidence_matrices \
19914 ▷ ▷ ▷ 7.3_nst Doinc \
19915 ▷ ▷ -end
19916 ▷ $(PDFLATEX) 7.3_nst_incma.tex
19917 ▷ $(OPEN) 7.3_nst_incma.pdf
19918
19919
19920
19921 geo.7.3_orderly:
19922 ▷ $(ORBITER) -v 200 \
19923 ▷ ▷ -define Test_lines -set -loop 3 8 1 -end \
19924 ▷ ▷ -define Geo -geometry_builder \
19925 ▷ ▷ ▷ -V 7 -B 7 -TD0 3 \
19926 ▷ ▷ ▷ -fuse 1 -fname_GEO 7.3 \
19927 ▷ ▷ ▷ -output_to_inc_file \
19928 ▷ ▷ ▷ -test Test_lines \
19929 ▷ ▷ ▷ -search_tree \
19930 ▷ ▷ ▷ -orderly \
19931 ▷ ▷ -end
19932
19933 geo.7.3_orderly_draw:
19934 ▷ $(ORBITER) -v 20 \
19935 ▷ ▷ -define Do -draw_options -embedded -radius 50 \
19936 ▷ ▷ ▷ -xin 10000 -yin 10000 \
19937 ▷ ▷ ▷ -xout 1000000 -yout 1000000 \
19938 ▷ ▷ ▷ -nodes_empty \
19939 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 \
19940 ▷ ▷ -end \
19941 ▷ ▷ -tree_draw \
19942 ▷ ▷ ▷ -file 7.3_tree.txt \
19943 ▷ ▷ ▷ -draw_options Do \
19944 ▷ ▷ -end
19945 ▷ $(PDFLATEX) 7.3_tree_draw.tex
19946 ▷ $(OPEN) 7.3_tree_draw.pdf
19947
19948
19949
19950 geo.7.3_orderly_mem_debug:
19951 ▷ $(ORBITER) -v 4 \
19952 ▷ ▷ -memory_debug 1 \
19953 ▷ ▷ -define Test_lines -set -loop 3 7 1 -end \
19954 ▷ ▷ -define Geo -geometry_builder \
19955 ▷ ▷ ▷ -V 7 -B 7 -TD0 3 \
19956 ▷ ▷ ▷ -fuse 1 -fname_GEO 7.3 \
19957 ▷ ▷ ▷ -output_to_inc_file \
19958 ▷ ▷ ▷ -test Test_lines \
19959 ▷ ▷ ▷ -search_tree \
19960 ▷ ▷ ▷ -orderly \
19961 ▷ ▷ -end

```

```

19962
19963
19964
19965
19966 geo.8.3:
19967 ▷ $(ORBITER) -v 2 \
19968 ▷ ▷ -define Test_lines -set -loop 3 9 1 -end \
19969 ▷ ▷ -define Geo -geometry_builder \
19970 ▷ ▷ ▷ -V 8 -B 8 -TDO 3 \
19971 ▷ ▷ ▷ -fuse 1 -fname_GEO 8.3 \
19972 ▷ ▷ ▷ -output_to_inc_file \
19973 ▷ ▷ ▷ -test Test_lines \
19974 ▷ ▷ -end
19975
19976 #-print_at_line 4
19977 # 1 geo: 0 11 18 29 30 38 44 54
19978 # ago=48
19979
19980
19981
19982
19983
19984 geo.9.3:
19985 ▷ $(ORBITER) -v 2 \
19986 ▷ ▷ -define Test_lines -set -loop 3 10 1 -end \
19987 ▷ ▷ -define Geo -geometry_builder \
19988 ▷ ▷ ▷ -V 9 -B 9 -TDO 3 \
19989 ▷ ▷ ▷ -fuse 1 -fname_GEO 9.3 \
19990 ▷ ▷ ▷ -output_to_inc_file \
19991 ▷ ▷ ▷ -test Test_lines \
19992 ▷ ▷ -end
19993
19994
19995 geo.10.3:
19996 ▷ $(ORBITER) -v 2 \
19997 ▷ ▷ -define Test_lines -set -loop 4 11 1 -end \
19998 ▷ ▷ -define Geo -geometry_builder \
19999 ▷ ▷ ▷ -V 10 -B 10 -TDO 3 -fuse 1 \
20000 ▷ ▷ ▷ -fname_GEO 10.3 \
20001 ▷ ▷ ▷ -output_to_inc_file \
20002 ▷ ▷ ▷ -output_to_sage_file \
20003 ▷ ▷ ▷ -output_to_blocks_file \
20004 ▷ ▷ ▷ -test Test_lines \
20005 ▷ ▷ ▷ -save_canonical_forms \
20006 ▷ ▷ -end
20007
20008
20009
20010 # 10 geos
20011 # 8/26/2021: 0 sec on Mac
20012
20013
20014 geo.10.3.inc.draw:
20015 ▷ $(ORBITER) -v 10 \
20016 ▷ ▷ -define Doinc -draw_incidence_structure_options \
20017 ▷ ▷ ▷ -width 60 -width_10 6 -end \
20018 ▷ ▷ -define C -combinatorial_object \
20019 ▷ ▷ ▷ -label geo.10.3 geo\10\3 \
20020 ▷ ▷ ▷ -file_of_incidence_geometries \

```

```

20021 ▷ ▷ ▷ 10.3.inc 10 10 30 \
20022 ▷ ▷ -end \
20023 ▷ ▷ -with C -do \
20024 ▷ ▷ -combinatorial_object_activity \
20025 ▷ ▷ ▷ -draw_incidence_matrices \
20026 ▷ ▷ ▷ 10.3.inc Doinc \
20027 ▷ ▷ -end
20028 ▷ $(PDFLATEX) 10.3.inc_incma.tex
20029 ▷ $(OPEN) 10.3.inc_incma.pdf
20030
20031
20032 geo.10.3.orderly:
20033 ▷ $(ORBITER) -v 20 \
20034 ▷ ▷ -define Test_lines -set -loop 4 11 1 -end \
20035 ▷ ▷ -define Geo -geometry_builder \
20036 ▷ ▷ ▷ -V 10 -B 10 -TD0 3 -fuse 1 \
20037 ▷ ▷ ▷ -fname_GEO 10.3 \
20038 ▷ ▷ ▷ -output_to_inc_file \
20039 ▷ ▷ ▷ -test Test_lines \
20040 ▷ ▷ ▷ -orderly \
20041 ▷ ▷ -end
20042
20043 geo.10.3.orderly.mem.debug:
20044 ▷ $(ORBITER) -v 2 \
20045 ▷ ▷ -memory_debug 1 \
20046 ▷ ▷ -define Test_lines -set -loop 4 10 1 -end \
20047 ▷ ▷ -define Geo -geometry_builder \
20048 ▷ ▷ ▷ -V 10 -B 10 -TD0 3 -fuse 1 \
20049 ▷ ▷ ▷ -fname_GEO 10.3 \
20050 ▷ ▷ ▷ -output_to_inc_file \
20051 ▷ ▷ ▷ -test Test_lines \
20052 ▷ ▷ ▷ -orderly \
20053 ▷ ▷ -end
20054
20055
20056 geo.10.3.tree:
20057 ▷ $(ORBITER) -v 20 \
20058 ▷ ▷ -define Test_lines -set -loop 0 11 1 -end \
20059 ▷ ▷ -define GEO -geometry_builder \
20060 ▷ ▷ ▷ -V 10 -B 10 -TD0 3 -fuse 1 \
20061 ▷ ▷ ▷ -fname_GEO 10.3 \
20062 ▷ ▷ ▷ -output_to_inc_file \
20063 ▷ ▷ ▷ -search_tree \
20064 ▷ ▷ ▷ -test Test_lines \
20065 ▷ ▷ -end
20066 ▷ $(ORBITER) -v 20 \
20067 ▷ ▷ -define Do -draw_options -embedded -radius 40 \
20068 ▷ ▷ ▷ -paperheight 220 \
20069 ▷ ▷ ▷ -paperwidth 330 \
20070 ▷ ▷ ▷ -xin 10000 -yin 10000 \
20071 ▷ ▷ ▷ -xout 1000000 -yout 500000 \
20072 ▷ ▷ ▷ -scale 2 -line_width 0.3 \
20073 ▷ ▷ ▷ -nodes_empty \
20074 ▷ ▷ -end \
20075 ▷ ▷ -tree_draw \
20076 ▷ ▷ ▷ -file 10.3.tree.txt \
20077 ▷ ▷ ▷ -draw_options Do \
20078 ▷ ▷ -end
20079 ▷ $(PDFLATEX) 10.3.tree_draw.tex

```

```

20080 ▷ $(OPEN) 10_3_tree.draw.pdf
20081
20082
20083
20084
20085
20086
20087 geo_11_3:
20088 ▷ $(ORBITER) -v 2 \
20089 ▷ ▷ -define Test_lines -set -loop 4 12 1 -end \
20090 ▷ ▷ -define Geo -geometry_builder \
20091 ▷ ▷ ▷ -V 11 -B 11 -TDO 3 \
20092 ▷ ▷ ▷ -fuse 1 -fname.GEO 11_3 \
20093 ▷ ▷ ▷ -output_to_inc_file \
20094 ▷ ▷ ▷ -test Test_lines \
20095 ▷ ▷ -end
20096
20097 # 31 geos
20098 # 8/26/2021: 0 sec on Mac
20099
20100 geo_12_3:
20101 ▷ $(ORBITER) -v 2 \
20102 ▷ ▷ -define Test_lines -set -loop 4 13 1 -end \
20103 ▷ ▷ -define Geo -geometry_builder \
20104 ▷ ▷ ▷ -V 12 -B 12 -TDO 3 \
20105 ▷ ▷ ▷ -fuse 1 -fname.GEO 12_3 \
20106 ▷ ▷ ▷ -output_to_inc_file \
20107 ▷ ▷ ▷ -test Test_lines \
20108 ▷ ▷ ▷ -output_to_sage_file \
20109 ▷ ▷ -end
20110
20111 # 229 geos
20112 #User time: 0.45 of a second, dt=45 tps = 100
20113 #nb.calls.to.densenaudy=24586
20114
20115
20116 geo_12_3_orderly:
20117 ▷ $(ORBITER) -v 2 \
20118 ▷ ▷ -define Test_lines -set -loop 4 13 1 -end \
20119 ▷ ▷ -define Geo -geometry_builder \
20120 ▷ ▷ ▷ -V 12 -B 12 -TDO 3 \
20121 ▷ ▷ ▷ -fuse 1 -fname.GEO 12_3 \
20122 ▷ ▷ ▷ -output_to_inc_file \
20123 ▷ ▷ ▷ -test Test_lines \
20124 ▷ ▷ ▷ -orderly \
20125 ▷ ▷ -end
20126
20127
20128
20129 geo_13_3:
20130 ▷ $(ORBITER) -v 2 \
20131 ▷ ▷ -define Test_lines -set -loop 4 14 1 -end \
20132 ▷ ▷ -define Geo -geometry_builder \
20133 ▷ ▷ ▷ -V 13 -B 13 -TDO 3 \
20134 ▷ ▷ ▷ -fuse 1 -fname.GEO 13_3 \
20135 ▷ ▷ ▷ -output_to_inc_file \
20136 ▷ ▷ ▷ -test Test_lines \
20137 ▷ ▷ ▷ -save_canonical_forms \
20138 ▷ ▷ -end

```

```
20139
20140 # 2036 geos, 96, 39, 13, 12^4, 8^3, 6^16, 4^30, 3^20, 2^190, 1^1770
20141 #User time: 0:4
20142 #nb.calls_to_densenaute=216777
20143
20144 geo.13.3.orderly:
20145 ▷ $(ORBITER) -v 2 \
20146 ▷ ▷ -define Test_lines -set -loop 4 14 1 -end \
20147 ▷ ▷ -define Geo -geometry_builder \
20148 ▷ ▷ ▷ -V 13 -B 13 -TDO 3 \
20149 ▷ ▷ ▷ -fuse 1 -fname_GEO 13.3 \
20150 ▷ ▷ ▷ -output_to_inc_file \
20151 ▷ ▷ ▷ -test Test_lines \
20152 ▷ ▷ ▷ -orderly \
20153 ▷ ▷ -end
20154
20155
20156
20157 geo.14.3:
20158 ▷ $(ORBITER) -v 2 \
20159 ▷ ▷ -define Test_lines -set -loop 4 15 1 -end \
20160 ▷ ▷ -define Geo -geometry_builder \
20161 ▷ ▷ ▷ -V 14 -B 14 -TDO 3 \
20162 ▷ ▷ ▷ -fuse 1 -fname_GEO 14.3 \
20163 ▷ ▷ ▷ -output_to_inc_file \
20164 ▷ ▷ ▷ -test Test_lines \
20165 ▷ ▷ -end
20166
20167 # 21399 geos, 56448, 128, 24^2, 16^3, 14^3, 12^7, 8^15, 7, 6^12, 4^91, 3^19, 2^91
    6, 1^20328
20168 #User time: 0:55
20169 #nb.calls_to_densenaute=2089344
20170
20171
20172 geo.14.3.orderly:
20173 ▷ $(ORBITER) -v 2 \
20174 ▷ ▷ -define Test_lines -set -loop 4 15 1 -end \
20175 ▷ ▷ -define Geo -geometry_builder \
20176 ▷ ▷ ▷ -V 14 -B 14 -TDO 3 \
20177 ▷ ▷ ▷ -fuse 1 -fname_GEO 14.3 \
20178 ▷ ▷ ▷ -output_to_inc_file \
20179 ▷ ▷ ▷ -test Test_lines \
20180 ▷ ▷ ▷ -orderly \
20181 ▷ ▷ -end
20182
20183 #User time: 0:50
20184
20185
20186 15.3.inc:
20187 ▷ $(ORBITER) -v 2 \
20188 ▷ ▷ -define Test_lines -set -loop 4 16 1 -end \
20189 ▷ ▷ -define Geo -geometry_builder \
20190 ▷ ▷ ▷ -V 15 -B 15 -TDO 3 \
20191 ▷ ▷ ▷ -fuse 1 -fname_GEO 15.3 \
20192 ▷ ▷ ▷ -output_to_inc_file \
20193 ▷ ▷ ▷ -test Test_lines \
20194 ▷ ▷ ▷ -special_test_not_orderly \
20195 ▷ ▷ -end
20196
```

```

20197 #15.3.inc
20198 # 245342 geos, 8064, 720, 192^2, 128, 72, 48^6, 32, 30^2, 24^2, 20^2, 18, 16^10,
      15^2, 12^11, 10^3, 8^34, 6^59, 5^5, 4^180, 3^69, 2^3709, 1^241240
20199 # 4 min 10 sec on Mac
20200 #-1 245342
20201 #8064, 720, 192^2, 128, 72, 48^6, 32, 30^2, 24^2, 20^2, 18, 16^10, 15^2, 12^11, 1
      0^3, 8^34, 6^59, 5^5, 4^180, 3^69, 2^3709, 1^241240
20202
20203
20204
20205
20206 geo.15.3.g4:
20207 ▷ $(ORBITER) -v 2 \
20208 ▷ ▷ -define Test_lines -set -loop 4 16 1 -end \
20209 ▷ ▷ -define Geo -geometry_builder \
20210 ▷ ▷ ▷ -V 15 -B 15 -TDO 3 \
20211 ▷ ▷ ▷ -fuse 1 -fname_GEO 15.3.g4 \
20212 ▷ ▷ ▷ -output_to_inc_file \
20213 ▷ ▷ ▷ -girth 4 \
20214 ▷ ▷ ▷ -search_tree \
20215 ▷ ▷ ▷ -special_test_not_orderly \
20216 ▷ ▷ ▷ -test Test_lines \
20217 ▷ ▷ -end
20218 ▷ $(ORBITER) -v 2 \
20219 ▷ ▷ -define Do -draw_options -embedded -radius 50 \
20220 ▷ ▷ ▷ -nodes_empty \
20221 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
20222 ▷ ▷ -tree_draw \
20223 ▷ ▷ ▷ -file 15.3.g4.tree.txt \
20224 ▷ ▷ ▷ -draw_options Do \
20225 ▷ ▷ -end
20226 ▷ $(PDFLATEX) 15.3.g4.tree_draw.tex
20227 ▷ $(OPEN) 15.3.g4.tree_draw.pdf
20228
20229
20230 # The unique Cremona Richmond configuration with group of order 720
20231 # cubic surface with 15 real lines and 15 real tritangent planes.
20232 #User time: 0 of a second, dt=0 tps = 100
20233 #nb.calls.to.densenauty=23
20234
20235 #-sideways
20236
20237
20238
20239
20240 geo.17.3.g4.orderly:
20241 ▷ $(ORBITER) -v 2 \
20242 ▷ ▷ -define Test_lines -set -loop 4 18 1 -end \
20243 ▷ ▷ -define Geo -geometry_builder \
20244 ▷ ▷ ▷ -V 17 -B 17 -TDO 3 \
20245 ▷ ▷ ▷ -fuse 1 -fname_GEO 17.3.g4 \
20246 ▷ ▷ ▷ -output_to_inc_file \
20247 ▷ ▷ ▷ -girth 4 \
20248 ▷ ▷ ▷ -test Test_lines \
20249 ▷ ▷ ▷ -orderly \
20250 ▷ ▷ -end
20251
20252 # 1 sol
20253

```



```

20254 geo.18.3.g4:
20255 ▷ $(ORBITER) -v 2 \
20256 ▷ ▷ -define Test_lines -set -loop 4 19 1 -end \
20257 ▷ ▷ -define Geo -geometry_builder \
20258 ▷ ▷ ▷ -V 18 -B 18 -TDO 3 \
20259 ▷ ▷ ▷ -fuse 1 -fname.GEO 18.3.g4 \
20260 ▷ ▷ ▷ -output_to_inc_file \
20261 ▷ ▷ ▷ -girth 4 \
20262 ▷ ▷ ▷ -search_tree \
20263 ▷ ▷ ▷ -special_test_not_orderly \
20264 ▷ ▷ ▷ -test Test_lines \
20265 ▷ ▷ -end
20266
20267 # 4 sol, 1 sec
20268
20269
20270 geo.19.3.g4:
20271 ▷ $(ORBITER) -v 2 \
20272 ▷ ▷ -define Test_lines -set -loop 4 20 1 -end \
20273 ▷ ▷ -define Geo -geometry_builder \
20274 ▷ ▷ ▷ -V 19 -B 19 -TDO 3 \
20275 ▷ ▷ ▷ -fuse 1 -fname.GEO 19.3.g4 \
20276 ▷ ▷ ▷ -output_to_inc_file \
20277 ▷ ▷ ▷ -girth 4 \
20278 ▷ ▷ ▷ -special_test_not_orderly \
20279 ▷ ▷ ▷ -test Test_lines \
20280 ▷ ▷ -end
20281
20282 # 14 sol, 10 sec on Mac
20283
20284 geo.20.3.g4:
20285 ▷ $(ORBITER) -v 2 \
20286 ▷ ▷ -define Test_lines -set -loop 4 21 1 -end \
20287 ▷ ▷ -define Geo -geometry_builder \
20288 ▷ ▷ ▷ -V 20 -B 20 -TDO 3 \
20289 ▷ ▷ ▷ -fuse 1 -fname.GEO 20.3.g4 \
20290 ▷ ▷ ▷ -output_to_inc_file \
20291 ▷ ▷ ▷ -girth 4 \
20292 ▷ ▷ ▷ -special_test_not_orderly \
20293 ▷ ▷ ▷ -test Test_lines \
20294 ▷ ▷ -end
20295
20296 # 162 sol, User time: 2:5 on Mac
20297
20298 geo.21.3.g4:
20299 ▷ $(ORBITER) -v 1 \
20300 ▷ ▷ -define Test_lines -set -loop 4 22 1 -end \
20301 ▷ ▷ -define Geo -geometry_builder \
20302 ▷ ▷ ▷ -V 21 -B 21 -TDO 3 \
20303 ▷ ▷ ▷ -fuse 1 -fname.GEO 21.3.g4 \
20304 ▷ ▷ ▷ -output_to_inc_file \
20305 ▷ ▷ ▷ -girth 4 \
20306 ▷ ▷ ▷ -special_test_not_orderly \
20307 ▷ ▷ ▷ -test Test_lines \
20308 ▷ ▷ -end
20309
20310
20311 # number of geo = 4713
20312 #User time: 9:31

```

```

20313
20314
20315
20316 geo_15.4:
20317 ▷ $(ORBITER) -v 1 \
20318 ▷ ▷ -define Test_lines -set -loop 4 16 1 -end \
20319 ▷ ▷ -define Geo -geometry_builder \
20320 ▷ ▷ ▷ -V 15 -B 15 -TDO 4 \
20321 ▷ ▷ ▷ -fuse 1 -fname_GEO 15.4 \
20322 ▷ ▷ ▷ -output_to_inc_file \
20323 ▷ ▷ ▷ -search_tree \
20324 ▷ ▷ ▷ -special_test_not_orderly \
20325 ▷ ▷ ▷ -test Test_lines \
20326 ▷ ▷ ▷ -save_canonical_forms \
20327 ▷ ▷ -end
20328 ▷ $(ORBITER) -v 2 \
20329 ▷ ▷ -define Do -draw_options -embedded -radius 50 \
20330 ▷ ▷ ▷ -nodes_empty \
20331 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
20332 ▷ ▷ -tree_draw \
20333 ▷ ▷ ▷ -file 15.4_tree.txt \
20334 ▷ ▷ ▷ -draw_options Do \
20335 ▷ ▷ -end
20336 ▷ $(PDFLATEX) 15.4_tree_draw.tex
20337 ▷ $(OPEN) 15.4_tree_draw.pdf
20338
20339 # 4 objects
20340 # ago=360, 30, 24, 15
20341 #User time: 0.15 of a second, dt=15 tps = 100
20342 #nb.calls_to_densenautey=6767
20343
20344
20345
20346 geo_16.4.g4:
20347 ▷ $(ORBITER) -v 1 \
20348 ▷ ▷ -define Test_lines -set -loop 4 17 1 -end \
20349 ▷ ▷ -define Geo -geometry_builder \
20350 ▷ ▷ ▷ -V 16 -B 16 -TDO 4 \
20351 ▷ ▷ ▷ -fuse 1 -fname_GEO 16.4.g4 \
20352 ▷ ▷ ▷ -output_to_inc_file \
20353 ▷ ▷ ▷ -special_test_not_orderly \
20354 ▷ ▷ ▷ -girth 4 \
20355 ▷ ▷ ▷ -test Test_lines \
20356 ▷ ▷ -end
20357
20358 # none
20359
20360
20361 geo_15.6:
20362 ▷ $(ORBITER) -v 1 \
20363 ▷ ▷ -define Test_lines -set -loop 4 16 1 -end \
20364 ▷ ▷ -define Geo -geometry_builder \
20365 ▷ ▷ ▷ -V 15 -B 15 -TDO 6 -fuse 1 \
20366 ▷ ▷ ▷ -fname_GEO 15.6 \
20367 ▷ ▷ ▷ -special_test_not_orderly \
20368 ▷ ▷ ▷ -output_to_inc_file \
20369 ▷ ▷ ▷ -test Test_lines \
20370 ▷ ▷ ▷ -output_to_sage_file \
20371 ▷ ▷ -end

```

```

20372
20373
20374
20375
20376 geo_16:
20377 ▷ $(ORBITER) -v 1 \
20378 ▷ ▷ -define Test_lines -set -loop 3 17 1 -end \
20379 ▷ ▷ -define Geo -geometry_builder \
20380 ▷ ▷ ▷ -V 16 -B 20 -TDO 5 \
20381 ▷ ▷ ▷ -fuse 1 -fname_GEO geo_16 \
20382 ▷ ▷ ▷ -special_test_not_orderly \
20383 ▷ ▷ ▷ -output_to_inc_file \
20384 ▷ ▷ ▷ -test Test_lines \
20385 ▷ ▷ -end
20386
20387
20388 geo_40_4.g4:
20389 ▷ $(ORBITER) -v 2 \
20390 ▷ ▷ -define Test_lines -set -loop 0 41 1 -end \
20391 ▷ ▷ -define Geo -geometry_builder \
20392 ▷ ▷ ▷ -V 40 -B 40 -TDO 4 \
20393 ▷ ▷ ▷ -fuse 1 \
20394 ▷ ▷ ▷ -fname_GEO 40_4.g4 \
20395 ▷ ▷ ▷ -girth 4 \
20396 ▷ ▷ ▷ -search_tree \
20397 ▷ ▷ ▷ -special_test_not_orderly \
20398 ▷ ▷ ▷ -test Test_lines \
20399 ▷ ▷ ▷ -output_to_sage_file \
20400 ▷ ▷ ▷ -output_to_inc_file \
20401 ▷ ▷ ▷ -save_canonical_forms \
20402 ▷ ▷ -end
20403 ▷ $(ORBITER) -v 2 \
20404 ▷ ▷ -define Do -draw_options -embedded -radius 50 \
20405 ▷ ▷ ▷ -xin 10000 -yin 10000 \
20406 ▷ ▷ ▷ -xout 1000000 -yout 1000000 \
20407 ▷ ▷ ▷ -nodes_empty \
20408 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
20409 ▷ ▷ -tree_draw \
20410 ▷ ▷ ▷ -file 40_4.g4.tree.txt \
20411 ▷ ▷ ▷ -draw_options Do \
20412 ▷ ▷ -end
20413 ▷ $(PDFLATEX) 40_4.g4.tree_draw.tex
20414 ▷ $(OPEN) 40_4.g4.tree_draw.pdf
20415
20416
20417 #-special_test_not_orderly is important for speed purposes
20418 # 2 geos, ago=51840^2
20419 # 40_4.g4.inc
20420
20421
20422 geo_63_3.g6:
20423 ▷ $(ORBITER) -v 1 \
20424 ▷ ▷ -define Test_lines -set -loop 4 64 1 -end \
20425 ▷ ▷ -define Geo -geometry_builder \
20426 ▷ ▷ ▷ -V 63 -B 63 -TDO 3 \
20427 ▷ ▷ ▷ -fuse 1 -fname_GEO 63_3.g6 \
20428 ▷ ▷ ▷ -girth 6 \
20429 ▷ ▷ ▷ -test Test_lines \
20430 ▷ ▷ ▷ -special_test_not_orderly \

```

```

20431 ▷ ▷ ▷ -output_to_sage_file \
20432 ▷ ▷ ▷ -output_to_inc_file \
20433 ▷ ▷ ▷ -search_tree \
20434 ▷ ▷ ▷ -save_canonical_forms \
20435 ▷ ▷ -end
20436
20437 # time 0:38 on Mac Oct 15, 2022
20438 # 2 geos, ago: 12096^2
20439 # 63_3_g6.inc
20440 # split Cayley hexagon H(2) and its dual
20441 # group = G_2(2) = PGU(3,3)
20442 #User time: 1:37 on Mac Mar 28, 2026 (with many options on, including -search_tre
e)
20443
20444
20445
20446
20447
20448 geo_LSQ6.inc:
20449 ▷ $(ORBITER) -v 1 \
20450 ▷ ▷ -define Test_lines -set -loop 1 19 1 -end \
20451 ▷ ▷ -define Geo -geometry_builder \
20452 ▷ ▷ ▷ -V 6,6,6 -B 1,1,1,36 -TDO \
20453 ▷ ▷ ▷ "1,0,0,6, 0,1,0,6, 0,0,1,6" \
20454 ▷ ▷ ▷ -fuse 3 \
20455 ▷ ▷ ▷ -fname_GEO LSQ6 \
20456 ▷ ▷ ▷ -output_to_inc_file \
20457 ▷ ▷ ▷ -test Test_lines \
20458 ▷ ▷ -end
20459
20460 #LSQ6.inc
20461
20462 geo_MOL6.inc:
20463 ▷ $(ORBITER) -v 1 \
20464 ▷ ▷ -define Test_lines -set -loop 1 25 1 -end \
20465 ▷ ▷ -define Geo -geometry_builder \
20466 ▷ ▷ ▷ -V 6,6,6,6 -B 1,1,1,1,36 -TDO \
20467 ▷ ▷ ▷ "1,0,0,0,6, 0,1,0,0,6, 0,0,1,0,6, 0,0,0,1,6" \
20468 ▷ ▷ ▷ -fuse 4 \
20469 ▷ ▷ ▷ -fname_GEO MOL6 \
20470 ▷ ▷ ▷ -output_to_inc_file \
20471 ▷ ▷ ▷ -test Test_lines \
20472 ▷ ▷ -end
20473
20474
20475
20476
20477 #####
20478 # Section 12.7: Tactical Decompositions
20479
20480 SECTION_TACTICAL_DECOMPOSITIONS:
20481
20482 test_12.7:
20483 ▷ make max_arc_16_4.start
20484 ▷ make max_arc_16_4.convert_stack_tdo
20485 ▷ make max_arc_16_4.refine
20486 ▷ make max_arc_16_4r.print
20487
20488

```

```

20489 max_arc_16_4.start:
20490 ▷ $(ORBITER) -v 4 -maximal_arc_parameters 16 4
20491
20492
20493 max_arc_16_4.convert_stack.tdo:
20494 ▷ $(ORBITER) -v 4 -convert_stack_to_tdo max_arc_q16_r4.stack
20495
20496 max_arc_16_4.refine:
20497 ▷ $(ORBITER) -v 4 -tdo_refinement \
20498 ▷ ▷ -input_file max_arc_q16_r4.tdo -dual_is_linear_space -end
20499 ▷
20500 max_arc_16_4r.print:
20501 ▷ $(ORBITER) -v 4 -tdo_print max_arc_q16_r4r.tdo
20502
20503
20504
20505 #####
20506 # Section 12.8: Plesken Rings
20507
20508
20509 SECTION_CREATING_GRAPHS:
20510
20511
20512 test_12.8:
20513 ▷ make subspaces_GOp_6_2
20514 ▷ make subspaces_GOp_6_2.draw.Asup
20515 ▷ make subspaces_GOp_6_2.plesken_ring
20516 ▷ make subspaces_GOp_6_2.plesken_ring.evaluate.join
20517 ▷ make subspaces_GOp_6_2.make_pair_relations
20518
20519
20520
20521
20522
20523 subspaces_GOp_6_2:
20524 ▷ $(ORBITER) -v 5 \
20525 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
20526 ▷ ▷ -define Do -draw_options \
20527 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
20528 ▷ ▷ ▷ -poset_orbits_x_stretch 750 \
20529 ▷ ▷ ▷ -embedded \
20530 ▷ ▷ ▷ -poset_type_Asup \
20531 ▷ ▷ -end \
20532 ▷ ▷ -define Control -poset_classification_control \
20533 ▷ ▷ ▷ -problem_label GOp_6_2 -W -depth 6 \
20534 ▷ ▷ -end \
20535 ▷ ▷ -define G -linear_group -PGL 6 2 -orthogonal 1 -end \
20536 ▷ ▷ -define Orb -orbits \
20537 ▷ ▷ ▷ -group G \
20538 ▷ ▷ ▷ -group_action G \
20539 ▷ ▷ ▷ -generators G \
20540 ▷ ▷ ▷ -on_subspaces 6 Control \
20541 ▷ ▷ -end \
20542 ▷ ▷ -define ro \
20543 ▷ ▷ ▷ -poset_classification_report_options \
20544 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
20545 ▷ ▷ ▷ -draw_options Do \
20546 ▷ ▷ -end \
20547 ▷ ▷ -with Orb -do -orbits_activity \

```

```

20548 ▷ ▷ ▷ -report "" \
20549 ▷ ▷ ▷ -report_options ro \
20550 ▷ ▷ -end \
20551 ▷ ▷ -print_symbols
20552 ▷ $(PDFLATEX) GOp_6_2_poset.tex
20553 ▷ $(OPEN) GOp_6_2_poset.pdf
20554
20555
20556
20557 subspaces_GOp_6_2_draw.Asup:
20558 ▷ $(ORBITER) -v 5 \
20559 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
20560 ▷ ▷ -define Do -draw_options \
20561 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
20562 ▷ ▷ ▷ -poset_orbits_x_stretch 750 \
20563 ▷ ▷ ▷ -embedded \
20564 ▷ ▷ ▷ -poset_type_Asup \
20565 ▷ ▷ -end \
20566 ▷ ▷ -define Control -poset_classification_control \
20567 ▷ ▷ ▷ -problem_label GOp_6_2 -W -depth 6 \
20568 ▷ ▷ -end \
20569 ▷ ▷ -define G -linear_group -PGL 6 2 -orthogonal 1 -end \
20570 ▷ ▷ -define Orb -orbits \
20571 ▷ ▷ ▷ -group G \
20572 ▷ ▷ ▷ -group_action G \
20573 ▷ ▷ ▷ -generators G \
20574 ▷ ▷ ▷ -on_subspaces 6 Control \
20575 ▷ ▷ -end \
20576 ▷ ▷ -define PC_Activity -poset_classification_activity \
20577 ▷ ▷ ▷ -draw_full_poset Do \
20578 ▷ ▷ -end \
20579 ▷ ▷ -with Orb -do -orbits_activity \
20580 ▷ ▷ ▷ -poset_classification_activity PC_Activity \
20581 ▷ ▷ -end \
20582 ▷ ▷ -print_symbols
20583 ▷ $(ORBITER) -v 3 \
20584 ▷ ▷ -draw_layered_graph \
20585 ▷ ▷ ▷ GOp_6_2_poset_full_lvl_6.Asup.layered_graph \
20586 ▷ ▷ -radius 50 -embedded \
20587 ▷ ▷ ▷ -line_width 0.2 -x_stretch 1.4 -scale 0.5 \
20588 ▷ ▷ -end
20589 ▷ $(PDFLATEX) GOp_6_2_poset_full_lvl_6.Asup.draw.tex
20590 ▷ $(OPEN) GOp_6_2_poset_full_lvl_6.Asup.draw.pdf
20591
20592
20593 #cp GOp_6_2_poset_full_lvl_6.Asup.draw.pdf ~/TEX.26/ORBITER.USERS.GUIDE/GRAPHICS/
    PLESKEN_RING
20594
20595
20596
20597
20598 subspaces_GOp_6_2_plesken_ring:
20599 ▷ $(ORBITER) -v 5 \
20600 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
20601 ▷ ▷ -define Do -draw_options \
20602 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
20603 ▷ ▷ ▷ -poset_orbits_x_stretch 750 \
20604 ▷ ▷ ▷ -embedded \
20605 ▷ ▷ ▷ -poset_type_Asup \

```

```

20606 ▷ ▷ -end \
20607 ▷ ▷ -define Control -poset_classification_control \
20608 ▷ ▷ ▷ -problem_label GOp_6_2 -W -depth 6 \
20609 ▷ ▷ -end \
20610 ▷ ▷ -define G -linear_group -PGL 6 2 -orthogonal 1 -end \
20611 ▷ ▷ -define Orb -orbits \
20612 ▷ ▷ ▷ -group G \
20613 ▷ ▷ ▷ -group_action G \
20614 ▷ ▷ ▷ -generators G \
20615 ▷ ▷ ▷ -on_subspaces 6 Control \
20616 ▷ ▷ -end \
20617 ▷ ▷ -define PC_Plesken -poset_classification_activity \
20618 ▷ ▷ ▷ -plesken_ring \
20619 ▷ ▷ -end \
20620 ▷ ▷ -assign P -from \
20621 ▷ ▷ -with Orb -do -orbits_activity \
20622 ▷ ▷ ▷ -poset_classification_activity PC_Plesken \
20623 ▷ ▷ -end \
20624 ▷ ▷ -with P -do -plesken_ring_activity \
20625 ▷ ▷ ▷ -report Do \
20626 ▷ ▷ -end \
20627 ▷ ▷ -print_symbols
20628 ▷ $(PDFLATEX) GOp_6_2_plesken_ring.tex
20629 ▷ $(OPEN) GOp_6_2_plesken_ring.pdf
20630
20631
20632 # there are 19 orbits, so we need 19 variables:
20633
20634 subspaces_GOp_6_2_plesken_ring_evaluate_join:
20635 ▷ $(ORBITER) -v 5 \
20636 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
20637 ▷ ▷ -define Do -draw_options \
20638 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
20639 ▷ ▷ ▷ -poset_orbits_x_stretch 750 \
20640 ▷ ▷ ▷ -embedded \
20641 ▷ ▷ ▷ -poset_type_Asup \
20642 ▷ ▷ -end \
20643 ▷ ▷ -define Control -poset_classification_control \
20644 ▷ ▷ ▷ -problem_label Op_6_2 -W -depth 6 \
20645 ▷ ▷ -end \
20646 ▷ ▷ -define G -linear_group -PGL 6 2 -orthogonal 1 -end \
20647 ▷ ▷ -define Orb -orbits \
20648 ▷ ▷ ▷ -group G \
20649 ▷ ▷ ▷ -group_action G \
20650 ▷ ▷ ▷ -generators G \
20651 ▷ ▷ ▷ -on_subspaces 6 Control \
20652 ▷ ▷ -end \
20653 ▷ ▷ -define ro \
20654 ▷ ▷ ▷ -poset_classification_report_options \
20655 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
20656 ▷ ▷ ▷ -draw_options Do \
20657 ▷ ▷ -end \
20658 ▷ ▷ -with Orb -do -orbits_activity \
20659 ▷ ▷ ▷ -report "" \
20660 ▷ ▷ ▷ -report_options ro \
20661 ▷ ▷ -end \
20662 ▷ ▷ -define PC_Plesken -poset_classification_activity \
20663 ▷ ▷ ▷ -plesken_ring \
20664 ▷ ▷ -end \

```

```

20665 ▷ ▷ -assign P -from \
20666 ▷ ▷ -with Orb -do -orbits.activity \
20667 ▷ ▷ ▷ -poset.classification.activity PC_Plesken \
20668 ▷ ▷ -end \
20669 ▷ ▷ -with P -do -plesken.ring.activity \
20670 ▷ ▷ ▷ -report Do \
20671 ▷ ▷ -end \
20672 ▷ ▷ -define F -finite.field -q 2 -end \
20673 ▷ ▷ -define formula -symbolic.object \
20674 ▷ ▷ ▷ -field F \
20675 ▷ ▷ ▷ -text "X6^2"\
20676 ▷ ▷ -end \
20677 ▷ ▷ -define R -polynomial.ring \
20678 ▷ ▷ ▷ -field F \
20679 ▷ ▷ ▷ -number_of_variables 19 \
20680 ▷ ▷ ▷ -homogeneous_of_degree 2 \
20681 ▷ ▷ ▷ -monomial_ordering_partition \
20682 ▷ ▷ ▷ -variables "X0,X1,X2,X3,X4,X5,X6,X7,X8,X9,X10,X11,X12,X13,X14,X15,X16,X17,X
18" \
20683 ▷ ▷ ▷ "X_0,X_1,X_2,X_3,X_4,X_5,X_6,X_7,X_8,X_9,X_{10},X_{11},X_{12},X_{13},X_{14}
,X_{15},X_{16},X_{17},X_{18}" \
20684 ▷ ▷ -end \
20685 ▷ ▷ -with P -do -plesken.ring.activity \
20686 ▷ ▷ ▷ -evaluate_join R formula \
20687 ▷ ▷ -end \
20688 ▷ ▷ -print_symbols
20689
20690
20691 subspaces_GOp_6_2.make_pair_relations:
20692 ▷ ▷ $(ORBITER) -v 5 \
20693 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
20694 ▷ ▷ -define Do -draw.options \
20695 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
20696 ▷ ▷ ▷ -poset.orbits.x.stretch 750 \
20697 ▷ ▷ ▷ -embedded \
20698 ▷ ▷ ▷ -poset.type.Asup \
20699 ▷ ▷ -end \
20700 ▷ ▷ -define Control -poset.classification.control \
20701 ▷ ▷ ▷ -problem_label Op_6_2 -W -depth 6 \
20702 ▷ ▷ -end \
20703 ▷ ▷ -define G -linear.group -PGL 6 2 -orthogonal 1 -end \
20704 ▷ ▷ -define Orb -orbits \
20705 ▷ ▷ ▷ -group G \
20706 ▷ ▷ ▷ -group.action G \
20707 ▷ ▷ ▷ -generators G \
20708 ▷ ▷ ▷ -on_subspaces 6 Control \
20709 ▷ ▷ -end \
20710 ▷ ▷ -define ro \
20711 ▷ ▷ ▷ -poset.classification.report.options \
20712 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
20713 ▷ ▷ ▷ -draw.options Do \
20714 ▷ ▷ -end \
20715 ▷ ▷ -with Orb -do -orbits.activity \
20716 ▷ ▷ ▷ -report "" \
20717 ▷ ▷ ▷ -report_options ro \
20718 ▷ ▷ -end \
20719 ▷ ▷ -define PC_Activity -poset.classification.activity \
20720 ▷ ▷ ▷ -pair_relations.within_orbit 6 \
20721 ▷ ▷ -end \

```



```

20722 ▷ ▷ -with Orb -do -orbits_activity \
20723 ▷ ▷ ▷ -poset_classification_activity PC_Activity \
20724 ▷ ▷ -end \
20725 ▷ ▷ -print_symbols
20726
20727
20728
20729
20730 #####
20731 # Chapter 13 - Graph Theory
20732 #####
20733
20734
20735 test_13:
20736 ▷ make test_13_1
20737 ▷ make test_13_2
20738 ▷ make test_13_3
20739
20740
20741 #####
20742 # Section 13.1: Creating Graphs
20743
20744
20745 SECTION.CREATING_GRAPHS:
20746
20747
20748 test_13_1:
20749 ▷ make make_graph_from_adjacency_matrix
20750 ▷ make make_triangle_graph
20751 ▷ make Cycle_graph_5
20752 ▷ make Cycle_graph_13
20753 ▷ make Chain_232
20754 ▷ make Paley_13_graph
20755 ▷ make Paley_13_graph_complement
20756 ▷ make Winnie_Li_graph_q16_index2
20757 ▷ make Sarnak_graph_29_5
20758 ▷ make Neumaier_graph_16
20759 ▷ make Neumaier_graph_16_reordered
20760 ▷ make tritangent_planes_graph
20761 ▷ make trihedral_pair_graph
20762 ▷ make small_graph
20763 ▷ make petersen
20764 ▷ make Johnson_6_2_0
20765 ▷ make Hamming_graph_3
20766 ▷ make Hamming_graph_7
20767 ▷ make Op_6_2_coll_graph
20768 ▷ make HJ_graph
20769 ▷ make Cayley_Z11_1mod3
20770 ▷ make Cayley_Sym4_coxeter
20771 ▷ make Cayley_Sym4_star
20772 ▷ make Queens_graph
20773 ▷ make Op_4_2_affine_polar_graph
20774 ▷ make Paulus_1
20775 ▷ make Paulus_15
20776 ▷ make Higman_Sims_graph
20777
20778
20779 make_graph_from_adjacency_matrix:
20780 ▷ $(ORBITER) -v 6 \

```

```

20781 ▷ ▷ -define triangle -vector -dense "0,1,1, 1,0,1, 1,1,0" -end \
20782 ▷ ▷ -define G -graph \
20783 ▷ ▷ ▷ -adjacency_matrix \
20784 ▷ ▷ ▷ triangle \
20785 ▷ ▷ -end
20786
20787
20788 make_triangle_graph:
20789 ▷ echo $(TRIANGLE_GRAPH) >triangle_graph.csv
20790 ▷ $(ORBITER) -v 6 \
20791 ▷ ▷ -define G -graph \
20792 ▷ ▷ ▷ -load_csv.no.border \
20793 ▷ ▷ ▷ triangle_graph.csv \
20794 ▷ ▷ -end
20795
20796
20797 Cycle_graph_5:
20798 ▷ $(ORBITER) -v 2 \
20799 ▷ ▷ -define Gamma -graph \
20800 ▷ ▷ ▷ -cycle 5 \
20801 ▷ ▷ -end \
20802 ▷ ▷ -with Gamma -do \
20803 ▷ ▷ ▷ -graph_theoretic_activity -export_csv \
20804 ▷ ▷ -end
20805
20806
20807
20808
20809
20810 Cycle_graph_13:
20811 ▷ $(ORBITER) -v 2 \
20812 ▷ ▷ -define Gamma -graph \
20813 ▷ ▷ ▷ -cycle 13 \
20814 ▷ ▷ -end \
20815 ▷ ▷ -with Gamma -do \
20816 ▷ ▷ ▷ -graph_theoretic_activity -export_csv \
20817 ▷ ▷ -end
20818
20819
20820
20821
20822
20823
20824 Chain_232:
20825 ▷ $(ORBITER) -v 2 \
20826 ▷ ▷ -define P1 -vector -dense 2,3,2 -end \
20827 ▷ ▷ -define P2 -vector -dense 2,3,2 -end \
20828 ▷ ▷ -define Gamma -graph \
20829 ▷ ▷ ▷ -chain_graph P1 P2 \
20830 ▷ ▷ -end
20831
20832
20833
20834
20835 Paley_13_graph:
20836 ▷ $(ORBITER) -v 2 \
20837 ▷ ▷ -define F -finite_field -q 13 -end \
20838 ▷ ▷ -define Gamma -graph -Paley F -end \
20839 ▷ ▷ -with Gamma -do \

```

```

20840 ▷ ▷ ▷ -graph.theoretic.activity -export.csv \
20841 ▷ ▷ -end
20842
20843
20844 Paley_13_graph_complement:
20845 ▷ $(ORBITER) -v 2 \
20846 ▷ ▷ -define F -finite.field -q 13 -end \
20847 ▷ ▷ -define Gamma -graph -Paley F -complement -end \
20848
20849 Winnie_Li_graph_q16_index2:
20850 ▷ $(ORBITER) -v 2 \
20851 ▷ ▷ -define F -finite.field -q 16 -end \
20852 ▷ ▷ -define Gamma -graph -Winnie_Li F 2 -end \
20853
20854
20855 Sarnak_graph_29_5:
20856 ▷ $(ORBITER) -v 2 \
20857 ▷ ▷ -define Gamma -graph -Sarnak 29 5 -end \
20858
20859
20860 Neumaier_graph_16:
20861 ▷ $(ORBITER) -v 2 \
20862 ▷ ▷ -define Gamma -graph -Neumaier_graph_16 -end \
20863
20864 Neumaier_graph_16_reordered:
20865 ▷ $(ORBITER) -v 2 \
20866 ▷ ▷ -define perm -vector -dense "0,1,2,3,4,5,12,13,6,7,8,9,10,11,14,15" -end \
20867 ▷ ▷ -define Gamma -graph -Neumaier_graph_16 -reorder perm -end \
20868
20869
20870 tritangent_planes_graph:
20871 ▷ $(ORBITER) -v 2 \
20872 ▷ ▷ -define Gamma \
20873 ▷ ▷ ▷ -graph -tritangent_planes_disjointness_graph \
20874 ▷ ▷ -end
20875
20876
20877 trihedral_pair_graph:
20878 ▷ $(ORBITER) -v 2 \
20879 ▷ ▷ -define Gamma \
20880 ▷ ▷ ▷ -graph -trihedral_pair_disjointness_graph \
20881 ▷ ▷ -end
20882
20883
20884
20885 small_graph:
20886 ▷ $(ORBITER) -v 2 \
20887 ▷ ▷ -define G -graph -edges_as_pairs \
20888 ▷ ▷ ▷ 5 "0,1,0,2,0,3,0,4,1,3,1,4,2,4" \
20889 ▷ ▷ -end
20890
20891
20892
20893
20894 petersen:
20895 ▷ $(ORBITER) -v 2 \
20896 ▷ ▷ -define G -graph -Johnson 5 2 0 -end
20897
20898

```

```

20899
20900
20901 Johnson_6_2_0:
20902 ▷ $(ORBITER) -v 2 \
20903 ▷ ▷ -define G -graph -Johnson 6 2 0 -end
20904
20905 Hamming_graph_3:
20906 ▷ $(ORBITER) -v 2 \
20907 ▷ ▷ -define G -graph -Hamming 3 2 -end
20908
20909
20910 Hamming_graph_7:
20911 ▷ $(ORBITER) -v 2 \
20912 ▷ ▷ -define G -graph -Hamming 7 2 -end
20913
20914 # needs halljanko315.csv
20915 # from https://www.win.tue.nl/~aeb/drg/graphs/HJ315.html
20916 #There is a unique distance-regular graph Gamma with intersection array {10,8,2
      ; 1,1,4,5}. It was constructed in Cohen (1981), and uniqueness (given the interse
      ction array) was proved in Cohen & Tits (1985).
20917
20918
20919 Op_6_2_coll_graph:
20920 ▷ $(ORBITER) -v 2 \
20921 ▷ ▷ -define F -finite_field -q 13 -end \
20922 ▷ ▷ -define O -orthogonal_space 1 6 F -end \
20923 ▷ ▷ -define v -vector -loop 0 35 1 -end \
20924 ▷ ▷ -define Gamma -graph -coll_orthogonal O v -end \
20925 ▷ ▷ -with Gamma -do \
20926 ▷ ▷ ▷ -graph_theoretic_activity -save \
20927 ▷ ▷ -end
20928
20929
20930 #0_1_6_13_coll_v.colored_graph
20931
20932
20933 HJ_graph:
20934 ▷ cp $(ORBITER_PATH)/examples/users_guide/halljanko315.csv .
20935 ▷ $(ORBITER) -v 6 \
20936 ▷ ▷ -define G -graph \
20937 ▷ ▷ ▷ -load_csv.no.border \
20938 ▷ ▷ ▷ halljanko315.csv \
20939 ▷ ▷ -end
20940
20941
20942
20943
20944
20945 Cayley_Z11_1mod3:
20946 ▷ $(ORBITER) -v 2 \
20947 ▷ ▷ -define F -finite_field -q 11 -end \
20948 ▷ ▷ -define S -vector -dense \
20949 ▷ ▷ ▷ "1,1, 1,4, 1,7, 1,10" -end \
20950 ▷ ▷ -define G -linear_group -AGL 1 F \
20951 ▷ ▷ ▷ -subgroup_by_generators "Z11" 11 1 "1,1" \
20952 ▷ ▷ -end \
20953 ▷ ▷ -define Gamma -graph \
20954 ▷ ▷ ▷ -Cayley_graph G S \
20955 ▷ ▷ -end

```

```

20956
20957
20958 Cayley_Sym4_coxeter:
20959 ▷ $(ORBITER) -v 2 \
20960 ▷ ▷ -define S -vector -dense "1,0,2,3, 0,2,1,3, 0,1,3,2" -end \
20961 ▷ ▷ -define G -permutation_group -symmetric_group 4 \
20962 ▷ ▷ -end \
20963 ▷ ▷ -define Gamma -graph \
20964 ▷ ▷ ▷ -Cayley_graph G S \
20965 ▷ ▷ -end
20966
20967
20968 Cayley_Sym4_star:
20969 ▷ $(ORBITER) -v 2 \
20970 ▷ ▷ -define S -vector -dense "1,0,2,3, 2,1,0,3, 3,1,2,0" -end \
20971 ▷ ▷ -define G -permutation_group -symmetric_group 4 \
20972 ▷ ▷ -end \
20973 ▷ ▷ -define Gamma -graph \
20974 ▷ ▷ ▷ -Cayley_graph G S \
20975 ▷ ▷ -end
20976
20977 Queens_graph:
20978 ▷ $(ORBITER) -v 2 \
20979 ▷ ▷ -define G -graph -non_attacking_queens_graph 8 -end \
20980
20981 Op_4_2_affine_polar_graph:
20982 ▷ $(ORBITER) -v 2 \
20983 ▷ ▷ -define F -finite_field -q 2 -end \
20984 ▷ ▷ -define O -orthogonal_space 1 4 F -end \
20985 ▷ ▷ -define Gamma -graph -affine_polar O -end \
20986 ▷ ▷ -with Gamma -do \
20987 ▷ ▷ ▷ -graph_theoretic_activity -save \
20988 ▷ ▷ -end
20989
20990 Paulus_1:
20991 ▷ $(ORBITER) -v 6 \
20992 ▷ ▷ -define P25_1 -graph -load_Brouwer \
20993 ▷ ▷ ▷ $(ORBITER_PATH)/examples/users_guide/PAULUS_ROSENFELD/P25_1.txt \
20994 ▷ ▷ -end
20995
20996 Paulus_15:
20997 ▷ $(ORBITER) -v 6 \
20998 ▷ ▷ -define P25_15 -graph -load_Brouwer \
20999 ▷ ▷ ▷ $(ORBITER_PATH)/examples/users_guide/PAULUS_ROSENFELD/P25_15.txt \
21000 ▷ ▷ -end
21001
21002
21003
21004
21005 Higman_Sims_graph:
21006 ▷ $(ORBITER) -v 2 \
21007 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
21008 ▷ ▷ -define Do -draw_options \
21009 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
21010 ▷ ▷ -end \
21011 ▷ ▷ -define code -vector \
21012 ▷ ▷ ▷ -dense $(GOLAY_24_CODE_PROJECTIVE) \
21013 ▷ ▷ -end \
21014 ▷ ▷ -define Control2 -poset_classification_control \

```

```

21015 ▷ ▷ ▷ -problem_label M24.pairs -W -depth 2 \
21016 ▷ ▷ ▷ -draw_options Do \
21017 ▷ ▷ -end \
21018 ▷ ▷ -define F -finite_field -q 2 -end \
21019 ▷ ▷ -define G -linear_group -PGL 12 2 \
21020 ▷ ▷ -subgroup_by_generators "M24" "244823040" 12 \
21021 ▷ ▷ ▷ $(GOLAY_24.CODE_AUT_GEN) \
21022 ▷ ▷ -end \
21023 ▷ ▷ -define M24.perm -modified_group -from G \
21024 ▷ ▷ ▷ -restricted_action code \\_on\\_code \
21025 ▷ ▷ -end \
21026 ▷ ▷ -define M22.perm -modified_group -from M24.perm \
21027 ▷ ▷ ▷ -set_stabilizer 0,1 Control2 \
21028 ▷ ▷ -end \
21029 ▷ ▷ -define pts -set -loop 2 22 1 -end \
21030 ▷ ▷ -define M22 -modified_group -from M22.perm \
21031 ▷ ▷ ▷ -restricted_action pts M22 \
21032 ▷ ▷ -end \
21033 ▷ ▷ -with M22 -do \
21034 ▷ ▷ -group_theoretic_activity \
21035 ▷ ▷ ▷ -report Do \
21036 ▷ ▷ -end \
21037 ▷ ▷ -define Control -poset_classification_control \
21038 ▷ ▷ ▷ -problem_label M22 -W -depth 6 \
21039 ▷ ▷ ▷ -draw_options Do \
21040 ▷ ▷ -end \
21041 ▷ ▷ -define Orb -orbits \
21042 ▷ ▷ ▷ -group M22 \
21043 ▷ ▷ ▷ -group_action M22 \
21044 ▷ ▷ ▷ -generators M22 \
21045 ▷ ▷ ▷ -on_subsets 6 Control \
21046 ▷ ▷ -end \
21047 ▷ ▷ -with Orb -do -orbits_activity \
21048 ▷ ▷ ▷ -Kramer_Mesner_matrix 3 6 \
21049 ▷ ▷ -end \
21050 ▷ ▷ -define ro \
21051 ▷ ▷ ▷ -poset_classification_report_options \
21052 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
21053 ▷ ▷ ▷ -draw_options Do \
21054 ▷ ▷ -end \
21055 ▷ ▷ -with Orb -do -orbits_activity \
21056 ▷ ▷ ▷ -report "" \
21057 ▷ ▷ ▷ -report_options ro \
21058 ▷ ▷ -end \
21059 ▷ ▷ -define D -design -orbit_on_sets \
21060 ▷ ▷ ▷ Orb 6 1 \
21061 ▷ ▷ -end \
21062 ▷ ▷ -with D -do \
21063 ▷ ▷ ▷ -design_activity \
21064 ▷ ▷ ▷ ▷ -export_flags \
21065 ▷ ▷ -end \
21066 ▷ ▷ -with D -do \
21067 ▷ ▷ ▷ -design_activity \
21068 ▷ ▷ ▷ ▷ -intersection_matrix \
21069 ▷ ▷ -end \
21070 ▷ ▷ -define Control_block_orbits -poset_classification_control \
21071 ▷ ▷ ▷ -problem_label M22bar_block_orbits -W -depth 2 \
21072 ▷ ▷ ▷ -draw_options Do \
21073 ▷ ▷ -end \

```

```

21074 ▷ ▷ -with D -do \
21075 ▷ ▷ ▷ -design_activity \
21076 ▷ ▷ ▷ ▷ -orbits_on_blocks 2 Control_block_orbits \
21077 ▷ ▷ -end \
21078 ▷ ▷ -with D -do \
21079 ▷ ▷ ▷ -design_activity \
21080 ▷ ▷ ▷ ▷ -one_point_extension 1 Control_block_orbits \
21081 ▷ ▷ -end \
21082 ▷ ▷ -define Gamma -graph \
21083 ▷ ▷ ▷ -load_adjacency_matrix_from_csv_and_select_value \
21084 ▷ ▷ ▷ blocks.v22_k6_one_pt_ext_orb1.csv 1 \
21085 ▷ ▷ -end
21086
21087
21088
21089
21090 #####
21091 # Section 13.2: Graph Theoretic Activities
21092
21093
21094 SECTION_GRAPH_THEORETIC_ACTIVITIES:
21095
21096
21097 test_13.2:
21098 ▷ make triangle_graph_properties
21099 ▷ make Hamming_graph_7_drg
21100 ▷ make Johnson_8_3_2_drg
21101 ▷ make Cycle_13_draw
21102 ▷ make Cycle_9_eigenvalues
21103 ▷ make Paley_13_draw
21104 ▷ make Paley_13_eigenvalues
21105 ▷ make tritangent_planes_graph_eigenvalues
21106 ▷ make Cayley_Z11_1mod3_eigenvalues_and_draw
21107 ▷ make Cayley_Sym4_coxeter_draw
21108 ▷ make Cayley_Sym5_coxeter_draw
21109 ▷ make Cayley_Sym4_star_eigenvalues_and_draw
21110 ▷ make graph_v5_e7.colored_graph
21111 ▷ make small_graph_draw
21112 ▷ make petersen_draw
21113 ▷ make Johnson_6_2_0_draw
21114 ▷ make Hamming_graph_3_draw
21115 ▷ make Hamming_graph_7_draw
21116 ▷ make Chain_232_properties
21117 ▷ make Chain_232_eigen
21118 ▷ make HJ_properties
21119 ▷ make HJ_d2_properties
21120 ▷ make PG0_5_2_collinearity_graph
21121 ▷ make trihedral_pair_graph_draw
21122 ▷ make Neumaier_graph_16_reordered_export_csv
21123 ▷ make Op_4_2_affine_polar_graph_info
21124 ▷ make Neumaier_16_test_SRG_property
21125 ▷ make Paulus_1_aut
21126 ▷ make Paulus_15_aut
21127 ▷ make PG_3_2_packing_make_group_and_test
21128 ▷ make Petersen_graph_distance_poset
21129 ▷ make Petersen_graph_distance_poset_draw
21130 ▷ make Petersen_test_if_distance_regular
21131 ▷ make Cayley_Sym5_distance_poset
21132

```

```

21133
21134 triangle_graph_properties:
21135 ▷ echo $(TRIANGLE_GRAPH) >triangle_graph.csv
21136 ▷ $(ORBITER) -v 6 \
21137 ▷ ▷ -define G -graph \
21138 ▷ ▷ ▷ -load_csv_no_border \
21139 ▷ ▷ ▷ triangle_graph.csv \
21140 ▷ ▷ -end \
21141 ▷ ▷ -with G -do \
21142 ▷ ▷ ▷ -graph_theoretic_activity -properties \
21143 ▷ ▷ -end
21144
21145 Hamming_graph_7_drg:
21146 ▷ $(ORBITER) -v 2 \
21147 ▷ ▷ -define Gamma -graph -Hamming 7 2 -end \
21148 ▷ ▷ -with Gamma -do -graph_theoretic_activity \
21149 ▷ ▷ ▷ -test_if_distance_regular 0 \
21150 ▷ ▷ -end
21151
21152
21153 Johnson_8_3_2_drg:
21154 ▷ $(ORBITER) -v 2 \
21155 ▷ ▷ -define Gamma -graph -Johnson 8 3 2 -end \
21156 ▷ ▷ -with Gamma -do -graph_theoretic_activity \
21157 ▷ ▷ ▷ -test_if_distance_regular 0 \
21158 ▷ ▷ -end
21159
21160
21161
21162 Cycle_13_draw:
21163 ▷ $(ORBITER) -v 2 \
21164 ▷ ▷ -define Gamma -graph -cycle 13 -end \
21165 ▷ ▷ -with Gamma -do \
21166 ▷ ▷ -graph_theoretic_activity -export_csv -end \
21167 ▷ ▷ -with Gamma -do \
21168 ▷ ▷ -graph_theoretic_activity -export_graphviz -end
21169 ▷ $(ORBITER) -v 2 -draw_matrix \
21170 ▷ ▷ -input_csv_file Cycle_13.csv \
21171 ▷ ▷ -box_width 20 -bit_depth 8 -partition 4 13 13 -end
21172 ▷ dot -Tpng Cycle_13.gv >Cycle_13.png
21173 ▷ #twopi -Tpng Cycle_13.gv >Cycle_13.png
21174 ▷ #$(OPEN) Cycle_13_draw.bmp
21175 ▷ #$(PDFLATEX) Cycle_13_report.tex
21176 ▷ #$(OPEN) Cycle_13_report.pdf
21177
21178
21179 Cycle_9_eigenvalues:
21180 ▷ $(ORBITER) -v 2 \
21181 ▷ ▷ -define Gamma -graph \
21182 ▷ ▷ ▷ -cycle 9 \
21183 ▷ ▷ -end \
21184 ▷ ▷ -with Gamma -do \
21185 ▷ ▷ -graph_theoretic_activity -eigenvalues -end \
21186 ▷ ▷ -with Gamma -do \
21187 ▷ ▷ -graph_theoretic_activity -eigenvalue_report -end
21188 ▷ $(PDFLATEX) Cycle_9_eigenvalues.tex
21189 ▷ $(OPEN) Cycle_9_eigenvalues.pdf
21190
21191

```



```

21192 Paley_13_draw:
21193 ▷ $(ORBITER) -v 2 \
21194 ▷ ▷ -define F -finite_field -q 13 -end \
21195 ▷ ▷ -define Gamma -graph -Paley F -end \
21196 ▷ ▷ -with Gamma -do \
21197 ▷ ▷ -graph_theoretic.activity -export_csv -end \
21198 ▷ ▷ -with Gamma -do \
21199 ▷ ▷ -graph_theoretic.activity -export_graphviz -end
21200 ▷ $(ORBITER) -v 2 -draw_matrix \
21201 ▷ ▷ -input_csv_file Paley_13.csv \
21202 ▷ ▷ -box_width 20 -bit_depth 8 -partition 4 13 13 -end
21203 ▷ dot -Tpng Paley_13.gv >Paley_13.png
21204 ▷ $(OPEN) Paley_13_draw.bmp
21205
21206
21207 Paley_13_eigenvalues:
21208 ▷ $(ORBITER) -v 2 \
21209 ▷ ▷ -define F -finite_field -q 13 -end \
21210 ▷ ▷ -define Gamma -graph -Paley F -end \
21211 ▷ ▷ -with Gamma -do \
21212 ▷ ▷ -graph_theoretic.activity -eigenvalues -end \
21213 ▷ ▷ -with Gamma -do \
21214 ▷ ▷ -graph_theoretic.activity -eigenvalue_report -end
21215 ▷ $(PDFLATEX) Paley_13_eigenvalues.tex
21216 ▷ $(OPEN) Paley_13_eigenvalues.pdf
21217
21218 #Paley_13_eigenvalues.csv
21219
21220 tritangent_planes_graph_eigenvalues:
21221 ▷ $(ORBITER) -v 2 \
21222 ▷ ▷ -define Gamma \
21223 ▷ ▷ ▷ -graph -tritangent_planes.disjointness_graph \
21224 ▷ ▷ -end \
21225 ▷ ▷ -with Gamma -do \
21226 ▷ ▷ -graph_theoretic.activity -eigenvalues -end \
21227 ▷ ▷ -with Gamma -do \
21228 ▷ ▷ -graph_theoretic.activity -eigenvalue_report -end
21229
21230
21231
21232 Cayley_Z11_1mod3_eigenvalues_and.draw:
21233 ▷ $(ORBITER) -v 2 \
21234 ▷ ▷ -define Do -draw_options -xin 20000 \
21235 ▷ ▷ ▷ -yin 20000 -embedded -radius 200 -end \
21236 ▷ ▷ -define F -finite_field -q 11 -end \
21237 ▷ ▷ -define S -vector -dense \
21238 ▷ ▷ ▷ "1,1, 1,4, 1,7, 1,10" -end \
21239 ▷ ▷ -define G -linear_group -AGL 1 F \
21240 ▷ ▷ ▷ -subgroup_by_generators "Z11" 11 1 "1,1" \
21241 ▷ ▷ -end \
21242 ▷ ▷ -define Gamma -graph \
21243 ▷ ▷ ▷ -Cayley_graph G S \
21244 ▷ ▷ -end \
21245 ▷ ▷ -with Gamma -do \
21246 ▷ ▷ -graph_theoretic.activity -eigenvalues -end \
21247 ▷ ▷ -with Gamma -do \
21248 ▷ ▷ -graph_theoretic.activity -draw Do -end
21249 ▷ $(PDFLATEX) Cayley_graph_AGL_1_11.draw.tex
21250 ▷ $(OPEN) Cayley_graph_AGL_1_11.draw.pdf

```

```

21251
21252
21253 Cayley_Sym4_coxeter_draw:
21254 ▷ $(ORBITER) -v 2 \
21255 ▷ ▷ -define Do -draw.options -xin 2000000 -yin 2000000 \
21256 ▷ ▷ ▷ -radius 20000 -embedded -nodes.empty -end \
21257 ▷ ▷ -define S -vector -dense \
21258 ▷ ▷ ▷ "1,0,2,3, 0,2,1,3, 0,1,3,2" -end \
21259 ▷ ▷ -define G -permutation_group -symmetric_group 4 \
21260 ▷ ▷ -end \
21261 ▷ ▷ -define Gamma -graph \
21262 ▷ ▷ ▷ -Cayley_graph G S \
21263 ▷ ▷ -end \
21264 ▷ ▷ -with Gamma -do \
21265 ▷ ▷ -graph_theoretic_activity -draw Do -end
21266 ▷ $(PDFLATEX) Cayley_graph_Sym_4.draw.tex
21267 ▷ $(OPEN) Cayley_graph_Sym_4.draw.pdf
21268
21269 Cayley_Sym5_coxeter_draw:
21270 ▷ $(ORBITER) -v 2 \
21271 ▷ ▷ -define Do -draw.options -xin 1000000 -yin 1000000 \
21272 ▷ ▷ ▷ -embedded -radius 10000 -nodes.empty -end \
21273 ▷ ▷ -define S -vector -dense \
21274 ▷ ▷ ▷ "1,0,2,3,4, 0,2,1,3,4, 0,1,3,2,4, 0,1,2,4,3" -end \
21275 ▷ ▷ -define G -permutation_group -symmetric_group 5 \
21276 ▷ ▷ -end \
21277 ▷ ▷ -define Gamma -graph \
21278 ▷ ▷ ▷ -Cayley_graph G S \
21279 ▷ ▷ -end \
21280 ▷ ▷ -with Gamma -do \
21281 ▷ ▷ -graph_theoretic_activity -draw Do -end
21282 ▷ $(PDFLATEX) Cayley_graph_Sym_5.draw.tex
21283 ▷ $(OPEN) Cayley_graph_Sym_5.draw.pdf
21284
21285
21286 Cayley_Sym4_star_eigenvalues_and.draw:
21287 ▷ $(ORBITER) -v 2 \
21288 ▷ ▷ -define Do -draw.options -xin 1000000 -yin 1000000 -embedded -end \
21289 ▷ ▷ -define S -vector -dense "1,0,2,3, 2,1,0,3, 3,1,2,0" -end \
21290 ▷ ▷ -define G -permutation_group -symmetric_group 4 \
21291 ▷ ▷ -end \
21292 ▷ ▷ -define Gamma -graph \
21293 ▷ ▷ ▷ -Cayley_graph G S \
21294 ▷ ▷ -end \
21295 ▷ ▷ -with Gamma -do \
21296 ▷ ▷ -graph_theoretic_activity -eigenvalues -end \
21297 ▷ ▷ -with Gamma -do \
21298 ▷ ▷ -graph_theoretic_activity -eigenvalue_report -end \
21299 ▷ ▷ -with Gamma -do \
21300 ▷ ▷ -graph_theoretic_activity -draw Do -end
21301 ▷ $(PDFLATEX) Cayley_graph_Sym_4.draw.tex
21302 ▷ $(OPEN) Cayley_graph_Sym_4.draw.pdf
21303 ▷ $(PDFLATEX) Cayley_graph_Sym_4.eigenvalues.tex
21304 ▷ $(OPEN) Cayley_graph_Sym_4.eigenvalues.pdf
21305
21306
21307
21308
21309 graph_v5_e7.colored_graph:

```

```

21310 ▷ $(ORBITER) -v 2 \
21311 ▷ ▷ -define G -graph -edges_as_pairs 5 \
21312 ▷ ▷ ▷ "0,1,0,2,0,3,0,4,1,3,1,4,2,4" \
21313 ▷ ▷ -end \
21314 ▷ ▷ -with G -do \
21315 ▷ ▷ -graph_theoretic_activity -save -end
21316
21317
21318
21319 small_graph_draw:
21320 ▷ $(ORBITER) -v 2 \
21321 ▷ ▷ -define G -graph -edges_as_pairs 5 \
21322 ▷ ▷ ▷ "0,1,0,2,0,3,0,4,1,3,1,4,2,4" \
21323 ▷ ▷ -end \
21324 ▷ ▷ -with G -do \
21325 ▷ ▷ -graph_theoretic_activity -export_csv -end \
21326 ▷ ▷ -with G -do \
21327 ▷ ▷ -graph_theoretic_activity -export_graphviz -end \
21328 ▷ ▷ -with G -do \
21329 ▷ ▷ -graph_theoretic_activity -save -end
21330 ▷ $(ORBITER) -v 2 -draw_matrix \
21331 ▷ ▷ -input_csv_file graph_v5_e7.csv \
21332 ▷ ▷ -box_width 40 -bit_depth 24 \
21333 ▷ ▷ -partition 4 "1,1,1,1,1" "1,1,1,1,1" -end
21334 ▷ dot -Tpng graph_v5_e7.gv >graph_v5_e7.png
21335
21336 # creates graph_v5_e7.csv
21337 # creates graph_v5_e7.colored.graph
21338
21339
21340 petersen_draw:
21341 ▷ $(ORBITER) -v 2 \
21342 ▷ ▷ -define G -graph -Johnson 5 2 0 -end \
21343 ▷ ▷ -with G -do \
21344 ▷ ▷ -graph_theoretic_activity -export_csv -end \
21345 ▷ ▷ -with G -do \
21346 ▷ ▷ -graph_theoretic_activity -export_graphviz -end \
21347 ▷ ▷ -with G -do \
21348 ▷ ▷ -graph_theoretic_activity -save -end
21349 ▷ $(ORBITER) -v 2 -draw_matrix \
21350 ▷ ▷ -input_csv_file Johnson_5.2_0.csv \
21351 ▷ ▷ -box_width 40 -bit_depth 24 -partition 4 "10" "10" -end
21352 ▷ dot -Tpng Johnson_5.2_0.gv >Johnson_5.2_0.png
21353
21354
21355 Johnson_6.2_0_draw:
21356 ▷ $(ORBITER) -v 2 \
21357 ▷ ▷ -define G -graph -Johnson 6 2 0 -end \
21358 ▷ ▷ -with G -do \
21359 ▷ ▷ -graph_theoretic_activity -export_csv -end \
21360 ▷ ▷ -with G -do \
21361 ▷ ▷ -graph_theoretic_activity -export_graphviz -end \
21362 ▷ ▷ -with G -do \
21363 ▷ ▷ -graph_theoretic_activity -save -end
21364 ▷ $(ORBITER) -v 2 -draw_matrix \
21365 ▷ ▷ -input_csv_file Johnson_6.2_0.csv \
21366 ▷ ▷ -box_width 40 -bit_depth 24 -partition 4 "10" "10" -end
21367 ▷ dot -Tpng Johnson_6.2_0.gv >Johnson_6.2_0.png
21368

```

```

21369
21370
21371 Hamming_graph_3_draw:
21372 ▷ $(ORBITER) -v 2 \
21373 ▷ ▷ -define G -graph -Hamming 3 2 -end \
21374 ▷ ▷ -with G -do \
21375 ▷ ▷ -graph_theoretic.activity -export_csv -end \
21376 ▷ ▷ -with G -do \
21377 ▷ ▷ -graph_theoretic.activity -export_graphviz -end \
21378 ▷ ▷ -with G -do \
21379 ▷ ▷ -graph_theoretic.activity -save -end
21380 ▷ $(ORBITER) -v 2 -draw_matrix \
21381 ▷ ▷ -input_csv_file Hamming_3.2.csv \
21382 ▷ ▷ -box_width 40 -bit_depth 24 \
21383 ▷ ▷ -partition 4 "1,1,1,1,1,1,1,1" "1,1,1,1,1,1,1,1" -end
21384 ▷ dot -Tpng Hamming_3.2.gv >Hamming_3.2.png
21385
21386
21387 Hamming_graph_7_draw:
21388 ▷ $(ORBITER) -v 2 \
21389 ▷ ▷ -define G -graph -Hamming 7 2 -end \
21390 ▷ ▷ -with G -do \
21391 ▷ ▷ -graph_theoretic.activity -export_csv -end \
21392 ▷ ▷ -with G -do \
21393 ▷ ▷ -graph_theoretic.activity -export_graphviz -end \
21394 ▷ ▷ -with G -do \
21395 ▷ ▷ -graph_theoretic.activity -save -end
21396 ▷ $(ORBITER) -v 2 -draw_matrix \
21397 ▷ ▷ -input_csv_file Hamming_7.2.csv \
21398 ▷ ▷ -box_width 8 -bit_depth 24 -partition 4 128 128 -end
21399 ▷ dot -Tpng Hamming_7.2.gv >Hamming_7.2.png
21400
21401
21402
21403
21404
21405
21406 Chain_232_properties:
21407 ▷ $(ORBITER) -v 2 \
21408 ▷ ▷ -define P1 -vector -dense 2,3,2 -end \
21409 ▷ ▷ -define P2 -vector -dense 2,3,2 -end \
21410 ▷ ▷ -define Gamma -graph \
21411 ▷ ▷ ▷ -chain_graph P1 P2 \
21412 ▷ ▷ -end \
21413 ▷ ▷ -with Gamma -do \
21414 ▷ ▷ ▷ -graph_theoretic.activity -export_csv \
21415 ▷ ▷ -end \
21416 ▷ ▷ -with Gamma -do \
21417 ▷ ▷ ▷ -graph_theoretic.activity -properties \
21418 ▷ ▷ -end
21419
21420 Chain_232_eigen:
21421 ▷ $(ORBITER) -v 2 \
21422 ▷ ▷ -define P1 -vector -dense 2,3,2 -end \
21423 ▷ ▷ -define P2 -vector -dense 2,3,2 -end \
21424 ▷ ▷ -define Gamma -graph \
21425 ▷ ▷ ▷ -chain_graph P1 P2 \
21426 ▷ ▷ -end \
21427 ▷ ▷ -with Gamma -do \

```

```

21428 ▷ ▷ ▷ -graph.theoretic.activity \
21429 ▷ ▷ ▷ -eigenvalues \
21430 ▷ ▷ -end \
21431 ▷ ▷ -with Gamma -do \
21432 ▷ ▷ ▷ -graph.theoretic.activity \
21433 ▷ ▷ ▷ -eigenvalue.report \
21434 ▷ ▷ -end
21435 ▷ $(PDFLATEX) chain_graph.eigenvalues.tex
21436 ▷ $(OPEN) chain_graph.eigenvalues.pdf
21437
21438
21439
21440 # need the file halljanko315.csv
21441
21442 HJ_properties:
21443 ▷ $(ORBITER) -v 6 \
21444 ▷ ▷ -define G -graph \
21445 ▷ ▷ ▷ -load.csv.no.border \
21446 ▷ ▷ ▷ halljanko315.csv \
21447 ▷ ▷ -end \
21448 ▷ ▷ -with G -do \
21449 ▷ ▷ ▷ -graph.theoretic.activity -properties \
21450 ▷ ▷ -end
21451
21452 #Degree type: (10^{315} )
21453
21454
21455
21456 HJ_d2_properties:
21457 ▷ $(ORBITER) -v 6 \
21458 ▷ ▷ -define G -graph \
21459 ▷ ▷ ▷ -load.csv.no.border \
21460 ▷ ▷ ▷ halljanko315.csv \
21461 ▷ ▷ ▷ -distance.2 \
21462 ▷ ▷ -end \
21463 ▷ ▷ -with G -do \
21464 ▷ ▷ ▷ -graph.theoretic.activity \
21465 ▷ ▷ ▷ -properties \
21466 ▷ ▷ -end
21467
21468
21469 #Degree type: (80^{315} )
21470
21471
21472
21473
21474 PG0.5.2.collinearity_graph:
21475 ▷ $(ORBITER) -v 2 \
21476 ▷ ▷ -define F -finite.field -q 2 -end \
21477 ▷ ▷ -define O -orthogonal.space 0 5 F -without.group -end \
21478 ▷ ▷ -with O -do -orthogonal.space.activity \
21479 ▷ ▷ ▷ -export.point.line.incidence.matrix \
21480 ▷ ▷ -end
21481 ▷ $(ORBITER) -v 3 \
21482 ▷ ▷ -define Inc -vector -file 0.5.2.incidence.matrix.csv -end \
21483 ▷ ▷ -define Gamma -graph -collinearity_graph Inc -end \
21484 ▷ ▷ -with Gamma -do \
21485 ▷ ▷ -graph.theoretic.activity \
21486 ▷ ▷ ▷ -properties \

```

```

21487 ▷ ▷ -end
21488
21489
21490 trihedral_pair_graph_draw:
21491 ▷ $(ORBITER) -v 2 -define Gamma \
21492 ▷ ▷ -graph -trihedral_pair_disjointness_graph -end \
21493 ▷ ▷ -with Gamma -do \
21494 ▷ ▷ -graph_theoretic_activity -export_csv -end
21495 ▷ $(ORBITER) -v 2 -draw_matrix \
21496 ▷ ▷ -input_csv_file trihedral_pair_disjointness.csv \
21497 ▷ ▷ -box_width 20 -bit_depth 8 -end
21498 ▷ $(OPEN) trihedral_pair_disjointness_draw.bmp
21499
21500
21501
21502 Neumaier_graph_16_reordered_export_csv:
21503 ▷ $(ORBITER) -v 2 \
21504 ▷ ▷ -define perm -vector -dense "0,1,2,3,4,5,12,13,6,7,8,9,10,11,14,15" -end \
21505 ▷ ▷ -define Gamma -graph -Neumaier_graph_16 -reorder perm -end \
21506 ▷ ▷ -with Gamma -do \
21507 ▷ ▷ -graph_theoretic_activity -export_csv -end
21508
21509
21510 Op_4_2_affine_polar_graph_info:
21511 ▷ $(ORBITER) -v 2 \
21512 ▷ ▷ -define F -finite_field -q 2 -end \
21513 ▷ ▷ -define O -orthogonal_space 1 4 F -end \
21514 ▷ ▷ -define Gamma -graph -affine_polar O -end \
21515 ▷ ▷ -with Gamma -do \
21516 ▷ ▷ ▷ -graph_theoretic_activity -save \
21517 ▷ ▷ -end \
21518 ▷ ▷ -with Gamma -do \
21519 ▷ ▷ ▷ -graph_theoretic_activity -test_SRG_property \
21520 ▷ ▷ -end \
21521 ▷ ▷ -with Gamma -do \
21522 ▷ ▷ -graph_theoretic_activity \
21523 ▷ ▷ ▷ -find_cliques -target_size 4 \
21524 ▷ ▷ -end \
21525 ▷ ▷ -with Gamma -do \
21526 ▷ ▷ -graph_theoretic_activity \
21527 ▷ ▷ ▷ -automorphism_group \
21528 ▷ ▷ -end
21529 ▷ #$(PDFLATEX) affine_polar_0_1_4_2_report.tex
21530 ▷ #$(OPEN) affine_polar_0_1_4_2_report.pdf
21531
21532
21533 # 24 cliques in affine_polar_0_1_4_2_solutions.csv
21534 # ago = 1152
21535
21536
21537 Neumaier_16_test_SRG_property:
21538 ▷ $(ORBITER) -v 10 \
21539 ▷ ▷ -define Gamma -graph -Neumaier_graph_16 -end \
21540 ▷ ▷ -with Gamma -do \
21541 ▷ ▷ -graph_theoretic_activity \
21542 ▷ ▷ ▷ -test_SRG_property \
21543 ▷ ▷ -end
21544
21545

```

```

21546 Paulus_1_aut:
21547 ▷ $(ORBITER) -v 6 \
21548 ▷ ▷ -define P25_1 -graph -load_Brouwer \
21549 ▷ ▷ ▷ $(ORBITER_PATH)/examples/users_guide/PAULUS_ROSENFELD/P25_1.txt \
21550 ▷ ▷ -end \
21551 ▷ ▷ -with P25_1 -do \
21552 ▷ ▷ -graph_theoretic_activity \
21553 ▷ ▷ ▷ -automorphism_group \
21554 ▷ ▷ -end
21555 ▷ $(PDFLATEX) P25_1_report.tex
21556 ▷ $(OPEN) P25_1_report.pdf
21557
21558 Paulus_15_aut:
21559 ▷ $(ORBITER) -v 6 \
21560 ▷ ▷ -define P25_15 -graph -load_Brouwer \
21561 ▷ ▷ ▷ $(ORBITER_PATH)/examples/users_guide/PAULUS_ROSENFELD/P25_15.txt \
21562 ▷ ▷ -end \
21563 ▷ ▷ -with P25_15 -do \
21564 ▷ ▷ -graph_theoretic_activity \
21565 ▷ ▷ ▷ -automorphism_group \
21566 ▷ ▷ -end
21567 ▷ $(PDFLATEX) P25_15_report.tex
21568 ▷ $(OPEN) P25_15_report.pdf
21569
21570 # ago = 600
21571
21572
21573 PG_3_2_packing_make_group_and_test:
21574 ▷ - mkdir SPREAD_TABLES_2
21575 ▷ $(ORBITER) -v 5 \
21576 ▷ ▷ -define Control -poset_classification_control \
21577 ▷ ▷ ▷ -problem_label spreads_PG_3_2 \
21578 ▷ ▷ ▷ -W \
21579 ▷ ▷ -end \
21580 ▷ ▷ -define F -finite_field -q 2 -end \
21581 ▷ ▷ -define P3 -projective_space -n 3 -field F -v 0 -end \
21582 ▷ ▷ -define P5 -projective_space -n 5 -field F -v 0 -end \
21583 ▷ ▷ -define T -spread_table -space P3 -rk 2 \
21584 ▷ ▷ ▷ -iso_types "0" \
21585 ▷ ▷ ▷ -path "SPREAD_TABLES_2/" \
21586 ▷ ▷ ▷ -control Control \
21587 ▷ ▷ ▷ -end \
21588 ▷ ▷ -define PC -packing_classify P3 P5 T \
21589 ▷ ▷ -with PC -do -packing_classify_activity \
21590 ▷ ▷ ▷ -make_graph_of_disjoint_spreads \
21591 ▷ ▷ -end
21592 ▷ cp SPREAD_TABLES_2/spread_4_disjoint.graph .
21593 ▷ $(ORBITER) -v 5 \
21594 ▷ ▷ -define Control -poset_classification_control \
21595 ▷ ▷ ▷ -problem_label packings_PG_3_2 \
21596 ▷ ▷ ▷ -W \
21597 ▷ ▷ -end \
21598 ▷ ▷ -define F -finite_field -q 2 -end \
21599 ▷ ▷ -define P3 -projective_space -n 3 -field F -v 0 -end \
21600 ▷ ▷ -define P5 -projective_space -n 5 -field F -v 0 -end \
21601 ▷ ▷ -define T -spread_table -space P3 -rk 2 \
21602 ▷ ▷ ▷ -iso_types "0" \
21603 ▷ ▷ ▷ -path "SPREAD_TABLES_2/" \
21604 ▷ ▷ ▷ -control Control \

```

```

21605 ▷ ▷ ▷ -end \
21606 ▷ ▷ -define PC -packing_classify P3 P5 T \
21607 ▷ ▷ -with PC -do -packing_classify_activity \
21608 ▷ ▷ ▷ -export_group_on_spreads \
21609 ▷ ▷ -end \
21610 ▷ ▷ -define G -modified_group \
21611 ▷ ▷ ▷ -import \
21612 ▷ ▷ -end \
21613 ▷ ▷ -define Orb -orbits -group G \
21614 ▷ ▷ ▷ -on_points \
21615 ▷ ▷ -end \
21616 ▷ ▷ -define Gamma -graph \
21617 ▷ ▷ ▷ -load \
21618 ▷ ▷ ▷ spread_4_disjoint.graph \
21619 ▷ ▷ -end \
21620 ▷ ▷ -with Gamma -do \
21621 ▷ ▷ ▷ -graph_theoretic_activity \
21622 ▷ ▷ ▷ -test_automorphism_property_of_group G \
21623 ▷ ▷ -end \
21624
21625 Petersen_graph_distance_poset:
21626 ▷ $(ORBITER) -v 2 \
21627 ▷ ▷ -define Gamma -graph -Johnson 5 2 0 -end \
21628 ▷ ▷ -with Gamma -do \
21629 ▷ ▷ ▷ -graph_theoretic_activity -create_distance_poset 0 \
21630 ▷ ▷ -end
21631
21632 Petersen_graph_distance_poset_draw:
21633 ▷ $(ORBITER) -v 3 \
21634 ▷ ▷ -draw_layered_graph \
21635 ▷ ▷ ▷ Johnson_5_2_0_distance_poset.layered_graph \
21636 ▷ ▷ ▷ -place \
21637 ▷ ▷ ▷ -radius 250 -embedded \
21638 ▷ ▷ ▷ -nodes \
21639 ▷ ▷ ▷ -line_width 1.1 -x_stretch 1.4 -scale 0.25 \
21640 ▷ ▷ -end
21641 ▷ $(PDFLATEX) Johnson_5_2_0_distance_poset_draw.tex
21642 ▷ $(OPEN) Johnson_5_2_0_distance_poset_draw.pdf
21643
21644
21645 #Johnson_5_2_0_distance_poset.layered_graph
21646
21647 Petersen_test_if_distance_regular:
21648 ▷ $(ORBITER) -v 2 \
21649 ▷ ▷ -define Gamma -graph -Johnson 5 2 0 -end \
21650 ▷ ▷ -test_if_distance_regular_graph \
21651 ▷ ▷ Johnson_5_2_0_distance_poset.layered_graph
21652
21653
21654
21655 Cayley_Sym5_distance_poset:
21656 ▷ $(ORBITER) -v 2 \
21657 ▷ ▷ -define Do -draw_options -xin 1000000 -yin 1000000 \
21658 ▷ ▷ ▷ -embedded -radius 10000 -nodes_empty -end \
21659 ▷ ▷ -define S -vector -dense \
21660 ▷ ▷ ▷ "1,0,2,3,4, 0,2,1,3,4, 0,1,3,2,4, 0,1,2,4,3" -end \
21661 ▷ ▷ -define G -permutation_group -symmetric_group 5 \
21662 ▷ ▷ -end \
21663 ▷ ▷ -define Gamma -graph \

```



```

21664 ▷ ▷ ▷ -Cayley_graph G S \
21665 ▷ ▷ -end \
21666 ▷ ▷ -with Gamma -do \
21667 ▷ ▷ ▷ -graph_theoretic_activity -create_distance_poset 0 \
21668 ▷ ▷ -end
21669 ▷ $(ORBITER) -v 3 \
21670 ▷ ▷ -draw_layered_graph \
21671 ▷ ▷ ▷ Cayley_graph_Sym_5_distance_poset.layered_graph \
21672 ▷ ▷ ▷ -place \
21673 ▷ ▷ ▷ -radius 250 -embedded \
21674 ▷ ▷ ▷ -nodes \
21675 ▷ ▷ ▷ -line_width 0.4 -x_stretch 1.4 -scale 0.25 \
21676 ▷ ▷ -end
21677 ▷ $(PDFLATEX) Cayley_graph_Sym_5_distance_poset_draw.tex
21678 ▷ $(OPEN) Cayley_graph_Sym_5_distance_poset_draw.pdf
21679
21680
21681 #####
21682 # Section 13.3: Graph Theory: Classification
21683
21684
21685 SECTION_GRAPH_THEORY_CLASSIFICATION:
21686
21687
21688 test_13.3:
21689 ▷ make graph_classify_5
21690 ▷ make tournament_classify_4
21691 ▷ make graph_classify_8_r3
21692 ▷ make Symmetric_4_inversion_graph_recognize
21693 ▷ make Symmetric_5_inversion_graph_recognize
21694
21695
21696
21697 graph_classify_5:
21698 ▷ $(ORBITER) -v 2 \
21699 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
21700 ▷ ▷ -define Do -draw_options \
21701 ▷ ▷ ▷ -radius 300 -nodes_empty \
21702 ▷ ▷ ▷ -line_width 1.5 \
21703 ▷ ▷ ▷ -scale 0.1 \
21704 ▷ ▷ -end \
21705 ▷ ▷ -define GC -graph_classification \
21706 ▷ ▷ ▷ -n 5 \
21707 ▷ ▷ ▷ -poset_classification_control \
21708 ▷ ▷ ▷ -problem_label graphs.v5 \
21709 ▷ ▷ ▷ -depth 10 \
21710 ▷ ▷ ▷ -draw_options Do \
21711 ▷ ▷ ▷ -end \
21712 ▷ ▷ -end \
21713 ▷ ▷ -with GC -do \
21714 ▷ ▷ -graph_classification_activity \
21715 ▷ ▷ ▷ -list_graphs_at_level 5 5 \
21716 ▷ ▷ -end \
21717 ▷ ▷ -with GC -do \
21718 ▷ ▷ -graph_classification_activity \
21719 ▷ ▷ ▷ -draw_options Do \
21720 ▷ ▷ ▷ -draw_graphs_at_level 5 \
21721 ▷ ▷ -end \
21722 ▷ ▷ -print_symbols

```

```

21723 ▷ $(PDFLATEX) graphs_v5_level_5.reps.tex
21724 ▷ $(OPEN) graphs_v5_level_5.reps.pdf
21725 ▷ #$(PDFLATEX) graphs_v5_poset.tex
21726 ▷ #$(OPEN) graphs_v5_poset.pdf
21727
21728
21729 tournament_classify_4:
21730 ▷ $(ORBITER) -v 2 \
21731 ▷ ▷ -define Do -draw_options \
21732 ▷ ▷ ▷ -radius 400 \
21733 ▷ ▷ ▷ -line_width 2 -scale 0.10 \
21734 ▷ ▷ -end \
21735 ▷ ▷ -define GC -graph_classification \
21736 ▷ ▷ ▷ -n 4 -tournament \
21737 ▷ ▷ ▷ -poset_classification_control \
21738 ▷ ▷ ▷ ▷ -problem_label tournament_4 \
21739 ▷ ▷ ▷ ▷ -depth 6 \
21740 ▷ ▷ ▷ ▷ -draw_options Do \
21741 ▷ ▷ ▷ -end \
21742 ▷ ▷ -end \
21743 ▷ ▷ -with GC -do \
21744 ▷ ▷ -graph_classification_activity \
21745 ▷ ▷ ▷ -draw_options Do \
21746 ▷ ▷ ▷ -draw_graphs_at_level 6 \
21747 ▷ ▷ -end \
21748 ▷ ▷ -print_symbols
21749 ▷ $(PDFLATEX) tournament_4_level_6.reps.tex
21750 ▷ $(OPEN) tournament_4_level_6.reps.pdf
21751 ▷
21752 # error after
21753 #do_orbiter_session after execute_command_line
21754
21755
21756
21757
21758 graph_classify_8_r3:
21759 ▷ $(ORBITER) -v 3 \
21760 ▷ ▷ -define Do -draw_options \
21761 ▷ ▷ ▷ -radius 400 \
21762 ▷ ▷ ▷ -line_width 2 -scale 0.10 \
21763 ▷ ▷ -end \
21764 ▷ ▷ -define GC -graph_classification \
21765 ▷ ▷ ▷ -n 8 -regular 3 \
21766 ▷ ▷ ▷ -poset_classification_control \
21767 ▷ ▷ ▷ ▷ -problem_label graphs_v8_r3 \
21768 ▷ ▷ ▷ ▷ -depth 12 \
21769 ▷ ▷ ▷ ▷ -draw_options Do \
21770 ▷ ▷ ▷ -end \
21771 ▷ ▷ -end \
21772 ▷ ▷ -with GC -do \
21773 ▷ ▷ -graph_classification_activity \
21774 ▷ ▷ ▷ -draw_options Do \
21775 ▷ ▷ ▷ -draw_graphs_at_level 12 \
21776 ▷ ▷ -end \
21777 ▷ ▷ -print_symbols
21778 ▷ #$(PDFLATEX) graphs_v8_r3_poset_lvl12.tex
21779 ▷ #$(OPEN) graphs_v8_r3_poset_lvl12.pdf
21780
21781

```

```

21782
21783
21784 Symmetric_4_inversion_graph_recognize:
21785 ▷ $(ORBITER) -v 10 \
21786 ▷ ▷ -define G -permutation_group -symmetric_group 4 -end \
21787 ▷ ▷ -with G -do \
21788 ▷ ▷ -group_theoretic_activity \
21789 ▷ ▷ ▷ -export_inversion_graphs "Symmetric4_inversion_graphs.csv" \
21790 ▷ ▷ -end
21791 ▷ $(ORBITER) -v 2 \
21792 ▷ ▷ -define Do -draw_options \
21793 ▷ ▷ ▷ -radius 250 -embedded \
21794 ▷ ▷ -end \
21795 ▷ ▷ -define GC -graph_classification \
21796 ▷ ▷ ▷ -n 4 \
21797 ▷ ▷ ▷ -poset_classification_control \
21798 ▷ ▷ ▷ ▷ -draw_options Do \
21799 ▷ ▷ ▷ -end \
21800 ▷ ▷ -end \
21801 ▷ ▷ -with GC -do \
21802 ▷ ▷ -graph_classification_activity \
21803 ▷ ▷ ▷ -recognize_graphs_from_adjacency_matrix_csv \
21804 ▷ ▷ ▷ ▷ Symmetric4_inversion_graphs.csv \
21805 ▷ ▷ -end \
21806 ▷ ▷ -print_symbols
21807
21808
21809 Symmetric_5_inversion_graph_recognize:
21810 ▷ $(ORBITER) -v 10 \
21811 ▷ ▷ -define G -permutation_group -symmetric_group 5 -end \
21812 ▷ ▷ -with G -do \
21813 ▷ ▷ -group_theoretic_activity \
21814 ▷ ▷ ▷ -export_inversion_graphs \
21815 ▷ ▷ ▷ ▷ "Symmetric5_inversion_graphs.csv" \
21816 ▷ ▷ -end
21817 ▷ $(ORBITER) -v 2 \
21818 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
21819 ▷ ▷ -define Do -draw_options \
21820 ▷ ▷ ▷ -radius 250 -embedded \
21821 ▷ ▷ -end \
21822 ▷ ▷ -define GC -graph_classification \
21823 ▷ ▷ ▷ -n 5 \
21824 ▷ ▷ ▷ -poset_classification_control \
21825 ▷ ▷ ▷ ▷ -problem_label graphs_v5 \
21826 ▷ ▷ ▷ -depth 10 \
21827 ▷ ▷ ▷ ▷ -draw_options Do \
21828 ▷ ▷ ▷ -end \
21829 ▷ ▷ -end \
21830 ▷ ▷ -with GC -do \
21831 ▷ ▷ -graph_classification_activity \
21832 ▷ ▷ ▷ -recognize_graphs_from_adjacency_matrix_csv \
21833 ▷ ▷ ▷ ▷ Symmetric5_inversion_graphs.csv \
21834 ▷ ▷ -end \
21835 ▷ ▷ -print_symbols
21836 ▷ #$(PDFLATEX) graphs_v5_poset.tex
21837 ▷ #$(OPEN) graphs_v5_poset.pdf
21838
21839
21840

```

```

21841
21842 #####
21843 # Chapter 14 - Design Theory
21844 #####
21845
21846
21847
21848 test_14:
21849 ▷ make test_14_1
21850 ▷ make test_14_2
21851 ▷ make test_14_3
21852 ▷ make test_14_4
21853
21854
21855
21856
21857 #####
21858 # Section 14.1: Design Theory
21859
21860 SECTION_DESIGN_THEORY:
21861
21862
21863 test_14_1:
21864 ▷ make design_PG_2_2
21865 ▷ make design_PG_2_3
21866 ▷ make design_PG_2_4
21867 ▷ make design_elliptic_semiplane_Baker1978
21868 ▷ make wreath_product_designs_n4_k2_inc.txt
21869 ▷ make wreath_product_designs_n8_k6_inc.txt
21870 ▷ make design_20
21871 ▷ make design_600
21872 ▷ make LSQ_5A
21873 ▷ make LSQ_5B
21874 ▷ make LSQ_5_c
21875
21876
21877 design_PG_2_2:
21878 ▷ $(ORBITER) -v 8 \
21879 ▷ ▷ -define F -finite_field -q 2 -end \
21880 ▷ ▷ -define D -design -field F -family PG_2_q -end \
21881 ▷ ▷ -with D -do \
21882 ▷ ▷ ▷ -design_activity \
21883 ▷ ▷ ▷ ▷ -export_flags \
21884 ▷ ▷ -end \
21885 ▷ ▷ -with D -do \
21886 ▷ ▷ ▷ -design_activity \
21887 ▷ ▷ ▷ ▷ -export_blocks \
21888 ▷ ▷ -end
21889 ▷ $(ORBITER) -v 8 \
21890 ▷ ▷ -csv_file_latex 1 PG_2_q2_blocks.csv
21891 ▷ $(PDFLATEX) PG_2_q2_blocks.tex
21892 ▷ $(OPEN) PG_2_q2_blocks.pdf
21893
21894
21895 design_PG_2_3:
21896 ▷ $(ORBITER) -v 8 \
21897 ▷ ▷ -define F -finite_field -q 3 -end \
21898 ▷ ▷ -define D -design -field F -family PG_2_q -end \
21899 ▷ ▷ -with D -do \

```

```

21900 ▷ ▷ ▷ -design_activity \
21901 ▷ ▷ ▷ ▷ -export_flags \
21902 ▷ ▷ -end \
21903 ▷ ▷ -with D -do \
21904 ▷ ▷ ▷ -design_activity \
21905 ▷ ▷ ▷ ▷ -tactical_decomposition \
21906 ▷ ▷ -end
21907
21908 # writes PG_2.q3.inc.txt
21909
21910 design_PG_2_4:
21911 ▷ $(ORBITER) -v 8 \
21912 ▷ ▷ -define F -finite_field -q 4 -end \
21913 ▷ ▷ -define D -design -field F -family PG_2.q -end \
21914 ▷ ▷ -with D -do \
21915 ▷ ▷ ▷ -design_activity \
21916 ▷ ▷ ▷ ▷ -export_flags \
21917 ▷ ▷ -end \
21918 ▷ ▷ -with D -do \
21919 ▷ ▷ ▷ -design_activity \
21920 ▷ ▷ ▷ ▷ -export_blocks \
21921 ▷ ▷ -end
21922
21923
21924 design_elliptic_semiplane_Baker1978:
21925 ▷ $(ORBITER) -v 8 \
21926 ▷ ▷ -define D -design -sporadic Baker_1978 -end \
21927 ▷ ▷ -with D -do \
21928 ▷ ▷ ▷ -design_activity \
21929 ▷ ▷ ▷ ▷ -export_flags \
21930 ▷ ▷ -end \
21931 ▷ ▷ -with D -do \
21932 ▷ ▷ ▷ -design_activity \
21933 ▷ ▷ ▷ ▷ -export_blocks \
21934 ▷ ▷ -end
21935
21936
21937 #Baker1978_flags.txt
21938 #Baker1978_blocks.csv
21939
21940
21941 wreath_product_designs_n4_k2.inc.txt:
21942 ▷ $(ORBITER) -v 8 \
21943 ▷ ▷ -define D -design -wreath_product_designs 4 2 -end \
21944 ▷ ▷ -with D -do \
21945 ▷ ▷ ▷ -design_activity \
21946 ▷ ▷ ▷ ▷ -export_flags \
21947 ▷ ▷ -end
21948
21949
21950
21951 wreath_product_designs_n8_k6.inc.txt:
21952 ▷ $(ORBITER) -v 8 \
21953 ▷ ▷ -define D -design -wreath_product_designs 8 6 -end \
21954 ▷ ▷ -with D -do \
21955 ▷ ▷ ▷ -design_activity \
21956 ▷ ▷ ▷ ▷ -export_flags \
21957 ▷ ▷ -end
21958

```

```

21959
21960 # wreath_product_designs_n8_k6_inc.txt
21961 # The design has 16 points and 3920 blocks of size 6.
21962
21963
21964 design_20:
21965 ▷ $(ORBITER) -v 8 \
21966 ▷ ▷ -define D -design -list_of_blocks_coded 15 3 \
21967 ▷ ▷ "61, 67, 72, 76, 129, 147, 152, 156, 197, 204, 215, 224, 249, 261, 267, 276,
21968 ▷ ▷ 296, 303, 309, 319" \
21969 ▷ ▷ -end \
21970 ▷ ▷ -with D -do \
21971 ▷ ▷ ▷ -design_activity \
21972 ▷ ▷ ▷ -export_flags \
21973 ▷ ▷ -end
21974
21975 design_600:
21976 ▷ echo $(GEO_BLOCKS_600) | sed 's/ //' >geo_600.blocks.csv
21977 ▷ $(ORBITER) -v 8 \
21978 ▷ ▷ -define D -design -list_of_blocks_from_file \
21979 ▷ ▷ 15 geo_600.blocks.csv Block \
21980 ▷ ▷ -end \
21981 ▷ ▷ -with D -do \
21982 ▷ ▷ ▷ -design_activity \
21983 ▷ ▷ ▷ -export_flags \
21984 ▷ ▷ -end \
21985 ▷ ▷ -with D -do \
21986 ▷ ▷ ▷ -design_activity \
21987 ▷ ▷ ▷ -save \
21988 ▷ ▷ ▷ -intersection_matrix \
21989 ▷ ▷ -end
21990
21991
21992 # 25 blocks of size 3
21993
21994
21995
21996 # see this for why we need to use sed:
21997 #https://stackoverflow.com/questions/2003536/bash-why-is-echo-adding-extra-space
21998
21999 LSQ_5A:
22000 ▷ $(ORBITER) -v 8 \
22001 ▷ ▷ -define LSQ5A -vector -format 5 -dense $(LSQ_5A.TABLE) -end \
22002 ▷ ▷ -define D -design -linear_space_from_latin_square \
22003 ▷ ▷ ▷ LSQ5A \
22004 ▷ ▷ -end \
22005 ▷ ▷ -with D -do \
22006 ▷ ▷ ▷ -design_activity \
22007 ▷ ▷ ▷ -export_flags \
22008 ▷ ▷ -end
22009
22010
22011 #latin_square_order5_LSQ5A_flags.txt
22012
22013 LSQ_5B:
22014 ▷ $(ORBITER) -v 8 \
22015 ▷ ▷ -define LSQ5B -vector -format 5 -dense $(LSQ_5B.TABLE) -end \
22016 ▷ ▷ -define D -design -linear_space_from_latin_square \

```

```

22017 ▷ ▷ ▷ LSQ5B \
22018 ▷ ▷ -end \
22019 ▷ ▷ -with D -do \
22020 ▷ ▷ ▷ -design_activity \
22021 ▷ ▷ ▷ ▷ -export_flags \
22022 ▷ ▷ -end
22023
22024 #latin_square_order5_LSQ5B_flags.txt
22025
22026
22027 LSQ_5.c:
22028 ▷ $(ORBITER) -v 3 \
22029 ▷ ▷ -define Doinc -draw_incidence_structure_options \
22030 ▷ ▷ ▷ -width 60 -width_10 6 -end \
22031 ▷ ▷ -define C -combinatorial_object \
22032 ▷ ▷ ▷ -label LSQ_5 LSQ\5 \
22033 ▷ ▷ ▷ -file_of_incidence_geometries latin_square_order5_LSQ5A_flags.txt 15 28 90
22034 ▷ ▷ ▷ -file_of_incidence_geometries latin_square_order5_LSQ5B_flags.txt 15 28 90
22035 ▷ ▷ -end \
22036 ▷ ▷ -with C -do \
22037 ▷ ▷ -combinatorial_object_activity \
22038 ▷ ▷ ▷ -canonical_form \
22039 ▷ ▷ ▷ ▷ -save_ago \
22040 ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
22041 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "LSQ5_nauty_log.txt" -end \
22042 ▷ ▷ ▷ -end \
22043 ▷ ▷ -end \
22044 ▷ ▷ -with C -do \
22045 ▷ ▷ -combinatorial_object_activity \
22046 ▷ ▷ ▷ -post_processing \
22047 ▷ ▷ -end \
22048 ▷ ▷ -with C -do \
22049 ▷ ▷ -combinatorial_object_activity \
22050 ▷ ▷ ▷ -report \
22051 ▷ ▷ ▷ ▷ -export_flag_orbits \
22052 ▷ ▷ ▷ ▷ -show_TDO \
22053 ▷ ▷ ▷ ▷ -show_TDA \
22054 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
22055 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
22056 ▷ ▷ ▷ -end \
22057 ▷ ▷ -end
22058 ▷ $(PDFLATEX) LSQ_5_classification.tex
22059 ▷ $(OPEN) LSQ_5_classification.pdf
22060
22061
22062
22063
22064
22065 #####
22066 # Section 14.2: Assuming Symmetry
22067
22068 SECTION_ASSUMING_SYMMETRY:
22069
22070 test_14.2:
22071 ▷ make KM_cyclic_7
22072 ▷ make KM_PGGL_2_32
22073 ▷ #make KM_PGGL_2_32.solve # too slow

```

```

22074 ▷ #make KM.PSL_3.5 # too slow
22075 ▷ make C5_KM_2_3
22076 ▷ make C5_KM_1_3
22077 ▷ make C5_concatenate
22078 ▷ make C5_solve
22079 ▷ make C5_design_classify
22080 ▷ make Witt_design_on_22_points
22081
22082
22083
22084
22085
22086
22087 KM_cyclic_7:
22088 ▷ $(ORBITER) -v 3 \
22089 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
22090 ▷ ▷ -define gens -vector -dense "1,2,3,4,5,6,0" -end \
22091 ▷ ▷ -define G -permutation_group -symmetric_group 7 \
22092 ▷ ▷ ▷ -subgroup_by_generators "C7" 7 1 gens \
22093 ▷ ▷ -end \
22094 ▷ ▷ -define Do -draw_options -embedded -sideways -radius 50 \
22095 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 \
22096 ▷ ▷ -end \
22097 ▷ ▷ -define Control -poset_classification_control \
22098 ▷ ▷ ▷ -problem_label C7 -W -depth 3 \
22099 ▷ ▷ ▷ -draw_options Do \
22100 ▷ ▷ -end \
22101 ▷ ▷ -define Orb -orbits \
22102 ▷ ▷ ▷ -group G \
22103 ▷ ▷ ▷ -group_action G \
22104 ▷ ▷ ▷ -generators G \
22105 ▷ ▷ ▷ -on_subsets 3 Control \
22106 ▷ ▷ -end \
22107 ▷ ▷ -with Orb -do -orbits_activity \
22108 ▷ ▷ ▷ -Kramer_Mesner_matrix 2 3 \
22109 ▷ ▷ -end \
22110 ▷ ▷ -define ro \
22111 ▷ ▷ ▷ -poset_classification_report_options \
22112 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
22113 ▷ ▷ ▷ ▷ -draw_options Do \
22114 ▷ ▷ -end \
22115 ▷ ▷ -with Orb -do -orbits_activity \
22116 ▷ ▷ ▷ -report "" \
22117 ▷ ▷ ▷ -report_options ro \
22118 ▷ ▷ -end
22119 ▷ $(ORBITER) -v 4 \
22120 ▷ ▷ -define A -vector -file C7_KM_2.3.csv -end \
22121 ▷ ▷ -define D -diophant \
22122 ▷ ▷ -label "C7_KM_2.3_system" \
22123 ▷ ▷ -coefficient_matrix A \
22124 ▷ ▷ -RHS_constant "EQ=1" \
22125 ▷ ▷ -x_min_global 0 -x_max_global 1 \
22126 ▷ ▷ -end \
22127 ▷ ▷ -with D -do \
22128 ▷ ▷ ▷ -diophant_activity -solve_mckay \
22129 ▷ ▷ -end
22130
22131
22132 # to create simple 7-designs on 33 points with block size 8 and lambda = 10 invar

```



```

        iant under PGGL(2,32):
22133
22134 KM_PGGL_2_32:
22135 ▷ $(ORBITER) -v 3 \
22136 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
22137 ▷ ▷ -define Do -draw_options -embedded -sideways -radius 50 \
22138 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 \
22139 ▷ ▷ -end \
22140 ▷ ▷ -define Control -poset_classification_control \
22141 ▷ ▷ ▷ -problem_label KM_PGGL_2_32 \
22142 ▷ ▷ ▷ -W -depth 8 \
22143 ▷ ▷ ▷ -draw_options Do \
22144 ▷ ▷ -end \
22145 ▷ ▷ -define G -linear_group -PGGL 2 32 -end \
22146 ▷ ▷ -define Orb -orbits \
22147 ▷ ▷ ▷ -group G \
22148 ▷ ▷ ▷ -group_action G \
22149 ▷ ▷ ▷ -generators G \
22150 ▷ ▷ ▷ -on_subsets 8 Control \
22151 ▷ ▷ -end \
22152 ▷ ▷ -with Orb -do -orbits_activity \
22153 ▷ ▷ ▷ -Kramer_Mesner_matrix 7 8 \
22154 ▷ ▷ -end \
22155 ▷ ▷ -define ro \
22156 ▷ ▷ ▷ -poset_classification_report_options \
22157 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
22158 ▷ ▷ ▷ ▷ -draw_options Do \
22159 ▷ ▷ -end \
22160 ▷ ▷ -with Orb -do -orbits_activity \
22161 ▷ ▷ ▷ -report "" \
22162 ▷ ▷ ▷ -report_options ro \
22163 ▷ ▷ -end
22164 ▷ $(ORBITER) -v 2 -draw_matrix \
22165 ▷ ▷ -input_csv_file KM_PGGL_2_32_KM_7_8.csv \
22166 ▷ ▷ -box_width 20 -bit_depth 24 \
22167 ▷ ▷ -partition 3 32 97 -end
22168 ▷ $(PDFLATEX) KM_PGGL_2_32_poset_lvl8.tex
22169 ▷ $(OPEN) KM_PGGL_2_32_poset_lvl8.pdf
22170 ▷ $(OPEN) KM_PGGL_2_32_KM_7_8_draw.bmp
22171
22172
22173 KM_PGGL_2_32_solve:
22174 ▷ $(ORBITER) -v 4 \
22175 ▷ ▷ -define A -vector -file KM_PGGL_2_32_KM_7_8.csv -end \
22176 ▷ ▷ -define D -diophant \
22177 ▷ ▷ -label "KM_PGGL_2_32_KM_7_8.system" \
22178 ▷ ▷ -coefficient_matrix A \
22179 ▷ ▷ -RHS_constant "EQ=10" \
22180 ▷ ▷ -x_min_global 0 -x_max_global 1 \
22181 ▷ ▷ -end \
22182 ▷ ▷ -with D -do \
22183 ▷ ▷ ▷ -diophant_activity -solve_mckay \
22184 ▷ ▷ -end
22185
22186 # this takes a long time
22187
22188
22189
22190

```

```

22191 KM.PSL_3.5:
22192 ▷ $(ORBITER) -v 3 \
22193 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
22194 ▷ ▷ -define Do -draw_options -embedded -sideways \
22195 ▷ ▷ ▷ -radius 50 -scale 0.5 -line_width 0.3 \
22196 ▷ ▷ -end \
22197 ▷ ▷ -define Control -poset_classification_control \
22198 ▷ ▷ ▷ -problem_label KM.PSL_3.5 \
22199 ▷ ▷ ▷ -W -depth 10 \
22200 ▷ ▷ ▷ -draw_options Do \
22201 ▷ ▷ -end \
22202 ▷ ▷ -define G -linear_group -PSL 3 5 -end \
22203 ▷ ▷ -define Orb -orbits \
22204 ▷ ▷ ▷ -group G \
22205 ▷ ▷ ▷ -group_action G \
22206 ▷ ▷ ▷ -generators G \
22207 ▷ ▷ ▷ -on_subsets 10 Control \
22208 ▷ ▷ -end \
22209 ▷ ▷ -with Orb -do -orbits_activity \
22210 ▷ ▷ ▷ -Kramer_Mesner_matrix 8 10 \
22211 ▷ ▷ -end \
22212 ▷ ▷ -define ro \
22213 ▷ ▷ ▷ -poset_classification_report_options \
22214 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
22215 ▷ ▷ ▷ ▷ -draw_options Do \
22216 ▷ ▷ -end \
22217 ▷ ▷ -with Orb -do -orbits_activity \
22218 ▷ ▷ ▷ -report "" \
22219 ▷ ▷ ▷ -report_options ro \
22220 ▷ ▷ -end
22221 ▷ $(ORBITER) -v 2 -draw_matrix \
22222 ▷ ▷ -input_csv_file KM.PSL_3.5.KM.8.10.csv \
22223 ▷ ▷ -box_width 10 -bit_depth 8 -partition 3 42 174 -end
22224 ▷ $(ORBITER) -v 4 \
22225 ▷ ▷ -define A -vector -file KM.PSL_3.5.KM.8.10.csv -end \
22226 ▷ ▷ -define D -diophant \
22227 ▷ ▷ ▷ -label "KM.PSL_3.5.KM.8.10.system" \
22228 ▷ ▷ ▷ -coefficient_matrix A \
22229 ▷ ▷ ▷ -RHS_constant "EQ=93" \
22230 ▷ ▷ ▷ -x_min_global 0 -x_max_global 1 \
22231 ▷ ▷ -end \
22232 ▷ ▷ -with D -do \
22233 ▷ ▷ ▷ -diophant_activity -solve_mckay \
22234 ▷ ▷ -end
22235
22236 # this takes a long time
22237
22238
22239
22240
22241 C5.KM.2.3:
22242 ▷ $(ORBITER) -v 3 \
22243 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
22244 ▷ ▷ -define gens -vector -dense $(GEN.C5) -end \
22245 ▷ ▷ -define G -permutation_group -symmetric_group 15 \
22246 ▷ ▷ ▷ -subgroup_by_generators "C5" 5 1 gens \
22247 ▷ ▷ -end \
22248 ▷ ▷ -define Do -draw_options -embedded -sideways -radius 50 \
22249 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 \

```

```

22250 ▷ ▷ -end \
22251 ▷ ▷ -define Control -poset_classification_control \
22252 ▷ ▷ ▷ -problem_label C5 -W -depth 3 \
22253 ▷ ▷ ▷ -draw_options Do \
22254 ▷ ▷ -end \
22255 ▷ ▷ -define Orb -orbits \
22256 ▷ ▷ ▷ -group G \
22257 ▷ ▷ ▷ -group_action G \
22258 ▷ ▷ ▷ -generators G \
22259 ▷ ▷ ▷ -on_subsets 3 Control \
22260 ▷ ▷ -end \
22261 ▷ ▷ -with Orb -do -orbits_activity \
22262 ▷ ▷ ▷ -Kramer_Mesner_matrix 2 3 \
22263 ▷ ▷ -end
22264
22265 #C5_KM_2.3.csv
22266
22267
22268 C5_KM_1.3:
22269 ▷ $(ORBITER) -v 3 \
22270 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
22271 ▷ ▷ -define gens -vector -dense $(GEN_C5) -end \
22272 ▷ ▷ -define G -permutation_group -symmetric_group 15 \
22273 ▷ ▷ ▷ -subgroup_by_generators "C5" 5 1 gens \
22274 ▷ ▷ -end \
22275 ▷ ▷ -define Do -draw_options -embedded -sideways -radius 50 \
22276 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 \
22277 ▷ ▷ -end \
22278 ▷ ▷ -define Control -poset_classification_control \
22279 ▷ ▷ ▷ -problem_label C5 -W -depth 3 \
22280 ▷ ▷ ▷ -draw_options Do \
22281 ▷ ▷ -end \
22282 ▷ ▷ -define Orb -orbits \
22283 ▷ ▷ ▷ -group G \
22284 ▷ ▷ ▷ -group_action G \
22285 ▷ ▷ ▷ -generators G \
22286 ▷ ▷ ▷ -on_subsets 3 Control \
22287 ▷ ▷ -end \
22288 ▷ ▷ -with Orb -do -orbits_activity \
22289 ▷ ▷ ▷ -Kramer_Mesner_matrix 1 3 \
22290 ▷ ▷ -end \
22291 ▷ ▷ -with Orb -do -orbits_activity \
22292 ▷ ▷ ▷ -export_something set_orbits 3 \
22293 ▷ ▷ -end \
22294
22295
22296 # 91 orbits at level 3
22297
22298 #C5_KM_1.3.csv
22299 #C5_set_orbits_level.3.csv
22300
22301
22302 C5_concatenate:
22303 ▷ $(ORBITER) -v 3 \
22304 ▷ ▷ -csv_file_concatenate C5_KM_combined.csv 2 C5_KM_1.3.csv C5_KM_2.3.csv
22305
22306 #C5_KM_combined.csv
22307 # in the combined system, there are three rows
22308 # from the first system and 21 rows from the second system

```

```

22309
22310
22311 C5.solve:
22312 ▷ $(ORBITER) -v 4 \
22313 ▷ ▷ -define A -vector -file C5_KM_combined.csv -end \
22314 ▷ ▷ -define D -diophant \
22315 ▷ ▷ ▷ -label "C5_KM_combined_system" \
22316 ▷ ▷ ▷ -coefficient_matrix A \
22317 ▷ ▷ ▷ -x_min_global 0 \
22318 ▷ ▷ ▷ -x_max_global 1 \
22319 ▷ ▷ ▷ -RHS "mult=3,EQ=5" \
22320 ▷ ▷ ▷ -RHS "mult=21,LE=1" \
22321 ▷ ▷ -end \
22322 ▷ ▷ -with D -do \
22323 ▷ ▷ ▷ -diophant_activity -solve_mckay \
22324 ▷ ▷ -end
22325
22326 #diophant::solve_all_mckay found 21540 solutions in 61001 backtrack nodes
22327 #Found 21540 solutions with 61001 backtrack steps
22328
22329 #C5_KM_combined_system_sol.csv
22330
22331
22332 C5.design_classify:
22333 ▷ $(ORBITER) -v 2 \
22334 ▷ ▷ -define C -combinatorial_object \
22335 ▷ ▷ ▷ -label C5_config C5\_config \
22336 ▷ ▷ ▷ -file_of_designs_through_block_orbits \
22337 ▷ ▷ ▷ ▷ C5_KM_combined_system_sol.csv \
22338 ▷ ▷ ▷ ▷ C5_set_orbits_level_3.csv \
22339 ▷ ▷ ▷ ▷ 15 3 \
22340 ▷ ▷ -end \
22341 ▷ ▷ -with C -do \
22342 ▷ ▷ -combinatorial_object_activity \
22343 ▷ ▷ ▷ -canonical_form \
22344 ▷ ▷ ▷ ▷ -save_ago \
22345 ▷ ▷ ▷ ▷ -save_transversal \
22346 ▷ ▷ ▷ ▷ -nauty_control -end \
22347 ▷ ▷ ▷ -end \
22348 ▷ ▷ -end \
22349 ▷ ▷ -with C -do \
22350 ▷ ▷ -combinatorial_object_activity \
22351 ▷ ▷ ▷ -post_processing \
22352 ▷ ▷ -end \
22353 ▷ ▷ -define Doinc -draw_incidence_structure_options \
22354 ▷ ▷ ▷ -width 60 -width_10 6 -end \
22355 ▷ ▷ -with C -do \
22356 ▷ ▷ -combinatorial_object_activity \
22357 ▷ ▷ ▷ -report \
22358 ▷ ▷ ▷ ▷ -show_incidence_matrices \
22359 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
22360 ▷ ▷ ▷ -end \
22361 ▷ ▷ -end \
22362 ▷ ▷ -print_symbols
22363 ▷ $(PDFLATEX) C5_config_classification.tex
22364 ▷ $(OPEN) C5_config_classification.pdf
22365
22366
22367 # finds 42 isomorphism types

```

```

22368
22369 # number 0 has group order 600
22370 # it is flag transitive and anti flag transitive
22371 # number 14 has group order 120
22372 # flag orbits: 15+60
22373 # anti flag orbits: 60+120
22374
22375
22376
22377 Witt_design_on_22_points:
22378 ▷ $(ORBITER) -v 2 \
22379 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
22380 ▷ ▷ -define Do -draw_options \
22381 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
22382 ▷ ▷ -end \
22383 ▷ ▷ -define code -vector \
22384 ▷ ▷ ▷ -dense $(GOLAY_24_CODE_PROJECTIVE) \
22385 ▷ ▷ -end \
22386 ▷ ▷ -define Do_embedded -draw_options -embedded -sideways -radius 50 \
22387 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 \
22388 ▷ ▷ -end \
22389 ▷ ▷ -define Control2 -poset_classification_control \
22390 ▷ ▷ ▷ -problem_label M24_pairs -W -depth 2 \
22391 ▷ ▷ ▷ -draw_options Do \
22392 ▷ ▷ -end \
22393 ▷ ▷ -define F -finite_field -q 2 -end \
22394 ▷ ▷ -define G -linear_group -PGL 12 2 \
22395 ▷ ▷ -subgroup_by_generators "M24" "244823040" 12 \
22396 ▷ ▷ ▷ $(GOLAY_24_CODE_AUT_GENES) \
22397 ▷ ▷ -end \
22398 ▷ ▷ -define M24_perm -modified_group \
22399 ▷ ▷ ▷ -from G -restricted_action code \_on\_code \
22400 ▷ ▷ -end \
22401 ▷ ▷ -define M22_perm -modified_group \
22402 ▷ ▷ ▷ -from M24_perm \
22403 ▷ ▷ ▷ -set_stabilizer 0,1 Control2 \
22404 ▷ ▷ -end \
22405 ▷ ▷ -define pts -set -loop 2 22 1 -end \
22406 ▷ ▷ -define M22 -modified_group -from M22_perm \
22407 ▷ ▷ ▷ -restricted_action pts M22 \
22408 ▷ ▷ -end \
22409 ▷ ▷ -with M22 -do \
22410 ▷ ▷ -group_theoretic_activity \
22411 ▷ ▷ ▷ -report Do \
22412 ▷ ▷ -end \
22413 ▷ ▷ -define Control -poset_classification_control \
22414 ▷ ▷ ▷ -problem_label M22 -W -depth 6 \
22415 ▷ ▷ ▷ -draw_options Do \
22416 ▷ ▷ -end \
22417 ▷ ▷ -define Orb -orbits \
22418 ▷ ▷ ▷ -group M22 \
22419 ▷ ▷ ▷ -group_action M22 \
22420 ▷ ▷ ▷ -generators M22 \
22421 ▷ ▷ ▷ -on_subsets 6 Control \
22422 ▷ ▷ -end \
22423 ▷ ▷ -with Orb -do -orbits_activity \
22424 ▷ ▷ ▷ -Kramer_Mesner_matrix 3 6 \
22425 ▷ ▷ -end \
22426 ▷ ▷ -define ro \

```

```

22427 ▷ ▷ ▷ -poset_classification_report_options \
22428 ▷ ▷ ▷ ▷ -draw_poset -type_ordinary \
22429 ▷ ▷ ▷ ▷ -draw_options Do_embedded \
22430 ▷ ▷ -end \
22431 ▷ ▷ -with Orb -do -orbits_activity \
22432 ▷ ▷ ▷ -report "" \
22433 ▷ ▷ ▷ -report_options ro \
22434 ▷ ▷ -end \
22435 ▷ ▷ -define D -design -orbit_on_sets \
22436 ▷ ▷ ▷ Orb 6 1 \
22437 ▷ ▷ -end \
22438 ▷ ▷ -with D -do \
22439 ▷ ▷ ▷ -design_activity \
22440 ▷ ▷ ▷ ▷ -export_incidence_matrix \
22441 ▷ ▷ -end \
22442 ▷ ▷ -with D -do \
22443 ▷ ▷ ▷ -design_activity \
22444 ▷ ▷ ▷ ▷ -intersection_matrix \
22445 ▷ ▷ -end
22446
22447
22448
22449
22450
22451 #####
22452 # Section 14.3: Design Theory - Large Sets
22453
22454 SECTION.DESIGN.THEORY.LARGE.SETS:
22455
22456 test_14.3:
22457 ▷ make AG_2.3.inc
22458 ▷ make LS_AG_2.3.design_table.create
22459 ▷ make AG_2.3.on_designs
22460 ▷ make AG_2.3.stab_orb.0
22461 ▷ make AG_2.3.stab_orb.0.Perm840_res192
22462 ▷ make LS_AG_2.3.disjoint_sets_graph_and_cliques
22463 ▷ make LS_AG_2.3.disjoint_sets_split
22464 ▷ make LS_AG_2.3.export_large_sets
22465 ▷ make design_PG_2.3.table.create # very slow
22466 ▷ make design_PG_2.3.group.5
22467 ▷ #make design_PG_2.3.group.5.sol.0
22468
22469
22470
22471 AG_2.3.inc:
22472 ▷ $(ORBITER) -v 2 \
22473 ▷ ▷ -define Geo -geometry_builder \
22474 ▷ ▷ ▷ -V 9 -B 12 \
22475 ▷ ▷ ▷ -TDO 4 -fuse 1 \
22476 ▷ ▷ ▷ -fname_GEO AG_2.3 \
22477 ▷ ▷ ▷ -test 3,4,5,6,7,8,9 \
22478 ▷ ▷ ▷ -output_to_inc_file \
22479 ▷ ▷ -end
22480
22481 #9 12 3
22482 #0 13 22 27 35 41 47 53 55 59 71 76
22483 #-1 1
22484 #432
22485

```

```

22486
22487
22488
22489
22490 LS_AG_2_3_design_table.create:
22491 ▷ $(ORBITER) -v 50 \
22492 ▷ ▷ -define B -vector -dense $(AG_2_3_BLOCKS) -end \
22493 ▷ ▷ -define D -design -list_of_blocks_coded 9 3 B -end \
22494 ▷ ▷ -define Sym9 -permutation_group -symmetric_group 9 -end \
22495 ▷ ▷ -define T -design_table D "AG_2_3" Sym9 -end
22496
22497 # creates AG_2_3_design_table.csv
22498 # and AG_2_3.makefile
22499
22500 #0,0,13,22,27,35,41,47,53,55,59,71,76
22501 # is the first design in AG_2_3_design_table.csv
22502
22503 #poset_orbit_node::init_root_node storing strong generators for a group of order
      362880
22504 # stabilizer order 432
22505 # 840 designs
22506
22507 #User time: 0.13 of a second, dt=13 tps = 100
22508
22509
22510 AG_2_3_on_designs:
22511 ▷ $(ORBITER) -v 2 \
22512 ▷ ▷ -define gens -vector -file AG_2_3_gens.csv -end \
22513 ▷ ▷ -define G -permutation_group \
22514 ▷ ▷ ▷ -bsgs AG_2_3 "AG_2_3" 840 362880 "0,1,2,3,4,5,6,7" \
22515 ▷ ▷ ▷ 8 gens \
22516 ▷ ▷ -end \
22517 ▷ ▷ -define Orb -orbits -group G \
22518 ▷ ▷ ▷ -on_points \
22519 ▷ ▷ -end \
22520 ▷ ▷ -with Orb -do -orbits_activity \
22521 ▷ ▷ ▷ -stabilizer 0 \
22522 ▷ ▷ -end
22523
22524 #Written file AG_2_3_stab_pt_0.makefile of size 239
22525
22526
22527
22528
22529
22530 # the stabilizer of the first design:
22531
22532
22533
22534
22535 AG_2_3_stab_orb_0:
22536 ▷ $(ORBITER) -v 2 \
22537 ▷ ▷ -define gens -vector -file AG_2_3_stab_pt_0_gens.csv -end \
22538 ▷ ▷ -define Set192 -set -here $(LARGE_SET_AG_2_3_NEIGHBOR_SET) -end \
22539 ▷ ▷ -define G -permutation_group \
22540 ▷ ▷ -bsgs AG_2_3_stab_orb_0 "AG_2_3_stab_pt_0" \
22541 ▷ ▷ ▷ 840 432 "0,1,2,3,4,5,6,7,8" 4 gens \
22542 ▷ ▷ -end \
22543 ▷ ▷ -define Gr -modified_group -from G \

```

```

22544 ▷ ▷ ▷ -restricted_action Set192 Set192 \
22545 ▷ ▷ -end \
22546 ▷ ▷ -with Gr -do \
22547 ▷ ▷ -group_theoretic_activity \
22548 ▷ ▷ ▷ -export_orbiter \
22549 ▷ ▷ -end
22550
22551 #Perm_840_res_Set192_gens.csv
22552
22553 AG_2_3_stab_orb_0.Perm840_res192:
22554 ▷ $(ORBITER) -v 2 \
22555 ▷ ▷ -define Do -draw_options \
22556 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
22557 ▷ ▷ -end \
22558 ▷ ▷ -define gens -vector -file Perm840_res_Set192_gens.csv -end \
22559 ▷ ▷ -define G -permutation_group \
22560 ▷ ▷ -bsgs Perm840_res192 "Perm840 {\rm res192}" \
22561 ▷ ▷ ▷ 192 432 "0,1,2,3,4,5,6,7,8" 4 gens \
22562 ▷ ▷ -end \
22563 ▷ ▷ -with G -do \
22564 ▷ ▷ -group_theoretic_activity \
22565 ▷ ▷ ▷ -report Do \
22566 ▷ ▷ -end
22567 ▷ $(PDFLATEX) Perm840_res192_report.tex
22568 ▷ $(OPEN) Perm840_res192_report.pdf
22569
22570
22571
22572 LS_AG_2_3_disjoint_sets_graph_and_cliques:
22573 ▷ $(ORBITER) -v 2 \
22574 ▷ ▷ -define Gamma -graph \
22575 ▷ ▷ ▷ -disjoint_sets_graph \
22576 ▷ ▷ ▷ AG_2_3_design_table.csv \
22577 ▷ ▷ -end \
22578 ▷ ▷ -with Gamma -do \
22579 ▷ ▷ -graph_theoretic_activity \
22580 ▷ ▷ ▷ -save \
22581 ▷ ▷ -end \
22582 ▷ ▷ -with Gamma -do \
22583 ▷ ▷ -graph_theoretic_activity \
22584 ▷ ▷ ▷ -find_cliques -target_size 7 \
22585 ▷ ▷ ▷ -output_file AG_2_3_design_table_cliques.csv \
22586 ▷ ▷ -end \
22587 ▷ ▷ -print_symbols
22588
22589 #AG_2_3_design_table_cliques.csv
22590
22591 #AG_2_3_design_table_disjoint_sets_colored_graph
22592 #User time: 0.66 of a second, dt=66 tps = 100
22593 #AG_2_3_design_table_disjoint_sets_solutions.csv
22594
22595 #15360 solutions
22596
22597 LS_AG_2_3_disjoint_sets_split:
22598 ▷ $(ORBITER) -v 4 \
22599 ▷ ▷ -define Gamma -graph -load \
22600 ▷ ▷ ▷ AG_2_3_design_table_disjoint_sets_colored_graph \
22601 ▷ ▷ -end \
22602 ▷ ▷ -with Gamma -do \

```



```

22603 ▷ ▷ -graph_theoretic.activity \
22604 ▷ ▷ ▷ -split_by_clique "0" "0" \
22605 ▷ ▷ -end
22606
22607
22608 #AG.2.3_design_table_disjoint_sets_0.graph
22609 #AG.2.3_design_table_disjoint_sets_0_subset.txt
22610
22611
22612
22613 LS.AG.2.3_export_large_sets:
22614 ▷ $(ORBITER) -v 20 \
22615 ▷ ▷ -define B -vector -dense $(AG.2.3_BLOCKS) -end \
22616 ▷ ▷ -define D -design -list_of_blocks_coded 9 3 B -end \
22617 ▷ ▷ -define Sym9 -permutation_group -symmetric_group 9 -end \
22618 ▷ ▷ -define T -design_table D "AG.2.3" Sym9 -end \
22619 ▷ ▷ -with D -do \
22620 ▷ ▷ -design_activity \
22621 ▷ ▷ ▷ -extract_solutions_by_index_csv "AG.2.3" Sym9 \
22622 ▷ ▷ ▷ AG.2.3_design_table_cliques.csv \
22623 ▷ ▷ ▷ "Solution" \
22624 ▷ ▷ ▷ AG.2.3_large_sets.csv \
22625 ▷ ▷ ▷ "" \
22626 ▷ ▷ -end
22627
22628 #AG.2.3_large_sets.csv
22629 # data is in column "large_set"
22630 #User time: 0.39 of a second, dt=39 tps = 100
22631
22632
22633
22634 design.PG.2.3_table_create:
22635 ▷ $(ORBITER) -v 2 \
22636 ▷ ▷ -define F -finite_field -q 3 -end \
22637 ▷ ▷ -define D -design -field F -family PG.2.q -end \
22638 ▷ ▷ -define Sym13 -permutation_group -symmetric_group 13 -end \
22639 ▷ ▷ -define T -design_table D "PG.2.13" Sym13 -end
22640
22641 # written file PG.2.13_design_table.csv
22642 # 1108800 designs
22643 #User time: 7:30
22644
22645 design.PG.2.3_group.5:
22646 ▷ $(ORBITER) -v 2 \
22647 ▷ ▷ -define F -finite_field -q 3 -end \
22648 ▷ ▷ -define D -design -field F -family PG.2.q -end \
22649 ▷ ▷ -define Sym13 -permutation_group -symmetric_group 13 -end \
22650 ▷ ▷ -define T -design_table D "PG.2.13" Sym13 -end \
22651 ▷ ▷ -define LSW -large_set_with_symmetry_assumption T \
22652 ▷ ▷ ▷ -H "5" $(GENERATORS_H5) \
22653 ▷ ▷ ▷ -N "1200" $(GENERATORS_N5) \
22654 ▷ ▷ ▷ -prefix "H5" \
22655 ▷ ▷ ▷ -selected_orbit_length 5 \
22656 ▷ ▷ ▷ -end \
22657 ▷ ▷ -with LSW -do \
22658 ▷ ▷ ▷ -large_set_with_symmetry_assumption_activity \
22659 ▷ ▷ ▷ -normalizer_on_orbits_of_a_given_length 5 \
22660 ▷ ▷ -end
22661

```

```

22662 #H5.N.orbit_reps.csv
22663 # 678 orbits
22664 #User time: 2:39
22665
22666
22667 # needs case_0.sol.txt
22668
22669 design_PG_2_3_group_5_sol_0:
22670 ▷ $(ORBITER) -v 2 \
22671 ▷ ▷ -define F -finite_field -q 3 -end \
22672 ▷ ▷ -define D -design -field F -family PG_2_q -end \
22673 ▷ ▷ -define Sym13 -permutation_group -symmetric_group 13 -end \
22674 ▷ ▷ -define T -design_table D "PG_2_13" Sym13 -end \
22675 ▷ ▷ -define LSW -large_set_with_symmetry_assumption T \
22676 ▷ ▷ ▷ -H "5" $(GENERATORS_H5) \
22677 ▷ ▷ ▷ -N "1200" $(GENERATORS_N5) \
22678 ▷ ▷ ▷ -prefix "H5" \
22679 ▷ ▷ ▷ -selected_orbit_length 5 \
22680 ▷ ▷ ▷ -end \
22681 ▷ ▷ -with LSW -do \
22682 ▷ ▷ ▷ -large_set_with_symmetry_assumption_activity \
22683 ▷ ▷ ▷ -read_solution_file 5 case_0.sol.txt \
22684 ▷ ▷ ▷ -end
22685
22686
22687
22688
22689 #####
22690 # Section 14.4: Design Theory - Delandtsheer Doyen
22691
22692 SECTION_DESIGN_THEORY_DELANDTSHEER_DOYEN:
22693
22694 test_14_4:
22695 ▷ make DD_PP4
22696 ▷ make DD_PP4_span_design
22697 ▷ make DD_PP4_design_orbit
22698 ▷ #make PG_2_27_special
22699
22700
22701 DD_PP4:
22702 ▷ $(ORBITER) -v 6 \
22703 ▷ ▷ -define Do -draw_options -end \
22704 ▷ ▷ -define pair_search_control -poset_classification_control \
22705 ▷ ▷ ▷ -problem_label PP4_pairs -W \
22706 ▷ ▷ ▷ -draw_options Do \
22707 ▷ ▷ -end \
22708 ▷ ▷ -define search_control -poset_classification_control \
22709 ▷ ▷ ▷ -problem_label PP4_search -W \
22710 ▷ ▷ ▷ -draw_options Do \
22711 ▷ ▷ ▷ -depth 5 \
22712 ▷ ▷ -end \
22713 ▷ ▷ -Delandtsheer_Doyen \
22714 ▷ ▷ ▷ -pair_search_control pair_search_control \
22715 ▷ ▷ ▷ -search_control search_control \
22716 ▷ ▷ ▷ $(PP4) $(PP4_GROUP1) $(PP4_MASK1) \
22717 ▷ ▷ ▷ -create_starter \
22718 ▷ ▷ ▷ -end \
22719
22720

```

```

22721 #-group_label
22722
22723 DD_PP4_span_design:
22724 ▷ $(ORBITER) -v 6 \
22725 ▷ ▷ -define G1 -linear_group -AGL 1 3 \
22726 ▷ ▷ ▷ -end \
22727 ▷ ▷ -define G2 -linear_group -AGL 1 7 \
22728 ▷ ▷ ▷ -end \
22729 ▷ ▷ -define G -modified_group -direct_product "G1,G2" \
22730 ▷ ▷ ▷ "21" "1,1,1,1" \
22731 ▷ ▷ -end \
22732 ▷ ▷ -define D -design -list_of_base_blocks \
22733 ▷ ▷ ▷ G \
22734 ▷ ▷ ▷ PP4_no_mask_cyclic21_cases.csv
22735 ▷ ▷ ▷ Starter \
22736 ▷ ▷ -end
22737
22738
22739 #csv_file_support::read_column_as_set_of_sets file PP4_no_mask_cyclic21_cases.csv
      does not exist or is empty
22740
22741
22742 DD_PP4_design_orbit:
22743 ▷ $(ORBITER) -v 4 \
22744 ▷ ▷ -define Do -draw_options \
22745 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
22746 ▷ ▷ -end \
22747 ▷ ▷ -define G1 -linear_group -AGL 1 3 \
22748 ▷ ▷ ▷ -end \
22749 ▷ ▷ -define G2 -linear_group -AGL 1 7 \
22750 ▷ ▷ ▷ -end \
22751 ▷ ▷ -define G -modified_group -direct_product "G1,G2" \
22752 ▷ ▷ ▷ "21" "1,1,1,1" \
22753 ▷ ▷ -end \
22754 ▷ ▷ -with G -do \
22755 ▷ ▷ -group_theoretic_activity \
22756 ▷ ▷ ▷ -report Do \
22757 ▷ ▷ -end \
22758 ▷ ▷ -define D -design -list_of_base_blocks \
22759 ▷ ▷ ▷ G \
22760 ▷ ▷ ▷ PP4_no_mask_cyclic21_cases.csv \
22761 ▷ ▷ ▷ Starter \
22762 ▷ ▷ -end
22763
22764
22765
22766
22767
22768
22769
22770 PG_2_27_special:
22771 ▷ $(ORBITER) -v 2 \
22772 ▷ ▷ -define Do -draw_options \
22773 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
22774 ▷ ▷ -end \
22775 ▷ ▷ -define F -finite_field -q 27 -override_polynomial 46 -end \
22776 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
22777 ▷ ▷ -with P -do -projective_space_activity \
22778 ▷ ▷ ▷ -cheat_sheet Do \

```

```

22779 ▷ ▷ -end
22780 ▷ $(PDFLATEX) PG_2_27.tex
22781 ▷ $(OPEN) PG_2_27.pdf
22782
22783
22784
22785
22786
22787 #####
22788 # Section 14.5: Design Theory - Flag transitive designs
22789
22790 SECTION.DESIGN.THEORY.FLAG.TRANSITIVE.DESIGNS:
22791
22792 test_14.5:
22793 ▷ make Praeger_design_45_12_3
22794 ▷ make Praeger_design_change_values
22795 ▷ make Praeger_45_12_3_design
22796 ▷ make Praeger_45_12_3_design.c
22797 ▷
22798
22799
22800 Praeger_design_45_12_3:
22801 ▷ $(ORBITER) -v 4 \
22802 ▷ ▷ -define Do -draw.options \
22803 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
22804 ▷ ▷ -end \
22805 ▷ ▷ -define F -finite_field -q 81 -compute_related_fields \
22806 ▷ ▷ ▷ -override_polynomial 89 \
22807 ▷ ▷ -end \
22808 ▷ ▷ -define AGGL -linear_group -AGGL 1 F \
22809 ▷ ▷ ▷ -end \
22810 ▷ ▷ -define G -linear_group -AGGL 1 F \
22811 ▷ ▷ ▷ -subgroup_by_generators G 3240 \
22812 ▷ ▷ ▷ 8 $(PRAEGER_DESIGN_45_12_3.GENS) \
22813 ▷ ▷ ▷ -end \
22814 ▷ ▷ -with G -do \
22815 ▷ ▷ -group_theoretic_activity \
22816 ▷ ▷ ▷ -report Do \
22817 ▷ ▷ -end \
22818 ▷ ▷ -define v -vector \
22819 ▷ ▷ ▷ -dense $(PRAEGER_DESIGN_45_12_3.GENS) \
22820 ▷ ▷ -end \
22821 ▷ ▷ -define gens -vector_ge \
22822 ▷ ▷ ▷ -action AGGL \
22823 ▷ ▷ ▷ -vector_data v \
22824 ▷ ▷ -end \
22825 ▷ ▷ -with gens -do \
22826 ▷ ▷ -vector_ge_activity \
22827 ▷ ▷ ▷ -report \
22828 ▷ ▷ -end \
22829 ▷ ▷ -with gens -do \
22830 ▷ ▷ -vector_ge_activity \
22831 ▷ ▷ ▷ -field_reduction 4 \
22832 ▷ ▷ -end \
22833 ▷ ▷ -with G -do \
22834 ▷ ▷ -group_theoretic_activity \
22835 ▷ ▷ ▷ -subgroup_lattice_magma \
22836 ▷ ▷ -end \
22837 ▷ ▷ -define L -modified_group -from G \

```

```

22838 ▷ ▷ ▷ -subgroup_by_lattice 43 \
22839 ▷ ▷ -end \
22840 ▷ ▷ -define R -modified_group -from G \
22841 ▷ ▷ ▷ -subgroup_by_lattice 44 \
22842 ▷ ▷ -end \
22843 ▷ ▷ -with L -do \
22844 ▷ ▷ -group_theoretic_activity \
22845 ▷ ▷ ▷ -report Do \
22846 ▷ ▷ -end \
22847 ▷ ▷ -with R -do \
22848 ▷ ▷ -group_theoretic_activity \
22849 ▷ ▷ ▷ -report Do \
22850 ▷ ▷ -end \
22851 ▷ ▷ -with G -do \
22852 ▷ ▷ -group_theoretic_activity \
22853 ▷ ▷ ▷ -create_flag_transitive_geometry L R \
22854 ▷ ▷ -end
22855
22856
22857 # AGGL_1.81_Subgroup_G.3240_AGGL_1.81_Subgroup_G.3240_subgroup_by_lattice.43_AGGL
    _1.81_Subgroup_G.3240_subgroup_by_lattice.44_intersection_size.csv
22858
22859
22860
22861 Praeger_design_change_values:
22862 ▷ $(ORBITER) -v 3 \
22863 ▷ ▷ -orbiter_path $(ORBITER_PATH) \
22864 ▷ ▷ -change_values \
22865 ▷ ▷ ▷ AGGL_1.81_Subgroup_G.3240_AGGL_1.81_Subgroup_G.3240_subgroup_by_lattice.43_
    AGGL_1.81_Subgroup_G.3240_subgroup_by_lattice.44_intersection_size.csv \
22866 ▷ ▷ ▷ Praeger_6.csv \
22867 ▷ ▷ ▷ 3,6,8 \
22868 ▷ ▷ ▷ 0,1,0
22869
22870
22871
22872 Praeger_45_12.3_design:
22873 ▷ $(ORBITER) -v 8 \
22874 ▷ ▷ -define Praeger_45 -vector \
22875 ▷ ▷ ▷ -file Praeger_6.csv -format 45 \
22876 ▷ ▷ -end \
22877 ▷ ▷ -define D -design \
22878 ▷ ▷ ▷ -from_incidence_matrix Praeger_45 \
22879 ▷ ▷ -end \
22880 ▷ ▷ -with D -do \
22881 ▷ ▷ ▷ -design_activity \
22882 ▷ ▷ ▷ ▷ -export_flags \
22883 ▷ ▷ -end \
22884 ▷ ▷ -with D -do \
22885 ▷ ▷ ▷ -design_activity \
22886 ▷ ▷ ▷ ▷ -export_blocks \
22887 ▷ ▷ -end \
22888 ▷ ▷ -with D -do \
22889 ▷ ▷ ▷ -design_activity \
22890 ▷ ▷ ▷ ▷ -save \
22891 ▷ ▷ ▷ ▷ -intersection_matrix \
22892 ▷ ▷ -end
22893
22894 #Praeger_45_flags.txt

```

```

22895
22896 Praeger_45_12.3.design-c:
22897 ▷ $(ORBITER) -v 3 \
22898 ▷ ▷ -define Doinc -draw_incidence_structure_options \
22899 ▷ ▷ ▷ -width 60 -width_10 6 -end \
22900 ▷ ▷ -define C -combinatorial_object \
22901 ▷ ▷ ▷ -label Praeger Praeger \
22902 ▷ ▷ ▷ -file_of_incidence_geometries \
22903 ▷ ▷ ▷ ▷ Praeger_45.flags.txt 45 45 540 \
22904 ▷ ▷ -end \
22905 ▷ ▷ -with C -do \
22906 ▷ ▷ -combinatorial_object_activity \
22907 ▷ ▷ ▷ -canonical_form \
22908 ▷ ▷ ▷ ▷ -save_ago \
22909 ▷ ▷ ▷ ▷ -save_transversal \
22910 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "praeger_design_nauty_log.txt" -end \
22911 ▷ ▷ ▷ -end \
22912 ▷ ▷ -end \
22913 ▷ ▷ -with C -do \
22914 ▷ ▷ -combinatorial_object_activity \
22915 ▷ ▷ ▷ -post_processing \
22916 ▷ ▷ -end \
22917 ▷ ▷ -with C -do \
22918 ▷ ▷ -combinatorial_object_activity \
22919 ▷ ▷ ▷ -report \
22920 ▷ ▷ ▷ ▷ -export_flag_orbits \
22921 ▷ ▷ ▷ ▷ -show_incidence_matrices \
22922 ▷ ▷ ▷ ▷ -export_group_GAP \
22923 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
22924 ▷ ▷ ▷ -end \
22925 ▷ ▷ -end
22926 ▷ $(PDFLATEX) Praeger_classification.tex
22927 ▷ $(OPEN) Praeger_classification.pdf
22928 ▷ $(ORBITER) -v 2 -draw_matrix \
22929 ▷ ▷ -input_csv_file Praeger_object_0_INP_flag_orbits.csv \
22930 ▷ ▷ -secondary_input_csv_file Praeger_object_0_INP.csv \
22931 ▷ ▷ -box_width 32 -bit_depth 24 \
22932 ▷ ▷ -end
22933 ▷ magick \
22934 ▷ ▷ Praeger_object_0_INP_flag_orbits.draw.bmp \
22935 ▷ ▷ Praeger_object_0_INP_flag_orbits.draw.png
22936 ▷ $(OPEN) Praeger_object_0_INP_flag_orbits.draw.png
22937 ▷ $(ORBITER) -v 2 \
22938 ▷ ▷ -define Do -draw_options -embedded -radius 150 \
22939 ▷ ▷ ▷ -nodes_empty \
22940 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
22941 ▷ ▷ -tree_draw \
22942 ▷ ▷ ▷ -file praeger_design_nauty_log.txt \
22943 ▷ ▷ ▷ -draw_options Do \
22944 ▷ ▷ -end
22945 ▷ $(PDFLATEX) praeger_design_nauty_log.draw.tex
22946 ▷ $(OPEN) praeger_design_nauty_log.draw.pdf
22947
22948 # ago = 19440
22949
22950
22951 #####
22952 # Chapter 15 - Finite Geometry
22953 #####

```

```

22954
22955
22956 test_15:
22957 ▷ make test_15_1
22958 ▷ make test_15_2
22959 ▷ make test_15_3
22960 ▷ make test_15_4
22961
22962
22963
22964
22965 #####
22966 # Section 15.1: Spreads
22967
22968 SECTION.SPREADS:
22969
22970 test_15_1:
22971 ▷ make create_spread_9a
22972 ▷ make create_spread_9b
22973 ▷ make create_spread_25_7
22974 ▷ make create_spread_Rao_Rao_27
22975 ▷ make desarguesian_spread_in_PG_3_2
22976 ▷ make desarguesian_spread_in_PG_5_2
22977 ▷ make desarguesian_spread_in_PG_3_4
22978 ▷ make desarguesian_spread_in_PG_3_5
22979 ▷ make PG_3_2_spread_table
22980 ▷ make classify_spreads_4
22981 ▷ make classify_spreads_16_4
22982 ▷ make classify_spreads_25_starter_lift_case_0
22983 ▷ make spreads_25_starter_0_cliques
22984 ▷ make classify_spreads_25_starter_lift_all_cases
22985 ▷ make spreads_25_starter_cliques
22986 ▷ make classify_spreads_25_isomorph
22987 ▷ make classify_spreads_27_3_starter
22988 ▷ make classify_spreads_27_starter_lift_all_cases
22989 ▷ make spreads_27_starter_cliques
22990 ▷ make classify_spreads_27_isomorph_and_recognize
22991 ▷ make create_spread_27_0
22992 ▷ make create_spread_27_1
22993 ▷ make create_spread_27_2
22994 ▷ make create_spread_27_3
22995 ▷ make create_spread_27_4
22996 ▷ make create_spread_27_5
22997 ▷ make create_spread_27_6
22998 ▷ make PG_3_5_packing_PWP_0_dualize
22999
23000
23001
23002 create_spread_9a:
23003 ▷ $(ORBITER) -v 3 \
23004 ▷ ▷ -define F -finite_field -q 3 -end \
23005 ▷ ▷ -define G -linear_group -PGL 4 F -end \
23006 ▷ ▷ -define S -spread -kernel_field F \
23007 ▷ ▷ ▷ -group G -k 2 -catalogue 0 \
23008 ▷ ▷ ▷ -end \
23009 ▷ ▷ -with S -do -spread_activity \
23010 ▷ ▷ ▷ -report \
23011 ▷ ▷ -end
23012

```

```

23013 # desarguesian spread, ago = 5760
23014
23015 create_spread_9b:
23016 ▷ $(ORBITER) -v 3 \
23017 ▷ ▷ -define F -finite_field -q 3 -end \
23018 ▷ ▷ -define G -linear_group -PGL 4 F -end \
23019 ▷ ▷ -define S -spread -kernel_field F \
23020 ▷ ▷ ▷ -group G -k 2 -catalogue 1 \
23021 ▷ ▷ ▷ -end \
23022 ▷ ▷ -with S -do -spread_activity \
23023 ▷ ▷ ▷ -report \
23024 ▷ ▷ -end
23025
23026
23027 # Hall spread, ago = 1920
23028
23029
23030 create_spread_25.7:
23031 ▷ $(ORBITER) -v 3 \
23032 ▷ ▷ -define F -finite_field -q 5 -end \
23033 ▷ ▷ -define G -linear_group -PGL 4 F -end \
23034 ▷ ▷ -define S -spread -kernel_field F \
23035 ▷ ▷ ▷ -group G -k 2 -catalogue 7 \
23036 ▷ ▷ ▷ -end \
23037 ▷ ▷ -with S -do -spread_activity \
23038 ▷ ▷ ▷ -report \
23039 ▷ ▷ -end
23040
23041
23042
23043 create_spread_Rao_Rao_27:
23044 ▷ $(ORBITER) -v 3 \
23045 ▷ ▷ -define F -finite_field -q 3 -end \
23046 ▷ ▷ -define SS -vector -dense $(SPREAD_SET_27_RAO_RAO) -end \
23047 ▷ ▷ -define G -linear_group -PGL 6 F -end \
23048 ▷ ▷ -define S -spread -kernel_field F \
23049 ▷ ▷ ▷ -group G -k 3 -spread_set "Rao_Rao" "Rao\Rao" SS \
23050 ▷ ▷ ▷ -end
23051
23052
23053 desarguesian_spread_in_PG.3.2:
23054 ▷ $(ORBITER) -v 3 \
23055 ▷ ▷ -define FQ -finite_field -q 4 -end \
23056 ▷ ▷ -define Fq -finite_field -q 2 -end \
23057 ▷ ▷ -with FQ -and Fq -do -finite_field_activity \
23058 ▷ ▷ ▷ -cheat_sheet_desarguesian_spread 2 -end
23059 ▷ $(PDFLATEX) Desarguesian_Spread.3.2.tex
23060 ▷ $(OPEN) Desarguesian_Spread.3.2.pdf
23061
23062 desarguesian_spread_in_PG.5.2:
23063 ▷ $(ORBITER) -v 3 \
23064 ▷ ▷ -define FQ -finite_field -q 8 -end \
23065 ▷ ▷ -define Fq -finite_field -q 2 -end \
23066 ▷ ▷ -with FQ -and Fq -do -finite_field_activity \
23067 ▷ ▷ ▷ -cheat_sheet_desarguesian_spread 2 -end
23068 ▷ $(PDFLATEX) Desarguesian_Spread.5.2.tex
23069 ▷ $(OPEN) Desarguesian_Spread.5.2.pdf
23070
23071 desarguesian_spread_in_PG.3.4:

```



```

23072 ▷ $(ORBITER) -v 3 \
23073 ▷ ▷ -define FQ -finite_field -q 16 -end \
23074 ▷ ▷ -define Fq -finite_field -q 4 -end \
23075 ▷ ▷ -with FQ -and Fq -do -finite_field_activity \
23076 ▷ ▷ ▷ -cheat_sheet_desarguesian_spread 2 -end
23077 ▷ $(PDFLATEX) Desarguesian_Spread_3.4.tex
23078 ▷ $(OPEN) Desarguesian_Spread_3.4.pdf
23079
23080 desarguesian_spread_in_PG_3.5:
23081 ▷ $(ORBITER) -v 3 \
23082 ▷ ▷ -define FQ -finite_field -q 25 -end \
23083 ▷ ▷ -define Fq -finite_field -q 5 -end \
23084 ▷ ▷ -with FQ -and Fq -do -finite_field_activity \
23085 ▷ ▷ ▷ -cheat_sheet_desarguesian_spread 2 -end
23086 ▷ $(PDFLATEX) Desarguesian_Spread_3.5.tex
23087 ▷ $(OPEN) Desarguesian_Spread_3.5.pdf
23088
23089
23090
23091 PG_3.2_spread_table:
23092 ▷ - mkdir SPREAD_TABLES_2
23093 ▷ $(ORBITER) -v 5 \
23094 ▷ ▷ -define Control -poset_classification_control \
23095 ▷ ▷ ▷ -problem_label spreads_PG_3.2 \
23096 ▷ ▷ ▷ -W \
23097 ▷ ▷ -end \
23098 ▷ ▷ -define F -finite_field -q 2 -end \
23099 ▷ ▷ -define P3 -projective_space -n 3 -field F -v 0 -end \
23100 ▷ ▷ -define T -spread_table -space P3 -rk 2 \
23101 ▷ ▷ ▷ -iso_types "0" \
23102 ▷ ▷ ▷ -path "SPREAD_TABLES_2/" \
23103 ▷ ▷ ▷ -control Control \
23104 ▷ ▷ ▷ -end \
23105
23106
23107 # 56 spreads
23108 #SPREAD_TABLES_2/spread_4_table.csv
23109
23110 # ToDo -report
23111
23112 classify_spreads_4:
23113 ▷ $(ORBITER) -v 3 \
23114 ▷ ▷ -define Do -draw_options \
23115 ▷ ▷ ▷ -embedded \
23116 ▷ ▷ -end \
23117 ▷ ▷ -define Control -poset_classification_control \
23118 ▷ ▷ ▷ -W \
23119 ▷ ▷ ▷ -problem_label spreads_2.2 \
23120 ▷ ▷ ▷ -draw_options Do \
23121 ▷ ▷ -end \
23122 ▷ ▷ -define F -finite_field -q 2 -end \
23123 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
23124 ▷ ▷ -define C -spread_classifier \
23125 ▷ ▷ ▷ -projective_space P \
23126 ▷ ▷ ▷ -poset_classification_control Control \
23127 ▷ ▷ ▷ -k 2 \
23128 ▷ ▷ ▷ -starter_size 5 \
23129 ▷ ▷ ▷ -output_prefix "." \
23130 ▷ ▷ -end \

```

```

23131 ▷ ▷ -with C -do -spread_classify_activity \
23132 ▷ ▷ ▷ -compute_starter \
23133 ▷ ▷ -end
23134 ▷ #$(PDFLATEX) spreads_2.2_poset_lvl5.tex
23135 ▷ #$(OPEN) spreads_2.2_poset_lvl5.pdf
23136
23137
23138 classify_spreads_16_4:
23139 ▷ $(ORBITER) -v 4 \
23140 ▷ ▷ -define Do -draw_options \
23141 ▷ ▷ ▷ -radius 20 \
23142 ▷ ▷ ▷ -nodes_empty \
23143 ▷ ▷ ▷ -line_width 0.2 \
23144 ▷ ▷ ▷ -embedded \
23145 ▷ ▷ -end \
23146 ▷ ▷ -define Control -poset_classification_control \
23147 ▷ ▷ ▷ -problem_label spreads_16_4 \
23148 ▷ ▷ ▷ -draw_options Do \
23149 ▷ ▷ -end \
23150 ▷ ▷ -define F -finite_field -q 4 -end \
23151 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
23152 ▷ ▷ -define C -spread_classifier \
23153 ▷ ▷ ▷ -projective_space P \
23154 ▷ ▷ ▷ -poset_classification_control Control \
23155 ▷ ▷ ▷ -k 2 \
23156 ▷ ▷ ▷ -starter_size 17 \
23157 ▷ ▷ ▷ -output_prefix "." \
23158 ▷ ▷ -end \
23159 ▷ ▷ -with C -do -spread_classify_activity \
23160 ▷ ▷ ▷ -compute_starter \
23161 ▷ ▷ -end
23162 ▷ #$(PDFLATEX) spreads_16_4_poset_lvl17.tex
23163 ▷ #$(OPEN) spreads_16_4_poset_lvl17.pdf
23164
23165
23166
23167
23168
23169 classify_spreads_25_starter_lift_case0:
23170 ▷ $(ORBITER) -v 3 \
23171 ▷ ▷ -define Do -draw_options \
23172 ▷ ▷ ▷ -radius 20 \
23173 ▷ ▷ ▷ -nodes_empty \
23174 ▷ ▷ ▷ -line_width 0.2 \
23175 ▷ ▷ ▷ -embedded \
23176 ▷ ▷ -end \
23177 ▷ ▷ -define Control -poset_classification_control \
23178 ▷ ▷ ▷ -W \
23179 ▷ ▷ ▷ -problem_label spreads_25 \
23180 ▷ ▷ ▷ -draw_options Do \
23181 ▷ ▷ -end \
23182 ▷ ▷ -define F -finite_field -q 5 -end \
23183 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
23184 ▷ ▷ -define C -spread_classifier \
23185 ▷ ▷ ▷ -projective_space P \
23186 ▷ ▷ ▷ -k 2 \
23187 ▷ ▷ ▷ -starter_size 5 \
23188 ▷ ▷ ▷ -recoordinatize \
23189 ▷ ▷ ▷ -poset_classification_control Control \

```

```

23190 ▷ ▷ ▷ -output_prefix "" \
23191 ▷ ▷ -end \
23192 ▷ ▷ -with C -do -spread_classify_activity \
23193 ▷ ▷ ▷ -compute_starter \
23194 ▷ ▷ -end \
23195 ▷ ▷ -with C -do -spread_classify_activity \
23196 ▷ ▷ ▷ -prepare_lifting_single_case 0 \
23197 ▷ ▷ -end
23198
23199
23200 #save_colored_graph fname=spreads_25_graph_0.bin
23201 #save_colored_graph nb_vertices=225
23202 #save_colored_graph nb_colors=21
23203 #save_colored_graph nb_colors_per_vertex=1
23204 #save_colored_graph done
23205 #colored_graph::save done
23206 #Written file spreads_25_graph_0.bin of size 5914
23207
23208
23209 spreads_25_starter_0_cliques:
23210 ▷ $(ORBITER) -v 2 \
23211 ▷ ▷ ▷ -define G -graph -load spreads_25_graph_0.bin -end \
23212 ▷ ▷ ▷ -with G -do \
23213 ▷ ▷ ▷ -graph_theoretic_activity \
23214 ▷ ▷ ▷ ▷ -find_cliques -rainbow -target_size 21 \
23215 ▷ ▷ ▷ ▷ -output_file spreads_25_0_cliques.csv \
23216 ▷ ▷ ▷ -end \
23217
23218 #graph_theoretic_activity::perform_activity Gr->label=spreads_25_graph_0 nb_sol =
      7680
23219
23220
23221 classify_spreads_25_starter_lift_all_cases:
23222 ▷ $(ORBITER) -v 3 \
23223 ▷ ▷ -define Do -draw_options \
23224 ▷ ▷ ▷ -radius 20 \
23225 ▷ ▷ ▷ -nodes_empty \
23226 ▷ ▷ ▷ -line_width 0.2 \
23227 ▷ ▷ ▷ -embedded \
23228 ▷ ▷ -end \
23229 ▷ ▷ -define Control -poset_classification_control \
23230 ▷ ▷ ▷ -W \
23231 ▷ ▷ ▷ -problem_label spreads_25 \
23232 ▷ ▷ ▷ -draw_options Do \
23233 ▷ ▷ -end \
23234 ▷ ▷ -define F -finite_field -q 5 -end \
23235 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
23236 ▷ ▷ -define C -spread_classifier \
23237 ▷ ▷ ▷ -projective_space P \
23238 ▷ ▷ ▷ -k 2 \
23239 ▷ ▷ ▷ -starter_size 5 \
23240 ▷ ▷ ▷ -recoordinatize \
23241 ▷ ▷ ▷ -poset_classification_control Control \
23242 ▷ ▷ ▷ -output_prefix "" \
23243 ▷ ▷ -end \
23244 ▷ ▷ -with C -do -spread_classify_activity \
23245 ▷ ▷ ▷ -compute_starter \
23246 ▷ ▷ -end \
23247 ▷ ▷ -with C -do -spread_classify_activity \

```

```

23248 ▷ ▷ ▷ -prepare_lifting_all_cases \
23249 ▷ ▷ -end
23250
23251
23252 # target size = 25 + 1 - starter_size = 26 - 5 = 21
23253
23254
23255 spreads_25_starter_cliques:
23256 ▷ $(ORBITER) -v 2 \
23257 ▷ ▷ -loop L 0 29 1 \
23258 ▷ ▷ ▷ -define G -graph -load spreads_25_graph_%L.bin -end \
23259 ▷ ▷ ▷ -with G -do \
23260 ▷ ▷ ▷ -graph_theoretic_activity \
23261 ▷ ▷ ▷ ▷ -find_cliques -rainbow -target_size 21 \
23262 ▷ ▷ ▷ ▷ -output_file spreads_25_graph_%L.sol.csv \
23263 ▷ ▷ ▷ -end \
23264 ▷ ▷ -end_loop L
23265
23266 #spreads_25_graph_0_sol.csv
23267 #spreads_25_graph_28_sol.csv
23268 # are all empty!
23269
23270
23271 # not: spreads_25_graph_0_cliques.csv
23272
23273 classify_spreads_25_isomorph:
23274 ▷ $(ORBITER) -v 4 \
23275 ▷ ▷ -define Do -draw_options \
23276 ▷ ▷ ▷ -radius 20 \
23277 ▷ ▷ ▷ -nodes_empty \
23278 ▷ ▷ ▷ -line_width 0.2 \
23279 ▷ ▷ ▷ -embedded \
23280 ▷ ▷ -end \
23281 ▷ ▷ -define Control -poset_classification_control \
23282 ▷ ▷ ▷ -W \
23283 ▷ ▷ ▷ -problem_label spreads_25 \
23284 ▷ ▷ ▷ -draw_options Do \
23285 ▷ ▷ -end \
23286 ▷ ▷ -define Iso -isomorph_arguments \
23287 ▷ ▷ ▷ -prefix_iso "../spreads_25" \
23288 ▷ ▷ ▷ -use_database_for_starter \
23289 ▷ ▷ ▷ -solution_prefix "spreads_25_graph" \
23290 ▷ ▷ ▷ -base_fname "" \
23291 ▷ ▷ -end \
23292 ▷ ▷ -define F -finite_field -q 5 -end \
23293 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
23294 ▷ ▷ -define C -spread_classifier \
23295 ▷ ▷ ▷ -projective_space P \
23296 ▷ ▷ ▷ -k 2 \
23297 ▷ ▷ ▷ -starter_size 5 \
23298 ▷ ▷ ▷ -recoordinatize \
23299 ▷ ▷ ▷ -poset_classification_control Control \
23300 ▷ ▷ ▷ -output_prefix "" \
23301 ▷ ▷ -end \
23302 ▷ ▷ -with C -do -spread_classify_activity \
23303 ▷ ▷ ▷ -compute_starter \
23304 ▷ ▷ -end \
23305 ▷ ▷ -with C -do -spread_classify_activity \
23306 ▷ ▷ ▷ -isomorph Iso \

```

```

23307 ▷ ▷ ▷ -build.db \
23308 ▷ ▷ -end \
23309 ▷ ▷ -with C -do -spread_classify_activity \
23310 ▷ ▷ ▷ -isomorph Iso \
23311 ▷ ▷ ▷ -read_solutions \
23312 ▷ ▷ -end \
23313 ▷ ▷ -with C -do -spread_classify_activity \
23314 ▷ ▷ ▷ -isomorph Iso \
23315 ▷ ▷ ▷ -compute_orbits \
23316 ▷ ▷ -end \
23317 ▷ ▷ -with C -do -spread_classify_activity \
23318 ▷ ▷ ▷ -isomorph Iso \
23319 ▷ ▷ ▷ -isomorph_testing \
23320 ▷ ▷ -end \
23321 ▷ ▷ -with C -do -spread_classify_activity \
23322 ▷ ▷ ▷ -isomorph Iso \
23323 ▷ ▷ ▷ -isomorph_report \
23324 ▷ ▷ -end
23325 ▷ $(PDFLATEX) spreads_25_isomorphism_types.tex
23326 ▷ $(OPEN) spreads_25_isomorphism_types.pdf
23327
23328 #We found 21 isomorphism types
23329 #1:33
23330
23331
23332
23333 classify_spreads_27_3_starter:
23334 ▷ $(ORBITER) -v 10 \
23335 ▷ ▷ -define Do -draw_options \
23336 ▷ ▷ ▷ -radius 20 \
23337 ▷ ▷ ▷ -nodes_empty \
23338 ▷ ▷ ▷ -line_width 0.2 \
23339 ▷ ▷ ▷ -embedded \
23340 ▷ ▷ -end \
23341 ▷ ▷ -define Control -poset_classification_control \
23342 ▷ ▷ ▷ -W \
23343 ▷ ▷ ▷ -problem_label spreads_27_3 \
23344 ▷ ▷ ▷ -draw_options Do \
23345 ▷ ▷ -end \
23346 ▷ ▷ -define F -finite_field -q 3 -end \
23347 ▷ ▷ -define P -projective_space -n 5 -field F -v 0 -end \
23348 ▷ ▷ -define C -spread_classifier \
23349 ▷ ▷ ▷ -projective_space P \
23350 ▷ ▷ ▷ -poset_classification_control Control \
23351 ▷ ▷ ▷ -k 3 \
23352 ▷ ▷ ▷ -starter_size 5 \
23353 ▷ ▷ ▷ -recoordinatize \
23354 ▷ ▷ ▷ -output_prefix "." \
23355 ▷ ▷ -end \
23356 ▷ ▷ -with C -do -spread_classify_activity \
23357 ▷ ▷ ▷ -compute_starter \
23358 ▷ ▷ -end \
23359 ▷ ▷ -with C -do -spread_classify_activity \
23360 ▷ ▷ ▷ -prepare_lifting_single_case 0 \
23361 ▷ ▷ -end
23362 ▷ #$(PDFLATEX) spreads_27_3_poset_detailed_lvl_5.tex
23363 ▷ #$(OPEN) spreads_27_3_poset_detailed_lvl_5.pdf
23364
23365 # 50 orbits at level 5:

```

```

23366 #5 : 50 orbits
23367 #total: 60
23368 #(39^2, 26^2, 10, 6^2, 5, 3^9, 2^9, 1^{24}) average is 4 + 26 / 50
23369
23370 # time 4:31
23371
23372
23373
23374 classify_spreads_27_starter_lift_all_cases:
23375 ▷ $(ORBITER) -v 3 \
23376 ▷ ▷ -define Do -draw.options \
23377 ▷ ▷ ▷ -radius 20 \
23378 ▷ ▷ ▷ -nodes.empty \
23379 ▷ ▷ ▷ -line_width 0.2 \
23380 ▷ ▷ ▷ -embedded \
23381 ▷ ▷ -end \
23382 ▷ ▷ -define Control -poset_classification_control \
23383 ▷ ▷ ▷ -W \
23384 ▷ ▷ ▷ -problem_label spreads_27_3 \
23385 ▷ ▷ ▷ -draw.options Do \
23386 ▷ ▷ -end \
23387 ▷ ▷ -define F -finite_field -q 3 -end \
23388 ▷ ▷ -define P -projective_space -n 5 -field F -v 0 -end \
23389 ▷ ▷ -define C -spread_classifier \
23390 ▷ ▷ ▷ -projective_space P \
23391 ▷ ▷ ▷ -k 3 \
23392 ▷ ▷ ▷ -starter_size 5 \
23393 ▷ ▷ ▷ -recoordinatize \
23394 ▷ ▷ ▷ -poset_classification_control Control \
23395 ▷ ▷ ▷ -output_prefix "" \
23396 ▷ ▷ -end \
23397 ▷ ▷ -with C -do -spread_classify_activity \
23398 ▷ ▷ ▷ -compute_starter \
23399 ▷ ▷ -end \
23400 ▷ ▷ -with C -do -spread_classify_activity \
23401 ▷ ▷ ▷ -prepare_lifting_all_cases \
23402 ▷ ▷ -end
23403
23404
23405 # 50 graphs
23406
23407
23408
23409 spreads_27_starter_cliques:
23410 ▷ $(ORBITER) -v 2 \
23411 ▷ ▷ -loop L 0 50 1 \
23412 ▷ ▷ ▷ -define G -graph -load spreads_27_3_graph_%L.bin -end \
23413 ▷ ▷ ▷ -with G -do \
23414 ▷ ▷ ▷ -graph_theoretic_activity \
23415 ▷ ▷ ▷ ▷ -find_cliques -rainbow -target_size 23 -end \
23416 ▷ ▷ ▷ -end \
23417 ▷ ▷ -end_loop L
23418
23419 #4:41
23420
23421
23422
23423 classify_spreads_27_isomorph_and_recognize:
23424 ▷ $(ORBITER) -v 3 \

```

```

23425 ▷ ▷ -define Do -draw.options \
23426 ▷ ▷ ▷ -radius 20 \
23427 ▷ ▷ ▷ -nodes.empty \
23428 ▷ ▷ ▷ -line_width 0.2 \
23429 ▷ ▷ ▷ -embedded \
23430 ▷ ▷ -end \
23431 ▷ ▷ -define Control -poset_classification_control \
23432 ▷ ▷ ▷ -W \
23433 ▷ ▷ ▷ -problem_label spreads_27_3 \
23434 ▷ ▷ ▷ -draw.options Do \
23435 ▷ ▷ -end \
23436 ▷ ▷ -define F -finite_field -q 3 -end \
23437 ▷ ▷ -define P -projective_space -n 5 -field F -v 0 -end \
23438 ▷ ▷ -define C -spread_classifier \
23439 ▷ ▷ ▷ -projective_space P \
23440 ▷ ▷ ▷ -k 3 \
23441 ▷ ▷ ▷ -starter_size 5 \
23442 ▷ ▷ ▷ -recoordinatize \
23443 ▷ ▷ ▷ -poset_classification_control Control \
23444 ▷ ▷ ▷ -output_prefix "" \
23445 ▷ ▷ -end \
23446 ▷ ▷ -with C -do -spread_classify_activity \
23447 ▷ ▷ ▷ -compute_starter \
23448 ▷ ▷ -end \
23449 ▷ ▷ -with C -do -spread_classify_activity \
23450 ▷ ▷ ▷ -isomorph \
23451 ▷ ▷ ▷ ▷ -prefix_iso "./spreads_27_3" \
23452 ▷ ▷ ▷ ▷ -use_database_for_starter \
23453 ▷ ▷ ▷ ▷ -build_db \
23454 ▷ ▷ ▷ ▷ -solution_prefix "" \
23455 ▷ ▷ ▷ ▷ -base_fname "" \
23456 ▷ ▷ ▷ -end \
23457 ▷ ▷ -end \
23458 ▷ ▷ -with C -do -spread_classify_activity \
23459 ▷ ▷ ▷ -isomorph \
23460 ▷ ▷ ▷ ▷ -prefix_iso "./spreads_27_3" \
23461 ▷ ▷ ▷ ▷ -use_database_for_starter \
23462 ▷ ▷ ▷ ▷ -read_solutions \
23463 ▷ ▷ ▷ ▷ -solution_prefix "" \
23464 ▷ ▷ ▷ ▷ -base_fname "spreads_27_3_graph" \
23465 ▷ ▷ ▷ -end \
23466 ▷ ▷ -end \
23467 ▷ ▷ -with C -do -spread_classify_activity \
23468 ▷ ▷ ▷ -isomorph \
23469 ▷ ▷ ▷ ▷ -prefix_iso "./spreads_27_3" \
23470 ▷ ▷ ▷ ▷ -use_database_for_starter \
23471 ▷ ▷ ▷ ▷ -compute_orbits \
23472 ▷ ▷ ▷ ▷ -solution_prefix "" \
23473 ▷ ▷ ▷ ▷ -base_fname "spreads_27_3_graph" \
23474 ▷ ▷ ▷ -end \
23475 ▷ ▷ -end \
23476 ▷ ▷ -with C -do -spread_classify_activity \
23477 ▷ ▷ ▷ -isomorph \
23478 ▷ ▷ ▷ ▷ -prefix_iso "./spreads_27_3" \
23479 ▷ ▷ ▷ ▷ -use_database_for_starter \
23480 ▷ ▷ ▷ ▷ -isomorph_testing \
23481 ▷ ▷ ▷ ▷ -solution_prefix "" \
23482 ▷ ▷ ▷ ▷ -base_fname "spreads_27_3_graph" \
23483 ▷ ▷ ▷ -end \

```

```

23484 ▷ ▷ -end \
23485 ▷ ▷ -with C -do -spread_classify_activity \
23486 ▷ ▷ ▷ -isomorph \
23487 ▷ ▷ ▷ ▷ -prefix_iso "./spreads_27_3" \
23488 ▷ ▷ ▷ ▷ -use_database_for_starter \
23489 ▷ ▷ ▷ ▷ -isomorph_report \
23490 ▷ ▷ ▷ ▷ -solution_prefix "" \
23491 ▷ ▷ ▷ ▷ -base_fname "spreads_27_3_graph" \
23492 ▷ ▷ ▷ -end \
23493 ▷ ▷ -end \
23494 ▷ ▷ -with C -do -spread_classify_activity \
23495 ▷ ▷ ▷ -isomorph \
23496 ▷ ▷ ▷ ▷ -prefix_iso "./spreads_27_3" \
23497 ▷ ▷ ▷ ▷ -use_database_for_starter \
23498 ▷ ▷ ▷ ▷ -recognize $(SPREAD_S27_RA0_RA0) \
23499 ▷ ▷ ▷ ▷ -solution_prefix "" \
23500 ▷ ▷ ▷ ▷ -base_fname "spreads_27_3_graph" \
23501 ▷ ▷ ▷ -end \
23502 ▷ ▷ -end
23503 ▷ #$(PDFLATEX) spreads_27_isomorphism_types.tex
23504 ▷ #$(OPEN) spreads_27_isomorphism_types.pdf
23505 ▷ #$(PDFLATEX) spreads_27_aut_group.tex
23506 ▷ #$(OPEN) spreads_27_aut_group.pdf
23507
23508
23509 # SPREAD_S27_RA0_RA0 is isomorphic to spread 0 in the list
23510 # (which is different from the ordering of the Orbiter catalogue)
23511 # the stabilizer of the spread has order 84.
23512
23513
23514 #substructure_lifting_data::write_hash_and_datref_file id.to.hash tallied:
23515 #( 1^6076, 2^289, 3^35, 4^5 )
23516 # using 64 bit hash values, based on a modified version of Paul Hsieh's SuperFast
    Hash
23517
23518 #We found 7 isomorphism types
23519 #0:36
23520
23521 #generators for the stabilizer of the Rao/Rao spread:
23522 #1,2,2,2,0,2,1,1,2,2,2,1,2,1,2,1,1,0,2,2,0,2,0,0,1,1,2,2,2,0,0,1,1,2,0,1, #1,0,1,
    0,1,2,0,1,2,2,2,2,1,1,1,0,1,1,2,0,1,1,2,1,1,1,0,2,2,2,0,2,0,1,1,0,
23523
23524
23525
23526 create_spread_27_0:
23527 ▷ $(ORBITER) -v 3 \
23528 ▷ ▷ -define F -finite_field -q 3 -end \
23529 ▷ ▷ -define G -linear_group -PGL 6 F -end \
23530 ▷ ▷ -define S -spread -kernel_field F \
23531 ▷ ▷ ▷ -group G -k 3 -catalogue 0 \
23532 ▷ ▷ ▷ -end \
23533 ▷ ▷ -with S -do -spread_activity \
23534 ▷ ▷ ▷ -report \
23535 ▷ ▷ -end
23536 ▷ #$(PDFLATEX) catalogue_q3_k3_0_report.tex
23537 ▷ #$(OPEN) catalogue_q3_k3_0_report.pdf
23538
23539 create_spread_27_1:
23540 ▷ $(ORBITER) -v 3 \

```



```

23541 ▷ ▷ -define F -finite_field -q 3 -end \
23542 ▷ ▷ -define G -linear_group -PGL 6 F -end \
23543 ▷ ▷ -define S -spread -kernel_field F \
23544 ▷ ▷ ▷ -group G -k 3 -catalogue 1 \
23545 ▷ ▷ ▷ -end \
23546 ▷ ▷ -with S -do -spread_activity \
23547 ▷ ▷ ▷ -report \
23548 ▷ ▷ -end
23549 ▷ $(PDFLATEX) catalogue_q3_k3_1.report.tex
23550 ▷ $(OPEN) catalogue_q3_k3_1.report.pdf
23551
23552 create_spread_27.2:
23553 ▷ $(ORBITER) -v 3 \
23554 ▷ ▷ -define F -finite_field -q 3 -end \
23555 ▷ ▷ -define G -linear_group -PGL 6 F -end \
23556 ▷ ▷ -define S -spread -kernel_field F \
23557 ▷ ▷ ▷ -group G -k 3 -catalogue 2 \
23558 ▷ ▷ ▷ -end \
23559 ▷ ▷ -with S -do -spread_activity \
23560 ▷ ▷ ▷ -report \
23561 ▷ ▷ -end
23562 ▷ $(PDFLATEX) catalogue_q3_k3_2.report.tex
23563 ▷ $(OPEN) catalogue_q3_k3_2.report.pdf
23564
23565 create_spread_27.3:
23566 ▷ $(ORBITER) -v 3 \
23567 ▷ ▷ -define F -finite_field -q 3 -end \
23568 ▷ ▷ -define G -linear_group -PGL 6 F -end \
23569 ▷ ▷ -define S -spread -kernel_field F \
23570 ▷ ▷ ▷ -group G -k 3 -catalogue 3 \
23571 ▷ ▷ ▷ -end \
23572 ▷ ▷ -with S -do -spread_activity \
23573 ▷ ▷ ▷ -report \
23574 ▷ ▷ -end
23575 ▷ $(PDFLATEX) catalogue_q3_k3_3.report.tex
23576 ▷ $(OPEN) catalogue_q3_k3_3.report.pdf
23577
23578 create_spread_27.4:
23579 ▷ $(ORBITER) -v 3 \
23580 ▷ ▷ -define F -finite_field -q 3 -end \
23581 ▷ ▷ -define G -linear_group -PGL 6 F -end \
23582 ▷ ▷ -define S -spread -kernel_field F \
23583 ▷ ▷ ▷ -group G -k 3 -catalogue 4 \
23584 ▷ ▷ ▷ -end \
23585 ▷ ▷ -with S -do -spread_activity \
23586 ▷ ▷ ▷ -report \
23587 ▷ ▷ -end
23588 ▷ $(PDFLATEX) catalogue_q3_k3_4.report.tex
23589 ▷ $(OPEN) catalogue_q3_k3_4.report.pdf
23590
23591 create_spread_27.5:
23592 ▷ $(ORBITER) -v 3 \
23593 ▷ ▷ -define F -finite_field -q 3 -end \
23594 ▷ ▷ -define G -linear_group -PGL 6 F -end \
23595 ▷ ▷ -define S -spread -kernel_field F \
23596 ▷ ▷ ▷ -group G -k 3 -catalogue 5 \
23597 ▷ ▷ ▷ -end \
23598 ▷ ▷ -with S -do -spread_activity \
23599 ▷ ▷ ▷ -report \

```

```

23600 ▷ ▷ -end
23601 ▷ $(PDFLATEX) catalogue_q3_k3_5_report.tex
23602 ▷ $(OPEN) catalogue_q3_k3_5_report.pdf
23603
23604 create_spread_27_6:
23605 ▷ $(ORBITER) -v 3 \
23606 ▷ ▷ -define F -finite_field -q 3 -end \
23607 ▷ ▷ -define G -linear_group -PGL 6 F -end \
23608 ▷ ▷ -define S -spread -kernel_field F \
23609 ▷ ▷ ▷ -group G -k 3 -catalogue 6 \
23610 ▷ ▷ ▷ -end \
23611 ▷ ▷ -with S -do -spread_activity \
23612 ▷ ▷ ▷ -report \
23613 ▷ ▷ -end
23614 ▷ $(PDFLATEX) catalogue_q3_k3_6_report.tex
23615 ▷ $(OPEN) catalogue_q3_k3_6_report.pdf
23616
23617
23618
23619
23620
23621 PG_3_5_packing_PWP_0_dualize:
23622 ▷ $(ORBITER_PATH)orbiter.out -v 5 \
23623 ▷ ▷ -define F -finite_field -q 5 -end \
23624 ▷ ▷ -define P -projective_space 3 F -end \
23625 ▷ ▷ -define T -spread_table -space P3 -rk 2 \
23626 ▷ ▷ ▷ -iso_types "12" \
23627 ▷ ▷ ▷ -path "SPREAD_TABLES_5_REG/" \
23628 ▷ ▷ ▷ -control Control \
23629 ▷ ▷ ▷ -end \
23630 ▷ ▷ -with T -do \
23631 ▷ ▷ ▷ -spread_table_activity \
23632 ▷ ▷ ▷ -dualize_packing $(PENTTILA_WILLIAMS_PRINCE_REG_PACKING_0) \
23633 ▷ ▷ -end
23634
23635
23636
23637
23638
23639
23640 #####
23641 # Section 15.2: Translation planes
23642
23643
23644 SECTION_TRANSLATION_PLANES:
23645
23646
23647 test_15_2:
23648 ▷ make create_translation_plane_9b
23649 ▷ make create_translation_plane_16_4_0
23650 ▷ make create_translation_plane_16_2_0
23651 ▷ make RREF_plane_16_2_0_rank_of_incma
23652 ▷ make create_translation_plane_25_14_rank
23653 ▷ make create_translation_plane_27_Rao_Rao
23654 ▷ make RREF_Rao_Rao_plane_incma_rank
23655 ▷ make create_translation_plane_27_p_rank_of_incidence_matrix
23656 ▷ make create_translation_plane_27_5_Rao_Rao
23657 ▷ make translation_plane_27_4_orbits
23658 ▷ make translation_plane_27_5_orbits

```

```

23659 ▷ make translation_plane.27.6.orbits
23660
23661
23662
23663
23664 create_translation_plane.9b:
23665 ▷ $(ORBITER) -v 3 \
23666 ▷ ▷ -define F -finite_field -q 3 -end \
23667 ▷ ▷ -define G -linear_group -PGL 4 F -end \
23668 ▷ ▷ -define G1 -linear_group -PGL 5 F -end \
23669 ▷ ▷ -define S -spread -kernel_field F \
23670 ▷ ▷ ▷ -group G -k 2 -catalogue 1 \
23671 ▷ ▷ ▷ -end \
23672 ▷ ▷ -define T -translation_plane S G G1 -end \
23673 ▷ ▷ -with T -do -translation_plane_activity \
23674 ▷ ▷ ▷ -export_incma \
23675 ▷ ▷ -end \
23676 ▷ ▷ -with T -do -translation_plane_activity \
23677 ▷ ▷ ▷ -report \
23678 ▷ ▷ -end \
23679 ▷ ▷ -define A -linear_group -import_group_of_plane T -end \
23680 ▷ ▷ -define Orb -orbits -group A \
23681 ▷ ▷ ▷ -on_points \
23682 ▷ ▷ -end \
23683 ▷ ▷ -define Do -draw_options -embedded -sideways -radius 50 \
23684 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 \
23685 ▷ ▷ -end \
23686 ▷ ▷ -with Orb -do -orbits_activity \
23687 ▷ ▷ ▷ -report Do \
23688 ▷ ▷ -end \
23689 ▷ ▷ -with Orb -do -orbits_activity \
23690 ▷ ▷ ▷ -stabilizer 92 \
23691 ▷ ▷ -end \
23692 ▷ ▷ -with Orb -do -orbits_activity \
23693 ▷ ▷ ▷ -export_trees \
23694 ▷ ▷ -end
23695 ▷ $(ORBITER) -v 2 \
23696 ▷ ▷ -draw_matrix \
23697 ▷ ▷ ▷ -input_csv_file plane_catalogue.q3.k2.1.incma.csv \
23698 ▷ ▷ ▷ -box_width 6 -bit_depth 8 \
23699 ▷ ▷ ▷ -partition 2 91 91 \
23700 ▷ ▷ -end
23701 ▷ $(ORBITER) -v 3 \
23702 ▷ ▷ -draw_layered_graph \
23703 ▷ ▷ ▷ orbit_PGL_5.3.on.andre.3.layered.graph \
23704 ▷ ▷ ▷ -place \
23705 ▷ ▷ ▷ -radius 250 -spanning_tree -embedded -nodes_empty \
23706 ▷ ▷ ▷ -line_width 1.1 -x_stretch 2.4 -scale 0.15 \
23707 ▷ ▷ -end
23708 ▷ $(PDFLATEX) orbit_PGL_5.3.on.andre.3.draw.tex
23709 ▷ $(OPEN) orbit_PGL_5.3.on.andre.3.draw.pdf
23710 ▷ $(PDFLATEX) group_of_plane.plane_catalogue.q3.k2.1.orbits_report.tex
23711 ▷ $(OPEN) group_of_plane.plane_catalogue.q3.k2.1.orbits_report.pdf
23712 ▷ $(PDFLATEX) group_of_plane.plane_catalogue.q3.k2.1.stab.pt.92.report.tex
23713 ▷ $(OPEN) group_of_plane.plane_catalogue.q3.k2.1.stab.pt.92.report.pdf
23714
23715
23716
23717

```

```

23718 create_translation_plane.16.4.0:
23719 ▷ $(ORBITER) -v 3 \
23720 ▷ ▷ -define F -finite_field -q 4 -end \
23721 ▷ ▷ -define G -linear_group -PGGL 4 F -end \
23722 ▷ ▷ -define G1 -linear_group -PGGL 5 F -end \
23723 ▷ ▷ -define S -spread -kernel_field F \
23724 ▷ ▷ ▷ -group G -k 2 -catalogue 0 \
23725 ▷ ▷ ▷ -end \
23726 ▷ ▷ -define T -translation_plane S G G1 -end \
23727 ▷ ▷ -with T -do -translation_plane_activity \
23728 ▷ ▷ ▷ -export_incma \
23729 ▷ ▷ -end
23730 ▷ $(ORBITER) -v 2 \
23731 ▷ ▷ -draw_matrix \
23732 ▷ ▷ ▷ -input_csv_file plane_catalogue.q4.k2.0.incma.csv \
23733 ▷ ▷ ▷ -box_width 6 -bit_depth 8 \
23734 ▷ ▷ ▷ -partition 4 273 273 \
23735 ▷ ▷ -end
23736 ▷ $(OPEN) plane_catalogue.q4.k2.0.incma_draw.bmp
23737
23738
23739 #0 : "1200", // Hall spread
23740 #1 : "81600", // Desarguesian spread
23741 #2 : "576", // Semifield spread
23742
23743
23744 create_translation_plane.16.2.0:
23745 ▷ $(ORBITER) -v 3 \
23746 ▷ ▷ -define F -finite_field -q 2 -end \
23747 ▷ ▷ -define G -linear_group -PGL 8 F -end \
23748 ▷ ▷ -define G1 -linear_group -PGL 9 F -end \
23749 ▷ ▷ -define S -spread -kernel_field F \
23750 ▷ ▷ ▷ -group G -k 4 -catalogue 0 \
23751 ▷ ▷ ▷ -end \
23752 ▷ ▷ -define T -translation_plane S G G1 -end \
23753 ▷ ▷ -with T -do -translation_plane_activity \
23754 ▷ ▷ ▷ -export_incma \
23755 ▷ ▷ -end
23756 ▷ $(ORBITER) -v 2 \
23757 ▷ ▷ -draw_matrix \
23758 ▷ ▷ ▷ -input_csv_file plane_catalogue.q2.k4.0.incma.csv \
23759 ▷ ▷ ▷ -box_width 6 -bit_depth 8 \
23760 ▷ ▷ ▷ -partition 4 273 273 \
23761 ▷ ▷ -end
23762 ▷ $(OPEN) plane_catalogue.q2.k4.0.incma_draw.bmp
23763
23764 #0 : "1008",
23765 #1 : "1008",
23766 #2 : "1728",
23767 #3 : "216",
23768 #4 : "360",
23769 #5 : "288",
23770 #6 : "3600",
23771 #7 : "244800",
23772
23773 RREF_plane.16.2.0.rank_of_incma:
23774 ▷ $(ORBITER) -v 2 \
23775 ▷ ▷ -define F -finite_field -q 2 -end \
23776 ▷ ▷ -define v -vector -field F \

```

```

23777 ▷ ▷ ▷ -file plane_catalogue.q2.k4.0.incma.csv \
23778 ▷ ▷ -end \
23779 ▷ ▷ -with F -do -finite_field_activity \
23780 ▷ ▷ -RREF v \
23781 ▷ ▷ -end
23782
23783 # 2-rank is 106, so the plane is Lorimer-Rahilly
23784
23785
23786 create_translation_plane.25.14.rank:
23787 ▷ $(ORBITER) -v 3 \
23788 ▷ ▷ -define F -finite_field -q 5 -end \
23789 ▷ ▷ -define G -linear_group -PGL 4 F -end \
23790 ▷ ▷ -define G1 -linear_group -PGL 5 F -end \
23791 ▷ ▷ -define S -spread -kernel_field F \
23792 ▷ ▷ ▷ -group G -k 2 -catalogue 14 \
23793 ▷ ▷ ▷ -end \
23794 ▷ ▷ -define T -translation_plane S G G1 -end \
23795 ▷ ▷ -with T -do -translation_plane_activity \
23796 ▷ ▷ ▷ -export_incma \
23797 ▷ ▷ -end
23798 ▷ $(ORBITER) -v 2 \
23799 ▷ ▷ -define F -finite_field -q 2 -end \
23800 ▷ ▷ -define v -vector -field F \
23801 ▷ ▷ ▷ -file plane_catalogue.q5.k2.14.incma.csv \
23802 ▷ ▷ -end \
23803 ▷ ▷ -with F -do -finite_field_activity \
23804 ▷ ▷ ▷ -RREF v \
23805 ▷ ▷ -end
23806
23807
23808
23809 create_translation_plane.27.Rao.Rao:
23810 ▷ $(ORBITER) -v 3 \
23811 ▷ ▷ -define F -finite_field -q 3 -end \
23812 ▷ ▷ -define SS -vector -dense $(SPREAD_SET_27.RAO.RAO) -end \
23813 ▷ ▷ -define G -linear_group -PGL 6 F -end \
23814 ▷ ▷ -define G1 -linear_group -PGL 7 F -end \
23815 ▷ ▷ -define S -spread -kernel_field F \
23816 ▷ ▷ ▷ -group G -k 3 -spread_set "Rao.Rao" "Rao\Rao" SS \
23817 ▷ ▷ ▷ -end \
23818 ▷ ▷ -define T -translation_plane S G G1 -end \
23819 ▷ ▷ -with T -do -translation_plane_activity \
23820 ▷ ▷ ▷ -export_incma \
23821 ▷ ▷ -end
23822
23823
23824 # creates plane_Rao_Rao_incma.csv
23825
23826
23827 RREF_Rao_Rao_plane_incma_rank:
23828 ▷ $(ORBITER) -v 2 \
23829 ▷ ▷ -define F -finite_field -q 3 -end \
23830 ▷ ▷ -define v -vector -field F \
23831 ▷ ▷ ▷ -file plane_Rao_Rao_incma.csv \
23832 ▷ ▷ -end \
23833 ▷ ▷ -with F -do -finite_field_activity \
23834 ▷ ▷ ▷ -RREF v \
23835 ▷ ▷ -end

```

```

23836
23837 # 3-rank is 271, so the Rao / Rao plane is Moorhouse IV.
23838
23839
23840 create_translation_plane.27.p_rank.of_incidence_matrix:
23841 ▷ $(ORBITER) -v 3 \
23842 ▷ ▷ -define F -finite_field -q 3 -end \
23843 ▷ ▷ -define G -linear_group -PGL 6 F -end \
23844 ▷ ▷ -define G1 -linear_group -PGL 7 F -end \
23845 ▷ ▷ -define S -spread -kernel_field F \
23846 ▷ ▷ ▷ -group G -k 3 -catalogue 6 \
23847 ▷ ▷ ▷ -end \
23848 ▷ ▷ -define T -translation_plane S G G1 -end \
23849 ▷ ▷ -with T -do -translation_plane_activity \
23850 ▷ ▷ ▷ -p_rank 3 \
23851 ▷ ▷ -end
23852
23853 # OCN : 3-rank : spread stab : pt-orb : line-orb : Moorhouse list
23854 # 0 : 217 : 766584 : 757 : 757 : flag trans : desarguesian = Moorhouse I
23855 # 1 : 262 : 2106 : 1,27,729 : 1,27,729 : generalized twisted field = Moorhouse
      II
23856 # 2 : 268 : 1014 : 2,26,729 : 1,54,702 : Andre = Moorhouse VII
23857 # 3 : 273 : 81 : 1,27,729 : 1,27,729 : Sherk = Moorhouse V
23858 # 4 : 274 : 1092 : 28,729 : 1,756 : Hering = Moorhouse III
23859 # 5 : 271 : 84 : 28,729 : 1,756 : flag trans : Moorhouse IV = Rao Rao
23860 # 6 : 265 : 84 : 28,729 : 1,756 : flag trans : Moorhouse VI
23861
23862 # so, Rao / Rao is OCN=5
23863
23864
23865
23866 create_translation_plane.27.5.Rao.Rao:
23867 ▷ $(ORBITER) -v 3 \
23868 ▷ ▷ -define F -finite_field -q 3 -end \
23869 ▷ ▷ -define G -linear_group -PGL 6 F -end \
23870 ▷ ▷ -define G1 -linear_group -PGL 7 F -end \
23871 ▷ ▷ -define S -spread -kernel_field F \
23872 ▷ ▷ ▷ -group G -k 3 -catalogue 5 \
23873 ▷ ▷ ▷ -end \
23874 ▷ ▷ -define T -translation_plane S G G1 -end \
23875 ▷ ▷ -with T -do -translation_plane_activity \
23876 ▷ ▷ ▷ -export_incma \
23877 ▷ ▷ -end \
23878 ▷ ▷ -with T -do -translation_plane_activity \
23879 ▷ ▷ ▷ -report \
23880 ▷ ▷ -end \
23881 ▷ ▷ -define A -linear_group -import_group_of_plane T -end \
23882 ▷ ▷ -define Orb -orbits -group A \
23883 ▷ ▷ ▷ -on_points \
23884 ▷ ▷ -end
23885 ▷ $(PDFLATEX) plane_catalogue.q3.k3.5.report.tex
23886 ▷ $(OPEN) plane_catalogue.q3.k3.5.report.pdf
23887
23888
23889
23890
23891 translation_plane.27.4.orbits:
23892 ▷ $(ORBITER) -v 3 \
23893 ▷ ▷ -define F -finite_field -q 3 -end \

```

```

23894 ▷ ▷ -define G -linear_group -PGL 6 F -end \
23895 ▷ ▷ -define G1 -linear_group -PGL 7 F -end \
23896 ▷ ▷ -define S -spread -kernel.field F \
23897 ▷ ▷ ▷ -group G -k 3 -catalogue 4 \
23898 ▷ ▷ ▷ -end \
23899 ▷ ▷ -define T -translation_plane S G G1 -end \
23900 ▷ ▷ -with T -do -translation_plane_activity \
23901 ▷ ▷ ▷ -export_incma \
23902 ▷ ▷ -end \
23903 ▷ ▷ -with T -do -translation_plane_activity \
23904 ▷ ▷ ▷ -report \
23905 ▷ ▷ -end \
23906 ▷ ▷ -define A -linear_group -import_group_of_plane T -end \
23907 ▷ ▷ -define Orb -orbits -group A \
23908 ▷ ▷ ▷ -on_points \
23909 ▷ ▷ -end
23910
23911
23912
23913 translation_plane.27.5.orbits:
23914 ▷ $(ORBITER) -v 3 \
23915 ▷ ▷ -define F -finite_field -q 3 -end \
23916 ▷ ▷ -define G -linear_group -PGL 6 F -end \
23917 ▷ ▷ -define G1 -linear_group -PGL 7 F -end \
23918 ▷ ▷ -define S -spread -kernel.field F \
23919 ▷ ▷ ▷ -group G -k 3 -catalogue 5 \
23920 ▷ ▷ ▷ -end \
23921 ▷ ▷ -define T -translation_plane S G G1 -end \
23922 ▷ ▷ -with T -do -translation_plane_activity \
23923 ▷ ▷ ▷ -export_incma \
23924 ▷ ▷ -end \
23925 ▷ ▷ -with T -do -translation_plane_activity \
23926 ▷ ▷ ▷ -report \
23927 ▷ ▷ -end \
23928 ▷ ▷ -define A -linear_group -import_group_of_plane T -end \
23929 ▷ ▷ -define Orb -orbits -group A \
23930 ▷ ▷ ▷ -on_points \
23931 ▷ ▷ -end
23932
23933
23934 translation_plane.27.6.orbits:
23935 ▷ $(ORBITER) -v 3 \
23936 ▷ ▷ -define F -finite_field -q 3 -end \
23937 ▷ ▷ -define G -linear_group -PGL 6 F -end \
23938 ▷ ▷ -define G1 -linear_group -PGL 7 F -end \
23939 ▷ ▷ -define S -spread -kernel.field F \
23940 ▷ ▷ ▷ -group G -k 3 -catalogue 6 \
23941 ▷ ▷ ▷ -end \
23942 ▷ ▷ -define T -translation_plane S G G1 -end \
23943 ▷ ▷ -with T -do -translation_plane_activity \
23944 ▷ ▷ ▷ -export_incma \
23945 ▷ ▷ -end \
23946 ▷ ▷ -with T -do -translation_plane_activity \
23947 ▷ ▷ ▷ -report \
23948 ▷ ▷ -end \
23949 ▷ ▷ -define A -linear_group -import_group_of_plane T -end \
23950 ▷ ▷ -define Orb -orbits -group A \
23951 ▷ ▷ ▷ -on_points \
23952 ▷ ▷ -end

```

```

23953
23954
23955
23956
23957
23958
23959
23960
23961
23962
23963 #####
23964 # Section 15.3: Packings
23965
23966
23967 SECTION_PACKINGS:
23968
23969
23970 test_15_3:
23971 ▷ make PG_3_3_spread_table
23972 ▷ make spread_table_PG_3_5_regular
23973 ▷ make PGL_4_3_classes
23974 ▷ make PG_3_3_assume_5_graph
23975 ▷ make PG_3_3_assume_5_fpc0_lo_cliques
23976 ▷ make PG_3_3_assume_5_fpc1_lo_cliques
23977 ▷ make PG_3_3_assume_5_fpc2_lo_cliques
23978 ▷ make PG_3_3_assume_5_read_solution
23979 ▷ make PG_3_3_assume_5_packings_classify
23980
23981
23982
23983 PG_3_3_spread_table:
23984 ▷ - mkdir SPREAD_TABLES_3
23985 ▷ $(ORBITER) -v 5 \
23986 ▷ ▷ -define Control -poset_classification_control \
23987 ▷ ▷ ▷ -problem_label spreads_PG_3_3 \
23988 ▷ ▷ ▷ -W \
23989 ▷ ▷ -end \
23990 ▷ ▷ -define F -finite_field -q 3 -end \
23991 ▷ ▷ -define P3 -projective_space -n 3 -field F -v 0 -end \
23992 ▷ ▷ -define P5 -projective_space -n 5 -field F -v 0 -end \
23993 ▷ ▷ -define T -spread_table -space P3 -rk 2 \
23994 ▷ ▷ ▷ -iso_types "0,1" \
23995 ▷ ▷ ▷ -path "SPREAD_TABLES_3/" \
23996 ▷ ▷ ▷ -control Control \
23997 ▷ ▷ ▷ -end \
23998
23999
24000
24001
24002
24003 spread_table_PG_3_5_regular:
24004 ▷ - mkdir SPREAD_TABLES_5_REG
24005 ▷ $(ORBITER) -v 6 \
24006 ▷ ▷ -define Control -poset_classification_control \
24007 ▷ ▷ ▷ -problem_label spreads_PG_3_5 \
24008 ▷ ▷ -end \
24009 ▷ ▷ -define F -finite_field -q 5 -end \
24010 ▷ ▷ -define P3 -projective_space -n 3 -field F -end \
24011 ▷ ▷ -define T -spread_table -space P3 -rk 2 \

```



```

24012 ▷ ▷ ▷ -iso_types "12" \
24013 ▷ ▷ ▷ -path "SPREAD.TABLES.5.REG/" \
24014 ▷ ▷ ▷ -control Control \
24015 ▷ ▷ ▷ -end \
24016 ▷ ▷ -print_symbols
24017
24018
24019 # 21 isomorphism types of spreads in PG(3,5)
24020 # 12 is the index of the regular spread in the classification of spreads
24021 # 12/9/2020: 34 sec on Mac
24022 # time: 0:19 1/11/2024 on Mac
24023 # User time: 1:17 12/20/2025 on Mac
24024 # 155000 spreads
24025
24026
24027
24028
24029
24030 PGL_4_3_classes:
24031 ▷ $(ORBITER) -v 2 \
24032 ▷ ▷ -define Do -draw_options \
24033 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
24034 ▷ ▷ -end \
24035 ▷ ▷ -define G -linear_group -PGL 4 3 -end \
24036 ▷ ▷ -with G -do \
24037 ▷ ▷ -group_theoretic_activity \
24038 ▷ ▷ ▷ -report Do \
24039 ▷ ▷ -end \
24040 ▷ ▷ -with G -do \
24041 ▷ ▷ -group_theoretic_activity \
24042 ▷ ▷ ▷ -classes \
24043 ▷ ▷ -end
24044 ▷ $(PDFLATEX) PGL_4_3_report_classes_of_elements.tex
24045 ▷ $(OPEN) PGL_4_3_report_classes_of_elements.pdf
24046
24047
24048
24049
24050
24051
24052 PG_3_3_assume_5_graph:
24053 ▷ $(ORBITER) -v 5 \
24054 ▷ ▷ -define Control -poset_classification_control \
24055 ▷ ▷ ▷ -problem_label spreads_PG_3_3 \
24056 ▷ ▷ ▷ -W \
24057 ▷ ▷ -end \
24058 ▷ ▷ -define Control_Fix -poset_classification_control \
24059 ▷ ▷ ▷ -problem_label N5_fixp_cliques \
24060 ▷ ▷ ▷ -W \
24061 ▷ ▷ -end \
24062 ▷ ▷ -define F -finite_field -q 3 -end \
24063 ▷ ▷ -define P3 -projective_space -n 3 -field F -v 0 -end \
24064 ▷ ▷ -define P5 -projective_space -n 5 -field F -v 0 -end \
24065 ▷ ▷ -define T -spread_table -space P3 -rk 2 \
24066 ▷ ▷ ▷ -iso_types "0,1" \
24067 ▷ ▷ ▷ -path "SPREAD.TABLES.3/" \
24068 ▷ ▷ ▷ -control Control \
24069 ▷ ▷ ▷ -end \
24070 ▷ ▷ -define PC -packing_classify P3 P5 T \

```

```

24071 ▷ ▷ -define PW -packing_with_symmetry_assumption PC \
24072 ▷ ▷ ▷ ▷ -H "H5" -PGL 4 3 -subgroup_by_generators \
24073 ▷ ▷ ▷ ▷ ▷ "5" 5 1 $(PGL_4_3_CLASS_5A_REP) -end \
24074 ▷ ▷ ▷ ▷ -N "N5" -PGL 4 3 -subgroup_by_generators \
24075 ▷ ▷ ▷ ▷ ▷ "N5" 160 3 $(PGL_4_3_CLASS_5A_N_GENS) -end \
24076 ▷ ▷ ▷ -end \
24077 ▷ ▷ -define PWF -packing_choose_fixed_points PW 3 Control_Fix \
24078 ▷ ▷ -define Long -packing_long_orbits PWF \
24079 ▷ ▷ ▷ -orbit_length 5 \
24080 ▷ ▷ ▷ -list_of_cases_from_file N5_fixp_cliques.csv \
24081 ▷ ▷ ▷ -create_graphs \
24082 ▷ ▷ ▷ -end \
24083 ▷ ▷ -print_symbols
24084
24085 # 3 orbits on 3 fixpoints
24086
24087 #-rw-r--r-- 1 betten staff 317 Dec 30 14:20 H5_fpc0_lo.graph
24088 #-rw-r--r-- 1 betten staff 602 Dec 30 14:20 H5_fpc1_lo.graph
24089 #-rw-r--r-- 1 betten staff 347 Dec 30 14:20 H5_fpc2_lo.graph
24090
24091
24092 PG_3_3_assume_5_fpc0_lo_cliques:
24093 ▷ $(ORBITER) -v 2 \
24094 ▷ ▷ -define G -graph -load H5_fpc0_lo.graph -end \
24095 ▷ ▷ -with G -do \
24096 ▷ ▷ ▷ -graph_theoretic_activity \
24097 ▷ ▷ ▷ -find_cliques -target_size 2 -output_file H5_fpc0_lo_cliques.csv -end \
24098 ▷ ▷ -print_symbols
24099
24100
24101 #colored_graph_cliques::all_cliques_black_and_white after CG->all_cliques_of_size
    _k_ignore_colors, nb_cliques = 0
24102
24103
24104 PG_3_3_assume_5_fpc1_lo_cliques:
24105 ▷ $(ORBITER) -v 2 \
24106 ▷ ▷ -define G -graph -load H5_fpc1_lo.graph -end \
24107 ▷ ▷ -with G -do \
24108 ▷ ▷ ▷ -graph_theoretic_activity \
24109 ▷ ▷ ▷ -find_cliques -target_size 2 -output_file H5_fpc1_lo_cliques.csv -end \
24110 ▷ ▷ -print_symbols
24111
24112 #colored_graph_cliques::all_cliques_black_and_white after CG->all_cliques_of_size
    _k_ignore_colors, nb_cliques = 4
24113
24114
24115
24116 PG_3_3_assume_5_fpc2_lo_cliques:
24117 ▷ $(ORBITER) -v 2 \
24118 ▷ ▷ -define G -graph -load H5_fpc2_lo.graph -end \
24119 ▷ ▷ -with G -do \
24120 ▷ ▷ ▷ -graph_theoretic_activity \
24121 ▷ ▷ ▷ -find_cliques -target_size 2 -output_file H5_fpc2_lo_cliques.csv -end \
24122 ▷ ▷ -print_symbols
24123
24124 #colored_graph_cliques::all_cliques_black_and_white after CG->all_cliques_of_size
    _k_ignore_colors, nb_cliques = 10
24125
24126 PG_3_3_assume_5_read_solution:

```

```

24127 ▷ $(ORBITER) -v 5 \
24128 ▷ ▷ -define Control -poset_classification_control \
24129 ▷ ▷ ▷ -problem_label spreads_PG_3_3 \
24130 ▷ ▷ ▷ -W \
24131 ▷ ▷ -end \
24132 ▷ ▷ -define Control_Fix -poset_classification_control \
24133 ▷ ▷ ▷ -problem_label N5_fixp_cliques \
24134 ▷ ▷ ▷ -W \
24135 ▷ ▷ -end \
24136 ▷ ▷ -define F -finite_field -q 3 -end \
24137 ▷ ▷ -define P3 -projective_space -n 3 -field F -v 0 -end \
24138 ▷ ▷ -define P5 -projective_space -n 5 -field F -v 0 -end \
24139 ▷ ▷ -define T -spread_table -space P3 -rk 2 \
24140 ▷ ▷ ▷ -iso_types "0,1" \
24141 ▷ ▷ ▷ -path "SPREAD_TABLES_3/" \
24142 ▷ ▷ ▷ -control Control \
24143 ▷ ▷ ▷ -end \
24144 ▷ ▷ -define PC -packing_classify P3 P5 T \
24145 ▷ ▷ -define PW -packing_with_symmetry_assumption PC \
24146 ▷ ▷ ▷ -H "H5" -PGL 4 3 -subgroup_by_generators \
24147 ▷ ▷ ▷ ▷ "5" 5 1 $(PGL_4_3_CLASS_5A_REP) -end \
24148 ▷ ▷ ▷ -N "N5" -PGL 4 3 -subgroup_by_generators \
24149 ▷ ▷ ▷ ▷ "N5" 160 3 $(PGL_4_3_CLASS_5A_N_GEN) -end \
24150 ▷ ▷ ▷ -end \
24151 ▷ ▷ -define PWF -packing_choose_fixed_points PW 3 Control_Fix \
24152 ▷ ▷ -define Long -packing_long_orbits PWF \
24153 ▷ ▷ ▷ -orbit_length 5 \
24154 ▷ ▷ ▷ -list_of_cases_from_file N5_fixp_cliques.csv \
24155 ▷ ▷ ▷ -read_solutions \
24156 ▷ ▷ ▷ -end \
24157 ▷ ▷ -print_symbols
24158
24159 # we find 14 packings.
24160 #-rw-r--r-- 1 betten staff 1131 Dec 31 00:53 H5_packings.csv
24161 #-rw-r--r-- 1 betten staff 581 Dec 31 00:53 H5_packings_flag_orbits.csv
24162
24163
24164 PG_3_3_assume_5_packings_classify:
24165 ▷ $(ORBITER) -v 2 \
24166 ▷ ▷ -define C -combinatorial_object \
24167 ▷ ▷ ▷ -label PG_3_3_assuming_5 \
24168 ▷ ▷ ▷ "${\rm PG}_3_3\_assuming_5" \
24169 ▷ ▷ ▷ -file_of_packings_through_spread_table \
24170 ▷ ▷ ▷ ▷ H5_packings.csv \
24171 ▷ ▷ ▷ ▷ SPREAD_TABLES_3/spread_9_table.csv \
24172 ▷ ▷ ▷ ▷ SpreadRep \
24173 ▷ ▷ ▷ ▷ 3 \
24174 ▷ ▷ -end \
24175 ▷ ▷ -define F -finite_field -q 3 -end \
24176 ▷ ▷ -define P3 -projective_space -n 3 -field F -v 0 -end \
24177 ▷ ▷ -with C -do \
24178 ▷ ▷ -combinatorial_object_activity \
24179 ▷ ▷ ▷ -canonical_form_PG P3 \
24180 ▷ ▷ ▷ -nauty_control -nauty_log "PG_3_3_packings_5_nauty_log.txt" -end \
24181 ▷ ▷ ▷ -end \
24182 ▷ ▷ -end \
24183 ▷ ▷ -define Doinc -draw_incidence_structure_options \
24184 ▷ ▷ ▷ -width 40 -width_10 6 \
24185 ▷ ▷ -end \

```

```

24186 ▷ ▷ -with C -do \
24187 ▷ ▷ -combinatorial_object_activity \
24188 ▷ ▷ ▷ -post_processing \
24189 ▷ ▷ -end \
24190 ▷ ▷ -with C -do \
24191 ▷ ▷ -combinatorial_object_activity \
24192 ▷ ▷ ▷ -report \
24193 ▷ ▷ ▷ ▷ -export_flag_orbits \
24194 ▷ ▷ ▷ ▷ -show_TDO \
24195 ▷ ▷ ▷ ▷ -show_TDA \
24196 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
24197 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
24198 ▷ ▷ ▷ -end \
24199 ▷ ▷ -end
24200 ▷ $(PDFLATEX) PG_3_3.assuming_5_classification.tex
24201 ▷ $(OPEN) PG_3_3.assuming_5_classification.pdf
24202
24203
24204
24205 # 7 isomorphism types
24206
24207
24208
24209 #####
24210 # Section 15.4: BLT-sets
24211
24212 SECTION.BLT.SETS:
24213
24214
24215 test_15.4:
24216 ▷ make BLT_5.1
24217 ▷ make BLT_5.1.export_gap
24218 ▷ make BLT_5.Linear
24219 ▷ make BLT_9.0
24220 ▷ make BLT_9.Kantor1
24221 ▷ make BLT_11.0
24222 ▷ make BLT_11.Fisher
24223 ▷ make BLT_11.Mondello
24224 ▷ make BLT_11.FTWKB
24225 ▷ make BLT_13.Kantor2
24226 ▷ make BLT_67.4
24227 ▷ make BLT_67.4.export_gap
24228 ▷ make BLT_9.deep_search
24229 ▷ make BLT_11.deep_search
24230 ▷ make BLT_13.deep_search
24231 ▷ make BLT_13.classify_starter
24232 ▷ make BLT_13.cliques
24233 ▷ make BLT_13.isomorph_read.DB
24234 ▷ make BLT_13.isomorph_read.solutions
24235 ▷ make BLT_13.isomorph_stabilizer_orbits
24236 ▷ make BLT_13.isomorph_testing
24237 ▷ make blt.set.export_gap.all
24238
24239
24240
24241 BLT_5.1:
24242 ▷ $(ORBITER) -v 4 \
24243 ▷ ▷ -define F -finite.field -q 5 -end \
24244 ▷ ▷ -define 0 -orthogonal.space 0 5 F -create_extension_fields -end \

```

```

24245 ▷ ▷ -define BLT -BLT_set \
24246 ▷ ▷ ▷ -space 0 -catalogue 1 -invariants \
24247 ▷ ▷ -end \
24248 ▷ ▷ -with BLT -do -blt_set_activity \
24249 ▷ ▷ ▷ -report \
24250 ▷ ▷ -end
24251 ▷ $(PDFLATEX) BLT_catalogue.q5_iso1.tex
24252 ▷ $(OPEN) BLT_catalogue.q5_iso1.pdf
24253
24254
24255 #blt_set_domain::test.flock.condition done
24256 #outcome : ( 0^10 )
24257 #make: *** [BLT_5_1] Segmentation fault: 11
24258
24259
24260
24261 BLT_5_1_export_gap:
24262 ▷ $(ORBITER) -v 2 \
24263 ▷ ▷ -define F -finite_field -q 5 -end \
24264 ▷ ▷ -define O -orthogonal_space 0 5 F -create_extension_fields -end \
24265 ▷ ▷ -define BLT -BLT_set \
24266 ▷ ▷ ▷ -space 0 -catalogue 1 \
24267 ▷ ▷ -end \
24268 ▷ ▷ -with BLT -do -blt_set_activity \
24269 ▷ ▷ ▷ -export_gap \
24270 ▷ ▷ -end
24271
24272
24273 BLT_5_Linear:
24274 ▷ $(ORBITER) -v 2 \
24275 ▷ ▷ -define F -finite_field -q 5 -end \
24276 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
24277 ▷ ▷ -define BLT -BLT_set \
24278 ▷ ▷ ▷ -space 0 -family "Linear" -invariants \
24279 ▷ ▷ -end \
24280 ▷ ▷ -with BLT -do -blt_set_activity \
24281 ▷ ▷ ▷ -report \
24282 ▷ ▷ -end
24283 ▷ $(PDFLATEX) BLT_Linear.q5.tex
24284 ▷ $(OPEN) BLT_Linear.q5.pdf
24285
24286
24287 BLT_9_0:
24288 ▷ $(ORBITER) -v 10 \
24289 ▷ ▷ -define F -finite_field -q 9 -end \
24290 ▷ ▷ -define O -orthogonal_space 0 5 F -create_extension_fields -end \
24291 ▷ ▷ -define BLT -BLT_set \
24292 ▷ ▷ ▷ -space 0 -catalogue 0 -invariants \
24293 ▷ ▷ -end \
24294 ▷ ▷ -with BLT -do -blt_set_activity \
24295 ▷ ▷ ▷ -report \
24296 ▷ ▷ -end
24297 ▷ $(PDFLATEX) BLT_catalogue.q9_iso0.tex
24298 ▷ $(OPEN) BLT_catalogue.q9_iso0.pdf
24299
24300
24301
24302
24303 BLT_9_Kantor1:

```

```

24304 ▷ $(ORBITER) -v 2 \
24305 ▷ ▷ -define F -finite.field -q 9 -end \
24306 ▷ ▷ -define O -orthogonal.space 0 5 F -end \
24307 ▷ ▷ -define BLT -BLT.set \
24308 ▷ ▷ ▷ -space 0 -family "Kantor1" -invariants \
24309 ▷ ▷ -end \
24310 ▷ ▷ -with BLT -do -blt.set.activity \
24311 ▷ ▷ ▷ -report \
24312 ▷ ▷ -end
24313 ▷ $(PDFLATEX) BLT_Kantor1.q9.tex
24314 ▷ $(OPEN) BLT_Kantor1.q9.pdf
24315
24316
24317
24318
24319 BLT_11_0:
24320 ▷ $(ORBITER) -v 2 \
24321 ▷ ▷ -define F -finite.field -q 11 -end \
24322 ▷ ▷ -define O -orthogonal.space 0 5 F -create.extension.fields -end \
24323 ▷ ▷ -define BLT -BLT.set \
24324 ▷ ▷ ▷ -space 0 -catalogue 0 -invariants \
24325 ▷ ▷ -end \
24326 ▷ ▷ -with BLT -do -blt.set.activity \
24327 ▷ ▷ ▷ -report \
24328 ▷ ▷ -end
24329 ▷ $(PDFLATEX) BLT_catalogue.q11_iso0.tex
24330 ▷ $(OPEN) BLT_catalogue.q11_iso0.pdf
24331
24332
24333 BLT_11_Fisher:
24334 ▷ $(ORBITER) -v 2 \
24335 ▷ ▷ -define F -finite.field -q 11 -end \
24336 ▷ ▷ -define O -orthogonal.space 0 5 F -end \
24337 ▷ ▷ -define BLT -BLT.set \
24338 ▷ ▷ ▷ -space 0 -family "Fisher" -invariants \
24339 ▷ ▷ -end \
24340 ▷ ▷ -with BLT -do -blt.set.activity \
24341 ▷ ▷ ▷ -report \
24342 ▷ ▷ -end
24343 ▷ $(PDFLATEX) BLT_Fisher.q11.tex
24344 ▷ $(OPEN) BLT_Fisher.q11.pdf
24345
24346 BLT_11_Mondello:
24347 ▷ $(ORBITER) -v 2 \
24348 ▷ ▷ -define F -finite.field -q 11 -end \
24349 ▷ ▷ -define O -orthogonal.space 0 5 F -end \
24350 ▷ ▷ -define BLT -BLT.set \
24351 ▷ ▷ ▷ -space 0 -family "Mondello" -invariants \
24352 ▷ ▷ -end \
24353 ▷ ▷ -with BLT -do -blt.set.activity \
24354 ▷ ▷ ▷ -report \
24355 ▷ ▷ -end
24356 ▷ $(PDFLATEX) BLT_Mondello.q11.tex
24357 ▷ $(OPEN) BLT_Mondello.q11.pdf
24358
24359
24360 BLT_11_FTWKB:
24361 ▷ $(ORBITER) -v 2 \
24362 ▷ ▷ -define F -finite.field -q 11 -end \

```

```

24363 ▷ ▷ -define 0 -orthogonal_space 0 5 F -end \
24364 ▷ ▷ -define BLT -BLT_set \
24365 ▷ ▷ ▷ -space 0 -family "FTWKB" -invariants \
24366 ▷ ▷ -end \
24367 ▷ ▷ -with BLT -do -blt_set_activity \
24368 ▷ ▷ ▷ -report \
24369 ▷ ▷ -end
24370 ▷ $(PDFLATEX) BLT_FTWKB_q11.tex
24371 ▷ $(OPEN) BLT_FTWKB_q11.pdf
24372
24373
24374 # for K2, q must be congruent to 2 or 3 mod 5
24375 BLT_13.Kantor2:
24376 ▷ $(ORBITER) -v 2 \
24377 ▷ ▷ -define F -finite_field -q 13 -end \
24378 ▷ ▷ -define 0 -orthogonal_space 0 5 F -end \
24379 ▷ ▷ -define BLT -BLT_set \
24380 ▷ ▷ ▷ -space 0 -family "Kantor2" -invariants \
24381 ▷ ▷ -end \
24382 ▷ ▷ -with BLT -do -blt_set_activity \
24383 ▷ ▷ ▷ -report \
24384 ▷ ▷ -end
24385 ▷ $(PDFLATEX) BLT_Kantor2.q13.tex
24386 ▷ $(OPEN) BLT_Kantor2.q13.pdf
24387
24388
24389 BLT_67.4:
24390 ▷ $(ORBITER) -v 4 \
24391 ▷ ▷ -define F -finite_field -q 67 -end \
24392 ▷ ▷ -define 0 -orthogonal_space 0 5 F -create_extension_fields -end \
24393 ▷ ▷ -define BLT -BLT_set \
24394 ▷ ▷ ▷ -space 0 -catalogue 4 -invariants \
24395 ▷ ▷ -end \
24396 ▷ ▷ -with BLT -do -blt_set_activity \
24397 ▷ ▷ ▷ -report \
24398 ▷ ▷ -end
24399
24400 #orthogonal_space_with_action::init_group before A->Known_groups->init_orthogonal
    _group_with_0
24401 # is slow
24402
24403 #knowledge_base::get_primitive_polynomial
24404 #knowledge_base::get_primitive_polynomial I do not have a polynomial
24405 #of that degree over that field
24406 #requested: degree 3 polynomial over GF(67)
24407
24408
24409
24410 BLT_67.4_export_gap:
24411 ▷ $(ORBITER) -v 2 \
24412 ▷ ▷ -define F -finite_field -q 67 -end \
24413 ▷ ▷ -define 0 -orthogonal_space 0 5 F -end \
24414 ▷ ▷ -define BLT -BLT_set \
24415 ▷ ▷ ▷ -space 0 -catalogue 4 \
24416 ▷ ▷ -end \
24417 ▷ ▷ -with BLT -do -blt_set_activity \
24418 ▷ ▷ ▷ -export_gap \
24419 ▷ ▷ -end
24420

```

```

24421
24422
24423
24424 BLT_9_deep_search:
24425 ▷ $(ORBITER) -v 2 \
24426 ▷ ▷ -define Do -draw.options \
24427 ▷ ▷ ▷ -xin 400000 \
24428 ▷ ▷ ▷ -xout 3000000 \
24429 ▷ ▷ ▷ -yout 3000000 \
24430 ▷ ▷ ▷ -y_stretch 0.5 \
24431 ▷ ▷ -end \
24432 ▷ ▷ -define F -finite_field -q 9 -end \
24433 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
24434 ▷ ▷ -define C -BLT_set_classifier \
24435 ▷ ▷ ▷ -orthogonal_space 0 -starter_size 10 \
24436 ▷ ▷ -end \
24437 ▷ ▷ -define Activity -poset_classification_activity \
24438 ▷ ▷ ▷ -report -draw.options Do -end \
24439 ▷ ▷ -end \
24440 ▷ ▷ -define Activity_export -poset_classification_activity \
24441 ▷ ▷ ▷ -export_level_to_cpp 10 -end \
24442 ▷ ▷ -with C -do -BLT_set_classify_activity \
24443 ▷ ▷ ▷ -compute_starter \
24444 ▷ ▷ ▷ ▷ -problem_label BLT.q9 \
24445 ▷ ▷ ▷ ▷ -W -depth 10 \
24446 ▷ ▷ ▷ -end \
24447 ▷ ▷ -end \
24448 ▷ ▷ -with C -do -BLT_set_classify_activity \
24449 ▷ ▷ ▷ -poset_classification_activity \
24450 ▷ ▷ ▷ ▷ Activity \
24451 ▷ ▷ -end \
24452 ▷ ▷ -with C -do -BLT_set_classify_activity \
24453 ▷ ▷ ▷ -poset_classification_activity \
24454 ▷ ▷ ▷ ▷ Activity_export \
24455 ▷ ▷ -end
24456 ▷ $(PDFLATEX) BLT.q9_poset.tex
24457 ▷ $(OPEN) BLT.q9_poset.pdf
24458
24459
24460
24461 BLT_11_deep_search:
24462 ▷ $(ORBITER) -v 2 \
24463 ▷ ▷ -define F -finite_field -q 11 -end \
24464 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
24465 ▷ ▷ -define C -BLT_set_classifier \
24466 ▷ ▷ ▷ -orthogonal_space 0 -starter_size 12 \
24467 ▷ ▷ -end \
24468 ▷ ▷ -with C -do -BLT_set_classify_activity \
24469 ▷ ▷ ▷ -compute_starter \
24470 ▷ ▷ ▷ ▷ -problem_label BLT.q11 \
24471 ▷ ▷ ▷ ▷ -W -depth 12 \
24472 ▷ ▷ ▷ -end \
24473 ▷ ▷ -end
24474 ▷ #$(PDFLATEX) BLT.q11_poset.tex
24475 ▷ #$(OPEN) BLT.q11_poset.pdf
24476
24477
24478
24479

```



```

24480 BLT_13.deep_search:
24481 ▷ $(ORBITER) -v 2 \
24482 ▷ ▷ -define F -finite_field -q 13 -end \
24483 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
24484 ▷ ▷ -define C -BLT_set_classifier \
24485 ▷ ▷ ▷ -orthogonal_space 0 -starter_size 14 \
24486 ▷ ▷ -end \
24487 ▷ ▷ -with C -do -BLT_set_classify_activity \
24488 ▷ ▷ ▷ -compute_starter \
24489 ▷ ▷ ▷ ▷ -problem_label BLT_q13 \
24490 ▷ ▷ ▷ ▷ -W -depth 14 \
24491 ▷ ▷ ▷ -end \
24492 ▷ ▷ -end
24493 ▷ #$(PDFLATEX) BLT_q13_poset.tex
24494 ▷ #$(OPEN) BLT_q13_poset.pdf
24495
24496
24497
24498
24499 BLT_13.classify_starter:
24500 ▷ $(ORBITER) -v 2 \
24501 ▷ ▷ -define F -finite_field -q 13 -end \
24502 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
24503 ▷ ▷ -define C -BLT_set_classifier \
24504 ▷ ▷ ▷ -orthogonal_space 0 -starter_size 5 \
24505 ▷ ▷ -end \
24506 ▷ ▷ -with C -do -BLT_set_classify_activity \
24507 ▷ ▷ ▷ -compute_starter \
24508 ▷ ▷ ▷ ▷ -problem_label BLT_q13 \
24509 ▷ ▷ ▷ ▷ -W -depth 5 \
24510 ▷ ▷ ▷ -end \
24511 ▷ ▷ -end \
24512 ▷ ▷ -with C -do -BLT_set_classify_activity \
24513 ▷ ▷ ▷ -create_graphs \
24514 ▷ ▷ -end
24515
24516
24517
24518
24519 BLT_13.cliques:
24520 ▷ $(ORBITER) -v 2 \
24521 ▷ ▷ -loop L 0 38 1 \
24522 ▷ ▷ ▷ -define G -graph \
24523 ▷ ▷ ▷ ▷ -load BLT_q13_graph_5_%L.bin \
24524 ▷ ▷ ▷ -end \
24525 ▷ ▷ ▷ -with G -do \
24526 ▷ ▷ ▷ -graph_theoretic_activity \
24527 ▷ ▷ ▷ ▷ -find_cliques -rainbow -target_size 9 \
24528 ▷ ▷ ▷ -end \
24529 ▷ ▷ -end_loop L
24530
24531
24532 # 3 solutions:
24533 #BLT_q13_graph_5_0_sol.txt
24534 #BLT_q13_graph_5_0_sol.csv
24535
24536
24537
24538 BLT_13.isomorph_read.DB:

```

```

24539 ▷ $(ORBITER) -v 2 \
24540 ▷ ▷ -define F -finite_field -q 13 -end \
24541 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
24542 ▷ ▷ -define C -BLT_set_classifier \
24543 ▷ ▷ ▷ -orthogonal_space 0 -starter_size 5 \
24544 ▷ ▷ -end \
24545 ▷ ▷ -with C -do -BLT_set_classify_activity \
24546 ▷ ▷ ▷ -compute_starter \
24547 ▷ ▷ ▷ ▷ -problem_label BLT.q13 \
24548 ▷ ▷ ▷ ▷ -W -depth 5 \
24549 ▷ ▷ ▷ -end \
24550 ▷ ▷ -end \
24551 ▷ ▷ -with C -do -BLT_set_classify_activity \
24552 ▷ ▷ ▷ -isomorph \
24553 ▷ ▷ ▷ ▷ -prefix_iso "/BLT.q13" \
24554 ▷ ▷ ▷ ▷ -use_database_for_starter \
24555 ▷ ▷ ▷ ▷ -build_db \
24556 ▷ ▷ ▷ ▷ -solution_prefix "" \
24557 ▷ ▷ ▷ ▷ -base_fname "" \
24558 ▷ ▷ ▷ -end \
24559 ▷ ▷ -end
24560
24561 #blt_set_classify_activity_description::read_arguments
24562 #-isomorph -prefix_iso
24563 #blt_set_classify_activity_description::read_arguments unrecognized option ./BLT_
    q13
24564
24565
24566 BLT_13_isomorph_read_solutions:
24567 ▷ $(ORBITER) -v 2 \
24568 ▷ ▷ -define F -finite_field -q 13 -end \
24569 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
24570 ▷ ▷ -define C -BLT_set_classifier \
24571 ▷ ▷ ▷ -orthogonal_space 0 -starter_size 5 \
24572 ▷ ▷ -end \
24573 ▷ ▷ -with C -do -BLT_set_classify_activity \
24574 ▷ ▷ ▷ -compute_starter \
24575 ▷ ▷ ▷ ▷ -problem_label BLT.q13 \
24576 ▷ ▷ ▷ ▷ -W -depth 5 \
24577 ▷ ▷ ▷ -end \
24578 ▷ ▷ -end \
24579 ▷ ▷ -with C -do -BLT_set_classify_activity \
24580 ▷ ▷ ▷ -isomorph \
24581 ▷ ▷ ▷ ▷ -prefix_iso "/BLT.q13" \
24582 ▷ ▷ ▷ ▷ -use_database_for_starter \
24583 ▷ ▷ ▷ ▷ -read_solutions \
24584 ▷ ▷ ▷ ▷ -list_of_cases BLT.q13_list_of_cases.5_0.1.csv \
24585 ▷ ▷ ▷ ▷ -solution_prefix "" \
24586 ▷ ▷ ▷ ▷ -base_fname "BLT.q13_graph" \
24587 ▷ ▷ ▷ -end \
24588 ▷ ▷ -end
24589
24590
24591 BLT_13_isomorph_stabilizer_orbits:
24592 ▷ $(ORBITER) -v 2 \
24593 ▷ ▷ -define F -finite_field -q 13 -end \
24594 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
24595 ▷ ▷ -define C -BLT_set_classifier \
24596 ▷ ▷ ▷ -orthogonal_space 0 -starter_size 5 \

```

```

24597 ▷ ▷ -end \
24598 ▷ ▷ -with C -do -BLT_set_classify_activity \
24599 ▷ ▷ ▷ -compute_starter \
24600 ▷ ▷ ▷ ▷ -problem_label BLT_q13 \
24601 ▷ ▷ ▷ ▷ -W -depth 5 \
24602 ▷ ▷ ▷ -end \
24603 ▷ ▷ -end \
24604 ▷ ▷ -with C -do -BLT_set_classify_activity \
24605 ▷ ▷ ▷ -isomorph \
24606 ▷ ▷ ▷ ▷ -prefix_iso "../BLT_q13" \
24607 ▷ ▷ ▷ ▷ -use_database_for_starter \
24608 ▷ ▷ ▷ ▷ -compute_orbits \
24609 ▷ ▷ ▷ ▷ -list_of_cases BLT_q13_list_of_cases_5_0_1.csv \
24610 ▷ ▷ ▷ ▷ -solution_prefix "" \
24611 ▷ ▷ ▷ ▷ -base_fname "BLT_q13_graph" \
24612 ▷ ▷ ▷ -end \
24613 ▷ ▷ -end
24614
24615 BLT_13_isomorph_testing:
24616 ▷ $(ORBITER) -v 4 \
24617 ▷ ▷ -define F -finite_field -q 13 -end \
24618 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
24619 ▷ ▷ -define C -BLT_set_classifier \
24620 ▷ ▷ ▷ -orthogonal_space 0 -starter_size 5 \
24621 ▷ ▷ -end \
24622 ▷ ▷ -with C -do -BLT_set_classify_activity \
24623 ▷ ▷ ▷ -compute_starter \
24624 ▷ ▷ ▷ ▷ -problem_label BLT_q13 \
24625 ▷ ▷ ▷ ▷ -W -depth 5 \
24626 ▷ ▷ ▷ -end \
24627 ▷ ▷ -end \
24628 ▷ ▷ -with C -do -BLT_set_classify_activity \
24629 ▷ ▷ ▷ -isomorph \
24630 ▷ ▷ ▷ ▷ -prefix_iso "../BLT_q13" \
24631 ▷ ▷ ▷ ▷ -use_database_for_starter \
24632 ▷ ▷ ▷ ▷ -isomorph_testing \
24633 ▷ ▷ ▷ ▷ -solution_prefix "" \
24634 ▷ ▷ ▷ ▷ -base_fname "BLT_q13_graph" \
24635 ▷ ▷ ▷ -end \
24636 ▷ ▷ -end \
24637 ▷ ▷ -with C -do -BLT_set_classify_activity \
24638 ▷ ▷ ▷ -isomorph \
24639 ▷ ▷ ▷ ▷ -prefix_iso "../BLT_q13" \
24640 ▷ ▷ ▷ ▷ -use_database_for_starter \
24641 ▷ ▷ ▷ ▷ -isomorph_report \
24642 ▷ ▷ ▷ ▷ -solution_prefix "" \
24643 ▷ ▷ ▷ ▷ -base_fname "BLT_q13_graph" \
24644 ▷ ▷ ▷ -end \
24645 ▷ ▷ -end
24646 ▷ $(PDFLATEX) BLT_q13_isomorphism_types.tex
24647 ▷ $(OPEN) BLT_q13_isomorphism_types.pdf
24648
24649
24650
24651
24652
24653 blt_set_export_gap_all:
24654 ▷ $(ORBITER) -v 3 \
24655 ▷ ▷ -define Q -vector -dense \

```

```

24656 ▷ ▷ ▷ $(BLT_ORDER_Q) \
24657 ▷ ▷ -end \
24658 ▷ ▷ -define NB -vector -dense \
24659 ▷ ▷ ▷ $(BLT_NUMBER_ISO) \
24660 ▷ ▷ -end \
24661 ▷ ▷ -loop_over i Q \
24662 ▷ ▷ ▷ -define F%i -finite_field -q "%i[Q]" -end \
24663 ▷ ▷ ▷ -define O%i -orthogonal_space 0 5 F%i -create_extension_fields -end \
24664 ▷ ▷ ▷ -loop j 0 %i[NB] 1 \
24665 ▷ ▷ ▷ -define BLT_%i_%j -BLT_set \
24666 ▷ ▷ ▷ ▷ -space O%i -catalogue %j \
24667 ▷ ▷ ▷ ▷ -end \
24668 ▷ ▷ ▷ ▷ -with BLT_%i_%j -do -blt_set_activity \
24669 ▷ ▷ ▷ ▷ ▷ -export_gap \
24670 ▷ ▷ ▷ ▷ -end \
24671 ▷ ▷ ▷ -end_loop j \
24672 ▷ ▷ -end_loop_over i \
24673 ▷ ▷ -print_symbols
24674
24675
24676 #flock::init q = 17 point_idx = 6
24677 #projective_space_basic::PG_element_normalize_from_a_given_position zero vector
24678
24679
24680
24681
24682
24683
24684 #####
24685 # Chapter 16 - Computer Science Primitives
24686 #####
24687
24688
24689
24690 test_16:
24691 ▷ make test_16.1
24692 ▷ make test_16.2
24693 ▷ make test_16.3
24694
24695
24696
24697
24698 #####
24699 # Section 16.1: Clique finding
24700
24701
24702 SECTION_GRAPH_THEORY_CLIQUES_FINDING:
24703
24704
24705 test_16.1:
24706 ▷ make small_graph_cliques
24707 ▷ make BLT_q13_graph_5.0_cliques_bw
24708 ▷ make BLT_q13_graph_5.0_cliques_rainbow
24709 ▷ make small_graph_cliques_Sajeeb
24710 ▷ make Paley_13_aut
24711 ▷ make Paley_13
24712 ▷ make Paley_13_cliques_classify
24713 ▷ make Paley_13_cliques_all
24714 ▷ make Neumaier_16_cliques_all

```

```

24715 ▷ make tritangent_planes_graph_aut
24716 ▷ make tritangent_planes_graph_cliques_classify
24717 ▷ make tritangent_planes_graph_cliques_all
24718 ▷ make PGO_5_2_cliques
24719 ▷ make Doily_disjoint_sets_graph_cliques_3
24720 ▷ make Doily_disjoint_sets_graph_cliques_5
24721 ▷ make PG_3_2_test
24722 ▷ make HJ_d2_c5
24723 ▷ make HJ64_cliques5
24724 ▷ make HJ64_cliques5_classify
24725 ▷ make Op_4_2_affine_polar_graph_cliques_4
24726
24727
24728
24729 small_graph_cliques:
24730 ▷ $(ORBITER) -v 2 \
24731 ▷ ▷ -define G -graph -edges_as_pairs 5 \
24732 ▷ ▷ ▷ "0,1,0,2,0,3,0,4,1,3,1,4,2,4" \
24733 ▷ ▷ -end \
24734 ▷ ▷ -with G -do \
24735 ▷ ▷ -graph_theoretic_activity -save -end
24736 ▷ $(ORBITER) -v 2 \
24737 ▷ ▷ -define G -graph -load graph.v5.e7.colored.graph -end \
24738 ▷ ▷ -with G -do \
24739 ▷ ▷ -graph_theoretic_activity \
24740 ▷ ▷ ▷ -find_cliques -target_size 3 \
24741 ▷ ▷ -end
24742
24743 # nb_sol = 3
24744
24745
24746
24747
24748 BLT_q13_graph_5_0_cliques.bw:
24749 ▷ $(ORBITER) -v 2 \
24750 ▷ ▷ -define F -finite_field -q 13 -end \
24751 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
24752 ▷ ▷ -define C -BLT_set_classifier \
24753 ▷ ▷ ▷ -orthogonal_space 0 -starter_size 5 \
24754 ▷ ▷ -end \
24755 ▷ ▷ -with C -do -BLT_set_classify_activity \
24756 ▷ ▷ ▷ -compute_starter \
24757 ▷ ▷ ▷ ▷ -problem_label BLT_q13 \
24758 ▷ ▷ ▷ ▷ -W -depth 5 \
24759 ▷ ▷ ▷ -end \
24760 ▷ ▷ -end \
24761 ▷ ▷ -with C -do -BLT_set_classify_activity \
24762 ▷ ▷ ▷ -create_graphs \
24763 ▷ ▷ -end
24764 ▷ $(ORBITER) -v 2 \
24765 ▷ ▷ -define G -graph -load BLT_q13_graph_5_0.bin -end \
24766 ▷ ▷ -with G -do \
24767 ▷ ▷ -graph_theoretic_activity \
24768 ▷ ▷ ▷ -find_cliques -target_size 9 \
24769 ▷ ▷ -end
24770
24771 # all_cliques_black_and_white
24772
24773

```

```

24774 BLT_q13_graph_5.0_cliques_rainbow:
24775 ▷ $(ORBITER) -v 2 \
24776 ▷ ▷ -define F -finite_field -q 13 -end \
24777 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
24778 ▷ ▷ -define C -BLT_set_classifier \
24779 ▷ ▷ ▷ -orthogonal_space 0 -starter_size 5 \
24780 ▷ ▷ -end \
24781 ▷ ▷ -with C -do -BLT_set_classify_activity \
24782 ▷ ▷ ▷ -compute_starter \
24783 ▷ ▷ ▷ ▷ -problem_label BLT_q13 \
24784 ▷ ▷ ▷ ▷ -W -depth 5 \
24785 ▷ ▷ ▷ -end \
24786 ▷ ▷ -end \
24787 ▷ ▷ -with C -do -BLT_set_classify_activity \
24788 ▷ ▷ ▷ -create_graphs \
24789 ▷ ▷ -end
24790 ▷ $(ORBITER) -v 2 \
24791 ▷ ▷ -define G -graph -load BLT_q13_graph_5.0.bin -end \
24792 ▷ ▷ -with G -do \
24793 ▷ ▷ -graph_theoretic_activity \
24794 ▷ ▷ ▷ -find_cliques -rainbow -target_size 9 \
24795 ▷ ▷ -end
24796
24797 # all_rainbow_cliques
24798
24799
24800 small_graph_cliques_Sajeeb:
24801 ▷ $(ORBITER) -v 2 \
24802 ▷ ▷ -define G -graph -edges_as_pairs 5 \
24803 ▷ ▷ ▷ "0,1,0,2,0,3,0,4,1,3,1,4,2,4" \
24804 ▷ ▷ -end \
24805 ▷ ▷ -with G -do \
24806 ▷ ▷ -graph_theoretic_activity -save -end
24807 ▷ $(ORBITER) -v 2 \
24808 ▷ ▷ -define G -graph -load graph_v5.e7.colored_graph -end \
24809 ▷ ▷ -with G -do \
24810 ▷ ▷ -graph_theoretic_activity \
24811 ▷ ▷ ▷ -find_cliques -Sajeeb -target_size 3 \
24812 ▷ ▷ -end
24813
24814 # nb_sol = 3
24815
24816 Paley_13_aut:
24817 ▷ $(ORBITER) -v 2 \
24818 ▷ ▷ -define F -finite_field -q 13 -end \
24819 ▷ ▷ -define Gamma -graph -Paley F -end \
24820 ▷ ▷ -with Gamma -do \
24821 ▷ ▷ -graph_theoretic_activity \
24822 ▷ ▷ ▷ -automorphism_group \
24823 ▷ ▷ -end
24824
24825 # writes Paley_13_group.makefile
24826 #User time: 0 of a second, dt=0 tps = 100
24827 #nb_calls_to_densenaute=1
24828
24829
24830
24831 Paley_13:
24832 ▷ $(ORBITER) -v 2 \

```

```

24833 ▷ ▷ -define gens -vector -file Paley_13.gens.csv -end \
24834 ▷ ▷ -define G -permutation_group \
24835 ▷ ▷ -bsgs Paley_13 "Paley\13" 13 78 "0,1" 3 gens -end \
24836
24837
24838 Paley_13_cliques_classify:
24839 ▷ $(ORBITER) -v 4 \
24840 ▷ ▷ -define Control -poset_classification_control \
24841 ▷ ▷ ▷ -W \
24842 ▷ ▷ ▷ -problem_label Paley13_cliques \
24843 ▷ ▷ ▷ -clique_test Gamma \
24844 ▷ ▷ ▷ -depth 5 \
24845 ▷ ▷ -end \
24846 ▷ ▷ -define F -finite_field -q 13 -end \
24847 ▷ ▷ -define gens -vector -file Paley_13.gens.csv -end \
24848 ▷ ▷ -define G -permutation_group \
24849 ▷ ▷ ▷ -bsgs Paley_13 "Paley\13" 13 78 "0,1" 3 gens -end \
24850 ▷ ▷ -define Gamma -graph -Paley F -end \
24851 ▷ ▷ -define Orb -orbits \
24852 ▷ ▷ ▷ -group G \
24853 ▷ ▷ ▷ -group_action G \
24854 ▷ ▷ ▷ -generators G \
24855 ▷ ▷ ▷ -on_subsets 5 Control \
24856 ▷ ▷ -end
24857
24858 #User time: 0.01 of a second, dt=1 tps = 100
24859
24860
24861 Paley_13_cliques_all:
24862 ▷ $(ORBITER) -v 2 \
24863 ▷ ▷ -define F -finite_field -q 13 -end \
24864 ▷ ▷ -define Gamma -graph -Paley F -end \
24865 ▷ ▷ -with Gamma -do \
24866 ▷ ▷ -graph_theoretic_activity \
24867 ▷ ▷ ▷ -find_cliques -target_size 3 \
24868 ▷ ▷ -end
24869
24870
24871 #colored_graph_cliques::all_cliques_black_and_white after CG->all_cliques_of_size
    _k_ignore_colors, nb_cliques = 26
24872
24873
24874
24875 Neumaier_16_cliques_all:
24876 ▷ $(ORBITER) -v 10 \
24877 ▷ ▷ -define Gamma -graph -Neumaier_graph_16 -end \
24878 ▷ ▷ -with Gamma -do \
24879 ▷ ▷ -graph_theoretic_activity \
24880 ▷ ▷ ▷ -find_cliques -target_size 4 \
24881 ▷ ▷ -end
24882
24883
24884 # 8 cliques of size 4
24885
24886
24887
24888 tritangent_planes_graph_aut:
24889 ▷ $(ORBITER) -v 2 \
24890 ▷ ▷ -define Gamma \

```

```

24891 ▷ ▷ ▷ -graph -tritangent_planes_disjointness_graph \
24892 ▷ ▷ -end \
24893 ▷ ▷ -with Gamma -do \
24894 ▷ ▷ -graph_theoretic_activity \
24895 ▷ ▷ ▷ -automorphism_group \
24896 ▷ ▷ -end
24897 ▷ $(PDFLATEX) tritangent_planes_disjointness_report.tex
24898 ▷ $(OPEN) tritangent_planes_disjointness_report.pdf
24899
24900 #tritangent_planes_disjointness_report.tex
24901 #tritangent_planes_disjointness_gens.csv
24902 #tritangent_planes_disjointness_group.makefile
24903
24904 tritangent_planes_graph_cliques_classify:
24905 ▷ $(ORBITER) -v 4 \
24906 ▷ ▷ -define Control -poset_classification_control \
24907 ▷ ▷ ▷ -W \
24908 ▷ ▷ ▷ -problem_label tri \
24909 ▷ ▷ ▷ -clique_test Gamma \
24910 ▷ ▷ ▷ -depth 9 \
24911 ▷ ▷ -end \
24912 ▷ ▷ -define gens -vector -file \
24913 ▷ ▷ ▷ tritangent_planes_disjointness_gens.csv \
24914 ▷ ▷ -end \
24915 ▷ ▷ -define G -permutation_group \
24916 ▷ ▷ -bsgs tritangent_planes_disjointness \
24917 ▷ ▷ ▷ "tritangent_planes_disjointness" 45 51840 \
24918 ▷ ▷ ▷ "0,3,12,24" 6 gens -end \
24919 ▷ ▷ -define Gamma \
24920 ▷ ▷ ▷ -graph -tritangent_planes_disjointness_graph \
24921 ▷ ▷ -end \
24922 ▷ ▷ -define Orb -orbits \
24923 ▷ ▷ ▷ -group G \
24924 ▷ ▷ ▷ -group_action G \
24925 ▷ ▷ ▷ -generators G \
24926 ▷ ▷ ▷ -on_subsets 9 Control \
24927 ▷ ▷ -end \
24928 ▷ ▷ -with Orb -do -orbits_activity \
24929 ▷ ▷ ▷ -recognize "4,9,11,14,21,26,30,35,40" \
24930 ▷ ▷ -end
24931
24932 #tri_reps_lvl_9.csv
24933
24934
24935 tritangent_planes_graph_cliques_all:
24936 ▷ $(ORBITER) -v 2 \
24937 ▷ ▷ -define Gamma \
24938 ▷ ▷ ▷ -graph -tritangent_planes_disjointness_graph \
24939 ▷ ▷ -end \
24940 ▷ ▷ -with Gamma -do \
24941 ▷ ▷ -graph_theoretic_activity \
24942 ▷ ▷ ▷ -find_cliques -target_size 9 \
24943 ▷ ▷ -end
24944
24945
24946 #tritangent_planes_disjointness_sol.csv
24947
24948
24949

```



```

24950 PGO.5.2_cliques:
24951 ▷ $(ORBITER) -v 2 \
24952 ▷ ▷ -define F -finite.field -q 2 -end \
24953 ▷ ▷ -define O -orthogonal.space 0 5 F -without.group -end \
24954 ▷ ▷ -with O -do -orthogonal.space.activity \
24955 ▷ ▷ ▷ -export.point.line.incidence.matrix \
24956 ▷ ▷ -end
24957 ▷ $(ORBITER) -v 3 \
24958 ▷ ▷ -define Inc -vector -file 0.5.2.incidence.matrix.csv -end \
24959 ▷ ▷ -define Gamma -graph -collinearity.graph Inc -end \
24960 ▷ ▷ -with Gamma -do \
24961 ▷ ▷ -graph.theoretic.activity \
24962 ▷ ▷ ▷ -find.cliques -target.size 3 \
24963 ▷ ▷ -end
24964
24965
24966
24967
24968 Doily_disjoint_sets_graph_cliques_3:
24969 ▷ echo $(DOILY) >doily.csv
24970 ▷ $(ORBITER) -v 2 \
24971 ▷ ▷ -define G -graph -disjoint.sets.graph \
24972 ▷ ▷ ▷ doily.csv \
24973 ▷ ▷ -end \
24974 ▷ ▷ -with G -do \
24975 ▷ ▷ ▷ -graph.theoretic.activity \
24976 ▷ ▷ ▷ -find.cliques \
24977 ▷ ▷ ▷ ▷ -target.size 3 \
24978 ▷ ▷ ▷ ▷ -output.file doily.cliques.csv \
24979 ▷ ▷ -end \
24980 ▷ ▷ -print.symbols
24981 ▷ $(ORBITER) -v 2 \
24982 ▷ ▷ -union doily.csv doily.cliques.csv doily.cliques.union.csv
24983
24984 # the files doily.csv and doily.cliques.csv have different formats
24985 # 80 cliques
24986
24987 Doily_disjoint_sets_graph_cliques_5:
24988 ▷ echo $(DOILY) >doily.csv
24989 ▷ $(ORBITER) -v 2 \
24990 ▷ ▷ -define G -graph -disjoint.sets.graph \
24991 ▷ ▷ ▷ doily.csv \
24992 ▷ ▷ -end \
24993 ▷ ▷ -with G -do \
24994 ▷ ▷ ▷ -graph.theoretic.activity \
24995 ▷ ▷ ▷ -find.cliques \
24996 ▷ ▷ ▷ ▷ -target.size 5 \
24997 ▷ ▷ ▷ ▷ -output.file doily.cliques_5 \
24998 ▷ ▷ ▷ -end \
24999 ▷ ▷ -end \
25000 ▷ ▷ -print.symbols
25001
25002 # 6 cliques
25003 # doily.cliques_5.csv
25004
25005
25006 PG.3.2_test:
25007 ▷ $(ORBITER) -v 2 \
25008 ▷ ▷ -define Do -draw.options \

```

```

25009 ▷ ▷ ▷ -radius 200 -line_width 0.3 -nodes_empty \
25010 ▷ ▷ -end \
25011 ▷ ▷ -define F -finite_field -q 2 -end \
25012 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
25013 ▷ ▷ -with P -do -projective_space_activity \
25014 ▷ ▷ ▷ -cheat_sheet Do \
25015 ▷ ▷ -end
25016 ▷ $(PDFLATEX) PG_3_2.tex
25017 ▷ $(OPEN) PG_3_2.pdf
25018
25019
25020
25021 HJ_d2_c5:
25022 ▷ $(ORBITER) -v 6 \
25023 ▷ ▷ -define G -graph \
25024 ▷ ▷ ▷ -load_csv_no_border \
25025 ▷ ▷ ▷ halljanko315.csv \
25026 ▷ ▷ ▷ -distance_2 \
25027 ▷ ▷ -end \
25028 ▷ ▷ -with G -do \
25029 ▷ ▷ -graph_theoretic_activity \
25030 ▷ ▷ ▷ -find_cliques -target_size 5 -end \
25031 ▷ ▷ -end
25032
25033
25034
25035 #graph_theoretic_activity::perform_activity Gr->label=halljanko315 nb_sol = 26208
25036 0
25037
25038 HJ64_cliques5:
25039 ▷ $(ORBITER) -v 6 \
25040 ▷ ▷ -define Gamma -graph \
25041 ▷ ▷ ▷ -load \
25042 ▷ ▷ ▷ Group_Perm315_Orbital_3.colored_graph \
25043 ▷ ▷ -end \
25044 ▷ ▷ -with Gamma -do \
25045 ▷ ▷ -graph_theoretic_activity \
25046 ▷ ▷ ▷ -find_cliques -target_size 5 -end \
25047 ▷ ▷ -end
25048
25049 #graph_theoretic_activity::perform_activity Gr->label=Group_Perm315_Orbital_3 nb_
25050 sol = 1008
25051 #Group_Perm315_Orbital_3_sol.csv
25052
25053
25054 HJ64_cliques5_classify:
25055 ▷ $(ORBITER) -v 6 \
25056 ▷ ▷ -define Control -poset_classification_control \
25057 ▷ ▷ ▷ -W \
25058 ▷ ▷ ▷ -problem_label HJ64_cliques \
25059 ▷ ▷ ▷ -clique_test Gamma \
25060 ▷ ▷ ▷ -depth 5 \
25061 ▷ ▷ -end \
25062 ▷ ▷ -define Gamma -graph \
25063 ▷ ▷ ▷ -load \
25064 ▷ ▷ ▷ Group_Perm315_Orbital_3.colored_graph \
25065 ▷ ▷ -end \

```

```

25066 ▷ ▷ -define gens -vector \
25067 ▷ ▷ ▷ -file halljanko315.gens.csv \
25068 ▷ ▷ -end \
25069 ▷ ▷ -define G -permutation_group \
25070 ▷ ▷ -bsgs halljanko315 "File\_halljanko315" \
25071 ▷ ▷ ▷ 315 1209600 "0,1,42,95" 5 gens -end \
25072 ▷ ▷ -define Orb -orbits -group G \
25073 ▷ ▷ ▷ -on_subsets 5 Control \
25074 ▷ ▷ -end
25075
25076 #HJ64_cliques.reps.lvl.5.csv
25077
25078 # 1 orbit
25079 #ROW,REP,AGO,OL
25080 #0,"0,8,31,110,283",1200,1008
25081 #END
25082
25083
25084 Op_4_2.affine.polar.graph.cliques.4:
25085 ▷ $(ORBITER) -v 2 \
25086 ▷ ▷ -define F -finite_field -q 2 -end \
25087 ▷ ▷ -define O -orthogonal_space 1 4 F -end \
25088 ▷ ▷ -define Gamma -graph -affine.polar O -end \
25089 ▷ ▷ -with Gamma -do \
25090 ▷ ▷ ▷ -graph.theoretic.activity -save \
25091 ▷ ▷ -end \
25092 ▷ ▷ -with Gamma -do \
25093 ▷ ▷ ▷ -graph.theoretic.activity -test_SRG.property \
25094 ▷ ▷ -end \
25095 ▷ ▷ -with Gamma -do \
25096 ▷ ▷ -graph.theoretic.activity \
25097 ▷ ▷ ▷ -find.cliques -target.size 4 -end \
25098 ▷ ▷ -end \
25099
25100
25101
25102 #####
25103 # Section 16.2: Rainbow cliques
25104
25105
25106 SECTION.GRAPH.THEORY.RAINBOW.CLiques:
25107
25108
25109 test_16_2:
25110 ▷ make BLT_11.classify_starter
25111 ▷ make BLT_11.clique_0
25112 ▷ make BLT_11.clique_0.Sajeeb
25113 ▷ make BLT_11_0.sort
25114 ▷ make BLT_11_0.sorted.draw
25115
25116
25117 BLT_11.classify_starter:
25118 ▷ $(ORBITER) -v 2 \
25119 ▷ ▷ -define F -finite_field -q 11 -end \
25120 ▷ ▷ -define O -orthogonal_space 0 5 F -end \
25121 ▷ ▷ -define C -BLT_set_classifier -orthogonal_space 0 -starter_size 5 -end \
25122 ▷ ▷ -with C -do -BLT_set_classify_activity \
25123 ▷ ▷ ▷ -compute_starter \
25124 ▷ ▷ ▷ ▷ -problem_label BLT.q11 \

```

```

25125 ▷ ▷ ▷ ▷ -W -depth 5 \
25126 ▷ ▷ ▷ -end \
25127 ▷ ▷ -end \
25128 ▷ ▷ -with C -do -BLT_set_classify_activity \
25129 ▷ ▷ ▷ -create_graphs \
25130 ▷ ▷ -end
25131
25132 #NB_CASES_Q11=
25133
25134
25135 BLT_11_clique_0:
25136 ▷ $(ORBITER) -v 2 \
25137 ▷ ▷ -define G -graph \
25138 ▷ ▷ ▷ -load BLT_q11_graph_5_0.bin \
25139 ▷ ▷ -end \
25140 ▷ ▷ -with G -do \
25141 ▷ ▷ -graph_theoretic_activity \
25142 ▷ ▷ ▷ -find_cliques -rainbow -target_size 12 \
25143 ▷ ▷ -end
25144
25145 # nb_sol = 3
25146
25147 BLT_11_clique_0_Sajeeb:
25148 ▷ $(ORBITER) -v 2 \
25149 ▷ ▷ -define G -graph \
25150 ▷ ▷ ▷ -load BLT_q11_graph_5_0.bin \
25151 ▷ ▷ -end \
25152 ▷ ▷ -with G -do \
25153 ▷ ▷ -graph_theoretic_activity \
25154 ▷ ▷ ▷ -find_cliques -rainbow -Sajeeb \
25155 ▷ ▷ -end
25156
25157 # nb_sol = 3
25158
25159
25160 BLT_11_0_sort:
25161 ▷ $(ORBITER) -v 2 \
25162 ▷ ▷ -define G -graph \
25163 ▷ ▷ ▷ -load BLT_q11_graph_5_0.bin \
25164 ▷ ▷ -end \
25165 ▷ ▷ -with G -do \
25166 ▷ ▷ -graph_theoretic_activity \
25167 ▷ ▷ ▷ -sort_by_colors \
25168 ▷ ▷ -end
25169
25170 #BLT_q11_graph_5_0_sorted.bin
25171
25172
25173 BLT_11_0_sorted_draw:
25174 ▷ $(ORBITER) -v 2 \
25175 ▷ ▷ -define Do -draw_options \
25176 ▷ ▷ ▷ -radius 100 \
25177 ▷ ▷ ▷ -scale 0.4 \
25178 ▷ ▷ ▷ -line_width 1.0 -embedded \
25179 ▷ ▷ -end \
25180 ▷ ▷ -define G -graph \
25181 ▷ ▷ ▷ -load BLT_q11_graph_5_0_sorted.bin \
25182 ▷ ▷ -end \
25183 ▷ ▷ -with G -do \

```

```

25184 ▷ ▷ -graph_theoretic.activity \
25185 ▷ ▷ ▷ -draw Do \
25186 ▷ ▷ -end
25187 ▷ $(PDFLATEX) BLT_q11_graph_5_0_sorted_draw.tex
25188 ▷ $(OPEN) BLT_q11_graph_5_0_sorted_draw.pdf
25189
25190
25191
25192
25193 #####
25194 # Section 16.3: Diophantine Systems
25195
25196 SECTION.DIOPHANT:
25197
25198
25199 test_16_3:
25200 ▷ make part10
25201 ▷ make octic_monomials
25202 ▷ make solve_test_system
25203 ▷ make McKay_test
25204 ▷ make DLX_test
25205
25206
25207
25208 part10:
25209 ▷ $(ORBITER) -v 4 \
25210 ▷ ▷ -define A -vector -format 1 -dense "10,9,8,7,6,5,4,3,2,1" -end \
25211 ▷ ▷ -define D -diophant \
25212 ▷ ▷ ▷ -label part10 \
25213 ▷ ▷ ▷ -coefficient_matrix A \
25214 ▷ ▷ ▷ -RHS "mult=1,EQ=10" \
25215 ▷ ▷ ▷ -x_min_global 0 -x_max_global 10 \
25216 ▷ ▷ -end \
25217 ▷ ▷ -with D -do \
25218 ▷ ▷ ▷ -diophant_activity -solve_mckay \
25219 ▷ ▷ -end
25220 ▷ ▷
25221
25222
25223 ▷
25224 # Finds 42 solutions with 67 backtrack steps
25225
25226
25227 octic_monomials:
25228 ▷ $(ORBITER) -v 4 \
25229 ▷ ▷ -define A -vector -format 1 -dense "1,1,1,1" -end \
25230 ▷ ▷ -define D -diophant \
25231 ▷ ▷ ▷ -label octic_monomials \
25232 ▷ ▷ ▷ -coefficient_matrix A \
25233 ▷ ▷ ▷ -RHS "mult=1,EQ=9" \
25234 ▷ ▷ ▷ -x_min_global 0 -x_max_global 8 \
25235 ▷ ▷ -end \
25236 ▷ ▷ -with D -do \
25237 ▷ ▷ ▷ -diophant_activity -solve_mckay \
25238 ▷ ▷ -end
25239 ▷ sort -r octic_monomials.sol.csv >octic_monomials_sorted.csv
25240
25241 #Found 165 solutions with 210 backtrack steps
25242 # 165=binomial(11,3)

```

```

25243
25244
25245 solve_test_system:
25246 ▷ $(ORBITER) -v 4 \
25247 ▷ ▷ -define A -vector -format 7 -dense $(TEST_SYSTEM) -end \
25248 ▷ ▷ -define D -diophant \
25249 ▷ ▷ ▷ -label test_system \
25250 ▷ ▷ ▷ -coefficient_matrix A \
25251 ▷ ▷ ▷ -RHS "mult=7,EQ=1" \
25252 ▷ ▷ ▷ -x_min_global 0 -x_max_global 1 \
25253 ▷ ▷ -end
25254
25255
25256 McKay_test:
25257 ▷ $(ORBITER) -v 4 \
25258 ▷ ▷ -define A -vector -format 7 -dense $(TEST_SYSTEM) -end \
25259 ▷ ▷ -define D -diophant \
25260 ▷ ▷ ▷ -label test_system \
25261 ▷ ▷ ▷ -coefficient_matrix A \
25262 ▷ ▷ ▷ -RHS "mult=7,EQ=1" \
25263 ▷ ▷ ▷ -x_min_global 0 -x_max_global 1 \
25264 ▷ ▷ -end \
25265 ▷ ▷ -with D -do \
25266 ▷ ▷ ▷ -diophant_activity -solve_mckay \
25267 ▷ ▷ -end
25268
25269 DLX_test:
25270 ▷ $(ORBITER) -v 4 \
25271 ▷ ▷ -define A -vector -format 7 -dense $(TEST_SYSTEM) -end \
25272 ▷ ▷ -define D \
25273 ▷ ▷ -diophant -label test_system \
25274 ▷ ▷ ▷ -coefficient_matrix A \
25275 ▷ ▷ ▷ -RHS "mult=7,EQ=1" \
25276 ▷ ▷ ▷ -x_min_global 0 -x_max_global 1 \
25277 ▷ ▷ -end \
25278 ▷ ▷ -with D -do \
25279 ▷ ▷ ▷ -diophant_activity -solve_DLX \
25280 ▷ ▷ -end
25281
25282 #DLX_test.sol
25283 # 1 solution in 6 backtrack steps
25284
25285
25286
25287
25288
25289 solve_praeger_design_line_type_system:
25290 ▷ $(ORBITER) -v 4 \
25291 ▷ ▷ -define A -vector -format 4 -dense $(PRAEGER_DESIGN_LINE_TYPE) -end \
25292 ▷ ▷ -define D -diophant \
25293 ▷ ▷ ▷ -label test_system \
25294 ▷ ▷ ▷ -coefficient_matrix A \
25295 ▷ ▷ ▷ -RHS "EQ=18" \
25296 ▷ ▷ ▷ -RHS "EQ=55" \
25297 ▷ ▷ ▷ -RHS "EQ=88" \
25298 ▷ ▷ ▷ -RHS "EQ=57" \
25299 ▷ ▷ ▷ -x_min_global 0 -x_max_global 57 \
25300 ▷ ▷ -end \
25301 ▷ ▷ -with D -do \

```

```

25302 ▷ ▷ ▷ -diophant.activity -solve_mckay \
25303 ▷ ▷ -end
25304
25305 # there are 8 solutions
25306
25307
25308
25309 #####
25310 # Chapter 17 - Canonical Forms with Nauty
25311 #####
25312
25313
25314
25315 test_17:
25316 ▷ make test_17_2
25317 ▷ make test_17_3
25318 ▷ make test_17_4
25319 ▷ make test_17_5
25320 ▷ make test_17_6
25321 ▷ make test_17_7
25322 ▷ make test_17_8
25323
25324
25325
25326 #####
25327 # Section 17.1: Overview of Canonical Forms
25328
25329
25330 SECTION.OVERVIEW.CANONICAL.FORMS:
25331
25332
25333
25334
25335
25336 #####
25337 # Section 17.2: Objects in projective Space
25338
25339
25340 SECTION.OBJECTS.IN.PROJECTIVE.SPACE:
25341
25342
25343 test_17_2:
25344 ▷ make quadrangles_PG_2_2
25345 ▷ #make Barth_mod_11_singular_pts_c
25346 ▷ make quartic_curves_q13_classify_nauty
25347 ▷ make quartic_curves_q13_filter_isomorphism_reps
25348 ▷ make quartic_curves_q13_tally_ago
25349 ▷ make Klein_cubic_threefold_q3
25350 ▷ make EC_canon
25351 ▷ make Hirschfeld_q4_c
25352 ▷ make Hirschfeld_stab_group
25353 ▷ make Hirschfeld_stab_subgroup
25354 ▷ make Hirschfeld_stab_orbits
25355 ▷ make Hirschfeld_q4_set_c
25356 ▷ make Dickson_sets_stabilizer
25357 ▷ #make Endrass_7c # variable is undefined
25358 ▷ make hyperoval_16_canonical_form
25359 ▷ make cubic_curves_PG_2_8_canon
25360 ▷ make ovoid_q8_canon

```

```

25361 ▷ make ovoid_ST_q8_canon
25362 ▷ make Suzuki_8
25363 ▷ make quartic_curve_25_0_0_canonical
25364 ▷ #make CRC_BCH_Q16_N51_D3_canonical # slow
25365 ▷ make Veronese_1_3_q7_c
25366
25367
25368 quadrangles_PG_2_2:
25369 ▷ $(ORBITER) -v 3 \
25370 ▷ ▷ -define C -combinatorial_object \
25371 ▷ ▷ ▷ -label quadrangle quadrangle \
25372 ▷ ▷ ▷ -set_of_points "0,1,2,3" \
25373 ▷ ▷ ▷ -set_of_points "1,3,4,6" \
25374 ▷ ▷ -end \
25375 ▷ ▷ -define F -finite_field -q 2 -end \
25376 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
25377 ▷ ▷ -with C -do \
25378 ▷ ▷ -combinatorial_object_activity \
25379 ▷ ▷ ▷ -canonical_form_PG P \
25380 ▷ ▷ ▷ -nauty_control -nauty_log "quadrangles_PG_2_2_nauty_log.txt" -end \
25381 ▷ ▷ ▷ -end \
25382 ▷ ▷ -end \
25383 ▷ ▷ -define Doinc -draw_incidence_structure_options \
25384 ▷ ▷ ▷ -width 40 -width_10 6 \
25385 ▷ ▷ -end \
25386 ▷ ▷ -with C -do \
25387 ▷ ▷ -combinatorial_object_activity \
25388 ▷ ▷ ▷ -post_processing \
25389 ▷ ▷ -end \
25390 ▷ ▷ -with C -do \
25391 ▷ ▷ -combinatorial_object_activity \
25392 ▷ ▷ ▷ -report \
25393 ▷ ▷ ▷ -export_flag_orbits \
25394 ▷ ▷ ▷ -show_TDO \
25395 ▷ ▷ ▷ -show_TDA \
25396 ▷ ▷ ▷ -dont_show_incidence_matrices \
25397 ▷ ▷ ▷ -incidence_draw_options Doinc \
25398 ▷ ▷ ▷ -end \
25399 ▷ ▷ -end
25400 ▷ $(PDFLATEX) quadrangle_classification.tex
25401 ▷ $(OPEN) quadrangle_classification.pdf
25402
25403
25404
25405
25406
25407 Barth_mod_11_singular_pts.c:
25408 ▷ $(ORBITER) -v 4 \
25409 ▷ ▷ -define H -set -here \
25410 ▷ ▷ ▷ $(BARTH_SEXTIC_MOD_11_SINGULAR_POINTS) \
25411 ▷ ▷ -end \
25412 ▷ ▷ -define C -combinatorial_object \
25413 ▷ ▷ ▷ -label Barth_q11_sing Barth\_q11\_sing \
25414 ▷ ▷ ▷ -set_of_points H \
25415 ▷ ▷ -end \
25416 ▷ ▷ -define F -finite_field -q 11 -end \
25417 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
25418 ▷ ▷ -with C -do \
25419 ▷ ▷ -combinatorial_object_activity \

```



```

25420 ▷ ▷ ▷ -canonical_form_PG P \
25421 ▷ ▷ ▷ ▷ -save_ago \
25422 ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
25423 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "Barth_mod.11_nauty_log.txt" -end \
25424 ▷ ▷ ▷ -end \
25425 ▷ ▷ -end \
25426 ▷ ▷ -define Doinc -draw_incidence_structure_options \
25427 ▷ ▷ ▷ -width 40 -width_10 6 \
25428 ▷ ▷ -end \
25429 ▷ ▷ -with C -do \
25430 ▷ ▷ -combinatorial_object_activity \
25431 ▷ ▷ ▷ -post_processing \
25432 ▷ ▷ -end \
25433 ▷ ▷ -with C -do \
25434 ▷ ▷ -combinatorial_object_activity \
25435 ▷ ▷ ▷ -report \
25436 ▷ ▷ ▷ ▷ -show_TDO \
25437 ▷ ▷ ▷ ▷ -show_TDA \
25438 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
25439 ▷ ▷ ▷ -end \
25440 ▷ ▷ -end
25441 ▷ $(PDFLATEX) Barth_q11_sing_classification.tex
25442 ▷ $(OPEN) Barth_q11_sing_classification.pdf
25443
25444
25445
25446 # too slow
25447
25448
25449
25450
25451 quartic_curves_q13_classify_nauty:
25452 ▷ $(ORBITER) -v 3 \
25453 ▷ ▷ -define F -finite_field -q 13 -end \
25454 ▷ ▷ -define P -projective_space -n 3 -field F -end \
25455 ▷ ▷ -define P2 -projective_space -n 2 -field F -v 0 -end \
25456 ▷ ▷ -define Control -poset_classification_control \
25457 ▷ ▷ ▷ -problem_label nc_arcs_q12 \
25458 ▷ ▷ ▷ -W -depth 6 \
25459 ▷ ▷ -end \
25460 ▷ ▷ -define Arc_control -arc_generator_control \
25461 ▷ ▷ ▷ -projective_space P2 \
25462 ▷ ▷ ▷ -control Control \
25463 ▷ ▷ ▷ -target_size 6 \
25464 ▷ ▷ ▷ -d 2 \
25465 ▷ ▷ -end \
25466 ▷ ▷ -define Orb -orbits \
25467 ▷ ▷ ▷ -on_arcs Arc_control \
25468 ▷ ▷ -end \
25469 ▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q13) 1 \
25470 ▷ ▷ ▷ -define S_%L -cubic_surface -space P -catalogue %L -end \
25471 ▷ ▷ -end_loop L \
25472 ▷ ▷ -print_symbols \
25473 ▷ ▷ -loop L 0 $(NB_CUBIC_SURFACES_Q13) 1 \
25474 ▷ ▷ ▷ -with S_%L -do \
25475 ▷ ▷ ▷ -cubic_surface_activity \
25476 ▷ ▷ ▷ -export_all_quartic_curves Orb \
25477 ▷ ▷ ▷ -end \
25478 ▷ ▷ -end_loop L \

```

```

25479 ▷ ▷ -print_symbols
25480 ▷ $(ORBITER) -v 3 \
25481 ▷ ▷ -csv_file_concatenate_from_mask 0 $(NB_CUBIC_SURFACES_Q13) \
25482 ▷ ▷ ▷ surface_catalogue_q13_iso%ld_quartics.csv \
25483 ▷ ▷ ▷ quartics_q13.csv
25484 ▷ $(ORBITER) -v 5 \
25485 ▷ ▷ -define F -finite_field -q 13 -end \
25486 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
25487 ▷ ▷ -define R -polynomial_ring \
25488 ▷ ▷ ▷ -field F \
25489 ▷ ▷ ▷ -number_of_variables 3 \
25490 ▷ ▷ ▷ -homogeneous_of_degree 4 \
25491 ▷ ▷ ▷ -monomial_ordering_partition \
25492 ▷ ▷ ▷ -variables "y0,y1,y2" "y_0,y_1,y_2" \
25493 ▷ ▷ -end \
25494 ▷ ▷ -define Orb -orbits -classification_by_canonical_form \
25495 ▷ ▷ ▷ -space P \
25496 ▷ ▷ ▷ -ring R \
25497 ▷ ▷ ▷ -input_fname_mask quartics_q13.csv \
25498 ▷ ▷ ▷ -nb_files 1 \
25499 ▷ ▷ ▷ -output_fname quartic_curves_q13 \
25500 ▷ ▷ ▷ -label_po "File" \
25501 ▷ ▷ ▷ -label_so "orbit" \
25502 ▷ ▷ ▷ -carry_through "PT" \
25503 ▷ ▷ ▷ -carry_through "PTCOEFF" \
25504 ▷ ▷ ▷ -carry_through "TANGENT_PLANE_RK" \
25505 ▷ ▷ ▷ -carry_through "abcdef" \
25506 ▷ ▷ ▷ -carry_through "NB_E" \
25507 ▷ ▷ ▷ -carry_through "NB_ET" \
25508 ▷ ▷ ▷ -carry_through "NB_K" \
25509 ▷ ▷ ▷ -carry_through "NB_DOUBLE" \
25510 ▷ ▷ ▷ -carry_through "NB_SINGLE" \
25511 ▷ ▷ ▷ -carry_through "NB_ZERO" \
25512 ▷ ▷ ▷ -carry_through "SEQN" \
25513 ▷ ▷ ▷ -carry_through "SEQNALG" \
25514 ▷ ▷ ▷ -carry_through "curve" \
25515 ▷ ▷ ▷ -carry_through "CURVEALG" \
25516 ▷ ▷ ▷ -label_equation_by_coefficients "curve" \
25517 ▷ ▷ ▷ -label_points "pts_on_curve" \
25518 ▷ ▷ ▷ -label_lines "bitangents" \
25519 ▷ ▷ ▷ -carry_through "Kovalevski_pts" \
25520 ▷ ▷ ▷ -nauty_control -reduce_memory_footprint 1 -end \
25521 ▷ ▷ ▷ -end \
25522 ▷ ▷ -end \
25523 ▷ ▷ -with Orb -do \
25524 ▷ ▷ ▷ -orbits_activity \
25525 ▷ ▷ ▷ -export_source_code \
25526 ▷ ▷ -end \
25527 ▷ ▷ -print_symbols
25528 ▷ #$(PDFLATEX) quartic_curves_q13_orbits.tex
25529 ▷ #$(OPEN) quartic_curves_q13_orbits.pdf
25530
25531
25532 # the file quartics_q13.csv does not exist
25533
25534
25535 quartic_curves_q13_filter_isomorphism_reps:
25536 ▷ $(ORBITER) -v 3 \
25537 ▷ ▷ -csv_file_filter \

```

```

25538 ▷ ▷ ▷ quartic_curves.q13_classified.csv \
25539 ▷ ▷ ▷ "F_Fst" \
25540 ▷ ▷ ▷ "1"
25541 ▷ mv quartic_curves.q13_classified.f_Fst_1.csv quartic_curves.q13_iso.csv
25542
25543 quartic_curves.q13.tally.ago:
25544 ▷ $(ORBITER) -v 3 \
25545 ▷ ▷ -tally_column \
25546 ▷ ▷ quartic_curves.q13_iso.csv AGO
25547
25548
25549
25550
25551 Klein_cubic_threefold.q3:
25552 ▷ $(ORBITER) -v 10 \
25553 ▷ ▷ -define F -finite_field -q 3 -end \
25554 ▷ ▷ -define P -projective_space -n 4 -field F -v 0 -end \
25555 ▷ ▷ -define R -polynomial_ring \
25556 ▷ ▷ ▷ -field F \
25557 ▷ ▷ ▷ -number_of_variables 5 \
25558 ▷ ▷ ▷ -homogeneous_of_degree 3 \
25559 ▷ ▷ ▷ -monomial_ordering_partition \
25560 ▷ ▷ ▷ -variables "X0,X1,X2,X3,X4" "X_0,X_1,X_2,X_3,X_4" \
25561 ▷ ▷ -end \
25562 ▷ ▷ -define V -variety \
25563 ▷ ▷ ▷ -projective_space P \
25564 ▷ ▷ ▷ -ring R \
25565 ▷ ▷ ▷ -equation_in_algebraic_form $(KLEIN_CUBIC_THREEFOLD_EQN) \
25566 ▷ ▷ ▷ -label_txt Klein3.q3 \
25567 ▷ ▷ ▷ -label_tex Klein3\q3 \
25568 ▷ ▷ -end \
25569 ▷ ▷ -with V -do -variety_activity -compute_group -nauty_control -nauty_log "Klein
_cubic_threefold.nauty_log.txt" -end -end \
25570 ▷ ▷ -with V -do -variety_activity -singular_points -end \
25571 ▷ ▷ -with V -do -variety_activity -report -end
25572 ▷ $(PDFLATEX) variety_Klein3.q3.report.tex
25573 ▷ $(OPEN) variety_Klein3.q3.report.pdf
25574 ▷ $(ORBITER) -v 2 \
25575 ▷ ▷ -define Do -draw_options -embedded -radius 150 \
25576 ▷ ▷ ▷ -nodes_empty \
25577 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
25578 ▷ ▷ -tree_draw \
25579 ▷ ▷ ▷ -file Klein_cubic_threefold.nauty_log.txt \
25580 ▷ ▷ ▷ -draw_options Do \
25581 ▷ ▷ -end
25582 ▷ $(PDFLATEX) Klein_cubic_threefold.nauty_log_draw.tex
25583 ▷ $(OPEN) Klein_cubic_threefold.nauty_log_draw.pdf
25584
25585 # 1 singular point
25586 #stabilizer_of_set_of_rational_points::orbit_of_equation_under_set_stabilizer fou
nd an orbit of length 1
25587 #Klein_cubic_threefold.nauty_log.txt
25588
25589
25590
25591 EC.canon:
25592 ▷ $(ORBITER) -v 2 \
25593 ▷ ▷ -define F -finite_field -q 11 -end \
25594 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \

```

```

25595 ▷ ▷ -define Geo -geometric_object P \
25596 ▷ ▷ ▷ -elliptic_curve 1 3 \
25597 ▷ ▷ -end \
25598 ▷ ▷ -define C -combinatorial_object \
25599 ▷ ▷ ▷ -label EC EC \
25600 ▷ ▷ ▷ -geometric_object Geo \
25601 ▷ ▷ -end \
25602 ▷ ▷ -with C -do \
25603 ▷ ▷ -combinatorial_object_activity \
25604 ▷ ▷ ▷ -canonical_form_PG P \
25605 ▷ ▷ ▷ -save_ago \
25606 ▷ ▷ ▷ ▷ -save_transversal \
25607 ▷ ▷ ▷ ▷ -max_TDO_depth 400 \
25608 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "EC.nauty.log.txt" -end \
25609 ▷ ▷ ▷ -end \
25610 ▷ ▷ -end \
25611 ▷ ▷ -define Doinc -draw_incidence_structure_options \
25612 ▷ ▷ ▷ -width 60 -width_10 6 \
25613 ▷ ▷ -end \
25614 ▷ ▷ -with C -do \
25615 ▷ ▷ -combinatorial_object_activity \
25616 ▷ ▷ ▷ -post_processing \
25617 ▷ ▷ -end \
25618 ▷ ▷ -with C -do \
25619 ▷ ▷ -combinatorial_object_activity \
25620 ▷ ▷ ▷ -report \
25621 ▷ ▷ ▷ ▷ -export_flag_orbits \
25622 ▷ ▷ ▷ ▷ -show_TDO \
25623 ▷ ▷ ▷ ▷ -show_TDA \
25624 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
25625 ▷ ▷ ▷ ▷ -export_group_GAP \
25626 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
25627 ▷ ▷ ▷ -end \
25628 ▷ ▷ -end
25629 ▷ $(PDFLATEX) EC_classification.tex
25630 ▷ $(OPEN) EC_classification.pdf
25631 ▷ $(ORBITER) -v 2 -draw_matrix \
25632 ▷ ▷ -input_csv_file EC_object_0_TDA_flag_orbits.csv \
25633 ▷ ▷ -secondary_input_csv_file EC_object_0_TDA.csv \
25634 ▷ ▷ -box_width 20 -bit_depth 24 \
25635 ▷ ▷ -end
25636 ▷ $(OPEN) EC_object_0_TDA_flag_orbits.draw.bmp
25637
25638
25639
25640
25641 Hirschfeld.q4.c:
25642 ▷ $(ORBITER) -v 2 \
25643 ▷ ▷ -define F -finite_field -q 4 -end \
25644 ▷ ▷ -define R -polynomial_ring -field F \
25645 ▷ ▷ ▷ -number_of_variables 4 \
25646 ▷ ▷ ▷ -homogeneous_of_degree 3 \
25647 ▷ ▷ ▷ -end \
25648 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
25649 ▷ ▷ -assign Eqn -from \
25650 ▷ ▷ -with R -do \
25651 ▷ ▷ ▷ -ring_theoretic_activity \
25652 ▷ ▷ ▷ -parse_equation_wo_parameters \
25653 ▷ ▷ ▷ ▷ "Hirschfeld.surface.q4" "Hirschfeld\surface\_q4" \

```

```

25654 ▷ ▷ ▷ ▷ $(HIRSCHFELD_SURFACE_EQUATION_ALGEBRAIC_FORM) \
25655 ▷ ▷ -end \
25656 ▷ ▷ -define Geo -geometric_object P \
25657 ▷ ▷ ▷ -projective_variety R \
25658 ▷ ▷ ▷ ▷ "Hirschfeld_surface.q4" "Hirschfeld\_surface\_q4" \
25659 ▷ ▷ ▷ ▷ Eqn \
25660 ▷ ▷ -end \
25661 ▷ ▷ -define C -combinatorial_object \
25662 ▷ ▷ ▷ -label Hirschfeld_surface.q4 Hirschfeld\_surface\_q4 \
25663 ▷ ▷ ▷ -geometric_object Geo \
25664 ▷ ▷ -end \
25665 ▷ ▷ -with C -do \
25666 ▷ ▷ -combinatorial_object_activity \
25667 ▷ ▷ ▷ -canonical_form_PG P \
25668 ▷ ▷ ▷ ▷ -save_ago \
25669 ▷ ▷ ▷ ▷ -max_TDO_depth 10 \
25670 ▷ ▷ ▷ ▷ -nauty_control \
25671 ▷ ▷ ▷ ▷ ▷ -nauty_log "Hirschfeld.q4.nauty.log.txt" \
25672 ▷ ▷ ▷ ▷ ▷ -save_nauty_input_graphs Hirschfeld.q4 \
25673 ▷ ▷ ▷ ▷ -end \
25674 ▷ ▷ ▷ -end \
25675 ▷ ▷ -end \
25676 ▷ ▷ -define Doinc -draw_incidence_structure_options \
25677 ▷ ▷ ▷ -width 60 -width_10 6 \
25678 ▷ ▷ -end \
25679 ▷ ▷ -with C -do \
25680 ▷ ▷ -combinatorial_object_activity \
25681 ▷ ▷ ▷ -post_processing \
25682 ▷ ▷ -end \
25683 ▷ ▷ -with C -do \
25684 ▷ ▷ -combinatorial_object_activity \
25685 ▷ ▷ ▷ -report \
25686 ▷ ▷ ▷ ▷ -export_flag_orbits \
25687 ▷ ▷ ▷ ▷ -show_TDO \
25688 ▷ ▷ ▷ ▷ -show_TDA \
25689 ▷ ▷ ▷ ▷ -export_labels \
25690 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
25691 ▷ ▷ ▷ ▷ -export_group_GAP \
25692 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
25693 ▷ ▷ ▷ -end \
25694 ▷ ▷ -end
25695 ▷ $(PDFLATEX) Hirschfeld_surface.q4_classification.tex
25696 ▷ $(OPEN) Hirschfeld_surface.q4_classification.pdf
25697 ▷ $(ORBITER) -v 2 \
25698 ▷ ▷ -define Do -draw_options -embedded -radius 150 \
25699 ▷ ▷ ▷ -nodes_empty \
25700 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
25701 ▷ ▷ -tree_draw \
25702 ▷ ▷ ▷ -file Hirschfeld.q4.nauty.log.txt \
25703 ▷ ▷ ▷ -draw_options Do \
25704 ▷ ▷ -end
25705 ▷ $(PDFLATEX) Hirschfeld.q4.nauty.log_draw.tex
25706 ▷ $(OPEN) Hirschfeld.q4.nauty.log_draw.pdf
25707
25708
25709 #Hirschfeld.q4.nauty.log.txt
25710 # group order is 51840
25711
25712

```

```

25713 Hirschfeld_stab_group:
25714 ▷ $(ORBITER) -v 9 \
25715 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
25716 ▷ ▷ -define Do -draw_options \
25717 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
25718 ▷ ▷ -end \
25719 ▷ ▷ -define G -linear_group -PGGL 4 4 \
25720 ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld.Stab" \
25721 ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
25722 ▷ ▷ ▷ -end \
25723 ▷ ▷ -with G -do \
25724 ▷ ▷ -group_theoretic_activity \
25725 ▷ ▷ ▷ -report Do \
25726 ▷ ▷ -end
25727 ▷ $(PDFLATEX) PGGL_4_4_Subgroup.Hirschfeld.Stab.51840_report.tex
25728 ▷ $(OPEN) PGGL_4_4_Subgroup.Hirschfeld.Stab.51840_report.pdf
25729
25730 Hirschfeld_stab_subgroup:
25731 ▷ $(ORBITER) -v 9 \
25732 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
25733 ▷ ▷ -define Do -draw_options \
25734 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
25735 ▷ ▷ -end \
25736 ▷ ▷ -define G -linear_group -PGGL 4 4 \
25737 ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld.Stab" \
25738 ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
25739 ▷ ▷ ▷ -end \
25740 ▷ ▷ -define Gsp -modified_group -from G \
25741 ▷ ▷ ▷ -create_special_subgroup \
25742 ▷ ▷ -end \
25743 ▷ ▷ -with Gsp -do \
25744 ▷ ▷ -group_theoretic_activity \
25745 ▷ ▷ ▷ -report Do \
25746 ▷ ▷ -end \
25747 ▷ ▷ -define Orb -orbits -group Gsp \
25748 ▷ ▷ ▷ -on_points \
25749 ▷ ▷ -end
25750
25751 Hirschfeld_stab_orbits:
25752 ▷ $(ORBITER) -v 9 \
25753 ▷ ▷ -orbiter_path $(ORBITER_EXE_PATH) \
25754 ▷ ▷ -define Do -draw_options \
25755 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
25756 ▷ ▷ -end \
25757 ▷ ▷ -define G -linear_group -PGGL 4 4 \
25758 ▷ ▷ ▷ -subgroup_by_generators "Hirschfeld.Stab" \
25759 ▷ ▷ ▷ 51840 6 $(HIRSCHFELD_STAB_GENERATORS) \
25760 ▷ ▷ ▷ -end \
25761 ▷ ▷ -with G -do \
25762 ▷ ▷ -group_theoretic_activity \
25763 ▷ ▷ ▷ -report Do \
25764 ▷ ▷ -end \
25765 ▷ ▷ -define Orb -orbits -group G \
25766 ▷ ▷ ▷ -on_points \
25767 ▷ ▷ -end \
25768 ▷ ▷ -with Orb -do -orbits_activity \
25769 ▷ ▷ ▷ -export_something "orbit" 0 \
25770 ▷ ▷ -end
25771

```

```

25772 #orbits.PGGL_4.4.Subgroup_Hirschfeld_Stab.51840_orbit_0.csv
25773
25774
25775 Hirschfeld.q4.set.c:
25776 ▷ $(ORBITER) -v 4 \
25777 ▷ ▷ -define H -set -here \
25778 ▷ ▷ ▷ $(HIRSCHFELD_SURFACE_Q4_SET_OF_POINTS) \
25779 ▷ ▷ -end \
25780 ▷ ▷ -define C -combinatorial.object \
25781 ▷ ▷ ▷ -label Hirschfeld_surface_q4 Hirschfeld\_surface\_q4 \
25782 ▷ ▷ ▷ -set_of_points H \
25783 ▷ ▷ -end \
25784 ▷ ▷ -define F -finite_field -q 4 -end \
25785 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
25786 ▷ ▷ -with C -do \
25787 ▷ ▷ -combinatorial_object_activity \
25788 ▷ ▷ ▷ -canonical_form_PG P \
25789 ▷ ▷ ▷ ▷ -save_ago \
25790 ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
25791 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "Hirschfeld.q4.set.nauty.log.txt" -end \
25792 ▷ ▷ ▷ -end \
25793 ▷ ▷ -end \
25794 ▷ ▷ -define Doinc -draw_incidence_structure_options \
25795 ▷ ▷ ▷ -width 40 -width_10 6 \
25796 ▷ ▷ -end \
25797 ▷ ▷ -with C -do \
25798 ▷ ▷ -combinatorial_object_activity \
25799 ▷ ▷ ▷ -post_processing \
25800 ▷ ▷ -end \
25801 ▷ ▷ -with C -do \
25802 ▷ ▷ -combinatorial_object_activity \
25803 ▷ ▷ ▷ -report \
25804 ▷ ▷ ▷ ▷ -show_TDO \
25805 ▷ ▷ ▷ ▷ -show_TDA \
25806 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
25807 ▷ ▷ ▷ -end \
25808 ▷ ▷ -end
25809 ▷ $(PDFLATEX) Hirschfeld_surface.q4_classification.tex
25810 ▷ $(OPEN) Hirschfeld_surface.q4_classification.pdf
25811
25812
25813
25814 Dickson.sets.stabilizer:
25815 ▷ $(ORBITER) -v 3 \
25816 ▷ ▷ -define C -combinatorial.object \
25817 ▷ ▷ ▷ -label Dickson.sets Dickson\_sets \
25818 ▷ ▷ ▷ -set_of_points "0,1,2,5,6" \
25819 ▷ ▷ ▷ -set_of_points "0,1,2,3,6" \
25820 ▷ ▷ ▷ -set_of_points "0,1,2,3,4" \
25821 ▷ ▷ ▷ -set_of_points "0,1,2,3,8" \
25822 ▷ ▷ ▷ -set_of_points "0,1,2,5,6,7,8" \
25823 ▷ ▷ ▷ -set_of_points "0,1,2,3,5,6,7" \
25824 ▷ ▷ ▷ -set_of_points "0,1,2,3,5,6,9" \
25825 ▷ ▷ ▷ -set_of_points "0,1,2,3,5,6,10" \
25826 ▷ ▷ ▷ -set_of_points "0,1,2,3,5,6,4" \
25827 ▷ ▷ ▷ -set_of_points "0,1,2,3,8,11,13" \
25828 ▷ ▷ ▷ -set_of_points "3,6,9,7,10,12,8,11,13,14,4" \
25829 ▷ ▷ ▷ -set_of_points "3,5,6,9,7,10,12,11,13,14,4" \
25830 ▷ ▷ ▷ -set_of_points "0,1,2,3,5,6,9,7,10,12,4" \

```

```

25831 ▷ ▷ -end \
25832 ▷ ▷ -define F -finite_field -q 2 -end \
25833 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
25834 ▷ ▷ -with C -do \
25835 ▷ ▷ -combinatorial_object_activity \
25836 ▷ ▷ ▷ -canonical_form_PG P \
25837 ▷ ▷ ▷ ▷ -save_ago \
25838 ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
25839 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "Dickson_sets_nauty_log.txt" -end \
25840 ▷ ▷ ▷ -end \
25841 ▷ ▷ -end \
25842 ▷ ▷ -define Doinc -draw_incidence_structure_options \
25843 ▷ ▷ ▷ -width 40 -width_10 6 \
25844 ▷ ▷ -end \
25845 ▷ ▷ -with C -do \
25846 ▷ ▷ -combinatorial_object_activity \
25847 ▷ ▷ ▷ -post_processing \
25848 ▷ ▷ -end \
25849 ▷ ▷ -with C -do \
25850 ▷ ▷ -combinatorial_object_activity \
25851 ▷ ▷ ▷ -report \
25852 ▷ ▷ ▷ ▷ -show_TDO \
25853 ▷ ▷ ▷ ▷ -show_TDA \
25854 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
25855 ▷ ▷ ▷ -end \
25856 ▷ ▷ -end
25857 ▷ $(PDFLATEX) Dickson_sets_classification.tex
25858 ▷ $(OPEN) Dickson_sets_classification.pdf
25859
25860
25861
25862 Endrass_7c:
25863 ▷ $(ORBITER) -v 2 \
25864 ▷ ▷ -define F -finite_field -q 7 -end \
25865 ▷ ▷ -define R -polynomial_ring -field F \
25866 ▷ ▷ ▷ -number_of_variables 4 \
25867 ▷ ▷ ▷ -homogeneous_of_degree 8 \
25868 ▷ ▷ ▷ -monomial_ordering_partition \
25869 ▷ ▷ ▷ -end \
25870 ▷ ▷ -define eqn -vector -field F -sparse 165 \
25871 ▷ ▷ ▷ $(ENDRASS_SPARSE) -end \
25872 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
25873 ▷ ▷ -define Endrass_F7 -geometric_object P \
25874 ▷ ▷ ▷ -projective_variety R \
25875 ▷ ▷ ▷ ▷ "Endrass_F7" \
25876 ▷ ▷ ▷ ▷ "Endrass\F7" \
25877 ▷ ▷ ▷ ▷ eqn \
25878 ▷ ▷ -end \
25879 ▷ ▷ -with Endrass_F7 -do \
25880 ▷ ▷ -combinatorial_object_activity -save \
25881 ▷ ▷ -end
25882 ▷ $(ORBITER) -v 2 \
25883 ▷ ▷ -define C -combinatorial_object \
25884 ▷ ▷ ▷ -label Endrass_F7 Endrass\F7 \
25885 ▷ ▷ ▷ -file_of_points Endrass_F7.txt \
25886 ▷ ▷ -end \
25887 ▷ ▷ -define F -finite_field -q 7 -end \
25888 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
25889 ▷ ▷ -with C -do \

```



```

25890 ▷ ▷ -combinatorial_object_activity \
25891 ▷ ▷ ▷ -canonical_form_PG P \
25892 ▷ ▷ ▷ ▷ -save_ago \
25893 ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
25894 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "Endrass_nauty_log.txt" -end \
25895 ▷ ▷ ▷ -end \
25896 ▷ ▷ -end \
25897 ▷ ▷ -define Doinc -draw_incidence_structure_options \
25898 ▷ ▷ ▷ -width 40 -width_10 6 \
25899 ▷ ▷ -end \
25900 ▷ ▷ -with C -do \
25901 ▷ ▷ -combinatorial_object_activity \
25902 ▷ ▷ ▷ -post_processing \
25903 ▷ ▷ -end \
25904 ▷ ▷ -with C -do \
25905 ▷ ▷ -combinatorial_object_activity \
25906 ▷ ▷ ▷ -report \
25907 ▷ ▷ ▷ -incidence_draw_options Doinc \
25908 ▷ ▷ ▷ -end \
25909 ▷ ▷ -end
25910 ▷ $(PDFLATEX) Endrass_F7_classification.tex
25911 ▷ $(OPEN) Endrass_F7_classification.pdf
25912
25913 #▷ ▷ ▷ ▷ -show_TDO \
25914 #▷ ▷ ▷ ▷ -show_TDA \
25915 # ToDo latex file is too big
25916 # ToDo group order is computed wrong
25917 # should be: group order is 32
25918
25919
25920 #ENDRASS_SPARSE is undefined
25921
25922
25923
25924 hyperoval_16_canonical_form:
25925 ▷ $(ORBITER) -v 2 \
25926 ▷ ▷ -define C -combinatorial_object \
25927 ▷ ▷ ▷ -label hyperoval_q16 hyperoval\_q16 \
25928 ▷ ▷ ▷ -set_of_points $(HYPEROVAL_16_16320) \
25929 ▷ ▷ ▷ -set_of_points $(HYPEROVAL_16_144) \
25930 ▷ ▷ -end \
25931 ▷ ▷ -define F -finite_field -q 16 -end \
25932 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
25933 ▷ ▷ -with C -do \
25934 ▷ ▷ -combinatorial_object_activity \
25935 ▷ ▷ ▷ -canonical_form_PG P \
25936 ▷ ▷ ▷ ▷ -save_ago \
25937 ▷ ▷ ▷ ▷ -save_transversal \
25938 ▷ ▷ ▷ ▷ -max_TDO_depth 10 \
25939 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "hyperoval_16_nauty_log.txt" -end \
25940 ▷ ▷ ▷ -end \
25941 ▷ ▷ -end \
25942 ▷ ▷ -define Doinc -draw_incidence_structure_options \
25943 ▷ ▷ ▷ -width 40 -width_10 6 \
25944 ▷ ▷ -end \
25945 ▷ ▷ -with C -do \
25946 ▷ ▷ -combinatorial_object_activity \
25947 ▷ ▷ ▷ -post_processing \
25948 ▷ ▷ -end \

```

```

25949 ▷ ▷ -with C -do \
25950 ▷ ▷ -combinatorial_object_activity \
25951 ▷ ▷ ▷ -report \
25952 ▷ ▷ ▷ ▷ -export_flag_orbits \
25953 ▷ ▷ ▷ ▷ -show_TDO \
25954 ▷ ▷ ▷ ▷ -show_TDA \
25955 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
25956 ▷ ▷ ▷ ▷ -export_group_GAP \
25957 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
25958 ▷ ▷ ▷ -end \
25959 ▷ ▷ -end
25960 ▷ $(PDFLATEX) hyperoval_q16_classification.tex
25961 ▷ $(OPEN) hyperoval_q16_classification.pdf
25962 ▷ $(ORBITER) -v 2 -draw_matrix \
25963 ▷ ▷ -input_csv_file hyperoval_q16_object_0_TDA_flag_orbits.csv \
25964 ▷ ▷ -secondary_input_csv_file hyperoval_q16_object_0_TDA.csv \
25965 ▷ ▷ -box_width 4 -bit_depth 24 \
25966 ▷ ▷ -end
25967 ▷ $(OPEN) hyperoval_q16_object_0_TDA_flag_orbits.draw.bmp
25968 ▷ $(ORBITER) -v 2 -draw_matrix \
25969 ▷ ▷ -input_csv_file hyperoval_q16_object_1_TDA_flag_orbits.csv \
25970 ▷ ▷ -secondary_input_csv_file hyperoval_q16_object_1_TDA.csv \
25971 ▷ ▷ -box_width 4 -bit_depth 24 \
25972 ▷ ▷ -end
25973 ▷ $(OPEN) hyperoval_q16_object_1_TDA_flag_orbits.draw.bmp
25974 ▷ $(ORBITER) -v 2 \
25975 ▷ ▷ -define Do -draw_options -embedded -radius 150 \
25976 ▷ ▷ ▷ -nodes_empty \
25977 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
25978 ▷ ▷ -tree_draw \
25979 ▷ ▷ ▷ -file hyperoval_16_nauty_log.txt \
25980 ▷ ▷ ▷ -draw_options Do \
25981 ▷ ▷ -end
25982 ▷ $(PDFLATEX) hyperoval_16_nauty_log_draw.tex
25983 ▷ $(OPEN) hyperoval_16_nauty_log_draw.pdf
25984
25985
25986 #hyperoval_16_nauty_log.txt
25987
25988
25989 cubic_curves_PG_2.8.canon:
25990 ▷ $(ORBITER) -v 6 \
25991 ▷ ▷ -define C -combinatorial_object \
25992 ▷ ▷ ▷ -label cc_8 cc\8 \
25993 ▷ ▷ ▷ -set_of_points "2,3,28,46,51,61,40,71" \
25994 ▷ ▷ -end \
25995 ▷ ▷ -define F -finite_field -q 8 -end \
25996 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
25997 ▷ ▷ -with C -do \
25998 ▷ ▷ -combinatorial_object_activity \
25999 ▷ ▷ ▷ -canonical_form_PG P \
26000 ▷ ▷ ▷ ▷ -save_ago \
26001 ▷ ▷ ▷ ▷ -max_TDO_depth 10 \
26002 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "cubic_curves_q8_nauty_log.txt" -show_canonical
    _form -end \
26003 ▷ ▷ ▷ -end \
26004 ▷ ▷ -end \
26005 ▷ ▷ -define Doinc -draw_incidence_structure_options \
26006 ▷ ▷ ▷ -width 40 -width_10 6 \

```

```

26007 ▷ ▷ -end \
26008 ▷ ▷ -with C -do \
26009 ▷ ▷ -combinatorial_object_activity \
26010 ▷ ▷ ▷ -post_processing \
26011 ▷ ▷ -end \
26012 ▷ ▷ -with C -do \
26013 ▷ ▷ -combinatorial_object_activity \
26014 ▷ ▷ ▷ -report \
26015 ▷ ▷ ▷ ▷ -export_flag_orbits \
26016 ▷ ▷ ▷ ▷ -show_TDO \
26017 ▷ ▷ ▷ ▷ -show_TDA \
26018 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
26019 ▷ ▷ ▷ ▷ -export_group_GAP \
26020 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
26021 ▷ ▷ ▷ -end \
26022 ▷ ▷ -end
26023 ▷ $(PDFLATEX) cc_8_classification.tex
26024 ▷ $(OPEN) cc_8_classification.pdf
26025
26026
26027
26028
26029 ovoid.q8.canon:
26030 ▷ $(ORBITER) -v 2 \
26031 ▷ ▷ -define F -finite_field -q 8 -end \
26032 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
26033 ▷ ▷ -define Geo -geometric_object P \
26034 ▷ ▷ ▷ -elliptic_quadric_ovoid \
26035 ▷ ▷ -end \
26036 ▷ ▷ -define C -combinatorial_object \
26037 ▷ ▷ ▷ -label ovoid ovoid \
26038 ▷ ▷ ▷ -geometric_object Geo \
26039 ▷ ▷ -end \
26040 ▷ ▷ -with C -do \
26041 ▷ ▷ -combinatorial_object_activity \
26042 ▷ ▷ ▷ -canonical_form_PG P \
26043 ▷ ▷ ▷ ▷ -save_ago \
26044 ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
26045 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "ovoid.q8.nauty_log.txt" -show_canonical_form
        -end \
26046 ▷ ▷ ▷ -end \
26047 ▷ ▷ -end \
26048 ▷ ▷ -define Doinc -draw_incidence_structure_options \
26049 ▷ ▷ ▷ -width 40 -width_10 6 \
26050 ▷ ▷ -end \
26051 ▷ ▷ -with C -do \
26052 ▷ ▷ -combinatorial_object_activity \
26053 ▷ ▷ ▷ -post_processing \
26054 ▷ ▷ -end \
26055 ▷ ▷ -with C -do \
26056 ▷ ▷ -combinatorial_object_activity \
26057 ▷ ▷ ▷ -report \
26058 ▷ ▷ ▷ ▷ -show_TDO \
26059 ▷ ▷ ▷ ▷ -show_TDA \
26060 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
26061 ▷ ▷ ▷ ▷ -export_group_GAP \
26062 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
26063 ▷ ▷ ▷ -end \
26064 ▷ ▷ -end

```

```

26065 ▷ $(PDFLATEX) ovoid_classification.tex
26066 ▷ $(OPEN) ovoid_classification.pdf
26067
26068 # group order 1572480
26069
26070 ovoid_ST_q8_canon:
26071 ▷ $(ORBITER) -v 2 \
26072 ▷ ▷ -define F -finite_field -q 8 -end \
26073 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
26074 ▷ ▷ -define Geo -geometric_object P \
26075 ▷ ▷ ▷ -ovoid_ST \
26076 ▷ ▷ -end \
26077 ▷ ▷ -define C -combinatorial_object \
26078 ▷ ▷ ▷ -label ovoid_ST ovoid\_ST \
26079 ▷ ▷ ▷ -geometric_object Geo \
26080 ▷ ▷ -end \
26081 ▷ ▷ -with C -do \
26082 ▷ ▷ -combinatorial_object_activity \
26083 ▷ ▷ ▷ -canonical_form_PG P \
26084 ▷ ▷ ▷ ▷ -save_ago \
26085 ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
26086 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "ovoid_ST_nauty_log.txt" -end \
26087 ▷ ▷ ▷ -end \
26088 ▷ ▷ -end \
26089 ▷ ▷ -define Doinc -draw_incidence_structure_options \
26090 ▷ ▷ ▷ -width 40 -width_10 6 \
26091 ▷ ▷ -end \
26092 ▷ ▷ -with C -do \
26093 ▷ ▷ -combinatorial_object_activity \
26094 ▷ ▷ ▷ -post_processing \
26095 ▷ ▷ -end \
26096 ▷ ▷ -with C -do \
26097 ▷ ▷ -combinatorial_object_activity \
26098 ▷ ▷ ▷ -report \
26099 ▷ ▷ ▷ ▷ -show_TDO \
26100 ▷ ▷ ▷ ▷ -show_TDA \
26101 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
26102 ▷ ▷ ▷ ▷ -export_group_GAP \
26103 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
26104 ▷ ▷ ▷ -end \
26105 ▷ ▷ -end
26106 ▷ $(PDFLATEX) ovoid_ST_classification.tex
26107 ▷ $(OPEN) ovoid_ST_classification.pdf
26108
26109 #User time: 0:8
26110 #ago = 87360
26111
26112 # note: 18 * 87360 = 1572480
26113
26114
26115
26116 Suzuki_8:
26117 ▷ $(ORBITER) -v 6 \
26118 ▷ ▷ -define Do -draw_options \
26119 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
26120 ▷ ▷ -end \
26121 ▷ ▷ -define F -finite_field -q 8 -end \
26122 ▷ ▷ -define gens -vector -field F \
26123 ▷ ▷ ▷ -compact $(SUZUKI_8_GENERATORS) -end \

```

```

26124 ▷ ▷ -define G -linear_group -PGL 4 8 \
26125 ▷ ▷ -subgroup_by_generators "Sz8" "87360" 5 gens \
26126 ▷ ▷ ▷ -end \
26127 ▷ ▷ -with G -do \
26128 ▷ ▷ -group_theoretic_activity \
26129 ▷ ▷ ▷ -report Do \
26130 ▷ ▷ -end
26131 ▷ $(PDFLATEX) PGL_4_8_Subgroup.Sz8.87360_report.tex
26132 ▷ $(OPEN) PGL_4_8_Subgroup.Sz8.87360_report.pdf
26133
26134 #User time: 0.010000 of a second, dt=1 tps = 100
26135
26136
26137 quartic_curve_25_0_0.canonical:
26138 ▷ $(ORBITER) -v 3 \
26139 ▷ ▷ -define F -finite_field -q 25 -end \
26140 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
26141 ▷ ▷ -define C -combinatorial_object \
26142 ▷ ▷ ▷ -label quartic_25_0_0 quartic\25\0\0 \
26143 ▷ ▷ ▷ -set_of_points "10,11,59,63,124,135,136,170,206,257,275,284,285,367,378,393
,433,619,641,644" \
26144 ▷ ▷ ▷ -set_of_points "9, 24, 62, 67, 77, 84, 87, 89, 125, 130, 158, 172, 197, 219
, 266, 271, 325, 356, 391, 392, 400, 429, 454, 458, 470, 503, 531, 553, 605, 625,
627, 646" \
26145 ▷ ▷ ▷ -set_of_points "9, 24, 55, 75, 79, 88, 93, 94, 112, 142, 186, 213, 230, 249
, 316, 329, 337, 371, 381, 383, 392, 405, 417, 439, 464, 466, 470, 524, 530, 567,
583, 594" \
26146 ▷ ▷ ▷ -set_of_points "10, 23, 34, 51, 65, 83, 117, 126, 144, 146, 147, 159, 172,
181, 185, 197, 198, 207, 240, 281, 283, 293, 300, 301, 305, 346, 357, 384, 409, 4
19, 444, 459, 463, 465, 468, 514, 533, 543, 547, 549, 586, 615, 618, 628" \
26147 ▷ ▷ ▷ -set_of_points "2, 12, 48, 65, 87, 120, 189, 246, 305, 323, 354, 375, 434,
435, 455, 482, 496, 557, 586, 595" \
26148 ▷ ▷ -end \
26149 ▷ ▷ -with C -do \
26150 ▷ ▷ -combinatorial_object_activity \
26151 ▷ ▷ ▷ -canonical_form_PG P \
26152 ▷ ▷ ▷ ▷ -save_ago \
26153 ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
26154 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "quartic_curve_q25_0.nauty.log.txt" -show_canon
ical_form -end \
26155 ▷ ▷ ▷ -end \
26156 ▷ ▷ -end \
26157 ▷ ▷ -define Doinc -draw_incidence_structure_options \
26158 ▷ ▷ ▷ -width 40 -width_10 6 \
26159 ▷ ▷ -end \
26160 ▷ ▷ -with C -do \
26161 ▷ ▷ -combinatorial_object_activity \
26162 ▷ ▷ ▷ -post_processing \
26163 ▷ ▷ -end \
26164 ▷ ▷ -with C -do \
26165 ▷ ▷ -combinatorial_object_activity \
26166 ▷ ▷ ▷ -report \
26167 ▷ ▷ ▷ ▷ -show_TDO \
26168 ▷ ▷ ▷ ▷ -show_TDA \
26169 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
26170 ▷ ▷ ▷ ▷ -export_group_GAP \
26171 ▷ ▷ ▷ -incidence_draw_options Doinc \
26172 ▷ ▷ ▷ -end \
26173 ▷ ▷ -end

```

```

26174 ▷ #$(PDFLATEX) quartic.25.0.0_classification.tex
26175 ▷ #$(OPEN) quartic.25.0.0_classification.pdf
26176
26177 #User time: 0:1
26178 #quartic_curve.q25.0.nauty.log.txt
26179
26180
26181
26182 CRC_BCH_Q16_N51_D3_canonical:
26183 ▷ $(ORBITER) -v 12 \
26184 ▷ ▷ -define F -finite_field -q 16 -override_polynomial 19 -end \
26185 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
26186 ▷ ▷ -define C -combinatorial_object \
26187 ▷ ▷ ▷ -label BCH_Q16_N51 BCH\_Q16\_N51 \
26188 ▷ ▷ ▷ -set_of_points $(CRC_BCH_Q16_N51_D3_PROJECTIVE_SET) \
26189 ▷ ▷ -end \
26190 ▷ ▷ -with C -do \
26191 ▷ ▷ -combinatorial_object_activity \
26192 ▷ ▷ ▷ -canonical_form_PG P \
26193 ▷ ▷ ▷ -save_ago \
26194 ▷ ▷ ▷ -max_TDO_depth 4 \
26195 ▷ ▷ ▷ -nauty_control -nauty_log "CRC_BCH_nauty_log.txt" -end \
26196 ▷ ▷ ▷ -end \
26197 ▷ ▷ -end \
26198 ▷ ▷ -define Doinc -draw_incidence_structure_options \
26199 ▷ ▷ ▷ -width 40 -width_10 6 \
26200 ▷ ▷ -end \
26201 ▷ ▷ -with C -do \
26202 ▷ ▷ -combinatorial_object_activity \
26203 ▷ ▷ ▷ -post_processing \
26204 ▷ ▷ -end \
26205 ▷ ▷ -with C -do \
26206 ▷ ▷ -combinatorial_object_activity \
26207 ▷ ▷ ▷ -report \
26208 ▷ ▷ ▷ ▷ -show_TDO \
26209 ▷ ▷ ▷ ▷ -show_TDA \
26210 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
26211 ▷ ▷ ▷ ▷ -export_group_GAP \
26212 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
26213 ▷ ▷ ▷ -end \
26214 ▷ ▷ -end
26215 ▷ $(PDFLATEX) BCH_Q16_N51_classification.tex
26216 ▷ $(OPEN) BCH_Q16_N51_classification.pdf
26217
26218 # too slow
26219
26220 # stabilizer of order 408 is generated by:
26221 #1,0,0,0,0,0,1,0,1,5,9,3,12,10,8,10,1, 0,1,0,0,0,0,1,0,0,0,0,1,1,2,13,5,0,
26222
26223
26224 Veronese_1.3.q7.c:
26225 ▷ $(ORBITER) -v 6 \
26226 ▷ ▷ -define C -combinatorial_object \
26227 ▷ ▷ ▷ -label veronese_1.3.q7 veronese\_1\_3\_q7 \
26228 ▷ ▷ ▷ -set_of_points "3, 0, 4, 268, 336, 184, 224, 364" \
26229 ▷ ▷ -end \
26230 ▷ ▷ -define F -finite_field -q 7 -end \
26231 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
26232 ▷ ▷ -with C -do \

```

```

26233 ▷ ▷ -combinatorial_object_activity \
26234 ▷ ▷ ▷ -canonical_form_PG P \
26235 ▷ ▷ ▷ ▷ -save_ago \
26236 ▷ ▷ ▷ ▷ -max_TDO_depth 4 \
26237 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "Veronese_1.3.q7.nauty.log.txt" -end \
26238 ▷ ▷ ▷ -end \
26239 ▷ ▷ -end \
26240 ▷ ▷ -define Doinc -draw_incidence_structure_options \
26241 ▷ ▷ ▷ -width 40 -width_10 6 \
26242 ▷ ▷ -end \
26243 ▷ ▷ -with C -do \
26244 ▷ ▷ -combinatorial_object_activity \
26245 ▷ ▷ ▷ -post_processing \
26246 ▷ ▷ -end \
26247 ▷ ▷ -with C -do \
26248 ▷ ▷ -combinatorial_object_activity \
26249 ▷ ▷ ▷ -report \
26250 ▷ ▷ ▷ ▷ -show_TDO \
26251 ▷ ▷ ▷ ▷ -show_TDA \
26252 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
26253 ▷ ▷ ▷ ▷ -export_group_GAP \
26254 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
26255 ▷ ▷ ▷ -end \
26256 ▷ ▷ -end
26257 ▷ $(PDFLATEX) veronese_1.3.q7_classification.tex
26258 ▷ $(OPEN) veronese_1.3.q7_classification.pdf
26259
26260 # ago = 336
26261
26262
26263
26264 #####
26265 # Section 17.3: Incidence Geometries
26266
26267
26268 SECTION_INCIDENCE_GEOMETRIES:
26269
26270
26271
26272 test_17_3:
26273 ▷ make geo_7_3_c
26274 ▷ make geo_10_3_c
26275 ▷ #make geo_10_3_c.lex_least
26276 ▷ #make geo_14_3_c
26277 ▷ make geo_15_3_c.filter72
26278 ▷ make geo_15_3.filtered_c
26279 ▷ make TFC_24_3_c
26280 ▷ #make geo_40_4.g4_c
26281 ▷ #make geo_17_3.g4_c
26282 ▷ make AG_2_3_c
26283 ▷ make geo_LSQ6_c
26284
26285
26286 geo_7_3_c:
26287 ▷ $(ORBITER) -v 2 \
26288 ▷ ▷ -define Test_lines -set -loop 3 5 1 -end \
26289 ▷ ▷ -define Geo -geometry_builder \
26290 ▷ ▷ ▷ -V 7 -B 7 -TDO 3 \
26291 ▷ ▷ ▷ -fuse 1 \

```

```

26292 ▷ ▷ ▷ -fname_GEO 7.3 \
26293 ▷ ▷ ▷ -output_to_inc_file \
26294 ▷ ▷ ▷ -test Test_lines \
26295 ▷ ▷ -end
26296 ▷ $(ORBITER) -v 10 \
26297 ▷ ▷ -define Doinc -draw_incidence_structure_options \
26298 ▷ ▷ ▷ -width 60 -width_10 6 -end \
26299 ▷ ▷ -define C -combinatorial_object \
26300 ▷ ▷ ▷ -label geo_7.3 geo\_7\_3 \
26301 ▷ ▷ ▷ -file_of_incidence_geometries 7.3.inc 7 7 21 \
26302 ▷ ▷ -end \
26303 ▷ ▷ -with C -do \
26304 ▷ ▷ -combinatorial_object_activity \
26305 ▷ ▷ ▷ -canonical_form \
26306 ▷ ▷ ▷ ▷ -save_ago \
26307 ▷ ▷ ▷ ▷ -save_transversal \
26308 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "geo_7.3.nauty_log.txt" -show_canonical_form -e
        nd \
26309 ▷ ▷ ▷ -end \
26310 ▷ ▷ -end \
26311 ▷ ▷ -with C -do \
26312 ▷ ▷ -combinatorial_object_activity \
26313 ▷ ▷ ▷ -post_processing \
26314 ▷ ▷ -end \
26315 ▷ ▷ -with C -do \
26316 ▷ ▷ -combinatorial_object_activity \
26317 ▷ ▷ ▷ -report \
26318 ▷ ▷ ▷ ▷ -export_flag_orbits \
26319 ▷ ▷ ▷ ▷ -show_incidence_matrices \
26320 ▷ ▷ ▷ ▷ -export_group_GAP \
26321 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
26322 ▷ ▷ ▷ -end \
26323 ▷ ▷ -end
26324 ▷ $(PDFLATEX) geo_7.3_classification.tex
26325 ▷ $(OPEN) geo_7.3_classification.pdf
26326 ▷ $(ORBITER) -v 2 -draw_matrix \
26327 ▷ ▷ -input_csv_file geo_7.3-object_0.TDA_flag_orbits.csv \
26328 ▷ ▷ -secondary_input_csv_file geo_7.3-object_0.TDA.csv \
26329 ▷ ▷ -box_width 32 -bit_depth 24 \
26330 ▷ ▷ -end
26331 ▷ $(ORBITER) -v 2 -draw_matrix \
26332 ▷ ▷ -input_csv_file geo_7.3-object_0.INP_flag_orbits.csv \
26333 ▷ ▷ -secondary_input_csv_file geo_7.3-object_0.INP.csv \
26334 ▷ ▷ -box_width 32 -bit_depth 24 \
26335 ▷ ▷ -end
26336 ▷ $(OPEN) geo_7.3-object_0.INP_flag_orbits_draw.bmp
26337
26338
26339
26340 geo_10.3.c:
26341 ▷ $(ORBITER) -v 2 \
26342 ▷ ▷ -define Test_lines -set -loop 3 8 1 -end \
26343 ▷ ▷ -define Geo -geometry_builder \
26344 ▷ ▷ ▷ -V 10 -B 10 -TDO 3 \
26345 ▷ ▷ ▷ -fuse 1 \
26346 ▷ ▷ ▷ -fname_GEO 10.3 \
26347 ▷ ▷ ▷ -output_to_inc_file \
26348 ▷ ▷ ▷ -test Test_lines \
26349 ▷ ▷ -end

```



```

26350 ▷ $(ORBITER) -v 10 \
26351 ▷ ▷ -define Doinc -draw_incidence_structure_options \
26352 ▷ ▷ ▷ -width 60 -width_10 6 -end \
26353 ▷ ▷ -define C -combinatorial_object \
26354 ▷ ▷ ▷ -label geo_10_3 geo\_10\_3 \
26355 ▷ ▷ ▷ -file_of_incidence_geometries 10_3.inc 10 10 30 \
26356 ▷ ▷ -end \
26357 ▷ ▷ -with C -do \
26358 ▷ ▷ -combinatorial_object_activity \
26359 ▷ ▷ ▷ -canonical_form \
26360 ▷ ▷ ▷ ▷ -save_ago \
26361 ▷ ▷ ▷ ▷ -save_transversal \
26362 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "geo_10_3.nauty_log.txt" -end \
26363 ▷ ▷ ▷ -end \
26364 ▷ ▷ -end \
26365 ▷ ▷ -with C -do \
26366 ▷ ▷ -combinatorial_object_activity \
26367 ▷ ▷ ▷ -post_processing \
26368 ▷ ▷ -end \
26369 ▷ ▷ -with C -do \
26370 ▷ ▷ -combinatorial_object_activity \
26371 ▷ ▷ ▷ -report \
26372 ▷ ▷ ▷ ▷ -export_flag_orbits \
26373 ▷ ▷ ▷ ▷ -show_incidence_matrices \
26374 ▷ ▷ ▷ ▷ -export_group_GAP \
26375 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
26376 ▷ ▷ ▷ -end \
26377 ▷ ▷ -end
26378 ▷ $(PDFLATEX) geo_10_3_classification.tex
26379 ▷ $(OPEN) geo_10_3_classification.pdf
26380 ▷ $(ORBITER) -v 2 \
26381 ▷ ▷ -loop L 0 10 1 \
26382 ▷ ▷ ▷ -draw_matrix \
26383 ▷ ▷ ▷ -input_csv_file geo_10_3_object_%L_TDA_flag_orbits.csv \
26384 ▷ ▷ ▷ -secondary_input_csv_file geo_10_3_object_%L_TDA.csv \
26385 ▷ ▷ ▷ -box_width 16 -bit_depth 24 \
26386 ▷ ▷ ▷ -end \
26387 ▷ ▷ -end_loop L
26388 ▷ $(ORBITER) -v 2 \
26389 ▷ ▷ -loop L 0 10 1 \
26390 ▷ ▷ ▷ -draw_matrix \
26391 ▷ ▷ ▷ -input_csv_file geo_10_3_object_%L_INP_flag_orbits.csv \
26392 ▷ ▷ ▷ -secondary_input_csv_file geo_10_3_object_%L_INP.csv \
26393 ▷ ▷ ▷ -box_width 16 -bit_depth 24 \
26394 ▷ ▷ ▷ -end \
26395 ▷ ▷ -end_loop L
26396 ▷ #magick font.[0-3].gif -delete 1 +append seq_delete.gif
26397 ▷ magick geo_10_3_object_[0-5]_TDA_flag_orbits_draw.bmp -frame 5 +append a0.png
26398 ▷ magick geo_10_3_object_[5-9]_TDA_flag_orbits_draw.bmp geo_10_3_object_9_TDA fla
g_orbits_draw.bmp -frame 5 +append a1.png
26399 ▷ magick a0.png a1.png -frame 5 -append 10_3.png
26400
26401
26402
26403
26404
26405
26406 geo_10_3_c_lex_least:
26407 ▷ $(ORBITER) -v 10 \

```

```

26408 ▷ ▷ -define Doinc -draw_incidence_structure_options \
26409 ▷ ▷ ▷ -width 60 -width_10 6 -end \
26410 ▷ ▷ -define Test_lines -set -loop 4 11 1 -end \
26411 ▷ ▷ -define Geo -geometry_builder \
26412 ▷ ▷ ▷ -V 10 -B 10 -TDO 3 -fuse 1 \
26413 ▷ ▷ ▷ -fname.GEO 10_3 \
26414 ▷ ▷ ▷ -test Test_lines \
26415 ▷ ▷ ▷ -output_to_inc_file \
26416 ▷ ▷ -end \
26417 ▷ ▷ -define C -combinatorial_object \
26418 ▷ ▷ ▷ -label geo_10_3 geo\_10\_3 \
26419 ▷ ▷ ▷ -file_of_incidence_geometries 10_3.inc 10 10 30 \
26420 ▷ ▷ -end \
26421 ▷ ▷ -with C -do \
26422 ▷ ▷ -combinatorial_object_activity \
26423 ▷ ▷ ▷ -canonical_form \
26424 ▷ ▷ ▷ ▷ -save_ago \
26425 ▷ ▷ ▷ ▷ -save_transversal \
26426 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "geo_10_3.nauty_log.txt" -end \
26427 ▷ ▷ ▷ -end \
26428 ▷ ▷ -end \
26429 ▷ ▷ -with C -do \
26430 ▷ ▷ -combinatorial_object_activity \
26431 ▷ ▷ ▷ -post_processing \
26432 ▷ ▷ -end \
26433 ▷ ▷ -with C -do \
26434 ▷ ▷ -combinatorial_object_activity \
26435 ▷ ▷ ▷ -report \
26436 ▷ ▷ ▷ ▷ -export_flag_orbits \
26437 ▷ ▷ ▷ ▷ -show_incidence_matrices \
26438 ▷ ▷ ▷ ▷ -export_group_GAP \
26439 ▷ ▷ ▷ ▷ -show_TDO \
26440 ▷ ▷ ▷ ▷ -show_TDA \
26441 ▷ ▷ ▷ ▷ -lex_least Geo \
26442 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
26443 ▷ ▷ ▷ -end \
26444 ▷ ▷ -end
26445 ▷ $(PDFLATEX) geo_10_3_classification.tex
26446 ▷ $(OPEN) geo_10_3_classification.pdf
26447 ▷ $(ORBITER) -v 2 -draw_matrix \
26448 ▷ ▷ -input_csv_file geo_10_3_object7_TDA_flag_orbits.csv \
26449 ▷ ▷ -secondary_input_csv_file geo_10_3_object7_TDA.csv \
26450 ▷ ▷ -box_width 16 -bit_depth 24 \
26451 ▷ ▷ -end
26452 ▷ $(ORBITER) -v 2 -draw_matrix \
26453 ▷ ▷ -input_csv_file geo_10_3_object7_INP_flag_orbits.csv \
26454 ▷ ▷ -secondary_input_csv_file geo_10_3_object7_INP.csv \
26455 ▷ ▷ -box_width 16 -bit_depth 24 \
26456 ▷ ▷ -end
26457
26458 # error!
26459
26460 #10_3_object7_TDA_flag_orbits.csv
26461
26462 #0 1 2 10 13 14 20 25 26 31 33 35 41 44 46 52 53 56 62 67 68 74 77 79 85 88 89
26463
26464
26465
26466

```

```

26467
26468
26469 geo_14_3.c:
26470 ▷ $(ORBITER) -v 2 \
26471 ▷ ▷ -define Doinc -draw_incidence_structure_options \
26472 ▷ ▷ ▷ -width 60 -width_10 6 -end \
26473 ▷ ▷ -define Test_lines -set -loop 4 15 1 -end \
26474 ▷ ▷ -define C -combinatorial_object \
26475 ▷ ▷ ▷ -label geo_14_3 geo\_14\_3 \
26476 ▷ ▷ ▷ -file_of_incidence_geometries 14_3.inc 14 14 42 \
26477 ▷ ▷ -end \
26478 ▷ ▷ -with C -do \
26479 ▷ ▷ -combinatorial_object_activity \
26480 ▷ ▷ ▷ -canonical_form \
26481 ▷ ▷ ▷ ▷ -save_ago \
26482 ▷ ▷ ▷ ▷ -save_transversal \
26483 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "geo_14_3.nauty_log.txt" -end \
26484 ▷ ▷ ▷ -end \
26485 ▷ ▷ -end
26486
26487 #-rw-r--r-- 1 betten staff 160116 Dec 18 17:44 geo_14_3.ago.csv
26488 #-rw-r--r-- 1 betten staff 234589 Dec 18 17:44 geo_14_3.transversal.csv
26489
26490
26491 geo_15_3.c_filter72:
26492 ▷ $(ORBITER) -v 2 \
26493 ▷ ▷ -define Doinc -draw_incidence_structure_options \
26494 ▷ ▷ ▷ -width 50 -width_10 5 -end \
26495 ▷ ▷ -define C -combinatorial_object \
26496 ▷ ▷ ▷ -label geo_15_3 geo\_15\_3 \
26497 ▷ ▷ ▷ -file_of_incidence_geometries 15_3.inc 15 15 45 \
26498 ▷ ▷ -end \
26499 ▷ ▷ -with C -do \
26500 ▷ ▷ -combinatorial_object_activity \
26501 ▷ ▷ ▷ -canonical_form \
26502 ▷ ▷ ▷ -save_ago \
26503 ▷ ▷ ▷ -nauty_control -end \
26504 ▷ ▷ ▷ -filter_ago_at_least 72 \
26505 ▷ ▷ ▷ -end \
26506 ▷ ▷ -end \
26507
26508
26509 geo_15_3.filtered.c:
26510 ▷ $(ORBITER) -v 2 \
26511 ▷ ▷ -define Doinc -draw_incidence_structure_options \
26512 ▷ ▷ ▷ -width 50 -width_10 5 -end \
26513 ▷ ▷ -define C -combinatorial_object \
26514 ▷ ▷ ▷ -label geo_15_3.ago_ge72 geo\_15\_3\_ago\_ge72 \
26515 ▷ ▷ ▷ -file_of_incidence_geometries 15_3.filtered.inc 15 15 45 \
26516 ▷ ▷ -end \
26517 ▷ ▷ -with C -do \
26518 ▷ ▷ -combinatorial_object_activity \
26519 ▷ ▷ ▷ -canonical_form \
26520 ▷ ▷ ▷ -save_ago \
26521 ▷ ▷ ▷ -nauty_control -end \
26522 ▷ ▷ ▷ -end \
26523 ▷ ▷ -end \
26524 ▷ ▷ -with C -do \
26525 ▷ ▷ -combinatorial_object_activity \

```

```

26526 ▷ ▷ ▷ -post_processing \
26527 ▷ ▷ -end \
26528 ▷ ▷ -with C -do \
26529 ▷ ▷ -combinatorial_object_activity \
26530 ▷ ▷ ▷ -report \
26531 ▷ ▷ ▷ ▷ -export_flag_orbits \
26532 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
26533 ▷ ▷ ▷ -end \
26534 ▷ ▷ -end
26535 ▷ $(PDFLATEX) geo.15.3_ago_ge72_classification.tex
26536 ▷ $(OPEN) geo.15.3_ago_ge72_classification.pdf
26537
26538
26539
26540 TFC_24.3_c:
26541 ▷ echo $(FILE_24.3_TFC_INC) >24.3_TFC.inc
26542 ▷ $(ORBITER) -v 20 \
26543 ▷ ▷ -define C -combinatorial_object \
26544 ▷ ▷ ▷ -label 24.3_TFC 24\3\_TFC \
26545 ▷ ▷ ▷ -file_of_incidence_geometries 24.3_TFC.inc 24 24 72 \
26546 ▷ ▷ -end \
26547 ▷ ▷ -with C -do \
26548 ▷ ▷ -combinatorial_object_activity \
26549 ▷ ▷ ▷ -canonical_form \
26550 ▷ ▷ ▷ -save_ago \
26551 ▷ ▷ ▷ -nauty_control -nauty_log "TFC_24.3_nauty_log.txt" -end \
26552 ▷ ▷ ▷ -end \
26553 ▷ ▷ -end \
26554 ▷ ▷ -define Doinc -draw_incidence_structure_options \
26555 ▷ ▷ ▷ -width 40 -width_10 6 \
26556 ▷ ▷ -end \
26557 ▷ ▷ -with C -do \
26558 ▷ ▷ -combinatorial_object_activity \
26559 ▷ ▷ ▷ -post_processing \
26560 ▷ ▷ -end \
26561 ▷ ▷ -with C -do \
26562 ▷ ▷ -combinatorial_object_activity \
26563 ▷ ▷ ▷ -report \
26564 ▷ ▷ ▷ ▷ -export_flag_orbits \
26565 ▷ ▷ ▷ ▷ -show_TDO \
26566 ▷ ▷ ▷ ▷ -show_TDA \
26567 ▷ ▷ ▷ ▷ -show_incidence_matrices \
26568 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
26569 ▷ ▷ ▷ -end \
26570 ▷ ▷ -end
26571 ▷ $(PDFLATEX) 24.3_TFC_classification.tex
26572 ▷ $(OPEN) 24.3_TFC_classification.pdf
26573 ▷ $(ORBITER) -v 2 -draw_matrix \
26574 ▷ ▷ -input_csv_file 24.3_TFC_object_2_TDA_flag_orbits.csv \
26575 ▷ ▷ -secondary_input_csv_file 24.3_TFC_object_2_TDA.csv \
26576 ▷ ▷ -box_width 40 -bit_depth 24 \
26577 ▷ ▷ -end
26578 ▷ $(OPEN) 24.3_TFC_object_2_TDA_flag_orbits.draw.bmp
26579 ▷ $(ORBITER) -v 2 \
26580 ▷ ▷ -define Do -draw_options -embedded -radius 150 \
26581 ▷ ▷ ▷ -nodes_empty \
26582 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
26583 ▷ ▷ -tree_draw \
26584 ▷ ▷ ▷ -file TFC_24.3_nauty_log.txt \

```

```

26585 ▷ ▷ ▷ -draw.options Do \
26586 ▷ ▷ -end
26587 ▷ $(PDFLATEX) TFC_24_3_nauty_log_draw.tex
26588 ▷ $(OPEN) TFC_24_3_nauty_log_draw.pdf
26589
26590
26591 geo.40_4_g4.c:
26592 ▷ $(ORBITER) -v 5 \
26593 ▷ ▷ -define Test_lines -set -loop 0 41 1 -end \
26594 ▷ ▷ -define Geo -geometry_builder \
26595 ▷ ▷ ▷ -V 40 -B 40 -TDO 4 \
26596 ▷ ▷ ▷ -fuse 1 \
26597 ▷ ▷ ▷ -fname_GEO 40_4_g4 \
26598 ▷ ▷ ▷ -girth 4 \
26599 ▷ ▷ ▷ -search_tree \
26600 ▷ ▷ ▷ -special_test_not_orderly \
26601 ▷ ▷ ▷ -test Test_lines \
26602 ▷ ▷ ▷ -output_to_sage_file \
26603 ▷ ▷ ▷ -output_to_inc_file \
26604 ▷ ▷ -end
26605 ▷ $(ORBITER) -v 2 \
26606 ▷ ▷ -define Doinc -draw_incidence_structure_options \
26607 ▷ ▷ ▷ -width 50 -width_10 5 -end \
26608 ▷ ▷ -define C -combinatorial_object \
26609 ▷ ▷ ▷ -label 40_4_g4 40\4\g4 \
26610 ▷ ▷ ▷ -file_of_incidence_geometries \
26611 ▷ ▷ ▷ ▷ 40_4_g4.inc 40 40 160 \
26612 ▷ ▷ -end \
26613 ▷ ▷ -with C -do \
26614 ▷ ▷ -combinatorial_object_activity \
26615 ▷ ▷ ▷ -canonical_form \
26616 ▷ ▷ ▷ ▷ -save_ago \
26617 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "geo.40_4_g4.nauty_log.txt" -end \
26618 ▷ ▷ ▷ -end \
26619 ▷ ▷ -end \
26620 ▷ ▷ -with C -do \
26621 ▷ ▷ -combinatorial_object_activity \
26622 ▷ ▷ ▷ -post_processing \
26623 ▷ ▷ -end \
26624 ▷ ▷ -with C -do \
26625 ▷ ▷ -combinatorial_object_activity \
26626 ▷ ▷ ▷ -report \
26627 ▷ ▷ ▷ ▷ -export_flag_orbits \
26628 ▷ ▷ ▷ ▷ -show_TDO \
26629 ▷ ▷ ▷ ▷ -show_TDA \
26630 ▷ ▷ ▷ ▷ -show_incidence_matrices \
26631 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
26632 ▷ ▷ ▷ -end \
26633 ▷ ▷ -end
26634 ▷ $(ORBITER) -v 2 \
26635 ▷ ▷ -loop L 0 2 1 \
26636 ▷ ▷ ▷ -draw_matrix \
26637 ▷ ▷ ▷ -input_csv_file 40_4_g4.object.%L.TDA_flag_orbits.csv \
26638 ▷ ▷ ▷ -secondary_input_csv_file 40_4_g4.object.%L.TDA.csv \
26639 ▷ ▷ ▷ -box_width 32 -bit_depth 24 \
26640 ▷ ▷ ▷ -end \
26641 ▷ ▷ ▷ -system "magick \
26642 ▷ ▷ ▷ ▷ 40_4_g4.object.%L.TDA_flag_orbits.draw.bmp \
26643 ▷ ▷ ▷ ▷ 40_4_g4.object.%L.TDA_flag_orbits.draw.png" \

```

```

26644 ▷ ▷ -end_loop L
26645 ▷ $(PDFLATEX) 40_4_g4.classification.tex
26646 ▷ $(OPEN) 40_4_g4.classification.pdf
26647 ▷ $(ORBITER) -v 2 \
26648 ▷ ▷ -define Do -draw_options -embedded -radius 150 \
26649 ▷ ▷ ▷ -nodes_empty \
26650 ▷ ▷ ▷ -scale 0.5 -line_width 0.3 -end \
26651 ▷ ▷ -tree_draw \
26652 ▷ ▷ ▷ -file geo.40_4_g4.nauty.log.txt \
26653 ▷ ▷ ▷ -draw_options Do \
26654 ▷ ▷ -end
26655 ▷ $(PDFLATEX) geo.40_4_g4.nauty.log.draw.tex
26656 ▷ $(OPEN) geo.40_4_g4.nauty.log.draw.pdf
26657
26658
26659 #geo_40_4_g4.nauty.log.txt
26660
26661
26662 geo.17_3_g4.c:
26663 ▷ $(ORBITER) -v 2 \
26664 ▷ ▷ -define Doinc -draw_incidence_structure_options \
26665 ▷ ▷ ▷ -width 50 -width_10 5 -end \
26666 ▷ ▷ -define C -combinatorial_object \
26667 ▷ ▷ ▷ -label 17_3_g4 17\3\g4 \
26668 ▷ ▷ ▷ -file_of_incidence_geometries 17_3_g4.inc 17 17 51 \
26669 ▷ ▷ -end \
26670 ▷ ▷ -with C -do \
26671 ▷ ▷ -combinatorial_object_activity \
26672 ▷ ▷ ▷ -canonical_form \
26673 ▷ ▷ ▷ ▷ -save_ago \
26674 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "geo.17_3_g4.nauty.log.txt" -end \
26675 ▷ ▷ ▷ -end \
26676 ▷ ▷ -end \
26677 ▷ ▷ -define Doinc -draw_incidence_structure_options \
26678 ▷ ▷ ▷ -width 40 -width_10 6 \
26679 ▷ ▷ -end \
26680 ▷ ▷ -with C -do \
26681 ▷ ▷ -combinatorial_object_activity \
26682 ▷ ▷ ▷ -post_processing \
26683 ▷ ▷ -end \
26684 ▷ ▷ -with C -do \
26685 ▷ ▷ -combinatorial_object_activity \
26686 ▷ ▷ ▷ -report \
26687 ▷ ▷ ▷ ▷ -export_flag_orbits \
26688 ▷ ▷ ▷ -show_TDO \
26689 ▷ ▷ ▷ ▷ -show_TDA \
26690 ▷ ▷ ▷ ▷ -show_incidence_matrices \
26691 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
26692 ▷ ▷ ▷ -end \
26693 ▷ ▷ -end
26694 ▷ $(PDFLATEX) 17_3_g4.classification.tex
26695 ▷ $(OPEN) 17_3_g4.classification.pdf
26696
26697
26698 AG.2_3.c: AG.2_3.inc
26699 ▷ $(ORBITER) -v 2 \
26700 ▷ ▷ -define C -combinatorial_object \
26701 ▷ ▷ ▷ -label AG.2_3 AG\2\3 \
26702 ▷ ▷ ▷ -file_of_incidence_geometries \

```

```

26703 ▷ ▷ ▷ AG_2.3.inc 9 12 36 \
26704 ▷ ▷ -end \
26705 ▷ ▷ -with C -do \
26706 ▷ ▷ -combinatorial_object_activity \
26707 ▷ ▷ ▷ -canonical_form \
26708 ▷ ▷ ▷ ▷ -save_ago \
26709 ▷ ▷ ▷ ▷ -max_TDO_depth 10 \
26710 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "AG_2.3_nauty_log.txt" -end \
26711 ▷ ▷ ▷ -end \
26712 ▷ ▷ -end \
26713 ▷ ▷ -define Doinc -draw_incidence_structure_options \
26714 ▷ ▷ ▷ -width 40 -width_10 6 \
26715 ▷ ▷ -end \
26716 ▷ ▷ -with C -do \
26717 ▷ ▷ -combinatorial_object_activity \
26718 ▷ ▷ ▷ -post_processing \
26719 ▷ ▷ -end \
26720 ▷ ▷ -with C -do \
26721 ▷ ▷ -combinatorial_object_activity \
26722 ▷ ▷ ▷ -report \
26723 ▷ ▷ ▷ ▷ -export_flag_orbits \
26724 ▷ ▷ ▷ ▷ -show_TDO \
26725 ▷ ▷ ▷ ▷ -show_TDA \
26726 ▷ ▷ ▷ ▷ -show_incidence_matrices \
26727 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
26728 ▷ ▷ ▷ -end \
26729 ▷ ▷ -end
26730 ▷ $(PDFLATEX) AG_2.3.classification.tex
26731 ▷ $(OPEN) AG_2.3.classification.pdf
26732 ▷ $(ORBITER) -v 2 -draw_matrix \
26733 ▷ ▷ -input_csv_file AG_2.3.object_0_INP_flag_orbits.csv \
26734 ▷ ▷ -secondary_input_csv_file AG_2.3.object_0_INP.csv \
26735 ▷ ▷ -box_width 40 -bit_depth 24 \
26736 ▷ ▷ -end
26737 ▷ $(OPEN) AG_2.3.object_0_INP_flag_orbits_draw.bmp
26738
26739
26740
26741
26742 geo_LSQ6.c:
26743 ▷ $(ORBITER) -v 2 \
26744 ▷ ▷ -define Test_lines -set -loop 1 19 1 -end \
26745 ▷ ▷ -define Geo -geometry_builder \
26746 ▷ ▷ ▷ -V 6,6,6 -B 1,1,1,36 -TDO \
26747 ▷ ▷ ▷ "1,0,0,6, 0,1,0,6, 0,0,1,6" \
26748 ▷ ▷ ▷ -fuse 3 \
26749 ▷ ▷ ▷ -fname_GEO LSQ6 \
26750 ▷ ▷ ▷ -output_to_inc_file \
26751 ▷ ▷ ▷ -test Test_lines \
26752 ▷ ▷ -end
26753 ▷ $(ORBITER) -v 3 \
26754 ▷ ▷ -define Doinc -draw_incidence_structure_options \
26755 ▷ ▷ ▷ -width 60 -width_10 6 -end \
26756 ▷ ▷ -define C -combinatorial_object \
26757 ▷ ▷ ▷ -label LSQ6 LSQ6 \
26758 ▷ ▷ ▷ -file_of_incidence_geometries \
26759 ▷ ▷ ▷ LSQ6.inc 18 39 126 \
26760 ▷ ▷ -end \
26761 ▷ ▷ -with C -do \

```

```

26762 ▷ ▷ -combinatorial_object_activity \
26763 ▷ ▷ ▷ -canonical_form \
26764 ▷ ▷ ▷ ▷ -save_ago \
26765 ▷ ▷ ▷ ▷ -save_transversal \
26766 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "LSQ6_nauty_log.txt" -end \
26767 ▷ ▷ ▷ -end \
26768 ▷ ▷ -end \
26769 ▷ ▷ -with C -do \
26770 ▷ ▷ -combinatorial_object_activity \
26771 ▷ ▷ ▷ -post_processing \
26772 ▷ ▷ ▷ -end \
26773 ▷ ▷ -with C -do \
26774 ▷ ▷ -combinatorial_object_activity \
26775 ▷ ▷ ▷ -report \
26776 ▷ ▷ ▷ ▷ -export_flag_orbits \
26777 ▷ ▷ ▷ ▷ -show_incidence_matrices \
26778 ▷ ▷ ▷ ▷ -export_group_GAP \
26779 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
26780 ▷ ▷ ▷ -end \
26781 ▷ ▷ -end
26782 ▷ $(PDFLATEX) LSQ6_classification.tex
26783 ▷ $(OPEN) LSQ6_classification.pdf
26784 ▷ $(ORBITER) -v 2 \
26785 ▷ ▷ -loop L 0 12 1 \
26786 ▷ ▷ ▷ -draw_matrix \
26787 ▷ ▷ ▷ -input_csv_file LSQ6_object_%L_TDA_flag_orbits.csv \
26788 ▷ ▷ ▷ -secondary_input_csv_file LSQ6_object_%L_TDA.csv \
26789 ▷ ▷ ▷ -box_width 32 -bit_depth 24 \
26790 ▷ ▷ ▷ -end \
26791 ▷ ▷ ▷ -system "magick \
26792 ▷ ▷ ▷ ▷ LSQ6_object_%L_TDA_flag_orbits.draw.bmp \
26793 ▷ ▷ ▷ ▷ LSQ6_object_%L_TDA_flag_orbits.draw.png" \
26794 ▷ ▷ -end_loop L
26795 ▷ $(ORBITER) -v 2 \
26796 ▷ ▷ -loop L 0 12 1 \
26797 ▷ ▷ ▷ -draw_matrix \
26798 ▷ ▷ ▷ -input_csv_file LSQ6_object_%L_INP_flag_orbits.csv \
26799 ▷ ▷ ▷ -secondary_input_csv_file LSQ6_object_%L_INP.csv \
26800 ▷ ▷ ▷ -box_width 32 -bit_depth 24 \
26801 ▷ ▷ ▷ -end \
26802 ▷ ▷ -end_loop L
26803 ▷ magick LSQ6_object_[0-5]_TDA_flag_orbits.draw.bmp -frame 10 -append a0.png
26804 ▷ magick LSQ6_object_[6-9]_TDA_flag_orbits.draw.bmp \
26805 ▷ ▷ LSQ6_object_10_TDA_flag_orbits.draw.bmp \
26806 ▷ ▷ LSQ6_object_11_TDA_flag_orbits.draw.bmp \
26807 ▷ ▷ -frame 10 -append a1.png
26808 ▷ magick a0.png a1.png -frame 10 +append LSQ6.png
26809 ▷ cp LSQ6.png ~/TEX.26/ORBITER_USERS_GUIDE/GRAPHICS/LSQ_6
26810
26811
26812
26813
26814
26815
26816
26817
26818
26819
26820

```



```

26821 #####
26822 # Section 17.4: Objects from Design Theory
26823
26824
26825 SECTION.OBJECTS_FROM_DESIGN.THEORY:
26826
26827
26828 test_17.4:
26829 ▷ make design_27c
26830 ▷ make design_PG_2_3_canonical
26831 ▷ make wreath_product_designs_n4_k2_c
26832 ▷ #make wreath_product_designs_n8_k6_c
26833 ▷ make LS_AG_2_3_solutions_classify # needs Section 14.3
26834 ▷ #make neuimaier_graphs_16_classify # needs the file NG_16_9_4_2_4.csv
26835
26836
26837
26838
26839
26840
26841 design_27c:
26842 ▷ $(ORBITER) -v 4 \
26843 ▷ ▷ -define Doinc -draw_incidence_structure_options \
26844 ▷ ▷ ▷ -width 20 -width_10 2 -end \
26845 ▷ ▷ -define C -combinatorial_object \
26846 ▷ ▷ ▷ -label design design \
26847 ▷ ▷ ▷ -set_of_points "2,56,30,112,253,90,440,508" \
26848 ▷ ▷ -end \
26849 ▷ ▷ -define F -finite_field -q 27 \
26850 ▷ ▷ ▷ -override_polynomial 46 \
26851 ▷ ▷ -end \
26852 ▷ ▷ -define P -projective_space -n 2 -field F -end \
26853 ▷ ▷ -with C -do \
26854 ▷ ▷ -combinatorial_object_activity \
26855 ▷ ▷ ▷ -canonical_form_PG P \
26856 ▷ ▷ ▷ -nauty_control -end \
26857 ▷ ▷ ▷ -end \
26858 ▷ ▷ -end \
26859 ▷ ▷ -with C -do \
26860 ▷ ▷ -combinatorial_object_activity \
26861 ▷ ▷ ▷ -post_processing \
26862 ▷ ▷ -end \
26863 ▷ ▷ -with C -do \
26864 ▷ ▷ -combinatorial_object_activity \
26865 ▷ ▷ ▷ -report \
26866 ▷ ▷ ▷ -incidence_draw_options Doinc \
26867 ▷ ▷ -end
26868 ▷ $(PDFLATEX) design_classification.tex
26869 ▷ $(OPEN) design_classification.pdf
26870
26871
26872 # ago = 1 ?? ToDo
26873
26874
26875 design_PG_2_3_canonical:
26876 ▷ $(ORBITER) -v 3 \
26877 ▷ ▷ -define F -finite_field -q 3 -end \
26878 ▷ ▷ -define D -design -field F -family PG_2_q -end \
26879 ▷ ▷ -with D -do \

```

```

26880 ▷ ▷ ▷ -design_activity \
26881 ▷ ▷ ▷ -export_incidence_matrix_as_flags \
26882 ▷ ▷ -end
26883 ▷ $(ORBITER) -v 3 \
26884 ▷ ▷ -define Doinc -draw_incidence_structure_options \
26885 ▷ ▷ ▷ -width 60 -width_10 6 -end \
26886 ▷ ▷ -define C -combinatorial_object \
26887 ▷ ▷ ▷ -label PG_2_3 PG_2_3 \
26888 ▷ ▷ ▷ -file_of_incidence_geometries_csv \
26889 ▷ ▷ ▷ PG_2_3_flags.csv flags 13 13 52 \
26890 ▷ ▷ -end \
26891 ▷ ▷ -with C -do \
26892 ▷ ▷ -combinatorial_object_activity \
26893 ▷ ▷ ▷ -canonical_form \
26894 ▷ ▷ ▷ -save_ago \
26895 ▷ ▷ ▷ -save_transversal \
26896 ▷ ▷ ▷ -nauty_control -end \
26897 ▷ ▷ ▷ -end \
26898 ▷ ▷ -end \
26899 ▷ ▷ -with C -do \
26900 ▷ ▷ -combinatorial_object_activity \
26901 ▷ ▷ ▷ -post_processing \
26902 ▷ ▷ -end \
26903 ▷ ▷ -with C -do \
26904 ▷ ▷ -combinatorial_object_activity \
26905 ▷ ▷ ▷ -report \
26906 ▷ ▷ ▷ -export_flag_orbits \
26907 ▷ ▷ ▷ -show_incidence_matrices \
26908 ▷ ▷ ▷ -export_group_GAP \
26909 ▷ ▷ ▷ -incidence_draw_options Doinc \
26910 ▷ ▷ ▷ -end \
26911 ▷ ▷ -end
26912 ▷ $(PDFLATEX) PG_2_3_classification.tex
26913 ▷ $(OPEN) PG_2_3_classification.pdf
26914 ▷ $(ORBITER) -v 2 -draw_matrix \
26915 ▷ ▷ -input_csv_file PG_2_3_object_0_TDA_flag_orbits.csv \
26916 ▷ ▷ -secondary_input_csv_file PG_2_3_object_0_TDA.csv \
26917 ▷ ▷ -box_width 32 -bit_depth 24 \
26918 ▷ ▷ -end
26919 ▷ $(OPEN) PG_2_3_object_0_TDA_flag_orbits_draw.bmp
26920 ▷
26921
26922
26923 wreath_product_designs_n4_k2_c:
26924 ▷ $(ORBITER) -v 8 \
26925 ▷ ▷ -define D -design -wreath_product_designs 4 2 -end \
26926 ▷ ▷ -with D -do \
26927 ▷ ▷ ▷ -design_activity \
26928 ▷ ▷ ▷ -export_incidence_matrix_as_flags \
26929 ▷ ▷ -end
26930 ▷ $(ORBITER) -v 10 \
26931 ▷ ▷ -define Doinc -draw_incidence_structure_options \
26932 ▷ ▷ ▷ -width 60 -width_10 6 -end \
26933 ▷ ▷ -define C -combinatorial_object \
26934 ▷ ▷ ▷ -label wreath_4_2 wreath_4_2 \
26935 ▷ ▷ ▷ -file_of_incidence_geometries \
26936 ▷ ▷ ▷ wreath_product_designs_n4_k2_flags.txt \
26937 ▷ ▷ ▷ 8 12 24 \
26938 ▷ ▷ -end \

```

```

26939 ▷ ▷ -with C -do \
26940 ▷ ▷ -combinatorial_object_activity \
26941 ▷ ▷ ▷ -canonical_form \
26942 ▷ ▷ ▷ ▷ -save_ago \
26943 ▷ ▷ ▷ ▷ -save_transversal \
26944 ▷ ▷ ▷ ▷ -nauty_control -end \
26945 ▷ ▷ ▷ -end \
26946 ▷ ▷ -end \
26947 ▷ ▷ -with C -do \
26948 ▷ ▷ -combinatorial_object_activity \
26949 ▷ ▷ ▷ -post_processing \
26950 ▷ ▷ -end \
26951 ▷ ▷ -with C -do \
26952 ▷ ▷ -combinatorial_object_activity \
26953 ▷ ▷ ▷ -report \
26954 ▷ ▷ ▷ ▷ -export_flag_orbits \
26955 ▷ ▷ ▷ ▷ -show_incidence_matrices \
26956 ▷ ▷ ▷ ▷ -export_group_GAP \
26957 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
26958 ▷ ▷ ▷ -end \
26959 ▷ ▷ -end
26960 ▷ $(PDFLATEX) wreath_4.2.classification.tex
26961 ▷ $(OPEN) wreath_4.2.classification.pdf
26962
26963
26964 wreath_product_designs_n8_k6.c:
26965 ▷ $(ORBITER) -v 8 \
26966 ▷ ▷ -define D -design -wreath_product_designs 8 6 -end \
26967 ▷ ▷ -with D -do \
26968 ▷ ▷ ▷ -design_activity \
26969 ▷ ▷ ▷ ▷ -export_incidence_matrix_as_flags \
26970 ▷ ▷ -end
26971 ▷ $(ORBITER) -v 10 \
26972 ▷ ▷ -define Doinc -draw_incidence_structure_options \
26973 ▷ ▷ ▷ -width 60 -width_10 6 -end \
26974 ▷ ▷ -define C -combinatorial_object \
26975 ▷ ▷ ▷ -label wreath_8_6 wreath\8\6 \
26976 ▷ ▷ ▷ -file_of_incidence_geometries \
26977 ▷ ▷ ▷ wreath_product_designs_n8_k6_flags.txt \
26978 ▷ ▷ ▷ 16 3920 23520 \
26979 ▷ ▷ -end \
26980 ▷ ▷ -with C -do \
26981 ▷ ▷ -combinatorial_object_activity \
26982 ▷ ▷ ▷ -canonical_form \
26983 ▷ ▷ ▷ ▷ -save_ago \
26984 ▷ ▷ ▷ ▷ -save_transversal \
26985 ▷ ▷ ▷ ▷ -nauty_control -end \
26986 ▷ ▷ ▷ -end \
26987 ▷ ▷ -end \
26988 ▷ ▷ -with C -do \
26989 ▷ ▷ -combinatorial_object_activity \
26990 ▷ ▷ ▷ -post_processing \
26991 ▷ ▷ -end \
26992 ▷ ▷ -with C -do \
26993 ▷ ▷ -combinatorial_object_activity \
26994 ▷ ▷ ▷ -report \
26995 ▷ ▷ ▷ ▷ -export_flag_orbits \
26996 ▷ ▷ ▷ ▷ -export_group_GAP \
26997 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \

```

```

26998 ▷ ▷ ▷ -end \
26999 ▷ ▷ -end
27000 ▷ $(PDFLATEX) wreath_8_6_classification.tex
27001 ▷ $(OPEN) wreath_8_6_classification.pdf
27002
27003 #group_action_on_combinatorial_object::export_INP_with_flag_orbits flag_orbits ha
ve not been computed
27004
27005
27006 # continuation of an example in Section 14.3
27007 # requires the file AG_2_3_large_sets.csv
27008
27009 LS_AG_2_3_solutions_classify:
27010 ▷ $(ORBITER) -v 2 \
27011 ▷ ▷ -define Doinc -draw_incidence_structure_options \
27012 ▷ ▷ ▷ -width 20 -width_10 2 -end \
27013 ▷ ▷ -define C -combinatorial_object \
27014 ▷ ▷ ▷ -label LS_AG_2_3 LS_AG_2_3 \
27015 ▷ ▷ ▷ -file_of_designs \
27016 ▷ ▷ ▷ AG_2_3_large_sets.csv large_set 9 84 3 12 \
27017 ▷ ▷ -end \
27018 ▷ ▷ -with C -do \
27019 ▷ ▷ -combinatorial_object_activity \
27020 ▷ ▷ ▷ -canonical_form \
27021 ▷ ▷ ▷ ▷ -save_ago \
27022 ▷ ▷ ▷ ▷ -save_transversal \
27023 ▷ ▷ ▷ ▷ -max_TDO_depth 10 \
27024 ▷ ▷ ▷ ▷ -nauty_control -end \
27025 ▷ ▷ ▷ -end \
27026 ▷ ▷ -end \
27027 ▷ ▷ -with C -do \
27028 ▷ ▷ -combinatorial_object_activity \
27029 ▷ ▷ ▷ -post_processing \
27030 ▷ ▷ -end \
27031 ▷ ▷ -with C -do \
27032 ▷ ▷ -combinatorial_object_activity \
27033 ▷ ▷ ▷ -report \
27034 ▷ ▷ ▷ ▷ -export_flag_orbits \
27035 ▷ ▷ ▷ ▷ -show_TDO \
27036 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
27037 ▷ ▷ ▷ -end \
27038 ▷ ▷ -end
27039 ▷ $(PDFLATEX) LS_AG_2_3_classification.tex
27040 ▷ $(OPEN) LS_AG_2_3_classification.pdf
27041 ▷ $(ORBITER) -v 2 -draw_matrix \
27042 ▷ ▷ -input_csv_file LS_AG_2_3_object_0_INP_flag_orbits.csv \
27043 ▷ ▷ -secondary_input_csv_file LS_AG_2_3_object_0_INP.csv \
27044 ▷ ▷ -box_width 12 -bit_depth 24 \
27045 ▷ ▷ -end
27046 ▷ $(OPEN) LS_AG_2_3_object_0_INP_flag_orbits_draw.bmp
27047 ▷ $(ORBITER) -v 2 -draw_matrix \
27048 ▷ ▷ -input_csv_file LS_AG_2_3_object_1_INP_flag_orbits.csv \
27049 ▷ ▷ -secondary_input_csv_file LS_AG_2_3_object_1_INP.csv \
27050 ▷ ▷ -box_width 12 -bit_depth 24 \
27051 ▷ ▷ -end
27052 ▷ $(OPEN) LS_AG_2_3_object_1_INP_flag_orbits_draw.bmp
27053
27054
27055

```

```

27056
27057
27058
27059 # needs the file NG_16.9.4.2.4.csv from the search for Neumaier graphs
27060
27061
27062 neumaier_graphs_16.classify:
27063 ▷ $(ORBITER) -v 2 \
27064 ▷ ▷ -define Doinc -draw_incidence_structure_options \
27065 ▷ ▷ ▷ -width 20 -width_10 2 -end \
27066 ▷ ▷ -define C -combinatorial_object \
27067 ▷ ▷ ▷ -label neumaier_graph neumaier\_graph \
27068 ▷ ▷ ▷ -graph_object Gamma \
27069 ▷ ▷ -end \
27070 ▷ ▷ -with C -do \
27071 ▷ ▷ -combinatorial_object_activity \
27072 ▷ ▷ ▷ -canonical_form \
27073 ▷ ▷ ▷ ▷ -save_ago \
27074 ▷ ▷ ▷ ▷ -save_transversal \
27075 ▷ ▷ ▷ ▷ -max_TDO_depth 10 \
27076 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "neumaier_graphs_16_nauty_log.txt" -end \
27077 ▷ ▷ ▷ -end \
27078 ▷ ▷ -end \
27079 ▷ ▷ -with C -do \
27080 ▷ ▷ -combinatorial_object_activity \
27081 ▷ ▷ ▷ -post_processing \
27082 ▷ ▷ -end \
27083 ▷ ▷ -with C -do \
27084 ▷ ▷ -combinatorial_object_activity \
27085 ▷ ▷ ▷ -report \
27086 ▷ ▷ ▷ ▷ -export_flag_orbits \
27087 ▷ ▷ ▷ ▷ -show_TDO \
27088 ▷ ▷ ▷ ▷ -show_TDA \
27089 ▷ ▷ ▷ ▷ -export_labels \
27090 ▷ ▷ ▷ ▷ -show_incidence_matrices \
27091 ▷ ▷ ▷ ▷ -export_group_GAP \
27092 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
27093 ▷ ▷ ▷ -end \
27094 ▷ ▷ -end
27095 ▷ $(ORBITER) -v 4 -csv_file_select_rows_by_file \
27096 ▷ ▷ NG_16.9.4.2.4.csv neumaier_graph_transversal.csv
27097 ▷ $(PDFLATEX) neumaier_graph_classification.tex
27098 ▷ $(OPEN) neumaier_graph_classification.pdf
27099
27100
27101 #NG_16.9.4.2.4_transversal.csv
27102
27103
27104
27105 #####
27106 # Section 17.5: Linear Codes
27107
27108
27109 SECTION_CANONICAL_FORMS_OF_LINEAR_CODES:
27110
27111
27112 test_17.5:
27113 ▷ make code_3.2_aut
27114 ▷ make code_6.3_aut

```

```

27115 ▷ make RM_3_1_group
27116 ▷ make RM_3_1_group_and_diagram
27117 ▷ make RM_4_1_group
27118 ▷ make RS_6_4_7_group
27119 ▷ make GV_n15_k6_d5_group
27120 ▷ make code_n15_k6_d6_a_group
27121 ▷ make code_n15_k6_d6_b_group
27122
27123
27124
27125 code_3_2_aut:
27126 ▷ $(ORBITER) -v 2 \
27127 ▷ ▷ -define F -finite_field -q 2 -end \
27128 ▷ ▷ -define genma -vector -field F -format 2 \
27129 ▷ ▷ ▷ -dense $(CODE_N3_K2_Q2_GENMA) \
27130 ▷ ▷ -end \
27131 ▷ ▷ -define P -projective_space -n 1 -field F -v 0 -end \
27132 ▷ ▷ -with P -do \
27133 ▷ ▷ -projective_space_activity \
27134 ▷ ▷ ▷ -canonical_form_of_code \
27135 ▷ ▷ ▷ ▷ "3_2" genma \
27136 ▷ ▷ ▷ ▷ -save_ago \
27137 ▷ ▷ ▷ ▷ -save_classification "3_2" \
27138 ▷ ▷ ▷ ▷ -nauty_control -end \
27139 ▷ ▷ ▷ -end \
27140 ▷ ▷ -end
27141 ▷ $(PDFLATEX) 3_2_classification.tex
27142 ▷ $(OPEN) 3_2_classification.pdf
27143 ▷ $(ORBITER) -v 2 -draw_matrix \
27144 ▷ ▷ -input_csv_file 3_2_object0_TDA_flag_orbits.csv \
27145 ▷ ▷ -secondary_input_csv_file 3_2_object0_TDA.csv \
27146 ▷ ▷ -box_width 16 -bit_depth 24 \
27147 ▷ ▷ -end
27148 ▷ $(OPEN) 3_2_object0_TDA_flag_orbits.draw.bmp
27149
27150 #projective_space_activity::perform_activity PA->canonical_form_of_code is disabl
    ed
27151
27152
27153
27154 code_6_3_aut:
27155 ▷ $(ORBITER) -v 2 \
27156 ▷ ▷ -define F -finite_field -q 2 -end \
27157 ▷ ▷ -define genma -vector -field F -format 3 \
27158 ▷ ▷ ▷ -compact $(CODE_N6_K3_Q2_GENMA) \
27159 ▷ ▷ -end \
27160 ▷ ▷ -define P -projective_space -n 2 -field F -v 0 -end \
27161 ▷ ▷ -with P -do \
27162 ▷ ▷ -projective_space_activity \
27163 ▷ ▷ ▷ -canonical_form_of_code \
27164 ▷ ▷ ▷ ▷ "6_3" genma \
27165 ▷ ▷ ▷ ▷ -save_ago \
27166 ▷ ▷ ▷ ▷ -save_classification "6_3" \
27167 ▷ ▷ ▷ ▷ -nauty_control -end \
27168 ▷ ▷ ▷ -end \
27169 ▷ ▷ -end
27170 ▷ $(PDFLATEX) 6_3_classification.tex
27171 ▷ $(OPEN) 6_3_classification.pdf
27172 ▷ $(ORBITER) -v 2 -draw_matrix \

```

```

27173 ▷ ▷ -input_csv_file 6_3.object0_TDA_flag_orbits.csv \
27174 ▷ ▷ -secondary_input_csv_file 6_3.object0_TDA.csv \
27175 ▷ ▷ -box_width 16 -bit_depth 24 \
27176 ▷ ▷ -end
27177 ▷ $(OPEN) 6_3.object0_TDA_flag_orbits.draw.bmp
27178
27179 # group of order 24
27180
27181
27182 RM_3_1_group:
27183 ▷ $(ORBITER) -v 2 \
27184 ▷ ▷ -define F -finite_field -q 2 -end \
27185 ▷ ▷ -define genma -vector -field F -format 4 \
27186 ▷ ▷ ▷ -compact $(CODE_RM_3_1_GENMA) \
27187 ▷ ▷ -end \
27188 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
27189 ▷ ▷ -with P -do \
27190 ▷ ▷ -projective_space_activity \
27191 ▷ ▷ ▷ -canonical_form_of_code \
27192 ▷ ▷ ▷ ▷ "RM_3_1" genma \
27193 ▷ ▷ ▷ ▷ -save_ago \
27194 ▷ ▷ ▷ ▷ -save_classification "RM_3_1" \
27195 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "RM_3_1_nauty_log.txt" -end \
27196 ▷ ▷ ▷ -end \
27197 ▷ ▷ -end
27198 ▷ $(PDFLATEX) RM_3_1_classification.tex
27199 ▷ $(OPEN) RM_3_1_classification.pdf
27200
27201 # group order 1344
27202 #RM_3_1_object0_INP_flag_orbits.csv
27203
27204 RM_3_1_group_and_diagram:
27205 ▷ $(ORBITER) -v 2 \
27206 ▷ ▷ -define F -finite_field -q 2 -end \
27207 ▷ ▷ -define genma -vector -field F -format 4 \
27208 ▷ ▷ ▷ -compact $(CODE_RM_3_1_GENMA) \
27209 ▷ ▷ -end \
27210 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
27211 ▷ ▷ -with P -do \
27212 ▷ ▷ -projective_space_activity \
27213 ▷ ▷ ▷ -canonical_form_of_code \
27214 ▷ ▷ ▷ ▷ "RM_3_1" genma \
27215 ▷ ▷ ▷ ▷ -save_ago \
27216 ▷ ▷ ▷ ▷ -save_classification "RM_3_1" \
27217 ▷ ▷ ▷ ▷ -nauty_control -end \
27218 ▷ ▷ ▷ -end \
27219 ▷ ▷ -end
27220 ▷ $(PDFLATEX) RM_3_1_classification.tex
27221 ▷ $(OPEN) RM_3_1_classification.pdf
27222 ▷ $(ORBITER) -v 2 -draw_matrix \
27223 ▷ ▷ -input_csv_file RM_3_1_object0_INP_flag_orbits.csv \
27224 ▷ ▷ -secondary_input_csv_file RM_3_1_object0_INP.csv \
27225 ▷ ▷ -box_width 16 -bit_depth 24 \
27226 ▷ ▷ -end
27227 ▷ $(ORBITER) -v 2 -draw_matrix \
27228 ▷ ▷ -input_csv_file RM_3_1_object0_TDA_flag_orbits.csv \
27229 ▷ ▷ -secondary_input_csv_file RM_3_1_object0_TDA.csv \
27230 ▷ ▷ -box_width 16 -bit_depth 24 \
27231 ▷ ▷ -end

```

```

27232 ▷ $(OPEN) RM_3_1_object0_INP_flag_orbits.draw.bmp
27233 ▷ $(OPEN) RM_3_1_object0_TDA_flag_orbits.draw.bmp
27234
27235
27236
27237 RM_4_1_group:
27238 ▷ $(ORBITER) -v 2 \
27239 ▷ ▷ -define F -finite_field -q 2 -end \
27240 ▷ ▷ -define genma -vector -field F -format 5 \
27241 ▷ ▷ ▷ -compact $(CODE_RM_4_1_GENMA) \
27242 ▷ ▷ -end \
27243 ▷ ▷ -define P -projective_space -n 4 -field F -v 0 -end \
27244 ▷ ▷ -with P -do \
27245 ▷ ▷ -projective_space_activity \
27246 ▷ ▷ ▷ -canonical_form_of_code \
27247 ▷ ▷ ▷ ▷ "RM_4_1" genma \
27248 ▷ ▷ ▷ ▷ -save_ago \
27249 ▷ ▷ ▷ ▷ -save_classification "RM_4_1" \
27250 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "RM_4_1_nauty_log.txt" -end \
27251 ▷ ▷ ▷ -end \
27252 ▷ ▷ -end
27253 ▷ $(PDFLATEX) RM_4_1_classification.tex
27254 ▷ $(OPEN) RM_4_1_classification.pdf
27255 ▷ $(ORBITER) -v 2 -draw_matrix \
27256 ▷ ▷ -input_csv_file RM_4_1_object0_INP_flag_orbits.csv \
27257 ▷ ▷ -secondary_input_csv_file RM_4_1_object0_INP.csv \
27258 ▷ ▷ -box_width 16 -bit_depth 24 \
27259 ▷ ▷ -end
27260 ▷ $(ORBITER) -v 2 -draw_matrix \
27261 ▷ ▷ -input_csv_file RM_4_1_object0_TDA_flag_orbits.csv \
27262 ▷ ▷ -secondary_input_csv_file RM_4_1_object0_TDA.csv \
27263 ▷ ▷ -box_width 16 -bit_depth 24 \
27264 ▷ ▷ -end
27265 ▷ $(OPEN) RM_4_1_object0_INP_flag_orbits.draw.bmp
27266 ▷ $(OPEN) RM_4_1_object0_TDA_flag_orbits.draw.bmp
27267
27268
27269 # group order 322560 = 24*30*28*16
27270
27271
27272 RS_6_4_7_group:
27273 ▷ $(ORBITER) -v 2 \
27274 ▷ ▷ -define F -finite_field -q 7 -end \
27275 ▷ ▷ -define genma -vector -field F -format 4 \
27276 ▷ ▷ ▷ -compact $(CODE_RS_6_4_7) \
27277 ▷ ▷ -end \
27278 ▷ ▷ -define P -projective_space -n 3 -field F -v 0 -end \
27279 ▷ ▷ -with P -do \
27280 ▷ ▷ -projective_space_activity \
27281 ▷ ▷ ▷ -canonical_form_of_code \
27282 ▷ ▷ ▷ ▷ "RS_6" genma \
27283 ▷ ▷ ▷ ▷ -save_ago \
27284 ▷ ▷ ▷ ▷ -save_classification "RS_6" \
27285 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "RS_6_4_7_nauty_log.txt" -end \
27286 ▷ ▷ ▷ -end \
27287 ▷ ▷ -end
27288
27289
27290 GV_n15.k6.d5_group:

```



```

27291 ▷ $(ORBITER) -v 2 \
27292 ▷ ▷ -define F -finite_field -q 2 -end \
27293 ▷ ▷ -define genma -vector -field F -format 6 \
27294 ▷ ▷ ▷ -compact $(CODE_GV_N15_K6) \
27295 ▷ ▷ -end \
27296 ▷ ▷ -define P -projective_space -n 5 -field F -v 0 -end \
27297 ▷ ▷ -with P -do \
27298 ▷ ▷ -projective_space_activity \
27299 ▷ ▷ ▷ -canonical_form_of_code \
27300 ▷ ▷ ▷ ▷ "GV_n15_k6_d5" genma \
27301 ▷ ▷ ▷ ▷ -save_ago \
27302 ▷ ▷ ▷ ▷ -save_classification "GV_n15_k6_d5" \
27303 ▷ ▷ ▷ ▷ -nauty_control -end \
27304 ▷ ▷ ▷ -end \
27305 ▷ ▷ -end
27306 ▷ $(PDFLATEX) GV_n15_k6_d5_classification.tex
27307 ▷ $(OPEN) GV_n15_k6_d5_classification.pdf
27308
27309 #ago=12
27310
27311
27312
27313 code_n15_k6_d6_a.group:
27314 ▷ $(ORBITER) -v 2 \
27315 ▷ ▷ -define F -finite_field -q 2 -end \
27316 ▷ ▷ -define genma -vector -field F -format 6 \
27317 ▷ ▷ ▷ -compact $(CODE_15_6_6.A) \
27318 ▷ ▷ -end \
27319 ▷ ▷ -define P -projective_space -n 5 -field F -v 0 -end \
27320 ▷ ▷ -with P -do \
27321 ▷ ▷ -projective_space_activity \
27322 ▷ ▷ ▷ -canonical_form_of_code \
27323 ▷ ▷ ▷ ▷ "n15_k6_d6_a" genma \
27324 ▷ ▷ ▷ ▷ -save_ago \
27325 ▷ ▷ ▷ ▷ -save_classification "n15_k6_d6_a" \
27326 ▷ ▷ ▷ ▷ -nauty_control -end \
27327 ▷ ▷ ▷ -end \
27328 ▷ ▷ -end
27329 ▷ $(PDFLATEX) n15_k6_d6_a_classification.tex
27330 ▷ $(OPEN) n15_k6_d6_a_classification.pdf
27331
27332
27333 code_n15_k6_d6_b.group:
27334 ▷ $(ORBITER) -v 2 \
27335 ▷ ▷ -define F -finite_field -q 2 -end \
27336 ▷ ▷ -define genma -vector -field F -format 6 \
27337 ▷ ▷ ▷ -compact $(CODE_15_6_6.B) \
27338 ▷ ▷ -end \
27339 ▷ ▷ -define P -projective_space -n 5 -field F -v 0 -end \
27340 ▷ ▷ -with P -do \
27341 ▷ ▷ -projective_space_activity \
27342 ▷ ▷ ▷ -canonical_form_of_code \
27343 ▷ ▷ ▷ ▷ "n15_k6_d6_b" genma \
27344 ▷ ▷ ▷ ▷ -save_ago \
27345 ▷ ▷ ▷ ▷ -save_classification "n15_k6_d6_b" \
27346 ▷ ▷ ▷ ▷ -nauty_control -end \
27347 ▷ ▷ ▷ -end \
27348 ▷ ▷ -end
27349 ▷ $(PDFLATEX) n15_k6_d6_b_classification.tex

```

```

27350 ▷ $(OPEN) n15.k6.d6.b.classification.pdf
27351
27352
27353
27354
27355 #####
27356 # Section 17.6: General Codes
27357
27358
27359 SECTION.CANONICAL_FORMS_OF_GENERAL_CODES:
27360
27361 test_17.6:
27362 ▷ make Hamming_graph_7_with_Hamming_code
27363
27364
27365
27366 Hamming_graph_7_with_Hamming_code:
27367 ▷ $(ORBITER) -v 2 \
27368 ▷ ▷ -define G -graph -Hamming 7 2 \
27369 ▷ ▷ ▷ -subset "_Hamming_code" "\\_with\\_Hamming\\_code" \
27370 ▷ ▷ ▷ $(HAMMING_CODE_CODEWORDS) -end \
27371 ▷ ▷ -with G -do \
27372 ▷ ▷ -graph_theoretic_activity -export_csv -end \
27373 ▷ ▷ -with G -do \
27374 ▷ ▷ -graph_theoretic_activity -export_graphviz -end \
27375 ▷ ▷ -with G -do \
27376 ▷ ▷ -graph_theoretic_activity -save -end \
27377 ▷ ▷ -with G -do \
27378 ▷ ▷ -graph_theoretic_activity -automorphism_group -end
27379 ▷ $(PDFLATEX) Hamming_7_2_Hamming_code_report.tex
27380 ▷ $(OPEN) Hamming_7_2_Hamming_code_report.pdf
27381
27382 # group of order 2688 = 16 * 168
27383
27384
27385
27386
27387 #####
27388 # Section 17.7: Graphs
27389
27390
27391 SECTION.CANONICAL_FORMS_OF_GRAPHS:
27392
27393
27394 test_17.7:
27395 ▷ make Cycle_13_aut
27396 ▷ make Neumaier_16_aut
27397 ▷ make Queens_graph_aut
27398 ▷ make Queens_graph_aut_orbits_draw
27399 ▷ make inversion_graph
27400 ▷ make Chain_232_aut
27401 ▷ #make JK_graph_pp16_1
27402 ▷ #make JK_graph_pp16_2
27403 ▷ #make JK_graph_pp16_9
27404 ▷ #make JK_graph_grid_3_3
27405 ▷ #make JK_graph_sts_13
27406 ▷ make HJ_group
27407 ▷ make HJ_group_and_orbits
27408 ▷ make HJ_orbital_graph_3

```

```

27409 ▷ make Group_Perm315_Orbital_3.colored_graph
27410 ▷ make HJ_d2_graph
27411 ▷ make PGO_5_2_graph_group
27412 ▷ make Paulus_c
27413 ▷ make Neumaier_graph_25.c
27414
27415
27416 Cycle_13_aut:
27417 ▷ $(ORBITER) -v 2 \
27418 ▷ ▷ -define Gamma -graph -cycle 13 -end \
27419 ▷ ▷ -with Gamma -do \
27420 ▷ ▷ ▷ -graph_theoretic_activity -automorphism_group \
27421 ▷ ▷ -end
27422 ▷ $(PDFLATEX) Cycle_13_report.tex
27423 ▷ $(OPEN) Cycle_13_report.pdf
27424
27425
27426 # ago=26
27427
27428
27429 Neumaier_16_aut:
27430 ▷ $(ORBITER) -v 2 \
27431 ▷ ▷ -define perm -vector -dense "0,1,2,3,4,5,12,13,6,7,8,9,10,11,14,15" -end \
27432 ▷ ▷ -define Gamma -graph -Neumaier_graph_16 -reorder perm -end \
27433 ▷ ▷ -with Gamma -do \
27434 ▷ ▷ ▷ -graph_theoretic_activity -automorphism_group \
27435 ▷ ▷ -end
27436 ▷ ▷ $(PDFLATEX) Neumaier_graph_16_report.tex
27437 ▷ ▷ $(OPEN) Neumaier_graph_16_report.pdf
27438
27439 # ago=256
27440
27441 Queens_graph_aut:
27442 ▷ $(ORBITER) -v 2 \
27443 ▷ ▷ -define Gamma -graph -non_attacking_queens_graph 8 -end \
27444 ▷ ▷ -with Gamma -do \
27445 ▷ ▷ ▷ -graph_theoretic_activity -automorphism_group \
27446 ▷ ▷ -end
27447 ▷ $(PDFLATEX) non_attacking_queens_graph_8_report.tex
27448 ▷ $(OPEN) non_attacking_queens_graph_8_report.pdf
27449
27450 #non_attacking_queens_graph_8_orbits.csv
27451 # ago=8
27452
27453
27454 Queens_graph_aut_orbits_draw:
27455 ▷ $(ORBITER) -v 2 -reformat \
27456 ▷ ▷ non_attacking_queens_graph_8_orbits.csv \
27457 ▷ ▷ Queens_orbits_on_points_8x8.csv 8
27458 ▷ $(ORBITER) -v 2 \
27459 ▷ ▷ -define all_one_r -vector -repeat 1 8 -end \
27460 ▷ ▷ -define all_one_c -vector -repeat 1 8 -end \
27461 ▷ ▷ -draw_matrix \
27462 ▷ ▷ -input_csv_file Queens_orbits_on_points_8x8.csv \
27463 ▷ ▷ -box_width 40 -bit_depth 24 \
27464 ▷ ▷ ▷ -partition 1 \
27465 ▷ ▷ ▷ all_one_r all_one_c \
27466 ▷ ▷ -end
27467 ▷ $(OPEN) Queens_orbits_on_points_8x8_draw.bmp

```

```

27468
27469
27470
27471 inversion_graph:
27472 ▷ $(ORBITER) -v 6 \
27473 ▷ ▷ -define G -graph \
27474 ▷ ▷ ▷ -inversion_graph "1,0,2,3" \
27475 ▷ ▷ -end \
27476 ▷ ▷ -with G -do \
27477 ▷ ▷ ▷ -graph_theoretic_activity -properties \
27478 ▷ ▷ -end \
27479 ▷ ▷ -with G -do \
27480 ▷ ▷ ▷ -graph_theoretic_activity -automorphism_group \
27481 ▷ ▷ -end
27482
27483
27484
27485 Chain_232_aut:
27486 ▷ $(ORBITER) -v 2 \
27487 ▷ ▷ -define P1 -vector -dense 2,3,2 -end \
27488 ▷ ▷ -define P2 -vector -dense 2,3,2 -end \
27489 ▷ ▷ -define Gamma -graph \
27490 ▷ ▷ ▷ -chain_graph P1 P2 \
27491 ▷ ▷ -end \
27492 ▷ ▷ -with Gamma -do \
27493 ▷ ▷ ▷ -graph_theoretic_activity -automorphism_group \
27494 ▷ ▷ -end
27495 ▷ $(PDFLATEX) chain_graph_report.tex
27496 ▷ $(OPEN) chain_graph_report.pdf
27497
27498
27499
27500
27501 JK_graph_pp16_1:
27502 ▷ $(ORBITER) -v 2 \
27503 ▷ ▷ -define Gamma -graph -load_dimacs \
27504 ▷ ▷ ../JUNTTILA_KASKI/benchmarks/pp/pp16-1 \
27505 ▷ ▷ -end \
27506 ▷ ▷ -with Gamma -do \
27507 ▷ ▷ -graph_theoretic_activity -save -end \
27508 ▷ ▷ -with Gamma -do \
27509 ▷ ▷ -graph_theoretic_activity -automorphism_group -end \
27510
27511 # go=34217164800
27512 #nauty_interface_with_group::create_automorphism_group_of_graph_with_partition_and_labeling: nb_backtrack1 = 6
27513 #nauty_interface_with_group::create_automorphism_group_of_graph_with_partition_and_labeling: nb_backtrack2 = 134
27514
27515 JK_graph_pp16_2:
27516 ▷ $(ORBITER) -v 2 \
27517 ▷ ▷ -define Gamma -graph -load_dimacs \
27518 ▷ ▷ ../JUNTTILA_KASKI/benchmarks/pp/pp16-2 \
27519 ▷ ▷ -end \
27520 ▷ ▷ -with Gamma -do \
27521 ▷ ▷ -graph_theoretic_activity -save -end \
27522 ▷ ▷ -with Gamma -do \
27523 ▷ ▷ -graph_theoretic_activity -automorphism_group -end \
27524

```

```

27525 # does not finish
27526
27527 JK_graph_pp16_9:
27528 ▷ $(ORBITER) -v 2 \
27529 ▷ ▷ -define Gamma -graph -load_dimacs \
27530 ▷ ▷ ../JUNTTILA_KASKI/benchmarks/pp/pp16-9 \
27531 ▷ ▷ -end \
27532 ▷ ▷ -with Gamma -do \
27533 ▷ ▷ -graph_theoretic_activity -save -end \
27534 ▷ ▷ -with Gamma -do \
27535 ▷ ▷ -graph_theoretic_activity -automorphism_group -end \
27536
27537
27538 JK_graph_grid_3_3:
27539 ▷ $(ORBITER) -v 2 \
27540 ▷ ▷ -define Gamma -graph -load_dimacs \
27541 ▷ ▷ ../JUNTTILA_KASKI/benchmarks/grid/grid-w-3-3 \
27542 ▷ ▷ -end \
27543 ▷ ▷ -with Gamma -do \
27544 ▷ ▷ -graph_theoretic_activity -save -end \
27545 ▷ ▷ -with Gamma -do \
27546 ▷ ▷ -graph_theoretic_activity -automorphism_group -end \
27547
27548
27549
27550
27551 JK_graph_sts_13:
27552 ▷ $(ORBITER) -v 2 \
27553 ▷ ▷ -define Gamma -graph -load_dimacs \
27554 ▷ ▷ ▷ ../JUNTTILA_KASKI/benchmarks/srg/sts-13 \
27555 ▷ ▷ -end \
27556 ▷ ▷ -with Gamma -do \
27557 ▷ ▷ -graph_theoretic_activity -save -end \
27558 ▷ ▷ -with Gamma -do \
27559 ▷ ▷ -graph_theoretic_activity -automorphism_group -end
27560 ▷ make ORBITER_PATH=$(ORBITER_EXE_PATH) -f sts-13_group.makefile sts-13
27561
27562
27563
27564 HJ_group:
27565 ▷ cp $(ORBITER_PATH)/examples/users_guide/halljanko315.csv .
27566 ▷ $(ORBITER) -v 6 \
27567 ▷ ▷ -define G -graph \
27568 ▷ ▷ ▷ -load_csv_no_border \
27569 ▷ ▷ ▷ halljanko315.csv \
27570 ▷ ▷ -end \
27571 ▷ ▷ -with G -do \
27572 ▷ ▷ ▷ -graph_theoretic_activity -automorphism_group \
27573 ▷ ▷ -end \
27574 ▷ ▷ -with G -do \
27575 ▷ ▷ ▷ -graph_theoretic_activity -properties \
27576 ▷ ▷ -end
27577 ▷ $(PDFLATEX) File_halljanko315_report.tex
27578 ▷ $(OPEN) File_halljanko315_report.pdf
27579
27580
27581 # ago = 1209600
27582
27583 #-rw-r--r-- 1 betten staff 1785 Sep 16 23:16 File_halljanko315_orbits.csv

```

```

27584 #-rw-r--r-- 1 betten staff 23892 Sep 16 23:16 File_halljanko315_report.tex

27585 #-rw-r--r-- 1 betten staff 7211 Sep 16 23:16 File_halljanko315_gens.csv
27586 #-rw-r--r-- 1 betten staff 231 Sep 16 23:16 File_halljanko315_group.make
file
27587
27588
27589 HJ_group_and_orbits:
27590 ▷ cp $(ORBITER_PATH)/examples/users_guide/halljanko315.csv .
27591 ▷ $(ORBITER) -v 6 \
27592 ▷ ▷ -define G -graph \
27593 ▷ ▷ ▷ -load_csv_no_border \
27594 ▷ ▷ ▷ halljanko315.csv \
27595 ▷ ▷ -end \
27596 ▷ ▷ -with G -do \
27597 ▷ ▷ ▷ -graph_theoretic_activity -automorphism_group \
27598 ▷ ▷ -end \
27599 ▷ ▷ -with G -do \
27600 ▷ ▷ ▷ -graph_theoretic_activity -properties \
27601 ▷ ▷ -end
27602 ▷ $(ORBITER) -v 2 \
27603 ▷ ▷ -define Control -poset_classification_control \
27604 ▷ ▷ ▷ -W \
27605 ▷ ▷ ▷ -problem_label HJ_orbits \
27606 ▷ ▷ ▷ -depth 2 \
27607 ▷ ▷ -end \
27608 ▷ ▷ -define gens -vector -file \
27609 ▷ ▷ ▷ File_halljanko315_gens.csv -end \
27610 ▷ ▷ -define G -permutation_group \
27611 ▷ ▷ ▷ -bsgs halljanko315 "File\halljanko315" \
27612 ▷ ▷ ▷ 315 1209600 "0,1,2" 5 gens \
27613 ▷ ▷ -end \
27614 ▷ ▷ -define Orb -orbits \
27615 ▷ ▷ ▷ -group G \
27616 ▷ ▷ ▷ -group_action G \
27617 ▷ ▷ ▷ -generators G \
27618 ▷ ▷ ▷ -on_subsets 2 Control \
27619 ▷ ▷ -end
27620
27621
27622
27623
27624 #ROW,REP,AGO,OL
27625 #0,"0,1",96,12600
27626 #1,"0,2",48,25200
27627 #2,"0,4",768,1575
27628 #3,"0,8",120,10080
27629 #END
27630
27631 HJ_orbital_graph_3:
27632 ▷ $(ORBITER) -v 2 \
27633 ▷ ▷ -define gens -vector -file \
27634 ▷ ▷ ▷ File_halljanko315_gens.csv -end \
27635 ▷ ▷ -define G -permutation_group \
27636 ▷ ▷ ▷ -bsgs halljanko315 "File\halljanko315" \
27637 ▷ ▷ ▷ 315 1209600 "0,1,2" 5 gens \
27638 ▷ ▷ -end \
27639 ▷ ▷ -define Gamma -graph \

```

```

27640 ▷ ▷ ▷ -orbital_graph G 3 \
27641 ▷ ▷ -end \
27642 ▷ ▷ -with Gamma -do \
27643 ▷ ▷ ▷ -graph_theoretic_activity \
27644 ▷ ▷ ▷ -properties \
27645 ▷ ▷ -end \
27646 ▷ ▷ -with Gamma -do \
27647 ▷ ▷ ▷ -graph_theoretic_activity \
27648 ▷ ▷ ▷ -save \
27649 ▷ ▷ -end
27650
27651 #Group_Perm315_Orbital_3.colored_graph
27652 #Degree type: (64^{315} )
27653
27654
27655 Group_Perm315_Orbital_3.colored_graph:
27656 ▷ $(ORBITER) -v 2 \
27657 ▷ ▷ -define gens -vector -format 5 -file \
27658 ▷ ▷ ▷ File_halljanko315.gens.csv -end \
27659 ▷ ▷ -define G -permutation_group \
27660 ▷ ▷ ▷ -bsgs halljanko315 "File\halljanko315" \
27661 ▷ ▷ ▷ 315 1209600 "0,1,2" 5 gens \
27662 ▷ ▷ -end \
27663 ▷ ▷ -define Gamma -graph \
27664 ▷ ▷ ▷ -orbital_graph G 3 \
27665 ▷ ▷ -end \
27666 ▷ ▷ -with Gamma -do \
27667 ▷ ▷ ▷ -graph_theoretic_activity -save \
27668 ▷ ▷ -end
27669
27670
27671 #halljanko315.gens.csv is missing
27672
27673
27674 HJ_d2_graph:
27675 ▷ cp $(ORBITER_PATH)/examples/users_guide/halljanko315.csv .
27676 ▷ $(ORBITER) -v 6 \
27677 ▷ ▷ -define G -graph \
27678 ▷ ▷ ▷ -load_csv_no_border \
27679 ▷ ▷ ▷ halljanko315.csv \
27680 ▷ ▷ ▷ -distance_2 \
27681 ▷ ▷ -end
27682
27683
27684
27685 PG0.5.2_graph_group:
27686 ▷ $(ORBITER) -v 2 \
27687 ▷ ▷ -define F -finite_field -q 2 -end \
27688 ▷ ▷ -define O -orthogonal_space 0 5 F -without_group -end \
27689 ▷ ▷ -with O -do -orthogonal_space_activity \
27690 ▷ ▷ ▷ -export_point_line_incidence_matrix \
27691 ▷ ▷ -end
27692 ▷ $(ORBITER) -v 3 \
27693 ▷ ▷ -define Inc -vector -file 0.5.2.incidence_matrix.csv -end \
27694 ▷ ▷ -define Gamma -graph -collinearity_graph Inc -end \
27695 ▷ ▷ -with Gamma -do \
27696 ▷ ▷ -graph_theoretic_activity \
27697 ▷ ▷ ▷ -automorphism_group \
27698 ▷ ▷ -end \

```

```

27699 ▷ ▷ -with Gamma -do \
27700 ▷ ▷ -graph_theoretic.activity \
27701 ▷ ▷ ▷ -eigenvalues \
27702 ▷ ▷ -end \
27703 ▷ ▷ -with Gamma -do \
27704 ▷ ▷ -graph_theoretic.activity \
27705 ▷ ▷ ▷ -eigenvalue_report \
27706 ▷ ▷ -end
27707 ▷ $(PDFLATEX) collinearity_graph_eigenvalues.tex
27708 ▷ $(OPEN) collinearity_graph_eigenvalues.pdf
27709
27710
27711
27712 Paulus.c:
27713 ▷ $(ORBITER) -v 6 \
27714 ▷ ▷ -define Doinc -draw_incidence_structure_options \
27715 ▷ ▷ ▷ -width 20 -width_10 2 -end \
27716 ▷ ▷ -loop L 1 16 1 \
27717 ▷ ▷ ▷ -define P25_%L -graph -load Brouwer \
27718 ▷ ▷ ▷ ▷ $(ORBITER_PATH)/examples/users_guide/PAULUS_ROSENFELD/P25_%L.txt \
27719 ▷ ▷ ▷ -end \
27720 ▷ ▷ -end_loop L \
27721 ▷ ▷ -define C -combinatorial.object \
27722 ▷ ▷ ▷ -label P25 P25 \
27723 ▷ ▷ ▷ -graph.object P25_1 \
27724 ▷ ▷ ▷ -graph.object P25_2 \
27725 ▷ ▷ ▷ -graph.object P25_3 \
27726 ▷ ▷ ▷ -graph.object P25_4 \
27727 ▷ ▷ ▷ -graph.object P25_5 \
27728 ▷ ▷ ▷ -graph.object P25_6 \
27729 ▷ ▷ ▷ -graph.object P25_7 \
27730 ▷ ▷ ▷ -graph.object P25_8 \
27731 ▷ ▷ ▷ -graph.object P25_9 \
27732 ▷ ▷ ▷ -graph.object P25_10 \
27733 ▷ ▷ ▷ -graph.object P25_11 \
27734 ▷ ▷ ▷ -graph.object P25_12 \
27735 ▷ ▷ ▷ -graph.object P25_13 \
27736 ▷ ▷ ▷ -graph.object P25_14 \
27737 ▷ ▷ ▷ -graph.object P25_15 \
27738 ▷ ▷ -end \
27739 ▷ ▷ -with C -do \
27740 ▷ ▷ -combinatorial.object.activity \
27741 ▷ ▷ ▷ -canonical_form \
27742 ▷ ▷ ▷ ▷ -save_ago \
27743 ▷ ▷ ▷ ▷ -save_transversal \
27744 ▷ ▷ ▷ ▷ -max_TDO_depth 10 \
27745 ▷ ▷ ▷ ▷ -nauty_control -end \
27746 ▷ ▷ ▷ -end \
27747 ▷ ▷ -end \
27748 ▷ ▷ -with C -do \
27749 ▷ ▷ -combinatorial.object.activity \
27750 ▷ ▷ ▷ -post_processing \
27751 ▷ ▷ -end \
27752 ▷ ▷ -with C -do \
27753 ▷ ▷ -combinatorial.object.activity \
27754 ▷ ▷ ▷ -report \
27755 ▷ ▷ ▷ ▷ -export_flag_orbits \
27756 ▷ ▷ ▷ ▷ -show_TDO \
27757 ▷ ▷ ▷ ▷ -show_TDA \

```



```

27758 ▷ ▷ ▷ ▷ -export_labels \
27759 ▷ ▷ ▷ ▷ -show_incidence_matrices \
27760 ▷ ▷ ▷ ▷ -export_group_GAP \
27761 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
27762 ▷ ▷ ▷ -end \
27763 ▷ ▷ -end
27764 ▷ $(PDFLATEX) P25_classification.tex
27765 ▷ $(OPEN) P25_classification.pdf
27766
27767
27768 Neumaier_graph_25.c:
27769 ▷ $(ORBITER) -v 10 \
27770 ▷ ▷ -define Gamma -graph -Neumaier_graph_25 -end \
27771 ▷ ▷ -with Gamma -do \
27772 ▷ ▷ -graph_theoretic_activity \
27773 ▷ ▷ ▷ -test_SRG_property \
27774 ▷ ▷ -end \
27775 ▷ ▷ -with Gamma -do \
27776 ▷ ▷ -graph_theoretic_activity \
27777 ▷ ▷ ▷ -eigenvalues \
27778 ▷ ▷ -end \
27779 ▷ ▷ -with Gamma -do \
27780 ▷ ▷ -graph_theoretic_activity \
27781 ▷ ▷ ▷ -eigenvalue_report \
27782 ▷ ▷ -end \
27783 ▷ ▷ -with Gamma -do \
27784 ▷ ▷ -graph_theoretic_activity \
27785 ▷ ▷ ▷ -find_cliques -target_size 5 \
27786 ▷ ▷ -end \
27787 ▷ ▷ -with Gamma -do \
27788 ▷ ▷ -graph_theoretic_activity \
27789 ▷ ▷ ▷ -automorphism_group \
27790 ▷ ▷ -end \
27791 ▷ ▷ -with Gamma -do \
27792 ▷ ▷ -graph_theoretic_activity \
27793 ▷ ▷ ▷ -test_Neumaier_property 5 \
27794 ▷ ▷ -end \
27795 ▷ ▷ -with Gamma -do \
27796 ▷ ▷ -graph_theoretic_activity \
27797 ▷ ▷ ▷ -print \
27798 ▷ ▷ -end \
27799 ▷ ▷ -define C -combinatorial_object \
27800 ▷ ▷ ▷ -label N25 N25 \
27801 ▷ ▷ ▷ -graph_object Gamma \
27802 ▷ ▷ -end \
27803 ▷ ▷ -with C -do \
27804 ▷ ▷ -combinatorial_object_activity \
27805 ▷ ▷ ▷ -canonical_form \
27806 ▷ ▷ ▷ ▷ -save_ago \
27807 ▷ ▷ ▷ ▷ -save_transversal \
27808 ▷ ▷ ▷ ▷ -max_TDO_depth 10 \
27809 ▷ ▷ ▷ ▷ -nauty_control -nauty_log "Neumaier_graph_25_nauty_log.txt" -end \
27810 ▷ ▷ ▷ -end \
27811 ▷ ▷ -end \
27812 ▷ ▷ -define Doinc -draw_incidence_structure_options \
27813 ▷ ▷ ▷ -width 20 -width_10 2 -end \
27814 ▷ ▷ -with C -do \
27815 ▷ ▷ -combinatorial_object_activity \
27816 ▷ ▷ ▷ -post_processing \

```

```

27817 ▷ ▷ -end \
27818 ▷ ▷ -with C -do \
27819 ▷ ▷ -combinatorial_object_activity \
27820 ▷ ▷ ▷ -report \
27821 ▷ ▷ ▷ ▷ -export_flag_orbits \
27822 ▷ ▷ ▷ ▷ -show_TDO \
27823 ▷ ▷ ▷ ▷ -show_TDA \
27824 ▷ ▷ ▷ ▷ -export_labels \
27825 ▷ ▷ ▷ ▷ -show_incidence_matrices \
27826 ▷ ▷ ▷ ▷ -export_group_GAP \
27827 ▷ ▷ ▷ ▷ -incidence_draw_options Doinc \
27828 ▷ ▷ ▷ -end \
27829 ▷ ▷ -end
27830 ▷ $(PDFLATEX) N25_classification.tex
27831 ▷ $(OPEN) N25_classification.pdf
27832
27833
27834 #ago = 18
27835
27836
27837 #####
27838 # Section 17.8: Quartic Curves
27839
27840
27841 SECTION_CANONICAL_FORMS_OF_QUARTIC_CURVES:
27842
27843
27844 test_17.8:
27845 ▷ make F_17_edge
27846 ▷ make Edge_curve_17_nauty
27847 ▷ make Edge_curve_17_group
27848
27849
27850
27851 F_17_edge:
27852 ▷ $(ORBITER) -v 3 \
27853 ▷ ▷ -define F -finite_field -q 17 -end \
27854 ▷ ▷ -with F -do -finite_field_activity \
27855 ▷ ▷ ▷ -cheat_sheet_GF -end
27856 ▷ $(PDFLATEX) GF_17.tex
27857 ▷ $(OPEN) GF_17.pdf
27858
27859
27860
27861
27862
27863 Edge_curve_17_nauty:
27864 ▷ $(ORBITER) -v 6 \
27865 ▷ ▷ -define F -finite_field -q 17 -end \
27866 ▷ ▷ -define R -polynomial_ring \
27867 ▷ ▷ ▷ -field F \
27868 ▷ ▷ ▷ -number_of_variables 3 \
27869 ▷ ▷ ▷ -homogeneous_of_degree 4 \
27870 ▷ ▷ ▷ -monomial_ordering_partition \
27871 ▷ ▷ ▷ -variables "X,Y,Z" "X,Y,Z" \
27872 ▷ ▷ -end \
27873 ▷ ▷ -assign Eqn -from \
27874 ▷ ▷ -with R -do \
27875 ▷ ▷ ▷ -ring_theoretic_activity \

```

```

27876 ▷ ▷ ▷ -parse.equation \
27877 ▷ ▷ ▷ ▷ "Edge_curve" "Edge\_curve" \
27878 ▷ ▷ ▷ ▷ $(EDGE_CURVE_EQUATION_ALGEBRAIC) \
27879 ▷ ▷ ▷ ▷ f 3 \
27880 ▷ ▷ -end \
27881 ▷ ▷ -define P -projective.space \
27882 ▷ ▷ ▷ -n 2 -field F -v 0 \
27883 ▷ ▷ -end \
27884 ▷ ▷ -define Geo -geometric.object P \
27885 ▷ ▷ ▷ -projective.variety R \
27886 ▷ ▷ ▷ ▷ "Edge_curve_q17" "Edge\_curve\_q17" \
27887 ▷ ▷ ▷ ▷ Eqn \
27888 ▷ ▷ -end \
27889 ▷ ▷ -define C -combinatorial.object \
27890 ▷ ▷ ▷ -label "Edge_curve_q17" "Edge\_curve\_q17" \
27891 ▷ ▷ ▷ -geometric.object Geo \
27892 ▷ ▷ -end \
27893 ▷ ▷ -with C -do \
27894 ▷ ▷ -combinatorial.object.activity \
27895 ▷ ▷ ▷ -canonical.form_PG P \
27896 ▷ ▷ ▷ ▷ -save.ago \
27897 ▷ ▷ ▷ ▷ -save.transversal \
27898 ▷ ▷ ▷ ▷ -max.TD0.depth 10 \
27899 ▷ ▷ ▷ ▷ -nauty.control -nauty.log "Edge_curve_q17_nauty.log.txt" -end \
27900 ▷ ▷ ▷ ▷ -end \
27901 ▷ ▷ ▷ -end \
27902 ▷ ▷ -define Doinc -draw.incidence.structure.options \
27903 ▷ ▷ ▷ -width 40 -width_10 6 \
27904 ▷ ▷ -end \
27905 ▷ ▷ -with C -do \
27906 ▷ ▷ -combinatorial.object.activity \
27907 ▷ ▷ ▷ -post.processing \
27908 ▷ ▷ -end \
27909 ▷ ▷ -with C -do \
27910 ▷ ▷ -combinatorial.object.activity \
27911 ▷ ▷ ▷ -report \
27912 ▷ ▷ ▷ ▷ -export.flag.orbits \
27913 ▷ ▷ ▷ ▷ -show.TD0 \
27914 ▷ ▷ ▷ ▷ -show.TDA \
27915 ▷ ▷ ▷ ▷ -dont.show.incidence.matrices \
27916 ▷ ▷ ▷ ▷ -export.group.GAP \
27917 ▷ ▷ ▷ ▷ -incidence.draw.options Doinc \
27918 ▷ ▷ ▷ -end \
27919 ▷ ▷ -end
27920 ▷ $(PDFLATEX) Edge_curve_q17_classification.tex
27921 ▷ $(OPEN) Edge_curve_q17_classification.pdf
27922 ▷ $(ORBITER) -v 2 -draw.matrix \
27923 ▷ ▷ -input.csv.file Edge_curve_q17_object_0.TDA.flag.orbits.csv \
27924 ▷ ▷ -secondary.input.csv.file Edge_curve_q17_object_0.TDA.csv \
27925 ▷ ▷ -box.width 4 -bit.depth 24 \
27926 ▷ ▷ -end
27927 ▷ $(OPEN) Edge_curve_q17_object_0.TDA.flag.orbits.draw.bmp
27928 ▷ $(ORBITER) -v 2 \
27929 ▷ ▷ -define Do -draw.options -embedded -radius 150 \
27930 ▷ ▷ ▷ -nodes.empty \
27931 ▷ ▷ ▷ -scale 0.5 -line.width 0.3 -end \
27932 ▷ ▷ -tree.draw \
27933 ▷ ▷ ▷ -file Edge_curve_q17_nauty.log.txt \
27934 ▷ ▷ ▷ -draw.options Do \

```

```

27935 ▷ ▷ -end
27936 ▷ $(PDFLATEX) Edge_curve.q17.nauty.log.draw.tex
27937 ▷ $(OPEN) Edge_curve.q17.nauty.log.draw.pdf
27938
27939 # 9 backtrack nodes total
27940 #Edge_curve.q17.nauty.log.txt
27941
27942 # aut = 24
27943 # User time: 0.04 of a second, dt=4 tps = 100
27944
27945
27946 # generators for a group of order 24:
27947 #1,0,0,0,13,0,0,0,4,
27948 #1,0,0,0,0,16,0,16,0,
27949 #0,1,16,2,4,4,15,4,4,
27950
27951
27952 Edge_curve.17.group:
27953 ▷ $(ORBITER) -v 3 \
27954 ▷ ▷ -define Do -draw.options \
27955 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
27956 ▷ ▷ -end \
27957 ▷ ▷ -define G -linear_group -PGL 3 17 \
27958 ▷ ▷ -subgroup_by_generators "Stab_Edge" "24" 3 \
27959 ▷ ▷ ▷ "1,0,0,0,13,0,0,0,4, \
27960 ▷ ▷ ▷ 1,0,0,0,0,16,0,16,0, \
27961 ▷ ▷ ▷ 0,1,16,2,4,4,15,4,4" \
27962 ▷ ▷ -end \
27963 ▷ ▷ -with G -do \
27964 ▷ ▷ ▷ -group.theoretic.activity \
27965 ▷ ▷ ▷ ▷ -group_table Do \
27966 ▷ ▷ -end \
27967 ▷ ▷ -with G -do \
27968 ▷ ▷ ▷ -group.theoretic.activity \
27969 ▷ ▷ ▷ ▷ -report Do \
27970 ▷ ▷ -end \
27971 ▷ ▷ -with G -do \
27972 ▷ ▷ ▷ -group.theoretic.activity \
27973 ▷ ▷ ▷ ▷ -print_elements_tex \
27974 ▷ ▷ -end
27975 ▷ $(PDFLATEX) PGL_3_17_Subgroup_Stab_Edge_24_group_table_report.tex
27976 ▷ $(OPEN) PGL_3_17_Subgroup_Stab_Edge_24_group_table_report.pdf
27977 ▷ $(PDFLATEX) PGL_3_17_Subgroup_Stab_Edge_24_report.tex
27978 ▷ $(OPEN) PGL_3_17_Subgroup_Stab_Edge_24_report.pdf
27979
27980
27981
27982
27983
27984
27985
27986
27987 #####
27988 # Chapter 18 - Applied Algebra
27989 #####
27990
27991
27992 test_18:
27993 ▷ make test_18.1

```

```

27994
27995
27996
27997 #####
27998 # Section 18.1: Automorphism group of Sym(6)
27999
28000 SECTION.MISCELLANEOUS:
28001
28002
28003
28004 test_18_1:
28005 ▷ make Sym_6
28006 ▷ make Sym_6_classes
28007 ▷ make Sym_6_Class_2A
28008 ▷ make Sym_6_Class_2B
28009 ▷ make Sym_6_Class_2C
28010 ▷ make Sym_6_concatenate_classes
28011 ▷ make Sym_6_class_2ABC_order_of_pairs
28012 ▷ make Sym_6_order_of_products_change_values
28013 ▷ make Sym_6_class_2ABC_make_graph_G2
28014 ▷ make Sym_6_class_2ABC_make_graph_G3
28015 ▷ make Sym_6_class_2ABC_subgraph_A5
28016 ▷ make Sym_6_aut_group
28017 ▷ make Sym_6_Schreier_tree_draw
28018 ▷ make Aut_Sym_6_group
28019 ▷ #make Aut_Sym_6_subgroup_lattice # too slow
28020 ▷ #make Aut_Sym_6_subgroup_lattice_load_and_draw
28021
28022
28023
28024
28025
28026
28027
28028 Sym_6:
28029 ▷ $(ORBITER) -v 3 \
28030 ▷ ▷ -define Do -draw.options \
28031 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
28032 ▷ ▷ -end \
28033 ▷ ▷ -define G -permutation_group -symmetric_group 6 -end \
28034 ▷ ▷ -with G -do \
28035 ▷ ▷ -group_theoretic_activity \
28036 ▷ ▷ ▷ -report Do \
28037 ▷ ▷ -end
28038 ▷ $(PDFLATEX) Sym_6_report.tex
28039 ▷ $(OPEN) Sym_6_report.pdf
28040
28041
28042 Sym_6_classes:
28043 ▷ $(ORBITER) -v 3 \
28044 ▷ ▷ -define G -permutation_group -symmetric_group 6 -end \
28045 ▷ ▷ -with G -do \
28046 ▷ ▷ -group_theoretic_activity \
28047 ▷ ▷ ▷ -classes \
28048 ▷ ▷ -end
28049 ▷ $(PDFLATEX) Sym_6_classes_out.tex
28050 ▷ $(OPEN) Sym_6_classes_out.pdf
28051
28052

```

```

28053 #of order 2 and with 0 fixed points. 1,0,3,2,5,4
28054 #of order 2 and with 2 fixed points. 1,0,3,2,4,5
28055 #of order 2 and with 4 fixed points. 1,0,2,3,4,5
28056
28057
28058 Sym_6_Class_2A:
28059 ▷ $(ORBITER) -v 9 \
28060 ▷ ▷ -orbiter_path $(ORBITER_PATH) \
28061 ▷ ▷ -define G -permutation-group -symmetric.group 6 -end \
28062 ▷ ▷ -with G -do \
28063 ▷ ▷ -group_theoretic.activity \
28064 ▷ ▷ ▷ -conjugacy_class_of "2A" $(SYM_6_CLASS_2A.REP) \
28065 ▷ ▷ -end
28066
28067 # writes the file Sym_6_class_of_2A.csv
28068 # which contains 15 elements in coded (data) form.
28069 # The coding of one element consists of 6 integers
28070
28071
28072 Sym_6_Class_2B:
28073 ▷ $(ORBITER) -v 9 \
28074 ▷ ▷ -orbiter_path $(ORBITER_PATH) \
28075 ▷ ▷ -define G -permutation-group -symmetric.group 6 -end \
28076 ▷ ▷ -with G -do \
28077 ▷ ▷ -group_theoretic.activity \
28078 ▷ ▷ ▷ -conjugacy_class_of "2B" $(SYM_6_CLASS_2B.REP) \
28079 ▷ ▷ -end
28080
28081
28082 # writes the file Sym_6_class_of_2B.csv
28083 # which contains 45 elements in coded (data) form.
28084 # The coding of one element consists of 6 integers
28085
28086
28087 Sym_6_Class_2C:
28088 ▷ $(ORBITER) -v 9 \
28089 ▷ ▷ -orbiter_path $(ORBITER_PATH) \
28090 ▷ ▷ -define G -permutation-group -symmetric.group 6 -end \
28091 ▷ ▷ -with G -do \
28092 ▷ ▷ -group_theoretic.activity \
28093 ▷ ▷ ▷ -conjugacy_class_of "2C" $(SYM_6_CLASS_2C.REP) \
28094 ▷ ▷ -end
28095
28096 # writes the file Sym_6_class_of_2C.csv.csv
28097 # which contains 15 elements in coded (data) form.
28098 # The coding of one element consists of 6 integers
28099
28100
28101
28102
28103
28104
28105 Sym_6_concatenate_classes:
28106 ▷ $(ORBITER) -v 3 \
28107 ▷ ▷ -csv_file_concatenate Sym_6_class_2ABC.csv 3 \
28108 ▷ ▷ ▷ $(FILE_NAME_SYM_6_CLASS_2A) \
28109 ▷ ▷ ▷ $(FILE_NAME_SYM_6_CLASS_2B) \
28110 ▷ ▷ ▷ $(FILE_NAME_SYM_6_CLASS_2C)
28111

```

```

28112
28113 #Sym_6_class_2ABC.csv
28114 # 75 = 15+45+15 rows
28115
28116 Sym_6_class_2ABC_order_of_pairs:
28117 ▷ $(ORBITER) -v 3 \
28118 ▷ ▷ -orbiter_path $(ORBITER_PATH) \
28119 ▷ ▷ -define G -permutation_group -symmetric_group 6 -end \
28120 ▷ ▷ -define Sym_6_class_2ABC -vector_ge \
28121 ▷ ▷ ▷ -action G \
28122 ▷ ▷ ▷ -read_csv Sym_6_class_2ABC.csv elements \
28123 ▷ ▷ ▷ -end \
28124 ▷ ▷ ▷ -with Sym_6_class_2ABC -do -vector_ge_activity \
28125 ▷ ▷ ▷ -order_of_products_of_pairs \
28126 ▷ ▷ ▷ -end \
28127
28128
28129 #Sym_6_class_2ABC_order_of_product_of_pairs.csv
28130 #Sym_6_class_2ABC_order_of_products_of_pairs.csv
28131
28132 Sym_6_order_of_products_change_values:
28133 ▷ $(ORBITER) -v 3 \
28134 ▷ ▷ -orbiter_path $(ORBITER_PATH) \
28135 ▷ ▷ -change_values \
28136 ▷ ▷ ▷ Sym_6_class_2ABC_order_of_product_of_pairs.csv \
28137 ▷ ▷ ▷ Sym_6_class_2ABC_order_3.csv \
28138 ▷ ▷ ▷ 1,2,3,4,5,6 \
28139 ▷ ▷ ▷ 0,0,1,0,0,0
28140 ▷ $(ORBITER) -v 3 \
28141 ▷ ▷ -orbiter_path $(ORBITER_PATH) \
28142 ▷ ▷ -change_values \
28143 ▷ ▷ ▷ Sym_6_class_2ABC_order_of_product_of_pairs.csv \
28144 ▷ ▷ ▷ Sym_6_class_2ABC_order_2.csv \
28145 ▷ ▷ ▷ 1,2,3,4,5,6 \
28146 ▷ ▷ ▷ 0,1,0,0,0,0
28147
28148 #Sym_6_class_2ABC_order_2.csv
28149 #Sym_6_class_2ABC_order_3.csv
28150
28151 Sym_6_class_2ABC_make_graph_G2:
28152 ▷ $(ORBITER) -v 3 \
28153 ▷ ▷ -define G -graph \
28154 ▷ ▷ ▷ -load_adjacency_matrix_from_csv_and_select_value \
28155 ▷ ▷ ▷ Sym_6_class_2ABC_order_2.csv 1 \
28156 ▷ ▷ ▷ -end \
28157 ▷ ▷ ▷ -with G -do \
28158 ▷ ▷ ▷ -graph_theoretic_activity -properties \
28159 ▷ ▷ ▷ -end \
28160 ▷ ▷ ▷ -with G -do \
28161 ▷ ▷ ▷ -graph_theoretic_activity -eigenvalues \
28162 ▷ ▷ ▷ -end \
28163 ▷ ▷ ▷ -with G -do \
28164 ▷ ▷ ▷ -graph_theoretic_activity -automorphism_group \
28165 ▷ ▷ ▷ -end
28166 ▷ $(PDFLATEX) File_Sym_6_class_2ABC_order_2_report.tex
28167 ▷ $(OPEN) File_Sym_6_class_2ABC_order_2_report.pdf
28168
28169 # ago = 1440
28170 # degree = 75

```

```

28171
28172
28173 Sym_6_class_2ABC_make_graph_G3:
28174 ▷ $(ORBITER) -v 3 \
28175 ▷ ▷ -define G -graph \
28176 ▷ ▷ ▷ -load_adjacency_matrix_from_csv_and_select_value \
28177 ▷ ▷ ▷ Sym_6_class_2ABC_order_3.csv 1 \
28178 ▷ ▷ -end \
28179 ▷ ▷ -with G -do \
28180 ▷ ▷ ▷ -graph_theoretic_activity -properties \
28181 ▷ ▷ -end \
28182 ▷ ▷ -with G -do \
28183 ▷ ▷ ▷ -graph_theoretic_activity -eigenvalues \
28184 ▷ ▷ -end \
28185 ▷ ▷ -with G -do \
28186 ▷ ▷ ▷ -graph_theoretic_activity -automorphism_group \
28187 ▷ ▷ -end
28188 ▷ $(PDFLATEX) File_Sym_6_class_2ABC_order_3_report.tex
28189 ▷ $(OPEN) File_Sym_6_class_2ABC_order_3_report.pdf
28190
28191 #ago=3762339840000
28192
28193
28194
28195 Sym_6_class_2ABC_subgraph_A5:
28196 ▷ $(ORBITER) -v 3 \
28197 ▷ ▷ -define G2 -graph \
28198 ▷ ▷ ▷ -load_adjacency_matrix_from_csv_and_select_value \
28199 ▷ ▷ ▷ Sym_6_class_2ABC_order_2.csv 1 \
28200 ▷ ▷ -end \
28201 ▷ ▷ -define G3 -graph \
28202 ▷ ▷ ▷ -load_adjacency_matrix_from_csv_and_select_value \
28203 ▷ ▷ ▷ Sym_6_class_2ABC_order_3.csv 1 \
28204 ▷ ▷ -end \
28205 ▷ ▷ -with G2 -and G3 -do \
28206 ▷ ▷ ▷ -graph_theoretic_activity -find_subgraph "A5" \
28207 ▷ ▷ -end \
28208
28209 #File_Sym_6_class_2ABC_order_2_all_A5.csv
28210 #1440 solutions
28211
28212
28213
28214 Sym_6_aut_group:
28215 ▷ $(ORBITER) -v 3 \
28216 ▷ ▷ -define images -vector -file_column \
28217 ▷ ▷ ▷ File_Sym_6_class_2ABC_order_2_all_A5.csv "C0" \
28218 ▷ ▷ -end \
28219 ▷ ▷ -define G -permutation_group -symmetric_group 6 -end \
28220 ▷ ▷ -define elements_of_order2_ge -vector_ge \
28221 ▷ ▷ ▷ -action G \
28222 ▷ ▷ ▷ -read_csv Sym_6_class_2ABC.csv "elements" \
28223 ▷ ▷ -end \
28224 ▷ ▷ -with G -do \
28225 ▷ ▷ -group_theoretic_activity \
28226 ▷ ▷ ▷ -group_of_automorphisms_by_images_of_generators \
28227 ▷ ▷ ▷ sym_6_aut_group \
28228 ▷ ▷ ▷ elements_of_order2_ge \
28229 ▷ ▷ ▷ images \

```



```

28230 ▷ ▷ -end
28231
28232
28233
28234 Sym_6_Schreier_tree_draw:
28235 ▷ $(ORBITER) -v 3 \
28236 ▷ ▷ -draw_layered_graph \
28237 ▷ ▷ ▷ images_tree.layered_graph \
28238 ▷ ▷ ▷ -place \
28239 ▷ ▷ ▷ -embedded \
28240 ▷ ▷ ▷ -nodes_empty \
28241 ▷ ▷ ▷ -spanning_tree \
28242 ▷ ▷ ▷ -radius 25 -line_width 1.3 \
28243 ▷ ▷ ▷ -y_stretch 1.0 -scale 0.5 \
28244 ▷ ▷ -end
28245 ▷ $(PDFLATEX) images_tree.draw.tex
28246 ▷ $(OPEN) images_tree.draw.pdf
28247
28248
28249 Aut_Sym_6_group:
28250 ▷ $(ORBITER) -v 3 \
28251 ▷ ▷ -define Do -draw_options \
28252 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
28253 ▷ ▷ -end \
28254 ▷ ▷ -define gens -vector -dense $(AUT_SYM_6_GENS) -end \
28255 ▷ ▷ -define G -permutation_group \
28256 ▷ ▷ ▷ -bsgs AutSym6 "{\rm AutSym}(6)" \
28257 ▷ ▷ ▷ $(AUT_SYM_6_DEGREE) \
28258 ▷ ▷ ▷ $(AUT_SYM_6_GROUP_ORDER) \
28259 ▷ ▷ ▷ $(AUT_SYM_6_BASE) \
28260 ▷ ▷ ▷ $(AUT_SYM_6_NB_GENS) \
28261 ▷ ▷ ▷ gens \
28262 ▷ ▷ -end \
28263 ▷ ▷ -with G -do \
28264 ▷ ▷ -group_theoretic_activity \
28265 ▷ ▷ ▷ -report Do \
28266 ▷ ▷ -end
28267 ▷ $(PDFLATEX) AutSym6_report.tex
28268 ▷ $(OPEN) AutSym6_report.pdf
28269
28270
28271 Aut_Sym_6_subgroup_lattice:
28272 ▷ $(ORBITER) -v 3 \
28273 ▷ ▷ -define gens -vector -dense $(AUT_SYM_6_GENS) -end \
28274 ▷ ▷ -define G -permutation_group \
28275 ▷ ▷ ▷ -bsgs AutSym6 "{\rm AutSym}(6)" \
28276 ▷ ▷ ▷ $(AUT_SYM_6_DEGREE) \
28277 ▷ ▷ ▷ $(AUT_SYM_6_GROUP_ORDER) \
28278 ▷ ▷ ▷ $(AUT_SYM_6_BASE) \
28279 ▷ ▷ ▷ $(AUT_SYM_6_NB_GENS) \
28280 ▷ ▷ ▷ gens \
28281 ▷ ▷ -end \
28282 ▷ ▷ -with G -do \
28283 ▷ ▷ -group_theoretic_activity \
28284 ▷ ▷ ▷ -subgroup_lattice \
28285 ▷ ▷ -end
28286
28287 # too slow
28288 # 457 zuppos

```

```

28289 #AutSym6_subgroup_lattice.csv
28290
28291 Aut_Sym_6_subgroup_lattice_load_and_draw:
28292 ▷ $(ORBITER) -v 3 \
28293 ▷ ▷ -define Do -draw.options \
28294 ▷ ▷ ▷ -radius 200 -line.width 0.5 \
28295 ▷ ▷ -end \
28296 ▷ ▷ -define gens -vector -dense $(AUT_SYM_6_GENS) -end \
28297 ▷ ▷ -define G -permutation_group \
28298 ▷ ▷ ▷ -bsgs AutSym6 "{\rm AutSym}(6)" \
28299 ▷ ▷ ▷ $(AUT_SYM_6_DEGREE) \
28300 ▷ ▷ ▷ $(AUT_SYM_6_GROUP_ORDER) \
28301 ▷ ▷ ▷ $(AUT_SYM_6_BASE) \
28302 ▷ ▷ ▷ $(AUT_SYM_6_NB_GENS) \
28303 ▷ ▷ ▷ gens \
28304 ▷ ▷ -end \
28305 ▷ ▷ -with G -do \
28306 ▷ ▷ -group_theoretic_activity \
28307 ▷ ▷ ▷ -subgroup_lattice_load AutSym6_subgroup_lattice.csv \
28308 ▷ ▷ -end \
28309 ▷ ▷ -with G -do \
28310 ▷ ▷ -group_theoretic_activity \
28311 ▷ ▷ ▷ -subgroup_lattice_draw_by_orbits Do \
28312 ▷ ▷ -end
28313 ▷ #$(PDFLATEX) AutSym6_drawing.tex
28314 ▷ #$(OPEN) AutSym6_drawing.pdf
28315 ▷ $(PDFLATEX) AutSym6_drawing_by_orbits.tex
28316 ▷ $(OPEN) AutSym6_drawing_by_orbits.pdf
28317
28318
28319
28320
28321
28322
28323
28324
28325
28326 #####
28327 # Chapter 19 - Interfaces
28328 #####
28329
28330
28331 test_19:
28332 ▷ make test_19_1
28333 ▷ make test_19_2
28334 ▷ make test_19_3
28335 ▷ make test_19_4
28336 ▷ make test_19_5
28337
28338
28339 #####
28340 # Section 19.1: Graphical Output
28341
28342 SECTION_GRAPHICAL_OUTPUT:
28343
28344
28345 test_19_1:
28346 ▷ make PGL_6_2_Wedge_4_2_detached_report.tex
28347 ▷ make schreier_tree_graphical_output

```

```

28348 ▷ make Queens_graph_again
28349
28350
28351
28352
28353
28354 PGL_6_2_Wedge_4_2_detached_report.tex:
28355 ▷ $(ORBITER) -v 4 \
28356 ▷ ▷ -define Do -draw_options \
28357 ▷ ▷ ▷ -radius 200 -line_width 0.3 \
28358 ▷ ▷ ▷ -end \
28359 ▷ ▷ -define G -linear_group -PGL 4 2 \
28360 ▷ ▷ ▷ -wedge_detached \
28361 ▷ ▷ -end \
28362 ▷ ▷ -with G -do \
28363 ▷ ▷ -group_theoretic_activity \
28364 ▷ ▷ ▷ -report Do \
28365 ▷ ▷ -end
28366 ▷ $(PDFLATEX) PGL_6_2_Wedge_4_2_detached_report.tex
28367 ▷ $(OPEN) PGL_6_2_Wedge_4_2_detached_report.pdf
28368
28369
28370 schreier_tree_graphical_output:
28371 ▷ $(ORBITER) -v 4 \
28372 ▷ ▷ -define Do -draw_options \
28373 ▷ ▷ ▷ -yout 500000 \
28374 ▷ ▷ ▷ -radius 15 -nodes_empty \
28375 ▷ ▷ ▷ -line_width 0.5 -y_stretch 0.25 \
28376 ▷ ▷ ▷ -embedded \
28377 ▷ ▷ -end \
28378 ▷ ▷ -define F -finite_field -q 2 -end \
28379 ▷ ▷ -define G -linear_group -PGL 4 F -end \
28380 ▷ ▷ -define R -polynomial_ring \
28381 ▷ ▷ ▷ -field F \
28382 ▷ ▷ ▷ -number_of_variables 4 \
28383 ▷ ▷ ▷ -homogeneous_of_degree 3 \
28384 ▷ ▷ ▷ -monomial_ordering_partition \
28385 ▷ ▷ ▷ -variables "X,Y,Z,W" "X,Y,Z,W" \
28386 ▷ ▷ -end \
28387 ▷ ▷ -define Orb -orbits \
28388 ▷ ▷ ▷ -group G \
28389 ▷ ▷ ▷ -print_interval 1000000 \
28390 ▷ ▷ ▷ -on_polynomials R \
28391 ▷ ▷ -end \
28392 ▷ ▷ -with Orb -do -orbits_activity \
28393 ▷ ▷ ▷ -export_something "orbit" 6 \
28394 ▷ ▷ -end
28395 ▷ ▷ -with Orb -do -orbits_activity \
28396 ▷ ▷ ▷ -draw_tree 6 Do \
28397 ▷ ▷ -end
28398
28399
28400 # slow
28401
28402
28403
28404 Queens_graph_again:
28405 ▷ $(ORBITER) -v 2 \
28406 ▷ ▷ -define G -graph -non_attacking_queens_graph 8 -end \

```

```

28407 ▷ ▷ -with G -do \
28408 ▷ ▷ -graph_theoretic.activity -export_csv -end \
28409 ▷ ▷ -with G -do \
28410 ▷ ▷ -graph_theoretic.activity -export_graphviz -end \
28411 ▷ ▷ -with G -do \
28412 ▷ ▷ -graph_theoretic.activity -save -end \
28413 ▷ ▷ -with G -do \
28414 ▷ ▷ -graph_theoretic.activity -automorphism_group -end \
28415 ▷ ▷ -with G -do \
28416 ▷ ▷ -graph_theoretic.activity -find_cliques \
28417 ▷ ▷ -target_size 8 -output_file 8queens \
28418 ▷ ▷ -end
28419
28420
28421
28422
28423 #####
28424 # Section 19.2: The Povray Interface
28425
28426
28427 SECTION_POVRAY:
28428
28429 test_19_2:
28430 ▷ make cube
28431 ▷ make math261_test
28432 ▷ make plane1
28433 ▷ make plane2
28434 ▷ make analytic_geo_1
28435 ▷ #make analytic_geo_1_video # needs ffmpeg
28436 ▷ make monkey
28437 ▷ make Eckardt
28438 ▷ make Eckardt.deform
28439 ▷ make Eckardt.deform_2
28440 ▷ make Clebsch
28441 ▷ make endrass8
28442
28443
28444 cube:
28445 ▷ cp $(ORBITER_PATH)/examples/users_guide/cube_centered.obj .
28446 ▷ $(ORBITER) -v 2 -povray \
28447 ▷ ▷ -round 0 -nb_frames_default 30 \
28448 ▷ ▷ -output_mask cube_%d.%03d.pov \
28449 ▷ ▷ -video_options -W 1024 -H 768 \
28450 ▷ ▷ -global_picture_scale 0.5 \
28451 ▷ ▷ -default_angle 75 \
28452 ▷ ▷ -clipping_radius 2.7 \
28453 ▷ ▷ -end \
28454 ▷ ▷ -scene_objects \
28455 ▷ ▷ ▷ -obj_file cube_centered.obj \
28456 ▷ ▷ ▷ -edge "0, 1" \
28457 ▷ ▷ ▷ -edge "0, 2" \
28458 ▷ ▷ ▷ -edge "0, 4" \
28459 ▷ ▷ ▷ -edge "1, 3" \
28460 ▷ ▷ ▷ -edge "1, 5" \
28461 ▷ ▷ ▷ -edge "2, 3" \
28462 ▷ ▷ ▷ -edge "2, 6" \
28463 ▷ ▷ ▷ -edge "3, 7" \
28464 ▷ ▷ ▷ -edge "4, 5" \
28465 ▷ ▷ ▷ -edge "4, 6" \

```

```

28466 ▷ ▷ ▷ -edge "5, 7" \
28467 ▷ ▷ ▷ -edge "6, 7" \
28468 ▷ ▷ ▷ -group_of_things.as.interval 0 8 \
28469 ▷ ▷ ▷ -spheres 0 0.3 $(POLISHED_CHROME_WHITE) \
28470 ▷ ▷ ▷ -group_of_things.as.interval 0 6 \
28471 ▷ ▷ ▷ -prisms 1 0.05 $(YELLOW_TRANSPARENT) \
28472 ▷ ▷ ▷ -group_of_things.as.interval 0 12 \
28473 ▷ ▷ ▷ -cylinders 2 0.15 $(COLOR_RED) \
28474 ▷ ▷ -scene_objects_end \
28475 ▷ ▷ -povray_end
28476 ▷ - rm -rf POV
28477 ▷ mkdir POV
28478 ▷ mv cube_0_*.pov POV
28479 ▷ mv makefile_animation POV
28480
28481
28482 math261_test:
28483 ▷ $(ORBITER) -v 2 -povray \
28484 ▷ ▷ -round 0 -nb_frames_default 30 \
28485 ▷ ▷ -output_mask math261_%d.%03d.pov \
28486 ▷ ▷ -video_options -W 1024 -H 768 \
28487 ▷ ▷ -global_picture_scale 0.1 \
28488 ▷ ▷ -default_angle 75 \
28489 ▷ ▷ -clipping_radius 2.7 \
28490 ▷ ▷ -end \
28491 ▷ ▷ -scene_objects \
28492 ▷ ▷ ▷ -point "0,0,0" \
28493 ▷ ▷ ▷ -point "5,0,0" \
28494 ▷ ▷ ▷ -point "0,5,0" \
28495 ▷ ▷ ▷ -point "0,0,5" \
28496 ▷ ▷ ▷ -point "1,2,3" \
28497 ▷ ▷ ▷ -point "4,5,6" \
28498 ▷ ▷ ▷ -point "5,7,9" \
28499 ▷ ▷ ▷ -edge "0,1" \
28500 ▷ ▷ ▷ -edge "0,2" \
28501 ▷ ▷ ▷ -edge "0,3" \
28502 ▷ ▷ ▷ -edge "0,4" \
28503 ▷ ▷ ▷ -edge "0,5" \
28504 ▷ ▷ ▷ -edge "4,6" \
28505 ▷ ▷ ▷ -edge "5,6" \
28506 ▷ ▷ ▷ -face "0,4,6,5" \
28507 ▷ ▷ ▷ -group_of_things.as.interval 0 7 \
28508 ▷ ▷ ▷ -spheres 0 0.1 $(POLISHED_CHROME_WHITE) \
28509 ▷ ▷ ▷ -group_of_things.as.interval 0 7 \
28510 ▷ ▷ ▷ -cylinders 1 0.05 $(COLOR_RED) \
28511 ▷ ▷ ▷ -prisms 2 0.05 $(YELLOW_TRANSPARENT) \
28512 ▷ ▷ ▷ -group_of_things.as.interval 0 1 \
28513 ▷ ▷ -scene_objects_end \
28514 ▷ ▷ -povray_end
28515 ▷ - rm -rf POV
28516 ▷ mkdir POV
28517 ▷ mv math261_0_*.pov POV
28518 ▷ mv makefile_animation POV
28519
28520
28521
28522 plane1:
28523 ▷ cp $(ORBITER_PATH)/examples/users_guide/coordinate_grid.csv .
28524 ▷ $(ORBITER) -v 2 -povray \

```

```

28525 ▷ ▷ -round 0 -nb_frames_default 30 \
28526 ▷ ▷ -output_mask plane1_%d.%03d.pov \
28527 ▷ ▷ -video_options -W 1024 -H 768 \
28528 ▷ ▷ -global_picture_scale 0.40 \
28529 ▷ ▷ -default_angle 75 \
28530 ▷ ▷ -clipping_radius 5 -omit_bottom_plane \
28531 ▷ ▷ -camera 0 "0,0,1" "5,5,3" "0,0,0" \
28532 ▷ ▷ -rotate_about_z_axis \
28533 ▷ ▷ -boundary_box \
28534 ▷ ▷ -end \
28535 ▷ ▷ -scene_objects \
28536 ▷ ▷ ▷ -line_through_two_points_recentered_from_csv_file coordinate_grid.csv \
28537 ▷ ▷ ▷ -plane_by_dual_coordinates "0,0,1,0" \
28538 ▷ ▷ ▷ -plane_by_dual_coordinates "0,1,0,0" \
28539 ▷ ▷ ▷ -plane_by_dual_coordinates "1,0,0,0" \
28540 ▷ ▷ ▷ -point "-2.25,0,0" \
28541 ▷ ▷ ▷ -point "0,-1.8,0" \
28542 ▷ ▷ ▷ -point "0,0,9" \
28543 ▷ ▷ ▷ -face "0,1,2,0" \
28544 ▷ ▷ ▷ -group_of_things "0" \
28545 ▷ ▷ ▷ -group_of_things "1" \
28546 ▷ ▷ ▷ -group_of_things "2" \
28547 ▷ ▷ ▷ -lines 0 0.15 $(COLOR_RED_SHINY) \
28548 ▷ ▷ ▷ -lines 1 0.15 $(COLOR_GREEN_SHINY) \
28549 ▷ ▷ ▷ -lines 2 0.15 $(COLOR_BLUE_SHINY) \
28550 ▷ ▷ ▷ -group_of_things_as_interval 3 39 \
28551 ▷ ▷ ▷ -lines 3 0.05 $(COLOR_BLACK_SHINY) \
28552 ▷ ▷ ▷ -group_of_things "0" \
28553 ▷ ▷ ▷ -planes 0 $(COLOR_BLUE_SEE_THROUGH) \
28554 ▷ ▷ ▷ -group_of_things "1" \
28555 ▷ ▷ ▷ -group_of_things "2" \
28556 ▷ ▷ ▷ -group_of_things "0" \
28557 ▷ ▷ ▷ -prisms 0 0.05 $(COLOR_YELLOW_THICK) \
28558 ▷ ▷ -scene_objects_end \
28559 ▷ ▷ -povray_end
28560 ▷ - rm -rf POV
28561 ▷ mkdir POV
28562 ▷ mv plane1_0_*.pov POV
28563 ▷ mv makefile_animation POV
28564
28565
28566
28567 plane2:
28568 ▷ cp $(ORBITER_PATH)/examples/users_guide/coordinate_grid.csv .
28569 ▷ $(ORBITER) -v 2 -povray \
28570 ▷ ▷ -round 0 -nb_frames_default 30 \
28571 ▷ ▷ -output_mask plane2_%d.%03d.pov \
28572 ▷ ▷ -video_options -W 2560 -H 1920 \
28573 ▷ ▷ -global_picture_scale 0.40 \
28574 ▷ ▷ -default_angle 75 \
28575 ▷ ▷ -clipping_radius 5 -omit_bottom_plane \
28576 ▷ ▷ ▷ -camera 0 "0,0,1" "6,6,2" "0,0,0" \
28577 ▷ ▷ ▷ -rotate_about_z_axis \
28578 ▷ ▷ ▷ -boundary_box \
28579 ▷ ▷ -end \
28580 ▷ ▷ -scene_objects \
28581 ▷ ▷ ▷ -line_through_two_points_recentered_from_csv_file coordinate_grid.csv \
28582 ▷ ▷ ▷ -plane_by_dual_coordinates "0,0,1,0" \
28583 ▷ ▷ ▷ -plane_by_dual_coordinates "0,1,0,0" \

```

```

28584 ▷ ▷ ▷ -plane.by_dual_coordinates "1,0,0,0" \
28585 ▷ ▷ ▷ -plane.by_dual_coordinates "4,5,-1,9" \
28586 ▷ ▷ ▷ -group_of_things "0" \
28587 ▷ ▷ ▷ -group_of_things "1" \
28588 ▷ ▷ ▷ -group_of_things "2" \
28589 ▷ ▷ ▷ -group_of_things.as.interval 3 39 \
28590 ▷ ▷ ▷ -lines 0 0.15 $(COLOR_RED_SHINY) \
28591 ▷ ▷ ▷ -lines 1 0.15 $(COLOR_GREEN_SHINY) \
28592 ▷ ▷ ▷ -lines 2 0.15 $(COLOR_BLUE_SHINY) \
28593 ▷ ▷ ▷ -lines 3 0.05 $(COLOR_BLACK_SHINY) \
28594 ▷ ▷ ▷ -group_of_things "0" \
28595 ▷ ▷ ▷ -planes 4 $(COLOR_BLUE_SEE_THROUGH) \
28596 ▷ ▷ ▷ -group_of_things "3" \
28597 ▷ ▷ -scene_objects_end \
28598 ▷ ▷ -povray_end
28599 ▷ - rm -rf POV
28600 ▷ mkdir POV
28601 ▷ mv plane2.0*.pov POV
28602 ▷ mv makefile_animation POV
28603
28604
28605
28606
28607 analytic_geo_1:
28608 ▷ cp $(ORBITER_PATH)/examples/users_guide/coordinate_grid.csv .
28609 ▷ $(ORBITER) -v 2 -povray \
28610 ▷ ▷ -round 0 -nb_frames_default 30 \
28611 ▷ ▷ -output_mask analytic_geo_1.%d.%03d.pov \
28612 ▷ ▷ -video_options -W 2560 -H 1920 \
28613 ▷ ▷ -global_picture_scale 0.80 \
28614 ▷ ▷ -default_angle 75 \
28615 ▷ ▷ -clipping_radius 5 -omit_bottom_plane \
28616 ▷ ▷ ▷ -camera 0 "0,0,1" "6,6,2" "0,0,0" \
28617 ▷ ▷ ▷ -rotate_about_z_axis \
28618 ▷ ▷ ▷ -boundary_box \
28619 ▷ ▷ -end \
28620 ▷ ▷ -scene_objects \
28621 ▷ ▷ ▷ -line_through_two_points_recentered_from_csv_file coordinate_grid.csv \
28622 ▷ ▷ ▷ -plane.by_dual_coordinates "0,0,1,0" \
28623 ▷ ▷ ▷ -plane.by_dual_coordinates "0,1,0,0" \
28624 ▷ ▷ ▷ -plane.by_dual_coordinates "1,0,0,0" \
28625 ▷ ▷ ▷ -group_of_things "0" \
28626 ▷ ▷ ▷ -group_of_things "1" \
28627 ▷ ▷ ▷ -group_of_things "2" \
28628 ▷ ▷ ▷ -group_of_things.as.interval 3 39 \
28629 ▷ ▷ ▷ -lines 0 0.05 $(COLOR_RED_SHINY) \
28630 ▷ ▷ ▷ -lines 1 0.05 $(COLOR_GREEN_SHINY) \
28631 ▷ ▷ ▷ -lines 2 0.05 $(COLOR_BLUE_SHINY) \
28632 ▷ ▷ ▷ -lines 3 0.04 $(COLOR_BLACK_SHINY) \
28633 ▷ ▷ ▷ -group_of_things "0" \
28634 ▷ ▷ ▷ -group_of_things "1" \
28635 ▷ ▷ ▷ -group_of_things "2" \
28636 ▷ ▷ ▷ -planes 4 $(COLOR_BLUE_SEE_THROUGH) \
28637 ▷ ▷ ▷ -planes 5 $(COLOR_GREEN_SEE_THROUGH) \
28638 ▷ ▷ ▷ -planes 6 $(COLOR_RED_SEE_THROUGH) \
28639 ▷ ▷ ▷ -point "0,0,0" \
28640 ▷ ▷ ▷ -point "1,0,0" \
28641 ▷ ▷ ▷ -point "1,2,0" \
28642 ▷ ▷ ▷ -point "1,2,3" \

```

```

28643 ▷ ▷ ▷ -edge "84,85" \
28644 ▷ ▷ ▷ -edge "85,86" \
28645 ▷ ▷ ▷ -edge "86,87" \
28646 ▷ ▷ ▷ -edge "84,87" \
28647 ▷ ▷ ▷ -group_of_things "84,85,86" \
28648 ▷ ▷ ▷ -spheres 7 0.1 $(POLISHED_CHROME_WHITE) \
28649 ▷ ▷ ▷ -group_of_things "87" \
28650 ▷ ▷ ▷ -spheres 8 0.10 $(COLOR_YELLOW_SHINY) \
28651 ▷ ▷ ▷ -group_of_things "0,1,2" \
28652 ▷ ▷ ▷ -cylinders 9 0.075 $(POLISHED_CHROME_WHITE) \
28653 ▷ ▷ ▷ -group_of_things "3" \
28654 ▷ ▷ ▷ -cylinders 10 0.075 $(COLOR_YELLOW_SHINY) \
28655 ▷ ▷ -scene_objects_end \
28656 ▷ ▷ -povray_end
28657 ▷ - rm -rf POV
28658 ▷ mkdir POV
28659 ▷ mv analytic_geo.1_0_*.pov POV
28660 ▷ mv makefile_animation POV
28661
28662
28663 analytic_geo.1_video:
28664 ▷ - rm -r FRAMES
28665 ▷ - mkdir FRAMES
28666 ▷ - rm analytic_geo.1.mp4
28667 ▷ $(ORBITER) \
28668 ▷ ▷ -prepare_frames \
28669 ▷ ▷ ▷ -i 0 30 PNG/ANALYTIC_GEO.1/analytic_geo.1_0_%03d.png \
28670 ▷ ▷ ▷ -output_starts_at 0 \
28671 ▷ ▷ ▷ -o FRAMES/frame%04d.png \
28672 ▷ ▷ -end
28673 ▷ ffmpeg -r 5 -f image2 -i FRAMES/frame%04d.png \
28674 ▷ ▷ -f mp4 -q:v 0 -vcodec mpeg4 analytic_geo.1.mp4
28675
28676 monkey:
28677 ▷ $(ORBITER) -v 2 -povray \
28678 ▷ ▷ -round 0 -nb_frames_default 30 \
28679 ▷ ▷ -output_mask monkey_%d_%03d.pov \
28680 ▷ ▷ -video_options -W 1024 -H 768 \
28681 ▷ ▷ -global_picture_scale 0.8 \
28682 ▷ ▷ -default_angle 75 \
28683 ▷ ▷ -clipping_radius 0.8 \
28684 ▷ ▷ -camera 0 "0,0,1" "1,1,0.5" "0,0,0" \
28685 ▷ ▷ -rotate_about_z_axis \
28686 ▷ ▷ -end \
28687 ▷ ▷ -scene_objects \
28688 ▷ ▷ ▷ -cubic_lex $(MONKEY_SADDLE_CUBIC) \
28689 ▷ ▷ ▷ -plane_by_dual_coordinates "0,0,1,0" \
28690 ▷ ▷ ▷ -group_of_things "0" \
28691 ▷ ▷ ▷ -group_of_things "0" \
28692 ▷ ▷ ▷ -cubics 0 $(COLOR_GOLD) \
28693 ▷ ▷ ▷ -planes 1 $(COLOR_BLUE) \
28694 ▷ ▷ -scene_objects_end \
28695 ▷ ▷ -povray_end
28696 ▷ - rm -rf POV
28697 ▷ mkdir POV
28698 ▷ mv monkey_0_*.pov POV
28699 ▷ mv makefile_animation POV
28700
28701

```



```

28702
28703
28704 Eckardt:
28705 ▷ $(ORBITER) -v 2 -povray \
28706 ▷ ▷ -round 0 -nb.frames.default 30 \
28707 ▷ ▷ -output_mask Eckardt.%d.%03d.pov \
28708 ▷ ▷ -video_options -W 1024 -H 768 \
28709 ▷ ▷ -global_picture_scale 0.9 \
28710 ▷ ▷ -default_angle 75 \
28711 ▷ ▷ -clipping_radius 2.4 \
28712 ▷ ▷ -camera 0 "1,1,1" "-3,1,3" "0.12,0.12,0.12" \
28713 ▷ ▷ -end \
28714 ▷ ▷ -scene_objects \
28715 ▷ ▷ ▷ -Hilbert_Cohn_Vossen_surface \
28716 ▷ ▷ ▷ -group_of_things "0" \
28717 ▷ ▷ ▷ -cubics 0 $(SURFACE_COLOR) \
28718 ▷ ▷ ▷ -group_of_things.as.interval 0 6 \
28719 ▷ ▷ ▷ -group_of_things.as.interval 6 6 \
28720 ▷ ▷ ▷ -group_of_things.as.interval.with.exceptions 12 15 \
28721 ▷ ▷ ▷ ▷ "14,19,23" \
28722 ▷ ▷ ▷ -lines 1 0.02 $(COLOR_RED_SHINY) \
28723 ▷ ▷ ▷ -lines 2 0.02 $(COLOR_BLUE_SHINY) \
28724 ▷ ▷ ▷ -lines 3 0.02 $(COLOR_YELLOW_SHINY) \
28725 ▷ ▷ ▷ -label 0 "a1" \
28726 ▷ ▷ ▷ -label 2 "a2" \
28727 ▷ ▷ ▷ -label 4 "a3" \
28728 ▷ ▷ ▷ -label 6 "a4" \
28729 ▷ ▷ ▷ -label 8 "a5" \
28730 ▷ ▷ ▷ -label 10 "a6" \
28731 ▷ ▷ ▷ -label 12 "b1" \
28732 ▷ ▷ ▷ -label 14 "b2" \
28733 ▷ ▷ ▷ -label 16 "b3" \
28734 ▷ ▷ ▷ -label 18 "b4" \
28735 ▷ ▷ ▷ -label 20 "b5" \
28736 ▷ ▷ ▷ -label 22 "b6" \
28737 ▷ ▷ ▷ -label 24 "c12" \
28738 ▷ ▷ ▷ -label 26 "c13" \
28739 ▷ ▷ ▷ -label 30 "c15" \
28740 ▷ ▷ ▷ -label 32 "c16" \
28741 ▷ ▷ ▷ -label 34 "c23" \
28742 ▷ ▷ ▷ -label 36 "c24" \
28743 ▷ ▷ ▷ -label 40 "c26" \
28744 ▷ ▷ ▷ -label 42 "c34" \
28745 ▷ ▷ ▷ -label 44 "c35" \
28746 ▷ ▷ ▷ -label 48 "c45" \
28747 ▷ ▷ ▷ -label 50 "c46" \
28748 ▷ ▷ ▷ -label 52 "c56" \
28749 ▷ ▷ ▷ -group_of_things.as.interval 0 6 \
28750 ▷ ▷ ▷ -texts 4 0.2 0.15 $(COLOR_BLACK_NO_SHADOW) \
28751 ▷ ▷ ▷ -group_of_things.as.interval 6 6 \
28752 ▷ ▷ ▷ -texts 5 0.2 0.15 $(COLOR_BLACK_NO_SHADOW) \
28753 ▷ ▷ ▷ -group_of_things.as.interval 12 12 \
28754 ▷ ▷ ▷ -texts 6 0.2 0.15 $(COLOR_BLACK_NO_SHADOW) \
28755 ▷ ▷ -scene_objects_end \
28756 ▷ ▷ -povray_end
28757 ▷ - rm -rf POV
28758 ▷ mkdir POV
28759 ▷ mv Eckardt_0*.pov POV
28760 ▷ mv makefile_animation POV

```

```

28761
28762
28763 #"-3,2.333,4" * 1.5 = "-4.5,3.5,6"
28764 #M := Matrix([[ -4.5, 3.5, 6], [1, 1, 1]])
28765 #NullSpace(M)
28766 #=0.186080731891197,-0.781539073943026,0.595458342051830
28767 #> > -rotate_about_custom_axis "0.186080731891197,-0.781539073943026,0.5954583420
51830" \
28768 #-W 1024 -H 768
28769 #-W 2560 -H 1920
28770 #-W 4096 -H 3072
28771
28772
28773
28774
28775 Eckardt_deform:
28776 > $(ORBITER) -v 2 -povray \
28777 > > -round 0 -nb_frames_default 93 \
28778 > > -output_mask Eckardt_deform.%d.%03d.pov \
28779 > > -video_options -W 1024 -H 768 \
28780 > > -global_picture_scale 0.9 \
28781 > > -default_angle 75 \
28782 > > -clipping_radius 2.4 \
28783 > > -camera 0 "1,1,1" "-3,1,3" "0.12,0.12,0.12" \
28784 > > -end \
28785 > > -scene_objects \
28786 > > > -Hilbert_Cohn_Vossen_surface \
28787 > > > -group_of_things "0" \
28788 > > > -deformation_of_cubic_lex 93 1.107148718 1.570796327 0 \
28789 > > > > $(ECKARDT_CUBIC_DEFORM1_LEX) \
28790 > > > > $(ECKARDT_CUBIC_DEFORM2_LEX) \
28791 > > > -group_of_things_as_interval 0 93 \
28792 > > > -group_is_animated 1 \
28793 > > > -cubics 1 $(SURFACE_COLOR_SEETHROUGH) \
28794 > > -scene_objects_end \
28795 > > -povray_end
28796 > - rm -rf POV
28797 > mkdir POV
28798 > mv Eckardt_deform_0_*.pov POV
28799 > mv makefile_animation POV
28800
28801
28802
28803
28804
28805 Eckardt_deform.2:
28806 > $(ORBITER) -v 2 -povray \
28807 > > -round 0 -nb_frames_default 30 \
28808 > > -output_mask Eckardt_deform.%d.%03d.pov \
28809 > > -video_options -W 1024 -H 768 \
28810 > > -global_picture_scale 0.9 \
28811 > > -default_angle 75 \
28812 > > -clipping_radius 2.4 \
28813 > > -camera 0 "1,1,1" "-3,1,3" "0.12,0.12,0.12" \
28814 > > -end \
28815 > > -scene_objects \
28816 > > > -Hilbert_Cohn_Vossen_surface \
28817 > > > -group_of_things "0" \
28818 > > > -deformation_of_cubic_lex 93 1.107148718 1.570796327 0 \

```

```

28819 ▷ ▷ ▷ ▷ $(ECKARDT_CUBIC_DEFORM1_LEX) \
28820 ▷ ▷ ▷ ▷ $(ECKARDT_CUBIC_DEFORM2_LEX) \
28821 ▷ ▷ ▷ --group_of_things_as_interval 0 93 \
28822 ▷ ▷ ▷ --group_is_animated 1 \
28823 ▷ ▷ ▷ -group_of_things "0" \
28824 ▷ ▷ ▷ -cubics 1 $(SURFACE_COLOR_SEETHROUGH) \
28825 ▷ ▷ ▷ -group_of_things "24" \
28826 ▷ ▷ ▷ -cubics 2 $(COLOR_RED) \
28827 ▷ ▷ ▷ -group_of_things "70" \
28828 ▷ ▷ ▷ -cubics 3 $(COLOR_BLUE) \
28829 ▷ ▷ -scene_objects_end \
28830 ▷ ▷ -povray_end
28831 ▷ - rm -rf POV
28832 ▷ mkdir POV
28833 ▷ mv Eckardt_deform_0_*.pov POV
28834 ▷ mv makefile_animation POV
28835
28836
28837
28838 Clebsch:
28839 ▷ $(ORBITER) -v 2 -povray \
28840 ▷ ▷ -round 0 -nb_frames_default 30 \
28841 ▷ ▷ -output_mask Clebsch.%d.%03d.pov \
28842 ▷ ▷ -video_options -W 1024 -H 768 \
28843 ▷ ▷ -global_picture_scale 0.9 \
28844 ▷ ▷ -default_angle 80 \
28845 ▷ ▷ -clipping_radius 2.4 \
28846 ▷ ▷ -camera 0 "1,1,1" "-4.5,3.5,6" "0,0,0" \
28847 ▷ ▷ -end \
28848 ▷ ▷ -scene_objects \
28849 ▷ ▷ ▷ -Clebsch_surface \
28850 ▷ ▷ ▷ -group_of_things "0" \
28851 ▷ ▷ ▷ -cubics 0 $(SURFACE_COLOR) \
28852 ▷ ▷ ▷ -group_of_things_as_interval 0 6 \
28853 ▷ ▷ ▷ -group_of_things_as_interval 6 6 \
28854 ▷ ▷ ▷ -group_of_things_as_interval 12 15 \
28855 ▷ ▷ ▷ -lines 1 0.02 $(COLOR_RED_SHINY) \
28856 ▷ ▷ ▷ -lines 2 0.02 $(COLOR_BLUE_SHINY) \
28857 ▷ ▷ ▷ -lines 3 0.02 $(COLOR_YELLOW_SHINY) \
28858 ▷ ▷ ▷ -group_of_things_as_interval 0 12 \
28859 ▷ ▷ ▷ -spheres 4 0.08 $(COLOR_TURQUOISE) \
28860 ▷ ▷ -scene_objects_end \
28861 ▷ ▷ -povray_end
28862 ▷ - rm -rf POV
28863 ▷ mkdir POV
28864 ▷ mv Clebsch_0_*.pov POV
28865 ▷ mv makefile_animation POV
28866
28867
28868
28869 endrass8:
28870 ▷ cp $(ORBITER_PATH)/examples/users_guide/coordinate_grid.csv .
28871 ▷ $(ORBITER) -v 2 -povray \
28872 ▷ ▷ -round 0 -nb_frames_default 30 \
28873 ▷ ▷ -output_mask endrass8.octic.%d.%03d.pov \
28874 ▷ ▷ -video_options -W 1024 -H 768 \
28875 ▷ ▷ -global_picture_scale 0.75 \
28876 ▷ ▷ -default_angle 75 \
28877 ▷ ▷ -clipping_radius 3.7 \

```

```

28878 ▷ ▷ -no_bottom_plane 0 \
28879 ▷ ▷ -camera 0 "1,1,1" "6,6,3" "0,0,0" \
28880 ▷ ▷ -rotate_about_111 \
28881 ▷ ▷ -end \
28882 ▷ ▷ -scene_objects \
28883 ▷ ▷ ▷ -line_through_two_points_recentered_from_csv_file \
28884 ▷ ▷ ▷ ▷ coordinate_grid.csv \
28885 ▷ ▷ ▷ -group_of_things "0" \
28886 ▷ ▷ ▷ -group_of_things "1" \
28887 ▷ ▷ ▷ -group_of_things "2" \
28888 ▷ ▷ ▷ -group_of_things_as_interval 3 39 \
28889 ▷ ▷ ▷ -lines 0 0.15 $(COLOR_RED_SHINY) \
28890 ▷ ▷ ▷ -lines 1 0.15 $(COLOR_GREEN_SHINY) \
28891 ▷ ▷ ▷ -lines 2 0.15 $(COLOR_BLUE_SHINY) \
28892 ▷ ▷ ▷ -lines 3 0.05 $(COLOR_BLACK_SHINY) \
28893 ▷ ▷ ▷ -octic_lex_165 $(ENDRASS_OCTIC_LEX_165) \
28894 ▷ ▷ ▷ -plane_by_dual_coordinates "0,0,1,0" \
28895 ▷ ▷ ▷ -group_of_things "0" \
28896 ▷ ▷ ▷ -group_of_things "0" \
28897 ▷ ▷ ▷ -octics 4 $(SURFACE_COLOR_SEETHROUGH) \
28898 ▷ ▷ ▷ -planes 5 "texture{ pigment{ color Blue transmit 0.5 } } \
28899 finish { diffuse 0.9 phong 1} }" \
28900 ▷ ▷ -scene_objects_end \
28901 ▷ ▷ -povray_end
28902 ▷ - rm -rf POV
28903 ▷ mkdir POV
28904 ▷ mv endrass_octic_0_*.pov POV
28905 ▷ mv makefile_animation POV
28906
28907
28908
28909
28910
28911 #####
28912 # Section 19.3: Creating Animations
28913
28914 SECTION_ANIMATIONS:
28915
28916
28917 test_19_3:
28918 ▷ make dode
28919 ▷ #make dode_video # need ffmpeg
28920 ▷ #make monkey_video
28921 ▷ #make Eckardt_deform_video
28922 ▷ make Eckardt_surface
28923 ▷ make Kummer_surface
28924 ▷ #make Kummer_video
28925 ▷ make Beauville_surface
28926 ▷ make Clebsch_up_create_points
28927 ▷ #make Clebsch_surface # needs function.Clebsch_map_of_circle_N1000_points.csv
28928 ▷ make Clebsch_surface_defining_equation
28929 ▷ #make Clebsch_surface_defining_equation_and_curves # needs function.Clebsch_map
    _of_circle_N1000_points.csv
28930 ▷ make F7_povray
28931 ▷ #make F7_video
28932 ▷ make McKean_povray
28933 ▷ #make McKean_video
28934
28935

```

```

28936
28937 dode:
28938 ▷ $(ORBITER) -v 2 \
28939 ▷ ▷ -povray \
28940 ▷ ▷ -round 0 -nb.frames.default 30 \
28941 ▷ ▷ -output_mask dode_%d.%03d.pov \
28942 ▷ ▷ -video_options -W 1024 -H 768 \
28943 ▷ ▷ -global_picture_scale 0.50 \
28944 ▷ ▷ -default_angle 45 \
28945 ▷ ▷ -clipping_radius 5 \
28946 ▷ ▷ -camera 0 "1,1,1" "-2,2,4" "0,0,0" \
28947 ▷ ▷ -rotate_about_111 \
28948 ▷ ▷ -end \
28949 ▷ ▷ -scene_objects \
28950 ▷ ▷ ▷ -dodecahedron \
28951 ▷ ▷ ▷ -group_of_things.as.interval 0 20 \
28952 ▷ ▷ ▷ -spheres 0 0.075 $(POLISHED_CHROME_WHITE) \
28953 ▷ ▷ ▷ -group_of_things.as.interval 0 30 \
28954 ▷ ▷ ▷ -cylinders 1 0.05 $(COLOR_RED_SHINY) \
28955 ▷ ▷ ▷ -group_of_things.as.interval 0 12 \
28956 ▷ ▷ ▷ -prisms 2 0.02 $(YELLOW_TRANSPARENT) \
28957 ▷ ▷ -scene_objects_end \
28958 ▷ ▷ -povray_end
28959 ▷ - rm -rf POV
28960 ▷ mkdir POV
28961 ▷ mv dode_0_*.pov POV
28962 ▷ mv makefile_animation POV
28963
28964
28965 dode_video:
28966 ▷ - rm -r FRAMES
28967 ▷ - mkdir FRAMES
28968 ▷ - rm dode.mp4
28969 ▷ $(ORBITER) \
28970 ▷ ▷ -prepare_frames \
28971 ▷ ▷ ▷ -i 0 30 DODE/dode_0_%03d.png \
28972 ▷ ▷ ▷ -output_starts_at 0 \
28973 ▷ ▷ ▷ -o FRAMES/frame%04d.png \
28974 ▷ ▷ -end
28975 ▷ ffmpeg -r 5 -f image2 -i FRAMES/frame%04d.png \
28976 ▷ ▷ -f mp4 -q:v 0 -vcodec mpeg4 dode.mp4
28977
28978
28979 monkey_video:
28980 ▷ - rm -r FRAMES
28981 ▷ - mkdir FRAMES
28982 ▷ - rm monkey.mp4
28983 ▷ $(ORBITER) \
28984 ▷ ▷ -prepare_frames \
28985 ▷ ▷ ▷ -i 0 30 monkey_0_%03d.png \
28986 ▷ ▷ ▷ -output_starts_at 0 \
28987 ▷ ▷ ▷ -o FRAMES/frame%04d.png \
28988 ▷ ▷ -end
28989 ▷ ffmpeg -r 5 -f image2 -i FRAMES/frame%04d.png \
28990 ▷ ▷ -f mp4 -q:v 0 -vcodec mpeg4 monkey.mp4
28991
28992 Eckardt_deform_video:
28993 ▷ - rm -r FRAMES
28994 ▷ - mkdir FRAMES

```

```

28995 ▷ - rm Eckardt_deform.mp4
28996 ▷ $(ORBITER) \
28997 ▷ ▷ -prepare_frames \
28998 ▷ ▷ ▷ -i 0 93 Eckardt_deform_0/Eckardt_deform_0_%03d.png \
28999 ▷ ▷ ▷ -output_starts_at 0 \
29000 ▷ ▷ ▷ -o FRAMES/frame%04d.png \
29001 ▷ ▷ -end
29002 ▷ ffmpeg -r 5 -f image2 -i FRAMES/frame%04d.png \
29003 ▷ ▷ -f mp4 -q:v 0 -vcodec mpeg4 Eckardt_deform.mp4
29004
29005
29006 Eckardt_surface:
29007 ▷ $(ORBITER) -v 2 -povray \
29008 ▷ ▷ -round 0 -nb_frames_default 30 \
29009 ▷ ▷ -output_mask Eckardt_%d_%03d.pov \
29010 ▷ ▷ -video_options -W 1024 -H 768 \
29011 ▷ ▷ -global_picture_scale 0.9 \
29012 ▷ ▷ -default_angle 75 \
29013 ▷ ▷ -clipping_radius 2.4 \
29014 ▷ ▷ -camera 0 "1,1,1" "-3,1,3" "0.12,0.12,0.12" \
29015 ▷ ▷ -end \
29016 ▷ ▷ -scene_objects \
29017 ▷ ▷ ▷ -cubic_Goursat "6,3,-15" \
29018 ▷ ▷ ▷ -group_of_things "0" \
29019 ▷ ▷ ▷ -cubics 0 $(SURFACE_COLOR_SEETHROUGH) \
29020 ▷ ▷ -scene_objects_end \
29021 ▷ ▷ -povray_end
29022 ▷ - rm -rf POV
29023 ▷ mkdir POV
29024 ▷ mv Eckardt_0*.pov POV
29025 ▷ mv makefile_animation POV
29026
29027
29028
29029 Kummer_surface:
29030 ▷ $(ORBITER) -v 2 -povray \
29031 ▷ ▷ -round 0 -nb_frames_default 30 \
29032 ▷ ▷ -output_mask Kummer_%d_%03d.pov \
29033 ▷ ▷ -video_options -W 1024 -H 768 \
29034 ▷ ▷ -global_picture_scale 0.9 \
29035 ▷ ▷ -default_angle 75 \
29036 ▷ ▷ -clipping_radius 2.4 \
29037 ▷ ▷ -camera 0 "1,1,1" "-3,1,3" "0.12,0.12,0.12" \
29038 ▷ ▷ -end \
29039 ▷ ▷ -scene_objects \
29040 ▷ ▷ ▷ -quartic_lex_35 $(KUMMER_QUARTIC_LEX_35) \
29041 ▷ ▷ ▷ -group_of_things "0" \
29042 ▷ ▷ ▷ -quartics 0 $(SURFACE_COLOR_SEETHROUGH) \
29043 ▷ ▷ -scene_objects_end \
29044 ▷ ▷ -povray_end
29045 ▷ - rm -rf POV
29046 ▷ mkdir POV
29047 ▷ mv Kummer_0*.pov POV
29048 ▷ mv makefile_animation POV
29049
29050
29051 # Maple:
29052 #Kummer := expand((x0^2 + x1^2 + x2^2 + x3^2)^2 - 3*(x0^4 + x1^4 + x2^4 + x3^4))
29053

```

```

29054
29055 Kummer_video:
29056 ▷ - rm -r FRAMES
29057 ▷ - mkdir FRAMES
29058 ▷ - rm Kummer.mp4
29059 ▷ $(ORBITER) \
29060 ▷ ▷ -prepare_frames \
29061 ▷ ▷ ▷ -i 0 30 KUMMER/Kummer.0.%03d.png \
29062 ▷ ▷ ▷ -output_starts_at 0 \
29063 ▷ ▷ ▷ -o FRAMES/frame%04d.png \
29064 ▷ ▷ -end
29065 ▷ ffmpeg -r 5 -f image2 -i FRAMES/frame%04d.png \
29066 ▷ ▷ -f mp4 -q:v 0 -vcodec mpeg4 Kummer.mp4
29067
29068
29069
29070 Beauville_surface:
29071 ▷ $(ORBITER) -v 2 -povray \
29072 ▷ ▷ -round 0 -nb_frames_default 30 \
29073 ▷ ▷ -output_mask Beauville.%d.%03d.pov \
29074 ▷ ▷ -video_options -W 1024 -H 768 \
29075 ▷ ▷ -global_picture_scale 0.3 \
29076 ▷ ▷ -default_angle 75 \
29077 ▷ ▷ -clipping_radius 2.4 \
29078 ▷ ▷ -camera 0 "1,1,1" "-3,1,3" "0.12,0.12,0.12" \
29079 ▷ ▷ -end \
29080 ▷ ▷ -scene_objects \
29081 ▷ ▷ ▷ -quintic_lex.56 $(BEAUVILLE_QUINTIC_LEX.56) \
29082 ▷ ▷ ▷ -group_of_things "0" \
29083 ▷ ▷ ▷ -quintics 0 $(SURFACE_COLOR_SEETHROUGH) \
29084 ▷ ▷ -scene_objects_end \
29085 ▷ ▷ -povray_end
29086 ▷ - rm -rf POV
29087 ▷ mkdir POV
29088 ▷ mv Beauville.0*.pov POV
29089 ▷ mv makefile_animation POV
29090
29091
29092
29093
29094
29095
29096
29097
29098 Clebsch_up_create_points:
29099 ▷ $(ORBITER) -v 2 \
29100 ▷ ▷ -smooth_curve "Clebsch_map_of_circle_to_defininig_eqn.r2" \
29101 ▷ ▷ ▷ 0.07 1000 5 0 $(TWO_PI) \
29102 ▷ ▷ -const a $(CLEBSCH_A) b $(CLEBSCH_B) c $(CLEBSCH_C) d $(CLEBSCH_D) \
29103 ▷ ▷ ▷ t00 $(T00) t01 $(T01) t02 $(T02) t03 $(T03) \
29104 ▷ ▷ ▷ t10 $(T10) t11 $(T11) t12 $(T12) t13 $(T13) \
29105 ▷ ▷ ▷ t20 $(T20) t21 $(T21) t22 $(T22) t23 $(T23) \
29106 ▷ ▷ ▷ t30 $(T30) t31 $(T31) t32 $(T32) t33 $(T33) \
29107 ▷ ▷ ▷ r 2 one 1 m -1 \
29108 ▷ ▷ -const_end \
29109 ▷ ▷ -var t \
29110 ▷ ▷ ▷ c001 c002 c011 c012 \
29111 ▷ ▷ ▷ d001 d011 d012 d112 \
29112 ▷ ▷ ▷ m002 m012 m022 m122 \

```

```

29113 ▷ ▷ ▷ n002 n012 n112 n022 n122 \
29114 ▷ ▷ ▷ y0 y1 y2 \
29115 ▷ ▷ ▷ y001 y002 y011 y012 y022 y112 y122 \
29116 ▷ ▷ ▷ x0 x1 x2 x3 \
29117 ▷ ▷ ▷ -var_end \
29118 ▷ ▷ ▷ -code \
29119 ▷ ▷ ▷ push t cos push r mult store y0 \
29120 ▷ ▷ ▷ push t sin push r mult store y1 \
29121 ▷ ▷ ▷ push one store y2 \
29122 ▷ ▷ ▷ push y0 push y0 push y1 mult mult store y001 \
29123 ▷ ▷ ▷ push y0 push y0 push y2 mult mult store y002 \
29124 ▷ ▷ ▷ push y0 push y1 push y1 mult mult store y011 \
29125 ▷ ▷ ▷ push y0 push y1 push y2 mult mult store y012 \
29126 ▷ ▷ ▷ push y0 push y2 push y2 mult mult store y022 \
29127 ▷ ▷ ▷ push y1 push y1 push y2 mult mult store y112 \
29128 ▷ ▷ ▷ push y1 push y2 push y2 mult mult store y122 \
29129 ▷ ▷ ▷ $(CLEBSCH_CUBICS) \
29130 ▷ ▷ ▷ push c001 push y001 mult \
29131 ▷ ▷ ▷ push c002 push y002 mult add \
29132 ▷ ▷ ▷ push c011 push y011 mult add \
29133 ▷ ▷ ▷ push c012 push y012 mult add \
29134 ▷ ▷ ▷ store x0 \
29135 ▷ ▷ ▷ push d001 push y001 mult \
29136 ▷ ▷ ▷ push d011 push y011 mult add \
29137 ▷ ▷ ▷ push d012 push y012 mult add \
29138 ▷ ▷ ▷ push d112 push y112 mult add \
29139 ▷ ▷ ▷ store x1 \
29140 ▷ ▷ ▷ push m002 push y002 mult \
29141 ▷ ▷ ▷ push m012 push y012 mult add \
29142 ▷ ▷ ▷ push m022 push y022 mult add \
29143 ▷ ▷ ▷ push m122 push y122 mult add \
29144 ▷ ▷ ▷ store x2 \
29145 ▷ ▷ ▷ push n002 push y002 mult \
29146 ▷ ▷ ▷ push n012 push y012 mult add \
29147 ▷ ▷ ▷ push n022 push y022 mult add \
29148 ▷ ▷ ▷ push n112 push y112 mult add \
29149 ▷ ▷ ▷ push n122 push y122 mult add \
29150 ▷ ▷ ▷ store x3 \
29151 ▷ ▷ ▷ push x0 push t00 mult \
29152 ▷ ▷ ▷ push x1 push t10 mult add \
29153 ▷ ▷ ▷ push x2 push t20 mult add \
29154 ▷ ▷ ▷ push x3 push t30 mult add \
29155 ▷ ▷ ▷ return \
29156 ▷ ▷ ▷ push x0 push t01 mult \
29157 ▷ ▷ ▷ push x1 push t11 mult add \
29158 ▷ ▷ ▷ push x2 push t21 mult add \
29159 ▷ ▷ ▷ push x3 push t31 mult add \
29160 ▷ ▷ ▷ return \
29161 ▷ ▷ ▷ push x0 push t02 mult \
29162 ▷ ▷ ▷ push x1 push t12 mult add \
29163 ▷ ▷ ▷ push x2 push t22 mult add \
29164 ▷ ▷ ▷ push x3 push t32 mult add \
29165 ▷ ▷ ▷ return \
29166 ▷ ▷ ▷ push x0 push t03 mult \
29167 ▷ ▷ ▷ push x1 push t13 mult add \
29168 ▷ ▷ ▷ push x2 push t23 mult add \
29169 ▷ ▷ ▷ push x3 push t33 mult add \
29170 ▷ ▷ ▷ return \
29171 ▷ ▷ ▷ -code_end

```



```

29172
29173
29174 Clebsch_surface:
29175 ▷ $(ORBITER) -v 2 -povray \
29176 ▷ ▷ -round 0 -nb_frames_default 30 \
29177 ▷ ▷ -output_mask Clebsch.%d.%03d.pov \
29178 ▷ ▷ -video_options -W 1024 -H 768 \
29179 ▷ ▷ -global_picture_scale 0.9 \
29180 ▷ ▷ -default_angle 75 \
29181 ▷ ▷ -clipping_radius 2.4 \
29182 ▷ ▷ -camera 0 "1,1,1" "-3,1,3" "0.12,0.12,0.12" \
29183 ▷ ▷ -end \
29184 ▷ ▷ -scene_objects \
29185 ▷ ▷ ▷ -cubic_orbiter "0,0,0,0,0,-4.236067972,\
29186 0,0,4.236067972,4.236067972,17.94427188,\
29187 -17.94427188,0,0,- 9.472135941,0,0,5.236067971,\
29188 8.472135938,- 27.41640782" \
29189 ▷ ▷ ▷ -group_of_things "0" \
29190 ▷ ▷ ▷ -cubics 0 $(SURFACE_COLOR_SEETHROUGH) \
29191 ▷ ▷ ▷ -point_list_from_csv_file \
29192 ▷ ▷ ▷ function_Clebsch_map_of_circle_N1000_points.csv \
29193 ▷ ▷ ▷ -group_of_things_as_interval 0 954 \
29194 ▷ ▷ ▷ -spheres 1 0.07 $(COLOR_RED) \
29195 ▷ ▷ -scene_objects_end \
29196 ▷ ▷ -povray_end
29197 ▷ - rm -rf POV
29198 ▷ mkdir POV
29199 ▷ mv Clebsch_0*.pov POV
29200 ▷ mv makefile_animation POV
29201
29202 #needs: function_Clebsch_map_of_circle_N1000_points.csv
29203
29204
29205
29206 Clebsch_surface_defining_equation:
29207 ▷ $(ORBITER) -v 2 -povray \
29208 ▷ ▷ -round 0 -nb_frames_default 30 \
29209 ▷ ▷ -output_mask Clebsch.%d.%03d.pov \
29210 ▷ ▷ -video_options -W 1024 -H 768 \
29211 ▷ ▷ -global_picture_scale 0.6 \
29212 ▷ ▷ -default_angle 75 \
29213 ▷ ▷ -clipping_radius 1.6 \
29214 ▷ ▷ -camera 0 "1,1,1" "-2,0,2" "0,0,0" \
29215 ▷ ▷ -end \
29216 ▷ ▷ -scene_objects \
29217 ▷ ▷ ▷ -cubic_orbiter "0,0,0,0,1,1,1,1,1,1,1,1,1,1,2,2,2" \
29218 ▷ ▷ ▷ -group_of_things "0" \
29219 ▷ ▷ ▷ -cubics 0 $(SURFACE_COLOR_SEETHROUGH) \
29220 ▷ ▷ -scene_objects_end \
29221 ▷ ▷ -povray_end
29222 ▷ - rm -rf POV
29223 ▷ mkdir POV
29224 ▷ mv Clebsch_0*.pov POV
29225 ▷ mv makefile_animation POV
29226
29227
29228
29229 Clebsch_surface_defining_equation_and_curves:
29230 ▷ $(ORBITER) -v 2 -povray \

```

```

29231 ▷ ▷ -round 0 -nb_frames_default 30 \
29232 ▷ ▷ -output_mask Clebsch_2curves_%d%03d.pov \
29233 ▷ ▷ -video_options -W 1024 -H 768 \
29234 ▷ ▷ -global_picture_scale 0.6 \
29235 ▷ ▷ -default_angle 75 \
29236 ▷ ▷ -clipping_radius 1.6 \
29237 ▷ ▷ -camera 0 "1,1,1" "-2,0,2" "0,0,0" \
29238 ▷ ▷ -end \
29239 ▷ ▷ -scene_objects \
29240 ▷ ▷ ▷ -cubic_orbiter "0,0,0,0,1,1,1,1,1,1,1,1,1,1,1,1,2,2,2" \
29241 ▷ ▷ ▷ -group_of_things "0" \
29242 ▷ ▷ ▷ -cubics 0 $(SURFACE_COLOR_SEETHROUGH) \
29243 ▷ ▷ ▷ -point_list_from_csv_file \
29244 ▷ ▷ ▷ function_Clebsch_map_of_circle_to_defininig_eqn_N1000_points.csv \
29245 ▷ ▷ ▷ -group_of_things_as_interval 0 656 \
29246 ▷ ▷ ▷ -spheres 1 0.07 $(COLOR_RED) \
29247 ▷ ▷ ▷ -point_list_from_csv_file \
29248 ▷ ▷ ▷ function_Clebsch_map_of_circle_to_defininig_eqn_r2_N1000_points.csv \
29249 ▷ ▷ ▷ -group_of_things_as_interval 656 1042 \
29250 ▷ ▷ ▷ -spheres 2 0.07 "texture{ pigment{ color Blue } }" \
29251 finish { diffuse 0.9 phong 1}}" \
29252 ▷ ▷ -scene_objects_end \
29253 ▷ ▷ -povray_end
29254 ▷ - rm -rf POV
29255 ▷ mkdir POV
29256 ▷ mv Clebsch_2curves_0_*.pov POV
29257 ▷ mv makefile_animation POV
29258
29259 #▷ ▷ ▷ -point_list_from_csv_file function_Clebsch_map_of_circle_N1000_points.csv
\
29260 #▷ ▷ ▷ -group_of_things_as_interval 0 954 \
29261 #▷ ▷ ▷ -spheres 1 0.07 "texture{ pigment{ color Red } finish { diffuse 0.9 phong
1}}" \
29262
29263
29264
29265
29266 F7_povray:
29267 ▷ $(ORBITER) -v 2 -povray \
29268 ▷ ▷ -round 0 -nb_frames_default 30 \
29269 ▷ ▷ -output_mask F7_15_lines_%d%03d.pov \
29270 ▷ ▷ -video_options -W 1024 -H 768 \
29271 ▷ ▷ -global_picture_scale 1.5 \
29272 ▷ ▷ -default_angle 80 \
29273 ▷ ▷ -clipping_radius 4.4 \
29274 ▷ ▷ -omit_bottom_plane \
29275 ▷ ▷ -camera 0 "1,1,1" "-4.5,3.5,6" "0,0,0" \
29276 ▷ ▷ -end \
29277 ▷ ▷ -scene_objects \
29278 ▷ ▷ ▷ -cubic_lex "0, 0, 6, 0, 0, -13.39014946, -3.341901346, -6.972931640, 5.8271
82718, 0, 0, 7.390149464, 7.390149464, 6.972931640, -1.512349728, -8.485281372, 0
, 0, 0, 0" \
29279 ▷ ▷ ▷ -group_of_things "0" \
29280 ▷ ▷ ▷ -cubics 0 $(SURFACE_COLOR_SEETHROUGH) \
29281 ▷ ▷ ▷ -line_through_point_with_direction "0, 0, 0, 1, 0, 0" \
29282 ▷ ▷ ▷ -line_through_point_with_direction "0, 0, -1, 0, 1, 0" \
29283 ▷ ▷ ▷ -line_through_point_with_direction "0, 0, 0, 0, 0, -1" \
29284 ▷ ▷ ▷ -line_through_point_with_direction "1, 0, 0, 1, 1, 1" \
29285 ▷ ▷ ▷ -line_through_point_with_direction "-1.414213562, 0, 0, 4.146264370, 1.7320

```

```

50808, 1.732050808" \
29286 ▷ ▷ ▷ -line_through_point_with_direction "0, 1.732050808, -1, 2.414213562, -0.317
837246, 2.414213562" \
29287 ▷ ▷ ▷ -line_through_point_with_direction "-2.133352390, 0, -1, 1.674708020, 1, 0"
\
29288 ▷ ▷ ▷ -line_through_point_with_direction "-2.539058015, 0, 0, 2.211360755, 1, 0"
\
29289 ▷ ▷ ▷ -line_through_point_with_direction "0, 1.148188060, 0, 0, -0.9435440612, 1"
\
29290 ▷ ▷ ▷ -line_through_point_with_direction "-0.9711971171, 0, 0, 1.162155272, 0, 1"
\
29291 ▷ ▷ ▷ -line_through_point_with_direction "2.096037870, 2.096037870, 0, -1.0658519
05, -1.065851905, 1" \
29292 ▷ ▷ ▷ -line_through_point_with_direction "3.921555783, 2.921555781, 0, -1.7224565
85, -1.722456585, 1" \
29293 ▷ ▷ ▷ -group_of_things_as_interval 0 12 \
29294 ▷ ▷ ▷ -lines 1 0.04 $(COLOR_YELLOW) \
29295 ▷ ▷ ▷ -scene_objects_end \
29296 ▷ ▷ ▷ -povray_end
29297 ▷ - rm -rf POV
29298 ▷ mkdir POV
29299 ▷ mv F7_15_lines_0*.pov POV
29300 ▷ mv makefile_animation POV
29301
29302
29303
29304
29305
29306
29307 F7_video:
29308 ▷ - rm -r FRAMES
29309 ▷ - mkdir FRAMES
29310 ▷ - rm fifteen_with_lines.mp4
29311 ▷ $(ORBITER) \
29312 ▷ ▷ -prepare_frames \
29313 ▷ ▷ ▷ -i 0 30 F7b/F7_15_lines_0_%03d.png \
29314 ▷ ▷ ▷ -output_starts_at 0 \
29315 ▷ ▷ ▷ -o FRAMES/frame%04d.png \
29316 ▷ ▷ -end
29317 ▷ ffmpeg -r 5 -f image2 -i FRAMES/frame%04d.png \
29318 ▷ ▷ -f mp4 -q:v 0 -vcodec mpeg4 fifteen_with_lines.mp4
29319
29320
29321 McKean_povray:
29322 ▷ $(ORBITER) -v 2 -povray \
29323 ▷ ▷ -round 0 -nb_frames_default 30 \
29324 ▷ ▷ -output_mask McKean.%d.%03d.pov \
29325 ▷ ▷ -video_options -W 1024 -H 768 \
29326 ▷ ▷ -global_picture_scale 1.5 \
29327 ▷ ▷ -default_angle 80 \
29328 ▷ ▷ -clipping_radius 4.4 \
29329 ▷ ▷ -omit_bottom_plane \
29330 ▷ ▷ -camera 0 "1,1,1" "-4.5,3.5,6" "0,0,0" \
29331 ▷ ▷ -end \
29332 ▷ ▷ -scene_objects \
29333 ▷ ▷ ▷ -cubic_lex "0, 0, 1, 0, 0, -1, -2, 1, \
29334 2, 0, 0, 1, 1,-1, -1, -1, 0, 0, 0, 0" \
29335 ▷ ▷ ▷ -group_of_things "0" \
29336 ▷ ▷ ▷ -cubics 0 $(SURFACE_COLOR_SEETHROUGH) \

```

```

29337 ▷ ▷ -scene_objects_end \
29338 ▷ ▷ -povray_end
29339 ▷ - rm -rf POV
29340 ▷ mkdir POV
29341 ▷ mv McKean_0_*.pov POV
29342 ▷ mv makefile_animation POV
29343
29344 McKean_video:
29345 ▷ - rm -r FRAMES
29346 ▷ - mkdir FRAMES
29347 ▷ - rm McKean.mp4
29348 ▷ $(ORBITER) \
29349 ▷ ▷ -prepare_frames \
29350 ▷ ▷ ▷ -i 0 30 MCKEAN/McKean_0_%03d.png \
29351 ▷ ▷ ▷ -output_starts_at 0 \
29352 ▷ ▷ ▷ -o FRAMES/frame%04d.png \
29353 ▷ ▷ -end
29354 ▷ ffmpeg -r 5 -f image2 -i FRAMES/frame%04d.png \
29355 ▷ ▷ -f mp4 -q:v 0 -vcodec mpeg4 McKean.mp4
29356
29357
29358
29359
29360 #####
29361 # Section 19.4: Continuous Function Plotter
29362
29363
29364
29365 SECTION.CONTINUOUS.FUNCTION.PLOTTER:
29366
29367
29368
29369 test_19_4:
29370 ▷ make lissajous
29371 ▷ make lissajous.plot
29372 ▷ make lissajous_3d
29373 ▷ make lissajous_3d.plot
29374
29375
29376
29377
29378 lissajous:
29379 ▷ $(ORBITER) -v 2 \
29380 ▷ ▷ -smooth_curve "lissajous" 0.07 2000 15 0 18.85 \
29381 ▷ ▷ -const a 3 b 2 c 1.57 r 7 -const_end \
29382 ▷ ▷ -var t -var_end \
29383 ▷ ▷ -code \
29384 ▷ ▷ ▷ push t push a mult push c add sin push r mult return \
29385 ▷ ▷ ▷ push t push b mult sin push r mult return \
29386 ▷ ▷ -code_end \
29387
29388 #function_lissajous_N2000_points.csv
29389
29390 lissajous.plot:
29391 ▷ cp $(ORBITER_PATH)/examples/users_guide/coordinate_grid.csv .
29392 ▷ $(ORBITER) -v 2 -povray \
29393 ▷ ▷ -round 0 -nb_frames_default 1 \
29394 ▷ ▷ -output_mask lissajous_%d_%03d.pov \
29395 ▷ ▷ -video_options -W 1024 -H 768 \

```

```

29396 ▷ ▷ -global.picture_scale 0.40 \
29397 ▷ ▷ -default_angle 45 \
29398 ▷ ▷ -clipping_radius 5 \
29399 ▷ ▷ -omit_bottom_plane \
29400 ▷ ▷ -camera 0 "0,-1,0" "0,0,12" "0,0,0" \
29401 ▷ ▷ -rotate_about_z_axis \
29402 ▷ ▷ -end \
29403 ▷ ▷ -scene_objects \
29404 ▷ ▷ ▷ -line_through_two_points_recentered_from_csv_file \
29405 ▷ ▷ ▷ coordinate_grid.csv \
29406 ▷ ▷ ▷ -group_of_things "0" \
29407 ▷ ▷ ▷ -group_of_things "1" \
29408 ▷ ▷ ▷ -group_of_things "2" \
29409 ▷ ▷ ▷ -lines 0 0.09 "texture{ pigment{ color Yellow } }" \
29410 ▷ ▷ ▷ -lines 1 0.09 "texture{ pigment{ color Yellow } }" \
29411 ▷ ▷ ▷ -lines 2 0.09 "texture{ pigment{ color Yellow } }" \
29412 ▷ ▷ ▷ -group_of_things_as_interval 3 39 \
29413 ▷ ▷ ▷ -lines 3 0.02 "texture{ pigment{ color Black } }" \
29414 ▷ ▷ ▷ -point_list_from_csv_file \
29415 ▷ ▷ ▷ function_lissajous_N2000.points.csv \
29416 ▷ ▷ ▷ -group_of_things_as_interval 0 6524 \
29417 ▷ ▷ ▷ -spheres 4 0.1 "texture{ pigment{ color Red } }" \
29418 finish { diffuse 0.9 phong 1} }" \
29419 ▷ ▷ ▷ -plane_by_dual_coordinates "0,0,1,0" \
29420 ▷ ▷ ▷ -group_of_things "0" \
29421 ▷ ▷ ▷ -planes 5 "texture{ pigment{ color Blue*0.5 \
29422 transmit 0.5 } }" \
29423 ▷ ▷ -scene_objects_end \
29424 ▷ ▷ -povray_end
29425 ▷ - rm -rf POV
29426 ▷ mkdir POV
29427 ▷ mv lissajous_0-*.pov POV
29428 ▷ mv makefile_animation POV
29429
29430 lissajous_3d:
29431 ▷ $(ORBITER) -v 2 \
29432 ▷ ▷ -smooth_curve "lissajous_3d" 0.07 2000 50 0 18.85 \
29433 ▷ ▷ -const a 3 b 2 c 1.57 r 7 -const_end \
29434 ▷ ▷ -var t -var_end \
29435 ▷ ▷ -code \
29436 ▷ ▷ ▷ push t push a mult push c add sin push r mult return \
29437 ▷ ▷ ▷ push t push b mult sin push r mult return \
29438 ▷ ▷ ▷ push t return \
29439 ▷ ▷ -code_end \
29440
29441 lissajous_3d_plot:
29442 ▷ cp $(ORBITER_PATH)/examples/users_guide/coordinate_grid.csv .
29443 ▷ $(ORBITER) -v 2 -povray \
29444 ▷ ▷ -round 0 -nb_frames_default 30 \
29445 ▷ ▷ -output_mask lissajous_3d_%d_%03d.pov \
29446 ▷ ▷ -video_options -W 1024 -H 768 \
29447 ▷ ▷ -global.picture_scale 0.40 \
29448 ▷ ▷ -default_angle 45 \
29449 ▷ ▷ -clipping_radius 5 \
29450 ▷ ▷ -omit_bottom_plane \
29451 ▷ ▷ -camera 0 "0,0,1" "7,7,5" "0,0,1" \
29452 ▷ ▷ -rotate_about_z_axis \
29453 ▷ ▷ -end \
29454 ▷ ▷ -scene_objects \

```

```

29455 ▷ ▷ ▷ -line_through_two_points_recentered_from_csv_file \
29456 ▷ ▷ ▷ coordinate_grid.csv \
29457 ▷ ▷ ▷ -group_of_things "0" \
29458 ▷ ▷ ▷ -group_of_things "1" \
29459 ▷ ▷ ▷ -group_of_things "2" \
29460 ▷ ▷ ▷ -lines 0 0.09 "texture{ pigment{ color Yellow } }" \
29461 ▷ ▷ ▷ -lines 1 0.09 "texture{ pigment{ color Yellow } }" \
29462 ▷ ▷ ▷ -lines 2 0.09 "texture{ pigment{ color Yellow } }" \
29463 ▷ ▷ ▷ -group_of_things_as_interval 3 39 \
29464 ▷ ▷ ▷ -lines 3 0.02 "texture{ pigment{ color Black } }" \
29465 ▷ ▷ ▷ -point_list_from_csv_file \
29466 ▷ ▷ ▷ function_lissajous_3d.N2000.points.csv \
29467 ▷ ▷ ▷ -group_of_things_as_interval 0 6538\
29468 ▷ ▷ ▷ -spheres 4 0.1 "texture{ pigment{ color Red } }" \
29469 finish { diffuse 0.9 phong 1}}" \
29470 ▷ ▷ ▷ -plane_by_dual_coordinates "0,0,1,0" \
29471 ▷ ▷ ▷ -group_of_things "0" \
29472 ▷ ▷ ▷ -scene_objects_end \
29473 ▷ ▷ ▷ -povray_end
29474 ▷ - rm -rf POV
29475 ▷ mkdir POV
29476 ▷ mv lissajous_3d_0_*.pov POV
29477 ▷ mv makefile_animation POV
29478
29479
29480 #####
29481 # Section 19.5: Gnuplot interface
29482
29483
29484
29485 SECTION_GNUPLOT_INTERFACE:
29486
29487
29488
29489 test_19.5:
29490 ▷ make gnuplot_test_data
29491 ▷ make gnuplot.Eulerfunction.50
29492
29493 gnuplot_test_data:
29494 ▷ echo $(GNUPLOT_TEST_INPUT) >gnuplot_test_data.csv
29495 ▷ $(ORBITER) -v 3 \
29496 ▷ ▷ -gnuplot gnuplot_test_data.csv \
29497 ▷ ▷ "Gnuplot test data" \
29498 ▷ ▷ "Input" \
29499 ▷ ▷ "Output"
29500 ▷ $(OPEN) gnuplot_test_data_gnuplot.png
29501
29502
29503 gnuplot.Eulerfunction.50:
29504 ▷ $(ORBITER) -v 1 -eulerfunction_interval 1 50
29505 ▷ $(ORBITER) -v 3 \
29506 ▷ ▷ -gnuplot table_eulerfunction_1.50.csv \
29507 ▷ ▷ "Euler function" \
29508 ▷ ▷ "Input" \
29509 ▷ ▷ "Output"
29510
29511
29512 #Row,N,PHI,NBPF,NBDPF
29513

```

```

29514
29515 #####
29516 # Chapter 20 - Mathematical Data in Orbiter
29517 #####
29518
29519
29520 test_20:
29521 ▷ make test_20.1
29522 ▷ make test_20.2
29523
29524
29525 #####
29526 # Section 20.1: Mathematical Data on Cubic Surfaces
29527
29528 SECTION.MATHEMATICAL.DATA:
29529
29530
29531 test_20.1:
29532 ▷ make make_table_of_surfaces
29533 ▷ make make_table_of_quartic_curves
29534 ▷ make cubic_surfaces_table.q17
29535 ▷ make quartic_curves_table.q17
29536
29537
29538 make_table_of_surfaces:
29539 ▷ $(ORBITER) -v 3 \
29540 ▷ ▷ -make_table_of_surfaces
29541 ▷ $(PDFLATEX) surfaces_report.tex
29542 ▷ $(OPEN) surfaces_report.pdf
29543
29544
29545 make_table_of_quartic_curves:
29546 ▷ $(ORBITER) -v 3 \
29547 ▷ ▷ -make_table_of_quartic_curves
29548 ▷ $(PDFLATEX) quartic_curves_report.tex
29549 ▷ $(OPEN) quartic_curves_report.pdf
29550
29551
29552
29553 cubic_surfaces_table.q17:
29554 ▷ $(ORBITER) -v 3 \
29555 ▷ ▷ -define F -finite_field -q 17 -end \
29556 ▷ ▷ -define P -projective_space \
29557 ▷ ▷ ▷ -n 3 -field F -v 0 \
29558 ▷ ▷ -end \
29559 ▷ ▷ -with P -do \
29560 ▷ ▷ ▷ -projective_space_activity \
29561 ▷ ▷ ▷ ▷ -table_of_cubic_surfaces \
29562 ▷ ▷ -end
29563
29564 quartic_curves_table.q17:
29565 ▷ $(ORBITER) -v 3 \
29566 ▷ ▷ -define F -finite_field -q 17 -end \
29567 ▷ ▷ -define P -projective_space \
29568 ▷ ▷ ▷ -n 2 -field F -v 0 \
29569 ▷ ▷ -end \
29570 ▷ ▷ -with P -do \
29571 ▷ ▷ ▷ -projective_space_activity \
29572 ▷ ▷ ▷ ▷ -table_of_quartic_curves \

```

```

29573 ▷ ▷ -end
29574
29575 # quartic_curves_q17_info.csv
29576
29577
29578
29579 #####
29580 # Section 20.2: Mathematical Data on BLT-Sets
29581
29582 SECTION.MATHEMATICAL_DATA:
29583
29584
29585 test_20_2:
29586 ▷ make blt_set_table.q7
29587 ▷ make blt_set_table.q9
29588 ▷ make blt_set_table.q13
29589 ▷ #make blt_set_table
29590 ▷ #make blt_set_export_gap
29591
29592
29593
29594 blt_set_table.q7:
29595 ▷ $(ORBITER) -v 3 \
29596 ▷ ▷ -define F -finite_field -q 7 -end \
29597 ▷ ▷ -define O -orthogonal_space 0 5 F -create_extension_fields -end \
29598 ▷ ▷ -with O -do \
29599 ▷ ▷ ▷ -orthogonal_space_activity \
29600 ▷ ▷ ▷ ▷ -table_of_blt_sets \
29601 ▷ ▷ -end
29602
29603 blt_set_table.q9:
29604 ▷ $(ORBITER) -v 3 \
29605 ▷ ▷ -define F -finite_field -q 9 -end \
29606 ▷ ▷ -define O -orthogonal_space 0 5 F -create_extension_fields -end \
29607 ▷ ▷ -with O -do \
29608 ▷ ▷ ▷ -orthogonal_space_activity \
29609 ▷ ▷ ▷ ▷ -table_of_blt_sets \
29610 ▷ ▷ -end
29611
29612
29613 blt_set_table.q13:
29614 ▷ $(ORBITER) -v 3 \
29615 ▷ ▷ -define F -finite_field -q 13 -end \
29616 ▷ ▷ -define O -orthogonal_space 0 5 F -create_extension_fields -end \
29617 ▷ ▷ -with O -do \
29618 ▷ ▷ ▷ -orthogonal_space_activity \
29619 ▷ ▷ ▷ ▷ -table_of_blt_sets \
29620 ▷ ▷ -end
29621
29622
29623 blt_set_table:
29624 ▷ $(ORBITER) -v 30 \
29625 ▷ ▷ -define Q -vector -dense "3,5,7,9,11,13,17,19,23,25,27,29,31" -end \
29626 ▷ ▷ -define NB -vector -dense "1,2,2,3,4,3,6,5,9,6,6,9,8" -end \
29627 ▷ ▷ -loop_over i Q \
29628 ▷ ▷ ▷ -define F%i -finite_field -q "%i[Q]" -end \
29629 ▷ ▷ ▷ -define O%i -orthogonal_space 0 5 F%i -create_extension_fields -end \
29630 ▷ ▷ ▷ -with O%i -do \
29631 ▷ ▷ ▷ ▷ -orthogonal_space_activity \

```



```

29632 ▷ ▷ ▷ ▷ ▷ -table_of_blt_sets \
29633 ▷ ▷ ▷ ▷ -end \
29634 ▷ ▷ ▷ -end \
29635 ▷ ▷ -end_loop_over i \
29636 ▷ ▷ -print_symbols
29637
29638
29639 blt_set_export_gap:
29640 ▷ $(ORBITER) -v 30 \
29641 ▷ ▷ -define Q -vector -dense \
29642 ▷ ▷ ▷ $(BLT_ORDER_Q) \
29643 ▷ ▷ -end \
29644 ▷ ▷ -define NB -vector -dense \
29645 ▷ ▷ ▷ $(BLT_NUMBER_ISO) \
29646 ▷ ▷ -end \
29647 ▷ ▷ -loop_over i Q \
29648 ▷ ▷ ▷ -define F%i -finite_field -q "%i[Q]" -end \
29649 ▷ ▷ ▷ -define O%i -orthogonal_space 0 5 F%i -create_extension_fields -end \
29650 ▷ ▷ ▷ -loop j 0 %i[NB] 1 \
29651 ▷ ▷ ▷ ▷ -define BLT_%i_%j -BLT_set \
29652 ▷ ▷ ▷ ▷ ▷ -space O%i -catalogue %j \
29653 ▷ ▷ ▷ ▷ -end \
29654 ▷ ▷ ▷ ▷ -with BLT_%i_%j -do -blt_set_activity \
29655 ▷ ▷ ▷ ▷ ▷ -export_gap \
29656 ▷ ▷ ▷ ▷ -end \
29657 ▷ ▷ ▷ -end_loop j \
29658 ▷ ▷ -end_loop_over i \
29659 ▷ ▷ -print_symbols
29660
29661
29662 #####
29663 # Chapter 21 - Miscellaneous
29664 #####
29665
29666
29667 test_21:
29668 ▷ make test_21_1
29669 ▷ make test_21_2
29670
29671
29672
29673 #####
29674 # Section 21.1: Miscellaneous
29675
29676 SECTION.MISCELLANEOUS:
29677
29678
29679
29680 test_21_1:
29681 ▷ make misc_select
29682 ▷ #make misc_join #poly_orbits_d3_n3_q2_select_F2.csv etc.
29683 ▷ #make table_mod_12 #missing: clock_mult_excel.csv
29684
29685
29686
29687 misc_select:
29688 ▷ $(ORBITER) -v 3 \
29689 ▷ ▷ -define F -finite_field -q 7 -end \
29690 ▷ ▷ -with F -do -finite_field_activity \

```

```

29691 ▷ ▷ ▷ -export_tables \
29692 ▷ ▷ -end
29693 ▷ $(ORBITER) -v 3 \
29694 ▷ ▷ -define F -finite_field -q 7 -end \
29695 ▷ ▷ -with F -do -finite_field_activity -cheat_sheet_GF -end
29696 ▷ $(ORBITER) -v 4 -csv_file_select_rows_and_cols \
29697 ▷ ▷ GF_q7_table_mul_r.csv \
29698 ▷ ▷ "0,2,4" "0,2,4"
29699
29700
29701
29702
29703
29704 misc_join:
29705 ▷ $(ORBITER) -v 4 \
29706 ▷ ▷ -csv_file_join 5 \
29707 ▷ ▷ ▷ poly_orbits_d3_n3_q2_select_F2.csv Orbit_idx \
29708 ▷ ▷ ▷ poly_orbits_d3_n3_q2_select_F4.csv Orbit_idx \
29709 ▷ ▷ ▷ poly_orbits_d3_n3_q2_select_F8.csv Orbit_idx \
29710 ▷ ▷ ▷ poly_orbits_d3_n3_q2_select_F16.csv Orbit_idx \
29711 ▷ ▷ ▷ poly_orbits_d3_n3_q2_select_F32.csv Orbit_idx
29712
29713
29714 table_mod_12:
29715 ▷ $(ORBITER) -v 2 \
29716 ▷ ▷ -define M -vector -load_csv_no_border clock_mult_excel.csv -end \
29717 ▷ ▷ -define all_one_r -vector -repeat 1 12 -end \
29718 ▷ ▷ -define all_one_c -vector -repeat 1 12 -end \
29719 ▷ ▷ -draw_matrix \
29720 ▷ ▷ -input_object M \
29721 ▷ ▷ -box_width 50 -bit_depth 24 \
29722 ▷ ▷ ▷ -partition 3 \
29723 ▷ ▷ ▷ ▷ all_one_r all_one_c \
29724 ▷ ▷ -end
29725
29726
29727 #missing: clock_mult_excel.csv
29728
29729
29730
29731 #####
29732 # Section 21.2: Limitations
29733
29734 SECTION_LIMITATIONS:
29735
29736 test_21_2:
29737 ▷ make F_15bit
29738 ▷ #make F_31bit
29739 ▷ #make F_32bit
29740 ▷ #make F_63bit
29741 ▷ #make F_64bit
29742
29743
29744 # 2^15 - 19 is prime
29745 # https://primes.utm.edu/lists/2small/0bit.html
29746
29747 F_15bit:
29748 ▷ $(ORBITER) -v 10 \
29749 ▷ ▷ -define F -finite_field -q 32749 -without_tables -end \

```

```

29750 ▷ ▷ -define v -vector -field F -allow_negatives \
29751 ▷ ▷ ▷ -dense "-1,-1" -end \
29752 ▷ ▷ -with F -do -finite_field_activity -product_of v -end
29753 ▷
29754
29755 F_31bit:
29756 ▷ $(ORBITER) -v 10 \
29757 ▷ ▷ -define F -finite_field -q 2147483647 -without_tables -end \
29758 ▷ ▷ -define v -vector -field F -allow_negatives \
29759 ▷ ▷ ▷ -dense "-1,-1" -end \
29760 ▷ ▷ -with F -do -finite_field_activity -product_of v -end
29761
29762
29763 # too slow
29764
29765 # The next command fail on purpose. The goal is to
29766 # show the limits of the integer domain.
29767
29768 F_32bit:
29769 ▷ $(ORBITER) -v 10 \
29770 ▷ ▷ -define F -finite_field -q 4294967291 -without_tables -end \
29771
29772 # we get an error message. The order of the field is too large for 32 bit int:
29773 #  $2^{32} - 5 = 4294967291$ 
29774
29775
29776 F_63bit:
29777 ▷ $(ORBITER) -v 10 \
29778 ▷ ▷ -define F -finite_field -q 9223372036854775783 -without_tables -end \
29779 ▷ ▷ -define v -vector -field F -allow_negatives \
29780 ▷ ▷ ▷ -dense "-1,-1" -end \
29781 ▷ ▷ -with F -do -finite_field_activity -product_of v -end
29782
29783 #  $2^{63}-25 = 9223372036854775783$  is prime
29784
29785 F_64bit:
29786 ▷ $(ORBITER) -v 10 \
29787 ▷ ▷ -define F -finite_field -q 18446744073709551557 -without_tables -end \
29788 ▷ ▷ -define v -vector -field F -allow_negatives \
29789 ▷ ▷ ▷ -dense "-1,-1" -end \
29790 ▷ ▷ -with F -do -finite_field_activity -product_of v -end
29791
29792 #  $2^{64}-59 = 18446744073709551557$  is prime
29793
29794
29795
29796
29797 #####
29798 # unclassified:
29799
29800
29801 stabilizer_of_mmx:
29802 ▷ $(ORBITER) -v 7 \
29803 ▷ ▷ -define C -combinatorial_object \
29804 ▷ ▷ ▷ -label decomp decomp \
29805 ▷ ▷ ▷ -multi_matrix "8,5,36" "3,3,3,3,3,3,3, 1,1,1,1,36, 1,0,0,0,6, 0,1,0,0,6,
0,0,1,0,6, 0,0,0,1,6, 1,0,0,0,6, 0,1,0,0,6, 0,0,1,0,6, 0,0,0,1,6" \
29806 ▷ ▷ ▷ -multi_matrix "8,5,36" "3,3,3,3,3,3,3, 36,1,1,1,1, 6,1,0,0,0, 6,0,1,0,0,
6,0,0,1,0, 6,0,0,0,1, 6,1,0,0,0, 6,0,1,0,0, 6,0,0,1,0, 6,0,0,0,1" \

```

```

29807 ▷ ▷ -end \
29808 ▷ ▷ -with C -do \
29809 ▷ ▷ -combinatorial_object_activity \
29810 ▷ ▷ ▷ -canonical_form \
29811 ▷ ▷ ▷ ▷ -save_ago \
29812 ▷ ▷ ▷ ▷ -save_transversal \
29813 ▷ ▷ ▷ ▷ -max_TDO_depth 400 \
29814 ▷ ▷ ▷ -end \
29815 ▷ ▷ -end \
29816 ▷ ▷ -with C -do \
29817 ▷ ▷ -combinatorial_object_activity \
29818 ▷ ▷ ▷ -post_processing \
29819 ▷ ▷ -end \
29820 ▷ ▷ -with C -do \
29821 ▷ ▷ -combinatorial_object_activity \
29822 ▷ ▷ ▷ -report \
29823 ▷ ▷ ▷ ▷ -export_flag_orbits \
29824 ▷ ▷ ▷ ▷ -show_TDO \
29825 ▷ ▷ ▷ ▷ -show_TDA \
29826 ▷ ▷ ▷ ▷ -dont_show_incidence_matrices \
29827 ▷ ▷ ▷ ▷ -export_group_GAP \
29828 ▷ ▷ ▷ -end \
29829 ▷ ▷ -end
29830 ▷ $(PDFLATEX) decomp_classification.tex
29831 ▷ $(OPEN) decomp_classification.pdf
29832
29833
29834 draw_desargues:
29835 ▷ $(ORBITER) -v 2 \
29836 ▷ ▷ -define Do -draw_options \
29837 ▷ ▷ ▷ -radius 150 \
29838 ▷ ▷ ▷ -line_width 1.0 -embedded \
29839 ▷ ▷ -end \
29840 ▷ ▷ -draw_mod_n \
29841 ▷ ▷ ▷ "desargues=on" \
29842 ▷ ▷ ▷ -file desargues \
29843 ▷ ▷ ▷ -n 51 \
29844 ▷ ▷ ▷ -radius 50 \
29845 ▷ ▷ ▷ -label_nodes \
29846 ▷ ▷ ▷ -draw_options Do \
29847 ▷ ▷ -end
29848 ▷ pdflatex desargues_draw.tex
29849 ▷ open desargues_draw.pdf
29850
29851 draw_vector_decomposition:
29852 ▷ $(ORBITER) -v 2 \
29853 ▷ ▷ -define Do -draw_options \
29854 ▷ ▷ ▷ -xin 20000 -yin 20000 \
29855 ▷ ▷ ▷ -xout 600000 -yout 600000 \
29856 ▷ ▷ ▷ -radius 50 \
29857 ▷ ▷ ▷ -line_width 1.0 -embedded \
29858 ▷ ▷ -end \
29859 ▷ ▷ -draw_mod_n \
29860 ▷ ▷ ▷ "vector_decomposition=on" \
29861 ▷ ▷ ▷ -file vector_decomposition \
29862 ▷ ▷ ▷ -n 51 \
29863 ▷ ▷ ▷ -radius 50 \
29864 ▷ ▷ ▷ -label_nodes \
29865 ▷ ▷ ▷ -draw_options Do \

```

```
29866 ▷ ▷ -end
29867 ▷ pdflatex vector_decomposition_draw.tex
29868 ▷ open vector_decomposition_draw.pdf
29869
29870
29871
29872
29873
```


Bibliography

- [1] Aida Abiad, Maarten De Boeck, and Sjanne Zeijlemaker. On the existence of small strictly Neumaier graphs. *Graphs Combin.*, 40(3):Paper No. 51, 25, 2024.
- [2] Abdullah Al-Azemi, Anton Betten, and Sajeeb Roy Chowdhury. A rainbow-clique search algorithm for BLT-sets. In *ICMS 2018—Proceedings of the International Congress on Mathematical Software; James H. Davenport, Manuel Kauers, George Labahn, Josef Urban (ed.)*, pages 71–79. Springer, 2018.
- [3] Awss Al-ogaidi and Anton Betten. Large Arcs in Small Planes, *Congressus Numerantium* 232 (2019), 119–136.
- [4] Abdullah Alazemi and Anton Betten. Computing the Group of an Algebraic Variety over a Finite Field, In: K. Buzzard, et al. (ed): *ICMS 2024*, Springer Lecture Notes in Computer Science, 14749, pp 89–96 (2024).
- [5] Johannes André. Über nicht-Desarguessche Ebenen mit transitiver Translationsgruppe. *Math. Z.*, 60:156–186, 1954.
- [6] Laura Bader, Guglielmo Lunardon, and Joseph A. Thas. Derivation of flocks of quadratic cones. *Forum Math.*, 2(2):163–174, 1990.
- [7] John Bamberg, Anton Betten, Philippe Cara, Jan De Beule, Michel Lavrauw, and Max Neunhöffer. *FinInG – Finite Incidence Geometry, Version 1.4.1*, 2018.
- [8] W. Barth. Two projective surfaces with many nodes, admitting the symmetries of the icosahedron. *J. Algebraic Geom.*, 5(1):173–186, 1996.
- [9] Susan Barwick and Gary Ebert. *Unitals in projective planes*. Springer Monographs in Mathematics. Springer, New York, 2008.
- [10] R. Bayley. <https://www.distanceregular.org>.
- [11] Anton Betten. Twisted tensor product codes. *Des. Codes Cryptogr.*, 47(1-3):191–219, 2008.
- [12] Anton Betten. Rainbow cliques and the classification of small BLT-sets. In *ISSAC 2013—Proceedings of the 38th International Symposium on Symbolic and Algebraic Computation*, pages 53–60. ACM, New York, 2013.
- [13] Anton Betten. The orbiter ecosystem for combinatorial objects. In *ISSAC 2020—Proceedings of the 45th International Symposium on Symbolic and Algebraic Computation*, pages 30–37. ACM, New York, 2020.
- [14] Anton Betten and Dieter Betten. There is no Drake/Larson linear space on 30 points. *J. Combin. Des.*, 18(1):48–70, 2010.
- [15] Anton Betten, Eun Ju Cheon, Seon Jeong Kim, and Tatsuya Maruta. Three families of multiple blocking sets in Desarguesian projective planes of even order. *Des. Codes Cryptogr.*, 68(1-3):49–59, 2013.
- [16] Anton Betten and Fatma Karaoglu. Cubic surfaces over small finite fields. *Des. Codes Cryptogr.*, 87(4):931–953, 2019.

- [17] Anton Betten and Fatma Karaoglu. The Eckardt point configuration of cubic surfaces revisited. *Des. Codes Cryptogr.*, 90(9):2159–2180, 2022.
- [18] Anton Betten and Fatma Karaoglu. The Eckardt point configuration of cubic surfaces revisited. *Des. Codes Cryptogr.*, 90(9):2159–2180, 2022.
- [19] Dieter Betten. 4-dimensionale Translationsebenen mit 8-dimensionaler Kollineationsgruppe. *Geometriae Dedicata*, 2:327–339, 1973.
- [20] Wieb Bosma, John Cannon, and Catherine Playoust. The Magma algebra system. I. The user language. *J. Symbolic Comput.*, 24(3-4):235–265, 1997. Computational algebra and number theory (London, 1993).
- [21] A. Brouwer. The Cohen-Tits near octagon on 315 points, <https://www.win.tue.nl/~aeb/drg/graphs/HJ315.html>.
- [22] R. H. Bruck and R. C. Bose. The construction of translation planes from projective spaces. *J. Algebra*, 1:85–102, 1964.
- [23] G. Butler. *Fundamental algorithms for permutation groups*, volume 559 of *Lecture Notes in Computer Science*. Springer-Verlag, Berlin, 1991.
- [24] Arjeh M. Cohen. Geometries originating from certain distance-regular graphs. In *Finite geometries and designs (Proc. Conf., Chelwood Gate, 1980)*, volume 49 of *London Math. Soc. Lecture Note Ser.*, pages 81–87. Cambridge Univ. Press, Cambridge-New York, 1981.
- [25] Henri Cohen. *A course in computational algebraic number theory*, volume 138 of *Graduate Texts in Mathematics*. Springer-Verlag, Berlin, 1993.
- [26] Jenny Anne Cooley. Cubic Surfaces over Finite Fields, Ph.D. thesis, University of Warwick, 2014.
- [27] Terry Czerwinski and David Oakden. The translation planes of order twenty-five. *J. Combin. Theory Ser. A*, 59(2):193–217, 1992.
- [28] Anne Delandtsheer and Jean Doyen. Most block-transitive t -designs are point-primitive. *Geom. Dedicata*, 29(3):307–310, 1989.
- [29] L.E. Dickson. Projective classification of cubic surfaces modulo 2, *Ann. of Math.* 16 (1915), 139–157.
- [30] S. Endrass. A projective surface of degree eight with 168 nodes. *J. Algebraic Geom.*, 6(2):325–334, 1997.
- [31] J. Chris Fisher and Joseph A. Thas. Flocks in $PG(3, q)$. *Math. Z.*, 169(1):1–11, 1979.
- [32] The GAP Group. *GAP – Groups, Algorithms, and Programming, Version 4.11.1*, 2021. <http://www.gap-system.org>.
- [33] D. G. Glynn and J. W. P. Hirschfeld. On the classification of geometric codes by polynomial functions. *Des. Codes Cryptogr.*, 6(3):189–204, 1995.
- [34] Gnuplot. Gnuplot <http://www.gnuplot.info>.
- [35] R. Hill. On Pellegrino’s 20-caps in $S_{4,3}$. In *Combinatorics ’81 (Rome, 1981)*, volume 78 of *North-Holland Math. Stud.*, pages 433–447. North-Holland, Amsterdam, 1983.
- [36] J. W. P. Hirschfeld. del Pezzo surfaces over finite fields. *Tensor (N.S.)*, 37(1):79–84, 1982.
- [37] James W. P. Hirschfeld. The double-six of lines over $PG(3, 4)$. *J. Austral. Math. Soc.*, 4:83–89, 1964.
- [38] J.W.P. Hirschfeld. Curves of genus 3, *Rendiconti di Matematica, Serie VII Volume 30*, Roma (2010), 77–88.
- [39] John D. Hobby. Metapost, a user’s manual. 2010, 2010.

- [40] Jeffrey Hoffstein, Jill Pipher, and Joseph H. Silverman. *An introduction to mathematical cryptography*. Undergraduate Texts in Mathematics. Springer, New York, second edition, 2014.
- [41] Derek F. Holt, Bettina Eick, and Eamonn A. O'Brien. *Handbook of computational group theory*. Discrete Mathematics and its Applications (Boca Raton). Chapman & Hall/CRC, Boca Raton, FL, 2005.
- [42] Christopher Jefferson, Markus Pfeiffer, Rebecca Waldecker, and Eliza Jonauskyste, year_unknown. GAP package images. 2019. <https://www.gap-system.org/Packages/images.html>.
- [43] Fatma Karaoglu. The cubic surfaces with twenty-seven lines over finite fields, Ph.D. thesis, University of Sussex, 2018.
- [44] Fatma Karaoglu and Anton Betten. The number of cubic surfaces with 27 lines over a finite field. *J. Algebraic Combin.*, 56(1):43–57, 2022.
- [45] Brian Kernighan and Dennis Ritchie. *The C Programming Language*. Prentice Hall. 1978 and 1988.
- [46] Earl S. Kramer and Dale M. Mesner. t -designs on hypergraphs. *Discrete Math.*, 15(3):263–296, 1976.
- [47] Joseph P. S. Kung. The cycle structure of a linear transformation over a finite field. *Linear Algebra Appl.*, 36:141–155, 1981.
- [48] Maska Law. Flocks, generalizes quadrangles and translation planes from BLT-sets, Ph.D. thesis, University of Western Australia, 2003.
- [49] Maska Law and Tim Penttila. Classification of flocks of the quadratic cone over fields of order at most 29. Number suppl., pages S232–S244. 2003. Special issue dedicated to Adriano Barlotti.
- [50] Maska Law and Tim Penttila. Construction of BLT-sets over small fields. *European J. Combin.*, 25(1):1–22, 2004.
- [51] Wen-Ch'ing Winnie Li. Character sums and abelian Ramanujan graphs. *J. Number Theory*, 41(2):199–217, 1992.
- [52] A. Lubotzky, R. Phillips, and P. Sarnak. Ramanujan graphs. *Combinatorica*, 8(3):261–277, 1988.
- [53] Heinz Lüneburg. *On the rational normal form of endomorphisms*. Bibliographisches Institut, Mannheim, 1987. A primer to constructive algebra.
- [54] L. Lunelli and M. Sce. *k-archi completi nei piani proiettivi desarguesiani di rango 8 e 16*. Centro di Calcoli Numerici, Politecnico di Milano, Milan, 1958.
- [55] 2021. Maple 21. Maplesoft, a division of Waterloo Maple Inc., Waterloo, Ontario. 2021.
- [56] Brendan McKay, 1995. Possolve, A program to solve positive integer systems, Australian National University, private communication.
- [57] Brendan McKay, 2020. Nauty and Traces (Version 2.7r1), Australian National University, 2020.
- [58] G. Eric Moorhouse. Projective Planes of Order 16. <http://ericmoorhouse.org/pub/planes16/>.
- [59] G. Eric Moorhouse. Projective Planes of Order 27. <http://ericmoorhouse.org/pub/planes27/>.
- [60] M. L. Narayana Rao and K. Kuppuswamy Rao. A new flag transitive affine plane of order 27. *Proc. Amer. Math. Soc.*, 59(2):337–345, 1976.
- [61] T. Penttila. Regular cyclic BLT-sets. Number 53, pages 167–172. 1998. Combinatorics '98 (Mondello).
- [62] Tim Penttila and Gordon F. Royle. BLT-sets over small fields. *Australas. J. Combin.*, 17:295–307, 1998.
- [63] Wilhelm Plesken. Counting with groups and rings. *J. Reine Angew. Math.*, 334:40–68, 1982.

- [64] 2021. POV-RAY Developers. POV-RAY, Persistence of Vision Raytracer Pty. Ltd. <http://povray.org>, accessed 1/23/2021.
- [65] Cheryl E. Praeger. The flag-transitive symmetric designs with 45 points, blocks of size 12, and 3 blocks on every point pair. *Des. Codes Cryptogr.*, 44(1-3):115–132, 2007.
- [66] L. Schläfli, 1858. An attempt to determine the twenty-seven lines upon a surface of the third order and to divide such surfaces into species in reference to the reality of the lines upon the surface, *Quart. J. Math.* **2** (1858), 55–110.
- [67] Bernd Schmalz. Verwendung von Untergruppenleitern zur Bestimmung von Doppelnebenklassen. *Bayreuth. Math. Schr.*, (31):109–143, 1990.
- [68] Ákos Seress. *Permutation group algorithms*, volume 152 of *Cambridge Tracts in Mathematics*. Cambridge University Press, Cambridge, 2003.
- [69] William Stein and David Joyner. SAGE: System for algebra and geometry experimentation. *ACM SIGSAM Bulletin*, 39(2):61–64, 2005. http://www.sagemath.org/files/sage_stein2005.pdf.
- [70] Ibrahim A. I. Suleiman and Robert A. Wilson. The 2-modular characters of Conway’s third group Co_3 . *J. Symbolic Comput.*, 24(3-4):493–506, 1997.
- [71] Magma system, year_unknown. Magma Calculator <http://magma.maths.usyd.edu.au/calc/>, accessed 11/24/2019.
- [72] Till Tantau. The TikZ and PGF Packages. Manual for Version 3.1.9a – 2022 <https://tikz.dev>.
- [73] D.E. Taylor. *The geometry of the classical groups*, volume 9 of *Sigma Series in Pure Mathematics*. Heldermann Verlag, Berlin, 1992.
- [74] Michael Walker. A class of translation planes. *Geometriae Dedicata*, 5(2):135–146, 1976.
- [75] Wikipedia. Bmp file format. https://en.wikipedia.org/wiki/BMP_file_format.
- [76] Robert Wilson, Peter Walsh, Jonathan Tripp, Ibrahim Suleiman, Richard Parker, Simon Norton, Simon Nickerson, Steve Linton, John Bray, and Rachel Abbott, 2019. ATLAS of Finite Group Representations - Version 3, <http://brauer.maths.qmul.ac.uk/Atlas/v3/>, accessed 11/24/2019.